Arm® A64 Instruction Set

for A-profile architecture

Document number	111182 (ID102025)
Document quality	BETA
Document version	A
Document confidentiality	Non-Confidential
Date of issue	30 September 2026

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved.
Document number: 111182 (ID102025)

Arm® A64 Instruction Set
for A-profile architecture

Copyright © 2010-2026 Arm Limited (or its affiliates). All rights reserved.

Release information

For information on the change history and known issues for this release, see the Release notes in the A64 ISA XML for A-profile architecture (2026-09).

Non-Confidential Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). **No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.**

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm's trademark usage guidelines at https://www.arm.com/company/policies/trademarks. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-20349

8 March 2024

Confidentiality Status

This document is Non-Confidential. The right to use, copy and disclose this document may be subject to license restrictions in accordance with the terms of the agreement entered into by Arm and the party that Arm delivered this document to.

Product Status

The information relating to the A-profile Architecture in this document is at Beta quality. Beta quality means that all major features of the specification are described, but some details might be missing.

Web Address

https://www.arm.com

Progressive Terminology Commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used terms that can be offensive. Arm strives to lead the industry and create change.

Previous issues of this document included terms that can be offensive. We have replaced these terms.

If you find offensive terms in this document, please contact terms@arm.com.

Feedback on this document

If you have any comments or queries about this document, create a ticket at support.arm.com/support-cases.

As part of the ticket, include:

- The title, *Arm® A64 Instruction Set for A-profile architecture*.
- The number, 111182 (ID102025).
- The section name to which your comments refer.
- The page number(s) to which your comments refer.
- The rule identifier(s) to which your comments refer if applicable.
- A concise explanation of your comments.

Arm also welcomes general suggestions for additions and improvements.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved.
Non-Confidential

A64 -- Base Instructions (alphabetic order)

ABS: Absolute value.

ADC: Add with carry.

ADCS: Add with carry, setting flags.

ADD (extended register): Add extended and scaled register.

ADD (immediate): Add immediate value.

ADD (shifted register): Add optionally-shifted register.

ADDG: Add with tag.

ADDPT: Add checked pointer.

ADDS (extended register): Add extended and scaled register, setting flags.

ADDS (immediate): Add immediate value, setting flags.

ADDS (shifted register): Add optionally-shifted register, setting flags.

ADR: Form PC-relative address.

ADRP: Form PC-relative address to 4KB page.

AND (immediate): Bitwise AND (immediate).

AND (shifted register): Bitwise AND (shifted register).

ANDS (immediate): Bitwise AND (immediate), setting flags.

ANDS (shifted register): Bitwise AND (shifted register), setting flags.

APAS: Associate physical address space: an alias of SYS.

ASR (immediate): Arithmetic shift right (immediate): an alias of SBFM.

ASR (register): Arithmetic shift right (register): an alias of ASRV.

ASRV: Arithmetic shift right variable.

AT: Address translate: an alias of SYS.

AUTDA, AUTDZA: Authenticate data address, using key A.

AUTDB, AUTDZB: Authenticate data address, using key B.

AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA: Authenticate instruction address, using key A.

AUTIA171615: Authenticate instruction address, using key A.

AUTIASPPC: Authenticate return address using key A, using an immediate offset.

AUTIASPPCR: Authenticate return address using key A, using a register.

AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB: Authenticate instruction address, using key B.

AUTIB171615: Authenticate instruction address, using key B.

AUTIBSPPC: Authenticate return address using key B, using an immediate offset.

AUTIBSPPCR: Authenticate return address using key B, using a register.

AXFLAG: Convert floating-point condition flags from Arm to external format.

B: Branch.

B.cond: Branch conditionally.

BC.cond: Branch consistent conditionally.

BFC: Bitfield clear: an alias of BFM.

BFI: Bitfield insert: an alias of BFM.

BFM: Bitfield move.

BFXIL: Bitfield extract and insert at low end: an alias of BFM.

BIC (shifted register): Bitwise bit clear (shifted register).

BICS (shifted register): Bitwise bit clear (shifted register), setting flags.

BL: Branch with link.

BLR: Branch with link to register.

BLRAA, BLRAAZ, BLRAB, BLRABZ: Branch with link to register, with pointer authentication.

BR: Branch to register.

BRAA, BRAAZ, BRAB, BRABZ: Branch to register, with pointer authentication.

BRB: Branch record buffer: an alias of SYS.

BRK: Breakpoint instruction.

BTI: Branch target identification.

CAS, CASA, CASAL, CASL: Compare and swap word or doubleword in memory.

CASB, CASAB, CASALB, CASLB: Compare and swap byte in memory.

CASH, CASAH, CASALH, CASLH: Compare and swap halfword in memory.

CASP, CASPA, CASPAL, CASPL: Compare and swap pair of words or doublewords in memory.

CASPT, CASPAT, CASPALT, CASPLT: Compare and swap pair unprivileged.

CAST, CASAT, CASALT, CASLT: Compare and swap unprivileged.

CB<cc> (immediate): Compare register with immediate and branch.

CB<cc> (register): Compare registers and branch.

CBB<cc>: Compare bytes and branch.

CBBLE: Compare signed less than or equal to bytes and branch: an alias of CBB<cc>.

CBBLO: Compare unsigned lower than bytes and branch: an alias of CBB<cc>.

CBBLS: Compare unsigned lower than or same as bytes and branch: an alias of CBB<cc>.

CBBLT: Compare signed less than bytes and branch: an alias of CBB<cc>.

CBGE (immediate): Compare signed greater than or equal to immediate and branch: an alias of CB<cc> (immediate).

CBH<cc>: Compare halfwords and branch.

CBHLE: Compare signed less than or equal to halfwords and branch: an alias of CBH<cc>.

CBHLO: Compare unsigned lower than halfwords and branch: an alias of CBH<cc>.

CBHLS: Compare unsigned lower than or same as halfwords and branch: an alias of CBH<cc>.

CBHLT: Compare signed less than halfwords and branch: an alias of CBH<cc>.

CBHS (immediate): Compare unsigned higher than or same as immediate and branch: an alias of CB<cc> (immediate).

CBLE (immediate): Compare signed less than or equal to immediate and branch: an alias of CB<cc> (immediate).

CBLE (register): Compare signed less than or equal to register and branch: an alias of CB<cc> (register).

CBLO (register): Compare unsigned lower than register and branch: an alias of CB<cc> (register).

CBLS (immediate): Compare unsigned lower than or same as immediate and branch: an alias of CB<cc> (immediate).

CBLS (register): Compare unsigned lower than or same as register and branch: an alias of CB<cc> (register).

CBLT (register): Compare signed less than register and branch: an alias of CB<cc> (register).

CBNZ: Compare and branch on nonzero.

CBZ: Compare and branch on zero.

CCMN (immediate): Conditional compare negative (immediate).

CCMN (register): Conditional compare negative (register).

CCMP (immediate): Conditional compare (immediate).

CCMP (register): Conditional compare (register).

CFINV: Invert carry flag.

CFLT<cc> (immediate): Compare register with immediate and fault.

CFLT<cc> (register): Compare registers and fault.

CFLTGE (immediate): Compare signed greater than or equal to immediate and fault: an alias of CFLT<cc> (immediate).

CFLTHS (immediate): Compare unsigned higher than or same as immediate and fault: an alias of CFLT<cc> (immediate).

CFLTLE (immediate): Compare signed less than or equal to immediate and fault: an alias of CFLT<cc> (immediate).

CFLTLE (register): Compare registers and fault if less than or equal: an alias of CFLT<cc> (register).

CFLTLO (register): Compare registers and fault if lower: an alias of CFLT<cc> (register).

CFLTLS (immediate): Compare unsigned lower than or same as immediate and fault: an alias of CFLT<cc> (immediate).

CFLTLS (register): Compare registers and fault if lower than or same: an alias of CFLT<cc> (register).

CFLTLT (register): Compare registers and fault if less than: an alias of CFLT<cc> (register).

CFLTNZ: Compare and fault on nonzero: an alias of CFLT<cc> (immediate).

CFLTZ: Compare and fault on zero: an alias of CFLT<cc> (immediate).

CFP: Control flow prediction restriction by context: an alias of SYS.

CHKFEAT: Check feature status.

CINC: Conditional increment: an alias of CSINC.

CINV: Conditional invert: an alias of CSINV.

CLRBHB: Clear branch history.

CLREX: Clear exclusive.

CLS: Count leading sign bits.

CLZ: Count leading zeros.

CMN (extended register): Compare negative (extended register): an alias of ADDS (extended register).

CMN (immediate): Compare negative (immediate): an alias of ADDS (immediate).

CMN (shifted register): Compare negative (shifted register): an alias of ADDS (shifted register).

CMP (extended register): Compare (extended register): an alias of SUBS (extended register).

CMP (immediate): Compare (immediate): an alias of SUBS (immediate).

CMP (shifted register): Compare (shifted register): an alias of SUBS (shifted register).

CMPP: Compare with tag: an alias of SUBPS.

CNEG: Conditional negate: an alias of CSNEG.

CNT: Count bits.

COSP: Clear other speculative prediction restriction by context: an alias of SYS.

CPP: Cache prefetch prediction restriction by context: an alias of SYS.

CPYFP, CPYFM, CPYFE: Memory copy forward-only.

CPYFPN, CPYFMN, CPYFEN: Memory copy forward-only, reads and writes non-temporal.

CPYFPRN, CPYFMRN, CPYFERN: Memory copy forward-only, reads non-temporal.

CPYFPRT, CPYFMRT, CPYFERT: Memory copy forward-only, reads unprivileged.

CPYFPRTN, CPYFMRTN, CPYFERTN: Memory copy forward-only, reads unprivileged, reads and writes non-temporal.

CPYFPRTRN, CPYFMRTRN, CPYFERTRN: Memory copy forward-only, reads unprivileged and non-temporal.

CPYFPRTWN, CPYFMRTWN, CPYFERTWN: Memory copy forward-only, reads unprivileged, writes non-temporal.

CPYFPT, CPYFMT, CPYFET: Memory copy forward-only, reads and writes unprivileged.

CPYFPTN, CPYFMTN, CPYFETN: Memory copy forward-only, reads and writes unprivileged and non-temporal.

CPYFPTRN, CPYFMTRN, CPYFETRN: Memory copy forward-only, reads and writes unprivileged, reads non-temporal.

CPYFPTWN, CPYFMTWN, CPYFETWN: Memory copy forward-only, reads and writes unprivileged, writes non-temporal.

CPYFPWN, CPYFMWN, CPYFEWN: Memory copy forward-only, writes non-temporal.

CPYFPWT, CPYFMWT, CPYFEWT: Memory copy forward-only, writes unprivileged.

CPYFPWTN, CPYFMWTN, CPYFEWTN: Memory copy forward-only, writes unprivileged, reads and writes non-temporal.

CPYFPWTRN, CPYFMWTRN, CPYFEWTRN: Memory copy forward-only, writes unprivileged, reads non-temporal.

CPYFPWTWN, CPYFMWTWN, CPYFEWTWN: Memory copy forward-only, writes unprivileged and non-temporal.

CPYP, CPYM, CPYE: Memory copy.

CPYPN, CPYMN, CPYEN: Memory copy, reads and writes non-temporal.

CPYPRN, CPYMRN, CPYERN: Memory copy, reads non-temporal.

CPYPRT, CPYMRT, CPYERT: Memory copy, reads unprivileged.

CPYPRTN, CPYMRTN, CPYERTN: Memory copy, reads unprivileged, reads and writes non-temporal.

CPYPRTRN, CPYMRTRN, CPYERTRN: Memory copy, reads unprivileged and non-temporal.

CPYPRTWN, CPYMRTWN, CPYERTWN: Memory copy, reads unprivileged, writes non-temporal.

CPYPT, CPYMT, CPYET: Memory copy, reads and writes unprivileged.

CPYPTN, CPYMTN, CPYETN: Memory copy, reads and writes unprivileged and non-temporal.

CPYPTRN, CPYMTRN, CPYETRN: Memory copy, reads and writes unprivileged, reads non-temporal.

CPYPTWN, CPYMTWN, CPYETWN: Memory copy, reads and writes unprivileged, writes non-temporal.

CPYPWN, CPYMWN, CPYEWN: Memory copy, writes non-temporal.

CPYPWT, CPYMWT, CPYEWT: Memory copy, writes unprivileged.

CPYPWTN, CPYMWTN, CPYEWTN: Memory copy, writes unprivileged, reads and writes non-temporal.

CPYPWTRN, CPYMWTRN, CPYEWTRN: Memory copy, writes unprivileged, reads non-temporal.

CPYPWTWN, CPYMWTWN, CPYEWTWN: Memory copy, writes unprivileged and non-temporal.

CRC32B, CRC32H, CRC32W, CRC32X: CRC32 checksum.

CRC32CB, CRC32CH, CRC32CW, CRC32CX: CRC32C checksum.

CSDB: Consumption of speculative data barrier.

CSEL: Conditional select.

CSET: Conditional set: an alias of CSINC.

CSETM: Conditional set mask: an alias of CSINV.

CSINC: Conditional select increment.

CSINV: Conditional select invert.

CSNEG: Conditional select negation.

CTZ: Count trailing zeros.

DC: Data cache operation: an alias of SYS.

DCPS1: Debug change PE state to EL1.

DCPS2: Debug change PE state to EL2.

DCPS3: Debug change PE state to EL3.

DGH: Data gathering hint.

DMB: Data memory barrier.

DRPS: Debug restore PE state.

DSB: Data synchronization barrier.

DVP: Data value prediction restriction by context: an alias of SYS.

EON (shifted register): Bitwise exclusive-OR NOT (shifted register).

EOR (immediate): Bitwise exclusive-OR (immediate).

EOR (shifted register): Bitwise exclusive-OR (shifted register).

ERET: Exception return.

ERETAA, ERETAB: Exception return, with pointer authentication.

ESB: Error synchronization barrier.

EXTR: Extract register.

FLT.cond: Conditional fault.

GCSB: Guarded Control Stack barrier.

GCSPOPCX: Guarded Control Stack pop and compare exception return record: an alias of SYS.

GCSPOPM: Guarded Control Stack pop: an alias of SYSL.

GCSPOPX: Guarded Control Stack pop exception return record: an alias of SYS.

GCSPUSHM: Guarded Control Stack push: an alias of SYS.

GCSPUSHX: Guarded Control Stack push exception return record: an alias of SYS.

GCSSS1: Guarded Control Stack switch stack 1: an alias of SYS.

GCSSS2: Guarded Control Stack switch stack 2: an alias of SYSL.

GCSSTR: Guarded Control Stack store register.

GCSSTTR: Guarded Control Stack store register (unprivileged).

GIC: Generic Interrupt Controller: an alias of SYS.

GICR: Generic Interrupt Controller with result: an alias of SYSL.

GMI: Tag mask insert.

GSB: Generic Interrupt Controller synchronization barrier: an alias of SYS.

HINT: Hint instruction.

HINTE: Hint instruction (extended).

HLT: Halt instruction.

HVC: Hypervisor call.

IC: Instruction cache operation: an alias of SYS.

IRG: Insert random tag.

ISB: Instruction synchronization barrier.

LD64B: Single-copy atomic 64-byte load.

LDA64B: Single-copy atomic 64-byte load-acquire.

LDADD, LDADDA, LDADDAL, LDADDL: Atomic add on word or doubleword.

LDADDB, LDADDAB, LDADDALB, LDADDLB: Atomic add on byte.

LDADDH, LDADDAH, LDADDALH, LDADDLH: Atomic add on halfword.

LDAP: Load-acquire pair of registers.

LDAPP: Load-acquire RCpc pair of registers.

LDAPR: Load-acquire RCpc register.

LDAPRB: Load-acquire RCpc register byte.

LDAPRH: Load-acquire RCpc register halfword.

LDAPUR: Load-acquire RCpc register (unscaled).

LDAPURB: Load-acquire RCpc register byte (unscaled).

LDAPURH: Load-acquire RCpc register halfword (unscaled).

LDAPURSB: Load-acquire RCpc register signed byte (unscaled).

LDAPURSH: Load-acquire RCpc register signed halfword (unscaled).

LDAPURSW: Load-acquire RCpc register signed word (unscaled).

LDAR: Load-acquire register.

LDARB: Load-acquire register byte.

LDARH: Load-acquire register halfword.

LDATXR: Load-acquire unprivileged exclusive register.

LDAXP: Load-acquire exclusive pair of registers.

LDAXR: Load-acquire exclusive register.

LDAXRB: Load-acquire exclusive register byte.

LDAXRH: Load-acquire exclusive register halfword.

LDCLR, LDCLRA, LDCLRAL, LDCLRL: Atomic bit clear on word or doubleword.

LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB: Atomic bit clear on byte.

LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH: Atomic bit clear on halfword.

LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL: Atomic bit clear on quadword.

LDEOR, LDEORA, LDEORAL, LDEORL: Atomic exclusive-OR on word or doubleword.

LDEORB, LDEORAB, LDEORALB, LDEORLB: Atomic exclusive-OR on byte.

LDEORH, LDEORAH, LDEORALH, LDEORLH: Atomic exclusive-OR on halfword.

LDG: Load Allocation Tag.

LDGM: Load tag multiple.

LDIAPP: Load-Acquire RCpc ordered pair of registers.

LDLAR: Load LOAcquire register.

LDLARB: Load LOAcquire register byte.

LDLARH: Load LOAcquire register halfword.

LDNP: Load pair of registers, with non-temporal hint.

LDP: Load pair of registers.

LDPSW: Load pair of registers signed word.

LDR (immediate): Load register (immediate).

LDR (literal): Load register (literal).

LDR (register): Load register (register).

LDRAA, LDRAB: Load register, with pointer authentication.

LDRB (immediate): Load register byte (immediate).

LDRB (register): Load register byte (register).

LDRH (immediate): Load register halfword (immediate).

LDRH (register): Load register halfword (register).

LDRSB (immediate): Load register signed byte (immediate).

LDRSB (register): Load register signed byte (register).

LDRSH (immediate): Load register signed halfword (immediate).

LDRSH (register): Load register signed halfword (register).

LDRSW (immediate): Load register signed word (immediate).

LDRSW (literal): Load register signed word (literal).

LDRSW (register): Load register signed word (register).

LDSET, LDSETA, LDSETAL, LDSETL: Atomic bit set on word or doubleword.

LDSETB, LDSETAB, LDSETALB, LDSETLB: Atomic bit set on byte.

LDSETH, LDSETAH, LDSETALH, LDSETLH: Atomic bit set on halfword.

LDSETP, LDSETPA, LDSETPAL, LDSETPL: Atomic bit set on quadword.

LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL: Atomic signed maximum on word or doubleword.

LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB: Atomic signed maximum on byte.

LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH: Atomic signed maximum on halfword.

LDSMIN, LDSMINA, LDSMINAL, LDSMINL: Atomic signed minimum on word or doubleword.

LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB: Atomic signed minimum on byte.

LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH: Atomic signed minimum on halfword.

LDTADD, LDTADDA, LDTADDAL, LDTADDL: Atomic add unprivileged.

LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL: Atomic bit clear unprivileged.

LDTNP: Load unprivileged pair of registers, with non-temporal hint.

LDTP: Load unprivileged pair of registers.

LDTR: Load register (unprivileged).

LDTRB: Load register byte (unprivileged).

LDTRH: Load register halfword (unprivileged).

LDTRSB: Load register signed byte (unprivileged).

LDTRSH: Load register signed halfword (unprivileged).

LDTRSW: Load register signed word (unprivileged).

LDTSET, LDTSETA, LDTSETAL, LDTSETL: Atomic bit set unprivileged.

LDTXR: Load unprivileged exclusive register.

LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL: Atomic unsigned maximum on word or doubleword.

LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB: Atomic unsigned maximum on byte.

LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH: Atomic unsigned maximum on halfword.

LDUMIN, LDUMINA, LDUMINAL, LDUMINL: Atomic unsigned minimum on word or doubleword.

LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB: Atomic unsigned minimum on byte.

LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH: Atomic unsigned minimum on halfword.

LDUR: Load register (unscaled).

LDURB: Load register byte (unscaled).

LDURH: Load register halfword (unscaled).

LDURSB: Load register signed byte (unscaled).

LDURSH: Load register signed halfword (unscaled).

LDURSW: Load register signed word (unscaled).

LDXP: Load exclusive pair of registers.

LDXR: Load exclusive register.

LDXRB: Load exclusive register byte.

LDXRH: Load exclusive register halfword.

LSL (immediate): Logical shift left (immediate): an alias of UBFM.

LSL (register): Logical shift left (register): an alias of LSLV.

LSLV: Logical shift left variable.

LSR (immediate): Logical shift right (immediate): an alias of UBFM.

LSR (register): Logical shift right (register): an alias of LSRV.

LSRV: Logical shift right variable.

MADD: Multiply-add.

MADDPT: Multiply-add checked pointer.

MNEG: Multiply-negate: an alias of MSUB.

MOV (bitmask immediate): Move bitmask immediate value: an alias of ORR (immediate).

MOV (inverted wide immediate): Move inverted wide immediate value: an alias of MOVN.

MOV (register): Move register value: an alias of ORR (shifted register).

MOV (to/from SP): Move register value to or from SP: an alias of ADD (immediate).

MOV (wide immediate): Move wide immediate value: an alias of MOVZ.

MOVK: Move wide with keep.

MOVN: Move wide with NOT.

MOVZ: Move wide with zero.

MRRS: Move System register to two adjacent general-purpose registers.

MRS: Move System register to general-purpose register.

MSR (immediate): Move immediate value to special register.

MSR (register): Move general-purpose register to System register.

MSRR: Move two adjacent general-purpose registers to System register.

MSUB: Multiply-subtract.

MSUBPT: Multiply-subtract checked pointer.

MUL: Multiply: an alias of MADD.

MVN: Bitwise NOT: an alias of ORN (shifted register).

NEG (shifted register): Negate (shifted register): an alias of SUB (shifted register).

NEGS: Negate, setting flags: an alias of SUBS (shifted register).

NGC: Negate with carry: an alias of SBC.

NGCS: Negate with carry, setting flags: an alias of SBCS.

NOP: No operation.

ORN (shifted register): Bitwise OR NOT (shifted register).

ORR (immediate): Bitwise OR (immediate).

ORR (shifted register): Bitwise OR (shifted register).

PACDA, PACDZA: Pointer Authentication Code for data address, using key A.

PACDB, PACDZB: Pointer Authentication Code for data address, using key B.

PACGA: Pointer Authentication Code, using generic key.

PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA: Pointer Authentication Code for instruction address, using key A.

PACIA171615: Pointer Authentication Code for instruction address, using key A.

PACIASPPC: Pointer Authentication Code for return address, using key A.

PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB: Pointer Authentication Code for instruction address, using key B.

PACIB171615: Pointer Authentication Code for instruction address, using key B.

PACIBSPPC: Pointer Authentication Code for return address, using key B.

PACM: Pointer authentication modifier.

PACNBIASPPC: Pointer Authentication Code for return address, using key A, not a branch target.

PACNBIBSPPC: Pointer Authentication Code for return address, using key B, not a branch target.

PRFM (immediate): Prefetch memory (immediate).

PRFM (literal): Prefetch memory (literal).

PRFM (register): Prefetch memory (register).

PRFUM: Prefetch memory (unscaled offset).

PSB: Profiling synchronization barrier.

PSSBB: Physical speculative store bypass barrier: an alias of DSB.

RBIT: Reverse bits.

RCWCAS, RCWCASA, RCWCASAL, RCWCASL: Read check write compare and swap doubleword in memory.

RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL: Read check write compare and swap quadword in memory.

RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL: Read check write atomic bit clear on doubleword in memory.

RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL: Read check write atomic bit clear on quadword in memory.

RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL: Read check write software compare and swap doubleword in memory.

RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL: Read check write software compare and swap quadword in memory.

RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL: Read check write software atomic bit clear on doubleword in memory.

RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL: Read check write software atomic bit clear on quadword in memory.

RCWSET, RCWSETA, RCWSETAL, RCWSETL: Read check write atomic bit set on doubleword in memory.

RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL: Read check write atomic bit set on quadword in memory.

RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL: Read check write software atomic bit set on doubleword in memory.

RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL: Read check write software atomic bit set on quadword in memory.

RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL: Read check write software swap doubleword in memory.

RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL: Read check write software swap quadword in memory.

RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL: Read check write swap doubleword in memory.

RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL: Read check write swap quadword in memory.

RET: Return from subroutine.

RETAA, RETAB: Return from subroutine, with pointer authentication.

RETAASPPC, RETABSPPC: Return from subroutine, with enhanced pointer authentication using an immediate offset.

RETAASPPCR, RETABSPPCR: Return from subroutine, with enhanced pointer authentication using a register.

REV: Reverse bytes.

REV16: Reverse bytes in 16-bit halfwords.

REV32: Reverse bytes in 32-bit words.

REV64: Reverse bytes: an alias of REV.

RMIF: Rotate, mask insert flags.

ROR (immediate): Rotate right (immediate): an alias of EXTR.

ROR (register): Rotate right (register): an alias of RORV.

RORV: Rotate right variable.

RPRFM: Range prefetch memory.

SB: Speculation barrier.

SBC: Subtract with carry.

SBCS: Subtract with carry, setting flags.

SBFIZ: Signed bitfield insert in zeros: an alias of SBFM.

SBFM: Signed bitfield move.

SBFX: Signed bitfield extract: an alias of SBFM.

SDIV (quotient): Signed divide.

SETF8, SETF16: Evaluation of 8-bit or 16-bit flag values.

SETGP, SETGM, SETGE: Memory set with tag setting.

SETGPN, SETGMN, SETGEN: Memory set with tag setting, non-temporal.

SETGPT, SETGMT, SETGET: Memory set with tag setting, unprivileged.

SETGPTN, SETGMTN, SETGETN: Memory set with tag setting, unprivileged and non-temporal.

SETP, SETM, SETE: Memory set.

SETPN, SETMN, SETEN: Memory set, non-temporal.

SETPT, SETMT, SETET: Memory set, unprivileged.

SETPTN, SETMTN, SETETN: Memory set, unprivileged and non-temporal.

SEV: Send event.

SEVL: Send event local.

SHUH: Shared update hint.

SLBND: Scoped release boundary.

SMADDL: Signed multiply-add long.

SMAX (immediate): Signed maximum (immediate).

SMAX (register): Signed maximum (register).

SMC: Secure monitor call.

SMIN (immediate): Signed minimum (immediate).

SMIN (register): Signed minimum (register).

SMNEGL: Signed multiply-negate long: an alias of SMSUBL.

SMSTART: Enable access to Streaming SVE mode and SME architectural state: an alias of MSR (immediate).

SMSTOP: Disable access to Streaming SVE mode and SME architectural state: an alias of MSR (immediate).

SMSUBL: Signed multiply-subtract long.

SMULH: Signed multiply high.

SMULL (32-bit): Signed multiply long: an alias of SMADDL.

SRLS: Scope release.

SSBB: Speculative store bypass barrier: an alias of DSB.

ST2G: Store Allocation Tags.

ST64B: Single-copy atomic 64-byte store without status result.

ST64BV: Single-copy atomic 64-byte store with status result.

ST64BV0: Single-copy atomic 64-byte EL0 store with status result.

STADD, STADDL: Atomic add on word or doubleword, without return: an alias of LDADD, LDADDA, LDADDAL, LDADDL.

STADDB, STADDLB: Atomic add on byte, without return: an alias of LDADDB, LDADDAB, LDADDALB, LDADDLB.

STADDH, STADDLH: Atomic add on halfword, without return: an alias of LDADDH, LDADDAH, LDADDALH, LDADDLH.

STCLR, STCLRL: Atomic bit clear on word or doubleword, without return: an alias of LDCLR, LDCLRA, LDCLRAL, LDCLRL.

STCLRB, STCLRLB: Atomic bit clear on byte, without return: an alias of LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB.

STCLRH, STCLRLH: Atomic bit clear on halfword, without return: an alias of LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH.

STCPH: Store concurrent priority hint.

STEOR, STEORL: Atomic exclusive-OR on word or doubleword, without return: an alias of LDEOR, LDEORA, LDEORAL, LDEORL.

STEORB, STEORLB: Atomic exclusive-OR on byte, without return: an alias of LDEORB, LDEORAB, LDEORALB, LDEORLB.

STEORH, STEORLH: Atomic exclusive-OR on halfword, without return: an alias of LDEORH, LDEORAH, LDEORALH, LDEORLH.

STG: Store Allocation Tag.

STGM: Store Allocation Tag multiple.

STGP: Store Allocation Tag and pair of registers.

STILP: Store-release ordered pair of registers.

STL64B: Single-copy atomic 64-byte store-release without status result.

STL64BV: Single-copy atomic 64-byte store-release with status result.

STL64BV0: Single-copy atomic 64-byte EL0 store-release with status result.

STLLR: Store LORelease register.

STLLRB: Store LORelease register byte.

STLLRH: Store LORelease register halfword.

STLP: Store-release pair of registers.

STLR: Store-release register.

STLRB: Store-release register byte.

STLRH: Store-release register halfword.

STLTXR: Store-release unprivileged exclusive register.

STLUR: Store-release register (unscaled).

STLURB: Store-release register byte (unscaled).

STLURH: Store-release register halfword (unscaled).

STLXP: Store-release exclusive pair of registers.

STLXR: Store-release exclusive register.

STLXRB: Store-release exclusive register byte.

STLXRH: Store-release exclusive register halfword.

STNP: Store pair of registers, with non-temporal hint.

STP: Store pair of registers.

STR (immediate): Store register (immediate).

STR (register): Store register (register).

STRB (immediate): Store register byte (immediate).

STRB (register): Store register byte (register).

STRH (immediate): Store register halfword (immediate).

STRH (register): Store register halfword (register).

STSET, STSETL: Atomic bit set on word or doubleword, without return: an alias of LDSET, LDSETA, LDSETAL, LDSETL.

STSETB, STSETLB: Atomic bit set on byte, without return: an alias of LDSETB, LDSETAB, LDSETALB, LDSETLB.

STSETH, STSETLH: Atomic bit set on halfword, without return: an alias of LDSETH, LDSETAH, LDSETALH, LDSETLH.

STSHH: Store shared hint.

STSMAX, STSMAXL: Atomic signed maximum on word or doubleword, without return: an alias of LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL.

STSMAXB, STSMAXLB: Atomic signed maximum on byte, without return: an alias of LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB.

STSMAXH, STSMAXLH: Atomic signed maximum on halfword, without return: an alias of LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH.

STSMIN, STSMINL: Atomic signed minimum on word or doubleword, without return: an alias of LDSMIN, LDSMINA, LDSMINAL, LDSMINL.

STSMINB, STSMINLB: Atomic signed minimum on byte, without return: an alias of LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB.

STSMINH, STSMINLH: Atomic signed minimum on halfword, without return: an alias of LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH.

STTADD, STTADDL: Atomic add unprivileged, without return: an alias of LDTADD, LDTADDA, LDTADDAL, LDTADDL.

STTCLR, STTCLRL: Atomic bit clear unprivileged, without return: an alias of LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL.

STTNP: Store unprivileged pair of registers, with non-temporal hint.

STTP: Store unprivileged pair of registers.

STTR: Store register (unprivileged).

STTRB: Store register byte (unprivileged).

STTRH: Store register halfword (unprivileged).

STTSET, STTSETL: Atomic bit set unprivileged, without return: an alias of LDTSET, LDTSETA, LDTSETAL, LDTSETL.

STTXR: Store unprivileged exclusive register.

STUMAX, STUMAXL: Atomic unsigned maximum on word or doubleword, without return: an alias of LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL.

STUMAXB, STUMAXLB: Atomic unsigned maximum on byte, without return: an alias of LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB.

STUMAXH, STUMAXLH: Atomic unsigned maximum on halfword, without return: an alias of LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH.

STUMIN, STUMINL: Atomic unsigned minimum on word or doubleword, without return: an alias of LDUMIN, LDUMINA, LDUMINAL, LDUMINL.

STUMINB, STUMINLB: Atomic unsigned minimum on byte, without return: an alias of LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB.

STUMINH, STUMINLH: Atomic unsigned minimum on halfword, without return: an alias of LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH.

STUR: Store register (unscaled).

STURB: Store register byte (unscaled).

STURH: Store register halfword (unscaled).

STXP: Store exclusive pair of registers.

STXR: Store exclusive register.

STXRB: Store exclusive register byte.

STXRH: Store exclusive register halfword.

STZ2G: Store Allocation Tags, zeroing.

STZG: Store Allocation Tag, zeroing.

STZGM: Store Allocation Tag and zero multiple.

SUB (extended register): Subtract extended and scaled register.

SUB (immediate): Subtract immediate value.

SUB (shifted register): Subtract optionally-shifted register.

SUBG: Subtract with tag.

SUBP: Subtract pointer.

SUBPS: Subtract pointer, setting flags.

SUBPT: Subtract checked pointer.

SUBS (extended register): Subtract extended and scaled register, setting flags.

SUBS (immediate): Subtract immediate value, setting flags.

SUBS (shifted register): Subtract optionally-shifted register, setting flags.

SVC: Supervisor call.

SWP, SWPA, SWPAL, SWPL: Swap word or doubleword in memory.

SWPB, SWPAB, SWPALB, SWPLB: Swap byte in memory.

SWPH, SWPAH, SWPALH, SWPLH: Swap halfword in memory.

SWPP, SWPPA, SWPPAL, SWPPL: Swap quadword in memory.

SWPT, SWPTA, SWPTAL, SWPTL: Swap unprivileged.

SXTB: Signed extend byte: an alias of SBFM.

SXTH: Sign extend halfword: an alias of SBFM.

SXTW: Sign extend word: an alias of SBFM.

SYS: System instruction.

SYSL: System instruction with result.

SYSP: 128-bit system instruction.

TBNZ: Test bit and branch if nonzero.

TBZ: Test bit and branch if zero.

TFLTNZ: Test bit and fault if nonzero.

TFLTZ: Test bit and fault if zero.

TLBI: TLB invalidate operation: an alias of SYS.

TLBIP: TLB invalidate pair operation: an alias of SYSP.

TRCIT: Trace instrumentation: an alias of SYS.

TSB: Trace synchronization barrier.

TST (immediate): Test bits (immediate): an alias of ANDS (immediate).

TST (shifted register): Test (shifted register): an alias of ANDS (shifted register).

UBFIZ: Unsigned bitfield insert in zeros: an alias of UBFM.

UBFM: Unsigned bitfield move.

UBFX: Unsigned bitfield extract: an alias of UBFM.

UDF: Permanently undefined.

UDIV (quotient): Unsigned divide.

UMADDL: Unsigned multiply-add long.

UMAX (immediate): Unsigned maximum (immediate).

UMAX (register): Unsigned maximum (register).

UMIN (immediate): Unsigned minimum (immediate).

UMIN (register): Unsigned minimum (register).

UMNEGL: Unsigned multiply-negate long: an alias of UMSUBL.

UMSUBL: Unsigned multiply-subtract long.

UMULH: Unsigned multiply high.

UMULL (32-bit): Unsigned multiply long: an alias of UMADDL.

UXTB: Unsigned extend byte: an alias of UBFM.

UXTH: Unsigned extend halfword: an alias of UBFM.

WFE: Wait for event.

WFET: Wait for event with timeout.

WFI: Wait for interrupt.

WFIT: Wait for interrupt with timeout.

XAFLAG: Convert floating-point condition flags from external format to Arm format.

XPACD, XPACI, XPACLRI: Strip Pointer Authentication Code.

YIELD: Yield.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

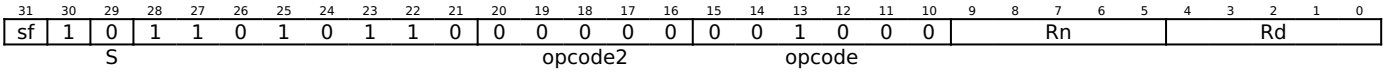
ABS

Absolute value

This instruction calculates the absolute value of the signed integer value in the source register, and writes the result to the destination register. If the value in the source register is the most negative value, the result is the same as the value in the source register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ABS <Wd>, <Wn>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ABS <Xd>, <Xn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let datasize : integer{} = 32 << UInt(sf);
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let result : integer = Abs(SInt(operand1));

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

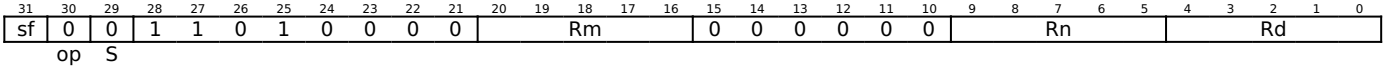
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADC

Add with carry

This instruction adds two register values and the Carry flag value, and writes the result to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ADC <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ADC <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);

var result : bits(datasize);
(result, -) = AddWithCarry{datasize}(operand1, operand2, PSTATE.C);

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

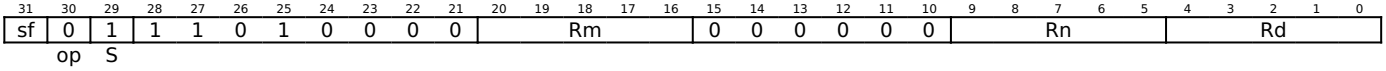
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADCS

Add with carry, setting flags

This instruction adds two register values and the Carry flag value, and writes the result to the destination register. It updates the condition flags based on the result.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ADCS <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ADCS <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);

var result : bits(datasize);
var nzcv : bits(4);

(result, nzcv) = AddWithCarry{datasize}(operand1, operand2, PSTATE.C);

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = nzcv;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

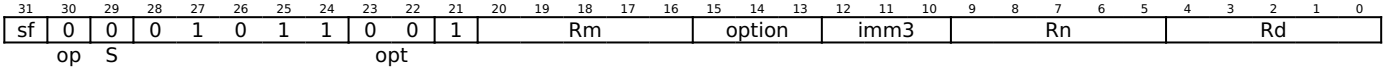
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADD (extended register)

Add extended and scaled register

This instruction adds a register value and a sign-extended or zero-extended register value, followed by an optional left shift amount, and writes the result to the destination register. The argument that is extended from the <Rm> register can be a byte, halfword, word, or doubleword.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ADD <Wd|WSP>, <Wn|WSP>, <Wm>{, <extend> {#<amount>}}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ADD <Xd|SP>, <Xn|SP>, <R><m>{, <extend> {#<amount>}}
```

Decode for all variants of this encoding

```
if imm3 IN {'101', '110', '111'} then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let shift : integer{} = UInt(imm3) as integer{0..4};
let datasize : integer{} = 32 << UInt(sf);
let extend_type : ExtendType = DecodeRegExtend(option);
```

Assembler Symbols

<Wd|WSP> Is the 32-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.

<Wn|WSP> Is the 32-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<extend> For the "32-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	LSL|UXTW
011	UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rd" or "Rn" is '11111' (WSP) and "option" is '010' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTW when "option" is '010'.

For the "64-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	UXTW
011	LSL|UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rd" or "Rn" is '11111' (SP) and "option" is '011' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTX when "option" is '011'.

<amount> Is the left shift amount to be applied after extension in the range 0 to 4, defaulting to 0, encoded in the "imm3" field. It must be absent when <extend> is absent, is required when <extend> is LSL, and is optional when <extend> is present but not LSL.

<Xd|SP> Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.

<Xn|SP> Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<R> Is a width specifier, encoded in “option”:

option	<R>
00x	W
010	W
x11	X
10x	W
110	W

<m> Is the number [0-30] of the second general-purpose source register or the name ZR (31), encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = if n == 31 then SP{datasize}() else X{datasize}(n);
let operand2 : bits(datasize) = ExtendReg{datasize}(m, extend_type, shift);

var result : bits(datasize);
(result, -) = AddWithCarry{datasize}(operand1, operand2, '0');

if d == 31 then
    SP{64}() = ZeroExtend{64}(result);
else
    X{datasize}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

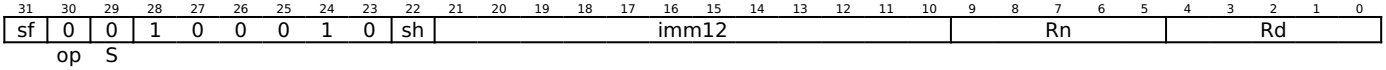
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADD (immediate)

Add immediate value

This instruction adds a register value and an optionally-shifted immediate value, and writes the result to the destination register.
This instruction is used by the alias MOV (to/from SP).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ADD <Wd|WSP>, <Wn|WSP>, #<imm>{, <shift>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ADD <Xd|SP>, <Xn|SP>, #<imm>{, <shift>}
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);

let imm : bits(24) = if sh == '0' then Zeros{12}::imm12 else imm12::Zeros{12};
```

Assembler Symbols

<Wd|WSP>	Is the 32-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Wn|WSP>	Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.
<imm>	Is an unsigned immediate, in the range 0 to 4095, encoded in the "imm12" field.
<shift>	Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #12

<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
MOV (to/from SP)	`sh == '0' && imm12 == '000000000000' && (Rd == '11111' || Rn == '11111')`

Operation

```
let operand1 : bits(datasize) = if n == 31 then SP{datasize}() else X{datasize}(n);
let operand2 : bits(datasize) = ZeroExtend{}(imm);

var result : bits(datasize);
(result, -) = AddWithCarry{datasize}(operand1, operand2, '0');

if d == 31 then
    SP{64}() = ZeroExtend{64}(result);
else
    X{datasize}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

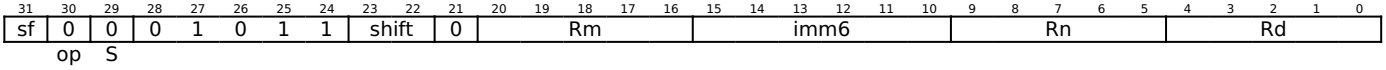
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADD (shifted register)

Add optionally-shifted register

This instruction adds a register value and an optionally-shifted register value, and writes the result to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ADD <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ADD <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if shift == '11' then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = ShiftReg{datasize}(m, shift_type, shift_amount);

var result : bits(datasize);
(result, -) = AddWithCarry{datasize}(operand1, operand2, '0');

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

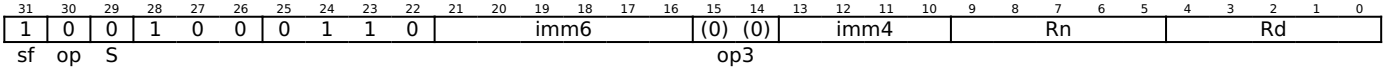
ADDG

Add with tag

This instruction adds an immediate value scaled by the Tag Granule to the address in the source register, modifies the Logical Address Tag of the address using an immediate value, and writes the result to the destination register. Tags specified in GCR_EL1.Exclude are excluded from the possible outputs when modifying the Logical Address Tag.

Integer

(FEAT_MTE)



Encoding

```
ADDG <Xd|SP>, <Xn|SP>, #<uimm6>, #<uimm4>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let tag_offset : bits(4) = imm4;
let offset : bits(64) = LSL(ZeroExtend{64}(imm6), LOG2_TAG_GRANULE);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.
<uimm6>	Is an unsigned immediate, a multiple of 16 in the range 0 to 1008, encoded in the "imm6" field.
<uimm4>	Is an unsigned immediate, in the range 0 to 15, encoded in the "imm4" field.

Operation

```
let operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let start_tag : bits(4) = AArch64_AllocationTagFromAddress(operand1);
let exclude : bits(16) = GCR_EL1().Exclude;
let rtag : bits(4) = AArch64_ChooseNonExcludedTagOrZero(start_tag, tag_offset, exclude);
var result : bits(64);

(result, -) = AddWithCarry{64}(operand1, offset, '0');

result = AArch64_AddressWithAllocationTag(result, rtag);

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDPT

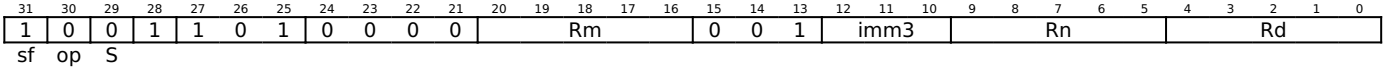
Add checked pointer

This instruction adds a base address register value and an optionally-shifted register value, and writes the result to the destination register. The optionally-shifted register value is treated as the offset.

If the operation would have generated a result where the most significant 8 bits of the result register differ from the most significant 8 bits of the base register, then the result is modified such that it is likely to be non-canonical when used as an address.

Integer

(FEAT_CPA)



Encoding

```
ADDPT <Xd|SP>, <Xn|SP>, <Xm>{, LSL #<amount>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CPA) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let shift : integer = UInt(imm3);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the general-purpose destination register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the first general-purpose source register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.
<amount>	Is the left shift amount, in the range 0 to 7, defaulting to 0, encoded in the "imm3" field.

Operation

```
var result : bits(64);
let base : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let offset : bits(64) = LSL(X{64}(m), shift);

result = base + offset;
result = PointerAddCheck(result, base);

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

2026-09_rel 2026-09-21 14:22:38

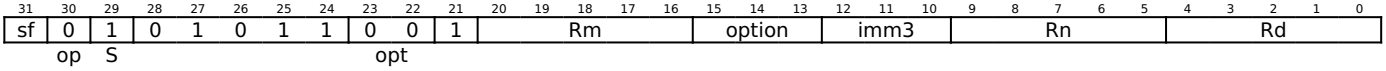
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDS (extended register)

Add extended and scaled register, setting flags

This instruction adds a register value and a sign-extended or zero-extended register value, followed by an optional left shift amount, and writes the result to the destination register. The argument that is extended from the <Rm> register can be a byte, halfword, word, or doubleword. It updates the condition flags based on the result.

This instruction is used by the alias CMN (extended register).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ADDS <Wd>, <Wn|WSP>, <Wm>{, <extend> {#<amount>}}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ADDS <Xd>, <Xn|SP>, <R><m>{, <extend> {#<amount>}}
```

Decode for all variants of this encoding

```
if imm3 IN {'101', '110', '111'} then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let shift : integer{} = UInt(imm3) as integer{0..4};
let datasize : integer{} = 32 << UInt(sf);
let extend_type : ExtendType = DecodeRegExtend(option);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn|WSP> Is the 32-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<extend> For the "32-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	LSL|UXTW
011	UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rn" is '11111' (WSP) and "option" is '010' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTW when "option" is '010'.

For the "64-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	UXTW
011	LSL|UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rn" is '11111' (SP) and "option" is '011' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTX when "option" is '011'.

<amount> Is the left shift amount to be applied after extension in the range 0 to 4, defaulting to 0, encoded in the "imm3" field. It must be absent when <extend> is absent, is required when <extend> is LSL, and is optional when <extend> is present but not LSL.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn|SP> Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<R> Is a width specifier, encoded in “option”:

option	<R>
00x	W
010	W
x11	X
10x	W
110	W

<m> Is the number [0-30] of the second general-purpose source register or the name ZR (31), encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CMN (extended register)	`Rd == '11111'`

Operation

```
let operand1 : bits(datasize) = if n == 31 then SP{datasize}() else X{datasize}(n);
let operand2 : bits(datasize) = ExtendReg{datasize}(m, extend_type, shift);

var result : bits(datasize);
var nzcv : bits(4);

(result, nzcv) = AddWithCarry{datasize}(operand1, operand2, '0');

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = nzcv;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

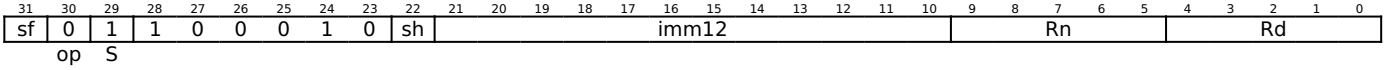
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDS (immediate)

Add immediate value, setting flags

This instruction adds a register value and an optionally-shifted immediate value, and writes the result to the destination register. It updates the condition flags based on the result.

This instruction is used by the alias CMN (immediate).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ADDS <Wd>, <Wn|WSP>, #<imm>{, <shift>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ADDS <Xd>, <Xn|SP>, #<imm>{, <shift>}
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);

let imm : bits(24) = if sh == '0' then Zeros{12}::imm12 else imm12::Zeros{12};
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn|WSP>	Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.
<imm>	Is an unsigned immediate, in the range 0 to 4095, encoded in the "imm12" field.
<shift>	Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #12

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
CMN (immediate)	`Rd == '11111'`

Operation

```
let operand1 : bits(datasize) = if n == 31 then SP{datasize}() else X{datasize}(n);
let operand2 : bits(datasize) = ZeroExtend{}(imm);

var result : bits(datasize);
var nzcv : bits(4);

(result, nzcv) = AddWithCarry{datasize}(operand1, operand2, '0');

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = nzcv;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

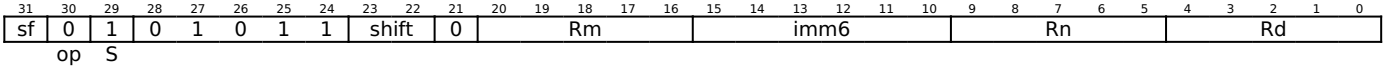
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDS (shifted register)

Add optionally-shifted register, setting flags

This instruction adds a register value and an optionally-shifted register value, and writes the result to the destination register. It updates the condition flags based on the result.

This instruction is used by the alias CMN (shifted register).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ADDS <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ADDS <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if shift == '11' then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CMN (shifted register)	`Rd == '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = ShiftReg{datasize}(m, shift_type, shift_amount);

var result : bits(datasize);
var nzcv : bits(4);

(result, nzcv) = AddWithCarry{datasize}(operand1, operand2, '0');

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = nzcv;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

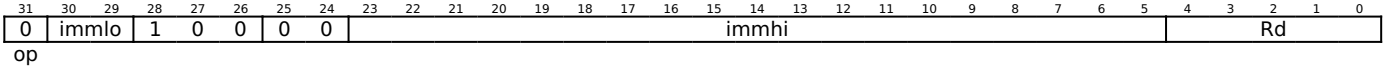
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADR

Form PC-relative address

This instruction adds an immediate value to the PC value to form a PC-relative address, and writes the result to the destination register.



Encoding

```
ADR <Xd>, <label>
```

Decode for this encoding

```
let d : integer{} = UInt(Rd);
let imm : bits(64) = SignExtend{}(immhi::immlo);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<label>	Is the program label whose address is to be calculated. Its offset from the address of this instruction, in the range +/-1MB, is encoded in "immhi:immlo".

Operation

```
X{64}(d) = PC64() + imm;
```

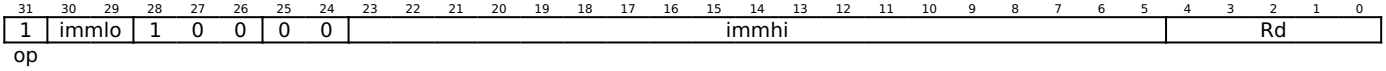
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADRP

Form PC-relative address to 4KB page

This instruction adds an immediate value that is shifted left by 12 bits, to the PC value to form a PC-relative address, with the bottom 12 bits masked out, and writes the result to the destination register.



Encoding

```
ADRP <Xd>, <label>
```

Decode for this encoding

```
let d : integer{} = UInt(Rd);
let imm : bits(64) = SignExtend{}(immhi::immlo::Zeros{12});
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<label>	Is the program label whose 4KB page address is to be calculated. Its offset from the page address of this instruction, in the range +/-4GB, is encoded as "immhi:immlo" times 4096.

Operation

```
let base : bits(64) = PC64()[63:12]::Zeros{12};
X{64}(d) = base + imm;
```

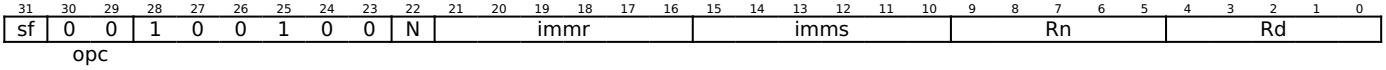
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AND (immediate)

Bitwise AND (immediate)

This instruction performs a bitwise AND between a register value and an immediate value, and writes the result to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
AND <Wd|WSP>, <Wn>, #<imm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
AND <Xd|SP>, <Xn>, #<imm>
```

Decode for all variants of this encoding

```
if sf == '0' && N != '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
var imm : bits(datasize);
(imm, -) = DecodeBitMasks{datasize}(N, imms, immr, TRUE);
```

Assembler Symbols

<Wd|WSP>	Is the 32-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<imm>	Is the bitmask immediate. For encoding information, see *Logical (immediate)*.
<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = imm;
let result : bits(datasize) = operand1 AND operand2;

if d == 31 then
    SP{64}() = ZeroExtend{64}(result);
else
    X{datasize}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

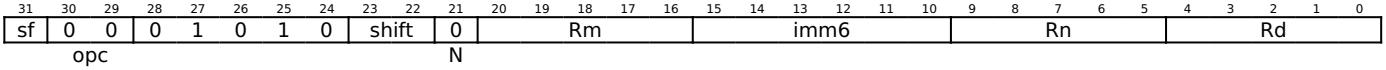
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AND (shifted register)

Bitwise AND (shifted register)

This instruction performs a bitwise AND between a register value and an optionally-shifted register value, and writes the result to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
AND <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
AND <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift to be applied to the final source, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	ROR

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = ShiftReg{datasize}(m, shift_type, shift_amount);
let result : bits(datasize) = operand1 AND operand2;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

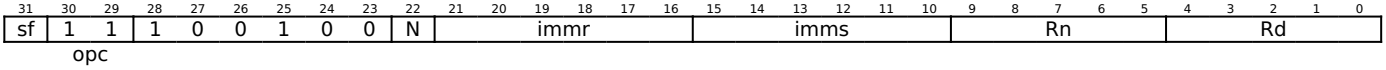
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ANDS (immediate)

Bitwise AND (immediate), setting flags

This instruction performs a bitwise AND between a register value and an immediate value, and writes the result to the destination register. It updates the condition flags based on the result.

This instruction is used by the alias TST (immediate).



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
ANDS <Wd>, <Wn>, #<imm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ANDS <Xd>, <Xn>, #<imm>
```

Decode for all variants of this encoding

```
if sf == '0' && N != '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
var imm : bits(datasize);
(imm, -) = DecodeBitMasks{datasize}(N, imms, immr, TRUE);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<imm>	Is the bitmask immediate. For encoding information, see *Logical (immediate)*.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
TST (immediate)	`Rd == '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = imm;
let result : bits(datasize) = operand1 AND operand2;

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = result[datasize-1]::IsZeroBit{datasize}(result)::'00';
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

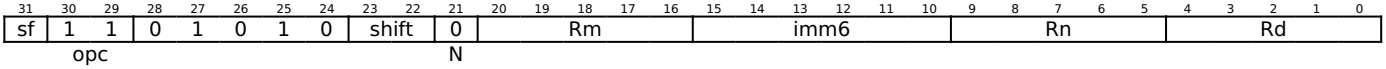
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ANDS (shifted register)

Bitwise AND (shifted register), setting flags

This instruction performs a bitwise AND between a register value and an optionally-shifted register value, and writes the result to the destination register. It updates the condition flags based on the result.

This instruction is used by the alias TST (shifted register).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ANDS <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ANDS <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift to be applied to the final source, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	ROR

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
TST (shifted register)	`Rd == '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = ShiftReg{datasize}(m, shift_type, shift_amount);
let result : bits(datasize) = operand1 AND operand2;

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = result[datasize-1]::IsZeroBit{datasize}(result)::'00';
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

APAS

Associate physical address space

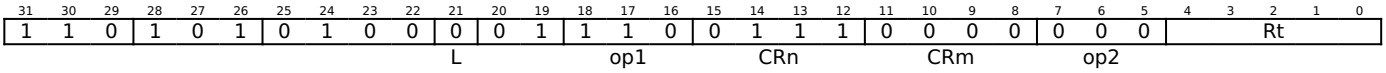
This instruction associates a physical address space with a memory-mapped location that is protected by a memory-side physical address space filter.
For more information, see APAS, Associate PA space.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_RME_GPC3)



Encoding

```
APAS <Xt>
```

is equivalent to

```
SYS #6, C7, C0, #0, <Xt>
```

and is always the preferred disassembly.

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

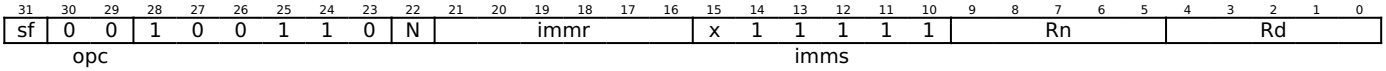
ASR (immediate)

Arithmetic shift right (immediate)

This instruction shifts a register value right by an immediate number of bits, shifting in copies of the sign bit in the upper bits and zeros in the lower bits, and writes the result to the destination register.

This is an alias of SBFM. This means:

- The encodings in this description are named to match the encodings of SBFM.
- The description of SBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0 && imms == 011111)`

```
ASR <Wd>, <Wn>, #<shift>
```

is equivalent to

```
SBFM <Wd>, <Wn>, #<shift>, #31
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1 && imms == 111111)`

```
ASR <Xd>, <Xn>, #<shift>
```

is equivalent to

```
SBFM <Xd>, <Xn>, #<shift>, #63
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<shift>	For the "32-bit" variant: is the shift amount, in the range 0 to 31, encoded in the "immr" field. For the "64-bit" variant: is the shift amount, in the range 0 to 63, encoded in the "immr" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of SBFM gives the operational pseudocode for this instruction.

Operational Information

The description of SBFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

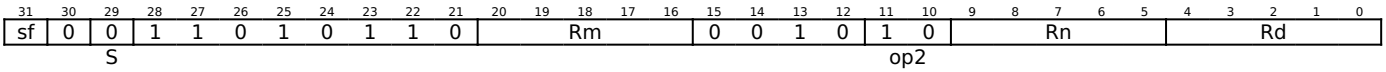
ASR (register)

Arithmetic shift right (register)

This instruction shifts a register value right by a variable number of bits, shifting in copies of its sign bit, and writes the result to the destination register. The value of the second source register modulo the register size in bits gives the number of bits by which the first source register is right-shifted.

This is an alias of ASRV. This means:

- The encodings in this description are named to match the encodings of ASRV.
- The description of ASRV gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ASR <Wd>, <Wn>, <Wm>
```

is equivalent to

```
ASRV <Wd>, <Wn>, <Wm>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ASR <Xd>, <Xn>, <Xm>
```

is equivalent to

```
ASRV <Xd>, <Xn>, <Xm>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding a shift amount from 0 to 31 in its bottom 5 bits, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding a shift amount from 0 to 63 in its bottom 6 bits, encoded in the "Rm" field.

Operation

The description of ASRV gives the operational pseudocode for this instruction.

Operational Information

The description of ASRV gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

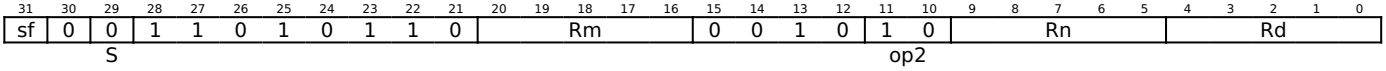
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ASRV

Arithmetic shift right variable

This instruction shifts a register value right by a variable number of bits, shifting in copies of its sign bit, and writes the result to the destination register. The value of the second source register modulo the register size in bits gives the number of bits by which the first source register is right-shifted.

This instruction is used by the alias ASR (register).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ASRV <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ASRV <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(op2);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding a shift amount from 0 to 31 in its bottom 5 bits, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding a shift amount from 0 to 63 in its bottom 6 bits, encoded in the "Rm" field.

Operation

```
let operand2 : bits(datasize) = X{datasize}(m);

X{datasize}(d) = ShiftReg{datasize}(n, shift_type, UInt(operand2) MOD datasize);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

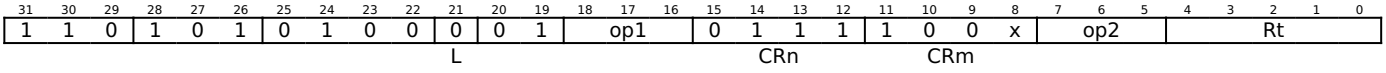
AT

Address translate

For more information, see *op0 == 0b01, cache maintenance, TLB maintenance, address translation, prediction restriction, BRBE, Trace Extension, Guarded Control Stack, and FEAT_RME_GPC3 instructions*.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

`AT <at_op>, <Xt>`

is equivalent to

`SYS #<op1>, C7, <Cm>, #<op2>, <Xt>`

and is the preferred disassembly when `SysOp(op1, '0111', CRm, op2) == Sys_AT`.

Assembler Symbols

<at_op> Is an AT operation name, as listed for the AT system instruction group, encoded in “(op1 :: CRm :: op2)”:

op1	CRm	op2	<at_op>	Architectural Feature	Description
000	1000	000	S1E1R	–	For more information, see AT S1E1R.
000	1000	001	S1E1W	–	For more information, see AT S1E1W.
000	1000	010	S1E0R	–	For more information, see AT S1E0R.
000	1000	011	S1E0W	–	For more information, see AT S1E0W.
000	1001	000	S1E1RP	FEAT_PAN2	For more information, see AT S1E1RP.
000	1001	001	S1E1WP	FEAT_PAN2	For more information, see AT S1E1WP.
000	1001	010	S1E1A	FEAT_ATS1A	For more information, see AT S1E1A.
100	1000	000	S1E2R	–	For more information, see AT S1E2R.
100	1000	001	S1E2W	–	For more information, see AT S1E2W.
100	1000	100	S12E1R	–	For more information, see AT S12E1R.
100	1000	101	S12E1W	–	For more information, see AT S12E1W.
100	1000	110	S12E0R	–	For more information, see AT S12E0R.
100	1000	111	S12E0W	–	For more information, see AT S12E0W.
100	1001	010	S1E2A	FEAT_ATS1A	For more information, see AT S1E2A.
110	1000	000	S1E3R	–	For more information, see AT S1E3R.
110	1000	001	S1E3W	–	For more information, see AT S1E3W.
110	1001	010	S1E3A	FEAT_ATS1A	For more information, see AT S1E3A.

<Xt> Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AUTDA, AUTDZA

Authenticate data address, using key A

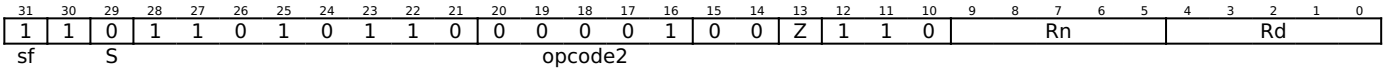
This instruction authenticates a data address, using a modifier and key A.
The address is in the general-purpose register that is specified by <Xd>.
The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xn|SP>, for `AUTDA`.
- The value zero, for `AUTDZA`.

If the authentication passes, the upper bits of the address are restored to enable subsequent use of the address. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.

Integer

(FEAT_PAuth)



Encoding for the AUTDA variant

Applies when `(Z == 0)`

```
AUTDA <Xd>, <Xn|SP>
```

Encoding for the AUTDZA variant

Applies when `(Z == 1 && Rn == 11111)`

```
AUTDZA <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;
let auth_combined : boolean = FALSE;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let source_is_sp : boolean = Z == '0' && n == 31;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

```
if source_is_sp then
    X{64}(d) = AuthDA(X{64}(d), SP{64}(), auth_combined);
else
    X{64}(d) = AuthDA(X{64}(d), X{64}(n), auth_combined);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AUTDB, AUTDZB

Authenticate data address, using key B

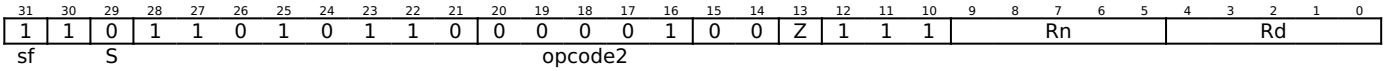
This instruction authenticates a data address, using a modifier and key B.
The address is in the general-purpose register that is specified by <Xd>.
The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xn|SP> for AUTDB.
- The value zero, for AUTDZB.

If the authentication passes, the upper bits of the address are restored to enable subsequent use of the address. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.

Integer

(FEAT_PAuth)



Encoding for the AUTDB variant

Applies when `(Z == 0)`

```
AUTDB <Xd>, <Xn|SP>
```

Encoding for the AUTDZB variant

Applies when `(Z == 1 && Rn == 11111)`

```
AUTDZB <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;
let auth_combined : boolean = FALSE;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let source_is_sp : boolean = Z == '0' && n == 31;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

```
if source_is_sp then
    X{64}(d) = AuthDB(X{64}(d), SP{64}(), auth_combined);
else
    X{64}(d) = AuthDB(X{64}(d), X{64}(n), auth_combined);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA

Authenticate instruction address, using key A

This instruction authenticates an instruction address, using a modifier and key A.
If the authentication passes, the upper bits of the address are restored to enable subsequent use of the address. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.
The address is:

- In the general-purpose register that is specified by <Xd> for AUTIA and AUTIZA.
- In X17, for AUTIA1716.
- In X30, for AUTIASP and AUTIAZ.

The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xn|SP> for AUTIA.
- The value zero, for AUTIZA and AUTIAZ.
- In X16, for AUTIA1716.
- In SP, for AUTIASP.

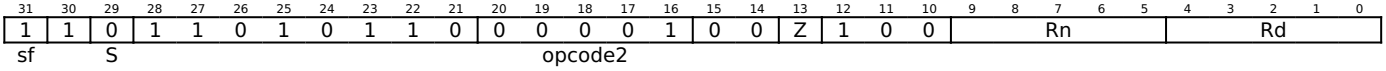
If *FEAT_PAuth_LR* is implemented and PSTATE.PACM is 1, then AUTIA1716 and AUTIASP include a second modifier that is:

- In X15, for AUTIA1716.
- In X16, for AUTIASP.

It has encodings from 2 classes: Integer and System

Integer

(FEAT_PAuth)



Encoding for the AUTIA variant

Applies when (Z == 0)

```
AUTIA <Xd>, <Xn|SP>
```

Encoding for the AUTIZA variant

Applies when (Z == 1 && Rn == 11111)

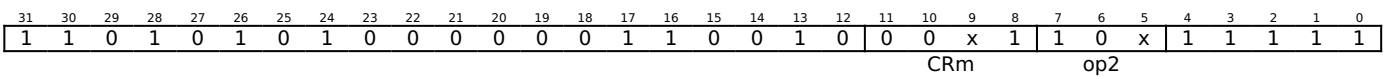
```
AUTIZA <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let source_is_sp : boolean = Z == '0' && n == 31;
let autia1716 : boolean = FALSE;
let auth_combined : boolean = FALSE;
```

System

(FEAT_PAuth)



Encoding for the AUTIA1716 variant

Applies when (CRm == 0001 && op2 == 100)

```
AUTIA1716
```

Encoding for the AUTIASP variant

Applies when (CRm == 0011 && op2 == 101)

```
AUTIASP
```

Encoding for the AUTIAZ variant

Applies when (CRm == 0011 && op2 == 100)

AUTIAZ

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_NOP); end;

var d : integer;
var n : integer;
var source_is_sp : boolean = FALSE;
var autia1716 : boolean = FALSE;
let auth_combined : boolean = FALSE;

case CRm::op2 of
    when '0011 100' => // AUTIAZ
        d = 30;
        n = 31;
    when '0011 101' => // AUTIASP
        d = 30;
        source_is_sp = TRUE;
    when '0001 100' => // AUTIA1716
        d = 17;
        n = 16;
        autia1716 = TRUE;
end;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

```
if source_is_sp then
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' then
        X{64}(d) = AuthIA2(X{64}(d), SP{64}(), X{64}(16), auth_combined);
    else
        X{64}(d) = AuthIA(X{64}(d), SP{64}(), auth_combined);
    end;
else
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' && autia1716 then
        X{64}(d) = AuthIA2(X{64}(d), X{64}(n), X{64}(15), auth_combined);
    else
        X{64}(d) = AuthIA(X{64}(d), X{64}(n), auth_combined);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AUTIA171615

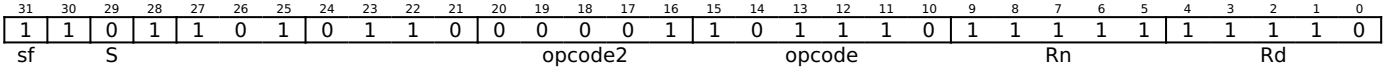
Authenticate instruction address, using key A

This instruction authenticates an instruction address, using two modifiers and key A. The address is in X17. The 64-bit value of modifier1 is the value in X16. The 64-bit value of modifier2 is the value in X15.

If the authentication passes, the upper bits of the address are restored to enable subsequent use of the address. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.

Integer

(FEAT_PAuth_LR)



Encoding

AUTIA171615

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
let is_combined : boolean = FALSE;
X{64}(17) = AuthIA2(X{64}(17), X{64}(16), X{64}(15), is_combined);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

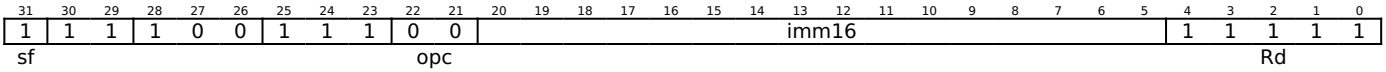
AUTIASPPC

Authenticate return address using key A, using an immediate offset

This instruction authenticates an instruction address, using two modifiers and key A.
If the authentication passes, the upper bits of the address are restored to enable subsequent use of the address. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.
The address is in X30.
The first modifier is in SP.
The second modifier is the address of a program label.

Integer

(FEAT_PAuth_LR)



Encoding

```
AUTIASPPC <label>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;

let d : integer = 30;
let offset : bits(64) = ZeroExtend{}(imm16::'00');
let auth_combined : boolean = FALSE;
```

Assembler Symbols

<label>	Is the program label whose address is to be calculated. Its negative offset from the address of this instruction, a multiple of 4 in the range -262140 to 0, is encoded as an unsigned value in the "imm16" field as <label>/4.

Operation

```
let pac_addr : bits(64) = PC64() - offset;

X{64}(d) = AuthIA2(X{64}(d), SP{64}(), pac_addr, auth_combined);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

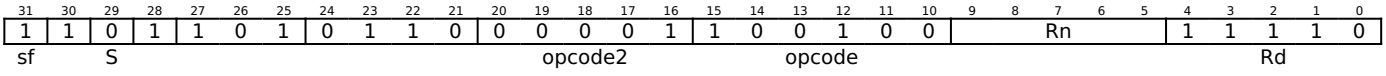
AUTIASPPCR

Authenticate return address using key A, using a register

This instruction authenticates an instruction address, using two modifiers and key A.
If the authentication passes, the upper bits of the address are restored to enable subsequent use of the address. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.
The address is in X30.
The first modifier is in SP.
The second modifier is in the general-purpose register that is specified by <Xn>.

Integer

(FEAT_PAuth_LR)



Encoding

```
AUTIASPPCR <Xn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;

let d : integer = 30;
let n : integer = UInt(Rn);
let auth_combined : boolean = FALSE;
```

Assembler Symbols

<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
X{64}(d) = AuthIA2(X{64}(d), SP{64}(), X{64}(n), auth_combined);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB

Authenticate instruction address, using key B

This instruction authenticates an instruction address, using a modifier and key B.
If the authentication passes, the upper bits of the address are restored to enable subsequent use of the address. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.
The address is:

- In the general-purpose register that is specified by <Xd> for `AUTIB` and `AUTIZB`.
- In X17, for `AUTIB1716`.
- In X30, for `AUTIBSP` and `AUTIBZ`.

The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xn|SP> for `AUTIB`.
- The value zero, for `AUTIZB` and `AUTIBZ`.
- In X16, for `AUTIB1716`.
- In SP, for `AUTIBSP`.

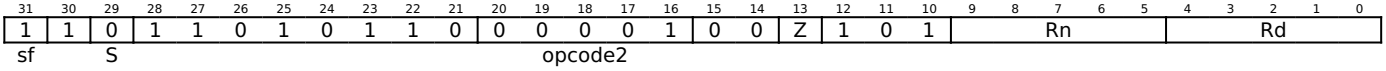
If *FEAT_PAuth_LR* is implemented and PSTATE.PACM is 1, then `AUTIB1716` and `AUTIBSP` include a second modifier that is:

- In X15, for `AUTIB1716`.
- In X16, for `AUTIBSP`.

It has encodings from 2 classes: Integer and System

Integer

(FEAT_PAuth)



Encoding for the AUTIB variant

Applies when `(Z == 0)`

```
AUTIB <Xd>, <Xn|SP>
```

Encoding for the AUTIZB variant

Applies when `(Z == 1 && Rn == 11111)`

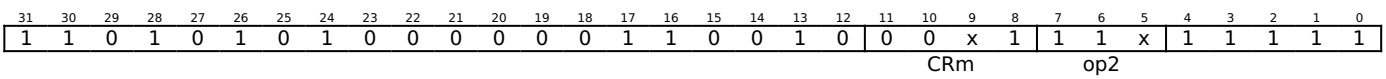
```
AUTIZB <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let source_is_sp : boolean = Z == '0' && n == 31;
let autib1716 : boolean = FALSE;
let auth_combined : boolean = FALSE;
```

System

(FEAT_PAuth)



Encoding for the AUTIB1716 variant

Applies when `(CRm == 0001 && op2 == 110)`

```
AUTIB1716
```

Encoding for the AUTIBSP variant

Applies when `(CRm == 0011 && op2 == 111)`

```
AUTIBSP
```

Encoding for the AUTIBZ variant

Applies when `(CRm == 0011 && op2 == 110)`

AUTIBZ

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_NOP); end;

var d : integer;
var n : integer;
var source_is_sp : boolean = FALSE;
var autib1716 : boolean = FALSE;
let auth_combined : boolean = FALSE;

case CRm::op2 of
    when '0011 110' => // AUTIBZ
        d = 30;
        n = 31;
    when '0011 111' => // AUTIBSP
        d = 30;
        source_is_sp = TRUE;
    when '0001 110' => // AUTIB1716
        d = 17;
        n = 16;
        autib1716 = TRUE;
end;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

```
if source_is_sp then
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' then
        X{64}(d) = AuthIB2(X{64}(d), SP{64}(), X{64}(16), auth_combined);
    else
        X{64}(d) = AuthIB(X{64}(d), SP{64}(), auth_combined);
    end;
else
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' && autib1716 then
        X{64}(d) = AuthIB2(X{64}(d), X{64}(n), X{64}(15), auth_combined);
    else
        X{64}(d) = AuthIB(X{64}(d), X{64}(n), auth_combined);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AUTIB171615

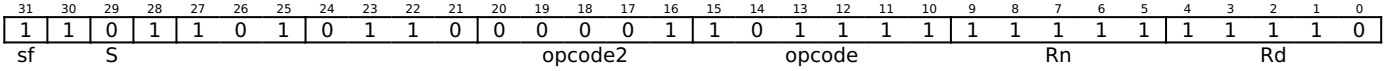
Authenticate instruction address, using key B

This instruction authenticates an instruction address, using two modifiers and key B. The address is in X17. The 64-bit value of modifier1 is the value in X16. The 64-bit value of modifier2 is the value in X15.

If the authentication passes, the upper bits of the address are restored to enable subsequent use of the address. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.

Integer

(FEAT_PAuth_LR)



Encoding

AUTIB171615

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
let is_combined : boolean = FALSE;
X{64}(17) = AuthIB2(X{64}(17), X{64}(16), X{64}(15), is_combined);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

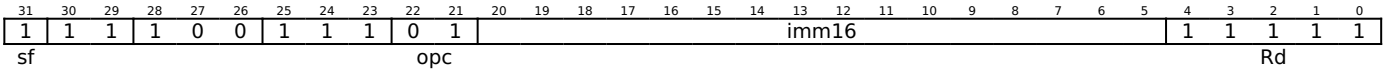
AUTIBSPPC

Authenticate return address using key B, using an immediate offset

This instruction authenticates an instruction address, using two modifiers and key B.
If the authentication passes, the upper bits of the address are restored to enable subsequent use of the address. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.
The address is in X30.
The first modifier is in SP.
The second modifier is the address of a program label.

Integer

(FEAT_PAuth_LR)



Encoding

```
AUTIBSPPC <label>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;

let d : integer = 30;
let offset : bits(64) = ZeroExtend{}(imm16::'00');
let auth_combined : boolean = FALSE;
```

Assembler Symbols

<label>	Is the program label whose address is to be calculated. Its negative offset from the address of this instruction, a multiple of 4 in the range -262140 to 0, is encoded as an unsigned value in the "imm16" field as <label>/4.

Operation

```
let pac_addr : bits(64) = PC64() - offset;

X{64}(d) = AuthIB2(X{64}(d), SP{64}(), pac_addr, auth_combined);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

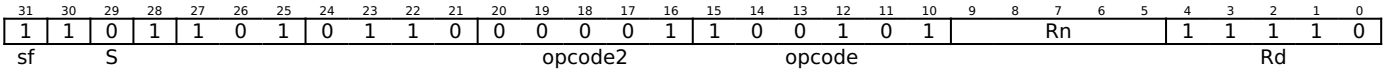
AUTIBSPPCR

Authenticate return address using key B, using a register

This instruction authenticates an instruction address, using two modifiers and key B.
If the authentication passes, the upper bits of the address are restored to enable subsequent use of the address. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.
The address is in X30.
The first modifier is in SP.
The second modifier is in the general-purpose register that is specified by <Xn>.

Integer

(FEAT_PAuth_LR)



Encoding

```
AUTIBSPPCR <Xn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;

let d : integer = 30;
let n : integer = UInt(Rn);
let auth_combined : boolean = FALSE;
```

Assembler Symbols

<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
X{64}(d) = AuthIB2(X{64}(d), SP{64}(), X{64}(n), auth_combined);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

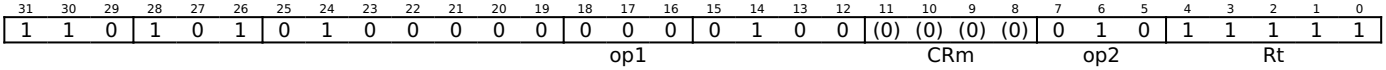
AXFLAG

Convert floating-point condition flags from Arm to external format

This instruction converts the state of the PSTATE.{N,Z,C,V} flags from a form representing the result of an Arm floating-point scalar compare instruction to an alternative representation required by some software.

System

(FEAT_FlagM2)



Encoding

AXFLAG

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FlagM2) then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
let z : bit = PSTATE.Z OR PSTATE.V;
let c : bit = PSTATE.C AND NOT PSTATE.V;

PSTATE.[N,Z,C,V] = '0' :: z :: c :: '0';
```

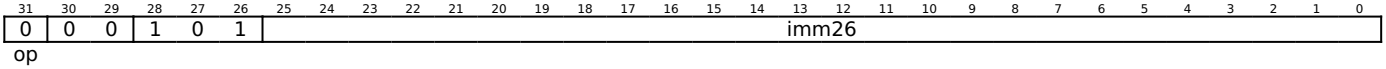
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

B

Branch

This instruction branches unconditionally to a label at a PC-relative offset. This instruction provides a hint that this is not a subroutine call or return.



Encoding

```
B <label>
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm26::'00');
```

Assembler Symbols

<label>	Is the program label to be unconditionally branched to. Its offset from the address of this instruction, in the range +/-128MB, is encoded as "imm26" times 4.

Operation

```
let branch_conditional : boolean = FALSE;
BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
```

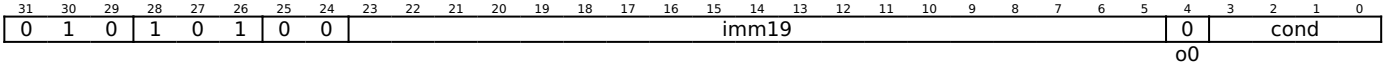
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

B.cond

Branch conditionally

This instruction branches conditionally to a label at a PC-relative offset. This instruction provides a hint that this is not a subroutine call or return.



Encoding

B.<cond> <label>

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm19::'00');
let condition : bits(4) = cond;
```

Assembler Symbols

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<label> Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range +/-1MB, is encoded as "imm19" times 4.

Operation

```
let branch_conditional : boolean = TRUE;
if ConditionHolds(condition) then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

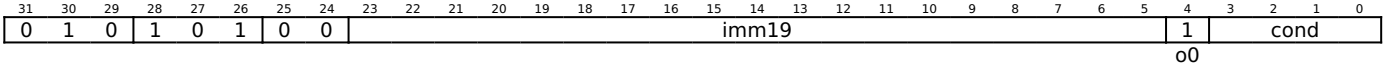
BC.cond

Branch consistent conditionally

This instruction branches conditionally to a label at a PC-relative offset, with a hint that this branch is very unlikely to change direction. This instruction provides a hint that this is not a subroutine call or return.

19-bit signed PC-relative branch offset

(FEAT_HBC)



Encoding

```
BC.<cond> <label>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_HBC) then EndOfDecode(Decode_UNDEF); end;
let offset : bits(64) = SignExtend{}(imm19::'00');
let condition : bits(4) = cond;
```

Assembler Symbols

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<label> Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range +/-1MB, is encoded as "imm19" times 4.

Operation

```
let branch_conditional : boolean = TRUE;
if ConditionHolds(condition) then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFC

Bitfield clear

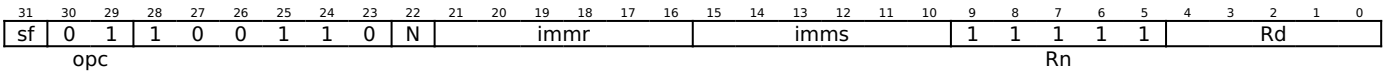
This instruction sets a bitfield of <width> bits at bit position <lsb> of the destination register to zero, leaving the other destination bits unchanged.

This is an alias of BFM. This means:

- The encodings in this description are named to match the encodings of BFM.
- The description of BFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Leaving other bits unchanged

(FEAT_ASMv8p2)



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
BFC <Wd>, #<lsb>, #<width>
```

is equivalent to

```
BFM <Wd>, WZR, #(-<lsb> MOD 32), #(<width>-1)
```

and is the preferred disassembly when `UInt(imms) < UInt(immr)`.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
BFC <Xd>, #<lsb>, #<width>
```

is equivalent to

```
BFM <Xd>, XZR, #(-<lsb> MOD 64), #(<width>-1)
```

and is the preferred disassembly when `UInt(imms) < UInt(immr)`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<lsb>	For the "32-bit" variant: is the bit number of the lsb of the destination bitfield, in the range 0 to 31. For the "64-bit" variant: is the bit number of the lsb of the destination bitfield, in the range 0 to 63.
<width>	For the "32-bit" variant: is the width of the bitfield, in the range 1 to 32-<lsb>. For the "64-bit" variant: is the width of the bitfield, in the range 1 to 64-<lsb>.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

The description of BFM gives the operational pseudocode for this instruction.

Operational Information

The description of BFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

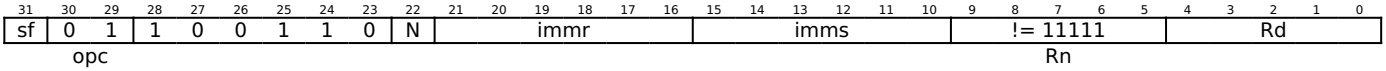
BFI

Bitfield insert

This instruction copies a bitfield of <width> bits from the least significant bits of the source register to bit position <lsb> of the destination register, leaving the other destination bits unchanged.

This is an alias of BFM. This means:

- The encodings in this description are named to match the encodings of BFM.
- The description of BFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
BFI <Wd>, <Wn>, #<lsb>, #<width>
```

is equivalent to

```
BFM <Wd>, <Wn>, #(-<lsb> MOD 32), #(<width>-1)
```

and is the preferred disassembly when `UInt(imms) < UInt(immr)`.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
BFI <Xd>, <Xn>, #<lsb>, #<width>
```

is equivalent to

```
BFM <Xd>, <Xn>, #(-<lsb> MOD 64), #(<width>-1)
```

and is the preferred disassembly when `UInt(imms) < UInt(immr)`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<lsb>	For the "32-bit" variant: is the bit number of the lsb of the destination bitfield, in the range 0 to 31. For the "64-bit" variant: is the bit number of the lsb of the destination bitfield, in the range 0 to 63.
<width>	For the "32-bit" variant: is the width of the bitfield, in the range 1 to 32-<lsb>. For the "64-bit" variant: is the width of the bitfield, in the range 1 to 64-<lsb>.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of BFM gives the operational pseudocode for this instruction.

Operational Information

The description of BFM gives the operational information for this instruction.

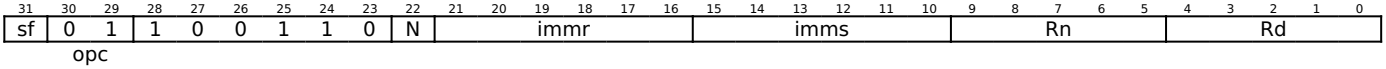
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFM

Bitfield move

This instruction is usually accessed using one of its aliases, which are always preferred for disassembly.
If <imms> is greater than or equal to <immr>, this copies a bitfield of (<imms>-<immr>+1) bits starting from bit position <immr> in the source register to the least significant bits of the destination register.
If <imms> is less than <immr>, this copies a bitfield of (<imms>+1) bits from the least significant bits of the source register to bit position (regsize-<immr>) of the destination register, where regsize is the destination register size of 32 or 64 bits.
In both cases, the other bits of the destination register remain unchanged.
This instruction is used by the aliases BFC, BFI, and BFXIL.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
BFM <Wd>, <Wn>, #<immr>, #<imms>
```

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
BFM <Xd>, <Xn>, #<immr>, #<imms>
```

Decode for all variants of this encoding

```
if sf == '1' && N != '1' then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && (N != '0' || immr[5] != '0' || imms[5] != '0') then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let r : integer = UInt(immr);

var wmask : bits(datasize);
var tmask : bits(datasize);
(wmask, tmask) = DecodeBitMasks{datasize}(N, imms, immr, FALSE);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<immr>	For the "32-bit" variant: is the right rotate amount, in the range 0 to 31, encoded in the "immr" field. For the "64-bit" variant: is the right rotate amount, in the range 0 to 63, encoded in the "immr" field.
<imms>	For the "32-bit" variant: is the leftmost bit number to be moved from the source, in the range 0 to 31, encoded in the "imms" field. For the "64-bit" variant: is the leftmost bit number to be moved from the source, in the range 0 to 63, encoded in the "imms" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
BFC	`Rn == '11111' && UInt(imms) < UInt(immr)`
BFI	`Rn != '11111' && UInt(imms) < UInt(immr)`
BFXIL	`UInt(imms) >= UInt(immr)`

Operation

```
let src : bits(datasize) = X{}(n);

let dst : bits(datasize) = X{}(d);

// Perform bitfield move on low bits
let bot : bits(datasize) = (dst AND NOT wmask) OR (ROR(src, r) AND wmask);

// Combine extension bits and result bits
X{datasize}(d) = (dst AND NOT tmask) OR (bot AND tmask);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

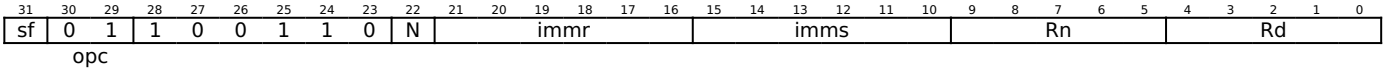
BFXIL

Bitfield extract and insert at low end

This instruction copies a bitfield of <width> bits starting from bit position <lsb> in the source register to the least significant bits of the destination register, leaving the other destination bits unchanged.

This is an alias of BFM. This means:

- The encodings in this description are named to match the encodings of BFM.
- The description of BFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
BFXIL <Wd>, <Wn>, #<lsb>, #<width>
```

is equivalent to

```
BFM <Wd>, <Wn>, #<lsb>, #(<lsb>+<width>-1)
```

and is the preferred disassembly when `UInt(imms) >= UInt(immr)`.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
BFXIL <Xd>, <Xn>, #<lsb>, #<width>
```

is equivalent to

```
BFM <Xd>, <Xn>, #<lsb>, #(<lsb>+<width>-1)
```

and is the preferred disassembly when `UInt(imms) >= UInt(immr)`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<lsb>	For the "32-bit" variant: is the bit number of the lsb of the source bitfield, in the range 0 to 31. For the "64-bit" variant: is the bit number of the lsb of the source bitfield, in the range 0 to 63.
<width>	For the "32-bit" variant: is the width of the bitfield, in the range 1 to 32-<lsb>. For the "64-bit" variant: is the width of the bitfield, in the range 1 to 64-<lsb>.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of BFM gives the operational pseudocode for this instruction.

Operational Information

The description of BFM gives the operational information for this instruction.

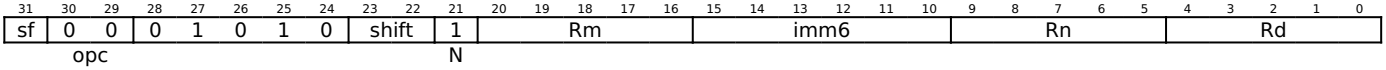
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BIC (shifted register)

Bitwise bit clear (shifted register)

This instruction performs a bitwise AND between a register value and the complement of an optionally-shifted register value, and writes the result to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
BIC <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
BIC <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift to be applied to the final source, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	ROR

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = ShiftReg{datasize}(m, shift_type, shift_amount);
let result : bits(datasize) = operand1 AND NOT operand2;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

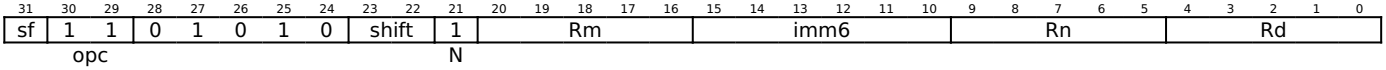
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BICS (shifted register)

Bitwise bit clear (shifted register), setting flags

This instruction performs a bitwise AND between a register value and the complement of an optionally-shifted register value, and writes the result to the destination register. It updates the condition flags based on the result.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
BICS <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
BICS <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift to be applied to the final source, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	ROR

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = ShiftReg{datasize}(m, shift_type, shift_amount);
let result : bits(datasize) = operand1 AND NOT operand2;

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = result[datasize-1]::IsZeroBit{datasize}(result)::'00';
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

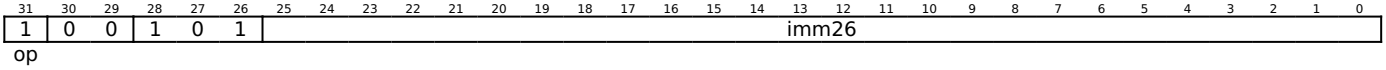
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BL

Branch with link

This instruction branches to a PC-relative offset, setting register X30 to PC+4. This instruction provides a hint that this is a subroutine call.



Encoding

BL <label>

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm26::'00');
let d : integer = 30;
```

Assembler Symbols

<label>	Is the program label to be unconditionally branched to. Its offset from the address of this instruction, in the range +/-128MB, is encoded as "imm26" times 4.

Operation

```
if IsFeatureImplemented(FEAT_GCS) && GCSPCREnabled(PSTATE.EL) then
    AddGCSRecord(PC64() + 4);
end;
X{64}(d) = PC64() + 4;

let branch_conditional : boolean = FALSE;
BranchTo{64}(PC64() + offset, BranchType_DIRCALL, branch_conditional);
```

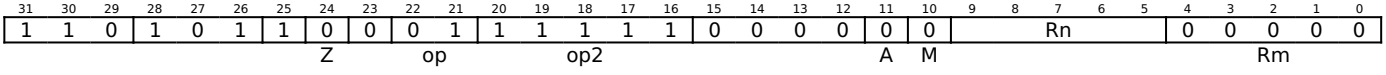
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BLR

Branch with link to register

This instruction calls a subroutine at an address in a register, setting register X30 to PC+4. This instruction provides a hint that this is a subroutine call.



Encoding

```
BLR <Xn>
```

Decode for this encoding

```
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xn>	Is the 64-bit name of the general-purpose register holding the address to be branched to, encoded in the "Rn" field.

Operation

```
let target : bits(64) = X{}(n);
if IsFeatureImplemented(FEAT_GCS) && GCSPCREnabled(PSTATE.EL) then
    AddGCSRecord(PC64() + 4);
end;

// Value in BTypeNext will be used to set PSTATE.BTYPE
BTypeNext = '10';

X{64}(30) = PC64() + 4;

let branch_conditional : boolean = FALSE;
BranchTo{64}(target, BranchType_INDCALL, branch_conditional);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BLRAA, BLRAAZ, BLRAB, BLRABZ

Branch with link to register, with pointer authentication

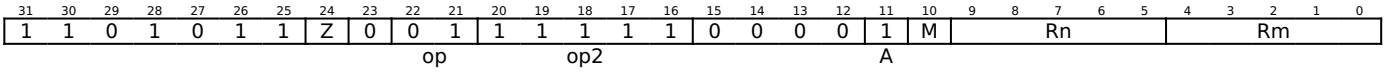
This instruction authenticates the address in the general-purpose register that is specified by <Xn>, using a modifier and the specified key, and calls a subroutine at the authenticated address, setting register X30 to PC+4.
The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xm|SP>, for BLRAA and BLRAB.
- The value zero, for BLRAAZ and BLRABZ.

Key A is used for BLRAA and BLRAAZ. Key B is used for BLRAB and BLRABZ.
If the authentication passes, the PE continues execution at the target of the branch. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.
The authenticated address is not written back to the general-purpose register.
This instruction provides a hint that this is a subroutine call.

Integer

(FEAT_PAuth)



Encoding for the Key A, register modifier variant

Applies when `(Z == 1 && M == 0)`

```
BLRAA <Xn>, <Xm|SP>
```

Encoding for the Key A, zero modifier variant

Applies when `(Z == 0 && M == 0 && Rm == 11111)`

```
BLRAAZ <Xn>
```

Encoding for the Key B, register modifier variant

Applies when `(Z == 1 && M == 1)`

```
BLRAB <Xn>, <Xm|SP>
```

Encoding for the Key B, zero modifier variant

Applies when `(Z == 0 && M == 1 && Rm == 11111)`

```
BLRABZ <Xn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let use_key_a : boolean = (M == '0');
let source_is_sp : boolean = Z == '1' && m == 31;
let auth_then_branch : boolean = TRUE;
```

Assembler Symbols

<Xn>	Is the 64-bit name of the general-purpose register holding the address to be branched to, encoded in the "Rn" field.
<Xm|SP>	Is the 64-bit name of the general-purpose source register or stack pointer holding the modifier, encoded in the "Rm" field.

Operation

```
var target : bits(64) = X{}(n);
let modifier : bits(64) = if source_is_sp then SP{64}() else X{64}(m);

if use_key_a then
    target = AuthIA(target, modifier, auth_then_branch);
else
    target = AuthIB(target, modifier, auth_then_branch);
end;
if IsFeatureImplemented(FEAT_GCS) && GCSPCREnabled(PSTATE.EL) then
    AddGCSRecord(PC64() + 4);
end;

// Value in BTypeNext will be used to set PSTATE.BTYPE
BTypeNext = '10';

X{64}(30) = PC64() + 4;

let branch_conditional : boolean = FALSE;
BranchTo{64}(target, BranchType_INDCALL, branch_conditional);
```

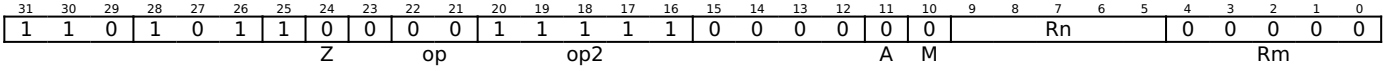
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BR

Branch to register

This instruction branches unconditionally to an address in a register. This instruction provides a hint that this is not a subroutine call or return.



Encoding

```
BR <Xn>
```

Decode for this encoding

```
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xn> Is the 64-bit name of the general-purpose register holding the address to be branched to, encoded in the "Rn" field.

Operation

```
let target : bits(64) = X{}(n);
// Value in BTypeNext will be used to set PSTATE.BTYPE
if InGuardedPage then
    if n == 16 || n == 17 then
        BTypeNext = '01';
    else
        BTypeNext = '11';
    end;
else
    BTypeNext = '01';
end;

let branch_conditional : boolean = FALSE;
BranchTo{64}(target, BranchType_INDIR, branch_conditional);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BRAA, BRAAZ, BRAB, BRABZ

Branch to register, with pointer authentication

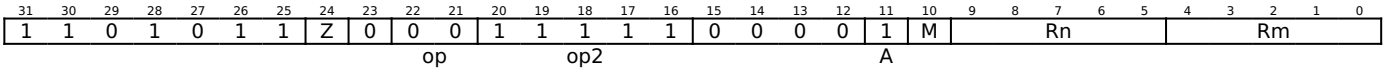
This instruction authenticates the address in the general-purpose register that is specified by <Xn>, using a modifier and the specified key, and branches to the authenticated address.
The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xm|SP>, for BRAA and BRAB.
- The value zero, for BRAAZ and BRABZ.

Key A is used for BRAA and BRAAZ. Key B is used for BRAB and BRABZ.
If the authentication passes, the PE continues execution at the target of the branch. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.
The authenticated address is not written back to the general-purpose register.
This instruction provides a hint that this is not a subroutine call or return.

Integer

(FEAT_PAuth)



Encoding for the Key A, register modifier variant

Applies when `(Z == 1 && M == 0)`

```
BRAA <Xn>, <Xm|SP>
```

Encoding for the Key A, zero modifier variant

Applies when `(Z == 0 && M == 0 && Rm == 11111)`

```
BRAAZ <Xn>
```

Encoding for the Key B, register modifier variant

Applies when `(Z == 1 && M == 1)`

```
BRAB <Xn>, <Xm|SP>
```

Encoding for the Key B, zero modifier variant

Applies when `(Z == 0 && M == 1 && Rm == 11111)`

```
BRABZ <Xn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let use_key_a : boolean = (M == '0');
let source_is_sp : boolean = Z == '1' && m == 31;
let auth_then_branch : boolean = TRUE;
```

Assembler Symbols

<Xn>	Is the 64-bit name of the general-purpose register holding the address to be branched to, encoded in the "Rn" field.
<Xm|SP>	Is the 64-bit name of the general-purpose source register or stack pointer holding the modifier, encoded in the "Rm" field.

Operation

```
var target : bits(64) = X{}(n);
let modifier : bits(64) = if source_is_sp then SP{64}() else X{64}(m);

if use_key_a then
    target = AuthIA(target, modifier, auth_then_branch);
else
    target = AuthIB(target, modifier, auth_then_branch);
end;
// Value in BTypeNext will be used to set PSTATE.BTYPE
if InGuardedPage then
    if n == 16 || n == 17 then
        BTypeNext = '01';
    else
        BTypeNext = '11';
    end;
else
    BTypeNext = '01';
end;

let branch_conditional : boolean = FALSE;
BranchTo{64}(target, BranchType_INDIR, branch_conditional);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BRB

Branch record buffer

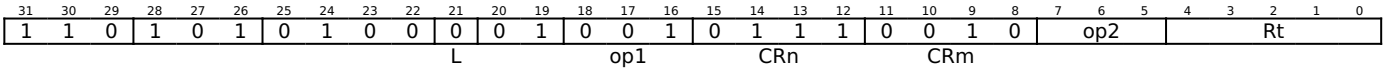
For more information, see *op0 == 0b01, cache maintenance, TLB maintenance, address translation, prediction restriction, BRBE, Trace Extension, Guarded Control Stack, and FEAT_RME_GPC3 instructions*.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_BRBE)



Encoding

```
BRB <brb_op>
```

is equivalent to

```
SYS #1, C7, C2, #<op2>{, <Xt>}
```

and is the preferred disassembly when `SysOp('001', '0111', '0010', op2) == Sys_BRB`.

Assembler Symbols

<brb_op> Is a BRB operation name, as listed for the BRB system instruction group, encoded in "op2":

op2	<brb_op>	Architectural Feature	Description
100	IALL	FEAT_BRBE	For more information, see BRB IALL.
101	INJ	FEAT_BRBE	For more information, see BRB INJ.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

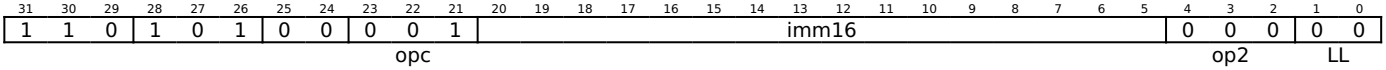
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BRK

Breakpoint instruction

This instruction generates a Breakpoint Instruction exception. The PE records the exception in *ESR_ELx*, using the EC value 0x3C, and captures the value of the immediate argument in *ESR_ELx*.ISS.

Within a guarded memory region, while *PSTATE*.BTYPE != 0b00, a `BRK` instruction will not generate a Branch Target exception and will generate a Breakpoint Instruction exception as normal. For more information, see *PSTATE.BTYPE*.



Encoding

```
BRK #<imm>
```

Decode for this encoding

```
let comment : bits(16) = imm16;
if IsFeatureImplemented(FEAT_BTI) then
    SetBTypeCompatible(TRUE);
end;
```

Assembler Symbols

<imm>	Is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field.

Operation

```
AArch64_SoftwareBreakpoint(comment);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

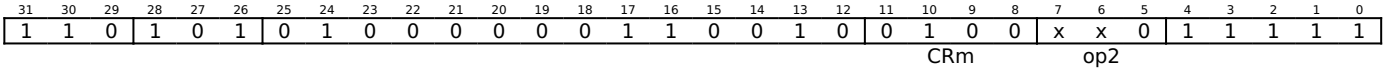
BTI

Branch target identification

This instruction is used to guard against the execution of instructions that are not the intended target of a branch.
Outside of a guarded memory region, a `BTI` instruction executes as a `NOP`. Within a guarded memory region, while *PSTATE*.BTYPE != 0b00, a `BTI` instruction compatible with the current value of PSTATE.BTYPE will not generate a Branch Target Exception and will allow execution of subsequent instructions within the memory region. For more information, see *PSTATE.BTYPE*.
The operand <targets> passed to a `BTI` instruction determines the values of *PSTATE*.BTYPE that the `BTI` instruction is compatible with.

System

(FEAT_BTI)



Encoding

```
BTI {<targets>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_BTI) then EndOfDecode(Decode_NOP); end;

// Check branch target compatibility between BTI instruction and PSTATE.BTYPE
SetBTypeCompatible(BTypeCompatible_BTI(op2[2:1]));
```

Assembler Symbols

<targets> Is the type of indirection, encoded in “op2[2:1]”:

op2[2:1]	<targets>
00	[absent]
01	c
10	j
11	jc

Operation

```
SetBTypeNext('00');
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CAS, CASA, CASAL, CASL

Compare and swap word or doubleword in memory

This instruction reads a 32-bit word or 64-bit doubleword from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not one of WZR or XZR, `CASA` and `CASAL` load from memory with acquire semantics.
- `CASL` and `CASAL` store to memory with release semantics.
- `CAS` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Ws>, or <Xs>, is restored to the value held in the register before the instruction was executed.

For a `CAS` or `CASA` instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, this signals to the memory system that an additional subsequent `CAS`, `CASA`, `CASAL`, or `CASL` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CAS`, `CASA`, `CASAL`, or `CASL` access to succeed when it does occur.

A code sequence starting with a `CAS` or `CASA` instruction for which <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, and ending with a subsequent `CAS`, `CASA`, `CASAL`, or `CASL` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> or <Xs> of the first `CAS` or `CASA` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CAS` or `CASA` instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, the value in memory is not modified, because the `CAS` or `CASA` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CAS` or `CASA` instruction when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> in a routine that includes a subsequent `CAS`, `CASA`, `CASAL`, or `CASL` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CAS`, `CASA`, `CASAL`, or `CASL`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CAS`, `CASA`, `CASAL`, or `CASL`.

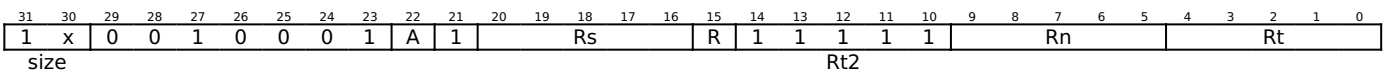
This guidance does not apply when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> for a `CAS` or `CASA` instruction and the compare between the `CAS` or `CASA` and the subsequent `CAS`, `CASA`, `CASAL`, or `CASL` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CAS`, `CASA`, `CASAL`, or `CASL` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)



Encoding for the 32-bit CAS variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
CAS <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASA variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
CASA <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASAL variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
CASAL <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASL variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
CASL <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CAS variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
CAS <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASA variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
CASA <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASAL variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
CASAL <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASL variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
CASL <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;
let acquire : boolean = A == '1' && s != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.

Operation

```
var address : bits(64);
var comparevalue : bits(datasize);
var newvalue : bits(datasize);

let privileged : boolean = PSTATE.EL != EL0;

let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, s);

comparevalue = X{datasize}(s);
newvalue = X{datasize}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(datasize) = MemAtomic{}(address, comparevalue, newvalue, accdesc);
X{regsize}(s) = ZeroExtend{regsize}(data);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CASB, CASAB, CASALB, CASLB

Compare and swap byte in memory

This instruction reads an 8-bit byte from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not WZR, `CASAB` and `CASALB` load from memory with acquire semantics.
- `CASLB` and `CASALB` store to memory with release semantics.
- `CASB` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Ws>, is restored to the values held in the register before the instruction was executed.

For a `CASB` or `CASAB` instruction, when <Ws> specifies the same register as <Wt>, this signals to the memory system that an additional subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` access to succeed when it does occur.

A code sequence starting with a `CASB` or `CASAB` instruction for which <Ws> specifies the same register as <Wt>, and ending with a subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> of the first `CASB` or `CASAB` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CASB` or `CASAB` instruction, when <Ws> specifies the same register as <Wt>, the value in memory is not modified, because the `CASB` or `CASAB` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CASB` or `CASAB` instruction when <Ws> specifies the same register as <Wt> in a routine that includes a subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB`.

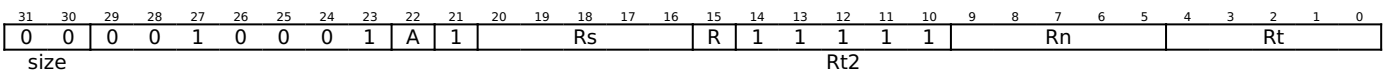
This guidance does not apply when <Ws> specifies the same register as <Wt> for a `CASB` or `CASAB` instruction and the compare between the `CASB` or `CASAB` and the subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CASB`, `CASAB`, `CASALB`, or `CASLB` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)



Encoding for the CASB variant

Applies when `(A == 0 && R == 0)`

```
CASB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASAB variant

Applies when `(A == 1 && R == 0)`

```
CASAB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASALB variant

Applies when `(A == 1 && R == 1)`

```
CASALB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASLB variant

Applies when `(A == 0 && R == 1)`

```
CASLB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquire : boolean = A == '1' && s != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(8);
var newvalue : bits(8);

let privileged : boolean = PSTATE.EL != EL0;

let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, s);

comparevalue = X{8}(s);
newvalue = X{8}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(8) = MemAtomic{}(address, comparevalue, newvalue, accdesc);
X{32}(s) = ZeroExtend{32}(data);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CASH, CASAH, CASALH, CASLH

Compare and swap halfword in memory

This instruction reads a 16-bit halfword from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not WZR, `CASAH` and `CASALH` load from memory with acquire semantics.
- `CASLH` and `CASALH` store to memory with release semantics.
- `CASH` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Ws>, is restored to the values held in the register before the instruction was executed.

For a `CASH` or `CASAH` instruction, when <Ws> specifies the same register as <Wt>, this signals to the memory system that an additional subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` access to succeed when it does occur.

A code sequence starting with a `CASH` or `CASAH` instruction for which <Ws> specifies the same register as <Wt>, and ending with a subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> of the first `CASH` or `CASAH` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CASH` or `CASAH` instruction, when <Ws> specifies the same register as <Wt>, the value in memory is not modified, because the `CASH` or `CASAH` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CASH` or `CASAH` instruction when <Ws> specifies the same register as <Wt> in a routine that includes a subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH`.

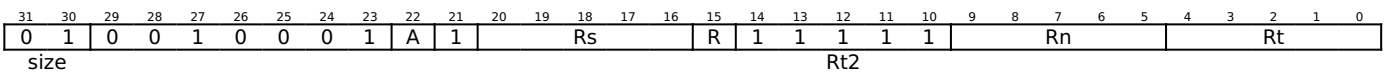
This guidance does not apply when <Ws> specifies the same register as <Wt> for a `CASH` or `CASAH` instruction and the compare between the `CASH` or `CASAH` and the subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CASH`, `CASAH`, `CASALH`, or `CASLH` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)



Encoding for the CASH variant

Applies when `(A == 0 && R == 0)`

```
CASH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASAH variant

Applies when `(A == 1 && R == 0)`

```
CASAH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASALH variant

Applies when `(A == 1 && R == 1)`

```
CASALH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the CASLH variant

Applies when `(A == 0 && R == 1)`

```
CASLH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquire : boolean = A == '1' && s != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(16);
var newvalue : bits(16);

let privileged : boolean = PSTATE.EL != EL0;

let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, s);

comparevalue = X{16}(s);
newvalue = X{16}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(16) = MemAtomic{}(address, comparevalue, newvalue, accdesc);
X{32}(s) = ZeroExtend{32}(data);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CASP, CASPA, CASPAL, CASPL

Compare and swap pair of words or doublewords in memory

This instruction reads a pair of 32-bit words or 64-bit doublewords from memory, and compares them against the values held in the first pair of registers. If the comparison is equal, the values in the second pair of registers are written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the writes are performed, the reads and writes occur atomically such that no other modification of the memory location can take place between the reads and writes.

- `CASPA` and `CASPAL` load from memory with acquire semantics.
- `CASPL` and `CASPAL` store to memory with release semantics.
- `CASP` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the registers which are compared and loaded, that is <Ws> and <W(s+1)>, or <Xs> and <X(s+1)>, are restored to the values held in the registers before the instruction was executed.

For a `CASP` or `CASPA` instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, this signals to the memory system that an additional subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` access to succeed when it does occur.

A code sequence starting with a `CASP` or `CASPA` instruction for which <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, and ending with a subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Ws> or <Xs> of the first `CASP` or `CASPA` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CASP` or `CASPA` instruction, when <Ws> or <Xs> specifies the same register as <Wt> or <Xt>, the value in memory is not modified, because the `CASP` or `CASPA` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CASP` or `CASPA` instruction when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> in a routine that includes a subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL`.

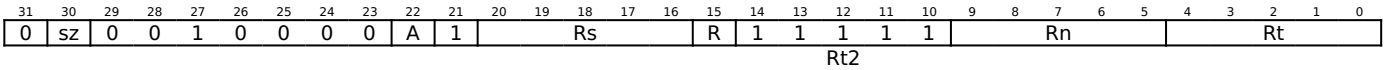
This guidance does not apply when <Ws> or <Xs> specifies the same register as <Wt> or <Xt> for a `CASP` or `CASPA` instruction and the compare between the `CASP` or `CASPA` and the subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CASP`, `CASPA`, `CASPAL`, or `CASPL` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSE)



Encoding for the 32-bit CASP variant

Applies when `(sz == 0 && A == 0 && R == 0)`

```
CASP <Ws>, <W(s+1)>, <Wt>, <W(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASPA variant

Applies when `(sz == 0 && A == 1 && R == 0)`

```
CASPA <Ws>, <W(s+1)>, <Wt>, <W(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASPAL variant

Applies when `(sz == 0 && A == 1 && R == 1)`

```
CASPAL <Ws>, <W(s+1)>, <Wt>, <W(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 32-bit CASPL variant

Applies when `(sz == 0 && A == 0 && R == 1)`

```
CASPL <Ws>, <W(s+1)>, <Wt>, <W(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASP variant

Applies when `(sz == 1 && A == 0 && R == 0)`

```
CASP <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASPA variant

Applies when `(sz == 1 && A == 1 && R == 0)`

```
CASPA <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASPAL variant

Applies when `(sz == 1 && A == 1 && R == 1)`

```
CASPAL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit CASPL variant

Applies when `(sz == 1 && A == 0 && R == 1)`

```
CASPL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
if Rs[0] == '1' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sz);
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Ws> must be an even-numbered register.
<W(s+1)>	Is the 32-bit name of the second general-purpose register to be compared and loaded.
<Wt>	Is the 32-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Wt> must be an even-numbered register.
<W(t+1)>	Is the 32-bit name of the second general-purpose register to be conditionally stored.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Xs> must be an even-numbered register.
<X(s+1)>	Is the 64-bit name of the second general-purpose register to be compared and loaded.
<Xt>	Is the 64-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Xt> must be an even-numbered register.
<X(t+1)>	Is the 64-bit name of the second general-purpose register to be conditionally stored.

Operation

```
var address : bits(64);
var comparevalue : bits(2*datasize);
var newvalue : bits(2*datasize);

let s1 : bits(datasize) = X{}(s);
let s2 : bits(datasize) = X{}(s+1);
let t1 : bits(datasize) = X{}(t);
let t2 : bits(datasize) = X{}(t+1);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, t+1, s, s+1);

comparevalue = if BigEndian(accdesc.acctype) then s1::s2 else s2::s1;
newvalue     = if BigEndian(accdesc.acctype) then t1::t2 else t2::t1;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(2*datasize) = MemAtomic{}(address, comparevalue, newvalue, accdesc);

if BigEndian(accdesc.acctype) then
    X{datasize}(s)   = data[2*datasize-1:datasize];
    X{datasize}(s+1) = data[datasize-1:0];
else
    X{datasize}(s)   = data[datasize-1:0];
    X{datasize}(s+1) = data[2*datasize-1:datasize];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CASPT, CASPAT, CASPALT, CASPLT

Compare and swap pair unprivileged

This instruction reads a pair of 64-bit doublewords from memory, and compares them against the values held in the first pair of registers. If the comparison is equal, the values in the second pair of registers are written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the writes are performed, the reads and writes occur atomically such that no other modification of the memory location can take place between the reads and writes.

- `CASPAT` and `CASPALT` load from memory with acquire semantics.
- `CASPLT` and `CASPALT` store to memory with release semantics.
- `CASPT` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the registers which are compared and loaded, that is <Xs> and <X(s+1)>, are restored to the values held in the registers before the instruction was executed.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For a `CASPT` or `CASPAT` instruction, when <Xs> specifies the same register as <Xt>, this signals to the memory system that an additional subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` access to succeed when it does occur.

A code sequence starting with a `CASPT` or `CASPAT` instruction for which <Xs> specifies the same register as <Xt>, and ending with a subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Xs> of the first `CASPT` or `CASPAT` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CASPT` or `CASPAT` instruction, when <Xs> specifies the same register as <Xt>, the value in memory is not modified, because the `CASPT` or `CASPAT` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CASPT` or `CASPAT` instruction when <Xs> specifies the same register as <Xt> in a routine that includes a subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT`.

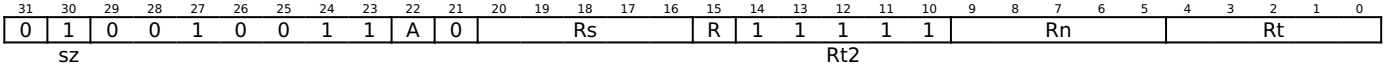
This guidance does not apply when <Xs> specifies the same register as <Xt> for a `CASPT` or `CASPAT` instruction and the compare between the `CASPT` or `CASPAT` and the subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CASPT`, `CASPAT`, `CASPALT`, or `CASPLT` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)



Encoding for the CASPT variant

Applies when `(A == 0 && R == 0)`

```
CASPT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the CASPAT variant

Applies when `(A == 1 && R == 0)`

```
CASPAT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the CASPALT variant

Applies when `(A == 1 && R == 1)`

```
CASPALT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Encoding for the CASPLT variant

Applies when `(A == 0 && R == 1)`

```
CASPLT <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
if Rs[0] == '1' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 64;
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Xs> must be an even-numbered register.
<X(s+1)>	Is the 64-bit name of the second general-purpose register to be compared and loaded.
<Xt>	Is the 64-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Xt> must be an even-numbered register.
<X(t+1)>	Is the 64-bit name of the second general-purpose register to be conditionally stored.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(2*datasize);
var newvalue : bits(2*datasize);

let s1 : bits(datasize) = X{}(s);
let s2 : bits(datasize) = X{}(s+1);
let t1 : bits(datasize) = X{}(t);
let t2 : bits(datasize) = X{}(t+1);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, t+1, s, s+1);

comparevalue = if BigEndian(accdesc.acctype) then s1::s2 else s2::s1;
newvalue     = if BigEndian(accdesc.acctype) then t1::t2 else t2::t1;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(2*datasize) = MemAtomic{}(address, comparevalue, newvalue, accdesc);

if BigEndian(accdesc.acctype) then
    X{datasize}(s)   = data[2*datasize-1:datasize];
    X{datasize}(s+1) = data[datasize-1:0];
else
    X{datasize}(s)   = data[datasize-1:0];
    X{datasize}(s+1) = data[2*datasize-1:datasize];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CAST, CASAT, CASALT, CASLT

Compare and swap unprivileged

This instruction reads a 64-bit doubleword from memory, and compares it against the value held in a first register. If the comparison is equal, the value in a second register is written to memory. If the comparison is not equal, the architecture permits writing the value read from the location to memory. If the write is performed, the read and write occur atomically such that no other modification of the memory location can take place between the read and write.

- If the destination register is not XZR, `CASAT` and `CASALT` load from memory with acquire semantics.
- `CASLT` and `CASALT` store to memory with release semantics.
- `CAST` has neither acquire nor release semantics.

The architecture permits that the data read clears any exclusive monitors associated with that location, even if the compare subsequently fails.

If the instruction generates a synchronous Data Abort, the register which is compared and loaded, that is <Xs>, is restored to the value held in the register before the instruction was executed.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For a `CAST` or `CASAT` instruction, when <Xs> specifies the same register as <Xt>, this signals to the memory system that an additional subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` access to the specified location is expected to occur in the near future in the absence of taking an exception or exceptional software conditions. The memory system can respond by taking actions that are expected to enable the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` access to succeed when it does occur.

A code sequence starting with a `CAST` or `CASAT` instruction for which <Xs> specifies the same register as <Xt>, and ending with a subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` to the same location, exhibits the following properties for best performance when the location may be accessed concurrently, on one or more other PEs:

- The sequence does not contain any direct System register writes, address translation instructions, cache or TLB maintenance operations, exception producing instructions, exception returns, or `ISB` barriers.
- The execution of the sequence includes 32 or fewer instructions.
- The value provided in <Xs> of the first `CAST` or `CASAT` is a value likely to result in the comparison failing. A failing comparison result may lead to better performance due to the hardware not performing a write to memory.

Note

For a `CAST` or `CASAT` instruction, when <Xs> specifies the same register as <Xt>, the value in memory is not modified, because the `CAST` or `CASAT` either fails its compare or writes the same value back to memory.

For performance reasons, Arm recommends that software not use a `CAST` or `CASAT` instruction when <Xs> specifies the same register as <Xt> in a routine that includes a subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` for any of the following purposes:

- To load the value that will be compared to conditionally determine whether or not to execute the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT`.
- As a form of prefetch executed between the instruction loading the value that will be compared and the subsequent compare instruction that conditionally determines whether or not to execute the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT`.

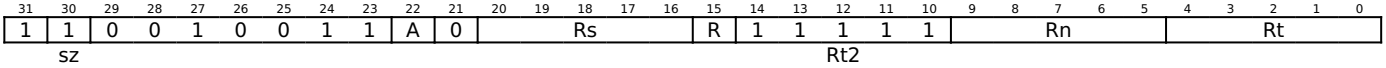
This guidance does not apply when <Xs> specifies the same register as <Xt> for a `CAST` or `CASAT` instruction and the compare between the `CAST` or `CASAT` and the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` is intended to detect an exceptional software condition such that the result of the compare does not prevent the subsequent `CAST`, `CASAT`, `CASALT`, or `CASLT` from executing under normal operating conditions.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)



Encoding for the CAST variant

Applies when `(A == 0 && R == 0)`

```
CAST <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the CASAT variant

Applies when `(A == 1 && R == 0)`

```
CASAT <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the CASALT variant

Applies when `(A == 1 && R == 1)`

```
CASALT <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Encoding for the CASLT variant

Applies when `(A == 0 && R == 1)`

```
CASLT <Xs>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquire : boolean = A == '1' && s != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var comparevalue : bits(64);
var newvalue : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();

let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_CAS, acquire, release,
                                                       tagchecked, privileged, t, s);

comparevalue = X{64}(s);
newvalue = X{64}(t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

X{64}(s) = MemAtomic{64}(address, comparevalue, newvalue, accdesc);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CB<cc> (immediate)

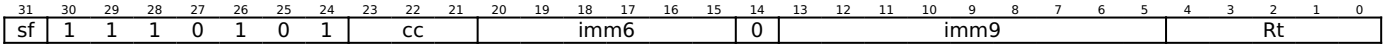
Compare register with immediate and branch

This instruction compares the value in a register with an immediate, and conditionally branches to a label at a PC-relative offset if the comparison is true. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This instruction is used by the pseudo-instructions CBGE (immediate), CBHS (immediate), CBLE (immediate), and CBLS (immediate).

Branch

(FEAT_CMPBR)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0 && cc == 000)`

```
CBGT <Wt>, #<imm>, <label>
```

Encoding for the 32-bit less than variant

Applies when `(sf == 0 && cc == 001)`

```
CBLT <Wt>, #<imm>, <label>
```

Encoding for the 32-bit higher variant

Applies when `(sf == 0 && cc == 010)`

```
CBHI <Wt>, #<imm>, <label>
```

Encoding for the 32-bit lower variant

Applies when `(sf == 0 && cc == 011)`

```
CBLO <Wt>, #<imm>, <label>
```

Encoding for the 32-bit equal variant

Applies when `(sf == 0 && cc == 110)`

```
CBEQ <Wt>, #<imm>, <label>
```

Encoding for the 32-bit not equal variant

Applies when `(sf == 0 && cc == 111)`

```
CBNE <Wt>, #<imm>, <label>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1 && cc == 000)`

```
CBGT <Xt>, #<imm>, <label>
```

Encoding for the 64-bit less than variant

Applies when `(sf == 1 && cc == 001)`

```
CBLT <Xt>, #<imm>, <label>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1 && cc == 010)`

```
CBHI <Xt>, #<imm>, <label>
```

Encoding for the 64-bit lower variant

Applies when `(sf == 1 && cc == 011)`

```
CBLO <Xt>, #<imm>, <label>
```

Encoding for the 64-bit equal variant

Applies when `(sf == 1 && cc == 110)`

```
CBEQ <Xt>, #<imm>, <label>
```

Encoding for the 64-bit not equal variant

Applies when `(sf == 1 && cc == 111)`

```
CBNE <Xt>, #<imm>, <label>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CMPBR) then EndOfDecode(Decode_UNDEF); end;
let datasize : integer{} = 32 << UInt(sf);
let t : integer = UInt(Rt);
let offset : bits(64) = SignExtend{}(imm9::'00');
var op : CmpOp;
var unsigned : boolean;

case cc of
    when '000' => op = Cmp_GT; unsigned = FALSE;
    when '001' => op = Cmp_LT; unsigned = FALSE;
    when '010' => op = Cmp_GT; unsigned = TRUE;
    when '011' => op = Cmp_LT; unsigned = TRUE;
    when '110' => op = Cmp_EQ; unsigned = TRUE;
    when '111' => op = Cmp_NE; unsigned = TRUE;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
let value2 : integer = UInt(imm6);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<imm>	Is an unsigned immediate, in the range 0 to 63, encoded in the "imm6" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(t);
let branch_conditional : boolean = TRUE;

let value1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
var cond : boolean;
case op of
    when Cmp_EQ => cond = value1 == value2;
    when Cmp_NE => cond = value1 != value2;
    when Cmp_LT => cond = value1 <  value2;
    when Cmp_GT => cond = value1 >  value2;
end;

if cond then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CB<cc> (register)

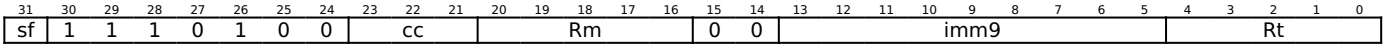
Compare registers and branch

This instruction compares the values in two registers, and conditionally branches to a label at a PC-relative offset if the condition is true. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This instruction is used by the pseudo-instructions CBLE (register), CBLO (register), CBLS (register), and CBLT (register).

Branch

(FEAT_CMPBR)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0 && cc == 000)`

```
CBGT <Wt>, <Wm>, <label>
```

Encoding for the 32-bit greater than or equal variant

Applies when `(sf == 0 && cc == 001)`

```
CBGE <Wt>, <Wm>, <label>
```

Encoding for the 32-bit higher variant

Applies when `(sf == 0 && cc == 010)`

```
CBHI <Wt>, <Wm>, <label>
```

Encoding for the 32-bit higher or same variant

Applies when `(sf == 0 && cc == 011)`

```
CBHS <Wt>, <Wm>, <label>
```

Encoding for the 32-bit equal variant

Applies when `(sf == 0 && cc == 110)`

```
CBEQ <Wt>, <Wm>, <label>
```

Encoding for the 32-bit not equal variant

Applies when `(sf == 0 && cc == 111)`

```
CBNE <Wt>, <Wm>, <label>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1 && cc == 000)`

```
CBGT <Xt>, <Xm>, <label>
```

Encoding for the 64-bit greater than or equal variant

Applies when `(sf == 1 && cc == 001)`

```
CBGE <Xt>, <Xm>, <label>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1 && cc == 010)`

```
CBHI <Xt>, <Xm>, <label>
```

Encoding for the 64-bit higher or same variant

Applies when `(sf == 1 && cc == 011)`

```
CBHS <Xt>, <Xm>, <label>
```

Encoding for the 64-bit equal variant

Applies when `(sf == 1 && cc == 110)`

```
CBEQ <Xt>, <Xm>, <label>
```

Encoding for the 64-bit not equal variant

Applies when `(sf == 1 && cc == 111)`

```
CBNE <Xt>, <Xm>, <label>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CMPBR) then EndOfDecode(Decode_UNDEF); end;
let datasize : integer{} = 32 << UInt(sf);
let t : integer = UInt(Rt);
let m : integer = UInt(Rm);
let offset : bits(64) = SignExtend{}(imm9::'00');
var op : CmpOp;
var unsigned : boolean;

case cc of
    when '000' => op = Cmp_GT; unsigned = FALSE;
    when '001' => op = Cmp_GE; unsigned = FALSE;
    when '010' => op = Cmp_GT; unsigned = TRUE;
    when '011' => op = Cmp_GE; unsigned = TRUE;
    when '110' => op = Cmp_EQ; unsigned = TRUE;
    when '111' => op = Cmp_NE; unsigned = TRUE;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(t);
let operand2 : bits(datasize) = X{datasize}(m);
let branch_conditional : boolean = TRUE;

let value1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
let value2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
var cond : boolean;
case op of
    when Cmp_EQ => cond = value1 == value2;
    when Cmp_NE => cond = value1 != value2;
    when Cmp_GE => cond = value1 >= value2;
    when Cmp_GT => cond = value1 >  value2;
end;

if cond then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBB<cc>

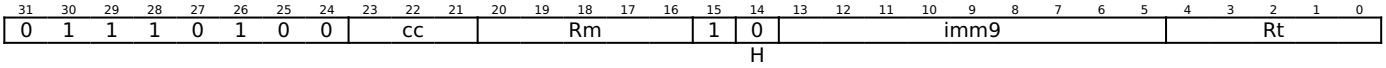
Compare bytes and branch

This instruction compares the byte values in two registers, and conditionally branches to a label at a PC-relative offset if the condition is true. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This instruction is used by the pseudo-instructions CBBLE, CBBLO, CBBLS, and CBBLT.

Branch

(FEAT_CMPBR)



Encoding for the Greater than variant

Applies when (cc == 000)

```
CBBGT <Wt>, <Wm>, <label>
```

Encoding for the Greater than or equal variant

Applies when (cc == 001)

```
CBBGE <Wt>, <Wm>, <label>
```

Encoding for the Higher variant

Applies when (cc == 010)

```
CBBHI <Wt>, <Wm>, <label>
```

Encoding for the Higher or same variant

Applies when (cc == 011)

```
CBBHS <Wt>, <Wm>, <label>
```

Encoding for the Equal variant

Applies when (cc == 110)

```
CBBEQ <Wt>, <Wm>, <label>
```

Encoding for the Not equal variant

Applies when (cc == 111)

```
CBBNE <Wt>, <Wm>, <label>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CMPBR) then EndOfDecode(Decode_UNDEF); end;
let datasize : integer{} = 8 << UInt(H);
let t : integer = UInt(Rt);
let m : integer = UInt(Rm);
let offset : bits(64) = SignExtend{}(imm9::'00');
var op : CmpOp;
var unsigned : boolean;

case cc of
    when '000' => op = Cmp_GT; unsigned = FALSE;
    when '001' => op = Cmp_GE; unsigned = FALSE;
    when '010' => op = Cmp_GT; unsigned = TRUE;
    when '011' => op = Cmp_GE; unsigned = TRUE;
    when '110' => op = Cmp_EQ; unsigned = TRUE;
    when '111' => op = Cmp_NE; unsigned = TRUE;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.

Operation

```
let operand1 : bits(datasize) = X{datasize}(t);
let operand2 : bits(datasize) = X{datasize}(m);
let branch_conditional : boolean = TRUE;

let value1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
let value2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
var cond : boolean;
case op of
    when Cmp_EQ => cond = value1 == value2;
    when Cmp_NE => cond = value1 != value2;
    when Cmp_GE => cond = value1 >= value2;
    when Cmp_GT => cond = value1 >  value2;
end;

if cond then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBBLE

Compare signed less than or equal to bytes and branch

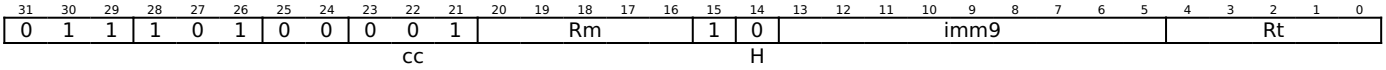
This instruction compares the signed byte values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is less than or equal to the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CBB<cc>. This means:

- The encodings in this description are named to match the encodings of CBB<cc>.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CBB<cc> gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding

```
CBBLE <Wm>, <Wt>, <label>
```

is equivalent to

```
CBBGE <Wt>, <Wm>, <label>
```

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.

Operation

The description of CBB<cc> gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBBLO

Compare unsigned lower than bytes and branch

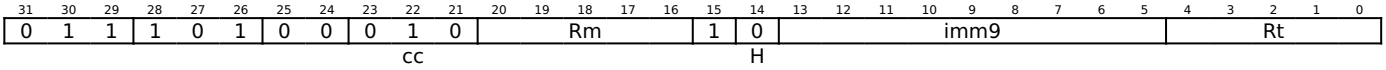
This instruction compares the unsigned byte values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is lower than the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CBB<cc>. This means:

- The encodings in this description are named to match the encodings of CBB<cc>.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CBB<cc> gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding

```
CBBLO <Wm>, <Wt>, <label>
```

is equivalent to

```
CBBHI <Wt>, <Wm>, <label>
```

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.

Operation

The description of CBB<cc> gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBBLS

Compare unsigned lower than or same as bytes and branch

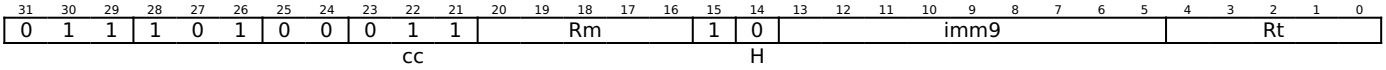
This instruction compares the unsigned byte values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is lower than or the same as the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CBB<cc>. This means:

- The encodings in this description are named to match the encodings of CBB<cc>.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CBB<cc> gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding

```
CBBLS <Wm>, <Wt>, <label>
```

is equivalent to

```
CBBHS <Wt>, <Wm>, <label>
```

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.

Operation

The description of CBB<cc> gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBBLT

Compare signed less than bytes and branch

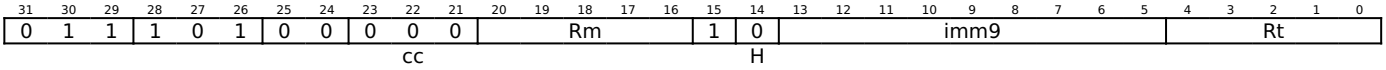
This instruction compares the signed byte values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is less than the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CBB<cc>. This means:

- The encodings in this description are named to match the encodings of CBB<cc>.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CBB<cc> gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding

```
CBBLT <Wm>, <Wt>, <label>
```

is equivalent to

```
CBBGT <Wt>, <Wm>, <label>
```

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.

Operation

The description of CBB<cc> gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBGE (immediate)

Compare signed greater than or equal to immediate and branch

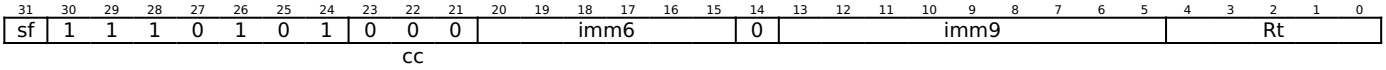
This instruction compares the signed value in a register with an immediate, and conditionally branches to a label at a PC-relative offset if the register value is greater than or equal to the immediate. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CB<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CB<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CB<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0)`

```
CBGE <Wt>, #<immp1>, <label>
```

is equivalent to

```
CBGT <Wt>, #<imm>, <label>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1)`

```
CBGE <Xt>, #<immp1>, <label>
```

is equivalent to

```
CBGT <Xt>, #<imm>, <label>
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<immp1>	Is an unsigned immediate, in the range 1 to 64, encoded as "imm6" plus 1.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CB<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBH<cc>

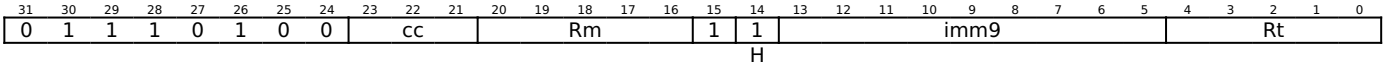
Compare halfwords and branch

This instruction compares the halfword values in two registers, and conditionally branches to a label at a PC-relative offset if the condition is true. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This instruction is used by the pseudo-instructions CBHLE, CBHLO, CBHLS, and CBHLT.

Branch

(FEAT_CMPBR)



Encoding for the Greater than variant

Applies when (cc == 000)

```
CBHGT <Wt>, <Wm>, <label>
```

Encoding for the Greater than or equal variant

Applies when (cc == 001)

```
CBHGE <Wt>, <Wm>, <label>
```

Encoding for the Higher variant

Applies when (cc == 010)

```
CBHHI <Wt>, <Wm>, <label>
```

Encoding for the Higher or same variant

Applies when (cc == 011)

```
CBHHS <Wt>, <Wm>, <label>
```

Encoding for the Equal variant

Applies when (cc == 110)

```
CBHEQ <Wt>, <Wm>, <label>
```

Encoding for the Not equal variant

Applies when (cc == 111)

```
CBHNE <Wt>, <Wm>, <label>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CMPBR) then EndOfDecode(Decode_UNDEF); end;
let datasize : integer{} = 8 << UInt(H);
let t : integer = UInt(Rt);
let m : integer = UInt(Rm);
let offset : bits(64) = SignExtend{}(imm9::'00');
var op : CmpOp;
var unsigned : boolean;

case cc of
    when '000' => op = Cmp_GT; unsigned = FALSE;
    when '001' => op = Cmp_GE; unsigned = FALSE;
    when '010' => op = Cmp_GT; unsigned = TRUE;
    when '011' => op = Cmp_GE; unsigned = TRUE;
    when '110' => op = Cmp_EQ; unsigned = TRUE;
    when '111' => op = Cmp_NE; unsigned = TRUE;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.

Operation

```
let operand1 : bits(datasize) = X{datasize}(t);
let operand2 : bits(datasize) = X{datasize}(m);
let branch_conditional : boolean = TRUE;

let value1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
let value2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
var cond : boolean;
case op of
    when Cmp_EQ => cond = value1 == value2;
    when Cmp_NE => cond = value1 != value2;
    when Cmp_GE => cond = value1 >= value2;
    when Cmp_GT => cond = value1 >  value2;
end;

if cond then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBHLE

Compare signed less than or equal to halfwords and branch

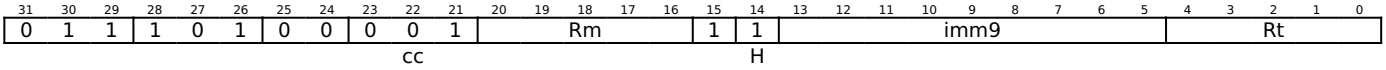
This instruction compares the signed halfword values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is less than or equal to the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CBH<cc>. This means:

- The encodings in this description are named to match the encodings of CBH<cc>.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CBH<cc> gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding

```
CBHLE <Wm>, <Wt>, <label>
```

is equivalent to

```
CBHGE <Wt>, <Wm>, <label>
```

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.

Operation

The description of CBH<cc> gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBHLO

Compare unsigned lower than halfwords and branch

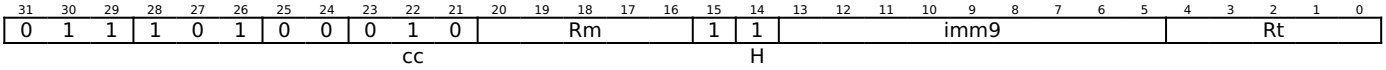
This instruction compares the unsigned halfword values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is lower than the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CBH<cc>. This means:

- The encodings in this description are named to match the encodings of CBH<cc>.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CBH<cc> gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding

CBHLO <Wm>, <Wt>, <label>

is equivalent to

CBHHI <Wt>, <Wm>, <label>

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.

Operation

The description of CBH<cc> gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBHLS

Compare unsigned lower than or same as halfwords and branch

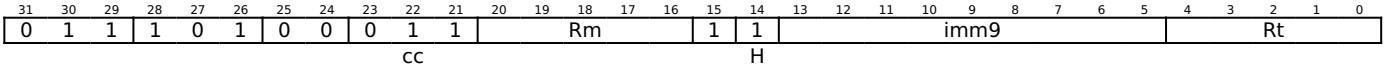
This instruction compares the unsigned halfword values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is lower than or the same as the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CBH<cc>. This means:

- The encodings in this description are named to match the encodings of CBH<cc>.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CBH<cc> gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding

CBHLS <Wm>, <Wt>, <label>

is equivalent to

CBHHS <Wt>, <Wm>, <label>

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.

Operation

The description of CBH<cc> gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBHLT

Compare signed less than halfwords and branch

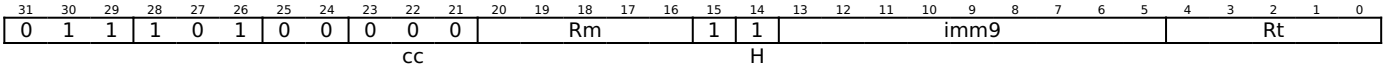
This instruction compares the signed halfword values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is less than the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CBH<cc>. This means:

- The encodings in this description are named to match the encodings of CBH<cc>.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CBH<cc> gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding

```
CBHLT <Wm>, <Wt>, <label>
```

is equivalent to

```
CBHGT <Wt>, <Wm>, <label>
```

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.

Operation

The description of CBH<cc> gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBHS (immediate)

Compare unsigned higher than or same as immediate and branch

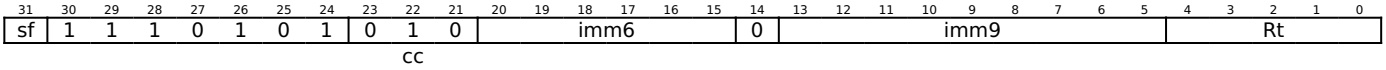
This instruction compares the unsigned value in a register with an immediate, and conditionally branches to a label at a PC-relative offset if the register value is higher than or the same as the immediate. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CB<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CB<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CB<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding for the 32-bit higher variant

Applies when `(sf == 0)`

```
CBHS <Wt>, #<immp1>, <label>
```

is equivalent to

```
CBHI <Wt>, #<imm>, <label>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1)`

```
CBHS <Xt>, #<immp1>, <label>
```

is equivalent to

```
CBHI <Xt>, #<imm>, <label>
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<immp1>	Is an unsigned immediate, in the range 1 to 64, encoded as "imm6" plus 1.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CB<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBLE (immediate)

Compare signed less than or equal to immediate and branch

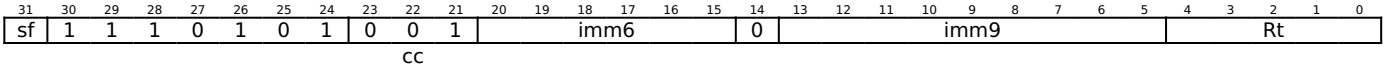
This instruction compares the signed value in a register with an immediate, and conditionally branches to a label at a PC-relative offset if the register value is less than or equal to the immediate. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CB<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CB<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CB<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding for the 32-bit less than variant

Applies when `(sf == 0)`

```
CBLE <Wt>, #<imms1>, <label>
```

is equivalent to

```
CBLT <Wt>, #<imm>, <label>
```

Encoding for the 64-bit less than variant

Applies when `(sf == 1)`

```
CBLE <Xt>, #<imms1>, <label>
```

is equivalent to

```
CBLT <Xt>, #<imm>, <label>
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<imms1>	Is a signed immediate, in the range -1 to 62, encoded as "imm6" minus 1.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CB<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBLE (register)

Compare signed less than or equal to register and branch

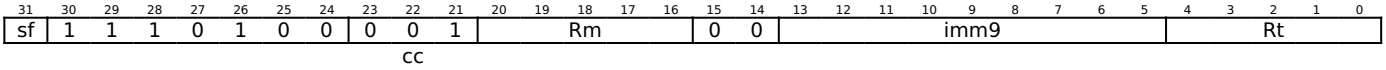
This instruction compares the signed values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is less than or equal to the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CB<cc> (register). This means:

- The encodings in this description are named to match the encodings of CB<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CB<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding for the 32-bit greater than or equal variant

Applies when `(sf == 0)`

```
CBLE <Wm>, <Wt>, <label>
```

is equivalent to

```
CBGE <Wt>, <Wm>, <label>
```

Encoding for the 64-bit greater than or equal variant

Applies when `(sf == 1)`

```
CBLE <Xm>, <Xt>, <label>
```

is equivalent to

```
CBGE <Xt>, <Xm>, <label>
```

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CB<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBLO (register)

Compare unsigned lower than register and branch

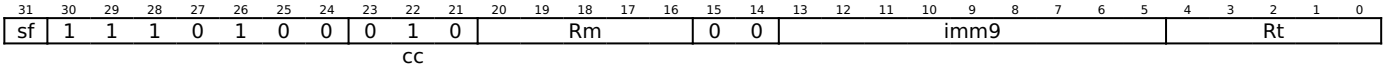
This instruction compares the unsigned values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is lower than the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CB<cc> (register). This means:

- The encodings in this description are named to match the encodings of CB<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CB<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding for the 32-bit higher variant

Applies when `(sf == 0)`

```
CBLO <Wm>, <Wt>, <label>
```

is equivalent to

```
CBHI <Wt>, <Wm>, <label>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1)`

```
CBLO <Xm>, <Xt>, <label>
```

is equivalent to

```
CBHI <Xt>, <Xm>, <label>
```

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CB<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBLS (immediate)

Compare unsigned lower than or same as immediate and branch

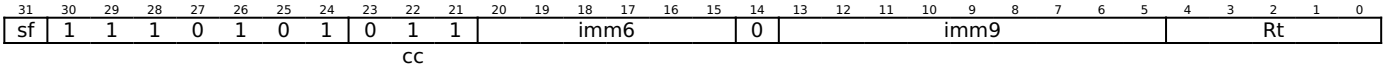
This instruction compares the unsigned value in a register with an immediate, and conditionally branches to a label at a PC-relative offset if the register value is lower than or the same as the immediate. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CB<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CB<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CB<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding for the 32-bit lower variant

Applies when `(sf == 0)`

```
CBLS <Wt>, #<imms1>, <label>
```

is equivalent to

```
CBLO <Wt>, #<imm>, <label>
```

Encoding for the 64-bit lower variant

Applies when `(sf == 1)`

```
CBLS <Xt>, #<imms1>, <label>
```

is equivalent to

```
CBLO <Xt>, #<imm>, <label>
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<imms1>	Is a signed immediate, in the range -1 to 62, encoded as "imm6" minus 1.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CB<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBLS (register)

Compare unsigned lower than or same as register and branch

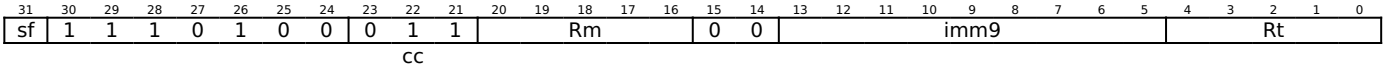
This instruction compares the unsigned values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is lower than or the same as the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CB<cc> (register). This means:

- The encodings in this description are named to match the encodings of CB<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CB<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding for the 32-bit higher or same variant

Applies when `(sf == 0)`

```
CBLS <Wm>, <Wt>, <label>
```

is equivalent to

```
CBHS <Wt>, <Wm>, <label>
```

Encoding for the 64-bit higher or same variant

Applies when `(sf == 1)`

```
CBLS <Xm>, <Xt>, <label>
```

is equivalent to

```
CBHS <Xt>, <Xm>, <label>
```

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CB<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBLT (register)

Compare signed less than register and branch

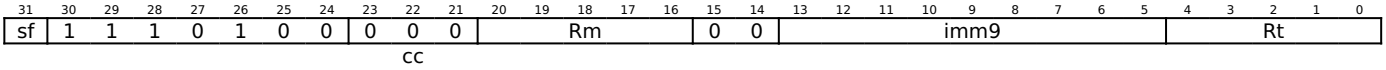
This instruction compares the signed values in two registers, and conditionally branches to a label at a PC-relative offset if the second value is less than the first. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.

This is a pseudo-instruction of CB<cc> (register). This means:

- The encodings in this description are named to match the encodings of CB<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CB<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Branch

(FEAT_CMPBR)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0)`

```
CBLT <Wm>, <Wt>, <label>
```

is equivalent to

```
CBGT <Wt>, <Wm>, <label>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1)`

```
CBLT <Xm>, <Xt>, <label>
```

is equivalent to

```
CBGT <Xt>, <Xm>, <label>
```

Assembler Symbols

<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range -1024 to 1020, is encoded as "imm9" times 4.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CB<cc> (register) gives the operational pseudocode for this instruction.

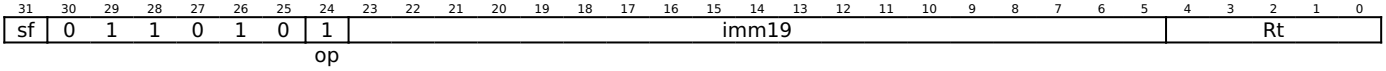
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBNZ

Compare and branch on nonzero

This instruction compares the value in a register with zero, and conditionally branches to a label at a PC-relative offset if the comparison is not equal. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect the condition flags.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CBNZ <Wt>, <label>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CBNZ <Xt>, <label>
```

Decode for all variants of this encoding

```
let t : integer = UInt(Rt);
let datasize : integer{} = 32 << UInt(sf);
let offset : bits(64) = SignExtend{}(imm19::'00');
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range +/-1MB, is encoded as "imm19" times 4.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

```
let branch_conditional : boolean = TRUE;
let operand1 : bits(datasize) = X{datasize}(t);
if !IsZero(operand1) then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

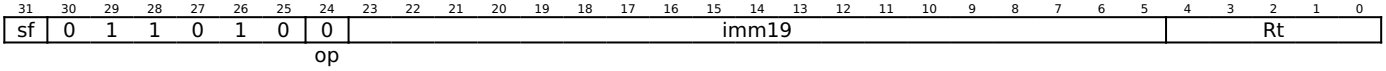
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CBZ

Compare and branch on zero

This instruction compares the value in a register with zero, and conditionally branches to a label at a PC-relative offset if the comparison is equal. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect condition flags.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CBZ <Wt>, <label>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CBZ <Xt>, <label>
```

Decode for all variants of this encoding

```
let t : integer = UInt(Rt);
let datasize : integer{} = 32 << UInt(sf);
let offset : bits(64) = SignExtend{}(imm19::'00');
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<label>	Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range +/-1MB, is encoded as "imm19" times 4.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

```
let branch_conditional : boolean = TRUE;
let operand1 : bits(datasize) = X{datasize}(t);
if IsZero(operand1) then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

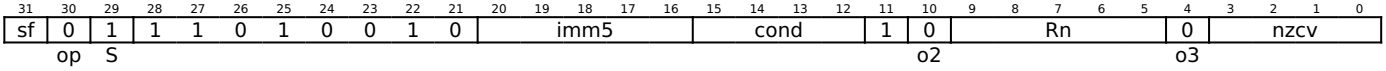
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CCMN (immediate)

Conditional compare negative (immediate)

This instruction sets the value of the condition flags to the result of the comparison of a register value and a negated immediate value if the condition is TRUE, and an immediate value otherwise.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CCMN <Wn>, #<imm>, #<nzcv>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CCMN <Xn>, #<imm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
let imm : bits(datasize) = ZeroExtend{}(imm5);
```

Assembler Symbols

<Wn> Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.

<imm> Is a five bit unsigned (positive) immediate encoded in the "imm5" field.

<nzcv> Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xn> Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
if ConditionHolds(condition) then
    let operand1 : bits(datasize) = X{datasize}(n);
    let operand2 : bits(datasize) = imm;

    (-, flags) = AddWithCarry{datasize}(operand1, operand2, '0');
end;
PSTATE.[N,Z,C,V] = flags;
```

Operational information

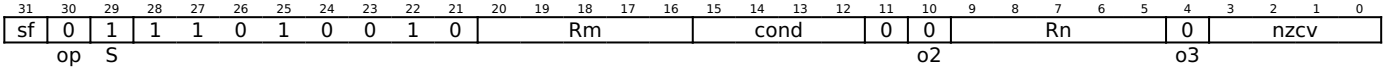
This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CCMN (register)

Conditional compare negative (register)

This instruction sets the value of the condition flags to the result of the comparison of a register value and the inverse of another register value if the condition is TRUE, and an immediate value otherwise.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CCMN <Wn>, <Wm>, #<nzcv>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CCMN <Xn>, <Xm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
```

Assembler Symbols

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<nzcv> Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
if ConditionHolds(condition) then
    let operand1 : bits(datasize) = X{datasize}(n);
    let operand2 : bits(datasize) = X{datasize}(m);

    (-, flags) = AddWithCarry{datasize}(operand1, operand2, '0');
end;
PSTATE.[N,Z,C,V] = flags;
```

Operational information

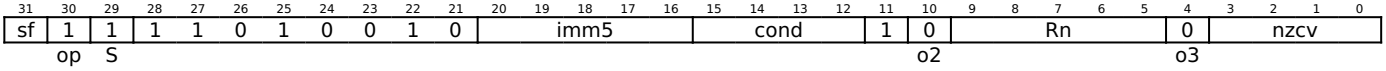
This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CCMP (immediate)

Conditional compare (immediate)

This instruction sets the value of the condition flags to the result of the comparison of a register value and an immediate value if the condition is TRUE, and an immediate value otherwise.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CCMP <Wn>, #<imm>, #<nzcv>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CCMP <Xn>, #<imm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
let imm : bits(datasize) = ZeroExtend{}(imm5);
```

Assembler Symbols

<Wn> Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.

<imm> Is a five bit unsigned (positive) immediate encoded in the "imm5" field.

<nzcv> Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xn> Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
if ConditionHolds(condition) then
    let operand1 : bits(datasize) = X{datasize}(n);
    let operand2 : bits(datasize) = imm;

    (-, flags) = AddWithCarry{datasize}(operand1, NOT operand2, '1');
end;
PSTATE.[N,Z,C,V] = flags;
```

Operational information

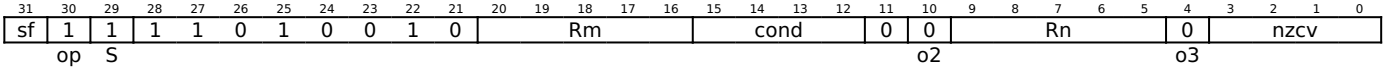
This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CCMP (register)

Conditional compare (register)

This instruction sets the value of the condition flags to the result of the comparison of two registers if the condition is TRUE, and an immediate value otherwise.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CCMP <Wn>, <Wm>, #<nzcv>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CCMP <Xn>, <Xm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
```

Assembler Symbols

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<nzcv> Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
if ConditionHolds(condition) then
    let operand1 : bits(datasize) = X{datasize}(n);
    let operand2 : bits(datasize) = X{datasize}(m);

    (-, flags) = AddWithCarry{datasize}(operand1, NOT operand2, '1');
end;
PSTATE.[N,Z,C,V] = flags;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

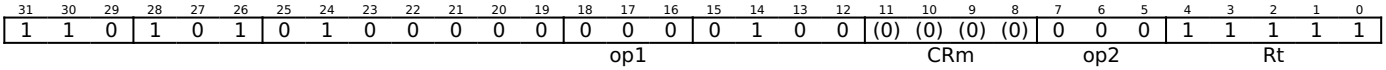
CFINV

Invert carry flag

This instruction inverts the value of the PSTATE.C flag.

System

(FEAT_FlagM)



Encoding

```
CFINV
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FlagM) then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
PSTATE.C = NOT PSTATE.C;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLT<cc> (immediate)

Compare register with immediate and fault

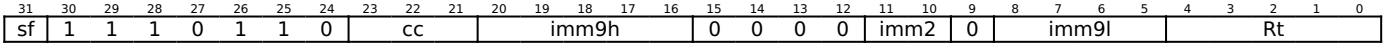
This instruction compares the value in a register with an immediate, and generates a Conditional fault exception if the comparison is true. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.

This instruction does not affect the condition flags.

This instruction is used by the pseudo-instructions CFLTGE (immediate), CFLTHS (immediate), CFLTLE (immediate), CFLTLS (immediate), CFLTNZ, and CFLTZ.

Fault

(FEAT_CFLT)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0 && cc == 000)`

```
CFLTGT #<reason>, <Wt>, #<imm>
```

Encoding for the 32-bit less than variant

Applies when `(sf == 0 && cc == 001)`

```
CFLTLT #<reason>, <Wt>, #<imm>
```

Encoding for the 32-bit higher variant

Applies when `(sf == 0 && cc == 010)`

```
CFLTHI #<reason>, <Wt>, #<imm>
```

Encoding for the 32-bit lower variant

Applies when `(sf == 0 && cc == 011)`

```
CFLTLO #<reason>, <Wt>, #<imm>
```

Encoding for the 32-bit equal variant

Applies when `(sf == 0 && cc == 100)`

```
CFLTEQ #<reason>, <Wt>, #<imm>
```

Encoding for the 32-bit not equal variant

Applies when `(sf == 0 && cc == 101)`

```
CFLTNE #<reason>, <Wt>, #<imm>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1 && cc == 000)`

```
CFLTGT #<reason>, <Xt>, #<imm>
```

Encoding for the 64-bit less than variant

Applies when `(sf == 1 && cc == 001)`

```
CFLTLT #<reason>, <Xt>, #<imm>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1 && cc == 010)`

```
CFLTHI #<reason>, <Xt>, #<imm>
```

Encoding for the 64-bit lower variant

Applies when `(sf == 1 && cc == 011)`

```
CFLTLO #<reason>, <Xt>, #<imm>
```

Encoding for the 64-bit equal variant

Applies when `(sf == 1 && cc == 100)`

```
CFLTEQ #<reason>, <Xt>, #<imm>
```

Encoding for the 64-bit not equal variant

Applies when `(sf == 1 && cc == 101)`

```
CFLTNE #<reason>, <Xt>, #<imm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CFLT) then EndOfDecode(Decode_UNDEF); end;
let datasize : integer{} = 32 << UInt(sf);
let t : integer = UInt(Rt);
let imm : bits(9) = imm9h::imm9l;
let reason : bits(16) = ZeroExtend{}(imm2);

var cmp : CmpOp;
var unsigned : boolean;

case cc of
    when '000' => cmp = Cmp_GT; unsigned = FALSE;
    when '001' => cmp = Cmp_LT; unsigned = FALSE;
    when '010' => cmp = Cmp_GT; unsigned = TRUE;
    when '011' => cmp = Cmp_LT; unsigned = TRUE;
    when '100' => cmp = Cmp_EQ; unsigned = FALSE;
    when '101' => cmp = Cmp_NE; unsigned = FALSE;
    otherwise  => EndOfDecode(Decode_UNDEF);
end;
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<imm>	For the "32-bit equal", "32-bit greater than", "32-bit less than", "32-bit not equal", "64-bit equal", "64-bit greater than", "64-bit less than", and "64-bit not equal" variants: is a signed immediate, in the range -256 to 255, encoded in "imm9h:imm9l". For the "32-bit higher", "32-bit lower", "64-bit higher", and "64-bit lower" variants: is an unsigned immediate, in the range 0 to 511, encoded in "imm9h:imm9l".
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

```
CheckCFLTEnabled();

let operand1 : bits(datasize) = X{}(t);

var cond : boolean;
let value1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
let value2 : integer = if unsigned then UInt(imm) else SInt(imm);

case cmp of
    when Cmp_EQ => cond = value1 == value2;
    when Cmp_NE => cond = value1 != value2;
    when Cmp_LT => cond = value1 < value2;
    when Cmp_GT => cond = value1 > value2;
end;
if cond then
    let isflt : boolean = FALSE;
    AArch64_ConditionalFault(isflt, reason);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLT<cc> (register)

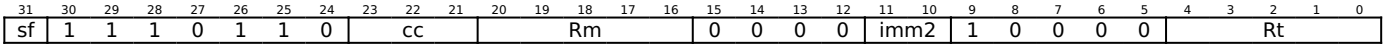
Compare registers and fault

This instruction compares the values in two registers, and generates a Conditional fault exception if the condition is true. This instruction does not affect the condition flags.

This instruction is used by the pseudo-instructions CFLTLE (register), CFLTLO (register), CFLTLS (register), and CFLTLT (register).

Fault

(FEAT_CFLT)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0 && cc == 000)`

```
CFLTGT #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 32-bit greater than or equal variant

Applies when `(sf == 0 && cc == 001)`

```
CFLTGE #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 32-bit higher variant

Applies when `(sf == 0 && cc == 010)`

```
CFLTHI #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 32-bit higher or same variant

Applies when `(sf == 0 && cc == 011)`

```
CFLTHS #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 32-bit equal variant

Applies when `(sf == 0 && cc == 110)`

```
CFLTEQ #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 32-bit not equal variant

Applies when `(sf == 0 && cc == 111)`

```
CFLTNE #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1 && cc == 000)`

```
CFLTGT #<reason>, <Xt|SP>, <Xm|SP>
```

Encoding for the 64-bit greater than or equal variant

Applies when `(sf == 1 && cc == 001)`

```
CFLTGE #<reason>, <Xt|SP>, <Xm|SP>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1 && cc == 010)`

```
CFLTHI #<reason>, <Xt|SP>, <Xm|SP>
```

Encoding for the 64-bit higher or same variant

Applies when `(sf == 1 && cc == 011)`

```
CFLTHS #<reason>, <Xt|SP>, <Xm|SP>
```

Encoding for the 64-bit equal variant

Applies when `(sf == 1 && cc == 110)`

```
CFLTEQ #<reason>, <Xt|SP>, <Xm|SP>
```

Encoding for the 64-bit not equal variant

Applies when `(sf == 1 && cc == 111)`

```
CFLTNE #<reason>, <Xt|SP>, <Xm|SP>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CFLT) then EndOfDecode(Decode_UNDEF); end;

let datasize : integer{} = 32 << UInt(sf);
let t : integer = UInt(Rt);
let m : integer = UInt(Rm);
let reason : bits(16) = ZeroExtend{}(imm2);

var cmp : CmpOp;
var unsigned : boolean;

case cc of
    when '000' => cmp = Cmp_GT; unsigned = FALSE;
    when '001' => cmp = Cmp_GE; unsigned = FALSE;
    when '010' => cmp = Cmp_GT; unsigned = TRUE;
    when '011' => cmp = Cmp_GE; unsigned = TRUE;
    when '110' => cmp = Cmp_EQ; unsigned = TRUE;
    when '111' => cmp = Cmp_NE; unsigned = TRUE;
    otherwise  => EndOfDecode(Decode_UNDEF);
end;
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt|WSP>	Is the 32-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rt" field.
<Wm|WSP>	Is the 32-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.
<Xt|SP>	Is the 64-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rt" field.
<Xm|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.

Operation

```
CheckCFLTEnabled();

let operand1 : bits(datasize) = if t == 31 then SP{datasize}() else X{datasize}(t);
let operand2 : bits(datasize) = if m == 31 then SP{datasize}() else X{datasize}(m);

var cond : boolean;
let value1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
let value2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
case cmp of
    when Cmp_EQ => cond = value1 == value2;
    when Cmp_NE => cond = value1 != value2;
    when Cmp_GT => cond = value1 >  value2;
    when Cmp_GE => cond = value1 >= value2;
end;

if cond then
    let isflt : boolean = FALSE;
    AArch64_ConditionalFault(isflt, reason);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTGE (immediate)

Compare signed greater than or equal to immediate and fault

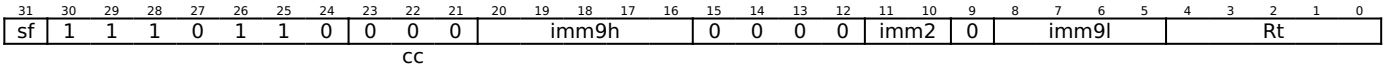
This instruction compares the signed value in a register with an immediate, and generates a Conditional fault exception if the register value is greater than or equal to the immediate. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0)`

```
CFLTGE #<reason>, <Wt>, #<immp1>
```

is equivalent to

```
CFLTGT #<reason>, <Wt>, #<imm>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1)`

```
CFLTGE #<reason>, <Xt>, #<immp1>
```

is equivalent to

```
CFLTGT #<reason>, <Xt>, #<imm>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<immp1>	Is a signed immediate, in the range -255 to 256, encoded as "imm9h:imm9l" plus 1.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTHS (immediate)

Compare unsigned higher than or same as immediate and fault

This instruction compares the unsigned value in a register with an immediate, and generates a Conditional fault exception if the register value is higher than or the same as the immediate. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.

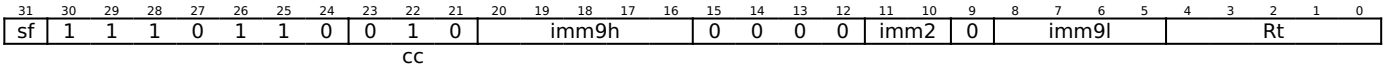
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit higher variant

Applies when `(sf == 0)`

```
CFLTHS #<reason>, <Wt>, #<immp1>
```

is equivalent to

```
CFLTHI #<reason>, <Wt>, #<imm>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1)`

```
CFLTHS #<reason>, <Xt>, #<immp1>
```

is equivalent to

```
CFLTHI #<reason>, <Xt>, #<imm>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<immp1>	Is an unsigned immediate, in the range 1 to 512, encoded as "imm9h:imm9l" plus 1.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTLE (immediate)

Compare signed less than or equal to immediate and fault

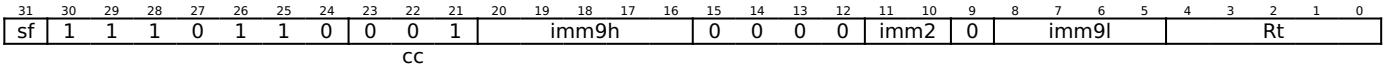
This instruction compares the signed value in a register with an immediate, and generates a Conditional fault exception if the register value is less than or equal to the immediate. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit less than variant

Applies when `(sf == 0)`

```
CFLTLE #<reason>, <Wt>, #<imms1>
```

is equivalent to

```
CFLTLT #<reason>, <Wt>, #<imm>
```

Encoding for the 64-bit less than variant

Applies when `(sf == 1)`

```
CFLTLE #<reason>, <Xt>, #<imms1>
```

is equivalent to

```
CFLTLT #<reason>, <Xt>, #<imm>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<imms1>	Is a signed immediate, in the range -257 to 254, encoded as "imm9h:imm9l" minus 1.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTLE (register)

Compare registers and fault if less than or equal

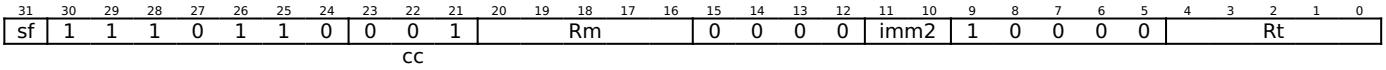
This instruction compares the values in two registers, and generates a Conditional fault exception if the first value is less than or equal to the second. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (register). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit greater than or equal variant

Applies when `(sf == 0)`

```
CFLTLE #<reason>, <Wm|WSP>, <Wt|WSP>
```

is equivalent to

```
CFLTGE #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 64-bit greater than or equal variant

Applies when `(sf == 1)`

```
CFLTLE #<reason>, <Xm|SP>, <Xt|SP>
```

is equivalent to

```
CFLTGE #<reason>, <Xt|SP>, <Xm|SP>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wm|WSP>	Is the 32-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Wt|WSP>	Is the 32-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xm|SP>	Is the 64-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Xt|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTLO (register)

Compare registers and fault if lower

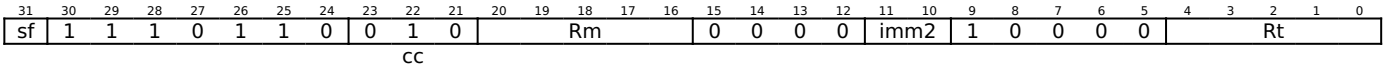
This instruction compares the values in two registers, and generates a Conditional fault exception if the first value is lower than the second. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (register). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit higher variant

Applies when `(sf == 0)`

```
CFLTLO #<reason>, <Wm|WSP>, <Wt|WSP>
```

is equivalent to

```
CFLTHI #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 64-bit higher variant

Applies when `(sf == 1)`

```
CFLTLO #<reason>, <Xm|SP>, <Xt|SP>
```

is equivalent to

```
CFLTHI #<reason>, <Xt|SP>, <Xm|SP>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wm|WSP>	Is the 32-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Wt|WSP>	Is the 32-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xm|SP>	Is the 64-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Xt|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTLS (immediate)

Compare unsigned lower than or same as immediate and fault

This instruction compares the unsigned value in a register with an immediate, and generates a Conditional fault exception if the register value is lower than or the same as the immediate. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.

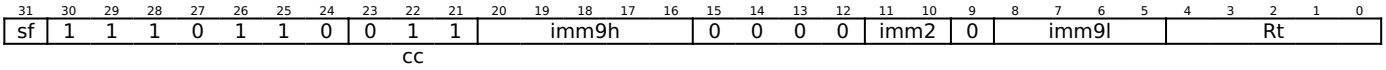
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit lower variant

Applies when `(sf == 0)`

```
CFLTLS #<reason>, <Wt>, #<imms1>
```

is equivalent to

```
CFLTLO #<reason>, <Wt>, #<imm>
```

Encoding for the 64-bit lower variant

Applies when `(sf == 1)`

```
CFLTLS #<reason>, <Xt>, #<imms1>
```

is equivalent to

```
CFLTLO #<reason>, <Xt>, #<imm>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<imms1>	Is a signed immediate, in the range -1 to 510, encoded as "imm9h:imm9l" minus 1.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTLS (register)

Compare registers and fault if lower than or same

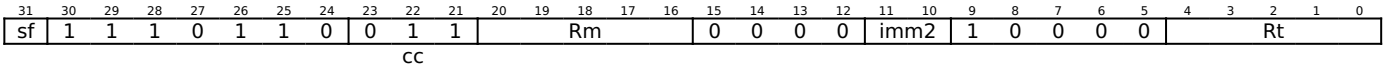
This instruction compares the values in two registers, and generates a Conditional fault exception if the first value is lower than or the same as the second. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (register). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit higher or same variant

Applies when `(sf == 0)`

```
CFLTLS #<reason>, <Wm|WSP>, <Wt|WSP>
```

is equivalent to

```
CFLTHS #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 64-bit higher or same variant

Applies when `(sf == 1)`

```
CFLTLS #<reason>, <Xm|SP>, <Xt|SP>
```

is equivalent to

```
CFLTHS #<reason>, <Xt|SP>, <Xm|SP>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wm|WSP>	Is the 32-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Wt|WSP>	Is the 32-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xm|SP>	Is the 64-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Xt|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTLT (register)

Compare registers and fault if less than

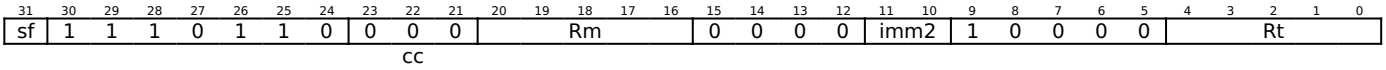
This instruction compares the values in two registers, and generates a Conditional fault exception if the first value is less than the second. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (register). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (register).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit greater than variant

Applies when `(sf == 0)`

```
CFLTLT #<reason>, <Wm|WSP>, <Wt|WSP>
```

is equivalent to

```
CFLTGT #<reason>, <Wt|WSP>, <Wm|WSP>
```

Encoding for the 64-bit greater than variant

Applies when `(sf == 1)`

```
CFLTLT #<reason>, <Xm|SP>, <Xt|SP>
```

is equivalent to

```
CFLTGT #<reason>, <Xt|SP>, <Xm|SP>
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wm|WSP>	Is the 32-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Wt|WSP>	Is the 32-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xm|SP>	Is the 64-bit name of the general-purpose register or stack pointer to be tested, encoded in the "Rm" field.
<Xt|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (register) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTNZ

Compare and fault on nonzero

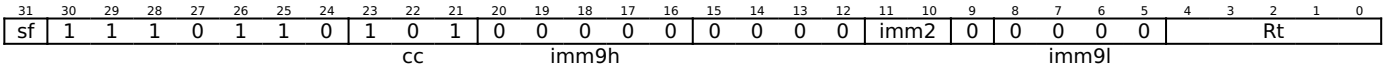
This instruction generates a Conditional fault exception if the register value is nonzero. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit not equal variant

Applies when `(sf == 0)`

```
CFLTNZ #<reason>, <Wt>
```

is equivalent to

```
CFLTNE #<reason>, <Wt>, #0
```

Encoding for the 64-bit not equal variant

Applies when `(sf == 1)`

```
CFLTNZ #<reason>, <Xt>
```

is equivalent to

```
CFLTNE #<reason>, <Xt>, #0
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFLTZ

Compare and fault on zero

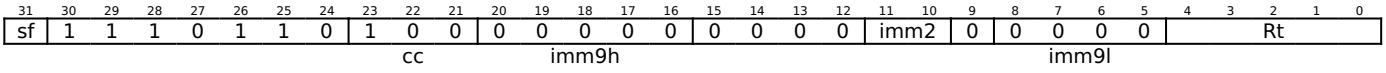
This instruction generates a Conditional fault exception if the register value is zero. If an exception is generated, the <reason> operand is copied to *ESR_ELx*.
This instruction does not affect the condition flags.

This is a pseudo-instruction of CFLT<cc> (immediate). This means:

- The encodings in this description are named to match the encodings of CFLT<cc> (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CFLT<cc> (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Fault

(FEAT_CFLT)



Encoding for the 32-bit equal variant

Applies when `(sf == 0)`

```
CFLTZ #<reason>, <Wt>
```

is equivalent to

```
CFLTEQ #<reason>, <Wt>, #0
```

Encoding for the 64-bit equal variant

Applies when `(sf == 1)`

```
CFLTZ #<reason>, <Xt>
```

is equivalent to

```
CFLTEQ #<reason>, <Xt>, #0
```

Assembler Symbols

<reason>	Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.
<Wt>	Is the 32-bit name of the general-purpose register to be tested, encoded in the "Rt" field.
<Xt>	Is the 64-bit name of the general-purpose register to be tested, encoded in the "Rt" field.

Operation

The description of CFLT<cc> (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CFP

Control flow prediction restriction by context

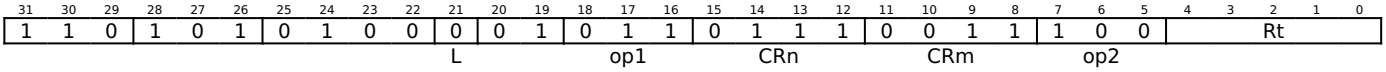
This instruction prevents control flow predictions that predict execution addresses based on information gathered from earlier execution within a particular execution context. Control flow predictions determined by the actions of code in the target execution context or contexts appearing in program order before the instruction cannot be used to exploitatively control speculative execution occurring after the instruction is complete and synchronized.
For more information, see CFP RCTX, Control Flow Prediction Restriction by Context.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_SPECRES)



Encoding

```
CFP RCTX, <Xt>
```

is equivalent to

```
SYS #3, C7, C3, #4, <Xt>
```

and is always the preferred disassembly.

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

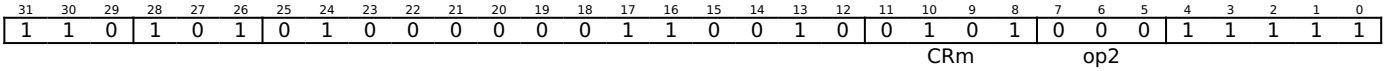
CHKFEAT

Check feature status

This instruction indicates the status of features. For more information, see *Check Feature*.
If *FEAT_CHK* is not implemented, this instruction executes as a `NOP`.

System

(FEAT_CHK)



Encoding

```
CHKFEAT X16
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CHK) then EndOfDecode(Decode_NOP); end;
```

Operation

```
X{64}(16) = AArch64_ChkFeat(X{64}(16));
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

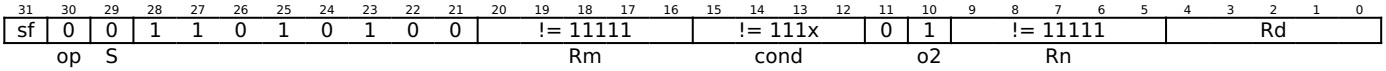
CINC

Conditional increment

This instruction returns, in the destination register, the value of the source register incremented by 1 if the condition is TRUE, and otherwise returns the value of the source register.

This is an alias of CSINC. This means:

- The encodings in this description are named to match the encodings of CSINC.
- The description of CSINC gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CINC <Wd>, <Wn>, <invcond>
```

is equivalent to

```
CSINC <Wd>, <Wn>, <Wm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CINC <Xd>, <Xn>, <invcond>
```

is equivalent to

```
CSINC <Xd>, <Xn>, <Xm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

<invcond> Is one of the standard conditions, excluding AL and NV, encoded with its least significant bit inverted, and encoded in “cond”:

cond	<invcond>	Description
0000	NE	Maps to <cond> EQ.
0001	EQ	Maps to <cond> NE.
0010	CC|LO	Maps to <cond> CS.
0011	CS|HS	Maps to <cond> CC.
0100	PL	Maps to <cond> MI.
0101	MI	Maps to <cond> PL.
0110	VC	Maps to <cond> VS.
0111	VS	Maps to <cond> VC.
1000	LS	Maps to <cond> HI.
1001	HI	Maps to <cond> LS.
1010	LT	Maps to <cond> GE.
1011	GE	Maps to <cond> LT.
1100	LE	Maps to <cond> GT.
1101	GT	Maps to <cond> LE.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

Operation

The description of CSINC gives the operational pseudocode for this instruction.

Operational Information

The description of CSINC gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

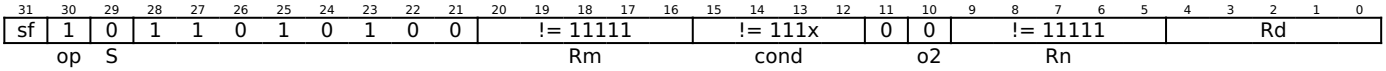
CINV

Conditional invert

This instruction returns, in the destination register, the bitwise inversion of the value of the source register if the condition is TRUE, and otherwise returns the value of the source register.

This is an alias of CSINV. This means:

- The encodings in this description are named to match the encodings of CSINV.
- The description of CSINV gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CINV <Wd>, <Wn>, <invcond>
```

is equivalent to

```
CSINV <Wd>, <Wn>, <Wm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CINV <Xd>, <Xn>, <invcond>
```

is equivalent to

```
CSINV <Xd>, <Xn>, <Xm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

<invcond> Is one of the standard conditions, excluding AL and NV, encoded with its least significant bit inverted, and encoded in “cond”:

cond	<invcond>	Description
0000	NE	Maps to <cond> EQ.
0001	EQ	Maps to <cond> NE.
0010	CC|LO	Maps to <cond> CS.
0011	CS|HS	Maps to <cond> CC.
0100	PL	Maps to <cond> MI.
0101	MI	Maps to <cond> PL.
0110	VC	Maps to <cond> VS.
0111	VS	Maps to <cond> VC.
1000	LS	Maps to <cond> HI.
1001	HI	Maps to <cond> LS.
1010	LT	Maps to <cond> GE.
1011	GE	Maps to <cond> LT.
1100	LE	Maps to <cond> GT.
1101	GT	Maps to <cond> LE.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

Operation

The description of CSINV gives the operational pseudocode for this instruction.

Operational Information

The description of CSINV gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

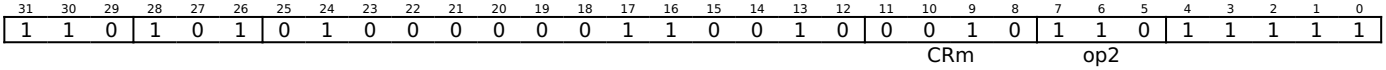
CLRBHB

Clear branch history

This instruction can be used to clear the branch history.
For more information, see *Branch history*.

System

(FEAT_CLRBHB)



Encoding

CLRBHB

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CLRBHB) then EndOfDecode(Decode_NOP); end;
```

Operation

```
Hint_CLRBHB();
```

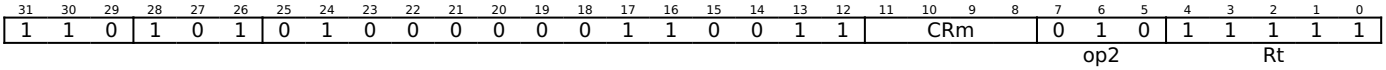
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CLREX

Clear exclusive

This instruction clears the local monitor of the executing PE.



Encoding

```
CLREX {#<imm>}
```

Decode for this encoding

```
// CRm field is ignored
```

Assembler Symbols

<imm>	Is an optional 4-bit unsigned immediate, in the range 0 to 15, defaulting to 15 and encoded in the "CRm" field.

Operation

```
ClearExclusiveLocal(ProcessorID());
```

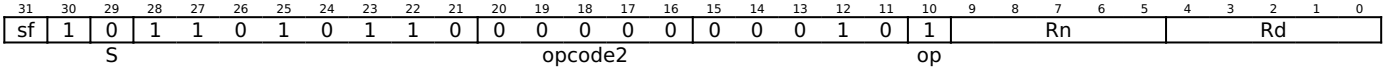
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CLS

Count leading sign bits

This instruction determines the number of leading bits of the source register that have the same value as the most significant bit of the register, and writes the result to the destination register. The result does not include the most significant bit of the source register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CLS <Wd>, <Wn>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CLS <Xd>, <Xn>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let result : integer = CountLeadingSignBits(operand1);

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

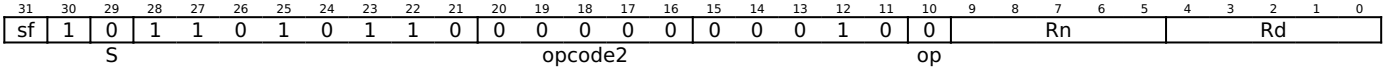
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CLZ

Count leading zeros

This instruction determines the number of consecutive binary zero bits, starting from the most significant bit in the source register, and places the result in the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CLZ <Wd>, <Wn>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CLZ <Xd>, <Xn>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let result : integer = CountLeadingZeroBits(operand1);

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

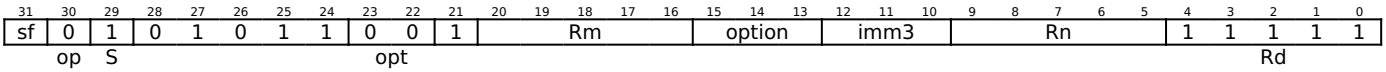
CMN (extended register)

Compare negative (extended register)

This instruction adds a register value and a sign-extended or zero-extended register value, followed by an optional left shift amount. The argument that is extended from the <Rm> register can be a byte, halfword, word, or doubleword. It updates the condition flags based on the result, and discards the result.

This is an alias of ADDS (extended register). This means:

- The encodings in this description are named to match the encodings of ADDS (extended register).
- The description of ADDS (extended register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CMN <Wn|WSP>, <Wm>{, <extend> {#<amount>}}
```

is equivalent to

```
ADDS WZR, <Wn|WSP>, <Wm>{, <extend> {#<amount>}}
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CMN <Xn|SP>, <R><m>{, <extend> {#<amount>}}
```

is equivalent to

```
ADDS XZR, <Xn|SP>, <R><m>{, <extend> {#<amount>}}
```

and is always the preferred disassembly.

Assembler Symbols

<Wn|WSP> Is the 32-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<extend> For the "32-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	LSL|UXTW
011	UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rn" is '11111' (WSP) and "option" is '010' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTW when "option" is '010'.

For the "64-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	UXTW
011	LSL|UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rn" is '11111' (SP) and "option" is '011' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTX when "option" is '011'.

<amount> Is the left shift amount to be applied after extension in the range 0 to 4, defaulting to 0, encoded in the "imm3" field. It must be absent when <extend> is absent, is required when <extend> is LSL, and is optional when <extend> is present but not LSL.

<Xn|SP> Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<R> Is a width specifier, encoded in “option”:

option	<R>
00x	W
010	W
x11	X
10x	W
110	W

<m> Is the number [0-30] of the second general-purpose source register or the name ZR (31), encoded in the "Rm" field.

Operation

The description of ADDS (extended register) gives the operational pseudocode for this instruction.

Operational Information

The description of ADDS (extended register) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

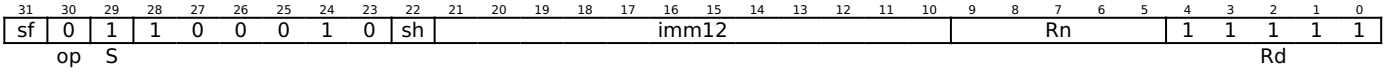
CMN (immediate)

Compare negative (immediate)

This instruction adds a register value and an optionally-shifted immediate value. It updates the condition flags based on the result, and discards the result.

This is an alias of ADDS (immediate). This means:

- The encodings in this description are named to match the encodings of ADDS (immediate).
- The description of ADDS (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CMN <Wn|WSP>, #<imm>{, <shift>}
```

is equivalent to

```
ADDS WZR, <Wn|WSP>, #<imm>{, <shift>}
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CMN <Xn|SP>, #<imm>{, <shift>}
```

is equivalent to

```
ADDS XZR, <Xn|SP>, #<imm>{, <shift>}
```

and is always the preferred disassembly.

Assembler Symbols

<Wn|WSP> Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

<imm> Is an unsigned immediate, in the range 0 to 4095, encoded in the "imm12" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #12

<Xn|SP> Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Operation

The description of ADDS (immediate) gives the operational pseudocode for this instruction.

Operational Information

The description of ADDS (immediate) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

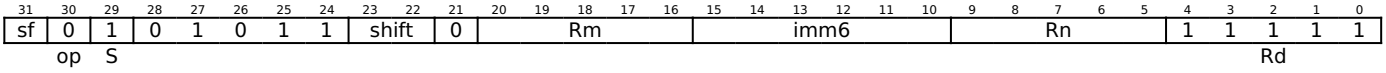
CMN (shifted register)

Compare negative (shifted register)

This instruction adds a register value and an optionally-shifted register value. It updates the condition flags based on the result, and discards the result.

This is an alias of ADDS (shifted register). This means:

- The encodings in this description are named to match the encodings of ADDS (shifted register).
- The description of ADDS (shifted register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CMN <Wn>, <Wm>{, <shift> #<amount>}
```

is equivalent to

```
ADDS WZR, <Wn>, <Wm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CMN <Xn>, <Xm>{, <shift> #<amount>}
```

is equivalent to

```
ADDS XZR, <Xn>, <Xm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Assembler Symbols

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

The description of ADDS (shifted register) gives the operational pseudocode for this instruction.

Operational Information

The description of ADDS (shifted register) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

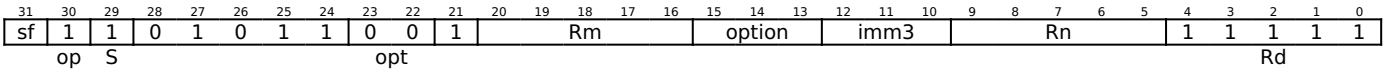
CMP (extended register)

Compare (extended register)

This instruction subtracts a sign-extended or zero-extended register value, followed by an optional left shift amount, from a register value. The argument that is extended from the <Rm> register can be a byte, halfword, word, or doubleword. It updates the condition flags based on the result, and discards the result.

This is an alias of SUBS (extended register). This means:

- The encodings in this description are named to match the encodings of SUBS (extended register).
- The description of SUBS (extended register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CMP <Wn|WSP>, <Wm>{, <extend> {#<amount>}}
```

is equivalent to

```
SUBS WZR, <Wn|WSP>, <Wm>{, <extend> {#<amount>}}
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CMP <Xn|SP>, <R><m>{, <extend> {#<amount>}}
```

is equivalent to

```
SUBS XZR, <Xn|SP>, <R><m>{, <extend> {#<amount>}}
```

and is always the preferred disassembly.

Assembler Symbols

<Wn|WSP> Is the 32-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<extend> For the "32-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	LSL|UXTW
011	UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rn" is '11111' (WSP) and "option" is '010' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTW when "option" is '010'.

For the "64-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	UXTW
011	LSL|UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rn" is '11111' (SP) and "option" is '011' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTX when "option" is '011'.

<amount> Is the left shift amount to be applied after extension in the range 0 to 4, defaulting to 0, encoded in the "imm3" field. It must be absent when <extend> is absent, is required when <extend> is LSL, and is optional when <extend> is present but not LSL.

<Xn|SP> Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<R> Is a width specifier, encoded in “option”:

option	<R>
00x	W
010	W
x11	X
10x	W
110	W

<m> Is the number [0-30] of the second general-purpose source register or the name ZR (31), encoded in the "Rm" field.

Operation

The description of SUBS (extended register) gives the operational pseudocode for this instruction.

Operational Information

The description of SUBS (extended register) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

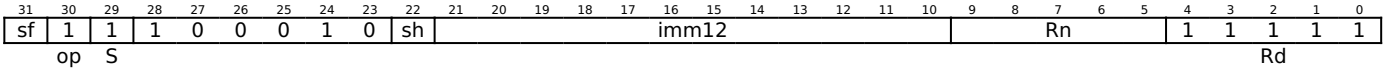
CMP (immediate)

Compare (immediate)

This instruction subtracts an optionally-shifted immediate value from a register value. It updates the condition flags based on the result, and discards the result.

This is an alias of SUBS (immediate). This means:

- The encodings in this description are named to match the encodings of SUBS (immediate).
- The description of SUBS (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CMP <Wn|WSP>, #<imm>{, <shift>}
```

is equivalent to

```
SUBS WZR, <Wn|WSP>, #<imm>{, <shift>}
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CMP <Xn|SP>, #<imm>{, <shift>}
```

is equivalent to

```
SUBS XZR, <Xn|SP>, #<imm>{, <shift>}
```

and is always the preferred disassembly.

Assembler Symbols

<Wn|WSP> Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

<imm> Is an unsigned immediate, in the range 0 to 4095, encoded in the "imm12" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #12

<Xn|SP> Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Operation

The description of SUBS (immediate) gives the operational pseudocode for this instruction.

Operational Information

The description of SUBS (immediate) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

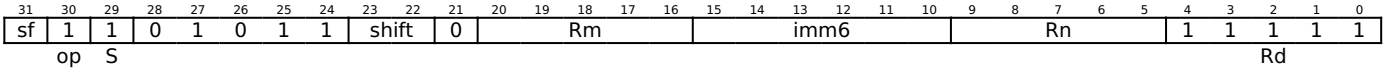
CMP (shifted register)

Compare (shifted register)

This instruction subtracts an optionally-shifted register value. It updates the condition flags based on the result, and discards the result.

This is an alias of SUBS (shifted register). This means:

- The encodings in this description are named to match the encodings of SUBS (shifted register).
- The description of SUBS (shifted register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CMP <Wn>, <Wm>{, <shift> #<amount>}
```

is equivalent to

```
SUBS WZR, <Wn>, <Wm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CMP <Xn>, <Xm>{, <shift> #<amount>}
```

is equivalent to

```
SUBS XZR, <Xn>, <Xm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Assembler Symbols

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

The description of SUBS (shifted register) gives the operational pseudocode for this instruction.

Operational Information

The description of SUBS (shifted register) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMPP

Compare with tag

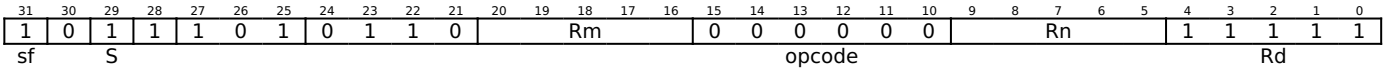
This instruction subtracts the 56-bit address held in the second source register from the 56-bit address held in the first source register, updates the condition flags based on the result of the subtraction, and discards the result.

This is an alias of SUBPS. This means:

- The encodings in this description are named to match the encodings of SUBPS.
- The description of SUBPS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_MTE)



Encoding

```
CMPP <Xn|SP>, <Xm|SP>
```

is equivalent to

```
SUBPS XZR, <Xn|SP>, <Xm|SP>
```

and is always the preferred disassembly.

Assembler Symbols

<Xn|SP>	Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.
<Xm|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.

Operation

The description of SUBPS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

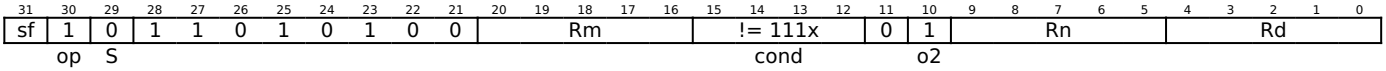
CNEG

Conditional negate

This instruction returns, in the destination register, the negated value of the source register if the condition is TRUE, and otherwise returns the value of the source register.

This is an alias of CSNEG. This means:

- The encodings in this description are named to match the encodings of CSNEG.
- The description of CSNEG gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CNEG <Wd>, <Wn>, <invcond>
```

is equivalent to

```
CSNEG <Wd>, <Wn>, <Wm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CNEG <Xd>, <Xn>, <invcond>
```

is equivalent to

```
CSNEG <Xd>, <Xn>, <Xm>, <cond>
```

and is the preferred disassembly when `Rn == Rm`.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

<invcond> Is one of the standard conditions, excluding AL and NV, encoded with its least significant bit inverted, and encoded in “cond”:

cond	<invcond>	Description
0000	NE	Maps to <cond> EQ.
0001	EQ	Maps to <cond> NE.
0010	CC|LO	Maps to <cond> CS.
0011	CS|HS	Maps to <cond> CC.
0100	PL	Maps to <cond> MI.
0101	MI	Maps to <cond> PL.
0110	VC	Maps to <cond> VS.
0111	VS	Maps to <cond> VC.
1000	LS	Maps to <cond> HI.
1001	HI	Maps to <cond> LS.
1010	LT	Maps to <cond> GE.
1011	GE	Maps to <cond> LT.
1100	LE	Maps to <cond> GT.
1101	GT	Maps to <cond> LE.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

Operation

The description of CSNEG gives the operational pseudocode for this instruction.

Operational Information

The description of CSNEG gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

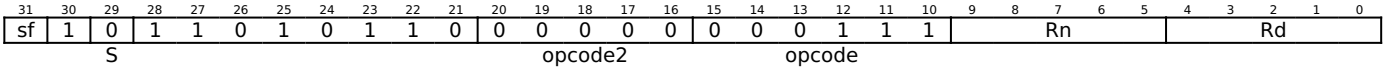
CNT

Count bits

This instruction determines the number of binary one bits in the value of the source register, and writes the result to the destination register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CNT <Wd>, <Wn>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CNT <Xd>, <Xn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let datasize : integer{} = 32 << UInt(sf);
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let result : integer = BitCount(operand1);

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

COSP

Clear other speculative prediction restriction by context

This instruction prevents predictions, other than Cache prefetch, Control flow, and Data Value predictions, that predict execution addresses based on information gathered from earlier execution within a particular execution context. Predictions, other than Cache prefetch, Control flow, and Data Value predictions, determined by the actions of code in the target execution context or contexts appearing in program order before the instruction cannot exploitatively control any speculative access occurring after the instruction is complete and synchronized.

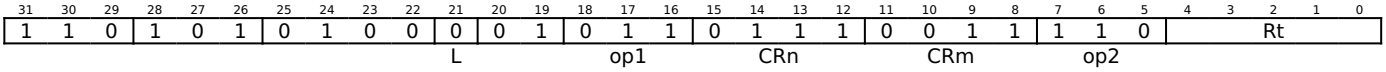
For more information, see COSP RCTX, Clear Other Speculative Prediction Restriction by Context.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_SPECRES2)



Encoding

```
COSP RCTX, <Xt>
```

is equivalent to

```
SYS #3, C7, C3, #6, <Xt>
```

and is always the preferred disassembly.

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPP

Cache prefetch prediction restriction by context

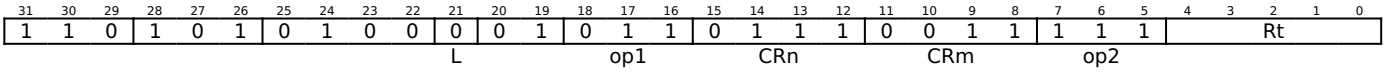
This instruction prevents cache allocation predictions that predict execution addresses based on information gathered from earlier execution within a particular execution context. The actions of code in the target execution context or contexts appearing in program order before the instruction cannot exploitatively control cache prefetch predictions occurring after the instruction is complete and synchronized.
For more information, see CPP RCTX, Cache Prefetch Prediction Restriction by Context.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_SPECRES)



Encoding

```
CPP RCTX, <Xt>
```

is equivalent to

```
SYS #3, C7, C3, #7, <Xt>
```

and is always the preferred disassembly.

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFP, CPYFM, CPYFE

Memory copy forward-only

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFP, then CPYFM, and then CPYFE.
CPYFP performs some preconditioning of the arguments suitable for using the CPYFM instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFM copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFE copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFP:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFP, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFP, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFM, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFM, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the lowest address that has not been copied from.
    - Xd holds the lowest address that has not been copied to.

For CPYFE, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

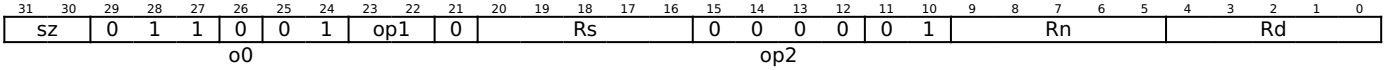
For CPYFE, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFP [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFM [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFE [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPN, CPYFMN, CPYFEN

Memory copy forward-only, reads and writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPN, then CPYFMN, and then CPYFEN.
CPYFPN performs some preconditioning of the arguments suitable for using the CPYFMN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFEN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFEN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

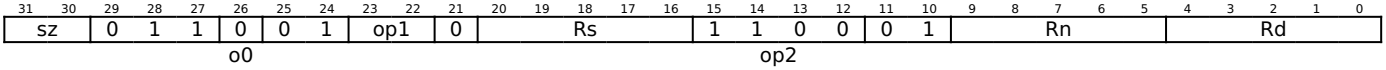
For CPYFEN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFEN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPRN, CPYFMRN, CPYFERN

Memory copy forward-only, reads non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPRN, then CPYFMRN, and then CPYFERN.
CPYFPRN performs some preconditioning of the arguments suitable for using the CPYFMRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFERN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPRN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPRN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPRN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMRN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMRN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFERN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

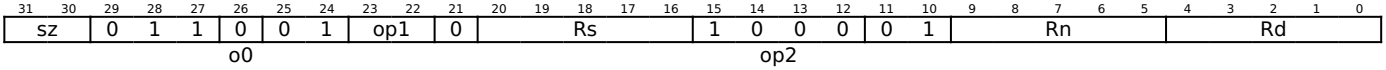
For CPYFERN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFERN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPRT, CPYFMRT, CPYFERT

Memory copy forward-only, reads unprivileged

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPRT, then CPYFMRT, and then CPYFERT.
CPYFPRT performs some preconditioning of the arguments suitable for using the CPYFMRT instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMRT copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFERT copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPRT:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPRT, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPRT, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMRT, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMRT, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFERT, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFERT, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

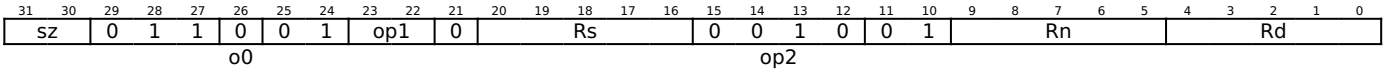
Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPRT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMRT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFERT [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPRTN, CPYFMRTN, CPYFERTN

Memory copy forward-only, reads unprivileged, reads and writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPRTN, then CPYFMRTN, and then CPYFERTN.
CPYFPRTN performs some preconditioning of the arguments suitable for using the CPYFMRTN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMRTN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFERTN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPRTN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPRTN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPRTN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMRTN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMRTN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFERTN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFERTN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

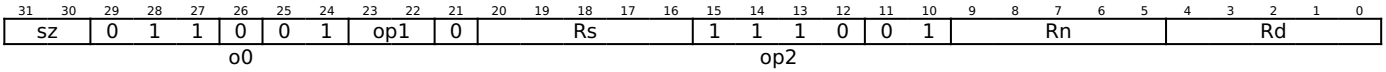
Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPRTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMRTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFERTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPRTRN, CPYFMRTRN, CPYFERTRN

Memory copy forward-only, reads unprivileged and non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPRTRN, then CPYFMRTRN, and then CPYFERTRN.
CPYFPRTRN performs some preconditioning of the arguments suitable for using the CPYFMRTRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMRTRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFERTRN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPRTRN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPRTRN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPRTRN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMRTRN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMRTRN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFERTRN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFERTRN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

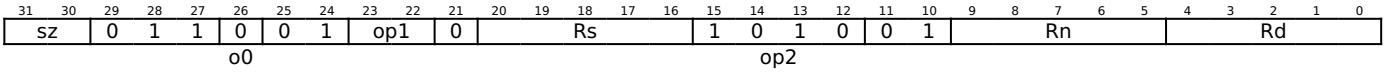
Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPRTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMRTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFERTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPRTWN, CPYFMRTWN, CPYFERTWN

Memory copy forward-only, reads unprivileged, writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPRTWN, then CPYFMRTWN, and then CPYFERTWN.
CPYFPRTWN performs some preconditioning of the arguments suitable for using the CPYFMRTWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMRTWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFERTWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPRTWN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPRTWN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPRTWN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMRTWN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMRTWN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFERTWN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFERTWN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

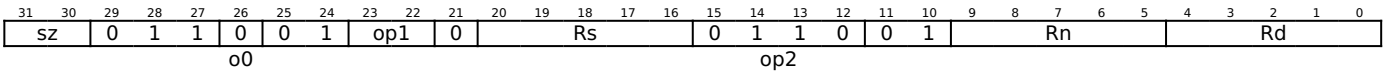
Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPRTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMRTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFERTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPT, CPYFMT, CPYFET

Memory copy forward-only, reads and writes unprivileged

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPT, then CPYFMT, and then CPYFET.
CPYFPT performs some preconditioning of the arguments suitable for using the CPYFMT instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMT copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFET copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPT:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPT, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPT, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMT, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMT, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the lowest address that has not been copied from.
    - Xd holds the lowest address that has not been copied to.

For CPYFET, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFET, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

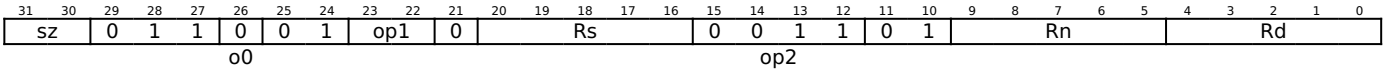
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFET [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPTN, CPYFMTN, CPYFETN

Memory copy forward-only, reads and writes unprivileged and non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPTN, then CPYFMTN, and then CPYFETN.
CPYFPTN performs some preconditioning of the arguments suitable for using the CPYFMTN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMTN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFETN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPTN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPTN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPTN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMTN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMTN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the lowest address that has not been copied from.
    - Xd holds the lowest address that has not been copied to.

For CPYFETN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFETN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

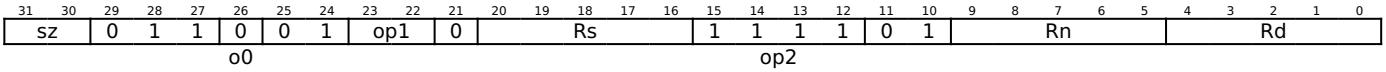
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFETN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPTRN, CPYFMTRN, CPYFETRN

Memory copy forward-only, reads and writes unprivileged, reads non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPTRN, then CPYFMTRN, and then CPYFETRN.
CPYFPTRN performs some preconditioning of the arguments suitable for using the CPYFMTRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMTRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFETRN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPTRN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPTRN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPTRN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMTRN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMTRN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the lowest address that has not been copied from.
    - Xd holds the lowest address that has not been copied to.

For CPYFETRN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFETRN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

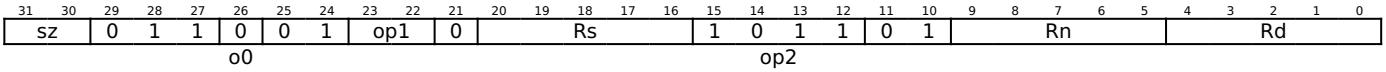
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFETRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPTWN, CPYFMTWN, CPYFETWN

Memory copy forward-only, reads and writes unprivileged, writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPTWN, then CPYFMTWN, and then CPYFETWN.
CPYFPTWN performs some preconditioning of the arguments suitable for using the CPYFMTWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMTWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFETWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPTWN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPTWN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPTWN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMTWN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMTWN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFETWN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFETWN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

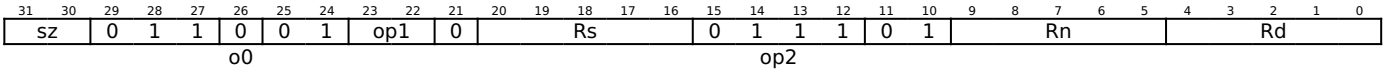
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFETWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPWN, CPYFMWN, CPYFEWN

Memory copy forward-only, writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPWN, then CPYFMWN, and then CPYFEWN.
CPYFPWN performs some preconditioning of the arguments suitable for using the CPYFMWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFEWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPWN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPWN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPWN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMWN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMWN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFEWN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

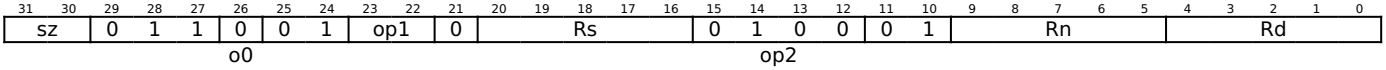
For CPYFEWN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFEWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPWT, CPYFMWT, CPYFEWT

Memory copy forward-only, writes unprivileged

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPWT, then CPYFMWT, and then CPYFEWT.
CPYFPWT performs some preconditioning of the arguments suitable for using the CPYFMWT instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMWT copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFEWT copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPWT:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPWT, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPWT, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMWT, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMWT, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the lowest address that has not been copied from.
    - Xd holds the lowest address that has not been copied to.

For CPYFEWT, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFEWT, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

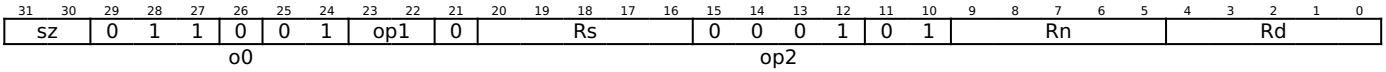
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPWT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMWT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFEWT [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPWTN, CPYFMWTN, CPYFEWTN

Memory copy forward-only, writes unprivileged, reads and writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPWTN, then CPYFMWTN, and then CPYFEWTN.
CPYFPWTN performs some preconditioning of the arguments suitable for using the CPYFMWTN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMWTN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFEWTN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPWTN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPWTN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPWTN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMWTN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMWTN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFEWTN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFEWTN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

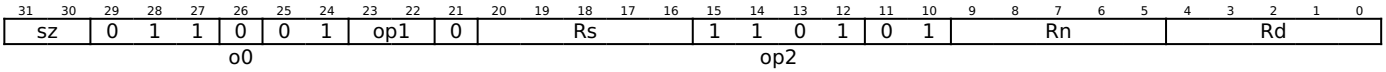
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPWTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMWTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFEWTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPWTRN, CPYFMWTRN, CPYFEWTRN

Memory copy forward-only, writes unprivileged, reads non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPWTRN, then CPYFMWTRN, and then CPYFEWTRN.
CPYFPWTRN performs some preconditioning of the arguments suitable for using the CPYFMWTRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMWTRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFEWTRN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPWTRN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPWTRN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPWTRN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMWTRN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMWTRN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFEWTRN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFEWTRN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

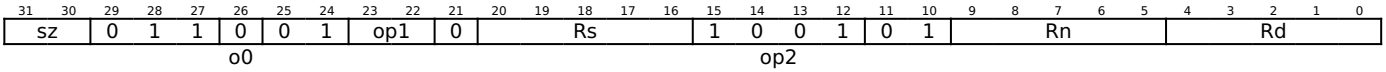
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPWTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMWTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFEWTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYFPWTWN, CPYFMWTWN, CPYFEWTWN

Memory copy forward-only, writes unprivileged and non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address in a forward direction. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYFPWTWN, then CPYFMWTWN, and then CPYFEWTWN.
CPYFPWTWN performs some preconditioning of the arguments suitable for using the CPYFMWTWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYFMWTWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYFEWTWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The memory copy performed by these instructions is in the forward direction only, so the instructions are suitable for a memory copy only where there is no overlap between the source and destination locations, or where the source address is greater than or equal to the destination address.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYFPWTWN:

- If Xn[63] == '1', the copy size is saturated to 0x7FFFFFFFFFFFFFF.

On completion of CPYFPWTWN, option A:

- Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the original Xs + saturated copy size.
- Xd holds the original Xd + saturated copy size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of CPYFPWTWN, option B:

- Xn holds the number of bytes in the saturated copy size remaining to be copied.
- Xs holds the lowest address that has not been copied from.
- Xd holds the lowest address that has not been copied to.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For CPYFMWTWN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.

For CPYFMWTWN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

For CPYFEWTWN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from - Xn.
- Xd holds the lowest address to be copied to - Xn.
- On completion of the instruction, Xn holds 0.

For CPYFEWTWN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be copied.
- Xs holds the lowest address to be copied from.
- Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds 0.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

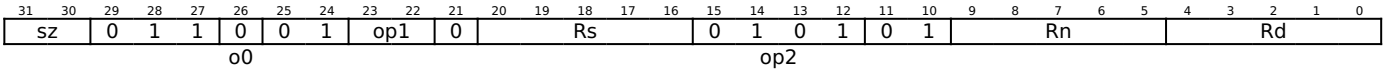
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYFPWTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYFMWTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYFEWTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);
memcpy.cpysize     = SInt(X{64}(memcpy.n));
memcpy.implements_option_a = CPYFOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize[63] == '1' then memcpy.cpysize = ArchMaxMOPSBlockSize; end;

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        // Copy in the forward direction offsets the arguments.
        memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
        memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
        memcpy.cpysize     = 0 - memcpy.cpysize;
    else
        memcpy.nzcv = '0010';
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
memcpy.forward = TRUE;
var fault : boolean  = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= -1 * memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                    memcpy.toaddress   + memcpy.cpysize,
                                                    memcpy.fromaddress + memcpy.cpysize,
                                                    memcpy.forward, B,
                                                    raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.cpysize      = memcpy.cpysize + B;
            memcpy.stagecpysize = memcpy.stagecpysize + B;
        end;
    end;
else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                memcpy.fromaddress,
                                                                memcpy.forward, B,
                                                                raccdesc, waccdesc);
        if copied != B then
            fault = TRUE;
        else
            memcpy.fromaddress  = memcpy.fromaddress + B;
            memcpy.toaddress    = memcpy.toaddress   + B;
            memcpy.cpysize      = memcpy.cpysize     - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);
```

```
if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if IsFault(memstatus) then
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYP, CPYM, CPYE

Memory copy

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYP, then CPYM, and then CPYE.

CPYP performs some preconditioning of the arguments suitable for using the CPYM instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYM copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYE copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.

The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYP:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYP, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYP, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYM, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYM, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.

- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYE, option A, when PSTATE.C = 0:

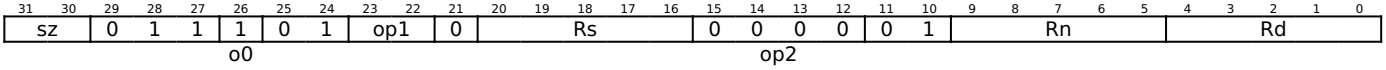
- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYE, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYP [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYM [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYE [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B,
                                                            raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPN, CPYMN, CPYEN

Memory copy, reads and writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPN, then CPYMN, and then CPYEN.
CPYPN performs some preconditioning of the arguments suitable for using the CPYMN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

        - Xd holds the lowest address to be copied to.
        - On completion of the instruction:
            - Xn holds the number of bytes remaining to be copied.
            - Xs holds the lowest address that has not been copied from.
            - Xd holds the lowest address that has not been copied to.
    - If the copy is in the backward direction (PSTATE.N == 1), then:
        - Xs holds the highest address to be copied from + 1.
        - Xd holds the highest address to be copied to + 1.
        - On completion of the instruction:
            - Xn holds the number of bytes remaining to be copied.
            - Xs holds the highest address that has not been copied from + 1.
            - Xd holds the highest address that has not been copied to + 1.

For CPYEN, option A, when PSTATE.C = 0:

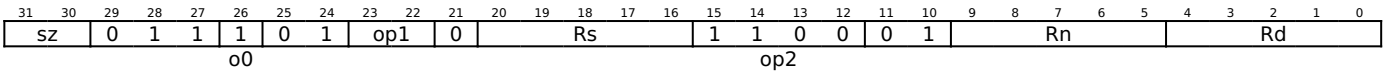
- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
    - Xn holds -1 times the number of bytes remaining to be copied.
    - Xs holds the lowest address to be copied from - Xn.
    - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address to be copied from + 1 - Xn.
    - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
    - Xn holds 0.
    - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied from.
        - The highest address that has been copied from + 1.
    - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied to.
        - The highest address that has not been copied to + 1.

For CPYEN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
    - Xs holds the lowest address to be copied from.
    - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
    - Xs holds the highest address to be copied from + 1.
    - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
    - Xn holds 0.
    - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied from.
        - The highest address that has been copied from + 1.
    - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied to.
        - The highest address that has not been copied to + 1.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYEN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPRN, CPYMRN, CPYERN

Memory copy, reads non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPRN, then CPYMRN, and then CPYERN.
CPYPRN performs some preconditioning of the arguments suitable for using the CPYMRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYERN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPRN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPRN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPRN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMRN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMRN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYERN, option A, when PSTATE.C = 0:

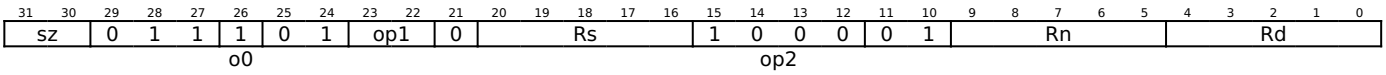
- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYERN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYERN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPRT, CPYMRT, CPYERT

Memory copy, reads unprivileged

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPRT, then CPYMRT, and then CPYERT.
CPYPRT performs some preconditioning of the arguments suitable for using the CPYMRT instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMRT copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYERT copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPRT:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPRT, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPRT, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMRT, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMRT, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYERT, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYERT, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

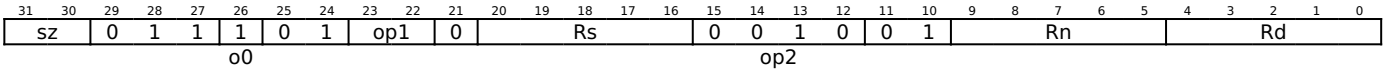
Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPRT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMRT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYERT [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPRTN, CPYMRTN, CPYERTN

Memory copy, reads unprivileged, reads and writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPRTN, then CPYMRTN, and then CPYERTN.
CPYPRTN performs some preconditioning of the arguments suitable for using the CPYMRTN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMRTN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYERTN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPRTN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPRTN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPRTN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMRTN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMRTN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

        - Xd holds the lowest address to be copied to.
        - On completion of the instruction:
            - Xn holds the number of bytes remaining to be copied.
            - Xs holds the lowest address that has not been copied from.
            - Xd holds the lowest address that has not been copied to.
    - If the copy is in the backward direction (PSTATE.N == 1), then:
        - Xs holds the highest address to be copied from + 1.
        - Xd holds the highest address to be copied to + 1.
        - On completion of the instruction:
            - Xn holds the number of bytes remaining to be copied.
            - Xs holds the highest address that has not been copied from + 1.
            - Xd holds the highest address that has not been copied to + 1.

For CPYERTN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
    - Xn holds -1 times the number of bytes remaining to be copied.
    - Xs holds the lowest address to be copied from - Xn.
    - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address to be copied from + 1 - Xn.
    - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
    - Xn holds 0.
    - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied from.
        - The highest address that has been copied from + 1.
    - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied to.
        - The highest address that has not been copied to + 1.

For CPYERTN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
    - Xs holds the lowest address to be copied from.
    - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
    - Xs holds the highest address to be copied from + 1.
    - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
    - Xn holds 0.
    - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied from.
        - The highest address that has been copied from + 1.
    - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied to.
        - The highest address that has not been copied to + 1.

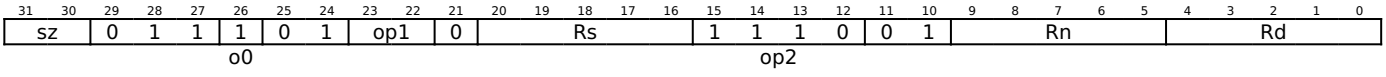
Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPRTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMRTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYERTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPRTRN, CPYMRTRN, CPYERTRN

Memory copy, reads unprivileged and non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPRTRN, then CPYMRTRN, and then CPYERTRN.
CPYPRTRN performs some preconditioning of the arguments suitable for using the CPYMRTRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMRTRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYERTRN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPRTRN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPRTRN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPRTRN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMRTRN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMRTRN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYERTRN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYERTRN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

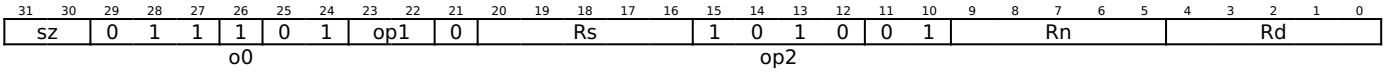
Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPRTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMRTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYERTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPRTWN, CPYMRTWN, CPYERTWN

Memory copy, reads unprivileged, writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPRTWN, then CPYMRTWN, and then CPYERTWN.
CPYPRTWN performs some preconditioning of the arguments suitable for using the CPYMRTWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMRTWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYERTWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPRTWN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPRTWN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPRTWN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMRTWN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMRTWN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYERTWN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYERTWN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

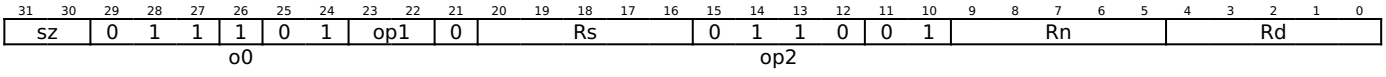
Explicit Memory Read effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Read effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPRTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMRTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYERTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPT, CPYMT, CPYET

Memory copy, reads and writes unprivileged

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPT, then CPYMT, and then CPYET.
CPYPT performs some preconditioning of the arguments suitable for using the CPYMT instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMT copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYET copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPT:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPT, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPT, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMT, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMT, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYET, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYET, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

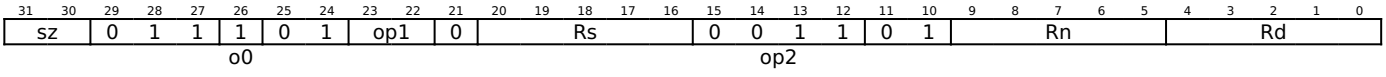
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYET [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPTN, CPYMTN, CPYETN

Memory copy, reads and writes unprivileged and non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPTN, then CPYMTN, and then CPYETN.
CPYPTN performs some preconditioning of the arguments suitable for using the CPYMTN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMTN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYETN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPTN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPTN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPTN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMTN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMTN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYETN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYETN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

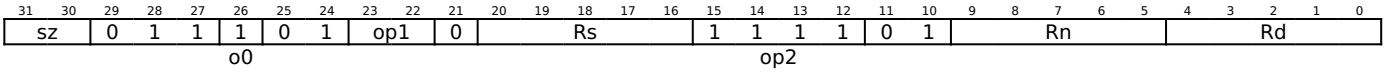
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYETN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPTRN, CPYMTRN, CPYETRN

Memory copy, reads and writes unprivileged, reads non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPTRN, then CPYMTRN, and then CPYETRN.
CPYPTRN performs some preconditioning of the arguments suitable for using the CPYMTRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMTRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYETRN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPTRN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPTRN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPTRN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMTRN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMTRN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

        - Xd holds the lowest address to be copied to.
        - On completion of the instruction:
            - Xn holds the number of bytes remaining to be copied.
            - Xs holds the lowest address that has not been copied from.
            - Xd holds the lowest address that has not been copied to.
    - If the copy is in the backward direction (PSTATE.N == 1), then:
        - Xs holds the highest address to be copied from + 1.
        - Xd holds the highest address to be copied to + 1.
        - On completion of the instruction:
            - Xn holds the number of bytes remaining to be copied.
            - Xs holds the highest address that has not been copied from + 1.
            - Xd holds the highest address that has not been copied to + 1.

For CPYETRN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
    - Xn holds -1 times the number of bytes remaining to be copied.
    - Xs holds the lowest address to be copied from - Xn.
    - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address to be copied from + 1 - Xn.
    - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
    - Xn holds 0.
    - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied from.
        - The highest address that has been copied from + 1.
    - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied to.
        - The highest address that has not been copied to + 1.

For CPYETRN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
    - Xs holds the lowest address to be copied from.
    - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
    - Xs holds the highest address to be copied from + 1.
    - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
    - Xn holds 0.
    - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied from.
        - The highest address that has been copied from + 1.
    - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied to.
        - The highest address that has not been copied to + 1.

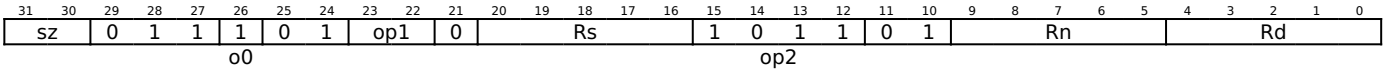
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYETRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPTWN, CPYMTWN, CPYETWN

Memory copy, reads and writes unprivileged, writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPTWN, then CPYMTWN, and then CPYETWN.
CPYPTWN performs some preconditioning of the arguments suitable for using the CPYMTWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMTWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYETWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPTWN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPTWN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPTWN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMTWN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMTWN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYETWN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYETWN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

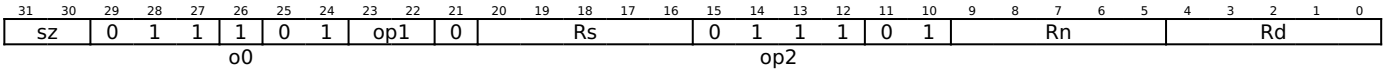
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYETWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPWN, CPYMWN, CPYEWN

Memory copy, writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPWN, then CPYMWN, and then CPYEWN.
CPYPWN performs some preconditioning of the arguments suitable for using the CPYMWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPWN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPWN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPWN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMWN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMWN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYEWN, option A, when PSTATE.C = 0:

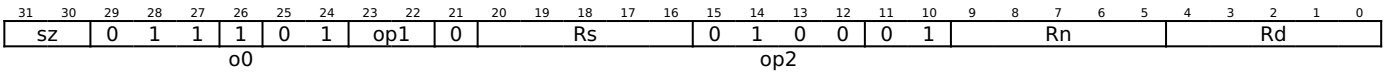
- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYEWN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYEWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPWT, CPYMWT, CPYEWT

Memory copy, writes unprivileged

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPWT, then CPYMWT, and then CPYEWT.
CPYPWT performs some preconditioning of the arguments suitable for using the CPYMWT instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMWT copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEWT copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPWT:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPWT, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPWT, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMWT, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMWT, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYEWT, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYEWT, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

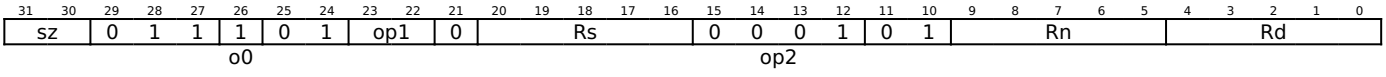
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPWT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMWT [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYEWT [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPWTN, CPYMWTN, CPYEWTN

Memory copy, writes unprivileged, reads and writes non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPWTN, then CPYMWTN, and then CPYEWTN.
CPYPWTN performs some preconditioning of the arguments suitable for using the CPYMWTN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMWTN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEWTN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPWTN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPWTN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPWTN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMWTN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMWTN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.

- Xd holds the lowest address to be copied to.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.

- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
  - On completion of the instruction:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address that has not been copied from + 1.
    - Xd holds the highest address that has not been copied to + 1.

For CPYEWTN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

For CPYEWTN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
  - Xs holds the lowest address to be copied from.
  - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
  - Xs holds the highest address to be copied from + 1.
  - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
  - Xn holds 0.
  - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied from.
    - The highest address that has been copied from + 1.
  - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
    - The lowest address that has been copied to.
    - The highest address that has not been copied to + 1.

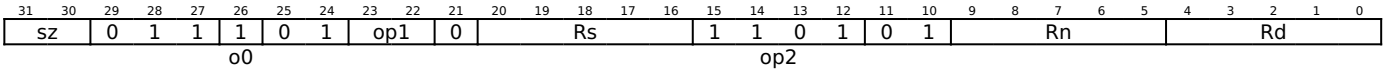
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPWTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMWTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYEWTN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B,
                                                            raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPWTRN, CPYMWTRN, CPYEWTRN

Memory copy, writes unprivileged, reads non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPWTRN, then CPYMWTRN, and then CPYEWTRN.
CPYPWTRN performs some preconditioning of the arguments suitable for using the CPYMWTRN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMWTRN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEWTRN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPWTRN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPWTRN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPWTRN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMWTRN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMWTRN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction (PSTATE.N == 0), then:
    - Xs holds the lowest address to be copied from.
    - Xd holds the lowest address to be copied to.
    - On completion of the instruction:
        - Xn holds the number of bytes remaining to be copied.
        - Xs holds the lowest address that has not been copied from.
        - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
    - Xs holds the highest address to be copied from + 1.
    - Xd holds the highest address to be copied to + 1.
    - On completion of the instruction:
        - Xn holds the number of bytes remaining to be copied.
        - Xs holds the highest address that has not been copied from + 1.
        - Xd holds the highest address that has not been copied to + 1.

For CPYEWTRN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
    - Xn holds -1 times the number of bytes remaining to be copied.
    - Xs holds the lowest address to be copied from - Xn.
    - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address to be copied from + 1 - Xn.
    - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
    - Xn holds 0.
    - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied from.
        - The highest address that has been copied from + 1.
    - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied to.
        - The highest address that has not been copied to + 1.

For CPYEWTRN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
    - Xs holds the lowest address to be copied from.
    - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
    - Xs holds the highest address to be copied from + 1.
    - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
    - Xn holds 0.
    - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied from.
        - The highest address that has been copied from + 1.
    - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied to.
        - The highest address that has not been copied to + 1.

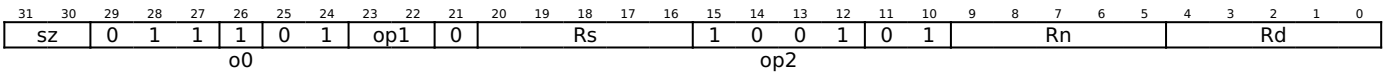
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPWTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMWTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYEWTRN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPYPWTWN, CPYMWTWN, CPYEWTWN

Memory copy, writes unprivileged and non-temporal

These instructions copy a requested number of bytes in memory from a source address to a destination address. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: CPYPWTWN, then CPYMWTWN, and then CPYEWTWN.
CPYPWTWN performs some preconditioning of the arguments suitable for using the CPYMWTWN instruction, and copies an IMPLEMENTATION DEFINED portion of the requested number of bytes. CPYMWTWN copies a further IMPLEMENTATION DEFINED portion of the remaining bytes. CPYEWTWN copies any final remaining bytes.

Note

The ability to copy an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being copied are divided between the different instructions.

For more information on exceptions specific to memory copy instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory copy: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For CPYPWTWN:

- If Xn[63:55] != '000000000', the copy size Xn is saturated to 0x007FFFFFFFFFFF.
- After saturation is performed, the direction of the memory copy is based on the following:
  If (Xs[55:0] > Xd[55:0]) and (Xd[55:0] + saturated copy size) > Xs[55:0], then the direction is forward.
  If (Xs[55:0] < Xd[55:0]) and (Xs[55:0] + saturated copy size) > Xd[55:0], then the direction is backward.
  Otherwise, the direction is an IMPLEMENTATION DEFINED choice between forward and backward.

On completion of CPYPWTWN, option A:

- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.
- If the copy is in the forward direction, then:
  - Xn holds -1 times the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the original Xs + saturated copy size.
  - Xd holds the original Xd + saturated copy size.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs and Xd are unchanged.

On completion of CPYPWTWN, option B:

- If the copy is in the forward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the lowest address that has not been copied from.
  - Xd holds the lowest address that has not been copied to.
  - PSTATE.{N,Z,C,V} are set to {0,0,1,0}.
- If the copy is in the backward direction, then:
  - Xn holds the number of bytes in the saturated copy size remaining to be copied.
  - Xs holds the highest address that has not been copied from + 1.
  - Xd holds the highest address that has not been copied to + 1.
  - PSTATE.{N,Z,C,V} are set to {1,0,1,0}.

For CPYMWTWN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
  - Xn holds -1 times the number of bytes remaining to be copied.
  - Xs holds the lowest address to be copied from - Xn.
  - Xd holds the lowest address to be copied to - Xn.
  - On completion of the instruction, Xn holds -1 times the number of bytes remaining to be copied.
- If the copy is in the backward direction (Xn holds a positive number), then:
  - Xn holds the number of bytes remaining to be copied.
  - Xs holds the highest address to be copied from + 1 - Xn.
  - Xd holds the highest address to be copied to + 1 - Xn.
  - On completion of the instruction, Xn holds the number of bytes remaining to be copied.

For CPYMWTWN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.

- If the copy is in the forward direction (PSTATE.N == 0), then:
    - Xs holds the lowest address to be copied from.
    - Xd holds the lowest address to be copied to.
    - On completion of the instruction:
        - Xn holds the number of bytes remaining to be copied.
        - Xs holds the lowest address that has not been copied from.
        - Xd holds the lowest address that has not been copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
    - Xs holds the highest address to be copied from + 1.
    - Xd holds the highest address to be copied to + 1.
    - On completion of the instruction:
        - Xn holds the number of bytes remaining to be copied.
        - Xs holds the highest address that has not been copied from + 1.
        - Xd holds the highest address that has not been copied to + 1.

For CPYEWTWN, option A, when PSTATE.C = 0:

- Xn holds a signed 64-bit integer.
- If the copy is in the forward direction (Xn holds a negative number), then:
    - Xn holds -1 times the number of bytes remaining to be copied.
    - Xs holds the lowest address to be copied from - Xn.
    - Xd holds the lowest address to be copied to - Xn.
- If the copy is in the backward direction (Xn holds a positive number), then:
    - Xn holds the number of bytes remaining to be copied.
    - Xs holds the highest address to be copied from + 1 - Xn.
    - Xd holds the highest address to be copied to + 1 - Xn.
- On completion of the instruction:
    - Xn holds 0.
    - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied from.
        - The highest address that has been copied from + 1.
    - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied to.
        - The highest address that has not been copied to + 1.

For CPYEWTWN, option B, when PSTATE.C = 1:

- Xn holds the number of bytes remaining to be copied.
- If the copy is in the forward direction (PSTATE.N == 0), then:
    - Xs holds the lowest address to be copied from.
    - Xd holds the lowest address to be copied to.
- If the copy is in the backward direction (PSTATE.N == 1), then:
    - Xs holds the highest address to be copied from + 1.
    - Xd holds the highest address to be copied to + 1.
- On completion of the instruction:
    - Xn holds 0.
    - Xs holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied from.
        - The highest address that has been copied from + 1.
    - Xd holds an IMPLEMENTATION SPECIFIC value in the range with bounds:
        - The lowest address that has been copied to.
        - The highest address that has not been copied to + 1.

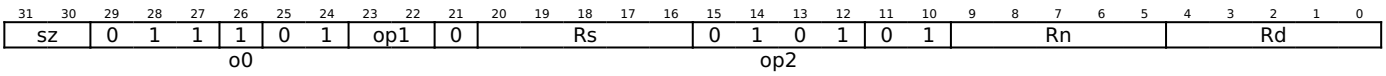
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op1 == 00)`

```
CPYPWTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Main variant

Applies when `(op1 == 01)`

```
CPYMWTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Encoding for the Epilogue variant

Applies when `(op1 == 10)`

```
CPYEWTWN [<Xd>]!, [<Xs>]!, <Xn>!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memcpy : CPYParams;
memcpy.d = UInt(Rd);
memcpy.s = UInt(Rs);
memcpy.n = UInt(Rn);
let options : bits(4) = op2;
let rnontemporal : boolean = options[3] == '1';
let wnontemporal : boolean = options[2] == '1';
case op1 of
    when '00' => memcpy.stage = MOPSStage_Prologue;
    when '01' => memcpy.stage = MOPSStage_Main;
    when '10' => memcpy.stage = MOPSStage_Epilogue;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set CPY** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address, encoded in the "Rd" field.
<Xs>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the source address and is updated by the instruction, encoded in the "Rs" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the source address, encoded in the "Rs" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be transferred and is updated by the instruction to encode the remaining size and destination, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be transferred and is set to zero on completion of the instruction, encoded in the "Rn" field.

Operation

```
CheckMOPSEnabled();

CheckCPYConstrainedUnpredictable(memcpy.n, memcpy.d, memcpy.s);

memcpy.nzcv        = PSTATE.[N,Z,C,V];
memcpy.toaddress   = X{64}(memcpy.d);
memcpy.fromaddress = X{64}(memcpy.s);

if memcpy.stage == MOPSStage_Prologue then
    memcpy.cpysize = UInt(X{64}(memcpy.n));
else
    memcpy.cpysize = SInt(X{64}(memcpy.n));
end;

memcpy.implements_option_a = CPYOptionA();

let rprivileged : boolean = (if options[1] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);
let wprivileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                             else PSTATE.EL != EL0);

let raccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_LOAD,  rprivileged, rnontemporal);
let waccdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, wprivileged, wnontemporal);

if memcpy.stage == MOPSStage_Prologue then
    if memcpy.cpysize > ArchMaxMOPSCPYSize then
        memcpy.cpysize = ArchMaxMOPSCPYSize;
    end;

    memcpy.forward = IsMemCpyForward(memcpy);

    if memcpy.implements_option_a then
        memcpy.nzcv = '0000';
        if memcpy.forward then
            // Copy in the forward direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.cpysize     = 0 - memcpy.cpysize;
        end;
    else
        if !memcpy.forward then
            // Copy in the reverse direction offsets the arguments.
            memcpy.toaddress   = memcpy.toaddress   + memcpy.cpysize;
            memcpy.fromaddress = memcpy.fromaddress + memcpy.cpysize;
            memcpy.nzcv = '1010';
        else
            memcpy.nzcv = '0010';
        end;
    end;
end;

memcpy.stagecpysize = MemCpyStageSize(memcpy);

if memcpy.stage != MOPSStage_Prologue then
    memcpy.forward = (memcpy.cpysize < 0 ||
                      (!memcpy.implements_option_a && memcpy.nzcv[3] == '0'));
    CheckMemCpyParams(memcpy, options);
end;

var copied : integer;
var iswrite : boolean;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var fault : boolean = FALSE;
var B : MOPSBlockSize = 0;

if memcpy.implements_option_a then
    while memcpy.stagecpysize != 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);

        if memcpy.forward then
            assert B <= -1 * memcpy.stagecpysize;
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                        memcpy.toaddress + memcpy.cpysize,
                                                        memcpy.fromaddress + memcpy.cpysize,
                                                        memcpy.forward, B,
                                                        raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;

        else
            assert B <= memcpy.stagecpysize;
            memcpy.cpysize      = memcpy.cpysize      - B;
```

```
            memcpy.stagecpysize = memcpy.stagecpysize - B;

            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(
                                                            memcpy.toaddress   + memcpy.cpysize,
                                                            memcpy.fromaddress + memcpy.cpysize,
                                                            memcpy.forward, B, raccdesc,
                                                            waccdesc);
            if copied != B then
                fault               = TRUE;
                memcpy.cpysize      = memcpy.cpysize      + B;
                memcpy.stagecpysize = memcpy.stagecpysize + B;
            end;
        end;
    end;

else
    while memcpy.stagecpysize > 0 && !fault looplimit ArchMaxMOPSCPYSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = CPYSizeChoice(memcpy);
        assert B <= memcpy.stagecpysize;

        if memcpy.forward then
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress,
                                                                    memcpy.fromaddress,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);
            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress + B;
                memcpy.toaddress   = memcpy.toaddress   + B;
            end;
        else
            (copied, iswrite, memaddrdesc, memstatus) = MemCpyBytes(memcpy.toaddress   - B,
                                                                    memcpy.fromaddress - B,
                                                                    memcpy.forward, B,
                                                                    raccdesc, waccdesc);

            if copied != B then
                fault = TRUE;
            else
                memcpy.fromaddress = memcpy.fromaddress - B;
                memcpy.toaddress   = memcpy.toaddress   - B;
            end;
        end;

        if !fault then
            memcpy.cpysize      = memcpy.cpysize      - B;
            memcpy.stagecpysize = memcpy.stagecpysize - B;
        end;
    end;
end;

UpdateCpyRegisters(memcpy, fault, copied);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let accdesc : AccessDescriptor = if iswrite then waccdesc else raccdesc;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memcpy.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memcpy.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CRC32B, CRC32H, CRC32W, CRC32X

CRC32 checksum

This instruction performs a cyclic redundancy check (CRC) calculation on a value held in a general-purpose register. It takes an input CRC value in the first source operand, performs a CRC on the input value in the second source operand, and returns the output CRC value. The second source operand can be 8, 16, 32, or 64 bits. To align with common usage, the bit order of the values is reversed as part of the operation, and the polynomial 0x04C11DB7 is used for the CRC calculation.

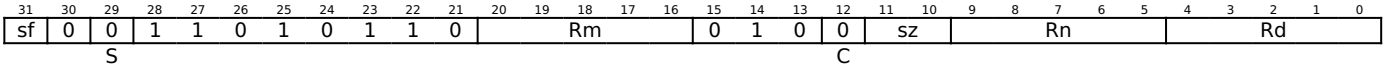
In an Armv8.0 implementation, this is an OPTIONAL instruction. From Armv8.1, it is mandatory for all implementations to implement this instruction.

Note

ID_AA64ISAR0_EL1.CRC32 indicates whether this instruction is supported.

CRC

(FEAT_CRC32)



Encoding for the CRC32B variant

Applies when `(sf == 0 && sz == 00)`

```
CRC32B <Wd>, <Wn>, <Wm>
```

Encoding for the CRC32H variant

Applies when `(sf == 0 && sz == 01)`

```
CRC32H <Wd>, <Wn>, <Wm>
```

Encoding for the CRC32W variant

Applies when `(sf == 0 && sz == 10)`

```
CRC32W <Wd>, <Wn>, <Wm>
```

Encoding for the CRC32X variant

Applies when `(sf == 1 && sz == 11)`

```
CRC32X <Wd>, <Wn>, <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CRC32) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
if sf == '1' && sz != '11' then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && sz == '11' then EndOfDecode(Decode_UNDEF); end;
let size : integer{} = 8 << UInt(sz);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose accumulator output register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose accumulator input register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the general-purpose data source register, encoded in the "Rm" field.
<Xm>	Is the 64-bit name of the general-purpose data source register, encoded in the "Rm" field.

Operation

```
let acc : bits(32) = X{32}(n);       // accumulator
let val : bits(size) = X{size}(m);   // input value
let poly : bits(32) = 0x4C11DB7[:32];

let tempacc : bits(32+size) = BitReverse{32}(acc)::Zeros{size};
let tempval : bits(size+32) = BitReverse{size}(val)::Zeros{32};

// Poly32Mod2 on a bitstring does a polynomial Modulus over {0,1} operation
X{32}(d) = BitReverse{32}(Poly32Mod2{32+size}(tempacc XOR tempval, poly));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CRC32CB, CRC32CH, CRC32CW, CRC32CX

CRC32C checksum

This instruction performs a cyclic redundancy check (CRC) calculation on a value held in a general-purpose register. It takes an input CRC value in the first source operand, performs a CRC on the input value in the second source operand, and returns the output CRC value. The second source operand can be 8, 16, 32, or 64 bits. To align with common usage, the bit order of the values is reversed as part of the operation, and the polynomial 0x1EDC6F41 is used for the CRC calculation.

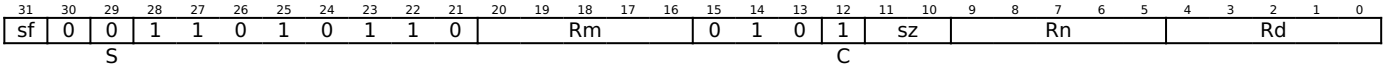
In an Armv8.0 implementation, this is an OPTIONAL instruction. From Armv8.1, it is mandatory for all implementations to implement this instruction.

Note

ID_AA64ISAR0_EL1.CRC32 indicates whether this instruction is supported.

CRC

(FEAT_CRC32)



Encoding for the CRC32CB variant

Applies when `(sf == 0 && sz == 00)`

```
CRC32CB <Wd>, <Wn>, <Wm>
```

Encoding for the CRC32CH variant

Applies when `(sf == 0 && sz == 01)`

```
CRC32CH <Wd>, <Wn>, <Wm>
```

Encoding for the CRC32CW variant

Applies when `(sf == 0 && sz == 10)`

```
CRC32CW <Wd>, <Wn>, <Wm>
```

Encoding for the CRC32CX variant

Applies when `(sf == 1 && sz == 11)`

```
CRC32CX <Wd>, <Wn>, <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CRC32) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
if sf == '1' && sz != '11' then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && sz == '11' then EndOfDecode(Decode_UNDEF); end;
let size : integer{} = 8 << UInt(sz);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose accumulator output register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose accumulator input register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the general-purpose data source register, encoded in the "Rm" field.
<Xm>	Is the 64-bit name of the general-purpose data source register, encoded in the "Rm" field.

Operation

```
let acc : bits(32) = X{32}(n);       // accumulator
let val : bits(size) = X{size}(m);   // input value
let poly : bits(32) = 0x1EDC6F41[:32];

let tempacc : bits(32+size) = BitReverse{32}(acc)::Zeros{size};
let tempval : bits(size+32) = BitReverse{size}(val)::Zeros{32};

// Poly32Mod2 on a bitstring does a polynomial Modulus over {0,1} operation
X{32}(d) = BitReverse{32}(Poly32Mod2{32+size}(tempacc XOR tempval, poly));
```

Operational information

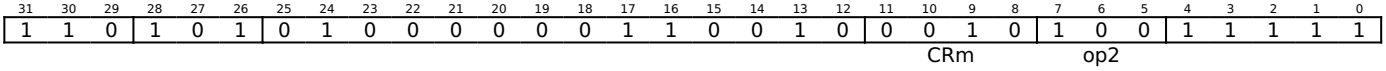
This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CSDB

Consumption of speculative data barrier

This instruction is a memory barrier that controls speculative execution arising from data value prediction. For more information and details of the semantics, see *Consumption of Speculative Data Barrier (CSDB)*.



Encoding

CSDB

Decode for this encoding

```
// Empty.
```

Operation

```
ConsumptionOfSpeculativeDataBarrier();
```

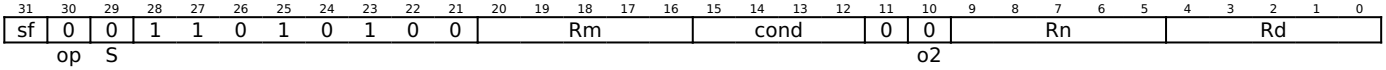
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CSEL

Conditional select

This instruction writes the value of the first source register to the destination register if the condition is TRUE. If the condition is FALSE, it writes the value of the second source register to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSEL <Wd>, <Wn>, <Wm>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSEL <Xd>, <Xn>, <Xm>, <cond>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
var result : bits(datasize);
if ConditionHolds(condition) then
    result = X{datasize}(n);
else
    result = X{datasize}(m);
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

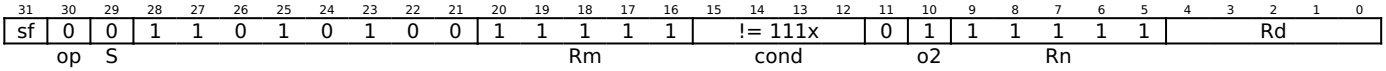
CSET

Conditional set

This instruction sets the destination register to 1 if the condition is TRUE, and otherwise sets it to 0.

This is an alias of CSINC. This means:

- The encodings in this description are named to match the encodings of CSINC.
- The description of CSINC gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSET <Wd>, <invcond>
```

is equivalent to

```
CSINC <Wd>, WZR, WZR, <cond>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSET <Xd>, <invcond>
```

is equivalent to

```
CSINC <Xd>, XZR, XZR, <cond>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<invcond> Is one of the standard conditions, excluding AL and NV, encoded with its least significant bit inverted, and encoded in “cond”:

cond	<invcond>	Description
0000	NE	Maps to <cond> EQ.
0001	EQ	Maps to <cond> NE.
0010	CC|LO	Maps to <cond> CS.
0011	CS|HS	Maps to <cond> CC.
0100	PL	Maps to <cond> MI.
0101	MI	Maps to <cond> PL.
0110	VC	Maps to <cond> VS.
0111	VS	Maps to <cond> VC.
1000	LS	Maps to <cond> HI.
1001	HI	Maps to <cond> LS.
1010	LT	Maps to <cond> GE.
1011	GE	Maps to <cond> LT.
1100	LE	Maps to <cond> GT.
1101	GT	Maps to <cond> LE.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

The description of CSINC gives the operational pseudocode for this instruction.

Operational Information

The description of CSINC gives the operational information for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

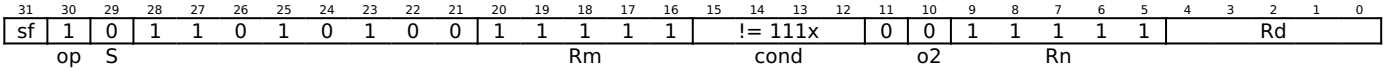
CSETM

Conditional set mask

This instruction sets all bits of the destination register to 1 if the condition is TRUE, and otherwise sets all bits to 0.

This is an alias of CSINV. This means:

- The encodings in this description are named to match the encodings of CSINV.
- The description of CSINV gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSETM <Wd>, <invcond>
```

is equivalent to

```
CSINV <Wd>, WZR, WZR, <cond>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSETM <Xd>, <invcond>
```

is equivalent to

```
CSINV <Xd>, XZR, XZR, <cond>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<invcond> Is one of the standard conditions, excluding AL and NV, encoded with its least significant bit inverted, and encoded in “cond”:

cond	<invcond>	Description
0000	NE	Maps to <cond> EQ.
0001	EQ	Maps to <cond> NE.
0010	CC|LO	Maps to <cond> CS.
0011	CS|HS	Maps to <cond> CC.
0100	PL	Maps to <cond> MI.
0101	MI	Maps to <cond> PL.
0110	VC	Maps to <cond> VS.
0111	VS	Maps to <cond> VC.
1000	LS	Maps to <cond> HI.
1001	HI	Maps to <cond> LS.
1010	LT	Maps to <cond> GE.
1011	GE	Maps to <cond> LT.
1100	LE	Maps to <cond> GT.
1101	GT	Maps to <cond> LE.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

The description of CSINV gives the operational pseudocode for this instruction.

Operational Information

The description of CSINV gives the operational information for this instruction.

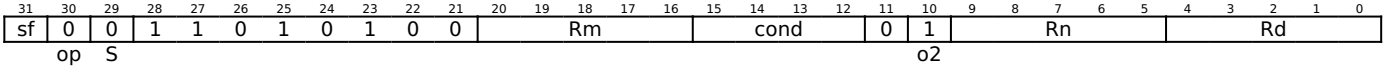
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CSINC

Conditional select increment

This instruction returns, in the destination register, the value of the first source register if the condition is TRUE, and otherwise returns the value of the second source register incremented by 1.

This instruction is used by the aliases CINC, and CSET.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSINC <Wd>, <Wn>, <Wm>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSINC <Xd>, <Xn>, <Xm>, <cond>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CINC	`Rm != '11111' && Rn != '11111' && !(cond IN {'111x'}) && Rn == Rm`
CSET	`Rm == '11111' && Rn == '11111' && !(cond IN {'111x'})`

Operation

```
var result : bits(datasize);
if ConditionHolds(condition) then
    result = X{datasize}(n);
else
    result = X{datasize}(m) + 1;
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

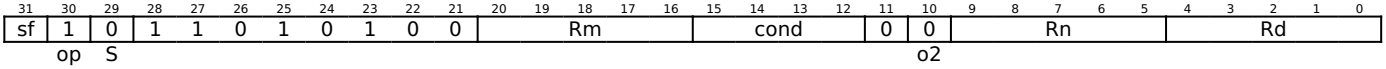
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CSINV

Conditional select invert

This instruction returns, in the destination register, the value of the first source register if the condition is TRUE, and otherwise returns the bitwise inversion value of the second source register.

This instruction is used by the aliases CINV, and CSETM.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSINV <Wd>, <Wn>, <Wm>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSINV <Xd>, <Xn>, <Xm>, <cond>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CINV	`Rm != '11111' && Rn != '11111' && !(cond IN {'111x'}) && Rn == Rm`
CSETM	`Rm == '11111' && Rn == '11111' && !(cond IN {'111x'})`

Operation

```
var result : bits(datasize);
if ConditionHolds(condition) then
    result = X{datasize}(n);
else
    result = NOT X{datasize}(m);
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

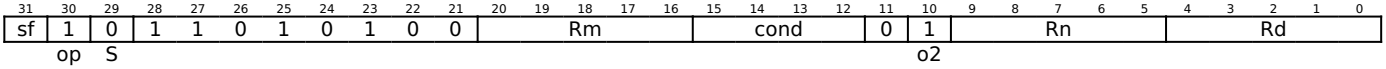
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CSNEG

Conditional select negation

This instruction returns, in the destination register, the value of the first source register if the condition is TRUE, and otherwise returns the negated value of the second source register.

This instruction is used by the alias CNEG.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CSNEG <Wd>, <Wn>, <Wm>, <cond>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CSNEG <Xd>, <Xn>, <Xm>, <cond>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let condition : bits(4) = cond;
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CNEG	`!(cond IN {'111x'}) && Rn == Rm`

Operation

```
var result : bits(datasize);
if ConditionHolds(condition) then
    result = X{datasize}(n);
else
    result = NOT X{datasize}(m) + 1;
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

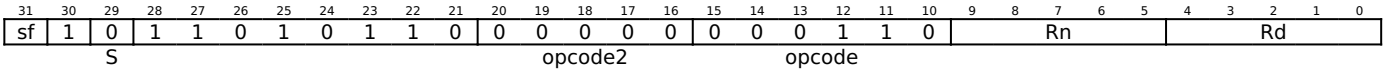
CTZ

Count trailing zeros

This instruction determines the number of consecutive binary zero bits, starting from the least significant bit in the source register, and places the result in the destination register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
CTZ <Wd>, <Wn>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
CTZ <Xd>, <Xn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let datasize : integer{} = 32 << UInt(sf);
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let result : integer = CountLeadingZeroBits(BitReverse{datasize}(operand1));

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

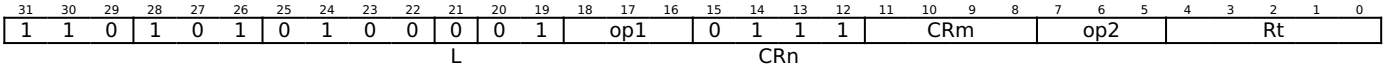
DC

Data cache operation

For more information, see *op0 == 0b01, cache maintenance, TLB maintenance, address translation, prediction restriction, BRBE, Trace Extension, Guarded Control Stack, and FEAT_RME_GPC3 instructions*.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

```
DC <dc_op>, <Xt>
```

is equivalent to

```
SYS #<op1>, C7, <Cm>, #<op2>, <Xt>
```

and is the preferred disassembly when `SysOp(op1, '0111', CRm, op2) == Sys_DC`.

Assembler Symbols

<dc_op> Is a DC operation name, as listed for the DC system instruction group, encoded in “(op1 :: CRm :: op2)”:

op1	CRm	op2	<dc_op>	Architectural Feature	Description
000	0110	001	IVAC	-	For more information, see DC IVAC.
000	0110	010	ISW	-	For more information, see DC ISW.
000	0110	011	IGVAC	FEAT_MTE2	For more information, see DC IGVAC.
000	0110	100	IGSW	FEAT_MTE2	For more information, see DC IGSW.
000	0110	101	IGDVAC	FEAT_MTE2	For more information, see DC IGDVAC.
000	0110	110	IGDSW	FEAT_MTE2	For more information, see DC IGDSW.
000	1010	010	CSW	-	For more information, see DC CSW.
000	1010	100	CGSW	FEAT_MTE2	For more information, see DC CGSW.
000	1010	110	CGDSW	FEAT_MTE2	For more information, see DC CGDSW.
000	1110	010	CISW	-	For more information, see DC CISW.
000	1110	100	CIGSW	FEAT_MTE2	For more information, see DC CIGSW.
000	1110	110	CIGDSW	FEAT_MTE2	For more information, see DC CIGDSW.
000	1111	001	CIVAPS	FEAT_PoPS	For more information, see DC CIVAPS.
000	1111	101	CIGDVAPS	FEAT_PoPS && FEAT_MTE2	For more information, see DC CIGDVAPS.
011	0100	001	ZVA	-	For more information, see DC ZVA.
011	0100	011	GVA	FEAT_MTE	For more information, see DC GVA.
011	0100	100	GZVA	FEAT_MTE	For more information, see DC GZVA.
011	0100	101	ZGBVA	FEAT_MTETC	For more information, see DC ZGBVA.
011	0100	111	GBVA	FEAT_MTETC	For more information, see DC GBVA.
011	1010	001	CVAC	-	For more information, see DC CVAC.
011	1010	011	CGVAC	FEAT_MTE	For more information, see DC CGVAC.
011	1010	101	CGDVAC	FEAT_MTE	For more information, see DC CGDVAC.
011	1011	000	CVAOC	FEAT_OCCMO	For more information, see DC CVAOC.
011	1011	001	CVAU	-	For more information, see DC CVAU.
011	1011	111	CGDVAOC	FEAT_OCCMO && FEAT_MTE	For more information, see DC CGDVAOC.
011	1100	001	CVAP	FEAT_DPB	For more information, see DC CVAP.
011	1100	011	CGVAP	FEAT_MTE	For more information, see DC CGVAP.
011	1100	101	CGDVAP	FEAT_MTE	For more information, see DC CGDVAP.
011	1101	001	CVADP	FEAT_DPB2	For more information, see DC CVADP.
011	1101	011	CGVADP	FEAT_MTE	For more information, see DC CGVADP.
011	1101	101	CGDVADP	FEAT_MTE	For more information, see DC CGDVADP.
011	1110	001	CIVAC	-	For more information, see DC CIVAC.
011	1110	011	CIGVAC	FEAT_MTE	For more information, see DC CIGVAC.
011	1110	101	CIGDVAC	FEAT_MTE	For more information, see DC CIGDVAC.

op1	CRm	op2	<dc_op>	Architectural Feature	Description
011	1111	000	CIVAOC	FEAT_OCCMO	For more information, see DC CIVAOC.
011	1111	111	CIGDVAOC	FEAT_OCCMO && FEAT_MTE	For more information, see DC CIGDVAOC.
100	1110	000	CIPAE	FEAT_MEC	For more information, see DC CIPAE.
100	1110	111	CIGDPAE	FEAT_MEC && FEAT_MTE2	For more information, see DC CIGDPAE.
110	1110	001	CIPAPA	FEAT_RME	For more information, see DC CIPAPA.
110	1110	101	CIGDPAPA	FEAT_RME && FEAT_MTE2	For more information, see DC CIGDPAPA.

<Xt> Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DCPS1

Debug change PE state to EL1

This instruction, when executed in Debug state:

- If executed at EL0, changes the current Exception level and SP to EL1 using SP_EL1.
- Otherwise, if executed at ELx, selects *SP_ELx*.

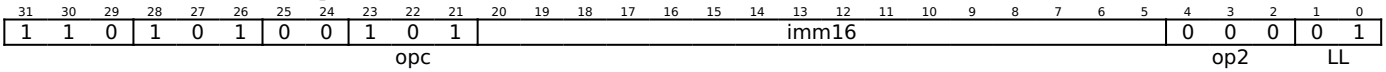
The target Exception level of a DCPS1 instruction is:

- EL1 if the instruction is executed at EL0.
- Otherwise, the Exception level at which the instruction is executed.

When the target Exception level of a DCPS1 instruction is ELx, on executing this instruction:

- *ELR_ELx* becomes UNKNOWN.
- *SPSR_ELx* becomes UNKNOWN.
- *ESR_ELx* becomes UNKNOWN.
- DLR_EL0 and DSPSR_EL0 become UNKNOWN.
- The endianness is set according to *SCTLR_ELx*.EE.

This instruction is always UNDEFINED in Non-debug state.
This instruction is UNDEFINED at EL0 if EL2 is implemented and enabled in the current Security state and *HCR_EL2*.TGE == 1.
For more information on the operation of the DCPS<n> instructions, see *DCPS*.



Encoding

```
DCPS1 {#<imm>}
```

Decode for this encoding

```
// Empty.
```

Assembler Symbols

<imm>	Is an optional 16-bit unsigned immediate, in the range 0 to 65535, defaulting to 0 and encoded in the "imm16" field.

Operation

```
if !Halted() then Undefined(); end;
DCPSInstruction(EL1);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DCPS2

Debug change PE state to EL2

This instruction, when executed in Debug state:

- If executed at EL0 or EL1, changes the current Exception level and SP to EL2 using SP_EL2.
- Otherwise, if executed at ELx, selects *SP_ELx*.

The target Exception level of a DCPS2 instruction is:

- EL2 if the instruction is executed at an Exception level that is not EL3.
- EL3 if the instruction is executed at EL3.

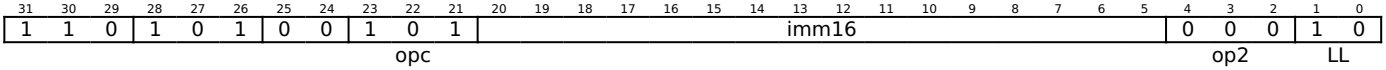
When the target Exception level of a DCPS2 instruction is ELx, on executing this instruction:

- *ELR_ELx* becomes UNKNOWN.
- *SPSR_ELx* becomes UNKNOWN.
- *ESR_ELx* becomes UNKNOWN.
- DLR_EL0 and DSPSR_EL0 become UNKNOWN.
- The endianness is set according to *SCTLR_ELx*.EE.

This instruction is always UNDEFINED in Non-debug state.
This instruction is UNDEFINED at the following Exception levels:

- All Exception levels if EL2 is not implemented.
- At EL0 and EL1 if EL2 is disabled in the current Security state.

For more information on the operation of the DCPS<n> instructions, see *DCPS*.



Encoding

```
DCPS2 {#<imm>}
```

Decode for this encoding

```
// Empty.
```

Assembler Symbols

<imm>	Is an optional 16-bit unsigned immediate, in the range 0 to 65535, defaulting to 0 and encoded in the "imm16" field.

Operation

```
if !Halted() then Undefined(); end;
DCPSInstruction(EL2);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DCPS3

Debug change PE state to EL3

This instruction, when executed in Debug state:

- If executed at EL3, selects SP_EL3.
- Otherwise, changes the current Exception level and SP to EL3 using SP_EL3.

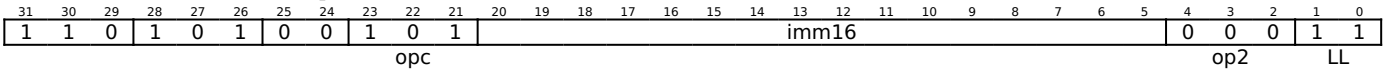
The target Exception level of a DCPS3 instruction is EL3.
On executing a DCPS3 instruction:

- ELR_EL3 becomes UNKNOWN.
- SPSR_EL3 becomes UNKNOWN.
- ESR_EL3 becomes UNKNOWN.
- DLR_EL0 and DSPSR_EL0 become UNKNOWN.
- The endianness is set according to SCTLR_EL3.EE.

This instruction is always UNDEFINED in Non-debug state.
This instruction is UNDEFINED at all Exception levels if either:

- *EDSCR*.SDD == 1.
- EL3 is not implemented.

For more information on the operation of the DCPS<n> instructions, see *DCPS*.



Encoding

```
DCPS3 {#<imm>}
```

Decode for this encoding

```
// Empty.
```

Assembler Symbols

<imm>	Is an optional 16-bit unsigned immediate, in the range 0 to 65535, defaulting to 0 and encoded in the "imm16" field.

Operation

```
if !Halted() then Undefined(); end;
DCPSInstruction(EL3);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

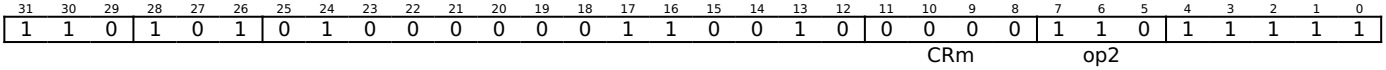
DGH

Data gathering hint

This instruction is a hint instruction that indicates that it is not expected to be performance optimal to merge memory accesses with Normal Non-cacheable or Device-GRE attributes appearing in program order before the hint instruction with any memory accesses appearing after the hint instruction into a single memory transaction on an interconnect.

System

(FEAT_DGH)



Encoding

DGH

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_DGH) then EndOfDecode(Decode_NOP); end;
```

Operation

```
Hint_DGH();
```

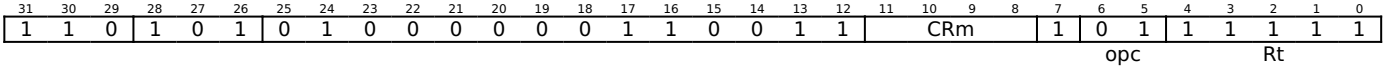
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DMB

Data memory barrier

This instruction is a memory barrier that ensures the ordering of observations of memory accesses, see *Data Memory Barrier*.



Encoding

```
DMB  (<option>|#<imm>)
```

Decode for this encoding

```
var types : MBReqTypes;
case CRm[1:0] of
    when '00' => types = MBReqTypes_All;
    when '01' => types = MBReqTypes_Reads;
    when '10' => types = MBReqTypes_Writes;
    when '11' => types = MBReqTypes_All;
end;
```

Assembler Symbols

<option> Specifies the limitation on the Memory Effects ordered by the barrier operation, and is encoded in "CRm". For more information on whether a Memory Effect is before or after a barrier instruction, see *Data Memory Barrier (DMB)*.

CRm	<option>	Description
xx00	RESERVED	
0001	OSHLD	This option has the same behavior as LD and is deprecated.
0010	OSHST	This option has the same behavior as ST and is deprecated.
0011	OSH	This option has the same behavior as SY and is deprecated.
0101	NSHLD	This option has the same behavior as LD and is deprecated.
0110	NSHST	This option has the same behavior as ST and is deprecated.
0111	NSH	This option has the same behavior as SY and is deprecated.
1001	ISHLD	This option has the same behavior as LD and is deprecated.
1010	ISHST	This option has the same behavior as ST and is deprecated.
1011	ISH	This option has the same behavior as SY and is deprecated.
1101	LD	Reads are the required access type before the barrier instruction. Reads and writes are the required access types after the barrier instruction.
1110	ST	Writes are the required access type, both before and after the barrier instruction.
1111	SY	Reads and writes are the required access types, both before and after the barrier instruction.

All encodings of "CRm" that are listed as reserved can be encoded using the #<imm> syntax. All reserved options have the same behavior as SY, but software must not rely on this behavior.

<imm> Is a 4-bit unsigned immediate, in the range 0 to 15, encoded in the "CRm" field.

Operation

```
DataMemoryBarrier(types);
```

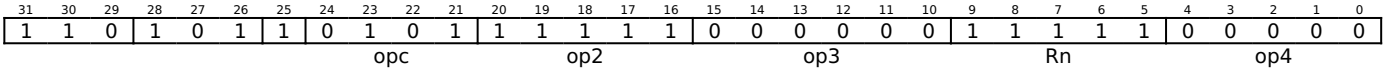
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DRPS

Debug restore PE state

This instruction restores *PSTATE* from the SPSR.
The SPSR is checked for the current Exception level for an illegal return event. See *Illegal exception returns from AArch64 state*.
This instruction is UNDEFINED in Non-debug state.
This instruction is UNDEFINED at EL0.
For more information on the operation of DRPS, see *DRPS*.



Encoding

DRPS

Decode for this encoding

```
// Empty.
```

Operation

```
if !Halted() || PSTATE.EL == EL0 then Undefined(); end;
DRPSInstruction();
```

2026-09_rel 2026-09-21 14:22:38

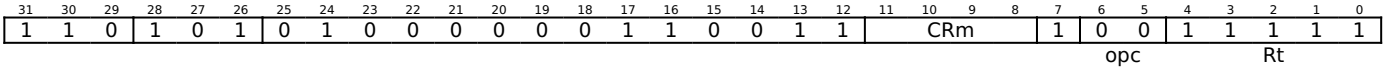
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DSB

Data synchronization barrier

This instruction is a memory barrier that ensures the completion of memory accesses, see *Data Synchronization Barrier*.
This instruction is used by the aliases PSSBB, and SSBB.
It has encodings from 2 classes: Memory barrier and Memory nXS barrier

Memory barrier



Encoding

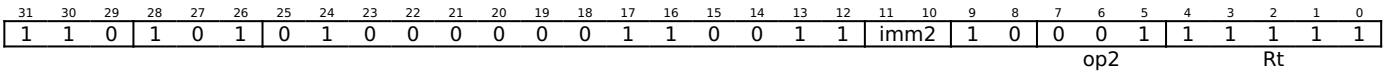
```
DSB (<option>|#<imm>)
```

Decode for this encoding

```
var types : MBReqTypes;
var nXS : boolean = FALSE;
var alias : DSBAlias;
var scope : MBMaintenanceScope;
case CRm of
    when '0000' => alias = DSBAlias_SSBB;
    when '0100' => alias = DSBAlias_PSSBB;
    otherwise   => alias = DSBAlias_DSB;
end;

case CRm[1:0] of
    when '00' =>
        types = MBReqTypes_All;
        scope = MBMaintenanceScope_OuterShareable;
    when '01' =>
        types = MBReqTypes_Reads;
        scope = MBMaintenanceScope_None;
    when '10' =>
        types = MBReqTypes_Writes;
        scope = MBMaintenanceScope_None;
    when '11' =>
        types = MBReqTypes_All;
        case CRm[3:2] of
            when '00' => scope = MBMaintenanceScope_OuterShareable;
            when '01' => scope = MBMaintenanceScope_Nonshareable;
            when '10' => scope = MBMaintenanceScope_InnerShareable;
            when '11' => scope = MBMaintenanceScope_OuterShareable;
        end;
end;
```

Memory nXS barrier

(FEAT_XS)



Encoding

```
DSB <option>nXS
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_XS) then EndOfDecode(Decode_UNDEF); end;
let types : MBReqTypes = MBReqTypes_All;
var nXS : boolean = TRUE;
let alias : DSBAlias = DSBAlias_DSB;
var scope : MBMaintenanceScope;
case imm2 of
    when '00' => scope = MBMaintenanceScope_OuterShareable;
    when '01' => scope = MBMaintenanceScope_Nonshareable;
    when '10' => scope = MBMaintenanceScope_InnerShareable;
    when '11' => scope = MBMaintenanceScope_OuterShareable;
end;
```

Assembler Symbols

<option> For the "Memory barrier" variant: specifies the limitation on the Memory Effects and maintenance operations ordered by the barrier operation, and is encoded in "CRm".

Note

The value 0b0000 is used to encode `SSBB` and the value 0b0100 is used to encode `PSSBB`.

For more information on whether an access is before or after a barrier instruction, see *Data Synchronization Barrier (DSB)*.

CRm	<option>	Description
0001	OSHLD	This option has the same behavior as LD and is deprecated.
0010	OSHST	This option has the same behavior as ST and is deprecated.
0011	OSH	This option has the same behavior as SY.
0101	NSHLD	This option has the same behavior as LD and is deprecated.
0110	NSHST	This option has the same behavior as ST and is deprecated.
0111	NSH	Non-shareable is the required maintenance scope. Reads and writes are the required access types, both before and after the barrier instruction.
1x00	RESERVED	
1001	ISHLD	This option has the same behavior as LD and is deprecated.
1010	ISHST	This option has the same behavior as ST and is deprecated.
1011	ISH	Inner Shareable is the required maintenance scope. Reads and writes are the required access types, both before and after the barrier instruction.
1101	LD	Reads are the required access type before the barrier instruction. Reads and writes are the required access types after the barrier instruction.
1110	ST	Writes are the required access type, both before and after the barrier instruction.
1111	SY	Outer Shareable is the required maintenance scope. Reads and writes are the required access types, both before and after the barrier instruction.

All encodings of "CRm" that are listed as reserved can be encoded using the #<imm> syntax. All reserved options have the same behavior as SY, but software must not rely on this behavior.

For the "Memory nXS barrier" variant: specifies the limitation on the barrier operation, and is encoded in “imm2”:

imm2	<option>	Description
00	OSH	This option has the same behavior as SY.
01	NSH	Non-shareable is the required maintenance scope. Reads and writes are the required access types, both before and after the barrier instruction.
10	ISH	Inner Shareable is the required maintenance scope. Reads and writes are the required access types, both before and after the barrier instruction.
11	SY	Outer Shareable is the required maintenance scope. Reads and writes are the required access types, both before and after the barrier instruction.

<imm> Is a 4-bit unsigned immediate, in the range 0 to 15, encoded in the "CRm" field.

Alias Conditions

Alias	Is preferred when
PSSBB	`CRm == '0100'`
SSBB	`CRm == '0000'`

Operation

```
case alias of
    when DSBAlias_SSBB =>
        SpeculativeStoreBypassBarrierToVA();
    when DSBAlias_PSSBB =>
        SpeculativeStoreBypassBarrierToPA();
    when DSBAlias_DSB =>
        if !nXS && IsFeatureImplemented(FEAT_XS) then
            nXS = PSTATE.EL IN {EL0, EL1} && IsHCRXEL2Enabled() && HCRX_EL2().FnXS == '1';
        end;
        DataSynchronizationBarrier(scope, types, nXS);
    otherwise =>
        unreachable;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DVP

Data value prediction restriction by context

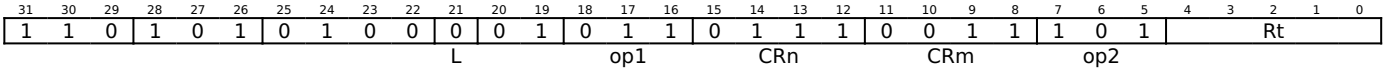
This instruction prevents data value predictions that predict execution addresses based on information gathered from earlier execution within a particular execution context. Data value predictions determined by the actions of code in the target execution context or contexts appearing in program order before the instruction cannot be used to exploitatively control speculative execution occurring after the instruction is complete and synchronized.
For more information, see DVP RCTX, Data Value Prediction Restriction by Context.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_SPECRES)



Encoding

```
DVP RCTX, <Xt>
```

is equivalent to

```
SYS #3, C7, C3, #5, <Xt>
```

and is always the preferred disassembly.

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

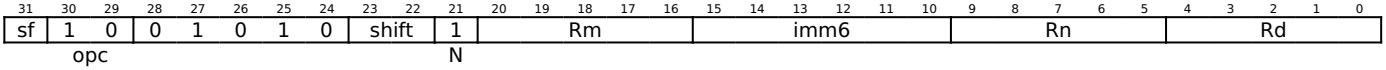
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

EON (shifted register)

Bitwise exclusive-OR NOT (shifted register)

This instruction performs a bitwise exclusive-OR NOT between a register value and an optionally-shifted register value, and writes the result to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
EON <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
EON <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift to be applied to the final source, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	ROR

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = ShiftReg{datasize}(m, shift_type, shift_amount);
let result : bits(datasize) = operand1 XOR NOT operand2;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

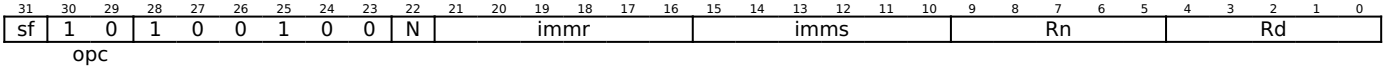
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

EOR (immediate)

Bitwise exclusive-OR (immediate)

This instruction performs a bitwise exclusive-OR between a register value and an immediate value, and writes the result to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
EOR <Wd|WSP>, <Wn>, #<imm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
EOR <Xd|SP>, <Xn>, #<imm>
```

Decode for all variants of this encoding

```
if sf == '0' && N != '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
var imm : bits(datasize);
(imm, -) = DecodeBitMasks{datasize}(N, imms, immr, TRUE);
```

Assembler Symbols

<Wd|WSP>	Is the 32-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<imm>	Is the bitmask immediate. For encoding information, see *Logical (immediate)*.
<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = imm;
let result : bits(datasize) = operand1 XOR operand2;

if d == 31 then
    SP{64}() = ZeroExtend{64}(result);
else
    X{datasize}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

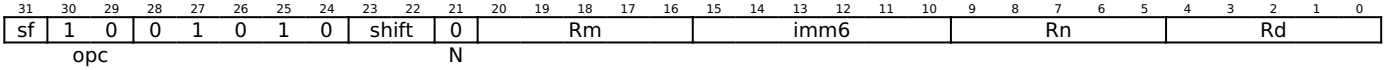
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

EOR (shifted register)

Bitwise exclusive-OR (shifted register)

This instruction performs a bitwise exclusive-OR between a register value and an optionally-shifted register value, and writes the result to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
EOR <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
EOR <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift to be applied to the final source, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	ROR

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = ShiftReg{datasize}(m, shift_type, shift_amount);
let result : bits(datasize) = operand1 XOR operand2;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

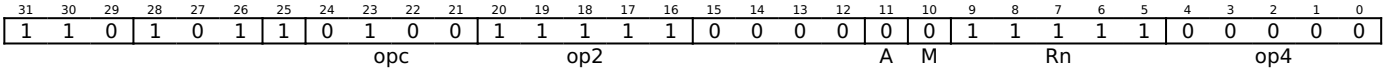
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ERET

Exception return

This instruction restores *PSTATE* from the SPSR, and branches to the address held in the ELR.
The SPSR is checked for the current Exception level for an illegal return event. See *Illegal exception returns from AArch64 state*.
ERET is UNDEFINED at EL0.



Encoding

ERET

Decode for this encoding

```
let pac : boolean = FALSE;
let use_key_a : boolean = TRUE;
```

Operation

```
if PSTATE.EL == EL0 then Undefined(); end;
AArch64_CheckForERetTrap(pac, use_key_a);
let target : bits(64) = ELR_ELx();

AArch64_ExceptionReturn(target, SPSR_ELx());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ERETAA, ERETAB

Exception return, with pointer authentication

This instruction authenticates the address in ELR, using SP as the modifier and the specified key, restores *PSTATE* from the SPSR for the current Exception level, and branches to the authenticated address.

Key A is used for ERETAA. Key B is used for ERETAB.

If the authentication passes, the PE continues execution at the target of the branch. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.

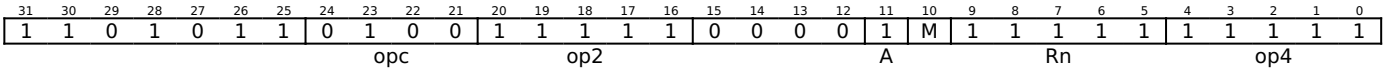
The authenticated address is not written back to ELR.

The SPSR is checked for the current Exception level for an illegal return event. See *Illegal exception returns from AArch64 state*.

ERETAA and ERETAB are UNDEFINED at EL0.

Integer

(FEAT_PAuth)



Encoding for the ERETAA variant

Applies when (M == 0)

ERETAA

Encoding for the ERETAB variant

Applies when (M == 1)

ERETAB

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let use_key_a : boolean = (M == '0');
let pac : boolean = TRUE;
let auth_then_branch : boolean = TRUE;
```

Operation

```
if PSTATE.EL == EL0 then Undefined(); end;
AArch64_CheckForERetTrap(pac, use_key_a);

var target : bits(64) = ELR_ELx();
let modifier : bits(64) = SP{64}();
if use_key_a then
    target = AuthIA(target, modifier, auth_then_branch);
else
    target = AuthIB(target, modifier, auth_then_branch);
end;

AArch64_ExceptionReturn(target, SPSR_ELx());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

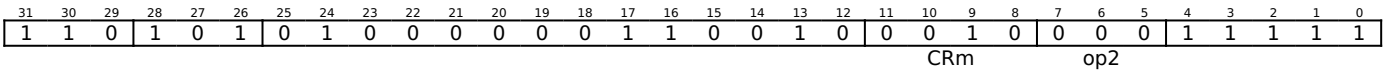
ESB

Error synchronization barrier

This instruction is an error synchronization event that might also update DISR_EL1 and VDISR_EL2.
This instruction can be used at all Exception levels and in Debug state.
In Debug state, this instruction behaves as if SError interrupts are masked at all Exception levels. For more information, see *RAS PE architecture* and Arm® Reliability, Availability, and Serviceability (RAS) System Architecture, for A-profile architecture (ARM IHI 0100).
If *FEAT_RAS* is not implemented, this instruction executes as a `NOP`.

System

(FEAT_RAS)



Encoding

```
ESB
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RAS) then EndOfDecode(Decode_NOP); end;
```

Operation

```
SynchronizeErrors();
AArch64_ESBOperation();
if PSTATE.EL IN {EL0, EL1} && EL2Enabled() then
    AArch64_vESBOperation();
elsif IsFeatureImplemented(FEAT_E3DSE) && PSTATE.EL != EL3 then
    AArch64_dESBOperation();
end;
TakeUnmaskedSErrorInterrupts();
```

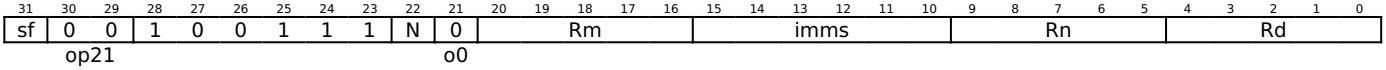
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

EXTR

Extract register

This instruction extracts a register from a pair of registers.
This instruction is used by the alias ROR (immediate).



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0 && imms == 0xxxxx)`

```
EXTR <Wd>, <Wn>, <Wm>, #<lsb>
```

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
EXTR <Xd>, <Xn>, <Xm>, #<lsb>
```

Decode for all variants of this encoding

```
if N != sf then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && imms[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let lsb : integer{} = UInt(imms);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<lsb>	For the "32-bit" variant: is the least significant bit position from which to extract, in the range 0 to 31, encoded in the "imms" field. For the "64-bit" variant: is the least significant bit position from which to extract, in the range 0 to 63, encoded in the "imms" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
ROR (immediate)	`Rn == Rm`

Operation

```
var result : bits(datasize);
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);
let concat : bits(2*datasize) = operand1::operand2;

result = concat[(lsb+datasize)-1:lsb];

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

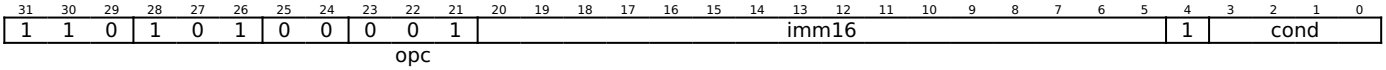
FLT.cond

Conditional fault

This instruction conditionally generates a Conditional fault exception. If an exception is generated, the immediate field is copied to ESR_ELx.

System

(FEAT_CFLT)



Encoding

```
FLT.<cond> #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CFLT) then EndOfDecode(Decode_UNDEF); end;
let imm : bits(16) = imm16;
let condition : bits(4) = cond;
```

Assembler Symbols

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<imm> Is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field.

Operation

```
CheckCFLTEnabled();

if ConditionHolds(condition) then
    let isflt : boolean = TRUE;
    AArch64_ConditionalFault(isflt, imm);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

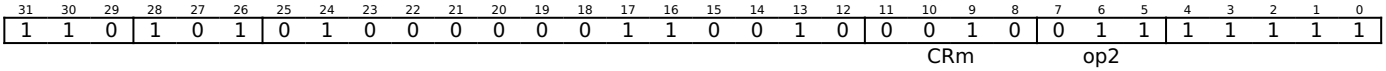
GCSB

Guarded Control Stack barrier

This instruction generates a GCSB effect.
If *FEAT_GCS* is not implemented, this instruction executes as a `NOP`.

System

(FEAT_GCS)



Encoding

```
GCSB DSYNC
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_GCS) then EndOfDecode(Decode_NOP); end;
```

Operation

```
GCSSynchronizationBarrier();
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GCSPOPCX

Guarded Control Stack pop and compare exception return record

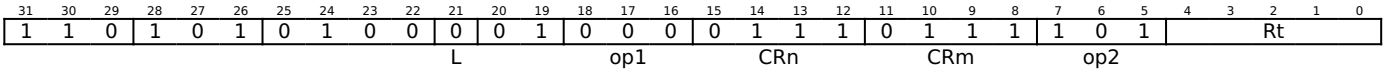
This instruction loads an exception return record from the location indicated by the current GCS pointer register, compares the loaded values with the current ELR_ELx, SPSR_ELx, and LR, and increments the pointer by the size of a GCS exception return record.
For more information, see GCSPOPCX, Guarded Control Stack Pop and Compare exception return record.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_GCS)



Encoding

```
GCSPOPCX
```

is equivalent to

```
SYS #0, C7, C7, #5{, <Xt>}
```

and is always the preferred disassembly.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GCSPOPM

Guarded Control Stack pop

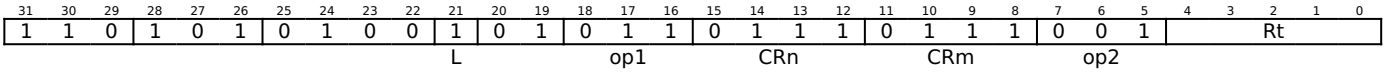
This instruction loads the 64-bit doubleword that is pointed to by the current GCS pointer register, writes it to the destination register, and increments the current GCS pointer register by the size of a GCS procedure return record.
For more information, see GCSPOPM, Guarded Control Stack Pop.

This is an alias of SYSL. This means:

- The encodings in this description are named to match the encodings of SYSL.
- The description of SYSL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_GCS)



Encoding

```
GCSPOPM {<Xt>}
```

is equivalent to

```
SYSL <Xt>, #3, C7, C7, #1
```

and is always the preferred disassembly.

Assembler Symbols

<Xt>	Is the 64-bit name of the optional general-purpose destination register, encoded in the "Rt" field. Defaults to XZR if absent.

Operation

The description of SYSL gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GCSPOPX

Guarded Control Stack pop exception return record

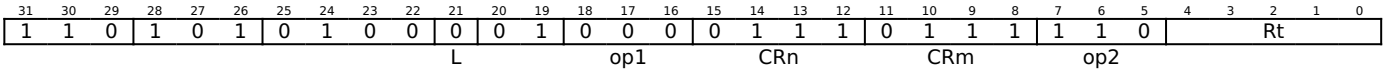
This instruction loads an exception return record from the location indicated by the current GCS pointer register, checks that the record is an exception return record, and increments the pointer by the size of a Guarded Control Stack exception return record.
For more information, see GCSPOPX, Guarded Control Stack Pop exception return record.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_GCS)



Encoding

GCSPOPX

is equivalent to

SYS #0, C7, C7, #6{, <Xt>}

and is always the preferred disassembly.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GCSPUSHM

Guarded Control Stack push

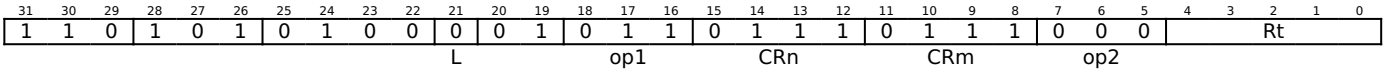
This instruction decrements the current GCS pointer register by the size of a GCS procedure return record and stores an entry to the Guarded Control Stack.
For more information, see GCSPUSHM, Guarded Control Stack Push.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_GCS)



Encoding

```
GCSPUSHM <Xt>
```

is equivalent to

```
SYS #3, C7, C7, #0, <Xt>
```

and is always the preferred disassembly.

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GCSPUSHX

Guarded Control Stack push exception return record

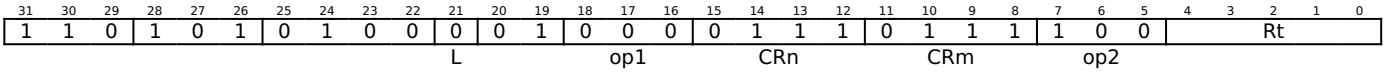
This instruction decrements the current GCS pointer register by the size of a Guarded Control Stack exception return record and stores an exception return record to the Guarded Control Stack.
For more information, see GCSPUSHX, Guarded Control Stack Push exception return record.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_GCS)



Encoding

GCSPUSHX

is equivalent to

SYS #0, C7, C7, #4{, <Xt>}

and is always the preferred disassembly.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GCSSS1

Guarded Control Stack switch stack 1

This instruction validates that the stack being switched to contains a Valid GCS cap record, stores an In-progress GCS cap record to the stack that is being switched to, and sets the current GCS pointer register to the stack that is being switched to.

If the instruction generates a synchronous Data Abort exception, Watchpoint exception, GPC exception, or GCS Data Check exception, the value of *GCSPR_ELx* for the current Exception level is restored to the value held in the register before the instruction was executed.

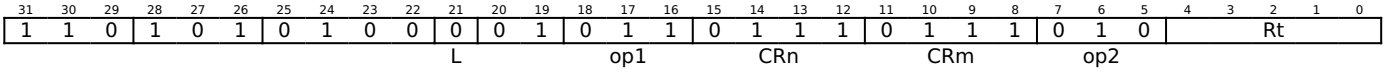
For more information, see GCSSS1, Guarded Control Stack Switch Stack 1.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_GCS)



Encoding

```
GCSSS1 <Xt>
```

is equivalent to

```
SYS #3, C7, C7, #2, <Xt>
```

and is always the preferred disassembly.

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GCSSS2

Guarded Control Stack switch stack 2

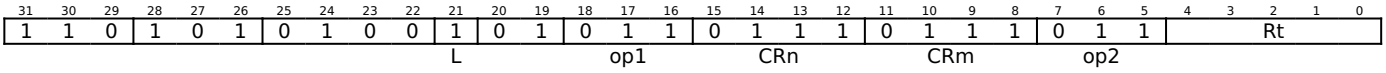
This instruction validates that the most recent entry of the Guarded Control Stack being switched to contains an In-progress GCS cap record, stores a Valid GCS cap record to the Guarded Control Stack that is being switched from, and sets Xt to the Guarded Control Stack pointer that is being switched from.
For more information, see GCSSS2, Guarded Control Stack Switch Stack 2.

This is an alias of SYSL. This means:

- The encodings in this description are named to match the encodings of SYSL.
- The description of SYSL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_GCS)



Encoding

```
GCSSS2 <Xt>
```

is equivalent to

```
SYSL <Xt>, #3, C7, C7, #3
```

and is always the preferred disassembly.

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rt" field.

Operation

The description of SYSL gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

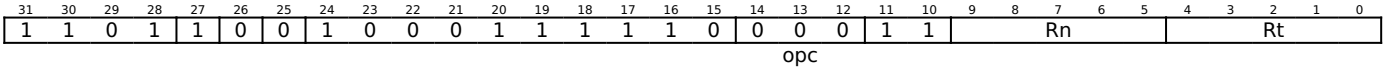
GCSSTR

Guarded Control Stack store register

This instruction stores a doubleword from a register to memory. The address that is used for the store is calculated from a base register.

Integer

(FEAT_GCS)



Encoding

```
GCSSTR <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_GCS) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);

let effective_el : bits(2) = PSTATE.EL;

if effective_el == PSTATE.EL then
    CheckGCSSTREnabled();
end;

let privileged : boolean = effective_el != EL0;
let accdesc : AccessDescriptor = CreateAccDescGCS(MemOp_STORE, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

Mem{64}(address, accdesc) = X{64}(t);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GCSSTTR

Guarded Control Stack store register (unprivileged)

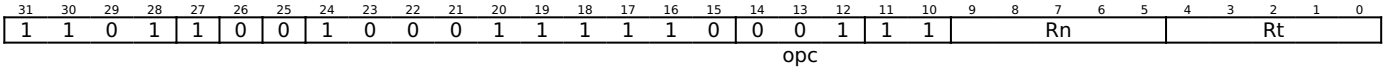
This instruction stores a doubleword from a register to memory. The address that is used for the store is calculated from a base register.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1 and the *Effective value* of *HCR_EL2*.{NV1, NV} is not {1, 1}.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_GCS)



Encoding

```
GCSSTTR <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_GCS) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);

let effective_el : bits(2) = if AArch64_IsUnprivAccessPriv() then PSTATE.EL else EL0;

if effective_el == PSTATE.EL then
    CheckGCSSTREnabled();
end;

let privileged : boolean = effective_el != EL0;
let accdesc : AccessDescriptor = CreateAccDescGCS(MemOp_STORE, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

Mem{64}(address, accdesc) = X{64}(t);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GIC

Generic Interrupt Controller

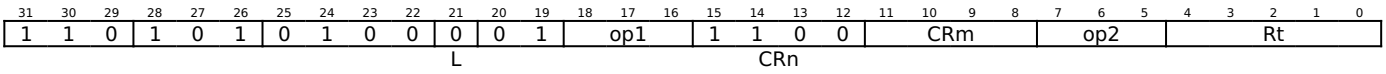
This instruction manages interrupts when FEAT_GCIE is implemented.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_GCIE)



Encoding

Applies when `(CRm IN {0001, 0010})`

```
GIC <gic_op>{, <Xt>}
```

is equivalent to

```
SYS #<op1>, C12, <Cm>, #<op2>{, <Xt>}
```

and is the preferred disassembly when `SysOp(op1, '1100', CRm, op2) == Sys_GIC`.

Assembler Symbols

<gic_op> Is a GIC operation name, as listed for the GIC system instruction group, encoded in "(op1 :: CRm :: op2)":

op1	CRm	op2	<gic_op>	Description
000	0001	000	CDDIS	For more information, see GIC CDDIS.
000	0001	001	CDEN	For more information, see GIC CDEN.
000	0001	010	CDPRI	For more information, see GIC CDPRI.
000	0001	011	CDAFF	For more information, see GIC CDAFF.
000	0001	100	CDPEND	For more information, see GIC CDPEND.
000	0001	101	CDRCFG	For more information, see GIC CDRCFG.
000	0001	111	CDEOI	For more information, see GIC CDEOI.
000	0010	000	CDDI	For more information, see GIC CDDI.
000	0010	001	CDHM	For more information, see GIC CDHM.
100	0001	000	VDDIS	For more information, see GIC VDDIS.
100	0001	001	VDEN	For more information, see GIC VDEN.
100	0001	010	VDPRI	For more information, see GIC VDPRI.
100	0001	011	VDAFF	For more information, see GIC VDAFF.
100	0001	100	VDPEND	For more information, see GIC VDPEND.
100	0001	101	VDRCFG	For more information, see GIC VDRCFG.
100	0010	000	VDDI	For more information, see GIC VDDI.
100	0010	001	VDHM	For more information, see GIC VDHM.
110	0001	000	LDDIS	For more information, see GIC LDDIS.
110	0001	001	LDEN	For more information, see GIC LDEN.
110	0001	010	LDPRI	For more information, see GIC LDPRI.
110	0001	011	LDAFF	For more information, see GIC LDAFF.
110	0001	100	LDPEND	For more information, see GIC LDPEND.
110	0001	101	LDRCFG	For more information, see GIC LDRCFG.
110	0010	000	LDDI	For more information, see GIC LDDI.
110	0010	001	LDHM	For more information, see GIC LDHM.

<Xt> Is the 64-bit name of the optional general-purpose source register, defaulting to '11111', encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GICR

Generic Interrupt Controller with result

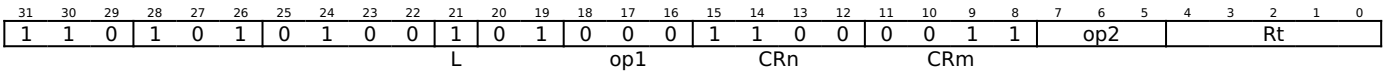
This instruction acknowledges interrupts when FEAT_GCIE is implemented.

This is an alias of SYSL. This means:

- The encodings in this description are named to match the encodings of SYSL.
- The description of SYSL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_GCIE)



Encoding

```
GICR <Xt>, <gicr_op>
```

is equivalent to

```
SYSL <Xt>, #0, C12, C3, #<op2>
```

and is the preferred disassembly when `SysLOp('000', '1100', '0011', op2) == Sysl_GICR`.

Assembler Symbols

<Xt> Is the 64-bit name of the general-purpose destination register, encoded in the "Rt" field.

<gicr_op> Is a GICR operation name, as listed for the GICR system instruction group, encoded in “op2”:

op2	<gicr_op>	Description
000	CDIA	For more information, see GICR CDIA.
001	CDNMIA	For more information, see GICR CDNMIA.

Operation

The description of SYSL gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

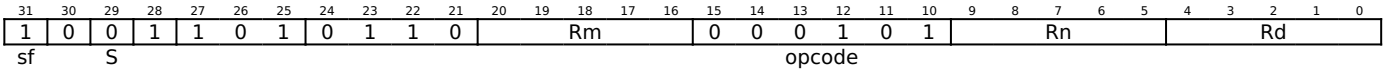
GMI

Tag mask insert

This instruction inserts the tag in the first source register into the excluded set specified in the second source register, writing the new excluded set to the destination register.

Integer

(FEAT_MTE)



Encoding

```
GMI <Xd>, <Xn|SP>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let address : bits(64) = if n == 31 then SP{64}() else X{64}(n);
var mask : bits(64) = X{64}(m);
let tag : bits(4) = AArch64_AllocationTagFromAddress(address);

mask[UInt(tag)] = '1';
X{64}(d) = mask;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

GSB

Generic Interrupt Controller synchronization barrier

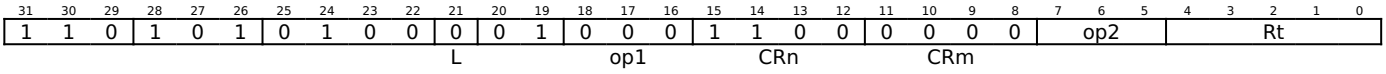
This instruction provides explicit synchronization mechanisms for GIC and GICR System instructions.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_GCIE)



Encoding

```
GSB <gsb_op>
```

is equivalent to

```
SYS #0, C12, C0, #<op2>{, <Xt>}
```

and is the preferred disassembly when `SysOp('000', '1100', '0000', op2) == Sys_GSB`.

Assembler Symbols

<gsb_op> Is a GSB operation name, as listed for the GSB system instruction group, encoded in “op2”:

op2	<gsb_op>	Description
000	SYS	For more information, see GSB SYS.
001	ACK	For more information, see GSB ACK.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

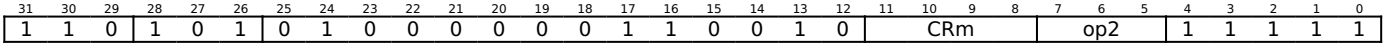
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

HINT

Hint instruction

This instruction is for the instruction set space that is reserved for architectural hint instructions.

Some encodings described here are not allocated in this revision of the architecture, and behave as NOPs. These encodings might be allocated to other hint functionality in future revisions of the architecture and therefore must not be used by software.



Encoding

```
HINT #<imm>
```

Decode for this encoding

```
var op : SystemHintOp;

var stream : boolean;
var priority : boolean;
var stshstrm : boolean;
case CRm::op2 of
    when '0000 000' => op = SystemHintOp_NOP;
    when '0000 001' => op = SystemHintOp_YIELD;
    when '0000 010' => op = SystemHintOp_WFE;
    when '0000 011' => op = SystemHintOp_WFI;
    when '0000 100' => op = SystemHintOp_SEV;
    when '0000 101' => op = SystemHintOp_SEVL;
    when '0000 110' =>
        if !IsFeatureImplemented(FEAT_DGH) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_DGH;
    when '0000 111' => See("XPACLRI");
    when '0001 xxx' =>
        case op2 of
            when '000' => See("PACIA1716");
            when '010' => See("PACIB1716");
            when '100' => See("AUTIA1716");
            when '110' => See("AUTIB1716");
            otherwise => EndOfDecode(Decode_NOP);
        end;
    when '0010 000' =>
        if !IsFeatureImplemented(FEAT_RAS) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_ESB;
    when '0010 001' =>
        if !IsFeatureImplemented(FEAT_SPE) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_PSB;
    when '0010 010' =>
        if !IsFeatureImplemented(FEAT_TRF) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_TSB;
    when '0010 011' =>
        if !IsFeatureImplemented(FEAT_GCS) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_GCSB;
    when '0010 100' =>
        op = SystemHintOp_CSDB;
    when '0010 110' =>
        if !IsFeatureImplemented(FEAT_CLRBHB) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_CLRBHB;
    when '0011 xxx' =>
        case op2 of
            when '000' => See("PACIAZ");
            when '001' => See("PACIASP");
            when '010' => See("PACIBZ");
            when '011' => See("PACIBSP");
            when '100' => See("AUTIAZ");
            when '101' => See("AUTIASP");
            when '110' => See("AUTIBZ");
            when '111' => See("AUTIBSP");
        end;
    when '0100 xx0' =>
        if !IsFeatureImplemented(FEAT_BTI) then EndOfDecode(Decode_NOP); end;

        // Check branch target compatibility between BTI instruction and PSTATE.BTYPE
        SetBTypeCompatible(BTypeCompatible_BTI(op2[2:1]));
        op = SystemHintOp_BTI;
    when '0100 111' => See("PACM");
    when '0101 000' =>
        if !IsFeatureImplemented(FEAT_CHK) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_CHKFEAT;
    when '0110 00x' =>
        if !IsFeatureImplemented(FEAT_PCDPHINT) then EndOfDecode(Decode_NOP); end;
        stream = op2[0] == '1';
        op = SystemHintOp_STSHH;
    when '0110 01x' =>
        if !IsFeatureImplemented(FEAT_CMH) then EndOfDecode(Decode_NOP); end;
        priority = op2[0] == '1';

        op = SystemHintOp_SHUH;
    when '0110 100' =>
        if !IsFeatureImplemented(FEAT_CMH) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_STCPH;
    when '0110 11x' =>
        if !IsFeatureImplemented(FEAT_RLCS) then EndOfDecode(Decode_NOP); end;
        stshstrm = op2[0] == '1';

        op = SystemHintOp_SRLS;
    when '0111 000' =>
        if !IsFeatureImplemented(FEAT_RLCS) then EndOfDecode(Decode_NOP); end;
        op = SystemHintOp_SLBND;
    otherwise => EndOfDecode(Decode_NOP);
end;
```

Assembler Symbols

<imm> Is a 7-bit unsigned immediate, in the range 0 to 127, encoded in the "CRm:op2" field. The encodings that are allocated to architectural hint functionality are described in the 'Hints' table in the 'Index by Encoding'.

Note

For allocated encodings of "CRm:op2":

- A disassembler will disassemble the allocated instruction, rather than the `HINT` instruction.
- An assembler may support assembly of allocated encodings using `HINT` with the corresponding <imm> value, but it is not required to do so.

Operation

```
case op of
    when SystemHintOp_YIELD =>
        Hint_Yield();

    when SystemHintOp_DGH =>
        Hint_DGH();

    when SystemHintOp_WFE =>
        Hint_WFE();

    when SystemHintOp_WFI =>
        Hint_WFI();

    when SystemHintOp_SEV =>
        SendEvent();

    when SystemHintOp_SEVL =>
        SendEventLocal();

    when SystemHintOp_ESB =>
        SynchronizeErrors();
        AArch64_ESBOperation();
        if PSTATE.EL IN {EL0, EL1} && EL2Enabled() then
            AArch64_vESBOperation();
        elsif IsFeatureImplemented(FEAT_E3DSE) && PSTATE.EL != EL3 then
            AArch64_dESBOperation();
        end;
        TakeUnmaskedSErrorInterrupts();

    when SystemHintOp_PSB =>
        CheckPSBTrap();

        ProfilingSynchronizationBarrier();

    when SystemHintOp_TSB =>
        CheckTSBTrap();

        TraceSynchronizationBarrier();

    when SystemHintOp_GCSB =>
        GCSSynchronizationBarrier();

    when SystemHintOp_CHKFEAT =>
        X{64}(16) = AArch64_ChkFeat(X{64}(16));

    when SystemHintOp_CSDB =>
        ConsumptionOfSpeculativeDataBarrier();

    when SystemHintOp_CLRBHB =>
        Hint_CLRBHB();

    when SystemHintOp_BTI =>
        SetBTypeNext('00');

    when SystemHintOp_STSHH =>
        Hint_StoreShared(stream);

    when SystemHintOp_SHUH =>
        Hint_SharedUpdate(priority);

    when SystemHintOp_STCPH =>
        Hint_StoreConcurrentPriority();

    when SystemHintOp_SLBND =>
        Hint_ScopedReleaseBoundary();

    when SystemHintOp_SRLS =>
        Hint_ScopeRelease(stshstrm);

    when SystemHintOp_NOP =>
        return; // Do nothing

    otherwise =>
        unreachable;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

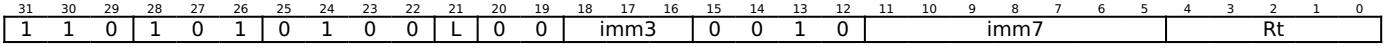
HINTE

Hint instruction (extended)

This instruction is the extended instruction set space that is reserved for future architectural hint instructions. The encodings described here are not allocated in this revision of the architecture, and behave as NOPs. These encodings might be allocated to other hint functionality in future revisions of the architecture and therefore must not be used by software.

System

(FEAT_HINTE)



Encoding

Applies when `(!(L == 0 && imm3 == 011 && Rt == 11111))`

```
HINTE #<imm16>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_HINTE) then EndOfDecode(Decode_UNDEF); end;
```

Assembler Symbols

<imm16>	Is a 16-bit unsigned immediate, in the range 0 to 65535, excluding the values 12319 to 16383 in increments of 32, encoded in the "L:imm3:imm7:Rt" fields.

Operation

```
return; // Do nothing
```

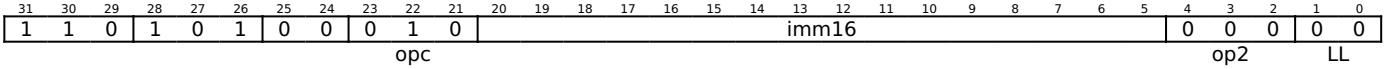
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

HLT

Halt instruction

This instruction can generate a Halt Instruction debug event, which causes entry into Debug state.
Within a guarded memory region, while *PSTATE*.BTYPE != 0b00, an `HLT` instruction that would cause entry into Debug state will not generate a Branch Target exception and will cause entry into Debug state as normal. For more information, see *PSTATE.BTYPE*.



Encoding

```
HLT #<imm>
```

Decode for this encoding

```
if EDSCR().HDE == '0' || !HaltingAllowed() then
    EndOfDecode(Decode_UNDEF);
elsif IsFeatureImplemented(FEAT_BTI) then
    SetBTypeCompatible(TRUE);
end;
```

Assembler Symbols

<imm>	Is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field.

Operation

```
let is_async : boolean = FALSE;
let fault : FaultRecord = NoFault();
Halt(DebugHalt_HaltInstruction, is_async, fault);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

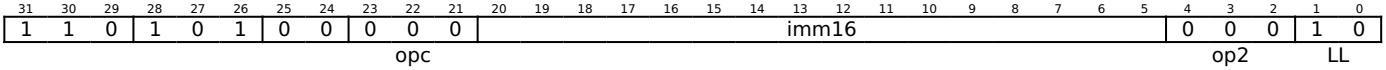
HVC

Hypervisor call

This instruction causes an exception to EL2. Software executing at EL1 can use this instruction to call the hypervisor to request a service.
The HVC instruction is UNDEFINED:

- When EL3 is implemented and SCR_EL3.HCE is set to 0.
- When EL3 is not implemented and *HCR_EL2*.HCD is set to 1.
- When EL2 is not implemented.
- At EL1 if EL2 is not enabled in the current Security state.
- At EL0.

On executing an HVC instruction, the PE records the exception as a Hypervisor Call exception in *ESR_ELx*, using the EC syndrome value 0x16, and the value of the immediate argument.



Encoding

```
HVC #<imm>
```

Decode for this encoding

```
if !HaveEL(EL2) then EndOfDecode(Decode_UNDEF); end;
let imm : bits(16) = imm16;
```

Assembler Symbols

<imm>	Is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field.

Operation

```
if PSTATE.EL == EL0 then Undefined(); end;
if PSTATE.EL == EL1 && !EL2Enabled() then Undefined(); end;
if !HaveEL(EL3) && HCR_EL2().HCD == '1' then Undefined(); end;
if HaveEL(EL3) && SCR_EL3().HCE == '0' then Undefined(); end;

AArch64_CallHypervisor(imm);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

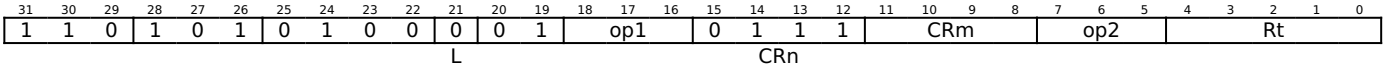
IC

Instruction cache operation

For more information, see *op0 == 0b01, cache maintenance, TLB maintenance, address translation, prediction restriction, BRBE, Trace Extension, Guarded Control Stack, and FEAT_RME_GPC3 instructions*.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

```
IC <ic_op>{, <Xt>}
```

is equivalent to

```
SYS #<op1>, C7, <Cm>, #<op2>{, <Xt>}
```

and is the preferred disassembly when `SysOp(op1, '0111', CRm, op2) == Sys_IC`.

Assembler Symbols

<ic_op> Is an IC operation name, as listed for the IC system instruction pages, encoded in “(op1 :: CRm :: op2)”:

op1	CRm	op2	<ic_op>	Description
000	0001	000	IALLUIS	For more information, see IC IALLUIS.
000	0101	000	IALLU	For more information, see IC IALLU.
011	0101	001	IVAU	For more information, see IC IVAU.

<Xt> Is the 64-bit name of the optional general-purpose source register, defaulting to '11111', encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

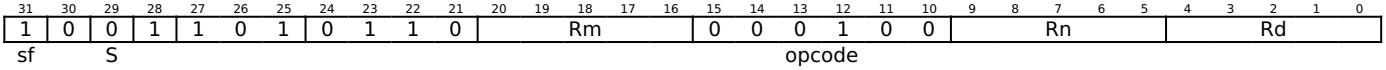
IRG

Insert random tag

This instruction inserts a random Logical Address Tag into the address in the first source register, and writes the result to the destination register. Any tags specified in the optional second source register or in GCR_EL1.Exclude are excluded from the selection of the random Logical Address Tag.

Integer

(FEAT_MTE)



Encoding

```
IRG <Xd|SP>, <Xn|SP>{, <Xm>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field. Defaults to XZR if absent.

Operation

```
let operand : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let exclude_reg : bits(64) = X{64}(m);
let exclude : bits(16) = exclude_reg[15:0] OR GCR_EL1().Exclude;

let rtag : bits(4) = AArch64_ChooseTagOrZero(exclude);
let result : bits(64) = AArch64_AddressWithAllocationTag(operand, rtag);

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

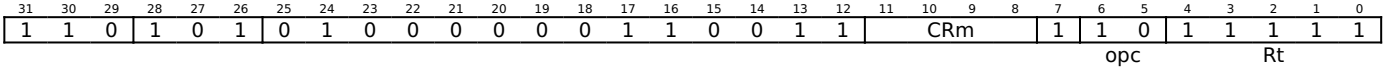
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ISB

Instruction synchronization barrier

This instruction flushes the pipeline in the PE and is a context synchronization event. For more information, see *Instruction Synchronization Barrier (ISB)*.



Encoding

```
ISB {<option>|#<imm>}
```

Decode for this encoding

```
// no additional decoding required
```

Assembler Symbols

<option> Specifies an optional limitation on the barrier operation. Values are:

SY
Full system barrier operation, encoded as CRm = 0b1111. Can be omitted.

All other encodings of "CRm" are reserved. The corresponding instructions execute as full system barrier operations, but must not be relied upon by software.

<imm> Is an optional 4-bit unsigned immediate, in the range 0 to 15, defaulting to 15 and encoded in the "CRm" field.

Operation

```
InstructionSynchronizationBarrier();

if IsFeatureImplemented(FEAT_BRBE) && BRBEBranchOnISB() then
    BRBEISB();
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD64B

Single-copy atomic 64-byte load

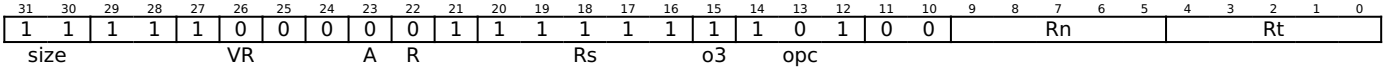
This instruction derives an address from a base register value, loads eight 64-bit doublewords from a memory location, and writes them to consecutive registers. The load starts at register Xt, with the data being read as X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt = Data[511:0]. The data is loaded atomically and is required to be 64-byte aligned.

It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports LD64B also supports LDA64B, ST64B, and STL64B.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64)



Encoding

```
LD64B <Xt>, [<Xn|SP> {, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LS64) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = FALSE;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = FALSE;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LD64B*.

Assembler Symbols

<Xt>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckLDST64BEnabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_LOAD, withstatus, acqrel, tagchecked);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

data = MemLoad64B(address, accdesc);

for i = 0 to 7 do
    value = data[63+64*i : 64*i];
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    X{64}(t+i) = value;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDA64B

Single-copy atomic 64-byte load-acquire

This instruction derives an address from a base register value, loads eight 64-bit doublewords from a memory location, and writes them to consecutive registers. The load starts at register Xt, with the data being read as X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt = Data[511:0]. The data is loaded atomically and is required to be 64-byte aligned.

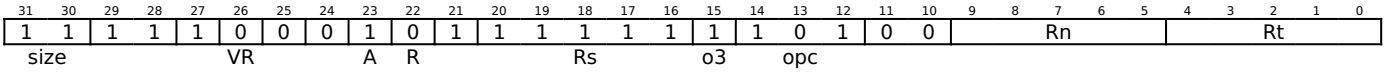
It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports LDA64B also supports LD64B, ST64B, and STL64B.

LDA64B loads from memory with acquire semantics.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64 && FEAT_LSC64B)



Encoding

```
LDA64B <Xt>, [<Xn|SP> {, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSC64B) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_LS64) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = FALSE;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = TRUE;
```

LDA64B has the same CONSTRAINED UNPREDICTABLE behavior as LD64B. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LD64B*.

Assembler Symbols

<Xt>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckLDST64BEnabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_LOAD, withstatus, acqrel, tagchecked);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

data = MemLoad64B(address, accdesc);

for i = 0 to 7 do
    value = data[63+64*i : 64*i];
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    X{64}(t+i) = value;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDADD, LDADDA, LDADDAL, LDADDL

Atomic add on word or doubleword

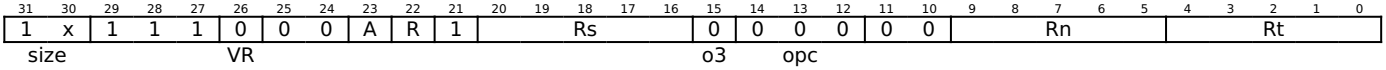
This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, adds the value held in a register to it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR or XZR, `LDADDA` and `LDADDAL` load from memory with acquire semantics.
- `LDADDL` and `LDADDAL` store to memory with release semantics.
- `LDADD` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STADD, STADDL.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDADD <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDADDA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDADDAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDADDL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDADD <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDADDA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDADDAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDADDL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STADD, STADDL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_ADD, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDADDB, LDADDAB, LDADDALB, LDADDLB

Atomic add on byte

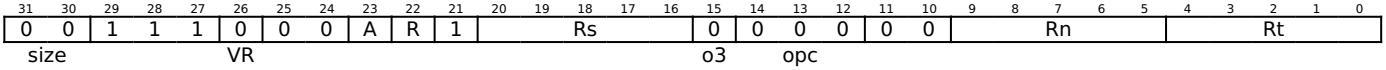
This instruction atomically loads an 8-bit byte from memory, adds the value held in a register to it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDADDAB` and `LDADDALB` load from memory with acquire semantics.
- `LDADDLB` and `LDADDALB` store to memory with release semantics.
- `LDADDB` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STADDB, STADDLB.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDADDB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDADDAB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDADDALB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDADDLB <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STADDB, STADDLB	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_ADD, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(8) = ARBITRARY : bits(8); // Irrelevant when not executing CAS
let value : bits(8) = X{8}(s);
let data : bits(8) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDADDH, LDADDAH, LDADDALH, LDADDLH

Atomic add on halfword

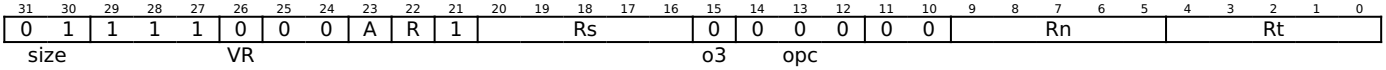
This instruction atomically loads a 16-bit halfword from memory, adds the value held in a register to it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDADDAH` and `LDADDALH` load from memory with acquire semantics.
- `LDADDLH` and `LDADDALH` store to memory with release semantics.
- `LDADDH` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STADDH, STADDLH.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDADDH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDADDAH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDADDALH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDADDLH <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STADDH, STADDLH	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_ADD, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(16) = ARBITRARY : bits(16); // Irrelevant when not executing CAS
let value : bits(16) = X{16}(s);
let data : bits(16) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAP

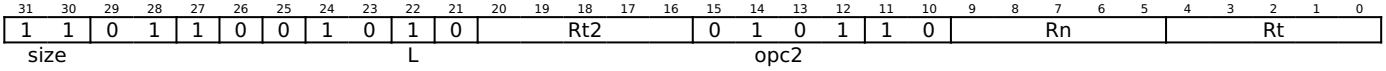
Load-acquire pair of registers

This instruction calculates an address from a base register value, loads two 64-bit doublewords from memory, and writes them to two registers. Explicit Memory effects produced by this instruction have Acquire semantics. For information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSCP)



Encoding

```
LDAP <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSCP) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let acquire : boolean = TRUE;
let tagchecked : boolean = n != 31;
let ispair : boolean = TRUE;
var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // Result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDP and LDIAPP*, which applies when t==t2.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data1 : bits(64);
var data2 : bits(64);
let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_LOAD, tagchecked, ispair,
                                                     acquire, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let full_data : bits(128) = Mem{128}(address, accdesc);
if BigEndian(accdesc.acctype) then
    data2 = full_data[63:0];
    data1 = full_data[127:64];
else
    data1 = full_data[63:0];
    data2 = full_data[127:64];
end;

if rt_unknown then
    data1 = ARBITRARY : bits(64);
    data2 = ARBITRARY : bits(64);
end;

X{64}(t) = data1;
X{64}(t2) = data2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPP

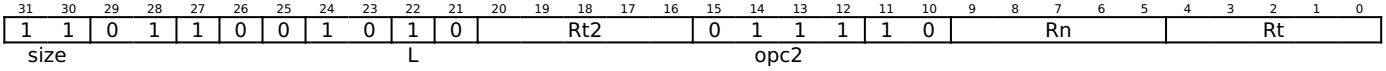
Load-acquire RCpc pair of registers

This instruction calculates an address from a base register value, loads two 64-bit doublewords from memory, and writes them to two registers. Explicit Memory effects produced by this instruction have AcquirePC semantics. For information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSCP)



Encoding

```
LDAPP <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSCP) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let acquire : boolean = TRUE;
let tagchecked : boolean = n != 31;
let ispair : boolean = TRUE;
var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // Result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDP and LDIAPP*, which applies when t==t2.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data1 : bits(64);
var data2 : bits(64);
let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, ispair, acquire, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let full_data : bits(128) = Mem{128}(address, accdesc);
if BigEndian(accdesc.acctype) then
    data2 = full_data[63:0];
    data1 = full_data[127:64];
else
    data1 = full_data[63:0];
    data2 = full_data[127:64];
end;

if rt_unknown then
    data1 = ARBITRARY : bits(64);
    data2 = ARBITRARY : bits(64);
end;

X{64}(t) = data1;
X{64}(t2) = data2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPR

Load-acquire RCpc register

This instruction derives an address from a base register value, loads a 32-bit word or 64-bit doubleword from the derived address in memory, and writes it to a register.

`LDAPR` loads from memory with AcquirePC semantics.

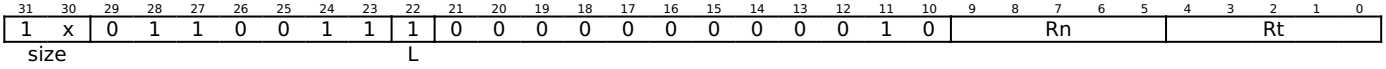
For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 2 classes: Post-index and No offset

Post-index

(FEAT_LRCPC3)



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDAPR <Wt>, [<Xn|SP>], #4
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

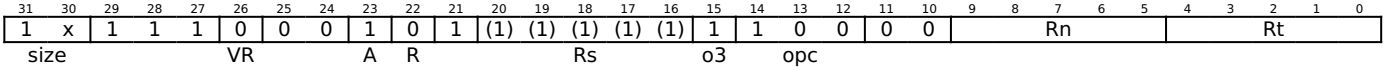
```
LDAPR <Xt>, [<Xn|SP>], #8
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC3) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
var wback : boolean = TRUE;
let acquirepc : boolean = TRUE;
let regsize : integer{} = if size == '11' then 64 else 32;
let datasize : integer{} = 8 << UInt(size);
let offset : integer = 1 << UInt(size);

let tagchecked : boolean = TRUE;

var wb_unknown : boolean = FALSE;

if n == t && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;        // writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;    // writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

No offset

(FEAT_LRCPC)



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDAPR <Wt>, [<Xn|SP> {, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
LDAPR <Xt>, [<Xn|SP> {, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let wback : boolean = FALSE;
let acquirepc : boolean = TRUE;
let regsize : integer{} = if size == '11' then 64 else 32;
let datasize : integer{} = 8 << UInt(size);
let offset : integer = 0;

let tagchecked : boolean = n != 31;

let wb_unknown : boolean = FALSE;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Operation

```
var address : bits(64);
var data : bits(datasize);

let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

data = Mem{datasize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    else
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPRB

Load-acquire RCpc register byte

This instruction derives an address from a base register value, loads a byte from the derived address in memory, zero-extends it and writes it to a register.

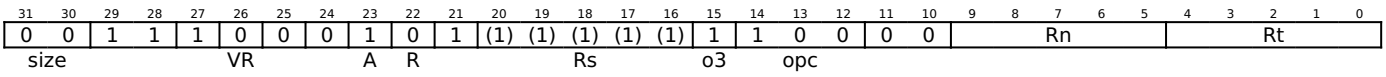
`LDAPRB` loads from memory with AcquirePC semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

Integer

(FEAT_LRCPC)



Encoding

```
LDAPRB <Wt>, [<Xn|SP> {, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquirepc : boolean = TRUE;

let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data : bits(8);

let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

data = Mem{8}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPRH

Load-acquire RCpc register halfword

This instruction derives an address from a base register value, loads a halfword from the derived address in memory, zero-extends it and writes it to a register.

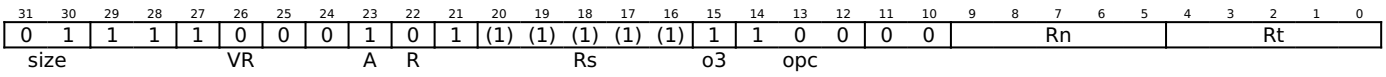
`LDAPRH` loads from memory with AcquirePC semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

Integer

(FEAT_LRCPC)



Encoding

```
LDAPRH <Wt>, [<Xn|SP> {, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let acquirepc : boolean = TRUE;

let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data : bits(16);

let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

data = Mem{16}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPUR

Load-acquire RCpc register (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a 32-bit word or 64-bit doubleword from memory, zero-extends it, and writes it to a register.

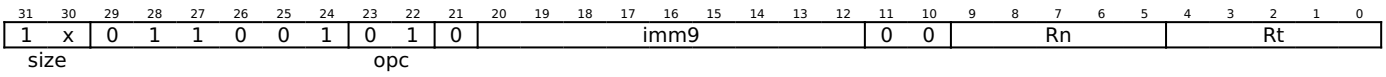
`LDAPUR` loads from memory with AcquirePC semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

Unscaled offset

(FEAT_LRCPC2)



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDAPUR <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
LDAPUR <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC2) then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << scale;
let regsize : integer{} = if datasize == 64 then 64 else 32;
let acquirepc : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPURB

Load-acquire RCpc register byte (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a byte from memory, zero-extends it, and writes it to a register.

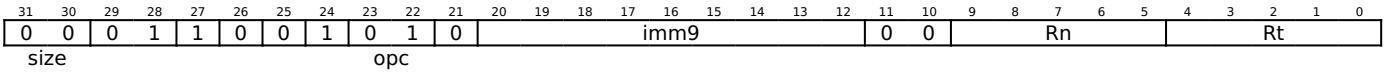
`LDAPURB` loads from memory with AcquirePC semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

Unscaled offset

(FEAT_LRCPC2)



Encoding

```
LDAPURB <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC2) then EndOfDecode(Decode_UNDEF); end;
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8;
let acquirepc : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPURH

Load-acquire RCpc register halfword (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a halfword from memory, zero-extends it, and writes it to a register.

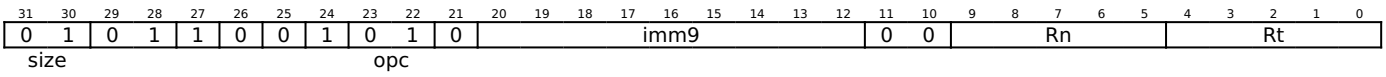
`LDAPURH` loads from memory with AcquirePC semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

Unscaled offset

(FEAT_LRCPC2)



Encoding

```
LDAPURH <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC2) then EndOfDecode(Decode_UNDEF); end;
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 16;
let acquirepc : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPURSB

Load-acquire RCpc register signed byte (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a signed byte from memory, sign-extends it, and writes it to a register.

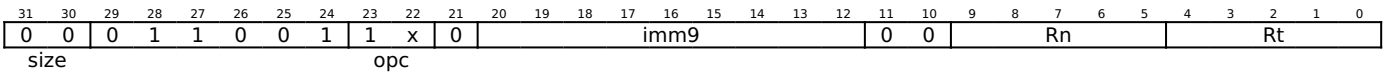
`LDAPURSB` loads from memory with AcquirePC semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

Unscaled offset

(FEAT_LRCPC2)



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDAPURSB <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
LDAPURSB <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC2) then EndOfDecode(Decode_UNDEF); end;
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8;
let regsize : integer{} = 64 >> UInt(opc[0]);
let acquirepc : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = SignExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPURSH

Load-acquire RCpc register signed halfword (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a signed halfword from memory, sign-extends it, and writes it to a register.

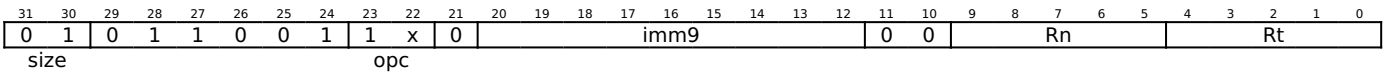
`LDAPURSH` loads from memory with AcquirePC semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

Unscaled offset

(FEAT_LRCPC2)



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDAPURSH <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
LDAPURSH <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC2) then EndOfDecode(Decode_UNDEF); end;
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 16;
let regsize : integer{} = 64 >> UInt(opc[0]);
let acquirepc : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = SignExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPURSW

Load-acquire RCpc register signed word (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a signed word from memory, sign-extends it, and writes it to a register.

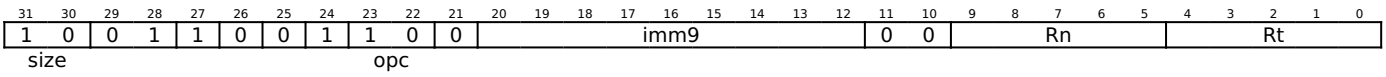
`LDAPURSW` loads from memory with AcquirePC semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

Unscaled offset

(FEAT_LRCPC2)



Encoding

```
LDAPURSW <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC2) then EndOfDecode(Decode_UNDEF); end;
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32;
let acquirepc : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{64}(t) = SignExtend{64}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

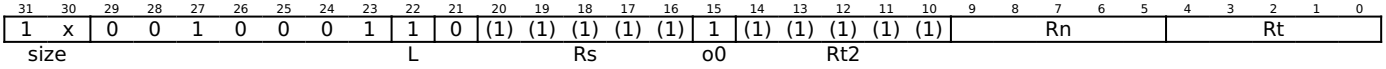
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAR

Load-acquire register

This instruction derives an address from a base register value, loads a 32-bit word or 64-bit doubleword from memory, and writes it to a register. `LDAR` loads from memory with Acquire semantics.
For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDAR <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
LDAR <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;
let acquire : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_LOAD, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

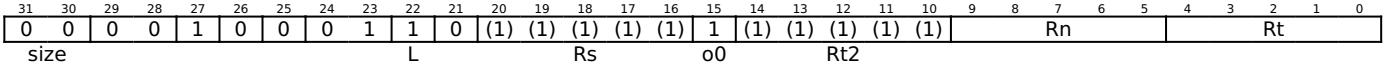
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDARB

Load-acquire register byte

This instruction derives an address from a base register value, loads a byte from memory, zero-extends it and writes it to a register.
`LDARB` loads from memory with Acquire semantics.
For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDARB <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_LOAD, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(8) = Mem{8}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

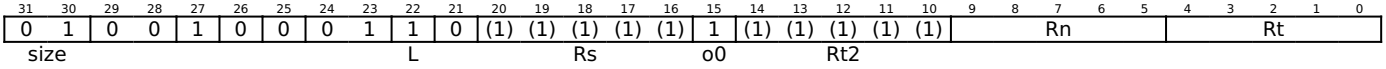
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDARH

Load-acquire register halfword

This instruction derives an address from a base register value, loads a halfword from memory, zero-extends it, and writes it to a register. `LDARH` loads from memory with Acquire semantics.
For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDARH <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_LOAD, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(16) = Mem{16}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDATXR

Load-acquire unprivileged exclusive register

This instruction derives an address from a base register value, loads a 32-bit word or 64-bit doubleword from memory, and writes it to a register. The memory access is atomic. The PE marks the physical address being accessed as an exclusive access. This exclusive access mark is checked by Store Exclusive instructions. See *Synchronization and semaphores*. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

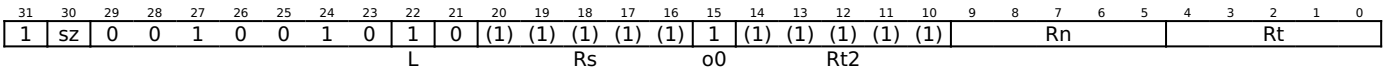
Note

For the purposes of the Exclusives monitors, and the forward progress guarantees for Load-Exclusive and Store-Exclusive loops, `LDATXR` is equivalent to `LDXR`.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
LDATXR <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
LDATXR <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let regsize : integer{} = if elsize == 64 then 64 else 32;
let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

AArch64_SetExclusiveMonitors(address, dbytes, accdesc);

data = Mem{elsize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAXP

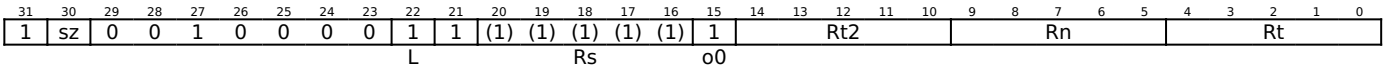
Load-acquire exclusive pair of registers

This instruction derives an address from a base register value, loads two 32-bit words or two 64-bit doublewords from memory, and writes them to two registers. For information on single-copy atomicity and alignment requirements, see *Requirements for single-copy atomicity* and *Alignment of data accesses*. The PE marks the physical address being accessed as an exclusive access. This exclusive access mark is checked by Store Exclusive instructions. See *Synchronization and semaphores*.

`LDAXP` loads from memory with Acquire semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
LDAXP <Wt1>, <Wt2>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
LDAXP <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let datasize : integer{} = elsize * 2;
let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;    // result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *CONSTRAINED UNPREDICTABLE behaviors with Load-Exclusive/Store-Exclusive pairs* and *LDAXP*.

Assembler Symbols

<Wt1>	Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Wt2>	Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Operation

```
var address : bits(64);
var data : bits(datasize);

let dbytes : integer{} = datasize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
// When elsize is 32, the access is single-copy atomic
let ispair : boolean = elsize == 64;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                     privileged, ispair, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

AArch64_SetExclusiveMonitors(address, dbytes, accdesc);

data = Mem{datasize}(address, accdesc);
if rt_unknown then
    // ConstrainedUNPREDICTABLE case
    X{datasize}(t)  = ARBITRARY : bits(datasize);        // In this case t = t2
elsif BigEndian(accdesc.acctype) then
    X{elsize}(t)  = data[elsize +: elsize];
    X{elsize}(t2) = data[0 +: elsize];
else
    X{elsize}(t)  = data[0 +: elsize];
    X{elsize}(t2) = data[elsize +: elsize];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAXR

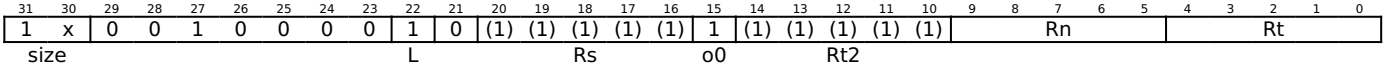
Load-acquire exclusive register

This instruction derives an address from a base register value, loads a 32-bit word or 64-bit doubleword from memory, and writes it to a register. The memory access is atomic. The PE marks the physical address being accessed as an exclusive access. This exclusive access mark is checked by Store Exclusive instructions. See *Synchronization and semaphores*.

`LDAXR` loads from memory with Acquire semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDAXR <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
LDAXR <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 8 << UInt(size);
let regsize : integer{} = if elsize == 64 then 64 else 32;
let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

AArch64_SetExclusiveMonitors(address, dbytes, accdesc);

data = Mem{elsize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAXRB

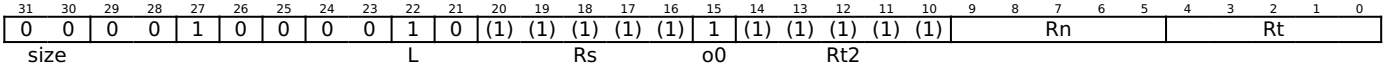
Load-acquire exclusive register byte

This instruction derives an address from a base register value, loads a byte from memory, zero-extends it and writes it to a register. The memory access is atomic. The PE marks the physical address being accessed as an exclusive access. This exclusive access mark is checked by Store Exclusive instructions. See *Synchronization and semaphores*.

`LDAXRB` loads from memory with Acquire semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDAXRB <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data : bits(8);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

AArch64_SetExclusiveMonitors(address, 1, accdesc);

data = Mem{8}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAXRH

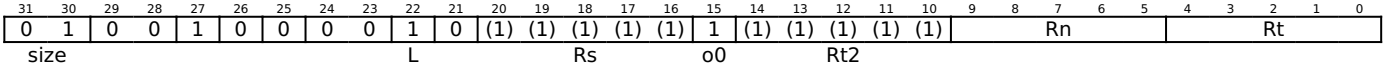
Load-acquire exclusive register halfword

This instruction derives an address from a base register value, loads a halfword from memory, zero-extends it and writes it to a register. The memory access is atomic. The PE marks the physical address being accessed as an exclusive access. This exclusive access mark is checked by Store Exclusive instructions. See *Synchronization and semaphores*.

`LDAXRH` loads from memory with Acquire semantics.

For more information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDAXRH <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data : bits(16);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

AArch64_SetExclusiveMonitors(address, 2, accdesc);

data = Mem{16}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDCLR, LDCLRA, LDCLRAL, LDCLRL

Atomic bit clear on word or doubleword

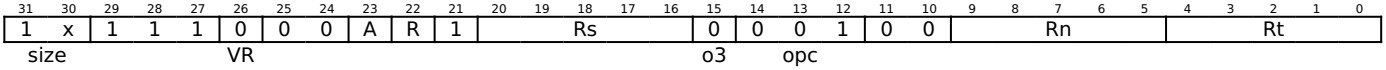
This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, performs a bitwise AND with the complement of the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR or XZR, `LDCLRA` and `LDCLRAL` load from memory with acquire semantics.
- `LDCLRL` and `LDCLRAL` store to memory with release semantics.
- `LDCLR` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STCLR, STCLRL.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDCLR <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDCLRA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDCLRAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDCLRL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDCLR <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDCLRA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDCLRAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDCLRL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STCLR, STCLRL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_BIC, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB

Atomic bit clear on byte

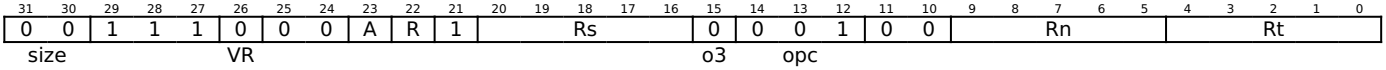
This instruction atomically loads an 8-bit byte from memory, performs a bitwise AND with the complement of the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDCLRAB` and `LDCLRALB` load from memory with acquire semantics.
- `LDCLRLB` and `LDCLRALB` store to memory with release semantics.
- `LDCLRB` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STCLRB, STCLRLB.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDCLRB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDCLRAB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDCLRALB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDCLRLB <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STCLRB, STCLRLB	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_BIC, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(8) = ARBITRARY : bits(8); // Irrelevant when not executing CAS
let value : bits(8) = X{8}(s);
let data : bits(8) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH

Atomic bit clear on halfword

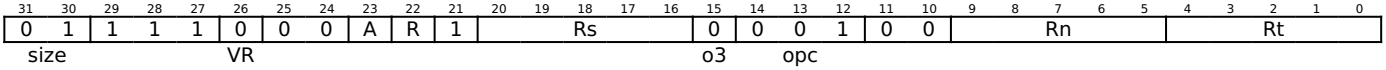
This instruction atomically loads a 16-bit halfword from memory, performs a bitwise AND with the complement of the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDCLRAH` and `LDCLRALH` load from memory with acquire semantics.
- `LDCLRLH` and `LDCLRALH` store to memory with release semantics.
- `LDCLRH` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STCLRH, STCLRLH.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDCLRH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDCLRAH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDCLRALH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDCLRLH <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STCLRH, STCLRLH	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_BIC, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(16) = ARBITRARY : bits(16); // Irrelevant when not executing CAS
let value : bits(16) = X{16}(s);
let data : bits(16) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL

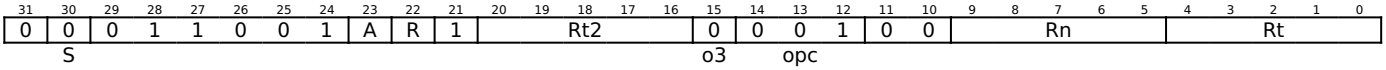
Atomic bit clear on quadword

This instruction atomically loads a 128-bit quadword from memory, performs a bitwise AND with the complement of the value held in a pair of registers on it, and stores the result back to memory. The value initially loaded from memory is returned in the same pair of registers.

- `LDCLRPA` and `LDCLRPAL` load from memory with acquire semantics.
- `LDCLRPL` and `LDCLRPAL` store to memory with release semantics.
- `LDCLRP` has neither acquire nor release semantics.

Integer

(FEAT_LSE128)



Encoding for the LDCLRP variant

Applies when `(A == 0 && R == 0)`

```
LDCLRP <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the LDCLRPA variant

Applies when `(A == 1 && R == 0)`

```
LDCLRPA <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the LDCLRPAL variant

Applies when `(A == 1 && R == 1)`

```
LDCLRPAL <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the LDCLRPL variant

Applies when `(A == 0 && R == 1)`

```
LDCLRPL <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE128) then EndOfDecode(Decode_UNDEF); end;
if Rt  == '11111' then EndOfDecode(Decode_UNDEF); end;
if Rt2 == '11111' then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LSE128OVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *128-bit Atomic Memory Operations*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
let value1 : bits(64) = X{}(t);
let value2 : bits(64) = X{}(t2);
var data : bits(128);
var store_value : bits(128);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_BIC, acquire, release,
                                                       tagchecked, privileged, t, t2, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

store_value = if BigEndian(accdesc.acctype) then value1::value2 else value2::value1;

let comparevalue : bits(128) = ARBITRARY : bits(128); // Irrelevant when not executing CAS
data = MemAtomic{128}(address, comparevalue, store_value, accdesc);

if rt_unknown then
    data = ARBITRARY : bits(128);
end;

if BigEndian(accdesc.acctype) then
    X{64}(t)  = data[127:64];
    X{64}(t2) = data[63:0];
else
    X{64}(t)  = data[63:0];
    X{64}(t2) = data[127:64];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDEOR, LDEORA, LDEORAL, LDEORL

Atomic exclusive-OR on word or doubleword

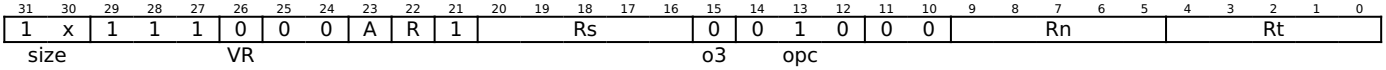
This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, performs an exclusive-OR with the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR or XZR, `LDEORA` and `LDEORAL` load from memory with acquire semantics.
- `LDEORL` and `LDEORAL` store to memory with release semantics.
- `LDEOR` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STEOR, STEORL.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDEOR <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDEORA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDEORAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDEORL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDEOR <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDEORA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDEORAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDEORL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STEOR, STEORL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_EOR, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDEORB, LDEORAB, LDEORALB, LDEORLB

Atomic exclusive-OR on byte

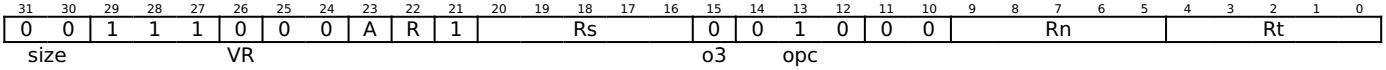
This instruction atomically loads an 8-bit byte from memory, performs an exclusive-OR with the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDEORAB` and `LDEORALB` load from memory with acquire semantics.
- `LDEORLB` and `LDEORALB` store to memory with release semantics.
- `LDEORB` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STEORB, STEORLB.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDEORB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDEORAB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDEORALB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDEORLB <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STEORB, STEORLB	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_EOR, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(8) = ARBITRARY : bits(8); // Irrelevant when not executing CAS
let value : bits(8) = X{8}(s);
let data : bits(8) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDEORH, LDEORAH, LDEORALH, LDEORLH

Atomic exclusive-OR on halfword

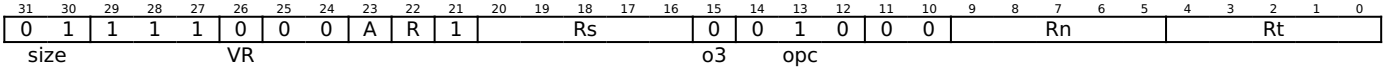
This instruction atomically loads a 16-bit halfword from memory, performs an exclusive-OR with the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDEORAH` and `LDEORALH` load from memory with acquire semantics.
- `LDEORLH` and `LDEORALH` store to memory with release semantics.
- `LDEORH` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STEORH, STEORLH.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDEORH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDEORAH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDEORALH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDEORLH <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STEORH, STEORLH	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_EOR, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(16) = ARBITRARY : bits(16); // Irrelevant when not executing CAS
let value : bits(16) = X{16}(s);
let data : bits(16) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

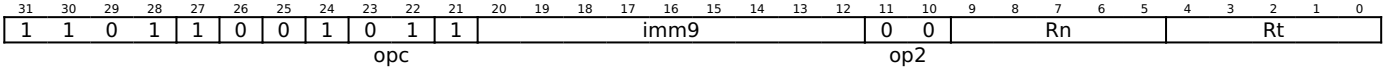
LDG

Load Allocation Tag

This instruction loads an Allocation Tag from a memory address, generates a Logical Address Tag from the Allocation Tag and merges it into the destination register. The address used for the load is calculated from the base register and an immediate signed offset scaled by the Tag Granule.

Integer

(FEAT_MTE)



Encoding

```
LDG <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate offset, a multiple of 16 in the range -4096 to 4080, defaulting to 0 and encoded in the "imm9" field.

Operation

```
var address : bits(64);
var tag : bits(4);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let stzgm : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_LOAD, stzgm, t);

address = AddressAdd(address, offset, accdesc);
address = AlignDownSize(address, TAG_GRANULE);

tag = AArch64_MemTag(address, accdesc);
X{64}(t) = AArch64_AddressWithAllocationTag(X{64}(t), tag);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDGM

Load tag multiple

This instruction reads a naturally aligned block of N Allocation Tags, where the size of N is identified in GMID_EL1.BS, and writes the Allocation Tag read from address A to the destination register at 4*A<7:4>+3:4*A<7:4>. Bits of the destination register not written with an Allocation Tag are set to 0.

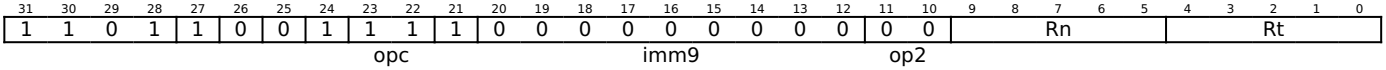
If the value of HCR_EL2.TGE is 0 and the Effective value of SCTLR2_EL1.GM0 is 0, this instruction is UNDEFINED at EL0.

If the value of HCR_EL2.TGE is 1 and the Effective value of SCTLR2_EL2.GM0 is 0, this instruction is UNDEFINED at EL0.

This instruction generates an Unchecked access.

Integer

(FEAT_MTE2)



Encoding

```
LDGM <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if PSTATE.EL == EL0 && !IsGMEnabledAtEL0() then Undefined(); end;

var data : bits(64) = Zeros{};
var address : bits(64);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let size : integer{} = 4 * 2 ^ UInt(GMID_EL1().BS);
address = AlignDownSize(address, size);
let count : integer = size >> LOG2_TAG_GRANULE;
var index : integer = UInt(address[LOG2_TAG_GRANULE+3:LOG2_TAG_GRANULE]);
let stzgm : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_LOAD, stzgm, t);

for i = 0 to count-1 do
    let tag : bits(4) = AArch64_MemTag(address, accdesc);
    data[index*:4] = tag;
    address = AddressIncrement(address, TAG_GRANULE, accdesc);
    index = index + 1;
end;

X{64}(t) = data;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDIAPP

Load-Acquire RCpc ordered pair of registers

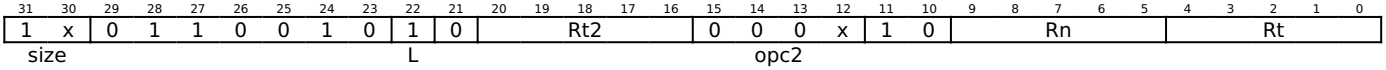
This instruction calculates an address from a base register value and an optional offset, loads two 32-bit words or two 64-bit doublewords from memory, and writes them to two registers. For information on single-copy atomicity and alignment requirements, see *Requirements for single-copy atomicity* and *Alignment of data accesses*. The instruction also has memory ordering semantics, as described in *Load-Acquire, Load-AcquirePC, and Store-Release*, with the additional requirement that:

- The Memory effects associated with Xt1/Wt1 are Ordered-before the Memory effects associated with Xt2/Wt2.
- `LDIAPP` loads from memory with Acquire semantics.

For information about addressing modes, see *Load/Store addressing modes*.

Integer

(FEAT_LRCPC3)



Encoding for the 32-bit post-index variant

Applies when `(size == 10 && opc2 == 0000)`

```
LDIAPP <Wt1>, <Wt2>, [<Xn|SP>], #8
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc2 == 0001)`

```
LDIAPP <Wt1>, <Wt2>, [<Xn|SP>]
```

Encoding for the 64-bit post-index variant

Applies when `(size == 11 && opc2 == 0000)`

```
LDIAPP <Xt1>, <Xt2>, [<Xn|SP>], #16
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc2 == 0001)`

```
LDIAPP <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC3) then EndOfDecode(Decode_UNDEF); end;
let ispair : boolean = TRUE;
let postindex : boolean = opc2[0] == '0';
var wback : boolean = opc2[0] == '0';
```

`LDIAPP` has the same CONSTRAINED UNPREDICTABLE behavior as `LDP`. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDP and LDIAPP*.

Assembler Symbols

<Wt1>	Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Wt2>	Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let scale : integer{} = 2 + UInt(size[0]);
let datasize : integer{} = 8 << scale;
let offset : integer = if opc2[0] == '0' then 2 << scale else 0;
let acqrel : boolean = TRUE;
let tagchecked : boolean = wback || n != 31;

var rt_unknown : boolean = FALSE;

var wb_unknown : boolean = FALSE;
if wback && (t == n || t2 == n) && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;        // writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;    // writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN => rt_unknown = TRUE;    // result is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);
var data1 : bits(datasize);
var data2 : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, ispair, acqrel, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let full_data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if BigEndian(accdesc.acctype) then
    data2 = full_data[(datasize-1):0];
    data1 = full_data[(2*datasize-1):datasize];
else
    data1 = full_data[(datasize-1):0];
    data2 = full_data[(2*datasize-1):datasize];
end;

if rt_unknown then
    data1 = ARBITRARY : bits(datasize);
    data2 = ARBITRARY : bits(datasize);
end;

X{datasize}(t) = data1;
X{datasize}(t2) = data2;

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

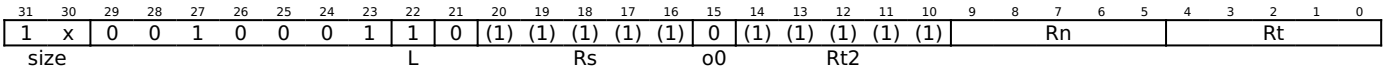
LDLAR

Load LOAcquire register

This instruction loads a 32-bit word or 64-bit doubleword from memory, and writes it to a register.
`LDLAR` loads from memory with Acquire semantics.
For more information about memory ordering semantics, see *Load LOAcquire, Store LORelease* and *Load-Acquire, Load-AcquirePC, and Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LOR)



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDLAR <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
LDLAR <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LOR) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;
let acquire : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLOR(MemOp_LOAD, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

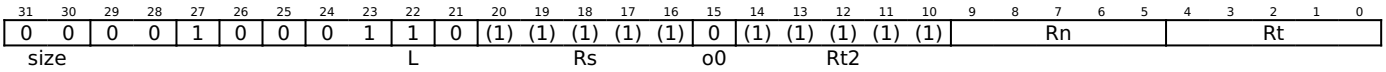
LDLARB

Load LOAcquire register byte

This instruction loads a byte from memory, zero-extends it and writes it to a register.
`LDLARB` loads from memory with Acquire semantics.
For more information about memory ordering semantics, see *Load LOAcquire, Store LORelease* and *Load-Acquire, Load-AcquirePC, and Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LOR)



Encoding

```
LDLARB <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LOR) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLOR(MemOp_LOAD, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(8) = Mem{8}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

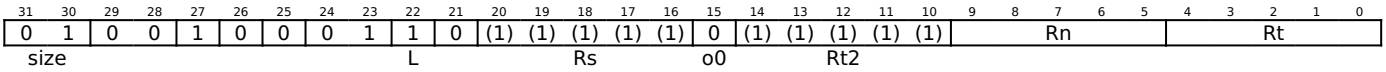
LDLARH

Load LOAcquire register halfword

This instruction loads a halfword from memory, zero-extends it, and writes it to a register.
`LDLARH` loads from memory with Acquire semantics.
For more information about memory ordering semantics, see *Load LOAcquire, Store LORelease* and *Load-Acquire, Load-AcquirePC, and Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LOR)



Encoding

```
LDLARH <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LOR) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLOR(MemOp_LOAD, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let data : bits(16) = Mem{16}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

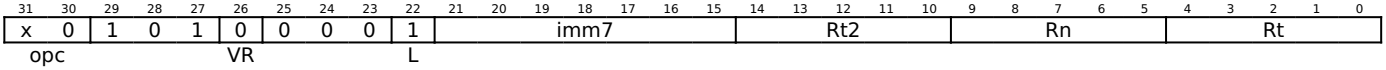
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNP

Load pair of registers, with non-temporal hint

This instruction calculates an address from a base register value and an immediate offset, loads two 32-bit words or two 64-bit doublewords from memory, and writes them to two registers.

For information about addressing modes, see *Load/Store addressing modes*. For information about non-temporal pair instructions, see *Load/Store non-temporal pair*.



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
LDNP <Wt1>, <Wt2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
LDNP <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for all variants of this encoding

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = TRUE;
let scale : integer{} = 2 + UInt(opc[1]);
let datasize : integer{} = 8 << scale;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // Result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDNP*.

Assembler Symbols

<Wt1>	Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Wt2>	Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "32-bit" variant: is the optional signed immediate byte offset, a multiple of 4 in the range -256 to 252, defaulting to 0 and encoded in the "imm7" field as <imm>/4. For the "64-bit" variant: is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8.
<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Operation

```
var address : bits(64);
let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, ispair, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);
let data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if rt_unknown then
    X{datasize}(t) = ARBITRARY : bits(datasize);
elsif BigEndian(accdesc.acctype) then
    X{datasize}(t2) = data[(datasize-1):0];
    X{datasize}(t)  = data[(2*datasize-1):datasize];
else
    X{datasize}(t)  = data[(datasize-1):0];
    X{datasize}(t2) = data[(2*datasize-1):datasize];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

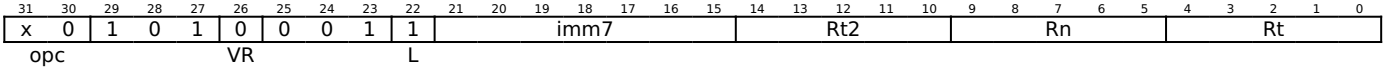
LDP

Load pair of registers

This instruction calculates an address from a base register value and an immediate offset, loads two 32-bit words or two 64-bit doublewords from memory, and writes them to two registers. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index



Encoding for the 32-bit variant

Applies when (opc == 00)

```
LDP <Wt1>, <Wt2>, [<Xn|SP>], #<imm>
```

Encoding for the 64-bit variant

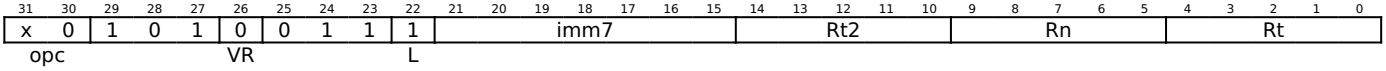
Applies when (opc == 10)

```
LDP <Xt1>, <Xt2>, [<Xn|SP>], #<imm>
```

Decode for all variants of this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index



Encoding for the 32-bit variant

Applies when (opc == 00)

```
LDP <Wt1>, <Wt2>, [<Xn|SP>, #<imm>]!
```

Encoding for the 64-bit variant

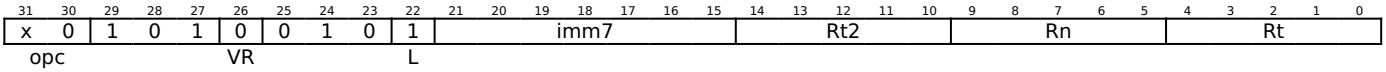
Applies when (opc == 10)

```
LDP <Xt1>, <Xt2>, [<Xn|SP>, #<imm>]!
```

Decode for all variants of this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset



Encoding for the 32-bit variant

Applies when (opc == 00)

```
LDP <Wt1>, <Wt2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 64-bit variant

Applies when (opc == 10)

```
LDP <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for all variants of this encoding

```
var wback : boolean = FALSE;
let postindex : boolean = FALSE;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDP*.

Assembler Symbols

<Wt1>	Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Wt2>	Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Post-index 32-bit" and "Pre-index 32-bit" variants: is the signed immediate byte offset, a multiple of 4 in the range -256 to 252, encoded in the "imm7" field as <imm>/4. For the "Post-index 64-bit" and "Pre-index 64-bit" variants: is the signed immediate byte offset, a multiple of 8 in the range -512 to 504, encoded in the "imm7" field as <imm>/8. For the "Signed offset 32-bit" variant: is the optional signed immediate byte offset, a multiple of 4 in the range -256 to 252, defaulting to 0 and encoded in the "imm7" field as <imm>/4. For the "Signed offset 64-bit" variant: is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8.
<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let scale : integer{} = 2 + UInt(opc[1]);
let datasize : integer{} = 8 << scale;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = wback || n != 31;

var rt_unknown : boolean = FALSE;
var wb_unknown : boolean = FALSE;

if wback && (t == n || t2 == n) && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;        // Writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;    // Writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // Result is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);
var data1 : bits(datasize);
var data2 : bits(datasize);
let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, ispair, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if rt_unknown then
    data1 = ARBITRARY : bits(datasize);
    data2 = ARBITRARY : bits(datasize);
elsif BigEndian(accdesc.acctype) then
    data2 = data[(datasize-1):0];
    data1 = data[(2*datasize-1):datasize];
else
    data1 = data[(datasize-1):0];
    data2 = data[(2*datasize-1):datasize];
end;

X{datasize}(t) = data1;
X{datasize}(t2) = data2;

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

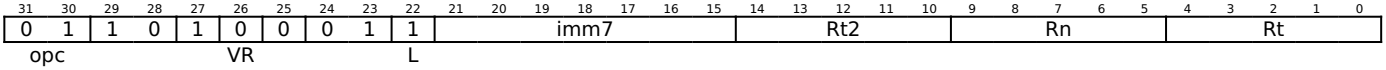
LDPSW

Load pair of registers signed word

This instruction calculates an address from a base register value and an immediate offset, loads two 32-bit words from memory, sign-extends them, and writes them to two registers. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index



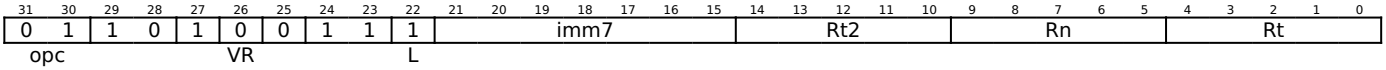
Encoding

```
LDPSW <Xt1>, <Xt2>, [<Xn|SP>], #<imm>
```

Decode for this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index



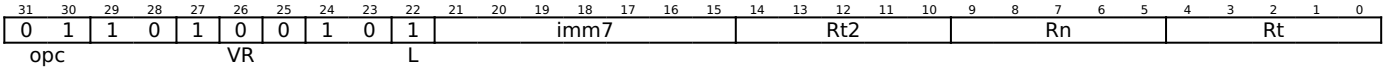
Encoding

```
LDPSW <Xt1>, <Xt2>, [<Xn|SP>, #<imm>]!
```

Decode for this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset



Encoding

```
LDPSW <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
var wback : boolean = FALSE;
let postindex : boolean = FALSE;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDPSW*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Post-index" and "Pre-index" variants: is the signed immediate byte offset, a multiple of 4 in the range -256 to 252, encoded in the "imm7" field as <imm>/4. For the "Signed offset" variant: is the optional signed immediate byte offset, a multiple of 4 in the range -256 to 252, defaulting to 0 and encoded in the "imm7" field as <imm>/4.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let scale : integer{} = 2 + UInt(opc[1]);
let datasize : integer{} = 32;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = wback || n != 31;

var rt_unknown : boolean = FALSE;
var wb_unknown : boolean = FALSE;

if wback && (t == n || t2 == n) && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;        // Writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;    // Writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // Result is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);
var data1 : bits(datasize);
var data2 : bits(datasize);
let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, ispair, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if rt_unknown then
    data1 = ARBITRARY : bits(datasize);
    data2 = ARBITRARY : bits(datasize);
elsif BigEndian(accdesc.acctype) then
    data2 = data[(datasize-1):0];
    data1 = data[(2*datasize-1):datasize];
else
    data1 = data[(datasize-1):0];
    data2 = data[(2*datasize-1):datasize];
end;

X{64}(t) = SignExtend{64}(data1);
X{64}(t2) = SignExtend{64}(data2);

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

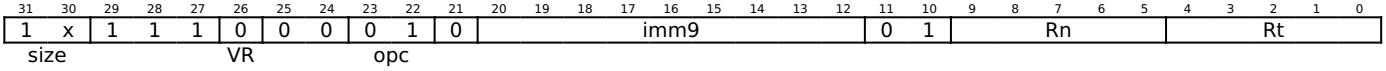
LDR (immediate)

Load register (immediate)

This instruction loads a word or doubleword from memory and writes it to a register. The address that is used for the load is calculated from a base register and an immediate offset. For information about addressing modes, see *Load/Store addressing modes*. The Unsigned offset variant scales the immediate offset value by the size of the value accessed before adding it to the base register value.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDR <Wt>, [<Xn|SP>], #<simm>
```

Encoding for the 64-bit variant

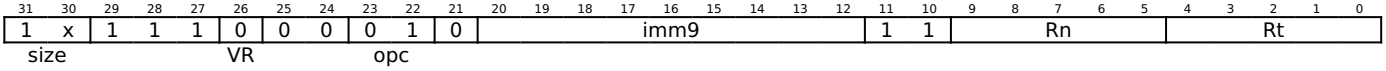
Applies when `(size == 11)`

```
LDR <Xt>, [<Xn|SP>], #<simm>
```

Decode for all variants of this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = TRUE;
let scale : integer{} = UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDR <Wt>, [<Xn|SP>, #<simm>]!
```

Encoding for the 64-bit variant

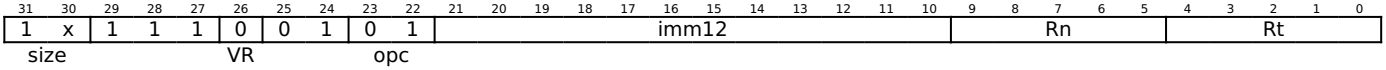
Applies when `(size == 11)`

```
LDR <Xt>, [<Xn|SP>, #<simm>]!
```

Decode for all variants of this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = FALSE;
let scale : integer{} = UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDR <Wt>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
LDR <Xt>, [<Xn|SP>{, #<pimm>}]
```

Decode for all variants of this encoding

```
var wback : boolean = FALSE;
let postindex : boolean = FALSE;
let scale : integer{} = UInt(size);
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), scale);
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDR (immediate)*.

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<pimm>	For the "32-bit" variant: is the optional positive immediate byte offset, a multiple of 4 in the range 0 to 16380, defaulting to 0 and encoded in the "imm12" field as <pimm>/4. For the "64-bit" variant: is the optional positive immediate byte offset, a multiple of 8 in the range 0 to 32760, defaulting to 0 and encoded in the "imm12" field as <pimm>/8.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << scale;
let regsize : integer{} = if datasize == 64 then 64 else 32;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var c : Constraint;
var wb_unknown : boolean = FALSE;
if wback && n == t && n != 31 then
    c = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;       // Writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;   // Writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

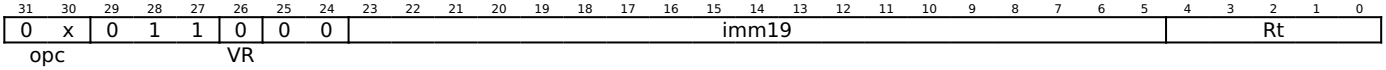
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDR (literal)

Load register (literal)

This instruction calculates an address from the PC value and an immediate offset, loads a word from memory, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
LDR <Wt>, <label>
```

Encoding for the 64-bit variant

Applies when `(opc == 01)`

```
LDR <Xt>, <label>
```

Decode for all variants of this encoding

```
let t : integer{} = UInt(Rt);
let size : integer{} = 4 << UInt(opc[0]);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = FALSE;

let offset : bits(64) = SignExtend{}(imm19::'00');
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<label>	Is the program label from which the data is to be loaded. Its offset from the address of this instruction, in the range +/-1MB, is encoded as "imm19" times 4.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Operation

```
let address : bits(64) = PC64() + offset;
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

X{size * 8}(t) = Mem{size * 8}(address, accdesc);
```

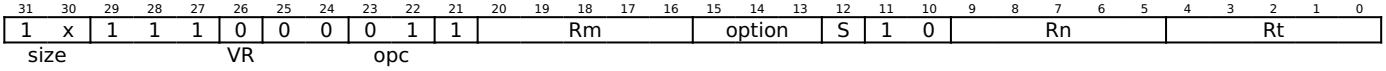
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDR (register)

Load register (register)

This instruction calculates an address from a base register value and an offset register value, loads a word from memory, and writes it to a register. The offset register value can optionally be shifted and extended. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDR <Wt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
LDR <Xt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Decode for all variants of this encoding

```
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
let extend_type : ExtendType = DecodeRegExtend(option);
let scale : integer{} = UInt(size);
let shift : integer{} = if S == '1' then scale else 0;
```

Assembler Symbols

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Wm> When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.

<Xm> When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.

<extend> Is the index extend/shift specifier, defaulting to LSL, and which must be omitted for the LSL option when <amount> is omitted, encoded in “option”:

option	<extend>
010	UXTW
011	LSL
110	SXTW
111	SXTX

<amount> For the "32-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#2

For the "64-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#3

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 8 << scale;
let regsize : integer{} = if datasize == 64 then 64 else 32;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDRAA, LDRAB

Load register, with pointer authentication

This instruction authenticates an address from a base register using a modifier of zero and the specified key, adds an immediate offset to the authenticated address, and loads a 64-bit doubleword from memory at this resulting address into a register.

Key A is used for `LDRAA`. Key B is used for `LDRAB`.

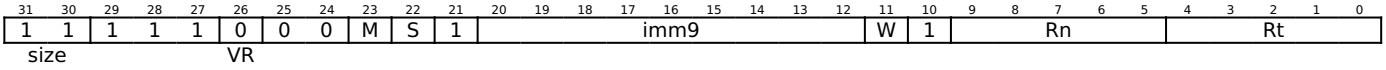
If the authentication passes, the PE behaves the same as for an `LDR` instruction. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.

The authenticated address is not written back to the base register, unless the pre-indexed variant of the instruction is used. In this case, the address that is written back to the base register does not include the pointer authentication code.

For information about addressing modes, see *Load/Store addressing modes*.

PAC

(FEAT_PAuth)



Encoding for the Key A, offset variant

Applies when `(M == 0 && W == 0)`

```
LDRAA <Xt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the Key A, pre-indexed variant

Applies when `(M == 0 && W == 1)`

```
LDRAA <Xt>, [<Xn|SP>{, #<simm>}]!
```

Encoding for the Key B, offset variant

Applies when `(M == 1 && W == 0)`

```
LDRAB <Xt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the Key B, pre-indexed variant

Applies when `(M == 1 && W == 1)`

```
LDRAB <Xt>, [<Xn|SP>{, #<simm>}]!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
var wback : boolean = W == '1';
let use_key_a : boolean = M == '0';
let S10 : bits(10) = S::imm9;
let offset : bits(64) = LSL(SignExtend{64}(S10), 3);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var wb_unknown : boolean = FALSE;
if wback && n == t && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;       // writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;   // writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, a multiple of 8 in the range -4096 to 4088, defaulting to 0 and encoded in the "S:imm9" field as <simm>/8.

Operation

```
var address : bits(64);
let privileged : boolean = PSTATE.EL != EL0;
let auth_then_branch : boolean = TRUE;

let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);
if n == 31 then
    address = SP{64}();
else
    address = X{64}(n);
end;

if use_key_a then
    address = AuthDA(address, X{64}(31), auth_then_branch);
else
    address = AuthDB(address, X{64}(31), auth_then_branch);
end;

if n == 31 then
    CheckSPAlignment();
end;

address = AddressAdd(address, offset, accdesc);
X{64}(t) = Mem{64}(address, accdesc);

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

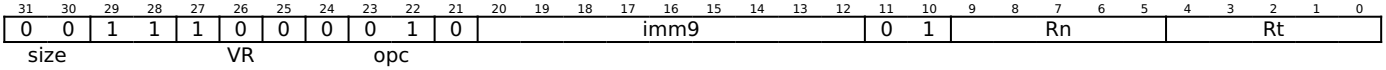
LDRB (immediate)

Load register byte (immediate)

This instruction loads a byte from memory, zero-extends it, and writes the result to a register. The address that is used for the load is calculated from a base register and an immediate offset. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index



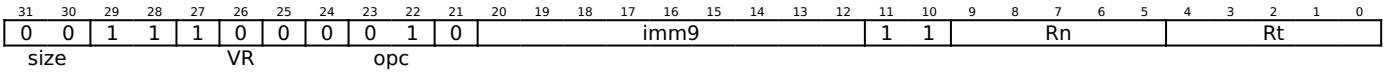
Encoding

```
LDRB <Wt>, [<Xn|SP>], #<simm>
```

Decode for this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = TRUE;
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index



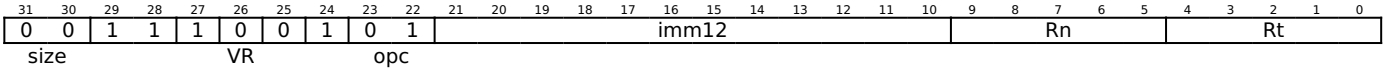
Encoding

```
LDRB <Wt>, [<Xn|SP>, #<simm>]!
```

Decode for this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = FALSE;
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset



Encoding

```
LDRB <Wt>, [<Xn|SP>{, #<pimm>}]
```

Decode for this encoding

```
var wback : boolean = FALSE;
let postindex : boolean = FALSE;
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), 0);
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDRB (immediate)*.

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<pimm>	Is the optional positive immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var c : Constraint;
var wb_unknown : boolean = FALSE;
if wback && n == t && n != 31 then
    c = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;       // Writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;   // Writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(8) = Mem{8}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

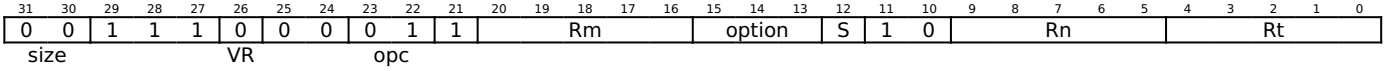
This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDRB (register)

Load register byte (register)

This instruction calculates an address from a base register value and an offset register value, loads a byte from memory, zero-extends it, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the Extended register variant

Applies when `(option != 011)`

```
LDRB <Wt>, [<Xn|SP>, (<Wm>|<Xm>), <extend> {<amount>}]
```

Encoding for the Shifted register variant

Applies when `(option == 011)`

```
LDRB <Wt>, [<Xn|SP>, <Xm>{, LSL <amount>}]
```

Decode for all variants of this encoding

```
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
let extend_type : ExtendType = DecodeRegExtend(option);
let shift : integer{} = 0;
```

Assembler Symbols

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Wm> When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.

<Xm> When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.

<extend> Is the index extend specifier, encoded in “option”:

option	<extend>
010	UXTW
110	SXTW
111	SXTX

<amount> Is the index shift amount, it must be #0, encoded in "S" as 0 if omitted, or as 1 if present.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(8) = Mem{8}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

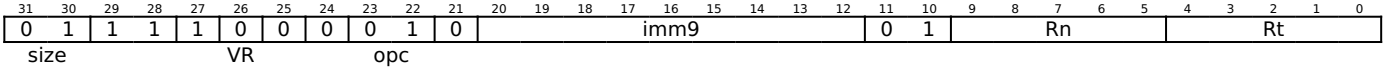
LDRH (immediate)

Load register halfword (immediate)

This instruction loads a halfword from memory, zero-extends it, and writes the result to a register. The address that is used for the load is calculated from a base register and an immediate offset. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index



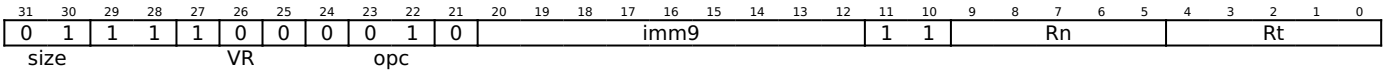
Encoding

```
LDRH <Wt>, [<Xn|SP>], #<simm>
```

Decode for this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = TRUE;
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index



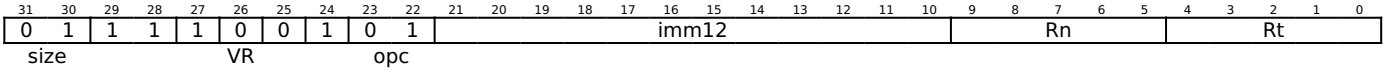
Encoding

```
LDRH <Wt>, [<Xn|SP>, #<simm>]!
```

Decode for this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = FALSE;
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset



Encoding

```
LDRH <Wt>, [<Xn|SP>{, #<pimm>}]
```

Decode for this encoding

```
var wback : boolean = FALSE;
let postindex : boolean = FALSE;
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), 1);
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDRH (immediate)*.

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<pimm>	Is the optional positive immediate byte offset, a multiple of 2 in the range 0 to 8190, defaulting to 0 and encoded in the "imm12" field as <pimm>/2.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var c : Constraint;
var wb_unknown : boolean = FALSE;
if wback && n == t && n != 31 then
    c = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;       // Writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;   // Writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(16) = Mem{16}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

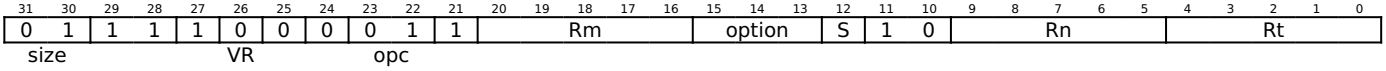
This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDRH (register)

Load register halfword (register)

This instruction calculates an address from a base register value and an offset register value, loads a halfword from memory, zero-extends it, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDRH <Wt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Decode for this encoding

```
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
let extend_type : ExtendType = DecodeRegExtend(option);
let shift : integer{} = if S == '1' then 1 else 0;
```

Assembler Symbols

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Wm> When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.

<Xm> When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.

<extend> Is the index extend/shift specifier, defaulting to LSL, and which must be omitted for the LSL option when <amount> is omitted, encoded in “option”:

option	<extend>
010	UXTW
011	LSL
110	SXTW
111	SXTX

<amount> Is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#1

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(16) = Mem{16}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

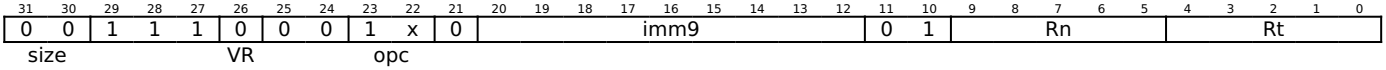
LDRSB (immediate)

Load register signed byte (immediate)

This instruction loads a byte from memory, sign-extends it to either 32 bits or 64 bits, and writes the result to a register. The address that is used for the load is calculated from a base register and an immediate offset. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDRSB <Wt>, [<Xn|SP>], #<simm>
```

Encoding for the 64-bit variant

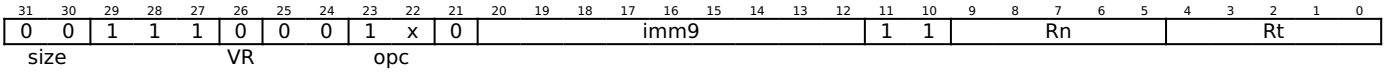
Applies when `(opc == 10)`

```
LDRSB <Xt>, [<Xn|SP>], #<simm>
```

Decode for all variants of this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = TRUE;
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDRSB <Wt>, [<Xn|SP>, #<simm>]!
```

Encoding for the 64-bit variant

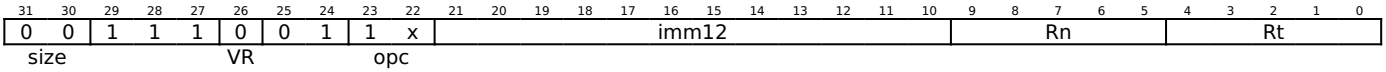
Applies when `(opc == 10)`

```
LDRSB <Xt>, [<Xn|SP>, #<simm>]!
```

Decode for all variants of this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = FALSE;
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDRSB <Wt>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
LDRSB <Xt>, [<Xn|SP>{, #<pimm>}]
```

Decode for all variants of this encoding

```
var wback : boolean = FALSE;
let postindex : boolean = FALSE;
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), 0);
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDRSB (immediate)*.

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<pimm>	Is the optional positive immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let regsize : integer{} = 64 >> UInt(opc[0]);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var c : Constraint;
var wb_unknown : boolean = FALSE;
if wback && n == t && n != 31 then
    c = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;       // Writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;   // Writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(8) = Mem{8}(address, accdesc);
X{regsize}(t) = SignExtend{regsize}(data);

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

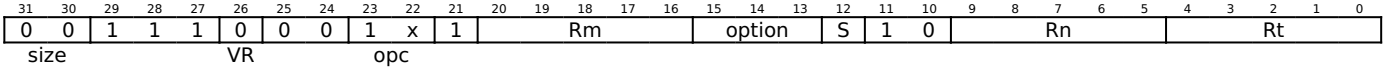
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDRSB (register)

Load register signed byte (register)

This instruction calculates an address from a base register value and an offset register value, loads a byte from memory, sign-extends it, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit with extended register offset variant

Applies when `(opc == 11 && option != 011)`

```
LDRSB <Wt>, [<Xn|SP>, (<Wm>|<Xm>), <extend> {<amount>}]
```

Encoding for the 32-bit with shifted register offset variant

Applies when `(opc == 11 && option == 011)`

```
LDRSB <Wt>, [<Xn|SP>, <Xm>{, LSL <amount>}]
```

Encoding for the 64-bit with extended register offset variant

Applies when `(opc == 10 && option != 011)`

```
LDRSB <Xt>, [<Xn|SP>, (<Wm>|<Xm>), <extend> {<amount>}]
```

Encoding for the 64-bit with shifted register offset variant

Applies when `(opc == 10 && option == 011)`

```
LDRSB <Xt>, [<Xn|SP>, <Xm>{, LSL <amount>}]
```

Decode for all variants of this encoding

```
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
let extend_type : ExtendType = DecodeRegExtend(option);
let shift : integer{} = 0;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Wm>	When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.
<Xm>	When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.
<extend>	Is the index extend specifier, encoded in “option”:

option	<extend>
010	UXTW
110	SXTW
111	SXTX

<amount>	Is the index shift amount, it must be #0, encoded in "S" as 0 if omitted, or as 1 if present.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let regsize : integer{} = 64 >> UInt(opc[0]);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(8) = Mem{8}(address, accdesc);
X{regsize}(t) = SignExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

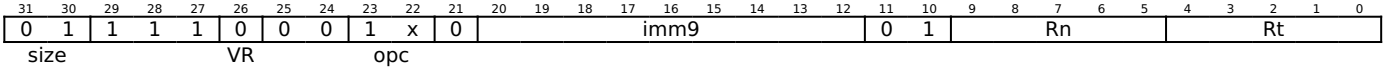
LDRSH (immediate)

Load register signed halfword (immediate)

This instruction loads a halfword from memory, sign-extends it to 32 bits or 64 bits, and writes the result to a register. The address that is used for the load is calculated from a base register and an immediate offset. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDRSH <Wt>, [<Xn|SP>], #<simm>
```

Encoding for the 64-bit variant

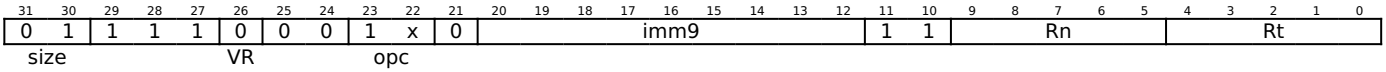
Applies when `(opc == 10)`

```
LDRSH <Xt>, [<Xn|SP>], #<simm>
```

Decode for all variants of this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = TRUE;
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDRSH <Wt>, [<Xn|SP>, #<simm>]!
```

Encoding for the 64-bit variant

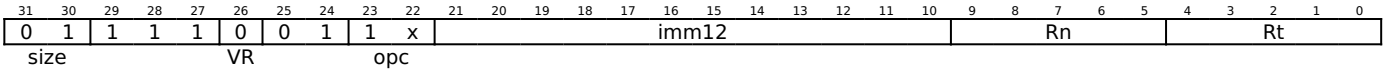
Applies when `(opc == 10)`

```
LDRSH <Xt>, [<Xn|SP>, #<simm>]!
```

Decode for all variants of this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = FALSE;
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDRSH <Wt>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
LDRSH <Xt>, [<Xn|SP>{, #<pimm>}]
```

Decode for all variants of this encoding

```
var wback : boolean = FALSE;
let postindex : boolean = FALSE;
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), 1);
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDRSH (immediate)*.

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<pimm>	Is the optional positive immediate byte offset, a multiple of 2 in the range 0 to 8190, defaulting to 0 and encoded in the "imm12" field as <pimm>/2.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let regsize : integer{} = 64 >> UInt(opc[0]);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var c : Constraint;
var wb_unknown : boolean = FALSE;
if wback && n == t && n != 31 then
    c = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;       // Writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;   // Writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(16) = Mem{16}(address, accdesc);
X{regsize}(t) = SignExtend{regsize}(data);

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

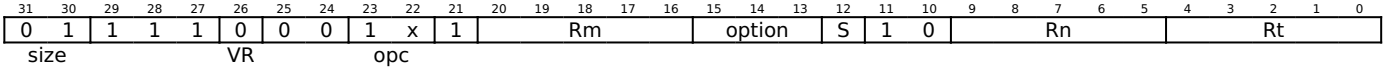
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDRSH (register)

Load register signed halfword (register)

This instruction calculates an address from a base register value and an offset register value, loads a halfword from memory, sign-extends it, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDRSH <Wt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
LDRSH <Xt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Decode for all variants of this encoding

```
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
let extend_type : ExtendType = DecodeRegExtend(option);
let shift : integer{} = if S == '1' then 1 else 0;
```

Assembler Symbols

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Wm> When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.

<Xm> When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.

<extend> Is the index extend/shift specifier, defaulting to LSL, and which must be omitted for the LSL option when <amount> is omitted, encoded in “option”:

option	<extend>
010	UXTW
011	LSL
110	SXTW
111	SXTX

<amount> Is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#1

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let regsize : integer{} = 64 >> UInt(opc[0]);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(16) = Mem{16}(address, accdesc);
X{regsize}(t) = SignExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

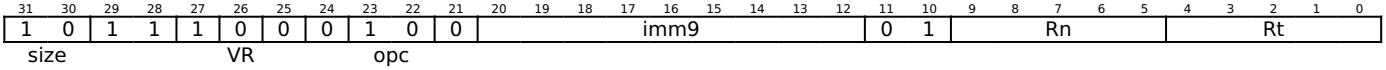
LDRSW (immediate)

Load register signed word (immediate)

This instruction loads a word from memory, sign-extends it to 64 bits, and writes the result to a register. The address that is used for the load is calculated from a base register and an immediate offset. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index



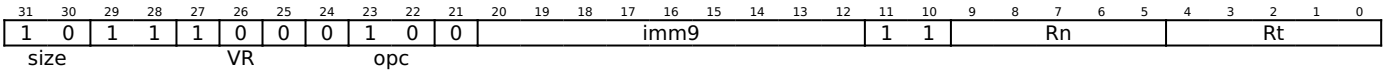
Encoding

```
LDRSW <Xt>, [<Xn|SP>], #<simm>
```

Decode for this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = TRUE;
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index



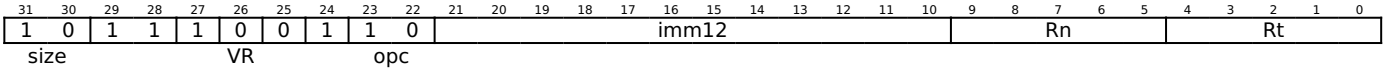
Encoding

```
LDRSW <Xt>, [<Xn|SP>, #<simm>]!
```

Decode for this encoding

```
var wback : boolean = TRUE;
let postindex : boolean = FALSE;
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset



Encoding

```
LDRSW <Xt>, [<Xn|SP>{, #<pimm>}]
```

Decode for this encoding

```
var wback : boolean = FALSE;
let postindex : boolean = FALSE;
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), 2);
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDRSW (immediate)*.

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<pimm>	Is the optional positive immediate byte offset, a multiple of 4 in the range 0 to 16380, defaulting to 0 and encoded in the "imm12" field as <pimm>/4.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var c : Constraint;
var wb_unknown : boolean = FALSE;
if wback && n == t && n != 31 then
    c = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;       // Writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;   // Writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(32) = Mem{32}(address, accdesc);
X{64}(t) = SignExtend{64}(data);

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

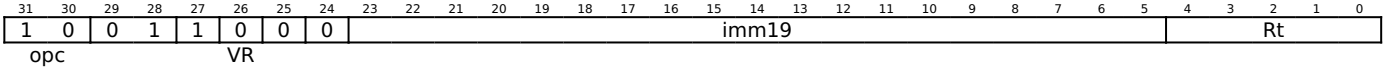
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDRSW (literal)

Load register signed word (literal)

This instruction calculates an address from the PC value and an immediate offset, loads a word from memory, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDRSW <Xt>, <label>
```

Decode for this encoding

```
let t : integer{} = UInt(Rt);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = FALSE;

let offset : bits(64) = SignExtend{}(imm19::'00');
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<label>	Is the program label from which the data is to be loaded. Its offset from the address of this instruction, in the range +/-1MB, is encoded as "imm19" times 4.

Operation

```
let address : bits(64) = PC64() + offset;
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

let data : bits(32) = Mem{32}(address, accdesc);
X{64}(t) = SignExtend{64}(data);
```

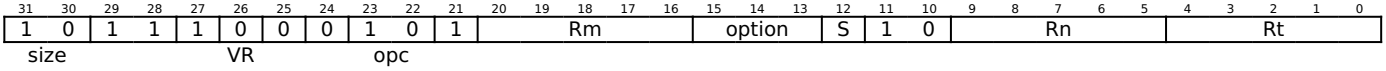
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDRSW (register)

Load register signed word (register)

This instruction calculates an address from a base register value and an offset register value, loads a word from memory, sign-extends it to form a 64-bit value, and writes it to a register. The offset register value can be shifted left by 0 or 2 bits. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDRSW <Xt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Decode for this encoding

```
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
let extend_type : ExtendType = DecodeRegExtend(option);
let shift : integer{} = if S == '1' then 2 else 0;
```

Assembler Symbols

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Wm> When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.

<Xm> When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.

<extend> Is the index extend/shift specifier, defaulting to LSL, and which must be omitted for the LSL option when <amount> is omitted, encoded in “option”:

option	<extend>
010	UXTW
011	LSL
110	SXTW
111	SXTX

<amount> Is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#2

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(32) = Mem{32}(address, accdesc);
X{64}(t) = SignExtend{64}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDSET, LDSETA, LDSETAL, LDSETL

Atomic bit set on word or doubleword

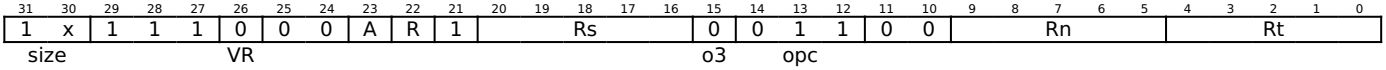
This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, performs a bitwise OR with the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR or XZR, `LDSETA` and `LDSETAL` load from memory with acquire semantics.
- `LDSETL` and `LDSETAL` store to memory with release semantics.
- `LDSET` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STSET, STSETL.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDSET <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDSETA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDSETAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDSETL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDSET <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDSETA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDSETAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDSETL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STSET, STSETL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_ORR, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDSETB, LDSETAB, LDSETALB, LDSETLB

Atomic bit set on byte

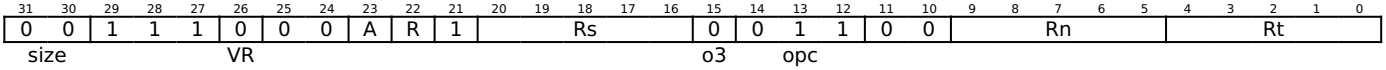
This instruction atomically loads an 8-bit byte from memory, performs a bitwise OR with the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDSETAB` and `LDSETALB` load from memory with acquire semantics.
- `LDSETLB` and `LDSETALB` store to memory with release semantics.
- `LDSETB` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STSETB, STSETLB.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDSETB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDSETAB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDSETALB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDSETLB <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STSETB, STSETLB	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_ORR, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(8) = ARBITRARY : bits(8); // Irrelevant when not executing CAS
let value : bits(8) = X{8}(s);
let data : bits(8) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDSETH, LDSETAH, LDSETALH, LDSETLH

Atomic bit set on halfword

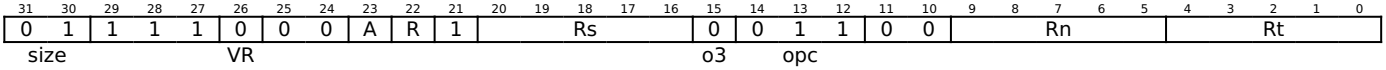
This instruction atomically loads a 16-bit halfword from memory, performs a bitwise OR with the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDSETAH` and `LDSETALH` load from memory with acquire semantics.
- `LDSETLH` and `LDSETALH` store to memory with release semantics.
- `LDSETH` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STSETH, STSETLH.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDSETH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDSETAH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDSETALH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDSETLH <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STSETH, STSETLH	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_ORR, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(16) = ARBITRARY : bits(16); // Irrelevant when not executing CAS
let value : bits(16) = X{16}(s);
let data : bits(16) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDSETP, LDSETPA, LDSETPAL, LDSETPL

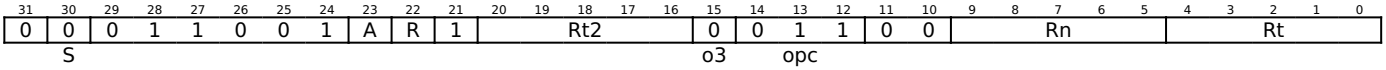
Atomic bit set on quadword

This instruction atomically loads a 128-bit quadword from memory, performs a bitwise OR with the value held in a pair of registers on it, and stores the result back to memory. The value initially loaded from memory is returned in the same pair of registers.

- `LDSETPA` and `LDSETPAL` load from memory with acquire semantics.
- `LDSETPL` and `LDSETPAL` store to memory with release semantics.
- `LDSETP` has neither acquire nor release semantics.

Integer

(FEAT_LSE128)



Encoding for the LDSETP variant

Applies when `(A == 0 && R == 0)`

```
LDSETP <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the LDSETPA variant

Applies when `(A == 1 && R == 0)`

```
LDSETPA <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the LDSETPAL variant

Applies when `(A == 1 && R == 1)`

```
LDSETPAL <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the LDSETPL variant

Applies when `(A == 0 && R == 1)`

```
LDSETPL <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE128) then EndOfDecode(Decode_UNDEF); end;
if Rt  == '11111' then EndOfDecode(Decode_UNDEF); end;
if Rt2 == '11111' then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LSE128OVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *128-bit Atomic Memory Operations*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
let value1 : bits(64) = X{}(t);
let value2 : bits(64) = X{}(t2);
var data : bits(128);
var store_value : bits(128);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_ORR, acquire, release,
                                                       tagchecked, privileged, t, t2, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

store_value = if BigEndian(accdesc.acctype) then value1::value2 else value2::value1;

let comparevalue : bits(128) = ARBITRARY : bits(128); // Irrelevant when not executing CAS
data = MemAtomic{128}(address, comparevalue, store_value, accdesc);

if rt_unknown then
    data = ARBITRARY : bits(128);
end;

if BigEndian(accdesc.acctype) then
    X{64}(t)  = data[127:64];
    X{64}(t2) = data[63:0];
else
    X{64}(t)  = data[63:0];
    X{64}(t2) = data[127:64];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL

Atomic signed maximum on word or doubleword

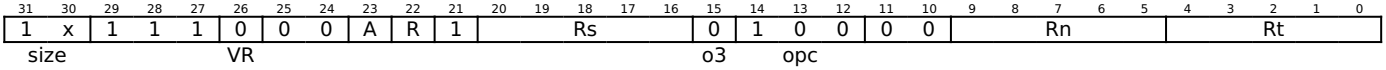
This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as signed numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR or XZR, `LDSMAXA` and `LDSMAXAL` load from memory with acquire semantics.
- `LDSMAXL` and `LDSMAXAL` store to memory with release semantics.
- `LDSMAX` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STSMAX, STSMAXL.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDSMAX <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDSMAXA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDSMAXAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDSMAXL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDSMAX <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDSMAXA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDSMAXAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDSMAXL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STSMAX, STSMAXL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SMAX, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB

Atomic signed maximum on byte

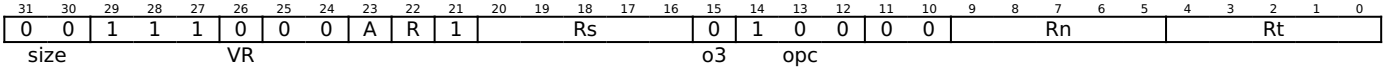
This instruction atomically loads an 8-bit byte from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as signed numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDSMAXAB` and `LDSMAXALB` load from memory with acquire semantics.
- `LDSMAXLB` and `LDSMAXALB` store to memory with release semantics.
- `LDSMAXB` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STSMAXB, STSMAXLB.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDSMAXB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDSMAXAB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDSMAXALB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDSMAXLB <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STSMAXB, STSMAXLB	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SMAX, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(8) = ARBITRARY : bits(8); // Irrelevant when not executing CAS
let value : bits(8) = X{8}(s);
let data : bits(8) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH

Atomic signed maximum on halfword

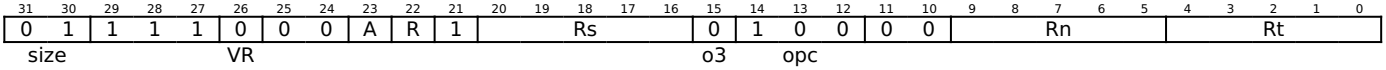
This instruction atomically loads a 16-bit halfword from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as signed numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDSMAXAH` and `LDSMAXALH` load from memory with acquire semantics.
- `LDSMAXLH` and `LDSMAXALH` store to memory with release semantics.
- `LDSMAXH` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STSMAXH, STSMAXLH.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDSMAXH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDSMAXAH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDSMAXALH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDSMAXLH <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STSMAXH, STSMAXLH	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SMAX, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(16) = ARBITRARY : bits(16); // Irrelevant when not executing CAS
let value : bits(16) = X{16}(s);
let data : bits(16) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDSMIN, LDSMINA, LDSMINAL, LDSMINL

Atomic signed minimum on word or doubleword

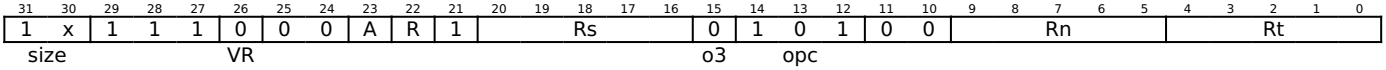
This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as signed numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR or XZR, `LDSMINA` and `LDSMINAL` load from memory with acquire semantics.
- `LDSMINL` and `LDSMINAL` store to memory with release semantics.
- `LDSMIN` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STSMIN, STSMINL.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDSMIN <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDSMINA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDSMINAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDSMINL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDSMIN <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDSMINA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDSMINAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDSMINL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STSMIN, STSMINL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SMIN, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB

Atomic signed minimum on byte

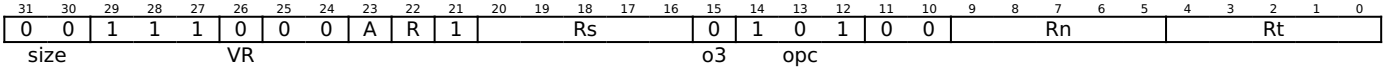
This instruction atomically loads an 8-bit byte from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as signed numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDSMINAB` and `LDSMINALB` load from memory with acquire semantics.
- `LDSMINLB` and `LDSMINALB` store to memory with release semantics.
- `LDSMINB` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STSMINB, STSMINLB.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDSMINB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDSMINAB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDSMINALB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDSMINLB <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STSMINB, STSMINLB	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SMIN, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(8) = ARBITRARY : bits(8); // Irrelevant when not executing CAS
let value : bits(8) = X{8}(s);
let data : bits(8) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH

Atomic signed minimum on halfword

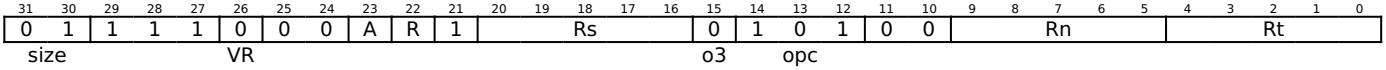
This instruction atomically loads a 16-bit halfword from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as signed numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDSMINAH` and `LDSMINALH` load from memory with acquire semantics.
- `LDSMINLH` and `LDSMINALH` store to memory with release semantics.
- `LDSMINH` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STSMINH, STSMINLH.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDSMINH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDSMINAH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDSMINALH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDSMINLH <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STSMINH, STSMINLH	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SMIN, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(16) = ARBITRARY : bits(16); // Irrelevant when not executing CAS
let value : bits(16) = X{16}(s);
let data : bits(16) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTADD, LDTADDA, LDTADDAL, LDTADDL

Atomic add unprivileged

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, adds the value held in a register to it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR, or XZR, `LDTADDA` and `LDTADDAL` load from memory with acquire semantics.
- `LDTADDL` and `LDTADDAL` store to memory with release semantics.
- `LDTADD` has neither acquire nor release semantics.

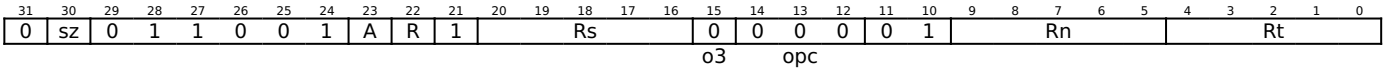
For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.
This instruction is used by the alias STTADD, STTADDL.

Integer

(FEAT_LSUI)



Encoding for the 32-bit no memory ordering variant

Applies when `(sz == 0 && A == 0 && R == 0)`

```
LDTADD <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(sz == 0 && A == 1 && R == 0)`

```
LDTADDA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(sz == 0 && A == 1 && R == 1)`

```
LDTADDAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(sz == 0 && A == 0 && R == 1)`

```
LDTADDL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(sz == 1 && A == 0 && R == 0)`

```
LDTADD <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(sz == 1 && A == 1 && R == 0)`

```
LDTADDA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(sz == 1 && A == 1 && R == 1)`

```
LDTADDAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(sz == 1 && A == 0 && R == 1)`

```
LDTADDL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 32 << UInt(sz);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STTADD, STTADDL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_ADD, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL

Atomic bit clear unprivileged

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, performs a bitwise AND with the complement of the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR, or XZR, `LDTCLRA` and `LDTCLRAL` load from memory with acquire semantics.
- `LDTCLRL` and `LDTCLRAL` store to memory with release semantics.
- `LDTCLR` has neither acquire nor release semantics.

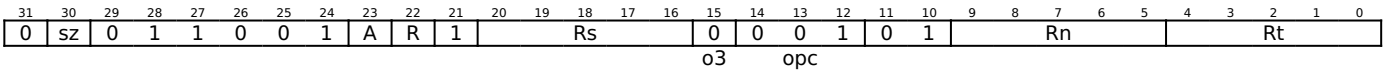
For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.
This instruction is used by the alias STTCLR, STTCLRL.

Integer

(FEAT_LSUI)



Encoding for the 32-bit no memory ordering variant

Applies when `(sz == 0 && A == 0 && R == 0)`

```
LDTCLR <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(sz == 0 && A == 1 && R == 0)`

```
LDTCLRA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(sz == 0 && A == 1 && R == 1)`

```
LDTCLRAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(sz == 0 && A == 0 && R == 1)`

```
LDTCLRL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(sz == 1 && A == 0 && R == 0)`

```
LDTCLR <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(sz == 1 && A == 1 && R == 0)`

```
LDTCLRA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(sz == 1 && A == 1 && R == 1)`

```
LDTCLRAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(sz == 1 && A == 0 && R == 1)`

```
LDTCLRL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 32 << UInt(sz);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STTCLR, STTCLRL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_BIC, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTNP

Load unprivileged pair of registers, with non-temporal hint

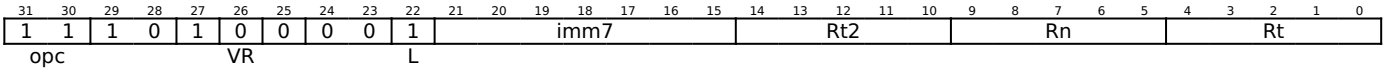
This instruction calculates an address from a base register value and an immediate offset, loads two 64-bit doublewords from memory, and writes them to two registers. For information about addressing modes, see *Load/Store addressing modes*. For information about non-temporal pair instructions, see *Load/Store non-temporal pair*.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Signed offset

(FEAT_LSUI)



Encoding

```
LDTNP <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = TRUE;
let datasize : integer{} = 64;
let offset : bits(64) = LSL(SignExtend{64}(imm7), 3);
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // Result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

LDTNP has the same CONSTRAINED UNPREDICTABLE behavior as LDNP. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDNP*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8.

Operation

```
var address : bits(64);
let privileged : boolean = AArch64_IsUnprivAccessPriv();
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, ispair, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);
let data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if rt_unknown then
    X{datasize}(t) = ARBITRARY : bits(datasize);
elsif BigEndian(accdesc.acctype) then
    X{datasize}(t2) = data[(datasize-1):0];
    X{datasize}(t)  = data[(2*datasize-1):datasize];
else
    X{datasize}(t)  = data[(datasize-1):0];
    X{datasize}(t2) = data[(2*datasize-1):datasize];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTP

Load unprivileged pair of registers

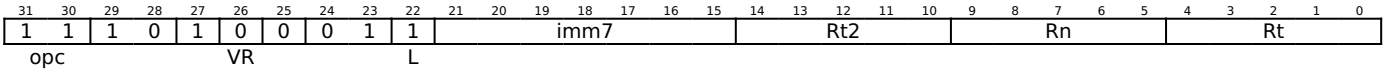
This instruction calculates an address from a base register value and an immediate offset, loads two 64-bit doublewords from memory, and writes them to two registers.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.
For information about addressing modes, see *Load/Store addressing modes*.
It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_LSUI)



Encoding

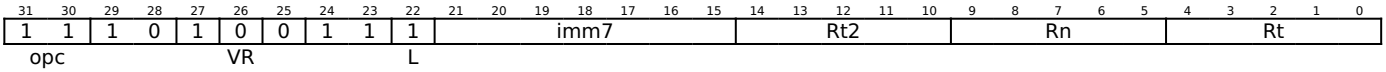
```
LDTP <Xt1>, <Xt2>, [<Xn|SP>], #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;

var wback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_LSUI)



Encoding

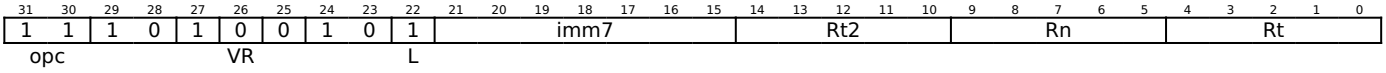
```
LDTP <Xt1>, <Xt2>, [<Xn|SP>, #<imm>]!
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;

var wback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_LSUI)



Encoding

```
LDTP <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;

var wback : boolean = FALSE;
let postindex : boolean = FALSE;
```

LDTP has the same CONSTRAINED UNPREDICTABLE behavior as LDP. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDP*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xt2> Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> For the "Post-index" and "Pre-index" variants: is the signed immediate byte offset, a multiple of 8 in the range -512 to 504, encoded in the "imm7" field as <imm>/8.

For the "Signed offset" variant: is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let scale : integer{} = 2 + UInt(opc[1]);
let datasize : integer{} = 64;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = wback || n != 31;

var rt_unknown : boolean = FALSE;
var wb_unknown : boolean = FALSE;

if wback && (t == n || t2 == n) && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPLD);
    assert c IN {Constraint_WBSUPPRESS, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_WBSUPPRESS => wback = FALSE;        // Writeback is suppressed
        when Constraint_UNKNOWN =>    wb_unknown = TRUE;    // Writeback is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // Result is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);
var data1 : bits(datasize);
var data2 : bits(datasize);
let privileged : boolean = AArch64.IsUnprivAccessPriv();
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp.LOAD, nontemporal, privileged,
                                                  tagchecked, ispair, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if rt_unknown then
    data1 = ARBITRARY : bits(datasize);
    data2 = ARBITRARY : bits(datasize);
elsif BigEndian(accdesc.acctype) then
    data2 = data[(datasize-1):0];
    data1 = data[(2*datasize-1):datasize];
else
    data1 = data[(datasize-1):0];
    data2 = data[(2*datasize-1):datasize];
end;

X{datasize}(t) = data1;
X{datasize}(t2) = data2;

if wback then
    if wb_unknown then
        address = ARBITRARY : bits(64);
    elsif postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTR

Load register (unprivileged)

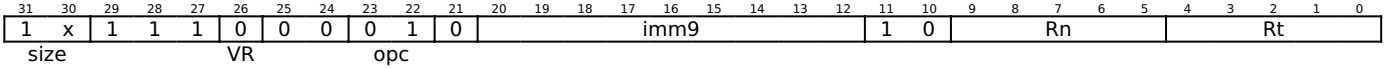
This instruction loads a word or doubleword from memory, and writes it to a register. The address that is used for the load is calculated from a base register and an immediate offset.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDTR <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
LDTR <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
let scale : integer{} = UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let datasize : integer{} = 8 << scale;
let regsize : integer{} = if datasize == 64 then 64 else 32;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTRB

Load register byte (unprivileged)

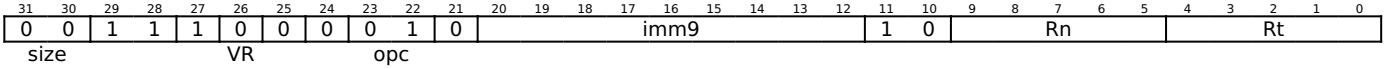
This instruction loads a byte from memory, zero-extends it, and writes the result to a register. The address that is used for the load is calculated from a base register and an immediate offset.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDTRB <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let datasize : integer{} = 8;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTRH

Load register halfword (unprivileged)

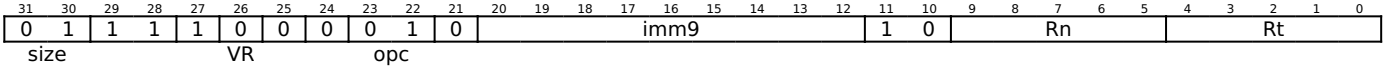
This instruction loads a halfword from memory, zero-extends it, and writes the result to a register. The address that is used for the load is calculated from a base register and an immediate offset.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDTRH <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let datasize : integer{} = 16;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTRSB

Load register signed byte (unprivileged)

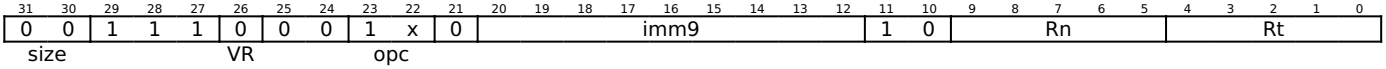
This instruction loads a byte from memory, sign-extends it to 32 bits or 64 bits, and writes the result to a register. The address that is used for the load is calculated from a base register and an immediate offset.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDTRSB <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
LDTRSB <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let datasize : integer{} = 8;
let regsize : integer{} = 64 >> UInt(opc[0]);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = SignExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTRSH

Load register signed halfword (unprivileged)

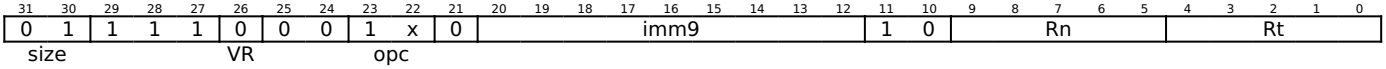
This instruction loads a halfword from memory, sign-extends it to 32 bits or 64 bits, and writes the result to a register. The address that is used for the load is calculated from a base register and an immediate offset.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDTRSH <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
LDTRSH <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let datasize : integer{} = 16;
let regsize : integer{} = 64 >> UInt(opc[0]);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = SignExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTRSW

Load register signed word (unprivileged)

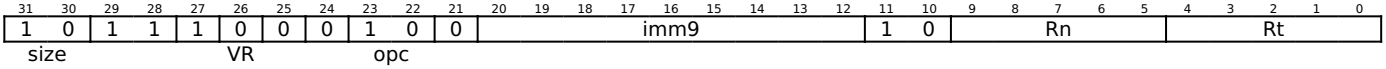
This instruction loads a word from memory, sign-extends it to 64 bits, and writes the result to a register. The address that is used for the load is calculated from a base register and an immediate offset.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDTRSW <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let datasize : integer{} = 32;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{64}(t) = SignExtend{64}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTSET, LDTSETA, LDTSETAL, LDTSETL

Atomic bit set unprivileged

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, performs a bitwise OR with the value held in a register on it, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR, or XZR, `LDTSETA` and `LDTSETAL` load from memory with acquire semantics.
- `LDTSETL` and `LDTSETAL` store to memory with release semantics.
- `LDTSET` has neither acquire nor release semantics.

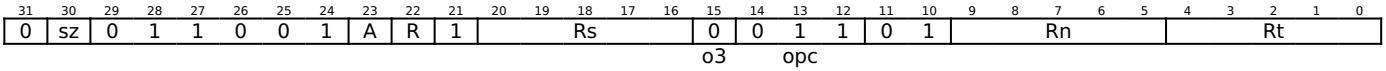
For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.
This instruction is used by the alias STTSET, STTSETL.

Integer

(FEAT_LSUI)



Encoding for the 32-bit no memory ordering variant

Applies when `(sz == 0 && A == 0 && R == 0)`

```
LDTSET <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(sz == 0 && A == 1 && R == 0)`

```
LDTSETA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(sz == 0 && A == 1 && R == 1)`

```
LDTSETAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(sz == 0 && A == 0 && R == 1)`

```
LDTSETL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(sz == 1 && A == 0 && R == 0)`

```
LDTSET <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(sz == 1 && A == 1 && R == 0)`

```
LDTSETA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(sz == 1 && A == 1 && R == 1)`

```
LDTSETAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(sz == 1 && A == 0 && R == 1)`

```
LDTSETL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 32 << UInt(sz);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STTSET, STTSETL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_ORR, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTXR

Load unprivileged exclusive register

This instruction derives an address from a base register value, loads a 32-bit word or a 64-bit doubleword from memory, and writes it to a register. The memory access is atomic. The PE marks the physical address being accessed as an exclusive access. This exclusive access mark is checked by Store Exclusive instructions. See *Synchronization and semaphores*.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

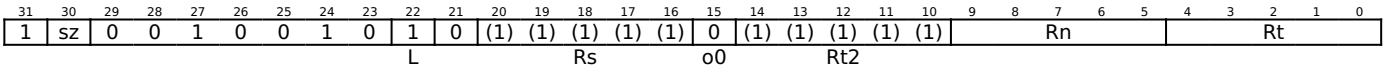
Note

For the purposes of the Exclusives monitors, and the forward progress guarantees for Load-Exclusive and Store-Exclusive loops, `LDTXR` is equivalent to `LDXR`.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
LDTXR <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
LDTXR <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let regsize : integer{} = if elsize == 64 then 64 else 32;
let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

AArch64_SetExclusiveMonitors(address, dbytes, accdesc);

data = Mem{elsize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL

Atomic unsigned maximum on word or doubleword

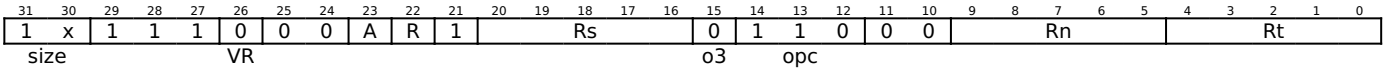
This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as unsigned numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR or XZR, `LDUMAXA` and `LDUMAXAL` load from memory with acquire semantics.
- `LDUMAXL` and `LDUMAXAL` store to memory with release semantics.
- `LDUMAX` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STUMAX, STUMAXL.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDUMAX <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDUMAXA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDUMAXAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDUMAXL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDUMAX <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDUMAXA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDUMAXAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDUMAXL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STUMAX, STUMAXL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_UMAX, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB

Atomic unsigned maximum on byte

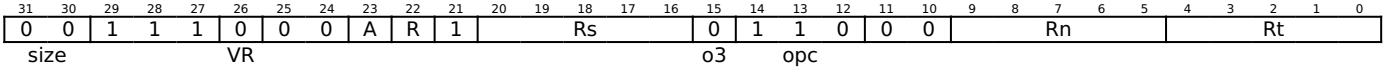
This instruction atomically loads an 8-bit byte from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as unsigned numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDUMAXAB` and `LDUMAXALB` load from memory with acquire semantics.
- `LDUMAXLB` and `LDUMAXALB` store to memory with release semantics.
- `LDUMAXB` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STUMAXB, STUMAXLB.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDUMAXB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDUMAXAB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDUMAXALB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDUMAXLB <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STUMAXB, STUMAXLB	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_UMAX, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(8) = ARBITRARY : bits(8); // Irrelevant when not executing CAS
let value : bits(8) = X{8}(s);
let data : bits(8) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH

Atomic unsigned maximum on halfword

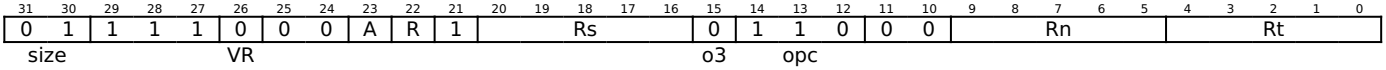
This instruction atomically loads a 16-bit halfword from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as unsigned numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDUMAXAH` and `LDUMAXALH` load from memory with acquire semantics.
- `LDUMAXLH` and `LDUMAXALH` store to memory with release semantics.
- `LDUMAXH` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STUMAXH, STUMAXLH.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDUMAXH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDUMAXAH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDUMAXALH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDUMAXLH <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STUMAXH, STUMAXLH	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_UMAX, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(16) = ARBITRARY : bits(16); // Irrelevant when not executing CAS
let value : bits(16) = X{16}(s);
let data : bits(16) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDUMIN, LDUMINA, LDUMINAL, LDUMINL

Atomic unsigned minimum on word or doubleword

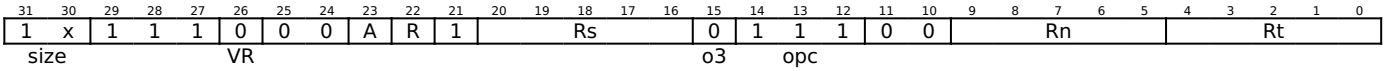
This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as unsigned numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR or XZR, `LDUMINA` and `LDUMINAL` load from memory with acquire semantics.
- `LDUMINL` and `LDUMINAL` store to memory with release semantics.
- `LDUMIN` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STUMIN, STUMINL.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDUMIN <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDUMINA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDUMINAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDUMINL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDUMIN <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDUMINA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDUMINAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDUMINL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
STUMIN, STUMINL	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_UMIN, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(datasize) = ARBITRARY : bits(datasize); // Irrelevant when not executing CAS
let value : bits(datasize) = X{datasize}(s);
let data : bits(datasize) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{regsize}(t) = ZeroExtend{regsize}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB

Atomic unsigned minimum on byte

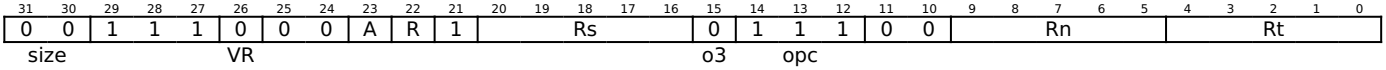
This instruction atomically loads an 8-bit byte from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as unsigned numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDUMINAB` and `LDUMINALB` load from memory with acquire semantics.
- `LDUMINLB` and `LDUMINALB` store to memory with release semantics.
- `LDUMINB` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STUMINB, STUMINLB.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDUMINB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDUMINAB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDUMINALB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDUMINLB <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STUMINB, STUMINLB	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_UMIN, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(8) = ARBITRARY : bits(8); // Irrelevant when not executing CAS
let value : bits(8) = X{8}(s);
let data : bits(8) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH

Atomic unsigned minimum on halfword

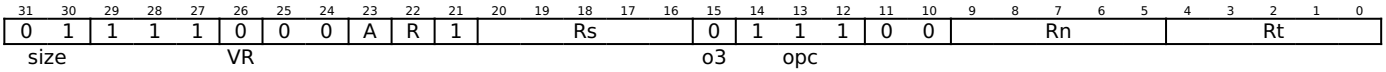
This instruction atomically loads a 16-bit halfword from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as unsigned numbers. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `LDUMINAH` and `LDUMINALH` load from memory with acquire semantics.
- `LDUMINLH` and `LDUMINALH` store to memory with release semantics.
- `LDUMINH` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.
This instruction is used by the alias STUMINH, STUMINLH.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDUMINH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDUMINAH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDUMINALH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDUMINLH <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
STUMINH, STUMINLH	`A == '0' && Rt == '11111'`

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_UMIN, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let comparevalue : bits(16) = ARBITRARY : bits(16); // Irrelevant when not executing CAS
let value : bits(16) = X{16}(s);
let data : bits(16) = MemAtomic{}(address, comparevalue, value, accdesc);

if t != 31 then
    X{32}(t) = ZeroExtend{32}(data);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

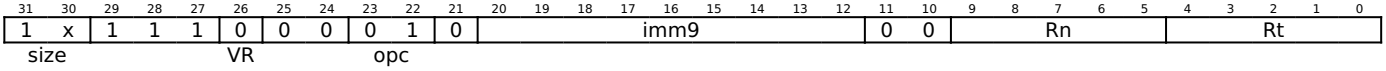
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDUR

Load register (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a 32-bit word or 64-bit doubleword from memory, zero-extends it, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDUR <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
LDUR <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
let scale : integer{} = UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let datasize : integer{} = 8 << scale;
let regsize : integer{} = if datasize == 64 then 64 else 32;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

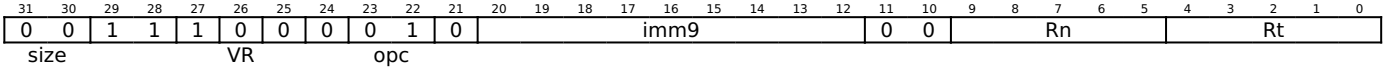
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDURB

Load register byte (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a byte from memory, zero-extends it, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDURB <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let datasize : integer{} = 8;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

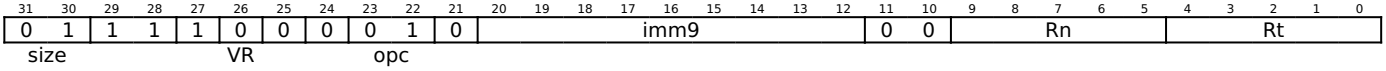
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDURH

Load register halfword (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a halfword from memory, zero-extends it, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDURH <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let datasize : integer{} = 16;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

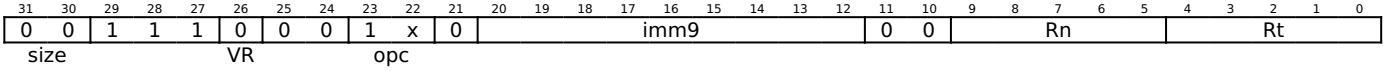
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDURSB

Load register signed byte (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a signed byte from memory, sign-extends it, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDURSB <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
LDURSB <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let datasize : integer{} = 8;
let regsize : integer{} = 64 >> UInt(opc[0]);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = SignExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

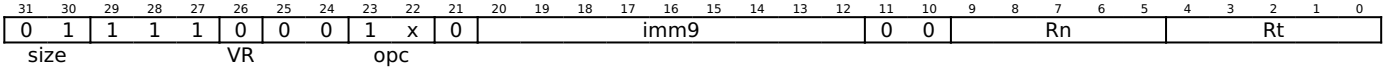
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDURSH

Load register signed halfword (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a signed halfword from memory, sign-extends it, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(opc == 11)`

```
LDURSH <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
LDURSH <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let datasize : integer{} = 16;
let regsize : integer{} = 64 >> UInt(opc[0]);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{regsize}(t) = SignExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

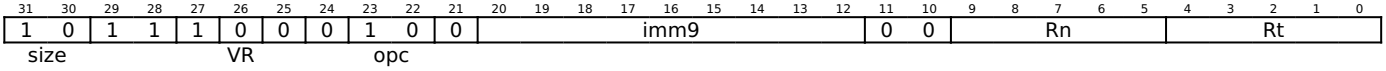
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDURSW

Load register signed word (unscaled)

This instruction calculates an address from a base register and an immediate offset, loads a signed word from memory, sign-extends it, and writes it to a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDURSW <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let datasize : integer{} = 32;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(datasize) = Mem{datasize}(address, accdesc);
X{64}(t) = SignExtend{64}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

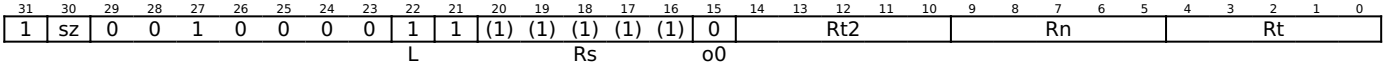
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDXP

Load exclusive pair of registers

This instruction derives an address from a base register value, loads two 32-bit words or two 64-bit doublewords from memory, and writes them to two registers. For information on single-copy atomicity and alignment requirements, see *Requirements for single-copy atomicity* and *Alignment of data accesses*. The PE marks the physical address being accessed as an exclusive access. This exclusive access mark is checked by Store Exclusive instructions. See *Synchronization and semaphores*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
LDXP <Wt1>, <Wt2>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
LDXP <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let datasize : integer{} = elsize * 2;
let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // result is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *CONSTRAINED UNPREDICTABLE behaviors with Load-Exclusive/Store-Exclusive pairs* and *LDXP*.

Assembler Symbols

<Wt1>	Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Wt2>	Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Operation

```
var address : bits(64);
var data : bits(datasize);

let dbytes : integer{} = datasize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
// When elsize is 32, the access is single-copy atomic
let ispair : boolean = elsize == 64;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                     privileged, ispair, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

AArch64_SetExclusiveMonitors(address, dbytes, accdesc);

data = Mem{datasize}(address, accdesc);
if rt_unknown then
    // ConstrainedUNPREDICTABLE case
    X{datasize}(t)  = ARBITRARY : bits(datasize);        // In this case t = t2
elsif BigEndian(accdesc.acctype) then
    X{elsize}(t)  = data[elsize +: elsize];
    X{elsize}(t2) = data[0 +: elsize];
else
    X{elsize}(t)  = data[0 +: elsize];
    X{elsize}(t2) = data[elsize +: elsize];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

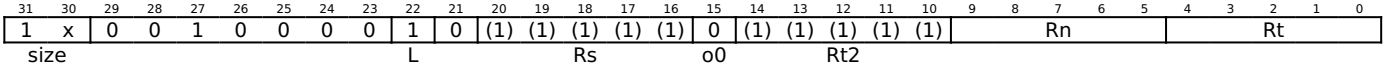
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDXR

Load exclusive register

This instruction derives an address from a base register value, loads a 32-bit word or a 64-bit doubleword from memory, and writes it to a register. The memory access is atomic. The PE marks the physical address being accessed as an exclusive access. This exclusive access mark is checked by Store Exclusive instructions. See *Synchronization and semaphores*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
LDXR <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
LDXR <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 8 << UInt(size);
let regsize : integer{} = if elsize == 64 then 64 else 32;
let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

AArch64_SetExclusiveMonitors(address, dbytes, accdesc);

data = Mem{elsize}(address, accdesc);
X{regsize}(t) = ZeroExtend{regsize}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

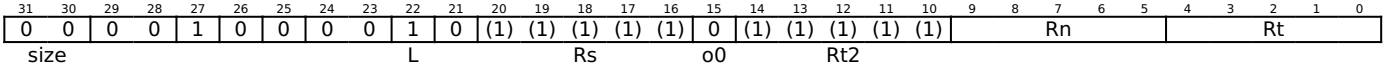
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDXRB

Load exclusive register byte

This instruction derives an address from a base register value, loads a byte from memory, zero-extends it and writes it to a register. The memory access is atomic. The PE marks the physical address being accessed as an exclusive access. This exclusive access mark is checked by Store Exclusive instructions. See *Synchronization and semaphores*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDXRB <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data : bits(8);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

AArch64_SetExclusiveMonitors(address, 1, accdesc);

data = Mem{8}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

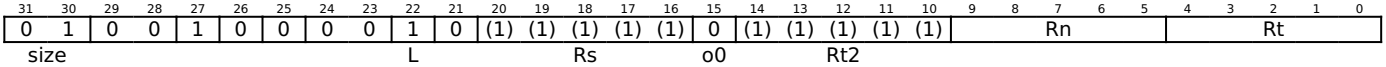
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDXRH

Load exclusive register halfword

This instruction derives an address from a base register value, loads a halfword from memory, zero-extends it and writes it to a register. The memory access is atomic. The PE marks the physical address being accessed as an exclusive access. This exclusive access mark is checked by Store Exclusive instructions. See *Synchronization and semaphores*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
LDXRH <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data : bits(16);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

AArch64_SetExclusiveMonitors(address, 2, accdesc);

data = Mem{16}(address, accdesc);
X{32}(t) = ZeroExtend{32}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

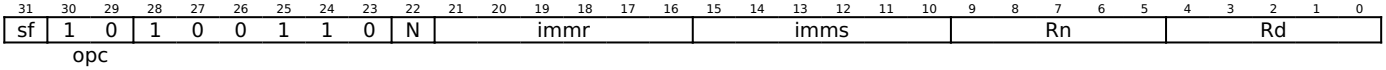
LSL (immediate)

Logical shift left (immediate)

This instruction shifts a register value left by an immediate number of bits, shifting in zeros, and writes the result to the destination register.

This is an alias of UBFM. This means:

- The encodings in this description are named to match the encodings of UBFM.
- The description of UBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0 && imms != 011111)`

```
LSL <Wd>, <Wn>, #<shift>
```

is equivalent to

```
UBFM <Wd>, <Wn>, #(-<shift> MOD 32), #(31-<shift>)
```

and is the preferred disassembly when `UInt(imms) + 1 == UInt(immr)`.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1 && imms != 111111)`

```
LSL <Xd>, <Xn>, #<shift>
```

is equivalent to

```
UBFM <Xd>, <Xn>, #(-<shift> MOD 64), #(63-<shift>)
```

and is the preferred disassembly when `UInt(imms) + 1 == UInt(immr)`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<shift>	For the "32-bit" variant: is the shift amount, in the range 0 to 31. For the "64-bit" variant: is the shift amount, in the range 0 to 63.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of UBFM gives the operational pseudocode for this instruction.

Operational Information

The description of UBFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

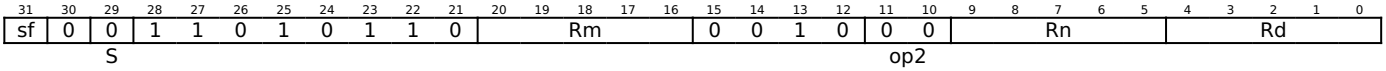
LSL (register)

Logical shift left (register)

This instruction shifts a register value left by a variable number of bits, shifting in zeros, and writes the result to the destination register. The value of the second source register modulo the register size in bits gives the number of bits by which the first source register is left-shifted.

This is an alias of LSLV. This means:

- The encodings in this description are named to match the encodings of LSLV.
- The description of LSLV gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
LSL <Wd>, <Wn>, <Wm>
```

is equivalent to

```
LSLV <Wd>, <Wn>, <Wm>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
LSL <Xd>, <Xn>, <Xm>
```

is equivalent to

```
LSLV <Xd>, <Xn>, <Xm>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding a shift amount from 0 to 31 in its bottom 5 bits, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding a shift amount from 0 to 63 in its bottom 6 bits, encoded in the "Rm" field.

Operation

The description of LSLV gives the operational pseudocode for this instruction.

Operational Information

The description of LSLV gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

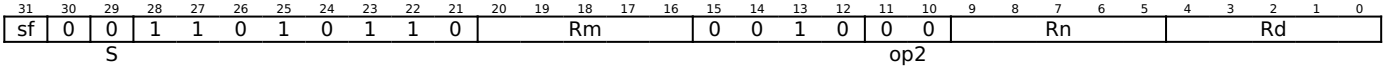
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LSLV

Logical shift left variable

This instruction shifts a register value left by a variable number of bits, shifting in zeros, and writes the result to the destination register. The value of the second source register modulo the register size in bits gives the number of bits by which the first source register is left-shifted.

This instruction is used by the alias LSL (register).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
LSLV <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
LSLV <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(op2);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding a shift amount from 0 to 31 in its bottom 5 bits, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding a shift amount from 0 to 63 in its bottom 6 bits, encoded in the "Rm" field.

Operation

```
let operand2 : bits(datasize) = X{datasize}(m);

X{datasize}(d) = ShiftReg{datasize}(n, shift_type, UInt(operand2) MOD datasize);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

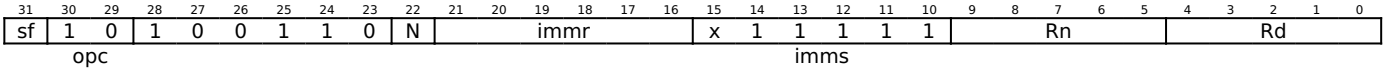
LSR (immediate)

Logical shift right (immediate)

This instruction shifts a register value right by an immediate number of bits, shifting in zeros, and writes the result to the destination register.

This is an alias of UBFM. This means:

- The encodings in this description are named to match the encodings of UBFM.
- The description of UBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0 && imms == 011111)`

```
LSR <Wd>, <Wn>, #<shift>
```

is equivalent to

```
UBFM <Wd>, <Wn>, #<shift>, #31
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1 && imms == 111111)`

```
LSR <Xd>, <Xn>, #<shift>
```

is equivalent to

```
UBFM <Xd>, <Xn>, #<shift>, #63
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<shift>	For the "32-bit" variant: is the shift amount, in the range 0 to 31, encoded in the "immr" field. For the "64-bit" variant: is the shift amount, in the range 0 to 63, encoded in the "immr" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of UBFM gives the operational pseudocode for this instruction.

Operational Information

The description of UBFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

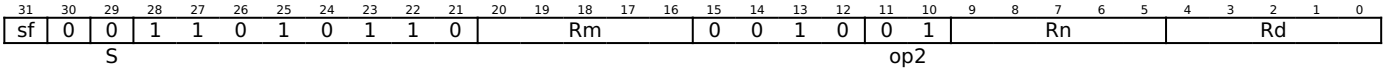
LSR (register)

Logical shift right (register)

This instruction shifts a register value right by a variable number of bits, shifting in zeros, and writes the result to the destination register. The value of the second source register modulo the register size in bits gives the number of bits by which the first source register is right-shifted.

This is an alias of LSRV. This means:

- The encodings in this description are named to match the encodings of LSRV.
- The description of LSRV gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
LSR <Wd>, <Wn>, <Wm>
```

is equivalent to

```
LSRV <Wd>, <Wn>, <Wm>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
LSR <Xd>, <Xn>, <Xm>
```

is equivalent to

```
LSRV <Xd>, <Xn>, <Xm>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding a shift amount from 0 to 31 in its bottom 5 bits, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding a shift amount from 0 to 63 in its bottom 6 bits, encoded in the "Rm" field.

Operation

The description of LSRV gives the operational pseudocode for this instruction.

Operational Information

The description of LSRV gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

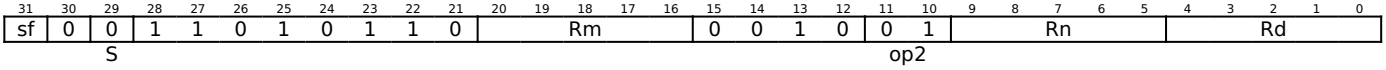
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LSRV

Logical shift right variable

This instruction shifts a register value right by a variable number of bits, shifting in zeros, and writes the result to the destination register. The value of the second source register modulo the register size in bits gives the number of bits by which the first source register is right-shifted.

This instruction is used by the alias LSR (register).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
LSRV <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
LSRV <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(op2);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding a shift amount from 0 to 31 in its bottom 5 bits, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding a shift amount from 0 to 63 in its bottom 6 bits, encoded in the "Rm" field.

Operation

```
let operand2 : bits(datasize) = X{datasize}(m);

X{datasize}(d) = ShiftReg{datasize}(n, shift_type, UInt(operand2) MOD datasize);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

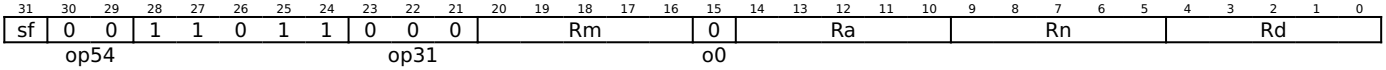
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MADD

Multiply-add

This instruction multiplies two register values, adds a third register value, and writes the result to the destination register.
This instruction is used by the alias MUL.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
MADD <Wd>, <Wn>, <Wm>, <Wa>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MADD <Xd>, <Xn>, <Xm>, <Xa>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Wa>	Is the 32-bit name of the third general-purpose source register holding the addend, encoded in the "Ra" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Xa>	Is the 64-bit name of the third general-purpose source register holding the addend, encoded in the "Ra" field.

Alias Conditions

Alias	Is preferred when
MUL	`Ra == '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);
let operand3 : bits(datasize) = X{datasize}(a);

let result : integer = UInt(operand3) + UInt(operand1) * UInt(operand2);

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MADDPT

Multiply-add checked pointer

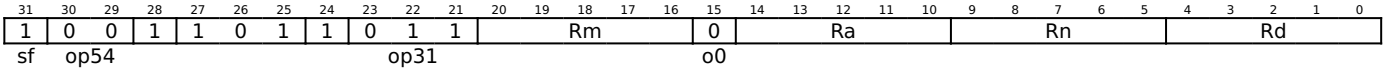
This instruction multiplies two register values, adds a third register value, and writes the result to the destination register. The intermediate product is treated as the offset.

If the operation would have generated a result where the most significant 8 bits of the result register differ from the most significant 8 bits of the base register, then the result is modified such that it is likely to be non-canonical when used as an address.

If the intermediate product cannot be correctly represented as a 64-bit two's complement value, then the result is modified such that it is likely to be non-canonical when used as an address.

Integer

(FEAT_CPA)



Encoding

```
MADDPT <Xd>, <Xn>, <Xm>, <Xa>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CPA) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Xa>	Is the 64-bit name of the third general-purpose source register holding the addend, encoded in the "Ra" field.

Operation

```
let operand1 : bits(64) = X{64}(n);
let operand2 : bits(64) = X{64}(m);
let operand3 : bits(64) = X{64}(a);

var result : bits(64);

let product : integer = SInt(operand1) * SInt(operand2);

// Signed and unsigned twos complement arithmetic are equivalent if only a
// fixed number of bits are considered.
result = operand3 + product[63:0];

let overflow : boolean = (product != SInt(product[63:0]));
result = PointerMultiplyAddCheck(result, operand3, overflow);

X{64}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

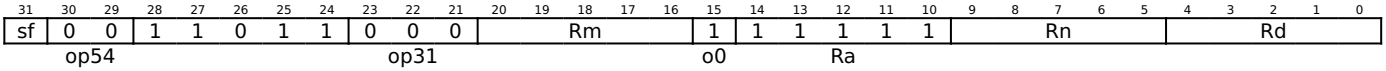
MNEG

Multiply-negate

This instruction multiplies two register values, negates the product, and writes the result to the destination register.

This is an alias of MSUB. This means:

- The encodings in this description are named to match the encodings of MSUB.
- The description of MSUB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
MNEG <Wd>, <Wn>, <Wm>
```

is equivalent to

```
MSUB <Wd>, <Wn>, <Wm>, WZR
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MNEG <Xd>, <Xn>, <Xm>
```

is equivalent to

```
MSUB <Xd>, <Xn>, <Xm>, XZR
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation

The description of MSUB gives the operational pseudocode for this instruction.

Operational Information

The description of MSUB gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

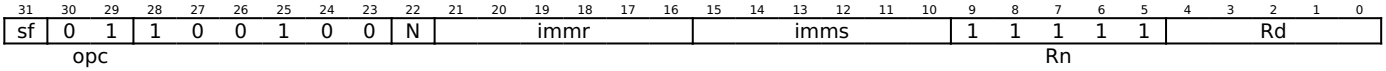
MOV (bitmask immediate)

Move bitmask immediate value

This instruction writes a bitmask immediate value to a register.

This is an alias of ORR (immediate). This means:

- The encodings in this description are named to match the encodings of ORR (immediate).
- The description of ORR (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
MOV <Wd|WSP>, #<imm>
```

is equivalent to

```
ORR <Wd|WSP>, WZR, #<imm>
```

and is the preferred disassembly when `!MoveWidePreferred(sf, N, imms, immr)`.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MOV <Xd|SP>, #<imm>
```

is equivalent to

```
ORR <Xd|SP>, XZR, #<imm>
```

and is the preferred disassembly when `!MoveWidePreferred(sf, N, imms, immr)`.

Assembler Symbols

<Wd|WSP>	Is the 32-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<imm>	For the "32-bit" variant: is the bitmask immediate, encoded in "imms:immr", but excluding values which could be encoded by MOVZ or MOVN. For the "64-bit" variant: is the bitmask immediate, encoded in "N:imms:immr", but excluding values which could be encoded by MOVZ or MOVN.
<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.

Operation

The description of ORR (immediate) gives the operational pseudocode for this instruction.

Operational Information

The description of ORR (immediate) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

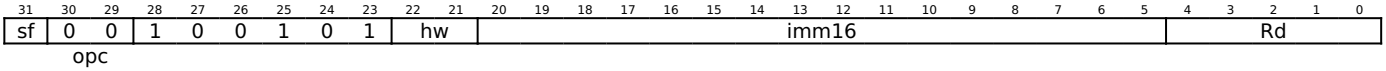
MOV (inverted wide immediate)

Move inverted wide immediate value

This instruction moves an inverted 16-bit immediate value to a register.

This is an alias of MOVN. This means:

- The encodings in this description are named to match the encodings of MOVN.
- The description of MOVN gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && hw == 0x)`

```
MOV <Wd>, #<imm>
```

is equivalent to

```
MOVN <Wd>, #<imm16>, LSL #<shift>
```

and is the preferred disassembly when `!(IsZero(imm16) && hw != '00') && !IsOnes(imm16)`.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MOV <Xd>, #<imm>
```

is equivalent to

```
MOVN <Xd>, #<imm16>, LSL #<shift>
```

and is the preferred disassembly when `!(IsZero(imm16) && hw != '00')`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<imm>	For the "32-bit" variant: is a 32-bit immediate, the bitwise inverse of which can be encoded in "imm16:hw", but excluding 0xFFFF0000 and 0x0000FFFF. For the "64-bit" variant: is a 64-bit immediate, the bitwise inverse of which can be encoded in "imm16:hw".
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

The description of MOVN gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

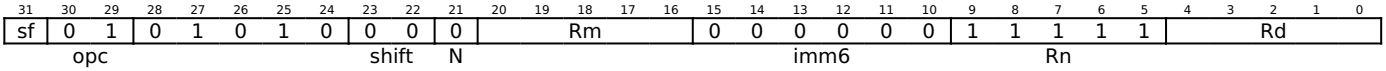
MOV (register)

Move register value

This instruction copies the value in a source register to the destination register.

This is an alias of ORR (shifted register). This means:

- The encodings in this description are named to match the encodings of ORR (shifted register).
- The description of ORR (shifted register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
MOV <Wd>, <Wm>
```

is equivalent to

```
ORR <Wd>, WZR, <Wm>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MOV <Xd>, <Xm>
```

is equivalent to

```
ORR <Xd>, XZR, <Xm>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wm>	Is the 32-bit name of the general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xm>	Is the 64-bit name of the general-purpose source register, encoded in the "Rm" field.

Operation

The description of ORR (shifted register) gives the operational pseudocode for this instruction.

Operational Information

The description of ORR (shifted register) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

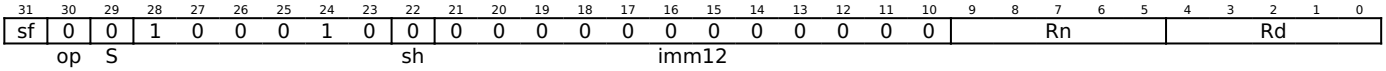
MOV (to/from SP)

Move register value to or from SP

This instruction copies the value of a register to or from the stack pointer.

This is an alias of ADD (immediate). This means:

- The encodings in this description are named to match the encodings of ADD (immediate).
- The description of ADD (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
MOV <Wd|WSP>, <Wn|WSP>
```

is equivalent to

```
ADD <Wd|WSP>, <Wn|WSP>, #0
```

and is the preferred disassembly when `Rd == '11111' || Rn == '11111'`.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MOV <Xd|SP>, <Xn|SP>
```

is equivalent to

```
ADD <Xd|SP>, <Xn|SP>, #0
```

and is the preferred disassembly when `Rd == '11111' || Rn == '11111'`.

Assembler Symbols

<Wd|WSP>	Is the 32-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Wn|WSP>	Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.
<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Operation

The description of ADD (immediate) gives the operational pseudocode for this instruction.

Operational Information

The description of ADD (immediate) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

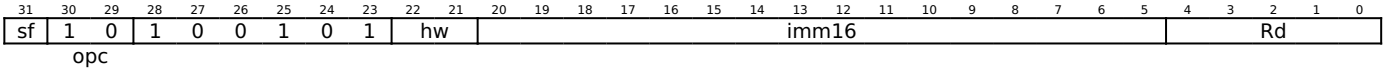
MOV (wide immediate)

Move wide immediate value

This instruction moves a 16-bit immediate value to a register.

This is an alias of MOVZ. This means:

- The encodings in this description are named to match the encodings of MOVZ.
- The description of MOVZ gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && hw == 0x)`

```
MOV <Wd>, #<imm>
```

is equivalent to

```
MOVZ <Wd>, #<imm16>, LSL #<shift>
```

and is the preferred disassembly when `!(IsZero(imm16) && hw != '00')`.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MOV <Xd>, #<imm>
```

is equivalent to

```
MOVZ <Xd>, #<imm16>, LSL #<shift>
```

and is the preferred disassembly when `!(IsZero(imm16) && hw != '00')`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<imm>	For the "32-bit" variant: is a 32-bit immediate which can be encoded in "imm16:hw". For the "64-bit" variant: is a 64-bit immediate which can be encoded in "imm16:hw".
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

The description of MOVZ gives the operational pseudocode for this instruction.

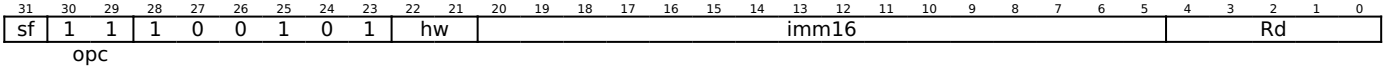
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVK

Move wide with keep

This instruction moves an optionally-shifted 16-bit immediate value into a register, keeping other bits unchanged.



Encoding for the 32-bit variant

Applies when `(sf == 0 && hw == 0x)`

```
MOVK <Wd>, #<imm>{, LSL #<shift>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MOVK <Xd>, #<imm>{, LSL #<shift>}
```

Decode for all variants of this encoding

```
if sf == '0' && hw[1] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let datasize : integer{} = 32 << UInt(sf);
let imm : bits(16) = imm16;
let pos : integer{} = UInt(hw) << 4;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<imm>	Is the 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field.
<shift>	For the "32-bit" variant: is the amount by which to shift the immediate left, either 0 (the default) or 16, encoded in the "hw" field as <shift>/16. For the "64-bit" variant: is the amount by which to shift the immediate left, either 0 (the default), 16, 32, or 48, encoded in the "hw" field as <shift>/16.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

```
var result : bits(datasize) = X{datasize}(d);

result[pos+15:pos] = imm;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

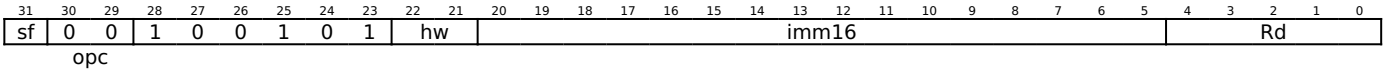
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVN

Move wide with NOT

This instruction moves the inverse of an optionally-shifted 16-bit immediate value to a register.
This instruction is used by the alias MOV (inverted wide immediate).



Encoding for the 32-bit variant

Applies when `(sf == 0 && hw == 0x)`

```
MOVN <Wd>, #<imm>{, LSL #<shift>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MOVN <Xd>, #<imm>{, LSL #<shift>}
```

Decode for all variants of this encoding

```
if sf == '0' && hw[1] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let datasize : integer{} = 32 << UInt(sf);
let imm : bits(16) = imm16;
let pos : integer{} = UInt(hw) << 4;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<imm>	Is the 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field.
<shift>	For the "32-bit" variant: is the amount by which to shift the immediate left, either 0 (the default) or 16, encoded in the "hw" field as <shift>/16. For the "64-bit" variant: is the amount by which to shift the immediate left, either 0 (the default), 16, 32, or 48, encoded in the "hw" field as <shift>/16.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Alias Conditions

Alias	Of variant	Is preferred when
MOV (inverted wide immediate)	32-bit	`!(IsZero(imm16) && hw != '00') && !IsOnes(imm16)`
MOV (inverted wide immediate)	64-bit	`!(IsZero(imm16) && hw != '00')`

Operation

```
var result : bits(datasize) = Zeros{};

result[pos+15:pos] = imm;

X{datasize}(d) = NOT result;
```

2026-09_rel 2026-09-21 14:22:38

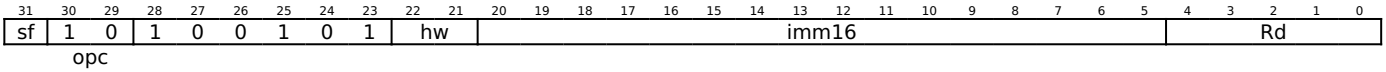
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVZ

Move wide with zero

This instruction moves an optionally-shifted 16-bit immediate value to a register.

This instruction is used by the alias MOV (wide immediate).



Encoding for the 32-bit variant

Applies when `(sf == 0 && hw == 0x)`

```
MOVZ <Wd>, #<imm>{, LSL #<shift>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MOVZ <Xd>, #<imm>{, LSL #<shift>}
```

Decode for all variants of this encoding

```
if sf == '0' && hw[1] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let datasize : integer{} = 32 << UInt(sf);
let imm : bits(16) = imm16;
let pos : integer{} = UInt(hw) << 4;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<imm>	Is the 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field.
<shift>	For the "32-bit" variant: is the amount by which to shift the immediate left, either 0 (the default) or 16, encoded in the "hw" field as <shift>/16. For the "64-bit" variant: is the amount by which to shift the immediate left, either 0 (the default), 16, 32, or 48, encoded in the "hw" field as <shift>/16.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Alias Conditions

Alias	Is preferred when
MOV (wide immediate)	`!(IsZero(imm16) && hw != '00')`

Operation

```
var result : bits(datasize) = Zeros{};

result[pos+15:pos] = imm;

X{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

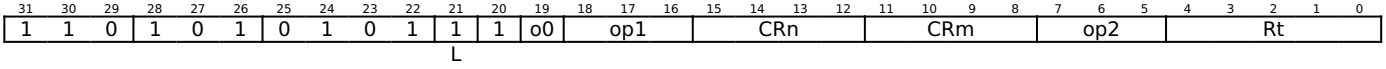
MRRS

Move System register to two adjacent general-purpose registers

This instruction allows the PE to read an AArch64 128-bit System register into two adjacent 64-bit general-purpose registers.

System

(FEAT_SYSREG128)



Encoding

```
MRRS <Xt>, <Xt+1>, (<systemreg>|S<op0>_<op1>_<Cn>_<Cm>_<op2>)
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SYSREG128) then EndOfDecode(Decode_UNDEF); end;
if Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;

let t : integer       = UInt(Rt);
let t2 : integer      = UInt(Rt+1);
let sys_op0 : bits(2) = '1' :: o0;
let sys_op1 : bits(3) = op1;
let sys_op2 : bits(3) = op2;
let sys_crn : bits(4) = CRn;
let sys_crm : bits(4) = CRm;
```

Assembler Symbols

<Xt>	Is the 64-bit name of the first general-purpose destination register, encoded in the "Rt" field.
<Xt+1>	Is the 64-bit name of the second general-purpose destination register, encoded as "Rt" +1.
<systemreg>	Is a System register name, encoded in "o0:op1:CRn:CRm:op2".
<op0>	Is an unsigned immediate, encoded in “o0”:

o0	<op0>
0	2
1	3

<op1>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op1" field.
<Cn>	Is a name 'Cn', with 'n' in the range 0 to 15, encoded in the "CRn" field.
<Cm>	Is a name 'Cm', with 'm' in the range 0 to 15, encoded in the "CRm" field.
<op2>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op2" field.

Operation

```
AArch64_SysRegRead128(sys_op0, sys_op1, sys_crn, sys_crm, sys_op2, t, t2);
```

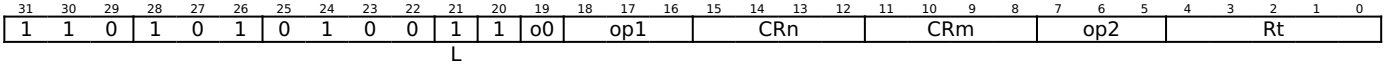
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MRS

Move System register to general-purpose register

This instruction allows the PE to read an AArch64 System register into a general-purpose register.



Encoding

```
MRS <Xt>, (<systemreg>|S<op0>_<op1>_<Cn>_<Cm>_<op2>)
```

Decode for this encoding

```
let t : integer       = UInt(Rt);
let sys_op0 : bits(2) = '1' :: o0;
let sys_op1 : bits(3) = op1;
let sys_op2 : bits(3) = op2;
let sys_crn : bits(4) = CRn;
let sys_crm : bits(4) = CRm;
```

Assembler Symbols

<Xt> Is the 64-bit name of the general-purpose destination register, encoded in the "Rt" field.

<systemreg> Is a System register name, encoded in "o0:op1:CRn:CRm:op2". The System register names are defined in *AArch64 System Register Descriptions*.

<op0> Is an unsigned immediate, encoded in “o0”:

o0	<op0>
0	2
1	3

<op1> Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op1" field.

<Cn> Is a name 'Cn', with 'n' in the range 0 to 15, encoded in the "CRn" field.

<Cm> Is a name 'Cm', with 'm' in the range 0 to 15, encoded in the "CRm" field.

<op2> Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op2" field.

Operation

```
AArch64_SysRegRead(sys_op0, sys_op1, sys_crn, sys_crm, sys_op2, t);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

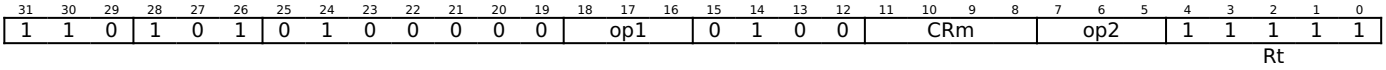
MSR (immediate)

Move immediate value to special register

This instruction moves an immediate value to selected bits of the PSTATE. For more information, see *Process state, PSTATE*.
The bits that can be written by this instruction are:

- PSTATE.D, PSTATE.A, PSTATE.I, PSTATE.F, and PSTATE.SP.
- If *FEAT_SSBS* is implemented, PSTATE.SSBS.
- If *FEAT_PAN* is implemented, PSTATE.PAN.
- If *FEAT_UAO* is implemented, PSTATE.UAO.
- If *FEAT_DIT* is implemented, PSTATE.DIT.
- If *FEAT_MTE* is implemented, PSTATE.TCO.
- If *FEAT_NMI* is implemented, PSTATE.ALLINT.
- If *FEAT_SME* is implemented, PSTATE.SM and PSTATE.ZA.
- If *FEAT_EBEP*, *FEAT_SPE_EXC*, or *FEAT_TRBE_EXC* is implemented, PSTATE.PM.

If *FEAT_MTE* is implemented and *FEAT_MTE2* is not implemented, it is IMPLEMENTATION DEFINED whether writes to PSTATE.TCO by this instruction are ignored.
This instruction is used by the aliases SMSTART, and SMSTOP.



Encoding

Applies when `(!(op1 == 000 && op2 IN {00x, 010}))`

```
MSR <pstatefield>, #<imm>
```

Decode for this encoding

```
if op1 == '000' && op2 == '000' then See("CFINV"); end;
if op1 == '000' && op2 == '001' then See("XAFLAG"); end;
if op1 == '000' && op2 == '010' then See("AXFLAG"); end;

var min_EL : bits(2);
var need_secure : boolean = FALSE;

case op1 of
    when '00x' =>
        min_EL = EL1;
    when '010' =>
        min_EL = EL1;
    when '011' =>
        min_EL = EL0;
    when '100' =>
        min_EL = EL2;
    when '101' =>
        if !IsFeatureImplemented(FEAT_VHE) then EndOfDecode(Decode_UNDEF); end;
        min_EL = EL2;
    when '110' =>
        min_EL = EL3;
    when '111' =>
        min_EL = EL1;
        need_secure = TRUE;
end;

let operand : bits(4) = CRm;
var field : PSTATEField;
case op1::op2 of
    when '000 011' =>
        if !IsFeatureImplemented(FEAT_UAO) then EndOfDecode(Decode_UNDEF); end;
        field = PSTATEField_UAO;
    when '000 100' =>
        if !IsFeatureImplemented(FEAT_PAN) then EndOfDecode(Decode_UNDEF); end;
        field = PSTATEField_PAN;
    when '000 101' => field = PSTATEField_SP;
    when '001 000' =>
        case CRm of
            when '000x' =>
                if !IsFeatureImplemented(FEAT_NMI) then EndOfDecode(Decode_UNDEF); end;
                field = PSTATEField_ALLINT;
            when '001x' =>
                if (!IsFeatureImplemented(FEAT_EBEP) &&
                      !IsFeatureImplemented(FEAT_SPE_EXC) &&
                      !IsFeatureImplemented(FEAT_TRBE_EXC)) then EndOfDecode(Decode_UNDEF); end;
                field = PSTATEField_PM;
            otherwise =>
                EndOfDecode(Decode_UNDEF);
        end;
    when '011 010' =>
        if !IsFeatureImplemented(FEAT_DIT) then EndOfDecode(Decode_UNDEF); end;
        field = PSTATEField_DIT;
    when '011 011' =>
        case CRm of
            when '001x' =>
                if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
                field = PSTATEField_SVCRSM;
            when '010x' =>
                if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
                field = PSTATEField_SVCRZA;
            when '011x' =>
                if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
                field = PSTATEField_SVCRSMZA;
            otherwise =>
                EndOfDecode(Decode_UNDEF);
        end;
    when '011 100' =>
        if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
        field = PSTATEField_TCO;
    when '011 110' => field = PSTATEField_DAIFSet;
    when '011 111' => field = PSTATEField_DAIFClr;
    when '011 001' =>
        if !IsFeatureImplemented(FEAT_SSBS) then EndOfDecode(Decode_UNDEF); end;
        field = PSTATEField_SSBS;
    otherwise =>      EndOfDecode(Decode_UNDEF);
end;
```

Assembler Symbols

<pstatefield> Is a PSTATE field name. For the MSR instruction, this is encoded in “(CRm :: op1 :: op2)”:

CRm	op1	op2	<pstatefield>	Architectural Feature
xxxx	000	011	UAO	FEAT_UAO
xxxx	000	100	PAN	FEAT_PAN
xxxx	000	101	SPSel	-
xxxx	000	11x	RESERVED	-
000x	001	000	ALLINT	FEAT_NMI
xxxx	001	001	RESERVED	-
xxxx	001	01x	RESERVED	-
xxxx	001	1xx	RESERVED	-
xxxx	010	xxx	RESERVED	-
xxxx	011	000	RESERVED	-
xxxx	011	001	SSBS	FEAT_SSBS
xxxx	011	010	DIT	FEAT_DIT
000x	011	011	RESERVED	-
xxxx	011	100	TCO	FEAT_MTE
xxxx	011	101	RESERVED	-
xxxx	011	110	DAIFSet	-
xxxx	011	111	DAIFClr	-
xxxx	1xx	xxx	RESERVED	-
001x	001	000	PM	FEAT_EBEP || FEAT_SPE_EXC || FEAT_TRBE_EXC
001x	011	011	SVCRSM	FEAT_SME
01xx	001	000	RESERVED	-
010x	011	011	SVCRZA	FEAT_SME
011x	011	011	SVCRSMZA	FEAT_SME
1xxx	001	000	RESERVED	-
1xxx	011	011	RESERVED	-

For the SMSTART and SMSTOP aliases, this is encoded in "CRm<2:1>", where 0b01 specifies SVCRSM, 0b10 specifies SVCRZA, and 0b11 specifies SVCRSMZA.

<imm> Is a 4-bit unsigned immediate, in the range 0 to 15, encoded in the "CRm" field. Restricted to the range 0 to 1, encoded in "CRm<0>", when <pstatefield> is ALLINT, PM, SVCRSM, SVCRSMZA, or SVCRZA.

Alias Conditions

Alias	Is preferred when
SMSTART	`op1 == '011' && CRm IN {'0xx1'} && op2 == '011'`
SMSTOP	`op1 == '011' && CRm IN {'0xx0'} && op2 == '011'`

Operation

```
if UInt(PSTATE.EL) < UInt(min_EL) then Undefined(); end;
if need_secure && CurrentSecurityState() != SS_Secure then Undefined(); end;

case field of
    when PSTATEField_SSBS =>
        PSTATE.SSBS = operand[0];
    when PSTATEField_SP =>
        PSTATE.SP = operand[0];
    when PSTATEField_DAIFSet =>
        AArch64_CheckDAIFAccess(PSTATEField_DAIFSet);
        PSTATE.D = PSTATE.D OR operand[3];
        PSTATE.A = PSTATE.A OR operand[2];
        PSTATE.I = PSTATE.I OR operand[1];
        PSTATE.F = PSTATE.F OR operand[0];
    when PSTATEField_DAIFClr =>
        AArch64_CheckDAIFAccess(PSTATEField_DAIFClr);
        PSTATE.D = PSTATE.D AND NOT(operand[3]);
        PSTATE.A = PSTATE.A AND NOT(operand[2]);
        PSTATE.I = PSTATE.I AND NOT(operand[1]);
        PSTATE.F = PSTATE.F AND NOT(operand[0]);
    when PSTATEField_PAN =>
        PSTATE.PAN = operand[0];
    when PSTATEField_UAO =>
        PSTATE.UAO = operand[0];
    when PSTATEField_DIT =>
        PSTATE.DIT = operand[0];
    when PSTATEField_TCO =>
        PSTATE.TCO = operand[0];
    when PSTATEField_ALLINT =>
        if (PSTATE.EL == EL1 && IsHCRXEL2Enabled() &&
              HCRX_EL2().TALLINT == '1' && operand[0] == '1') then
            AArch64_SystemAccessTrap(EL2, 0x18);
        end;
        PSTATE.ALLINT = operand[0];
    when PSTATEField_SVCRSM =>
        CheckSMEAccess();
        SetPSTATE_SM(operand[0]);
    when PSTATEField_SVCRZA =>
        CheckSMEAccess();
        SetPSTATE_ZA(operand[0]);
    when PSTATEField_SVCRSMZA =>
        CheckSMEAccess();
        SetPSTATE_SM(operand[0]);
        SetPSTATE_ZA(operand[0]);
    when PSTATEField_PM =>
        PSTATE.PM = operand[0];
end;
```

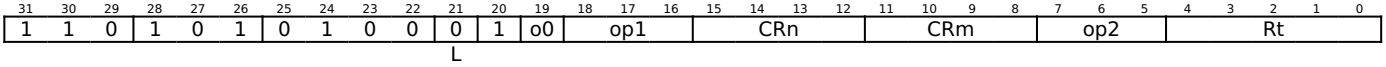
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MSR (register)

Move general-purpose register to System register

This instruction allows the PE to write an AArch64 System register from a general-purpose register.



Encoding

```
MSR (<systemreg>|S<op0>_<op1>_<Cn>_<Cm>_<op2>), <Xt>
```

Decode for this encoding

```
let t : integer       = UInt(Rt);
let sys_op0 : bits(2) = '1' :: o0;
let sys_op1 : bits(3) = op1;
let sys_op2 : bits(3) = op2;
let sys_crn : bits(4) = CRn;
let sys_crm : bits(4) = CRm;
```

Assembler Symbols

<systemreg> Is a System register name, encoded in "o0:op1:CRn:CRm:op2". The System register names are defined in *AArch64 System Register Descriptions*.

<op0> Is an unsigned immediate, encoded in “o0”:

o0	<op0>
0	2
1	3

<op1> Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op1" field.

<Cn> Is a name 'Cn', with 'n' in the range 0 to 15, encoded in the "CRn" field.

<Cm> Is a name 'Cm', with 'm' in the range 0 to 15, encoded in the "CRm" field.

<op2> Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op2" field.

<Xt> Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

```
AArch64_SysRegWrite(sys_op0, sys_op1, sys_crn, sys_crm, sys_op2, t);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

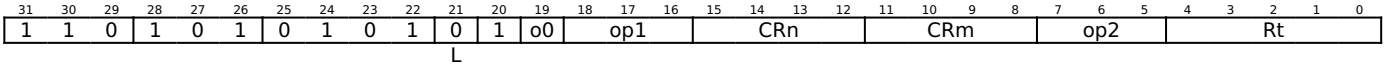
MSRR

Move two adjacent general-purpose registers to System register

This instruction allows the PE to write an AArch64 128-bit System register from two adjacent 64-bit general-purpose registers.

System

(FEAT_SYSREG128)



Encoding

```
MSRR (<systemreg>|S<op0>_<op1>_<Cn>_<Cm>_<op2>), <Xt>, <Xt+1>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SYSREG128) then EndOfDecode(Decode_UNDEF); end;
if Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;

let t : integer       = UInt(Rt);
let t2 : integer      = UInt(Rt+1);
let sys_op0 : bits(2) = '1' :: o0;
let sys_op1 : bits(3) = op1;
let sys_op2 : bits(3) = op2;
let sys_crn : bits(4) = CRn;
let sys_crm : bits(4) = CRm;
```

Assembler Symbols

<systemreg> Is a System register name, encoded in "o0:op1:CRn:CRm:op2".

<op0> Is an unsigned immediate, encoded in “o0”:

o0	<op0>
0	2
1	3

<op1> Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op1" field.

<Cn> Is a name 'Cn', with 'n' in the range 0 to 15, encoded in the "CRn" field.

<Cm> Is a name 'Cm', with 'm' in the range 0 to 15, encoded in the "CRm" field.

<op2> Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op2" field.

<Xt> Is the 64-bit name of the first general-purpose source register, encoded in the "Rt" field.

<Xt+1> Is the 64-bit name of the second general-purpose source register, encoded as "Rt" +1.

Operation

```
AArch64_SysRegWrite128(sys_op0, sys_op1, sys_crn, sys_crm, sys_op2, t, t2);
```

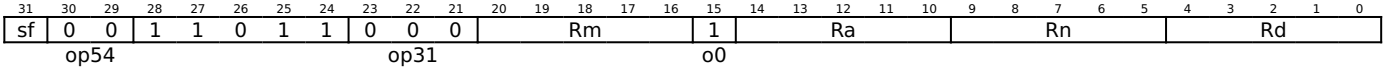
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MSUB

Multiply-subtract

This instruction multiplies two register values, subtracts the product from a third register value, and writes the result to the destination register. This instruction is used by the alias MNEG.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
MSUB <Wd>, <Wn>, <Wm>, <Wa>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MSUB <Xd>, <Xn>, <Xm>, <Xa>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Wa>	Is the 32-bit name of the third general-purpose source register holding the minuend, encoded in the "Ra" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Xa>	Is the 64-bit name of the third general-purpose source register holding the minuend, encoded in the "Ra" field.

Alias Conditions

Alias	Is preferred when
MNEG	`Ra == '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);
let operand3 : bits(datasize) = X{datasize}(a);

let result : integer = UInt(operand3) - UInt(operand1) * UInt(operand2);

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MSUBPT

Multiply-subtract checked pointer

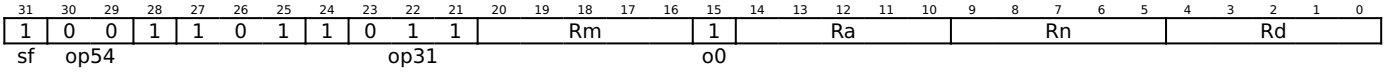
This instruction multiplies two register values, subtracts the product from a third register value, and writes the result to the destination register. The intermediate product is treated as the offset.

If the operation would have generated a result where the most significant 8 bits of the result register differ from the most significant 8 bits of the base register, then the result is modified such that it is likely to be non-canonical when used as an address.

If the intermediate product cannot be correctly represented as a 64-bit two's complement value, then the result is modified such that it is likely to be non-canonical when used as an address.

Integer

(FEAT_CPA)



Encoding

```
MSUBPT <Xd>, <Xn>, <Xm>, <Xa>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CPA) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Xa>	Is the 64-bit name of the third general-purpose source register holding the minuend, encoded in the "Ra" field.

Operation

```
let operand1 : bits(64) = X{64}(n);
let operand2 : bits(64) = X{64}(m);
let operand3 : bits(64) = X{64}(a);

var result : bits(64);

let product : integer = SInt(operand1) * SInt(operand2);

// Signed and unsigned twos complement arithmetic are equivalent if only a
// fixed number of bits are considered.
result = operand3 - product[63:0];

let overflow : boolean = (product != SInt(product[63:0]));
result = PointerMultiplyAddCheck(result, operand3, overflow);

X{64}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

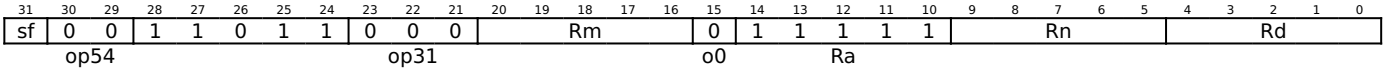
MUL

Multiply

This instruction multiplies two register values and writes the result to the destination register.

This is an alias of MADD. This means:

- The encodings in this description are named to match the encodings of MADD.
- The description of MADD gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
MUL <Wd>, <Wn>, <Wm>
```

is equivalent to

```
MADD <Wd>, <Wn>, <Wm>, WZR
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MUL <Xd>, <Xn>, <Xm>
```

is equivalent to

```
MADD <Xd>, <Xn>, <Xm>, XZR
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation

The description of MADD gives the operational pseudocode for this instruction.

Operational Information

The description of MADD gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

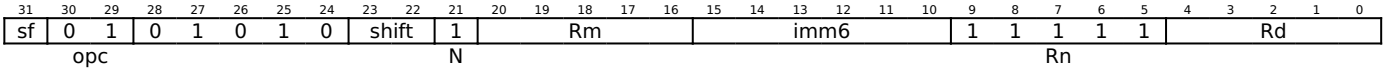
MVN

Bitwise NOT

This instruction writes the bitwise inverse of a register value to the destination register.

This is an alias of ORN (shifted register). This means:

- The encodings in this description are named to match the encodings of ORN (shifted register).
- The description of ORN (shifted register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
MVN <Wd>, <Wm>{, <shift> #<amount>}
```

is equivalent to

```
ORN <Wd>, WZR, <Wm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
MVN <Xd>, <Xm>{, <shift> #<amount>}
```

is equivalent to

```
ORN <Xd>, XZR, <Xm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wm> Is the 32-bit name of the general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift to be applied to the final source, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	ROR

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xm> Is the 64-bit name of the general-purpose source register, encoded in the "Rm" field.

Operation

The description of ORN (shifted register) gives the operational pseudocode for this instruction.

Operational Information

The description of ORN (shifted register) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

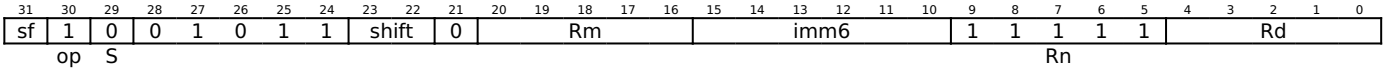
NEG (shifted register)

Negate (shifted register)

This instruction negates an optionally-shifted register value, and writes the result to the destination register.

This is an alias of SUB (shifted register). This means:

- The encodings in this description are named to match the encodings of SUB (shifted register).
- The description of SUB (shifted register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
NEG <Wd>, <Wm>{, <shift> #<amount>}
```

is equivalent to

```
SUB <Wd>, WZR, <Wm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
NEG <Xd>, <Xm>{, <shift> #<amount>}
```

is equivalent to

```
SUB <Xd>, XZR, <Xm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wm> Is the 32-bit name of the general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xm> Is the 64-bit name of the general-purpose source register, encoded in the "Rm" field.

Operation

The description of SUB (shifted register) gives the operational pseudocode for this instruction.

Operational Information

The description of SUB (shifted register) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

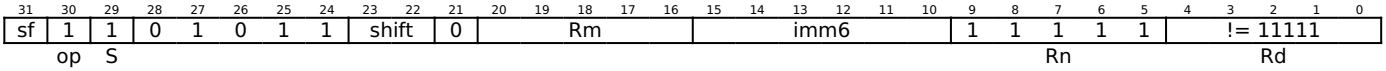
NEGS

Negate, setting flags

This instruction negates an optionally-shifted register value, and writes the result to the destination register. It updates the condition flags based on the result.

This is an alias of SUBS (shifted register). This means:

- The encodings in this description are named to match the encodings of SUBS (shifted register).
- The description of SUBS (shifted register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
NEGS <Wd>, <Wm>{, <shift> #<amount>}
```

is equivalent to

```
SUBS <Wd>, WZR, <Wm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
NEGS <Xd>, <Xm>{, <shift> #<amount>}
```

is equivalent to

```
SUBS <Xd>, XZR, <Xm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wm> Is the 32-bit name of the general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xm> Is the 64-bit name of the general-purpose source register, encoded in the "Rm" field.

Operation

The description of SUBS (shifted register) gives the operational pseudocode for this instruction.

Operational Information

The description of SUBS (shifted register) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

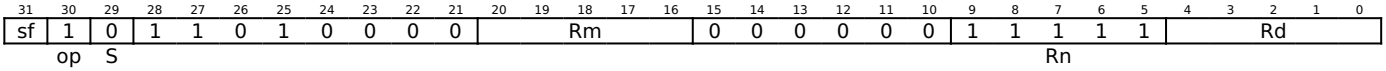
NGC

Negate with carry

This instruction negates the sum of a register value and the value of NOT (Carry flag), and writes the result to the destination register.

This is an alias of SBC. This means:

- The encodings in this description are named to match the encodings of SBC.
- The description of SBC gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
NGC <Wd>, <Wm>
```

is equivalent to

```
SBC <Wd>, WZR, <Wm>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
NGC <Xd>, <Xm>
```

is equivalent to

```
SBC <Xd>, XZR, <Xm>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wm>	Is the 32-bit name of the general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xm>	Is the 64-bit name of the general-purpose source register, encoded in the "Rm" field.

Operation

The description of SBC gives the operational pseudocode for this instruction.

Operational Information

The description of SBC gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

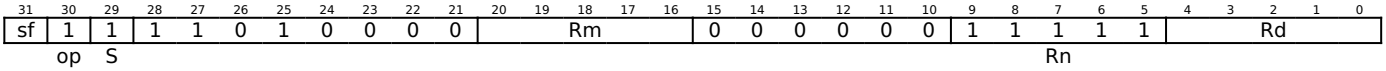
NGCS

Negate with carry, setting flags

This instruction negates the sum of a register value and the value of NOT (Carry flag), and writes the result to the destination register. It updates the condition flags based on the result.

This is an alias of SBCS. This means:

- The encodings in this description are named to match the encodings of SBCS.
- The description of SBCS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
NGCS <Wd>, <Wm>
```

is equivalent to

```
SBCS <Wd>, WZR, <Wm>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
NGCS <Xd>, <Xm>
```

is equivalent to

```
SBCS <Xd>, XZR, <Xm>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wm>	Is the 32-bit name of the general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xm>	Is the 64-bit name of the general-purpose source register, encoded in the "Rm" field.

Operation

The description of SBCS gives the operational pseudocode for this instruction.

Operational Information

The description of SBCS gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

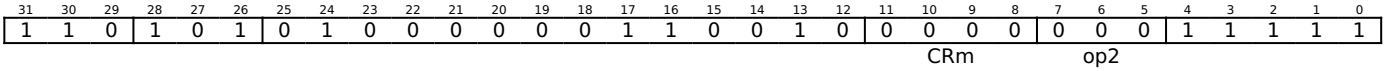
NOP

No operation

This instruction does nothing, other than advance the value of the program counter by 4. This instruction can be used for instruction alignment purposes.

Note

The timing effects of including a `NOP` instruction in a program are not guaranteed. It can increase execution time, leave it unchanged, or even reduce it. Therefore, `NOP` instructions are not suitable for timing loops.



Encoding

`NOP`

Decode for this encoding

```
// Empty.
```

Operation

```
return; // Do nothing
```

2026-09_rel 2026-09-21 14:22:38

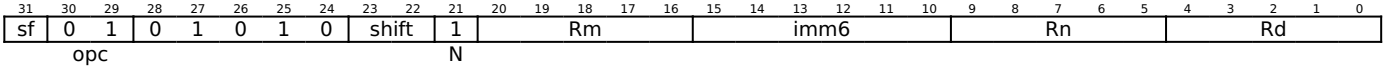
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ORN (shifted register)

Bitwise OR NOT (shifted register)

This instruction performs a bitwise (inclusive) OR between a register value and the complement of an optionally-shifted register value, and writes the result to the destination register.

This instruction is used by the alias MVN.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ORN <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ORN <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift to be applied to the final source, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	ROR

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
MVN	`Rn == '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = ShiftReg{datasize}(m, shift_type, shift_amount);
let result : bits(datasize) = operand1 OR NOT operand2;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

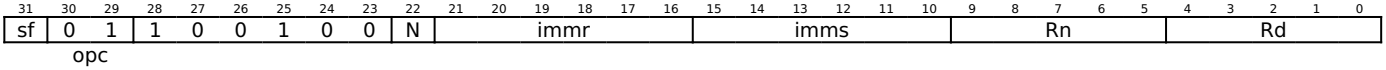
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ORR (immediate)

Bitwise OR (immediate)

This instruction performs a bitwise (inclusive) OR between a register value and an immediate value, and writes the result to the destination register. This instruction is used by the alias MOV (bitmask immediate).



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
ORR <Wd|WSP>, <Wn>, #<imm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ORR <Xd|SP>, <Xn>, #<imm>
```

Decode for all variants of this encoding

```
if sf == '0' && N != '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
var imm : bits(datasize);
(imm, -) = DecodeBitMasks{datasize}(N, imms, immr, TRUE);
```

Assembler Symbols

<Wd|WSP>	Is the 32-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<imm>	Is the bitmask immediate. For encoding information, see *Logical (immediate)*.
<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
MOV (bitmask immediate)	`Rn == '11111' && !MoveWidePreferred(sf, N, imms, immr)`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = imm;
let result : bits(datasize) = operand1 OR operand2;

if d == 31 then
    SP{64}() = ZeroExtend{64}(result);
else
    X{datasize}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

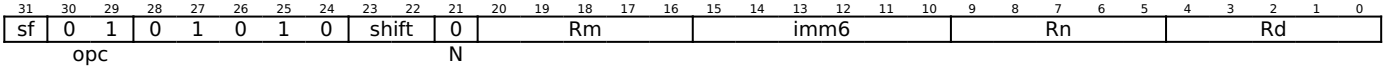
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ORR (shifted register)

Bitwise OR (shifted register)

This instruction performs a bitwise (inclusive) OR between a register value and an optionally-shifted register value, and writes the result to the destination register.

This instruction is used by the alias MOV (register).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ORR <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ORR <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift to be applied to the final source, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	ROR

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
MOV (register)	`shift == '00' && imm6 == '000000' && Rn == '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = ShiftReg{datasize}(m, shift_type, shift_amount);
let result : bits(datasize) = operand1 OR operand2;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PACDA, PACDZA

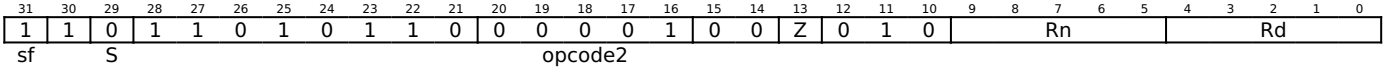
Pointer Authentication Code for data address, using key A

This instruction calculates and inserts a Pointer Authentication Code for a data address, using a modifier and key A.
The address is in the general-purpose register that is specified by <Xd>.
The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xn|SP> for PACDA.
- The value zero, for PACDZA.

Integer

(FEAT_PAuth)



Encoding for the PACDA variant

Applies when `(Z == 0)`

```
PACDA <Xd>, <Xn|SP>
```

Encoding for the PACDZA variant

Applies when `(Z == 1 && Rn == 11111)`

```
PACDZA <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

var source_is_sp : boolean = FALSE;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

if Z == '0' then // PACDA
    if n == 31 then source_is_sp = TRUE; end;
else // PACDZA
    if n != 31 then EndOfDecode(Decode_UNDEF); end;
end;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

```
if source_is_sp then
    X{64}(d) = AddPACDA(X{64}(d), SP{64}());
else
    X{64}(d) = AddPACDA(X{64}(d), X{64}(n));
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PACDB, PACDZB

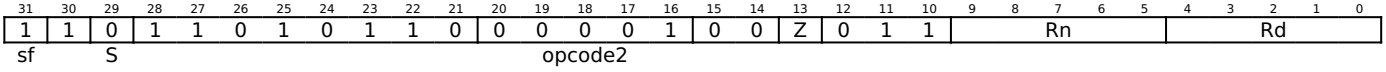
Pointer Authentication Code for data address, using key B

This instruction calculates and inserts a Pointer Authentication Code for a data address, using a modifier and key B.
The address is in the general-purpose register that is specified by <Xd>.
The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xn|SP> for PACDB.
- The value zero, for PACDZB.

Integer

(FEAT_PAuth)



Encoding for the PACDB variant

Applies when `(Z == 0)`

```
PACDB <Xd>, <Xn|SP>
```

Encoding for the PACDZB variant

Applies when `(Z == 1 && Rn == 11111)`

```
PACDZB <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

var source_is_sp : boolean = FALSE;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

if Z == '0' then // PACDB
    if n == 31 then source_is_sp = TRUE; end;
else // PACDZB
    if n != 31 then EndOfDecode(Decode_UNDEF); end;
end;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

```
if source_is_sp then
    X{64}(d) = AddPACDB(X{64}(d), SP{64}());
else
    X{64}(d) = AddPACDB(X{64}(d), X{64}(n));
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

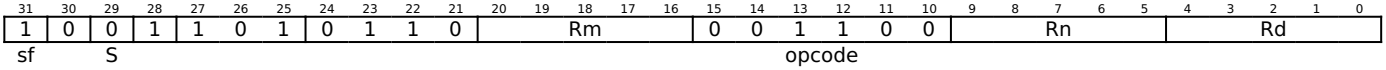
PACGA

Pointer Authentication Code, using generic key

This instruction calculates the Pointer Authentication Code for a 64-bit value in the first source register, using a modifier in the second source register, and the generic key. The calculated Pointer Authentication Code is written to the most significant 32 bits of the destination register, and the least significant 32 bits of the destination register are set to zero.

Integer

(FEAT_PAuth)



Encoding

```
PACGA <Xd>, <Xn>, <Xm|SP>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let source_is_sp : boolean = m == 31;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.

Operation

```
if source_is_sp then
    X{64}(d) = AddPACGA(X{64}(n), SP{64}());
else
    X{64}(d) = AddPACGA(X{64}(n), X{64}(m));
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA

Pointer Authentication Code for instruction address, using key A

This instruction calculates and inserts a Pointer Authentication Code for an instruction address, using a modifier and key A.
The address is:

- In the general-purpose register that is specified by <Xd> for PACIA and PACIZA.
- In X17, for PACIA1716.
- In X30, for PACIASP and PACIAZ.

The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xn|SP> for PACIA.
- The value zero, for PACIZA and PACIAZ.
- In X16, for PACIA1716.
- In SP, for PACIASP.

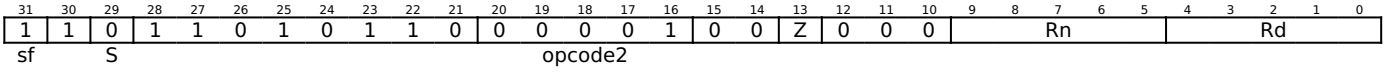
If *FEAT_PAuth_LR* is implemented and PSTATE.PACM is 1, then PACIA1716 and PACIASP include a second modifier that is:

- In X15, for PACIA1716.
- The value of PC, for PACIASP.

A PACIASP instruction has an implicit BTI instruction. The implicit BTI instruction of a PACIASP instruction is always compatible with *PSTATE*.BTYPE == 0b01 and *PSTATE*.BTYPE == 0b10. Controls in *SCTLR_ELx* configure whether the implicit BTI instruction of a PACIASP instruction is compatible with *PSTATE*.BTYPE == 0b11. For more information, see *PSTATE.BTYPE*.
It has encodings from 2 classes: Integer and System

Integer

(FEAT_PAuth)



Encoding for the PACIA variant

Applies when (Z == 0)

```
PACIA <Xd>, <Xn|SP>
```

Encoding for the PACIZA variant

Applies when (Z == 1 && Rn == 11111)

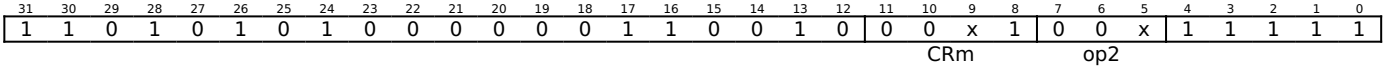
```
PACIZA <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
var source_is_sp : boolean = FALSE;
let pacia1716 : boolean = FALSE;

if Z == '0' then // PACIA
    if n == 31 then source_is_sp = TRUE; end;
else // PACIZA
    if n != 31 then EndOfDecode(Decode_UNDEF); end;
end;
```

System

(FEAT_PAuth)



Encoding for the PACIA1716 variant

Applies when (CRm == 0001 && op2 == 000)

```
PACIA1716
```

Encoding for the PACIASP variant

Applies when (CRm == 0011 && op2 == 001)

PACIASP

Encoding for the PACIAZ variant

Applies when (CRm == 0011 && op2 == 000)

PACIAZ

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_NOP); end;

var d : integer;
var n : integer;
var source_is_sp : boolean = FALSE;
var pacia1716 : boolean = FALSE;

case CRm::op2 of
    when '0011 000' => // PACIAZ
        d = 30;
        n = 31;
    when '0011 001' => // PACIASP
        d = 30;
        source_is_sp = TRUE;
        if IsFeatureImplemented(FEAT_BTI) then
            // Check for branch target compatibility between PSTATE.BTYPE
            // and implicit branch target of PACIASP instruction.
            let pacinst : PACInstType = PACIxSP;
            SetBTypeCompatible(BTypeCompatible_PAC(pacinst));
        end;
    when '0001 000' => // PACIA1716
        d = 17;
        n = 16;
        pacia1716 = TRUE;
end;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

```
if source_is_sp then
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' then
        X{64}(d) = AddPACIA2(X{64}(d), SP{64}(), PC64());
    else
        X{64}(d) = AddPACIA(X{64}(d), SP{64}());
    end;
else
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' && pacia1716 then
        X{64}(d) = AddPACIA2(X{64}(d), X{64}(n), X{64}(15));
    else
        X{64}(d) = AddPACIA(X{64}(d), X{64}(n));
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

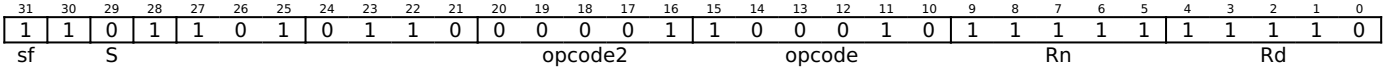
PACIA171615

Pointer Authentication Code for instruction address, using key A

This instruction calculates and inserts a pointer authentication code for an instruction address, using two modifiers and key A. The address is in X17. The 64-bit value of modifier1 is the value in X16. The 64-bit value of modifier2 is the value in X15.

Integer

(FEAT_PAuth_LR)



Encoding

PACIA171615

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
X{64}(17) = AddPACIA2(X{64}(17), X{64}(16), X{64}(15));
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

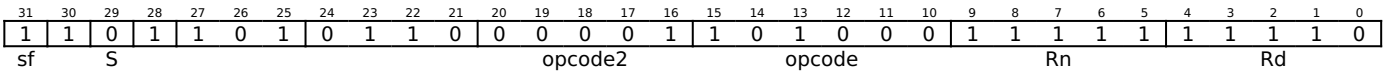
PACIASPPC

Pointer Authentication Code for return address, using key A

This instruction calculates and inserts a Pointer Authentication Code for an instruction address, using two modifiers and key A.
The address is in X30.
The first modifier is in SP.
The second modifier is the value of PC.
A `PACIASPPC` instruction has an implicit `BTI` instruction. The implicit `BTI` instruction of a `PACIASPPC` instruction is always compatible with *PSTATE*.BTYPE == 0b01 and *PSTATE*.BTYPE == 0b10. Controls in *SCTLR_ELx* configure whether the implicit `BTI` instruction of a `PACIASPPC` instruction is compatible with *PSTATE*.BTYPE == 0b11. For more information, see *PSTATE.BTYPE*.

Integer

(FEAT_PAuth_LR)



Encoding

```
PACIASPPC
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;

let d : integer = 30;
if IsFeatureImplemented(FEAT_BTI) then
    // Check for branch target compatibility between PSTATE.BTYPE
    // and implicit branch target of PACIxSPPC instruction.
    let pacinst : PACInstType = PACIxSPPC;
    SetBTypeCompatible(BTypeCompatible_PAC(pacinst));
end;
```

Operation

```
X{64}(d) = AddPACIA2(X{64}(d), SP{64}(), PC64());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB

Pointer Authentication Code for instruction address, using key B

This instruction calculates and inserts a Pointer Authentication Code for an instruction address, using a modifier and key B.
The address is:

- In the general-purpose register that is specified by <Xd> for PACIB and PACIZB.
- In X17, for PACIB1716.
- In X30, for PACIBSP and PACIBZ.

The modifier is:

- In the general-purpose register or stack pointer that is specified by <Xn|SP> for PACIB.
- The value zero, for PACIZB and PACIBZ.
- In X16, for PACIB1716.
- In SP, for PACIBSP.

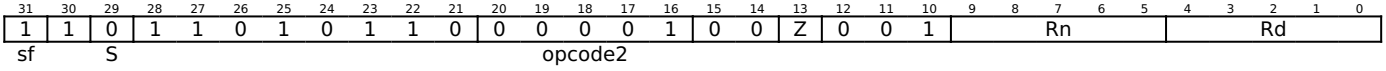
If *FEAT_PAuth_LR* is implemented and PSTATE.PACM is 1, then PACIB1716 and PACIBSP include a second modifier that is:

- In X15, for PACIB1716.
- The value of PC, for PACIBSP.

A PACIBSP instruction has an implicit BTI instruction. The implicit BTI instruction of a PACIBSP instruction is always compatible with *PSTATE*.BTYPE == 0b01 and *PSTATE*.BTYPE == 0b10. Controls in *SCTLR_ELx* configure whether the implicit BTI instruction of a PACIBSP instruction is compatible with *PSTATE*.BTYPE == 0b11. For more information, see *PSTATE.BTYPE*.
It has encodings from 2 classes: Integer and System

Integer

(FEAT_PAuth)



Encoding for the PACIB variant

Applies when (Z == 0)

```
PACIB <Xd>, <Xn|SP>
```

Encoding for the PACIZB variant

Applies when (Z == 1 && Rn == 11111)

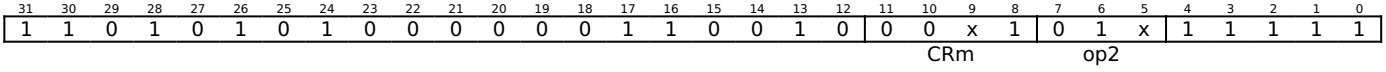
```
PACIZB <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
var source_is_sp : boolean = FALSE;
let pacib1716 : boolean = FALSE;

if Z == '0' then // PACIB
    if n == 31 then source_is_sp = TRUE; end;
else // PACIZB
    if n != 31 then EndOfDecode(Decode_UNDEF); end;
end;
```

System

(FEAT_PAuth)



Encoding for the PACIB1716 variant

Applies when (CRm == 0001 && op2 == 010)

```
PACIB1716
```

Encoding for the PACIBSP variant

Applies when (CRm == 0011 && op2 == 011)

PACIBSP

Encoding for the PACIBZ variant

Applies when (CRm == 0011 && op2 == 010)

PACIBZ

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_NOP); end;

var d : integer;
var n : integer;
var source_is_sp : boolean = FALSE;
var pacib1716 : boolean = FALSE;

case CRm::op2 of
    when '0011 010' => // PACIBZ
        d = 30;
        n = 31;
    when '0011 011' => // PACIBSP
        d = 30;
        source_is_sp = TRUE;
        if IsFeatureImplemented(FEAT_BTI) then
            // Check for branch target compatibility between PSTATE.BTYPE
            // and implicit branch target of PACIBSP instruction.
            let pacinst : PACInstType = PACIxSP;
            SetBTypeCompatible(BTypeCompatible_PAC(pacinst));
        end;
    when '0001 010' => // PACIB1716
        d = 17;
        n = 16;
        pacib1716 = TRUE;
end;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rn" field.

Operation

```
if source_is_sp then
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' then
        X{64}(d) = AddPACIB2(X{64}(d), SP{64}(), PC64());
    else
        X{64}(d) = AddPACIB(X{64}(d), SP{64}());
    end;
else
    if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' && pacib1716 then
        X{64}(d) = AddPACIB2(X{64}(d), X{64}(n), X{64}(15));
    else
        X{64}(d) = AddPACIB(X{64}(d), X{64}(n));
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

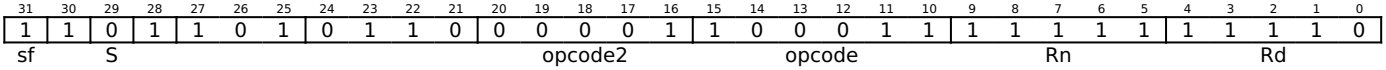
PACIB171615

Pointer Authentication Code for instruction address, using key B

This instruction calculates and inserts a pointer authentication code for an instruction address, using two modifiers and key B. The address is in X17. The 64-bit value of modifier1 is the value in X16. The 64-bit value of modifier2 is the value in X15.

Integer

(FEAT_PAuth_LR)



Encoding

PACIB171615

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
X{64}(17) = AddPACIB2(X{64}(17), X{64}(16), X{64}(15));
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

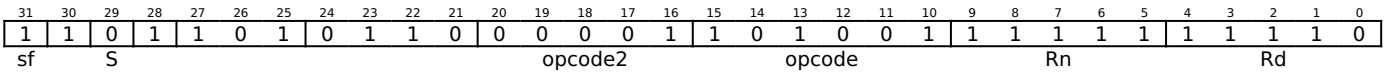
PACIBSPPC

Pointer Authentication Code for return address, using key B

This instruction calculates and inserts a Pointer Authentication Code for an instruction address, using two modifiers and key B.
The address is in X30.
The first modifier is in SP.
The second modifier is the value of PC.
A `PACIBSPPC` instruction has an implicit `BTI` instruction. The implicit `BTI` instruction of a `PACIBSPPC` instruction is always compatible with *PSTATE*.BTYPE == 0b01 and *PSTATE*.BTYPE == 0b10. Controls in *SCTLR_ELx* configure whether the implicit `BTI` instruction of a `PACIBSPPC` instruction is compatible with *PSTATE*.BTYPE == 0b11. For more information, see *PSTATE.BTYPE*.

Integer

(FEAT_PAuth_LR)



Encoding

PACIBSPPC

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;

let d : integer = 30;
if IsFeatureImplemented(FEAT_BTI) then
    // Check for branch target compatibility between PSTATE.BTYPE
    // and implicit branch target of PACIxSPPC instruction.
    let pacinst : PACInstType = PACIxSPPC;
    SetBTypeCompatible(BTypeCompatible_PAC(pacinst));
end;
```

Operation

```
X{64}(d) = AddPACIB2(X{64}(d), SP{64}(), PC64());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

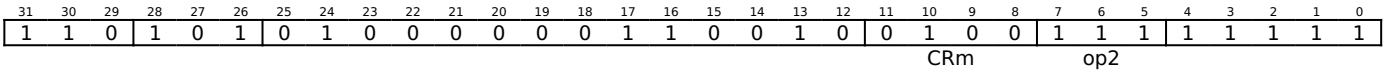
PACM

Pointer authentication modifier

This instruction is used to set the value of PSTATE.PACM to 1.
If *FEAT_PAuth_LR* is not implemented, this instruction behaves as a NOP.

System

(FEAT_PAuth_LR)



Encoding

PACM

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_NOP); end;
```

Operation

```
PSTATE.PACM = if IsPACMEnabled() then '1' else '0';
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

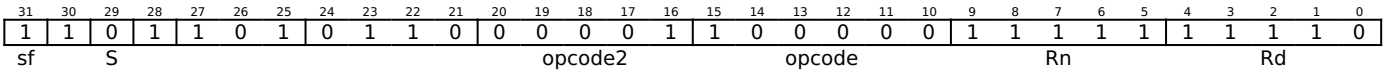
PACNBIASPPC

Pointer Authentication Code for return address, using key A, not a branch target

This instruction calculates and inserts a Pointer Authentication Code for an instruction address, using two modifiers and key A.
The address is in X30.
The first modifier is in SP.
The second modifier is the value of PC.

Integer

(FEAT_PAuth_LR)



Encoding

```
PACNBIASPPC
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;

let d : integer = 30;
```

Operation

```
X{64}(d) = AddPACIA2(X{64}(d), SP{64}(), PC64());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

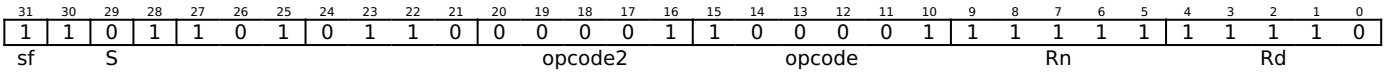
PACNBIBSPPC

Pointer Authentication Code for return address, using key B, not a branch target

This instruction calculates and inserts a Pointer Authentication Code for an instruction address, using two modifiers and key B.
The address is in X30.
The first modifier is in SP.
The second modifier is the value of PC.

Integer

(FEAT_PAuth_LR)



Encoding

```
PACNBIBSPPC
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;

let d : integer = 30;
```

Operation

```
X{64}(d) = AddPACIB2(X{64}(d), SP{64}(), PC64());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFM (immediate)

Prefetch memory (immediate)

This instruction provides a hint to the memory system.

The PRFM IR variant of this instruction signals to the memory system that the thread of execution on the current PE will read a location that might not yet have been updated with the value the thread needs, and that the memory system should optimize for low-latency delivery of that value when the write by the other observer does occur.

The other variants of this instruction signal to the memory system that memory accesses to a location are likely to occur soon. The memory system can respond by taking actions that are expected to speed up the memory access when it does occur, such as making the cache line containing the specified address available at the level of cache specified by the instruction.

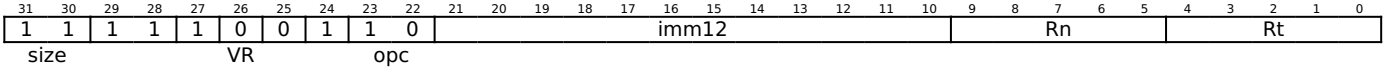
For all variants of this instruction, the address for memory accesses is calculated from a base register value and an immediate offset.

The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - When FEAT_PCDPHINT is implemented, IR for intent to read on update.
  - PLD for prefetch for load.
  - PLI for prefetch for execute.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
  - When FEAT_PRFMSLC is implemented, SLC for system level cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

The effect of a PRFM instruction is IMPLEMENTATION DEFINED. For more information, see *Prefetch memory*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
PRFM (<prfop>|#<imm5>), [<Xn|SP>{, #<pimm>}]
```

Decode for this encoding

```
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), 3);
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = FALSE;
```

Assembler Symbols

<prfop> Is the prefetch operation, encoded in “Rt”:

Rt	<prfop>	Architectural Feature
00000	PLDL1KEEP	-
00001	PLDL1STRM	-
00010	PLDL2KEEP	-
00011	PLDL2STRM	-
00100	PLDL3KEEP	-
00101	PLDL3STRM	-
00110	PLDSLCKEEP	FEAT_PRFMSLC
00111	PLDSLCSTRM	FEAT_PRFMSLC
01000	PLIL1KEEP	-
01001	PLIL1STRM	-
01010	PLIL2KEEP	-
01011	PLIL2STRM	-
01100	PLIL3KEEP	-
01101	PLIL3STRM	-
01110	PLISLCKEEP	FEAT_PRFMSLC
01111	PLISLCSTRM	FEAT_PRFMSLC
10000	PSTL1KEEP	-
10001	PSTL1STRM	-
10010	PSTL2KEEP	-
10011	PSTL2STRM	-
10100	PSTL3KEEP	-
10101	PSTL3STRM	-
10110	PSTSLCKEEP	FEAT_PRFMSLC
10111	PSTSLCSTRM	FEAT_PRFMSLC
11000	IR	FEAT_PCDPHINT

For other encodings of the "Rt" field, use <imm5>.

<imm5> Is the prefetch operation encoding as an immediate, in the range 0 to 31, encoded in the "Rt" field. This syntax is only for encodings that are not accessible using <prfop>.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<pimm> Is the optional positive immediate byte offset, a multiple of 8 in the range 0 to 32760, defaulting to 0 and encoded in the "imm12" field as <pimm>/8.

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_PREFETCH, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);
let supports_ir : boolean = IsFeatureImplemented(FEAT_PCDPHINT);
Prefetch(address, t[4:0], supports_ir);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFM (literal)

Prefetch memory (literal)

This instruction signals to the memory system that memory accesses to a location are likely to occur soon. The memory system can respond by taking actions that are expected to speed up the memory access when it does occur, such as making the cache line containing the specified address available at the level of cache specified by the instruction.

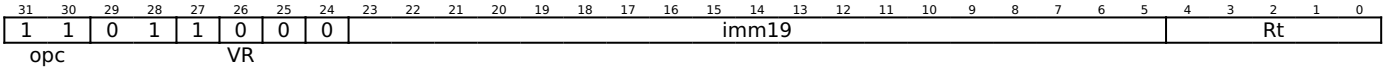
The address for memory accesses is calculated from the PC value and an immediate offset.

The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PLI for prefetch for execute.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
  - When FEAT_PRFMSLC is implemented, SLC for system level cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

The effect of a PRFM instruction is IMPLEMENTATION DEFINED. For more information, see *Prefetch memory*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
PRFM (<prfop>|#<imm5>), <label>
```

Decode for this encoding

```
let t : integer = UInt(Rt);

let offset : bits(64) = SignExtend{}(imm19::'00');
```

Assembler Symbols

<prfop> Is the prefetch operation, encoded in “Rt”:

Rt	<prfop>	Architectural Feature
00000	PLDL1KEEP	-
00001	PLDL1STRM	-
00010	PLDL2KEEP	-
00011	PLDL2STRM	-
00100	PLDL3KEEP	-
00101	PLDL3STRM	-
00110	PLDSLCKEEP	FEAT_PRFMSLC
00111	PLDSLCSTRM	FEAT_PRFMSLC
01000	PLIL1KEEP	-
01001	PLIL1STRM	-
01010	PLIL2KEEP	-
01011	PLIL2STRM	-
01100	PLIL3KEEP	-
01101	PLIL3STRM	-
01110	PLISLCKEEP	FEAT_PRFMSLC
01111	PLISLCSTRM	FEAT_PRFMSLC
10000	PSTL1KEEP	-
10001	PSTL1STRM	-
10010	PSTL2KEEP	-
10011	PSTL2STRM	-
10100	PSTL3KEEP	-
10101	PSTL3STRM	-
10110	PSTSLCKEEP	FEAT_PRFMSLC
10111	PSTSLCSTRM	FEAT_PRFMSLC

For other encodings of the "Rt" field, use <imm5>.

<imm5> Is the prefetch operation encoding as an immediate, in the range 0 to 31, encoded in the "Rt" field. This syntax is only for encodings that are not accessible using <prfop>.

<label> Is the program label from which the data is to be loaded. Its offset from the address of this instruction, in the range +/-1MB, is encoded as "imm19" times 4.

Operation

```
let address : bits(64) = PC64() + offset;

let supports_ir : boolean = FALSE;
Prefetch(address, t[4:0], supports_ir);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFM (register)

Prefetch memory (register)

This instruction signals to the memory system that memory accesses to a location are likely to occur soon. The memory system can respond by taking actions that are expected to speed up the memory access when it does occur, such as making the cache line containing the specified address available at the level of cache specified by the instruction.

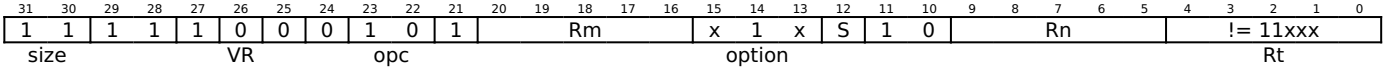
The address for memory accesses is calculated from a base register value and an offset register value. The offset register value can optionally be shifted and extended.

The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PLI for prefetch for execute.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
  - When FEAT_PRFMSLC is implemented, SLC for system level cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

The effect of a PRFM instruction is IMPLEMENTATION DEFINED. For more information, see *Prefetch memory*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
PRFM (<prfop>|#<imm5>), [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Decode for this encoding

```
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
let extend_type : ExtendType = DecodeRegExtend(option);
let shift : integer{} = if S == '1' then 3 else 0;
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let m : integer = UInt(Rm);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = FALSE;
```

Assembler Symbols

<prfop> Is the prefetch operation, encoded in “Rt”:

Rt	<prfop>	Architectural Feature
00000	PLDL1KEEP	-
00001	PLDL1STRM	-
00010	PLDL2KEEP	-
00011	PLDL2STRM	-
00100	PLDL3KEEP	-
00101	PLDL3STRM	-
00110	PLDSLCKEEP	FEAT_PRFMSLC
00111	PLDSLCSTRM	FEAT_PRFMSLC
01000	PLIL1KEEP	-
01001	PLIL1STRM	-
01010	PLIL2KEEP	-
01011	PLIL2STRM	-
01100	PLIL3KEEP	-
01101	PLIL3STRM	-
01110	PLISLCKEEP	FEAT_PRFMSLC
01111	PLISLCSTRM	FEAT_PRFMSLC
10000	PSTL1KEEP	-
10001	PSTL1STRM	-
10010	PSTL2KEEP	-
10011	PSTL2STRM	-
10100	PSTL3KEEP	-
10101	PSTL3STRM	-
10110	PSTSLCKEEP	FEAT_PRFMSLC
10111	PSTSLCSTRM	FEAT_PRFMSLC

For other encodings of the "Rt" field, use <imm5>.

<imm5> Is the prefetch operation encoding as an immediate, in the range 0 to 31, encoded in the "Rt" field. This syntax is only for encodings that are not accessible using <prfop>.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Wm> When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.

<Xm> When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.

<extend> Is the index extend/shift specifier, defaulting to LSL, and which must be omitted for the LSL option when <amount> is omitted, encoded in “option”:

option	<extend>
010	UXTW
011	LSL
110	SXTW
111	SXTX

<amount> Is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#3

Operation

```
var address : bits(64);

let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_PREFETCH, nontemporal,
                                                  privileged, tagchecked, t);

if n == 31 then
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);
let supports_ir : boolean = FALSE;
Prefetch(address, t[4:0], supports_ir);
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFUM

Prefetch memory (unscaled offset)

This instruction signals to the memory system that memory accesses to a location are likely to occur soon. The memory system can respond by taking actions that are expected to speed up the memory access when it does occur, such as making the cache line containing the specified address available at the level of cache specified by the instruction.

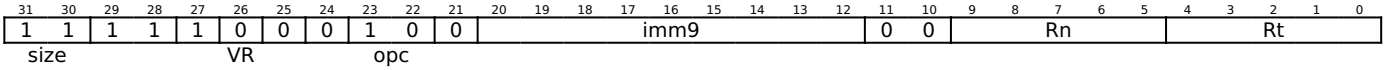
The address for memory accesses is calculated from a base register value and a signed, unscaled immediate offset.

The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PLI for prefetch for execute.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
  - When FEAT_PRFMSLC is implemented, SLC for system level cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

The effect of a PRFUM instruction is IMPLEMENTATION DEFINED. For more information, see *Prefetch memory*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
PRFUM (<prfop>|#<imm5>), [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = FALSE;
```

Assembler Symbols

<prfop> Is the prefetch operation, encoded in “Rt”:

Rt	<prfop>	Architectural Feature
00000	PLDL1KEEP	-
00001	PLDL1STRM	-
00010	PLDL2KEEP	-
00011	PLDL2STRM	-
00100	PLDL3KEEP	-
00101	PLDL3STRM	-
00110	PLDSLCKEEP	FEAT_PRFMSLC
00111	PLDSLCSTRM	FEAT_PRFMSLC
01000	PLIL1KEEP	-
01001	PLIL1STRM	-
01010	PLIL2KEEP	-
01011	PLIL2STRM	-
01100	PLIL3KEEP	-
01101	PLIL3STRM	-
01110	PLISLCKEEP	FEAT_PRFMSLC
01111	PLISLCSTRM	FEAT_PRFMSLC
10000	PSTL1KEEP	-
10001	PSTL1STRM	-
10010	PSTL2KEEP	-
10011	PSTL2STRM	-
10100	PSTL3KEEP	-
10101	PSTL3STRM	-
10110	PSTSLCKEEP	FEAT_PRFMSLC
10111	PSTSLCSTRM	FEAT_PRFMSLC

For other encodings of the "Rt" field, use <imm5>.

<imm5> Is the prefetch operation encoding as an immediate, in the range 0 to 31, encoded in the "Rt" field. This syntax is only for encodings that are not accessible using <prfop>.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<simm> Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_PREFETCH, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);
let supports_ir : boolean = FALSE;
Prefetch(address, t[4:0], supports_ir);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PSB

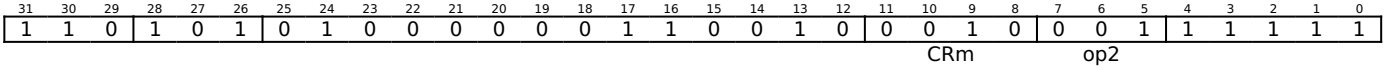
Profiling synchronization barrier

This instruction is a barrier that ensures that all existing profiling data for the current PE has been formatted, and profiling buffer addresses have been translated such that all writes to the profiling buffer have been initiated. For information about the synchronization behavior when a `DSB` instruction follows a `PSB` instruction in program order, see *Synchronization and Statistical Profiling*.

If *FEAT_SPE* is not implemented, this instruction executes as a `NOP`.

System

(FEAT_SPE)



Encoding

```
PSB CSYNC
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SPE) then EndOfDecode(Decode_NOP); end;
```

Operation

```
CheckPSBTrap();

ProfilingSynchronizationBarrier();
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

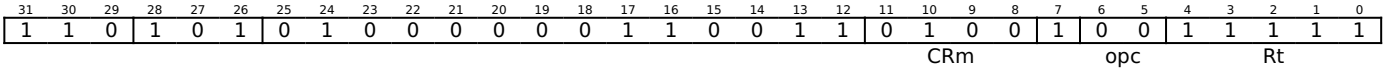
PSSBB

Physical speculative store bypass barrier

This instruction is a memory barrier that prevents speculative loads from bypassing earlier stores to the same physical address under certain conditions. For more information and details of the semantics, see *Physical Speculative Store Bypass Barrier (PSSBB)*.

This is an alias of DSB. This means:

- The encodings in this description are named to match the encodings of DSB.
- The description of DSB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

PSSBB

is equivalent to

```
DSB #4
```

and is always the preferred disassembly.

Operation

The description of DSB gives the operational pseudocode for this instruction.

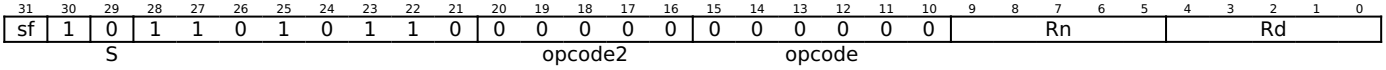
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RBIT

Reverse bits

This instruction reverses the bit order in a register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
RBIT <Wd>, <Wn>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
RBIT <Xd>, <Xn>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand : bits(datasize) = X{datasize}(n);
var result : bits(datasize);

for i = 0 to datasize-1 do
    result[(datasize-1)-i] = operand[i];
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWCAS, RCWCASA, RCWCASAL, RCWCASL

Read check write compare and swap doubleword in memory

This instruction reads a 64-bit doubleword from memory, and compares it against the value held in a register. If the comparison is equal, the value in a second register is conditionally written to memory. Storing back to memory is conditional on RCW Checks. If the compare fails or the RCW Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the update of memory.

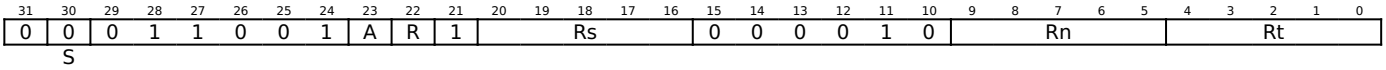
- If the destination register is not XZR, `RCWCASA` and `RCWCASAL` load from memory with acquire semantics.
- `RCWCASL` and `RCWCASAL` store to memory with release semantics.
- `RCWCAS` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_THE)



Encoding for the RCWCAS variant

Applies when `(A == 0 && R == 0)`

```
RCWCAS <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWCASA variant

Applies when `(A == 1 && R == 0)`

```
RCWCASA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWCASAL variant

Applies when `(A == 1 && R == 1)`

```
RCWCASAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWCASL variant

Applies when `(A == 0 && R == 1)`

```
RCWCASL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_THE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let soft : boolean = FALSE;

let acquire : boolean = A == '1' && s != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
let newdata : bits(64) = X{}(t);
let compdata : bits(64) = X{}(s);
var readdata : bits(64);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_CAS, soft, acquire,
                                                  release, tagchecked, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

(nzcv, readdata) = MemAtomicRCW{64}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
X{64}(s) = readdata;   // Return the old value when s!=31
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL

Read check write compare and swap quadword in memory

This instruction reads a 128-bit quadword from memory, and compares it against the value held in a pair of registers. If the comparison is equal, the value in a second pair of registers is conditionally written to memory. Storing back to memory is conditional on RCW Checks. If the compare fails or the RCW Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the update of memory.

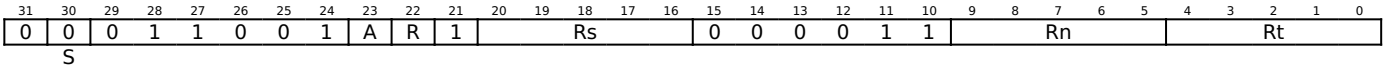
- RCWCASPA and RCWCASPAL load from memory with acquire semantics.
- RCWCASPL and RCWCASPAL store to memory with release semantics.
- RCWCASP has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_D128 && FEAT_THE)



Encoding for the RCWCASP variant

Applies when (A == 0 && R == 0)

```
RCWCASP <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>]
```

Encoding for the RCWCASPA variant

Applies when (A == 1 && R == 0)

```
RCWCASPA <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>]
```

Encoding for the RCWCASPAL variant

Applies when (A == 1 && R == 1)

```
RCWCASPAL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>]
```

Encoding for the RCWCASPL variant

Applies when (A == 0 && R == 1)

```
RCWCASPL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_D128) || !IsFeatureImplemented(FEAT_THE) then
    EndOfDecode(Decode_UNDEF);
end;
if Rs[0] == '1' then EndOfDecode(Decode_UNDEF); end;
if Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1';
let release : boolean = R == '1';
let soft : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Xs> must be an even-numbered register.
<X(s+1)>	Is the 64-bit name of the second general-purpose register to be compared and loaded.
<Xt>	Is the 64-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Xt> must be an even-numbered register.
<X(t+1)>	Is the 64-bit name of the second general-purpose register to be conditionally stored.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
var newdata : bits(128);
var compdata : bits(128);
var readdata : bits(128);
var nzcv : bits(4);

let s1 : bits(64) = X{}(s);
let s2 : bits(64) = X{}(s + 1);
let t1 : bits(64) = X{}(t);
let t2 : bits(64) = X{}(t + 1);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_CAS, soft, acquire, release,
                                                  tagchecked, t, t + 1, s, s + 1);

compdata = if BigEndian(accdesc.acctype) then s1::s2 else s2::s1;
newdata  = if BigEndian(accdesc.acctype) then t1::t2 else t2::t1;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

(nzcv, readdata) = MemAtomicRCW{128}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
if BigEndian(accdesc.acctype) then
    X{64}(s)     = readdata[127:64];
    X{64}(s + 1) = readdata[63:0];
else
    X{64}(s)     = readdata[63:0];
    X{64}(s + 1) = readdata[127:64];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL

Read check write atomic bit clear on doubleword in memory

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise AND with the complement of the value held in a register on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks. The value initially loaded from memory is returned in the destination register. If the RCW Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks.

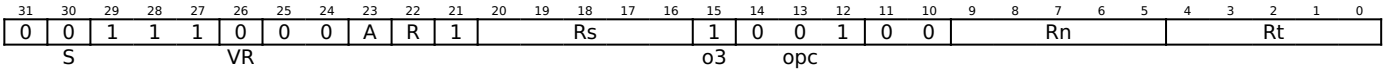
- If the destination register is not XZR, `RCWCLRA` and `RCWCLRAL` load from memory with acquire semantics.
- `RCWCLRL` and `RCWCLRAL` store to memory with release semantics.
- `RCWCLR` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_THE)



Encoding for the RCWCLR variant

Applies when `(A == 0 && R == 0)`

```
RCWCLR <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWCLRA variant

Applies when `(A == 1 && R == 0)`

```
RCWCLRA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWCLRAL variant

Applies when `(A == 1 && R == 1)`

```
RCWCLRAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWCLRL variant

Applies when `(A == 0 && R == 1)`

```
RCWCLRL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_THE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let soft : boolean = FALSE;

let acquire : boolean = A == '1' && t != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if IsD128Enabled(PSTATE.EL) then Undefined(); end;

var address : bits(64);
let newdata : bits(64) = X{}(s);
var readdata : bits(64);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_BIC, soft, acquire, release,
                                                  tagchecked, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let compdata : bits(64) = ARBITRARY : bits(64);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{64}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
X{64}(t) = readdata;    // Return the old value when t!=31
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL

Read check write atomic bit clear on quadword in memory

This instruction atomically loads a 128-bit quadword from memory, performs a bitwise AND with the complement of the value held in a pair of registers on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks. The value initially loaded from memory is returned in the same pair of registers. If the RCW Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks.

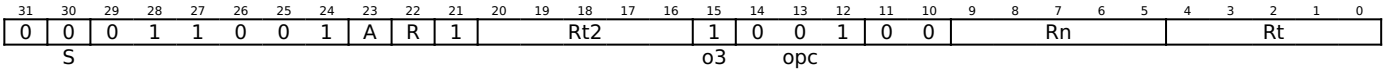
- `RCWCLRPA` and `RCWCLRPAL` load from memory with acquire semantics.
- `RCWCLRPL` and `RCWCLRPAL` store to memory with release semantics.
- `RCWCLRP` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_D128 && FEAT_THE)



Encoding for the RCWCLRP variant

Applies when `(A == 0 && R == 0)`

```
RCWCLRP <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWCLRPA variant

Applies when `(A == 1 && R == 0)`

```
RCWCLRPA <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWCLRPAL variant

Applies when `(A == 1 && R == 1)`

```
RCWCLRPAL <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWCLRPL variant

Applies when `(A == 0 && R == 1)`

```
RCWCLRPL <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_D128) || !IsFeatureImplemented(FEAT_THE) then
    EndOfDecode(Decode_UNDEF);
end;
if Rt  == '11111' then EndOfDecode(Decode_UNDEF); end;
if Rt2 == '11111' then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let soft : boolean = FALSE;

let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LSE128OVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN => rt_unknown = TRUE;    // result is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *128-bit Atomic Memory Operations*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
var value1 : bits(64);
var value2 : bits(64);
var newdata : bits(128);
var readdata : bits(128);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_BIC, soft, acquire, release,
                                                  tagchecked, t, t2, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

value1 = X{64}(t);
value2 = X{64}(t2);

newdata = if BigEndian(accdesc.acctype) then value1::value2 else value2::value1;

let compdata : bits(128) = ARBITRARY : bits(128);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{128}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
if rt_unknown then
    readdata = ARBITRARY : bits(128);
end;

if BigEndian(accdesc.acctype) then
    X{64}(t) = readdata[127:64];
    X{64}(t2) = readdata[63:0];
else
    X{64}(t) = readdata[63:0];
    X{64}(t2) = readdata[127:64];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL

Read check write software compare and swap doubleword in memory

This instruction reads a 64-bit doubleword from memory, and compares it against the value held in a register. If the comparison is equal, the value in a second register is conditionally written to memory. Storing back to memory is conditional on RCW Checks and RCWS Checks. If the compare fails, the RCW Checks fail, or the RCWS Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the update of memory.

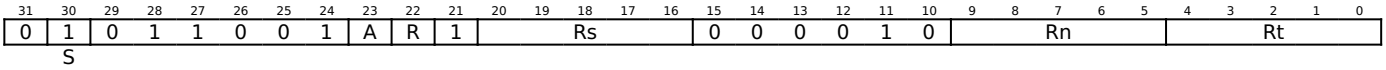
- If the destination register is not XZR, `RCWSCASA` and `RCWSCASAL` load from memory with acquire semantics.
- `RCWSCASL` and `RCWSCASAL` store to memory with release semantics.
- `RCWSCAS` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_THE)



Encoding for the RCWSCAS variant

Applies when `(A == 0 && R == 0)`

```
RCWSCAS <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSCASA variant

Applies when `(A == 1 && R == 0)`

```
RCWSCASA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSCASAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSCASAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSCASL variant

Applies when `(A == 0 && R == 1)`

```
RCWSCASL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_THE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let soft : boolean = TRUE;

let acquire : boolean = A == '1' && s != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be compared and loaded, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be conditionally stored, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
let newdata : bits(64) = X{}(t);
let compdata : bits(64) = X{}(s);
var readdata : bits(64);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_CAS, soft, acquire,
                                                  release, tagchecked, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

(nzcv, readdata) = MemAtomicRCW{64}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
X{64}(s) = readdata;   // Return the old value when s!=31
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL

Read check write software compare and swap quadword in memory

This instruction reads a 128-bit quadword from memory, and compares it against the value held in a pair of registers. If the comparison is equal, the value in a second pair of registers is conditionally written to memory. Storing back to memory is conditional on RCW Checks and RCWS Checks. If the compare fails, the RCW Checks fail, or the RCWS Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the update of memory.

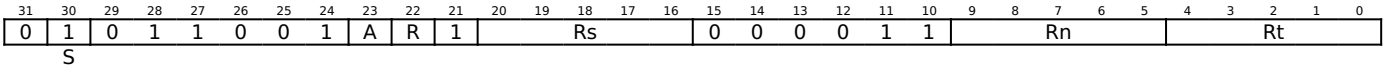
- `RCWSCASPA` and `RCWSCASPAL` load from memory with acquire semantics.
- `RCWSCASPL` and `RCWSCASPAL` store to memory with release semantics.
- `RCWSCASP` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_D128 && FEAT_THE)



Encoding for the RCWSCASP variant

Applies when `(A == 0 && R == 0)`

```
RCWSCASP <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>]
```

Encoding for the RCWSCASPA variant

Applies when `(A == 1 && R == 0)`

```
RCWSCASPA <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>]
```

Encoding for the RCWSCASPAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSCASPAL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>]
```

Encoding for the RCWSCASPL variant

Applies when `(A == 0 && R == 1)`

```
RCWSCASPL <Xs>, <X(s+1)>, <Xt>, <X(t+1)>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_D128) || !IsFeatureImplemented(FEAT_THE) then
    EndOfDecode(Decode_UNDEF);
end;
if Rs[0] == '1' then EndOfDecode(Decode_UNDEF); end;
if Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1';
let release : boolean = R == '1';
let soft : boolean = TRUE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the first general-purpose register to be compared and loaded, encoded in the "Rs" field. <Xs> must be an even-numbered register.
<X(s+1)>	Is the 64-bit name of the second general-purpose register to be compared and loaded.
<Xt>	Is the 64-bit name of the first general-purpose register to be conditionally stored, encoded in the "Rt" field. <Xt> must be an even-numbered register.
<X(t+1)>	Is the 64-bit name of the second general-purpose register to be conditionally stored.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
var newdata : bits(128);
var compdata : bits(128);
var readdata : bits(128);
var nzcv : bits(4);

let s1 : bits(64) = X{}(s);
let s2 : bits(64) = X{}(s + 1);
let t1 : bits(64) = X{}(t);
let t2 : bits(64) = X{}(t + 1);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_CAS, soft, acquire, release,
                                                  tagchecked, t, t + 1, s, s + 1);

compdata = if BigEndian(accdesc.acctype) then s1::s2 else s2::s1;
newdata  = if BigEndian(accdesc.acctype) then t1::t2 else t2::t1;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

(nzcv, readdata) = MemAtomicRCW{128}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
if BigEndian(accdesc.acctype) then
    X{64}(s)     = readdata[127:64];
    X{64}(s + 1) = readdata[63:0];
else
    X{64}(s)     = readdata[63:0];
    X{64}(s + 1) = readdata[127:64];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL

Read check write software atomic bit clear on doubleword in memory

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise AND with the complement of the value held in a register on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks and RCWS Checks. The value initially loaded from memory is returned in the destination register. If the RCW Checks fail or the RCWS Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks and RCWS Checks.

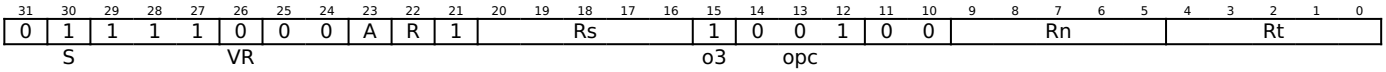
- If the destination register is not XZR, `RCWSCLRA` and `RCWSCLRAL` load from memory with acquire semantics.
- `RCWSCLRL` and `RCWSCLRAL` store to memory with release semantics.
- `RCWSCLR` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_THE)



Encoding for the RCWSCLR variant

Applies when `(A == 0 && R == 0)`

```
RCWSCLR <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSCLRA variant

Applies when `(A == 1 && R == 0)`

```
RCWSCLRA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSCLRAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSCLRAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSCLRL variant

Applies when `(A == 0 && R == 1)`

```
RCWSCLRL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_THE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let soft : boolean = TRUE;

let acquire : boolean = A == '1' && t != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if IsD128Enabled(PSTATE.EL) then Undefined(); end;

var address : bits(64);
let newdata : bits(64) = X{}(s);
var readdata : bits(64);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_BIC, soft, acquire, release,
                                                  tagchecked, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let compdata : bits(64) = ARBITRARY : bits(64);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{64}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
X{64}(t) = readdata;    // Return the old value when t!=31
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL

Read check write software atomic bit clear on quadword in memory

This instruction atomically loads a 128-bit quadword from memory, performs a bitwise AND with the complement of the value held in a pair of registers on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks and RCWS Checks. The value initially loaded from memory is returned in the same pair of registers. If the RCW Checks fail or the RCWS Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks and RCWS Checks.

- `RCWSCLRPA` and `RCWSCLRPAL` load from memory with acquire semantics.
- `RCWSCLRPL` and `RCWSCLRPAL` store to memory with release semantics.
- `RCWSCLRP` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_D128 && FEAT_THE)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	1	0	0	1	A	R	1	Rt2					1	0	0	1	0	0	Rn					Rt				
	S															o3		opc													

Encoding for the RCWSCLRP variant

Applies when `(A == 0 && R == 0)`

```
RCWSCLRP <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSCLRPA variant

Applies when `(A == 1 && R == 0)`

```
RCWSCLRPA <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSCLRPAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSCLRPAL <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSCLRPL variant

Applies when `(A == 0 && R == 1)`

```
RCWSCLRPL <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_D128) || !IsFeatureImplemented(FEAT_THE) then
    EndOfDecode(Decode_UNDEF);
end;
if Rt  == '11111' then EndOfDecode(Decode_UNDEF); end;
if Rt2 == '11111' then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let soft : boolean = TRUE;

let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LSE128OVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN => rt_unknown = TRUE;    // result is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *128-bit Atomic Memory Operations*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
var value1 : bits(64);
var value2 : bits(64);
var newdata : bits(128);
var readdata : bits(128);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_BIC, soft, acquire, release,
                                                  tagchecked, t, t2, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

value1 = X{64}(t);
value2 = X{64}(t2);

newdata = if BigEndian(accdesc.acctype) then value1::value2 else value2::value1;

let compdata : bits(128) = ARBITRARY : bits(128);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{128}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
if rt_unknown then
    readdata = ARBITRARY : bits(128);
end;

if BigEndian(accdesc.acctype) then
    X{64}(t) = readdata[127:64];
    X{64}(t2) = readdata[63:0];
else
    X{64}(t) = readdata[63:0];
    X{64}(t2) = readdata[127:64];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSET, RCWSETA, RCWSETAL, RCWSETL

Read check write atomic bit set on doubleword in memory

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise OR with the value held in a register on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks. The value initially loaded from memory is returned in the destination register. If the RCW Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks.

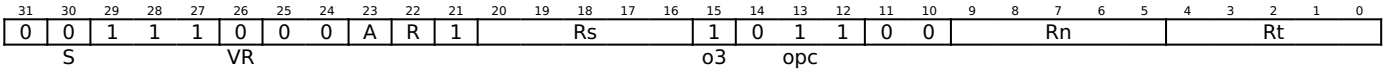
- If the destination register is not XZR, `RCWSETA` and `RCWSETAL` load from memory with acquire semantics.
- `RCWSETL` and `RCWSETAL` store to memory with release semantics.
- `RCWSET` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_THE)



Encoding for the RCWSET variant

Applies when `(A == 0 && R == 0)`

```
RCWSET <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSETA variant

Applies when `(A == 1 && R == 0)`

```
RCWSETA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSETAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSETAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSETL variant

Applies when `(A == 0 && R == 1)`

```
RCWSETL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_THE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let soft : boolean = FALSE;

let acquire : boolean = A == '1' && t != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if IsD128Enabled(PSTATE.EL) then Undefined(); end;

var address : bits(64);
let newdata : bits(64) = X{}(s);
var readdata : bits(64);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_ORR, soft, acquire, release,
                                                  tagchecked, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let compdata : bits(64) = ARBITRARY : bits(64);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{64}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
X{64}(t) = readdata;    // Return the old value when t!=31
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL

Read check write atomic bit set on quadword in memory

This instruction atomically loads a 128-bit quadword from memory, performs a bitwise OR with the value held in a pair of registers on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks. The value initially loaded from memory is returned in the same pair of registers. If the RCW Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks.

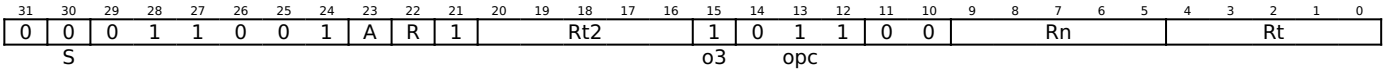
- `RCWSETPA` and `RCWSETPAL` load from memory with acquire semantics.
- `RCWSETPL` and `RCWSETPAL` store to memory with release semantics.
- `RCWSETP` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_D128 && FEAT_THE)



Encoding for the RCWSETP variant

Applies when `(A == 0 && R == 0)`

```
RCWSETP <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSETPA variant

Applies when `(A == 1 && R == 0)`

```
RCWSETPA <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSETPAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSETPAL <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSETPL variant

Applies when `(A == 0 && R == 1)`

```
RCWSETPL <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_D128) || !IsFeatureImplemented(FEAT_THE) then
    EndOfDecode(Decode_UNDEF);
end;
if Rt  == '11111' then EndOfDecode(Decode_UNDEF); end;
if Rt2 == '11111' then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let soft : boolean = FALSE;

let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LSE128OVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN => rt_unknown = TRUE;    // result is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *128-bit Atomic Memory Operations*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
var value1 : bits(64);
var value2 : bits(64);
var newdata : bits(128);
var readdata : bits(128);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_ORR, soft, acquire, release,
                                                  tagchecked, t, t2, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

value1 = X{64}(t);
value2 = X{64}(t2);

newdata = if BigEndian(accdesc.acctype) then value1::value2 else value2::value1;

let compdata : bits(128) = ARBITRARY : bits(128);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{128}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
if rt_unknown then
    readdata = ARBITRARY : bits(128);
end;

if BigEndian(accdesc.acctype) then
    X{64}(t) = readdata[127:64];
    X{64}(t2) = readdata[63:0];
else
    X{64}(t) = readdata[63:0];
    X{64}(t2) = readdata[127:64];
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL

Read check write software atomic bit set on doubleword in memory

This instruction atomically loads a 64-bit doubleword from memory, performs a bitwise OR with the value held in a register on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks and RCWS Checks. The value initially loaded from memory is returned in the destination register. If the RCW Checks fail or the RCWS Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks and RCWS Checks.

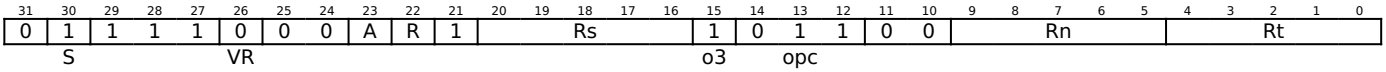
- If the destination register is not XZR, `RCWSSETA` and `RCWSSETAL` load from memory with acquire semantics.
- `RCWSSETL` and `RCWSSETAL` store to memory with release semantics.
- `RCWSSET` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_THE)



Encoding for the RCWSSET variant

Applies when `(A == 0 && R == 0)`

```
RCWSSET <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSSETA variant

Applies when `(A == 1 && R == 0)`

```
RCWSSETA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSSETAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSSETAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSSETL variant

Applies when `(A == 0 && R == 1)`

```
RCWSSETL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_THE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let soft : boolean = TRUE;

let acquire : boolean = A == '1' && t != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if IsD128Enabled(PSTATE.EL) then Undefined(); end;

var address : bits(64);
let newdata : bits(64) = X{}(s);
var readdata : bits(64);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_ORR, soft, acquire, release,
                                                  tagchecked, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let compdata : bits(64) = ARBITRARY : bits(64);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{64}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
X{64}(t) = readdata;    // Return the old value when t!=31
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL

Read check write software atomic bit set on quadword in memory

This instruction atomically loads a 128-bit quadword from memory, performs a bitwise OR with the value held in a pair of registers on it, and conditionally stores the result back to memory. Storing of the result back to memory is conditional on RCW Checks and RCWS Checks. The value initially loaded from memory is returned in the same pair of registers. If the RCW Checks fail or the RCWS Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks and RCWS Checks.

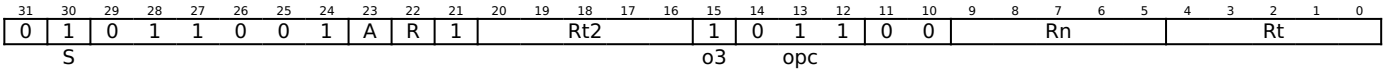
- `RCWSSETPA` and `RCWSSETPAL` load from memory with acquire semantics.
- `RCWSSETPL` and `RCWSSETPAL` store to memory with release semantics.
- `RCWSSETP` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_D128 && FEAT_THE)



Encoding for the RCWSSETP variant

Applies when `(A == 0 && R == 0)`

```
RCWSSETP <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSSETPA variant

Applies when `(A == 1 && R == 0)`

```
RCWSSETPA <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSSETPAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSSETPAL <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSSETPL variant

Applies when `(A == 0 && R == 1)`

```
RCWSSETPL <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_D128) || !IsFeatureImplemented(FEAT_THE) then
    EndOfDecode(Decode_UNDEF);
end;
if Rt  == '11111' then EndOfDecode(Decode_UNDEF); end;
if Rt2 == '11111' then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let soft : boolean = TRUE;

let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LSE128OVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN => rt_unknown = TRUE;    // result is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *128-bit Atomic Memory Operations*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
var value1 : bits(64);
var value2 : bits(64);
var newdata : bits(128);
var readdata : bits(128);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_ORR, soft, acquire, release,
                                                  tagchecked, t, t2, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

value1 = X{64}(t);
value2 = X{64}(t2);

newdata = if BigEndian(accdesc.acctype) then value1::value2 else value2::value1;

let compdata : bits(128) = ARBITRARY : bits(128);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{128}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
if rt_unknown then
    readdata = ARBITRARY : bits(128);
end;

if BigEndian(accdesc.acctype) then
    X{64}(t) = readdata[127:64];
    X{64}(t2) = readdata[63:0];
else
    X{64}(t) = readdata[63:0];
    X{64}(t2) = readdata[127:64];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL

Read check write software swap doubleword in memory

This instruction atomically loads a 64-bit doubleword from a memory location, and conditionally stores the value held in a register back to the same memory location. Storing back to memory is conditional on RCW Checks and RCWS Checks. The value initially loaded from memory is returned in the destination register. If the RCW Checks fail or the RCWS Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks and RCWS Checks.

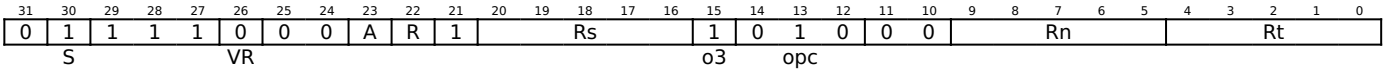
- If the destination register is not XZR, `RCWSSWPA` and `RCWSSWPAL` load from memory with acquire semantics.
- `RCWSSWPL` and `RCWSSWPAL` store to memory with release semantics.
- `RCWSSWP` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_THE)



Encoding for the RCWSSWP variant

Applies when `(A == 0 && R == 0)`

```
RCWSSWP <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSSWPA variant

Applies when `(A == 1 && R == 0)`

```
RCWSSWPA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSSWPAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSSWPAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSSWPL variant

Applies when `(A == 0 && R == 1)`

```
RCWSSWPL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_THE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let soft : boolean = TRUE;

let acquire : boolean = A == '1' && t != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
let newdata : bits(64) = X{}(s);
var readdata : bits(64);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_SWP, soft, acquire,
                                                  release, tagchecked, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let compdata : bits(64) = ARBITRARY : bits(64);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{64}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
X{64}(t) = readdata;   // Return the old value when t!=31
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL

Read check write software swap quadword in memory

This instruction atomically loads a 128-bit quadword from a memory location, and conditionally stores the value held in a pair of registers back to the same memory location. Storing back to memory is conditional on RCW Checks and RCWS Checks. The value initially loaded from memory is returned in the same pair of registers. If the RCW Checks fail or the RCWS Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks and RCWS Checks.

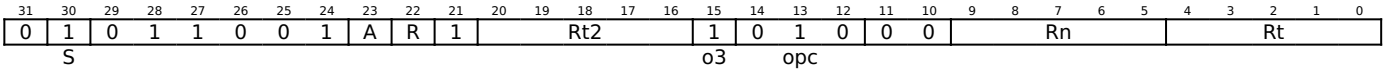
- `RCWSSWPPA` and `RCWSSWPPAL` load from memory with acquire semantics.
- `RCWSSWPPL` and `RCWSSWPPAL` store to memory with release semantics.
- `RCWSSWPP` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_D128 && FEAT_THE)



Encoding for the RCWSSWPP variant

Applies when `(A == 0 && R == 0)`

```
RCWSSWPP <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSSWPPA variant

Applies when `(A == 1 && R == 0)`

```
RCWSSWPPA <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSSWPPAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSSWPPAL <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSSWPPL variant

Applies when `(A == 0 && R == 1)`

```
RCWSSWPPL <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_D128) || !IsFeatureImplemented(FEAT_THE) then
    EndOfDecode(Decode_UNDEF);
end;
if Rt  == '11111' then EndOfDecode(Decode_UNDEF); end;
if Rt2 == '11111' then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let soft : boolean = TRUE;

let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LSE128OVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *128-bit Atomic Memory Operations*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
var value1 : bits(64);
var value2 : bits(64);
var newdata : bits(128);
var readdata : bits(128);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_SWP, soft, acquire,
                                                  release, tagchecked, t, t2, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

value1 = X{64}(t);
value2 = X{64}(t2);

newdata = if BigEndian(accdesc.acctype) then value1::value2 else value2::value1;

let compdata : bits(128) = ARBITRARY : bits(128);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{128}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
if rt_unknown then
    readdata = ARBITRARY : bits(128);
end;

if BigEndian(accdesc.acctype) then
    X{64}(t)  = readdata[127:64];
    X{64}(t2) = readdata[63:0];
else
    X{64}(t)  = readdata[63:0];
    X{64}(t2) = readdata[127:64];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL

Read check write swap doubleword in memory

This instruction atomically loads a 64-bit doubleword from a memory location, and conditionally stores the value held in a register back to the same memory location. Storing back to memory is conditional on RCW Checks. The value initially loaded from memory is returned in the destination register. If the RCW Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks.

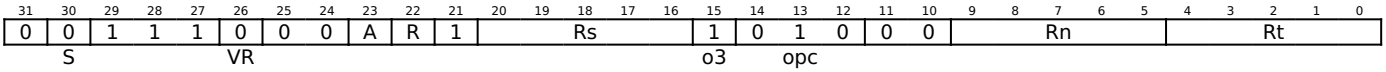
- If the destination register is not XZR, `RCWSWPA` and `RCWSWPAL` load from memory with acquire semantics.
- `RCWSWPL` and `RCWSWPAL` store to memory with release semantics.
- `RCWSWP` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_THE)



Encoding for the RCWSWP variant

Applies when `(A == 0 && R == 0)`

```
RCWSWP <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSWPA variant

Applies when `(A == 1 && R == 0)`

```
RCWSWPA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSWPAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSWPAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the RCWSWPL variant

Applies when `(A == 0 && R == 1)`

```
RCWSWPL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_THE) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let soft : boolean = FALSE;

let acquire : boolean = A == '1' && t != 31;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xs>	Is the 64-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
let newdata : bits(64) = X{}(s);
var readdata : bits(64);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_SWP, soft, acquire,
                                                  release, tagchecked, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let compdata : bits(64) = ARBITRARY : bits(64);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{64}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
X{64}(t) = readdata;   // Return the old value when t!=31
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL

Read check write swap quadword in memory

This instruction atomically loads a 128-bit quadword from a memory location, and conditionally stores the value held in a pair of registers back to the same memory location. Storing back to memory is conditional on RCW Checks. The value initially loaded from memory is returned in the same pair of registers. If the RCW Checks fail, the architecture permits writing the value read from the location to memory. If the write is performed, the read and the write occur atomically such that no other modification of the memory location can take place between the read and the write. This instruction updates the condition flags based on the result of the RCW Checks.

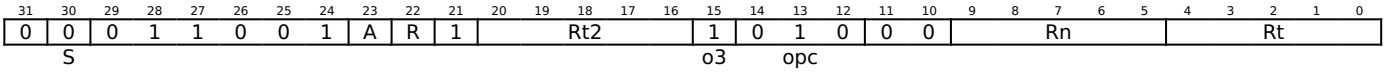
- `RCWSWPPA` and `RCWSWPPAL` load from memory with acquire semantics.
- `RCWSWPPL` and `RCWSWPPAL` store to memory with release semantics.
- `RCWSWPP` has neither acquire nor release semantics.

Note

This instruction is for performing atomic updates of translation table entries and not for general use.

Integer

(FEAT_D128 && FEAT_THE)



Encoding for the RCWSWPP variant

Applies when `(A == 0 && R == 0)`

```
RCWSWPP <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSWPPA variant

Applies when `(A == 1 && R == 0)`

```
RCWSWPPA <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSWPPAL variant

Applies when `(A == 1 && R == 1)`

```
RCWSWPPAL <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the RCWSWPPL variant

Applies when `(A == 0 && R == 1)`

```
RCWSWPPL <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_D128) || !IsFeatureImplemented(FEAT_THE) then
    EndOfDecode(Decode_UNDEF);
end;
if Rt  == '11111' then EndOfDecode(Decode_UNDEF); end;
if Rt2 == '11111' then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let soft : boolean = FALSE;

let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LSE128OVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *128-bit Atomic Memory Operations*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !IsD128Enabled(PSTATE.EL) then Undefined(); end;
var address : bits(64);
var value1 : bits(64);
var value2 : bits(64);
var newdata : bits(128);
var readdata : bits(128);
var nzcv : bits(4);

let accdesc : AccessDescriptor = CreateAccDescRCW(MemAtomicOp_SWP, soft, acquire,
                                                  release, tagchecked, t, t2, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

value1 = X{64}(t);
value2 = X{64}(t2);

newdata = if BigEndian(accdesc.acctype) then value1::value2 else value2::value1;

let compdata : bits(128) = ARBITRARY : bits(128);    // Irrelevant when not executing CAS
(nzcv, readdata) = MemAtomicRCW{128}(address, compdata, newdata, accdesc);

PSTATE.[N,Z,C,V] = nzcv;
if rt_unknown then
    readdata = ARBITRARY : bits(128);
end;

if BigEndian(accdesc.acctype) then
    X{64}(t)  = readdata[127:64];
    X{64}(t2) = readdata[63:0];
else
    X{64}(t)  = readdata[63:0];
    X{64}(t2) = readdata[127:64];
end;
```

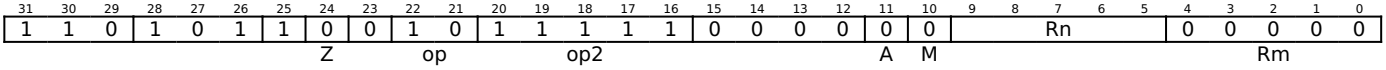
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RET

Return from subroutine

This instruction branches unconditionally to an address in a register. This instruction provides a hint that this is a subroutine return.



Encoding

```
RET {<Xn>}
```

Decode for this encoding

```
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xn>	Is the 64-bit name of the general-purpose register holding the address to be branched to, encoded in the "Rn" field. Defaults to X30 if absent.

Operation

```
var target : bits(64) = X{}(n);
if IsFeatureImplemented(FEAT_GCS) && GCSPCREnabled(PSTATE.EL) then
    target = LoadCheckGCSRecord(target, GCSInstType_PRET);
    SetCurrentGCSPointer(GetCurrentGCSPointer() + 8);
end;

// Value in BTypeNext will be used to set PSTATE.BTYPE
BTypeNext = '00';

let branch_conditional : boolean = FALSE;
BranchTo{64}(target, BranchType_RET, branch_conditional);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RETAA, RETAB

Return from subroutine, with pointer authentication

This instruction authenticates the address that is held in LR, using SP as the modifier and the specified key, and branches to the authenticated address.

Key A is used for `RETAA`. Key B is used for `RETAB`.

If the authentication passes, the PE continues execution at the target of the branch. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.

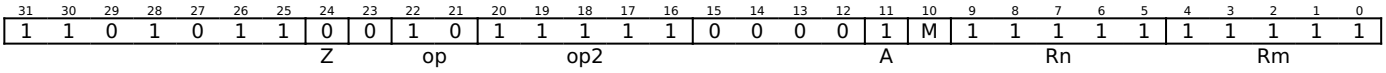
The authenticated address is not written back to LR.

This instruction provides a hint that this is a subroutine return.

If *FEAT_PAuth_LR* is implemented and PSTATE.PACM is 1, then `RETAA` and `RETAB` include a second modifier that is in X16.

Integer

(FEAT_PAuth)



Encoding for the RETAA variant

Applies when `(M == 0)`

`RETAA`

Encoding for the RETAB variant

Applies when `(M == 1)`

`RETAB`

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let use_key_a : boolean = (M == '0');
let auth_then_branch : boolean = TRUE;
```

Operation

```
var inst_type : GCSInstruction;
var target : bits(64) = X{}(30);

let modifier : bits(64) = SP{}();
var modifier2 : bits(64);
var use_modifier2 : boolean = FALSE;
if IsFeatureImplemented(FEAT_PAuth_LR) && PSTATE.PACM == '1' then
    modifier2 = X{64}(16);
    use_modifier2 = TRUE;
end;

if use_key_a then
    if use_modifier2 && IsFeatureImplemented(FEAT_PAuth_LR) then
        target = AuthIA2(target, modifier, modifier2, auth_then_branch);
    else
        target = AuthIA(target, modifier, auth_then_branch);
    end;
else
    if use_modifier2 && IsFeatureImplemented(FEAT_PAuth_LR) then
        target = AuthIB2(target, modifier, modifier2, auth_then_branch);
    else
        target = AuthIB(target, modifier, auth_then_branch);
    end;
end;

if IsFeatureImplemented(FEAT_GCS) && GCSPCREnabled(PSTATE.EL) then
    inst_type = if use_key_a then GCSInstType_PRETAA else GCSInstType_PRETAB;
    target = LoadCheckGCSRecord(target, inst_type);
    SetCurrentGCSPointer(GetCurrentGCSPointer() + 8);
end;

// Value in BTypeNext will be used to set PSTATE.BTYPE
BTypeNext = '00';

let branch_conditional : boolean = FALSE;
BranchTo{64}(target, BranchType_RET, branch_conditional);
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RETAASPPC, RETABSPPC

Return from subroutine, with enhanced pointer authentication using an immediate offset

This instruction authenticates the address that is held in LR, using SP as the first modifier, the specified immediate subtracted from PC as the second modifier, and the specified key, and branches to the authenticated address.

Key A is used for `RETAASPPC`. Key B is used for `RETABSPPC`.

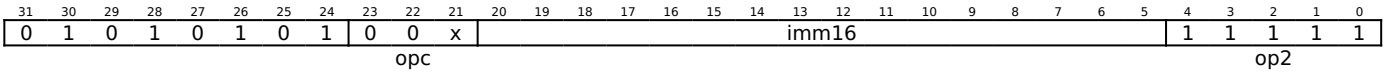
If the authentication passes, the PE continues execution at the target of the branch. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.

The authenticated address is not written back to LR.

This instruction provides a hint that this is a subroutine return.

Integer

(FEAT_PAuth_LR)



Encoding for the RETAASPPC variant

Applies when `(opc == 000)`

```
RETAASPPC <label>
```

Encoding for the RETABSPPC variant

Applies when `(opc == 001)`

```
RETABSPPC <label>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;

let use_key_a : boolean = opc[0] == '0';
let offset : bits(64) = ZeroExtend{}(imm16::'00');
let auth_then_branch : boolean = TRUE;
```

Assembler Symbols

<label>	Is the program label whose address is to be calculated. Its negative offset from the address of this instruction, a multiple of 4 in the range -262140 to 0, is encoded as an unsigned value in the "imm16" field as <label>/4.

Operation

```
var inst_type : GCSInstruction;
var target : bits(64) = X{64}(30);

let modifier : bits(64) = SP{64}();
let modifier2 : bits(64) = PC64() - offset;

if use_key_a then
    target = AuthIA2(target, modifier, modifier2, auth_then_branch);
else
    target = AuthIB2(target, modifier, modifier2, auth_then_branch);
end;

if IsFeatureImplemented(FEAT_GCS) && GCSPCREnabled(PSTATE.EL) then
    inst_type = if use_key_a then GCSInstType_PRETAA else GCSInstType_PRETAB;
    target = LoadCheckGCSRecord(target, inst_type);
    SetCurrentGCSPointer(GetCurrentGCSPointer() + 8);
end;

// Value in BTypeNext will be used to set PSTATE.BTYPE
BTypeNext = '00';

let branch_conditional : boolean = FALSE;
BranchTo{64}(target, BranchType_RET, branch_conditional);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RETAASPPCR, RETABSPPCR

Return from subroutine, with enhanced pointer authentication using a register

This instruction authenticates the address that is held in LR, using SP as the first modifier, the value in the specified register as the second modifier, and the specified key, and branches to the authenticated address.

Key A is used for `RETAASPPCR`. Key B is used for `RETABSPPCR`.

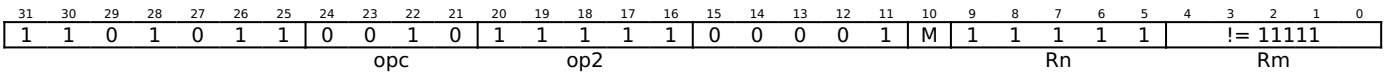
If the authentication passes, the PE continues execution at the target of the branch. For information on behavior if the authentication fails, see *Faulting on pointer authentication*.

The authenticated address is not written back to LR.

This instruction provides a hint that this is a subroutine return.

Integer

(FEAT_PAuth_LR)



Encoding for the RETAASPPCR variant

Applies when `(M == 0)`

```
RETAASPPCR <Xm>
```

Encoding for the RETABSPPCR variant

Applies when `(M == 1)`

```
RETABSPPCR <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth_LR) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Rm);
let use_key_a : boolean = M == '0';
let auth_then_branch : boolean = TRUE;
```

Assembler Symbols

<Xm>	Is the 64-bit name of the general-purpose source register, encoded in the "Rm" field.

Operation

```
var inst_type : GCSInstruction;
var target : bits(64) = X{64}(30);

let modifier : bits(64) = SP{64}();
let modifier2 : bits(64) = X{64}(m);

if use_key_a then
    target = AuthIA2(target, modifier, modifier2, auth_then_branch);
else
    target = AuthIB2(target, modifier, modifier2, auth_then_branch);
end;

if IsFeatureImplemented(FEAT_GCS) && GCSPCREnabled(PSTATE.EL) then
    inst_type = if use_key_a then GCSInstType_PRETAA else GCSInstType_PRETAB;
    target = LoadCheckGCSRecord(target, inst_type);
    SetCurrentGCSPointer(GetCurrentGCSPointer() + 8);
end;

// Value in BTypeNext will be used to set PSTATE.BTYPE
BTypeNext = '00';

let branch_conditional : boolean = FALSE;
BranchTo{64}(target, BranchType_RET, branch_conditional);
```

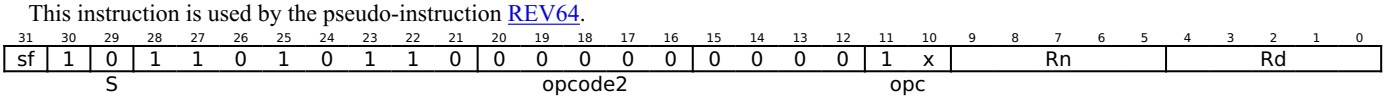
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

REV

Reverse bytes

This instruction reverses the byte order in a register.
This instruction is used by the pseudo-instruction REV64.



Encoding for the 32-bit variant

Applies when `(sf == 0 && opc == 10)`

```
REV <Wd>, <Wn>
```

Encoding for the 64-bit variant

Applies when `(sf == 1 && opc == 11)`

```
REV <Xd>, <Xn>
```

Decode for all variants of this encoding

```
if opc == '11' && sf == '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let container_size : integer{} = 8 << UInt(opc);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand : bits(datasize) = X{datasize}(n);
var result : bits(datasize);

let containers : integer = datasize DIV container_size;
for c = 0 to containers-1 do
    let container : bits(container_size) = operand[c*:container_size];
    result[c*:container_size] = Reverse{container_size}(container, 8);
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

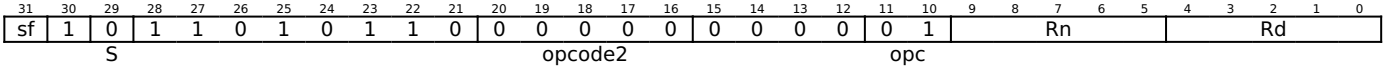
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

REV16

Reverse bytes in 16-bit halfwords

This instruction reverses the byte order in each 16-bit halfword of a register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
REV16 <Wd>, <Wn>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
REV16 <Xd>, <Xn>
```

Decode for all variants of this encoding

```
if opc == '11' && sf == '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let container_size : integer{} = 8 << UInt(opc);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand : bits(datasize) = X{datasize}(n);
var result : bits(datasize);

let containers : integer = datasize DIV container_size;
for c = 0 to containers-1 do
    let container : bits(container_size) = operand[c*:container_size];
    result[c*:container_size] = Reverse{container_size}(container, 8);
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

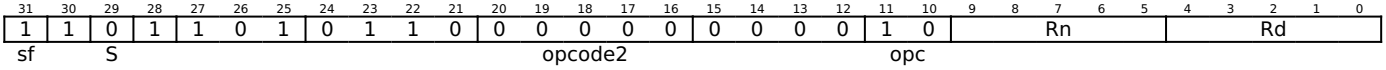
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

REV32

Reverse bytes in 32-bit words

This instruction reverses the byte order in each 32-bit word of a register.



Encoding

```
REV32 <Xd>, <Xn>
```

Decode for this encoding

```
if opc == '11' && sf == '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let container_size : integer{} = 8 << UInt(opc);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand : bits(datasize) = X{datasize}(n);
var result : bits(datasize);

let containers : integer = datasize DIV container_size;
for c = 0 to containers-1 do
    let container : bits(container_size) = operand[c*:container_size];
    result[c*:container_size] = Reverse{container_size}(container, 8);
end;

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

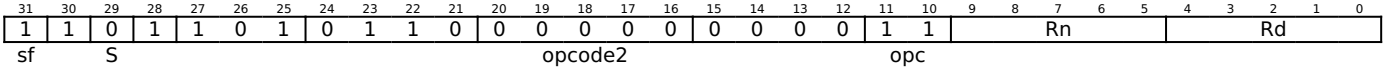
REV64

Reverse bytes

This instruction reverses the byte order in a 64-bit general-purpose register.
When assembling for Armv8.2, an assembler must support this pseudo-instruction. It is OPTIONAL whether an assembler supports this pseudo-instruction when assembling for an architecture earlier than Armv8.2.

This is a pseudo-instruction of REV. This means:

- The encodings in this description are named to match the encodings of REV.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of REV gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

```
REV64 <Xd>, <Xn>
```

is equivalent to

```
REV <Xd>, <Xn>
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of REV gives the operational pseudocode for this instruction.

Operational Information

The description of REV gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

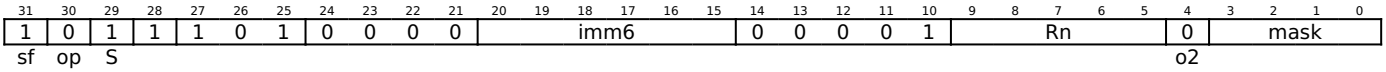
RMIF

Rotate, mask insert flags

This instruction performs a rotation right of a value held in a general-purpose register by an immediate value, and then inserts a selection of the bottom four bits of the result of the rotation into the PSTATE flags, under the control of a second immediate mask.

Integer

(FEAT_FlagM)



Encoding

```
RMIF <Xn>, #<shift>, #<mask>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FlagM) then EndOfDecode(Decode_UNDEF); end;
let imm : integer{} = UInt(imm6);
let flagmask : bits(4) = mask;
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.
<shift>	Is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.
<mask>	Is the flag bit mask, an immediate in the range 0 to 15, which selects the bits that are inserted into the NZCV condition flags, encoded in the "mask" field.

Operation

```
let reg : bits(64) = X{64}(n);
let flags : bits(4) = (reg::reg)[imm+3:imm];
if flagmask[3] == '1' then PSTATE.N = flags[3]; end;
if flagmask[2] == '1' then PSTATE.Z = flags[2]; end;
if flagmask[1] == '1' then PSTATE.C = flags[1]; end;
if flagmask[0] == '1' then PSTATE.V = flags[0]; end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

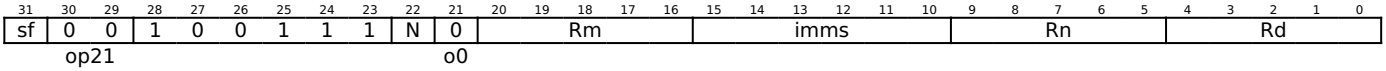
ROR (immediate)

Rotate right (immediate)

This instruction provides the value of the contents of a register rotated by a variable number of bits. The bits that are rotated off the right end are inserted into the vacated bit positions on the left.

This is an alias of EXTR. This means:

- The encodings in this description are named to match the encodings of EXTR.
- The description of EXTR gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0 && imms == 0xxxxx)`

```
ROR <Wd>, <Ws>, #<shift>
```

is equivalent to

```
EXTR <Wd>, <Ws>, <Ws>, #<shift>
```

and is the preferred disassembly when `Rn == Rm`.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
ROR <Xd>, <Xs>, #<shift>
```

is equivalent to

```
EXTR <Xd>, <Xs>, <Xs>, #<shift>
```

and is the preferred disassembly when `Rn == Rm`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Ws>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.
<shift>	For the "32-bit" variant: is the amount by which to rotate, in the range 0 to 31, encoded in the "imms" field. For the "64-bit" variant: is the amount by which to rotate, in the range 0 to 63, encoded in the "imms" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xs>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" and "Rm" fields.

Operation

The description of EXTR gives the operational pseudocode for this instruction.

Operational Information

The description of EXTR gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

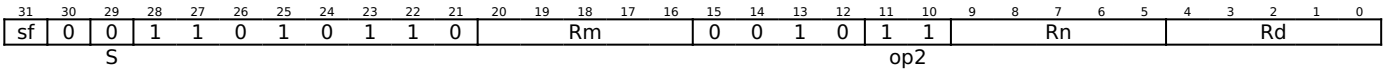
ROR (register)

Rotate right (register)

This instruction provides the value of the contents of a register rotated by a variable number of bits. The bits that are rotated off the right end are inserted into the vacated bit positions on the left. The value of the second source register modulo the register size in bits gives the number of bits by which the first source register is right-shifted.

This is an alias of RORV. This means:

- The encodings in this description are named to match the encodings of RORV.
- The description of RORV gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
ROR <Wd>, <Wn>, <Wm>
```

is equivalent to

```
RORV <Wd>, <Wn>, <Wm>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
ROR <Xd>, <Xn>, <Xm>
```

is equivalent to

```
RORV <Xd>, <Xn>, <Xm>
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding a shift amount from 0 to 31 in its bottom 5 bits, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding a shift amount from 0 to 63 in its bottom 6 bits, encoded in the "Rm" field.

Operation

The description of RORV gives the operational pseudocode for this instruction.

Operational Information

The description of RORV gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

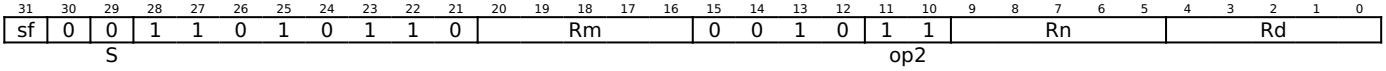
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RORV

Rotate right variable

This instruction provides the value of the contents of a register rotated by a variable number of bits. The bits that are rotated off the right end are inserted into the vacated bit positions on the left. The value of the second source register modulo the register size in bits gives the number of bits by which the first source register is right-shifted.

This instruction is used by the alias ROR (register).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
RORV <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
RORV <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(op2);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding a shift amount from 0 to 31 in its bottom 5 bits, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding a shift amount from 0 to 63 in its bottom 6 bits, encoded in the "Rm" field.

Operation

```
let operand2 : bits(datasize) = X{datasize}(m);

X{datasize}(d) = ShiftReg{datasize}(n, shift_type, UInt(operand2) MOD datasize);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RPRFM

Range prefetch memory

This instruction signals the memory system that data memory accesses from a specified range of addresses are likely to occur in the near future. The instruction may also signal the memory system about the likelihood of data reuse of the specified range of addresses. The memory system can respond by taking actions that are expected to speed up the memory accesses when they do occur, such as prefetching locations within the specified address ranges into one or more caches. The memory system may also exploit the data reuse hints to decide whether to retain the data in other caches upon eviction from the innermost caches or to discard it.

The effect of an RPRFM instruction is IMPLEMENTATION DEFINED. For more information, see *Prefetch memory*.

An RPRFM instruction specifies the type of accesses and range of addresses using the following parameters:

- 'Access type', encoded in Rt<0>, specifies:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- 'Policy', encoded in Rt<2:1>, specifies:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once. If a streaming prefetch is specified, then the 'ReuseDistance' parameter is ignored.
- 'BaseAddress', in the 64-bit base register, holds the initial block address for the accesses.
- 'ReuseDistance', in the metadata register bits[63:60], indicates the maximum number of bytes to be accessed by this PE before executing the next RPRFM instruction that specifies the same range. This includes the total number of bytes inside and outside of the range that will be accessed by the same PE. This parameter can be used to influence cache eviction and replacement policies, in order to retain the data in the most optimal levels of the memory hierarchy after each access. If software cannot easily determine the amount of other memory that will be accessed, these bits can be set to zero to indicate that 'ReuseDistance' is not known. Otherwise, these four bits encode decreasing powers of two in the range 512MiB (0b0001) to 32KiB (0b1111).
- 'Stride', in the metadata register bits[59:38], is a signed, two's complement integer encoding of the number of bytes to advance the block address after 'Length' bytes have been accessed, in the range -2MiB to +2MiB-1B. A negative value indicates that the block address is advanced in a descending direction.
- 'Count', in the metadata register bits[37:22], is an unsigned integer encoding of the number of blocks of data to be accessed minus 1, representing the range 1 to 65536 blocks. If 'Count' is 0, then the 'Stride' parameter is ignored and only a single block of contiguous bytes from 'BaseAddress' to ('BaseAddress' + 'Length' - 1) is described.
- 'Length', in the metadata register bits[21:0], is a signed, two's complement integer encoding of the number of contiguous bytes to be accessed starting from the current block address, without changing the block address, in the range -2MiB to +2MiB-1B. A negative value indicates that the bytes are accessed in a descending direction.

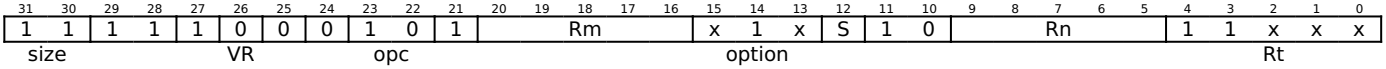
Note

Software is expected to honor the parameters it provides to the RPRFM instruction, and the same PE should access all locations in the range, in the direction specified by the sign of the 'Length' and 'Stride' parameters. A range prefetch is considered active on a PE until all locations in the range have been accessed by the PE. A range prefetch might also be inactivated by the PE prior to completion, for example due to a software context switch or lack of hardware resources.

Software should not specify overlapping addresses in multiple active ranges. If a range is expected to be accessed by both load and store instructions (read-modify-write), then a single range with an 'Access type' parameter of PST (prefetch for store) should be specified.

Integer

(FEAT_RPRFM)



Encoding

```
RPRFM  (<rprfop>|#<imm6>), <Xm>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RPRFM) then EndOfDecode(Decode_NOP); end;
let operation : bits(6) = option[2]::option[0]::S::Rt[2:0];
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
```

Assembler Symbols

<rprfop> Is the range prefetch operation, encoded in “(option :: S :: Rt)”:

option	S	Rt	<rprfop>
010	0	11000	PLDKEEP
010	0	11001	PSTKEEP
010	0	11100	PLDSTRM
010	0	11101	PSTSTRM

For other encodings, use <imm6>.

<imm6> Is the range prefetch operation encoding as an immediate, in the range 0 to 63, encoded in "option<2>:option<0>:S:Rt<2:0>". This syntax is only for encodings that are not representable using <rprfop>.

<Xm> Is the 64-bit name of the general-purpose register that holds an encoding of the metadata, encoded in the "Rm" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
let address : bits(64)  = if n == 31 then SP{64}() else X{64}(n);
let metadata : bits(64) = X{}(m);
let stride : integer = SInt(metadata[59:38]);
let count : integer  = UInt(metadata[37:22]) + 1;
let length : integer = SInt(metadata[21:0]);
var reuse : integer;

if metadata[63:60] == '0000' then
    reuse = -1; // Not known
else
    reuse = 32768 << (15 - UInt(metadata[63:60]));
end;

Hint_RangePrefetch(address, length, stride, count, reuse, operation);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

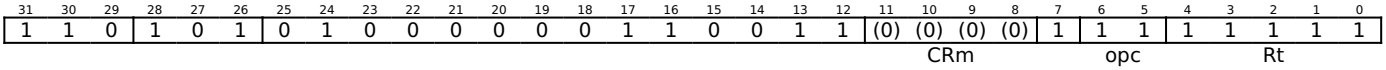
SB

Speculation barrier

This instruction is a barrier that controls speculation. For more information and details of the semantics, see *Speculation Barrier (SB)*.

System

(FEAT_SB)



Encoding

SB

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SB) then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
SpeculationBarrier();
```

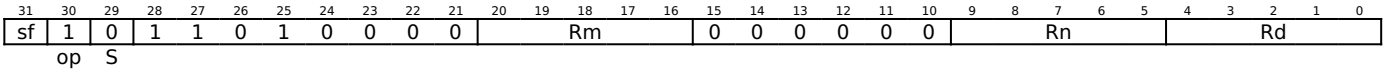
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SBC

Subtract with carry

This instruction subtracts a register value and the value of NOT (Carry flag) from a register value, and writes the result to the destination register. This instruction is used by the alias NGC.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SBC <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SBC <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
NGC	`Rn == '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = NOT X{datasize}(m);

var result : bits(datasize);
(result, -) = AddWithCarry{datasize}(operand1, operand2, PSTATE.C);

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

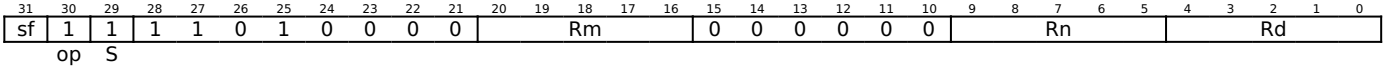
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SBCS

Subtract with carry, setting flags

This instruction subtracts a register value and the value of NOT (Carry flag) from a register value, and writes the result to the destination register. It updates the condition flags based on the result.

This instruction is used by the alias NGCS.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SBCS <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SBCS <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
NGCS	`Rn == '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = NOT X{datasize}(m);

var result : bits(datasize);
var nzcv : bits(4);

(result, nzcv) = AddWithCarry{datasize}(operand1, operand2, PSTATE.C);

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = nzcv;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

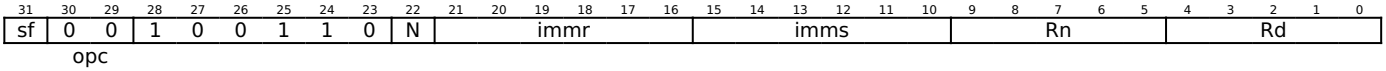
SBFIZ

Signed bitfield insert in zeros

This instruction copies a bitfield of <width> bits from the least significant bits of the source register to bit position <lsb> of the destination register, setting the destination bits below the bitfield to zero, and the bits above the bitfield to a copy of the most significant bit of the bitfield.

This is an alias of SBFM. This means:

- The encodings in this description are named to match the encodings of SBFM.
- The description of SBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
SBFIZ <Wd>, <Wn>, #<lsb>, #<width>
```

is equivalent to

```
SBFM <Wd>, <Wn>, #(-<lsb> MOD 32), #(<width>-1)
```

and is the preferred disassembly when `UInt(imms) < UInt(immr)`.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
SBFIZ <Xd>, <Xn>, #<lsb>, #<width>
```

is equivalent to

```
SBFM <Xd>, <Xn>, #(-<lsb> MOD 64), #(<width>-1)
```

and is the preferred disassembly when `UInt(imms) < UInt(immr)`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<lsb>	For the "32-bit" variant: is the bit number of the lsb of the destination bitfield, in the range 0 to 31. For the "64-bit" variant: is the bit number of the lsb of the destination bitfield, in the range 0 to 63.
<width>	For the "32-bit" variant: is the width of the bitfield, in the range 1 to 32-<lsb>. For the "64-bit" variant: is the width of the bitfield, in the range 1 to 64-<lsb>.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of SBFM gives the operational pseudocode for this instruction.

Operational Information

The description of SBFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SBFM

Signed bitfield move

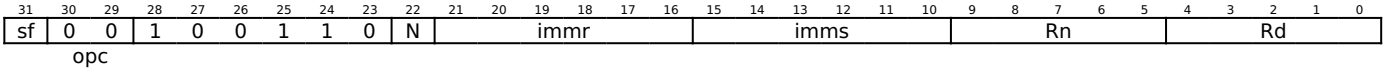
This instruction is usually accessed using one of its aliases, which are always preferred for disassembly.

If <imms> is greater than or equal to <immr>, this copies a bitfield of (<imms>-<immr>+1) bits starting from bit position <immr> in the source register to the least significant bits of the destination register.

If <imms> is less than <immr>, this copies a bitfield of (<imms>+1) bits from the least significant bits of the source register to bit position (regsize-<immr>) of the destination register, where regsize is the destination register size of 32 or 64 bits.

In both cases, the destination bits below the bitfield are set to zero, and the bits above the bitfield are set to a copy of the most significant bit of the bitfield.

This instruction is used by the aliases ASR (immediate), SBFIZ, SBFX, SXTB, SXTH, and SXTW.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
SBFM <Wd>, <Wn>, #<immr>, #<imms>
```

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
SBFM <Xd>, <Xn>, #<immr>, #<imms>
```

Decode for all variants of this encoding

```
if sf == '1' && N != '1' then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && (N != '0' || immr[5] != '0' || imms[5] != '0') then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let s : integer = UInt(imms);
let r : integer = UInt(immr);

var wmask : bits(datasize);
var tmask : bits(datasize);
(wmask, tmask) = DecodeBitMasks{datasize}(N, imms, immr, FALSE);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<immr>	For the "32-bit" variant: is the right rotate amount, in the range 0 to 31, encoded in the "immr" field. For the "64-bit" variant: is the right rotate amount, in the range 0 to 63, encoded in the "immr" field.
<imms>	For the "32-bit" variant: is the leftmost bit number to be moved from the source, in the range 0 to 31, encoded in the "imms" field. For the "64-bit" variant: is the leftmost bit number to be moved from the source, in the range 0 to 63, encoded in the "imms" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Alias Conditions

Alias	Of variant	Is preferred when
ASR (immediate)	32-bit	`imms == '011111'`
ASR (immediate)	64-bit	`imms == '111111'`
SBFIZ		`UInt(imms) < UInt(immr)`
SBFX		`BFXPreferred(sf, opc[1], imms, immr)`
SXTB		`immr == '000000' && imms == '000111'`
SXTH		`immr == '000000' && imms == '001111'`
SXTW		`immr == '000000' && imms == '011111'`

Operation

```
let src : bits(datasize) = X{}(n);

// Perform bitfield move on low bits
let bot : bits(datasize) = ROR(src, r) AND wmask;
let top : bits(datasize) = Replicate{}(src[s]);

// Combine extension bits and result bits
X{datasize}(d) = (top AND NOT tmask) OR (bot AND tmask);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

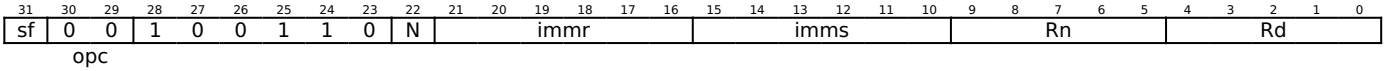
SBFX

Signed bitfield extract

This instruction copies a bitfield of <width> bits starting from bit position <lsb> in the source register to the least significant bits of the destination register, and sets destination bits above the bitfield to a copy of the most significant bit of the bitfield.

This is an alias of SBFM. This means:

- The encodings in this description are named to match the encodings of SBFM.
- The description of SBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
SBFX <Wd>, <Wn>, #<lsb>, #<width>
```

is equivalent to

```
SBFM <Wd>, <Wn>, #<lsb>, #(<lsb>+<width>-1)
```

and is the preferred disassembly when `BFXPreferred(sf, opc[1], imms, immr)`.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
SBFX <Xd>, <Xn>, #<lsb>, #<width>
```

is equivalent to

```
SBFM <Xd>, <Xn>, #<lsb>, #(<lsb>+<width>-1)
```

and is the preferred disassembly when `BFXPreferred(sf, opc[1], imms, immr)`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<lsb>	For the "32-bit" variant: is the bit number of the lsb of the source bitfield, in the range 0 to 31. For the "64-bit" variant: is the bit number of the lsb of the source bitfield, in the range 0 to 63.
<width>	For the "32-bit" variant: is the width of the bitfield, in the range 1 to 32-<lsb>. For the "64-bit" variant: is the width of the bitfield, in the range 1 to 64-<lsb>.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of SBFM gives the operational pseudocode for this instruction.

Operational Information

The description of SBFM gives the operational information for this instruction.

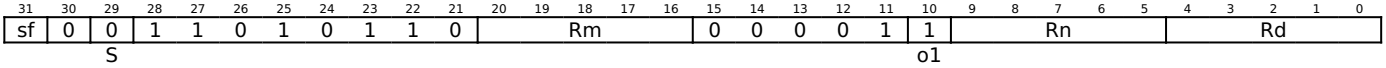
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDIV (quotient)

Signed divide

This instruction divides the first signed source register value by the second signed source register value, and writes the result to the destination register. Dividing by zero writes the value zero to the destination register. The condition flags are not affected.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SDIV <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SDIV <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);
let dividend : integer = SInt(operand1);
let divisor : integer = SInt(operand2);

var result : integer;
if divisor == 0 then
    result = 0;
elsif dividend < 0 == divisor < 0 then
    result = Abs(dividend) DIVRM Abs(divisor);    // same signs - positive result
else
    result = -(Abs(dividend) DIVRM Abs(divisor)); // different signs - negative result
end;

X{datasize}(d) = result[datasize-1:0];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

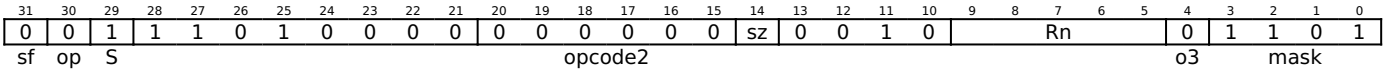
SETF8, SETF16

Evaluation of 8-bit or 16-bit flag values

This instruction sets the PSTATE.NZV flags based on the value in the specified general-purpose register. `SETF8` treats the value as an 8-bit value. `SETF16` treats the value as a 16-bit value.
The PSTATE.C flag is not affected by these instructions.

Integer

(FEAT_FlagM)



Encoding for the SETF8 variant

Applies when `(sz == 0)`

```
SETF8 <Wn>
```

Encoding for the SETF16 variant

Applies when `(sz == 1)`

```
SETF16 <Wn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FlagM) then EndOfDecode(Decode_UNDEF); end;
let size : integer{} = 8 << UInt(sz);
let n : integer = UInt(Rn);
```

Assembler Symbols

<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let reg : bits(32) = X{32}(n);
PSTATE.N = reg[size-1];
PSTATE.Z = if reg[size-1:0] == Zeros{size} then '1' else '0';
PSTATE.V = reg[size] XOR reg[size-1];
//PSTATE.C unchanged;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SETGP, SETGM, SETGE

Memory set with tag setting

These instructions set a requested number of bytes in memory to the value in the least significant byte of the source data register and store an Allocation Tag to memory for each Tag Granule written. The Allocation Tag is calculated from the Logical Address Tag in the register that holds the first address to be set. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETGP, then SETGM, and then SETGE.

SETGP performs some preconditioning of the arguments suitable for using the SETGM instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETGM sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETGE sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see *Memory Copy and Memory Set exceptions*.

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETGP:

- If Xn[63] == '1', the set size is saturated to 0x7FFFFFFFFFFFFF0.

On completion of SETGP, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of SETGP, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For SETGM, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETGM, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be set.
  - Xd holds the lowest address that has not been set.

For SETGE, option A, when PSTATE.C = '0':

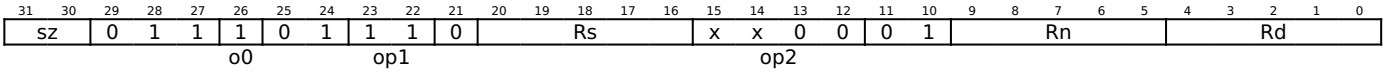
- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds 0.

For SETGE, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds 0.
  - Xd holds the lowest address that has not been set.

Integer

(FEAT_MOPS && FEAT_MTE)



Encoding for the Prologue variant

Applies when `(op2 == 0000)`

```
SETGP [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when `(op2 == 0100)`

```
SETGM [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when `(op2 == 1000)`

```
SETGE [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if (!IsFeatureImplemented(FEAT_MOPS) ||
      !IsFeatureImplemented(FEAT_MTE) || sz != '00') then
    EndOfDecode(Decode_UNDEF);
end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	For the "Main" and "Prologue" variants: is the 64-bit name of the general-purpose register that holds the source data in bits<7:0>, encoded in the "Rs" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.

Operation

```
CheckMOPSEnabled();

memset.is_setg  = TRUE;

CheckSETConstrainedUnpredictable(memset);

let data : bits(8) = X{8}(memset.s);
var B : MOPSBlockSize = 0;

memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETGOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                                                 else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescSTGMOPS(privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSSETGSize then
        memset.setsize = ArchMaxMOPSSETGSize;
    end;

    if ((memset.setsize != 0 && !IsAlignedSize(memset.toaddress, TAG_GRANULE)) ||
            !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, memset.toaddress);
        AArch64_Abort(fault);
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize   = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);

    var fault_address : bits(64);
    if memset.implements_option_a then
        fault_address = memset.toaddress + memset.setsize;
    else
        fault_address = memset.toaddress;
    end;

    if (memset.setsize != 0 && (memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
          !IsAlignedSize(memset.toaddress, TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
    if ((memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
           !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
end;

var tags_set : integer;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var tagmemaddrdesc : AddressDescriptor;
var tagmemstatus : PhysMemRetStatus;
var fault : boolean    = FALSE;
var tagfault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault && !tagfault looplimit ArchMaxMOPSSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= -1 * memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
                                                  data, B, accdesc);
        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress + memset.setsize);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress + memset.setsize,
                                                              tag, B, accdesc);
        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);
```

```
        if !fault && !tagfault then
            memset.setsize      = memset.setsize      + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;

else
    while memset.stagesetsize > 0 && !fault && !tagfault looplimit ArchMaxMOPSSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);
        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress, tag,
                                                              B, accdesc);

        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

        if !fault && !tagfault then
            memset.toaddress    = memset.toaddress    + B;
            memset.setsize      = memset.setsize      - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault || tagfault, tags_set * TAG_GRANULE);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
elsif tagfault then
    if IsFault(tagmemaddrdesc) then
        AArch64_Abort(tagmemaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(tagmemstatus, iswrite, tagmemaddrdesc, 1, accdesc);
    end;
end;

if memset.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SETGPN, SETGMN, SETGEN

Memory set with tag setting, non-temporal

These instructions set a requested number of bytes in memory to the value in the least significant byte of the source data register and store an Allocation Tag to memory for each Tag Granule written. The Allocation Tag is calculated from the Logical Address Tag in the register that holds the first address to be set. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETGPN, then SETGMN, and then SETGEN.

SETGPN performs some preconditioning of the arguments suitable for using the SETGMN instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETGMN sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETGEN sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see *Memory Copy and Memory Set exceptions*.

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETGPN:

- If Xn[63] == '1', the set size is saturated to 0x7FFFFFFFFFFFFF0.

On completion of SETGPN, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of SETGPN, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For SETGMN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETGMN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be set.
  - Xd holds the lowest address that has not been set.

For SETGEN, option A, when PSTATE.C = '0':

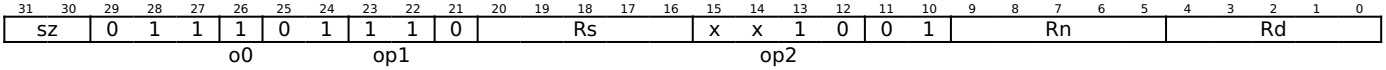
- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds 0.

For SETGEN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds 0.
  - Xd holds the lowest address that has not been set.

Integer

(FEAT_MOPS && FEAT_MTE)



Encoding for the Prologue variant

Applies when (op2 == 0010)

```
SETGPN [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when (op2 == 0110)

```
SETGMN [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when (op2 == 1010)

```
SETGEN [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if (!IsFeatureImplemented(FEAT_MOPS) ||
      !IsFeatureImplemented(FEAT_MTE) || sz != '00') then
    EndOfDecode(Decode_UNDEF);
end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	For the "Main" and "Prologue" variants: is the 64-bit name of the general-purpose register that holds the source data in bits<7:0>, encoded in the "Rs" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.

Operation

```
CheckMOPSEnabled();

memset.is_setg  = TRUE;

CheckSETConstrainedUnpredictable(memset);

let data : bits(8) = X{8}(memset.s);
var B : MOPSBlockSize = 0;

memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETGOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                                                 else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescSTGMOPS(privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSSETGSize then
        memset.setsize = ArchMaxMOPSSETGSize;
    end;

    if ((memset.setsize != 0 && !IsAlignedSize(memset.toaddress, TAG_GRANULE)) ||
            !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, memset.toaddress);
        AArch64_Abort(fault);
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize   = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);

    var fault_address : bits(64);
    if memset.implements_option_a then
        fault_address = memset.toaddress + memset.setsize;
    else
        fault_address = memset.toaddress;
    end;

    if (memset.setsize != 0 && (memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
          !IsAlignedSize(memset.toaddress, TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
    if ((memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
           !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
end;

var tags_set : integer;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var tagmemaddrdesc : AddressDescriptor;
var tagmemstatus : PhysMemRetStatus;
var fault : boolean    = FALSE;
var tagfault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault && !tagfault looplimit ArchMaxMOPSSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= -1 * memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
                                                  data, B, accdesc);
        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress + memset.setsize);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress + memset.setsize,
                                                              tag, B, accdesc);
        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);
```

```
        if !fault && !tagfault then
            memset.setsize      = memset.setsize      + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;

else
    while memset.stagesetsize > 0 && !fault && !tagfault looplimit ArchMaxMOPSSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);
        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress, tag,
                                                              B, accdesc);

        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

        if !fault && !tagfault then
            memset.toaddress    = memset.toaddress    + B;
            memset.setsize      = memset.setsize      - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault || tagfault, tags_set * TAG_GRANULE);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
elsif tagfault then
    if IsFault(tagmemaddrdesc) then
        AArch64_Abort(tagmemaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(tagmemstatus, iswrite, tagmemaddrdesc, 1, accdesc);
    end;
end;

if memset.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SETGPT, SETGMT, SETGET

Memory set with tag setting, unprivileged

These instructions set a requested number of bytes in memory to the value in the least significant byte of the source data register and store an Allocation Tag to memory for each Tag Granule written. The Allocation Tag is calculated from the Logical Address Tag in the register that holds the first address to be set. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETGPT, then SETGMT, and then SETGET.

SETGPT performs some preconditioning of the arguments suitable for using the SETGMT instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETGMT sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETGET sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see *Memory Copy and Memory Set exceptions*.

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETGPT:

- If Xn[63] == '1', the set size is saturated to 0x7FFFFFFFFFFFFF0.

On completion of SETGPT, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of SETGPT, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For SETGMT, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETGMT, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be set.
  - Xd holds the lowest address that has not been set.

For SETGET, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds 0.

For SETGET, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds 0.
  - Xd holds the lowest address that has not been set.

Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

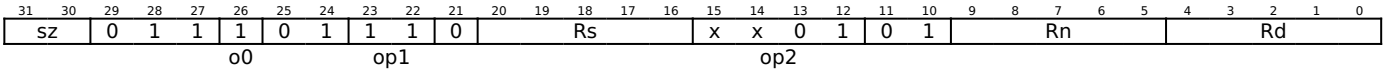
- The instruction is executed at EL1.

- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS && FEAT_MTE)



Encoding for the Prologue variant

Applies when (op2 == 0001)

```
SETGPT [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when (op2 == 0101)

```
SETGMT [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when (op2 == 1001)

```
SETGET [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if (!IsFeatureImplemented(FEAT_MOPS) ||
      !IsFeatureImplemented(FEAT_MTE) || sz != '00') then
    EndOfDecode(Decode_UNDEF);
end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	For the "Main" and "Prologue" variants: is the 64-bit name of the general-purpose register that holds the source data in bits<7:0>, encoded in the "Rs" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.

Operation

```
CheckMOPSEnabled();

memset.is_setg  = TRUE;

CheckSETConstrainedUnpredictable(memset);

let data : bits(8) = X{8}(memset.s);
var B : MOPSBlockSize = 0;

memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETGOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                                                 else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescSTGMOPS(privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSSETGSize then
        memset.setsize = ArchMaxMOPSSETGSize;
    end;

    if ((memset.setsize != 0 && !IsAlignedSize(memset.toaddress, TAG_GRANULE)) ||
            !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, memset.toaddress);
        AArch64_Abort(fault);
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize   = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);

    var fault_address : bits(64);
    if memset.implements_option_a then
        fault_address = memset.toaddress + memset.setsize;
    else
        fault_address = memset.toaddress;
    end;

    if (memset.setsize != 0 && (memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
          !IsAlignedSize(memset.toaddress, TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
    if ((memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
           !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
end;

var tags_set : integer;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var tagmemaddrdesc : AddressDescriptor;
var tagmemstatus : PhysMemRetStatus;
var fault : boolean    = FALSE;
var tagfault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault && !tagfault looplimit ArchMaxMOPSSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= -1 * memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
                                                  data, B, accdesc);
        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress + memset.setsize);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress + memset.setsize,
                                                              tag, B, accdesc);
        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);
```

```
        if !fault && !tagfault then
            memset.setsize      = memset.setsize      + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;

else
    while memset.stagesetsize > 0 && !fault && !tagfault looplimit ArchMaxMOPSSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);
        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress, tag,
                                                              B, accdesc);

        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

        if !fault && !tagfault then
            memset.toaddress    = memset.toaddress    + B;
            memset.setsize      = memset.setsize      - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault || tagfault, tags_set * TAG_GRANULE);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
elsif tagfault then
    if IsFault(tagmemaddrdesc) then
        AArch64_Abort(tagmemaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(tagmemstatus, iswrite, tagmemaddrdesc, 1, accdesc);
    end;
end;

if memset.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SETGPTN, SETGMTN, SETGETN

Memory set with tag setting, unprivileged and non-temporal

These instructions set a requested number of bytes in memory to the value in the least significant byte of the source data register and store an Allocation Tag to memory for each Tag Granule written. The Allocation Tag is calculated from the Logical Address Tag in the register that holds the first address to be set. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETGPTN, then SETGMTN, and then SETGETN.

SETGPTN performs some preconditioning of the arguments suitable for using the SETGMTN instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETGMTN sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETGETN sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see *Memory Copy and Memory Set exceptions*.

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETGPTN:

- If Xn[63] == '1', the set size is saturated to 0x7FFFFFFFFFFFFF0.

On completion of SETGPTN, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of SETGPTN, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For SETGMTN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETGMTN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be set.
  - Xd holds the lowest address that has not been set.

For SETGETN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds 0.

For SETGETN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds 0.
  - Xd holds the lowest address that has not been set.

Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

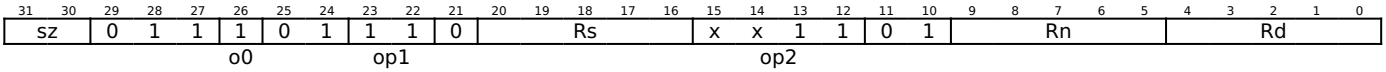
- The instruction is executed at EL1.

- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS && FEAT_MTE)



Encoding for the Prologue variant

Applies when `(op2 == 0011)`

```
SETGPTN [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when `(op2 == 0111)`

```
SETGMTN [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when `(op2 == 1011)`

```
SETGETN [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if (!IsFeatureImplemented(FEAT_MOPS) ||
      !IsFeatureImplemented(FEAT_MTE) || sz != '00') then
    EndOfDecode(Decode_UNDEF);
end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address (an integer multiple of 16) and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set (an integer multiple of 16) and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	For the "Main" and "Prologue" variants: is the 64-bit name of the general-purpose register that holds the source data in bits<7:0>, encoded in the "Rs" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.

Operation

```
CheckMOPSEnabled();

memset.is_setg  = TRUE;

CheckSETConstrainedUnpredictable(memset);

let data : bits(8) = X{8}(memset.s);
var B : MOPSBlockSize = 0;

memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETGOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                                                 else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescSTGMOPS(privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSSETGSize then
        memset.setsize = ArchMaxMOPSSETGSize;
    end;

    if ((memset.setsize != 0 && !IsAlignedSize(memset.toaddress, TAG_GRANULE)) ||
            !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, memset.toaddress);
        AArch64_Abort(fault);
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize   = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);

    var fault_address : bits(64);
    if memset.implements_option_a then
        fault_address = memset.toaddress + memset.setsize;
    else
        fault_address = memset.toaddress;
    end;

    if (memset.setsize != 0 && (memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
          !IsAlignedSize(memset.toaddress, TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
    if ((memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
           !IsAlignedSize(memset.setsize[63:0], TAG_GRANULE)) then
        let fault : FaultRecord = AlignmentFault(accdesc, fault_address);
        AArch64_Abort(fault);
    end;
end;

var tags_set : integer;
var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var tagmemaddrdesc : AddressDescriptor;
var tagmemstatus : PhysMemRetStatus;
var fault : boolean    = FALSE;
var tagfault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault && !tagfault looplimit ArchMaxMOPSSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= -1 * memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
                                                  data, B, accdesc);
        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress + memset.setsize);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress + memset.setsize,
                                                              tag, B, accdesc);
        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);
```

```
        if !fault && !tagfault then
            memset.setsize      = memset.setsize      + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;

else
    while memset.stagesetsize > 0 && !fault && !tagfault looplimit ArchMaxMOPSSETGSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, TAG_GRANULE);
        assert B <= memset.stagesetsize && B[3:0] == '0000';

        (-, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);
        let tag : bits(4) = AArch64_AllocationTagFromAddress(memset.toaddress);
        (tags_set, tagmemaddrdesc, tagmemstatus) = MemSetTags(memset.toaddress, tag,
                                                              B, accdesc);

        fault    = IsFault(memaddrdesc)    || IsFault(memstatus);
        tagfault = IsFault(tagmemaddrdesc) || IsFault(tagmemstatus);

        if !fault && !tagfault then
            memset.toaddress    = memset.toaddress    + B;
            memset.setsize      = memset.setsize      - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault || tagfault, tags_set * TAG_GRANULE);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
elsif tagfault then
    if IsFault(tagmemaddrdesc) then
        AArch64_Abort(tagmemaddrdesc.fault);
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(tagmemstatus, iswrite, tagmemaddrdesc, 1, accdesc);
    end;
end;

if memset.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SETP, SETM, SETE

Memory set

These instructions set a required number of bytes in memory to the value in the least significant byte of the source data register. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETP, then SETM, and then SETE.

SETP performs some preconditioning of the arguments suitable for using the SETM instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETM sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETE sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see *Memory Copy and Memory Set exceptions*.

The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETP:

- If Xn[63] == '1', the set size is saturated to 0x7FFFFFFFFFFFFFFF.

On completion of SETP, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of SETP, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For SETM, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETM, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be set.
  - Xd holds the lowest address that has not been set.

For SETE, option A, when PSTATE.C = '0':

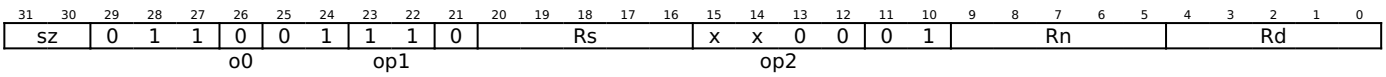
- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds 0.

For SETE, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds 0.
  - Xd holds the lowest address that has not been set.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op2 == 0000)`

```
SETP [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when `(op2 == 0100)`

```
SETM [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when `(op2 == 1000)`

```
SETE [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.

Operation

```
CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset);

let data : bits(8) = X{}(memset.s);
var B : MOPSBlockSize = 0;

memset.is_setg = FALSE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                            else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSBlockSize then
        memset.setsize = ArchMaxMOPSBlockSize;
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize   = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);
end;

var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var memory_set : integer;
var fault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= -1 * memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
                                                           data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.setsize      = memset.setsize      + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;

else
    while memset.stagesetsize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.toaddress    = memset.toaddress    + B;
            memset.setsize      = memset.setsize      - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault, memory_set);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
```

```
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memset.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SETPN, SETMN, SETEN

Memory set, non-temporal

These instructions set a required number of bytes in memory to the value in the least significant byte of the source data register. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETPN, then SETMN, and then SETEN.
SETPN performs some preconditioning of the arguments suitable for using the SETMN instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETMN sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETEN sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETPN:

- If Xn[63] == '1', the set size is saturated to 0x7FFFFFFFFFFFFFFF.

On completion of SETPN, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of SETPN, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For SETMN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETMN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be set.
  - Xd holds the lowest address that has not been set.

For SETEN, option A, when PSTATE.C = '0':

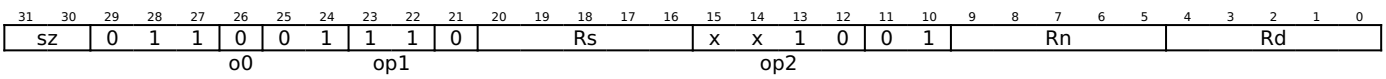
- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds 0.

For SETEN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds 0.
  - Xd holds the lowest address that has not been set.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when (op2 == 0010)

```
SETPN [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when (op2 == 0110)

```
SETMN [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when (op2 == 1010)

```
SETEN [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.

Operation

```
CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset);

let data : bits(8) = X{}(memset.s);
var B : MOPSBlockSize = 0;

memset.is_setg = FALSE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                            else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSBlockSize then
        memset.setsize = ArchMaxMOPSBlockSize;
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize   = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);
end;

var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var memory_set : integer;
var fault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= -1 * memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
                                                           data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.setsize      = memset.setsize      + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;

else
    while memset.stagesetsize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.toaddress    = memset.toaddress    + B;
            memset.setsize      = memset.setsize      - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault, memory_set);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
```

```
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memset.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SETPT, SETMT, SETET

Memory set, unprivileged

These instructions set a required number of bytes in memory to the value in the least significant byte of the source data register. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETPT, then SETMT, and then SETET.
SETPT performs some preconditioning of the arguments suitable for using the SETMT instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETMT sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETET sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETPT:

- If Xn[63] == '1', the set size is saturated to 0x7FFFFFFFFFFFFFFF.

On completion of SETPT, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of SETPT, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For SETMT, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETMT, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be set.
  - Xd holds the lowest address that has not been set.

For SETET, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds 0.

For SETET, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds 0.
  - Xd holds the lowest address that has not been set.

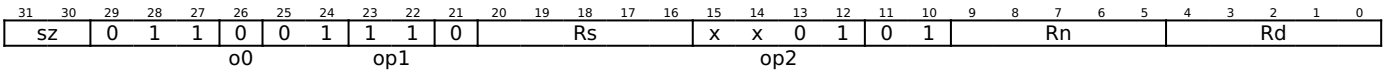
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when (op2 == 0001)

```
SETPT [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when (op2 == 0101)

```
SETMT [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when (op2 == 1001)

```
SETET [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.

Operation

```
CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset);

let data : bits(8) = X{}(memset.s);
var B : MOPSBlockSize = 0;

memset.is_setg = FALSE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                            else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSBlockSize then
        memset.setsize = ArchMaxMOPSBlockSize;
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize   = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);
end;

var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var memory_set : integer;
var fault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= -1 * memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
                                                           data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.setsize      = memset.setsize      + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;

else
    while memset.stagesetsize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.toaddress    = memset.toaddress    + B;
            memset.setsize      = memset.setsize      - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault, memory_set);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
```

```
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memset.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SETPTN, SETMTN, SETETN

Memory set, unprivileged and non-temporal

These instructions set a required number of bytes in memory to the value in the least significant byte of the source data register. The prologue, main, and epilogue instructions are expected to be run in succession and to appear consecutively in memory: SETPTN, then SETMTN, and then SETETN.
SETPTN performs some preconditioning of the arguments suitable for using the SETMTN instruction, and sets an IMPLEMENTATION DEFINED portion of the requested number of bytes. SETMTN sets a further IMPLEMENTATION DEFINED portion of the remaining bytes. SETETN sets any final remaining bytes.

Note

The ability to set an IMPLEMENTATION DEFINED number of bytes allows an implementation to optimize how the bytes being set are divided between the different instructions.

For more information on exceptions specific to memory set instructions, see *Memory Copy and Memory Set exceptions*.
The architecture supports two algorithms for the memory set: option A and option B. Which algorithm is used is IMPLEMENTATION DEFINED.

Note

Portable software should not assume that the choice of algorithm is constant.

For SETPTN:

- If Xn[63] == '1', the set size is saturated to 0x7FFFFFFFFFFFFFFF.

On completion of SETPTN, option A:

- Xn holds -1 times the number of bytes in the saturated set size remaining to be set.
- Xd holds the original Xd + saturated set size.
- PSTATE.{N,Z,C,V} are set to {0,0,0,0}.

On completion of SETPTN, option B:

- Xn holds the number of bytes in the saturated set size remaining to be set.
- Xd holds the lowest address that has not been set.
- PSTATE.{N,Z,C,V} are set to {0,0,1,0}.

For SETMTN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds -1 times the number of bytes remaining to be set.

For SETMTN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds the number of bytes remaining to be set.
  - Xd holds the lowest address that has not been set.

For SETETN, option A, when PSTATE.C = '0':

- Xn holds a signed 64-bit integer.
- Xn holds -1 times the number of bytes remaining to be set.
- Xd holds the lowest address to be set - Xn.
- On completion of the instruction, Xn holds 0.

For SETETN, option B, when PSTATE.C = '1':

- Xn holds the number of bytes remaining to be set.
- Xd holds the lowest address to be set.
- On completion of the instruction:
  - Xn holds 0.
  - Xd holds the lowest address that has not been set.

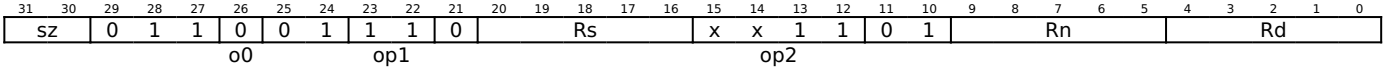
Explicit Memory Write effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory Write effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_MOPS)



Encoding for the Prologue variant

Applies when `(op2 == 0011)`

```
SETPTN [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Main variant

Applies when `(op2 == 0111)`

```
SETMTN [<Xd>]!, <Xn>!, <Xs>
```

Encoding for the Epilogue variant

Applies when `(op2 == 1011)`

```
SETETN [<Xd>]!, <Xn>!, <Xs>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_MOPS) || sz != '00' then EndOfDecode(Decode_UNDEF); end;

var memset : SETParams;
memset.d = UInt(Rd);
memset.s = UInt(Rs);
memset.n = UInt(Rn);
let options : bits(2) = op2[1:0];
let nontemporal : boolean = options[1] == '1';

case op2[3:2] of
    when '00' => memset.stage = MOPSStage_Prologue;
    when '01' => memset.stage = MOPSStage_Main;
    when '10' => memset.stage = MOPSStage_Epilogue;
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *Memory Copy and Memory Set SET** and *Crossing a page boundary with different memory types or Shareability attributes*.

Assembler Symbols

<Xd>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the destination address and is updated by the instruction, encoded in the "Rd" field. For the "Epilogue" and "Main" variants: is the 64-bit name of the general-purpose register that holds an encoding of the destination address and for option B is updated by the instruction, encoded in the "Rd" field.
<Xn>	For the "Prologue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Main" variant: is the 64-bit name of the general-purpose register that holds an encoding of the number of bytes to be set and is updated by the instruction, encoded in the "Rn" field. For the "Epilogue" variant: is the 64-bit name of the general-purpose register that holds the number of bytes to be set and is set to zero on completion of the instruction, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register that holds the source data, encoded in the "Rs" field.

Operation

```
CheckMOPSEnabled();

CheckSETConstrainedUnpredictable(memset);

let data : bits(8) = X{}(memset.s);
var B : MOPSBlockSize = 0;

memset.is_setg = FALSE;
memset.nzcv = PSTATE.[N,Z,C,V];
memset.toaddress = X{64}(memset.d);
if memset.stage == MOPSStage_Prologue then
    memset.setsize = UInt(X{64}(memset.n));
else
    memset.setsize = SInt(X{64}(memset.n));
end;
memset.implements_option_a = SETOptionA();

let privileged : boolean = (if options[0] == '1' then AArch64_IsUnprivAccessPriv()
                            else PSTATE.EL != EL0);

let accdesc : AccessDescriptor = CreateAccDescMOPS(MemOp_STORE, privileged, nontemporal);

if memset.stage == MOPSStage_Prologue then
    if memset.setsize > ArchMaxMOPSBlockSize then
        memset.setsize = ArchMaxMOPSBlockSize;
    end;

    if memset.implements_option_a then
        memset.nzcv = '0000';
        memset.toaddress = memset.toaddress + memset.setsize;
        memset.setsize   = 0 - memset.setsize;
    else
        memset.nzcv = '0010';
    end;
end;

memset.stagesetsize = MemSetStageSize(memset);

if memset.stage != MOPSStage_Prologue then
    CheckMemSetParams(memset, options);
end;

var memaddrdesc : AddressDescriptor;
var memstatus : PhysMemRetStatus;
var memory_set : integer;
var fault : boolean = FALSE;

if memset.implements_option_a then
    while memset.stagesetsize < 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= -1 * memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress + memset.setsize,
                                                           data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.setsize      = memset.setsize      + B;
            memset.stagesetsize = memset.stagesetsize + B;
        end;
    end;

else
    while memset.stagesetsize > 0 && !fault looplimit ArchMaxMOPSBlockSize do
        // IMP DEF selection of the block size that is worked on. While many
        // implementations might make this constant, that is not assumed.
        B = SETSizeChoice(memset, 1);
        assert B <= memset.stagesetsize;

        (memory_set, memaddrdesc, memstatus) = MemSetBytes(memset.toaddress, data, B, accdesc);

        if memory_set != B then
            fault = TRUE;
        else
            memset.toaddress    = memset.toaddress    + B;
            memset.setsize      = memset.setsize      - B;
            memset.stagesetsize = memset.stagesetsize - B;
        end;
    end;
end;

UpdateSetRegisters(memset, fault, memory_set);

if fault then
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
```

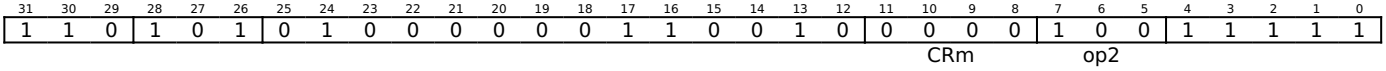
```
    else
        let iswrite : boolean = TRUE;
        HandleExternalAbort(memstatus, iswrite, memaddrdesc, B, accdesc);
    end;
end;

if memset.stage == MOPSStage_Prologue then
    PSTATE.[N,Z,C,V] = memset.nzcv;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SEV

Send event

This instruction is a hint instruction that causes an event to be signaled to all PEs in the multiprocessor system. For more information, see *Wait for Event mechanism and Send event*.



Encoding

SEV

Decode for this encoding

```
// Empty.
```

Operation

```
SendEvent();
```

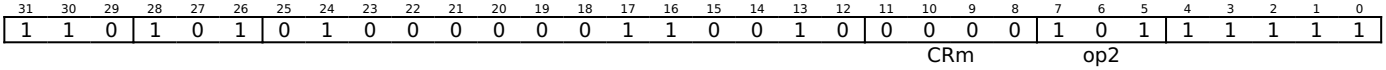
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SEVL

Send event local

This instruction is a hint instruction that causes an event to be signaled locally without requiring the event to be signaled to other PEs in the multiprocessor system. It can prime a wait-loop that starts with a `WFE` instruction.



Encoding

```
SEVL
```

Decode for this encoding

```
// Empty.
```

Operation

```
SendEventLocal();
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SHUH

Shared update hint

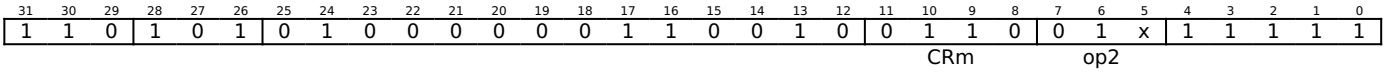
This instruction signals to the memory system that if the next instruction in program order generates an Explicit Memory Effect, then it is to a location that one or more other threads of execution are likely to subsequently update, and there is a performance benefit to ensuring subsequent updates by other threads of execution complete with minimal latency.

Note

This instruction might increase the latency of subsequent accesses to the location by the executing PE.

System

(FEAT_CMH)



Encoding

```
SHUH {PH}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CMH) then EndOfDecode(Decode_NOP); end;
let priority : boolean = op2[0] == '1';
```

Assembler Symbols

PH — Is a priority hint. If present, it additionally provides the same hint as the STCPH instruction, and is encoded in "op2[0]":

op2[0]	PH
0	[absent]
1	[present]

Operation

```
Hint_SharedUpdate(priority);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SLBND

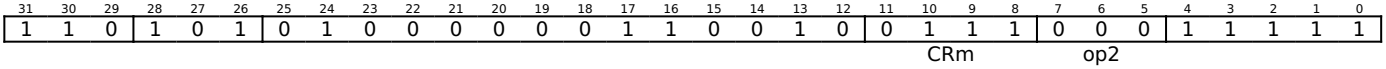
Scoped release boundary

This instruction signals to the memory system that a corresponding instruction with Release semantics or DMB instruction will subsequently occur in program order, and if that corresponding instruction defines a Barrier scope with the SLBND instruction, then these instructions do not create a Barrier-ordered-before relationship between any Effect in program order before the SLBND instruction and any of the following:

- The Effect with Release semantics.
- Any Effect in program order after the DMB Effect.

System

(FEAT_RLCS)



Encoding

SLBND

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RLCS) then EndOfDecode(Decode_NOP); end;
```

Operation

```
Hint_ScopedReleaseBoundary();
```

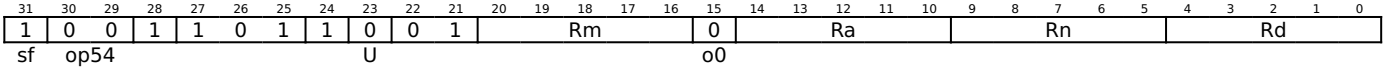
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMADDL

Signed multiply-add long

This instruction multiplies two 32-bit register values, adds a 64-bit register value, and writes the result to the 64-bit destination register.
This instruction is used by the alias SMULL (32-bit).



Encoding

```
SMADDL <Xd>, <Wn>, <Wm>, <Xa>
```

Decode for this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Xa>	Is the 64-bit name of the third general-purpose source register holding the addend, encoded in the "Ra" field.

Alias Conditions

Alias	Is preferred when
SMULL (32-bit)	`Ra == '11111'`

Operation

```
let operand1 : bits(32) = X{32}(n);
let operand2 : bits(32) = X{32}(m);
let operand3 : bits(64) = X{64}(a);

let result : integer = SInt(operand3) + SInt(operand1) * SInt(operand2);

X{64}(d) = result[63:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

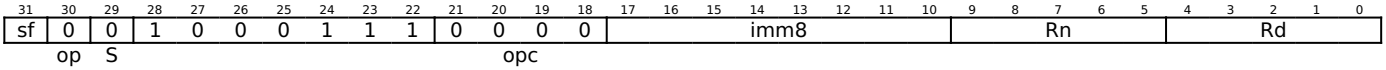
SMAX (immediate)

Signed maximum (immediate)

This instruction determines the signed maximum of the source register value and immediate, and writes the result to the destination register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SMAX <Wd>, <Wn>, #<simm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SMAX <Xd>, <Xn>, #<simm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let imm : integer = SInt(imm8);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<simm>	Is a signed immediate, in the range -128 to 127, encoded in the "imm8" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let result : integer = Max(SInt(operand1), imm);

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

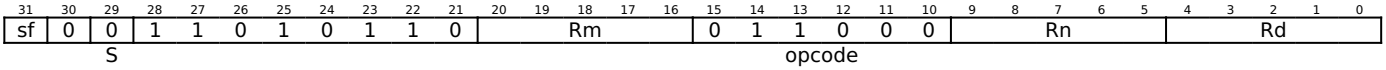
SMAX (register)

Signed maximum (register)

This instruction determines the signed maximum of the two source register values and writes the result to the destination register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SMAX <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SMAX <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);
let result : integer = Max(SInt(operand1), SInt(operand2));

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMC

Secure monitor call

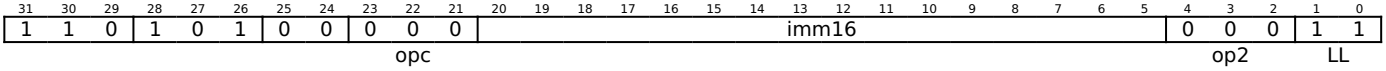
This instruction causes an exception to EL3.

SMC is available only for software executing at EL1 or higher. It is UNDEFINED at EL0.

If the values of *HCR_EL2*.TSC and SCR_EL3.SMD are both 0, execution of an SMC instruction at EL1 or higher generates a Secure Monitor Call exception, recording it in *ESR_ELx*, using the EC syndrome value 0x17, that is taken to EL3.

If the value of *HCR_EL2*.TSC is 1 and EL2 is enabled in the current Security state, execution of an SMC instruction at EL1 generates an exception that is taken to EL2, regardless of the value of SCR_EL3.SMD.

If the value of *HCR_EL2*.TSC is 0 and the value of SCR_EL3.SMD is 1, the SMC instruction is UNDEFINED.



Encoding

```
SMC #<imm>
```

Decode for this encoding

```
let imm : bits(16) = imm16;
```

Assembler Symbols

<imm>	Is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field.

Operation

```
if PSTATE.EL == EL0 then Undefined(); end;
AArch64_CheckForSMCUndefOrTrap(imm);
AArch64_CallSecureMonitor(imm);
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

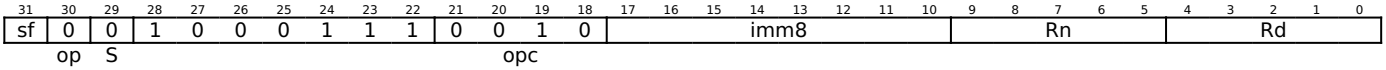
SMIN (immediate)

Signed minimum (immediate)

This instruction determines the signed minimum of the source register value and immediate, and writes the result to the destination register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SMIN <Wd>, <Wn>, #<simm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SMIN <Xd>, <Xn>, #<simm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let imm : integer = SInt(imm8);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<simm>	Is a signed immediate, in the range -128 to 127, encoded in the "imm8" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let result : integer = Min(SInt(operand1), imm);

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

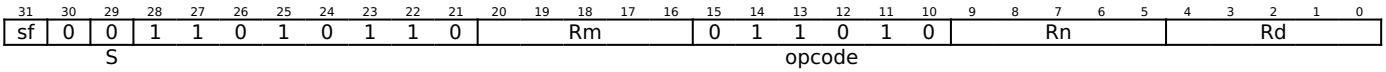
SMIN (register)

Signed minimum (register)

This instruction determines the signed minimum of the two source register values and writes the result to the destination register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SMIN <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SMIN <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);
let result : integer = Min(SInt(operand1), SInt(operand2));

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

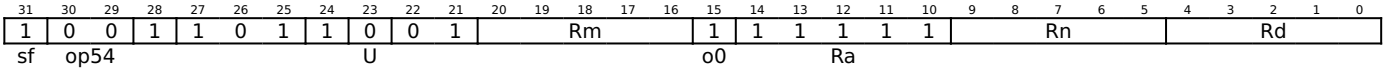
SMNEGL

Signed multiply-negate long

This instruction multiplies two 32-bit register values, negates the product, and writes the result to the 64-bit destination register.

This is an alias of SMSUBL. This means:

- The encodings in this description are named to match the encodings of SMSUBL.
- The description of SMSUBL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

```
SMNEGL <Xd>, <Wn>, <Wm>
```

is equivalent to

```
SMSUBL <Xd>, <Wn>, <Wm>, XZR
```

and is always the preferred disassembly.

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation

The description of SMSUBL gives the operational pseudocode for this instruction.

Operational Information

The description of SMSUBL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMSTART

Enable access to Streaming SVE mode and SME architectural state

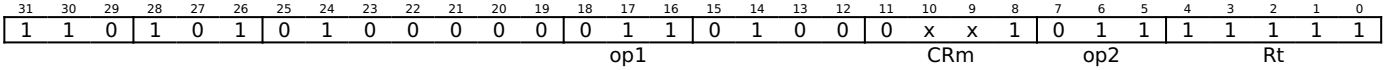
This instruction enables access to Streaming SVE mode and SME architectural state.
SMSTART enters Streaming SVE mode, and enables the SME ZA storage.
SMSTART SM enters Streaming SVE mode, but does not enable the SME ZA storage.
SMSTART ZA enables the SME ZA storage, but does not cause an entry to Streaming SVE mode.

This is an alias of MSR (immediate). This means:

- The encodings in this description are named to match the encodings of MSR (immediate).
- The description of MSR (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_SME)



Encoding

```
SMSTART {<option>}
```

is equivalent to

```
MSR <pstatefield>, #1
```

and is always the preferred disassembly.

Assembler Symbols

<option> Is an optional mode, encoded in "CRm[2:1]":

CRm[2:1]	<option>	Description
00	RESERVED	
01	SM	Maps to <pstatefield> SVCRSM.
10	ZA	Maps to <pstatefield> SVCRZA.
11	[absent]	Maps to <pstatefield> SVCRSMZA.

Operation

The description of MSR (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMSTOP

Disable access to Streaming SVE mode and SME architectural state

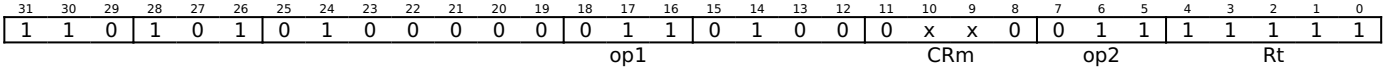
This instruction disables access to Streaming SVE mode and SME architectural state.
SMSTOP exits Streaming SVE mode, and disables the SME ZA storage.
SMSTOP SM exits Streaming SVE mode, but does not disable the SME ZA storage.
SMSTOP ZA disables the SME ZA storage, but does not cause an exit from Streaming SVE mode.

This is an alias of MSR (immediate). This means:

- The encodings in this description are named to match the encodings of MSR (immediate).
- The description of MSR (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_SME)



Encoding

```
SMSTOP {<option>}
```

is equivalent to

```
MSR <pstatefield>, #0
```

and is always the preferred disassembly.

Assembler Symbols

<option> Is an optional mode, encoded in "CRm[2:1]":

CRm[2:1]	<option>	Description
00	RESERVED	
01	SM	Maps to <pstatefield> SVCRSM.
10	ZA	Maps to <pstatefield> SVCRZA.
11	[absent]	Maps to <pstatefield> SVCRSMZA.

Operation

The description of MSR (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

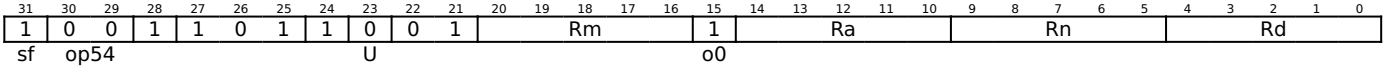
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMSUBL

Signed multiply-subtract long

This instruction multiplies two 32-bit register values, subtracts the product from a 64-bit register value, and writes the result to the 64-bit destination register.

This instruction is used by the alias SMNEGL.



Encoding

```
SMSUBL <Xd>, <Wn>, <Wm>, <Xa>
```

Decode for this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Xa>	Is the 64-bit name of the third general-purpose source register holding the minuend, encoded in the "Ra" field.

Alias Conditions

Alias	Is preferred when
SMNEGL	`Ra == '11111'`

Operation

```
let operand1 : bits(32) = X{32}(n);
let operand2 : bits(32) = X{32}(m);
let operand3 : bits(64) = X{64}(a);

let result : integer = SInt(operand3) - SInt(operand1) * SInt(operand2);

X{64}(d) = result[63:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

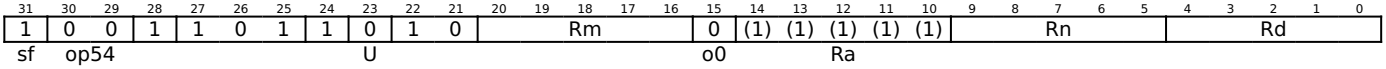
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMULH

Signed multiply high

This instruction multiplies two 64-bit register values, and writes bits[127:64] of the 128-bit result to the 64-bit destination register.



Encoding

```
SMULH <Xd>, <Xn>, <Xm>
```

Decode for this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation

```
let operand1 : bits(64) = X{64}(n);
let operand2 : bits(64) = X{64}(m);

let result : integer = SInt(operand1) * SInt(operand2);

X{64}(d) = result[127:64];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

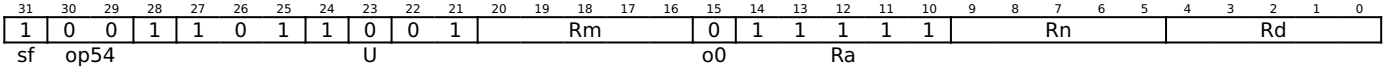
SMULL (32-bit)

Signed multiply long

This instruction multiplies two 32-bit register values, and writes the result to the 64-bit destination register.

This is an alias of SMADDL. This means:

- The encodings in this description are named to match the encodings of SMADDL.
- The description of SMADDL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

`SMULL <Xd>, <Wn>, <Wm>`

is equivalent to

`SMADDL <Xd>, <Wn>, <Wm>, XZR`

and is always the preferred disassembly.

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation

The description of SMADDL gives the operational pseudocode for this instruction.

Operational Information

The description of SMADDL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SRLS

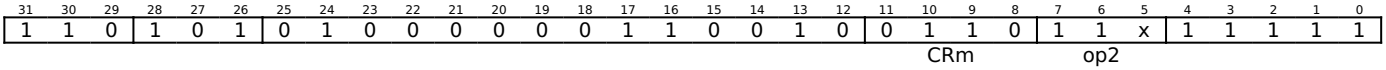
Scope release

This instruction signals to the memory system that if the next instruction in program order has Release semantics or is a DMB instruction that defines a Barrier scope with a preceding SLBND instruction, then these instructions do not create a Barrier-ordered-before relationship between any Effect in program order before the SLBND instruction and any of the following:

- The Effect with Release semantics.
- Any Effect in program order after the DMB Effect.

System

(FEAT_RLCS)



Encoding

```
SRLS {STSHSTRM}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RLCS) then EndOfDecode(Decode_NOP); end;
let stshstrm : boolean = op2[0] == '1';
```

Assembler Symbols

STSHSTRM — Is a store shared stream hint, and is encoded in "op2[0]". If present, it additionally provides the same hint as the STSHH STRM instruction.

Note

If FEAT_PCDPHINT is not implemented, the optional STSHSTRM hint may not have the performance benefits that are associated with the hint generated by the STSHH STRM instruction.

op2[0]	STSHSTRM
0	[absent]
1	[present]

Operation

```
Hint_ScopeRelease(stshstrm);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

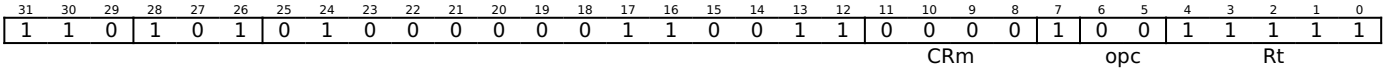
SSBB

Speculative store bypass barrier

This instruction is a memory barrier that prevents speculative loads from bypassing earlier stores to the same virtual address under certain conditions. For more information and details of the semantics, see *Speculative Store Bypass Barrier (SSBB)*.

This is an alias of DSB. This means:

- The encodings in this description are named to match the encodings of DSB.
- The description of DSB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

SSBB

is equivalent to

DSB #0

and is always the preferred disassembly.

Operation

The description of DSB gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2G

Store Allocation Tags

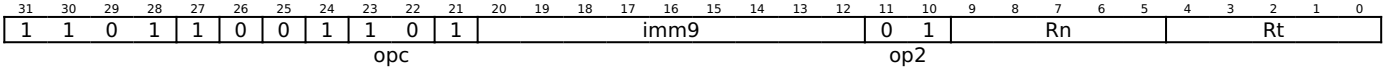
This instruction stores an Allocation Tag to two Tag Granules of memory. The address used for the store is calculated from the base register and an immediate signed offset scaled by the Tag Granule. The Allocation Tag is calculated from the Logical Address Tag in the source register.

This instruction generates an Unchecked access.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_MTE)



Encoding

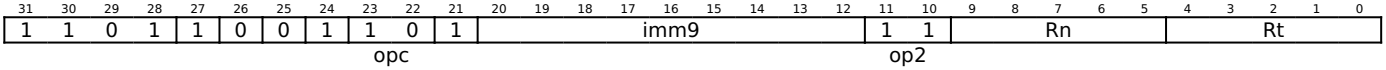
```
ST2G <Xt|SP>, [<Xn|SP>], #<simm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_MTE)



Encoding

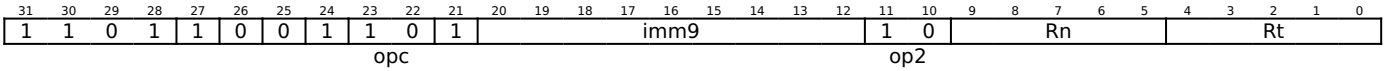
```
ST2G <Xt|SP>, [<Xn|SP>, #<simm>]!
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_MTE)



Encoding

```
ST2G <Xt|SP>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = FALSE;
let postindex : boolean = FALSE;
```

Assembler Symbols

<Xt|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate offset, a multiple of 16 in the range -4096 to 4080, defaulting to 0 and encoded in the "imm9" field.

Operation

```
var address : bits(64);
var address2 : bits(64);
let data : bits(64) = if t == 31 then SP{64}() else X{64}(t);
let tag : bits(4) = AArch64_AllocationTagFromAddress(data);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let stzgm : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_STORE, stzgm, t);

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

address2 = AddressIncrement(address, TAG_GRANULE, accdesc);

AArch64_MemTag(address, accdesc) = tag;
AArch64_MemTag(address2, accdesc) = tag;

if writeback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;

    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST64B

Single-copy atomic 64-byte store without status result

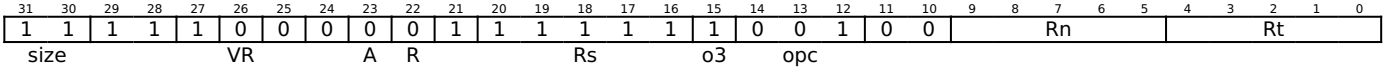
This instruction stores eight 64-bit doublewords from consecutive registers to a memory location. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt. The data is stored atomically and is required to be 64-byte aligned.

It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports `ST64B` also supports `LD64B`, `LDA64B`, and `STL64B`.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64)



Encoding

```
ST64B <Xt>, [<Xn|SP> {, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LS64) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = FALSE;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = FALSE;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *ST64B*.

Assembler Symbols

<Xt>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckLDST64BEnabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);

for i = 0 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

MemStore64B(address, data, accdesc);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST64BV

Single-copy atomic 64-byte store with status result

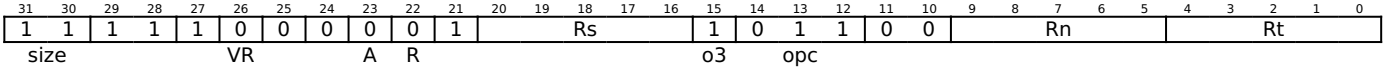
This instruction stores eight 64-bit doublewords from consecutive registers to a memory location, and writes the status result of the store to a register. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt. The data is stored atomically and is required to be 64-byte aligned.

It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports ST64BV also supports ST64BV0, STL64BV, and STL64BV0.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64_V)



Encoding

```
ST64BV <Xs>, <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LS64_V) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = TRUE;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = FALSE;
```

Assembler Symbols

<Xs> Is the 64-bit name of the general-purpose register into which the status result of this instruction is written, encoded in the "Rs" field. The value returned is:

0xFFFFFFFFFFFFFFFF
: If the memory location accessed does not support this instruction. In this case, the value at the memory location is UNKNOWN.

!= 0xFFFFFFFFFFFFFFFF
: If the memory location accessed does support this instruction. In this case, the peripheral that provides the response defines the returned value and provides information on the state of the memory update at the memory location.

If XZR is used, then the return value is ignored.

<Xt> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckST64BVEnabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);

for i = 0 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let status : bits(64) = MemStore64BWithRet(address, data, accdesc);

if s != 31 then X{64}(s) = status; end;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST64BV0

Single-copy atomic 64-byte EL0 store with status result

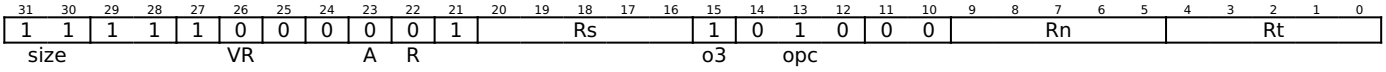
This instruction stores eight 64-bit doublewords from consecutive registers to a memory location, with the bottom 32 bits taken from ACCDATA_EL1, and writes the status result of the store to a register. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt[63:32]::ACCDATA_EL1()[31:0]. The data is stored atomically and is required to be 64-byte aligned.

It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports `ST64BV0` also supports `ST64BV`, `STL64BV`, and `STL64BV0`.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64_ACCDATA)



Encoding

```
ST64BV0 <Xs>, <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LS64_ACCDATA) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = TRUE;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = FALSE;
```

Assembler Symbols

<Xs> Is the 64-bit name of the general-purpose register into which the status result of this instruction is written, encoded in the "Rs" field. The value returned is:

0xFFFFFFFFFFFFFFFF
: If the memory location accessed does not support this instruction. In this case, the value at the memory location is UNKNOWN.

!= 0xFFFFFFFFFFFFFFFF
: If the memory location accessed does support this instruction. In this case, the peripheral that provides the response defines the returned value and provides information on the state of the memory update at the memory location.

If XZR is used, then the return value is ignored.

<Xt> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckST64BV0Enabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);

let Xt : bits(64) = X{}(t);
value[31:0] = ACCDATA_EL1()[31:0];
value[63:32] = Xt[63:32];
if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
data[63:0] = value;

for i = 1 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let status : bits(64) = MemStore64BWithRet(address, data, accdesc);

if s != 31 then X{64}(s) = status; end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STADD, STADDL

Atomic add on word or doubleword, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, adds the value held in a register to it, and stores the result back to memory.

- `STADD` does not have release semantics.
- `STADDL` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

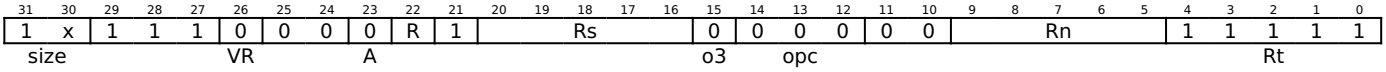
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDADD, LDADDA, LDADDAL, LDADDL. This means:

- The encodings in this description are named to match the encodings of LDADD, LDADDA, LDADDAL, LDADDL.
- The description of LDADD, LDADDA, LDADDAL, LDADDL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STADD <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDADD <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(size == 10 && R == 1)`

```
STADDL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDADDL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STADD <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDADD <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(size == 11 && R == 1)`

```
STADDL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDADDL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDADD, LDADDA, LDADDAL, LDADDL gives the operational pseudocode for this instruction.

Operational Information

The description of LDADD, LDADDA, LDADDAL, LDADDL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STADDB, STADDLB

Atomic add on byte, without return

This instruction atomically loads an 8-bit byte from memory, adds the value held in a register to it, and stores the result back to memory.

- `STADDB` does not have release semantics.
- `STADDLB` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

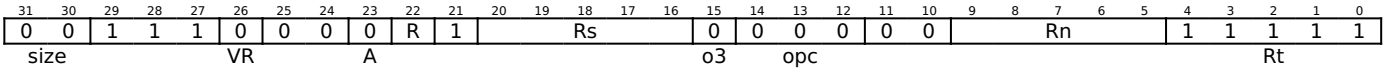
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDADDB, LDADDAB, LDADDALB, LDADDLB. This means:

- The encodings in this description are named to match the encodings of LDADDB, LDADDAB, LDADDALB, LDADDLB.
- The description of LDADDB, LDADDAB, LDADDALB, LDADDLB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STADDB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDADDB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STADDLB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDADDLB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDADDB, LDADDAB, LDADDALB, LDADDLB gives the operational pseudocode for this instruction.

Operational Information

The description of LDADDB, LDADDAB, LDADDALB, LDADDLB gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STADDH, STADDLH

Atomic add on halfword, without return

This instruction atomically loads a 16-bit halfword from memory, adds the value held in a register to it, and stores the result back to memory.

- `STADDH` does not have release semantics.
- `STADDLH` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

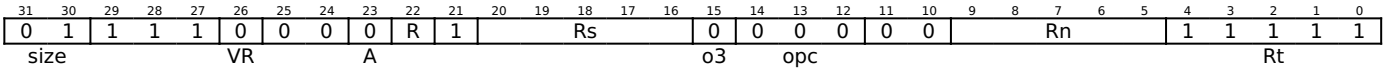
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDADDH, LDADDAH, LDADDALH, LDADDLH. This means:

- The encodings in this description are named to match the encodings of LDADDH, LDADDAH, LDADDALH, LDADDLH.
- The description of LDADDH, LDADDAH, LDADDALH, LDADDLH gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STADDH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDADDH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STADDLH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDADDLH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDADDH, LDADDAH, LDADDALH, LDADDLH gives the operational pseudocode for this instruction.

Operational Information

The description of LDADDH, LDADDAH, LDADDALH, LDADDLH gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STCLR, STCLRL

Atomic bit clear on word or doubleword, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, performs a bitwise AND with the complement of the value held in a register on it, and stores the result back to memory.

- `STCLR` does not have release semantics.
- `STCLRL` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

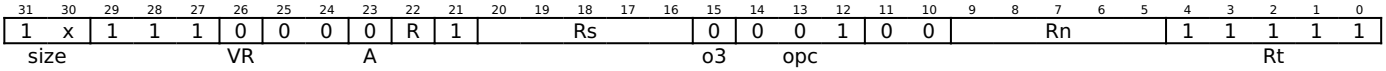
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDCLR, LDCLRA, LDCLRAL, LDCLRL. This means:

- The encodings in this description are named to match the encodings of LDCLR, LDCLRA, LDCLRAL, LDCLRL.
- The description of LDCLR, LDCLRA, LDCLRAL, LDCLRL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STCLR <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDCLR <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(size == 10 && R == 1)`

```
STCLRL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDCLRL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STCLR <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDCLR <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(size == 11 && R == 1)`

```
STCLRL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDCLRL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDCLR, LDCLRA, LDCLRAL, LDCLRL gives the operational pseudocode for this instruction.

Operational Information

The description of LDCLR, LDCLRA, LDCLRAL, LDCLRL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STCLRB, STCLRLB

Atomic bit clear on byte, without return

This instruction atomically loads an 8-bit byte from memory, performs a bitwise AND with the complement of the value held in a register on it, and stores the result back to memory.

- `STCLRB` does not have release semantics.
- `STCLRLB` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

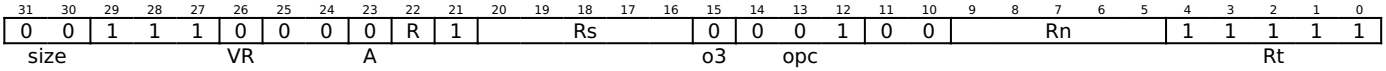
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB. This means:

- The encodings in this description are named to match the encodings of LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB.
- The description of LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STCLRB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDCLRB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STCLRLB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDCLRLB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB gives the operational pseudocode for this instruction.

Operational Information

The description of LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STCLRH, STCLRLH

Atomic bit clear on halfword, without return

This instruction atomically loads a 16-bit halfword from memory, performs a bitwise AND with the complement of the value held in a register on it, and stores the result back to memory.

- `STCLRH` does not have release semantics.
- `STCLRLH` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

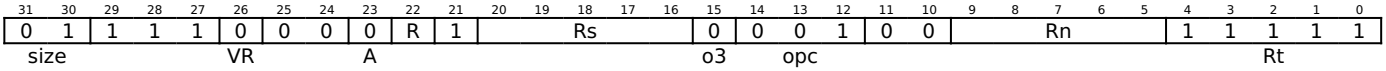
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH. This means:

- The encodings in this description are named to match the encodings of LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH.
- The description of LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STCLRH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDCLRH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STCLRLH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDCLRLH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH gives the operational pseudocode for this instruction.

Operational Information

The description of LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

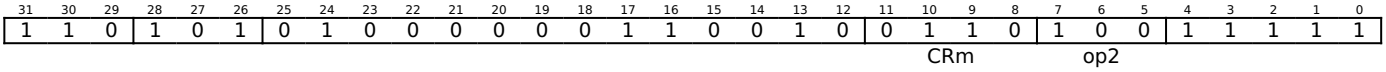
STCPH

Store concurrent priority hint

This instruction signals to the memory system that if the next instruction in program order generates an Explicit Memory Write Effect, then there is a performance benefit if that Explicit Memory Write Effect is sequenced before Memory Effects from other threads of execution in the coherence order to the same location. For atomic instructions, there is a performance benefit if the corresponding Explicit Memory Read Effect is sequenced before Memory Effects from other threads of execution in the coherence order to the same location. This includes cases in which any instruction that conditionally performs a write generates an Explicit Memory Read Effect before it is known whether the write is performed.

System

(FEAT_CMH)



Encoding

STCPH

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CMH) then EndOfDecode(Decode_NOP); end;
```

Operation

```
Hint_StoreConcurrentPriority();
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STEOR, STEORL

Atomic exclusive-OR on word or doubleword, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, performs an exclusive-OR with the value held in a register on it, and stores the result back to memory.

- `STEOR` does not have release semantics.
- `STEORL` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

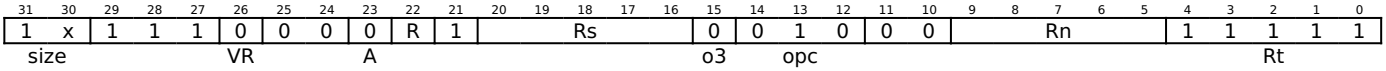
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDEOR, LDEORA, LDEORAL, LDEORL. This means:

- The encodings in this description are named to match the encodings of LDEOR, LDEORA, LDEORAL, LDEORL.
- The description of LDEOR, LDEORA, LDEORAL, LDEORL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STEOR <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDEOR <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(size == 10 && R == 1)`

```
STEORL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDEORL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STEOR <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDEOR <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(size == 11 && R == 1)`

```
STEORL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDEORL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDEOR, LDEORA, LDEORAL, LDEORL gives the operational pseudocode for this instruction.

Operational Information

The description of LDEOR, LDEORA, LDEORAL, LDEORL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STEORB, STEORLB

Atomic exclusive-OR on byte, without return

This instruction atomically loads an 8-bit byte from memory, performs an exclusive-OR with the value held in a register on it, and stores the result back to memory.

- `STEORB` does not have release semantics.
- `STEORLB` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

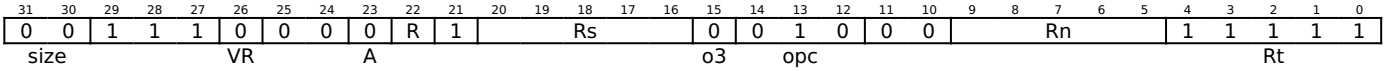
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDEORB, LDEORAB, LDEORALB, LDEORLB. This means:

- The encodings in this description are named to match the encodings of LDEORB, LDEORAB, LDEORALB, LDEORLB.
- The description of LDEORB, LDEORAB, LDEORALB, LDEORLB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STEORB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDEORB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STEORLB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDEORLB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDEORB, LDEORAB, LDEORALB, LDEORLB gives the operational pseudocode for this instruction.

Operational Information

The description of LDEORB, LDEORAB, LDEORALB, LDEORLB gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STEORH, STEORLH

Atomic exclusive-OR on halfword, without return

This instruction atomically loads a 16-bit halfword from memory, performs an exclusive-OR with the value held in a register on it, and stores the result back to memory.

- `STEORH` does not have release semantics.
- `STEORLH` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

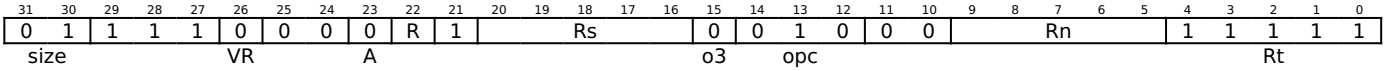
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDEORH, LDEORAH, LDEORALH, LDEORLH. This means:

- The encodings in this description are named to match the encodings of LDEORH, LDEORAH, LDEORALH, LDEORLH.
- The description of LDEORH, LDEORAH, LDEORALH, LDEORLH gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STEORH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDEORH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STEORLH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDEORLH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDEORH, LDEORAH, LDEORALH, LDEORLH gives the operational pseudocode for this instruction.

Operational Information

The description of LDEORH, LDEORAH, LDEORALH, LDEORLH gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STG

Store Allocation Tag

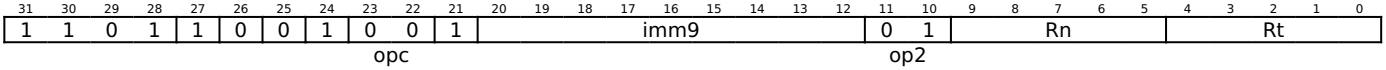
This instruction stores an Allocation Tag to memory. The address used for the store is calculated from the base register and an immediate signed offset scaled by the Tag Granule. The Allocation Tag is calculated from the Logical Address Tag in the source register.

This instruction generates an Unchecked access.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_MTE)



Encoding

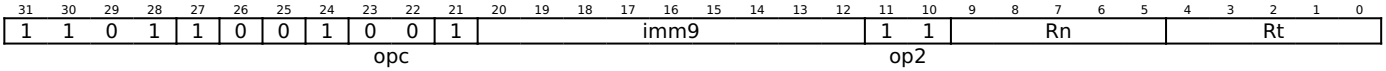
```
STG <Xt|SP>, [<Xn|SP>], #<simm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_MTE)



Encoding

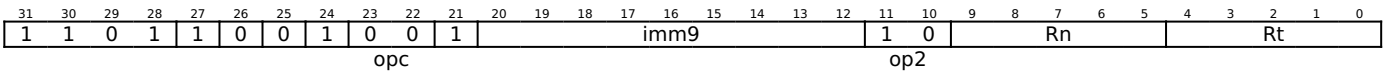
```
STG <Xt|SP>, [<Xn|SP>, #<simm>]!
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_MTE)



Encoding

```
STG <Xt|SP>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = FALSE;
let postindex : boolean = FALSE;
```

Assembler Symbols

<Xt|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate offset, a multiple of 16 in the range -4096 to 4080, defaulting to 0 and encoded in the "imm9" field.

Operation

```
var address : bits(64);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let stzgm : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_STORE, stzgm, t);

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(64) = if t == 31 then SP{64}() else X{64}(t);
let tag : bits(4) = AArch64_AllocationTagFromAddress(data);
AArch64_MemTag(address, accdesc) = tag;

if writeback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;

    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STGM

Store Allocation Tag multiple

This instruction writes a naturally aligned block of N Allocation Tags, where the size of N is identified in GMID_EL1.BS, and the Allocation Tag written to address A is taken from the source register at 4*A<7:4>+3:4*A<7:4>.

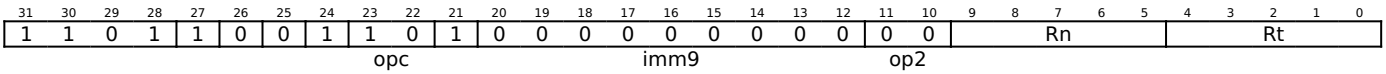
If the value of HCR_EL2.TGE is 0 and the Effective value of SCTLR2_EL1.GM0 is 0, this instruction is UNDEFINED at EL0.

If the value of HCR_EL2.TGE is 1 and the Effective value of SCTLR2_EL2.GM0 is 0, this instruction is UNDEFINED at EL0.

This instruction generates an Unchecked access.

Integer

(FEAT_MTE2)



Encoding

```
STGM <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !IsGMEnabledAtEL0() && PSTATE.EL == EL0 then Undefined(); end;

let data : bits(64) = X{64}(t);
var address : bits(64);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let size : integer{} = 4 * 2 ^ UInt(GMID_EL1().BS);
address = AlignDownSize(address, size);
let count : integer = size >> LOG2_TAG_GRANULE;
var index : integer = UInt(address[LOG2_TAG_GRANULE+3:LOG2_TAG_GRANULE]);
let stzgm : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_STORE, stzgm, t);

for i = 0 to count-1 do
    let tag : bits(4) = data[index*:4];
    AArch64_MemTag(address, accdesc) = tag;
    address = AddressIncrement(address, TAG_GRANULE, accdesc);
    index = index + 1;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STGP

Store Allocation Tag and pair of registers

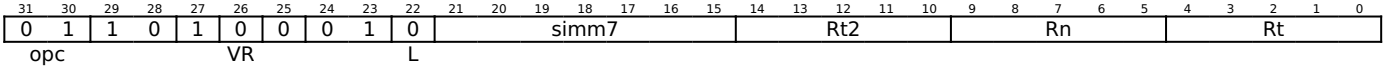
This instruction stores an Allocation Tag and two 64-bit doublewords to memory, from two registers. The address used for the store is calculated from the base register and an immediate signed offset scaled by the Tag Granule. The Allocation Tag is calculated from the Logical Address Tag in the base register.

This instruction generates an Unchecked access.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_MTE)



Encoding

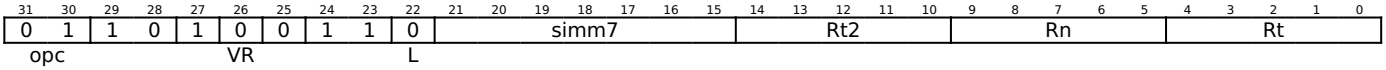
```
STGP <Xt1>, <Xt2>, [<Xn|SP>], #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(simm7), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_MTE)



Encoding

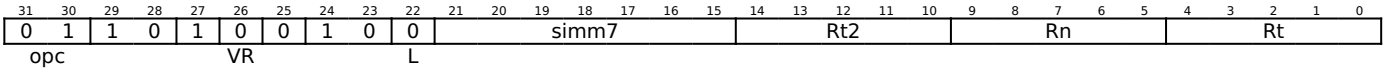
```
STGP <Xt1>, <Xt2>, [<Xn|SP>, #<imm>]!
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(simm7), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_MTE)



Encoding

```
STGP <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(simm7), LOG2_TAG_GRANULE);
let writeback : boolean = FALSE;
let postindex : boolean = FALSE;
```

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> For the "Post-index" and "Pre-index" variants: is the signed immediate offset, a multiple of 16 in the range -1024 to 1008, encoded in the "simm7" field.

For the "Signed offset" variant: is the optional signed immediate offset, a multiple of 16 in the range -1024 to 1008, defaulting to 0 and encoded in the "simm7" field.

Operation

```
var address : bits(64);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let stzgm : boolean = FALSE;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_STORE, stzgm, ispair, t, t2);

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

if !IsAlignedSize(address, TAG_GRANULE) then
    let fault : FaultRecord = AlignmentFault(accdesc, address);
    AArch64_Abort(fault);
end;

let data : bits(128) = (if BigEndian(accdesc.acctype) then X{64}(t) :: X{64}(t2)
                        else X{64}(t2) :: X{64}(t));
Mem{128}(address, accdesc) = data;

AArch64_MemTag(address, accdesc) = AArch64_AllocationTagFromAddress(address);

if writeback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;

    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STILP

Store-release ordered pair of registers

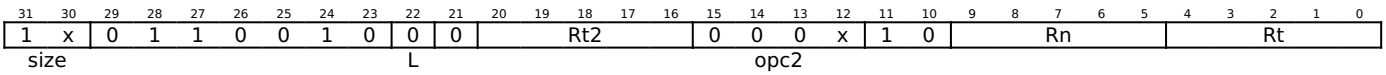
This instruction calculates an address from a base register value and an optional offset, and stores two 32-bit words or two 64-bit doublewords to the calculated address, from two registers. For information on single-copy atomicity and alignment requirements, see *Requirements for single-copy atomicity* and *Alignment of data accesses*. The instruction also has memory ordering semantics, as described in *Load-Acquire, Load-AcquirePC, and Store-Release*, with the additional requirement that:

- When using the pre-index addressing mode, the Memory effects associated with Xt2/Wt2 are Ordered-before the Memory effects associated with Xt1/Wt1.
- For all other addressing modes, the Memory effects associated with Xt1/Wt1 are Ordered-before the Memory effects associated with Xt2/Wt2.

For information about addressing modes, see *Load/Store addressing modes*.

Integer

(FEAT_LRCPC3)



Encoding for the 32-bit pre-index variant

Applies when `(size == 10 && opc2 == 0000)`

```
STILP <Wt1>, <Wt2>, [<Xn|SP>, #-8]!
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc2 == 0001)`

```
STILP <Wt1>, <Wt2>, [<Xn|SP>]
```

Encoding for the 64-bit pre-index variant

Applies when `(size == 11 && opc2 == 0000)`

```
STILP <Xt1>, <Xt2>, [<Xn|SP>, #-16]!
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc2 == 0001)`

```
STILP <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC3) then EndOfDecode(Decode_UNDEF); end;
let ispair : boolean = TRUE;
let wback : boolean = opc2[0] == '0';
```

STILP has the same CONSTRAINED UNPREDICTABLE behavior as STP. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STP and STILP*.

Assembler Symbols

<Wt1>	Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Wt2>	Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let scale : integer{} = 2 + UInt(size[0]);
let datasize : integer{} = 8 << scale;
let offset : integer = if opc2[0] == '0' then -1 * (2 << scale) else 0;
let acqrel : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var rt_unknown : boolean = FALSE;

if wback && (t == n || t2 == n) && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPST);
    assert c IN {Constraint_NONE, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_NONE =>    rt_unknown = FALSE;   // value stored is pre-writeback
        when Constraint_UNKNOWN => rt_unknown = TRUE;    // value stored is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);
var data1 : bits(datasize);
var data2 : bits(datasize);
var accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked,
                                                     ispair, acqrel, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

if rt_unknown && t == n then
    data1 = ARBITRARY : bits(datasize);
else
    data1 = X{datasize}(t);
end;
if rt_unknown && t2 == n then
    data2 = ARBITRARY : bits(datasize);
else
    data2 = X{datasize}(t2);
end;

var full_data : bits(2*datasize);
if BigEndian(accdesc.acctype) then
    full_data = data1::data2;
else
    full_data = data2::data1;
end;
accdesc.highestaddressfirst = offset < 0;
Mem{2*datasize}(address, accdesc) = full_data;

if wback then
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STL64B

Single-copy atomic 64-byte store-release without status result

This instruction stores eight 64-bit doublewords from consecutive registers to a memory location. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt. The data is stored atomically and is required to be 64-byte aligned.

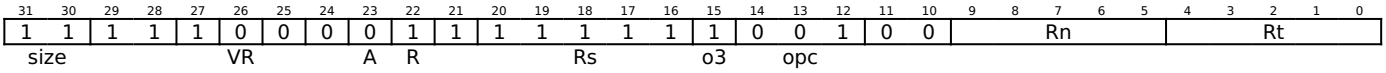
It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports STL64B also supports LD64B, LDA64B, and ST64B.

STL64B stores to memory with release semantics.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64 && FEAT_LSC64B)



Encoding

```
STL64B <Xt>, [<Xn|SP> {, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSC64B) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_LS64) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = FALSE;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = TRUE;
```

STL64B has the same CONSTRAINED UNPREDICTABLE behavior as ST64B. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *ST64B*.

Assembler Symbols

<Xt>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckLDST64BEnabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);

for i = 0 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

MemStore64B(address, data, accdesc);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STL64BV

Single-copy atomic 64-byte store-release with status result

This instruction stores eight 64-bit doublewords from consecutive registers to a memory location, and writes the status result of the store to a register. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt. The data is stored atomically and is required to be 64-byte aligned.

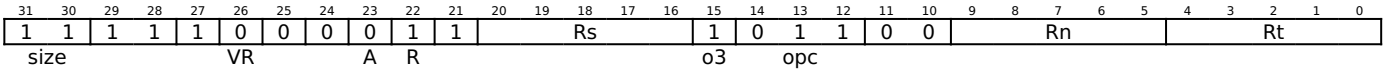
It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports `STL64BV` also supports `ST64BV`, `ST64BV0`, and `STL64BV0`.

`STL64BV` stores to memory with release semantics.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64_V && FEAT_LSC64B)



Encoding

```
STL64BV <Xs>, <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSC64B) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_LS64_V) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = TRUE;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = TRUE;
```

Assembler Symbols

<Xs> Is the 64-bit name of the general-purpose register into which the status result of this instruction is written, encoded in the "Rs" field. The value returned is:

0xFFFFFFFFFFFFFFFF
: If the memory location accessed does not support this instruction. In this case, the value at the memory location is UNKNOWN.

!= 0xFFFFFFFFFFFFFFFF
: If the memory location accessed does support this instruction. In this case, the peripheral that provides the response defines the returned value and provides information on the state of the memory update at the memory location.

If XZR is used, then the return value is ignored.

<Xt> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckST64BVEnabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);

for i = 0 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let status : bits(64) = MemStore64BWithRet(address, data, accdesc);

if s != 31 then X{64}(s) = status; end;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STL64BV0

Single-copy atomic 64-byte EL0 store-release with status result

This instruction stores eight 64-bit doublewords from consecutive registers to a memory location, with the bottom 32 bits taken from ACCDATA_EL1, and writes the status result of the store to a register. The store starts at register Xt, with the data being formed as Data[511:0] = X(t+7)::X(t+6)::X(t+5)::X(t+4)::X(t+3)::X(t+2)::X(t+1)::Xt[63:32]::ACCDATA_EL1()[31:0]. The data is stored atomically and is required to be 64-byte aligned.

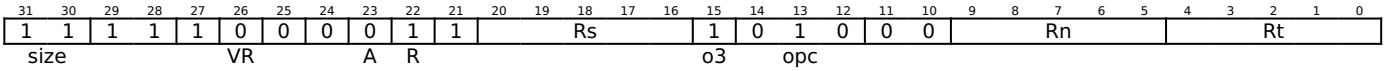
It is IMPLEMENTATION DEFINED which memory locations support this instruction. A memory location that supports `STL64BV0` also supports `ST64BV`, `ST64BV0`, and `STL64BV`.

`STL64BV0` stores to memory with release semantics.

For more information, including about the memory types accessible and how the accesses are performed, see *Single-copy atomic 64-byte load/store*.

Integer

(FEAT_LS64_ACCDATA && FEAT_LSC64B)



Encoding

```
STL64BV0 <Xs>, <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSC64B) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_LS64_ACCDATA) then EndOfDecode(Decode_UNDEF); end;
if Rt[4:3] == '11' || Rt[0] == '1' then EndOfDecode(Decode_UNDEF); end;
let withstatus : boolean = TRUE;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let tagchecked : boolean = n != 31;
let acqrel : boolean = TRUE;
```

Assembler Symbols

<Xs> Is the 64-bit name of the general-purpose register into which the status result of this instruction is written, encoded in the "Rs" field. The value returned is:

0xFFFFFFFFFFFFFFFF
: If the memory location accessed does not support this instruction. In this case, the value at the memory location is UNKNOWN.

!= 0xFFFFFFFFFFFFFFFF
: If the memory location accessed does support this instruction. In this case, the peripheral that provides the response defines the returned value and provides information on the state of the memory update at the memory location.

If XZR is used, then the return value is ignored.

<Xt> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckST64BV0Enabled(acqrel);

var data : bits(512);
var address : bits(64);
var value : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLS64(MemOp_STORE, withstatus, acqrel, tagchecked);

let Xt : bits(64) = X{}(t);
value[31:0] = ACCDATA_EL1()[31:0];
value[63:32] = Xt[63:32];
if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
data[63:0] = value;

for i = 1 to 7 do
    value = X{64}(t+i);
    if BigEndian(accdesc.acctype) then value = BigEndianReverse{64}(value); end;
    data[63+64*i : 64*i] = value;
end;

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let status : bits(64) = MemStore64BWithRet(address, data, accdesc);

if s != 31 then X{64}(s) = status; end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

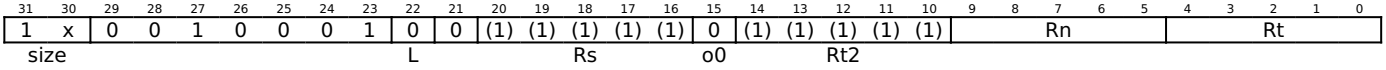
STLLR

Store LORelease register

This instruction stores a 32-bit word or a 64-bit doubleword to a memory location, from a register. The instruction also has memory ordering semantics as described in *Load LOAcquire, Store LORelease*. For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LOR)



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STLLR <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STLLR <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LOR) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLOR(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

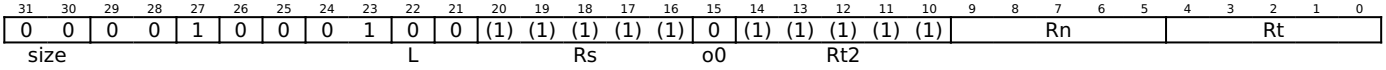
STLLRB

Store LORelease register byte

This instruction stores a byte from a 32-bit register to a memory location. The instruction also has memory ordering semantics as described in *Load LOAcquire, Store LORelease*. For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LOR)



Encoding

```
STLLRB <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LOR) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLOR(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

Mem{8}(address, accdesc) = X{8}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

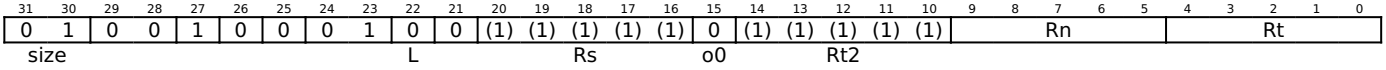
STLLRH

Store LORelease register halfword

This instruction stores a halfword from a 32-bit register to a memory location. The instruction also has memory ordering semantics as described in *Load LOAcquire, Store LORelease*. For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LOR)



Encoding

```
STLLRH <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LOR) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescLOR(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

Mem{16}(address, accdesc) = X{16}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLP

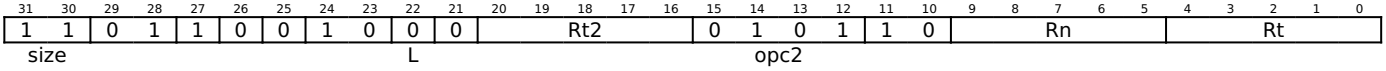
Store-release pair of registers

This instruction calculates an address from a base register value, and stores two 64-bit doublewords to the calculated address, from two registers. Explicit Memory effects produced by this instruction have Release semantics. For information about memory ordering semantics, see *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSCP)



Encoding

```
STLP <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSCP) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let acquire : boolean = FALSE;
let tagchecked : boolean = n != 31;
let ispair : boolean = TRUE;
```

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data1 : bits(64);
var data2 : bits(64);
let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked, ispair,
                                                     acquire, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

data1 = X{64}(t);
data2 = X{64}(t2);
let full_data : bits(128) = (if BigEndian(accdesc.acctype) then data1::data2
                                                           else data2::data1);
Mem{128}(address, accdesc) = full_data;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

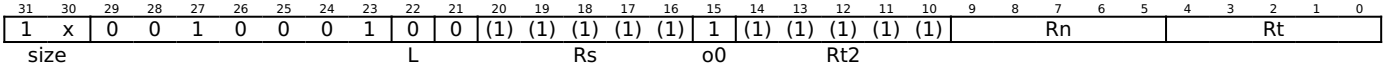
STLR

Store-release register

This instruction stores a 32-bit word or a 64-bit doubleword to a memory location, from a register. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 2 classes: No offset and Pre-index

No offset



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STLR <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

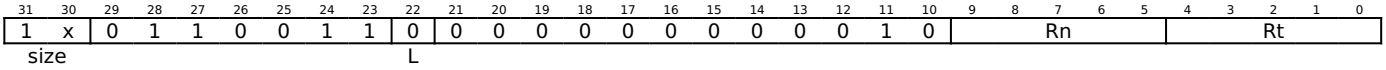
```
STLR <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let wback : boolean = FALSE;

let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let offset : integer = 0;
let acquire : boolean = FALSE;
let tagchecked : boolean = n != 31;

let rt_unknown : boolean = FALSE;
```

Pre-index

(FEAT_LRCPC3)



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STLR <Wt>, [<Xn|SP>, #-4]!
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STLR <Xt>, [<Xn|SP>, #-8]!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC3) then EndOfDecode(Decode_UNDEF); end;
let wback : boolean = TRUE;

let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let offset : integer = -1 * (1 << UInt(size));
let acquire : boolean = FALSE;
let tagchecked : boolean = TRUE;

var rt_unknown : boolean = FALSE;

if n == t && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPST);
    assert c IN {Constraint_NONE, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_NONE =>    rt_unknown = FALSE;   // value stored is original value
        when Constraint_UNKNOWN => rt_unknown = TRUE;    // value stored is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);
var data : bits(datasize);
if rt_unknown then
    data = ARBITRARY : bits(datasize);
else
    data = X{datasize}(t);
end;
Mem{datasize}(address, accdesc) = data;
if wback then
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

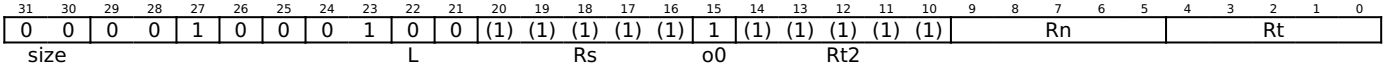
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLRB

Store-release register byte

This instruction stores a byte from a 32-bit register to a memory location. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STLRB <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

Mem{8}(address, accdesc) = X{8}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

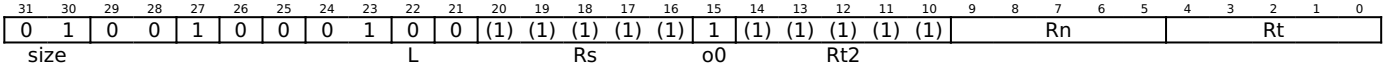
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLRH

Store-release register halfword

This instruction stores a halfword from a 32-bit register to a memory location. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STLRH <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

Mem{16}(address, accdesc) = X{16}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLTXR

Store-release unprivileged exclusive register

This instruction stores a 32-bit word or a 64-bit doubleword to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

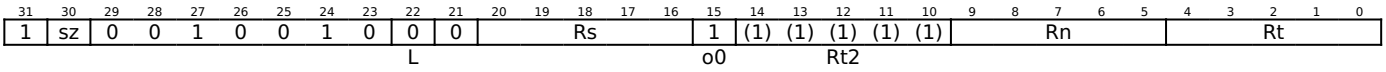
Note

For the purposes of the Exclusives monitors, and the forward progress guarantees for Load-Exclusive and Store-Exclusive loops, `STLTXR` is equivalent to `STXR`.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
STLTXR <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
STLTXR <Ws>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

`STLTXR` has the same CONSTRAINED UNPREDICTABLE behavior as `STLXR`. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STLXR*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(elsize);
else
    data = X{elsize}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{elsize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLUR

Store-release register (unscaled)

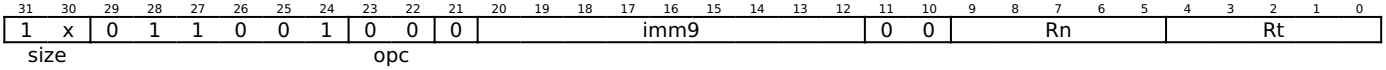
This instruction calculates an address from a base register value and an immediate offset, and stores a 32-bit word or a 64-bit doubleword to the calculated address, from a register.

The instruction has memory ordering semantics as described in *Load-Acquire, Load-AcquirePC, and Store-Release*

For information about addressing modes, see *Load/Store addressing modes*.

Unscaled offset

(FEAT_LRCPC2)



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STLUR <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STLUR <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC2) then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);

let datasize : integer{} = 8 << scale;
let acquire : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLURB

Store-release register byte (unscaled)

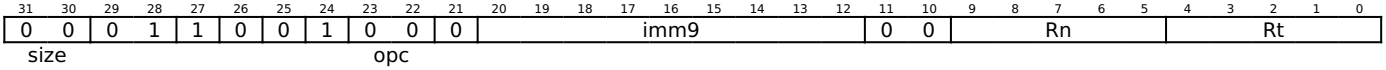
This instruction calculates an address from a base register value and an immediate offset, and stores a byte to the calculated address, from a 32-bit register.

The instruction has memory ordering semantics as described in *Load-Acquire, Load-AcquirePC, and Store-Release*

For information about addressing modes, see *Load/Store addressing modes*.

Unscaled offset

(FEAT_LRCPC2)



Encoding

```
STLURB <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC2) then EndOfDecode(Decode_UNDEF); end;
let offset : bits(64) = SignExtend{}(imm9);
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);

let datasize : integer{} = 8;
let acquire : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLURH

Store-release register halfword (unscaled)

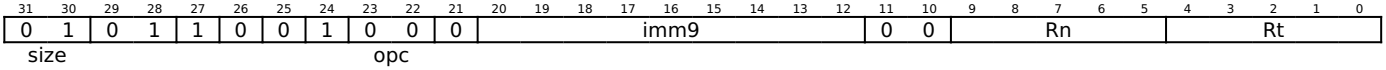
This instruction calculates an address from a base register value and an immediate offset, and stores a halfword to the calculated address, from a 32-bit register.

The instruction has memory ordering semantics as described in *Load-Acquire, Load-AcquirePC, and Store-Release*

For information about addressing modes, see *Load/Store addressing modes*.

Unscaled offset

(FEAT_LRCPC2)



Encoding

```
STLURH <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LRCPC2) then EndOfDecode(Decode_UNDEF); end;
let offset : bits(64) = SignExtend{}(imm9);
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);

let datasize : integer{} = 16;
let acquire : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Operation

```
var address : bits(64);

let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

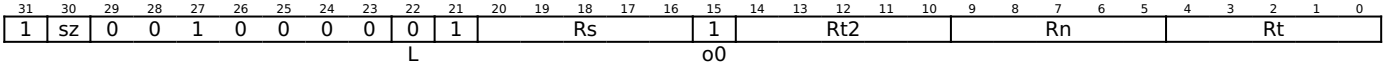
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLXP

Store-release exclusive pair of registers

This instruction stores two 32-bit words or two 64-bit doublewords to a memory location if the PE has exclusive access to the memory address, from two registers, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. For information on single-copy atomicity and alignment requirements, see *Requirements for single-copy atomicity* and *Alignment of data accesses*. If a 64-bit pair Store-Exclusive succeeds, it causes a single-copy atomic update of the 128-bit memory location being updated. The instruction also has memory ordering semantics, as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
STLXP <Ws>, <Wt1>, <Wt2>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
STLXP <Ws>, <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let datasize : integer{} = elsize * 2;
let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t || s == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // store UNKNOWN value
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rn_unknown = TRUE;    // address is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *CONSTRAINED UNPREDICTABLE behaviors with Load-Exclusive/Store-Exclusive pairs* and *STLXP*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt1> Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Wt2> Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt1> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xt2> Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(datasize);

let dbytes : integer{} = datasize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, ispair, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(datasize);
else
    let el1 : bits(datasize DIV 2) = X{}(t);
    let el2 : bits(datasize DIV 2) = X{}(t2);
    data = if BigEndian(accdesc.acctype) then el1::el2 else el2::el1;
end;
var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{datasize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

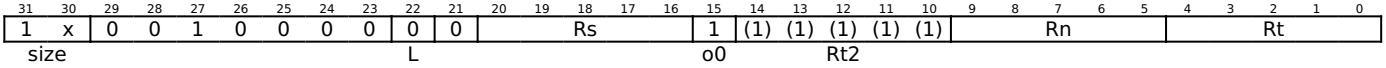
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLXR

Store-release exclusive register

This instruction stores a 32-bit word or a 64-bit doubleword to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STLXR <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STLXR <Ws>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 8 << UInt(size);
let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STLXR*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(elsize);
else
    data = X{elsize}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{elsize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

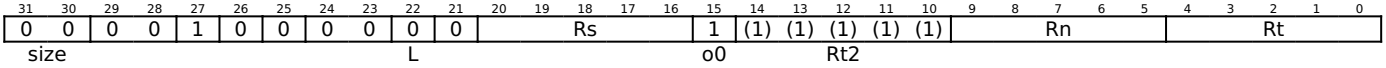
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLXRB

Store-release exclusive register byte

This instruction stores a byte from a 32-bit register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STLXRB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STLXRB*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Aborts

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(8);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(8);
else
    data = X{8}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, 1, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{8}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

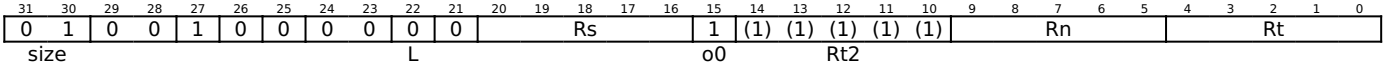
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLXRH

Store-release exclusive register halfword

This instruction stores a halfword from a 32-bit register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic. The instruction also has memory ordering semantics as described in *Load-Acquire, Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STLXRH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = TRUE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STLXRH*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

A non halfword-aligned memory address causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(16);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(16);
else
    data = X{16}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, 2, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{16}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

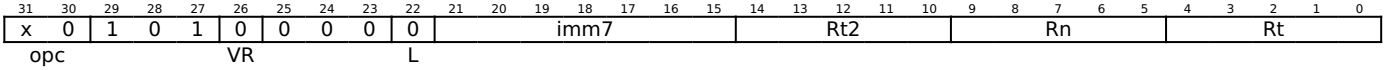
This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNP

Store pair of registers, with non-temporal hint

This instruction calculates an address from a base register value and an immediate offset, and stores two 32-bit words or two 64-bit doublewords to the calculated address, from two registers. For information about addressing modes, see *Load/Store addressing modes*. For information about non-temporal pair instructions, see *Load/Store non-temporal pair*.



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
STNP <Wt1>, <Wt2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
STNP <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for all variants of this encoding

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = TRUE;
let scale : integer{} = 2 + UInt(opc[1]);
let datasize : integer{} = 8 << scale;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Wt1>	Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Wt2>	Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "32-bit" variant: is the optional signed immediate byte offset, a multiple of 4 in the range -256 to 252, defaulting to 0 and encoded in the "imm7" field as <imm>/4. For the "64-bit" variant: is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8.
<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Operation

```
var address : bits(64);
let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, ispair, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);
let data : bits(2*datasize) = (if BigEndian(accdesc.acctype)
                                 then X{datasize}(t) :: X{datasize}(t2)
                                 else X{datasize}(t2) :: X{datasize}(t));
Mem{2*datasize}(address, accdesc) = data;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

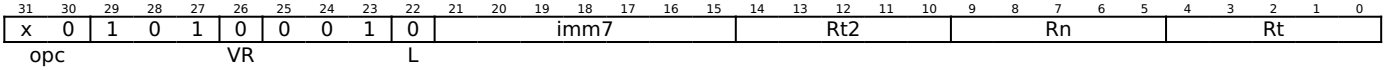
STP

Store pair of registers

This instruction calculates an address from a base register value and an immediate offset, and stores two 32-bit words or two 64-bit doublewords to the calculated address, from two registers. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
STP <Wt1>, <Wt2>, [<Xn|SP>], #<imm>
```

Encoding for the 64-bit variant

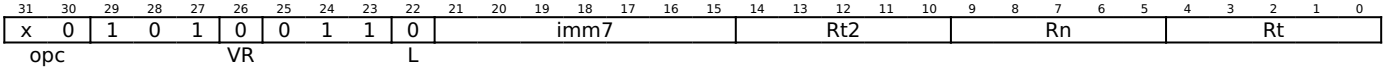
Applies when `(opc == 10)`

```
STP <Xt1>, <Xt2>, [<Xn|SP>], #<imm>
```

Decode for all variants of this encoding

```
let wback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
STP <Wt1>, <Wt2>, [<Xn|SP>, #<imm>]!
```

Encoding for the 64-bit variant

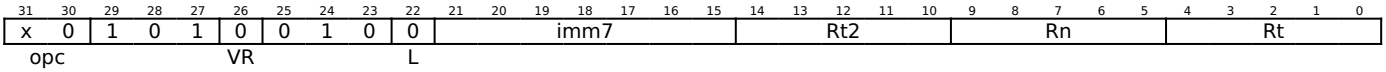
Applies when `(opc == 10)`

```
STP <Xt1>, <Xt2>, [<Xn|SP>, #<imm>]!
```

Decode for all variants of this encoding

```
let wback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
STP <Wt1>, <Wt2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 10)`

```
STP <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for all variants of this encoding

```
let wback : boolean = FALSE;
let postindex : boolean = FALSE;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STP*.

Assembler Symbols

<Wt1>	Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Wt2>	Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Post-index 32-bit" and "Pre-index 32-bit" variants: is the signed immediate byte offset, a multiple of 4 in the range -256 to 252, encoded in the "imm7" field as <imm>/4. For the "Post-index 64-bit" and "Pre-index 64-bit" variants: is the signed immediate byte offset, a multiple of 8 in the range -512 to 504, encoded in the "imm7" field as <imm>/8. For the "Signed offset 32-bit" variant: is the optional signed immediate byte offset, a multiple of 4 in the range -256 to 252, defaulting to 0 and encoded in the "imm7" field as <imm>/4. For the "Signed offset 64-bit" variant: is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8.
<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let scale : integer{} = 2 + UInt(opc[1]);
let datasize : integer{} = 8 << scale;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = wback || n != 31;

var rt_unknown : boolean = FALSE;

if wback && (t == n || t2 == n) && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPST);
    assert c IN {Constraint_NONE, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_NONE =>       rt_unknown = FALSE;   // Value stored is pre-writeback
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // Value stored is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);
var data1 : bits(datasize);
var data2 : bits(datasize);
let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, ispair, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

if rt_unknown && t == n then
    data1 = ARBITRARY : bits(datasize);
else
    data1 = X{datasize}(t);
end;
if rt_unknown && t2 == n then
    data2 = ARBITRARY : bits(datasize);
else
    data2 = X{datasize}(t2);
end;

let data : bits(2*datasize) = (if BigEndian(accdesc.acctype) then data1::data2
                                                             else data2::data1);
Mem{2 * datasize}(address, accdesc) = data;

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

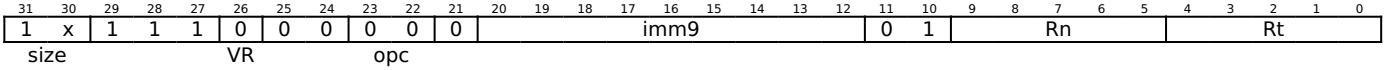
STR (immediate)

Store register (immediate)

This instruction stores a word or a doubleword from a register to memory. The address that is used for the store is calculated from a base register and an immediate offset. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STR <Wt>, [<Xn|SP>], #<simm>
```

Encoding for the 64-bit variant

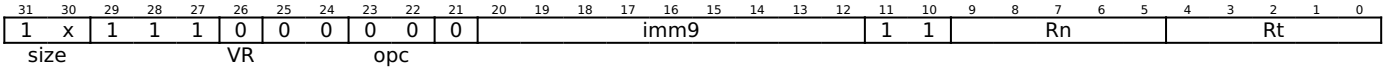
Applies when `(size == 11)`

```
STR <Xt>, [<Xn|SP>], #<simm>
```

Decode for all variants of this encoding

```
let wback : boolean = TRUE;
let postindex : boolean = TRUE;
let scale : integer{} = UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STR <Wt>, [<Xn|SP>, #<simm>]!
```

Encoding for the 64-bit variant

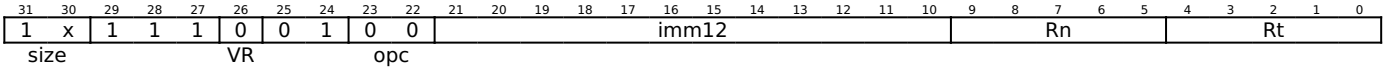
Applies when `(size == 11)`

```
STR <Xt>, [<Xn|SP>, #<simm>]!
```

Decode for all variants of this encoding

```
let wback : boolean = TRUE;
let postindex : boolean = FALSE;
let scale : integer{} = UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STR <Wt>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STR <Xt>, [<Xn|SP>{, #<pimm>}]
```

Decode for all variants of this encoding

```
let wback : boolean = FALSE;
let postindex : boolean = FALSE;
let scale : integer{} = UInt(size);
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), scale);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<pimm>	For the "32-bit" variant: is the optional positive immediate byte offset, a multiple of 4 in the range 0 to 16380, defaulting to 0 and encoded in the "imm12" field as <pimm>/4. For the "64-bit" variant: is the optional positive immediate byte offset, a multiple of 8 in the range 0 to 32760, defaulting to 0 and encoded in the "imm12" field as <pimm>/8.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << scale;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var c : Constraint;
var rt_unknown : boolean = FALSE;
if wback && n == t && n != 31 then
    c = ConstrainUnpredictable(Unpredictable_WBOVERLAPST);
    assert c IN {Constraint_NONE, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_NONE =>    rt_unknown = FALSE;    // Value stored is original value
        when Constraint_UNKNOWN => rt_unknown = TRUE;     // Value stored is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

var data : bits(datasize);
if rt_unknown then
    data = ARBITRARY : bits(datasize);
else
    data = X{datasize}(t);
end;

Mem{datasize}(address, accdesc) = data;

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

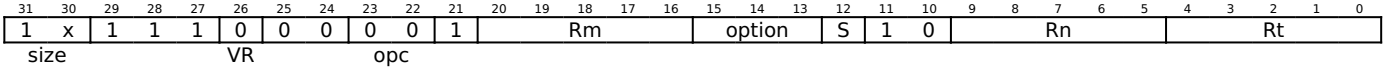
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STR (register)

Store register (register)

This instruction calculates an address from a base register value and an offset register value, and stores a 32-bit word or a 64-bit doubleword to the calculated address, from a register. For information about addressing modes, see *Load/Store addressing modes*.

The instruction uses an offset addressing mode, that calculates the address used for the memory access from a base register value and an offset register value. The offset can be optionally shifted and extended.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STR <Wt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STR <Xt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Decode for all variants of this encoding

```
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
let extend_type : ExtendType = DecodeRegExtend(option);
let scale : integer{} = UInt(size);
let shift : integer{} = if S == '1' then scale else 0;
```

Assembler Symbols

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Wm> When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.

<Xm> When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.

<extend> Is the index extend/shift specifier, defaulting to LSL, and which must be omitted for the LSL option when <amount> is omitted, encoded in “option”:

option	<extend>
010	UXTW
011	LSL
110	SXTW
111	SXTX

<amount> For the "32-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#2

For the "64-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#3

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 8 << scale;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

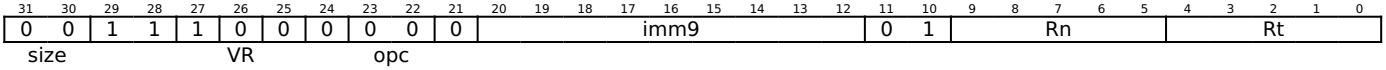
STRB (immediate)

Store register byte (immediate)

This instruction stores the least significant byte of a 32-bit register to memory. The address that is used for the store is calculated from a base register and an immediate offset. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index



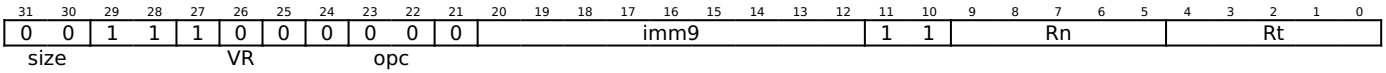
Encoding

```
STRB <Wt>, [<Xn|SP>], #<simm>
```

Decode for this encoding

```
let wback : boolean = TRUE;
let postindex : boolean = TRUE;
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index



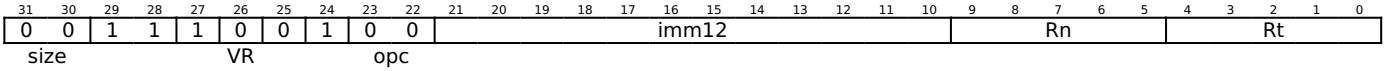
Encoding

```
STRB <Wt>, [<Xn|SP>, #<simm>]!
```

Decode for this encoding

```
let wback : boolean = TRUE;
let postindex : boolean = FALSE;
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset



Encoding

```
STRB <Wt>, [<Xn|SP>{, #<pimm>}]
```

Decode for this encoding

```
let wback : boolean = FALSE;
let postindex : boolean = FALSE;
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), 0);
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STRB (immediate)*.

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<pimm>	Is the optional positive immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var c : Constraint;
var rt_unknown : boolean = FALSE;
if wback && n == t && n != 31 then
    c = ConstrainUnpredictable(Unpredictable_WBOVERLAPST);
    assert c IN {Constraint_NONE, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_NONE =>    rt_unknown = FALSE;    // Value stored is original value
        when Constraint_UNKNOWN => rt_unknown = TRUE;     // Value stored is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

var data : bits(8);
if rt_unknown then
    data = ARBITRARY : bits(8);
else
    data = X{8}(t);
end;

Mem{8}(address, accdesc) = data;

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

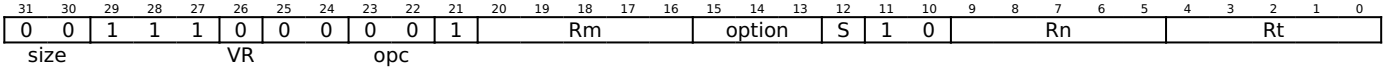
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STRB (register)

Store register byte (register)

This instruction calculates an address from a base register value and an offset register value, and stores a byte from a 32-bit register to the calculated address. For information about addressing modes, see *Load/Store addressing modes*.

The instruction uses an offset addressing mode, that calculates the address used for the memory access from a base register value and an offset register value. The offset can be optionally shifted and extended.



Encoding for the Extended register variant

Applies when `(option != 011)`

```
STRB <Wt>, [<Xn|SP>, (<Wm>|<Xm>), <extend> {<amount>}]
```

Encoding for the Shifted register variant

Applies when `(option == 011)`

```
STRB <Wt>, [<Xn|SP>, <Xm>{, LSL <amount>}]
```

Decode for all variants of this encoding

```
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
let extend_type : ExtendType = DecodeRegExtend(option);
let shift : integer{} = 0;
```

Assembler Symbols

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Wm> When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.

<Xm> When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.

<extend> Is the index extend specifier, encoded in “option”:

option	<extend>
010	UXTW
110	SXTW
111	SXTX

<amount> Is the index shift amount, it must be #0, encoded in "S" as 0 if omitted, or as 1 if present.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{8}(address, accdesc) = X{8}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

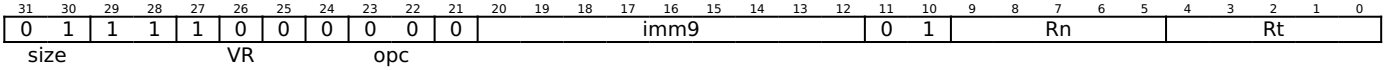
STRH (immediate)

Store register halfword (immediate)

This instruction stores the least significant halfword of a 32-bit register to memory. The address that is used for the store is calculated from a base register and an immediate offset. For information about addressing modes, see *Load/Store addressing modes*.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index



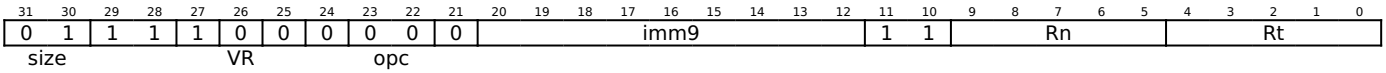
Encoding

```
STRH <Wt>, [<Xn|SP>], #<simm>
```

Decode for this encoding

```
let wback : boolean = TRUE;
let postindex : boolean = TRUE;
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index



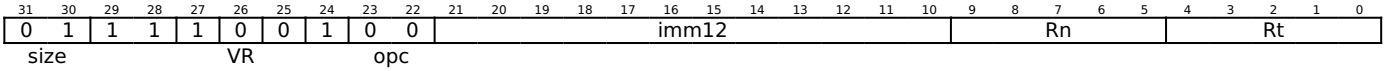
Encoding

```
STRH <Wt>, [<Xn|SP>, #<simm>]!
```

Decode for this encoding

```
let wback : boolean = TRUE;
let postindex : boolean = FALSE;
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset



Encoding

```
STRH <Wt>, [<Xn|SP>{, #<pimm>}]
```

Decode for this encoding

```
let wback : boolean = FALSE;
let postindex : boolean = FALSE;
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), 1);
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STRH (immediate)*.

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<pimm>	Is the optional positive immediate byte offset, a multiple of 2 in the range 0 to 8190, defaulting to 0 and encoded in the "imm12" field as <pimm>/2.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;

var c : Constraint;
var rt_unknown : boolean = FALSE;
if wback && n == t && n != 31 then
    c = ConstrainUnpredictable(Unpredictable_WBOVERLAPST);
    assert c IN {Constraint_NONE, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_NONE =>    rt_unknown = FALSE;    // Value stored is original value
        when Constraint_UNKNOWN => rt_unknown = TRUE;     // Value stored is UNKNOWN
        when Constraint_UNDEF =>   EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>     EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

var data : bits(16);
if rt_unknown then
    data = ARBITRARY : bits(16);
else
    data = X{16}(t);
end;

Mem{16}(address, accdesc) = data;

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

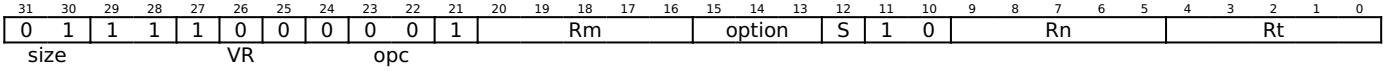
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STRH (register)

Store register halfword (register)

This instruction calculates an address from a base register value and an offset register value, and stores a halfword from a 32-bit register to the calculated address. For information about addressing modes, see *Load/Store addressing modes*.

The instruction uses an offset addressing mode, that calculates the address used for the memory access from a base register value and an offset register value. The offset can be optionally shifted and extended.



Encoding

```
STRH <Wt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Decode for this encoding

```
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
let extend_type : ExtendType = DecodeRegExtend(option);
let shift : integer{} = if S == '1' then 1 else 0;
```

Assembler Symbols

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Wm> When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.

<Xm> When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.

<extend> Is the index extend/shift specifier, defaulting to LSL, and which must be omitted for the LSL option when <amount> is omitted, encoded in “option”:

option	<extend>
010	UXTW
011	LSL
110	SXTW
111	SXTX

<amount> Is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#1

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{16}(address, accdesc) = X{16}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STSET, STSETL

Atomic bit set on word or doubleword, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, performs a bitwise OR with the value held in a register on it, and stores the result back to memory.

- `STSET` does not have release semantics.
- `STSETL` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

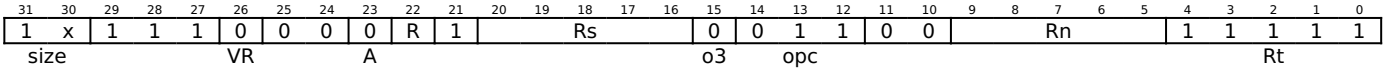
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDSET, LDSETA, LDSETAL, LDSETL. This means:

- The encodings in this description are named to match the encodings of LDSET, LDSETA, LDSETAL, LDSETL.
- The description of LDSET, LDSETA, LDSETAL, LDSETL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STSET <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSET <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(size == 10 && R == 1)`

```
STSETL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSETL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STSET <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDSET <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(size == 11 && R == 1)`

```
STSETL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDSETL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDSET, LDSETA, LDSETAL, LDSETL gives the operational pseudocode for this instruction.

Operational Information

The description of LDSET, LDSETA, LDSETAL, LDSETL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STSETB, STSETLB

Atomic bit set on byte, without return

This instruction atomically loads an 8-bit byte from memory, performs a bitwise OR with the value held in a register on it, and stores the result back to memory.

- `STSETB` does not have release semantics.
- `STSETLB` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

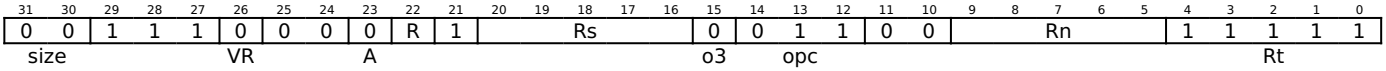
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDSETB, LDSETAB, LDSETALB, LDSETLB. This means:

- The encodings in this description are named to match the encodings of LDSETB, LDSETAB, LDSETALB, LDSETLB.
- The description of LDSETB, LDSETAB, LDSETALB, LDSETLB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STSETB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSETB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STSETLB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSETLB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDSETB, LDSETAB, LDSETALB, LDSETLB gives the operational pseudocode for this instruction.

Operational Information

The description of LDSETB, LDSETAB, LDSETALB, LDSETLB gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STSETH, STSETLH

Atomic bit set on halfword, without return

This instruction atomically loads a 16-bit halfword from memory, performs a bitwise OR with the value held in a register on it, and stores the result back to memory.

- `STSETH` does not have release semantics.
- `STSETLH` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

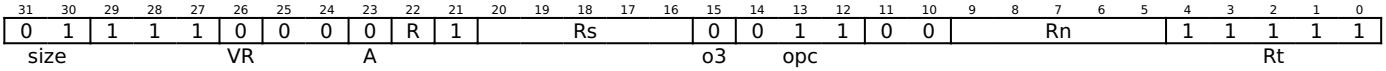
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDSETH, LDSETAH, LDSETALH, LDSETLH. This means:

- The encodings in this description are named to match the encodings of LDSETH, LDSETAH, LDSETALH, LDSETLH.
- The description of LDSETH, LDSETAH, LDSETALH, LDSETLH gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STSETH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSETH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STSETLH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSETLH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDSETH, LDSETAH, LDSETALH, LDSETLH gives the operational pseudocode for this instruction.

Operational Information

The description of LDSETH, LDSETAH, LDSETALH, LDSETLH gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STSHH

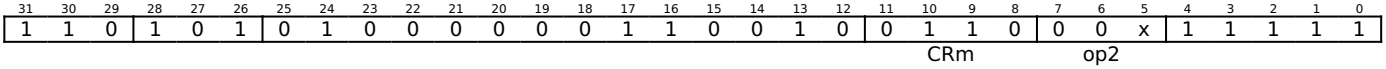
Store shared hint

This instruction signals to the memory system that if the next instruction in program order generates an explicit write effect, then it is to a location that one or more other threads of execution will observe, and there is a performance benefit to ensuring that the updated value from the write to that location propagates to those other observers with minimal latency.

The thread of execution on the other observers might be polling the location using load or Load-Exclusive instructions, or may have executed a `PRFM IR` instruction targeting the location.

System

(FEAT_PCDPHINT)



Encoding

`STSHH` <policy>

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PCDPHINT) then EndOfDecode(Decode_NOP); end;
let stream : boolean = op2[0] == '1';
```

Assembler Symbols

<policy> <policy> is one of:

KEEP
: Signals to the memory system that there may be a performance benefit to retaining the updated location in the local cache of the updating PE.

STRM
: Signals to the memory system that there may be a performance benefit to ensuring the updated location is not retained in the local cache of the updating PE.

op2[0]	<policy>
0	KEEP
1	STRM

Operation

```
Hint_StoreShared(stream);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STSMAX, STSMAXL

Atomic signed maximum on word or doubleword, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as signed numbers.

- `STSMAX` does not have release semantics.
- `STSMAXL` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

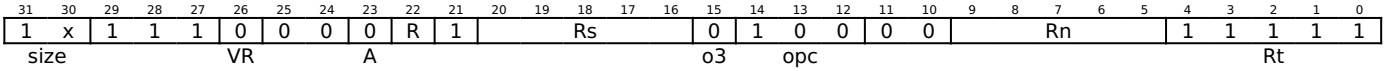
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL. This means:

- The encodings in this description are named to match the encodings of LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL.
- The description of LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STSMAX <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMAX <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(size == 10 && R == 1)`

```
STSMAXL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMAXL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STSMAX <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDSMAX <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(size == 11 && R == 1)`

```
STSMAXL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDSMAXL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL gives the operational pseudocode for this instruction.

Operational Information

The description of LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STSMAXB, STSMAXLB

Atomic signed maximum on byte, without return

This instruction atomically loads an 8-bit byte from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as signed numbers.

- `STSMAXB` does not have release semantics.
- `STSMAXLB` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

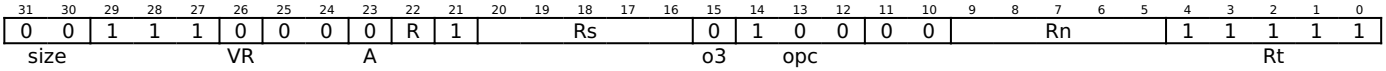
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB. This means:

- The encodings in this description are named to match the encodings of LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB.
- The description of LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STSMAXB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMAXB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STSMAXLB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMAXLB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB gives the operational pseudocode for this instruction.

Operational Information

The description of LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STSMAXH, STSMAXLH

Atomic signed maximum on halfword, without return

This instruction atomically loads a 16-bit halfword from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as signed numbers.

- `STSMAXH` does not have release semantics.
- `STSMAXLH` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

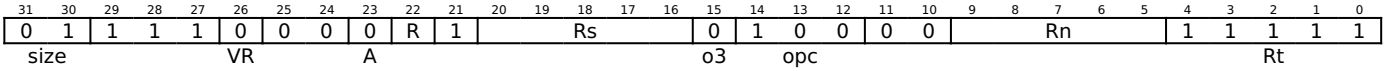
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH. This means:

- The encodings in this description are named to match the encodings of LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH.
- The description of LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STSMAXH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMAXH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STSMAXLH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMAXLH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH gives the operational pseudocode for this instruction.

Operational Information

The description of LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STSMIN, STSMINL

Atomic signed minimum on word or doubleword, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as signed numbers.

- `STSMIN` does not have release semantics.
- `STSMINL` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

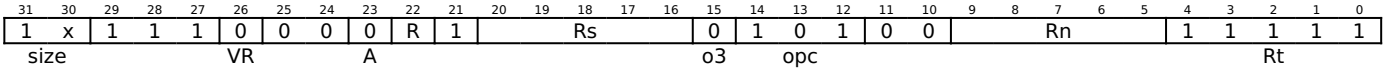
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDSMIN, LDSMINA, LDSMINAL, LDSMINL. This means:

- The encodings in this description are named to match the encodings of LDSMIN, LDSMINA, LDSMINAL, LDSMINL.
- The description of LDSMIN, LDSMINA, LDSMINAL, LDSMINL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STSMIN <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMIN <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(size == 10 && R == 1)`

```
STSMINL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMINL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STSMIN <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDSMIN <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(size == 11 && R == 1)`

```
STSMINL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDSMINL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDSMIN, LDSMINA, LDSMINAL, LDSMINL gives the operational pseudocode for this instruction.

Operational Information

The description of LDSMIN, LDSMINA, LDSMINAL, LDSMINL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STSMINB, STSMINLB

Atomic signed minimum on byte, without return

This instruction atomically loads an 8-bit byte from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as signed numbers.

- `STSMINB` does not have release semantics.
- `STSMINLB` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

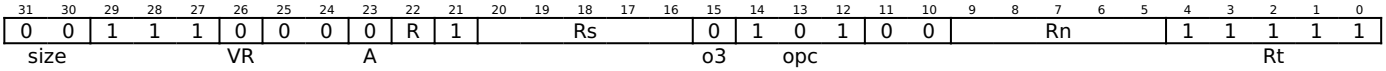
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB. This means:

- The encodings in this description are named to match the encodings of LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB.
- The description of LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STSMINB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMINB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STSMINLB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMINLB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB gives the operational pseudocode for this instruction.

Operational Information

The description of LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STSMINH, STSMINLH

Atomic signed minimum on halfword, without return

This instruction atomically loads a 16-bit halfword from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as signed numbers.

- `STSMINH` does not have release semantics.
- `STSMINLH` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

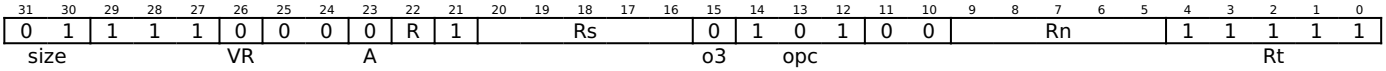
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH. This means:

- The encodings in this description are named to match the encodings of LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH.
- The description of LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STSMINH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMINH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STSMINLH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDSMINLH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH gives the operational pseudocode for this instruction.

Operational Information

The description of LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTADD, STTADDL

Atomic add unprivileged, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, adds the value held in a register to it, and stores the result back to memory.

- `STTADD` does not have release semantics.
- `STTADDL` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

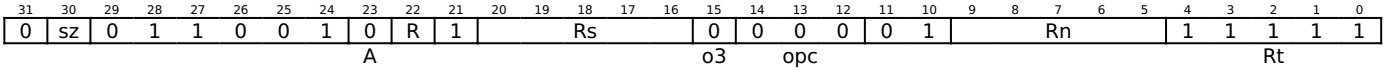
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDTADD, LDTADDA, LDTADDAL, LDTADDL. This means:

- The encodings in this description are named to match the encodings of LDTADD, LDTADDA, LDTADDAL, LDTADDL.
- The description of LDTADD, LDTADDA, LDTADDAL, LDTADDL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSUI)



Encoding for the 32-bit no memory ordering variant

Applies when `(sz == 0 && R == 0)`

```
STTADD <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDTADD <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(sz == 0 && R == 1)`

```
STTADDL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDTADDL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(sz == 1 && R == 0)`

```
STTADD <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDTADD <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(sz == 1 && R == 1)`

```
STTADDL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDTADDL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDTADD, LDTADDA, LDTADDAL, LDTADDL gives the operational pseudocode for this instruction.

Operational Information

The description of LDTADD, LDTADDA, LDTADDAL, LDTADDL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTCLR, STTCLRL

Atomic bit clear unprivileged, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, performs a bitwise AND with the complement of the value held in a register on it, and stores the result back to memory.

- `STTCLR` does not have release semantics.
- `STTCLRL` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

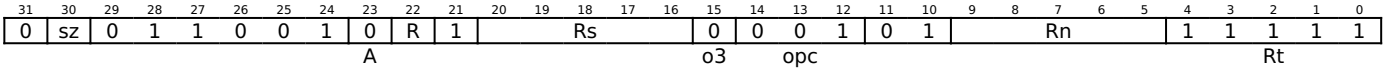
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL. This means:

- The encodings in this description are named to match the encodings of LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL.
- The description of LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSUI)



Encoding for the 32-bit no memory ordering variant

Applies when `(sz == 0 && R == 0)`

```
STTCLR <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDTCLR <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(sz == 0 && R == 1)`

```
STTCLRL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDTCLRL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(sz == 1 && R == 0)`

```
STTCLR <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDTCLR <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(sz == 1 && R == 1)`

```
STTCLRL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDTCLRL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL gives the operational pseudocode for this instruction.

Operational Information

The description of LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTNP

Store unprivileged pair of registers, with non-temporal hint

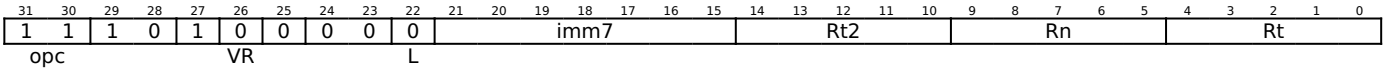
This instruction calculates an address from a base register value and an immediate offset, and stores two 64-bit doublewords to the calculated address, from two registers. For information about addressing modes, see *Load/Store addressing modes*. For information about non-temporal pair instructions, see *Load/Store non-temporal pair*.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Signed offset

(FEAT_LSUI)



Encoding

```
STTNP <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = TRUE;
let datasize : integer{} = 64;
let offset : bits(64) = LSL(SignExtend{64}(imm7), 3);
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8.

Operation

```
var address : bits(64);
let privileged : boolean = AArch64_IsUnprivAccessPriv();
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, ispair, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);
let data : bits(2*datasize) = (if BigEndian(accdesc.acctype)
                                 then X{datasize}(t) :: X{datasize}(t2)
                                 else X{datasize}(t2) :: X{datasize}(t));
Mem{2*datasize}(address, accdesc) = data;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTP

Store unprivileged pair of registers

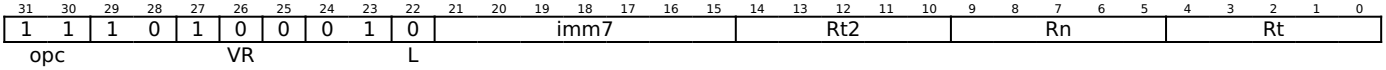
This instruction calculates an address from a base register value and an immediate offset, and stores two 64-bit doublewords to the calculated address, from two registers.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.
For information about addressing modes, see *Load/Store addressing modes*.
It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_LSUI)



Encoding

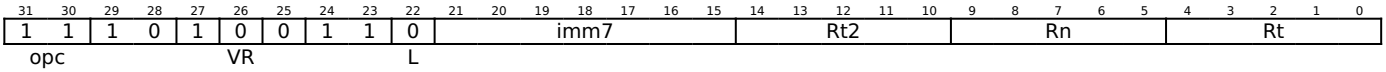
```
STTP <Xt1>, <Xt2>, [<Xn|SP>], #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;

let wback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_LSUI)



Encoding

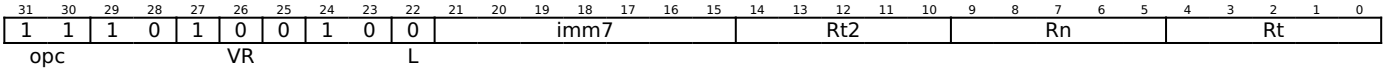
```
STTP <Xt1>, <Xt2>, [<Xn|SP>, #<imm>]!
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;

let wback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_LSUI)



Encoding

```
STTP <Xt1>, <Xt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;

let wback : boolean = FALSE;
let postindex : boolean = FALSE;
```

STTP has the same CONSTRAINED UNPREDICTABLE behavior as STP. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STP*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Post-index" and "Pre-index" variants: is the signed immediate byte offset, a multiple of 8 in the range -512 to 504, encoded in the "imm7" field as <imm>/8. For the "Signed offset" variant: is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let scale : integer{} = 2 + UInt(opc[1]);
let datasize : integer{} = 64;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = wback || n != 31;

var rt_unknown : boolean = FALSE;

if wback && (t == n || t2 == n) && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_WBOVERLAPST);
    assert c IN {Constraint_NONE, Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_NONE =>       rt_unknown = FALSE;   // Value stored is pre-writeback
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // Value stored is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
var address : bits(64);
var data1 : bits(datasize);
var data2 : bits(datasize);
let privileged : boolean = AArch64_IsUnprivAccessPriv();
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, ispair, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

if rt_unknown && t == n then
    data1 = ARBITRARY : bits(datasize);
else
    data1 = X{datasize}(t);
end;
if rt_unknown && t2 == n then
    data2 = ARBITRARY : bits(datasize);
else
    data2 = X{datasize}(t2);
end;

let data : bits(2*datasize) = (if BigEndian(accdesc.acctype) then data1::data2
                                                             else data2::data1);
Mem{2 * datasize}(address, accdesc) = data;

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTR

Store register (unprivileged)

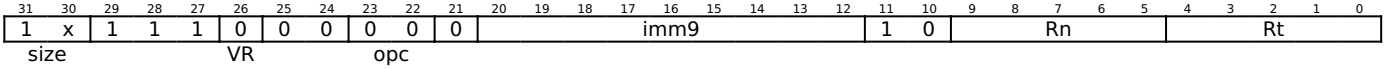
This instruction stores a word or doubleword from a register to memory. The address that is used for the store is calculated from a base register and an immediate offset.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STTR <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STTR <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
let scale : integer{} = UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let datasize : integer{} = 8 << scale;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTRB

Store register byte (unprivileged)

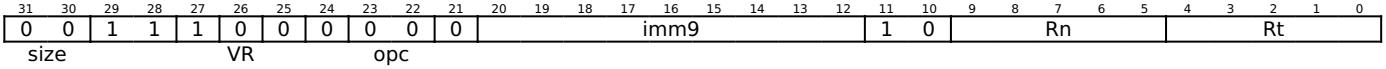
This instruction stores a byte from a 32-bit register to memory. The address that is used for the store is calculated from a base register and an immediate offset.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STTRB <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let datasize : integer{} = 8;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTRH

Store register halfword (unprivileged)

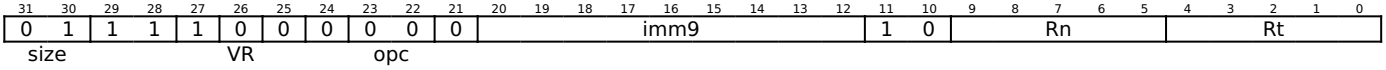
This instruction stores a halfword from a 32-bit register to memory. The address that is used for the store is calculated from a base register and an immediate offset.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STTRH <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let datasize : integer{} = 16;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTSET, STTSETL

Atomic bit set unprivileged, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, performs a bitwise OR with the value held in a register on it, and stores the result back to memory.

- `STTSET` does not have release semantics.
- `STTSETL` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

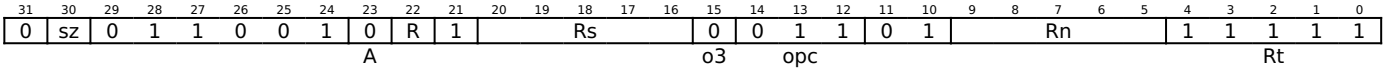
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDTSET, LDTSETA, LDTSETAL, LDTSETL. This means:

- The encodings in this description are named to match the encodings of LDTSET, LDTSETA, LDTSETAL, LDTSETL.
- The description of LDTSET, LDTSETA, LDTSETAL, LDTSETL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSUI)



Encoding for the 32-bit no memory ordering variant

Applies when `(sz == 0 && R == 0)`

```
STTSET <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDTSET <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(sz == 0 && R == 1)`

```
STTSETL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDTSETL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(sz == 1 && R == 0)`

```
STTSET <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDTSET <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(sz == 1 && R == 1)`

```
STTSETL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDTSETL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDTSET, LDTSETA, LDTSETAL, LDTSETL gives the operational pseudocode for this instruction.

Operational Information

The description of LDTSET, LDTSETA, LDTSETAL, LDTSETL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTXR

Store unprivileged exclusive register

This instruction stores a 32-bit word or a 64-bit doubleword from a register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*.
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

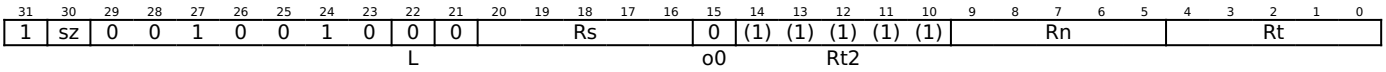
Note

For the purposes of the Exclusives monitors, and the forward progress guarantees for Load-Exclusive and Store-Exclusive loops, STTXR is equivalent to STXR.

For information about addressing modes, see *Load/Store addressing modes*.

No offset

(FEAT_LSUI)



Encoding for the 32-bit variant

Applies when (sz == 0)

```
STTXR <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when (sz == 1)

```
STTXR <Ws>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // store UNKNOWN value
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rn_unknown = TRUE;    // address is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

STTXR has the same CONSTRAINED UNPREDICTABLE behavior as STXR. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STXR*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(elsize);
else
    data = X{elsize}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{elsize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STUMAX, STUMAXL

Atomic unsigned maximum on word or doubleword, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as unsigned numbers.

- STUMAX does not have release semantics.
- STUMAXL stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

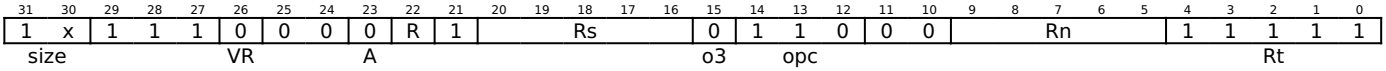
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL. This means:

- The encodings in this description are named to match the encodings of LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL.
- The description of LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STUMAX <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMAX <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(size == 10 && R == 1)`

```
STUMAXL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMAXL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STUMAX <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDUMAX <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(size == 11 && R == 1)`

```
STUMAXL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDUMAXL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL gives the operational pseudocode for this instruction.

Operational Information

The description of LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STUMAXB, STUMAXLB

Atomic unsigned maximum on byte, without return

This instruction atomically loads an 8-bit byte from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as unsigned numbers.

- `STUMAXB` does not have release semantics.
- `STUMAXLB` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

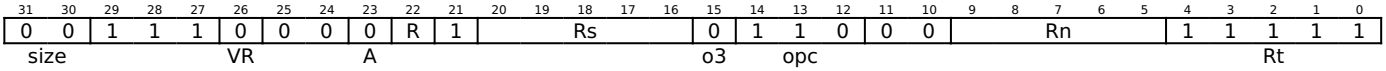
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB. This means:

- The encodings in this description are named to match the encodings of LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB.
- The description of LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STUMAXB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMAXB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STUMAXLB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMAXLB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB gives the operational pseudocode for this instruction.

Operational Information

The description of LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STUMAXH, STUMAXLH

Atomic unsigned maximum on halfword, without return

This instruction atomically loads a 16-bit halfword from memory, compares it against the value held in a register, and stores the larger value back to memory, treating the values as unsigned numbers.

- `STUMAXH` does not have release semantics.
- `STUMAXLH` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

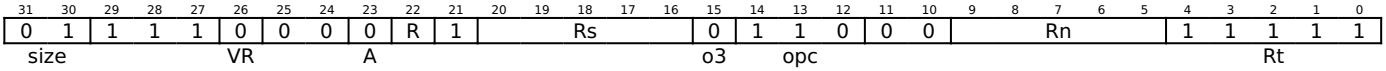
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH. This means:

- The encodings in this description are named to match the encodings of LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH.
- The description of LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STUMAXH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMAXH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STUMAXLH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMAXLH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH gives the operational pseudocode for this instruction.

Operational Information

The description of LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STUMIN, STUMINL

Atomic unsigned minimum on word or doubleword, without return

This instruction atomically loads a 32-bit word or 64-bit doubleword from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as unsigned numbers.

- `STUMIN` does not have release semantics.
- `STUMINL` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

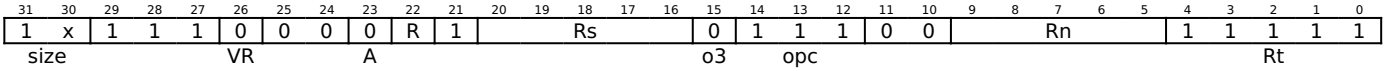
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDUMIN, LDUMINA, LDUMINAL, LDUMINL. This means:

- The encodings in this description are named to match the encodings of LDUMIN, LDUMINA, LDUMINAL, LDUMINL.
- The description of LDUMIN, LDUMINA, LDUMINAL, LDUMINL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the 32-bit no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STUMIN <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMIN <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 32-bit release variant

Applies when `(size == 10 && R == 1)`

```
STUMINL <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMINL <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STUMIN <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDUMIN <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the 64-bit release variant

Applies when `(size == 11 && R == 1)`

```
STUMINL <Xs>, [<Xn|SP>]
```

is equivalent to

```
LDUMINL <Xs>, XZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xs> Is the 64-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

The description of LDUMIN, LDUMINA, LDUMINAL, LDUMINL gives the operational pseudocode for this instruction.

Operational Information

The description of LDUMIN, LDUMINA, LDUMINAL, LDUMINL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STUMINB, STUMINLB

Atomic unsigned minimum on byte, without return

This instruction atomically loads an 8-bit byte from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as unsigned numbers.

- `STUMINB` does not have release semantics.
- `STUMINLB` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

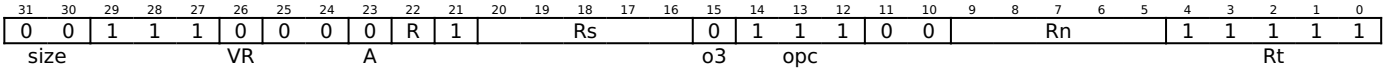
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB. This means:

- The encodings in this description are named to match the encodings of LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB.
- The description of LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STUMINB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMINB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STUMINLB <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMINLB <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB gives the operational pseudocode for this instruction.

Operational Information

The description of LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STUMINH, STUMINLH

Atomic unsigned minimum on halfword, without return

This instruction atomically loads a 16-bit halfword from memory, compares it against the value held in a register, and stores the smaller value back to memory, treating the values as unsigned numbers.

- `STUMINH` does not have release semantics.
- `STUMINLH` stores to memory with release semantics, as described in *Load-Acquire, Store-Release*.

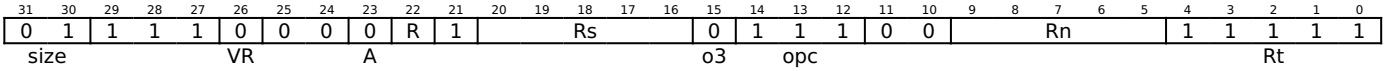
For information about addressing modes, see *Load/Store addressing modes*.

This is an alias of LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH. This means:

- The encodings in this description are named to match the encodings of LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH.
- The description of LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Integer

(FEAT_LSE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STUMINH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMINH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Encoding for the Release variant

Applies when `(R == 1)`

```
STUMINLH <Ws>, [<Xn|SP>]
```

is equivalent to

```
LDUMINLH <Ws>, WZR, [<Xn|SP>]
```

and is always the preferred disassembly.

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

The description of LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH gives the operational pseudocode for this instruction.

Operational Information

The description of LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH gives the operational information for this instruction.

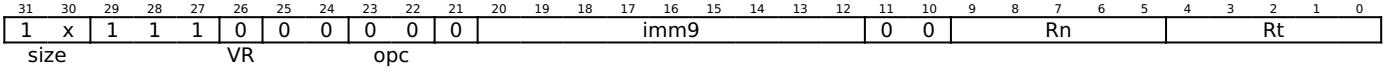
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STUR

Store register (unscaled)

This instruction calculates an address from a base register value and an immediate offset, and stores a 32-bit word or a 64-bit doubleword to the calculated address, from a register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STUR <Wt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STUR <Xt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
let scale : integer{} = UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let datasize : integer{} = 8 << scale;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

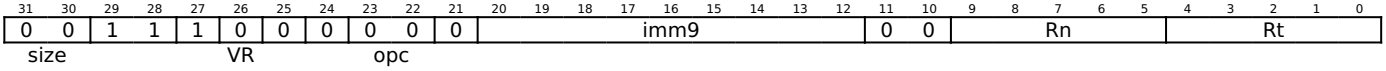
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STURB

Store register byte (unscaled)

This instruction calculates an address from a base register value and an immediate offset, and stores a byte to the calculated address, from a 32-bit register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STURB <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let datasize : integer{} = 8;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

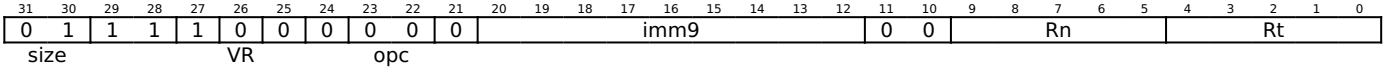
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STURH

Store register halfword (unscaled)

This instruction calculates an address from a base register value and an immediate offset, and stores a halfword to the calculated address, from a 32-bit register. For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STURH <Wt>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Wt>	Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.

Shared Decode

```
let n : integer = UInt(Rn);
let t : integer = UInt(Rt);
let datasize : integer{} = 16;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                  tagchecked, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = X{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

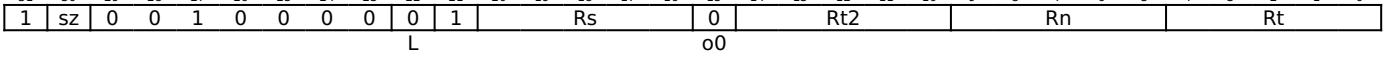
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STXP

Store exclusive pair of registers

This instruction stores two 32-bit words or two 64-bit doublewords from two registers to a memory location if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. For information on single-copy atomicity and alignment requirements, see *Requirements for single-copy atomicity* and *Alignment of data accesses*. If a 64-bit pair Store-Exclusive succeeds, it causes a single-copy atomic update of the 128-bit memory location being updated. For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(sz == 0)`

```
STXP <Ws>, <Wt1>, <Wt2>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(sz == 1)`

```
STXP <Ws>, <Xt1>, <Xt2>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 32 << UInt(sz);
let datasize : integer{} = elsize * 2;
let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t || s == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // store UNKNOWN value
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rn_unknown = TRUE;    // address is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *CONSTRAINED UNPREDICTABLE behaviors with Load-Exclusive/Store-Exclusive pairs* and *STXP*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt1> Is the 32-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Wt2> Is the 32-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt1> Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.

<Xt2> Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(datasize);

let dbytes : integer{} = datasize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, ispair, t, t2);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(datasize);
else
    let el1 : bits(datasize DIV 2) = X{}(t);
    let el2 : bits(datasize DIV 2) = X{}(t2);
    data = if BigEndian(accdesc.acctype) then el1::el2 else el2::el1;
end;
var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{datasize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

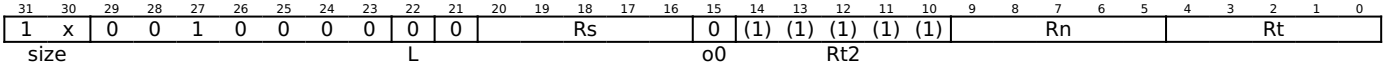
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STXR

Store exclusive register

This instruction stores a 32-bit word or a 64-bit doubleword from a register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding for the 32-bit variant

Applies when `(size == 10)`

```
STXR <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
STXR <Ws>, <Xt>, [<Xn|SP>{, #0}]
```

Decode for all variants of this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let elsize : integer{} = 8 << UInt(size);
let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STXR*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xt> Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

Accessing an address that is not aligned to the size of the data being accessed causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(elsize);

let dbytes : integer{} = elsize DIV 8;
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(elsize);
else
    data = X{elsize}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, dbytes, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{elsize}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

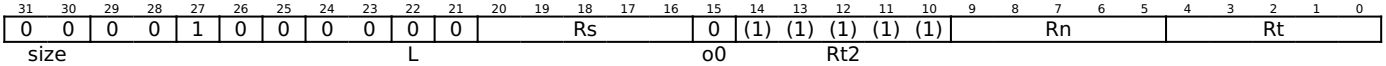
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STXRB

Store exclusive register byte

This instruction stores a byte from a register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STXRB <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STXRB*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Aborts

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(8);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(8);
else
    data = X{8}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, 1, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{8}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

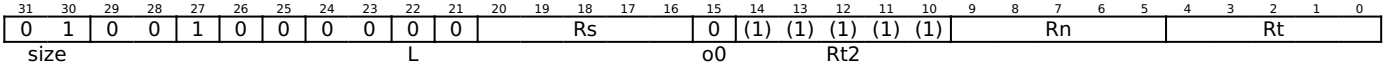
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STXRH

Store exclusive register halfword

This instruction stores a halfword from a register to memory if the PE has exclusive access to the memory address, and returns a status value of 0 if the store was successful, or of 1 if no store was performed. See *Synchronization and semaphores*. The memory access is atomic.

For information about addressing modes, see *Load/Store addressing modes*.



Encoding

```
STXRH <Ws>, <Wt>, [<Xn|SP>{, #0}]
```

Decode for this encoding

```
let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acqrel : boolean = FALSE;
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
var rn_unknown : boolean = FALSE;
if s == t then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_DATAOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // store UNKNOWN value
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
if s == n && n != 31 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_BASEOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rn_unknown = TRUE;     // address is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *STXRH*.

Assembler Symbols

<Ws> Is the 32-bit name of the general-purpose register into which the status result of the store exclusive is written, encoded in the "Rs" field. The value returned is:

0
: If the operation updates memory.

1
: If the operation fails to update memory.

<Wt> Is the 32-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Aborts and alignment

If a synchronous Data Abort or Watchpoint exception is generated by the execution of this instruction:

- Memory is not updated.
- <Ws> is not updated.

A non halfword-aligned memory address causes an Alignment fault Data Abort exception to be generated, subject to the following rules:

- If AArch64_ExclusiveMonitorsPass() returns TRUE, the exception is generated.
- Otherwise, it is IMPLEMENTATION DEFINED whether the exception is generated.

If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address, if accessed, would generate a synchronous Data Abort or Watchpoint exception, it is IMPLEMENTATION DEFINED whether the exception is generated.

Operation

```
var address : bits(64);
var data : bits(16);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                     privileged, t);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
elsif rn_unknown then
    address = ARBITRARY : bits(64);
else
    address = X{64}(n);
end;

if rt_unknown then
    data = ARBITRARY : bits(16);
else
    data = X{16}(t);
end;

var status : bit = '1';
// Check whether the Exclusives monitors are set to include the
// physical memory locations corresponding to virtual address
// range [address, address+dbytes-1].
// If AArch64_ExclusiveMonitorsPass() returns FALSE and the memory address,
// if accessed, would generate a synchronous Data Abort or Watchpoint exception,
// it is IMPLEMENTATION DEFINED whether the exception is generated.

// If FEAT_SPE is implemented, it is also IMPLEMENTATION DEFINED whether or not the
// physical address packet is output when permitted and when
// AArch64_ExclusiveMonitorPass() returns FALSE for a Store Exclusive instruction.
// This behavior is not reflected here due to the previously stated limitation.
if AArch64_ExclusiveMonitorsPass(address, 2, accdesc) then
    // This atomic write will be rejected if it does not refer
    // to the same physical locations after address translation.
    Mem{16}(address, accdesc) = data;
    status = ExclusiveMonitorsStatus();
end;
X{32}(s) = ZeroExtend{32}(status);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STZ2G

Store Allocation Tags, zeroing

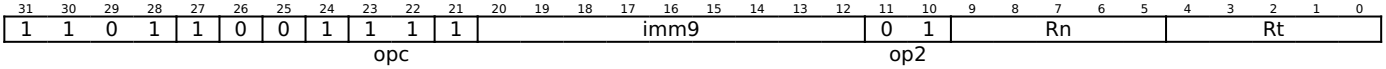
This instruction stores an Allocation Tag to two Tag Granules of memory, zeroing the associated data locations. The address used for the store is calculated from the base register and an immediate signed offset scaled by the Tag Granule. The Allocation Tag is calculated from the Logical Address Tag in the source register.

This instruction generates an Unchecked access.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_MTE)



Encoding

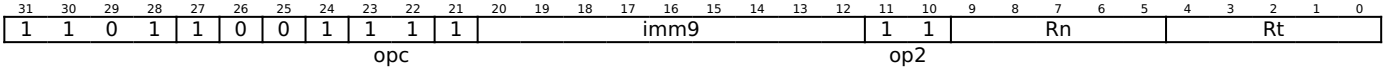
```
STZ2G <Xt|SP>, [<Xn|SP>], #<simm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_MTE)



Encoding

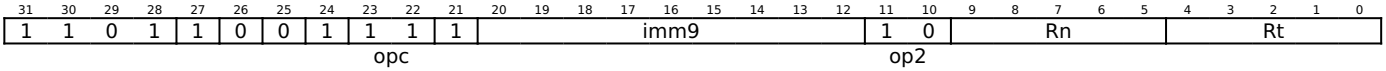
```
STZ2G <Xt|SP>, [<Xn|SP>, #<simm>]!
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_MTE)



Encoding

```
STZ2G <Xt|SP>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = FALSE;
let postindex : boolean = FALSE;
```

Assembler Symbols

<Xt|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate offset, a multiple of 16 in the range -4096 to 4080, defaulting to 0 and encoded in the "imm9" field.

Operation

```
var address : bits(64);
var address2 : bits(64);
let data : bits(64) = if t == 31 then SP{64}() else X{64}(t);
let tag : bits(4) = AArch64_AllocationTagFromAddress(data);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let stzgm : boolean = FALSE;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_STORE, stzgm, ispair, t, t);

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

address2 = AddressIncrement(address, TAG_GRANULE, accdesc);

if !IsAlignedSize(address, TAG_GRANULE) then
    let fault : FaultRecord = AlignmentFault(accdesc, address);
    AArch64_Abort(fault);
end;

Mem{16*TAG_GRANULE}(address, accdesc) = Zeros{TAG_GRANULE * 16};

AArch64_MemTag(address, accdesc) = tag;
AArch64_MemTag(address2, accdesc) = tag;

if writeback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;

    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STZG

Store Allocation Tag, zeroing

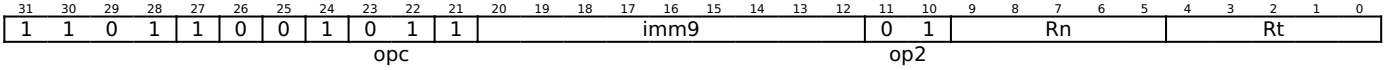
This instruction stores an Allocation Tag to memory, zeroing the associated data location. The address used for the store is calculated from the base register and an immediate signed offset scaled by the Tag Granule. The Allocation Tag is calculated from the Logical Address Tag in the source register.

This instruction generates an Unchecked access.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_MTE)



Encoding

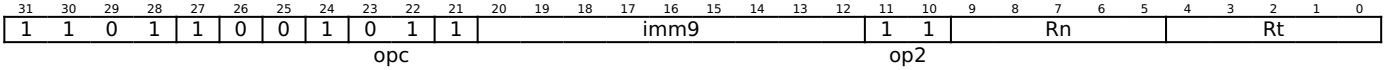
```
STZG <Xt|SP>, [<Xn|SP>], #<simm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_MTE)



Encoding

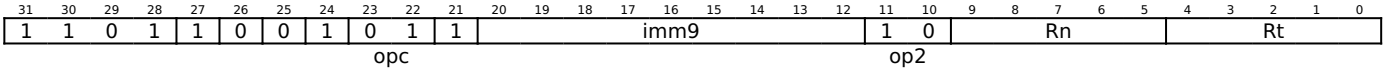
```
STZG <Xt|SP>, [<Xn|SP>, #<simm>]!
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_MTE)



Encoding

```
STZG <Xt|SP>, [<Xn|SP>{, #<simm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let offset : bits(64) = LSL(SignExtend{64}(imm9), LOG2_TAG_GRANULE);
let writeback : boolean = FALSE;
let postindex : boolean = FALSE;
```

Assembler Symbols

<Xt|SP>	Is the 64-bit name of the general-purpose source register or stack pointer, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate offset, a multiple of 16 in the range -4096 to 4080, defaulting to 0 and encoded in the "imm9" field.

Operation

```
var address : bits(64);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let stzgm : boolean = FALSE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_STORE, stzgm, t);

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

if !IsAlignedSize(address, TAG_GRANULE) then
    let fault : FaultRecord = AlignmentFault(accdesc, address);
    AArch64_Abort(fault);
end;

Mem{8*TAG_GRANULE}(address, accdesc) = Zeros{TAG_GRANULE * 8};

let data : bits(64) = if t == 31 then SP{64}() else X{64}(t);
let tag : bits(4) = AArch64_AllocationTagFromAddress(data);
AArch64_MemTag(address, accdesc) = tag;

if writeback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;

    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STZGM

Store Allocation Tag and zero multiple

This instruction writes a naturally aligned block of N Allocation Tags and stores zero to the associated data locations, where the size of N is identified in DCZID_EL0.BS, and the Allocation Tag is taken from the source register bits<3:0>.

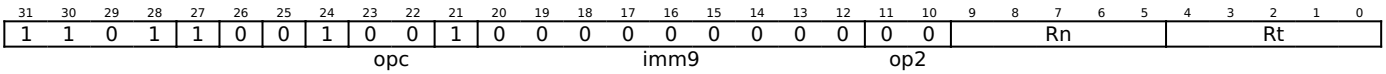
If the value of HCR_EL2.TGE is 0 and the Effective value of SCTLR2_EL1.GM0 is 0, this instruction is UNDEFINED at EL0.

If the value of HCR_EL2.TGE is 1 and the Effective value of SCTLR2_EL2.GM0 is 0, this instruction is UNDEFINED at EL0.

This instruction generates an Unchecked access.

Integer

(FEAT_MTE2)



Encoding

```
STZGM <Xt>, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
if !IsGMEnabledAtEL0() && PSTATE.EL == EL0 then Undefined(); end;

let data : bits(64) = X{64}(t);
let tag : bits(4) = data[3:0];
var address : bits(64);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

let size : integer{} = 4 * 2 ^ UInt(DCZID_EL0().BS);
address = AlignDownSize(address, size);
let count : integer = size >> LOG2_TAG_GRANULE;
let stzgm : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescLDGSTG(MemOp_STORE, stzgm, t);

for i = 0 to count-1 do
    AArch64_MemTag(address, accdesc) = tag;
    Mem{8*TAG_GRANULE}(address, accdesc) = Zeros{8*TAG_GRANULE};
    address = AddressIncrement(address, TAG_GRANULE, accdesc);
end;
```

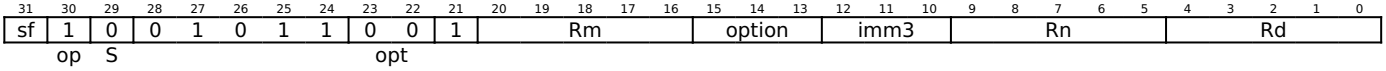
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUB (extended register)

Subtract extended and scaled register

This instruction subtracts a sign-extended or zero-extended register value, followed by an optional left shift amount, from a register value, and writes the result to the destination register. The argument that is extended from the <Rm> register can be a byte, halfword, word, or doubleword.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SUB <Wd|WSP>, <Wn|WSP>, <Wm>{, <extend> {#<amount>}}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SUB <Xd|SP>, <Xn|SP>, <R><m>{, <extend> {#<amount>}}
```

Decode for all variants of this encoding

```
if imm3 IN {'101', '110', '111'} then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let shift : integer{} = UInt(imm3) as integer{0..4};
let datasize : integer{} = 32 << UInt(sf);
let extend_type : ExtendType = DecodeRegExtend(option);
```

Assembler Symbols

<Wd|WSP> Is the 32-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.

<Wn|WSP> Is the 32-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<extend> For the "32-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	LSL|UXTW
011	UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rd" or "Rn" is '11111' (WSP) and "option" is '010' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTW when "option" is '010'.

For the "64-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	UXTW
011	LSL|UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rd" or "Rn" is '11111' (SP) and "option" is '011' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTX when "option" is '011'.

<amount> Is the left shift amount to be applied after extension in the range 0 to 4, defaulting to 0, encoded in the "imm3" field. It must be absent when <extend> is absent, is required when <extend> is LSL, and is optional when <extend> is present but not LSL.

<Xd|SP> Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.

<Xn|SP> Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<R> Is a width specifier, encoded in “option”:

option	<R>
00x	W
010	W
x11	X
10x	W
110	W

<m> Is the number [0-30] of the second general-purpose source register or the name ZR (31), encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = if n == 31 then SP{datasize}() else X{datasize}(n);
let operand2 : bits(datasize) = NOT ExtendReg{datasize}(m, extend_type, shift);

var result : bits(datasize);
(result, -) = AddWithCarry{datasize}(operand1, operand2, '1');

if d == 31 then
    SP{64}() = ZeroExtend{64}(result);
else
    X{datasize}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

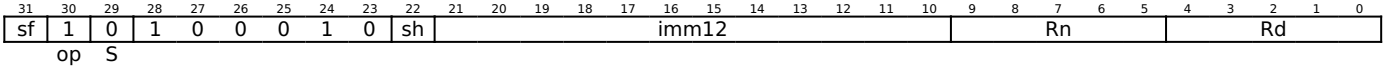
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUB (immediate)

Subtract immediate value

This instruction subtracts an optionally-shifted immediate value from a register value, and writes the result to the destination register.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SUB <Wd|WSP>, <Wn|WSP>, #<imm>{, <shift>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SUB <Xd|SP>, <Xn|SP>, #<imm>{, <shift>}
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);

let imm : bits(24) = if sh == '0' then Zeros{12}::imm12 else imm12::Zeros{12};
```

Assembler Symbols

<Wd|WSP> Is the 32-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.

<Wn|WSP> Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

<imm> Is an unsigned immediate, in the range 0 to 4095, encoded in the "imm12" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #12

<Xd|SP> Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.

<Xn|SP> Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = if n == 31 then SP{datasize}() else X{datasize}(n);
let operand2 : bits(datasize) = NOT ZeroExtend{datasize}(imm);

var result : bits(datasize);
(result, -) = AddWithCarry{datasize}(operand1, operand2, '1');

if d == 31 then
    SP{64}() = ZeroExtend{64}(result);
else
    X{datasize}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

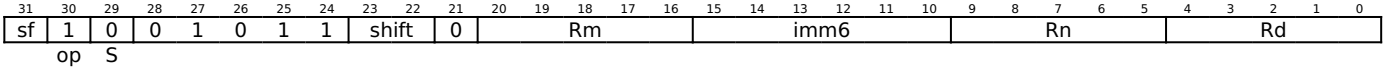
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUB (shifted register)

Subtract optionally-shifted register

This instruction subtracts an optionally-shifted register value from a register value, and writes the result to the destination register.
This instruction is used by the alias NEG (shifted register).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SUB <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SUB <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if shift == '11' then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
NEG (shifted register)	`Rn == '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = NOT ShiftReg{datasize}(m, shift_type, shift_amount);

var result : bits(datasize);
(result, -) = AddWithCarry{datasize}(operand1, operand2, '1');

X{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

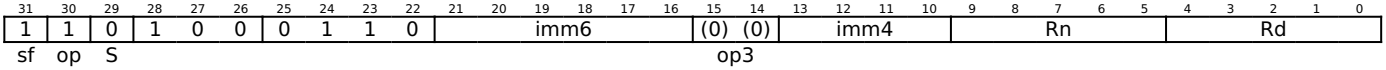
SUBG

Subtract with tag

This instruction subtracts an immediate value scaled by the Tag Granule from the address in the source register, modifies the Logical Address Tag of the address using an immediate value, and writes the result to the destination register. Tags specified in GCR_EL1.Exclude are excluded from the possible outputs when modifying the Logical Address Tag.

Integer

(FEAT_MTE)



Encoding

```
SUBG <Xd|SP>, <Xn|SP>, #<uimm6>, #<uimm4>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let tag_offset : bits(4) = imm4;
let offset : bits(64) = LSL(ZeroExtend{64}(imm6), LOG2_TAG_GRANULE);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.
<uimm6>	Is an unsigned immediate, a multiple of 16 in the range 0 to 1008, encoded in the "imm6" field.
<uimm4>	Is an unsigned immediate, in the range 0 to 15, encoded in the "imm4" field.

Operation

```
let operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let start_tag : bits(4) = AArch64_AllocationTagFromAddress(operand1);
let exclude : bits(16) = GCR_EL1().Exclude;
let rtag : bits(4) = AArch64_ChooseNonExcludedTagOrZero(start_tag, tag_offset, exclude);
var result : bits(64);

(result, -) = AddWithCarry{64}(operand1, NOT offset, '1');

result = AArch64_AddressWithAllocationTag(result, rtag);

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

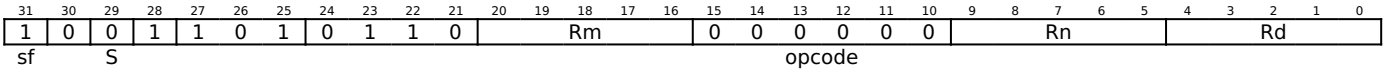
SUBP

Subtract pointer

This instruction subtracts the 56-bit address held in the second source register from the 56-bit address held in the first source register, sign-extends the result to 64 bits, and writes the result to the destination register.

Integer

(FEAT_MTE)



Encoding

```
SUBP <Xd>, <Xn|SP>, <Xm|SP>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.
<Xm|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.

Operation

```
var operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
var operand2 : bits(64) = if m == 31 then SP{64}() else X{64}(m);
operand1 = SignExtend{64}(operand1[55:0]);
operand2 = NOT SignExtend{64}(operand2[55:0]);
var result : bits(64);

(result, -) = AddWithCarry{64}(operand1, operand2, '1');

X{64}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUBPS

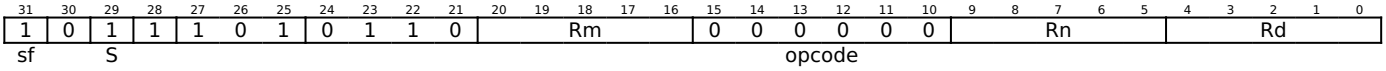
Subtract pointer, setting flags

This instruction subtracts the 56-bit address held in the second source register from the 56-bit address held in the first source register, sign-extends the result to 64 bits, and writes the result to the destination register. It updates the condition flags based on the result of the subtraction.

This instruction is used by the alias CMPP.

Integer

(FEAT_MTE)



Encoding

```
SUBPS <Xd>, <Xn|SP>, <Xm|SP>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_MTE) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.
<Xm|SP>	Is the 64-bit name of the second general-purpose source register or stack pointer, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CMPP	`S == '1' && Rd == '11111'`

Operation

```
var operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
var operand2 : bits(64) = if m == 31 then SP{64}() else X{64}(m);
operand1 = SignExtend{64}(operand1[55:0]);
operand2 = NOT SignExtend{64}(operand2[55:0]);
var result : bits(64);

var nzcv : bits(4);

(result, nzcv) = AddWithCarry{64}(operand1, operand2, '1');

X{64}(d) = result;
PSTATE.[N,Z,C,V] = nzcv;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUBPT

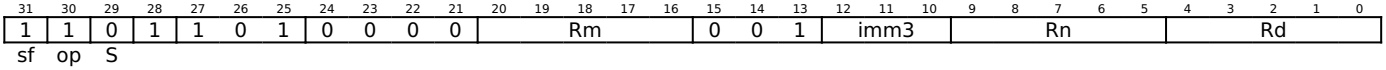
Subtract checked pointer

This instruction subtracts an optionally-shifted register value from a base address register value, and writes the result to the destination register. The optionally-shifted register value is treated as the offset.

If the operation would have generated a result where the most significant 8 bits of the result register differ from the most significant 8 bits of the base register, then the result is modified such that it is likely to be non-canonical when used as an address.

Integer

(FEAT_CPA)



Encoding

```
SUBPT <Xd|SP>, <Xn|SP>, <Xm>{, LSL #<amount>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CPA) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let shift : integer = UInt(imm3);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the general-purpose destination register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the first general-purpose source register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.
<amount>	Is the left shift amount, in the range 0 to 7, defaulting to 0, encoded in the "imm3" field.

Operation

```
var result : bits(64);
let base : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let offset : bits(64) = LSL(X{64}(m), shift);

result = base - offset;
result = PointerAddCheck(result, base);

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

2026-09_rel 2026-09-21 14:22:38

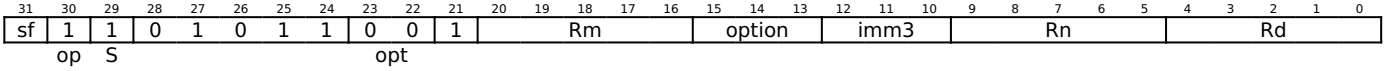
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUBS (extended register)

Subtract extended and scaled register, setting flags

This instruction subtracts a sign-extended or zero-extended register value, followed by an optional left shift amount, from a register value, and writes the result to the destination register. The argument that is extended from the <Rm> register can be a byte, halfword, word, or doubleword. It updates the condition flags based on the result.

This instruction is used by the alias CMP (extended register).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SUBS <Wd>, <Wn|WSP>, <Wm>{, <extend> {#<amount>}}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SUBS <Xd>, <Xn|SP>, <R><m>{, <extend> {#<amount>}}
```

Decode for all variants of this encoding

```
if imm3 IN {'101', '110', '111'} then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let shift : integer{} = UInt(imm3) as integer{0..4};
let datasize : integer{} = 32 << UInt(sf);
let extend_type : ExtendType = DecodeRegExtend(option);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn|WSP> Is the 32-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<extend> For the "32-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	LSL|UXTW
011	UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rn" is '11111' (WSP) and "option" is '010' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTW when "option" is '010'.

For the "64-bit" variant: is the extension to be applied to the second source operand, encoded in “option”:

option	<extend>
000	UXTB
001	UXTH
010	UXTW
011	LSL|UXTX
100	SXTB
101	SXTH
110	SXTW
111	SXTX

If "Rn" is '11111' (SP) and "option" is '011' then LSL is preferred, but may be omitted when "imm3" is '000'. In all other cases <extend> is required and must be UXTX when "option" is '011'.

<amount> Is the left shift amount to be applied after extension in the range 0 to 4, defaulting to 0, encoded in the "imm3" field. It must be absent when <extend> is absent, is required when <extend> is LSL, and is optional when <extend> is present but not LSL.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn|SP> Is the 64-bit name of the first source general-purpose register or stack pointer, encoded in the "Rn" field.

<R> Is a width specifier, encoded in “option”:

option	<R>
00x	W
010	W
x11	X
10x	W
110	W

<m> Is the number [0-30] of the second general-purpose source register or the name ZR (31), encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CMP (extended register)	`Rd == '11111'`

Operation

```
let operand1 : bits(datasize) = if n == 31 then SP{datasize}() else X{datasize}(n);
let operand2 : bits(datasize) = NOT ExtendReg{datasize}(m, extend_type, shift);

var result : bits(datasize);
var nzcv : bits(4);

(result, nzcv) = AddWithCarry{datasize}(operand1, operand2, '1');

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = nzcv;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

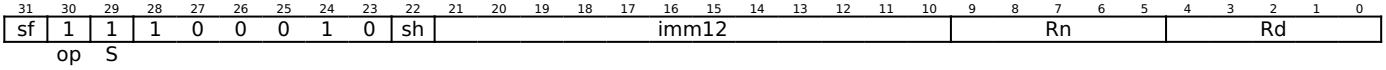
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUBS (immediate)

Subtract immediate value, setting flags

This instruction subtracts an optionally-shifted immediate value from a register value, and writes the result to the destination register. It updates the condition flags based on the result.

This instruction is used by the alias CMP (immediate).



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SUBS <Wd>, <Wn|WSP>, #<imm>{, <shift>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SUBS <Xd>, <Xn|SP>, #<imm>{, <shift>}
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);

let imm : bits(24) = if sh == '0' then Zeros{12}::imm12 else imm12::Zeros{12};
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn|WSP> Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

<imm> Is an unsigned immediate, in the range 0 to 4095, encoded in the "imm12" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #12

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn|SP> Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Alias Conditions

Alias	Is preferred when
CMP (immediate)	`Rd == '11111'`

Operation

```
let operand1 : bits(datasize) = if n == 31 then SP{datasize}() else X{datasize}(n);
let operand2 : bits(datasize) = NOT ZeroExtend{datasize}(imm);

var result : bits(datasize);
var nzcv : bits(4);

(result, nzcv) = AddWithCarry{datasize}(operand1, operand2, '1');

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = nzcv;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

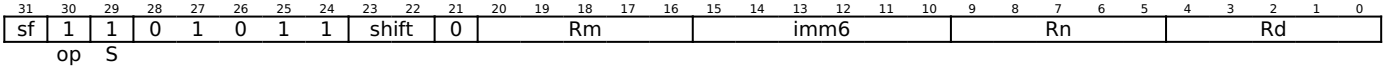
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUBS (shifted register)

Subtract optionally-shifted register, setting flags

This instruction subtracts an optionally-shifted register value, and writes the result to the destination register. It updates the condition flags based on the result.

This instruction is used by the aliases CMP (shifted register), and NEGS.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
SUBS <Wd>, <Wn>, <Wm>{, <shift> #<amount>}
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
SUBS <Xd>, <Xn>, <Xm>{, <shift> #<amount>}
```

Decode for all variants of this encoding

```
if shift == '11' then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && imm6[5] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
let shift_type : ShiftType = DecodeShift(shift);
let shift_amount : integer = UInt(imm6);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift type to be applied to the second source operand, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	RESERVED

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
CMP (shifted register)	`Rd == '11111'`
NEGS	`Rn == '11111' && Rd != '11111'`

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = NOT ShiftReg{datasize}(m, shift_type, shift_amount);

var result : bits(datasize);
var nzcv : bits(4);

(result, nzcv) = AddWithCarry{datasize}(operand1, operand2, '1');

X{datasize}(d) = result;
PSTATE.[N,Z,C,V] = nzcv;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

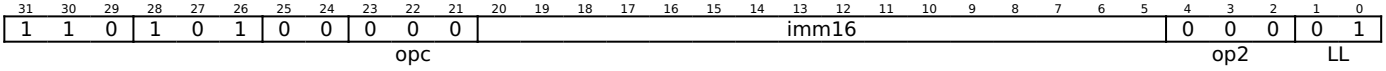
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SVC

Supervisor call

This instruction causes an exception to be taken to EL1.
On executing an `SVC` instruction, the PE records the exception as a Supervisor Call exception in *ESR_ELx*, using the EC syndrome value 0x15, and the value of the immediate argument.



Encoding

```
SVC #<imm>
```

Decode for this encoding

```
let imm : bits(16) = imm16;
```

Assembler Symbols

<imm>	Is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field.

Operation

```
AArch64_CheckForSVCTrap(imm);
AArch64_CallSupervisor(imm);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SWP, SWPA, SWPAL, SWPL

Swap word or doubleword in memory

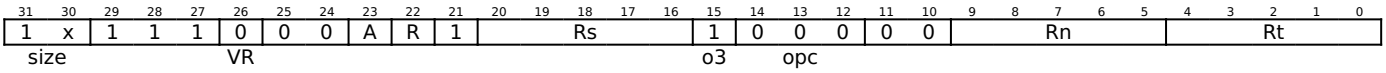
This instruction atomically loads a 32-bit word or 64-bit doubleword from a memory location, and stores the value held in a register back to the same memory location. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR or XZR, `SWPA` and `SWPAL` load from memory with acquire semantics.
- `SWPL` and `SWPAL` store to memory with release semantics.
- `SWP` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Integer

(FEAT_LSE)



Encoding for the 32-bit SWP variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
SWP <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit SWPA variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
SWPA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit SWPAL variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
SWPAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit SWPL variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
SWPL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit SWP variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
SWP <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit SWPA variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
SWPA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit SWPAL variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
SWPAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit SWPL variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
SWPL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;

let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 8 << UInt(size);
let regsize : integer{} = if datasize == 64 then 64 else 32;
let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Operation

```
var address : bits(64);
var data : bits(datasize);
var store_value : bits(datasize);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SWP, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

store_value = X{datasize}(s);

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, store_value, accdesc);

X{regsize}(t) = ZeroExtend{regsize}(data);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SWPB, SWPAB, SWPALB, SWPLB

Swap byte in memory

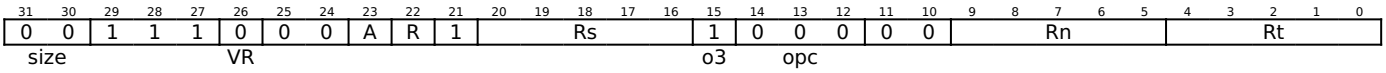
This instruction atomically loads an 8-bit byte from a memory location, and stores the value held in a register back to the same memory location. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `SWPAB` and `SWPALB` load from memory with acquire semantics.
- `SWPLB` and `SWPALB` store to memory with release semantics.
- `SWPB` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Integer

(FEAT_LSE)



Encoding for the SWPB variant

Applies when `(A == 0 && R == 0)`

```
SWPB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the SWPAB variant

Applies when `(A == 1 && R == 0)`

```
SWPAB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the SWPALB variant

Applies when `(A == 1 && R == 1)`

```
SWPALB <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the SWPLB variant

Applies when `(A == 0 && R == 1)`

```
SWPLB <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;

let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data : bits(8);
var store_value : bits(8);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SWP, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

store_value = X{8}(s);

// Irrelevant when not executing CAS
let comparevalue : bits(8) = ARBITRARY : bits(8);
data = MemAtomic{8}(address, comparevalue, store_value, accdesc);

X{32}(t) = ZeroExtend{32}(data);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SWPH, SWPAH, SWPALH, SWPLH

Swap halfword in memory

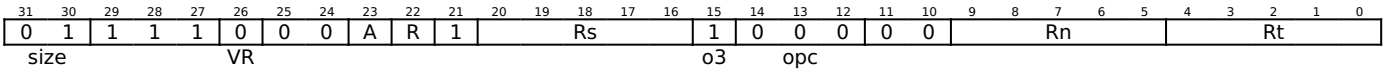
This instruction atomically loads a 16-bit halfword from a memory location, and stores the value held in a register back to the same memory location. The value initially loaded from memory is returned in the destination register.

- If the destination register is not WZR, `SWPAH` and `SWPALH` load from memory with acquire semantics.
- `SWPLH` and `SWPALH` store to memory with release semantics.
- `SWPH` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Integer

(FEAT_LSE)



Encoding for the SWPH variant

Applies when `(A == 0 && R == 0)`

```
SWPH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the SWPAH variant

Applies when `(A == 1 && R == 0)`

```
SWPAH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the SWPALH variant

Applies when `(A == 1 && R == 1)`

```
SWPALH <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the SWPLH variant

Applies when `(A == 0 && R == 1)`

```
SWPLH <Ws>, <Wt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE) then EndOfDecode(Decode_UNDEF); end;

let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
var data : bits(16);
var store_value : bits(16);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SWP, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

store_value = X{16}(s);

// Irrelevant when not executing CAS
let comparevalue : bits(16) = ARBITRARY : bits(16);
data = MemAtomic{16}(address, comparevalue, store_value, accdesc);

X{32}(t) = ZeroExtend{32}(data);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SWPP, SWPPA, SWPPAL, SWPPL

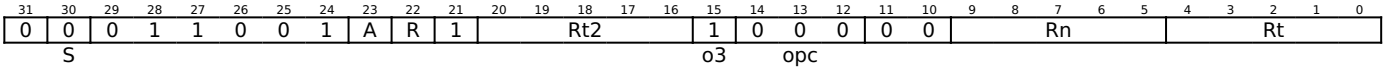
Swap quadword in memory

This instruction atomically loads a 128-bit quadword from a memory location, and stores the value held in a pair of registers back to the same memory location. The value initially loaded from memory is returned in the same pair of registers.

- SWPPA and SWPPAL load from memory with acquire semantics.
- SWPPL and SWPPAL store to memory with release semantics.
- SWPP has neither acquire nor release semantics.

Integer

(FEAT_LSE128)



Encoding for the SWPP variant

Applies when `(A == 0 && R == 0)`

```
SWPP <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the SWPPA variant

Applies when `(A == 1 && R == 0)`

```
SWPPA <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the SWPPAL variant

Applies when `(A == 1 && R == 1)`

```
SWPPAL <Xt1>, <Xt2>, [<Xn|SP>]
```

Encoding for the SWPPL variant

Applies when `(A == 0 && R == 1)`

```
SWPPL <Xt1>, <Xt2>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSE128) then EndOfDecode(Decode_UNDEF); end;
if Rt  == '11111' then EndOfDecode(Decode_UNDEF); end;
if Rt2 == '11111' then EndOfDecode(Decode_UNDEF); end;

let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;

if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LSE128OVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *128-bit Atomic Memory Operations*.

Assembler Symbols

<Xt1>	Is the 64-bit name of the first general-purpose register to be transferred, encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second general-purpose register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
var address : bits(64);
let value1 : bits(64) = X{}(t);
let value2 : bits(64) = X{}(t2);
var data : bits(128);
var store_value : bits(128);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SWP, acquire, release,
                                                       tagchecked, privileged, t, t2, t, t2);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

store_value = if BigEndian(accdesc.acctype) then value1::value2 else value2::value1;

let comparevalue : bits(128) = ARBITRARY : bits(128); // Irrelevant when not executing CAS
data = MemAtomic{128}(address, comparevalue, store_value, accdesc);

if rt_unknown then
    data = ARBITRARY : bits(128);
end;

if BigEndian(accdesc.acctype) then
    X{64}(t)  = data[127:64];
    X{64}(t2) = data[63:0];
else
    X{64}(t)  = data[63:0];
    X{64}(t2) = data[127:64];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SWPT, SWPTA, SWPTAL, SWPTL

Swap unprivileged

This instruction atomically loads a 32-bit word or 64-bit doubleword from a memory location, and stores the value held in a register back to the same memory location. The value initially loaded from memory is returned in the destination register.

- If the destination register is not one of WZR or XZR, `SWPTA` and `SWPTAL` load from memory with acquire semantics.
- `SWPTL` and `SWPTAL` store to memory with release semantics.
- `SWPT` has neither acquire nor release semantics.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

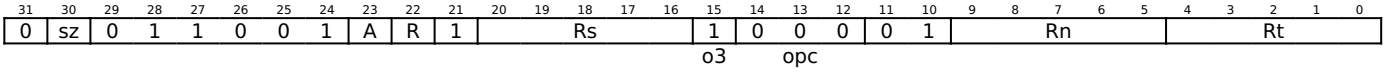
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Integer

(FEAT_LSUI)



Encoding for the 32-bit SWPT variant

Applies when `(sz == 0 && A == 0 && R == 0)`

```
SWPT <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit SWPTA variant

Applies when `(sz == 0 && A == 1 && R == 0)`

```
SWPTA <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit SWPTAL variant

Applies when `(sz == 0 && A == 1 && R == 1)`

```
SWPTAL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 32-bit SWPTL variant

Applies when `(sz == 0 && A == 0 && R == 1)`

```
SWPTL <Ws>, <Wt>, [<Xn|SP>]
```

Encoding for the 64-bit SWPT variant

Applies when `(sz == 1 && A == 0 && R == 0)`

```
SWPT <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit SWPTA variant

Applies when `(sz == 1 && A == 1 && R == 0)`

```
SWPTA <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit SWPTAL variant

Applies when `(sz == 1 && A == 1 && R == 1)`

```
SWPTAL <Xs>, <Xt>, [<Xn|SP>]
```

Encoding for the 64-bit SWPTL variant

Applies when `(sz == 1 && A == 0 && R == 1)`

```
SWPTL <Xs>, <Xt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSUI) then EndOfDecode(Decode_UNDEF); end;

let s : integer{} = UInt(Rs);
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);

let datasize : integer{} = 32 << UInt(sz);
let regsize : integer{} = if datasize == 64 then 64 else 32;
let acquire : boolean = A == '1' && Rt != '11111';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Ws>	Is the 32-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Wt>	Is the 32-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xs>	Is the 64-bit name of the general-purpose register to be stored, encoded in the "Rs" field.
<Xt>	Is the 64-bit name of the general-purpose register to be loaded, encoded in the "Rt" field.

Operation

```
var address : bits(64);
var data : bits(datasize);
var store_value : bits(datasize);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let accdesc : AccessDescriptor = CreateAccDescAtomicOp(MemAtomicOp_SWP, acquire, release,
                                                       tagchecked, privileged, t, s);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

store_value = X{datasize}(s);

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, store_value, accdesc);

X{regsize}(t) = ZeroExtend{regsize}(data);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

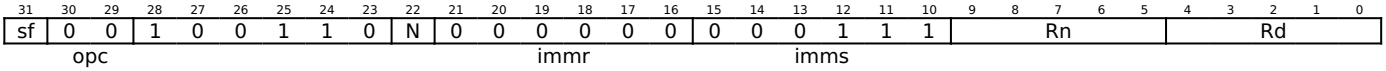
SXTB

Signed extend byte

This instruction extracts an 8-bit value from a register, sign-extends it to the size of the register, and writes the result to the destination register.

This is an alias of SBFM. This means:

- The encodings in this description are named to match the encodings of SBFM.
- The description of SBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
SXTB <Wd>, <Wn>
```

is equivalent to

```
SBFM <Wd>, <Wn>, #0, #7
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
SXTB <Xd>, <Wn>
```

is equivalent to

```
SBFM <Xd>, <Xn>, #0, #7
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

The description of SBFM gives the operational pseudocode for this instruction.

Operational Information

The description of SBFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

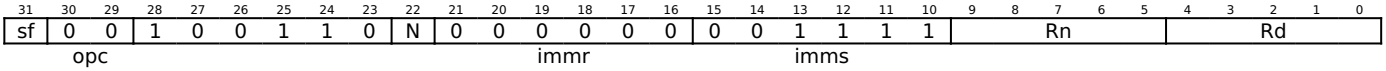
SXTH

Sign extend halfword

This instruction extracts a 16-bit value, sign-extends it to the size of the register, and writes the result to the destination register.

This is an alias of SBFM. This means:

- The encodings in this description are named to match the encodings of SBFM.
- The description of SBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
SXTH <Wd>, <Wn>
```

is equivalent to

```
SBFM <Wd>, <Wn>, #0, #15
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
SXTH <Xd>, <Wn>
```

is equivalent to

```
SBFM <Xd>, <Xn>, #0, #15
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

The description of SBFM gives the operational pseudocode for this instruction.

Operational Information

The description of SBFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

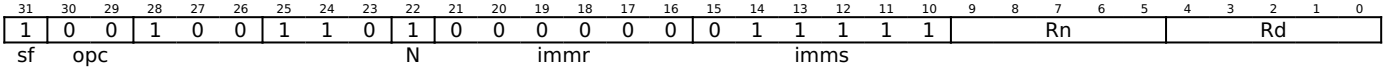
SXTW

Sign extend word

This instruction sign-extends a word to the size of the register, and writes the result to the destination register.

This is an alias of SBFM. This means:

- The encodings in this description are named to match the encodings of SBFM.
- The description of SBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

```
SXTW <Xd>, <Wn>
```

is equivalent to

```
SBFM <Xd>, <Xn>, #0, #31
```

and is always the preferred disassembly.

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of SBFM gives the operational pseudocode for this instruction.

Operational Information

The description of SBFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

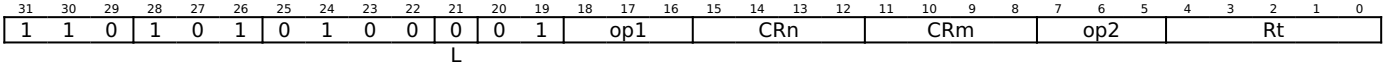
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SYS

System instruction

For more information, see *op0 == 0b01, cache maintenance, TLB maintenance, address translation, prediction restriction, BRBE, Trace Extension, Guarded Control Stack, and FEAT_RME_GPC3 instructions* for the encodings of System instructions.

This instruction is used by the aliases APAS, AT, BRB, CFP, COSP, CPP, DC, DVP, GCSPOPCX, GCSPOPX, GCSPUSHM, GCSPUSHX, GCSSS1, GIC, GSB, IC, TLBI, and TRCIT.



Encoding

```
SYS #<op1>, <Cn>, <Cm>, #<op2>{, <Xt>}
```

Decode for this encoding

```
let t : integer       = UInt(Rt);
let sys_op0 : bits(2) = '01';
let sys_op1 : bits(3) = op1;
let sys_op2 : bits(3) = op2;
let sys_crn : bits(4) = CRn;
let sys_crm : bits(4) = CRm;
```

Assembler Symbols

<op1>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op1" field.
<Cn>	Is a name 'Cn', with 'n' in the range 0 to 15, encoded in the "CRn" field.
<Cm>	Is a name 'Cm', with 'm' in the range 0 to 15, encoded in the "CRm" field.
<op2>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op2" field.
<Xt>	Is the 64-bit name of the optional general-purpose source register, defaulting to '11111', encoded in the "Rt" field.

Alias Conditions

Alias	Is preferred when
APAS	`op1 == '110' && CRn == '0111' && CRm == '0000' && op2 == '000'`
AT	`CRn == '0111' && CRm IN {'100x'} && SysOp(op1, '0111', CRm, op2) == Sys_AT`
BRB	`op1 == '001' && CRn == '0111' && CRm == '0010' && SysOp('001', '0111', '0010', op2) == Sys_BRB`
CFP	`op1 == '011' && CRn == '0111' && CRm == '0011' && op2 == '100'`
COSP	`op1 == '011' && CRn == '0111' && CRm == '0011' && op2 == '110'`
CPP	`op1 == '011' && CRn == '0111' && CRm == '0011' && op2 == '111'`
DC	`CRn == '0111' && SysOp(op1, '0111', CRm, op2) == Sys_DC`
DVP	`op1 == '011' && CRn == '0111' && CRm == '0011' && op2 == '101'`
GCSPOPCX	`op1 == '000' && CRn == '0111' && CRm == '0111' && op2 == '101'`
GCSPOPX	`op1 == '000' && CRn == '0111' && CRm == '0111' && op2 == '110'`
GCSPUSHM	`op1 == '011' && CRn == '0111' && CRm == '0111' && op2 == '000'`
GCSPUSHX	`op1 == '000' && CRn == '0111' && CRm == '0111' && op2 == '100'`
GCSSS1	`op1 == '011' && CRn == '0111' && CRm == '0111' && op2 == '010'`
GIC	`CRn == '1100' && CRm IN {'0001', '0010'} && SysOp(op1, '1100', CRm, op2) == Sys_GIC`
GSB	`op1 == '000' && CRn == '1100' && CRm == '0000' && SysOp('000', '1100', '0000', op2) == Sys_GSB`
IC	`CRn == '0111' && SysOp(op1, '0111', CRm, op2) == Sys_IC`
TLBI	`CRn IN {'100x'} && SysOp(op1, CRn, CRm, op2) == Sys_TLBI`
TRCIT	`op1 == '011' && CRn == '0111' && CRm == '0010' && op2 == '111'`

Operation

```
AArch64_SysInstr(sys_op0, sys_op1, sys_crn, sys_crm, sys_op2, t);
```

2026-09_rel 2026-09-21 14:22:38

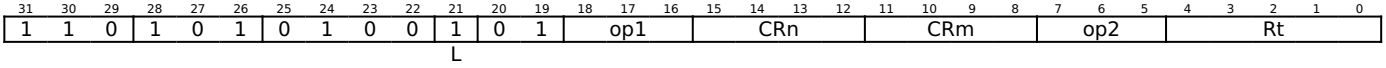
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SYSL

System instruction with result

For more information, see *op0 == 0b01, cache maintenance, TLB maintenance, address translation, prediction restriction, BRBE, Trace Extension, Guarded Control Stack, and FEAT_RME_GPC3 instructions* for the encodings of System instructions.

This instruction is used by the aliases GCSPOPM, GCSSS2, and GICR.



Encoding

```
SYSL <Xt>, #<op1>, <Cn>, <Cm>, #<op2>
```

Decode for this encoding

```
let t : integer       = UInt(Rt);
let sys_op0 : bits(2) = '01';
let sys_op1 : bits(3) = op1;
let sys_op2 : bits(3) = op2;
let sys_crn : bits(4) = CRn;
let sys_crm : bits(4) = CRm;
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rt" field.
<op1>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op1" field.
<Cn>	Is a name 'Cn', with 'n' in the range 0 to 15, encoded in the "CRn" field.
<Cm>	Is a name 'Cm', with 'm' in the range 0 to 15, encoded in the "CRm" field.
<op2>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op2" field.

Alias Conditions

Alias	Is preferred when
GCSPOPM	`op1 == '011' && CRn == '0111' && CRm == '0111' && op2 == '001'`
GCSSS2	`op1 == '011' && CRn == '0111' && CRm == '0111' && op2 == '011'`
GICR	`op1 == '000' && CRn == '1100' && CRm == '0011' && SysLOp('000', '1100', '0011', op2) == Sysl_GICR`

Operation

```
AArch64_SysInstrWithResult(sys_op0, sys_op1, sys_crn, sys_crm, sys_op2, t);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SYSP

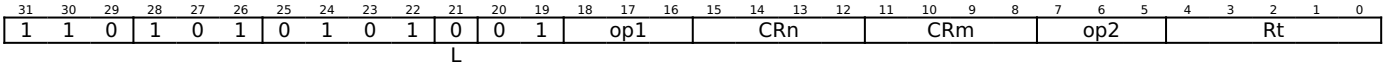
128-bit system instruction

For more information, see *op0 == 0b01, cache maintenance, TLB maintenance, address translation, prediction restriction, BRBE, Trace Extension, Guarded Control Stack, and FEAT_RME_GPC3 instructions* for the encodings of System instructions.

This instruction is used by the alias TLBIP.

System

(FEAT_SYSINSTR128)



Encoding

```
SYSP #<op1>, <Cn>, <Cm>, #<op2>{, <Xt1>, <Xt2>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SYSINSTR128) then EndOfDecode(Decode_UNDEF); end;
if Rt[0] == '1' && Rt != '11111' then EndOfDecode(Decode_UNDEF); end;

let t : integer       = UInt(Rt);
let t2 : integer      = if t == 31 then 31 else t + 1;
let sys_op0 : bits(2) = '01';
let sys_op1 : bits(3) = op1;
let sys_op2 : bits(3) = op2;
let sys_crn : bits(4) = CRn;
let sys_crm : bits(4) = CRm;
```

Assembler Symbols

<op1>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op1" field.
<Cn>	Is a name 'Cn', with 'n' in the range 0 to 15, encoded in the "CRn" field.
<Cm>	Is a name 'Cm', with 'm' in the range 0 to 15, encoded in the "CRm" field.
<op2>	Is a 3-bit unsigned immediate, in the range 0 to 7, encoded in the "op2" field.
<Xt1>	Is the 64-bit name of the first optional general-purpose source register, defaulting to '11111', encoded in the "Rt" field.
<Xt2>	Is the 64-bit name of the second optional general-purpose source register, defaulting to '11111', encoded as "Rt" +1. Defaults to '11111' if "Rt" = '11111'.

Alias Conditions

Alias	Is preferred when
TLBIP	`CRn IN {'100x'} && SysOp128(op1, CRn, CRm, op2) == Sys_TLBIP`

Operation

```
AArch64_SysInstr128(sys_op0, sys_op1, sys_crn, sys_crm, sys_op2, t, t2);
```

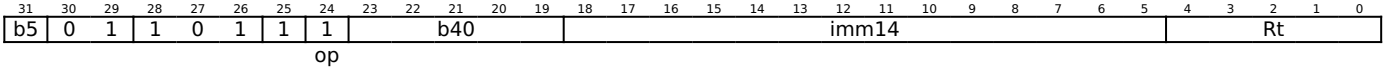
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TBNZ

Test bit and branch if nonzero

This instruction compares the value of a bit in a general-purpose register with zero, and conditionally branches to a label at a PC-relative offset if the comparison is not equal. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect condition flags.



Encoding

```
TBNZ <R><t>, #<imm>, <label>
```

Decode for this encoding

```
let t : integer = UInt(Rt);

let datasize : integer{} = 32 << UInt(b5);
let bit_pos : integer = UInt(b5::b40);
let offset : bits(64) = SignExtend{}(imm14::'00');
```

Assembler Symbols

<R> Is a width specifier, encoded in “b5”:

b5	<R>
0	W
1	X

In assembler source code an 'X' specifier is always permitted, but a 'W' specifier is permitted only when the bit number is less than 32.

<t> Is the number [0-30] of the general-purpose register to be tested or the name ZR (31), encoded in the "Rt" field.

<imm> Is the bit number to be tested, in the range 0 to 63, encoded in "b5:b40".

<label> Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range +/-32KB, is encoded as "imm14" times 4.

Operation

```
let operand : bits(datasize) = X{datasize}(t);
let branch_conditional : boolean = TRUE;
if operand[bit_pos] != '0' then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

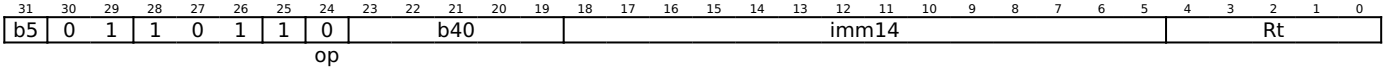
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TBZ

Test bit and branch if zero

This instruction compares the value of a test bit with zero, and conditionally branches to a label at a PC-relative offset if the comparison is equal. This instruction provides a hint that this is not a subroutine call or return. This instruction does not affect condition flags.



Encoding

```
TBZ <R><t>, #<imm>, <label>
```

Decode for this encoding

```
let t : integer = UInt(Rt);

let datasize : integer{} = 32 << UInt(b5);
let bit_pos : integer = UInt(b5::b40);
let offset : bits(64) = SignExtend{}(imm14::'00');
```

Assembler Symbols

<R> Is a width specifier, encoded in “b5”:

b5	<R>
0	W
1	X

In assembler source code an 'X' specifier is always permitted, but a 'W' specifier is permitted only when the bit number is less than 32.

<t> Is the number [0-30] of the general-purpose register to be tested or the name ZR (31), encoded in the "Rt" field.

<imm> Is the bit number to be tested, in the range 0 to 63, encoded in "b5:b40".

<label> Is the program label to be conditionally branched to. Its offset from the address of this instruction, in the range +/-32KB, is encoded as "imm14" times 4.

Operation

```
let operand : bits(datasize) = X{datasize}(t);
let branch_conditional : boolean = TRUE;
if operand[bit_pos] == '0' then
    BranchTo{64}(PC64() + offset, BranchType_DIR, branch_conditional);
else
    BranchNotTaken(BranchType_DIR, branch_conditional);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

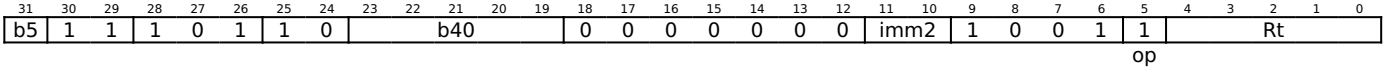
TFLTNZ

Test bit and fault if nonzero

This instruction compares the value of a bit in a general-purpose register with zero, and conditionally generates a Conditional fault exception if the comparison is not equal. This instruction does not affect condition flags.

Fault

(FEAT_CFLT)



Encoding

```
TFLTNZ #<reason>, <R><t>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CFLT) then EndOfDecode(Decode_UNDEF); end;

let datasize : integer{} = 32 << UInt(b5);
let t : integer = UInt(Rt);
let bit_pos : integer = UInt(b5::b40);
let reason : bits(16) = ZeroExtend{}(imm2);
```

Assembler Symbols

<reason> Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.

<R> Is a width specifier, encoded in “b5”:

b5	<R>
0	W
1	X

In assembler source code an 'X' specifier is always permitted, but a 'W' specifier is permitted only when the bit number is less than 32.

<t> Is the number [0-30] of the general-purpose register to be tested or the name ZR (31), encoded in the "Rt" field.

<imm> Is the bit number to be tested, in the range 0 to 63, encoded in "b5:b40".

Operation

```
CheckCFLTEnabled();

let operand : bits(datasize) = X{}(t);

if operand[bit_pos] != '0' then
    let isflt : boolean = FALSE;
    AArch64_ConditionalFault(isflt, reason);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

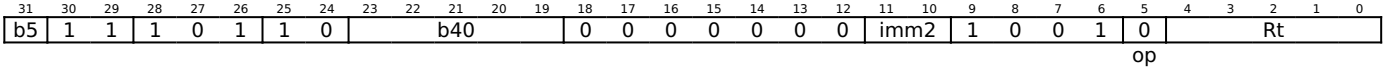
TFLTZ

Test bit and fault if zero

This instruction compares the value of a bit in a general-purpose register with zero, and conditionally generates a Conditional fault exception if the comparison is equal. This instruction does not affect condition flags.

Fault

(FEAT_CFLT)



Encoding

```
TFLTZ #<reason>, <R><t>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_CFLT) then EndOfDecode(Decode_UNDEF); end;

let datasize : integer{} = 32 << UInt(b5);
let t : integer = UInt(Rt);
let bit_pos : integer = UInt(b5::b40);
let reason : bits(16) = ZeroExtend{}(imm2);
```

Assembler Symbols

<reason> Is a 2-bit unsigned immediate, in the range 0 to 3, encoded in the "imm2" field.

<R> Is a width specifier, encoded in “b5”:

b5	<R>
0	W
1	X

In assembler source code an 'X' specifier is always permitted, but a 'W' specifier is permitted only when the bit number is less than 32.

<t> Is the number [0-30] of the general-purpose register to be tested or the name ZR (31), encoded in the "Rt" field.

<imm> Is the bit number to be tested, in the range 0 to 63, encoded in "b5:b40".

Operation

```
CheckCFLTEnabled();

let operand : bits(datasize) = X{}(t);

if operand[bit_pos] == '0' then
    let isflt : boolean = FALSE;
    AArch64_ConditionalFault(isflt, reason);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

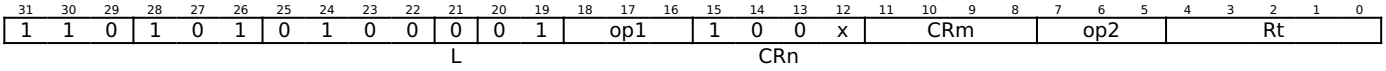
TLBI

TLB invalidate operation

For more information, see *op0 == 0b01, cache maintenance, TLB maintenance, address translation, prediction restriction, BRBE, Trace Extension, Guarded Control Stack, and FEAT_RME_GPC3 instructions*.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

```
TLBI <tlbi_op>{, <Xt>}
```

is equivalent to

```
SYS #<op1>, <Cn>, <Cm>, #<op2>{, <Xt>}
```

and is the preferred disassembly when `SysOp(op1, CRn, CRm, op2) == Sys_TLBI`.

<tlbi_op> Is a TLBI operation name, as listed for the TLBI system instruction group, encoded in “(op1 :: CRn :: CRm :: op2)”:

op1	CRn	CRm	op2	<tlbi_op>	Architectural Feature	Description
000	1000	0001	000	VMALLE1OS	FEAT_TLBIOS	For more information, see TLBI VMALLE1OS, TLBI VMALLE1OSNXS.
000	1000	0001	001	VAE1OS	FEAT_TLBIOS	For more information, see TLBI VAE1OS, TLBI VAE1OSNXS.
000	1000	0001	010	ASIDE1OS	FEAT_TLBIOS	For more information, see TLBI ASIDE1OS, TLBI ASIDE1OSNXS.
000	1000	0001	011	VAAE1OS	FEAT_TLBIOS	For more information, see TLBI VAAE1OS, TLBI VAAE1OSNXS.
000	1000	0001	101	VALE1OS	FEAT_TLBIOS	For more information, see TLBI VALE1OS, TLBI VALE1OSNXS.
000	1000	0001	111	VAALE1OS	FEAT_TLBIOS	For more information, see TLBI VAALE1OS, TLBI VAALE1OSNXS.
000	1000	0010	001	RVAE1IS	FEAT_TLBIRANGE	For more information, see TLBI RVAE1IS, TLBI RVAE1ISNXS.
000	1000	0010	011	RVAAE1IS	FEAT_TLBIRANGE	For more information, see TLBI RVAAE1IS, TLBI RVAAE1ISNXS.
000	1000	0010	101	RVALE1IS	FEAT_TLBIRANGE	For more information, see TLBI RVALE1IS, TLBI RVALE1ISNXS.
000	1000	0010	111	RVAALE1IS	FEAT_TLBIRANGE	For more information, see TLBI RVAALE1IS, TLBI RVAALE1ISNXS.
000	1000	0011	000	VMALLE1IS	-	For more information, see TLBI VMALLE1IS, TLBI VMALLE1ISNXS.
000	1000	0011	001	VAE1IS	-	For more information, see TLBI VAE1IS, TLBI VAE1ISNXS.
000	1000	0011	010	ASIDE1IS	-	For more information, see TLBI ASIDE1IS, TLBI ASIDE1ISNXS.
000	1000	0011	011	VAAE1IS	-	For more information, see TLBI VAAE1IS, TLBI VAAE1ISNXS.
000	1000	0011	101	VALE1IS	-	For more information, see TLBI VALE1IS, TLBI VALE1ISNXS.
000	1000	0011	111	VAALE1IS	-	For more information, see TLBI VAALE1IS, TLBI VAALE1ISNXS.
000	1000	0101	001	RVAE1OS	FEAT_TLBIRANGE	For more information, see TLBI RVAE1OS, TLBI RVAE1OSNXS.
000	1000	0101	011	RVAAE1OS	FEAT_TLBIRANGE	For more information, see TLBI RVAAE1OS, TLBI RVAAE1OSNXS.
000	1000	0101	101	RVALE1OS	FEAT_TLBIRANGE	For more information, see TLBI RVALE1OS, TLBI RVALE1OSNXS.
000	1000	0101	111	RVAALE1OS	FEAT_TLBIRANGE	For more information, see TLBI RVAALE1OS, TLBI RVAALE1OSNXS.
000	1000	0110	001	RVAE1	FEAT_TLBIRANGE	For more information, see TLBI RVAE1, TLBI RVAE1NXS.
000	1000	0110	011	RVAAE1	FEAT_TLBIRANGE	For more information, see TLBI RVAAE1, TLBI RVAAE1NXS.
000	1000	0110	101	RVALE1	FEAT_TLBIRANGE	For more information, see TLBI RVALE1, TLBI RVALE1NXS.

op1	CRn	CRm	op2	<tlbi_op>	Architectural Feature	Description
000	1000	0110	111	RVAALE1	FEAT_TLBIRANGE	For more information, see TLBI RVAALE1, TLBI RVAALE1NXS.
000	1000	0111	000	VMALLE1	-	For more information, see TLBI VMALLE1, TLBI VMALLE1NXS.
000	1000	0111	001	VAE1	-	For more information, see TLBI VAE1, TLBI VAE1NXS.
000	1000	0111	010	ASIDE1	-	For more information, see TLBI ASIDE1, TLBI ASIDE1NXS.
000	1000	0111	011	VAAE1	-	For more information, see TLBI VAAE1, TLBI VAAE1NXS.
000	1000	0111	101	VALE1	-	For more information, see TLBI VALE1, TLBI VALE1NXS.
000	1000	0111	111	VAALE1	-	For more information, see TLBI VAALE1, TLBI VAALE1NXS.
000	1001	0001	000	VMALLE1OSNXS	FEAT_XS	For more information, see TLBI VMALLE1OS, TLBI VMALLE1OSNXS.
000	1001	0001	001	VAE1OSNXS	FEAT_XS	For more information, see TLBI VAE1OS, TLBI VAE1OSNXS.
000	1001	0001	010	ASIDE1OSNXS	FEAT_XS	For more information, see TLBI ASIDE1OS, TLBI ASIDE1OSNXS.
000	1001	0001	011	VAAE1OSNXS	FEAT_XS	For more information, see TLBI VAAE1OS, TLBI VAAE1OSNXS.
000	1001	0001	101	VALE1OSNXS	FEAT_XS	For more information, see TLBI VALE1OS, TLBI VALE1OSNXS.
000	1001	0001	111	VAALE1OSNXS	FEAT_XS	For more information, see TLBI VAALE1OS, TLBI VAALE1OSNXS.
000	1001	0010	001	RVAE1ISNXS	FEAT_XS	For more information, see TLBI RVAE1IS, TLBI RVAE1ISNXS.
000	1001	0010	011	RVAAE1ISNXS	FEAT_XS	For more information, see TLBI RVAAE1IS, TLBI RVAAE1ISNXS.
000	1001	0010	101	RVALE1ISNXS	FEAT_XS	For more information, see TLBI RVALE1IS, TLBI RVALE1ISNXS.
000	1001	0010	111	RVAALE1ISNXS	FEAT_XS	For more information, see TLBI RVAALE1IS, TLBI RVAALE1ISNXS.
000	1001	0011	000	VMALLE1ISNXS	FEAT_XS	For more information, see TLBI VMALLE1IS, TLBI VMALLE1ISNXS.
000	1001	0011	001	VAE1ISNXS	FEAT_XS	For more information, see TLBI VAE1IS, TLBI VAE1ISNXS.
000	1001	0011	010	ASIDE1ISNXS	FEAT_XS	For more information, see TLBI ASIDE1IS, TLBI ASIDE1ISNXS.
000	1001	0011	011	VAAE1ISNXS	FEAT_XS	For more information, see TLBI VAAE1IS, TLBI VAAE1ISNXS.
000	1001	0011	101	VALE1ISNXS	FEAT_XS	For more information, see TLBI VALE1IS, TLBI VALE1ISNXS.
000	1001	0011	111	VAALE1ISNXS	FEAT_XS	For more information, see TLBI VAALE1IS, TLBI VAALE1ISNXS.

op1	CRn	CRm	op2	<tlbi_op>	Architectural Feature	Description
000	1001	0101	001	RVAE1OSNXS	FEAT_XS	For more information, see TLBI RVAE1OS, TLBI RVAE1OSNXS.
000	1001	0101	011	RVAAE1OSNXS	FEAT_XS	For more information, see TLBI RVAAE1OS, TLBI RVAAE1OSNXS.
000	1001	0101	101	RVALE1OSNXS	FEAT_XS	For more information, see TLBI RVALE1OS, TLBI RVALE1OSNXS.
000	1001	0101	111	RVAALE1OSNXS	FEAT_XS	For more information, see TLBI RVAALE1OS, TLBI RVAALE1OSNXS.
000	1001	0110	001	RVAE1NXS	FEAT_XS	For more information, see TLBI RVAE1, TLBI RVAE1NXS.
000	1001	0110	011	RVAAE1NXS	FEAT_XS	For more information, see TLBI RVAAE1, TLBI RVAAE1NXS.
000	1001	0110	101	RVALE1NXS	FEAT_XS	For more information, see TLBI RVALE1, TLBI RVALE1NXS.
000	1001	0110	111	RVAALE1NXS	FEAT_XS	For more information, see TLBI RVAALE1, TLBI RVAALE1NXS.
000	1001	0111	000	VMALLE1NXS	FEAT_XS	For more information, see TLBI VMALLE1, TLBI VMALLE1NXS.
000	1001	0111	001	VAE1NXS	FEAT_XS	For more information, see TLBI VAE1, TLBI VAE1NXS.
000	1001	0111	010	ASIDE1NXS	FEAT_XS	For more information, see TLBI ASIDE1, TLBI ASIDE1NXS.
000	1001	0111	011	VAAE1NXS	FEAT_XS	For more information, see TLBI VAAE1, TLBI VAAE1NXS.
000	1001	0111	101	VALE1NXS	FEAT_XS	For more information, see TLBI VALE1, TLBI VALE1NXS.
000	1001	0111	111	VAALE1NXS	FEAT_XS	For more information, see TLBI VAALE1, TLBI VAALE1NXS.
100	1000	0000	001	IPAS2E1IS	-	For more information, see TLBI IPAS2E1IS, TLBI IPAS2E1ISNXS.
100	1000	0000	010	RIPAS2E1IS	FEAT_TLBIRANGE	For more information, see TLBI RIPAS2E1IS, TLBI RIPAS2E1ISNXS.
100	1000	0000	101	IPAS2LE1IS	-	For more information, see TLBI IPAS2LE1IS, TLBI IPAS2LE1ISNXS.
100	1000	0000	110	RIPAS2LE1IS	FEAT_TLBIRANGE	For more information, see TLBI RIPAS2LE1IS, TLBI RIPAS2LE1ISNXS.
100	1000	0001	000	ALLE2OS	FEAT_TLBIOS	For more information, see TLBI ALLE2OS, TLBI ALLE2OSNXS.
100	1000	0001	001	VAE2OS	FEAT_TLBIOS	For more information, see TLBI VAE2OS, TLBI VAE2OSNXS.
100	1000	0001	100	ALLE1OS	FEAT_TLBIOS	For more information, see TLBI ALLE1OS, TLBI ALLE1OSNXS.
100	1000	0001	101	VALE2OS	FEAT_TLBIOS	For more information, see TLBI VALE2OS, TLBI VALE2OSNXS.

op1	CRn	CRm	op2	<tlbi_op>	Architectural Feature	Description
100	1000	0001	110	VMALLS12E1OS	FEAT_TLBIOS	For more information, see TLBI VMALLS12E1OS, TLBI VMALLS12E1OSNXS.
100	1000	0010	001	RVAE2IS	FEAT_TLBIRANGE	For more information, see TLBI RVAE2IS, TLBI RVAE2ISNXS.
100	1000	0010	010	VMALLWS2E1IS	FEAT_TLBIW	For more information, see TLBI VMALLWS2E1IS, TLBI VMALLWS2E1ISNXS.
100	1000	0010	101	RVALE2IS	FEAT_TLBIRANGE	For more information, see TLBI RVALE2IS, TLBI RVALE2ISNXS.
100	1000	0011	000	ALLE2IS	-	For more information, see TLBI ALLE2IS, TLBI ALLE2ISNXS.
100	1000	0011	001	VAE2IS	-	For more information, see TLBI VAE2IS, TLBI VAE2ISNXS.
100	1000	0011	100	ALLE1IS	-	For more information, see TLBI ALLE1IS, TLBI ALLE1ISNXS.
100	1000	0011	101	VALE2IS	-	For more information, see TLBI VALE2IS, TLBI VALE2ISNXS.
100	1000	0011	110	VMALLS12E1IS	-	For more information, see TLBI VMALLS12E1IS, TLBI VMALLS12E1ISNXS.
100	1000	0100	000	IPAS2E1OS	FEAT_TLBIOS	For more information, see TLBI IPAS2E1OS, TLBI IPAS2E1OSNXS.
100	1000	0100	001	IPAS2E1	-	For more information, see TLBI IPAS2E1, TLBI IPAS2E1NXS.
100	1000	0100	010	RIPAS2E1	FEAT_TLBIRANGE	For more information, see TLBI RIPAS2E1, TLBI RIPAS2E1NXS.
100	1000	0100	011	RIPAS2E1OS	FEAT_TLBIRANGE	For more information, see TLBI RIPAS2E1OS, TLBI RIPAS2E1OSNXS.
100	1000	0100	100	IPAS2LE1OS	FEAT_TLBIOS	For more information, see TLBI IPAS2LE1OS, TLBI IPAS2LE1OSNXS.
100	1000	0100	101	IPAS2LE1	-	For more information, see TLBI IPAS2LE1, TLBI IPAS2LE1NXS.
100	1000	0100	110	RIPAS2LE1	FEAT_TLBIRANGE	For more information, see TLBI RIPAS2LE1, TLBI RIPAS2LE1NXS.
100	1000	0100	111	RIPAS2LE1OS	FEAT_TLBIRANGE	For more information, see TLBI RIPAS2LE1OS, TLBI RIPAS2LE1OSNXS.
100	1000	0101	001	RVAE2OS	FEAT_TLBIRANGE	For more information, see TLBI RVAE2OS, TLBI RVAE2OSNXS.
100	1000	0101	010	VMALLWS2E1OS	FEAT_TLBIW	For more information, see TLBI VMALLWS2E1OS, TLBI VMALLWS2E1OSNXS.
100	1000	0101	101	RVALE2OS	FEAT_TLBIRANGE	For more information, see TLBI RVALE2OS, TLBI RVALE2OSNXS.
100	1000	0110	001	RVAE2	FEAT_TLBIRANGE	For more information, see TLBI RVAE2, TLBI RVAE2NXS.

op1	CRn	CRm	op2	<tlbi_op>	Architectural Feature	Description
100	1000	0110	010	VMALLWS2E1	FEAT_TLBIW	For more information, see TLBI VMALLWS2E1, TLBI VMALLWS2E1NXS.
100	1000	0110	101	RVALE2	FEAT_TLBIRANGE	For more information, see TLBI RVALE2, TLBI RVALE2NXS.
100	1000	0111	000	ALLE2	-	For more information, see TLBI ALLE2, TLBI ALLE2NXS.
100	1000	0111	001	VAE2	-	For more information, see TLBI VAE2, TLBI VAE2NXS.
100	1000	0111	100	ALLE1	-	For more information, see TLBI ALLE1, TLBI ALLE1NXS.
100	1000	0111	101	VALE2	-	For more information, see TLBI VALE2, TLBI VALE2NXS.
100	1000	0111	110	VMALLS12E1	-	For more information, see TLBI VMALLS12E1, TLBI VMALLS12E1NXS.
100	1001	0000	001	IPAS2E1ISNXS	FEAT_XS	For more information, see TLBI IPAS2E1IS, TLBI IPAS2E1ISNXS.
100	1001	0000	010	RIPAS2E1ISNXS	FEAT_XS	For more information, see TLBI RIPAS2E1IS, TLBI RIPAS2E1ISNXS.
100	1001	0000	101	IPAS2LE1ISNXS	FEAT_XS	For more information, see TLBI IPAS2LE1IS, TLBI IPAS2LE1ISNXS.
100	1001	0000	110	RIPAS2LE1ISNXS	FEAT_XS	For more information, see TLBI RIPAS2LE1IS, TLBI RIPAS2LE1ISNXS.
100	1001	0001	000	ALLE2OSNXS	FEAT_XS	For more information, see TLBI ALLE2OS, TLBI ALLE2OSNXS.
100	1001	0001	001	VAE2OSNXS	FEAT_XS	For more information, see TLBI VAE2OS, TLBI VAE2OSNXS.
100	1001	0001	100	ALLE1OSNXS	FEAT_XS	For more information, see TLBI ALLE1OS, TLBI ALLE1OSNXS.
100	1001	0001	101	VALE2OSNXS	FEAT_XS	For more information, see TLBI VALE2OS, TLBI VALE2OSNXS.
100	1001	0001	110	VMALLS12E1OSNXS	FEAT_XS	For more information, see TLBI VMALLS12E1OS, TLBI VMALLS12E1OSNXS.
100	1001	0010	001	RVAE2ISNXS	FEAT_XS	For more information, see TLBI RVAE2IS, TLBI RVAE2ISNXS.
100	1001	0010	010	VMALLWS2E1ISNXS	FEAT_TLBIW	For more information, see TLBI VMALLWS2E1IS, TLBI VMALLWS2E1ISNXS.
100	1001	0010	101	RVALE2ISNXS	FEAT_XS	For more information, see TLBI RVALE2IS, TLBI RVALE2ISNXS.
100	1001	0011	000	ALLE2ISNXS	FEAT_XS	For more information, see TLBI ALLE2IS, TLBI ALLE2ISNXS.
100	1001	0011	001	VAE2ISNXS	FEAT_XS	For more information, see TLBI VAE2IS, TLBI VAE2ISNXS.

op1	CRn	CRm	op2	<tlbi_op>	Architectural Feature	Description
100	1001	0011	100	ALLE1ISNXS	FEAT_XS	For more information, see TLBI ALLE1IS, TLBI ALLE1ISNXS.
100	1001	0011	101	VALE2ISNXS	FEAT_XS	For more information, see TLBI VALE2IS, TLBI VALE2ISNXS.
100	1001	0011	110	VMALLS12E1ISNXS	FEAT_XS	For more information, see TLBI VMALLS12E1IS, TLBI VMALLS12E1ISNXS.
100	1001	0100	000	IPAS2E1OSNXS	FEAT_XS	For more information, see TLBI IPAS2E1OS, TLBI IPAS2E1OSNXS.
100	1001	0100	001	IPAS2E1NXS	FEAT_XS	For more information, see TLBI IPAS2E1, TLBI IPAS2E1NXS.
100	1001	0100	010	RIPAS2E1NXS	FEAT_XS	For more information, see TLBI RIPAS2E1, TLBI RIPAS2E1NXS.
100	1001	0100	011	RIPAS2E1OSNXS	FEAT_XS	For more information, see TLBI RIPAS2E1OS, TLBI RIPAS2E1OSNXS.
100	1001	0100	100	IPAS2LE1OSNXS	FEAT_XS	For more information, see TLBI IPAS2LE1OS, TLBI IPAS2LE1OSNXS.
100	1001	0100	101	IPAS2LE1NXS	FEAT_XS	For more information, see TLBI IPAS2LE1, TLBI IPAS2LE1NXS.
100	1001	0100	110	RIPAS2LE1NXS	FEAT_XS	For more information, see TLBI RIPAS2LE1, TLBI RIPAS2LE1NXS.
100	1001	0100	111	RIPAS2LE1OSNXS	FEAT_XS	For more information, see TLBI RIPAS2LE1OS, TLBI RIPAS2LE1OSNXS.
100	1001	0101	001	RVAE2OSNXS	FEAT_XS	For more information, see TLBI RVAE2OS, TLBI RVAE2OSNXS.
100	1001	0101	010	VMALLWS2E1OSNXS	FEAT_TLBIW	For more information, see TLBI VMALLWS2E1OS, TLBI VMALLWS2E1OSNXS.
100	1001	0101	101	RVALE2OSNXS	FEAT_XS	For more information, see TLBI RVALE2OS, TLBI RVALE2OSNXS.
100	1001	0110	001	RVAE2NXS	FEAT_XS	For more information, see TLBI RVAE2, TLBI RVAE2NXS.
100	1001	0110	010	VMALLWS2E1NXS	FEAT_TLBIW	For more information, see TLBI VMALLWS2E1, TLBI VMALLWS2E1NXS.
100	1001	0110	101	RVALE2NXS	FEAT_XS	For more information, see TLBI RVALE2, TLBI RVALE2NXS.
100	1001	0111	000	ALLE2NXS	FEAT_XS	For more information, see TLBI ALLE2, TLBI ALLE2NXS.
100	1001	0111	001	VAE2NXS	FEAT_XS	For more information, see TLBI VAE2, TLBI VAE2NXS.
100	1001	0111	100	ALLE1NXS	FEAT_XS	For more information, see TLBI ALLE1, TLBI ALLE1NXS.
100	1001	0111	101	VALE2NXS	FEAT_XS	For more information, see TLBI VALE2, TLBI VALE2NXS.

op1	CRn	CRm	op2	<tlbi_op>	Architectural Feature	Description
100	1001	0111	110	VMALLS12E1NXS	FEAT_XS	For more information, see TLBI VMALLS12E1, TLBI VMALLS12E1NXS.
110	1000	0001	000	ALLE3OS	FEAT_TLBIOS	For more information, see TLBI ALLE3OS, TLBI ALLE3OSNXS.
110	1000	0001	001	VAE3OS	FEAT_TLBIOS	For more information, see TLBI VAE3OS, TLBI VAE3OSNXS.
110	1000	0001	100	PAALLOS	FEAT_RME	For more information, see TLBI PAALLOS.
110	1000	0001	101	VALE3OS	FEAT_TLBIOS	For more information, see TLBI VALE3OS, TLBI VALE3OSNXS.
110	1000	0010	001	RVAE3IS	FEAT_TLBIRANGE	For more information, see TLBI RVAE3IS, TLBI RVAE3ISNXS.
110	1000	0010	101	RVALE3IS	FEAT_TLBIRANGE	For more information, see TLBI RVALE3IS, TLBI RVALE3ISNXS.
110	1000	0011	000	ALLE3IS	-	For more information, see TLBI ALLE3IS, TLBI ALLE3ISNXS.
110	1000	0011	001	VAE3IS	-	For more information, see TLBI VAE3IS, TLBI VAE3ISNXS.
110	1000	0011	101	VALE3IS	-	For more information, see TLBI VALE3IS, TLBI VALE3ISNXS.
110	1000	0100	011	RPAOS	FEAT_RME	For more information, see TLBI RPAOS.
110	1000	0100	111	RPALOS	FEAT_RME	For more information, see TLBI RPALOS.
110	1000	0101	001	RVAE3OS	FEAT_TLBIRANGE	For more information, see TLBI RVAE3OS, TLBI RVAE3OSNXS.
110	1000	0101	101	RVALE3OS	FEAT_TLBIRANGE	For more information, see TLBI RVALE3OS, TLBI RVALE3OSNXS.
110	1000	0110	001	RVAE3	FEAT_TLBIRANGE	For more information, see TLBI RVAE3, TLBI RVAE3NXS.
110	1000	0110	101	RVALE3	FEAT_TLBIRANGE	For more information, see TLBI RVALE3, TLBI RVALE3NXS.
110	1000	0111	000	ALLE3	-	For more information, see TLBI ALLE3, TLBI ALLE3NXS.
110	1000	0111	001	VAE3	-	For more information, see TLBI VAE3, TLBI VAE3NXS.
110	1000	0111	100	PAALL	FEAT_RME	For more information, see TLBI PAALL.
110	1000	0111	101	VALE3	-	For more information, see TLBI VALE3, TLBI VALE3NXS.
110	1001	0001	000	ALLE3OSNXS	FEAT_XS	For more information, see TLBI ALLE3OS, TLBI ALLE3OSNXS.
110	1001	0001	001	VAE3OSNXS	FEAT_XS	For more information, see TLBI VAE3OS, TLBI VAE3OSNXS.
110	1001	0001	101	VALE3OSNXS	FEAT_XS	For more information, see TLBI VALE3OS, TLBI VALE3OSNXS.
110	1001	0010	001	RVAE3ISNXS	FEAT_XS	For more information, see TLBI RVAE3IS, TLBI RVAE3ISNXS.

op1	CRn	CRm	op2	<tlbi_op>	Architectural Feature	Description
110	1001	0010	101	RVALE3ISNXS	FEAT_XS	For more information, see TLBI RVALE3IS, TLBI RVALE3ISNXS.
110	1001	0011	000	ALLE3ISNXS	FEAT_XS	For more information, see TLBI ALLE3IS, TLBI ALLE3ISNXS.
110	1001	0011	001	VAE3ISNXS	FEAT_XS	For more information, see TLBI VAE3IS, TLBI VAE3ISNXS.
110	1001	0011	101	VALE3ISNXS	FEAT_XS	For more information, see TLBI VALE3IS, TLBI VALE3ISNXS.
110	1001	0101	001	RVAE3OSNXS	FEAT_XS	For more information, see TLBI RVAE3OS, TLBI RVAE3OSNXS.
110	1001	0101	101	RVALE3OSNXS	FEAT_XS	For more information, see TLBI RVALE3OS, TLBI RVALE3OSNXS.
110	1001	0110	001	RVAE3NXS	FEAT_XS	For more information, see TLBI RVAE3, TLBI RVAE3NXS.
110	1001	0110	101	RVALE3NXS	FEAT_XS	For more information, see TLBI RVALE3, TLBI RVALE3NXS.
110	1001	0111	000	ALLE3NXS	FEAT_XS	For more information, see TLBI ALLE3, TLBI ALLE3NXS.
110	1001	0111	001	VAE3NXS	FEAT_XS	For more information, see TLBI VAE3, TLBI VAE3NXS.
110	1001	0111	101	VALE3NXS	FEAT_XS	For more information, see TLBI VALE3, TLBI VALE3NXS.

<Xt> Is the 64-bit name of the optional general-purpose source register, defaulting to '11111', encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TLBIP

TLB invalidate pair operation

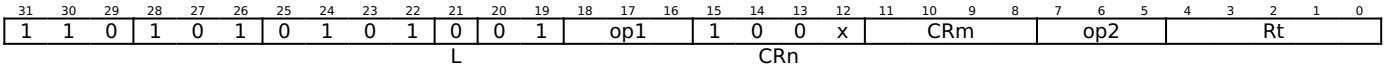
For more information, see *op0 == 0b01, cache maintenance, TLB maintenance, address translation, prediction restriction, BRBE, Trace Extension, Guarded Control Stack, and FEAT_RME_GPC3 instructions*.

This is an alias of SYSP. This means:

- The encodings in this description are named to match the encodings of SYSP.
- The description of SYSP gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_SYSINSTR128)



Encoding

```
TLBIP <tlbip_op>{, <Xt1>, <Xt2>}
```

is equivalent to

```
SYSP #<op1>, <Cn>, <Cm>, #<op2>{, <Xt1>, <Xt2>}
```

and is the preferred disassembly when `SysOp128(op1, CRn, CRm, op2) == Sys_TLBIP`.

<tlbip_op> Is a TLBIP operation name, as listed for the TLBIP system pair instruction group, encoded in “(op1 :: CRn :: CRm :: op2)”:

op1	CRn	CRm	op2	<tlbip_op>	Architectural Feature	Description
000	1000	0001	001	VAE1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAE1OS, TLBIP VAE1OSNXS.
000	1000	0001	011	VAAE1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAAE1OS, TLBIP VAAE1OSNXS.
000	1000	0001	101	VALE1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VALE1OS, TLBIP VALE1OSNXS.
000	1000	0001	111	VAALE1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAALE1OS, TLBIP VAALE1OSNXS.
000	1000	0010	001	RVAE1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAE1IS, TLBIP RVAE1ISNXS.
000	1000	0010	011	RVAAE1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAAE1IS, TLBIP RVAAE1ISNXS.
000	1000	0010	101	RVALE1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVALE1IS, TLBIP RVALE1ISNXS.
000	1000	0010	111	RVAALE1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAALE1IS, TLBIP RVAALE1ISNXS.
000	1000	0011	001	VAE1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAE1IS, TLBIP VAE1ISNXS.
000	1000	0011	011	VAAE1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAAE1IS, TLBIP VAAE1ISNXS.
000	1000	0011	101	VALE1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VALE1IS, TLBIP VALE1ISNXS.
000	1000	0011	111	VAALE1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAALE1IS, TLBIP VAALE1ISNXS.
000	1000	0101	001	RVAE1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAE1OS, TLBIP RVAE1OSNXS.
000	1000	0101	011	RVAAE1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAAE1OS, TLBIP RVAAE1OSNXS.
000	1000	0101	101	RVALE1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVALE1OS, TLBIP RVALE1OSNXS.
000	1000	0101	111	RVAALE1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAALE1OS, TLBIP RVAALE1OSNXS.
000	1000	0110	001	RVAE1	FEAT_D128	For more information, see TLBIP RVAE1, TLBIP RVAE1NXS.
000	1000	0110	011	RVAAE1	FEAT_D128	For more information, see TLBIP RVAAE1, TLBIP RVAAE1NXS.
000	1000	0110	101	RVALE1	FEAT_D128	For more information, see TLBIP RVALE1, TLBIP RVALE1NXS.
000	1000	0110	111	RVAALE1	FEAT_D128	For more information, see TLBIP RVAALE1, TLBIP RVAALE1NXS.

op1	CRn	CRm	op2	<tlbip_op>	Architectural Feature	Description
000	1000	0111	001	VAE1	FEAT_D128	For more information, see TLBIP VAE1, TLBIP VAE1NXS.
000	1000	0111	011	VAAE1	FEAT_D128	For more information, see TLBIP VAAE1, TLBIP VAAE1NXS.
000	1000	0111	101	VALE1	FEAT_D128	For more information, see TLBIP VALE1, TLBIP VALE1NXS.
000	1000	0111	111	VAALE1	FEAT_D128	For more information, see TLBIP VAALE1, TLBIP VAALE1NXS.
000	1001	0001	001	VAE1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAE1OS, TLBIP VAE1OSNXS.
000	1001	0001	011	VAAE1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAAE1OS, TLBIP VAAE1OSNXS.
000	1001	0001	101	VALE1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VALE1OS, TLBIP VALE1OSNXS.
000	1001	0001	111	VAALE1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAALE1OS, TLBIP VAALE1OSNXS.
000	1001	0010	001	RVAE1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAE1IS, TLBIP RVAE1ISNXS.
000	1001	0010	011	RVAAE1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAAE1IS, TLBIP RVAAE1ISNXS.
000	1001	0010	101	RVALE1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVALE1IS, TLBIP RVALE1ISNXS.
000	1001	0010	111	RVAALE1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAALE1IS, TLBIP RVAALE1ISNXS.
000	1001	0011	001	VAE1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAE1IS, TLBIP VAE1ISNXS.
000	1001	0011	011	VAAE1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAAE1IS, TLBIP VAAE1ISNXS.
000	1001	0011	101	VALE1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VALE1IS, TLBIP VALE1ISNXS.
000	1001	0011	111	VAALE1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAALE1IS, TLBIP VAALE1ISNXS.
000	1001	0101	001	RVAE1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAE1OS, TLBIP RVAE1OSNXS.
000	1001	0101	011	RVAAE1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAAE1OS, TLBIP RVAAE1OSNXS.
000	1001	0101	101	RVALE1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVALE1OS, TLBIP RVALE1OSNXS.
000	1001	0101	111	RVAALE1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAALE1OS, TLBIP RVAALE1OSNXS.

op1	CRn	CRm	op2	<tlbip_op>	Architectural Feature	Description
000	1001	0110	001	RVAE1NXS	FEAT_D128	For more information, see TLBIP RVAE1, TLBIP RVAE1NXS.
000	1001	0110	011	RVAAE1NXS	FEAT_D128	For more information, see TLBIP RVAAE1, TLBIP RVAAE1NXS.
000	1001	0110	101	RVALE1NXS	FEAT_D128	For more information, see TLBIP RVALE1, TLBIP RVALE1NXS.
000	1001	0110	111	RVAALE1NXS	FEAT_D128	For more information, see TLBIP RVAALE1, TLBIP RVAALE1NXS.
000	1001	0111	001	VAE1NXS	FEAT_D128	For more information, see TLBIP VAE1, TLBIP VAE1NXS.
000	1001	0111	011	VAAE1NXS	FEAT_D128	For more information, see TLBIP VAAE1, TLBIP VAAE1NXS.
000	1001	0111	101	VALE1NXS	FEAT_D128	For more information, see TLBIP VALE1, TLBIP VALE1NXS.
000	1001	0111	111	VAALE1NXS	FEAT_D128	For more information, see TLBIP VAALE1, TLBIP VAALE1NXS.
100	1000	0000	001	IPAS2E1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP IPAS2E1IS, TLBIP IPAS2E1ISNXS.
100	1000	0000	010	RIPAS2E1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RIPAS2E1IS, TLBIP RIPAS2E1ISNXS.
100	1000	0000	101	IPAS2LE1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP IPAS2LE1IS, TLBIP IPAS2LE1ISNXS.
100	1000	0000	110	RIPAS2LE1IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RIPAS2LE1IS, TLBIP RIPAS2LE1ISNXS.
100	1000	0001	001	VAE2OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAE2OS, TLBIP VAE2OSNXS.
100	1000	0001	101	VALE2OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VALE2OS, TLBIP VALE2OSNXS.
100	1000	0010	001	RVAE2IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAE2IS, TLBIP RVAE2ISNXS.
100	1000	0010	101	RVALE2IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVALE2IS, TLBIP RVALE2ISNXS.
100	1000	0011	001	VAE2IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAE2IS, TLBIP VAE2ISNXS.
100	1000	0011	101	VALE2IS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VALE2IS, TLBIP VALE2ISNXS.
100	1000	0100	000	IPAS2E1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP IPAS2E1OS, TLBIP IPAS2E1OSNXS.
100	1000	0100	001	IPAS2E1	FEAT_D128	For more information, see TLBIP IPAS2E1, TLBIP IPAS2E1NXS.
100	1000	0100	010	RIPAS2E1	FEAT_D128	For more information, see TLBIP RIPAS2E1, TLBIP RIPAS2E1NXS.

op1	CRn	CRm	op2	<tlbip_op>	Architectural Feature	Description
100	1000	0100	011	RIPAS2E1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RIPAS2E1OS, TLBIP RIPAS2E1OSNXS.
100	1000	0100	100	IPAS2LE1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP IPAS2LE1OS, TLBIP IPAS2LE1OSNXS.
100	1000	0100	101	IPAS2LE1	FEAT_D128	For more information, see TLBIP IPAS2LE1, TLBIP IPAS2LE1NXS.
100	1000	0100	110	RIPAS2LE1	FEAT_D128	For more information, see TLBIP RIPAS2LE1, TLBIP RIPAS2LE1NXS.
100	1000	0100	111	RIPAS2LE1OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RIPAS2LE1OS, TLBIP RIPAS2LE1OSNXS.
100	1000	0101	001	RVAE2OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAE2OS, TLBIP RVAE2OSNXS.
100	1000	0101	101	RVALE2OS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVALE2OS, TLBIP RVALE2OSNXS.
100	1000	0110	001	RVAE2	FEAT_D128	For more information, see TLBIP RVAE2, TLBIP RVAE2NXS.
100	1000	0110	101	RVALE2	FEAT_D128	For more information, see TLBIP RVALE2, TLBIP RVALE2NXS.
100	1000	0111	001	VAE2	FEAT_D128	For more information, see TLBIP VAE2, TLBIP VAE2NXS.
100	1000	0111	101	VALE2	FEAT_D128	For more information, see TLBIP VALE2, TLBIP VALE2NXS.
100	1001	0000	001	IPAS2E1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP IPAS2E1IS, TLBIP IPAS2E1ISNXS.
100	1001	0000	010	RIPAS2E1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RIPAS2E1IS, TLBIP RIPAS2E1ISNXS.
100	1001	0000	101	IPAS2LE1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP IPAS2LE1IS, TLBIP IPAS2LE1ISNXS.
100	1001	0000	110	RIPAS2LE1ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RIPAS2LE1IS, TLBIP RIPAS2LE1ISNXS.
100	1001	0001	001	VAE2OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAE2OS, TLBIP VAE2OSNXS.
100	1001	0001	101	VALE2OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VALE2OS, TLBIP VALE2OSNXS.
100	1001	0010	001	RVAE2ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAE2IS, TLBIP RVAE2ISNXS.
100	1001	0010	101	RVALE2ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVALE2IS, TLBIP RVALE2ISNXS.
100	1001	0011	001	VAE2ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VAE2IS, TLBIP VAE2ISNXS.

op1	CRn	CRm	op2	<tlbip_op>	Architectural Feature	Description
100	1001	0011	101	VALE2ISNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP VALE2IS, TLBIP VALE2ISNXS.
100	1001	0100	000	IPAS2E1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP IPAS2E1OS, TLBIP IPAS2E1OSNXS.
100	1001	0100	001	IPAS2E1NXS	FEAT_D128	For more information, see TLBIP IPAS2E1, TLBIP IPAS2E1NXS.
100	1001	0100	010	RIPAS2E1NXS	FEAT_D128	For more information, see TLBIP RIPAS2E1, TLBIP RIPAS2E1NXS.
100	1001	0100	011	RIPAS2E1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RIPAS2E1OS, TLBIP RIPAS2E1OSNXS.
100	1001	0100	100	IPAS2LE1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP IPAS2LE1OS, TLBIP IPAS2LE1OSNXS.
100	1001	0100	101	IPAS2LE1NXS	FEAT_D128	For more information, see TLBIP IPAS2LE1, TLBIP IPAS2LE1NXS.
100	1001	0100	110	RIPAS2LE1NXS	FEAT_D128	For more information, see TLBIP RIPAS2LE1, TLBIP RIPAS2LE1NXS.
100	1001	0100	111	RIPAS2LE1OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RIPAS2LE1OS, TLBIP RIPAS2LE1OSNXS.
100	1001	0101	001	RVAE2OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVAE2OS, TLBIP RVAE2OSNXS.
100	1001	0101	101	RVALE2OSNXS	FEAT_D128 || FEAT_TLBID	For more information, see TLBIP RVALE2OS, TLBIP RVALE2OSNXS.
100	1001	0110	001	RVAE2NXS	FEAT_D128	For more information, see TLBIP RVAE2, TLBIP RVAE2NXS.
100	1001	0110	101	RVALE2NXS	FEAT_D128	For more information, see TLBIP RVALE2, TLBIP RVALE2NXS.
100	1001	0111	001	VAE2NXS	FEAT_D128	For more information, see TLBIP VAE2, TLBIP VAE2NXS.
100	1001	0111	101	VALE2NXS	FEAT_D128	For more information, see TLBIP VALE2, TLBIP VALE2NXS.
110	1000	0001	001	VAE3OS	FEAT_D128	For more information, see TLBIP VAE3OS, TLBIP VAE3OSNXS.
110	1000	0001	101	VALE3OS	FEAT_D128	For more information, see TLBIP VALE3OS, TLBIP VALE3OSNXS.
110	1000	0010	001	RVAE3IS	FEAT_D128	For more information, see TLBIP RVAE3IS, TLBIP RVAE3ISNXS.
110	1000	0010	101	RVALE3IS	FEAT_D128	For more information, see TLBIP RVALE3IS, TLBIP RVALE3ISNXS.
110	1000	0011	001	VAE3IS	FEAT_D128	For more information, see TLBIP VAE3IS, TLBIP VAE3ISNXS.
110	1000	0011	101	VALE3IS	FEAT_D128	For more information, see TLBIP VALE3IS, TLBIP VALE3ISNXS.

op1	CRn	CRm	op2	<tlbip_op>	Architectural Feature	Description
110	1000	0101	001	RVAE3OS	FEAT_D128	For more information, see TLBIP RVAE3OS, TLBIP RVAE3OSNXS.
110	1000	0101	101	RVALE3OS	FEAT_D128	For more information, see TLBIP RVALE3OS, TLBIP RVALE3OSNXS.
110	1000	0110	001	RVAE3	FEAT_D128	For more information, see TLBIP RVAE3, TLBIP RVAE3NXS.
110	1000	0110	101	RVALE3	FEAT_D128	For more information, see TLBIP RVALE3, TLBIP RVALE3NXS.
110	1000	0111	001	VAE3	FEAT_D128	For more information, see TLBIP VAE3, TLBIP VAE3NXS.
110	1000	0111	101	VALE3	FEAT_D128	For more information, see TLBIP VALE3, TLBIP VALE3NXS.
110	1001	0001	001	VAE3OSNXS	FEAT_D128	For more information, see TLBIP VAE3OS, TLBIP VAE3OSNXS.
110	1001	0001	101	VALE3OSNXS	FEAT_D128	For more information, see TLBIP VALE3OS, TLBIP VALE3OSNXS.
110	1001	0010	001	RVAE3ISNXS	FEAT_D128	For more information, see TLBIP RVAE3IS, TLBIP RVAE3ISNXS.
110	1001	0010	101	RVALE3ISNXS	FEAT_D128	For more information, see TLBIP RVALE3IS, TLBIP RVALE3ISNXS.
110	1001	0011	001	VAE3ISNXS	FEAT_D128	For more information, see TLBIP VAE3IS, TLBIP VAE3ISNXS.
110	1001	0011	101	VALE3ISNXS	FEAT_D128	For more information, see TLBIP VALE3IS, TLBIP VALE3ISNXS.
110	1001	0101	001	RVAE3OSNXS	FEAT_D128	For more information, see TLBIP RVAE3OS, TLBIP RVAE3OSNXS.
110	1001	0101	101	RVALE3OSNXS	FEAT_D128	For more information, see TLBIP RVALE3OS, TLBIP RVALE3OSNXS.
110	1001	0110	001	RVAE3NXS	FEAT_D128	For more information, see TLBIP RVAE3, TLBIP RVAE3NXS.
110	1001	0110	101	RVALE3NXS	FEAT_D128	For more information, see TLBIP RVALE3, TLBIP RVALE3NXS.
110	1001	0111	001	VAE3NXS	FEAT_D128	For more information, see TLBIP VAE3, TLBIP VAE3NXS.
110	1001	0111	101	VALE3NXS	FEAT_D128	For more information, see TLBIP VALE3, TLBIP VALE3NXS.

<Xt1> Is the 64-bit name of the first optional general-purpose source register, defaulting to '11111', encoded in the "Rt" field.

<Xt2> Is the 64-bit name of the second optional general-purpose source register, defaulting to '11111', encoded as "Rt" +1. Defaults to '11111' if "Rt" = '11111'.

Operation

The description of SYSP gives the operational pseudocode for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TRCIT

Trace instrumentation

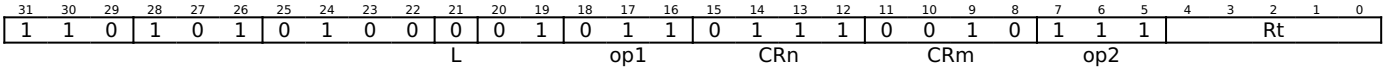
This instruction generates an instrumentation trace packet that contains the value of the provided register.
For more information, see TRCIT, Trace Instrumentation.

This is an alias of SYS. This means:

- The encodings in this description are named to match the encodings of SYS.
- The description of SYS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

System

(FEAT_ITE)



Encoding

```
TRCIT <Xt>
```

is equivalent to

```
SYS #3, C7, C2, #7, <Xt>
```

and is always the preferred disassembly.

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rt" field.

Operation

The description of SYS gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TSB

Trace synchronization barrier

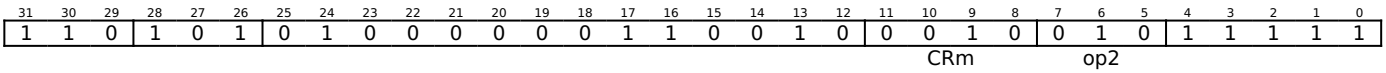
This instruction is a barrier that synchronizes the trace operations of instructions, see *Trace Synchronization Barrier (TSB)*. For information about the synchronization behavior when a `DSB` instruction follows a `TSB` instruction in program order, see:

- *Trace synchronization and memory barriers*.
- *Synchronization in self-hosted trace*.

If *FEAT_TRF* is not implemented, this instruction executes as a `NOP`.

System

(FEAT_TRF)



Encoding

```
TSB CSYNC
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_TRF) then EndOfDecode(Decode_NOP); end;
```

Operation

```
CheckTSBTrap();

TraceSynchronizationBarrier();
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

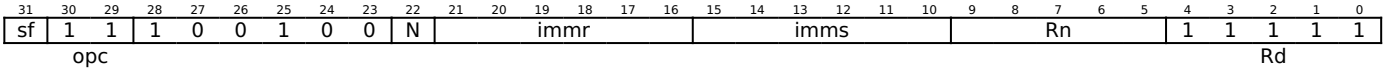
TST (immediate)

Test bits (immediate)

This instruction performs a bitwise AND between a register value and an immediate value, and discards the results. It updates the condition flags based on the result.

This is an alias of ANDS (immediate). This means:

- The encodings in this description are named to match the encodings of ANDS (immediate).
- The description of ANDS (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
TST <Wn>, #<imm>
```

is equivalent to

```
ANDS WZR, <Wn>, #<imm>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
TST <Xn>, #<imm>
```

is equivalent to

```
ANDS XZR, <Xn>, #<imm>
```

and is always the preferred disassembly.

Assembler Symbols

<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<imm>	Is the bitmask immediate. For encoding information, see *Logical (immediate)*.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of ANDS (immediate) gives the operational pseudocode for this instruction.

Operational Information

The description of ANDS (immediate) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

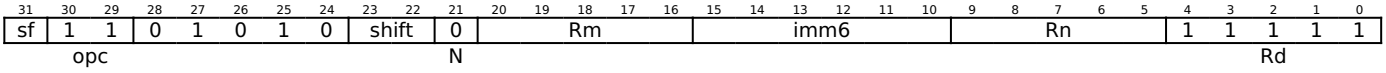
TST (shifted register)

Test (shifted register)

This instruction performs a bitwise AND between a register value and an optionally-shifted register value. It updates the condition flags based on the result, and discards the result.

This is an alias of ANDS (shifted register). This means:

- The encodings in this description are named to match the encodings of ANDS (shifted register).
- The description of ANDS (shifted register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
TST <Wn>, <Wm>{, <shift> #<amount>}
```

is equivalent to

```
ANDS WZR, <Wn>, <Wm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
TST <Xn>, <Xm>{, <shift> #<amount>}
```

is equivalent to

```
ANDS XZR, <Xn>, <Xm>{, <shift> #<amount>}
```

and is always the preferred disassembly.

Assembler Symbols

<Wn> Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Wm> Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.

<shift> Is the optional shift to be applied to the final source, defaulting to LSL and encoded in “shift”:

shift	<shift>
00	LSL
01	LSR
10	ASR
11	ROR

<amount> For the "32-bit" variant: is the shift amount, in the range 0 to 31, defaulting to 0 and encoded in the "imm6" field.
For the "64-bit" variant: is the shift amount, in the range 0 to 63, defaulting to 0 and encoded in the "imm6" field.

<Xn> Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

The description of ANDS (shifted register) gives the operational pseudocode for this instruction.

Operational Information

The description of ANDS (shifted register) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

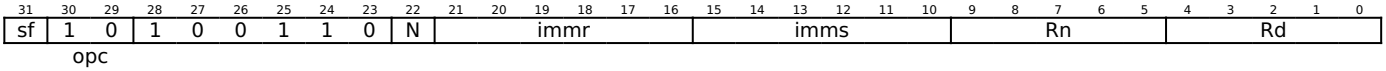
UBFIZ

Unsigned bitfield insert in zeros

This instruction copies a bitfield of <width> bits from the least significant bits of the source register to bit position <lsb> of the destination register, setting the destination bits above and below the bitfield to zero.

This is an alias of UBFM. This means:

- The encodings in this description are named to match the encodings of UBFM.
- The description of UBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
UBFIZ <Wd>, <Wn>, #<lsb>, #<width>
```

is equivalent to

```
UBFM <Wd>, <Wn>, #(-<lsb> MOD 32), #(<width>-1)
```

and is the preferred disassembly when `UInt(imms) < UInt(immr)`.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
UBFIZ <Xd>, <Xn>, #<lsb>, #<width>
```

is equivalent to

```
UBFM <Xd>, <Xn>, #(-<lsb> MOD 64), #(<width>-1)
```

and is the preferred disassembly when `UInt(imms) < UInt(immr)`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<lsb>	For the "32-bit" variant: is the bit number of the lsb of the destination bitfield, in the range 0 to 31. For the "64-bit" variant: is the bit number of the lsb of the destination bitfield, in the range 0 to 63.
<width>	For the "32-bit" variant: is the width of the bitfield, in the range 1 to 32-<lsb>. For the "64-bit" variant: is the width of the bitfield, in the range 1 to 64-<lsb>.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of UBFM gives the operational pseudocode for this instruction.

Operational Information

The description of UBFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UBFM

Unsigned bitfield move

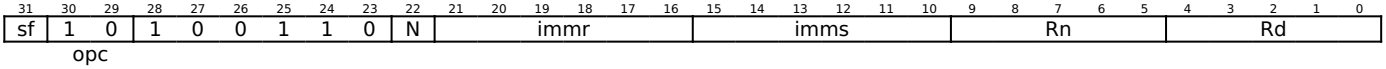
This instruction is usually accessed using one of its aliases, which are always preferred for disassembly.

If <imms> is greater than or equal to <immr>, this copies a bitfield of (<imms>-<immr>+1) bits starting from bit position <immr> in the source register to the least significant bits of the destination register.

If <imms> is less than <immr>, this copies a bitfield of (<imms>+1) bits from the least significant bits of the source register to bit position (regsize-<immr>) of the destination register, where regsize is the destination register size of 32 or 64 bits.

In both cases, the destination bits below and above the bitfield are set to zero.

This instruction is used by the aliases LSL (immediate), LSR (immediate), UBFIZ, UBFX, UXTB, and UXTH.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
UBFM <Wd>, <Wn>, #<immr>, #<imms>
```

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
UBFM <Xd>, <Xn>, #<immr>, #<imms>
```

Decode for all variants of this encoding

```
if sf == '1' && N != '1' then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && (N != '0' || immr[5] != '0' || imms[5] != '0') then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let r : integer = UInt(immr);

var wmask : bits(datasize);
var tmask : bits(datasize);
(wmask, tmask) = DecodeBitMasks{datasize}(N, imms, immr, FALSE);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<immr>	For the "32-bit" variant: is the right rotate amount, in the range 0 to 31, encoded in the "immr" field. For the "64-bit" variant: is the right rotate amount, in the range 0 to 63, encoded in the "immr" field.
<imms>	For the "32-bit" variant: is the leftmost bit number to be moved from the source, in the range 0 to 31, encoded in the "imms" field. For the "64-bit" variant: is the leftmost bit number to be moved from the source, in the range 0 to 63, encoded in the "imms" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Alias Conditions

Alias	Of variant	Is preferred when
LSL (immediate)	32-bit	`imms != '011111' && UInt(imms) + 1 == UInt(immr)`
LSL (immediate)	64-bit	`imms != '111111' && UInt(imms) + 1 == UInt(immr)`
LSR (immediate)	32-bit	`imms == '011111'`
LSR (immediate)	64-bit	`imms == '111111'`
UBFIZ		`UInt(imms) < UInt(immr)`
UBFX		`BFXPreferred(sf, opc[1], imms, immr)`
UXTB		`immr == '000000' && imms == '000111'`
UXTH		`immr == '000000' && imms == '001111'`

Operation

```
let src : bits(datasize) = X{}(n);

// Perform bitfield move on low bits
let bot : bits(datasize) = ROR(src, r) AND wmask;

// Combine extension bits and result bits
X{datasize}(d) = bot AND tmask;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

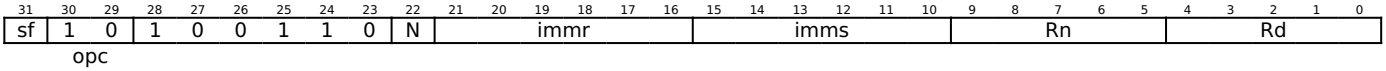
UBFX

Unsigned bitfield extract

This instruction copies a bitfield of <width> bits starting from bit position <lsb> in the source register to the least significant bits of the destination register, and sets destination bits above the bitfield to zero.

This is an alias of UBFM. This means:

- The encodings in this description are named to match the encodings of UBFM.
- The description of UBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding for the 32-bit variant

Applies when `(sf == 0 && N == 0)`

```
UBFX <Wd>, <Wn>, #<lsb>, #<width>
```

is equivalent to

```
UBFM <Wd>, <Wn>, #<lsb>, #(<lsb>+<width>-1)
```

and is the preferred disassembly when `BFXPreferred(sf, opc[1], imms, immr)`.

Encoding for the 64-bit variant

Applies when `(sf == 1 && N == 1)`

```
UBFX <Xd>, <Xn>, #<lsb>, #<width>
```

is equivalent to

```
UBFM <Xd>, <Xn>, #<lsb>, #(<lsb>+<width>-1)
```

and is the preferred disassembly when `BFXPreferred(sf, opc[1], imms, immr)`.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<lsb>	For the "32-bit" variant: is the bit number of the lsb of the source bitfield, in the range 0 to 31. For the "64-bit" variant: is the bit number of the lsb of the source bitfield, in the range 0 to 63.
<width>	For the "32-bit" variant: is the width of the bitfield, in the range 1 to 32-<lsb>. For the "64-bit" variant: is the width of the bitfield, in the range 1 to 64-<lsb>.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of UBFM gives the operational pseudocode for this instruction.

Operational Information

The description of UBFM gives the operational information for this instruction.

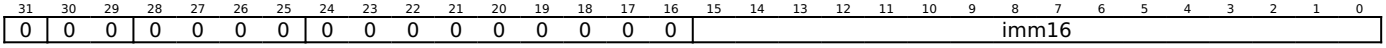
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDF

Permanently undefined

This instruction generates an Undefined Instruction exception (*ESR_ELx*.EC = 0b000000). The encodings for `UDF` used in this section are defined as permanently UNDEFINED.



Encoding

```
UDF #<imm>
```

Decode for this encoding

```
// The imm16 field is ignored by hardware.
EndOfDecode(Decode_UNDEF);
```

Assembler Symbols

<imm>	is a 16-bit unsigned immediate, in the range 0 to 65535, encoded in the "imm16" field. The PE ignores the value of this constant.

Operation

```
// No operation.
```

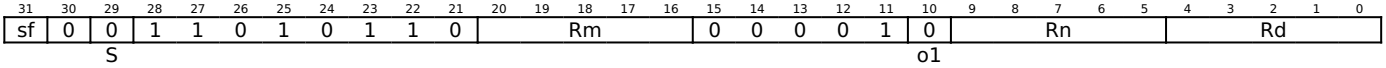
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDIV (quotient)

Unsigned divide

This instruction divides the first unsigned source register value by the second unsigned source register value, and writes the result to the destination register. Dividing by zero writes the value zero to the destination register. The condition flags are not affected.



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
UDIV <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
UDIV <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);
let dividend : integer = UInt(operand1);
let divisor : integer = UInt(operand2);

var result : integer;
if divisor == 0 then
    result = 0;
else
    result = dividend DIVRM divisor;
end;

X{datasize}(d) = result[datasize-1:0];
```

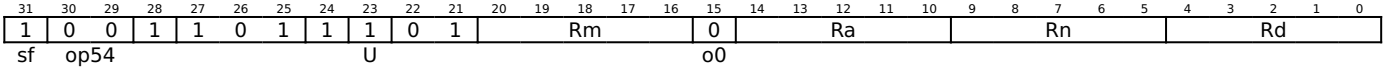
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMADDL

Unsigned multiply-add long

This instruction multiplies two 32-bit register values, adds a 64-bit register value, and writes the result to the 64-bit destination register.
This instruction is used by the alias UMULL (32-bit).



Encoding

```
UMADDL <Xd>, <Wn>, <Wm>, <Xa>
```

Decode for this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Xa>	Is the 64-bit name of the third general-purpose source register holding the addend, encoded in the "Ra" field.

Alias Conditions

Alias	Is preferred when
UMULL (32-bit)	`Ra == '11111'`

Operation

```
let operand1 : bits(32) = X{32}(n);
let operand2 : bits(32) = X{32}(m);
let operand3 : bits(64) = X{64}(a);

let result : integer = UInt(operand3) + UInt(operand1) * UInt(operand2);

X{64}(d) = result[63:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

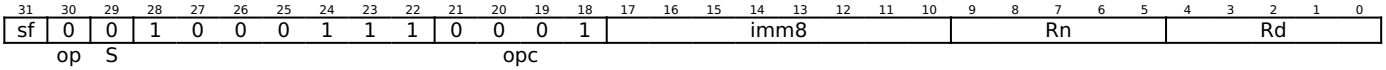
UMAX (immediate)

Unsigned maximum (immediate)

This instruction determines the unsigned maximum of the source register value and immediate, and writes the result to the destination register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
UMAX <Wd>, <Wn>, #<uimm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
UMAX <Xd>, <Xn>, #<uimm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let imm : integer = UInt(imm8);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<uimm>	Is an unsigned immediate, in the range 0 to 255, encoded in the "imm8" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let result : integer = Max(UInt(operand1), imm);

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

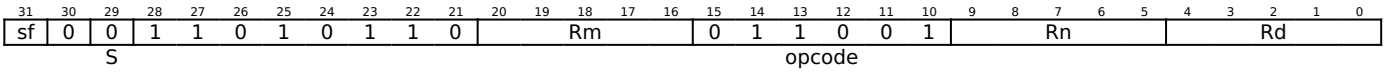
UMAX (register)

Unsigned maximum (register)

This instruction determines the unsigned maximum of the two source register values and writes the result to the destination register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
UMAX <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
UMAX <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);
let result : integer = Max(UInt(operand1), UInt(operand2));

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

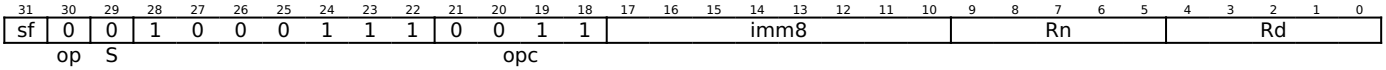
UMIN (immediate)

Unsigned minimum (immediate)

This instruction determines the unsigned minimum of the source register value and immediate, and writes the result to the destination register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
UMIN <Wd>, <Wn>, #<uimm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
UMIN <Xd>, <Xn>, #<uimm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 32 << UInt(sf);
let imm : integer = UInt(imm8);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<uimm>	Is an unsigned immediate, in the range 0 to 255, encoded in the "imm8" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let result : integer = Min(UInt(operand1), imm);

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

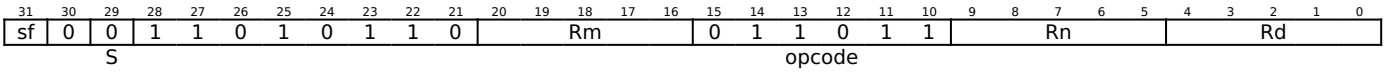
UMIN (register)

Unsigned minimum (register)

This instruction determines the unsigned minimum of the two source register values and writes the result to the destination register.

Integer

(FEAT_CSSC)



Encoding for the 32-bit variant

Applies when `(sf == 0)`

```
UMIN <Wd>, <Wn>, <Wm>
```

Encoding for the 64-bit variant

Applies when `(sf == 1)`

```
UMIN <Xd>, <Xn>, <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_CSSC) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 32 << UInt(sf);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register, encoded in the "Rm" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register, encoded in the "Rm" field.

Operation

```
let operand1 : bits(datasize) = X{datasize}(n);
let operand2 : bits(datasize) = X{datasize}(m);
let result : integer = Min(UInt(operand1), UInt(operand2));

X{datasize}(d) = result[datasize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

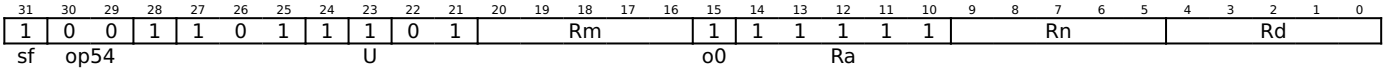
UMNEGL

Unsigned multiply-negate long

This instruction multiplies two 32-bit register values, negates the product, and writes the result to the 64-bit destination register.

This is an alias of UMSUBL. This means:

- The encodings in this description are named to match the encodings of UMSUBL.
- The description of UMSUBL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

`UMNEGL <Xd>, <Wn>, <Wm>`

is equivalent to

`UMSUBL <Xd>, <Wn>, <Wm>, XZR`

and is always the preferred disassembly.

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation

The description of UMSUBL gives the operational pseudocode for this instruction.

Operational Information

The description of UMSUBL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

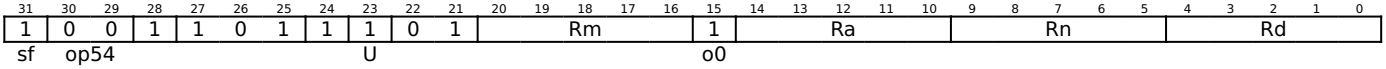
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMSUBL

Unsigned multiply-subtract long

This instruction multiplies two 32-bit register values, subtracts the product from a 64-bit register value, and writes the result to the 64-bit destination register.

This instruction is used by the alias UMNEGL.



Encoding

```
UMSUBL <Xd>, <Wn>, <Wm>, <Xa>
```

Decode for this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.
<Xa>	Is the 64-bit name of the third general-purpose source register holding the minuend, encoded in the "Ra" field.

Alias Conditions

Alias	Is preferred when
UMNEGL	`Ra == '11111'`

Operation

```
let operand1 : bits(32) = X{32}(n);
let operand2 : bits(32) = X{32}(m);
let operand3 : bits(64) = X{64}(a);

let result : integer = UInt(operand3) - UInt(operand1) * UInt(operand2);

X{64}(d) = result[63:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

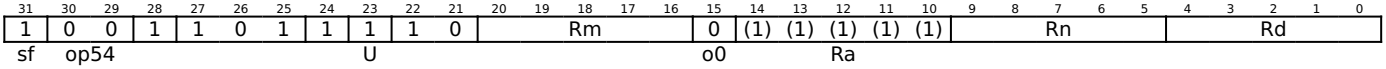
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMULH

Unsigned multiply high

This instruction multiplies two 64-bit register values, and writes bits[127:64] of the 128-bit result to the 64-bit destination register.



Encoding

```
UMULH <Xd>, <Xn>, <Xm>
```

Decode for this encoding

```
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation

```
let operand1 : bits(64) = X{64}(n);
let operand2 : bits(64) = X{64}(m);

let result : integer = UInt(operand1) * UInt(operand2);

X{64}(d) = result[127:64];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

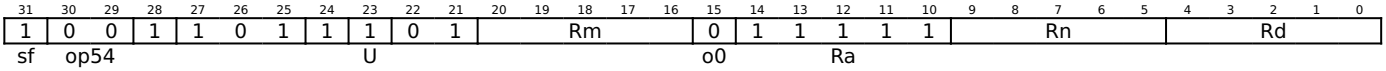
UMULL (32-bit)

Unsigned multiply long

This instruction multiplies two 32-bit register values, and writes the result to the 64-bit destination register.

This is an alias of UMADDL. This means:

- The encodings in this description are named to match the encodings of UMADDL.
- The description of UMADDL gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

`UMULL` <Xd>, <Wn>, <Wm>

is equivalent to

UMADDL <Xd>, <Wn>, <Wm>, `XZR`

and is always the preferred disassembly.

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the first general-purpose source register holding the multiplicand, encoded in the "Rn" field.
<Wm>	Is the 32-bit name of the second general-purpose source register holding the multiplier, encoded in the "Rm" field.

Operation

The description of UMADDL gives the operational pseudocode for this instruction.

Operational Information

The description of UMADDL gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

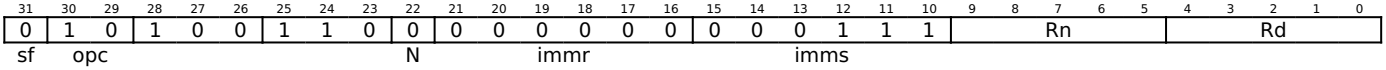
UXTB

Unsigned extend byte

This instruction extracts an 8-bit value from a register, zero-extends it to the size of the register, and writes the result to the destination register.

This is an alias of UBFM. This means:

- The encodings in this description are named to match the encodings of UBFM.
- The description of UBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

```
UXTB <Wd>, <Wn>
```

is equivalent to

```
UBFM <Wd>, <Wn>, #0, #7
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of UBFM gives the operational pseudocode for this instruction.

Operational Information

The description of UBFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

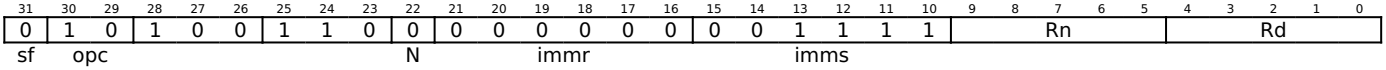
UXTH

Unsigned extend halfword

This instruction extracts a 16-bit value from a register, zero-extends it to the size of the register, and writes the result to the destination register.

This is an alias of UBFM. This means:

- The encodings in this description are named to match the encodings of UBFM.
- The description of UBFM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.



Encoding

```
UXTH <Wd>, <Wn>
```

is equivalent to

```
UBFM <Wd>, <Wn>, #0, #15
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

The description of UBFM gives the operational pseudocode for this instruction.

Operational Information

The description of UBFM gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

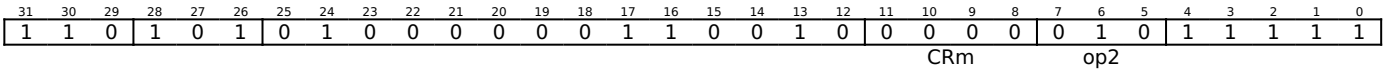
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WFE

Wait for event

This instruction is a hint instruction that indicates that the PE can enter a low-power state and remain there until a wakeup event occurs. Wakeup events include the event signaled as a result of executing the `SEV` instruction on any PE in the multiprocessor system. For more information, see *Wait For Event mechanism and Send event*.

As described in *Wait For Event mechanism and Send event*, the execution of a `WFE` instruction that would otherwise cause entry to a low-power state can be trapped to a higher Exception level.



Encoding

```
WFE
```

Decode for this encoding

```
// Empty.
```

Operation

```
Hint_WFE();
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WFET

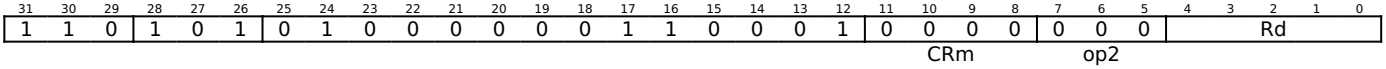
Wait for event with timeout

This instruction provides a hint that the PE can enter a low-power state and remain there until either a local timeout event or a wakeup event occurs. Wakeup events include the event signaled as a result of executing the `SEV` instruction on any PE in the multiprocessor system. For more information, see *Wait For Event mechanism and Send event*.

As described in *Wait For Event mechanism and Send event*, the execution of a `WFET` instruction that would otherwise cause entry to a low-power state can be trapped to a higher Exception level.

System

(FEAT_WFxT)



Encoding

```
WFET <Xt>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_WFxT) then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rd" field.

Operation

```
let localtimeout : integer = UInt(X{64}(d));

if Halted() && ConstrainUnpredictableBool(Unpredictable_WFxTDEBUG) then
    ExecuteAsNOP();
end;

Hint_WFET(localtimeout);
```

2026-09_rel 2026-09-21 14:22:38

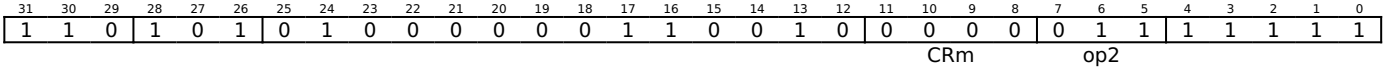
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WFI

Wait for interrupt

This instruction is a hint instruction that indicates that the PE can enter a low-power state and remain there until a wakeup event occurs. For more information, see *Wait For Interrupt*.

As described in *Wait For Interrupt*, the execution of a `WFI` instruction that would otherwise cause entry to a low-power state can be trapped to a higher Exception level.



Encoding

```
WFI
```

Decode for this encoding

```
// Empty.
```

Operation

```
Hint_WFI();
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WFIT

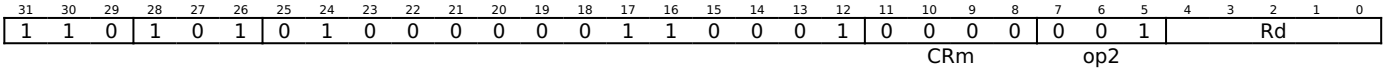
Wait for interrupt with timeout

This instruction provides a hint that the PE can enter a low-power state and remain there until either a local timeout event or a wakeup event occurs. For more information, see *Wait For Interrupt*.

As described in *Wait For Interrupt*, the execution of a `WFIT` instruction that would otherwise cause entry to a low-power state can be trapped to a higher Exception level.

System

(FEAT_WFxT)



Encoding

```
WFIT <Xt>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_WFxT) then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose source register, encoded in the "Rd" field.

Operation

```
let localtimeout : integer = UInt(X{64}(d));

if Halted() && ConstrainUnpredictableBool(Unpredictable_WFxTDEBUG) then
    ExecuteAsNOP();
end;

Hint_WFIT(localtimeout);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

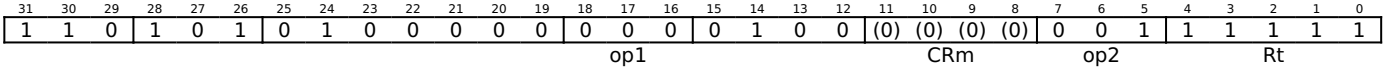
XAFLAG

Convert floating-point condition flags from external format to Arm format

This instruction converts the state of the PSTATE.{N,Z,C,V} flags from an alternative representation required by some software to a form representing the result of an Arm floating-point scalar compare instruction.

System

(FEAT_FlagM2)



Encoding

XAFLAG

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FlagM2) then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
let n : bit = NOT PSTATE.C AND NOT PSTATE.Z;
let z : bit = PSTATE.Z AND PSTATE.C;
let c : bit = PSTATE.C OR PSTATE.Z;
let v : bit = NOT PSTATE.C AND PSTATE.Z;

PSTATE.N = n;
PSTATE.Z = z;
PSTATE.C = c;
PSTATE.V = v;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

XPACD, XPACI, XPACLRI

Strip Pointer Authentication Code

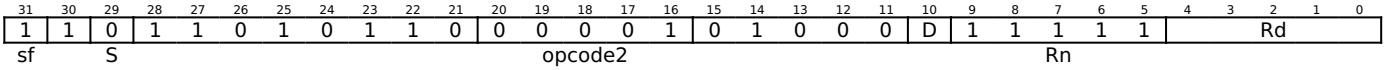
This instruction removes the Pointer Authentication Code from an address. The address is in the specified general-purpose register for `XPACI` and `XPACD`, and is in LR for `XPACLRI`.

The `XPACD` instruction is used for data addresses, and `XPACI` and `XPACLRI` are used for instruction addresses.

It has encodings from 2 classes: Integer and System

Integer

(FEAT_PAuth)



Encoding for the XPACD variant

Applies when `(D == 1)`

```
XPACD <Xd>
```

Encoding for the XPACI variant

Applies when `(D == 0)`

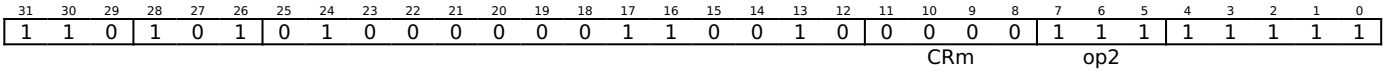
```
XPACI <Xd>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let data : boolean = D == '1';
```

System

(FEAT_PAuth)



Encoding

```
XPACLRI
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_PAuth) then EndOfDecode(Decode_NOP); end;

let d : integer{} = 30;
let data : boolean = FALSE;
```

Assembler Symbols

<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

```
X{64}(d) = Strip(X{64}(d), data);
```

2026-09_rel 2026-09-21 14:22:38

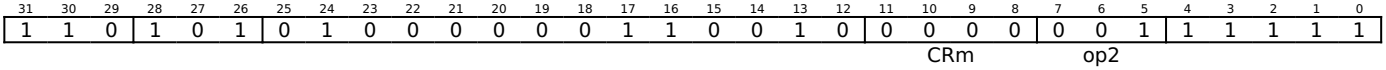
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

YIELD

Yield

This instruction is a hint instruction. Software with a multithreading capability can use a `YIELD` instruction to indicate to the PE that it is performing a task, for example a spin-lock, that could be swapped out to improve overall system performance. The PE can use this hint to suspend and resume multiple software threads if it supports the capability.

For more information about the recommended use of this instruction, see *The YIELD instruction*.



Encoding

```
YIELD
```

Decode for this encoding

```
// Empty.
```

Operation

```
Hint_Yield();
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

A64 -- SIMD and Floating-point Instructions (alphabetic order)

ABS: Absolute value (vector).

ADD (vector): Add (vector).

ADDHN, ADDHN2: Add returning high narrow.

ADDP (scalar): Add pair of elements (scalar).

ADDP (vector): Add pairwise (vector).

ADDV: Add across vector.

AESD: AES single round decryption.

AESE: AES single round encryption.

AESIMC: AES inverse mix columns.

AESMC: AES mix columns.

AND (vector): Bitwise AND (vector).

BCAX: Bit clear and exclusive-OR.

BF1CVTL, BF1CVTL2, BF2CVTL, BF2CVTL2: 8-bit floating-point convert to BFloat16 (vector).

BFCVT: Single-precision convert to BFloat16 (scalar).

BFCVTN, BFCVTN2: Single-precision convert to BFloat16 (vector).

BFDOT (by element): BFloat16 dot product to single-precision (vector, by element).

BFDOT (vector): BFloat16 dot product to single-precision (vector).

BFMLALB, BFMLALT (by element): BFloat16 multiply-add to single-precision (by element).

BFMLALB, BFMLALT (vector): BFloat16 multiply-add to single-precision (vector).

BFMMLA (widening): BFloat16 matrix multiply-accumulate to single-precision.

BIC (vector, immediate): Bitwise bit clear (vector, immediate).

BIC (vector, register): Bitwise bit clear (vector, register).

BIF: Bitwise insert if false.

BIT: Bitwise insert if true.

BSL: Bitwise select.

CLS (vector): Count leading sign bits (vector).

CLZ (vector): Count leading zero bits (vector).

CMEQ (register): Compare bitwise equal (vector).

CMEQ (zero): Compare bitwise equal to zero (vector).

CMGE (register): Compare signed greater than or equal (vector).

CMGE (zero): Compare signed greater than or equal to zero (vector).

CMGT (register): Compare signed greater than (vector).

CMGT (zero): Compare signed greater than zero (vector).

CMHI (register): Compare unsigned higher (vector).

CMHS (register): Compare unsigned higher or same (vector).

CMLE (zero): Compare signed less than or equal to zero (vector).

CMLT (zero): Compare signed less than zero (vector).

CMTST: Compare bitwise test bits nonzero (vector).

CNT: Population count per byte.

DUP (element): Duplicate vector element to vector or scalar.

DUP (general): Duplicate general-purpose register to vector.

EOR (vector): Bitwise exclusive-OR (vector).

EOR3: Three-way exclusive-OR.

EXT: Extract vector from pair of vectors.

F1CVTL, F1CVTL2, F2CVTL, F2CVTL2: 8-bit floating-point convert to half-precision (vector).

FABD: Floating-point absolute difference (vector).

FABS (scalar): Floating-point absolute value (scalar).

FABS (vector): Floating-point absolute value (vector).

FACGE: Floating-point absolute compare greater than or equal (vector).

FACGT: Floating-point absolute compare greater than (vector).

FADD (scalar): Floating-point add (scalar).

FADD (vector): Floating-point add (vector).

FADDP (scalar): Floating-point add pair of elements (scalar).

FADDP (vector): Floating-point add pairwise (vector).

FAMAX: Floating-point absolute maximum.

FAMIN: Floating-point absolute minimum.

FCADD: Floating-point complex add.

FCCMP: Floating-point conditional quiet compare (scalar).

FCCMPE: Floating-point conditional signaling compare (scalar).

FCMEQ (register): Floating-point compare equal (vector).

FCMEQ (zero): Floating-point compare equal to zero (vector).

FCMGE (register): Floating-point compare greater than or equal (vector).

FCMGE (zero): Floating-point compare greater than or equal to zero (vector).

FCMGT (register): Floating-point compare greater than (vector).

FCMGT (zero): Floating-point compare greater than zero (vector).

FCMLA: Floating-point complex multiply accumulate.

FCMLA (by element): Floating-point complex multiply accumulate (by element).

FCMLE (zero): Floating-point compare less than or equal to zero (vector).

FCMLT (zero): Floating-point compare less than zero (vector).

FCMP: Floating-point quiet compare (scalar).

FCMPE: Floating-point signaling compare (scalar).

FCSEL: Floating-point conditional select (scalar).

FCVT: Floating-point convert precision (scalar).

FCVTAS (scalar SIMD&FP): Floating-point convert to signed integer, rounding to nearest with ties to away (scalar SIMD&FP).

FCVTAS (scalar): Floating-point convert to signed integer, rounding to nearest with ties to away (scalar).

FCVTAS (vector): Floating-point convert to signed integer, rounding to nearest with ties to away (vector).

FCVTAU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding to nearest with ties to away (scalar SIMD&FP).

FCVTAU (scalar): Floating-point convert to unsigned integer, rounding to nearest with ties to away (scalar).

FCVTAU (vector): Floating-point convert to unsigned integer, rounding to nearest with ties to away (vector).

FCVTL, FCVTL2: Floating-point convert to higher precision long (vector).

FCVTMS (scalar SIMD&FP): Floating-point convert to signed integer, rounding toward minus infinity (scalar SIMD&FP).

FCVTMS (scalar): Floating-point convert to signed integer, rounding toward minus infinity (scalar).

FCVTMS (vector): Floating-point convert to signed integer, rounding toward minus infinity (vector).

FCVTMU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding toward minus infinity (scalar SIMD&FP).

FCVTMU (scalar): Floating-point convert to unsigned integer, rounding toward minus infinity (scalar).

FCVTMU (vector): Floating-point convert to unsigned integer, rounding toward minus infinity (vector).

FCVTN (half-precision to 8-bit floating-point): Half-precision convert to 8-bit floating-point (vector).

FCVTN, FCVTN2 (double to single-precision, single to half-precision): Floating-point convert to lower precision narrow (vector).

FCVTN, FCVTN2 (single-precision to 8-bit floating-point): Single-precision convert to 8-bit floating-point (vector).

FCVTNS (scalar SIMD&FP): Floating-point convert to signed integer, rounding to nearest with ties to even (scalar SIMD&FP).

FCVTNS (scalar): Floating-point convert to signed integer, rounding to nearest with ties to even (scalar).

FCVTNS (vector): Floating-point convert to signed integer, rounding to nearest with ties to even (vector).

FCVTNU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding to nearest with ties to even (scalar SIMD&FP).

FCVTNU (scalar): Floating-point convert to unsigned integer, rounding to nearest with ties to even (scalar).

FCVTNU (vector): Floating-point convert to unsigned integer, rounding to nearest with ties to even (vector).

FCVTPS (scalar SIMD&FP): Floating-point convert to signed integer, rounding toward plus infinity (scalar SIMD&FP).

FCVTPS (scalar): Floating-point convert to signed integer, rounding toward plus infinity (scalar).

FCVTPS (vector): Floating-point convert to signed integer, rounding toward plus infinity (vector).

FCVTPU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding toward plus infinity (scalar SIMD&FP).

FCVTPU (scalar): Floating-point convert to unsigned integer, rounding toward plus infinity (scalar).

FCVTPU (vector): Floating-point convert to unsigned integer, rounding toward plus infinity (vector).

FCVTXN, FCVTXN2: Floating-point convert to lower precision narrow, rounding to odd (vector).

FCVTZS (scalar SIMD&FP): Floating-point convert to signed integer, rounding toward zero (scalar SIMD&FP).

FCVTZS (scalar, fixed-point): Floating-point convert to signed fixed-point, rounding toward zero (scalar).

FCVTZS (scalar, integer): Floating-point convert to signed integer, rounding toward zero (scalar).

FCVTZS (vector, fixed-point): Floating-point convert to signed fixed-point, rounding toward zero (vector).

FCVTZS (vector, integer): Floating-point convert to signed integer, rounding toward zero (vector).

FCVTZU (scalar SIMD&FP): Floating-point convert to unsigned integer, rounding toward zero (scalar SIMD&FP).

FCVTZU (scalar, fixed-point): Floating-point convert to unsigned fixed-point, rounding toward zero (scalar).

FCVTZU (scalar, integer): Floating-point convert to unsigned integer, rounding toward zero (scalar).

FCVTZU (vector, fixed-point): Floating-point convert to unsigned fixed-point, rounding toward zero (vector).

FCVTZU (vector, integer): Floating-point convert to unsigned integer, rounding toward zero (vector).

FDIV (scalar): Floating-point divide (scalar).

FDIV (vector): Floating-point divide (vector).

FDOT (8-bit floating-point to half-precision, by element): 8-bit floating-point dot product to half-precision (vector, by element).

FDOT (8-bit floating-point to half-precision, vector): 8-bit floating-point dot product to half-precision (vector).

FDOT (8-bit floating-point to single-precision, by element): 8-bit floating-point dot product to single-precision (vector, by element).

FDOT (8-bit floating-point to single-precision, vector): 8-bit floating-point dot product to single-precision (vector).

FDOT (half-precision to single-precision, by element): Half-precision dot product to single-precision (vector, by element).

FDOT (half-precision to single-precision, vector): Half-precision dot product to single-precision (vector).

FJCVTZS: Floating-point JavaScript convert to signed fixed-point, rounding toward zero.

FMADD: Floating-point fused multiply-add (scalar).

FMAX (scalar): Floating-point maximum (scalar).

FMAX (vector): Floating-point maximum (vector).

FMAXNM (scalar): Floating-point maximum number (scalar).

FMAXNM (vector): Floating-point maximum number (vector).

FMAXNMP (scalar): Floating-point maximum number of pair of elements (scalar).

FMAXNMP (vector): Floating-point maximum number pairwise (vector).

FMAXNMV: Floating-point maximum number across vector.

FMAXP (scalar): Floating-point maximum of pair of elements (scalar).

FMAXP (vector): Floating-point maximum pairwise (vector).

FMAXV: Floating-point maximum across vector.

FMIN (scalar): Floating-point minimum (scalar).

FMIN (vector): Floating-point minimum (vector).

FMINNM (scalar): Floating-point minimum number (scalar).

FMINNM (vector): Floating-point minimum number (vector).

FMINNMP (scalar): Floating-point minimum number of pair of elements (scalar).

FMINNMP (vector): Floating-point minimum number pairwise (vector).

FMINNMV: Floating-point minimum number across vector.

FMINP (scalar): Floating-point minimum of pair of elements (scalar).

FMINP (vector): Floating-point minimum pairwise (vector).

FMINV: Floating-point minimum across vector.

FMLA (by element): Floating-point fused multiply-add to accumulator (by element).

FMLA (vector): Floating-point fused multiply-add to accumulator (vector).

FMLAL, FMLAL2 (by element): Floating-point fused multiply-add long to accumulator (by element).

FMLAL, FMLAL2 (vector): Floating-point fused multiply-add long to accumulator (vector).

FMLALB, FMLALT (by element): 8-bit floating-point multiply-add to half-precision (vector, by element).

FMLALB, FMLALT (vector): 8-bit floating-point multiply-add to half-precision (vector).

FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element): 8-bit floating-point multiply-add to single-precision (vector, by element).

FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector): 8-bit floating-point multiply-add to single-precision (vector).

FMLS (by element): Floating-point fused multiply-subtract from accumulator (by element).

FMLS (vector): Floating-point fused multiply-subtract from accumulator (vector).

FMLSL, FMLSL2 (by element): Floating-point fused multiply-subtract long from accumulator (by element).

FMLSL, FMLSL2 (vector): Floating-point fused multiply-subtract long from accumulator (vector).

FMMLA (non-widening): Half-precision matrix multiply-accumulate.

FMMLA (widening, 8-bit floating-point to half-precision): 8-bit floating-point matrix multiply-accumulate to half-precision.

FMMLA (widening, 8-bit floating-point to single-precision): 8-bit floating-point matrix multiply-accumulate to single-precision.

FMMLA (widening, half-precision to single-precision): Half-precision matrix multiply-accumulate to single-precision.

FMOV (general): Floating-point move to or from general-purpose register without conversion.

FMOV (register): Floating-point move register without conversion.

FMOV (scalar, immediate): Floating-point move immediate (scalar).

FMOV (vector, immediate): Floating-point move immediate (vector).

FMSUB: Floating-point fused multiply-subtract (scalar).

FMUL (by element): Floating-point multiply (by element).

FMUL (scalar): Floating-point multiply (scalar).

FMUL (vector): Floating-point multiply (vector).

FMULX: Floating-point multiply extended.

FMULX (by element): Floating-point multiply extended (by element).

FNEG (scalar): Floating-point negate (scalar).

FNEG (vector): Floating-point negate (vector).

FNMADD: Floating-point negated fused multiply-add (scalar).

FNMSUB: Floating-point negated fused multiply-subtract (scalar).

FNMUL (scalar): Floating-point multiply-negate (scalar).

FRECPE: Floating-point reciprocal estimate.

FRECPS: Floating-point reciprocal step.

FRECPX: Floating-point reciprocal exponent (scalar).

FRINT32X (scalar): Floating-point round to 32-bit integer, using current rounding mode (scalar).

FRINT32X (vector): Floating-point round to 32-bit integer, using current rounding mode (vector).

FRINT32Z (scalar): Floating-point round to 32-bit integer toward zero (scalar).

FRINT32Z (vector): Floating-point round to 32-bit integer toward zero (vector).

FRINT64X (scalar): Floating-point round to 64-bit integer, using current rounding mode (scalar).

FRINT64X (vector): Floating-point round to 64-bit integer, using current rounding mode (vector).

FRINT64Z (scalar): Floating-point round to 64-bit integer toward zero (scalar).

FRINT64Z (vector): Floating-point round to 64-bit integer toward zero (vector).

FRINTA (scalar): Floating-point round to integral, to nearest with ties to away (scalar).

FRINTA (vector): Floating-point round to integral, to nearest with ties to away (vector).

FRINTI (scalar): Floating-point round to integral, using current rounding mode (scalar).

FRINTI (vector): Floating-point round to integral, using current rounding mode (vector).

FRINTM (scalar): Floating-point round to integral, toward minus infinity (scalar).

FRINTM (vector): Floating-point round to integral, toward minus infinity (vector).

FRINTN (scalar): Floating-point round to integral, to nearest with ties to even (scalar).

FRINTN (vector): Floating-point round to integral, to nearest with ties to even (vector).

FRINTP (scalar): Floating-point round to integral, toward plus infinity (scalar).

FRINTP (vector): Floating-point round to integral, toward plus infinity (vector).

FRINTX (scalar): Floating-point round to integral exact, using current rounding mode (scalar).

FRINTX (vector): Floating-point round to integral exact, using current rounding mode (vector).

FRINTZ (scalar): Floating-point round to integral, toward zero (scalar).

FRINTZ (vector): Floating-point round to integral, toward zero (vector).

FRSQRTE: Floating-point reciprocal square root estimate.

FRSQRTS: Floating-point reciprocal square root step.

FSCALE: Floating-point adjust exponent by vector.

FSQRT (scalar): Floating-point square root (scalar).

FSQRT (vector): Floating-point square root (vector).

FSUB (scalar): Floating-point subtract (scalar).

FSUB (vector): Floating-point subtract (vector).

INS (element): Insert vector element from another vector element.

INS (general): Insert vector element from general-purpose register.

LD1 (multiple structures): Load multiple single-element structures to one, two, three, or four registers.

LD1 (single structure): Load one single-element structure to one lane of one register.

LD1R: Load one single-element structure and replicate to all lanes (of one register).

LD2 (multiple structures): Load multiple 2-element structures to two registers.

LD2 (single structure): Load single 2-element structure to one lane of two registers.

LD2R: Load single 2-element structure and replicate to all lanes of two registers.

LD3 (multiple structures): Load multiple 3-element structures to three registers.

LD3 (single structure): Load single 3-element structure to one lane of three registers.

LD3R: Load single 3-element structure and replicate to all lanes of three registers.

LD4 (multiple structures): Load multiple 4-element structures to four registers.

LD4 (single structure): Load single 4-element structure to one lane of four registers.

LD4R: Load single 4-element structure and replicate to all lanes of four registers.

LDAP1 (SIMD&FP): Load-acquire RCpc one single-element structure to one lane of one register.

LDAPUR (SIMD&FP): Load-acquire RCpc SIMD&FP register (unscaled offset).

LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL: Atomic BFloat16 add.

LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL: Atomic BFloat16 maximum.

LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML: Atomic BFloat16 maximum number.

LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL: Atomic BFloat16 minimum.

LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML: Atomic BFloat16 minimum number.

LDFADD, LDFADDA, LDFADDAL, LDFADDL: Atomic floating-point add.

LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL: Atomic floating-point maximum.

LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML: Atomic floating-point maximum number.

LDFMIN, LDFMINA, LDFMINAL, LDFMINL: Atomic floating-point minimum.

LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML: Atomic floating-point minimum number.

LDNP (SIMD&FP): Load pair of SIMD&FP registers, with non-temporal hint.

LDP (SIMD&FP): Load pair of SIMD&FP registers.

LDR (immediate, SIMD&FP): Load SIMD&FP register (immediate offset).

LDR (literal, SIMD&FP): Load SIMD&FP register (PC-relative literal).

LDR (register, SIMD&FP): Load SIMD&FP register (register offset).

LDTNP (SIMD&FP): Load unprivileged pair of SIMD&FP registers, with non-temporal hint.

LDTP (SIMD&FP): Load unprivileged pair of SIMD&FP registers.

LDUR (SIMD&FP): Load SIMD&FP register (unscaled offset).

LUTI2: Lookup table read with 2-bit indices.

LUTI4: Lookup table read with 4-bit indices.

MLA (by element): Multiply-add to accumulator (vector, by element).

MLA (vector): Multiply-add to accumulator (vector).

MLS (by element): Multiply-subtract from accumulator (vector, by element).

MLS (vector): Multiply-subtract from accumulator (vector).

MOV (element): Move vector element to another vector element: an alias of INS (element).

MOV (from general): Move general-purpose register to a vector element: an alias of INS (general).

MOV (scalar): Move vector element to scalar: an alias of DUP (element).

MOV (to general): Move vector element to general-purpose register: an alias of UMOV.

MOV (vector): Move vector: an alias of ORR (vector, register).

MOVI: Move immediate (vector).

MUL (by element): Multiply (vector, by element).

MUL (vector): Multiply (vector).

MVN: Bitwise NOT (vector): an alias of NOT.

MVNI: Move inverted immediate (vector).

NEG (vector): Negate (vector).

NOT: Bitwise NOT (vector).

ORN (vector): Bitwise inclusive OR NOT (vector).

ORR (vector, immediate): Bitwise inclusive OR (vector, immediate).

ORR (vector, register): Bitwise inclusive OR (vector, register).

PMUL: Polynomial multiply.

PMULL, PMULL2: Polynomial multiply long.

RADDHN, RADDHN2: Rounding add returning high narrow.

RAX1: Rotate and exclusive-OR.

RBIT (vector): Reverse bit order (vector).

REV16 (vector): Reverse elements in 16-bit halfwords (vector).

REV32 (vector): Reverse elements in 32-bit words (vector).

REV64: Reverse elements in 64-bit doublewords (vector).

RSHRN, RSHRN2: Rounding shift right narrow (immediate).

RSUBHN, RSUBHN2: Rounding subtract returning high narrow.

SABA: Signed absolute difference and accumulate.

SABAL, SABAL2: Signed absolute difference and accumulate long.

SABD: Signed absolute difference.

SABDL, SABDL2: Signed absolute difference long.

SADALP: Signed add and accumulate long pairwise.

SADDL, SADDL2: Signed add long (vector).

SADDLP: Signed add long pairwise.

SADDLV: Signed add long across vector.

SADDW, SADDW2: Signed add wide.

SCVTF (scalar SIMD&FP): Signed integer convert to floating-point (scalar SIMD&FP).

SCVTF (scalar, fixed-point): Signed fixed-point convert to floating-point (scalar).

SCVTF (scalar, integer): Signed integer convert to floating-point (scalar).

SCVTF (vector, fixed-point): Signed fixed-point convert to floating-point (vector).

SCVTF (vector, integer): Signed integer convert to floating-point (vector).

SDOT (by element): Dot product signed arithmetic (vector, by element).

SDOT (vector): Dot product signed arithmetic (vector).

SHA1C: SHA1 hash update (choose).

SHA1H: SHA1 fixed rotate.

SHA1M: SHA1 hash update (majority).

SHA1P: SHA1 hash update (parity).

SHA1SU0: SHA1 schedule update 0.

SHA1SU1: SHA1 schedule update 1.

SHA256H: SHA-256 hash update (part 1).

SHA256H2: SHA-256 hash update (part 2).

SHA256SU0: SHA-256 schedule update 0.

SHA256SU1: SHA-256 schedule update 1.

SHA512H: SHA-512 hash update part 1.

SHA512H2: SHA-512 hash update part 2.

SHA512SU0: SHA-512 schedule update 0.

SHA512SU1: SHA-512 schedule update 1.

SHADD: Signed halving add.

SHL: Shift left (immediate).

SHLL, SHLL2: Shift left long (by element size).

SHRN, SHRN2: Shift right narrow (immediate).

SHSUB: Signed halving subtract.

SLI: Shift left and insert (immediate).

SM3PARTW1: SM3PARTW1.

SM3PARTW2: SM3PARTW2.

SM3SS1: SM3SS1.

SM3TT1A: SM3TT1A.

SM3TT1B: SM3TT1B.

SM3TT2A: SM3TT2A.

SM3TT2B: SM3TT2B.

SM4E: SM4 encode.

SM4EKEY: SM4 key.

SMAX: Signed maximum (vector).

SMAXP: Signed maximum pairwise.

SMAXV: Signed maximum across vector.

SMIN: Signed minimum (vector).

SMINP: Signed minimum pairwise.

SMINV: Signed minimum across vector.

SMLAL, SMLAL2 (by element): Signed multiply-add long (vector, by element).

SMLAL, SMLAL2 (vector): Signed multiply-add long (vector).

SMLSL, SMLSL2 (by element): Signed multiply-subtract long (vector, by element).

SMLSL, SMLSL2 (vector): Signed multiply-subtract long (vector).

SMMLA (vector): Signed 8-bit integer matrix multiply-accumulate to 32-bit integer (vector).

SMOV: Signed move vector element to general-purpose register.

SMULL, SMULL2 (by element): Signed multiply long (vector, by element).

SMULL, SMULL2 (vector): Signed multiply long (vector).

SQABS: Signed saturating absolute value.

SQADD: Signed saturating add.

SQDMLAL, SQDMLAL2 (by element): Signed saturating doubling multiply-add long (by element).

SQDMLAL, SQDMLAL2 (vector): Signed saturating doubling multiply-add long.

SQDMLSL, SQDMLSL2 (by element): Signed saturating doubling multiply-subtract long (by element).

SQDMLSL, SQDMLSL2 (vector): Signed saturating doubling multiply-subtract long.

SQDMULH (by element): Signed saturating doubling multiply returning high half (by element).

SQDMULH (vector): Signed saturating doubling multiply returning high half.

SQDMULL, SQDMULL2 (by element): Signed saturating doubling multiply long (by element).

SQDMULL, SQDMULL2 (vector): Signed saturating doubling multiply long.

SQNEG: Signed saturating negate.

SQRDMLAH (by element): Signed saturating rounding doubling multiply accumulate returning high half (by element).

SQRDMLAH (vector): Signed saturating rounding doubling multiply accumulate returning high half (vector).

SQRDMLSH (by element): Signed saturating rounding doubling multiply subtract returning high half (by element).

SQRDMLSH (vector): Signed saturating rounding doubling multiply subtract returning high half (vector).

SQRDMULH (by element): Signed saturating rounding doubling multiply returning high half (by element).

SQRDMULH (vector): Signed saturating rounding doubling multiply returning high half.

SQRSHL: Signed saturating rounding shift left (register).

SQRSHRN, SQRSHRN2: Signed saturating rounded shift right narrow (immediate).

SQRSHRUN, SQRSHRUN2: Signed saturating rounded shift right unsigned narrow (immediate).

SQSHL (immediate): Signed saturating shift left (immediate).

SQSHL (register): Signed saturating shift left (register).

SQSHLU: Signed saturating shift left unsigned (immediate).

SQSHRN, SQSHRN2: Signed saturating shift right narrow (immediate).

SQSHRUN, SQSHRUN2: Signed saturating shift right unsigned narrow (immediate).

SQSUB: Signed saturating subtract.

SQXTN, SQXTN2: Signed saturating extract narrow.

SQXTUN, SQXTUN2: Signed saturating extract unsigned narrow.

SRHADD: Signed rounding halving add.

SRI: Shift right and insert (immediate).

SRSHL: Signed rounding shift left (register).

SRSHR: Signed rounding shift right (immediate).

SRSRA: Signed rounding shift right and accumulate (immediate).

SSHL: Signed shift left (register).

SSHLL, SSHLL2: Signed shift left long (immediate).

SSHR: Signed shift right (immediate).

SSRA: Signed shift right and accumulate (immediate).

SSUBL, SSUBL2: Signed subtract long.

SSUBW, SSUBW2: Signed subtract wide.

ST1 (multiple structures): Store multiple single-element structures from one, two, three, or four registers.

ST1 (single structure): Store a single-element structure from one lane of one register.

ST2 (multiple structures): Store multiple 2-element structures from two registers.

ST2 (single structure): Store single 2-element structure from one lane of two registers.

ST3 (multiple structures): Store multiple 3-element structures from three registers.

ST3 (single structure): Store single 3-element structure from one lane of three registers.

ST4 (multiple structures): Store multiple 4-element structures from four registers.

ST4 (single structure): Store single 4-element structure from one lane of four registers.

STBFADD, STBFADDL: Atomic BFloat16 floating-point add, without return.

STBFMAX, STBFMAXL: Atomic BFloat16 floating-point maximum, without return.

STBFMAXNM, STBFMAXNML: Atomic BFloat16 floating-point maximum number, without return.

STBFMIN, STBFMINL: Atomic BFloat16 floating-point minimum, without return.

STBFMINNM, STBFMINNML: Atomic BFloat16 floating-point minimum number, without return.

STFADD, STFADDL: Atomic floating-point add, without return.

STFMAX, STFMAXL: Atomic floating-point maximum, without return.

STFMAXNM, STFMAXNML: Atomic floating-point maximum number, without return.

STFMIN, STFMINL: Atomic floating-point minimum, without return.

STFMINNM, STFMINNML: Atomic floating-point minimum number, without return.

STL1 (SIMD&FP): Store-release a single-element structure from one lane of one register.

STLUR (SIMD&FP): Store-release SIMD&FP register (unscaled offset).

STNP (SIMD&FP): Store pair of SIMD&FP registers, with non-temporal hint.

STP (SIMD&FP): Store pair of SIMD&FP registers.

STR (immediate, SIMD&FP): Store SIMD&FP register (immediate offset).

STR (register, SIMD&FP): Store SIMD&FP register (register offset).

STTNP (SIMD&FP): Store unprivileged pair of SIMD&FP registers, with non-temporal hint.

STTP (SIMD&FP): Store unprivileged pair of SIMD&FP registers.

STUR (SIMD&FP): Store SIMD&FP register (unscaled offset).

SUB (vector): Subtract (vector).

SUBHN, SUBHN2: Subtract returning high narrow.

SUDOT (by element): Dot product with signed and unsigned integers (vector, by element).

SUQADD: Signed saturating accumulate of unsigned value.

SXTL, SXTL2: Signed extend long: an alias of SSHLL, SSHLL2.

TBL: Table vector lookup.

TBX: Table vector lookup extension.

TRN1: Transpose vectors (primary).

TRN2: Transpose vectors (secondary).

UABA: Unsigned absolute difference and accumulate.

UABAL, UABAL2: Unsigned absolute difference and accumulate long.

UABD: Unsigned absolute difference (vector).

UABDL, UABDL2: Unsigned absolute difference long.

UADALP: Unsigned add and accumulate long pairwise.

UADDL, UADDL2: Unsigned add long (vector).

UADDLP: Unsigned add long pairwise.

UADDLV: Unsigned sum long across vector.

UADDW, UADDW2: Unsigned add wide.

UCVTF (scalar SIMD&FP): Unsigned integer convert to floating-point (scalar SIMD&FP).

UCVTF (scalar, fixed-point): Unsigned fixed-point convert to floating-point (scalar).

UCVTF (scalar, integer): Unsigned integer convert to floating-point (scalar).

UCVTF (vector, fixed-point): Unsigned fixed-point convert to floating-point (vector).

UCVTF (vector, integer): Unsigned integer convert to floating-point (vector).

UDOT (by element): Dot product unsigned arithmetic (vector, by element).

UDOT (vector): Dot product unsigned arithmetic (vector).

UHADD: Unsigned halving add.

UHSUB: Unsigned halving subtract.

UMAX: Unsigned maximum (vector).

UMAXP: Unsigned maximum pairwise.

UMAXV: Unsigned maximum across vector.

UMIN: Unsigned minimum (vector).

UMINP: Unsigned minimum pairwise.

UMINV: Unsigned minimum across vector.

UMLAL, UMLAL2 (by element): Unsigned multiply-add long (vector, by element).

UMLAL, UMLAL2 (vector): Unsigned multiply-add long (vector).

UMLSL, UMLSL2 (by element): Unsigned multiply-subtract long (vector, by element).

UMLSL, UMLSL2 (vector): Unsigned multiply-subtract long (vector).

UMMLA (vector): Unsigned 8-bit integer matrix multiply-accumulate to 32-bit integer (vector).

UMOV: Unsigned move vector element to general-purpose register.

UMULL, UMULL2 (by element): Unsigned multiply long (vector, by element).

UMULL, UMULL2 (vector): Unsigned multiply long (vector).

UQADD: Unsigned saturating add.

UQRSHL: Unsigned saturating rounding shift left (register).

UQRSHRN, UQRSHRN2: Unsigned saturating rounded shift right narrow (immediate).

UQSHL (immediate): Unsigned saturating shift left (immediate).

UQSHL (register): Unsigned saturating shift left (register).

UQSHRN, UQSHRN2: Unsigned saturating shift right narrow (immediate).

UQSUB: Unsigned saturating subtract.

UQXTN, UQXTN2: Unsigned saturating extract narrow.

URECPE: Unsigned reciprocal estimate.

URHADD: Unsigned rounding halving add.

URSHL: Unsigned rounding shift left (register).

URSHR: Unsigned rounding shift right (immediate).

URSQRTE: Unsigned reciprocal square root estimate.

URSRA: Unsigned rounding shift right and accumulate (immediate).

USDOT (by element): Dot product with unsigned and signed integers (vector, by element).

USDOT (vector): Dot product with unsigned and signed integers (vector).

USHL: Unsigned shift left (register).

USHLL, USHLL2: Unsigned shift left long (immediate).

USHR: Unsigned shift right (immediate).

USMMLA (vector): Unsigned and signed 8-bit integer matrix multiply-accumulate to 32-bit integer (vector).

USQADD: Unsigned saturating accumulate of signed value.

USRA: Unsigned shift right and accumulate (immediate).

USUBL, USUBL2: Unsigned subtract long.

USUBW, USUBW2: Unsigned subtract wide.

UXTL, UXTL2: Unsigned extend long: an alias of USHLL, USHLL2.

UZP1: Unzip vectors (primary).

UZP2: Unzip vectors (secondary).

XAR: Exclusive-OR and rotate.

XTN, XTN2: Extract narrow.

ZIP1: Zip vectors (primary).

ZIP2: Zip vectors (secondary).

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ABS

Absolute value (vector)

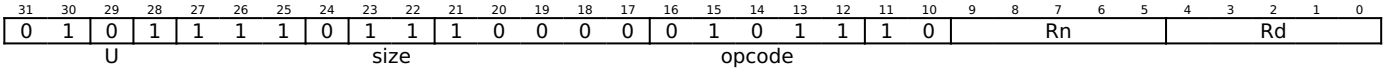
This instruction calculates the absolute value of each vector element in the source SIMD&FP register, puts the result into a vector, and writes the vector to the destination SIMD&FP register. If the value in an element of the source vector is the most negative value, the result for that element is the same as the value in that element of the source vector.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

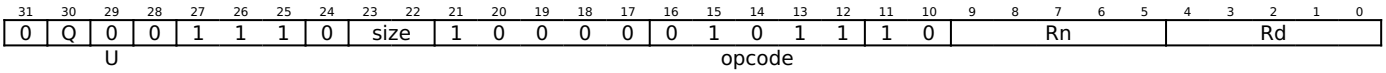
```
ABS D<d>, D<n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
ABS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    element = SInt(operand[e*:esize]);
    element = Abs(element);
    result[e*:esize] = element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADD (vector)

Add (vector)

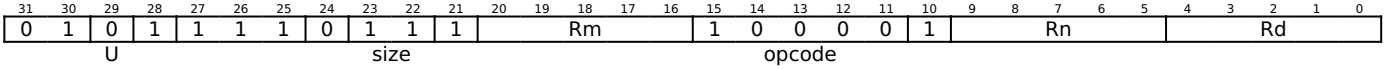
This instruction adds corresponding elements in the two source SIMD&FP registers, places the results into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

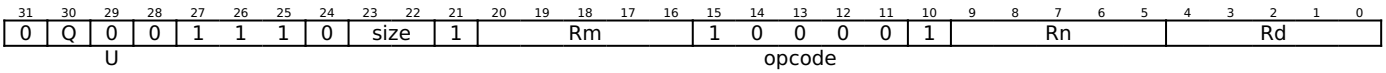
```
ADD D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
ADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = element1 + element2;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDHN, ADDHN2

Add returning high narrow

This instruction adds each vector element in the first source SIMD&FP register to the corresponding vector element in the second source SIMD&FP register, places the most significant half of the result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register.

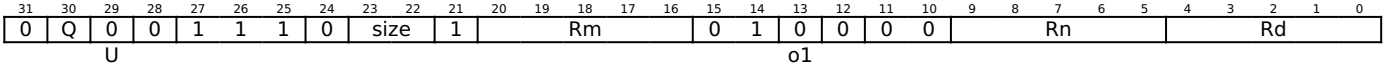
The results are truncated. For rounded results, see *RADDHN*.

The ADDHN instruction writes the vector to the lower half of the destination register and clears the upper half. The ADDHN2 instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
ADDHN{2} <Vd>.<Tb>, <Vn>.<Ta>, <Vm>.<Ta>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
let round : boolean = FALSE;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(2*datasize) = V{}(n);
let operand2 : bits(2*datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:(2*esize)]);
    element2 = UInt(operand2[e*:(2*esize)]);
    sum = element1 + element2;
    sum = RShr(sum, esize, round);
    result[e*:esize] = sum[esize-1:0];
end;

Vpart{datasize}(d, part) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

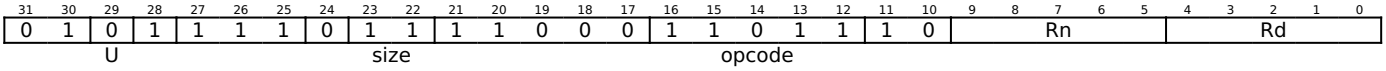
ADDP (scalar)

Add pair of elements (scalar)

This instruction adds two vector elements in the source SIMD&FP register and writes the scalar result into the destination SIMD&FP register. Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
ADDP D<d>, <Vn>.2D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 64;
let datasize : integer{} = 128;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
V{esize}(d) = IntReduce{esize, datasize}(ReduceOp_ADD, operand);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDP (vector)

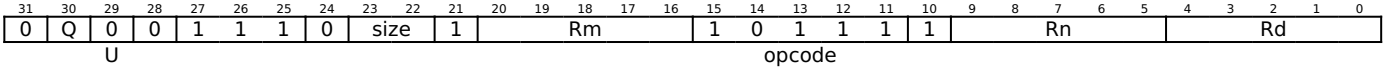
Add pairwise (vector)

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements from the concatenated vector, adds each pair of values together, places the result into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
ADDP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = concat[(2*e)*:esize];
    element2 = concat[((2*e)+1)*:esize];
    result[e*:esize] = element1 + element2;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDV

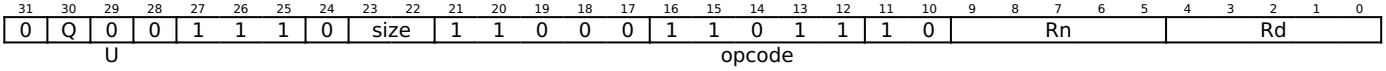
Add across vector

This instruction adds every vector element in the source SIMD&FP register together, and writes the scalar result to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
ADDV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '100' then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<V> Is the destination width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	RESERVED
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
V{esize}(d) = IntReduce{esize, datasize}(ReduceOp_ADD, operand);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

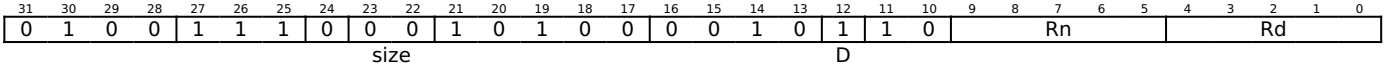
AESD

AES single round decryption

This instruction reads a 16-byte state array from the first source vector, together with a round key from the second source vector. The state array undergoes a single round of the AddRoundKey(), InvShiftRows(), and InvSubBytes() transformations in accordance with the AES standard. The updated state array is destructively placed in the first source vector.

Advanced SIMD

(FEAT_AES)



Encoding

```
AESD <Vd>.16B, <Vn>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AES) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
var result : bits(128) = operand1 XOR operand2;
result = AESInvShiftRows(result);
V{128}(d) = AESInvSubBytes(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

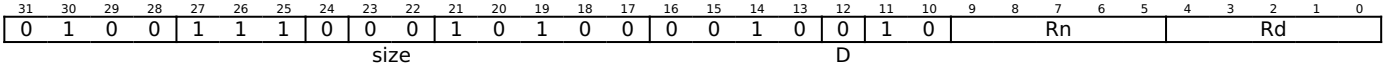
AESE

AES single round encryption

This instruction reads a 16-byte state array from the first source vector, together with a round key from the second source vector. The state array undergoes a single round of the AddRoundKey(), ShiftRows(), and SubBytes() transformations in accordance with the AES standard. The updated state array is destructively placed in the first source vector.

Advanced SIMD

(FEAT_AES)



Encoding

```
AESE <Vd>.16B, <Vn>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AES) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
var result : bits(128) = operand1 XOR operand2;
result = AESShiftRows(result);
V{128}(d) = AESSubBytes(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

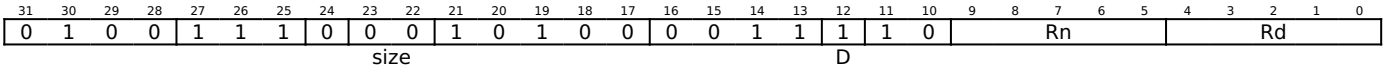
AESIMC

AES inverse mix columns

This instruction reads a 16-byte state array from the source vector and performs a single round of the InvMixColumns() transformation on the state array in accordance with the AES standard. The updated state array is placed in the destination vector.

Advanced SIMD

(FEAT_AES)



Encoding

```
AESIMC <Vd>.16B, <Vn>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AES) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand : bits(128) = V{}(n);
V{128}(d) = AESInvMixColumns(operand);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

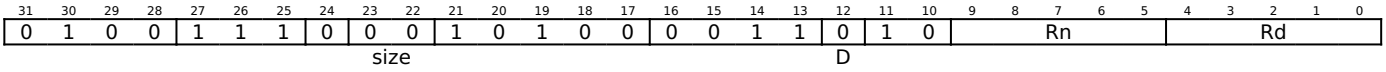
AESMC

AES mix columns

This instruction reads a 16-byte state array from the source vector and performs a single round of the MixColumns() transformation on the state array in accordance with the AES standard. The updated state array is placed in the destination vector.

Advanced SIMD

(FEAT_AES)



Encoding

```
AESMC <Vd>.16B, <Vn>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AES) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand : bits(128) = V{}(n);
V{128}(d) = AESMixColumns(operand);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

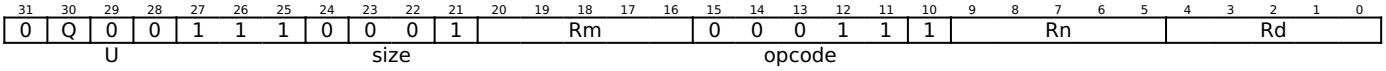
AND (vector)

Bitwise AND (vector)

This instruction performs a bitwise AND between the two source SIMD&FP registers, and writes the result to the destination SIMD&FP register. Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
AND <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
V{datasize}(d) = operand1 AND operand2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

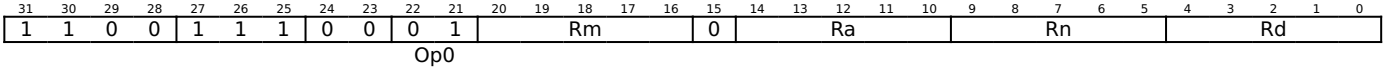
BCAX

Bit clear and exclusive-OR

This instruction performs a bitwise AND between the 128-bit vector in a source SIMD&FP register and the complement of the vector in another source SIMD&FP register, then performs a bitwise exclusive-OR between the resulting vector and the vector in a third source SIMD&FP register, and writes the result to the destination SIMD&FP register.

Advanced SIMD

(FEAT_SHA3)



Encoding

```
BCAX <Vd>.16B, <Vn>.16B, <Vm>.16B, <Va>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.
<Va>	Is the name of the third SIMD&FP source register, encoded in the "Ra" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(m);
let operand2 : bits(128) = V{}(n);
let operand3 : bits(128) = V{}(a);

V{128}(d) = operand2 XOR (operand1 AND NOT(operand3));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

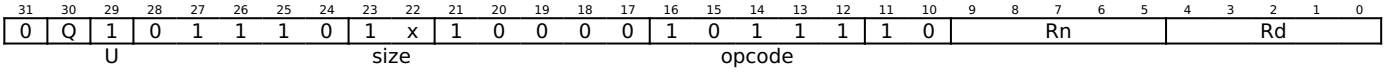
BF1CVTL, BF1CVTL2, BF2CVTL, BF2CVTL2

8-bit floating-point convert to BFloat16 (vector)

This instruction converts each 8-bit floating-point element from the lower or upper half of the source vector to BFloat16 while downscaling the value, and places the results in the 16-bit elements of the destination vector. BF1CVTL and BF2CVTL convert the elements from the lower half of the source vector while scaling the values by $2^{-UInt(FPMR.LSCALE[5:0])}$ and $2^{-UInt(FPMR.LSCALE2[5:0])}$, respectively. BF1CVTL2 and BF2CVTL2 convert the elements from the upper half of the source vector while scaling the values by $2^{-UInt(FPMR.LSCALE[5:0])}$ and $2^{-UInt(FPMR.LSCALE2[5:0])}$, respectively. The 8-bit floating-point encoding format for BF1CVTL and BF1CVTL2 is selected by FPMR.F8S1. The 8-bit floating-point encoding format for BF2CVTL and BF2CVTL2 is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_FP8)



Encoding for the BF1CVTL{2} variant

Applies when `(size == 10)`

```
BF1CVTL{2} <Vd>.8H, <Vn>.<Ta>
```

Encoding for the BF2CVTL{2} variant

Applies when `(size == 11)`

```
BF2CVTL{2} <Vd>.8H, <Vn>.<Ta>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP8) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let part : integer = UInt(Q);
let elements : integer = 64 DIV 8;
let issrc2 : boolean = size == '11';
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> — Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> — Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	8B
1	16B

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(64) = Vpart{}(n, part);
var result : bits(128);

for e = 0 to elements-1 do
    result[e*:16] = FP8ConvertBF(operand[e*:8], issrc2, FPCR(), FPMR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFCVT

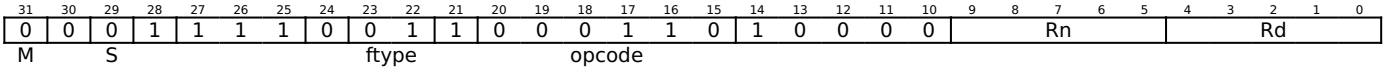
Single-precision convert to BFloat16 (scalar)

This instruction converts the single-precision floating-point value in the 32-bit SIMD&FP source register to BFloat16 format and writes the result in the 16-bit SIMD&FP destination register.

ID_AA64ISAR1_EL1.BF16 indicates whether this instruction is supported.

Single-precision to BFloat16

(FEAT_BF16)



Encoding

```
BFCVT <Hd>, <Sn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_BF16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(32) = V{}(n);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

result[0+:16] = FPConvertBF(operand, FPCR());
V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFCVTN, BFCVTN2

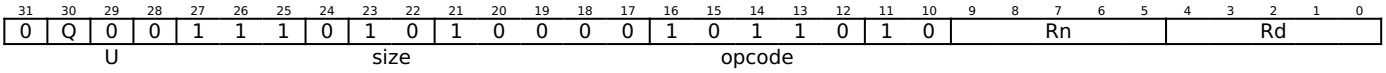
Single-precision convert to BFloat16 (vector)

This instruction reads each single-precision element in the SIMD&FP source vector, converts each value to BFloat16 format, and writes the results in the lower or upper half of the SIMD&FP destination vector. The result elements are half the width of the source elements.

The `BFCVTN` instruction writes the half-width results to the lower half of the 128-bit SIMD&FP destination vector and clears the upper half to zero. The `BFCVTN2` instruction writes the results to the upper half of the 128-bit SIMD&FP destination vector without affecting the lower half.

Vector single-precision to BFloat16

(FEAT_BF16)



Encoding

```
BFCVTN{2} <Vd>.<Ta>, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_BF16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let part : integer = UInt(Q);
let elements : integer = 64 DIV 16;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	4H
1	8H

<Vn> — Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(128) = V{}(n);
var result : bits(64);

for e = 0 to elements-1 do
    result[e*:16] = FPConvertBF(operand[e*:32], FPCR());
end;

Vpart{64}(d, part) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFDOT (by element)

BFloat16 dot product to single-precision (vector, by element)

This instruction delimits the source vectors into pairs of BFloat16 elements. The BFloat16 pair within the second source vector is specified using an immediate index. The index range is from 0 to 3 inclusive.

If FEAT_EBF16 is not implemented or FPCR.EBF is 0, this instruction:

- Performs an unfused sum-of-products of each pair of adjacent BFloat16 elements in the first source vector with the specified pair of elements in the second source vector. The intermediate single-precision products are rounded before they are summed, and the intermediate sum is rounded before accumulation into the single-precision destination element that overlaps with the corresponding pair of BFloat16 elements in the first source vector.
- Uses the non-IEEE 754 Round-to-Odd rounding mode, which forces bit 0 of an inexact result to 1, and rounds an overflow to an appropriately signed Infinity.
- Flushes denormalized inputs and results to zero, as if FPCR.{FZ, FIZ} is {1, 1}.
- Honors FPCR.AH when generating a default NaN result, otherwise disables alternative floating point behaviors, as if FPCR.AH is 0.

If FEAT_EBF16 is implemented and FPCR.EBF is 1, then this instruction:

- Performs a fused sum-of-products of each pair of adjacent BFloat16 elements in the first source vector with the specified pair of elements in the second source vector. The intermediate single-precision products are not rounded before they are summed, but the intermediate sum is rounded before accumulation into the single-precision destination element that overlaps with the corresponding pair of BFloat16 elements in the first source vector.
- Follows all other floating-point behaviors that apply to single-precision arithmetic, as governed by FPCR.RMode, FPCR.FZ, FPCR.AH, and FPCR.FIZ.

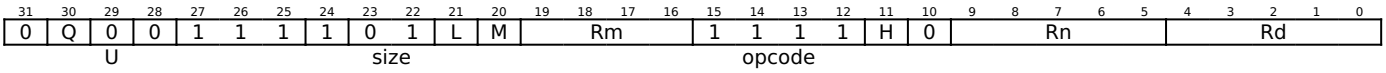
Irrespective of FEAT_EBF16 and FPCR.EBF, this instruction:

- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.
- Generates only the default NaN, as if FPCR.DN is 1.

ID_AA64ISAR1_EL1.BF16 indicates whether this instruction is supported.

Advanced SIMD

(FEAT_BF16)



Encoding

```
BFDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.2H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_BF16) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(M::Rm);
let d : integer = UInt(Rd);
let i : integer = UInt(H::L);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 32;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	4H
1	8H

<Vm> Is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> Is the immediate index of a pair of 16-bit elements in the range 0 to 3, encoded in the "H:L" fields.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    let elt1_a : bits(16) = operand1[(2 * e + 0)*:16];
    let elt1_b : bits(16) = operand1[(2 * e + 1)*:16];
    let elt2_a : bits(16) = operand2[(2 * i + 0)*:16];
    let elt2_b : bits(16) = operand2[(2 * i + 1)*:16];

    let sum : bits(32) = operand3[e*:32];
    result[e*:32] = BFDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFDOT (vector)

BFloat16 dot product to single-precision (vector)

This instruction delimits the source vectors into pairs of BFloat16 elements.
If FEAT_EBF16 is not implemented or FPCR.EBF is 0, this instruction:

- Performs an unfused sum-of-products of each pair of adjacent BFloat16 elements in the source vectors. The intermediate single-precision products are rounded before they are summed, and the intermediate sum is rounded before accumulation into the single-precision destination element that overlaps with the corresponding pair of BFloat16 elements in the source vectors.
- Uses the non-IEEE 754 Round-to-Odd rounding mode, which forces bit 0 of an inexact result to 1, and rounds an overflow to an appropriately signed Infinity.
- Flushes denormalized inputs and results to zero, as if FPCR.{FZ, FIZ} is {1, 1}.
- Honors FPCR.AH when generating a default NaN result, otherwise disables alternative floating point behaviors, as if FPCR.AH is 0.

If FEAT_EBF16 is implemented and FPCR.EBF is 1, then this instruction:

- Performs a fused sum-of-products of each pair of adjacent BFloat16 elements in the source vectors. The intermediate single-precision products are not rounded before they are summed, but the intermediate sum is rounded before accumulation into the single-precision destination element that overlaps with the corresponding pair of BFloat16 elements in the source vectors.
- Follows all other floating-point behaviors that apply to single-precision arithmetic, as governed by FPCR.RMode, FPCR.FZ, FPCR.AH, and FPCR.FIZ.

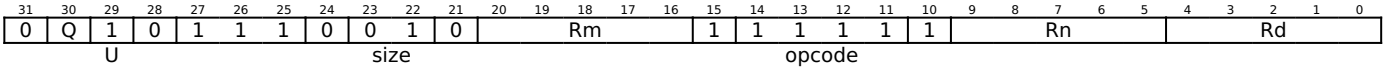
Irrespective of FEAT_EBF16 and FPCR.EBF, this instruction:

- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.
- Generates only the default NaN, as if FPCR.DN is 1.

ID_AA64ISAR1_EL1.BF16 indicates whether this instruction is supported.

Advanced SIMD

(FEAT_BF16)



Encoding

```
BFDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_BF16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 32;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	4H
1	8H

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    let elt1_a : bits(16) = operand1[(2 * e + 0)*:16];
    let elt1_b : bits(16) = operand1[(2 * e + 1)*:16];
    let elt2_a : bits(16) = operand2[(2 * e + 0)*:16];
    let elt2_b : bits(16) = operand2[(2 * e + 1)*:16];

    let sum : bits(32) = operand3[e*:32];
    result[e*:32] = BFDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLALB, BFMLALT (by element)

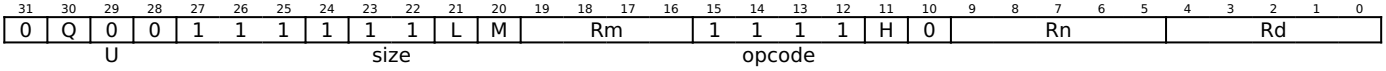
BFloat16 multiply-add to single-precision (by element)

This instruction widens the even-numbered (bottom) or odd-numbered (top) 16-bit elements in the first source vector, and the indexed element in the second source vector from Bfloat16 to single-precision format. The instruction then multiplies and adds these values without intermediate rounding to single-precision elements of the destination vector that overlap with the corresponding BFloat16 elements in the first source vector.

ID_AA64ISAR1_EL1.BF16 indicates whether this instruction is supported.

Vector

(FEAT_BF16)



Encoding

```
BFMLAL<bt> <Vd>.4S, <Vn>.8H, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_BF16) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt('0'::Rm);
let d : integer = UInt(Rd);
let index : integer = UInt(H::L::M);

let elements : integer = 128 DIV 32;
let sel : integer = UInt(Q);
```

Assembler Symbols

<bt> Is the bottom or top element specifier, encoded in “Q”:

Q	<bt>
0	B
1	T

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, in the range V0 to V15, encoded in the "Rm" field.

<index> Is the element index, in the range 0 to 7, encoded in the "H:L:M" fields.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
var result : bits(128);
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(128) = V{}(d);
let element2 : bits(16)  = operand2[index*:16];

for e = 0 to elements-1 do
    let element1 : bits(16)   = operand1[(2 * e + sel)*:16];
    let addend : bits(32)     = operand3[e*:32];
    result[e*:32] = BFMulAddH(addend, element1, element2, FPCR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLALB, BFMLALT (vector)

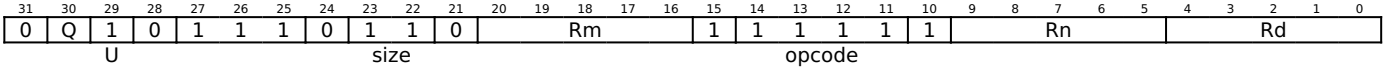
BFloat16 multiply-add to single-precision (vector)

This instruction widens the even-numbered (bottom) or odd-numbered (top) 16-bit elements in the first and second source vectors from Bfloat16 to single-precision format. The instruction then multiplies and adds these values without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding BFloat16 elements in the source vectors.

ID_AA64ISAR1_EL1.BF16 indicates whether this instruction is supported.

Vector

(FEAT_BF16)



Encoding

```
BFMLAL<bt> <Vd>.4S, <Vn>.8H, <Vm>.8H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_BF16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let elements : integer = 128 DIV 32;
let sel : integer = UInt(Q);
```

Assembler Symbols

<bt> Is the bottom or top element specifier, encoded in “Q”:

Q	<bt>
0	B
1	T

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(128) = V{}(d);
var result : bits(128);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[(2 * e + sel)*:16];
    let element2 : bits(16) = operand2[(2 * e + sel)*:16];
    let addend : bits(32)   = operand3[e*:32];
    result[e*:32] = BFMulAddH(addend, element1, element2, FPCR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMMLA (widening)

BFloat16 matrix multiply-accumulate to single-precision

This instruction multiplies the 2x4 matrix of BFloat16 values held in the first source vector by the 4x2 matrix of BFloat16 values in the second source vector.
If FEAT_EBF16 is not implemented or FPCR.EBF is 0, this instruction:

- Calculates the intermediate single-precision results as a sum of two unfused sum-of-products using two pairs of adjacent BFloat16 values from each of the source vectors, and then rounds the results before accumulation into the 2x2 matrix of single-precision results in the destination vector. This is equivalent to accumulating a 4-way dot product per destination element.
- Uses the non-IEEE 754 Round-to-Odd rounding mode, which forces bit 0 of an inexact result to 1, and rounds an overflow to an appropriately signed Infinity.
- Flushes denormalized inputs and results to zero, as if FPCR.{FZ, FIZ} is {1, 1}.
- Honors FPCR.AH when generating a default NaN result, otherwise disables alternative floating point behaviors, as if FPCR.AH is 0.

If FEAT_EBF16 is implemented and FPCR.EBF is 1, this instruction:

- Calculates the intermediate single-precision results as a sum of two fused sum-of-products using two pairs of adjacent BFloat16 values from each of the source vectors, and then rounds the results before accumulation into the 2x2 matrix of single-precision results in the destination vector. This is equivalent to accumulating a 4-way dot product per destination element.
- Follows all other floating-point behaviors that apply to single-precision arithmetic, as governed by FPCR.RMode, FPCR.FZ, FPCR.AH, and FPCR.FIZ.

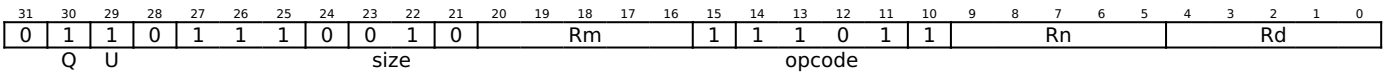
Irrespective of FEAT_EBF16 and FPCR.EBF, this instruction:

- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.
- Generates only the default NaN, as if FPCR.DN is 1.

ID_AA64ISAR1_EL1.BF16 indicates whether this instruction is supported.

Advanced SIMD

(FEAT_BF16)



Encoding

```
BFMMLA <Vd>.4S, <Vn>.8H, <Vm>.8H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_BF16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let op1 : bits(128) = V{}(n);
let op2 : bits(128) = V{}(m);
let acc : bits(128) = V{}(d);

V{128}(d) = BFMatMulAddH(acc, op1, op2, FPCR());
```

Operational information

Arm expects that the BFMMLA instruction will deliver a peak BFloat16 multiply throughput that is at least as high as can be achieved using two BFDOT (vector) instructions, with a goal that it should have significantly higher throughput.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BIC (vector, immediate)

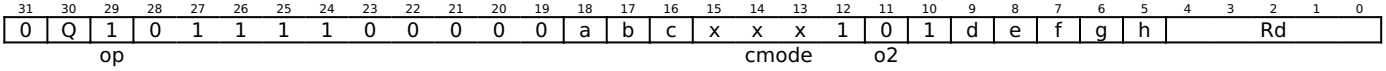
Bitwise bit clear (vector, immediate)

This instruction reads each vector element from the destination SIMD&FP register, performs a bitwise AND between each result and the complement of an immediate constant, places the result into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Shifted immediate

(FEAT_AdvSIMD)



Encoding for the 16-bit variant

Applies when `(cmode == 10x1)`

```
BIC <Vd>.<T>, #<imm8>{, LSL #<amount>}
```

Encoding for the 32-bit variant

Applies when `(cmode == 0xx1)`

```
BIC <Vd>.<T>, #<imm8>{, LSL #<amount>}
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let rd : integer = UInt(Rd);
let datasize : integer{} = 64 << UInt(Q);
let imm64 : bits(64) = AdvSIMDExpandImm(op, cmode, a::b::c::d::e::f::g::h);
let imm : bits(datasize) = Replicate{}(imm64);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP register, encoded in the "Rd" field.

<T> For the "16-bit" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "32-bit" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	2S
1	4S

<imm8> Is an 8-bit immediate encoded in "a:b:c:d:e:f:g:h".

<amount> For the "16-bit" variant: is the shift amount encoded in “cmode[1]”:

cmode[1]	<amount>
0	0
1	8

defaulting to 0 if LSL is omitted.

For the "32-bit" variant: is the shift amount encoded in “cmode[2:1]”:

cmode[2:1]	<amount>
00	0
01	8
10	16
11	24

defaulting to 0 if LSL is omitted.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(rd);
V{datasize}(rd) = operand AND NOT(imm);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BIC (vector, register)

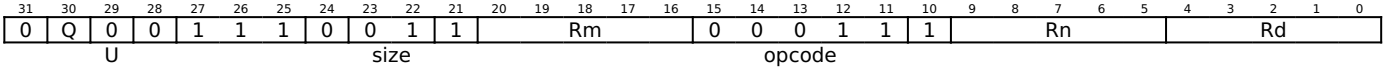
Bitwise bit clear (vector, register)

This instruction performs a bitwise AND between the first source SIMD&FP register and the complement of the second source SIMD&FP register, and writes the result to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
BIC <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = NOT(V{datasize}(m));
V{datasize}(d) = operand1 AND operand2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BIF

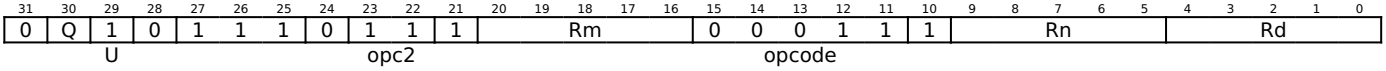
Bitwise insert if false

This instruction inserts each bit from the first source SIMD&FP register into the destination SIMD&FP register if the corresponding bit of the second source SIMD&FP register is 0, otherwise leaves the bit in the destination register unchanged.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
BIF <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(datasize) = V{}(d);
let operand2 : bits(datasize) = NOT(V{datasize}(m));
let operand3 : bits(datasize) = V{}(n);
V{datasize}(d) = operand1 XOR ((operand1 XOR operand3) AND operand2);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BIT

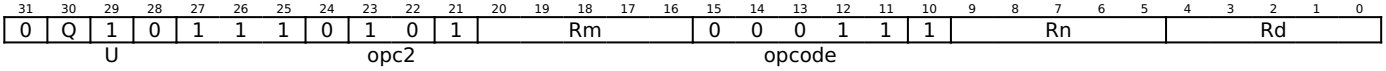
Bitwise insert if true

This instruction inserts each bit from the first source SIMD&FP register into the SIMD&FP destination register if the corresponding bit of the second source SIMD&FP register is 1, otherwise leaves the bit in the destination register unchanged.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
BIT <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(datasize) = V{}(d);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(n);
V{datasize}(d) = operand1 XOR ((operand1 XOR operand3) AND operand2);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BSL

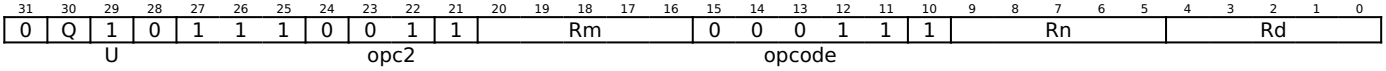
Bitwise select

This instruction sets each bit in the destination SIMD&FP register to the corresponding bit from the first source SIMD&FP register when the original destination bit was 1, otherwise from the second source SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
BSL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(datasize) = V{}(m);
let operand2 : bits(datasize) = V{}(d);
let operand3 : bits(datasize) = V{}(n);
V{datasize}(d) = operand1 XOR ((operand1 XOR operand3) AND operand2);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CLS (vector)

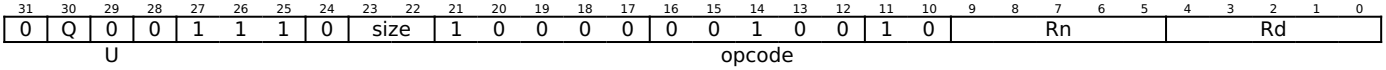
Count leading sign bits (vector)

This instruction determines the number of consecutive bits following the most significant bit that are the same as the most significant bit in each vector element in the source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register. The result does not include the most significant bit itself.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
CLS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

for e = 0 to elements-1 do
    let count : integer = CountLeadingSignBits(operand[e*:esize]);
    result[e*:esize] = count[esize-1:0];
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CLZ (vector)

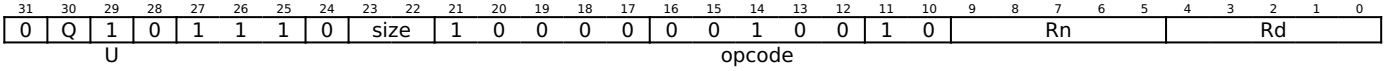
Count leading zero bits (vector)

This instruction determines the number of consecutive zeros, starting from the most significant bit, in each vector element in the source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
CLZ <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

for e = 0 to elements-1 do
    let count : integer = CountLeadingZeroBits(operand[e*:esize]);
    result[e*:esize] = count[esize-1:0];
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMEQ (register)

Compare bitwise equal (vector)

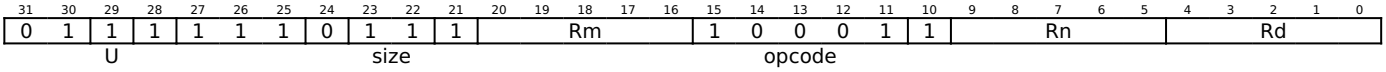
This instruction compares each vector element from the first source SIMD&FP register with the corresponding vector element from the second source SIMD&FP register, and if the comparison is equal sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

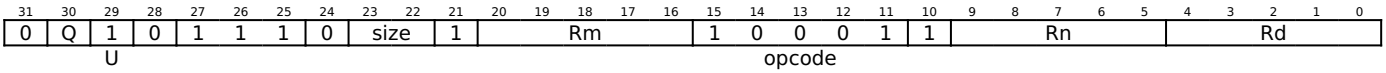
```
CMEQ D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMEQ <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = if element1 == element2 then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMEQ (zero)

Compare bitwise equal to zero (vector)

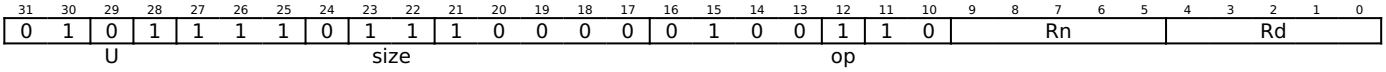
This instruction reads each vector element in the source SIMD&FP register and if the value is equal to zero sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

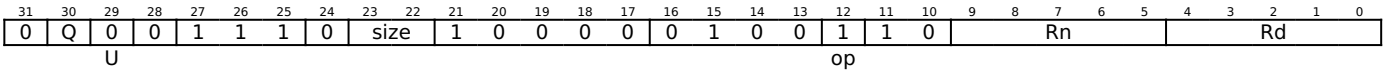
```
CMEQ D<d>, D<n>, #0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMEQ <Vd>.<T>, <Vn>.<T>, #0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    element = SInt(operand[e*:esize]);
    result[e*:esize] = if element == 0 then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMGE (register)

Compare signed greater than or equal (vector)

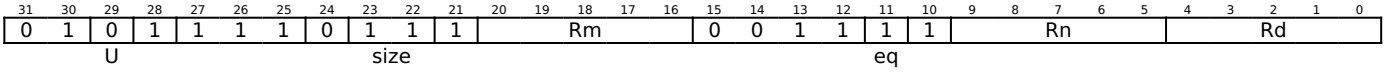
This instruction compares each vector element in the first source SIMD&FP register with the corresponding vector element in the second source SIMD&FP register and if the first signed integer value is greater than or equal to the second signed integer value sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

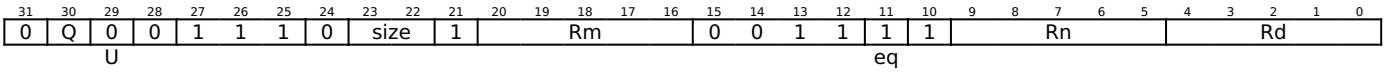
```
CMGE D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMGE <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the first SIMD&FP source register, encoded in the "Rn" field.
<m>	Is the number of the second SIMD&FP source register, encoded in the "Rm" field.
<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<T>	Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    result[e*:esize] = if element1 >= element2 then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMGE (zero)

Compare signed greater than or equal to zero (vector)

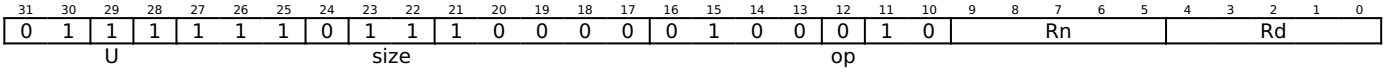
This instruction reads each vector element in the source SIMD&FP register and if the signed integer value is greater than or equal to zero sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

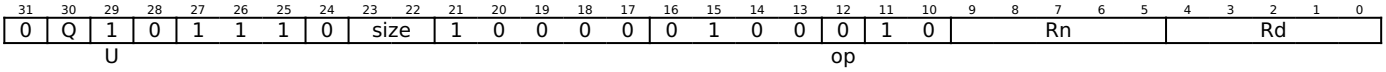
```
CMGE D<d>, D<n>, #0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMGE <Vd>.<T>, <Vn>.<T>, #0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    element = SInt(operand[e*:esize]);
    result[e*:esize] = if element >= 0 then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMGT (register)

Compare signed greater than (vector)

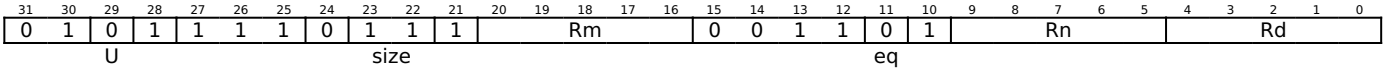
This instruction compares each vector element in the first source SIMD&FP register with the corresponding vector element in the second source SIMD&FP register and if the first signed integer value is greater than the second signed integer value sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

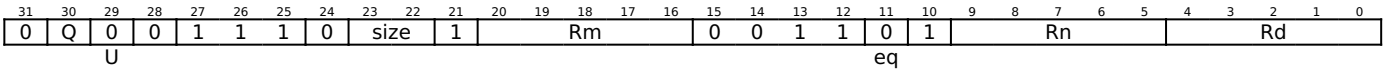
```
CMGT D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMGT <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the first SIMD&FP source register, encoded in the "Rn" field.
<m>	Is the number of the second SIMD&FP source register, encoded in the "Rm" field.
<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<T>	Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    result[e*:esize] = if element1 > element2 then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMGT (zero)

Compare signed greater than zero (vector)

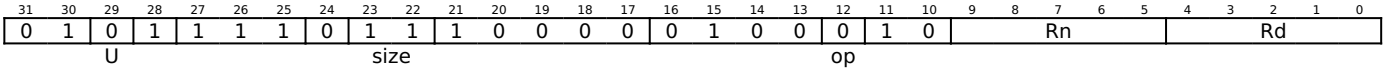
This instruction reads each vector element in the source SIMD&FP register and if the signed integer value is greater than zero sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

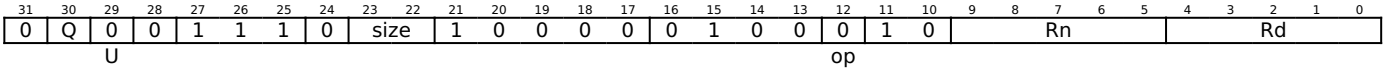
```
CMGT D<d>, D<n>, #0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMGT <Vd>.<T>, <Vn>.<T>, #0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    element = SInt(operand[e*:esize]);
    result[e*:esize] = if element > 0 then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMHI (register)

Compare unsigned higher (vector)

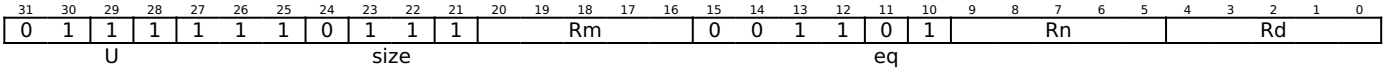
This instruction compares each vector element in the first source SIMD&FP register with the corresponding vector element in the second source SIMD&FP register and if the first unsigned integer value is greater than the second unsigned integer value sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

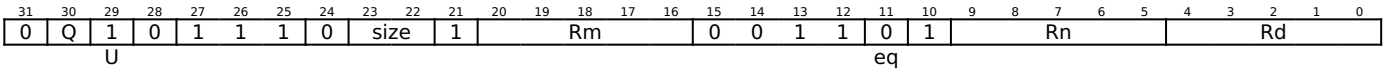
```
CMHI D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMHI <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the first SIMD&FP source register, encoded in the "Rn" field.
<m>	Is the number of the second SIMD&FP source register, encoded in the "Rm" field.
<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<T>	Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    result[e*:esize] = if element1 > element2 then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMHS (register)

Compare unsigned higher or same (vector)

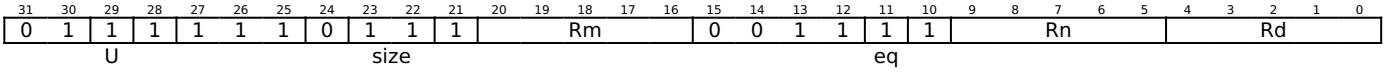
This instruction compares each vector element in the first source SIMD&FP register with the corresponding vector element in the second source SIMD&FP register and if the first unsigned integer value is greater than or equal to the second unsigned integer value sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

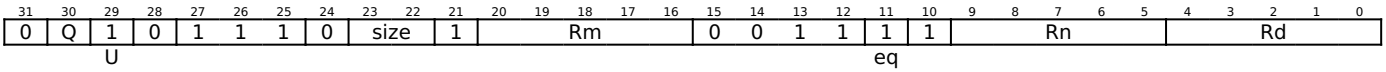
```
CMHS D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMHS <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the first SIMD&FP source register, encoded in the "Rn" field.
<m>	Is the number of the second SIMD&FP source register, encoded in the "Rm" field.
<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<T>	Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    result[e*:esize] = if element1 >= element2 then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMLE (zero)

Compare signed less than or equal to zero (vector)

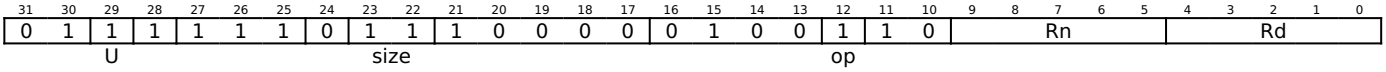
This instruction reads each vector element in the source SIMD&FP register and if the signed integer value is less than or equal to zero sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

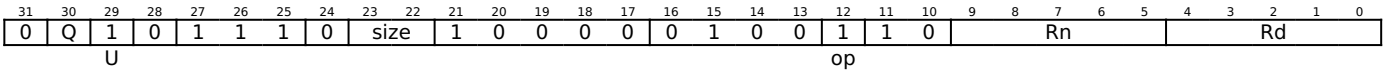
```
CMLE D<d>, D<n>, #0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMLE <Vd>.<T>, <Vn>.<T>, #0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    element = SInt(operand[e*:esize]);
    result[e*:esize] = if element <= 0 then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMLT (zero)

Compare signed less than zero (vector)

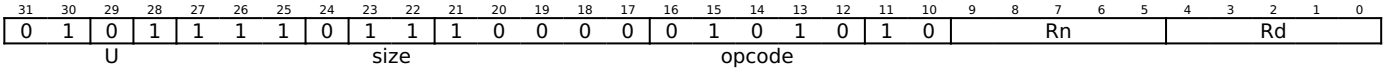
This instruction reads each vector element in the source SIMD&FP register and if the signed integer value is less than zero sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

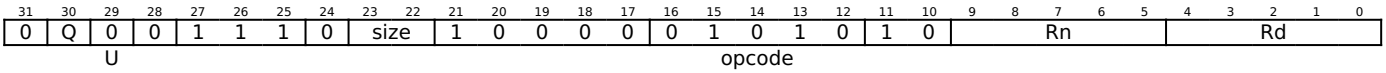
```
CMLT D<d>, D<n>, #0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMLT <Vd>.<T>, <Vn>.<T>, #0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    element = SInt(operand[e*:esize]);
    result[e*:esize] = if element < 0 then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMTST

Compare bitwise test bits nonzero (vector)

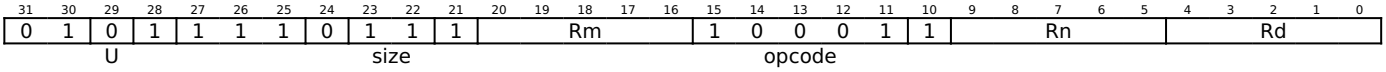
This instruction reads each vector element in the first source SIMD&FP register, performs an AND with the corresponding vector element in the second source SIMD&FP register, and if the result is not zero, sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

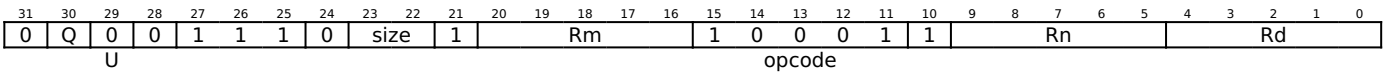
```
CMTST D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
CMTST <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = if !IsZero(element1 AND element2) then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CNT

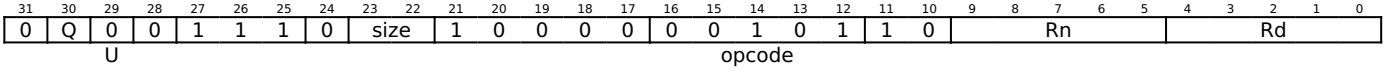
Population count per byte

This instruction determines the number of bits that have a value of one in each vector element in the source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
CNT <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '00' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 8;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	x	RESERVED
1x	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

for e = 0 to elements-1 do
    let count : integer = BitCount(operand[e*:esize]);
    result[e*:esize] = count[esize-1:0];
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DUP (element)

Duplicate vector element to vector or scalar

This instruction duplicates the vector element at the specified element index in the source SIMD&FP register into a scalar or each element in a vector, and writes the result to the destination SIMD&FP register.

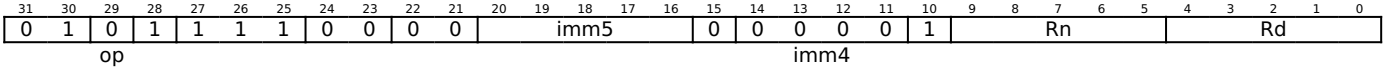
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This instruction is used by the alias MOV (scalar).

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

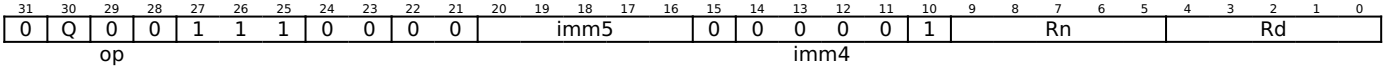
```
DUP <V><d>, <Vn>.<T>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if imm5 == 'x0000' then EndOfDecode(Decode_UNDEF); end;
let size : integer{} = LowestSetBitNZ(imm5[3:0]);

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let index : integer = UInt(imm5[4:size+1]);
let idxdsize : integer{} = 64 << UInt(imm5[4]);
let esize : integer{} = 8 << size;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
DUP <Vd>.<T>, <Vn>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if imm5 == 'x0000' then EndOfDecode(Decode_UNDEF); end;
if imm5 == 'x1000' && Q == '0' then EndOfDecode(Decode_UNDEF); end;
let size : integer{} = LowestSetBitNZ(imm5[3:0]);

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let index : integer = UInt(imm5[4:size+1]);
let idxdsize : integer{} = 64 << UInt(imm5[4]);
let esize : integer{} = 8 << size;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<V> Is the destination width specifier, encoded in “imm5”:

imm5	<V>
x0000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> For the "Scalar" variant: is the element width specifier, encoded in “imm5”:

imm5	<T>
x0000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D

For the "Vector" variant: is an arrangement specifier, encoded in “(imm5 :: Q)”:

imm5	Q	<T>
x0000	x	RESERVED
xxxx1	0	8B
xxxx1	1	16B
xxx10	0	4H
xxx10	1	8H
xx100	0	2S
xx100	1	4S
x1000	0	RESERVED
x1000	1	2D

<index> Is the element index encoded in “imm5”:

imm5	<index>
x0000	RESERVED
xxxx1	UInt(imm5[4:1])
xxx10	UInt(imm5[4:2])
xx100	UInt(imm5[4:3])
x1000	UInt(imm5[4])

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ts> Is an element size specifier, encoded in “imm5”:

imm5	<Ts>
x0000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(idxdsize) = V{idxdsize}(n);
var result : bits(datasize);
let element : bits(esize) = operand[index*:esize];

for e = 0 to elements-1 do
    result[e*:esize] = element;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DUP (general)

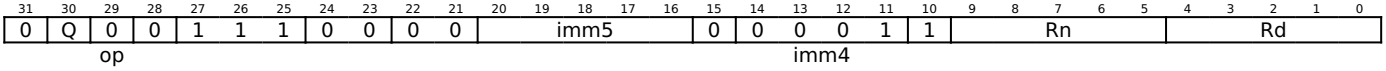
Duplicate general-purpose register to vector

This instruction duplicates the contents of the source general-purpose register into a scalar or each element in a vector, and writes the result to the SIMD&FP destination register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
DUP <Vd>.<T>, <R><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if imm5 == 'x0000' then EndOfDecode(Decode_UNDEF); end;
if imm5 == 'x1000' && Q == '0' then EndOfDecode(Decode_UNDEF); end;
let size : integer{} = LowestSetBitNZ(imm5[3:0]);

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

// imm5[4:size+1] is IGNORED
let esize : integer{} = 8 << size;
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(imm5 :: Q)”:

imm5	Q	<T>
x0000	x	RESERVED
xxxx1	0	8B
xxxx1	1	16B
xxx10	0	4H
xxx10	1	8H
xx100	0	2S
xx100	1	4S
x1000	0	RESERVED
x1000	1	2D

<R> Is the width specifier for the general-purpose source register, encoded in “imm5”:

imm5	<R>
x0000	RESERVED
xxxx1	W
xxx10	W
xx100	W
x1000	X

Unspecified bits in "imm5" are ignored but should be set to zero by an assembler.

<n> Is the number [0-30] of the general-purpose source register or ZR (31), encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let element : bits(esize) = X{esize}(n);
let elements : integer = datasize DIV esize;
var result : bits(datasize);

for e = 0 to elements-1 do
    result[e*:esize] = element;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

EOR (vector)

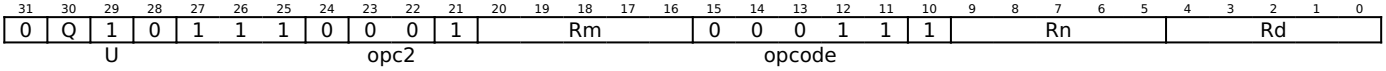
Bitwise exclusive-OR (vector)

This instruction performs a bitwise exclusive-OR between the first source SIMD&FP register and the second source SIMD&FP register, and places the result in the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
EOR <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(datasize) = V{}(m);
let operand2 : bits(datasize) = Zeros{};
let operand3 : bits(datasize) = Ones{};
let operand4 : bits(datasize) = V{}(n);
V{datasize}(d) = operand1 XOR ((operand2 XOR operand4) AND operand3);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

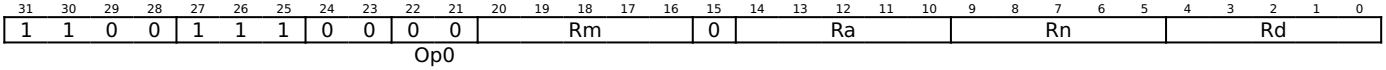
EOR3

Three-way exclusive-OR

This instruction performs a three-way exclusive-OR of the values in the three source SIMD&FP registers, and writes the result to the destination SIMD&FP register.

Advanced SIMD

(FEAT_SHA3)



Encoding

```
EOR3 <Vd>.16B, <Vn>.16B, <Vm>.16B, <Va>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.
<Va>	Is the name of the third SIMD&FP source register, encoded in the "Ra" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(m);
let operand2 : bits(128) = V{}(n);
let operand3 : bits(128) = V{}(a);
V{128}(d) = operand2 XOR operand1 XOR operand3;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

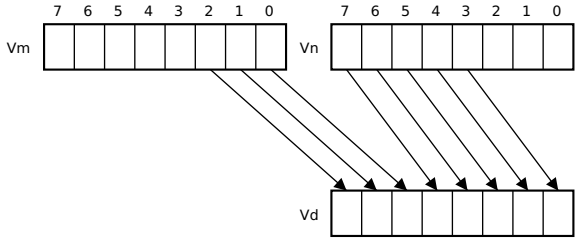
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

EXT

Extract vector from pair of vectors

This instruction extracts the lowest vector elements from the second source SIMD&FP register and the highest vector elements from the first source SIMD&FP register, concatenates the results into a vector, and writes the vector to the destination SIMD&FP register vector. The index value specifies the lowest vector element to extract from the first source register, and consecutive elements are extracted from the first, then second, source registers until the destination vector is filled.

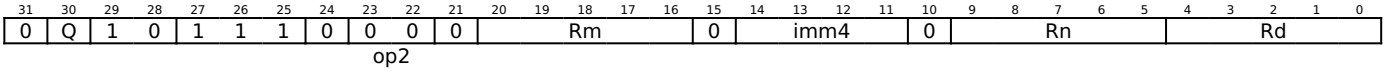
The following figure shows an example of the operation of EXT doubleword operation for Q = 0 and imm4<2:0> = 3.



Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
EXT <Vd>.<T>, <Vn>.<T>, <Vm>.<T>, #<index>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if Q == '0' && imm4[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let datasize : integer{} = 64 << UInt(Q);
let position : integer{} = 8 * UInt(imm4);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

<index> Is the lowest numbered byte element to be extracted, encoded in “(Q :: imm4)”:

Q	imm4[3]	<index>
0	0	UInt(imm4[2:0])
0	1	RESERVED
1	x	UInt(imm4)

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let hi : bits(datasize) = V{datasize}(m);
let lo : bits(datasize) = V{datasize}(n);
let concat : bits(datasize*2) = hi :: lo;

V{datasize}(d) = concat[(position+datasize)-1:position];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

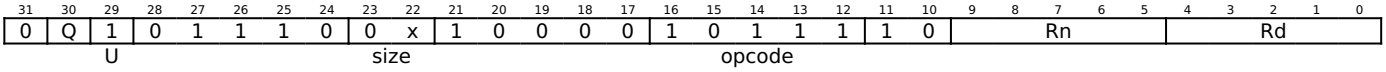
F1CVTL, F1CVTL2, F2CVTL, F2CVTL2

8-bit floating-point convert to half-precision (vector)

This instruction converts each 8-bit floating-point element from the lower or upper half of the source vector to half-precision while downscaling the value, and places the results in the 16-bit elements of the destination vector. `F1CVTL` and `F2CVTL` convert the elements from the lower half of the source vector while scaling the values by $2^{-UInt(FPMR.LSCALE[3:0])}$ and $2^{-UInt(FPMR.LSCALE2[3:0])}$, respectively. `F1CVTL2` and `F2CVTL2` convert the elements from the upper half of the source vector while scaling the values by $2^{-UInt(FPMR.LSCALE[3:0])}$ and $2^{-UInt(FPMR.LSCALE2[3:0])}$, respectively. The 8-bit floating-point encoding format for `F1CVTL` and `F1CVTL2` is selected by FPMR.F8S1. The 8-bit floating-point encoding format for `F2CVTL` and `F2CVTL2` is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_FP8)



Encoding for the F1CVTL{2} variant

Applies when `(size == 00)`

```
F1CVTL{2} <Vd>.8H, <Vn>.<Ta>
```

Encoding for the F2CVTL{2} variant

Applies when `(size == 01)`

```
F2CVTL{2} <Vd>.8H, <Vn>.<Ta>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP8) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let d : integer = UInt(Rd);
let part : integer = UInt(Q);
let elements : integer = 64 DIV 8;
let issrc2 : boolean = size == '01';
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> — Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> — Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	8B
1	16B

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(64) = Vpart{}(n, part);
var result : bits(128);

for e = 0 to elements-1 do
    result[e*:16] = FP8ConvertFP(operand[e*:8], issrc2, FPCR(), FPMR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FABD

Floating-point absolute difference (vector)

This instruction subtracts the floating-point values in the elements of the second source SIMD&FP register, from the corresponding floating-point values in the elements of the first source SIMD&FP register, places the absolute value of each result in a vector, and writes the vector to the destination SIMD&FP register.

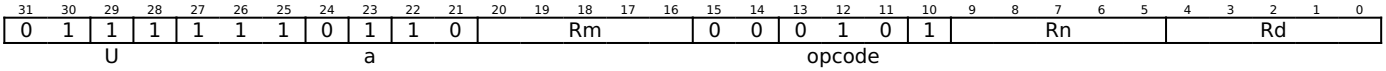
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

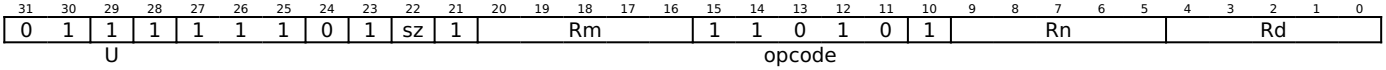
```
FABD <Hd>, <Hn>, <Hm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

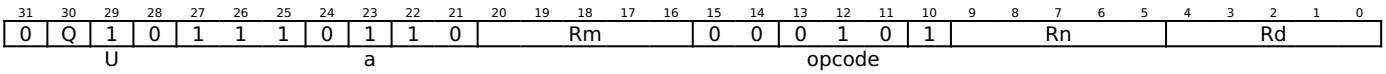
```
FABD <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

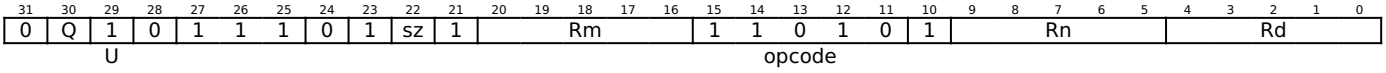
```
FABD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FABD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

var element1 : bits(esize);
var element2 : bits(esize);
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(n) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    let diff : bits(esize) = FPSub{}(element1, element2, FPCR());
    result[e*:esize] = FPAbs{esize}(diff, FPCR());
end;

V{128}(d) = result;
```

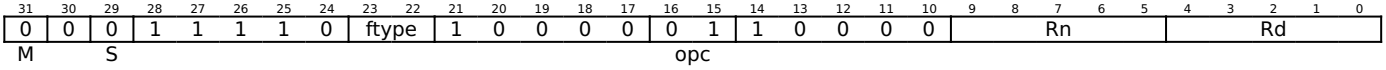
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FABS (scalar)

Floating-point absolute value (scalar)

This instruction calculates the absolute value in the SIMD&FP source register and writes the result to the SIMD&FP destination register. Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FABS <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FABS <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FABS <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPAbs{esize}(operand, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FABS (vector)

Floating-point absolute value (vector)

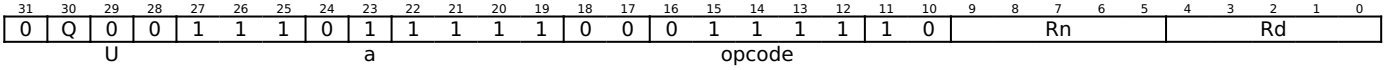
This instruction calculates the absolute value of each vector element in the source SIMD&FP register, writes the result to a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

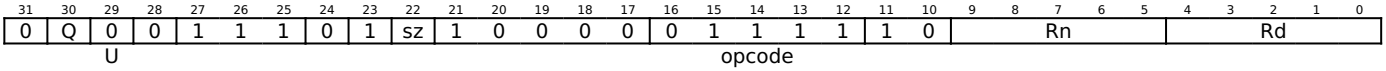
```
FABS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FABS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[e*:esize];
    result[e*:esize] = FPAbs{esize}(element, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FACGE

Floating-point absolute compare greater than or equal (vector)

This instruction compares the absolute value of each floating-point value in the first source SIMD&FP register with the absolute value of the corresponding floating-point value in the second source SIMD&FP register and if the first value is greater than or equal to the second value sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

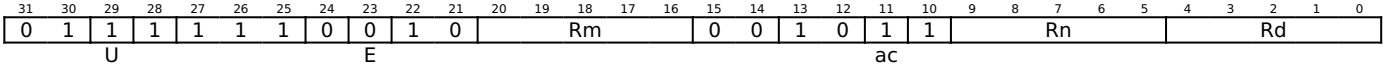
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

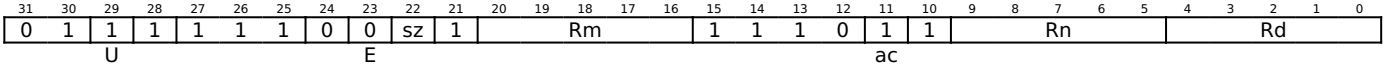
```
FACGE <Hd>, <Hn>, <Hm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

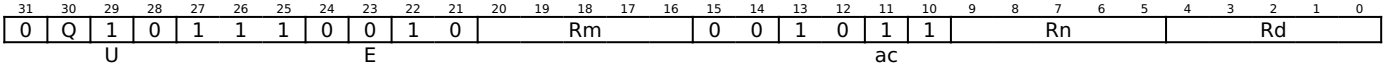
```
FACGE <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

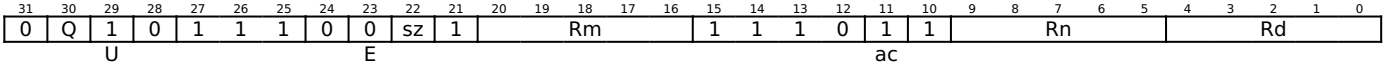
```
FACGE <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FACGE <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

var element1 : bits(esize);
var element2 : bits(esize);
var test_passed : boolean;
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(m) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    element1 = FPAbs{esize}(element1, FPCR());
    element2 = FPAbs{esize}(element2, FPCR());
    test_passed = FPCompareGE{esize}(element1, element2, FPCR());
    result[e*:esize] = if test_passed then Ones{esize} else Zeros{esize};
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FACGT

Floating-point absolute compare greater than (vector)

This instruction compares the absolute value of each vector element in the first source SIMD&FP register with the absolute value of the corresponding vector element in the second source SIMD&FP register and if the first value is greater than the second value sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

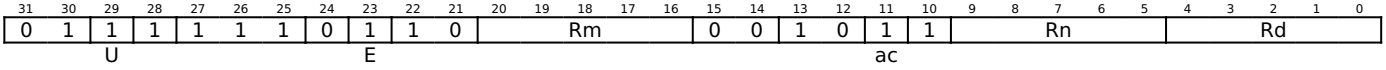
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

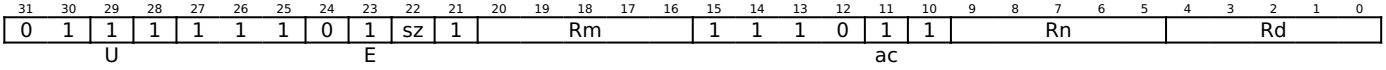
```
FACGT <Hd>, <Hn>, <Hm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

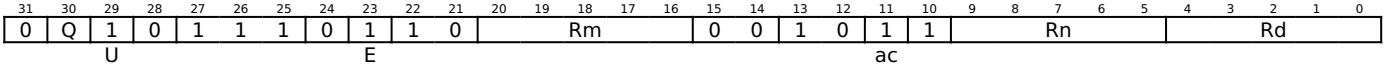
```
FACGT <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

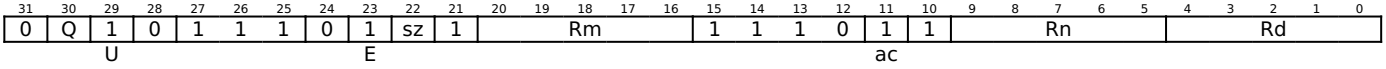
```
FACGT <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FACGT <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

var element1 : bits(esize);
var element2 : bits(esize);
var test_passed : boolean;
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(m) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    element1 = FPAbs{esize}(element1, FPCR());
    element2 = FPAbs{esize}(element2, FPCR());
    test_passed = FPCompareGT{esize}(element1, element2, FPCR());
    result[e*:esize] = if test_passed then Ones{esize} else Zeros{esize};
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

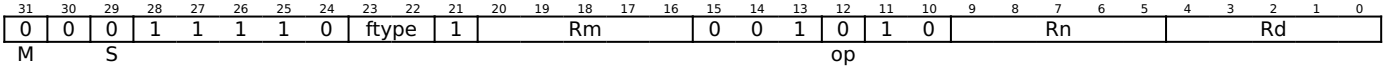
FADD (scalar)

Floating-point add (scalar)

This instruction adds the floating-point values of the two source SIMD&FP registers, and writes the result to the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FADD <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FADD <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FADD <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPAdd{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FADD (vector)

Floating-point add (vector)

This instruction adds corresponding vector elements in the two source SIMD&FP registers, writes the result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

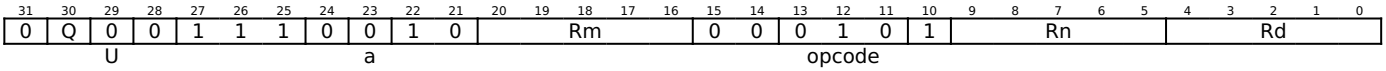
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

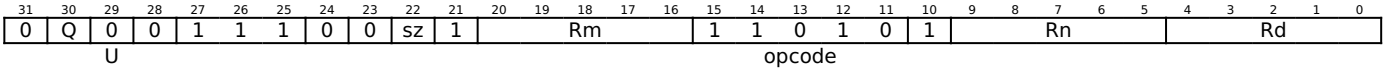
```
FADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPAdd{esize}(element1, element2, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FADDP (scalar)

Floating-point add pair of elements (scalar)

This instruction adds two floating-point vector elements in the source SIMD&FP register and writes the scalar result into the destination SIMD&FP register.

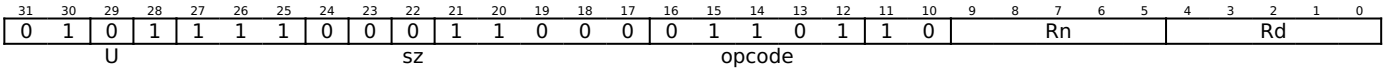
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

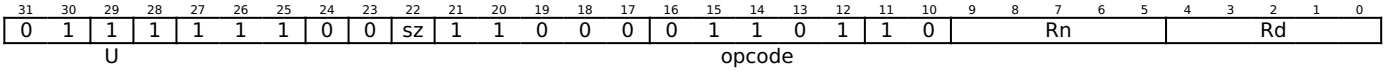
```
FADDP H<d>, <Vn>.2H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 32;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FADDP <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize * 2;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is the destination width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<T> Is the source arrangement specifier, encoded in “sz”:

sz	<T>
0	2S
1	2D

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand :  bits(datasize) = V{}(n);
let element1 :  bits(esize) = operand[0*:esize];
let element2 : bits(esize) = operand[1*:esize];
V{esize}(d) = FPAdd{esize}(element1, element2, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FADDP (vector)

Floating-point add pairwise (vector)

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements from the concatenated vector, adds each pair of values together, places the result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

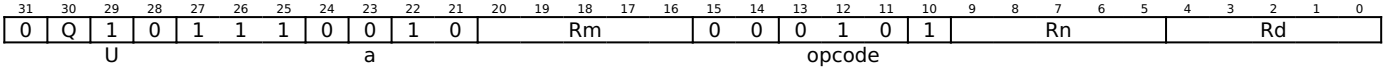
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

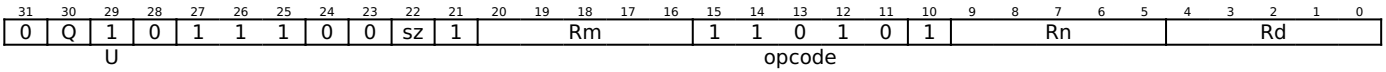
```
FADDP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FADDP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = concat[(2*e)*:esize];
    element2 = concat[((2*e)+1)*:esize];
    result[e*:esize] = FPAdd{esize}(element1, element2, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FAMAX

Floating-point absolute maximum

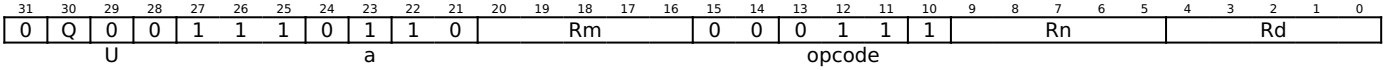
This instruction determines the maximum absolute value from floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and places the results in the corresponding elements of the destination vector.
The behavior is as follows:

- Alternate floating-point behaviors do not apply, as if FPCR.AH is 0.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FAMINMAX)



Encoding

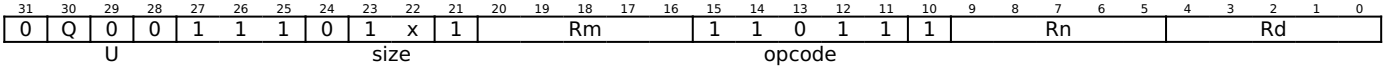
```
FAMAX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD && FEAT_FAMINMAX)



Encoding

```
FAMAX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if Q == '0' && size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(size[0] :: Q)”:

size[0]	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

for e = 0 to elements-1 do
    let op1 : bits(esize) = operand1[e*:esize];
    let op2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPAbsMax{esize}(op1, op2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FAMIN

Floating-point absolute minimum

This instruction determines the minimum absolute value from floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and places the results in the corresponding elements of the destination vector.

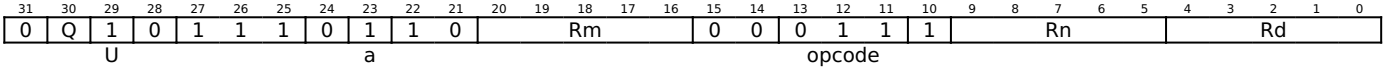
The behavior is as follows:

- Alternate floating-point behaviors do not apply, as if FPCR.AH is 0.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FAMINMAX)



Encoding

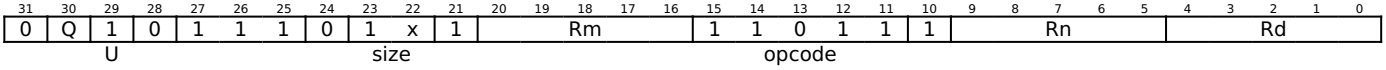
```
FAMIN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD && FEAT_FAMINMAX)



Encoding

```
FAMIN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if Q == '0' && size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(size[0] :: Q)”:

size[0]	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

for e = 0 to elements-1 do
    let op1 : bits(esize) = operand1[e*:esize];
    let op2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPAbsMin{esize}(op1, op2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCADD

Floating-point complex add

This instruction operates on complex numbers that are represented in SIMD&FP registers as pairs of elements, with the more significant element holding the imaginary part of the number and the less significant element holding the real part of the number. Each element holds a floating-point value. It performs the following calculation on the corresponding complex number element pairs from the two source registers:

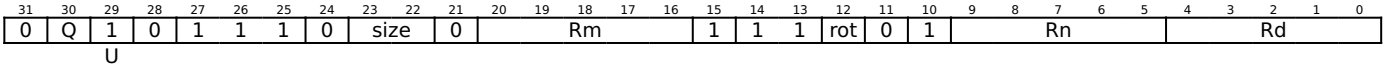
- Considering the complex number from the second source register on an Argand diagram, the number is rotated counterclockwise by 90 or 270 degrees.
- The rotated complex number is added to the complex number from the first source register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_FCMA)



Encoding

```
FCADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>, #<rotate>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FCMA) then EndOfDecode(Decode_UNDEF); end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
if size == '01' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
if Q == '0' && size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

<rotate> Is the rotation, encoded in “rot”:

rot	<rotate>
0	90
1	270

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element3 : bits(esize);

for e = 0 to (elements DIV 2)-1 do
    case rot of
        when '0' =>
            element1 = FPNeg{esize}(operand2[(e*2+1)*:esize], FPCR());
            element3 = operand2[(e*2)*:esize];
        when '1' =>
            element1 = operand2[(e*2+1)*:esize];
            element3 = FPNeg{esize}(operand2[(e*2)*:esize], FPCR());
    end;
    result[(e*2)*:esize] = FPAdd{esize}(operand1[(e*2)*:esize], element1, FPCR());
    result[(e*2+1)*:esize] = FPAdd{esize}(operand1[(e*2+1)*:esize], element3, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCCMP

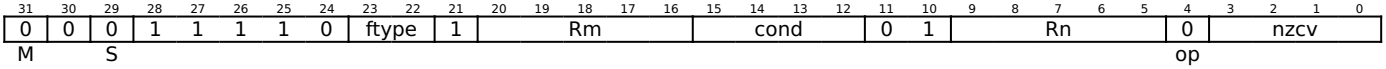
Floating-point conditional quiet compare (scalar)

This instruction, if the condition passes, has the following behavior:

- This instruction compares the two SIMD&FP source register values and writes the result to the *PSTATE*.{N, Z, C, V} flags.
- This instruction raises an Invalid Operation floating-point exception if either or both of the operands is a signaling NaN.
- This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

If the condition does not pass, then the comparison is not performed and the *PSTATE*.{N, Z, C, V} flags are set to the flag bit specifier.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FCCMP <Hn>, <Hm>, #<nzcv>, <cond>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FCCMP <Sn>, <Sm>, #<nzcv>, <cond>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FCCMP <Dn>, <Dm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 8 << UInt(ftype XOR '10');
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
```

Assembler Symbols

<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<nzcv>	Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Sn> Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Sm> Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<Dn> Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Dm> Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();

let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

if ConditionHolds(condition) then
    let signal_all_nans : boolean = FALSE;
    flags = FPCompare{datasize}(operand1, operand2, signal_all_nans, FPCR());
end;

PSTATE.[N,Z,C,V] = flags;
```

Operational information

The IEEE 754 standard specifies that the result of a comparison is precisely one of <, ==, >, or unordered. If either or both of the operands is a NaN, they are unordered, and all three of (Operand1 < Operand2), (Operand1 == Operand2), and (Operand1 > Operand2) are false. An unordered comparison sets the PSTATE condition flags to N=0, Z=0, C=1, and V=1.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

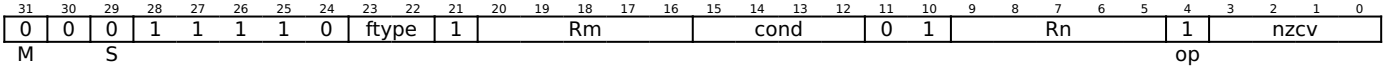
FCCMPE

Floating-point conditional signaling compare (scalar)

This instruction, if the condition passes, has the following behavior:

- This instruction compares the two SIMD&FP source register values and writes the result to the *PSTATE*.{N, Z, C, V} flags.
- This instruction raises an Invalid Operation floating-point exception if either or both of the operands is any type of NaN.
- This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

If the condition does not pass, then the comparison is not performed and the *PSTATE*.{N, Z, C, V} flags are set to the flag bit specifier.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FCCMPE <Hn>, <Hm>, #<nzcv>, <cond>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FCCMPE <Sn>, <Sm>, #<nzcv>, <cond>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FCCMPE <Dn>, <Dm>, #<nzcv>, <cond>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 8 << UInt(ftype XOR '10');
let condition : bits(4) = cond;
var flags : bits(4) = nzcv;
```

Assembler Symbols

<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<nzcv>	Is the flag bit specifier, an immediate in the range 0 to 15, giving the alternative state for the 4-bit NZCV condition flags, encoded in the "nzcv" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Sn> Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Sm> Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<Dn> Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Dm> Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();

let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

if ConditionHolds(condition) then
    let signal_all_nans : boolean = TRUE;
    flags = FPCompare{datasize}(operand1, operand2, signal_all_nans, FPCR());
end;

PSTATE.[N,Z,C,V] = flags;
```

Operational information

The IEEE 754 standard specifies that the result of a comparison is precisely one of <, ==, >, or unordered. If either or both of the operands is a NaN, they are unordered, and all three of (Operand1 < Operand2), (Operand1 == Operand2), and (Operand1 > Operand2) are false. An unordered comparison sets the PSTATE condition flags to N=0, Z=0, C=1, and V=1.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMEQ (register)

Floating-point compare equal (vector)

This instruction compares each floating-point value from the first source SIMD&FP register, with the corresponding floating-point value from the second source SIMD&FP register, and if the comparison is equal sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

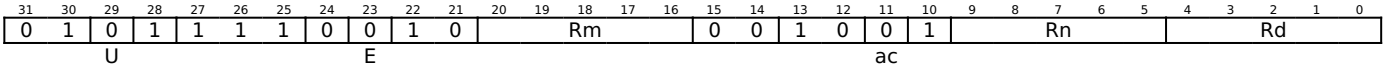
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

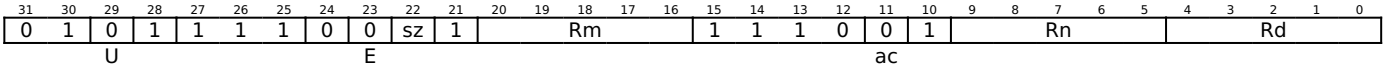
```
FCMEQ <Hd>, <Hn>, <Hm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

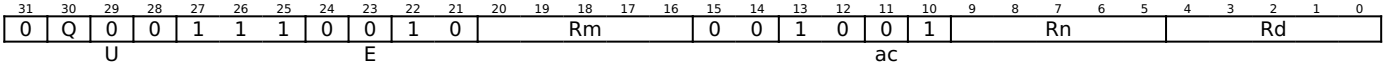
```
FCMEQ <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

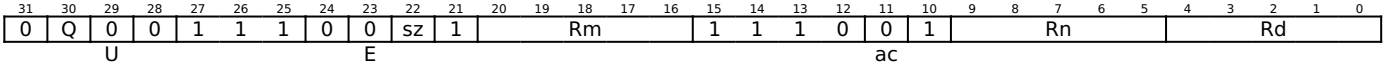
```
FCMEQ <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCMEQ <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

var element1 : bits(esize);
var element2 : bits(esize);
var test_passed : boolean;
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(m) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    test_passed = FPCompareEQ{esize}(element1, element2, FPCR());
    result[e*:esize] = if test_passed then Ones{esize} else Zeros{esize};
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMEQ (zero)

Floating-point compare equal to zero (vector)

This instruction reads each floating-point value in the source SIMD&FP register and if the value is equal to zero sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

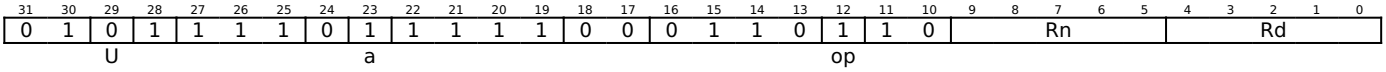
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

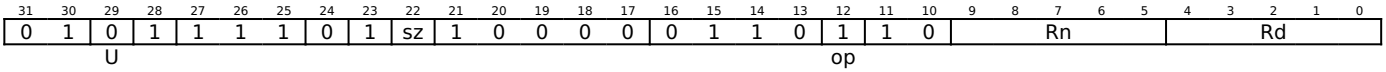
```
FCMEQ <Hd>, <Hn>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

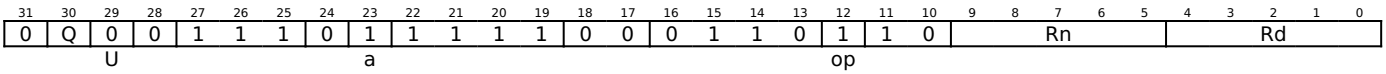
```
FCMEQ <V><d>, <V><n>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

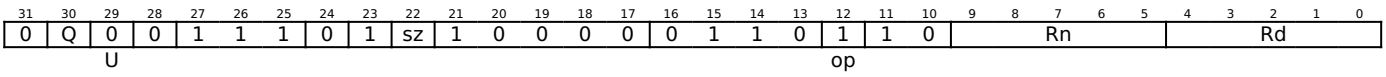
```
FCMEQ <Vd>.<T>, <Vn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCMEQ <Vd>.<T>, <Vn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
let zero : bits(esize) = FPZero{}('0');
var element : bits(esize);
var test_passed : boolean;

for e = 0 to elements-1 do
    element = operand[e*:esize];
    test_passed = FPCompareEQ{esize}(element, zero, FPCR());
    result[e*:esize] = if test_passed then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMGE (register)

Floating-point compare greater than or equal (vector)

This instruction reads each floating-point value in the first source SIMD&FP register and if the value is greater than or equal to the corresponding floating-point value in the second source SIMD&FP register sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

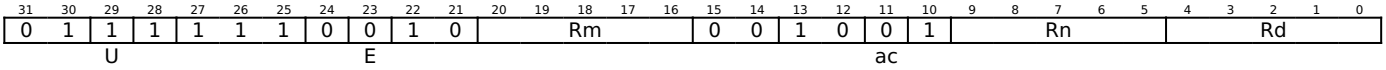
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

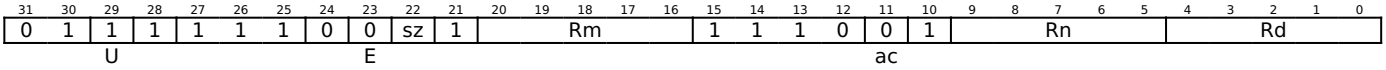
```
FCMGE <Hd>, <Hn>, <Hm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

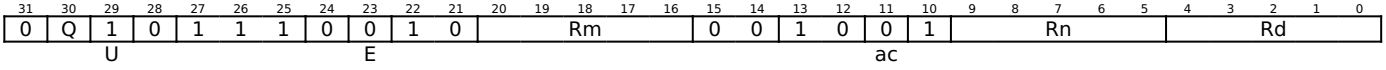
```
FCMGE <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

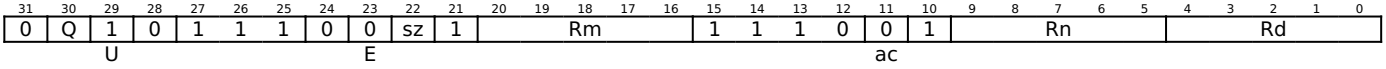
```
FCMGE <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCMGE <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

var element1 : bits(esize);
var element2 : bits(esize);
var test_passed : boolean;
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(m) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    test_passed = FPCompareGE{esize}(element1, element2, FPCR());
    result[e*:esize] = if test_passed then Ones{esize} else Zeros{esize};
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMGE (zero)

Floating-point compare greater than or equal to zero (vector)

This instruction reads each floating-point value in the source SIMD&FP register and if the value is greater than or equal to zero sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

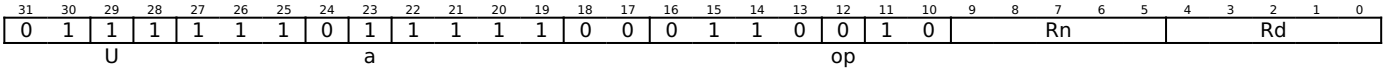
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

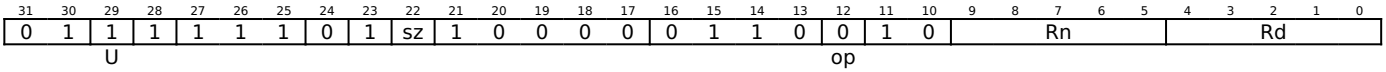
```
FCMGE <Hd>, <Hn>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

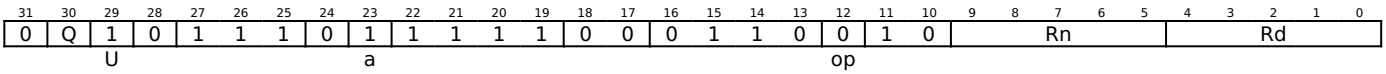
```
FCMGE <V><d>, <V><n>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

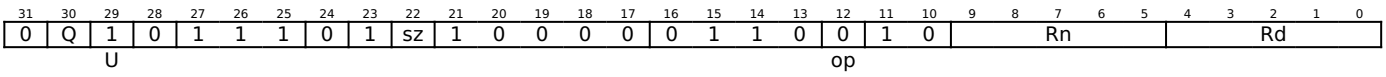
```
FCMGE <Vd>.<T>, <Vn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCMGE <Vd>.<T>, <Vn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
let zero : bits(esize) = FPZero{}('0');
var element : bits(esize);
var test_passed : boolean;

for e = 0 to elements-1 do
    element = operand[e*:esize];
    test_passed = FPCompareGE{esize}(element, zero, FPCR());
    result[e*:esize] = if test_passed then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMGT (register)

Floating-point compare greater than (vector)

This instruction reads each floating-point value in the first source SIMD&FP register and if the value is greater than the corresponding floating-point value in the second source SIMD&FP register sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

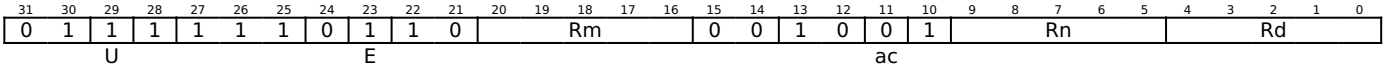
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

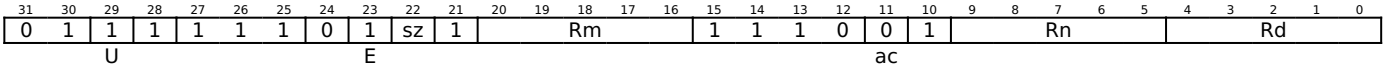
```
FCMGT <Hd>, <Hn>, <Hm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

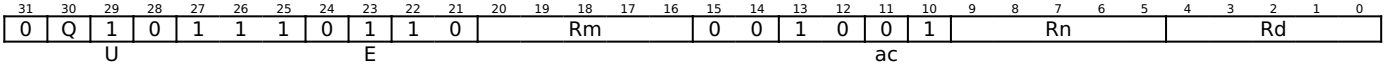
```
FCMGT <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

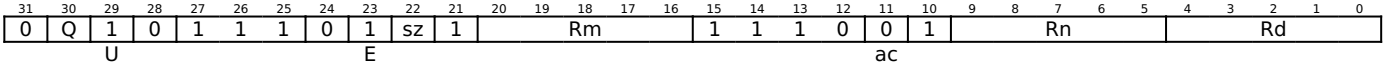
```
FCMGT <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCMGT <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

var element1 : bits(esize);
var element2 : bits(esize);
var test_passed : boolean;
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(m) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    test_passed = FPCompareGT{esize}(element1, element2, FPCR());
    result[e*:esize] = if test_passed then Ones{esize} else Zeros{esize};
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMGT (zero)

Floating-point compare greater than zero (vector)

This instruction reads each floating-point value in the source SIMD&FP register and if the value is greater than zero sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

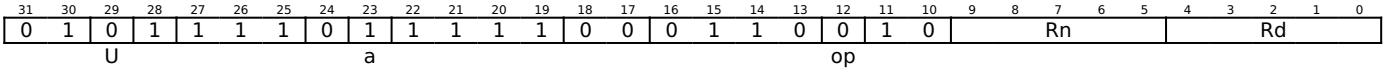
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

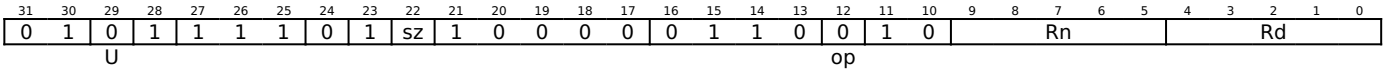
```
FCMGT <Hd>, <Hn>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

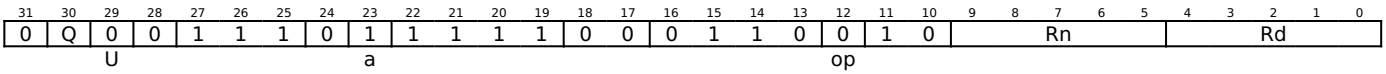
```
FCMGT <V><d>, <V><n>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

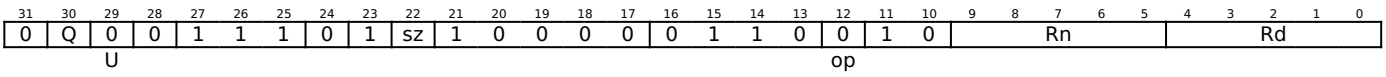
```
FCMGT <Vd>.<T>, <Vn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCMGT <Vd>.<T>, <Vn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
let zero : bits(esize) = FPZero{}('0');
var element : bits(esize);
var test_passed : boolean;

for e = 0 to elements-1 do
    element = operand[e*:esize];
    test_passed = FPCompareGT{esize}(element, zero, FPCR());
    result[e*:esize] = if test_passed then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMLA

Floating-point complex multiply accumulate

This instruction operates on complex numbers that are represented in SIMD&FP registers as pairs of elements, with the more significant element holding the imaginary part of the number and the less significant element holding the real part of the number. Each element holds a floating-point value. It performs the following calculation on the corresponding complex number element pairs from the two source registers and the destination register:

- Considering the complex number from the second source register on an Argand diagram, the number is rotated counterclockwise by 0, 90, 180, or 270 degrees.
- The two elements of the transformed complex number are multiplied by:
  - The real element of the complex number from the first source register, if the transformation was a rotation by 0 or 180 degrees.
  - The imaginary element of the complex number from the first source register, if the transformation was a rotation by 90 or 270 degrees.
- The complex number resulting from that multiplication is added to the complex number from the destination register.

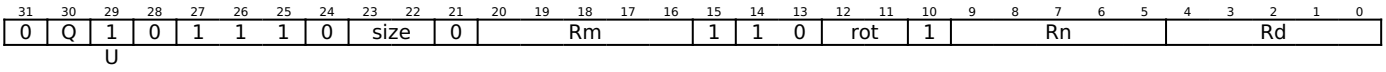
The multiplication and addition operations are performed as a fused multiply-add, without any intermediate rounding.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_FCMA)



Encoding

```
FCMLA <Vd>.<T>, <Vn>.<T>, <Vm>.<T>, #<rotate>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FCMA) then EndOfDecode(Decode_UNDEF); end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_FP16) && size == '01' then EndOfDecode(Decode_UNDEF); end;
if Q == '0' && size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

<rotate> Is the rotation, encoded in “rot”:

rot	<rotate>
00	0
01	90
10	180
11	270

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);
var element3 : bits(esize);
var element4 : bits(esize);

for e = 0 to (elements DIV 2)-1 do
    case rot of
        when '00' =>
            element1 = operand2[(e*2)*:esize];
            element2 = operand1[(e*2)*:esize];
            element3 = operand2[(e*2+1)*:esize];
            element4 = operand1[(e*2)*:esize];
        when '01' =>
            element1 = FPNeg{esize}(operand2[(e*2+1)*:esize], FPCR());
            element2 = operand1[(e*2+1)*:esize];
            element3 = operand2[(e*2)*:esize];
            element4 = operand1[(e*2+1)*:esize];
        when '10' =>
            element1 = FPNeg{esize}(operand2[(e*2)*:esize], FPCR());
            element2 = operand1[(e*2)*:esize];
            element3 = FPNeg{esize}(operand2[(e*2+1)*:esize], FPCR());
            element4 = operand1[(e*2)*:esize];
        when '11' =>
            element1 = operand2[(e*2+1)*:esize];
            element2 = operand1[(e*2+1)*:esize];
            element3 = FPNeg{esize}(operand2[(e*2)*:esize], FPCR());
            element4 = operand1[(e*2+1)*:esize];
    end;

    result[(e*2)*:esize] = FPMulAdd{esize}(operand3[(e*2)*:esize], element2, element1, FPCR());
    result[(e*2+1)*:esize] = FPMulAdd{esize}(operand3[(e*2+1)*:esize], element4, element3, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMLA (by element)

Floating-point complex multiply accumulate (by element)

This instruction operates on complex numbers that are represented in SIMD&FP registers as pairs of elements, with the more significant element holding the imaginary part of the number and the less significant element holding the real part of the number. Each element holds a floating-point value. It performs the following calculation on complex numbers from the first source register and the destination register with the specified complex number from the second source register:

- Considering the complex number from the second source register on an Argand diagram, the number is rotated counterclockwise by 0, 90, 180, or 270 degrees.
- The two elements of the transformed complex number are multiplied by:
  - The real element of the complex number from the first source register, if the transformation was a rotation by 0 or 180 degrees.
  - The imaginary element of the complex number from the first source register, if the transformation was a rotation by 90 or 270 degrees.
- The complex number resulting from that multiplication is added to the complex number from the destination register.

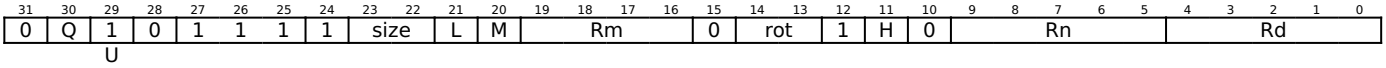
The multiplication and addition operations are performed as a fused multiply-add, without any intermediate rounding.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_FCMA)



Encoding

```
FCMLA <Vd>.<T>, <Vn>.<T>, <Vm>.<Ts>[<index>], #<rotate>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FCMA) then EndOfDecode(Decode_UNDEF); end;
if size == '00' || size == '11' then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_FP16) && size == '01' then EndOfDecode(Decode_UNDEF); end;
if size == '10' && (L == '1' || Q == '0') then EndOfDecode(Decode_UNDEF); end;
if size == '01' && H == '1' && Q == '0' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(M::Rm);
var index : integer;
if size == '01' then index = UInt(H::L); end;
if size == '10' then index = UInt(H); end;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	RESERVED
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L)”:

size	<index>
00	RESERVED
01	UInt(H :: L)
10	UInt(H)
11	RESERVED

<rotate> Is the rotation, encoded in “rot”:

rot	<rotate>
00	0
01	90
10	180
11	270

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to (elements DIV 2)-1 do
    var element1 : bits(esize);
    var element2 : bits(esize);
    var element3 : bits(esize);
    var element4 : bits(esize);
    case rot of
        when '00' =>
            element1 = operand2[(index*2)*:esize];
            element2 = operand1[(e*2)*:esize];
            element3 = operand2[(index*2+1)*:esize];
            element4 = operand1[(e*2)*:esize];
        when '01' =>
            element1 = FPNeg{esize}(operand2[(index*2+1)*:esize], FPCR());
            element2 = operand1[(e*2+1)*:esize];
            element3 = operand2[(index*2)*:esize];
            element4 = operand1[(e*2+1)*:esize];
        when '10' =>
            element1 = FPNeg{esize}(operand2[(index*2)*:esize], FPCR());
            element2 = operand1[(e*2)*:esize];
            element3 = FPNeg{esize}(operand2[(index*2+1)*:esize], FPCR());
            element4 = operand1[(e*2)*:esize];
        when '11' =>
            element1 = operand2[(index*2+1)*:esize];
            element2 = operand1[(e*2+1)*:esize];
            element3 = FPNeg{esize}(operand2[(index*2)*:esize], FPCR());
            element4 = operand1[(e*2+1)*:esize];
    end;

    result[(e*2)*:esize] = FPMulAdd{esize}(operand3[(e*2)*:esize], element2, element1, FPCR());
    result[(e*2+1)*:esize] = FPMulAdd{esize}(operand3[(e*2+1)*:esize], element4, element3, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMLE (zero)

Floating-point compare less than or equal to zero (vector)

This instruction reads each floating-point value in the source SIMD&FP register and if the value is less than or equal to zero sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

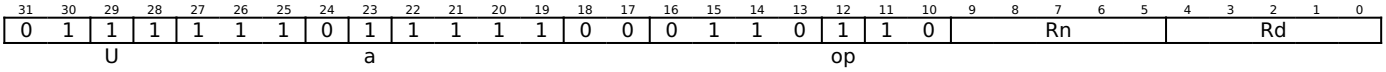
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

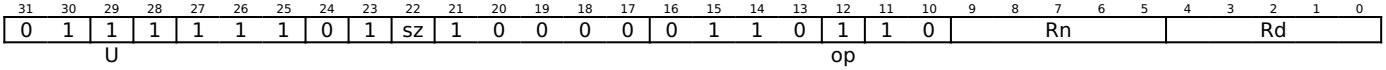
```
FCMLE <Hd>, <Hn>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

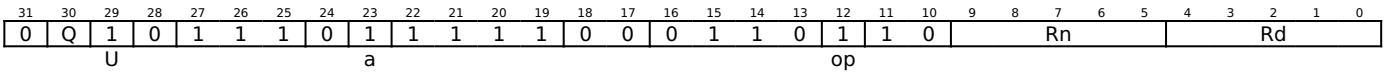
```
FCMLE <V><d>, <V><n>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

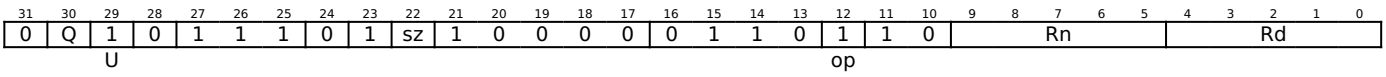
```
FCMLE <Vd>.<T>, <Vn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCMLE <Vd>.<T>, <Vn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
let zero : bits(esize) = FPZero{}('0');
var element : bits(esize);
var test_passed : boolean;

for e = 0 to elements-1 do
    element = operand[e*:esize];
    test_passed = FPCompareGE{esize}(zero, element, FPCR());
    result[e*:esize] = if test_passed then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMLT (zero)

Floating-point compare less than zero (vector)

This instruction reads each floating-point value in the source SIMD&FP register and if the value is less than zero sets every bit of the corresponding vector element in the destination SIMD&FP register to one, otherwise sets every bit of the corresponding vector element in the destination SIMD&FP register to zero.

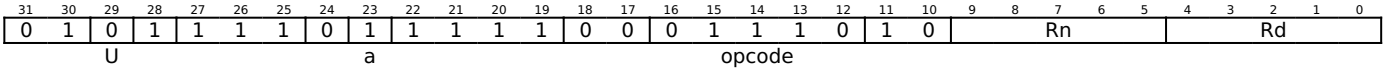
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

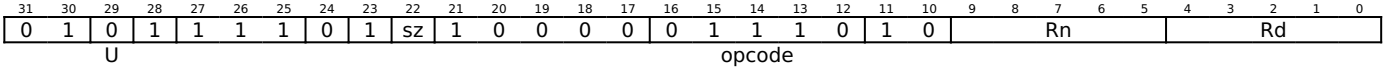
```
FCMLT <Hd>, <Hn>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

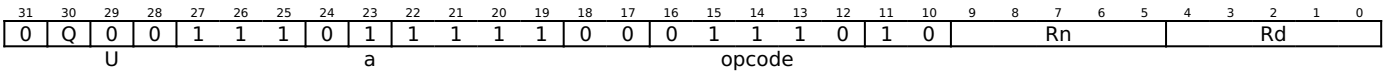
```
FCMLT <V><d>, <V><n>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

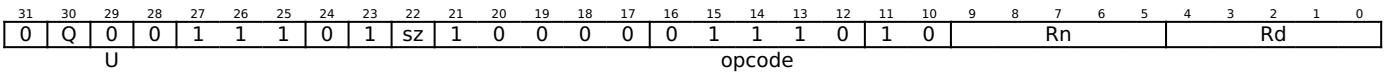
```
FCMLT <Vd>.<T>, <Vn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCMLT <Vd>.<T>, <Vn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
let zero : bits(esize) = FPZero{}('0');
var element : bits(esize);
var test_passed : boolean;

for e = 0 to elements-1 do
    element = operand[e*:esize];
    test_passed = FPCompareGT{esize}(zero, element, FPCR());
    result[e*:esize] = if test_passed then Ones{esize} else Zeros{esize};
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMP

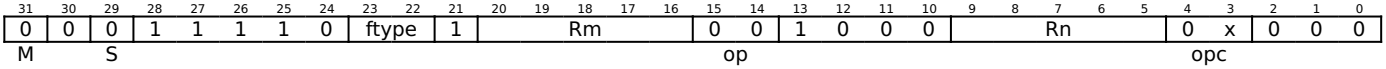
Floating-point quiet compare (scalar)

This instruction compares the two SIMD&FP source register values, or the first SIMD&FP source register value and zero. It writes the result to the *PSTATE*.{N, Z, C, V} flags.

This instruction raises an Invalid Operation floating-point exception if either or both of the operands is a signaling NaN.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant
(FEAT_FP16)

Applies when `(ftype == 11 && opc == 00)`

```
FCMP <Hn>, <Hm>
```

Encoding for the Half-precision, zero variant
(FEAT_FP16)

Applies when `(ftype == 11 && Rm == (00000) && opc == 01)`

```
FCMP <Hn>, #0.0
```

Encoding for the Single-precision variant
(FEAT_FP)

Applies when `(ftype == 00 && opc == 00)`

```
FCMP <Sn>, <Sm>
```

Encoding for the Single-precision, zero variant
(FEAT_FP)

Applies when `(ftype == 00 && Rm == (00000) && opc == 01)`

```
FCMP <Sn>, #0.0
```

Encoding for the Double-precision variant
(FEAT_FP)

Applies when `(ftype == 01 && opc == 00)`

```
FCMP <Dn>, <Dm>
```

Encoding for the Double-precision, zero variant
(FEAT_FP)

Applies when `(ftype == 01 && Rm == (00000) && opc == 01)`

```
FCMP <Dn>, #0.0
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let m : integer = UInt(Rm);   // ignored when opc<0> == '1'

let datasize : integer{} = 8 << UInt(ftype XOR '10');
let signal_all_nans : boolean = FALSE;
let cmp_with_zero : boolean = (opc[0] == '1');
```

Assembler Symbols

<Hn>	For the "Half-precision" variant: is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field. For the "Half-precision, zero" variant: is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sn>	For the "Single-precision" variant: is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

	For the "Single-precision, zero" variant: is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dn>	For the "Double-precision" variant: is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
	For the "Double-precision, zero" variant: is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();

let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = if cmp_with_zero then FPZero{datasize}('0') else V{datasize}(m);

PSTATE.[N,Z,C,V] = FPCompare{datasize}(operand1, operand2, signal_all_nans, FPCR());
```

Operational information

The IEEE 754 standard specifies that the result of a comparison is precisely one of <, ==, >, or unordered. If either or both of the operands is a NaN, they are unordered, and all three of (Operand1 < Operand2), (Operand1 == Operand2), and (Operand1 > Operand2) are false. An unordered comparison sets the PSTATE condition flags to N=0, Z=0, C=1, and V=1.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMPE

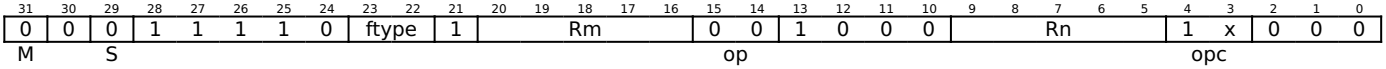
Floating-point signaling compare (scalar)

This instruction compares the two SIMD&FP source register values, or the first SIMD&FP source register value and zero. It writes the result to the *PSTATE*.{N, Z, C, V} flags.

This instruction raises an Invalid Operation floating-point exception if either or both of the operands is any type of NaN.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant
(FEAT_FP16)

Applies when `(ftype == 11 && opc == 10)`

```
FCMPE <Hn>, <Hm>
```

Encoding for the Half-precision, zero variant
(FEAT_FP16)

Applies when `(ftype == 11 && Rm == (00000) && opc == 11)`

```
FCMPE <Hn>, #0.0
```

Encoding for the Single-precision variant
(FEAT_FP)

Applies when `(ftype == 00 && opc == 10)`

```
FCMPE <Sn>, <Sm>
```

Encoding for the Single-precision, zero variant
(FEAT_FP)

Applies when `(ftype == 00 && Rm == (00000) && opc == 11)`

```
FCMPE <Sn>, #0.0
```

Encoding for the Double-precision variant
(FEAT_FP)

Applies when `(ftype == 01 && opc == 10)`

```
FCMPE <Dn>, <Dm>
```

Encoding for the Double-precision, zero variant
(FEAT_FP)

Applies when `(ftype == 01 && Rm == (00000) && opc == 11)`

```
FCMPE <Dn>, #0.0
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let m : integer = UInt(Rm);   // ignored when opc<0> == '1'

let datasize : integer{} = 8 << UInt(ftype XOR '10');
let signal_all_nans : boolean = TRUE;
let cmp_with_zero : boolean = (opc[0] == '1');
```

Assembler Symbols

<Hn>	For the "Half-precision" variant: is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field. For the "Half-precision, zero" variant: is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sn>	For the "Single-precision" variant: is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

	For the "Single-precision, zero" variant: is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dn>	For the "Double-precision" variant: is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
	For the "Double-precision, zero" variant: is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();

let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = if cmp_with_zero then FPZero{datasize}('0') else V{datasize}(m);

PSTATE.[N,Z,C,V] = FPCompare{datasize}(operand1, operand2, signal_all_nans, FPCR());
```

Operational information

The IEEE 754 standard specifies that the result of a comparison is precisely one of <, ==, >, or unordered. If either or both of the operands is a NaN, they are unordered, and all three of (Operand1 < Operand2), (Operand1 == Operand2), and (Operand1 > Operand2) are false. An unordered comparison sets the PSTATE condition flags to N=0, Z=0, C=1, and V=1.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

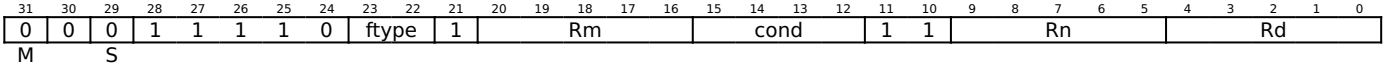
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCSEL

Floating-point conditional select (scalar)

This instruction allows the SIMD&FP destination register to take the value from either one or the other of two SIMD&FP source registers. If the condition passes, the first SIMD&FP source register value is taken, otherwise the second SIMD&FP source register value is taken.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FCSEL <Hd>, <Hn>, <Hm>, <cond>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FCSEL <Sd>, <Sn>, <Sm>, <cond>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FCSEL <Dd>, <Dn>, <Dm>, <cond>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let datasize : integer{} = 8 << UInt(ftype XOR '10');
let condition : bits(4) = cond;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<cond> Is one of the standard conditions, encoded in the standard way, and encoded in “cond”:

cond	<cond>
0000	EQ
0001	NE
0010	CS|HS
0011	CC|LO
0100	MI
0101	PL
0110	VS
0111	VC
1000	HI
1001	LS
1010	GE
1011	LT
1100	GT
1101	LE
1110	AL
1111	NV

<Sd> Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Sn> Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
var result : bits(datasize);

let condition_holds : boolean = ConditionHolds(condition);
result = if condition_holds then V{datasize}(n) else V{datasize}(m);

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVT

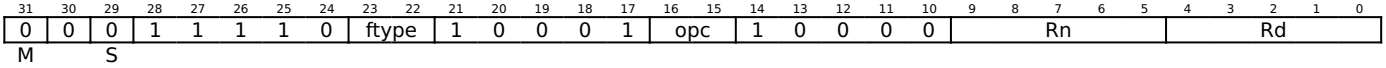
Floating-point convert precision (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to the precision for the destination register data type using the rounding mode that is determined by the FPCR and writes the result to the SIMD&FP destination register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FP)



Encoding for the Half-precision to single-precision variant

Applies when `(ftype == 11 && opc == 00)`

```
FCVT <Sd>, <Hn>
```

Encoding for the Half-precision to double-precision variant

Applies when `(ftype == 11 && opc == 01)`

```
FCVT <Dd>, <Hn>
```

Encoding for the Single-precision to half-precision variant

Applies when `(ftype == 00 && opc == 11)`

```
FCVT <Hd>, <Sn>
```

Encoding for the Single-precision to double-precision variant

Applies when `(ftype == 00 && opc == 01)`

```
FCVT <Dd>, <Sn>
```

Encoding for the Double-precision to half-precision variant

Applies when `(ftype == 01 && opc == 11)`

```
FCVT <Hd>, <Dn>
```

Encoding for the Double-precision to single-precision variant

Applies when `(ftype == 01 && opc == 00)`

```
FCVT <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == opc || ftype == '10' || opc == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let srcsize : integer{} = 8 << UInt(ftype XOR '10');
let dstsize : integer{} = 8 << UInt(opc XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(srcsize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:dstsize] = FPConvert{dstsize, srcsize}(operand, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTAS (scalar SIMD&FP)

Floating-point convert to signed integer, rounding to nearest with ties to away (scalar SIMD&FP)

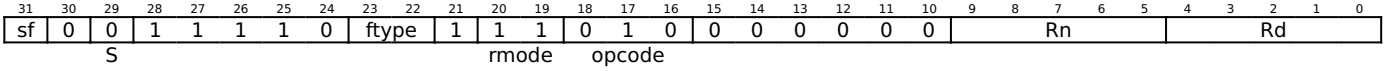
This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit signed integer using the Round to Nearest with Ties to Away rounding mode, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FPRCVT)



Encoding for the Half-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 11)`

```
FCVTAS <Sd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 11)`

```
FCVTAS <Dd>, <Hn>
```

Encoding for the Single-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 00)`

```
FCVTAS <Dd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 01)`

```
FCVTAS <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

fltval = V{fltsize}(n);
intval = FPToFixed{intsize, fltsize}(fltval, 0, FALSE, FPCR(), FPRounding_TIEAWAY);
result[0+:intsize] = intval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

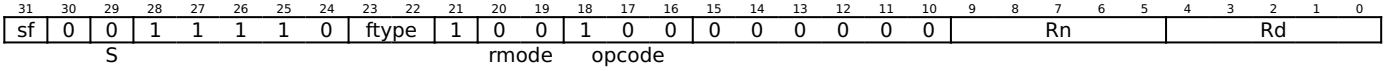
FCVTAS (scalar)

Floating-point convert to signed integer, rounding to nearest with ties to away (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit signed integer using the Round to Nearest with Ties to Away rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTAS <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTAS <Xd>, <Hn>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTAS <Wd>, <Sn>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTAS <Xd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTAS <Wd>, <Dn>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTAS <Xd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');

let rounding : FPRounding = FPRounding_TIEAWAY;
let unsigned : boolean    = FALSE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let fltval : bits(fltsize) = V{}(n);
let fracbits : integer = 0;

X{intsize}(d) = FPToFixed{intsize, fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTAS (vector)

Floating-point convert to signed integer, rounding to nearest with ties to away (vector)

This instruction converts each element in a vector from a floating-point value to a signed integer value using the Round to Nearest with Ties to Away rounding mode and writes the result to the SIMD&FP destination register.

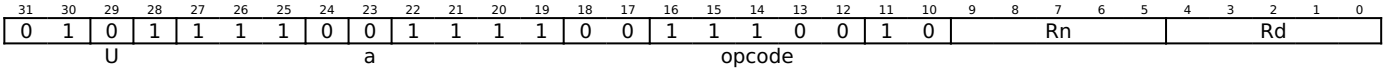
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

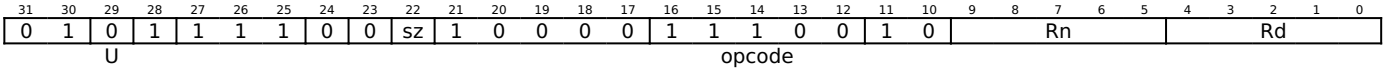
```
FCVTAS <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPRounding_TIEAWAY;
let unsigned : boolean = FALSE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

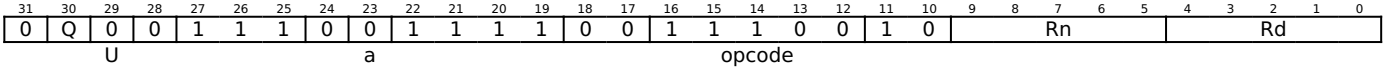
```
FCVTAS <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPRounding_TIEAWAY;
let unsigned : boolean = FALSE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

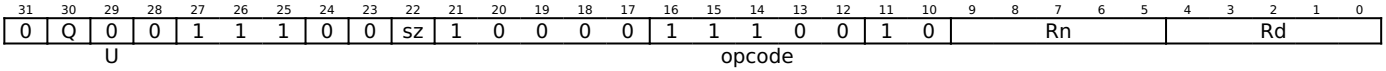
```
FCVTAS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPRounding_TIEAWAY;
let unsigned : boolean = FALSE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTAS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPRounding_TIEAWAY;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTAU (scalar SIMD&FP)

Floating-point convert to unsigned integer, rounding to nearest with ties to away (scalar SIMD&FP)

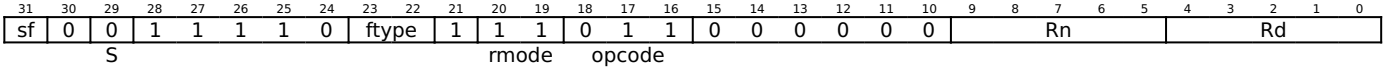
This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit unsigned integer using the Round to Nearest with Ties to Away rounding mode, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FPRCVT)



Encoding for the Half-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 11)`

```
FCVTAU <Sd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 11)`

```
FCVTAU <Dd>, <Hn>
```

Encoding for the Single-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 00)`

```
FCVTAU <Dd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 01)`

```
FCVTAU <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

fltval = V{fltsize}(n);
intval = FPToFixed{intsize, fltsize}(fltval, 0, TRUE, FPCR(), FPRounding_TIEAWAY);
result[0+:intsize] = intval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

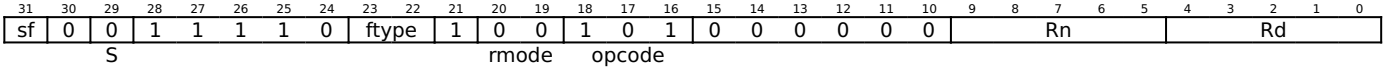
FCVTAU (scalar)

Floating-point convert to unsigned integer, rounding to nearest with ties to away (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit unsigned integer using the Round to Nearest with Ties to Away rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTAU <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTAU <Xd>, <Hn>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTAU <Wd>, <Sn>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTAU <Xd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTAU <Wd>, <Dn>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTAU <Xd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');

let rounding : FPRounding = FPRounding_TIEAWAY;
let unsigned : boolean    = TRUE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltval : bits(fltsize) = V{}(n);
let fracbits : integer = 0;

X{intsize}(d) = FPToFixed{intsize, fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTAU (vector)

Floating-point convert to unsigned integer, rounding to nearest with ties to away (vector)

This instruction converts each element in a vector from a floating-point value to an unsigned integer value using the Round to Nearest with Ties to Away rounding mode and writes the result to the SIMD&FP destination register.

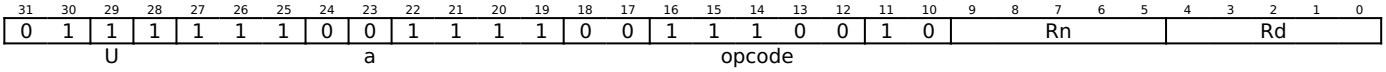
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

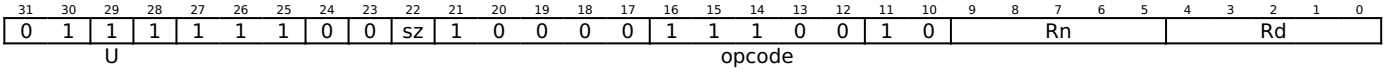
```
FCVTAU <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPRounding_TIEAWAY;
let unsigned : boolean = TRUE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

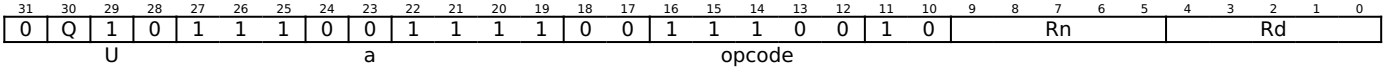
```
FCVTAU <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPRounding_TIEAWAY;
let unsigned : boolean = TRUE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

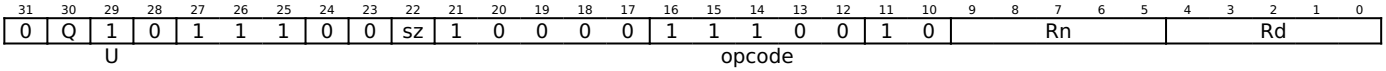
```
FCVTAU <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPRounding_TIEAWAY;
let unsigned : boolean = TRUE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTAU <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPRounding_TIEAWAY;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTL, FCVTL2

Floating-point convert to higher precision long (vector)

This instruction reads each element in a vector in the SIMD&FP source register, converts each value to double the precision of the source element using the rounding mode that is determined by the FPCR, and writes each result to the equivalent element of the vector in the SIMD&FP destination register.

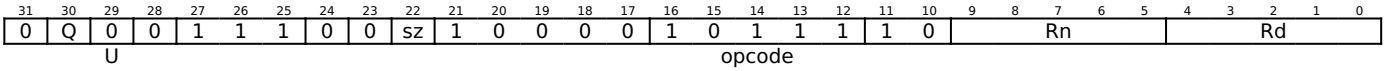
Where the operation lengthens a 64-bit vector to a 128-bit vector, the `FCVTL2` variant operates on the elements in the top 64 bits of the source register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTL{2} <Vd>.<Ta>, <Vn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 16 << UInt(sz);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “sz”:

sz	<Ta>
0	4S
1	2D

<Vn> — Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<Tb>
0	0	4H
0	1	8H
1	0	2S
1	1	4S

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = Vpart{}(n, part);
var result : bits(2*datasize);

for e = 0 to elements-1 do
    result[e*:(2*esize)] = FPConvert{2*esize, esize}(operand[e*:esize], FPCR());
end;

V{2*datasize}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTMS (scalar SIMD&FP)

Floating-point convert to signed integer, rounding toward minus infinity (scalar SIMD&FP)

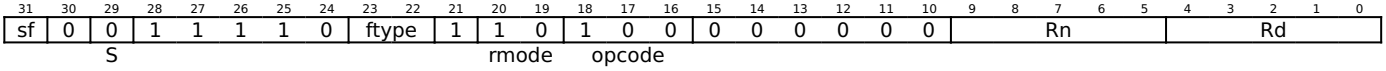
This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit signed integer using the Round toward Minus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FPRCVT)



Encoding for the Half-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 11)`

```
FCVTMS <Sd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 11)`

```
FCVTMS <Dd>, <Hn>
```

Encoding for the Single-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 00)`

```
FCVTMS <Dd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 01)`

```
FCVTMS <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

fltval = V{fltsize}(n);
intval = FPToFixed{intsize, fltsize}(fltval, 0, FALSE, FPCR(), FPRounding_NEGINF);
result[0+:intsize] = intval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

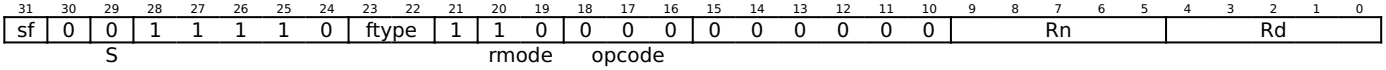
FCVTMS (scalar)

Floating-point convert to signed integer, rounding toward minus infinity (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit signed integer using the Round towards Minus Infinity rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTMS <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTMS <Xd>, <Hn>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTMS <Wd>, <Sn>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTMS <Xd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTMS <Wd>, <Dn>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTMS <Xd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');

let rounding : FPRounding = FPRounding_NEGINF;
let unsigned : boolean    = FALSE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltval : bits(fltsize) = V{}(n);
let fracbits : integer = 0;

X{intsize}(d) = FPToFixed{intsize, fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTMS (vector)

Floating-point convert to signed integer, rounding toward minus infinity (vector)

This instruction converts a scalar or each element in a vector from a floating-point value to a signed integer value using the Round towards Minus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

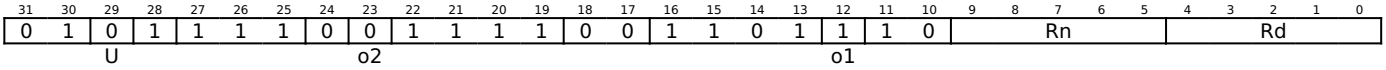
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

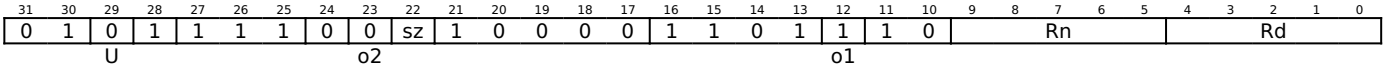
```
FCVTMS <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

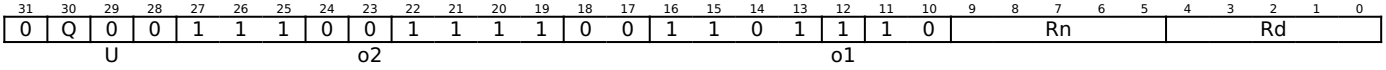
```
FCVTMS <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

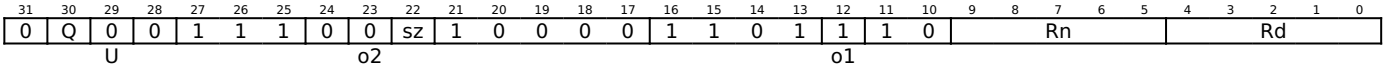
```
FCVTMS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTMS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTMU (scalar SIMD&FP)

Floating-point convert to unsigned integer, rounding toward minus infinity (scalar SIMD&FP)

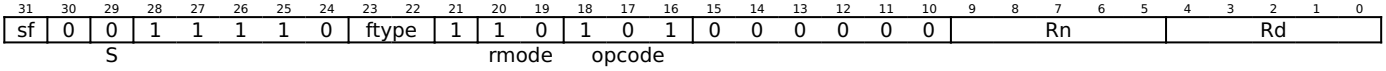
This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit unsigned integer using the Round toward Minus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FPRCVT)



Encoding for the Half-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 11)`

```
FCVTMU <Sd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 11)`

```
FCVTMU <Dd>, <Hn>
```

Encoding for the Single-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 00)`

```
FCVTMU <Dd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 01)`

```
FCVTMU <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

fltval = V{fltsize}(n);
intval = FPToFixed{intsize, fltsize}(fltval, 0, TRUE, FPCR(), FPRounding_NEGINF);
result[0+:intsize] = intval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

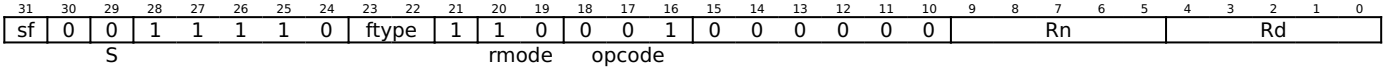
FCVTMU (scalar)

Floating-point convert to unsigned integer, rounding toward minus infinity (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit unsigned integer using the Round towards Minus Infinity rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTMU <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTMU <Xd>, <Hn>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTMU <Wd>, <Sn>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTMU <Xd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTMU <Wd>, <Dn>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTMU <Xd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');

let rounding : FPRounding = FPRounding_NEGINF;
let unsigned : boolean    = TRUE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltval : bits(fltsize) = V{}(n);
let fracbits : integer = 0;

X{intsize}(d) = FPToFixed{intsize, fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTMU (vector)

Floating-point convert to unsigned integer, rounding toward minus infinity (vector)

This instruction converts a scalar or each element in a vector from a floating-point value to an unsigned integer value using the Round towards Minus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

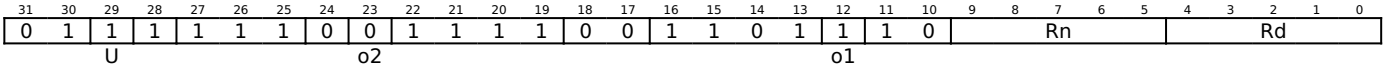
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

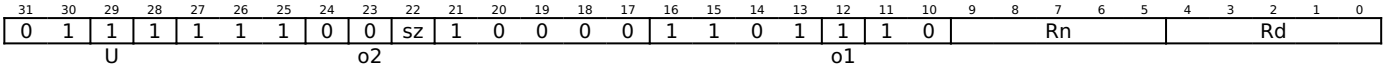
```
FCVTMU <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

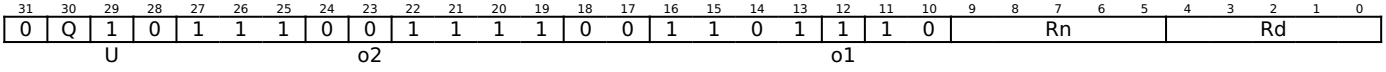
```
FCVTMU <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

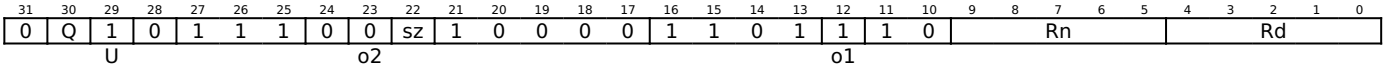
```
FCVTMU <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTMU <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTN (half-precision to 8-bit floating-point)

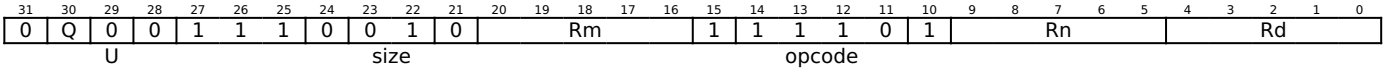
Half-precision convert to 8-bit floating-point (vector)

This instruction converts half-precision elements of the two source vectors to 8-bit floating-point while scaling the values by $2^{\mathrm{SInt(FPMR.NSCALE[4:0])}}$, and places the in-order results in the 8-bit elements of the destination vector.

The 8-bit floating-point encoding format is selected by FPMR.F8D.

Advanced SIMD

(FEAT_FP8)



Encoding

```
FCVTN <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP8) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV 16;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	8B
1	16B

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	4H
1	8H

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

for e = 0 to elements-1 do
    result[(0*elements + e)*:8] = FPConvertFP8{8, 16}(operand1[e*:16], FPCR(), FPMR());
    result[(1*elements + e)*:8] = FPConvertFP8{8, 16}(operand2[e*:16], FPCR(), FPMR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTN, FCVTN2 (double to single-precision, single to half-precision)

Floating-point convert to lower precision narrow (vector)

This instruction reads each vector element in the SIMD&FP source register, converts each result to half the precision of the source element, writes the final result to a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. The destination vector elements are half as long as the source vector elements. The rounding mode is determined by the FPCR.

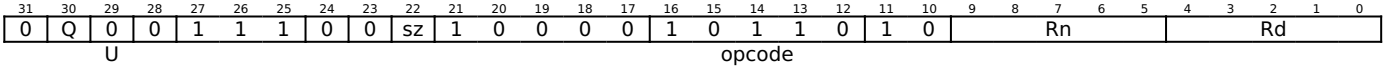
FCVTN writes the vector to the lower half of the destination register and clears the upper half. FCVTN2 writes the vector to the upper half of the destination register without affecting the other bits of the register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTN{2} <Vd>.<Tb>, <Vn>.<Ta>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 16 << UInt(sz);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> — Is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<Tb>
0	0	4H
0	1	8H
1	0	2S
1	1	4S

<Vn> — Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> — Is an arrangement specifier, encoded in “sz”:

sz	<Ta>
0	4S
1	2D

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(2*datasize) = V{}(n);
var result : bits(datasize);

for e = 0 to elements-1 do
    result[e*:esize] = FPConvert{esize, 2*esize}(operand[e*:(2*esize)], FPCR());
end;

Vpart{datasize}(d, part) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTN, FCVTN2 (single-precision to 8-bit floating-point)

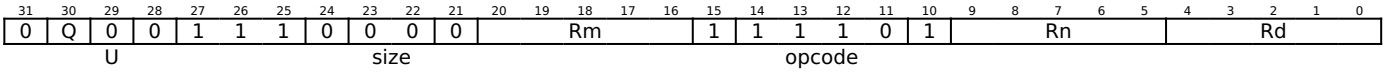
Single-precision convert to 8-bit floating-point (vector)

This instruction converts each single-precision element of the two source vectors to 8-bit floating-point while scaling the value by $2^{\mathrm{SInt(FPMR.NSCALE)}}$, and places the in-order results in the 8-bit elements of the lower or upper half of the destination vector. `FCVTN` writes the results to the lower half of the destination vector and clears the upper half. `FCVTN2` writes the results to the upper half of the destination vector without affecting the other bits of the vector.

The 8-bit floating-point encoding format is selected by FPMR.F8D.

Advanced SIMD

(FEAT_FP8)



Encoding

```
FCVTN{2} <Vd>.<Ta>, <Vn>.4S, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP8) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let part : integer = UInt(Q);
let elements : integer = 128 DIV 32;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	8B
1	16B

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
var result : bits(64);

for e = 0 to elements-1 do
    result[(0*elements + e)*:8] = FPConvertFP8{8, 32}(operand1[e*:32], FPCR(), FPMR());
    result[(1*elements + e)*:8] = FPConvertFP8{8, 32}(operand2[e*:32], FPCR(), FPMR());
end;

Vpart{64}(d, part) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTNS (scalar SIMD&FP)

Floating-point convert to signed integer, rounding to nearest with ties to even (scalar SIMD&FP)

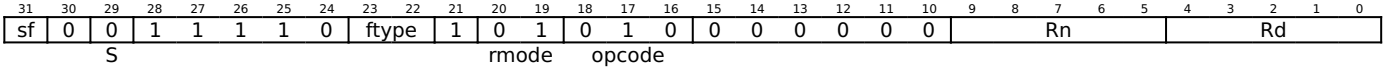
This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit signed integer using the Round to Nearest with Ties to Even rounding mode, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FPRCVT)



Encoding for the Half-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 11)`

```
FCVTNS <Sd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 11)`

```
FCVTNS <Dd>, <Hn>
```

Encoding for the Single-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 00)`

```
FCVTNS <Dd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 01)`

```
FCVTNS <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

fltval = V{fltsize}(n);
intval = FPToFixed{intsize, fltsize}(fltval, 0, FALSE, FPCR(), FPRounding_TIEEVEN);
result[0+:intsize] = intval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

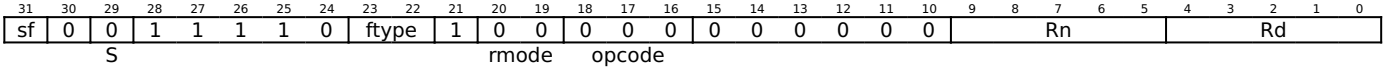
FCVTNS (scalar)

Floating-point convert to signed integer, rounding to nearest with ties to even (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit signed integer using the Round to Nearest rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTNS <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTNS <Xd>, <Hn>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTNS <Wd>, <Sn>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTNS <Xd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTNS <Wd>, <Dn>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTNS <Xd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');

let rounding : FPRounding = FPRounding_TIEEVEN;
let unsigned : boolean    = FALSE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltval : bits(fltsize) = V{}(n);
let fracbits : integer = 0;

X{intsize}(d) = FPToFixed{intsize, fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTNS (vector)

Floating-point convert to signed integer, rounding to nearest with ties to even (vector)

This instruction converts a scalar or each element in a vector from a floating-point value to a signed integer value using the Round to Nearest rounding mode, and writes the result to the SIMD&FP destination register.

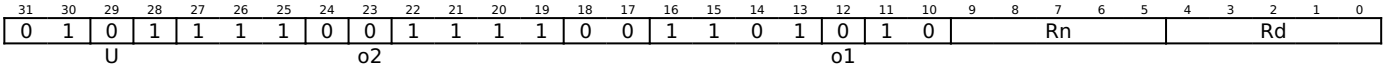
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

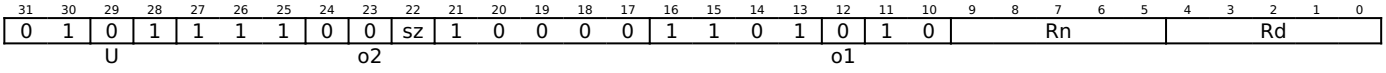
```
FCVTNS <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

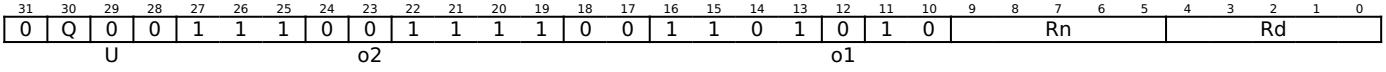
```
FCVTNS <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

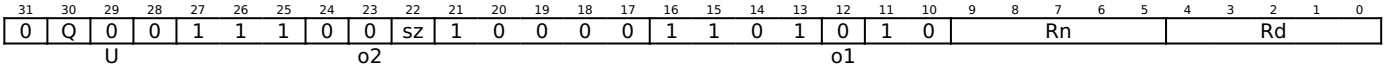
```
FCVTNS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTNS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTNU (scalar SIMD&FP)

Floating-point convert to unsigned integer, rounding to nearest with ties to even (scalar SIMD&FP)

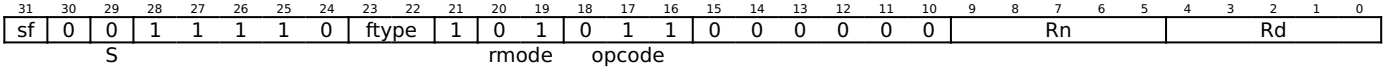
This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit unsigned integer using the Round to Nearest with Ties to Even rounding mode, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FPRCVT)



Encoding for the Half-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 11)`

```
FCVTNU <Sd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 11)`

```
FCVTNU <Dd>, <Hn>
```

Encoding for the Single-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 00)`

```
FCVTNU <Dd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 01)`

```
FCVTNU <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

fltval = V{fltsize}(n);
intval = FPToFixed{intsize, fltsize}(fltval, 0, TRUE, FPCR(), FPRounding_TIEEVEN);
result[0+:intsize] = intval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

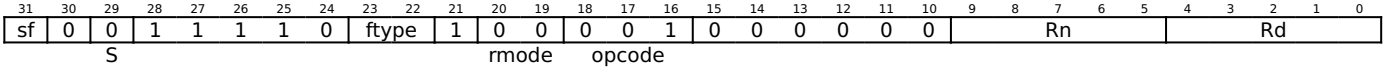
FCVTNU (scalar)

Floating-point convert to unsigned integer, rounding to nearest with ties to even (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit unsigned integer using the Round to Nearest rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTNU <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTNU <Xd>, <Hn>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTNU <Wd>, <Sn>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTNU <Xd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTNU <Wd>, <Dn>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTNU <Xd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');

let rounding : FPRounding = FPRounding_TIEEVEN;
let unsigned : boolean    = TRUE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltval : bits(fltsize) = V{}(n);
let fracbits : integer = 0;

X{intsize}(d) = FPToFixed{intsize, fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTNU (vector)

Floating-point convert to unsigned integer, rounding to nearest with ties to even (vector)

This instruction converts a scalar or each element in a vector from a floating-point value to an unsigned integer value using the Round to Nearest rounding mode, and writes the result to the SIMD&FP destination register.

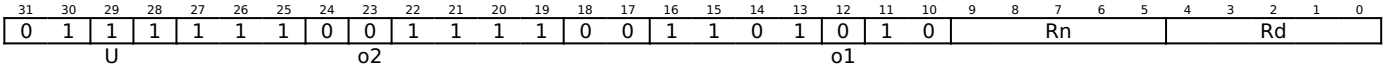
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

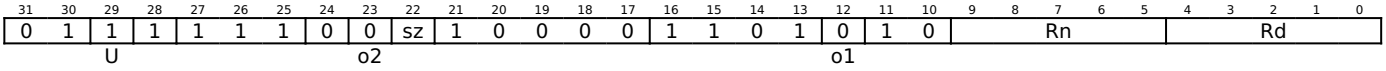
```
FCVTNU <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

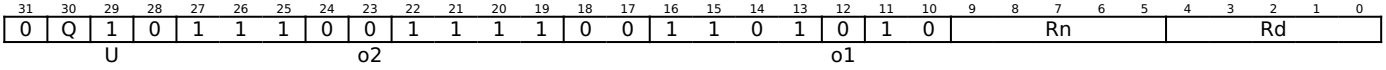
```
FCVTNU <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

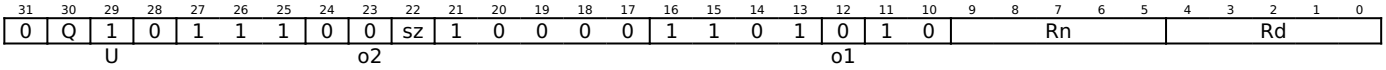
```
FCVTNU <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTNU <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTPS (scalar SIMD&FP)

Floating-point convert to signed integer, rounding toward plus infinity (scalar SIMD&FP)

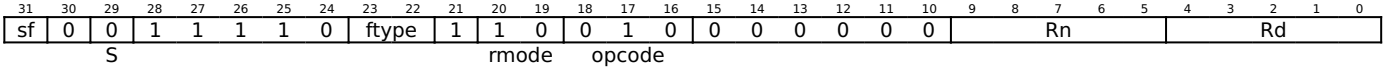
This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit signed integer using the Round toward Plus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FPRCVT)



Encoding for the Half-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 11)`

```
FCVTPS <Sd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 11)`

```
FCVTPS <Dd>, <Hn>
```

Encoding for the Single-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 00)`

```
FCVTPS <Dd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 01)`

```
FCVTPS <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

fltval = V{fltsize}(n);
intval = FPToFixed{intsize, fltsize}(fltval, 0, FALSE, FPCR(), FPRounding_POSINF);
result[0+:intsize] = intval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

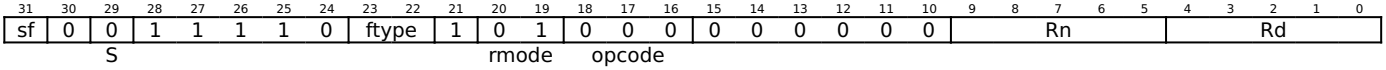
FCVTPS (scalar)

Floating-point convert to signed integer, rounding toward plus infinity (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit signed integer using the Round towards Plus Infinity rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTPS <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTPS <Xd>, <Hn>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTPS <Wd>, <Sn>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTPS <Xd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTPS <Wd>, <Dn>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTPS <Xd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');

let rounding : FPRounding = FPRounding_POSINF;
let unsigned : boolean    = FALSE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltval : bits(fltsize) = V{}(n);
let fracbits : integer = 0;

X{intsize}(d) = FPToFixed{intsize, fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTPS (vector)

Floating-point convert to signed integer, rounding toward plus infinity (vector)

This instruction converts a scalar or each element in a vector from a floating-point value to a signed integer value using the Round towards Plus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

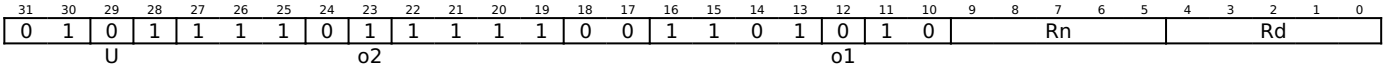
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

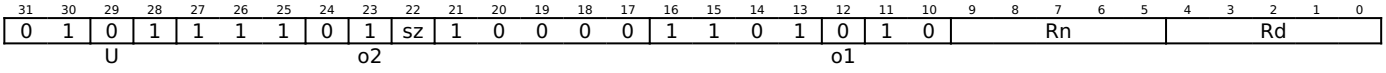
```
FCVTPS <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

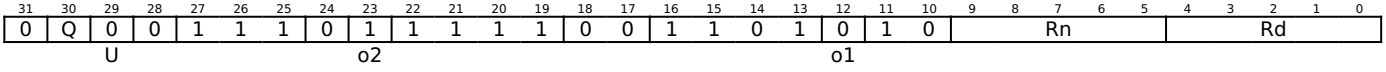
```
FCVTPS <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

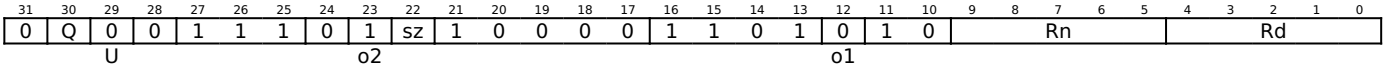
```
FCVTPS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTPS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTPU (scalar SIMD&FP)

Floating-point convert to unsigned integer, rounding toward plus infinity (scalar SIMD&FP)

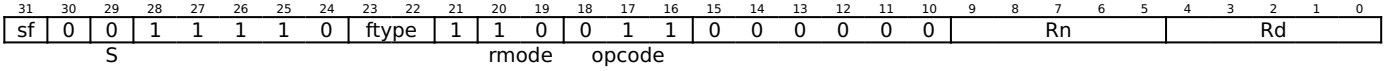
This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit unsigned integer using the Round toward Plus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FPRCVT)



Encoding for the Half-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 11)`

```
FCVTPU <Sd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 11)`

```
FCVTPU <Dd>, <Hn>
```

Encoding for the Single-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 00)`

```
FCVTPU <Dd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 01)`

```
FCVTPU <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

fltval = V{fltsize}(n);
intval = FPToFixed{intsize, fltsize}(fltval, 0, TRUE, FPCR(), FPRounding_POSINF);
result[0+:intsize] = intval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

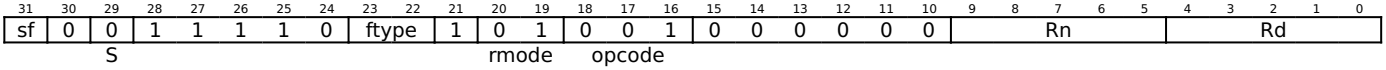
FCVTPU (scalar)

Floating-point convert to unsigned integer, rounding toward plus infinity (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit unsigned integer using the Round towards Plus Infinity rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTPU <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTPU <Xd>, <Hn>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTPU <Wd>, <Sn>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTPU <Xd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTPU <Wd>, <Dn>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTPU <Xd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');

let rounding : FPRounding = FPRounding_POSINF;
let unsigned : boolean    = TRUE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltval : bits(fltsize) = V{}(n);
let fracbits : integer = 0;

X{intsize}(d) = FPToFixed{intsize, fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTPU (vector)

Floating-point convert to unsigned integer, rounding toward plus infinity (vector)

This instruction converts a scalar or each element in a vector from a floating-point value to an unsigned integer value using the Round towards Plus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

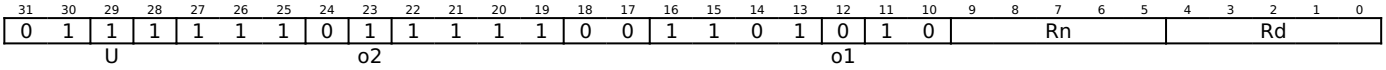
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

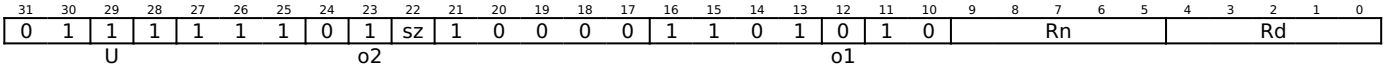
```
FCVTPU <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

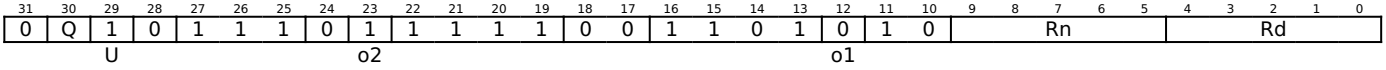
```
FCVTPU <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

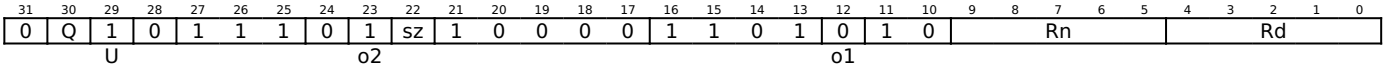
```
FCVTPU <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTPU <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTXN, FCVTXN2

Floating-point convert to lower precision narrow, rounding to odd (vector)

This instruction reads each double-precision element in the source SIMD&FP register, converts each value to single-precision format using the Round to Odd rounding mode, and writes the results to the destination SIMD&FP register.

Note

This instruction uses the Round to Odd rounding mode, which is not defined by the IEEE 754-2008 standard. This rounding mode ensures that if the result of the conversion is inexact, the least significant bit of the mantissa is forced to 1. This rounding mode enables a floating-point value to be converted to a lower precision format using an intermediate precision format while avoiding double rounding errors. For example, a 64-bit floating-point value can be converted to a correctly rounded 16-bit floating-point value by first using this instruction to produce a 32-bit value and then using another instruction with the wanted rounding mode to convert the 32-bit value to the final 16-bit floating-point value.

The Vector variant of `FCVTXN` writes the half-width results to the lower half of the destination register and clears the upper half to 0. `FCVTXN2` writes the half-width results to the upper half of the destination register without affecting the other bits of the register.

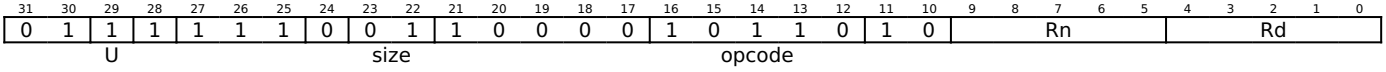
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

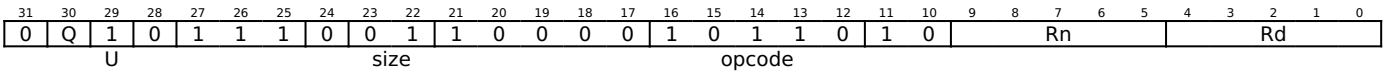
```
FCVTXN S<d>, D<n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32;
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
FCVTXN{2} <Vd>.<Tb>, <Vn>.2D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32;
let datasize : integer{} = 64;
let elements : integer = 2;
let part : integer = UInt(Q);
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the SIMD&FP source register, encoded in the "Rn" field.

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	2S
1	4S

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand : bits(2*datasize) = V{}(n);
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

for e = 0 to elements-1 do
    result[e*:esize] = FPConvert{esize, 2*esize}(operand[e*:(2*esize)], FPCR(), FPRounding_ODD);
end;

if merge then
    V{128}(d) = result;
else
    Vpart{datasize}(d, part) = result[0+:datasize];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZS (scalar SIMD&FP)

Floating-point convert to signed integer, rounding toward zero (scalar SIMD&FP)

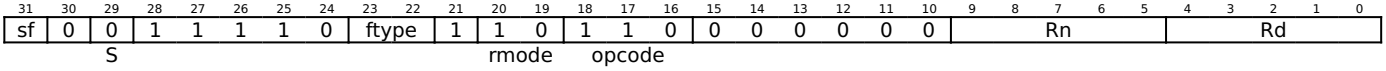
This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit signed integer using the Round toward Zero rounding mode, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FPRCVT)



Encoding for the Half-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 11)`

```
FCVTZS <Sd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 11)`

```
FCVTZS <Dd>, <Hn>
```

Encoding for the Single-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 00)`

```
FCVTZS <Dd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 01)`

```
FCVTZS <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

fltval = V{fltsize}(n);
intval = FPToFixed{intsize, fltsize}(fltval, 0, FALSE, FPCR(), FPRounding_ZERO);
result[0+:intsize] = intval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

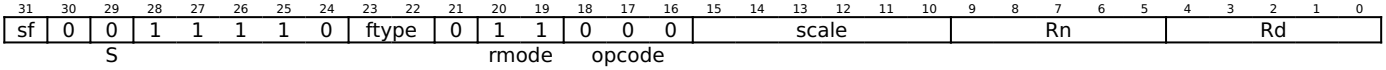
FCVTZS (scalar, fixed-point)

Floating-point convert to signed fixed-point, rounding toward zero (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit fixed-point signed integer using the Round towards Zero rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTZS <Wd>, <Hn>, #<fbits>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTZS <Xd>, <Hn>, #<fbits>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTZS <Wd>, <Sn>, #<fbits>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTZS <Xd>, <Sn>, #<fbits>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTZS <Wd>, <Dn>, #<fbits>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTZS <Xd>, <Dn>, #<fbits>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && scale[5] == '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let decode_fltsize : integer{} = 8 << UInt(ftype XOR '10');

let fracbits : integer = 64 - UInt(scale);

let rounding : FPRounding = FPRounding_ZERO;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<fbits>	For the "Double-precision to 32-bit", "Half-precision to 32-bit", and "Single-precision to 32-bit" variants: is the number of bits after the binary point in the fixed-point destination, in the range 1 to 32, encoded as 64 minus "scale". For the "Double-precision to 64-bit", "Half-precision to 64-bit", and "Single-precision to 64-bit" variants: is the number of bits after the binary point in the fixed-point destination, in the range 1 to 64, encoded as 64 minus "scale".
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let fltval : bits(decode_fltsize) = V{}(n);
X{intsize}(d) = FPToFixed{intsize, decode_fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

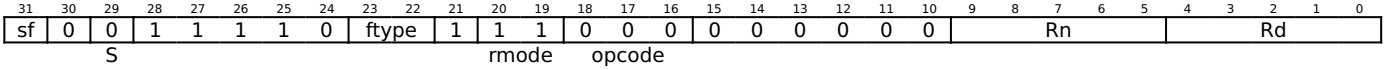
FCVTZS (scalar, integer)

Floating-point convert to signed integer, rounding toward zero (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit signed integer using the Round towards Zero rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTZS <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTZS <Xd>, <Hn>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTZS <Wd>, <Sn>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTZS <Xd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTZS <Wd>, <Dn>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTZS <Xd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');

let rounding : FPRounding = FPRounding_ZERO;
let unsigned : boolean    = FALSE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltval : bits(fltsize) = V{}(n);
let fracbits : integer = 0;

X{intsize}(d) = FPToFixed{intsize, fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZS (vector, fixed-point)

Floating-point convert to signed fixed-point, rounding toward zero (vector)

This instruction converts a scalar or each element in a vector from floating-point to fixed-point signed integer using the Round towards Zero rounding mode, and writes the result to the SIMD&FP destination register.

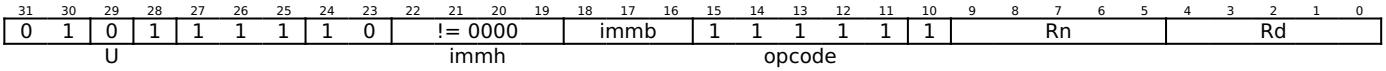
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

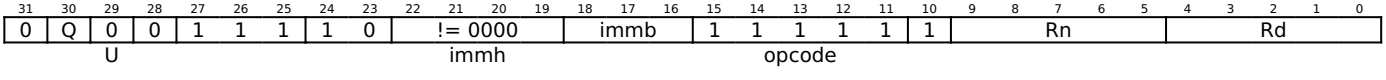
```
FCVTZS <V><d>, <V><n>, #<fbits>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh IN {'000x'} || (immh IN {'001x'} && !IsFeatureImplemented(FEAT_FP16)) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = if immh IN {'1xxx'} then 64 else if immh IN {'01xx'} then 32 else 16;
let datasize : integer{} = esize;
let elements : integer = 1;

let fracbits : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
FCVTZS <Vd>.<T>, <Vn>.<T>, #<fbits>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh IN {'000x'} || (immh IN {'001x'} && !IsFeatureImplemented(FEAT_FP16)) then
    EndOfDecode(Decode_UNDEF);
end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = if immh IN {'1xxx'} then 64 else if immh IN {'01xx'} then 32 else 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let fracbits : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<V> Is a width specifier, encoded in “immh”:

immh	<V>	Architectural Feature
0001	RESERVED	-
001x	H	FEAT_FP16
01xx	S	-
1xxx	D	-

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<fbits> For the "Scalar" variant: is the number of fractional bits, in the range 1 to the operand width, encoded in “(immh :: immb)”:

immh	<fbits>
0001	RESERVED
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

For the "Vector" variant: is the number of fractional bits, in the range 1 to the element width, encoded in “(immh :: immb)”:

immh	<fbits>
0001	RESERVED
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>	Architectural Feature
0001	x	RESERVED	-
001x	0	4H	FEAT_FP16
001x	1	8H	FEAT_FP16
01xx	0	2S	-
01xx	1	4S	-
1xxx	0	RESERVED	-
1xxx	1	2D	-

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZS (vector, integer)

Floating-point convert to signed integer, rounding toward zero (vector)

This instruction converts a scalar or each element in a vector from a floating-point value to a signed integer value using the Round towards Zero rounding mode, and writes the result to the SIMD&FP destination register.

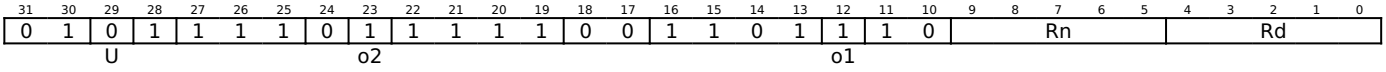
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

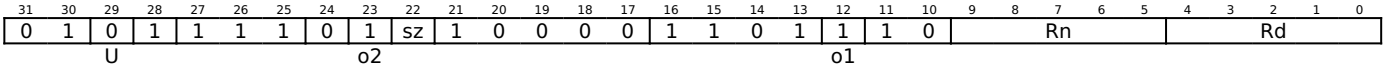
```
FCVTZS <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

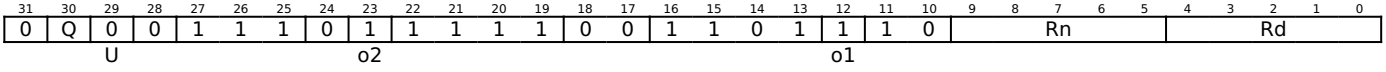
```
FCVTZS <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

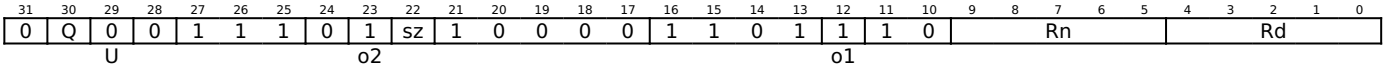
```
FCVTZS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTZS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZU (scalar SIMD&FP)

Floating-point convert to unsigned integer, rounding toward zero (scalar SIMD&FP)

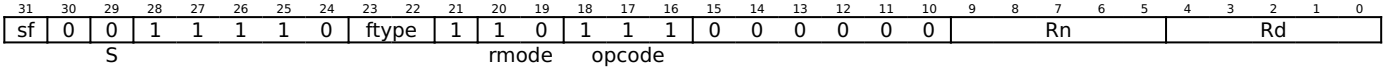
This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit unsigned integer using the Round toward Zero rounding mode, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FPRCVT)



Encoding for the Half-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 11)`

```
FCVTZU <Sd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 11)`

```
FCVTZU <Dd>, <Hn>
```

Encoding for the Single-precision to 64-bit variant

Applies when `(sf == 1 && ftype == 00)`

```
FCVTZU <Dd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant

Applies when `(sf == 0 && ftype == 01)`

```
FCVTZU <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

fltval = V{fltsize}(n);
intval = FPToFixed{intsize, fltsize}(fltval, 0, TRUE, FPCR(), FPRounding_ZERO);
result[0+:intsize] = intval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

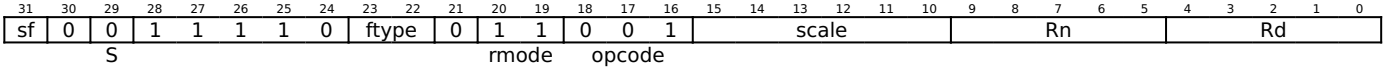
FCVTZU (scalar, fixed-point)

Floating-point convert to unsigned fixed-point, rounding toward zero (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit fixed-point unsigned integer using the Round towards Zero rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTZU <Wd>, <Hn>, #<fbits>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTZU <Xd>, <Hn>, #<fbits>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTZU <Wd>, <Sn>, #<fbits>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTZU <Xd>, <Sn>, #<fbits>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTZU <Wd>, <Dn>, #<fbits>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTZU <Xd>, <Dn>, #<fbits>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && scale[5] == '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let decode_fltsize : integer{} = 8 << UInt(ftype XOR '10');

let fracbits : integer = 64 - UInt(scale);

let rounding : FPRounding = FPRounding_ZERO;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<fbits>	For the "Double-precision to 32-bit", "Half-precision to 32-bit", and "Single-precision to 32-bit" variants: is the number of bits after the binary point in the fixed-point destination, in the range 1 to 32, encoded as 64 minus "scale". For the "Double-precision to 64-bit", "Half-precision to 64-bit", and "Single-precision to 64-bit" variants: is the number of bits after the binary point in the fixed-point destination, in the range 1 to 64, encoded as 64 minus "scale".
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltval : bits(decode_fltsize) = V{}(n);
X{intsize}(d) = FPToFixed{intsize, decode_fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

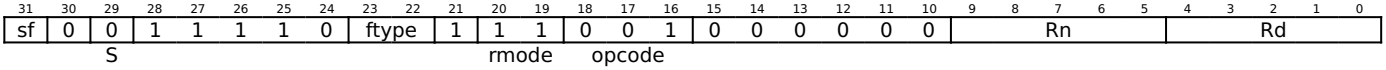
FCVTZU (scalar, integer)

Floating-point convert to unsigned integer, rounding toward zero (scalar)

This instruction converts the floating-point value in the SIMD&FP source register to a 32-bit or 64-bit unsigned integer using the Round towards Zero rounding mode, and writes the result to the general-purpose destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
FCVTZU <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
FCVTZU <Xd>, <Hn>
```

Encoding for the Single-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
FCVTZU <Wd>, <Sn>
```

Encoding for the Single-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
FCVTZU <Xd>, <Sn>
```

Encoding for the Double-precision to 32-bit variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
FCVTZU <Wd>, <Dn>
```

Encoding for the Double-precision to 64-bit variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
FCVTZU <Xd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');

let rounding : FPRounding = FPRounding_ZERO;
let unsigned : boolean    = TRUE;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltval : bits(fltsize) = V{}(n);
let fracbits : integer = 0;

X{intsize}(d) = FPToFixed{intsize, fltsize}(fltval, fracbits, unsigned, FPCR(), rounding);
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZU (vector, fixed-point)

Floating-point convert to unsigned fixed-point, rounding toward zero (vector)

This instruction converts a scalar or each element in a vector from floating-point to fixed-point unsigned integer using the Round towards Zero rounding mode, and writes the result to the general-purpose destination register.

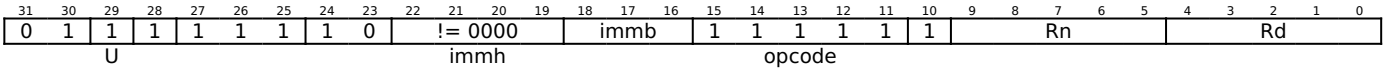
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

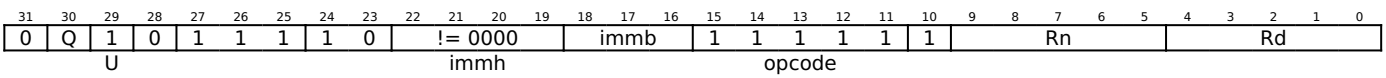
```
FCVTZU <V><d>, <V><n>, #<fbits>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh IN {'000x'} || (immh IN {'001x'} && !IsFeatureImplemented(FEAT_FP16)) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = if immh IN {'1xxx'} then 64 else if immh IN {'01xx'} then 32 else 16;
let datasize : integer{} = esize;
let elements : integer = 1;

let fracbits : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
FCVTZU <Vd>.<T>, <Vn>.<T>, #<fbits>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh IN {'000x'} || (immh IN {'001x'} && !IsFeatureImplemented(FEAT_FP16)) then
    EndOfDecode(Decode_UNDEF);
end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = if immh IN {'1xxx'} then 64 else if immh IN {'01xx'} then 32 else 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let fracbits : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<V> Is a width specifier, encoded in “immh”:

immh	<V>	Architectural Feature
0001	RESERVED	-
001x	H	FEAT_FP16
01xx	S	-
1xxx	D	-

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<fbits> For the "Scalar" variant: is the number of fractional bits, in the range 1 to the operand width, encoded in “(immh :: immb)”:

immh	<fbits>
0001	RESERVED
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

For the "Vector" variant: is the number of fractional bits, in the range 1 to the element width, encoded in “(immh :: immb)”:

immh	<fbits>
0001	RESERVED
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>	Architectural Feature
0001	x	RESERVED	-
001x	0	4H	FEAT_FP16
001x	1	8H	FEAT_FP16
01xx	0	2S	-
01xx	1	4S	-
1xxx	0	RESERVED	-
1xxx	1	2D	-

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZU (vector, integer)

Floating-point convert to unsigned integer, rounding toward zero (vector)

This instruction converts a scalar or each element in a vector from a floating-point value to an unsigned integer value using the Round towards Zero rounding mode, and writes the result to the SIMD&FP destination register.

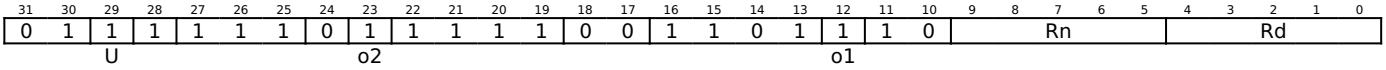
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

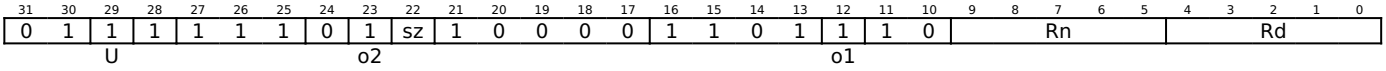
```
FCVTZU <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

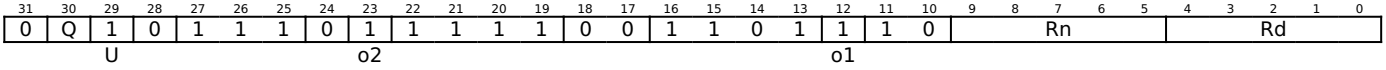
```
FCVTZU <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

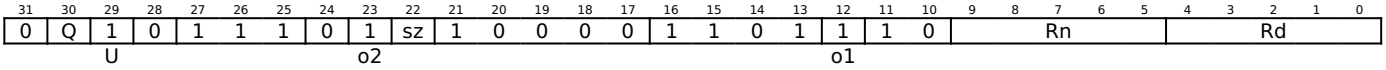
```
FCVTZU <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FCVTZU <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : FPRounding = FPDecodeRounding(o1::o2);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPToFixed{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

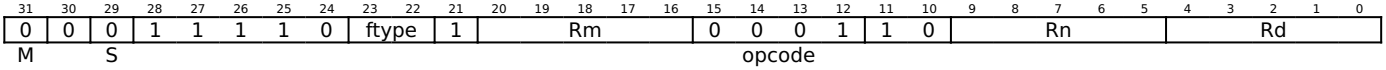
FDIV (scalar)

Floating-point divide (scalar)

This instruction divides the floating-point value of the first source SIMD&FP register by the floating-point value of the second source SIMD&FP register, and writes the result to the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FDIV <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FDIV <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FDIV <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPDiv{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDIV (vector)

Floating-point divide (vector)

This instruction divides the floating-point values in the elements in the first source SIMD&FP register, by the floating-point values in the corresponding elements in the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register.

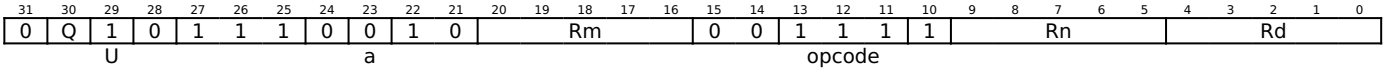
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

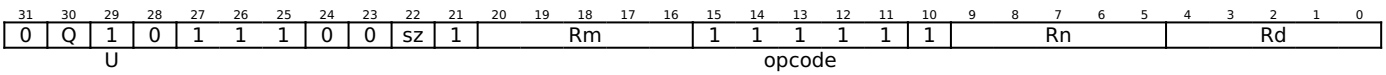
```
FDIV <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FDIV <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPDiv{esize}(element1, element2, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (8-bit floating-point to half-precision, by element)

8-bit floating-point dot product to half-precision (vector, by element)

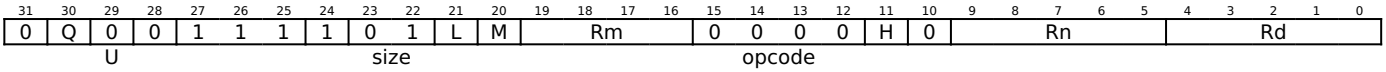
This instruction calculates the fused sum-of-products of a pair of 8-bit floating-point values held in each 16-bit element of the first source vector and a pair of 8-bit floating-point values in an indexed 16-bit element of the second source vector. The half-precision sum-of-products are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$, before being destructively added without intermediate rounding to the corresponding half-precision elements of the destination vector.

The 8-bit floating-point groups within the second source vector are specified using an immediate index.

The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_FP8DOT2)



Encoding

```
FDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.2B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP8DOT2) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let d : integer = UInt(Rd);
let m : integer = UInt('0'::Rm);
let i : integer = UInt(H::L::M);

let datasize : integer{} = if Q == '1' then 128 else 64;
let esize : integer{} = 16;
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	4H
1	8H

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, in the range V0 to V15, encoded in the "Rm" field.

<index> Is the immediate index of a pair of 8-bit elements, in the range 0 to 7, encoded in the "H:L:M" fields.

Operation

```
CheckFPMREnabled(); AArch64.CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    let op1 : bits(esize) = operand1[e*:esize];
    let op2 : bits(esize) = operand2[i*:esize];
    let sum : bits(esize) = operand3[e*:esize];

    result[e*:esize] = FP8DotAddFP{esize, esize}(sum, op1, op2, FPCR(), FPMR());
end;

V{datasize}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (8-bit floating-point to half-precision, vector)

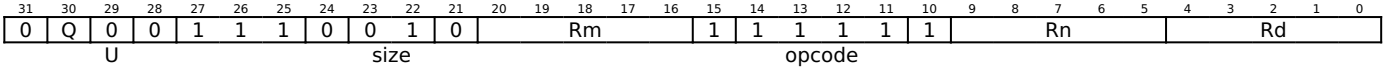
8-bit floating-point dot product to half-precision (vector)

This instruction calculates the fused sum-of-products of a pair of 8-bit floating-point values held in each 16-bit element of the first and second source vectors. The half-precision sum-of-products are scaled by $2^{-UInt(FPMR.LSCALE[3:0])}$, before being destructively added without intermediate rounding to the corresponding half-precision elements of the destination vector.

The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_FP8DOT2)



Encoding

```
FDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP8DOT2) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = if Q == '1' then 128 else 64;
let esize : integer{} = 16;
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	4H
1	8H

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    let op1 : bits(esize) = operand1[e*:esize];
    let op2 : bits(esize) = operand2[e*:esize];
    var sum : bits(esize) = operand3[e*:esize];

    sum = FP8DotAddFP{esize, esize}(sum, op1, op2, FPCR(), FPMR());
    result[e*:esize] = sum;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (8-bit floating-point to single-precision, by element)

8-bit floating-point dot product to single-precision (vector, by element)

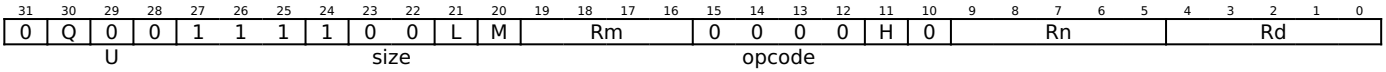
This instruction calculates the fused sum-of-products of a group of four 8-bit floating-point values held in each 32-bit element of the first source vector and a group of four 8-bit floating-point values in an indexed 32-bit element of the second source vector. The single-precision sum-of-products are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$, before being destructively added without intermediate rounding to the corresponding single-precision elements of the destination vector.

The 8-bit floating-point groups within the second source vector are specified using an immediate index.

The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_FP8DOT4)



Encoding

```
FDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.4B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP8DOT4) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let d : integer = UInt(Rd);
let m : integer = UInt(M::Rm);
let i : integer = UInt(H::L);

let datasize : integer{} = if Q == '1' then 128 else 64;
let esize : integer{} = 32;
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> Is the immediate index of a group of four 8-bit values, in the range 0 to 3, encoded in the "H:L" fields.

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    let op1 : bits(esize) = operand1[e*:esize];
    let op2 : bits(esize) = operand2[i*:esize];
    let sum : bits(esize) = operand3[e*:esize];

    result[e*:esize] = FP8DotAddFP{esize, esize}(sum, op1, op2, FPCR(), FPMR());
end;

V{datasize}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (8-bit floating-point to single-precision, vector)

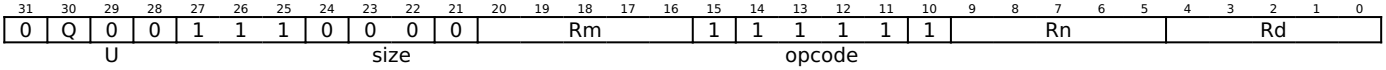
8-bit floating-point dot product to single-precision (vector)

This instruction calculates the fused sum-of-products of a group of four 8-bit floating-point values held in each 32-bit element of the first and second source vectors. The single-precision sum-of-products are scaled by $2^{-UInt(FPMR.LSCALE)}$, before being destructively added without intermediate rounding to the corresponding single-precision elements of the destination vector.

The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_FP8DOT4)



Encoding

```
FDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP8DOT4) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = if Q == '1' then 128 else 64;
let esize : integer{} = 32;
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    let op1 : bits(esize) = operand1[e*:esize];
    let op2 : bits(esize) = operand2[e*:esize];
    var sum : bits(esize) = operand3[e*:esize];

    sum = FP8DotAddFP{esize, esize}(sum, op1, op2, FPCR(), FPMR());
    result[e*:esize] = sum;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

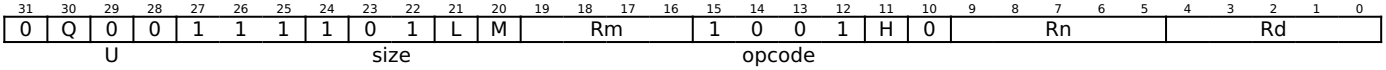
FDOT (half-precision to single-precision, by element)

Half-precision dot product to single-precision (vector, by element)

This instruction calculates the fused sum-of-products of a pair of half-precision values held in each 32-bit element of the first source vector and a pair of half-precision values held in an indexed 32-bit element of the second source vector, without intermediate rounding, and then destructively adds the single-precision sum-of-products to the corresponding single-precision element of the destination vector.

Advanced SIMD

(FEAT_F16F32DOT)



Encoding

```
FDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.2H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F16F32DOT) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(M::Rm);
let d : integer = UInt(Rd);
let i : integer = UInt(H::L);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 32;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	4H
1	8H

<Vm> Is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> Is the immediate index of a pair of 16-bit elements in the range 0 to 3, encoded in the "H:L" fields.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    let elt1_a : bits(16) = operand1[(2 * e + 0)*:16];
    let elt1_b : bits(16) = operand1[(2 * e + 1)*:16];
    let elt2_a : bits(16) = operand2[(2 * i + 0)*:16];
    let elt2_b : bits(16) = operand2[(2 * i + 1)*:16];

    let sum : bits(32) = operand3[e*:32];
    result[e*:32] = FPDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

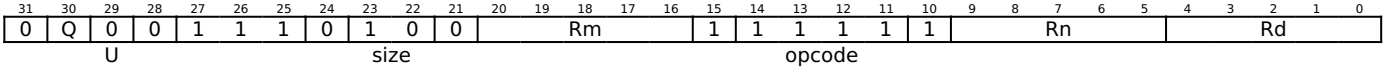
FDOT (half-precision to single-precision, vector)

Half-precision dot product to single-precision (vector)

This instruction calculates the fused sum-of-products of a pair of half-precision values held in each 32-bit element of the first and second source vectors, without intermediate rounding, and then destructively adds the single-precision sum-of-products to the corresponding single-precision element of the destination vector.

Advanced SIMD

(FEAT_F16F32DOT)



Encoding

```
FDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F16F32DOT) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 32;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	4H
1	8H

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    let elt1_a : bits(16) = operand1[(2 * e + 0)*:16];
    let elt1_b : bits(16) = operand1[(2 * e + 1)*:16];
    let elt2_a : bits(16) = operand2[(2 * e + 0)*:16];
    let elt2_b : bits(16) = operand2[(2 * e + 1)*:16];

    let sum : bits(32) = operand3[e*:32];
    result[e*:32] = FPDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FJCVTZS

Floating-point JavaScript convert to signed fixed-point, rounding toward zero

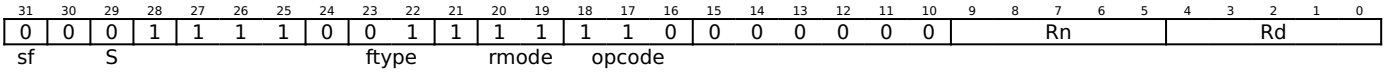
This instruction converts the double-precision floating-point value in the SIMD&FP source register to a 32-bit signed integer using the Round towards Zero rounding mode, and writes the result to the general-purpose destination register. If the result is too large to be represented as a signed 32-bit integer, then the result is the integer modulo 2^{32}, as held in a 32-bit signed integer.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Double-precision to 32-bit

(FEAT_JSCVT)



Encoding

```
FJCVTZS <Wd>, <Dn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_JSCVT) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let fltval : bits(64) = V{}(n);

var intval : bits(32);
var z : bit;
(intval, z) = FPToFixedJS(fltval, FPCR());

X{32}(d) = intval;
PSTATE.[N,Z,C,V] = '0'::z::'00';
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

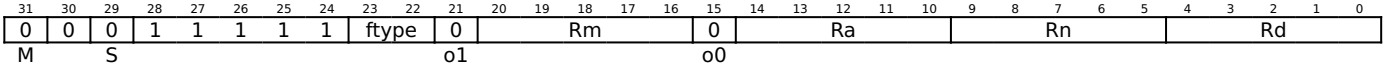
FMADD

Floating-point fused multiply-add (scalar)

This instruction multiplies the values of the first two SIMD&FP source registers, adds the product to the value of the third SIMD&FP source register, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant
(FEAT_FP16)

Applies when `(ftype == 11)`

```
FMADD <Hd>, <Hn>, <Hm>, <Ha>
```

Encoding for the Single-precision variant
(FEAT_FP)

Applies when `(ftype == 00)`

```
FMADD <Sd>, <Sn>, <Sm>, <Sa>
```

Encoding for the Double-precision variant
(FEAT_FP)

Applies when `(ftype == 01)`

```
FMADD <Dd>, <Dn>, <Dm>, <Da>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Ha>	Is the 16-bit name of the third SIMD&FP source register holding the addend, encoded in the "Ra" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Sa>	Is the 32-bit name of the third SIMD&FP source register holding the addend, encoded in the "Ra" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Da>	Is the 64-bit name of the third SIMD&FP source register holding the addend, encoded in the "Ra" field.

Operation

```
AArch64.CheckFPEnabled();

let addend : bits(esize)   = V{esize}(a);
let operand1 : bits(esize) = V{esize}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(a) else Zeros{128};

result[0+:esize] = FPMulAdd{esize}(addend, operand1, operand2, FPCR());
V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAX (scalar)

Floating-point maximum (scalar)

This instruction compares the two source SIMD&FP registers, and writes the larger of the two floating-point values to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows:

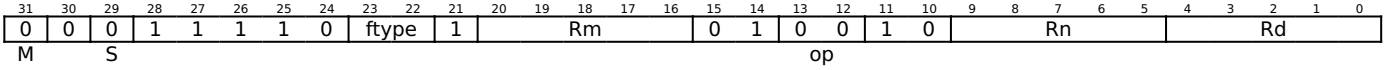
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FMAX <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FMAX <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FMAX <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPMax{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAX (vector)

Floating-point maximum (vector)

This instruction compares corresponding vector elements in the two source SIMD&FP registers, places the larger of each of the two floating-point values into a vector, and writes the vector to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

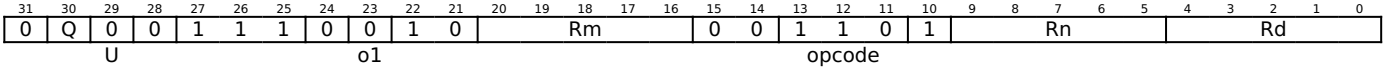
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

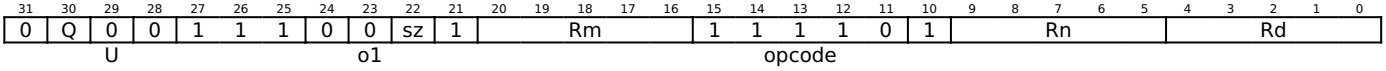
```
FMAX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMax{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNM (scalar)

Floating-point maximum number (scalar)

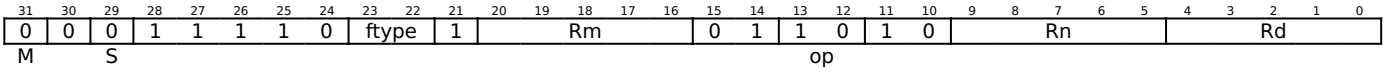
This instruction compares the first and second source SIMD&FP register values, and writes the larger of the two floating-point values to the destination SIMD&FP register.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant
(FEAT_FP16)

Applies when `(ftype == 11)`

```
FMAXNM <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant
(FEAT_FP)

Applies when `(ftype == 00)`

```
FMAXNM <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant
(FEAT_FP)

Applies when `(ftype == 01)`

```
FMAXNM <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPMaxNum{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNM (vector)

Floating-point maximum number (vector)

This instruction compares corresponding vector elements in the two source SIMD&FP registers, writes the larger of the two floating-point values into a vector, and writes the vector to the destination SIMD&FP register.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

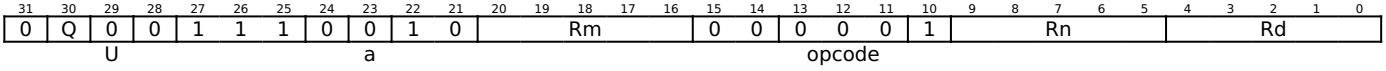
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

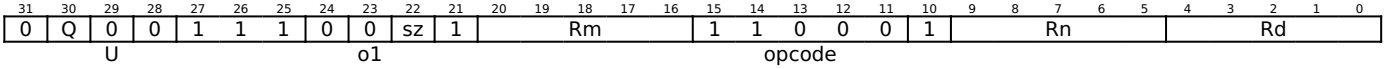
```
FMAXNM <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXNM <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNMP (scalar)

Floating-point maximum number of pair of elements (scalar)

This instruction compares two vector elements in the source SIMD&FP register and writes the largest of the floating-point values as a scalar to the destination SIMD&FP register.

The behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

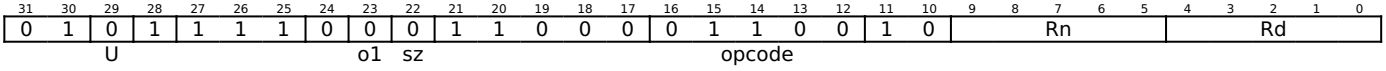
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

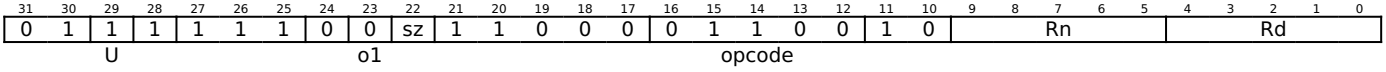
```
FMAXNMP H<d>, <Vn>.2H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 32;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXNMP <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize * 2;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.
<V>	Is the destination width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<T> Is the source arrangement specifier, encoded in “sz”:

sz	<T>
0	2S
1	2D

Operation

```
AArch64.CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
let element1 : bits(esize) = operand[0*:esize];
let element2 : bits(esize) = operand[1*:esize];
V{esize}(d) = FPMaxNum{esize}(element1, element2, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNMP (vector)

Floating-point maximum number pairwise (vector)

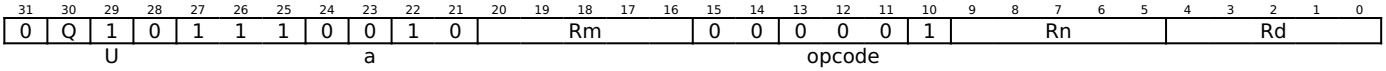
This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements in the two source SIMD&FP registers, writes the largest of each pair of values into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.
The behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

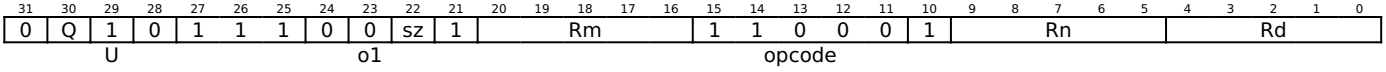
```
FMAXNMP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXNMP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = concat[(2*e)*:esize];
    element2 = concat[((2*e)+1)*:esize];
    result[e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNMV

Floating-point maximum number across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the largest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are floating-point values.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

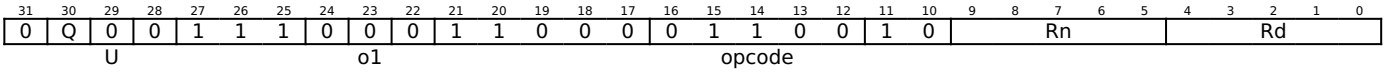
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

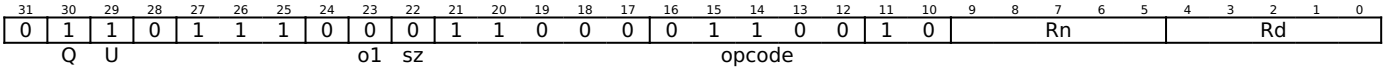
```
FMAXNMV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
```

Single-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXNMV S<d>, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q != '01' then EndOfDecode(Decode_UNDEF); end;    // .4S only

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<V>	Is the destination width specifier, H.
<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
V{esize}(d) = FPReduce{esize, datasize}(ReduceOp_FMAXNUM, operand, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXP (scalar)

Floating-point maximum of pair of elements (scalar)

This instruction compares two vector elements in the source SIMD&FP register and writes the largest of the floating-point values as a scalar to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

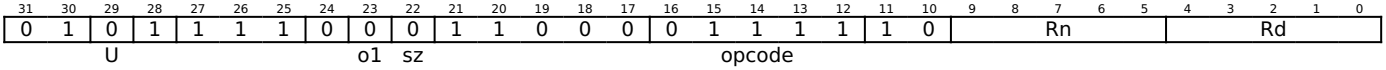
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

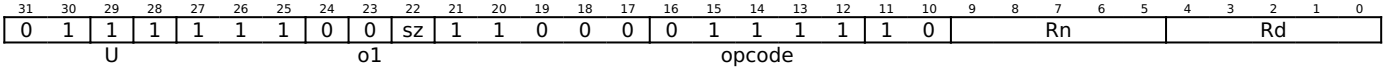
```
FMAXP H<d>, <Vn>.2H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 32;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXP <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize * 2;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is the destination width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<T> Is the source arrangement specifier, encoded in “sz”:

sz	<T>
0	2S
1	2D

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand :  bits(datasize) = V{}(n);
let element1 : bits(esize) = operand[0*:esize];
let element2 : bits(esize) = operand[1*:esize];
V{esize}(d) = FPMax{esize}(element1, element2, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXP (vector)

Floating-point maximum pairwise (vector)

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements from the concatenated vector, writes the larger of each pair of values into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

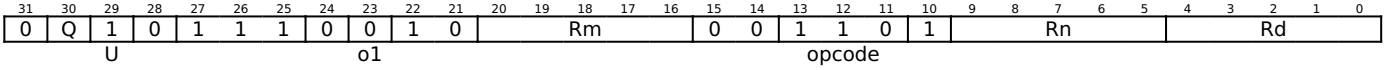
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

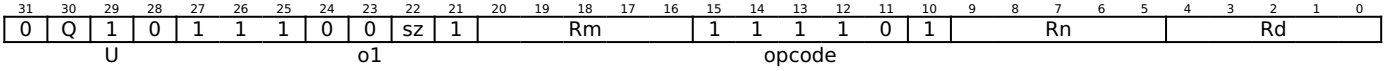
```
FMAXP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = concat[(2*e)*:esize];
    element2 = concat[((2*e)+1)*:esize];
    result[e*:esize] = FPMax{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXV

Floating-point maximum across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the largest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are floating-point values.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

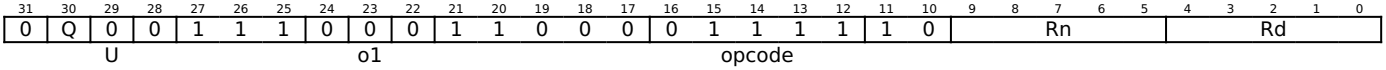
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

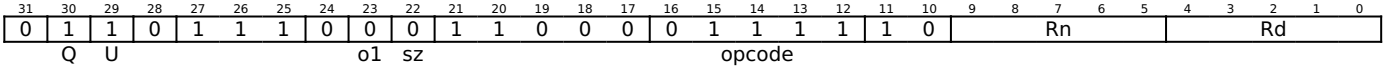
```
FMAXV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
```

Single-precision

(FEAT_AdvSIMD)



Encoding

```
FMAXV S<d>, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q != '01' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<V>	Is the destination width specifier, H.
<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand :  bits(datasize) = V{}(n);
V{esize}(d) = FPReduce{esize, datasize}(ReduceOp_FMAX, operand, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMIN (scalar)

Floating-point minimum (scalar)

This instruction compares the first and second source SIMD&FP register values, and writes the smaller of the two floating-point values to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows:

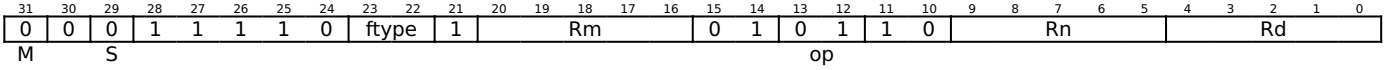
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FMIN <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FMIN <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FMIN <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPMin{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMIN (vector)

Floating-point minimum (vector)

This instruction compares corresponding elements in the vectors in the two source SIMD&FP registers, places the smaller of each of the two floating-point values into a vector, and writes the vector to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

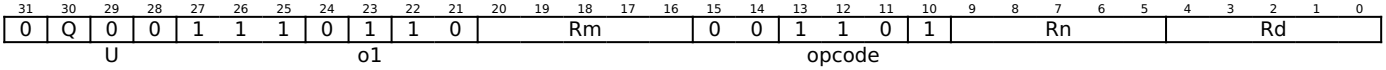
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

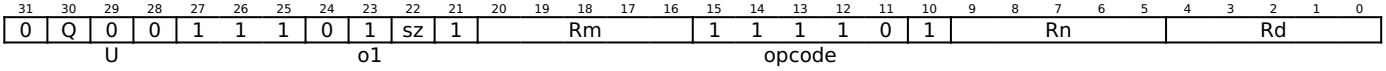
```
FMIN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMIN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMin{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNM (scalar)

Floating-point minimum number (scalar)

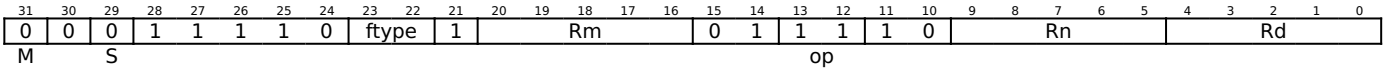
This instruction compares the first and second source SIMD&FP register values, and writes the smaller of the two floating-point values to the destination SIMD&FP register.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant
(FEAT_FP16)

Applies when `(ftype == 11)`

```
FMINNM <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant
(FEAT_FP)

Applies when `(ftype == 00)`

```
FMINNM <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant
(FEAT_FP)

Applies when `(ftype == 01)`

```
FMINNM <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPMinNum{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNM (vector)

Floating-point minimum number (vector)

This instruction compares corresponding vector elements in the two source SIMD&FP registers, writes the smaller of the two floating-point values into a vector, and writes the vector to the destination SIMD&FP register.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

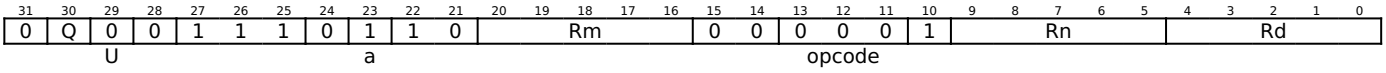
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

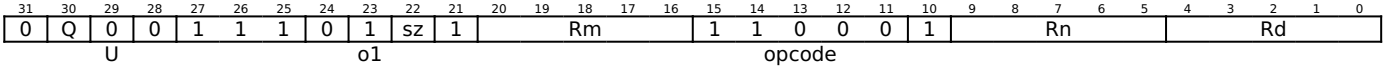
```
FMINNM <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMINNM <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNMP (scalar)

Floating-point minimum number of pair of elements (scalar)

This instruction compares two vector elements in the source SIMD&FP register and writes the smallest of the floating-point values as a scalar to the destination SIMD&FP register.

The behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

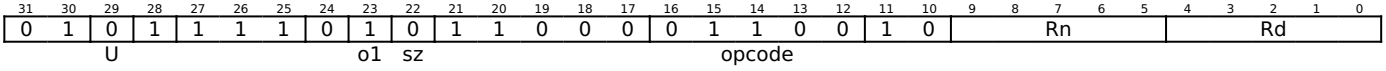
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

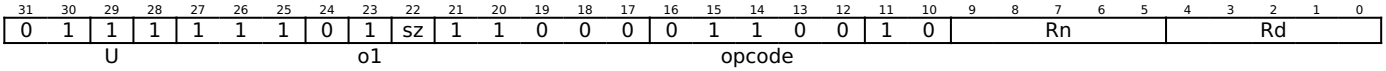
```
FMINNMP H<d>, <Vn>.2H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 32;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMINNMP <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize * 2;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.
<V>	Is the destination width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<T> Is the source arrangement specifier, encoded in “sz”:

sz	<T>
0	2S
1	2D

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
let element1 : bits(esize) = operand[0*:esize];
let element2 : bits(esize) = operand[1*:esize];
V{esize}(d) = FPMinNum{esize}(element1, element2, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNMP (vector)

Floating-point minimum number pairwise (vector)

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements in the two source SIMD&FP registers, writes the smallest of each pair of floating-point values into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

The behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

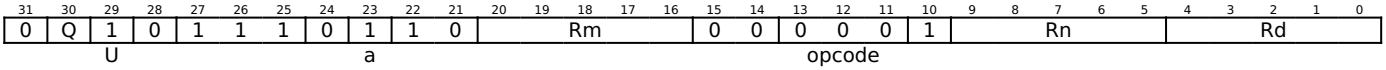
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

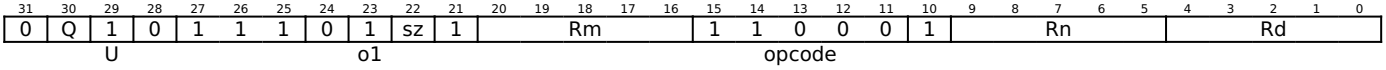
```
FMINNMP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMINNMP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = concat[(2*e)*:esize];
    element2 = concat[((2*e)+1)*:esize];
    result[e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNMV

Floating-point minimum number across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the smallest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are floating-point values.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

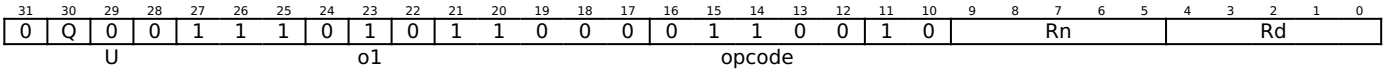
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

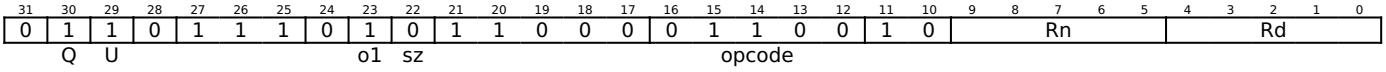
```
FMINNMV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
```

Single-precision

(FEAT_AdvSIMD)



Encoding

```
FMINNMV S<d>, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q != '01' then EndOfDecode(Decode_UNDEF); end;    // .4S only

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<V>	Is the destination width specifier, H.
<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
V{esize}(d) = FPReduce{esize, datasize}(ReduceOp_FMINNUM, operand, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINP (scalar)

Floating-point minimum of pair of elements (scalar)

This instruction compares two vector elements in the source SIMD&FP register and writes the smallest of the floating-point values as a scalar to the destination SIMD&FP register.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

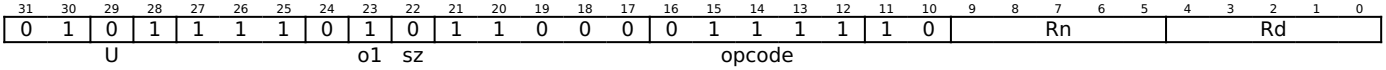
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

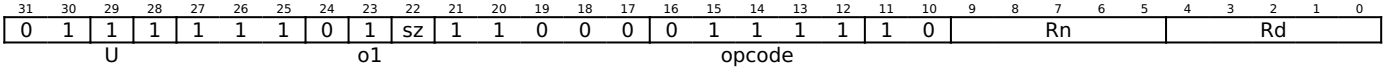
```
FMINP H<d>, <Vn>.2H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 32;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMINP <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize * 2;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is the destination width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<T> Is the source arrangement specifier, encoded in “sz”:

sz	<T>
0	2S
1	2D

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand :  bits(datasize) = V{}(n);
let element1 : bits(esize) = operand[0*:esize];
let element2 : bits(esize) = operand[1*:esize];
V{esize}(d) = FPMin{esize}(element1, element2, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINP (vector)

Floating-point minimum pairwise (vector)

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements from the concatenated vector, writes the smaller of each pair of values into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

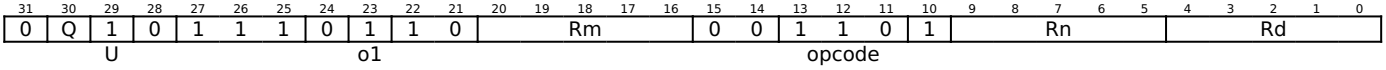
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

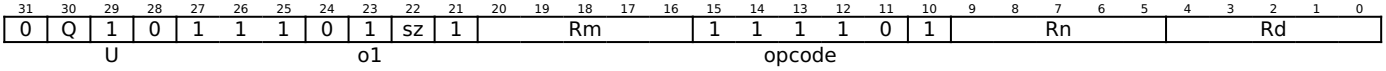
```
FMINP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMINP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = concat[(2*e)*:esize];
    element2 = concat[((2*e)+1)*:esize];
    result[e*:esize] = FPMin{esize}(element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINV

Floating-point minimum across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the smallest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are floating-point values.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

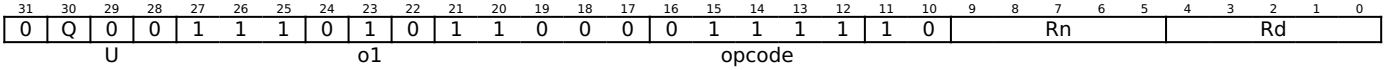
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

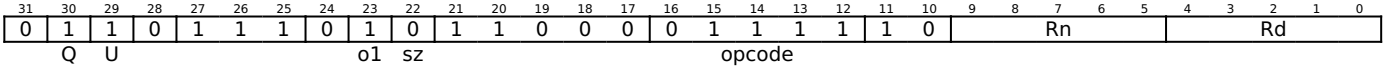
```
FMINV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
```

Single-precision

(FEAT_AdvSIMD)



Encoding

```
FMINV S<d>, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q != '01' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<V>	Is the destination width specifier, H.
<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand :  bits(datasize) = V{}(n);
V{esize}(d) = FPReduce{esize, datasize}(ReduceOp_FMIN, operand, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLA (by element)

Floating-point fused multiply-add to accumulator (by element)

This instruction multiplies the vector elements in the first source SIMD&FP register by the specified value in the second source SIMD&FP register, and accumulates the results in the vector elements of the destination SIMD&FP register. All the values in this instruction are floating-point values.

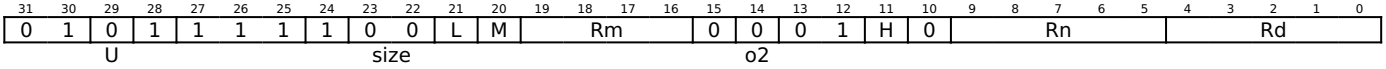
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar, half-precision , Scalar, single-precision and double-precision , Vector, half-precision and Vector, single-precision and double-precision

Scalar, half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

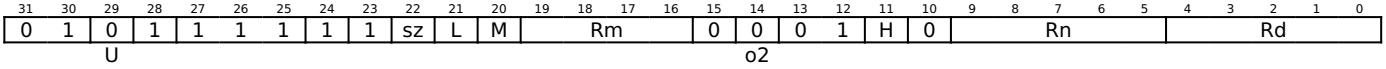
```
FMLA <Hd>, <Hn>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let idxdsize : integer{} = 64 << UInt(H);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Rd);
let index : integer = UInt(H::L::M);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar, single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

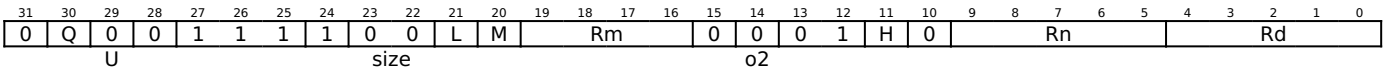
```
FMLA <V><d>, <V><n>, <Vm>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
let Rmhi : bit = M;
case sz::L of
    when '0x' => index = UInt(H::L);
    when '10' => index = UInt(H);
    when '11' => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector, half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

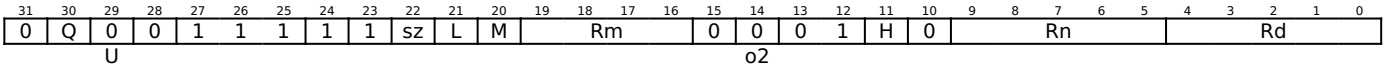
```
FMLA <Vd>.<T>, <Vn>.<T>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let idxdsize : integer{} = 64 << UInt(H);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Rd);
let index : integer = UInt(H::L::M);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector, single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMLA <Vd>.<T>, <Vn>.<T>, <Vm>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
let Rmhi : bit = M;
case sz::L of
    when '0x' => index = UInt(H::L);
    when '10' => index = UInt(H);
    when '11' => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> For the "Scalar, half-precision" and "Vector, half-precision" variants: is the name of the second SIMD&FP source register, in the range V0 to V15, encoded in the "Rm" field.

For the "Scalar, single-precision and double-precision" and "Vector, single-precision and double-precision" variants: is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> For the "Scalar, half-precision" and "Vector, half-precision" variants: is the element index, in the range 0 to 7, encoded in the "H:L:M" fields.

For the "Scalar, single-precision and double-precision" and "Vector, single-precision and double-precision" variants: is the element index, encoded in “(sz :: L :: H)”:

sz	L	<index>
0	x	UInt(H :: L)
1	0	UInt(H)
1	1	RESERVED

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<Ts> Is an element size specifier, encoded in “sz”:

sz	<Ts>
0	S
1	D

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector, half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector, single-precision and double-precision" variant: is an arrangement specifier, encoded in “(Q :: sz)”:

Q	sz	<T>
0	0	2S
0	1	RESERVED
1	0	4S
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var element1 : bits(esize);
let element2 : bits(esize) = operand2[index*:esize];
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    result[e*:esize] = FPMulAdd{esize}(operand3[e*:esize], element1, element2, FPCR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLA (vector)

Floating-point fused multiply-add to accumulator (vector)

This instruction multiplies corresponding floating-point values in the vectors in the two source SIMD&FP registers, adds the product to the corresponding vector element of the destination SIMD&FP register, and writes the result to the destination SIMD&FP register.

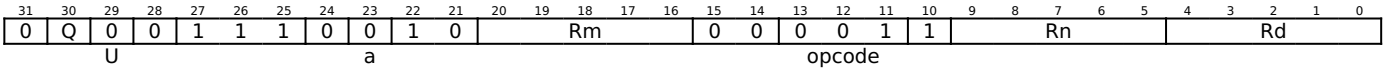
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

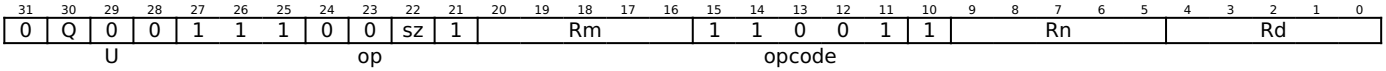
```
FMLA <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMLA <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMulAdd{esize}(operand3[e*:esize], element1, element2, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLAL, FMLAL2 (by element)

Floating-point fused multiply-add long to accumulator (by element)

This instruction multiplies the half-precision vector elements in the first source SIMD&FP register by the specified half-precision value in the second source SIMD&FP register, and accumulates the intermediate product without rounding to the corresponding single-precision vector element of the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

In Armv8.2 and Armv8.3, this is an OPTIONAL instruction. From Armv8.4, it is mandatory for all implementations to support it.

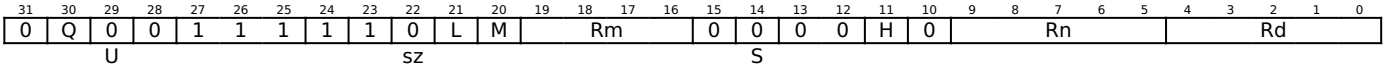
Note

ID_AA64ISAR0_EL1.FHM indicates whether this instruction is supported.

It has encodings from 2 classes: FMLAL and FMLAL2

FMLAL

(FEAT_FHM)



Encoding

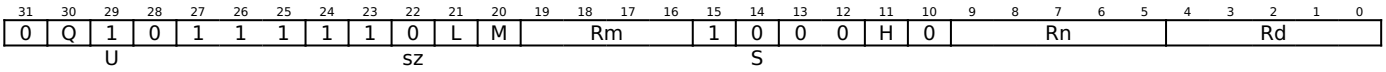
```
FMLAL <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FHM) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt('0'::Rm);    // Vm can only be in bottom 16 registers.
let index : integer = UInt(H::L::M);

let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let part : integer = 0;
```

FMLAL2

(FEAT_FHM)



Encoding

```
FMLAL2 <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FHM) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt('0'::Rm);    // Vm can only be in bottom 16 registers.
let index : integer = UInt(H::L::M);

let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let part : integer = 1;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	2H
1	4H

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

<index> Is the element index, encoded in the "H:L:M" fields.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize DIV 2) = Vpart{}(n, part);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : bits(esize DIV 2);
let element2 : bits(esize DIV 2) = operand2[index*:(esize DIV 2)];

for e = 0 to elements-1 do
    element1 = operand1[e*:(esize DIV 2)];
    result[e*:esize] = FPMulAddH(operand3[e*:esize], element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLAL, FMLAL2 (vector)

Floating-point fused multiply-add long to accumulator (vector)

This instruction multiplies corresponding half-precision floating-point values in the vectors in the two source SIMD&FP registers, and accumulates the intermediate product without rounding to the corresponding single-precision vector element of the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

In Armv8.2 and Armv8.3, this is an OPTIONAL instruction. From Armv8.4, it is mandatory for all implementations to support it.

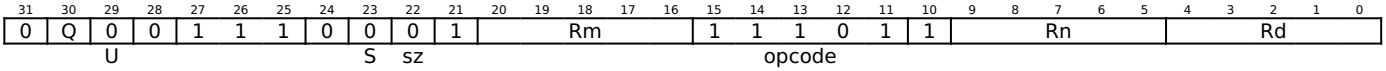
Note

ID_AA64ISAR0_EL1.FHM indicates whether this instruction is supported.

It has encodings from 2 classes: FMLAL and FMLAL2

FMLAL

(FEAT_FHM)



Encoding

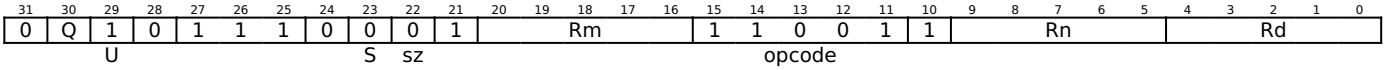
```
FMLAL <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FHM) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let part : integer = 0;
```

FMLAL2

(FEAT_FHM)



Encoding

```
FMLAL2 <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FHM) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let part : integer = 1;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	2H
1	4H

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize DIV 2) = Vpart{}(n, part);
let operand2 : bits(datasize DIV 2) = Vpart{}(m, part);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : bits(esize DIV 2);
var element2 : bits(esize DIV 2);

for e = 0 to elements-1 do
    element1 = operand1[e*:(esize DIV 2)];
    element2 = operand2[e*:(esize DIV 2)];
    result[e*:esize] = FPMulAddH(operand3[e*:esize], element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

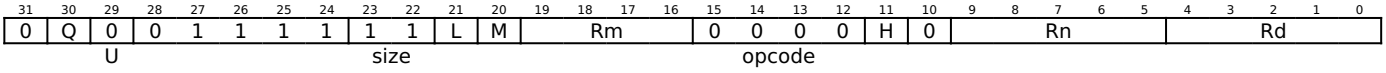
FMLALB, FMLALT (by element)

8-bit floating-point multiply-add to half-precision (vector, by element)

This instruction widens the even-numbered (bottom) or odd-numbered (top) 8-bit elements in the first source vector and the indexed element from the second source vector to half-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$, before being destructively added without intermediate rounding to the half-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the first source vector.
The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_FP8FMA)



Encoding for the FMLALB variant

Applies when `(Q == 0)`

```
FMLALB <Vd>.8H, <Vn>.16B, <Vm>.B[<index>]
```

Encoding for the FMLALT variant

Applies when `(Q == 1)`

```
FMLALT <Vd>.8H, <Vn>.16B, <Vm>.B[<index>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP8FMA) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt('00'::Rm[2:0]);
let d : integer = UInt(Rd);
let index : integer = UInt(H::L::M::Rm[3]);
let elements : integer = 128 DIV 16;
let sel : integer = UInt(Q);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, in the range V0 to V7, encoded in the "Rm<2:0>" field.
<index>	Is the element index, in the range 0 to 15, encoded in the "H:L:M:Rm<3>" fields.

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(128) = V{}(d);
var result : bits(128);

for e = 0 to elements-1 do
    let element1 : bits(8) = operand1[(2 * e + sel)*:8];
    let element2 : bits(8) = operand2[index*:8];
    let element3 : bits(16) = operand3[e*:16];
    result[e*:16] = FP8MulAddFP{16}(element3, element1, element2, FPCR(), FPMR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALB, FMLALT (vector)

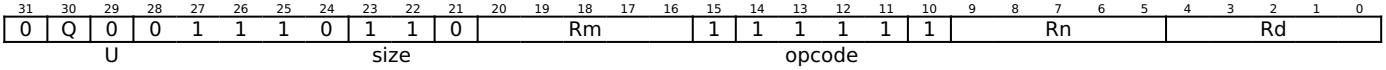
8-bit floating-point multiply-add to half-precision (vector)

This instruction widens the even-numbered (bottom) or odd-numbered (top) 8-bit elements in the first and second source vectors to half-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$, before being destructively added without intermediate rounding to the half-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the source vectors.

The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_FP8FMA)



Encoding for the FMLALB variant

Applies when `(Q == 0)`

```
FMLALB <Vd>.8H, <Vn>.16B, <Vm>.16B
```

Encoding for the FMLALT variant

Applies when `(Q == 1)`

```
FMLALT <Vd>.8H, <Vn>.16B, <Vm>.16B
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP8FMA) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let elements : integer = 128 DIV 16;
let sel : integer = UInt(Q);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(128) = V{}(d);
var result : bits(128);

for e = 0 to elements-1 do
    let element1 : bits(8) = operand1[(2 * e + sel)*:8];
    let element2 : bits(8) = operand2[(2 * e + sel)*:8];
    let element3 : bits(16) = operand3[e*:16];
    result[e*:16] = FP8MulAddFP{16}(element3, element1, element2, FPCR(), FPMR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

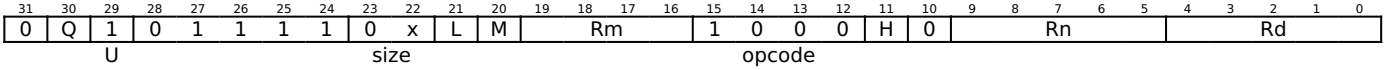
FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element)

8-bit floating-point multiply-add to single-precision (vector, by element)

This instruction widens the first (bottom bottom), second (bottom top), third (top bottom), or fourth (top top) 8-bit element of each 32-bit container in the first source vector and the indexed element from the second source vector to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$, before being destructively added without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the first source vector. The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_FP8FMA)



Encoding for the FMLALLBB variant

Applies when (Q == 0 && size == 00)

```
FMLALLBB <Vd>.4S, <Vn>.16B, <Vm>.B[<index>]
```

Encoding for the FMLALLBT variant

Applies when (Q == 0 && size == 01)

```
FMLALLBT <Vd>.4S, <Vn>.16B, <Vm>.B[<index>]
```

Encoding for the FMLALLTB variant

Applies when (Q == 1 && size == 00)

```
FMLALLTB <Vd>.4S, <Vn>.16B, <Vm>.B[<index>]
```

Encoding for the FMLALLTT variant

Applies when (Q == 1 && size == 01)

```
FMLALLTT <Vd>.4S, <Vn>.16B, <Vm>.B[<index>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP8FMA) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt('00'::Rm[2:0]);
let d : integer = UInt(Rd);
let index : integer = UInt(H::L::M::Rm[3]);
let elements : integer = 128 DIV 32;
let sel : integer = UInt(Q::size[0]);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, in the range V0 to V7, encoded in the "Rm<2:0>" field.
<index>	Is the element index, in the range 0 to 15, encoded in the "H:L:M:Rm<3>" fields.

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(128) = V{}(d);
var result : bits(128);

for e = 0 to elements-1 do
    let element1 : bits(8)  = operand1[(4 * e + sel)*:8];
    let element2 : bits(8)  = operand2[index*:8];
    let element3 : bits(32) = operand3[e*:32];
    result[e*:32] = FP8MulAddFP{32}(element3, element1, element2, FPCR(), FPMR());
end;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

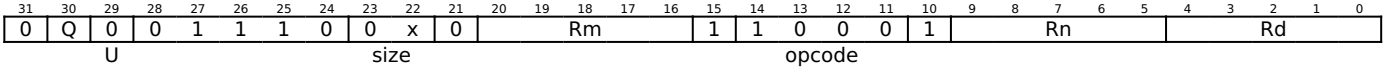
FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector)

8-bit floating-point multiply-add to single-precision (vector)

This instruction widens the first (bottom bottom), second (bottom top), third (top bottom), or fourth (top top) 8-bit element of each 32-bit container in the first and second source vectors to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$, before being destructively added without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the source vectors.
The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_FP8FMA)



Encoding for the FMLALLBB variant

Applies when (Q == 0 && size == 00)

```
FMLALLBB <Vd>.4S, <Vn>.16B, <Vm>.16B
```

Encoding for the FMLALLBT variant

Applies when (Q == 0 && size == 01)

```
FMLALLBT <Vd>.4S, <Vn>.16B, <Vm>.16B
```

Encoding for the FMLALLTB variant

Applies when (Q == 1 && size == 00)

```
FMLALLTB <Vd>.4S, <Vn>.16B, <Vm>.16B
```

Encoding for the FMLALLTT variant

Applies when (Q == 1 && size == 01)

```
FMLALLTT <Vd>.4S, <Vn>.16B, <Vm>.16B
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP8FMA) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let elements : integer = 128 DIV 32;
let sel : integer = UInt(Q::size[0]);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
CheckFPMREnabled(); AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(128) = V{}(d);
var result : bits(128);

for e = 0 to elements-1 do
    let element1 : bits(8)  = operand1[(4 * e + sel)*:8];
    let element2 : bits(8)  = operand2[(4 * e + sel)*:8];
    let element3 : bits(32) = operand3[e*:32];
    result[e*:32] = FP8MulAddFP{32}(element3, element1, element2, FPCR(), FPMR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLS (by element)

Floating-point fused multiply-subtract from accumulator (by element)

This instruction multiplies the vector elements in the first source SIMD&FP register by the specified value in the second source SIMD&FP register, and subtracts the results from the vector elements of the destination SIMD&FP register. All the values in this instruction are floating-point values.

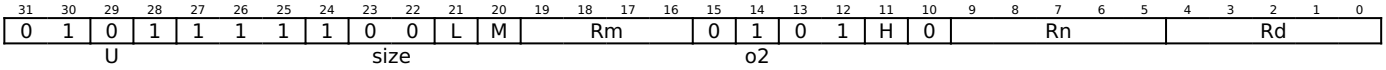
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar, half-precision , Scalar, single-precision and double-precision , Vector, half-precision and Vector, single-precision and double-precision

Scalar, half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

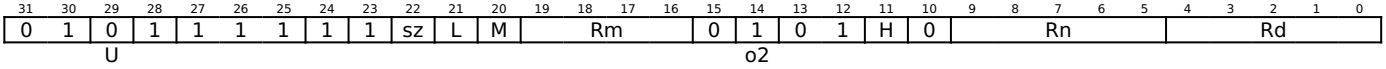
```
FMLS <Hd>, <Hn>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let idxdsize : integer{} = 64 << UInt(H);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Rd);
let index : integer = UInt(H::L::M);

let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar, single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

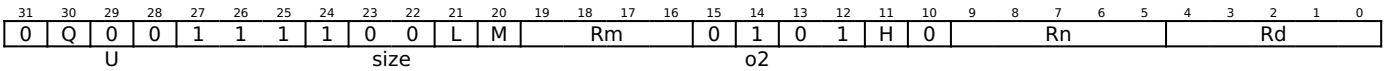
```
FMLS <V><d>, <V><n>, <Vm>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
let Rmhi : bit = M;
case sz::L of
    when '0x' => index = UInt(H::L);
    when '10' => index = UInt(H);
    when '11' => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector, half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

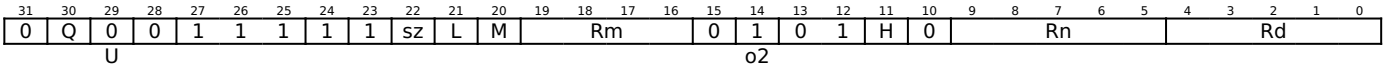
```
FMLS <Vd>.<T>, <Vn>.<T>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let idxdsize : integer{} = 64 << UInt(H);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Rd);
let index : integer = UInt(H::L::M);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector, single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMLS <Vd>.<T>, <Vn>.<T>, <Vm>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
let Rmhi : bit = M;
case sz::L of
    when '0x' => index = UInt(H::L);
    when '10' => index = UInt(H);
    when '11' => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> For the "Scalar, half-precision" and "Vector, half-precision" variants: is the name of the second SIMD&FP source register, in the range V0 to V15, encoded in the "Rm" field.

For the "Scalar, single-precision and double-precision" and "Vector, single-precision and double-precision" variants: is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> For the "Scalar, half-precision" and "Vector, half-precision" variants: is the element index, in the range 0 to 7, encoded in the "H:L:M" fields.

For the "Scalar, single-precision and double-precision" and "Vector, single-precision and double-precision" variants: is the element index, encoded in “(sz :: L :: H)”:

sz	L	<index>
0	x	UInt(H :: L)
1	0	UInt(H)
1	1	RESERVED

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<Ts> Is an element size specifier, encoded in “sz”:

sz	<Ts>
0	S
1	D

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector, half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector, single-precision and double-precision" variant: is an arrangement specifier, encoded in “(Q :: sz)”:

Q	sz	<T>
0	0	2S
0	1	RESERVED
1	0	4S
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var element1 : bits(esize);
let element2 : bits(esize) = operand2[index*:esize];
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

for e = 0 to elements-1 do
    element1 = FPNeg{esize}(operand1[e*:esize], FPCR());
    result[e*:esize] = FPMulAdd{esize}(operand3[e*:esize], element1, element2, FPCR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLS (vector)

Floating-point fused multiply-subtract from accumulator (vector)

This instruction multiplies corresponding floating-point values in the vectors in the two source SIMD&FP registers, negates the product, adds the result to the corresponding vector element of the destination SIMD&FP register, and writes the result to the destination SIMD&FP register.

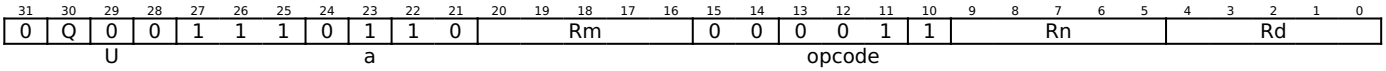
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

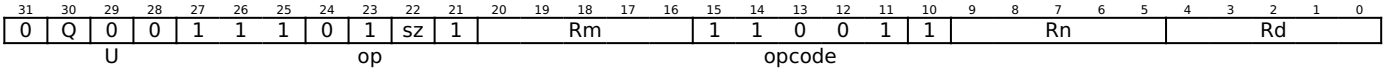
```
FMLS <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMLS <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = FPNeg{esize}(operand1[e*:esize], FPCR());
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMulAdd{esize}(operand3[e*:esize], element1, element2, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLSL, FMLSL2 (by element)

Floating-point fused multiply-subtract long from accumulator (by element)

This instruction multiplies the negated half-precision vector elements in the first source SIMD&FP register by the specified half-precision value in the second source SIMD&FP register, and accumulates the intermediate product without rounding to the corresponding single-precision vector element of the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

In Armv8.2 and Armv8.3, this is an OPTIONAL instruction. From Armv8.4, it is mandatory for all implementations to support it.

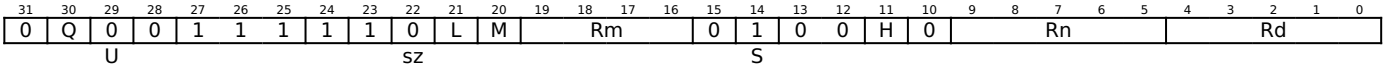
Note

ID_AA64ISAR0_EL1.FHM indicates whether this instruction is supported.

It has encodings from 2 classes: FMLSL and FMLSL2

FMLSL

(FEAT_FHM)



Encoding

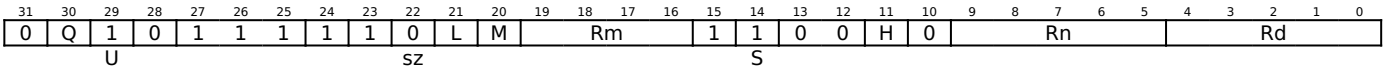
```
FMLSL <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FHM) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt('0'::Rm);    // Vm can only be in bottom 16 registers.
let index : integer = UInt(H::L::M);

let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let part : integer = 0;
```

FMLSL2

(FEAT_FHM)



Encoding

```
FMLSL2 <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FHM) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt('0'::Rm);    // Vm can only be in bottom 16 registers.
let index : integer = UInt(H::L::M);

let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let part : integer = 1;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	2H
1	4H

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

<index> Is the element index, encoded in the "H:L:M" fields.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize DIV 2) = Vpart{}(n, part);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : bits(esize DIV 2);
let element2 : bits(esize DIV 2) = operand2[index*:(esize DIV 2)];

for e = 0 to elements-1 do
    element1 = FPNeg{esize DIV 2}(operand1[e*:(esize DIV 2)], FPCR());
    result[e*:esize] = FPMulAddH(operand3[e*:esize], element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLSL, FMLSL2 (vector)

Floating-point fused multiply-subtract long from accumulator (vector)

This instruction negates the half-precision values in the vector of one SIMD&FP register, multiplies these with the corresponding half-precision values in another vector, and accumulates the intermediate product without rounding to the corresponding single-precision vector element of the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

In Armv8.2 and Armv8.3, this is an OPTIONAL instruction. From Armv8.4, it is mandatory for all implementations to support it.

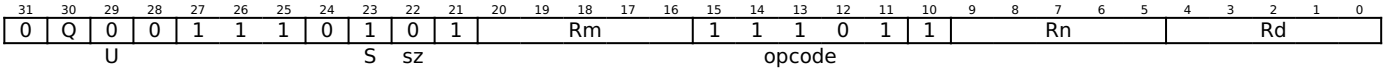
Note

ID_AA64ISAR0_EL1.FHM indicates whether this instruction is supported.

It has encodings from 2 classes: FMLSL and FMLSL2

FMLSL

(FEAT_FHM)



Encoding

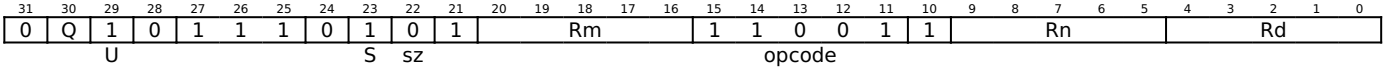
```
FMLSL <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FHM) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let part : integer = 0;
```

FMLSL2

(FEAT_FHM)



Encoding

```
FMLSL2 <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FHM) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let part : integer = 1;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	2H
1	4H

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize DIV 2) = Vpart{}(n, part);
let operand2 : bits(datasize DIV 2) = Vpart{}(m, part);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : bits(esize DIV 2);
var element2 : bits(esize DIV 2);

for e = 0 to elements-1 do
    element1 = FPNeg{esize DIV 2}(operand1[e*:(esize DIV 2)], FPCR());
    element2 = operand2[e*:(esize DIV 2)];
    result[e*:esize] = FPMulAddH(operand3[e*:esize], element1, element2, FPCR());
end;
V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

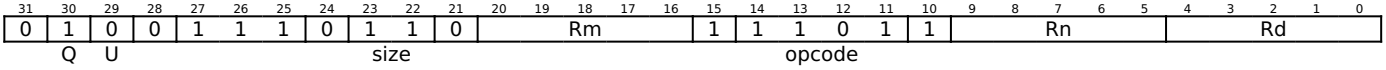
FMMLA (non-widening)

Half-precision matrix multiply-accumulate

This instruction multiplies the 2x2 matrix of half-precision values in each 64-bit segment of the first source vector by the 2x2 matrix of half-precision values in the corresponding 64-bit segment of the second source vector. The intermediate products are rounded before they are summed, and the intermediate sum is rounded before accumulation into the 2x2 half-precision matrix held in the corresponding 64-bit segment of the addend and destination vector. This is equivalent to performing a 2-way dot product per destination element.

Advanced SIMD

(FEAT_F16MM)



Encoding

```
FMMLA <Vd>.8H, <Vn>.8H, <Vm>.8H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F16MM) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(128) = V{}(d);

var result : bits(128);

for s = 0 to 1 do
    let op1 : bits(64) = operand1[s*:64];
    let op2 : bits(64) = operand2[s*:64];
    let acc : bits(64) = operand3[s*:64];
    result[s*:64] = FPMatMulAdd{64}(acc, op1, op2, 16, FPCR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMMLA (widening, 8-bit floating-point to half-precision)

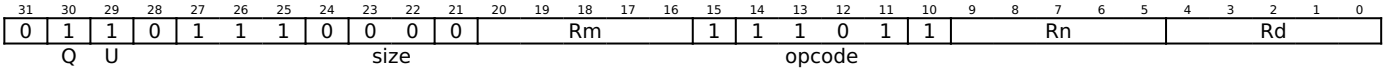
8-bit floating-point matrix multiply-accumulate to half-precision

This instruction multiplies the 2×4 matrix of 8-bit floating-point values held in each 64-bit segment of the first source vector by the 4×2 matrix of 8-bit floating-point values in the corresponding segment of the second source vector. The intermediate half-precision results, calculated as a fused sum-of-products using four adjacent 8-bit floating-point values from each of the source vectors, are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$ before being destructively added without intermediate rounding to the 2x2 half-precision matrix in the corresponding segment of the destination vector. This is equivalent to accumulating 4-way dot product per destination element.

The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_F8F16MM)



Encoding

```
FMMLA <Vd>.8H, <Vn>.16B, <Vm>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F8F16MM) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Rd);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
CheckFPMREnabled();
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(128) = V{}(d);

var result : bits(128);
var op1, op2, acc : bits(64);

for s = 0 to 1 do
    op1 = operand1[s*:64];
    op2 = operand2[s*:64];
    acc = operand3[s*:64];
    result[s*:64] = FP8MatMulAddFP{64}(acc, op1, op2, 4, FPCR(), FPMR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMMLA (widening, 8-bit floating-point to single-precision)

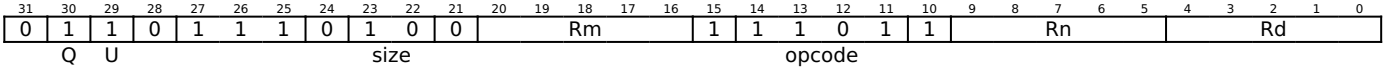
8-bit floating-point matrix multiply-accumulate to single-precision

This instruction multiplies the 2×8 matrix of 8-bit floating-point values held in the first source vector by the 8×2 matrix of 8-bit floating-point values in the second source vector. The intermediate single-precision results, calculated as a fused sum-of-products using eight adjacent 8-bit floating-point values from each of the source vectors, are scaled by $2^{-UInt(FPMR.LSCALE)}$ before being destructively added without intermediate rounding to the 2x2 single-precision matrix in the destination vector. This is equivalent to accumulating 8-way dot product per destination element.

The 8-bit floating-point encoding format for the elements of the first source vector is selected by FPMR.F8S1. The 8-bit floating-point encoding format for the elements of the second source vector is selected by FPMR.F8S2.

Advanced SIMD

(FEAT_F8F32MM)



Encoding

```
FMMLA <Vd>.4S, <Vn>.16B, <Vm>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F8F32MM) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Rd);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
CheckFPMREnabled();
AArch64_CheckFPAdvSIMDEnabled();
let op1 : bits(128) = V{}(n);
let op2 : bits(128) = V{}(m);
let acc : bits(128) = V{}(d);

V{128}(d) = FP8MatMulAddFP{128}(acc, op1, op2, 8, FPCR(), FPMR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

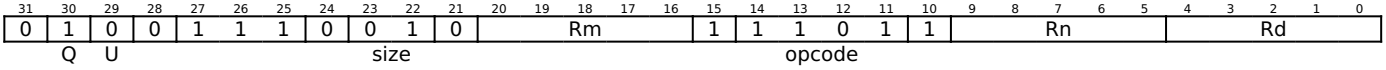
FMMLA (widening, half-precision to single-precision)

Half-precision matrix multiply-accumulate to single-precision

This instruction multiplies the 2×4 matrix of half-precision values held in the first source vector by the 4×2 matrix of half-precision values in the second source vector. The intermediate single-precision results are calculated as a sum of two fused sum-of-products using two pairs of adjacent half-precision values from each of the source vectors, and rounded before accumulation into the 2×2 matrix of single-precision results in the destination vector. This is equivalent to performing a 4-way dot product per destination element.

Advanced SIMD

(FEAT_F16F32MM)



Encoding

```
FMMLA <Vd>.4S, <Vn>.8H, <Vm>.8H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F16F32MM) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let op1 : bits(128) = V{}(n);
let op2 : bits(128) = V{}(m);
let acc : bits(128) = V{}(d);

V{128}(d) = FPMatMulAddH{128}(acc, op1, op2, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

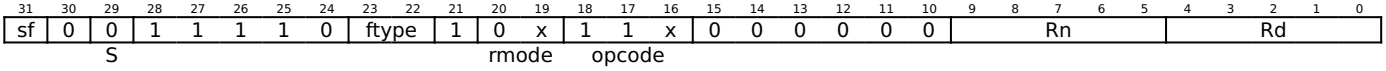
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOV (general)

Floating-point move to or from general-purpose register without conversion

This instruction transfers the contents of a SIMD&FP register to a general-purpose register, or the contents of a general-purpose register to a SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision to 32-bit variant
(FEAT_FP16)

Applies when `(sf == 0 && ftype == 11 && rmode == 00 && opcode == 110)`

```
FMOV <Wd>, <Hn>
```

Encoding for the Half-precision to 64-bit variant
(FEAT_FP16)

Applies when `(sf == 1 && ftype == 11 && rmode == 00 && opcode == 110)`

```
FMOV <Xd>, <Hn>
```

Encoding for the 32-bit to half-precision variant
(FEAT_FP16)

Applies when `(sf == 0 && ftype == 11 && rmode == 00 && opcode == 111)`

```
FMOV <Hd>, <Wn>
```

Encoding for the 32-bit to single-precision variant
(FEAT_FP)

Applies when `(sf == 0 && ftype == 00 && rmode == 00 && opcode == 111)`

```
FMOV <Sd>, <Wn>
```

Encoding for the Single-precision to 32-bit variant
(FEAT_FP)

Applies when `(sf == 0 && ftype == 00 && rmode == 00 && opcode == 110)`

```
FMOV <Wd>, <Sn>
```

Encoding for the 64-bit to half-precision variant
(FEAT_FP16)

Applies when `(sf == 1 && ftype == 11 && rmode == 00 && opcode == 111)`

```
FMOV <Hd>, <Xn>
```

Encoding for the 64-bit to double-precision variant
(FEAT_FP)

Applies when `(sf == 1 && ftype == 01 && rmode == 00 && opcode == 111)`

```
FMOV <Dd>, <Xn>
```

Encoding for the 64-bit to top half of 128-bit variant
(FEAT_FP)

Applies when `(sf == 1 && ftype == 10 && rmode == 01 && opcode == 111)`

```
FMOV <Vd>.D[1], <Xn>
```

Encoding for the Double-precision to 64-bit variant
(FEAT_FP)

Applies when `(sf == 1 && ftype == 01 && rmode == 00 && opcode == 110)`

```
FMOV <Xd>, <Dn>
```

Encoding for the Top half of 128-bit to 64-bit variant
(FEAT_FP)

Applies when `(sf == 1 && ftype == 10 && rmode == 01 && opcode == 110)`

```
FMOV <Xd>, <Vn>.D[1]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' && opcode[2:1]::rmode != '11 01' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = if ftype == '10' then 64 else (8 << UInt(ftype XOR '10'));
let part : integer = UInt(rmode[0]);
var op : FPConvOp;

case opcode[2:1]::rmode of
    when '11 00' =>        // FMOV
        if fltsize != 16 && fltsize != intsize then EndOfDecode(Decode_UNDEF); end;
        op = if opcode[0] == '1' then FPConvOp_MOV_ItoF else FPConvOp_MOV_FtoI;
    when '11 01' =>        // FMOV D[1]
        if intsize != 64 || ftype != '10' then EndOfDecode(Decode_UNDEF); end;
        op = if opcode[0] == '1' then FPConvOp_MOV_ItoF else FPConvOp_MOV_FtoI;
    otherwise =>
        unreachable;
end;
```

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);

case op of
    when FPConvOp_MOV_FtoI =>
        fltval = Vpart{fltsize}(n, part);
        X{intsize}(d) = ZeroExtend{intsize}(fltval);
    when FPConvOp_MOV_ItoF =>
        intval = X{intsize}(n);
        Vpart{fltsize}(d, part) = intval[fltsize-1:0];
    otherwise =>
        unreachable;
end;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

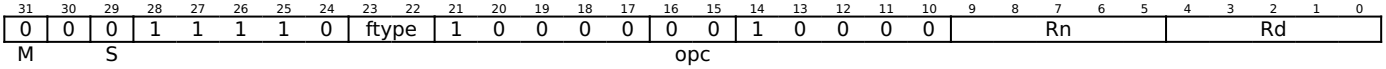
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOV (register)

Floating-point move register without conversion

This instruction copies the floating-point value in the SIMD&FP source register to the SIMD&FP destination register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FMOV <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FMOV <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FMOV <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = Zeros{};

result[0+:esize] = operand;

V{128}(d) = result;
```

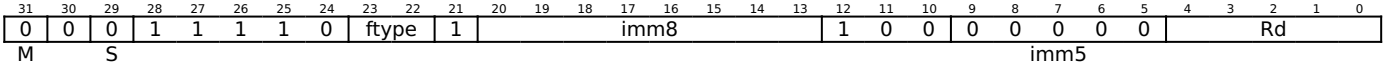
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOV (scalar, immediate)

Floating-point move immediate (scalar)

This instruction copies a floating-point immediate constant into the SIMD&FP destination register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FMOV <Hd>, #<imm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FMOV <Sd>, #<imm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FMOV <Dd>, #<imm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);

let datasize : integer{} = 8 << UInt(ftype XOR '10');
let imm : bits(datasize) = VFPExpandImm{}(imm8);
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<imm>	Is a signed floating-point constant with 3-bit exponent and normalized 4 bits of precision, encoded in the "imm8" field. For details of the range of constants available and the encoding of <imm>, see *Modified immediate constants in A64 floating-point instructions*.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

Operation

```
AArch64_CheckFPEnabled();

V{datasize}(d) = imm;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

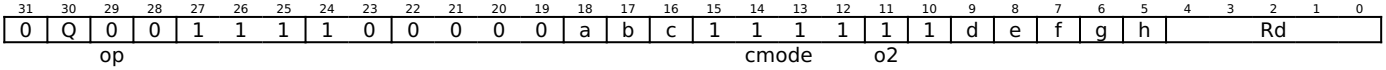
FMOV (vector, immediate)

Floating-point move immediate (vector)

This instruction copies an immediate floating-point constant into every element of the SIMD&FP destination register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

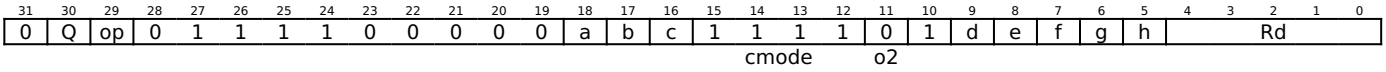
```
FMOV <Vd>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let rd : integer = UInt(Rd);

let datasize : integer{} = 64 << UInt(Q);
let imm8 : bits(8) = a::b::c::d::e::f::g::h;
let imm16 : bits(16) = imm8[7]::NOT(imm8[6])::Replicate{2}(imm8[6])::imm8[5:0]::Zeros{6};
let imm : bits(datasize) = Replicate{}(imm16);
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding for the Single-precision variant

Applies when `(op == 0)`

```
FMOV <Vd>.<T>, #<imm>
```

Encoding for the Double-precision variant

Applies when `(Q == 1 && op == 1)`

```
FMOV <Vd>.2D, #<imm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if cmode::op == '11111' then
    // FMOV Dn,#imm is in main FP instruction set
    if Q == '0' then EndOfDecode(Decode_UNDEF); end;
end;

let rd : integer = UInt(Rd);
let datasize : integer{} = 64 << UInt(Q);
let imm64 : bits(64) = AdvSIMDExpandImm(op, cmode, a::b::c::d::e::f::g::h);
let imm : bits(datasize) = Replicate{}(imm64);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	2S
1	4S

<imm> Is a signed floating-point constant with 3-bit exponent and normalized 4 bits of precision, encoded in "a:b:c:d:e:f:g:h". For details of the range of constants available and the encoding of <imm>, see *Modified immediate constants in A64 floating-point instructions*.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
V{datasize}(rd) = imm;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

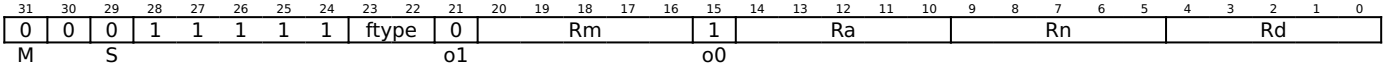
FMSUB

Floating-point fused multiply-subtract (scalar)

This instruction multiplies the values of the first two SIMD&FP source registers, negates the product, adds that to the value of the third SIMD&FP source register, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FMSUB <Hd>, <Hn>, <Hm>, <Ha>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FMSUB <Sd>, <Sn>, <Sm>, <Sa>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FMSUB <Dd>, <Dn>, <Dm>, <Da>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Ha>	Is the 16-bit name of the third SIMD&FP source register holding the minuend, encoded in the "Ra" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Sa>	Is the 32-bit name of the third SIMD&FP source register holding the minuend, encoded in the "Ra" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Da>	Is the 64-bit name of the third SIMD&FP source register holding the minuend, encoded in the "Ra" field.

Operation

```
AArch64_CheckFPEnabled();

let addend : bits(esize)   = V{esize}(a);
let operand1 : bits(esize) = FPNeg{esize}(V{esize}(n), FPCR());
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(a) else Zeros{128};

result[0+:esize] = FPMulAdd{esize}(addend, operand1, operand2, FPCR());
V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMUL (by element)

Floating-point multiply (by element)

This instruction multiplies the vector elements in the first source SIMD&FP register by the specified value in the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are floating-point values.

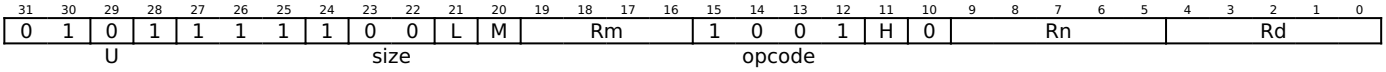
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar, half-precision , Scalar, single-precision and double-precision , Vector, half-precision and Vector, single-precision and double-precision

Scalar, half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

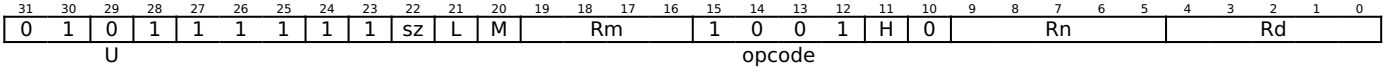
```
FMUL <Hd>, <Hn>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let idxdsize : integer{} = 64 << UInt(H);
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let index : integer = UInt(H::L::M);

let esize : integer{} = 16;
let datasize : integer{} = esize;

let elements : integer = 1;
```

Scalar, single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

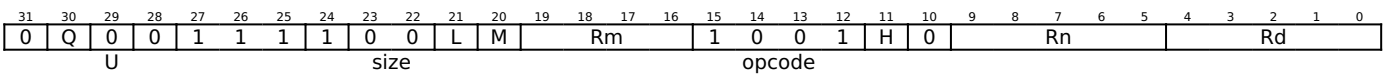
```
FMUL <V><d>, <V><n>, <Vm>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
let Rmhi : bit = M;
case sz::L of
    when '0x' => index = UInt(H::L);
    when '10' => index = UInt(H);
    when '11' => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector, half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

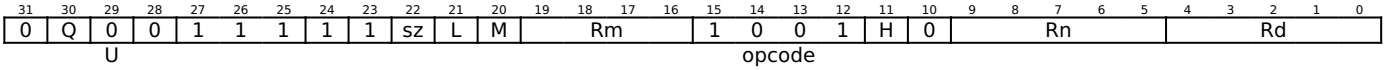
```
FMUL <Vd>.<T>, <Vn>.<T>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let idxdsize : integer{} = 64 << UInt(H);
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let index : integer = UInt(H::L::M);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector, single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMUL <Vd>.<T>, <Vn>.<T>, <Vm>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
let Rmhi : bit = M;
case sz::L of
    when '0x' => index = UInt(H::L);
    when '10' => index = UInt(H);
    when '11' => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> For the "Scalar, half-precision" and "Vector, half-precision" variants: is the name of the second SIMD&FP source register, in the range V0 to V15, encoded in the "Rm" field.

For the "Scalar, single-precision and double-precision" and "Vector, single-precision and double-precision" variants: is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> For the "Scalar, half-precision" and "Vector, half-precision" variants: is the element index, in the range 0 to 7, encoded in the "H:L:M" fields.

For the "Scalar, single-precision and double-precision" and "Vector, single-precision and double-precision" variants: is the element index, encoded in “(sz :: L :: H)”:

sz	L	<index>
0	x	UInt(H :: L)
1	0	UInt(H)
1	1	RESERVED

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<Ts> Is an element size specifier, encoded in “sz”:

sz	<Ts>
0	S
1	D

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector, half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector, single-precision and double-precision" variant: is an arrangement specifier, encoded in “(Q :: sz)”:

Q	sz	<T>
0	0	2S
0	1	RESERVED
1	0	4S
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
var element1 : bits(esize);
let element2 : bits(esize) = operand2[index*:esize];

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(n) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    result[e*:esize] = FPMul{esize}(element1, element2, FPCR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

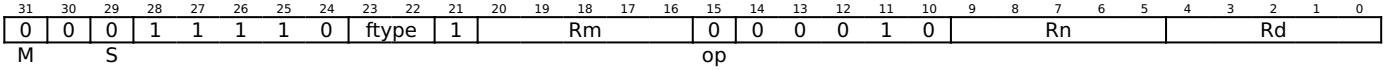
FMUL (scalar)

Floating-point multiply (scalar)

This instruction multiplies the floating-point values of the two source SIMD&FP registers, and writes the result to the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FMUL <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FMUL <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FMUL <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize]  = FPMul{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMUL (vector)

Floating-point multiply (vector)

This instruction multiplies corresponding floating-point values in the vectors in the two source SIMD&FP registers, places the result in a vector, and writes the vector to the destination SIMD&FP register.

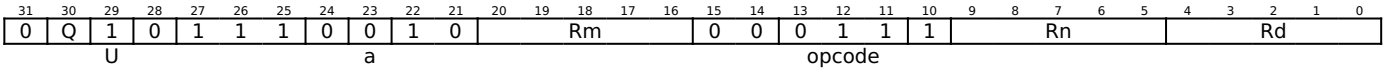
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

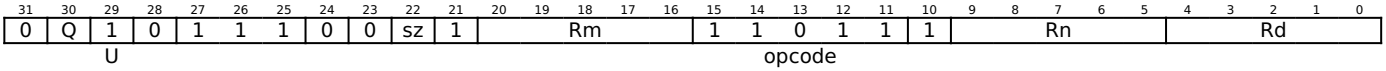
```
FMUL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMUL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMul{esize}(element1, element2, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMULX

Floating-point multiply extended

This instruction multiplies corresponding floating-point values in the vectors of the two source SIMD&FP registers, places the resulting floating-point values in a vector, and writes the vector to the destination SIMD&FP register.

If one value is zero and the other value is infinite, the result is 2.0. In this case, the result is negative if only one of the values is negative, otherwise the result is positive.

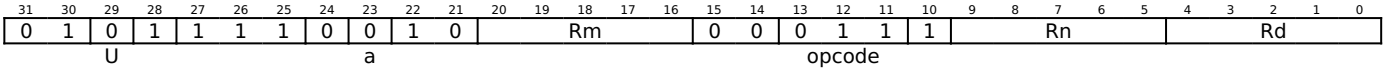
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

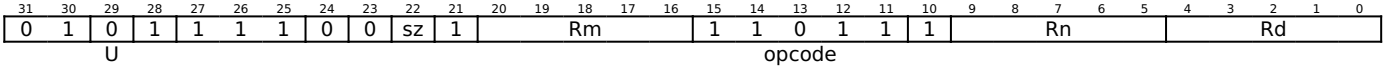
```
FMULX <Hd>, <Hn>, <Hm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

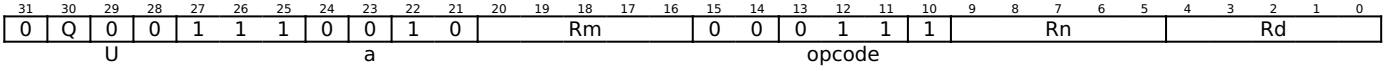
```
FMULX <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

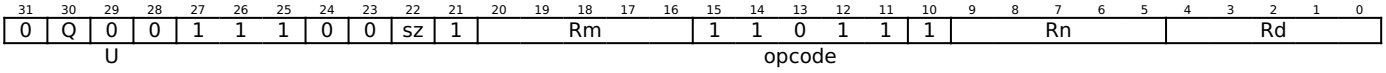
```
FMULX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMULX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
if elements == 1 then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

var element1 : bits(esize);
var element2 : bits(esize);
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(n) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPMulX{esize}(element1, element2, FPCR());
end;
V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMULX (by element)

Floating-point multiply extended (by element)

This instruction multiplies the floating-point values in the vector elements in the first source SIMD&FP register by the specified floating-point value in the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register.

If one value is zero and the other value is infinite, the result is 2.0. In this case, the result is negative if only one of the values is negative, otherwise the result is positive.

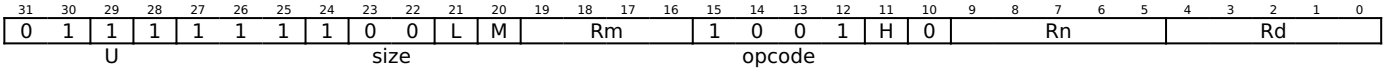
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar, half-precision , Scalar, single-precision and double-precision , Vector, half-precision and Vector, single-precision and double-precision

Scalar, half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

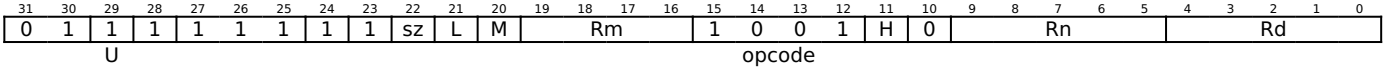
```
FMULX <Hd>, <Hn>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let idxdsize : integer{} = 64 << UInt(H);
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let index : integer = UInt(H::L::M);

let esize : integer{} = 16;
let datasize : integer{} = esize;

let elements : integer = 1;
```

Scalar, single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

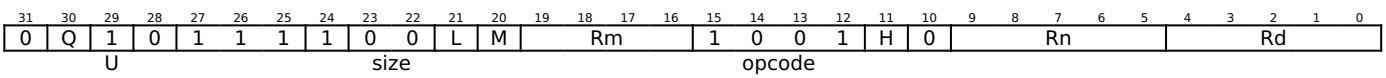
```
FMULX <V><d>, <V><n>, <Vm>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
let Rmhi : bit = M;
case sz::L of
    when '0x' => index = UInt(H::L);
    when '10' => index = UInt(H);
    when '11' => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector, half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

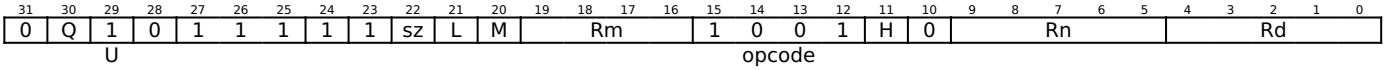
```
FMULX <Vd>.<T>, <Vn>.<T>, <Vm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let idxdsize : integer{} = 64 << UInt(H);
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let index : integer = UInt(H::L::M);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector, single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FMULX <Vd>.<T>, <Vn>.<T>, <Vm>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
let Rmhi : bit = M;
case sz::L of
    when '0x' => index = UInt(H::L);
    when '10' => index = UInt(H);
    when '11' => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> For the "Scalar, half-precision" and "Vector, half-precision" variants: is the name of the second SIMD&FP source register, in the range V0 to V15, encoded in the "Rm" field.

For the "Scalar, single-precision and double-precision" and "Vector, single-precision and double-precision" variants: is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> For the "Scalar, half-precision" and "Vector, half-precision" variants: is the element index, in the range 0 to 7, encoded in the "H:L:M" fields.

For the "Scalar, single-precision and double-precision" and "Vector, single-precision and double-precision" variants: is the element index, encoded in “(sz :: L :: H)”:

sz	L	<index>
0	x	UInt(H :: L)
1	0	UInt(H)
1	1	RESERVED

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<Ts> Is an element size specifier, encoded in “sz”:

sz	<Ts>
0	S
1	D

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector, half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector, single-precision and double-precision" variant: is an arrangement specifier, encoded in “(Q :: sz)”:

Q	sz	<T>
0	0	2S
0	1	RESERVED
1	0	4S
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
var element1 : bits(esize);
let element2 : bits(esize) = operand2[index*:esize];

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(n) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    result[e*:esize] = FPMulX{esize}(element1, element2, FPCR());
end;

V{128}(d) = result;
```

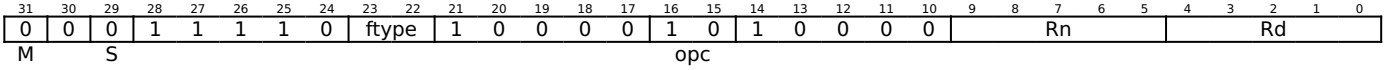
2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FNEG (scalar)

Floating-point negate (scalar)

This instruction negates the value in the SIMD&FP source register and writes the result to the SIMD&FP destination register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FNEG <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FNEG <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FNEG <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPNeg{esize}(operand, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FNEG (vector)

Floating-point negate (vector)

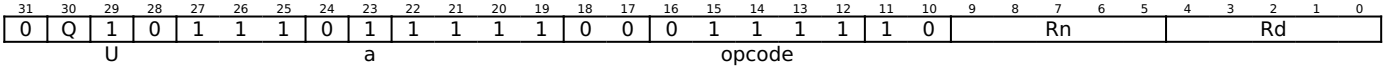
This instruction negates the value of each vector element in the source SIMD&FP register, writes the result to a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

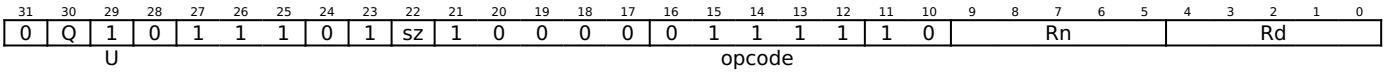
```
FNEG <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FNEG <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[e*:esize];
    result[e*:esize] = FPNeg{esize}(element, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

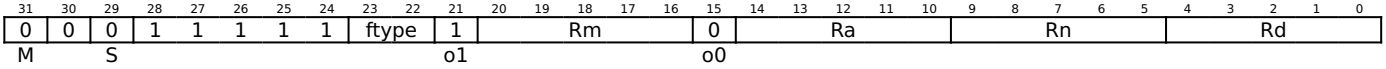
FNMADD

Floating-point negated fused multiply-add (scalar)

This instruction multiplies the values of the first two SIMD&FP source registers, negates the product, subtracts the value of the third SIMD&FP source register, and writes the result to the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FNMADD <Hd>, <Hn>, <Hm>, <Ha>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FNMADD <Sd>, <Sn>, <Sm>, <Sa>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FNMADD <Dd>, <Dn>, <Dm>, <Da>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Ha>	Is the 16-bit name of the third SIMD&FP source register holding the addend, encoded in the "Ra" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Sa>	Is the 32-bit name of the third SIMD&FP source register holding the addend, encoded in the "Ra" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Da>	Is the 64-bit name of the third SIMD&FP source register holding the addend, encoded in the "Ra" field.

Operation

```
AArch64_CheckFPEnabled();

let addend : bits(esize)   = FPNeg{esize}(V{esize}(a), FPCR());
let operand1 : bits(esize) = FPNeg{esize}(V{esize}(n), FPCR());
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(a) else Zeros{128};

result[0+:esize] = FPMulAdd{esize}(addend, operand1, operand2, FPCR());
V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

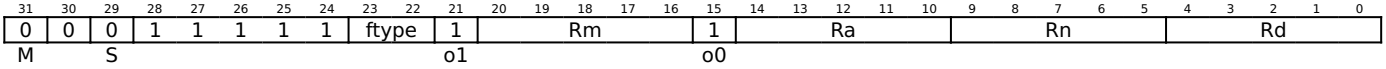
FNMSUB

Floating-point negated fused multiply-subtract (scalar)

This instruction multiplies the values of the first two SIMD&FP source registers, subtracts the value of the third SIMD&FP source register, and writes the result to the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant
(FEAT_FP16)

Applies when `(ftype == 11)`

```
FNMSUB <Hd>, <Hn>, <Hm>, <Ha>
```

Encoding for the Single-precision variant
(FEAT_FP)

Applies when `(ftype == 00)`

```
FNMSUB <Sd>, <Sn>, <Sm>, <Sa>
```

Encoding for the Double-precision variant
(FEAT_FP)

Applies when `(ftype == 01)`

```
FNMSUB <Dd>, <Dn>, <Dm>, <Da>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Ha>	Is the 16-bit name of the third SIMD&FP source register holding the minuend, encoded in the "Ra" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Sa>	Is the 32-bit name of the third SIMD&FP source register holding the minuend, encoded in the "Ra" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register holding the multiplicand, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register holding the multiplier, encoded in the "Rm" field.
<Da>	Is the 64-bit name of the third SIMD&FP source register holding the minuend, encoded in the "Ra" field.

Operation

```
AArch64_CheckFPEnabled();

let addend : bits(esize)   = FPNeg{esize}(V{esize}(a), FPCR());
let operand1 : bits(esize) = V{esize}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(a) else Zeros{128};

result[0+:esize] = FPMulAdd{esize}(addend, operand1, operand2, FPCR());
V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

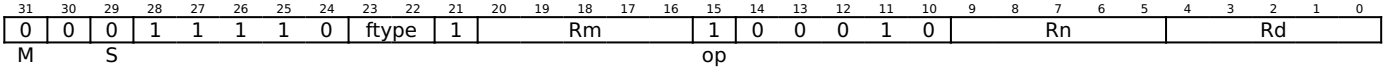
FNMUL (scalar)

Floating-point multiply-negate (scalar)

This instruction multiplies the floating-point values of the two source SIMD&FP registers, and writes the negation of the result to the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FNMUL <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FNMUL <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FNMUL <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

let product : bits(esize) = FPMul{}(operand1, operand2, FPCR());
result[0+:esize]  = FPNeg{esize}(product, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRECPE

Floating-point reciprocal estimate

This instruction finds an approximate reciprocal estimate for each vector element in the source SIMD&FP register, places the result in a vector, and writes the vector to the destination SIMD&FP register.

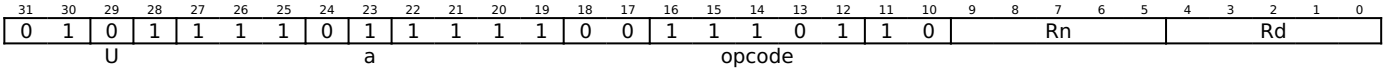
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

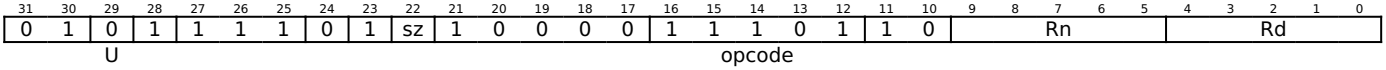
```
FRECPE <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

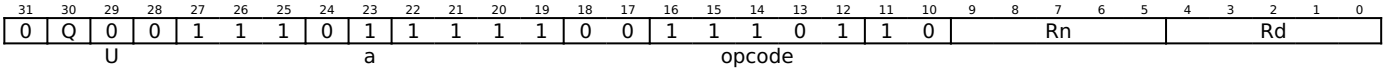
```
FRECPE <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



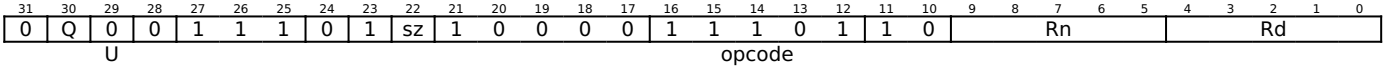
Encoding

```
FRECPE <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision
(FEAT_AdvSIMD)



Encoding

```
FRECPE <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;
let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

for e = 0 to elements-1 do
    let element : bits(esize) = operand[e*:esize];
    result[e*:esize] = FPRecipEstimate{esize}(element, FPCR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRECPS

Floating-point reciprocal step

This instruction multiplies the corresponding floating-point values in the vectors of the two source SIMD&FP registers, subtracts each of the products from 2.0, places the resulting floating-point values in a vector, and writes the vector to the destination SIMD&FP register.

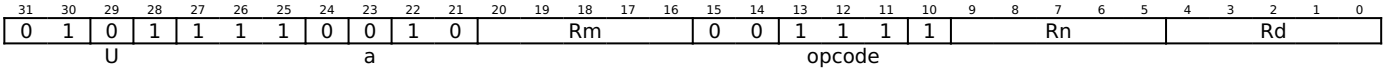
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

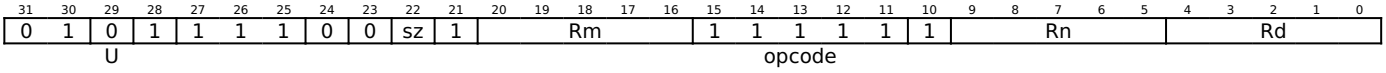
```
FRECPS <Hd>, <Hn>, <Hm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

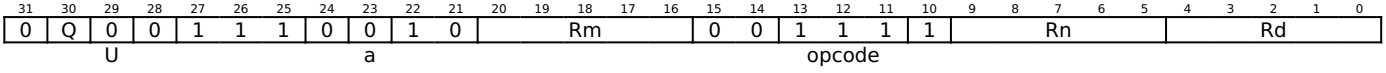
```
FRECPS <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

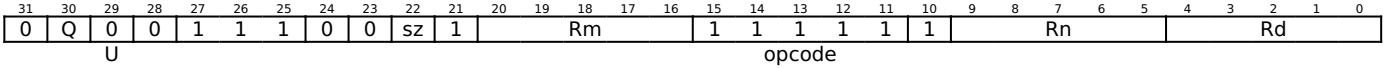
```
FRECPS <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FRECPS <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
if elements == 1 then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

var element1 : bits(esize);
var element2 : bits(esize);
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(n) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPRecipStepFused{esize}(element1, element2, FPCR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRECPX

Floating-point reciprocal exponent (scalar)

This instruction finds an approximate reciprocal exponent for the source SIMD&FP register and writes the result to the destination SIMD&FP register.

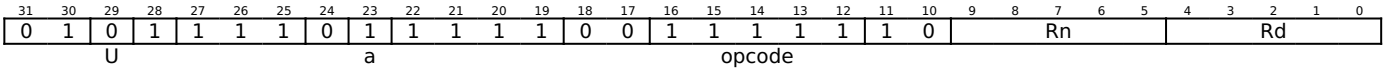
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

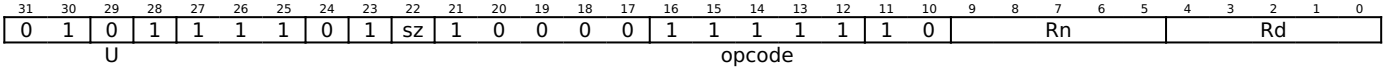
```
FRECPX <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 16;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FRECPX <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
let operand : bits(esize) = V{}(n);

let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

result[0+:esize] = FPRecpX{esize}(operand, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT32X (scalar)

Floating-point round to 32-bit integer, using current rounding mode (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value that fits into a 32-bit integer size using the rounding mode that is determined by the FPCR, and writes the result to the SIMD&FP destination register.

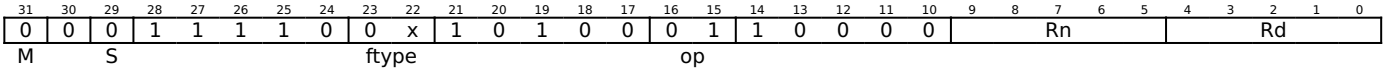
A zero input returns a zero result with the same sign. When the result value is not numerically equal to the input value, an Inexact exception is raised. When the input is infinite, NaN or out-of-range, the instruction returns the most negative integer representable in the destination size, and an Invalid Operation floating-point exception is raised.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FRINTTS)



Encoding for the Single-precision variant

Applies when `(ftype == 00)`

```
FRINT32X <Sd>, <Sn>
```

Encoding for the Double-precision variant

Applies when `(ftype == 01)`

```
FRINT32X <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FRINTTS) then EndOfDecode(Decode_UNDEF); end;
if ftype IN {'1x'} then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(ftype[0]);
let intsize : integer{} = 32;
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
let rounding : FPRounding = FPRoundingMode(FPCR());
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundIntN{esize}(operand, FPCR(), rounding, intsize);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT32X (vector)

Floating-point round to 32-bit integer, using current rounding mode (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values that fit into a 32-bit integer size using the rounding mode that is determined by the FPCR, and writes the result to the SIMD&FP destination register.

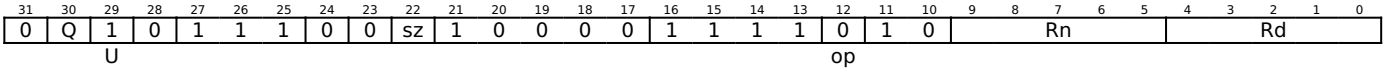
A zero input returns a zero result with the same sign. When one of the result values is not numerically equal to the corresponding input value, an Inexact exception is raised. When an input is infinite, NaN or out-of-range, the instruction returns for the corresponding result value the most negative integer representable in the destination size, and an Invalid Operation floating-point exception is raised.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector single-precision and double-precision

(FEAT_FRINTTS)



Encoding

```
FRINT32X <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FRINTTS) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let intsize : integer{} = 32;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
let rounding : FPRounding = FPRoundingMode(FPCR());
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundIntN{esize}(element, FPCR(), rounding, intsize);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT32Z (scalar)

Floating-point round to 32-bit integer toward zero (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value that fits into a 32-bit integer size using the Round towards Zero rounding mode, and writes the result to the SIMD&FP destination register.

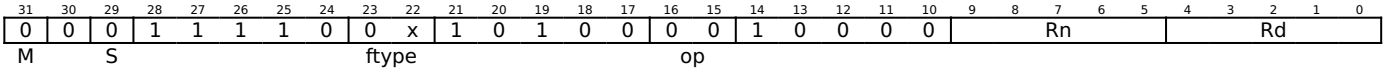
A zero input returns a zero result with the same sign. When the result value is not numerically equal to the input value, an Inexact exception is raised. When the input is infinite, NaN or out-of-range, the instruction returns the most negative integer representable in the destination size, and an Invalid Operation floating-point exception is raised.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FRINTTS)



Encoding for the Single-precision variant

Applies when `(ftype == 00)`

```
FRINT32Z <Sd>, <Sn>
```

Encoding for the Double-precision variant

Applies when `(ftype == 01)`

```
FRINT32Z <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FRINTTS) then EndOfDecode(Decode_UNDEF); end;
if ftype IN {'1x'} then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(ftype[0]);
let intsize : integer{} = 32;

let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundIntN{esize}(operand, FPCR(), rounding, intsize);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT32Z (vector)

Floating-point round to 32-bit integer toward zero (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values that fit into a 32-bit integer size using the Round towards Zero rounding mode, and writes the result to the SIMD&FP destination register.

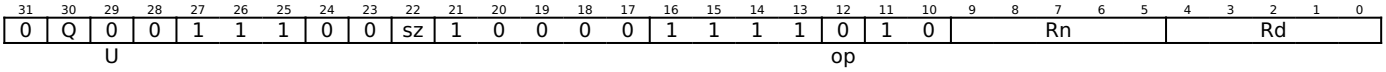
A zero input returns a zero result with the same sign. When one of the result values is not numerically equal to the corresponding input value, an Inexact exception is raised. When an input is infinite, NaN or out-of-range, the instruction returns for the corresponding result value the most negative integer representable in the destination size, and an Invalid Operation floating-point exception is raised.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector single-precision and double-precision

(FEAT_FRINTTS)



Encoding

```
FRINT32Z <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FRINTTS) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let intsize : integer{} = 32;
let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundIntN{esize}(element, FPCR(), rounding, intsize);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT64X (scalar)

Floating-point round to 64-bit integer, using current rounding mode (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value that fits into a 64-bit integer size using the rounding mode that is determined by the FPCR, and writes the result to the SIMD&FP destination register.

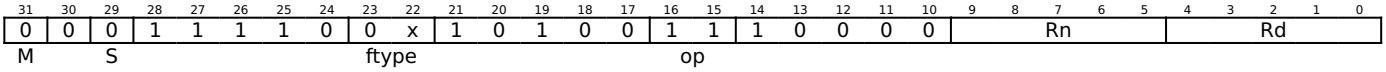
A zero input returns a zero result with the same sign. When the result value is not numerically equal to the input value, an Inexact exception is raised. When the input is infinite, NaN or out-of-range, the instruction returns the most negative integer representable in the destination size, and an Invalid Operation floating-point exception is raised.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FRINTTS)



Encoding for the Single-precision variant

Applies when `(ftype == 00)`

```
FRINT64X <Sd>, <Sn>
```

Encoding for the Double-precision variant

Applies when `(ftype == 01)`

```
FRINT64X <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FRINTTS) then EndOfDecode(Decode_UNDEF); end;
if ftype IN {'1x'} then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(ftype[0]);
let intsize : integer{} = 64;
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
let rounding : FPRounding = FPRoundingMode(FPCR());
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundIntN{esize}(operand, FPCR(), rounding, intsize);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT64X (vector)

Floating-point round to 64-bit integer, using current rounding mode (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values that fit into a 64-bit integer size using the rounding mode that is determined by the FPCR, and writes the result to the SIMD&FP destination register.

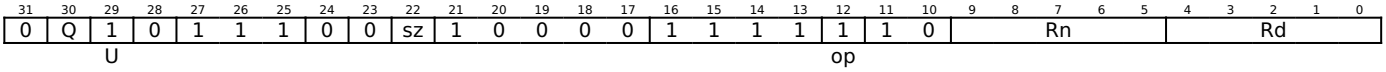
A zero input returns a zero result with the same sign. When one of the result values is not numerically equal to the corresponding input value, an Inexact exception is raised. When an input is infinite, NaN or out-of-range, the instruction returns for the corresponding result value the most negative integer representable in the destination size, and an Invalid Operation floating-point exception is raised.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector single-precision and double-precision

(FEAT_FRINTTS)



Encoding

```
FRINT64X <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FRINTTS) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let intsize : integer{} = 64;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
let rounding : FPRounding = FPRoundingMode(FPCR());
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundIntN{esize}(element, FPCR(), rounding, intsize);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT64Z (scalar)

Floating-point round to 64-bit integer toward zero (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value that fits into a 64-bit integer size using the Round towards Zero rounding mode, and writes the result to the SIMD&FP destination register.

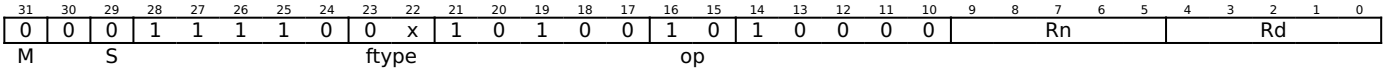
A zero input returns a zero result with the same sign. When the result value is not numerically equal to the input value, an Inexact exception is raised. When the input is infinite, NaN or out-of-range, the instruction returns the most negative integer representable in the destination size, and an Invalid Operation floating-point exception is raised.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Floating-point

(FEAT_FRINTTS)



Encoding for the Single-precision variant

Applies when `(ftype == 00)`

```
FRINT64Z <Sd>, <Sn>
```

Encoding for the Double-precision variant

Applies when `(ftype == 01)`

```
FRINT64Z <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FRINTTS) then EndOfDecode(Decode_UNDEF); end;
if ftype IN {'1x'} then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(ftype[0]);
let intsize : integer{} = 64;

let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundIntN{esize}(operand, FPCR(), rounding, intsize);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT64Z (vector)

Floating-point round to 64-bit integer toward zero (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values that fit into a 64-bit integer size using the Round towards Zero rounding mode, and writes the result to the SIMD&FP destination register.

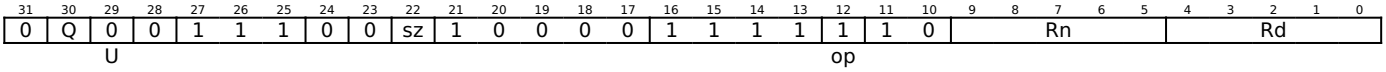
A zero input returns a zero result with the same sign. When one of the result values is not numerically equal to the corresponding input value, an Inexact exception is raised. When an input is infinite, NaN or out-of-range, the instruction returns for the corresponding result value the most negative integer representable in the destination size, and an Invalid Operation floating-point exception is raised.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector single-precision and double-precision

(FEAT_FRINTTS)



Encoding

```
FRINT64Z <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FRINTTS) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let intsize : integer{} = 64;
let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundIntN{esize}(element, FPCR(), rounding, intsize);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTA (scalar)

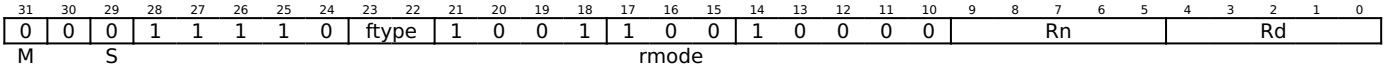
Floating-point round to integral, to nearest with ties to away (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value of the same size using the Round to Nearest with Ties to Away rounding mode, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FRINTA <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FRINTA <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FRINTA <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEAWAY;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundInt{esize}(operand, FPCR(), rounding, exact);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTA (vector)

Floating-point round to integral, to nearest with ties to away (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values of the same size using the Round to Nearest with Ties to Away rounding mode, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

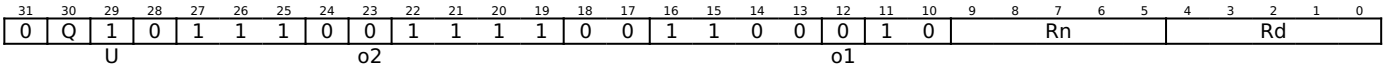
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

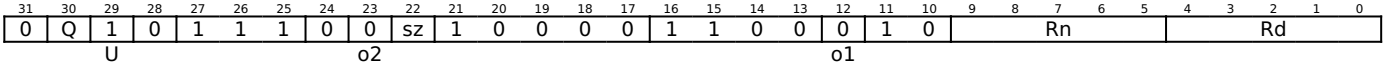
```
FRINTA <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEAWAY;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FRINTA <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEAWAY;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundInt{esize}(element, FPCR(), rounding, exact);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTI (scalar)

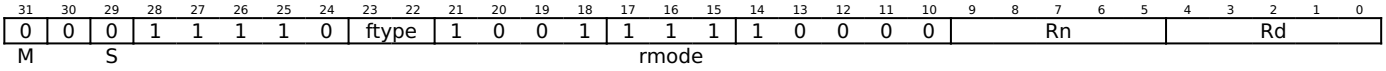
Floating-point round to integral, using current rounding mode (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value of the same size using the rounding mode that is determined by the FPCR, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FRINTI <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FRINTI <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FRINTI <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
let exact : boolean = FALSE;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
let rounding : FPRounding = FPRoundingMode(FPCR());
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundInt{esize}(operand, FPCR(), rounding, exact);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTI (vector)

Floating-point round to integral, using current rounding mode (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values of the same size using the rounding mode that is determined by the FPCR, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

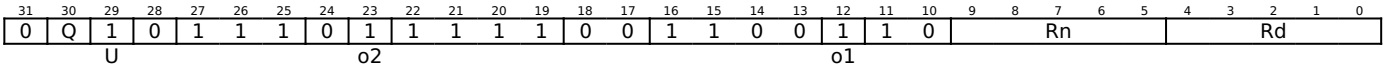
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

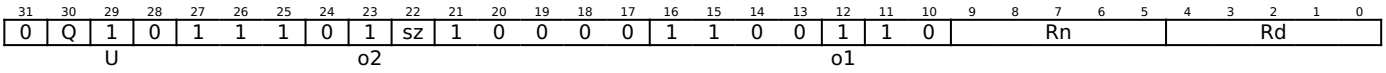
```
FRINTI <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FRINTI <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);
let rounding : FPRounding = FPRoundingMode(FPCR());

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundInt{esize}(element, FPCR(), rounding, exact);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTM (scalar)

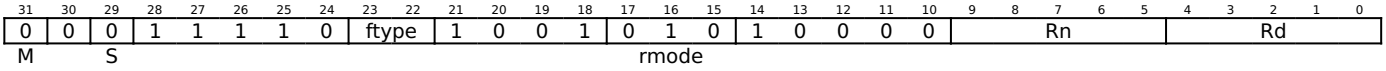
Floating-point round to integral, toward minus infinity (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value of the same size using the Round towards Minus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FRINTM <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FRINTM <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FRINTM <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_NEGINF;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundInt{esize}(operand, FPCR(), rounding, exact);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTM (vector)

Floating-point round to integral, toward minus infinity (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values of the same size using the Round towards Minus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

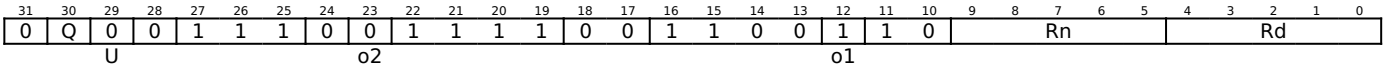
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

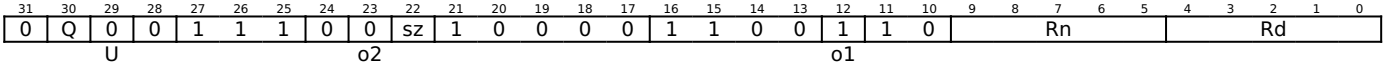
```
FRINTM <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_NEGINF;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FRINTM <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_NEGINF;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundInt{esize}(element, FPCR(), rounding, exact);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTN (scalar)

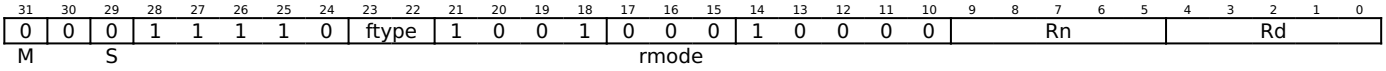
Floating-point round to integral, to nearest with ties to even (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value of the same size using the Round to Nearest rounding mode, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FRINTN <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FRINTN <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FRINTN <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEEVEN;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundInt{esize}(operand, FPCR(), rounding, exact);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTN (vector)

Floating-point round to integral, to nearest with ties to even (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values of the same size using the Round to Nearest rounding mode, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

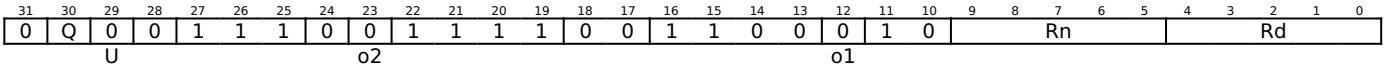
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

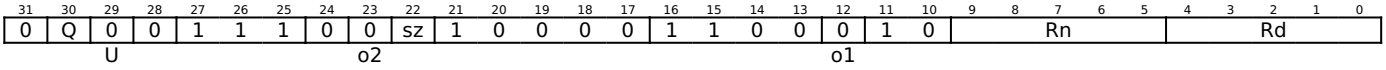
```
FRINTN <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEEVEN;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FRINTN <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEEVEN;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundInt{esize}(element, FPCR(), rounding, exact);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTP (scalar)

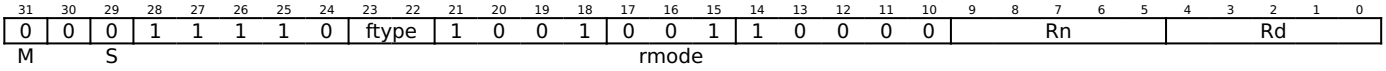
Floating-point round to integral, toward plus infinity (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value of the same size using the Round towards Plus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FRINTP <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FRINTP <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FRINTP <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_POSINF;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundInt{esize}(operand, FPCR(), rounding, exact);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTP (vector)

Floating-point round to integral, toward plus infinity (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values of the same size using the Round towards Plus Infinity rounding mode, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

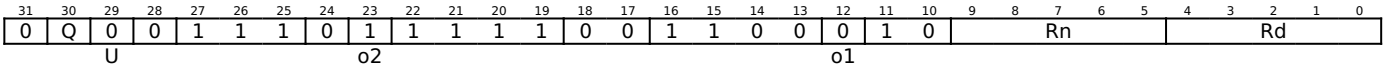
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

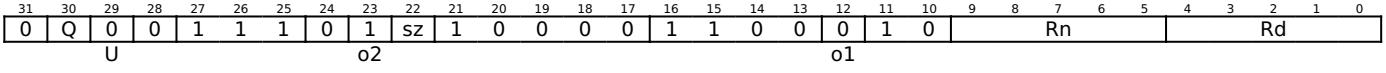
```
FRINTP <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_POSINF;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FRINTP <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_POSINF;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundInt{esize}(element, FPCR(), rounding, exact);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTX (scalar)

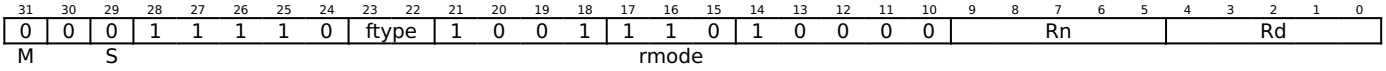
Floating-point round to integral exact, using current rounding mode (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value of the same size using the rounding mode that is determined by the FPCR, and writes the result to the SIMD&FP destination register.

When the result value is not numerically equal to the input value, an Inexact exception is raised. A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FRINTX <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FRINTX <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FRINTX <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
let exact : boolean = TRUE;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
let rounding : FPRounding = FPRoundingMode(FPCR());
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundInt{esize}(operand, FPCR(), rounding, exact);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTX (vector)

Floating-point round to integral exact, using current rounding mode (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values of the same size using the rounding mode that is determined by the FPCR, and writes the result to the SIMD&FP destination register.

When a result value is not numerically equal to the corresponding input value, an Inexact exception is raised. A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

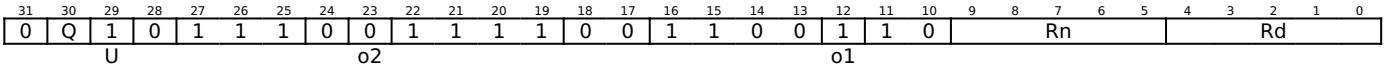
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

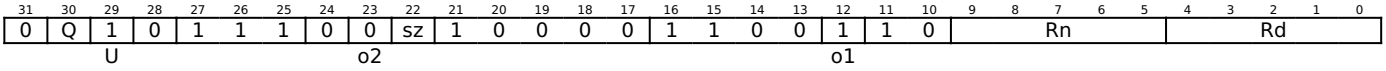
```
FRINTX <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = TRUE;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FRINTX <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = TRUE;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);
let rounding : FPRounding = FPRoundingMode(FPCR());

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundInt{esize}(element, FPCR(), rounding, exact);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTZ (scalar)

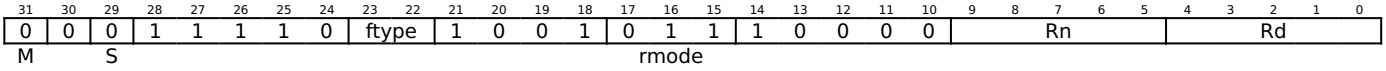
Floating-point round to integral, toward zero (scalar)

This instruction rounds a floating-point value in the SIMD&FP source register to an integral floating-point value of the same size using the Round towards Zero rounding mode, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FRINTZ <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FRINTZ <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FRINTZ <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPRoundInt{esize}(operand, FPCR(), rounding, exact);

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTZ (vector)

Floating-point round to integral, toward zero (vector)

This instruction rounds a vector of floating-point values in the SIMD&FP source register to integral floating-point values of the same size using the Round towards Zero rounding mode, and writes the result to the SIMD&FP destination register.

A zero input gives a zero result with the same sign, an infinite input gives an infinite result with the same sign, and a NaN is propagated as for normal arithmetic.

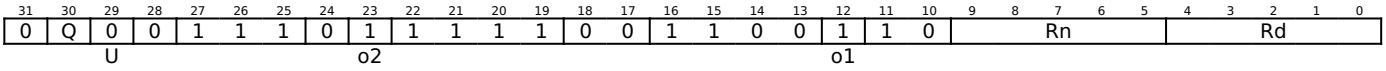
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

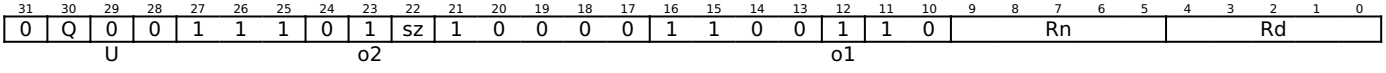
```
FRINTZ <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FRINTZ <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FPRoundInt{esize}(element, FPCR(), rounding, exact);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRSQRTE

Floating-point reciprocal square root estimate

This instruction calculates an approximate square root for each vector element in the source SIMD&FP register, places the result in a vector, and writes the vector to the destination SIMD&FP register.

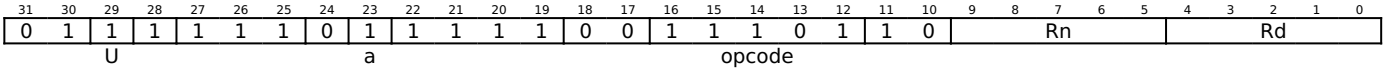
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

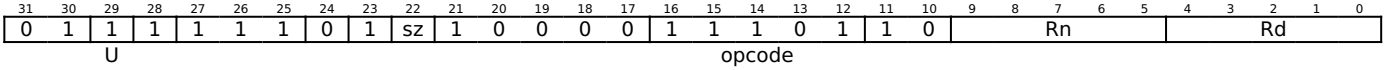
```
FRSQRTE <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

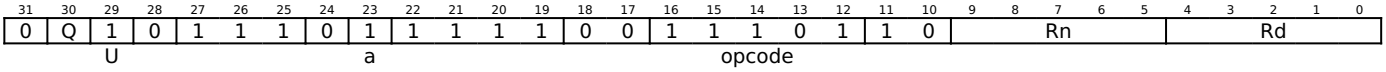
```
FRSQRTE <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



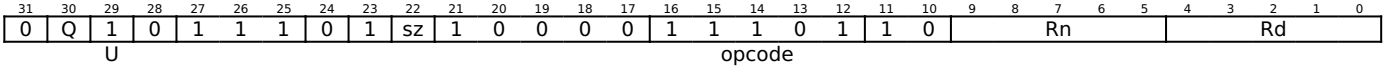
Encoding

```
FRSQRTE <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision
(FEAT_AdvSIMD)



Encoding

```
FRSQRTE <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;
let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

for e = 0 to elements-1 do
    let element : bits(esize) = operand[e*:esize];
    result[e*:esize] = FPRSqrtEstimate{esize}(element, FPCR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRSQRTS

Floating-point reciprocal square root step

This instruction multiplies corresponding floating-point values in the vectors of the two source SIMD&FP registers, subtracts each of the products from 3.0, divides these results by 2.0, places the results into a vector, and writes the vector to the destination SIMD&FP register.

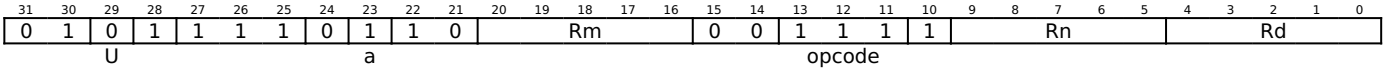
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

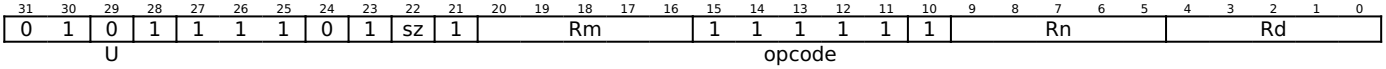
```
FRSQRTS <Hd>, <Hn>, <Hm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

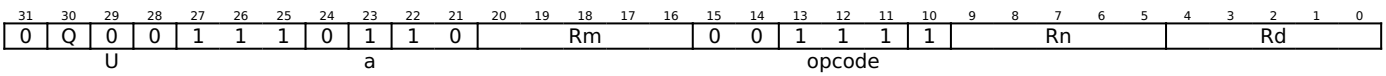
```
FRSQRTS <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

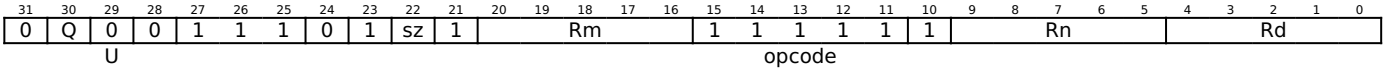
```
FRSQRTS <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FRSQRTS <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.

<Hm> Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
if elements == 1 then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

var element1 : bits(esize);
var element2 : bits(esize);
let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(n) else Zeros{128};

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPRSqrtStepFused{esize}(element1, element2, FPCR());
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FSCALE

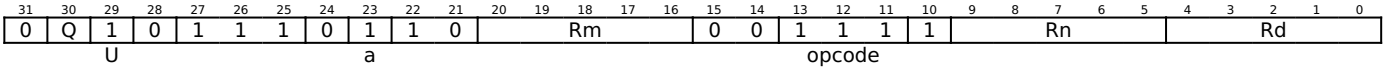
Floating-point adjust exponent by vector

This instruction multiplies the floating-point elements of the first source vector by 2.0 to the power of the signed integer values in the corresponding elements of the second source vector, and places the results in the corresponding elements of the destination vector.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_FP8)



Encoding

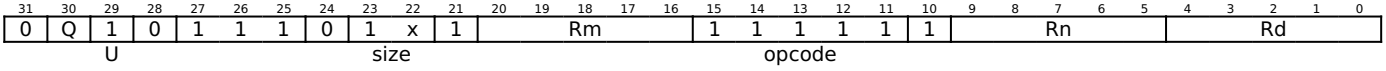
```
FSCALE <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP8) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_FP8)



Encoding

```
FSCALE <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP8) then EndOfDecode(Decode_UNDEF); end;
if Q == '0' && size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = if Q == '1' then 128 else 64;
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(size[0] :: Q)”:

size[0]	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

for e = 0 to elements-1 do
    let op1 : bits(esize) = operand1[e*:esize];
    let op2 : integer     = SInt(operand2[e*:esize]);
    result[e*:esize] = FPScale{esize}(op1, op2, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

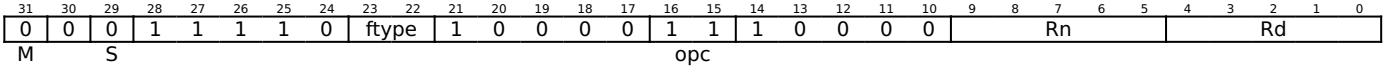
FSQRT (scalar)

Floating-point square root (scalar)

This instruction calculates the square root of the value in the SIMD&FP source register and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FSQRT <Hd>, <Hn>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FSQRT <Sd>, <Sn>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FSQRT <Dd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();

let operand : bits(esize) = V{}(n);
var result : bits(128) = if IsMerging(FPCR()) then V{128}(d) else Zeros{128};

result[0+:esize] = FPSqrt{esize}(operand, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FSQRT (vector)

Floating-point square root (vector)

This instruction calculates the square root for each vector element in the source SIMD&FP register, places the result in a vector, and writes the vector to the destination SIMD&FP register.

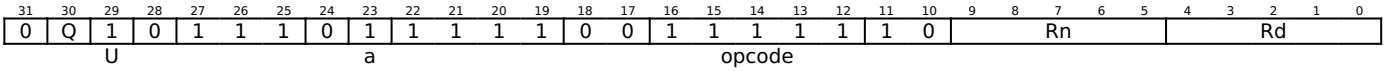
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

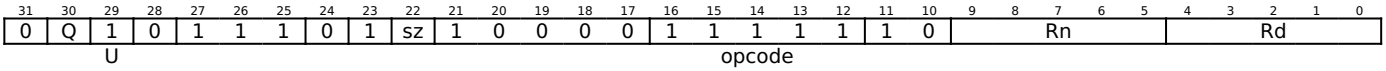
```
FSQRT <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FSQRT <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[e*:esize];
    result[e*:esize] = FPSqrt{esize}(element, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

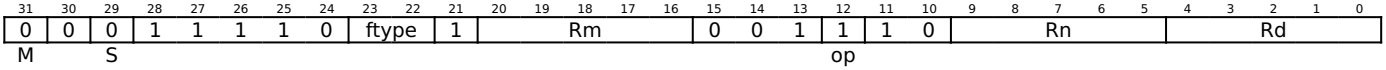
FSUB (scalar)

Floating-point subtract (scalar)

This instruction subtracts the floating-point value of the second source SIMD&FP register from the floating-point value of the first source SIMD&FP register, and writes the result to the destination SIMD&FP register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the Half-precision variant (FEAT_FP16)

Applies when `(ftype == 11)`

```
FSUB <Hd>, <Hn>, <Hm>
```

Encoding for the Single-precision variant (FEAT_FP)

Applies when `(ftype == 00)`

```
FSUB <Sd>, <Sn>, <Sm>
```

Encoding for the Double-precision variant (FEAT_FP)

Applies when `(ftype == 01)`

```
FSUB <Dd>, <Dn>, <Dm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(ftype XOR '10');
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Hn>	Is the 16-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Hm>	Is the 16-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Sm>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rm" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the first SIMD&FP source register, encoded in the "Rn" field.
<Dm>	Is the 64-bit name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPEnabled();
let operand1 : bits(esize) = V{}(n);
let operand2 : bits(esize) = V{}(m);

var result : bits(128) = if IsMerging(FPCR()) then V{128}(n) else Zeros{128};

result[0+:esize] = FPSub{esize}(operand1, operand2, FPCR());

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FSUB (vector)

Floating-point subtract (vector)

This instruction subtracts the elements in the vector in the second source SIMD&FP register, from the corresponding elements in the vector in the first source SIMD&FP register, places each result into elements of a vector, and writes the vector to the destination SIMD&FP register.

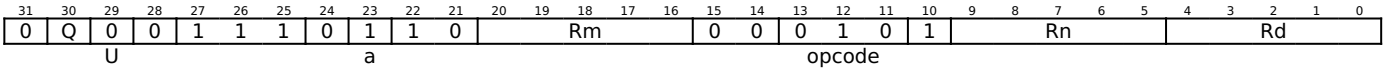
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Half-precision and Single-precision and double-precision

Half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

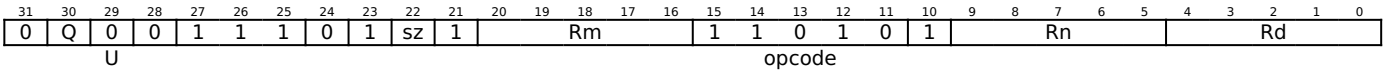
```
FSUB <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
FSUB <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);

var element1 : bits(esize);
var element2 : bits(esize);
var result : bits(datasize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = FPSub{esize}(element1, element2, FPCR());
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

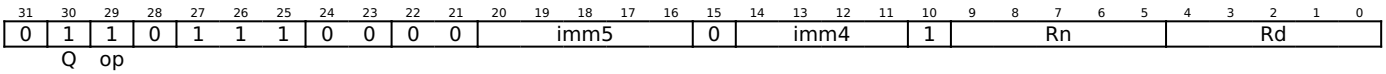
INS (element)

Insert vector element from another vector element

This instruction copies the vector element of the source SIMD&FP register to the specified vector element of the destination SIMD&FP register. This instruction can insert data into individual elements within a SIMD&FP register without clearing the remaining bits to zero.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
This instruction is used by the alias MOV (element).

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
INS <Vd>.<Ts>[<index1>], <Vn>.<Ts>[<index2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if imm5 == 'x0000' then EndOfDecode(Decode_UNDEF); end;
let size : integer{} = LowestSetBitNZ(imm5[3:0]);

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let dst_index : integer = UInt(imm5[4:size+1]);
let src_index : integer = UInt(imm4[3:size]);
let idxdsize : integer{} = 64 << UInt(imm4[3]);
// imm4[size-1:0] is IGNORED
let esize : integer{} = 8 << size;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ts> Is an element size specifier, encoded in “imm5”:

imm5	<Ts>
x0000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D

<index1> Is the destination element index encoded in “imm5”:

imm5	<index1>
x0000	RESERVED
xxxx1	UInt(imm5[4:1])
xxx10	UInt(imm5[4:2])
xx100	UInt(imm5[4:3])
x1000	UInt(imm5[4])

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<index2> Is the source element index encoded in “(imm5 :: imm4)”:

imm5	<index2>
x0000	RESERVED
xxxx1	UInt(imm4)
xxx10	UInt(imm4[3:1])
xx100	UInt(imm4[3:2])
x1000	UInt(imm4[3])

Unspecified bits in "imm4" are ignored but should be set to zero by an assembler.

Operation

```
AArch64.CheckFPAdvSIMDEnabled();
let operand : bits(idxdsize) = V{idxdsize}(n);
var result : bits(128) = V{128}(d);

result[dst_index*:esize] = operand[src_index*:esize];
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

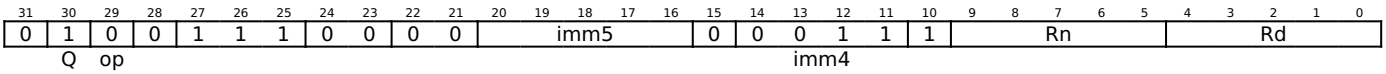
INS (general)

Insert vector element from general-purpose register

This instruction copies the contents of the source general-purpose register to the specified vector element in the destination SIMD&FP register.
This instruction can insert data into individual elements within a SIMD&FP register without clearing the remaining bits to zero.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
This instruction is used by the alias MOV (from general).

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
INS <Vd>.<Ts>[<index>], <R><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if imm5 == 'x0000' then EndOfDecode(Decode_UNDEF); end;
let size : integer{} = LowestSetBitNZ(imm5[3:0]);

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let index : integer = UInt(imm5[4:size+1]);
let esize : integer{} = 8 << size;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ts> Is an element size specifier, encoded in “imm5”:

imm5	<Ts>
x0000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D

<index> Is the element index encoded in “imm5”:

imm5	<index>
x0000	RESERVED
xxxx1	UInt(imm5[4:1])
xxx10	UInt(imm5[4:2])
xx100	UInt(imm5[4:3])
x1000	UInt(imm5[4])

<R> Is the width specifier for the general-purpose source register, encoded in “imm5”:

imm5	<R>
x0000	RESERVED
xxxx1	W
xxx10	W
xx100	W
x1000	X

<n> Is the number [0-30] of the general-purpose source register or ZR (31), encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let element : bits(esize) = X{esize}(n);
var result : bits(128) = V{128}(d);

result[index*:esize] = element;
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

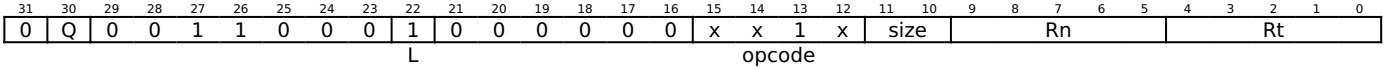
LD1 (multiple structures)

Load multiple single-element structures to one, two, three, or four registers

This instruction loads multiple single-element structures from memory and writes the result to one, two, three, or four SIMD&FP registers. Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding for the One register variant

Applies when `(opcode == 0111)`

```
LD1 { <Vt>.<T> }, [<Xn|SP>]
```

Encoding for the Two registers variant

Applies when `(opcode == 1010)`

```
LD1 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>]
```

Encoding for the Three registers variant

Applies when `(opcode == 0110)`

```
LD1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>]
```

Encoding for the Four registers variant

Applies when `(opcode == 0010)`

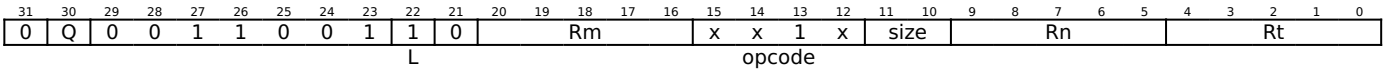
```
LD1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the One register, immediate offset variant

Applies when `(Rm == 11111 && opcode == 0111)`

```
LD1 { <Vt>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the One register, register offset variant

Applies when `(Rm != 11111 && opcode == 0111)`

```
LD1 { <Vt>.<T> }, [<Xn|SP>], <Xm>
```

Encoding for the Two registers, immediate offset variant

Applies when `(Rm == 11111 && opcode == 1010)`

```
LD1 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Two registers, register offset variant

Applies when `(Rm != 11111 && opcode == 1010)`

```
LD1 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>], <Xm>
```

Encoding for the Three registers, immediate offset variant

Applies when `(Rm == 11111 && opcode == 0110)`

```
LD1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Three registers, register offset variant

Applies when `(Rm != 11111 && opcode == 0110)`

```
LD1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>], <Xm>
```

Encoding for the Four registers, immediate offset variant

Applies when `(Rm == 11111 && opcode == 0010)`

```
LD1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Four registers, register offset variant

Applies when `(Rm != 11111 && opcode == 0010)`

```
LD1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	1D
11	1	2D

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Vt3> Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.

<Vt4> Is the name of the fourth SIMD&FP register to be transferred, encoded as "Rt" plus 3 modulo 32.

<imm> For the "One register, immediate offset" variant: is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#8
1	#16

For the "Two registers, immediate offset" variant: is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#16
1	#32

For the "Three registers, immediate offset" variant: is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#24
1	#48

For the "Four registers, immediate offset" variant: is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#32
1	#64

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(size);
let elements : integer = datasize DIV esize;

var rpt : integer;                // number of iterations
let selem : integer = 1;          // structure elements

case opcode of
    when '0010' => rpt = 4;    // LD/ST1 (4 registers)
    when '0110' => rpt = 3;    // LD/ST1 (3 registers)
    when '1010' => rpt = 2;    // LD/ST1 (2 registers)
    when '0111' => rpt = 1;    // LD/ST1 (1 register)
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(datasize);
var tt : integer;
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

for r = 0 to rpt - 1 do
    for e = 0 to elements - 1 do
        tt = (t + r) MOD 32;
        for s = 0 to selem - 1 do
            rval = V{datasize}(tt);
            eaddr = AddressIncrement(address, offs, accdesc);
            rval[e *: esize] = Mem{esize}(eaddr, accdesc);
            V{datasize}(tt) = rval;
            offs = offs + ebytes;
            tt = (tt + 1) MOD 32;
        end;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1 (single structure)

Load one single-element structure to one lane of one register

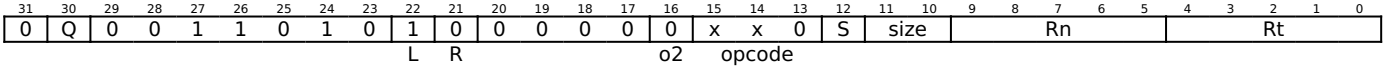
This instruction loads a single-element structure from memory and writes the result to the specified lane of the SIMD&FP register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding for the 8-bit variant

Applies when `(opcode == 000)`

```
LD1 { <Vt>.B }[<index>], [<Xn|SP>]
```

Encoding for the 16-bit variant

Applies when `(opcode == 010 && size == x0)`

```
LD1 { <Vt>.H }[<index>], [<Xn|SP>]
```

Encoding for the 32-bit variant

Applies when `(opcode == 100 && size == 00)`

```
LD1 { <Vt>.S }[<index>], [<Xn|SP>]
```

Encoding for the 64-bit variant

Applies when `(opcode == 100 && S == 0 && size == 01)`

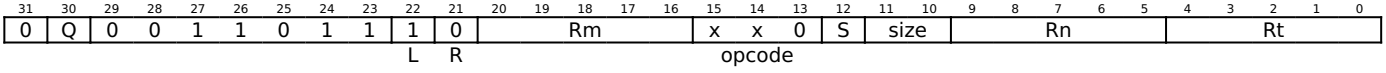
```
LD1 { <Vt>.D }[<index>], [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the 8-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 000)`

```
LD1 { <Vt>.B }[<index>], [<Xn|SP>], #1
```

Encoding for the 8-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 000)`

```
LD1 { <Vt>.B }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 64-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 100 && S == 0 && size == 01)`

```
LD1 { <Vt>.D }[<index>], [<Xn|SP>], #8
```

Encoding for the 64-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 100 && S == 0 && size == 01)`

```
LD1 { <Vt>.D }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 16-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 010 && size == x0)`

```
LD1 { <Vt>.H }[<index>], [<Xn|SP>], #2
```

Encoding for the 16-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 010 && size == x0)`

```
LD1 { <Vt>.H }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 32-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 100 && size == 00)`

```
LD1 { <Vt>.S }[<index>], [<Xn|SP>], #4
```

Encoding for the 32-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 100 && size == 00)`

```
LD1 { <Vt>.S }[<index>], [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt>	Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.
<index>	For the "8-bit", "8-bit, immediate offset", and "8-bit, register offset" variants: is the element index, encoded in "Q:S:size". For the "16-bit", "16-bit, immediate offset", and "16-bit, register offset" variants: is the element index, encoded in "Q:S:size<1>". For the "32-bit", "32-bit, immediate offset", and "32-bit, register offset" variants: is the element index, encoded in "Q:S". For the "64-bit", "64-bit, immediate offset", and "64-bit, register offset" variants: is the element index, encoded in "Q".
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        rval[index *: esize] = Mem{esize}(eaddr, accdesc);
        V{128}(t) = rval;
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

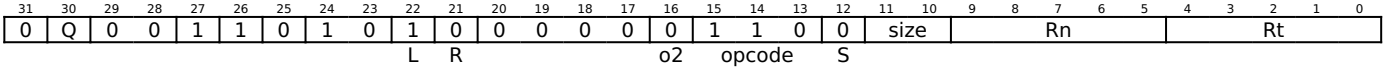
LD1R

Load one single-element structure and replicate to all lanes (of one register)

This instruction loads a single-element structure from memory and replicates the structure to all the lanes of the SIMD&FP register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding

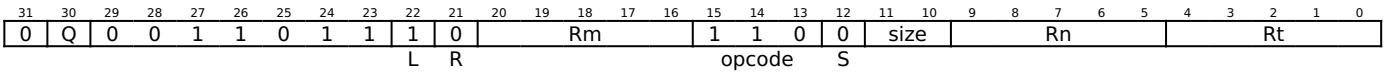
```
LD1R { <Vt>.<T> }, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the Immediate offset variant

Applies when `(Rm == 11111)`

```
LD1R { <Vt>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Register offset variant

Applies when `(Rm != 11111)`

```
LD1R { <Vt>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	1D
11	1	2D

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the post-index immediate offset, encoded in “size”:

size	<imm>
00	#1
01	#2
10	#4
11	#8

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        rval[index *: esize] = Mem{esize}(eaddr, accdesc);
        V{128}(t) = rval;
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

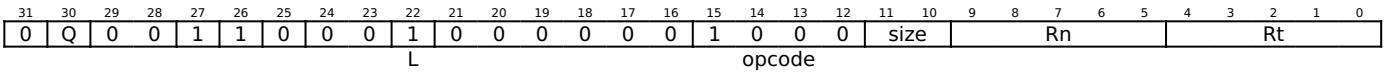
LD2 (multiple structures)

Load multiple 2-element structures to two registers

This instruction loads multiple 2-element structures from memory and writes the result to the two SIMD&FP registers, with de-interleaving. For an example of de-interleaving, see *LD3 (multiple structures)*.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding

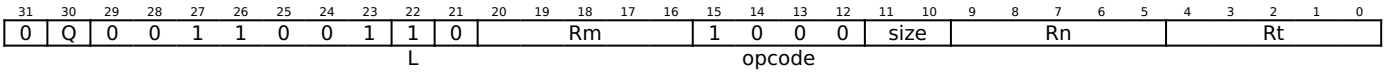
```
LD2 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the Immediate offset variant

Applies when `(Rm == 11111)`

```
LD2 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Register offset variant

Applies when `(Rm != 11111)`

```
LD2 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#16
1	#32

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(size);
let elements : integer = datasize DIV esize;

let rpt : integer = 1;                  // number of iterations
let selem : integer = 2;           // structure elements

// .1D format only permitted with LD1 & ST1
if size::Q == '110' && selem != 1 then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(datasize);
var tt : integer;
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

for r = 0 to rpt - 1 do
    for e = 0 to elements - 1 do
        tt = (t + r) MOD 32;
        for s = 0 to selem - 1 do
            rval = V{datasize}(tt);
            eaddr = AddressIncrement(address, offs, accdesc);
            rval[e *: esize] = Mem{esize}(eaddr, accdesc);
            V{datasize}(tt) = rval;
            offs = offs + ebytes;
            tt = (tt + 1) MOD 32;
        end;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2 (single structure)

Load single 2-element structure to one lane of two registers

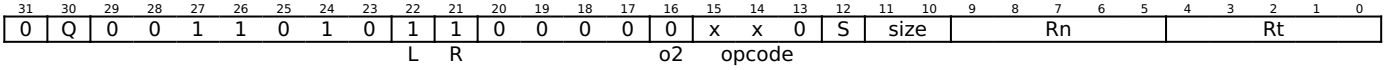
This instruction loads a 2-element structure from memory and writes the result to the corresponding elements of the two SIMD&FP registers without affecting the other bits of the registers.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding for the 8-bit variant

Applies when `(opcode == 000)`

```
LD2 { <Vt>.B, <Vt2>.B }[<index>], [<Xn|SP>]
```

Encoding for the 16-bit variant

Applies when `(opcode == 010 && size == x0)`

```
LD2 { <Vt>.H, <Vt2>.H }[<index>], [<Xn|SP>]
```

Encoding for the 32-bit variant

Applies when `(opcode == 100 && size == 00)`

```
LD2 { <Vt>.S, <Vt2>.S }[<index>], [<Xn|SP>]
```

Encoding for the 64-bit variant

Applies when `(opcode == 100 && S == 0 && size == 01)`

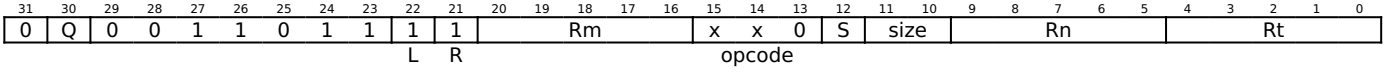
```
LD2 { <Vt>.D, <Vt2>.D }[<index>], [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the 8-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 000)`

```
LD2 { <Vt>.B, <Vt2>.B }[<index>], [<Xn|SP>], #2
```

Encoding for the 8-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 000)`

```
LD2 { <Vt>.B, <Vt2>.B }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 16-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 010 && size == x0)`

```
LD2 { <Vt>.H, <Vt2>.H }[<index>], [<Xn|SP>], #4
```

Encoding for the 16-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 010 && size == x0)`

```
LD2 { <Vt>.H, <Vt2>.H }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 32-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 100 && size == 00)`

```
LD2 { <Vt>.S, <Vt2>.S }[<index>], [<Xn|SP>], #8
```

Encoding for the 32-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 100 && size == 00)`

```
LD2 { <Vt>.S, <Vt2>.S }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 64-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 100 && S == 0 && size == 01)`

```
LD2 { <Vt>.D, <Vt2>.D }[<index>], [<Xn|SP>], #16
```

Encoding for the 64-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 100 && S == 0 && size == 01)`

```
LD2 { <Vt>.D, <Vt2>.D }[<index>], [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt>	Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.
<Vt2>	Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.
<index>	For the "8-bit", "8-bit, immediate offset", and "8-bit, register offset" variants: is the element index, encoded in "Q:S:size". For the "16-bit", "16-bit, immediate offset", and "16-bit, register offset" variants: is the element index, encoded in "Q:S:size<1>". For the "32-bit", "32-bit, immediate offset", and "32-bit, register offset" variants: is the element index, encoded in "Q:S". For the "64-bit", "64-bit, immediate offset", and "64-bit, register offset" variants: is the element index, encoded in "Q".
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        rval[index *: esize] = Mem{esize}(eaddr, accdesc);
        V{128}(t) = rval;
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

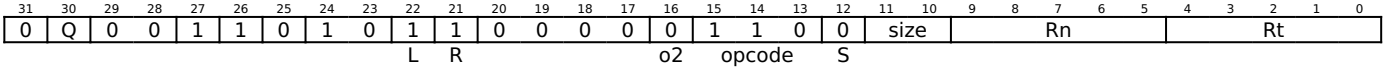
LD2R

Load single 2-element structure and replicate to all lanes of two registers

This instruction loads a 2-element structure from memory and replicates the structure to all the lanes of the two SIMD&FP registers.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding

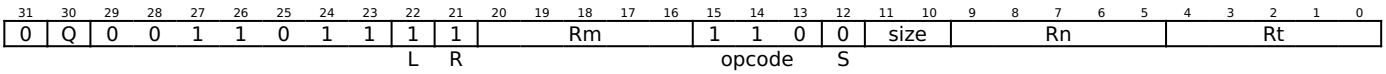
```
LD2R { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the Immediate offset variant

Applies when `(Rm == 11111)`

```
LD2R { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Register offset variant

Applies when `(Rm != 11111)`

```
LD2R { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	1D
11	1	2D

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the post-index immediate offset, encoded in “size”:

size	<imm>
00	#2
01	#4
10	#8
11	#16

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        rval[index *: esize] = Mem{esize}(eaddr, accdesc);
        V{128}(t) = rval;
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

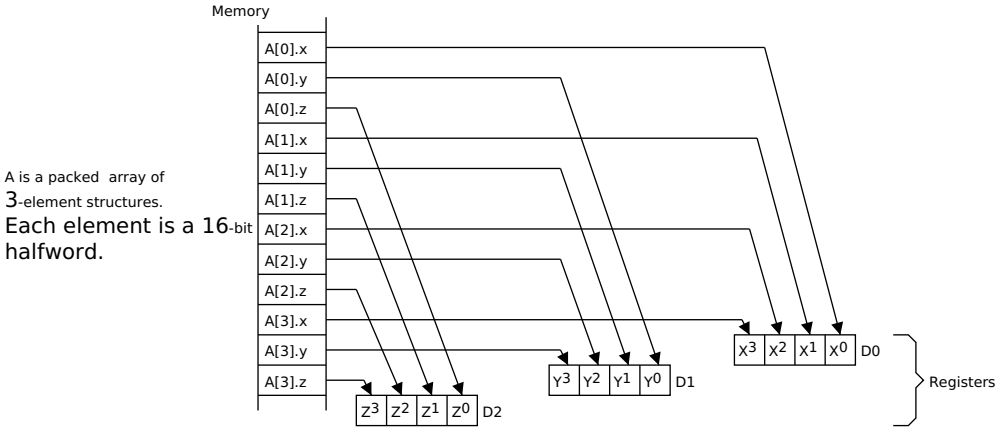
This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3 (multiple structures)

Load multiple 3-element structures to three registers

This instruction loads multiple 3-element structures from memory and writes the result to the three SIMD&FP registers, with de-interleaving. The following figure shows an example of the operation of de-interleaving of an LD3.16 (multiple 3-element structures) instruction.

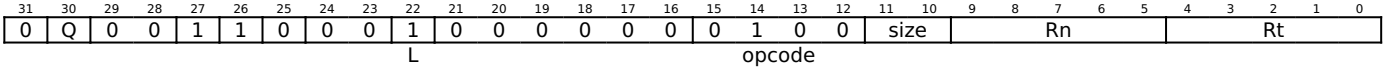


Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding

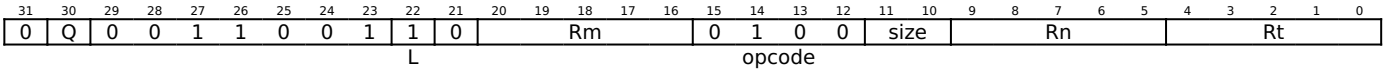
```
LD3 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the Immediate offset variant

Applies when `(Rm == 11111)`

```
LD3 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Register offset variant

Applies when `(Rm != 11111)`

```
LD3 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Vt3> Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#24
1	#48

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(size);
let elements : integer = datasize DIV esize;

let rpt : integer = 1;                // number of iterations
let selem : integer = 3;           // structure elements

// .1D format only permitted with LD1 & ST1
if size::Q == '110' && selem != 1 then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(datasize);
var tt : integer;
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

for r = 0 to rpt - 1 do
    for e = 0 to elements - 1 do
        tt = (t + r) MOD 32;
        for s = 0 to selem - 1 do
            rval = V{datasize}(tt);
            eaddr = AddressIncrement(address, offs, accdesc);
            rval[e *: esize] = Mem{esize}(eaddr, accdesc);
            V{datasize}(tt) = rval;
            offs = offs + ebytes;
            tt = (tt + 1) MOD 32;
        end;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3 (single structure)

Load single 3-element structure to one lane of three registers

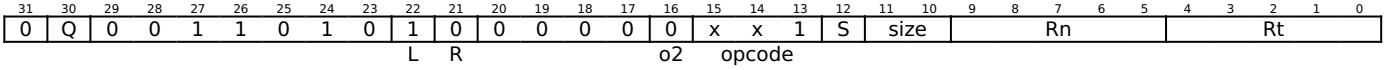
This instruction loads a 3-element structure from memory and writes the result to the corresponding elements of the three SIMD&FP registers without affecting the other bits of the registers.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding for the 8-bit variant

Applies when `(opcode == 001)`

```
LD3 { <Vt>.B, <Vt2>.B, <Vt3>.B }[<index>], [<Xn|SP>]
```

Encoding for the 16-bit variant

Applies when `(opcode == 011 && size == x0)`

```
LD3 { <Vt>.H, <Vt2>.H, <Vt3>.H }[<index>], [<Xn|SP>]
```

Encoding for the 32-bit variant

Applies when `(opcode == 101 && size == 00)`

```
LD3 { <Vt>.S, <Vt2>.S, <Vt3>.S }[<index>], [<Xn|SP>]
```

Encoding for the 64-bit variant

Applies when `(opcode == 101 && S == 0 && size == 01)`

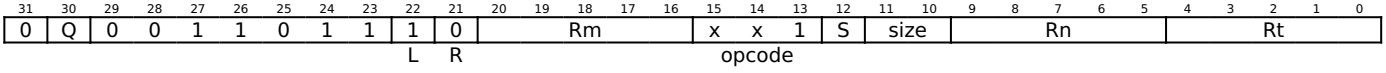
```
LD3 { <Vt>.D, <Vt2>.D, <Vt3>.D }[<index>], [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the 8-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 001)`

```
LD3 { <Vt>.B, <Vt2>.B, <Vt3>.B }[<index>], [<Xn|SP>], #3
```

Encoding for the 8-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 001)`

```
LD3 { <Vt>.B, <Vt2>.B, <Vt3>.B }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 16-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 011 && size == x0)`

```
LD3 { <Vt>.H, <Vt2>.H, <Vt3>.H }[<index>], [<Xn|SP>], #6
```

Encoding for the 16-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 011 && size == x0)`

```
LD3 { <Vt>.H, <Vt2>.H, <Vt3>.H }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 32-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 101 && size == 00)`

```
LD3 { <Vt>.S, <Vt2>.S, <Vt3>.S }[<index>], [<Xn|SP>], #12
```

Encoding for the 32-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 101 && size == 00)`

```
LD3 { <Vt>.S, <Vt2>.S, <Vt3>.S }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 64-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 101 && S == 0 && size == 01)`

```
LD3 { <Vt>.D, <Vt2>.D, <Vt3>.D }[<index>], [<Xn|SP>], #24
```

Encoding for the 64-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 101 && S == 0 && size == 01)`

```
LD3 { <Vt>.D, <Vt2>.D, <Vt3>.D }[<index>], [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt>	Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.
<Vt2>	Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.
<Vt3>	Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.
<index>	For the "8-bit", "8-bit, immediate offset", and "8-bit, register offset" variants: is the element index, encoded in "Q:S:size". For the "16-bit", "16-bit, immediate offset", and "16-bit, register offset" variants: is the element index, encoded in "Q:S:size<1>". For the "32-bit", "32-bit, immediate offset", and "32-bit, register offset" variants: is the element index, encoded in "Q:S". For the "64-bit", "64-bit, immediate offset", and "64-bit, register offset" variants: is the element index, encoded in "Q".
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        rval[index *: esize] = Mem{esize}(eaddr, accdesc);
        V{128}(t) = rval;
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

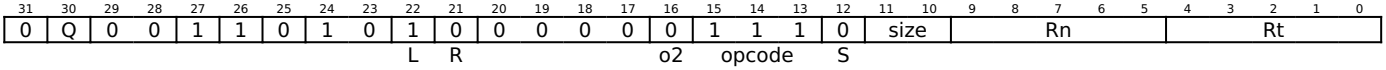
LD3R

Load single 3-element structure and replicate to all lanes of three registers

This instruction loads a 3-element structure from memory and replicates the structure to all the lanes of the three SIMD&FP registers.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding

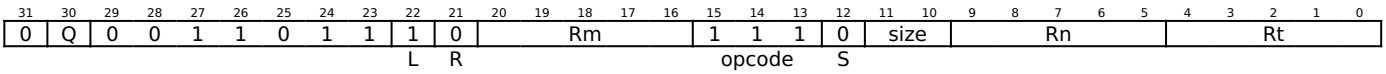
```
LD3R { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the Immediate offset variant

Applies when `(Rm == 11111)`

```
LD3R { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Register offset variant

Applies when `(Rm != 11111)`

```
LD3R { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	1D
11	1	2D

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Vt3> Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the post-index immediate offset, encoded in “size”:

size	<imm>
00	#3
01	#6
10	#12
11	#24

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        rval[index *: esize] = Mem{esize}(eaddr, accdesc);
        V{128}(t) = rval;
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

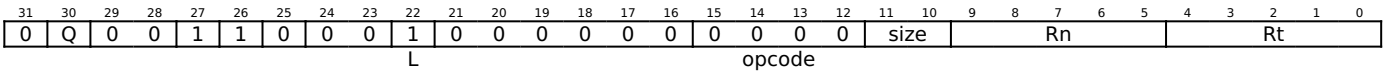
LD4 (multiple structures)

Load multiple 4-element structures to four registers

This instruction loads multiple 4-element structures from memory and writes the result to the four SIMD&FP registers, with de-interleaving. For an example of de-interleaving, see *LD3 (multiple structures)*.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding

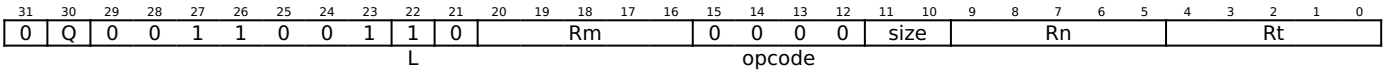
```
LD4 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the Immediate offset variant

Applies when `(Rm == 11111)`

```
LD4 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Register offset variant

Applies when `(Rm != 11111)`

```
LD4 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Vt3> Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.

<Vt4> Is the name of the fourth SIMD&FP register to be transferred, encoded as "Rt" plus 3 modulo 32.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#32
1	#64

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(size);
let elements : integer = datasize DIV esize;

let rpt : integer = 1;                 // number of iterations
let selem : integer = 4;           // structure elements

// .1D format only permitted with LD1 & ST1
if size::Q == '110' && selem != 1 then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(datasize);
var tt : integer;
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

for r = 0 to rpt - 1 do
    for e = 0 to elements - 1 do
        tt = (t + r) MOD 32;
        for s = 0 to selem - 1 do
            rval = V{datasize}(tt);
            eaddr = AddressIncrement(address, offs, accdesc);
            rval[e *: esize] = Mem{esize}(eaddr, accdesc);
            V{datasize}(tt) = rval;
            offs = offs + ebytes;
            tt = (tt + 1) MOD 32;
        end;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4 (single structure)

Load single 4-element structure to one lane of four registers

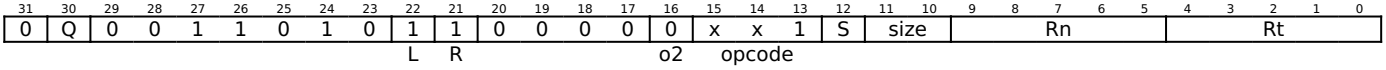
This instruction loads a 4-element structure from memory and writes the result to the corresponding elements of the four SIMD&FP registers without affecting the other bits of the registers.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding for the 8-bit variant

Applies when `(opcode == 001)`

```
LD4 { <Vt>.B, <Vt2>.B, <Vt3>.B, <Vt4>.B }[<index>], [<Xn|SP>]
```

Encoding for the 16-bit variant

Applies when `(opcode == 011 && size == x0)`

```
LD4 { <Vt>.H, <Vt2>.H, <Vt3>.H, <Vt4>.H }[<index>], [<Xn|SP>]
```

Encoding for the 32-bit variant

Applies when `(opcode == 101 && size == 00)`

```
LD4 { <Vt>.S, <Vt2>.S, <Vt3>.S, <Vt4>.S }[<index>], [<Xn|SP>]
```

Encoding for the 64-bit variant

Applies when `(opcode == 101 && S == 0 && size == 01)`

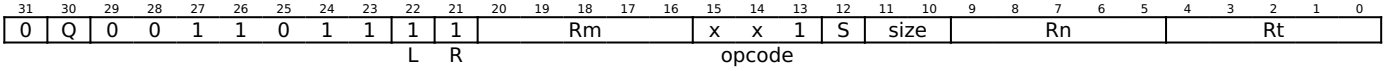
```
LD4 { <Vt>.D, <Vt2>.D, <Vt3>.D, <Vt4>.D }[<index>], [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the 8-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 001)`

```
LD4 { <Vt>.B, <Vt2>.B, <Vt3>.B, <Vt4>.B }[<index>], [<Xn|SP>], #4
```

Encoding for the 8-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 001)`

```
LD4 { <Vt>.B, <Vt2>.B, <Vt3>.B, <Vt4>.B }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 16-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 011 && size == x0)`

```
LD4 { <Vt>.H, <Vt2>.H, <Vt3>.H, <Vt4>.H }[<index>], [<Xn|SP>], #8
```

Encoding for the 16-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 011 && size == x0)`

```
LD4 { <Vt>.H, <Vt2>.H, <Vt3>.H, <Vt4>.H }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 32-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 101 && size == 00)`

```
LD4 { <Vt>.S, <Vt2>.S, <Vt3>.S, <Vt4>.S }[<index>], [<Xn|SP>], #16
```

Encoding for the 32-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 101 && size == 00)`

```
LD4 { <Vt>.S, <Vt2>.S, <Vt3>.S, <Vt4>.S }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 64-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 101 && S == 0 && size == 01)`

```
LD4 { <Vt>.D, <Vt2>.D, <Vt3>.D, <Vt4>.D }[<index>], [<Xn|SP>], #32
```

Encoding for the 64-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 101 && S == 0 && size == 01)`

```
LD4 { <Vt>.D, <Vt2>.D, <Vt3>.D, <Vt4>.D }[<index>], [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt>	Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.
<Vt2>	Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.
<Vt3>	Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.
<Vt4>	Is the name of the fourth SIMD&FP register to be transferred, encoded as "Rt" plus 3 modulo 32.
<index>	For the "8-bit", "8-bit, immediate offset", and "8-bit, register offset" variants: is the element index, encoded in "Q:S:size". For the "16-bit", "16-bit, immediate offset", and "16-bit, register offset" variants: is the element index, encoded in "Q:S:size<1>". For the "32-bit", "32-bit, immediate offset", and "32-bit, register offset" variants: is the element index, encoded in "Q:S". For the "64-bit", "64-bit, immediate offset", and "64-bit, register offset" variants: is the element index, encoded in "Q".
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        rval[index *: esize] = Mem{esize}(eaddr, accdesc);
        V{128}(t) = rval;
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

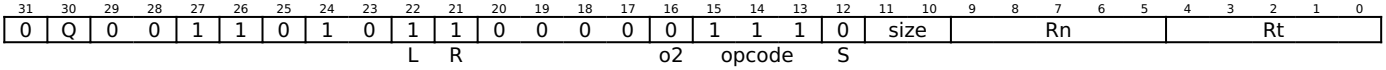
LD4R

Load single 4-element structure and replicate to all lanes of four registers

This instruction loads a 4-element structure from memory and replicates the structure to all the lanes of the four SIMD&FP registers.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding

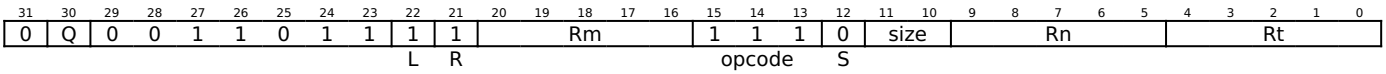
```
LD4R { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the Immediate offset variant

Applies when `(Rm == 11111)`

```
LD4R { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Register offset variant

Applies when `(Rm != 11111)`

```
LD4R { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	1D
11	1	2D

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Vt3> Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.

<Vt4> Is the name of the fourth SIMD&FP register to be transferred, encoded as "Rt" plus 3 modulo 32.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the post-index immediate offset, encoded in “size”:

size	<imm>
00	#4
01	#8
10	#16
11	#32

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        rval[index *: esize] = Mem{esize}(eaddr, accdesc);
        V{128}(t) = rval;
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAP1 (SIMD&FP)

Load-acquire RCpc one single-element structure to one lane of one register

This instruction loads a single-element structure from memory and writes the result to the specified lane of the SIMD&FP register without affecting the other bits of the register.

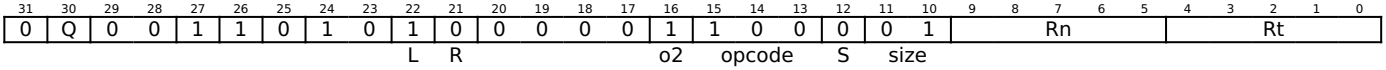
The instruction has memory ordering semantics, as described in *Load-Acquire, Load-AcquirePC, and Store-Release*.

For information about addressing modes, see *Load/Store addressing modes*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

64-bit

(FEAT_AdvSIMD && FEAT_LRCPC3)



Encoding

```
LDAP1 { <Vt>.D }[<index>], [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_LRCPC3) then
    EndOfDecode(Decode_UNDEF);
end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt>	Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.
<index>	Is the element index, encoded in "Q".
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let accdesc : AccessDescriptor = CreateAccDescASIMDAcqRel(MemOp_LOAD, tagchecked);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        rval[index *: esize] = Mem{esize}(eaddr, accdesc);
        V{128}(t) = rval;
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPUR (SIMD&FP)

Load-acquire RCpc SIMD&FP register (unscaled offset)

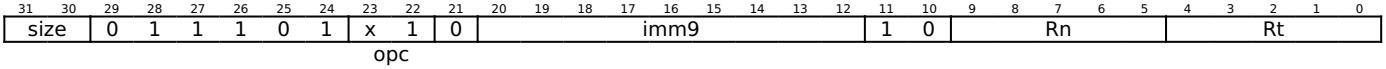
This instruction loads a SIMD&FP register from memory. The address that is used for the load is calculated from a base register value and an optional immediate offset.

The instruction has memory ordering semantics as described in *Load-Acquire, Load-AcquirePC, and Store-Release*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Unscaled offset

(FEAT_FP && FEAT_LRCPC3)



Encoding for the 8-bit variant

Applies when `(size == 00 && opc == 01)`

```
LDAPUR <Bt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 01)`

```
LDAPUR <Ht>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 01)`

```
LDAPUR <St>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 01)`

```
LDAPUR <Dt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 11)`

```
LDAPUR <Qt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) || !IsFeatureImplemented(FEAT_LRCPC3) then
    EndOfDecode(Decode_UNDEF);
end;
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Bt>	Is the 8-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<St>	Is the 32-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt>	Is the 128-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << scale;
let tagchecked : boolean = n != 31;
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
var address : bits(64);
let accdesc : AccessDescriptor = CreateAccDescASIMDAcqRel(MemOp_LOAD, tagchecked);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

V{datasize}(t) = Mem{datasize}(address, accdesc);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL

Atomic BFloat16 add

This instruction atomically loads a 16-bit value from memory, adds it to the BFloat16 value held in a register, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- `LDBFADDA` and `LDBFADDAL` load from memory with acquire semantics.
- `LDBFADDL` and `LDBFADDAL` store to memory with release semantics.
- `LDBFADD` has neither acquire nor release semantics.

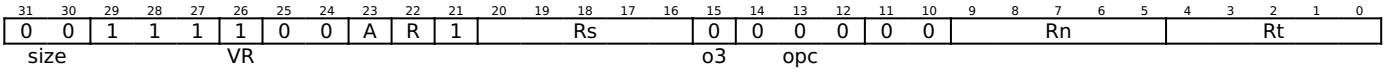
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDBFADD <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDBFADDA <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDBFADDAL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDBFADDL <Hs>, <Ht>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 16;
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_BFADD, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);

V{datasize}(t) = data;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL

Atomic BFloat16 maximum

This instruction atomically loads a 16-bit value from memory, calculates the BFloat16 maximum with the value held in a register, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- `LDBFMAXA` and `LDBFMAXAL` load from memory with acquire semantics.
- `LDBFMAXL` and `LDBFMAXAL` store to memory with release semantics.
- `LDBFMAX` has neither acquire nor release semantics.

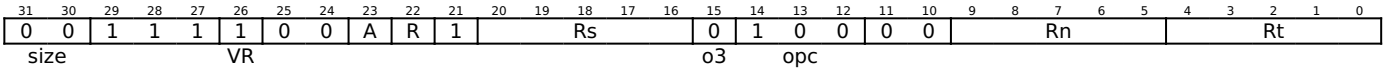
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDBFMAX <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDBFMAXA <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDBFMAXAL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDBFMAXL <Hs>, <Ht>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 16;
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_BFMAX, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);

V{datasize}(t) = data;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML

Atomic BFloat16 maximum number

This instruction atomically loads a 16-bit value from memory, calculates the BFloat16 maximum number with the value held in a register, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- `LDBFMAXNMA` and `LDBFMAXNMAL` load from memory with acquire semantics.
- `LDBFMAXNML` and `LDBFMAXNMAL` store to memory with release semantics.
- `LDBFMAXNM` has neither acquire nor release semantics.

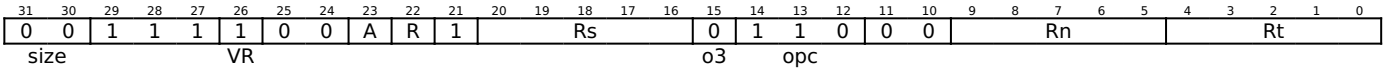
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDBFMAXNM <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDBFMAXNMA <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDBFMAXNMAL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDBFMAXNML <Hs>, <Ht>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 16;
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_BFMAXNM, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);

V{datasize}(t) = data;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL

Atomic BFloat16 minimum

This instruction atomically loads a 16-bit value from memory, calculates the BFloat16 minimum with the value held in a register, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- `LDBFMINA` and `LDBFMINAL` load from memory with acquire semantics.
- `LDBFMINL` and `LDBFMINAL` store to memory with release semantics.
- `LDBFMIN` has neither acquire nor release semantics.

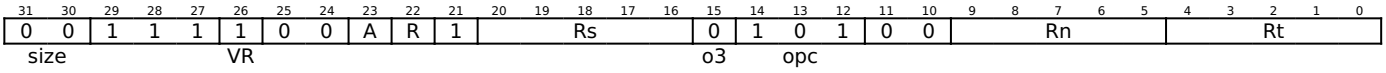
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDBFMIN <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDBFMINA <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDBFMINAL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDBFMINL <Hs>, <Ht>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 16;
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_BFMIN, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);

V{datasize}(t) = data;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML

Atomic BFloat16 minimum number

This instruction atomically loads a 16-bit value from memory, calculates the BFloat16 minimum number with the value held in a register, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- `LDBFMINNMA` and `LDBFMINNMAL` load from memory with acquire semantics.
- `LDBFMINNML` and `LDBFMINNMAL` store to memory with release semantics.
- `LDBFMINNM` has neither acquire nor release semantics.

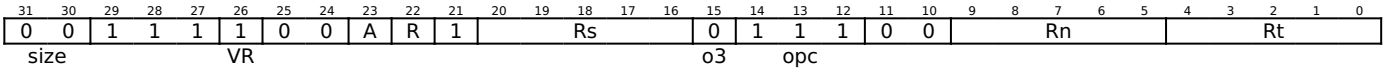
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the No memory ordering variant

Applies when `(A == 0 && R == 0)`

```
LDBFMINNM <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Acquire variant

Applies when `(A == 1 && R == 0)`

```
LDBFMINNMA <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Acquire-release variant

Applies when `(A == 1 && R == 1)`

```
LDBFMINNMAL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(A == 0 && R == 1)`

```
LDBFMINNML <Hs>, <Ht>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 16;
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_BFMINNM, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);

V{datasize}(t) = data;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFADD, LDFADDA, LDFADDAL, LDFADDL

Atomic floating-point add

This instruction atomically loads a 16-bit, 32-bit, or 64-bit value from memory, performs a floating-point add with the value held in a register, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- `LDFADDA` and `LDFADDAL` load from memory with acquire semantics.
- `LDFADDL` and `LDFADDAL` store to memory with release semantics.
- `LDFADD` has neither acquire nor release semantics.

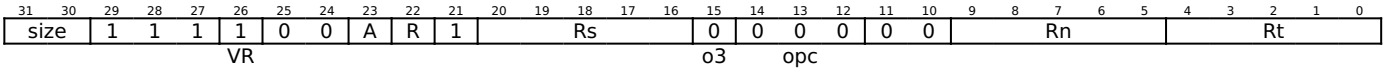
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the Half-precision no memory ordering variant

Applies when `(size == 01 && A == 0 && R == 0)`

```
LDFADD <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision acquire variant

Applies when `(size == 01 && A == 1 && R == 0)`

```
LDFADDA <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision acquire-release variant

Applies when `(size == 01 && A == 1 && R == 1)`

```
LDFADDAL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision release variant

Applies when `(size == 01 && A == 0 && R == 1)`

```
LDFADDL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Single-precision no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDFADD <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDFADDA <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDFADDAL <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDFADDL <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Double-precision no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDFADD <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDFADDA <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDFADDAL <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDFADDL <Ds>, <Dt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Ss>	Is the 32-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<St>	Is the 32-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Ds>	Is the 64-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_FPADD, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);

V{datasize}(t) = data;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL

Atomic floating-point maximum

This instruction atomically loads a 16-bit, 32-bit, or 64-bit value from memory, calculates the floating-point maximum with the value held in a register, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- `LDFMAXA` and `LDFMAXAL` load from memory with acquire semantics.
- `LDFMAXL` and `LDFMAXAL` store to memory with release semantics.
- `LDFMAX` has neither acquire nor release semantics.

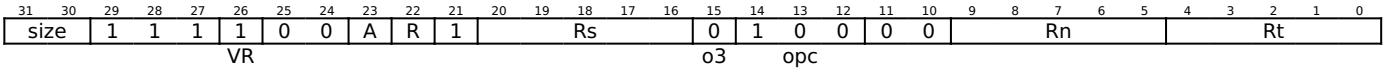
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the Half-precision no memory ordering variant

Applies when `(size == 01 && A == 0 && R == 0)`

```
LDFMAX <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision acquire variant

Applies when `(size == 01 && A == 1 && R == 0)`

```
LDFMAXA <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision acquire-release variant

Applies when `(size == 01 && A == 1 && R == 1)`

```
LDFMAXAL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision release variant

Applies when `(size == 01 && A == 0 && R == 1)`

```
LDFMAXL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Single-precision no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDFMAX <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDFMAXA <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDFMAXAL <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDFMAXL <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Double-precision no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDFMAX <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDFMAXA <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDFMAXAL <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDFMAXL <Ds>, <Dt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Ss>	Is the 32-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<St>	Is the 32-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Ds>	Is the 64-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_FPMAX, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);

V{datasize}(t) = data;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML

Atomic floating-point maximum number

This instruction atomically loads a 16-bit, 32-bit, or 64-bit value from memory, calculates the floating-point maximum number with the value held in a register, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- `LDFMAXNMA` and `LDFMAXNMAL` load from memory with acquire semantics.
- `LDFMAXNML` and `LDFMAXNMAL` store to memory with release semantics.
- `LDFMAXNM` has neither acquire nor release semantics.

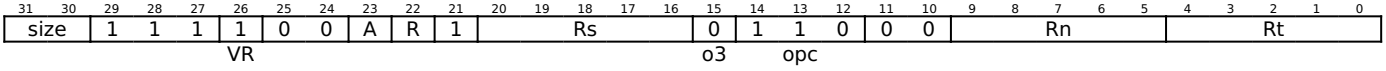
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the Half-precision no memory ordering variant

Applies when `(size == 01 && A == 0 && R == 0)`

```
LDFMAXNM <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision acquire variant

Applies when `(size == 01 && A == 1 && R == 0)`

```
LDFMAXNMA <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision acquire-release variant

Applies when `(size == 01 && A == 1 && R == 1)`

```
LDFMAXNMAL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision release variant

Applies when `(size == 01 && A == 0 && R == 1)`

```
LDFMAXNML <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Single-precision no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDFMAXNM <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDFMAXNMA <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDFMAXNMAL <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDFMAXNML <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Double-precision no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDFMAXNM <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDFMAXNMA <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDFMAXNMAL <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDFMAXNML <Ds>, <Dt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Ss>	Is the 32-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<St>	Is the 32-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Ds>	Is the 64-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_FPMAXNM, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);

V{datasize}(t) = data;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFMIN, LDFMINA, LDFMINAL, LDFMINL

Atomic floating-point minimum

This instruction atomically loads a 16-bit, 32-bit, or 64-bit value from memory, calculates the floating-point minimum with the value held in a register, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- `LDFMINA` and `LDFMINAL` load from memory with acquire semantics.
- `LDFMINL` and `LDFMINAL` store to memory with release semantics.
- `LDFMIN` has neither acquire nor release semantics.

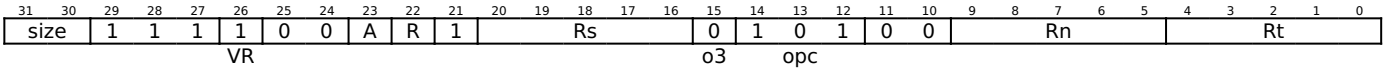
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the Half-precision no memory ordering variant

Applies when `(size == 01 && A == 0 && R == 0)`

```
LDFMIN <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision acquire variant

Applies when `(size == 01 && A == 1 && R == 0)`

```
LDFMINA <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision acquire-release variant

Applies when `(size == 01 && A == 1 && R == 1)`

```
LDFMINAL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision release variant

Applies when `(size == 01 && A == 0 && R == 1)`

```
LDFMINL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Single-precision no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDFMIN <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDFMINA <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDFMINAL <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDFMINL <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Double-precision no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDFMIN <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDFMINA <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDFMINAL <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDFMINL <Ds>, <Dt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Ss>	Is the 32-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<St>	Is the 32-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Ds>	Is the 64-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_FPMIN, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);

V{datasize}(t) = data;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML

Atomic floating-point minimum number

This instruction atomically loads a 16-bit, 32-bit, or 64-bit value from memory, calculates the floating-point minimum number with the value held in a register, and stores the result back to memory. The value initially loaded from memory is returned in the destination register.

- `LDFMINNMA` and `LDFMINNMAL` load from memory with acquire semantics.
- `LDFMINNML` and `LDFMINNMAL` store to memory with release semantics.
- `LDFMINNM` has neither acquire nor release semantics.

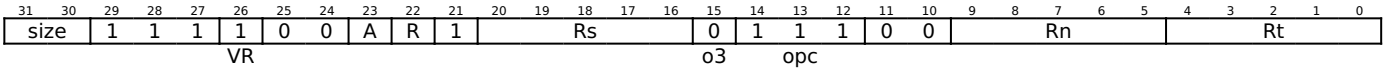
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the Half-precision no memory ordering variant

Applies when `(size == 01 && A == 0 && R == 0)`

```
LDFMINNM <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision acquire variant

Applies when `(size == 01 && A == 1 && R == 0)`

```
LDFMINNMA <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision acquire-release variant

Applies when `(size == 01 && A == 1 && R == 1)`

```
LDFMINNMAL <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Half-precision release variant

Applies when `(size == 01 && A == 0 && R == 1)`

```
LDFMINNML <Hs>, <Ht>, [<Xn|SP>]
```

Encoding for the Single-precision no memory ordering variant

Applies when `(size == 10 && A == 0 && R == 0)`

```
LDFMINNM <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision acquire variant

Applies when `(size == 10 && A == 1 && R == 0)`

```
LDFMINNMA <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision acquire-release variant

Applies when `(size == 10 && A == 1 && R == 1)`

```
LDFMINNMAL <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Single-precision release variant

Applies when `(size == 10 && A == 0 && R == 1)`

```
LDFMINNML <Ss>, <St>, [<Xn|SP>]
```

Encoding for the Double-precision no memory ordering variant

Applies when `(size == 11 && A == 0 && R == 0)`

```
LDFMINNM <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision acquire variant

Applies when `(size == 11 && A == 1 && R == 0)`

```
LDFMINNMA <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision acquire-release variant

Applies when `(size == 11 && A == 1 && R == 1)`

```
LDFMINNMAL <Ds>, <Dt>, [<Xn|SP>]
```

Encoding for the Double-precision release variant

Applies when `(size == 11 && A == 0 && R == 1)`

```
LDFMINNML <Ds>, <Dt>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = A == '1';
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Ss>	Is the 32-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<St>	Is the 32-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Ds>	Is the 64-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_FPMINNM, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);

V{datasize}(t) = data;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNP (SIMD&FP)

Load pair of SIMD&FP registers, with non-temporal hint

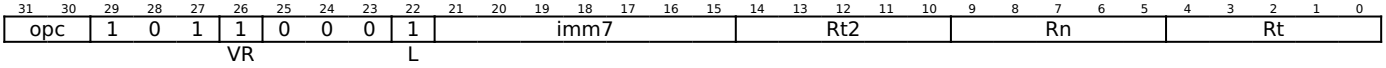
This instruction loads a pair of SIMD&FP registers from memory, issuing a hint to the memory system that the access is non-temporal. The address that is used for the load is calculated from a base register value and an optional immediate offset.

For information about non-temporal pair instructions, see *Load/Store SIMD and Floating-point non-temporal pair*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Signed offset

(FEAT_FP)



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
LDNP <St1>, <St2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 01)`

```
LDNP <Dt1>, <Dt2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 128-bit variant

Applies when `(opc == 10)`

```
LDNP <Qt1>, <Qt2>, [<Xn|SP>{, #<imm>}]
```

Decode for all variants of this encoding

```
// Empty.
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDNP (SIMD&FP)*.

Assembler Symbols

<St1>	Is the 32-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<St2>	Is the 32-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "32-bit" variant: is the optional signed immediate byte offset, a multiple of 4 in the range -256 to 252, defaulting to 0 and encoded in the "imm7" field as <imm>/4. For the "64-bit" variant: is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8. For the "128-bit" variant: is the optional signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, defaulting to 0 and encoded in the "imm7" field as <imm>/16.
<Dt1>	Is the 64-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Dt2>	Is the 64-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Qt1>	Is the 128-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt2>	Is the 128-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.

Shared Decode

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = TRUE;
let scale : integer{} = 2 + (UInt(opc) as integer{0..2});
let datasize : integer{} = 8 << scale;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // Result is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal, tagchecked,
                                                    privileged, ispair);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if rt_unknown then
    V{datasize}(t) = ARBITRARY : bits(datasize);
elsif BigEndian(accdesc.acctype) then
    V{datasize}(t2) = data[(datasize-1):0];
    V{datasize}(t)  = data[(2*datasize-1):datasize];
else
    V{datasize}(t)  = data[(datasize-1):0];
    V{datasize}(t2) = data[(2*datasize-1):datasize];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDP (SIMD&FP)

Load pair of SIMD&FP registers

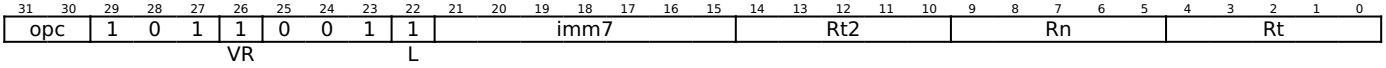
This instruction loads a pair of SIMD&FP registers from memory. The address that is used for the load is calculated from a base register value and an optional immediate offset.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_FP)



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
LDP <St1>, <St2>, [<Xn|SP>], #<imm>
```

Encoding for the 64-bit variant

Applies when `(opc == 01)`

```
LDP <Dt1>, <Dt2>, [<Xn|SP>], #<imm>
```

Encoding for the 128-bit variant

Applies when `(opc == 10)`

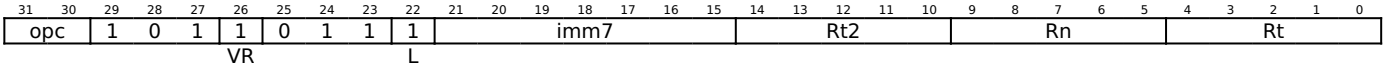
```
LDP <Qt1>, <Qt2>, [<Xn|SP>], #<imm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;

let wback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_FP)



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
LDP <St1>, <St2>, [<Xn|SP>, #<imm>]!
```

Encoding for the 64-bit variant

Applies when `(opc == 01)`

```
LDP <Dt1>, <Dt2>, [<Xn|SP>, #<imm>]!
```

Encoding for the 128-bit variant

Applies when `(opc == 10)`

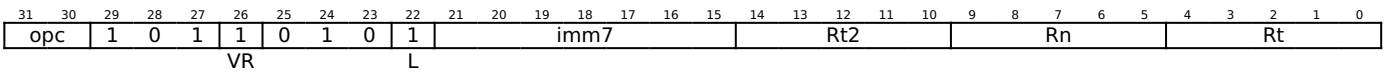
```
LDP <Qt1>, <Qt2>, [<Xn|SP>, #<imm>]!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;

let wback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_FP)



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
LDP <St1>, <St2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 01)`

```
LDP <Dt1>, <Dt2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 128-bit variant

Applies when `(opc == 10)`

```
LDP <Qt1>, <Qt2>, [<Xn|SP>{, #<imm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;

let wback : boolean = FALSE;
let postindex : boolean = FALSE;
```

For information about the CONSTRAINED UNPREDICTABLE behavior of this instruction, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDP (SIMD&FP)*.

Assembler Symbols

<St1>	Is the 32-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<St2>	Is the 32-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Post-index 32-bit" and "Pre-index 32-bit" variants: is the signed immediate byte offset, a multiple of 4 in the range -256 to 252, encoded in the "imm7" field as <imm>/4. For the "Post-index 64-bit" and "Pre-index 64-bit" variants: is the signed immediate byte offset, a multiple of 8 in the range -512 to 504, encoded in the "imm7" field as <imm>/8. For the "Post-index 128-bit" and "Pre-index 128-bit" variants: is the signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, encoded in the "imm7" field as <imm>/16. For the "Signed offset 32-bit" variant: is the optional signed immediate byte offset, a multiple of 4 in the range -256 to 252, defaulting to 0 and encoded in the "imm7" field as <imm>/4. For the "Signed offset 64-bit" variant: is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8. For the "Signed offset 128-bit" variant: is the optional signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, defaulting to 0 and encoded in the "imm7" field as <imm>/16.
<Dt1>	Is the 64-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Dt2>	Is the 64-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Qt1>	Is the 128-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt2>	Is the 128-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let scale : integer{} = 2 + (UInt(opc) as integer{0..2});
let datasize : integer{} = 8 << scale;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = wback || n != 31;

var rt_unknown : boolean = FALSE;
if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;    // Result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged, ispair);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if rt_unknown then
    V{datasize}(t) = ARBITRARY : bits(datasize);
elsif BigEndian(accdesc.acctype) then
    V{datasize}(t2) = data[(datasize-1):0];
    V{datasize}(t)  = data[(2*datasize-1):datasize];
else
    V{datasize}(t)  = data[(datasize-1):0];
    V{datasize}(t2) = data[(2*datasize-1):datasize];
end;

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDR (immediate, SIMD&FP)

Load SIMD&FP register (immediate offset)

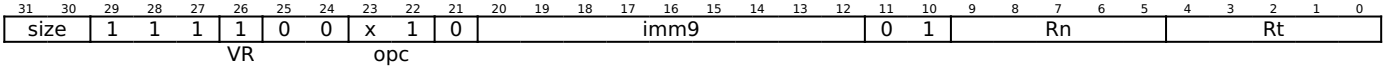
This instruction loads an element from memory, and writes the result as a scalar to the SIMD&FP register. The address that is used for the load is calculated from a base register value, a signed immediate offset, and an optional offset that is a multiple of the element size.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index

(FEAT_FP)



Encoding for the 8-bit variant

Applies when `(size == 00 && opc == 01)`

```
LDR <Bt>, [<Xn|SP>], #<simm>
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 01)`

```
LDR <Ht>, [<Xn|SP>], #<simm>
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 01)`

```
LDR <St>, [<Xn|SP>], #<simm>
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 01)`

```
LDR <Dt>, [<Xn|SP>], #<simm>
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 11)`

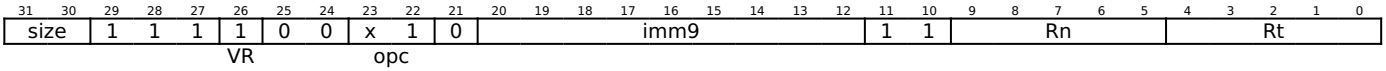
```
LDR <Qt>, [<Xn|SP>], #<simm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let wback : boolean = TRUE;
let postindex : boolean = TRUE;
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index

(FEAT_FP)



Encoding for the 8-bit variant

Applies when `(size == 00 && opc == 01)`

```
LDR <Bt>, [<Xn|SP>, #<simm>]!
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 01)`

```
LDR <Ht>, [<Xn|SP>, #<simm>]!
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 01)`

```
LDR <St>, [<Xn|SP>, #<simm>]!
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 01)`

```
LDR <Dt>, [<Xn|SP>, #<simm>]!
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 11)`

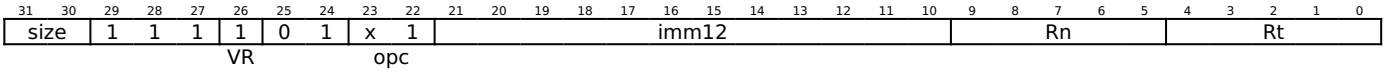
```
LDR <Qt>, [<Xn|SP>, #<simm>]!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let wback : boolean = TRUE;
let postindex : boolean = FALSE;
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset

(FEAT_FP)



Encoding for the 8-bit variant

Applies when `(size == 00 && opc == 01)`

```
LDR <Bt>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 01)`

```
LDR <Ht>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 01)`

```
LDR <St>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 01)`

```
LDR <Dt>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 11)`

```
LDR <Qt>, [<Xn|SP>{, #<pimm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let wback : boolean = FALSE;
let postindex : boolean = FALSE;
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), scale);
```

Assembler Symbols

<Bt>	Is the 8-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<St>	Is the 32-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt>	Is the 128-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

<pimm> For the "8-bit" variant: is the optional positive immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field.

For the "16-bit" variant: is the optional positive immediate byte offset, a multiple of 2 in the range 0 to 8190, defaulting to 0 and encoded in the "imm12" field as <pimm>/2.

For the "32-bit" variant: is the optional positive immediate byte offset, a multiple of 4 in the range 0 to 16380, defaulting to 0 and encoded in the "imm12" field as <pimm>/4.

For the "64-bit" variant: is the optional positive immediate byte offset, a multiple of 8 in the range 0 to 32760, defaulting to 0 and encoded in the "imm12" field as <pimm>/8.

For the "128-bit" variant: is the optional positive immediate byte offset, a multiple of 16 in the range 0 to 65520, defaulting to 0 and encoded in the "imm12" field as <pimm>/16.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << scale;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal, tagchecked,
                                                    privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

V{datasize}(t) = Mem{datasize}(address, accdesc);

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDR (literal, SIMD&FP)

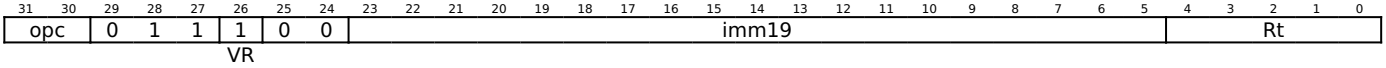
Load SIMD&FP register (PC-relative literal)

This instruction loads a SIMD&FP register from memory. The address that is used for the load is calculated from the PC value and an immediate offset.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Literal

(FEAT_FP)



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
LDR <St>, <label>
```

Encoding for the 64-bit variant

Applies when `(opc == 01)`

```
LDR <Dt>, <label>
```

Encoding for the 128-bit variant

Applies when `(opc == 10)`

```
LDR <Qt>, <label>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
if opc == '11' then EndOfDecode(Decode_UNDEF); end;
let size : integer{} = 4 << (UInt(opc) as integer{0..2});
let nontemporal : boolean = FALSE;
let tagchecked : boolean = FALSE;

let offset : bits(64) = SignExtend{}(imm19::'00');
```

Assembler Symbols

<St>	Is the 32-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<label>	Is the program label from which the data is to be loaded. Its offset from the address of this instruction, in the range +/-1MB, is encoded as "imm19" times 4.
<Dt>	Is the 64-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.
<Qt>	Is the 128-bit name of the SIMD&FP register to be loaded, encoded in the "Rt" field.

Operation

```
let address : bits(64) = PC64() + offset;
AArch64_CheckFPEnabled();
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);
let data : bits(size*8) = Mem{size*8}(address, accdesc);
V{size*8}(t) = data;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDR (register, SIMD&FP)

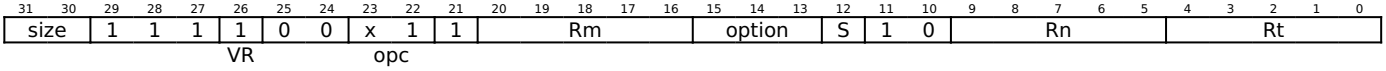
Load SIMD&FP register (register offset)

This instruction loads a SIMD&FP register from memory. The address that is used for the load is calculated from a base register value and an offset register value. The offset can be optionally shifted and extended.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

SIMD&FP registers

(FEAT_FP)



Encoding for the 8-bit with extended register offset variant

Applies when `(size == 00 && opc == 01 && option != 011)`

```
LDR <Bt>, [<Xn|SP>, (<Wm>|<Xm>), <extend> {<amount>}]
```

Encoding for the 8-bit with shifted register offset variant

Applies when `(size == 00 && opc == 01 && option == 011)`

```
LDR <Bt>, [<Xn|SP>, <Xm>{, LSL <amount>}]
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 01)`

```
LDR <Ht>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 01)`

```
LDR <St>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 01)`

```
LDR <Dt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 11)`

```
LDR <Qt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let extend_type : ExtendType = DecodeRegExtend(option);
let shift : integer{} = if S == '1' then scale else 0;
```

Assembler Symbols

<Bt>	Is the 8-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Wm>	When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.
<Xm>	When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.
<extend>	For the "8-bit with extended register offset" variant: is the index extend specifier, encoded in “option”:

option	<extend>
010	UXTW
110	SXTW
111	SXTX

For the "128-bit", "16-bit", "32-bit", and "64-bit" variants: is the index extend/shift specifier, defaulting to LSL, and which must be omitted for the LSL option when <amount> is omitted, encoded in “option”:

option	<extend>
010	UXTW
011	LSL
110	SXTW
111	SXTX

<amount> For the "8-bit with extended register offset" and "8-bit with shifted register offset" variants: is the index shift amount, it must be #0, encoded in "S" as 0 if omitted, or as 1 if present.

For the "16-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#1

For the "32-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#2

For the "64-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#3

For the "128-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#4

<Ht> Is the 16-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

<St> Is the 32-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

<Dt> Is the 64-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

<Qt> Is the 128-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 8 << scale;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
AArch64_CheckFPEnabled();
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal, tagchecked,
                                                    privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

V{datasize}(t) = Mem{datasize}(address, accdesc);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTNP (SIMD&FP)

Load unprivileged pair of SIMD&FP registers, with non-temporal hint

This instruction loads a pair of SIMD&FP registers from memory, issuing a hint to the memory system that the access is non-temporal. The address that is used for the load is calculated from a base register value and an optional immediate offset. For information about non-temporal pair instructions, see *Load/Store SIMD and Floating-point non-temporal pair*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

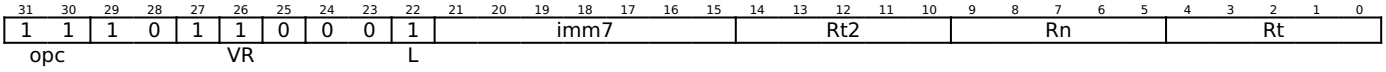
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Signed offset

(FEAT_FP && FEAT_LSUI)



Encoding

```
LDTNP <Qt1>, <Qt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP) || !IsFeatureImplemented(FEAT_LSUI) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = TRUE;
let datasize : integer{} = 128;
let offset : bits(64) = LSL(SignExtend{64}(imm7), 4);
let tagchecked : boolean = n != 31;

var rt_unknown : boolean = FALSE;
if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>     rt_unknown = TRUE;     // Result is UNKNOWN
        when Constraint_UNDEF =>       EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>         EndOfDecode(Decode_NOP);
    end;
end;
```

`LDTNP (SIMD&FP)` has the same CONSTRAINED UNPREDICTABLE behavior as `LDNP (SIMD&FP)`. For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDNP (SIMD&FP)*.

Assembler Symbols

<Qt1>	Is the 128-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt2>	Is the 128-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, defaulting to 0 and encoded in the "imm7" field as <imm>/16.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal, tagchecked,
                                                    privileged, ispair);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

let data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if rt_unknown then
    V{datasize}(t) = ARBITRARY : bits(datasize);
elsif BigEndian(accdesc.acctype) then
    V{datasize}(t2) = data[(datasize-1):0];
    V{datasize}(t)  = data[(2*datasize-1):datasize];
else
    V{datasize}(t)  = data[(datasize-1):0];
    V{datasize}(t2) = data[(2*datasize-1):datasize];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDTP (SIMD&FP)

Load unprivileged pair of SIMD&FP registers

This instruction loads a pair of SIMD&FP registers from memory. The address that is used for the load is calculated from a base register value and an optional immediate offset.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

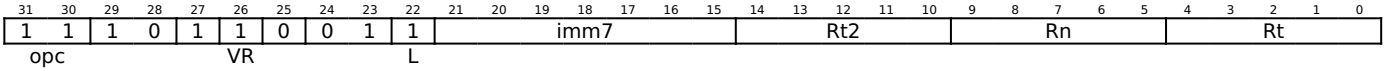
- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_FP && FEAT_LSUI)



Encoding

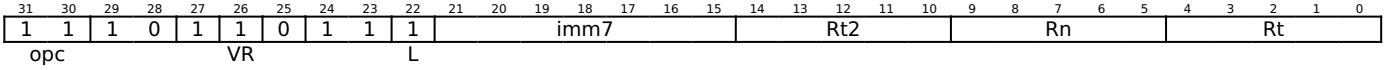
```
LDTP <Qt1>, <Qt2>, [<Xn|SP>], #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP) || !IsFeatureImplemented(FEAT_LSUI) then
    EndOfDecode(Decode_UNDEF);
end;

let wback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_FP && FEAT_LSUI)



Encoding

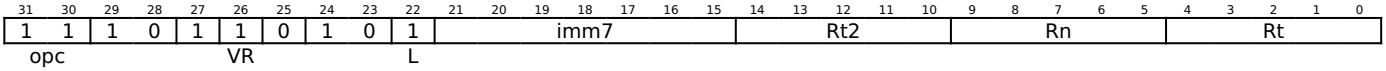
```
LDTP <Qt1>, <Qt2>, [<Xn|SP>, #<imm>]!
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP) || !IsFeatureImplemented(FEAT_LSUI) then
    EndOfDecode(Decode_UNDEF);
end;

let wback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_FP && FEAT_LSUI)



Encoding

```
LDTP <Qt1>, <Qt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP) || !IsFeatureImplemented(FEAT_LSUI) then
    EndOfDecode(Decode_UNDEF);
end;

let wback : boolean = FALSE;
let postindex : boolean = FALSE;
```

LDTP has the same CONSTRAINED UNPREDICTABLE behavior as LDP (SIMD&FP). For information about this CONSTRAINED UNPREDICTABLE behavior, see *Architectural Constraints on UNPREDICTABLE behaviors*, and particularly *LDP (SIMD&FP)*.

Assembler Symbols

<Qt1>	Is the 128-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt2>	Is the 128-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Post-index" and "Pre-index" variants: is the signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, encoded in the "imm7" field as <imm>/16. For the "Signed offset" variant: is the optional signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, defaulting to 0 and encoded in the "imm7" field as <imm>/16.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let datasize : integer{} = 128;
let offset : bits(64) = LSL(SignExtend{64}(imm7), 4);
let tagchecked : boolean = wback || n != 31;

var rt_unknown : boolean = FALSE;
if t == t2 then
    let c : Constraint = ConstrainUnpredictable(Unpredictable_LDPOVERLAP);
    assert c IN {Constraint_UNKNOWN, Constraint_UNDEF, Constraint_NOP};
    case c of
        when Constraint_UNKNOWN =>    rt_unknown = TRUE;    // Result is UNKNOWN
        when Constraint_UNDEF =>      EndOfDecode(Decode_UNDEF);
        when Constraint_NOP =>        EndOfDecode(Decode_NOP);
    end;
end;
```

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged, ispair);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

let data : bits(2*datasize) = Mem{2*datasize}(address, accdesc);
if rt_unknown then
    V{datasize}(t) = ARBITRARY : bits(datasize);
elsif BigEndian(accdesc.acctype) then
    V{datasize}(t2) = data[(datasize-1):0];
    V{datasize}(t)  = data[(2*datasize-1):datasize];
else
    V{datasize}(t)  = data[(datasize-1):0];
    V{datasize}(t2) = data[(2*datasize-1):datasize];
end;

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDUR (SIMD&FP)

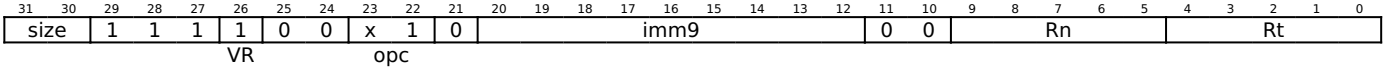
Load SIMD&FP register (unscaled offset)

This instruction loads a SIMD&FP register from memory. The address that is used for the load is calculated from a base register value and an optional immediate offset.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Unscaled offset

(FEAT_FP)



Encoding for the 8-bit variant

Applies when `(size == 00 && opc == 01)`

```
LDUR <Bt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 01)`

```
LDUR <Ht>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 01)`

```
LDUR <St>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 01)`

```
LDUR <Dt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 11)`

```
LDUR <Qt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Bt>	Is the 8-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<St>	Is the 32-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt>	Is the 128-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << scale;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_LOAD, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

V{datasize}(t) = Mem{datasize}(address, accdesc);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

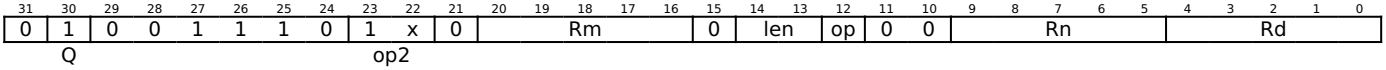
LUTI2

Lookup table read with 2-bit indices

This instruction copies indexed 8-bit or 16-bit elements from the table vector to the destination vector using packed 2-bit indices from a segment of the source vector. A segment corresponds to a portion of the source vector that is consumed in order to fill the destination vector. The segment is selected by the vector segment index.

Advanced SIMD

(FEAT_AdvSIMD && FEAT_LUT)



Encoding for the Byte variant

Applies when `(op2 == 10 && op == 1)`

```
LUTI2 <Vd>.16B, { <Vn>.16B }, <Vm>[<index>]
```

Encoding for the Halfword variant

Applies when `(op2 == 11)`

```
LUTI2 <Vd>.8H, { <Vn>.8H }, <Vm>[<index>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_LUT) then
    EndOfDecode(Decode_UNDEF);
end;
if op2 == '10' && op == '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let isize : integer{} = 2;
let esize : integer{} = if op2 == '10' then 8 else 16;
let part : integer = if op2 == '10' then UInt(len) else UInt(len::op);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP table register, encoded in the "Rn" field.
<Vm>	Is the name of the SIMD&FP source register, encoded in the "Rm" field.
<index>	For the "Byte" variant: is the vector segment index, in the range 0 to 3, encoded in the "len" field. For the "Halfword" variant: is the vector segment index, in the range 0 to 7, encoded in the "len:op" fields.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let elements : integer = 128 DIV esize;
let ibase : integer = elements * part;
let indices : bits(128) = V{128}(m);
let table : bits(128)   = V{128}(n);
var result : bits(128);

for e = 0 to elements-1 do
    let index : integer = UInt(indices[(ibase + e)*:isize]);
    result[e*:esize] = table[index*:esize];
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

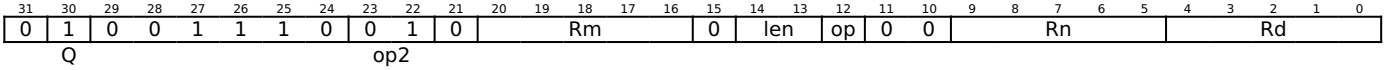
LUTI4

Lookup table read with 4-bit indices

This instruction copies indexed 8-bit or 16-bit elements from the one or two table vectors to the destination vector using packed 4-bit indices from a segment of the source vector. A segment corresponds to a portion of the source vector that is consumed in order to fill the destination vector. The segment is selected by the vector segment index.

Advanced SIMD

(FEAT_AdvSIMD && FEAT_LUT)



Encoding for the Byte variant

Applies when `(len == x1 && op == 0)`

```
LUTI4 <Vd>.16B, { <Vn>.16B }, <Vm>[<index>]
```

Encoding for the Halfword variant

Applies when `(op == 1)`

```
LUTI4 <Vd>.8H, { <Vn1>.8H, <Vn2>.8H }, <Vm>[<index>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_LUT) then
    EndOfDecode(Decode_UNDEF);
end;
if len[0] == '0' && op == '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let isize : integer{} = 4;
let esize : integer{} = 8 << UInt(op);
let ntblr : integer = 1 << UInt(op);
let part : integer = if op == '0' then UInt(len[1]) else UInt(len);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP table register, encoded in the "Rn" field.
<Vm>	Is the name of the SIMD&FP source register, encoded in the "Rm" field.
<index>	For the "Byte" variant: is the vector segment index, in the range 0 to 1, encoded in the "len<1>" field. For the "Halfword" variant: is the vector segment index, in the range 0 to 3, encoded in the "len" field.
<Vn1>	Is the name of the first SIMD&FP table register, encoded in the "Rn" field.
<Vn2>	Is the name of the second SIMD&FP table register, encoded as "Rn" plus 1 modulo 32.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let elements : integer = 128 DIV esize;
let ibase : integer = elements * part;
let indices : bits(128) = V{128}(m);
let table1 : bits(128)  = V{128}(n+0);
let table2 : bits(128)  = if ntblr == 2 then V{128}((n+1) MOD 32) else Zeros{128};
var result : bits(128);
var res : bits(esize);

for e = 0 to elements-1 do
    let index : integer = UInt(indices[(ibase + e)*:isize]);
    if index < elements then
        res = table1[index*:esize];
    else
        assert ntblr == 2;
        res = table2[(index - elements)*:esize];
    end;
    result[e*:esize] = res;
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MLA (by element)

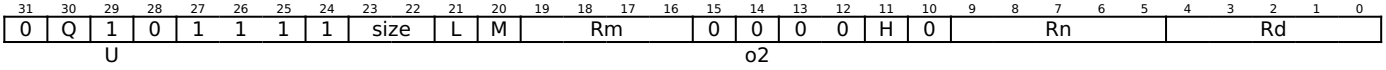
Multiply-add to accumulator (vector, by element)

This instruction multiplies the vector elements in the first source SIMD&FP register by the specified value in the second source SIMD&FP register, and accumulates the results with the vector elements of the destination SIMD&FP register. All the values in this instruction are unsigned integer values.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
MLA <Vd>.<T>, <Vn>.<T>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L);    Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var product : bits(esize);

element2 = UInt(operand2[index*:esize]);
for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    product = (element1 * element2)[esize-1:0];
    result[e*:esize] = operand3[e*:esize] + product;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MLA (vector)

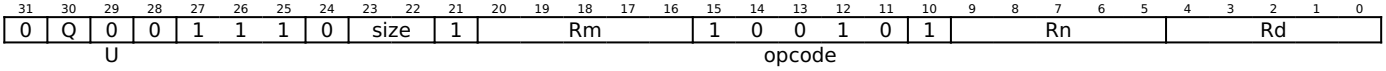
Multiply-add to accumulator (vector)

This instruction multiplies corresponding elements in the vectors of the two source SIMD&FP registers, and accumulates the results with the vector elements of the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
MLA <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);
var product : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    product = (UInt(element1) * UInt(element2))[esize-1:0];
    result[e*:esize] = operand3[e*:esize] + product;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MLS (by element)

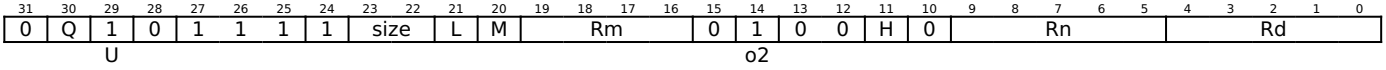
Multiply-subtract from accumulator (vector, by element)

This instruction multiplies the vector elements in the first source SIMD&FP register by the specified value in the second source SIMD&FP register, and subtracts the results from the vector elements of the destination SIMD&FP register. All the values in this instruction are unsigned integer values.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
MLS <Vd>.<T>, <Vn>.<T>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L);    Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var product : bits(esize);

element2 = UInt(operand2[index*:esize]);
for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    product = (element1 * element2)[esize-1:0];
    result[e*:esize] = operand3[e*:esize] - product;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MLS (vector)

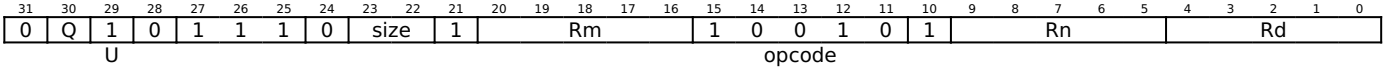
Multiply-subtract from accumulator (vector)

This instruction multiplies corresponding elements in the vectors of the two source SIMD&FP registers, and subtracts the results from the vector elements of the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
MLS <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);
var product : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    product = (UInt(element1) * UInt(element2))[esize-1:0];
    result[e*:esize] = operand3[e*:esize] - product;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (element)

Move vector element to another vector element

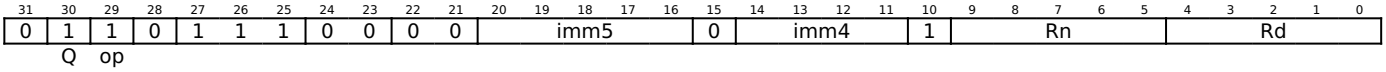
This instruction copies the vector element of the source SIMD&FP register to the specified vector element of the destination SIMD&FP register. This instruction can insert data into individual elements within a SIMD&FP register without clearing the remaining bits to zero.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This is an alias of INS (element). This means:

- The encodings in this description are named to match the encodings of INS (element).
- The description of INS (element) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
MOV <Vd>.<Ts>[<index1>], <Vn>.<Ts>[<index2>]
```

is equivalent to

```
INS <Vd>.<Ts>[<index1>], <Vn>.<Ts>[<index2>]
```

and is always the preferred disassembly.

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ts> Is an element size specifier, encoded in “imm5”:

imm5	<Ts>
x0000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D

<index1> Is the destination element index encoded in “imm5”:

imm5	<index1>
x0000	RESERVED
xxxx1	UInt(imm5[4:1])
xxx10	UInt(imm5[4:2])
xx100	UInt(imm5[4:3])
x1000	UInt(imm5[4])

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<index2> Is the source element index encoded in “(imm5 :: imm4)”:

imm5	<index2>
x0000	RESERVED
xxxx1	UInt(imm4)
xxx10	UInt(imm4[3:1])
xx100	UInt(imm4[3:2])
x1000	UInt(imm4[3])

Unspecified bits in "imm4" are ignored but should be set to zero by an assembler.

Operation

The description of INS (element) gives the operational pseudocode for this instruction.

Operational Information

The description of INS (element) gives the operational information for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (from general)

Move general-purpose register to a vector element

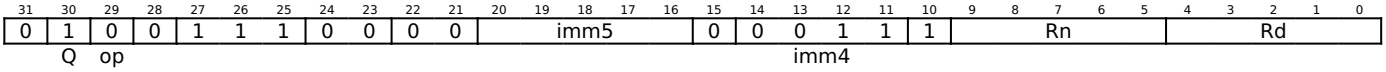
This instruction copies the contents of the source general-purpose register to the specified vector element in the destination SIMD&FP register. This instruction can insert data into individual elements within a SIMD&FP register without clearing the remaining bits to zero.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This is an alias of INS (general). This means:

- The encodings in this description are named to match the encodings of INS (general).
- The description of INS (general) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
MOV <Vd>.<Ts>[<index>], <R><n>
```

is equivalent to

```
INS <Vd>.<Ts>[<index>], <R><n>
```

and is always the preferred disassembly.

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ts> Is an element size specifier, encoded in “imm5”:

imm5	<Ts>
x0000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D

<index> Is the element index encoded in “imm5”:

imm5	<index>
x0000	RESERVED
xxxx1	UInt(imm5[4:1])
xxx10	UInt(imm5[4:2])
xx100	UInt(imm5[4:3])
x1000	UInt(imm5[4])

<R> Is the width specifier for the general-purpose source register, encoded in “imm5”:

imm5	<R>
x0000	RESERVED
xxxx1	W
xxx10	W
xx100	W
x1000	X

<n> Is the number [0-30] of the general-purpose source register or ZR (31), encoded in the "Rn" field.

Operation

The description of INS (general) gives the operational pseudocode for this instruction.

Operational Information

The description of INS (general) gives the operational information for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (scalar)

Move vector element to scalar

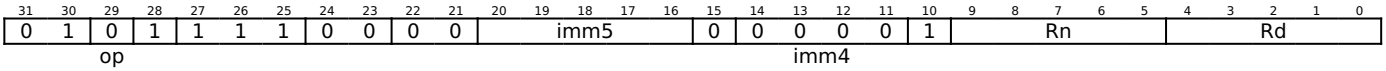
This instruction duplicates the specified vector element in the SIMD&FP source register into a scalar, and writes the result to the SIMD&FP destination register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This is an alias of DUP (element). This means:

- The encodings in this description are named to match the encodings of DUP (element).
- The description of DUP (element) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Scalar

(FEAT_AdvSIMD)



Encoding

```
MOV <V><d>, <Vn>.<T>[<index>]
```

is equivalent to

```
DUP <V><d>, <Vn>.<T>[<index>]
```

and is always the preferred disassembly.

Assembler Symbols

<V> Is the destination width specifier, encoded in “imm5”:

imm5	<V>
x0000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is the element width specifier, encoded in “imm5”:

imm5	<T>
x0000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D

<index> Is the element index encoded in “imm5”:

imm5	<index>
x0000	RESERVED
xxxx1	UInt(imm5[4:1])
xxx10	UInt(imm5[4:2])
xx100	UInt(imm5[4:3])
x1000	UInt(imm5[4])

Operation

The description of DUP (element) gives the operational pseudocode for this instruction.

Operational Information

The description of DUP (element) gives the operational information for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (to general)

Move vector element to general-purpose register

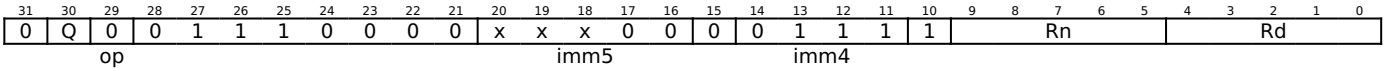
This instruction reads the unsigned integer from the source SIMD&FP register, zero-extends it to form a 32-bit or 64-bit value, and writes the result to the destination general-purpose register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This is an alias of UMOV. This means:

- The encodings in this description are named to match the encodings of UMOV.
- The description of UMOV gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding for the 32-bit variant

Applies when `(Q == 0 && imm5 == xx100)`

```
MOV <Wd>, <Vn>.S[<index>]
```

is equivalent to

```
UMOV <Wd>, <Vn>.S[<index>]
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(Q == 1 && imm5 == x1000)`

```
MOV <Xd>, <Vn>.D[<index>]
```

is equivalent to

```
UMOV <Xd>, <Vn>.D[<index>]
```

and is always the preferred disassembly.

Assembler Symbols

<Wd>	Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.
<Vn>	Is the name of the SIMD&FP source register, encoded in the "Rn" field.
<index>	For the "32-bit" variant: is the element index encoded in "imm5<4:3>". For the "64-bit" variant: is the element index encoded in "imm5<4>".
<Xd>	Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

The description of UMOV gives the operational pseudocode for this instruction.

Operational Information

The description of UMOV gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (vector)

Move vector

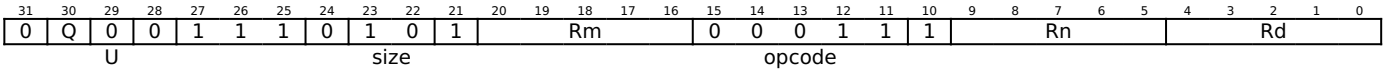
This instruction copies the vector in the source SIMD&FP register into the destination SIMD&FP register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This is an alias of ORR (vector, register). This means:

- The encodings in this description are named to match the encodings of ORR (vector, register).
- The description of ORR (vector, register) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
MOV <Vd>.<T>, <Vn>.<T>
```

is equivalent to

```
ORR <Vd>.<T>, <Vn>.<T>, <Vn>.<T>
```

and is the preferred disassembly when `Rm == Rn`.

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

The description of ORR (vector, register) gives the operational pseudocode for this instruction.

Operational Information

The description of ORR (vector, register) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

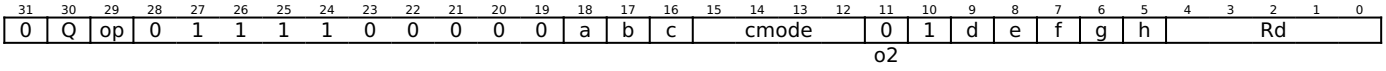
MOVI

Move immediate (vector)

This instruction places an immediate constant into every vector element of the destination SIMD&FP register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding for the 8-bit variant

Applies when `(op == 0 && cmode == 1110)`

```
MOVI <Vd>.<T>, #<imm8>{, LSL #0}
```

Encoding for the 16-bit shifted immediate variant

Applies when `(op == 0 && cmode == 10x0)`

```
MOVI <Vd>.<T>, #<imm8>{, LSL #<amount>}
```

Encoding for the 32-bit shifted immediate variant

Applies when `(op == 0 && cmode == 0xx0)`

```
MOVI <Vd>.<T>, #<imm8>{, LSL #<amount>}
```

Encoding for the 32-bit shifting ones variant

Applies when `(op == 0 && cmode == 110x)`

```
MOVI <Vd>.<T>, #<imm8>, MSL #<amount>
```

Encoding for the 64-bit scalar variant

Applies when `(Q == 0 && op == 1 && cmode == 1110)`

```
MOVI <Dd>, #<imm>
```

Encoding for the 64-bit vector variant

Applies when `(Q == 1 && op == 1 && cmode == 1110)`

```
MOVI <Vd>.2D, #<imm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let rd : integer = UInt(Rd);
let datasize : integer{} = 64 << UInt(Q);
let imm64 : bits(64) = AdvSIMDExpandImm(op, cmode, a::b::c::d::e::f::g::h);
let imm : bits(datasize) = Replicate{}(imm64);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "8-bit" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

For the "16-bit shifted immediate" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "32-bit shifted immediate" and "32-bit shifting ones" variants: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	2S
1	4S

<imm8> Is an 8-bit immediate encoded in "a:b:c:d:e:f:g:h".

<amount> For the "16-bit shifted immediate" variant: is the shift amount encoded in “cmode[1]”:

cmode[1]	<amount>
0	0
1	8

defaulting to 0 if LSL is omitted.

For the "32-bit shifted immediate" variant: is the shift amount encoded in “cmode[2:1]”:

cmode[2:1]	<amount>
00	0
01	8
10	16
11	24

defaulting to 0 if LSL is omitted.

For the "32-bit shifting ones" variant: is the shift amount encoded in “cmode[0]”:

cmode[0]	<amount>
0	8
1	16

<Dd> Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<imm> Is a 64-bit immediate 'aaaaaaaabbbbbbbbccccccccddddddddeeeeeeeeffffffffgggggggghhhhhhhh', encoded in "a:b:c:d:e:f:g:h".

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
V{datasize}(rd) = imm;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MUL (by element)

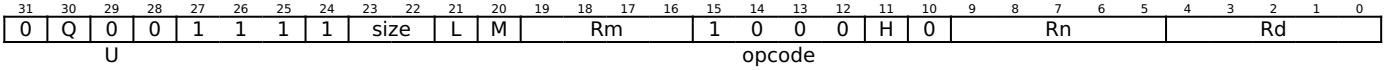
Multiply (vector, by element)

This instruction multiplies the vector elements in the first source SIMD&FP register by the specified value in the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are unsigned integer values.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
MUL <Vd>.<T>, <Vn>.<T>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L);    Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var product : bits(esize);

element2 = UInt(operand2[index*:esize]);
for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    product = (element1 * element2)[esize-1:0];
    result[e*:esize] = product;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MUL (vector)

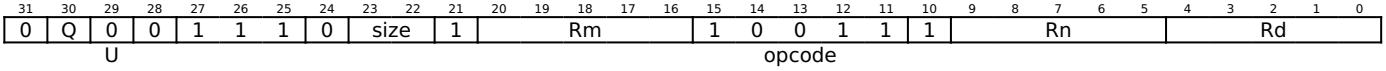
Multiply (vector)

This instruction multiplies corresponding elements in the vectors of the two source SIMD&FP registers, places the results in a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
MUL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if U == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);
var product : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    product = (UInt(element1) * UInt(element2))[esize-1:0];
    result[e*:esize] = product;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MVN

Bitwise NOT (vector)

This instruction reads each vector element from the source SIMD&FP register, places the inverse of each value into a vector, and writes the vector to the destination SIMD&FP register.

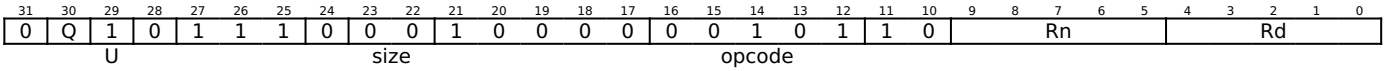
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This is an alias of NOT. This means:

- The encodings in this description are named to match the encodings of NOT.
- The description of NOT gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Vector

(FEAT_AdvSIMD)



Encoding

```
MVN <Vd>.<T>, <Vn>.<T>
```

is equivalent to

```
NOT <Vd>.<T>, <Vn>.<T>
```

and is always the preferred disassembly.

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

The description of NOT gives the operational pseudocode for this instruction.

Operational Information

The description of NOT gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

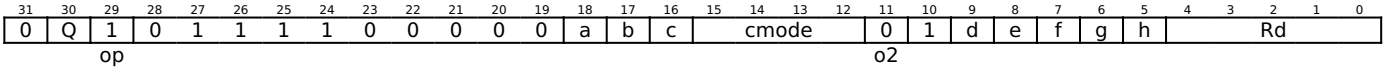
MVNI

Move inverted immediate (vector)

This instruction places the inverse of an immediate constant into every vector element of the destination SIMD&FP register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding for the 16-bit shifted immediate variant

Applies when `(cmode == 10x0)`

```
MVNI <Vd>.<T>, #<imm8>{, LSL #<amount>}
```

Encoding for the 32-bit shifted immediate variant

Applies when `(cmode == 0xx0)`

```
MVNI <Vd>.<T>, #<imm8>{, LSL #<amount>}
```

Encoding for the 32-bit shifting ones variant

Applies when `(cmode == 110x)`

```
MVNI <Vd>.<T>, #<imm8>, MSL #<amount>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let rd : integer = UInt(Rd);
let datasize : integer{} = 64 << UInt(Q);
let imm64 : bits(64) = AdvSIMDExpandImm(op, cmode, a::b::c::d::e::f::g::h);
let imm : bits(datasize) = Replicate{}(imm64);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "16-bit shifted immediate" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "32-bit shifted immediate" and "32-bit shifting ones" variants: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	2S
1	4S

<imm8> Is an 8-bit immediate encoded in "a:b:c:d:e:f:g:h".

<amount> For the "16-bit shifted immediate" variant: is the shift amount encoded in “cmode[1]”:

cmode[1]	<amount>
0	0
1	8

defaulting to 0 if LSL is omitted.

For the "32-bit shifted immediate" variant: is the shift amount encoded in “cmode[2:1]”:

cmode[2:1]	<amount>
00	0
01	8
10	16
11	24

defaulting to 0 if LSL is omitted.

For the "32-bit shifting ones" variant: is the shift amount encoded in “cmode[0]”:

cmode[0]	<amount>
0	8
1	16

Operation

```
AArch64.CheckFPAdvSIMDEnabled();
V{datasize}(rd) = NOT(imm);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

NEG (vector)

Negate (vector)

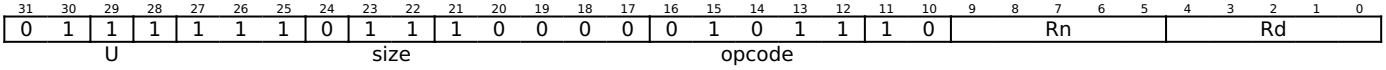
This instruction reads each vector element from the source SIMD&FP register, negates each value, puts the result into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

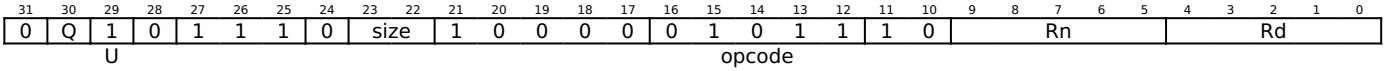
```
NEG D<d>, D<n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
NEG <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    element = SInt(operand[e*:esize]);
    element = -element;
    result[e*:esize] = element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

NOT

Bitwise NOT (vector)

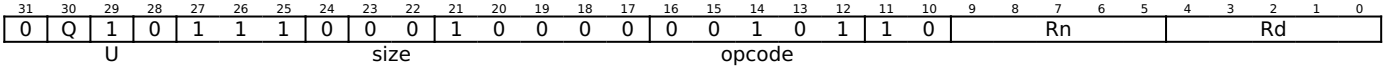
This instruction reads each vector element from the source SIMD&FP register, places the inverse of each value into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This instruction is used by the alias MVN.

Vector

(FEAT_AdvSIMD)



Encoding

```
NOT <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 8;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = NOT(element);
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ORN (vector)

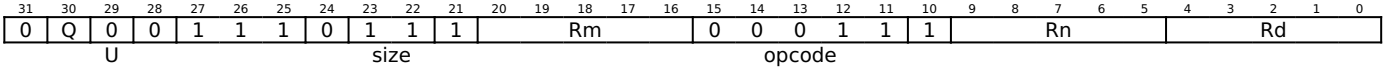
Bitwise inclusive OR NOT (vector)

This instruction performs a bitwise OR NOT between the two source SIMD&FP registers, and writes the result to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
ORN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = NOT(V{datasize}(m));
V{datasize}(d) = operand1 OR operand2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ORR (vector, immediate)

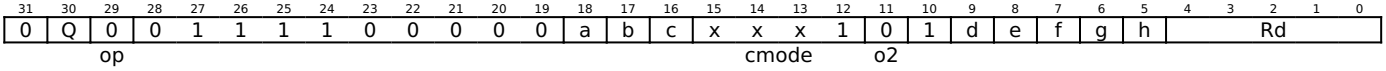
Bitwise inclusive OR (vector, immediate)

This instruction reads each vector element from the destination SIMD&FP register, performs a bitwise OR between each result and an immediate constant, places the result into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Shifted immediate

(FEAT_AdvSIMD)



Encoding for the 16-bit variant

Applies when `(cmode == 10x1)`

```
ORR <Vd>.<T>, #<imm8>{, LSL #<amount>}
```

Encoding for the 32-bit variant

Applies when `(cmode == 0xx1)`

```
ORR <Vd>.<T>, #<imm8>{, LSL #<amount>}
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let rd : integer = UInt(Rd);
let datasize : integer{} = 64 << UInt(Q);
let imm64 : bits(64) = AdvSIMDExpandImm(op, cmode, a::b::c::d::e::f::g::h);
let imm : bits(datasize) = Replicate{}(imm64);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP register, encoded in the "Rd" field.

<T> For the "16-bit" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "32-bit" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	2S
1	4S

<imm8> Is an 8-bit immediate encoded in "a:b:c:d:e:f:g:h".

<amount> For the "16-bit" variant: is the shift amount encoded in “cmode[1]”:

cmode[1]	<amount>
0	0
1	8

defaulting to 0 if LSL is omitted.

For the "32-bit" variant: is the shift amount encoded in “cmode[2:1]”:

cmode[2:1]	<amount>
00	0
01	8
10	16
11	24

defaulting to 0 if LSL is omitted.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(rd);
V{datasize}(rd) = operand OR imm;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

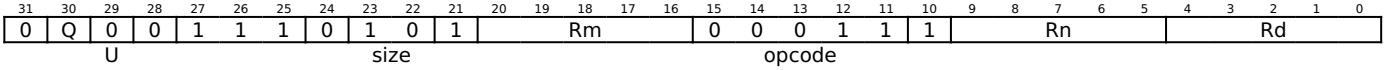
ORR (vector, register)

Bitwise inclusive OR (vector, register)

This instruction performs a bitwise OR between the two source SIMD&FP registers, and writes the result to the destination SIMD&FP register.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
This instruction is used by the alias MOV (vector).

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
ORR <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Alias Conditions

Alias	Is preferred when
MOV (vector)	`Rm == Rn`

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
V{datasize}(d) = operand1 OR operand2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PMUL

Polynomial multiply

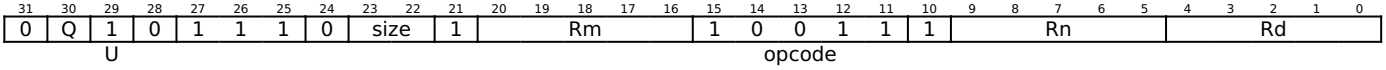
This instruction multiplies corresponding elements in the vectors of the two source SIMD&FP registers, places the results in a vector, and writes the vector to the destination SIMD&FP register.

For information about multiplying polynomials, see *Polynomial arithmetic over {0, 1}*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
PMUL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if U == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	x	RESERVED
1x	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);
var product : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    product = PolynomialMult{esize, esize}(element1, element2)[esize-1:0];
    result[e*:esize] = product;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PMULL, PMULL2

Polynomial multiply long

This instruction multiplies corresponding elements in the lower or upper half of the vectors of the two source SIMD&FP registers, places the results in a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

For information about multiplying polynomials, see *Polynomial arithmetic over {0, 1}*.

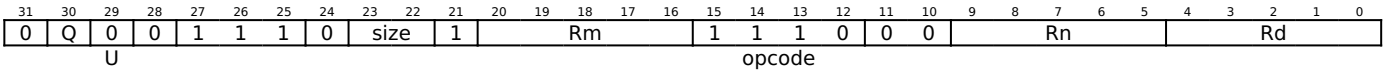
The `PMULL` instruction extracts each source vector from the lower half of each source register. The `PMULL2` instruction extracts each source vector from the upper half of each source register.

The `PMULL` and `PMULL2` variants that operate on 64-bit source elements are defined only when FEAT_PMULL is implemented.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
PMULL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '01' || size == '10' then EndOfDecode(Decode_UNDEF); end;
if size == '11' && !IsFeatureImplemented(FEAT_PMULL) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	RESERVED
10	RESERVED
11	1Q

The '1Q' arrangement is only allocated in an implementation that includes the Cryptographic Extension, and is otherwise RESERVED.

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	x	RESERVED
10	x	RESERVED
11	0	1D
11	1	2D

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:(2*esize)] = PolynomialMult{esize, esize}(element1, element2);
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RADDHN, RADDHN2

Rounding add returning high narrow

This instruction adds each vector element in the first source SIMD&FP register to the corresponding vector element in the second source SIMD&FP register, places the most significant half of the result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register.

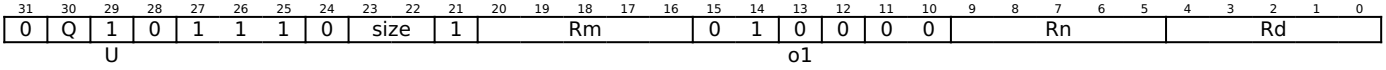
The results are rounded. For truncated results, see *ADDHN*.

The `RADDHN` instruction writes the vector to the lower half of the destination register and clears the upper half. The `RADDHN2` instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
RADDHN{2} <Vd>.<Tb>, <Vn>.<Ta>, <Vm>.<Ta>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
let round : boolean = TRUE;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(2*datasize) = V{}(n);
let operand2 : bits(2*datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:(2*esize)]);
    element2 = UInt(operand2[e*:(2*esize)]);
    sum = element1 + element2;
    sum = RShr(sum, esize, round);
    result[e*:esize] = sum[esize-1:0];
end;

Vpart{datasize}(d, part) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

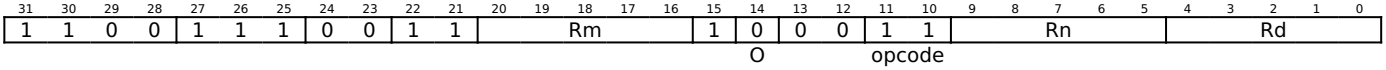
RAX1

Rotate and exclusive-OR

This instruction rotates each 64-bit element of the 128-bit vector in a source SIMD&FP register left by 1, performs a bitwise exclusive-OR of the resulting 128-bit vector and the vector in another source SIMD&FP register, and writes the result to the destination SIMD&FP register.

Advanced SIMD

(FEAT_SHA3)



Encoding

```
RAX1 <Vd>.2D, <Vn>.2D, <Vm>.2D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vm : bits(128) = V{}(m);
let Vn : bits(128) = V{}(n);
V{128}(d) = Vn XOR (ROL(Vm[127:64], 1)::ROL(Vm[63:0], 1));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RBIT (vector)

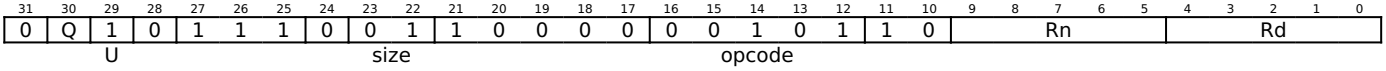
Reverse bit order (vector)

This instruction reads each vector element from the source SIMD&FP register, reverses the bits of the element, places the results into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
RBIT <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 8;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “Q”:

Q	<T>
0	8B
1	16B

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(esize);
var rev : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    for i = 0 to esize-1 do
        rev[(esize-1)-i] = element[i];
    end;
    result[e*:esize] = rev;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

REV16 (vector)

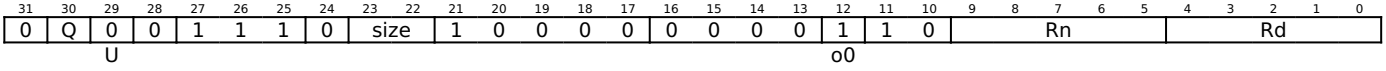
Reverse elements in 16-bit halfwords (vector)

This instruction reverses the order of 8-bit elements in each halfword of the vector in the source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
REV16 <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let csize : integer{} = 64 >> UInt(o0::U);
let esize : integer{} = 8 << UInt(size);
if csize <= esize then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 64 << UInt(Q);
let containers : integer = datasize DIV csize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	x	RESERVED
1x	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
for c = 0 to containers-1 do
    let container : bits(csize) = operand[c*:csize];
    result[c*:csize] = Reverse{csize}(container, esize);
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

REV32 (vector)

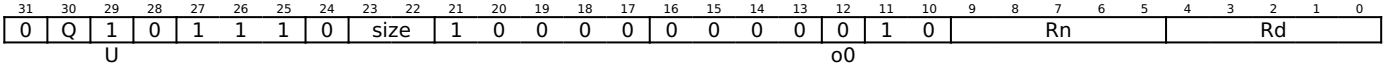
Reverse elements in 32-bit words (vector)

This instruction reverses the order of 8-bit or 16-bit elements in each word of the vector in the source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
REV32 <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let csize : integer{} = 64 >> UInt(o0::U);
let esize : integer{} = 8 << UInt(size);
if csize <= esize then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 64 << UInt(Q);
let containers : integer = datasize DIV csize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
1x	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
for c = 0 to containers-1 do
    let container : bits(csize) = operand[c*:csize];
    result[c*:csize] = Reverse{csize}(container, esize);
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

REV64

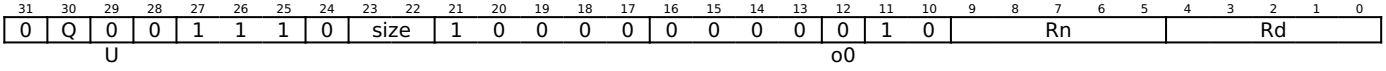
Reverse elements in 64-bit doublewords (vector)

This instruction reverses the order of 8-bit, 16-bit, or 32-bit elements in each doubleword of the vector in the source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
REV64 <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let csize : integer{} = 64 >> UInt(o0::U);
let esize : integer{} = 8 << UInt(size);
if csize <= esize then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 64 << UInt(Q);
let containers : integer = datasize DIV csize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
for c = 0 to containers-1 do
    let container : bits(csize) = operand[c*:csize];
    result[c*:csize] = Reverse{csize}(container, esize);
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RSHRN, RSHRN2

Rounding shift right narrow (immediate)

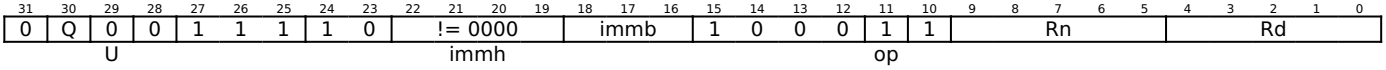
This instruction reads each unsigned integer value from the vector in the source SIMD&FP register, right shifts each result by an immediate value, writes the final result to a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. The destination vector elements are half as long as the source vector elements. The results are rounded. For truncated results, see *SHRN*.

The RSHRN instruction writes the vector to the lower half of the destination register and clears the upper half. The RSHRN2 instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
RSHRN{2} <Vd>.<Tb>, <Vn>.<Ta>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = TRUE;
```

Assembler Symbols

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

<shift> Is the right shift amount, in the range 1 to the destination element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize*2) = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    element = RShr(UInt(operand[e*:(2*esize)]), shift, round);
    result[e*:esize] = element[esize-1:0];
end;

Vpart{datasize}(d, part) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RSUBHN, RSUBHN2

Rounding subtract returning high narrow

This instruction subtracts each vector element of the second source SIMD&FP register from the corresponding vector element of the first source SIMD&FP register, places the most significant half of the result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register.

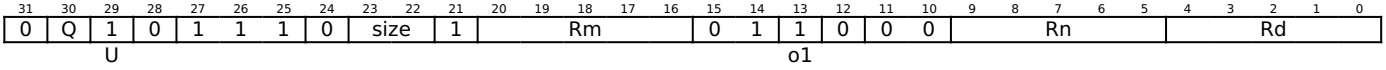
The results are rounded. For truncated results, see *SUBHN*.

The RSUBHN instruction writes the vector to the lower half of the destination register and clears the upper half. The RSUBHN2 instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
RSUBHN{2} <Vd>.<Tb>, <Vn>.<Ta>, <Vm>.<Ta>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
let round : boolean = TRUE;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(2*datasize) = V{}(n);
let operand2 : bits(2*datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:(2*esize)]);
    element2 = UInt(operand2[e*:(2*esize)]);
    sum = element1 - element2;
    sum = RShr(sum, esize, round);
    result[e*:esize] = sum[esize-1:0];
end;

Vpart{datasize}(d, part) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SABA

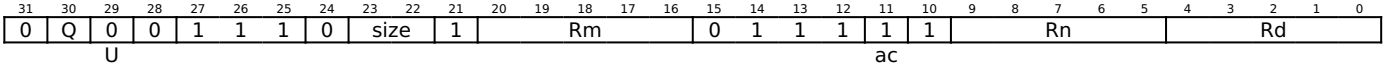
Signed absolute difference and accumulate

This instruction subtracts the elements of the vector of the second source SIMD&FP register from the corresponding elements of the first source SIMD&FP register, and accumulates the absolute values of the results into the elements of the vector of the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
SABA <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize) = V{datasize}(d);
var element1 : integer;
var element2 : integer;
var absdiff : bits(esize);

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    absdiff = Abs(element1 - element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + absdiff;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SABAL, SABAL2

Signed absolute difference and accumulate long

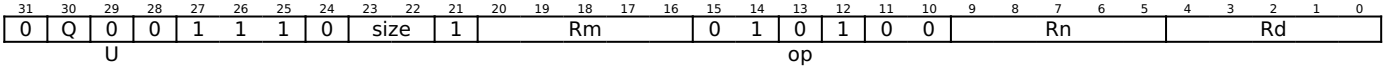
This instruction subtracts the vector elements in the lower or upper half of the second source SIMD&FP register from the corresponding vector elements of the first source SIMD&FP register, and accumulates the absolute values of the results into the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements.

The SABAL instruction extracts each source vector from the lower half of each source register. The SABAL2 instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
SABAL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize) = V{2*datasize}(d);
var element1 : integer;
var element2 : integer;
var absdiff : bits(2*esize);

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    absdiff = Abs(element1-element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] + absdiff;
end;
V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SABD

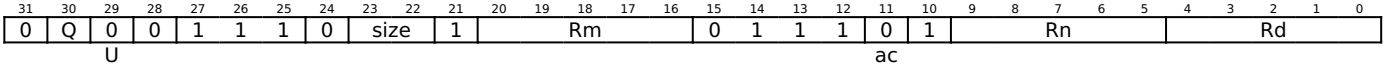
Signed absolute difference

This instruction subtracts the elements of the vector of the second source SIMD&FP register from the corresponding elements of the first source SIMD&FP register, places the absolute values of the results into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
SABD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize) = Zeros{datasize};
var element1 : integer;
var element2 : integer;
var absdiff : bits(esize);

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    absdiff = Abs(element1 - element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + absdiff;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SABDL, SABDL2

Signed absolute difference long

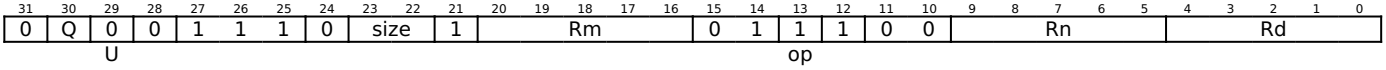
This instruction subtracts the vector elements in the lower or upper half of the second source SIMD&FP register from the corresponding vector elements of the first source SIMD&FP register, places the absolute value of the results into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements.

The SABDL instruction extracts each source vector from the lower half of each source register. The SABDL2 instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
SABDL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize) = Zeros{2*datasize};
var element1 : integer;
var element2 : integer;
var absdiff : bits(2*esize);

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    absdiff = Abs(element1-element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] + absdiff;
end;
V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SADALP

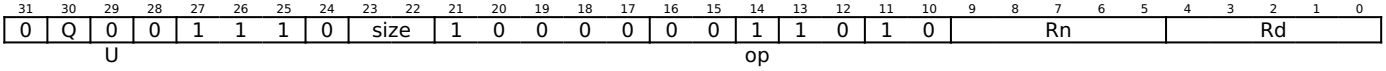
Signed add and accumulate long pairwise

This instruction adds pairs of adjacent signed integer values from the vector in the source SIMD&FP register and accumulates the results into the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
SADALP <Vd>.<Ta>, <Vn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV (2 * esize);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Ta>
00	0	4H
00	1	8H
01	0	2S
01	1	4S
10	0	1D
10	1	2D
11	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize)  = V{}(d);

var sum : bits(2*esize);
var op1 : integer;
var op2 : integer;

for e = 0 to elements-1 do
    op1 = SInt(operand[(2*e+0)*:esize]);
    op2 = SInt(operand[(2*e+1)*:esize]);
    sum = (op1+op2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] + sum;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SADDL, SADDL2

Signed add long (vector)

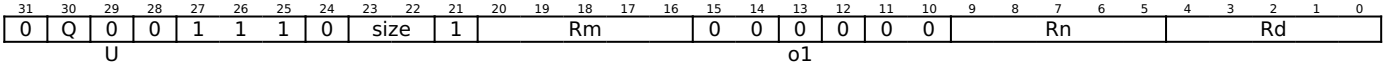
This instruction adds each vector element in the lower or upper half of the first source SIMD&FP register to the corresponding vector element of the second source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements. All the values in this instruction are signed integer values.

The SADDL instruction extracts each source vector from the lower half of each source register. The SADDL2 instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
SADDL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    sum = element1 + element2;
    result[e*:(2*esize)] = sum[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SADDLP

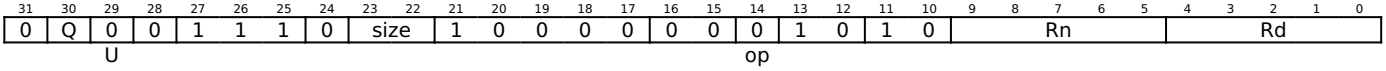
Signed add long pairwise

This instruction adds pairs of adjacent signed integer values from the vector in the source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
SADDLP <Vd>.<Ta>, <Vn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV (2 * esize);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Ta>
00	0	4H
00	1	8H
01	0	2S
01	1	4S
10	0	1D
10	1	2D
11	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

var sum : bits(2*esize);
var op1 : integer;
var op2 : integer;

for e = 0 to elements-1 do
    op1 = SInt(operand[(2*e+0)*:esize]);
    op2 = SInt(operand[(2*e+1)*:esize]);
    sum = (op1+op2)[2*esize-1:0];
    result[e*:(2*esize)] = sum;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SADDLV

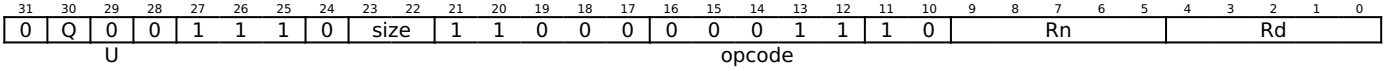
Signed add long across vector

This instruction adds every vector element in the source SIMD&FP register together, and writes the scalar result to the destination SIMD&FP register. The destination scalar is twice as long as the source vector elements. All the values in this instruction are signed integer values.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
SADDLV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '100' then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let unsigned : boolean = FALSE;
```

Assembler Symbols

<V> Is the destination width specifier, encoded in “size”:

size	<V>
00	H
01	S
10	D
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	RESERVED
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var opelt : bits(esize) = operand[0+:esize];
var sum : integer = if unsigned then UInt(opelt) else SInt(opelt);

for e = 1 to elements-1 do
    opelt = operand[e*:esize];
    let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
    sum = sum + element;
end;

V{2*esize}(d) = sum[2*esize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SADDW, SADDW2

Signed add wide

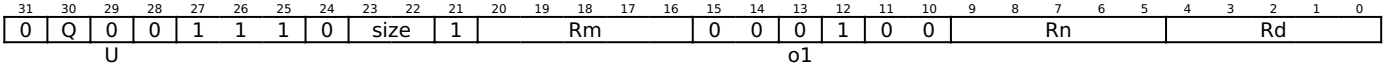
This instruction adds vector elements of the first source SIMD&FP register to the corresponding vector elements in the lower or upper half of the second source SIMD&FP register, places the results in a vector, and writes the vector to the SIMD&FP destination register.

The SADDW instruction extracts the second source vector from the lower half of the second source register. The SADDW2 instruction extracts the second source vector from the upper half of the second source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
SADDW{2} <Vd>.<Ta>, <Vn>.<Ta>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(2*datasize) = V{}(n);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:(2*esize)]);
    element2 = SInt(operand2[e*:esize]);
    sum = element1 + element2;
    result[e*:(2*esize)] = sum[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SCVTF (scalar SIMD&FP)

Signed integer convert to floating-point (scalar SIMD&FP)

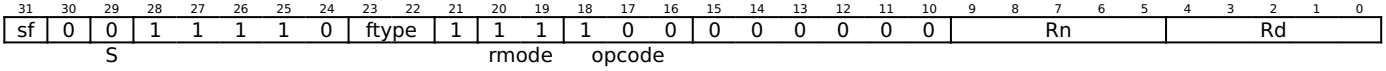
This instruction converts the signed integer value in the SIMD&FP source register to a floating-point value using the rounding mode that is specified by the FPCR, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Integer

(FEAT_FPRCVT)



Encoding for the 32-bit to half-precision variant

Applies when `(sf == 0 && ftype == 11)`

```
SCVTF <Hd>, <Sn>
```

Encoding for the 32-bit to double-precision variant

Applies when `(sf == 0 && ftype == 01)`

```
SCVTF <Dd>, <Sn>
```

Encoding for the 64-bit to half-precision variant

Applies when `(sf == 1 && ftype == 11)`

```
SCVTF <Hd>, <Dn>
```

Encoding for the 64-bit to single-precision variant

Applies when `(sf == 1 && ftype == 00)`

```
SCVTF <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
let rounding : FPRounding = FPRoundingMode(FPCR());
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

intval = V{intsize}(n);
fltval = FixedToFP{fltsize, intsize}(intval, 0, FALSE, FPCR(), rounding);
result[0+:fltsize] = fltval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

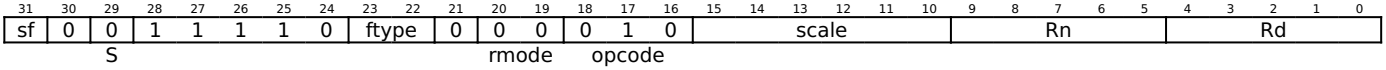
SCVTF (scalar, fixed-point)

Signed fixed-point convert to floating-point (scalar)

This instruction converts the signed value in the 32-bit or 64-bit general-purpose source register to a floating-point value using the rounding mode that is specified by the FPCR, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the 32-bit to half-precision variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
SCVTF <Hd>, <Wn>, #<fbits>
```

Encoding for the 64-bit to half-precision variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
SCVTF <Hd>, <Xn>, #<fbits>
```

Encoding for the 32-bit to single-precision variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
SCVTF <Sd>, <Wn>, #<fbits>
```

Encoding for the 64-bit to single-precision variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
SCVTF <Sd>, <Xn>, #<fbits>
```

Encoding for the 32-bit to double-precision variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
SCVTF <Dd>, <Wn>, #<fbits>
```

Encoding for the 64-bit to double-precision variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
SCVTF <Dd>, <Xn>, #<fbits>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && scale[5] == '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let decode_fltsize : integer{} = 8 << UInt(ftype XOR '10');

let fracbits : integer = 64 - UInt(scale);

let unsigned : boolean = FALSE;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<fbits>	For the "32-bit to double-precision", "32-bit to half-precision", and "32-bit to single-precision" variants: is the number of bits after the binary point in the fixed-point source, in the range 1 to 32, encoded as 64 minus "scale".

	For the "64-bit to double-precision", "64-bit to half-precision", and "64-bit to single-precision" variants: is the number of bits after the binary point in the fixed-point source, in the range 1 to 64, encoded as 64 minus "scale".
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

Operation

```
AArch64_CheckFPEnabled();

let merge : boolean = IsMerging(FPCR());
let fltsize : integer{} = if merge then 128 else decode_fltsize;
var fltval : bits(fltsize) = if merge then V{fltsize}(d) else Zeros{fltsize};
let intval : bits(intsize) = X{}(n);
let rounding : FPRounding = FPRoundingMode(FPCR());

fltval[0+:decode_fltsize] = FixedToFP{decode_fltsize, intsize}(intval, fracbits, unsigned,
                                                               FPCR(), rounding);

V{fltsize}(d) = fltval;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

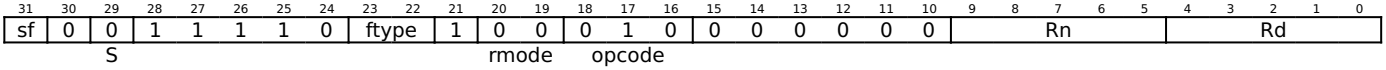
SCVTF (scalar, integer)

Signed integer convert to floating-point (scalar)

This instruction converts the signed integer value in the general-purpose source register to a floating-point value using the rounding mode that is specified by the FPCR, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the 32-bit to half-precision variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
SCVTF <Hd>, <Wn>
```

Encoding for the 32-bit to single-precision variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
SCVTF <Sd>, <Wn>
```

Encoding for the 32-bit to double-precision variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
SCVTF <Dd>, <Wn>
```

Encoding for the 64-bit to half-precision variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
SCVTF <Hd>, <Xn>
```

Encoding for the 64-bit to single-precision variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
SCVTF <Sd>, <Xn>
```

Encoding for the 64-bit to double-precision variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
SCVTF <Dd>, <Xn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let decode_fltsize : integer{} = 8 << UInt(ftype XOR '10');
let unsigned : boolean    = FALSE;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltsize : integer{} = if IsMerging(FPCR()) then 128 else decode_fltsize;

let intval : bits(intsize) = X{}(n);
let rounding : FPRounding  = FPRoundingMode(FPCR());
let fracbits : integer = 0;
var fltval : bits(fltsize) = if IsMerging(FPCR()) then V{fltsize}(d) else Zeros{fltsize};
fltval[0+:decode_fltsize] = FixedToFP{decode_fltsize, intsize}(intval, fracbits, unsigned,
                                                               FPCR(), rounding);

V{fltsize}(d) = fltval;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SCVTF (vector, fixed-point)

Signed fixed-point convert to floating-point (vector)

This instruction converts each element in a vector from fixed-point to floating-point using the rounding mode that is specified by the FPCR, and writes the result to the SIMD&FP destination register.

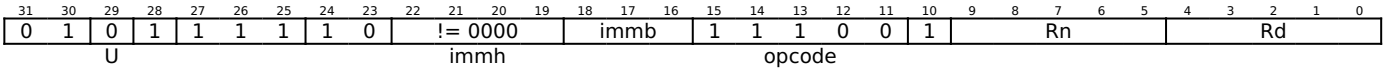
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

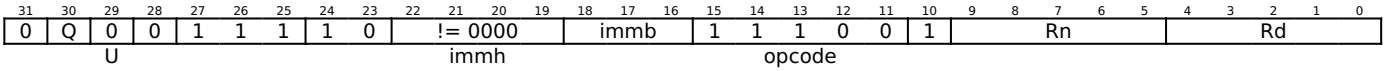
```
SCVTF <V><d>, <V><n>, #<fbits>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh IN {'000x'} || (immh IN {'001x'} && !IsFeatureImplemented(FEAT_FP16)) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = if immh IN {'1xxx'} then 64 else if immh IN {'01xx'} then 32 else 16;
let datasize : integer{} = esize;
let elements : integer = 1;

let fracbits : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SCVTF <Vd>.<T>, <Vn>.<T>, #<fbits>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh IN {'000x'} || (immh IN {'001x'} && !IsFeatureImplemented(FEAT_FP16)) then
    EndOfDecode(Decode_UNDEF);
end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = if immh IN {'1xxx'} then 64 else if immh IN {'01xx'} then 32 else 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let fracbits : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “immh”:

immh	<V>	Architectural Feature
0001	RESERVED	-
001x	H	FEAT_FP16
01xx	S	-
1xxx	D	-

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<fbits> For the "Scalar" variant: is the number of fractional bits, in the range 1 to the operand width, encoded in “(immh :: immb)”:

immh	<fbits>
0001	RESERVED
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

For the "Vector" variant: is the number of fractional bits, in the range 1 to the element width, encoded in “(immh :: immb)”:

immh	<fbits>
0001	RESERVED
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>	Architectural Feature
0001	x	RESERVED	-
001x	0	4H	FEAT_FP16
001x	1	8H	FEAT_FP16
01xx	0	2S	-
01xx	1	4S	-
1xxx	0	RESERVED	-
1xxx	1	2D	-

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
let rounding : FPRounding = FPRoundingMode(FPCR());

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FixedToFP{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SCVTF (vector, integer)

Signed integer convert to floating-point (vector)

This instruction converts each element in a vector from signed integer to floating-point using the rounding mode that is specified by the FPCR, and writes the result to the SIMD&FP destination register.

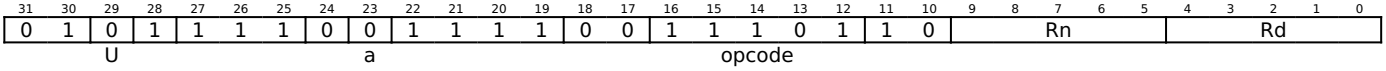
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

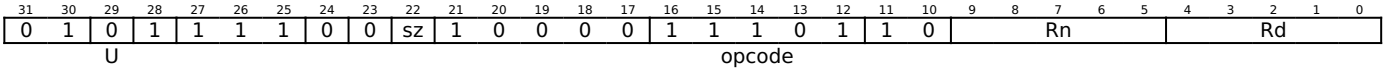
```
SCVTF <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;

let unsigned : boolean = FALSE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

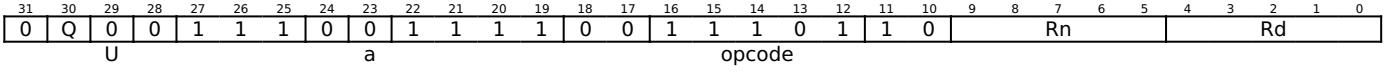
```
SCVTF <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let unsigned : boolean = FALSE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

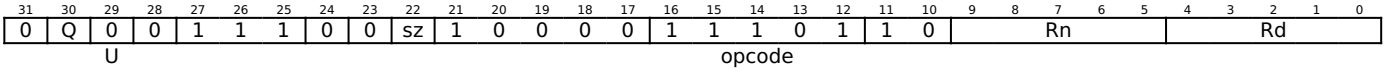
```
SCVTF <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let unsigned : boolean = FALSE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
SCVTF <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let unsigned : boolean = FALSE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let rounding : FPRounding = FPRoundingMode(FPCR());
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FixedToFP{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (by element)

Dot product signed arithmetic (vector, by element)

This instruction performs the dot product of the four 8-bit elements in each 32-bit element of the first source register with the four 8-bit elements of an indexed 32-bit element in the second source register, accumulating the result into the corresponding 32-bit element of the destination register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

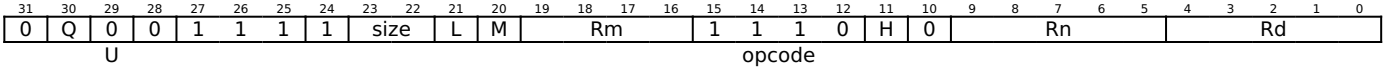
In Armv8.2 and Armv8.3, this is an OPTIONAL instruction. From Armv8.4 it is mandatory for all implementations to support it.

Note

ID_AA64ISAR0_EL1.DP indicates whether this instruction is supported.

Vector

(FEAT_DotProd)



Encoding

```
SDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.4B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_DotProd) then EndOfDecode(Decode_UNDEF); end;
if size  != '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(M::Rm);
let index : integer = UInt(H::L);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> Is the element index, encoded in the "H:L" fields.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(128) = V{}(m);
var result : bits(datasize) = V{}(d);
for e = 0 to elements-1 do
    var res : integer = 0;
    var element1, element2 : integer;
    for i = 0 to 3 do
        element1 = SInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        element2 = SInt(operand2[(4 * index + i)*:(esize DIV 4)]);

        res = res + element1 * element2;
    end;
    result[e*:esize] = result[e*:esize] + res;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (vector)

Dot product signed arithmetic (vector)

This instruction performs the dot product of the four signed 8-bit elements in each 32-bit element of the first source register with the four signed 8-bit elements of the corresponding 32-bit element in the second source register, accumulating the result into the corresponding 32-bit element of the destination register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

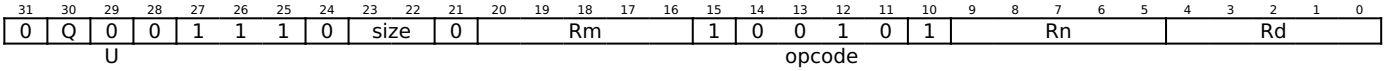
In Armv8.2 and Armv8.3, this is an OPTIONAL instruction. From Armv8.4 it is mandatory for all implementations to support it.

Note

ID_AA64ISAR0_EL1.DP indicates whether this instruction is supported.

Vector

(FEAT_DotProd)



Encoding

```
SDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_DotProd) then EndOfDecode(Decode_UNDEF); end;
if size != '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

result = V{datasize}(d);
for e = 0 to elements-1 do
    var res : integer = 0;
    var element1, element2 : integer;
    for i = 0 to 3 do
        element1 = SInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        element2 = SInt(operand2[(4 * e + i)*:(esize DIV 4)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = result[e*:esize] + res;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

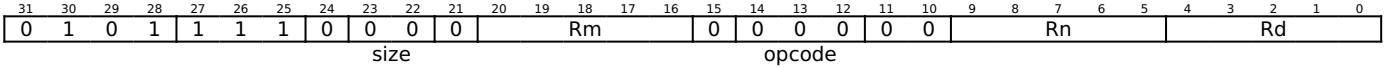
SHA1C

SHA1 hash update (choose)

This instruction reads the five 32-bit words of the SHA-1 working state from the first and second source vectors, together with four 32-bit values from the third source vector, each containing the sum of a message schedule word and the corresponding round constant. The working state undergoes four rounds of the SHA-1 hash computation, applying the choose function and additions defined by the SHA-1 standard. The updated first four words of the working state are destructively placed in the first source vector.

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1C <Qd>, <Sn>, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var x : bits(128) = V{}(d);
var y : bits(32) = V{}(n);    // Note: 32 not 128 bits wide
let w : bits(128) = V{}(m);

for e = 0 to 3 do
    let t : bits(32) = SHAchoose(x[63:32], x[95:64], x[127:96]);
    y = y + ROL(x[31:0], 5) + t + w[e*:32];
    x[63:32] = ROL(x[63:32], 30);
    let yx : bits(160) = ROL(y::x, 32);
    (y, x) = (yx[159:128], yx[127:0]);
end;
V{128}(d) = x;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

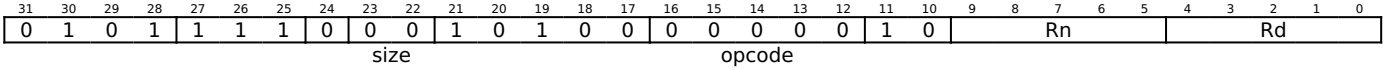
SHA1H

SHA1 fixed rotate

This instruction reads a 32-bit value from the source vector. The value undergoes the fixed 30-bit left rotate operation defined by the SHA-1 standard. The resulting 32-bit value is placed in the destination vector.

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1H <Sd>, <Sn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand : bits(32) = V{}(n);        // read element [0] only,  [1-3] zeroed
V{32}(d) = ROL(operand, 30);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

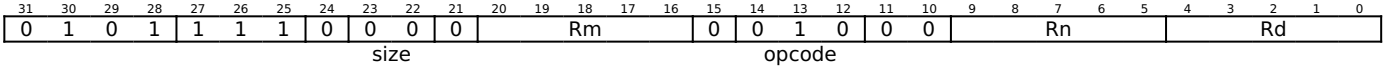
SHA1M

SHA1 hash update (majority)

This instruction reads the five 32-bit words of the SHA-1 working state from the first and second source vectors, together with four 32-bit values from the third source vector, each containing the sum of a message schedule word and the corresponding round constant. The working state undergoes four rounds of the SHA-1 hash computation, applying the majority function and additions defined by the SHA-1 standard. The updated first four words of the working state are destructively placed in the first source vector.

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1M <Qd>, <Sn>, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var x : bits(128) = V{}(d);
var y : bits(32) = V{}(n);    // Note: 32 not 128 bits wide
let w : bits(128) = V{}(m);

for e = 0 to 3 do
    let t : bits(32) = SHAmajority{32}(x[63:32], x[95:64], x[127:96]);
    y = y + ROL(x[31:0], 5) + t + w[e*:32];
    x[63:32] = ROL(x[63:32], 30);
    let yx : bits(160) = ROL(y::x, 32);
    (y, x) = (yx[159:128], yx[127:0]);
end;
V{128}(d) = x;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

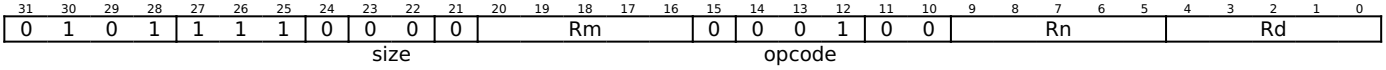
SHA1P

SHA1 hash update (parity)

This instruction reads the five 32-bit words of the SHA-1 working state from the first and second source vectors, together with four 32-bit values from the third source vector, each containing the sum of a message schedule word and the corresponding round constant. The working state undergoes four rounds of the SHA-1 hash computation, applying the parity function and additions defined by the SHA-1 standard. The updated first four words of the working state are destructively placed in the first source vector.

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1P <Qd>, <Sn>, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var x : bits(128) = V{}(d);
var y : bits(32) = V{}(n);    // Note: 32 not 128 bits wide
let w : bits(128) = V{}(m);

for e = 0 to 3 do
    let t : bits(32) = SHAparity(x[63:32], x[95:64], x[127:96]);
    y = y + ROL(x[31:0], 5) + t + w[e*:32];
    x[63:32] = ROL(x[63:32], 30);
    let yx : bits(160) = ROL(y::x, 32);
    (y, x) = (yx[159:128], yx[127:0]);
end;
V{128}(d) = x;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

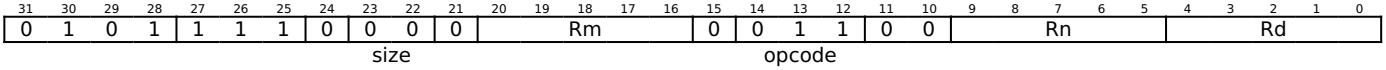
SHA1SU0

SHA1 schedule update 0

This instruction reads previously computed SHA-1 message schedule words from the three source vectors. The values undergo the first part of four successive SHA-1 message schedule updates, applying the exclusive-OR operations defined by the SHA-1 standard. The resulting four 32-bit intermediate values are destructively placed in the first source vector.

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1SU0 <Vd>.4S, <Vn>.4S, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
let operand3 : bits(128) = V{}(m);
var result : bits(128) = operand2[63:0] :: operand1[127:64];
result = result XOR operand1 XOR operand3;
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

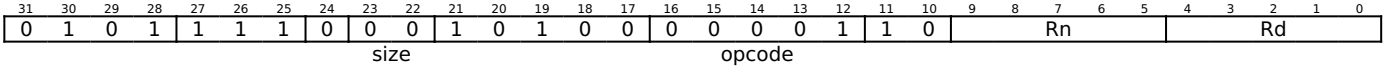
SHA1SU1

SHA1 schedule update 1

This instruction reads four intermediate values for SHA-1 message schedule updates from the first source vector, together with previously computed SHA-1 message schedule words from the second source vector. The values undergo the second part of four successive SHA-1 message schedule updates, applying the exclusive-OR and rotate operations defined by the SHA-1 standard. The resulting four 32-bit message schedule words are destructively placed in the first source vector.

Advanced SIMD

(FEAT_SHA1)



Encoding

```
SHA1SU1 <Vd>.4S, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA1) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
let T : bits(128) = operand1 XOR LSR(operand2, 32);

var result : bits(128);
result[31:0]   = ROL(T[31:0],   1);
result[63:32]  = ROL(T[63:32],  1);
result[95:64]  = ROL(T[95:64],  1);
result[127:96] = ROL(T[127:96], 1) XOR ROL(T[31:0], 2);
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

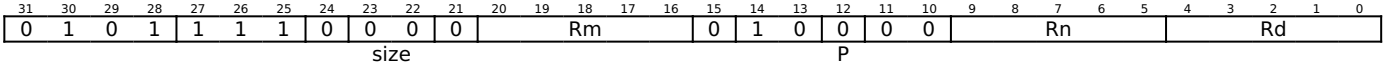
SHA256H

SHA-256 hash update (part 1)

This instruction reads the eight 32-bit words of the SHA-256 working state from the first and second source vectors, together with four 32-bit values from the third source vector, each containing the sum of a message schedule word and the corresponding round constant. The working state undergoes four rounds of the SHA-256 hash computation in accordance with the SHA-256 standard. The updated first half of the working state is destructively placed in the first source vector.

Advanced SIMD

(FEAT_SHA256)



Encoding

```
SHA256H <Qd>, <Qn>, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA256) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination, encoded in the "Rd" field.
<Qn>	Is the 128-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let part1 : boolean = TRUE;
V{128}(d) = SHA256hash(V{128}(d), V{128}(n), V{128}(m), part1);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

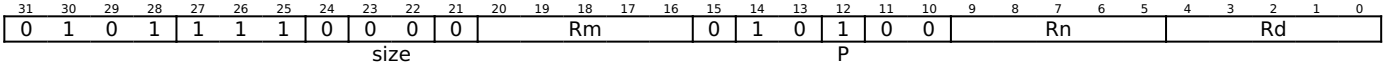
SHA256H2

SHA-256 hash update (part 2)

This instruction reads the eight 32-bit words of the SHA-256 working state from the first and second source vectors, together with four 32-bit values from the third source vector, each containing the sum of a message schedule word and the corresponding round constant. The two halves of the working state are supplied in the opposite order to SHA256H. The working state undergoes four rounds of the SHA-256 hash computation in accordance with the SHA-256 standard. The updated second half of the working state is destructively placed in the first source vector.

Advanced SIMD

(FEAT_SHA256)



Encoding

```
SHA256H2 <Qd>, <Qn>, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA256) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination, encoded in the "Rd" field.
<Qn>	Is the 128-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let part1 : boolean = FALSE;
V{128}(d) = SHA256hash(V{128}(n), V{128}(d), V{128}(m), part1);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

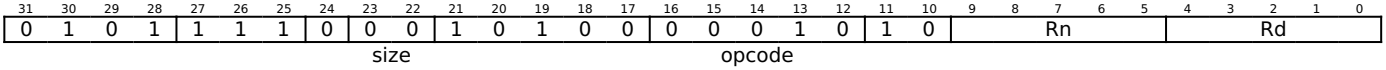
SHA256SU0

SHA-256 schedule update 0

This instruction reads previously computed SHA-256 message schedule words from the first and second source vectors. The values undergo the first part of four successive SHA-256 message schedule updates, applying the sigma0 function and additions defined by the SHA-256 standard. The resulting four 32-bit intermediate values are destructively placed in the first source vector.

Advanced SIMD

(FEAT_SHA256)



Encoding

```
SHA256SU0 <Vd>.4S, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA256) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
let T : bits(128) = operand2[31:0] :: operand1[127:32];
var result : bits(128);
var elt : bits(32);

for e = 0 to 3 do
    elt = T[e*:32];
    elt = ROR(elt, 7) XOR ROR(elt, 18) XOR LSR(elt, 3);
    result[e*:32] = elt + operand1[e*:32];
end;
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

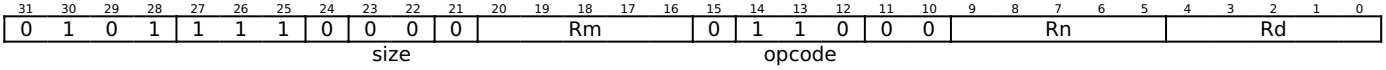
SHA256SU1

SHA-256 schedule update 1

This instruction reads four intermediate values for SHA-256 message schedule updates from the first source vector, together with previously computed SHA-256 message schedule words from the second and third source vectors. The values undergo the second part of four successive SHA-256 message schedule updates, applying the sigma1 function and additions defined by the SHA-256 standard. The resulting four 32-bit message schedule words are destructively placed in the first source vector.

Advanced SIMD

(FEAT_SHA256)



Encoding

```
SHA256SU1 <Vd>.4S, <Vn>.4S, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA256) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(128) = V{}(d);
let operand2 : bits(128) = V{}(n);
let operand3 : bits(128) = V{}(m);
let T0 : bits(128) = operand3[31:0] :: operand2[127:32];

var T1 : bits(64);
var elt : bits(32);
var result : bits(128);

T1 = operand3[127:64];
for e = 0 to 1 do
    elt = T1[e*:32];
    elt = ROR(elt, 17) XOR ROR(elt, 19) XOR LSR(elt, 10);
    elt = elt + operand1[e*:32] + T0[e*:32];
    result[e*:32] = elt;
end;

T1 = result[63:0];
for e = 2 to 3 do
    elt = T1[(e - 2)*:32];
    elt = ROR(elt, 17) XOR ROR(elt, 19) XOR LSR(elt, 10);
    elt = elt + operand1[e*:32] + T0[e*:32];
    result[e*:32] = elt;
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

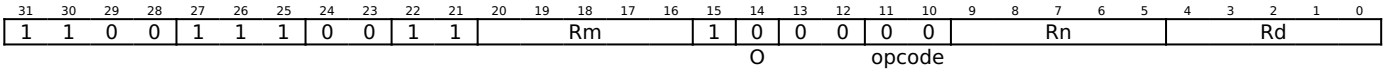
SHA512H

SHA-512 hash update part 1

This instruction takes the values from the three 128-bit source SIMD&FP registers and produces a 128-bit output value that combines the sigma1 and chi functions of two iterations of the SHA-512 calculation. It returns this value to the destination SIMD&FP register.

Advanced SIMD

(FEAT_SHA512)



Encoding

```
SHA512H <Qd>, <Qn>, <Vm>.2D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA512) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Qn>	Is the 128-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var Vtmp : bits(128);
var MSigma1 : bits(64);
var tmp : bits(64);
let x : bits(128) = V{}(n);
let y : bits(128) = V{}(m);
let w : bits(128) = V{}(d);

MSigma1 =  ROR(y[127:64], 14) XOR ROR(y[127:64], 18) XOR ROR(y[127:64], 41);
Vtmp[127:64] =  (y[127:64] AND x[63:0]) XOR (NOT(y[127:64]) AND x[127:64]);
Vtmp[127:64] = (Vtmp[127:64] + MSigma1 +  w[127:64]);
tmp = Vtmp[127:64] + y[63:0];
MSigma1 = ROR(tmp, 14) XOR ROR(tmp, 18) XOR ROR(tmp, 41);
Vtmp[63:0] = (tmp AND y[127:64]) XOR (NOT(tmp) AND x[63:0]);
Vtmp[63:0] = (Vtmp[63:0] + MSigma1 + w[63:0]);
V{128}(d) =  Vtmp;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

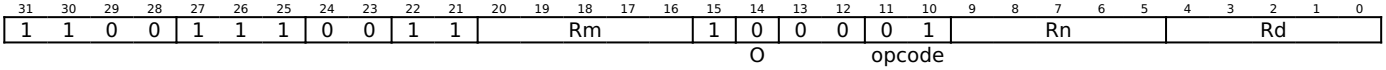
SHA512H2

SHA-512 hash update part 2

This instruction takes the values from the three 128-bit source SIMD&FP registers and produces a 128-bit output value that combines the sigma0 and majority functions of two iterations of the SHA-512 calculation. It returns this value to the destination SIMD&FP register.

Advanced SIMD

(FEAT_SHA512)



Encoding

```
SHA512H2 <Qd>, <Qn>, <Vm>.2D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA512) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Qd>	Is the 128-bit name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Qn>	Is the 128-bit name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var Vtmp : bits(128);
var NSigma0 : bits(64);
let x : bits(128) = V{}(n);
let y : bits(128) = V{}(m);
let w : bits(128) = V{}(d);

NSigma0 =  ROR(y[63:0], 28) XOR ROR(y[63:0], 34) XOR ROR(y[63:0], 39);
Vtmp[127:64] = SHAmajority{64}(x[63:0], y[127:64], y[63:0]);
Vtmp[127:64] = (Vtmp[127:64] + NSigma0 +  w[127:64]);
NSigma0 =  ROR(Vtmp[127:64], 28) XOR ROR(Vtmp[127:64], 34) XOR ROR(Vtmp[127:64], 39);
Vtmp[63:0] = SHAmajority{64}(Vtmp[127:64], y[63:0], y[127:64]);
Vtmp[63:0] =   (Vtmp[63:0] + NSigma0 + w[63:0]);

V{128}(d) = Vtmp;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

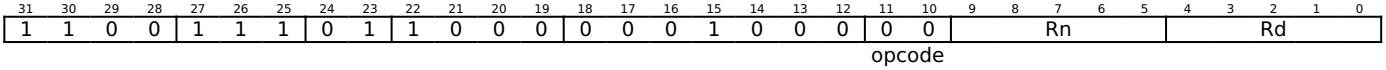
SHA512SU0

SHA-512 schedule update 0

This instruction takes the values from the two 128-bit source SIMD&FP registers and produces a 128-bit output value that combines the gamma0 functions of two iterations of the SHA-512 schedule update that are performed after the first 16 iterations within a block. It returns this value to the destination SIMD&FP register.

Advanced SIMD

(FEAT_SHA512)



Encoding

```
SHA512SU0 <Vd>.2D, <Vn>.2D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA512) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var sig0 : bits(64);
var Vtmp : bits(128);
let x : bits(128) = V{}(n);
let w : bits(128) = V{}(d);
sig0 = ROR(w[127:64], 1) XOR ROR(w[127:64], 8) XOR ('0000000'::w[127:71]);
Vtmp[63:0] = w[63:0] + sig0;
sig0 = ROR(x[63:0], 1) XOR ROR(x[63:0], 8) XOR ('0000000'::x[63:7]);
Vtmp[127:64] = w[127:64] + sig0;
V{128}(d) = Vtmp;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

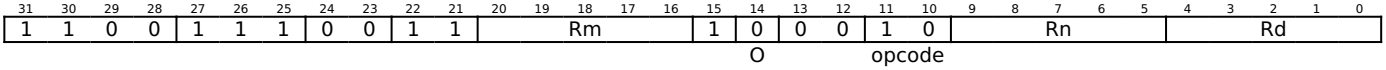
SHA512SU1

SHA-512 schedule update 1

This instruction takes the values from the three source SIMD&FP registers and produces a 128-bit output value that combines the gamma1 functions of two iterations of the SHA-512 schedule update that are performed after the first 16 iterations within a block. It returns this value to the destination SIMD&FP register.

Advanced SIMD

(FEAT_SHA512)



Encoding

```
SHA512SU1 <Vd>.2D, <Vn>.2D, <Vm>.2D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA512) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var sig1 : bits(64);
var Vtmp : bits(128);
let x : bits(128) = V{}(n);
let y : bits(128) = V{}(m);
let w : bits(128) = V{}(d);

sig1 = ROR(x[127:64], 19) XOR ROR(x[127:64], 61) XOR ('000000'::x[127:70]);
Vtmp[127:64] = w[127:64] + sig1 + y[127:64];
sig1 = ROR(x[63:0], 19) XOR ROR(x[63:0], 61) XOR ('000000'::x[63:6]);
Vtmp[63:0] = w[63:0] + sig1 + y[63:0];
V{128}(d) = Vtmp;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SHADD

Signed halving add

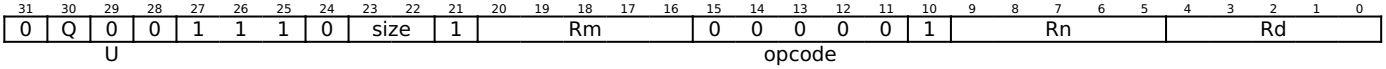
This instruction adds corresponding signed integer values from the two source SIMD&FP registers, shifts each result right one bit, places the results into a vector, and writes the vector to the destination SIMD&FP register.

The results are truncated. For rounded results, see *SRHADD*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
SHADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    sum = (element1 + element2) >> 1;
    result[e*:esize] = sum[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SHL

Shift left (immediate)

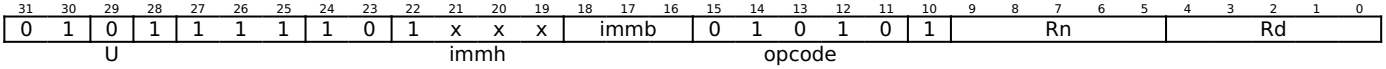
This instruction reads each value from a vector, left shifts each result by an immediate value, writes the final result to a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

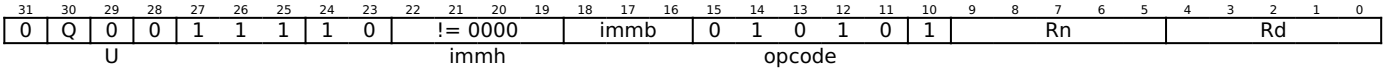
```
SHL D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;
let shift : integer = UInt(immh::immb) - esize;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SHL <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let shift : integer = UInt(immh::immb) - esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the left shift amount, in the range 0 to 63, encoded as UInt("immh:immb") - 64.

For the "Vector" variant: is the left shift amount, in the range 0 to the element width in bits minus 1, encoded in "(immh :: immb)":

immh	<shift>
0001	UInt(immh :: immb) - 8
001x	UInt(immh :: immb) - 16
01xx	UInt(immh :: immb) - 32
1xxx	UInt(immh :: immb) - 64

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

for e = 0 to elements-1 do
    result[e*:esize] = LSL(operand[e*:esize], shift);
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SHLL, SHLL2

Shift left long (by element size)

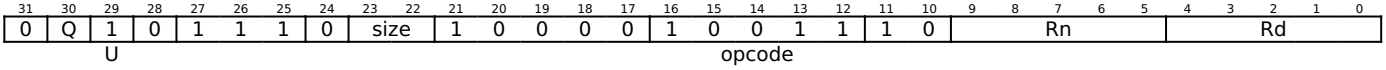
This instruction reads each vector element in the lower or upper half of the source SIMD&FP register, left shifts each result by the element size, writes the final result to a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements.

The `SHLL` instruction extracts vector elements from the lower half of the source register. The `SHLL2` instruction extracts vector elements from the upper half of the source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
SHLL{2} <Vd>.<Ta>, <Vn>.<Tb>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = esize;
```

Assembler Symbols

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<shift> Is the left shift amount, which must be equal to the source element width in bits, encoded in “size”:

size	<shift>
00	8
01	16
10	32
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = Vpart{}(n, part);
var result : bits(2*datasize);
var element : integer;

for e = 0 to elements-1 do
    element = SInt(operand[e*:esize]) << shift;
    result[e*:(2*esize)] = element[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SHRN, SHRN2

Shift right narrow (immediate)

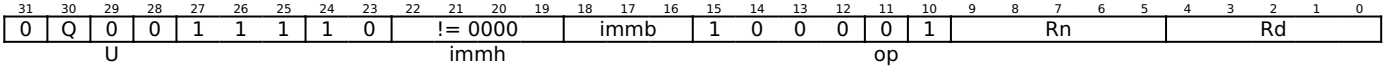
This instruction reads each unsigned integer value from the source SIMD&FP register, right shifts each result by an immediate value, puts the final result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. The destination vector elements are half as long as the source vector elements. The results are truncated. For rounded results, see *RSHRN*.

The SHRN instruction writes the vector to the lower half of the destination register and clears the upper half. The SHRN2 instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
SHRN{2} <Vd>.<Tb>, <Vn>.<Ta>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = FALSE;
```

Assembler Symbols

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

<shift> Is the right shift amount, in the range 1 to the destination element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize*2) = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    element = RShr(UInt(operand[e*:(2*esize)]), shift, round);
    result[e*:esize] = element[esize-1:0];
end;

Vpart{datasize}(d, part) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SHSUB

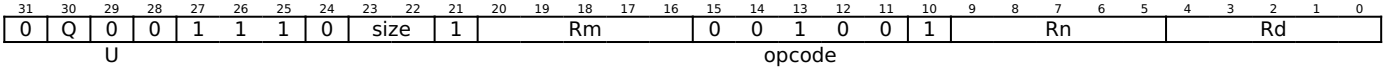
Signed halving subtract

This instruction subtracts the elements in the vector in the second source SIMD&FP register from the corresponding elements in the vector in the first source SIMD&FP register, shifts each result right one bit, places each result into elements of a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
SHSUB <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var diff : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    diff = (element1 - element2) >> 1;
    result[e*:esize] = diff[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

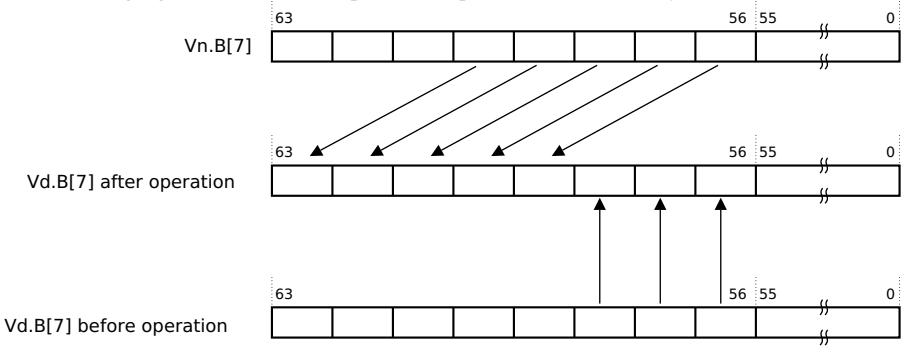
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SLI

Shift left and insert (immediate)

This instruction reads each vector element in the source SIMD&FP register, left shifts each vector element by an immediate value, and inserts the result into the corresponding vector element in the destination SIMD&FP register such that the new zero bits created by the shift are not inserted but retain their existing value. Bits shifted out of the left of each vector element in the source register are lost.

The following figure shows an example of the operation of shift left by 3 for an 8-bit vector element.

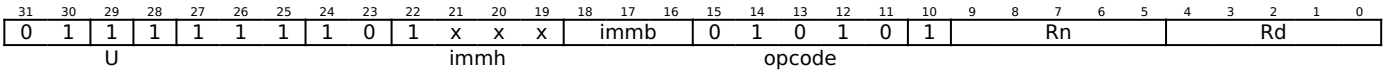


Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

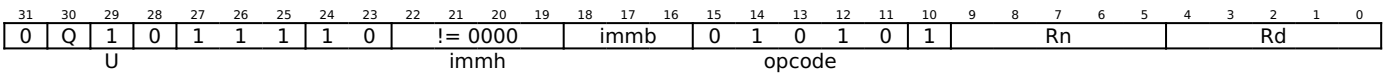
```
SLI D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;
let shift : integer = UInt(immh::immb) - esize;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SLI <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let shift : integer = UInt(immh::immb) - esize;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the SIMD&FP source register, encoded in the "Rn" field.
<shift>	For the "Scalar" variant: is the left shift amount, in the range 0 to 63, encoded as UInt("immh:immb") - 64.

For the "Vector" variant: is the left shift amount, in the range 0 to the element width in bits minus 1, encoded in “(immh :: immb)”:

immh	<shift>
0001	UInt(immh :: immb) - 8
001x	UInt(immh :: immb) - 16
01xx	UInt(immh :: immb) - 32
1xxx	UInt(immh :: immb) - 64

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
let operand2 : bits(datasize) = V{}(d);
let mask : bits(esize) = LSL(Ones{esize}, shift);
var result : bits(datasize);
var shifted : bits(esize);

for e = 0 to elements-1 do
    shifted = LSL(operand[e*:esize], shift);
    result[e*:esize] = (operand2[e*:esize] AND NOT(mask)) OR shifted;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

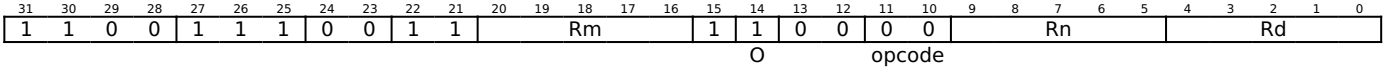
SM3PARTW1

SM3PARTW1

This instruction takes three 128-bit vectors from the three source SIMD&FP registers and returns a 128-bit result in the destination SIMD&FP register. The result is obtained by a three-way exclusive-OR of the elements within the input vectors with some fixed rotations, see the Operation pseudocode for more information.

Advanced SIMD

(FEAT_SM3)



Encoding

```
SM3PARTW1 <Vd>.4S, <Vn>.4S, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SM3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vm : bits(128) = V{}(m);
let Vn : bits(128) = V{}(n);
let Vd : bits(128) = V{}(d);
var result : bits(128);

result[95:0] = (Vd XOR Vn)[95:0] XOR (ROL(Vm[127:96], 15)::ROL(Vm[95:64], 15)::ROL(Vm[63:32], 15));

for i = 0 to 3 do
    if i == 3 then
        result[127:96] = (Vd XOR Vn)[127:96] XOR (ROL(result[31:0], 15));
    end;
    result[(32*i)+31:(32*i)] = (result[(32*i)+31:(32*i)] XOR ROL(result[(32*i)+31:(32*i)], 15) XOR
                                ROL(result[(32*i)+31:(32*i)], 23));
end;
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

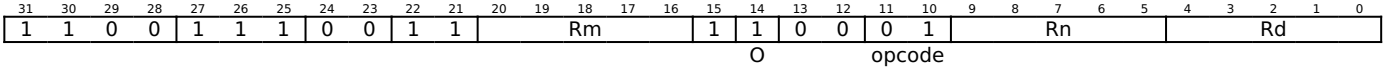
SM3PARTW2

SM3PARTW2

This instruction takes three 128-bit vectors from three source SIMD&FP registers and returns a 128-bit result in the destination SIMD&FP register. The result is obtained by a three-way exclusive-OR of the elements within the input vectors with some fixed rotations, see the Operation pseudocode for more information.

Advanced SIMD

(FEAT_SM3)



Encoding

```
SM3PARTW2 <Vd>.4S, <Vn>.4S, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SM3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vm : bits(128) = V{}(m);
let Vn : bits(128) = V{}(n);
let Vd : bits(128) = V{}(d);

var result : bits(128);
var tmp : bits(128);
var tmp2 : bits(32);

tmp[127:0] = (Vn XOR
              (ROL(Vm[127:96], 7) :: ROL(Vm[95:64], 7) :: ROL(Vm[63:32], 7) :: ROL(Vm[31:0], 7)));
result[127:0] = Vd[127:0] XOR tmp[127:0];
tmp2 = ROL(tmp[31:0], 15);
tmp2 = tmp2 XOR ROL(tmp2, 15) XOR ROL(tmp2, 23);
result[127:96] = result[127:96] XOR tmp2;
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

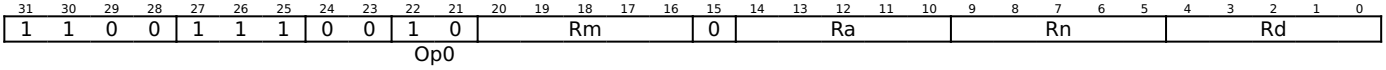
SM3SS1

SM3SS1

This instruction rotates the top 32 bits of the 128-bit vector in the first source SIMD&FP register by 12, and adds that 32-bit value to the two other 32-bit values held in the top 32 bits of each of the 128-bit vectors in the second and third source SIMD&FP registers, rotating this result left by 7 and writing the final result into the top 32 bits of the vector in the destination SIMD&FP register, with the bottom 96 bits of the vector being written to 0.

Advanced SIMD

(FEAT_SM3)



Encoding

```
SM3SS1 <Vd>.4S, <Vn>.4S, <Vm>.4S, <Va>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SM3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let a : integer{} = UInt(Ra);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.
<Va>	Is the name of the third SIMD&FP source register, encoded in the "Ra" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vm : bits(128) = V{}(m);
let Vn : bits(128) = V{}(n);
let Va : bits(128) = V{}(a);

var result : bits(128);
result[127:96] = ROL((ROL(Vn[127:96], 12) + Vm[127:96] + Va[127:96]), 7);
result[95:0] = Zeros{96};
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SM3TT1A

SM3TT1A

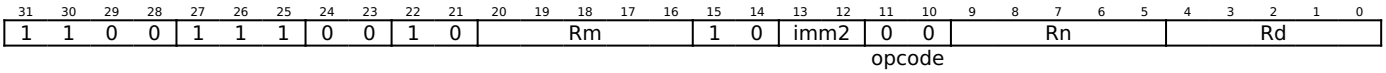
This instruction takes three 128-bit vectors from three source SIMD&FP registers and a 2-bit immediate index value, and returns a 128-bit result in the destination SIMD&FP register. It performs a three-way exclusive-OR of the three 32-bit fields held in the upper three elements of the first source vector, and adds the resulting 32-bit value and the following three other 32-bit values:

- The bottom 32-bit element of the first source vector, Vd, that was used for the three-way exclusive-OR.
- The result of the exclusive-OR of the top 32-bit element of the second source vector, Vn, with a rotation left by 12 of the top 32-bit element of the first source vector.
- A 32-bit element indexed out of the third source vector, Vm.

The result of this addition is returned as the top element of the result. The other elements of the result are taken from elements of the first source vector, with the element returned in bits<63:32> being rotated left by 9.

Advanced SIMD

(FEAT_SM3)



Encoding

```
SM3TT1A <Vd>.4S, <Vn>.4S, <Vm>.S[<imm2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SM3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let i : integer = UInt(imm2);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.
<imm2>	Is a 32-bit element indexed out of <Vm>, encoded in "imm2".

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vm : bits(128) = V{}(m);
let Vn : bits(128) = V{}(n);
let Vd : bits(128) = V{}(d);

var WjPrime : bits(32);
var result : bits(128);
var TT1 : bits(32);
var SS2 : bits(32);

WjPrime = Vm[i*:32];
SS2 = Vn[127:96] XOR ROL(Vd[127:96], 12);
TT1 = Vd[63:32] XOR (Vd[127:96] XOR Vd[95:64]);
TT1 = (TT1 + Vd[31:0] + SS2 + WjPrime)[31:0];
result[31:0] = Vd[63:32];
result[63:32] = ROL(Vd[95:64], 9);
result[95:64] = Vd[127:96];
result[127:96] = TT1;
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SM3TT1B

SM3TT1B

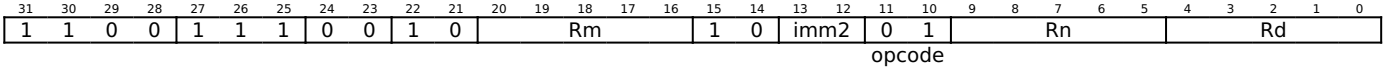
This instruction takes three 128-bit vectors from three source SIMD&FP registers and a 2-bit immediate index value, and returns a 128-bit result in the destination SIMD&FP register. It performs a 32-bit majority function between the three 32-bit fields held in the upper three elements of the first source vector, and adds the resulting 32-bit value and the following three other 32-bit values:

- The bottom 32-bit element of the first source vector, Vd, that was used for the 32-bit majority function.
- The result of the exclusive-OR of the top 32-bit element of the second source vector, Vn, with a rotation left by 12 of the top 32-bit element of the first source vector.
- A 32-bit element indexed out of the third source vector, Vm.

The result of this addition is returned as the top element of the result. The other elements of the result are taken from elements of the first source vector, with the element returned in bits<63:32> being rotated left by 9.

Advanced SIMD

(FEAT_SM3)



Encoding

```
SM3TT1B <Vd>.4S, <Vn>.4S, <Vm>.S[<imm2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SM3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let i : integer = UInt(imm2);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.
<imm2>	Is a 32-bit element indexed out of <Vm>, encoded in "imm2".

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vm : bits(128) = V{}(m);
let Vn : bits(128) = V{}(n);
let Vd : bits(128) = V{}(d);

var WjPrime : bits(32);
var result : bits(128);
var TT1 : bits(32);
var SS2 : bits(32);

WjPrime = Vm[i*:32];
SS2 = Vn[127:96] XOR ROL(Vd[127:96], 12);
TT1 = (Vd[127:96] AND Vd[63:32]) OR (Vd[127:96] AND Vd[95:64]) OR (Vd[63:32] AND Vd[95:64]);
TT1 = (TT1 + Vd[31:0] + SS2 + WjPrime)[31:0];
result[31:0] = Vd[63:32];
result[63:32] = ROL(Vd[95:64], 9);
result[95:64] = Vd[127:96];
result[127:96] = TT1;
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SM3TT2A

SM3TT2A

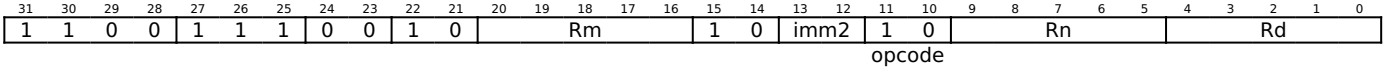
This instruction takes three 128-bit vectors from three source SIMD&FP register and a 2-bit immediate index value, and returns a 128-bit result in the destination SIMD&FP register. It performs a three-way exclusive-OR of the three 32-bit fields held in the upper three elements of the first source vector, and adds the resulting 32-bit value and the following three other 32-bit values:

- The bottom 32-bit element of the first source vector, Vd, that was used for the three-way exclusive-OR.
- The 32-bit element held in the top 32 bits of the second source vector, Vn.
- A 32-bit element indexed out of the third source vector, Vm.

A three-way exclusive-OR is performed of the result of this addition, the result of the addition rotated left by 9, and the result of the addition rotated left by 17. The result of this exclusive-OR is returned as the top element of the returned result. The other elements of this result are taken from elements of the first source vector, with the element returned in bits<63:32> being rotated left by 19.

Advanced SIMD

(FEAT_SM3)



Encoding

```
SM3TT2A <Vd>.4S, <Vn>.4S, <Vm>.S[<imm2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SM3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let i : integer = UInt(imm2);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.
<imm2>	Is a 32-bit element indexed out of <Vm>, encoded in "imm2".

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vm : bits(128) = V{}(m);
let Vn : bits(128) = V{}(n);
let Vd : bits(128) = V{}(d);

var Wj : bits(32);
var result : bits(128);
var TT2 : bits(32);

Wj = Vm[i*:32];
TT2 = Vd[63:32] XOR (Vd[127:96] XOR Vd[95:64]);
TT2 = (TT2 + Vd[31:0] + Vn[127:96] + Wj)[31:0];

result[31:0] = Vd[63:32];
result[63:32] = ROL(Vd[95:64], 19);
result[95:64] = Vd[127:96];
result[127:96] = TT2 XOR ROL(TT2, 9) XOR ROL(TT2, 17);
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SM3TT2B

SM3TT2B

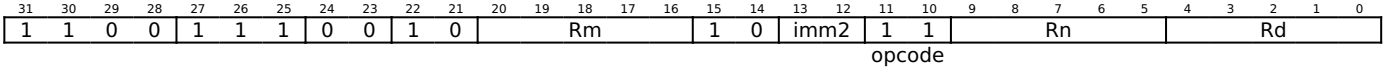
This instruction takes three 128-bit vectors from three source SIMD&FP registers, and a 2-bit immediate index value, and returns a 128-bit result in the destination SIMD&FP register. It performs a 32-bit majority function between the three 32-bit fields held in the upper three elements of the first source vector, and adds the resulting 32-bit value and the following three other 32-bit values:

- The bottom 32-bit element of the first source vector, Vd, that was used for the 32-bit majority function.
- The 32-bit element held in the top 32 bits of the second source vector, Vn.
- A 32-bit element indexed out of the third source vector, Vm.

A three-way exclusive-OR is performed of the result of this addition, the result of the addition rotated left by 9, and the result of the addition rotated left by 17. The result of this exclusive-OR is returned as the top element of the returned result. The other elements of this result are taken from elements of the first source vector, with the element returned in bits<63:32> being rotated left by 19.

Advanced SIMD

(FEAT_SM3)



Encoding

```
SM3TT2B <Vd>.4S, <Vn>.4S, <Vm>.S[<imm2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SM3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let i : integer = UInt(imm2);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the third SIMD&FP source register, encoded in the "Rm" field.
<imm2>	Is a 32-bit element indexed out of <Vm>, encoded in "imm2".

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vm : bits(128) = V{}(m);
let Vn : bits(128) = V{}(n);
let Vd : bits(128) = V{}(d);

var Wj : bits(32);
var result : bits(128);
var TT2 : bits(32);

Wj = Vm[i*:32];
TT2 = (Vd[127:96] AND Vd[95:64]) OR (NOT(Vd[127:96]) AND Vd[63:32]);
TT2 = (TT2 + Vd[31:0] + Vn[127:96] + Wj)[31:0];

result[31:0] = Vd[63:32];
result[63:32] = ROL(Vd[95:64], 19);
result[95:64] = Vd[127:96];
result[127:96] = TT2 XOR ROL(TT2, 9) XOR ROL(TT2, 17);
V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

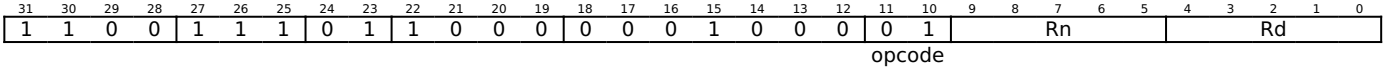
SM4E

SM4 encode

This instruction takes input data as a 128-bit vector from the first source SIMD&FP register, and four iterations of the round key held as the elements of the 128-bit vector in the second source SIMD&FP register. It encrypts the data by four rounds, in accordance with the SM4 standard, returning the 128-bit result to the destination SIMD&FP register.

Advanced SIMD

(FEAT_SM4)



Encoding

```
SM4E <Vd>.4S, <Vn>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SM4) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the second SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vn : bits(128) = V{}(n);
var intval : bits(32);
var roundresult : bits(128);
var roundkey : bits(32);

roundresult = V{128}(d);
for index = 0 to 3 do
    roundkey = Vn[index*:32];

    intval = roundresult[127:96] XOR roundresult[95:64] XOR roundresult[63:32] XOR roundkey;

    for i = 0 to 3 do
        intval[i*:8]  = Sbox(intval[i*:8]);
    end;

    intval = intval XOR ROL(intval, 2) XOR ROL(intval, 10) XOR ROL(intval, 18) XOR ROL(intval, 24);
    intval = intval XOR roundresult[31:0];

    roundresult[31:0]   = roundresult[63:32];
    roundresult[63:32]  = roundresult[95:64];
    roundresult[95:64]  = roundresult[127:96];
    roundresult[127:96] = intval;
end;

V{128}(d) = roundresult;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

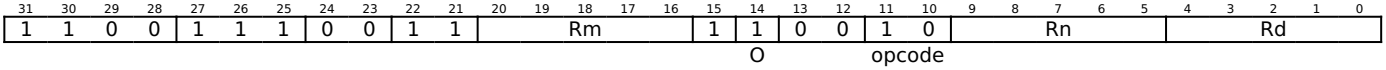
SM4EKEY

SM4 key

This instruction takes an input as a 128-bit vector from the first source SIMD&FP register and a 128-bit constant from the second SIMD&FP register. It derives four iterations of the output key, in accordance with the SM4 standard, returning the 128-bit result to the destination SIMD&FP register.

Advanced SIMD

(FEAT_SM4)



Encoding

```
SM4EKEY <Vd>.4S, <Vn>.4S, <Vm>.4S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SM4) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vm : bits(128) = V{}(m);
var intval : bits(32);
var const : bits(32);
var roundresult : bits(128);

roundresult = V{128}(n);
for index = 0 to 3 do
    const = Vm[index*:32];

    intval = roundresult[127:96] XOR roundresult[95:64] XOR roundresult[63:32] XOR const;

    for i = 0 to 3 do
        intval[i*:8] = Sbox(intval[i*:8]);
    end;

    intval = intval XOR ROL(intval, 13) XOR ROL(intval, 23);
    intval = intval XOR roundresult[31:0];

    roundresult[31:0]   = roundresult[63:32];
    roundresult[63:32]  = roundresult[95:64];
    roundresult[95:64]  = roundresult[127:96];
    roundresult[127:96] = intval;
end;

V{128}(d) = roundresult;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMAX

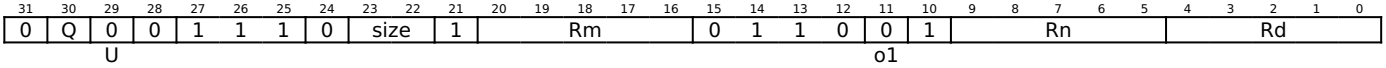
Signed maximum (vector)

This instruction compares corresponding elements in the vectors in the two source SIMD&FP registers, places the larger of each pair of signed integer values into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
SMAX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var max : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    max = Max(element1, element2);
    result[e*:esize] = max[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMAXP

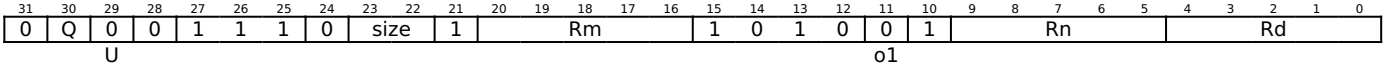
Signed maximum pairwise

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements in the two source SIMD&FP registers, writes the largest of each pair of signed integer values into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
SMAXP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : integer;
var element2 : integer;
var max : integer;

for e = 0 to elements-1 do
    element1 = SInt(concat[(2*e)*:esize]);
    element2 = SInt(concat[((2*e)+1)*:esize]);
    max = Max(element1, element2);
    result[e*:esize] = max[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMAXV

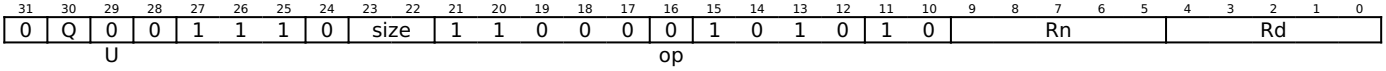
Signed maximum across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the largest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are signed integer values.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
SMAXV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '100' then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let unsigned : boolean = FALSE;
```

Assembler Symbols

<V> Is the destination width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	RESERVED
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var opelt : bits(esize) = operand[0+:esize];
var max : integer = if unsigned then UInt(opelt) else SInt(opelt);

for e = 1 to elements-1 do
    opelt = operand[e*:esize];
    let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
    max = Max(max, element);
end;

V{esize}(d) = max[esize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMIN

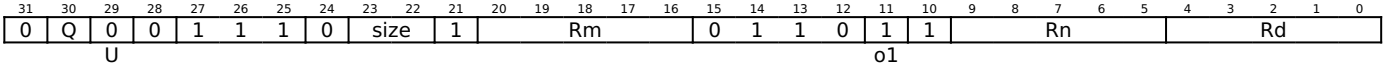
Signed minimum (vector)

This instruction compares corresponding elements in the vectors in the two source SIMD&FP registers, places the smaller of each of the two signed integer values into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
SMIN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var min : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    min = Min(element1, element2);
    result[e*:esize] = min[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMINP

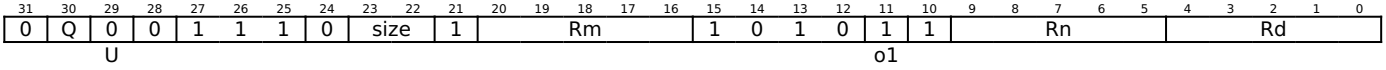
Signed minimum pairwise

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements in the two source SIMD&FP registers, writes the smallest of each pair of signed integer values into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
SMINP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : integer;
var element2 : integer;
var min : integer;

for e = 0 to elements-1 do
    element1 = SInt(concat[(2*e)*:esize]);
    element2 = SInt(concat[((2*e)+1)*:esize]);
    min = Min(element1, element2);
    result[e*:esize] = min[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMINV

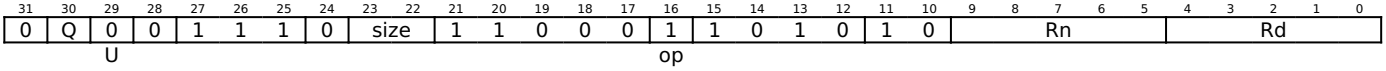
Signed minimum across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the smallest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are signed integer values.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
SMINV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '100' then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let unsigned : boolean = FALSE;
```

Assembler Symbols

<V> Is the destination width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	RESERVED
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var opelt : bits(esize) = operand[0+:esize];
var min : integer = if unsigned then UInt(opelt) else SInt(opelt);

for e = 1 to elements-1 do
    opelt = operand[e*:esize];
    let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
    min = Min(min, element);
end;

V{esize}(d) = min[esize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLAL, SMLAL2 (by element)

Signed multiply-add long (vector, by element)

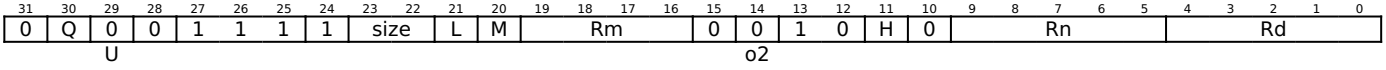
This instruction multiplies each vector element in the lower or upper half of the first source SIMD&FP register by the specified vector element in the second source SIMD&FP register, and accumulates the results with the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied. All the values in this instruction are signed integer values.

The SMLAL instruction extracts vector elements from the lower half of the first source register. The SMLAL2 instruction extracts vector elements from the upper half of the first source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
SMLAL{2} <Vd>.<Ta>, <Vn>.<Tb>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
let element2 : integer = SInt(operand2[index*:esize]);
var product : bits(2*esize);

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    product = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = operand3[e*:(2*esize)] + product;
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLAL, SMLAL2 (vector)

Signed multiply-add long (vector)

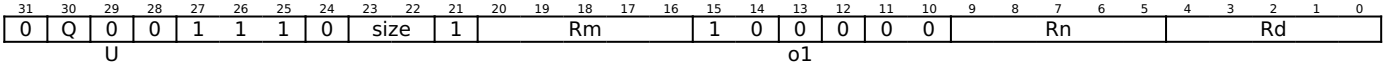
This instruction multiplies corresponding signed integer values in the lower or upper half of the vectors of the two source SIMD&FP registers, and accumulates the results with the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

The SMLAL instruction extracts each source vector from the lower half of each source register. The SMLAL2 instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
SMLAL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var product : bits(2*esize);
var accum : bits(2*esize);

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    product = (element1 * element2)[2*esize-1:0];
    accum = operand3[e*:(2*esize)] + product;
    result[e*:(2*esize)] = accum;
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLSL, SMLSL2 (by element)

Signed multiply-subtract long (vector, by element)

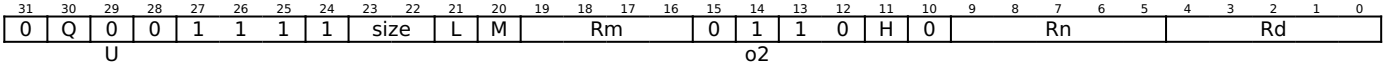
This instruction multiplies each vector element in the lower or upper half of the first source SIMD&FP register by the specified vector element of the second source SIMD&FP register and subtracts the results from the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

The `SMLSL` instruction extracts vector elements from the lower half of the first source register. The `SMLSL2` instruction extracts vector elements from the upper half of the first source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
SMLSL{2} <Vd>.<Ta>, <Vn>.<Tb>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
let element2 : integer = SInt(operand2[index*:esize]);
var product : bits(2*esize);

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    product = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = operand3[e*:(2*esize)] - product;
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLSL, SMLSL2 (vector)

Signed multiply-subtract long (vector)

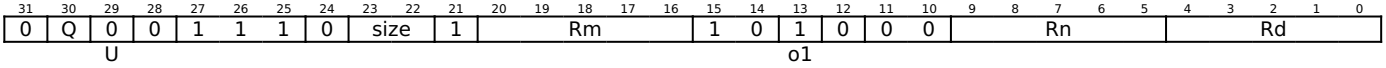
This instruction multiplies corresponding signed integer values in the lower or upper half of the vectors of the two source SIMD&FP registers, and subtracts the results from the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

The `SMLSL` instruction extracts each source vector from the lower half of each source register. The `SMLSL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
SMLSL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var product : bits(2*esize);
var accum : bits(2*esize);

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    product = (element1 * element2)[2*esize-1:0];
    accum = operand3[e*:(2*esize)] - product;
    result[e*:(2*esize)] = accum;
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMMLA (vector)

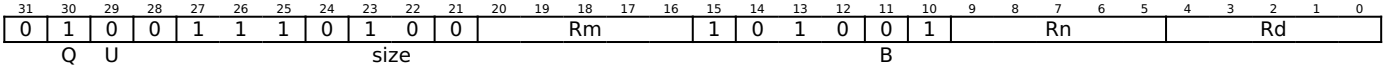
Signed 8-bit integer matrix multiply-accumulate to 32-bit integer (vector)

This instruction multiplies the 2x8 matrix of signed 8-bit integer values in the first source vector by the 8x2 matrix of signed 8-bit integer values in the second source vector. The resulting 2x2 32-bit integer matrix product is destructively added to the 32-bit integer matrix accumulator in the destination vector. This is equivalent to performing an 8-way dot product per destination element.

From Armv8.2 to Armv8.5, this is an OPTIONAL instruction. From Armv8.6 it is mandatory for implementations that include Advanced SIMD to support it. ID_AA64ISAR1_EL1.I8MM indicates whether this instruction is supported.

Vector

(FEAT_I8MM)



Encoding

```
SMMLA <Vd>.4S, <Vn>.16B, <Vm>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_I8MM) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let addend : bits(128)   = V{}(d);

V{128}(d) = MatMulAdd(addend, operand1, operand2, op1_unsigned, op2_unsigned);
```

Operational information

Arm expects that the SMMLA (vector) instruction will deliver a peak integer multiply throughput that is at least as high as can be achieved using two SDOT (vector) instructions, with a goal that it should have significantly higher throughput.

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMOV

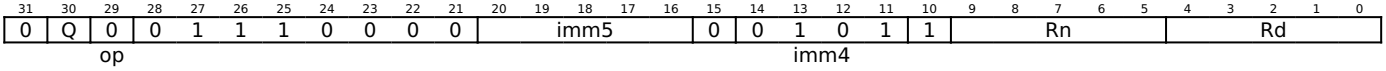
Signed move vector element to general-purpose register

This instruction reads the signed integer from the source SIMD&FP register, sign-extends it to form a 32-bit or 64-bit value, and writes the result to destination general-purpose register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding for the 32-bit variant

Applies when `(Q == 0)`

```
SMOV <Wd>, <Vn>.<Ts>[<index>]
```

Encoding for the 64-bit variant

Applies when `(Q == 1)`

```
SMOV <Xd>, <Vn>.<Ts>[<index>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if imm5 == 'xx000' then EndOfDecode(Decode_UNDEF); end;

let size : integer{} = LowestSetBitNZ(imm5[2:0]);

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << size;
let datasize : integer{} = 32 << UInt(Q);

if datasize <= esize then EndOfDecode(Decode_UNDEF); end;

let index : integer = UInt(imm5[4:size+1]);
let idxdsize : integer{} = 64 << UInt(imm5[4]);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ts> For the "32-bit" variant: is an element size specifier, encoded in “imm5”:

imm5	<Ts>
xxx00	RESERVED
xxxx1	B
xxx10	H

For the "64-bit" variant: is an element size specifier, encoded in “imm5”:

imm5	<Ts>
xx000	RESERVED
xxxx1	B
xxx10	H
xx100	S

<index> For the "32-bit" variant: is the element index encoded in “imm5”:

imm5	<index>
xxx00	RESERVED
xxxx1	UInt(imm5[4:1])
xxx10	UInt(imm5[4:2])

For the "64-bit" variant: is the element index encoded in “imm5”:

imm5	<index>
xx000	RESERVED
xxxx1	UInt(imm5[4:1])
xxx10	UInt(imm5[4:2])
xx100	UInt(imm5[4:3])

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Operation

```
if index == 0 then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;
let operand : bits(idxdsize) = V{idxdsize}(n);

X{datasize}(d) = SignExtend{datasize}(operand[index*:esize]);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMULL, SMULL2 (by element)

Signed multiply long (vector, by element)

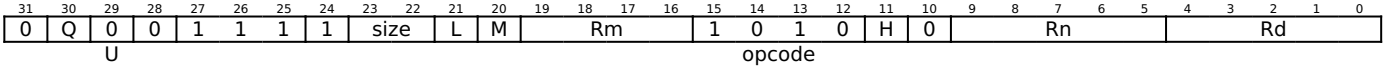
This instruction multiplies each vector element in the lower or upper half of the first source SIMD&FP register by the specified vector element of the second source SIMD&FP register, places the result in a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

The `SMULL` instruction extracts vector elements from the lower half of the first source register. The `SMULL2` instruction extracts vector elements from the upper half of the first source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
SMULL{2} <Vd>.<Ta>, <Vn>.<Tb>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(idxdsize) = V{}(m);
var result : bits(2*datasize);
var element1 : integer;
let element2 : integer = SInt(operand2[index*:esize]);
var product : bits(2*esize);

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    product = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = product;
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMULL, SMULL2 (vector)

Signed multiply long (vector)

This instruction multiplies corresponding signed integer values in the lower or upper half of the vectors of the two source SIMD&FP registers, places the results in a vector, and writes the vector to the destination SIMD&FP register.

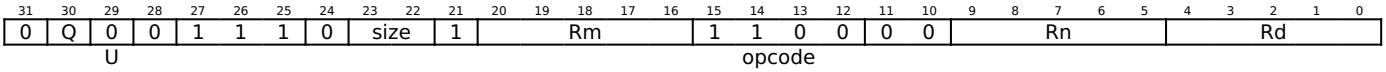
The destination vector elements are twice as long as the elements that are multiplied.

The `SMULL` instruction extracts each source vector from the lower half of each source register. The `SMULL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
SMULL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    result[e*:(2*esize)] = (element1 * element2)[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQABS

Signed saturating absolute value

This instruction reads each vector element from the source SIMD&FP register, puts the absolute value of the result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are signed integer values.

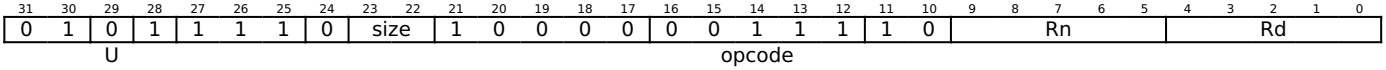
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

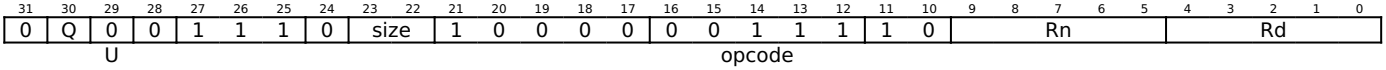
```
SQABS <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQABS <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element = SInt(operand[e*:esize]);
    element = Abs(element);
    (result[e*:esize], sat) = SignedSatQ{esize}(element);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQADD

Signed saturating add

This instruction adds the values of corresponding elements of the two source SIMD&FP registers, places the results into a vector, and writes the vector to the destination SIMD&FP register.

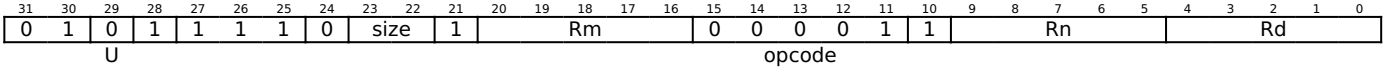
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

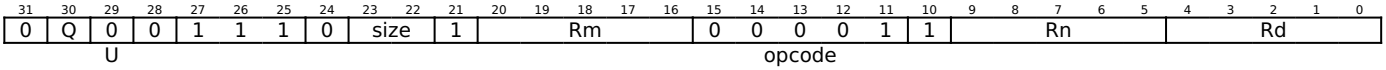
```
SQADD <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let unsigned : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    sum = element1 + element2;
    (result[e*:esize], sat) = SatQ{esize}(sum, unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMLAL, SQDMLAL2 (by element)

Signed saturating doubling multiply-add long (by element)

This instruction multiplies each vector element in the lower or upper half of the first source SIMD&FP register by the specified vector element of the second source SIMD&FP register, doubles the results, and accumulates the final results with the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

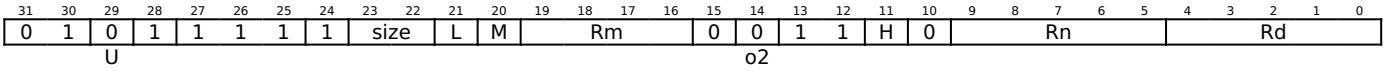
The `SQDMLAL` instruction extracts vector elements from the lower half of the first source register. The `SQDMLAL2` instruction extracts vector elements from the upper half of the first source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

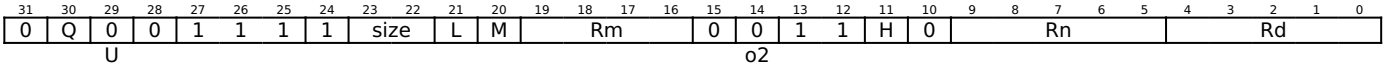
```
SQDMLAL <Va><d>, <Vb><n>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQDMLAL{2} <Vd>.<Ta>, <Vn>.<Tb>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Va> Is the destination width specifier, encoded in “size”:

size	<Va>
00	RESERVED
01	S
10	D
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vb> Is the source width specifier, encoded in “size”:

size	<Vb>
00	RESERVED
01	H
10	S
11	RESERVED

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var product : bits(2*esize);
var accum : integer;
var sat1 : boolean;
var sat2 : boolean;

element2 = SInt(operand2[index*:esize]);
for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    (product, sat1) = SignedSatQ{2*esize}(2 * element1 * element2);
    accum = SInt(operand3[e*:(2*esize)]) + SInt(product);
    (result[e*:(2*esize)], sat2) = SignedSatQ{2*esize}(accum);
    if sat1 || sat2 then FPSR().QC = '1'; end;
end;

V{2*datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMLAL, SQDMLAL2 (vector)

Signed saturating doubling multiply-add long

This instruction multiplies corresponding signed integer values in the lower or upper half of the vectors of the two source SIMD&FP registers, doubles the results, and accumulates the final results with the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

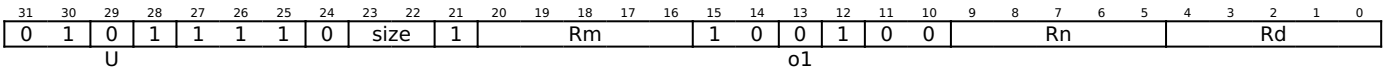
The `SQDMLAL` instruction extracts each source vector from the lower half of each source register. The `SQDMLAL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

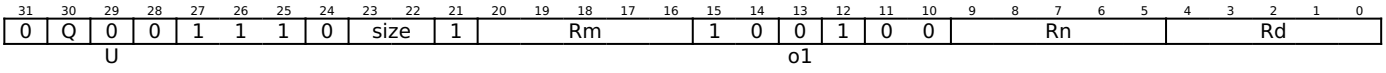
```
SQDMLAL <Va><d>, <Vb><n>, <Vb><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '00' || size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQDMLAL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '00' || size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Va> Is the destination width specifier, encoded in “size”:

size	<Va>
00	RESERVED
01	S
10	D
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vb> Is the source width specifier, encoded in “size”:

size	<Vb>
00	RESERVED
01	H
10	S
11	RESERVED

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var product : bits(2*esize);
var accum : integer;
var sat1 : boolean;
var sat2 : boolean;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    (product, sat1) = SignedSatQ{2*esize}(2 * element1 * element2);
    accum = SInt(operand3[e*:(2*esize)]) + SInt(product);
    (result[e*:(2*esize)], sat2) = SignedSatQ{2*esize}(accum);
    if sat1 || sat2 then FPSR().QC = '1'; end;
end;

V{2*datasize}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMLSL, SQDMLSL2 (by element)

Signed saturating doubling multiply-subtract long (by element)

This instruction multiplies each vector element in the lower or upper half of the first source SIMD&FP register by the specified vector element of the second source SIMD&FP register, doubles the results, and subtracts the final results from the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied. All the values in this instruction are signed integer values.

If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

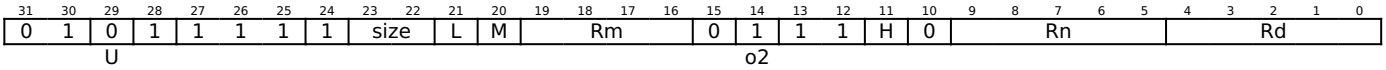
The `SQDMLSL` instruction extracts vector elements from the lower half of the first source register. The `SQDMLSL2` instruction extracts vector elements from the upper half of the first source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

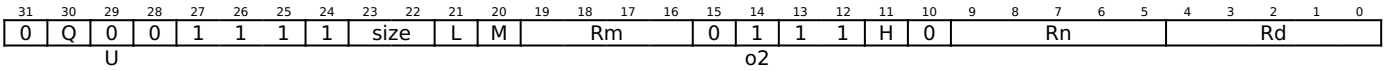
```
SQDMLSL <Va><d>, <Vb><n>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQDMLSL{2} <Vd>.<Ta>, <Vn>.<Tb>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Va> Is the destination width specifier, encoded in “size”:

size	<Va>
00	RESERVED
01	S
10	D
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vb> Is the source width specifier, encoded in “size”:

size	<Vb>
00	RESERVED
01	H
10	S
11	RESERVED

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var product : bits(2*esize);
var accum : integer;
var sat1 : boolean;
var sat2 : boolean;

element2 = SInt(operand2[index*:esize]);
for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    (product, sat1) = SignedSatQ{2*esize}(2 * element1 * element2);
    accum = SInt(operand3[e*:(2*esize)]) - SInt(product);
    (result[e*:(2*esize)], sat2) = SignedSatQ{2*esize}(accum);
    if sat1 || sat2 then FPSR().QC = '1'; end;
end;

V{2*datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMLSL, SQDMLSL2 (vector)

Signed saturating doubling multiply-subtract long

This instruction multiplies corresponding signed integer values in the lower or upper half of the vectors of the two source SIMD&FP registers, doubles the results, and subtracts the final results from the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

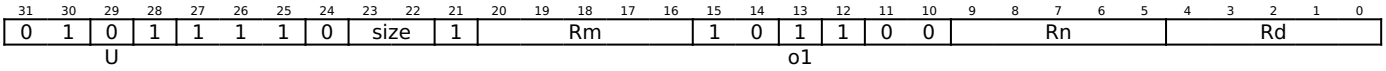
The `SQDMLSL` instruction extracts each source vector from the lower half of each source register. The `SQDMLSL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

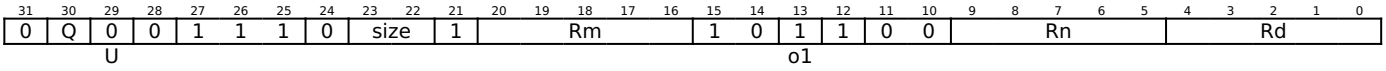
```
SQDMLSL <Va><d>, <Vb><n>, <Vb><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '00' || size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQDMLSL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '00' || size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Va> Is the destination width specifier, encoded in “size”:

size	<Va>
00	RESERVED
01	S
10	D
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vb> Is the source width specifier, encoded in “size”:

size	<Vb>
00	RESERVED
01	H
10	S
11	RESERVED

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var product : bits(2*esize);
var accum : integer;
var sat1 : boolean;
var sat2 : boolean;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    (product, sat1) = SignedSatQ{2*esize}(2 * element1 * element2);
    accum = SInt(operand3[e*:(2*esize)]) - SInt(product);
    (result[e*:(2*esize)], sat2) = SignedSatQ{2*esize}(accum);
    if sat1 || sat2 then FPSR().QC = '1'; end;
end;

V{2*datasize}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMULH (by element)

Signed saturating doubling multiply returning high half (by element)

This instruction multiplies each vector element in the first source SIMD&FP register by the specified vector element of the second source SIMD&FP register, doubles the results, places the most significant half of the final results into a vector, and writes the vector to the destination SIMD&FP register.

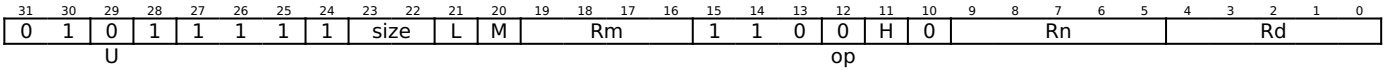
The results are truncated. For rounded results, see *SQRDMULH*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

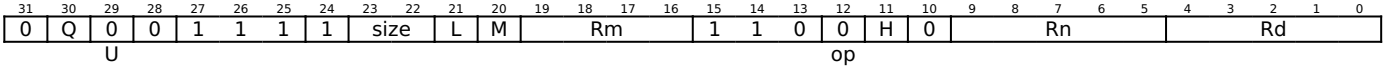
```
SQDMULH <V><d>, <V><n>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L);    Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;

let round : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQDMULH <Vd>.<T>, <Vn>.<T>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L);    Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let round : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var product : integer;
var sat : boolean;

element2 = SInt(operand2[index*:esize]);
for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    product = 2 * element1 * element2;
    product = RShr(product, esize, round);
    // The following only saturates if element1 and element2 equal -(2^(esize-1))
    (result[e*:esize], sat) = SignedSatQ{esize}(product);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMULH (vector)

Signed saturating doubling multiply returning high half

This instruction multiplies the values of corresponding elements of the two source SIMD&FP registers, doubles the results, places the most significant half of the final results into a vector, and writes the vector to the destination SIMD&FP register.

The results are truncated. For rounded results, see *SQRDMULH*.

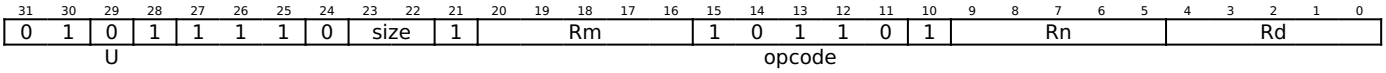
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

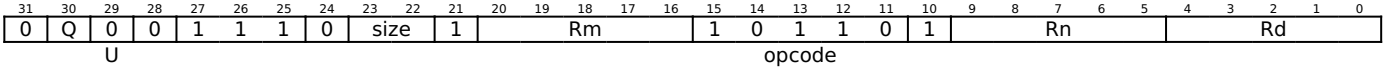
```
SQDMULH <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' || size == '00' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQDMULH <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' || size == '00' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var product : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    product = 2 * element1 * element2;
    product = RShr(product, esize, rounding);
    (result[e*:esize], sat) = SignedSatQ{esize}(product);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMULL, SQDMULL2 (by element)

Signed saturating doubling multiply long (by element)

This instruction multiplies each vector element in the lower or upper half of the first source SIMD&FP register by the specified vector element of the second source SIMD&FP register, doubles the results, places the final results in a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are signed integer values.

If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

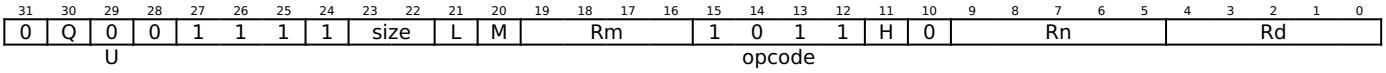
The `SQDMULL` instruction extracts the first source vector from the lower half of the first source register. The `SQDMULL2` instruction extracts the first source vector from the upper half of the first source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Vector and Scalar

Vector

(FEAT_AdvSIMD)



Encoding

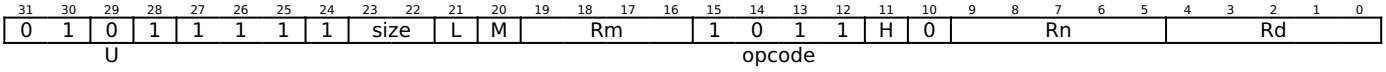
```
SQDMULL{2} <Vd>.<Ta>, <Vn>.<Tb>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Scalar

(FEAT_AdvSIMD)



Encoding

```
SQDMULL <Va><d>, <Vb><n>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;
```

Assembler Symbols

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

<Va> Is the destination width specifier, encoded in “size”:

size	<Va>
00	RESERVED
01	S
10	D
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vb> Is the source width specifier, encoded in “size”:

size	<Vb>
00	RESERVED
01	H
10	S
11	RESERVED

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(idxdsize) = V{}(m);
var result : bits(2*datasize);
var element1 : integer;
let element2 : integer = SInt(operand2[index*:esize]);
var product : bits(2*esize);
var sat : boolean;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    (product, sat) = SignedSatQ{2*esize}(2 * element1 * element2);
    result[e*:(2*esize)] = product;
    if sat then FPSR().QC = '1'; end;
end;

V{2*datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMULL, SQDMULL2 (vector)

Signed saturating doubling multiply long

This instruction multiplies corresponding vector elements in the lower or upper half of the two source SIMD&FP registers, doubles the result, places the final results in a vector, and writes the vector to the destination SIMD&FP register.

If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

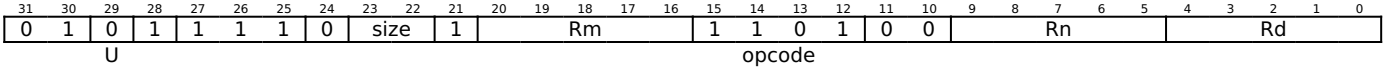
The `SQDMULL` instruction extracts each source vector from the lower half of each source register. The `SQDMULL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

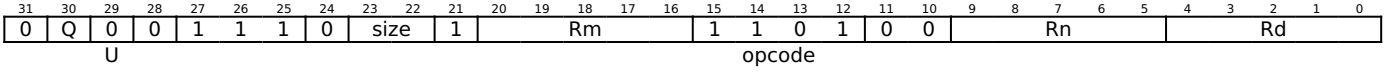
```
SQDMULL <Va><d>, <Vb><n>, <Vb><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '00' || size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQDMULL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '00' || size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Va> Is the destination width specifier, encoded in “size”:

size	<Va>
00	RESERVED
01	S
10	D
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vb> Is the source width specifier, encoded in “size”:

size	<Vb>
00	RESERVED
01	H
10	S
11	RESERVED

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var product : bits(2*esize);
var sat : boolean;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    (product, sat) = SignedSatQ{2*esize}(2 * element1 * element2);
    result[e*:(2*esize)] = product;
    if sat then FPSR().QC = '1'; end;
end;

V{2*datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQNEG

Signed saturating negate

This instruction reads each vector element from the source SIMD&FP register, negates each value, places the result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are signed integer values.

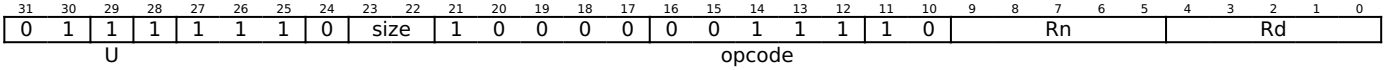
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

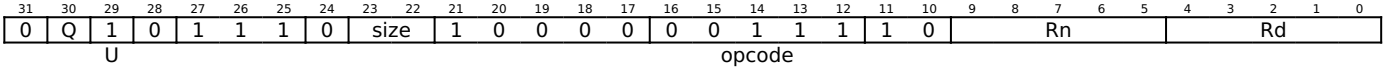
```
SQNEG <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQNEG <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element = SInt(operand[e*:esize]);
    element = -element;
    (result[e*:esize], sat) = SignedSatQ{esize}(element);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDMLAH (by element)

Signed saturating rounding doubling multiply accumulate returning high half (by element)

This instruction multiplies the vector elements of the first source SIMD&FP register with the value of a vector element of the second source SIMD&FP register without saturating the multiply results, doubles the results, and accumulates the most significant half of the final results with the vector elements of the destination SIMD&FP register. The results are rounded.

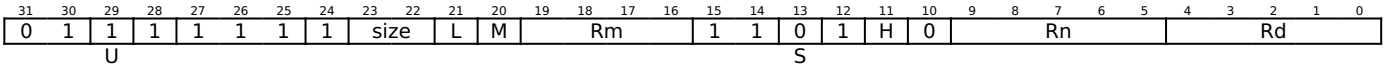
If any of the results overflow, they are saturated. The cumulative saturation bit, FPSR.QC, is set if saturation occurs.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_RDM)



Encoding

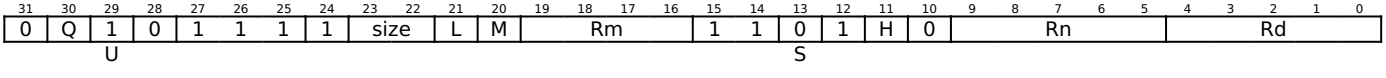
```
SQRDMLAH <V><d>, <V><n>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RDM) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;

case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;

let rounding : boolean = TRUE;
```

Vector

(FEAT_RDM)



Encoding

```
SQRDMLAH <Vd>.<T>, <Vn>.<T>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RDM) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;

case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let rounding : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var element3 : integer;
var accum : integer;
var sat : boolean;

element2 = SInt(operand2[index*:esize]);
for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element3 = SInt(operand3[e*:esize]);
    accum = (element3 << esize) + 2 * (element1 * element2);
    accum = RShr(accum, esize, rounding);
    (result[e*:esize], sat) = SignedSatQ{esize}(accum);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDMLAH (vector)

Signed saturating rounding doubling multiply accumulate returning high half (vector)

This instruction multiplies the vector elements of the first source SIMD&FP register with the corresponding vector elements of the second source SIMD&FP register without saturating the multiply results, doubles the results, and accumulates the most significant half of the final results with the vector elements of the destination SIMD&FP register. The results are rounded.

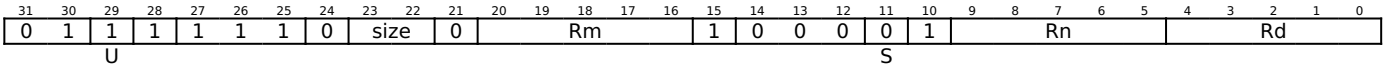
If any of the results overflow, they are saturated. The cumulative saturation bit, FPSR.QC, is set if saturation occurs.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_RDM)



Encoding

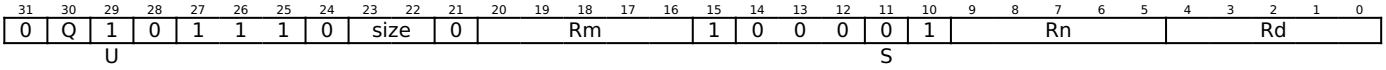
```
SQRDMLAH <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RDM) then EndOfDecode(Decode_UNDEF); end;
if size == '11' || size == '00' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : boolean = TRUE;
```

Vector

(FEAT_RDM)



Encoding

```
SQRDMLAH <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RDM) then EndOfDecode(Decode_UNDEF); end;
if size == '11' || size == '00' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var element3 : integer;
var accum : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    element3 = SInt(operand3[e*:esize]);
    accum = (element3 << esize) + 2 * (element1 * element2);
    accum = RShr(accum, esize, rounding);
    (result[e*:esize], sat) = SignedSatQ{esize}(accum);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDMLSH (by element)

Signed saturating rounding doubling multiply subtract returning high half (by element)

This instruction multiplies the vector elements of the first source SIMD&FP register with the value of a vector element of the second source SIMD&FP register without saturating the multiply results, doubles the results, and subtracts the most significant half of the final results from the vector elements of the destination SIMD&FP register. The results are rounded.

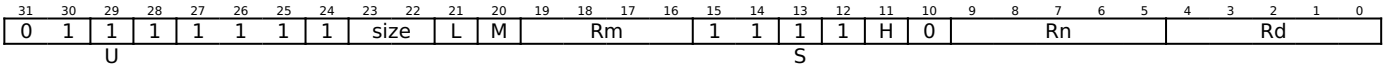
If any of the results overflow, they are saturated. The cumulative saturation bit, FPSR.QC, is set if saturation occurs.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_RDM)



Encoding

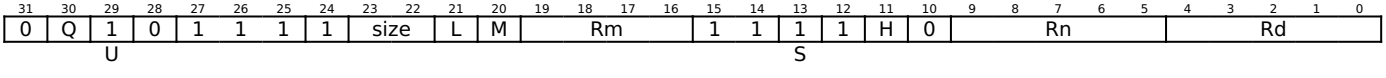
```
SQRDMLSH <V><d>, <V><n>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RDM) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;

case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;

let rounding : boolean = TRUE;
```

Vector

(FEAT_RDM)



Encoding

```
SQRDMLSH <Vd>.<T>, <Vn>.<T>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RDM) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;

case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let rounding : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var element3 : integer;
var accum : integer;
var sat : boolean;

element2 = SInt(operand2[index*:esize]);
for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element3 = SInt(operand3[e*:esize]);
    accum = (element3 << esize) - 2 * (element1 * element2);
    accum = RShr(accum, esize, rounding);
    (result[e*:esize], sat) = SignedSatQ{esize}(accum);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDMLSH (vector)

Signed saturating rounding doubling multiply subtract returning high half (vector)

This instruction multiplies the vector elements of the first source SIMD&FP register with the corresponding vector elements of the second source SIMD&FP register without saturating the multiply results, doubles the results, and subtracts the most significant half of the final results from the vector elements of the destination SIMD&FP register. The results are rounded.

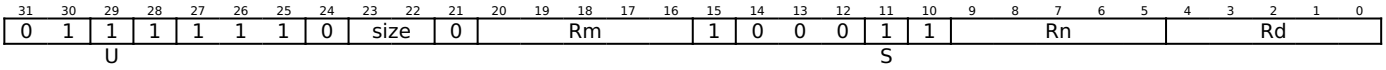
If any of the results overflow, they are saturated. The cumulative saturation bit, FPSR.QC, is set if saturation occurs.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_RDM)



Encoding

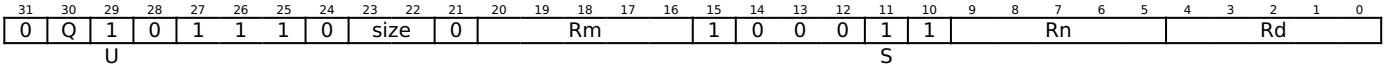
```
SQRDMLSH <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RDM) then EndOfDecode(Decode_UNDEF); end;
if size == '11' || size == '00' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : boolean = TRUE;
```

Vector

(FEAT_RDM)



Encoding

```
SQRDMLSH <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_RDM) then EndOfDecode(Decode_UNDEF); end;
if size == '11' || size == '00' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var element3 : integer;
var accum : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    element3 = SInt(operand3[e*:esize]);
    accum = (element3 << esize) - 2 * (element1 * element2);
    accum = RShr(accum, esize, rounding);
    (result[e*:esize], sat) = SignedSatQ{esize}(accum);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDMULH (by element)

Signed saturating rounding doubling multiply returning high half (by element)

This instruction multiplies each vector element in the first source SIMD&FP register by the specified vector element of the second source SIMD&FP register, doubles the results, places the most significant half of the final results into a vector, and writes the vector to the destination SIMD&FP register.

The results are rounded. For truncated results, see *SQDMULH*.

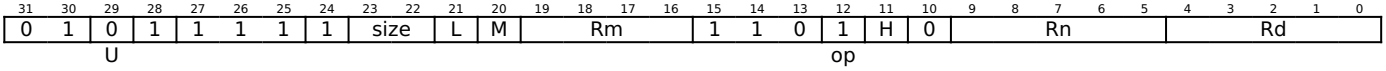
If any of the results overflows, they are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

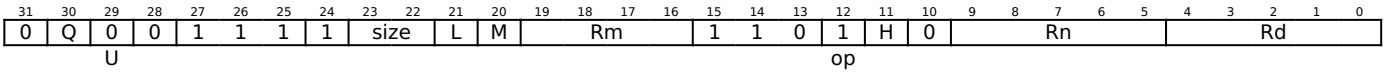
```
SQRDMULH <V><d>, <V><n>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L);    Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;

let round : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQRDMULH <Vd>.<T>, <Vn>.<T>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L);    Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let round : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(idxdsize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var product : integer;
var sat : boolean;

element2 = SInt(operand2[index*:esize]);
for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    product = 2 * element1 * element2;
    product = RShr(product, esize, round);
    // The following only saturates if element1 and element2 equal -(2^(esize-1))
    (result[e*:esize], sat) = SignedSatQ{esize}(product);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDMULH (vector)

Signed saturating rounding doubling multiply returning high half

This instruction multiplies the values of corresponding elements of the two source SIMD&FP registers, doubles the results, places the most significant half of the final results into a vector, and writes the vector to the destination SIMD&FP register.

The results are rounded. For truncated results, see *SQDMULH*.

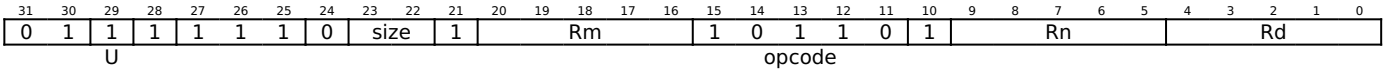
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

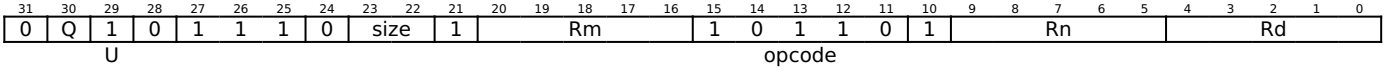
```
SQRDMULH <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' || size == '00' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let rounding : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQRDMULH <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' || size == '00' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let rounding : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var product : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    product = 2 * element1 * element2;
    product = RShr(product, esize, rounding);
    (result[e*:esize], sat) = SignedSatQ{esize}(product);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRSHL

Signed saturating rounding shift left (register)

This instruction takes each vector element in the first source SIMD&FP register, shifts it by a value from the least significant byte of the corresponding vector element of the second source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register.

If the shift value is positive, the operation is a left shift. Otherwise, it is a right shift. The results are rounded. For truncated results, see *SQSHL*.

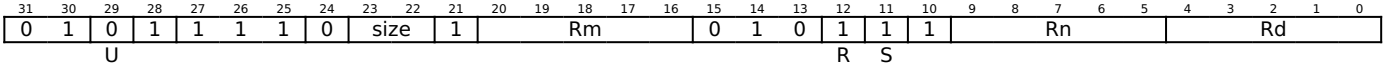
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

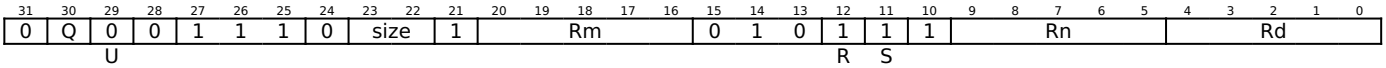
```
SQRSHL <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if S == '0' && size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let unsigned : boolean = FALSE;
let rounding : boolean = TRUE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQRSHL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let unsigned : boolean = FALSE;
let rounding : boolean = TRUE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    var element : integer = SInt(operand1[e*:esize]);
    var shift : integer = ShiftSat(SInt(operand2[e*:esize][7:0]), esize);
    if shift >= 0 then // left shift
        element = element << shift;
    else               // right shift
        shift = -shift;
        element = RShr(element, shift, rounding);
    end;

    (result[e*:esize], sat) = SatQ{esize}(element, unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRSHRN, SQRSHRN2

Signed saturating rounded shift right narrow (immediate)

This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, saturates each shifted result to a value that is half the original width, puts the final result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. All the values in this instruction are signed integer values. The destination vector elements are half as long as the source vector elements. The results are rounded. For truncated results, see *SQSHRN*.

The SQRSHRN instruction writes the vector to the lower half of the destination register and clears the upper half. The SQRSHRN2 instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

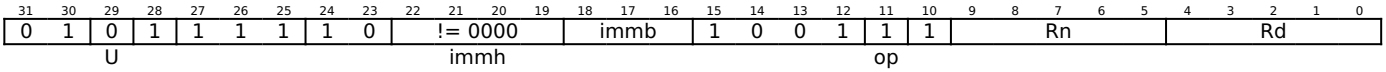
If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

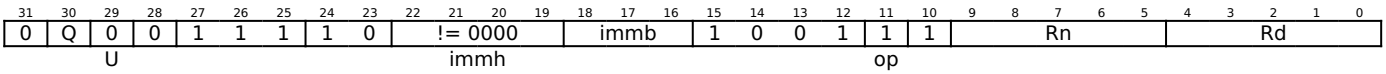
```
SQRSHRN <Vb><d>, <Va><n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = TRUE;
let unsigned : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQRSHRN{2} <Vd>.<Tb>, <Vn>.<Ta>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = TRUE;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Vb> Is the destination width specifier, encoded in “immh”:

immh	<Vb>
0001	B
001x	H
01xx	S
1xxx	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Va> Is the source width specifier, encoded in “immh”:

immh	<Va>
0001	H
001x	S
01xx	D
1xxx	RESERVED

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to the destination operand width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

For the "Vector" variant: is the right shift amount, in the range 1 to the destination element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize*2) = V{}(n);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    let opelt : bits(2*esize) = operand[e*:(2*esize)];
    if unsigned then
        let element : integer = RShr(UInt(opelt), shift, round);
        (result[e*:esize], sat) = UnsignedSatQ{esize}(element);
    else
        let element : integer = RShr(SInt(opelt), shift, round);
        (result[e*:esize], sat) = SignedSatQ{esize}(element);
    end;
    if sat then FPSR().QC = '1'; end;
end;

Vpart{datasize}(d, part) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRSHRUN, SQRSHRUN2

Signed saturating rounded shift right unsigned narrow (immediate)

This instruction reads each signed integer value in the vector of the source SIMD&FP register, right shifts each value by an immediate value, saturates the result to an unsigned integer value that is half the original width, places the final result into a vector, and writes the vector to the destination SIMD&FP register. The results are rounded. For truncated results, see *SQSHRUN*.

The SQRSHRUN instruction writes the vector to the lower half of the destination register and clears the upper half. The SQRSHRUN2 instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

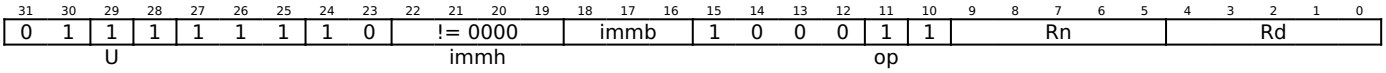
If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

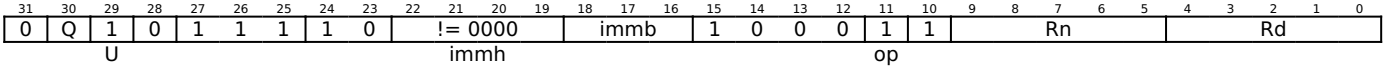
```
SQRSHRUN <Vb><d>, <Va><n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQRSHRUN{2} <Vd>.<Tb>, <Vn>.<Ta>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = TRUE;
```

Assembler Symbols

<Vb> Is the destination width specifier, encoded in “immh”:

immh	<Vb>
0001	B
001x	H
01xx	S
1xxx	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Va> Is the source width specifier, encoded in “immh”:

immh	<Va>
0001	H
001x	S
01xx	D
1xxx	RESERVED

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to the destination operand width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

For the "Vector" variant: is the right shift amount, in the range 1 to the destination element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize*2) = V{}(n);
var result : bits(datasize);
var element : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element = RShr(SInt(operand[e*:(2*esize)]), shift, round);
    (result[e*:esize], sat) = UnsignedSatQ{esize}(element);
    if sat then FPSR().QC = '1'; end;
end;

Vpart{datasize}(d, part) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQSHL (immediate)

Signed saturating shift left (immediate)

This instruction reads each vector element in the source SIMD&FP register, shifts each result by an immediate value, places the final result in a vector, and writes the vector to the destination SIMD&FP register. The results are truncated. For rounded results, see *UQRSHL*.

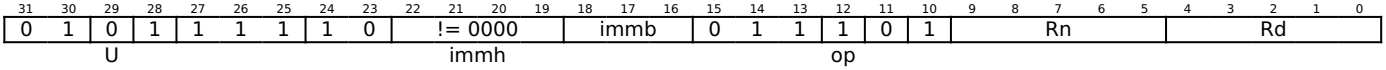
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

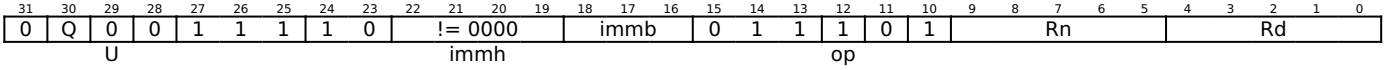
```
SQSHL <V><d>, <V><n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = esize;
let elements : integer = 1;
let shift : integer = UInt(immh::immb) - esize;

let src_unsigned : boolean = FALSE;
let dst_unsigned : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQSHL <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let shift : integer = UInt(immh::immb) - esize;

let src_unsigned : boolean = FALSE;
let dst_unsigned : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “immh”:

immh	<V>
0001	B
001x	H
01xx	S
1xxx	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the left shift amount, in the range 0 to the operand width in bits minus 1, encoded in “(immh :: immb)”:

immh	<shift>
0001	UInt(immh :: immb) - 8
001x	UInt(immh :: immb) - 16
01xx	UInt(immh :: immb) - 32
1xxx	UInt(immh :: immb) - 64

For the "Vector" variant: is the left shift amount, in the range 0 to the element width in bits minus 1, encoded in “(immh :: immb)”:

immh	<shift>
0001	UInt(immh :: immb) - 8
001x	UInt(immh :: immb) - 16
01xx	UInt(immh :: immb) - 32
1xxx	UInt(immh :: immb) - 64

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    let element : integer = if src_unsigned then UInt(opelt) else SInt(opelt);
    (result[e*:esize], sat) = SatQ{esize}(element << shift, dst_unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQSHL (register)

Signed saturating shift left (register)

This instruction takes each element in the vector of the first source SIMD&FP register, shifts each element by a value from the least significant byte of the corresponding element of the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register.

If the shift value is positive, the operation is a left shift. Otherwise, it is a right shift. The results are truncated. For rounded results, see *SQRSHL*.

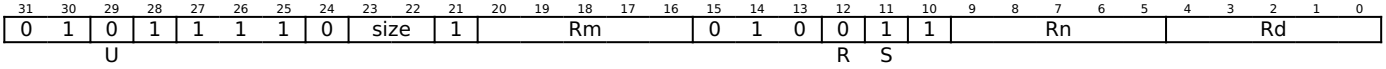
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

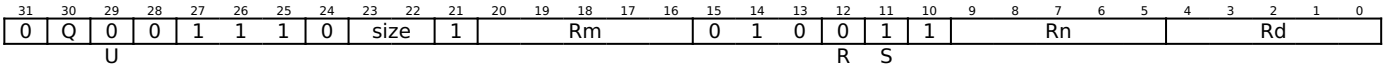
```
SQSHL <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if S == '0' && size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let unsigned : boolean = FALSE;
let rounding : boolean = FALSE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQSHL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let unsigned : boolean = FALSE;
let rounding : boolean = FALSE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    var element : integer = SInt(operand1[e*:esize]);
    var shift : integer = ShiftSat(SInt(operand2[e*:esize][7:0]), esize);
    if shift >= 0 then // left shift
        element = element << shift;
    else               // right shift
        shift = -shift;
        element = RShr(element, shift, rounding);
    end;

    (result[e*:esize], sat) = SatQ{esize}(element, unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQSHLU

Signed saturating shift left unsigned (immediate)

This instruction reads each signed integer value in the vector of the source SIMD&FP register, shifts each value by an immediate value, saturates the shifted result to an unsigned integer value, places the result in a vector, and writes the vector to the destination SIMD&FP register. The results are truncated. For rounded results, see *UQRSHL*.

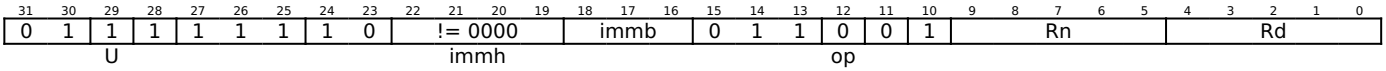
If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

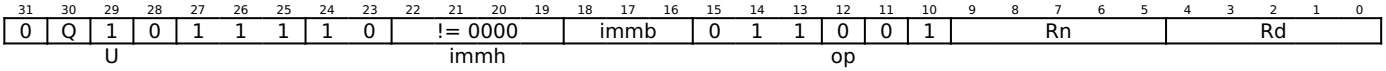
```
SQSHLU <V><d>, <V><n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = esize;
let elements : integer = 1;
let shift : integer = UInt(immh::immb) - esize;

let src_unsigned : boolean = FALSE;
let dst_unsigned : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQSHLU <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let shift : integer = UInt(immh::immb) - esize;

let src_unsigned : boolean = FALSE;
let dst_unsigned : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “immh”:

immh	<V>
0001	B
001x	H
01xx	S
1xxx	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the left shift amount, in the range 0 to the operand width in bits minus 1, encoded in “(immh :: immb)”:

immh	<shift>
0001	UInt(immh :: immb) - 8
001x	UInt(immh :: immb) - 16
01xx	UInt(immh :: immb) - 32
1xxx	UInt(immh :: immb) - 64

For the "Vector" variant: is the left shift amount, in the range 0 to the element width in bits minus 1, encoded in “(immh :: immb)”:

immh	<shift>
0001	UInt(immh :: immb) - 8
001x	UInt(immh :: immb) - 16
01xx	UInt(immh :: immb) - 32
1xxx	UInt(immh :: immb) - 64

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    let element : integer = if src_unsigned then UInt(opelt) else SInt(opelt);
    (result[e*:esize], sat) = SatQ{esize}(element << shift, dst_unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQSHRN, SQSHRN2

Signed saturating shift right narrow (immediate)

This instruction reads each vector element in the source SIMD&FP register, right shifts and truncates each result by an immediate value, saturates each shifted result to a value that is half the original width, puts the final result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. All the values in this instruction are signed integer values. The destination vector elements are half as long as the source vector elements. For rounded results, see *SQRSHRN*.

The SQSHRN instruction writes the vector to the lower half of the destination register and clears the upper half. The SQSHRN2 instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

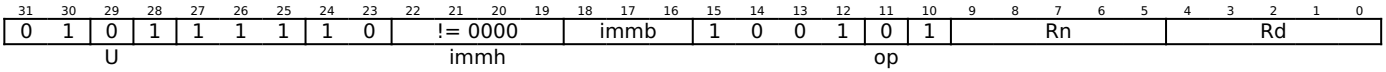
If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

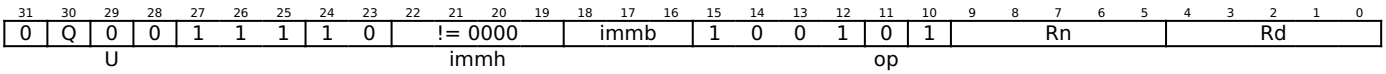
```
SQSHRN <Vb><d>, <Va><n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = FALSE;
let unsigned : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQSHRN{2} <Vd>.<Tb>, <Vn>.<Ta>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = FALSE;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Vb> Is the destination width specifier, encoded in “immh”:

immh	<Vb>
0001	B
001x	H
01xx	S
1xxx	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Va> Is the source width specifier, encoded in “immh”:

immh	<Va>
0001	H
001x	S
01xx	D
1xxx	RESERVED

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to the destination operand width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

For the "Vector" variant: is the right shift amount, in the range 1 to the destination element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize*2) = V{}(n);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    let opelt : bits(2*esize) = operand[e*:(2*esize)];
    if unsigned then
        let element : integer = RShr(UInt(opelt), shift, round);
        (result[e*:esize], sat) = UnsignedSatQ{esize}(element);
    else
        let element : integer = RShr(SInt(opelt), shift, round);
        (result[e*:esize], sat) = SignedSatQ{esize}(element);
    end;
    if sat then FPSR().QC = '1'; end;
end;

Vpart{datasize}(d, part) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQSHRUN, SQSHRUN2

Signed saturating shift right unsigned narrow (immediate)

This instruction reads each signed integer value in the vector of the source SIMD&FP register, right shifts each value by an immediate value, saturates the result to an unsigned integer value that is half the original width, places the final result into a vector, and writes the vector to the destination SIMD&FP register. The results are truncated. For rounded results, see *SQRSHRUN*.

The SQSHRUN instruction writes the vector to the lower half of the destination register and clears the upper half. The SQSHRUN2 instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

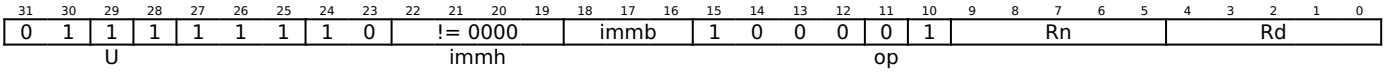
If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

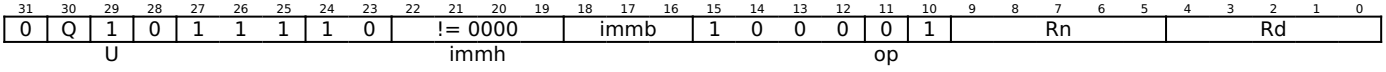
```
SQSHRUN <Vb><d>, <Va><n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQSHRUN{2} <Vd>.<Tb>, <Vn>.<Ta>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = FALSE;
```

Assembler Symbols

<Vb> Is the destination width specifier, encoded in “immh”:

immh	<Vb>
0001	B
001x	H
01xx	S
1xxx	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Va> Is the source width specifier, encoded in “immh”:

immh	<Va>
0001	H
001x	S
01xx	D
1xxx	RESERVED

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to the destination operand width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

For the "Vector" variant: is the right shift amount, in the range 1 to the destination element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize*2) = V{}(n);
var result : bits(datasize);
var element : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element = RShr(SInt(operand[e*:(2*esize)]), shift, round);
    (result[e*:esize], sat) = UnsignedSatQ{esize}(element);
    if sat then FPSR().QC = '1'; end;
end;

Vpart{datasize}(d, part) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQSUB

Signed saturating subtract

This instruction subtracts the element values of the second source SIMD&FP register from the corresponding element values of the first source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register.

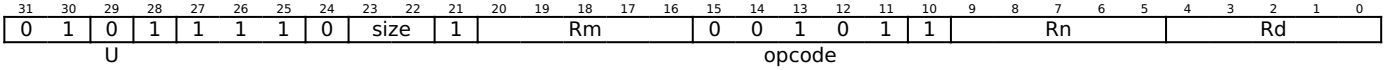
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

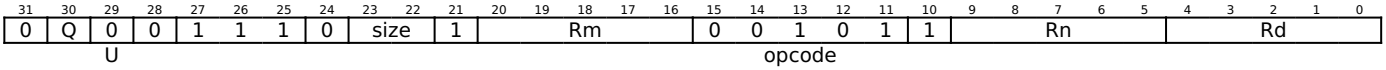
```
SQSUB <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let unsigned : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQSUB <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var diff : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    diff = element1 - element2;
    (result[e*:esize], sat) = SatQ{esize}(diff, unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQXTN, SQXTN2

Signed saturating extract narrow

This instruction reads each vector element from the source SIMD&FP register, saturates the value to half the original width, places the result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. The destination vector elements are half as long as the source vector elements. All the values in this instruction are signed integer values.

If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

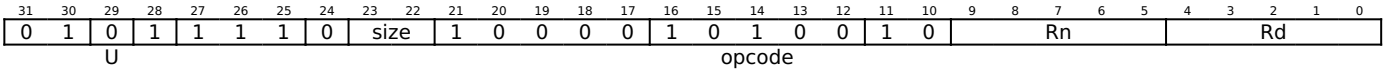
The `SQXTN` instruction writes the vector to the lower half of the destination register and clears the upper half. The `SQXTN2` instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

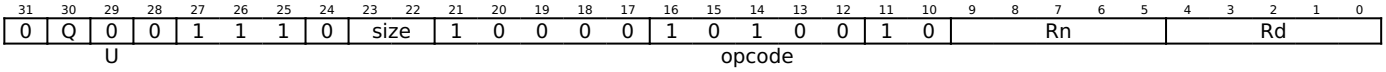
```
SQXTN <Vb><d>, <Va><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let part : integer = 0;
let elements : integer = 1;
let unsigned : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQXTN{2} <Vd>.<Tb>, <Vn>.<Ta>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Vb> Is the destination width specifier, encoded in “size”:

size	<Vb>
00	B
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Va> Is the source width specifier, encoded in “size”:

size	<Va>
00	H
01	S
10	D
11	RESERVED

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(2*datasize) = V{}(n);
var result : bits(datasize);
var element : bits(2*esize);
var sat : boolean;

for e = 0 to elements-1 do
    element = operand[e*:(2*esize)];
    (result[e*:esize], sat) = SatQ{esize}(SInt(element), unsigned);
    if sat then FPSR().QC = '1'; end;
end;

Vpart{datasize}(d, part) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQXTUN, SQXTUN2

Signed saturating extract unsigned narrow

This instruction reads each signed integer value in the vector of the source SIMD&FP register, saturates the value to an unsigned integer value that is half the original width, places the result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. The destination vector elements are half as long as the source vector elements.

If saturation occurs, the cumulative saturation bit FPSR.QC is set.

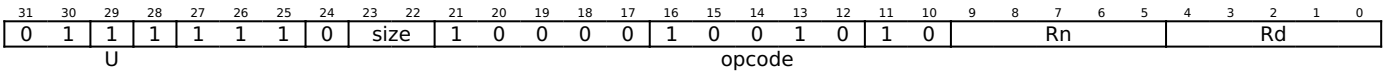
The `SQXTUN` instruction writes the vector to the lower half of the destination register and clears the upper half. The `SQXTUN2` instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

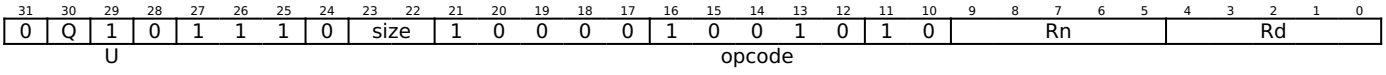
```
SQXTUN <Vb><d>, <Va><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let part : integer = 0;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SQXTUN{2} <Vd>.<Tb>, <Vn>.<Ta>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vb> Is the destination width specifier, encoded in “size”:

size	<Vb>
00	B
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Va> Is the source width specifier, encoded in “size”:

size	<Va>
00	H
01	S
10	D
11	RESERVED

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(2*datasize) = V{}(n);
var result : bits(datasize);
var element : bits(2*esize);
var sat : boolean;

for e = 0 to elements-1 do
    element = operand[e*:(2*esize)];
    (result[e*:esize], sat) = UnsignedSatQ{esize}(SInt(element));
    if sat then FPSR().QC = '1'; end;
end;

Vpart{datasize}(d, part) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SRHADD

Signed rounding halving add

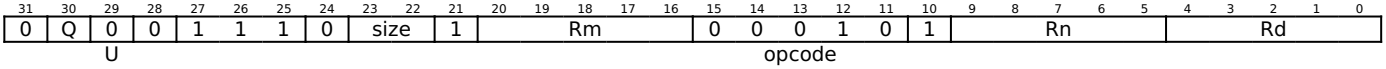
This instruction adds corresponding signed integer values from the two source SIMD&FP registers, shifts each result right one bit, places the results into a vector, and writes the vector to the destination SIMD&FP register.

The results are rounded. For truncated results, see *SHADD*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
SRHADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    sum = (element1 + element2 + 1) >> 1;
    result[e*:esize] = sum[esize-1:0];
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

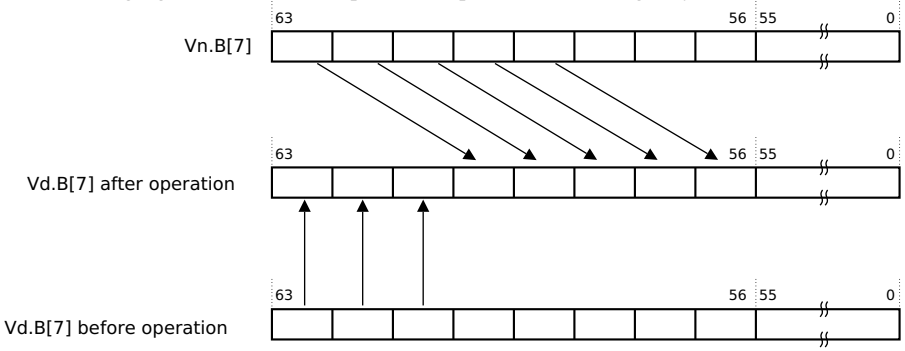
Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SRI

Shift right and insert (immediate)

This instruction reads each vector element in the source SIMD&FP register, right shifts each vector element by an immediate value, and inserts the result into the corresponding vector element in the destination SIMD&FP register such that the new zero bits created by the shift are not inserted but retain their existing value. Bits shifted out of the right of each vector element of the source register are lost.

The following figure shows an example of the operation of shift right by 3 for an 8-bit vector element.



Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	1	1	1	1	0	1	x	x	x	immb			0	1	0	0	0	1	Rn					Rd				
		U								immh						opcode															

Encoding

```
SRI D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;
let shift : integer = (esize * 2) - UInt(immh::immb);
```

Vector

(FEAT_AdvSIMD)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	1	0	1	1	1	1	0	!= 0000				immb			0	1	0	0	0	1	Rn					Rd				
		U							immh							opcode															

Encoding

```
SRI <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let shift : integer = (esize * 2) - UInt(immh::immb);
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the SIMD&FP source register, encoded in the "Rn" field.
<shift>	For the "Scalar" variant: is the right shift amount, in the range 1 to 64, encoded as 128 - UInt("immh:immb").

For the "Vector" variant: is the right shift amount, in the range 1 to the element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
let operand2 : bits(datasize) = V{}(d);
let mask : bits(esize) = LSR(Ones{esize}, shift);
var result : bits(datasize);
var shifted : bits(esize);

for e = 0 to elements-1 do
    shifted = LSR(operand[e*:esize], shift);
    result[e*:esize] = (operand2[e*:esize] AND NOT(mask)) OR shifted;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SRSHL

Signed rounding shift left (register)

This instruction takes each signed integer value in the vector of the first source SIMD&FP register, shifts it by a value from the least significant byte of the corresponding element of the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register.

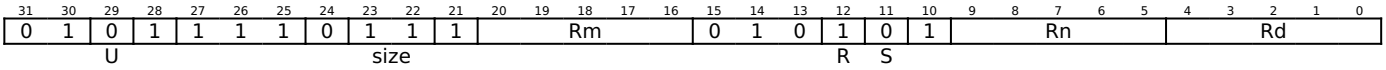
If the shift value is positive, the operation is a left shift. If the shift value is negative, it is a rounding right shift. For a truncating shift, see *SSHL*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

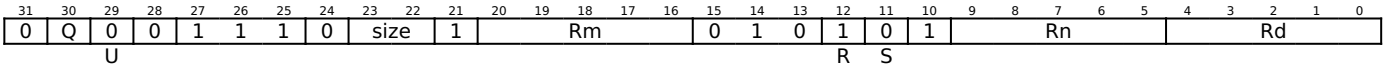
```
SRSHL D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if S == '0' && size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let rounding : boolean = TRUE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SRSHL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let rounding : boolean = TRUE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the first SIMD&FP source register, encoded in the "Rn" field.
<m>	Is the number of the second SIMD&FP source register, encoded in the "Rm" field.
<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

for e = 0 to elements-1 do
    var element : integer = SInt(operand1[e*:esize]);
    var shift : integer = ShiftSat(SInt(operand2[e*:esize][7:0]), esize);
    if shift >= 0 then // left shift
        element = element << shift;
    else               // right shift
        shift = -shift;
        element = RShr(element, shift, rounding);
    end;

    result[e*:esize] = element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SRSHR

Signed rounding shift right (immediate)

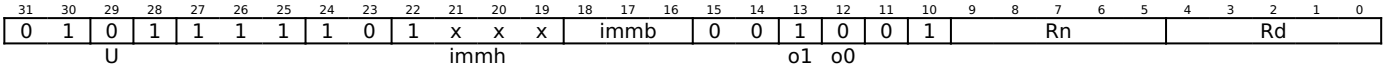
This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, places the final result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are signed integer values. The results are rounded. For truncated results, see *SSHR*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

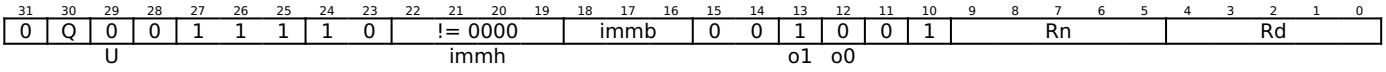
```
SRSHR D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
let round : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SRSHR <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
let round : boolean = TRUE;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to 64, encoded as 128 - UInt("immh:immb").

For the "Vector" variant: is the right shift amount, in the range 1 to the element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    if unsigned then
        element = RShr(UInt(opelt), shift, round);
    else
        element = RShr(SInt(opelt), shift, round);
    end;
    result[e*:esize] = element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SRSRA

Signed rounding shift right and accumulate (immediate)

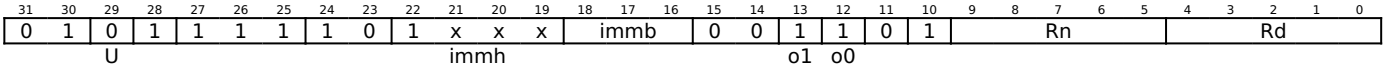
This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, and accumulates the final results with the vector elements of the destination SIMD&FP register. All the values in this instruction are signed integer values. The results are rounded. For truncated results, see *SSRA*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

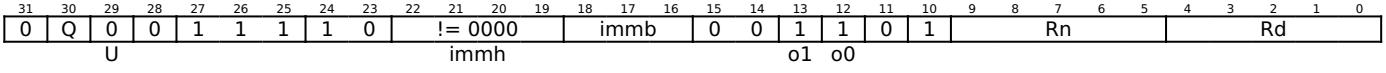
```
SRSRA D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
let round : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SRSRA <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
let round : boolean = TRUE;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to 64, encoded as 128 - UInt("immh:immb").

For the "Vector" variant: is the right shift amount, in the range 1 to the element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
let operand2 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    if unsigned then
        element = RShr(UInt(opelt), shift, round);
    else
        element = RShr(SInt(opelt), shift, round);
    end;
    result[e*:esize] = operand2[e*:esize] + element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SSHL

Signed shift left (register)

This instruction takes each signed integer value in the vector of the first source SIMD&FP register, shifts each value by a value from the least significant byte of the corresponding element of the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register.

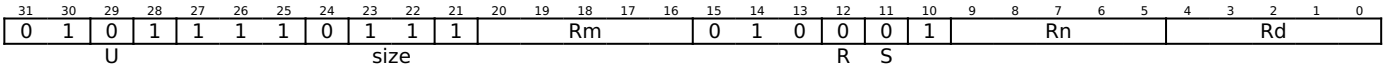
If the shift value is positive, the operation is a left shift. If the shift value is negative, it is a truncating right shift. For a rounding shift, see *SRSHL*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

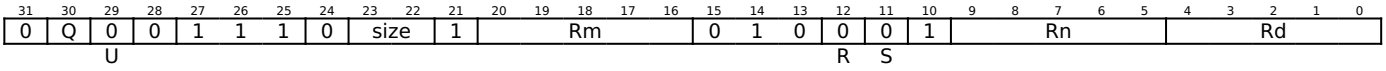
```
SSHL D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if S == '0' && size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let rounding : boolean = FALSE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SSHL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let rounding : boolean = FALSE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the first SIMD&FP source register, encoded in the "Rn" field.
<m>	Is the number of the second SIMD&FP source register, encoded in the "Rm" field.
<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

for e = 0 to elements-1 do
    var element : integer = SInt(operand1[e*:esize]);
    var shift : integer = ShiftSat(SInt(operand2[e*:esize][7:0]), esize);
    if shift >= 0 then // left shift
        element = element << shift;
    else               // right shift
        shift = -shift;
        element = RShr(element, shift, rounding);
    end;

    result[e*:esize] = element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SSHLL, SSHLL2

Signed shift left long (immediate)

This instruction reads each vector element from the source SIMD&FP register, left shifts each vector element by the specified shift amount, places the result into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements. All the values in this instruction are signed integer values.

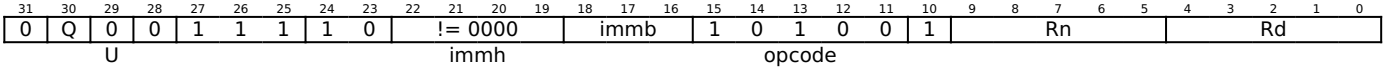
The `SSHLL` instruction extracts vector elements from the lower half of the source register. The `SSHLL2` instruction extracts vector elements from the upper half of the source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This instruction is used by the alias SXTL, SXTL2.

Vector

(FEAT_AdvSIMD)



Encoding

```
SSHLL{2} <Vd>.<Ta>, <Vn>.<Tb>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = UInt(immh::immb) - esize;
let unsigned : boolean = FALSE;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

<Vn> — Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

<shift> Is the left shift amount, in the range 0 to the source element width in bits minus 1, encoded in “(immh :: immb)”:

immh	<shift>
0001	UInt(immh :: immb) - 8
001x	UInt(immh :: immb) - 16
01xx	UInt(immh :: immb) - 32
1xxx	RESERVED

Alias Conditions

Alias	Is preferred when
SXTL, SXTL2	immb == '000' && BitCount(immh) == 1

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = Vpart{}(n, part);
var result : bits(datasize*2);

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
    result[e*:(2*esize)] = (element << shift)[2*esize-1:0];
end;

V{datasize*2}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SSHR

Signed shift right (immediate)

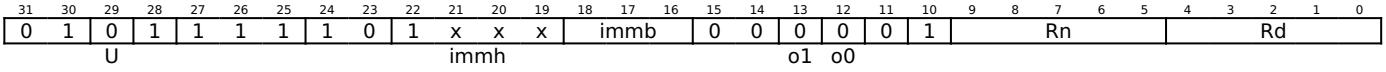
This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, places the final result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are signed integer values. The results are truncated. For rounded results, see *SRSHR*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

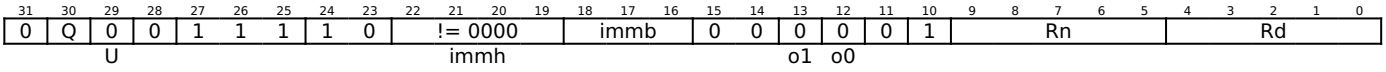
```
SSHR D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
let round : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SSHR <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
let round : boolean = FALSE;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to 64, encoded as 128 - UInt("immh:immb").

For the "Vector" variant: is the right shift amount, in the range 1 to the element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    if unsigned then
        element = RShr(UInt(opelt), shift, round);
    else
        element = RShr(SInt(opelt), shift, round);
    end;
    result[e*:esize] = element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SSRA

Signed shift right and accumulate (immediate)

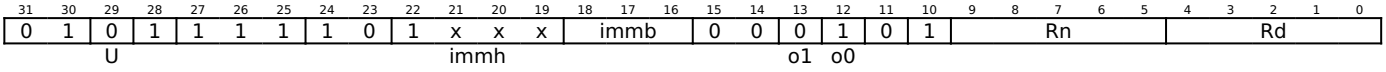
This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, and accumulates the final results with the vector elements of the destination SIMD&FP register. All the values in this instruction are signed integer values. The results are truncated. For rounded results, see *SRSRA*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

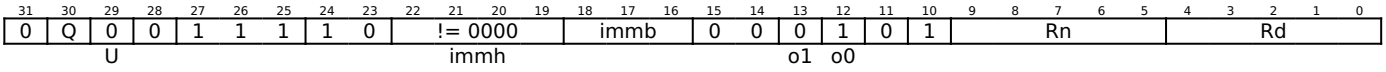
```
SSRA D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
let round : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SSRA <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = FALSE;
let round : boolean = FALSE;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to 64, encoded as 128 - UInt("immh:immb").

For the "Vector" variant: is the right shift amount, in the range 1 to the element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
let operand2 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    if unsigned then
        element = RShr(UInt(opelt), shift, round);
    else
        element = RShr(SInt(opelt), shift, round);
    end;
    result[e*:esize] = operand2[e*:esize] + element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SSUBL, SSUBL2

Signed subtract long

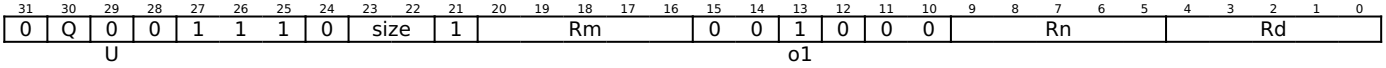
This instruction subtracts each vector element in the lower or upper half of the second source SIMD&FP register from the corresponding vector element of the first source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are signed integer values. The destination vector elements are twice as long as the source vector elements.

The `SSUBL` instruction extracts each source vector from the lower half of each source register. The `SSUBL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
SSUBL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:esize]);
    element2 = SInt(operand2[e*:esize]);
    sum = element1 - element2;
    result[e*:(2*esize)] = sum[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SSUBW, SSUBW2

Signed subtract wide

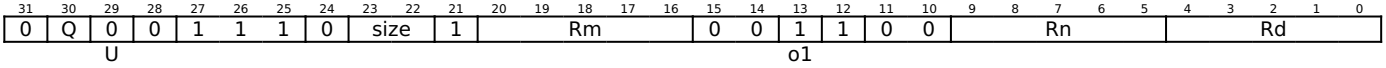
This instruction subtracts each vector element in the lower or upper half of the second source SIMD&FP register from the corresponding vector element in the first source SIMD&FP register, places the result in a vector, and writes the vector to the SIMD&FP destination register. All the values in this instruction are signed integer values.

The `SSUBW` instruction extracts the second source vector from the lower half of the second source register. The `SSUBW2` instruction extracts the second source vector from the upper half of the second source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
SSUBW{2} <Vd>.<Ta>, <Vn>.<Ta>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(2*datasize) = V{}(n);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = SInt(operand1[e*:(2*esize)]);
    element2 = SInt(operand2[e*:esize]);
    sum = element1 - element2;
    result[e*:(2*esize)] = sum[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1 (multiple structures)

Store multiple single-element structures from one, two, three, or four registers

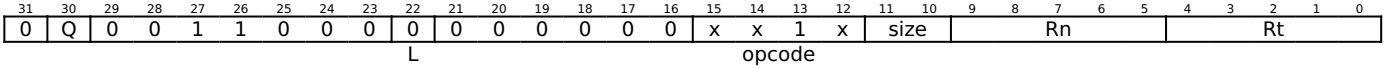
This instruction stores elements to memory from one, two, three, or four SIMD&FP registers, without interleaving. Every element of each register is stored.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding for the One register variant

Applies when (opcode == 0111)

```
ST1 { <Vt>.<T> }, [<Xn|SP>]
```

Encoding for the Two registers variant

Applies when (opcode == 1010)

```
ST1 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>]
```

Encoding for the Three registers variant

Applies when (opcode == 0110)

```
ST1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>]
```

Encoding for the Four registers variant

Applies when (opcode == 0010)

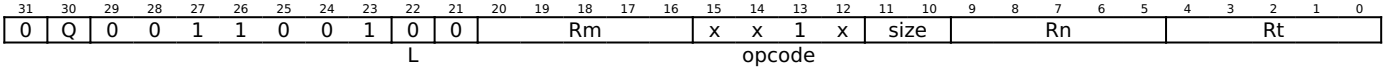
```
ST1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the One register, immediate offset variant

Applies when (Rm == 11111 && opcode == 0111)

```
ST1 { <Vt>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the One register, register offset variant

Applies when (Rm != 11111 && opcode == 0111)

```
ST1 { <Vt>.<T> }, [<Xn|SP>], <Xm>
```

Encoding for the Two registers, immediate offset variant

Applies when (Rm == 11111 && opcode == 1010)

```
ST1 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Two registers, register offset variant

Applies when (Rm != 11111 && opcode == 1010)

```
ST1 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>], <Xm>
```

Encoding for the Three registers, immediate offset variant

Applies when `(Rm == 11111 && opcode == 0110)`

```
ST1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Three registers, register offset variant

Applies when `(Rm != 11111 && opcode == 0110)`

```
ST1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>], <Xm>
```

Encoding for the Four registers, immediate offset variant

Applies when `(Rm == 11111 && opcode == 0010)`

```
ST1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Four registers, register offset variant

Applies when `(Rm != 11111 && opcode == 0010)`

```
ST1 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	1D
11	1	2D

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Vt3> Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.

<Vt4> Is the name of the fourth SIMD&FP register to be transferred, encoded as "Rt" plus 3 modulo 32.

<imm> For the "One register, immediate offset" variant: is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#8
1	#16

For the "Two registers, immediate offset" variant: is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#16
1	#32

For the "Three registers, immediate offset" variant: is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#24
1	#48

For the "Four registers, immediate offset" variant: is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#32
1	#64

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(size);
let elements : integer = datasize DIV esize;

var rpt : integer;                // number of iterations
let selem : integer = 1;          // structure elements

case opcode of
    when '0010' => rpt = 4;    // LD/ST1 (4 registers)
    when '0110' => rpt = 3;    // LD/ST1 (3 registers)
    when '1010' => rpt = 2;    // LD/ST1 (2 registers)
    when '0111' => rpt = 1;    // LD/ST1 (1 register)
    otherwise => EndOfDecode(Decode_UNDEF);
end;
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(datasize);
var tt : integer;
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

for r = 0 to rpt - 1 do
    for e = 0 to elements - 1 do
        tt = (t + r) MOD 32;
        for s = 0 to selem - 1 do
            rval = V{datasize}(tt);
            eaddr = AddressIncrement(address, offs, accdesc);
            Mem{esize}(eaddr, accdesc) = rval[e *: esize];
            offs = offs + ebytes;
            tt = (tt + 1) MOD 32;
        end;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

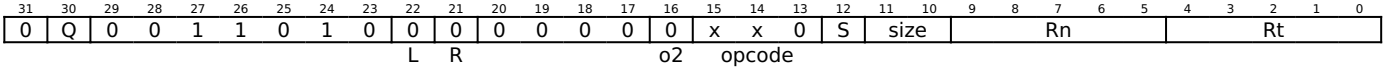
ST1 (single structure)

Store a single-element structure from one lane of one register

This instruction stores the specified element of a SIMD&FP register to memory.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding for the 8-bit variant

Applies when (opcode == 000)

```
ST1 { <Vt>.B }[<index>], [<Xn|SP>]
```

Encoding for the 16-bit variant

Applies when (opcode == 010 && size == x0)

```
ST1 { <Vt>.H }[<index>], [<Xn|SP>]
```

Encoding for the 32-bit variant

Applies when (opcode == 100 && size == 00)

```
ST1 { <Vt>.S }[<index>], [<Xn|SP>]
```

Encoding for the 64-bit variant

Applies when (opcode == 100 && S == 0 && size == 01)

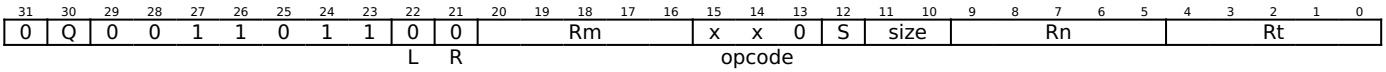
```
ST1 { <Vt>.D }[<index>], [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the 8-bit, immediate offset variant

Applies when (Rm == 11111 && opcode == 000)

```
ST1 { <Vt>.B }[<index>], [<Xn|SP>], #1
```

Encoding for the 8-bit, register offset variant

Applies when (Rm != 11111 && opcode == 000)

```
ST1 { <Vt>.B }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 16-bit, immediate offset variant

Applies when (Rm == 11111 && opcode == 010 && size == x0)

```
ST1 { <Vt>.H }[<index>], [<Xn|SP>], #2
```

Encoding for the 16-bit, register offset variant

Applies when (Rm != 11111 && opcode == 010 && size == x0)

```
ST1 { <Vt>.H }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 32-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 100 && size == 00)`

```
ST1 { <Vt>.S }[<index>], [<Xn|SP>], #4
```

Encoding for the 32-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 100 && size == 00)`

```
ST1 { <Vt>.S }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 64-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 100 && S == 0 && size == 01)`

```
ST1 { <Vt>.D }[<index>], [<Xn|SP>], #8
```

Encoding for the 64-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 100 && S == 0 && size == 01)`

```
ST1 { <Vt>.D }[<index>], [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt>	Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.
<index>	For the "8-bit", "8-bit, immediate offset", and "8-bit, register offset" variants: is the element index, encoded in "Q:S:size". For the "16-bit", "16-bit, immediate offset", and "16-bit, register offset" variants: is the element index, encoded in "Q:S:size<1>". For the "32-bit", "32-bit, immediate offset", and "32-bit, register offset" variants: is the element index, encoded in "Q:S". For the "64-bit", "64-bit, immediate offset", and "64-bit, register offset" variants: is the element index, encoded in "Q".
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        // extract from one lane of 128-bit register
        Mem{esize}(eaddr, accdesc) = rval[index *: esize];
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2 (multiple structures)

Store multiple 2-element structures from two registers

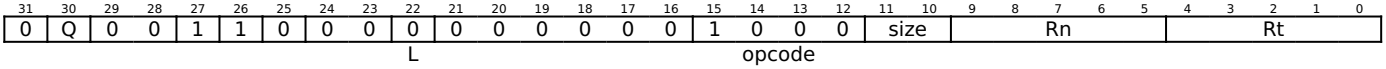
This instruction stores multiple 2-element structures from two SIMD&FP registers to memory, with interleaving. Every element of each register is stored.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding

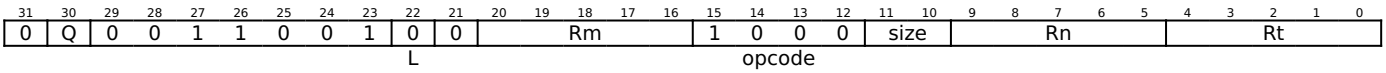
```
ST2 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the Immediate offset variant

Applies when `(Rm == 11111)`

```
ST2 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Register offset variant

Applies when `(Rm != 11111)`

```
ST2 { <Vt>.<T>, <Vt2>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#16
1	#32

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(size);
let elements : integer = datasize DIV esize;

let rpt : integer = 1;                  // number of iterations
let selem : integer = 2;            // structure elements

// .1D format only permitted with LD1 & ST1
if size::Q == '110' && selem != 1 then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(datasize);
var tt : integer;
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

for r = 0 to rpt - 1 do
    for e = 0 to elements - 1 do
        tt = (t + r) MOD 32;
        for s = 0 to selem - 1 do
            rval = V{datasize}(tt);
            eaddr = AddressIncrement(address, offs, accdesc);
            Mem{esize}(eaddr, accdesc) = rval[e *: esize];
            offs = offs + ebytes;
            tt = (tt + 1) MOD 32;
        end;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

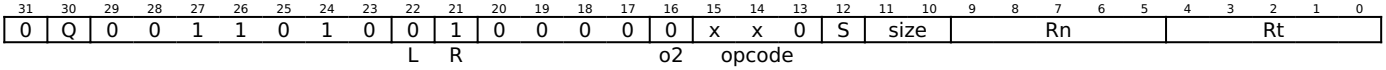
ST2 (single structure)

Store single 2-element structure from one lane of two registers

This instruction stores a 2-element structure to memory from corresponding elements of two SIMD&FP registers.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding for the 8-bit variant

Applies when (opcode == 000)

```
ST2 { <Vt>.B, <Vt2>.B }[<index>], [<Xn|SP>]
```

Encoding for the 16-bit variant

Applies when (opcode == 010 && size == x0)

```
ST2 { <Vt>.H, <Vt2>.H }[<index>], [<Xn|SP>]
```

Encoding for the 32-bit variant

Applies when (opcode == 100 && size == 00)

```
ST2 { <Vt>.S, <Vt2>.S }[<index>], [<Xn|SP>]
```

Encoding for the 64-bit variant

Applies when (opcode == 100 && S == 0 && size == 01)

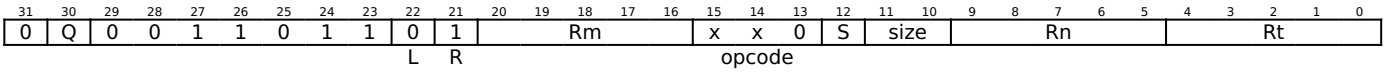
```
ST2 { <Vt>.D, <Vt2>.D }[<index>], [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the 8-bit, immediate offset variant

Applies when (Rm == 11111 && opcode == 000)

```
ST2 { <Vt>.B, <Vt2>.B }[<index>], [<Xn|SP>], #2
```

Encoding for the 8-bit, register offset variant

Applies when (Rm != 11111 && opcode == 000)

```
ST2 { <Vt>.B, <Vt2>.B }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 16-bit, immediate offset variant

Applies when (Rm == 11111 && opcode == 010 && size == x0)

```
ST2 { <Vt>.H, <Vt2>.H }[<index>], [<Xn|SP>], #4
```

Encoding for the 16-bit, register offset variant

Applies when (Rm != 11111 && opcode == 010 && size == x0)

```
ST2 { <Vt>.H, <Vt2>.H }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 32-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 100 && size == 00)`

```
ST2 { <Vt>.S, <Vt2>.S }[<index>], [<Xn|SP>], #8
```

Encoding for the 32-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 100 && size == 00)`

```
ST2 { <Vt>.S, <Vt2>.S }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 64-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 100 && S == 0 && size == 01)`

```
ST2 { <Vt>.D, <Vt2>.D }[<index>], [<Xn|SP>], #16
```

Encoding for the 64-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 100 && S == 0 && size == 01)`

```
ST2 { <Vt>.D, <Vt2>.D }[<index>], [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt>	Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.
<Vt2>	Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.
<index>	For the "8-bit", "8-bit, immediate offset", and "8-bit, register offset" variants: is the element index, encoded in "Q:S:size". For the "16-bit", "16-bit, immediate offset", and "16-bit, register offset" variants: is the element index, encoded in "Q:S:size<1>". For the "32-bit", "32-bit, immediate offset", and "32-bit, register offset" variants: is the element index, encoded in "Q:S". For the "64-bit", "64-bit, immediate offset", and "64-bit, register offset" variants: is the element index, encoded in "Q".
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        // extract from one lane of 128-bit register
        Mem{esize}(eaddr, accdesc) = rval[index *: esize];
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3 (multiple structures)

Store multiple 3-element structures from three registers

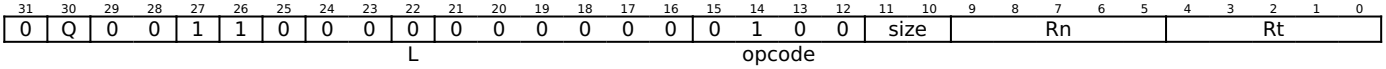
This instruction stores multiple 3-element structures to memory from three SIMD&FP registers, with interleaving. Every element of each register is stored.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding

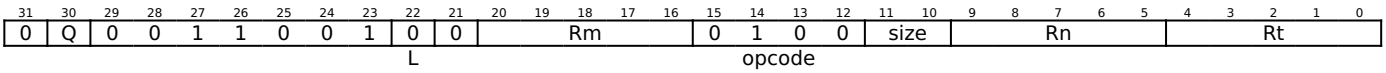
```
ST3 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the Immediate offset variant

Applies when `(Rm == 11111)`

```
ST3 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Register offset variant

Applies when `(Rm != 11111)`

```
ST3 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Vt3> Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#24
1	#48

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(size);
let elements : integer = datasize DIV esize;

let rpt : integer = 1;                  // number of iterations
let selem : integer = 3;            // structure elements

// .1D format only permitted with LD1 & ST1
if size::Q == '110' && selem != 1 then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(datasize);
var tt : integer;
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

for r = 0 to rpt - 1 do
    for e = 0 to elements - 1 do
        tt = (t + r) MOD 32;
        for s = 0 to selem - 1 do
            rval = V{datasize}(tt);
            eaddr = AddressIncrement(address, offs, accdesc);
            Mem{esize}(eaddr, accdesc) = rval[e *: esize];
            offs = offs + ebytes;
            tt = (tt + 1) MOD 32;
        end;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

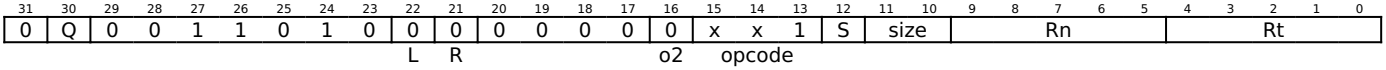
ST3 (single structure)

Store single 3-element structure from one lane of three registers

This instruction stores a 3-element structure to memory from corresponding elements of three SIMD&FP registers.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding for the 8-bit variant

Applies when (opcode == 001)

```
ST3 { <Vt>.B, <Vt2>.B, <Vt3>.B }[<index>], [<Xn|SP>]
```

Encoding for the 16-bit variant

Applies when (opcode == 011 && size == x0)

```
ST3 { <Vt>.H, <Vt2>.H, <Vt3>.H }[<index>], [<Xn|SP>]
```

Encoding for the 32-bit variant

Applies when (opcode == 101 && size == 00)

```
ST3 { <Vt>.S, <Vt2>.S, <Vt3>.S }[<index>], [<Xn|SP>]
```

Encoding for the 64-bit variant

Applies when (opcode == 101 && S == 0 && size == 01)

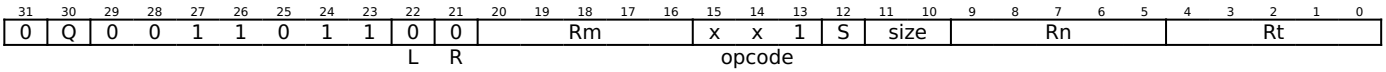
```
ST3 { <Vt>.D, <Vt2>.D, <Vt3>.D }[<index>], [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the 8-bit, immediate offset variant

Applies when (Rm == 11111 && opcode == 001)

```
ST3 { <Vt>.B, <Vt2>.B, <Vt3>.B }[<index>], [<Xn|SP>], #3
```

Encoding for the 8-bit, register offset variant

Applies when (Rm != 11111 && opcode == 001)

```
ST3 { <Vt>.B, <Vt2>.B, <Vt3>.B }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 16-bit, immediate offset variant

Applies when (Rm == 11111 && opcode == 011 && size == x0)

```
ST3 { <Vt>.H, <Vt2>.H, <Vt3>.H }[<index>], [<Xn|SP>], #6
```

Encoding for the 16-bit, register offset variant

Applies when (Rm != 11111 && opcode == 011 && size == x0)

```
ST3 { <Vt>.H, <Vt2>.H, <Vt3>.H }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 32-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 101 && size == 00)`

```
ST3 { <Vt>.S, <Vt2>.S, <Vt3>.S }[<index>], [<Xn|SP>], #12
```

Encoding for the 32-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 101 && size == 00)`

```
ST3 { <Vt>.S, <Vt2>.S, <Vt3>.S }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 64-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 101 && S == 0 && size == 01)`

```
ST3 { <Vt>.D, <Vt2>.D, <Vt3>.D }[<index>], [<Xn|SP>], #24
```

Encoding for the 64-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 101 && S == 0 && size == 01)`

```
ST3 { <Vt>.D, <Vt2>.D, <Vt3>.D }[<index>], [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt>	Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.
<Vt2>	Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.
<Vt3>	Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.
<index>	For the "8-bit", "8-bit, immediate offset", and "8-bit, register offset" variants: is the element index, encoded in "Q:S:size". For the "16-bit", "16-bit, immediate offset", and "16-bit, register offset" variants: is the element index, encoded in "Q:S:size<1>". For the "32-bit", "32-bit, immediate offset", and "32-bit, register offset" variants: is the element index, encoded in "Q:S". For the "64-bit", "64-bit, immediate offset", and "64-bit, register offset" variants: is the element index, encoded in "Q".
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        // extract from one lane of 128-bit register
        Mem{esize}(eaddr, accdesc) = rval[index *: esize];
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4 (multiple structures)

Store multiple 4-element structures from four registers

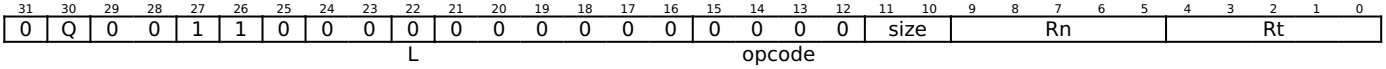
This instruction stores multiple 4-element structures to memory from four SIMD&FP registers, with interleaving. Every element of each register is stored.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding

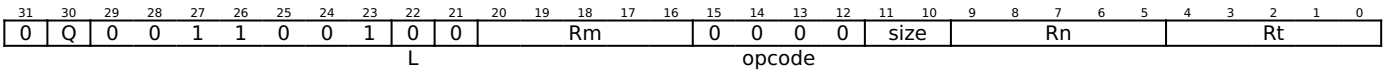
```
ST4 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the Immediate offset variant

Applies when `(Rm == 11111)`

```
ST4 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>], <imm>
```

Encoding for the Register offset variant

Applies when `(Rm != 11111)`

```
ST4 { <Vt>.<T>, <Vt2>.<T>, <Vt3>.<T>, <Vt4>.<T> }, [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt> Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vt2> Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.

<Vt3> Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.

<Vt4> Is the name of the fourth SIMD&FP register to be transferred, encoded as "Rt" plus 3 modulo 32.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the post-index immediate offset, encoded in “Q”:

Q	<imm>
0	#32
1	#64

<Xm> Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(size);
let elements : integer = datasize DIV esize;

let rpt : integer = 1;                 // number of iterations
let selem : integer = 4;           // structure elements

// .1D format only permitted with LD1 & ST1
if size::Q == '110' && selem != 1 then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(datasize);
var tt : integer;
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

for r = 0 to rpt - 1 do
    for e = 0 to elements - 1 do
        tt = (t + r) MOD 32;
        for s = 0 to selem - 1 do
            rval = V{datasize}(tt);
            eaddr = AddressIncrement(address, offs, accdesc);
            Mem{esize}(eaddr, accdesc) = rval[e *: esize];
            offs = offs + ebytes;
            tt = (tt + 1) MOD 32;
        end;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

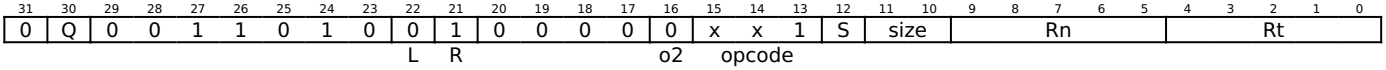
ST4 (single structure)

Store single 4-element structure from one lane of four registers

This instruction stores a 4-element structure to memory from corresponding elements of four SIMD&FP registers.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.
It has encodings from 2 classes: No offset and Post-index

No offset

(FEAT_AdvSIMD)



Encoding for the 8-bit variant

Applies when (opcode == 001)

```
ST4 { <Vt>.B, <Vt2>.B, <Vt3>.B, <Vt4>.B }[<index>], [<Xn|SP>]
```

Encoding for the 16-bit variant

Applies when (opcode == 011 && size == x0)

```
ST4 { <Vt>.H, <Vt2>.H, <Vt3>.H, <Vt4>.H }[<index>], [<Xn|SP>]
```

Encoding for the 32-bit variant

Applies when (opcode == 101 && size == 00)

```
ST4 { <Vt>.S, <Vt2>.S, <Vt3>.S, <Vt4>.S }[<index>], [<Xn|SP>]
```

Encoding for the 64-bit variant

Applies when (opcode == 101 && S == 0 && size == 01)

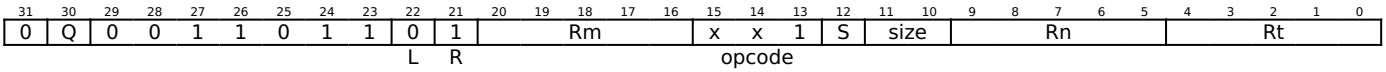
```
ST4 { <Vt>.D, <Vt2>.D, <Vt3>.D, <Vt4>.D }[<index>], [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Post-index

(FEAT_AdvSIMD)



Encoding for the 8-bit, immediate offset variant

Applies when (Rm == 11111 && opcode == 001)

```
ST4 { <Vt>.B, <Vt2>.B, <Vt3>.B, <Vt4>.B }[<index>], [<Xn|SP>], #4
```

Encoding for the 8-bit, register offset variant

Applies when (Rm != 11111 && opcode == 001)

```
ST4 { <Vt>.B, <Vt2>.B, <Vt3>.B, <Vt4>.B }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 16-bit, immediate offset variant

Applies when (Rm == 11111 && opcode == 011 && size == x0)

```
ST4 { <Vt>.H, <Vt2>.H, <Vt3>.H, <Vt4>.H }[<index>], [<Xn|SP>], #8
```

Encoding for the 16-bit, register offset variant

Applies when (Rm != 11111 && opcode == 011 && size == x0)

```
ST4 { <Vt>.H, <Vt2>.H, <Vt3>.H, <Vt4>.H }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 32-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 101 && size == 00)`

```
ST4 { <Vt>.S, <Vt2>.S, <Vt3>.S, <Vt4>.S }[<index>], [<Xn|SP>], #16
```

Encoding for the 32-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 101 && size == 00)`

```
ST4 { <Vt>.S, <Vt2>.S, <Vt3>.S, <Vt4>.S }[<index>], [<Xn|SP>], <Xm>
```

Encoding for the 64-bit, immediate offset variant

Applies when `(Rm == 11111 && opcode == 101 && S == 0 && size == 01)`

```
ST4 { <Vt>.D, <Vt2>.D, <Vt3>.D, <Vt4>.D }[<index>], [<Xn|SP>], #32
```

Encoding for the 64-bit, register offset variant

Applies when `(Rm != 11111 && opcode == 101 && S == 0 && size == 01)`

```
ST4 { <Vt>.D, <Vt2>.D, <Vt3>.D, <Vt4>.D }[<index>], [<Xn|SP>], <Xm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let wback : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt>	Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.
<Vt2>	Is the name of the second SIMD&FP register to be transferred, encoded as "Rt" plus 1 modulo 32.
<Vt3>	Is the name of the third SIMD&FP register to be transferred, encoded as "Rt" plus 2 modulo 32.
<Vt4>	Is the name of the fourth SIMD&FP register to be transferred, encoded as "Rt" plus 3 modulo 32.
<index>	For the "8-bit", "8-bit, immediate offset", and "8-bit, register offset" variants: is the element index, encoded in "Q:S:size". For the "16-bit", "16-bit, immediate offset", and "16-bit, register offset" variants: is the element index, encoded in "Q:S:size<1>". For the "32-bit", "32-bit, immediate offset", and "32-bit, register offset" variants: is the element index, encoded in "Q:S". For the "64-bit", "64-bit, immediate offset", and "64-bit, register offset" variants: is the element index, encoded in "Q".
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose post-index register, excluding XZR, encoded in the "Rm" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        // extract from one lane of 128-bit register
        Mem{esize}(eaddr, accdesc) = rval[index *: esize];
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STBFADD, STBFADDL

Atomic BFloat16 floating-point add, without return

This instruction atomically loads a 16-bit value from memory, adds it to the BFloat16 value held in a register, and stores the result back to memory.

- `STBFADDL` stores to memory with release semantics.
- `STBFADD` has no release semantics.

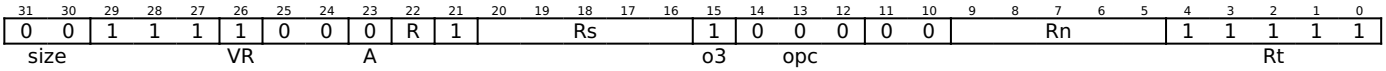
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STBFADD <Hs>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(R == 1)`

```
STBFADDL <Hs>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 16;
let acquire : boolean = FALSE;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_BFADD, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STBFMAX, STBFMAXL

Atomic BFloat16 floating-point maximum, without return

This instruction atomically loads a 16-bit value from memory, calculates the BFloat16 maximum with the value held in a register, and stores the result back to memory.

- `STBFMAXL` stores to memory with release semantics.
- `STBFMAX` has no release semantics.

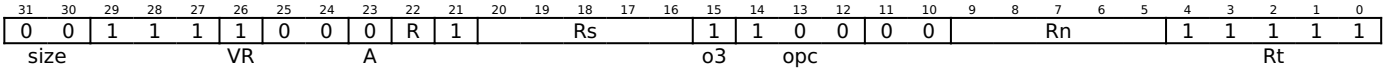
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STBFMAX <Hs>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(R == 1)`

```
STBFMAXL <Hs>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 16;
let acquire : boolean = FALSE;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_BFMAX, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STBFMAXNM, STBFMAXNML

Atomic BFloat16 floating-point maximum number, without return

This instruction atomically loads a 16-bit value from memory, calculates the BFloat16 maximum number with the value held in a register, and stores the result back to memory.

- `STBFMAXNML` stores to memory with release semantics.
- `STBFMAXNM` has no release semantics.

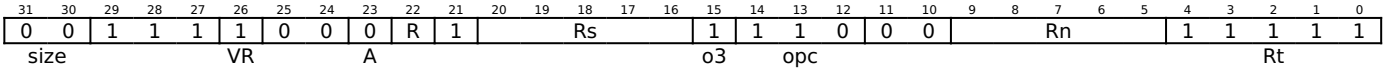
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STBFMAXNM <Hs>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(R == 1)`

```
STBFMAXNML <Hs>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 16;
let acquire : boolean = FALSE;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_BFMAXNM, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STBFMIN, STBFMINL

Atomic BFloat16 floating-point minimum, without return

This instruction atomically loads a 16-bit value from memory, calculates the BFloat16 minimum with the value held in a register, and stores the result back to memory.

- `STBFMINL` stores to memory with release semantics.
- `STBFMIN` has no release semantics.

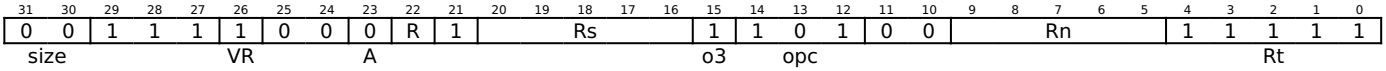
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STBFMIN <Hs>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(R == 1)`

```
STBFMINL <Hs>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 16;
let acquire : boolean = FALSE;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_BFMIN, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STBFMINNM, STBFMINNML

Atomic BFloat16 floating-point minimum number, without return

This instruction atomically loads a 16-bit value from memory, calculates the BFloat16 minimum number with the value held in a register, and stores the result back to memory.

- `STBFMINNML` stores to memory with release semantics.
- `STBFMINNM` has no release semantics.

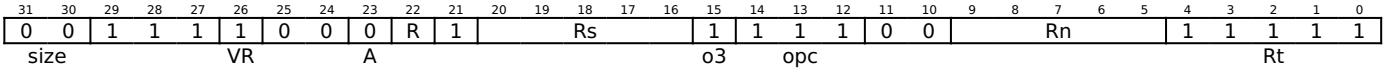
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the No memory ordering variant

Applies when `(R == 0)`

```
STBFMINNM <Hs>, [<Xn|SP>]
```

Encoding for the Release variant

Applies when `(R == 1)`

```
STBFMINNML <Hs>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 16;
let acquire : boolean = FALSE;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_BFMINNM, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STFADD, STFADDL

Atomic floating-point add, without return

This instruction atomically loads a 16-bit, 32-bit, or 64-bit value from memory, performs a floating-point add with the value held in a register, and stores the result back to memory.

- `STFADDL` stores to memory with release semantics.
- `STFADD` has no release semantics.

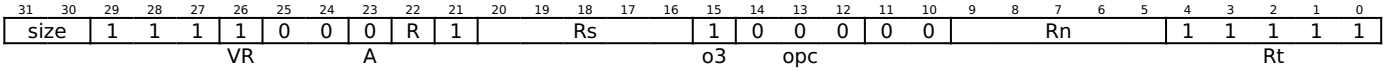
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the Half-precision no memory ordering variant

Applies when `(size == 01 && R == 0)`

```
STFADD <Hs>, [<Xn|SP>]
```

Encoding for the Half-precision release variant

Applies when `(size == 01 && R == 1)`

```
STFADDL <Hs>, [<Xn|SP>]
```

Encoding for the Single-precision no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STFADD <Ss>, [<Xn|SP>]
```

Encoding for the Single-precision release variant

Applies when `(size == 10 && R == 1)`

```
STFADDL <Ss>, [<Xn|SP>]
```

Encoding for the Double-precision no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STFADD <Ds>, [<Xn|SP>]
```

Encoding for the Double-precision release variant

Applies when `(size == 11 && R == 1)`

```
STFADDL <Ds>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = FALSE;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Ss> Is the 32-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

<Ds> Is the 64-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_FPADD, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STFMAX, STFMAXL

Atomic floating-point maximum, without return

This instruction atomically loads a 16-bit, 32-bit, or 64-bit value from memory, calculates the floating-point maximum with the value held in a register, and stores the result back to memory.

- `STFMAXL` stores to memory with release semantics.
- `STFMAX` has no release semantics.

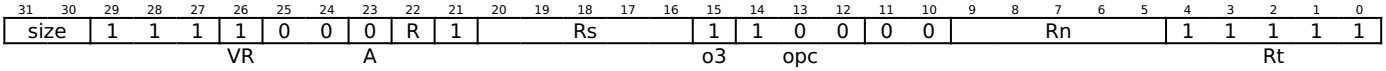
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the Half-precision no memory ordering variant

Applies when `(size == 01 && R == 0)`

```
STFMAX <Hs>, [<Xn|SP>]
```

Encoding for the Half-precision release variant

Applies when `(size == 01 && R == 1)`

```
STFMAXL <Hs>, [<Xn|SP>]
```

Encoding for the Single-precision no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STFMAX <Ss>, [<Xn|SP>]
```

Encoding for the Single-precision release variant

Applies when `(size == 10 && R == 1)`

```
STFMAXL <Ss>, [<Xn|SP>]
```

Encoding for the Double-precision no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STFMAX <Ds>, [<Xn|SP>]
```

Encoding for the Double-precision release variant

Applies when `(size == 11 && R == 1)`

```
STFMAXL <Ds>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = FALSE;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Ss> Is the 32-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

<Ds> Is the 64-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_FPMAX, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STFMAXNM, STFMAXNML

Atomic floating-point maximum number, without return

This instruction atomically loads a 16-bit, 32-bit, or 64-bit value from memory, calculates the floating-point maximum number with the value held in a register, and stores the result back to memory.

- `STFMAXNML` stores to memory with release semantics.
- `STFMAXNM` has no release semantics.

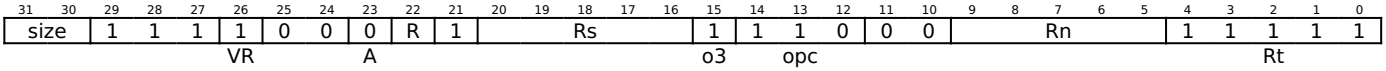
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the Half-precision no memory ordering variant

Applies when `(size == 01 && R == 0)`

```
STFMAXNM <Hs>, [<Xn|SP>]
```

Encoding for the Half-precision release variant

Applies when `(size == 01 && R == 1)`

```
STFMAXNML <Hs>, [<Xn|SP>]
```

Encoding for the Single-precision no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STFMAXNM <Ss>, [<Xn|SP>]
```

Encoding for the Single-precision release variant

Applies when `(size == 10 && R == 1)`

```
STFMAXNML <Ss>, [<Xn|SP>]
```

Encoding for the Double-precision no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STFMAXNM <Ds>, [<Xn|SP>]
```

Encoding for the Double-precision release variant

Applies when `(size == 11 && R == 1)`

```
STFMAXNML <Ds>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = FALSE;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Ss> Is the 32-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

<Ds> Is the 64-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_FPMAXNM, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STFMIN, STFMINL

Atomic floating-point minimum, without return

This instruction atomically loads a 16-bit, 32-bit, or 64-bit value from memory, calculates the floating-point minimum with the value held in a register, and stores the result back to memory.

- `STFMINL` stores to memory with release semantics.
- `STFMIN` has no release semantics.

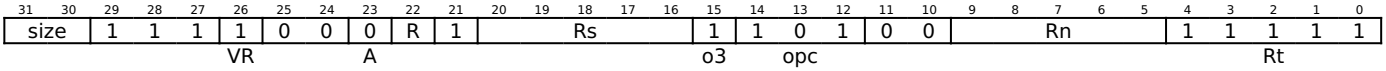
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the Half-precision no memory ordering variant

Applies when `(size == 01 && R == 0)`

```
STFMIN <Hs>, [<Xn|SP>]
```

Encoding for the Half-precision release variant

Applies when `(size == 01 && R == 1)`

```
STFMINL <Hs>, [<Xn|SP>]
```

Encoding for the Single-precision no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STFMIN <Ss>, [<Xn|SP>]
```

Encoding for the Single-precision release variant

Applies when `(size == 10 && R == 1)`

```
STFMINL <Ss>, [<Xn|SP>]
```

Encoding for the Double-precision no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STFMIN <Ds>, [<Xn|SP>]
```

Encoding for the Double-precision release variant

Applies when `(size == 11 && R == 1)`

```
STFMINL <Ds>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = FALSE;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Ss> Is the 32-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

<Ds> Is the 64-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_FPMIN, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STFMINNM, STFMINNML

Atomic floating-point minimum number, without return

This instruction atomically loads a 16-bit, 32-bit, or 64-bit value from memory, calculates the floating-point minimum number with the value held in a register, and stores the result back to memory.

- `STFMINNML` stores to memory with release semantics.
- `STFMINNM` has no release semantics.

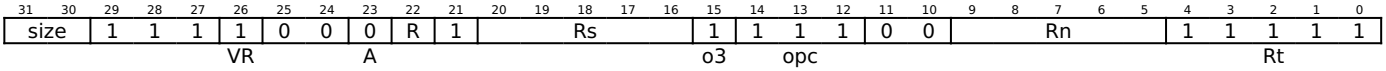
This instruction:

- Disables alternative floating-point behaviors, as if FPCR.AH is 0.
- Generates only the default NaN, as if FPCR.DN is 1.
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.

For more information about memory ordering semantics, see *Load-Acquire, Store-Release*.
For information about addressing modes, see *Load/Store addressing modes*.

Floating-point

(FEAT_LSFE)



Encoding for the Half-precision no memory ordering variant

Applies when `(size == 01 && R == 0)`

```
STFMINNM <Hs>, [<Xn|SP>]
```

Encoding for the Half-precision release variant

Applies when `(size == 01 && R == 1)`

```
STFMINNML <Hs>, [<Xn|SP>]
```

Encoding for the Single-precision no memory ordering variant

Applies when `(size == 10 && R == 0)`

```
STFMINNM <Ss>, [<Xn|SP>]
```

Encoding for the Single-precision release variant

Applies when `(size == 10 && R == 1)`

```
STFMINNML <Ss>, [<Xn|SP>]
```

Encoding for the Double-precision no memory ordering variant

Applies when `(size == 11 && R == 0)`

```
STFMINNM <Ds>, [<Xn|SP>]
```

Encoding for the Double-precision release variant

Applies when `(size == 11 && R == 1)`

```
STFMINNML <Ds>, [<Xn|SP>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_LSFE) then EndOfDecode(Decode_UNDEF); end;

let n : integer = UInt(Rn);
let s : integer = UInt(Rs);

let datasize : integer{} = 8 << UInt(size);
let acquire : boolean = FALSE;
let release : boolean = R == '1';
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Hs>	Is the 16-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Ss> Is the 32-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

<Ds> Is the 64-bit name of the SIMD&FP register holding the data value to be operated on with the contents of the memory location, encoded in the "Rs" field.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
var value : bits(datasize);
var data : bits(datasize);
let accdesc : AccessDescriptor = CreateAccDescFPAtomicOp(MemAtomicOp_FPMINNM, acquire,
                                                         release, tagchecked);

value = V{datasize}(s);
if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

// Irrelevant when not executing CAS
let comparevalue : bits(datasize) = ARBITRARY : bits(datasize);
data = MemAtomic{datasize}(address, comparevalue, value, accdesc);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

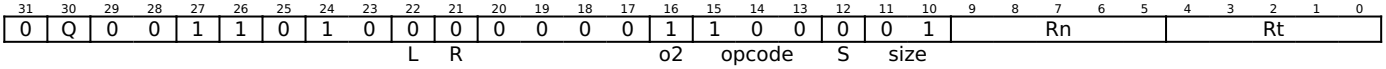
STL1 (SIMD&FP)

Store-release a single-element structure from one lane of one register

This instruction stores the specified element of a SIMD&FP register to memory.
The instruction also has memory ordering semantics, as described in *Load-Acquire, Load-AcquirePC, and Store-Release*. For information about addressing modes, see *Load/Store addressing modes*.
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

64-bit

(FEAT_AdvSIMD && FEAT_LRCPC3)



Encoding

```
STL1 { <Vt>.D }[<index>], [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_LRCPC3) then
    EndOfDecode(Decode_UNDEF);
end;
var t : integer = UInt(Rt);
let n : integer = UInt(Rn);
let m : integer = ARBITRARY : integer;
let wback : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Assembler Symbols

<Vt>	Is the name of the first or only SIMD&FP register to be transferred, encoded in the "Rt" field.
<index>	Is the element index, encoded in "Q".
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Shared Decode

```
var scale : bits(2) = opcode[2:1];
let selem : integer = UInt(opcode[0]::R) + 1;
var replicate : boolean = FALSE;
var index : integer;

case scale of
    when '11' =>
        // load and replicate
        if L == '0' || S == '1' then EndOfDecode(Decode_UNDEF); end;
        scale = size;
        replicate = TRUE;
    when '00' =>
        index = UInt(Q::S::size);     // B[0-15]
    when '01' =>
        if size[0] == '1' then EndOfDecode(Decode_UNDEF); end;
        index = UInt(Q::S::size[1]);  // H[0-7]
    when '10' =>
        if size[1] == '1' then EndOfDecode(Decode_UNDEF); end;
        if size[0] == '0' then
            index = UInt(Q::S);       // S[0-3]
        else
            if S == '1' then EndOfDecode(Decode_UNDEF); end;
            index = UInt(Q);          // D[0-1]
            scale = '11';
        end;
end;

let datasize : integer{} = 64 << UInt(Q);
let esize : integer{} = 8 << UInt(scale);
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

var address : bits(64);
var eaddr : bits(64);
var rval : bits(128);
var element : bits(esize);
var offs : bits(64) = Zeros{};
let ebytes : integer{} = esize DIV 8;

let accdesc : AccessDescriptor = CreateAccDescASIMDAcqRel(MemOp_STORE, tagchecked);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if replicate then
    // load and replicate to all elements
    for s = 0 to selem - 1 do
        eaddr = AddressIncrement(address, offs, accdesc);
        element = Mem{esize}(eaddr, accdesc);
        // replicate to fill 128- or 64-bit register
        V{datasize}(t) = Replicate{datasize}(element);
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
else
    // load/store one element per register
    for s = 0 to selem - 1 do
        rval = V{128}(t);
        eaddr = AddressIncrement(address, offs, accdesc);
        // extract from one lane of 128-bit register
        Mem{esize}(eaddr, accdesc) = rval[index *: esize];
        offs = offs + ebytes;
        t = (t + 1) MOD 32;
    end;
end;
if wback then
    if m != 31 then
        offs = X{64}(m);
    end;
    address = AddressAdd(address, offs, accdesc);
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLUR (SIMD&FP)

Store-release SIMD&FP register (unscaled offset)

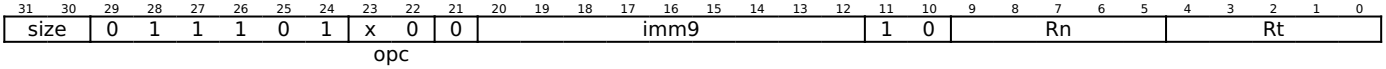
This instruction stores a single SIMD&FP register to memory. The address that is used for the store is calculated from a base register value and an optional immediate offset.

The instruction has memory ordering semantics, as described in *Load-Acquire, Load-AcquirePC, and Store-Release*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Unscaled offset

(FEAT_FP && FEAT_LRCPC3)



Encoding for the 8-bit variant

Applies when `(size == 00 && opc == 00)`

```
STLUR <Bt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 00)`

```
STLUR <Ht>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 00)`

```
STLUR <St>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 00)`

```
STLUR <Dt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 10)`

```
STLUR <Qt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) || !IsFeatureImplemented(FEAT_LRCPC3) then
    EndOfDecode(Decode_UNDEF);
end;
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Bt>	Is the 8-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<St>	Is the 32-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt>	Is the 128-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << scale;
let tagchecked : boolean = n != 31;
```

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
var address : bits(64);
let accdesc : AccessDescriptor = CreateAccDescASIMDAcqRel(MemOp_STORE, tagchecked);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = V{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNP (SIMD&FP)

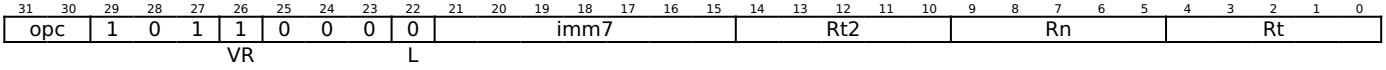
Store pair of SIMD&FP registers, with non-temporal hint

This instruction stores a pair of SIMD&FP registers to memory, issuing a hint to the memory system that the access is non-temporal. The address used for the store is calculated from an address from a base register value and an immediate offset. For information about non-temporal pair instructions, see *Load/Store SIMD and Floating-point non-temporal pair*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Signed offset

(FEAT_FP)



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
STNP <St1>, <St2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 01)`

```
STNP <Dt1>, <Dt2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 128-bit variant

Applies when `(opc == 10)`

```
STNP <Qt1>, <Qt2>, [<Xn|SP>{, #<imm>}]
```

Decode for all variants of this encoding

```
// Empty.
```

Assembler Symbols

<St1>	Is the 32-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<St2>	Is the 32-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "32-bit" variant: is the optional signed immediate byte offset, a multiple of 4 in the range -256 to 252, defaulting to 0 and encoded in the "imm7" field as <imm>/4. For the "64-bit" variant: is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8. For the "128-bit" variant: is the optional signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, defaulting to 0 and encoded in the "imm7" field as <imm>/16.
<Dt1>	Is the 64-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Dt2>	Is the 64-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Qt1>	Is the 128-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt2>	Is the 128-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.

Shared Decode

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = TRUE;
let scale : integer{} = 2 + (UInt(opc) as integer{0..2});
let datasize : integer{} = 8 << scale;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = n != 31;
```

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal, tagchecked,
                                                    privileged, ispair);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

var data : bits(2*datasize);
if BigEndian(accdesc.acctype) then
    data = V{datasize}(t) :: V{datasize}(t2);
else
    data = V{datasize}(t2) :: V{datasize}(t);
end;
Mem{2*datasize}(address, accdesc) = data;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STP (SIMD&FP)

Store pair of SIMD&FP registers

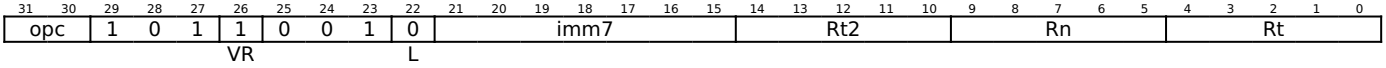
This instruction stores a pair of SIMD&FP registers to memory. The address used for the store is calculated from a base register value and an immediate offset.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_FP)



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
STP <St1>, <St2>, [<Xn|SP>], #<imm>
```

Encoding for the 64-bit variant

Applies when `(opc == 01)`

```
STP <Dt1>, <Dt2>, [<Xn|SP>], #<imm>
```

Encoding for the 128-bit variant

Applies when `(opc == 10)`

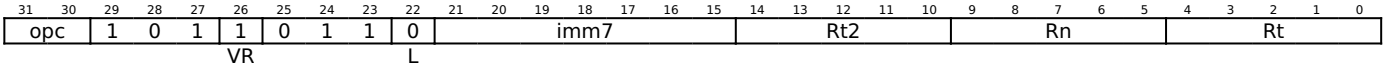
```
STP <Qt1>, <Qt2>, [<Xn|SP>], #<imm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;

let wback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_FP)



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
STP <St1>, <St2>, [<Xn|SP>, #<imm>]!
```

Encoding for the 64-bit variant

Applies when `(opc == 01)`

```
STP <Dt1>, <Dt2>, [<Xn|SP>, #<imm>]!
```

Encoding for the 128-bit variant

Applies when `(opc == 10)`

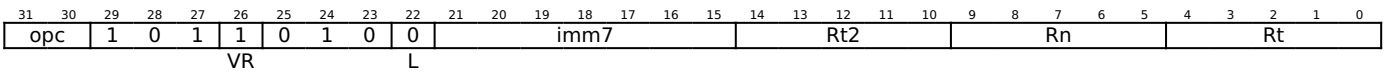
```
STP <Qt1>, <Qt2>, [<Xn|SP>, #<imm>]!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;

let wback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_FP)



Encoding for the 32-bit variant

Applies when `(opc == 00)`

```
STP <St1>, <St2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 64-bit variant

Applies when `(opc == 01)`

```
STP <Dt1>, <Dt2>, [<Xn|SP>{, #<imm>}]
```

Encoding for the 128-bit variant

Applies when `(opc == 10)`

```
STP <Qt1>, <Qt2>, [<Xn|SP>{, #<imm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;

let wback : boolean = FALSE;
let postindex : boolean = FALSE;
```

Assembler Symbols

<St1>	Is the 32-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<St2>	Is the 32-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Post-index 32-bit" and "Pre-index 32-bit" variants: is the signed immediate byte offset, a multiple of 4 in the range -256 to 252, encoded in the "imm7" field as <imm>/4. For the "Post-index 64-bit" and "Pre-index 64-bit" variants: is the signed immediate byte offset, a multiple of 8 in the range -512 to 504, encoded in the "imm7" field as <imm>/8. For the "Post-index 128-bit" and "Pre-index 128-bit" variants: is the signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, encoded in the "imm7" field as <imm>/16. For the "Signed offset 32-bit" variant: is the optional signed immediate byte offset, a multiple of 4 in the range -256 to 252, defaulting to 0 and encoded in the "imm7" field as <imm>/4. For the "Signed offset 64-bit" variant: is the optional signed immediate byte offset, a multiple of 8 in the range -512 to 504, defaulting to 0 and encoded in the "imm7" field as <imm>/8. For the "Signed offset 128-bit" variant: is the optional signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, defaulting to 0 and encoded in the "imm7" field as <imm>/16.
<Dt1>	Is the 64-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Dt2>	Is the 64-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Qt1>	Is the 128-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt2>	Is the 128-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let scale : integer{} = 2 + (UInt(opc) as integer{0..2});
let datasize : integer{} = 8 << scale;
let offset : bits(64) = LSL(SignExtend{64}(imm7), scale);
let tagchecked : boolean = wback || n != 31;
```

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged, ispair);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

var data : bits(2*datasize);
if BigEndian(accdesc.acctype) then
    data = V{datasize}(t) :: V{datasize}(t2);
else
    data = V{datasize}(t2) :: V{datasize}(t);
end;

Mem{2*datasize}(address, accdesc) = data;

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STR (immediate, SIMD&FP)

Store SIMD&FP register (immediate offset)

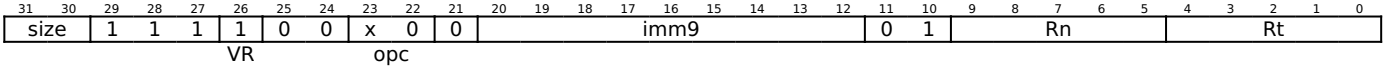
This instruction stores a single SIMD&FP register to memory. The address that is used for the store is calculated from a base register value and an immediate offset.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 3 classes: Post-index , Pre-index and Unsigned offset

Post-index

(FEAT_FP)



Encoding for the 8-bit variant

Applies when `(size == 00 && opc == 00)`

```
STR <Bt>, [<Xn|SP>], #<simm>
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 00)`

```
STR <Ht>, [<Xn|SP>], #<simm>
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 00)`

```
STR <St>, [<Xn|SP>], #<simm>
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 00)`

```
STR <Dt>, [<Xn|SP>], #<simm>
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 10)`

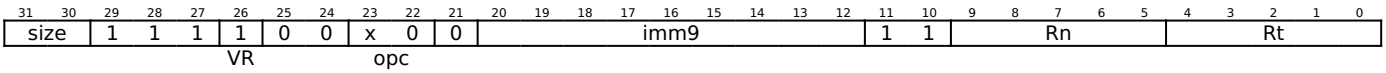
```
STR <Qt>, [<Xn|SP>], #<simm>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let wback : boolean = TRUE;
let postindex : boolean = TRUE;
let offset : bits(64) = SignExtend{}(imm9);
```

Pre-index

(FEAT_FP)



Encoding for the 8-bit variant

Applies when `(size == 00 && opc == 00)`

```
STR <Bt>, [<Xn|SP>, #<simm>]!
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 00)`

```
STR <Ht>, [<Xn|SP>, #<simm>]!
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 00)`

```
STR <St>, [<Xn|SP>, #<simm>]!
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 00)`

```
STR <Dt>, [<Xn|SP>, #<simm>]!
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 10)`

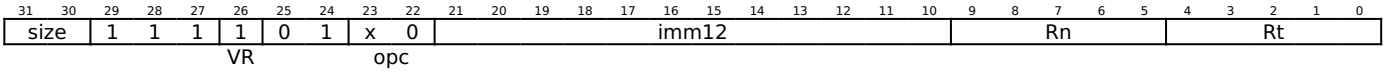
```
STR <Qt>, [<Xn|SP>, #<simm>]!
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let wback : boolean = TRUE;
let postindex : boolean = FALSE;
let offset : bits(64) = SignExtend{}(imm9);
```

Unsigned offset

(FEAT_FP)



Encoding for the 8-bit variant

Applies when `(size == 00 && opc == 00)`

```
STR <Bt>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 00)`

```
STR <Ht>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 00)`

```
STR <St>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 00)`

```
STR <Dt>, [<Xn|SP>{, #<pimm>}]
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 10)`

```
STR <Qt>, [<Xn|SP>{, #<pimm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let wback : boolean = FALSE;
let postindex : boolean = FALSE;
let offset : bits(64) = LSL(ZeroExtend{64}(imm12), scale);
```

Assembler Symbols

<Bt>	Is the 8-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the signed immediate byte offset, in the range -256 to 255, encoded in the "imm9" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<St>	Is the 32-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt>	Is the 128-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

<pimm> For the "8-bit" variant: is the optional positive immediate byte offset, in the range 0 to 4095, defaulting to 0 and encoded in the "imm12" field.

For the "16-bit" variant: is the optional positive immediate byte offset, a multiple of 2 in the range 0 to 8190, defaulting to 0 and encoded in the "imm12" field as <pimm>/2.

For the "32-bit" variant: is the optional positive immediate byte offset, a multiple of 4 in the range 0 to 16380, defaulting to 0 and encoded in the "imm12" field as <pimm>/4.

For the "64-bit" variant: is the optional positive immediate byte offset, a multiple of 8 in the range 0 to 32760, defaulting to 0 and encoded in the "imm12" field as <pimm>/8.

For the "128-bit" variant: is the optional positive immediate byte offset, a multiple of 16 in the range 0 to 65520, defaulting to 0 and encoded in the "imm12" field as <pimm>/16.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << scale;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = wback || n != 31;
```

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal, tagchecked,
                                                    privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

Mem{datasize}(address, accdesc) = V{datasize}(t);

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STR (register, SIMD&FP)

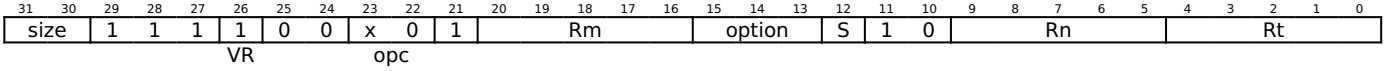
Store SIMD&FP register (register offset)

This instruction stores a single SIMD&FP register to memory. The address that is used for the store is calculated from a base register value and an offset register value. The offset can be optionally shifted and extended.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

SIMD&FP registers

(FEAT_FP)



Encoding for the 8-bit with extended register offset variant

Applies when `(size == 00 && opc == 00 && option != 011)`

```
STR <Bt>, [<Xn|SP>, (<Wm>|<Xm>), <extend> {<amount>}]
```

Encoding for the 8-bit with shifted register offset variant

Applies when `(size == 00 && opc == 00 && option == 011)`

```
STR <Bt>, [<Xn|SP>, <Xm>{, LSL <amount>}]
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 00)`

```
STR <Ht>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 00)`

```
STR <St>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 00)`

```
STR <Dt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 10)`

```
STR <Qt>, [<Xn|SP>, (<Wm>|<Xm>){, <extend> {<amount>}}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if option[1] == '0' then EndOfDecode(Decode_UNDEF); end;          // sub-word index
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let extend_type : ExtendType = DecodeRegExtend(option);
let shift : integer{} = if S == '1' then scale else 0;
```

Assembler Symbols

<Bt>	Is the 8-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Wm>	When option<0> is set to 0, is the 32-bit name of the general-purpose index register, encoded in the "Rm" field.
<Xm>	When option<0> is set to 1, is the 64-bit name of the general-purpose index register, encoded in the "Rm" field.
<extend>	For the "8-bit with extended register offset" variant: is the index extend specifier, encoded in “option”:

option	<extend>
010	UXTW
110	SXTW
111	SXTX

For the "128-bit", "16-bit", "32-bit", and "64-bit" variants: is the index extend/shift specifier, defaulting to LSL, and which must be omitted for the LSL option when <amount> is omitted, encoded in “option”:

option	<extend>
010	UXTW
011	LSL
110	SXTW
111	SXTX

<amount>

For the "8-bit with extended register offset" and "8-bit with shifted register offset" variants: is the index shift amount, it must be #0, encoded in "S" as 0 if omitted, or as 1 if present.

For the "16-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#1

For the "32-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#2

For the "64-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#3

For the "128-bit" variant: is the index shift amount, optional only when <extend> is not LSL. Where it is permitted to be optional, it defaults to #0. It is encoded in “S”:

S	<amount>
0	#0
1	#4

<Ht>

Is the 16-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

<St>

Is the 32-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

<Dt>

Is the 64-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

<Qt>

Is the 128-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 8 << scale;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
```

Operation

```
AArch64_CheckFPEnabled();
let offset : bits(64) = ExtendReg{}(m, extend_type, shift);
var address : bits(64);

let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal, tagchecked,
                                                    privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = V{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTNP (SIMD&FP)

Store unprivileged pair of SIMD&FP registers, with non-temporal hint

This instruction stores a pair of SIMD&FP registers to memory, issuing a hint to the memory system that the access is non-temporal. The address used for the store is calculated from an address from a base register value and an immediate offset. For information about non-temporal pair instructions, see *Load/Store SIMD and Floating-point non-temporal pair*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

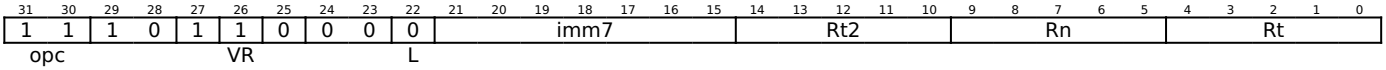
Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

Signed offset

(FEAT_FP && FEAT_LSUI)



Encoding

```
STTNP <Qt1>, <Qt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP) || !IsFeatureImplemented(FEAT_LSUI) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = TRUE;
let datasize : integer{} = 128;
let offset : bits(64) = LSL(SignExtend{64}(imm7), 4);
let tagchecked : boolean = n != 31;
```

Assembler Symbols

<Qt1>	Is the 128-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt2>	Is the 128-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, defaulting to 0 and encoded in the "imm7" field as <imm>/16.

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal, tagchecked,
                                                    privileged, ispair);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

var data : bits(2*datasize);
if BigEndian(accdesc.acctype) then
    data = V{datasize}(t) :: V{datasize}(t2);
else
    data = V{datasize}(t2) :: V{datasize}(t);
end;
Mem{2*datasize}(address, accdesc) = data;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STTP (SIMD&FP)

Store unprivileged pair of SIMD&FP registers

This instruction stores a pair of SIMD&FP registers to memory. The address used for the store is calculated from a base register value and an immediate offset.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Explicit Memory effects produced by the instruction behave as if the instruction was executed at EL0 if the *Effective value* of PSTATE.UAO is 0 and either:

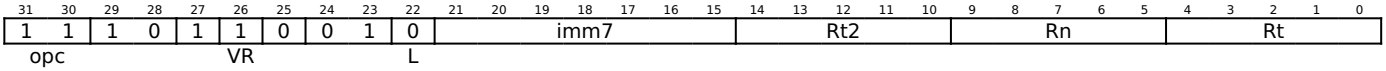
- The instruction is executed at EL1.
- The instruction is executed at EL2 when the *Effective value* of *HCR_EL2()*.{E2H, TGE} is '11'.

Otherwise, the Explicit Memory effects operate with the restrictions determined by the Exception level at which the instruction is executed.

It has encodings from 3 classes: Post-index , Pre-index and Signed offset

Post-index

(FEAT_FP && FEAT_LSUI)



Encoding

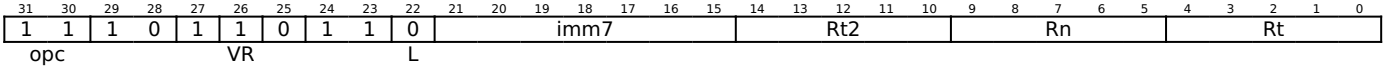
```
STTP <Qt1>, <Qt2>, [<Xn|SP>], #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP) || !IsFeatureImplemented(FEAT_LSUI) then
    EndOfDecode(Decode_UNDEF);
end;

let wback : boolean = TRUE;
let postindex : boolean = TRUE;
```

Pre-index

(FEAT_FP && FEAT_LSUI)



Encoding

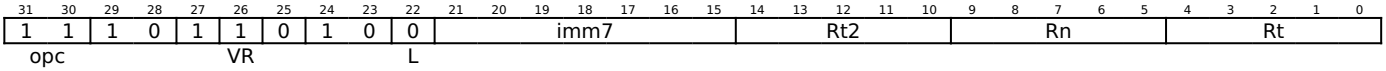
```
STTP <Qt1>, <Qt2>, [<Xn|SP>, #<imm>]!
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP) || !IsFeatureImplemented(FEAT_LSUI) then
    EndOfDecode(Decode_UNDEF);
end;

let wback : boolean = TRUE;
let postindex : boolean = FALSE;
```

Signed offset

(FEAT_FP && FEAT_LSUI)



Encoding

```
STTP <Qt1>, <Qt2>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_FP) || !IsFeatureImplemented(FEAT_LSUI) then
    EndOfDecode(Decode_UNDEF);
end;

let wback : boolean = FALSE;
let postindex : boolean = FALSE;
```

Assembler Symbols

<Qt1>	Is the 128-bit name of the first SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt2>	Is the 128-bit name of the second SIMD&FP register to be transferred, encoded in the "Rt2" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Post-index" and "Pre-index" variants: is the signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, encoded in the "imm7" field as <imm>/16. For the "Signed offset" variant: is the optional signed immediate byte offset, a multiple of 16 in the range -1024 to 1008, defaulting to 0 and encoded in the "imm7" field as <imm>/16.

Shared Decode

```
let t : integer{} = UInt(Rt);
let t2 : integer{} = UInt(Rt2);
let n : integer{} = UInt(Rn);
let nontemporal : boolean = FALSE;
let datasize : integer{} = 128;
let offset : bits(64) = LSL(SignExtend{64}(imm7), 4);
let tagchecked : boolean = wback || n != 31;
```

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);

let privileged : boolean = AArch64_IsUnprivAccessPriv();
let ispair : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged, ispair);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

if !postindex then
    address = AddressAdd(address, offset, accdesc);
end;

var data : bits(2*datasize);
if BigEndian(accdesc.acctype) then
    data = V{datasize}(t) :: V{datasize}(t2);
else
    data = V{datasize}(t2) :: V{datasize}(t);
end;

Mem{2*datasize}(address, accdesc) = data;

if wback then
    if postindex then
        address = AddressAdd(address, offset, accdesc);
    end;
    if n == 31 then
        SP{64}() = address;
    else
        X{64}(n) = address;
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STUR (SIMD&FP)

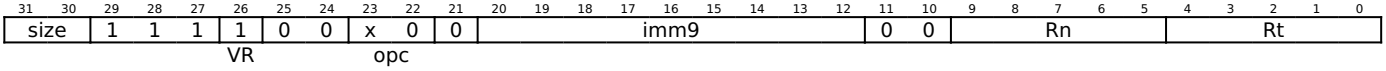
Store SIMD&FP register (unscaled offset)

This instruction stores a single SIMD&FP register to memory. The address that is used for the store is calculated from a base register value and an optional immediate offset.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Unscaled offset

(FEAT_FP)



Encoding for the 8-bit variant

Applies when `(size == 00 && opc == 00)`

```
STUR <Bt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 16-bit variant

Applies when `(size == 01 && opc == 00)`

```
STUR <Ht>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 32-bit variant

Applies when `(size == 10 && opc == 00)`

```
STUR <St>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 64-bit variant

Applies when `(size == 11 && opc == 00)`

```
STUR <Dt>, [<Xn|SP>{, #<simm>}]
```

Encoding for the 128-bit variant

Applies when `(size == 00 && opc == 10)`

```
STUR <Qt>, [<Xn|SP>{, #<simm>}]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if opc[1] == '1' && size != '00' then EndOfDecode(Decode_UNDEF); end;
let scale : integer{} = if opc[1] == '1' then 4 else UInt(size);
let offset : bits(64) = SignExtend{}(imm9);
```

Assembler Symbols

<Bt>	Is the 8-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<simm>	Is the optional signed immediate byte offset, in the range -256 to 255, defaulting to 0 and encoded in the "imm9" field.
<Ht>	Is the 16-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<St>	Is the 32-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Dt>	Is the 64-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.
<Qt>	Is the 128-bit name of the SIMD&FP register to be transferred, encoded in the "Rt" field.

Shared Decode

```
let t : integer{} = UInt(Rt);
let n : integer{} = UInt(Rn);
let datasize : integer{} = 8 << scale;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
```

Operation

```
AArch64_CheckFPEnabled();
var address : bits(64);
let privileged : boolean = PSTATE.EL != EL0;
let accdesc : AccessDescriptor = CreateAccDescASIMD(MemOp_STORE, nontemporal,
                                                    tagchecked, privileged);

if n == 31 then
    CheckSPAlignment();
    address = SP{64}();
else
    address = X{64}(n);
end;

address = AddressAdd(address, offset, accdesc);

Mem{datasize}(address, accdesc) = V{datasize}(t);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUB (vector)

Subtract (vector)

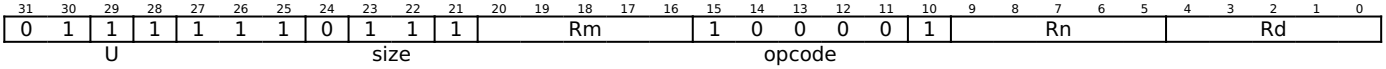
This instruction subtracts each vector element in the second source SIMD&FP register from the corresponding vector element in the first source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

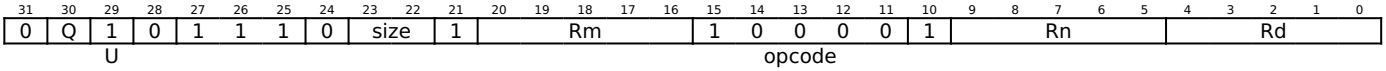
```
SUB D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SUB <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    element1 = operand1[e*:esize];
    element2 = operand2[e*:esize];
    result[e*:esize] = element1 - element2;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUBHN, SUBHN2

Subtract returning high narrow

This instruction subtracts each vector element in the second source SIMD&FP register from the corresponding vector element in the first source SIMD&FP register, places the most significant half of the result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. All the values in this instruction are signed integer values.

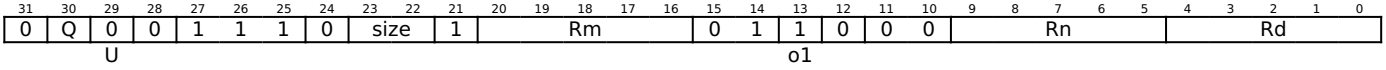
The results are truncated. For rounded results, see *RSUBHN*.

The SUBHN instruction writes the vector to the lower half of the destination register and clears the upper half. The SUBHN2 instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
SUBHN{2} <Vd>.<Tb>, <Vn>.<Ta>, <Vm>.<Ta>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
let round : boolean = FALSE;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(2*datasize) = V{}(n);
let operand2 : bits(2*datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:(2*esize)]);
    element2 = UInt(operand2[e*:(2*esize)]);
    sum = element1 - element2;
    sum = RShr(sum, esize, round);
    result[e*:esize] = sum[esize-1:0];
end;

Vpart{datasize}(d, part) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUDOT (by element)

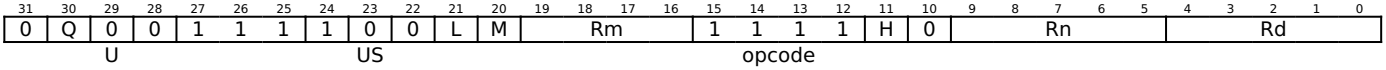
Dot product with signed and unsigned integers (vector, by element)

This instruction performs the dot product of the four signed 8-bit integer values in each 32-bit element of the first source register with the four unsigned 8-bit integer values in an indexed 32-bit element of the second source register, accumulating the result into the corresponding 32-bit element of the destination vector.

From Armv8.2 to Armv8.5, this is an OPTIONAL instruction. From Armv8.6 it is mandatory for implementations that include Advanced SIMD to support it. ID_AA64ISAR1_EL1.I8MM indicates whether this instruction is supported.

Vector

(FEAT_I8MM)



Encoding

```
SUDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.4B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_I8MM) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(M::Rm);
let d : integer = UInt(Rd);
let i : integer = UInt(H::L);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 32;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> Is the immediate index of a group of four 8-bit values, in the range 0 to 3, encoded in the "H:L" fields.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    var res : bits(32) = operand3[e*:32];
    for b = 0 to 3 do
        let element1 : integer = SInt(operand1[(4 * e + b)*:8]);
        let element2 : integer = UInt(operand2[(4 * i + b)*:8]);
        res = res + element1 * element2;
    end;
    result[e*:32] = res;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUQADD

Signed saturating accumulate of unsigned value

This instruction adds the unsigned integer values of the vector elements in the source SIMD&FP register to corresponding signed integer values of the vector elements in the destination SIMD&FP register, and writes the resulting signed integer values to the destination SIMD&FP register.

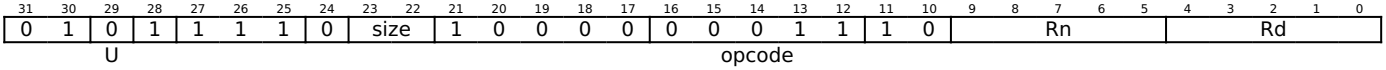
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

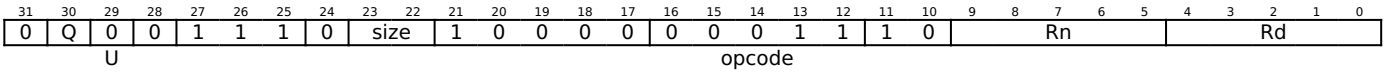
```
SUQADD <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let unsigned : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
SUQADD <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

let operand2 : bits(datasize) = V{}(d);
var op1 : integer;
var op2 : integer;
var sat : boolean;

for e = 0 to elements-1 do
    op1 = UInt(operand[e*:esize]);
    op2 = SInt(operand2[e*:esize]);
    (result[e*:esize], sat) = SatQ{esize}(op1 + op2, unsigned);
    if sat then FPSR().QC = '1'; end;
end;
V{datasize}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SXTL, SXTL2

Signed extend long

This instruction duplicates each vector element in the lower or upper half of the source SIMD&FP register into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements. All the values in this instruction are signed integer values.

The `SXTL` instruction extracts the source vector from the lower half of the source register. The `SXTL2` instruction extracts the source vector from the upper half of the source register.

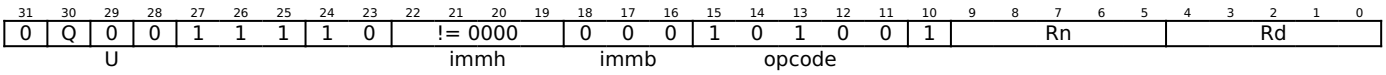
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This is an alias of SSHLL, SSHLL2. This means:

- The encodings in this description are named to match the encodings of SSHLL, SSHLL2.
- The description of SSHLL, SSHLL2 gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Vector

(FEAT_AdvSIMD)



Encoding

```
SXTL{2} <Vd>.<Ta>, <Vn>.<Tb>
```

is equivalent to

```
SSHLL{2} <Vd>.<Ta>, <Vn>.<Tb>, #0
```

and is the preferred disassembly when `BitCount(immh) == 1`.

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

<Vn> — Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

Operation

The description of SSHLL, SSHLL2 gives the operational pseudocode for this instruction.

Operational Information

The description of SSHLL, SSHLL2 gives the operational information for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TBL

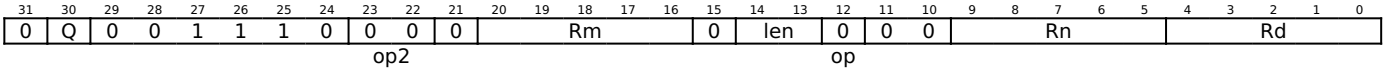
Table vector lookup

This instruction reads each value from the vector elements in the index source SIMD&FP register, uses each result as an index to perform a lookup in a table of bytes that is described by one to four source table SIMD&FP registers, places the lookup result in a vector, and writes the vector to the destination SIMD&FP register. If an index is out of range for the table, the result for that lookup is 0. If more than one source register is used to describe the table, the first source register describes the lowest bytes of the table.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding for the Single register table variant

Applies when `(len == 00)`

```
TBL <Vd>.<Ta>, { <Vn>.16B }, <Vm>.<Ta>
```

Encoding for the Two register table variant

Applies when `(len == 01)`

```
TBL <Vd>.<Ta>, { <Vn>.16B, <Vn+1>.16B }, <Vm>.<Ta>
```

Encoding for the Three register table variant

Applies when `(len == 10)`

```
TBL <Vd>.<Ta>, { <Vn>.16B, <Vn+1>.16B, <Vn+2>.16B }, <Vm>.<Ta>
```

Encoding for the Four register table variant

Applies when `(len == 11)`

```
TBL <Vd>.<Ta>, { <Vn>.16B, <Vn+1>.16B, <Vn+2>.16B, <Vn+3>.16B }, <Vm>.<Ta>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 8;
let regs : integer{} = UInt(len) + 1;
let is_tbl : boolean = (op == '0');
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	8B
1	16B

<Vn> For the "Single register table" variant: is the name of the SIMD&FP table register, encoded in the "Rn" field.

For the "Four register table", "Three register table", and "Two register table" variants: is the name of the first SIMD&FP table register, encoded in the "Rn" field.

<Vm> Is the name of the SIMD&FP index register, encoded in the "Rm" field.

<Vn+1> Is the name of the second SIMD&FP table register, encoded as "Rn" plus 1 modulo 32.

<Vn+2> Is the name of the third SIMD&FP table register, encoded as "Rn" plus 2 modulo 32.

<Vn+3> Is the name of the fourth SIMD&FP table register, encoded as "Rn" plus 3 modulo 32.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let indices : bits(datasize) = V{datasize}(m);
var table : bits(128*regs) = Zeros{};
var result : bits(datasize);
var index : integer;

// Create table from registers
for i = 0 to regs - 1 do
    table[i*:128] = V{128}((n+i) MOD 32);
end;

result = if is_tbl then Zeros{datasize} else V{datasize}(d);
for i = 0 to elements - 1 do
    index = UInt(indices[i*:8]);
    if index < 16 * regs then
        result[i*:8] = table[index*:8];
    end;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TBX

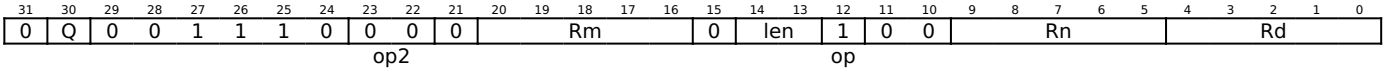
Table vector lookup extension

This instruction reads each value from the vector elements in the index source SIMD&FP register, uses each result as an index to perform a lookup in a table of bytes that is described by one to four source table SIMD&FP registers, places the lookup result in a vector, and writes the vector to the destination SIMD&FP register. If an index is out of range for the table, the existing value in the vector element of the destination register is left unchanged. If more than one source register is used to describe the table, the first source register describes the lowest bytes of the table.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding for the Single register table variant

Applies when `(len == 00)`

```
TBX <Vd>.<Ta>, { <Vn>.16B }, <Vm>.<Ta>
```

Encoding for the Two register table variant

Applies when `(len == 01)`

```
TBX <Vd>.<Ta>, { <Vn>.16B, <Vn+1>.16B }, <Vm>.<Ta>
```

Encoding for the Three register table variant

Applies when `(len == 10)`

```
TBX <Vd>.<Ta>, { <Vn>.16B, <Vn+1>.16B, <Vn+2>.16B }, <Vm>.<Ta>
```

Encoding for the Four register table variant

Applies when `(len == 11)`

```
TBX <Vd>.<Ta>, { <Vn>.16B, <Vn+1>.16B, <Vn+2>.16B, <Vn+3>.16B }, <Vm>.<Ta>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 8;
let regs : integer{} = UInt(len) + 1;
let is_tbl : boolean = (op == '0');
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	8B
1	16B

<Vn> For the "Single register table" variant: is the name of the SIMD&FP table register, encoded in the "Rn" field.

For the "Four register table", "Three register table", and "Two register table" variants: is the name of the first SIMD&FP table register, encoded in the "Rn" field.

<Vm> Is the name of the SIMD&FP index register, encoded in the "Rm" field.

<Vn+1> Is the name of the second SIMD&FP table register, encoded as "Rn" plus 1 modulo 32.

<Vn+2> Is the name of the third SIMD&FP table register, encoded as "Rn" plus 2 modulo 32.

<Vn+3> Is the name of the fourth SIMD&FP table register, encoded as "Rn" plus 3 modulo 32.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let indices : bits(datasize) = V{datasize}(m);
var table : bits(128*regs) = Zeros{};
var result : bits(datasize);
var index : integer;

// Create table from registers
for i = 0 to regs - 1 do
    table[i*:128] = V{128}((n+i) MOD 32);
end;

result = if is_tbl then Zeros{datasize} else V{datasize}(d);
for i = 0 to elements - 1 do
    index = UInt(indices[i*:8]);
    if index < 16 * regs then
        result[i*:8] = table[index*:8];
    end;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TRN1

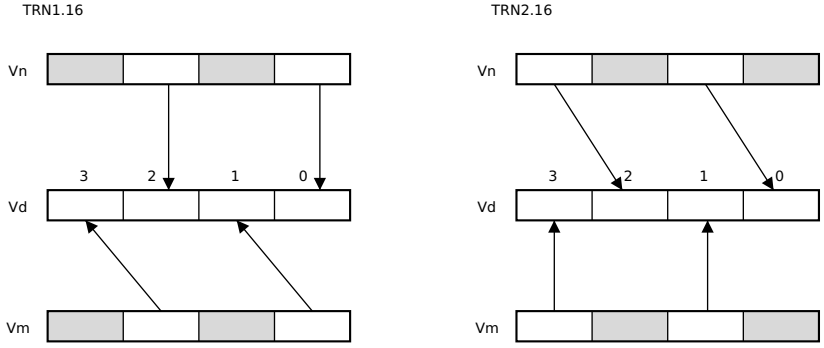
Transpose vectors (primary)

This instruction reads corresponding even-numbered vector elements from the two source SIMD&FP registers, starting at zero, places each result into consecutive elements of a vector, and writes the vector to the destination SIMD&FP register. Vector elements from the first source register are placed into even-numbered elements of the destination vector, starting at zero, while vector elements from the second source register are placed into odd-numbered elements of the destination vector.

Note

By using this instruction with `TRN2`, a 2 x 2 matrix can be transposed.

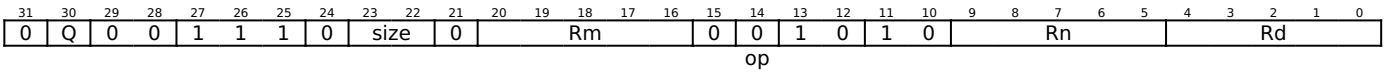
The following figure shows an example of the operation of TRN1 and TRN2 halfword operations where Q = 0.



Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
TRN1 <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let part : integer = UInt(op);
let pairs : integer = elements DIV 2;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{datasize}(n);
let operand2 : bits(datasize) = V{datasize}(m);
var result : bits(datasize);

for p = 0 to pairs-1 do
    result[(2*p+0)*:esize] = operand1[(2*p+part)*:esize];
    result[(2*p+1)*:esize] = operand2[(2*p+part)*:esize];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TRN2

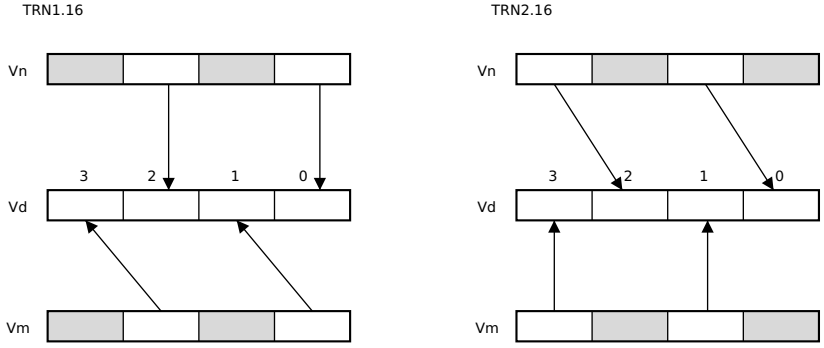
Transpose vectors (secondary)

This instruction reads corresponding odd-numbered vector elements from the two source SIMD&FP registers, places each result into consecutive elements of a vector, and writes the vector to the destination SIMD&FP register. Vector elements from the first source register are placed into even-numbered elements of the destination vector, starting at zero, while vector elements from the second source register are placed into odd-numbered elements of the destination vector.

Note

By using this instruction with `TRN1`, a 2 x 2 matrix can be transposed.

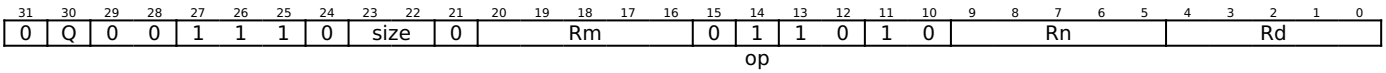
The following figure shows an example of the operation of TRN1 and TRN2 halfword operations where Q = 0.



Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
TRN2 <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let part : integer = UInt(op);
let pairs : integer = elements DIV 2;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{datasize}(n);
let operand2 : bits(datasize) = V{datasize}(m);
var result : bits(datasize);

for p = 0 to pairs-1 do
    result[(2*p+0)*:esize] = operand1[(2*p+part)*:esize];
    result[(2*p+1)*:esize] = operand2[(2*p+part)*:esize];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UABA

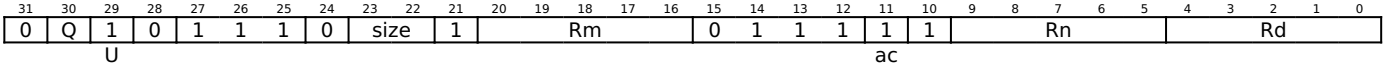
Unsigned absolute difference and accumulate

This instruction subtracts the elements of the vector of the second source SIMD&FP register from the corresponding elements of the first source SIMD&FP register, and accumulates the absolute values of the results into the elements of the vector of the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
UABA <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize) = V{datasize}(d);
var element1 : integer;
var element2 : integer;
var absdiff : bits(esize);

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    absdiff = Abs(element1 - element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + absdiff;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UABAL, UABAL2

Unsigned absolute difference and accumulate long

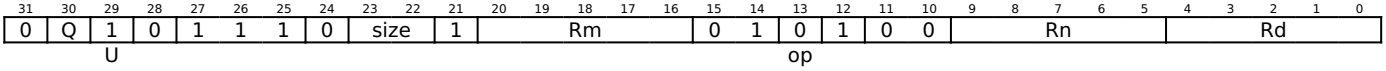
This instruction subtracts the vector elements in the lower or upper half of the second source SIMD&FP register from the corresponding vector elements of the first source SIMD&FP register, and accumulates the absolute values of the results into the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements. All the values in this instruction are unsigned integer values.

The `UABAL` instruction extracts each source vector from the lower half of each source register. The `UABAL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
UABAL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize) = V{2*datasize}(d);
var element1 : integer;
var element2 : integer;
var absdiff : bits(2*esize);

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    absdiff = Abs(element1-element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] + absdiff;
end;
V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UABD

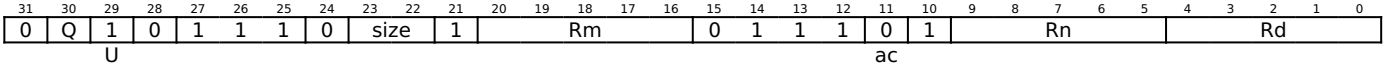
Unsigned absolute difference (vector)

This instruction subtracts the elements of the vector of the second source SIMD&FP register from the corresponding elements of the first source SIMD&FP register, places the absolute values of the results into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
UABD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize) = Zeros{datasize};
var element1 : integer;
var element2 : integer;
var absdiff : bits(esize);

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    absdiff = Abs(element1 - element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + absdiff;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UABDL, UABDL2

Unsigned absolute difference long

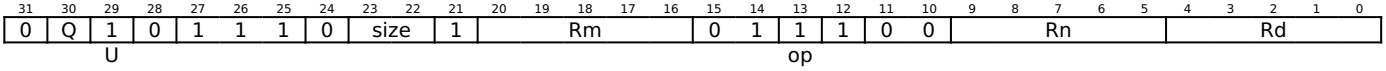
This instruction subtracts the vector elements in the lower or upper half of the second source SIMD&FP register from the corresponding vector elements of the first source SIMD&FP register, places the absolute value of the result into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements. All the values in this instruction are unsigned integer values.

The `UABDL` instruction extracts each source vector from the lower half of each source register. The `UABDL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
UABDL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize) = Zeros{2*datasize};
var element1 : integer;
var element2 : integer;
var absdiff : bits(2*esize);

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    absdiff = Abs(element1-element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] + absdiff;
end;
V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UADALP

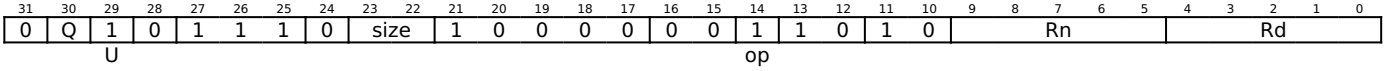
Unsigned add and accumulate long pairwise

This instruction adds pairs of adjacent unsigned integer values from the vector in the source SIMD&FP register and accumulates the results with the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
UADALP <Vd>.<Ta>, <Vn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV (2 * esize);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Ta>
00	0	4H
00	1	8H
01	0	2S
01	1	4S
10	0	1D
10	1	2D
11	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize)  = V{}(d);

var sum : bits(2*esize);
var op1 : integer;
var op2 : integer;

for e = 0 to elements-1 do
    op1 = UInt(operand[(2*e+0)*:esize]);
    op2 = UInt(operand[(2*e+1)*:esize]);
    sum = (op1+op2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] + sum;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UADDL, UADDL2

Unsigned add long (vector)

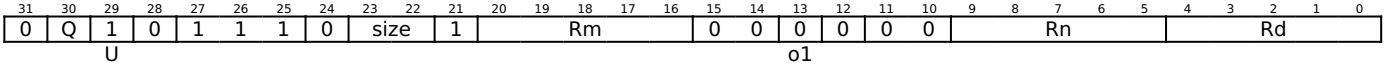
This instruction adds each vector element in the lower or upper half of the first source SIMD&FP register to the corresponding vector element of the second source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements. All the values in this instruction are unsigned integer values.

The `UADDL` instruction extracts each source vector from the lower half of each source register. The `UADDL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
UADDL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    sum = element1 + element2;
    result[e*:(2*esize)] = sum[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UADDLP

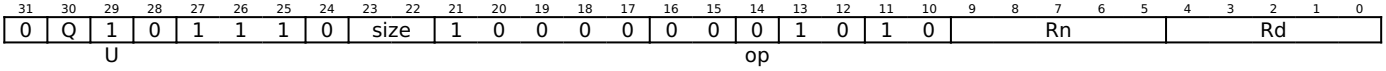
Unsigned add long pairwise

This instruction adds pairs of adjacent unsigned integer values from the vector in the source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
UADDLP <Vd>.<Ta>, <Vn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV (2 * esize);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Ta>
00	0	4H
00	1	8H
01	0	2S
01	1	4S
10	0	1D
10	1	2D
11	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

var sum : bits(2*esize);
var op1 : integer;
var op2 : integer;

for e = 0 to elements-1 do
    op1 = UInt(operand[(2*e+0)*:esize]);
    op2 = UInt(operand[(2*e+1)*:esize]);
    sum = (op1+op2)[2*esize-1:0];
    result[e*:(2*esize)] = sum;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UADDLV

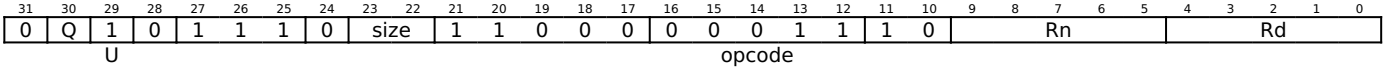
Unsigned sum long across vector

This instruction adds every vector element in the source SIMD&FP register together, and writes the scalar result to the destination SIMD&FP register. The destination scalar is twice as long as the source vector elements. All the values in this instruction are unsigned integer values.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
UADDLV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '100' then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let unsigned : boolean = TRUE;
```

Assembler Symbols

<V> Is the destination width specifier, encoded in “size”:

size	<V>
00	H
01	S
10	D
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	RESERVED
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var opelt : bits(esize) = operand[0+:esize];
var sum : integer = if unsigned then UInt(opelt) else SInt(opelt);

for e = 1 to elements-1 do
    opelt = operand[e*:esize];
    let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
    sum = sum + element;
end;

V{2*esize}(d) = sum[2*esize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UADDW, UADDW2

Unsigned add wide

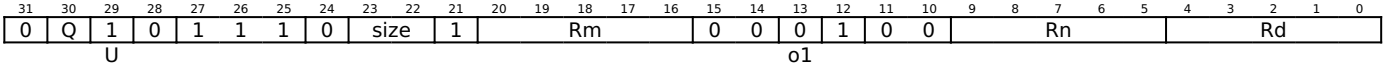
This instruction adds the vector elements of the first source SIMD&FP register to the corresponding vector elements in the lower or upper half of the second source SIMD&FP register, places the result in a vector, and writes the vector to the SIMD&FP destination register. The vector elements of the destination register and the first source register are twice as long as the vector elements of the second source register. All the values in this instruction are unsigned integer values.

The `UADDW` instruction extracts vector elements from the lower half of the second source register. The `UADDW2` instruction extracts vector elements from the upper half of the second source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
UADDW{2} <Vd>.<Ta>, <Vn>.<Ta>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(2*datasize) = V{}(n);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:(2*esize)]);
    element2 = UInt(operand2[e*:esize]);
    sum = element1 + element2;
    result[e*:(2*esize)] = sum[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UCVTF (scalar SIMD&FP)

Unsigned integer convert to floating-point (scalar SIMD&FP)

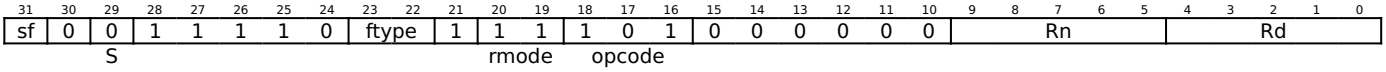
This instruction converts the unsigned integer value in the SIMD&FP source register to a floating-point value using the rounding mode that is specified by the FPCR, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Integer

(FEAT_FPRCVT)



Encoding for the 32-bit to half-precision variant

Applies when `(sf == 0 && ftype == 11)`

```
UCVTF <Hd>, <Sn>
```

Encoding for the 32-bit to double-precision variant

Applies when `(sf == 0 && ftype == 01)`

```
UCVTF <Dd>, <Sn>
```

Encoding for the 64-bit to half-precision variant

Applies when `(sf == 1 && ftype == 11)`

```
UCVTF <Hd>, <Dn>
```

Encoding for the 64-bit to single-precision variant

Applies when `(sf == 1 && ftype == 00)`

```
UCVTF <Sd>, <Dn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FPRCVT) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if (sf == '0' && ftype == '00') || (sf == '1' && ftype == '01') then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let fltsize : integer{} = 8 << UInt(ftype XOR '10');
let rounding : FPRounding = FPRoundingMode(FPCR());
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Sn>	Is the 32-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dn>	Is the 64-bit name of the SIMD&FP source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

Operation

```
AArch64_CheckFPEnabled();

var fltval : bits(fltsize);
var intval : bits(intsize);
let merge : boolean = IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};

intval = V{intsize}(n);
fltval = FixedToFP{fltsize, intsize}(intval, 0, TRUE, FPCR(), rounding);
result[0+:fltsize] = fltval;

V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

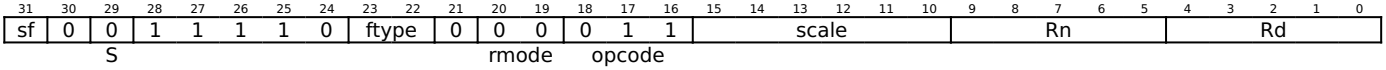
UCVTF (scalar, fixed-point)

Unsigned fixed-point convert to floating-point (scalar)

This instruction converts the unsigned value in the 32-bit or 64-bit general-purpose source register to a floating-point value using the rounding mode that is specified by the FPCR, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the 32-bit to half-precision variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
UCVTF <Hd>, <Wn>, #<fbits>
```

Encoding for the 64-bit to half-precision variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
UCVTF <Hd>, <Xn>, #<fbits>
```

Encoding for the 32-bit to single-precision variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
UCVTF <Sd>, <Wn>, #<fbits>
```

Encoding for the 64-bit to single-precision variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
UCVTF <Sd>, <Xn>, #<fbits>
```

Encoding for the 32-bit to double-precision variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
UCVTF <Dd>, <Wn>, #<fbits>
```

Encoding for the 64-bit to double-precision variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
UCVTF <Dd>, <Xn>, #<fbits>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;
if sf == '0' && scale[5] == '0' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let decode_fltsize : integer{} = 8 << UInt(ftype XOR '10');

let fracbits : integer = 64 - UInt(scale);

let unsigned : boolean = TRUE;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<fbits>	For the "32-bit to double-precision", "32-bit to half-precision", and "32-bit to single-precision" variants: is the number of bits after the binary point in the fixed-point source, in the range 1 to 32, encoded as 64 minus "scale".

	For the "64-bit to double-precision", "64-bit to half-precision", and "64-bit to single-precision" variants: is the number of bits after the binary point in the fixed-point source, in the range 1 to 64, encoded as 64 minus "scale".
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

Operation

```
AArch64_CheckFPEnabled();

let merge : boolean = IsMerging(FPCR());
let fltsize : integer{} = if merge then 128 else decode_fltsize;
var fltval : bits(fltsize) = if merge then V{fltsize}(d) else Zeros{fltsize};
let intval : bits(intsize) = X{}(n);
let rounding : FPRounding = FPRoundingMode(FPCR());

fltval[0+:decode_fltsize] = FixedToFP{decode_fltsize, intsize}(intval, fracbits, unsigned,
                                                               FPCR(), rounding);

V{fltsize}(d) = fltval;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

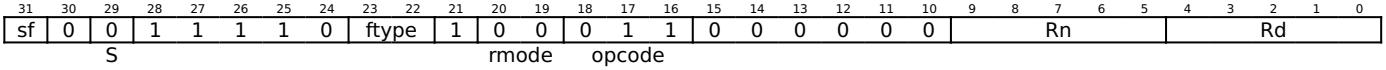
UCVTF (scalar, integer)

Unsigned integer convert to floating-point (scalar)

This instruction converts the unsigned integer value in the general-purpose source register to a floating-point value using the rounding mode that is specified by the FPCR, and writes the result to the SIMD&FP destination register.

This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.



Encoding for the 32-bit to half-precision variant (FEAT_FP16)

Applies when `(sf == 0 && ftype == 11)`

```
UCVTF <Hd>, <Wn>
```

Encoding for the 32-bit to single-precision variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 00)`

```
UCVTF <Sd>, <Wn>
```

Encoding for the 32-bit to double-precision variant (FEAT_FP)

Applies when `(sf == 0 && ftype == 01)`

```
UCVTF <Dd>, <Wn>
```

Encoding for the 64-bit to half-precision variant (FEAT_FP16)

Applies when `(sf == 1 && ftype == 11)`

```
UCVTF <Hd>, <Xn>
```

Encoding for the 64-bit to single-precision variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 00)`

```
UCVTF <Sd>, <Xn>
```

Encoding for the 64-bit to double-precision variant (FEAT_FP)

Applies when `(sf == 1 && ftype == 01)`

```
UCVTF <Dd>, <Xn>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_FP) then EndOfDecode(Decode_UNDEF); end;
if ftype == '10' then EndOfDecode(Decode_UNDEF); end;
if ftype == '11' && !IsFeatureImplemented(FEAT_FP16) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let intsize : integer{} = 32 << UInt(sf);
let decode_fltsize : integer{} = 8 << UInt(ftype XOR '10');
let unsigned : boolean    = TRUE;
```

Assembler Symbols

<Hd>	Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Wn>	Is the 32-bit name of the general-purpose source register, encoded in the "Rn" field.
<Sd>	Is the 32-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Dd>	Is the 64-bit name of the SIMD&FP destination register, encoded in the "Rd" field.
<Xn>	Is the 64-bit name of the general-purpose source register, encoded in the "Rn" field.

Operation

```
AArch64.CheckFPEnabled();

let fltsize : integer{} = if IsMerging(FPCR()) then 128 else decode_fltsize;

let intval : bits(intsize) = X{}(n);
let rounding : FPRounding  = FPRoundingMode(FPCR());
let fracbits : integer = 0;
var fltval : bits(fltsize) = if IsMerging(FPCR()) then V{fltsize}(d) else Zeros{fltsize};
fltval[0+:decode_fltsize] = FixedToFP{decode_fltsize, intsize}(intval, fracbits, unsigned,
                                                               FPCR(), rounding);

V{fltsize}(d) = fltval;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UCVTF (vector, fixed-point)

Unsigned fixed-point convert to floating-point (vector)

This instruction converts each element in a vector from fixed-point to floating-point using the rounding mode that is specified by the FPCR, and writes the result to the SIMD&FP destination register.

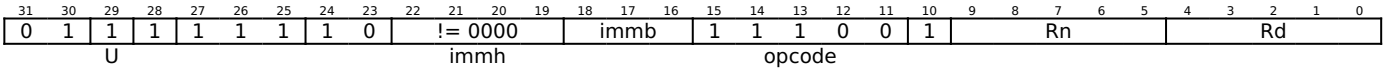
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

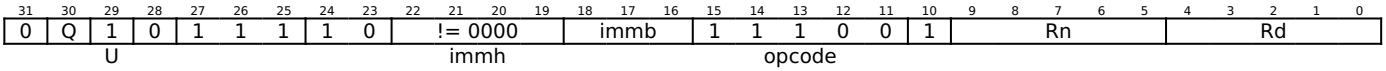
```
UCVTF <V><d>, <V><n>, #<fbits>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh IN {'000x'} || (immh IN {'001x'} && !IsFeatureImplemented(FEAT_FP16)) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = if immh IN {'1xxx'} then 64 else if immh IN {'01xx'} then 32 else 16;
let datasize : integer{} = esize;
let elements : integer = 1;

let fracbits : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
UCVTF <Vd>.<T>, <Vn>.<T>, #<fbits>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh IN {'000x'} || (immh IN {'001x'} && !IsFeatureImplemented(FEAT_FP16)) then
    EndOfDecode(Decode_UNDEF);
end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = if immh IN {'1xxx'} then 64 else if immh IN {'01xx'} then 32 else 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let fracbits : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<V>

Is a width specifier, encoded in “immh”:

immh	<V>	Architectural Feature
0001	RESERVED	-
001x	H	FEAT_FP16
01xx	S	-
1xxx	D	-

<d>

Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<fbits> For the "Scalar" variant: is the number of fractional bits, in the range 1 to the operand width, encoded in “(immh :: immb)”:

immh	<fbits>
0001	RESERVED
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

For the "Vector" variant: is the number of fractional bits, in the range 1 to the element width, encoded in “(immh :: immb)”:

immh	<fbits>
0001	RESERVED
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>	Architectural Feature
0001	x	RESERVED	-
001x	0	4H	FEAT_FP16
001x	1	8H	FEAT_FP16
01xx	0	2S	-
01xx	1	4S	-
1xxx	0	RESERVED	-
1xxx	1	2D	-

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
let rounding : FPRounding = FPRoundingMode(FPCR());

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FixedToFP{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UCVTF (vector, integer)

Unsigned integer convert to floating-point (vector)

This instruction converts each element in a vector from an unsigned integer value to a floating-point value using the rounding mode that is specified by the FPCR, and writes the result to the SIMD&FP destination register.

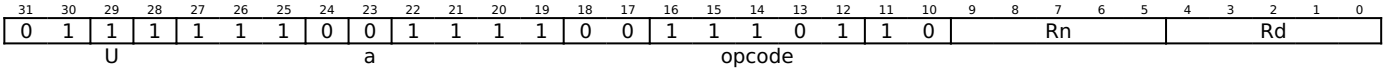
This instruction can generate a floating-point exception. Depending on the settings in FPCR, the exception results in either a flag being set in FPSR or a synchronous exception being generated. For more information, see *Floating-point exceptions and exception traps*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 4 classes: Scalar half-precision , Scalar single-precision and double-precision , Vector half-precision and Vector single-precision and double-precision

Scalar half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

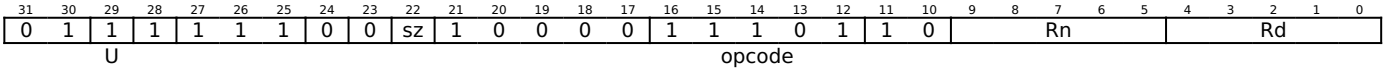
```
UCVTF <Hd>, <Hn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let esize : integer{} = 16;
let datasize : integer{} = esize;
let elements : integer = 1;

let unsigned : boolean = TRUE;
```

Scalar single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

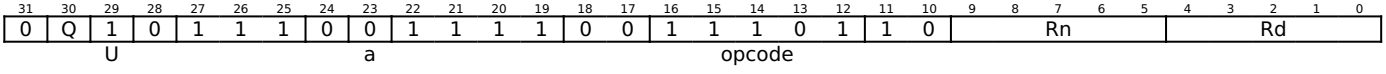
```
UCVTF <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = esize;
let elements : integer = 1;
let unsigned : boolean = TRUE;
```

Vector half-precision

(FEAT_AdvSIMD && FEAT_FP16)



Encoding

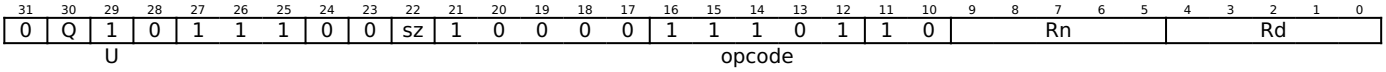
```
UCVTF <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) || !IsFeatureImplemented(FEAT_FP16) then
    EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

let esize : integer{} = 16;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let unsigned : boolean = TRUE;
```

Vector single-precision and double-precision

(FEAT_AdvSIMD)



Encoding

```
UCVTF <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

if sz::Q == '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32 << UInt(sz);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let unsigned : boolean = TRUE;
```

Assembler Symbols

<Hd> Is the 16-bit name of the SIMD&FP destination register, encoded in the "Rd" field.

<Hn> Is the 16-bit name of the SIMD&FP source register, encoded in the "Rn" field.

<V> Is a width specifier, encoded in “sz”:

sz	<V>
0	S
1	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> For the "Vector half-precision" variant: is an arrangement specifier, encoded in “Q”:

Q	<T>
0	4H
1	8H

For the "Vector single-precision and double-precision" variant: is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	0	RESERVED
1	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
if elements == 1 && IsFeatureImplemented(FEAT_FPRCVT) then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;

let operand : bits(datasize) = V{}(n);

let merge : boolean = elements == 1 && IsMerging(FPCR());
var result : bits(128) = if merge then V{128}(d) else Zeros{128};
let rounding : FPRounding = FPRoundingMode(FPCR());
let fracbits : integer = 0;
var element : bits(esize);

for e = 0 to elements-1 do
    element = operand[e*:esize];
    result[e*:esize] = FixedToFP{esize, esize}(element, fracbits, unsigned, FPCR(), rounding);
end;

V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (by element)

Dot product unsigned arithmetic (vector, by element)

This instruction performs the dot product of the four 8-bit elements in each 32-bit element of the first source register with the four 8-bit elements of an indexed 32-bit element in the second source register, accumulating the result into the corresponding 32-bit element of the destination register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

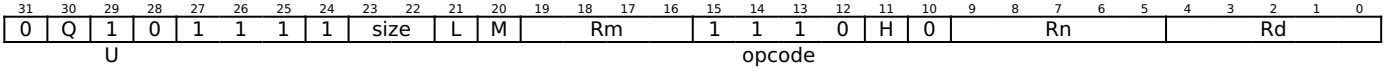
In Armv8.2 and Armv8.3, this is an OPTIONAL instruction. From Armv8.4 it is mandatory for all implementations to support it.

Note

ID_AA64ISAR0_EL1.DP indicates whether this instruction is supported.

Vector

(FEAT_DotProd)



Encoding

```
UDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.4B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_DotProd) then EndOfDecode(Decode_UNDEF); end;
if size  != '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(M::Rm);
let index : integer = UInt(H::L);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> Is the element index, encoded in the "H:L" fields.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(128) = V{}(m);
var result : bits(datasize) = V{}(d);
for e = 0 to elements-1 do
    var res : integer = 0;
    var element1, element2 : integer;
    for i = 0 to 3 do
        element1 = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        element2 = UInt(operand2[(4 * index + i)*:(esize DIV 4)]);

        res = res + element1 * element2;
    end;
    result[e*:esize] = result[e*:esize] + res;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (vector)

Dot product unsigned arithmetic (vector)

This instruction performs the dot product of the four unsigned 8-bit elements in each 32-bit element of the first source register with the four unsigned 8-bit elements of the corresponding 32-bit element in the second source register, accumulating the result into the corresponding 32-bit element of the destination register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

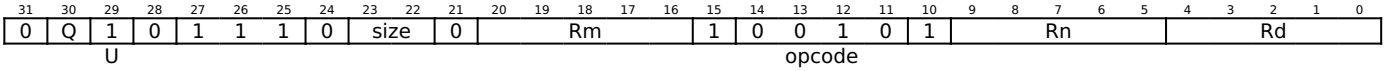
In Armv8.2 and Armv8.3, this is an OPTIONAL instruction. From Armv8.4 it is mandatory for all implementations to support it.

Note

ID_AA64ISAR0_EL1.DP indicates whether this instruction is supported.

Vector

(FEAT_DotProd)



Encoding

```
UDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_DotProd) then EndOfDecode(Decode_UNDEF); end;
if size != '10' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

result = V{datasize}(d);
for e = 0 to elements-1 do
    var res : integer = 0;
    var element1, element2 : integer;
    for i = 0 to 3 do
        element1 = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        element2 = UInt(operand2[(4 * e + i)*:(esize DIV 4)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = result[e*:esize] + res;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UHADD

Unsigned halving add

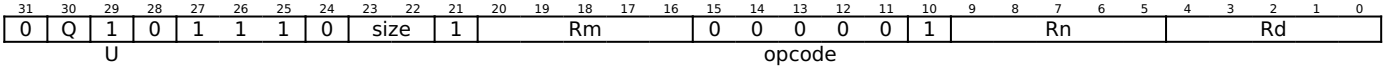
This instruction adds corresponding unsigned integer values from the two source SIMD&FP registers, shifts each result right one bit, places the results into a vector, and writes the vector to the destination SIMD&FP register.

The results are truncated. For rounded results, see *URHADD*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
UHADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    sum = (element1 + element2) >> 1;
    result[e*:esize] = sum[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UHSUB

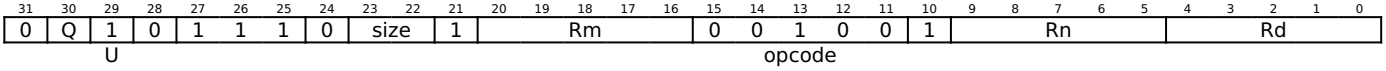
Unsigned halving subtract

This instruction subtracts the vector elements in the second source SIMD&FP register from the corresponding vector elements in the first source SIMD&FP register, shifts each result right one bit, places each result into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
UHSUB <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var diff : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    diff = (element1 - element2) >> 1;
    result[e*:esize] = diff[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMAX

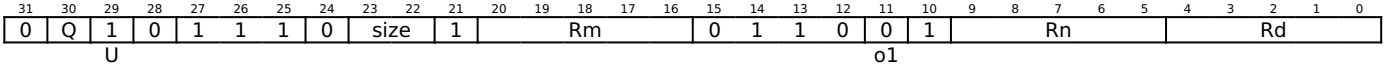
Unsigned maximum (vector)

This instruction compares corresponding elements in the vectors in the two source SIMD&FP registers, places the larger of each pair of unsigned integer values into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
UMAX <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var max : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    max = Max(element1, element2);
    result[e*:esize] = max[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMAXP

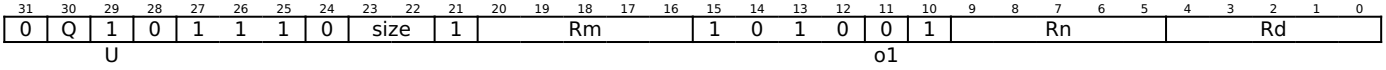
Unsigned maximum pairwise

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements in the two source SIMD&FP registers, writes the largest of each pair of unsigned integer values into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
UMAXP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : integer;
var element2 : integer;
var max : integer;

for e = 0 to elements-1 do
    element1 = UInt(concat[(2*e)*:esize]);
    element2 = UInt(concat[((2*e)+1)*:esize]);
    max = Max(element1, element2);
    result[e*:esize] = max[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMAXV

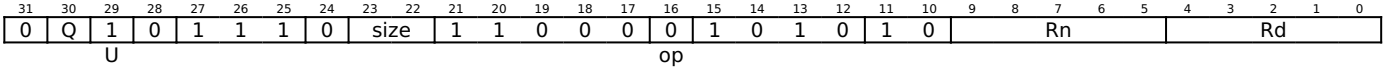
Unsigned maximum across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the largest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are unsigned integer values.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
UMAXV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '100' then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let unsigned : boolean = TRUE;
```

Assembler Symbols

<V> Is the destination width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	RESERVED
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var opelt : bits(esize) = operand[0+:esize];
var max : integer = if unsigned then UInt(opelt) else SInt(opelt);

for e = 1 to elements-1 do
    opelt = operand[e*:esize];
    let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
    max = Max(max, element);
end;

V{esize}(d) = max[esize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMIN

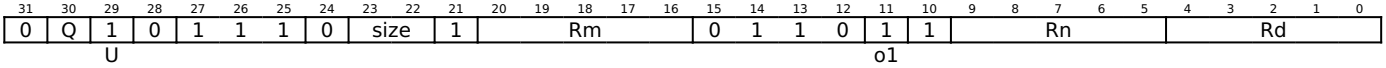
Unsigned minimum (vector)

This instruction compares corresponding vector elements in the two source SIMD&FP registers, places the smaller of each of the two unsigned integer values into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
UMIN <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var min : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    min = Min(element1, element2);
    result[e*:esize] = min[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMINP

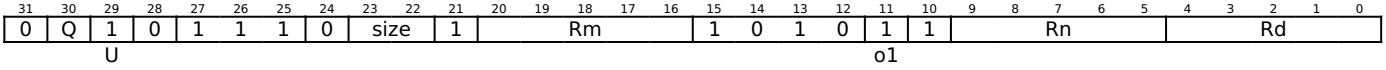
Unsigned minimum pairwise

This instruction creates a vector by concatenating the vector elements of the first source SIMD&FP register after the vector elements of the second source SIMD&FP register, reads each pair of adjacent vector elements in the two source SIMD&FP registers, writes the smallest of each pair of unsigned integer values into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
UMINP <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
let concat : bits(2*datasize) = operand2::operand1;
var element1 : integer;
var element2 : integer;
var min : integer;

for e = 0 to elements-1 do
    element1 = UInt(concat[(2*e)*:esize]);
    element2 = UInt(concat[((2*e)+1)*:esize]);
    min = Min(element1, element2);
    result[e*:esize] = min[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMINV

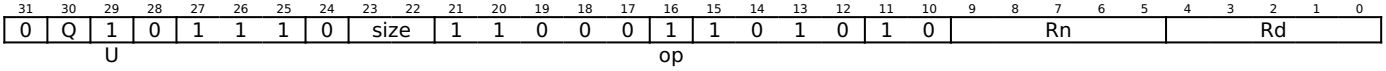
Unsigned minimum across vector

This instruction compares all the vector elements in the source SIMD&FP register, and writes the smallest of the values as a scalar to the destination SIMD&FP register. All the values in this instruction are unsigned integer values.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
UMINV <V><d>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '100' then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let unsigned : boolean = TRUE;
```

Assembler Symbols

<V> Is the destination width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	RESERVED
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var opelt : bits(esize) = operand[0+:esize];
var min : integer = if unsigned then UInt(opelt) else SInt(opelt);

for e = 1 to elements-1 do
    opelt = operand[e*:esize];
    let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
    min = Min(min, element);
end;

V{esize}(d) = min[esize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLAL, UMLAL2 (by element)

Unsigned multiply-add long (vector, by element)

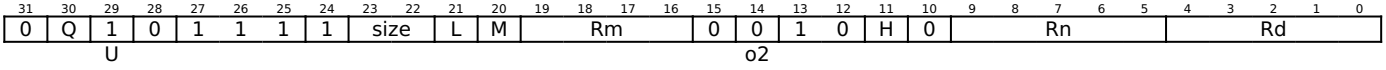
This instruction multiplies each vector element in the lower or upper half of the first source SIMD&FP register by the specified vector element of the second source SIMD&FP register and accumulates the results with the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

The `UMLAL` instruction extracts vector elements from the lower half of the first source register. The `UMLAL2` instruction extracts vector elements from the upper half of the first source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
UMLAL{2} <Vd>.<Ta>, <Vn>.<Tb>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
let element2 : integer = UInt(operand2[index*:esize]);
var product : bits(2*esize);

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    product = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = operand3[e*:(2*esize)] + product;
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLAL, UMLAL2 (vector)

Unsigned multiply-add long (vector)

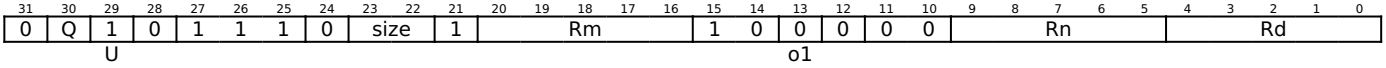
This instruction multiplies the vector elements in the lower or upper half of the first source SIMD&FP register by the corresponding vector elements of the second source SIMD&FP register, and accumulates the results with the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

The `UMLAL` instruction extracts vector elements from the lower half of the first source register. The `UMLAL2` instruction extracts vector elements from the upper half of the first source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
UMLAL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var product : bits(2*esize);
var accum : bits(2*esize);

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    product = (element1 * element2)[2*esize-1:0];
    accum = operand3[e*:(2*esize)] + product;
    result[e*:(2*esize)] = accum;
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLSL, UMLSL2 (by element)

Unsigned multiply-subtract long (vector, by element)

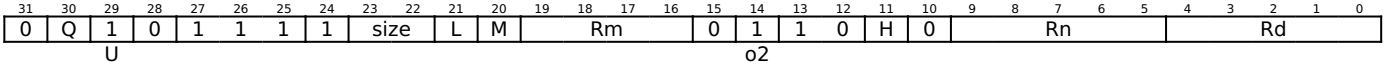
This instruction multiplies each vector element in the lower or upper half of the first source SIMD&FP register by the specified vector element of the second source SIMD&FP register and subtracts the results from the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

The `UMLSL` instruction extracts vector elements from the lower half of the first source register. The `UMLSL2` instruction extracts vector elements from the upper half of the first source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
UMLSL{2} <Vd>.<Ta>, <Vn>.<Tb>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(idxdsize) = V{}(m);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
let element2 : integer = UInt(operand2[index*:esize]);
var product : bits(2*esize);

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    product = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = operand3[e*:(2*esize)] - product;
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLSL, UMLSL2 (vector)

Unsigned multiply-subtract long (vector)

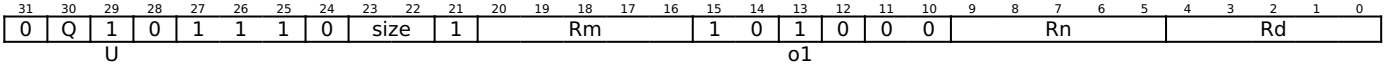
This instruction multiplies corresponding vector elements in the lower or upper half of the two source SIMD&FP registers, and subtracts the results from the vector elements of the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied. All the values in this instruction are unsigned integer values.

The `UMLSL` instruction extracts each source vector from the lower half of each source register. The `UMLSL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
UMLSL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
let operand3 : bits(2*datasize) = V{}(d);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var product : bits(2*esize);
var accum : bits(2*esize);

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    product = (element1 * element2)[2*esize-1:0];
    accum = operand3[e*:(2*esize)] - product;
    result[e*:(2*esize)] = accum;
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMMLA (vector)

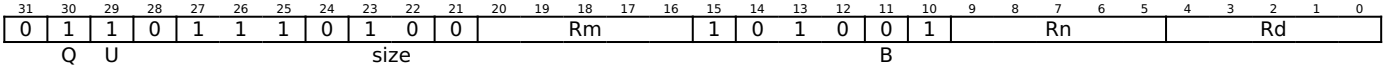
Unsigned 8-bit integer matrix multiply-accumulate to 32-bit integer (vector)

This instruction multiplies the 2x8 matrix of unsigned 8-bit integer values in the first source vector by the 8x2 matrix of unsigned 8-bit integer values in the second source vector. The resulting 2x2 32-bit integer matrix product is destructively added to the 32-bit integer matrix accumulator in the destination vector. This is equivalent to performing an 8-way dot product per destination element.

From Armv8.2 to Armv8.5, this is an OPTIONAL instruction. From Armv8.6 it is mandatory for implementations that include Advanced SIMD to support it. ID_AA64ISAR1_EL1.I8MM indicates whether this instruction is supported.

Vector

(FEAT_I8MM)



Encoding

```
UMMLA <Vd>.4S, <Vn>.16B, <Vm>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_I8MM) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let addend : bits(128)   = V{}(d);

V{128}(d) = MatMulAdd(addend, operand1, operand2, op1_unsigned, op2_unsigned);
```

Operational information

Arm expects that the UMMLA (vector) instruction will deliver a peak integer multiply throughput that is at least as high as can be achieved using two UDOT (vector) instructions, with a goal that it should have significantly higher throughput.

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMOV

Unsigned move vector element to general-purpose register

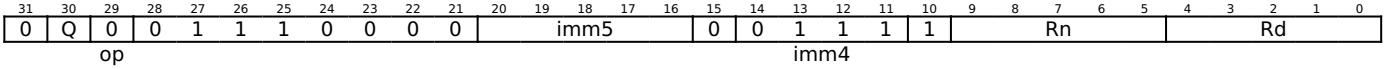
This instruction reads the unsigned integer from the source SIMD&FP register, zero-extends it to form a 32-bit or 64-bit value, and writes the result to the destination general-purpose register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This instruction is used by the alias MOV (to general).

Advanced SIMD

(FEAT_AdvSIMD)



Encoding for the 32-bit variant

Applies when `(Q == 0)`

```
UMOV <Wd>, <Vn>.<Ts>[<index>]
```

Encoding for the 64-bit variant

Applies when `(Q == 1 && imm5 == x1000)`

```
UMOV <Xd>, <Vn>.D[<index>]
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if imm5 == 'x0000' then EndOfDecode(Decode_UNDEF); end;

let size : integer{} = LowestSetBitNZ(imm5[3:0]);

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << size;
let datasize : integer{} = 32 << UInt(Q);

if datasize == 64 && esize < 64 then EndOfDecode(Decode_UNDEF); end;
if datasize == 32 && esize >= 64 then EndOfDecode(Decode_UNDEF); end;

let index : integer = UInt(imm5[4:size+1]);
let idxdsize : integer{} = 64 << UInt(imm5[4]);
```

Assembler Symbols

<Wd> Is the 32-bit name of the general-purpose destination register, encoded in the "Rd" field.

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ts> Is an element size specifier, encoded in “imm5”:

imm5	<Ts>
xx000	RESERVED
xxxx1	B
xxx10	H
xx100	S

<index> For the "32-bit" variant: is the element index encoded in “imm5”:

imm5	<index>
xx000	RESERVED
xxxx1	UInt(imm5[4:1])
xxx10	UInt(imm5[4:2])
xx100	UInt(imm5[4:3])

For the "64-bit" variant: is the element index encoded in "imm5<4>".

<Xd> Is the 64-bit name of the general-purpose destination register, encoded in the "Rd" field.

Alias Conditions

Alias	Of variant	Is preferred when
MOV (to general)	32-bit	`imm5 IN {'xx100'}`

Alias	Of variant	Is preferred when
MOV (to general)	64-bit	`imm5 IN {'x1000'}`

Operation

```
if index == 0 then
    AArch64_CheckFPEnabled();
else
    AArch64_CheckFPAdvSIMDEnabled();
end;
let operand : bits(idxdsize) = V{idxdsize}(n);

X{datasize}(d) = ZeroExtend{datasize}(operand[index*:esize]);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMULL, UMULL2 (by element)

Unsigned multiply long (vector, by element)

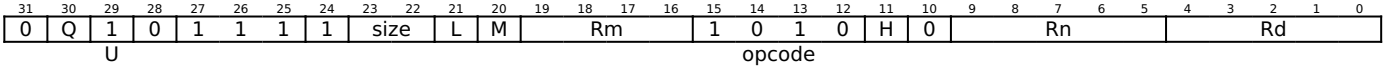
This instruction multiplies each vector element in the lower or upper half of the first source SIMD&FP register by the specified vector element of the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied.

The `UMULL` instruction extracts vector elements from the lower half of the first source register. The `UMULL2` instruction extracts vector elements from the upper half of the first source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
UMULL{2} <Vd>.<Ta>, <Vn>.<Tb>, V<m>.<Ts>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let idxdsize : integer{} = 64 << UInt(H);
var index : integer;
var Rmhi : bit;
case size of
    when '01' => index = UInt(H::L::M); Rmhi = '0';
    when '10' => index = UInt(H::L); Rmhi = M;
    otherwise => EndOfDecode(Decode_UNDEF);
end;

let d : integer = UInt(Rd);
let n : integer = UInt(Rn);
let m : integer = UInt(Rmhi::Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	RESERVED
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	x	RESERVED
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<m> Is the number of the second SIMD&FP source register, encoded in “(size :: M :: Rm)”:

size	<m>
00	RESERVED
01	UInt('0' :: Rm)
10	UInt(M :: Rm)
11	RESERVED

Restricted to 0-15 when element size <Ts> is H.

<Ts> Is an element size specifier, encoded in “size”:

size	<Ts>
00	RESERVED
01	H
10	S
11	RESERVED

<index> Is the element index, encoded in “(size :: H :: L :: M)”:

size	<index>
00	RESERVED
01	UInt(H :: L :: M)
10	UInt(H :: L)
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(idxdsize) = V{}(m);
var result : bits(2*datasize);
var element1 : integer;
let element2 : integer = UInt(operand2[index*:esize]);
var product : bits(2*esize);

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    product = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = product;
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMULL, UMULL2 (vector)

Unsigned multiply long (vector)

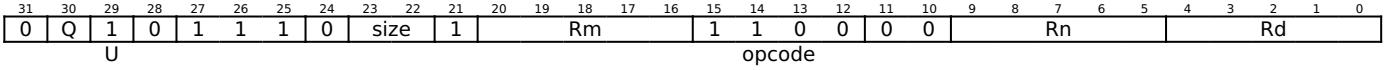
This instruction multiplies corresponding vector elements in the lower or upper half of the two source SIMD&FP registers, places the result in a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the elements that are multiplied. All the values in this instruction are unsigned integer values.

The `UMULL` instruction extracts each source vector from the lower half of each source register. The `UMULL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
UMULL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    result[e*:(2*esize)] = (element1 * element2)[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQADD

Unsigned saturating add

This instruction adds the values of corresponding elements of the two source SIMD&FP registers, places the results into a vector, and writes the vector to the destination SIMD&FP register.

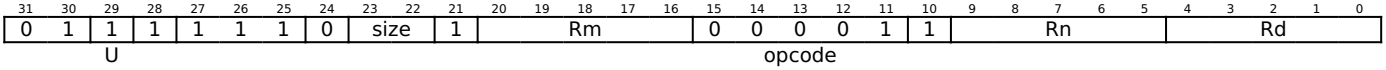
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

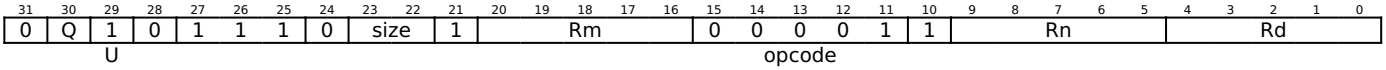
```
UQADD <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let unsigned : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
UQADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    sum = element1 + element2;
    (result[e*:esize], sat) = SatQ{esize}(sum, unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQRSHL

Unsigned saturating rounding shift left (register)

This instruction takes each vector element of the first source SIMD&FP register, shifts the vector element by a value from the least significant byte of the corresponding vector element of the second source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register.

If the shift value is positive, the operation is a left shift. Otherwise, it is a right shift. The results are rounded. For truncated results, see *UQSHL*.

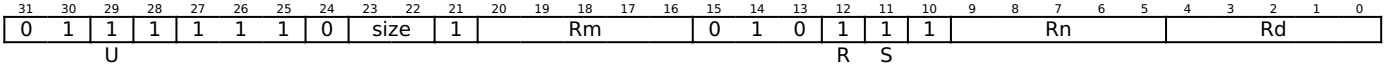
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

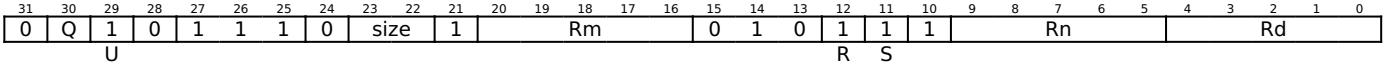
```
UQRSHL <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if S == '0' && size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let unsigned : boolean = TRUE;
let rounding : boolean = TRUE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
UQRSHL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let unsigned : boolean = TRUE;
let rounding : boolean = TRUE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    var element : integer = UInt(operand1[e*:esize]);
    var shift : integer = ShiftSat(SInt(operand2[e*:esize][7:0]), esize);
    if shift >= 0 then // left shift
        element = element << shift;
    else               // right shift
        shift = -shift;
        element = RShr(element, shift, rounding);
    end;

    (result[e*:esize], sat) = SatQ{esize}(element, unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQRSHRN, UQRSHRN2

Unsigned saturating rounded shift right narrow (immediate)

This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, puts the final result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. All the values in this instruction are unsigned integer values. The results are rounded. For truncated results, see *UQSHRN*.

The `UQRSHRN` instruction writes the vector to the lower half of the destination register and clears the upper half. The `UQRSHRN2` instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

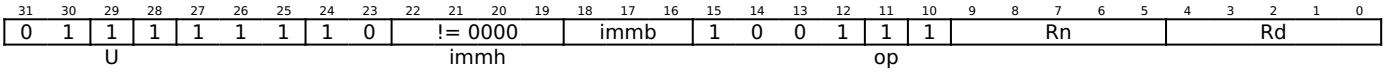
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

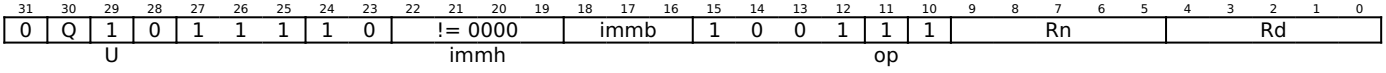
```
UQRSHRN <Vb><d>, <Va><n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = TRUE;
let unsigned : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
UQRSHRN{2} <Vd>.<Tb>, <Vn>.<Ta>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = TRUE;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Vb> Is the destination width specifier, encoded in “immh”:

immh	<Vb>
0001	B
001x	H
01xx	S
1xxx	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Va> Is the source width specifier, encoded in “immh”:

immh	<Va>
0001	H
001x	S
01xx	D
1xxx	RESERVED

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to the destination operand width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

For the "Vector" variant: is the right shift amount, in the range 1 to the destination element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize*2) = V{}(n);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    let opelt : bits(2*esize) = operand[e*:(2*esize)];
    if unsigned then
        let element : integer = RShr(UInt(opelt), shift, round);
        (result[e*:esize], sat) = UnsignedSatQ{esize}(element);
    else
        let element : integer = RShr(SInt(opelt), shift, round);
        (result[e*:esize], sat) = SignedSatQ{esize}(element);
    end;
    if sat then FPSR().QC = '1'; end;
end;

Vpart{datasize}(d, part) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQSHL (immediate)

Unsigned saturating shift left (immediate)

This instruction takes each vector element in the source SIMD&FP register, shifts it by an immediate value, places the results in a vector, and writes the vector to the destination SIMD&FP register. The results are truncated. For rounded results, see *UQRSHL*.

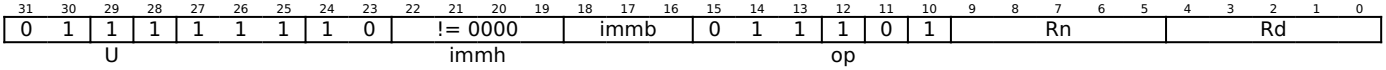
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

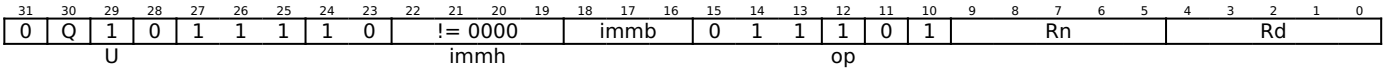
```
UQSHL <V><d>, <V><n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = esize;
let elements : integer = 1;
let shift : integer = UInt(immh::immb) - esize;

let src_unsigned : boolean = TRUE;
let dst_unsigned : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
UQSHL <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let shift : integer = UInt(immh::immb) - esize;

let src_unsigned : boolean = TRUE;
let dst_unsigned : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “immh”:

immh	<V>
0001	B
001x	H
01xx	S
1xxx	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the left shift amount, in the range 0 to the operand width in bits minus 1, encoded in “(immh :: immb)”:

immh	<shift>
0001	UInt(immh :: immb) - 8
001x	UInt(immh :: immb) - 16
01xx	UInt(immh :: immb) - 32
1xxx	UInt(immh :: immb) - 64

For the "Vector" variant: is the left shift amount, in the range 0 to the element width in bits minus 1, encoded in “(immh :: immb)”:

immh	<shift>
0001	UInt(immh :: immb) - 8
001x	UInt(immh :: immb) - 16
01xx	UInt(immh :: immb) - 32
1xxx	UInt(immh :: immb) - 64

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    let element : integer = if src_unsigned then UInt(opelt) else SInt(opelt);
    (result[e*:esize], sat) = SatQ{esize}(element << shift, dst_unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQSHL (register)

Unsigned saturating shift left (register)

This instruction takes each element in the vector of the first source SIMD&FP register, shifts the element by a value from the least significant byte of the corresponding element of the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register.

If the shift value is positive, the operation is a left shift. Otherwise, it is a right shift. The results are truncated. For rounded results, see *UQRSHL*.

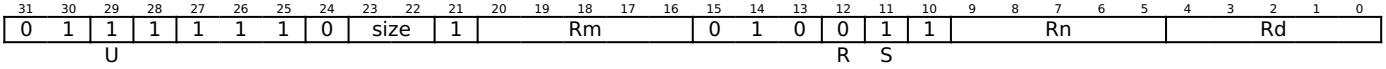
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

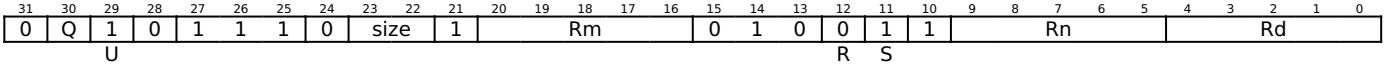
```
UQSHL <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if S == '0' && size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let unsigned : boolean = TRUE;
let rounding : boolean = FALSE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
UQSHL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let unsigned : boolean = TRUE;
let rounding : boolean = FALSE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    var element : integer = UInt(operand1[e*:esize]);
    var shift : integer = ShiftSat(SInt(operand2[e*:esize][7:0]), esize);
    if shift >= 0 then // left shift
        element = element << shift;
    else               // right shift
        shift = -shift;
        element = RShr(element, shift, rounding);
    end;

    (result[e*:esize], sat) = SatQ{esize}(element, unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQSHRN, UQSHRN2

Unsigned saturating shift right narrow (immediate)

This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, saturates each shifted result to a value that is half the original width, puts the final result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. All the values in this instruction are unsigned integer values. The results are truncated. For rounded results, see *UQRSHRN*.

The `UQSHRN` instruction writes the vector to the lower half of the destination register and clears the upper half. The `UQSHRN2` instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

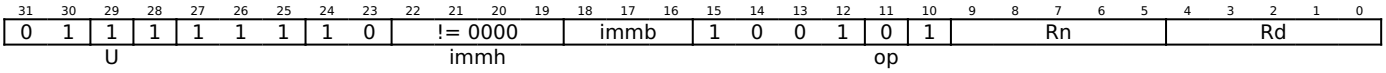
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

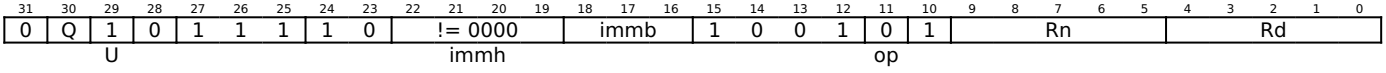
```
UQSHRN <Vb><d>, <Va><n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = esize;
let elements : integer = 1;
let part : integer = 0;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = FALSE;
let unsigned : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
UQSHRN{2} <Vd>.<Tb>, <Vn>.<Ta>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (2 * esize) - UInt(immh::immb);
let round : boolean = FALSE;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Vb> Is the destination width specifier, encoded in “immh”:

immh	<Vb>
0001	B
001x	H
01xx	S
1xxx	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Va> Is the source width specifier, encoded in “immh”:

immh	<Va>
0001	H
001x	S
01xx	D
1xxx	RESERVED

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to the destination operand width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

For the "Vector" variant: is the right shift amount, in the range 1 to the destination element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	RESERVED

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize*2) = V{}(n);
var result : bits(datasize);
var sat : boolean;

for e = 0 to elements-1 do
    let opelt : bits(2*esize) = operand[e*:(2*esize)];
    if unsigned then
        let element : integer = RShr(UInt(opelt), shift, round);
        (result[e*:esize], sat) = UnsignedSatQ{esize}(element);
    else
        let element : integer = RShr(SInt(opelt), shift, round);
        (result[e*:esize], sat) = SignedSatQ{esize}(element);
    end;
    if sat then FPSR().QC = '1'; end;
end;

Vpart{datasize}(d, part) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQSUB

Unsigned saturating subtract

This instruction subtracts the element values of the second source SIMD&FP register from the corresponding element values of the first source SIMD&FP register, places the results into a vector, and writes the vector to the destination SIMD&FP register.

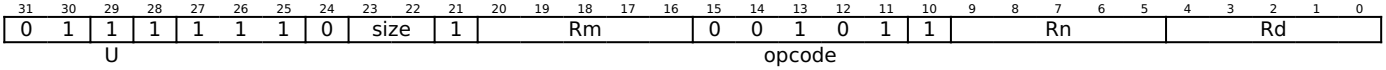
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

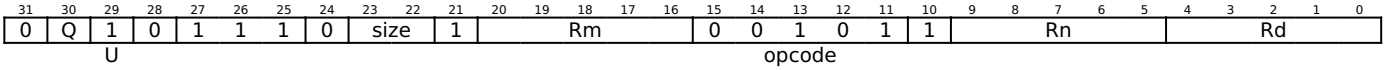
```
UQSUB <V><d>, <V><n>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let unsigned : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
UQSUB <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<m> Is the number of the second SIMD&FP source register, encoded in the "Rm" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var diff : integer;
var sat : boolean;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    diff = element1 - element2;
    (result[e*:esize], sat) = SatQ{esize}(diff, unsigned);
    if sat then FPSR().QC = '1'; end;
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQXTN, UQXTN2

Unsigned saturating extract narrow

This instruction reads each vector element from the source SIMD&FP register, saturates each value to half the original width, places the result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are unsigned integer values.

If saturation occurs, the cumulative saturation bit FPSR.QC is set.

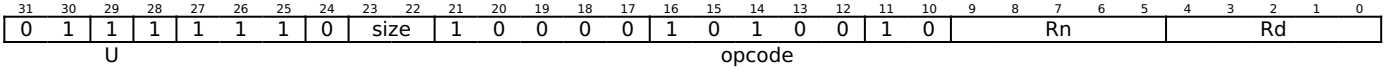
The `UQXTN` instruction writes the vector to the lower half of the destination register and clears the upper half. The `UQXTN2` instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

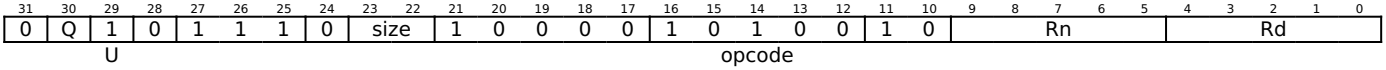
```
UQXTN <Vb><d>, <Va><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let part : integer = 0;
let elements : integer = 1;
let unsigned : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
UQXTN{2} <Vd>.<Tb>, <Vn>.<Ta>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Vb> Is the destination width specifier, encoded in “size”:

size	<Vb>
00	B
01	H
10	S
11	RESERVED

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<Va> Is the source width specifier, encoded in “size”:

size	<Va>
00	H
01	S
10	D
11	RESERVED

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

2 Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(2*datasize) = V{}(n);
var result : bits(datasize);
var element : bits(2*esize);
var sat : boolean;

for e = 0 to elements-1 do
    element = operand[e*:(2*esize)];
    (result[e*:esize], sat) = SatQ{esize}(UInt(element), unsigned);
    if sat then FPSR().QC = '1'; end;
end;

Vpart{datasize}(d, part) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

URECPE

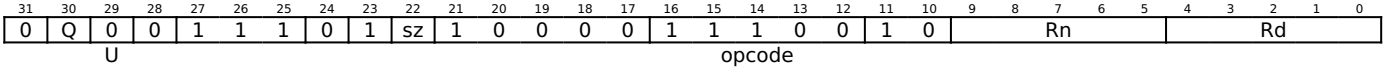
Unsigned reciprocal estimate

This instruction reads each vector element from the source SIMD&FP register, calculates an approximate inverse for the unsigned integer value, places the result into a vector, and writes the vector to the destination SIMD&FP register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
URECPE <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

if sz == '1' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(32);

for e = 0 to elements-1 do
    element = operand[e*:32];
    result[e*:32] = UnsignedRecipEstimate{32}(element);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

URHADD

Unsigned rounding halving add

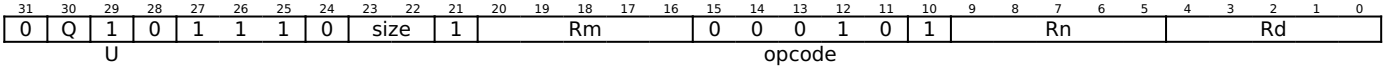
This instruction adds corresponding unsigned integer values from the two source SIMD&FP registers, shifts each result right one bit, places the results into a vector, and writes the vector to the destination SIMD&FP register.

The results are rounded. For truncated results, see *UHADD*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers of the same type

(FEAT_AdvSIMD)



Encoding

```
URHADD <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    sum = (element1 + element2 + 1) >> 1;
    result[e*:esize] = sum[esize-1:0];
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

URSHL

Unsigned rounding shift left (register)

This instruction takes each element in the vector of the first source SIMD&FP register, shifts the vector element by a value from the least significant byte of the corresponding element of the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register.

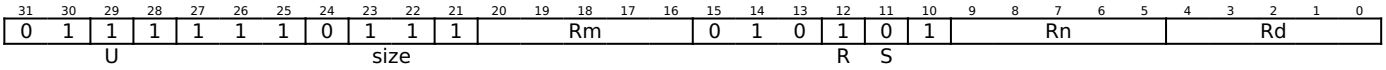
If the shift value is positive, the operation is a left shift. If the shift value is negative, it is a rounding right shift.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

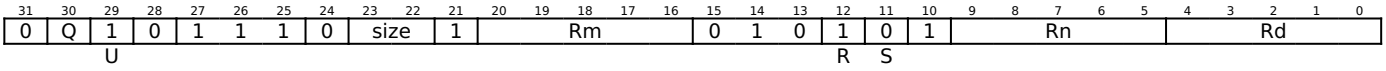
```
URSHL D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if S == '0' && size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let rounding : boolean = TRUE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
URSHL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let rounding : boolean = TRUE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the first SIMD&FP source register, encoded in the "Rn" field.
<m>	Is the number of the second SIMD&FP source register, encoded in the "Rm" field.
<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

for e = 0 to elements-1 do
    var element : integer = UInt(operand1[e*:esize]);
    var shift : integer = ShiftSat(SInt(operand2[e*:esize][7:0]), esize);
    if shift >= 0 then // left shift
        element = element << shift;
    else               // right shift
        shift = -shift;
        element = RShr(element, shift, rounding);
    end;

    result[e*:esize] = element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

URSHR

Unsigned rounding shift right (immediate)

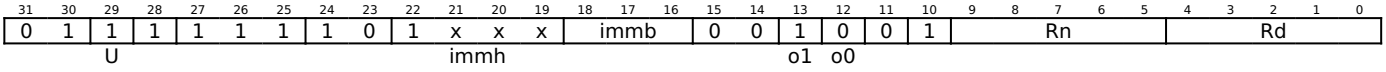
This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, writes the final result to a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are unsigned integer values. The results are rounded. For truncated results, see *USHR*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

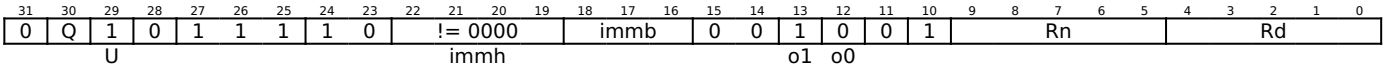
```
URSHR D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
let round : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
URSHR <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
let round : boolean = TRUE;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to 64, encoded as 128 - UInt("immh:immb").

For the "Vector" variant: is the right shift amount, in the range 1 to the element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    if unsigned then
        element = RShr(UInt(opelt), shift, round);
    else
        element = RShr(SInt(opelt), shift, round);
    end;
    result[e*:esize] = element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

URSQRTE

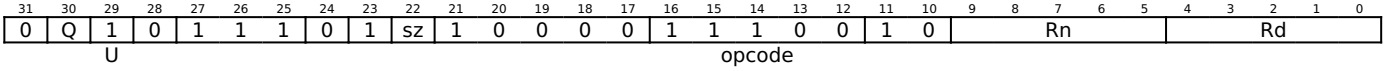
Unsigned reciprocal square root estimate

This instruction reads each vector element from the source SIMD&FP register, calculates an approximate inverse square root for each value, places the result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are unsigned integer values.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
URSQRTE <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let n : integer = UInt(Rn);

if sz == '1' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(sz :: Q)”:

sz	Q	<T>
0	0	2S
0	1	4S
1	x	RESERVED

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);
var element : bits(32);

for e = 0 to elements-1 do
    element = operand[e*:32];
    result[e*:32] = UnsignedRSqrtEstimate{32}(element);
end;

V{datasize}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

URSRA

Unsigned rounding shift right and accumulate (immediate)

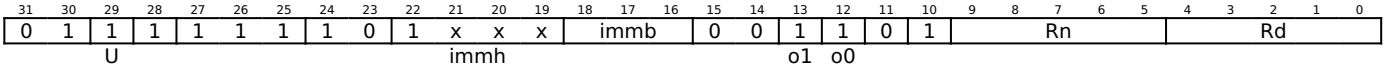
This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, and accumulates the final results with the vector elements of the destination SIMD&FP register. All the values in this instruction are unsigned integer values. The results are rounded. For truncated results, see *USRA*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

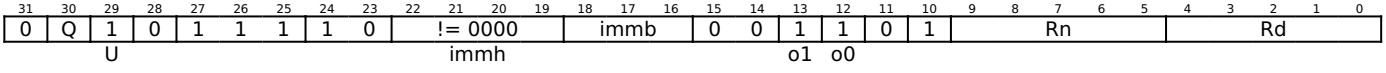
```
URSRA D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
let round : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
URSRA <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
let round : boolean = TRUE;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to 64, encoded as 128 - UInt("immh:immb").

For the "Vector" variant: is the right shift amount, in the range 1 to the element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
let operand2 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    if unsigned then
        element = RShr(UInt(opelt), shift, round);
    else
        element = RShr(SInt(opelt), shift, round);
    end;
    result[e*:esize] = operand2[e*:esize] + element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USDOT (by element)

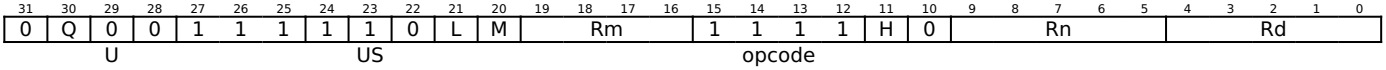
Dot product with unsigned and signed integers (vector, by element)

This instruction performs the dot product of the four unsigned 8-bit integer values in each 32-bit element of the first source register with the four signed 8-bit integer values in an indexed 32-bit element of the second source register, accumulating the result into the corresponding 32-bit element of the destination register.

From Armv8.2 to Armv8.5, this is an OPTIONAL instruction. From Armv8.6 it is mandatory for implementations that include Advanced SIMD to support it. ID_AA64ISAR1_EL1.I8MM indicates whether this instruction is supported.

Vector

(FEAT_I8MM)



Encoding

```
USDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.4B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_I8MM) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(M::Rm);
let d : integer = UInt(Rd);
let i : integer = UInt(H::L);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 32;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, encoded in the "M:Rm" fields.

<index> Is the immediate index of a group of four 8-bit values, in the range 0 to 3, encoded in the "H:L" fields.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(128) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    var res : bits(32) = operand3[e*:32];
    for b = 0 to 3 do
        let element1 : integer = UInt(operand1[(4 * e + b)*:8]);
        let element2 : integer = SInt(operand2[(4 * i + b)*:8]);
        res = res + element1 * element2;
    end;
    result[e*:32] = res;
end;
V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USDOT (vector)

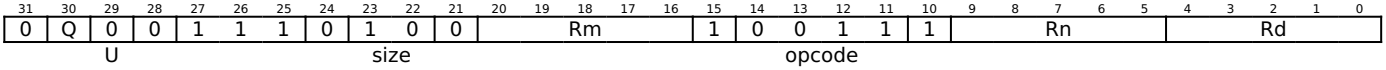
Dot product with unsigned and signed integers (vector)

This instruction performs the dot product of the four unsigned 8-bit integer values in each 32-bit element of the first source register with the four signed 8-bit integer values in the corresponding 32-bit element of the second source register, accumulating the result into the corresponding 32-bit element of the destination register.

From Armv8.2 to Armv8.5, this is an OPTIONAL instruction. From Armv8.6 it is mandatory for implementations that include Advanced SIMD to support it. ID_AA64ISAR1_EL1.I8MM indicates whether this instruction is supported.

Vector

(FEAT_I8MM)



Encoding

```
USDOT <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_I8MM) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV 32;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.

<Ta> Is an arrangement specifier, encoded in “Q”:

Q	<Ta>
0	2S
1	4S

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> Is an arrangement specifier, encoded in “Q”:

Q	<Tb>
0	8B
1	16B

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
let operand3 : bits(datasize) = V{}(d);
var result : bits(datasize);

for e = 0 to elements-1 do
    var res : bits(32) = operand3[e*:32];
    for b = 0 to 3 do
        let element1 : integer = UInt(operand1[(4 * e + b)*:8]);
        let element2 : integer = SInt(operand2[(4 * e + b)*:8]);
        res = res + element1 * element2;
    end;
    result[e*:32] = res;
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USHL

Unsigned shift left (register)

This instruction takes each element in the vector of the first source SIMD&FP register, shifts each element by a value from the least significant byte of the corresponding element of the second source SIMD&FP register, places the results in a vector, and writes the vector to the destination SIMD&FP register.

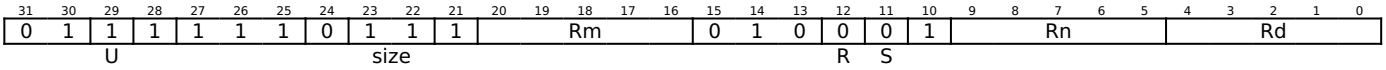
If the shift value is positive, the operation is a left shift. If the shift value is negative, it is a truncating right shift. For a rounding shift, see *URSHL*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

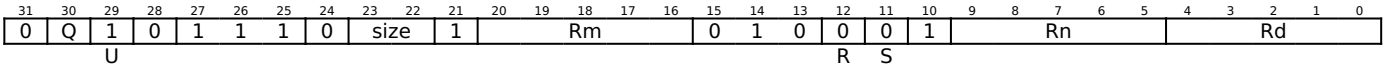
```
USHL D<d>, D<n>, D<m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if S == '0' && size != '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let rounding : boolean = FALSE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
USHL <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let rounding : boolean = FALSE;
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

<d>	Is the number of the SIMD&FP destination register, encoded in the "Rd" field.
<n>	Is the number of the first SIMD&FP source register, encoded in the "Rn" field.
<m>	Is the number of the second SIMD&FP source register, encoded in the "Rm" field.
<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{}(n);
let operand2 : bits(datasize) = V{}(m);
var result : bits(datasize);

for e = 0 to elements-1 do
    var element : integer = UInt(operand1[e*:esize]);
    var shift : integer = ShiftSat(SInt(operand2[e*:esize][7:0]), esize);
    if shift >= 0 then // left shift
        element = element << shift;
    else               // right shift
        shift = -shift;
        element = RShr(element, shift, rounding);
    end;

    result[e*:esize] = element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USHLL, USHLL2

Unsigned shift left long (immediate)

This instruction reads each vector element in the lower or upper half of the source SIMD&FP register, shifts the unsigned integer value left by the specified number of bits, places the result into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements.

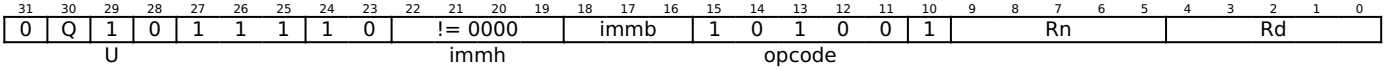
The `USHLL` instruction extracts vector elements from the lower half of the source register. The `USHLL2` instruction extracts vector elements from the upper half of the source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This instruction is used by the alias UXTL, UXTL2.

Vector

(FEAT_AdvSIMD)



Encoding

```
USHLL{2} <Vd>.<Ta>, <Vn>.<Tb>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] == '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh[2:0]);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = UInt(immh::immb) - esize;
let unsigned : boolean = TRUE;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

<Vn> — Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

<shift> Is the left shift amount, in the range 0 to the source element width in bits minus 1, encoded in “(immh :: immb)”:

immh	<shift>
0001	UInt(immh :: immb) - 8
001x	UInt(immh :: immb) - 16
01xx	UInt(immh :: immb) - 32
1xxx	RESERVED

Alias Conditions

Alias	Is preferred when
UXTL, UXTL2	immb == '000' && BitCount(immh) == 1

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = Vpart{}(n, part);
var result : bits(datasize*2);

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
    result[e*:(2*esize)] = (element << shift)[2*esize-1:0];
end;

V{datasize*2}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USHR

Unsigned shift right (immediate)

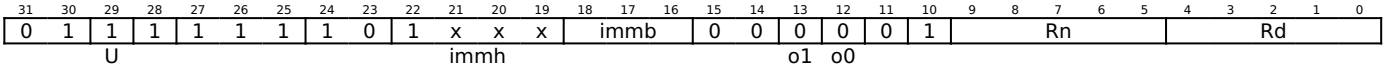
This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, writes the final result to a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are unsigned integer values. The results are truncated. For rounded results, see *URSHR*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

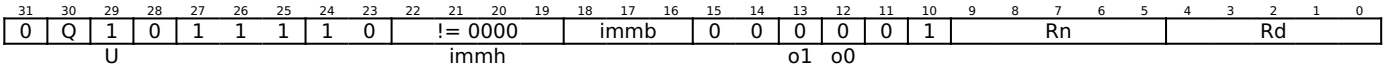
```
USHR D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
let round : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
USHR <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
let round : boolean = FALSE;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to 64, encoded as 128 - UInt("immh:immb").

For the "Vector" variant: is the right shift amount, in the range 1 to the element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    if unsigned then
        element = RShr(UInt(opelt), shift, round);
    else
        element = RShr(SInt(opelt), shift, round);
    end;
    result[e*:esize] = element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USMMLA (vector)

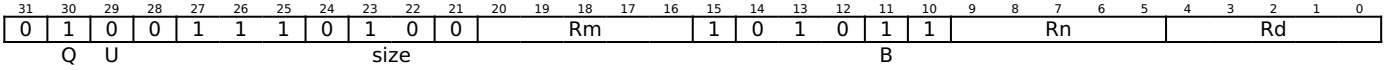
Unsigned and signed 8-bit integer matrix multiply-accumulate to 32-bit integer (vector)

This instruction multiplies the 2x8 matrix of unsigned 8-bit integer values in the first source vector by the 8x2 matrix of signed 8-bit integer values in the second source vector. The resulting 2x2 32-bit integer matrix product is destructively added to the 32-bit integer matrix accumulator in the destination vector. This is equivalent to performing an 8-way dot product per destination element.

From Armv8.2 to Armv8.5, this is an OPTIONAL instruction. From Armv8.6 it is mandatory for implementations that include Advanced SIMD to support it. ID_AA64ISAR1_EL1.I8MM indicates whether this instruction is supported.

Vector

(FEAT_I8MM)



Encoding

```
USMMLA <Vd>.4S, <Vn>.16B, <Vm>.16B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_I8MM) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP third source and destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(128) = V{}(n);
let operand2 : bits(128) = V{}(m);
let addend : bits(128)   = V{}(d);

V{128}(d) = MatMulAdd(addend, operand1, operand2, op1_unsigned, op2_unsigned);
```

Operational information

Arm expects that the USMMLA (vector) instruction will deliver a peak integer multiply throughput that is at least as high as can be achieved using two USDOT (vector) instructions, with a goal that it should have significantly higher throughput.

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USQADD

Unsigned saturating accumulate of signed value

This instruction adds the signed integer values of the vector elements in the source SIMD&FP register to corresponding unsigned integer values of the vector elements in the destination SIMD&FP register, and accumulates the resulting unsigned integer values with the vector elements of the destination SIMD&FP register.

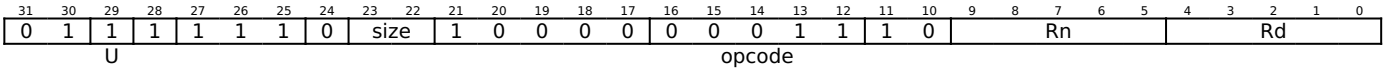
If overflow occurs with any of the results, those results are saturated. If saturation occurs, the cumulative saturation bit FPSR.QC is set.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

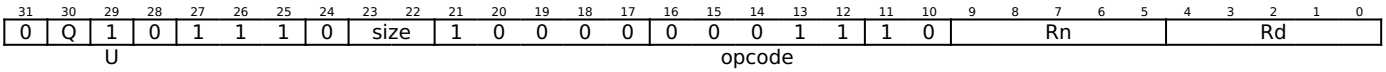
```
USQADD <V><d>, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = esize;
let elements : integer = 1;
let unsigned : boolean = TRUE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
USQADD <Vd>.<T>, <Vn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the SIMD&FP source register, encoded in the "Rn" field.

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize) = V{}(n);
var result : bits(datasize);

let operand2 : bits(datasize) = V{}(d);
var op1 : integer;
var op2 : integer;
var sat : boolean;

for e = 0 to elements-1 do
    op1 = SInt(operand[e*:esize]);
    op2 = UInt(operand2[e*:esize]);
    (result[e*:esize], sat) = SatQ{esize}(op1 + op2, unsigned);
    if sat then FPSR().QC = '1'; end;
end;
V{datasize}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USRA

Unsigned shift right and accumulate (immediate)

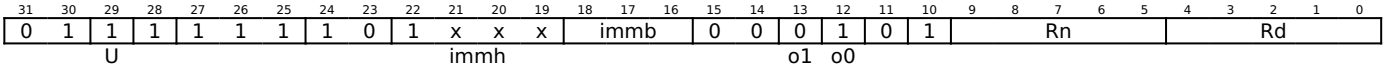
This instruction reads each vector element in the source SIMD&FP register, right shifts each result by an immediate value, and accumulates the final results with the vector elements of the destination SIMD&FP register. All the values in this instruction are unsigned integer values. The results are truncated. For rounded results, see *URSRA*.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

It has encodings from 2 classes: Scalar and Vector

Scalar

(FEAT_AdvSIMD)



Encoding

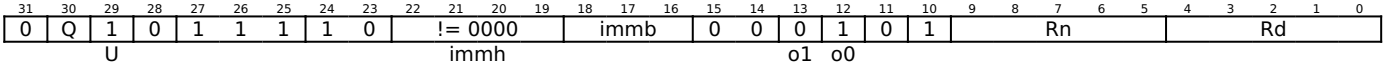
```
USRA D<d>, D<n>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3] != '1' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << 3;
let datasize : integer{} = esize;
let elements : integer = 1;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
let round : boolean = FALSE;
```

Vector

(FEAT_AdvSIMD)



Encoding

```
USRA <Vd>.<T>, <Vn>.<T>, #<shift>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if immh[3]::Q == '10' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let esize : integer{} = 8 << HighestSetBitNZ(immh);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;

let shift : integer = (esize * 2) - UInt(immh::immb);
let unsigned : boolean = TRUE;
let round : boolean = FALSE;
```

Assembler Symbols

<d> Is the number of the SIMD&FP destination register, encoded in the "Rd" field.

<n> Is the number of the first SIMD&FP source register, encoded in the "Rn" field.

<shift> For the "Scalar" variant: is the right shift amount, in the range 1 to 64, encoded as 128 - UInt("immh:immb").

For the "Vector" variant: is the right shift amount, in the range 1 to the element width in bits, encoded in “(immh :: immb)”:

immh	<shift>
0001	16 - UInt(immh :: immb)
001x	32 - UInt(immh :: immb)
01xx	64 - UInt(immh :: immb)
1xxx	128 - UInt(immh :: immb)

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<T>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	0	RESERVED
1xxx	1	2D

<Vn> Is the name of the SIMD&FP source register, encoded in the "Rn" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(datasize)  = V{}(n);
let operand2 : bits(datasize) = V{}(d);
var result : bits(datasize);
var element : integer;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand[e*:esize];
    if unsigned then
        element = RShr(UInt(opelt), shift, round);
    else
        element = RShr(SInt(opelt), shift, round);
    end;
    result[e*:esize] = operand2[e*:esize] + element[esize-1:0];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USUBL, USUBL2

Unsigned subtract long

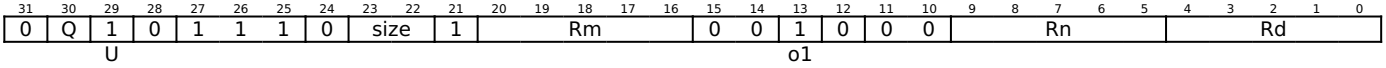
This instruction subtracts each vector element in the lower or upper half of the second source SIMD&FP register from the corresponding vector element of the first source SIMD&FP register, places the result into a vector, and writes the vector to the destination SIMD&FP register. All the values in this instruction are unsigned integer values. The destination vector elements are twice as long as the source vector elements.

The `USUBL` instruction extracts each source vector from the lower half of each source register. The `USUBL2` instruction extracts each source vector from the upper half of each source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
USUBL{2} <Vd>.<Ta>, <Vn>.<Tb>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = Vpart{}(n, part);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:esize]);
    element2 = UInt(operand2[e*:esize]);
    sum = element1 - element2;
    result[e*:(2*esize)] = sum[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USUBW, USUBW2

Unsigned subtract wide

This instruction subtracts each vector element in the lower or upper half of the second source SIMD&FP register from the corresponding vector element in the first source SIMD&FP register, places the result in a vector, and writes the vector to the SIMD&FP destination register. All the values in this instruction are unsigned integer values.

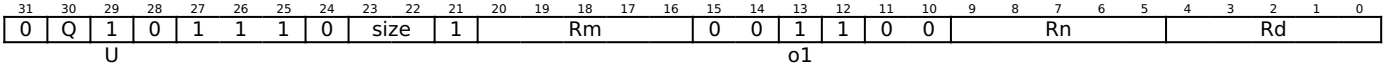
The vector elements of the destination register and the first source register are twice as long as the vector elements of the second source register.

The `USUBW` instruction extracts vector elements from the lower half of the second source register. The `USUBW2` instruction extracts vector elements from the upper half of the second source register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Three registers, not all the same type

(FEAT_AdvSIMD)



Encoding

```
USUBW{2} <Vd>.<Ta>, <Vn>.<Ta>, <Vm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

<Vn> — Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> — Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(2*datasize) = V{}(n);
let operand2 : bits(datasize) = Vpart{}(m, part);
var result : bits(2*datasize);
var element1 : integer;
var element2 : integer;
var sum : integer;

for e = 0 to elements-1 do
    element1 = UInt(operand1[e*:(2*esize)]);
    element2 = UInt(operand2[e*:esize]);
    sum = element1 - element2;
    result[e*:(2*esize)] = sum[2*esize-1:0];
end;

V{2*datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UXTL, UXTL2

Unsigned extend long

This instruction copies each vector element from the lower or upper half of the source SIMD&FP register into a vector, and writes the vector to the destination SIMD&FP register. The destination vector elements are twice as long as the source vector elements.

The `UXTL` instruction extracts vector elements from the lower half of the source register. The `UXTL2` instruction extracts vector elements from the upper half of the source register.

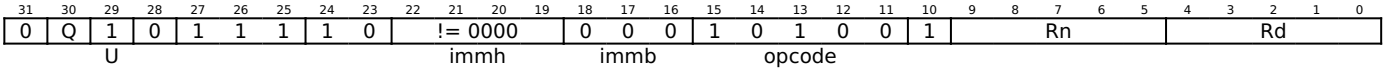
Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

This is an alias of USHLL, USHLL2. This means:

- The encodings in this description are named to match the encodings of USHLL, USHLL2.
- The description of USHLL, USHLL2 gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Vector

(FEAT_AdvSIMD)



Encoding

```
UXTL{2} <Vd>.<Ta>, <Vn>.<Tb>
```

is equivalent to

```
USHLL{2} <Vd>.<Ta>, <Vn>.<Tb>, #0
```

and is the preferred disassembly when `BitCount(immh) == 1`.

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Ta> — Is an arrangement specifier, encoded in “immh”:

immh	<Ta>
0001	8H
001x	4S
01xx	2D
1xxx	RESERVED

<Vn> — Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Tb> — Is an arrangement specifier, encoded in “(immh :: Q)”:

immh	Q	<Tb>
0001	0	8B
0001	1	16B
001x	0	4H
001x	1	8H
01xx	0	2S
01xx	1	4S
1xxx	x	RESERVED

Operation

The description of USHLL, USHLL2 gives the operational pseudocode for this instruction.

Operational Information

The description of USHLL, USHLL2 gives the operational information for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UZP1

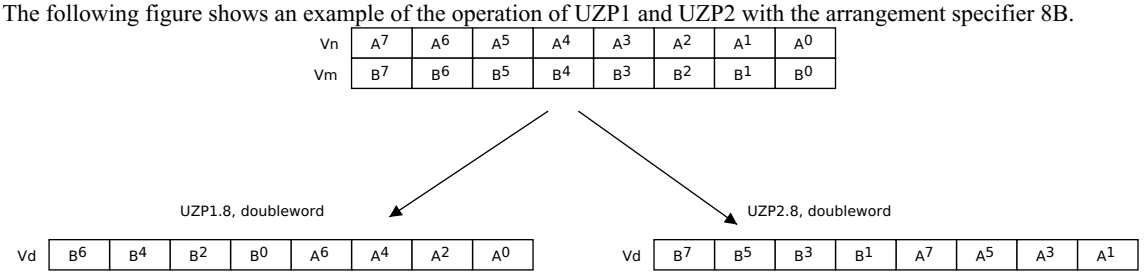
Unzip vectors (primary)

This instruction reads corresponding even-numbered vector elements from the two source SIMD&FP registers, starting at zero, places the result from the first source register into consecutive elements in the lower half of a vector, and the result from the second source register into consecutive elements in the upper half of a vector, and writes the vector to the destination SIMD&FP register.

Note

This instruction can be used with `UZP2` to de-interleave two vectors.

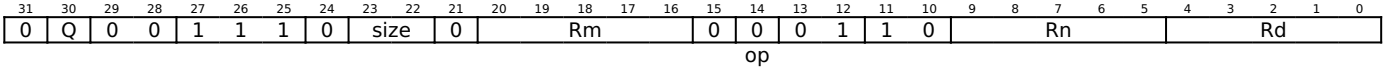
The following figure shows an example of the operation of UZP1 and UZP2 with the arrangement specifier 8B.



Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
UZP1 <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let part : integer = UInt(op);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operandl : bits(datasize) = V{datasize}(n);
let operandh : bits(datasize) = V{datasize}(m);
var result : bits(datasize);

let zipped : bits(datasize*2) = operandh::operandl;
for e = 0 to elements-1 do
    result[e*:esize] = zipped[(2*e+part)*:esize];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UZP2

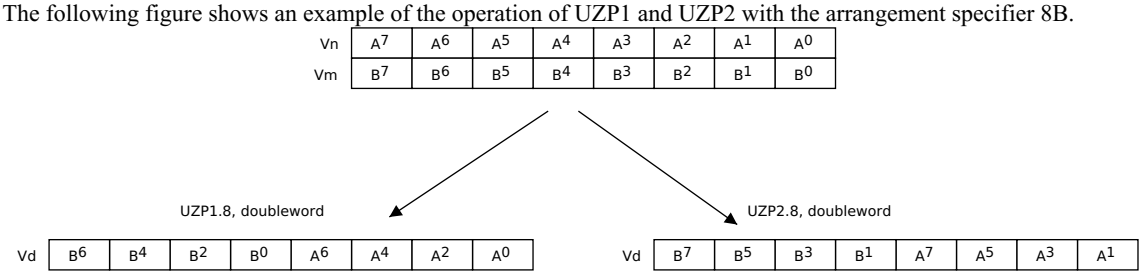
Unzip vectors (secondary)

This instruction reads corresponding odd-numbered vector elements from the two source SIMD&FP registers, places the result from the first source register into consecutive elements in the lower half of a vector, and the result from the second source register into consecutive elements in the upper half of a vector, and writes the vector to the destination SIMD&FP register.

Note

This instruction can be used with `UZP1` to de-interleave two vectors.

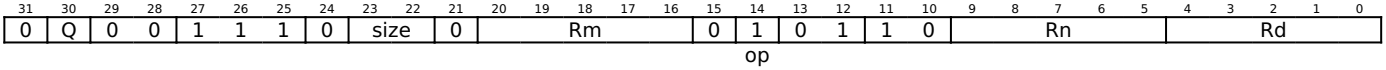
The following figure shows an example of the operation of UZP1 and UZP2 with the arrangement specifier 8B.



Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
UZP2 <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let part : integer = UInt(op);
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operandl : bits(datasize) = V{datasize}(n);
let operandh : bits(datasize) = V{datasize}(m);
var result : bits(datasize);

let zipped : bits(datasize*2) = operandh::operandl;
for e = 0 to elements-1 do
    result[e*:esize] = zipped[(2*e+part)*:esize];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

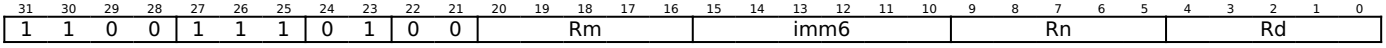
XAR

Exclusive-OR and rotate

This instruction performs a bitwise exclusive-OR between the 128-bit vector in the first source SIMD&FP register and the 128-bit vector in the second source SIMD&FP register, rotates each 64-bit element of the resulting 128-bit vector right by the value specified by a 6-bit immediate value, and writes the result to the destination SIMD&FP register.

Advanced SIMD

(FEAT_SHA3)



Encoding

```
XAR <Vd>.2D, <Vn>.2D, <Vm>.2D, #<imm6>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SHA3) then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);
```

Assembler Symbols

<Vd>	Is the name of the SIMD&FP destination register, encoded in the "Rd" field.
<Vn>	Is the name of the first SIMD&FP source register, encoded in the "Rn" field.
<Vm>	Is the name of the second SIMD&FP source register, encoded in the "Rm" field.
<imm6>	Is a rotation right, encoded in "imm6".

Operation

```
AArch64_CheckFPAdvSIMDEnabled();

let Vm : bits(128) = V{}(m);
let Vn : bits(128) = V{}(n);
let tmp : bits(128) = Vn XOR Vm;
V{128}(d) = ROR(tmp[127:64], UInt(imm6))::ROR(tmp[63:0], UInt(imm6));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

XTN, XTN2

Extract narrow

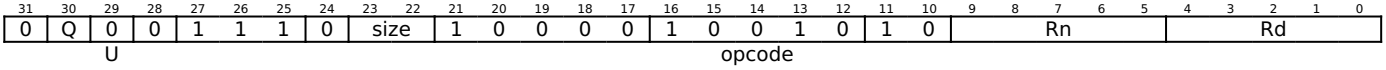
This instruction reads each vector element from the source SIMD&FP register, narrows each value to half the original width, places the result into a vector, and writes the vector to the lower or upper half of the destination SIMD&FP register. The destination vector elements are half as long as the source vector elements.

The `XTN` instruction writes the vector to the lower half of the destination register and clears the upper half. The `XTN2` instruction writes the vector to the upper half of the destination register without affecting the other bits of the register.

Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Vector

(FEAT_AdvSIMD)



Encoding

```
XTN{2} <Vd>.<Tb>, <Vn>.<Ta>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64;
let part : integer = UInt(Q);
let elements : integer = datasize DIV esize;
```

Assembler Symbols

2 — Is the second and upper half specifier. If present, it causes the operation to be performed on the upper 64 bits of the registers holding the narrower elements, and is encoded in “Q”:

Q	2
0	[absent]
1	[present]

<Vd> — Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<Tb> — Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<Tb>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	x	RESERVED

<Vn> — Is the name of the SIMD&FP source register, encoded in the "Rn" field.

<Ta> — Is an arrangement specifier, encoded in “size”:

size	<Ta>
00	8H
01	4S
10	2D
11	RESERVED

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand : bits(2*datasize) = V{}(n);
var result : bits(datasize);
var element : bits(2*esize);

for e = 0 to elements-1 do
    element = operand[e*:(2*esize)];
    result[e*:esize] = element[esize-1:0];
end;
Vpart{datasize}(d, part) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ZIP1

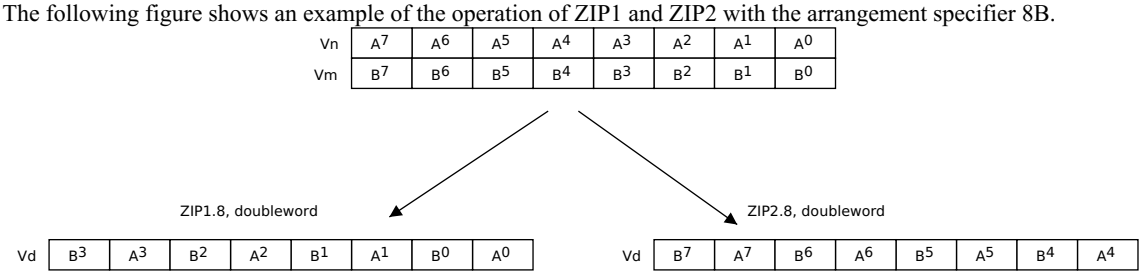
Zip vectors (primary)

This instruction reads adjacent vector elements from the lower half of two source SIMD&FP registers as pairs, interleaves the pairs and places them into a vector, and writes the vector to the destination SIMD&FP register. The first pair from the first source register is placed into the two lowest vector elements, with subsequent pairs taken alternately from each source register.

Note

This instruction can be used with `ZIP2` to interleave two vectors.

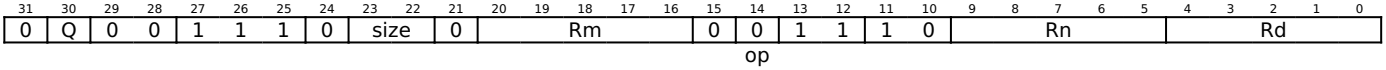
The following figure shows an example of the operation of ZIP1 and ZIP2 with the arrangement specifier 8B.



Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
ZIP1 <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let part : integer = UInt(op);
let pairs : integer = elements DIV 2;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{datasize}(n);
let operand2 : bits(datasize) = V{datasize}(m);
var result : bits(datasize);

let base : integer = part * pairs;

for p = 0 to pairs-1 do
    result[(2*p+0)*:esize] = operand1[(base+p)*:esize];
    result[(2*p+1)*:esize] = operand2[(base+p)*:esize];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ZIP2

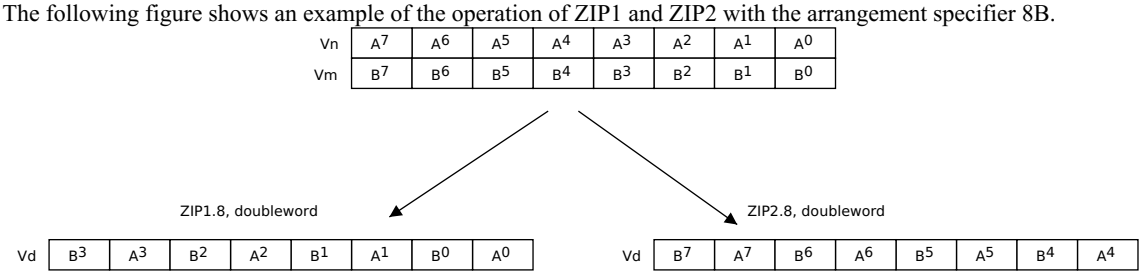
Zip vectors (secondary)

This instruction reads adjacent vector elements from the upper half of two source SIMD&FP registers as pairs, interleaves the pairs and places them into a vector, and writes the vector to the destination SIMD&FP register. The first pair from the first source register is placed into the two lowest vector elements, with subsequent pairs taken alternately from each source register.

Note

This instruction can be used with `ZIP1` to interleave two vectors.

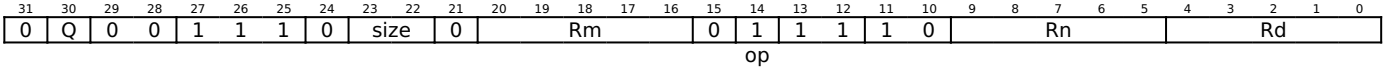
The following figure shows an example of the operation of ZIP1 and ZIP2 with the arrangement specifier 8B.



Depending on the settings in the CPACR_EL1, CPTR_EL2, and CPTR_EL3 registers, and the current Security state and Exception level, an attempt to execute the instruction might be trapped.

Advanced SIMD

(FEAT_AdvSIMD)



Encoding

```
ZIP2 <Vd>.<T>, <Vn>.<T>, <Vm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_AdvSIMD) then EndOfDecode(Decode_UNDEF); end;
if size::Q == '110' then EndOfDecode(Decode_UNDEF); end;

let d : integer{} = UInt(Rd);
let n : integer{} = UInt(Rn);
let m : integer{} = UInt(Rm);

let esize : integer{} = 8 << UInt(size);
let datasize : integer{} = 64 << UInt(Q);
let elements : integer = datasize DIV esize;
let part : integer = UInt(op);
let pairs : integer = elements DIV 2;
```

Assembler Symbols

<Vd> Is the name of the SIMD&FP destination register, encoded in the "Rd" field.

<T> Is an arrangement specifier, encoded in “(size :: Q)”:

size	Q	<T>
00	0	8B
00	1	16B
01	0	4H
01	1	8H
10	0	2S
10	1	4S
11	0	RESERVED
11	1	2D

<Vn> Is the name of the first SIMD&FP source register, encoded in the "Rn" field.

<Vm> Is the name of the second SIMD&FP source register, encoded in the "Rm" field.

Operation

```
AArch64_CheckFPAdvSIMDEnabled();
let operand1 : bits(datasize) = V{datasize}(n);
let operand2 : bits(datasize) = V{datasize}(m);
var result : bits(datasize);

let base : integer = part * pairs;

for p = 0 to pairs-1 do
    result[(2*p+0)*:esize] = operand1[(base+p)*:esize];
    result[(2*p+1)*:esize] = operand2[(base+p)*:esize];
end;

V{datasize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

A64 -- SVE Instructions (alphabetic order)

ABS: Absolute value (predicated).

ADCLB: Add with carry long (bottom).

ADCLT: Add with carry long (top).

ADD (immediate): Add immediate (unpredicated).

ADD (vectors, predicated): Add (predicated).

ADD (vectors, unpredicated): Add (unpredicated).

ADDHNB: Add narrow high part (bottom).

ADDHNT: Add narrow high part (top).

ADDP: Add pairwise.

ADDPL: Add multiple of predicate register size to scalar register.

ADDPT (predicated): Add checked pointer vectors (predicated).

ADDPT (unpredicated): Add checked pointer vectors (unpredicated).

ADDQP: Add pairwise within quadword vector segments (unpredicated).

ADDQV: Add reduction of quadword vector segments.

ADDSUBP: Add subtract pairwise.

ADDVL: Add multiple of vector register size to scalar register.

ADR: Calculate vector address.

AESD (indexed): Multi-vector AES single round decryption.

AESD (vectors): AES single round decryption.

AESDIMC: Multi-vector AES single round decryption and inverse mix columns.

AESE (indexed): Multi-vector AES single round encryption.

AESE (vectors): AES single round encryption.

AESEMC: Multi-vector AES single round encryption and mix columns.

AESIMC: AES inverse mix columns.

AESMC: AES mix columns.

AND (immediate): Bitwise AND with immediate (unpredicated).

AND (predicates): Bitwise AND predicates.

AND (vectors, predicated): Bitwise AND (predicated).

AND (vectors, unpredicated): Bitwise AND (unpredicated).

ANDQV: Bitwise AND reduction of quadword vector segments.

ANDS: Bitwise AND predicates, setting the condition flags.

ANDV: Bitwise AND reduction to scalar.

ASR (immediate, predicated): Arithmetic shift right by immediate (predicated).

ASR (immediate, unpredicated): Arithmetic shift right by immediate (unpredicated).

ASR (vectors): Arithmetic shift right by vector (predicated).

ASR (wide elements, predicated): Arithmetic shift right by 64-bit wide elements (predicated).

ASR (wide elements, unpredicated): Arithmetic shift right by 64-bit wide elements (unpredicated).

ASRD: Arithmetic shift right for divide by immediate (predicated).

ASRR: Reversed arithmetic shift right by vector (predicated).

BCAX: Bitwise clear and exclusive-OR.

BDEP: Scatter lower bits into positions selected by bitmask.

BEXT: Gather lower bits from positions selected by bitmask.

BF1CVT, BF2CVT: 8-bit floating-point convert to BFloat16.

BF1CVTLT, BF2CVTLT: 8-bit floating-point convert to BFloat16 (top).

BFADD (predicated): BFloat16 add (predicated).

BFADD (unpredicated): BFloat16 add (unpredicated).

BFCLAMP: BFloat16 clamp to minimum/maximum number.

BFCVT: Single-precision convert to BFloat16 (predicated).

BFCVTN: BFloat16 convert to interleaved 8-bit floating-point.

BFCVTNT: Single-precision convert to BFloat16 (top, predicated).

BFDOT (indexed): BFloat16 dot product by indexed element to single-precision.

BFDOT (vectors): BFloat16 dot product to single-precision.

BFMAX: BFloat16 maximum (predicated).

BFMAXNM: BFloat16 maximum number (predicated).

BFMIN: BFloat16 minimum (predicated).

BFMINNM: BFloat16 minimum number (predicated).

BFMLA (indexed): BFloat16 fused multiply-add by indexed element.

BFMLA (vectors): BFloat16 fused multiply-add.

BFMLALB (indexed): BFloat16 multiply-add by indexed element to single-precision (bottom).

BFMLALB (vectors): BFloat16 multiply-add to single-precision (bottom).

BFMLALT (indexed): BFloat16 multiply-add by indexed element to single-precision (top).

BFMLALT (vectors): BFloat16 multiply-add to single-precision (top).

BFMLS (indexed): BFloat16 fused multiply-subtract by indexed element.

BFMLS (vectors): BFloat16 fused multiply-subtract.

BFMLSLB (indexed): BFloat16 multiply-subtract by indexed element from single-precision (bottom).

BFMLSLB (vectors): BFloat16 multiply-subtract from single-precision (bottom).

BFMLSLT (indexed): BFloat16 multiply-subtract by indexed element from single-precision (top).

BFMLSLT (vectors): BFloat16 multiply-subtract from single-precision (top).

BFMMLA (non-widening): BFloat16 matrix multiply-accumulate.

BFMMLA (widening): BFloat16 matrix multiply-accumulate to single-precision.

BFMUL (indexed): BFloat16 multiply by indexed element.

BFMUL (vectors, predicated): BFloat16 multiply (predicated).

BFMUL (vectors, unpredicated): BFloat16 multiply (unpredicated).

BFSCALE: BFloat16 adjust exponent (predicated).

BFSUB (predicated): BFloat16 subtract (predicated).

BFSUB (unpredicated): BFloat16 subtract (unpredicated).

BGRP: Group bits to right or left as selected by bitmask.

BIC (immediate): Bitwise clear bits using immediate (unpredicated): an alias of AND (immediate).

BIC (predicates): Bitwise clear predicates.

BIC (vectors, predicated): Bitwise clear (predicated).

BIC (vectors, unpredicated): Bitwise clear (unpredicated).

BICS: Bitwise clear predicates, setting the condition flags.

BRKA: Break after first true condition.

BRKAS: Break after first true condition, setting the condition flags.

BRKB: Break before first true condition.

BRKBS: Break before first true condition, setting the condition flags.

BRKN: Propagate break to next partition.

BRKNS: Propagate break to next partition, setting the condition flags.

BRKPA: Break after first true condition, propagating from previous partition.

BRKPAS: Break after first true condition, propagating from previous partition and setting the condition flags.

BRKPB: Break before first true condition, propagating from previous partition.

BRKPBS: Break before first true condition, propagating from previous partition and setting the condition flags.

BSL: Bitwise select.

BSL1N: Bitwise select with first input inverted.

BSL2N: Bitwise select with second input inverted.

CADD: Complex integer add.

CDOT (indexed): Complex integer dot product by indexed element.

CDOT (vectors): Complex integer dot product.

CLASTA (scalar): Conditionally extract element after last to general-purpose register.

CLASTA (SIMD&FP scalar): Conditionally extract element after last to SIMD&FP scalar register.

CLASTA (vectors): Conditionally extract element after last to vector register.

CLASTB (scalar): Conditionally extract last element to general-purpose register.

CLASTB (SIMD&FP scalar): Conditionally extract last element to SIMD&FP scalar register.

CLASTB (vectors): Conditionally extract last element to vector register.

CLS: Count leading sign bits (predicated).

CLZ: Count leading zero bits (predicated).

CMLA (indexed): Complex integer multiply-add by indexed element.

CMLA (vectors): Complex integer multiply-add.

CMP<cc> (immediate): Compare vector to immediate.

CMP<cc> (vectors): Compare vectors.

CMP<cc> (wide elements): Compare vector to 64-bit wide elements.

CMPLE (vectors): Compare signed less than or equal to vector, setting the condition flags: an alias of CMP<cc> (vectors).

CMPLO (vectors): Compare unsigned lower than vector, setting the condition flags: an alias of CMP<cc> (vectors).

CMPLS (vectors): Compare unsigned lower than or same as vector, setting the condition flags: an alias of CMP<cc> (vectors).

CMPLT (vectors): Compare signed less than vector, setting the condition flags: an alias of CMP<cc> (vectors).

CNOT: Logically invert boolean condition (predicated).

CNT: Count nonzero bits (predicated).

CNTB, CNTD, CNTH, CNTW: Set scalar to multiple of predicate constraint element count.

CNTP (predicate as counter): Set scalar to count from predicate-as-counter.

CNTP (predicate): Set scalar to count of true predicate elements.

COMPACT: Copy Active vector elements to lower-numbered elements.

CPY (immediate, merging): Copy signed integer immediate to vector elements (merging).

CPY (immediate, zeroing): Copy signed integer immediate to vector elements (zeroing).

CPY (scalar): Copy general-purpose register to vector elements (predicated).

CPY (SIMD&FP scalar): Copy SIMD&FP scalar register to vector elements (predicated).

CTERMEQ, CTERMNE: Compare and terminate loop.

DECB, DECD, DECH, DECW (scalar): Decrement scalar by multiple of predicate constraint element count.

DECD, DECH, DECW (vector): Decrement vector by multiple of predicate constraint element count.

DECP (scalar): Decrement scalar by count of true predicate elements.

DECP (vector): Decrement vector by count of true predicate elements.

DUP (immediate): Broadcast signed immediate to vector elements (unpredicated).

DUP (indexed): Broadcast indexed element to vector (unpredicated).

DUP (scalar): Broadcast general-purpose register to vector elements (unpredicated).

DUPM: Broadcast logical bitmask immediate to vector (unpredicated).

DUPQ: Broadcast indexed element within each quadword vector segment (unpredicated).

EON: Bitwise exclusive-OR with inverted immediate (unpredicated): an alias of EOR (immediate).

EOR (immediate): Bitwise exclusive-OR with immediate (unpredicated).

EOR (predicates): Bitwise exclusive-OR predicates.

EOR (vectors, predicated): Bitwise exclusive-OR (predicated).

EOR (vectors, unpredicated): Bitwise exclusive-OR (unpredicated).

EOR3: Bitwise exclusive-OR between three vectors.

EORBT: Interleaving exclusive-OR (bottom, top).

EORQV: Bitwise exclusive-OR reduction of quadword vector segments.

EORS: Bitwise exclusive-OR predicates, setting the condition flags.

EORTB: Interleaving exclusive-OR (top, bottom).

EORV: Bitwise exclusive-OR reduction to scalar.

EXPAND: Copy lower-numbered vector elements to Active elements.

EXT: Extract vector from pair of vectors.

EXTQ: Extract vector segment from each pair of quadword vector segments.

F1CVT, F2CVT: 8-bit floating-point convert to half-precision.

F1CVTLT, F2CVTLT: 8-bit floating-point convert to half-precision (top).

FABD: Floating-point absolute difference (predicated).

FABS: Floating-point absolute value (predicated).

FAC<cc>: Floating-point absolute compare.

FACLE: Floating-point absolute compare less than or equal: an alias of FAC<cc>.

FACLT: Floating-point absolute compare less than: an alias of FAC<cc>.

FADD (immediate): Floating-point add immediate (predicated).

FADD (vectors, predicated): Floating-point add (predicated).

FADD (vectors, unpredicated): Floating-point add (unpredicated).

FADDA: Floating-point add strictly-ordered reduction, accumulating in scalar.

FADDP: Floating-point add pairwise.

FADDQV: Floating-point add recursive reduction of quadword vector segments.

FADDV: Floating-point add recursive reduction to scalar.

FAMAX: Floating-point absolute maximum (predicated).

FAMIN: Floating-point absolute minimum (predicated).

FCADD: Floating-point complex add (predicated).

FCLAMP: Floating-point clamp to minimum/maximum number.

FCM<cc> (vectors): Floating-point compare.

FCM<cc> (zero): Floating-point compare with zero.

FCMLA (indexed): Floating-point complex multiply-add by indexed element.

FCMLA (vectors): Floating-point complex multiply-add (predicated).

FCMLE (vectors): Floating-point compare less than or equal to vector: an alias of FCM<cc> (vectors).

FCMLT (vectors): Floating-point compare less than vector: an alias of FCM<cc> (vectors).

FCPY: Copy floating-point immediate to vector elements (predicated).

FCVT: Floating-point convert (predicated).

FCVTLT: Floating-point widening convert (top, predicated).

FCVTN: Half-precision convert to interleaved 8-bit floating-point.

FCVTNB: Single-precision convert to interleaved 8-bit floating-point (bottom).

FCVTNT (predicated): Floating-point narrowing convert (top, predicated).

FCVTNT (unpredicated): Single-precision convert to interleaved 8-bit floating-point (top).

FCVTX: Double-precision convert to single-precision, rounding to odd (predicated).

FCVTXNT: Double-precision convert to single-precision, rounding to odd (top, predicated).

FCVTZS: Floating-point convert to signed integer, rounding toward zero (predicated).

FCVTZSN: Floating-point narrowing convert to interleaved signed integer, rounding toward zero.

FCVTZU: Floating-point convert to unsigned integer, rounding toward zero (predicated).

FCVTZUN: Floating-point narrowing convert to interleaved unsigned integer, rounding toward zero.

FDIV: Floating-point divide (predicated).

FDIVR: Floating-point reversed divide (predicated).

FDOT (2-way, indexed, FP16 to FP32): Half-precision dot product by indexed element to single-precision.

FDOT (2-way, indexed, FP8 to FP16): 8-bit floating-point dot product by indexed element to half-precision.

FDOT (2-way, vectors, FP16 to FP32): Half-precision dot product to single-precision.

FDOT (2-way, vectors, FP8 to FP16): 8-bit floating-point dot product to half-precision.

FDOT (4-way, indexed): 8-bit floating-point dot product by indexed element to single-precision.

FDOT (4-way, vectors): 8-bit floating-point dot product to single-precision.

FDUP: Broadcast floating-point immediate to vector elements (unpredicated).

FEXPA: Floating-point exponential accelerator.

FIRSTP: Scalar index of first true predicate element (predicated).

FLOGB: Floating-point base 2 logarithm as integer.

FMAD: Floating-point fused multiply-add to multiplicand (predicated).

FMAX (immediate): Floating-point maximum with immediate (predicated).

FMAX (vectors): Floating-point maximum (predicated).

FMAXNM (immediate): Floating-point maximum number with immediate (predicated).

FMAXNM (vectors): Floating-point maximum number (predicated).

FMAXNMP: Floating-point maximum number pairwise.

FMAXNMQV: Floating-point maximum number recursive reduction of quadword vector segments.

FMAXNMV: Floating-point maximum number recursive reduction to scalar.

FMAXP: Floating-point maximum pairwise.

FMAXQV: Floating-point maximum reduction of quadword vector segments.

FMAXV: Floating-point maximum recursive reduction to scalar.

FMIN (immediate): Floating-point minimum with immediate (predicated).

FMIN (vectors): Floating-point minimum (predicated).

FMINNM (immediate): Floating-point minimum number with immediate (predicated).

FMINNM (vectors): Floating-point minimum number (predicated).

FMINNMP: Floating-point minimum number pairwise.

FMINNMQV: Floating-point minimum number recursive reduction of quadword vector segments.

FMINNMV: Floating-point minimum number recursive reduction to scalar.

FMINP: Floating-point minimum pairwise.

FMINQV: Floating-point minimum recursive reduction of quadword vector segments.

FMINV: Floating-point minimum recursive reduction to scalar.

FMLA (indexed): Floating-point fused multiply-add by indexed element.

FMLA (vectors): Floating-point fused multiply-add (predicated).

FMLALB (indexed, FP16 to FP32): Half-precision multiply-add by indexed element to single-precision (bottom).

FMLALB (indexed, FP8 to FP16): 8-bit floating-point multiply-add by indexed element to half-precision (bottom).

FMLALB (vectors, FP16 to FP32): Half-precision multiply-add to single-precision (bottom).

FMLALB (vectors, FP8 to FP16): 8-bit floating-point multiply-add to half-precision (bottom).

FMLALLBB (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (bottom bottom).

FMLALLBB (vectors): 8-bit floating-point multiply-add to single-precision (bottom bottom).

FMLALLBT (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (bottom top).

FMLALLBT (vectors): 8-bit floating-point multiply-add to single-precision (bottom top).

FMLALLTB (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (top bottom).

FMLALLTB (vectors): 8-bit floating-point multiply-add to single-precision (top bottom).

FMLALLTT (indexed): 8-bit floating-point multiply-add by indexed element to single-precision (top top).

FMLALLTT (vectors): 8-bit floating-point multiply-add to single-precision (top top).

FMLALT (indexed, FP16 to FP32): Half-precision multiply-add by indexed element to single-precision (top).

FMLALT (indexed, FP8 to FP16): 8-bit floating-point multiply-add by indexed element to half-precision (top).

FMLALT (vectors, FP16 to FP32): Half-precision multiply-add to single-precision (top).

FMLALT (vectors, FP8 to FP16): 8-bit floating-point multiply-add to half-precision (top).

FMLS (indexed): Floating-point fused multiply-subtract by indexed element.

FMLS (vectors): Floating-point fused multiply-subtract (predicated).

FMLSLB (indexed): Half-precision multiply-subtract by indexed element from single-precision (bottom).

FMLSLB (vectors): Half-precision multiply-subtract from single-precision (bottom).

FMLSLT (indexed): Half-precision multiply-subtract by indexed element from single-precision (top).

FMLSLT (vectors): Half-precision multiply-subtract from single-precision (top).

FMMLA (non-widening): Floating-point matrix multiply-accumulate.

FMMLA (widening, FP16 to FP32): Half-precision matrix multiply-accumulate to single-precision.

FMMLA (widening, FP8 to FP16): 8-bit floating-point matrix multiply-accumulate to half-precision.

FMMLA (widening, FP8 to FP32): 8-bit floating-point matrix multiply-accumulate to single-precision.

FMOV (immediate, predicated): Move floating-point immediate to vector elements (predicated): an alias of FCPY.

FMOV (immediate, unpredicated): Move floating-point immediate to vector elements (unpredicated): an alias of FDUP.

FMOV (zero, predicated): Move floating-point +0.0 to vector elements (predicated): an alias of CPY (immediate, merging).

FMOV (zero, unpredicated): Move floating-point +0.0 to vector elements (unpredicated): an alias of DUP (immediate).

FMSB: Floating-point fused multiply-subtract to multiplicand (predicated).

FMUL (immediate): Floating-point multiply by immediate (predicated).

FMUL (indexed): Floating-point multiply by indexed element.

FMUL (vectors, predicated): Floating-point multiply (predicated).

FMUL (vectors, unpredicated): Floating-point multiply (unpredicated).

FMULX: Floating-point multiply extended (predicated).

FNEG: Floating-point negate (predicated).

FNMAD: Floating-point negated fused multiply-add to multiplicand (predicated).

FNMLA: Floating-point negated fused multiply-add (predicated).

FNMLS: Floating-point negated fused multiply-subtract (predicated).

FNMSB: Floating-point negated fused multiply-subtract to multiplicand (predicated).

FRECPE: Floating-point reciprocal estimate (unpredicated).

FRECPS: Floating-point reciprocal step (unpredicated).

FRECPX: Floating-point reciprocal exponent (predicated).

FRINT32X: Floating-point round to 32-bit integer (predicated).

FRINT32Z: Floating-point round to 32-bit integer, rounding toward zero (predicated).

FRINT64X: Floating-point round to 64-bit integer (predicated).

FRINT64Z: Floating-point round to 64-bit integer, rounding toward zero (predicated).

FRINT<r>: Floating-point round to integral value (predicated).

FRSQRTE: Floating-point reciprocal square root estimate (unpredicated).

FRSQRTS: Floating-point reciprocal square root step (unpredicated).

FSCALE: Floating-point adjust exponent (predicated).

FSQRT: Floating-point square root (predicated).

FSUB (immediate): Floating-point subtract immediate (predicated).

FSUB (vectors, predicated): Floating-point subtract (predicated).

FSUB (vectors, unpredicated): Floating-point subtract (unpredicated).

FSUBR (immediate): Floating-point reversed subtract from immediate (predicated).

FSUBR (vectors): Floating-point reversed subtract (predicated).

FTMAD: Floating-point trigonometric multiply-add coefficient.

FTSMUL: Floating-point trigonometric starting value.

FTSSEL: Floating-point trigonometric select coefficient.

HISTCNT: Count matching elements in vector.

HISTSEG: Count matching elements in vector segments.

INCB, INCD, INCH, INCW (scalar): Increment scalar by multiple of predicate constraint element count.

INCD, INCH, INCW (vector): Increment vector by multiple of predicate constraint element count.

INCP (scalar): Increment scalar by count of true predicate elements.

INCP (vector): Increment vector by count of true predicate elements.

INDEX (immediate, scalar): Create index starting from immediate and incremented by general-purpose register.

INDEX (immediates): Create index starting from and incremented by immediate.

INDEX (scalar, immediate): Create index starting from general-purpose register and incremented by immediate.

INDEX (scalars): Create index starting from and incremented by general-purpose register.

INSR (scalar): Insert general-purpose register in shifted vector.

INSR (SIMD&FP scalar): Insert SIMD&FP scalar register in shifted vector.

LASTA (scalar): Extract element after last to general-purpose register.

LASTA (SIMD&FP scalar): Extract element after last to SIMD&FP scalar register.

LASTB (scalar): Extract last element to general-purpose register.

LASTB (SIMD&FP scalar): Extract last element to SIMD&FP scalar register.

LASTP: Scalar index of last true predicate element (predicated).

LD1B (scalar plus immediate, consecutive registers): Contiguous load of bytes to multiple consecutive vectors (immediate index).

LD1B (scalar plus immediate, single register): Contiguous load unsigned bytes to vector (immediate index).

LD1B (scalar plus scalar, consecutive registers): Contiguous load of bytes to multiple consecutive vectors (scalar index).

LD1B (scalar plus scalar, single register): Contiguous load unsigned bytes to vector (scalar index).

LD1B (scalar plus vector): Gather load unsigned bytes to vector (vector index).

LD1B (vector plus immediate): Gather load unsigned bytes to vector (immediate index).

LD1D (scalar plus immediate, consecutive registers): Contiguous load of doublewords to multiple consecutive vectors (immediate index).

LD1D (scalar plus immediate, single register): Contiguous load unsigned doublewords to vector (immediate index).

LD1D (scalar plus scalar, consecutive registers): Contiguous load of doublewords to multiple consecutive vectors (scalar index).

LD1D (scalar plus scalar, single register): Contiguous load unsigned doublewords to vector (scalar index).

LD1D (scalar plus vector): Gather load doublewords to vector (vector index).

LD1D (vector plus immediate): Gather load doublewords to vector (immediate index).

LD1H (scalar plus immediate, consecutive registers): Contiguous load of halfwords to multiple consecutive vectors (immediate index).

LD1H (scalar plus immediate, single register): Contiguous load unsigned halfwords to vector (immediate index).

LD1H (scalar plus scalar, consecutive registers): Contiguous load of halfwords to multiple consecutive vectors (scalar index).

LD1H (scalar plus scalar, single register): Contiguous load unsigned halfwords to vector (scalar index).

LD1H (scalar plus vector): Gather load unsigned halfwords to vector (vector index).

LD1H (vector plus immediate): Gather load unsigned halfwords to vector (immediate index).

LD1Q: Gather load quadwords.

LD1RB: Load and broadcast unsigned byte to vector.

LD1RD: Load and broadcast doubleword to vector.

LD1RH: Load and broadcast unsigned halfword to vector.

LD1ROB (scalar plus immediate): Contiguous load and replicate thirty-two bytes (immediate index).

LD1ROB (scalar plus scalar): Contiguous load and replicate thirty-two bytes (scalar index).

LD1ROD (scalar plus immediate): Contiguous load and replicate four doublewords (immediate index).

LD1ROD (scalar plus scalar): Contiguous load and replicate four doublewords (scalar index).

LD1ROH (scalar plus immediate): Contiguous load and replicate sixteen halfwords (immediate index).

LD1ROH (scalar plus scalar): Contiguous load and replicate sixteen halfwords (scalar index).

LD1ROW (scalar plus immediate): Contiguous load and replicate eight words (immediate index).

LD1ROW (scalar plus scalar): Contiguous load and replicate eight words (scalar index).

LD1RQB (scalar plus immediate): Contiguous load and replicate sixteen bytes (immediate index).

LD1RQB (scalar plus scalar): Contiguous load and replicate sixteen bytes (scalar index).

LD1RQD (scalar plus immediate): Contiguous load and replicate two doublewords (immediate index).

LD1RQD (scalar plus scalar): Contiguous load and replicate two doublewords (scalar index).

LD1RQH (scalar plus immediate): Contiguous load and replicate eight halfwords (immediate index).

LD1RQH (scalar plus scalar): Contiguous load and replicate eight halfwords (scalar index).

LD1RQW (scalar plus immediate): Contiguous load and replicate four words (immediate index).

LD1RQW (scalar plus scalar): Contiguous load and replicate four words (scalar index).

LD1RSB: Load and broadcast signed byte to vector.

LD1RSH: Load and broadcast signed halfword to vector.

LD1RSW: Load and broadcast signed word to vector.

LD1RW: Load and broadcast unsigned word to vector.

LD1SB (scalar plus immediate): Contiguous load signed bytes to vector (immediate index).

LD1SB (scalar plus scalar): Contiguous load signed bytes to vector (scalar index).

LD1SB (scalar plus vector): Gather load signed bytes to vector (vector index).

LD1SB (vector plus immediate): Gather load signed bytes to vector (immediate index).

LD1SH (scalar plus immediate): Contiguous load signed halfwords to vector (immediate index).

LD1SH (scalar plus scalar): Contiguous load signed halfwords to vector (scalar index).

LD1SH (scalar plus vector): Gather load signed halfwords to vector (vector index).

LD1SH (vector plus immediate): Gather load signed halfwords to vector (immediate index).

LD1SW (scalar plus immediate): Contiguous load signed words to vector (immediate index).

LD1SW (scalar plus scalar): Contiguous load signed words to vector (scalar index).

LD1SW (scalar plus vector): Gather load signed words to vector (vector index).

LD1SW (vector plus immediate): Gather load signed words to vector (immediate index).

LD1W (scalar plus immediate, consecutive registers): Contiguous load of words to multiple consecutive vectors (immediate index).

LD1W (scalar plus immediate, single register): Contiguous load unsigned words to vector (immediate index).

LD1W (scalar plus scalar, consecutive registers): Contiguous load of words to multiple consecutive vectors (scalar index).

LD1W (scalar plus scalar, single register): Contiguous load unsigned words to vector (scalar index).

LD1W (scalar plus vector): Gather load unsigned words to vector (vector index).

LD1W (vector plus immediate): Gather load unsigned words to vector (immediate index).

LD2B (scalar plus immediate): Contiguous load two-byte structures to two vectors (immediate index).

LD2B (scalar plus scalar): Contiguous load two-byte structures to two vectors (scalar index).

LD2D (scalar plus immediate): Contiguous load two-doubleword structures to two vectors (immediate index).

LD2D (scalar plus scalar): Contiguous load two-doubleword structures to two vectors (scalar index).

LD2H (scalar plus immediate): Contiguous load two-halfword structures to two vectors (immediate index).

LD2H (scalar plus scalar): Contiguous load two-halfword structures to two vectors (scalar index).

LD2Q (scalar plus immediate): Contiguous load two-quadword structures to two vectors (immediate index).

LD2Q (scalar plus scalar): Contiguous load two-quadword structures to two vectors (scalar index).

LD2W (scalar plus immediate): Contiguous load two-word structures to two vectors (immediate index).

LD2W (scalar plus scalar): Contiguous load two-word structures to two vectors (scalar index).

LD3B (scalar plus immediate): Contiguous load three-byte structures to three vectors (immediate index).

LD3B (scalar plus scalar): Contiguous load three-byte structures to three vectors (scalar index).

LD3D (scalar plus immediate): Contiguous load three-doubleword structures to three vectors (immediate index).

LD3D (scalar plus scalar): Contiguous load three-doubleword structures to three vectors (scalar index).

LD3H (scalar plus immediate): Contiguous load three-halfword structures to three vectors (immediate index).

LD3H (scalar plus scalar): Contiguous load three-halfword structures to three vectors (scalar index).

LD3Q (scalar plus immediate): Contiguous load three-quadword structures to three vectors (immediate index).

LD3Q (scalar plus scalar): Contiguous load three-quadword structures to three vectors (scalar index).

LD3W (scalar plus immediate): Contiguous load three-word structures to three vectors (immediate index).

LD3W (scalar plus scalar): Contiguous load three-word structures to three vectors (scalar index).

LD4B (scalar plus immediate): Contiguous load four-byte structures to four vectors (immediate index).

LD4B (scalar plus scalar): Contiguous load four-byte structures to four vectors (scalar index).

LD4D (scalar plus immediate): Contiguous load four-doubleword structures to four vectors (immediate index).

LD4D (scalar plus scalar): Contiguous load four-doubleword structures to four vectors (scalar index).

LD4H (scalar plus immediate): Contiguous load four-halfword structures to four vectors (immediate index).

LD4H (scalar plus scalar): Contiguous load four-halfword structures to four vectors (scalar index).

LD4Q (scalar plus immediate): Contiguous load four-quadword structures to four vectors (immediate index).

LD4Q (scalar plus scalar): Contiguous load four-quadword structures to four vectors (scalar index).

LD4W (scalar plus immediate): Contiguous load four-word structures to four vectors (immediate index).

LD4W (scalar plus scalar): Contiguous load four-word structures to four vectors (scalar index).

LDAPRO: Load-Acquire RCpc octaword.

LDFF1B (scalar plus scalar): Contiguous load first-fault unsigned bytes to vector (scalar index).

LDFF1B (scalar plus vector): Gather load first-fault unsigned bytes to vector (vector index).

LDFF1B (vector plus immediate): Gather load first-fault unsigned bytes to vector (immediate index).

LDFF1D (scalar plus scalar): Contiguous load first-fault doublewords to vector (scalar index).

LDFF1D (scalar plus vector): Gather load first-fault doublewords to vector (vector index).

LDFF1D (vector plus immediate): Gather load first-fault doublewords to vector (immediate index).

LDFF1H (scalar plus scalar): Contiguous load first-fault unsigned halfwords to vector (scalar index).

LDFF1H (scalar plus vector): Gather load first-fault unsigned halfwords to vector (vector index).

LDFF1H (vector plus immediate): Gather load first-fault unsigned halfwords to vector (immediate index).

LDFF1SB (scalar plus scalar): Contiguous load first-fault signed bytes to vector (scalar index).

LDFF1SB (scalar plus vector): Gather load first-fault signed bytes to vector (vector index).

LDFF1SB (vector plus immediate): Gather load first-fault signed bytes to vector (immediate index).

LDFF1SH (scalar plus scalar): Contiguous load first-fault signed halfwords to vector (scalar index).

LDFF1SH (scalar plus vector): Gather load first-fault signed halfwords to vector (vector index).

LDFF1SH (vector plus immediate): Gather load first-fault signed halfwords to vector (immediate index).

LDFF1SW (scalar plus scalar): Contiguous load first-fault signed words to vector (scalar index).

LDFF1SW (scalar plus vector): Gather load first-fault signed words to vector (vector index).

LDFF1SW (vector plus immediate): Gather load first-fault signed words to vector (immediate index).

LDFF1W (scalar plus scalar): Contiguous load first-fault unsigned words to vector (scalar index).

LDFF1W (scalar plus vector): Gather load first-fault unsigned words to vector (vector index).

LDFF1W (vector plus immediate): Gather load first-fault unsigned words to vector (immediate index).

LDNF1B: Contiguous load non-fault unsigned bytes to vector (immediate index).

LDNF1D: Contiguous load non-fault doublewords to vector (immediate index).

LDNF1H: Contiguous load non-fault unsigned halfwords to vector (immediate index).

LDNF1SB: Contiguous load non-fault signed bytes to vector (immediate index).

LDNF1SH: Contiguous load non-fault signed halfwords to vector (immediate index).

LDNF1SW: Contiguous load non-fault signed words to vector (immediate index).

LDNF1W: Contiguous load non-fault unsigned words to vector (immediate index).

LDNT1B (scalar plus immediate, consecutive registers): Contiguous load non-temporal of bytes to multiple consecutive vectors (immediate index).

LDNT1B (scalar plus immediate, single register): Contiguous load non-temporal bytes to vector (immediate index).

LDNT1B (scalar plus scalar, consecutive registers): Contiguous load non-temporal of bytes to multiple consecutive vectors (scalar index).

LDNT1B (scalar plus scalar, single register): Contiguous load non-temporal bytes to vector (scalar index).

LDNT1B (vector plus scalar): Gather load non-temporal unsigned bytes.

LDNT1D (scalar plus immediate, consecutive registers): Contiguous load non-temporal of doublewords to multiple consecutive vectors (immediate index).

LDNT1D (scalar plus immediate, single register): Contiguous load non-temporal doublewords to vector (immediate index).

LDNT1D (scalar plus scalar, consecutive registers): Contiguous load non-temporal of doublewords to multiple consecutive vectors (scalar index).

LDNT1D (scalar plus scalar, single register): Contiguous load non-temporal doublewords to vector (scalar index).

LDNT1D (vector plus scalar): Gather load non-temporal unsigned doublewords.

LDNT1H (scalar plus immediate, consecutive registers): Contiguous load non-temporal of halfwords to multiple consecutive vectors (immediate index).

LDNT1H (scalar plus immediate, single register): Contiguous load non-temporal halfwords to vector (immediate index).

LDNT1H (scalar plus scalar, consecutive registers): Contiguous load non-temporal of halfwords to multiple consecutive vectors (scalar index).

LDNT1H (scalar plus scalar, single register): Contiguous load non-temporal halfwords to vector (scalar index).

LDNT1H (vector plus scalar): Gather load non-temporal unsigned halfwords.

LDNT1SB: Gather load non-temporal signed bytes.

LDNT1SH: Gather load non-temporal signed halfwords.

LDNT1SW: Gather load non-temporal signed words.

LDNT1W (scalar plus immediate, consecutive registers): Contiguous load non-temporal of words to multiple consecutive vectors (immediate index).

LDNT1W (scalar plus immediate, single register): Contiguous load non-temporal words to vector (immediate index).

LDNT1W (scalar plus scalar, consecutive registers): Contiguous load non-temporal of words to multiple consecutive vectors (scalar index).

LDNT1W (scalar plus scalar, single register): Contiguous load non-temporal words to vector (scalar index).

LDNT1W (vector plus scalar): Gather load non-temporal unsigned words.

LDR (predicate): Load predicate register.

LDR (vector): Load vector register.

LSL (immediate, predicated): Logical shift left by immediate (predicated).

LSL (immediate, unpredicated): Logical shift left by immediate (unpredicated).

LSL (vectors): Logical shift left by vector (predicated).

LSL (wide elements, predicated): Logical shift left by 64-bit wide elements (predicated).

LSL (wide elements, unpredicated): Logical shift left by 64-bit wide elements (unpredicated).

LSLR: Reversed logical shift left by vector (predicated).

LSR (immediate, predicated): Logical shift right by immediate (predicated).

LSR (immediate, unpredicated): Logical shift right by immediate (unpredicated).

LSR (vectors): Logical shift right by vector (predicated).

LSR (wide elements, predicated): Logical shift right by 64-bit wide elements (predicated).

LSR (wide elements, unpredicated): Logical shift right by 64-bit wide elements (unpredicated).

LSRR: Reversed logical shift right by vector (predicated).

LUTI2 (8-bit and 16-bit): Lookup table read with 2-bit indices (8-bit and 16-bit).

LUTI4 (8-bit and 16-bit): Lookup table read with 4-bit indices (8-bit and 16-bit).

LUTI6 (16-bit): Lookup table read with 6-bit indices (16-bit).

LUTI6 (8-bit): Lookup table read with 6-bit indices (8-bit).

MAD: Multiply-add to multiplicand (predicated).

MADPT: Multiply-add checked pointer vectors to multiplicand.

MATCH: Detect any matching elements within quadwords, setting the condition flags.

MLA (indexed): Multiply-add by indexed element.

MLA (vectors): Multiply-add (predicated).

MLAPT: Multiply-add checked pointer vectors.

MLS (indexed): Multiply-subtract by indexed element.

MLS (vectors): Multiply-subtract (predicated).

MOV: Move logical bitmask immediate to vector (unpredicated): an alias of DUPM.

MOV: Move predicate (unpredicated): an alias of ORR (predicates).

MOV (immediate, merging): Move signed integer immediate to vector elements (merging): an alias of CPY (immediate, merging).

MOV (immediate, unpredicated): Move signed immediate to vector elements (unpredicated): an alias of DUP (immediate).

MOV (immediate, zeroing): Move signed integer immediate to vector elements (zeroing): an alias of CPY (immediate, zeroing).

MOV (predicate, merging): Move predicates (merging): an alias of SEL (predicates).

MOV (predicate, zeroing): Move predicates (zeroing): an alias of AND (predicates).

MOV (scalar, predicated): Move general-purpose register to vector elements (predicated): an alias of CPY (scalar).

MOV (scalar, unpredicated): Move general-purpose register to vector elements (unpredicated): an alias of DUP (scalar).

MOV (SIMD&FP scalar, predicated): Move SIMD&FP scalar register to vector elements (predicated): an alias of CPY (SIMD&FP scalar).

MOV (SIMD&FP scalar, unpredicated): Move indexed element or SIMD&FP scalar to vector (unpredicated): an alias of DUP (indexed).

MOV (vector, predicated): Move vector elements (predicated): an alias of SEL (vectors).

MOV (vector, unpredicated): Move vector register (unpredicated): an alias of ORR (vectors, unpredicated).

MOVPRFX (predicated): Move prefix (predicated).

MOVPRFX (unpredicated): Move prefix (unpredicated).

MOVS (predicated): Move predicates (zeroing), setting the condition flags: an alias of ANDS.

MOVS (unpredicated): Move predicate (unpredicated), setting the condition flags: an alias of ORRS.

MSB: Multiply-subtract to multiplicand.

MUL (immediate): Multiply by immediate (unpredicated).

MUL (indexed): Multiply by indexed element.

MUL (vectors, predicated): Multiply (predicated).

MUL (vectors, unpredicated): Multiply (unpredicated).

NAND: Bitwise NAND predicates.

NANDS: Bitwise NAND predicates, setting the condition flags.

NBSL: Bitwise inverted select.

NEG: Negate (predicated).

NMATCH: Detect no matching elements within quadwords, setting the condition flags.

NOR: Bitwise NOR predicates.

NORS: Bitwise NOR predicates, setting the condition flags.

NOT (predicate): Bitwise invert predicate: an alias of EOR (predicates).

NOT (vector): Bitwise invert (predicated).

NOTS: Bitwise invert predicate, setting the condition flags: an alias of EORS.

ORN (immediate): Bitwise inclusive OR with inverted immediate (unpredicated): an alias of ORR (immediate).

ORN (predicates): Bitwise inclusive OR inverted predicate.

ORNS: Bitwise inclusive OR inverted predicate, setting the condition flags.

ORQV: Bitwise inclusive OR reduction of quadword vector segments.

ORR (immediate): Bitwise inclusive OR with immediate (unpredicated).

ORR (predicates): Bitwise inclusive OR predicates.

ORR (vectors, predicated): Bitwise inclusive OR (predicated).

ORR (vectors, unpredicated): Bitwise inclusive OR (unpredicated).

ORRS: Bitwise inclusive OR predicates, setting the condition flags.

ORV: Bitwise inclusive OR reduction to scalar.

PEXT (predicate pair): Predicate extract pair from predicate-as-counter.

PEXT (predicate): Predicate extract from predicate-as-counter.

PFALSE: Set all predicate elements to false.

PFIRST: Set the First active predicate element to true.

PMLAL: Multi-vector polynomial multiply long and accumulate.

PMOV (to predicate): Move predicate from vector.

PMOV (to vector): Move predicate to vector.

PMUL: Polynomial multiply (unpredicated).

PMULL: Multi-vector polynomial multiply long.

PMULLB: Polynomial multiply long (bottom).

PMULLT: Polynomial multiply long (top).

PNEXT: Find next active predicate.

PRFB (scalar plus immediate): Contiguous prefetch bytes (immediate index).

PRFB (scalar plus scalar): Contiguous prefetch bytes (scalar index).

PRFB (scalar plus vector): Gather prefetch bytes (scalar plus vector).

PRFB (vector plus immediate): Gather prefetch bytes (vector plus immediate).

PRFD (scalar plus immediate): Contiguous prefetch doublewords (immediate index).

PRFD (scalar plus scalar): Contiguous prefetch doublewords (scalar index).

PRFD (scalar plus vector): Gather prefetch doublewords (scalar plus vector).

PRFD (vector plus immediate): Gather prefetch doublewords (vector plus immediate).

PRFH (scalar plus immediate): Contiguous prefetch halfwords (immediate index).

PRFH (scalar plus scalar): Contiguous prefetch halfwords (scalar index).

PRFH (scalar plus vector): Gather prefetch halfwords (scalar plus vector).

PRFH (vector plus immediate): Gather prefetch halfwords (vector plus immediate).

PRFW (scalar plus immediate): Contiguous prefetch words (immediate index).

PRFW (scalar plus scalar): Contiguous prefetch words (scalar index).

PRFW (scalar plus vector): Gather prefetch words (scalar plus vector).

PRFW (vector plus immediate): Gather prefetch words (vector plus immediate).

PSEL: Predicate select between predicate register or all-false.

PTEST: Set condition flags for predicate.

PTRUE (predicate as counter): Initialize predicate-as-counter to all active.

PTRUE (predicate): Initialize predicate from named constraint.

PTRUES: Initialize predicate from named constraint and set the condition flags.

PUNPKHI, PUNPKLO: Unpack and widen half of predicate.

RADDHNB: Rounding add narrow high part (bottom).

RADDHNT: Rounding add narrow high part (top).

RAX1: Bitwise rotate left by 1 and exclusive-OR.

RBIT: Reverse bits (predicated).

RDFFR (predicated): Return predicate of successfully loaded elements.

RDFFR (unpredicated): Read the first-fault register.

RDFFRS: Return predicate of successfully loaded elements, setting the condition flags.

RDVL: Read multiple of vector register size to scalar register.

REV (predicate): Reverse all elements in a predicate.

REV (vector): Reverse all elements in a vector (unpredicated).

REVB, REVH, REVW: Reverse bytes / halfwords / words within elements (predicated).

REVD: Reverse 64-bit doublewords in elements (predicated).

RSHRNB: Rounding shift right narrow by immediate (bottom).

RSHRNT: Rounding shift right narrow by immediate (top).

RSUBHNB: Rounding subtract narrow high part (bottom).

RSUBHNT: Rounding subtract narrow high part (top).

SABA: Signed absolute difference and accumulate.

SABAL: Two-way signed absolute difference sum and accumulate long.

SABALB: Signed absolute difference and accumulate long (bottom).

SABALT: Signed absolute difference and accumulate long (top).

SABD: Signed absolute difference (predicated).

SABDLB: Signed absolute difference long (bottom).

SABDLT: Signed absolute difference long (top).

SADALP: Signed add and accumulate long pairwise.

SADDLB: Signed add long (bottom).

SADDLBT: Signed add long (bottom + top).

SADDLT: Signed add long (top).

SADDV: Signed add reduction to scalar.

SADDWB: Signed add wide (bottom).

SADDWT: Signed add wide (top).

SBCLB: Subtract with carry long (bottom).

SBCLT: Subtract with carry long (top).

SCLAMP: Signed clamp to minimum/maximum.

SCVTF (bottom, unpredicated): Signed integer convert to floating-point (bottom, unpredicated).

SCVTF (predicated): Signed integer convert to floating-point (predicated).

SCVTFLT: Signed integer convert to floating-point (top, unpredicated).

SDIV: Signed divide (predicated).

SDIVR: Signed reversed divide (predicated).

SDOT (2-way, indexed): Signed integer dot product by indexed element (two-way).

SDOT (2-way, vectors): Signed integer dot product (two-way).

SDOT (4-way, indexed): Signed integer dot product by indexed element (four-way).

SDOT (4-way, vectors): Signed integer dot product (four-way).

SEL (predicates): Conditionally select elements from two predicates.

SEL (vectors): Conditionally select elements from two vectors.

SETFFR: Initialize the first-fault register to all true.

SHADD: Signed halving add.

SHRNB: Shift right narrow by immediate (bottom).

SHRNT: Shift right narrow by immediate (top).

SHSUB: Signed halving subtract.

SHSUBR: Signed halving subtract reversed.

SLI: Shift left and insert (immediate).

SM4E: SM4 encryption and decryption.

SM4EKEY: SM4 key updates.

SMAX (immediate): Signed maximum with immediate (unpredicated).

SMAX (vectors): Signed maximum (predicated).

SMAXP: Signed maximum pairwise.

SMAXQV: Signed maximum reduction of quadword vector segments.

SMAXV: Signed maximum reduction to scalar.

SMIN (immediate): Signed minimum with immediate (unpredicated).

SMIN (vectors): Signed minimum (predicated).

SMINP: Signed minimum pairwise.

SMINQV: Signed minimum reduction of quadword vector segments.

SMINV: Signed minimum reduction to scalar.

SMLALB (indexed): Signed multiply-add long by indexed element (bottom).

SMLALB (vectors): Signed multiply-add long (bottom).

SMLALT (indexed): Signed multiply-add long by indexed element (top).

SMLALT (vectors): Signed multiply-add long (top).

SMLSLB (indexed): Signed multiply-subtract long by indexed element (bottom).

SMLSLB (vectors): Signed multiply-subtract long (bottom).

SMLSLT (indexed): Signed multiply-subtract long by indexed element (top).

SMLSLT (vectors): Signed multiply-subtract long (top).

SMMLA: Signed 8-bit integer matrix multiply-accumulate to 32-bit integer.

SMULH (predicated): Signed multiply returning high half (predicated).

SMULH (unpredicated): Signed multiply returning high half (unpredicated).

SMULLB (indexed): Signed multiply long by indexed element (bottom).

SMULLB (vectors): Signed multiply long (bottom).

SMULLT (indexed): Signed multiply long by indexed element (top).

SMULLT (vectors): Signed multiply long (top).

SPLICE: Splice two vectors under predicate control.

SQABS: Signed saturating absolute value.

SQADD (immediate): Signed saturating add immediate (unpredicated).

SQADD (vectors, predicated): Signed saturating add (predicated).

SQADD (vectors, unpredicated): Signed saturating add (unpredicated).

SQCADD: Saturating complex integer add.

SQCVTN: Signed 32-bit integer saturating extract narrow to interleaved 16-bit integer.

SQCVTUN: Signed 32-bit integer saturating extract narrow to interleaved unsigned 16-bit integer.

SQDECB: Signed saturating decrement scalar by multiple of 8-bit predicate constraint element count.

SQDECD (scalar): Signed saturating decrement scalar by multiple of 64-bit predicate constraint element count.

SQDECD (vector): Signed saturating decrement vector by multiple of 64-bit predicate constraint element count.

SQDECH (scalar): Signed saturating decrement scalar by multiple of 16-bit predicate constraint element count.

SQDECH (vector): Signed saturating decrement vector by multiple of 16-bit predicate constraint element count.

SQDECP (scalar): Signed saturating decrement scalar by count of true predicate elements.

SQDECP (vector): Signed saturating decrement vector by count of true predicate elements.

SQDECW (scalar): Signed saturating decrement scalar by multiple of 32-bit predicate constraint element count.

SQDECW (vector): Signed saturating decrement vector by multiple of 32-bit predicate constraint element count.

SQDMLALB (indexed): Signed saturating doubling multiply-add by indexed element (bottom).

SQDMLALB (vectors): Signed saturating doubling multiply-add (bottom).

SQDMLALBT: Signed saturating doubling multiply-add (bottom × top).

SQDMLALT (indexed): Signed saturating doubling multiply-add by indexed element (top).

SQDMLALT (vectors): Signed saturating doubling multiply-add (top).

SQDMLSLB (indexed): Signed saturating doubling multiply-subtract by indexed element (bottom).

SQDMLSLB (vectors): Signed saturating doubling multiply-subtract (bottom).

SQDMLSLBT: Signed saturating doubling multiply-subtract (bottom × top).

SQDMLSLT (indexed): Signed saturating doubling multiply-subtract by indexed element (top).

SQDMLSLT (vectors): Signed saturating doubling multiply-subtract (top).

SQDMULH (indexed): Signed saturating doubling multiply high by indexed element.

SQDMULH (vectors): Signed saturating doubling multiply high (unpredicated).

SQDMULLB (indexed): Signed saturating doubling multiply by indexed element (bottom).

SQDMULLB (vectors): Signed saturating doubling multiply (bottom).

SQDMULLT (indexed): Signed saturating doubling multiply by indexed element (top).

SQDMULLT (vectors): Signed saturating doubling multiply (top).

SQINCB: Signed saturating increment scalar by multiple of 8-bit predicate constraint element count.

SQINCD (scalar): Signed saturating increment scalar by multiple of 64-bit predicate constraint element count.

SQINCD (vector): Signed saturating increment vector by multiple of 64-bit predicate constraint element count.

SQINCH (scalar): Signed saturating increment scalar by multiple of 16-bit predicate constraint element count.

SQINCH (vector): Signed saturating increment vector by multiple of 16-bit predicate constraint element count.

SQINCP (scalar): Signed saturating increment scalar by count of true predicate elements.

SQINCP (vector): Signed saturating increment vector by count of true predicate elements.

SQINCW (scalar): Signed saturating increment scalar by multiple of 32-bit predicate constraint element count.

SQINCW (vector): Signed saturating increment vector by multiple of 32-bit predicate constraint element count.

SQNEG: Signed saturating negate.

SQRDCMLAH (indexed): Saturating rounding doubling complex integer multiply-add high by indexed element.

SQRDCMLAH (vectors): Saturating rounding doubling complex integer multiply-add high.

SQRDMLAH (indexed): Signed saturating rounding doubling multiply-add high by indexed element.

SQRDMLAH (vectors): Signed saturating rounding doubling multiply-add high (unpredicated).

SQRDMLSH (indexed): Signed saturating rounding doubling multiply-subtract high by indexed element.

SQRDMLSH (vectors): Signed saturating rounding doubling multiply-subtract high (unpredicated).

SQRDMULH (indexed): Signed saturating rounding doubling multiply high by indexed element.

SQRDMULH (vectors): Signed saturating rounding doubling multiply high (unpredicated).

SQRSHL: Signed saturating rounding shift left (predicated).

SQRSHLR: Signed saturating rounding shift left reversed (predicated).

SQRSHRN: Signed saturating rounding shift right narrow by immediate to interleaved integer.

SQRSHRNB: Signed saturating rounding shift right narrow by immediate (bottom).

SQRSHRNT: Signed saturating rounding shift right narrow by immediate (top).

SQRSHRUN: Signed saturating rounding shift right unsigned narrow by immediate and interleave.

SQRSHRUNB: Signed saturating rounding shift right narrow by immediate to unsigned integer (bottom).

SQRSHRUNT: Signed saturating rounding shift right narrow by immediate to unsigned integer (top).

SQSHL (immediate): Signed saturating shift left by immediate.

SQSHL (vectors): Signed saturating shift left (predicated).

SQSHLR: Signed saturating shift left reversed (predicated).

SQSHLU: Signed saturating shift left unsigned by immediate.

SQSHRN: Signed saturating shift right narrow by immediate to interleaved integer.

SQSHRNB: Signed saturating shift right narrow by immediate (bottom).

SQSHRNT: Signed saturating shift right narrow by immediate (top).

SQSHRUN: Signed saturating shift right narrow by immediate to interleaved unsigned integer.

SQSHRUNB: Signed saturating shift right narrow by immediate to unsigned integer (bottom).

SQSHRUNT: Signed saturating shift right narrow by immediate to unsigned integer (top).

SQSUB (immediate): Signed saturating subtract immediate (unpredicated).

SQSUB (vectors, predicated): Signed saturating subtract (predicated).

SQSUB (vectors, unpredicated): Signed saturating subtract (unpredicated).

SQSUBR: Signed saturating subtract reversed (predicated).

SQXTNB: Signed saturating extract narrow (bottom).

SQXTNT: Signed saturating extract narrow (top).

SQXTUNB: Signed saturating extract narrow to unsigned integer (bottom).

SQXTUNT: Signed saturating extract narrow to unsigned integer (top).

SRHADD: Signed rounding halving add.

SRI: Shift right and insert (immediate).

SRSHL: Signed rounding shift left (predicated).

SRSHLR: Signed rounding shift left reversed (predicated).

SRSHR: Signed rounding shift right by immediate.

SRSRA: Signed rounding shift right and accumulate (immediate).

SSHLLB: Signed shift left long by immediate (bottom).

SSHLLT: Signed shift left long by immediate (top).

SSRA: Signed shift right and accumulate (immediate).

SSUBLB: Signed subtract long (bottom).

SSUBLBT: Signed subtract long (bottom - top).

SSUBLT: Signed subtract long (top).

SSUBLTB: Signed subtract long (top - bottom).

SSUBWB: Signed subtract wide (bottom).

SSUBWT: Signed subtract wide (top).

ST1B (scalar plus immediate, consecutive registers): Contiguous store of bytes from multiple consecutive vectors (immediate index).

ST1B (scalar plus immediate, single register): Contiguous store bytes from vector (immediate index).

ST1B (scalar plus scalar, consecutive registers): Contiguous store of bytes from multiple consecutive vectors (scalar index).

ST1B (scalar plus scalar, single register): Contiguous store bytes from vector (scalar index).

ST1B (scalar plus vector): Scatter store bytes from a vector (vector index).

ST1B (vector plus immediate): Scatter store bytes from a vector (immediate index).

ST1D (scalar plus immediate, consecutive registers): Contiguous store of doublewords from multiple consecutive vectors (immediate index).

ST1D (scalar plus immediate, single register): Contiguous store doublewords from vector (immediate index).

ST1D (scalar plus scalar, consecutive registers): Contiguous store of doublewords from multiple consecutive vectors (scalar index).

ST1D (scalar plus scalar, single register): Contiguous store doublewords from vector (scalar index).

ST1D (scalar plus vector): Scatter store doublewords from a vector (vector index).

ST1D (vector plus immediate): Scatter store doublewords from a vector (immediate index).

ST1H (scalar plus immediate, consecutive registers): Contiguous store of halfwords from multiple consecutive vectors (immediate index).

ST1H (scalar plus immediate, single register): Contiguous store halfwords from vector (immediate index).

ST1H (scalar plus scalar, consecutive registers): Contiguous store of halfwords from multiple consecutive vectors (scalar index).

ST1H (scalar plus scalar, single register): Contiguous store halfwords from vector (scalar index).

ST1H (scalar plus vector): Scatter store halfwords from a vector (vector index).

ST1H (vector plus immediate): Scatter store halfwords from a vector (immediate index).

ST1Q: Scatter store quadwords.

ST1W (scalar plus immediate, consecutive registers): Contiguous store of words from multiple consecutive vectors (immediate index).

ST1W (scalar plus immediate, single register): Contiguous store words from vector (immediate index).

ST1W (scalar plus scalar, consecutive registers): Contiguous store of words from multiple consecutive vectors (scalar index).

ST1W (scalar plus scalar, single register): Contiguous store words from vector (scalar index).

ST1W (scalar plus vector): Scatter store words from a vector (vector index).

ST1W (vector plus immediate): Scatter store words from a vector (immediate index).

ST2B (scalar plus immediate): Contiguous store two-byte structures from two vectors (immediate index).

ST2B (scalar plus scalar): Contiguous store two-byte structures from two vectors (scalar index).

ST2D (scalar plus immediate): Contiguous store two-doubleword structures from two vectors (immediate index).

ST2D (scalar plus scalar): Contiguous store two-doubleword structures from two vectors (scalar index).

ST2H (scalar plus immediate): Contiguous store two-halfword structures from two vectors (immediate index).

ST2H (scalar plus scalar): Contiguous store two-halfword structures from two vectors (scalar index).

ST2Q (scalar plus immediate): Contiguous store two-quadword structures from two vectors (immediate index).

ST2Q (scalar plus scalar): Contiguous store two-quadword structures from two vectors (scalar index).

ST2W (scalar plus immediate): Contiguous store two-word structures from two vectors (immediate index).

ST2W (scalar plus scalar): Contiguous store two-word structures from two vectors (scalar index).

ST3B (scalar plus immediate): Contiguous store three-byte structures from three vectors (immediate index).

ST3B (scalar plus scalar): Contiguous store three-byte structures from three vectors (scalar index).

ST3D (scalar plus immediate): Contiguous store three-doubleword structures from three vectors (immediate index).

ST3D (scalar plus scalar): Contiguous store three-doubleword structures from three vectors (scalar index).

ST3H (scalar plus immediate): Contiguous store three-halfword structures from three vectors (immediate index).

ST3H (scalar plus scalar): Contiguous store three-halfword structures from three vectors (scalar index).

ST3Q (scalar plus immediate): Contiguous store three-quadword structures from three vectors (immediate index).

ST3Q (scalar plus scalar): Contiguous store three-quadword structures from three vectors (scalar index).

ST3W (scalar plus immediate): Contiguous store three-word structures from three vectors (immediate index).

ST3W (scalar plus scalar): Contiguous store three-word structures from three vectors (scalar index).

ST4B (scalar plus immediate): Contiguous store four-byte structures from four vectors (immediate index).

ST4B (scalar plus scalar): Contiguous store four-byte structures from four vectors (scalar index).

ST4D (scalar plus immediate): Contiguous store four-doubleword structures from four vectors (immediate index).

ST4D (scalar plus scalar): Contiguous store four-doubleword structures from four vectors (scalar index).

ST4H (scalar plus immediate): Contiguous store four-halfword structures from four vectors (immediate index).

ST4H (scalar plus scalar): Contiguous store four-halfword structures from four vectors (scalar index).

ST4Q (scalar plus immediate): Contiguous store four-quadword structures from four vectors (immediate index).

ST4Q (scalar plus scalar): Contiguous store four-quadword structures from four vectors (scalar index).

ST4W (scalar plus immediate): Contiguous store four-word structures from four vectors (immediate index).

ST4W (scalar plus scalar): Contiguous store four-word structures from four vectors (scalar index).

STLRO: Store-release octaword.

STNT1B (scalar plus immediate, consecutive registers): Contiguous store non-temporal of bytes from multiple consecutive vectors (immediate index).

STNT1B (scalar plus immediate, single register): Contiguous store non-temporal bytes from vector (immediate index).

STNT1B (scalar plus scalar, consecutive registers): Contiguous store non-temporal of bytes from multiple consecutive vectors (scalar index).

STNT1B (scalar plus scalar, single register): Contiguous store non-temporal bytes from vector (scalar index).

STNT1B (vector plus scalar): Scatter store non-temporal bytes.

STNT1D (scalar plus immediate, consecutive registers): Contiguous store non-temporal of doublewords from multiple consecutive vectors (immediate index).

STNT1D (scalar plus immediate, single register): Contiguous store non-temporal doublewords from vector (immediate index).

STNT1D (scalar plus scalar, consecutive registers): Contiguous store non-temporal of doublewords from multiple consecutive vectors (scalar index).

STNT1D (scalar plus scalar, single register): Contiguous store non-temporal doublewords from vector (scalar index).

STNT1D (vector plus scalar): Scatter store non-temporal doublewords.

STNT1H (scalar plus immediate, consecutive registers): Contiguous store non-temporal of halfwords from multiple consecutive vectors (immediate index).

STNT1H (scalar plus immediate, single register): Contiguous store non-temporal halfwords from vector (immediate index).

STNT1H (scalar plus scalar, consecutive registers): Contiguous store non-temporal of halfwords from multiple consecutive vectors (scalar index).

STNT1H (scalar plus scalar, single register): Contiguous store non-temporal halfwords from vector (scalar index).

STNT1H (vector plus scalar): Scatter store non-temporal halfwords.

STNT1W (scalar plus immediate, consecutive registers): Contiguous store non-temporal of words from multiple consecutive vectors (immediate index).

STNT1W (scalar plus immediate, single register): Contiguous store non-temporal words from vector (immediate index).

STNT1W (scalar plus scalar, consecutive registers): Contiguous store non-temporal of words from multiple consecutive vectors (scalar index).

STNT1W (scalar plus scalar, single register): Contiguous store non-temporal words from vector (scalar index).

STNT1W (vector plus scalar): Scatter store non-temporal words.

STR (predicate): Store predicate register.

STR (vector): Store vector register.

SUB (immediate): Subtract immediate (unpredicated).

SUB (vectors, predicated): Subtract (predicated).

SUB (vectors, unpredicated): Subtract (unpredicated).

SUBHNB: Subtract narrow high part (bottom).

SUBHNT: Subtract narrow high part (top).

SUBP: Subtract pairwise.

SUBPT (predicated): Subtract checked pointer vectors (predicated).

SUBPT (unpredicated): Subtract checked pointer vectors (unpredicated).

SUBR (immediate): Reversed subtract from immediate (unpredicated).

SUBR (vectors): Reversed subtract (predicated).

SUDOT: Signed by unsigned 8-bit integer dot product by indexed element to 32-bit integer.

SUNPKHI, SUNPKLO: Signed unpack and extend half of vector.

SUQADD: Signed saturating unsigned add.

SXTB, SXTH, SXTW: Signed byte / halfword / word extend (predicated).

TBL: Programmable table lookup in one or two vector table (zeroing).

TBLQ: Programmable table lookup within each quadword vector segment (zeroing).

TBX: Programmable table lookup in single vector table (merging).

TBXQ: Programmable table lookup within each quadword vector segment (merging).

TRN1, TRN2 (predicates): Interleave even or odd elements from two predicates.

TRN1, TRN2 (vectors): Interleave even or odd elements from two vectors.

UABA: Unsigned absolute difference and accumulate.

UABAL: Two-way unsigned absolute difference sum and accumulate long.

UABALB: Unsigned absolute difference and accumulate long (bottom).

UABALT: Unsigned absolute difference and accumulate long (top).

UABD: Unsigned absolute difference (predicated).

UABDLB: Unsigned absolute difference long (bottom).

UABDLT: Unsigned absolute difference long (top).

UADALP: Unsigned add and accumulate long pairwise.

UADDLB: Unsigned add long (bottom).

UADDLT: Unsigned add long (top).

UADDV: Unsigned add reduction to scalar.

UADDWB: Unsigned add wide (bottom).

UADDWT: Unsigned add wide (top).

UCLAMP: Unsigned clamp to minimum/maximum.

UCVTF (bottom, unpredicated): Unsigned integer convert to floating-point (bottom, unpredicated).

UCVTF (predicated): Unsigned integer convert to floating-point (predicated).

UCVTFLT: Unsigned integer convert to floating-point (top, unpredicated).

UDIV: Unsigned divide (predicated).

UDIVR: Unsigned reversed divide (predicated).

UDOT (2-way, indexed): Unsigned integer dot product by indexed element (two-way).

UDOT (2-way, vectors): Unsigned integer dot product (two-way).

UDOT (4-way, indexed): Unsigned integer dot product by indexed element (four-way).

UDOT (4-way, vectors): Unsigned integer dot product (four-way).

UHADD: Unsigned halving add.

UHSUB: Unsigned halving subtract.

UHSUBR: Unsigned halving subtract reversed.

UMAX (immediate): Unsigned maximum with immediate (unpredicated).

UMAX (vectors): Unsigned maximum (predicated).

UMAXP: Unsigned maximum pairwise.

UMAXQV: Unsigned maximum reduction of quadword vector segments.

UMAXV: Unsigned maximum reduction to scalar.

UMIN (immediate): Unsigned minimum with immediate (unpredicated).

UMIN (vectors): Unsigned minimum (predicated).

UMINP: Unsigned minimum pairwise.

UMINQV: Unsigned minimum reduction of quadword vector segments.

UMINV: Unsigned minimum reduction to scalar.

UMLALB (indexed): Unsigned multiply-add long by indexed element (bottom).

UMLALB (vectors): Unsigned multiply-add long (bottom).

UMLALT (indexed): Unsigned multiply-add long by indexed element (top).

UMLALT (vectors): Unsigned multiply-add long (top).

UMLSLB (indexed): Unsigned multiply-subtract long by indexed element (bottom).

UMLSLB (vectors): Unsigned multiply-subtract long (bottom).

UMLSLT (indexed): Unsigned multiply-subtract long by indexed element (top).

UMLSLT (vectors): Unsigned multiply-subtract long (top).

UMMLA: Unsigned 8-bit integer matrix multiply-accumulate to 32-bit integer.

UMULH (predicated): Unsigned multiply returning high half (predicated).

UMULH (unpredicated): Unsigned multiply returning high half (unpredicated).

UMULLB (indexed): Unsigned multiply long by indexed element (bottom).

UMULLB (vectors): Unsigned multiply long (bottom).

UMULLT (indexed): Unsigned multiply long by indexed element (top).

UMULLT (vectors): Unsigned multiply long (top).

UQADD (immediate): Unsigned saturating add immediate (unpredicated).

UQADD (vectors, predicated): Unsigned saturating add (predicated).

UQADD (vectors, unpredicated): Unsigned saturating add (unpredicated).

UQCVTN: Unsigned 32-bit integer saturating extract narrow to interleaved 16-bit integer.

UQDECB: Unsigned saturating decrement scalar by multiple of 8-bit predicate constraint element count.

UQDECD (scalar): Unsigned saturating decrement scalar by multiple of 64-bit predicate constraint element count.

UQDECD (vector): Unsigned saturating decrement vector by multiple of 64-bit predicate constraint element count.

UQDECH (scalar): Unsigned saturating decrement scalar by multiple of 16-bit predicate constraint element count.

UQDECH (vector): Unsigned saturating decrement vector by multiple of 16-bit predicate constraint element count.

UQDECP (scalar): Unsigned saturating decrement scalar by count of true predicate elements.

UQDECP (vector): Unsigned saturating decrement vector by count of true predicate elements.

UQDECW (scalar): Unsigned saturating decrement scalar by multiple of 32-bit predicate constraint element count.

UQDECW (vector): Unsigned saturating decrement vector by multiple of 32-bit predicate constraint element count.

UQINCB: Unsigned saturating increment scalar by multiple of 8-bit predicate constraint element count.

UQINCD (scalar): Unsigned saturating increment scalar by multiple of 64-bit predicate constraint element count.

UQINCD (vector): Unsigned saturating increment vector by multiple of 64-bit predicate constraint element count.

UQINCH (scalar): Unsigned saturating increment scalar by multiple of 16-bit predicate constraint element count.

UQINCH (vector): Unsigned saturating increment vector by multiple of 16-bit predicate constraint element count.

UQINCP (scalar): Unsigned saturating increment scalar by count of true predicate elements.

UQINCP (vector): Unsigned saturating increment vector by count of true predicate elements.

UQINCW (scalar): Unsigned saturating increment scalar by multiple of 32-bit predicate constraint element count.

UQINCW (vector): Unsigned saturating increment vector by multiple of 32-bit predicate constraint element count.

UQRSHL: Unsigned saturating rounding shift left (predicated).

UQRSHLR: Unsigned saturating rounding shift left reversed (predicated).

UQRSHRN: Unsigned saturating rounding shift right narrow by immediate to interleaved integer.

UQRSHRNB: Unsigned saturating rounding shift right narrow by immediate (bottom).

UQRSHRNT: Unsigned saturating rounding shift right narrow by immediate (top).

UQSHL (immediate): Unsigned saturating shift left by immediate.

UQSHL (vectors): Unsigned saturating shift left (predicated).

UQSHLR: Unsigned saturating shift left reversed (predicated).

UQSHRN: Unsigned saturating shift right narrow by immediate to interleaved integer.

UQSHRNB: Unsigned saturating shift right narrow by immediate (bottom).

UQSHRNT: Unsigned saturating shift right narrow by immediate (top).

UQSUB (immediate): Unsigned saturating subtract immediate (unpredicated).

UQSUB (vectors, predicated): Unsigned saturating subtract (predicated).

UQSUB (vectors, unpredicated): Unsigned saturating subtract (unpredicated).

UQSUBR: Unsigned saturating subtract reversed (predicated).

UQXTNB: Unsigned saturating extract narrow (bottom).

UQXTNT: Unsigned saturating extract narrow (top).

URECPE: Unsigned reciprocal estimate (predicated).

URHADD: Unsigned rounding halving add.

URSHL: Unsigned rounding shift left (predicated).

URSHLR: Unsigned rounding shift left reversed (predicated).

URSHR: Unsigned rounding shift right by immediate.

URSQRTE: Unsigned reciprocal square root estimate (predicated).

URSRA: Unsigned rounding shift right and accumulate (immediate).

USDOT (indexed): Unsigned by signed 8-bit integer dot product by indexed element to 32-bit integer.

USDOT (vectors): Unsigned by signed 8-bit integer dot product to 32-bit integer.

USHLLB: Unsigned shift left long by immediate (bottom).

USHLLT: Unsigned shift left long by immediate (top).

USMMLA: Unsigned by signed 8-bit integer matrix multiply-accumulate to 32-bit integer.

USQADD: Unsigned saturating signed add.

USRA: Unsigned shift right and accumulate (immediate).

USUBLB: Unsigned subtract long (bottom).

USUBLT: Unsigned subtract long (top).

USUBWB: Unsigned subtract wide (bottom).

USUBWT: Unsigned subtract wide (top).

UUNPKHI, UUNPKLO: Unsigned unpack and extend half of vector.

UXTB, UXTH, UXTW: Unsigned byte / halfword / word extend (predicated).

UZP1, UZP2 (predicates): Concatenate even or odd elements from two predicates.

UZP1, UZP2 (vectors): Concatenate even or odd elements from two vectors.

UZPQ1: Concatenate even elements within each pair of quadword vector segments.

UZPQ2: Concatenate odd elements within each pair of quadword vector segments.

WHILEGE (predicate as counter): While decrementing signed scalar greater than or equal to scalar (predicate-as-counter).

WHILEGE (predicate pair): While decrementing signed scalar greater than or equal to scalar (pair of predicates).

WHILEGE (predicate): While decrementing signed scalar greater than or equal to scalar.

WHILEGT (predicate as counter): While decrementing signed scalar greater than scalar (predicate-as-counter).

WHILEGT (predicate pair): While decrementing signed scalar greater than scalar (pair of predicates).

WHILEGT (predicate): While decrementing signed scalar greater than scalar.

WHILEHI (predicate as counter): While decrementing unsigned scalar higher than scalar (predicate-as-counter).

WHILEHI (predicate pair): While decrementing unsigned scalar higher than scalar (pair of predicates).

WHILEHI (predicate): While decrementing unsigned scalar higher than scalar.

WHILEHS (predicate as counter): While decrementing unsigned scalar higher than or same as scalar (predicate-as-counter).

WHILEHS (predicate pair): While decrementing unsigned scalar higher than or same as scalar (pair of predicates).

WHILEHS (predicate): While decrementing unsigned scalar higher than or same as scalar.

WHILELE (predicate as counter): While incrementing signed scalar less than or equal to scalar (predicate-as-counter).

WHILELE (predicate pair): While incrementing signed scalar less than or equal to scalar (pair of predicates).

WHILELE (predicate): While incrementing signed scalar less than or equal to scalar.

WHILELO (predicate as counter): While incrementing unsigned scalar lower than scalar (predicate-as-counter).

WHILELO (predicate pair): While incrementing unsigned scalar lower than scalar (pair of predicates).

WHILELO (predicate): While incrementing unsigned scalar lower than scalar.

WHILELS (predicate as counter): While incrementing unsigned scalar lower than or same as scalar (predicate-as-counter).

WHILELS (predicate pair): While incrementing unsigned scalar lower than or same as scalar (pair of predicates).

WHILELS (predicate): While incrementing unsigned scalar lower than or same as scalar.

WHILELT (predicate as counter): While incrementing signed scalar less than scalar (predicate-as-counter).

WHILELT (predicate pair): While incrementing signed scalar less than scalar (pair of predicates).

WHILELT (predicate): While incrementing signed scalar less than scalar.

WHILERW: While free of read-after-write conflicts.

WHILEWR: While free of write-after-read/write conflicts.

WRFFR: Write the first-fault register.

XAR: Bitwise exclusive-OR and rotate right by immediate.

ZIP1, ZIP2 (predicates): Interleave elements from two half predicates.

ZIP1, ZIP2 (vectors): Interleave elements from two half vectors.

ZIPQ1: Interleave elements from low halves of each pair of quadword vector segments.

ZIPQ2: Interleave elements from high halves of each pair of quadword vector segments.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ABS

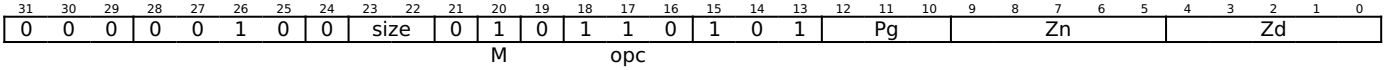
Absolute value (predicated)

This instruction calculates the absolute value of the signed integer in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. If the value in an element of the source vector is the most negative value, the result for that element is the same as the value in that element of the source vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

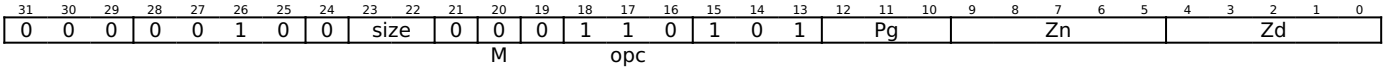
```
ABS <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
ABS <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        var element : integer = SInt(operand[e*:esize]);
        element = Abs(element);
        result[e*:esize] = element[esize-1:0];

    end;
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

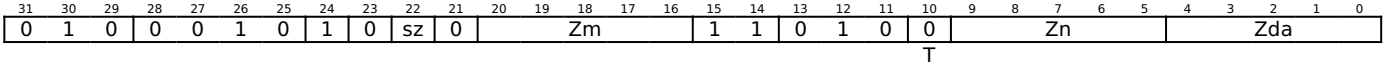
ADCLB

Add with carry long (bottom)

This instruction adds the even-numbered elements of the first source vector and the 1-bit carry from the least-significant bit of the odd-numbered elements of the second source vector to the even-numbered elements of the destination and accumulator vector. The 1-bit carry output is placed in the corresponding odd-numbered element of the destination vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
ADCLB <Zda>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (esize * 2);
let operand : bits(VL) = Z{}(n);
let carries : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for p = 0 to pairs-1 do
    let element1 : bits(esize) = result[(2*p + 0)*:esize];
    let element2 : bits(esize) = operand[(2*p + 0)*:esize];
    let carry_in : bit = carries[(2*p + 1)*:esize][0];

    let (res, nzcv) = AddWithCarry{esize}(element1, element2, carry_in);
    let carry_out : bit = nzcv[1];

    result[(2*p + 0)*:esize] = res;
    result[(2*p + 1)*:esize] = ZeroExtend{esize}(carry_out);
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

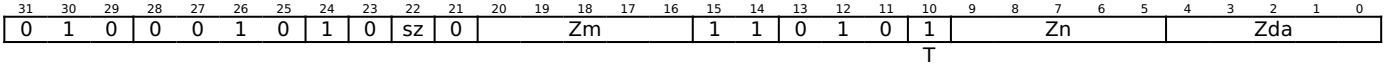
ADCLT

Add with carry long (top)

This instruction adds the odd-numbered elements of the first source vector and the 1-bit carry from the least-significant bit of the odd-numbered elements of the second source vector to the even-numbered elements of the destination and accumulator vector. The 1-bit carry output is placed in the corresponding odd-numbered element of the destination vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
ADCLT <Zda>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (esize * 2);
let operand : bits(VL) = Z{}(n);
let carries : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for p = 0 to pairs-1 do
    let element1 : bits(esize) = result[(2*p + 0)*:esize];
    let element2 : bits(esize) = operand[(2*p + 1)*:esize];
    let carry_in : bit = carries[(2*p + 1)*:esize][0];

    let (res, nzcv) = AddWithCarry{esize}(element1, element2, carry_in);
    let carry_out : bit = nzcv[1];

    result[(2*p + 0)*:esize] = res;
    result[(2*p + 1)*:esize] = ZeroExtend{esize}(carry_out);
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADD (immediate)

Add immediate (unpredicated)

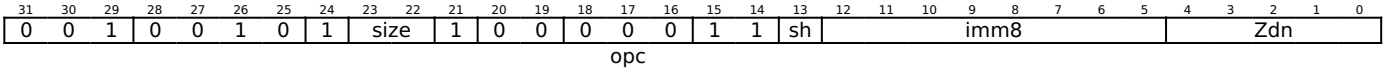
This instruction adds an unsigned immediate to each element of the source vector, and destructively places the results in the corresponding elements of the source vector. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ADD <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    result[e*:esize] = element1 + imm;

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

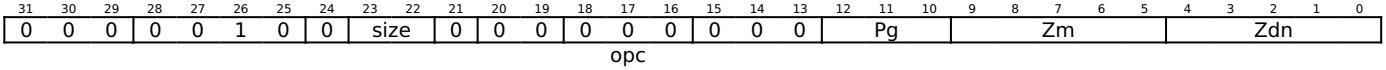
ADD (vectors, predicated)

Add (predicated)

This instruction adds Active elements of the first source vector to the corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = element1 + element2;
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

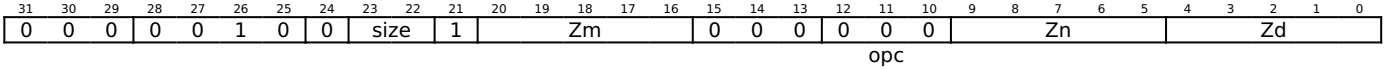
ADD (vectors, unpredicated)

Add (unpredicated)

This instruction adds all elements of the first source vector to the corresponding elements of the second source vector, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ADD <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = element1 + element2;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

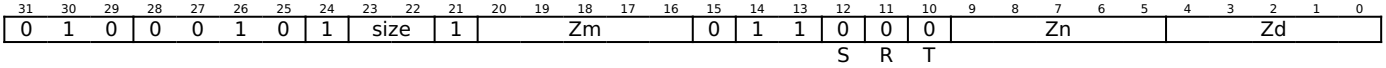
ADDHNB

Add narrow high part (bottom)

This instruction adds each vector element of the first source vector to the corresponding vector element of the second source vector, and places the most significant half of the result in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
ADDHNB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	B
10	H
11	S

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    let res : integer = (element1 + element2) >> halfesize;
    result[(2*e + 0)*:halfesize] = res[halfesize-1:0];
    result[(2*e + 1)*:halfesize] = Zeros{halfesize};

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

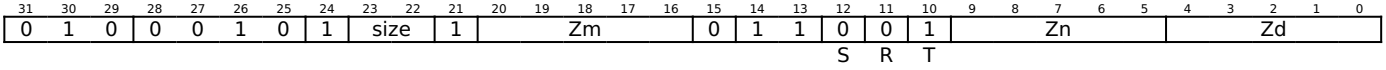
ADDHNT

Add narrow high part (top)

This instruction adds each vector element of the first source vector to the corresponding vector element of the second source vector, and places the most significant half of the result in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
ADDHNT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	B
10	H
11	S

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(d);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    let res : integer = (element1 + element2) >> halfesize;
    result[(2*e + 1)*:halfesize] = res[halfesize-1:0];

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

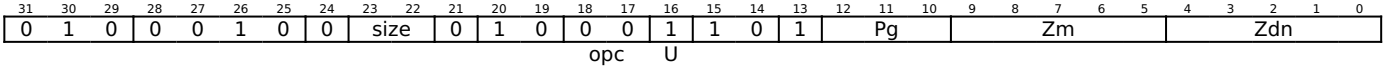
ADDP

Add pairwise

This instruction adds pairs of adjacent elements within each source vector, and destructively places the interleaved results in the first source vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
ADDP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    if !ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand1[e*:esize];
    else
        if IsEven(e) then
            element1 = UInt(operand1[(e + 0)*:esize]);
            element2 = UInt(operand1[(e + 1)*:esize]);
        else
            element1 = UInt(operand2[(e - 1)*:esize]);
            element2 = UInt(operand2[(e + 0)*:esize]);
        end;
        let res : integer = element1 + element2;
        result[e*:esize] = res[esize-1:0];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

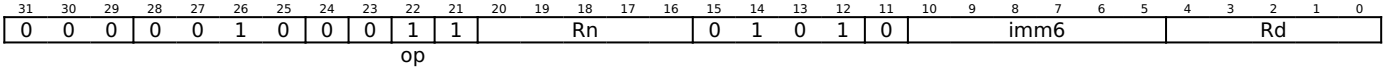
ADDPL

Add multiple of predicate register size to scalar register

This instruction adds the current predicate register size in bytes multiplied by an immediate in the range -32 to 31 to the 64-bit source general-purpose register or current stack pointer and places the result in the 64-bit destination general-purpose register or current stack pointer.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ADDPL <Xd|SP>, <Xn|SP>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let d : integer = UInt(Rd);
let imm : integer = SInt(imm6);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.
<imm>	Is the signed immediate operand, in the range -32 to 31, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let result : bits(64) = operand1 + (imm * (PL DIV 8));

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

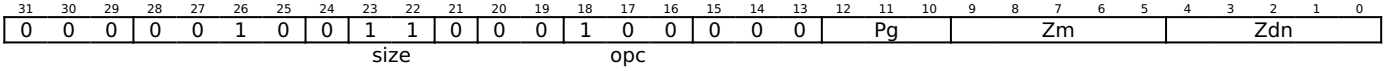
ADDPT (predicated)

Add checked pointer vectors (predicated)

This instruction adds, with pointer check, Active elements of the first source vector to the corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.
This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE && FEAT_CPA)



Encoding

```
ADDPT <Zdn>.D, <Pg>/M, <Zdn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_CPA) then
    EndOfDecode(Decode_UNDEF);
end;
let g : integer =  UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer =  UInt(Zm);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 64;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 64) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(64) = operand1[e*:64];
    let element2 : bits(64) = operand2[e*:64];
    if ActivePredicateElement{PL}(mask, e, 64) then
        let res : bits(64) = element1 + element2;
        result[e*:64] = PointerAddCheck(res, element1);
    else
        result[e*:64] = operand1[e*:64];
    end;

    Z{VL}(dn) = result;
end;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDPT (unpredicated)

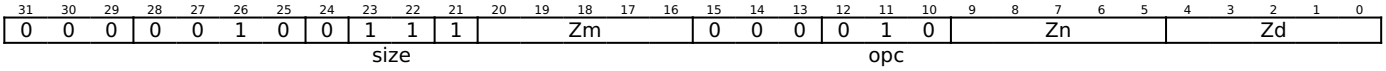
Add checked pointer vectors (unpredicated)

This instruction adds, with pointer check, all elements of the first source vector to the corresponding elements of the second source vector and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE && FEAT_CPA)



Encoding

```
ADDPT <Zd>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_CPA) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 64;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(64) = operand1[e*:64];
    let element2 : bits(64) = operand2[e*:64];
    let res : bits(64) = element1 + element2;
    result[e*:64] = PointerAddCheck(res, element1);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDQP

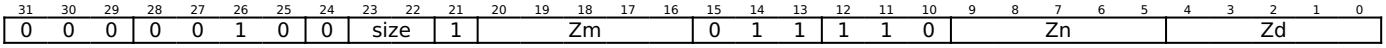
Add pairwise within quadword vector segments (unpredicated)

This instruction adds pairs of adjacent elements within each concatenated pair of 128-bit segments of the first and second source vectors, and places the results in the corresponding 128-bit segment of the destination vector.

This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
ADDQP <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for s = 0 to segments-1 do
    let seg1 : bits(128) = operand1[s*:128];
    let seg2 : bits(128) = operand2[s*:128];
    let concat : bits(256) = seg2::seg1;

    for e = 0 to elempersegment-1 do
        let element1 : bits(esize) = concat[(2*e + 0)*:esize];
        let element2 : bits(esize) = concat[(2*e + 1)*:esize];
        result[(s*elempersegment + e)*:esize] = element1 + element2;
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

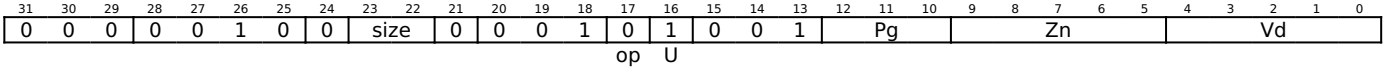
ADDQV

Add reduction of quadword vector segments

This instruction adds the elements of the same position from each 128-bit source vector segment, and places each result into the corresponding element of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as zero.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ADDQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	16B
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(128) = Zeros{};
var stmp : bits(128) = Zeros{};

var dtmp : integer;

for e = 0 to elempersegment-1 do
    dtmp = 0;
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp = operand[s*:128];
            dtmp = dtmp + UInt(stmp[e*:esize]);
        end;
    end;
    result[e*:esize] = dtmp[esize-1:0];
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDSUBP

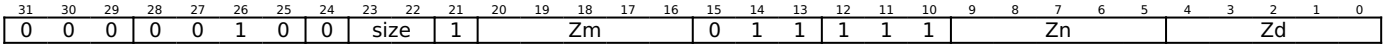
Add subtract pairwise

This instruction adds pairs of adjacent elements within the first source vector and subtracts pairs of adjacent elements within the second source vector, and places the interleaved results in the destination vector.

This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
ADDSUBP <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (esize * 2);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to pairs-1 do
    let elt1_a : integer = UInt(operand1[(2*e + 0)*:esize]);
    let elt1_b : integer = UInt(operand1[(2*e + 1)*:esize]);
    let elt2_a : integer = UInt(operand2[(2*e + 0)*:esize]);
    let elt2_b : integer = UInt(operand2[(2*e + 1)*:esize]);
    let res1 : integer = elt1_a + elt1_b;
    let res2 : integer = elt2_a - elt2_b;
    result[(2*e + 0)*:esize] = res1[esize-1:0];
    result[(2*e + 1)*:esize] = res2[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

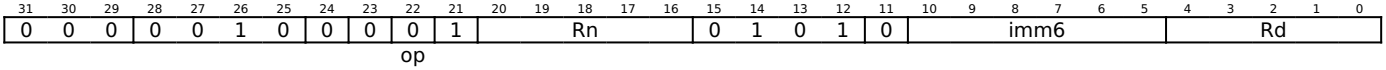
ADDVL

Add multiple of vector register size to scalar register

This instruction adds the current vector register size in bytes multiplied by an immediate in the range -32 to 31 to the 64-bit source general-purpose register or current stack pointer, and places the result in the 64-bit destination general-purpose register or current stack pointer.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ADDVL <Xd|SP>, <Xn|SP>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let d : integer = UInt(Rd);
let imm : integer = SInt(imm6);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.
<imm>	Is the signed immediate operand, in the range -32 to 31, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let result : bits(64) = operand1 + (imm * (VL DIV 8));

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADR

Calculate vector address

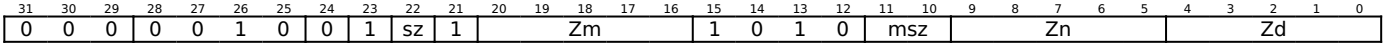
This instruction optionally sign-extends or zero-extends the least significant 32 bits of each element from a vector of offsets or indices in the second source vector, and scales each index by 2, 4, or 8. The scaled indices are then added to a vector of base addresses from the first source vector, and the resulting addresses are placed in the destination vector. This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: Packed offsets , Unpacked 32-bit signed offsets and Unpacked 32-bit unsigned offsets

Packed offsets

(FEAT_SVE)



Encoding

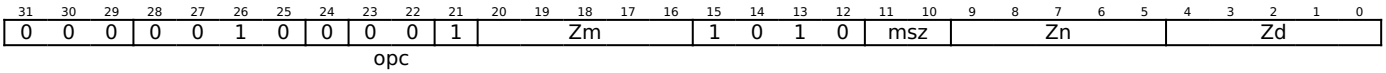
```
ADR <Zd>.<T>, [<Zn>.<T>, <Zm>.<T>{, <mod> <amount>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let osize : integer{} = esize;
let unsigned : boolean = TRUE;
let mbytes : integer{} = 1 << UInt(msz);
```

Unpacked 32-bit signed offsets

(FEAT_SVE)



Encoding

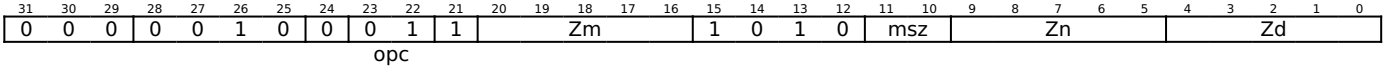
```
ADR <Zd>.D, [<Zn>.D, <Zm>.D, SXTW{<amount>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let osize : integer{} = 32;
let unsigned : boolean = FALSE;
let mbytes : integer{} = 1 << UInt(msz);
```

Unpacked 32-bit unsigned offsets

(FEAT_SVE)



Encoding

```
ADR <Zd>.D, [<Zn>.D, <Zm>.D, UXTW{<amount>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let osize : integer{} = 32;
let unsigned : boolean = TRUE;
let mbytes : integer{} = 1 << UInt(msz);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Zn> Is the name of the base scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the offset scalable vector register, encoded in the "Zm" field.

<mod> Is the index extend and shift specifier, encoded in “msz”:

msz	<mod>
00	[absent]
x1	LSL
10	LSL

<amount> Is the index shift amount, encoded in “msz”:

msz	<amount>
00	[absent]
01	#1
10	#2
11	#3

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let base : bits(VL) = Z{}(n);
let offs : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let addr : bits(esize) = base[e*:esize];
    let offselt : bits(osize) = offs[e*:esize][osize-1:0];
    let offset : integer = if unsigned then UInt(offselt) else SInt(offselt);
    result[e*:esize] = addr + (offset * mbytes);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AESD (indexed)

Multi-vector AES single round decryption

This instruction reads a 16-byte state array from each 128-bit segment of the two or four first source vectors, together with a round key from the indexed 128-bit segment of the corresponding 512-bit portion of the second source vector. Each state array undergoes a single round of the AddRoundKey(), InvShiftRows(), and InvSubBytes() transformations in accordance with the AES standard. Each updated state array is destructively placed in the corresponding segment of the two or four first source vectors.

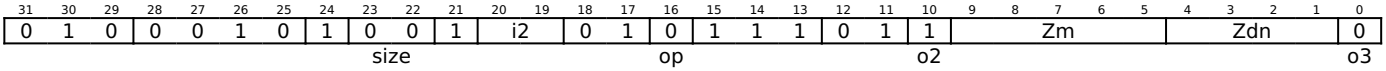
When the vector length is less than 512 bits, the most significant bits of the index are ignored to select the indexed 128-bit segment of the second source vector. This instruction is unpredicated.

This instruction is legal when executed in Streaming SVE mode if both FEAT_SSVE_AES and FEAT_SVE_AES2 are implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SVE_AES2)



Encoding

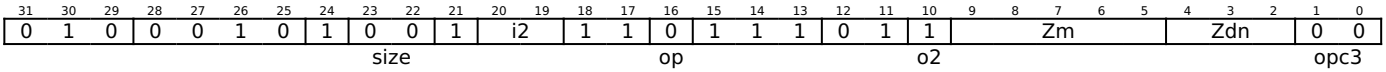
```
AESD { <Zdn1>.B-<Zdn2>.B }, { <Zdn1>.B-<Zdn2>.B }, <Zm>.Q[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES2) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn::'0');
var index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SVE_AES2)



Encoding

```
AESD { <Zdn1>.B-<Zdn4>.B }, { <Zdn1>.B-<Zdn4>.B }, <Zm>.Q[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES2) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn::'00');
var index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<index>	Is the round key index, in the range 0 to 3, encoded in the "i2" field.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_AES) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
if VL == 128 then index = 0; end;
if VL == 256 then index = index MOD 2; end;
let segments : integer = VL DIV 128;
let operand2 : bits(VL) = Z{}(m);
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{}(dn + r);
    for s = 0 to segments-1 do
        let keyindex : integer = (s - (s MOD 4)) + index;
        let res : bits(128) = operand1[s*:128] XOR operand2[keyindex*:128];
        results[[r]][s*:128] = AESInvSubBytes(AESInvShiftRows(res));

    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn + r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AESD (vectors)

AES single round decryption

This instruction reads a 16-byte state array from each 128-bit segment of the first source vector, together with a round key from the corresponding 128-bit segment of the second source vector. Each state array undergoes a single round of the AddRoundKey(), InvShiftRows(), and InvSubBytes() transformations in accordance with the AES standard. Each updated state array is destructively placed in the corresponding segment of the first source vector. This instruction is unpredicated.

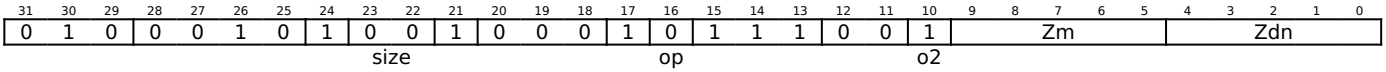
ID_AA64ZFR0_EL1.AES indicates whether this instruction is implemented.

This instruction is legal when executed in Streaming SVE mode if one of the following is true:

- Both FEAT_SSVE_AES and FEAT_SVE_AES are implemented.
- FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE_AES)



Encoding

```
AESD <Zdn>.B, <Zdn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_AES) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

result = operand1 XOR operand2;
for s = 0 to segments-1 do
    result[s*:128] = AESInvSubBytes(AESInvShiftRows(result[s*:128]));
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AESDIMC

Multi-vector AES single round decryption and inverse mix columns

This instruction reads a 16-byte state array from each 128-bit segment of the two or four first source vectors, together with a round key from the indexed 128-bit segment of the corresponding 512-bit portion of the second source vector. Each state array undergoes a single round of the AddRoundKey(), InvShiftRows(), InvSubBytes(), and InvMixColumns() transformations in accordance with the AES standard. Each updated state array is destructively placed in the corresponding segment of the two or four first source vectors.

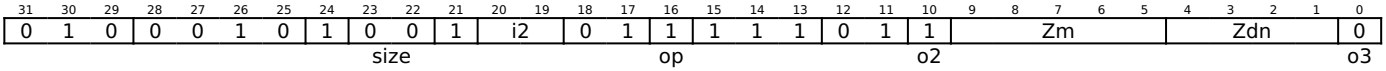
When the vector length is less than 512 bits, the most significant bits of the index are ignored to select the indexed 128-bit segment of the second source vector. This instruction is unpredicated.

This instruction is legal when executed in Streaming SVE mode if both FEAT_SSVE_AES and FEAT_SVE_AES2 are implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SVE_AES2)



Encoding

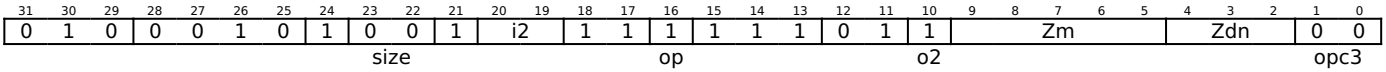
```
AESDIMC { <Zdn1>.B-<Zdn2>.B }, { <Zdn1>.B-<Zdn2>.B }, <Zm>.Q[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES2) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn::'0');
var index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SVE_AES2)



Encoding

```
AESDIMC { <Zdn1>.B-<Zdn4>.B }, { <Zdn1>.B-<Zdn4>.B }, <Zm>.Q[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES2) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn::'00');
var index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<index>	Is the round key index, in the range 0 to 3, encoded in the "i2" field.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_AES) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
if VL == 128 then index = 0; end;
if VL == 256 then index = index MOD 2; end;
let segments : integer = VL DIV 128;
let operand2 : bits(VL) = Z{}(m);
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{}(dn + r);
    for s = 0 to segments-1 do
        let keyindex : integer = (s - (s MOD 4)) + index;
        let res : bits(128) = operand1[s*:128] XOR operand2[keyindex*:128];
        results[[r]][s*:128] = AESInvMixColumns(AESInvSubBytes(AESInvShiftRows(res)));

    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn + r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AESE (indexed)

Multi-vector AES single round encryption

This instruction reads a 16-byte state array from each 128-bit segment of the two or four first source vectors, together with a round key from the indexed 128-bit segment of the corresponding 512-bit portion of the second source vector. Each state array undergoes a single round of the AddRoundKey(), ShiftRows(), and SubBytes() transformations in accordance with the AES standard. Each updated state array is destructively placed in the corresponding segment of the two or four first source vectors.

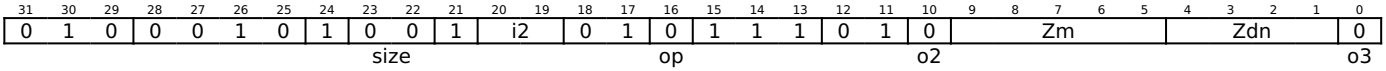
When the vector length is less than 512 bits, the most significant bits of the index are ignored to select the indexed 128-bit segment of the second source vector. This instruction is unpredicated.

This instruction is legal when executed in Streaming SVE mode if both FEAT_SSVE_AES and FEAT_SVE_AES2 are implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SVE_AES2)



Encoding

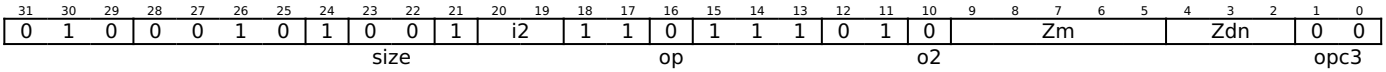
```
AESE { <Zdn1>.B-<Zdn2>.B }, { <Zdn1>.B-<Zdn2>.B }, <Zm>.Q[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES2) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn::'0');
var index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SVE_AES2)



Encoding

```
AESE { <Zdn1>.B-<Zdn4>.B }, { <Zdn1>.B-<Zdn4>.B }, <Zm>.Q[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES2) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn::'00');
var index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<index>	Is the round key index, in the range 0 to 3, encoded in the "i2" field.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_AES) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
if VL == 128 then index = 0; end;
if VL == 256 then index = index MOD 2; end;
let segments : integer = VL DIV 128;
let operand2 : bits(VL) = Z{}(m);
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{}(dn + r);
    for s = 0 to segments-1 do
        let keyindex : integer = (s - (s MOD 4)) + index;
        let res : bits(128) = operand1[s*:128] XOR operand2[keyindex*:128];
        results[[r]][s*:128] = AESSubBytes(AESShiftRows(res));

    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn + r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AESE (vectors)

AES single round encryption

This instruction reads a 16-byte state array from each 128-bit segment of the first source vector together with a round key from the corresponding 128-bit segment of the second source vector. Each state array undergoes a single round of the AddRoundKey(), ShiftRows(), and SubBytes() transformations in accordance with the AES standard. Each updated state array is destructively placed in the corresponding segment of the first source vector. This instruction is unpredicated.

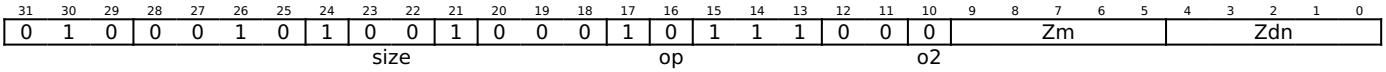
ID_AA64ZFR0_EL1.AES indicates whether this instruction is implemented.

This instruction is legal when executed in Streaming SVE mode if one of the following is true:

- Both FEAT_SSVE_AES and FEAT_SVE_AES are implemented.
- FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE_AES)



Encoding

```
AESE <Zdn>.B, <Zdn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_AES) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

result = operand1 XOR operand2;
for s = 0 to segments-1 do
    result[s*:128] = AESSubBytes(AESShiftRows(result[s*:128]));
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AESEMC

Multi-vector AES single round encryption and mix columns

This instruction reads a 16-byte state array from each 128-bit segment of the two or four first source vectors, together with a round key from the indexed 128-bit segment of the corresponding 512-bit portion of the second source vector. Each state array undergoes a single round of the AddRoundKey(), ShiftRows(), SubBytes(), and MixColumns() transformations in accordance with the AES standard. Each updated state array is destructively placed in the corresponding segment of the two or four first source vectors.

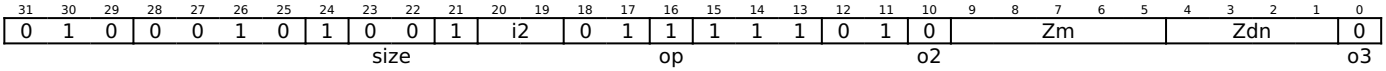
When the vector length is less than 512 bits, the most significant bits of the index are ignored to select the indexed 128-bit segment of the second source vector. This instruction is unpredicated.

This instruction is legal when executed in Streaming SVE mode if both FEAT_SSVE_AES and FEAT_SVE_AES2 are implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SVE_AES2)



Encoding

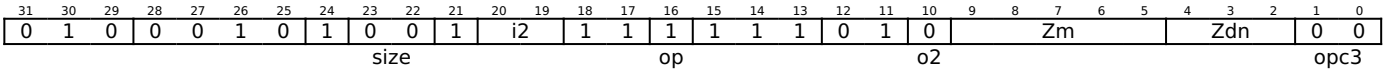
```
AESEMC { <Zdn1>.B-<Zdn2>.B }, { <Zdn1>.B-<Zdn2>.B }, <Zm>.Q[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES2) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn::'0');
var index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SVE_AES2)



Encoding

```
AESEMC { <Zdn1>.B-<Zdn4>.B }, { <Zdn1>.B-<Zdn4>.B }, <Zm>.Q[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES2) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn::'00');
var index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<index>	Is the round key index, in the range 0 to 3, encoded in the "i2" field.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_AES) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
if VL == 128 then index = 0; end;
if VL == 256 then index = index MOD 2; end;
let segments : integer = VL DIV 128;
let operand2 : bits(VL) = Z{}(m);
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{}(dn + r);
    for s = 0 to segments-1 do
        let keyindex : integer = (s - (s MOD 4)) + index;
        let res : bits(128) = operand1[s*:128] XOR operand2[keyindex*:128];
        results[[r]][s*:128] = AESMixColumns(AESSubBytes(AESShiftRows(res)));

    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn + r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AESIMC

AES inverse mix columns

This instruction reads a 16-byte state array from each 128-bit segment of the source register, and performs a single round of the InvMixColumns() transformation on each state array in accordance with the AES standard. Each updated state array is destructively placed in the corresponding segment of the first source vector. This instruction is unpredicated.

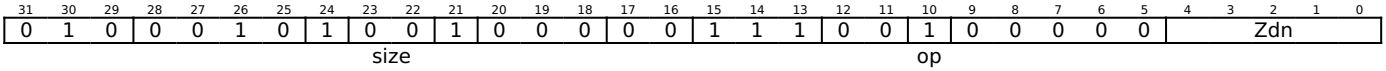
ID_AA64ZFR0_EL1.AES indicates whether this instruction is implemented.

This instruction is legal when executed in Streaming SVE mode if one of the following is true:

- Both FEAT_SSVE_AES and FEAT_SVE_AES are implemented.
- FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE_AES)



Encoding

```
AESIMC <Zdn>.B, <Zdn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES) then EndOfDecode(Decode_UNDEF); end;
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_AES) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand : bits(VL) = Z{}(dn);
var result : bits(VL);

for s = 0 to segments-1 do
    result[s*:128] = AESInvMixColumns(operand[s*:128]);
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AESMC

AES mix columns

This instruction reads a 16-byte state array from each 128-bit segment of the source register, and performs a single round of the MixColumns() transformation on each state array in accordance with the AES standard. Each updated state array is destructively placed in the corresponding segment of the first source vector. This instruction is unpredicated.

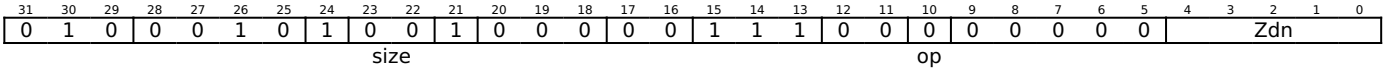
ID_AA64ZFR0_EL1.AES indicates whether this instruction is implemented.

This instruction is legal when executed in Streaming SVE mode if one of the following is true:

- Both FEAT_SSVE_AES and FEAT_SVE_AES are implemented.
- FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE_AES)



Encoding

```
AESMC <Zdn>.B, <Zdn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES) then EndOfDecode(Decode_UNDEF); end;
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_AES) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand : bits(VL) = Z{}(dn);
var result : bits(VL);

for s = 0 to segments-1 do
    result[s*:128] = AESMixColumns(operand[s*:128]);
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

AND (immediate)

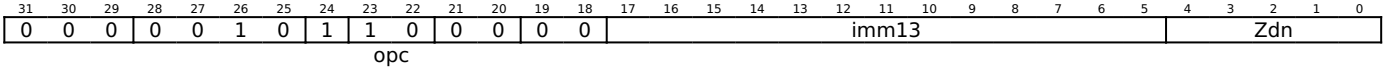
Bitwise AND with immediate (unpredicated)

This instruction performs a bitwise AND between an immediate and each 64-bit element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate is a 64-bit value consisting of a single run of ones or zeros repeating every 2, 4, 8, 16, 32, or 64 bits. This instruction is unpredicated.

This instruction is used by the pseudo-instruction BIC (immediate).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
AND <Zdn>.<T>, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn);
var imm : bits(64);
(imm, -) = DecodeBitMasks{64}(imm13[12], imm13[5:0], imm13[11:6], TRUE);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “imm13”:

imm13	<T>
0xxxxxx0xxxxx	S
0xxxxxx10xxxx	H
0xxxxxx110xxx	B
0xxxxxx1110xx	B
0xxxxxx11110x	B
0xxxxxx11111x	RESERVED
1xxxxxxxxxxxx	D

<const> Is a 64, 32, 16, or 8-bit bitmask consisting of replicated 2, 4, 8, 16, 32, or 64-bit fields, each field containing a rotated run of nonzero bits, encoded in the "imm13" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 64;
let operand : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(64) = operand[e*:64];
    result[e*:64] = element1 AND imm;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

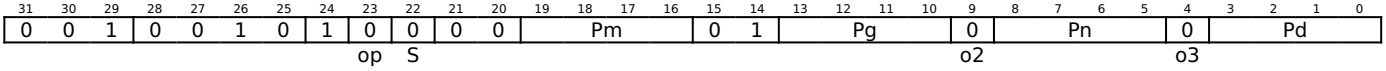
AND (predicates)

Bitwise AND predicates

This instruction performs a bitwise AND between the Active elements of the first source predicate and the corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.
This instruction is used by the alias MOV (predicate, zeroing).

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
AND <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Alias Conditions

Alias	Is preferred when
MOV (predicate, zeroing)	`S == '0' && Pn == Pm`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1 AND element2);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

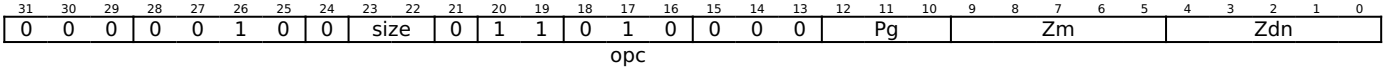
AND (vectors, predicated)

Bitwise AND (predicated)

This instruction performs a bitwise AND between the Active elements of the first source vector and the corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
AND <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = element1 AND element2;
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

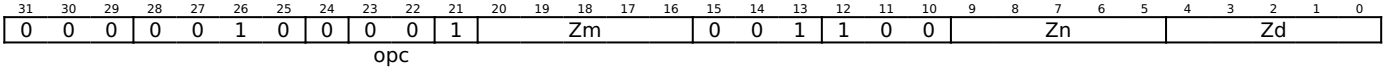
AND (vectors, unpredicated)

Bitwise AND (unpredicated)

This instruction performs a bitwise AND between the elements of the first source vector and the corresponding elements of the second source vector, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
AND <Zd>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);

Z{VL}(d) = operand1 AND operand2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

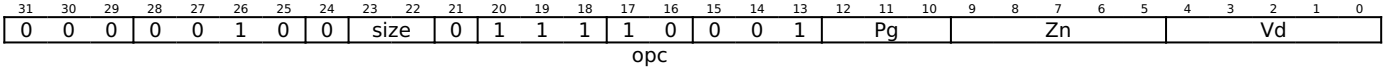
ANDQV

Bitwise AND reduction of quadword vector segments

This instruction performs a bitwise AND between elements in the same position from each 128-bit source vector segment, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as all ones.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ANDQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	16B
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(128) = Zeros{};
var stmp : bits(128) = Zeros{};

var dtmp : bits(esize);

for e = 0 to elempersegment-1 do
    dtmp = Ones{esize};
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp = operand[s*:128];
            dtmp = dtmp AND stmp[e*:esize];
        end;
    end;
    result[e*:esize] = dtmp[esize-1:0];
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ANDS

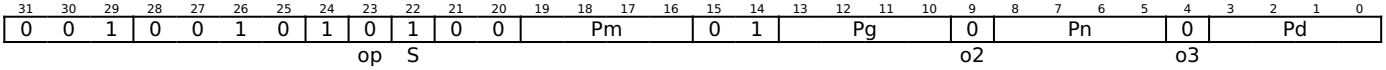
Bitwise AND predicates, setting the condition flags

This instruction performs a bitwise AND between the Active elements of the first source predicate and the corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This instruction is used by the alias MOVS (predicated).

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
ANDS <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Alias Conditions

Alias	Is preferred when
MOVS (predicated)	`S == '1' && Pn == Pm`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1 AND element2);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

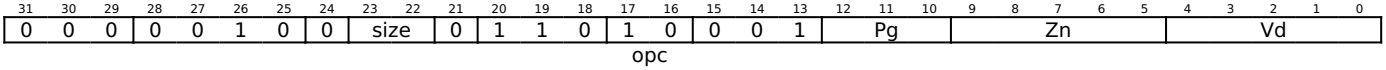
ANDV

Bitwise AND reduction to scalar

This instruction performs a bitwise AND between the elements of the source vector, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as all ones.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ANDV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(esize) = Ones{esize};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result = result AND operand[e*:esize];

    end;
end;
V{esize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

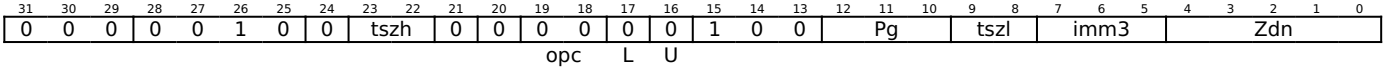
ASR (immediate, predicated)

Arithmetic shift right by immediate (predicated)

This instruction shifts right by immediate, preserving the sign bit, each Active element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ASR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let PL : integer{} = VL DIV 8;
let operand1 : bits(VL) = Z{}(dn);
let mask : bits(PL) = P{}(g);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = ASR(element1, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

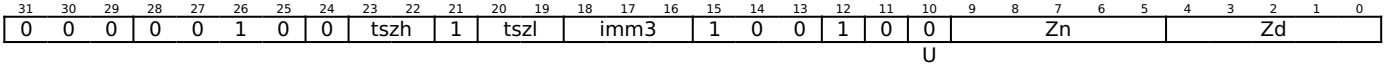
ASR (immediate, unpredicated)

Arithmetic shift right by immediate (unpredicated)

This instruction shifts right by immediate, preserving the sign bit, each element of the source vector, and places the results in the corresponding elements of the destination vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ASR <Zd>.<T>, <Zn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    result[e*:esize] = ASR(element1, shift);

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

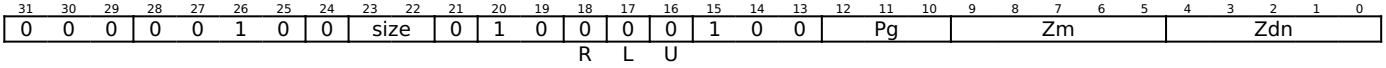
ASR (vectors)

Arithmetic shift right by vector (predicated)

This instruction shifts right, preserving the sign bit, Active elements of the first source vector by the corresponding elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. The shift amount operand is a vector of unsigned elements in which all bits are significant, and not used modulo the element size. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ASR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        let shift : integer = Min(UInt(element2), esize);
        result[e*:esize] = ASR(element1, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

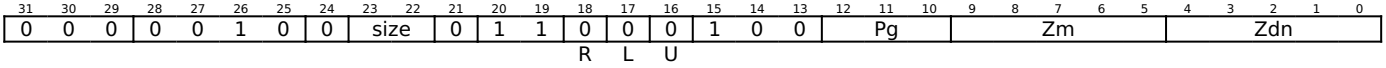
ASR (wide elements, predicated)

Arithmetic shift right by 64-bit wide elements (predicated)

This instruction shifts right, preserving the sign bit, Active elements of the first source vector by the corresponding overlapping 64-bit elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. The shift amount is a vector of unsigned 64-bit doubleword elements in which all bits are significant, and not used modulo the destination element size. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ASR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(64) = operand2[((e * esize) DIVRM 64)*:64];
        let shift : integer = Min(UInt(element2), esize);
        result[e*:esize] = ASR(element1, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

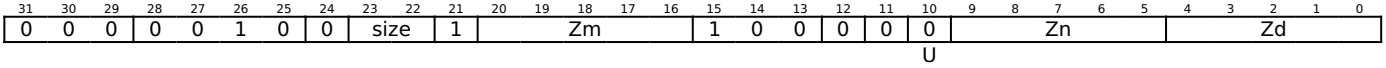
ASR (wide elements, unpredicated)

Arithmetic shift right by 64-bit wide elements (unpredicated)

This instruction shifts right, preserving the sign bit, all elements of the first source vector by the corresponding overlapping 64-bit elements of the second source vector and places the first in the corresponding elements of the destination vector. The shift amount is a vector of unsigned 64-bit doubleword elements in which all bits are significant, and not used modulo the destination element size. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ASR <Zd>.<T>, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(64) = operand2[((e * esize) DIVRM 64)*:64];
    let shift : integer = Min(UInt(element2), esize);
    result[e*:esize] = ASR(element1, shift);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

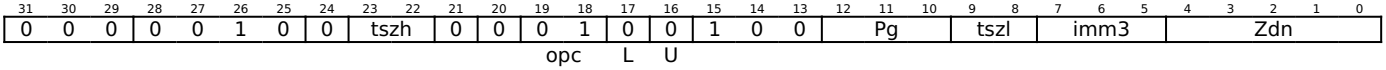
ASRD

Arithmetic shift right for divide by immediate (predicated)

This instruction shifts right by immediate, preserving the sign bit, each Active element of the source vector, and destructively places the results in the corresponding elements of the source vector. The result rounds toward zero as in a signed division. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ASRD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        var element1 : integer = SInt(operand1[e*:esize]);
        if element1 < 0 then
            element1 = element1 + ((1 << shift) - 1);
        end;
        result[e*:esize] = (element1 >> shift)[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.

- A predicated `MOVPRFX` must use the same governing predicate register as this instruction.
- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

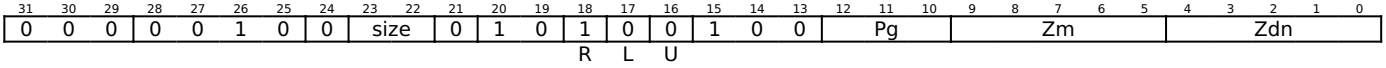
ASRR

Reversed arithmetic shift right by vector (predicated)

This instruction reverse shifts right, preserving the sign bit, Active elements of the second source vector by the corresponding elements of the first source vector, and destructively places the results in the corresponding elements of the first source vector. The shift amount operand is a vector of unsigned elements in which all bits are significant, and not used modulo the element size. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ASRR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        let shift : integer = Min(UInt(element1), esize);
        result[e*:esize] = ASR(element2, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

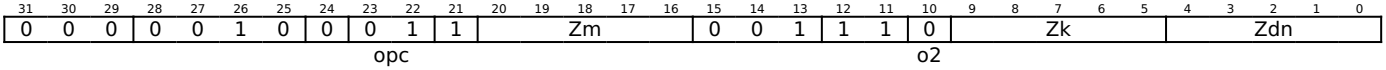
BCAX

Bitwise clear and exclusive-OR

This instruction performs a bitwise AND between elements of the second source vector and the inverted corresponding elements of the third source vector, then an exclusive-OR between the results and the corresponding elements of the first source vector. The final results are destructively placed in the corresponding elements of the destination and first source vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
BCAX <Zdn>.D, <Zdn>.D, <Zm>.D, <Zk>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let m : integer = UInt(Zm);
let k : integer = UInt(Zk);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the third source scalable vector register, encoded in the "Zk" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(k);

Z{VL}(dn) = operand1 XOR (operand2 AND NOT(operand3));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BDEP

Scatter lower bits into positions selected by bitmask

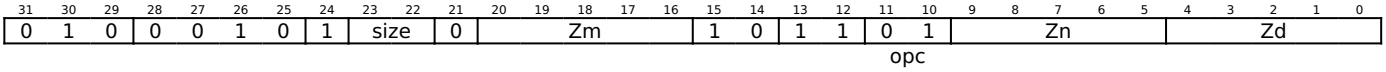
This instruction scatters the lowest-numbered contiguous bits within each element of the first source vector to the bit positions indicated by nonzero bits in the corresponding mask element of the second source vector, preserving their order, and sets the bits corresponding to a zero mask bit to zero. This instruction is unpredicated.

ID_AA64ZFR0_EL1.BitPerm indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled, or FEAT_SSVE_BitPerm is implemented.

SVE2

(FEAT_SVE_BitPerm)



Encoding

```
BDEP <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_BitPerm) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_BitPerm) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let data : bits(VL) = Z{}(n);
let mask : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements - 1 do
    result[e*:esize] = BitDeposit{esize}(data[e*:esize], mask[e*:esize]);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BEXT

Gather lower bits from positions selected by bitmask

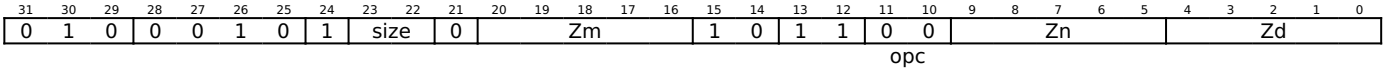
This instruction gathers bits in each element of the first source vector from the bit positions indicated by nonzero bits in the corresponding mask element of the second source vector to the lowest-numbered contiguous bits of the corresponding destination element, preserving their order, and sets the remaining higher-numbered bits to zero. This instruction is unpredicated.

ID_AA64ZFR0_EL1.BitPerm indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled, or FEAT_SSVE_BitPerm is implemented.

SVE2

(FEAT_SVE_BitPerm)



Encoding

```
BEXT <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_BitPerm) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_BitPerm) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let data : bits(VL) = Z{}(n);
let mask : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements - 1 do
    result[e*:esize] = BitExtract{esize}(data[e*:esize], mask[e*:esize]);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BF1CVT, BF2CVT

8-bit floating-point convert to BFloat16

This instruction converts each even-numbered 8-bit floating-point element of the source vector to BFloat16 while downscaling the value, and places the results in the overlapping 16-bit elements of the destination vector. BF1CVT scales the values by $2^{-UInt(FPMR.LSCALE[5:0])}$. BF2CVT scales the values by $2^{-UInt(FPMR.LSCALE2[5:0])}$.

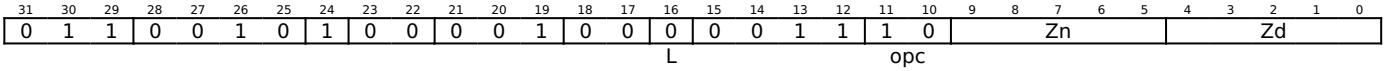
The 8-bit floating-point encoding format for BF1CVT is selected by FPMR.F8S1. The 8-bit floating-point encoding format for BF2CVT is selected by FPMR.F8S2.

This instruction is unpredicated.

It has encodings from 2 classes: BF1CVT and BF2CVT

BF1CVT

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

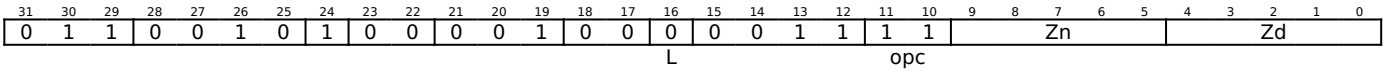
```
BF1CVT <Zd>.H, <Zn>.B
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_FP8)) then EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 8;
let d_esize : integer{} = 16;
let issrc2 : boolean = FALSE;
```

BF2CVT

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

```
BF2CVT <Zd>.H, <Zn>.B
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_FP8)) then EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 8;
let d_esize : integer{} = 16;
let issrc2 : boolean = TRUE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[e*:esize];
    let res : bits(d_esize) = FP8ConvertBF(element[s_esize-1:0], issrc2, FPCR(), FPMR());
    result[e*:esize] = ZeroExtend{esize}(res);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BF1CVTLT, BF2CVTLT

8-bit floating-point convert to BFloat16 (top)

This instruction converts each odd-numbered 8-bit floating-point element of the source vector to BFloat16 while downscaling the value, and places the results in the overlapping 16-bit elements of the destination vector. BF1CVTLT scales the values by $2^{-UInt(FPMR.LSCALE[5:0])}$. BF2CVTLT scales the values by $2^{-UInt(FPMR.LSCALE2[5:0])}$.

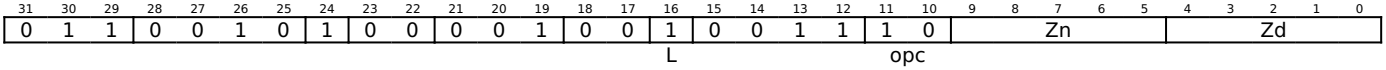
The 8-bit floating-point encoding format for BF1CVTLT is selected by FPMR.F8S1. The 8-bit floating-point encoding format for BF2CVTLT is selected by FPMR.F8S2.

This instruction is unpredicated.

It has encodings from 2 classes: BF1CVTLT and BF2CVTLT

BF1CVTLT

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

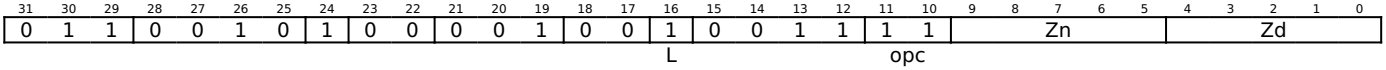
```
BF1CVTLT <Zd>.H, <Zn>.B
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_FP8)) then EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let issrc2 : boolean = FALSE;
```

BF2CVTLT

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

```
BF2CVTLT <Zd>.H, <Zn>.B
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_FP8)) then EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let issrc2 : boolean = TRUE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
for e = 0 to elements-1 do
    let element : bits(esize DIV 2) = operand[(2*e + 1)*:(esize DIV 2)];
    result[e*:esize] = FP8ConvertBF(element, issrc2, FPCR(), FPMR());

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFADD (predicated)

BFloat16 add (predicated)

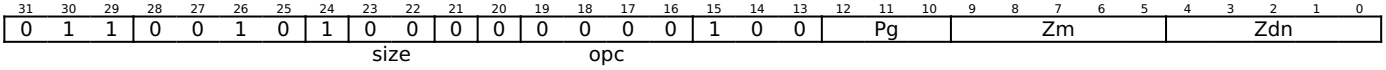
This instruction adds Active BFloat16 elements of the first source vector to the corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFADD <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFAdd{16}(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFADD (unpredicated)

BFloat16 add (unpredicated)

This instruction adds all BFloat16 elements of the first source vector to the corresponding elements of the second source vector and places the results in the corresponding elements of the destination vector.

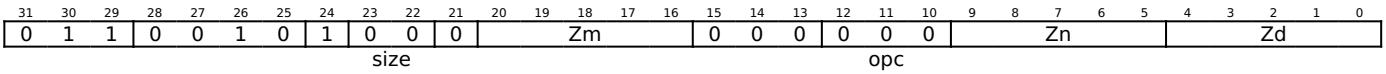
This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFADD <Zd>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    let element2 : bits(16) = operand2[e*:16];
    result[e*:16] = BFAdd{16}(element1, element2, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFCLAMP

BFloat16 clamp to minimum/maximum number

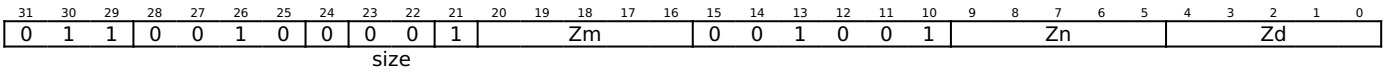
This instruction clamps each BFloat16 element in the destination vector to between the BFloat16 minimum value in the corresponding element of the first source vector and the BFloat16 maximum value in the corresponding element of the second source vector and destructively places the clamped results in the corresponding elements of the destination vector.
The behavior is as follows for each minimum number and maximum number operation:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.
This instruction is unpredicated.
ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFCLAMP <Zd>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var result : bits(VL);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    let element2 : bits(16) = operand2[e*:16];
    let element3 : bits(16) = operand3[e*:16];
    let maxelement : bits(16) = BFMaxNum{}(element1, element3, FPCR());
    result[e*:16] = BFMinNum{16}(maxelement, element2, FPCR());
end;
Z{VL}(d) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFCVT

Single-precision convert to BFloat16 (predicated)

This instruction converts to BFloat16 from single-precision in each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

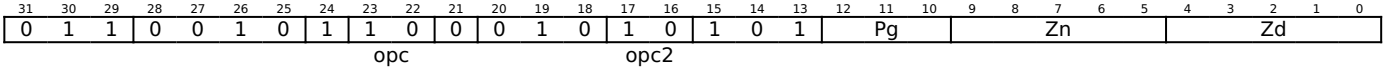
Since the result type is smaller than the input type, the results are zero-extended to fill each destination element.

ID_AA64ZFR0_EL1.BF16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Merging and Zeroing

Merging

((FEAT_SVE || FEAT_SME) && FEAT_BF16)



Encoding

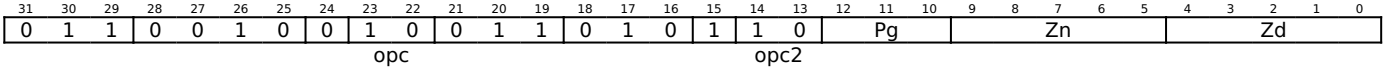
```
BFCVT <Zd>.H, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_BF16)) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
BFCVT <Zd>.H, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 32;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, 32) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, 32) then
        let element : bits(32) = operand[e*:32];
        result[(2*e)*:16] = FPConvertBF(element, FPCR());
        result[(2*e+1)*:16] = Zeros{16};

    end;
end;
Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:
The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFCVTN

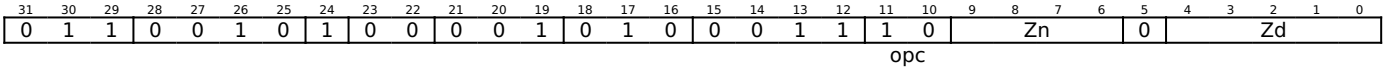
BFloat16 convert to interleaved 8-bit floating-point

This instruction converts each BFloat16 element of the pair of source vectors to 8-bit floating-point while scaling the value by $2^{\text{SInt(FPMR.NSCALE)}}$, and places the two-way interleaved results in the corresponding 8-bit elements of the destination vector. The 8-bit floating-point encoding format is selected by FPMR.F8D.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

```
BFCVTN <Zd>.B, { <Zn1>.H-<Zn2>.H }
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
      !IsFeatureImplemented(FEAT_FP8)) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    let element2 : bits(16) = operand2[e*:16];
    result[(2*e + 0)*:8] = BFConvertFP8(element1, FPCR(), FPMR());
    result[(2*e + 1)*:8] = BFConvertFP8(element2, FPCR(), FPMR());
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFCVTNT

Single-precision convert to BFloat16 (top, predicated)

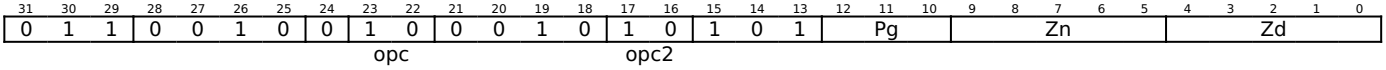
This instruction converts each Active floating-point element of the source vector from single-precision to BFloat16 floating-point, and places the results in the odd-numbered 16-bit elements of the destination vector, leaving the even-numbered elements unchanged. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

ID_AA64ZFR0_EL1.BF16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Merging and Zeroing

Merging

((FEAT_SVE || FEAT_SME) && FEAT_BF16)



Encoding

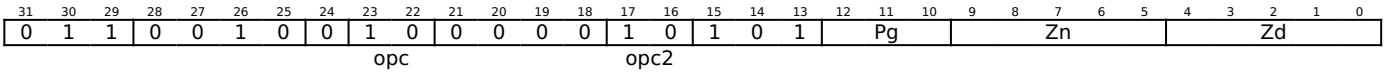
```
BFCVTNT <Zd>.H, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_BF16)) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
BFCVTNT <Zd>.H, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 32;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, 32) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, 32) then
        let element : bits(32) = operand[e*:32];
        result[(2*e+1)*:16] = FPConvertBF(element, FPCR());
    elsif !merging then
        result[(2*e+1)*:16] = Zeros{16};

    end;
end;
Z{VL}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFDOT (indexed)

BFloat16 dot product by indexed element to single-precision

This instruction delimits the source vectors into pairs of BFloat16 elements. The BFloat16 pairs within the second source vector are specified using an immediate index that selects the same BFloat16 pair position within each 128-bit vector segment. The index range is from 0 to 3.

If FEAT_EBF16 is not implemented or FPCR.EBF is 0, this instruction:

- Performs an unfused sum-of-products of each pair of adjacent BFloat16 elements in the first source vector with the specified pair of elements in the second vector. The intermediate single-precision products are rounded before they are summed, and the intermediate sum is rounded before accumulation into the single-precision destination element that overlaps with the corresponding pair of BFloat16 elements in the first source vector.
- Uses the non-IEEE 754 Round-to-Odd rounding mode, which forces bit 0 of an inexact result to 1, and rounds an overflow to an appropriately signed Infinity.
- Flushes denormalized inputs and results to zero, as if FPCR.{FZ, FIZ} is {1, 1}.
- Honors FPCR.AH when generating a default NaN result, otherwise disables alternative floating point behaviors, as if FPCR.AH is 0.

If FEAT_EBF16 is implemented and FPCR.EBF is 1, this instruction:

- Performs a fused sum-of-products of each pair of adjacent BFloat16 elements in the first source vector with the specified pair of elements in the second vector. The intermediate single-precision products are not rounded before they are summed, but the intermediate sum is rounded before accumulation into the single-precision destination element that overlaps with the corresponding pair of BFloat16 elements in the first source vector.
- Follows all other floating-point behaviors that apply to single-precision arithmetic, as governed by FPCR.RMode, FPCR.FZ, FPCR.AH, and FPCR.FIZ.

Irrespective of FEAT_EBF16 and FPCR.EBF, this instruction:

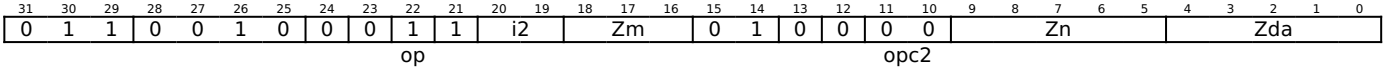
- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.
- Generates only the default NaN, as if FPCR.DN is 1.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.BF16 indicates whether this instruction is implemented.

SVE

((FEAT_SVE || FEAT_SME) && FEAT_BF16)



Encoding

```
BFDOT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_BF16)) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i2);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let elt1_a : bits(16) = operand1[(2 * e + 0)*:16];
    let elt1_b : bits(16) = operand1[(2 * e + 1)*:16];
    let elt2_a : bits(16) = operand2[(2 * s + 0)*:16];
    let elt2_b : bits(16) = operand2[(2 * s + 1)*:16];
    var sum : bits(32) = operand3[e*:32];

    sum = BFDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
    result[e*:32] = sum;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFDOT (vectors)

BFloat16 dot product to single-precision

This instruction delimits the source vectors into pairs of BFloat16 elements.
If FEAT_EBF16 is not implemented or FPCR.EBF is 0, this instruction:

- Performs an unfused sum-of-products of each pair of adjacent BFloat16 elements in the source vectors. The intermediate single-precision products are rounded before they are summed, and the intermediate sum is rounded before accumulation into the single-precision destination element that overlaps with the corresponding pair of BFloat16 elements in the source vectors.
- Uses the non-IEEE 754 Round-to-Odd rounding mode, which forces bit 0 of an inexact result to 1, and rounds an overflow to an appropriately signed Infinity.
- Flushes denormalized inputs and results to zero, as if FPCR.{FZ, FIZ} is {1, 1}.
- Honors FPCR.AH when generating a default NaN result, otherwise disables alternative floating point behaviors, as if FPCR.AH is 0.

If FEAT_EBF16 is implemented and FPCR.EBF is 1, this instruction:

- Performs a fused sum-of-products of each pair of adjacent BFloat16 elements in the source vectors. The intermediate single-precision products are not rounded before they are summed, but the intermediate sum is rounded before accumulation into the single-precision destination element that overlaps with the corresponding pair of BFloat16 elements in the source vectors.
- Follows all other floating-point behaviors that apply to single-precision arithmetic, as governed by FPCR.RMode, FPCR.FZ, FPCR.AH, and FPCR.FIZ.

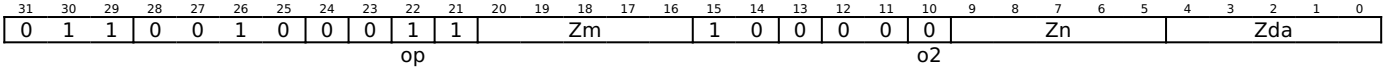
Irrespective of FEAT_EBF16 and FPCR.EBF, this instruction:

- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.
- Generates only the default NaN, as if FPCR.DN is 1.

This instruction is unpredicated.
ID_AA64ZFR0_EL1.BF16 indicates whether this instruction is implemented.

SVE

((FEAT_SVE || FEAT_SME) && FEAT_BF16)



Encoding

```
BFDOT <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_BF16)) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let elt1_a : bits(16) = operand1[(2 * e + 0)*:16];
    let elt1_b : bits(16) = operand1[(2 * e + 1)*:16];
    let elt2_a : bits(16) = operand2[(2 * e + 0)*:16];
    let elt2_b : bits(16) = operand2[(2 * e + 1)*:16];
    var sum : bits(32) = operand3[e*:32];

    sum = BFDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
    result[e*:32] = sum;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMAX

BFloat16 maximum (predicated)

This instruction determines the maximum of Active BFloat16 elements of the first source vector and the corresponding BFloat16 elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

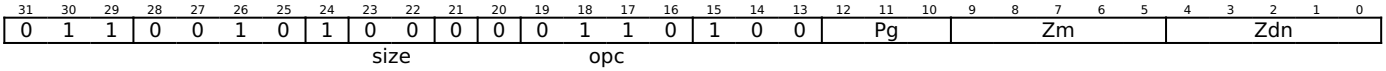
Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMAX <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFMax{16}(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMAXNM

BFloat16 maximum number (predicated)

This instruction determines the maximum number value of Active BFloat16 elements of the first source vector and the corresponding BFloat16 elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

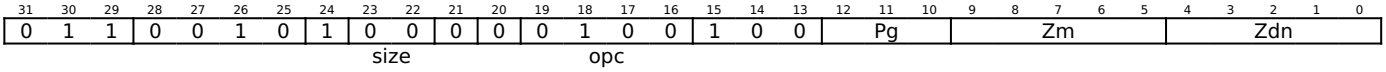
Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMAXNM <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFMaxNum{16}(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMIN

BFloat16 minimum (predicated)

This instruction determines the minimum of Active BFloat16 elements of the first source vector and the corresponding BFloat16 elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

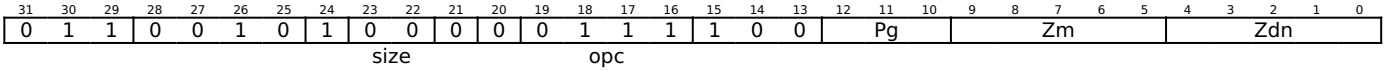
Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMIN <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFMin{16}(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMINNM

BFloat16 minimum number (predicated)

This instruction determines the minimum number value of Active BFloat16 elements of the first source vector and the corresponding BFloat16 elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

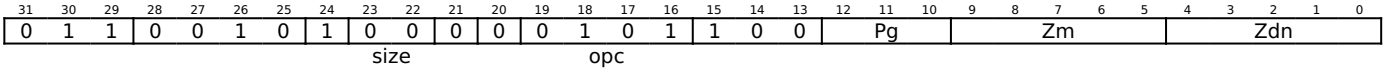
Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMINNM <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFMinNum{16}(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLA (indexed)

BFloat16 fused multiply-add by indexed element

This instruction multiplies all BFloat16 elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment. The products are then destructively added without intermediate rounding to the corresponding elements of the addend and destination vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to 7.

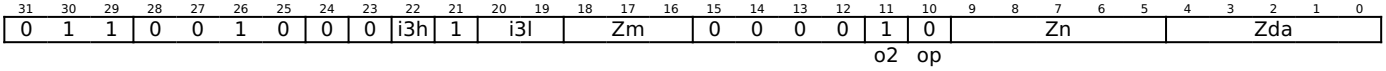
This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMLA <Zda>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
let op3_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let eltspersegment : integer = 128 DIV 16;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let elem2 : bits(16) = op2[s*:16];
    let elem1 : bits(16) = if op1_neg then BFNeg(op1[e*:16]) else op1[e*:16];
    let elem3 : bits(16) = if op3_neg then BFNeg(result[e*:16]) else result[e*:16];
    result[e*:16] = BFMulAdd(elem3, elem1, elem2, FPCR());

end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLA (vectors)

BFloat16 fused multiply-add

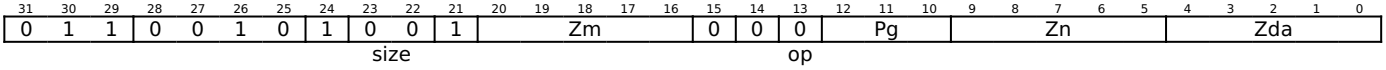
This instruction multiplies the Active BFloat16 elements of the first source vector by the corresponding BFloat16 elements of the second source vector. The results are then added to elements of the third source (addend) vector without intermediate rounding and destructively placed in the destination and third source (addend) vector. Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMLA <Zda>.H, <Pg>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
let op3_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(n) else Zeros{VL};
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, 16) then
        let elem1 : bits(16) = if op1_neg then BFNeg(op1[e*:16]) else op1[e*:16];
        let elem2 : bits(16) = op2[e*:16];
        let elem3 : bits(16) = if op3_neg then BFNeg(op3[e*:16]) else op3[e*:16];

        result[e*:16] = BFMulAdd(elem3, elem1, elem2, FPCR());
    else
        result[e*:16] = op3[e*:16];

    end;
end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLALB (indexed)

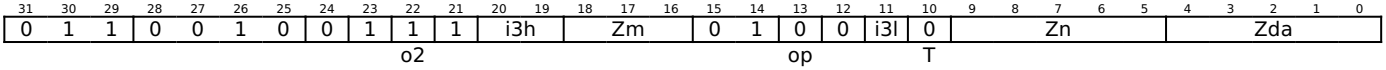
BFloat16 multiply-add by indexed element to single-precision (bottom)

This instruction widens the even-numbered BFloat16 elements in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and then destructively multiplies and adds these values without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding BFloat16 elements in the first source vector. This instruction is unpredicated.

ID_AA64ZFR0_EL1.BF16 indicates whether this instruction is implemented.

SVE

((FEAT_SVE || FEAT_SME) && FEAT_BF16)



Encoding

```
BFMLALB <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_BF16)) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i3h::i3l);
let op1_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 2 * segmentbase + index;
    let elem1 : bits(16) = (if op1_neg then BFNeg(op1[(2*e + 0)*:16])
                            else op1[(2*e + 0)*:16]);
    let elem2 : bits(16) = op2[s*:16];
    let elem3 : bits(32) = op3[e*:32];
    result[e*:32] = BFMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLALB (vectors)

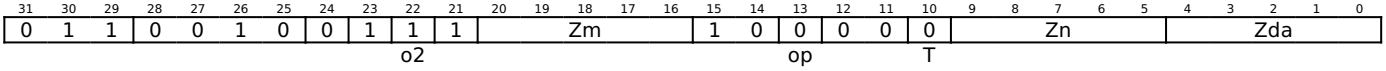
BFloat16 multiply-add to single-precision (bottom)

This instruction widens the even-numbered BFloat16 elements in the first source vector and the corresponding elements in the second source vector to single-precision format and then destructively multiplies and adds these values without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding BFloat16 elements in the source vectors. This instruction is unpredicated.

ID_AA64ZFR0_EL1.BF16 indicates whether this instruction is implemented.

SVE

((FEAT_SVE || FEAT_SME) && FEAT_BF16)



Encoding

```
BFMLALB <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_BF16)) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let elem1 : bits(16) = (if op1_neg then BFNeg(op1[(2*e + 0)*:16])
                           else op1[(2*e + 0)*:16]);
    let elem2 : bits(16) = op2[(2*e + 0)*:16];
    let elem3 : bits(32) = op3[e*:32];
    result[e*:32] = BFMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLALT (indexed)

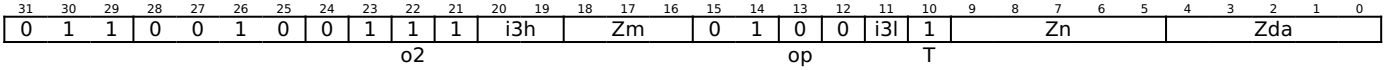
BFloat16 multiply-add by indexed element to single-precision (top)

This instruction widens the odd-numbered BFloat16 elements in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and then destructively multiplies and adds these values without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding BFloat16 elements in the first source vector. This instruction is unpredicated.

ID_AA64ZFR0_EL1.BF16 indicates whether this instruction is implemented.

SVE

((FEAT_SVE || FEAT_SME) && FEAT_BF16)



Encoding

```
BFMLALT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_BF16)) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i3h::i3l);
let op1_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 2 * segmentbase + index;
    let elem1 : bits(16) = (if op1_neg then BFNeg(op1[(2*e + 1)*:16])
                            else op1[(2*e + 1)*:16]);
    let elem2 : bits(16) = op2[s*:16];
    let elem3 : bits(32) = op3[e*:32];
    result[e*:32] = BFMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLALT (vectors)

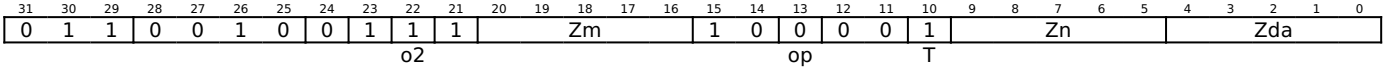
BFloat16 multiply-add to single-precision (top)

This instruction widens the odd-numbered BFloat16 elements in the first source vector and the corresponding elements in the second source vector to single-precision format and then destructively multiplies and adds these values without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding BFloat16 elements in the source vectors. This instruction is unpredicated.

ID_AA64ZFR0_EL1.BF16 indicates whether this instruction is implemented.

SVE

((FEAT_SVE || FEAT_SME) && FEAT_BF16)



Encoding

```
BFMLALT <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_BF16)) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let elem1 : bits(16) = (if op1_neg then BFNeg(op1[(2*e + 1)*:16])
                           else op1[(2*e + 1)*:16]);
    let elem2 : bits(16) = op2[(2*e + 1)*:16];
    let elem3 : bits(32) = op3[e*:32];
    result[e*:32] = BFMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLS (indexed)

BFloat16 fused multiply-subtract by indexed element

This instruction multiplies all BFloat16 elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment. The products are then destructively subtracted without intermediate rounding from the corresponding elements of the addend and destination vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to 7.

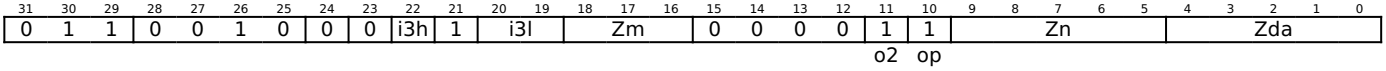
This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMLS <Zda>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
let op3_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let eltspersegment : integer = 128 DIV 16;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let elem2 : bits(16) = op2[s*:16];
    let elem1 : bits(16) = if op1_neg then BFNeg(op1[e*:16]) else op1[e*:16];
    let elem3 : bits(16) = if op3_neg then BFNeg(result[e*:16]) else result[e*:16];
    result[e*:16] = BFMulAdd(elem3, elem1, elem2, FPCR());

end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLS (vectors)

BFloat16 fused multiply-subtract

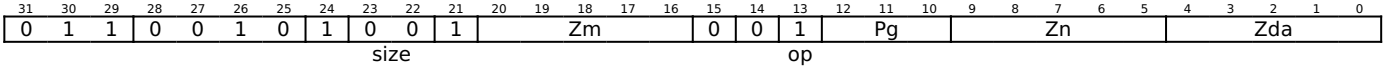
This instruction multiplies the Active BFloat16 elements of the first source vector by the corresponding BFloat16 elements of the second source vector. The results are then subtracted from elements of the third source (addend) vector without intermediate rounding and destructively placed in the destination and third source (addend) vector. Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMLS <Zda>.H, <Pg>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
let op3_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(n) else Zeros{VL};
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, 16) then
        let elem1 : bits(16) = if op1_neg then BFNeg(op1[e*:16]) else op1[e*:16];
        let elem2 : bits(16) = op2[e*:16];
        let elem3 : bits(16) = if op3_neg then BFNeg(op3[e*:16]) else op3[e*:16];

        result[e*:16] = BFMulAdd(elem3, elem1, elem2, FPCR());
    else
        result[e*:16] = op3[e*:16];

    end;
end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

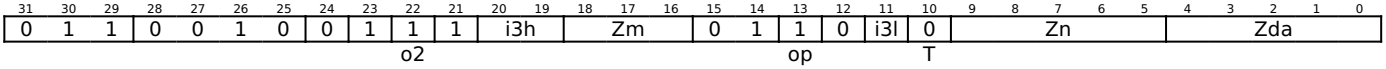
BFMLSLB (indexed)

BFloat16 multiply-subtract by indexed element from single-precision (bottom)

This instruction widens the even-numbered BFloat16 elements in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and then destructively multiplies and subtracts these values without intermediate rounding from the single-precision elements of the destination vector that overlap with the corresponding BFloat16 elements in the first source vector. This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
BFMLSLB <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i3h::i3l);
let op1_neg : boolean = TRUE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 2 * segmentbase + index;
    let elem1 : bits(16) = (if op1_neg then BFNeg(op1[(2*e + 0)*:16])
                            else op1[(2*e + 0)*:16]);
    let elem2 : bits(16) = op2[s*:16];
    let elem3 : bits(32) = op3[e*:32];
    result[e*:32] = BFMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

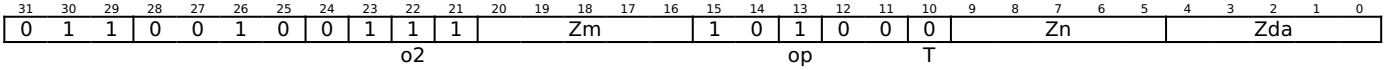
BFMLSLB (vectors)

BFloat16 multiply-subtract from single-precision (bottom)

This instruction widens the even-numbered BFloat16 elements in the first source vector and the corresponding elements in the second source vector to single-precision format and then destructively multiplies and subtracts these values without intermediate rounding from the single-precision elements of the destination vector that overlap with the corresponding BFloat16 elements in the source vectors. This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
BFMLSLB <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let elem1 : bits(16) = (if op1_neg then BFNeg(op1[(2*e + 0)*:16])
                            else op1[(2*e + 0)*:16]);
    let elem2 : bits(16) = op2[(2*e + 0)*:16];
    let elem3 : bits(32) = op3[e*:32];
    result[e*:32] = BFMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

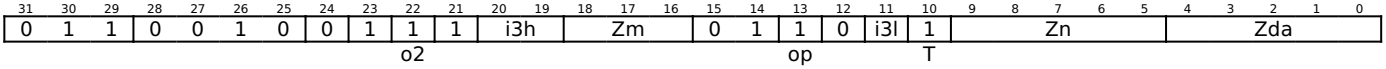
BFMLSLT (indexed)

BFloat16 multiply-subtract by indexed element from single-precision (top)

This instruction widens the odd-numbered BFloat16 elements in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and then destructively multiplies and subtracts these values without intermediate rounding from the single-precision elements of the destination vector that overlap with the corresponding BFloat16 elements in the first source vector. This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
BFMLSLT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i3h::i3l);
let op1_neg : boolean = TRUE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 2 * segmentbase + index;
    let elem1 : bits(16) = (if op1_neg then BFNeg(op1[(2*e + 1)*:16])
                            else op1[(2*e + 1)*:16]);
    let elem2 : bits(16) = op2[s*:16];
    let elem3 : bits(32) = op3[e*:32];
    result[e*:32] = BFMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

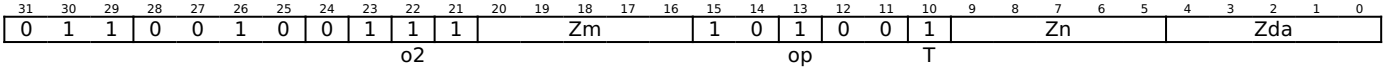
BFMLSLT (vectors)

BFloat16 multiply-subtract from single-precision (top)

This instruction widens the odd-numbered BFloat16 elements in the first source vector and the corresponding elements in the second source vector to single-precision format and then destructively multiplies and subtracts these values without intermediate rounding from the single-precision elements of the destination vector that overlap with the corresponding BFloat16 elements in the source vectors. This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
BFMLSLT <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let elem1 : bits(16) = (if op1_neg then BFNeg(op1[(2*e + 1)*:16])
                            else op1[(2*e + 1)*:16]);
    let elem2 : bits(16) = op2[(2*e + 1)*:16];
    let elem3 : bits(32) = op3[e*:32];
    result[e*:32] = BFMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMMLA (non-widening)

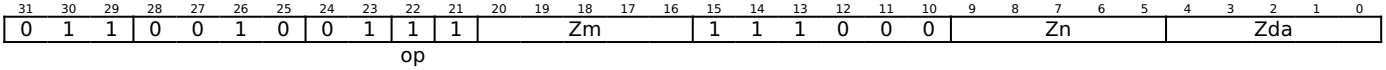
BFloat16 matrix multiply-accumulate

This instruction multiplies the 2x2 matrix of BFloat16 values in each 64-bit segment of the first source vector by the 2x2 matrix of BFloat16 values in the corresponding 64-bit segment of the second source vector. The intermediate products are rounded before they are summed, and the intermediate sum is rounded before accumulation into the 2x2 BFloat16 matrix held in the corresponding 64-bit segment of the addend and destination vector. This is equivalent to performing a 2-way dot product per destination element. This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE_B16MM)



Encoding

```
BFMMLA <Zda>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16MM) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer =  VL DIV 64;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);
var op1, op2 : bits(64);
var res, addend : bits(64);

for s = 0 to segments-1 do
    op1    = operand1[s*:(64)];
    op2    = operand2[s*:(64)];
    addend = operand3[s*:(64)];
    res    = BFMatMulAdd(addend, op1, op2,  FPCR());
    result[s*:(64)] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMMLA (widening)

BFloat16 matrix multiply-accumulate to single-precision

This instruction multiplies the 2×4 matrix of BFloat16 values held in each 128-bit segment of the first source vector by the 4×2 matrix of BFloat16 values in the corresponding segment of the second source vector.

If FEAT_EBF16 is not implemented or FPCR.EBF is 0, this instruction:

- Calculates the intermediate single-precision results as a sum of two unfused sum-of-products using two pairs of adjacent BFloat16 values from each of the source vectors, and rounds the results before accumulation into the 2×2 matrix of single-precision results in the corresponding segment of the destination vector. This is equivalent to accumulating a 4-way dot product per destination element.
- Uses the non-IEEE 754 Round-to-Odd rounding mode, which forces bit 0 of an inexact result to 1, and rounds an overflow to an appropriately signed Infinity.
- Flushes denormalized inputs and results to zero, as if FPCR.{FZ, FIZ} is {1, 1}.
- Honors FPCR.AH when generating a default NaN result, otherwise disables alternative floating point behaviors, as if FPCR.AH is 0.

If FEAT_EBF16 is implemented and FPCR.EBF is 1, this instruction:

- Calculates the intermediate single-precision results as a sum of two fused sum-of-products using two pairs of adjacent BFloat16 values from each of the source vectors, and rounds the results before accumulation into the 2×2 matrix of single-precision results in the destination vector. This is equivalent to accumulating a 4-way dot product per destination element.
- Follows all other floating-point behaviors that apply to single-precision arithmetic, as governed by FPCR.RMode, FPCR.FZ, FPCR.AH, and FPCR.FIZ.

Irrespective of FEAT_EBF16 and FPCR.EBF, this instruction:

- Does not modify the cumulative FPSR exception bits (IDC, IXC, UFC, OFC, DZC, and IOC).
- Disables trapped floating-point exceptions, as if the FPCR trap enable bits (IDE, IXE, UFE, OFE, DZE, and IOE) are all zero.
- Generates only the default NaN, as if FPCR.DN is 1.

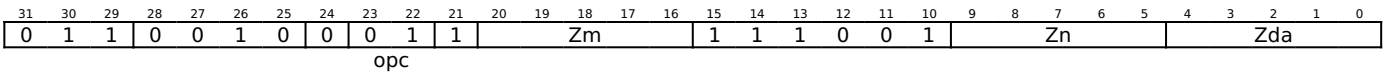
This instruction is unpredicated and vector length agnostic.

ID_AA64ZFR0_EL1.BF16 indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE && FEAT_BF16)



Encoding

```
BFMMLA <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_BF16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer =  VL DIV 128;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);
var op1, op2 : bits(128);
var res, addend : bits(128);

for s = 0 to segments-1 do
    op1    = operand1[s*:(128)];
    op2    = operand2[s*:(128)];
    addend = operand3[s*:(128)];
    res    = BFMatMulAddH(addend, op1, op2,  FPCR());
    result[s*:(128)] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMUL (indexed)

BFloat16 multiply by indexed element

This instruction multiplies all BFloat16 elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment and places the results in the corresponding elements of the destination vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to 7.

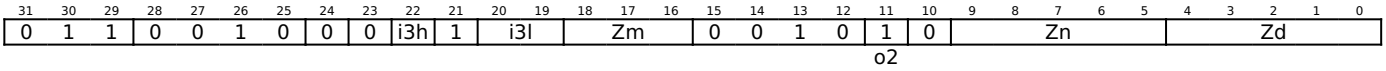
This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMUL <Zd>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let eltspersegment : integer = 128 DIV 16;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let elem2 : bits(16) = op2[s*:16];
    let elem1 : bits(16) = op1[e*:16];
    result[e*:16] = BFMul(elem1, elem2, FPCR());

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMUL (vectors, predicated)

BFloat16 multiply (predicated)

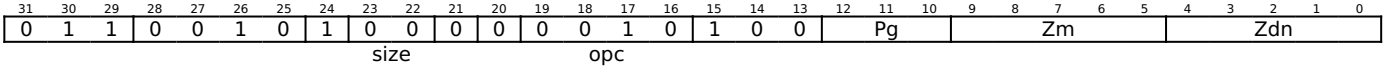
This instruction multiplies Active BFloat16 elements of the first source vector by the corresponding BFloat16 elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMUL <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFMul(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMUL (vectors, unpredicated)

BFloat16 multiply (unpredicated)

This instruction multiplies all BFloat16 elements of the first source vector by the corresponding BFloat16 elements of the second source vector, and places the results in the corresponding elements of the destination vector.

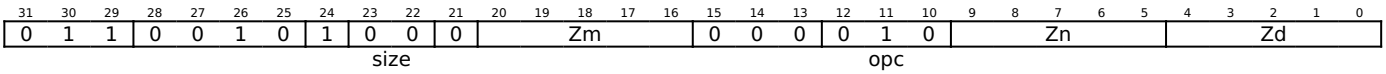
This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFMUL <Zd>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    let element2 : bits(16) = operand2[e*:16];
    result[e*:16] = BFMul(element1, element2, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

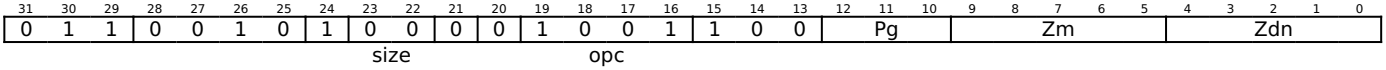
BFSCALE

BFloat16 adjust exponent (predicated)

This instruction multiplies the Active BFloat16 elements of the first source vector by 2.0 to the power of the signed integer values in the corresponding elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified. This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

SVE2

(FEAT_SVE_BFSCALE)



Encoding

```
BFSCALE <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_BFSCALE) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : integer = SInt(operand2[e*:16]);
        result[e*:16] = BFScale(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFSUB (predicated)

BFloat16 subtract (predicated)

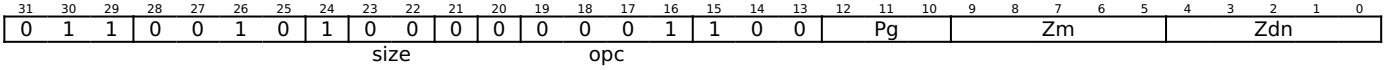
This instruction subtracts Active BFloat16 elements of the second source vector from corresponding BFloat16 elements of the first source vector and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFSUB <Zdn>.H, <Pg>/M, <Zdn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 16;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 16) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    if ActivePredicateElement{PL}(mask, e, 16) then
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFSub(element1, element2, FPCR());
    else
        result[e*:16] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFSUB (unpredicated)

BFloat16 subtract (unpredicated)

This instruction subtracts all BFloat16 elements of the second source vector from corresponding BFloat16 elements of the first source vector and places the results in the corresponding elements of the destination vector.

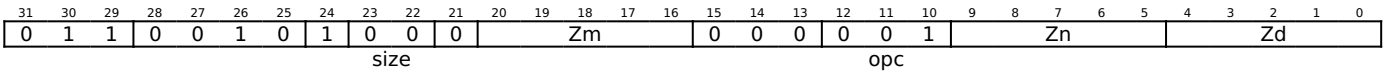
This instruction follows SVE2 non-widening BFloat16 numerical behaviors.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

SVE2

(FEAT_SVE_B16B16)



Encoding

```
BFSUB <Zd>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_B16B16) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    let element2 : bits(16) = operand2[e*:16];
    result[e*:16] = BFSub(element1, element2, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BGRP

Group bits to right or left as selected by bitmask

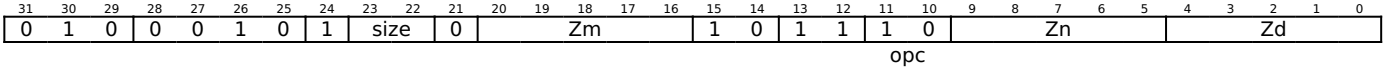
This instruction separates bits in each element of the first source vector by gathering from the bit positions indicated by nonzero bits in the corresponding mask element of the second source vector to the lowest-numbered contiguous bits of the corresponding destination element, and from positions indicated by zero bits to the highest-numbered bits of the destination element, preserving the bit order within each group. This instruction is unpredicated.

ID_AA64ZFR0_EL1.BitPerm indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled, or FEAT_SSVE_BitPerm is implemented.

SVE2

(FEAT_SVE_BitPerm)



Encoding

```
BGRP <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_BitPerm) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_BitPerm) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let data : bits(VL) = Z{}(n);
let mask : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements - 1 do
    result[e*:esize] = BitGroup{esize}(data[e*:esize], mask[e*:esize]);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BIC (immediate)

Bitwise clear bits using immediate (unpredicated)

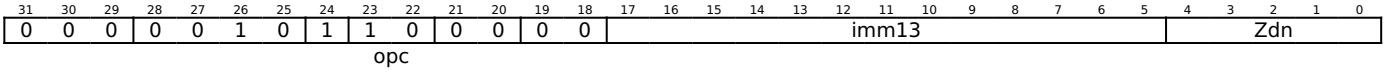
This instruction performs a bitwise clear bits using an immediate with each 64-bit element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate is a 64-bit value consisting of a single run of ones or zeros repeating every 2, 4, 8, 16, 32, or 64 bits. This instruction is unpredicated.

This is a pseudo-instruction of AND (immediate). This means:

- The encodings in this description are named to match the encodings of AND (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of AND (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
BIC <Zdn>.<T>, <Zdn>.<T>, #<const>
```

is equivalent to

```
AND <Zdn>.<T>, <Zdn>.<T>, #(-<const> - 1)
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “imm13”:

imm13	<T>
0xxxxxx0xxxxx	S
0xxxxxx10xxxx	H
0xxxxxx110xxx	B
0xxxxxx1110xx	B
0xxxxxx11110x	B
0xxxxxx11111x	RESERVED
1xxxxxxxxxxxx	D

<const> Is a 64, 32, 16, or 8-bit bitmask consisting of replicated 2, 4, 8, 16, 32, or 64-bit fields, each field containing a rotated run of nonzero bits, encoded in the "imm13" field.

Operation

The description of AND (immediate) gives the operational pseudocode for this instruction.

Operational Information

The description of AND (immediate) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

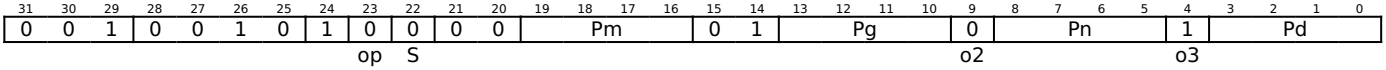
BIC (predicates)

Bitwise clear predicates

This instruction performs a bitwise AND between the Active elements of the first source predicate and the inverted corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BIC <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1 AND (NOT element2));
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

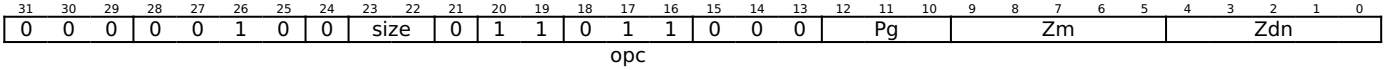
BIC (vectors, predicated)

Bitwise clear (predicated)

This instruction performs a bitwise AND between the Active elements of the first source vector and the inverted corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
BIC <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = element1 AND (NOT element2);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

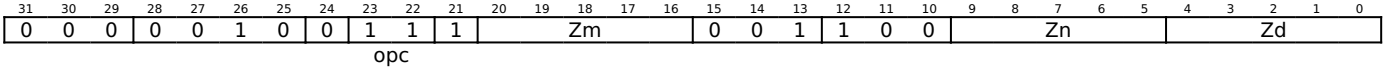
BIC (vectors, unpredicated)

Bitwise clear (unpredicated)

This instruction performs a bitwise AND between the elements of the first source vector and the inverted corresponding elements of the second source vector, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
BIC <Zd>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);

Z{VL}(d) = operand1 AND (NOT operand2);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

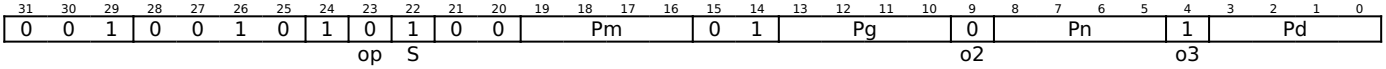
BICS

Bitwise clear predicates, setting the condition flags

This instruction performs a bitwise AND between the Active elements of the first source predicate and the inverted corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BICS <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1 AND (NOT element2));
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BRKA

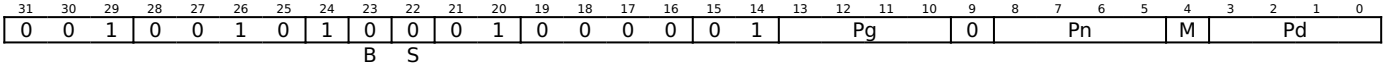
Break after first true condition

This instruction sets the destination predicate elements up to and including the First active and true source element to true, then sets subsequent elements to false.

Inactive elements in the destination predicate register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected. This instruction does not set the condition flags.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BRKA <Pd>.B, <Pg>/<ZM>, <Pn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let d : integer = UInt(Pd);
let merging : boolean = (M == '1');
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<ZM> Is the predication qualifier, encoded in “M”:

M	<ZM>
0	Z
1	M

<Pn> Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(PL) = P{}(n);
let operand2 : bits(PL) = if merging then P{PL}(d) else Zeros{PL};
var break : boolean = FALSE;
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element : boolean = ActivePredicateElement{PL}(operand, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let pbit : bit = if !break then '1' else '0';
        result[e*:psize] = ZeroExtend{psize}(pbit);
        break = break || element;
    elsif merging then
        let pbit : bit = PredicateElement{PL}(operand2, e, esize);
        result[e*:psize] = ZeroExtend{psize}(pbit);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

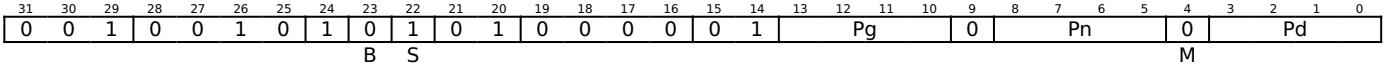
BRKAS

Break after first true condition, setting the condition flags

This instruction sets the destination predicate elements up to and including the First active and true source element to true, then sets subsequent elements to false. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BRKAS <Pd>.B, <Pg>/Z, <Pn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let d : integer = UInt(Pd);
let merging : boolean = FALSE;
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(PL) = P{}(n);
let operand2 : bits(PL) = if merging then P{PL}(d) else Zeros{PL};
var break : boolean = FALSE;
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element : boolean = ActivePredicateElement{PL}(operand, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let pbit : bit = if !break then '1' else '0';
        result[e*:psize] = ZeroExtend{psize}(pbit);
        break = break || element;
    elsif merging then
        let pbit : bit = PredicateElement{PL}(operand2, e, esize);
        result[e*:psize] = ZeroExtend{psize}(pbit);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

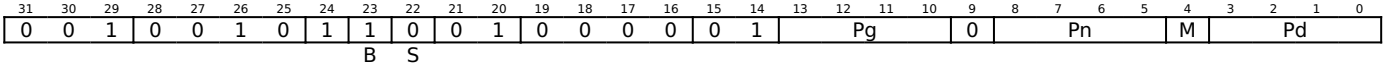
BRKB

Break before first true condition

This instruction sets the destination predicate elements up to but not including the First active and true source element to true, then sets subsequent elements to false.
Inactive elements in the destination predicate register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected. This instruction does not set the condition flags.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BRKB <Pd>.B, <Pg>/<ZM>, <Pn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let d : integer = UInt(Pd);
let merging : boolean = (M == '1');
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<ZM> Is the predication qualifier, encoded in “M”:

M	<ZM>
0	Z
1	M

<Pn> Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(PL) = P{}(n);
let operand2 : bits(PL) = if merging then P{PL}(d) else Zeros{PL};
var break : boolean = FALSE;
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element : boolean = ActivePredicateElement{PL}(operand, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        break = break || element;
        let pbit : bit = if !break then '1' else '0';
        result[e*:psize] = ZeroExtend{psize}(pbit);
    elsif merging then
        let pbit : bit = PredicateElement{PL}(operand2, e, esize);
        result[e*:psize] = ZeroExtend{psize}(pbit);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

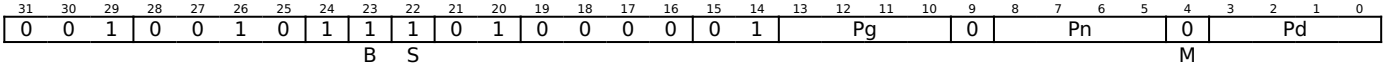
BRKBS

Break before first true condition, setting the condition flags

This instruction sets the destination predicate elements up to but not including the First active and true source element to true, then sets subsequent elements to false. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BRKBS <Pd>.B, <Pg>/Z, <Pn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let d : integer = UInt(Pd);
let merging : boolean = FALSE;
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(PL) = P{}(n);
let operand2 : bits(PL) = if merging then P{PL}(d) else Zeros{PL};
var break : boolean = FALSE;
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element : boolean = ActivePredicateElement{PL}(operand, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        break = break || element;
        let pbit : bit = if !break then '1' else '0';
        result[e*:psize] = ZeroExtend{psize}(pbit);
    elsif merging then
        let pbit : bit = PredicateElement{PL}(operand2, e, esize);
        result[e*:psize] = ZeroExtend{psize}(pbit);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

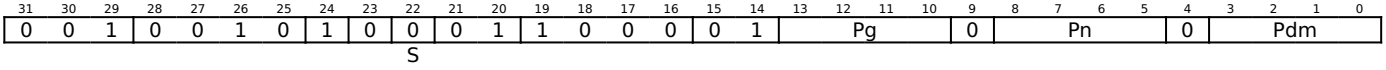
BRKN

Propagate break to next partition

This instruction propagates break to next partition. If the Last active element of the first source predicate is false, then the destination predicate is set to all-false. Otherwise, the destination and second source predicates are left unchanged. This instruction does not set the condition flags.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BRKN <Pdm>.B, <Pg>/Z, <Pn>.B, <Pdm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let dm : integer = UInt(Pdm);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pdm>	Is the name of the second source and destination scalable predicate register, encoded in the "Pdm" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(dm);
var result : bits(PL);

if LastActive{PL}(mask, operand1, 8) == '1' then
    result = operand2;
else
    result = Zeros{PL};
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(Ones{PL}, result, 8);
end;
P{PL}(dm) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

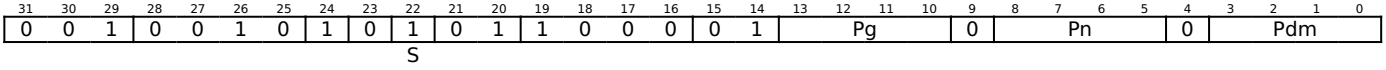
BRKNS

Propagate break to next partition, setting the condition flags

This instruction propagates break to next partition and sets the condition flags. If the Last active element of the first source predicate is false, then the destination predicate is set to all-false. Otherwise, the destination and second source predicates are left unchanged. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BRKNS <Pdm>.B, <Pg>/Z, <Pn>.B, <Pdm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let dm : integer = UInt(Pdm);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pdm>	Is the name of the second source and destination scalable predicate register, encoded in the "Pdm" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(dm);
var result : bits(PL);

if LastActive{PL}(mask, operand1, 8) == '1' then
    result = operand2;
else
    result = Zeros{PL};
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(Ones{PL}, result, 8);
end;
P{PL}(dm) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

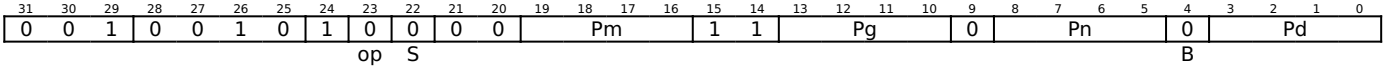
BRKPA

Break after first true condition, propagating from previous partition

This instruction breaks after first true condition, propagating from previous partition. If the Last active element of the first source predicate is false, then the destination predicate is set to all-false. Otherwise, the destination predicate elements up to and including the First active and true source element are set to true, then the subsequent elements are set to false. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BRKPA <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
var last : boolean = (LastActive{PL}(mask, operand1, 8) == '1');

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, 8) then
        let pbit : bit = if last then '1' else '0';
        result[e*:1] = ZeroExtend{1}(pbit);
        last = last && (!ActivePredicateElement{PL}(operand2, e, 8));
    else
        result[e*:1] = ZeroExtend{1}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

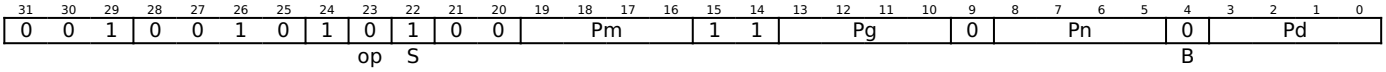
BRKPAS

Break after first true condition, propagating from previous partition and setting the condition flags

This instruction breaks after first true condition, propagating from previous partition, and sets the condition flags. If the Last active element of the first source predicate is false, then the destination predicate is set to all-false. Otherwise, the destination predicate elements up to and including the First active and true source element are set to true, then the subsequent elements are set to false. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BRKPAS <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
var last : boolean = (LastActive{PL}(mask, operand1, 8) == '1');

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, 8) then
        let pbit : bit = if last then '1' else '0';
        result[e*:1] = ZeroExtend{1}(pbit);
        last = last && (!ActivePredicateElement{PL}(operand2, e, 8));
    else
        result[e*:1] = ZeroExtend{1}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

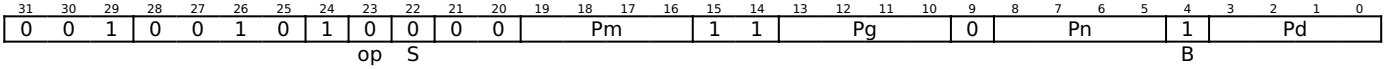
BRKPB

Break before first true condition, propagating from previous partition

This instruction breaks before first true condition, propagating from previous partition. If the Last active element of the first source predicate is false, then the destination predicate is set to all-false. Otherwise, the destination predicate elements up to but not including the First active and true source element are set to true, then the subsequent elements are set to false. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BRKPB <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
var last : boolean = (LastActive{PL}(mask, operand1, 8) == '1');

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, 8) then
        last = last && (!ActivePredicateElement{PL}(operand2, e, 8));
        let pbit : bit = if last then '1' else '0';
        result[e*:1] = ZeroExtend{1}(pbit);
    else
        result[e*:1] = ZeroExtend{1}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

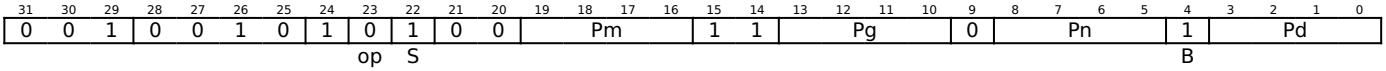
BRKPBS

Break before first true condition, propagating from previous partition and setting the condition flags

This instruction breaks before first true condition, propagating from previous partition, and sets the condition flags. If the Last active element of the first source predicate is false, then the destination predicate is set to all-false. Otherwise, the destination predicate elements up to but not including the First active and true source element are set to true, then the subsequent elements are set to false. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
BRKPBS <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
var last : boolean = (LastActive{PL}(mask, operand1, 8) == '1');

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, 8) then
        last = last && (!ActivePredicateElement{PL}(operand2, e, 8));
        let pbit : bit = if last then '1' else '0';
        result[e*:1] = ZeroExtend{1}(pbit);
    else
        result[e*:1] = ZeroExtend{1}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

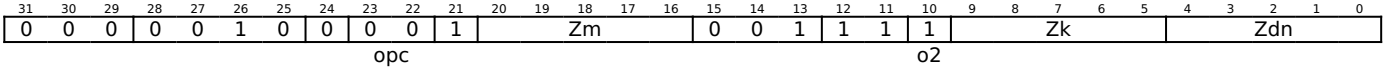
BSL

Bitwise select

This instruction selects bits from the first source vector where the corresponding bit in the third source vector is '1', and from the second source vector where the corresponding bit in the third source vector is '0'. The result is destructively placed in the destination and first source vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
BSL <Zdn>.D, <Zdn>.D, <Zm>.D, <Zk>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let m : integer = UInt(Zm);
let k : integer = UInt(Zk);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the third source scalable vector register, encoded in the "Zk" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(k);

Z{VL}(dn) = (operand1 AND operand3) OR (operand2 AND NOT(operand3));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

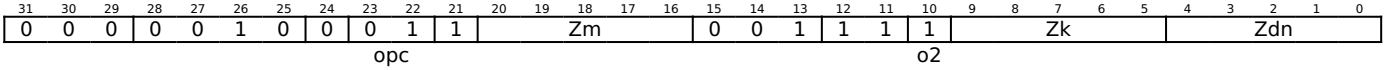
BSL1N

Bitwise select with first input inverted

This instruction selects bits from the inverted first source vector where the corresponding bit in the third source vector is '1', and from the second source vector where the corresponding bit in the third source vector is '0'. The result is destructively placed in the destination and first source vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
BSL1N <Zdn>.D, <Zdn>.D, <Zm>.D, <Zk>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let m : integer = UInt(Zm);
let k : integer = UInt(Zk);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the third source scalable vector register, encoded in the "Zk" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(k);

Z{VL}(dn) = (NOT(operand1) AND operand3) OR (operand2 AND NOT(operand3));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

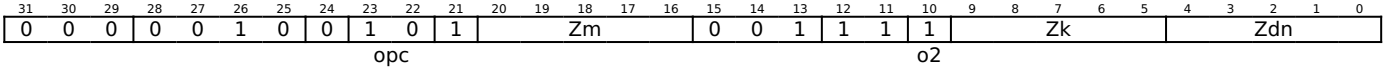
BSL2N

Bitwise select with second input inverted

This instruction selects bits from the first source vector where the corresponding bit in the third source vector is '1', and from the inverted second source vector where the corresponding bit in the third source vector is '0'. The result is destructively placed in the destination and first source vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
BSL2N <Zdn>.D, <Zdn>.D, <Zm>.D, <Zk>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let m : integer = UInt(Zm);
let k : integer = UInt(Zk);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the third source scalable vector register, encoded in the "Zk" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(k);

Z{VL}(dn) = (operand1 AND operand3) OR (NOT(operand2) AND NOT(operand3));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CADD

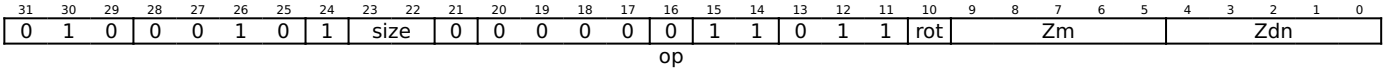
Complex integer add

This instruction adds the real and imaginary components of the integral complex numbers from the first source vector to the complex numbers from the second source vector that have first been rotated by 90 or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation, equivalent to multiplying the complex numbers in the second source vector by ±j beforehand. The results are destructively placed in the corresponding elements of the first source vector. This instruction is unpredicated.

Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
CADD <Zdn>.<T>, <Zdn>.<T>, <Zm>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
let sub_i : boolean = (rot == '0');
let sub_r : boolean = (rot == '1');
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
0	#90
1	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (2 * esize);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for p = 0 to pairs-1 do
    var acc_r : integer  = SInt(operand1[(2 * p + 0)*:esize]);
    var acc_i : integer  = SInt(operand1[(2 * p + 1)*:esize]);
    let elt2_r : integer = SInt(operand2[(2 * p + 0)*:esize]);
    let elt2_i : integer = SInt(operand2[(2 * p + 1)*:esize]);
    if sub_i then
        acc_r = acc_r - elt2_i;
        acc_i = acc_i + elt2_r;
    end;
    if sub_r then
        acc_r = acc_r + elt2_i;
        acc_i = acc_i - elt2_r;
    end;

    result[(2 * p + 0)*:esize] = acc_r[esize-1:0];
    result[(2 * p + 1)*:esize] = acc_i[esize-1:0];
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CDOT (indexed)

Complex integer dot product by indexed element

This instruction delimits the source vectors into pairs of 8-bit or 16-bit signed integer complex numbers. Within each pair, the complex numbers in the first source vector are multiplied by the corresponding complex numbers in the second source vector and the resulting wide real or wide imaginary part of the product is accumulated into a 32-bit or 64-bit destination vector element that overlaps all four of the elements that comprise a pair of complex number values in the first source vector.

As a result each instruction implicitly deinterleaves the real and imaginary components of their complex number inputs, so that the destination vector accumulates 4×wide real sums or 4×wide imaginary sums.

The complex numbers in the second source vector are rotated by 0, 90, 180, or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation, by performing the following transformations prior to the dot product operations:

- If the rotation is #0, the imaginary parts of the complex numbers in the second source vector are negated. The destination vector therefore accumulates the real parts of a complex dot product.
- If the rotation is #90, the real and imaginary parts of the complex numbers the second source vector are swapped. The destination vector therefore accumulates the imaginary parts of a complex dot product.
- If the rotation is #180, there is no transformation. The destination vector therefore accumulates the real parts of a complex conjugate dot product.
- If the rotation is #270, the real parts of the complex numbers in the second source vector are negated and then swapped with the imaginary parts. The destination vector therefore accumulates the imaginary parts of a complex conjugate dot product.

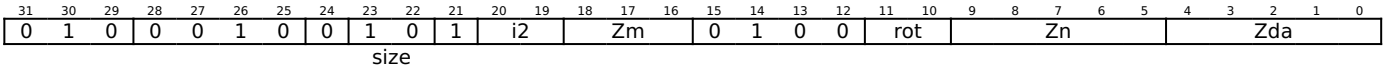
The indexed form of these instructions select a single pair of complex numbers within each 128-bit segment of the second source vector as the multiplier for all pairs of complex numbers within the corresponding 128-bit segment of the first source vector. The complex number pairs within the second source vector are specified using an immediate index that selects the same complex number pair position within each 128-bit vector segment. The index range is from 0 to one less than the number of complex number pairs per 128-bit segment.

Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element.

It has encodings from 2 classes: 8-bit to 32-bit and 16-bit to 64-bit

8-bit to 32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

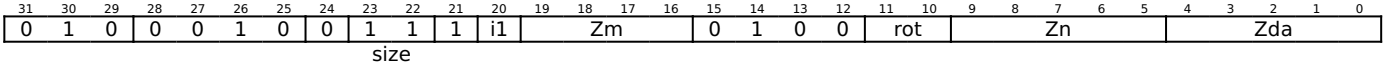
```
CDOT <Zda>.S, <Zn>.B, <Zm>.B[<imm>], <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT(rot[0]));
let sub_i : boolean = (rot[0] == rot[1]);
```

16-bit to 64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
CDOT <Zda>.D, <Zn>.H, <Zm>.H[<imm>], <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT(rot[0]));
let sub_i : boolean = (rot[0] == rot[1]);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> For the "8-bit to 32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.

For the "16-bit to 64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<imm> For the "8-bit to 32-bit" variant: is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

For the "16-bit to 64-bit" variant: is the immediate index of a group of four 16-bit values within each 128-bit vector segment, in the range 0 to 1, encoded in the "i1" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
00	#0
01	#90
10	#180
11	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 1 do
        let elt1_r : integer = SInt(operand1[(4 * e + 2 * i + 0)*:(esize DIV 4)]);
        let elt1_i : integer = SInt(operand1[(4 * e + 2 * i + 1)*:(esize DIV 4)]);
        let elt2_a : integer = SInt(operand2[(4 * s + 2 * i + sel_a)*:(esize DIV 4)]);
        let elt2_b : integer = SInt(operand2[(4 * s + 2 * i + sel_b)*:(esize DIV 4)]);
        if sub_i then
            res = (res + (elt1_r * elt2_a)) - (elt1_i * elt2_b);
        else
            res = res + (elt1_r * elt2_a) + (elt1_i * elt2_b);
        end;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CDOT (vectors)

Complex integer dot product

This instruction delimits the source vectors into pairs of 8-bit or 16-bit signed integer complex numbers. Within each pair, the complex numbers in the first source vector are multiplied by the corresponding complex numbers in the second source vector and the resulting wide real or wide imaginary part of the product is accumulated into a 32-bit or 64-bit destination vector element that overlaps all four of the elements that comprise a pair of complex number values in the first source vector.

As a result each instruction implicitly deinterleaves the real and imaginary components of their complex number inputs, so that the destination vector accumulates 4×wide real sums or 4×wide imaginary sums.

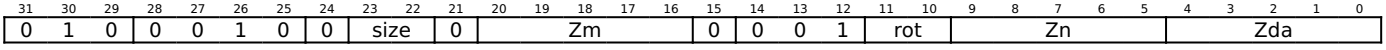
The complex numbers in the second source vector are rotated by 0, 90, 180, or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation, by performing the following transformations prior to the dot product operations:

- If the rotation is #0, the imaginary parts of the complex numbers in the second source vector are negated. The destination vector therefore accumulates the real parts of a complex dot product.
- If the rotation is #90, the real and imaginary parts of the complex numbers the second source vector are swapped. The destination vector therefore accumulates the imaginary parts of a complex dot product.
- If the rotation is #180, there is no transformation. The destination vector therefore accumulates the real parts of a complex conjugate dot product.
- If the rotation is #270, the real parts of the complex numbers in the second source vector are negated and then swapped with the imaginary parts. The destination vector therefore accumulates the imaginary parts of a complex conjugate dot product.

Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
CDOT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT(rot[0]));
let sub_i : boolean = (rot[0] == rot[1]);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size[0]”:

size[0]	<Tb>
0	B
1	H

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
00	#0
01	#90
10	#180
11	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 1 do
        let elt1_r : integer = SInt(operand1[(4 * e + 2 * i + 0)*:(esize DIV 4)]);
        let elt1_i : integer = SInt(operand1[(4 * e + 2 * i + 1)*:(esize DIV 4)]);
        let elt2_a : integer = SInt(operand2[(4 * e + 2 * i + sel_a)*:(esize DIV 4)]);
        let elt2_b : integer = SInt(operand2[(4 * e + 2 * i + sel_b)*:(esize DIV 4)]);
        if sub_i then
            res = (res + (elt1_r * elt2_a)) - (elt1_i * elt2_b);
        else
            res = res + (elt1_r * elt2_a) + (elt1_i * elt2_b);
        end;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

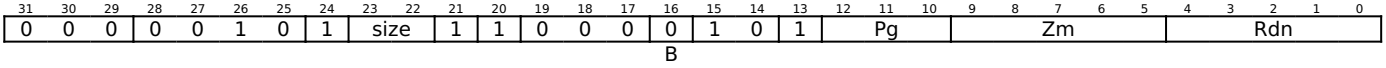
CLASTA (scalar)

Conditionally extract element after last to general-purpose register

This instruction extracts and zero-extends the element after the Last active element, or if the Last active element is the final element it extracts element zero, from the source vector register, and destructively places the zero-extended value in the destination and first source general-purpose register. If there are no Active elements, the least significant element-size bits of the destination and first source general-purpose register are destructively zero-extended.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
CLASTA <R><dn>, <Pg>, <R><dn>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Rdn);
let m : integer = UInt(Zm);
let csize : integer{} = if esize < 64 then 32 else 64;
let isBefore : boolean = FALSE;
```

Assembler Symbols

<R> Is a width specifier, encoded in “size”:

size	<R>
00	W
01	W
10	W
11	X

<dn> Is the number [0-30] of the source and destination general-purpose register or the name ZR (31), encoded in the "Rdn" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the source scalable vector register, encoded in the "Zm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(esize) = X{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(csize);
var last : integer = LastActiveElement{PL}(mask, esize);

if last < 0 then
    result = ZeroExtend{csize}(operand1);
else
    if !isBefore then
        last = last + 1;
        if last >= elements then last = 0; end;
    end;
    result = ZeroExtend{csize}(operand2[last*:esize]);

end;
X{csize}(dn) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

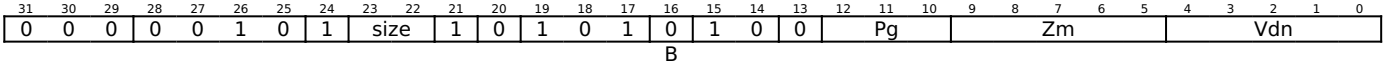
CLASTA (SIMD&FP scalar)

Conditionally extract element after last to SIMD&FP scalar register

This instruction extracts and zero-extends the element after the Last active element, or if the Last active element is the final element it extracts element zero, from the source vector register, and destructively places the zero-extended value in the destination and first source SIMD & floating-point scalar register. If there are no Active elements, the least significant element-size bits of the destination and first source SIMD & floating-point scalar register are destructively zero-extended.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
CLASTA <V><dn>, <Pg>, <V><dn>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Vdn);
let m : integer = UInt(Zm);
let isBefore : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<dn> Is the number [0-31] of the source and destination SIMD&FP register, encoded in the "Vdn" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the source scalable vector register, encoded in the "Zm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(esize) = V{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(esize);
var last : integer = LastActiveElement{PL}(mask, esize);

if last < 0 then
    result = ZeroExtend{esize}(operand1);
else
    if !isBefore then
        last = last + 1;
        if last >= elements then last = 0; end;
    end;
    result = operand2[last*:esize];

end;
V{esize}(dn) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

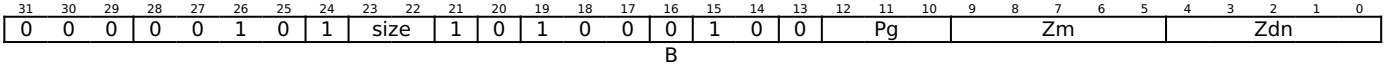
CLASTA (vectors)

Conditionally extract element after last to vector register

This instruction extracts the element after the Last active element, or if the Last active element is the final element it extracts element zero, from the second source vector register, and then replicates the extracted element to destructively fill the destination and first source vector. If there are no Active elements, the destination and source vector are unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
CLASTA <Zdn>.<T>, <Pg>, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let isBefore : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);
var last : integer = LastActiveElement{PL}(mask, esize);

if last < 0 then
    result = operand1;
else
    if !isBefore then
        last = last + 1;
        if last >= elements then last = 0; end;
    end;
    for e = 0 to elements-1 do
        result[e*:esize] = operand2[last*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

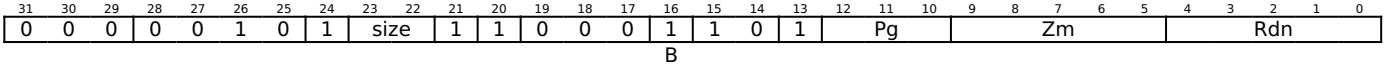
CLASTB (scalar)

Conditionally extract last element to general-purpose register

This instruction extracts and zero-extends the Last active element from the source vector register, and destructively places the zero-extended value in the destination and first source general-purpose register. If there are no Active elements, the least significant element-size bits of the destination and first source general-purpose register are destructively zero-extended.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
CLASTB <R><dn>, <Pg>, <R><dn>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Rdn);
let m : integer = UInt(Zm);
let csize : integer{} = if esize < 64 then 32 else 64;
let isBefore : boolean = TRUE;
```

Assembler Symbols

<R> Is a width specifier, encoded in “size”:

size	<R>
00	W
01	W
10	W
11	X

<dn> Is the number [0-30] of the source and destination general-purpose register or the name ZR (31), encoded in the "Rdn" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the source scalable vector register, encoded in the "Zm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(esize) = X{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(csize);
var last : integer = LastActiveElement{PL}(mask, esize);

if last < 0 then
    result = ZeroExtend{csize}(operand1);
else
    if !isBefore then
        last = last + 1;
        if last >= elements then last = 0; end;
    end;
    result = ZeroExtend{csize}(operand2[last*:esize]);

end;
X{csize}(dn) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

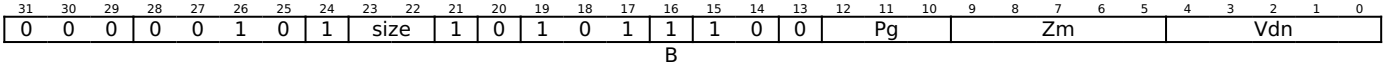
CLASTB (SIMD&FP scalar)

Conditionally extract last element to SIMD&FP scalar register

This instruction extracts and zero-extends the Last active element from the source vector register, and destructively places the zero-extended value in the destination and first source SIMD & floating-point scalar register. If there are no Active elements, the least significant element-size bits of the destination and first source SIMD & floating-point scalar register are destructively zero-extended.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
CLASTB <V><dn>, <Pg>, <V><dn>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Vdn);
let m : integer = UInt(Zm);
let isBefore : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<dn> Is the number [0-31] of the source and destination SIMD&FP register, encoded in the "Vdn" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the source scalable vector register, encoded in the "Zm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(esize) = V{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(esize);
var last : integer = LastActiveElement{PL}(mask, esize);

if last < 0 then
    result = ZeroExtend{esize}(operand1);
else
    if !isBefore then
        last = last + 1;
        if last >= elements then last = 0; end;
    end;
    result = operand2[last*:esize];

end;
V{esize}(dn) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

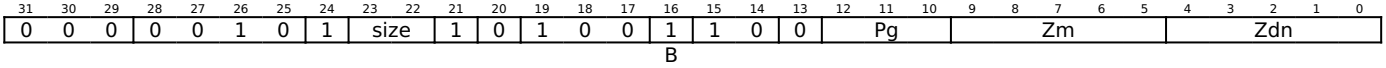
CLASTB (vectors)

Conditionally extract last element to vector register

This instruction extracts the Last active element from the second source vector register, and then replicates the extracted element to destructively fill the destination and first source vector. If there are no Active elements, the destination and source vector are unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
CLASTB <Zdn>.<T>, <Pg>, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let isBefore : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);
var last : integer = LastActiveElement{PL}(mask, esize);

if last < 0 then
    result = operand1;
else
    if !isBefore then
        last = last + 1;
        if last >= elements then last = 0; end;
    end;
    for e = 0 to elements-1 do
        result[e*:esize] = operand2[last*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CLS

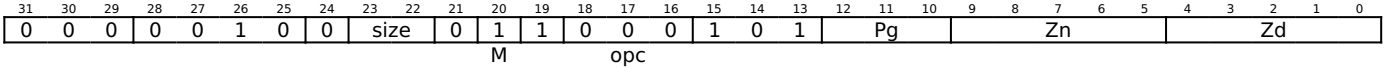
Count leading sign bits (predicated)

This instruction determines the number of consecutive sign bits, starting from the most significant bit in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

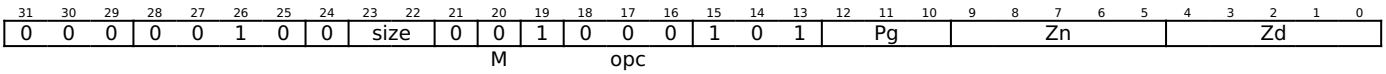
```
CLS <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
CLS <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = CountLeadingSignBits(element)[esize-1:0];

    end;
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CLZ

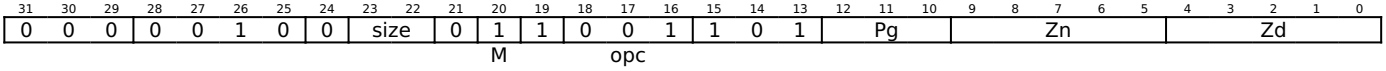
Count leading zero bits (predicated)

This instruction determines the number of consecutive binary zero bits, starting from the most significant bit in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

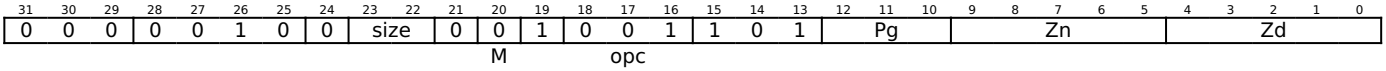
```
CLZ <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
CLZ <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = CountLeadingZeroBits(element)[esize-1:0];

    end;
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMLA (indexed)

Complex integer multiply-add by indexed element

This instruction multiplies the duplicated real components for rotations 0 and 180, or imaginary components for rotations 90 and 270, of the integral numbers in each 128-bit segment of the first source vector by the specified complex number in the corresponding the second source vector segment rotated by 0, 90, 180, or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation.

This instruction then adds the products to the corresponding components of the complex numbers in the addend vector, and destructively places the results in the corresponding elements of the addend vector. This instruction is unpredicated.

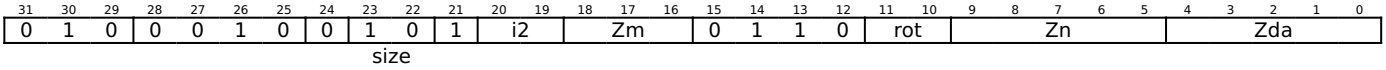
These transformations permit the creation of a variety of multiply-add and multiply-subtract operations on complex numbers by combining two of these instructions with the same vector operands but with rotations that are 90 degrees apart.

Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element.

It has encodings from 2 classes: 16-bit and 32-bit

16-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

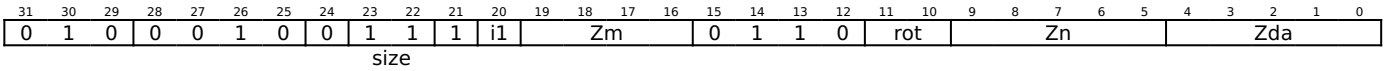
```
CMLA <Zda>.H, <Zn>.H, <Zm>.H[<imm>], <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT(rot[0]));
let sub_r : boolean = (rot[0] != rot[1]);
let sub_i : boolean = (rot[1] == '1');
```

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
CMLA <Zda>.S, <Zn>.S, <Zm>.S[<imm>], <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT(rot[0]));
let sub_r : boolean = (rot[0] != rot[1]);
let sub_i : boolean = (rot[1] == '1');
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "16-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "16-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2" field.

For the "32-bit" variant: is the element index, in the range 0 to 1, encoded in the "i1" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
00	#0
01	#90
10	#180
11	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (2 * esize);
let pairspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for p = 0 to pairs-1 do
    let segmentbase : integer = p - (p MOD pairspersegment);
    let s : integer = segmentbase + index;
    let elt1_a : integer = SInt(operand1[(2 * p + sel_a)*:esize]);
    let elt2_a : integer = SInt(operand2[(2 * s + sel_a)*:esize]);
    let elt2_b : integer = SInt(operand2[(2 * s + sel_b)*:esize]);
    let elt3_r : bits(esize) = operand3[(2 * p + 0)*:esize];
    let elt3_i : bits(esize) = operand3[(2 * p + 1)*:esize];
    let product_r : integer =  elt1_a * elt2_a;
    let product_i : integer =  elt1_a * elt2_b;
    if sub_r then
        result[(2 * p + 0)*:esize] = elt3_r - product_r;
    else
        result[(2 * p + 0)*:esize] = elt3_r + product_r;
    end;
    if sub_i then
        result[(2 * p + 1)*:esize] = elt3_i - product_i;
    else
        result[(2 * p + 1)*:esize] = elt3_i + product_i;
    end;

end;
Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMLA (vectors)

Complex integer multiply-add

This instruction multiplies the duplicated real components for rotations 0 and 180, or imaginary components for rotations 90 and 270, of the integral numbers in the first source vector by the corresponding complex number in the second source vector rotated by 0, 90, 180, or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation.

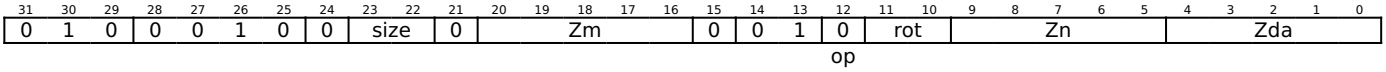
This instruction then adds the products to the corresponding components of the complex numbers in the addend vector, and destructively places the results in the corresponding elements of the addend vector. This instruction is unpredicated.

These transformations permit the creation of a variety of multiply-add and multiply-subtract operations on complex numbers by combining two of these instructions with the same vector operands but with rotations that are 90 degrees apart.

Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
CMLA <Zda>.<T>, <Zn>.<T>, <Zm>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT(rot[0]));
let sub_r : boolean = (rot[0] != rot[1]);
let sub_i : boolean = (rot[1] == '1');
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
00	#0
01	#90
10	#180
11	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for p = 0 to pairs-1 do
    let elt1_a : integer = SInt(operand1[(2 * p + sel_a)*:esize]);
    let elt2_a : integer = SInt(operand2[(2 * p + sel_a)*:esize]);
    let elt2_b : integer = SInt(operand2[(2 * p + sel_b)*:esize]);
    let elt3_r : bits(esize) = operand3[(2 * p + 0)*:esize];
    let elt3_i : bits(esize) = operand3[(2 * p + 1)*:esize];
    let product_r : integer =  elt1_a * elt2_a;
    let product_i : integer =  elt1_a * elt2_b;
    if sub_r then
        result[(2 * p + 0)*:esize] = elt3_r - product_r;
    else
        result[(2 * p + 0)*:esize] = elt3_r + product_r;
    end;
    if sub_i then
        result[(2 * p + 1)*:esize] = elt3_i - product_i;
    else
        result[(2 * p + 1)*:esize] = elt3_i + product_i;
    end;

end;
Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMP<cc> (immediate)

Compare vector to immediate

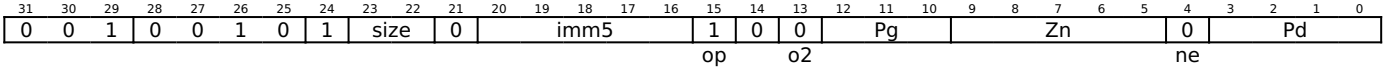
This instruction compares Active integer elements in the source vector with an immediate, and places the boolean results of the specified comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

<cc>	Comparison
EQ	equal
GE	signed greater than or equal
GT	signed greater than
HI	unsigned higher than
HS	unsigned higher than or same
LE	signed less than or equal
LO	unsigned lower than
LS	unsigned lower than or same
LT	signed less than
NE	not equal

It has encodings from 10 classes: Equal , Greater than , Greater than or equal , Higher , Higher or same , Less than , Less than or equal , Lower , Lower or same and Not equal

Equal

(FEAT_SVE || FEAT_SME)



Encoding

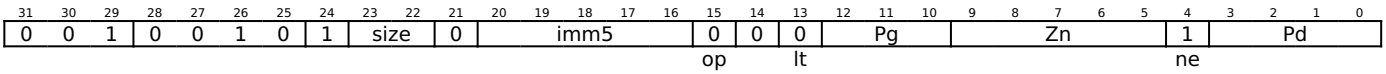
```
CMPEQ <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_EQ;
let imm : integer = SInt(imm5);
let unsigned : boolean = FALSE;
```

Greater than

(FEAT_SVE || FEAT_SME)



Encoding

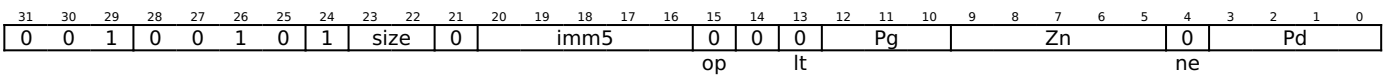
```
CMPGT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GT;
let imm : integer = SInt(imm5);
let unsigned : boolean = FALSE;
```

Greater than or equal

(FEAT_SVE || FEAT_SME)



Encoding

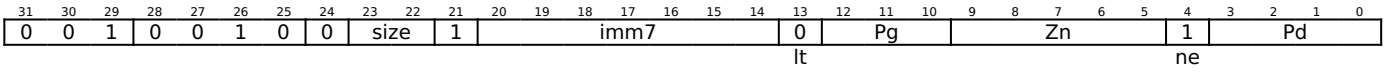
```
CMPGE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GE;
let imm : integer = SInt(imm5);
let unsigned : boolean = FALSE;
```

Higher

(FEAT_SVE || FEAT_SME)



Encoding

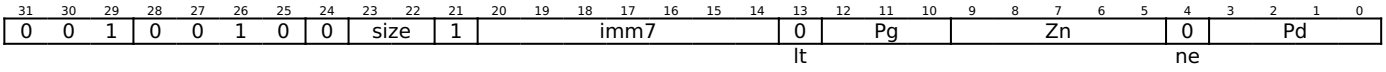
```
CMPHI <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GT;
let imm : integer = UInt(imm7);
let unsigned : boolean = TRUE;
```

Higher or same

(FEAT_SVE || FEAT_SME)



Encoding

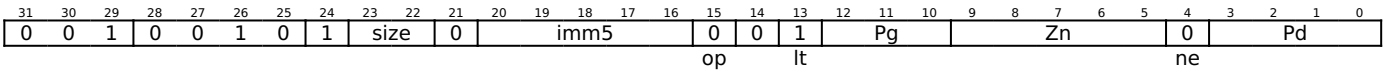
```
CMPHS <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GE;
let imm : integer = UInt(imm7);
let unsigned : boolean = TRUE;
```

Less than

(FEAT_SVE || FEAT_SME)



Encoding

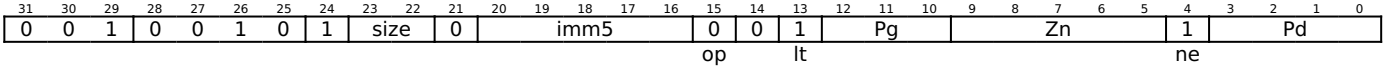
```
CMPLT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_LT;
let imm : integer = SInt(imm5);
let unsigned : boolean = FALSE;
```

Less than or equal

(FEAT_SVE || FEAT_SME)



Encoding

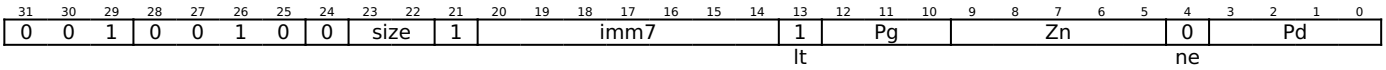
```
CMPLE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_LE;
let imm : integer = SInt(imm5);
let unsigned : boolean = FALSE;
```

Lower

(FEAT_SVE || FEAT_SME)



Encoding

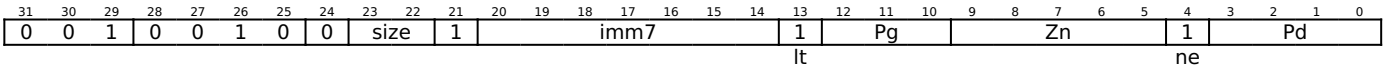
```
CMPLO <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_LT;
let imm : integer = UInt(imm7);
let unsigned : boolean = TRUE;
```

Lower or same

(FEAT_SVE || FEAT_SME)



Encoding

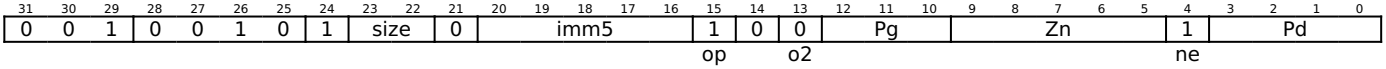
```
CMPLS <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_LE;
let imm : integer = UInt(imm7);
let unsigned : boolean = TRUE;
```

Not equal

(FEAT_SVE || FEAT_SME)



Encoding

```
CMPNE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_NE;
let imm : integer = SInt(imm5);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<imm> For the "Equal", "Greater than or equal", "Greater than", "Less than or equal", "Less than", and "Not equal" variants: is the signed immediate operand, in the range -16 to 15, encoded in the "imm5" field.

For the "Higher or same", "Higher", "Lower or same", and "Lower" variants: is the unsigned immediate operand, in the range 0 to 127, encoded in the "imm7" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    if ActivePredicateElement{PL}(mask, e, esize) then
        var cond : boolean;
        case cmp_op of
            when Cmp_EQ => cond = element1 == imm;
            when Cmp_NE => cond = element1 != imm;
            when Cmp_GE => cond = element1 >= imm;
            when Cmp_LT => cond = element1 <  imm;
            when Cmp_GT => cond = element1 >  imm;
            when Cmp_LE => cond = element1 <= imm;
        end;
        let pbit : bit = if cond then '1' else '0';
        result[e*:psize] = ZeroExtend{psize}(pbit);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the predicate register or NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMP<cc> (vectors)

Compare vectors

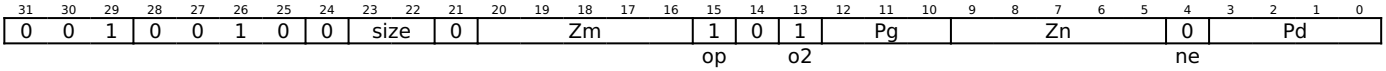
This instruction compares Active integer elements in the first source vector with the corresponding elements in the second source vector, and places the boolean results of the specified comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

<cc>	Comparison
EQ	equal
GE	signed greater than or equal
GT	signed greater than
HI	unsigned higher than
HS	unsigned higher than or same
NE	not equal

This instruction is used by the pseudo-instructions CMPLE (vectors), CMPLO (vectors), CMPLS (vectors), and CMPLT (vectors).
It has encodings from 6 classes: Equal , Greater than , Greater than or equal , Higher , Higher or same and Not equal

Equal

(FEAT_SVE || FEAT_SME)



Encoding

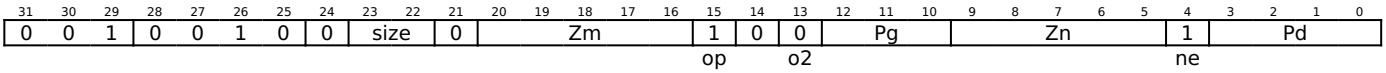
```
CMPEQ <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_EQ;
let unsigned : boolean = FALSE;
```

Greater than

(FEAT_SVE || FEAT_SME)



Encoding

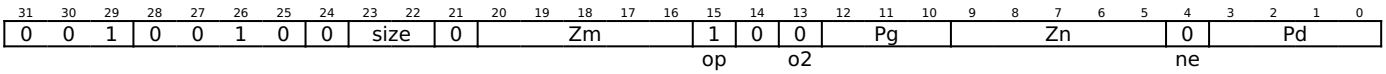
```
CMPGT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GT;
let unsigned : boolean = FALSE;
```

Greater than or equal

(FEAT_SVE || FEAT_SME)



Encoding

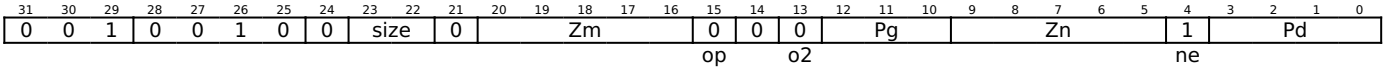
```
CMPGE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GE;
let unsigned : boolean = FALSE;
```

Higher

(FEAT_SVE || FEAT_SME)



Encoding

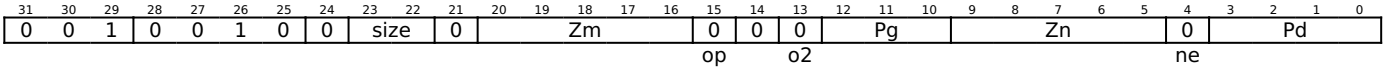
```
CMPHI <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GT;
let unsigned : boolean = TRUE;
```

Higher or same

(FEAT_SVE || FEAT_SME)



Encoding

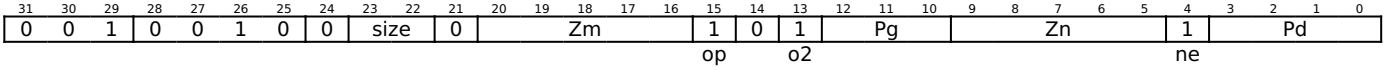
```
CMPHS <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GE;
let unsigned : boolean = TRUE;
```

Not equal

(FEAT_SVE || FEAT_SME)



Encoding

```
CMPNE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_NE;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    if ActivePredicateElement{PL}(mask, e, esize) then
        var cond : boolean;
        let op2elt : bits(esize) = operand2[e*:esize];
        let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
        case cmp_op of
            when Cmp_EQ => cond = element1 == element2;
            when Cmp_NE => cond = element1 != element2;
            when Cmp_GE => cond = element1 >= element2;
            when Cmp_LT => cond = element1 <  element2;
            when Cmp_GT => cond = element1 >  element2;
            when Cmp_LE => cond = element1 <= element2;
        end;
        let pbit : bit = if cond then '1' else '0';
        result[e*:psize] = ZeroExtend{psize}(pbit);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the predicate register or NZCV condition flags written by this instruction might be significantly delayed.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMP<cc> (wide elements)

Compare vector to 64-bit wide elements

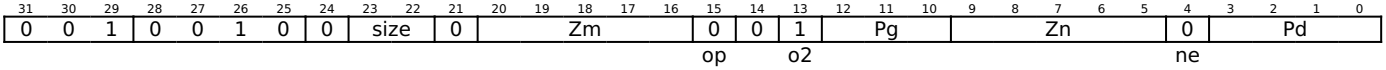
This instruction compares Active integer elements in the first source vector with overlapping 64-bit doubleword elements in the second source vector, and places the boolean results of the specified comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

<cc>	Comparison
EQ	equal
GE	signed greater than or equal
GT	signed greater than
HI	unsigned higher than
HS	unsigned higher than or same
LE	signed less than or equal
LO	unsigned lower than
LS	unsigned lower than or same
LT	signed less than
NE	not equal

It has encodings from 10 classes: Equal , Greater than , Greater than or equal , Higher , Higher or same , Less than , Less than or equal , Lower , Lower or same and Not equal

Equal

(FEAT_SVE || FEAT_SME)



Encoding

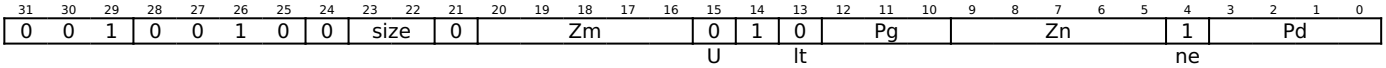
```
CMPEQ <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_EQ;
let unsigned : boolean = FALSE;
```

Greater than

(FEAT_SVE || FEAT_SME)



Encoding

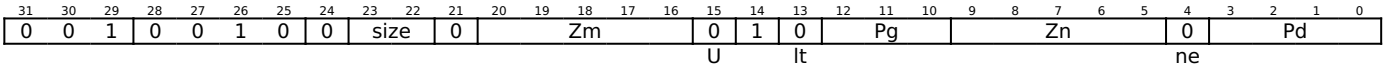
```
CMPGT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GT;
let unsigned : boolean = FALSE;
```

Greater than or equal

(FEAT_SVE || FEAT_SME)



Encoding

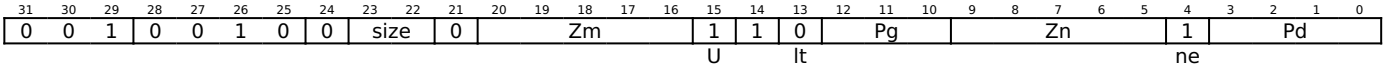
```
CMPGE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GE;
let unsigned : boolean = FALSE;
```

Higher

(FEAT_SVE || FEAT_SME)



Encoding

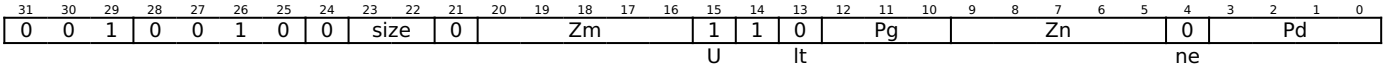
```
CMPHI <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GT;
let unsigned : boolean = TRUE;
```

Higher or same

(FEAT_SVE || FEAT_SME)



Encoding

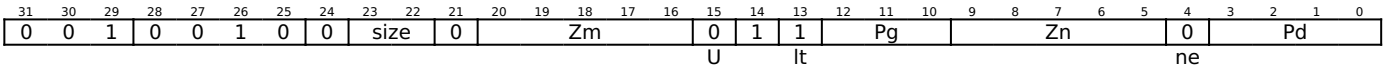
```
CMPHS <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GE;
let unsigned : boolean = TRUE;
```

Less than

(FEAT_SVE || FEAT_SME)



Encoding

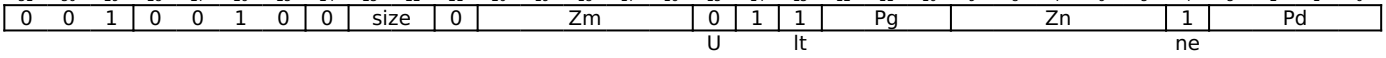
```
CMPLT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_LT;
let unsigned : boolean = FALSE;
```

Less than or equal

(FEAT_SVE || FEAT_SME)



Encoding

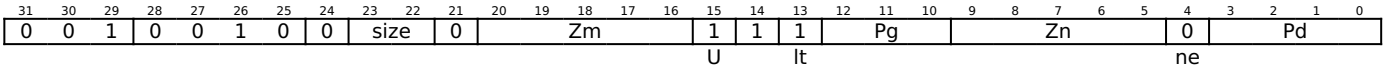
```
CMPLE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_LE;
let unsigned : boolean = FALSE;
```

Lower

(FEAT_SVE || FEAT_SME)



Encoding

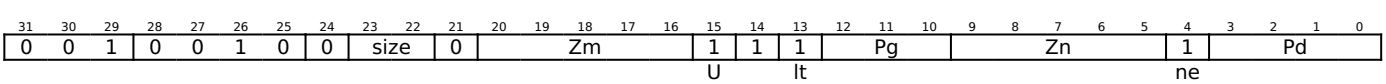
```
CMPLO <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_LT;
let unsigned : boolean = TRUE;
```

Lower or same

(FEAT_SVE || FEAT_SME)



Encoding

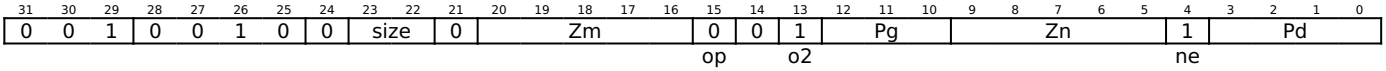
```
CMPLS <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_LE;
let unsigned : boolean = TRUE;
```

Not equal

(FEAT_SVE || FEAT_SME)



Encoding

```
CMPNE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_NE;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    if ActivePredicateElement{PL}(mask, e, esize) then
        var cond : boolean;
        let op2elt : bits(64) = operand2[((e * esize) DIVRM 64)*:64];
        let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
        case cmp_op of
            when Cmp_EQ => cond = element1 == element2;
            when Cmp_NE => cond = element1 != element2;
            when Cmp_GE => cond = element1 >= element2;
            when Cmp_LT => cond = element1 <  element2;
            when Cmp_GT => cond = element1 >  element2;
            when Cmp_LE => cond = element1 <= element2;
        end;
        let pbit : bit = if cond then '1' else '0';
        result[e*:psize] = ZeroExtend{psize}(pbit);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the predicate register or NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMPLE (vectors)

Compare signed less than or equal to vector, setting the condition flags

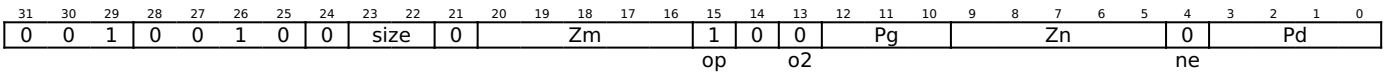
This instruction compares Active signed integer elements in the first source vector being less than or equal to the corresponding signed elements in the second source vector, and places the boolean results of the comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This is a pseudo-instruction of CMP<cc> (vectors). This means:

- The encodings in this description are named to match the encodings of CMP<cc> (vectors).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CMP<cc> (vectors) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Greater than or equal

(FEAT_SVE || FEAT_SME)



Encoding

```
CMPLE <Pd>.<T>, <Pg>/Z, <Zm>.<T>, <Zn>.<T>
```

is equivalent to

```
CMPGE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

Operation

The description of CMP<cc> (vectors) gives the operational pseudocode for this instruction.

Operational Information

The description of CMP<cc> (vectors) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMPLO (vectors)

Compare unsigned lower than vector, setting the condition flags

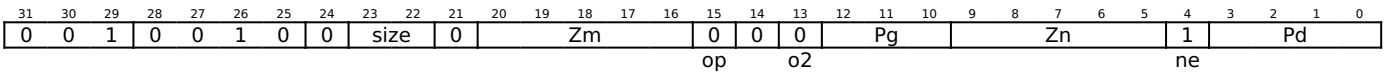
This instruction compares Active unsigned integer elements in the first source vector being lower than the corresponding unsigned elements in the second source vector, and places the boolean results of the comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This is a pseudo-instruction of CMP<cc> (vectors). This means:

- The encodings in this description are named to match the encodings of CMP<cc> (vectors).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CMP<cc> (vectors) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Higher

(FEAT_SVE || FEAT_SME)



Encoding

```
CMPLO <Pd>.<T>, <Pg>/Z, <Zm>.<T>, <Zn>.<T>
```

is equivalent to

```
CMPHI <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

Operation

The description of CMP<cc> (vectors) gives the operational pseudocode for this instruction.

Operational Information

The description of CMP<cc> (vectors) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMPLS (vectors)

Compare unsigned lower than or same as vector, setting the condition flags

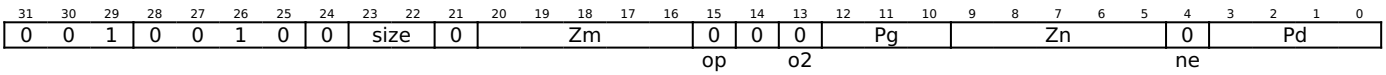
This instruction compares Active unsigned integer elements in the first source vector being lower than or the same as the corresponding unsigned elements in the second source vector, and places the boolean results of the comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This is a pseudo-instruction of CMP<cc> (vectors). This means:

- The encodings in this description are named to match the encodings of CMP<cc> (vectors).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CMP<cc> (vectors) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Higher or same

(FEAT_SVE || FEAT_SME)



Encoding

```
CMPLS <Pd>.<T>, <Pg>/Z, <Zm>.<T>, <Zn>.<T>
```

is equivalent to

```
CMPHS <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

Operation

The description of CMP<cc> (vectors) gives the operational pseudocode for this instruction.

Operational Information

The description of CMP<cc> (vectors) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CMPLT (vectors)

Compare signed less than vector, setting the condition flags

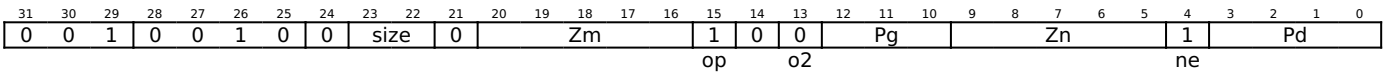
This instruction compares Active signed integer elements in the first source vector being less than the corresponding signed elements in the second source vector, and places the boolean results of the comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This is a pseudo-instruction of CMP<cc> (vectors). This means:

- The encodings in this description are named to match the encodings of CMP<cc> (vectors).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CMP<cc> (vectors) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Greater than

(FEAT_SVE || FEAT_SME)



Encoding

```
CMPLT <Pd>.<T>, <Pg>/Z, <Zm>.<T>, <Zn>.<T>
```

is equivalent to

```
CMPGT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

Operation

The description of CMP<cc> (vectors) gives the operational pseudocode for this instruction.

Operational Information

The description of CMP<cc> (vectors) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CNOT

Logically invert boolean condition (predicated)

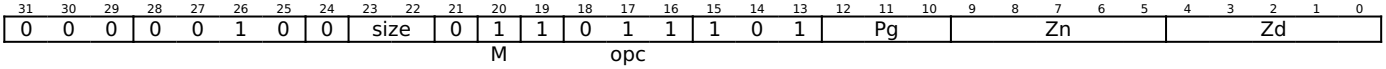
This instruction logically inverts the boolean value in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

Boolean TRUE is any nonzero value in a source, and one in a result element. Boolean FALSE is always zero.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

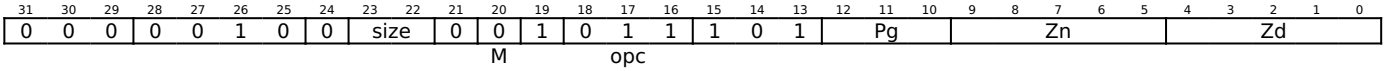
```
CNOT <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
CNOT <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = ZeroExtend{esize}(IsZeroBit{esize}(element));

    end;
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CNT

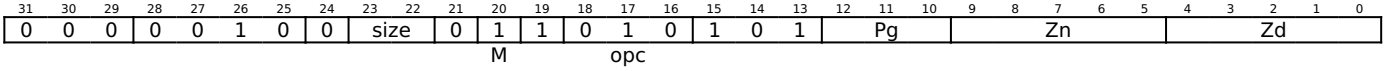
Count nonzero bits (predicated)

This instruction determines the number of nonzero bits in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

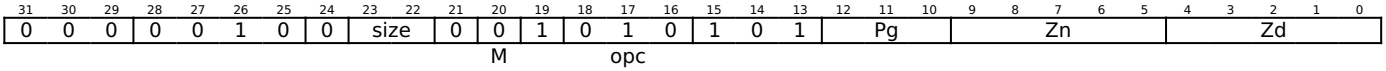
```
CNT <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
CNT <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = BitCount(element)[esize-1:0];

    end;
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CNTB, CNTD, CNTH, CNTW

Set scalar to multiple of predicate constraint element count

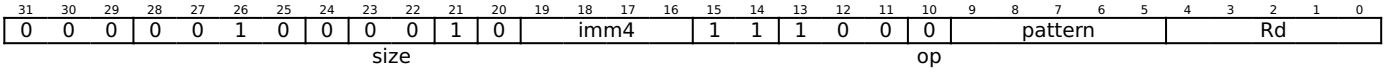
This instruction determines the number of Active elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then places the result in the scalar destination.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 4 classes: Byte , Doubleword , Halfword and Word

Byte

(FEAT_SVE || FEAT_SME)



Encoding

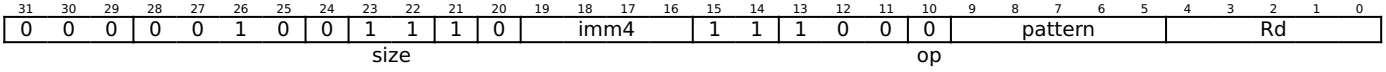
```
CNTB <Xd>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let d : integer = UInt(Rd);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Doubleword

(FEAT_SVE || FEAT_SME)



Encoding

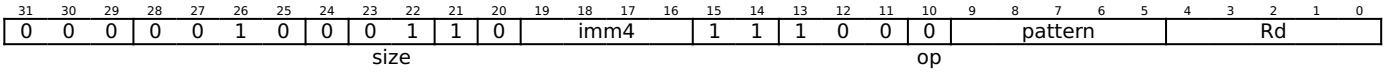
```
CNTD <Xd>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let d : integer = UInt(Rd);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Halfword

(FEAT_SVE || FEAT_SME)



Encoding

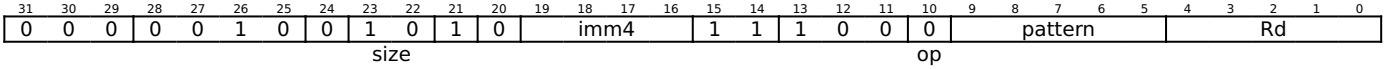
```
CNTH <Xd>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let d : integer = UInt(Rd);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Word

(FEAT_SVE || FEAT_SME)



Encoding

```
CNTW <Xd>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let d : integer = UInt(Rd);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Assembler Symbols

<Xd> Is the 64-bit name of the destination general-purpose register, encoded in the "Rd" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
X{64}(d) = (count * imm)[63:0];
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

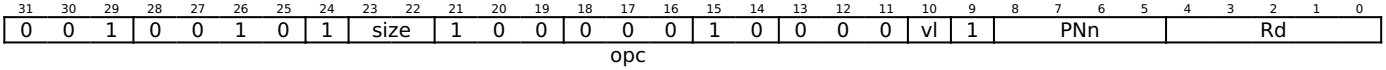
CNTP (predicate as counter)

Set scalar to count from predicate-as-counter

This instruction determines the number of true elements in the source predicate, up to the total number of elements in two or four vectors, and places the scalar result in the destination general-purpose register.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
CNTP <Xd>, <PNn>.<T>, <vl>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(PNn);
let d : integer = UInt(Rd);
let width : integer = 2 << UInt(vl);
```

Assembler Symbols

<Xd> Is the 64-bit name of the destination general-purpose register, encoded in the "Rd" field.

<PNn> Is the name of the source scalable predicate register, with predicate-as-counter encoding, encoded in the "PNn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<vl> Is the vl specifier, encoded in “vl”:

vl	<vl>
0	VLx2
1	VLx4

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then CheckSVEEnabled(); else CheckStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let pred : bits(PL) = P{}(n);
let mask : bits(PL*4) = CounterToPredicate{}(pred[15:0]);
var sum : bits(64) = Zeros{};
let maxelements : integer = elements * width;

for e = 0 to maxelements-1 do
    if ActivePredicateElement{PL*4}(mask, e, esize) then
        sum = sum + 1;
    end;
end;
X{64}(d) = sum;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

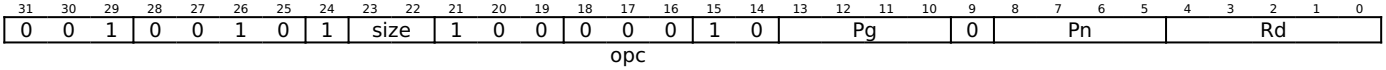
CNTP (predicate)

Set scalar to count of true predicate elements

This instruction determines the number of Active and true elements in the source predicate and places the scalar result in the destination general-purpose register. Inactive predicate elements are not counted.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
CNTP <Xd>, <Pg>, <Pn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let d : integer = UInt(Rd);
```

Assembler Symbols

<Xd> Is the 64-bit name of the destination general-purpose register, encoded in the "Rd" field.

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<Pn> Is the name of the source scalable predicate register, encoded in the "Pn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(PL) = P{}(n);
var sum : bits(64) = Zeros{};

for e = 0 to elements-1 do
    if (ActivePredicateElement{PL}(mask, e, esize)
         && ActivePredicateElement{PL}(operand, e, esize)) then
        sum = sum + 1;
    end;
end;
X{64}(d) = sum;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

COMPACT

Copy Active vector elements to lower-numbered elements

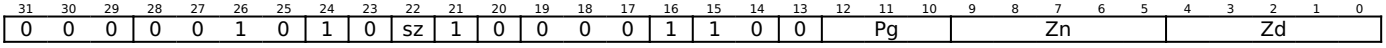
This instruction copies Active elements from the source vector to consecutive elements of the destination vector, in increasing order of element number, and sets any remaining elements to zero.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled, or FEAT_SME2p2 is implemented.

It has encodings from 2 classes: Byte and halfword and Word and doubleword

Byte and halfword

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

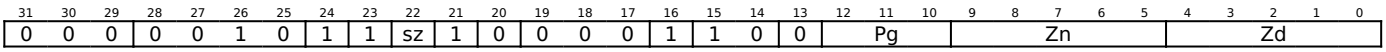
```
COMPACT <Zd>.<T>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(sz);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Word and doubleword

(FEAT_SVE || FEAT_SME2p2)



Encoding

```
COMPACT <Zd>.<T>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> For the "Byte and halfword" variant: is the size specifier, encoded in “sz”:

sz	<T>
0	B
1	H

For the "Word and doubleword" variant: is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2p2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = Zeros{};
var x : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand1[e*:esize];
        result[x*:esize] = element;
        x = x + 1;
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPY (immediate, merging)

Copy signed integer immediate to vector elements (merging)

This instruction copies a signed integer immediate to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.

The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

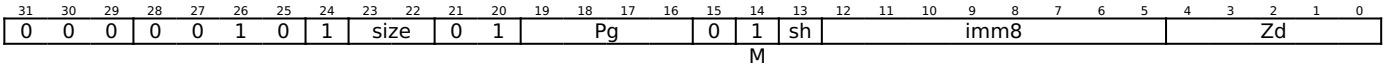
The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This instruction is used by the alias MOV (immediate, merging).

This instruction is used by the pseudo-instruction FMOV (zero, predicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
CPY <Zd>.<T>, <Pg>/M, #<imm>{, <shift>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
var imm : integer = SInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<imm> Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Alias Conditions

Alias	Is preferred when
FMOV (zero, predicated)	`Never`
MOV (immediate, merging)	`Unconditionally`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let dest : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = imm[esize-1:0];
    else
        result[e*:esize] = dest[e*:esize];
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CPY (immediate, zeroing)

Copy signed integer immediate to vector elements (zeroing)

This instruction copies a signed integer immediate to each Active element in the destination vector. Inactive elements in the destination vector register are set to zero.

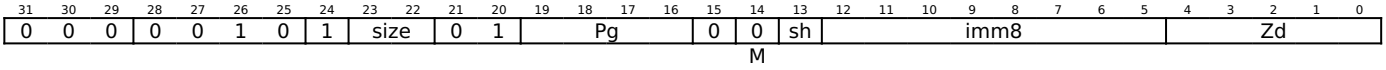
The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This instruction is used by the alias MOV (immediate, zeroing).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
CPY <Zd>.<T>, <Pg>/Z, #<imm>{, <shift>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
var imm : integer = SInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<imm> Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let dest : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = imm[esize-1:0];
    else
        result[e*:esize] = dest[e*:esize];
    end;
end;

Z{VL}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

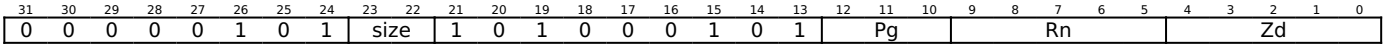
CPY (scalar)

Copy general-purpose register to vector elements (predicated)

This instruction copies the general-purpose scalar source register to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.
This instruction is used by the alias MOV (scalar, predicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding for the 32-bit variant

Applies when `(size != 11)`

```
CPY <Zd>.<T>, <Pg>/M, <Wn|WSP>
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
CPY <Zd>.D, <Pg>/M, <Xn|SP>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Wn|WSP> Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

<Xn|SP> Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var result : bits(VL) = Z{}(d);
if AnyActiveElement{PL}(mask, esize) then
    var operand1 : bits(64);
    if n == 31 then
        operand1 = SP{64}();
    else
        operand1 = X{64}(n);
    end;

    for e = 0 to elements-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            result[e*:esize] = operand1[esize-1:0];
        end;
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

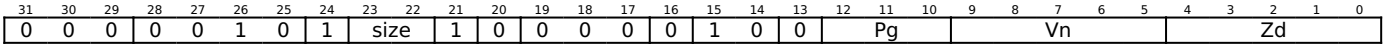
CPY (SIMD&FP scalar)

Copy SIMD&FP scalar register to vector elements (predicated)

This instruction copies the SIMD & floating-point scalar source register to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.
This instruction is used by the alias MOV (SIMD&FP scalar, predicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
CPY <Zd>.<T>, <Pg>/M, <V><n>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Vn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<n> Is the number [0-31] of the source SIMD&FP register, encoded in the "Vn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(esize) = (if AnyActiveElement{PL}(mask, esize) then
                              V{esize}(n) else Zeros{esize});
var result : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand1;
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.

- A predicated `MOVPRFX` must use the same governing predicate register as this instruction.
- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

CTERMEQ, CTERMNE

Compare and terminate loop

This instruction detects termination conditions in serialized vector loops by testing whether the comparison between the scalar source operands holds true. If the comparison is not successful, the instruction tests the state of the !Last condition flag (C), which indicates whether the previous flag-setting predicate instruction selected the last element of the vector partition.

The Z and C condition flags are preserved by this instruction. The N and V condition flags are set as a pair to generate one of the following conditions for a subsequent conditional instruction:

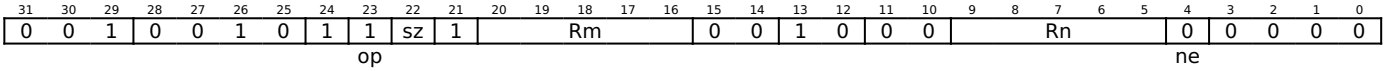
Condition	N	V	Meaning
GE	0	0	Continue loop (compare failed and last element not selected)
LT	0	1	Terminate loop (last element selected)
LT	1	0	Terminate loop (compare succeeded)
GE	1	1	Never generated

The scalar source operands are 32-bit or 64-bit general-purpose registers of the same size.

It has encodings from 2 classes: Equal and Not equal

Equal

(FEAT_SVE || FEAT_SME)



Encoding

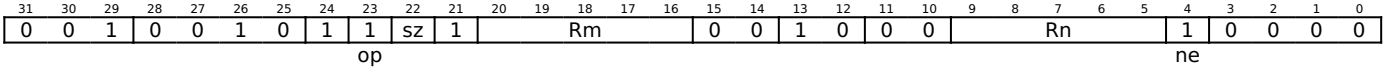
```
CTERMEQ <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let cmp_op : CmpOp = Cmp_EQ;
```

Not equal

(FEAT_SVE || FEAT_SME)



Encoding

```
CTERMNE <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let cmp_op : CmpOp = Cmp_NE;
```

Assembler Symbols

<R> Is a width specifier, encoded in “sz”:

sz	<R>
0	W
1	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let operand1 : bits(esize) = X{}(n);
let operand2 : bits(esize) = X{}(m);
let element1 : integer = UInt(operand1);
let element2 : integer = UInt(operand2);
var term : boolean;

case cmp_op of
    when Cmp_EQ => term = element1 == element2;
    when Cmp_NE => term = element1 != element2;
end;
if term then
    PSTATE.N = '1';
    PSTATE.V = '0';
else
    PSTATE.N = '0';
    PSTATE.V = (NOT PSTATE.C);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DECB, DECD, DECH, DECW (scalar)

Decrement scalar by multiple of predicate constraint element count

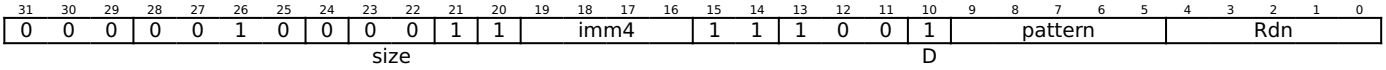
This instruction determines the number of Active elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement the scalar destination.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 4 classes: Byte , Doubleword , Halfword and Word

Byte

(FEAT_SVE || FEAT_SME)



Encoding

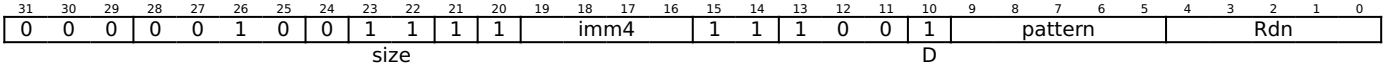
```
DECB <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Doubleword

(FEAT_SVE || FEAT_SME)



Encoding

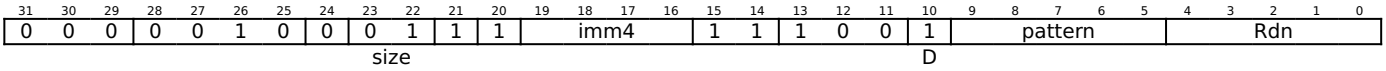
```
DECD <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Halfword

(FEAT_SVE || FEAT_SME)



Encoding

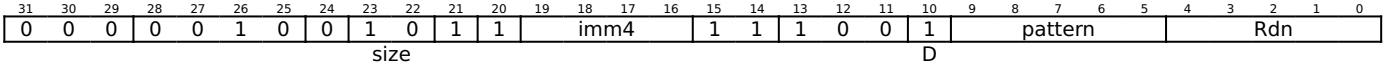
```
DECH <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Word

(FEAT_SVE || FEAT_SME)



Encoding

```
DECW <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Assembler Symbols

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let VL : integer{} = CurrentVL();
let operand1 : bits(64) = X{}(dn);
X{64}(dn) = operand1 - (count * imm);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DECD, DECH, DECW (vector)

Decrement vector by multiple of predicate constraint element count

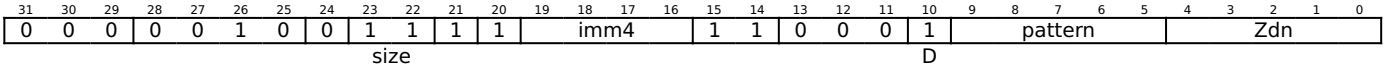
This instruction determines the number of Active elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement all destination vector elements.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 3 classes: Doubleword , Halfword and Word

Doubleword

(FEAT_SVE || FEAT_SME)



Encoding

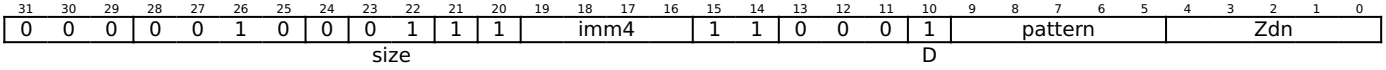
```
DECD <Zdn>.D{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Halfword

(FEAT_SVE || FEAT_SME)



Encoding

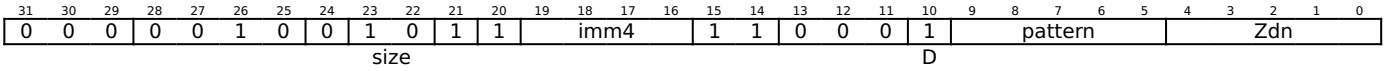
```
DECH <Zdn>.H{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Word

(FEAT_SVE || FEAT_SME)



Encoding

```
DECW <Zdn>.S{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    result[e*:esize] = operand1[e*:esize] - (count * imm);

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

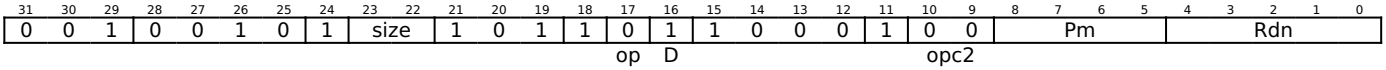
DECP (scalar)

Decrement scalar by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to decrement the scalar destination.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
DECP <Xdn>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Rdn);
```

Assembler Symbols

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(64) = X{}(dn);
let operand2 : bits(PL) = P{}(m);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

X{64}(dn) = operand1 - count;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DECP (vector)

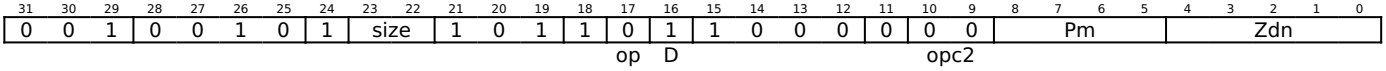
Decrement vector by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to decrement all destination vector elements.

The predicate size specifier may be omitted in assembler source code, but this is deprecated and will be prohibited in a future release of the architecture.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
DECP <Zdn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(PL) = P{}(m);
var result : bits(VL);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

for e = 0 to elements-1 do
    result[e*:esize] = operand1[e*:esize] - count;

end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DUP (immediate)

Broadcast signed immediate to vector elements (unpredicated)

This instruction unconditionally broadcasts the signed integer immediate into each element of the destination vector. This instruction is unpredicated.

The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

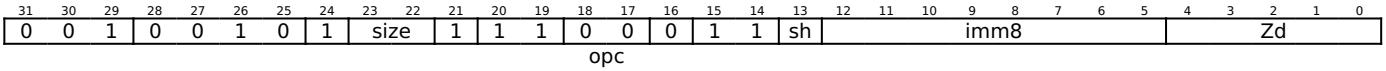
The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This instruction is used by the alias MOV (immediate, unpredicated).

This instruction is used by the pseudo-instruction FMOV (zero, unpredicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
DUP <Zd>.<T>, #<imm>{, <shift>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt(Zd);
var imm : integer = SInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Alias Conditions

Alias	Is preferred when
FMOV (zero, unpredicated)	Never
MOV (immediate, unpredicated)	Unconditionally

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let result : bits(VL) = Replicate{}(imm[esize-1:0]);
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DUP (indexed)

Broadcast indexed element to vector (unpredicated)

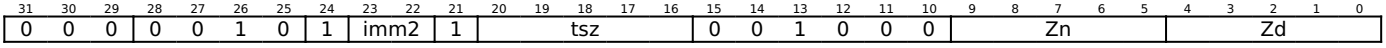
This instruction unconditionally broadcasts the indexed source vector element into each element of the destination vector. This instruction is unpredicated.

The immediate element index is in the range of 0 to 63 (bytes), 31 (halfwords), 15 (words), 7 (doublewords) or 3 (quadwords). Selecting an element beyond the accessible vector length causes the destination vector to be set to zero.

This instruction is used by the alias MOV (SIMD&FP scalar, unpredicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
DUP <Zd>.<T>, <Zn>.<T>[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if tsz == '00000' then EndOfDecode(Decode_UNDEF); end;
let lsb : integer{} = LowestSetBit(tsz);
let esize : integer{} = 8 << lsb;
let imm : bits(7) = imm2::tsz;
let index : integer = UInt(imm[6:(lsb+1)]);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsz”:

tsz	<T>
00000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D
10000	Q

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<imm> Is the immediate index, in the range 0 to one less than the number of elements in 512 bits, encoded in "imm2:tsz".

Alias Conditions

Alias	Is preferred when
MOV (SIMD&FP scalar, unpredicated)	`BitCount(imm2 :: tsz) == 1`
MOV (SIMD&FP scalar, unpredicated)	`BitCount(imm2 :: tsz) > 1`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = if index < elements then Z{VL}(n) else Zeros{VL};
var result : bits(VL);
var element : bits(esize);

if index >= elements then
    element = Zeros{esize};
else
    element = operand1[index*:esize];
end;
result = Replicate{VL}(element);

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DUP (scalar)

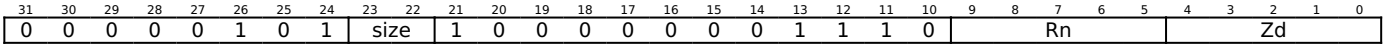
Broadcast general-purpose register to vector elements (unpredicated)

This instruction unconditionally broadcasts the general-purpose scalar source register into each element of the destination vector. This instruction is unpredicated.

This instruction is used by the alias MOV (scalar, unpredicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding for the 32-bit variant

Applies when `(size != 11)`

```
DUP <Zd>.<T>, <Wn|WSP>
```

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
DUP <Zd>.D, <Xn|SP>
```

Decode for all variants of this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Rn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S

<Wn|WSP> Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

<Xn|SP> Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var operand : bits(64);
if n == 31 then
    operand = SP{64}();
else
    operand = X{64}(n);
end;
var result : bits(VL);

for e = 0 to elements-1 do
    result[e*:esize] = operand[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DUPM

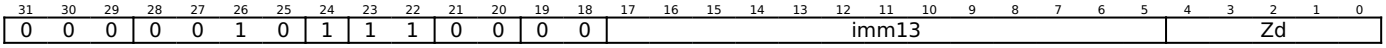
Broadcast logical bitmask immediate to vector (unpredicated)

This instruction unconditionally broadcasts the logical bitmask immediate into each element of the destination vector. This instruction is unpredicated. The immediate is a 64-bit value consisting of a single run of ones or zeros repeating every 2, 4, 8, 16, 32, or 64 bits.

This instruction is used by the alias MOV.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
DUPM <Zd>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let d : integer = UInt(Zd);
var imm : bits(esize);
(imm, -) = DecodeBitMasks{esize}(imm13[12], imm13[5:0], imm13[11:6], TRUE);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “imm13”:

imm13	<T>
0xxxxxx0xxxxx	S
0xxxxxx10xxxx	H
0xxxxxx110xxx	B
0xxxxxx1110xx	B
0xxxxxx11110x	B
0xxxxxx11111x	RESERVED
1xxxxxxxxxxxx	D

<const> Is a 64, 32, 16, or 8-bit bitmask consisting of replicated 2, 4, 8, 16, 32, or 64-bit fields, each field containing a rotated run of nonzero bits, encoded in the "imm13" field.

Alias Conditions

Alias	Is preferred when
MOV	`SVEMoveMaskPreferred(imm13)`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let result : bits(VL) = Replicate{}(imm);
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

DUPQ

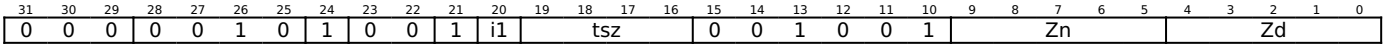
Broadcast indexed element within each quadword vector segment (unpredicated)

This instruction unconditionally broadcasts the indexed element within each 128-bit source vector segment to all elements of the corresponding destination vector segment. This instruction is unpredicated.

The immediate element index is in the range of 0 to 15 (bytes), 7 (halfwords), 3 (words) or 1 (doublewords).

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
DUPQ <Zd>.<T>, <Zn>.<T>[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
if tsz == '0000' then EndOfDecode(Decode_UNDEF); end;
let lsb : integer{} = LowestSetBit(tsz);
let esize : integer{} = 8 << lsb;
let imm : bits(5) = i1::tsz;
let index : integer = UInt(imm[4:(lsb+1)]);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsz”:

tsz	<T>
0000	RESERVED
xxx1	B
xx10	H
x100	S
1000	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<imm> Is the immediate index, in the range 0 to one less than the number of elements in 128 bits, encoded in "i1:tsz".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let elements : integer = 128 DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
var element : bits(esize);

for s = 0 to segments-1 do
    element = operand[(s * elements + index)*:esize];
    result[s*:128] = Replicate{128}(element);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

EON

Bitwise exclusive-OR with inverted immediate (unpredicated)

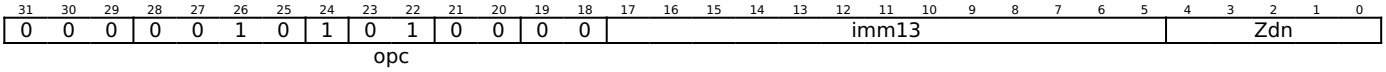
This instruction performs a bitwise exclusive-OR between an inverted immediate and each 64-bit element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate is a 64-bit value consisting of a single run of ones or zeros repeating every 2, 4, 8, 16, 32, or 64 bits. This instruction is unpredicated.

This is a pseudo-instruction of EOR (immediate). This means:

- The encodings in this description are named to match the encodings of EOR (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of EOR (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
EON <Zdn>.<T>, <Zdn>.<T>, #<const>
```

is equivalent to

```
EOR <Zdn>.<T>, <Zdn>.<T>, #(-<const> - 1)
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “imm13”:

imm13	<T>
0xxxxxx0xxxxx	S
0xxxxxx10xxxx	H
0xxxxxx110xxx	B
0xxxxxx1110xx	B
0xxxxxx11110x	B
0xxxxxx11111x	RESERVED
1xxxxxxxxxxxx	D

<const> Is a 64, 32, 16, or 8-bit bitmask consisting of replicated 2, 4, 8, 16, 32, or 64-bit fields, each field containing a rotated run of nonzero bits, encoded in the "imm13" field.

Operation

The description of EOR (immediate) gives the operational pseudocode for this instruction.

Operational Information

The description of EOR (immediate) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

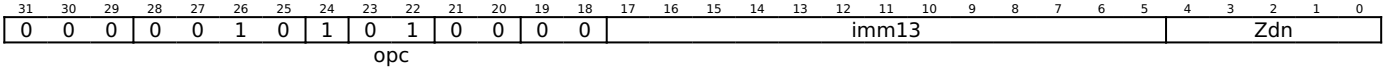
EOR (immediate)

Bitwise exclusive-OR with immediate (unpredicated)

This instruction performs a bitwise exclusive-OR between an immediate and each 64-bit element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate is a 64-bit value consisting of a single run of ones or zeros repeating every 2, 4, 8, 16, 32, or 64 bits. This instruction is unpredicated.
This instruction is used by the pseudo-instruction EON.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
EOR <Zdn>.<T>, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn);
var imm : bits(64);
(imm, -) = DecodeBitMasks{64}(imm13[12], imm13[5:0], imm13[11:6], TRUE);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “imm13”:

imm13	<T>
0xxxxxx0xxxxx	S
0xxxxxx10xxxx	H
0xxxxxx110xxx	B
0xxxxxx1110xx	B
0xxxxxx11110x	B
0xxxxxx11111x	RESERVED
1xxxxxxxxxxxx	D

<const> Is a 64, 32, 16, or 8-bit bitmask consisting of replicated 2, 4, 8, 16, 32, or 64-bit fields, each field containing a rotated run of nonzero bits, encoded in the "imm13" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 64;
let operand : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(64) = operand[e*:64];
    result[e*:64] = element1 XOR imm;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

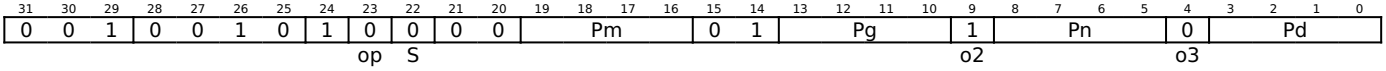
EOR (predicates)

Bitwise exclusive-OR predicates

This instruction performs a bitwise exclusive-OR between the Active elements of the first source predicate and the corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.
This instruction is used by the alias NOT (predicate).

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
EOR <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Alias Conditions

Alias	Is preferred when
NOT (predicate)	`Pm == Pg`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1 XOR element2);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

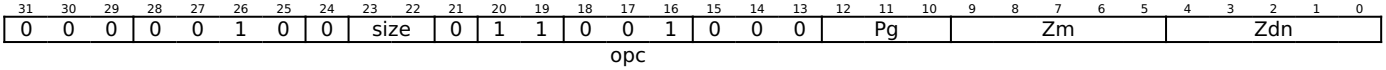
EOR (vectors, predicated)

Bitwise exclusive-OR (predicated)

This instruction performs a bitwise exclusive-OR between the Active elements of the first source vector and the corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
EOR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = element1 XOR element2;
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

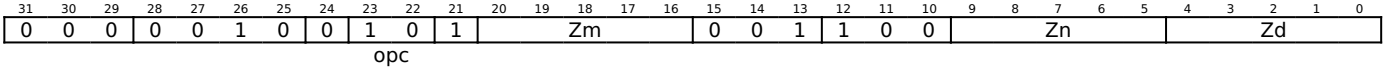
EOR (vectors, unpredicated)

Bitwise exclusive-OR (unpredicated)

This instruction performs a bitwise exclusive-OR between the elements of the first source vector and the corresponding elements of the second source vector, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
EOR <Zd>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);

Z{VL}(d) = operand1 XOR operand2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

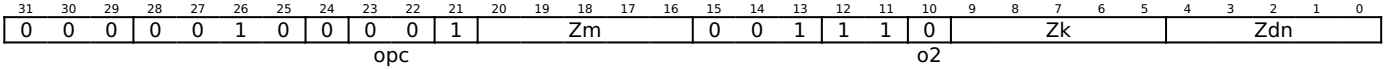
EOR3

Bitwise exclusive-OR between three vectors

This instruction performs a bitwise exclusive-OR between the corresponding elements of the three source vectors, and destructively places the results in the corresponding elements of the destination and first source vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
EOR3 <Zdn>.D, <Zdn>.D, <Zm>.D, <Zk>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let m : integer = UInt(Zm);
let k : integer = UInt(Zk);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the third source scalable vector register, encoded in the "Zk" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(k);

Z{VL}(dn) = operand1 XOR operand2 XOR operand3;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

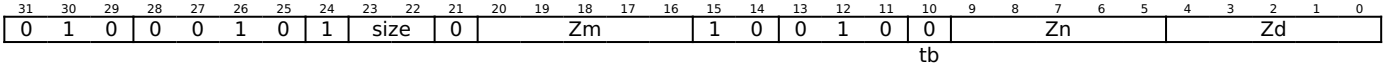
EORBT

Interleaving exclusive-OR (bottom, top)

This instruction performs an interleaving exclusive-OR between the even-numbered elements of the first source vector register and the odd-numbered elements of the second source vector register, placing the result in the even-numbered elements of the destination vector, leaving the odd-numbered elements unchanged. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
EORBT <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 0;
let sel2 : integer = 1;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[(2*e + sel1)*:esize];
    let element2 : bits(esize) = operand2[(2*e + sel2)*:esize];
    result[(2*e + sel1)*:esize] = element1 XOR element2;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

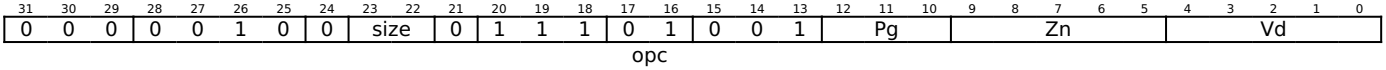
EORQV

Bitwise exclusive-OR reduction of quadword vector segments

This instruction performs a bitwise exclusive-OR between elements in the same position from each 128-bit source vector segment, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as all zeros.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
EORQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	16B
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(128) = Zeros{};
var stmp : bits(128) = Zeros{};

var dtmp : bits(esize);

for e = 0 to elempersegment-1 do
    dtmp = Zeros{esize};
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp = operand[s*:128];
            dtmp = dtmp XOR stmp[e*:esize];
        end;
    end;
    result[e*:esize] = dtmp[esize-1:0];
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

EORS

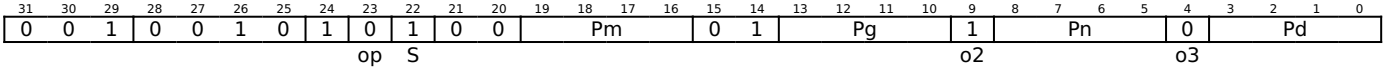
Bitwise exclusive-OR predicates, setting the condition flags

This instruction performs a bitwise exclusive-OR between the Active elements of the first source predicate and the corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This instruction is used by the alias NOTS.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
EORS <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Alias Conditions

Alias	Is preferred when
NOTS	`Pm == Pg`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1 XOR element2);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

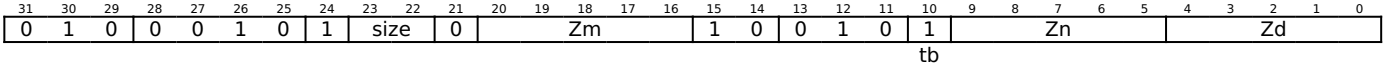
EORTB

Interleaving exclusive-OR (top, bottom)

This instruction performs an interleaving exclusive-OR between the odd-numbered elements of the first source vector register and the even-numbered elements of the second source vector register, placing the result in the odd-numbered elements of the destination vector, leaving the even-numbered elements unchanged. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
EORTB <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 1;
let sel2 : integer = 0;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[(2*e + sel1)*:esize];
    let element2 : bits(esize) = operand2[(2*e + sel2)*:esize];
    result[(2*e + sel1)*:esize] = element1 XOR element2;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

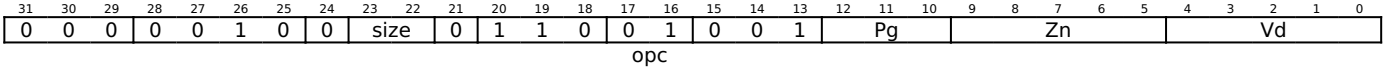
EORV

Bitwise exclusive-OR reduction to scalar

This instruction performs a bitwise exclusive-OR between the elements of the source vector, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
EORV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(esize) = Zeros{esize};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result = result XOR operand[e*:esize];

    end;
end;
V{esize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

EXPAND

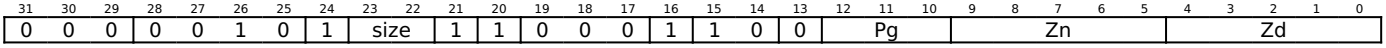
Copy lower-numbered vector elements to Active elements

This instruction copies consecutive elements from the source vector to Active elements of the destination vector, in increasing order of element number. Inactive elements in the destination vector register are set to zero.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled, or FEAT_SME2p2 is implemented.

SVE2

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
EXPAND <Zd>.<T>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2p2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL);
var x : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand1[x*:esize];
        x = x + 1;
    else
        result[e*:esize] = Zeros{esize};
    end;
end;

Z{VL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

EXT

Extract vector from pair of vectors

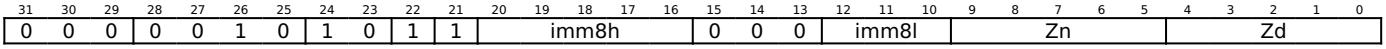
This instruction copies the indexed byte up to the last byte of the first source vector to the bottom of the result vector, then fills the remainder of the result starting from the first byte of the second source vector. The result is destructively placed in the destination and first source vector, or constructively placed in the destination vector. This instruction is unpredicated.

An index that is greater than or equal to the vector length in bytes is treated as zero, resulting in the first source vector being copied to the result unchanged.

It has encodings from 2 classes: Constructive and Destructive

Constructive

(FEAT_SVE2 || FEAT_SME)



Encoding

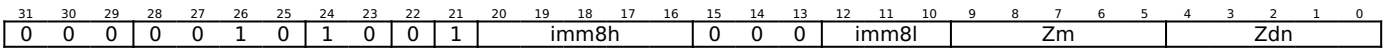
```
EXT <Zd>.B, { <Zn1>.B, <Zn2>.B }, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dst : integer = UInt(Zd);
let s1 : integer = UInt(Zn);
let s2 : integer = (s1 + 1) MOD 32;
let position : integer{} = UInt(imm8h::imm8l) * 8;
```

Destructive

(FEAT_SVE || FEAT_SME)



Encoding

```
EXT <Zdn>.B, <Zdn>.B, <Zm>.B, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dst : integer = UInt(Zdn);
let s1 : integer = dst;
let s2 : integer = UInt(Zm);
let position : integer{} = UInt(imm8h::imm8l) * 8;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded in the "Zn" field.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded in the "Zn" field.
<imm>	Is the unsigned immediate operand, in the range 0 to 255, encoded in the "imm8h:imm8l" fields.
<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(s1);
let operand2 : bits(VL) = Z{}(s2);
var result : bits(VL);

let concat : bits(VL*2) = operand2 :: operand1;

if position >= VL then
    result = concat[VL-1:0];
else
    result = concat[(position+VL)-1:position];
end;

Z{VL}(dst) = result;
```

Operational information

For the "Constructive" variant:

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

For the "Destructive" variant:

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The destructive variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the destructive variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

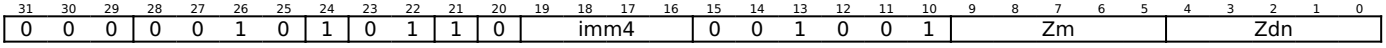
EXTQ

Extract vector segment from each pair of quadword vector segments

This instruction copies, for each 128-bit vector segment of the result, the indexed byte up to and including the last byte of the corresponding first source vector segment to the bottom of the result segment, and then fills the remainder of the result segment starting from the first byte of the corresponding second source vector segment. The result segments are destructively placed in the corresponding first source vector segment. This instruction is unpredicated.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
EXTQ <Zdn>.B, <Zdn>.B, <Zm>.B, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let position : integer{} = UInt(imm4) << 3;
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<imm>	Is the unsigned immediate operand, in the range 0 to 15, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for s = 0 to segments-1 do
    let concat : bits(256) = operand2[s*:128] :: operand1[s*:128];
    result[s*:128] = concat[position+127:position];
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

F1CVT, F2CVT

8-bit floating-point convert to half-precision

This instruction converts each even-numbered 8-bit floating-point element of the source vector to half-precision while downscaling the value, and places the results in the overlapping 16-bit elements of the destination vector. F1CVT scales the values by $2^{-UInt(FPMR.LSCALE[3:0])}$. F2CVT scales the values by $2^{-UInt(FPMR.LSCALE2[3:0])}$.

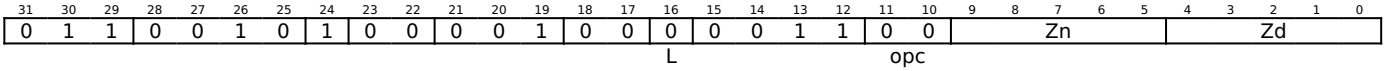
The 8-bit floating-point encoding format for F1CVT is selected by FPMR.F8S1. The 8-bit floating-point encoding format for F2CVT is selected by FPMR.F8S2.

This instruction is unpredicated.

It has encodings from 2 classes: F1CVT and F2CVT

F1CVT

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

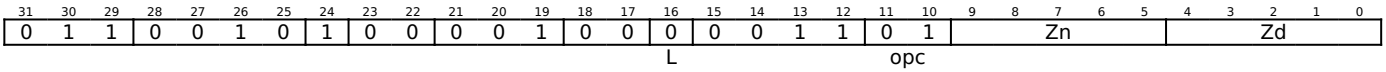
```
F1CVT <Zd>.H, <Zn>.B
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_FP8)) then EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 8;
let d_esize : integer{} = 16;
let issrc2 : boolean = FALSE;
```

F2CVT

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

```
F2CVT <Zd>.H, <Zn>.B
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_FP8)) then EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 8;
let d_esize : integer{} = 16;
let issrc2 : boolean = TRUE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[e*:esize];
    let res : bits(d_esize) = FP8ConvertFP(element[s_esize-1:0], issrc2, FPCR(), FPMR());
    result[e*:esize] = ZeroExtend{esize}(res);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

F1CVTLT, F2CVTLT

8-bit floating-point convert to half-precision (top)

This instruction converts each odd-numbered 8-bit floating-point element of the source vector to half-precision while downscaling the value, and places the results in the overlapping 16-bit elements of the destination vector. F1CVTLT scales the values by $2^{-UInt(FPMR.LSCALE[3:0])}$. F2CVTLT scales the values by $2^{-UInt(FPMR.LSCALE2[3:0])}$.

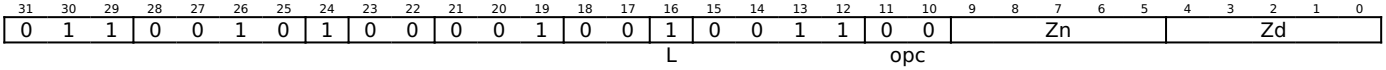
The 8-bit floating-point encoding format for F1CVTLT is selected by FPMR.F8S1. The 8-bit floating-point encoding format for F2CVTLT is selected by FPMR.F8S2.

This instruction is unpredicated.

It has encodings from 2 classes: F1CVTLT and F2CVTLT

F1CVTLT

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

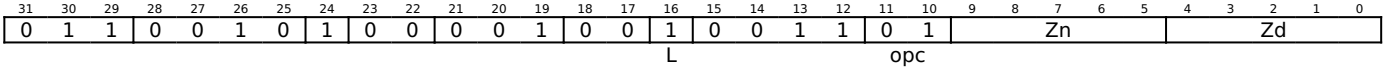
```
F1CVTLT <Zd>.H, <Zn>.B
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_FP8)) then EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let issrc2 : boolean = FALSE;
```

F2CVTLT

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

```
F2CVTLT <Zd>.H, <Zn>.B
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_FP8)) then EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let issrc2 : boolean = TRUE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
for e = 0 to elements-1 do
    let element : bits(esize DIV 2) = operand[(2*e + 1)*:(esize DIV 2)];
    result[e*:esize] = FP8ConvertFP(element, issrc2, FPCR(), FPMR());

end;
Z{VL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

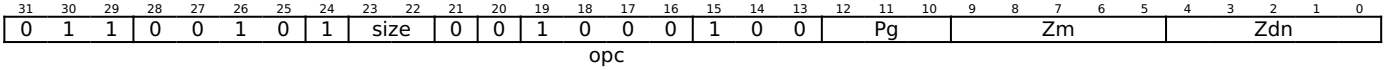
FABD

Floating-point absolute difference (predicated)

This instruction calculates the absolute difference of Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the result in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FABD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPAbs{esize}(FPSub{esize}(element1, element2, FPCR()), FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FABS

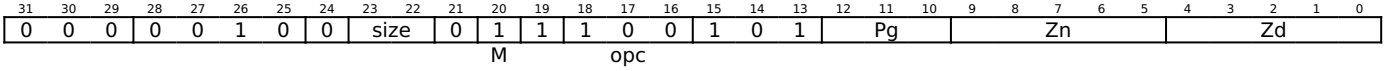
Floating-point absolute value (predicated)

This instruction takes the absolute value of each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. This clears the sign bit and cannot signal a floating-point exception. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

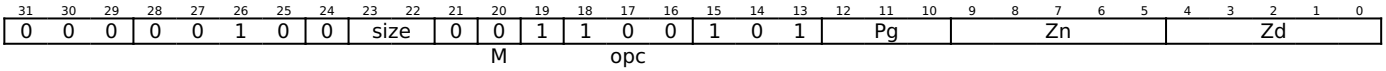
```
FABS <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FABS <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = FPAbs{esize}(element, FPCR());

    end;
end;
Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FAC<cc>

Floating-point absolute compare

This instruction compares Active absolute values of floating-point elements in the first source vector with the corresponding absolute values of elements in the second source vector, and places the boolean results of the specified comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

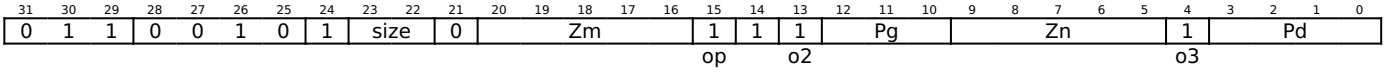
<cc>	Comparison
GE	greater than or equal
GT	greater than
LE	less than or equal
LT	less than

This instruction is used by the pseudo-instructions FACLE, and FACLT.

It has encodings from 2 classes: Greater than and Greater than or equal

Greater than

(FEAT_SVE || FEAT_SME)



Encoding

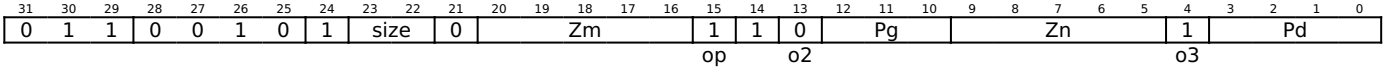
```
FACGT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GT;
```

Greater than or equal

(FEAT_SVE || FEAT_SME)



Encoding

```
FACGE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = FPAbs{}(operand1[e*:esize], FPCR());
        let element2 : bits(esize) = FPAbs{}(operand2[e*:esize], FPCR());
        var res : boolean;
        case cmp_op of
            when Cmp_GE => res = FPCompareGE{esize}(element1, element2, FPCR());
            when Cmp_GT => res = FPCompareGT{esize}(element1, element2, FPCR());
        end;
        let pbit : bit = if res then '1' else '0';
        result[e*:psize] = ZeroExtend{psize}(pbit);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

P{PL}(d) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the predicate register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FACLE

Floating-point absolute compare less than or equal

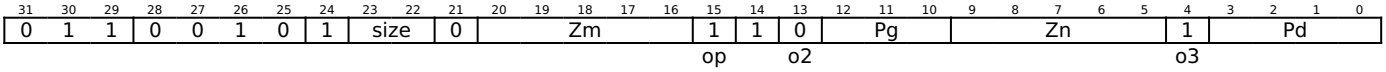
This instruction compares Active absolute values of floating-point elements in the first source vector being less than or equal to the corresponding absolute values of elements in the second source vector, and places the boolean results of the comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

This is a pseudo-instruction of FAC<cc>. This means:

- The encodings in this description are named to match the encodings of FAC<cc>.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of FAC<cc> gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Greater than or equal

(FEAT_SVE || FEAT_SME)



Encoding

```
FACLE <Pd>.<T>, <Pg>/Z, <Zm>.<T>, <Zn>.<T>
```

is equivalent to

```
FACGE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

Operation

The description of FAC<cc> gives the operational pseudocode for this instruction.

Operational Information

The description of FAC<cc> gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FACLT

Floating-point absolute compare less than

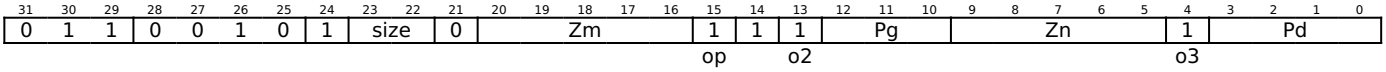
This instruction compares Active absolute values of floating-point elements in the first source vector being less than the corresponding absolute values of elements in the second source vector, and places the boolean results of the comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

This is a pseudo-instruction of FAC<cc>. This means:

- The encodings in this description are named to match the encodings of FAC<cc>.
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of FAC<cc> gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Greater than

(FEAT_SVE || FEAT_SME)



Encoding

```
FACLT <Pd>.<T>, <Pg>/Z, <Zm>.<T>, <Zn>.<T>
```

is equivalent to

```
FACGT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

Operation

The description of FAC<cc> gives the operational pseudocode for this instruction.

Operational Information

The description of FAC<cc> gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

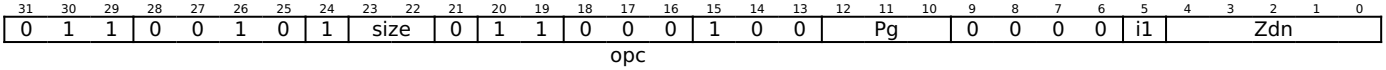
FADD (immediate)

Floating-point add immediate (predicated)

This instruction adds an immediate to each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.5 or +1.0 only. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then FPPointFive{esize}('0') else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.5
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPAdd{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

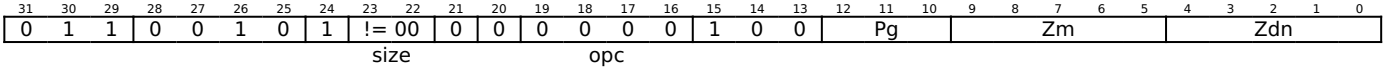
FADD (vectors, predicated)

Floating-point add (predicated)

This instruction adds Active floating-point elements of the first source vector to the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPAdd{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

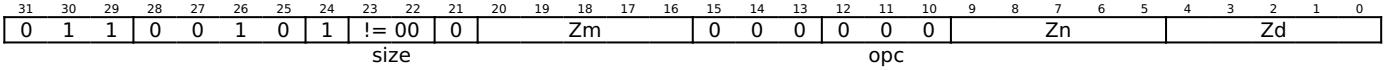
FADD (vectors, unpredicated)

Floating-point add (unpredicated)

This instruction adds all floating-point elements of the first source vector to the corresponding elements of the second source vector, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FADD <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPAdd{esize}(element1, element2, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FADDA

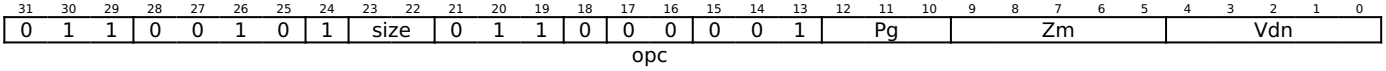
Floating-point add strictly-ordered reduction, accumulating in scalar

This instruction adds the floating-point values in a SIMD&FP scalar source and all active lanes of the vector source and destructively places the result in the SIMD&FP scalar source register. Vector elements are processed strictly in order from low to high, with the scalar source providing the initial value. Inactive elements in the source vector are ignored.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
FADDA <V><dn>, <Pg>, <V><dn>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Vdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	D

<dn> Is the number [0-31] of the source and destination SIMD&FP register, encoded in the "Vdn" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the source scalable vector register, encoded in the "Zm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(esize) = V{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(esize) = operand1;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand2[e*:esize];
        result = FPAdd{esize}(result, element, FPCR());
    end;
end;

V{esize}(dn) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

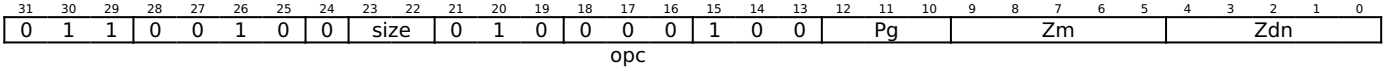
FADDP

Floating-point add pairwise

This instruction adds pairs of adjacent floating-point elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FADDP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL) = Z{}(dn);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if IsEven(e) then
            element1 = operand1[(e + 0)*:esize];
            element2 = operand1[(e + 1)*:esize];
        else
            element1 = operand2[(e - 1)*:esize];
            element2 = operand2[(e + 0)*:esize];
        end;
        result[e*:esize] = FPAdd{esize}(element1, element2, FPCR());
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

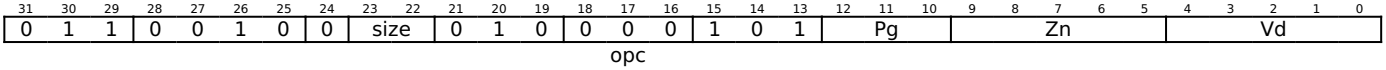
FADDQV

Floating-point add recursive reduction of quadword vector segments

This instruction performs a floating-point addition of the same element numbers from each 128-bit source vector segment using a recursive pairwise reduction, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as +0.0.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
FADDQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	RESERVED
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer{} = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let segbits : integer{} = segments*esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPZero{esize}('0');
var result : bits(128) = Zeros{};

for e = 0 to elempersegment-1 do
    var stmp : bits(segbits);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp[s*:esize] = operand[(s * elempersegment + e)*:esize];
        else
            stmp[s*:esize] = identity;
        end;
    end;
    result[e*:esize] = FPReduce{esize, segbits}(ReduceOp_FADD, stmp, FPCR());
end;
V{128}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

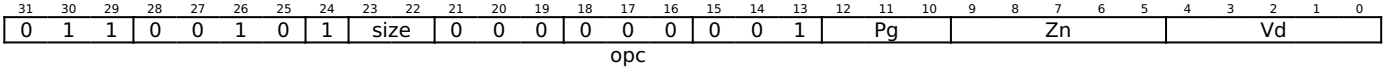
FADDV

Floating-point add recursive reduction to scalar

This instruction adds horizontally the floating-point values over all lanes of a vector using a recursive pairwise reduction, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as +0.0.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FADDV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPZero{esize}('0');

V{esize}(d) = FPReducePredicated{esize, VL, PL}(ReduceOp_FADD, operand, mask,
                                                identity, FPCR());
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FAMAX

Floating-point absolute maximum (predicated)

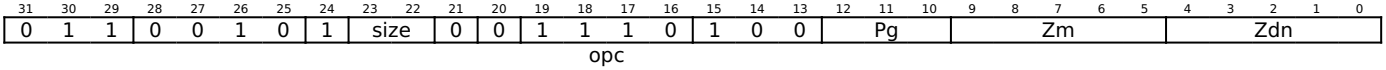
This instruction determines the maximum absolute value from Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.
The behavior is as follows:

- Alternate floating-point behaviors do not apply, as if FPCR.AH is 0.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

Inactive elements in the destination vector register remain unmodified.

SVE2

((FEAT_SVE2 || FEAT_SME2) && FEAT_FAMINMAX)



Encoding

```
FAMAX <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_FAMINMAX)) then EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPAbsMax{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FAMIN

Floating-point absolute minimum (predicated)

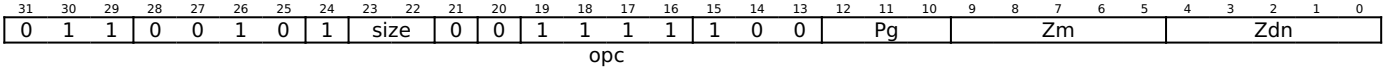
This instruction determines the minimum absolute value from Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.
The behavior is as follows:

- Alternate floating-point behaviors do not apply, as if FPCR.AH is 0.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

Inactive elements in the destination vector register remain unmodified.

SVE2

((FEAT_SVE2 || FEAT_SME2) && FEAT_FAMINMAX)



Encoding

```
FAMIN <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_FAMINMAX)) then EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPAbsMin{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCADD

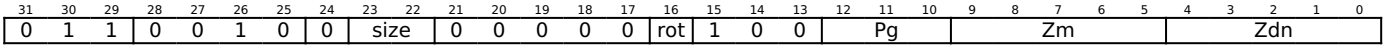
Floating-point complex add (predicated)

This instruction adds the real and imaginary components of the Active floating-point complex numbers from the first source vector to the complex numbers from the second source vector that have first been rotated by 90 or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation, equivalent to multiplying the complex numbers in the second source vector by ±j beforehand. The results are destructively placed in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FCADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let sub_i : boolean = (rot == '0');
let sub_r : boolean = (rot == '1');
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
0	#90
1	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let pairs : integer = VL DIV (2 * esize);
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for p = 0 to pairs-1 do
    var acc_r : bits(esize) = operand1[(2 * p + 0)*:esize];
    var acc_i : bits(esize) = operand1[(2 * p + 1)*:esize];
    if ActivePredicateElement{PL}(mask, 2 * p + 0, esize) then
        var elt2_i : bits(esize) = operand2[(2 * p + 1)*:esize];
        if sub_i then elt2_i = FPNeg{esize}(elt2_i, FPCR()); end;
        acc_r = FPAdd{esize}(acc_r, elt2_i, FPCR());
    end;
    if ActivePredicateElement{PL}(mask, 2 * p + 1, esize) then
        var elt2_r : bits(esize) = operand2[(2 * p + 0)*:esize];
        if sub_r then elt2_r = FPNeg{esize}(elt2_r, FPCR()); end;
        acc_i = FPAdd{esize}(acc_i, elt2_r, FPCR());
    end;
    result[(2 * p + 0)*:esize] = acc_r;
    result[(2 * p + 1)*:esize] = acc_i;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCLAMP

Floating-point clamp to minimum/maximum number

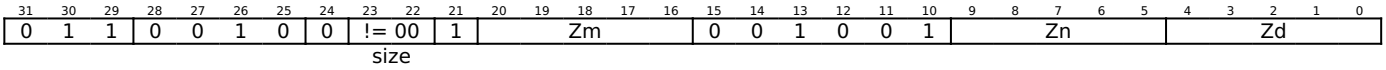
This instruction clamps each floating-point element in the destination vector to between the floating-point minimum value in the corresponding element of the first source vector and the floating-point maximum value in the corresponding element of the second source vector and destructively places the clamped results in the corresponding elements of the destination vector.
The behavior is as follows for each minimum number and maximum number operation:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
FCLAMP <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var result : bits(VL);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    let element3 : bits(esize) = operand3[e*:esize];
    let maxelement : bits(esize) = FPMaxNum{}(element1, element3, FPCR());
    result[e*:esize] = FPMinNum{esize}(maxelement, element2, FPCR());
end;
Z{VL}(d) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCM<cc> (vectors)

Floating-point compare

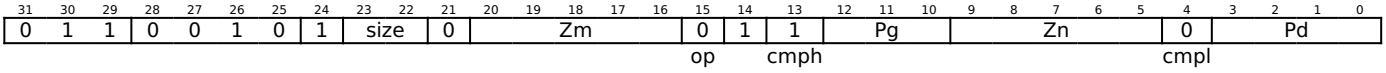
This instruction compares Active floating-point elements in the first source vector with the corresponding elements in the second source vector, and places the boolean results of the specified comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

<cc>	Comparison
EQ	equal
GE	greater than or equal
GT	greater than
NE	not equal
UO	unordered

This instruction is used by the pseudo-instructions FCMLE (vectors), and FCMLT (vectors).
It has encodings from 5 classes: Equal , Greater than , Greater than or equal , Not equal and Unordered

Equal

(FEAT_SVE || FEAT_SME)



Encoding

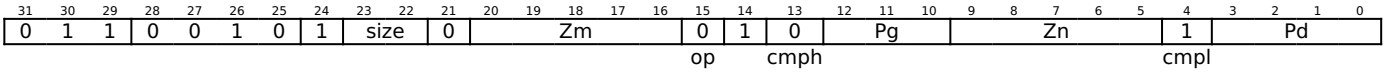
```
FCMEQ <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_EQ;
```

Greater than

(FEAT_SVE || FEAT_SME)



Encoding

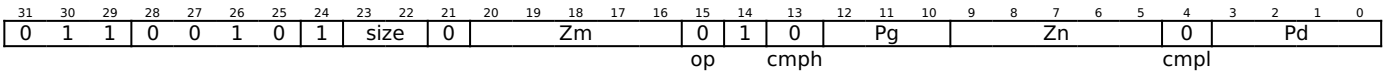
```
FCMGT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GT;
```

Greater than or equal

(FEAT_SVE || FEAT_SME)



Encoding

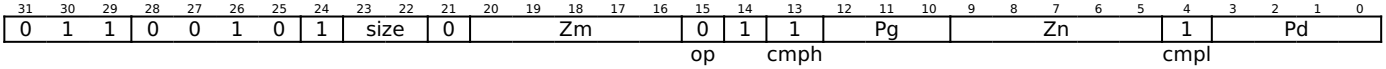
```
FCMGE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_GE;
```

Not equal

(FEAT_SVE || FEAT_SME)



Encoding

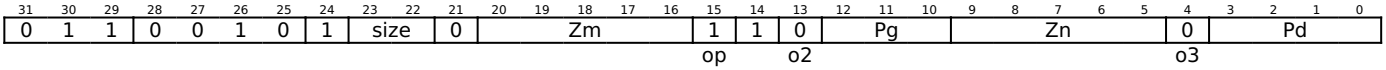
```
FCMNE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_NE;
```

Unordered

(FEAT_SVE || FEAT_SME)



Encoding

```
FCMUO <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Pd);
let cmp_op : CmpOp = Cmp_UN;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        var res : boolean;
        case cmp_op of
            when Cmp_EQ => res = FPCompareEQ{esize}(element1, element2, FPCR());
            when Cmp_GE => res = FPCompareGE{esize}(element1, element2, FPCR());
            when Cmp_GT => res = FPCompareGT{esize}(element1, element2, FPCR());
            when Cmp_UN => res = FPCompareUN{esize}(element1, element2, FPCR());
            when Cmp_NE => res = FPCompareNE{esize}(element1, element2, FPCR());
            when Cmp_LT => res = FPCompareGT{esize}(element2, element1, FPCR());
            when Cmp_LE => res = FPCompareGE{esize}(element2, element1, FPCR());
        end;
        let pbit : bit = if res then '1' else '0';
        result[e*:psize] = ZeroExtend{psize}(pbit);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

P{PL}(d) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the predicate register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCM<cc> (zero)

Floating-point compare with zero

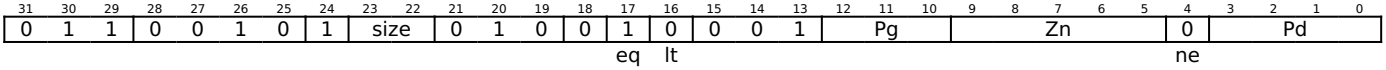
This instruction compares Active floating-point elements in the source vector with zero, and places the boolean results of the specified comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

<cc>	Comparison
EQ	equal
GE	greater than or equal
GT	greater than
LE	less than or equal
LT	less than
NE	not equal
UO	unordered

It has encodings from 6 classes: Equal , Greater than , Greater than or equal , Less than , Less than or equal and Not equal

Equal

(FEAT_SVE || FEAT_SME)



Encoding

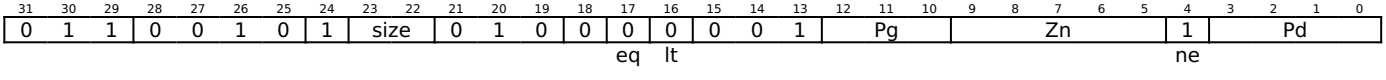
```
FCMEQ <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let op : CmpOp = Cmp_EQ;
```

Greater than

(FEAT_SVE || FEAT_SME)



Encoding

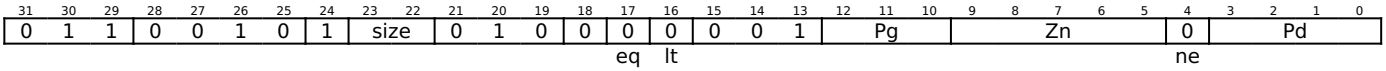
```
FCMGT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let op : CmpOp = Cmp_GT;
```

Greater than or equal

(FEAT_SVE || FEAT_SME)



Encoding

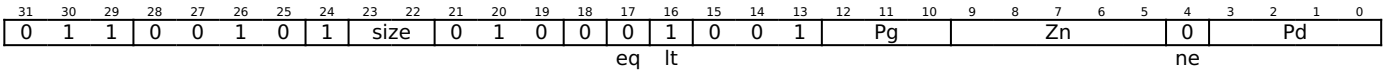
```
FCMGE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let op : CmpOp = Cmp_GE;
```

Less than

(FEAT_SVE || FEAT_SME)



Encoding

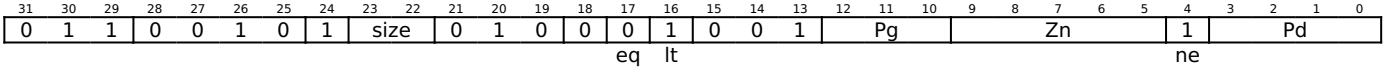
```
FCMLT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let op : CmpOp = Cmp_LT;
```

Less than or equal

(FEAT_SVE || FEAT_SME)



Encoding

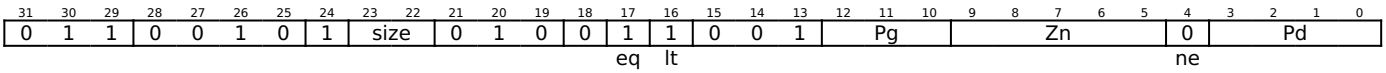
```
FCMLE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let op : CmpOp = Cmp_LE;
```

Not equal

(FEAT_SVE || FEAT_SME)



Encoding

```
FCMNE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, #0.0
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let op : CmpOp = Cmp_NE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        var res : boolean;
        case op of
            when Cmp_EQ => res = FPCompareEQ{esize}(element, 0[esize-1:0], FPCR());
            when Cmp_GE => res = FPCompareGE{esize}(element, 0[esize-1:0], FPCR());
            when Cmp_GT => res = FPCompareGT{esize}(element, 0[esize-1:0], FPCR());
            when Cmp_NE => res = FPCompareNE{esize}(element, 0[esize-1:0], FPCR());
            when Cmp_LT => res = FPCompareGT{esize}(0[esize-1:0], element, FPCR());
            when Cmp_LE => res = FPCompareGE{esize}(0[esize-1:0], element, FPCR());
        end;
        let pbit : bit = if res then '1' else '0';
        result[e*:psize] = ZeroExtend{psize}(pbit);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

P{PL}(d) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the predicate register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMLA (indexed)

Floating-point complex multiply-add by indexed element

This instruction multiplies the duplicated real components for rotations 0 and 180, or imaginary components for rotations 90 and 270, of the floating-point complex numbers in each 128-bit segment of the first source vector by the specified complex number in the corresponding the second source vector segment rotated by 0, 90, 180, or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation.

This instruction then destructively adds the products to the corresponding components of the complex numbers in the addend and destination vector, without intermediate rounding.

These transformations permit the creation of a variety of multiply-add and multiply-subtract operations on complex numbers by combining two of these instructions with the same vector operands but with rotations that are 90 degrees apart.

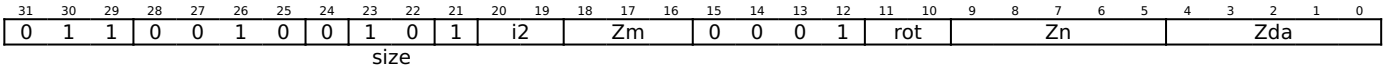
Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element.

The complex numbers within the second source vector are specified using an immediate index that selects the same complex number position within each 128-bit vector segment. The index range is from 0 to one less than the number of complex numbers per 128-bit segment. This instruction is unpredicated.

It has encodings from 2 classes: Half-precision and Single-precision

Half-precision

(FEAT_SVE || FEAT_SME)



Encoding

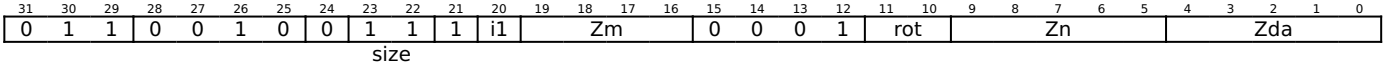
```
FCMLA <Zda>.H, <Zn>.H, <Zm>.H[<imm>], <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT rot[0]);
let neg_i : boolean = (rot[1] == '1');
let neg_r : boolean = (rot[0] != rot[1]);
```

Single-precision

(FEAT_SVE || FEAT_SME)



Encoding

```
FCMLA <Zda>.S, <Zn>.S, <Zm>.S[<imm>], <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT rot[0]);
let neg_i : boolean = (rot[1] == '1');
let neg_r : boolean = (rot[0] != rot[1]);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> For the "Half-precision" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.

For the "Single-precision" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<imm> For the "Half-precision" variant: is the index of a Real and Imaginary pair, in the range 0 to 3, encoded in the "i2" field.

For the "Single-precision" variant: is the index of a Real and Imaginary pair, in the range 0 to 1, encoded in the "i1" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
00	#0
01	#90
10	#180
11	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (2 * esize);
let pairspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for p = 0 to pairs-1 do
    let segmentbase : integer = p - p MOD pairspersegment;
    let s : integer = segmentbase + index;
    var addend_r : bits(esize) = operand3[(2 * p + 0)*:esize];
    var addend_i : bits(esize) = operand3[(2 * p + 1)*:esize];
    let elt1_a : bits(esize) = operand1[(2 * p + sel_a)*:esize];
    var elt2_a : bits(esize) = operand2[(2 * s + sel_a)*:esize];
    var elt2_b : bits(esize) = operand2[(2 * s + sel_b)*:esize];
    if neg_r then elt2_a = FPNeg{esize}(elt2_a, FPCR()); end;
    if neg_i then elt2_b = FPNeg{esize}(elt2_b, FPCR()); end;
    addend_r = FPMulAdd{esize}(addend_r, elt1_a, elt2_a, FPCR());
    addend_i = FPMulAdd{esize}(addend_i, elt1_a, elt2_b, FPCR());
    result[(2 * p + 0)*:esize] = addend_r;
    result[(2 * p + 1)*:esize] = addend_i;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMLA (vectors)

Floating-point complex multiply-add (predicated)

This instruction multiplies the duplicated real components for rotations 0 and 180, or imaginary components for rotations 90 and 270, of the floating-point complex numbers in the first source vector by the corresponding complex number in the second source vector rotated by 0, 90, 180, or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation.

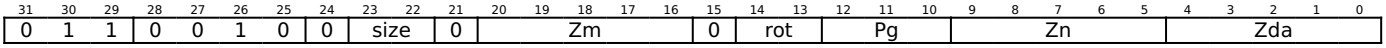
This instruction then destructively adds the products to the corresponding components of the complex numbers in the addend and destination vector, without intermediate rounding.

These transformations permit the creation of a variety of multiply-add and multiply-subtract operations on complex numbers by combining two of these instructions with the same vector operands but with rotations that are 90 degrees apart.

Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FCMLA <Zda>.<T>, <Pg>/M, <Zn>.<T>, <Zm>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT rot[0]);
let neg_i : boolean = (rot[1] == '1');
let neg_r : boolean = (rot[0] != rot[1]);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
00	#0
01	#90
10	#180
11	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let pairs : integer = VL DIV (2 * esize);
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for p = 0 to pairs-1 do
    var addend_r : bits(esize) = op3[(2 * p + 0)*:esize];
    var addend_i : bits(esize) = op3[(2 * p + 1)*:esize];
    if ActivePredicateElement{PL}(mask, 2 * p + 0, esize) then
        let elt1_a : bits(esize) = op1[(2*p + sel_a)*:esize];
        let elt2_a : bits(esize) =
            (if neg_r then
                 FPNeg{esize}(op2[(2 * p + sel_a)*:esize], FPCR())
             else
                 op2[(2 * p + sel_a)*:esize]);
        addend_r = FPMulAdd{esize}(addend_r, elt1_a, elt2_a, FPCR());
    end;
    if ActivePredicateElement{PL}(mask, 2 * p + 1, esize) then
        let elt1_a : bits(esize) = op1[(2 * p + sel_a)*:esize];
        let elt2_b : bits(esize) =
            (if neg_i then
                 FPNeg{esize}(op2[(2 * p + sel_b)*:esize], FPCR())
             else
                 op2[(2 * p + sel_b)*:esize]);
        addend_i = FPMulAdd{esize}(addend_i, elt1_a, elt2_b, FPCR());
    end;
    result[(2 * p + 0)*:esize] = addend_r;
    result[(2 * p + 1)*:esize] = addend_i;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMLE (vectors)

Floating-point compare less than or equal to vector

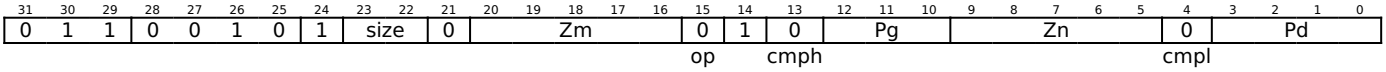
This instruction compares Active floating-point elements in the first source vector being less than or equal to the corresponding elements in the second source vector, and places the boolean results of the comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

This is a pseudo-instruction of FCM<cc> (vectors). This means:

- The encodings in this description are named to match the encodings of FCM<cc> (vectors).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of FCM<cc> (vectors) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Greater than or equal

(FEAT_SVE || FEAT_SME)



Encoding

```
FCMLE <Pd>.<T>, <Pg>/Z, <Zm>.<T>, <Zn>.<T>
```

is equivalent to

```
FCMGE <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

Operation

The description of FCM<cc> (vectors) gives the operational pseudocode for this instruction.

Operational Information

The description of FCM<cc> (vectors) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCMLT (vectors)

Floating-point compare less than vector

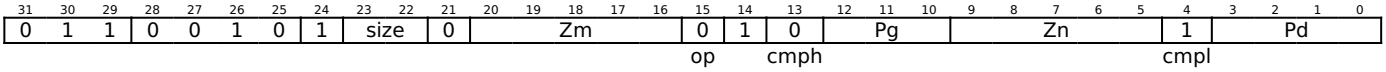
This instruction compares Active floating-point elements in the first source vector being less than the corresponding elements in the second source vector, and places the boolean results of the comparison in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

This is a pseudo-instruction of FCM<cc> (vectors). This means:

- The encodings in this description are named to match the encodings of FCM<cc> (vectors).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of FCM<cc> (vectors) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Greater than

(FEAT_SVE || FEAT_SME)



Encoding

```
FCMLT <Pd>.<T>, <Pg>/Z, <Zm>.<T>, <Zn>.<T>
```

is equivalent to

```
FCMGT <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

Operation

The description of FCM<cc> (vectors) gives the operational pseudocode for this instruction.

Operational Information

The description of FCM<cc> (vectors) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCPY

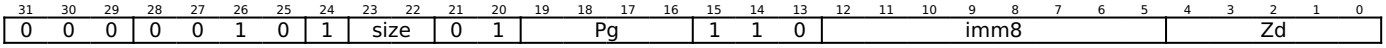
Copy floating-point immediate to vector elements (predicated)

This instruction copies a floating-point immediate into each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.

This instruction is used by the alias FMOV (immediate, predicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FCPY <Zd>.<T>, <Pg>/M, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let d : integer = UInt(Zd);
let imm : bits(esize) = VFPExpandImm{}(imm8);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<const> Is a floating-point immediate value expressible as ±n÷16×2^r, where n and r are integers such that 16 ≤ n ≤ 31 and -3 ≤ r ≤ 4, i.e. a normalized binary floating-point encoding with 1 sign bit, 3-bit exponent, and 4-bit fractional part, encoded in the "imm8" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var result : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = imm;
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVT

Floating-point convert (predicated)

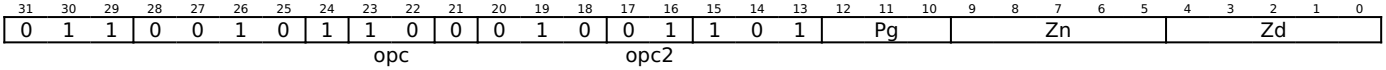
This instruction converts the size and precision of each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

Since the input and result types have a different size the smaller type is held unpacked in the least significant bits of elements of the larger size. When the input is the smaller type the upper bits of each source element are ignored. When the result is the smaller type the results are zero-extended to fill each destination element.

It has encodings from 12 classes: Half-precision to single-precision, merging , Half-precision to single-precision, zeroing , Half-precision to double-precision, merging , Half-precision to double-precision, zeroing , Single-precision to half-precision, merging , Single-precision to half-precision, zeroing , Single-precision to double-precision, merging , Single-precision to double-precision, zeroing , Double-precision to half-precision, merging , Double-precision to half-precision, zeroing , Double-precision to single-precision, merging and Double-precision to single-precision, zeroing

Half-precision to single-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

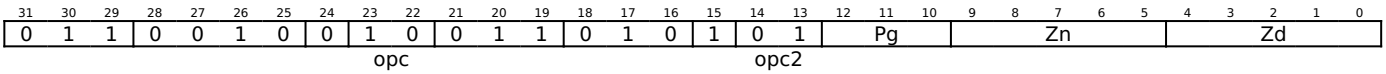
```
FCVT <Zd>.S, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 32;
let merging : boolean = TRUE;
```

Half-precision to single-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

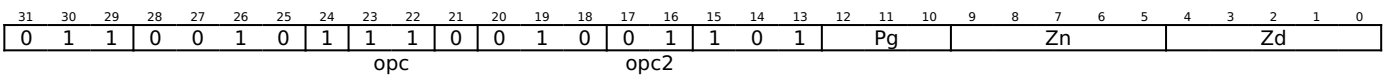
```
FCVT <Zd>.S, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 32;
let merging : boolean = FALSE;
```

Half-precision to double-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

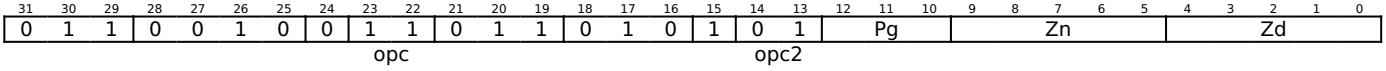
```
FCVT <Zd>.D, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 64;
let merging : boolean = TRUE;
```

Half-precision to double-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

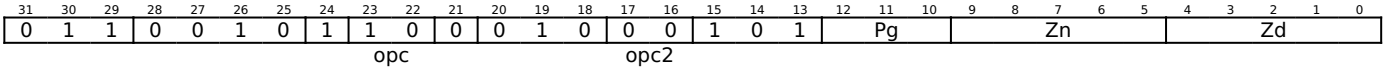
```
FCVT <Zd>.D, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 64;
let merging : boolean = FALSE;
```

Single-precision to half-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

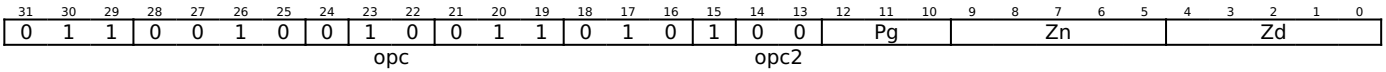
```
FCVT <Zd>.H, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 16;
let merging : boolean = TRUE;
```

Single-precision to half-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

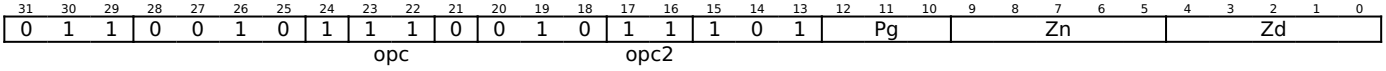
```
FCVT <Zd>.H, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 16;
let merging : boolean = FALSE;
```

Single-precision to double-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

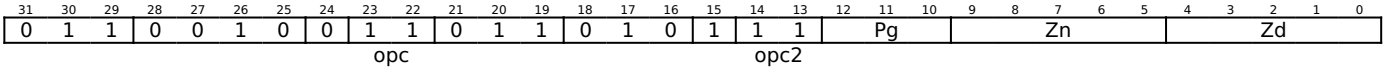
```
FCVT <Zd>.D, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 64;
let merging : boolean = TRUE;
```

Single-precision to double-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

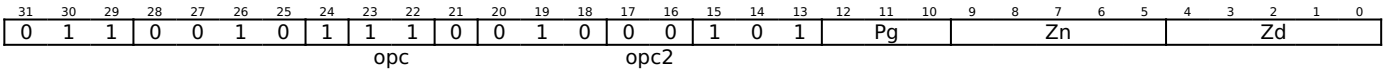
```
FCVT <Zd>.D, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 64;
let merging : boolean = FALSE;
```

Double-precision to half-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

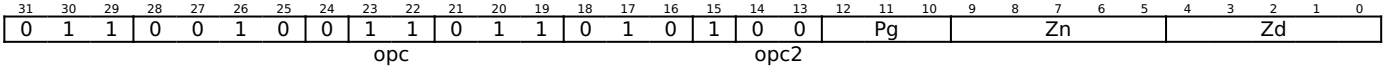
```
FCVT <Zd>.H, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 16;
let merging : boolean = TRUE;
```

Double-precision to half-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

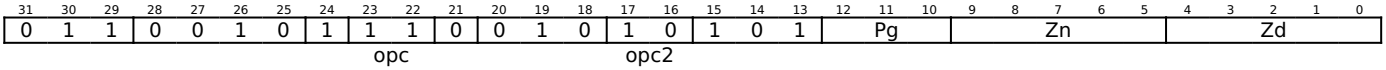
```
FCVT <Zd>.H, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 16;
let merging : boolean = FALSE;
```

Double-precision to single-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

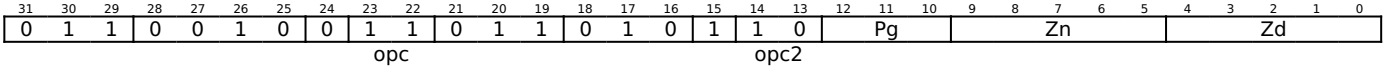
```
FCVT <Zd>.S, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let merging : boolean = TRUE;
```

Double-precision to single-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FCVT <Zd>.S, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        let res : bits(d_esize) = FPConvertSVE{d_esize, s_esize}(element[s_esize-1:0], FPCR());
        result[e*:esize] = ZeroExtend{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Half-precision to single-precision, merging" , "Half-precision to double-precision, merging" , "Single-precision to half-precision, merging" , "Single-precision to double-precision, merging" , "Double-precision to half-precision, merging" , "Double-precision to single-precision, merging" variants:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTLT

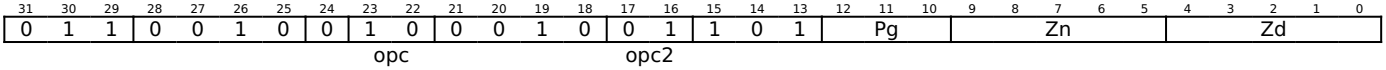
Floating-point widening convert (top, predicated)

This instruction converts odd-numbered floating-point elements from the source vector to the next higher precision, and places the results in the Active overlapping double-width elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 4 classes: Half-precision to single-precision, merging , Half-precision to single-precision, zeroing , Single-precision to double-precision, merging and Single-precision to double-precision, zeroing

Half-precision to single-precision, merging

(FEAT_SVE2 || FEAT_SME)



Encoding

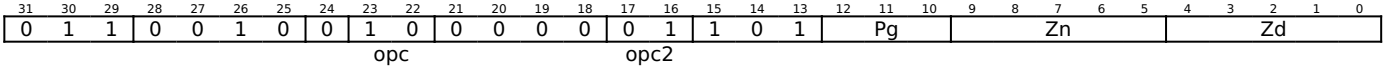
```
FCVTLT <Zd>.S, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Half-precision to single-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

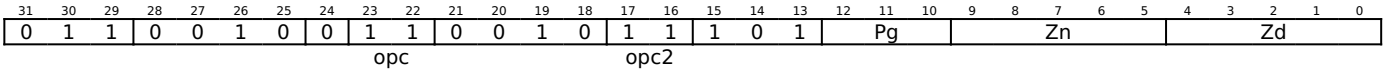
```
FCVTLT <Zd>.S, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Single-precision to double-precision, merging

(FEAT_SVE2 || FEAT_SME)



Encoding

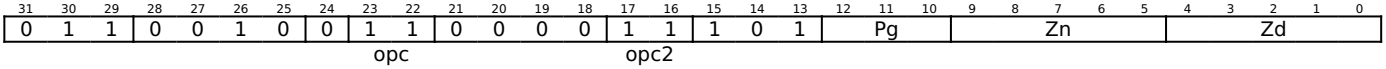
```
FCVTLT <Zd>.D, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Single-precision to double-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FCVTLT <Zd>.D, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let halfesize : integer{} = esize DIV 2;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(halfesize) = operand[(2*e + 1)*:halfesize];
        result[e*:esize] = FPConvertSVE{esize, halfesize}(element, FPCR());
    end;
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

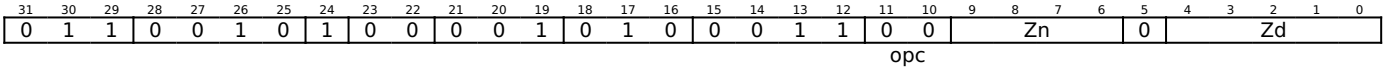
FCVTN

Half-precision convert to interleaved 8-bit floating-point

This instruction converts each half-precision element of the pair of source vectors to 8-bit floating-point while scaling the value by $2^{SInt(FPMR.NSCALE[4:0])}$, and places the two-way interleaved results in the corresponding 8-bit elements of the destination vector. The 8-bit floating-point encoding format is selected by FPMR.F8D.
This instruction is unpredicated.

SVE2

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

```
FCVTN <Zd>.B, { <Zn1>.H-<Zn2>.H }
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
      !IsFeatureImplemented(FEAT_FP8)) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    let element2 : bits(16) = operand2[e*:16];
    result[(2*e + 0)*:8] = FPConvertFP8{8, 16}(element1, FPCR(), FPMR());
    result[(2*e + 1)*:8] = FPConvertFP8{8, 16}(element2, FPCR(), FPMR());
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTNB

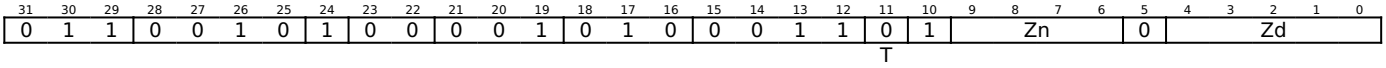
Single-precision convert to interleaved 8-bit floating-point (bottom)

This instruction converts each single-precision element of the pair of source vectors to 8-bit floating-point while scaling the value by $2^{SInt(FPMR.NSCALE)}$, and places the two-way interleaved results in the corresponding even-numbered 8-bit elements of the destination vector, zeroing the odd-numbered elements. The 8-bit floating-point encoding format is selected by FPMR.F8D.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

```
FCVTNB <Zd>.B, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
      !IsFeatureImplemented(FEAT_FP8)) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
for e = 0 to elements-1 do
    let element1 : bits(32) = operand1[e*:32];
    let element2 : bits(32) = operand2[e*:32];
    let res1 : bits(8) = FPConvertFP8{8, 32}(element1, FPCR(), FPMR());
    let res2 : bits(8) = FPConvertFP8{8, 32}(element2, FPCR(), FPMR());
    result[(2*e + 0)*:16] = ZeroExtend{16}(res1);
    result[(2*e + 1)*:16] = ZeroExtend{16}(res2);
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTNT (predicated)

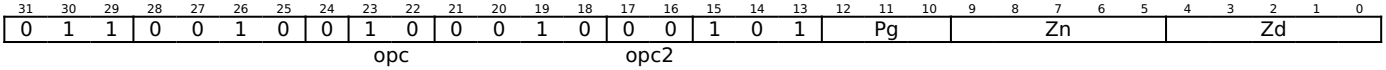
Floating-point narrowing convert (top, predicated)

This instruction converts Active floating-point elements from the source vector to the next lower precision, and places the results in the odd-numbered half-width elements of the destination vector, leaving the even-numbered elements unchanged. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 4 classes: Single-precision to half-precision, merging , Single-precision to half-precision, zeroing , Double-precision to single-precision, merging and Double-precision to single-precision, zeroing

Single-precision to half-precision, merging

(FEAT_SVE2 || FEAT_SME)



Encoding

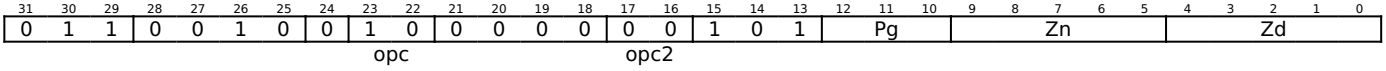
```
FCVTNT <Zd>.H, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Single-precision to half-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

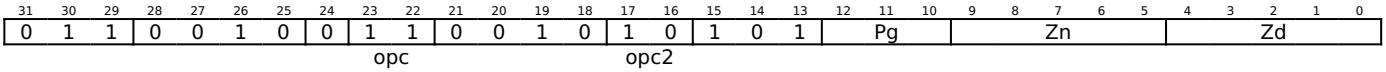
```
FCVTNT <Zd>.H, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Double-precision to single-precision, merging

(FEAT_SVE2 || FEAT_SME)



Encoding

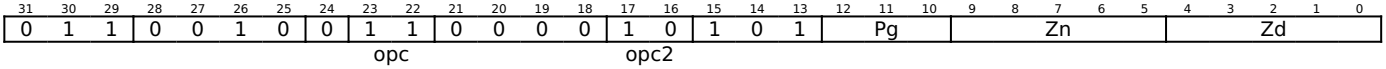
```
FCVTNT <Zd>.S, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Double-precision to single-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FCVTNT <Zd>.S, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let halfesize : integer{} = esize DIV 2;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[(2*e + 1)*:halfesize] = FPConvertSVE{halfesize, esize}(element, FPCR());

    elsif !merging then
        result[(2*e + 1)*:halfesize] = Zeros{halfesize};

    end;
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTNT (unpredicated)

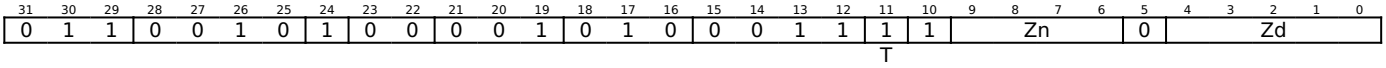
Single-precision convert to interleaved 8-bit floating-point (top)

This instruction converts each single-precision element of the pair of source vectors to 8-bit floating-point while scaling the value by $2^{\mathrm{SInt(FPMR.NSCALE)}}$, and places the two-way interleaved results in the corresponding odd-numbered 8-bit elements of the destination vector, leaving the even-numbered elements unchanged. The 8-bit floating-point encoding format is selected by FPMR.F8D.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 || FEAT_SME2) && FEAT_FP8)



Encoding

```
FCVTNT <Zd>.B, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
      !IsFeatureImplemented(FEAT_FP8)) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SME2) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var result : bits(VL) = Z{VL}(d);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
for e = 0 to elements-1 do
    let element1 : bits(32) = operand1[e*:32];
    let element2 : bits(32) = operand2[e*:32];
    result[(4*e + 1)*:8] = FPConvertFP8{8, 32}(element1, FPCR(), FPMR());
    result[(4*e + 3)*:8] = FPConvertFP8{8, 32}(element2, FPCR(), FPMR());
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTX

Double-precision convert to single-precision, rounding to odd (predicated)

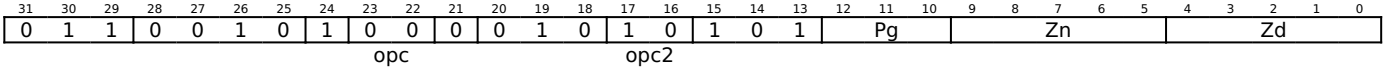
This instruction converts Active double-precision elements from the source vector to single-precision, rounding to Odd, and places the results in the even-numbered 32-bit elements of the destination vector, while setting the odd-numbered elements to zero. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

Rounding to Odd (aka Von Neumann rounding) permits a two-step conversion from double-precision to half-precision without incurring intermediate rounding errors.

It has encodings from 2 classes: Double-precision to single-precision, merging and Double-precision to single-precision, zeroing

Double-precision to single-precision, merging

(FEAT_SVE2 || FEAT_SME)



Encoding

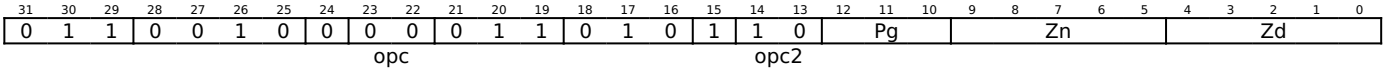
```
FCVTX <Zd>.S, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let merging : boolean = TRUE;
```

Double-precision to single-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FCVTX <Zd>.S, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        let res : bits(d_esize) = FPConvertSVE{d_esize, s_esize}(element[s_esize-1:0], FPCR(),
                                                                 FPRounding_ODD);
        result[e*:esize] = ZeroExtend{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Double-precision to single-precision, merging" variant:
The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTXNT

Double-precision convert to single-precision, rounding to odd (top, predicated)

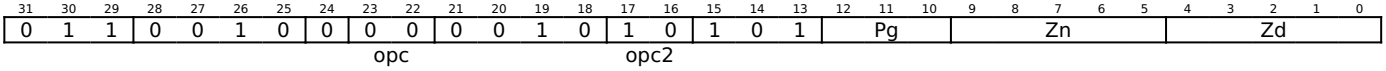
This instruction converts Active double-precision elements from the source vector to single-precision, rounding to Odd, and places the results in the odd-numbered 32-bit elements of the destination vector, leaving the even-numbered elements unchanged. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

Rounding to Odd (aka Von Neumann rounding) permits a two-step conversion from double-precision to half-precision without incurring intermediate rounding errors.

It has encodings from 2 classes: Double-precision to single-precision, merging and Double-precision to single-precision, zeroing

Double-precision to single-precision, merging

(FEAT_SVE2 || FEAT_SME)



Encoding

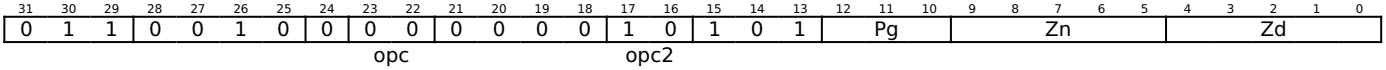
```
FCVTXNT <Zd>.S, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Double-precision to single-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FCVTXNT <Zd>.S, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let halfesize : integer{} = esize DIV 2;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[(2*e + 1)*:halfesize] = FPConvertSVE{halfesize, esize}(element, FPCR(),
                                                                      FPRounding_ODD);

    elsif !merging then
        result[(2*e + 1)*:halfesize] = Zeros{halfesize};

    end;
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZS

Floating-point convert to signed integer, rounding toward zero (predicated)

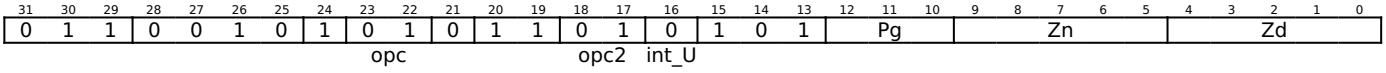
This instruction converts to the signed integer nearer to zero from each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

If the input and result types have a different size the smaller type is held unpacked in the least significant bits of elements of the larger size. When the input is the smaller type the upper bits of each source element are ignored. When the result is the smaller type the results are sign-extended to fill each destination element.

It has encodings from 14 classes: Half-precision to 16-bit, merging , Half-precision to 16-bit, zeroing , Half-precision to 32-bit, merging , Half-precision to 32-bit, zeroing , Half-precision to 64-bit, merging , Half-precision to 64-bit, zeroing , Single-precision to 32-bit, merging , Single-precision to 32-bit, zeroing , Single-precision to 64-bit, merging , Single-precision to 64-bit, zeroing , Double-precision to 32-bit, merging , Double-precision to 32-bit, zeroing , Double-precision to 64-bit, merging and Double-precision to 64-bit, zeroing

Half-precision to 16-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

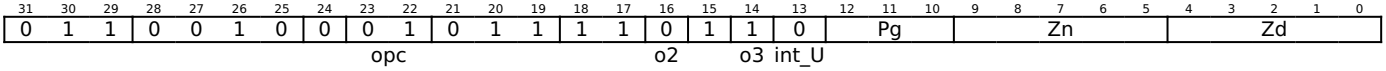
```
FCVTZS <Zd>.H, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 16;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Half-precision to 16-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

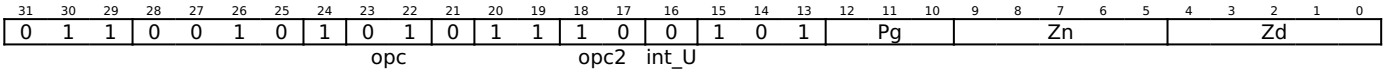
```
FCVTZS <Zd>.H, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 16;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Half-precision to 32-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

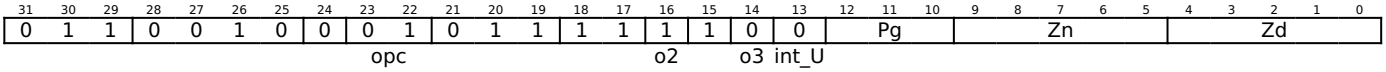
```
FCVTZS <Zd>.S, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 32;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Half-precision to 32-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

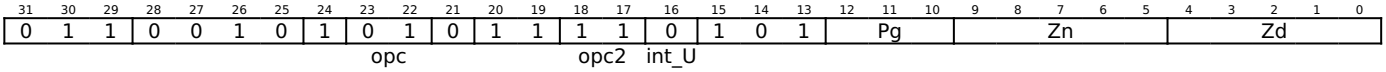
```
FCVTZS <Zd>.S, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 32;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Half-precision to 64-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

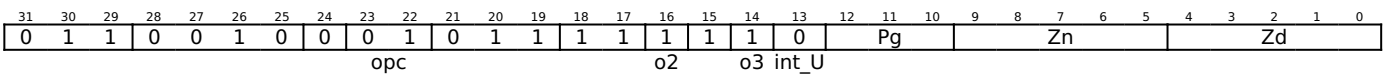
```
FCVTZS <Zd>.D, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 64;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Half-precision to 64-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

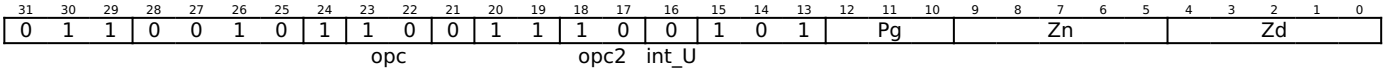
```
FCVTZS <Zd>.D, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 64;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Single-precision to 32-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

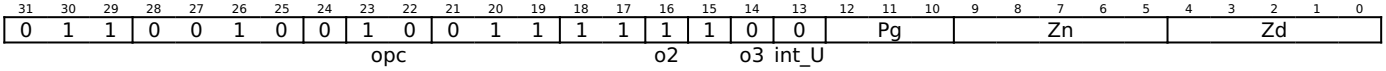
```
FCVTZS <Zd>.S, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 32;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Single-precision to 32-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

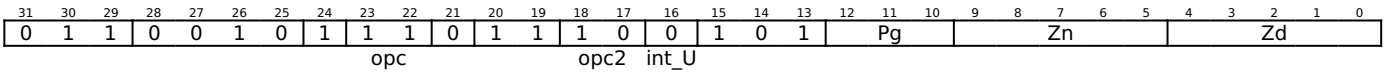
```
FCVTZS <Zd>.S, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 32;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Single-precision to 64-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

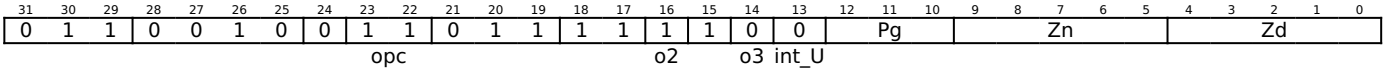
```
FCVTZS <Zd>.D, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 64;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Single-precision to 64-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

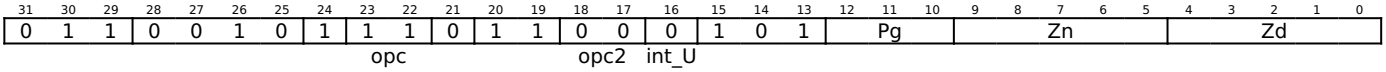
```
FCVTZS <Zd>.D, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 64;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Double-precision to 32-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

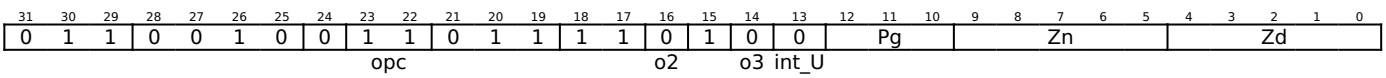
```
FCVTZS <Zd>.S, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Double-precision to 32-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

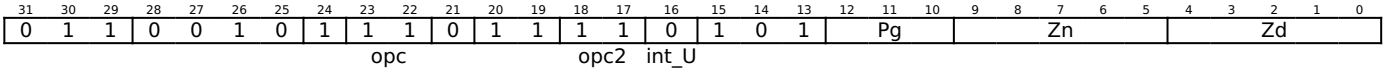
```
FCVTZS <Zd>.S, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Double-precision to 64-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

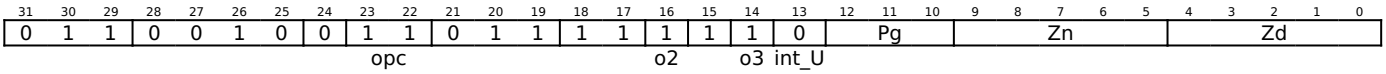
```
FCVTZS <Zd>.D, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 64;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Double-precision to 64-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FCVTZS <Zd>.D, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 64;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        let res : bits(d_esize) = FPToFixed{d_esize, s_esize}(element[s_esize-1:0], 0, unsigned,
                                                              FPCR(), rounding);
        result[e*:esize] = Extend{esize}(res, unsigned);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Half-precision to 16-bit, merging" , "Half-precision to 32-bit, merging" , "Half-precision to 64-bit, merging" , "Single-precision to 32-bit, merging" , "Single-precision to 64-bit, merging" , "Double-precision to 32-bit, merging" , "Double-precision to 64-bit, merging" variants:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZSN

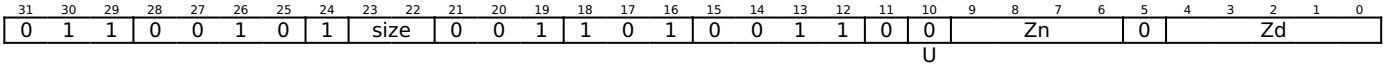
Floating-point narrowing convert to interleaved signed integer, rounding toward zero

This instruction converts each floating-point element of the pair of source vectors to half-width signed integer nearer to zero, and places the two-way interleaved results in the corresponding half-width elements of the destination vector.

This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
FCVTZSN <Zd>.<T>, { <Zn1>.<Tb>-<Zn2>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	B
10	H
11	S

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

<Zn2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n+0);
let operand2 : bits(VL) = Z{}(n+1);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[(2*e + 0)*:(esize DIV 2)] = FPToFixed{esize DIV 2, esize}(element1, 0, unsigned, FPCR(),
                                                                     rounding);
    result[(2*e + 1)*:(esize DIV 2)] = FPToFixed{esize DIV 2, esize}(element2, 0, unsigned, FPCR(),
                                                                     rounding);
end;

Z{VL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZU

Floating-point convert to unsigned integer, rounding toward zero (predicated)

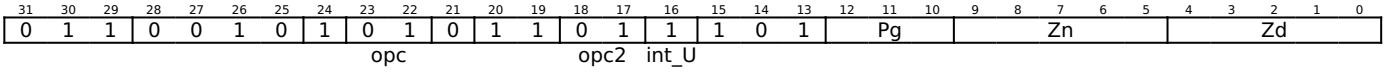
This instruction converts to the unsigned integer nearer to zero from each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

If the input and result types have a different size the smaller type is held unpacked in the least significant bits of elements of the larger size. When the input is the smaller type the upper bits of each source element are ignored. When the result is the smaller type the results are zero-extended to fill each destination element.

It has encodings from 14 classes: Half-precision to 16-bit, merging , Half-precision to 16-bit, zeroing , Half-precision to 32-bit, merging , Half-precision to 32-bit, zeroing , Half-precision to 64-bit, merging , Half-precision to 64-bit, zeroing , Single-precision to 32-bit, merging , Single-precision to 32-bit, zeroing , Single-precision to 64-bit, merging , Single-precision to 64-bit, zeroing , Double-precision to 32-bit, merging , Double-precision to 32-bit, zeroing , Double-precision to 64-bit, merging and Double-precision to 64-bit, zeroing

Half-precision to 16-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

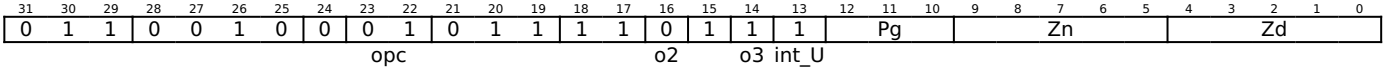
```
FCVTZU <Zd>.H, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 16;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Half-precision to 16-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

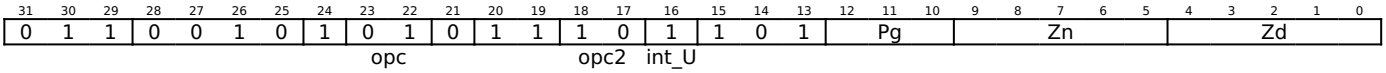
```
FCVTZU <Zd>.H, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 16;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Half-precision to 32-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

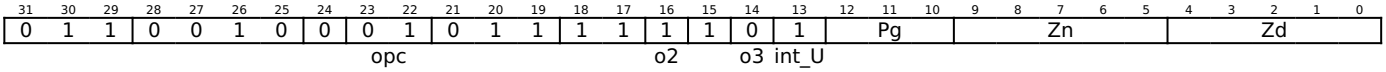
```
FCVTZU <Zd>.S, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 32;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Half-precision to 32-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

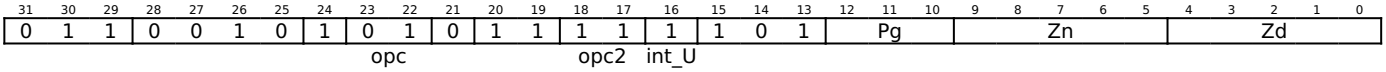
```
FCVTZU <Zd>.S, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 32;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Half-precision to 64-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

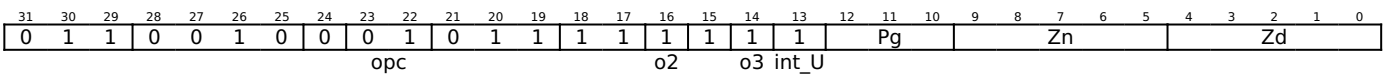
```
FCVTZU <Zd>.D, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 64;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Half-precision to 64-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

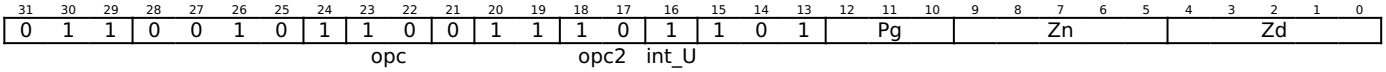
```
FCVTZU <Zd>.D, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 64;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Single-precision to 32-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

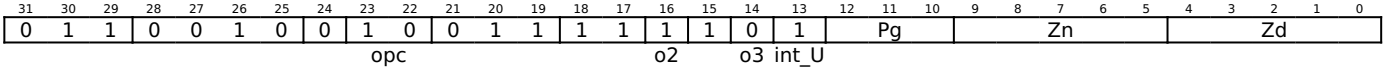
```
FCVTZU <Zd>.S, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 32;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Single-precision to 32-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

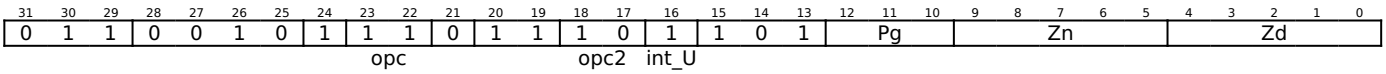
```
FCVTZU <Zd>.S, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 32;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Single-precision to 64-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

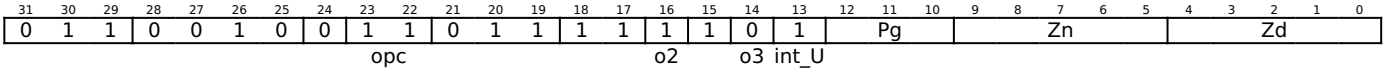
```
FCVTZU <Zd>.D, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 64;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Single-precision to 64-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

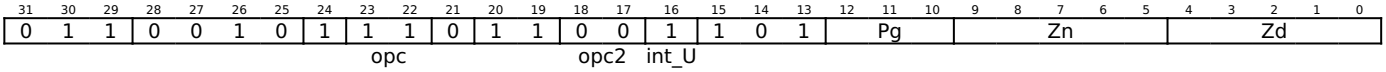
```
FCVTZU <Zd>.D, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 64;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Double-precision to 32-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

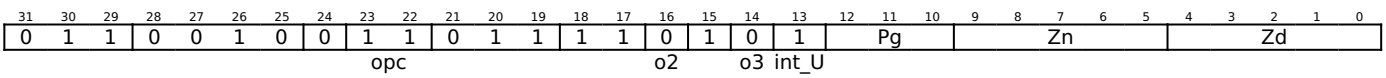
```
FCVTZU <Zd>.S, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Double-precision to 32-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

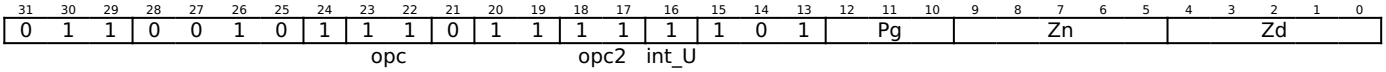
```
FCVTZU <Zd>.S, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Double-precision to 64-bit, merging

(FEAT_SVE || FEAT_SME)



Encoding

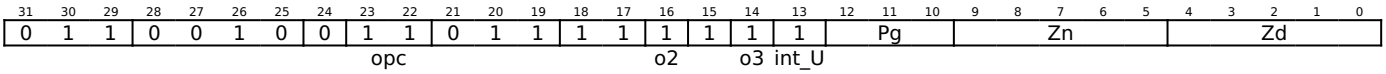
```
FCVTZU <Zd>.D, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 64;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Double-precision to 64-bit, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FCVTZU <Zd>.D, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 64;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        let res : bits(d_esize) = FPToFixed{d_esize, s_esize}(element[s_esize-1:0], 0, unsigned,
                                                              FPCR(), rounding);
        result[e*:esize] = Extend{esize}(res, unsigned);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Half-precision to 16-bit, merging" , "Half-precision to 32-bit, merging" , "Half-precision to 64-bit, merging" , "Single-precision to 32-bit, merging" , "Single-precision to 64-bit, merging" , "Double-precision to 32-bit, merging" , "Double-precision to 64-bit, merging" variants:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZUN

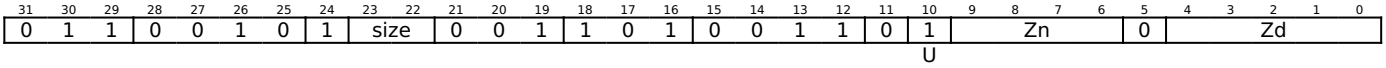
Floating-point narrowing convert to interleaved unsigned integer, rounding toward zero

This instruction converts each floating-point element of the pair of source vectors to half-width unsigned integer nearer to zero, and places the two-way interleaved results in the corresponding half-width elements of the destination vector.

This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
FCVTZUN <Zd>.<T>, { <Zn1>.<Tb>-<Zn2>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	B
10	H
11	S

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

<Zn2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n+0);
let operand2 : bits(VL) = Z{}(n+1);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[(2*e + 0)*:(esize DIV 2)] = FPToFixed{esize DIV 2, esize}(element1, 0, unsigned, FPCR(),
                                                                     rounding);
    result[(2*e + 1)*:(esize DIV 2)] = FPToFixed{esize DIV 2, esize}(element2, 0, unsigned, FPCR(),
                                                                     rounding);
end;

Z{VL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

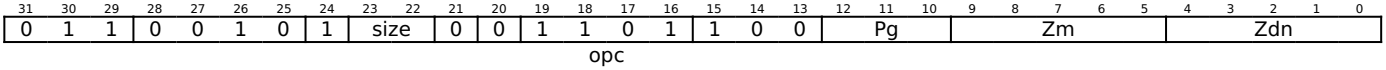
FDIV

Floating-point divide (predicated)

This instruction divides Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector and destructively places the quotient in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FDIV <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPDiv{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

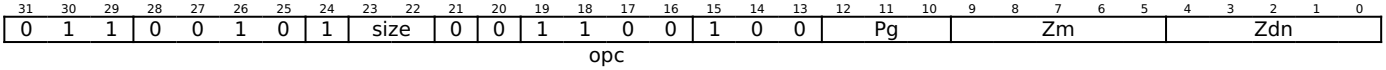
FDIVR

Floating-point reversed divide (predicated)

This instruction performs a reversed divide on Active floating-point elements of the second source vector by the corresponding floating-point elements of the first source vector, and destructively places the quotient in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FDIVR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPDiv{esize}(element2, element1, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (2-way, indexed, FP16 to FP32)

Half-precision dot product by indexed element to single-precision

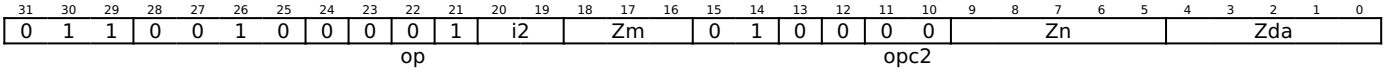
This instruction calculates the fused sum-of-products of a pair of half-precision values held in each 32-bit element of the first source vector and a pair of half-precision values in an indexed 32-bit element of the second source vector, without intermediate rounding, and then destructively adds the single-precision sum-of-products to the corresponding single-precision element of the destination vector.

The half-precision pairs within the second source vector are specified using an immediate index that selects the same pair position within each 128-bit vector segment. The index range is from 0 to 3.

This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
FDOT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i2);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let elt1_a : bits(16) = operand1[(2 * e + 0)*:16];
    let elt1_b : bits(16) = operand1[(2 * e + 1)*:16];
    let elt2_a : bits(16) = operand2[(2 * s + 0)*:16];
    let elt2_b : bits(16) = operand2[(2 * s + 1)*:16];
    var sum : bits(32) = operand3[e*:32];

    sum = FPDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
    result[e*:32] = sum;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (2-way, indexed, FP8 to FP16)

8-bit floating-point dot product by indexed element to half-precision

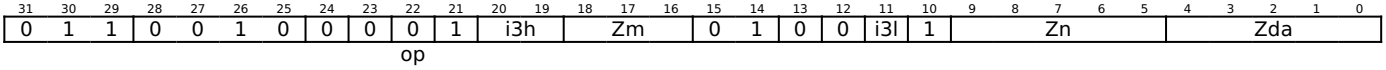
This instruction calculates the fused sum-of-products of a pair of 8-bit floating-point values held in each 16-bit element of the first source vector and a pair of 8-bit floating-point values in an indexed 16-bit element of the second source vector. The half-precision sum-of-products are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$, before being destructively added without intermediate rounding to the corresponding half-precision elements of the destination vector.

The 8-bit floating-point groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8DOT2) || FEAT_SSVE_FP8DOT2)



Encoding

```
FDOT <Zda>.H, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !HaveSVE2FP8DOT2() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i3h::i3l);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index of a pair of 8-bit elements within each 128-bit vector segment, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8DOT2) && IsFeatureImplemented(FEAT_FP8DOT2) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8DOT2) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let eltspersegment : integer = 128 DIV 16;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let op1 : bits(16) = operand1[e*:(16)];
    let op2 : bits(16) = operand2[s*:(16)];
    var sum : bits(16)  = operand3[e*:(16)];

    sum = FP8DotAddFP{16, 16}(sum, op1, op2, FPCR(), FPMR());
    result[e*:(16)] = sum;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (2-way, vectors, FP16 to FP32)

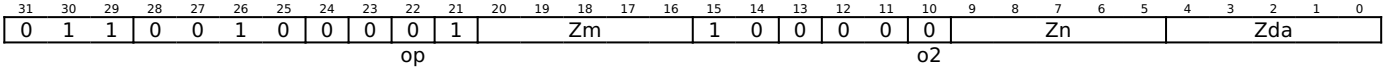
Half-precision dot product to single-precision

This instruction calculates the fused sum-of-products of a pair of half-precision values held in each 32-bit element of the first source and second source vectors, without intermediate rounding, and then destructively adds the single-precision sum-of-products to the corresponding single-precision element of the destination vector.

This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
FDOT <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let elt1_a : bits(16) = operand1[(2 * e + 0)*:16];
    let elt1_b : bits(16) = operand1[(2 * e + 1)*:16];
    let elt2_a : bits(16) = operand2[(2 * e + 0)*:16];
    let elt2_b : bits(16) = operand2[(2 * e + 1)*:16];
    var sum : bits(32) = operand3[e*:32];

    sum = FPDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
    result[e*:32] = sum;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (2-way, vectors, FP8 to FP16)

8-bit floating-point dot product to half-precision

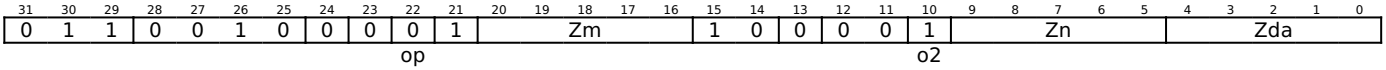
This instruction calculates the fused sum-of-products of a pair of 8-bit floating-point values held in each 16-bit element of the first source and second source vectors. The half-precision sum-of-products are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$, before being destructively added without intermediate rounding to the corresponding half-precision elements of the destination vector.

The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8DOT2) || FEAT_SSVE_FP8DOT2)



Encoding

```
FDOT <Zda>.H, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !HaveSVE2FP8DOT2() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8DOT2) && IsFeatureImplemented(FEAT_FP8DOT2) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8DOT2) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1 : bits(16) = operand1[e*:(16)];
    let op2 : bits(16) = operand2[e*:(16)];
    var sum : bits(16)  = operand3[e*:(16)];

    sum = FP8DotAddFP{16, 16}(sum, op1, op2, FPCR(), FPMR());
    result[e*:(16)] = sum;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (4-way, indexed)

8-bit floating-point dot product by indexed element to single-precision

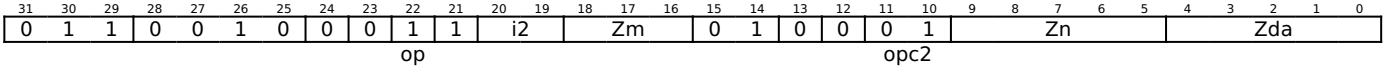
This instruction calculates the fused sum-of-products of a group of four 8-bit floating-point values held in each 32-bit element of the first source vector and a group of four 8-bit floating-point values in an indexed 32-bit element of the second source vector. The single-precision sum-of-products are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$, before being destructively added without intermediate rounding to the corresponding single-precision elements of the destination vector.

The 8-bit floating-point groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8DOT4) || FEAT_SSVE_FP8DOT4)



Encoding

```
FDOT <Zda>.S, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !HaveSVE2FP8DOT4() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i2);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8DOT4) && IsFeatureImplemented(FEAT_FP8DOT4) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8DOT4) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let op1 : bits(32) = operand1[e*:(32)];
    let op2 : bits(32) = operand2[s*:(32)];
    var sum : bits(32)  = operand3[e*:(32)];

    sum = FP8DotAddFP{32, 32}(sum, op1, op2, FPCR(), FPMR());
    result[e*:(32)] = sum;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (4-way, vectors)

8-bit floating-point dot product to single-precision

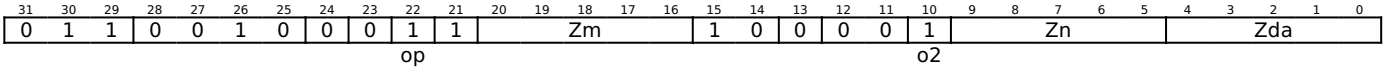
This instruction calculates the fused sum-of-products of a group of four 8-bit floating-point values held in each 32-bit element of the first source and second source vectors. The single-precision sum-of-products are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$, before being destructively added without intermediate rounding to the corresponding single-precision elements of the destination vector.

The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8DOT4) || FEAT_SSVE_FP8DOT4)



Encoding

```
FDOT <Zda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !HaveSVE2FP8DOT4() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8DOT4) && IsFeatureImplemented(FEAT_FP8DOT4) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8DOT4) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1 : bits(32) = operand1[e*:(32)];
    let op2 : bits(32) = operand2[e*:(32)];
    var sum : bits(32)  = operand3[e*:(32)];

    sum = FP8DotAddFP{32, 32}(sum, op1, op2, FPCR(), FPMR());
    result[e*:(32)] = sum;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDUP

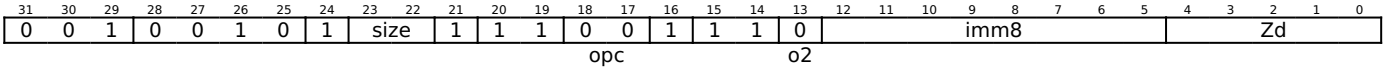
Broadcast floating-point immediate to vector elements (unpredicated)

This instruction unconditionally broadcasts the floating-point immediate into each element of the destination vector. This instruction is unpredicated.

This instruction is used by the alias FMOV (immediate, unpredicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FDUP <Zd>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt(Zd);
let imm : bits(esize) = VFPExpandImm{}(imm8);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<const> Is a floating-point immediate value expressible as ±n÷16×2^r, where n and r are integers such that 16 ≤ n ≤ 31 and -3 ≤ r ≤ 4, i.e. a normalized binary floating-point encoding with 1 sign bit, 3-bit exponent, and 4-bit fractional part, encoded in the "imm8" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var result : bits(VL);

for e = 0 to elements-1 do
    result[e*:esize] = imm;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FEXPA

Floating-point exponential accelerator

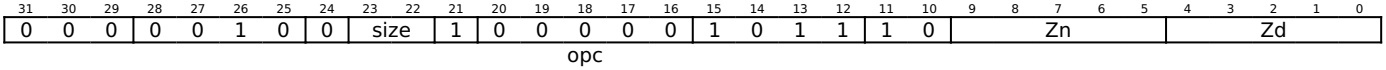
This instruction calculates an exponentiation acceleration operation on each floating-point element in the source vector, where the result sign is zero, the result exponent field is copied from a set of significant bits of the input fraction, and the result fraction is inserted from a lookup table indexed by the least-significant input fraction bits, and returns each result in the corresponding element of the destination vector.
This instruction is fully defined by its bit-manipulation semantics, does not generate floating-point exceptions, and does not guarantee NaN propagation.
For double-precision variants, the result element exponent is copied from the source element bits<16:6>, and the result fraction is set based on the source element to the rounded value of $2^{52} \times (2^{bits<5:0>/64} - 1)$.
For single-precision variants, the result element exponent is copied from the source element bits<13:6>, and the result fraction is set based on the source element to the rounded value of $2^{23} \times (2^{bits<5:0>/64} - 1)$.
For half-precision variants, the result element exponent is copied from the source element bits<9:5>, and the result fraction is set based on the source element to the rounded value of $2^{10} \times (2^{bits<4:0>/32} - 1)$.
This instruction is unpredicated.
This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled, or FEAT_SSVE_FEXPA is implemented.

Note

For a double-precision input value x in the range 70,368,744,177,655 <= x < 70,368,744,179,711, the operation performed by this instruction is equivalent to calculating $2^{x-70,368,744,178,687}$.
For a single-precision input value x in the range 131,073 <= x < 131,327, the operation performed by this instruction is equivalent to calculating $2^{x-131,199}$.
For a half-precision input value x in the range 33 <= x < 63, the operation performed by this instruction is equivalent to calculating 2^{x-47}.

SVE

(FEAT_SVE || FEAT_SSVE_FEXPA)



Encoding

```
FEXPA <Zd>.<T>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SSVE_FEXPA) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_FEXPA) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[e*:esize];
    result[e*:esize] = FPExpA{esize}(element);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

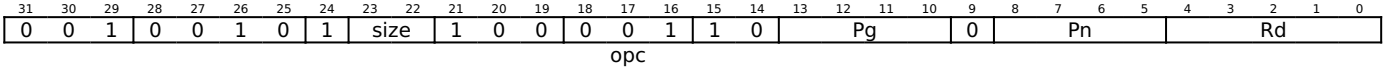
FIRSTP

Scalar index of first true predicate element (predicated)

This instruction determines the index of the First active and true element of the source predicate, and places the scalar result in the destination general-purpose register. If there are no Active and true elements, the destination general-purpose register is updated to the value minus one.

SVE2

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FIRSTP <Xd>, <Pg>, <Pn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let d : integer = UInt(Rd);
```

Assembler Symbols

<Xd> Is the 64-bit name of the destination general-purpose register, encoded in the "Rd" field.

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<Pn> Is the name of the source scalable predicate register, encoded in the "Pn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(PL) = P{}(n);
var index : integer = -1;

for e = 0 to elements-1 do
    if (index == -1 && ActivePredicateElement{PL}(mask, e, esize) &&
          ActivePredicateElement{PL}(operand, e, esize)) then
        index = e;
    end;
end;

X{64}(d) = index[63:0];
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FLOGB

Floating-point base 2 logarithm as integer

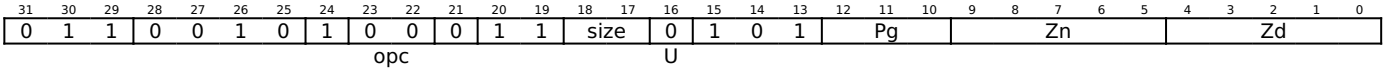
This instruction returns the signed integer base 2 logarithm of each floating-point input element |x| after normalization.
This is the unbiased exponent of x used in the representation of the floating-point value, such that, for positive x, x = significand × $2^{exponent}$.
The integer results are placed in elements of the destination vector that have the same width (esize) as the floating-point input elements:

- If x is normal, the result is the base 2 logarithm of x.
- If x is subnormal, the result corresponds to the normalized representation.
- If x is infinite, the result is $2^{(esize-1)}$-1.
- If x is ±0.0 or NaN, the result is $-2^{(esize-1)}$.

Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.
It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE2 || FEAT_SME)



Encoding

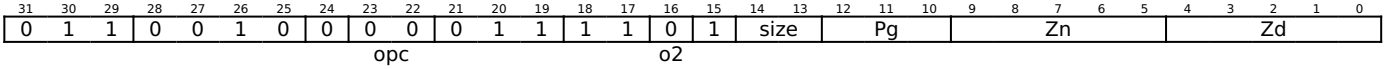
```
FLOGB <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FLOGB <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = FPLogB{esize}(element, FPCR());
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:
The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

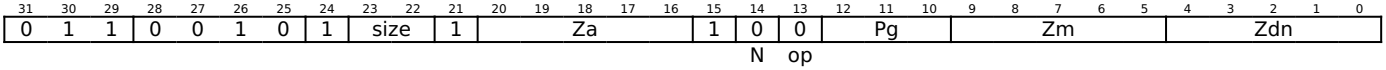
FMAD

Floating-point fused multiply-add to multiplicand (predicated)

This instruction multiplies the Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector, and adds the result to elements of the third (addend) vector without intermediate rounding. The results are destructively placed in the destination and first source (multiplicand) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAD <Zdn>.<T>, <Pg>/M, <Zm>.<T>, <Za>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let a : integer = UInt(Za);
let op1_neg : boolean = FALSE;
let op3_neg : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Za> Is the name of the third source scalable vector register, encoded in the "Za" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = Z{}(dn);
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(a) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let elem1 : bits(esize) = (if op1_neg then FPNeg{esize}(op1[e*:esize], FPCR())
                                   else op1[e*:esize]);
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = (if op3_neg then FPNeg{esize}(op3[e*:esize], FPCR())
                                   else op3[e*:esize]);

        result[e*:esize] = FPMulAdd{esize}(elem3, elem1, elem2, FPCR());
    else
        result[e*:esize] = op1[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAX (immediate)

Floating-point maximum with immediate (predicated)

This instruction determines the maximum of an immediate and each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.0 or +1.0 only.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if the element is a NaN, the result is a quiet NaN derived from it.
- When FPCR.DN is 1, if the element is a NaN, the result is default NaN.

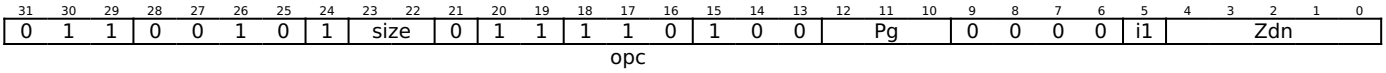
When FPCR.AH is 1, the behavior is as follows:

- If both the element and the immediate are zeros, regardless of the sign of either zero, the result is the immediate.
- If the element is a NaN, regardless of the value of FPCR.DN, the result is the immediate.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAX <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then Zeros{esize} else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.0
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPMax{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAX (vectors)

Floating-point maximum (predicated)

This instruction determines the maximum of Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.
When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

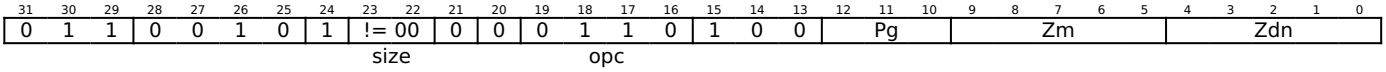
When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAX <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPMax{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNM (immediate)

Floating-point maximum number with immediate (predicated)

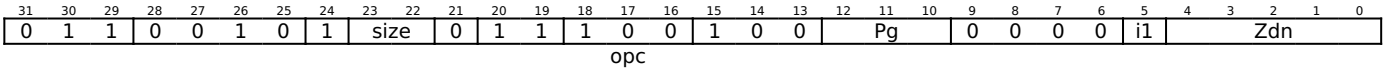
This instruction determines the maximum number value of an immediate and each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.0 or +1.0 only.
The behavior is as follows:

- Negative zero compares less than positive zero.
- If the element is a quiet NaN, the element is ignored and the result is the immediate value.
- When FPCR.DN is 0, if the element is a signaling NaN, the result is a quiet NaN derived from it.
- When FPCR.DN is 1, if the element is a signaling NaN, the result is default NaN.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAXNM <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then Zeros{esize} else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.0
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPMaxNum{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNM (vectors)

Floating-point maximum number (predicated)

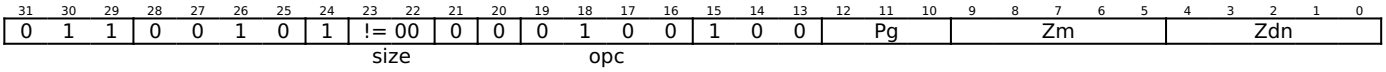
This instruction determines the maximum number value of Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.
The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAXNM <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNMP

Floating-point maximum number pairwise

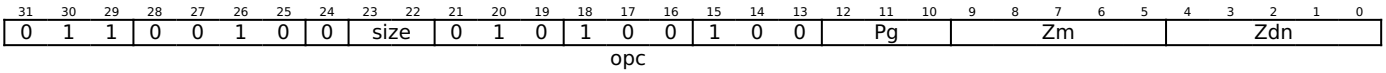
This instruction calculates the maximum value of each pair of adjacent floating-point elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

The behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMAXNMP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL) = Z{}(dn);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if IsEven(e) then
            element1 = operand1[(e + 0)*:esize];
            element2 = operand1[(e + 1)*:esize];
        else
            element1 = operand2[(e - 1)*:esize];
            element2 = operand2[(e + 0)*:esize];
        end;
        result[e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNMQV

Floating-point maximum number recursive reduction of quadword vector segments

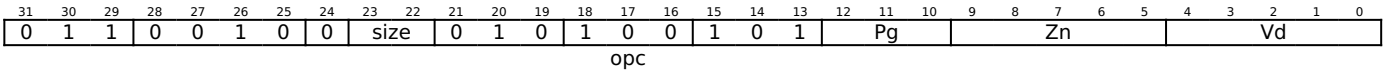
This instruction calculates a floating-point maximum number of the same element numbers from each 128-bit source vector segment using a recursive pairwise reduction, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as the default NaN.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
FMAXNMQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	RESERVED
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer{} = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let segbits : integer{} = segments*esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPDefaultNaN{esize}(FPCR());
var result : bits(128) = Zeros{};

for e = 0 to elempersegment-1 do
    var stmp : bits(segbits);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp[s*:esize] = operand[(s * elempersegment + e)*:esize];
        else
            stmp[s*:esize] = identity;
        end;
    end;
    result[e*:esize] = FPReduce{esize, segbits}(ReduceOp_FMAXNUM, stmp, FPCR());
end;
V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNMV

Floating-point maximum number recursive reduction to scalar

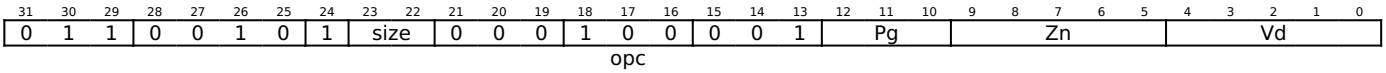
This instruction calculates a floating-point maximum number horizontally over all lanes of a vector using a recursive pairwise reduction, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as the default NaN.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAXNMV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPDefaultNaN{esize}(FPCR());

V{esize}(d) = FPReducePredicated{esize, VL, PL}(ReduceOp_FMAXNUM, operand, mask,
                                                identity, FPCR());
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXP

Floating-point maximum pairwise

This instruction calculates the maximum value of each pair of adjacent floating-point elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

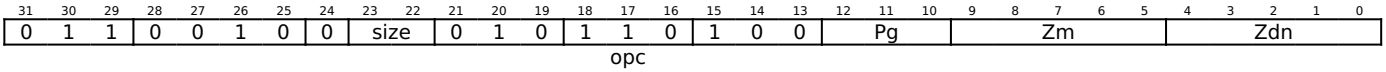
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMAXP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL) = Z{}(dn);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if IsEven(e) then
            element1 = operand1[(e + 0)*:esize];
            element2 = operand1[(e + 1)*:esize];
        else
            element1 = operand2[(e - 1)*:esize];
            element2 = operand2[(e + 0)*:esize];
        end;
        result[e*:esize] = FPMax{esize}(element1, element2, FPCR());
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXQV

Floating-point maximum reduction of quadword vector segments

This instruction calculates a floating-point maximum of the same element numbers from each 128-bit source vector segment using a recursive pairwise reduction, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as -Infinity.
When FPCR.AH is 0, the behavior is as follows:

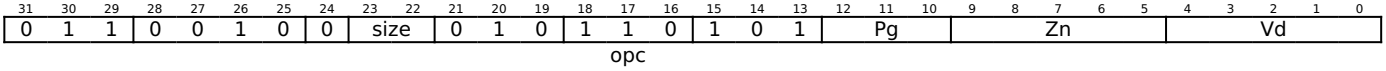
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
FMAXQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	RESERVED
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer{} = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let segbits : integer{} = segments*esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPInfinity{esize}('1');
var result : bits(128) = Zeros{};

for e = 0 to elempersegment-1 do
    var stmp : bits(segbits);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp[s*:esize] = operand[(s * elempersegment + e)*:esize];
        else
            stmp[s*:esize] = identity;
        end;
    end;
    result[e*:esize] = FPReduce{esize, segbits}(ReduceOp_FMAX, stmp, FPCR());
end;
V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXV

Floating-point maximum recursive reduction to scalar

This instruction calculates a floating-point maximum horizontally over all lanes of a vector using a recursive pairwise reduction, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as -Infinity.

When FPCR.AH is 0, the behavior is as follows:

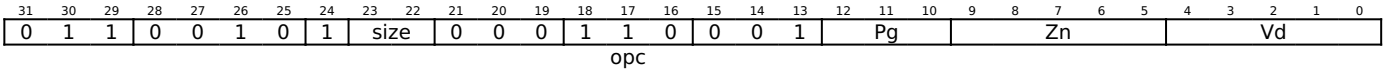
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMAXV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPInfinity{esize}('1');

V{esize}(d) = FPReducePredicated{esize, VL, PL}(ReduceOp_FMAX, operand, mask,
                                                identity, FPCR());
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMIN (immediate)

Floating-point minimum with immediate (predicated)

This instruction determines the minimum of an immediate and each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.0 or +1.0 only.
When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if the element is a NaN, the result is a quiet NaN derived from it.
- When FPCR.DN is 1, if the element is a NaN, the result is default NaN.

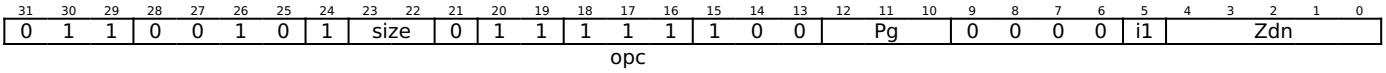
When FPCR.AH is 1, the behavior is as follows:

- If both the element and the immediate are zeros, regardless of the sign of either zero, the result is the immediate.
- If the element is a NaN, regardless of the value of FPCR.DN, the result is the immediate.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMIN <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then Zeros{esize} else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.0
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPMin{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMIN (vectors)

Floating-point minimum (predicated)

This instruction determines the minimum of Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

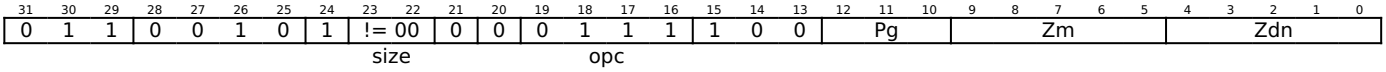
When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMIN <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPMin{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNM (immediate)

Floating-point minimum number with immediate (predicated)

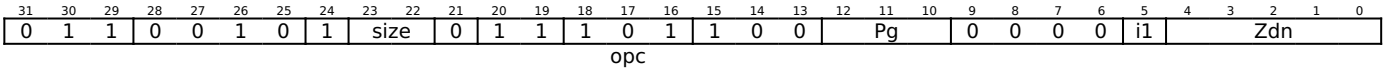
This instruction determines the minimum number value of an immediate and each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.0 or +1.0 only.
The behavior is as follows:

- Negative zero compares less than positive zero.
- If the element is a quiet NaN, the element is ignored and the result is the immediate value.
- When FPCR.DN is 0, if the element is a signaling NaN, the result is a quiet NaN derived from it.
- When FPCR.DN is 1, if the element is a signaling NaN, the result is default NaN.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMINNM <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then Zeros{esize} else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.0
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPMinNum{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNM (vectors)

Floating-point minimum number (predicated)

This instruction determines the minimum number value of Active floating-point elements of the first source vector and the corresponding floating-point elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector.

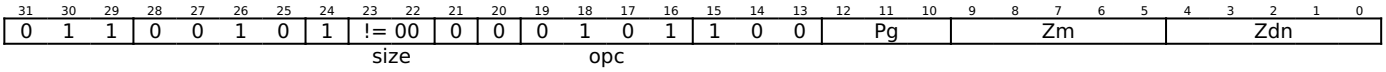
The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMINNM <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNMP

Floating-point minimum number pairwise

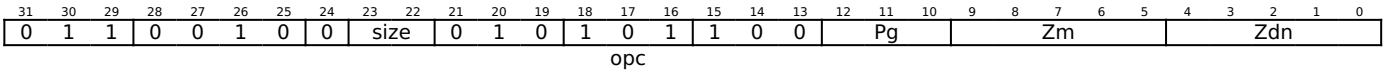
This instruction calculates the minimum value of each pair of adjacent floating-point elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

The behavior is as follows for each pairwise operation:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMINNMP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL) = Z{}(dn);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if IsEven(e) then
            element1 = operand1[(e + 0)*:esize];
            element2 = operand1[(e + 1)*:esize];
        else
            element1 = operand2[(e - 1)*:esize];
            element2 = operand2[(e + 0)*:esize];
        end;
        result[e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNMQV

Floating-point minimum number recursive reduction of quadword vector segments

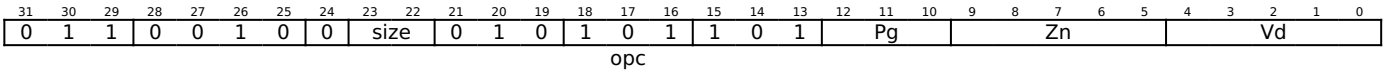
This instruction calculates a floating-point minimum number of the same element numbers from each 128-bit source vector segment using a recursive pairwise reduction, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as the default NaN.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
FMINNMQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	RESERVED
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer{} = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let segbits : integer{} = segments*esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPDefaultNaN{esize}(FPCR());
var result : bits(128) = Zeros{};

for e = 0 to elempersegment-1 do
    var stmp : bits(segbits);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp[s*:esize] = operand[(s * elempersegment + e)*:esize];
        else
            stmp[s*:esize] = identity;
        end;
    end;
    result[e*:esize] = FPReduce{esize, segbits}(ReduceOp_FMINNUM, stmp, FPCR());
end;
V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNMV

Floating-point minimum number recursive reduction to scalar

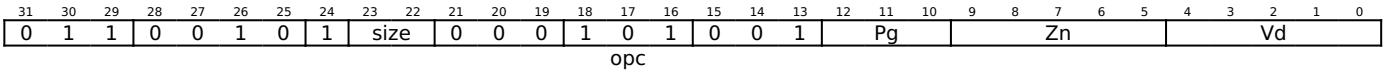
This instruction calculates a floating-point minimum number horizontally over all lanes of a vector using a recursive pairwise reduction, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as the default NaN.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMINNMV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPDefaultNaN{esize}(FPCR());

V{esize}(d) = FPReducePredicated{esize, VL, PL}(ReduceOp_FMINNUM, operand, mask,
                                                identity, FPCR());
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINP

Floating-point minimum pairwise

This instruction calculates the minimum value of each pair of adjacent floating-point elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

When FPCR.AH is 0, the behavior is as follows for each pairwise operation:

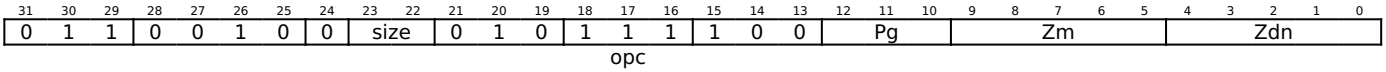
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows for each pairwise operation:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMINP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL) = Z{}(dn);
var element1 : bits(esize);
var element2 : bits(esize);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if IsEven(e) then
            element1 = operand1[(e + 0)*:esize];
            element2 = operand1[(e + 1)*:esize];
        else
            element1 = operand2[(e - 1)*:esize];
            element2 = operand2[(e + 0)*:esize];
        end;
        result[e*:esize] = FPMin{esize}(element1, element2, FPCR());
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINQV

Floating-point minimum recursive reduction of quadword vector segments

This instruction calculates a floating-point minimum of the same element numbers from each 128-bit source vector segment using a recursive pairwise reduction, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as +Infinity.
When FPCR.AH is 0, the behavior is as follows:

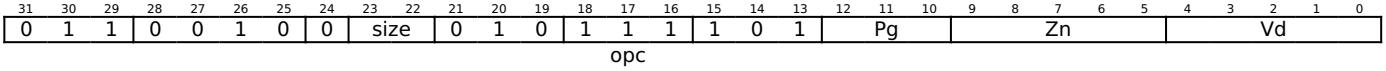
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
FMINQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	RESERVED
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer{} = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let segbits : integer{} = segments*esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPInfinity{esize}('0');
var result : bits(128) = Zeros{};

for e = 0 to elempersegment-1 do
    var stmp : bits(segbits);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp[s*:esize] = operand[(s * elempersegment + e)*:esize];
        else
            stmp[s*:esize] = identity;
        end;
    end;
    result[e*:esize] = FPReduce{esize, segbits}(ReduceOp_FMIN, stmp, FPCR());
end;
V{128}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINV

Floating-point minimum recursive reduction to scalar

This instruction calculates a floating-point minimum horizontally over all lanes of a vector using a recursive pairwise reduction, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as +Infinity.

When FPCR.AH is 0, the behavior is as follows:

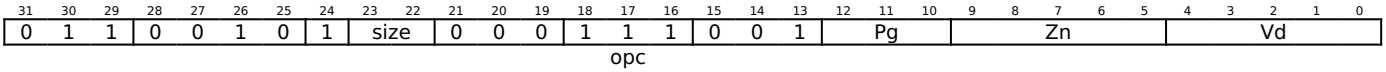
- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either value is a NaN, the result is a quiet NaN derived as follows:
  - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
  - Otherwise, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both values are zeros, regardless of the sign of either zero, the result is the second value.
- If either value is a NaN, regardless of the value of FPCR.DN, the result is the second value.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMINV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	RESERVED
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let identity : bits(esize) = FPInfinity{esize}('0');

V{esize}(d) = FPReducePredicated{esize, VL, PL}(ReduceOp_FMIN, operand, mask,
                                                identity, FPCR());
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLA (indexed)

Floating-point fused multiply-add by indexed element

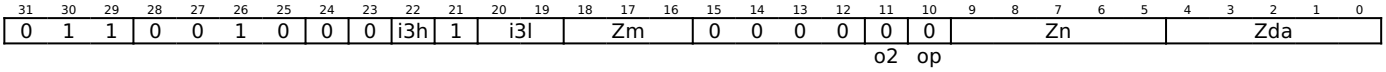
This instruction multiplies all floating-point elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment. The products are then destructively added without intermediate rounding to the corresponding elements of the addend and destination vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment. This instruction is unpredicated.

It has encodings from 3 classes: Half-precision , Single-precision and Double-precision

Half-precision

(FEAT_SVE || FEAT_SME)



Encoding

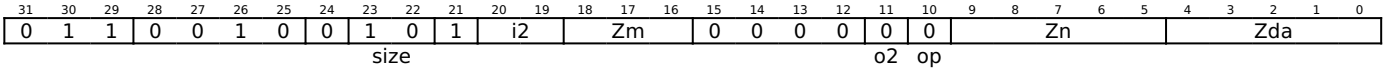
```
FMLA <Zda>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
let op3_neg : boolean = FALSE;
```

Single-precision

(FEAT_SVE || FEAT_SME)



Encoding

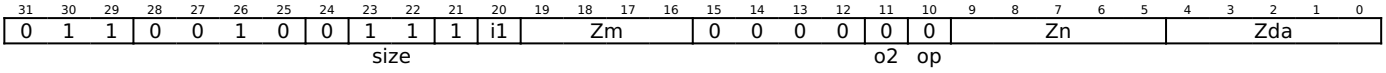
```
FMLA <Zda>.S, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
let op3_neg : boolean = FALSE;
```

Double-precision

(FEAT_SVE || FEAT_SME)



Encoding

```
FMLA <Zda>.D, <Zn>.D, <Zm>.D[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
let op3_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "Half-precision" and "Single-precision" variants: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "Double-precision" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "Half-precision" variant: is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "Single-precision" variant: is the immediate index, in the range 0 to 3, encoded in the "i2" field. For the "Double-precision" variant: is the immediate index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let elem2 : bits(esize) = op2[s*:esize];
    let elem1 : bits(esize) = (if op1_neg then FPNeg{esize}(op1[e*:esize], FPCR())
                               else op1[e*:esize]);
    let elem3 : bits(esize) = (if op3_neg then FPNeg{esize}(result[e*:esize], FPCR())
                               else result[e*:esize]);
    result[e*:esize] = FPMulAdd{esize}(elem3, elem1, elem2, FPCR());

end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

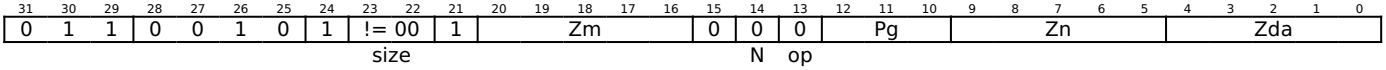
FMLA (vectors)

Floating-point fused multiply-add (predicated)

This instruction multiplies the Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector, and adds the result to elements of the third source (addend) vector without intermediate rounding. The results are destructively placed in the destination and third source (addend) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMLA <Zda>.<T>, <Pg>/M, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
let op3_neg : boolean = FALSE;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let elem1 : bits(esize) = (if op1_neg then FPNeg{esize}(op1[e*:esize], FPCR())
                                   else op1[e*:esize]);
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = (if op3_neg then FPNeg{esize}(op3[e*:esize], FPCR())
                                   else op3[e*:esize]);

        result[e*:esize] = FPMulAdd{esize}(elem3, elem1, elem2, FPCR());
    else
        result[e*:esize] = op3[e*:esize];

    end;
end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

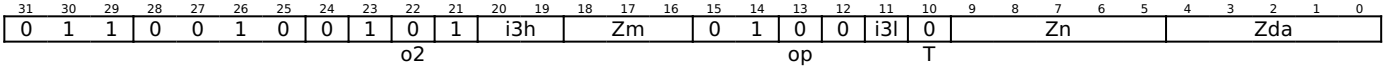
FMLALB (indexed, FP16 to FP32)

Half-precision multiply-add by indexed element to single-precision (bottom)

This instruction widens the even-numbered half-precision elements in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and then destructively multiplies and adds these values without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding half-precision elements in the first source vector. This instruction is unpredicated.

Single-precision

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMLALB <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i3h::i3l);
let op1_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 2 * segmentbase + index;
    let elem1 : bits(esize DIV 2) = (if op1_neg
                                     then FPNeg{esize DIV 2}(op1[(2*e + 0)*:(esize DIV 2)], FPCR())
                                     else op1[(2*e + 0)*:(esize DIV 2)]);
    let elem2 : bits(esize DIV 2) = op2[s*:(esize DIV 2)];
    let elem3 : bits(esize) = op3[e*:esize];
    result[e*:esize] = FPMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

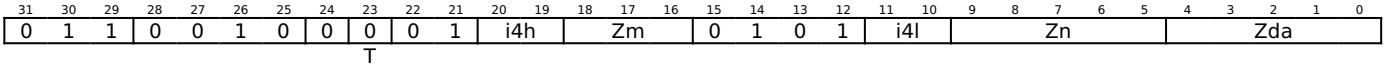
FMLALB (indexed, FP8 to FP16)

8-bit floating-point multiply-add by indexed element to half-precision (bottom)

This instruction widens the even 8-bit elements in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to half-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-UInt(FPMR.LSCALE[3:0])}$ before being destructively added without intermediate rounding to the half-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the first source vector. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.
This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALB <Zda>.H, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i4h::i4l);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 15, encoded in the "i4h:i4l" fields.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let eltspersegment : integer = 128 DIV 16;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 2 * segmentbase + index;
    let element1 : bits(8) = operand1[(2 * e + 0)*:8];
    let element2 : bits(8) = operand2[s*:8];
    let element3 : bits(16) = operand3[e*:16];
    result[e*:16] = FP8MulAddFP{16}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

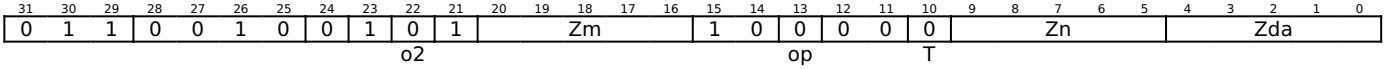
FMLALB (vectors, FP16 to FP32)

Half-precision multiply-add to single-precision (bottom)

This instruction widens the even-numbered half-precision elements in the first source vector and the corresponding elements in the second source vector to single-precision format and then destructively multiplies and adds these values without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding half-precision elements in the source vectors. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMLALB <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let elem1 : bits(esize DIV 2) = (if op1_neg
                                     then FPNeg{esize DIV 2}(op1[(2*e + 0)*:(esize DIV 2)], FPCR())
                                     else op1[(2*e + 0)*:(esize DIV 2)]);
    let elem2 : bits(esize DIV 2) = op2[(2*e + 0)*:(esize DIV 2)];
    let elem3 : bits(esize) = op3[e*:esize];
    result[e*:esize] = FPMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALB (vectors, FP8 to FP16)

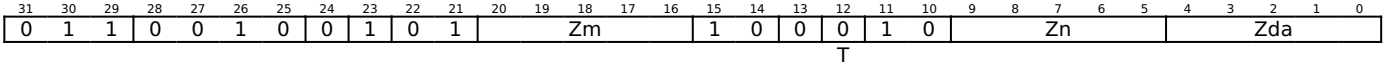
8-bit floating-point multiply-add to half-precision (bottom)

This instruction widens the even 8-bit elements in the first and second source vectors to half-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$ before being destructively added without intermediate rounding to the half-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the source vectors. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALB <Zda>.H, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(8) = operand1[(2 * e + 0)*:8];
    let element2 : bits(8) = operand2[(2 * e + 0)*:8];
    let element3 : bits(16) = operand3[e*:16];
    result[e*:16] = FP8MulAddFP{16}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALLBB (indexed)

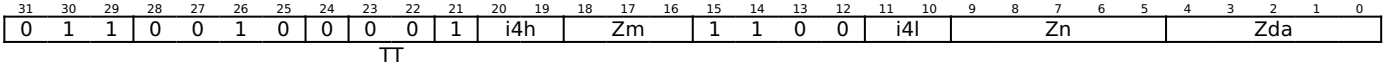
8-bit floating-point multiply-add by indexed element to single-precision (bottom bottom)

This instruction widens the first 8-bit element of each 32-bit container in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$ before being destructively added without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the first source vector. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALLBB <Zda>.S, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i4h::i4l);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 15, encoded in the "i4h:i4l" fields.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 4 * segmentbase + index;
    let element1 : bits(8) = operand1[(4 * e + 0)*:8];
    let element2 : bits(8) = operand2[s*:8];
    let element3 : bits(32) = operand3[e*:32];
    result[e*:32] = FP8MulAddFP{32}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALLBB (vectors)

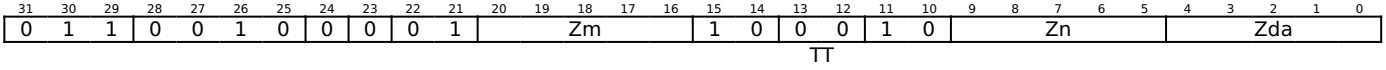
8-bit floating-point multiply-add to single-precision (bottom bottom)

This instruction widens the first 8-bit element of each 32-bit container in the first and second source vectors to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-UInt(FPMR.LSCALE)}$ before being destructively added without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the source vectors. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALLBB <Zda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(8) = operand1[(4 * e + 0)*:8];
    let element2 : bits(8) = operand2[(4 * e + 0)*:8];
    let element3 : bits(32) = operand3[e*:32];
    result[e*:32] = FP8MulAddFP{32}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALLBT (indexed)

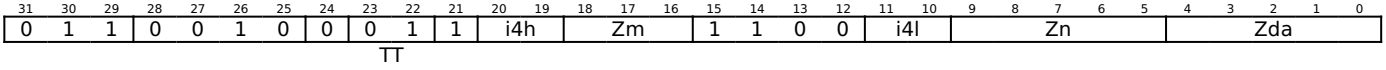
8-bit floating-point multiply-add by indexed element to single-precision (bottom top)

This instruction widens the second 8-bit element of each 32-bit container in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$ before being destructively added without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the first source vector. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALLBT <Zda>.S, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i4h::i4l);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 15, encoded in the "i4h:i4l" fields.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 4 * segmentbase + index;
    let element1 : bits(8) = operand1[(4 * e + 1)*:8];
    let element2 : bits(8) = operand2[s*:8];
    let element3 : bits(32) = operand3[e*:32];
    result[e*:32] = FP8MulAddFP{32}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALLBT (vectors)

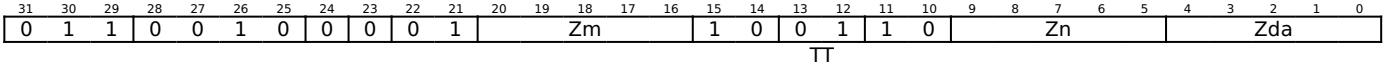
8-bit floating-point multiply-add to single-precision (bottom top)

This instruction widens the second 8-bit element of each 32-bit container in the first and second source vectors to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-UInt(FPMR.LSCALE)}$ before being destructively added without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the source vectors. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALLBT <Zda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(8) = operand1[(4 * e + 1)*:8];
    let element2 : bits(8) = operand2[(4 * e + 1)*:8];
    let element3 : bits(32) = operand3[e*:32];
    result[e*:32] = FP8MulAddFP{32}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALLTB (indexed)

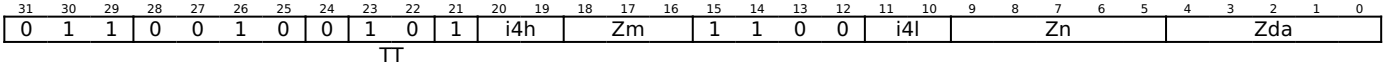
8-bit floating-point multiply-add by indexed element to single-precision (top bottom)

This instruction widens the third 8-bit element of each 32-bit container in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$ before being destructively added without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the first source vector. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALLTB <Zda>.S, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i4h::i4l);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 15, encoded in the "i4h:i4l" fields.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 4 * segmentbase + index;
    let element1 : bits(8) = operand1[(4 * e + 2)*:8];
    let element2 : bits(8) = operand2[s*:8];
    let element3 : bits(32) = operand3[e*:32];
    result[e*:32] = FP8MulAddFP{32}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALLTB (vectors)

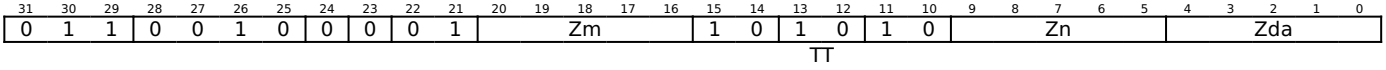
8-bit floating-point multiply-add to single-precision (top bottom)

This instruction widens the third 8-bit element of each 32-bit container in the first and second source vectors to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-UInt(FPMR.LSCALE)}$ before being destructively added without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the source vectors. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALLTB <Zda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(8) = operand1[(4 * e + 2)*:8];
    let element2 : bits(8) = operand2[(4 * e + 2)*:8];
    let element3 : bits(32) = operand3[e*:32];
    result[e*:32] = FP8MulAddFP{32}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALLTT (indexed)

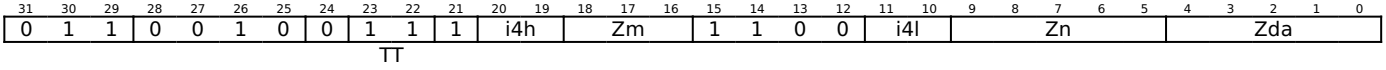
8-bit floating-point multiply-add by indexed element to single-precision (top top)

This instruction widens the fourth 8-bit element of each 32-bit container in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$ before being destructively added without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the first source vector. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALLTT <Zda>.S, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i4h::i4l);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 15, encoded in the "i4h:i4l" fields.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let eltspersegment : integer = 128 DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 4 * segmentbase + index;
    let element1 : bits(8) = operand1[(4 * e + 3)*:8];
    let element2 : bits(8) = operand2[s*:8];
    let element3 : bits(32) = operand3[e*:32];
    result[e*:32] = FP8MulAddFP{32}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALLTT (vectors)

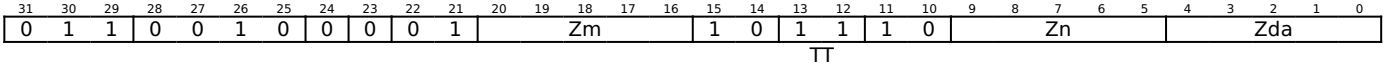
8-bit floating-point multiply-add to single-precision (top top)

This instruction widens the fourth 8-bit element of each 32-bit container in the first and second source vectors to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$ before being destructively added without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the source vectors. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALLTT <Zda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(8) = operand1[(4 * e + 3)*:8];
    let element2 : bits(8) = operand2[(4 * e + 3)*:8];
    let element3 : bits(32) = operand3[e*:32];
    result[e*:32] = FP8MulAddFP{32}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

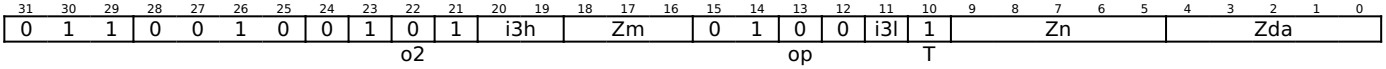
FMLALT (indexed, FP16 to FP32)

Half-precision multiply-add by indexed element to single-precision (top)

This instruction widens the odd-numbered half-precision elements in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and then destructively multiplies and adds these values without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding half-precision elements in the first source vector. This instruction is unpredicated.

Single-precision

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMLALT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i3h::i3l);
let op1_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 2 * segmentbase + index;
    let elem1 : bits(esize DIV 2) = (if op1_neg
                                     then FPNeg{esize DIV 2}(op1[(2*e + 1)*:(esize DIV 2)], FPCR())
                                     else op1[(2*e + 1)*:(esize DIV 2)]);
    let elem2 : bits(esize DIV 2) = op2[s*:(esize DIV 2)];
    let elem3 : bits(esize) = op3[e*:esize];
    result[e*:esize] = FPMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

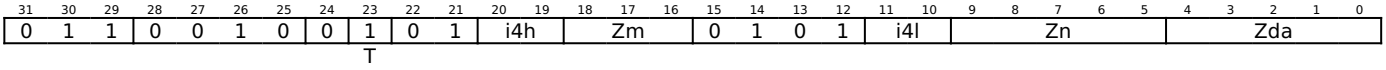
FMLALT (indexed, FP8 to FP16)

8-bit floating-point multiply-add by indexed element to half-precision (top)

This instruction widens the odd 8-bit elements in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to half-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-UInt(FPMR.LSCALE[3:0])}$ before being destructively added without intermediate rounding to the half-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the first source vector. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.
This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALT <Zda>.H, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i4h::i4l);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 15, encoded in the "i4h:i4l" fields.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let eltspersegment : integer = 128 DIV 16;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 2 * segmentbase + index;
    let element1 : bits(8) = operand1[(2 * e + 1)*:8];
    let element2 : bits(8) = operand2[s*:8];
    let element3 : bits(16) = operand3[e*:16];
    result[e*:16] = FP8MulAddFP{16}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

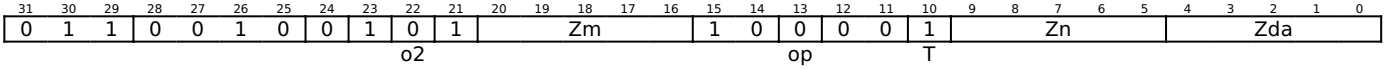
FMLALT (vectors, FP16 to FP32)

Half-precision multiply-add to single-precision (top)

This instruction widens the odd-numbered half-precision elements in the first source vector and the corresponding elements in the second source vector to single-precision format and then destructively multiplies and adds these values without intermediate rounding to the single-precision elements of the destination vector that overlap with the corresponding half-precision elements in the source vectors. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMLALT <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let elem1 : bits(esize DIV 2) = (if op1_neg
                                     then FPNeg{esize DIV 2}(op1[(2*e + 1)*:(esize DIV 2)], FPCR())
                                     else op1[(2*e + 1)*:(esize DIV 2)]);
    let elem2 : bits(esize DIV 2) = op2[(2*e + 1)*:(esize DIV 2)];
    let elem3 : bits(esize) = op3[e*:esize];
    result[e*:esize] = FPMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALT (vectors, FP8 to FP16)

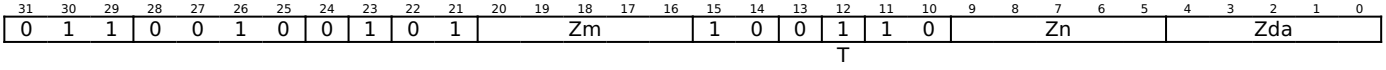
8-bit floating-point multiply-add to half-precision (top)

This instruction widens the odd 8-bit elements in the first and second source vectors to half-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-UInt(FPMR.LSCALE[3:0])}$ before being destructively added without intermediate rounding to the half-precision elements of the destination vector that overlap with the corresponding 8-bit floating-point elements in the source vectors. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SVE2

((FEAT_SVE2 && FEAT_FP8FMA) || FEAT_SSVE_FP8FMA)



Encoding

```
FMLALT <Zda>.H, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !HaveSVE2FP8FMA() then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
if IsFeatureImplemented(FEAT_SSVE_FP8FMA) && IsFeatureImplemented(FEAT_FP8FMA) then
    CheckSVEEnabled();
elsif IsFeatureImplemented(FEAT_FP8FMA) then
    CheckNonStreamingSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(8) = operand1[(2 * e + 1)*:8];
    let element2 : bits(8) = operand2[(2 * e + 1)*:8];
    let element3 : bits(16) = operand3[e*:16];
    result[e*:16] = FP8MulAddFP{16}(element3, element1, element2, FPCR(), FPMR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLS (indexed)

Floating-point fused multiply-subtract by indexed element

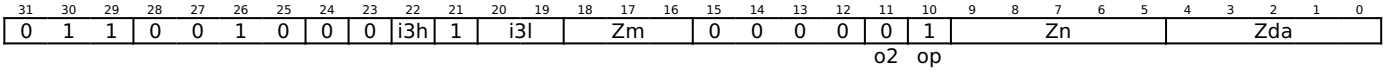
This instruction multiplies all floating-point elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment. The products are then destructively subtracted without intermediate rounding from the corresponding elements of the addend and destination vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment. This instruction is unpredicated.

It has encodings from 3 classes: Half-precision , Single-precision and Double-precision

Half-precision

(FEAT_SVE || FEAT_SME)



Encoding

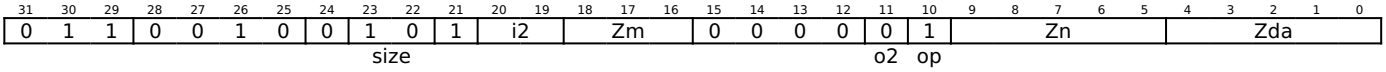
```
FMLS <Zda>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
let op3_neg : boolean = FALSE;
```

Single-precision

(FEAT_SVE || FEAT_SME)



Encoding

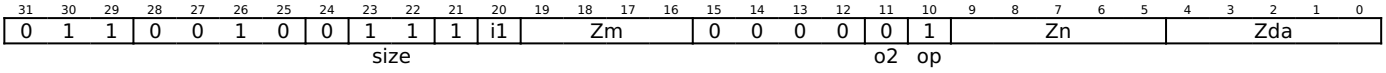
```
FMLS <Zda>.S, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
let op3_neg : boolean = FALSE;
```

Double-precision

(FEAT_SVE || FEAT_SME)



Encoding

```
FMLS <Zda>.D, <Zn>.D, <Zm>.D[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
let op3_neg : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "Half-precision" and "Single-precision" variants: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "Double-precision" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "Half-precision" variant: is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "Single-precision" variant: is the immediate index, in the range 0 to 3, encoded in the "i2" field. For the "Double-precision" variant: is the immediate index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let elem2 : bits(esize) = op2[s*:esize];
    let elem1 : bits(esize) = (if op1_neg then FPNeg{esize}(op1[e*:esize], FPCR())
                               else op1[e*:esize]);
    let elem3 : bits(esize) = (if op3_neg then FPNeg{esize}(result[e*:esize], FPCR())
                               else result[e*:esize]);
    result[e*:esize] = FPMulAdd{esize}(elem3, elem1, elem2, FPCR());

end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

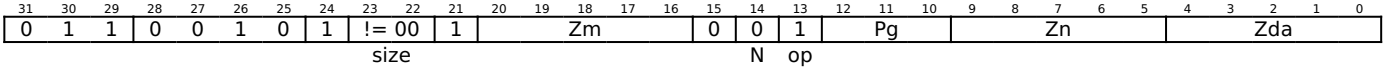
FMLS (vectors)

Floating-point fused multiply-subtract (predicated)

This instruction multiplies the Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector, and subtracts the result from elements of the third source (addend) vector without intermediate rounding. The results are destructively placed in the destination and third source (addend) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMLS <Zda>.<T>, <Pg>/M, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
let op3_neg : boolean = FALSE;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let elem1 : bits(esize) = (if op1_neg then FPNeg{esize}(op1[e*:esize], FPCR())
                                   else op1[e*:esize]);
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = (if op3_neg then FPNeg{esize}(op3[e*:esize], FPCR())
                                   else op3[e*:esize]);

        result[e*:esize] = FPMulAdd{esize}(elem3, elem1, elem2, FPCR());
    else
        result[e*:esize] = op3[e*:esize];

    end;
end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

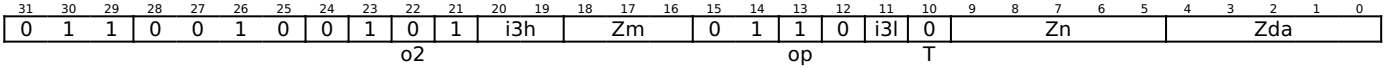
FMLSLB (indexed)

Half-precision multiply-subtract by indexed element from single-precision (bottom)

This instruction widens the even-numbered half-precision elements in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and then destructively multiplies and subtracts these values without intermediate rounding from the single-precision elements of the destination vector that overlap with the corresponding half-precision elements in the first source vector. This instruction is unpredicated.

Single-precision

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMLSLB <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i3h::i3l);
let op1_neg : boolean = TRUE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 2 * segmentbase + index;
    let elem1 : bits(esize DIV 2) = (if op1_neg
                                     then FPNeg{esize DIV 2}(op1[(2*e + 0)*:(esize DIV 2)], FPCR())
                                     else op1[(2*e + 0)*:(esize DIV 2)]);
    let elem2 : bits(esize DIV 2) = op2[s*:(esize DIV 2)];
    let elem3 : bits(esize) = op3[e*:esize];
    result[e*:esize] = FPMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

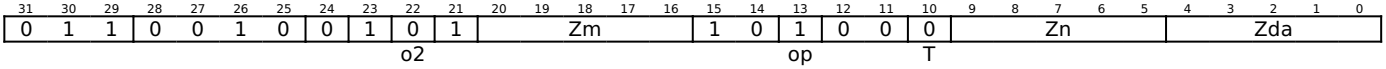
FMLSLB (vectors)

Half-precision multiply-subtract from single-precision (bottom)

This instruction widens the even-numbered half-precision elements in the first source vector and the corresponding elements in the second source vector to single-precision format and then destructively multiplies and subtracts these values without intermediate rounding from the single-precision elements of the destination vector that overlap with the corresponding half-precision elements in the source vectors. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMLSLB <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let elem1 : bits(esize DIV 2) = (if op1_neg
                                     then FPNeg{esize DIV 2}(op1[(2*e + 0)*:(esize DIV 2)], FPCR())
                                     else op1[(2*e + 0)*:(esize DIV 2)]);
    let elem2 : bits(esize DIV 2) = op2[(2*e + 0)*:(esize DIV 2)];
    let elem3 : bits(esize) = op3[e*:esize];
    result[e*:esize] = FPMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

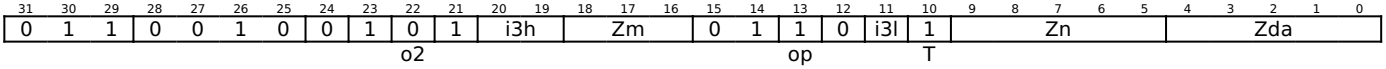
FMLSLT (indexed)

Half-precision multiply-subtract by indexed element from single-precision (top)

This instruction widens the odd-numbered half-precision elements in the first source vector and the indexed element from the corresponding 128-bit segment in the second source vector to single-precision format and then destructively multiplies and subtracts these values without intermediate rounding from the single-precision elements of the destination vector that overlap with the corresponding half-precision elements in the first source vector. This instruction is unpredicated.

Single-precision

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMLSLT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let index : integer = UInt(i3h::i3l);
let op1_neg : boolean = TRUE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = 2 * segmentbase + index;
    let elem1 : bits(esize DIV 2) = (if op1_neg
                                     then FPNeg{esize DIV 2}(op1[(2*e + 1)*:(esize DIV 2)], FPCR())
                                     else op1[(2*e + 1)*:(esize DIV 2)]);
    let elem2 : bits(esize DIV 2) = op2[s*:(esize DIV 2)];
    let elem3 : bits(esize) = op3[e*:esize];
    result[e*:esize] = FPMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

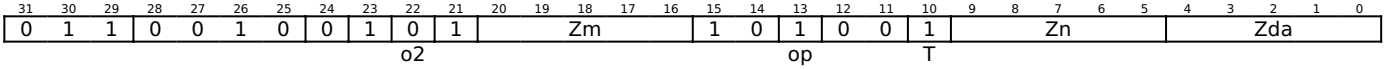
FMLSLT (vectors)

Half-precision multiply-subtract from single-precision (top)

This instruction widens the odd-numbered half-precision elements in the first source vector and the corresponding elements in the second source vector to single-precision format and then destructively multiplies and subtracts these values without intermediate rounding from the single-precision elements of the destination vector that overlap with the corresponding half-precision elements in the source vectors. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
FMLSLT <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let elem1 : bits(esize DIV 2) = (if op1_neg
                                     then FPNeg{esize DIV 2}(op1[(2*e + 1)*:(esize DIV 2)], FPCR())
                                     else op1[(2*e + 1)*:(esize DIV 2)]);
    let elem2 : bits(esize DIV 2) = op2[(2*e + 1)*:(esize DIV 2)];
    let elem3 : bits(esize) = op3[e*:esize];
    result[e*:esize] = FPMulAddH(elem3, elem1, elem2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMMLA (non-widening)

Floating-point matrix multiply-accumulate

This instruction supports half-precision, single-precision, and double-precision data types in a 2x2 matrix contained in segments of 64, 128, or 256 bits, respectively. It multiplies the 2x2 matrix in each segment of the first source vector by the 2x2 matrix in the corresponding segment of the second source vector. The intermediate products are rounded before they are summed, and the intermediate sum is rounded before accumulation into the 2x2 matrix held in the corresponding segment of the addend and destination vector. This is equivalent to performing a 2-way dot product per destination element. This instruction is unpredicated. The half-precision and single-precision variants are vector length agnostic. The double-precision variant requires that the Effective SVE vector length is at least 256 bits.

ID_AA64ZFR0_EL1.F32MM indicates whether the single-precision variant is implemented.

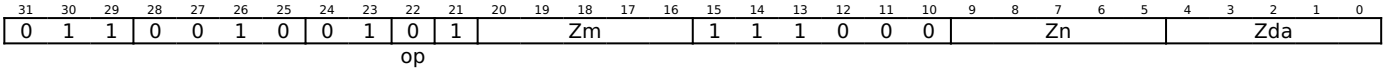
ID_AA64ZFR0_EL1.F64MM indicates whether the double-precision variant is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE2p2 && FEAT_F16MM)



Encoding

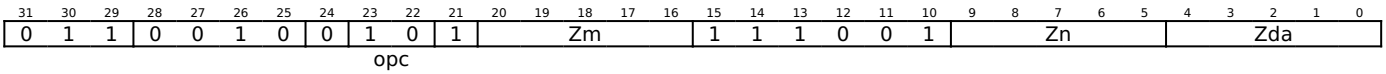
```
FMMLA <Zda>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) || !IsFeatureImplemented(FEAT_F16MM) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

32-bit element

(FEAT_F32MM)



Encoding

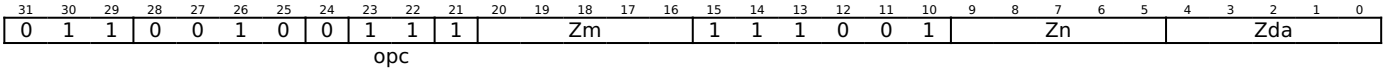
```
FMMLA <Zda>.S, <Zn>.S, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F32MM) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

64-bit element

(FEAT_F64MM)



Encoding

```
FMMLA <Zda>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < esize * 4 then EndOfDecode(Decode_UNDEF); end;
let segments : integer = VL DIV (4 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL) = Zeros{};
var op1, op2 : bits(4*esize);
var res, addend : bits(4*esize);

for s = 0 to segments-1 do
    op1    = operand1[s*:(4*esize)];
    op2    = operand2[s*:(4*esize)];
    addend = operand3[s*:(4*esize)];
    res    = FPMatMulAdd{4*esize}(addend, op1, op2, esize, FPCR());
    result[s*:(4*esize)] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMMLA (widening, FP16 to FP32)

Half-precision matrix multiply-accumulate to single-precision

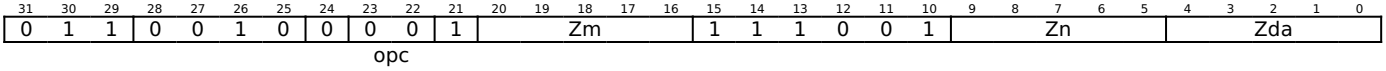
This instruction multiplies the 2×4 matrix of half-precision values held in each 128-bit segment of the first source vector by the 4×2 matrix of half-precision values in the corresponding segment of the second source vector. The intermediate single-precision results are calculated as a sum of two fused sum-of-products using two pairs of adjacent half-precision values from each of the source vectors, and rounded before accumulation into the 2×2 matrix of single-precision results in the corresponding segment of the destination vector. This is equivalent to performing a 4-way dot product per destination element.

This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE_F16F32MM)



Encoding

```
FMMLA <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_F16F32MM) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer =  VL DIV 128;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for s = 0 to segments-1 do
    let op1 : bits(128)    = operand1[s*:(128)];
    let op2 : bits(128)    = operand2[s*:(128)];
    let addend : bits(128) = operand3[s*:(128)];
    result[s*:(128)] = FPMatMulAddH{128}(addend, op1, op2, FPCR());
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMMLA (widening, FP8 to FP16)

8-bit floating-point matrix multiply-accumulate to half-precision

This instruction multiplies the 2×4 matrix of 8-bit floating-point values held in each 64-bit segment of the first source vector by the 4×2 matrix of 8-bit floating-point values in the corresponding segment of the second source vector. The intermediate half-precision results calculated as a fused sum-of-products using four adjacent 8-bit floating-point values from each of the source vectors, are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$, before being destructively added without intermediate rounding to the 2×2 half-precision matrix in the corresponding segment of the destination vector. This is equivalent to accumulating 4-way dot product per destination element.

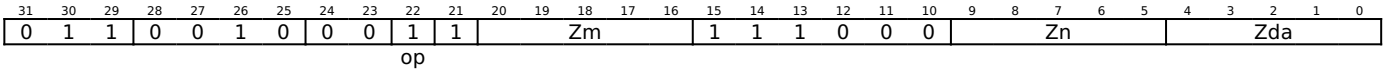
The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2 && FEAT_F8F16MM)



Encoding

```
FMMLA <Zda>.H, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) || !IsFeatureImplemented(FEAT_F8F16MM) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer =  VL DIV 64;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for s = 0 to segments-1 do
    let op1 : bits(64)    = operand1[s*:(64)];
    let op2 : bits(64)    = operand2[s*:(64)];
    let addend : bits(64) = operand3[s*:(64)];
    let way : integer{} = 4;
    result[s*:(64)] = FP8MatMulAddFP{64}(addend, op1, op2, way, FPCR(), FPMR());

end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMMLA (widening, FP8 to FP32)

8-bit floating-point matrix multiply-accumulate to single-precision

This instruction multiplies the 2×8 matrix of 8-bit floating-point values held in each 128-bit segment of the first source vector by the 8×2 matrix of 8-bit floating-point values in the corresponding segment of the second source vector. The intermediate single-precision results calculated as a fused sum-of-products using eight adjacent 8-bit floating-point values from each of the source vectors, are scaled by $2^{-\text{UInt(FPMR.LSCALE)}}$, before being destructively added without intermediate rounding to the 2×2 single-precision matrix in the corresponding segment of the destination vector. This is equivalent to accumulating 8-way dot product per destination element.

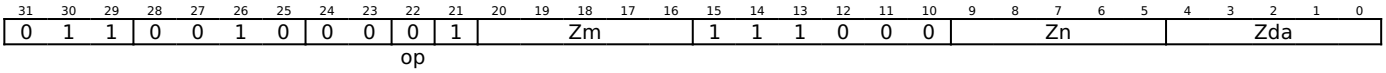
The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2 && FEAT_F8F32MM)



Encoding

```
FMMLA <Zda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) || !IsFeatureImplemented(FEAT_F8F32MM) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer =  VL DIV 128;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for s = 0 to segments-1 do
    let op1 : bits(128)    = operand1[s*:(128)];
    let op2 : bits(128)    = operand2[s*:(128)];
    let addend : bits(128) = operand3[s*:(128)];
    let way : integer{} = 8;
    result[s*:(128)] = FP8MatMulAddFP{128}(addend, op1, op2, way, FPCR(), FPMR());

end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOV (immediate, predicated)

Move floating-point immediate to vector elements (predicated)

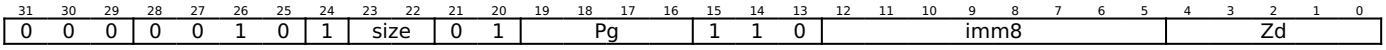
This instruction moves a floating-point immediate into each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.

This is an alias of FCPY. This means:

- The encodings in this description are named to match the encodings of FCPY.
- The description of FCPY gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMOV <Zd>.<T>, <Pg>/M, #<const>
```

is equivalent to

```
FCPY <Zd>.<T>, <Pg>/M, #<const>
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<const> Is a floating-point immediate value expressible as ±n÷16×2^r, where n and r are integers such that 16 ≤ n ≤ 31 and -3 ≤ r ≤ 4, i.e. a normalized binary floating-point encoding with 1 sign bit, 3-bit exponent, and 4-bit fractional part, encoded in the "imm8" field.

Operation

The description of FCPY gives the operational pseudocode for this instruction.

Operational Information

The description of FCPY gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOV (immediate, unpredicated)

Move floating-point immediate to vector elements (unpredicated)

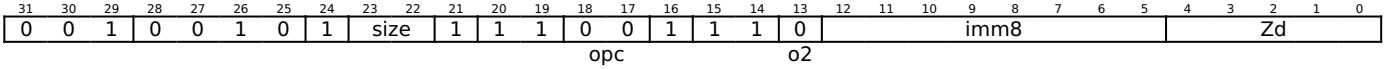
This instruction unconditionally broadcasts the floating-point immediate into each element of the destination vector. This instruction is unpredicated.

This is an alias of FDUP. This means:

- The encodings in this description are named to match the encodings of FDUP.
- The description of FDUP gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMOV <Zd>.<T>, #<const>
```

is equivalent to

```
FDUP <Zd>.<T>, #<const>
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<const> Is a floating-point immediate value expressible as ±n÷16×2^r, where n and r are integers such that 16 ≤ n ≤ 31 and -3 ≤ r ≤ 4, i.e. a normalized binary floating-point encoding with 1 sign bit, 3-bit exponent, and 4-bit fractional part, encoded in the "imm8" field.

Operation

The description of FDUP gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOV (zero, predicated)

Move floating-point +0.0 to vector elements (predicated)

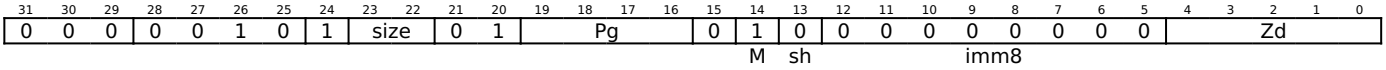
This instruction moves floating-point constant +0.0 to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.

This is a pseudo-instruction of CPY (immediate, merging). This means:

- The encodings in this description are named to match the encodings of CPY (immediate, merging).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of CPY (immediate, merging) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMOV <Zd>.<T>, <Pg>/M, #0.0
```

is equivalent to

```
CPY <Zd>.<T>, <Pg>/M, #0
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

Operation

The description of CPY (immediate, merging) gives the operational pseudocode for this instruction.

Operational Information

The description of CPY (immediate, merging) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOV (zero, unpredicated)

Move floating-point +0.0 to vector elements (unpredicated)

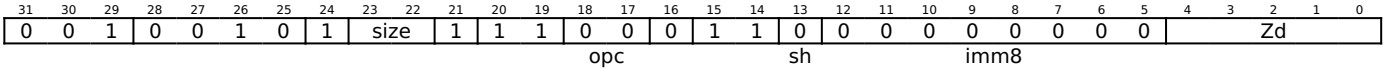
This instruction unconditionally broadcasts the floating-point constant +0.0 into each element of the destination vector. This instruction is unpredicated.

This is a pseudo-instruction of DUP (immediate). This means:

- The encodings in this description are named to match the encodings of DUP (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of DUP (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMOV <Zd>.<T>, #0.0
```

is equivalent to

```
DUP <Zd>.<T>, #0
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

Operation

The description of DUP (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

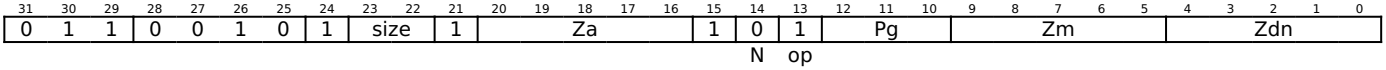
FMSB

Floating-point fused multiply-subtract to multiplicand (predicated)

This instruction multiplies the Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector, and subtracts the result from elements of the third (addend) vector without intermediate rounding. The results are destructively placed in the destination and first source (multiplicand) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMSB <Zdn>.<T>, <Pg>/M, <Zm>.<T>, <Za>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let a : integer = UInt(Za);
let op1_neg : boolean = TRUE;
let op3_neg : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Za> Is the name of the third source scalable vector register, encoded in the "Za" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = Z{}(dn);
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(a) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let elem1 : bits(esize) = (if op1_neg then FPNeg{esize}(op1[e*:esize], FPCR())
                                   else op1[e*:esize]);
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = (if op3_neg then FPNeg{esize}(op3[e*:esize], FPCR())
                                   else op3[e*:esize]);

        result[e*:esize] = FPMulAdd{esize}(elem3, elem1, elem2, FPCR());
    else
        result[e*:esize] = op1[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

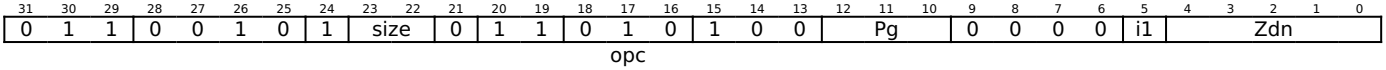
FMUL (immediate)

Floating-point multiply by immediate (predicated)

This instruction multiplies each Active floating-point element of the source vector by an immediate, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.5 or +2.0 only. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMUL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then FPPointFive{esize}('0') else FPTwo{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.5
1	#2.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPMul{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMUL (indexed)

Floating-point multiply by indexed element

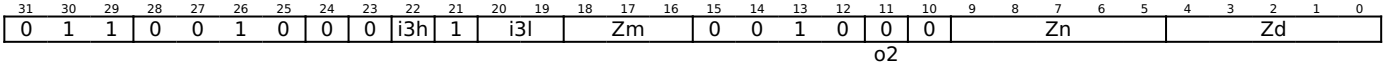
This instruction multiplies all floating-point elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment. The results are placed in the corresponding elements of the destination vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment. This instruction is unpredicated.

It has encodings from 3 classes: Half-precision , Single-precision and Double-precision

Half-precision

(FEAT_SVE || FEAT_SME)



Encoding

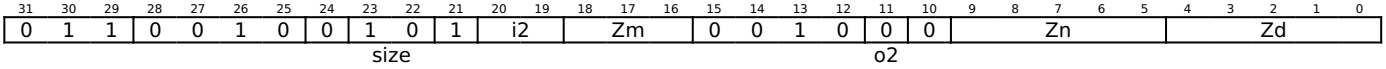
```
FMUL <Zd>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Single-precision

(FEAT_SVE || FEAT_SME)



Encoding

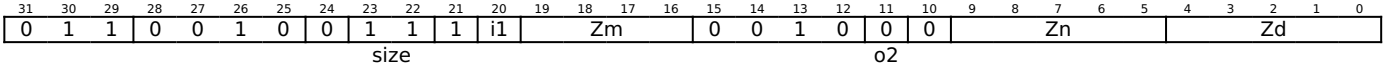
```
FMUL <Zd>.S, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Double-precision

(FEAT_SVE || FEAT_SME)



Encoding

```
FMUL <Zd>.D, <Zn>.D, <Zm>.D[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "Half-precision" and "Single-precision" variants: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "Double-precision" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "Half-precision" variant: is the immediate index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "Single-precision" variant: is the immediate index, in the range 0 to 3, encoded in the "i2" field. For the "Double-precision" variant: is the immediate index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let op1 : bits(VL) = Z{}(n);
let op2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let elem2 : bits(esize) = op2[s*:esize];
    let elem1 : bits(esize) = op1[e*:esize];
    result[e*:esize] = FPMul{esize}(elem1, elem2, FPCR());

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

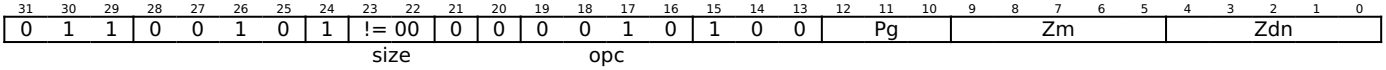
FMUL (vectors, predicated)

Floating-point multiply (predicated)

This instruction multiplies Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMUL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPMul{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

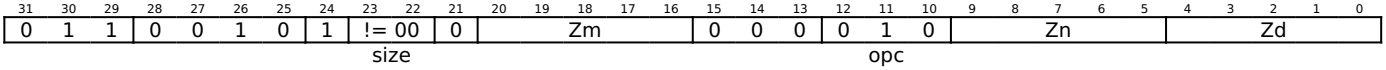
FMUL (vectors, unpredicated)

Floating-point multiply (unpredicated)

This instruction multiplies all elements of the first source vector by the corresponding floating-point elements of the second source vector and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMUL <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPMul{esize}(element1, element2, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMULX

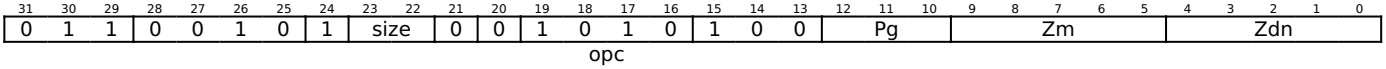
Floating-point multiply extended (predicated)

This instruction multiplies Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector except that ∞×0.0 gives 2.0 instead of NaN, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

The instruction can be used with FRECPX to safely convert arbitrary elements in mathematical vector space to unit vectors or direction vectors with length 1.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FMULX <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPMulX{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FNEG

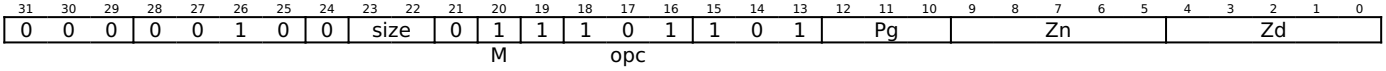
Floating-point negate (predicated)

This instruction negates each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. This inverts the sign bit and cannot signal a floating-point exception. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

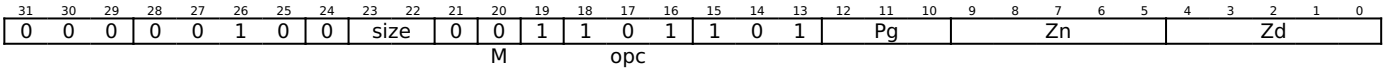
```
FNEG <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FNEG <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = FPNeg{esize}(element, FPCR());

    end;
end;
Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

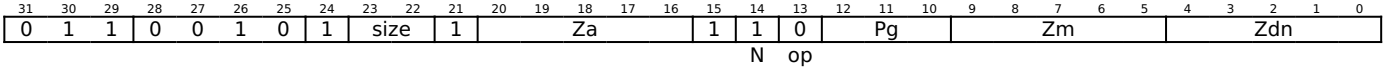
FNMAD

Floating-point negated fused multiply-add to multiplicand (predicated)

This instruction multiplies the Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector, and adds the result to elements of the third (addend) vector without intermediate rounding. The negated results are destructively placed in the destination and first source (multiplicand) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FNMAD <Zdn>.<T>, <Pg>/M, <Zm>.<T>, <Za>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let a : integer = UInt(Za);
let op1_neg : boolean = TRUE;
let op3_neg : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Za> Is the name of the third source scalable vector register, encoded in the "Za" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = Z{}(dn);
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(a) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let elem1 : bits(esize) = (if op1_neg then FPNeg{esize}(op1[e*:esize], FPCR())
                                   else op1[e*:esize]);
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = (if op3_neg then FPNeg{esize}(op3[e*:esize], FPCR())
                                   else op3[e*:esize]);

        result[e*:esize] = FPMulAdd{esize}(elem3, elem1, elem2, FPCR());
    else
        result[e*:esize] = op1[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

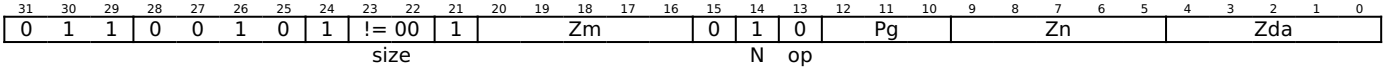
FNMLA

Floating-point negated fused multiply-add (predicated)

This instruction multiplies the Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector, and adds the result to elements of the third source (addend) vector without intermediate rounding. The negated results are destructively placed in the destination and third source (addend) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FNMLA <Zda>.<T>, <Pg>/M, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = TRUE;
let op3_neg : boolean = TRUE;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let elem1 : bits(esize) = (if op1_neg then FPNeg{esize}(op1[e*:esize], FPCR())
                                   else op1[e*:esize]);
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = (if op3_neg then FPNeg{esize}(op3[e*:esize], FPCR())
                                   else op3[e*:esize]);

        result[e*:esize] = FPMulAdd{esize}(elem3, elem1, elem2, FPCR());
    else
        result[e*:esize] = op3[e*:esize];

    end;
end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

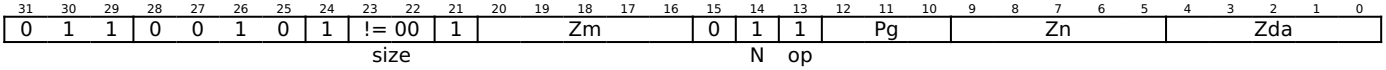
FNMLS

Floating-point negated fused multiply-subtract (predicated)

This instruction multiplies the Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector, and subtracts the result from elements of the third source (addend) vector without intermediate rounding. The negated results are destructively placed in the destination and third source (addend) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FNMLS <Zda>.<T>, <Pg>/M, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_neg : boolean = FALSE;
let op3_neg : boolean = TRUE;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let elem1 : bits(esize) = (if op1_neg then FPNeg{esize}(op1[e*:esize], FPCR())
                                   else op1[e*:esize]);
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = (if op3_neg then FPNeg{esize}(op3[e*:esize], FPCR())
                                   else op3[e*:esize]);

        result[e*:esize] = FPMulAdd{esize}(elem3, elem1, elem2, FPCR());
    else
        result[e*:esize] = op3[e*:esize];

    end;
end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

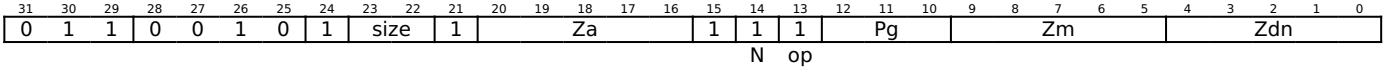
FNMSB

Floating-point negated fused multiply-subtract to multiplicand (predicated)

This instruction multiplies the Active floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector, and subtracts the result from elements of the third (addend) vector without intermediate rounding. The negated results are destructively placed in the destination and first source (multiplicand) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FNMSB <Zdn>.<T>, <Pg>/M, <Zm>.<T>, <Za>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let a : integer = UInt(Za);
let op1_neg : boolean = FALSE;
let op3_neg : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Za> Is the name of the third source scalable vector register, encoded in the "Za" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let op1 : bits(VL) = Z{}(dn);
let op2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let op3 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(a) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let elem1 : bits(esize) = (if op1_neg then FPNeg{esize}(op1[e*:esize], FPCR())
                                   else op1[e*:esize]);
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = (if op3_neg then FPNeg{esize}(op3[e*:esize], FPCR())
                                   else op3[e*:esize]);

        result[e*:esize] = FPMulAdd{esize}(elem3, elem1, elem2, FPCR());
    else
        result[e*:esize] = op1[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

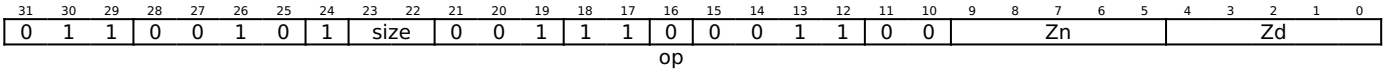
FRECPE

Floating-point reciprocal estimate (unpredicated)

This instruction finds the approximate reciprocal of each floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FRECPE <Zd>.<T>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[e*:esize];
    result[e*:esize] = FPRecipEstimate{esize}(element, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRECPS

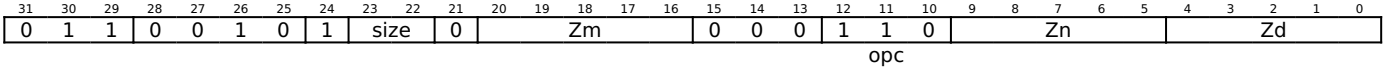
Floating-point reciprocal step (unpredicated)

This instruction multiplies the floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector, subtracts the products from 2.0 without intermediate rounding, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

This instruction can be used to perform a single Newton-Raphson iteration for calculating the reciprocal of a vector of floating-point values.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FRECPS <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPRecipStepFused{esize}(element1, element2, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRECPX

Floating-point reciprocal exponent (predicated)

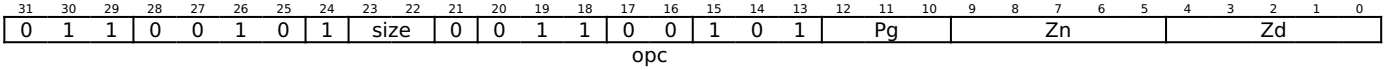
This instruction inverts the exponent, zeroes the fractional part of each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

The result of this instruction can be used with FMULX to convert arbitrary elements in mathematical vector space to "unit vectors" or "direction vectors" of length 1.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

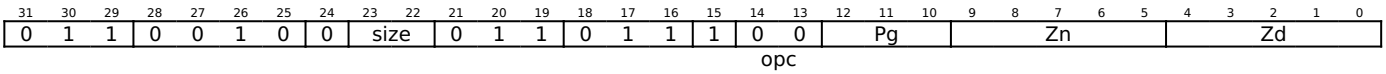
```
FRECPX <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FRECPX <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = FPRecpX{esize}(element, FPCR());
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT32X

Floating-point round to 32-bit integer (predicated)

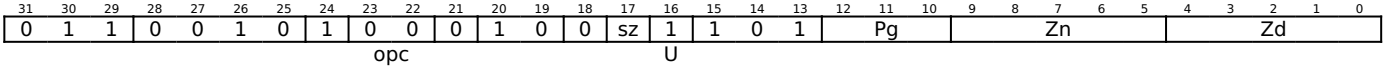
This instruction rounds to integral floating-point values that fit into a 32-bit integer size using the rounding mode that is determined by the FPCR from each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector.

Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

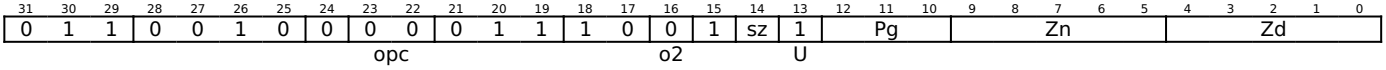
```
FRINT32X <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let intsize : integer{} = 32;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FRINT32X <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let intsize : integer{} = 32;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = FPRoundIntN{esize}(element, FPCR(), rounding, intsize);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT32Z

Floating-point round to 32-bit integer, rounding toward zero (predicated)

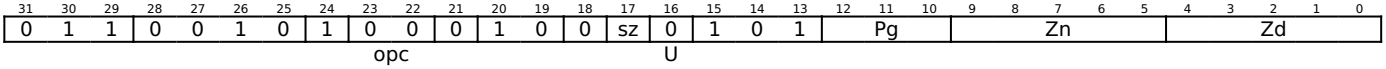
This instruction rounds to integral floating-point values that fit into a 32-bit integer size using the round towards zero rounding mode from each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector.

Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

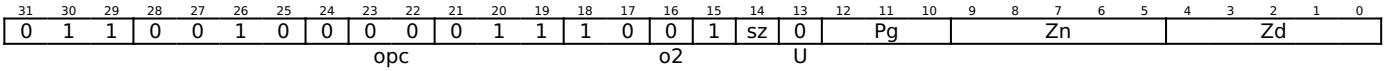
```
FRINT32Z <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let intsize : integer{} = 32;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FRINT32Z <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let intsize : integer{} = 32;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = FPRoundIntN{esize}(element, FPCR(), rounding, intsize);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT64X

Floating-point round to 64-bit integer (predicated)

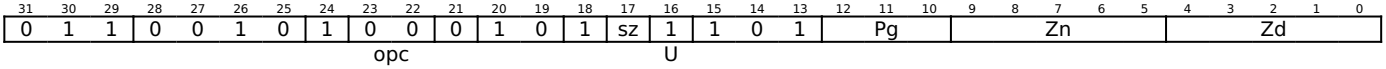
This instruction rounds to integral floating-point values that fit into a 64-bit integer size using the rounding mode that is determined by the FPCR from each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector.

Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

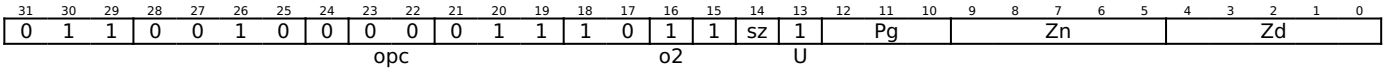
```
FRINT64X <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let intsize : integer{} = 64;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FRINT64X <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let intsize : integer{} = 64;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = FPRoundIntN{esize}(element, FPCR(), rounding, intsize);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT64Z

Floating-point round to 64-bit integer, rounding toward zero (predicated)

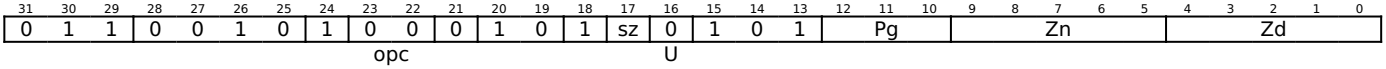
This instruction rounds to integral floating-point values that fit into a 64-bit integer size using the round towards zero rounding mode from each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector.

Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

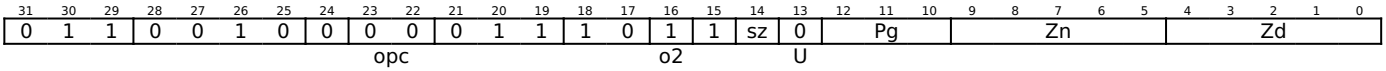
```
FRINT64Z <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let intsize : integer{} = 64;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FRINT64Z <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let intsize : integer{} = 64;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = FPRoundIntN{esize}(element, FPCR(), rounding, intsize);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINT<r>

Floating-point round to integral value (predicated)

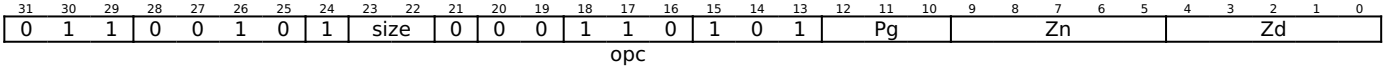
This instruction rounds to an integral floating-point value with the specified rounding option from each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

<r>	Rounding Option
N	to nearest, with ties to even
A	to nearest, with ties away from zero
M	toward minus Infinity
P	toward plus Infinity
Z	toward zero
I	current FPCR rounding mode
X	current FPCR rounding mode, signaling inexact

It has encodings from 14 classes: Current mode signaling inexact, merging , Current mode signaling inexact, zeroing , Current mode, merging , Current mode, zeroing , Nearest with ties to away, merging , Nearest with ties to away, zeroing , Nearest with ties to even, merging , Nearest with ties to even, zeroing , Toward zero, merging , Toward zero, zeroing , Toward minus infinity, merging , Toward minus infinity, zeroing , Toward plus infinity, merging and Toward plus infinity, zeroing

Current mode signaling inexact, merging

(FEAT_SVE || FEAT_SME)



Encoding

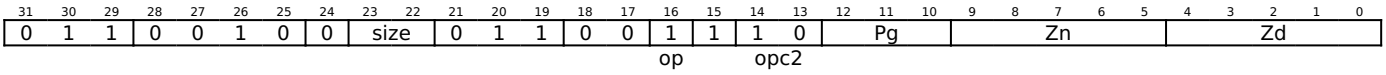
```
FRINTX <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

Current mode signaling inexact, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

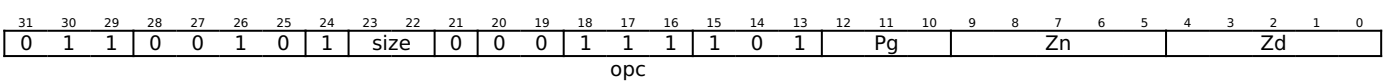
```
FRINTX <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

Current mode, merging

(FEAT_SVE || FEAT_SME)



Encoding

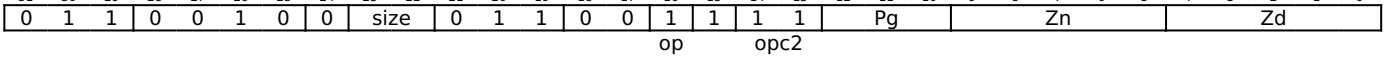
```
FRINTI <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

Current mode, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

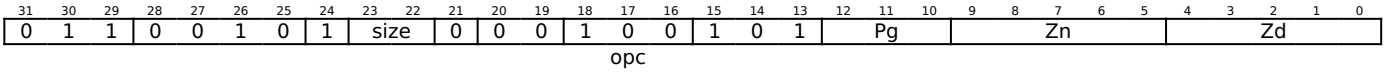
```
FRINTI <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

Nearest with ties to away, merging

(FEAT_SVE || FEAT_SME)



Encoding

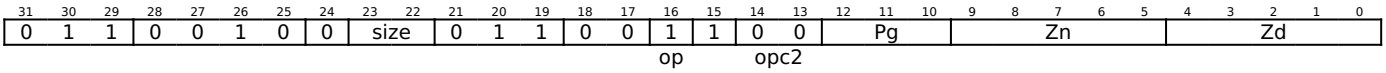
```
FRINTA <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEAWAY;
let merging : boolean = TRUE;
```

Nearest with ties to away, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

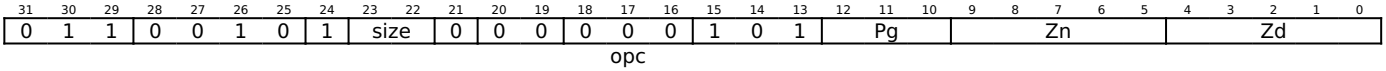
```
FRINTA <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEAWAY;
let merging : boolean = FALSE;
```

Nearest with ties to even, merging

(FEAT_SVE || FEAT_SME)



Encoding

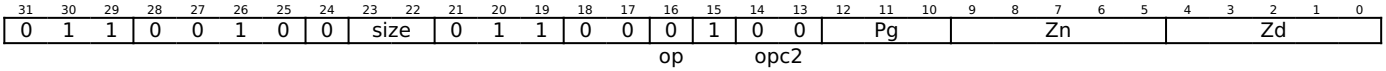
```
FRINTN <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEEVEN;
let merging : boolean = TRUE;
```

Nearest with ties to even, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

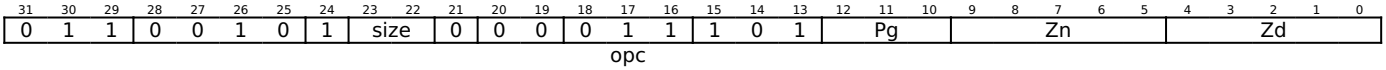
```
FRINTN <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEEVEN;
let merging : boolean = FALSE;
```

Toward zero, merging

(FEAT_SVE || FEAT_SME)



Encoding

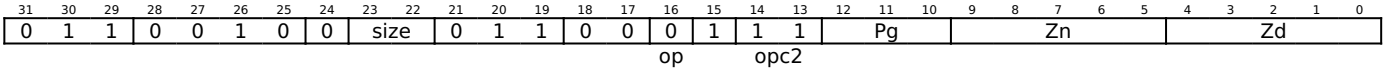
```
FRINTZ <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = TRUE;
```

Toward zero, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

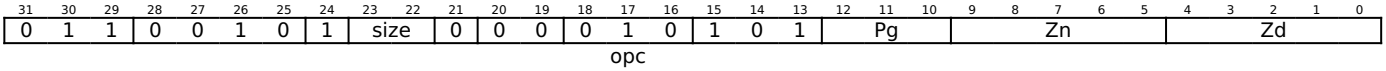
```
FRINTZ <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
let merging : boolean = FALSE;
```

Toward minus infinity, merging

(FEAT_SVE || FEAT_SME)



Encoding

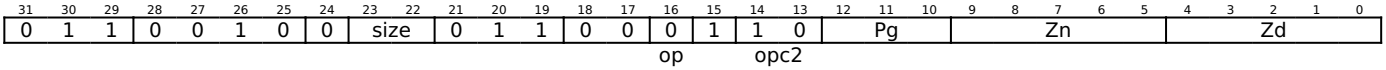
```
FRINTM <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_NEGINF;
let merging : boolean = TRUE;
```

Toward minus infinity, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

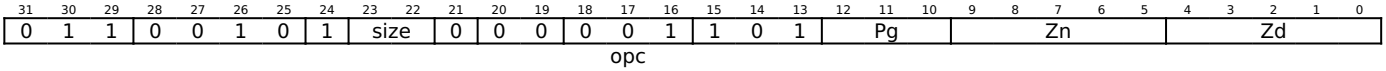
```
FRINTM <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_NEGINF;
let merging : boolean = FALSE;
```

Toward plus infinity, merging

(FEAT_SVE || FEAT_SME)



Encoding

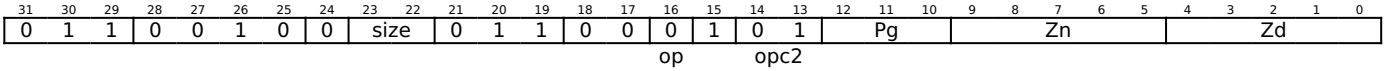
```
FRINTP <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_POSINF;
let merging : boolean = TRUE;
```

Toward plus infinity, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FRINTP <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_POSINF;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = FPRoundInt{esize}(element, FPCR(), rounding, exact);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Current mode signaling inexact, merging" , "Current mode, merging" , "Nearest with ties to away, merging" , "Nearest with ties to even, merging" , "Toward zero, merging" , "Toward minus infinity, merging" , "Toward plus infinity, merging" variants:
The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

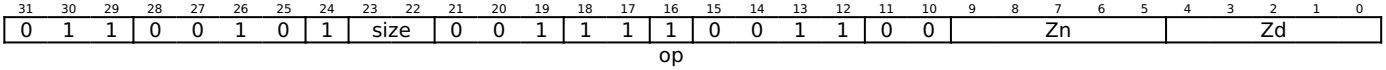
FRSQRTE

Floating-point reciprocal square root estimate (unpredicated)

This instruction finds the approximate reciprocal square root of each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FRSQRTE <Zd>.<T>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[e*:esize];
    result[e*:esize] = FPRSqrtEstimate{esize}(element, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRSQRTS

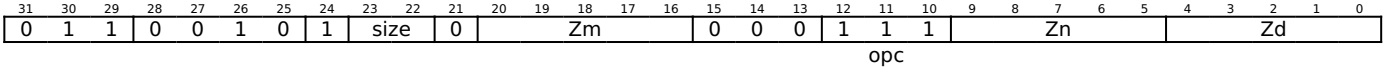
Floating-point reciprocal square root step (unpredicated)

This instruction multiplies the floating-point elements of the first source vector by the corresponding floating-point elements of the second source vector, subtracts the products from 3.0 and divides the results by 2.0 without any intermediate rounding, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

This instruction can be used to perform a single Newton-Raphson iteration for calculating the reciprocal square root of a vector of floating-point values.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FRSQRTS <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPRSqrtStepFused{esize}(element1, element2, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

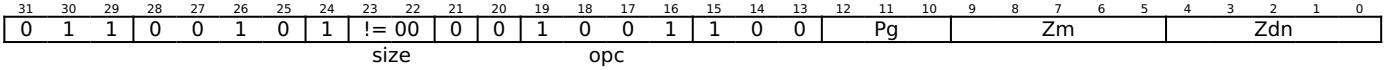
FSCALE

Floating-point adjust exponent (predicated)

This instruction multiplies the Active floating-point elements of the first source vector by 2.0 to the power of the signed integer values in the corresponding elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FSCALE <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : integer = SInt(operand2[e*:esize]);
        result[e*:esize] = FPScale{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FSQRT

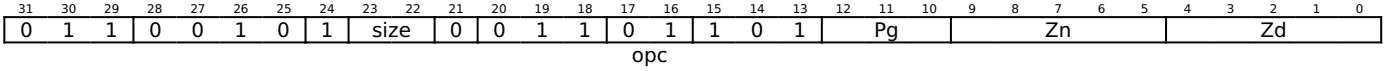
Floating-point square root (predicated)

This instruction calculates the square root of each Active floating-point element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

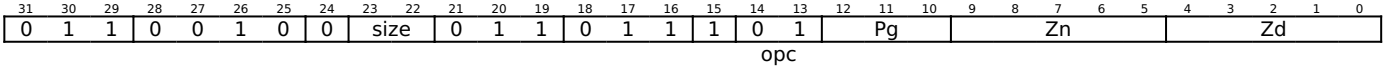
```
FSQRT <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
FSQRT <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = FPSqrt{esize}(element, FPCR());
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

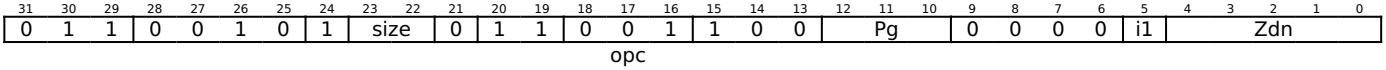
FSUB (immediate)

Floating-point subtract immediate (predicated)

This instruction subtracts an immediate from each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.5 or +1.0 only. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FSUB <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then FPPointFive{esize}('0') else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.5
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPSub{esize}(element1, imm, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

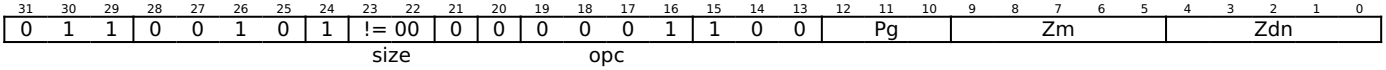
FSUB (vectors, predicated)

Floating-point subtract (predicated)

This instruction subtracts Active floating-point elements of the second source vector from the corresponding floating-point elements of the first source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FSUB <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPSub{esize}(element1, element2, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

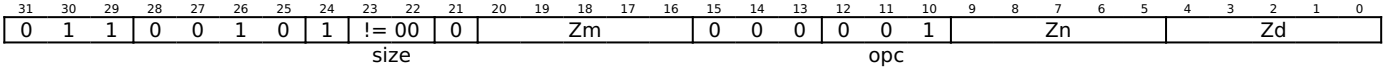
FSUB (vectors, unpredicated)

Floating-point subtract (unpredicated)

This instruction subtracts all floating-point elements of the second source vector from the corresponding elements of the first source vector, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FSUB <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPSub{esize}(element1, element2, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

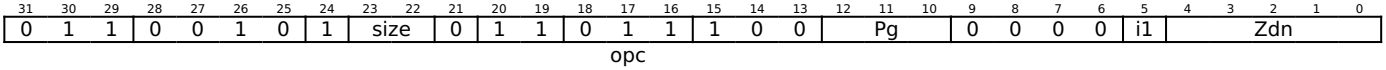
FSUBR (immediate)

Floating-point reversed subtract from immediate (predicated)

This instruction reverse subtracts from an immediate each Active floating-point element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate may take the value +0.5 or +1.0 only. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FSUBR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let imm : bits(esize) = if i1 == '0' then FPPointFive{esize}('0') else FPOne{esize}('0');
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the floating-point immediate value, encoded in “i1”:

i1	<const>
0	#0.5
1	#1.0

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = FPSub{esize}(imm, element1, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

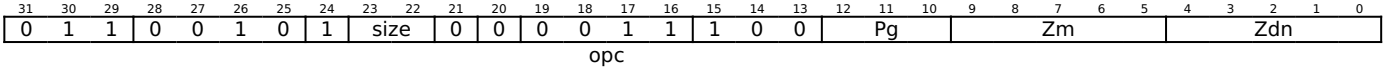
FSUBR (vectors)

Floating-point reversed subtract (predicated)

This instruction reverse subtracts Active floating-point elements of the first source vector from the corresponding floating-point elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
FSUBR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);

let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPSub{esize}(element2, element1, FPCR());
    else
        result[e*:esize] = element1;
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FTMAD

Floating-point trigonometric multiply-add coefficient

This instruction multiplies each element of the first source vector by the absolute value of the corresponding element of the second source vector and performs a fused addition of each product with a value obtained from a table of hard-wired coefficients, and destructively places the results in the first source vector. This instruction is unpredicated.

The coefficients are selected by a combination of the sign bit in the second source element and an immediate index in the range 0 to 7.

`FTMAD` can be combined with `FTSMUL` and `FTSSEL` to calculate values for sin(x) and cos(x). For more information, see `FTSMUL`. The coefficients are intended to provide accurate results for `FTSMUL` inputs in the range $-\pi/4 < x \leq \pi/4$.

For double-precision operations, the coefficients are:

element2<63>	Index	Hexadecimal	Approximate value
0	0	0x3FF0000000000000	1
0	1	0xBFC5555555555543	-1/3!
0	2	0x3F8111111110F30C	1/5!
0	3	0xBF2A01A019B92FC6	-1/7!
0	4	0x3EC71DE351F3D22B	1/9!
0	5	0xBE5AE5E2B60F7B91	-1/11!
0	6	0x3DE5D8408868552F	1/13!
0	7	0x0000000000000000	0
1	0	0x3FF0000000000000	1
1	1	0xBFE0000000000000	-1/2!
1	2	0x3FA5555555555536	1/4!
1	3	0xBF56C16C16C13A0B	-1/6!
1	4	0x3EFA01A019B1E8D8	1/8!
1	5	0xBE927E4F7282F468	-1/10!
1	6	0x3E21EE96D2641B13	1/12!
1	7	0xBDA8F76380FBB401	-1/14!

For single-precision operations, the coefficients are:

element2<63>	Index	Hexadecimal	Approximate value
0	0	0x3F800000	1
0	1	0xBE2AAAAB	-1/3!
0	2	0x3C088886	1/5!
0	3	0xB95008B9	-1/7!
0	4	0x36369D6D	1/9!
0	5	0x00000000	0
0	6	0x00000000	0
0	7	0x00000000	0
1	0	0x3F800000	1
1	1	0xBF000000	-1/2!
1	2	0x3D2AAAA6	1/4!
1	3	0xBAB60705	-1/6!
1	4	0x37CD37CC	1/8!
1	5	0x00000000	0
1	6	0x00000000	0
1	7	0x00000000	0

For half-precision operations, the coefficients are:

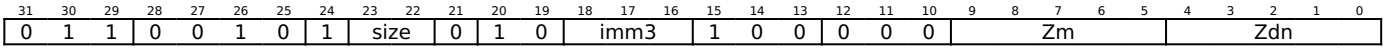
element2<63>	Index	Hexadecimal	Approximate value
0	0	0x3C00	1
0	1	0xB155	-1/3!
0	2	0x2030	1/5!
0	3	0x0000	0
0	4	0x0000	0
0	5	0x0000	0
0	6	0x0000	0
0	7	0x0000	0
1	0	0x3C00	1
1	1	0xB800	-1/2!

element2<63>	Index	Hexadecimal	Approximate value
1	2	0x293A	1/4!
1	3	0x0000	0
1	4	0x0000	0
1	5	0x0000	0
1	6	0x0000	0
1	7	0x0000	0

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
FTMAD <Zdn>.<T>, <Zdn>.<T>, <Zm>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let imm : integer = UInt(imm3);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<imm> Is the unsigned immediate operand, in the range 0 to 7, encoded in the "imm3" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPTrigMAdd{esize}(imm, element1, element2, FPCR());
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FTSMUL

Floating-point trigonometric starting value

This instruction multiplies each element of the first source vector by itself, replaces the sign-bit of each product with the least-significant bit of the corresponding element of the second source vector, and places the results in the destination vector. This instruction is unpredicated.

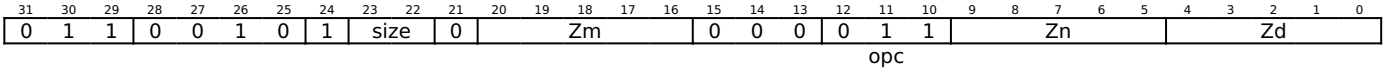
FTSMUL can be combined with FTMAD and FTSSEL to calculate values for sin(x) and cos(x). The use of the second operand is consistent with it holding an integer corresponding to the desired sine-wave quadrant when used in conjunction with FTMAD.

Note: FTSMUL, FTMAD, and FTSSEL can be used to calculate values for sin(k) and correspondingly cos(k-π/2) using an intermediate value x, in the range $-\pi/4 < x \leq \pi/4$, and a quadrant q where $k = q\pi/2 + x$, using a Taylor Series approximation. FTSMUL can be used to calculate x^2, and to insert q<0> into the most-significant bit, indicating the desired odd versus even Taylor Series to be used in FTMAD. Repeated uses of FTMAD can be performed on this value with decreasing immediate index operands, to produce a single accumulated value approximating the Taylor Series result with a single outstanding factor. FTSSEL can be used to apply the final factor of x, 1.0, -x, or -1.0, dependent on the corresponding sine-wave quadrant q<1:0>, to produce a final sin() or cos() value.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
FTSMUL <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPTrigSMul{esize}(element1, element2, FPCR());
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FTSSEL

Floating-point trigonometric select coefficient

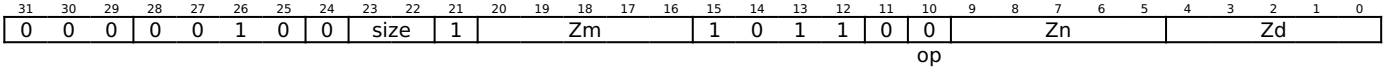
This instruction selects either the element of the first source vector or the value 1.0, based on bit<0> of the corresponding element of the second source element, negates the result if bit<1> of the corresponding element of the second source element is set, and places the results in the destination vector. This instruction is unpredicated.

FTSSEL can be combined with FTSMUL and FTMAD to calculate values for sin(k) and cos(k). For more information, see FTSMUL. The use of the second operand is consistent with it holding an integer corresponding to the desired sine-wave quadrant.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
FTSSEL <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = FPTrigSSel{esize}(element1, element2);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

HISTCNT

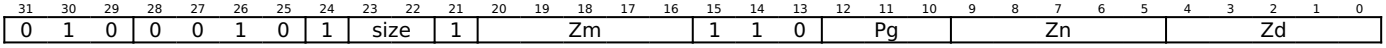
Count matching elements in vector

This instruction compares each Active 32 or 64-bit element of the first source vector with all Active elements with an element number less than or equal to its own in the second source vector, and places the count of matching elements in the corresponding element of the destination vector. Inactive elements in the destination vector are set to zero.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2)



Encoding

```
HISTCNT <Zd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let d : integer = UInt(Zd);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    var count : integer = 0;
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        for i = 0 to e do
            if ActivePredicateElement{PL}(mask, i, esize) then
                let element2 : bits(esize) = operand2[i*:esize];
                if element1 == element2 then
                    count = count + 1;
                end;
            end;
        end;
    end;
    result[e*:esize] = count[esize-1:0];
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

HISTSEG

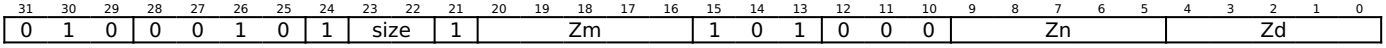
Count matching elements in vector segments

This instruction compares each 8-bit byte element of the first source vector with all of the elements in the corresponding 128-bit segment of the second source vector and places the count of matching elements in the corresponding element of the destination vector. This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2)



Encoding

```
HISTSEG <Zd>.B, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
if size != '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8;
let d : integer = UInt(Zd);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for b = 0 to segments-1 do
    for s = 0 to eltspersegment-1 do
        var count : integer = 0;
        let e : integer = eltspersegment * b + s;
        let element1 : bits(esize) = operand1[e*:esize];
        for i = 0 to eltspersegment-1 do
            let e2 : integer = eltspersegment * b + i;
            let element2 : bits(esize) = operand2[e2*:esize];
            if element1 == element2 then
                count = count + 1;
            end;
        end;
        result[e*:esize] = count[esize-1:0];
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

INCB, INCD, INCH, INCW (scalar)

Increment scalar by multiple of predicate constraint element count

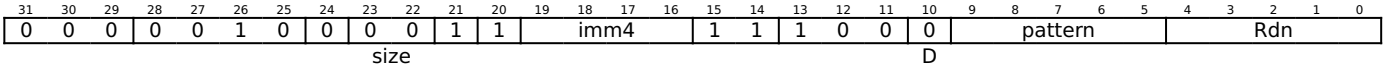
This instruction determines the number of Active elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment the scalar destination.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 4 classes: Byte , Doubleword , Halfword and Word

Byte

(FEAT_SVE || FEAT_SME)



Encoding

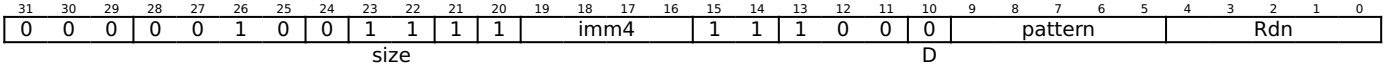
```
INCB <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Doubleword

(FEAT_SVE || FEAT_SME)



Encoding

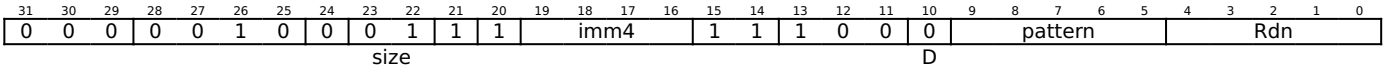
```
INCD <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Halfword

(FEAT_SVE || FEAT_SME)



Encoding

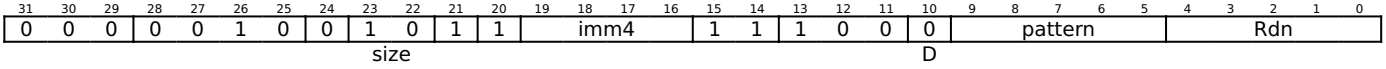
```
INCH <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Word

(FEAT_SVE || FEAT_SME)



Encoding

```
INCW <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Assembler Symbols

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let VL : integer{} = CurrentVL();
let operand1 : bits(64) = X{}(dn);
X{64}(dn) = operand1 + (count * imm);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

INCD, INCH, INCW (vector)

Increment vector by multiple of predicate constraint element count

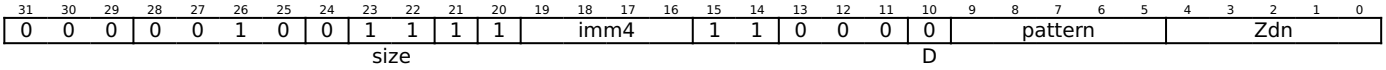
This instruction determines the number of Active elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment all destination vector elements.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 3 classes: Doubleword , Halfword and Word

Doubleword

(FEAT_SVE || FEAT_SME)



Encoding

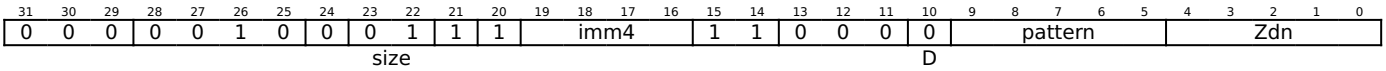
```
INCD <Zdn>.D{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Halfword

(FEAT_SVE || FEAT_SME)



Encoding

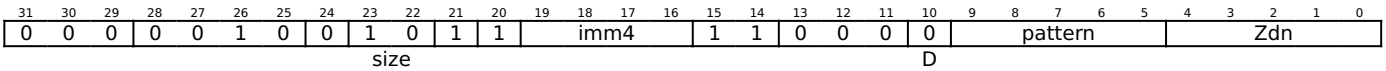
```
INCH <Zdn>.H{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Word

(FEAT_SVE || FEAT_SME)



Encoding

```
INCW <Zdn>.S{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    result[e*:esize] = operand1[e*:esize] + (count * imm);

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

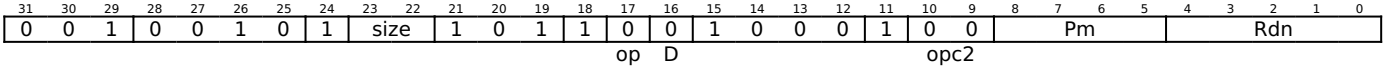
INCP (scalar)

Increment scalar by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to increment the scalar destination.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
INCP <Xdn>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Rdn);
```

Assembler Symbols

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(64) = X{}(dn);
let operand2 : bits(PL) = P{}(m);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

X{64}(dn) = operand1 + count;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

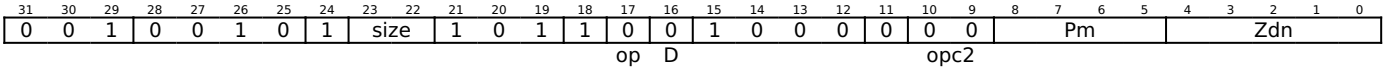
INCP (vector)

Increment vector by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to increment all destination vector elements. The predicate size specifier may be omitted in assembler source code, but this is deprecated and will be prohibited in a future release of the architecture.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
INCP <Zdn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(PL) = P{}(m);
var result : bits(VL);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

for e = 0 to elements-1 do
    result[e*:esize] = operand1[e*:esize] + count;

end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

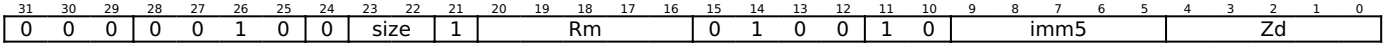
INDEX (immediate, scalar)

Create index starting from immediate and incremented by general-purpose register

This instruction populates the destination vector by setting the first element to the first signed immediate integer operand and monotonically incrementing the value by the second signed scalar integer operand for each subsequent element. The scalar source operand is a general-purpose register in which only the least significant bits corresponding to the vector element size are used and any remaining bits are ignored. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
INDEX <Zd>.<T>, #<imm>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Rm);
let d : integer = UInt(Zd);
let imm : integer = SInt(imm5);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is the signed immediate operand, in the range -16 to 15, encoded in the "imm5" field.

<R> Is a width specifier, encoded in “size”:

size	<R>
00	W
01	W
10	W
11	X

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand2 : bits(esize) = X{}(m);
let element2 : integer = SInt(operand2);
var result : bits(VL);

for e = 0 to elements-1 do
    let index : integer = imm + e * element2;
    result[e*:esize] = index[esize-1:0];
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

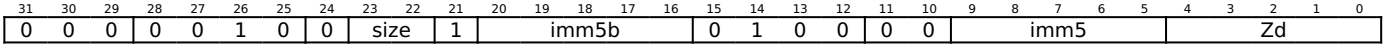
INDEX (immediates)

Create index starting from and incremented by immediate

This instruction populates the destination vector by setting the first element to the first signed immediate integer operand and monotonically incrementing the value by the second signed immediate integer operand for each subsequent element. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
INDEX <Zd>.<T>, #<imm1>, #<imm2>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt(Zd);
let imm1 : integer = SInt(imm5);
let imm2 : integer = SInt(imm5b);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm1> Is the first signed immediate operand, in the range -16 to 15, encoded in the "imm5" field.

<imm2> Is the second signed immediate operand, in the range -16 to 15, encoded in the "imm5b" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var result : bits(VL);

for e = 0 to elements-1 do
    let index : integer = imm1 + e * imm2;
    result[e*:esize] = index[esize-1:0];
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

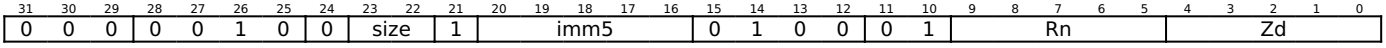
INDEX (scalar, immediate)

Create index starting from general-purpose register and incremented by immediate

This instruction populates the destination vector by setting the first element to the first signed scalar integer operand and monotonically incrementing the value by the second signed immediate integer operand for each subsequent element. The scalar source operand is a general-purpose register in which only the least significant bits corresponding to the vector element size are used and any remaining bits are ignored. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
INDEX <Zd>.<T>, <R><n>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Rn);
let d : integer = UInt(Zd);
let imm : integer = SInt(imm5);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “size”:

size	<R>
00	W
01	W
10	W
11	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<imm> Is the signed immediate operand, in the range -16 to 15, encoded in the "imm5" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(esize) = X{}(n);
let element1 : integer = SInt(operand1);
var result : bits(VL);

for e = 0 to elements-1 do
    let index : integer = element1 + e * imm;
    result[e*:esize] = index[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

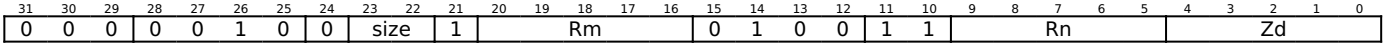
INDEX (scalars)

Create index starting from and incremented by general-purpose register

This instruction populates the destination vector by setting the first element to the first signed scalar integer operand and monotonically incrementing the value by the second signed scalar integer operand for each subsequent element. The scalar source operands are general-purpose registers in which only the least significant bits corresponding to the vector element size are used and any remaining bits are ignored. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
INDEX <Zd>.<T>, <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “size”:

size	<R>
00	W
01	W
10	W
11	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(esize) = X{}(n);
let element1 : integer = SInt(operand1);
let operand2 : bits(esize) = X{}(m);
let element2 : integer = SInt(operand2);
var result : bits(VL);

for e = 0 to elements-1 do
    let index : integer = element1 + e * element2;
    result[e*:esize] = index[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

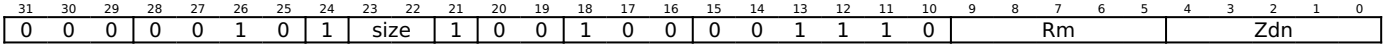
INSR (scalar)

Insert general-purpose register in shifted vector

This instruction shifts the destination vector left by one element, and then places a copy of the least-significant bits of the general-purpose register in element 0 of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
INSR <Zdn>.<T>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Rm);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “size”:

size	<R>
00	W
01	W
10	W
11	X

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let dest : bits(VL) = Z{}(dn);
let src : bits(esize) = X{}(m);
Z{VL}(dn) = dest[(VL-esize)-1:0] :: src;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

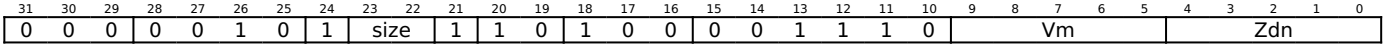
INSR (SIMD&FP scalar)

Insert SIMD&FP scalar register in shifted vector

This instruction shifts the destination vector left by one element, and then places a copy of the SIMD&FP scalar register in element 0 of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
INSR <Zdn>.<T>, <V><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Vm);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<m> Is the number [0-31] of the source SIMD&FP register, encoded in the "Vm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let dest : bits(VL) = Z{}(dn);
let src : bits(esize) = V{}(m);
Z{VL}(dn) = dest[(VL-esize)-1:0] :: src;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

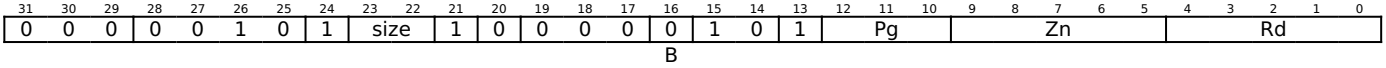
LASTA (scalar)

Extract element after last to general-purpose register

This instruction extracts the element after the Last active element to a general-purpose register. If there is an Active element, the element after the Last active element modulo the number of elements from the final source vector register is extracted. If there are no Active elements, element zero is extracted. Then the extracted element is zero-extended and placed in the destination general-purpose register.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LASTA <R><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = if esize < 64 then 32 else 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Rd);
let isBefore : boolean = FALSE;
```

Assembler Symbols

<R> Is a width specifier, encoded in “size”:

size	<R>
00	W
01	W
10	W
11	X

<d> Is the number [0-30] of the destination general-purpose register or the name ZR (31), encoded in the "Rd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = Z{}(n);
var result : bits(rsize);
var last : integer = LastActiveElement{PL}(mask, esize);

if isBefore then
    if last < 0 then last = elements - 1; end;
else
    last = last + 1;
    if last >= elements then last = 0; end;
end;
result = ZeroExtend{rsize}(operand[last*:esize]);

X{rsize}(d) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

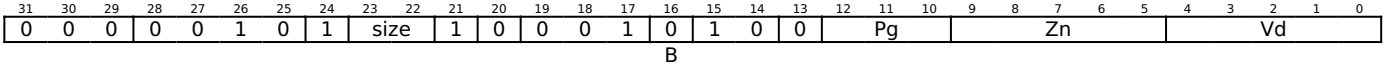
LASTA (SIMD&FP scalar)

Extract element after last to SIMD&FP scalar register

This instruction extracts the element after the Last active element to a SIMD&FP scalar register. If there is an Active element, the element after the Last active element modulo the number of elements from the final source vector register is extracted. If there are no Active elements, element zero is extracted. Then the extracted element is placed in the destination SIMD&FP scalar register.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LASTA <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
let isBefore : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = Z{}(n);
var last : integer = LastActiveElement{PL}(mask, esize);

if isBefore then
    if last < 0 then last = elements - 1; end;
else
    last = last + 1;
    if last >= elements then last = 0; end;
end;
V{esize}(d) = operand[last*:esize];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

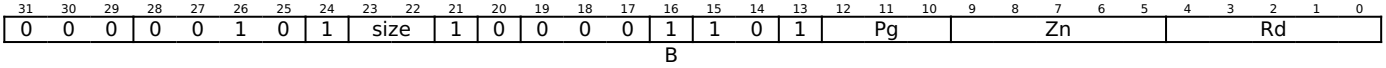
LASTB (scalar)

Extract last element to general-purpose register

This instruction extracts the Last active element to a general-purpose register. If there is an Active element, the Last active element from the final source vector register is extracted. If there are no Active elements, the highest-numbered element is extracted. Then the extracted element is zero-extended and placed in the destination general-purpose register.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LASTB <R><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = if esize < 64 then 32 else 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Rd);
let isBefore : boolean = TRUE;
```

Assembler Symbols

<R> Is a width specifier, encoded in “size”:

size	<R>
00	W
01	W
10	W
11	X

<d> Is the number [0-30] of the destination general-purpose register or the name ZR (31), encoded in the "Rd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = Z{}(n);
var result : bits(rsize);
var last : integer = LastActiveElement{PL}(mask, esize);

if isBefore then
    if last < 0 then last = elements - 1; end;
else
    last = last + 1;
    if last >= elements then last = 0; end;
end;
result = ZeroExtend{rsize}(operand[last*:esize]);

X{rsize}(d) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

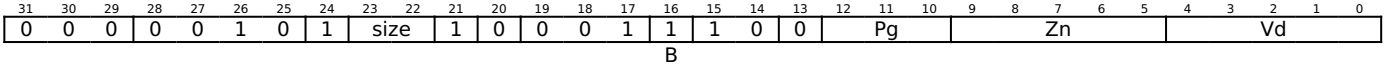
LASTB (SIMD&FP scalar)

Extract last element to SIMD&FP scalar register

This instruction extracts the Last active element to a SIMD&FP scalar register. If there is an Active element, the Last active element from the final source vector register is extracted. If there are no Active elements, the highest-numbered element is extracted. Then the extracted element is placed in the destination SIMD&FP register.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LASTB <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
let isBefore : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = Z{}(n);
var last : integer = LastActiveElement{PL}(mask, esize);

if isBefore then
    if last < 0 then last = elements - 1; end;
else
    last = last + 1;
    if last >= elements then last = 0; end;
end;
V{esize}(d) = operand[last*:esize];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

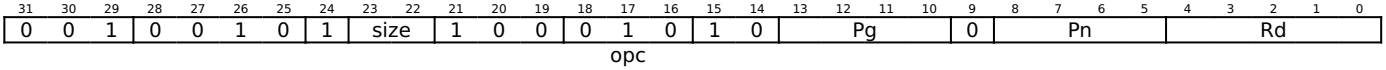
LASTP

Scalar index of last true predicate element (predicated)

This instruction determines the index of the Last active and true element of the source predicate, and places the scalar result in the destination general-purpose register. If there are no Active and true elements, the destination general-purpose register is updated to the value minus one.

SVE2

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
LASTP <Xd>, <Pg>, <Pn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let d : integer = UInt(Rd);
```

Assembler Symbols

<Xd> Is the 64-bit name of the destination general-purpose register, encoded in the "Rd" field.

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<Pn> Is the name of the source scalable predicate register, encoded in the "Pn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(PL) = P{}(n);
var index : integer = -1;

for e = elements-1 downto 0 do
    if (index == -1 && ActivePredicateElement{PL}(mask, e, esize) &&
          ActivePredicateElement{PL}(operand, e, esize)) then
        index = e;
    end;
end;

X{64}(d) = index[63:0];
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1B (scalar plus immediate, consecutive registers)

Contiguous load of bytes to multiple consecutive vectors (immediate index)

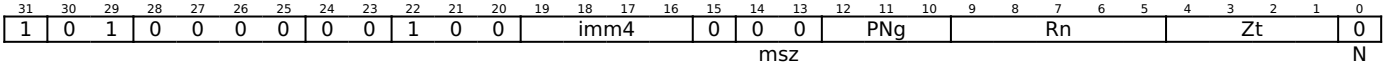
This instruction performs a contiguous load of unsigned bytes to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

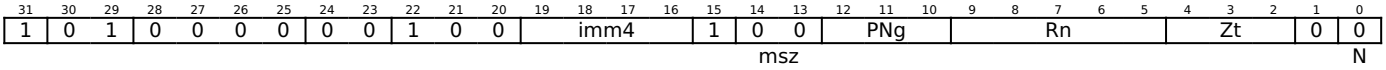
```
LD1B { <Zt1>.B-<Zt2>.B }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LD1B { <Zt1>.B-<Zt4>.B }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1B (scalar plus immediate, single register)

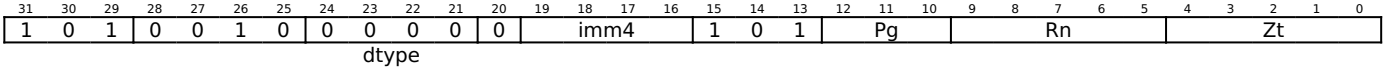
Contiguous load unsigned bytes to vector (immediate index)

This instruction performs a contiguous load of unsigned bytes to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 4 classes: 8-bit element , 16-bit element , 32-bit element and 64-bit element

8-bit element

(FEAT_SVE || FEAT_SME)



Encoding

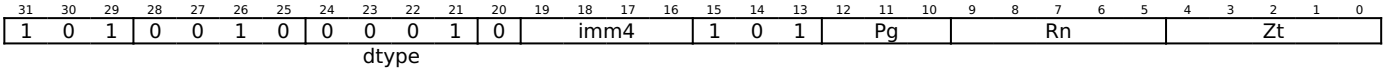
```
LD1B { <Zt>.B }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

16-bit element

(FEAT_SVE || FEAT_SME)



Encoding

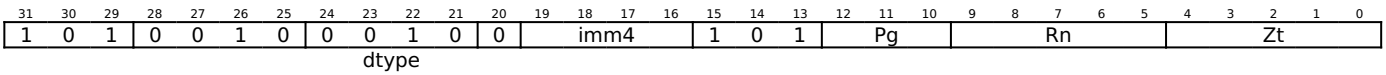
```
LD1B { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

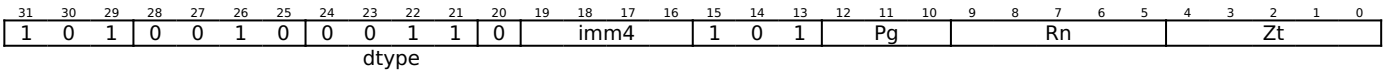
```
LD1B { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1B { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1B (scalar plus scalar, consecutive registers)

Contiguous load of bytes to multiple consecutive vectors (scalar index)

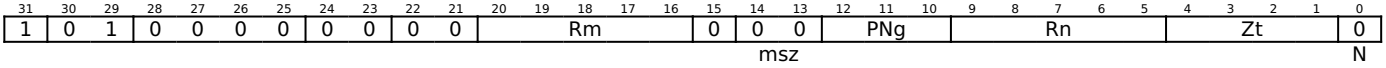
This instruction performs a contiguous load of unsigned bytes to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

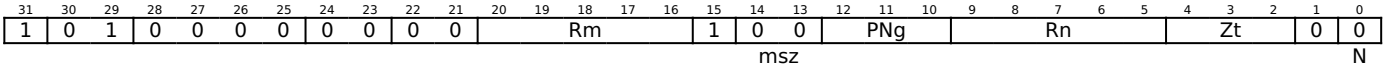
```
LD1B { <Zt1>.B-<Zt2>.B }, <PNg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 8;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LD1B { <Zt1>.B-<Zt4>.B }, <PNg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 8;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

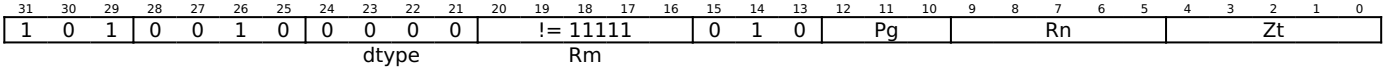
LD1B (scalar plus scalar, single register)

Contiguous load unsigned bytes to vector (scalar index)

This instruction performs a contiguous load of unsigned bytes to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.
It has encodings from 4 classes: 8-bit element , 16-bit element , 32-bit element and 64-bit element

8-bit element

(FEAT_SVE || FEAT_SME)



Encoding

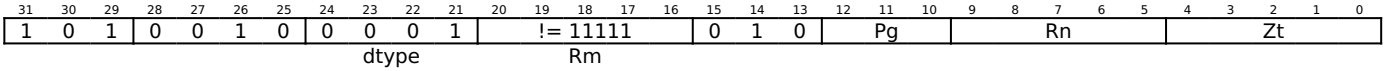
```
LD1B { <Zt>.B }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
```

16-bit element

(FEAT_SVE || FEAT_SME)



Encoding

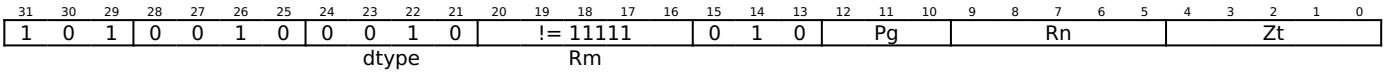
```
LD1B { <Zt>.H }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
```

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

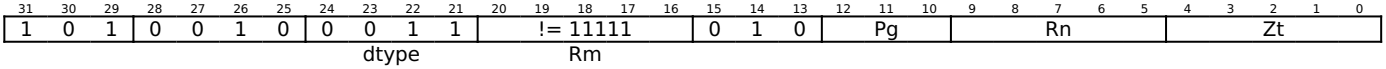
```
LD1B { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1B { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1B (scalar plus vector)

Gather load unsigned bytes to vector (vector index)

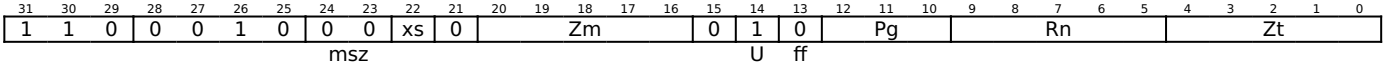
This instruction performs a gather load of unsigned bytes to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally sign-extended or zero-extended from 32 to 64 bits. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 32-bit unpacked unscaled offset , 32-bit unscaled offset and 64-bit unscaled offset

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

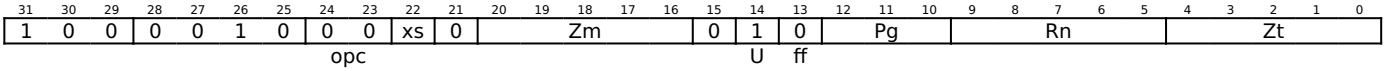
```
LD1B { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

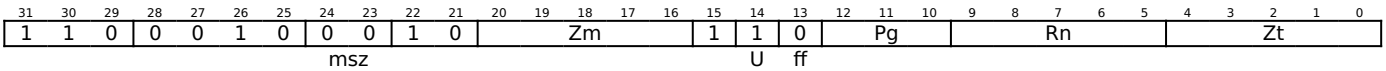
```
LD1B { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LD1B { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1B (vector plus immediate)

Gather load unsigned bytes to vector (immediate index)

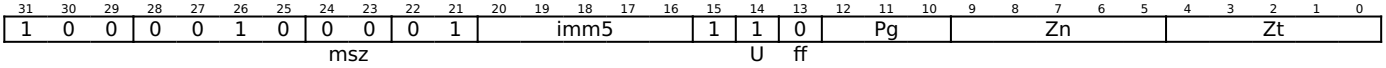
This instruction performs a gather load of unsigned bytes to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is in the range 0 to 31. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

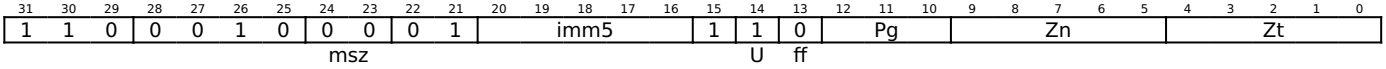
```
LD1B { <Zt>.S }, <Pg>/Z, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
LD1B { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, in the range 0 to 31, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1D (scalar plus immediate, consecutive registers)

Contiguous load of doublewords to multiple consecutive vectors (immediate index)

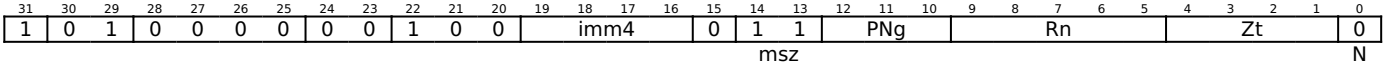
This instruction performs a contiguous load of unsigned doublewords to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

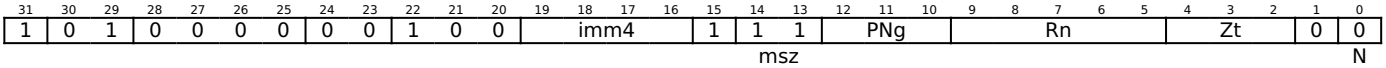
```
LD1D { <Zt1>.D-<Zt2>.D }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LD1D { <Zt1>.D-<Zt4>.D }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1D (scalar plus immediate, single register)

Contiguous load unsigned doublewords to vector (immediate index)

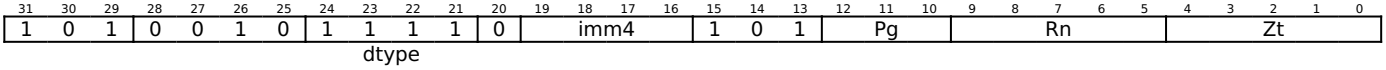
This instruction performs a contiguous load of unsigned doublewords to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

The 128-bit element variant is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 64-bit element and 128-bit element

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

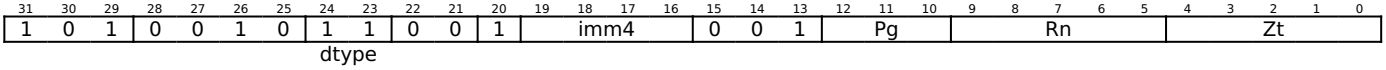
```
LD1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

128-bit element

(FEAT_SVE2p1)



Encoding

```
LD1D { <Zt>.Q }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let msize : integer{} = 64;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
if esize < 128 then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1D (scalar plus scalar, consecutive registers)

Contiguous load of doublewords to multiple consecutive vectors (scalar index)

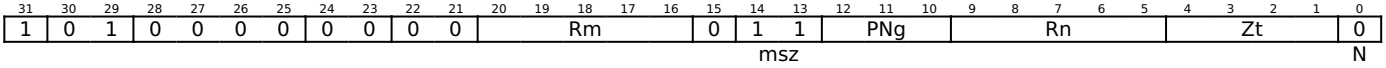
This instruction performs a contiguous load of unsigned doublewords to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

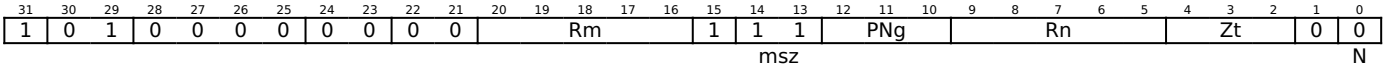
```
LD1D { <Zt1>.D-<Zt2>.D }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 64;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LD1D { <Zt1>.D-<Zt4>.D }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 64;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1D (scalar plus scalar, single register)

Contiguous load unsigned doublewords to vector (scalar index)

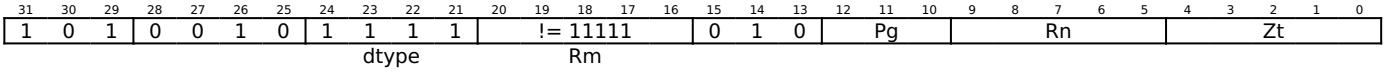
This instruction performs a contiguous load of unsigned doublewords to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 8 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

The 128-bit element variant is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 64-bit element and 128-bit element

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

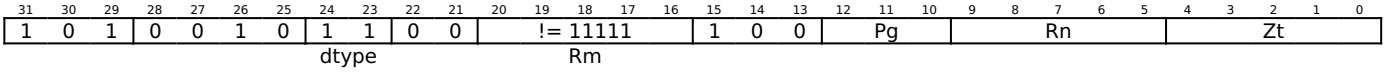
```
LD1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let unsigned : boolean = TRUE;
```

128-bit element

(FEAT_SVE2p1)



Encoding

```
LD1D { <Zt>.Q }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let msize : integer{} = 64;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
if esize < 128 then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1D (scalar plus vector)

Gather load doublewords to vector (vector index)

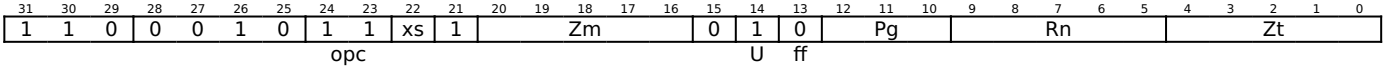
This instruction performs a gather load of doublewords to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 8. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 4 classes: 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

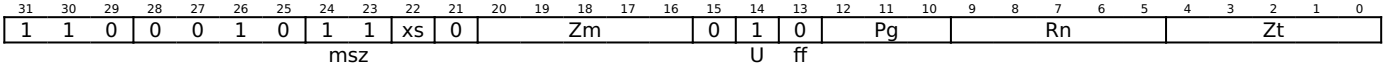
```
LD1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod> #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 3;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

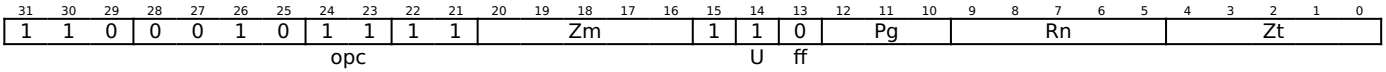
```
LD1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

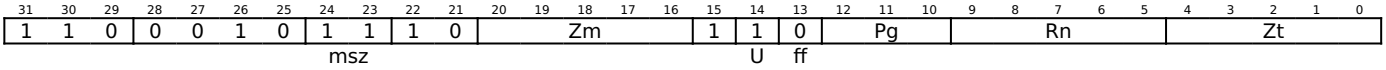
```
LD1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 3;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LD1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1D (vector plus immediate)

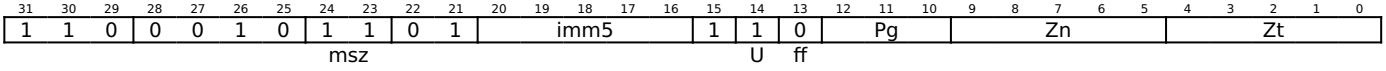
Gather load doublewords to vector (immediate index)

This instruction performs a gather load of doublewords to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is a multiple of 8 in the range 0 to 248. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
LD1D { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 8 in the range 0 to 248, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1H (scalar plus immediate, consecutive registers)

Contiguous load of halfwords to multiple consecutive vectors (immediate index)

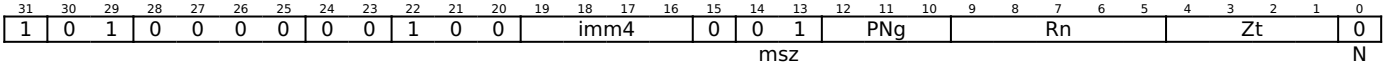
This instruction performs a contiguous load of unsigned halfwords to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

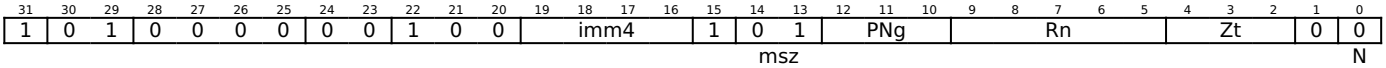
```
LD1H { <Zt1>.H-<Zt2>.H }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LD1H { <Zt1>.H-<Zt4>.H }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1H (scalar plus immediate, single register)

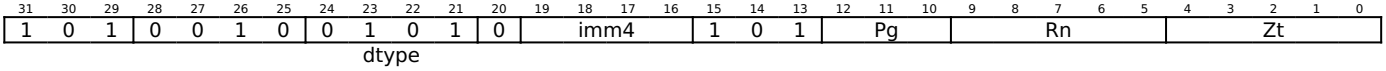
Contiguous load unsigned halfwords to vector (immediate index)

This instruction performs a contiguous load of unsigned halfwords to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE || FEAT_SME)



Encoding

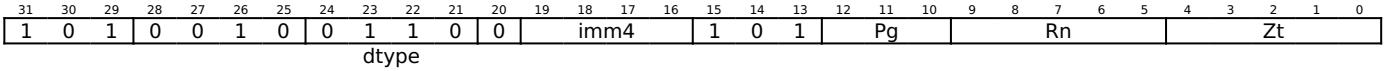
```
LD1H { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

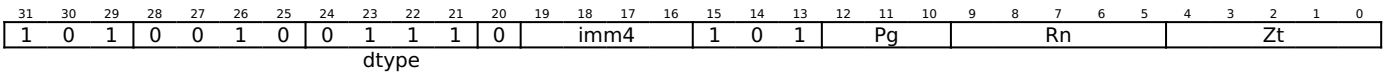
```
LD1H { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1H (scalar plus scalar, consecutive registers)

Contiguous load of halfwords to multiple consecutive vectors (scalar index)

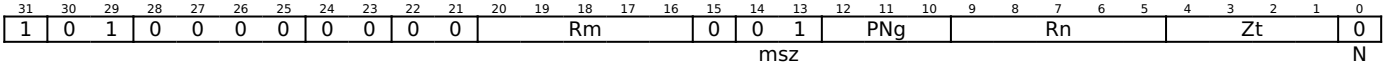
This instruction performs a contiguous load of unsigned halfwords to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

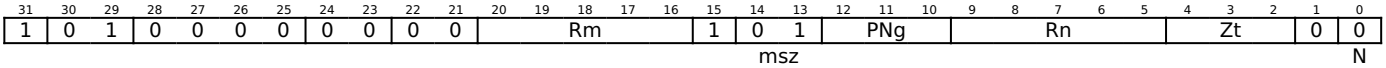
```
LD1H { <Zt1>.H-<Zt2>.H }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 16;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LD1H { <Zt1>.H-<Zt4>.H }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 16;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1H (scalar plus scalar, single register)

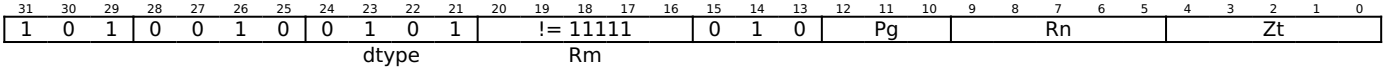
Contiguous load unsigned halfwords to vector (scalar index)

This instruction performs a contiguous load of unsigned halfwords to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 2 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE || FEAT_SME)



Encoding

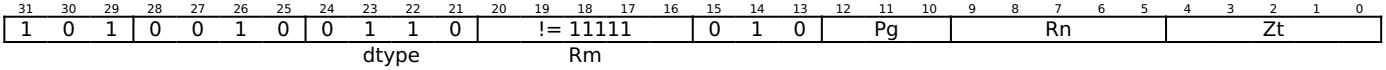
```
LD1H { <Zt>.H }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
```

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

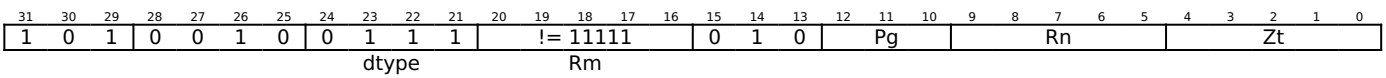
```
LD1H { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1H (scalar plus vector)

Gather load unsigned halfwords to vector (vector index)

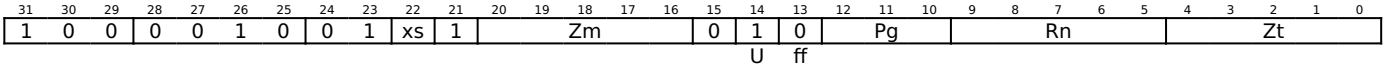
This instruction performs a gather load of unsigned halfwords to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 2. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 6 classes: 32-bit scaled offset , 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 32-bit unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

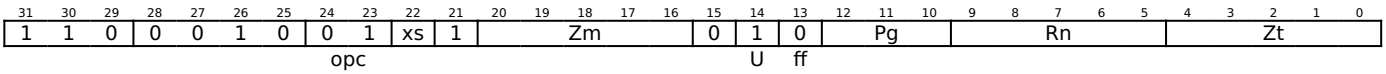
```
LD1H { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 1;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

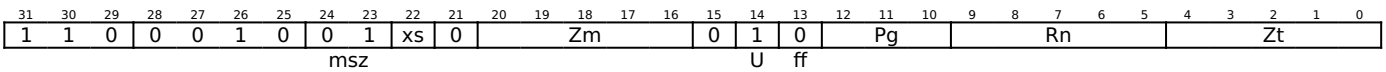
```
LD1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 1;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

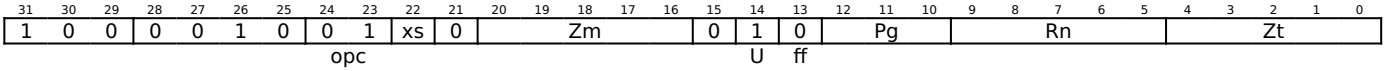
```
LD1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

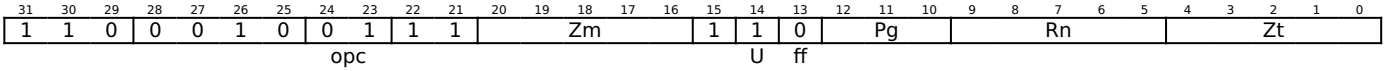
```
LD1H { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

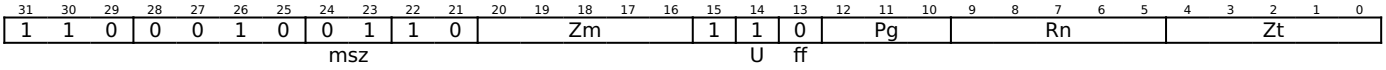
```
LD1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 1;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LD1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1H (vector plus immediate)

Gather load unsigned halfwords to vector (immediate index)

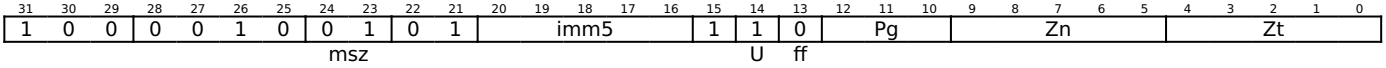
This instruction performs a gather load of unsigned halfwords to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is a multiple of 2 in the range 0 to 62. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

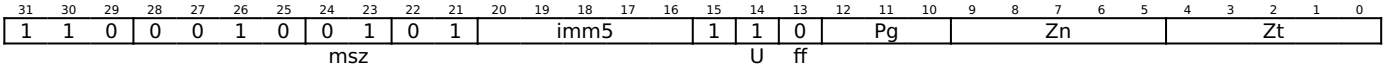
```
LD1H { <Zt>.S }, <Pg>/Z, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
LD1H { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 2 in the range 0 to 62, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

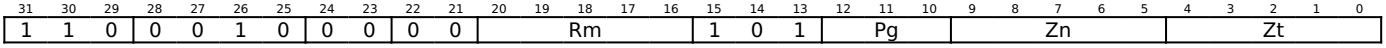
LD1Q

Gather load quadwords

This instruction performs a gather load of quadwords to Active elements of a vector register from memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.
This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2p1)



Encoding

```
LD1Q { <Zt>.Q }, <Pg>/Z, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
CheckNonStreamingSVEEnabled();
let elements : integer = VL DIV 128;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var result : bits(VL);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, 128) then
    base = Z{VL}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, 128) then
        let baddr : bits(64) = base[(2*e)*:64];
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        result[e*:128] = Mem{128}(addr, accdesc);
    else
        result[e*:128] = Zeros{128};
    end;
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RB

Load and broadcast unsigned byte to vector

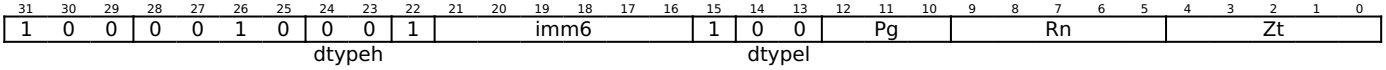
This instruction loads a single unsigned byte from a memory address generated by a 64-bit scalar base address plus an immediate offset that is in the range 0 to 63.

The loaded data is broadcast into all Active elements of the destination vector, setting the Inactive elements to zero. If all elements are inactive, then the instruction will not perform a read from Device memory or cause a data abort.

It has encodings from 4 classes: 8-bit element , 16-bit element , 32-bit element and 64-bit element

8-bit element

(FEAT_SVE || FEAT_SME)



Encoding

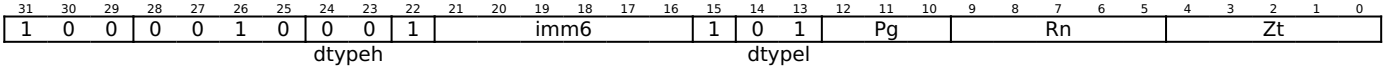
```
LD1RB { <Zt>.B }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm6);
```

16-bit element

(FEAT_SVE || FEAT_SME)



Encoding

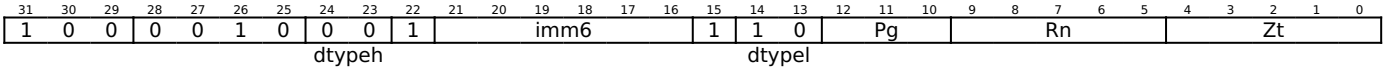
```
LD1RB { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm6);
```

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

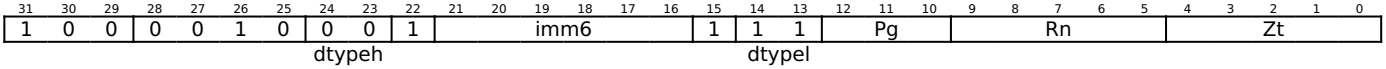
```
LD1RB { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm6);
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RB { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm6);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional unsigned immediate byte offset, in the range 0 to 63, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    let addr : bits(64) = AddressAdd(base, offset * mbytes, accdesc);
    data = Mem{msize}(addr, accdesc);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RD

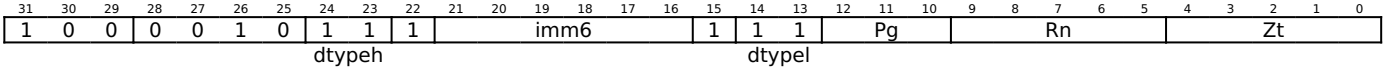
Load and broadcast doubleword to vector

This instruction loads a single doubleword from a memory address generated by a 64-bit scalar base address plus an immediate offset that is a multiple of 8 in the range 0 to 504.

The loaded data is broadcast into all Active elements of the destination vector, setting the Inactive elements to zero. If all elements are inactive, then the instruction will not perform a read from Device memory or cause a data abort.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RD { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm6);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 8 in the range 0 to 504, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    let addr : bits(64) = AddressAdd(base, offset * mbytes, accdesc);
    data = Mem{msize}(addr, accdesc);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RH

Load and broadcast unsigned halfword to vector

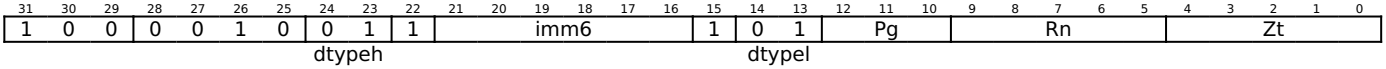
This instruction loads a single unsigned halfword from a memory address generated by a 64-bit scalar base address plus an immediate offset that is a multiple of 2 in the range 0 to 126.

The loaded data is broadcast into all Active elements of the destination vector, setting the Inactive elements to zero. If all elements are inactive, then the instruction will not perform a read from Device memory or cause a data abort.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE || FEAT_SME)



Encoding

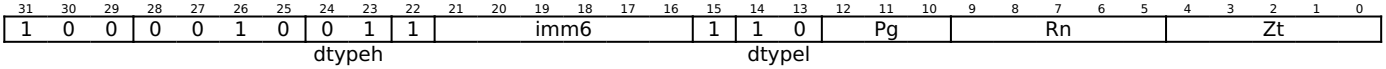
```
LD1RH { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm6);
```

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

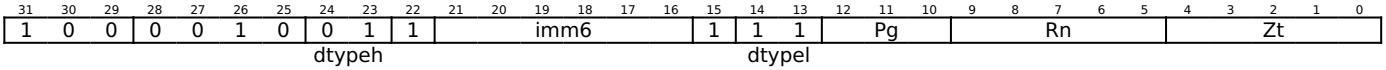
```
LD1RH { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm6);
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RH { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm6);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 2 in the range 0 to 126, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    let addr : bits(64) = AddressAdd(base, offset * mbytes, accdesc);
    data = Mem{msize}(addr, accdesc);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1ROB (scalar plus immediate)

Contiguous load and replicate thirty-two bytes (immediate index)

This instruction loads thirty-two contiguous bytes to elements of a 256-bit (octaword) vector from the memory address generated by a 64-bit scalar base address and immediate index that is a multiple of 32 in the range -256 to +224 added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero.

The resulting 256-bit vector is then replicated to fill the destination vector. The instruction requires that the Effective SVE vector length is at least 256 bits.

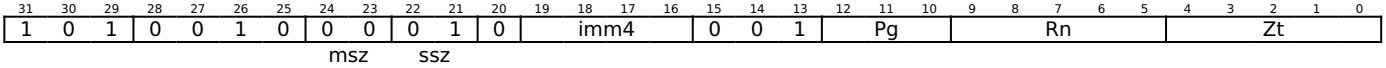
Only the first thirty-two predicate elements are used and higher numbered predicate elements are ignored.

ID_AA64ZFR0_EL1.F64MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_F64MM)



Encoding

```
LD1ROB { <Zt>.B }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 32 in the range -256 to 224, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = 256 DIV esize;
var base : bits(64);
let mask : bits(32) = P{PL}(g)[31:0]; // low 32 bits only
var addr : bits(64);
var result : bits(256);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{32}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{32}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = ZeroExtend{VL}(Replicate{VL}(result));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1ROB (scalar plus scalar)

Contiguous load and replicate thirty-two bytes (scalar index)

This instruction loads thirty-two contiguous bytes to elements of a 256-bit (octaword) vector from the memory address generated by a 64-bit scalar base address and scalar index that is added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero.

The resulting 256-bit vector is then replicated to fill the destination vector. The instruction requires that the Effective SVE vector length is at least 256 bits.

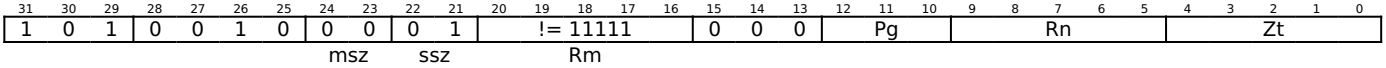
Only the first thirty-two predicate elements are used and higher numbered predicate elements are ignored.

ID_AA64ZFR0_EL1.F64MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_F64MM)



Encoding

```
LD1ROB { <Zt>.B }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = 256 DIV esize;
var base : bits(64);
let mask : bits(32) = P{PL}(g)[31:0]; // low 32 bits only
var offset : bits(64);
var addr : bits(64);
var result : bits(256);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{32}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{32}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = ZeroExtend{VL}(Replicate{VL}(result));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1ROD (scalar plus immediate)

Contiguous load and replicate four doublewords (immediate index)

This instruction loads four contiguous doublewords to elements of a 256-bit (octaword) vector from the memory address generated by a 64-bit scalar base address and immediate index that is a multiple of 32 in the range -256 to +224 added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero.

The resulting 256-bit vector is then replicated to fill the destination vector. The instruction requires that the Effective SVE vector length is at least 256 bits.

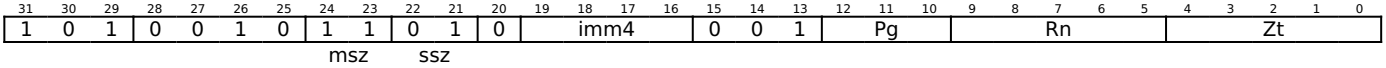
Only the first four predicate elements are used and higher numbered predicate elements are ignored.

ID_AA64ZFR0_EL1.F64MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_F64MM)



Encoding

```
LD1ROD { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 32 in the range -256 to 224, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = 256 DIV esize;
var base : bits(64);
let mask : bits(32) = P{PL}(g)[31:0]; // low 32 bits only
var addr : bits(64);
var result : bits(256);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{32}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{32}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = ZeroExtend{VL}(Replicate{VL}(result));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1ROD (scalar plus scalar)

Contiguous load and replicate four doublewords (scalar index)

This instruction loads four contiguous doublewords to elements of a 256-bit (octaword) vector from the memory address generated by a 64-bit scalar base address and scalar index that is multiplied by 8 and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero.

The resulting 256-bit vector is then replicated to fill the destination vector. The instruction requires that the Effective SVE vector length is at least 256 bits.

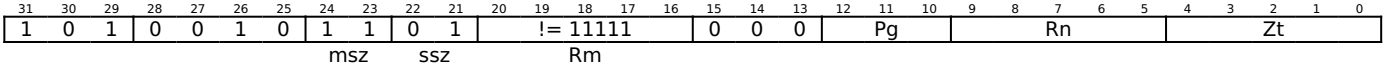
Only the first four predicate elements are used and higher numbered predicate elements are ignored.

ID_AA64ZFR0_EL1.F64MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_F64MM)



Encoding

```
LD1ROD { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = 256 DIV esize;
var base : bits(64);
let mask : bits(32) = P{PL}(g)[31:0]; // low 32 bits only
var offset : bits(64);
var addr : bits(64);
var result : bits(256);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{32}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{32}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = ZeroExtend{VL}(Replicate{VL}(result));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1ROH (scalar plus immediate)

Contiguous load and replicate sixteen halfwords (immediate index)

This instruction loads sixteen contiguous halfwords to elements of a 256-bit (octaword) vector from the memory address generated by a 64-bit scalar base address and immediate index that is a multiple of 32 in the range -256 to +224 added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero.

The resulting 256-bit vector is then replicated to fill the destination vector. The instruction requires that the Effective SVE vector length is at least 256 bits.

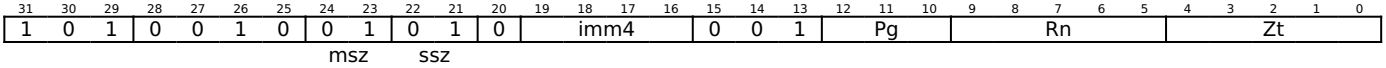
Only the first sixteen predicate elements are used and higher numbered predicate elements are ignored.

ID_AA64ZFR0_EL1.F64MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_F64MM)



Encoding

```
LD1ROH { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 32 in the range -256 to 224, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = 256 DIV esize;
var base : bits(64);
let mask : bits(32) = P{PL}(g)[31:0]; // low 32 bits only
var addr : bits(64);
var result : bits(256);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{32}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{32}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = ZeroExtend{VL}(Replicate{VL}(result));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1ROH (scalar plus scalar)

Contiguous load and replicate sixteen halfwords (scalar index)

This instruction loads sixteen contiguous halfwords to elements of a 256-bit (octaword) vector from the memory address generated by a 64-bit scalar base address and scalar index that is multiplied by 2 and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero.

The resulting 256-bit vector is then replicated to fill the destination vector. The instruction requires that the Effective SVE vector length is at least 256 bits.

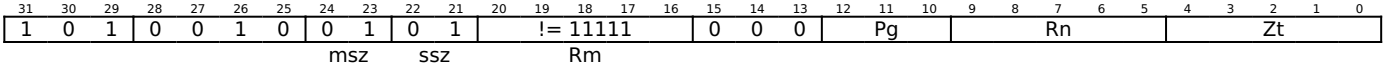
Only the first sixteen predicate elements are used and higher numbered predicate elements are ignored.

ID_AA64ZFR0_EL1.F64MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_F64MM)



Encoding

```
LD1ROH { <Zt>.H }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = 256 DIV esize;
var base : bits(64);
let mask : bits(32) = P{PL}(g)[31:0]; // low 32 bits only
var offset : bits(64);
var addr : bits(64);
var result : bits(256);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{32}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{32}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = ZeroExtend{VL}(Replicate{VL}(result));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1ROW (scalar plus immediate)

Contiguous load and replicate eight words (immediate index)

This instruction loads eight contiguous words to elements of a 256-bit (octaword) vector from the memory address generated by a 64-bit scalar base address and immediate index that is a multiple of 32 in the range -256 to +224 added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero.

The resulting 256-bit vector is then replicated to fill the destination vector. The instruction requires that the Effective SVE vector length is at least 256 bits.

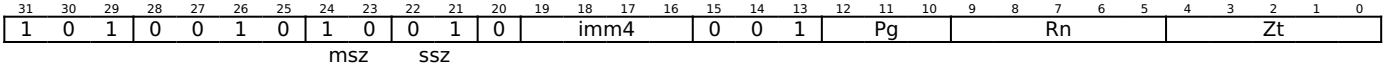
Only the first eight predicate elements are used and higher numbered predicate elements are ignored.

ID_AA64ZFR0_EL1.F64MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_F64MM)



Encoding

```
LD1ROW { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 32 in the range -256 to 224, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = 256 DIV esize;
var base : bits(64);
let mask : bits(32) = P{PL}(g)[31:0]; // low 32 bits only
var addr : bits(64);
var result : bits(256);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{32}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{32}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = ZeroExtend{VL}(Replicate{VL}(result));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1ROW (scalar plus scalar)

Contiguous load and replicate eight words (scalar index)

This instruction loads eight contiguous words to elements of a 256-bit (octaword) vector from the memory address generated by a 64-bit scalar base address and scalar index that is multiplied by 4 and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero.

The resulting 256-bit vector is then replicated to fill the destination vector. The instruction requires that the Effective SVE vector length is at least 256 bits.

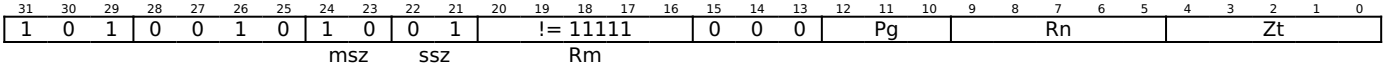
Only the first eight predicate elements are used and higher numbered predicate elements are ignored.

ID_AA64ZFR0_EL1.F64MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_F64MM)



Encoding

```
LD1ROW { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = 256 DIV esize;
var base : bits(64);
let mask : bits(32) = P{PL}(g)[31:0]; // low 32 bits only
var offset : bits(64);
var addr : bits(64);
var result : bits(256);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{32}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{32}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = ZeroExtend{VL}(Replicate{VL}(result));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RQB (scalar plus immediate)

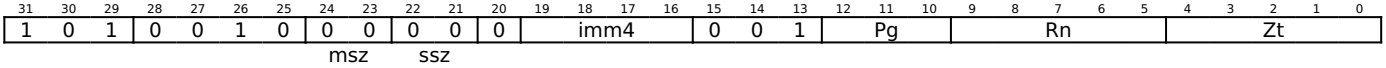
Contiguous load and replicate sixteen bytes (immediate index)

This instruction loads sixteen contiguous bytes to elements of a short, 128-bit (quadword) vector from the memory address generated by a 64-bit scalar base address and immediate index that is a multiple of 16 in the range -128 to +112 added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero. The resulting short vector is then replicated to fill the long destination vector. Only the first sixteen predicate elements are used and higher numbered predicate elements are ignored.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RQB { <Zt>.B }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 16 in the range -128 to 112, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = 128 DIV esize;
var base : bits(64);
let mask : bits(16) = P{PL}(g)[15:0]; // low 16 bits only
var addr : bits(64);
var result : bits(128);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{16}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * 16, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{16}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = Replicate{VL}(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RQB (scalar plus scalar)

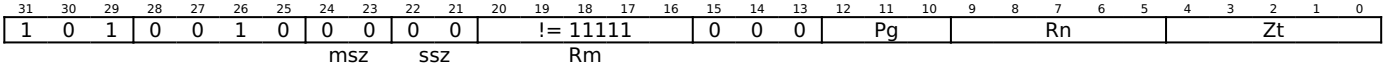
Contiguous load and replicate sixteen bytes (scalar index)

This instruction loads sixteen contiguous bytes to elements of a short, 128-bit (quadword) vector from the memory address generated by a 64-bit scalar base address and scalar index that is added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero. The resulting short vector is then replicated to fill the long destination vector. Only the first sixteen predicate elements are used and higher numbered predicate elements are ignored.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RQB { <Zt>.B }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = 128 DIV esize;
var base : bits(64);
let mask : bits(16) = P{PL}(g)[15:0]; // low 16 bits only
var offset : bits(64);
var addr : bits(64);
var result : bits(128);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{16}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{16}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = Replicate{VL}(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RQD (scalar plus immediate)

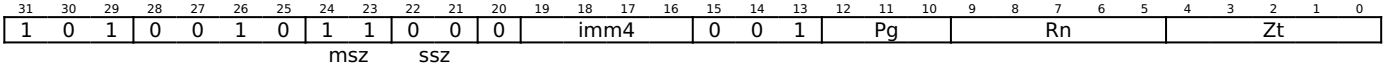
Contiguous load and replicate two doublewords (immediate index)

This instruction loads two contiguous doublewords to elements of a short, 128-bit (quadword) vector from the memory address generated by a 64-bit scalar base address and immediate index that is a multiple of 16 in the range -128 to +112 added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero. The resulting short vector is then replicated to fill the long destination vector. Only the first two predicate elements are used and higher numbered predicate elements are ignored.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RQD { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 16 in the range -128 to 112, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = 128 DIV esize;
var base : bits(64);
let mask : bits(16) = P{PL}(g)[15:0]; // low 16 bits only
var addr : bits(64);
var result : bits(128);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{16}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * 16, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{16}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = Replicate{VL}(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RQD (scalar plus scalar)

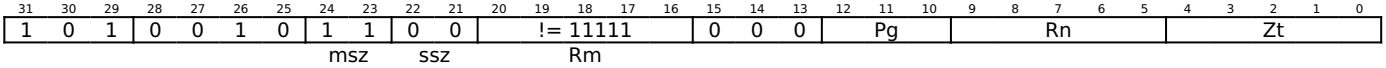
Contiguous load and replicate two doublewords (scalar index)

This instruction loads two contiguous doublewords to elements of a short, 128-bit (quadword) vector from the memory address generated by a 64-bit scalar base address and scalar index that is multiplied by 8 and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero. The resulting short vector is then replicated to fill the long destination vector. Only the first two predicate elements are used and higher numbered predicate elements are ignored.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RQD { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = 128 DIV esize;
var base : bits(64);
let mask : bits(16) = P{PL}(g)[15:0]; // low 16 bits only
var offset : bits(64);
var addr : bits(64);
var result : bits(128);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{16}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{16}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = Replicate{VL}(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RQH (scalar plus immediate)

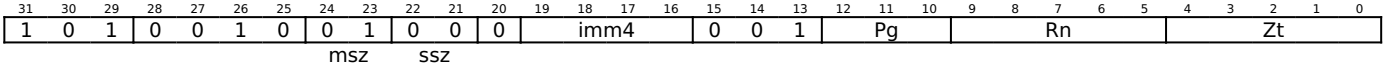
Contiguous load and replicate eight halfwords (immediate index)

This instruction loads eight contiguous halfwords to elements of a short, 128-bit (quadword) vector from the memory address generated by a 64-bit scalar base address and immediate index that is a multiple of 16 in the range -128 to +112 added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero. The resulting short vector is then replicated to fill the long destination vector. Only the first eight predicate elements are used and higher numbered predicate elements are ignored.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RQH { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 16 in the range -128 to 112, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = 128 DIV esize;
var base : bits(64);
let mask : bits(16) = P{PL}(g)[15:0]; // low 16 bits only
var addr : bits(64);
var result : bits(128);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{16}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * 16, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{16}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = Replicate{VL}(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RQH (scalar plus scalar)

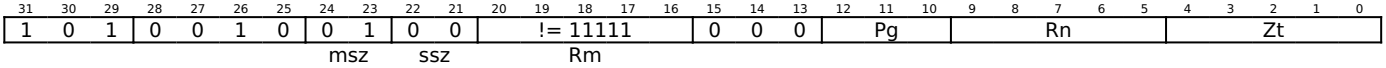
Contiguous load and replicate eight halfwords (scalar index)

This instruction loads eight contiguous halfwords to elements of a short, 128-bit (quadword) vector from the memory address generated by a 64-bit scalar base address and scalar index that is multiplied by 2 and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero. The resulting short vector is then replicated to fill the long destination vector. Only the first eight predicate elements are used and higher numbered predicate elements are ignored.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RQH { <Zt>.H }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = 128 DIV esize;
var base : bits(64);
let mask : bits(16) = P{PL}(g)[15:0]; // low 16 bits only
var offset : bits(64);
var addr : bits(64);
var result : bits(128);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{16}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{16}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = Replicate{VL}(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RQW (scalar plus immediate)

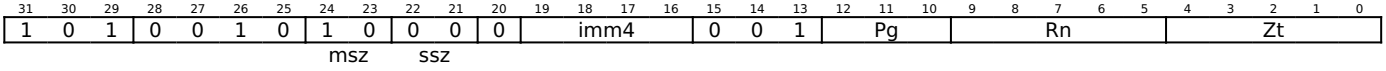
Contiguous load and replicate four words (immediate index)

This instruction loads four contiguous words to elements of a short, 128-bit (quadword) vector from the memory address generated by a 64-bit scalar base address and immediate index that is a multiple of 16 in the range -128 to +112 added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero. The resulting short vector is then replicated to fill the long destination vector. Only the first four predicate elements are used and higher numbered predicate elements are ignored.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RQW { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate byte offset, a multiple of 16 in the range -128 to 112, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = 128 DIV esize;
var base : bits(64);
let mask : bits(16) = P{PL}(g)[15:0]; // low 16 bits only
var addr : bits(64);
var result : bits(128);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{16}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * 16, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{16}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = Replicate{VL}(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RQW (scalar plus scalar)

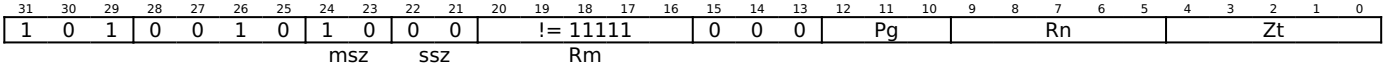
Contiguous load and replicate four words (scalar index)

This instruction loads four contiguous words to elements of a short, 128-bit (quadword) vector from the memory address generated by a 64-bit scalar base address and scalar index that is multiplied by 4 and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero. The resulting short vector is then replicated to fill the long destination vector. Only the first four predicate elements are used and higher numbered predicate elements are ignored.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RQW { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = 128 DIV esize;
var base : bits(64);
let mask : bits(16) = P{PL}(g)[15:0]; // low 16 bits only
var offset : bits(64);
var addr : bits(64);
var result : bits(128);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{16}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{16}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = Replicate{VL}(result);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RSB

Load and broadcast signed byte to vector

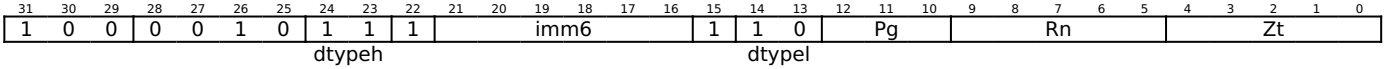
This instruction loads a single signed byte from a memory address generated by a 64-bit scalar base address plus an immediate offset that is in the range 0 to 63.

The loaded data is broadcast into all Active elements of the destination vector, setting the Inactive elements to zero. If all elements are inactive, then the instruction will not perform a read from Device memory or cause a data abort.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE || FEAT_SME)



Encoding

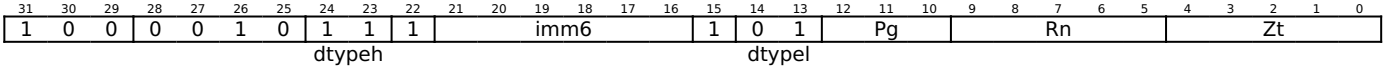
```
LD1RSB { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm6);
```

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

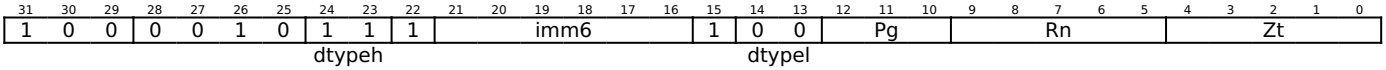
```
LD1RSB { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm6);
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RSB { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm6);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional unsigned immediate byte offset, in the range 0 to 63, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    let addr : bits(64) = AddressAdd(base, offset * mbytes, accdesc);
    data = Mem{msize}(addr, accdesc);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RSH

Load and broadcast signed halfword to vector

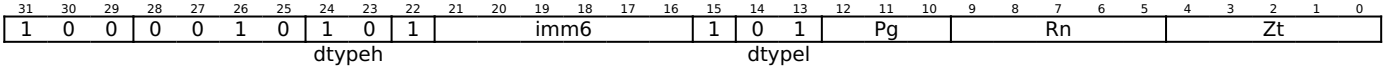
This instruction loads a single signed halfword from a memory address generated by a 64-bit scalar base address plus an immediate offset that is a multiple of 2 in the range 0 to 126.

The loaded data is broadcast into all Active elements of the destination vector, setting the Inactive elements to zero. If all elements are inactive, then the instruction will not perform a read from Device memory or cause a data abort.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

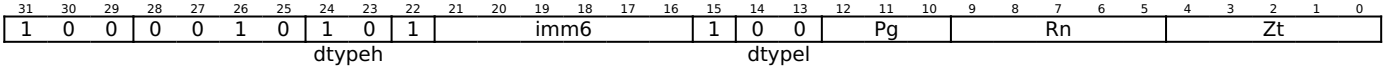
```
LD1RSH { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm6);
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RSH { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm6);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 2 in the range 0 to 126, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    let addr : bits(64) = AddressAdd(base, offset * mbytes, accdesc);
    data = Mem{msize}(addr, accdesc);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RSW

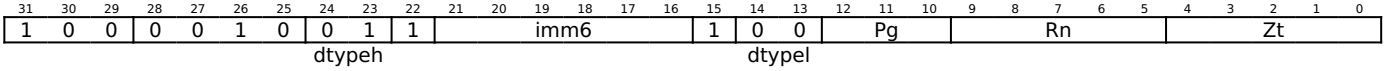
Load and broadcast signed word to vector

This instruction loads a single signed word from a memory address generated by a 64-bit scalar base address plus an immediate offset that is a multiple of 4 in the range 0 to 252.

The loaded data is broadcast into all Active elements of the destination vector, setting the Inactive elements to zero. If all elements are inactive, then the instruction will not perform a read from Device memory or cause a data abort.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RSW { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm6);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 4 in the range 0 to 252, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    let addr : bits(64) = AddressAdd(base, offset * mbytes, accdesc);
    data = Mem{msize}(addr, accdesc);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1RW

Load and broadcast unsigned word to vector

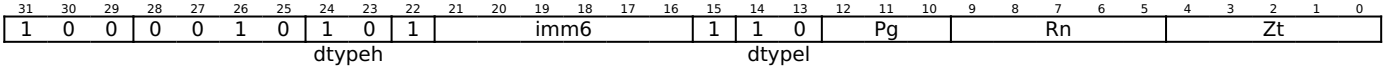
This instruction loads a single unsigned word from a memory address generated by a 64-bit scalar base address plus an immediate offset that is a multiple of 4 in the range 0 to 252.

The loaded data is broadcast into all Active elements of the destination vector, setting the Inactive elements to zero. If all elements are inactive, then the instruction will not perform a read from Device memory or cause a data abort.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

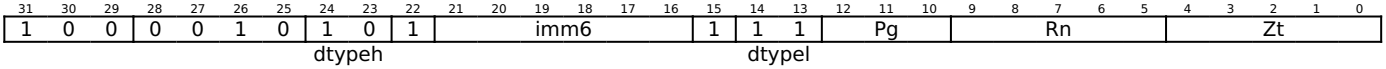
```
LD1RW { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm6);
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1RW { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm6);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 4 in the range 0 to 252, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    let addr : bits(64) = AddressAdd(base, offset * mbytes, accdesc);
    data = Mem{msize}(addr, accdesc);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1SB (scalar plus immediate)

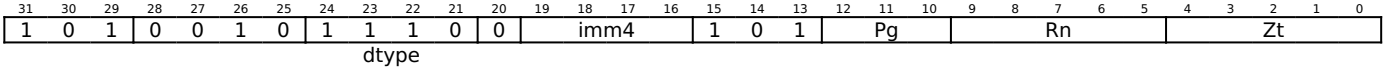
Contiguous load signed bytes to vector (immediate index)

This instruction performs a contiguous load of signed bytes to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE || FEAT_SME)



Encoding

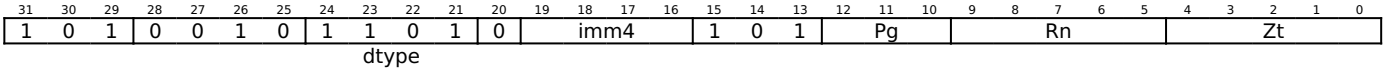
```
LD1SB { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

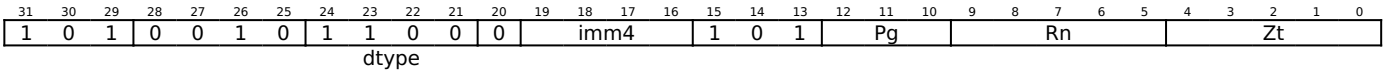
```
LD1SB { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1SB { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1SB (scalar plus scalar)

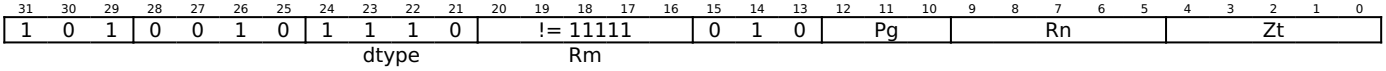
Contiguous load signed bytes to vector (scalar index)

This instruction performs a contiguous load of signed bytes to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE || FEAT_SME)



Encoding

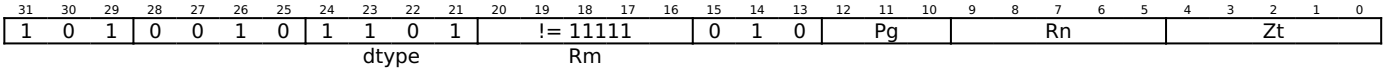
```
LD1SB { <Zt>.H }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
```

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

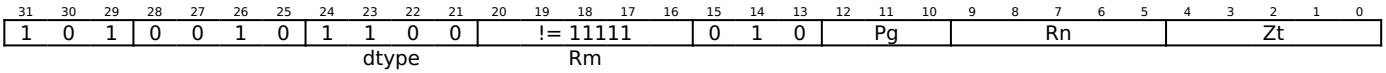
```
LD1SB { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1SB { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1SB (scalar plus vector)

Gather load signed bytes to vector (vector index)

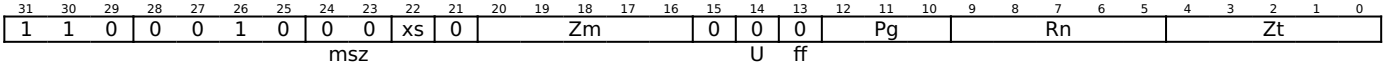
This instruction performs a gather load of signed bytes to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally sign-extended or zero-extended from 32 to 64 bits. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 32-bit unpacked unscaled offset , 32-bit unscaled offset and 64-bit unscaled offset

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

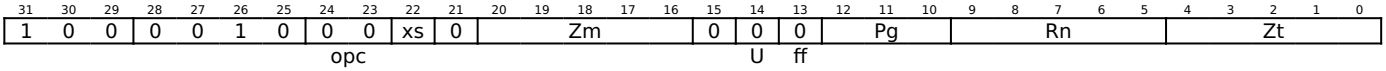
```
LD1SB { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

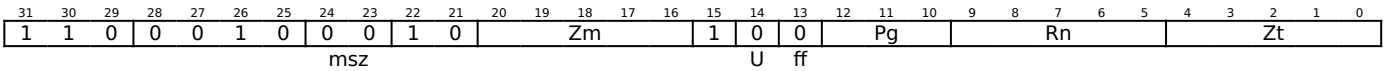
```
LD1SB { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LD1SB { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offs_size : integer{} = 64;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1SB (vector plus immediate)

Gather load signed bytes to vector (immediate index)

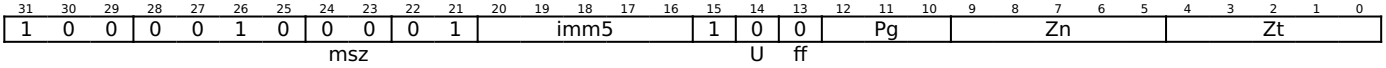
This instruction performs a gather load of signed bytes to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is in the range 0 to 31. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

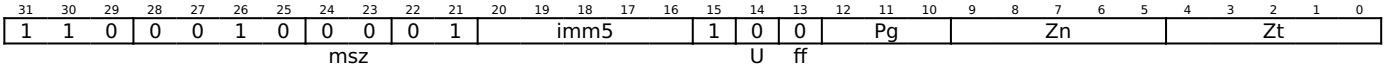
```
LD1SB { <Zt>.S }, <Pg>/Z, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
LD1SB { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, in the range 0 to 31, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1SH (scalar plus immediate)

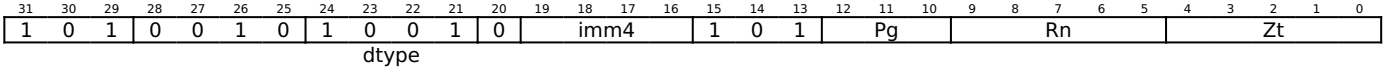
Contiguous load signed halfwords to vector (immediate index)

This instruction performs a contiguous load of signed halfwords to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

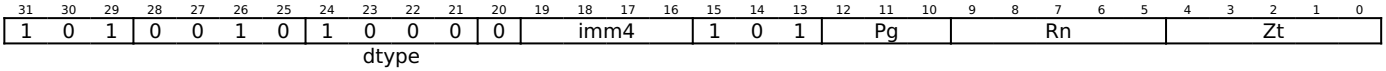
```
LD1SH { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1SH (scalar plus scalar)

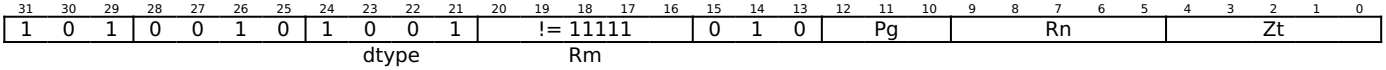
Contiguous load signed halfwords to vector (scalar index)

This instruction performs a contiguous load of signed halfwords to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 2 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

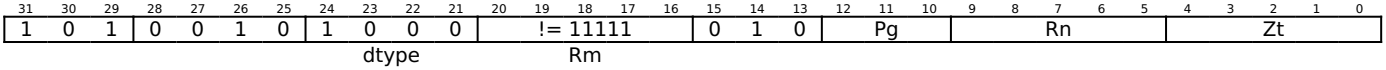
```
LD1SH { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1SH (scalar plus vector)

Gather load signed halfwords to vector (vector index)

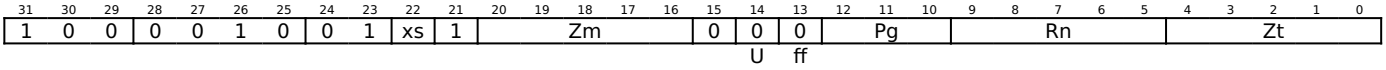
This instruction performs a gather load of signed halfwords to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 2. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 6 classes: 32-bit scaled offset , 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 32-bit unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

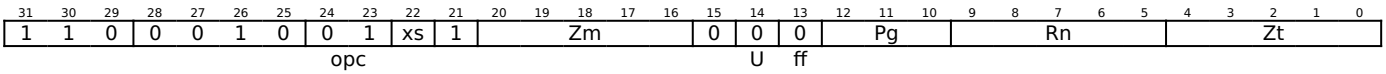
```
LD1SH { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 1;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

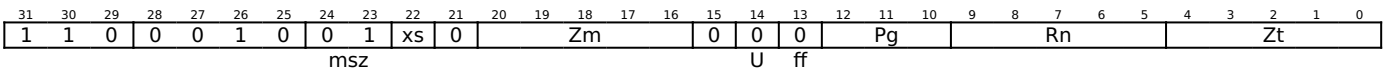
```
LD1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 1;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

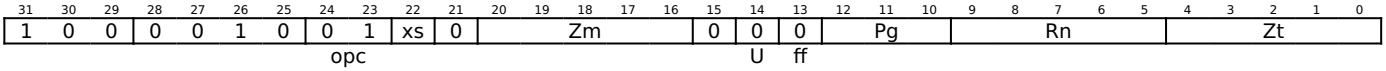
```
LD1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

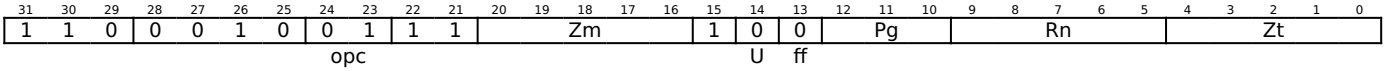
```
LD1SH { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

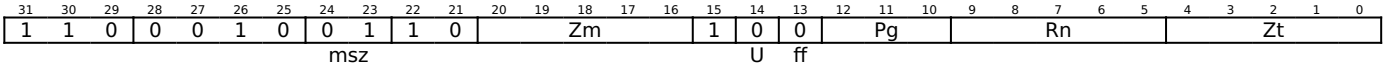
```
LD1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 64;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 1;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LD1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 64;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1SH (vector plus immediate)

Gather load signed halfwords to vector (immediate index)

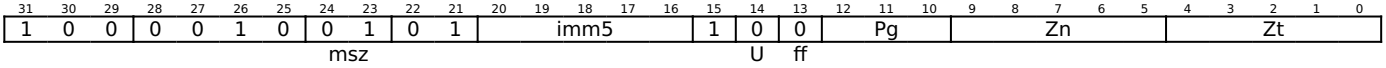
This instruction performs a gather load of signed halfwords to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is a multiple of 2 in the range 0 to 62. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

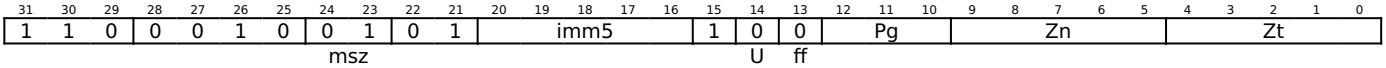
```
LD1SH { <Zt>.S }, <Pg>/Z, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
LD1SH { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 2 in the range 0 to 62, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

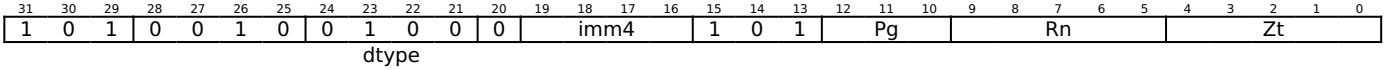
LD1SW (scalar plus immediate)

Contiguous load signed words to vector (immediate index)

This instruction performs a contiguous load of signed words to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

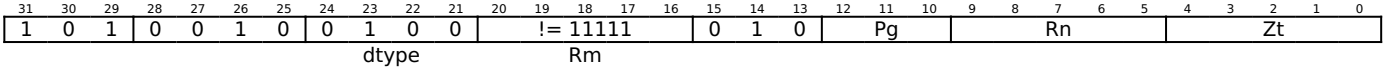
LD1SW (scalar plus scalar)

Contiguous load signed words to vector (scalar index)

This instruction performs a contiguous load of signed words to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 4 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1SW (scalar plus vector)

Gather load signed words to vector (vector index)

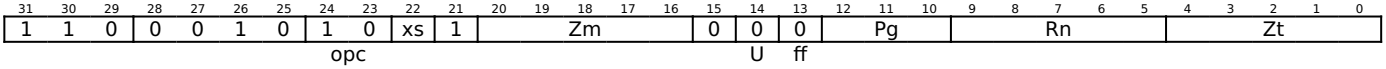
This instruction performs a gather load of signed words to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 4. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 4 classes: 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

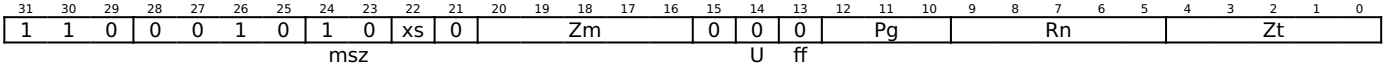
```
LD1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod> #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 2;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

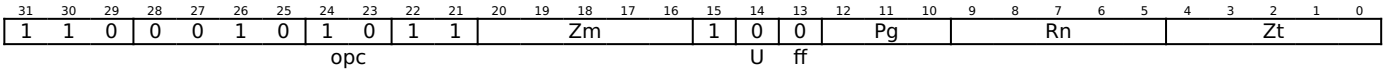
```
LD1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

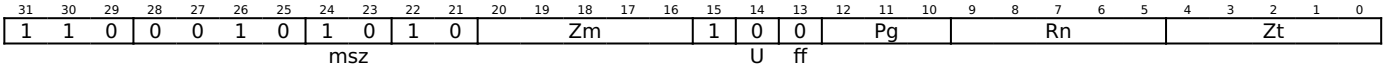
```
LD1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 64;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 2;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LD1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 64;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

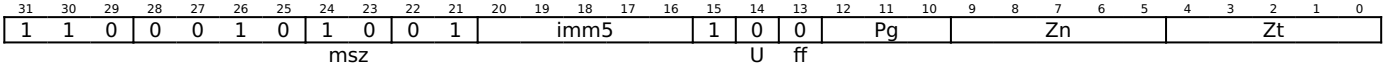
LD1SW (vector plus immediate)

Gather load signed words to vector (immediate index)

This instruction performs a gather load of signed words to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is a multiple of 4 in the range 0 to 124. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.
This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
LD1SW { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 4 in the range 0 to 124, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1W (scalar plus immediate, consecutive registers)

Contiguous load of words to multiple consecutive vectors (immediate index)

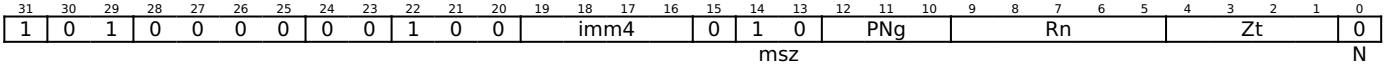
This instruction performs a contiguous load of unsigned words to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

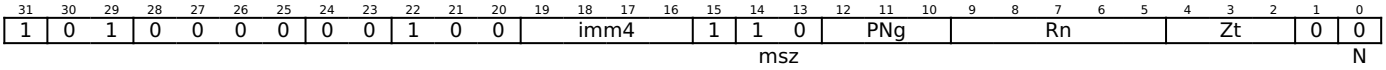
```
LD1W { <Zt1>.S-<Zt2>.S }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LD1W { <Zt1>.S-<Zt4>.S }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1W (scalar plus immediate, single register)

Contiguous load unsigned words to vector (immediate index)

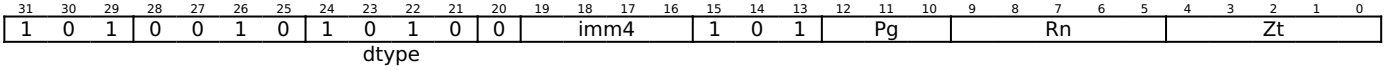
This instruction performs a contiguous load of unsigned words to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

The 128-bit element variant is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 32-bit element , 64-bit element and 128-bit element

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

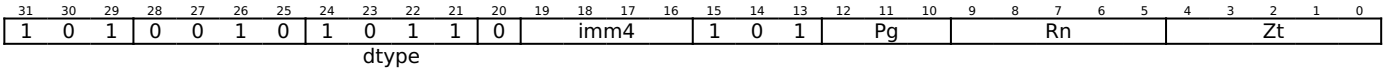
```
LD1W { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

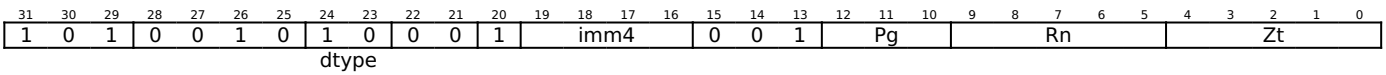
```
LD1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

128-bit element

(FEAT_SVE2p1)



Encoding

```
LD1W { <Zt>.Q }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
if esize < 128 then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1W (scalar plus scalar, consecutive registers)

Contiguous load of words to multiple consecutive vectors (scalar index)

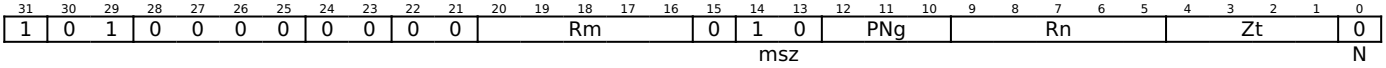
This instruction performs a contiguous load of unsigned words to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

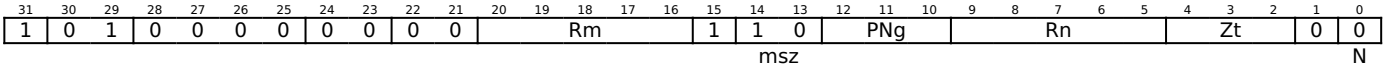
```
LD1W { <Zt1>.S-<Zt2>.S }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 32;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LD1W { <Zt1>.S-<Zt4>.S }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 32;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1W (scalar plus scalar, single register)

Contiguous load unsigned words to vector (scalar index)

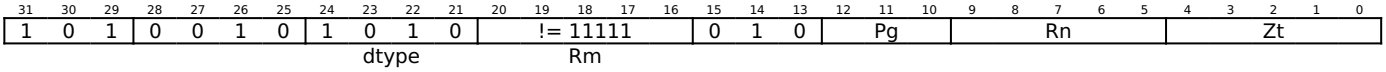
This instruction performs a contiguous load of unsigned words to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 4 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

The 128-bit element variant is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 32-bit element , 64-bit element and 128-bit element

32-bit element

(FEAT_SVE || FEAT_SME)



Encoding

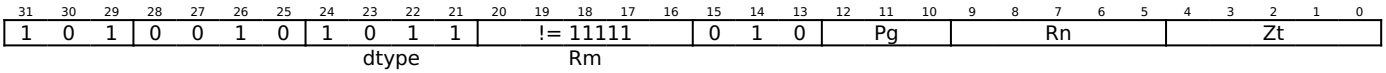
```
LD1W { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
```

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

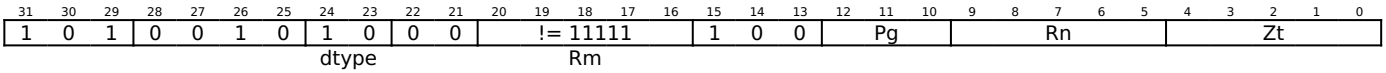
```
LD1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
```

128-bit element

(FEAT_SVE2p1)



Encoding

```
LD1W { <Zt>.Q }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
if esize < 128 then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1W (scalar plus vector)

Gather load unsigned words to vector (vector index)

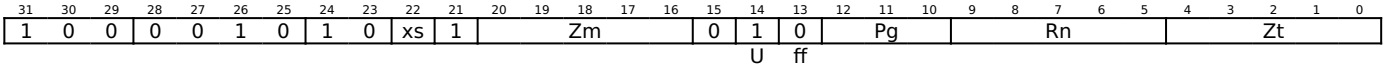
This instruction performs a gather load of unsigned words to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 4. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 6 classes: 32-bit scaled offset , 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 32-bit unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

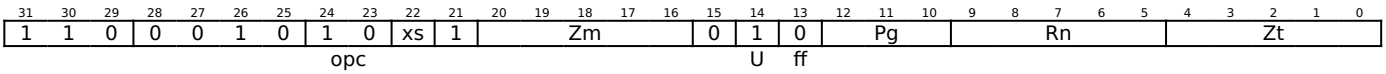
```
LD1W { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod> #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 2;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

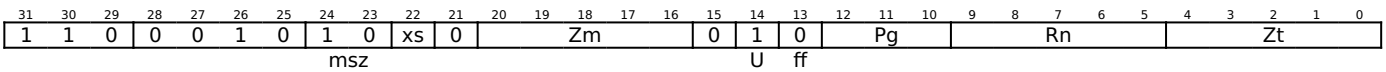
```
LD1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod> #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 2;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

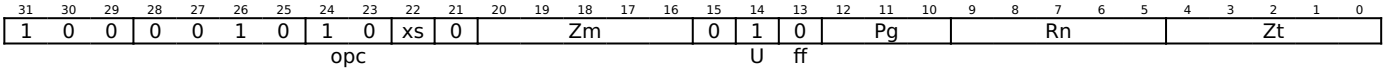
```
LD1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

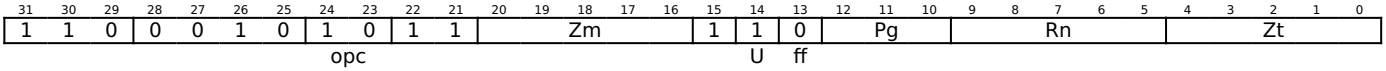
```
LD1W { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

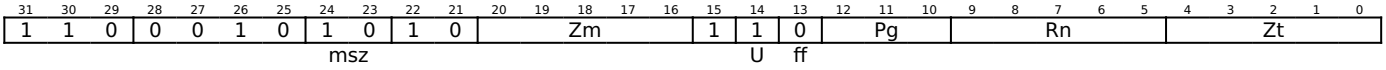
```
LD1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 2;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LD1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1W (vector plus immediate)

Gather load unsigned words to vector (immediate index)

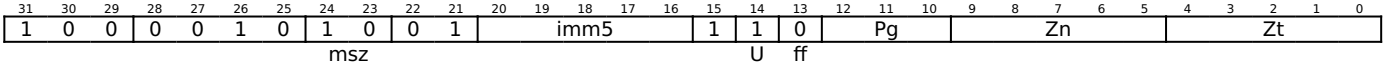
This instruction performs a gather load of unsigned words to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is a multiple of 4 in the range 0 to 124. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

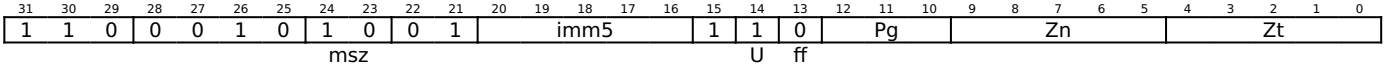
```
LD1W { <Zt>.S }, <Pg>/Z, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
LD1W { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 4 in the range 0 to 124, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);
if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;
Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2B (scalar plus immediate)

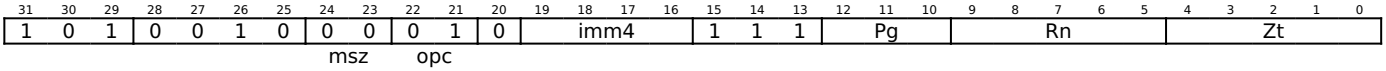
Contiguous load two-byte structures to two vectors (immediate index)

This instruction performs a contiguous load of two-byte structures, each to the same element number in two vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 2 in the range -16 to 14 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive bytes in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the two destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD2B { <Zt1>.B, <Zt2>.B }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2B (scalar plus scalar)

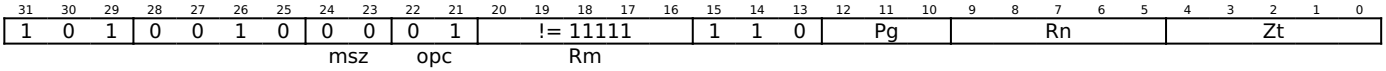
Contiguous load two-byte structures to two vectors (scalar index)

This instruction performs a contiguous load of two-byte structures, each to the same element number in two vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register and added to the base address. After each structure access the index value is incremented by two. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive bytes in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the two destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD2B { <Zt1>.B, <Zt2>.B }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2D (scalar plus immediate)

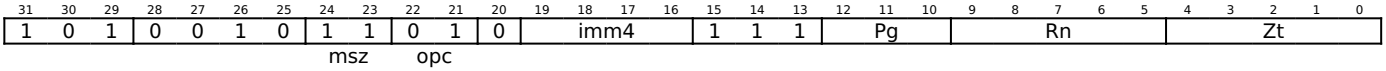
Contiguous load two-doubleword structures to two vectors (immediate index)

This instruction performs a contiguous load of two-doubleword structures, each to the same element number in two vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 2 in the range -16 to 14 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive doublewords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the two destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD2D { <Zt1>.D, <Zt2>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2D (scalar plus scalar)

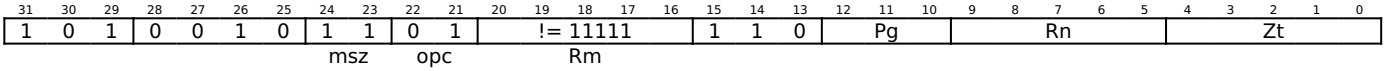
Contiguous load two-doubleword structures to two vectors (scalar index)

This instruction performs a contiguous load of two-doubleword structures, each to the same element number in two vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by two. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive doublewords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the two destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD2D { <Zt1>.D, <Zt2>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2H (scalar plus immediate)

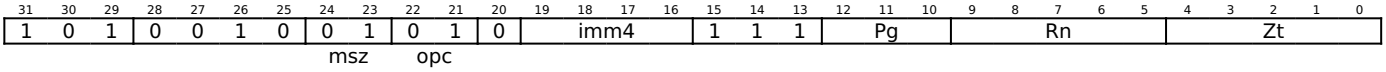
Contiguous load two-halfword structures to two vectors (immediate index)

This instruction performs a contiguous load of two-halfword structures, each to the same element number in two vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 2 in the range -16 to 14 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive halfwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the two destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD2H { <Zt1>.H, <Zt2>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2H (scalar plus scalar)

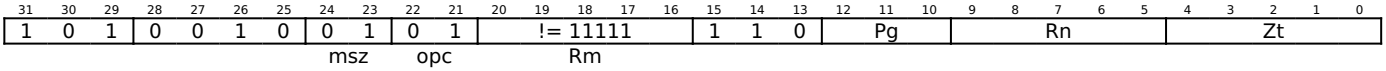
Contiguous load two-halfword structures to two vectors (scalar index)

This instruction performs a contiguous load of two-halfword structures, each to the same element number in two vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by two. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive halfwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the two destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD2H { <Zt1>.H, <Zt2>.H }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2Q (scalar plus immediate)

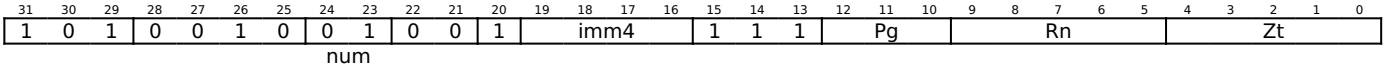
Contiguous load two-quadword structures to two vectors (immediate index)

This instruction performs a contiguous load of two-quadword structures, each to the same element number in two vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 2 in the range -16 to 14 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive quadwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the two destination vector registers.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
LD2Q { <Zt1>.Q, <Zt2>.Q }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let offset : integer = SInt(imm4);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2Q (scalar plus scalar)

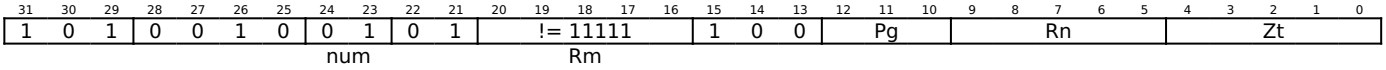
Contiguous load two-quadword structures to two vectors (scalar index)

This instruction performs a contiguous load of two-quadword structures, each to the same element number in two vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by two. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive quadwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the two destination vector registers.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
LD2Q { <Zt1>.Q, <Zt2>.Q }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #4]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2W (scalar plus immediate)

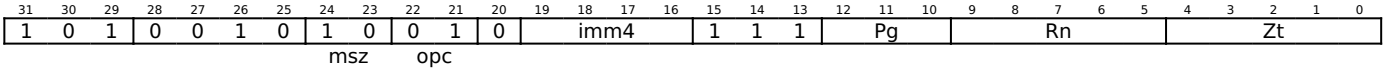
Contiguous load two-word structures to two vectors (immediate index)

This instruction performs a contiguous load of two-word structures, each to the same element number in two vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 2 in the range -16 to 14 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive words in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the two destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD2W { <Zt1>.S, <Zt2>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD2W (scalar plus scalar)

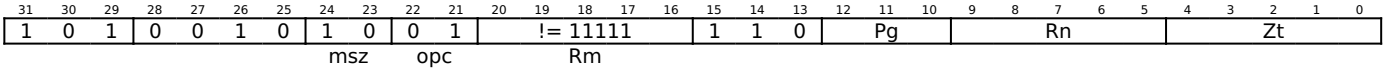
Contiguous load two-word structures to two vectors (scalar index)

This instruction performs a contiguous load of two-word structures, each to the same element number in two vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by two. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive words in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the two destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD2W { <Zt1>.S, <Zt2>.S }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3B (scalar plus immediate)

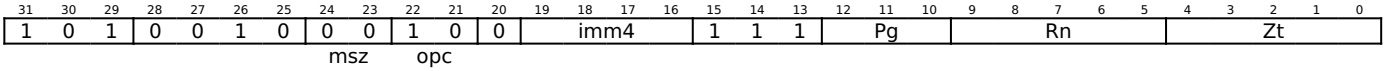
Contiguous load three-byte structures to three vectors (immediate index)

This instruction performs a contiguous load of three-byte structures, each to the same element number in three vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 3 in the range -24 to 21 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive bytes in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the three destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD3B { <Zt1>.B, <Zt2>.B, <Zt3>.B }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 3 in the range -24 to 21, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3B (scalar plus scalar)

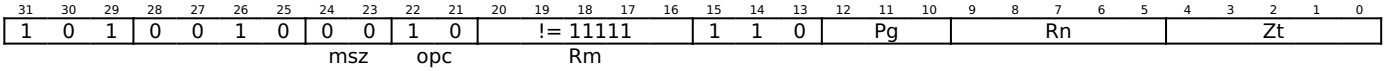
Contiguous load three-byte structures to three vectors (scalar index)

This instruction performs a contiguous load of three-byte structures, each to the same element number in three vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register and added to the base address. After each structure access the index value is incremented by three. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive bytes in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the three destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD3B { <Zt1>.B, <Zt2>.B, <Zt3>.B }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3D (scalar plus immediate)

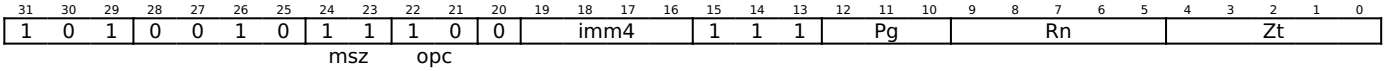
Contiguous load three-doubleword structures to three vectors (immediate index)

This instruction performs a contiguous load of three-doubleword structures, each to the same element number in three vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 3 in the range -24 to 21 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive doublewords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the three destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD3D { <Zt1>.D, <Zt2>.D, <Zt3>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 3 in the range -24 to 21, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3D (scalar plus scalar)

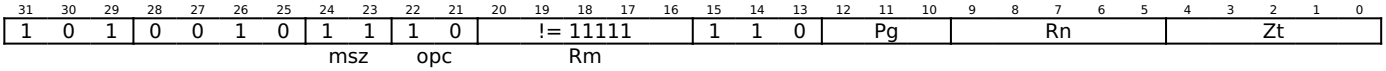
Contiguous load three-doubleword structures to three vectors (scalar index)

This instruction performs a contiguous load of three-doubleword structures, each to the same element number in three vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by three. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive doublewords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the three destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD3D { <Zt1>.D, <Zt2>.D, <Zt3>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3H (scalar plus immediate)

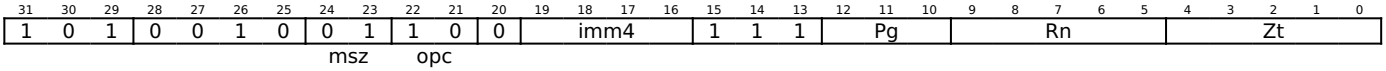
Contiguous load three-halfword structures to three vectors (immediate index)

This instruction performs a contiguous load of three-halfword structures, each to the same element number in three vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 3 in the range -24 to 21 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive halfwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the three destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD3H { <Zt1>.H, <Zt2>.H, <Zt3>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 3 in the range -24 to 21, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3H (scalar plus scalar)

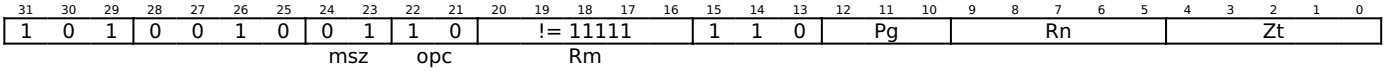
Contiguous load three-halfword structures to three vectors (scalar index)

This instruction performs a contiguous load of three-halfword structures, each to the same element number in three vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by three. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive halfwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the three destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD3H { <Zt1>.H, <Zt2>.H, <Zt3>.H }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3Q (scalar plus immediate)

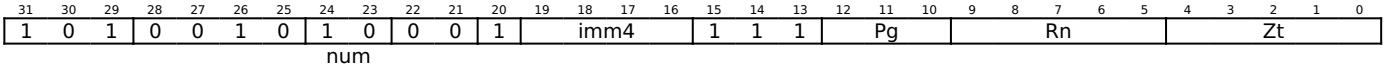
Contiguous load three-quadword structures to three vectors (immediate index)

This instruction performs a contiguous load of three-quadword structures, each to the same element number in three vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 3 in the range -24 to 21 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive quadwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the three destination vector registers.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
LD3Q { <Zt1>.Q, <Zt2>.Q, <Zt3>.Q }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let offset : integer = SInt(imm4);
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 3 in the range -24 to 21, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3Q (scalar plus scalar)

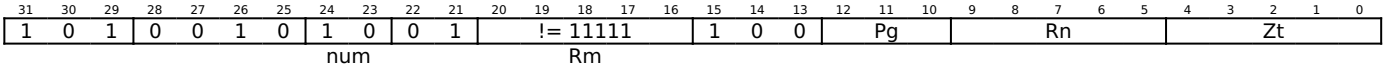
Contiguous load three-quadword structures to three vectors (scalar index)

This instruction performs a contiguous load of three-quadword structures, each to the same element number in three vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by three. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive quadwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the three destination vector registers.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
LD3Q { <Zt1>.Q, <Zt2>.Q, <Zt3>.Q }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #4]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3W (scalar plus immediate)

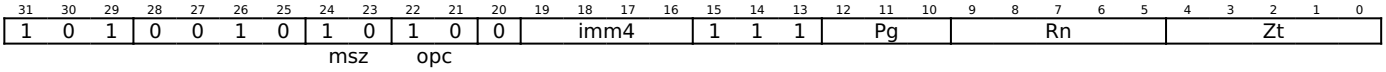
Contiguous load three-word structures to three vectors (immediate index)

This instruction performs a contiguous load of three-word structures, each to the same element number in three vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 3 in the range -24 to 21 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive words in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the three destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD3W { <Zt1>.S, <Zt2>.S, <Zt3>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 3 in the range -24 to 21, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD3W (scalar plus scalar)

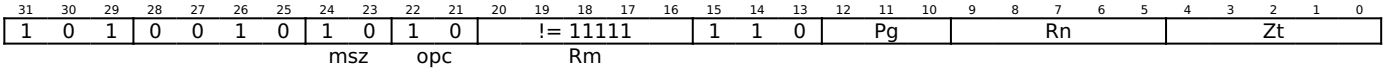
Contiguous load three-word structures to three vectors (scalar index)

This instruction performs a contiguous load of three-word structures, each to the same element number in three vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by three. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive words in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the three destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD3W { <Zt1>.S, <Zt2>.S, <Zt3>.S }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4B (scalar plus immediate)

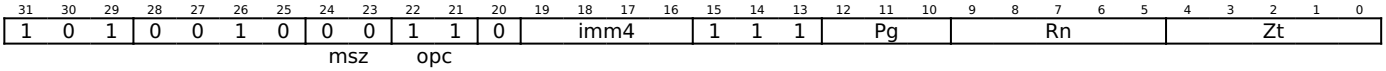
Contiguous load four-byte structures to four vectors (immediate index)

This instruction performs a contiguous load of four-byte structures, each to the same element number in four vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 4 in the range -32 to 28 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive bytes in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the four destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD4B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4B (scalar plus scalar)

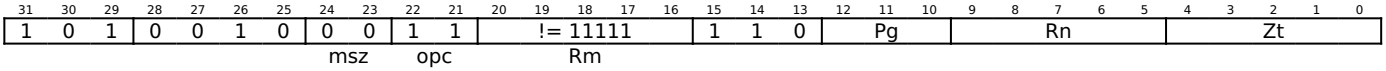
Contiguous load four-byte structures to four vectors (scalar index)

This instruction performs a contiguous load of four-byte structures, each to the same element number in four vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register and added to the base address. After each structure access the index value is incremented by four. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive bytes in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the four destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD4B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4D (scalar plus immediate)

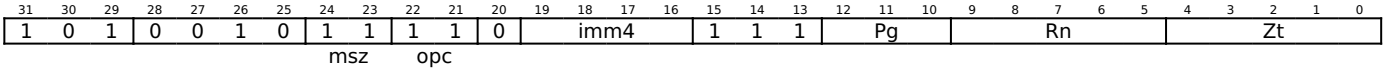
Contiguous load four-doubleword structures to four vectors (immediate index)

This instruction performs a contiguous load of four-doubleword structures, each to the same element number in four vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 4 in the range -32 to 28 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive doublewords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the four destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD4D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4D (scalar plus scalar)

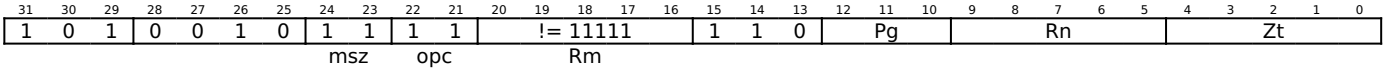
Contiguous load four-doubleword structures to four vectors (scalar index)

This instruction performs a contiguous load of four-doubleword structures, each to the same element number in four vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by four. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive doublewords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the four destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD4D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4H (scalar plus immediate)

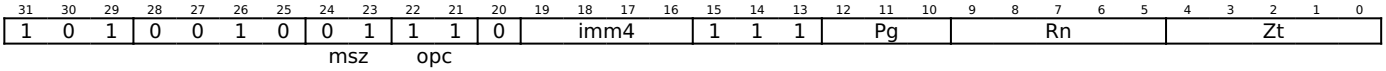
Contiguous load four-halfword structures to four vectors (immediate index)

This instruction performs a contiguous load of four-halfword structures, each to the same element number in four vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 4 in the range -32 to 28 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive halfwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the four destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD4H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4H (scalar plus scalar)

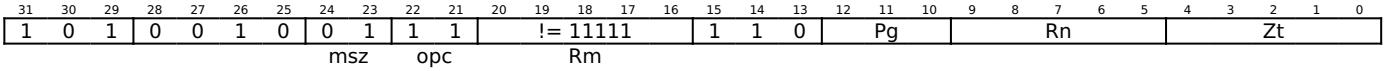
Contiguous load four-halfword structures to four vectors (scalar index)

This instruction performs a contiguous load of four-halfword structures, each to the same element number in four vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by four. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive halfwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the four destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD4H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4Q (scalar plus immediate)

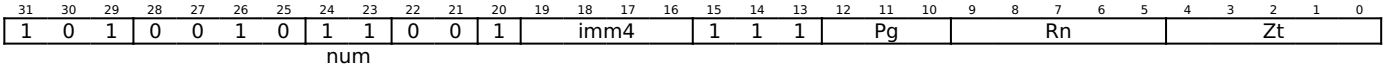
Contiguous load four-quadword structures to four vectors (immediate index)

This instruction performs a contiguous load of four-quadword structures, each to the same element number in four vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 4 in the range -32 to 28 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive quadwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the four destination vector registers.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
LD4Q { <Zt1>.Q, <Zt2>.Q, <Zt3>.Q, <Zt4>.Q }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let offset : integer = SInt(imm4);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4Q (scalar plus scalar)

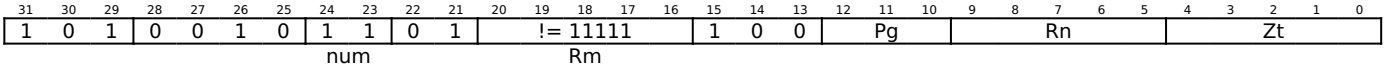
Contiguous load four-quadword structures to four vectors (scalar index)

This instruction performs a contiguous load of four-quadword structures, each to the same element number in four vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by four. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive quadwords in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the four destination vector registers.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
LD4Q { <Zt1>.Q, <Zt2>.Q, <Zt3>.Q, <Zt4>.Q }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #4]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4W (scalar plus immediate)

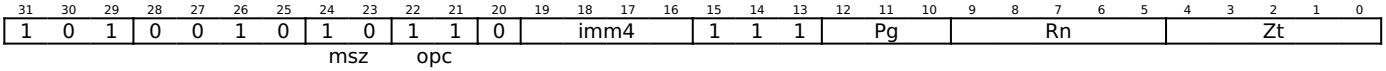
Contiguous load four-word structures to four vectors (immediate index)

This instruction performs a contiguous load of four-word structures, each to the same element number in four vector registers from the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 4 in the range -32 to 28 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive words in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the four destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD4W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD4W (scalar plus scalar)

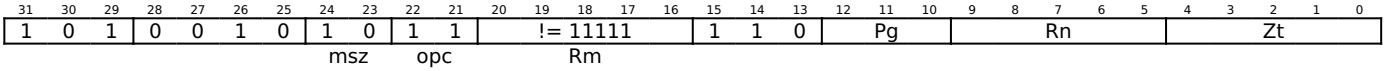
Contiguous load four-word structures to four vectors (scalar index)

This instruction performs a contiguous load of four-word structures, each to the same element number in four vector registers from the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by four. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive words in memory that make up each structure. Inactive elements will not cause a read from Device memory or signal a fault, and the corresponding element is set to zero in each of the four destination vector registers.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LD4W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}((t+r) MOD 32) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDAPRO

Load-Acquire RCpc octaword

This instruction loads thirty-two contiguous bytes to elements of a 256-bit (octaword) vector from the memory address generated by a 64-bit scalar base, plus a scaled immediate offset. The base register must be aligned to 32 bytes if alignment is checked.

This instruction is unpredicated.

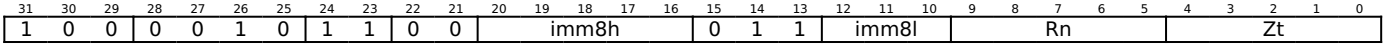
The instruction has memory ordering semantics as described in *Load-Acquire, Load-AcquirePC, and Store-Release*.

This instruction requires that the Effective SVE vector length is at least 256 bits.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE_LRCPC)



Encoding

```
LDAPRO <Zt>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_LRCPC) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let imm : integer = UInt(imm8h::imm8l);
let acquirepc : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional positive immediate byte offset, a multiple of 32 in the range 0 to 8160, defaulting to 0, encoded in the "imm8h:imm8l" fields.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
var base : bits(64);
let offset : integer = imm * 32;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescLDAcqPC(tagchecked, acquirepc, t);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

let addr : bits(64) = AddressAdd(base, offset, accdesc);

let data : bits(256) = Mem{256}(addr, accdesc);

Z{VL}(t) = ZeroExtend{VL}(data);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1B (scalar plus scalar)

Contiguous load first-fault unsigned bytes to vector (scalar index)

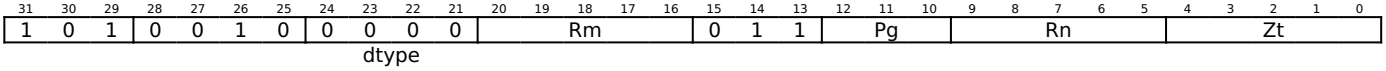
This instruction performs a contiguous load with first-faulting behavior of unsigned bytes to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 4 classes: 8-bit element , 16-bit element , 32-bit element and 64-bit element

8-bit element

(FEAT_SVE)



Encoding

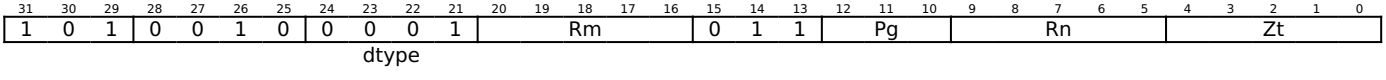
```
LDFF1B { <Zt>.B }, <Pg>/Z, [<Xn|SP>{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
```

16-bit element

(FEAT_SVE)



Encoding

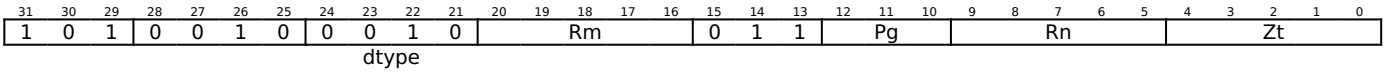
```
LDFF1B { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
```

32-bit element

(FEAT_SVE)



Encoding

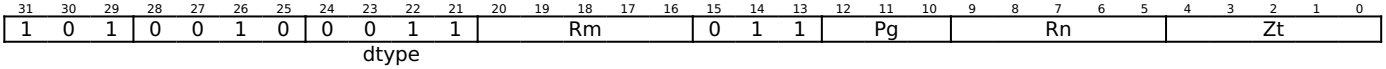
```
LDFF1B { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
```

64-bit element

(FEAT_SVE)



Encoding

```
LDFF1B { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;
Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1B (scalar plus vector)

Gather load first-fault unsigned bytes to vector (vector index)

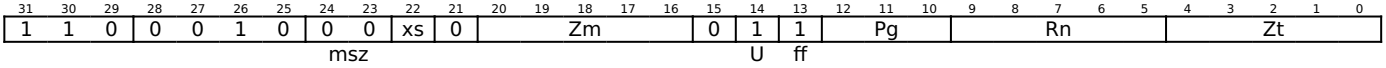
This instruction performs a gather load with first-faulting behavior of unsigned bytes to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally sign-extended or zero-extended from 32 to 64 bits. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 32-bit unpacked unscaled offset , 32-bit unscaled offset and 64-bit unscaled offset

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

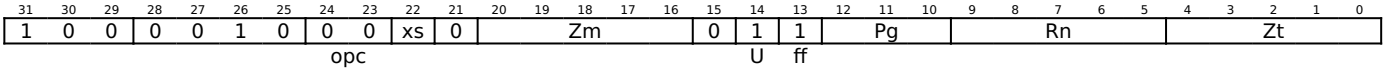
```
LDFF1B { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

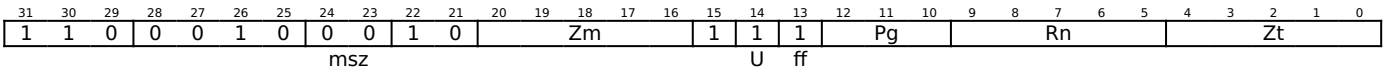
```
LDFF1B { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LDFF1B { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Zm> Is the name of the offset scalable vector register, encoded in the "Zm" field.

<mod> Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1B (vector plus immediate)

Gather load first-fault unsigned bytes to vector (immediate index)

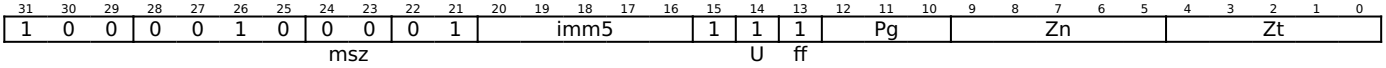
This instruction performs a gather load with first-faulting behavior of unsigned bytes to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is in the range 0 to 31. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

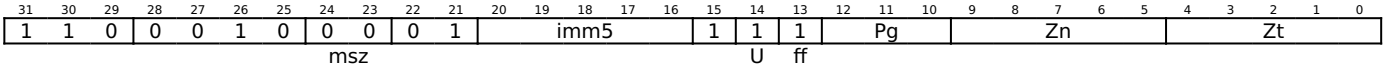
```
LDFF1B { <Zt>.S }, <Pg>/Z, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
LDFF1B { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, in the range 0 to 31, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1D (scalar plus scalar)

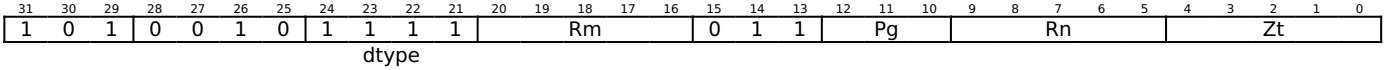
Contiguous load first-fault doublewords to vector (scalar index)

This instruction performs a contiguous load with first-faulting behavior of doublewords to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 8 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
LDFF1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #3}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;
Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1D (scalar plus vector)

Gather load first-fault doublewords to vector (vector index)

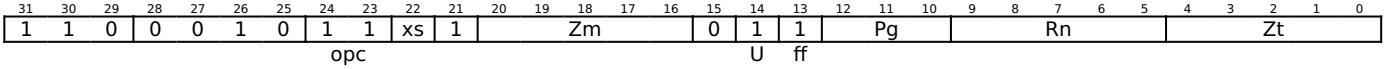
This instruction performs a gather load with first-faulting behavior of doublewords to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 8. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 4 classes: 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

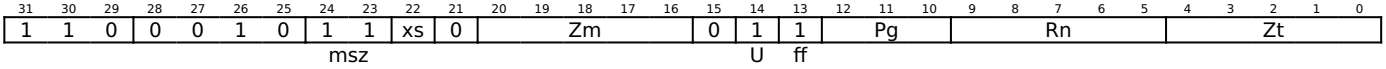
```
LDFF1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod> #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 3;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

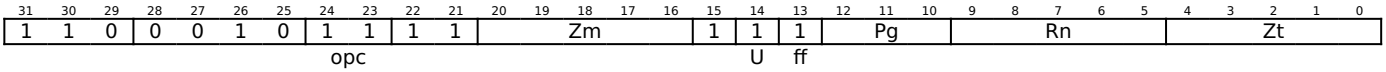
```
LDFF1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

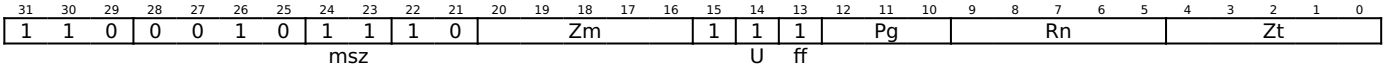
```
LDFF1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 3;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LDFF1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1D (vector plus immediate)

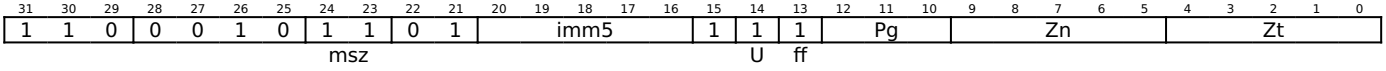
Gather load first-fault doublewords to vector (immediate index)

This instruction performs a gather load with first-faulting behavior of doublewords to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is a multiple of 8 in the range 0 to 248. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
LDFF1D { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 8 in the range 0 to 248, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1H (scalar plus scalar)

Contiguous load first-fault unsigned halfwords to vector (scalar index)

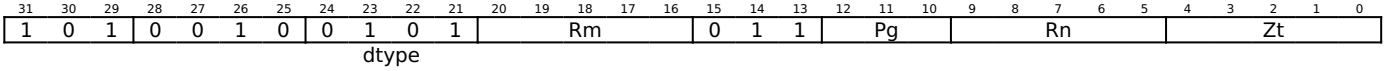
This instruction performs a contiguous load with first-faulting behavior of unsigned halfwords to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 2 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE)



Encoding

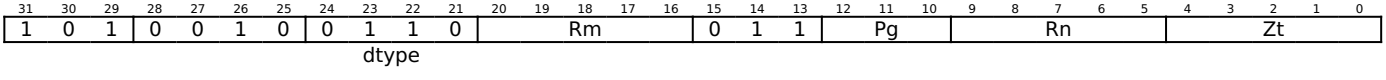
```
LDFF1H { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #1}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
```

32-bit element

(FEAT_SVE)



Encoding

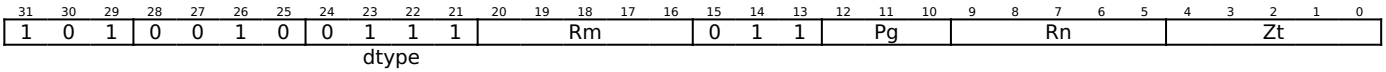
```
LDFF1H { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #1}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
```

64-bit element

(FEAT_SVE)



Encoding

```
LDFF1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #1}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;
Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1H (scalar plus vector)

Gather load first-fault unsigned halfwords to vector (vector index)

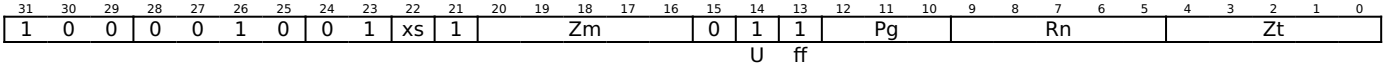
This instruction performs a gather load with first-faulting behavior of unsigned halfwords to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 2. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 6 classes: 32-bit scaled offset , 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 32-bit unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

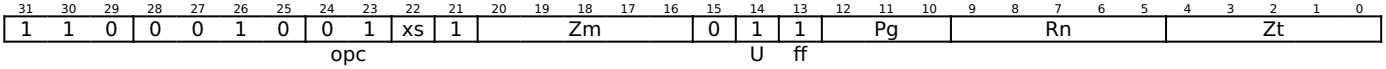
```
LDFF1H { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 1;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

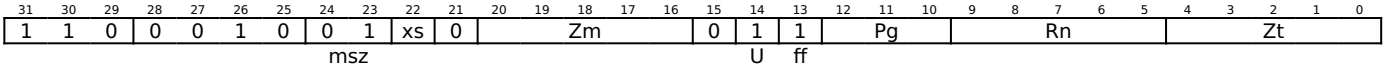
```
LDFF1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 1;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

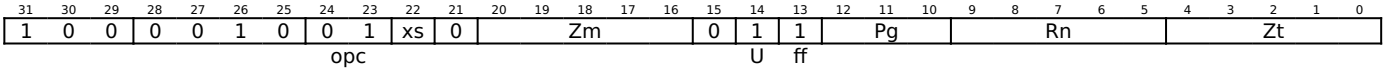
```
LDFF1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

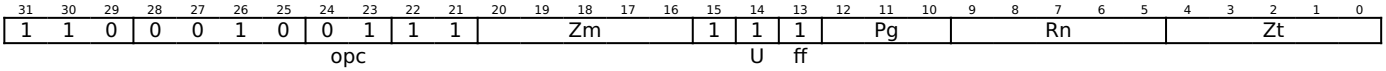
```
LDFF1H { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

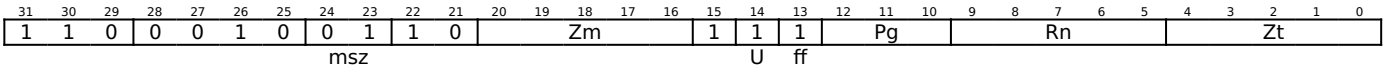
```
LDFF1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 1;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LDFF1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Zm> Is the name of the offset scalable vector register, encoded in the "Zm" field.

<mod> Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1H (vector plus immediate)

Gather load first-fault unsigned halfwords to vector (immediate index)

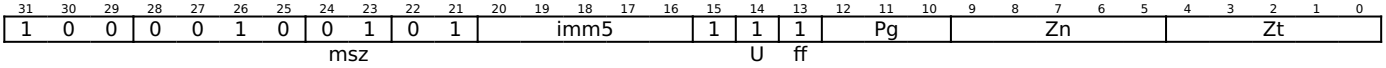
This instruction performs a gather load with first-faulting behavior of unsigned halfwords to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is a multiple of 2 in the range 0 to 62. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

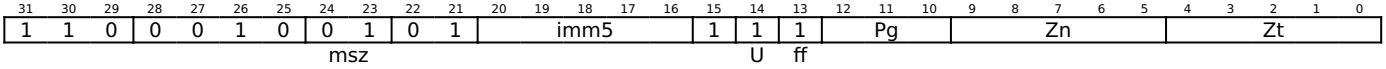
```
LDFF1H { <Zt>.S }, <Pg>/Z, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
LDFF1H { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 2 in the range 0 to 62, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1SB (scalar plus scalar)

Contiguous load first-fault signed bytes to vector (scalar index)

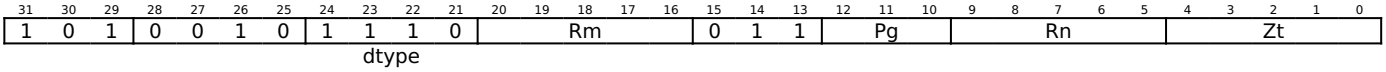
This instruction performs a contiguous load with first-faulting behavior of signed bytes to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE)



Encoding

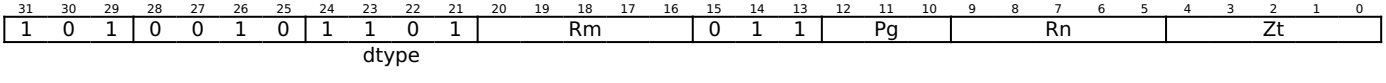
```
LDFF1SB { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
```

32-bit element

(FEAT_SVE)



Encoding

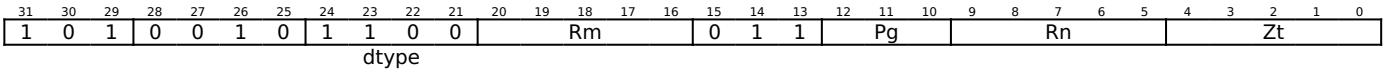
```
LDFF1SB { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
```

64-bit element

(FEAT_SVE)



Encoding

```
LDFF1SB { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;
Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1SB (scalar plus vector)

Gather load first-fault signed bytes to vector (vector index)

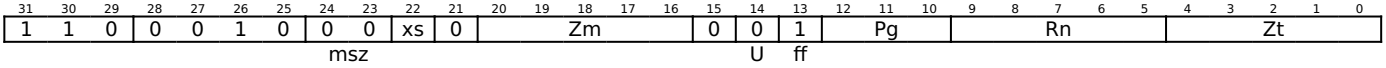
This instruction performs a gather load with first-faulting behavior of signed bytes to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally sign-extended or zero-extended from 32 to 64 bits. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 32-bit unpacked unscaled offset , 32-bit unscaled offset and 64-bit unscaled offset

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

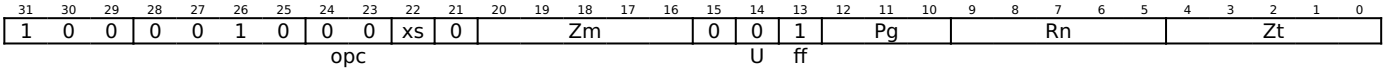
```
LDFF1SB { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

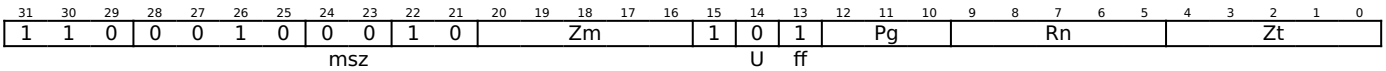
```
LDFF1SB { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LDFF1SB { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offs_size : integer{} = 64;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Zm> Is the name of the offset scalable vector register, encoded in the "Zm" field.

<mod> Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1SB (vector plus immediate)

Gather load first-fault signed bytes to vector (immediate index)

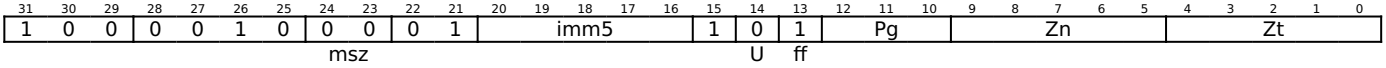
This instruction performs a gather load with first-faulting behavior of signed bytes to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is in the range 0 to 31. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

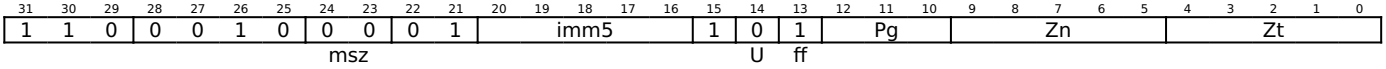
```
LDFF1SB { <Zt>.S }, <Pg>/Z, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
LDFF1SB { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, in the range 0 to 31, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1SH (scalar plus scalar)

Contiguous load first-fault signed halfwords to vector (scalar index)

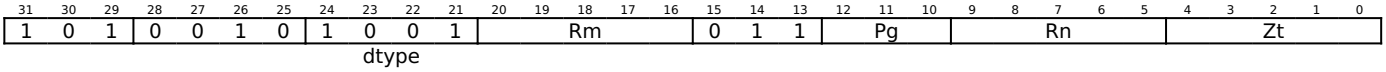
This instruction performs a contiguous load with first-faulting behavior of signed halfwords to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 2 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

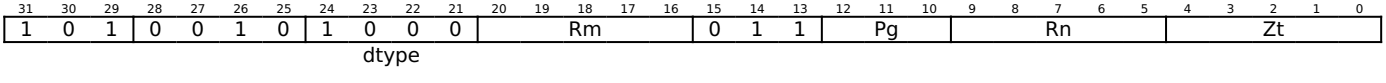
```
LDFF1SH { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #1}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
```

64-bit element

(FEAT_SVE)



Encoding

```
LDFF1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #1}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;
Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1SH (scalar plus vector)

Gather load first-fault signed halfwords to vector (vector index)

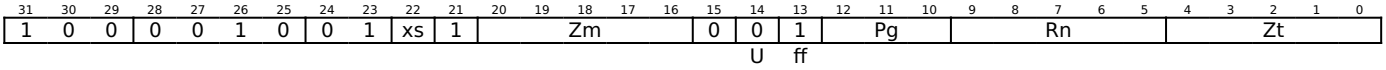
This instruction performs a gather load with first-faulting behavior of signed halfwords to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 2. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 6 classes: 32-bit scaled offset , 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 32-bit unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

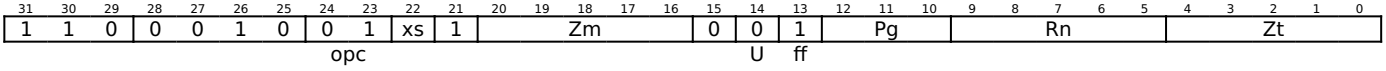
```
LDFF1SH { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 1;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

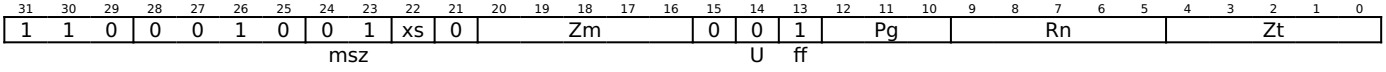
```
LDFF1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 1;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

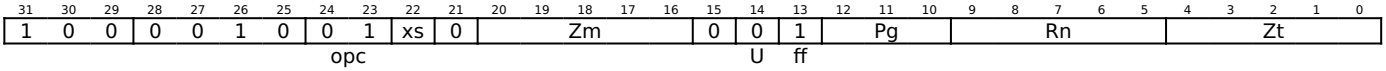
```
LDFF1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

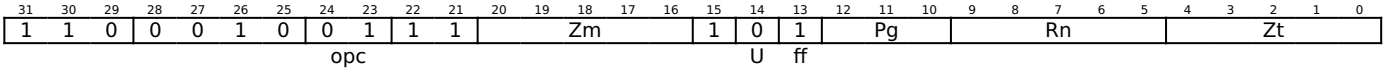
```
LDFF1SH { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

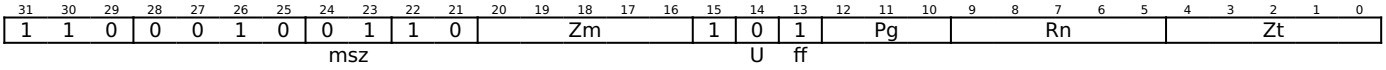
```
LDFF1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 64;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 1;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LDFF1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 64;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Zm> Is the name of the offset scalable vector register, encoded in the "Zm" field.

<mod> Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1SH (vector plus immediate)

Gather load first-fault signed halfwords to vector (immediate index)

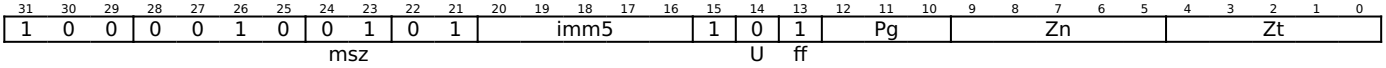
This instruction performs a gather load with first-faulting behavior of signed halfwords to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is a multiple of 2 in the range 0 to 62. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

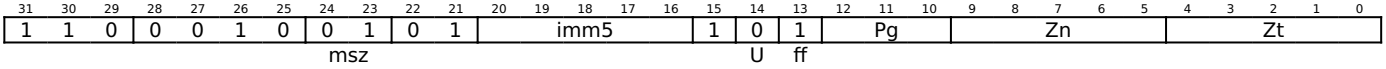
```
LDFF1SH { <Zt>.S }, <Pg>/Z, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
LDFF1SH { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 2 in the range 0 to 62, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1SW (scalar plus scalar)

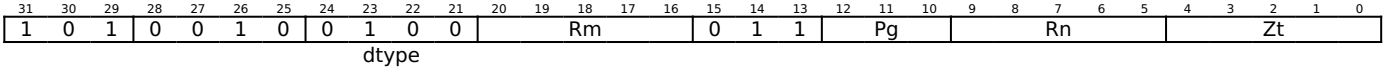
Contiguous load first-fault signed words to vector (scalar index)

This instruction performs a contiguous load with first-faulting behavior of signed words to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 4 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
LDFF1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #2}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;
Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1SW (scalar plus vector)

Gather load first-fault signed words to vector (vector index)

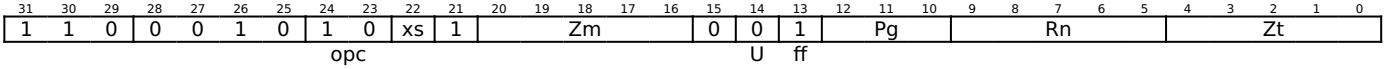
This instruction performs a gather load with first-faulting behavior of signed words to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 4. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 4 classes: 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

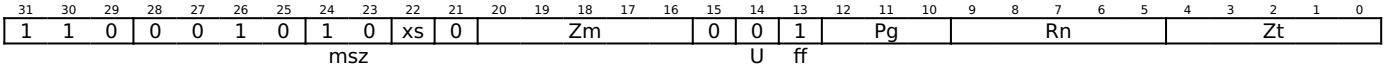
```
LDFF1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod> #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 2;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

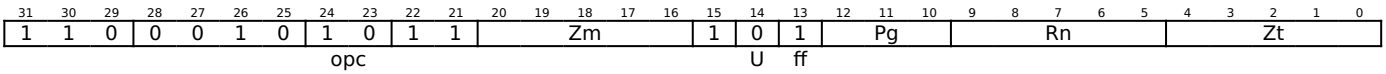
```
LDFF1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

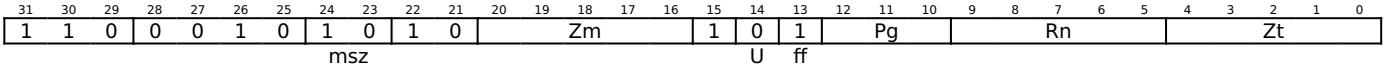
```
LDFF1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 64;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 2;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LDFF1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 64;
let unsigned : boolean = FALSE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1SW (vector plus immediate)

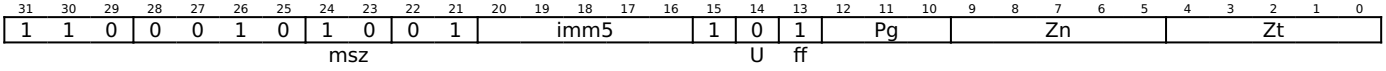
Gather load first-fault signed words to vector (immediate index)

This instruction performs a gather load with first-faulting behavior of signed words to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is a multiple of 4 in the range 0 to 124. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
LDFF1SW { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = FALSE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 4 in the range 0 to 124, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1W (scalar plus scalar)

Contiguous load first-fault unsigned words to vector (scalar index)

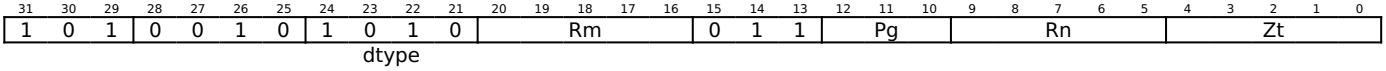
This instruction performs a contiguous load with first-faulting behavior of unsigned words to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 4 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

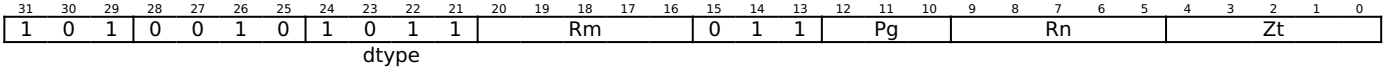
```
LDFF1W { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #2}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
```

64-bit element

(FEAT_SVE)



Encoding

```
LDFF1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #2}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;
Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1W (scalar plus vector)

Gather load first-fault unsigned words to vector (vector index)

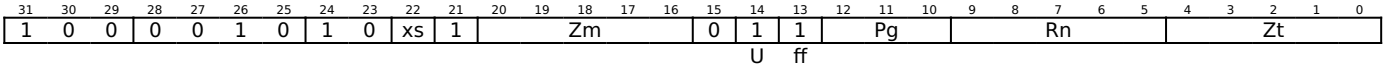
This instruction performs a gather load with first-faulting behavior of unsigned words to Active elements of a vector register from memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 4. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 6 classes: 32-bit scaled offset , 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 32-bit unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

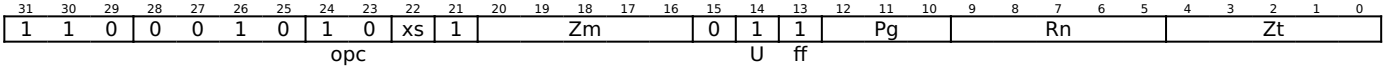
```
LDFF1W { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod> #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 2;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

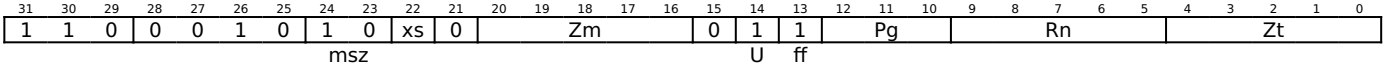
```
LDFF1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod> #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 2;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

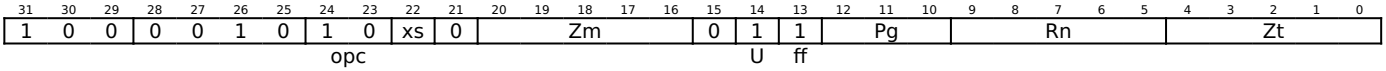
```
LDFF1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

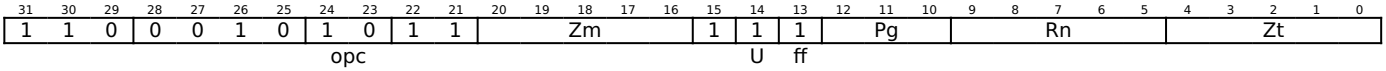
```
LDFF1W { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

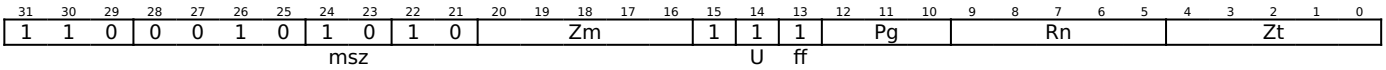
```
LDFF1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 2;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
LDFF1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 64;
let unsigned : boolean = TRUE;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Zm> Is the name of the offset scalable vector register, encoded in the "Zm" field.

<mod> Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDFF1W (vector plus immediate)

Gather load first-fault unsigned words to vector (immediate index)

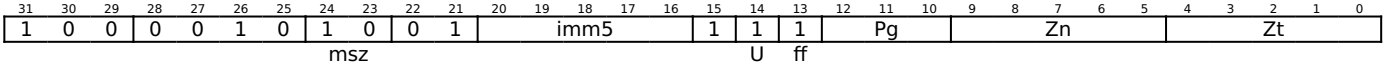
This instruction performs a gather load with first-faulting behavior of unsigned words to Active elements of a vector register from memory addresses generated by a vector base plus immediate index. The index is a multiple of 4 in the range 0 to 124. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

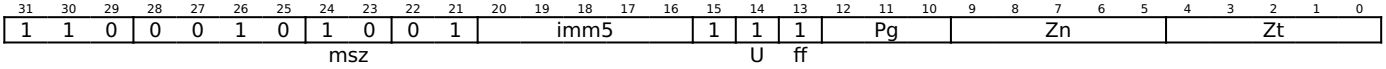
```
LDFF1W { <Zt>.S }, <Pg>/Z, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
LDFF1W { <Zt>.D }, <Pg>/Z, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 4 in the range 0 to 124, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = FALSE;
let tagchecked : boolean = TRUE;
var accdesc : AccessDescriptor = CreateAccDescSVEFF(contiguous, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

assert accdesc.first;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        if accdesc.first then
            // Mem[] will not return if a fault is detected for the First active element
            data = Mem{msize}(addr, accdesc);
            accdesc.first = FALSE;
        else
            // MemNF[] will return fault=TRUE if access is not performed for any reason
            (data, fault) = MemNF{msize}(addr, accdesc);
            faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
        end;
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNF1B

Contiguous load non-fault unsigned bytes to vector (immediate index)

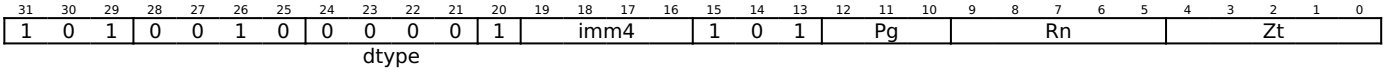
This instruction performs a contiguous load with non-faulting behavior of unsigned bytes to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 4 classes: 8-bit element , 16-bit element , 32-bit element and 64-bit element

8-bit element

(FEAT_SVE)



Encoding

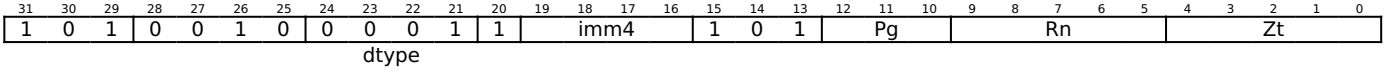
```
LDNF1B { <Zt>.B }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

16-bit element

(FEAT_SVE)



Encoding

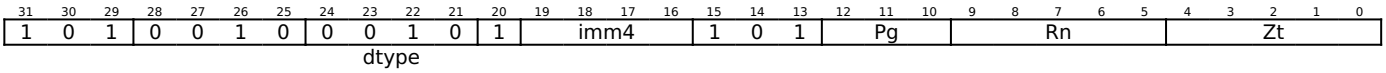
```
LDNF1B { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

32-bit element

(FEAT_SVE)



Encoding

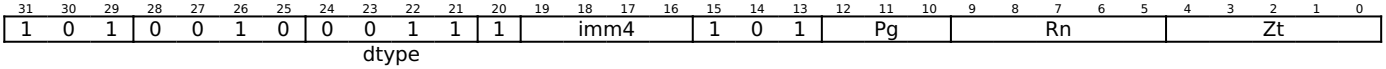
```
LDNF1B { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

64-bit element

(FEAT_SVE)



Encoding

```
LDNF1B { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVENF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        // MemNF[] will return fault=TRUE if access is not performed for any reason
        (data, fault) = MemNF{msize}(addr, accdesc);
        faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNF1D

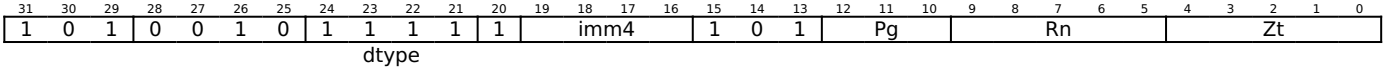
Contiguous load non-fault doublewords to vector (immediate index)

This instruction performs a contiguous load with non-faulting behavior of doublewords to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
LDNF1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVENF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        // MemNF[] will return fault=TRUE if access is not performed for any reason
        (data, fault) = MemNF{msize}(addr, accdesc);
        faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNF1H

Contiguous load non-fault unsigned halfwords to vector (immediate index)

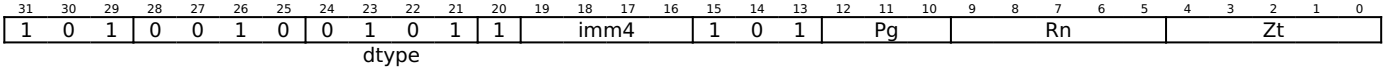
This instruction performs a contiguous load with non-faulting behavior of unsigned halfwords to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE)



Encoding

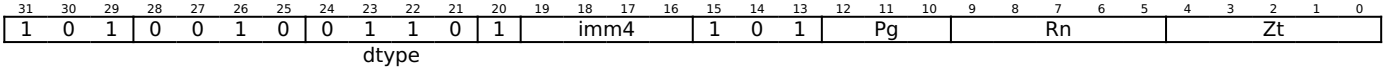
```
LDNF1H { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

32-bit element

(FEAT_SVE)



Encoding

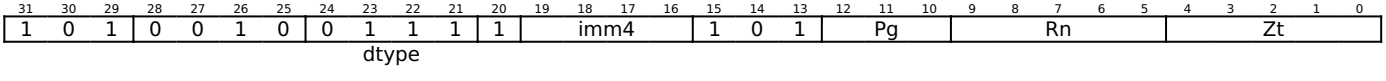
```
LDNF1H { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

64-bit element

(FEAT_SVE)



Encoding

```
LDNF1H { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVENF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        // MemNF[] will return fault=TRUE if access is not performed for any reason
        (data, fault) = MemNF{msize}(addr, accdesc);
        faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNF1SB

Contiguous load non-fault signed bytes to vector (immediate index)

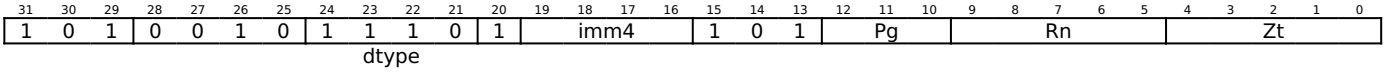
This instruction performs a contiguous load with non-faulting behavior of signed bytes to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 16-bit element , 32-bit element and 64-bit element

16-bit element

(FEAT_SVE)



Encoding

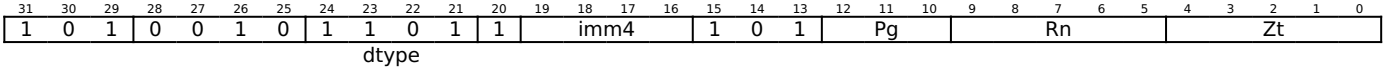
```
LDNF1SB { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

32-bit element

(FEAT_SVE)



Encoding

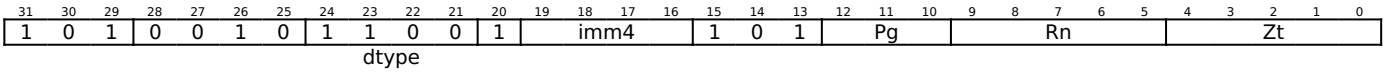
```
LDNF1SB { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

64-bit element

(FEAT_SVE)



Encoding

```
LDNF1SB { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVENF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        // MemNF[] will return fault=TRUE if access is not performed for any reason
        (data, fault) = MemNF{msize}(addr, accdesc);
        faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNF1SH

Contiguous load non-fault signed halfwords to vector (immediate index)

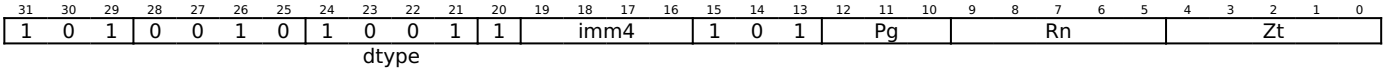
This instruction performs a contiguous load with non-faulting behavior of signed halfwords to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

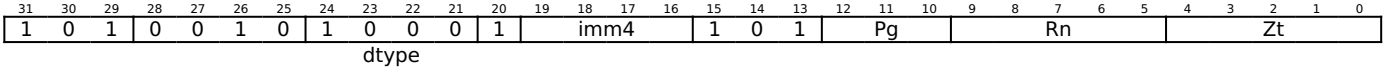
```
LDNF1SH { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

64-bit element

(FEAT_SVE)



Encoding

```
LDNF1SH { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVENF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        // MemNF[] will return fault=TRUE if access is not performed for any reason
        (data, fault) = MemNF{msize}(addr, accdesc);
        faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNF1SW

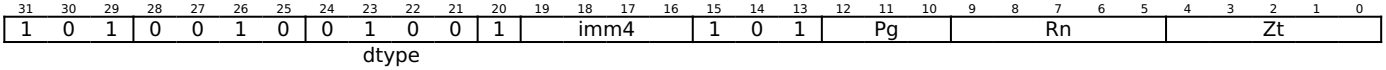
Contiguous load non-fault signed words to vector (immediate index)

This instruction performs a contiguous load with non-faulting behavior of signed words to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
LDNF1SW { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = FALSE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVENF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        // MemNF[] will return fault=TRUE if access is not performed for any reason
        (data, fault) = MemNF{msize}(addr, accdesc);
        faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNF1W

Contiguous load non-fault unsigned words to vector (immediate index)

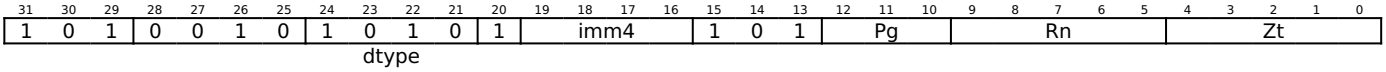
This instruction performs a contiguous load with non-faulting behavior of unsigned words to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

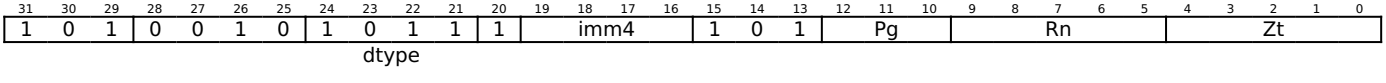
```
LDNF1W { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

64-bit element

(FEAT_SVE)



Encoding

```
LDNF1W { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let orig : bits(VL) = Z{}(t);
var data : bits(msize);
let mbytes : integer{} = msize DIV 8;
var fault : boolean = FALSE;
var faulted : boolean = FALSE;
var unknown : boolean = FALSE;
let contiguous : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVENF(contiguous, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        // MemNF[] will return fault=TRUE if access is not performed for any reason
        (data, fault) = MemNF{msize}(addr, accdesc);
        faulted = faulted || ConstrainUnpredictableBool(Unpredictable_NONFAULT);
    else
        (data, fault) = (Zeros{msize}, FALSE);
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);

    // FFR elements set to FALSE following a suppressed access/fault
    faulted = faulted || fault;
    if faulted then
        ElemFFR(e, esize) = '0';
    end;

    // Value becomes CONSTRAINED UNPREDICTABLE after an FFR element is FALSE
    unknown = unknown || ElemFFR(e, esize) == '0';
    if unknown then
        if !fault && ConstrainUnpredictableBool(Unpredictable_SVELDNFDATA) then
            result[e*:esize] = Extend{esize}(data, unsigned);
        elsif ConstrainUnpredictableBool(Unpredictable_SVELDNFZERO) then
            result[e*:esize] = Zeros{esize};
        else  // merge
            result[e*:esize] = orig[e*:esize];
        end;
    else
        result[e*:esize] = Extend{esize}(data, unsigned);
    end;
end;

Z{VL}(t) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1B (scalar plus immediate, consecutive registers)

Contiguous load non-temporal of bytes to multiple consecutive vectors (immediate index)

This instruction performs a contiguous non-temporal load of bytes to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

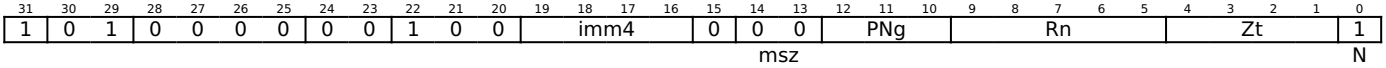
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

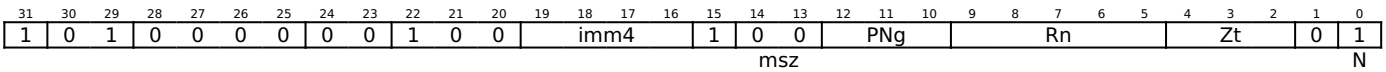
```
LDNT1B { <Zt1>.B-<Zt2>.B }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LDNT1B { <Zt1>.B-<Zt4>.B }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{}(g);
let mask : bits(PL * nreg) = CounterToPredicate{}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1B (scalar plus immediate, single register)

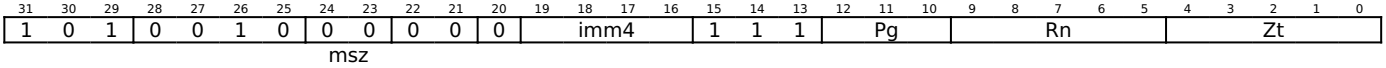
Contiguous load non-temporal bytes to vector (immediate index)

This instruction performs a contiguous non-temporal load of bytes to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LDNT1B { <Zt>.B }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1B (scalar plus scalar, consecutive registers)

Contiguous load non-temporal of bytes to multiple consecutive vectors (scalar index)

This instruction performs a contiguous non-temporal load of bytes to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

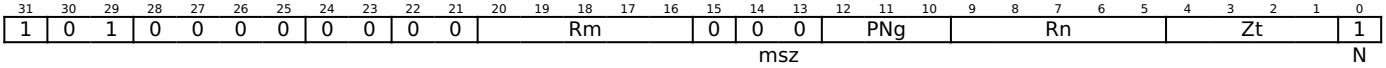
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

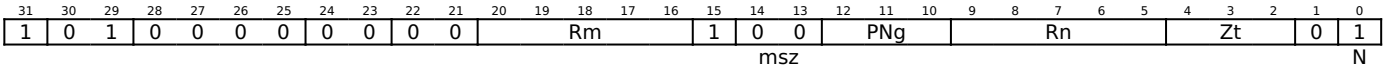
```
LDNT1B { <Zt1>.B-<Zt2>.B }, <PNg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;

let t : integer = UInt(Zt::'0');

let esize : integer{} = 8;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LDNT1B { <Zt1>.B-<Zt4>.B }, <PNg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;

let t : integer = UInt(Zt::'00');

let esize : integer{} = 8;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;

var offset : bits(64);
var base : bits(64);
var addr : bits(64);

let pred : bits(PL) = P{}(g);
let mask : bits(PL * nreg) = CounterToPredicate{}(pred[15:0]);

var values : array [[4]] of bits(VL);

let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 &&
       ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;

        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

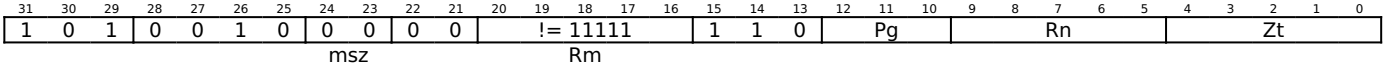
LDNT1B (scalar plus scalar, single register)

Contiguous load non-temporal bytes to vector (scalar index)

This instruction performs a contiguous non-temporal load of bytes to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.
A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LDNT1B { <Zt>.B }, <Pg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var offset : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1B (vector plus scalar)

Gather load non-temporal unsigned bytes

This instruction performs a non-temporal gather load of unsigned bytes to Active elements of a vector register from memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

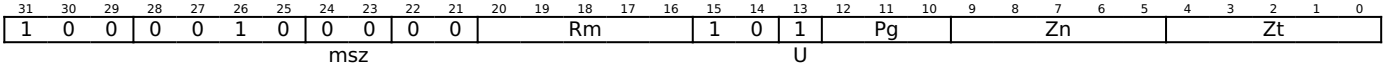
A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit unscaled offset and 64-bit unscaled offset

32-bit unscaled offset

(FEAT_SVE2)



Encoding

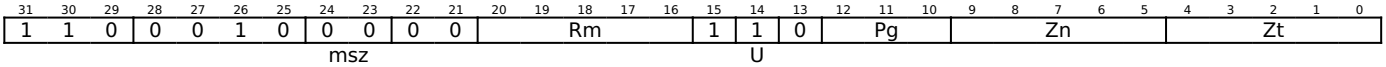
```
LDNT1B { <Zt>.S }, <Pg>/Z, [<Zn>.S{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
```

64-bit unscaled offset

(FEAT_SVE2)



Encoding

```
LDNT1B { <Zt>.D }, <Pg>/Z, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1D (scalar plus immediate, consecutive registers)

Contiguous load non-temporal of doublewords to multiple consecutive vectors (immediate index)

This instruction performs a contiguous non-temporal load of doublewords to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

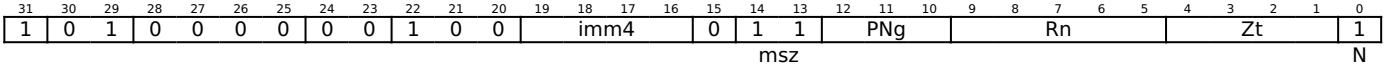
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

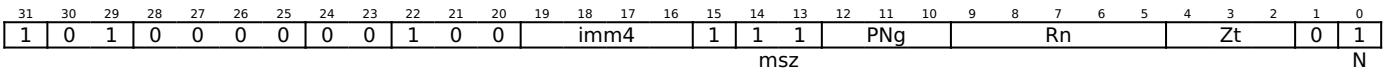
```
LDNT1D { <Zt1>.D-<Zt2>.D }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LDNT1D { <Zt1>.D-<Zt4>.D }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{}(g);
let mask : bits(PL * nreg) = CounterToPredicate{}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

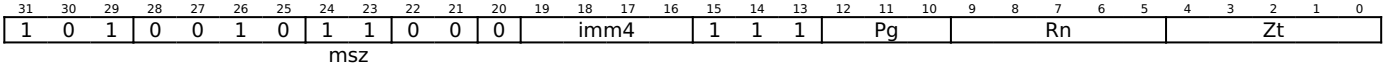
LDNT1D (scalar plus immediate, single register)

Contiguous load non-temporal doublewords to vector (immediate index)

This instruction performs a contiguous non-temporal load of doublewords to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.
A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LDNT1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1D (scalar plus scalar, consecutive registers)

Contiguous load non-temporal of doublewords to multiple consecutive vectors (scalar index)

This instruction performs a contiguous non-temporal load of doublewords to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

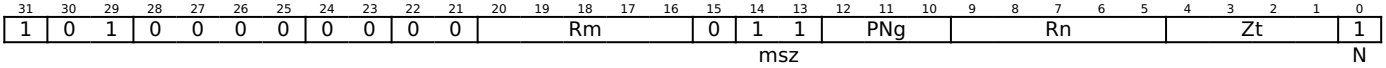
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

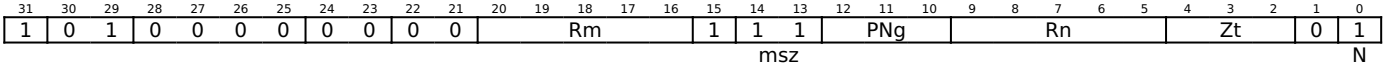
```
LDNT1D { <Zt1>.D-<Zt2>.D }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;

let t : integer = UInt(Zt::'0');

let esize : integer{} = 64;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LDNT1D { <Zt1>.D-<Zt4>.D }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;

let t : integer = UInt(Zt::'00');

let esize : integer{} = 64;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;

var offset : bits(64);
var base : bits(64);
var addr : bits(64);

let pred : bits(PL) = P{}(g);
let mask : bits(PL * nreg) = CounterToPredicate{}(pred[15:0]);

var values : array [[4]] of bits(VL);

let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 &&
       ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;

        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1D (scalar plus scalar, single register)

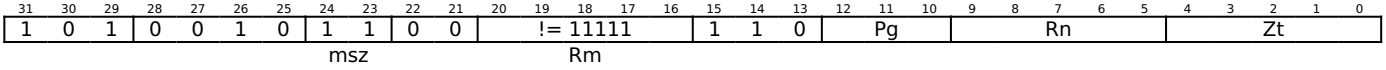
Contiguous load non-temporal doublewords to vector (scalar index)

This instruction performs a contiguous non-temporal load of doublewords to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 8 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LDNT1D { <Zt>.D }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var offset : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1D (vector plus scalar)

Gather load non-temporal unsigned doublewords

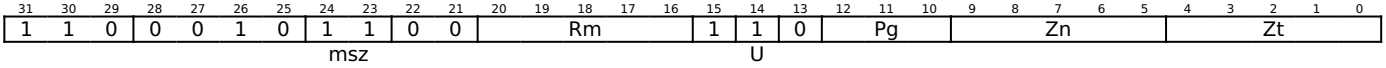
This instruction performs a non-temporal gather load of doublewords to Active elements of a vector register from memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2)



Encoding

```
LDNT1D { <Zt>.D }, <Pg>/Z, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1H (scalar plus immediate, consecutive registers)

Contiguous load non-temporal of halfwords to multiple consecutive vectors (immediate index)

This instruction performs a contiguous non-temporal load of halfwords to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

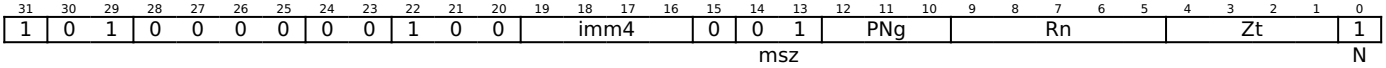
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

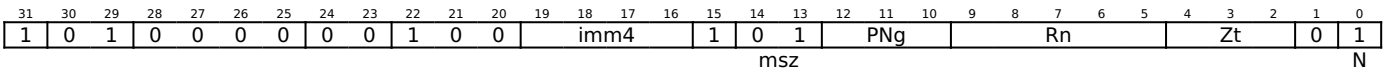
```
LDNT1H { <Zt1>.H-<Zt2>.H }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LDNT1H { <Zt1>.H-<Zt4>.H }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{}(g);
let mask : bits(PL * nreg) = CounterToPredicate{}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

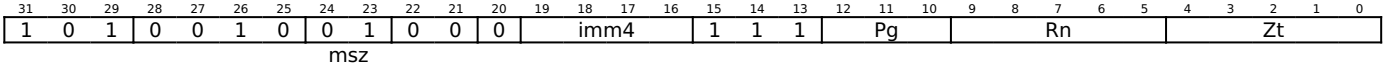
LDNT1H (scalar plus immediate, single register)

Contiguous load non-temporal halfwords to vector (immediate index)

This instruction performs a contiguous non-temporal load of halfwords to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.
A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LDNT1H { <Zt>.H }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1H (scalar plus scalar, consecutive registers)

Contiguous load non-temporal of halfwords to multiple consecutive vectors (scalar index)

This instruction performs a contiguous non-temporal load of halfwords to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

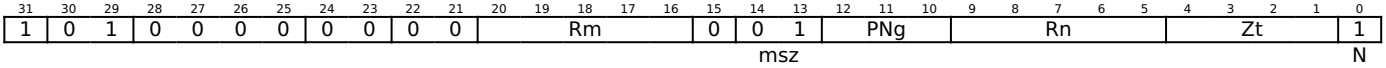
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

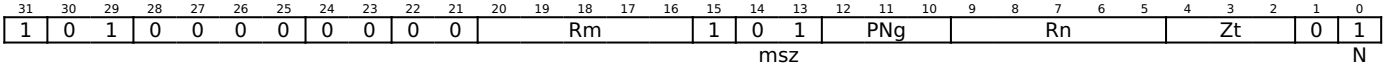
```
LDNT1H { <Zt1>.H-<Zt2>.H }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;

let t : integer = UInt(Zt::'0');

let esize : integer{} = 16;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LDNT1H { <Zt1>.H-<Zt4>.H }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;

let t : integer = UInt(Zt::'00');

let esize : integer{} = 16;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;

var offset : bits(64);
var base : bits(64);
var addr : bits(64);

let pred : bits(PL) = P{}(g);
let mask : bits(PL * nreg) = CounterToPredicate{}(pred[15:0]);

var values : array [[4]] of bits(VL);

let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 &&
       ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;

        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1H (scalar plus scalar, single register)

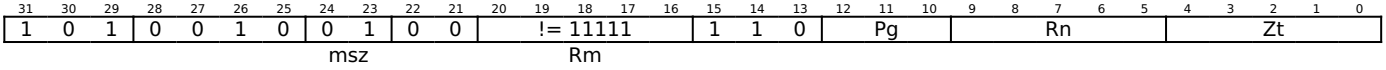
Contiguous load non-temporal halfwords to vector (scalar index)

This instruction performs a contiguous non-temporal load of halfwords to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 2 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LDNT1H { <Zt>.H }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var offset : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1H (vector plus scalar)

Gather load non-temporal unsigned halfwords

This instruction performs a non-temporal gather load of unsigned halfwords to Active elements of a vector register from memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

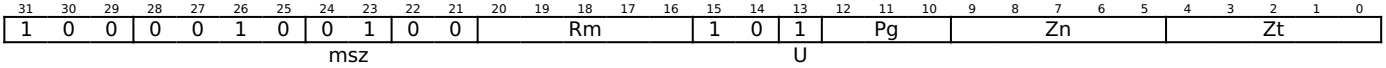
A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit unscaled offset and 64-bit unscaled offset

32-bit unscaled offset

(FEAT_SVE2)



Encoding

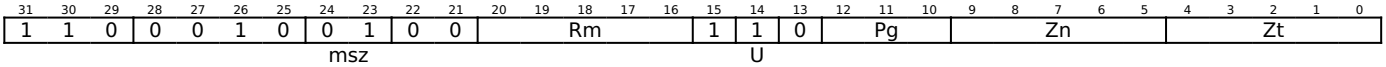
```
LDNT1H { <Zt>.S }, <Pg>/Z, [<Zn>.S{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
```

64-bit unscaled offset

(FEAT_SVE2)



Encoding

```
LDNT1H { <Zt>.D }, <Pg>/Z, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1SB

Gather load non-temporal signed bytes

This instruction performs a non-temporal gather load of signed bytes to Active elements of a vector register from memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

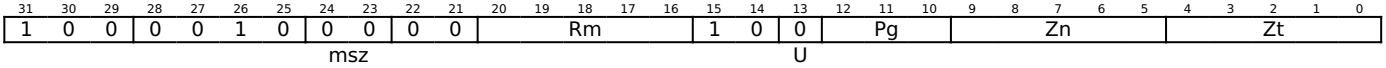
A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit unscaled offset and 64-bit unscaled offset

32-bit unscaled offset

(FEAT_SVE2)



Encoding

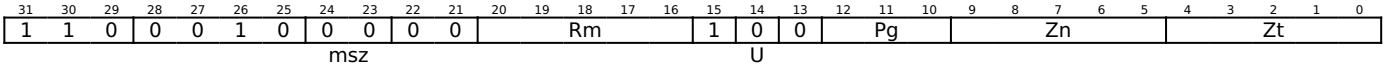
```
LDNT1SB { <Zt>.S }, <Pg>/Z, [<Zn>.S{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
```

64-bit unscaled offset

(FEAT_SVE2)



Encoding

```
LDNT1SB { <Zt>.D }, <Pg>/Z, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1SH

Gather load non-temporal signed halfwords

This instruction performs a non-temporal gather load of signed halfwords to Active elements of a vector register from memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

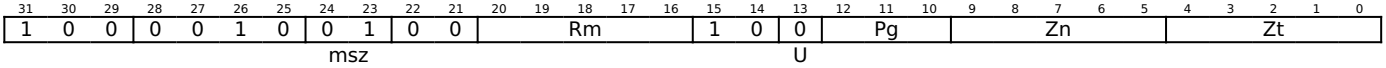
A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit unscaled offset and 64-bit unscaled offset

32-bit unscaled offset

(FEAT_SVE2)



Encoding

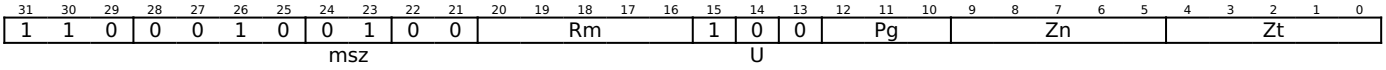
```
LDNT1SH { <Zt>.S }, <Pg>/Z, [<Zn>.S{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
```

64-bit unscaled offset

(FEAT_SVE2)



Encoding

```
LDNT1SH { <Zt>.D }, <Pg>/Z, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1SW

Gather load non-temporal signed words

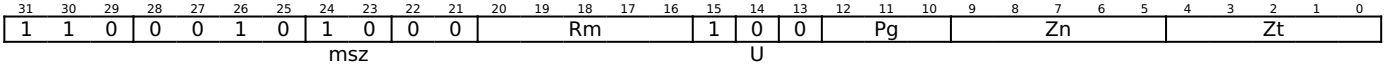
This instruction performs a non-temporal gather load of signed words to Active elements of a vector register from memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2)



Encoding

```
LDNT1SW { <Zt>.D }, <Pg>/Z, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1W (scalar plus immediate, consecutive registers)

Contiguous load non-temporal of words to multiple consecutive vectors (immediate index)

This instruction performs a contiguous non-temporal load of words to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

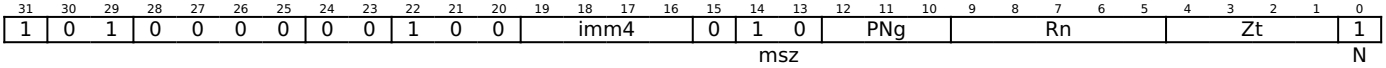
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

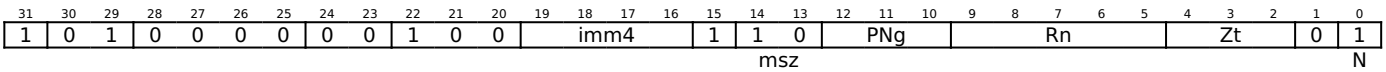
```
LDNT1W { <Zt1>.S-<Zt2>.S }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LDNT1W { <Zt1>.S-<Zt4>.S }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{}(g);
let mask : bits(PL * nreg) = CounterToPredicate{}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1W (scalar plus immediate, single register)

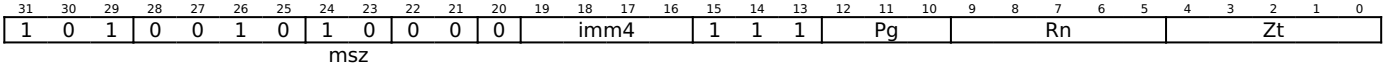
Contiguous load non-temporal words to vector (immediate index)

This instruction performs a contiguous non-temporal load of words to elements of a vector register from the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LDNT1W { <Zt>.S }, <Pg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1W (scalar plus scalar, consecutive registers)

Contiguous load non-temporal of words to multiple consecutive vectors (scalar index)

This instruction performs a contiguous non-temporal load of words to elements of two or four consecutive vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

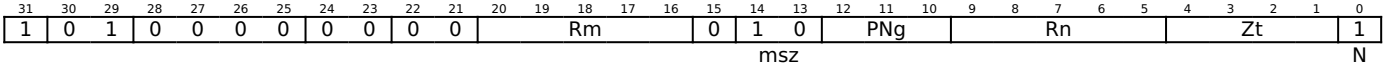
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

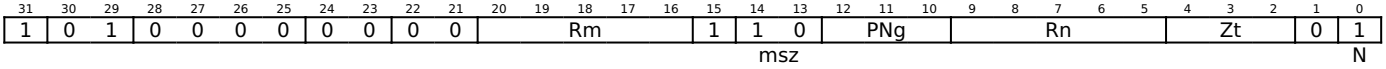
```
LDNT1W { <Zt1>.S-<Zt2>.S }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;

let t : integer = UInt(Zt::'0');

let esize : integer{} = 32;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
LDNT1W { <Zt1>.S-<Zt4>.S }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;

let t : integer = UInt(Zt::'00');

let esize : integer{} = 32;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;

var offset : bits(64);
var base : bits(64);
var addr : bits(64);

let pred : bits(PL) = P{}(g);
let mask : bits(PL * nreg) = CounterToPredicate{}(pred[15:0]);

var values : array [[4]] of bits(VL);

let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 &&
       ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;

        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer+r) = values[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1W (scalar plus scalar, single register)

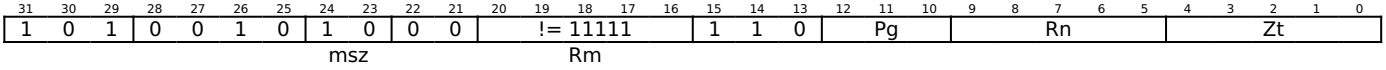
Contiguous load non-temporal words to vector (scalar index)

This instruction performs a contiguous non-temporal load of words to elements of a vector register from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 4 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LDNT1W { <Zt>.S }, <Pg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var offset : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1W (vector plus scalar)

Gather load non-temporal unsigned words

This instruction performs a non-temporal gather load of unsigned words to Active elements of a vector register from memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements will not cause a read from Device memory or signal faults, and are set to zero in the destination vector.

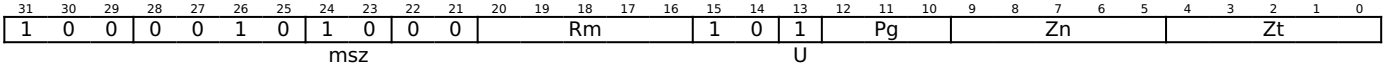
A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit unscaled offset and 64-bit unscaled offset

32-bit unscaled offset

(FEAT_SVE2)



Encoding

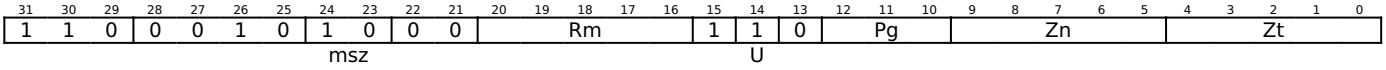
```
LDNT1W { <Zt>.S }, <Pg>/Z, [<Zn>.S{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
```

64-bit unscaled offset

(FEAT_SVE2)



Encoding

```
LDNT1W { <Zt>.D }, <Pg>/Z, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var result : bits(VL);
var data : bits(msize);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        data = Mem{msize}(addr, accdesc);
        result[e*:esize] = Extend{esize}(data, unsigned);
    else
        result[e*:esize] = Zeros{esize};
    end;
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDR (predicate)

Load predicate register

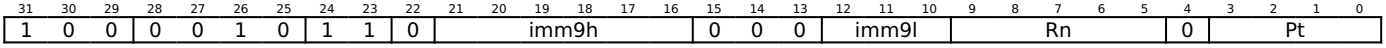
This instruction loads a predicate register from a memory address generated by a 64-bit scalar base, plus an immediate offset in the range -256 to 255 that is multiplied by the current predicate register size in bytes. This instruction is unpredicated.

The load is performed as contiguous byte accesses, each containing 8 consecutive predicate bits in ascending element order, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then a general-purpose base register must be aligned to 2 bytes.

For programmer convenience, an assembler must also accept a predicate-as-counter register name for the destination predicate register.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LDR <Pt>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Pt);
let n : integer = UInt(Rn);
let imm : integer = SInt(imm9h::imm9l);
```

Assembler Symbols

<Pt>	Is the name of the destination scalable predicate register, encoded in the "Pt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -256 to 255, defaulting to 0, encoded in the "imm9h:imm9l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = PL DIV 8;
var base : bits(64);
let offset : integer = imm * elements;
var result : bits(PL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

var addr : bits(64) = AddressAdd(base, offset, accdesc);

let aligned : boolean = IsAlignedSize(base, 2);

if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;

for e = 0 to elements-1 do
    result[e*:8] = AArch64_MemSingle{8}(addr, accdesc, aligned);
    addr = AddressIncrement(addr, 1, accdesc);
end;

P{PL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDR (vector)

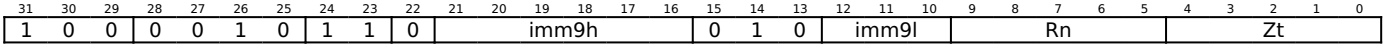
Load vector register

This instruction loads a vector register from a memory address generated by a 64-bit scalar base, plus an immediate offset in the range -256 to 255 that is multiplied by the current vector register size in bytes. This instruction is unpredicated.

The load is performed as contiguous byte accesses, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then the base register must be aligned to 16 bytes.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LDR <Zt>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let imm : integer = SInt(imm9h::imm9l);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -256 to 255, defaulting to 0, encoded in the "imm9h:imm9l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 8;
var base : bits(64);
let offset : integer = imm * elements;
var result : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  predicated, tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

var addr : bits(64) = AddressAdd(base, offset, accdesc);

let aligned : boolean = IsAlignedSize(base, 16);

if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;

for e = 0 to elements-1 do
    result[e*:8] = AArch64_MemSingle{8}(addr, accdesc, aligned);
    addr = AddressIncrement(addr, 1, accdesc);
end;

Z{VL}(t) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

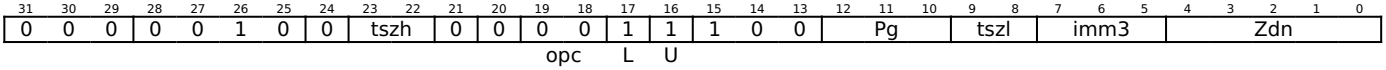
LSL (immediate, predicated)

Logical shift left by immediate (predicated)

This instruction shifts left by immediate each Active element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate shift amount is an unsigned value in the range 0 to number of bits per element minus 1. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let shift : integer = UInt(tsize::imm3) - esize;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the immediate shift amount, in the range 0 to number of bits per element minus 1, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let PL : integer{} = VL DIV 8;
let operand1 : bits(VL) = Z{}(dn);
let mask : bits(PL) = P{}(g);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = LSL(element1, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

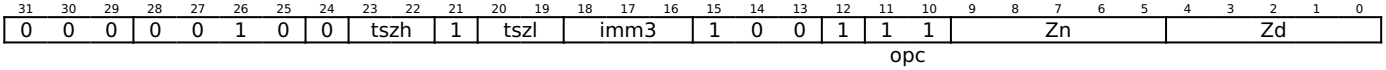
LSL (immediate, unpredicated)

Logical shift left by immediate (unpredicated)

This instruction shifts left by immediate each element of the source vector, and places the results in the corresponding elements of the destination vector. The immediate shift amount is an unsigned value in the range 0 to number of bits per element minus 1. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSL <Zd>.<T>, <Zn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = UInt(tsize::imm3) - esize;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<const> Is the immediate shift amount, in the range 0 to number of bits per element minus 1, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    result[e*:esize] = LSL(element1, shift);

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

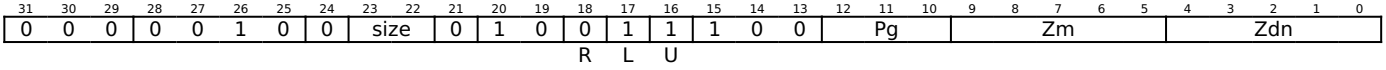
LSL (vectors)

Logical shift left by vector (predicated)

This instruction shifts left Active elements of the first source vector by the corresponding elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. The shift amount operand is a vector of unsigned elements in which all bits are significant, and not used modulo the element size. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        let shift : integer = Min(UInt(element2), esize);
        result[e*:esize] = LSL(element1, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

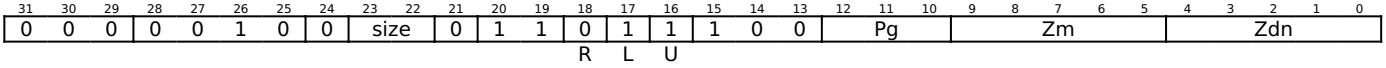
LSL (wide elements, predicated)

Logical shift left by 64-bit wide elements (predicated)

This instruction shifts left Active elements of the first source vector by the corresponding overlapping 64-bit elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. The shift amount is a vector of unsigned 64-bit doubleword elements in which all bits are significant, and not used modulo the destination element size. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(64) = operand2[((e * esize) DIVRM 64)*:64];
        let shift : integer = Min(UInt(element2), esize);
        result[e*:esize] = LSL(element1, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

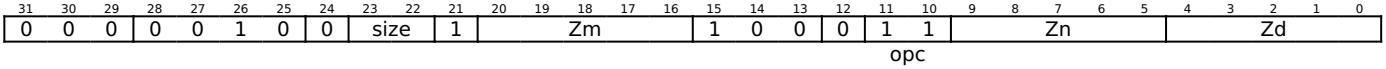
LSL (wide elements, unpredicated)

Logical shift left by 64-bit wide elements (unpredicated)

This instruction shifts left all elements of the first source vector by the corresponding overlapping 64-bit elements of the second source vector and places the first in the corresponding elements of the destination vector. The shift amount is a vector of unsigned 64-bit doubleword elements in which all bits are significant, and not used modulo the destination element size. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSL <Zd>.<T>, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(64) = operand2[((e * esize) DIVRM 64)*:64];
    let shift : integer = Min(UInt(element2), esize);
    result[e*:esize] = LSL(element1, shift);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

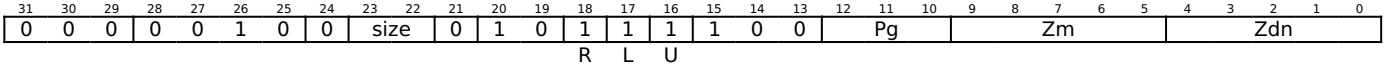
LSLR

Reversed logical shift left by vector (predicated)

This instruction reverse shifts left Active elements of the second source vector by the corresponding elements of the first source vector, and destructively places the results in the corresponding elements of the first source vector. The shift amount operand is a vector of unsigned elements in which all bits are significant, and not used modulo the element size. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSLR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        let shift : integer = Min(UInt(element1), esize);
        result[e*:esize] = LSL(element2, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

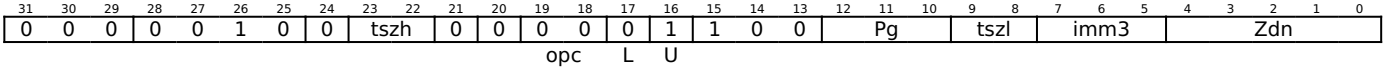
LSR (immediate, predicated)

Logical shift right by immediate (predicated)

This instruction shifts right by immediate, inserting zeros, each Active element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let PL : integer{} = VL DIV 8;
let operand1 : bits(VL) = Z{}(dn);
let mask : bits(PL) = P{}(g);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = LSR(element1, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

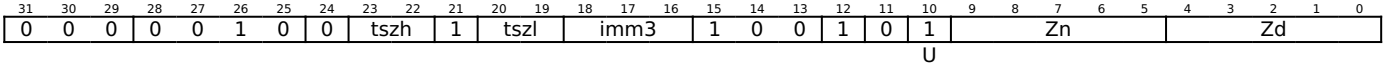
LSR (immediate, unpredicated)

Logical shift right by immediate (unpredicated)

This instruction shifts right by immediate, inserting zeros, each element of the source vector, and places the results in the corresponding elements of the destination vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSR <Zd>.<T>, <Zn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    result[e*:esize] = LSR(element1, shift);

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

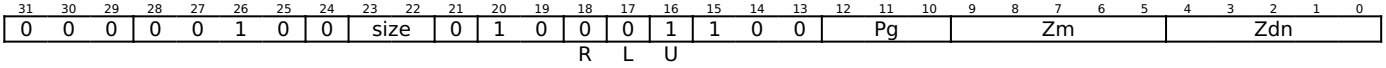
LSR (vectors)

Logical shift right by vector (predicated)

This instruction shifts right, inserting zeros, Active elements of the first source vector by the corresponding elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. The shift amount operand is a vector of unsigned elements in which all bits are significant, and not used modulo the element size. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        let shift : integer = Min(UInt(element2), esize);
        result[e*:esize] = LSR(element1, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

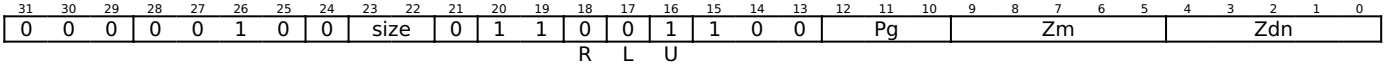
LSR (wide elements, predicated)

Logical shift right by 64-bit wide elements (predicated)

This instruction shifts right, inserting zeros, Active elements of the first source vector by the corresponding overlapping 64-bit elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. The shift amount is a vector of unsigned 64-bit doubleword elements in which all bits are significant, and not used modulo the destination element size. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(64) = operand2[((e * esize) DIVRM 64)*:64];
        let shift : integer = Min(UInt(element2), esize);
        result[e*:esize] = LSR(element1, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

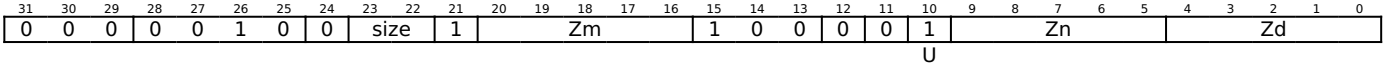
LSR (wide elements, unpredicated)

Logical shift right by 64-bit wide elements (unpredicated)

This instruction shifts right, inserting zeros, all elements of the first source vector by the corresponding overlapping 64-bit elements of the second source vector and places the first in the corresponding elements of the destination vector. The shift amount is a vector of unsigned 64-bit doubleword elements in which all bits are significant, and not used modulo the destination element size. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSR <Zd>.<T>, <Zn>.<T>, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(64) = operand2[((e * esize) DIVRM 64)*:64];
    let shift : integer = Min(UInt(element2), esize);
    result[e*:esize] = LSR(element1, shift);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

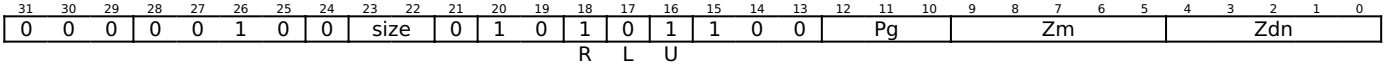
LSRR

Reversed logical shift right by vector (predicated)

This instruction reverse shifts right, inserting zeros, Active elements of the second source vector by the corresponding elements of the first source vector, and destructively places the results in the corresponding elements of the first source vector. The shift amount operand is a vector of unsigned elements in which all bits are significant, and not used modulo the element size. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
LSRR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        let shift : integer = Min(UInt(element1), esize);
        result[e*:esize] = LSR(element2, shift);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI2 (8-bit and 16-bit)

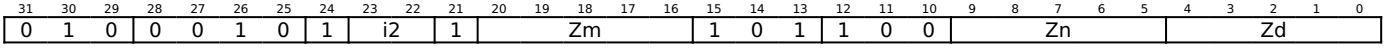
Lookup table read with 2-bit indices (8-bit and 16-bit)

This instruction copies indexed 8-bit or 16-bit elements from the low 128 bits of the table vector to the destination vector using packed 2-bit indices from a segment of the source vector. A segment corresponds to a portion of the source vector that is consumed in order to fill the destination vector. The segment is selected by the vector segment index. This instruction is unpredicated.

It has encodings from 2 classes: Byte and Halfword

Byte

((FEAT_SVE2 || FEAT_SME2) && FEAT_LUT)



Encoding

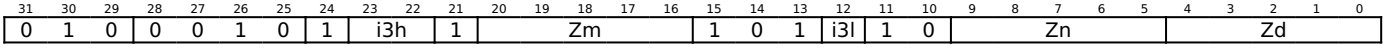
```
LUTI2 <Zd>.B, { <Zn>.B }, <Zm>[<index>]
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_LUT)) then EndOfDecode(Decode_UNDEF);
end;
let isize : integer{} = 2;
let esize : integer{} = 8;
let m : integer = UInt(Zm);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let part : integer = UInt(i2);
```

Halfword

((FEAT_SVE2 || FEAT_SME2) && FEAT_LUT)



Encoding

```
LUTI2 <Zd>.H, { <Zn>.H }, <Zm>[<index>]
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_LUT)) then EndOfDecode(Decode_UNDEF);
end;
let isize : integer{} = 2;
let esize : integer{} = 16;
let m : integer = UInt(Zm);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let part : integer = UInt(i3h::i3l);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the table vector register, encoded in the "Zn" field.
<Zm>	Is the name of the source scalable vector register, encoded in the "Zm" field.
<index>	For the "Byte" variant: is the vector segment index, in the range 0 to 3, encoded in the "i2" field. For the "Halfword" variant: is the vector segment index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let ibase : integer = elements * part;
let indexes : bits(VL) = Z{}(m);
let table : bits(VL)   = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let index : integer = UInt(indexes[(ibase + e)*:isize]);
    result[e*:esize] = table[index*:esize];

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI4 (8-bit and 16-bit)

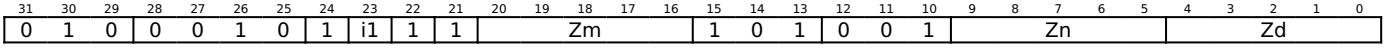
Lookup table read with 4-bit indices (8-bit and 16-bit)

This instruction copies indexed 8-bit or 16-bit elements from the low 128 or 256 bits of the table vector, or from the low 128 bits of the two table vectors to the destination vector using packed 4-bit indices from a segment of the source vector. A segment corresponds to a portion of the source vector that is consumed in order to fill the destination vector. The segment is selected by the vector segment index. This instruction is unpredicated.

It has encodings from 3 classes: Byte, single register table , Halfword, two register table and Halfword, single register table

Byte, single register table

((FEAT_SVE2 || FEAT_SME2) && FEAT_LUT)



Encoding

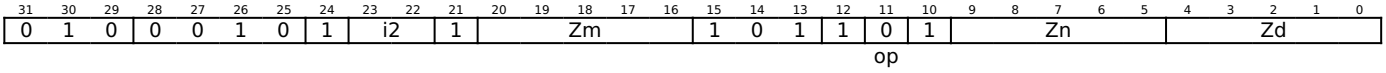
```
LUTI4 <Zd>.B, { <Zn>.B }, <Zm>[<index>]
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_LUT)) then EndOfDecode(Decode_UNDEF);
end;
let isize : integer{} = 4;
let esize : integer{} = 8;
let ntblr : integer = 1;
let m : integer = UInt(Zm);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let part : integer = UInt(i1);
```

Halfword, two register table

((FEAT_SVE2 || FEAT_SME2) && FEAT_LUT)



Encoding

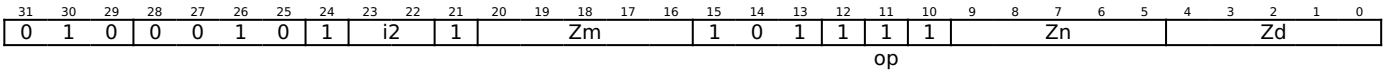
```
LUTI4 <Zd>.H, { <Zn1>.H, <Zn2>.H }, <Zm>[<index>]
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_LUT)) then EndOfDecode(Decode_UNDEF);
end;
let isize : integer{} = 4;
let esize : integer{} = 16;
let ntblr : integer = 2;
let m : integer = UInt(Zm);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let part : integer = UInt(i2);
```

Halfword, single register table

((FEAT_SVE2 || FEAT_SME2) && FEAT_LUT)



Encoding

```
LUTI4 <Zd>.H, { <Zn>.H }, <Zm>[<index>]
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME2)) ||
     !IsFeatureImplemented(FEAT_LUT)) then EndOfDecode(Decode_UNDEF);
end;
if MaxImplementedAnyVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let isize : integer{} = 4;
let esize : integer{} = 16;
let ntblr : integer = 1;
let m : integer = UInt(Zm);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let part : integer = UInt(i2);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the table vector register, encoded in the "Zn" field.
<Zm>	Is the name of the source scalable vector register, encoded in the "Zm" field.
<index>	For the "Byte, single register table" variant: is the vector segment index, in the range 0 to 1, encoded in the "i1" field. For the "Halfword, single register table" and "Halfword, two register table" variants: is the vector segment index, in the range 0 to 3, encoded in the "i2" field.
<Zn1>	Is the name of the first table vector register, encoded as "Zn".
<Zn2>	Is the name of the second table vector register, encoded as "Zn" plus 1 modulo 32.

Operation

```
if IsFeatureImplemented(FEAT_SME2) then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
if ntblr == 1 && esize == 16 && VL < 256 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = VL DIV esize;
let tablesize : integer = if ntblr == 1 && esize == 16 then 256 else 128;
let eltspertable : integer = tablesize DIV esize;
let ibase : integer = elements * part;
let indexes : bits(VL) = Z{}(m);
let table1 : bits(VL)  = Z{}(n+0);
let table2 : bits(VL)  = if ntblr == 2 then Z{VL}((n+1) MOD 32) else Zeros{VL};
var result : bits(VL);
var res : bits(esize);

for e = 0 to elements-1 do
    let index : integer = UInt(indexes[(ibase + e)*:isize]);
    if index < eltspertable then
        res = table1[index*:esize];
    else
        assert ntblr == 2;
        res = table2[(index - eltspertable)*:esize];
    end;
    result[e*:esize] = res;

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI6 (16-bit)

Lookup table read with 6-bit indices (16-bit)

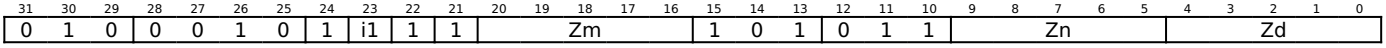
This instruction copies indexed 16-bit elements from the low 512 bits of the two table vectors to the destination vector using packed 6-bit indices from a segment in the bottom three quarters of the source vector register.

This instruction is unpredicated.

This instruction requires that the Effective SVE vector length is at least 512 bits.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
LUTI6 <Zd>.H, { <Zn1>.H, <Zn2>.H }, <Zm>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if MaxImplementedAnyVL() < 512 then EndOfDecode(Decode_UNDEF); end;
let isize : integer{} = 6;
let esize : integer{} = 16;
let m : integer = UInt(Zm);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let part : integer = UInt(i1);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first table vector register, encoded as "Zn".
<Zn2>	Is the name of the second table vector register, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the source scalable vector register, encoded in the "Zm" field.
<index>	Is the vector segment index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < 512 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = VL DIV esize;
let ibase : integer = elements * part;
let indexes : bits(VL) = Z{}(m); // Only bottom 3/4 used
let table : bits(1024)  = Z{VL}((n+1) MOD 32)[511:0] :: Z{VL}(n+0)[511:0];
var result : bits(VL);

for e = 0 to elements-1 do
    let index : integer = UInt(indexes[(ibase + e)*:isize]);
    let res : bits(esize) = table[index*:esize];
    result[e*:esize] = res;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI6 (8-bit)

Lookup table read with 6-bit indices (8-bit)

This instruction copies indexed 8-bit elements from the low 256 bits of the two table vectors to the destination vector using packed 6-bit indices from the bottom three quarters of the source vector.

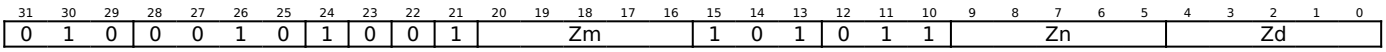
This instruction is unpredicated.

This instruction requires that the Effective SVE vector length is at least 256 bits.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2p3)



Encoding

```
LUTI6 <Zd>.B, { <Zn1>.B, <Zn2>.B }, <Zm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) then EndOfDecode(Decode_UNDEF);
end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let isize : integer{} = 6;
let esize : integer{} = 8;
let m : integer = UInt(Zm);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first table vector register, encoded as "Zn".
<Zn2>	Is the name of the second table vector register, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = VL DIV esize;
let indexes : bits(VL) = Z{}(m); // Only bottom 3/4 used
let table : bits(512)  = Z{VL}((n+1) MOD 32)[255:0] :: Z{VL}(n+0)[255:0];
var result : bits(VL);

for e = 0 to elements-1 do
    let index : integer = UInt(indexes[e*:isize]);
    let res : bits(esize) = table[index*:esize];
    result[e*:esize] = res;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

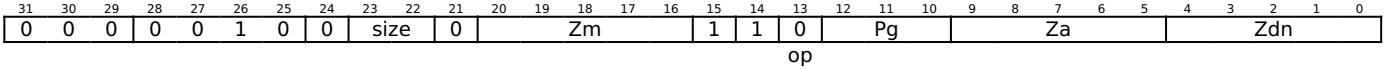
MAD

Multiply-add to multiplicand (predicated)

This instruction multiplies the Active elements of the first source vector by the corresponding elements of the second source vector, and adds the result to elements of the third (addend) vector. The results are destructively placed in the destination and first source (multiplicand) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MAD <Zdn>.<T>, <Pg>/M, <Zm>.<T>, <Za>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let a : integer = UInt(Za);
let sub_op : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Za> Is the name of the third source scalable vector register, encoded in the "Za" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let operand3 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(a) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : integer = UInt(operand1[e*:esize]);
        let element2 : integer = UInt(operand2[e*:esize]);
        let product : integer = element1 * element2;
        if sub_op then
            result[e*:esize] = operand3[e*:esize] - product;
        else
            result[e*:esize] = operand3[e*:esize] + product;
        end;
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

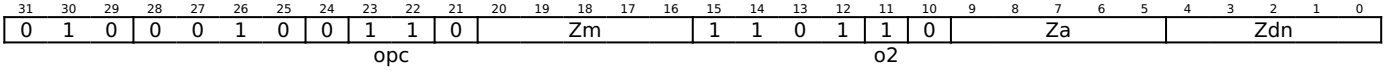
MADPT

Multiply-add checked pointer vectors to multiplicand

This instruction multiplies, with overflow check, the elements of the first and second source vectors and adds the results of multiplication, with pointer check, to elements of the third (addend) vector. The final results are destructively placed in the destination and first source (multiplicand) vector.
This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE && FEAT_CPA)



Encoding

```
MADPT <Zdn>.D, <Zm>.D, <Za>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_CPA) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn);
let m : integer  = UInt(Zm);
let a : integer  = UInt(Za);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Za>	Is the name of the third source scalable vector register, encoded in the "Za" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 64;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(a);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:64]);
    let element2 : integer = SInt(operand2[e*:64]);
    let product : integer = element1 * element2;
    let overflow : boolean = (product != SInt(product[63:0]));
    let addend : bits(64) = operand3[e*:64];
    result[e*:64] = PointerMultiplyAddCheck(addend + product, addend, overflow);
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MATCH

Detect any matching elements within quadwords, setting the condition flags

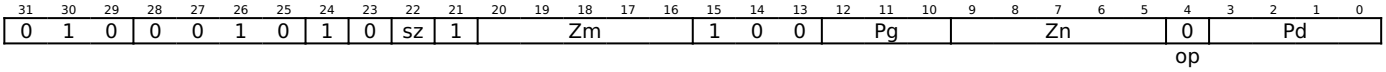
This instruction compares each Active 8-bit or 16-bit element in the first source vector with all of the elements in the corresponding 128-bit segment of the second source vector. For each element in the first source vector, if any matching element is found in the corresponding segment of the second source vector, the corresponding element of the destination predicate is set to true, otherwise it is set to false. Inactive elements in the destination predicate register are set to zero.

This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2)



Encoding

```
MATCH <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(sz);
let g : integer = UInt(Pg);
let d : integer = UInt(Pd);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	B
1	H

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let segmentbase : integer = e - (e MOD eltspersegment);
        result[e*:psize] = ZeroExtend{psize}('0');
        let element1 : bits(esize) = operand1[e*:esize];
        for i = segmentbase to (segmentbase + eltspersegment) - 1 do
            let element2 : bits(esize) = operand2[i*:esize];
            if element1 == element2 then
                result[e*:psize] = ZeroExtend{psize}('1');
            end;
        end;
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MLA (indexed)

Multiply-add by indexed element

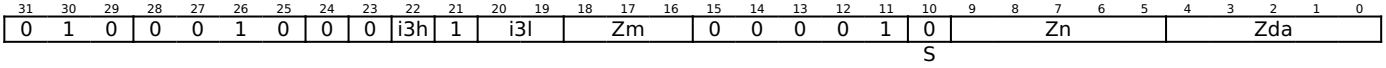
This instruction multiplies all integer elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment. The products are then destructively added to the corresponding elements of the addend and destination vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment. This instruction is unpredicated.

It has encodings from 3 classes: 16-bit , 32-bit and 64-bit

16-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

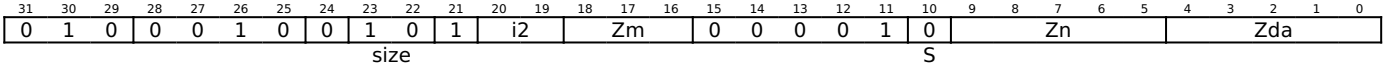
```
MLA <Zda>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

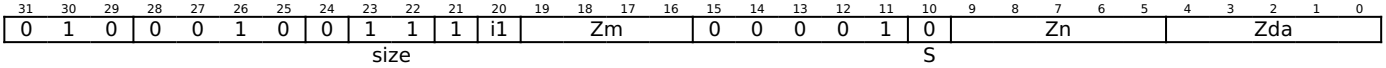
```
MLA <Zda>.S, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
MLA <Zda>.D, <Zn>.D, <Zm>.D[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "16-bit" and "32-bit" variants: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "16-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "32-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2" field. For the "64-bit" variant: is the element index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[s*:esize]);
    let product : bits(esize) = (element1 * element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

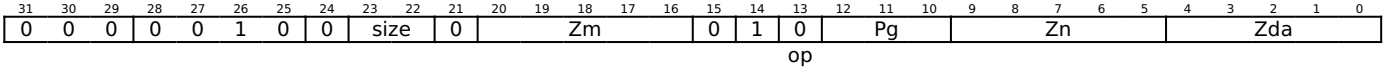
MLA (vectors)

Multiply-add (predicated)

This instruction multiplies the Active elements of the first source vector by the corresponding elements of the second source vector, and adds the result to elements of the third source (addend) vector. The results are destructively placed in the destination and third source (addend) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MLA <Zda>.<T>, <Pg>/M, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sub_op : boolean = FALSE;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : integer = UInt(operand1[e*:esize]);
        let element2 : integer = UInt(operand2[e*:esize]);
        let product : integer = element1 * element2;
        if sub_op then
            result[e*:esize] = operand3[e*:esize] - product;
        else
            result[e*:esize] = operand3[e*:esize] + product;
        end;
    else
        result[e*:esize] = operand3[e*:esize];
    end;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

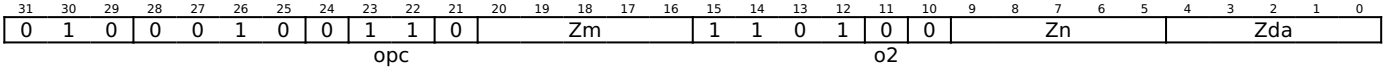
MLAPT

Multiply-add checked pointer vectors

This instruction multiplies, with overflow check, the elements of the first and second source vectors and adds the results of multiplication, with pointer check, to elements of the third source (addend) vector. The final results are destructively placed in the destination and third source (addend) vector.
This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE && FEAT_CPA)



Encoding

```
MLAPT <Zda>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_CPA) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 64;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:64]);
    let element2 : integer = SInt(operand2[e*:64]);
    let product : integer = element1 * element2;
    let overflow : boolean = (product != SInt(product[63:0]));
    let addend : bits(64) = operand3[e*:64];
    result[e*:64] = PointerMultiplyAddCheck(addend + product, addend, overflow);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MLS (indexed)

Multiply-subtract by indexed element

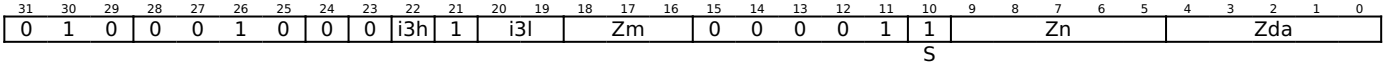
This instruction multiplies all integer elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment. The products are then destructively subtracted from the corresponding elements of the addend and destination vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment. This instruction is unpredicated.

It has encodings from 3 classes: 16-bit , 32-bit and 64-bit

16-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

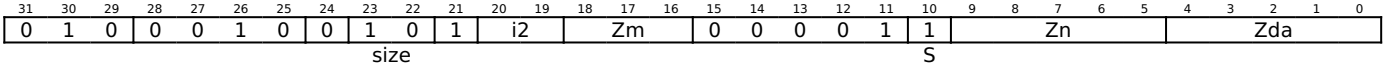
```
MLS <Zda>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

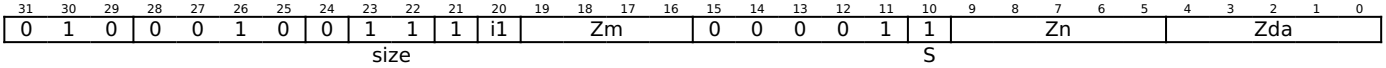
```
MLS <Zda>.S, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
MLS <Zda>.D, <Zn>.D, <Zm>.D[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "16-bit" and "32-bit" variants: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "16-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "32-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2" field. For the "64-bit" variant: is the element index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[s*:esize]);
    let product : bits(esize) = (element1 * element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] - product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

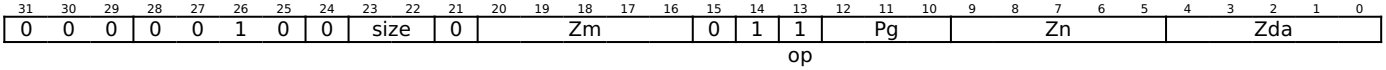
MLS (vectors)

Multiply-subtract (predicated)

This instruction multiplies the Active elements of the first source vector by the corresponding elements of the second source vector, and subtracts the result from elements of the third source (addend) vector. The results are destructively placed in the destination and third source (addend) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MLS <Zda>.<T>, <Pg>/M, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sub_op : boolean = TRUE;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : integer = UInt(operand1[e*:esize]);
        let element2 : integer = UInt(operand2[e*:esize]);
        let product : integer = element1 * element2;
        if sub_op then
            result[e*:esize] = operand3[e*:esize] - product;
        else
            result[e*:esize] = operand3[e*:esize] + product;
        end;
    else
        result[e*:esize] = operand3[e*:esize];
    end;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV

Move logical bitmask immediate to vector (unpredicated)

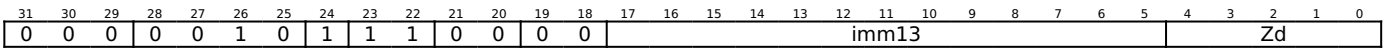
This instruction unconditionally broadcasts the logical bitmask immediate into each element of the destination vector. This instruction is unpredicated. The immediate is a 64-bit value consisting of a single run of ones or zeros repeating every 2, 4, 8, 16, 32, or 64 bits.

This is an alias of DUPM. This means:

- The encodings in this description are named to match the encodings of DUPM.
- The description of DUPM gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Zd>.<T>, #<const>
```

is equivalent to

```
DUPM <Zd>.<T>, #<const>
```

and is the preferred disassembly when `SVEMoveMaskPreferred(imm13)`.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “imm13”:

imm13	<T>
0xxxxxx0xxxxx	S
0xxxxxx10xxxx	H
0xxxxxx110xxx	B
0xxxxxx1110xx	B
0xxxxxx11110x	B
0xxxxxx11111x	RESERVED
1xxxxxxxxxxxx	D

<const> Is a 64, 32, 16, or 8-bit bitmask consisting of replicated 2, 4, 8, 16, 32, or 64-bit fields, each field containing a rotated run of nonzero bits, encoded in the "imm13" field.

Operation

The description of DUPM gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV

Move predicate (unpredicated)

This instruction reads all elements from the source predicate and places those elements in the destination predicate. This instruction is unpredicated. This instruction does not set the condition flags.

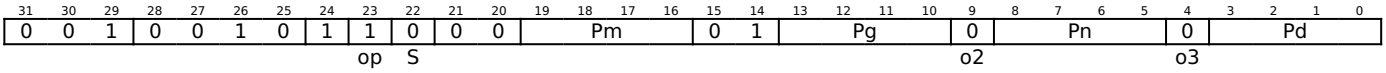
For programmer convenience, an assembler must also accept predicate-as-counter register names for the source and destination predicate registers.

This is an alias of ORR (predicates). This means:

- The encodings in this description are named to match the encodings of ORR (predicates).
- The description of ORR (predicates) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Pd>.B, <Pn>.B
```

is equivalent to

```
ORR <Pd>.B, <Pn>/Z, <Pn>.B, <Pn>.B
```

and is the preferred disassembly when `S == '0' && Pn == Pm && Pm == Pg`.

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

The description of ORR (predicates) gives the operational pseudocode for this instruction.

Operational Information

The description of ORR (predicates) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (immediate, merging)

Move signed integer immediate to vector elements (merging)

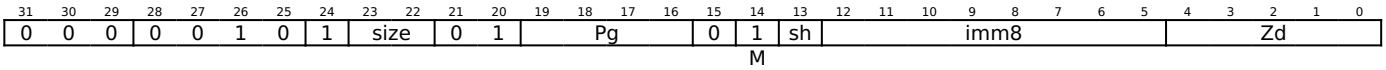
This instruction moves a signed integer immediate to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.
The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).
The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This is an alias of CPY (immediate, merging). This means:

- The encodings in this description are named to match the encodings of CPY (immediate, merging).
- The description of CPY (immediate, merging) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Zd>.<T>, <Pg>/M, #<imm>{, <shift>}
```

is equivalent to

```
CPY <Zd>.<T>, <Pg>/M, #<imm>{, <shift>}
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<imm> Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

The description of CPY (immediate, merging) gives the operational pseudocode for this instruction.

Operational Information

The description of CPY (immediate, merging) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (immediate, unpredicated)

Move signed immediate to vector elements (unpredicated)

This instruction unconditionally broadcasts the signed integer immediate into each element of the destination vector. This instruction is unpredicated.

The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

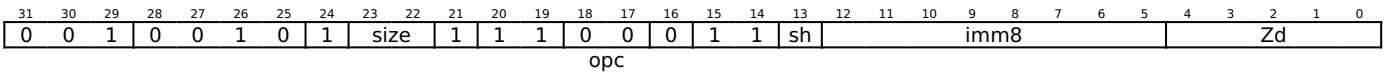
The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This is an alias of DUP (immediate). This means:

- The encodings in this description are named to match the encodings of DUP (immediate).
- The description of DUP (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Zd>.<T>, #<imm>{, <shift>}
```

is equivalent to

```
DUP <Zd>.<T>, #<imm>{, <shift>}
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

The description of DUP (immediate) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (immediate, zeroing)

Move signed integer immediate to vector elements (zeroing)

This instruction moves a signed integer immediate to each Active element in the destination vector. Inactive elements in the destination vector register are set to zero.

The immediate operand is a signed value in the range -128 to +127, and for element widths of 16 bits or higher it may also be a signed multiple of 256 in the range -32768 to +32512 (excluding 0).

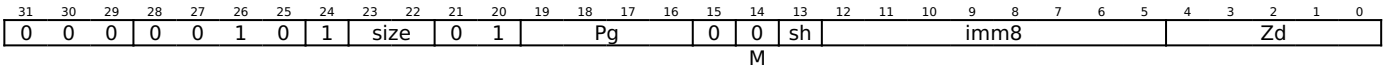
The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

This is an alias of CPY (immediate, zeroing). This means:

- The encodings in this description are named to match the encodings of CPY (immediate, zeroing).
- The description of CPY (immediate, zeroing) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Zd>.<T>, <Pg>/Z, #<imm>{, <shift>}
```

is equivalent to

```
CPY <Zd>.<T>, <Pg>/Z, #<imm>{, <shift>}
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register, encoded in the "Pg" field.

<imm> Is a signed immediate in the range -128 to 127, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

The description of CPY (immediate, zeroing) gives the operational pseudocode for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (predicate, merging)

Move predicates (merging)

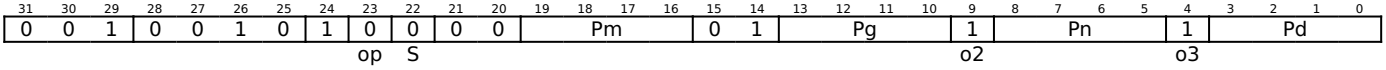
This instruction reads Active elements from the source predicate and places those elements in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register remain unmodified. This instruction does not set the condition flags.

This is an alias of SEL (predicates). This means:

- The encodings in this description are named to match the encodings of SEL (predicates).
- The description of SEL (predicates) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Pd>.B, <Pg>/M, <Pn>.B
```

is equivalent to

```
SEL <Pd>.B, <Pg>, <Pn>.B, <Pd>.B
```

and is the preferred disassembly when `Pd == Pm`.

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

The description of SEL (predicates) gives the operational pseudocode for this instruction.

Operational Information

The description of SEL (predicates) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (predicate, zeroing)

Move predicates (zeroing)

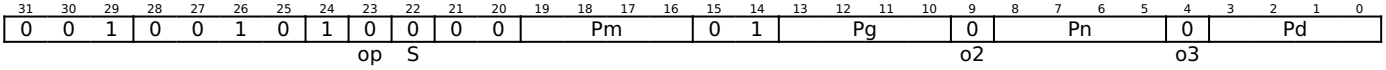
This instruction reads Active elements from the source predicate and places those elements in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

This is an alias of AND (predicates). This means:

- The encodings in this description are named to match the encodings of AND (predicates).
- The description of AND (predicates) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Pd>.B, <Pg>/Z, <Pn>.B
```

is equivalent to

```
AND <Pd>.B, <Pg>/Z, <Pn>.B, <Pn>.B
```

and is the preferred disassembly when `S == '0' && Pn == Pm`.

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

The description of AND (predicates) gives the operational pseudocode for this instruction.

Operational Information

The description of AND (predicates) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (scalar, predicated)

Move general-purpose register to vector elements (predicated)

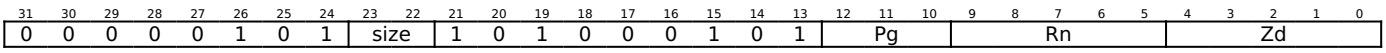
This instruction moves the general-purpose scalar source register to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.

This is an alias of CPY (scalar). This means:

- The encodings in this description are named to match the encodings of CPY (scalar).
- The description of CPY (scalar) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding for the 32-bit variant

Applies when `(size != 11)`

```
MOV <Zd>.<T>, <Pg>/M, <Wn|WSP>
```

is equivalent to

```
CPY <Zd>.<T>, <Pg>/M, <Wn|WSP>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
MOV <Zd>.D, <Pg>/M, <Xn|SP>
```

is equivalent to

```
CPY <Zd>.D, <Pg>/M, <Xn|SP>
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Wn|WSP> Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

<Xn|SP> Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Operation

The description of CPY (scalar) gives the operational pseudocode for this instruction.

Operational Information

The description of CPY (scalar) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (scalar, unpredicated)

Move general-purpose register to vector elements (unpredicated)

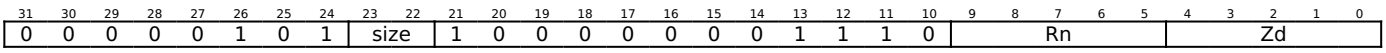
This instruction unconditionally broadcasts the general-purpose scalar source register into each element of the destination vector. This instruction is unpredicated.

This is an alias of DUP (scalar). This means:

- The encodings in this description are named to match the encodings of DUP (scalar).
- The description of DUP (scalar) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding for the 32-bit variant

Applies when `(size != 11)`

```
MOV <Zd>.<T>, <Wn|WSP>
```

is equivalent to

```
DUP <Zd>.<T>, <Wn|WSP>
```

and is always the preferred disassembly.

Encoding for the 64-bit variant

Applies when `(size == 11)`

```
MOV <Zd>.D, <Xn|SP>
```

is equivalent to

```
DUP <Zd>.D, <Xn|SP>
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S

<Wn|WSP> Is the 32-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

<Xn|SP> Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.

Operation

The description of DUP (scalar) gives the operational pseudocode for this instruction.

Operational Information

The description of DUP (scalar) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (SIMD&FP scalar, predicated)

Move SIMD&FP scalar register to vector elements (predicated)

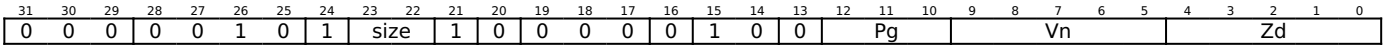
This instruction moves the SIMD & floating-point scalar source register to each Active element in the destination vector. Inactive elements in the destination vector register remain unmodified.

This is an alias of CPY (SIMD&FP scalar). This means:

- The encodings in this description are named to match the encodings of CPY (SIMD&FP scalar).
- The description of CPY (SIMD&FP scalar) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Zd>.<T>, <Pg>/M, <V><n>
```

is equivalent to

```
CPY <Zd>.<T>, <Pg>/M, <V><n>
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<n> Is the number [0-31] of the source SIMD&FP register, encoded in the "Vn" field.

Operation

The description of CPY (SIMD&FP scalar) gives the operational pseudocode for this instruction.

Operational Information

The description of CPY (SIMD&FP scalar) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (SIMD&FP scalar, unpredicated)

Move indexed element or SIMD&FP scalar to vector (unpredicated)

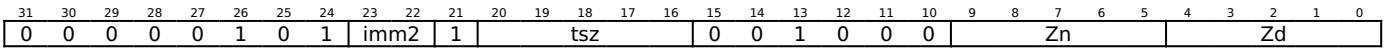
This instruction unconditionally broadcasts the SIMD&FP scalar into each element of the destination vector. This instruction is unpredicated.

This is an alias of DUP (indexed). This means:

- The encodings in this description are named to match the encodings of DUP (indexed).
- The description of DUP (indexed) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Zd>.<T>, <V><n>
```

is equivalent to

```
DUP <Zd>.<T>, <Zn>.<T>[0]
```

and is the preferred disassembly when `BitCount(imm2 :: tsz) == 1`.

Encoding

```
MOV <Zd>.<T>, <Zn>.<T>[<imm>]
```

is equivalent to

```
DUP <Zd>.<T>, <Zn>.<T>[<imm>]
```

and is the preferred disassembly when `BitCount(imm2 :: tsz) > 1`.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsz”:

tsz	<T>
00000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D
10000	Q

<V> Is a width specifier, encoded in “tsz”:

tsz	<V>
00000	RESERVED
xxxx1	B
xxx10	H
xx100	S
x1000	D
10000	Q

<n> Is the number [0-31] of the source SIMD&FP register, encoded in the "Zn" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<imm> Is the immediate index, in the range 0 to one less than the number of elements in 512 bits, encoded in "imm2:tsz".

Operation

The description of DUP (indexed) gives the operational pseudocode for this instruction.

Operational Information

The description of DUP (indexed) gives the operational information for this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (vector, predicated)

Move vector elements (predicated)

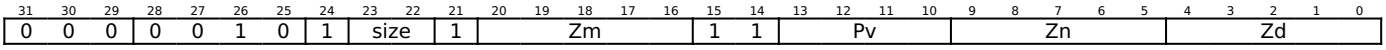
This instruction moves elements from the source vector to the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified.

This is an alias of SEL (vectors). This means:

- The encodings in this description are named to match the encodings of SEL (vectors).
- The description of SEL (vectors) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Zd>.<T>, <Pv>/M, <Zn>.<T>
```

is equivalent to

```
SEL <Zd>.<T>, <Pv>, <Zn>.<T>, <Zd>.<T>
```

and is the preferred disassembly when `Zd == Zm`.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pv> Is the name of the vector select predicate register, encoded in the "Pv" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

The description of SEL (vectors) gives the operational pseudocode for this instruction.

Operational Information

The description of SEL (vectors) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (vector, unpredicated)

Move vector register (unpredicated)

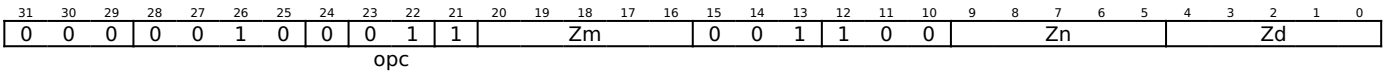
This instruction copies the contents of the source vector register to the destination vector register. This instruction is unpredicated.

This is an alias of ORR (vectors, unpredicated). This means:

- The encodings in this description are named to match the encodings of ORR (vectors, unpredicated).
- The description of ORR (vectors, unpredicated) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOV <Zd>.D, <Zn>.D
```

is equivalent to

```
ORR <Zd>.D, <Zn>.D, <Zn>.D
```

and is the preferred disassembly when `Zn == Zm`.

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

The description of ORR (vectors, unpredicated) gives the operational pseudocode for this instruction.

Operational Information

The description of ORR (vectors, unpredicated) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVPRFX (predicated)

Move prefix (predicated)

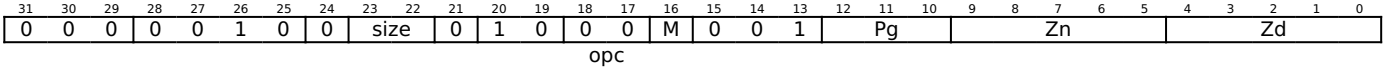
This instruction is a hint to hardware that the instruction may be combined with the destructive instruction that follows it in program order to create a single constructive operation. This instruction is also permitted to be implemented as a discrete vector copy, and the result of executing the pair of instructions with or without combining is identical. The choice of combined versus discrete operation may vary dynamically.

Unless the combination of a constructive operation with merging predication is specifically required, it is strongly recommended that for performance reasons software should prefer to use the zeroing form of predicated MOVPRFX or the unpredicated MOVPRFX instruction.

Although the operation of the instruction is defined as a simple predicated vector copy, it is required that the prefixed instruction at PC+4 must be an SVE destructive binary or ternary instruction encoding, or a unary operation with merging predication, but excluding other MOVPRFX instructions. The prefixed instruction must specify the same predicate register, and have the same maximum element size (ignoring a fixed 64-bit "wide vector" operand), and the same destination vector as the MOVPRFX instruction. The prefixed instruction must not use the destination register in any other operand position, even if they have different names but refer to the same architectural register state. Any other use is UNPREDICTABLE.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOVPRFX <Zd>.<T>, <Pg>/<ZM>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = (M == '1');
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<ZM> Is the predication qualifier, encoded in “M”:

M	<ZM>
0	Z
1	M

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let dest : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand1[e*:esize];
        result[e*:esize] = element;
    else
        result[e*:esize] = dest[e*:esize];
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVPRFX (unpredicated)

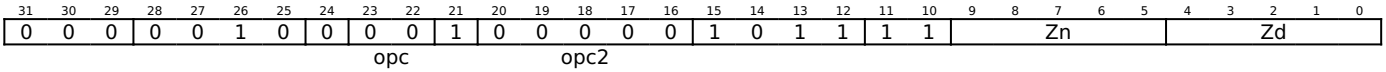
Move prefix (unpredicated)

This instruction is a hint to hardware that the instruction may be combined with the destructive instruction that follows it in program order to create a single constructive operation. This instruction is also permitted to be implemented as a discrete vector copy, and the result of executing the pair of instructions with or without combining is identical. The choice of combined versus discrete operation may vary dynamically.

Although the operation of the instruction is defined as a simple unpredicated vector copy, it is required that the prefixed instruction at PC+4 must be an SVE destructive binary or ternary instruction encoding, or a unary operation with merging predication, but excluding other `MOVPRFX` instructions. The prefixed instruction must specify the same destination vector as the `MOVPRFX` instruction. The prefixed instruction must not use the destination register in any other operand position, even if they have different names but refer to the same architectural register state. Any other use is UNPREDICTABLE.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MOVPRFX <Zd>, <Zn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let result : bits(VL) = Z{}(n);
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVS (predicated)

Move predicates (zeroing), setting the condition flags

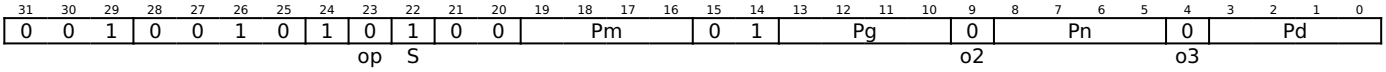
This instruction reads Active elements from the source predicate and places those elements in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This is an alias of ANDS. This means:

- The encodings in this description are named to match the encodings of ANDS.
- The description of ANDS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
MOVS <Pd>.B, <Pg>/Z, <Pn>.B
```

is equivalent to

```
ANDS <Pd>.B, <Pg>/Z, <Pn>.B, <Pn>.B
```

and is the preferred disassembly when `S == '1' && Pn == Pm`.

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

The description of ANDS gives the operational pseudocode for this instruction.

Operational Information

The description of ANDS gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVS (unpredicated)

Move predicate (unpredicated), setting the condition flags

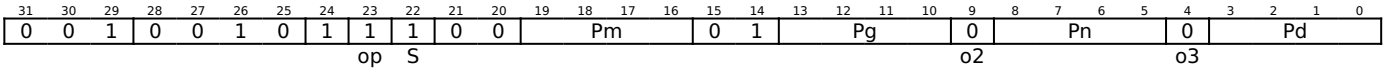
This instruction reads all elements from the source predicate and places those elements in the destination predicate. This instruction is unpredicated. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This is an alias of ORRS. This means:

- The encodings in this description are named to match the encodings of ORRS.
- The description of ORRS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
MOVS <Pd>.B, <Pn>.B
```

is equivalent to

```
ORRS <Pd>.B, <Pn>/Z, <Pn>.B, <Pn>.B
```

and is the preferred disassembly when `S == '1' && Pn == Pm && Pm == Pg`.

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

The description of ORRS gives the operational pseudocode for this instruction.

Operational Information

The description of ORRS gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

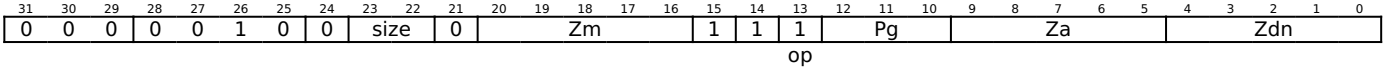
MSB

Multiply-subtract to multiplicand

This instruction multiplies the Active elements of the first source vector by the corresponding elements of the second source vector, and subtracts the result from elements of the third (addend) vector. The results are destructively placed in the destination and first source (multiplicand) vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MSB <Zdn>.<T>, <Pg>/M, <Zm>.<T>, <Za>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
let a : integer = UInt(Za);
let sub_op : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Za> Is the name of the third source scalable vector register, encoded in the "Za" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let operand3 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(a) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element1 : integer = UInt(operand1[e*:esize]);
        let element2 : integer = UInt(operand2[e*:esize]);
        let product : integer = element1 * element2;
        if sub_op then
            result[e*:esize] = operand3[e*:esize] - product;
        else
            result[e*:esize] = operand3[e*:esize] + product;
        end;
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

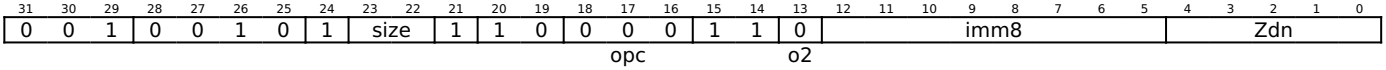
MUL (immediate)

Multiply by immediate (unpredicated)

This instruction multiplies each element of the source vector by an immediate, and destructively places the results in the corresponding elements of the source vector. The immediate is a signed 8-bit value in the range -128 to +127, inclusive. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MUL <Zdn>.<T>, <Zdn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
let imm : integer = SInt(imm8);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is the signed immediate operand, in the range -128 to 127, encoded in the "imm8" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    result[e*:esize] = (element1 * imm)[esize-1:0];

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MUL (indexed)

Multiply by indexed element

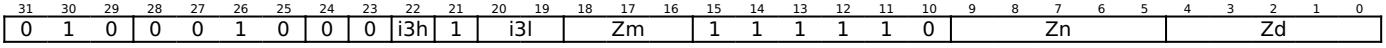
This instruction multiplies all integer elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment. The results are placed in the corresponding elements of the destination vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment. This instruction is unpredicated.

It has encodings from 3 classes: 16-bit , 32-bit and 64-bit

16-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

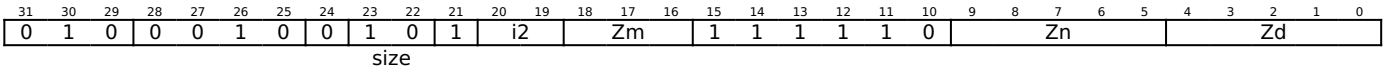
```
MUL <Zd>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

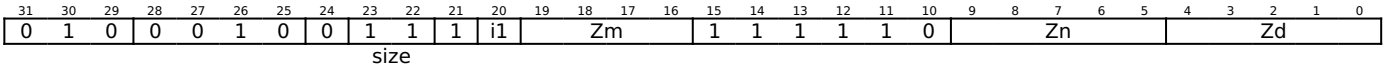
```
MUL <Zd>.S, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
MUL <Zd>.D, <Zn>.D, <Zm>.D[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.

<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "16-bit" and "32-bit" variants: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "16-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "32-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2" field. For the "64-bit" variant: is the element index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[s*:esize]);
    let res : integer = element1 * element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

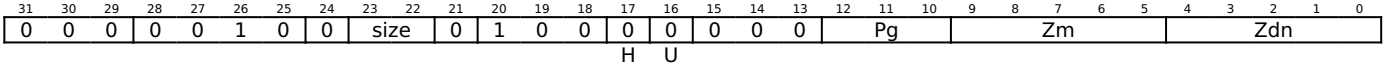
MUL (vectors, predicated)

Multiply (predicated)

This instruction multiplies Active elements of the first source vector by the corresponding elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
MUL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let product : integer = element1 * element2;
        result[e*:esize] = product[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

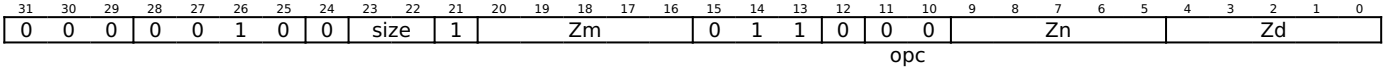
MUL (vectors, unpredicated)

Multiply (unpredicated)

This instruction multiplies elements of the first source vector by the corresponding elements of the second source vector and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
MUL <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    let product : integer = element1 * element2;
    result[e*:esize] = product[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

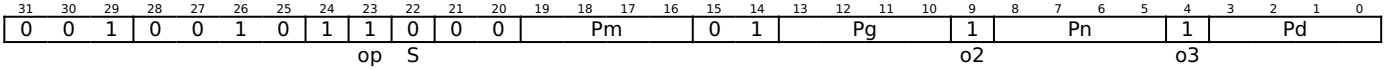
NAND

Bitwise NAND predicates

This instruction performs a bitwise NAND between the Active elements of the first source predicate and the corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
NAND <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(NOT(element1 AND element2));
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

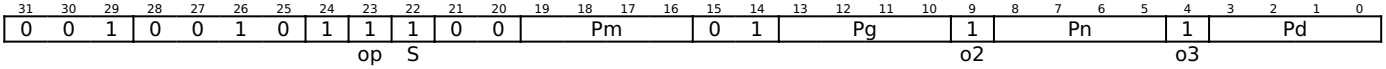
NANDS

Bitwise NAND predicates, setting the condition flags

This instruction performs a bitwise NAND between the Active elements of the first source predicate and the corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
NANDS <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(NOT(element1 AND element2));
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

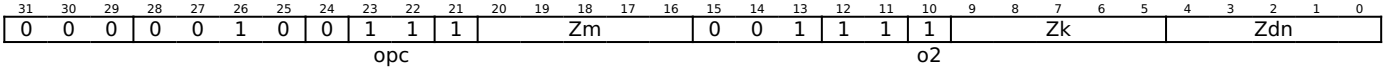
NBSL

Bitwise inverted select

This instruction selects bits from the first source vector where the corresponding bit in the third source vector is '1', and from the second source vector where the corresponding bit in the third source vector is '0'. The inverted result is destructively placed in the destination and first source vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
NBSL <Zdn>.D, <Zdn>.D, <Zm>.D, <Zk>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let m : integer = UInt(Zm);
let k : integer = UInt(Zk);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the third source scalable vector register, encoded in the "Zk" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(k);

Z{VL}(dn) = NOT((operand1 AND operand3) OR (operand2 AND NOT(operand3)));
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

NEG

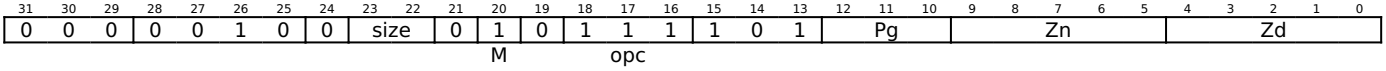
Negate (predicated)

This instruction negates the signed integer value in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

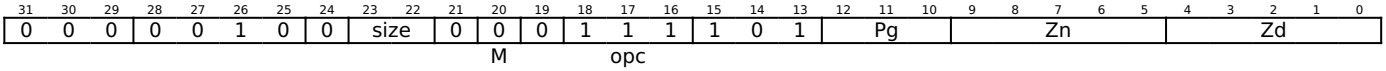
```
NEG <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
NEG <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        var element : integer = SInt(operand[e*:esize]);
        element = -element;
        result[e*:esize] = element[esize-1:0];

    end;
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

NMATCH

Detect no matching elements within quadwords, setting the condition flags

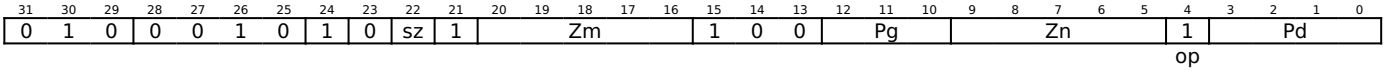
This instruction compares each Active 8-bit or 16-bit element in the first source vector with all of the elements in the corresponding 128-bit segment of the second source vector. For each element in the first source vector, if no matching element is found in the corresponding segment of the second source vector, the corresponding element of the destination predicate is set to true, otherwise it is set to false. Inactive elements in the destination predicate register are set to zero.

This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2)



Encoding

```
NMATCH <Pd>.<T>, <Pg>/Z, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(sz);
let g : integer = UInt(Pg);
let d : integer = UInt(Pd);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	B
1	H

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let segmentbase : integer = e - (e MOD eltspersegment);
        result[e*:psize] = ZeroExtend{psize}('1');
        let element1 : bits(esize) = operand1[e*:esize];
        for i = segmentbase to (segmentbase + eltspersegment) - 1 do
            let element2 : bits(esize) = operand2[i*:esize];
            if element1 == element2 then
                result[e*:psize] = ZeroExtend{psize}('0');
            end;
        end;
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

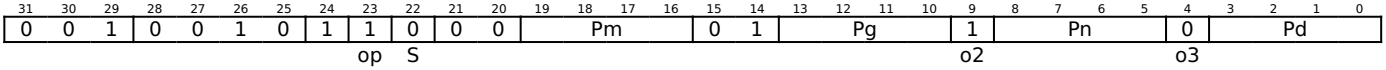
NOR

Bitwise NOR predicates

This instruction performs a bitwise NOR between the Active elements of the first source predicate and the corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
NOR <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(NOT(element1 OR element2));
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

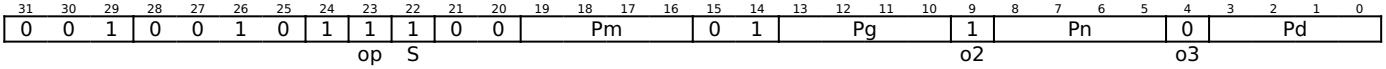
NORS

Bitwise NOR predicates, setting the condition flags

This instruction performs a bitwise NOR between the Active elements of the first source predicate and the corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
NORS <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(NOT(element1 OR element2));
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

NOT (predicate)

Bitwise invert predicate

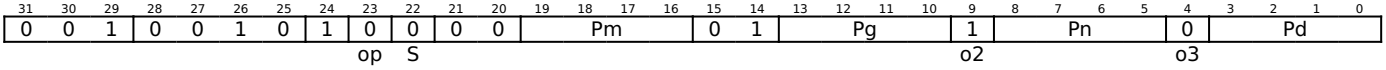
This instruction performs a bitwise invert of each Active element of the source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

This is an alias of EOR (predicates). This means:

- The encodings in this description are named to match the encodings of EOR (predicates).
- The description of EOR (predicates) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
NOT <Pd>.B, <Pg>/Z, <Pn>.B
```

is equivalent to

```
EOR <Pd>.B, <Pg>/Z, <Pn>.B, <Pg>.B
```

and is the preferred disassembly when `Pm == Pg`.

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

The description of EOR (predicates) gives the operational pseudocode for this instruction.

Operational Information

The description of EOR (predicates) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

NOT (vector)

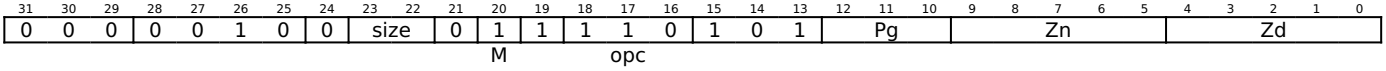
Bitwise invert (predicated)

This instruction performs a bitwise invert of each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

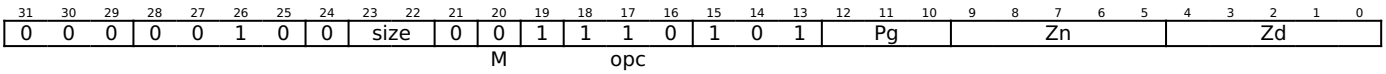
```
NOT <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
NOT <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = NOT element;

    end;
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

NOTS

Bitwise invert predicate, setting the condition flags

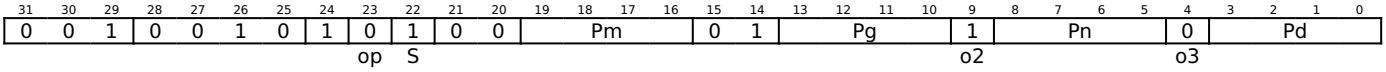
This instruction performs a bitwise invert of each Active element of the source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This is an alias of EORS. This means:

- The encodings in this description are named to match the encodings of EORS.
- The description of EORS gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
NOTS <Pd>.B, <Pg>/Z, <Pn>.B
```

is equivalent to

```
EORS <Pd>.B, <Pg>/Z, <Pn>.B, <Pg>.B
```

and is the preferred disassembly when `Pm == Pg`.

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

The description of EORS gives the operational pseudocode for this instruction.

Operational Information

The description of EORS gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ORN (immediate)

Bitwise inclusive OR with inverted immediate (unpredicated)

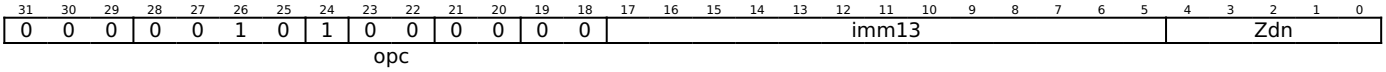
This instruction performs a bitwise inclusive OR between an inverted immediate and each 64-bit element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate is a 64-bit value consisting of a single run of ones or zeros repeating every 2, 4, 8, 16, 32, or 64 bits. This instruction is unpredicated.

This is a pseudo-instruction of ORR (immediate). This means:

- The encodings in this description are named to match the encodings of ORR (immediate).
- The assembler syntax is used only for assembly, and is not used on disassembly.
- The description of ORR (immediate) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ORN <Zdn>.<T>, <Zdn>.<T>, #<const>
```

is equivalent to

```
ORR <Zdn>.<T>, <Zdn>.<T>, #(-<const> - 1)
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “imm13”:

imm13	<T>
0xxxxxx0xxxxx	S
0xxxxxx10xxxx	H
0xxxxxx110xxx	B
0xxxxxx1110xx	B
0xxxxxx11110x	B
0xxxxxx11111x	RESERVED
1xxxxxxxxxxxx	D

<const> Is a 64, 32, 16, or 8-bit bitmask consisting of replicated 2, 4, 8, 16, 32, or 64-bit fields, each field containing a rotated run of nonzero bits, encoded in the "imm13" field.

Operation

The description of ORR (immediate) gives the operational pseudocode for this instruction.

Operational Information

The description of ORR (immediate) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

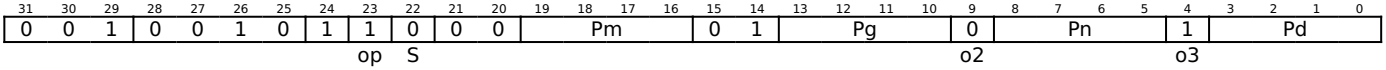
ORN (predicates)

Bitwise inclusive OR inverted predicate

This instruction performs a bitwise inclusive OR between the Active elements of the first source predicate and the inverted corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
ORN <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1 OR (NOT element2));
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

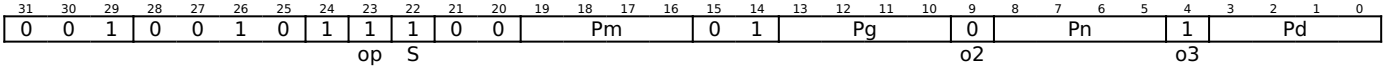
ORNS

Bitwise inclusive OR inverted predicate, setting the condition flags

This instruction performs a bitwise inclusive OR between the Active elements of the first source predicate and the inverted corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
ORNS <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1 OR (NOT element2));
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

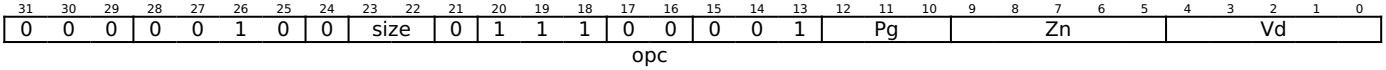
ORQV

Bitwise inclusive OR reduction of quadword vector segments

This instruction performs a bitwise inclusive OR between elements in the same position from each 128-bit source vector segment, and places each result into the corresponding element number of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as all zeros.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ORQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	16B
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(128) = Zeros{};
var stmp : bits(128) = Zeros{};

var dtmp : bits(esize);

for e = 0 to elempersegment-1 do
    dtmp = Zeros{esize};
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp = operand[s*:128];
            dtmp = dtmp OR stmp[e*:esize];
        end;
    end;
    result[e*:esize] = dtmp[esize-1:0];
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

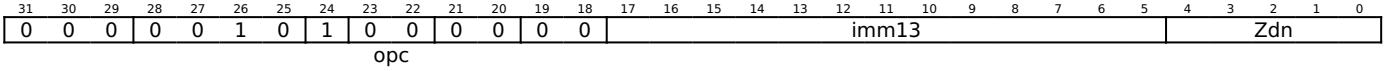
ORR (immediate)

Bitwise inclusive OR with immediate (unpredicated)

This instruction performs a bitwise inclusive OR between an immediate and each 64-bit element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate is a 64-bit value consisting of a single run of ones or zeros repeating every 2, 4, 8, 16, 32, or 64 bits. This instruction is unpredicated.
This instruction is used by the pseudo-instruction ORN (immediate).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ORR <Zdn>.<T>, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn);
var imm : bits(64);
(imm, -) = DecodeBitMasks{64}(imm13[12], imm13[5:0], imm13[11:6], TRUE);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “imm13”:

imm13	<T>
0xxxxxx0xxxxx	S
0xxxxxx10xxxx	H
0xxxxxx110xxx	B
0xxxxxx1110xx	B
0xxxxxx11110x	B
0xxxxxx11111x	RESERVED
1xxxxxxxxxxxx	D

<const> Is a 64, 32, 16, or 8-bit bitmask consisting of replicated 2, 4, 8, 16, 32, or 64-bit fields, each field containing a rotated run of nonzero bits, encoded in the "imm13" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 64;
let operand : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(64) = operand[e*:64];
    result[e*:64] = element1 OR imm;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

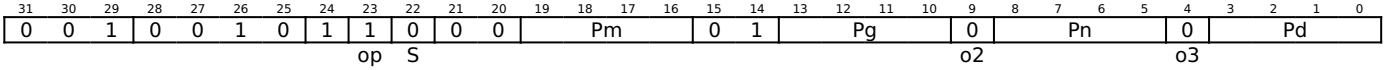
ORR (predicates)

Bitwise inclusive OR predicates

This instruction performs a bitwise inclusive OR between the Active elements of the first source predicate and the corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.
This instruction is used by the alias MOV.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
ORR <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Alias Conditions

Alias	Is preferred when
MOV	`S == '0' && Pn == Pm && Pm == Pg`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1 OR element2);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

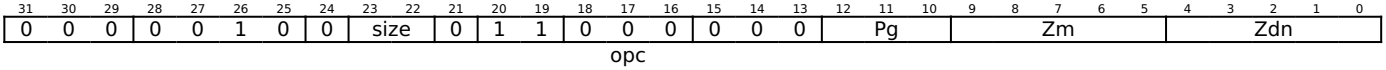
ORR (vectors, predicated)

Bitwise inclusive OR (predicated)

This instruction performs a bitwise inclusive OR between the Active elements of the first source vector and the corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ORR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = element1 OR element2;
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ORR (vectors, unpredicated)

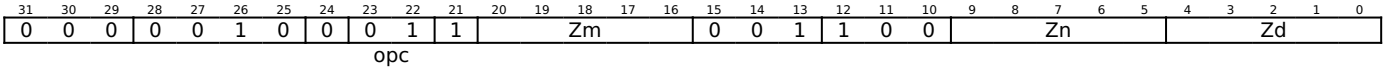
Bitwise inclusive OR (unpredicated)

This instruction performs a bitwise inclusive OR between the elements of the first source vector and the corresponding elements of the second source vector, and places the first in the corresponding elements of the destination vector. This instruction is unpredicated.

This instruction is used by the alias MOV (vector, unpredicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ORR <Zd>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Alias Conditions

Alias	Is preferred when
MOV (vector, unpredicated)	`Zn == Zm`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);

Z{VL}(d) = operand1 OR operand2;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ORRS

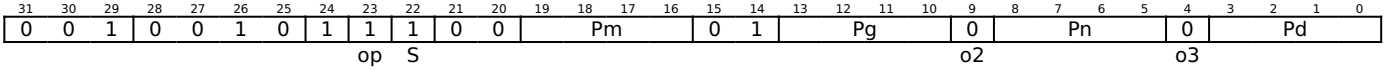
Bitwise inclusive OR predicates, setting the condition flags

This instruction performs a bitwise inclusive OR between the Active elements of the first source predicate and the corresponding elements of the second source predicate, and places the results in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This instruction is used by the alias MOVS (unpredicated).

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
ORRS <Pd>.B, <Pg>/Z, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Alias Conditions

Alias	Is preferred when
MOVS (unpredicated)	`S == '1' && Pn == Pm && Pm == Pg`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1 OR element2);
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
end;
P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

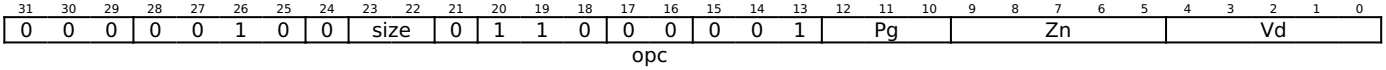
ORV

Bitwise inclusive OR reduction to scalar

This instruction performs a bitwise inclusive OR between the elements of the source vector, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ORV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(esize) = Zeros{esize};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result = result OR operand[e*:esize];

    end;
end;
V{esize}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

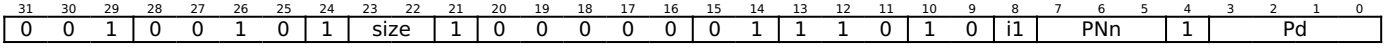
PEXT (predicate pair)

Predicate extract pair from predicate-as-counter

This instruction converts the source predicate-as-counter into a four register wide predicate-as-mask, and copies the two portions of the mask value selected by the portion index to the two destination predicate registers. A portion corresponds to a one predicate register fraction of the wider predicate-as-mask value.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
PEXT { <Pd1>.<T>, <Pd2>.<T> }, <PNn>[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt('1'::PNn);
let d0 : integer = UInt(Pd);
let d1 : integer = (UInt(Pd) + 1) MOD 16;
let part : integer = UInt(i1);
```

Assembler Symbols

<Pd1> Is the name of the first destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pd2> Is the name of the second destination scalable predicate register, encoded in the "Pd" field.

<PNn> Is the name of the first source scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNn" plus 8.

<imm> Is the portion index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then CheckSVEEnabled(); else CheckStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let pred : bits(PL) = P{}(n);
let mask : bits(PL*4) = CounterToPredicate{}(pred[15:0]);
var result0 : bits(PL);
var result1 : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let pbit : bit = PredicateElement{PL*4}(mask, part * 2 * elements + e, esize);
    result0[e*:psize] = ZeroExtend{psize}(pbit);
end;

for e = 0 to elements-1 do
    let pbit : bit = PredicateElement{PL*4}(mask, part * 2 * elements + elements + e, esize);
    result1[e*:psize] = ZeroExtend{psize}(pbit);
end;

P{PL}(d0) = result0;
P{PL}(d1) = result1;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

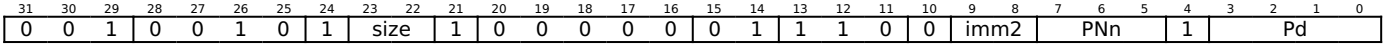
PEXT (predicate)

Predicate extract from predicate-as-counter

This instruction converts the source predicate-as-counter into a four register wide predicate-as-mask, and copies the portion of the mask value selected by the portion index to the destination predicate register. A portion corresponds to a one predicate register fraction of the wider predicate-as-mask value.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
PEXT <Pd>.<T>, <PNn>[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt('1'::PNn);
let d : integer = UInt(Pd);
let part : integer = UInt(imm2);
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<PNn> Is the name of the first source scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNn" plus 8.

<imm> Is the portion index, in the range 0 to 3, encoded in the "imm2" field.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then CheckSVEEnabled(); else CheckStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let pred : bits(PL) = P{}(n);
let mask : bits(PL*4) = CounterToPredicate{}(pred[15:0]);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let pbit : bit = PredicateElement{PL*4}(mask, part * elements + e, esize);
    result[e*:psize] = ZeroExtend{psize}(pbit);
end;

P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

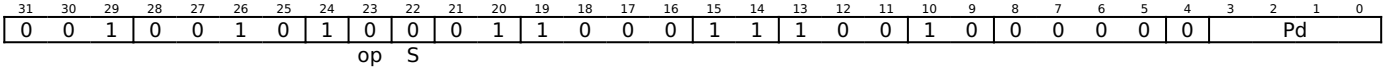
PFALSE

Set all predicate elements to false

This instruction sets all elements in the destination predicate to false.
For programmer convenience, an assembler must also accept predicate-as-counter register name for the destination predicate register.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PFALSE <Pd>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer = UInt(Pd);
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
P{PL}(d) = Zeros{PL};
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

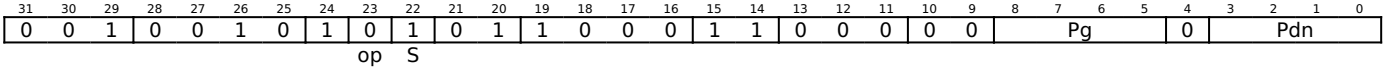
PFIRST

Set the First active predicate element to true

This instruction sets the First active element in the destination predicate to true, otherwise elements from the source predicate are passed through unchanged. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PFIRST <Pdn>.B, <Pg>, <Pdn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let dn : integer = UInt(Pdn);
```

Assembler Symbols

<Pdn>	Is the name of the source and destination scalable predicate register, encoded in the "Pdn" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var result : bits(PL) = P{}(dn);
var first : integer = -1;
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) && first == -1 then
        first = e;
    end;
end;

if first >= 0 then
    result[first*:psize] = ZeroExtend{psize}('1');
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(dn) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PMLAL

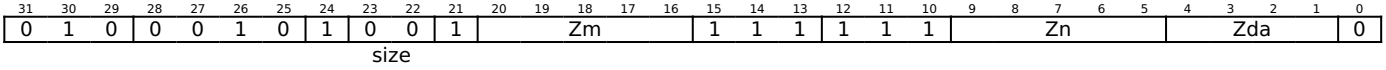
Multi-vector polynomial multiply long and accumulate

This instruction performs a polynomial multiplication over [0, 1] of the even-numbered elements of the first source vector by the corresponding elements of the second source vector, and then performs a bitwise exclusive-OR between the result and the overlapping double-width elements of the first destination vector. The same operation is performed with odd-numbered elements of the source vectors, writing to the second destination vector. This instruction is unpredicated.

This instruction is legal when executed in Streaming SVE mode if both FEAT_SSVE_AES and FEAT_SVE_AES2 are implemented.

SVE2

(FEAT_SVE_AES2)



Encoding

```
PMLAL { <Zda1>.Q-<Zda2>.Q }, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda::'0');
```

Assembler Symbols

<Zda1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zda" times 2.
<Zda2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zda" times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_AES) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result_lo : bits(VL) = Z{}(da + 0);
var result_hi : bits(VL) = Z{}(da + 1);

for e = 0 to elements-1 do
    let element1_lo : bits(esize DIV 2) = operand1[(2*e + 0)*:(esize DIV 2)];
    let element2_lo : bits(esize DIV 2) = operand2[(2*e + 0)*:(esize DIV 2)];
    let element1_hi : bits(esize DIV 2) = operand1[(2*e + 1)*:(esize DIV 2)];
    let element2_hi : bits(esize DIV 2) = operand2[(2*e + 1)*:(esize DIV 2)];
    let product_lo : bits(esize) = PolynomialMult{esize DIV 2, esize DIV 2}(element1_lo,
                                                                            element2_lo);
    let product_hi : bits(esize) = PolynomialMult{esize DIV 2, esize DIV 2}(element1_hi,
                                                                            element2_hi);
    result_lo[e*:esize] = result_lo[e*:esize] XOR product_lo;
    result_hi[e*:esize] = result_hi[e*:esize] XOR product_hi;
end;

Z{VL}(da + 0) = result_lo;
Z{VL}(da + 1) = result_hi;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PMOV (to predicate)

Move predicate from vector

This instruction copies a packed bitmap, where bit value 0b1 represents TRUE and bit value 0b0 represents FALSE, from a portion of the source vector register to elements of the destination SVE predicate register.

Because the number of bits in an SVE predicate element scales with the vector element size, the behavior varies according to the specified element size.

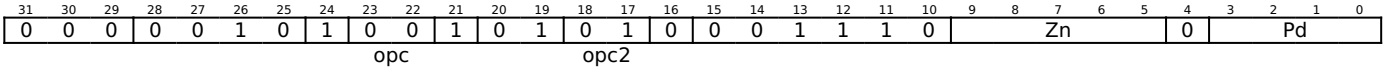
- When the predicate element specifier is B, each bit [N] from the least-significant VL/8 bits in the source vector register is copied to bit [N] of the destination predicate register. The portion index, if specified, must be 0.
- When the predicate element specifier is H, each bit [N] within the indexed block of VL/16 bits in the source vector register is copied to bit [N*2] of the destination predicate register, and the other bits in the predicate are set to zero. The portion index is in the range 0 to 1, inclusive.
- When the predicate element specifier is S, each bit [N] within the indexed block of VL/32 bits in the source vector register is copied to bit [N*4] of the destination predicate register, and the other bits in the predicate are set to zero. The portion index is in the range 0 to 3, inclusive.
- When the predicate element specifier is D, each bit [N] within the indexed block of VL/64 bits in the source vector register is copied to bit [N*8] of the destination predicate register, and the other bits in the predicate are set to zero. The portion index is in the range 0 to 7, inclusive.

The portion index is optional, defaulting to 0 if omitted.

It has encodings from 4 classes: Byte , Doubleword , Halfword and Word

Byte

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

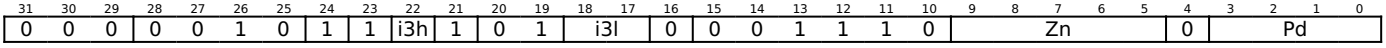
```
PMOV <Pd>.B, <Zn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let esize : integer{} = 8;
let imm : integer = 0;
```

Doubleword

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

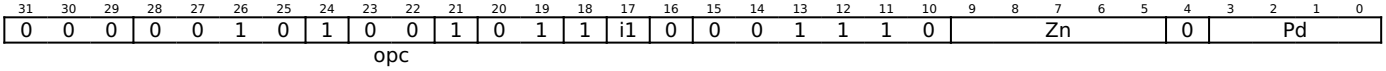
```
PMOV <Pd>.D, <Zn>{[<imm>]}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let esize : integer{} = 64;
let imm : integer = UInt(i3h::i3l);
```

Halfword

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

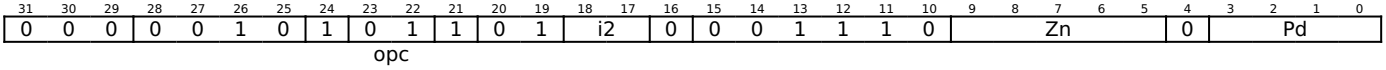
```
PMOV <Pd>.H, <Zn>{[<imm>]}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let esize : integer{} = 16;
let imm : integer = UInt(i1);
```

Word

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
PMOV <Pd>.S, <Zn>{[<imm>]}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Pd);
let esize : integer{} = 32;
let imm : integer = UInt(i2);
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.
<imm>	For the "Doubleword" variant: is the optional portion index, in the range 0 to 7, defaulting to 0, encoded in the "i3h:i3l" fields. For the "Halfword" variant: is the optional portion index, in the range 0 to 1, defaulting to 0, encoded in the "i1" field. For the "Word" variant: is the optional portion index, in the range 0 to 3, defaulting to 0, encoded in the "i2" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    result[e*:psize] = ZeroExtend{psize}(operand[(elements * imm) + e]);
end;

P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PMOV (to vector)

Move predicate to vector

This instruction copies the source SVE predicate register elements into the destination vector register as a packed bitmap with one bit per predicate element, where bit value 0b1 represents a TRUE predicate element, and bit value 0b0 represents a FALSE predicate element.

Because the number of bits in an SVE predicate element scales with the vector element size, the behavior varies according to the specified element size.

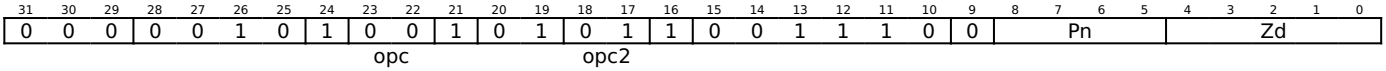
- When the predicate element specifier is B, every bit in the predicate register is copied to the least-significant VL/8 bits of the destination vector register. The portion index, if specified, must be 0.
- When the predicate element specifier is H, every second bit in the predicate register is copied to the indexed block of VL/16 bits in the destination vector register, where the portion index is in the range 0 to 1, inclusive.
- When the predicate element specifier is S, every fourth bit in the predicate register is copied to the indexed block of VL/32 bits in the destination vector register, where the portion index is in the range 0 to 3, inclusive.
- When the predicate element specifier is D, every eighth bit in the predicate register is copied to the indexed block of VL/64 bits in the destination vector register, where the portion index is in the range 0 to 7, inclusive.

The portion index is optional, defaulting to 0 if omitted. When the index is zero, the instruction writes zeros to the most significant VL-(VL/esize) bits of the destination vector register. When a nonzero index is specified, the packed bitmap is inserted into the destination vector register, and the unindexed blocks remain unchanged.

It has encodings from 4 classes: Byte , Doubleword , Halfword and Word

Byte

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

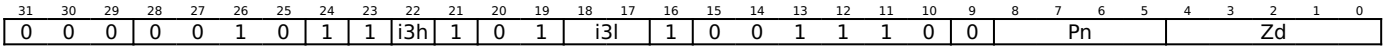
```
PMOV <Zd>, <Pn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Pn);
let d : integer = UInt(Zd);
let esize : integer{} = 8;
let imm : integer = 0;
```

Doubleword

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

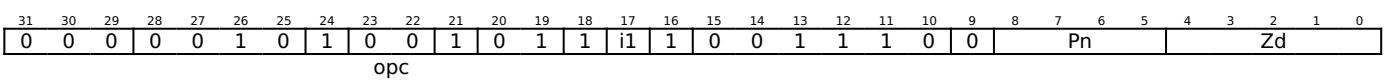
```
PMOV <Zd>{[<imm>]}, <Pn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Pn);
let d : integer = UInt(Zd);
let esize : integer{} = 64;
let imm : integer = UInt(i3h::i3l);
```

Halfword

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

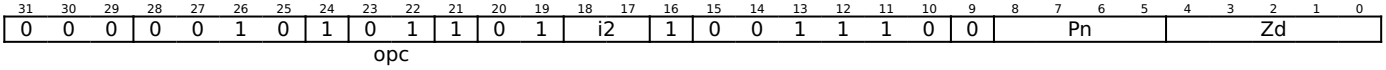
```
PMOV <Zd>{[<imm>]}, <Pn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Pn);
let d : integer = UInt(Zd);
let esize : integer{} = 16;
let imm : integer = UInt(i1);
```

Word

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
PMOV <Zd>{[<imm>]}, <Pn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Pn);
let d : integer = UInt(Zd);
let esize : integer{} = 32;
let imm : integer = UInt(i2);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.
<imm>	For the "Doubleword" variant: is the optional portion index, in the range 0 to 7, defaulting to 0, encoded in the "i3h:i3l" fields. For the "Halfword" variant: is the optional portion index, in the range 0 to 1, defaulting to 0, encoded in the "i1" field. For the "Word" variant: is the optional portion index, in the range 0 to 3, defaulting to 0, encoded in the "i2" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand : bits(PL) = P{}(n);
var result : bits(VL);

if imm == 0 then
    result = Zeros{VL};
else
    result = Z{VL}(d);
end;

for e = 0 to elements-1 do
    result[(elements * imm) + e] = PredicateElement{PL}(operand, e, esize);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

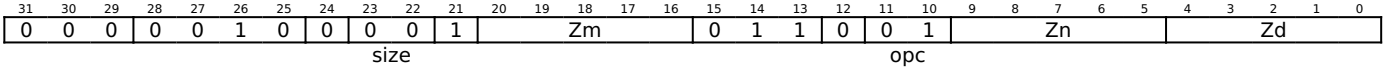
PMUL

Polynomial multiply (unpredicated)

This instruction performs a polynomial multiplication over [0, 1] of all elements of the first source vector by the corresponding elements of the second source vector, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
PMUL <Zd>.B, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = PolynomialMult{esize, esize}(element1, element2)[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PMULL

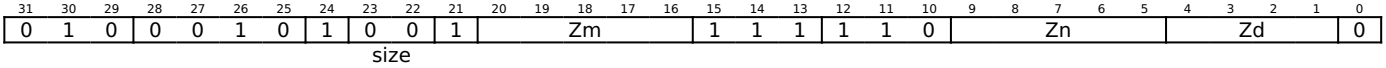
Multi-vector polynomial multiply long

This instruction performs a polynomial multiplication over [0, 1] of the even-numbered elements of the first source vector by the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the first destination vector. The same operation is performed with odd-numbered elements of the source vectors, writing to the second destination vector. This instruction is unpredicated.

This instruction is legal when executed in Streaming SVE mode if both FEAT_SSVE_AES and FEAT_SVE_AES2 are implemented.

SVE2

(FEAT_SVE_AES2)



Encoding

```
PMULL { <Zd1>.Q-<Zd2>.Q }, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_AES2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SSVE_AES) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result_lo : bits(VL);
var result_hi : bits(VL);

for e = 0 to elements-1 do
    let element1_lo : bits(esize DIV 2) = operand1[(2*e + 0)*:(esize DIV 2)];
    let element2_lo : bits(esize DIV 2) = operand2[(2*e + 0)*:(esize DIV 2)];
    let element1_hi : bits(esize DIV 2) = operand1[(2*e + 1)*:(esize DIV 2)];
    let element2_hi : bits(esize DIV 2) = operand2[(2*e + 1)*:(esize DIV 2)];
    result_lo[e*:esize] = PolynomialMult{esize DIV 2, esize DIV 2}(element1_lo, element2_lo);
    result_hi[e*:esize] = PolynomialMult{esize DIV 2, esize DIV 2}(element1_hi, element2_hi);
end;

Z{VL}(d + 0) = result_lo;
Z{VL}(d + 1) = result_hi;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PMULLB

Polynomial multiply long (bottom)

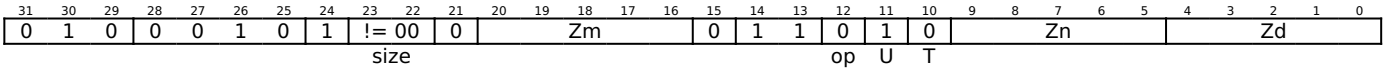
This instruction performs a polynomial multiplication over [0, 1] of the even-numbered elements of the first source vector by the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.
ID_AA64ZFR0_EL1.AES indicates whether the 128-bit element variant is implemented. The 128-bit element variant is legal when executed in Streaming SVE mode if one of the following is true:

- Both FEAT_SSVE_AES and FEAT_SVE_PMULL128 are implemented.
- FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 16-bit or 64-bit elements and 128-bit element

16-bit or 64-bit elements

(FEAT_SVE2 || FEAT_SME)



Encoding

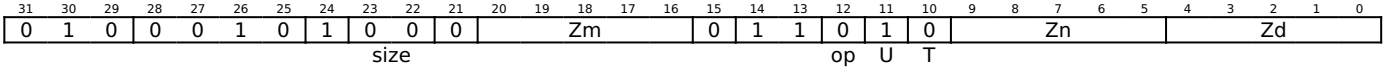
```
PMULLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

128-bit element

(FEAT_SVE_PMULL128)



Encoding

```
PMULLB <Zd>.Q, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_PMULL128) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	RESERVED
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
01	B
10	RESERVED
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if esize == 128 then
    if IsFeatureImplemented(FEAT_SSVE_AES) then
        CheckSVEEnabled();
    else
        CheckNonStreamingSVEEnabled();
    end;
else
    CheckSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize DIV 2) = operand1[(2*e + 0)*:(esize DIV 2)];
    let element2 : bits(esize DIV 2) = operand2[(2*e + 0)*:(esize DIV 2)];
    result[e*:esize] = PolynomialMult{esize DIV 2, esize DIV 2}(element1, element2);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PMULLT

Polynomial multiply long (top)

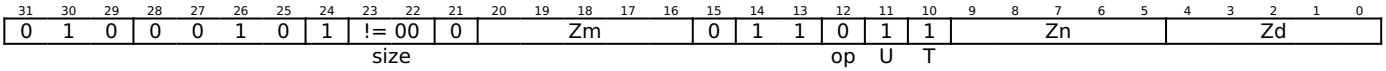
This instruction performs a polynomial multiplication over [0, 1] of the odd-numbered elements of the first source vector by the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.
ID_AA64ZFR0_EL1.AES indicates whether the 128-bit element variant is implemented. The 128-bit element variant is legal when executed in Streaming SVE mode if one of the following is true:

- Both FEAT_SSVE_AES and FEAT_SVE_PMULL128 are implemented.
- FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 16-bit or 64-bit elements and 128-bit element

16-bit or 64-bit elements

(FEAT_SVE2 || FEAT_SME)



Encoding

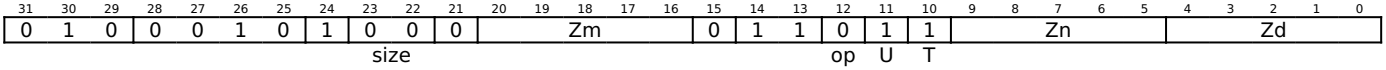
```
PMULLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

128-bit element

(FEAT_SVE_PMULL128)



Encoding

```
PMULLT <Zd>.Q, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_PMULL128) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	RESERVED
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
01	B
10	RESERVED
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if esize == 128 then
    if IsFeatureImplemented(FEAT_SSVE_AES) then
        CheckSVEEnabled();
    else
        CheckNonStreamingSVEEnabled();
    end;
else
    CheckSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize DIV 2) = operand1[(2*e + 1)*:(esize DIV 2)];
    let element2 : bits(esize DIV 2) = operand2[(2*e + 1)*:(esize DIV 2)];
    result[e*:esize] = PolynomialMult{esize DIV 2, esize DIV 2}(element1, element2);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

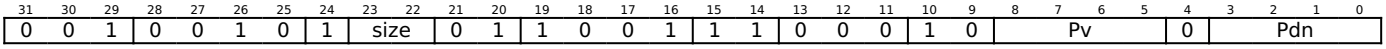
PNEXT

Find next active predicate

This instruction constructs a loop that iterates over all true elements in the vector select predicate register. If all elements in the first source predicate register are false it determines the first true element in the vector select predicate register, otherwise it determines the next true element in the vector select predicate register that follows the last true element in the first source predicate register. All elements of the destination predicate register are set to false, except the element corresponding to the determined vector select element, if any, which is set to true. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PNEXT <Pdn>.<T>, <Pv>, <Pdn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let v : integer = UInt(Pv);
let dn : integer = UInt(Pdn);
```

Assembler Symbols

<Pdn> Is the name of the first source and destination scalable predicate register, encoded in the "Pdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pv> Is the name of the vector select predicate register, encoded in the "Pv" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer{} = VL DIV esize;
let mask : bits(PL) = P{}(v);
let operand : bits(PL) = P{}(dn);
var result : bits(PL) = Zeros{PL};
let psize : integer{} = esize DIV 8;

var next : integer = LastActiveElement{PL}(operand, esize) + 1;

while next < elements && (!ActivePredicateElement{PL}(mask, next, esize)) looplimit elements do
    next = next + 1;
end;

if next < elements then
    result[next*:psize] = ZeroExtend{psize}('1');
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(dn) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFB (scalar plus immediate)

Contiguous prefetch bytes (immediate index)

This instruction prefetches contiguous byte elements from the memory address generated by a 64-bit scalar base and immediate index in the range -32 to 31 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

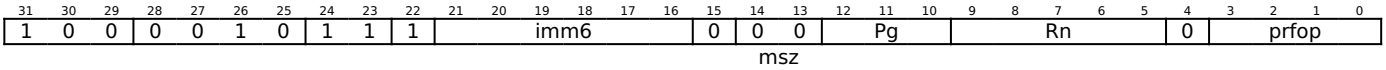
Note

Arm strongly recommends the following for this instruction:

- A PE does not perform a prefetch as a result of Inactive elements.
- Software uses the predicate operand to suppress prefetches from unwanted addresses.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PRFB <prfop>, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 0;
let offset : integer = SInt(imm6);
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -32 to 31, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let eoff : integer = offset * elements + e;
        let addr : bits(64) = base + eoff << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFB (scalar plus scalar)

Contiguous prefetch bytes (scalar index)

This instruction prefetches contiguous byte elements from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element prefetch the index value is incremented, but the index register is not updated.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

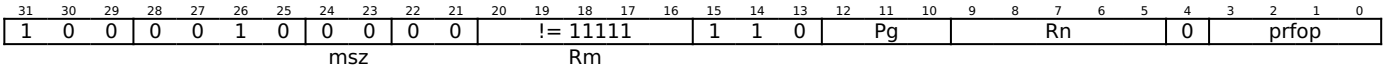
Note

Arm strongly recommends the following for this instruction:

- A PE does not perform a prefetch as a result of Inactive elements.
- Software uses the predicate operand to suppress prefetches from unwanted addresses.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PRFB <prfop>, <Pg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 0;
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(64);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let eoff : integer = UInt(offset) + e;
        let addr : bits(64) = base + eoff << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFB (scalar plus vector)

Gather prefetch bytes (scalar plus vector)

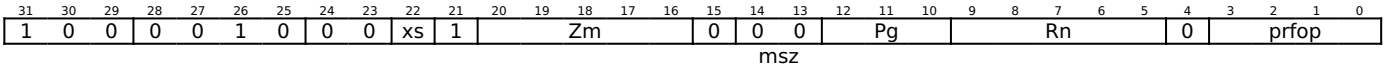
This instruction performs a gather prefetch of bytes from the active memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally sign-extended or zero-extended from 32 to 64 bits. Inactive addresses are not prefetched from memory.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.
It has encodings from 3 classes: 32-bit scaled offset , 32-bit unpacked scaled offset and 64-bit scaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

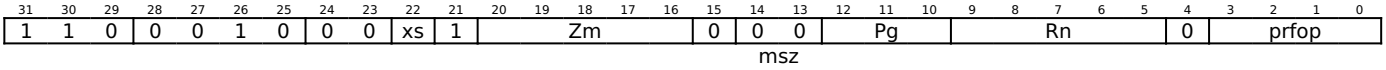
```
PRFB <prfop>, <Pg>, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = (xs == '0');
let scale : integer = 0;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

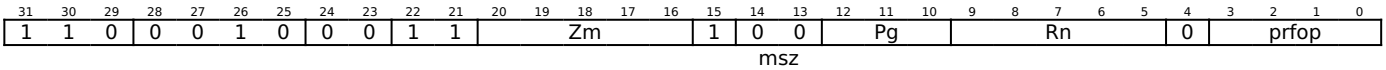
```
PRFB <prfop>, <Pg>, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = (xs == '0');
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

```
PRFB <prfop>, <Pg>, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Zm> Is the name of the offset scalable vector register, encoded in the "Zm" field.

<mod> Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = base + off << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFB (vector plus immediate)

Gather prefetch bytes (vector plus immediate)

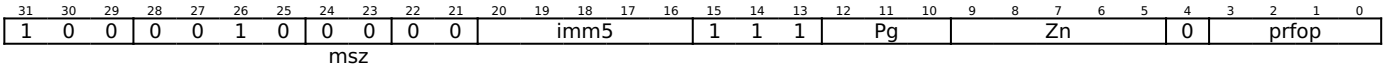
This instruction performs a gather prefetch of bytes from the active memory addresses generated by a vector base plus immediate index. The index is in the range 0 to 31. Inactive addresses are not prefetched from memory.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.
It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

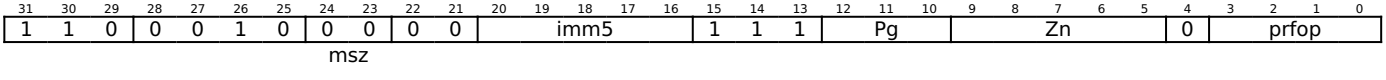
```
PRFB <prfop>, <Pg>, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 0;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
PRFB <prfop>, <Pg>, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 0;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the base scalable vector register, encoded in the "Zn" field.

<imm> Is the optional unsigned immediate byte offset, in the range 0 to 31, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let addr : bits(64) = ZeroExtend{64}(base[e*:esize]) + offset << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFD (scalar plus immediate)

Contiguous prefetch doublewords (immediate index)

This instruction prefetches contiguous doubleword elements from the memory address generated by a 64-bit scalar base and immediate index in the range -32 to 31 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

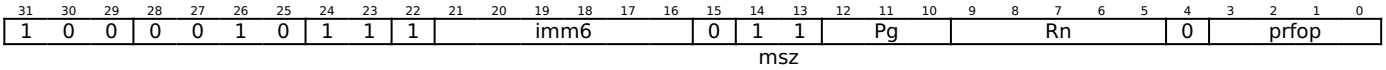
Note

Arm strongly recommends the following for this instruction:

- A PE does not perform a prefetch as a result of Inactive elements.
- Software uses the predicate operand to suppress prefetches from unwanted addresses.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PRFD <prfop>, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 3;
let offset : integer = SInt(imm6);
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -32 to 31, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let eoff : integer = offset * elements + e;
        let addr : bits(64) = base + eoff << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFD (scalar plus scalar)

Contiguous prefetch doublewords (scalar index)

This instruction prefetches contiguous doubleword elements from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 8 and added to the base address. After each element prefetch the index value is incremented, but the index register is not updated.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

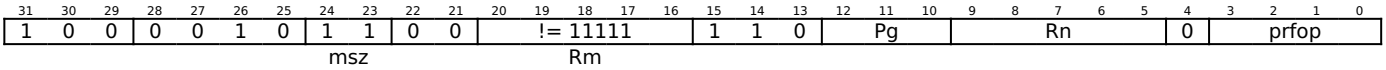
Note

Arm strongly recommends the following for this instruction:

- A PE does not perform a prefetch as a result of Inactive elements.
- Software uses the predicate operand to suppress prefetches from unwanted addresses.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PRFD <prfop>, <Pg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 3;
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(64);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let eoff : integer = UInt(offset) + e;
        let addr : bits(64) = base + eoff << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFD (scalar plus vector)

Gather prefetch doublewords (scalar plus vector)

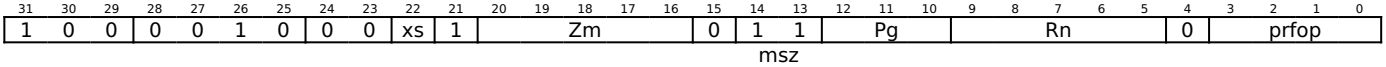
This instruction performs a gather prefetch of doublewords from the active memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then multiplied by 8. Inactive addresses are not prefetched from memory.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.
It has encodings from 3 classes: 32-bit scaled offset , 32-bit unpacked scaled offset and 64-bit scaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

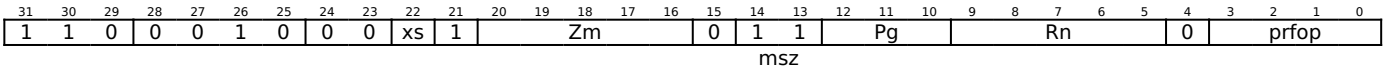
```
PRFD <prfop>, <Pg>, [<Xn|SP>, <Zm>.S, <mod> #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = (xs == '0');
let scale : integer = 3;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

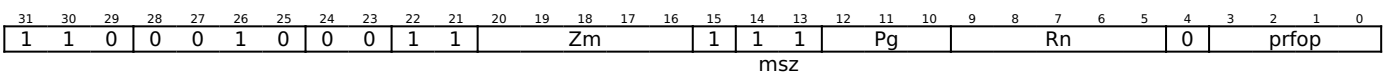
```
PRFD <prfop>, <Pg>, [<Xn|SP>, <Zm>.D, <mod> #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = (xs == '0');
let scale : integer = 3;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

```
PRFD <prfop>, <Pg>, [<Xn|SP>, <Zm>.D, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 3;
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Zm> Is the name of the offset scalable vector register, encoded in the "Zm" field.

<mod> Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = base + off << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFD (vector plus immediate)

Gather prefetch doublewords (vector plus immediate)

This instruction performs a gather prefetch of doublewords from the active memory addresses generated by a vector base plus immediate index. The index is a multiple of 8 in the range 0 to 248. Inactive addresses are not prefetched from memory.

The <prfop> operand specifies the prefetch hint as follows:

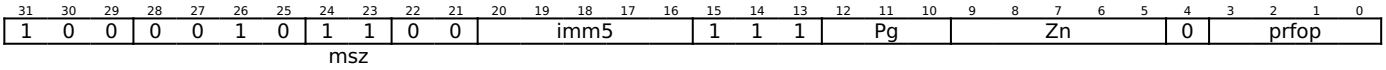
- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

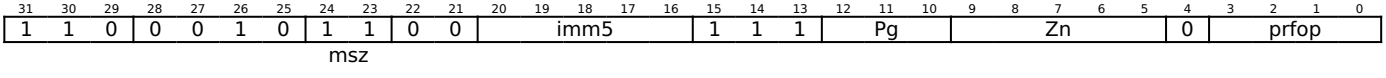
```
PRFD <prfop>, <Pg>, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 3;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
PRFD <prfop>, <Pg>, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 3;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the base scalable vector register, encoded in the "Zn" field.

<imm> Is the optional unsigned immediate byte offset, a multiple of 8 in the range 0 to 248, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let addr : bits(64) = ZeroExtend{64}(base[e*:esize]) + offset << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFH (scalar plus immediate)

Contiguous prefetch halfwords (immediate index)

This instruction prefetches contiguous halfword elements from the memory address generated by a 64-bit scalar base and immediate index in the range -32 to 31 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

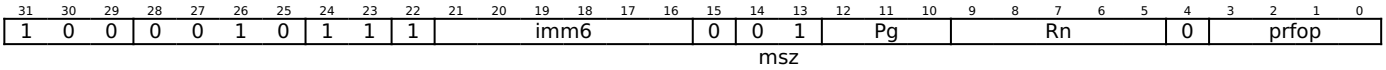
Note

Arm strongly recommends the following for this instruction:

- A PE does not perform a prefetch as a result of Inactive elements.
- Software uses the predicate operand to suppress prefetches from unwanted addresses.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PRFH <prfop>, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 1;
let offset : integer = SInt(imm6);
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -32 to 31, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let eoff : integer = offset * elements + e;
        let addr : bits(64) = base + eoff << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFH (scalar plus scalar)

Contiguous prefetch halfwords (scalar index)

This instruction prefetches contiguous halfword elements from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 2 and added to the base address. After each element prefetch the index value is incremented, but the index register is not updated.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

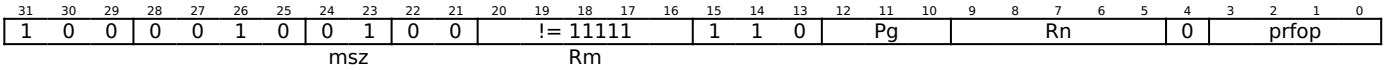
Note

Arm strongly recommends the following for this instruction:

- A PE does not perform a prefetch as a result of Inactive elements.
- Software uses the predicate operand to suppress prefetches from unwanted addresses.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PRFH <prfop>, <Pg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 1;
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(64);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let eoff : integer = UInt(offset) + e;
        let addr : bits(64) = base + eoff << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFH (scalar plus vector)

Gather prefetch halfwords (scalar plus vector)

This instruction performs a gather prefetch of halfwords from the active memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then multiplied by 2. Inactive addresses are not prefetched from memory.

The <prfop> operand specifies the prefetch hint as follows:

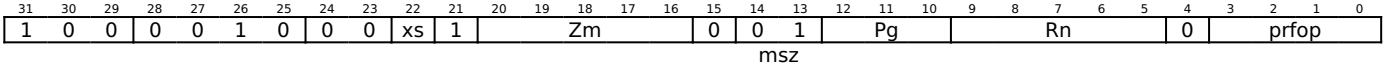
- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 32-bit scaled offset , 32-bit unpacked scaled offset and 64-bit scaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

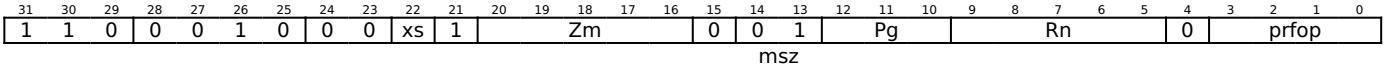
```
PRFH <prfop>, <Pg>, [<Xn|SP>, <Zm>.S, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = (xs == '0');
let scale : integer = 1;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

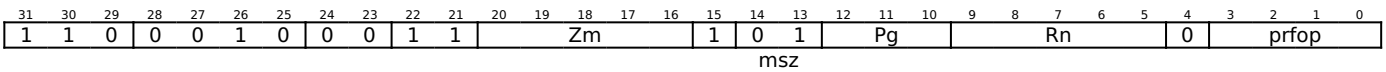
```
PRFH <prfop>, <Pg>, [<Xn|SP>, <Zm>.D, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = (xs == '0');
let scale : integer = 1;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

```
PRFH <prfop>, <Pg>, [<Xn|SP>, <Zm>.D, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 1;
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Zm> Is the name of the offset scalable vector register, encoded in the "Zm" field.

<mod> Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = base + off << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFH (vector plus immediate)

Gather prefetch halfwords (vector plus immediate)

This instruction performs a gather prefetch of halfwords from the active memory addresses generated by a vector base plus immediate index. The index is a multiple of 2 in the range 0 to 62. Inactive addresses are not prefetched from memory.

The <prfop> operand specifies the prefetch hint as follows:

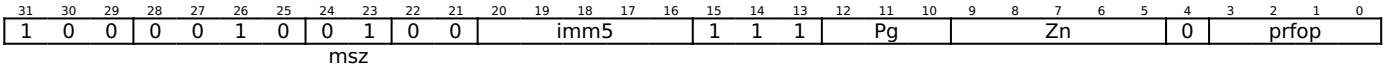
- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

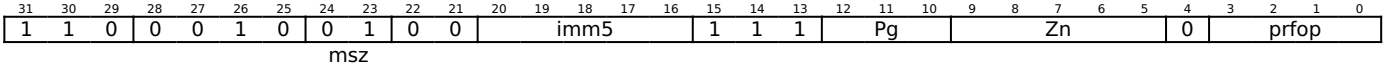
```
PRFH <prfop>, <Pg>, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 1;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
PRFH <prfop>, <Pg>, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 1;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the base scalable vector register, encoded in the "Zn" field.

<imm> Is the optional unsigned immediate byte offset, a multiple of 2 in the range 0 to 62, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let addr : bits(64) = ZeroExtend{64}(base[e*:esize]) + offset << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFW (scalar plus immediate)

Contiguous prefetch words (immediate index)

This instruction prefetches contiguous word elements from the memory address generated by a 64-bit scalar base and immediate index in the range -32 to 31 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

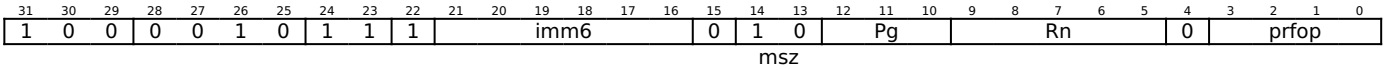
Note

Arm strongly recommends the following for this instruction:

- A PE does not perform a prefetch as a result of Inactive elements.
- Software uses the predicate operand to suppress prefetches from unwanted addresses.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PRFW <prfop>, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 2;
let offset : integer = SInt(imm6);
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -32 to 31, defaulting to 0, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let eoff : integer = offset * elements + e;
        let addr : bits(64) = base + eoff << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFW (scalar plus scalar)

Contiguous prefetch words (scalar index)

This instruction prefetches contiguous word elements from the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 4 and added to the base address. After each element prefetch the index value is incremented, but the index register is not updated.
The <prfop> operand specifies the prefetch hint as follows:

- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

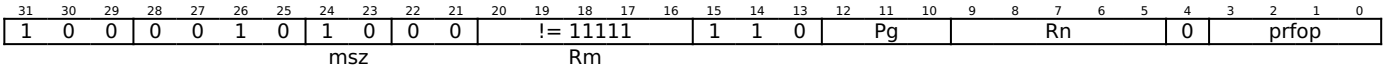
Note

Arm strongly recommends the following for this instruction:

- A PE does not perform a prefetch as a result of Inactive elements.
- Software uses the predicate operand to suppress prefetches from unwanted addresses.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PRFW <prfop>, <Pg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 2;
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(64);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = X{64}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let eoff : integer = UInt(offset) + e;
        let addr : bits(64) = base + eoff << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFW (scalar plus vector)

Gather prefetch words (scalar plus vector)

This instruction performs a gather prefetch of words from the active memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then multiplied by 4. Inactive addresses are not prefetched from memory.

The <prfop> operand specifies the prefetch hint as follows:

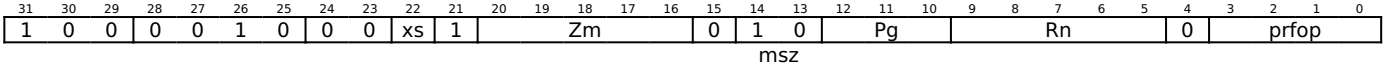
- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 32-bit scaled offset , 32-bit unpacked scaled offset and 64-bit scaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

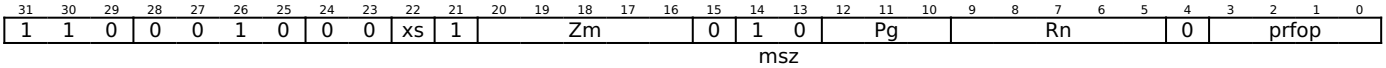
```
PRFW <prfop>, <Pg>, [<Xn|SP>, <Zm>.S, <mod> #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = (xs == '0');
let scale : integer = 2;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

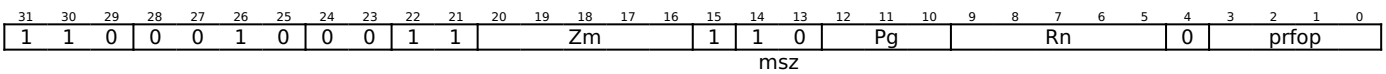
```
PRFW <prfop>, <Pg>, [<Xn|SP>, <Zm>.D, <mod> #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = (xs == '0');
let scale : integer = 2;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

```
PRFW <prfop>, <Pg>, [<Xn|SP>, <Zm>.D, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 2;
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Zm> Is the name of the offset scalable vector register, encoded in the "Zm" field.

<mod> Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(64);
var offset : bits(VL);

if AnyActiveElement{PL}(mask, esize) then
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = base + off << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PRFW (vector plus immediate)

Gather prefetch words (vector plus immediate)

This instruction performs a gather prefetch of words from the active memory addresses generated by a vector base plus immediate index. The index is a multiple of 4 in the range 0 to 124. Inactive addresses are not prefetched from memory.

The <prfop> operand specifies the prefetch hint as follows:

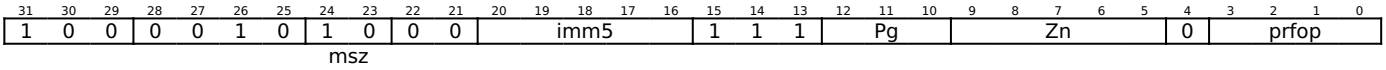
- Access type:
  - PLD for prefetch for load.
  - PST for prefetch for store.
- Target cache level:
  - L1 for Level 1 cache.
  - L2 for Level 2 cache.
  - L3 for Level 3 cache.
- Policy:
  - KEEP for retained or temporal prefetch, allocated in the cache normally.
  - STRM for streaming or non-temporal prefetch, for data that is used only once.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

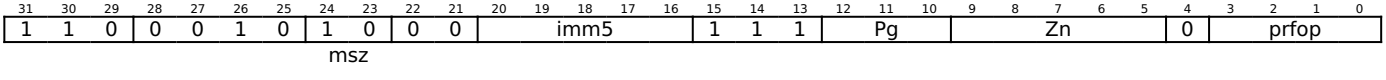
```
PRFW <prfop>, <Pg>, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 2;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
PRFW <prfop>, <Pg>, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let level : integer = UInt(prfop[2:1]);
let stream : boolean = (prfop[0] == '1');
let pref_hint : PrefetchHint = if prfop[3] == '0' then Prefetch_READ else Prefetch_WRITE;
let scale : integer = 2;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<prfop> Is the prefetch operation specifier, encoded in “prfop”:

prfop	<prfop>
0000	PLDL1KEEP
0001	PLDL1STRM
0010	PLDL2KEEP
0011	PLDL2STRM
0100	PLDL3KEEP
0101	PLDL3STRM
x11x	#uimm4
1000	PSTL1KEEP
1001	PSTL1STRM
1010	PSTL2KEEP
1011	PSTL2STRM
1100	PSTL3KEEP
1101	PSTL3STRM

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the base scalable vector register, encoded in the "Zn" field.

<imm> Is the optional unsigned immediate byte offset, a multiple of 4 in the range 0 to 124, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let addr : bits(64) = ZeroExtend{64}(base[e*:esize]) + offset << scale;
        Hint_Prefetch(addr, pref_hint, level, stream);
    end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

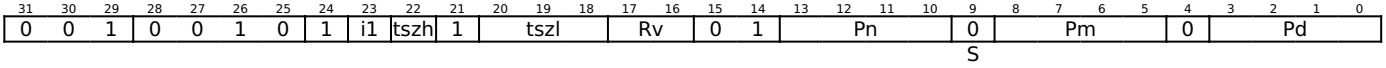
PSEL

Predicate select between predicate register or all-false

This instruction places the contents of the first source predicate register into the destination predicate register if the indexed element of the second source predicate is true, otherwise it sets the destination predicate to all-false. The indexed element is determined by the sum of a general-purpose index register and an immediate, modulo the number of elements. This instruction does not set the condition flags. For programmer convenience, an assembler must also accept predicate-as-counter register names for the destination predicate register and the first source predicate register.

SVE2

(FEAT_SME || FEAT_SVE2p1)



Encoding

```
PSEL <Pd>, <Pn>, <Pm>.<T>[<Wv>, <imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let imm5 : bits(5) = i1::tszh::tszl;
var esize : integer;
var imm : integer;
case tszh::tszl of
    when '0000' => EndOfDecode(Decode_UNDEF);
    when '1000' => esize = 64;  imm = UInt(imm5[4]);
    when 'x100' => esize = 32;  imm = UInt(imm5[4:3]);
    when 'xx10' => esize = 16;  imm = UInt(imm5[4:2]);
    when 'xxx1' => esize = 8;   imm = UInt(imm5[4:1]);
end;
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let v : integer = UInt('011'::Rv);
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<Pn> Is the name of the first source scalable predicate register, encoded in the "Pn" field.

<Pm> Is the name of the second source scalable predicate register, encoded in the "Pm" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	000	RESERVED
x	xx1	B
x	x10	H
x	100	S
1	000	D

<Wv> Is the 32-bit name of the vector select register W12-W15, encoded as "Rv" plus 12.

<imm> Is the element index, in the range 0 to one less than the number of vector elements in a 128-bit vector register, encoded in "i1:tszh:tszl".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
let idx : bits(32) = X{}(v);
let element : integer = (UInt(idx) + imm) MOD elements;
var result : bits(PL);

if ActivePredicateElement{PL}(operand2, element, esize) then
    result = operand1;
else
    result = Zeros{PL};
end;

P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

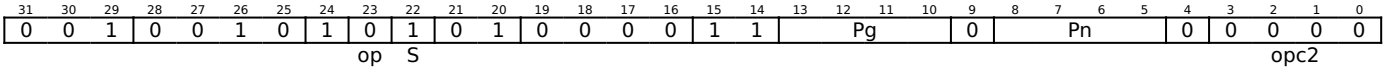
PTEST

Set condition flags for predicate

This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate source register, and sets the V flag to zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
PTEST <Pg>, <Pn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
```

Assembler Symbols

<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let result : bits(PL) = P{}(n);

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the predicate register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

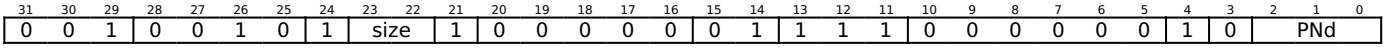
PTRUE (predicate as counter)

Initialize predicate-as-counter to all active

This instruction sets the destination predicate as all Active elements, using the predicate-as-counter encoding.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
PTRUE <PNd>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt('1'::PNd);
```

Assembler Symbols

<PNd> Is the name of the destination scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNd" plus 8.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then CheckSVEEnabled(); else CheckStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let result : bits(PL) = EncodePredCount{}(esize, elements, elements, FALSE);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PTRUE (predicate)

Initialize predicate from named constraint

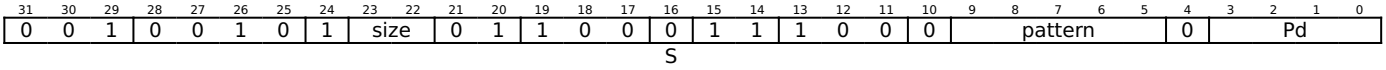
This instruction sets elements of the destination predicate to true if the element number satisfies the named predicate constraint, or to false otherwise. If the constraint specifies more elements than are available at the current vector length, then all elements of the destination predicate are set to false.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
This instruction does not set the condition flags.

Not setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
PTRUE <Pd>.<T>{, <pattern>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
let pat : bits(5) = pattern;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let pbit : bit = if e < count then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(result, result, esize);
end;
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PTRUES

Initialize predicate from named constraint and set the condition flags

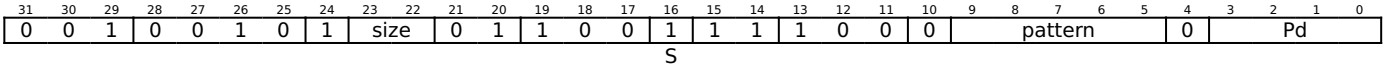
This instruction sets elements of the destination predicate to true if the element number satisfies the named predicate constraint, or to false otherwise. If the constraint specifies more elements than are available at the current vector length, then all elements of the destination predicate are set to false.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

Setting the condition flags

(FEAT_SVE || FEAT_SME)



Encoding

```
PTRUES <Pd>.<T>{, <pattern>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
let pat : bits(5) = pattern;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let pbit : bit = if e < count then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
end;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(result, result, esize);
end;
P{PL}(d) = result;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the NZCV condition flags written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

PUNPKHI, PUNPKLO

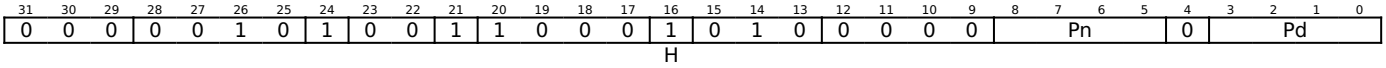
Unpack and widen half of predicate

This instruction unpacks elements from the lowest or highest half of the source predicate and places them in elements of twice their size within the destination predicate. This instruction is unpredicated.

It has encodings from 2 classes: High half and Low half

High half

(FEAT_SVE || FEAT_SME)



Encoding

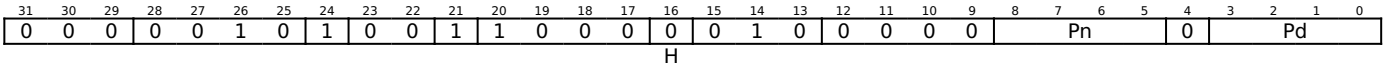
```
PUNPKHI <Pd>.H, <Pn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Pn);
let d : integer = UInt(Pd);
let hi : boolean = TRUE;
```

Low half

(FEAT_SVE || FEAT_SME)



Encoding

```
PUNPKLO <Pd>.H, <Pn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Pn);
let d : integer = UInt(Pd);
let hi : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand : bits(PL) = P{}(n);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let pbit : bit = PredicateElement{PL}(operand, if hi then e + elements else e, esize DIV 2);
    result[e*:psize] = ZeroExtend{psize}(pbit);
end;

P{PL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

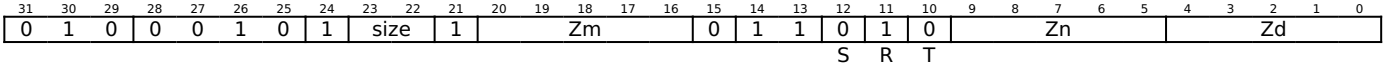
RADDHNB

Rounding add narrow high part (bottom)

This instruction adds each vector element of the first source vector to the corresponding vector element of the second source vector, and places the most significant rounded half of the result in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
RADDHNB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	B
10	H
11	S

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    let res : integer = ((element1 + element2) + (1 << (halfesize - 1))) >> halfesize;
    result[(2*e + 0)*:halfesize] = res[halfesize-1:0];
    result[(2*e + 1)*:halfesize] = Zeros{halfesize};

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

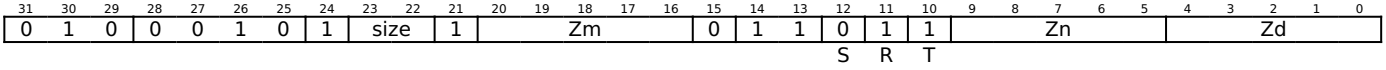
RADDHNT

Rounding add narrow high part (top)

This instruction adds each vector element of the first source vector to the corresponding vector element of the second source vector, and places the most significant rounded half of the result in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
RADDHNT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	B
10	H
11	S

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(d);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    let res : integer = ((element1 + element2) + (1 << (halfesize - 1))) >> halfesize;
    result[(2*e + 1)*:halfesize] = res[halfesize-1:0];

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RAX1

Bitwise rotate left by 1 and exclusive-OR

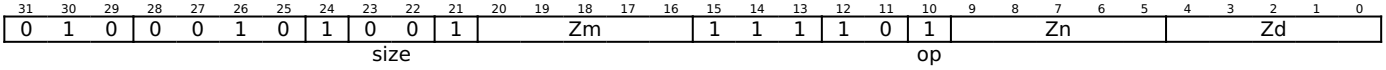
This instruction rotates each 64-bit element of the second source vector left by 1 and performs an exclusive-OR with the corresponding elements of the first source vector. The results are placed in the corresponding elements of the destination vector. This instruction is unpredicated.

ID_AA64ZFR0_EL1.SHA3 indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled, or FEAT_SME2p1 is implemented.

SVE2

(FEAT_SVE_SHA3)



Encoding

```
RAX1 <Zd>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_SHA3) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if IsFeatureImplemented(FEAT_SME2p1) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 64;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(64) = operand1[e*:64];
    let element2 : bits(64) = operand2[e*:64];
    result[e*:64] = element1 XOR ROL(element2, 1);
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RBIT

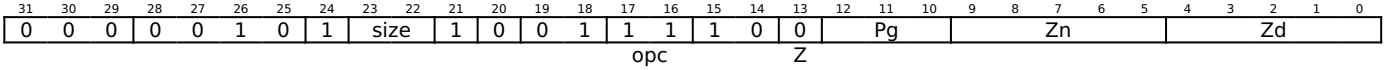
Reverse bits (predicated)

This instruction reverses the bits in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE || FEAT_SME)



Encoding

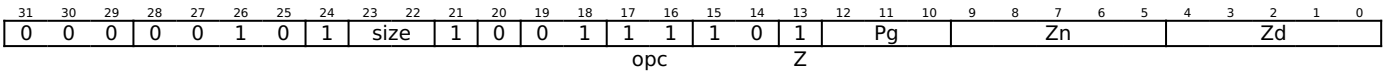
```
RBIT <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
RBIT <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = BitReverse{esize}(element);

    end;
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RDFFR (predicated)

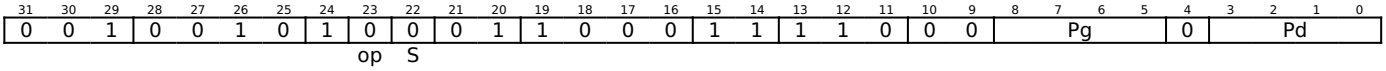
Return predicate of successfully loaded elements

This instruction reads the first-fault register (FFR) and places Active elements in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction does not set the condition flags.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

Not setting the condition flags

(FEAT_SVE)



Encoding

```
RDFFR <Pd>.B, <Pg>/Z
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let d : integer = UInt(Pd);
let setflags : boolean = FALSE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let ffr : bits(PL) = FFR{}();
let result : bits(PL) = ffr AND mask;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, 8);
end;
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

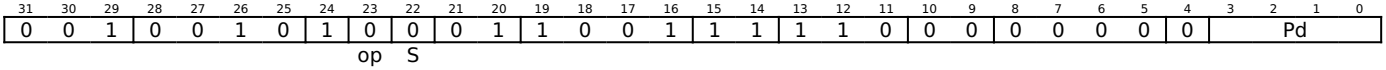
RDFFR (unpredicated)

Read the first-fault register

This instruction reads the first-fault register (FFR) and places the read values in the destination predicate without predication.
This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
RDFFR <Pd>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Pd);
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let ffr : bits(PL) = FFR{}();
P{PL}(d) = ffr;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RDFFRS

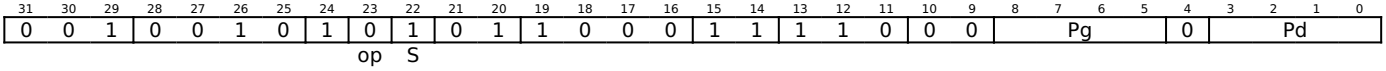
Return predicate of successfully loaded elements, setting the condition flags

This instruction reads the first-fault register (FFR) and places Active elements in the corresponding elements of the destination predicate. Inactive elements in the destination predicate register are set to zero. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

Setting the condition flags

(FEAT_SVE)



Encoding

```
RDFFRS <Pd>.B, <Pg>/Z
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let d : integer = UInt(Pd);
let setflags : boolean = TRUE;
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let mask : bits(PL) = P{}(g);
let ffr : bits(PL) = FFR{}();
let result : bits(PL) = ffr AND mask;

if setflags then
    PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, 8);
end;
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

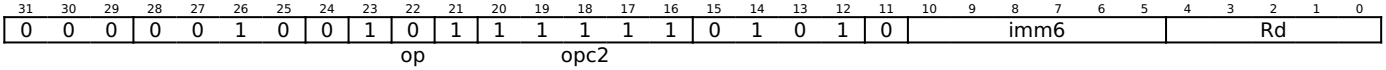
RDVL

Read multiple of vector register size to scalar register

This instruction multiplies the current vector register size in bytes by an immediate in the range -32 to 31 and places the result in the 64-bit destination general-purpose register.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
RDVL <Xd>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let d : integer = UInt(Rd);
let imm : integer = SInt(imm6);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the destination general-purpose register, encoded in the "Rd" field.
<imm>	Is the signed immediate operand, in the range -32 to 31, encoded in the "imm6" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let len : integer = imm * (VL DIV 8);
X{64}(d) = len[63:0];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

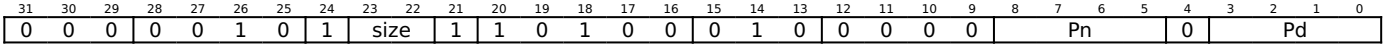
REV (predicate)

Reverse all elements in a predicate

This instruction reads all elements in the source predicate, reverses the order of all the elements, and places the result in the destination predicate. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
REV <Pd>.<T>, <Pn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Pn);
let d : integer = UInt(Pd);
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pn> Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let operand : bits(PL) = P{}(n);
let result : bits(PL) = Reverse{}(operand, esize DIV 8);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

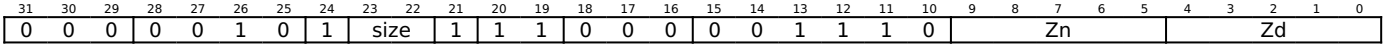
REV (vector)

Reverse all elements in a vector (unpredicated)

This instruction reads all elements in the source vector, reverses the order of all the elements, and places the result in the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
REV <Zd>.<T>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let operand : bits(VL) = Z{}(n);
let result : bits(VL) = Reverse{}(operand, esize);
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

REVB, REVH, REVW

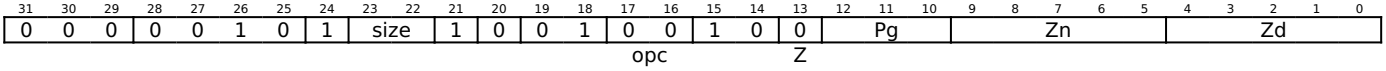
Reverse bytes / halfwords / words within elements (predicated)

This instruction reverses the order of 8-bit bytes, 16-bit halfwords or 32-bit words within each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 6 classes: Byte, merging , Byte, zeroing , Halfword, merging , Halfword, zeroing , Word, merging and Word, zeroing

Byte, merging

(FEAT_SVE || FEAT_SME)



Encoding

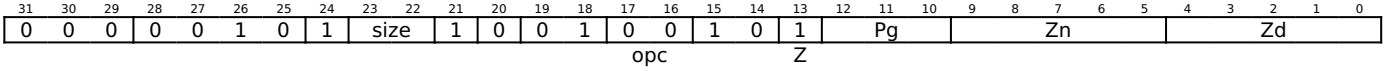
```
REVB <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let swsize : integer = 8;
let merging : boolean = TRUE;
```

Byte, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

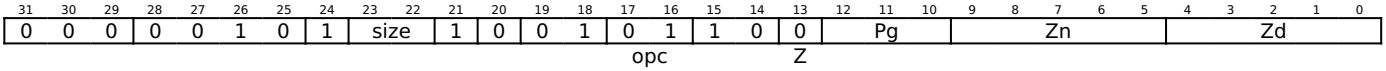
```
REVB <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let swsize : integer = 8;
let merging : boolean = FALSE;
```

Halfword, merging

(FEAT_SVE || FEAT_SME)



Encoding

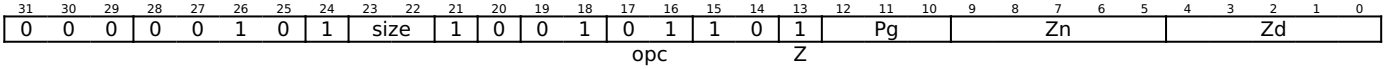
```
REVH <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let swsize : integer = 16;
let merging : boolean = TRUE;
```

Halfword, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

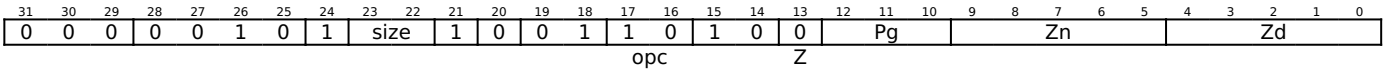
```
REVH <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let swsize : integer = 16;
let merging : boolean = FALSE;
```

Word, merging

(FEAT_SVE || FEAT_SME)



Encoding

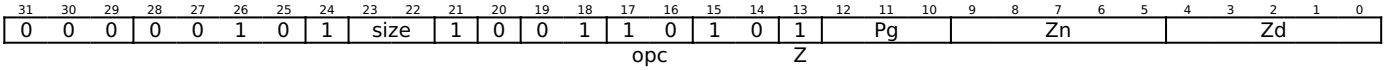
```
REVW <Zd>.D, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let swsize : integer = 32;
let merging : boolean = TRUE;
```

Word, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
REVW <Zd>.D, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let swsize : integer = 32;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> For the "Byte, merging" and "Byte, zeroing" variants: is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

For the "Halfword, merging" and "Halfword, zeroing" variants: is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = Reverse{esize}(element, swsize);
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

REVD

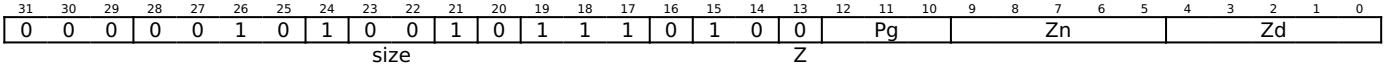
Reverse 64-bit doublewords in elements (predicated)

This instruction reverses the order of 64-bit doublewords within each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SME || FEAT_SVE2p1)



Encoding

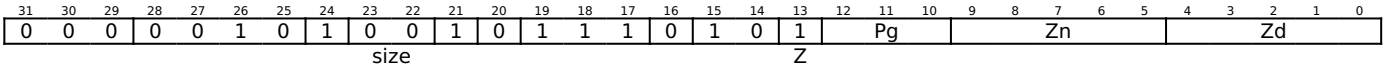
```
REVD <Zd>.Q, <Pg>/M, <Zn>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 128;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let swsize : integer = 64;
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
REVD <Zd>.Q, <Pg>/Z, <Zn>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 128;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let swsize : integer = 64;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = Reverse{esize}(element, swsize);
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

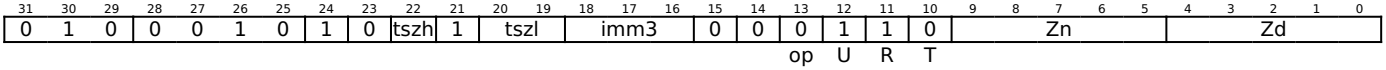
RSHRNB

Rounding shift right narrow by immediate (bottom)

This instruction shifts each unsigned integer value in the source vector elements right by an immediate value, and places the rounded results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
RSHRNB <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = (UInt(element) + (1 << (shift-1))) >> shift;
    result[(2*e + 0)*:esize] = res[esize-1:0];
    result[(2*e + 1)*:esize] = Zeros{esize};

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

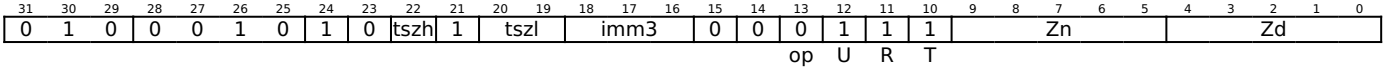
RSHRNT

Rounding shift right narrow by immediate (top)

This instruction shifts each unsigned integer value in the source vector elements right by an immediate value, and places the rounded results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
RSHRNT <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = (UInt(element) + (1 << (shift-1))) >> shift;
    result[(2*e + 1)*:esize] = res[esize-1:0];

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

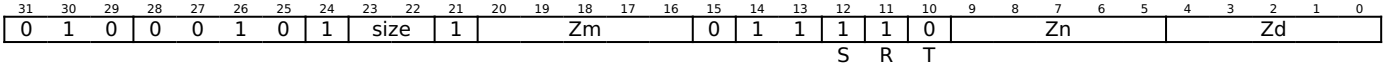
RSUBHNB

Rounding subtract narrow high part (bottom)

This instruction subtracts each vector element of the second source vector from the corresponding vector element in the first source vector, and places the most significant rounded half of the result in the even-numbered half-width destination elements, while setting the odd-numbered half-width destination elements to zero. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
RSUBHNB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	B
10	H
11	S

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    let res : integer = ((element1 - element2) + (1 << (halfesize - 1))) >> halfesize;
    result[(2*e + 0)*:halfesize] = res[halfesize-1:0];
    result[(2*e + 1)*:halfesize] = Zeros{halfesize};

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

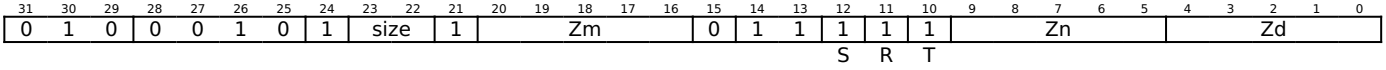
RSUBHNT

Rounding subtract narrow high part (top)

This instruction subtracts each vector element of the second source vector from the corresponding vector element in the first source vector, and places the most significant rounded half of the result in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
RSUBHNT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	B
10	H
11	S

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(d);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    let res : integer = ((element1 - element2) + (1 << (halfesize - 1))) >> halfesize;
    result[(2*e + 1)*:halfesize] = res[halfesize-1:0];

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

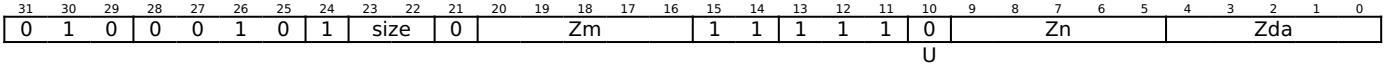
SABA

Signed absolute difference and accumulate

This instruction calculates the absolute difference between signed integer values in elements of the first source vector and the corresponding elements of the second source vector, and adds the difference to the corresponding elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SABA <Zda>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let op2elt : bits(esize) = operand2[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let absdiff : bits(esize) = Abs(element1 - element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + absdiff;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

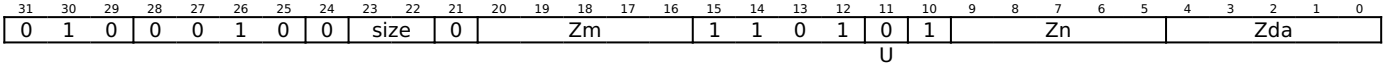
SABAL

Two-way signed absolute difference sum and accumulate long

This instruction calculates the sum of the absolute difference between even-numbered signed elements and the absolute difference between the odd-numbered signed elements of each double-width container of the first and second vectors, and destructively adds the results to the overlapping double-width elements of the addend vector.
This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
SABAL <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let elt1_a : integer = SInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let elt1_b : integer = SInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let elt2_a : integer = SInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let elt2_b : integer = SInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let sum = Abs(elt1_a - elt2_a) + Abs(elt1_b - elt2_b);
    result[e*:esize] = result[e*:esize] + sum[esize-1:0];
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

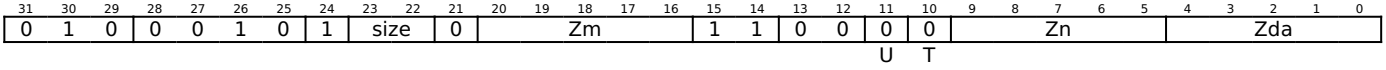
SABALB

Signed absolute difference and accumulate long (bottom)

This instruction calculates the absolute difference between even-numbered signed integer values in elements of the first source vector and the corresponding elements of the second source vector, and destructively adds the results to the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SABALB <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let absdiff = Abs(element1 - element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + absdiff;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

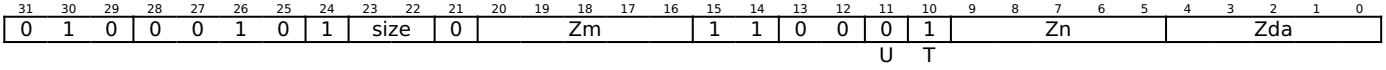
SABALT

Signed absolute difference and accumulate long (top)

This instruction calculates the absolute difference between odd-numbered signed elements of the first source vector and the corresponding elements of the second source vector, and destructively adds the results to the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SABALT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let absdiff = Abs(element1 - element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + absdiff;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

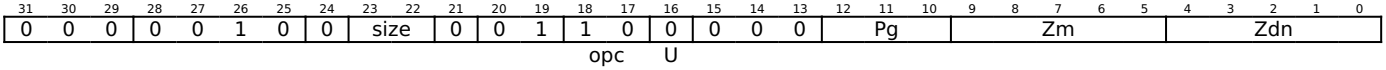
SABD

Signed absolute difference (predicated)

This instruction calculates the absolute difference between signed integer values in Active elements of the first source vector and the corresponding elements of the second source vector, and destructively places the difference in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SABD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let absdiff : integer = Abs(element1 - element2);
        result[e*:esize] = absdiff[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

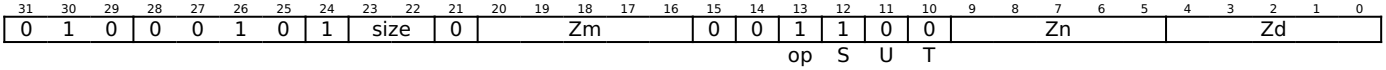
SABDLB

Signed absolute difference long (bottom)

This instruction calculates the absolute difference between even-numbered signed integer values in elements of the first source vector and the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SABDLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let res : integer = Abs(element1 - element2);
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

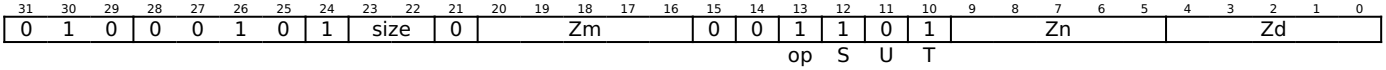
SABDLT

Signed absolute difference long (top)

This instruction calculates the absolute difference between odd-numbered signed integer values in elements of the first source vector and the corresponding elements of the second source vector, and places the results in overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SABDLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let res : integer = Abs(element1 - element2);
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

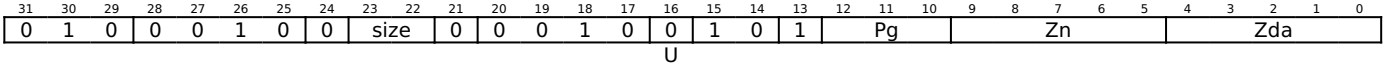
SADALP

Signed add and accumulate long pairwise

This instruction adds pairs of adjacent signed integer values and accumulates the results into the overlapping double-width elements of the destination vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SADALP <Zda>.<T>, <Pg>/M, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the second source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand_acc : bits(VL) = Z{}(da);
let operand_src : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if !ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand_acc[e*:esize];
    else
        let element1 : integer = SInt(operand_src[(2*e + 0)*:(esize DIV 2)]);
        let element2 : integer = SInt(operand_src[(2*e + 1)*:(esize DIV 2)]);
        let sum : bits(esize) = (element1 + element2)[esize-1:0];
        result[e*:esize] = operand_acc[e*:esize] + sum;
    end;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

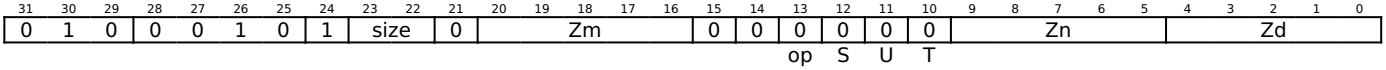
SADDLB

Signed add long (bottom)

This instruction adds the corresponding even-numbered signed elements of the first and second source vectors, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SADDLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 0;
let sel2 : integer = 0;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 + element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

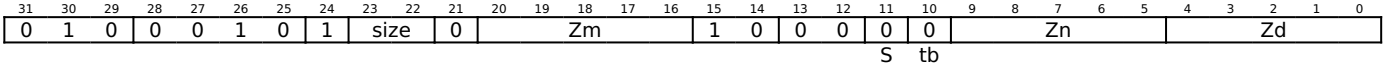
SADDLBT

Signed add long (bottom + top)

This instruction adds the even-numbered signed elements of the first source vector to the odd-numbered signed elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SADDLBT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 0;
let sel2 : integer = 1;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 + element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

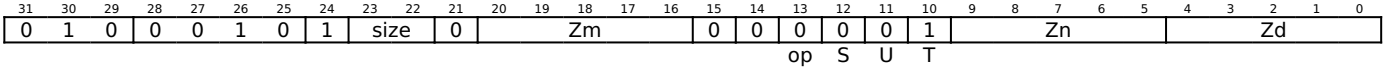
SADDLT

Signed add long (top)

This instruction adds the corresponding odd-numbered signed elements of the first and second source vectors, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SADDLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 1;
let sel2 : integer = 1;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 + element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

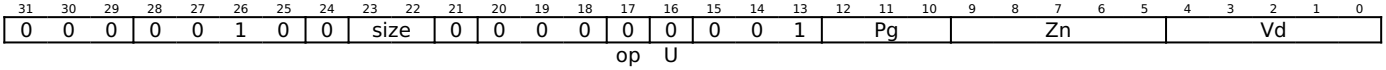
SADDV

Signed add reduction to scalar

This instruction adds all the signed elements of the source vector, and places the result in the SIMD&FP scalar destination register. Narrow elements are first sign-extended to 64 bits. Inactive elements in the source vector are treated as zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SADDV <Dd>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Dd> Is the 64-bit name of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = Z{}(n);
var sum : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = SInt(operand[e*:esize]);
        sum = sum + element;
    end;
end;

V{64}(d) = sum[63:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

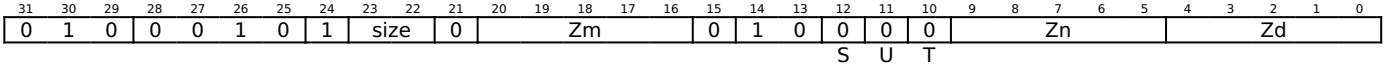
SADDWB

Signed add wide (bottom)

This instruction adds the even-numbered signed elements of the second source vector to the overlapping double-width elements of the first source vector and places the results in the corresponding double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SADDWB <Zd>.<T>, <Zn>.<T>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    result[e*:esize] = (element1 + element2)[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

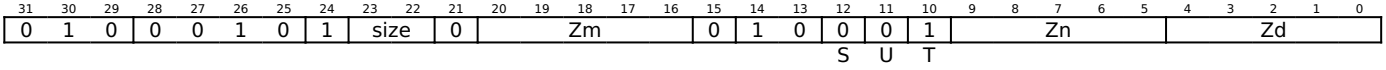
SADDWT

Signed add wide (top)

This instruction adds the odd-numbered signed elements of the second source vector to the overlapping double-width elements of the first source vector and places the results in the corresponding double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SADDWT <Zd>.<T>, <Zn>.<T>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    result[e*:esize] = (element1 + element2)[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

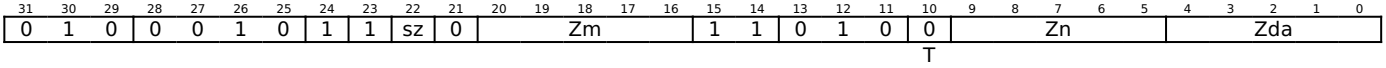
SBCLB

Subtract with carry long (bottom)

This instruction subtracts the even-numbered elements of the first source vector and the inverted 1-bit carry from the least-significant bit of the odd-numbered elements of the second source vector from the even-numbered elements of the destination and accumulator vector. The 1-bit carry output is placed in the corresponding odd-numbered element of the destination vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SBCLB <Zda>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (esize * 2);
let operand : bits(VL) = Z{}(n);
let carries : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for p = 0 to pairs-1 do
    let element1 : bits(esize) = result[(2*p + 0)*:esize];
    let element2 : bits(esize) = operand[(2*p + 0)*:esize];
    let carry_in : bit = carries[(2*p + 1)*:esize][0];

    let (res, nzcv) = AddWithCarry{esize}(element1, NOT(element2), carry_in);
    let carry_out : bit = nzcv[1];

    result[(2*p + 0)*:esize] = res;
    result[(2*p + 1)*:esize] = ZeroExtend{esize}(carry_out);
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

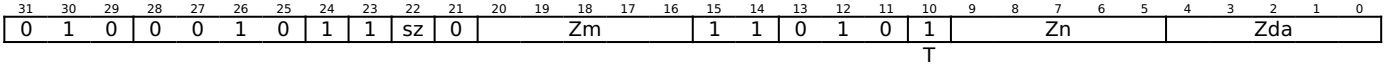
SBCLT

Subtract with carry long (top)

This instruction subtracts the odd-numbered elements of the first source vector and the inverted 1-bit carry from the least-significant bit of the odd-numbered elements of the second source vector from the even-numbered elements of the destination and accumulator vector. The 1-bit carry output is placed in the corresponding odd-numbered element of the destination vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SBCLT <Zda>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (esize * 2);
let operand : bits(VL) = Z{}(n);
let carries : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for p = 0 to pairs-1 do
    let element1 : bits(esize) = result[(2*p + 0)*:esize];
    let element2 : bits(esize) = operand[(2*p + 1)*:esize];
    let carry_in : bit = carries[(2*p + 1)*:esize][0];

    let (res, nzcv) = AddWithCarry{esize}(element1, NOT(element2), carry_in);
    let carry_out : bit = nzcv[1];

    result[(2*p + 0)*:esize] = res;
    result[(2*p + 1)*:esize] = ZeroExtend{esize}(carry_out);
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

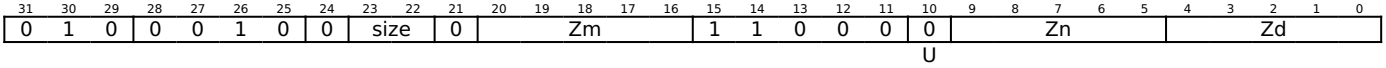
SCLAMP

Signed clamp to minimum/maximum

This instruction clamps each signed element in the destination vector to between the signed minimum value in the corresponding element of the first source vector and the signed maximum value in the corresponding element of the second source vector and destructively writes the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SME || FEAT_SVE2p1)



Encoding

```
SCLAMP <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(d);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    let element3 : integer = SInt(operand3[e*:esize]);
    let res : integer = Min(Max(element1, element3), element2);
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SCVTF (bottom, unpredicated)

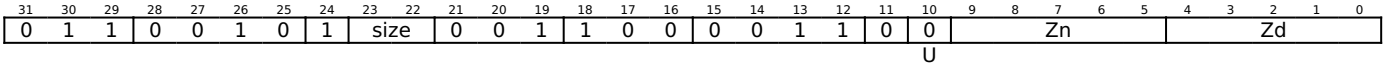
Signed integer convert to floating-point (bottom, unpredicated)

This instruction converts each even-numbered signed integer element of the source vector to floating-point, and places the results in the overlapping double-width elements of the destination vector.

This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
SCVTF <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize DIV 2) = operand[(2*e + 0)*:(esize DIV 2)];
    result[e*:esize] = FixedToFP{esize, esize DIV 2}(element, 0, unsigned, FPCR(), rounding);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SCVTF (predicated)

Signed integer convert to floating-point (predicated)

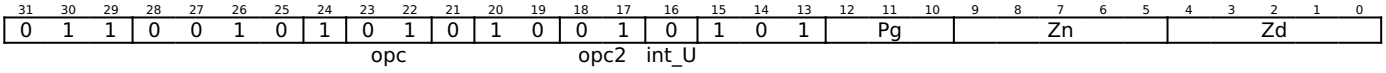
This instruction converts to floating-point from the signed integer in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

If the input and result types have a different size the smaller type is held unpacked in the least significant bits of elements of the larger size. When the input is the smaller type the upper bits of each source element are ignored. When the result is the smaller type the results are zero-extended to fill each destination element.

It has encodings from 14 classes: 16-bit to half-precision, merging , 16-bit to half-precision, zeroing , 32-bit to half-precision, merging , 32-bit to half-precision, zeroing , 32-bit to single-precision, merging , 32-bit to single-precision, zeroing , 32-bit to double-precision, merging , 32-bit to double-precision, zeroing , 64-bit to half-precision, merging , 64-bit to half-precision, zeroing , 64-bit to single-precision, merging , 64-bit to single-precision, zeroing , 64-bit to double-precision, merging and 64-bit to double-precision, zeroing

16-bit to half-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

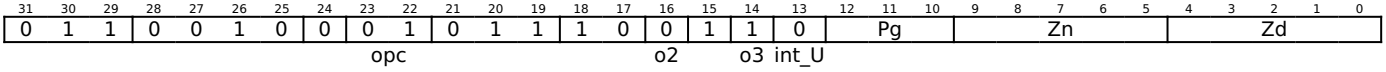
```
SCVTF <Zd>.H, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 16;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

16-bit to half-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

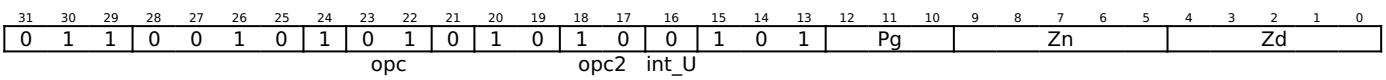
```
SCVTF <Zd>.H, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 16;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

32-bit to half-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

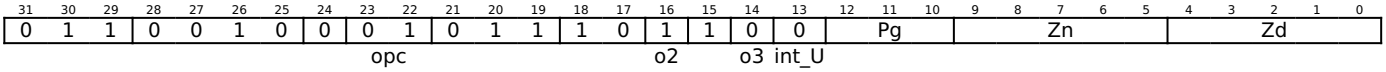
```
SCVTF <Zd>.H, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 16;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

32-bit to half-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

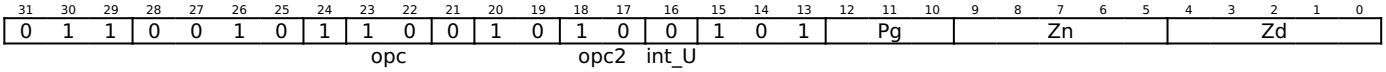
```
SCVTF <Zd>.H, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 16;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

32-bit to single-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

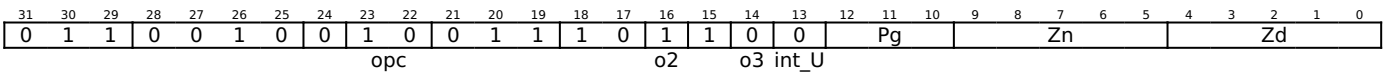
```
SCVTF <Zd>.S, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 32;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

32-bit to single-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

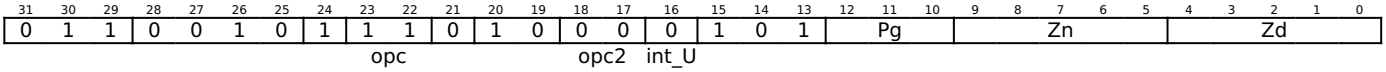
```
SCVTF <Zd>.S, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 32;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

32-bit to double-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

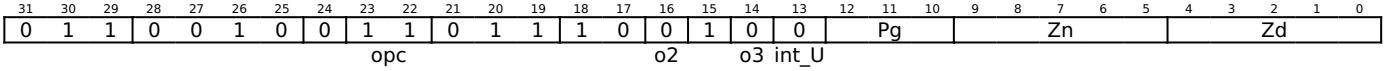
```
SCVTF <Zd>.D, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 64;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

32-bit to double-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

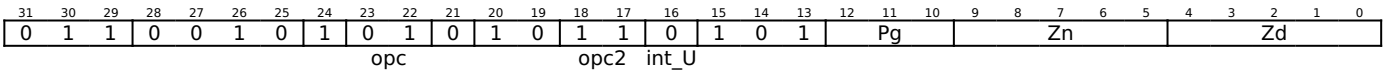
```
SCVTF <Zd>.D, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 64;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

64-bit to half-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

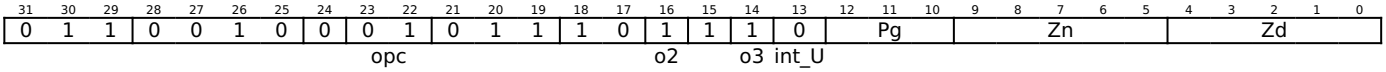
```
SCVTF <Zd>.H, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 16;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

64-bit to half-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

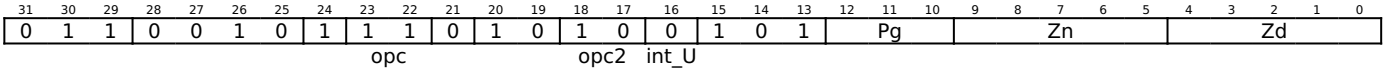
```
SCVTF <Zd>.H, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 16;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

64-bit to single-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

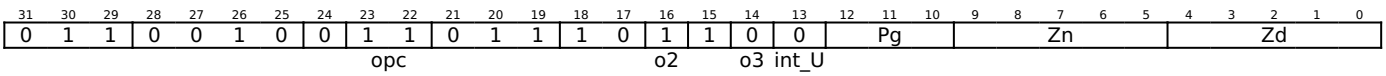
```
SCVTF <Zd>.S, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

64-bit to single-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

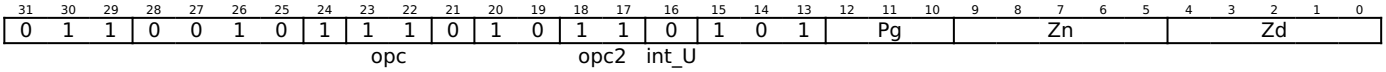
```
SCVTF <Zd>.S, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

64-bit to double-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

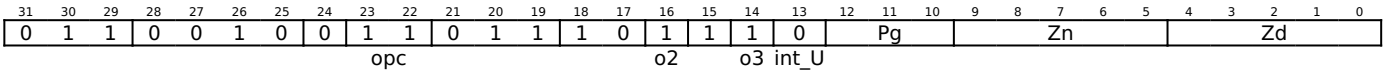
```
SCVTF <Zd>.D, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 64;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

64-bit to double-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
SCVTF <Zd>.D, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 64;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        let fpval : bits(d_esize) = FixedToFP{d_esize, s_esize}(element[s_esize-1:0], 0, unsigned,
                                                                FPCR(), rounding);
        result[e*:esize] = ZeroExtend{esize}(fpval);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "16-bit to half-precision, merging" , "32-bit to half-precision, merging" , "32-bit to single-precision, merging" , "32-bit to double-precision, merging" , "64-bit to half-precision, merging" , "64-bit to single-precision, merging" , "64-bit to double-precision, merging" variants:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SCVTFLT

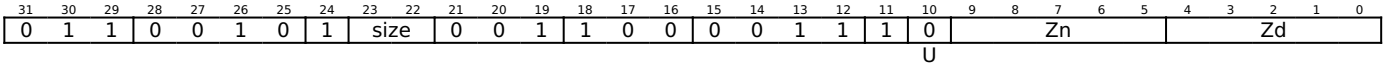
Signed integer convert to floating-point (top, unpredicated)

This instruction converts each odd-numbered signed integer element of the source vector to floating-point, and places the results in the overlapping double-width elements of the destination vector.

This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
SCVTFLT <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize DIV 2) = operand[(2*e + 1)*:(esize DIV 2)];
    result[e*:esize] = FixedToFP{esize, esize DIV 2}(element, 0, unsigned, FPCR(), rounding);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

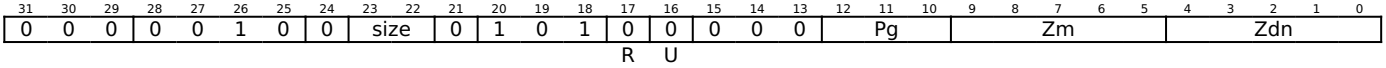
SDIV

Signed divide (predicated)

This instruction performs a signed divide on Active elements of the first source vector by the corresponding elements of the second source vector and destructively places the quotient in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SDIV <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let dividend : integer = SInt(operand1[e*:esize]);
        let divisor : integer = SInt(operand2[e*:esize]);
        var quotient : integer;
        if divisor == 0 then
            quotient = 0;
        elsif (dividend < 0) == (divisor < 0) then
            quotient = Abs(dividend) DIVRM Abs(divisor);    // same signs - positive result
        else
            quotient = -(Abs(dividend) DIVRM Abs(divisor)); // different signs - negative result
        end;
        result[e*:esize] = quotient[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.

- A predicated `MOVPRFX` must use the same governing predicate register as this instruction.
- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

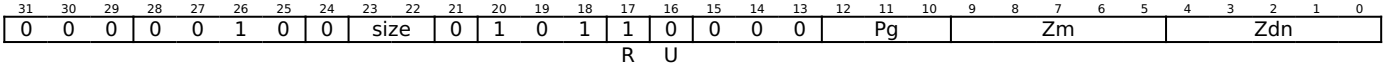
SDIVR

Signed reversed divide (predicated)

This instruction performs a signed reversed divide on Active elements of the second source vector by the corresponding elements of the first source vector, and destructively places the quotient in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SDIVR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let divisor : integer = SInt(operand1[e*:esize]);
        let dividend : integer = SInt(operand2[e*:esize]);
        var quotient : integer;
        if divisor == 0 then
            quotient = 0;
        elsif (dividend < 0) == (divisor < 0) then
            quotient = Abs(dividend) DIVRM Abs(divisor);    // same signs - positive result
        else
            quotient = -(Abs(dividend) DIVRM Abs(divisor)); // different signs - negative result
        end;
        result[e*:esize] = quotient[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.

- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (2-way, indexed)

Signed integer dot product by indexed element (two-way)

This instruction calculates the dot product of a pair of signed 8-bit or 16-bit integer values held in each 16-bit or 32-bit element of the first source vector multiplied by a pair of signed 8-bit or 16-bit integer values in an indexed 16-bit or 32-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 16-bit or 32-bit element of the destination vector.

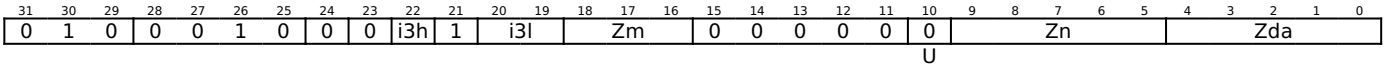
The groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment. The index range is from 0 to one less than the number of groups per 128-bit segment.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit to 16-bit and 16-bit to 32-bit

8-bit to 16-bit

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

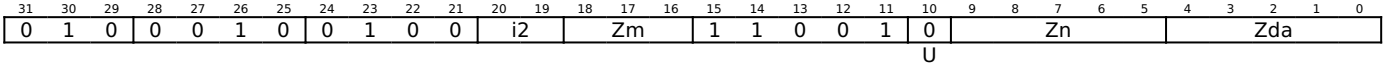
```
SDOT <Zda>.H, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

16-bit to 32-bit

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
SDOT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	For the "8-bit to 16-bit" variant: is the immediate index of a pair of 8-bit elements within each 128-bit vector segment, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "16-bit to 32-bit" variant: is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 1 do
        let element1 : integer = SInt(operand1[(2 * e + i)*:(esize DIV 2)]);
        let element2 : integer = SInt(operand2[(2 * s + i)*:(esize DIV 2)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (2-way, vectors)

Signed integer dot product (two-way)

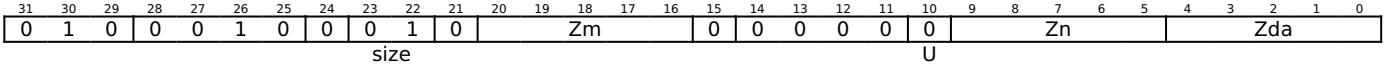
This instruction calculates the dot product of a pair of signed 8-bit or 16-bit integer values held in each 16-bit or 32-bit element of the first source vector multiplied by a pair of signed 8-bit or 16-bit integer values in the corresponding 16-bit or 32-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 16-bit or 32-bit element of the destination vector.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit to 16-bit and 16-bit to 32-bit

8-bit to 16-bit

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

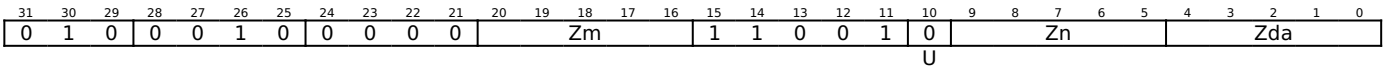
```
SDOT <Zda>.H, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

16-bit to 32-bit

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
SDOT <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 1 do
        let element1 : integer = SInt(operand1[(2 * e + i)*:(esize DIV 2)]);
        let element2 : integer = SInt(operand2[(2 * e + i)*:(esize DIV 2)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (4-way, indexed)

Signed integer dot product by indexed element (four-way)

This instruction calculates the dot product of a group of four signed 8-bit or 16-bit integer values held in each 32-bit or 64-bit element of the first source vector multiplied by a group of four signed 8-bit or 16-bit integer values in an indexed 32-bit or 64-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 32-bit or 64-bit element of the destination vector.

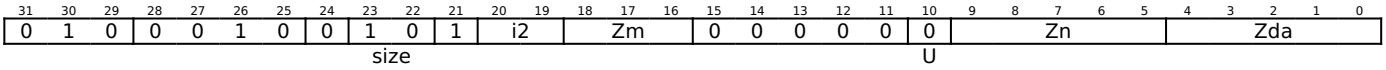
The groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment. The index range is from 0 to one less than the number of groups per 128-bit segment.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit to 32-bit and 16-bit to 64-bit

8-bit to 32-bit

(FEAT_SVE || FEAT_SME)



Encoding

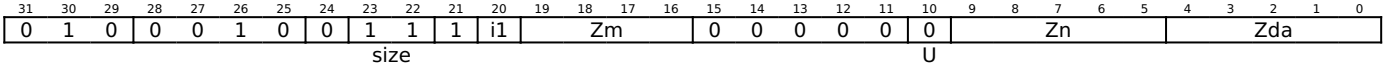
```
SDOT <Zda>.S, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

16-bit to 64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SDOT <Zda>.D, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "8-bit to 32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "16-bit to 64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "8-bit to 32-bit" variant: is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field. For the "16-bit to 64-bit" variant: is the immediate index of a group of four 16-bit values within each 128-bit vector segment, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 3 do
        let element1 : integer = SInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        let element2 : integer = SInt(operand2[(4 * s + i)*:(esize DIV 4)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

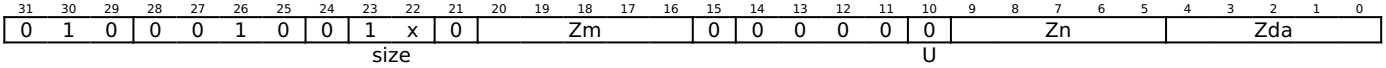
SDOT (4-way, vectors)

Signed integer dot product (four-way)

This instruction calculates the dot product of a group of four signed 8-bit or 16-bit integer values held in each 32-bit or 64-bit element of the first source vector multiplied by a group of four signed 8-bit or 16-bit integer values in the corresponding 32-bit or 64-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 32-bit or 64-bit element of the destination vector.
This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SDOT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size[0]”:

size[0]	<Tb>
0	B
1	H

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 3 do
        let element1 : integer = SInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        let element2 : integer = SInt(operand2[(4 * e + i)*:(esize DIV 4)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SEL (predicates)

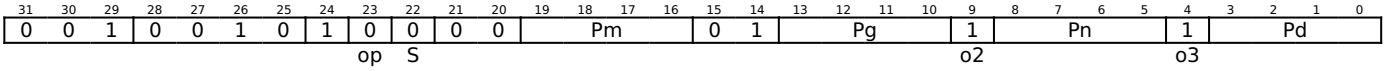
Conditionally select elements from two predicates

This instruction reads Active elements from the first source predicate and Inactive elements from the second source predicate and places them in the corresponding elements of the destination predicate. This instruction does not set the condition flags.

This instruction is used by the alias MOV (predicate, merging).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SEL <Pd>.B, <Pg>, <Pn>.B, <Pm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
```

Assembler Symbols

<Pd>	Is the name of the destination scalable predicate register, encoded in the "Pd" field.
<Pg>	Is the name of the governing scalable predicate register, encoded in the "Pg" field.
<Pn>	Is the name of the first source scalable predicate register, encoded in the "Pn" field.
<Pm>	Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Alias Conditions

Alias	Is preferred when
MOV (predicate, merging)	`Pd == Pm`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

for e = 0 to elements-1 do
    let element1 : bit = PredicateElement{PL}(operand1, e, esize);
    let element2 : bit = PredicateElement{PL}(operand2, e, esize);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:psize] = ZeroExtend{psize}(element1);
    else
        result[e*:psize] = ZeroExtend{psize}(element2);
    end;
end;

P{PL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SEL (vectors)

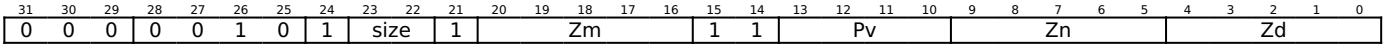
Conditionally select elements from two vectors

This instruction selects elements from the first source vector where the corresponding vector select predicate element is true, and from the second source vector where the predicate element is false, and places them in the corresponding elements of the destination vector.

This instruction is used by the alias MOV (vector, predicated).

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SEL <Zd>.<T>, <Pv>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let v : integer = UInt(Pv);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pv> Is the name of the vector select predicate register, encoded in the "Pv" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Alias Conditions

Alias	Is preferred when
MOV (vector, predicated)	`Zd == Zm`

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(v);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
let operand2 : bits(VL) = if AnyActiveElement{PL}(NOT(mask), esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand1[e*:esize];
    else
        result[e*:esize] = operand2[e*:esize];
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SETFFR

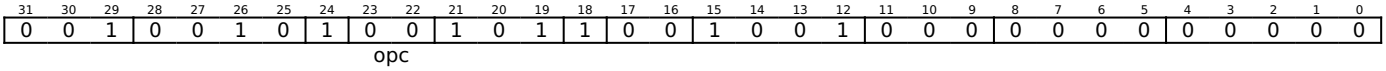
Initialize the first-fault register to all true

This instruction initializes the first-fault register (FFR) to all true prior to a sequence of first-fault or non-fault loads. This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

SETFFR

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
FFR{PL}() = Ones{PL};
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

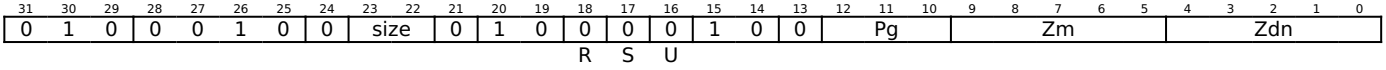
SHADD

Signed halving add

This instruction adds Active signed elements of the first source vector to the corresponding signed elements of the second source vector, shifts right one bit, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SHADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = (element1 + element2) >> 1;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

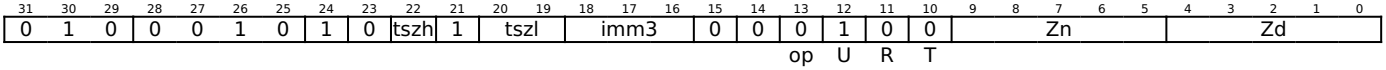
SHRNB

Shift right narrow by immediate (bottom)

This instruction shifts each unsigned integer value in the source vector elements right by an immediate value, and places the truncated results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SHRNB <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = UInt(element) >> shift;
    result[(2*e + 0)*:esize] = res[esize-1:0];
    result[(2*e + 1)*:esize] = Zeros{esize};

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

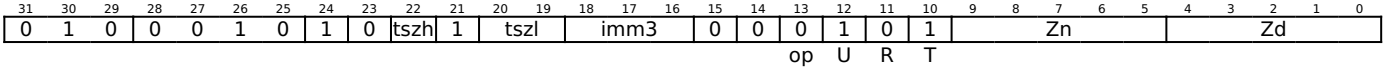
SHRNT

Shift right narrow by immediate (top)

This instruction shifts each unsigned integer value in the source vector elements right by an immediate value, and places the truncated results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SHRNT <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = UInt(element) >> shift;
    result[(2*e + 1)*:esize] = res[esize-1:0];

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

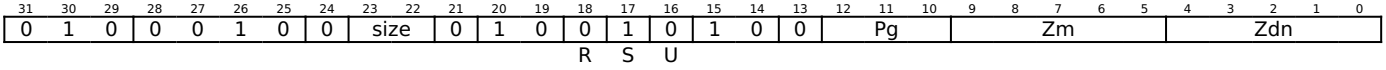
SHSUB

Signed halving subtract

This instruction subtracts Active signed elements of the second source vector from the corresponding signed elements of the first source vector, shifts right one bit, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SHSUB <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = (element1 - element2) >> 1;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

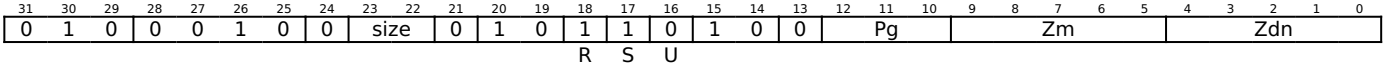
SHSUBR

Signed halving subtract reversed

This instruction subtracts Active signed elements of the first source vector from corresponding signed elements of the second source vector, shifts right one bit, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SHSUBR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = (element2 - element1) >> 1;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

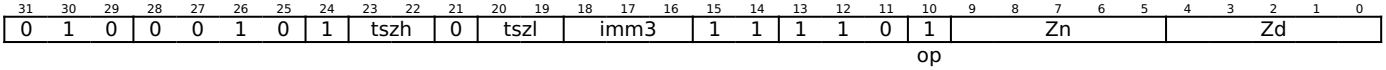
SLI

Shift left and insert (immediate)

This instruction shifts each source vector element left by an immediate value, and inserts the result into the corresponding vector element in the destination vector register, merging the shifted bits from each source element with existing bits in each destination vector element. The immediate shift amount is an unsigned value in the range 0 to number of bits per element minus 1. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SLI <Zd>.<T>, <Zn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = UInt(tsize::imm3) - esize;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<const> Is the immediate shift amount, in the range 0 to number of bits per element minus 1, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    let element1 : bits(esize) = result[e*:esize];
    let element2 : bits(esize) = operand[e*:esize];
    let mask : bits(esize) = LSL(Ones{esize}, shift);
    let shiftedval : bits(esize) = LSL(element2, shift);
    result[e*:esize] = (element1 AND (NOT mask)) OR shiftedval;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SM4E

SM4 encryption and decryption

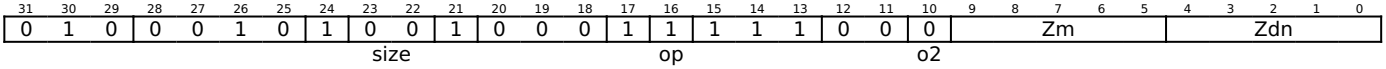
This instruction reads 16 bytes of input data from each 128-bit segment of the first source vector, together with four iterations of 32-bit round keys from the corresponding 128-bit segments of the second source vector. Each block of data is encrypted by four rounds in accordance with the SM4 standard, and destructively placed in the corresponding segments of the first source vector. This instruction is unpredicated.

ID_AA64ZFR0_EL1.SM4 indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE_SM4)



Encoding

```
SM4E <Zdn>.S, <Zdn>.S, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_SM4) then EndOfDecode(Decode_UNDEF); end;
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for s = 0 to segments-1 do
    let key : bits(128) = operand2[s*:128];
    var intval : bits(32);
    var roundresult : bits(128) = operand1[s*:128];
    var roundkey : bits(32);

    for index = 0 to 3 do
        roundkey = key[index*:32];
        intval = roundresult[127:96] XOR roundresult[95:64] XOR roundresult[63:32] XOR roundkey;

        for i = 0 to 3 do
            intval[i*:8]  = Sbox(intval[i*:8]);
        end;

        intval = (intval XOR
                  ROL(intval, 2) XOR
                  ROL(intval, 10) XOR
                  ROL(intval, 18) XOR
                  ROL(intval, 24));
        intval = intval XOR roundresult[31:0];

        roundresult[31:0] = roundresult[63:32];
        roundresult[63:32] = roundresult[95:64];
        roundresult[95:64] = roundresult[127:96];
        roundresult[127:96] = intval;
    end;

    result[s*:128] = roundresult;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SM4EKEY

SM4 key updates

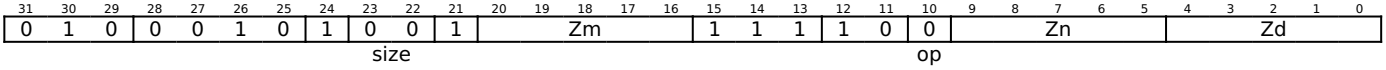
This instruction reads four rounds of 32-bit input key values from each 128-bit segment of the first source vector, along with four rounds of 32-bit constants from the corresponding 128-bit segment of the second source vector. The four rounds of output key values are derived in accordance with the SM4 standard, and placed in the corresponding segments of the destination vector. This instruction is unpredicated.

ID_AA64ZFR0_EL1.SM4 indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE_SM4)



Encoding

```
SM4EKEY <Zd>.S, <Zn>.S, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_SM4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for s = 0 to segments-1 do
    let source : bits(128) = operand2[s*:128];
    var intval : bits(32);
    var const : bits(32);
    var roundresult : bits(128) = operand1[s*:128];

    for index = 0 to 3 do
        const = source[index*:32];
        intval = roundresult[127:96] XOR roundresult[95:64] XOR roundresult[63:32] XOR const;
        for i = 0 to 3 do
            intval[i*:8] = Sbox(intval[i*:8]);
        end;

        intval = intval XOR ROL(intval, 13) XOR ROL(intval, 23);
        intval = intval XOR roundresult[31:0];

        roundresult[31:0] = roundresult[63:32];
        roundresult[63:32] = roundresult[95:64];
        roundresult[95:64] = roundresult[127:96];
        roundresult[127:96] = intval;
    end;

    result[s*:128] = roundresult;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

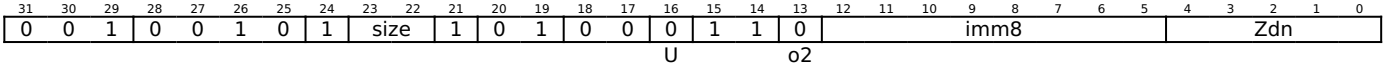
SMAX (immediate)

Signed maximum with immediate (unpredicated)

This instruction determines the signed maximum of an immediate and each element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate is a signed 8-bit value in the range -128 to +127, inclusive. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SMAX <Zdn>.<T>, <Zdn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
let unsigned : boolean = FALSE;
let imm : integer = if unsigned then UInt(imm8) else SInt(imm8);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is the signed immediate operand, in the range -128 to 127, encoded in the "imm8" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    result[e*:esize] = Max(element1, imm)[esize-1:0];

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

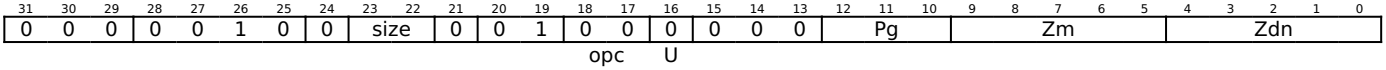
SMAX (vectors)

Signed maximum (predicated)

This instruction determines the signed maximum of Active elements of the first source vector and the corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SMAX <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let maximum : integer = Max(element1, element2);
        result[e*:esize] = maximum[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

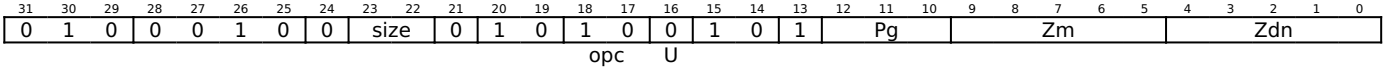
SMAXP

Signed maximum pairwise

This instruction calculates the maximum value of each pair of adjacent signed integer elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMAXP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    if !ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand1[e*:esize];
    else
        if IsEven(e) then
            element1 = SInt(operand1[(e + 0)*:esize]);
            element2 = SInt(operand1[(e + 1)*:esize]);
        else
            element1 = SInt(operand2[(e - 1)*:esize]);
            element2 = SInt(operand2[(e + 0)*:esize]);
        end;
        let res : integer = Max(element1, element2);
        result[e*:esize] = res[esize-1:0];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

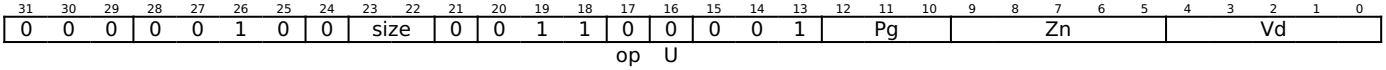
SMAXQV

Signed maximum reduction of quadword vector segments

This instruction determines the signed maximum of the elements in the same position from each 128-bit source vector segment, and places each result into the corresponding element of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as the minimum signed integer for the element size.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
SMAXQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	16B
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(128) = Zeros{};
var stmp : bits(128) = Zeros{};

var dtmp : integer;

for e = 0 to elempersegment-1 do
    dtmp = if unsigned then 0 else -(2^(esize-1));
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp = operand[s*:128];
            dtmp = Max(dtmp, SInt(stmp[e*:esize]));
        end;
    end;
    result[e*:esize] = dtmp[esize-1:0];
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

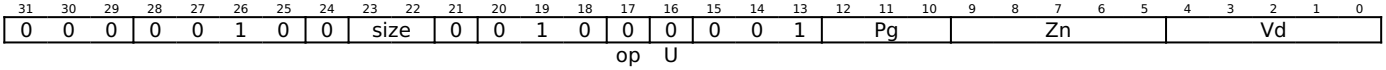
SMAXV

Signed maximum reduction to scalar

This instruction determines the signed maximum of the elements of the source vector, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as the minimum signed integer for the element size.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SMAXV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var maximum : integer = if unsigned then 0 else -(2^(esize-1));

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let opelt : bits(esize) = operand[e*:esize];
        let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
        maximum = Max(maximum, element);

    end;
end;
V{esize}(d) = maximum[esize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

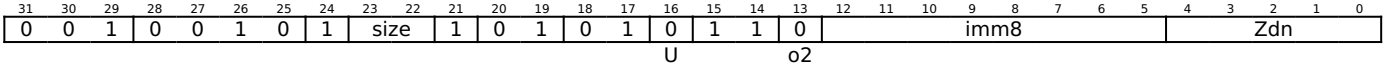
SMIN (immediate)

Signed minimum with immediate (unpredicated)

This instruction determines the signed minimum of an immediate and each element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate is a signed 8-bit value in the range -128 to +127, inclusive. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SMIN <Zdn>.<T>, <Zdn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
let unsigned : boolean = FALSE;
let imm : integer = if unsigned then UInt(imm8) else SInt(imm8);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is the signed immediate operand, in the range -128 to 127, encoded in the "imm8" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    result[e*:esize] = Min(element1, imm)[esize-1:0];

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

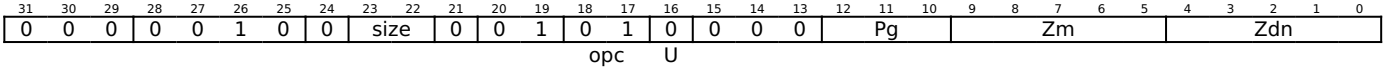
SMIN (vectors)

Signed minimum (predicated)

This instruction determines the signed minimum of Active elements of the first source vector and the corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SMIN <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let minimum : integer = Min(element1, element2);
        result[e*:esize] = minimum[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

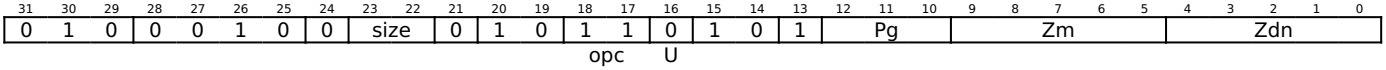
SMINP

Signed minimum pairwise

This instruction calculates the minimum value of each pair of adjacent signed integer elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMINP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    if !ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand1[e*:esize];
    else
        if IsEven(e) then
            element1 = SInt(operand1[(e + 0)*:esize]);
            element2 = SInt(operand1[(e + 1)*:esize]);
        else
            element1 = SInt(operand2[(e - 1)*:esize]);
            element2 = SInt(operand2[(e + 0)*:esize]);
        end;
        let res : integer = Min(element1, element2);
        result[e*:esize] = res[esize-1:0];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

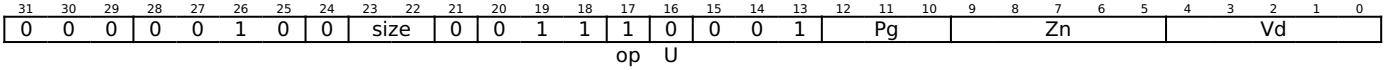
SMINQV

Signed minimum reduction of quadword vector segments

This instruction determines the signed minimum of the elements in the same position from each 128-bit source vector segment, and places each result into the corresponding element of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as the maximum signed integer for the element size.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
SMINQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	16B
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(128) = Zeros{};
var stmp : bits(128) = Zeros{};

var dtmp : integer;

for e = 0 to elempersegment-1 do
    dtmp = if unsigned then (2^esize - 1) else (2^(esize-1) - 1);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp = operand[s*:128];
            dtmp = Min(dtmp, SInt(stmp[e*:esize]));
        end;
    end;
    result[e*:esize] = dtmp[esize-1:0];
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

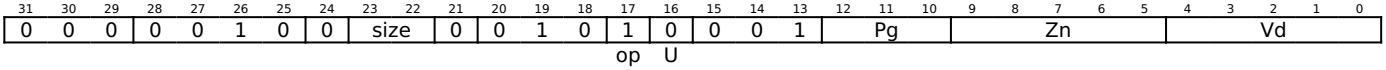
SMINV

Signed minimum reduction to scalar

This instruction determines the signed minimum of the elements of the source vector, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as the maximum signed integer for the element size.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SMINV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var minimum : integer = if unsigned then (2^esize - 1) else (2^(esize-1) - 1);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let opelt : bits(esize) = operand[e*:esize];
        let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
        minimum = Min(minimum, element);

    end;
end;
V{esize}(d) = minimum[esize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLALB (indexed)

Signed multiply-add long by indexed element (bottom)

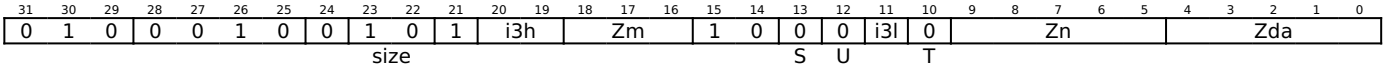
This instruction multiplies the even-numbered signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment, and destructively adds the results to the overlapping double-width elements of the addend vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

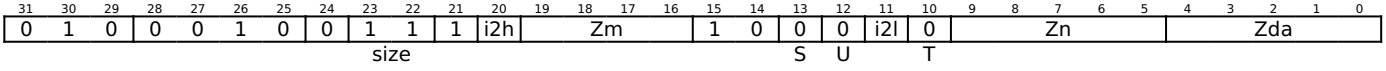
```
SMLALB <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMLALB <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let product : bits(2*esize) = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] + product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

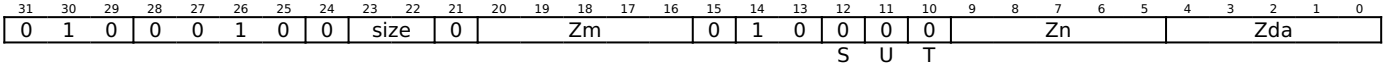
SMLALB (vectors)

Signed multiply-add long (bottom)

This instruction multiplies the even-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and destructively adds the results to the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMLALB <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let product = (element1 * element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLALT (indexed)

Signed multiply-add long by indexed element (top)

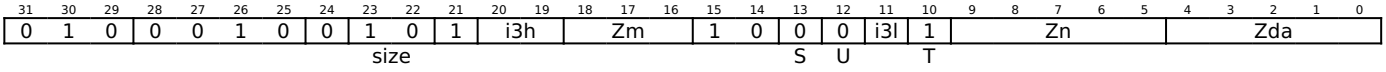
This instruction multiplies the odd-numbered signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment, and destructively adds the results to the overlapping double-width elements of the addend vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

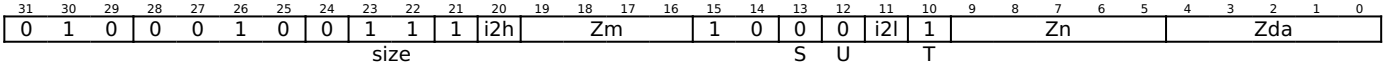
```
SMLALT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMLALT <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let product : bits(2*esize) = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] + product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

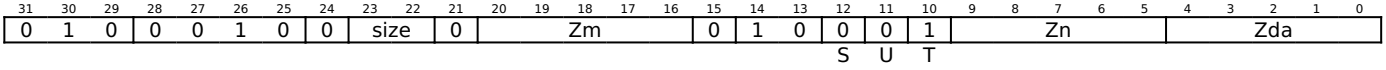
SMLALT (vectors)

Signed multiply-add long (top)

This instruction multiplies the odd-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and destructively adds the results to the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMLALT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let product = (element1 * element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLSLB (indexed)

Signed multiply-subtract long by indexed element (bottom)

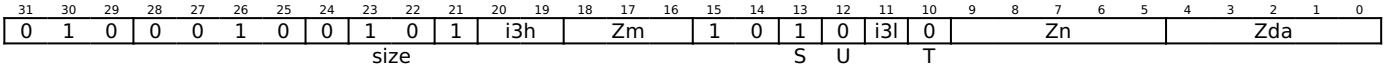
This instruction multiplies the even-numbered signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment, and destructively subtracts the results from the overlapping double-width elements of the addend vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

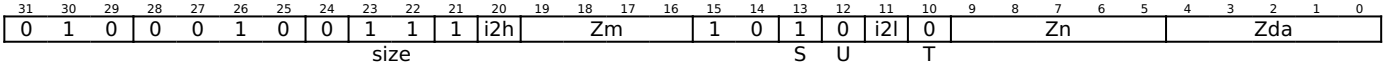
```
SMLSLB <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMLSLB <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let product : bits(2*esize) = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] - product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

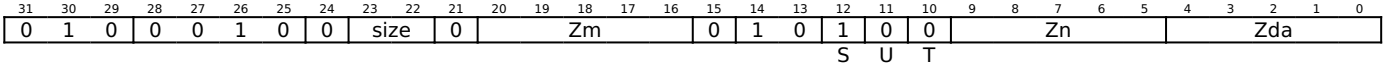
SMLSLB (vectors)

Signed multiply-subtract long (bottom)

This instruction multiplies the even-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and destructively subtracts the results from the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMLSLB <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let product = (element1 * element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] - product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLSLT (indexed)

Signed multiply-subtract long by indexed element (top)

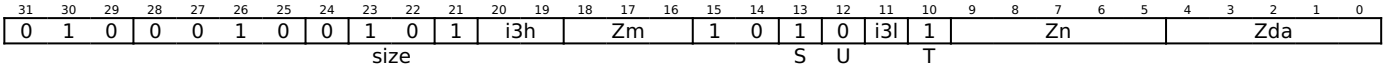
This instruction multiplies the odd-numbered signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment, and destructively subtracts the results from the overlapping double-width elements of the addend vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

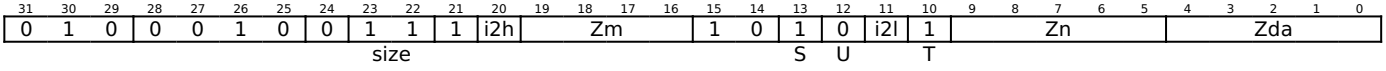
```
SMLSLT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMLSLT <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let product : bits(2*esize) = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] - product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

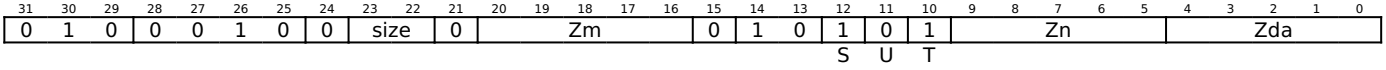
SMLSLT (vectors)

Signed multiply-subtract long (top)

This instruction multiplies the odd-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and destructively subtracts the results from the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMLSLT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let product = (element1 * element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] - product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMMLA

Signed 8-bit integer matrix multiply-accumulate to 32-bit integer

This instruction multiplies the 2×8 matrix of signed 8-bit integer values held in each 128-bit segment of the first source vector by the 8×2 matrix of signed 8-bit integer values in the corresponding segment of the second source vector. The resulting 2×2 widened 32-bit integer matrix product is then destructively added to the 32-bit integer matrix accumulator held in the corresponding segment of the addend and destination vector. This is equivalent to performing an 8-way dot product per destination element.

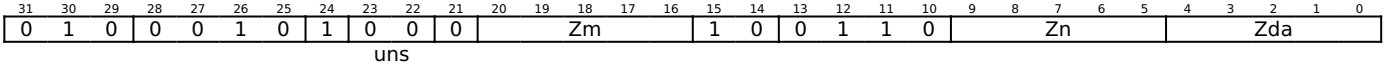
This instruction is unpredicated.

ID_AA64ZFR0_EL1.I8MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE && FEAT_I8MM)



Encoding

```
SMMLA <Zda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_I8MM) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL) = Zeros{};
var op1, op2 : bits(128);
var res, addend : bits(128);

for s = 0 to segments-1 do
    op1    = operand1[s*:(128)];
    op2    = operand2[s*:(128)];
    addend = operand3[s*:(128)];
    res    = MatMulAdd(addend, op1, op2, op1_unsigned, op2_unsigned);
    result[s*:(128)] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

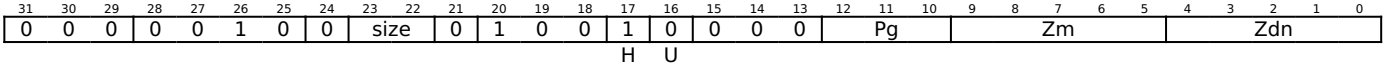
SMULH (predicated)

Signed multiply returning high half (predicated)

This instruction performs a widening multiplication of signed integer values in Active elements of the first source vector by the corresponding elements of the second source vector and destructively places the high half of the result in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SMULH <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let product : integer = (element1 * element2) >> esize;
        result[e*:esize] = product[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

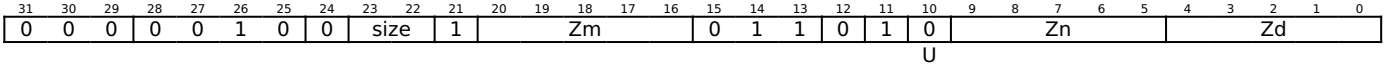
SMULH (unpredicated)

Signed multiply returning high half (unpredicated)

This instruction performs a widening multiplication of signed integer values of all elements of the first source vector by the corresponding elements of the second source vector and places the high half of the result in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMULH <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let op2elt : bits(esize) = operand2[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let product : integer = (element1 * element2) >> esize;
    result[e*:esize] = product[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMULLB (indexed)

Signed multiply long by indexed element (bottom)

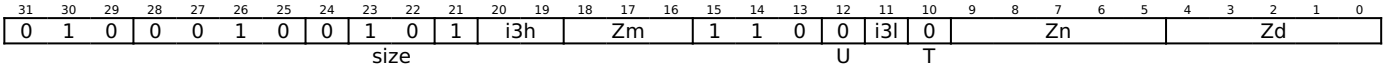
This instruction multiplies the even-numbered signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment, and places the results in the overlapping double-width elements of the destination vector register.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

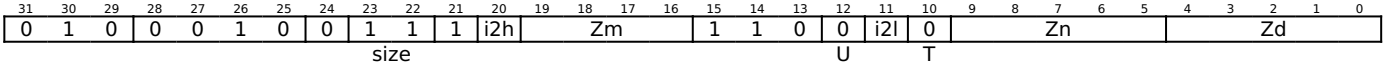
```
SMULLB <Zd>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 0;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMULLB <Zd>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 0;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let res : integer = element1 * element2;
    result[e*:(2*esize)] = res[2*esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

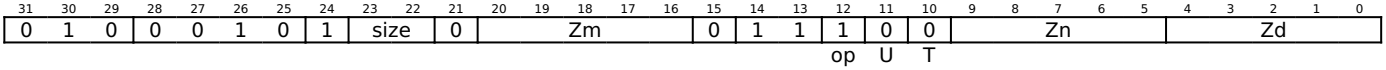
SMULLB (vectors)

Signed multiply long (bottom)

This instruction multiplies the even-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMULLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let res : integer = element1 * element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMULLT (indexed)

Signed multiply long by indexed element (top)

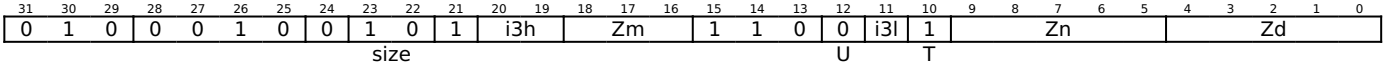
This instruction multiplies the odd-numbered signed elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment, and places the results in the overlapping double-width elements of the destination vector register.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

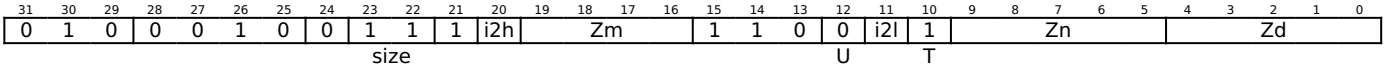
```
SMULLT <Zd>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 1;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMULLT <Zd>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 1;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let res : integer = element1 * element2;
    result[e*:(2*esize)] = res[2*esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

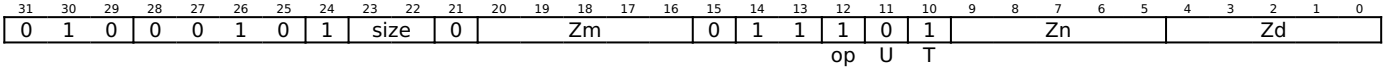
SMULLT (vectors)

Signed multiply long (top)

This instruction multiplies the odd-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SMULLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let res : integer = element1 * element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SPLICE

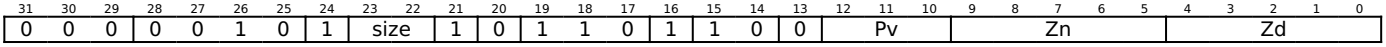
Splice two vectors under predicate control

This instruction selects a region from the first source vector and copies it to the lowest-numbered elements of the result. Any remaining elements of the result are set to a copy of the lowest-numbered elements from the second source vector. The region is selected using the first and last true elements in the vector select predicate register. The result is destructively placed in the destination and first source vector, or constructively placed in the destination vector.

It has encodings from 2 classes: Constructive and Destructive

Constructive

(FEAT_SVE2 || FEAT_SME)



Encoding

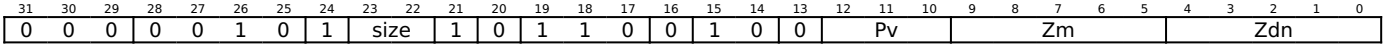
```
SPLICE <Zd>.<T>, <Pv>, { <Zn1>.<T>, <Zn2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let v : integer = UInt(Pv);
let dst : integer = UInt(Zd);
let s1 : integer = UInt(Zn);
let s2 : integer = (s1 + 1) MOD 32;
```

Destructive

(FEAT_SVE || FEAT_SME)



Encoding

```
SPLICE <Zdn>.<T>, <Pv>, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let v : integer = UInt(Pv);
let dst : integer = UInt(Zdn);
let s1 : integer = dst;
let s2 : integer = UInt(Zm);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pv> Is the name of the vector select predicate register P0-P7, encoded in the "Pv" field.

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded in the "Zn" field.

<Zn2> Is the name of the second scalable vector register of the source multi-vector group, encoded in the "Zn" field.

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(v);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(s1) else Zeros{VL};
let operand2 : bits(VL) = Z{}(s2);
var result : bits(VL);
var x : integer = 0;
var active : boolean = FALSE;
let lastnum : integer = LastActiveElement{PL}(mask, esize);

if lastnum >= 0 then
    for e = 0 to lastnum do
        active = active || ActivePredicateElement{PL}(mask, e, esize);
        if active then
            result[x*:esize] = operand1[e*:esize];
            x = x + 1;
        end;
    end;
end;

let nelements : integer = (elements - x) - 1;
for e = 0 to nelements do
    result[x*:esize] = operand2[e*:esize];
    x = x + 1;
end;

Z{VL}(dst) = result;
```

Operational information

For the "Destructive" variant:
The destructive variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the destructive variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQABS

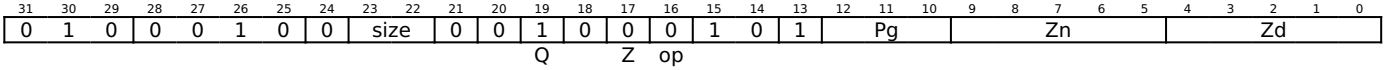
Signed saturating absolute value

This instruction calculates the absolute value of the signed integer in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE2 || FEAT_SME)



Encoding

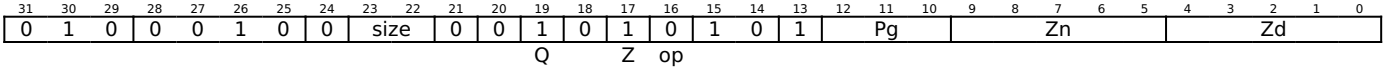
```
SQABS <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
SQABS <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        var element : integer = SInt(operand[e*:esize]);
        element = Abs(element);
        result[e*:esize] = SignedSat{esize}(element);

    end;
end;
Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:
The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQADD (immediate)

Signed saturating add immediate (unpredicated)

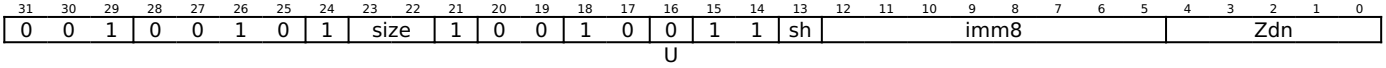
This instruction performs a signed saturating addition of an unsigned immediate to each element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQADD <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    (result[e*:esize], -) = SatQ{esize}(element1 + imm, unsigned);

end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

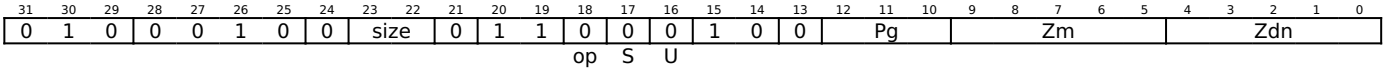
SQADD (vectors, predicated)

Signed saturating add (predicated)

This instruction adds Active signed elements of the first source vector to corresponding signed elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = SignedSat{esize}(element1 + element2);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

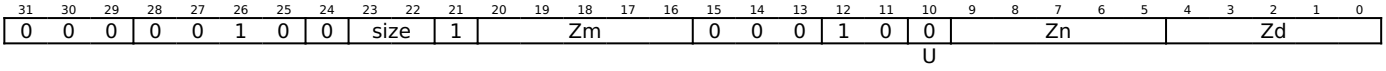
SQADD (vectors, unpredicated)

Signed saturating add (unpredicated)

This instruction performs a signed saturating addition of all elements of the first source vector to the corresponding elements of the second source vector, and places the results in the corresponding elements of the destination vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQADD <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let op2elt : bits(esize) = operand2[e*:esize];
    if unsigned then
        let element1 : integer = UInt(op1elt);
        let element2 : integer = UInt(op2elt);
        (result[e*:esize], -) = UnsignedSatQ{esize}(element1 + element2);
    else
        let element1 : integer = SInt(op1elt);
        let element2 : integer = SInt(op2elt);
        (result[e*:esize], -) = SignedSatQ{esize}(element1 + element2);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQCADD

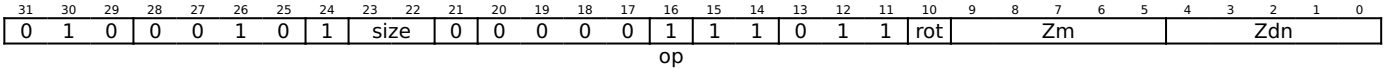
Saturating complex integer add

This instruction adds the real and imaginary components of the integral complex numbers from the first source vector to the complex numbers from the second source vector that have first been rotated by 90 or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation, equivalent to multiplying the complex numbers in the second source vector by ±j beforehand. The results are destructively placed in the corresponding elements of the first source vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQCADD <Zdn>.<T>, <Zdn>.<T>, <Zm>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
let sub_i : boolean = (rot == '0');
let sub_r : boolean = (rot == '1');
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
0	#90
1	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (2 * esize);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for p = 0 to pairs-1 do
    var acc_r : integer  = SInt(operand1[(2 * p + 0)*:esize]);
    var acc_i : integer  = SInt(operand1[(2 * p + 1)*:esize]);
    let elt2_r : integer = SInt(operand2[(2 * p + 0)*:esize]);
    let elt2_i : integer = SInt(operand2[(2 * p + 1)*:esize]);
    if sub_i then
        acc_r = acc_r - elt2_i;
        acc_i = acc_i + elt2_r;
    end;
    if sub_r then
        acc_r = acc_r + elt2_i;
        acc_i = acc_i - elt2_r;
    end;

    result[(2 * p + 0)*:esize] = SignedSat{esize}(acc_r);
    result[(2 * p + 1)*:esize] = SignedSat{esize}(acc_i);
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQCVTN

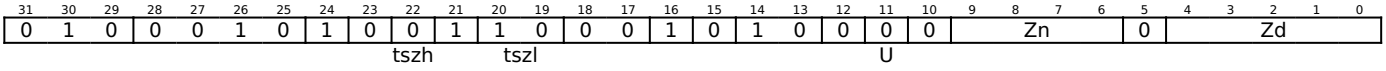
Signed 32-bit integer saturating extract narrow to interleaved 16-bit integer

This instruction saturates the signed integer value in each element of the pair of source vectors to half the original source element width, and places the two-way interleaved results in the half-width destination elements.

This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
SQCVTN <Zd>.H, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 1 do
        let operand : bits(VL) = Z{}(n+i);
        let element : integer = SInt(operand[e*:(2 * esize)]);
        result[(2*e + i)*:esize] = SignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQCVTUN

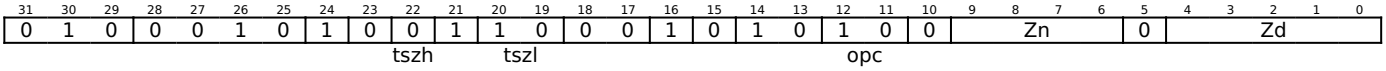
Signed 32-bit integer saturating extract narrow to interleaved unsigned 16-bit integer

This instruction saturates the signed integer value in each element of the pair of source vectors to unsigned integer value that is half the original source element width, and places the two-way interleaved results in the half-width destination elements.

This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
SQCVTUN <Zd>.H, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 1 do
        let operand : bits(VL) = Z{}(n+i);
        let element : integer = SInt(operand[e*:(2 * esize)]);
        result[(2*e + i)*:esize] = UnsignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDECB

Signed saturating decrement scalar by multiple of 8-bit predicate constraint element count

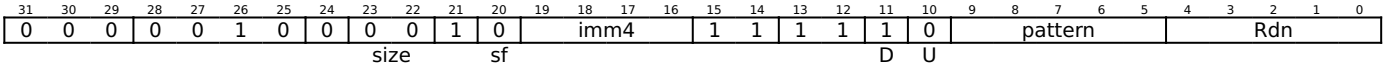
This instruction determines the number of Active 8-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement the scalar destination. The result is saturated to the source general-purpose register's signed integer range. A 32-bit saturated result is then sign-extended to 64 bits.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

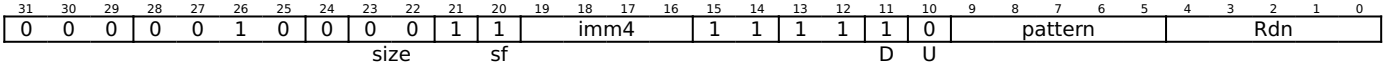
```
SQDECB <Xdn>, <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SQDECB <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Xdn>	Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.
<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) - (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) - (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDECD (scalar)

Signed saturating decrement scalar by multiple of 64-bit predicate constraint element count

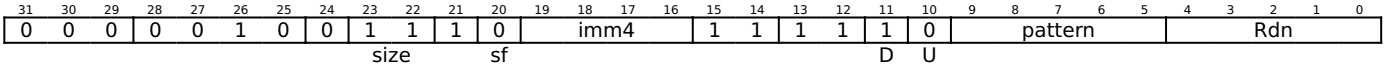
This instruction determines the number of Active 64-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement the scalar destination. The result is saturated to the source general-purpose register's signed integer range. A 32-bit saturated result is then sign-extended to 64 bits.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

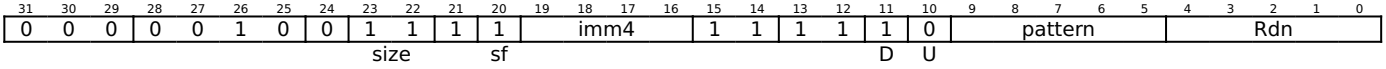
```
SQDECD <Xdn>, <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SQDECD <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Xdn>	Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.
<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) - (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) - (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDECD (vector)

Signed saturating decrement vector by multiple of 64-bit predicate constraint element count

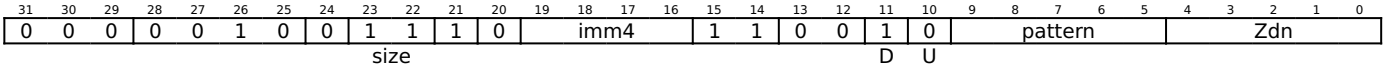
This instruction determines the number of Active 64-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement all destination vector elements. The results are saturated to the 64-bit signed integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQDECD <Zdn>.D{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) - (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) - (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDECH (scalar)

Signed saturating decrement scalar by multiple of 16-bit predicate constraint element count

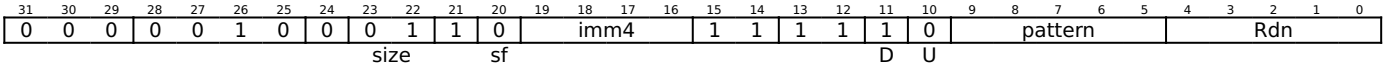
This instruction determines the number of Active 16-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement the scalar destination. The result is saturated to the source general-purpose register's signed integer range. A 32-bit saturated result is then sign-extended to 64 bits.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

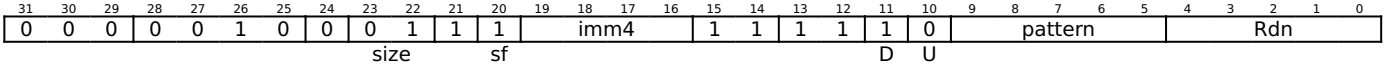
```
SQDECH <Xdn>, <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SQDECH <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Xdn>	Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.
<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) - (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) - (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDECH (vector)

Signed saturating decrement vector by multiple of 16-bit predicate constraint element count

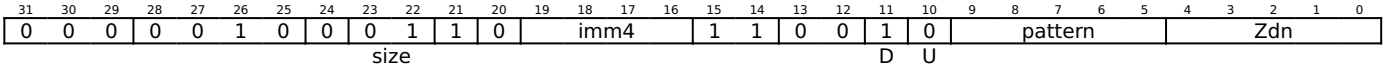
This instruction determines the number of Active 16-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement all destination vector elements. The results are saturated to the 16-bit signed integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQDECH <Zdn>.H{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) - (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) - (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDECP (scalar)

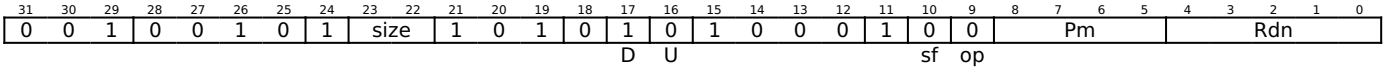
Signed saturating decrement scalar by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to decrement the scalar destination. The result is saturated to the source general-purpose register's signed integer range. A 32-bit saturated result is then sign-extended to 64 bits.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

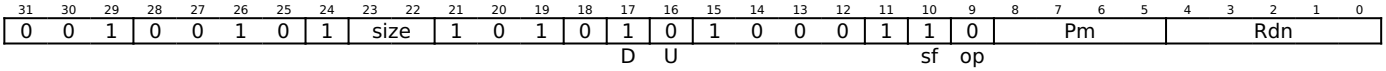
```
SQDECP <Xdn>, <Pm>.<T>, <Wdn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Rdn);
let unsigned : boolean = FALSE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SQDECP <Xdn>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Rdn);
let unsigned : boolean = FALSE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Wdn> Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(ssize) = X{}(dn);
let operand2 : bits(PL) = P{}(m);
var result : bits(ssize);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) - count);
    X{64}(dn) = ZeroExtend{64}(result);
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) - count);
    X{64}(dn) = SignExtend{64}(result);
end;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDECP (vector)

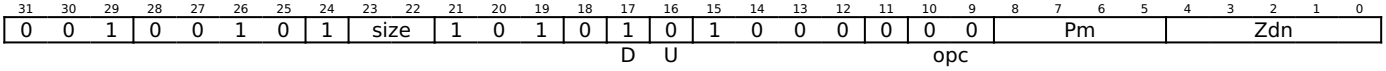
Signed saturating decrement vector by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to decrement all destination vector elements. The results are saturated to the element signed integer range.

The predicate size specifier may be omitted in assembler source code, but this is deprecated and will be prohibited in a future release of the architecture.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQDECP <Zdn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Zdn);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(PL) = P{}(m);
var result : bits(VL);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(opelt) - count);
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(opelt) - count);
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDECW (scalar)

Signed saturating decrement scalar by multiple of 32-bit predicate constraint element count

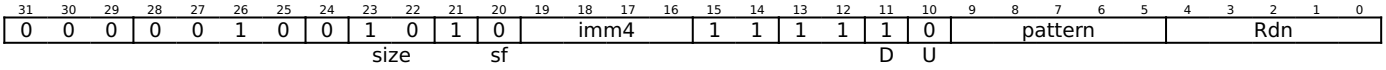
This instruction determines the number of Active 32-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement the scalar destination. The result is saturated to the source general-purpose register's signed integer range. A 32-bit saturated result is then sign-extended to 64 bits.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

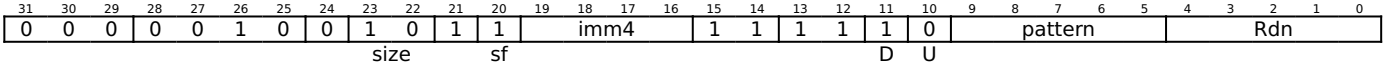
```
SQDECW <Xdn>, <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SQDECW <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Xdn>	Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.
<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) - (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) - (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDECW (vector)

Signed saturating decrement vector by multiple of 32-bit predicate constraint element count

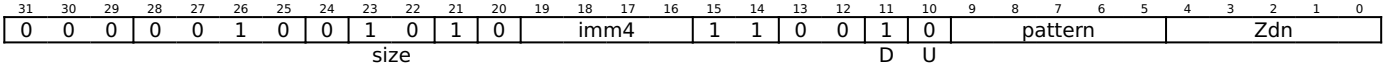
This instruction determines the number of Active 32-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement all destination vector elements. The results are saturated to the 32-bit signed integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQDECW <Zdn>.S{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) - (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) - (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMLALB (indexed)

Signed saturating doubling multiply-add by indexed element (bottom)

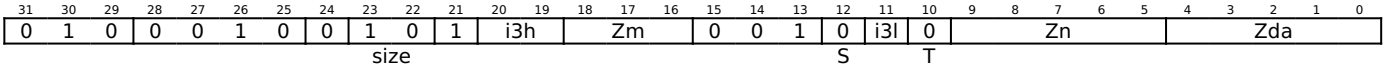
This instruction multiples the even-numbered signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment and then doubles the result of the multiplication. Each intermediate value is saturated to the double-width N-bit value's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$ and is then destructively added to the overlapping double-width elements of the addend and destination vector. Each destination element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

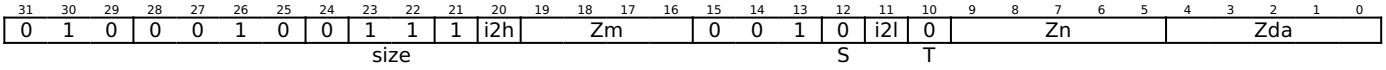
```
SQDMLALB <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMLALB <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let element3 : integer = SInt(result[e*:(2*esize)]);
    let product : integer = SInt(SignedSat{2*esize}(2 * element1 * element2));
    let res : integer = element3 + product;
    result[e*:(2*esize)] = SignedSat{2*esize}(res);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

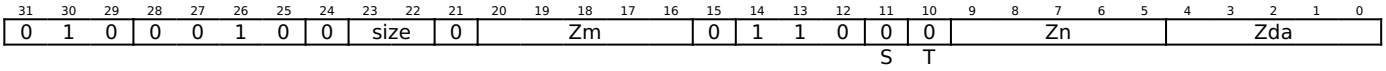
SQDMLALB (vectors)

Signed saturating doubling multiply-add (bottom)

This instruction multiples the even-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and then doubles the result of the multiplication. Each intermediate value is saturated to the double-width N-bit value's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$ and is then destructively added to the overlapping double-width elements of the addend and destination vector. Each destination element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMLALB <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel1 : integer = 0;
let sel2 : integer = 0;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2 * e + sel1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2 * e + sel2)*:(esize DIV 2)]);
    let element3 : integer = SInt(result[e*:esize]);
    let product : integer = SInt(SignedSat{esize}(2 * element1 * element2));
    result[e*:esize] = SignedSat{esize}(element3 + product);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

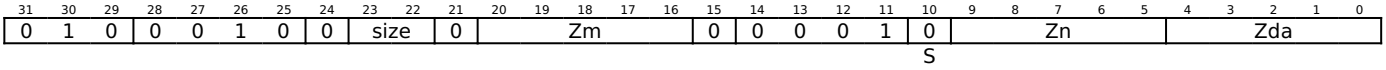
SQDMLALBT

Signed saturating doubling multiply-add (bottom × top)

This instruction multiplies the even-numbered signed elements of the first source vector by the corresponding odd-numbered signed elements of the second source vector, and then doubles the result of the multiplication. Each intermediate value is saturated to the double-width N-bit value's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$ and is then destructively added to the overlapping double-width elements of the addend and destination vector. Each destination element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMLALBT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel1 : integer = 0;
let sel2 : integer = 1;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2 * e + sel1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2 * e + sel2)*:(esize DIV 2)]);
    let element3 : integer = SInt(result[e*:esize]);
    let product : integer = SInt(SignedSat{esize}(2 * element1 * element2));
    result[e*:esize] = SignedSat{esize}(element3 + product);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

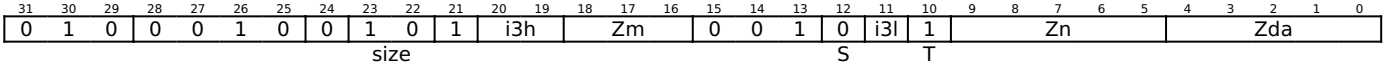
SQDMLALT (indexed)

Signed saturating doubling multiply-add by indexed element (top)

This instruction multiplies the odd-numbered signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment, and then doubles the result. Each intermediate value is saturated to the double-width N-bit value's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$ and is then destructively added to the overlapping double-width elements of the addend and destination vector. Each destination element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.
The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

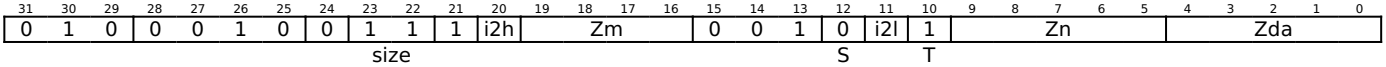
```
SQDMLALT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMLALT <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let element3 : integer = SInt(result[e*:(2*esize)]);
    let product : integer = SInt(SignedSat{2*esize}(2 * element1 * element2));
    let res : integer = element3 + product;
    result[e*:(2*esize)] = SignedSat{2*esize}(res);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

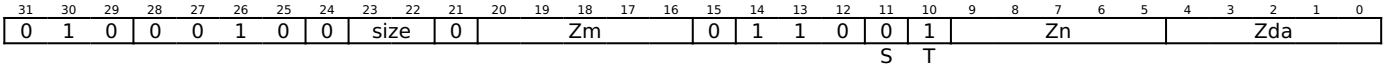
SQDMLALT (vectors)

Signed saturating doubling multiply-add (top)

This instruction multiplies the odd-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and then doubles the result. Each intermediate value is saturated to the double-width N-bit value's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$ and is then destructively added to the overlapping double-width elements of the addend and destination vector. Each destination element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMLALT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel1 : integer = 1;
let sel2 : integer = 1;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2 * e + sel1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2 * e + sel2)*:(esize DIV 2)]);
    let element3 : integer = SInt(result[e*:esize]);
    let product : integer = SInt(SignedSat{esize}(2 * element1 * element2));
    result[e*:esize] = SignedSat{esize}(element3 + product);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

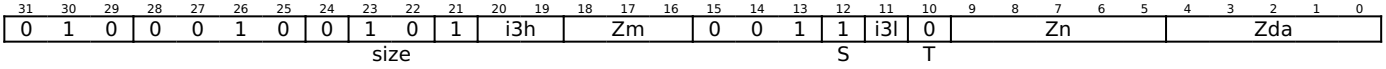
SQDMLSLB (indexed)

Signed saturating doubling multiply-subtract by indexed element (bottom)

This instruction multiplies the even-numbered signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment, and then doubles the result. Each intermediate value is saturated to the double-width N-bit value's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$ and is then destructively subtracted from the overlapping double-width elements of the addend and destination vector. Each destination element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

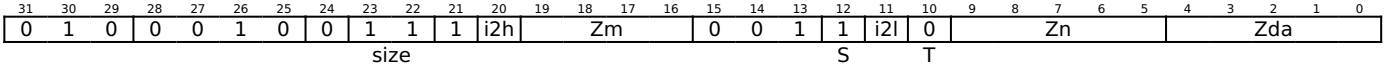
```
SQDMLSLB <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMLSLB <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let element3 : integer = SInt(result[e*:(2*esize)]);
    let product : integer = SInt(SignedSat{2*esize}(2 * element1 * element2));
    let res : integer = element3 - product;
    result[e*:(2*esize)] = SignedSat{2*esize}(res);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

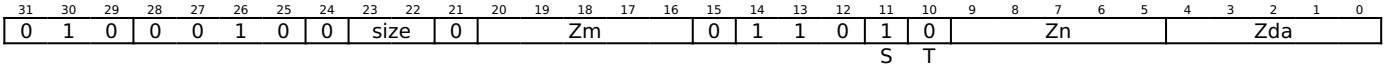
SQDMLSLB (vectors)

Signed saturating doubling multiply-subtract (bottom)

This instruction multiplies the even-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and then doubles the result. Each intermediate value is saturated to the double-width N-bit value's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$ and is then destructively subtracted from the overlapping double-width elements of the addend and destination vector. Each destination element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMLSLB <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel1 : integer = 0;
let sel2 : integer = 0;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2 * e + sel1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2 * e + sel2)*:(esize DIV 2)]);
    let element3 : integer = SInt(result[e*:esize]);
    let product : integer = SInt(SignedSat{esize}(2 * element1 * element2));
    result[e*:esize] = SignedSat{esize}(element3 - product);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

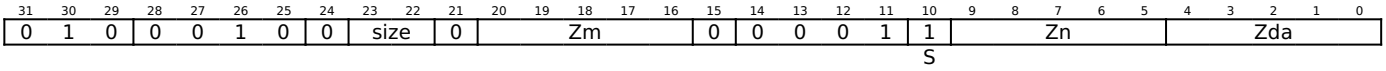
SQDMLSLBT

Signed saturating doubling multiply-subtract (bottom × top)

This instruction multiplies the even-numbered signed elements of the first source vector by the corresponding odd-numbered signed elements of the second source vector, and then doubles the result. Each intermediate value is saturated to the double-width N-bit value's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$ and is then destructively subtracted from the overlapping double-width elements of the addend and destination vector. Each destination element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMLSLBT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel1 : integer = 0;
let sel2 : integer = 1;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2 * e + sel1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2 * e + sel2)*:(esize DIV 2)]);
    let element3 : integer = SInt(result[e*:esize]);
    let product : integer = SInt(SignedSat{esize}(2 * element1 * element2));
    result[e*:esize] = SignedSat{esize}(element3 - product);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

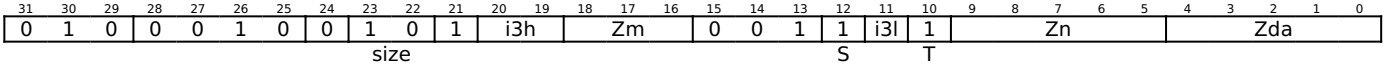
SQDMLSLT (indexed)

Signed saturating doubling multiply-subtract by indexed element (top)

This instruction multiplies the odd-numbered signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment, and then doubles the result. Each intermediate value is saturated to the double-width N-bit value's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$ and is then destructively subtracted from the overlapping double-width elements of the addend and destination vector. Each destination element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.
The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

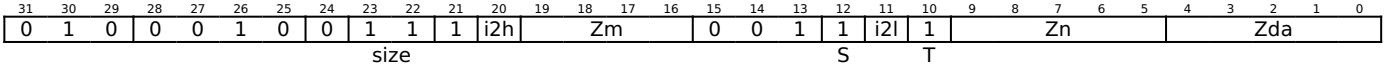
```
SQDMLSLT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMLSLT <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let element3 : integer = SInt(result[e*:(2*esize)]);
    let product : integer = SInt(SignedSat{2*esize}(2 * element1 * element2));
    let res : integer = element3 - product;
    result[e*:(2*esize)] = SignedSat{2*esize}(res);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

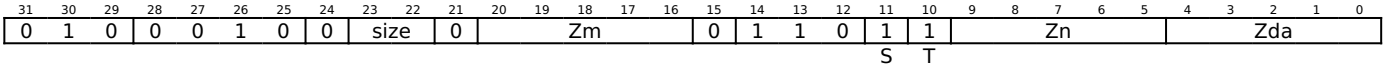
SQDMLSLT (vectors)

Signed saturating doubling multiply-subtract (top)

This instruction multiplies the odd-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and then doubles the result. Each intermediate value is saturated to the double-width N-bit value's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$ and is then destructively subtracted from the overlapping double-width elements of the addend and destination vector. Each destination element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMLSLT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel1 : integer = 1;
let sel2 : integer = 1;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2 * e + sel1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2 * e + sel2)*:(esize DIV 2)]);
    let element3 : integer = SInt(result[e*:esize]);
    let product : integer = SInt(SignedSat{esize}(2 * element1 * element2));
    result[e*:esize] = SignedSat{esize}(element3 - product);
end;

Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMULH (indexed)

Signed saturating doubling multiply high by indexed element

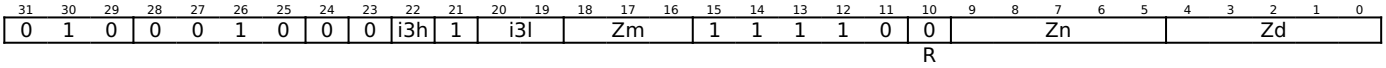
This instruction multiplies all signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment, then doubles and places the most significant half of the result in the corresponding elements of the destination vector register. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 3 classes: 16-bit , 32-bit and 64-bit

16-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

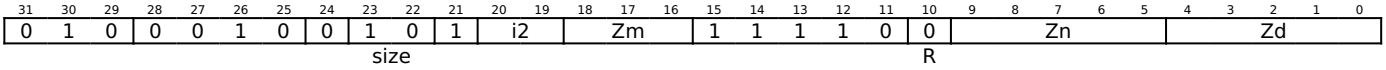
```
SQDMULH <Zd>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

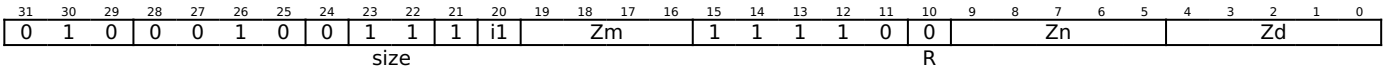
```
SQDMULH <Zd>.S, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMULH <Zd>.D, <Zn>.D, <Zm>.D[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "16-bit" and "32-bit" variants: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "16-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "32-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2" field. For the "64-bit" variant: is the element index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[s*:esize]);
    let res : integer = 2 * element1 * element2;
    result[e*:esize] = SignedSat{esize}(res >> esize);

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

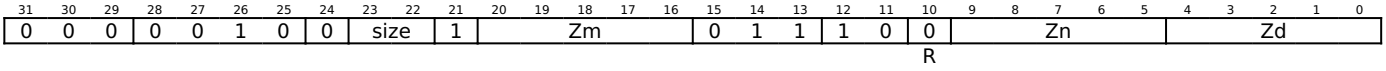
SQDMULH (vectors)

Signed saturating doubling multiply high (unpredicated)

This instruction multiplies the signed elements of the first source vector by the corresponding elements of the second source vector, and then doubles the result and places the most significant half of the results in the corresponding elements of the destination vector register. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMULH <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    let res : integer = 2 * element1 * element2;
    result[e*:esize] = SignedSat{esize}(res >> esize);

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMULLB (indexed)

Signed saturating doubling multiply by indexed element (bottom)

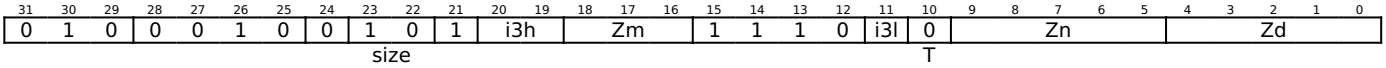
This instruction multiplies the even-numbered signed elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment, and then doubles and places the results in overlapping double-width elements of the destination vector register. Each result element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

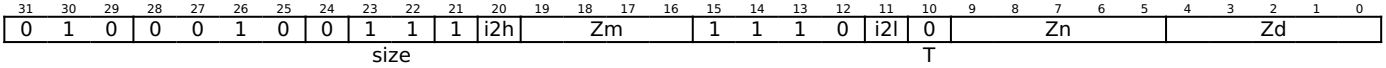
```
SQDMULLB <Zd>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 0;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMULLB <Zd>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 0;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let res : integer = 2 * element1 * element2;
    result[e*:(2*esize)] = SignedSat{2*esize}(res);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

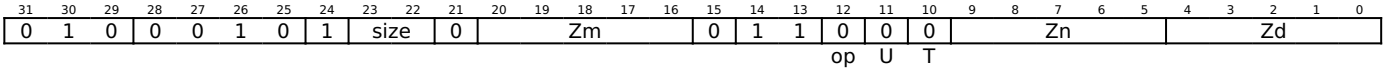
SQDMULLB (vectors)

Signed saturating doubling multiply (bottom)

This instruction multiplies the even-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and then doubles the result and places the results in the overlapping double-width elements of the destination vector. Each result element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMULLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let res : integer = 2 * element1 * element2;
    result[e*:esize] = SignedSat{esize}(res);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMULLT (indexed)

Signed saturating doubling multiply by indexed element (top)

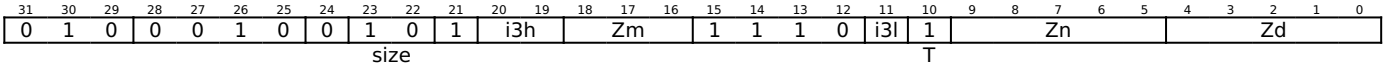
This instruction multiplies the odd-numbered signed elements within each 128-bit segment of the first source vector by the specified element in the corresponding second source vector segment, and then doubles and places the results in overlapping double-width elements of the destination vector register. Each result element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

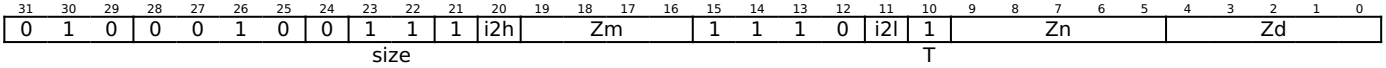
```
SQDMULLT <Zd>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 1;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMULLT <Zd>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 1;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = SInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = SInt(operand2[(2 * s + index)*:esize]);
    let res : integer = 2 * element1 * element2;
    result[e*:(2*esize)] = SignedSat{2*esize}(res);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

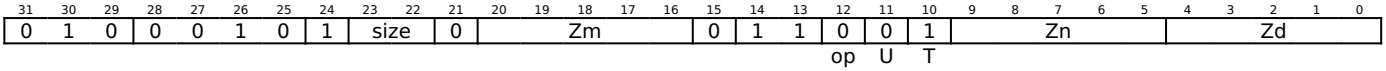
SQDMULLT (vectors)

Signed saturating doubling multiply (top)

This instruction multiplies the odd-numbered signed elements of the first source vector by the corresponding elements of the second source vector, and then doubles and places the results in the overlapping double-width elements of the destination vector. Each result element is saturated to the double-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQDMULLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = SInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let res : integer = 2 * element1 * element2;
    result[e*:esize] = SignedSat{esize}(res);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQINCB

Signed saturating increment scalar by multiple of 8-bit predicate constraint element count

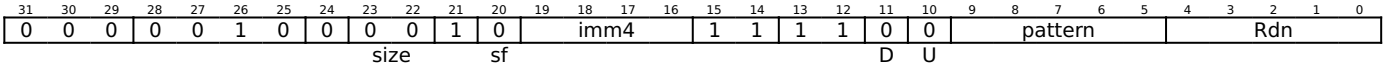
This instruction determines the number of Active 8-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment the scalar destination. The result is saturated to the source general-purpose register's signed integer range. A 32-bit saturated result is then sign-extended to 64 bits.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

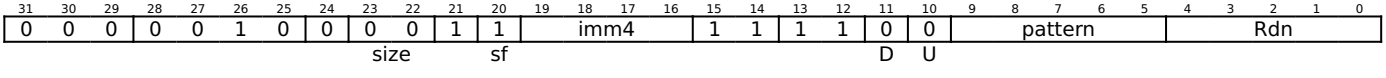
```
SQINCB <Xdn>, <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SQINCB <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Xdn>	Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.
<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) + (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) + (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQINCD (scalar)

Signed saturating increment scalar by multiple of 64-bit predicate constraint element count

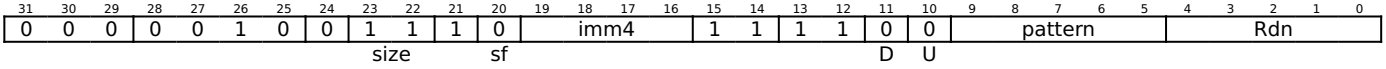
This instruction determines the number of Active 64-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment the scalar destination. The result is saturated to the source general-purpose register's signed integer range. A 32-bit saturated result is then sign-extended to 64 bits.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

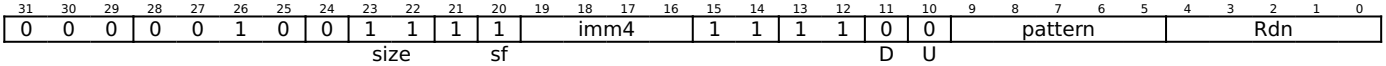
```
SQINCD <Xdn>, <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SQINCD <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Xdn>	Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.
<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) + (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) + (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQINCD (vector)

Signed saturating increment vector by multiple of 64-bit predicate constraint element count

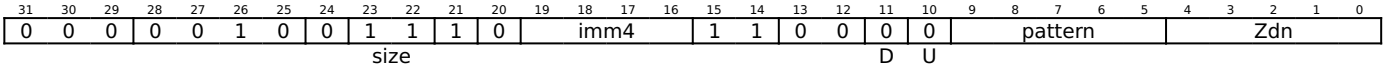
This instruction determines the number of Active 64-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment all destination vector elements. The results are saturated to the 64-bit signed integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQINCD <Zdn>.D{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) + (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) + (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQINCH (scalar)

Signed saturating increment scalar by multiple of 16-bit predicate constraint element count

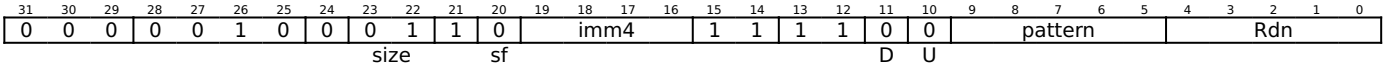
This instruction determines the number of Active 16-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment the scalar destination. The result is saturated to the source general-purpose register's signed integer range. A 32-bit saturated result is then sign-extended to 64 bits.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

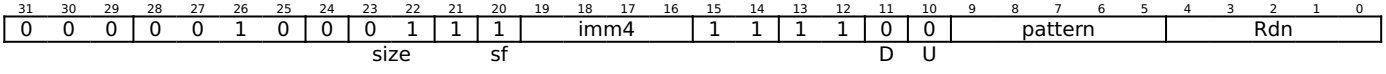
```
SQINCH <Xdn>, <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SQINCH <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Xdn>	Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.
<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) + (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) + (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQINCH (vector)

Signed saturating increment vector by multiple of 16-bit predicate constraint element count

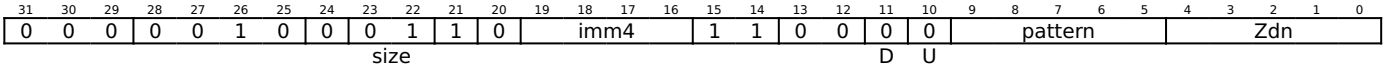
This instruction determines the number of Active 16-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment all destination vector elements. The results are saturated to the 16-bit signed integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQINCH <Zdn>.H{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) + (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) + (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQINCP (scalar)

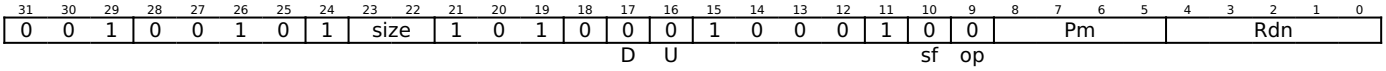
Signed saturating increment scalar by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to increment the scalar destination. The result is saturated to the source general-purpose register's signed integer range. A 32-bit saturated result is then sign-extended to 64 bits.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

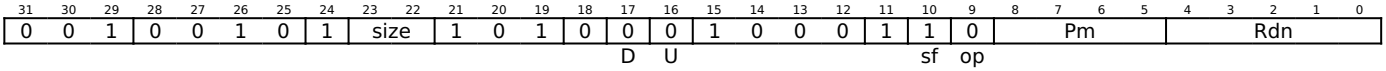
```
SQINCP <Xdn>, <Pm>.<T>, <Wdn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Rdn);
let unsigned : boolean = FALSE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SQINCP <Xdn>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Rdn);
let unsigned : boolean = FALSE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Wdn> Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(ssize) = X{}(dn);
let operand2 : bits(PL) = P{}(m);
var result : bits(ssize);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) + count);
    X{64}(dn) = ZeroExtend{64}(result);
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) + count);
    X{64}(dn) = SignExtend{64}(result);
end;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQINCP (vector)

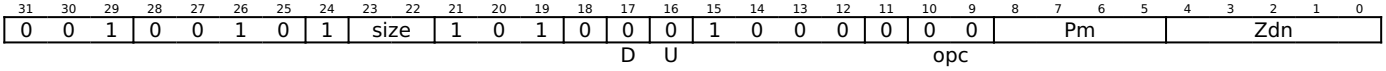
Signed saturating increment vector by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to increment all destination vector elements. The results are saturated to the element signed integer range.

The predicate size specifier may be omitted in assembler source code, but this is deprecated and will be prohibited in a future release of the architecture.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQINCP <Zdn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Zdn);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(PL) = P{}(m);
var result : bits(VL);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(opelt) + count);
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(opelt) + count);
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQINCW (scalar)

Signed saturating increment scalar by multiple of 32-bit predicate constraint element count

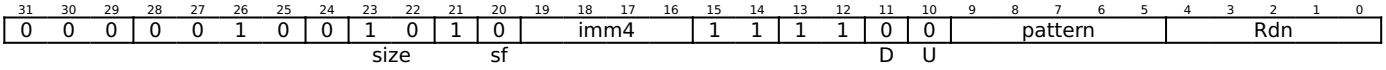
This instruction determines the number of Active 32-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment the scalar destination. The result is saturated to the source general-purpose register's signed integer range. A 32-bit saturated result is then sign-extended to 64 bits.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

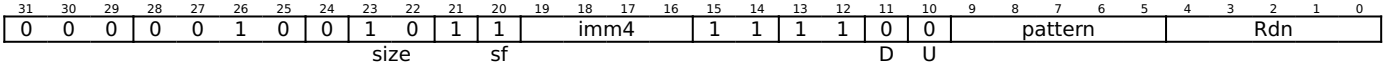
```
SQINCW <Xdn>, <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
SQINCW <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Xdn>	Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.
<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) + (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) + (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQINCW (vector)

Signed saturating increment vector by multiple of 32-bit predicate constraint element count

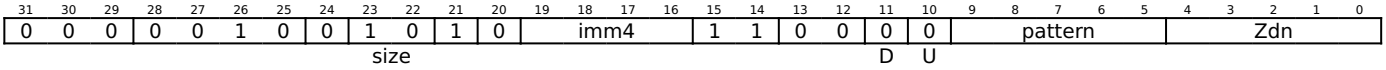
This instruction determines the number of Active 32-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment all destination vector elements. The results are saturated to the 32-bit signed integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQINCW <Zdn>.S{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) + (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) + (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQNEG

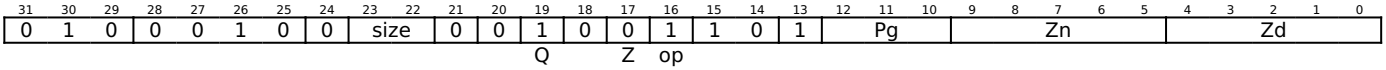
Signed saturating negate

This instruction negates the signed integer value in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE2 || FEAT_SME)



Encoding

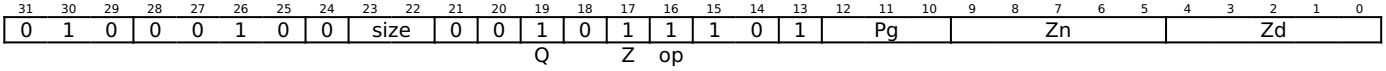
```
SQNEG <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
SQNEG <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        var element : integer = SInt(operand[e*:esize]);
        element = -element;
        result[e*:esize] = SignedSat{esize}(element);

    end;
end;
Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:
The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDCMLAH (indexed)

Saturating rounding doubling complex integer multiply-add high by indexed element

This instruction multiplies without saturation the duplicated real components for rotations 0 and 180, or imaginary components for rotations 90 and 270, of the integral numbers in each 128-bit segment of the first source vector by the specified complex number in the corresponding the second source vector segment rotated by 0, 90, 180, or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation.

This instruction then doubles and adds the products to the corresponding components of the complex numbers in the addend vector, and destructively places the most significant rounded half of the results in the corresponding elements of the addend vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

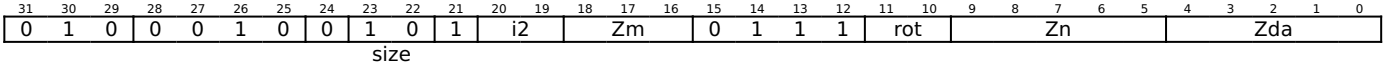
These transformations permit the creation of a variety of multiply-add and multiply-subtract operations on complex numbers by combining two of these instructions with the same vector operands but with rotations that are 90 degrees apart.

Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element.

It has encodings from 2 classes: 16-bit and 32-bit

16-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

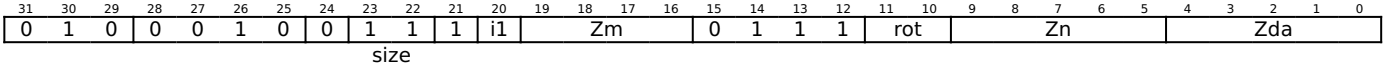
```
SQRDCMLAH <Zda>.H, <Zn>.H, <Zm>.H[<imm>], <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT(rot[0]));
let sub_r : boolean = (rot[0] != rot[1]);
let sub_i : boolean = (rot[1] == '1');
```

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRDCMLAH <Zda>.S, <Zn>.S, <Zm>.S[<imm>], <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT(rot[0]));
let sub_r : boolean = (rot[0] != rot[1]);
let sub_i : boolean = (rot[1] == '1');
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "16-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<imm> For the "16-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2" field.
For the "32-bit" variant: is the element index, in the range 0 to 1, encoded in the "i1" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
00	#0
01	#90
10	#180
11	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (2 * esize);
let pairspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

var res_r, res_i : integer;
for p = 0 to pairs-1 do
    let segmentbase : integer = p - (p MOD pairspersegment);
    let s : integer = segmentbase + index;
    let elt1_a : integer = SInt(operand1[(2 * p + sel_a)*:esize]);
    let elt2_a : integer = SInt(operand2[(2 * s + sel_a)*:esize]);
    let elt2_b : integer = SInt(operand2[(2 * s + sel_b)*:esize]);
    let elt3_r : bits(esize) = operand3[(2 * p + 0)*:esize];
    let elt3_i : bits(esize) = operand3[(2 * p + 1)*:esize];
    let product_r : integer =  elt1_a * elt2_a;
    let product_i : integer =  elt1_a * elt2_b;
    if sub_r then
        res_r = (SInt(elt3_r) << esize) - 2 * product_r;
    else
        res_r = (SInt(elt3_r) << esize) + 2 * product_r;
    end;
    res_r = (res_r + (1 << (esize-1))) >> esize;
    result[(2 * p + 0)*:esize] = SignedSat{esize}(res_r);
    if sub_i then
        res_i = (SInt(elt3_i) << esize) - 2 * product_i;
    else
        res_i = (SInt(elt3_i) << esize) + 2 * product_i;
    end;
    res_i = (res_i + (1 << (esize-1))) >> esize;
    result[(2 * p + 1)*:esize] = SignedSat{esize}(res_i);

end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDCMLAH (vectors)

Saturating rounding doubling complex integer multiply-add high

This instruction multiplies without saturation the duplicated real components for rotations 0 and 180, or imaginary components for rotations 90 and 270, of the integral numbers in the first source vector by the corresponding complex number in the second source vector rotated by 0, 90, 180, or 270 degrees in the direction from the positive real axis towards the positive imaginary axis, when considered in polar representation.

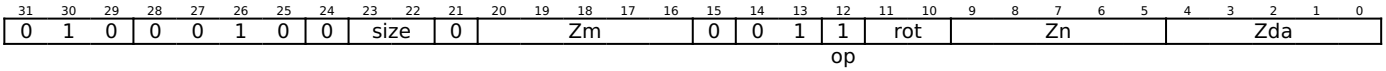
This instruction then doubles and adds the products to the corresponding components of the complex numbers in the addend vector, and destructively places the most significant rounded half of the results in the corresponding elements of the addend vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

These transformations permit the creation of a variety of multiply-add and multiply-subtract operations on complex numbers by combining two of these instructions with the same vector operands but with rotations that are 90 degrees apart.

Each complex number is represented in a vector register as an even/odd pair of elements with the real part in the even-numbered element and the imaginary part in the odd-numbered element.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRDCMLAH <Zda>.<T>, <Zn>.<T>, <Zm>.<T>, <const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel_a : integer = UInt(rot[0]);
let sel_b : integer = UInt(NOT(rot[0]));
let sub_r : boolean = (rot[0] != rot[1]);
let sub_i : boolean = (rot[1] == '1');
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<const> Is the const specifier, encoded in “rot”:

rot	<const>
00	#0
01	#90
10	#180
11	#270

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

var res_r, res_i : integer;
for p = 0 to pairs-1 do
    let elt1_a : integer = SInt(operand1[(2 * p + sel_a)*:esize]);
    let elt2_a : integer = SInt(operand2[(2 * p + sel_a)*:esize]);
    let elt2_b : integer = SInt(operand2[(2 * p + sel_b)*:esize]);
    let elt3_r : bits(esize) = operand3[(2 * p + 0)*:esize];
    let elt3_i : bits(esize) = operand3[(2 * p + 1)*:esize];
    let product_r : integer =  elt1_a * elt2_a;
    let product_i : integer =  elt1_a * elt2_b;
    if sub_r then
        res_r = (SInt(elt3_r) << esize) - 2 * product_r;
    else
        res_r = (SInt(elt3_r) << esize) + 2 * product_r;
    end;
    res_r = (res_r + (1 << (esize-1))) >> esize;
    result[(2 * p + 0)*:esize] = SignedSat{esize}(res_r);
    if sub_i then
        res_i = (SInt(elt3_i) << esize) - 2 * product_i;
    else
        res_i = (SInt(elt3_i) << esize) + 2 * product_i;
    end;
    res_i = (res_i + (1 << (esize-1))) >> esize;
    result[(2 * p + 1)*:esize] = SignedSat{esize}(res_i);

end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDMLAH (indexed)

Signed saturating rounding doubling multiply-add high by indexed element

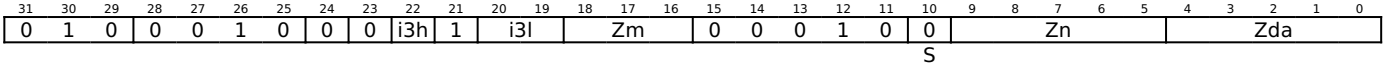
This instruction multiplies all signed elements within each 128-bit segment of the first source vector by the specified signed element of the corresponding second source vector segment, and then doubles the result and destructively adds the rounded high half of each result to the corresponding elements of the addend and destination vector. Each destination element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 3 classes: 16-bit , 32-bit and 64-bit

16-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

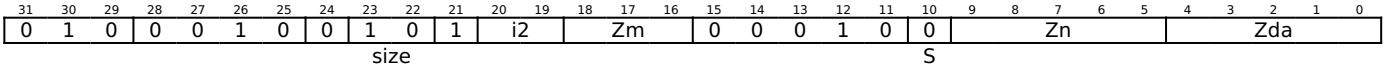
```
SQRDMLAH <Zda>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

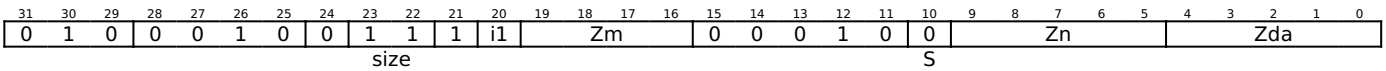
```
SQRDMLAH <Zda>.S, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRDMLAH <Zda>.D, <Zn>.D, <Zm>.D[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "16-bit" and "32-bit" variants: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "16-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "32-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2" field. For the "64-bit" variant: is the element index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[s*:esize]);
    let element3 : integer = SInt(operand3[e*:esize]);
    let res : integer = (element3 << esize) + (2 * element1 * element2);
    result[e*:esize] = SignedSat{esize}((res + (1 << (esize - 1))) >> esize);

end;
Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

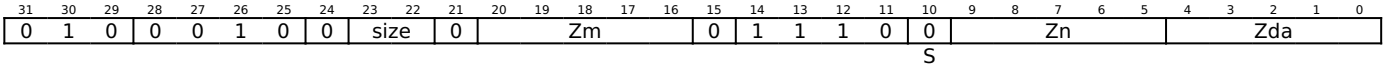
SQRDMLAH (vectors)

Signed saturating rounding doubling multiply-add high (unpredicated)

This instruction multiplies the signed elements of the first source vector by the corresponding elements of the second source vector, and then doubles the result and destructively adds the rounded high half of each result to the corresponding elements of the addend and destination vector. Each destination element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRDMLAH <Zda>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    let element3 : integer = SInt(operand3[e*:esize]);
    let res : integer = (element3 << esize) + (2 * element1 * element2);
    result[e*:esize] = SignedSat{esize}((res + (1 << (esize - 1))) >> esize);

end;
Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDMLSH (indexed)

Signed saturating rounding doubling multiply-subtract high by indexed element

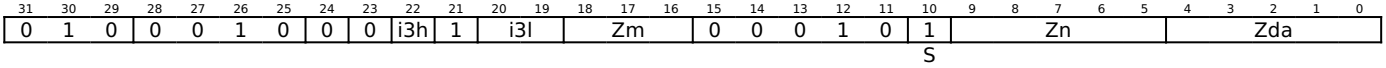
This instruction multiplies all signed elements within each 128-bit segment of the first source vector by the specified signed element of the corresponding second source vector segment, and then doubles the result and destructively subtracts the rounded high half of each result to the corresponding elements of the addend and destination vector. Each destination element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 3 classes: 16-bit , 32-bit and 64-bit

16-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

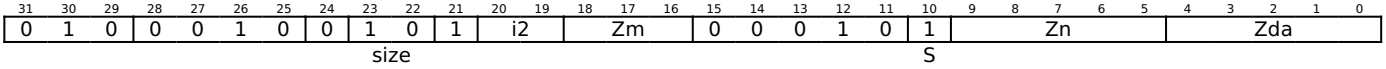
```
SQRDMLSH <Zda>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

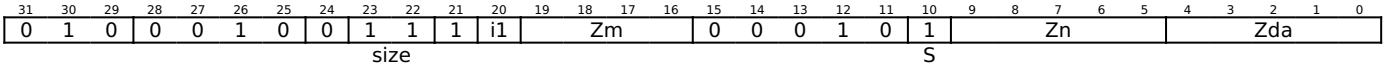
```
SQRDMLSH <Zda>.S, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRDMLSH <Zda>.D, <Zn>.D, <Zm>.D[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "16-bit" and "32-bit" variants: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "16-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "32-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2" field. For the "64-bit" variant: is the element index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[s*:esize]);
    let element3 : integer = SInt(operand3[e*:esize]);
    let res : integer = (element3 << esize) - (2 * element1 * element2);
    result[e*:esize] = SignedSat{esize}((res + (1 << (esize - 1))) >> esize);

end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

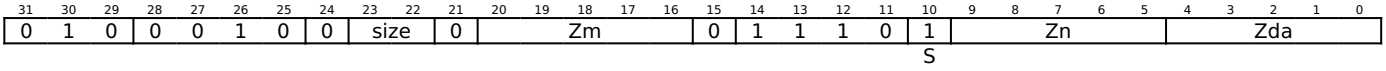
SQRDMLSH (vectors)

Signed saturating rounding doubling multiply-subtract high (unpredicated)

This instruction multiplies the signed elements of the first source vector by the corresponding elements of the second source vector, and then doubles the result and destructively subtracts the rounded high half of each result from the corresponding elements of the addend and destination vector. Each destination element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRDMLSH <Zda>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    let element3 : integer = SInt(operand3[e*:esize]);
    let res : integer = (element3 << esize) - (2 * element1 * element2);
    result[e*:esize] = SignedSat{esize}((res + (1 << (esize - 1))) >> esize);

end;
Z{VL}(da) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRDMULH (indexed)

Signed saturating rounding doubling multiply high by indexed element

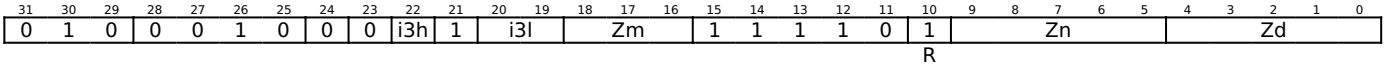
This instruction multiplies all signed elements within each 128-bit segment of the first source vector by the specified signed element in the corresponding second source vector segment, then doubles and places the most significant rounded half of the result in the corresponding elements of the destination vector register. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 3 classes: 16-bit , 32-bit and 64-bit

16-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

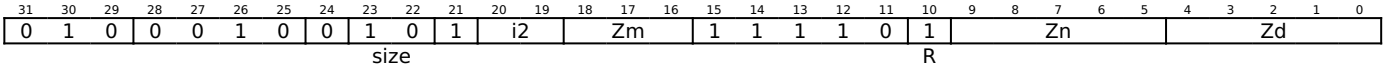
```
SQRDMULH <Zd>.H, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

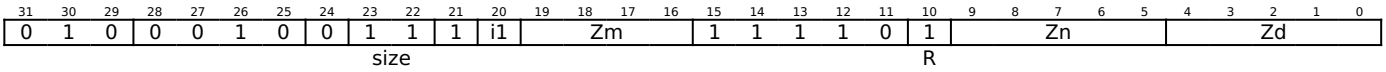
```
SQRDMULH <Zd>.S, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRDMULH <Zd>.D, <Zn>.D, <Zm>.D[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "16-bit" and "32-bit" variants: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "16-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "32-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2" field. For the "64-bit" variant: is the element index, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[s*:esize]);
    let res : integer = 2 * element1 * element2;
    result[e*:esize] = SignedSat{esize}((res + (1 << (esize - 1))) >> esize);

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

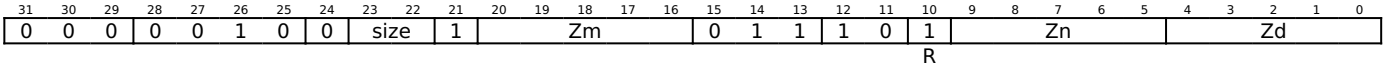
SQRDMULH (vectors)

Signed saturating rounding doubling multiply high (unpredicated)

This instruction multiplies the signed elements of the first source vector by the corresponding elements of the second source vector, and then doubles the result and places the most significant rounded half of the result in the corresponding elements of the destination vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRDMULH <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    let res : integer = 2 * element1 * element2;
    result[e*:esize] = SignedSat{esize}((res + (1 << (esize - 1))) >> esize);

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

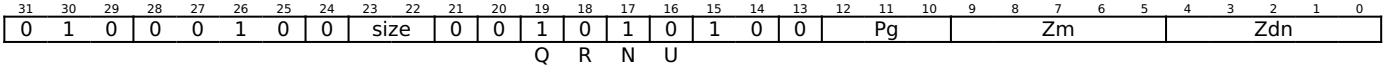
SQRSHL

Signed saturating rounding shift left (predicated)

This instruction shifts Active signed elements of the first source vector by corresponding elements of the second source vector, and destructively places the rounded results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRSHL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = SInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + (1 << (shift - 1))) >> shift;
        end;
        result[e*:esize] = SignedSat{esize}(res);
    else
        result[e*:esize] = operand1[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

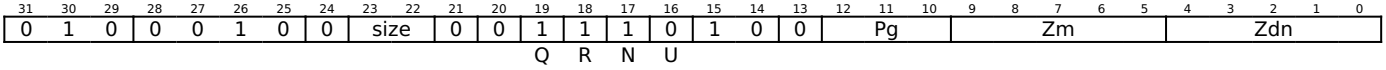
SQRSHLR

Signed saturating rounding shift left reversed (predicated)

This instruction shifts Active signed elements of the second source vector by corresponding elements of the first source vector and destructively places the rounded results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRSHLR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let operand2 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = SInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + (1 << (shift - 1))) >> shift;
        end;
        result[e*:esize] = SignedSat{esize}(res);
    else
        result[e*:esize] = operand2[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRSHRN

Signed saturating rounding shift right narrow by immediate to interleaved integer

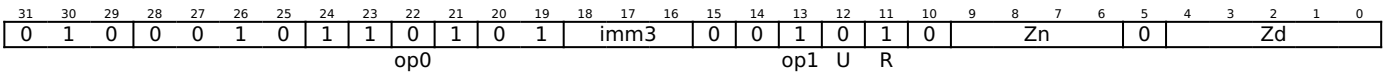
This instruction shifts right by an immediate value the signed integer value in each element of the two source vectors, and places the two-way interleaved rounded results in the half-width destination elements. Each result element is saturated to the half-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The immediate shift amount is an unsigned value in the range 1 to number of bits per destination element.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit and 16-bit

8-bit

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

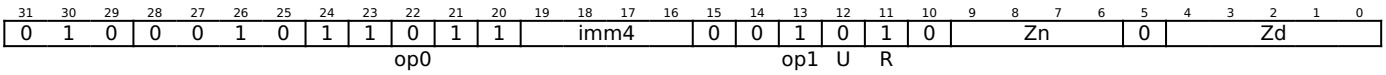
```
SQRSHRN <Zd>.B, { <Zn1>.H-<Zn2>.H }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = esize - UInt(imm3);
```

16-bit

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
SQRSHRN <Zd>.H, { <Zn1>.S-<Zn2>.S }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = esize - UInt(imm4);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<const>	For the "8-bit" variant: is the immediate shift amount, in the range 1 to 8, encoded as 8 minus "imm3". For the "16-bit" variant: is the immediate shift amount, in the range 1 to 16, encoded as 16 minus "imm4".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 1 do
        let operand : bits(VL) = Z{}(n+i);
        let element : bits(2 * esize) = operand[e*:(2 * esize)];
        let res : integer = (SInt(element) + (1 << (shift-1))) >> shift;
        result[(2*e + i)*:esize] = SignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

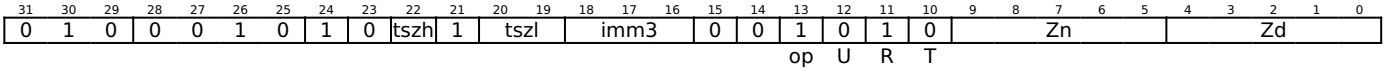
SQRSHRNB

Signed saturating rounding shift right narrow by immediate (bottom)

This instruction shifts each signed integer value in the source vector elements right by an immediate value, and places the rounded results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. Each result element is saturated to the half-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRSHRNB <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = (SInt(element) + (1 << (shift-1))) >> shift;
    result[(2*e + 0)*:esize] = SignedSat{esize}(res);
    result[(2*e + 1)*:esize] = Zeros{esize};

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

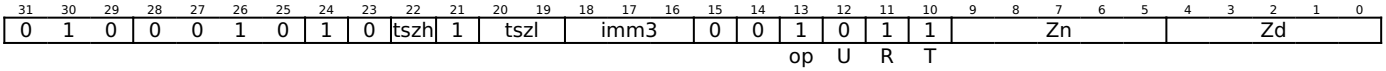
SQRSHRNT

Signed saturating rounding shift right narrow by immediate (top)

This instruction shifts each signed integer value in the source vector elements right by an immediate value, and places the rounded results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. Each result element is saturated to the half-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRSHRNT <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = (SInt(element) + (1 << (shift-1))) >> shift;
    result[(2*e + 1)*:esize] = SignedSat{esize}(res);

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRSHRUN

Signed saturating rounding shift right unsigned narrow by immediate and interleave

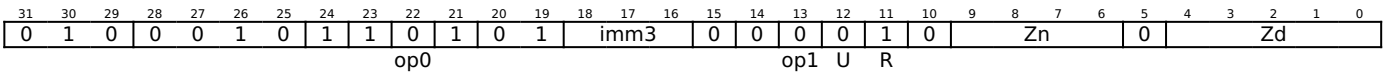
This instruction shifts right by an immediate value the signed integer value in each element of the two source vectors, and places the two-way interleaved rounded results in the half-width destination elements. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per destination element.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit and 16-bit

8-bit

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

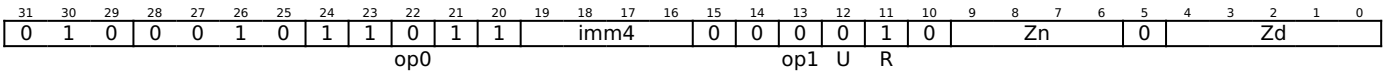
```
SQRSHRUN <Zd>.B, { <Zn1>.H-<Zn2>.H }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = esize - UInt(imm3);
```

16-bit

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
SQRSHRUN <Zd>.H, { <Zn1>.S-<Zn2>.S }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = esize - UInt(imm4);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<const>	For the "8-bit" variant: is the immediate shift amount, in the range 1 to 8, encoded as 8 minus "imm3". For the "16-bit" variant: is the immediate shift amount, in the range 1 to 16, encoded as 16 minus "imm4".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 1 do
        let operand : bits(VL) = Z{}(n+i);
        let element : bits(2 * esize) = operand[e*:(2 * esize)];
        let res : integer = (SInt(element) + (1 << (shift-1))) >> shift;
        result[(2*e + i)*:esize] = UnsignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

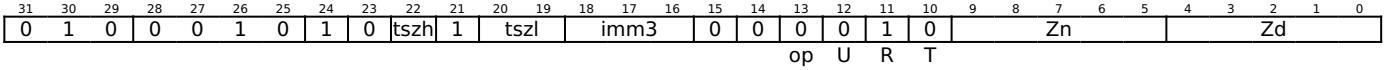
SQRSHRUNB

Signed saturating rounding shift right narrow by immediate to unsigned integer (bottom)

This instruction shifts each signed integer value in the source vector elements right by an immediate value, and places the rounded results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRSHRUNB <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = (SInt(element) + (1 << (shift-1))) >> shift;
    result[(2*e + 0)*:esize] = UnsignedSat{esize}(res);
    result[(2*e + 1)*:esize] = Zeros{esize};

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

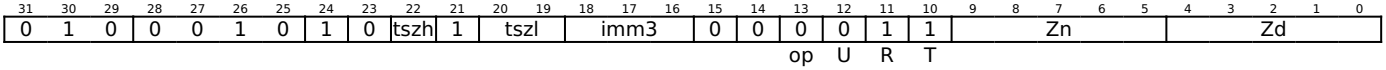
SQRSHRUNT

Signed saturating rounding shift right narrow by immediate to unsigned integer (top)

This instruction shifts each signed integer value in the source vector elements right by an immediate value, and places the rounded results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQRSHRUNT <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = (SInt(element) + (1 << (shift-1))) >> shift;
    result[(2*e + 1)*:esize] = UnsignedSat{esize}(res);

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

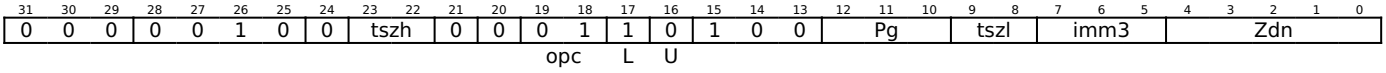
SQSHL (immediate)

Signed saturating shift left by immediate

This instruction shifts left by immediate each Active signed element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The immediate shift amount is an unsigned value in the range 0 to number of bits per element minus 1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQSHL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let shift : integer = UInt(tsize::imm3) - esize;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the immediate shift amount, in the range 0 to number of bits per element minus 1, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let mask : bits(PL) = P{}(g);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = element1 << shift;
        result[e*:esize] = SignedSat{esize}(res);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

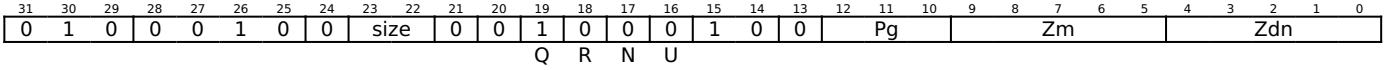
SQSHL (vectors)

Signed saturating shift left (predicated)

This instruction shifts Active signed elements of the first source vector by corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQSHL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = SInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = element >> shift;
        end;
        result[e*:esize] = SignedSat{esize}(res);
    else
        result[e*:esize] = operand1[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

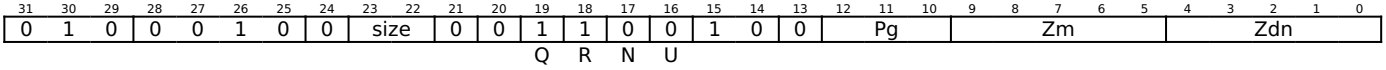
SQSHLR

Signed saturating shift left reversed (predicated)

This instruction shifts Active signed elements of the second source vector by the corresponding elements of the first source vector, and destructively places the results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQSHLR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let operand2 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = SInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = element >> shift;
        end;
        result[e*:esize] = SignedSat{esize}(res);
    else
        result[e*:esize] = operand2[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

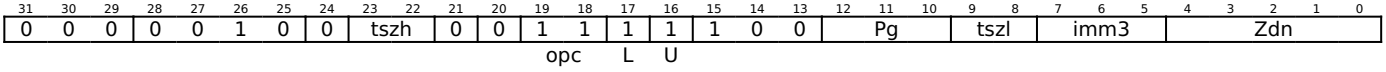
SQSHLU

Signed saturating shift left unsigned by immediate

This instruction shifts left by immediate each Active signed element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 0 to number of bits per element minus 1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQSHLU <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let shift : integer = UInt(tsize::imm3) - esize;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the immediate shift amount, in the range 0 to number of bits per element minus 1, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let mask : bits(PL) = P{}(g);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = element1 << shift;
        result[e*:esize] = UnsignedSat{esize}(res);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQSHRN

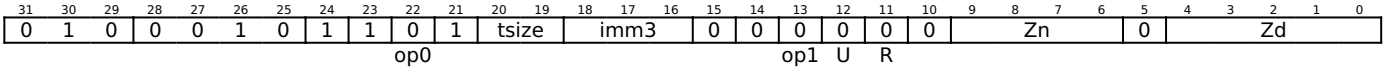
Signed saturating shift right narrow by immediate to interleaved integer

This instruction shifts right by an immediate value the signed integer value in each element of the two source vectors, and places the two-way interleaved truncated results in the half-width destination elements. Each result element is saturated to the half-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The immediate shift amount is an unsigned value in the range 1 to number of bits per destination element.

This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
SQSHRN <Zd>.<T>, { <Zn1>.<Tb>-<Zn2>.<Tb> }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if tsize == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsize”:

tsize	<T>
00	RESERVED
01	B
1x	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.

<Tb> Is the size specifier, encoded in “tsize”:

tsize	<Tb>
00	RESERVED
01	H
1x	S

<Zn2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in the "tsize:imm3" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 1 do
        let operand : bits(VL) = Z{}(n+i);
        let element : bits(2*esize) = operand[e*:(2*esize)];
        let res : integer = SInt(element) >> shift;
        result[(2*e + i)*:esize] = SignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

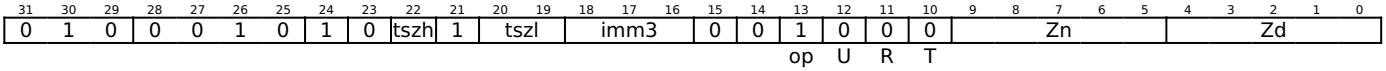
SQSHRNB

Signed saturating shift right narrow by immediate (bottom)

This instruction shifts each signed integer value in the source vector elements right by an immediate value, and places the truncated results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. Each result element is saturated to the half-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQSHRNB <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = SInt(element) >> shift;
    result[(2*e + 0)*:esize] = SignedSat{esize}(res);
    result[(2*e + 1)*:esize] = Zeros{esize};

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

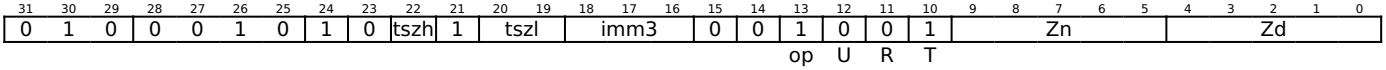
SQSHRNT

Signed saturating shift right narrow by immediate (top)

This instruction shifts each signed integer value in the source vector elements right by an immediate value, and places the truncated results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. Each result element is saturated to the half-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQSHRNT <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = SInt(element) >> shift;
    result[(2*e + 1)*:esize] = SignedSat{esize}(res);

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

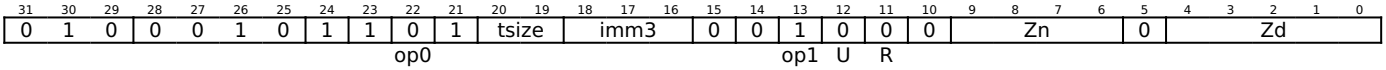
SQSHRUN

Signed saturating shift right narrow by immediate to interleaved unsigned integer

This instruction shifts right by an immediate value the signed integer value in each element of the two source vectors, and places the two-way interleaved truncated results in the half-width destination elements. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per destination element.
This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
SQSHRUN <Zd>.<T>, { <Zn1>.<Tb>-<Zn2>.<Tb> }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if tsize == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsize”:

tsize	<T>
00	RESERVED
01	B
1x	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.

<Tb> Is the size specifier, encoded in “tsize”:

tsize	<Tb>
00	RESERVED
01	H
1x	S

<Zn2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in the "tsize:imm3" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 1 do
        let operand : bits(VL) = Z{}(n+i);
        let element : bits(2*esize) = operand[e*:(2*esize)];
        let res : integer = SInt(element) >> shift;
        result[(2*e + i)*:esize] = UnsignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

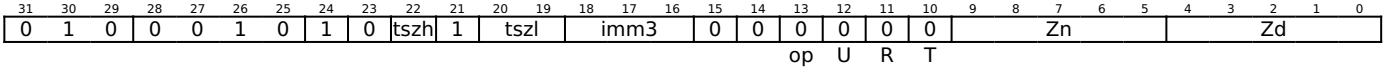
SQSHRUNB

Signed saturating shift right narrow by immediate to unsigned integer (bottom)

This instruction shifts each signed integer value in the source vector elements right by an immediate value, and places the truncated results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQSHRUNB <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = SInt(element) >> shift;
    result[(2*e + 0)*:esize] = UnsignedSat{esize}(res);
    result[(2*e + 1)*:esize] = Zeros{esize};

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

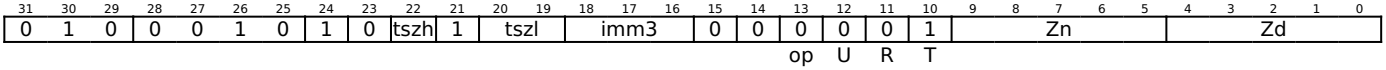
SQSHRUNT

Signed saturating shift right narrow by immediate to unsigned integer (top)

This instruction shifts each signed integer value in the source vector elements right by an immediate value, and places the truncated results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQSHRUNT <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = SInt(element) >> shift;
    result[(2*e + 1)*:esize] = UnsignedSat{esize}(res);

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQSUB (immediate)

Signed saturating subtract immediate (unpredicated)

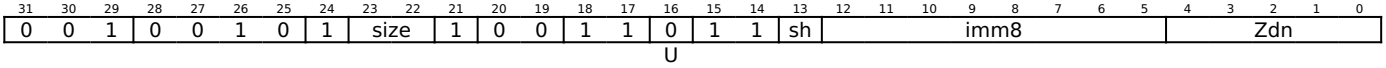
This instruction performs a signed saturating subtraction of an unsigned immediate from each element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQSUB <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    (result[e*:esize], -) = SatQ{esize}(element1 - imm, unsigned);

end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

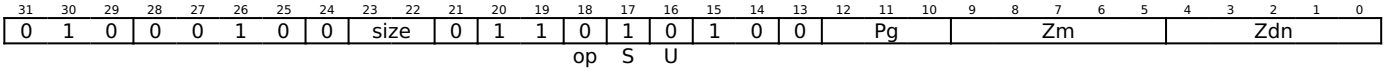
SQSUB (vectors, predicated)

Signed saturating subtract (predicated)

This instruction subtracts Active signed elements of the second source vector from corresponding signed elements of the first source vector and destructively places the results in the corresponding elements of the first source vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQSUB <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = SignedSat{esize}(element1 - element2);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

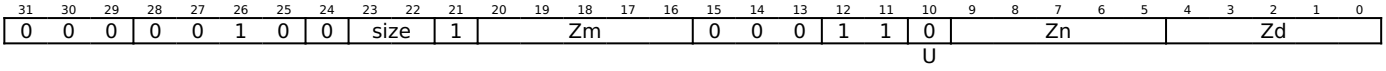
SQSUB (vectors, unpredicated)

Signed saturating subtract (unpredicated)

This instruction performs a signed saturating subtraction of all elements of the second source vector from the corresponding elements of the first source vector, and places the results in the corresponding elements of the destination vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SQSUB <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let op2elt : bits(esize) = operand2[e*:esize];
    if unsigned then
        let element1 : integer = UInt(op1elt);
        let element2 : integer = UInt(op2elt);
        (result[e*:esize], -) = UnsignedSatQ{esize}(element1 - element2);
    else
        let element1 : integer = SInt(op1elt);
        let element2 : integer = SInt(op2elt);
        (result[e*:esize], -) = SignedSatQ{esize}(element1 - element2);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

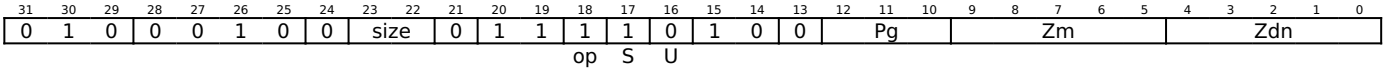
SQSUBR

Signed saturating subtract reversed (predicated)

This instruction subtracts Active signed elements of the first source vector from corresponding signed elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQSUBR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = SignedSat{esize}(element2 - element1);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

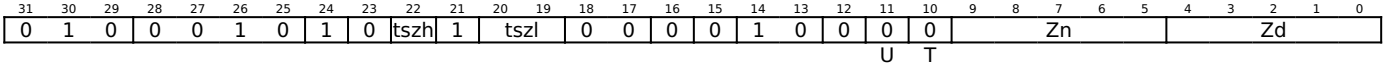
SQXTNB

Signed saturating extract narrow (bottom)

This instruction saturates the signed integer value in each source element to half the original source element width, and places the results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQXTNB <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if !(tsize IN {'001', '010', '100'}) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	10	H
0	11	RESERVED
1	00	S
1	01	RESERVED
1	1x	RESERVED

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	10	S
0	11	RESERVED
1	00	D
1	01	RESERVED
1	1x	RESERVED

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
var result : bits(VL);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let res : bits(halfesize) = SignedSat{halfesize}(element1);
    result[(2*e + 0)*:halfesize] = res;
    result[(2*e + 1)*:halfesize] = Zeros{halfesize};

end;
Z{VL}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

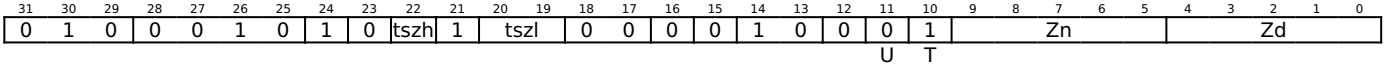
SQXTNT

Signed saturating extract narrow (top)

This instruction saturates the signed integer value in each source element to half the original source element width, and places the results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQXTNT <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if !(tsize IN {'001', '010', '100'}) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	10	H
0	11	RESERVED
1	00	S
1	01	RESERVED
1	1x	RESERVED

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	10	S
0	11	RESERVED
1	00	D
1	01	RESERVED
1	1x	RESERVED

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let res : bits(halfesize) = SignedSat{halfesize}(element1);
    result[(2*e + 1)*:halfesize] = res;

end;
Z{VL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

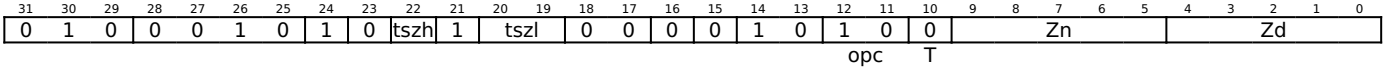
SQXTUNB

Signed saturating extract narrow to unsigned integer (bottom)

This instruction saturates the signed integer value in each source element to an unsigned integer value that is half the original source element width, and places the results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQXTUNB <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if !(tsize IN {'001', '010', '100'}) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	10	H
0	11	RESERVED
1	00	S
1	01	RESERVED
1	1x	RESERVED

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	10	S
0	11	RESERVED
1	00	D
1	01	RESERVED
1	1x	RESERVED

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
var result : bits(VL);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let res : bits(halfesize) = UnsignedSat{halfesize}(element1);
    result[(2*e + 0)*:halfesize] = res;
    result[(2*e + 1)*:halfesize] = Zeros{halfesize};

end;
Z{VL}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

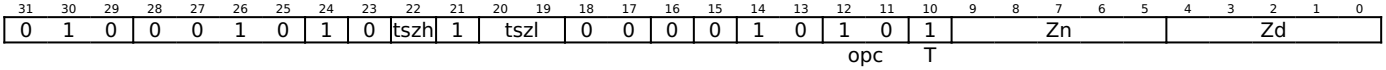
SQXTUNT

Signed saturating extract narrow to unsigned integer (top)

This instruction saturates the signed integer value in each source element to an unsigned integer value that is half the original source element width, and places the results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SQXTUNT <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if !(tsize IN {'001', '010', '100'}) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	10	H
0	11	RESERVED
1	00	S
1	01	RESERVED
1	1x	RESERVED

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	10	S
0	11	RESERVED
1	00	D
1	01	RESERVED
1	1x	RESERVED

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let res : bits(halfesize) = UnsignedSat{halfesize}(element1);
    result[(2*e + 1)*:halfesize] = res;

end;
Z{VL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

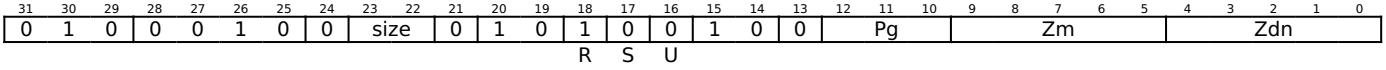
SRHADD

Signed rounding halving add

This instruction adds Active signed elements of the first source vector to corresponding signed elements of the second source vector, shifts right one bit, and destructively places the rounded results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SRHADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = (element1 + element2 + 1) >> 1;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

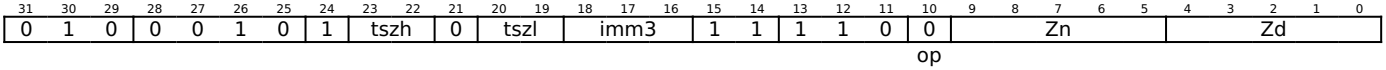
SRI

Shift right and insert (immediate)

This instruction shifts each source vector element right by an immediate value, and inserts the result into the corresponding vector element in the destination vector register, merging the shifted bits from each source element with existing bits in each destination vector element. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SRI <Zd>.<T>, <Zn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    let element1 : bits(esize) = result[e*:esize];
    let element2 : bits(esize) = operand[e*:esize];
    let mask : bits(esize) = LSR(Ones{esize}, shift);
    let shiftedval : bits(esize) = LSR(element2, shift);
    result[e*:esize] = (element1 AND (NOT mask)) OR shiftedval;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

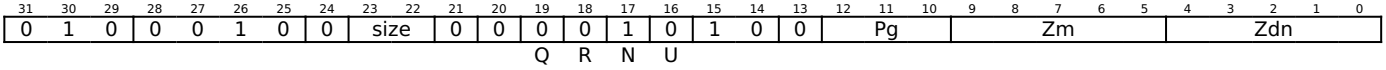
SRSHL

Signed rounding shift left (predicated)

This instruction shifts Active signed elements of the first source vector by corresponding elements of the second source vector, and destructively places the rounded results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SRSHL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = SInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + (1 << (shift - 1))) >> shift;
        end;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

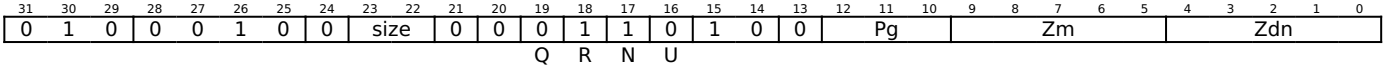
SRSHLR

Signed rounding shift left reversed (predicated)

This instruction shifts Active signed elements of the second source vector by corresponding elements of the first source vector and destructively places the rounded results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SRSHLR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let operand2 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = SInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + (1 << (shift - 1))) >> shift;
        end;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand2[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

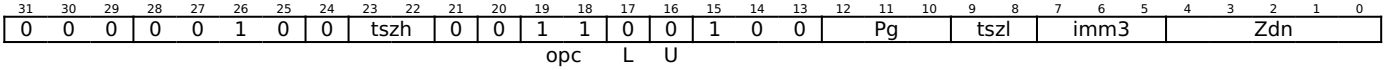
SRSHR

Signed rounding shift right by immediate

This instruction shifts right by immediate each Active signed element of the source vector, and destructively places the rounded results in the corresponding elements of the source vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SRSHR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let mask : bits(PL) = P{}(g);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = (element1 + (1 << (shift - 1))) >> shift;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

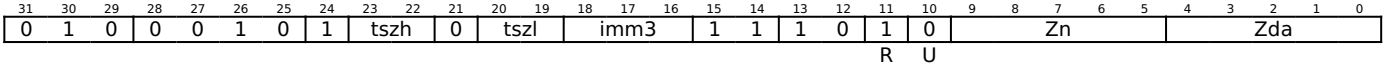
SRSRA

Signed rounding shift right and accumulate (immediate)

This instruction shifts right by immediate each signed element of the source vector, preserving the sign bit, and destructively adds the rounded intermediate result to the corresponding elements of the addend vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SRSRA <Zda>.<T>, <Zn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let da : integer = UInt(Zda);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : integer = (SInt(operand1[e*:esize]) + (1 << (shift - 1))) >> shift;
    result[e*:esize] = operand2[e*:esize] + element[esize-1:0];
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

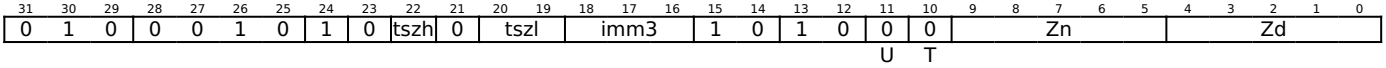
SSHLLB

Signed shift left long by immediate (bottom)

This instruction shifts left by immediate each even-numbered signed element of the source vector, and places the results in the overlapping double-width elements of the destination vector. The immediate shift amount is an unsigned value in the range 0 to number of bits per element minus 1. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SSHLLB <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = UInt(tsize::imm3) - esize;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<const> Is the immediate shift amount, in the range 0 to number of bits per element minus 1, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[(2*e + 0)*:esize];
    let shifted_value : integer = SInt(element) << shift;
    result[e*:(2*esize)] = shifted_value[2*esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

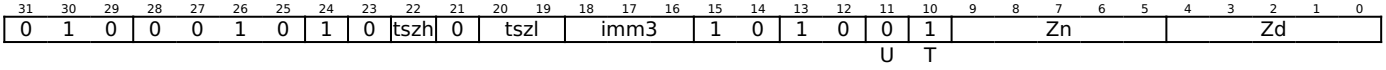
SSHLLT

Signed shift left long by immediate (top)

This instruction shifts left by immediate each odd-numbered signed element of the source vector, and places the results in the overlapping double-width elements of the destination vector. The immediate shift amount is an unsigned value in the range 0 to number of bits per element minus 1. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SSHLLT <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = UInt(tsize::imm3) - esize;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<const> Is the immediate shift amount, in the range 0 to number of bits per element minus 1, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[(2*e + 1)*:esize];
    let shifted_value : integer = SInt(element) << shift;
    result[e*:(2*esize)] = shifted_value[2*esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

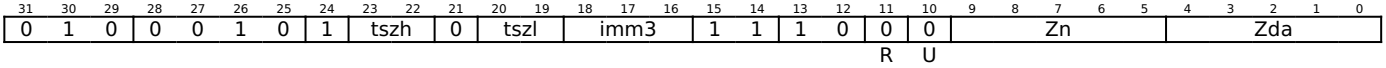
SSRA

Signed shift right and accumulate (immediate)

This instruction shifts right by immediate each signed element of the source vector, preserving the sign bit, and destructively adds the truncated intermediate result to the corresponding elements of the addend vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SSRA <Zda>.<T>, <Zn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let da : integer = UInt(Zda);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : integer = SInt(operand1[e*:esize]) >> shift;
    result[e*:esize] = operand2[e*:esize] + element[esize-1:0];
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

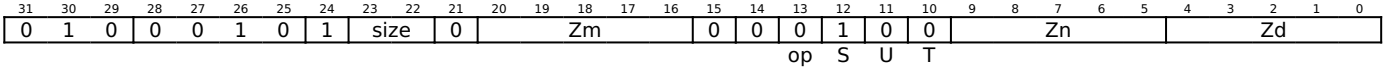
SSUBLB

Signed subtract long (bottom)

This instruction subtracts the even-numbered signed elements of the second source vector from the corresponding signed elements of the first source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SSUBLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 0;
let sel2 : integer = 0;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 - element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

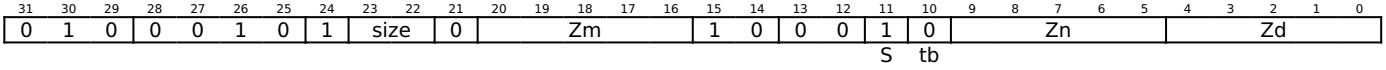
SSUBLBT

Signed subtract long (bottom - top)

This instruction subtracts the odd-numbered signed elements of the second source vector from the even-numbered signed elements of the first source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SSUBLBT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 0;
let sel2 : integer = 1;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 - element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

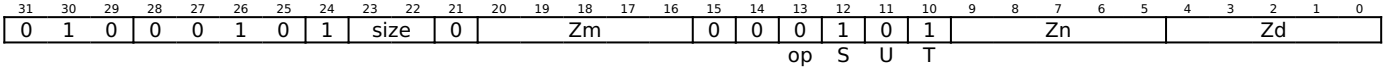
SSUBLT

Signed subtract long (top)

This instruction subtracts the odd-numbered signed elements of the second source vector from the corresponding signed elements of the first source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SSUBLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 1;
let sel2 : integer = 1;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 - element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

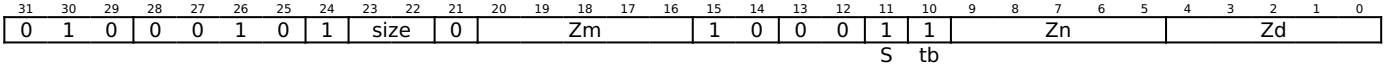
SSUBLTB

Signed subtract long (top - bottom)

This instruction subtracts the even-numbered signed elements of the second source vector from the odd-numbered signed elements of the first source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SSUBLTB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 1;
let sel2 : integer = 0;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 - element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

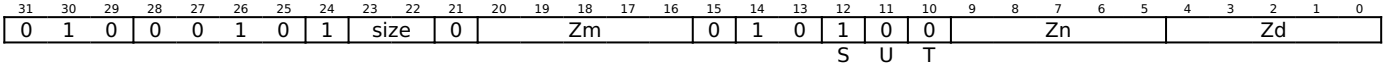
SSUBWB

Signed subtract wide (bottom)

This instruction subtracts the even-numbered signed elements of the second source vector from the overlapping double-width elements of the first source vector and places the results in the corresponding double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SSUBWB <Zd>.<T>, <Zn>.<T>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    result[e*:esize] = (element1 - element2)[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

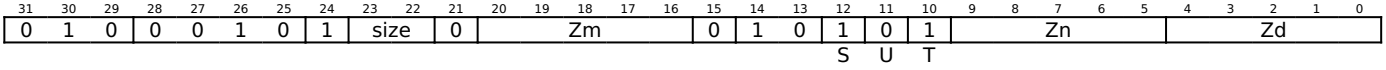
SSUBWT

Signed subtract wide (top)

This instruction subtracts the even-numbered signed elements of the second source vector from the overlapping double-width elements of the first source vector and places the results in the corresponding double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SSUBWT <Zd>.<T>, <Zn>.<T>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = SInt(operand1[e*:esize]);
    let element2 : integer = SInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    result[e*:esize] = (element1 - element2)[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1B (scalar plus immediate, consecutive registers)

Contiguous store of bytes from multiple consecutive vectors (immediate index)

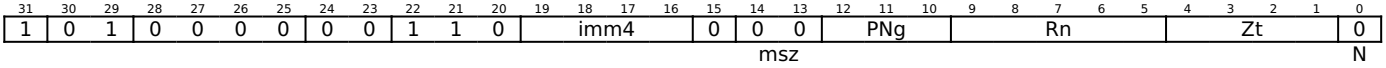
This instruction performs a contiguous store of bytes from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

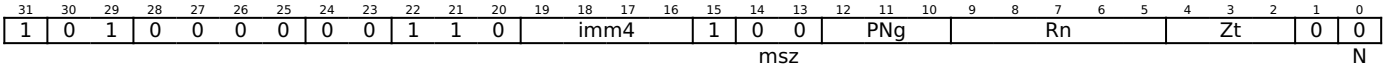
```
ST1B { <Zt1>.B-<Zt2>.B }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
ST1B { <Zt1>.B-<Zt4>.B }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

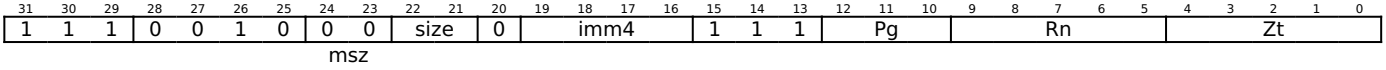
ST1B (scalar plus immediate, single register)

Contiguous store bytes from vector (immediate index)

This instruction performs a contiguous store of bytes from elements of a vector register to the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST1B { <Zt>.<T> }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8 << UInt(size);
let msize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1B (scalar plus scalar, consecutive registers)

Contiguous store of bytes from multiple consecutive vectors (scalar index)

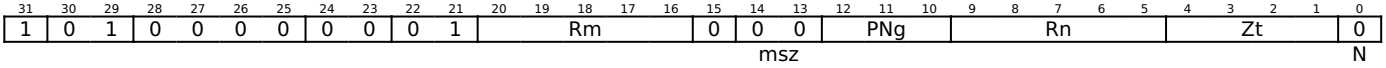
This instruction performs a contiguous store of bytes from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

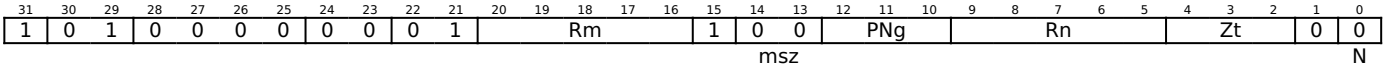
```
ST1B { <Zt1>.B-<Zt2>.B }, <PNg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 8;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
ST1B { <Zt1>.B-<Zt4>.B }, <PNg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 8;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

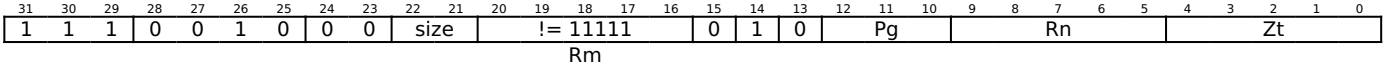
ST1B (scalar plus scalar, single register)

Contiguous store bytes from vector (scalar index)

This instruction performs a contiguous store of bytes from elements of a vector register to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST1B { <Zt>.<T> }, <Pg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8 << UInt(size);
let msize : integer{} = 8;
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1B (scalar plus vector)

Scatter store bytes from a vector (vector index)

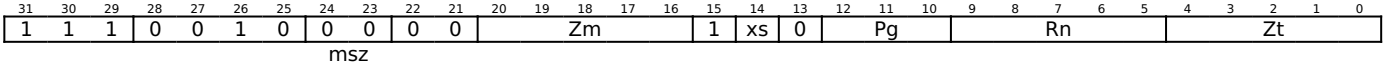
This instruction performs a scatter store of bytes from the Active elements of a vector register to the memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally sign-extended or zero-extended from 32 to 64 bits. Inactive elements are not written to memory.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 3 classes: 32-bit unpacked unscaled offset , 32-bit unscaled offset and 64-bit unscaled offset

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

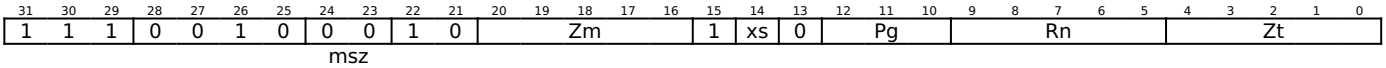
```
ST1B { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

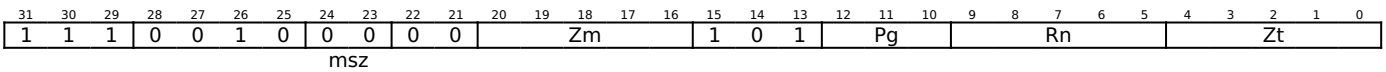
```
ST1B { <Zt>.S }, <Pg>, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
ST1B { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var src : bits(VL);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1B (vector plus immediate)

Scatter store bytes from a vector (immediate index)

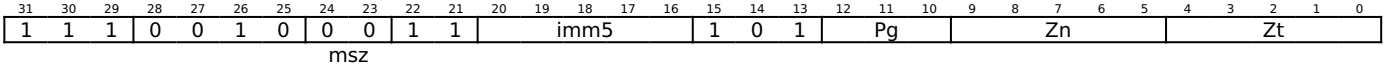
This instruction performs a scatter store of bytes from the Active elements of a vector register to the memory addresses generated by a vector base plus immediate index. The index is in the range 0 to 31. Inactive elements are not written to memory.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

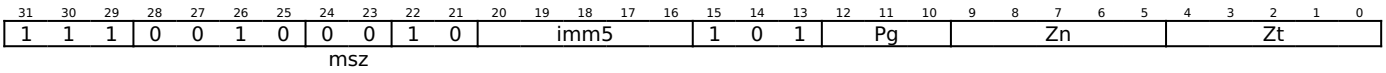
```
ST1B { <Zt>.S }, <Pg>, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
ST1B { <Zt>.D }, <Pg>, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, in the range 0 to 31, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1D (scalar plus immediate, consecutive registers)

Contiguous store of doublewords from multiple consecutive vectors (immediate index)

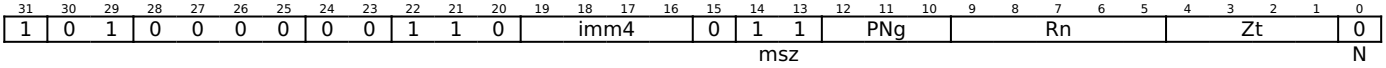
This instruction performs a contiguous store of doublewords from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

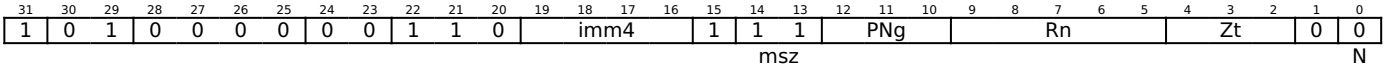
```
ST1D { <Zt1>.D-<Zt2>.D }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
ST1D { <Zt1>.D-<Zt4>.D }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1D (scalar plus immediate, single register)

Contiguous store doublewords from vector (immediate index)

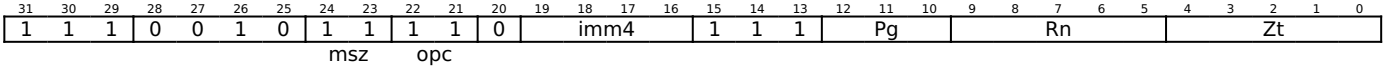
This instruction performs a contiguous store of doublewords from elements of a vector register to the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements are not written to memory.

The 128-bit element variant is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 64-bit element and 128-bit element

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

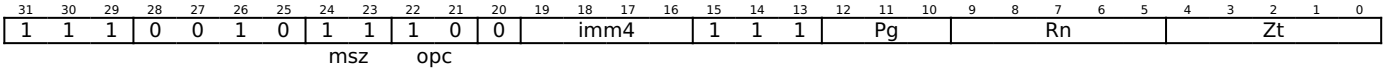
```
ST1D { <Zt>.D }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offset : integer = SInt(imm4);
```

128-bit element

(FEAT_SVE2p1)



Encoding

```
ST1D { <Zt>.Q }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let msize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
if esize < 128 then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1D (scalar plus scalar, consecutive registers)

Contiguous store of doublewords from multiple consecutive vectors (scalar index)

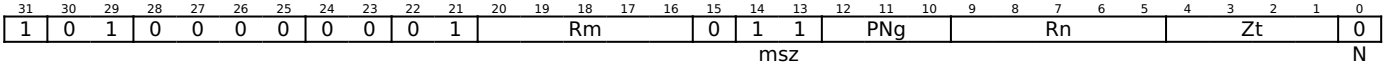
This instruction performs a contiguous store of doublewords from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

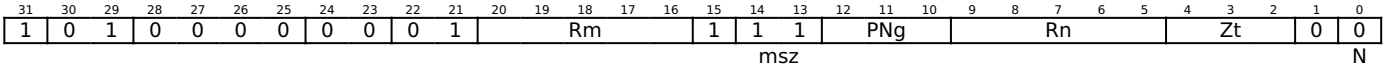
```
ST1D { <Zt1>.D-<Zt2>.D }, <PNg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 64;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
ST1D { <Zt1>.D-<Zt4>.D }, <PNg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 64;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1D (scalar plus scalar, single register)

Contiguous store doublewords from vector (scalar index)

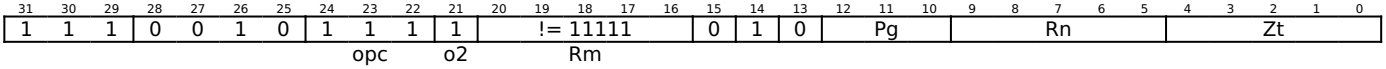
This instruction performs a contiguous store of doublewords from elements of a vector register to the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 8 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements are not written to memory.

The 128-bit element variant is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 64-bit element and 128-bit element

64-bit element

(FEAT_SVE || FEAT_SME)



Encoding

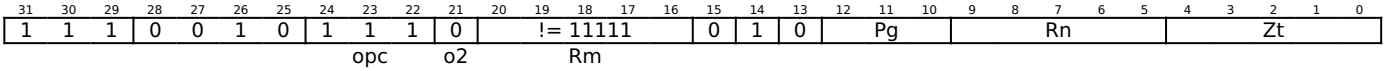
```
ST1D { <Zt>.D }, <Pg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
```

128-bit element

(FEAT_SVE2p1)



Encoding

```
ST1D { <Zt>.Q }, <Pg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let msize : integer{} = 64;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
if esize < 128 then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1D (scalar plus vector)

Scatter store doublewords from a vector (vector index)

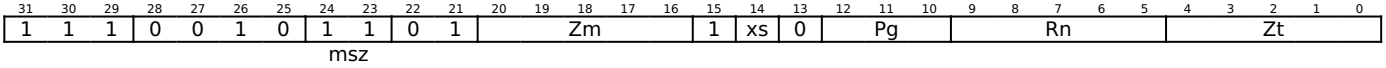
This instruction performs a scatter store of doublewords from the Active elements of a vector register to the memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 8. Inactive elements are not written to memory.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 4 classes: 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

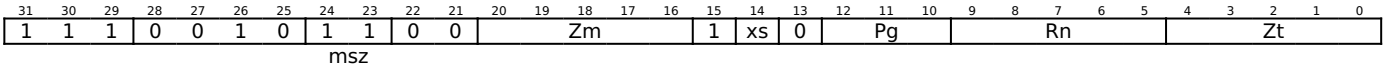
```
ST1D { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D, <mod> #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 3;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

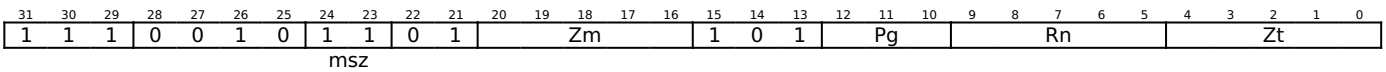
```
ST1D { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

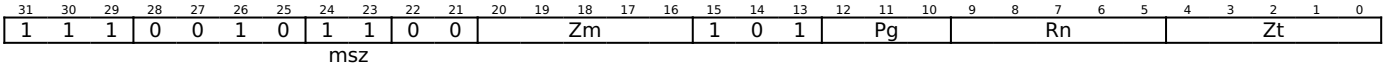
```
ST1D { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 3;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
ST1D { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var src : bits(VL);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1D (vector plus immediate)

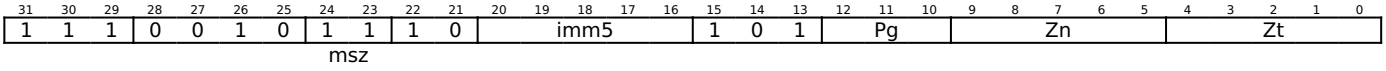
Scatter store doublewords from a vector (immediate index)

This instruction performs a scatter store of doublewords from the Active elements of a vector register to the memory addresses generated by a vector base plus immediate index. The index is a multiple of 8 in the range 0 to 248. Inactive elements are not written to memory.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
ST1D { <Zt>.D }, <Pg>, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 8 in the range 0 to 248, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1H (scalar plus immediate, consecutive registers)

Contiguous store of halfwords from multiple consecutive vectors (immediate index)

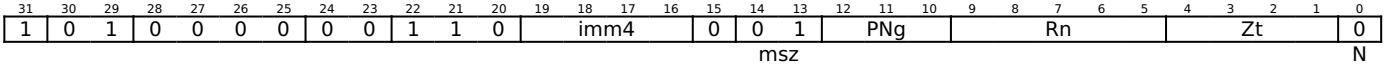
This instruction performs a contiguous store of halfwords from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

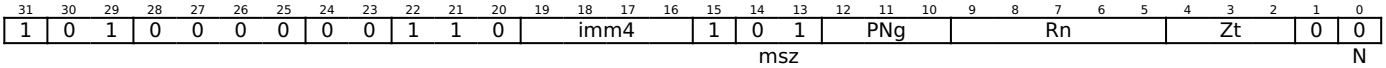
```
ST1H { <Zt1>.H-<Zt2>.H }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
ST1H { <Zt1>.H-<Zt4>.H }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

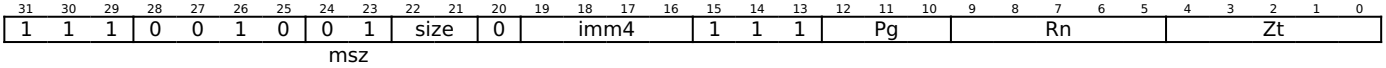
ST1H (scalar plus immediate, single register)

Contiguous store halfwords from vector (immediate index)

This instruction performs a contiguous store of halfwords from elements of a vector register to the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST1H { <Zt>.<T> }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8 << UInt(size);
let msize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1H (scalar plus scalar, consecutive registers)

Contiguous store of halfwords from multiple consecutive vectors (scalar index)

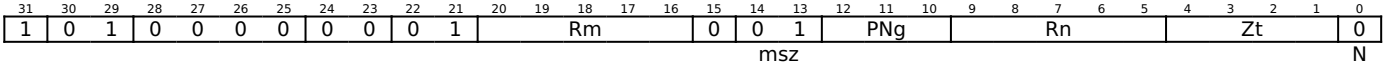
This instruction performs a contiguous store of halfwords from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

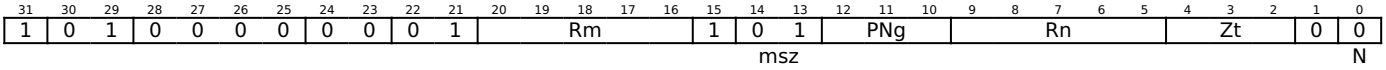
```
ST1H { <Zt1>.H-<Zt2>.H }, <PNg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 16;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
ST1H { <Zt1>.H-<Zt4>.H }, <PNg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 16;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

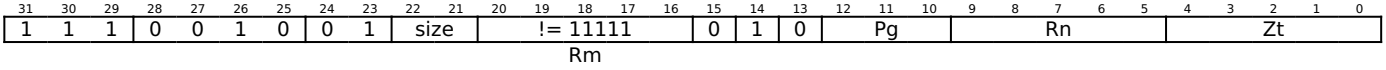
ST1H (scalar plus scalar, single register)

Contiguous store halfwords from vector (scalar index)

This instruction performs a contiguous store of halfwords from elements of a vector register to the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 2 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST1H { <Zt>.<T> }, <Pg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8 << UInt(size);
let msize : integer{} = 16;
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1H (scalar plus vector)

Scatter store halfwords from a vector (vector index)

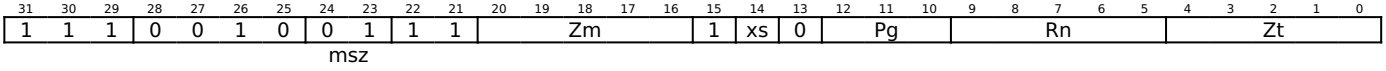
This instruction performs a scatter store of halfwords from the Active elements of a vector register to the memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 2. Inactive elements are not written to memory.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 6 classes: 32-bit scaled offset , 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 32-bit unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

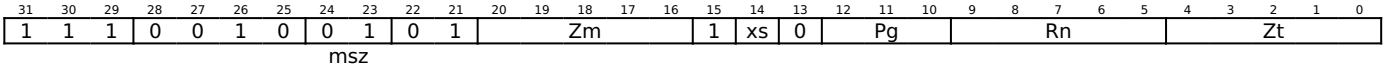
```
ST1H { <Zt>.S }, <Pg>, [<Xn|SP>, <Zm>.S, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 1;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

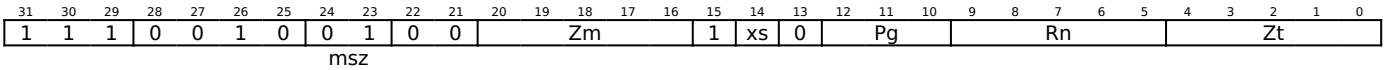
```
ST1H { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D, <mod> #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 1;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

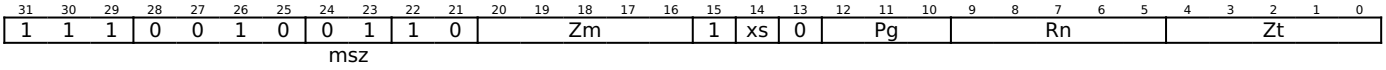
```
ST1H { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

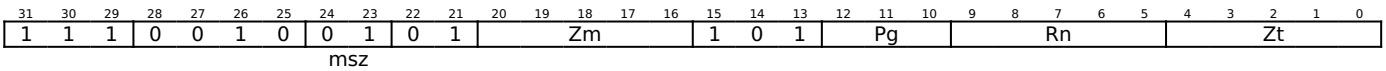
```
ST1H { <Zt>.S }, <Pg>, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

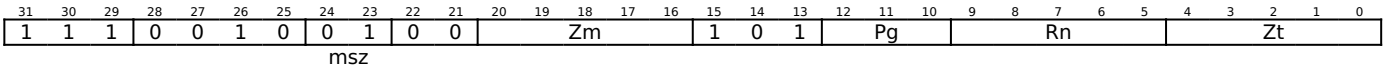
```
ST1H { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 1;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
ST1H { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var src : bits(VL);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1H (vector plus immediate)

Scatter store halfwords from a vector (immediate index)

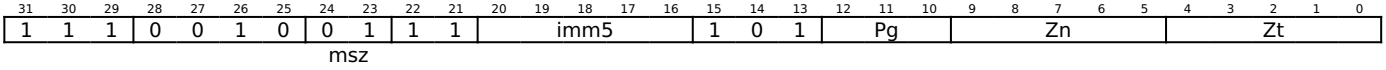
This instruction performs a scatter store of halfwords from the Active elements of a vector register to the memory addresses generated by a vector base plus immediate index. The index is a multiple of 2 in the range 0 to 62. Inactive elements are not written to memory.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

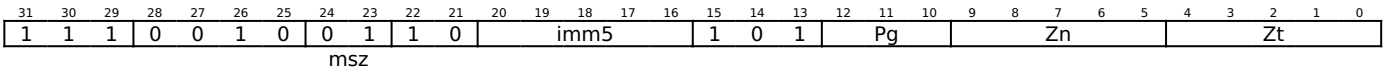
```
ST1H { <Zt>.S }, <Pg>, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
ST1H { <Zt>.D }, <Pg>, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 2 in the range 0 to 62, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1Q

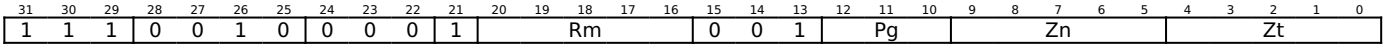
Scatter store quadwords

This instruction performs a scatter store of quadwords from the Active elements of a vector register to the memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements are not written to memory.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2p1)



Encoding

```
ST1Q { <Zt>.Q }, <Pg>, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
CheckNonStreamingSVEEnabled();
let elements : integer = VL DIV 128;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var src : bits(VL);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, 128) then
    base = Z{VL}(n);
    offset = X{64}(m);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, 128) then
        let baddr : bits(64) = base[(2*e)*:64];
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        Mem{128}(addr, accdesc) = src[e*:128];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1W (scalar plus immediate, consecutive registers)

Contiguous store of words from multiple consecutive vectors (immediate index)

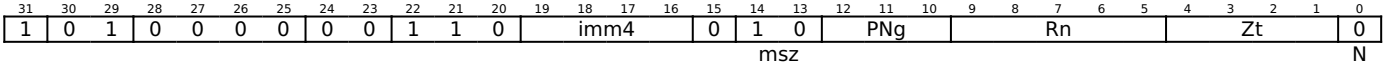
This instruction performs a contiguous store of words from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

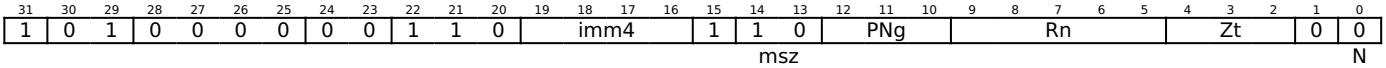
```
ST1W { <Zt1>.S-<Zt2>.S }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
ST1W { <Zt1>.S-<Zt4>.S }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1W (scalar plus immediate, single register)

Contiguous store words from vector (immediate index)

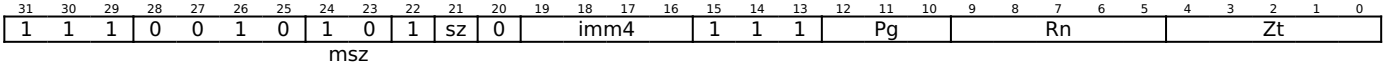
This instruction performs a contiguous store of words from elements of a vector register to the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements are not written to memory.

The 128-bit element variant is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit and 64-bit elements and 128-bit element

32-bit and 64-bit elements

(FEAT_SVE || FEAT_SME)



Encoding

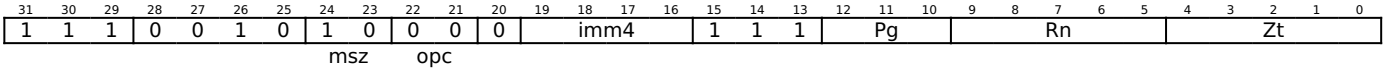
```
ST1W { <Zt>.<T> }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32 << UInt(sz);
let msize : integer{} = 32;
let offset : integer = SInt(imm4);
```

128-bit element

(FEAT_SVE2p1)



Encoding

```
ST1W { <Zt>.Q }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let msize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<imm> Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
if esize < 128 then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;
base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1W (scalar plus scalar, consecutive registers)

Contiguous store of words from multiple consecutive vectors (scalar index)

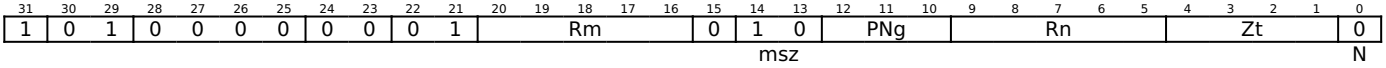
This instruction performs a contiguous store of words from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

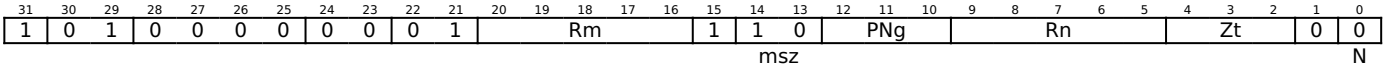
```
ST1W { <Zt1>.S-<Zt2>.S }, <PNg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 32;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
ST1W { <Zt1>.S-<Zt4>.S }, <PNg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 32;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1W (scalar plus scalar, single register)

Contiguous store words from vector (scalar index)

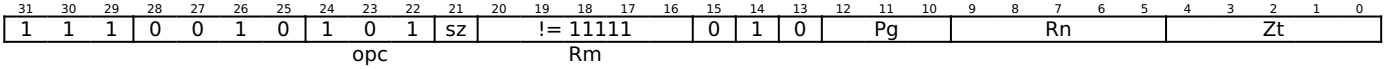
This instruction performs a contiguous store of words from elements of a vector register to the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 4 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements are not written to memory.

The 128-bit element variant is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit and 64-bit elements and 128-bit element

32-bit and 64-bit elements

(FEAT_SVE || FEAT_SME)



Encoding

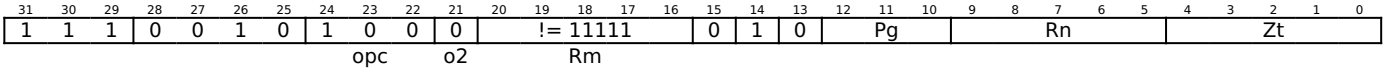
```
ST1W { <Zt>.<T> }, <Pg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32 << UInt(sz);
let msize : integer{} = 32;
```

128-bit element

(FEAT_SVE2p1)



Encoding

```
ST1W { <Zt>.Q }, <Pg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let msize : integer{} = 32;
```

Assembler Symbols

<Zt> Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
if esize < 128 then CheckSVEEnabled(); else CheckNonStreamingSVEEnabled(); end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);
if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;
base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);
for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1W (scalar plus vector)

Scatter store words from a vector (vector index)

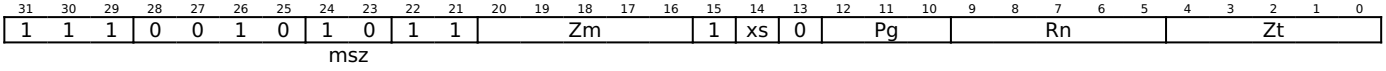
This instruction performs a scatter store of words from the Active elements of a vector register to the memory addresses generated by a 64-bit scalar base plus vector index. The index values are optionally first sign-extended or zero-extended from 32 to 64 bits and then optionally multiplied by 4. Inactive elements are not written to memory.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 6 classes: 32-bit scaled offset , 32-bit unpacked scaled offset , 32-bit unpacked unscaled offset , 32-bit unscaled offset , 64-bit scaled offset and 64-bit unscaled offset

32-bit scaled offset

(FEAT_SVE)



Encoding

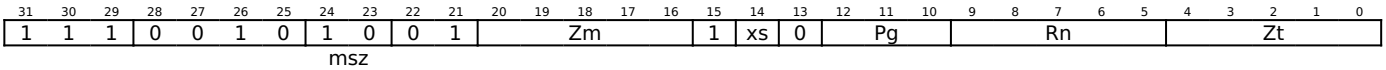
```
ST1W { <Zt>.S }, <Pg>, [<Xn|SP>, <Zm>.S, <mod> #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 2;
```

32-bit unpacked scaled offset

(FEAT_SVE)



Encoding

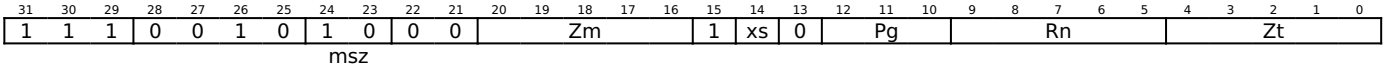
```
ST1W { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D, <mod> #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 2;
```

32-bit unpacked unscaled offset

(FEAT_SVE)



Encoding

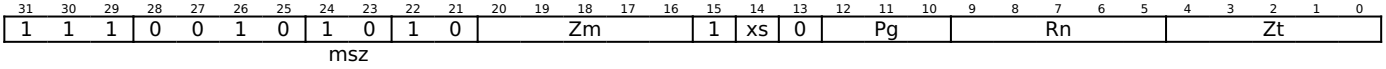
```
ST1W { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

32-bit unscaled offset

(FEAT_SVE)



Encoding

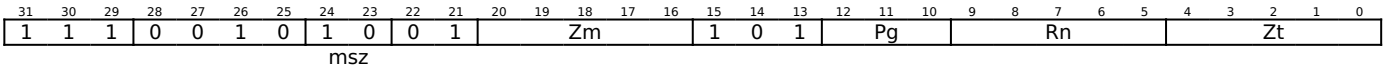
```
ST1W { <Zt>.S }, <Pg>, [<Xn|SP>, <Zm>.S, <mod>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let offs_size : integer{} = 32;
let offs_unsigned : boolean = xs == '0';
let scale : integer = 0;
```

64-bit scaled offset

(FEAT_SVE)



Encoding

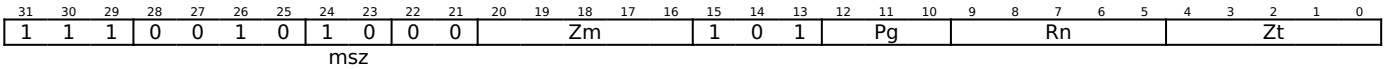
```
ST1W { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 2;
```

64-bit unscaled offset

(FEAT_SVE)



Encoding

```
ST1W { <Zt>.D }, <Pg>, [<Xn|SP>, <Zm>.D]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Zm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offs_size : integer{} = 64;
let offs_unsigned : boolean = TRUE;
let scale : integer = 0;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Zm>	Is the name of the offset scalable vector register, encoded in the "Zm" field.
<mod>	Is the index extend and shift specifier, encoded in “xs”:

xs	<mod>
0	UXTW
1	SXTW

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(VL);
var src : bits(VL);
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    base = if n == 31 then SP{64}() else X{64}(n);
    offset = Z{VL}(m);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let offselt : bits(offs_size) = offset[e*:esize][offs_size-1:0];
        let off : integer = if offs_unsigned then UInt(offselt) else SInt(offselt);
        let addr : bits(64) = AddressAdd(base, off << scale, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1W (vector plus immediate)

Scatter store words from a vector (immediate index)

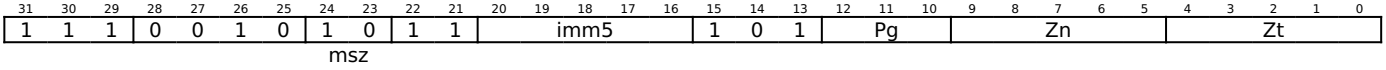
This instruction performs a scatter store of words from the Active elements of a vector register to the memory addresses generated by a vector base plus immediate index. The index is a multiple of 4 in the range 0 to 124. Inactive elements are not written to memory.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit element and 64-bit element

32-bit element

(FEAT_SVE)



Encoding

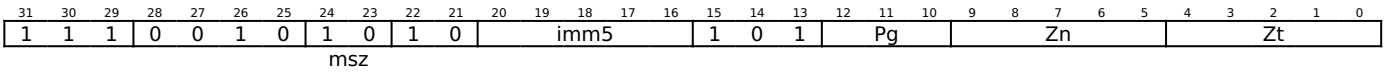
```
ST1W { <Zt>.S }, <Pg>, [<Zn>.S{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
let offset : integer = UInt(imm5);
```

64-bit element

(FEAT_SVE)



Encoding

```
ST1W { <Zt>.D }, <Pg>, [<Zn>.D{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
let offset : integer = UInt(imm5);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<imm>	Is the optional unsigned immediate byte offset, a multiple of 4 in the range 0 to 124, defaulting to 0, encoded in the "imm5" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var src : bits(VL);
let mbytes : integer{} = msize DIV 8;
let contiguous : boolean = FALSE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset * mbytes, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2B (scalar plus immediate)

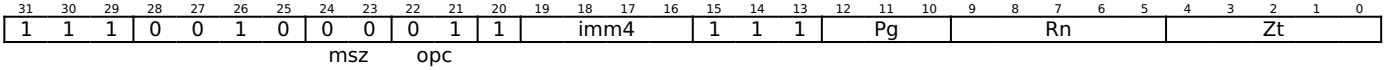
Contiguous store two-byte structures from two vectors (immediate index)

This instruction performs a contiguous store of two-byte structures, each from the same element number in two vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 2 in the range -16 to 14 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive bytes in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST2B { <Zt1>.B, <Zt2>.B }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2B (scalar plus scalar)

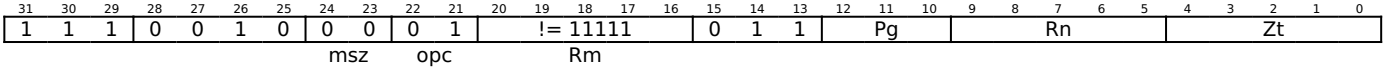
Contiguous store two-byte structures from two vectors (scalar index)

This instruction performs a contiguous store of two-byte structures, each from the same element number in two vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register and added to the base address. After each structure access the index value is incremented by two. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive bytes in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST2B { <Zt1>.B, <Zt2>.B }, <Pg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2D (scalar plus immediate)

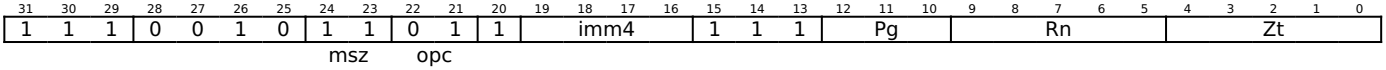
Contiguous store two-doubleword structures from two vectors (immediate index)

This instruction performs a contiguous store of two-doubleword structures, each from the same element number in two vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 2 in the range -16 to 14 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive doublewords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST2D { <Zt1>.D, <Zt2>.D }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2D (scalar plus scalar)

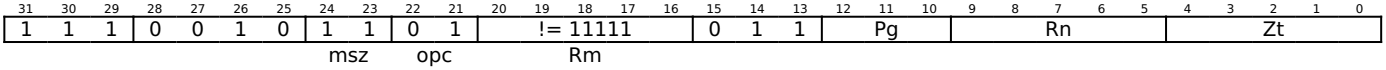
Contiguous store two-doubleword structures from two vectors (scalar index)

This instruction performs a contiguous store of two-doubleword structures, each from the same element number in two vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by two. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive doublewords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST2D { <Zt1>.D, <Zt2>.D }, <Pg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2H (scalar plus immediate)

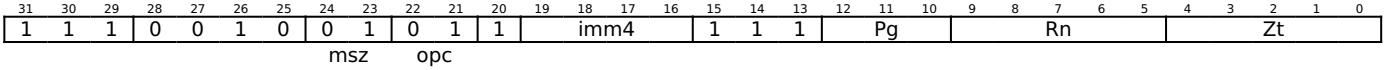
Contiguous store two-halfword structures from two vectors (immediate index)

This instruction performs a contiguous store of two-halfword structures, each from the same element number in two vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 2 in the range -16 to 14 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive halfwords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST2H { <Zt1>.H, <Zt2>.H }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2H (scalar plus scalar)

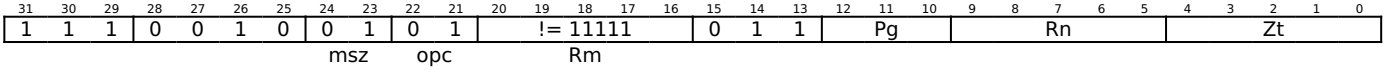
Contiguous store two-halfword structures from two vectors (scalar index)

This instruction performs a contiguous store of two-halfword structures, each from the same element number in two vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by two. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive halfwords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST2H { <Zt1>.H, <Zt2>.H }, <Pg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2Q (scalar plus immediate)

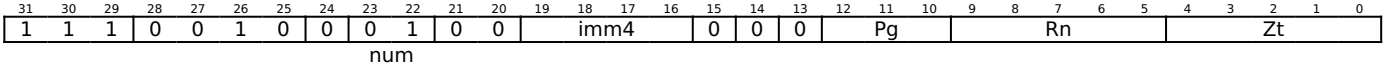
Contiguous store two-quadword structures from two vectors (immediate index)

This instruction performs a contiguous store of two-quadword structures, each from the same element number in two vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 2 in the range -16 to 14 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive quadwords in memory that make up each structure. Inactive structures are not written to memory.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ST2Q { <Zt1>.Q, <Zt2>.Q }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let offset : integer = SInt(imm4);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2Q (scalar plus scalar)

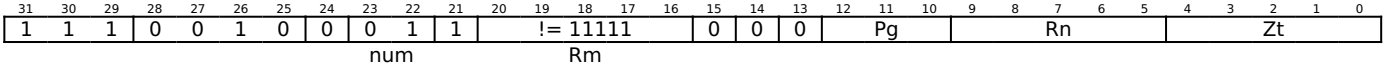
Contiguous store two-quadword structures from two vectors (scalar index)

This instruction performs a contiguous store of two-quadword structures, each from the same element number in two vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by two. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive quadwords in memory that make up each structure. Inactive structures are not written to memory.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ST2Q { <Zt1>.Q, <Zt2>.Q }, <Pg>, [<Xn|SP>, <Xm>, LSL #4]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2W (scalar plus immediate)

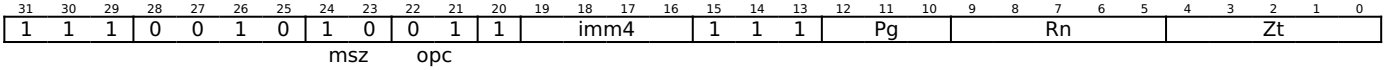
Contiguous store two-word structures from two vectors (immediate index)

This instruction performs a contiguous store of two-word structures, each from the same element number in two vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 2 in the range -16 to 14 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive words in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST2W { <Zt1>.S, <Zt2>.S }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST2W (scalar plus scalar)

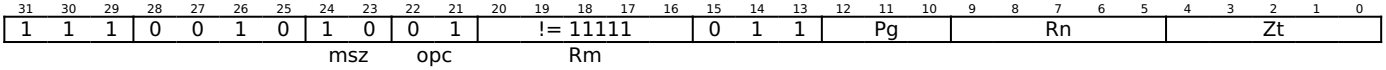
Contiguous store two-word structures from two vectors (scalar index)

This instruction performs a contiguous store of two-word structures, each from the same element number in two vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by two. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the two vector registers, or equivalently to the two consecutive words in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST2W { <Zt1>.S, <Zt2>.S }, <Pg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let nreg : integer{} = 2;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3B (scalar plus immediate)

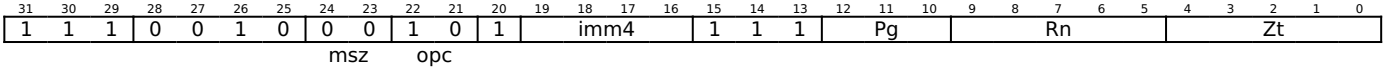
Contiguous store three-byte structures from three vectors (immediate index)

This instruction performs a contiguous store of three-byte structures, each from the same element number in three vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 3 in the range -24 to 21 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive bytes in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST3B { <Zt1>.B, <Zt2>.B, <Zt3>.B }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 3 in the range -24 to 21, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3B (scalar plus scalar)

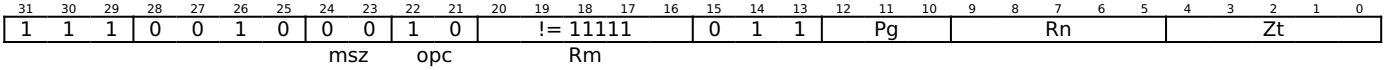
Contiguous store three-byte structures from three vectors (scalar index)

This instruction performs a contiguous store of three-byte structures, each from the same element number in three vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register and added to the base address. After each structure access the index value is incremented by three. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive bytes in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST3B { <Zt1>.B, <Zt2>.B, <Zt3>.B }, <Pg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3D (scalar plus immediate)

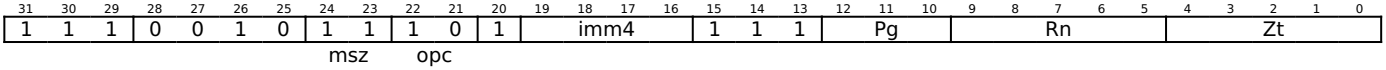
Contiguous store three-doubleword structures from three vectors (immediate index)

This instruction performs a contiguous store of three-doubleword structures, each from the same element number in three vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 3 in the range -24 to 21 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive doublewords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST3D { <Zt1>.D, <Zt2>.D, <Zt3>.D }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 3 in the range -24 to 21, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3D (scalar plus scalar)

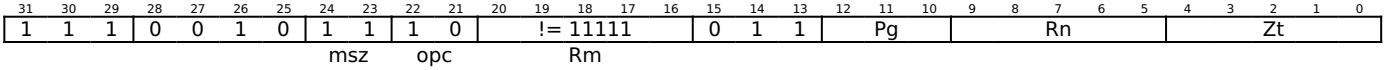
Contiguous store three-doubleword structures from three vectors (scalar index)

This instruction performs a contiguous store of three-doubleword structures, each from the same element number in three vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by three. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive doublewords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST3D { <Zt1>.D, <Zt2>.D, <Zt3>.D }, <Pg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3H (scalar plus immediate)

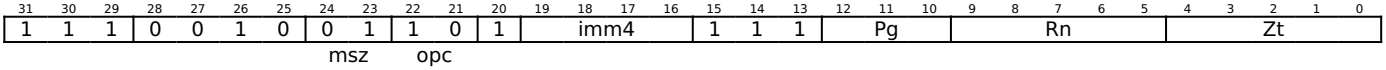
Contiguous store three-halfword structures from three vectors (immediate index)

This instruction performs a contiguous store of three-halfword structures, each from the same element number in three vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 3 in the range -24 to 21 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive halfwords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST3H { <Zt1>.H, <Zt2>.H, <Zt3>.H }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 3 in the range -24 to 21, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3H (scalar plus scalar)

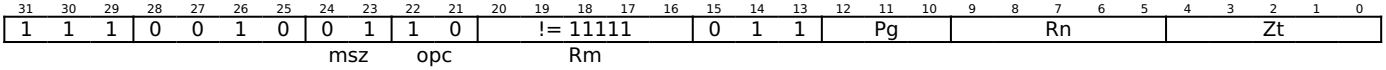
Contiguous store three-halfword structures from three vectors (scalar index)

This instruction performs a contiguous store of three-halfword structures, each from the same element number in three vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by three. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive halfwords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST3H { <Zt1>.H, <Zt2>.H, <Zt3>.H }, <Pg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3Q (scalar plus immediate)

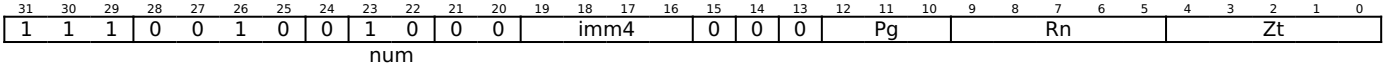
Contiguous store three-quadword structures from three vectors (immediate index)

This instruction performs a contiguous store of three-quadword structures, each from the same element number in three vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 3 in the range -24 to 21 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive quadwords in memory that make up each structure. Inactive structures are not written to memory.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ST3Q { <Zt1>.Q, <Zt2>.Q, <Zt3>.Q }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let offset : integer = SInt(imm4);
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 3 in the range -24 to 21, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3Q (scalar plus scalar)

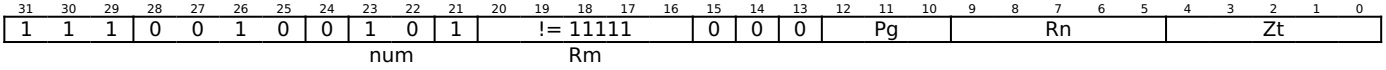
Contiguous store three-quadword structures from three vectors (scalar index)

This instruction performs a contiguous store of three-quadword structures, each from the same element number in three vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by three. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive quadwords in memory that make up each structure. Inactive structures are not written to memory.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ST3Q { <Zt1>.Q, <Zt2>.Q, <Zt3>.Q }, <Pg>, [<Xn|SP>, <Xm>, LSL #4]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3W (scalar plus immediate)

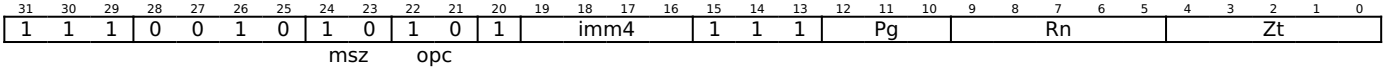
Contiguous store three-word structures from three vectors (immediate index)

This instruction performs a contiguous store of three-word structures, each from the same element number in three vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 3 in the range -24 to 21 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive words in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST3W { <Zt1>.S, <Zt2>.S, <Zt3>.S }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 3 in the range -24 to 21, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST3W (scalar plus scalar)

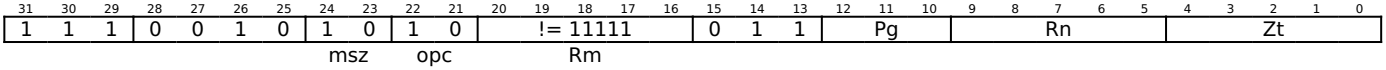
Contiguous store three-word structures from three vectors (scalar index)

This instruction performs a contiguous store of three-word structures, each from the same element number in three vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by three. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the three vector registers, or equivalently to the three consecutive words in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST3W { <Zt1>.S, <Zt2>.S, <Zt3>.S }, <Pg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let nreg : integer{} = 3;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4B (scalar plus immediate)

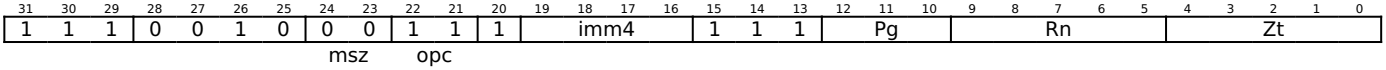
Contiguous store four-byte structures from four vectors (immediate index)

This instruction performs a contiguous store of four-byte structures, each from the same element number in four vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 4 in the range -32 to 28 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive bytes in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST4B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4B (scalar plus scalar)

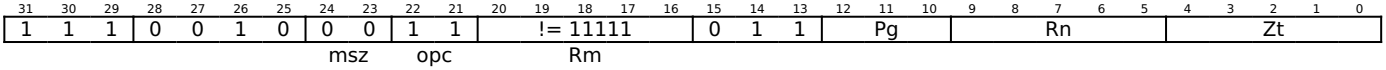
Contiguous store four-byte structures from four vectors (scalar index)

This instruction performs a contiguous store of four-byte structures, each from the same element number in four vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register and added to the base address. After each structure access the index value is incremented by four. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive bytes in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST4B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <Pg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4D (scalar plus immediate)

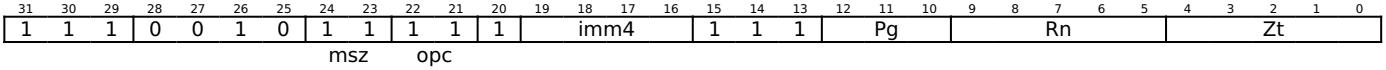
Contiguous store four-doubleword structures from four vectors (immediate index)

This instruction performs a contiguous store of four-doubleword structures, each from the same element number in four vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 4 in the range -32 to 28 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive doublewords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST4D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4D (scalar plus scalar)

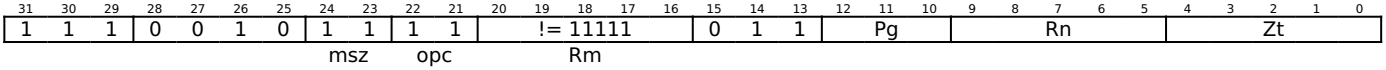
Contiguous store four-doubleword structures from four vectors (scalar index)

This instruction performs a contiguous store of four-doubleword structures, each from the same element number in four vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by four. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive doublewords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST4D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <Pg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4H (scalar plus immediate)

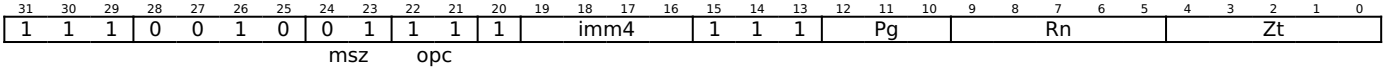
Contiguous store four-halfword structures from four vectors (immediate index)

This instruction performs a contiguous store of four-halfword structures, each from the same element number in four vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 4 in the range -32 to 28 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive halfwords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST4H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4H (scalar plus scalar)

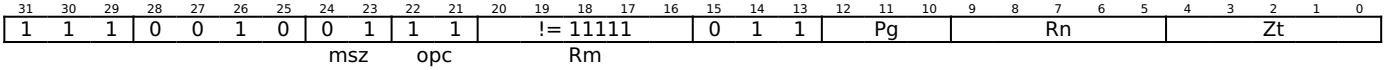
Contiguous store four-halfword structures from four vectors (scalar index)

This instruction performs a contiguous store of four-halfword structures, each from the same element number in four vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by four. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive halfwords in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST4H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <Pg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4Q (scalar plus immediate)

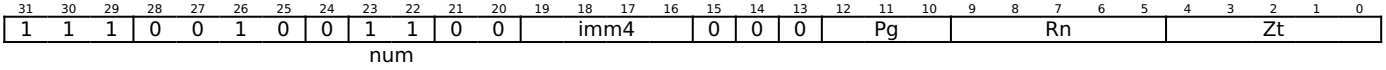
Contiguous store four-quadword structures from four vectors (immediate index)

This instruction performs a contiguous store of four-quadword structures, each from the same element number in four vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 4 in the range -32 to 28 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive quadwords in memory that make up each structure. Inactive structures are not written to memory.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ST4Q { <Zt1>.Q, <Zt2>.Q, <Zt3>.Q, <Zt4>.Q }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let offset : integer = SInt(imm4);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4Q (scalar plus scalar)

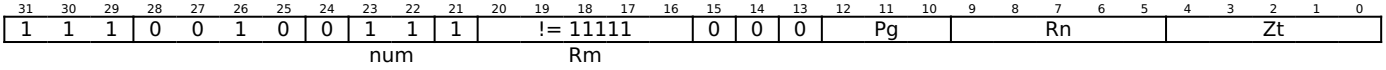
Contiguous store four-quadword structures from four vectors (scalar index)

This instruction performs a contiguous store of four-quadword structures, each from the same element number in four vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by four. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive quadwords in memory that make up each structure. Inactive structures are not written to memory.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ST4Q { <Zt1>.Q, <Zt2>.Q, <Zt3>.Q, <Zt4>.Q }, <Pg>, [<Xn|SP>, <Xm>, LSL #4]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 128;
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4W (scalar plus immediate)

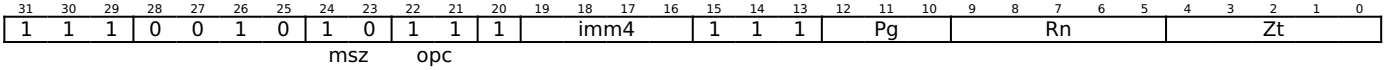
Contiguous store four-word structures from four vectors (immediate index)

This instruction performs a contiguous store of four-word structures, each from the same element number in four vector registers to the memory address generated by a 64-bit scalar base and an immediate index that is a multiple of 4 in the range -32 to 28 that is multiplied by the vector's in-memory size, irrespective of predication.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive words in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST4W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * nreg * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST4W (scalar plus scalar)

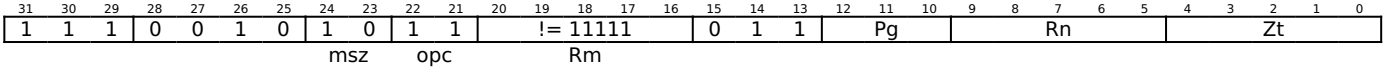
Contiguous store four-word structures from four vectors (scalar index)

This instruction performs a contiguous store of four-word structures, each from the same element number in four vector registers to the memory address generated by a 64-bit scalar base and a 64-bit scalar index register scaled by the element size (LSL option) and added to the base address. After each structure access the index value is incremented by four. The index register is not updated by the instruction.

Each predicate element applies to the same element number in each of the four vector registers, or equivalently to the four consecutive words in memory that make up each structure. Inactive structures are not written to memory.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
ST4W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <Pg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let nreg : integer{} = 4;
```

Assembler Symbols

<Zt1>	Is the name of the first scalable vector register to be transferred, encoded in the "Zt" field.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" plus 1 modulo 32.
<Zt3>	Is the name of the third scalable vector register to be transferred, encoded as "Zt" plus 2 modulo 32.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" plus 3 modulo 32.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
let mask : bits(PL) = P{}(g);
var offset : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var values : array [[nreg]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = TRUE;

let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    values[[r]] = Z{VL}((t+r) MOD 32);
end;

for e = 0 to elements-1 do
    for r = 0 to nreg-1 do
        if ActivePredicateElement{PL}(mask, e, esize) then
            Mem{esize}(addr, accdesc) = values[[r]][e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STLRO

Store-release octaword

This instruction stores a 32-byte (octaword) vector register to a memory address generated by a 64-bit scalar base, plus an immediate offset. The base register must be aligned to 32 bytes if alignment is checked.

This instruction is unpredicated.

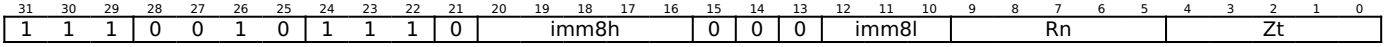
The instruction has memory ordering semantics as described in *Load-Acquire, Store-Release*.

This instruction requires that the Effective SVE vector length is at least 256 bits.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE_LRCPC)



Encoding

```
STLRO <Zt>, [<Xn|SP>{, #<imm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE_LRCPC) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let imm : integer = UInt(imm8h::imm8l);
let acquire : boolean = FALSE;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional positive immediate byte offset, a multiple of 32 in the range 0 to 8160, defaulting to 0, encoded in the "imm8h:imm8l" fields.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < 256 then EndOfDecode(Decode_UNDEF); end;
var src : bits(VL);
var base : bits(64);
let offset : integer = imm * 32;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked, acquire, t);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

src = Z{VL}(t);
let data : bits(256) = src[0 +: 256];

let addr : bits(64) = AddressAdd(base, offset, accdesc);

Mem{256}(addr, accdesc) = data;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1B (scalar plus immediate, consecutive registers)

Contiguous store non-temporal of bytes from multiple consecutive vectors (immediate index)

This instruction performs a contiguous non-temporal store of bytes from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

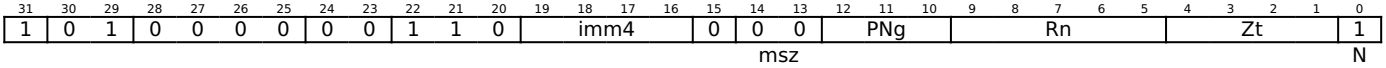
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

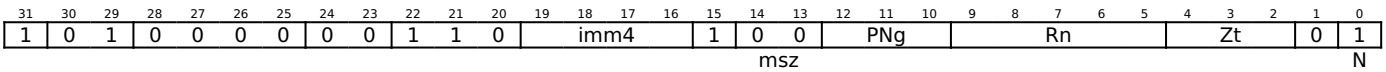
```
STNT1B { <Zt1>.B-<Zt2>.B }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
STNT1B { <Zt1>.B-<Zt4>.B }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1B (scalar plus immediate, single register)

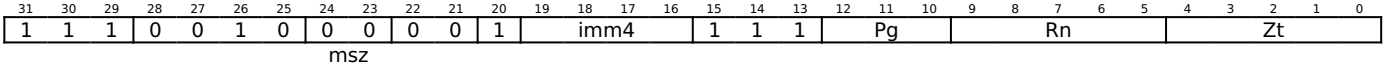
Contiguous store non-temporal bytes from vector (immediate index)

This instruction performs a contiguous non-temporal store of bytes from elements of a vector register to the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
STNT1B { <Zt>.B }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var src : bits(VL);
let mask : bits(PL) = P{}(g);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1B (scalar plus scalar, consecutive registers)

Contiguous store non-temporal of bytes from multiple consecutive vectors (scalar index)

This instruction performs a contiguous non-temporal store of bytes from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

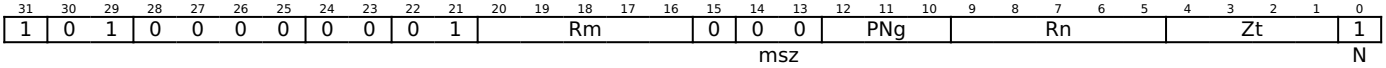
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

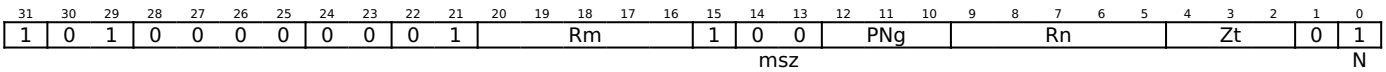
```
STNT1B { <Zt1>.B-<Zt2>.B }, <PNg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 8;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
STNT1B { <Zt1>.B-<Zt4>.B }, <PNg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 8;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

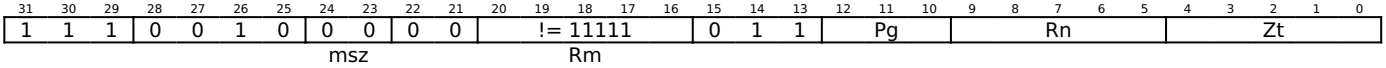
STNT1B (scalar plus scalar, single register)

Contiguous store non-temporal bytes from vector (scalar index)

This instruction performs a contiguous non-temporal store of bytes from elements of a vector register to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements are not written to memory.
A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
STNT1B { <Zt>.B }, <Pg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 8;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var offset : bits(64);
var addr : bits(64);
var src : bits(VL);
let mask : bits(PL) = P{}(g);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated  : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1B (vector plus scalar)

Scatter store non-temporal bytes

This instruction performs a non-temporal scatter store of bytes from the Active elements of a vector register to the memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements are not written to memory.

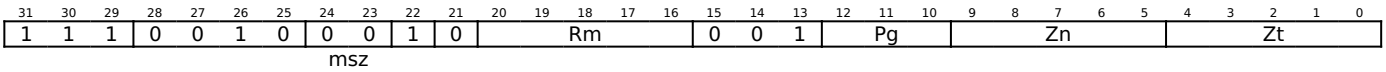
A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit unscaled offset and 64-bit unscaled offset

32-bit unscaled offset

(FEAT_SVE2)



Encoding

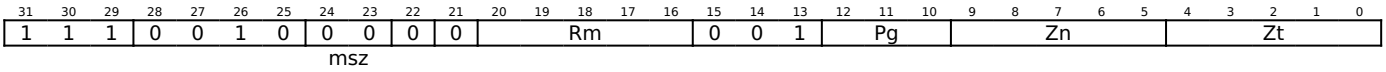
```
STNT1B { <Zt>.S }, <Pg>, [<Zn>.S{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 8;
```

64-bit unscaled offset

(FEAT_SVE2)



Encoding

```
STNT1B { <Zt>.D }, <Pg>, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 8;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var src : bits(VL);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1D (scalar plus immediate, consecutive registers)

Contiguous store non-temporal of doublewords from multiple consecutive vectors (immediate index)

This instruction performs a contiguous non-temporal store of doublewords from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

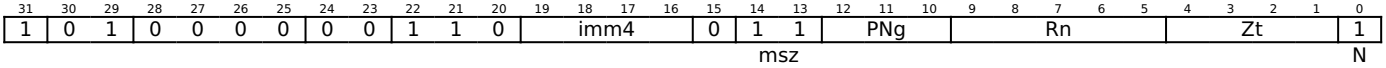
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

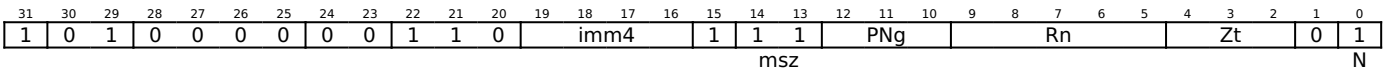
```
STNT1D { <Zt1>.D-<Zt2>.D }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
STNT1D { <Zt1>.D-<Zt4>.D }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

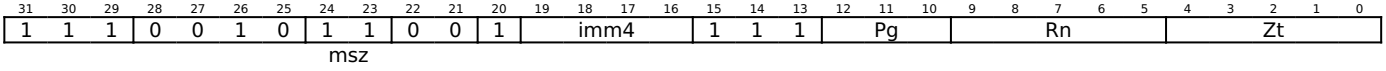
STNT1D (scalar plus immediate, single register)

Contiguous store non-temporal doublewords from vector (immediate index)

This instruction performs a contiguous non-temporal store of doublewords from elements of a vector register to the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements are not written to memory.
A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
STNT1D { <Zt>.D }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var src : bits(VL);
let mask : bits(PL) = P{}(g);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1D (scalar plus scalar, consecutive registers)

Contiguous store non-temporal of doublewords from multiple consecutive vectors (scalar index)

This instruction performs a contiguous non-temporal store of doublewords from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

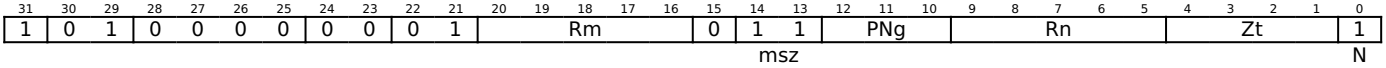
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

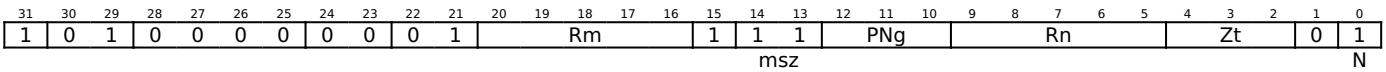
```
STNT1D { <Zt1>.D-<Zt2>.D }, <PNg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 64;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
STNT1D { <Zt1>.D-<Zt4>.D }, <PNg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 64;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

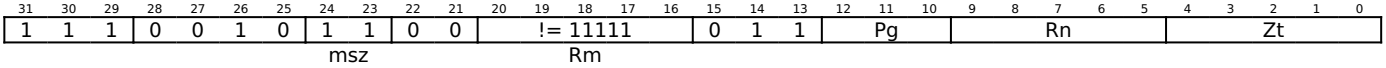
STNT1D (scalar plus scalar, single register)

Contiguous store non-temporal doublewords from vector (scalar index)

This instruction performs a contiguous non-temporal store of doublewords from elements of a vector register to the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 8 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements are not written to memory.
A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
STNT1D { <Zt>.D }, <Pg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var offset : bits(64);
var addr : bits(64);
var src : bits(VL);
let mask : bits(PL) = P{}(g);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated  : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1D (vector plus scalar)

Scatter store non-temporal doublewords

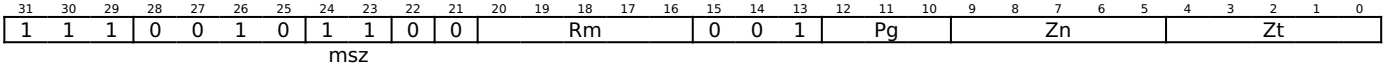
This instruction performs a non-temporal scatter store of doublewords from the Active elements of a vector register to the memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE2)



Encoding

```
STNT1D { <Zt>.D }, <Pg>, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 64;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var src : bits(VL);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1H (scalar plus immediate, consecutive registers)

Contiguous store non-temporal of halfwords from multiple consecutive vectors (immediate index)

This instruction performs a contiguous non-temporal store of halfwords from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

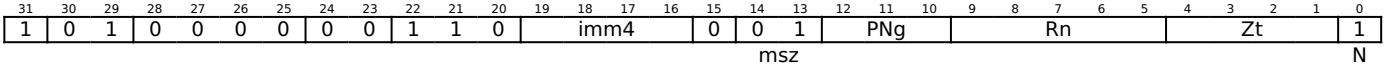
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

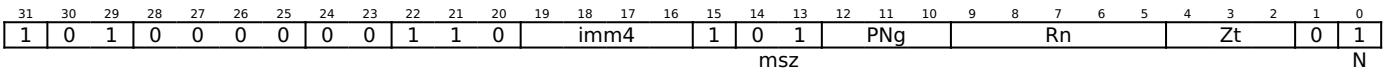
```
STNT1H { <Zt1>.H-<Zt2>.H }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
STNT1H { <Zt1>.H-<Zt4>.H }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

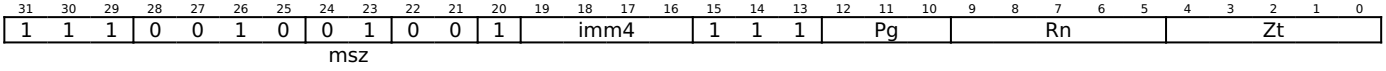
STNT1H (scalar plus immediate, single register)

Contiguous store non-temporal halfwords from vector (immediate index)

This instruction performs a contiguous non-temporal store of halfwords from elements of a vector register to the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements are not written to memory.
A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
STNT1H { <Zt>.H }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var src : bits(VL);
let mask : bits(PL) = P{}(g);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1H (scalar plus scalar, consecutive registers)

Contiguous store non-temporal of halfwords from multiple consecutive vectors (scalar index)

This instruction performs a contiguous non-temporal store of halfwords from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

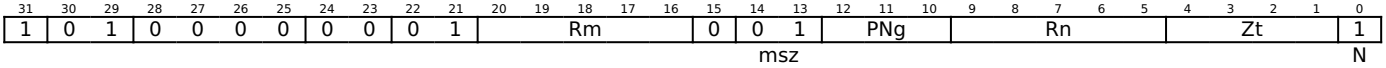
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

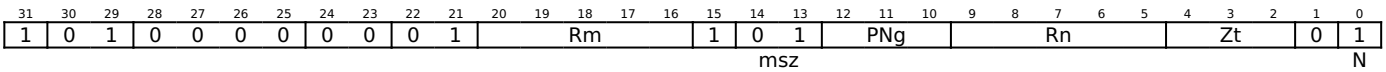
```
STNT1H { <Zt1>.H-<Zt2>.H }, <PNg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 16;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
STNT1H { <Zt1>.H-<Zt4>.H }, <PNg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 16;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

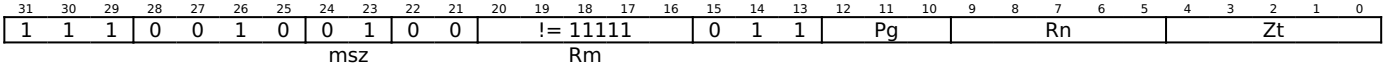
STNT1H (scalar plus scalar, single register)

Contiguous store non-temporal halfwords from vector (scalar index)

This instruction performs a contiguous non-temporal store of halfwords from elements of a vector register to the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 2 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements are not written to memory.
A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
STNT1H { <Zt>.H }, <Pg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 16;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var offset : bits(64);
var addr : bits(64);
var src : bits(VL);
let mask : bits(PL) = P{}(g);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated  : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1H (vector plus scalar)

Scatter store non-temporal halfwords

This instruction performs a non-temporal scatter store of halfwords from the Active elements of a vector register to the memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements are not written to memory.

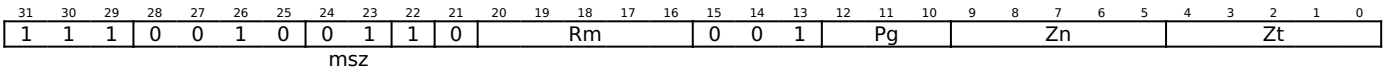
A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit unscaled offset and 64-bit unscaled offset

32-bit unscaled offset

(FEAT_SVE2)



Encoding

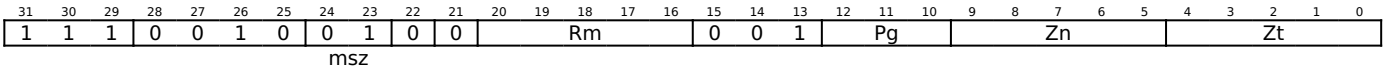
```
STNT1H { <Zt>.S }, <Pg>, [<Zn>.S{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 16;
```

64-bit unscaled offset

(FEAT_SVE2)



Encoding

```
STNT1H { <Zt>.D }, <Pg>, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 16;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var src : bits(VL);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1W (scalar plus immediate, consecutive registers)

Contiguous store non-temporal of words from multiple consecutive vectors (immediate index)

This instruction performs a contiguous non-temporal store of words from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

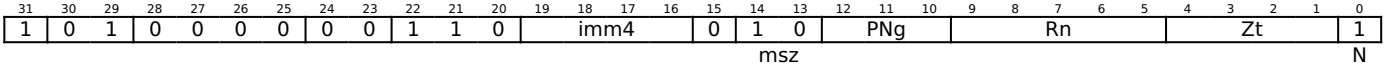
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

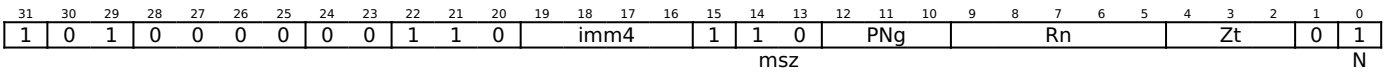
```
STNT1W { <Zt1>.S-<Zt2>.S }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
STNT1W { <Zt1>.S-<Zt4>.S }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1W (scalar plus immediate, single register)

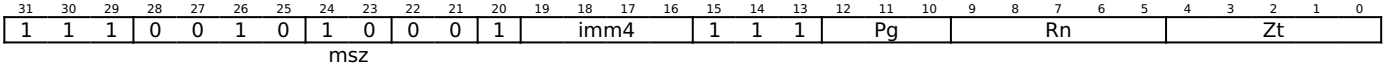
Contiguous store non-temporal words from vector (immediate index)

This instruction performs a contiguous non-temporal store of words from elements of a vector register to the memory address generated by a 64-bit scalar base and immediate index in the range -8 to 7 that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
STNT1W { <Zt>.S }, <Pg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -8 to 7, defaulting to 0, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mbytes : integer{} = esize DIV 8;
var src : bits(VL);
let mask : bits(PL) = P{}(g);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * elements * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1W (scalar plus scalar, consecutive registers)

Contiguous store non-temporal of words from multiple consecutive vectors (scalar index)

This instruction performs a contiguous non-temporal store of words from elements of two or four consecutive vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

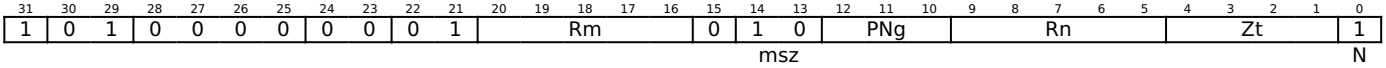
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

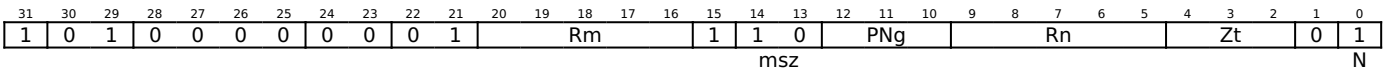
```
STNT1W { <Zt1>.S-<Zt2>.S }, <PNg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let t : integer = UInt(Zt::'0');
let esize : integer{} = 32;
```

Four registers

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
STNT1W { <Zt1>.S-<Zt4>.S }, <PNg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let t : integer = UInt(Zt::'00');
let esize : integer{} = 32;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 2. For the "Four registers" variant: is the name of the first scalable vector register to be transferred, encoded as "Zt" times 4.
<Zt2>	Is the name of the second scalable vector register to be transferred, encoded as "Zt" times 2 plus 1.
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt4>	Is the name of the fourth scalable vector register to be transferred, encoded as "Zt" times 4 plus 3.

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1W (scalar plus scalar, single register)

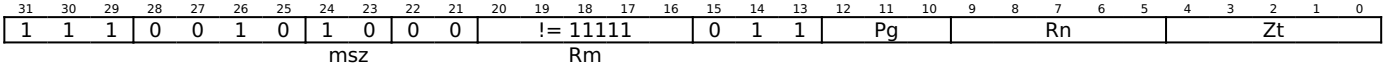
Contiguous store non-temporal words from vector (scalar index)

This instruction performs a contiguous non-temporal store of words from elements of a vector register to the memory address generated by a 64-bit scalar base and scalar index that is multiplied by 4 and added to the base address. After each element access the index value is incremented, but the index register is not updated. Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
STNT1W { <Zt>.S }, <Pg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var base : bits(64);
var offset : bits(64);
var addr : bits(64);
var src : bits(VL);
let mask : bits(PL) = P{}(g);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
let predicated  : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if !AnyActiveElement{PL}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
    src = Z{VL}(t);
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    addr = AddressIncrement(addr, mbytes, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1W (vector plus scalar)

Scatter store non-temporal words

This instruction performs a non-temporal scatter store of words from the Active elements of a vector register to the memory addresses generated by a vector base plus a 64-bit unscaled scalar register offset. Inactive elements are not written to memory.

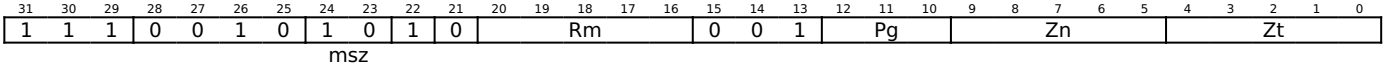
A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

It has encodings from 2 classes: 32-bit unscaled offset and 64-bit unscaled offset

32-bit unscaled offset

(FEAT_SVE2)



Encoding

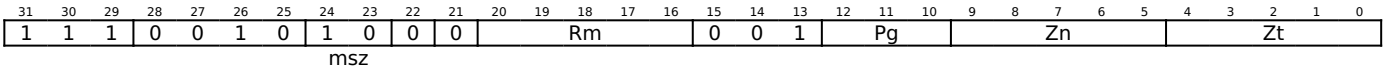
```
STNT1W { <Zt>.S }, <Pg>, [<Zn>.S{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 32;
let msize : integer{} = 32;
```

64-bit unscaled offset

(FEAT_SVE2)



Encoding

```
STNT1W { <Zt>.D }, <Pg>, [<Zn>.D{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let n : integer = UInt(Zn);
let m : integer = UInt(Rm);
let g : integer = UInt(Pg);
let esize : integer{} = 64;
let msize : integer{} = 32;
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the base scalable vector register, encoded in the "Zn" field.
<Xm>	Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
var base : bits(VL);
var offset : bits(64);
var src : bits(VL);
let contiguous : boolean = FALSE;
let nontemporal : boolean = TRUE;
let predicated : boolean = TRUE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if AnyActiveElement{PL}(mask, esize) then
    base = Z{VL}(n);
    offset = X{64}(m);
    src = Z{VL}(t);
end;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let baddr : bits(64) = ZeroExtend{}(base[e*:esize]);
        let addr : bits(64) = AddressAdd(baddr, offset, accdesc);
        Mem{msize}(addr, accdesc) = src[e*:esize][msize-1:0];
    end;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STR (predicate)

Store predicate register

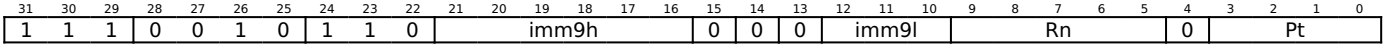
This instruction stores a predicate register to a memory address generated by a 64-bit scalar base, plus an immediate offset in the range -256 to 255 that is multiplied by the current predicate register size in bytes. This instruction is unpredicated.

The store is performed as contiguous byte accesses, each containing 8 consecutive predicate bits in ascending element order, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then a general-purpose base register must be aligned to 2 bytes.

For programmer convenience, an assembler must also accept a predicate-as-counter register name for the source predicate register.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
STR <Pt>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Pt);
let n : integer = UInt(Rn);
let imm : integer = SInt(imm9h::imm9l);
```

Assembler Symbols

<Pt>	Is the name of the scalable predicate transfer register, encoded in the "Pt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -256 to 255, defaulting to 0, encoded in the "imm9h:imm9l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = PL DIV 8;
var src : bits(PL);
var base : bits(64);
let offset : integer = imm * elements;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

src = P{PL}(t);

var addr : bits(64) = AddressAdd(base, offset, accdesc);

let aligned : boolean = IsAlignedSize(base, 2);

if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;

for e = 0 to elements-1 do
    AArch64_MemSingle{8}(addr, accdesc, aligned) = src[e*:8];
    addr = AddressIncrement(addr, 1, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STR (vector)

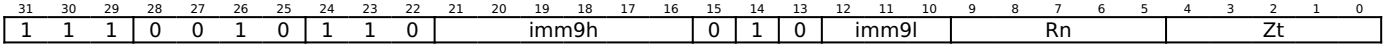
Store vector register

This instruction stores a vector register to a memory address generated by a 64-bit scalar base, plus an immediate offset in the range -256 to 255 that is multiplied by the current vector register size in bytes. This instruction is unpredicated.

The store is performed as contiguous byte accesses, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then the base register must be aligned to 16 bytes.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
STR <Zt>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let t : integer = UInt(Zt);
let n : integer = UInt(Rn);
let imm : integer = SInt(imm9h::imm9l);
```

Assembler Symbols

<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	Is the optional signed immediate vector offset, in the range -256 to 255, defaulting to 0, encoded in the "imm9h:imm9l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 8;
var src : bits(VL);
var base : bits(64);
let offset : integer = imm * elements;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let predicated : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  predicated, tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;

src = Z{VL}(t);

var addr : bits(64) = AddressAdd(base, offset, accdesc);

let aligned : boolean = IsAlignedSize(base, 16);

if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;

for e = 0 to elements-1 do
    AArch64_MemSingle{8}(addr, accdesc, aligned) = src[e*:8];
    addr = AddressIncrement(addr, 1, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUB (immediate)

Subtract immediate (unpredicated)

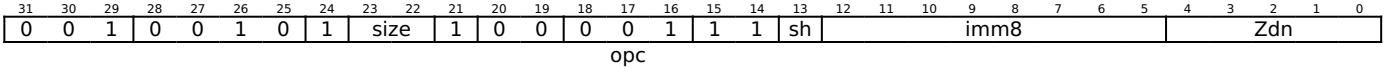
This instruction subtracts an unsigned immediate from each element of the source vector, and destructively places the results in the corresponding elements of the source vector. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SUB <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    result[e*:esize] = element1 - imm;

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

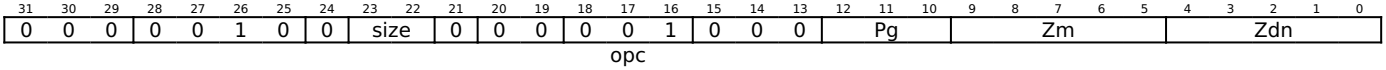
SUB (vectors, predicated)

Subtract (predicated)

This instruction subtracts Active elements of the second source vector from the corresponding elements of the first source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SUB <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = element1 - element2;
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

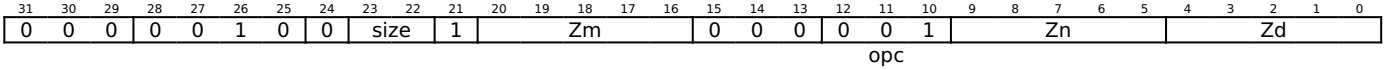
SUB (vectors, unpredicated)

Subtract (unpredicated)

This instruction subtracts elements of the second source vector from the corresponding elements of the first source vector, and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SUB <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = element1 - element2;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

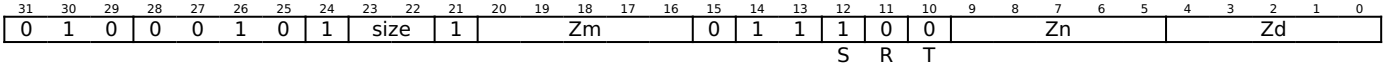
SUBHNB

Subtract narrow high part (bottom)

This instruction subtracts each vector element of the second source vector from the corresponding vector element in the first source vector, and places the most significant half of the result in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SUBHNB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	B
10	H
11	S

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    let res : integer = (element1 - element2) >> halfesize;
    result[(2*e + 0)*:halfesize] = res[halfesize-1:0];
    result[(2*e + 1)*:halfesize] = Zeros{halfesize};

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

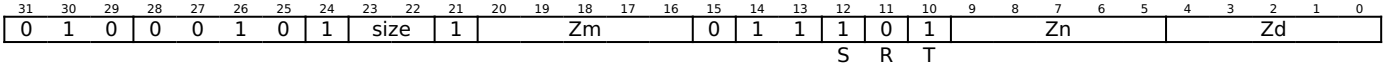
SUBHNT

Subtract narrow high part (top)

This instruction subtracts each vector element of the second source vector from the corresponding vector element in the first source vector, and places the most significant half of the result in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SUBHNT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	B
10	H
11	S

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	H
10	S
11	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(d);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    let res : integer = (element1 - element2) >> halfesize;
    result[(2*e + 1)*:halfesize] = res[halfesize-1:0];

end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

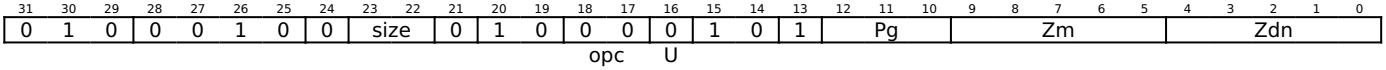
SUBP

Subtract pairwise

This instruction subtracts pairs of adjacent elements within each source vector, and destructively places the interleaved results in the first source vector.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
SUBP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    if !ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand1[e*:esize];
    else
        if IsEven(e) then
            element1 = UInt(operand1[(e + 0)*:esize]);
            element2 = UInt(operand1[(e + 1)*:esize]);
        else
            element1 = UInt(operand2[(e - 1)*:esize]);
            element2 = UInt(operand2[(e + 0)*:esize]);
        end;
        let res : integer = element1 - element2;
        result[e*:esize] = res[esize-1:0];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

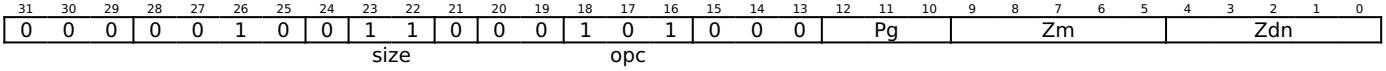
SUBPT (predicated)

Subtract checked pointer vectors (predicated)

This instruction subtracts, with pointer check, Active elements of the second source vector from corresponding elements of the first source vector and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.
This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE && FEAT_CPA)



Encoding

```
SUBPT <Zdn>.D, <Pg>/M, <Zdn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_CPA) then
    EndOfDecode(Decode_UNDEF);
end;
let g : integer =  UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer =  UInt(Zm);
```

Assembler Symbols

<Zdn>	Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV 64;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, 64) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(64) = operand1[e*:64];
    let element2 : bits(64) = operand2[e*:64];
    if ActivePredicateElement{PL}(mask, e, 64) then
        let res : bits(64) = element1 - element2;
        result[e*:64] = PointerAddCheck(res, element1);
    else
        result[e*:64] = operand1[e*:64];
    end;

    Z{VL}(dn) = result;
end;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUBPT (unpredicated)

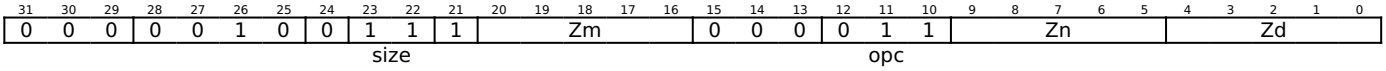
Subtract checked pointer vectors (unpredicated)

This instruction subtracts, with pointer check, all elements of the second source vector from corresponding elements of the first source vector and places the results in the corresponding elements of the destination vector. This instruction is unpredicated.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE2

(FEAT_SVE && FEAT_CPA)



Encoding

```
SUBPT <Zd>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_CPA) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 64;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(64) = operand1[e*:64];
    let element2 : bits(64) = operand2[e*:64];
    let res : bits(64) = element1 - element2;
    result[e*:64] = PointerAddCheck(res, element1);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUBR (immediate)

Reversed subtract from immediate (unpredicated)

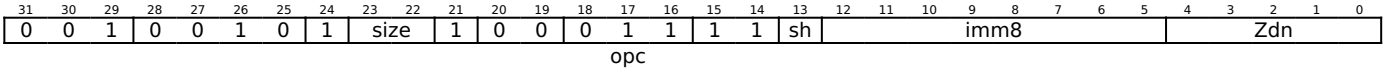
This instruction performs a reversed subtraction from an unsigned immediate each element of the source vector, and destructively places the results in the corresponding elements of the source vector. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SUBR <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    result[e*:esize] = (imm - element1)[esize-1:0];

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

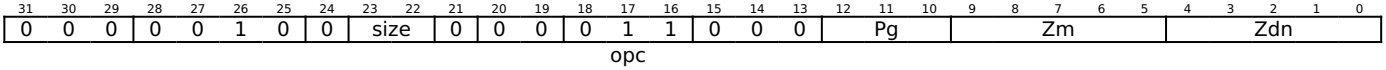
SUBR (vectors)

Reversed subtract (predicated)

This instruction performs a reversed subtraction on Active elements of the first source vector from the corresponding elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
SUBR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = element2 - element1;
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUDOT

Signed by unsigned 8-bit integer dot product by indexed element to 32-bit integer

This instruction calculates the dot product of a group of four signed 8-bit integer values held in each 32-bit element of the first source vector multiplied by a group of four unsigned 8-bit integer values in an indexed 32-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 32-bit element of the destination vector.

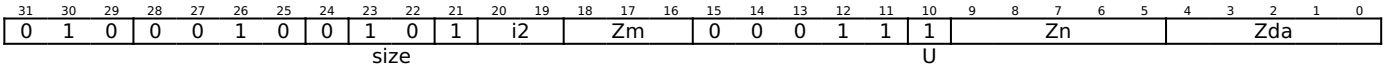
The groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment. The index range is from 0 to 3.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.I8MM indicates whether this instruction is implemented.

SVE

((FEAT_SVE || FEAT_SME) && FEAT_I8MM)



Encoding

```
SUDOT <Zda>.S, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_I8MM)) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 3 do
        let element1 : integer = SInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        let element2 : integer = UInt(operand2[(4 * s + i)*:(esize DIV 4)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUNPKHI, SUNPKLO

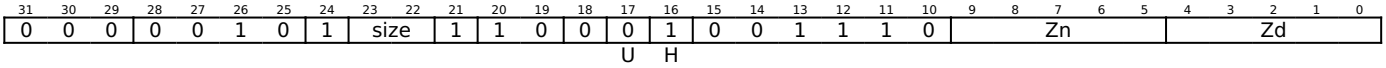
Signed unpack and extend half of vector

This instruction unpacks and sign-extends elements from the lowest or highest half of the source vector, and writes the sign-extended values into double-width elements of the destination vector. This instruction is unpredicated.

It has encodings from 2 classes: High half and Low half

High half

(FEAT_SVE || FEAT_SME)



Encoding

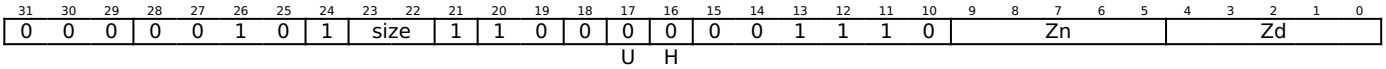
```
SUNPKHI <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let hi : boolean = TRUE;
```

Low half

(FEAT_SVE || FEAT_SME)



Encoding

```
SUNPKLO <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let hi : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let hsize : integer{} = esize DIV 2;
let offset : integer = if hi then elements else 0;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(hsize) = operand[(e + offset)*:hsize];
    result[e*:esize] = Extend{esize}(element, unsigned);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

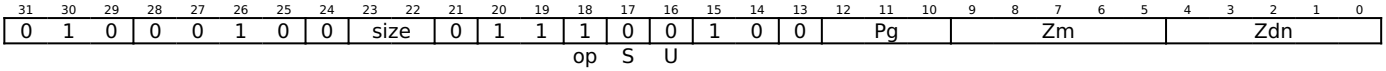
SUQADD

Signed saturating unsigned add

This instruction adds Active unsigned elements of the source vector to the corresponding signed elements of the addend vector, and destructively places the results in the corresponding elements of the addend vector. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
SUQADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = SignedSat{esize}(SInt(element1) + UInt(element2));
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

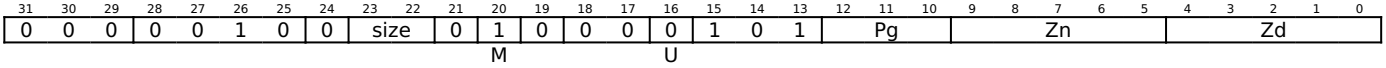
SXTB, SXTH, SXTW

Signed byte / halfword / word extend (predicated)

This instruction sign-extends the least-significant sub-element of each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.
It has encodings from 6 classes: Byte, merging , Byte, zeroing , Halfword, merging , Halfword, zeroing , Word, merging and Word, zeroing

Byte, merging

(FEAT_SVE || FEAT_SME)



Encoding

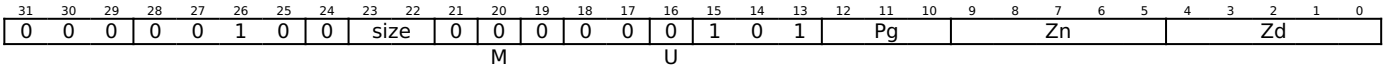
```
SXTB <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let merging : boolean = TRUE;
```

Byte, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

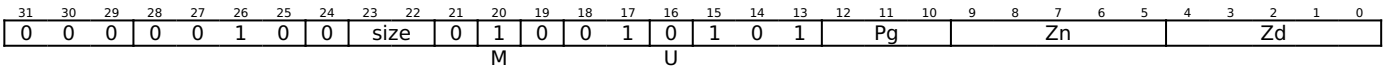
```
SXTB <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let merging : boolean = FALSE;
```

Halfword, merging

(FEAT_SVE || FEAT_SME)



Encoding

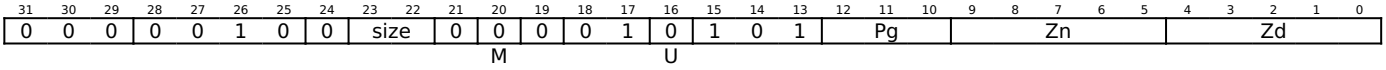
```
SXTH <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let merging : boolean = TRUE;
```

Halfword, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

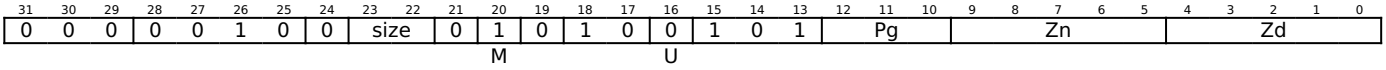
```
SXTH <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let merging : boolean = FALSE;
```

Word, merging

(FEAT_SVE || FEAT_SME)



Encoding

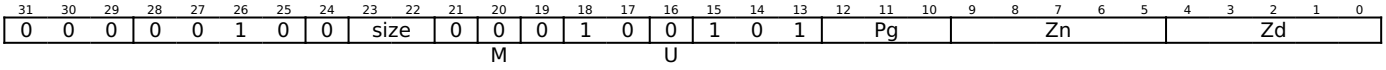
```
SXTW <Zd>.D, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let merging : boolean = TRUE;
```

Word, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
SXTW <Zd>.D, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = FALSE;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> For the "Byte, merging" and "Byte, zeroing" variants: is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

For the "Halfword, merging" and "Halfword, zeroing" variants: is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = Extend{esize}(element[s_esize-1:0], unsigned);
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TBL

Programmable table lookup in one or two vector table (zeroing)

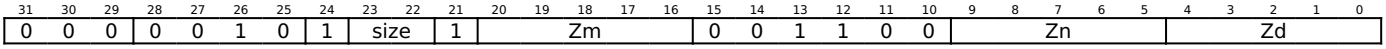
This instruction reads each element of the second source (index) vector and uses its value to select an indexed element from a table of elements consisting of one or two consecutive vector registers, where the first or only vector holds the lower numbered elements, and places the indexed table element in the destination vector element corresponding to the index vector element. If an index value is greater than or equal to the number of vector elements, then it places zero in the corresponding destination vector element.

Since the index values can select any element in a vector this operation is not naturally vector length agnostic.

It has encodings from 2 classes: Single register table and Two register table

Single register table

(FEAT_SVE || FEAT_SME)



Encoding

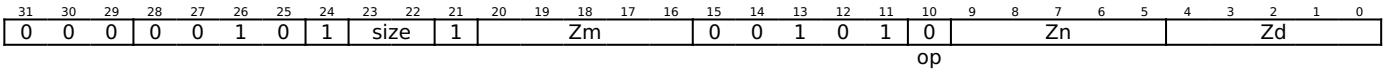
```
TBL <Zd>.<T>, { <Zn>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let double_table : boolean = FALSE;
```

Two register table

(FEAT_SVE2 || FEAT_SME)



Encoding

```
TBL <Zd>.<T>, { <Zn1>.<T>, <Zn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let double_table : boolean = TRUE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded in the "Zn" field.

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let indexes : bits(VL) = Z{}(m);
var result : bits(VL);
let table_size : integer{} = if double_table then VL*2 else VL;
let table_elems : integer = table_size DIV esize;
var table : bits(table_size);

if double_table then
    let top : bits(VL) = Z{}((n + 1) MOD 32);
    let bottom : bits(VL) = Z{}(n);
    table = (top::bottom)[table_size-1:0];
else
    table = Z{table_size}(n);
end;

for e = 0 to elements-1 do
    let idx : integer = UInt(indexes[e*:esize]);
    result[e*:esize] = if idx < table_elems then table[idx*:esize] else Zeros{esize};
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

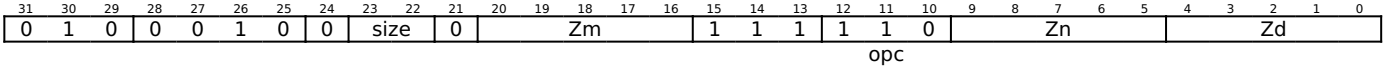
TBLQ

Programmable table lookup within each quadword vector segment (zeroing)

This instruction reads, for each 128-bit destination vector segment, each element of the corresponding second source (index) vector segment and uses its value to select an indexed element from the corresponding first source (table) vector segment. The indexed table element is placed in the element of the destination vector that corresponds to the index vector element. If an index value is greater than or equal to the number of elements in a 128-bit vector segment then it places zero in the corresponding destination vector element. This instruction is unpredicated.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
TBLQ <Zd>.<T>, { <Zn>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let elements : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for s = 0 to segments-1 do
    for e = 0 to elements-1 do
        let idx : integer = UInt(operand2[(s * elements + e)*:esize]);
        if idx < elements then
            result[(s * elements + e)*:esize] = operand1[(s * elements + idx)*:esize];
        else
            result[(s * elements + e)*:esize] = Zeros{esize};
        end;
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

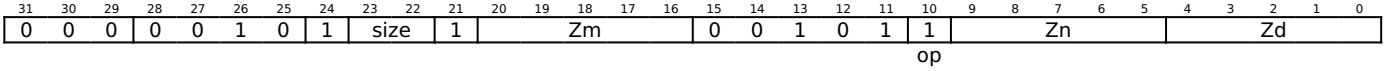
TBX

Programmable table lookup in single vector table (merging)

This instruction reads each element of the second source (index) vector and uses its value to select an indexed element from a table of elements in the first source vector, and places the indexed element in the destination vector element corresponding to the index vector element. If an index value is greater than or equal to the number of vector elements, then the corresponding destination vector element is left unchanged.
Since the index values can select any element in a vector this operation is not naturally vector length agnostic.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
TBX <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(d);

for e = 0 to elements-1 do
    let element2 : integer = UInt(operand2[e*:esize]);
    if element2 < elements then
        result[e*:esize] = operand1[element2*:esize];
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

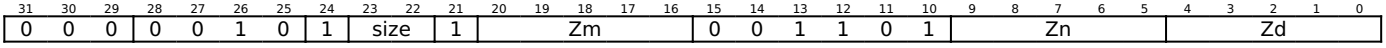
TBXQ

Programmable table lookup within each quadword vector segment (merging)

This instruction reads, for each 128-bit destination vector segment, each element of the corresponding second source (index) vector segment and uses its value to select an indexed element from the corresponding first source (table) vector segment. The indexed table element is placed in the element of the destination vector that corresponds to the index vector element. If an index value is greater than or equal to the number of elements in a 128-bit vector segment then the corresponding destination vector element is left unchanged. This instruction is unpredicated.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
TBXQ <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let elements : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(d);

for s = 0 to segments-1 do
    for e = 0 to elements-1 do
        let idx : integer = UInt(operand2[(s * elements + e)*:esize]);
        if idx < elements then
            result[(s * elements + e)*:esize] = operand1[(s * elements + idx)*:esize];
        end;
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TRN1, TRN2 (predicates)

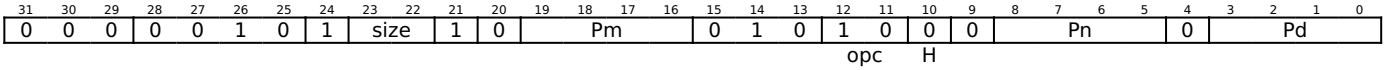
Interleave even or odd elements from two predicates

This instruction interleaves alternating even-numbered or odd-numbered elements from the first and second source predicates, and places the values in the destination predicate. This instruction is unpredicated.

It has encodings from 2 classes: Even and Odd

Even

(FEAT_SVE || FEAT_SME)



Encoding

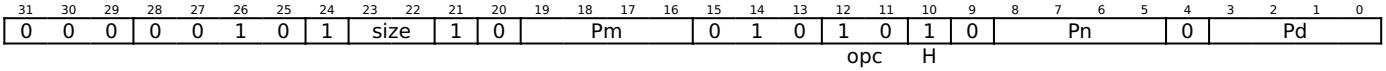
```
TRN1 <Pd>.<T>, <Pn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let part : integer = 0;
```

Odd

(FEAT_SVE || FEAT_SME)



Encoding

```
TRN2 <Pd>.<T>, <Pn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let part : integer = 1;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pn> Is the name of the first source scalable predicate register, encoded in the "Pn" field.

<Pm> Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let pairs : integer = VL DIV (esize * 2);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);

for p = 0 to pairs-1 do
    result[(2*p+0)*:(esize DIV 8)] = operand1[(2*p+part)*:(esize DIV 8)];
    result[(2*p+1)*:(esize DIV 8)] = operand2[(2*p+part)*:(esize DIV 8)];
end;

P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

TRN1, TRN2 (vectors)

Interleave even or odd elements from two vectors

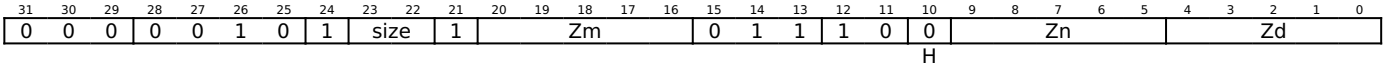
This instruction interleaves alternating even-numbered or odd-numbered elements from the first and second source vectors, and places the values in the destination vector. This instruction is unpredicated.

The Even (quadwords) and Odd (quadwords) variants require an Effective SVE vector length of at least 256 bits. ID_AA64ZFR0_EL1.F64MM or ID_AA64ZFR0_EL1.EltPerm indicates whether the Even (quadwords) and Odd (quadwords) variants are implemented. The Even (quadwords) and Odd (quadwords) variants are legal when executed in Streaming SVE mode if FEAT_SVE_PERM256 is implemented.

It has encodings from 4 classes: Even , Even (quadwords) , Odd and Odd (quadwords)

Even

(FEAT_SVE || FEAT_SME)



Encoding

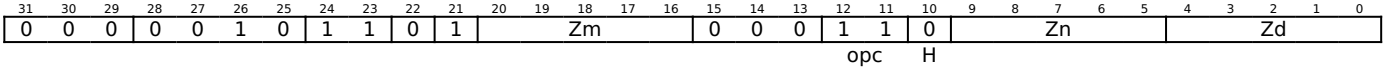
```
TRN1 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Even (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

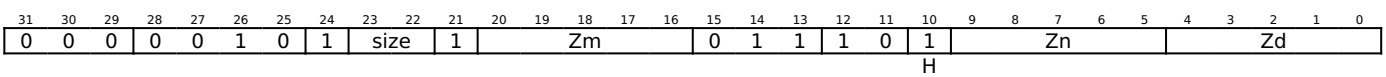
```
TRN1 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) then
    EndOfDecode(Decode_UNDEF);
end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Odd

(FEAT_SVE || FEAT_SME)



Encoding

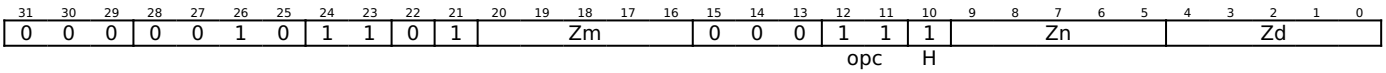
```
TRN2 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Odd (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

```
TRN2 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) then
    EndOfDecode(Decode_UNDEF);
end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if esize < 128 || IsFeatureImplemented(FEAT_SVE_PERM256) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
if VL < esize * 2 then EndOfDecode(Decode_UNDEF); end;
let pairs : integer = VL DIV (esize * 2);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Zeros{};

for p = 0 to pairs-1 do
    result[(2*p+0)*:esize] = operand1[(2*p+part)*:esize];
    result[(2*p+1)*:esize] = operand2[(2*p+part)*:esize];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

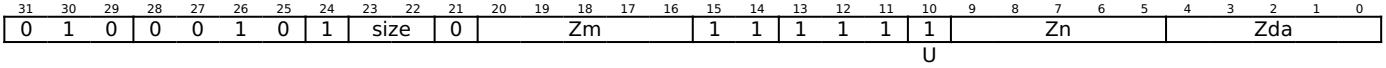
UABA

Unsigned absolute difference and accumulate

This instruction calculates the absolute difference between unsigned integer values in elements of the first source vector and the corresponding elements of the second source vector, and adds the difference to the corresponding elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UABA <Zda>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let op2elt : bits(esize) = operand2[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let absdiff : bits(esize) = Abs(element1 - element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + absdiff;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

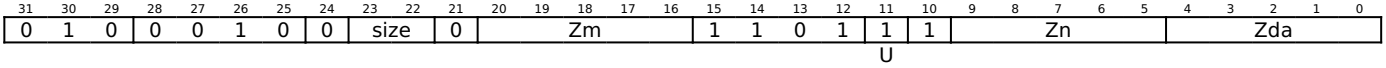
UABAL

Two-way unsigned absolute difference sum and accumulate long

This instruction calculates the sum of the absolute difference between even-numbered unsigned elements and the absolute difference between the odd-numbered unsigned elements of each double-width container of the first and second vectors, and destructively adds the results to the overlapping double-width elements of the addend vector.
This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
UABAL <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let elt1_a : integer = UInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let elt1_b : integer = UInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let elt2_a : integer = UInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let elt2_b : integer = UInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let sum = Abs(elt1_a - elt2_a) + Abs(elt1_b - elt2_b);
    result[e*:esize] = result[e*:esize] + sum[esize-1:0];
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

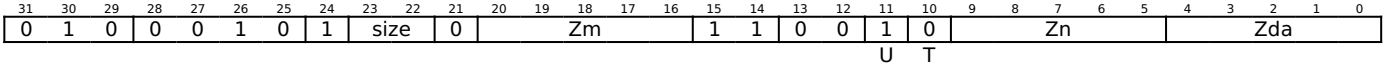
UABALB

Unsigned absolute difference and accumulate long (bottom)

This instruction calculates the absolute difference between even-numbered unsigned elements of the first source vector and the corresponding elements of the second source vector, and destructively adds the results to the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UABALB <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = UInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let absdiff = Abs(element1 - element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + absdiff;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

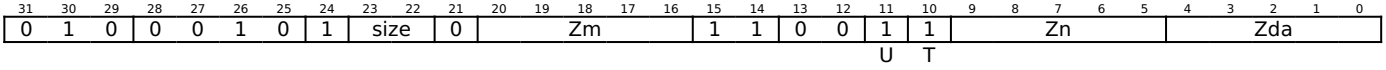
UABALT

Unsigned absolute difference and accumulate long (top)

This instruction calculates the absolute difference between odd-numbered unsigned elements of the first source vector and the corresponding elements of the second source vector, and destructively adds the results to the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UABALT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = UInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let absdiff = Abs(element1 - element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + absdiff;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

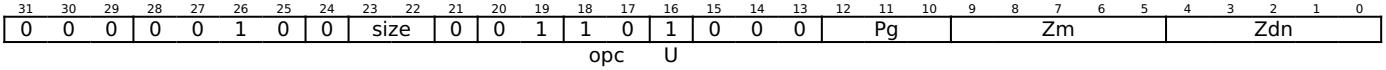
UABD

Unsigned absolute difference (predicated)

This instruction calculates the absolute difference between unsigned integer values in Active elements of the first source vector and the corresponding elements of the second source vector, and destructively places the difference in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UABD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let absdiff : integer = Abs(element1 - element2);
        result[e*:esize] = absdiff[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

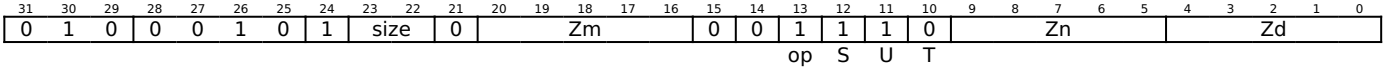
UABDLB

Unsigned absolute difference long (bottom)

This instruction calculates the absolute difference between the even-numbered unsigned integer values in elements of the first source vector and the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UABDLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = UInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let res : integer = Abs(element1 - element2);
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

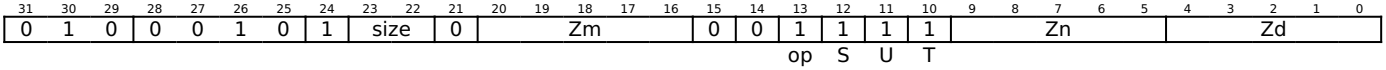
UABDLT

Unsigned absolute difference long (top)

This instruction calculates the absolute difference between the odd-numbered unsigned integer values in elements of the first source vector and the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UABDLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = UInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let res : integer = Abs(element1 - element2);
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

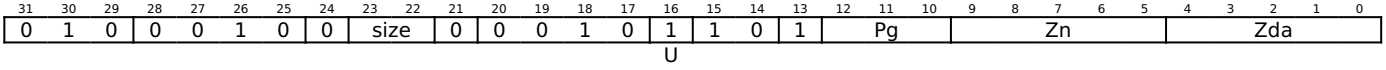
UADALP

Unsigned add and accumulate long pairwise

This instruction adds pairs of adjacent unsigned integer values and accumulates the results into the overlapping double-width elements of the destination vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UADALP <Zda>.<T>, <Pg>/M, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the second source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand_acc : bits(VL) = Z{}(da);
let operand_src : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if !ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand_acc[e*:esize];
    else
        let element1 : integer = UInt(operand_src[(2*e + 0)*:(esize DIV 2)]);
        let element2 : integer = UInt(operand_src[(2*e + 1)*:(esize DIV 2)]);
        let sum : bits(esize) = (element1 + element2)[esize-1:0];
        result[e*:esize] = operand_acc[e*:esize] + sum;
    end;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

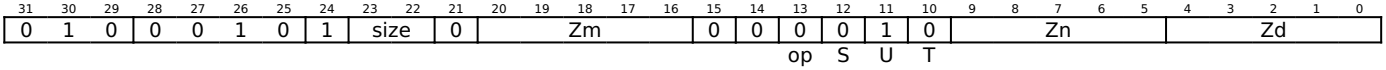
UADDLB

Unsigned add long (bottom)

This instruction adds the corresponding even-numbered unsigned elements of the first and second source vectors, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UADDLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 0;
let sel2 : integer = 0;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 + element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

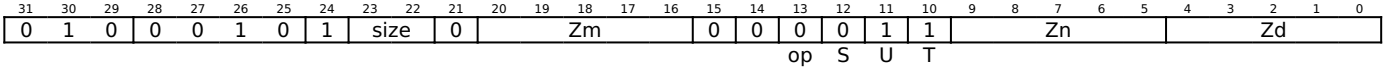
UADDLT

Unsigned add long (top)

This instruction adds the corresponding odd-numbered unsigned elements of the first and second source vectors, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UADDLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 1;
let sel2 : integer = 1;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 + element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

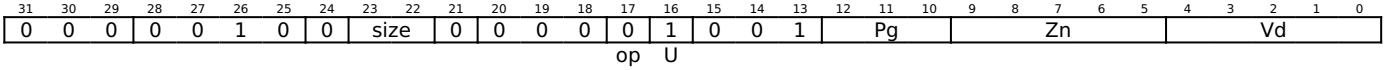
UADDV

Unsigned add reduction to scalar

This instruction adds all the unsigned elements of the source vector, and places the result in the SIMD&FP scalar destination register. Narrow elements are first zero-extended to 64 bits. Inactive elements in the source vector are treated as zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UADDV <Dd>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
```

Assembler Symbols

<Dd> Is the 64-bit name of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var sum : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = UInt(operand[e*:esize]);
        sum = sum + element;

    end;
end;
V{64}(d) = sum[63:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

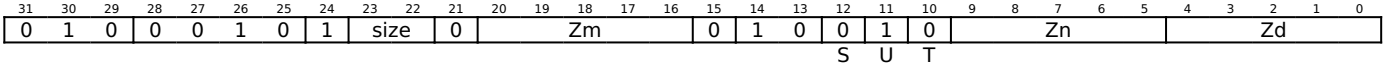
UADDWB

Unsigned add wide (bottom)

This instruction adds the even-numbered unsigned elements of the second source vector to the overlapping double-width elements of the first source vector and places the results in the corresponding double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UADDWB <Zd>.<T>, <Zn>.<T>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    result[e*:esize] = (element1 + element2)[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

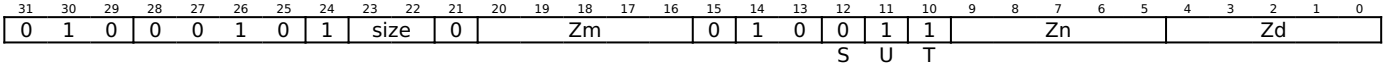
UADDWT

Unsigned add wide (top)

This instruction adds the odd-numbered unsigned elements of the second source vector to the overlapping double-width elements of the first source vector and places the results in the corresponding double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UADDWT <Zd>.<T>, <Zn>.<T>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    result[e*:esize] = (element1 + element2)[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

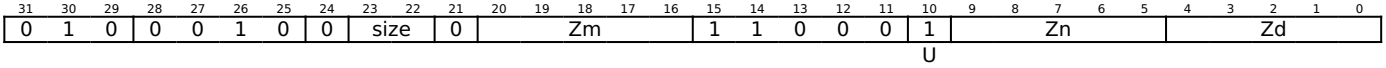
UCLAMP

Unsigned clamp to minimum/maximum

This instruction clamps each unsigned element in the destination vector to between the unsigned minimum value in the corresponding element of the first source vector and the unsigned maximum value in the corresponding element of the second source vector and destructively writes the results in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SME || FEAT_SVE2p1)



Encoding

```
UCLAMP <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(d);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    let element3 : integer = UInt(operand3[e*:esize]);
    let res : integer = Min(Max(element1, element3), element2);
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UCVTF (bottom, unpredicated)

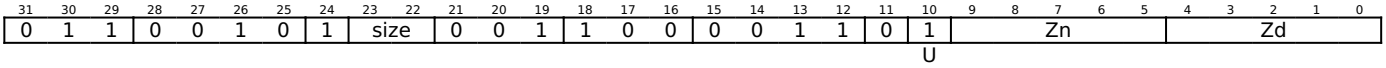
Unsigned integer convert to floating-point (bottom, unpredicated)

This instruction converts each even-numbered unsigned integer element of the source vector to floating-point, and places the results in the overlapping double-width elements of the destination vector.

This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
UCVTF <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize DIV 2) = operand[(2*e + 0)*:(esize DIV 2)];
    result[e*:esize] = FixedToFP{esize, esize DIV 2}(element, 0, unsigned, FPCR(), rounding);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UCVTF (predicated)

Unsigned integer convert to floating-point (predicated)

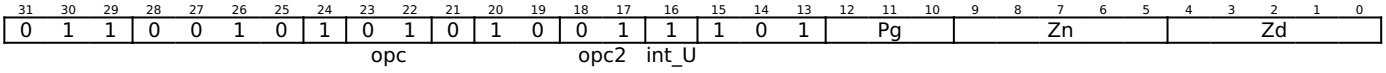
This instruction converts to floating-point from the unsigned integer in each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

If the input and result types have a different size the smaller type is held unpacked in the least significant bits of elements of the larger size. When the input is the smaller type the upper bits of each source element are ignored. When the result is the smaller type the results are zero-extended to fill each destination element.

It has encodings from 14 classes: 16-bit to half-precision, merging , 16-bit to half-precision, zeroing , 32-bit to half-precision, merging , 32-bit to half-precision, zeroing , 32-bit to single-precision, merging , 32-bit to single-precision, zeroing , 32-bit to double-precision, merging , 32-bit to double-precision, zeroing , 64-bit to half-precision, merging , 64-bit to half-precision, zeroing , 64-bit to single-precision, merging , 64-bit to single-precision, zeroing , 64-bit to double-precision, merging and 64-bit to double-precision, zeroing

16-bit to half-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

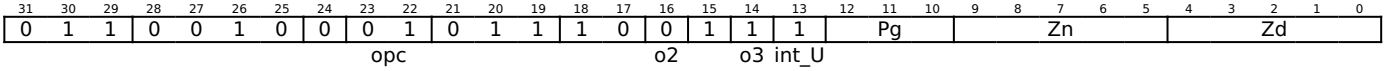
```
UCVTF <Zd>.H, <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 16;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

16-bit to half-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

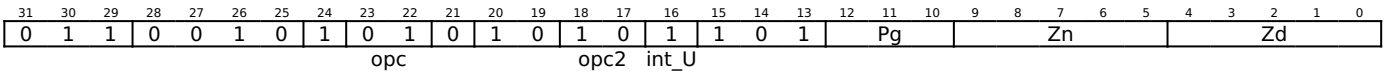
```
UCVTF <Zd>.H, <Pg>/Z, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 16;
let d_esize : integer{} = 16;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

32-bit to half-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

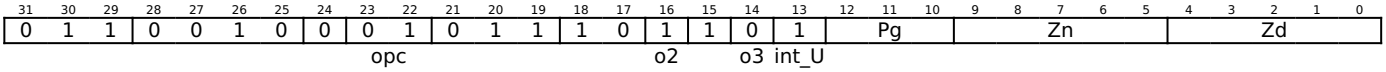
```
UCVTF <Zd>.H, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 16;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

32-bit to half-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

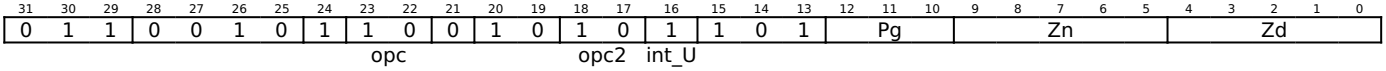
```
UCVTF <Zd>.H, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 16;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

32-bit to single-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

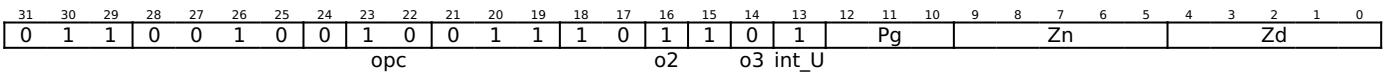
```
UCVTF <Zd>.S, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 32;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

32-bit to single-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

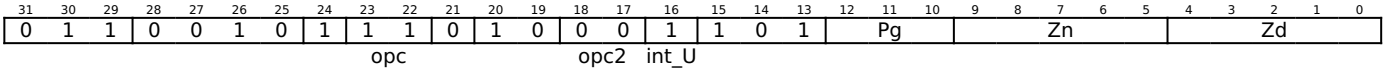
```
UCVTF <Zd>.S, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 32;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

32-bit to double-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

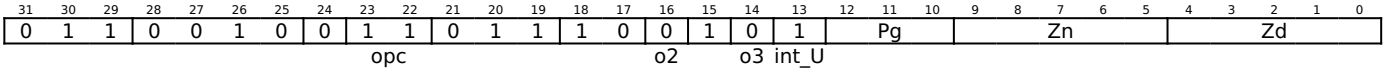
```
UCVTF <Zd>.D, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 64;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

32-bit to double-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

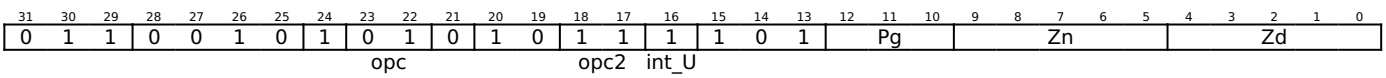
```
UCVTF <Zd>.D, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 32;
let d_esize : integer{} = 64;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

64-bit to half-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

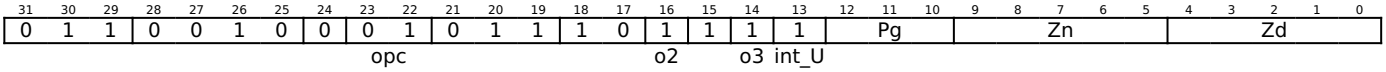
```
UCVTF <Zd>.H, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 16;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

64-bit to half-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

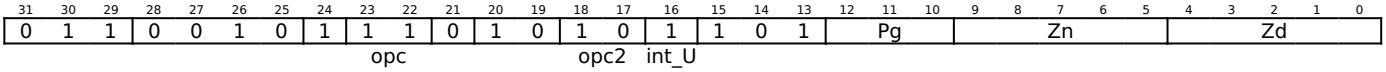
```
UCVTF <Zd>.H, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 16;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

64-bit to single-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

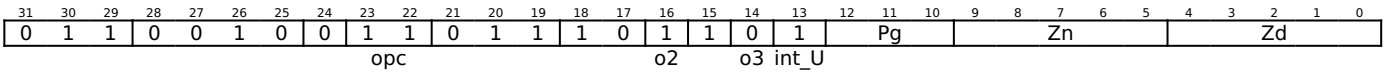
```
UCVTF <Zd>.S, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

64-bit to single-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

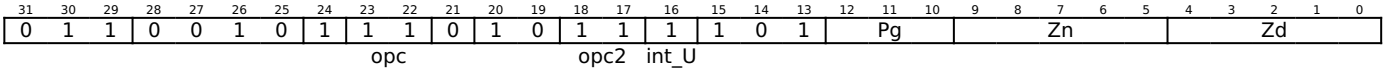
```
UCVTF <Zd>.S, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 32;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

64-bit to double-precision, merging

(FEAT_SVE || FEAT_SME)



Encoding

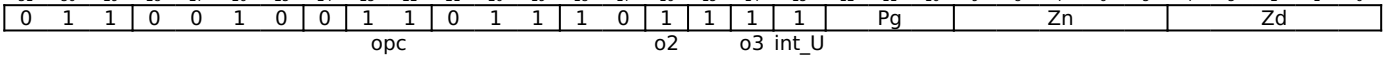
```
UCVTF <Zd>.D, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 64;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = TRUE;
```

64-bit to double-precision, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
UCVTF <Zd>.D, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let s_esize : integer{} = 64;
let d_esize : integer{} = 64;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        let fpval : bits(d_esize) = FixedToFP{d_esize, s_esize}(element[s_esize-1:0], 0, unsigned,
                                                                FPCR(), rounding);
        result[e*:esize] = ZeroExtend{esize}(fpval);
    end;
end;

Z{VL}(d) = result;
```

Operational information

For the "16-bit to half-precision, merging" , "32-bit to half-precision, merging" , "32-bit to single-precision, merging" , "32-bit to double-precision, merging" , "64-bit to half-precision, merging" , "64-bit to single-precision, merging" , "64-bit to double-precision, merging" variants:

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UCVTFLT

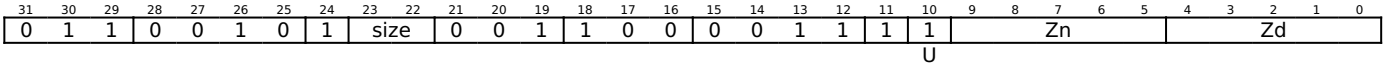
Unsigned integer convert to floating-point (top, unpredicated)

This instruction converts each odd-numbered unsigned integer element of the source vector to floating-point, and places the results in the overlapping double-width elements of the destination vector.

This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
UCVTFLT <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize DIV 2) = operand[(2*e + 1)*:(esize DIV 2)];
    result[e*:esize] = FixedToFP{esize, esize DIV 2}(element, 0, unsigned, FPCR(), rounding);
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

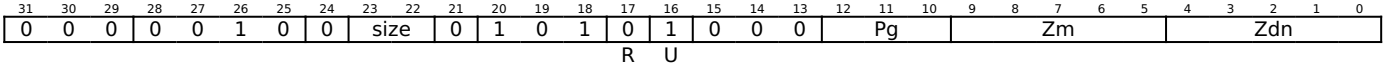
UDIV

Unsigned divide (predicated)

This instruction performs an unsigned division on Active elements of the first source vector by the corresponding elements of the second source vector and destructively places the quotient in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UDIV <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let dividend : integer = UInt(operand1[e*:esize]);
        let divisor : integer = UInt(operand2[e*:esize]);
        var quotient : integer;
        if divisor == 0 then
            quotient = 0;
        else
            quotient = dividend DIVRM divisor;
        end;
        result[e*:esize] = quotient[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

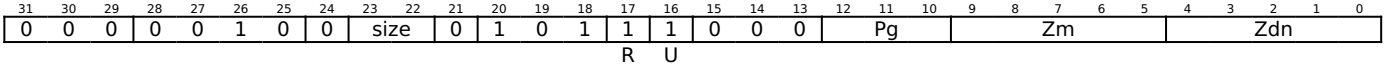
UDIVR

Unsigned reversed divide (predicated)

This instruction performs an unsigned reversed division on Active elements of the second source vector by the corresponding elements of the first source vector, and destructively places the quotient in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UDIVR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let divisor : integer = UInt(operand1[e*:esize]);
        let dividend : integer = UInt(operand2[e*:esize]);
        var quotient : integer;
        if divisor == 0 then
            quotient = 0;
        else
            quotient = dividend DIVRM divisor;
        end;
        result[e*:esize] = quotient[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (2-way, indexed)

Unsigned integer dot product by indexed element (two-way)

This instruction calculates the dot product of a pair of unsigned 8-bit or 16-bit integer values held in each 16-bit or 32-bit element of the first source vector multiplied by a pair of unsigned 8-bit or 16-bit integer values in an indexed 16-bit or 32-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 16-bit or 32-bit element of the destination vector.

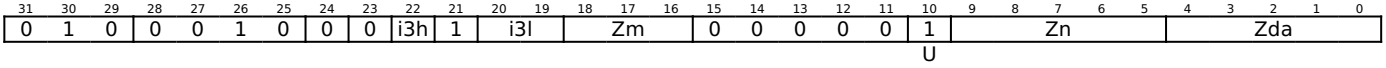
The groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment. The index range is from 0 to one less than the number of groups per 128-bit segment.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit to 16-bit and 16-bit to 32-bit

8-bit to 16-bit

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

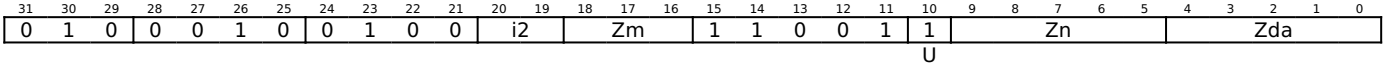
```
UDOT <Zda>.H, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

16-bit to 32-bit

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
UDOT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	For the "8-bit to 16-bit" variant: is the immediate index of a pair of 8-bit elements within each 128-bit vector segment, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "16-bit to 32-bit" variant: is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 1 do
        let element1 : integer = UInt(operand1[(2 * e + i)*:(esize DIV 2)]);
        let element2 : integer = UInt(operand2[(2 * s + i)*:(esize DIV 2)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (2-way, vectors)

Unsigned integer dot product (two-way)

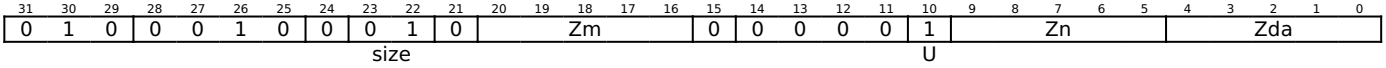
This instruction calculates the dot product of a pair of unsigned 8-bit or 16-bit integer values held in each 16-bit or 32-bit element of the first source vector multiplied by a pair of unsigned 8-bit or 16-bit integer values in the corresponding 16-bit or 32-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 16-bit or 32-bit element of the destination vector.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit to 16-bit and 16-bit to 32-bit

8-bit to 16-bit

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

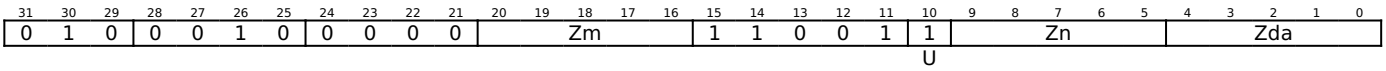
```
UDOT <Zda>.H, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

16-bit to 32-bit

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
UDOT <Zda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 1 do
        let element1 : integer = UInt(operand1[(2 * e + i)*:(esize DIV 2)]);
        let element2 : integer = UInt(operand2[(2 * e + i)*:(esize DIV 2)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (4-way, indexed)

Unsigned integer dot product by indexed element (four-way)

This instruction calculates the dot product of a group of four unsigned 8-bit or 16-bit integer values held in each 32-bit or 64-bit element of the first source vector multiplied by a group of four unsigned 8-bit or 16-bit integer values in an indexed 32-bit or 64-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 32-bit or 64-bit element of the destination vector.

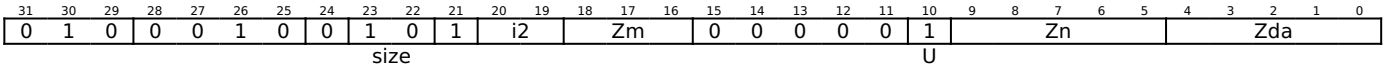
The groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment. The index range is from 0 to one less than the number of groups per 128-bit segment.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit to 32-bit and 16-bit to 64-bit

8-bit to 32-bit

(FEAT_SVE || FEAT_SME)



Encoding

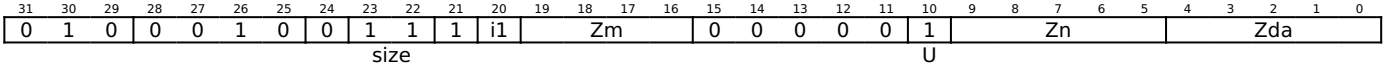
```
UDOT <Zda>.S, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

16-bit to 64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UDOT <Zda>.D, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let index : integer = UInt(i1);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "8-bit to 32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "16-bit to 64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "8-bit to 32-bit" variant: is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field. For the "16-bit to 64-bit" variant: is the immediate index of a group of four 16-bit values within each 128-bit vector segment, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 3 do
        let element1 : integer = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        let element2 : integer = UInt(operand2[(4 * s + i)*:(esize DIV 4)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

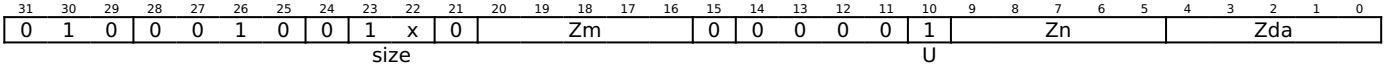
UDOT (4-way, vectors)

Unsigned integer dot product (four-way)

This instruction calculates the dot product of a group of four unsigned 8-bit or 16-bit integer values held in each 32-bit or 64-bit element of the first source vector multiplied by a group of four unsigned 8-bit or 16-bit integer values in the corresponding 32-bit or 64-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 32-bit or 64-bit element of the destination vector.
This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UDOT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size[0]”:

size[0]	<Tb>
0	B
1	H

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 3 do
        let element1 : integer = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        let element2 : integer = UInt(operand2[(4 * e + i)*:(esize DIV 4)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

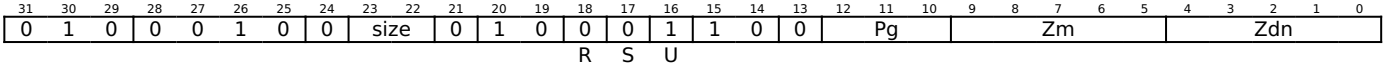
UHADD

Unsigned halving add

This instruction adds Active unsigned elements of the first source vector to corresponding unsigned elements of the second source vector, shifts right one bit, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UHADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = (element1 + element2) >> 1;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

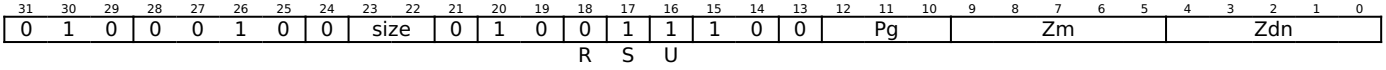
UHSUB

Unsigned halving subtract

This instruction subtracts Active unsigned elements of the second source vector from corresponding unsigned elements of the first source vector, shifts right one bit, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UHSUB <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = (element1 - element2) >> 1;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

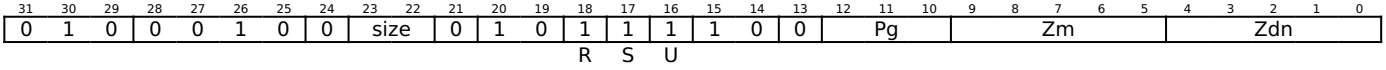
UHSUBR

Unsigned halving subtract reversed

This instruction subtracts Active unsigned elements of the first source vector from corresponding unsigned elements of the second source vector, shifts right one bit, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UHSUBR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = (element2 - element1) >> 1;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

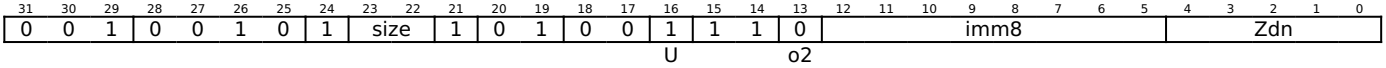
UMAX (immediate)

Unsigned maximum with immediate (unpredicated)

This instruction determines the unsigned maximum of an immediate and each element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate is an unsigned 8-bit value in the range 0 to 255, inclusive. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UMAX <Zdn>.<T>, <Zdn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
let unsigned : boolean = TRUE;
let imm : integer = if unsigned then UInt(imm8) else SInt(imm8);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is the unsigned immediate operand, in the range 0 to 255, encoded in the "imm8" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    result[e*:esize] = Max(element1, imm)[esize-1:0];

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

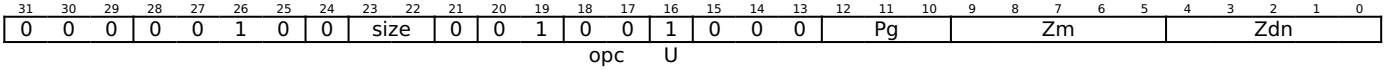
UMAX (vectors)

Unsigned maximum (predicated)

This instruction determines the unsigned maximum of Active elements of the first source vector and the corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UMAX <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let maximum : integer = Max(element1, element2);
        result[e*:esize] = maximum[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

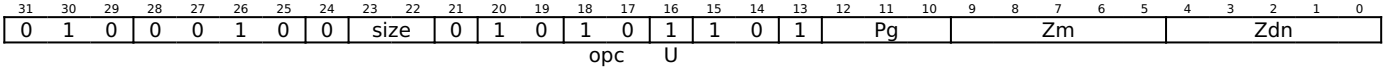
UMAXP

Unsigned maximum pairwise

This instruction calculates the maximum value of each pair of adjacent unsigned integer elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMAXP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    if !ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand1[e*:esize];
    else
        if IsEven(e) then
            element1 = UInt(operand1[(e + 0)*:esize]);
            element2 = UInt(operand1[(e + 1)*:esize]);
        else
            element1 = UInt(operand2[(e - 1)*:esize]);
            element2 = UInt(operand2[(e + 0)*:esize]);
        end;
        let res : integer = Max(element1, element2);
        result[e*:esize] = res[esize-1:0];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

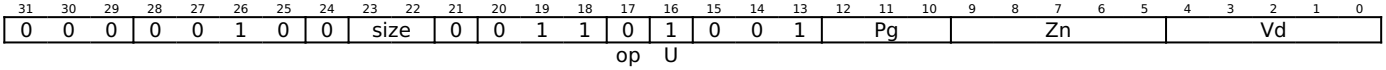
UMAXQV

Unsigned maximum reduction of quadword vector segments

This instruction determines the unsigned maximum of the elements in the same position from each 128-bit source vector segment, and places each result into the corresponding element of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as zero.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
UMAXQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	16B
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(128) = Zeros{};
var stmp : bits(128) = Zeros{};

var dtmp : integer;

for e = 0 to elempersegment-1 do
    dtmp = if unsigned then 0 else -(2^(esize-1));
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp = operand[s*:128];
            dtmp = Max(dtmp, UInt(stmp[e*:esize]));
        end;
    end;
    result[e*:esize] = dtmp[esize-1:0];
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

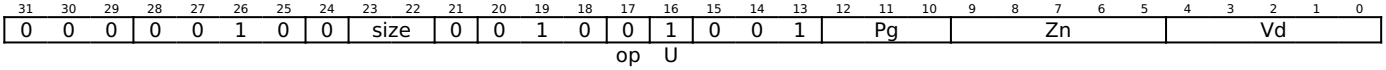
UMAXV

Unsigned maximum reduction to scalar

This instruction determines the unsigned maximum of the elements of the source vector, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UMAXV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var maximum : integer = if unsigned then 0 else -(2^(esize-1));

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let opelt : bits(esize) = operand[e*:esize];
        let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
        maximum = Max(maximum, element);

    end;
end;
V{esize}(d) = maximum[esize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

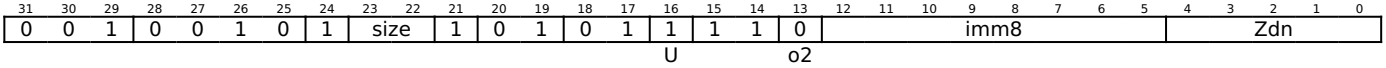
UMIN (immediate)

Unsigned minimum with immediate (unpredicated)

This instruction determines the unsigned minimum of an immediate and each element of the source vector, and destructively places the results in the corresponding elements of the source vector. The immediate is an unsigned 8-bit value in the range 0 to 255, inclusive. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UMIN <Zdn>.<T>, <Zdn>.<T>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
let unsigned : boolean = TRUE;
let imm : integer = if unsigned then UInt(imm8) else SInt(imm8);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is the unsigned immediate operand, in the range 0 to 255, encoded in the "imm8" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    result[e*:esize] = Min(element1, imm)[esize-1:0];

end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

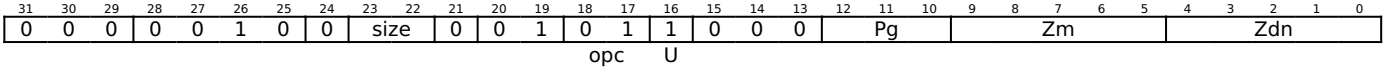
UMIN (vectors)

Unsigned minimum (predicated)

This instruction determines the unsigned minimum of Active elements of the first source vector and the corresponding elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UMIN <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let minimum : integer = Min(element1, element2);
        result[e*:esize] = minimum[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

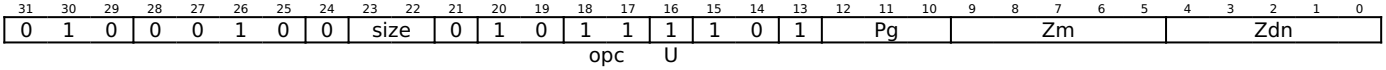
UMINP

Unsigned minimum pairwise

This instruction calculates the minimum value of each pair of adjacent unsigned integer elements within each source vector, and interleaves the results from corresponding lanes. The interleaved result values are destructively placed in the first source vector.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMINP <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);
var element1 : integer;
var element2 : integer;

for e = 0 to elements-1 do
    if !ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = operand1[e*:esize];
    else
        if IsEven(e) then
            element1 = UInt(operand1[(e + 0)*:esize]);
            element2 = UInt(operand1[(e + 1)*:esize]);
        else
            element1 = UInt(operand2[(e - 1)*:esize]);
            element2 = UInt(operand2[(e + 0)*:esize]);
        end;
        let res : integer = Min(element1, element2);
        result[e*:esize] = res[esize-1:0];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

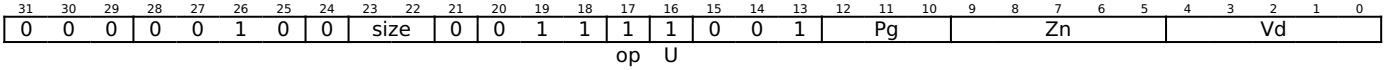
UMINQV

Unsigned minimum reduction of quadword vector segments

This instruction determines the unsigned minimum of the elements in the same position from each 128-bit source vector segment, and places each result into the corresponding element of the 128-bit SIMD&FP destination register. Inactive elements in the source vector are treated as the maximum unsigned integer for the element size.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
UMINQV <Vd>.<T>, <Pg>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Vd> Is the name of the destination SIMD&FP register, encoded in the "Vd" field.

<T> Is an arrangement specifier, encoded in “size”:

size	<T>
00	16B
01	8H
10	4S
11	2D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let segments : integer = VL DIV 128;
let elempersegment : integer = 128 DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(128) = Zeros{};
var stmp : bits(128) = Zeros{};

var dtmp : integer;

for e = 0 to elempersegment-1 do
    dtmp = if unsigned then (2^esize - 1) else (2^(esize-1) - 1);
    for s = 0 to segments-1 do
        if ActivePredicateElement{PL}(mask, s * elempersegment + e, esize) then
            stmp = operand[s*:128];
            dtmp = Min(dtmp, UInt(stmp[e*:esize]));
        end;
    end;
    result[e*:esize] = dtmp[esize-1:0];
end;

V{128}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

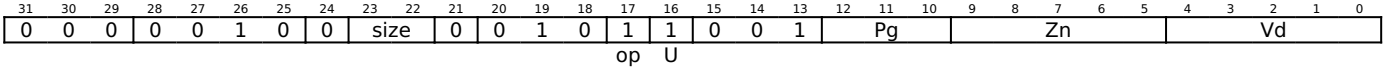
UMINV

Unsigned minimum reduction to scalar

This instruction determines the unsigned minimum of the elements of the source vector, and places the result in the SIMD&FP scalar destination register. Inactive elements in the source vector are treated as the maximum unsigned integer for the element size.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UMINV <V><d>, <Pg>, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Vd);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<V> Is a width specifier, encoded in “size”:

size	<V>
00	B
01	H
10	S
11	D

<d> Is the number [0-31] of the destination SIMD&FP register, encoded in the "Vd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var minimum : integer = if unsigned then (2^esize - 1) else (2^(esize-1) - 1);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let opelt : bits(esize) = operand[e*:esize];
        let element : integer = if unsigned then UInt(opelt) else SInt(opelt);
        minimum = Min(minimum, element);

    end;
end;
V{esize}(d) = minimum[esize-1:0];
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLALB (indexed)

Unsigned multiply-add long by indexed element (bottom)

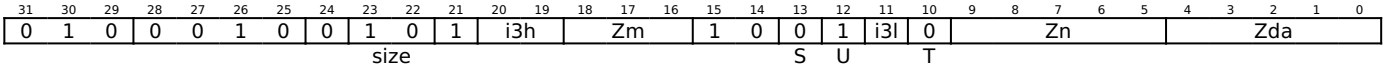
This instruction multiplies the even-numbered unsigned elements within each 128-bit segment of the first source vector by the specified unsigned element in the corresponding second source vector segment, and destructively adds the results to the overlapping double-width elements of the addend vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

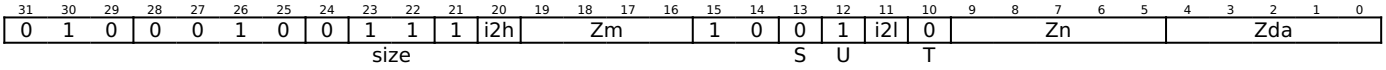
```
UMLALB <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMLALB <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = UInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = UInt(operand2[(2 * s + index)*:esize]);
    let product : bits(2*esize) = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] + product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

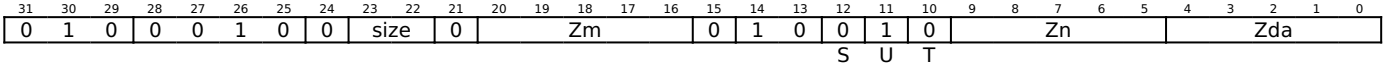
UMLALB (vectors)

Unsigned multiply-add long (bottom)

This instruction multiplies the even-numbered unsigned elements of the first source vector by the corresponding elements of the second source vector, and destructively adds the results to the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMLALB <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = UInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let product = (element1 * element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLALT (indexed)

Unsigned multiply-add long by indexed element (top)

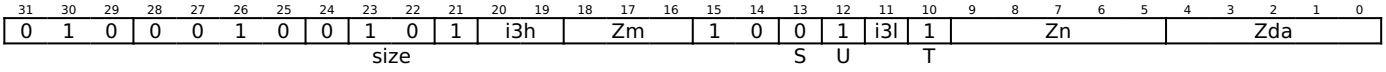
This instruction multiplies the odd-numbered unsigned elements within each 128-bit segment of the first source vector by the specified unsigned element in the corresponding second source vector segment, and destructively adds the results to the overlapping double-width elements of the addend vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

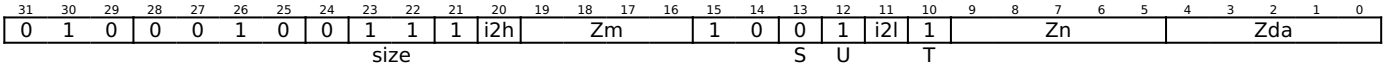
```
UMLALT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMLALT <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = UInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = UInt(operand2[(2 * s + index)*:esize]);
    let product : bits(2*esize) = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] + product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

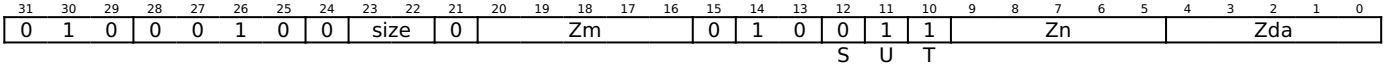
UMLALT (vectors)

Unsigned multiply-add long (top)

This instruction multiplies the odd-numbered unsigned elements of the first source vector by the corresponding elements of the second source vector, and destructively adds the results to the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMLALT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = UInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let product = (element1 * element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] + product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLSLB (indexed)

Unsigned multiply-subtract long by indexed element (bottom)

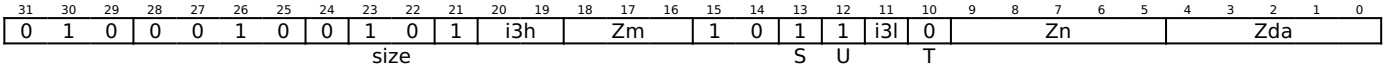
This instruction multiplies the even-numbered unsigned elements within each 128-bit segment of the first source vector by the specified unsigned element in the corresponding second source vector segment, and destructively subtracts the results from the overlapping double-width elements of the addend vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

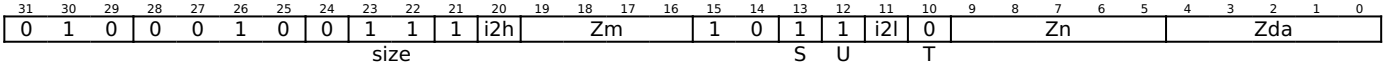
```
UMLSLB <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMLSLB <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 0;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = UInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = UInt(operand2[(2 * s + index)*:esize]);
    let product : bits(2*esize) = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] - product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

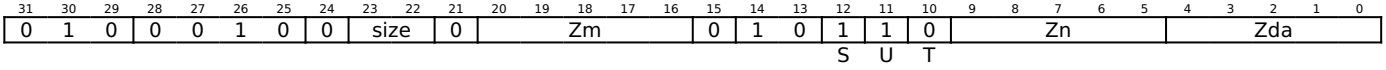
UMLSLB (vectors)

Unsigned multiply-subtract long (bottom)

This instruction multiplies the even-numbered unsigned elements of the first source vector by the corresponding elements of the second source vector, and destructively subtracts the results from the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMLSLB <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = UInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let product = (element1 * element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] - product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLSLT (indexed)

Unsigned multiply-subtract long by indexed element (top)

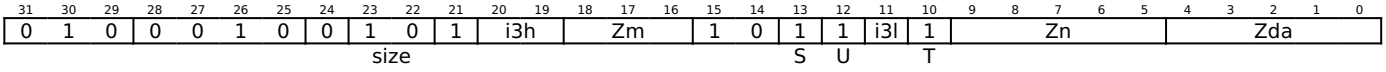
This instruction multiplies the odd-numbered unsigned elements within each 128-bit segment of the first source vector by the specified unsigned element in the corresponding second source vector segment, and destructively subtracts the results from the overlapping double-width elements of the addend vector.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

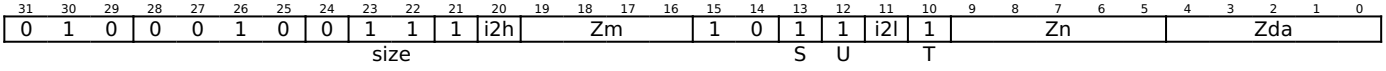
```
UMLSLT <Zda>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMLSLT <Zda>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let sel : integer = 1;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = UInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = UInt(operand2[(2 * s + index)*:esize]);
    let product : bits(2*esize) = (element1 * element2)[2*esize-1:0];
    result[e*:(2*esize)] = result[e*:(2*esize)] - product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

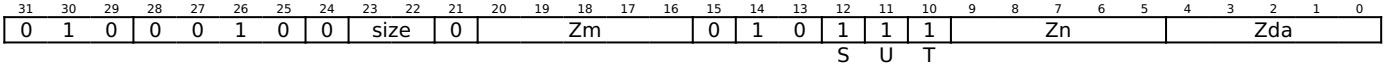
UMLSLT (vectors)

Unsigned multiply-subtract long (top)

This instruction multiplies the odd-numbered unsigned elements of the first source vector by the corresponding elements of the second source vector, and destructively subtracts the results from the overlapping double-width elements of the addend vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMLSLT <Zda>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Z{}(da);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = UInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let product = (element1 * element2)[esize-1:0];
    result[e*:esize] = result[e*:esize] - product;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMMLA

Unsigned 8-bit integer matrix multiply-accumulate to 32-bit integer

This instruction multiplies the 2×8 matrix of unsigned 8-bit integer values held in each 128-bit segment of the first source vector by the 8×2 matrix of unsigned 8-bit integer values in the corresponding segment of the second source vector. The resulting 2×2 widened 32-bit integer matrix product is then destructively added to the 32-bit integer matrix accumulator held in the corresponding segment of the addend and destination vector. This is equivalent to performing an 8-way dot product per destination element.

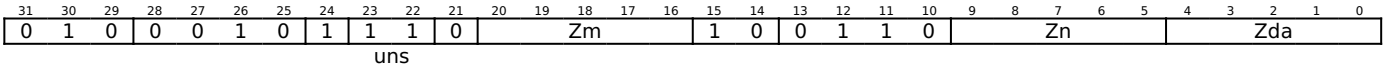
This instruction is unpredicated.

ID_AA64ZFR0_EL1.I8MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE && FEAT_I8MM)



Encoding

```
UMMLA <Zda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_I8MM) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL) = Zeros{};
var op1, op2 : bits(128);
var res, addend : bits(128);

for s = 0 to segments-1 do
    op1    = operand1[s*:(128)];
    op2    = operand2[s*:(128)];
    addend = operand3[s*:(128)];
    res    = MatMulAdd(addend, op1, op2, op1_unsigned, op2_unsigned);
    result[s*:(128)] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

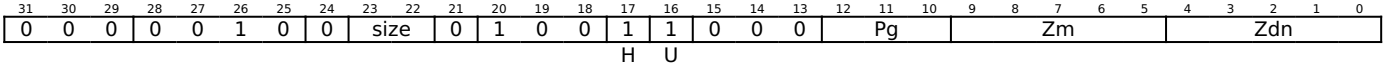
UMULH (predicated)

Unsigned multiply returning high half (predicated)

This instruction performs a widening multiplication of unsigned integer values in Active elements of the first source vector by the corresponding elements of the second source vector and destructively places the high half of the result in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UMULH <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let product : integer = (element1 * element2) >> esize;
        result[e*:esize] = product[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.

- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

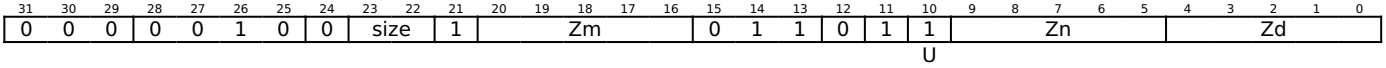
UMULH (unpredicated)

Unsigned multiply returning high half (unpredicated)

This instruction performs a widening multiplication of unsigned integer values of all elements of the first source vector by the corresponding elements of the second source vector and places the high half of the result in the corresponding elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMULH <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let op2elt : bits(esize) = operand2[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let product : integer = (element1 * element2) >> esize;
    result[e*:esize] = product[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMULLB (indexed)

Unsigned multiply long by indexed element (bottom)

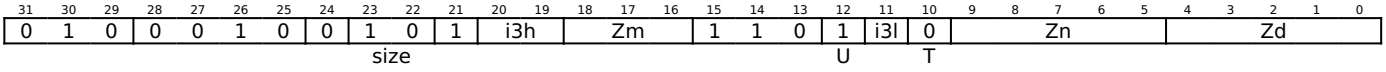
This instruction multiplies the even-numbered unsigned elements within each 128-bit segment of the first source vector by the specified unsigned element in the corresponding second source vector segment, and places the results in the overlapping double-width elements of the destination vector register.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

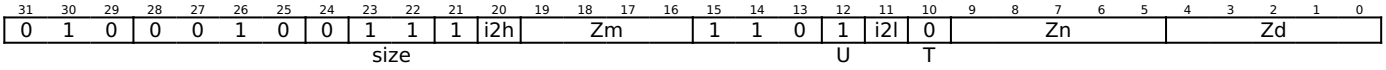
```
UMULLB <Zd>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 0;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMULLB <Zd>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 0;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = UInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = UInt(operand2[(2 * s + index)*:esize]);
    let res : integer = element1 * element2;
    result[e*:(2*esize)] = res[2*esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

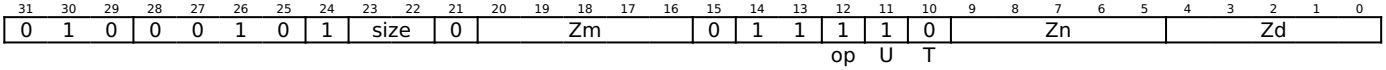
UMULLB (vectors)

Unsigned multiply long (bottom)

This instruction multiplies the even-numbered unsigned elements of the first source vector by the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMULLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[(2*e + 0)*:(esize DIV 2)]);
    let element2 : integer = UInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    let res : integer = element1 * element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMULLT (indexed)

Unsigned multiply long by indexed element (top)

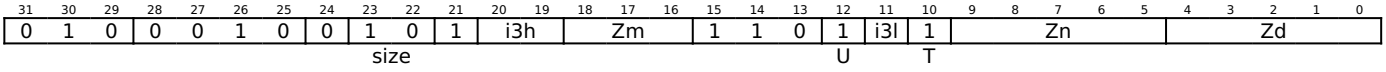
This instruction multiplies the odd-numbered unsigned elements within each 128-bit segment of the first source vector by the specified unsigned element in the corresponding second source vector segment, and places the results in the overlapping double-width elements of the destination vector register.

The elements within the second source vector are specified using an immediate index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

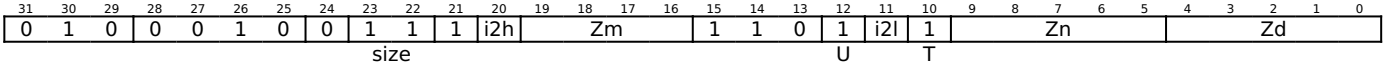
```
UMULLT <Zd>.S, <Zn>.H, <Zm>.H[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let index : integer = UInt(i3h::i3l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 1;
```

64-bit

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMULLT <Zd>.D, <Zn>.S, <Zm>.S[<imm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let index : integer = UInt(i2h::i2l);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel : integer = 1;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	For the "32-bit" variant: is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field. For the "64-bit" variant: is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<imm>	For the "32-bit" variant: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "64-bit" variant: is the element index, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let eltspersegment : integer = 128 DIV (2 * esize);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let s : integer = e - (e MOD eltspersegment);
    let element1 : integer = UInt(operand1[(2 * e + sel)*:esize]);
    let element2 : integer = UInt(operand2[(2 * s + index)*:esize]);
    let res : integer = element1 * element2;
    result[e*:(2*esize)] = res[2*esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

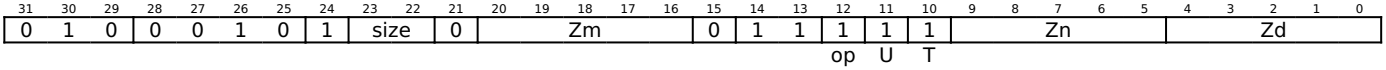
UMULLT (vectors)

Unsigned multiply long (top)

This instruction multiplies the odd-numbered unsigned elements of the first source vector by the corresponding elements of the second source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UMULLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[(2*e + 1)*:(esize DIV 2)]);
    let element2 : integer = UInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    let res : integer = element1 * element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQADD (immediate)

Unsigned saturating add immediate (unpredicated)

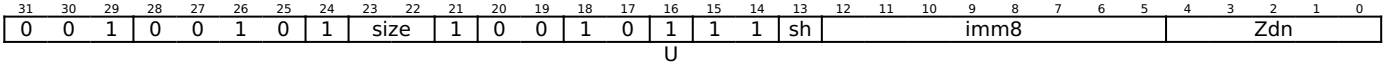
This instruction performs an unsigned saturating addition of an unsigned immediate to each element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQADD <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    (result[e*:esize], -) = SatQ{esize}(element1 + imm, unsigned);

end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

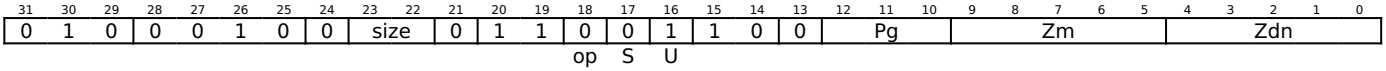
UQADD (vectors, predicated)

Unsigned saturating add (predicated)

This instruction adds Active unsigned elements of the first source vector to corresponding unsigned elements of the second source vector, and destructively places the results in the corresponding elements of the first source vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = UnsignedSat{esize}(element1 + element2);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

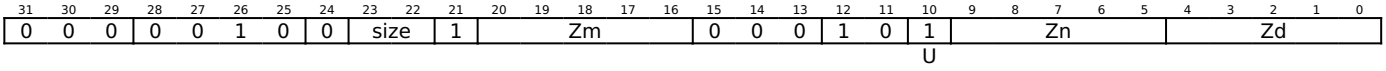
UQADD (vectors, unpredicated)

Unsigned saturating add (unpredicated)

This instruction performs an unsigned saturating addition of all elements of the first source vector to the corresponding elements of the second source vector, and places the results in the corresponding elements of the destination vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQADD <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let op2elt : bits(esize) = operand2[e*:esize];
    if unsigned then
        let element1 : integer = UInt(op1elt);
        let element2 : integer = UInt(op2elt);
        (result[e*:esize], -) = UnsignedSatQ{esize}(element1 + element2);
    else
        let element1 : integer = SInt(op1elt);
        let element2 : integer = SInt(op2elt);
        (result[e*:esize], -) = SignedSatQ{esize}(element1 + element2);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQCVTN

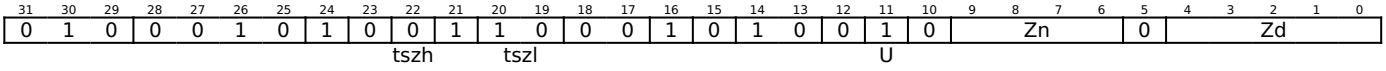
Unsigned 32-bit integer saturating extract narrow to interleaved 16-bit integer

This instruction saturates the unsigned integer value in each element of the pair of source vectors to half the original source element width, and places the two-way interleaved results in the half-width destination elements.

This instruction is unpredicated.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
UQCVTN <Zd>.H, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 1 do
        let operand : bits(VL) = Z{}(n+i);
        let element : integer = UInt(operand[e*:(2 * esize)]);
        result[(2*e + i)*:esize] = UnsignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQDECB

Unsigned saturating decrement scalar by multiple of 8-bit predicate constraint element count

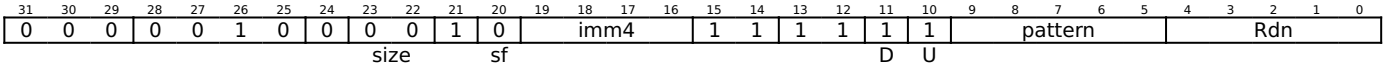
This instruction determines the number of Active 8-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement the scalar destination. The result is saturated to the general-purpose register's unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

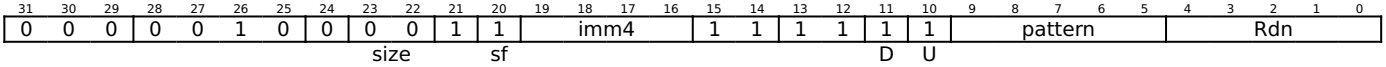
```
UQDECB <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UQDECB <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) - (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) - (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQDECD (scalar)

Unsigned saturating decrement scalar by multiple of 64-bit predicate constraint element count

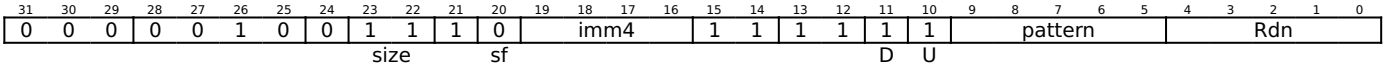
This instruction determines the number of Active 64-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement the scalar destination. The result is saturated to the general-purpose register's unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

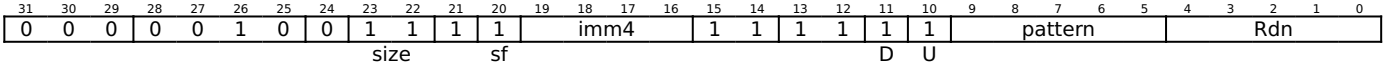
```
UQDECD <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UQDECD <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) - (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) - (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQDECD (vector)

Unsigned saturating decrement vector by multiple of 64-bit predicate constraint element count

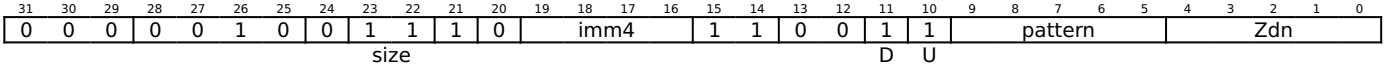
This instruction determines the number of Active 64-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement all destination vector elements. The results are saturated to the 64-bit unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQDECD <Zdn>.D{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) - (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) - (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQDECH (scalar)

Unsigned saturating decrement scalar by multiple of 16-bit predicate constraint element count

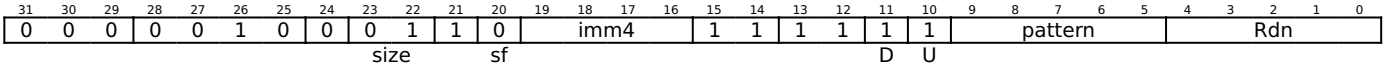
This instruction determines the number of Active 16-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement the scalar destination. The result is saturated to the general-purpose register's unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

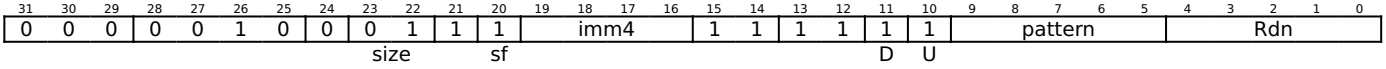
```
UQDECH <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UQDECH <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) - (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) - (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQDECH (vector)

Unsigned saturating decrement vector by multiple of 16-bit predicate constraint element count

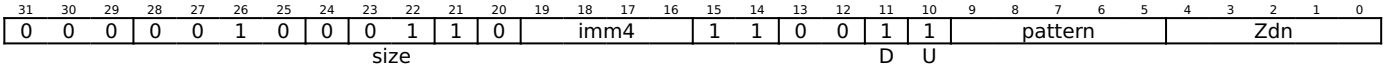
This instruction determines the number of Active 16-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement all destination vector elements. The results are saturated to the 16-bit unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQDECH <Zdn>.H{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) - (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) - (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQDECP (scalar)

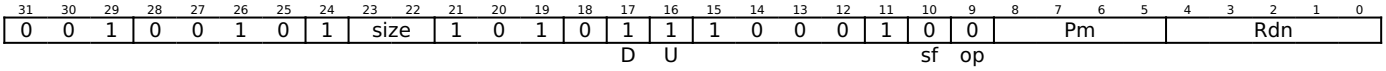
Unsigned saturating decrement scalar by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to decrement the scalar destination. The result is saturated to the general-purpose register's unsigned integer range.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

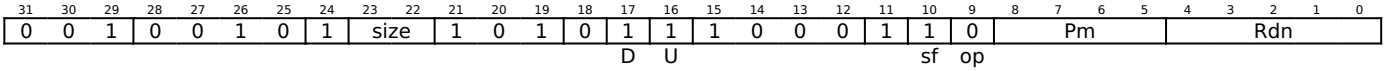
```
UQDECP <Wdn>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Rdn);
let unsigned : boolean = TRUE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UQDECP <Xdn>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Rdn);
let unsigned : boolean = TRUE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Wdn> Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(ssize) = X{}(dn);
let operand2 : bits(PL) = P{}(m);
var result : bits(ssize);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) - count);
    X{64}(dn) = ZeroExtend{64}(result);
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) - count);
    X{64}(dn) = SignExtend{64}(result);
end;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQDECP (vector)

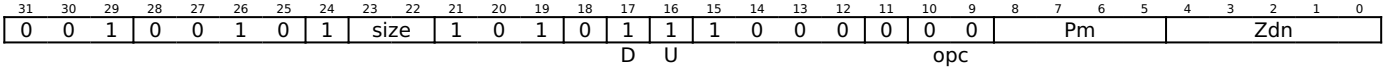
Unsigned saturating decrement vector by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to decrement all destination vector elements. The results are saturated to the element unsigned integer range.

The predicate size specifier may be omitted in assembler source code, but this is deprecated and will be prohibited in a future release of the architecture.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQDECP <Zdn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Zdn);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(PL) = P{}(m);
var result : bits(VL);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(opelt) - count);
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(opelt) - count);
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQDECW (scalar)

Unsigned saturating decrement scalar by multiple of 32-bit predicate constraint element count

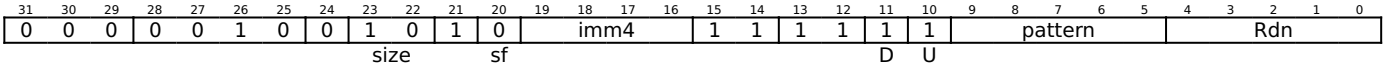
This instruction determines the number of Active 32-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement the scalar destination. The result is saturated to the general-purpose register's unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

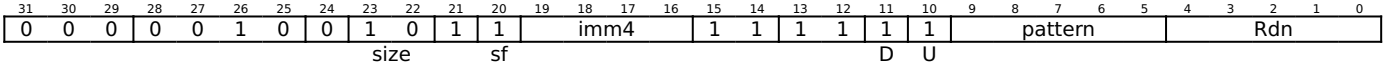
```
UQDECW <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UQDECW <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) - (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) - (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQDECW (vector)

Unsigned saturating decrement vector by multiple of 32-bit predicate constraint element count

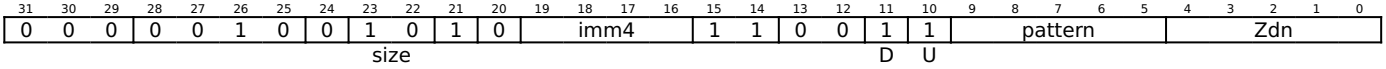
This instruction determines the number of Active 32-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to decrement all destination vector elements. The results are saturated to the 32-bit unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQDECW <Zdn>.S{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) - (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) - (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQINCB

Unsigned saturating increment scalar by multiple of 8-bit predicate constraint element count

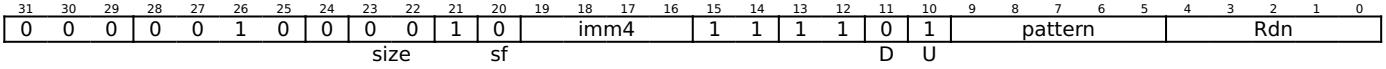
This instruction determines the number of Active 8-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment the scalar destination. The result is saturated to the general-purpose register's unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

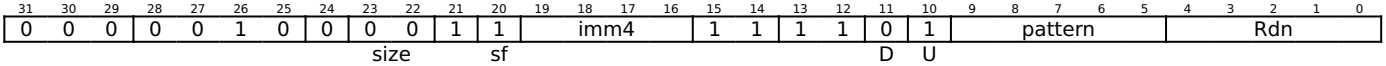
```
UQINCB <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UQINCB <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) + (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) + (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQINCD (scalar)

Unsigned saturating increment scalar by multiple of 64-bit predicate constraint element count

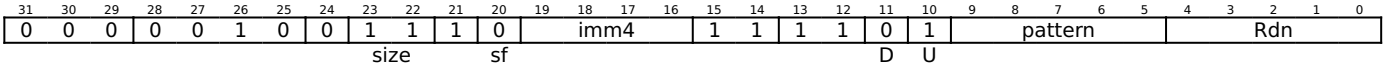
This instruction determines the number of Active 64-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment the scalar destination. The result is saturated to the general-purpose register's unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

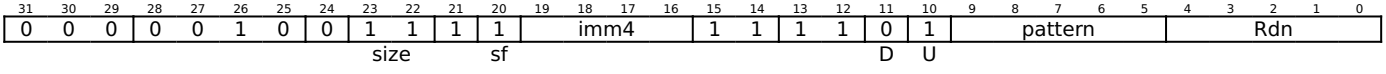
```
UQINCD <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UQINCD <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) + (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) + (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQINCD (vector)

Unsigned saturating increment vector by multiple of 64-bit predicate constraint element count

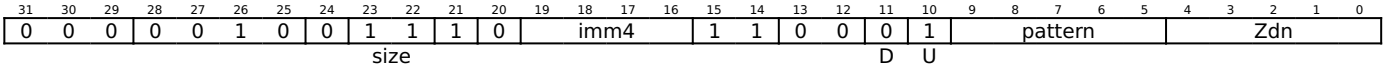
This instruction determines the number of Active 64-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment all destination vector elements. The results are saturated to the 64-bit unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQINCD <Zdn>.D{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) + (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) + (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQINCH (scalar)

Unsigned saturating increment scalar by multiple of 16-bit predicate constraint element count

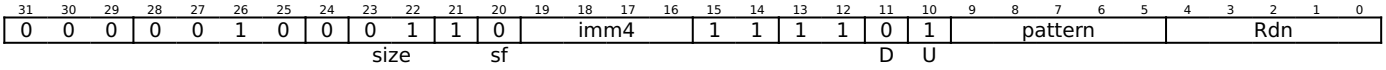
This instruction determines the number of Active 16-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment the scalar destination. The result is saturated to the general-purpose register's unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

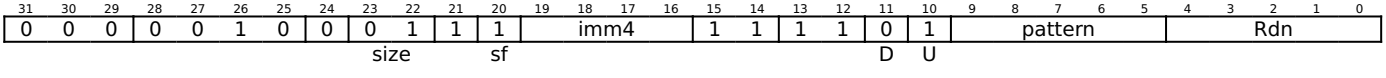
```
UQINCH <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UQINCH <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) + (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) + (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQINCH (vector)

Unsigned saturating increment vector by multiple of 16-bit predicate constraint element count

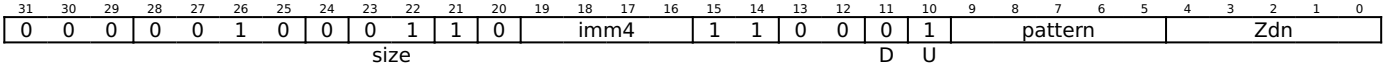
This instruction determines the number of Active 16-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment all destination vector elements. The results are saturated to the 16-bit unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQINCH <Zdn>.H{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) + (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) + (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQINCP (scalar)

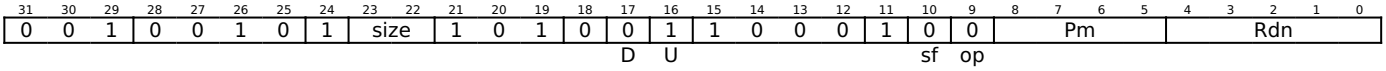
Unsigned saturating increment scalar by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to increment the scalar destination. The result is saturated to the general-purpose register's unsigned integer range.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

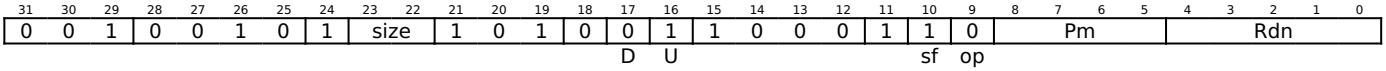
```
UQINCP <Wdn>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Rdn);
let unsigned : boolean = TRUE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UQINCP <Xdn>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Rdn);
let unsigned : boolean = TRUE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Wdn> Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(ssize) = X{}(dn);
let operand2 : bits(PL) = P{}(m);
var result : bits(ssize);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) + count);
    X{64}(dn) = ZeroExtend{64}(result);
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) + count);
    X{64}(dn) = SignExtend{64}(result);
end;
```

Operational information

If FEAT_SME is implemented and the PE is in Streaming SVE mode, then any subsequent instruction that is dependent on the general-purpose register written by this instruction might be significantly delayed.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQINCP (vector)

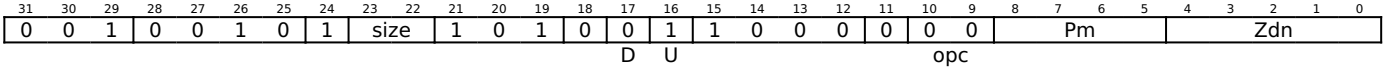
Unsigned saturating increment vector by count of true predicate elements

This instruction determines the number of true elements in the source predicate and then uses the result to increment all destination vector elements. The results are saturated to the element unsigned integer range.

The predicate size specifier may be omitted in assembler source code, but this is deprecated and will be prohibited in a future release of the architecture.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQINCP <Zdn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let m : integer = UInt(Pm);
let dn : integer = UInt(Zdn);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Pm> Is the name of the source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(PL) = P{}(m);
var result : bits(VL);
var count : integer = 0;

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(operand2, e, esize) then
        count = count + 1;
    end;
end;

for e = 0 to elements-1 do
    let opelt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(opelt) + count);
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(opelt) + count);
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQINCW (scalar)

Unsigned saturating increment scalar by multiple of 32-bit predicate constraint element count

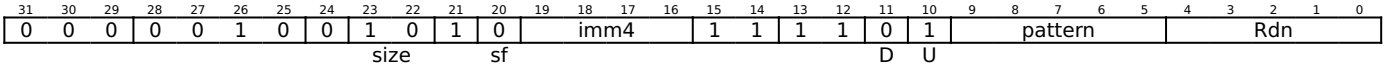
This instruction determines the number of Active 32-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment the scalar destination. The result is saturated to the general-purpose register's unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SVE || FEAT_SME)



Encoding

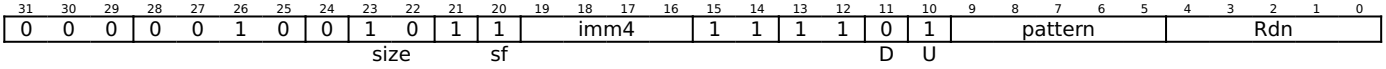
```
UQINCW <Wdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 32;
```

64-bit

(FEAT_SVE || FEAT_SME)



Encoding

```
UQINCW <Xdn>{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Rdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
let  ssize : integer{} = 64;
```

Assembler Symbols

<Wdn>	Is the 32-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

<Xdn> Is the 64-bit name of the source and destination general-purpose register, encoded in the "Rdn" field.

Operation

```
CheckSVEEnabled();
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(ssize) = X{}(dn);
var result : bits(ssize);
if unsigned then
    (result, -) = UnsignedSatQ{ssize}(UInt(operand1) + (count * imm));
else
    (result, -) = SignedSatQ{ssize}(SInt(operand1) + (count * imm));
end;
X{64}(dn) = Extend{64}(result, unsigned);
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQINCW (vector)

Unsigned saturating increment vector by multiple of 32-bit predicate constraint element count

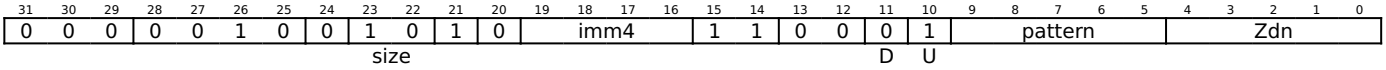
This instruction determines the number of Active 32-bit elements implied by the named predicate constraint, multiplies that by an immediate in the range 1 to 16 inclusive, and then uses the result to increment all destination vector elements. The results are saturated to the 32-bit unsigned integer range.
The named predicate constraint limits the number of Active elements in a single predicate to:

- A fixed number (VL1 to VL256)
- The largest power of two (POW2)
- The largest multiple of three or four (MUL3 or MUL4)
- All available, implicitly a multiple of two (ALL).

Unspecified or out of range constraint encodings generate an empty predicate or zero element count rather than Undefined Instruction exception.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQINCW <Zdn>.S{, <pattern>{, MUL #<imm>}}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32;
let dn : integer = UInt(Zdn);
let pat : bits(5) = pattern;
let imm : integer = UInt(imm4) + 1;

let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<pattern> Is the optional pattern specifier, defaulting to ALL, encoded in “pattern”:

pattern	<pattern>
00000	POW2
00001	VL1
00010	VL2
00011	VL3
00100	VL4
00101	VL5
00110	VL6
00111	VL7
01000	VL8
01001	VL16
01010	VL32
01011	VL64
01100	VL128
01101	VL256
0111x	#uimm5
1xx00	#uimm5
1x0x1	#uimm5
1x010	#uimm5
101x1	#uimm5
10110	#uimm5
11101	MUL4
11110	MUL3
11111	ALL

<imm> Is the immediate multiplier, in the range 1 to 16, defaulting to 1, encoded in the "imm4" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let count : integer = DecodePredCount(pat, esize);
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    if unsigned then
        (result[e*:esize], -) = UnsignedSatQ{esize}(UInt(op1elt) + (count * imm));
    else
        (result[e*:esize], -) = SignedSatQ{esize}(SInt(op1elt) + (count * imm));
    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

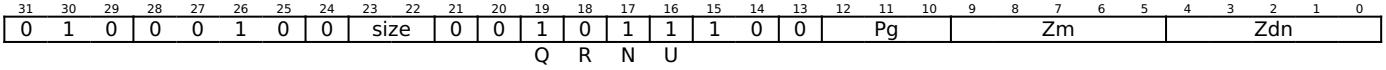
UQRSHL

Unsigned saturating rounding shift left (predicated)

This instruction shifts Active unsigned elements of the first source vector by corresponding elements of the second source vector and destructively places the rounded results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQRSHL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = UInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + (1 << (shift - 1))) >> shift;
        end;
        result[e*:esize] = UnsignedSat{esize}(res);
    else
        result[e*:esize] = operand1[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

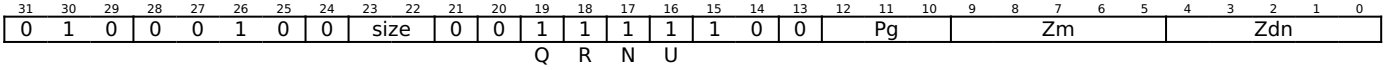
UQRSHLR

Unsigned saturating rounding shift left reversed (predicated)

This instruction shifts Active unsigned elements of the second source vector by corresponding elements of the first source vector and destructively places the rounded results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQRSHLR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let operand2 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = UInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + (1 << (shift - 1))) >> shift;
        end;
        result[e*:esize] = UnsignedSat{esize}(res);
    else
        result[e*:esize] = operand2[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQRSHRN

Unsigned saturating rounding shift right narrow by immediate to interleaved integer

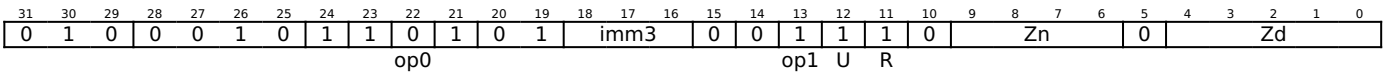
This instruction shifts right by an immediate value the unsigned integer value in each element of the two source vectors, and places the two-way interleaved rounded results in the half-width destination elements. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per destination element.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit and 16-bit

8-bit

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

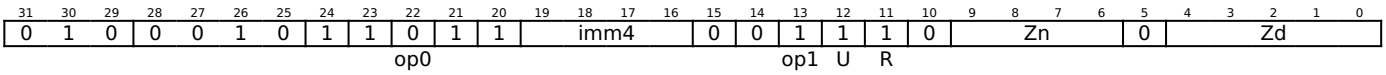
```
UQRSHRN <Zd>.B, { <Zn1>.H-<Zn2>.H }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = esize - UInt(imm3);
```

16-bit

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
UQRSHRN <Zd>.H, { <Zn1>.S-<Zn2>.S }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = esize - UInt(imm4);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<const>	For the "8-bit" variant: is the immediate shift amount, in the range 1 to 8, encoded as 8 minus "imm3". For the "16-bit" variant: is the immediate shift amount, in the range 1 to 16, encoded as 16 minus "imm4".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 1 do
        let operand : bits(VL) = Z{}(n+i);
        let element : bits(2 * esize) = operand[e*:(2 * esize)];
        let res : integer = (UInt(element) + (1 << (shift-1))) >> shift;
        result[(2*e + i)*:esize] = UnsignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

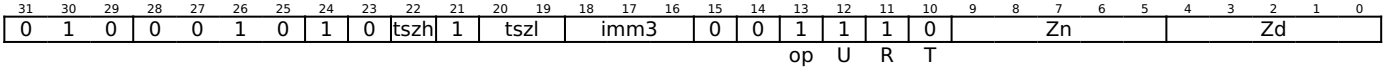
UQRSHRNB

Unsigned saturating rounding shift right narrow by immediate (bottom)

This instruction shifts each unsigned integer value in the source vector elements right by an immediate value, and places the rounded results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQRSHRNB <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = (UInt(element) + (1 << (shift-1))) >> shift;
    result[(2*e + 0)*:esize] = UnsignedSat{esize}(res);
    result[(2*e + 1)*:esize] = Zeros{esize};

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

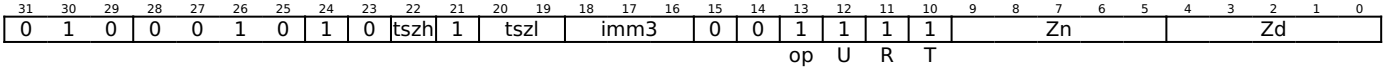
UQRSHRNT

Unsigned saturating rounding shift right narrow by immediate (top)

This instruction shifts each unsigned integer value in the source vector elements by an immediate value, and places the rounded results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQRSHRNT <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = (UInt(element) + (1 << (shift-1))) >> shift;
    result[(2*e + 1)*:esize] = UnsignedSat{esize}(res);

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

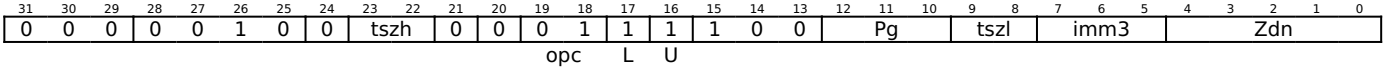
UQSHL (immediate)

Unsigned saturating shift left by immediate

This instruction shifts left by immediate each Active unsigned element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 0 to number of bits per element minus 1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQSHL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let shift : integer = UInt(tsize::imm3) - esize;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the immediate shift amount, in the range 0 to number of bits per element minus 1, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let mask : bits(PL) = P{}(g);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = element1 << shift;
        result[e*:esize] = UnsignedSat{esize}(res);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

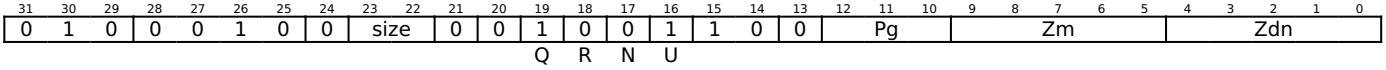
UQSHL (vectors)

Unsigned saturating shift left (predicated)

This instruction shifts Active unsigned elements of the first source vector by corresponding elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQSHL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = UInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = element >> shift;
        end;
        result[e*:esize] = UnsignedSat{esize}(res);
    else
        result[e*:esize] = operand1[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

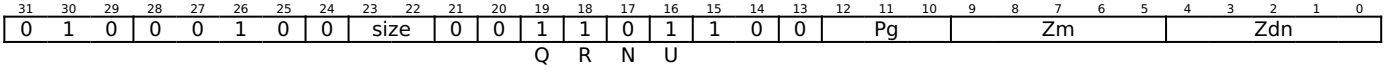
UQSHLR

Unsigned saturating shift left reversed (predicated)

This instruction shifts Active unsigned elements of the second source vector by corresponding elements of the first source vector and destructively places the results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQSHLR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let operand2 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = UInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = element >> shift;
        end;
        result[e*:esize] = UnsignedSat{esize}(res);
    else
        result[e*:esize] = operand2[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

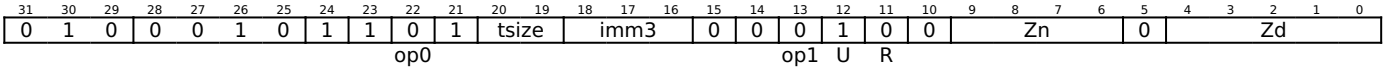
UQSHRN

Unsigned saturating shift right narrow by immediate to interleaved integer

This instruction shifts right by an immediate value the unsigned integer value in each element of the two source vectors, and places the two-way interleaved truncated results in the half-width destination elements. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per destination element.
This instruction is unpredicated.

SVE2

(FEAT_SVE2p3 || FEAT_SME2p3)



Encoding

```
UQSHRN <Zd>.<T>, { <Zn1>.<Tb>-<Zn2>.<Tb> }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p3) && !IsFeatureImplemented(FEAT_SME2p3) then
    EndOfDecode(Decode_UNDEF);
end;
if tsize == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsize”:

tsize	<T>
00	RESERVED
01	B
1x	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.

<Tb> Is the size specifier, encoded in “tsize”:

tsize	<Tb>
00	RESERVED
01	H
1x	S

<Zn2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in the "tsize:imm3" fields.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 1 do
        let operand : bits(VL) = Z{}(n+i);
        let element : bits(2*esize) = operand[e*:(2*esize)];
        let res : integer = UInt(element) >> shift;
        result[(2*e + i)*:esize] = UnsignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

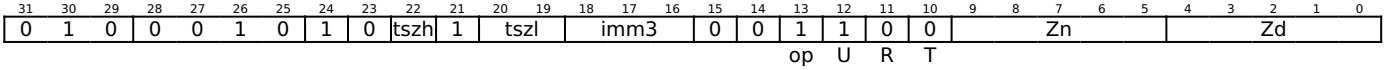
UQSHRNB

Unsigned saturating shift right narrow by immediate (bottom)

This instruction shifts each unsigned integer value in the source vector elements right by an immediate value, and places the truncated results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQSHRNB <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = UInt(element) >> shift;
    result[(2*e + 0)*:esize] = UnsignedSat{esize}(res);
    result[(2*e + 1)*:esize] = Zeros{esize};

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

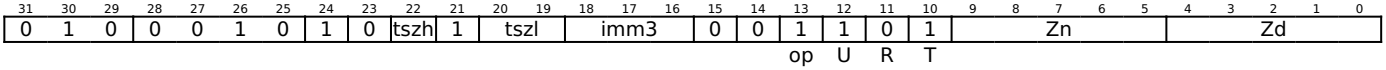
UQSHRNT

Unsigned saturating shift right narrow by immediate (top)

This instruction shifts each unsigned integer value in the source vector elements right by an immediate value, and places the truncated results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQSHRNT <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
for e = 0 to elements-1 do
    let element : bits(2*esize) = operand[e*:(2*esize)];
    let res : integer = UInt(element) >> shift;
    result[(2*e + 1)*:esize] = UnsignedSat{esize}(res);

end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQSUB (immediate)

Unsigned saturating subtract immediate (unpredicated)

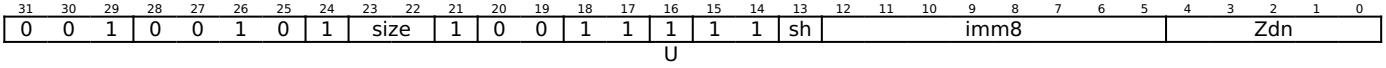
This instruction performs an unsigned saturating subtraction of an unsigned immediate from each element of the source vector, and destructively places the results in the corresponding elements of the source vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. This instruction is unpredicated.

The immediate is an unsigned value in the range 0 to 255, and for element widths of 16 bits or higher it may also be a positive multiple of 256 in the range 256 to 65280.

The immediate is encoded in 8 bits with an optional left shift by 8. The preferred disassembly when the shift option is specified is "#<imm>, LSL #8". However, an assembler and disassembler may also allow use of the shifted 16-bit value unless the immediate is 0 and the shift amount is 8, which must be unambiguously described as "#0, LSL #8".

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQSUB <Zdn>.<T>, <Zdn>.<T>, #<imm>{, <shift>}
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size::sh == '001' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn);
var imm : integer = UInt(imm8);
if sh == '1' then imm = imm << 8; end;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<imm> Is an unsigned immediate in the range 0 to 255, encoded in the "imm8" field.

<shift> Is the optional left shift to apply to the immediate, defaulting to LSL #0 and encoded in “sh”:

sh	<shift>
0	LSL #0
1	LSL #8

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    (result[e*:esize], -) = SatQ{esize}(element1 - imm, unsigned);

end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

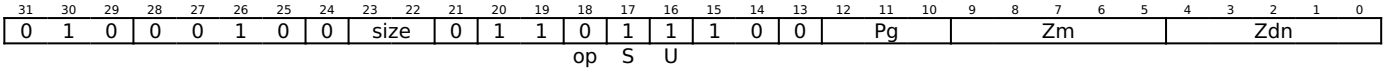
UQSUB (vectors, predicated)

Unsigned saturating subtract (predicated)

This instruction subtracts Active unsigned elements of the second source vector from corresponding unsigned elements of the first source vector and destructively places the results in the corresponding elements of the first source vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQSUB <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = UnsignedSat{esize}(element1 - element2);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

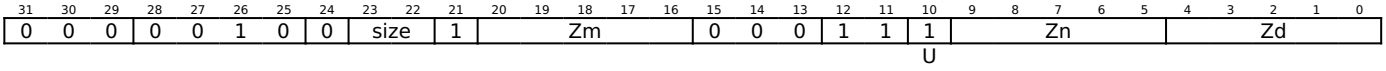
UQSUB (vectors, unpredicated)

Unsigned saturating subtract (unpredicated)

This instruction performs an unsigned saturating subtraction of all elements of the second source vector from the corresponding elements of the first source vector, and places the results in the corresponding elements of the destination vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. This instruction is unpredicated.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
UQSUB <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize) = operand1[e*:esize];
    let op2elt : bits(esize) = operand2[e*:esize];
    if unsigned then
        let element1 : integer = UInt(op1elt);
        let element2 : integer = UInt(op2elt);
        (result[e*:esize], -) = UnsignedSatQ{esize}(element1 - element2);
    else
        let element1 : integer = SInt(op1elt);
        let element2 : integer = SInt(op2elt);
        (result[e*:esize], -) = SignedSatQ{esize}(element1 - element2);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

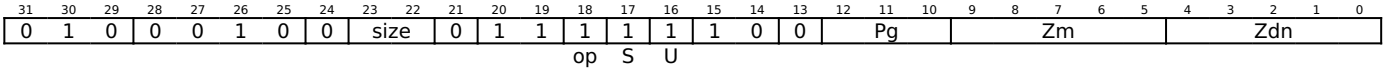
UQSUBR

Unsigned saturating subtract reversed (predicated)

This instruction subtracts Active unsigned elements of the first source vector from corresponding unsigned elements of the second source vector and destructively places the results in the corresponding elements of the first source vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQSUBR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = UnsignedSat{esize}(element2 - element1);
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

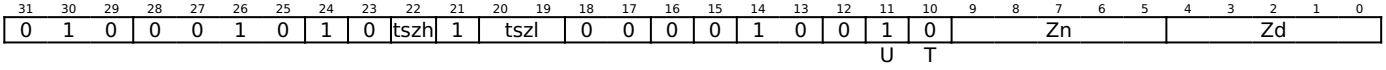
UQXTNB

Unsigned saturating extract narrow (bottom)

This instruction saturates the unsigned integer value in each source element to half the original source element width, and places the results in the even-numbered half-width destination elements, while setting the odd-numbered elements to zero.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQXTNB <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if !(tsize IN {'001', '010', '100'}) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	10	H
0	11	RESERVED
1	00	S
1	01	RESERVED
1	1x	RESERVED

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	10	S
0	11	RESERVED
1	00	D
1	01	RESERVED
1	1x	RESERVED

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
var result : bits(VL);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let res : bits(halfesize) = UnsignedSat{halfesize}(element1);
    result[(2*e + 0)*:halfesize] = res;
    result[(2*e + 1)*:halfesize] = Zeros{halfesize};

end;
Z{VL}(d) = result;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

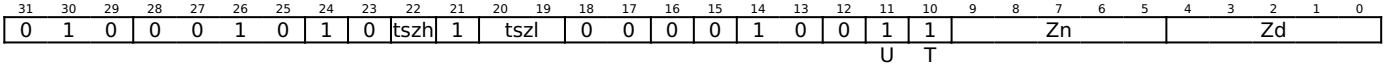
UQXTNT

Unsigned saturating extract narrow (top)

This instruction saturates the unsigned integer value in each source element to half the original source element width, and places the results in the odd-numbered half-width destination elements, leaving the even-numbered elements unchanged.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
UQXTNT <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if !(tsize IN {'001', '010', '100'}) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	B
0	10	H
0	11	RESERVED
1	00	S
1	01	RESERVED
1	1x	RESERVED

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	H
0	10	S
0	11	RESERVED
1	00	D
1	01	RESERVED
1	1x	RESERVED

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
var result : bits(VL) = Z{}(d);
let halfesize : integer{} = esize DIV 2;

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let res : bits(halfesize) = UnsignedSat{halfesize}(element1);
    result[(2*e + 1)*:halfesize] = res;

end;
Z{VL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

URECPE

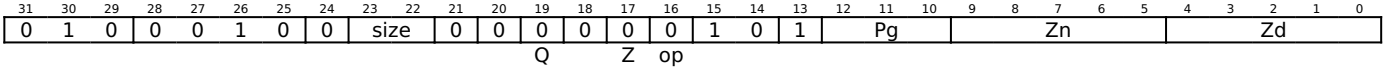
Unsigned reciprocal estimate (predicated)

This instruction finds the approximate reciprocal of each Active unsigned element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE2 || FEAT_SME)



Encoding

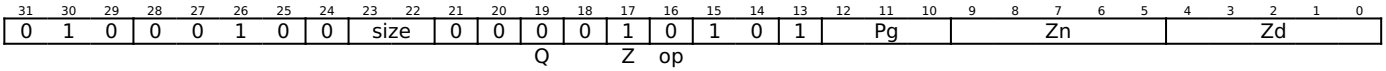
```
URECPE <Zd>.S, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
URECPE <Zd>.S, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = UnsignedRecipEstimate{esize}(element);

    end;
end;
Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:
The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

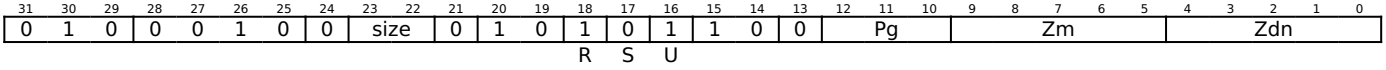
URHADD

Unsigned rounding halving add

This instruction adds Active unsigned elements of the first source vector to corresponding unsigned elements of the second source vector, shifts right one bit, and destructively places the rounded results in the corresponding elements of the first source vector. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
URHADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = (element1 + element2 + 1) >> 1;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.

• The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

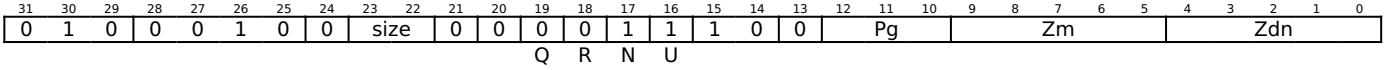
URSHL

Unsigned rounding shift left (predicated)

This instruction shifts Active unsigned elements of the first source vector by corresponding elements of the second source vector and destructively places the rounded results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
URSHL <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = UInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + (1 << (shift - 1))) >> shift;
        end;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

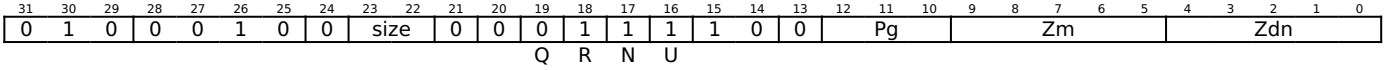
URSHLR

Unsigned rounding shift left reversed (predicated)

This instruction shifts Active unsigned elements of the second source vector by corresponding elements of the first source vector and destructively places the rounded results in the corresponding elements of the first source vector. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
URSHLR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
let operand2 : bits(VL) = Z{}(dn);
var result : bits(VL);

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : integer = UInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + (1 << (shift - 1))) >> shift;
        end;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand2[e*:esize];

    end;
end;
Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

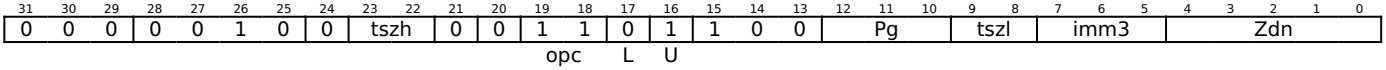
URSHR

Unsigned rounding shift right by immediate

This instruction shifts right by immediate each Active unsigned element of the source vector, and destructively places the rounded results in the corresponding elements of the source vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
URSHR <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zdn> Is the name of the source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let mask : bits(PL) = P{}(g);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    if ActivePredicateElement{PL}(mask, e, esize) then
        let res : integer = (element1 + (1 << (shift - 1))) >> shift;
        result[e*:esize] = res[esize-1:0];
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.

- A predicated `MOVPRFX` must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The `MOVPRFX` must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

URSQRTE

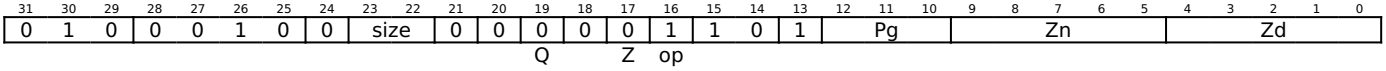
Unsigned reciprocal square root estimate (predicated)

This instruction finds the approximate reciprocal square root of each Active unsigned element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.

It has encodings from 2 classes: Merging and Zeroing

Merging

(FEAT_SVE2 || FEAT_SME)



Encoding

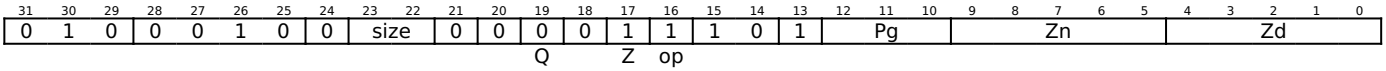
```
URSQRTE <Zd>.S, <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = TRUE;
```

Zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
URSQRTE <Zd>.S, <Pg>/Z, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '10' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Pg>	Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = UnsignedRSqrtEstimate{esize}(element);

    end;
end;
Z{VL}(d) = result;
```

Operational information

For the "Merging" variant:
The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

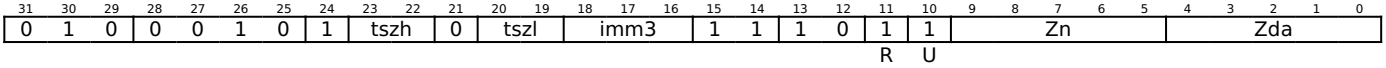
URSRA

Unsigned rounding shift right and accumulate (immediate)

This instruction shifts right by immediate each unsigned element of the source vector, inserting zeros, and destructively adds the rounded intermediate result to the corresponding elements of the addend vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
URSRA <Zda>.<T>, <Zn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let da : integer = UInt(Zda);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : integer = (UInt(operand1[e*:esize]) + (1 << (shift - 1))) >> shift;
    result[e*:esize] = operand2[e*:esize] + element[esize-1:0];
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USDOT (indexed)

Unsigned by signed 8-bit integer dot product by indexed element to 32-bit integer

This instruction calculates the dot product of a group of four unsigned 8-bit integer values held in each 32-bit element of the first source vector multiplied by a group of four signed 8-bit integer values in an indexed 32-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 32-bit element of the destination vector.

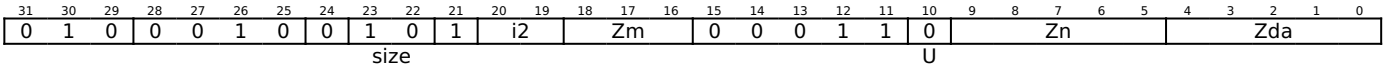
The groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment. The index range is from 0 to 3.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.I8MM indicates whether this instruction is implemented.

SVE

((FEAT_SVE || FEAT_SME) && FEAT_I8MM)



Encoding

```
USDOT <Zda>.S, <Zn>.B, <Zm>.B[<imm>]
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_I8MM)) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let index : integer = UInt(i2);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z7, encoded in the "Zm" field.
<imm>	Is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let eltspersegment : integer = 128 DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let segmentbase : integer = e - (e MOD eltspersegment);
    let s : integer = segmentbase + index;
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 3 do
        let element1 : integer = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        let element2 : integer = SInt(operand2[(4 * s + i)*:(esize DIV 4)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USDOT (vectors)

Unsigned by signed 8-bit integer dot product to 32-bit integer

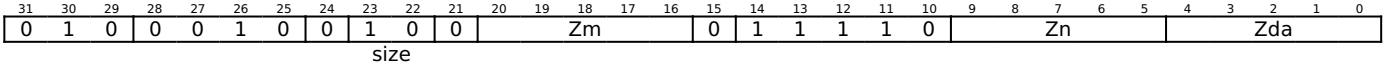
This instruction calculates the dot product of a group of four unsigned 8-bit integer values held in each 32-bit element of the first source vector multiplied by a group of four signed 8-bit integer values in the corresponding 32-bit element of the second source vector, and then destructively adds the widened dot product to the corresponding 32-bit element of the destination vector.

This instruction is unpredicated.

ID_AA64ZFR0_EL1.I8MM indicates whether this instruction is implemented.

SVE

((FEAT_SVE || FEAT_SME) && FEAT_I8MM)



Encoding

```
USDOT <Zda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if ((!IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME)) ||
    !IsFeatureImplemented(FEAT_I8MM)) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    var res : bits(esize) = operand3[e*:esize];
    for i = 0 to 3 do
        let element1 : integer = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
        let element2 : integer = SInt(operand2[(4 * e + i)*:(esize DIV 4)]);
        res = res + element1 * element2;
    end;
    result[e*:esize] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

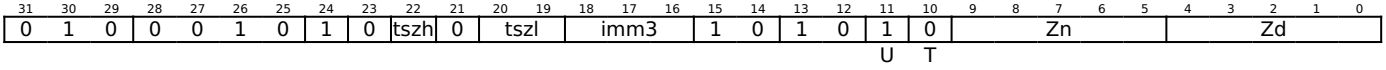
USHLLB

Unsigned shift left long by immediate (bottom)

This instruction shifts left by immediate each even-numbered unsigned element of the source vector, and places the results in the overlapping double-width elements of the destination vector. The immediate shift amount is an unsigned value in the range 0 to number of bits per element minus 1. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
USHLLB <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = UInt(tsize::imm3) - esize;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<const> Is the immediate shift amount, in the range 0 to number of bits per element minus 1, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[(2*e + 0)*:esize];
    let shifted_value : integer = UInt(element) << shift;
    result[e*:(2*esize)] = shifted_value[2*esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

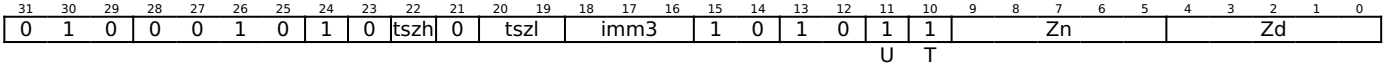
USHLLT

Unsigned shift left long by immediate (top)

This instruction shifts left by immediate each odd-numbered unsigned element of the source vector, and places the results in the overlapping double-width elements of the destination vector. The immediate shift amount is an unsigned value in the range 0 to number of bits per element minus 1. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
USHLLT <Zd>.<T>, <Zn>.<Tb>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(3) = tszh::tszl;
if tsize == '000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let shift : integer = UInt(tsize::imm3) - esize;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
0	00	RESERVED
0	01	H
0	1x	S
1	xx	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<Tb>
0	00	RESERVED
0	01	B
0	1x	H
1	xx	S

<const> Is the immediate shift amount, in the range 0 to number of bits per element minus 1, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(esize) = operand[(2*e + 1)*:esize];
    let shifted_value : integer = UInt(element) << shift;
    result[e*:(2*esize)] = shifted_value[2*esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USMMLA

Unsigned by signed 8-bit integer matrix multiply-accumulate to 32-bit integer

This instruction multiplies the 2×8 matrix of unsigned 8-bit integer values held in each 128-bit segment of the first source vector by the 8×2 matrix of signed 8-bit integer values in the corresponding segment of the second source vector. The resulting 2×2 widened 32-bit integer matrix product is then destructively added to the 32-bit integer matrix accumulator held in the corresponding segment of the addend and destination vector. This is equivalent to performing an 8-way dot product per destination element.

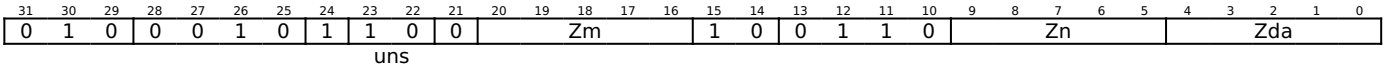
This instruction is unpredicated.

ID_AA64ZFR0_EL1.I8MM indicates whether this instruction is implemented.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE && FEAT_I8MM)



Encoding

```
USMMLA <Zda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) || !IsFeatureImplemented(FEAT_I8MM) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(Zda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<Zda>	Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckNonStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
let operand3 : bits(VL) = Z{}(da);
var result : bits(VL) = Zeros{};
var op1, op2 : bits(128);
var res, addend : bits(128);

for s = 0 to segments-1 do
    op1    = operand1[s*:(128)];
    op2    = operand2[s*:(128)];
    addend = operand3[s*:(128)];
    res    = MatMulAdd(addend, op1, op2, op1_unsigned, op2_unsigned);
    result[s*:(128)] = res;
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

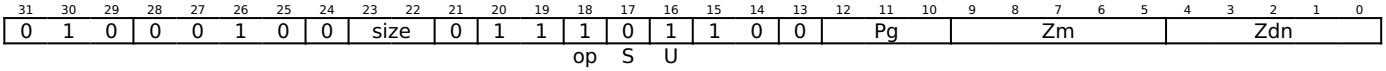
USQADD

Unsigned saturating signed add

This instruction adds Active signed elements of the source vector to the corresponding unsigned elements of the addend vector, and destructively places the results in the corresponding elements of the addend vector. Each result element is saturated to the N-bit element's unsigned integer range 0 to (2^N)-1. Inactive elements in the destination vector register remain unmodified.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
USQADD <Zdn>.<T>, <Pg>/M, <Zdn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let g : integer = UInt(Pg);
let dn : integer = UInt(Zdn);
let m : integer = UInt(Zm);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(m) else Zeros{VL};
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = UnsignedSat{esize}(UInt(element1) + SInt(element2));
    else
        result[e*:esize] = operand1[e*:esize];
    end;
end;

Z{VL}(dn) = result;
```

Operational information

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of this instruction.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

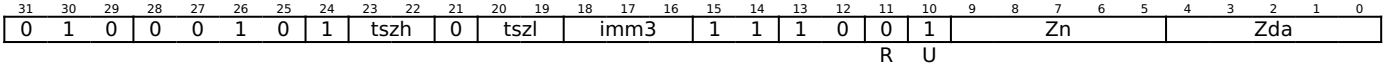
USRA

Unsigned shift right and accumulate (immediate)

This instruction shifts right by immediate each unsigned element of the source vector, inserting zeros, and destructively adds the truncated intermediate result to the corresponding elements of the addend vector. The immediate shift amount is an unsigned value in the range 1 to number of bits per element. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
USRA <Zda>.<T>, <Zn>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn);
let da : integer = UInt(Zda);
let shift : integer = (2 * esize) - UInt(tsize::imm3);
```

Assembler Symbols

<Zda> Is the name of the third source and destination scalable vector register, encoded in the "Zda" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(da);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : integer = UInt(operand1[e*:esize]) >> shift;
    result[e*:esize] = operand2[e*:esize] + element[esize-1:0];
end;

Z{VL}(da) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.
This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

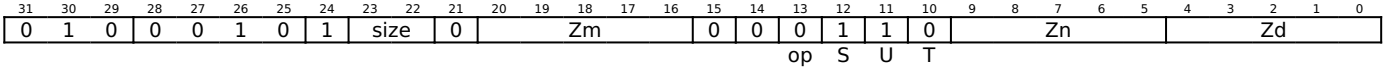
USUBLB

Unsigned subtract long (bottom)

This instruction subtracts the even-numbered unsigned elements of the second source vector from the corresponding unsigned elements of the first source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
USUBLB <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 0;
let sel2 : integer = 0;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 - element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

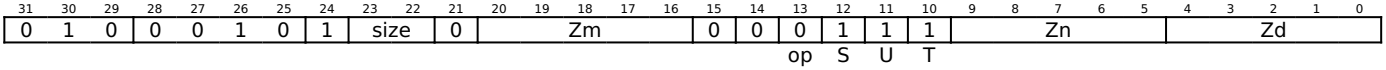
USUBLT

Unsigned subtract long (top)

This instruction subtracts the odd-numbered unsigned elements of the second source vector from the corresponding unsigned elements of the first source vector, and places the results in the overlapping double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
USUBLT <Zd>.<T>, <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let sel1 : integer = 1;
let sel2 : integer = 1;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let op1elt : bits(esize DIV 2) = operand1[(2*e + sel1)*:(esize DIV 2)];
    let op2elt : bits(esize DIV 2) = operand2[(2*e + sel2)*:(esize DIV 2)];
    let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
    let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
    let res : integer = element1 - element2;
    result[e*:esize] = res[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

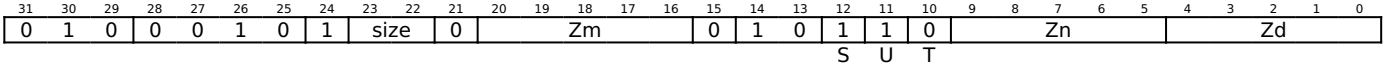
USUBWB

Unsigned subtract wide (bottom)

This instruction subtracts the even-numbered unsigned elements of the second source vector from the overlapping double-width elements of the first source vector and places the results in the corresponding double-width elements of the destination vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
USUBWB <Zd>.<T>, <Zn>.<T>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[(2*e + 0)*:(esize DIV 2)]);
    result[e*:esize] = (element1 - element2)[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

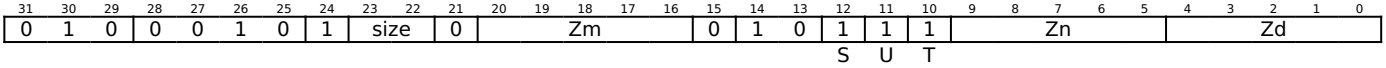
USUBWT

Unsigned subtract wide (top)

This instruction subtracts the odd-numbered unsigned elements of the second source vector from the overlapping double-width elements of the first source vector and places the results in the corresponding double-width elements of the destination vector. This instruction is unpredicated. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
USUBWT <Zd>.<T>, <Zn>.<T>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : integer = UInt(operand1[e*:esize]);
    let element2 : integer = UInt(operand2[(2*e + 1)*:(esize DIV 2)]);
    result[e*:esize] = (element1 - element2)[esize-1:0];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UUNPKHI, UUNPKLO

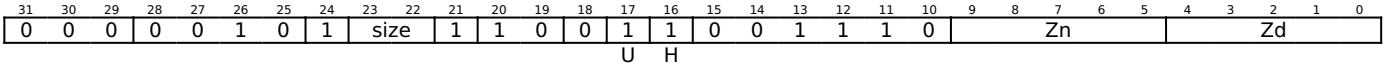
Unsigned unpack and extend half of vector

This instruction unpacks and zero-extends elements from the lowest or highest half of the source vector, and writes the zero-extended values into double-width elements of the destination vector. This instruction is unpredicated.

It has encodings from 2 classes: High half and Low half

High half

(FEAT_SVE || FEAT_SME)



Encoding

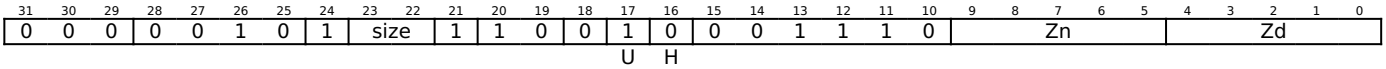
```
UUNPKHI <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let hi : boolean = TRUE;
```

Low half

(FEAT_SVE || FEAT_SME)



Encoding

```
UUNPKLO <Zd>.<T>, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let hi : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let hsize : integer{} = esize DIV 2;
let offset : integer = if hi then elements else 0;
let operand : bits(VL) = Z{}(n);
var result : bits(VL);

for e = 0 to elements-1 do
    let element : bits(hsize) = operand[(e + offset)*:hsize];
    result[e*:esize] = Extend{esize}(element, unsigned);
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

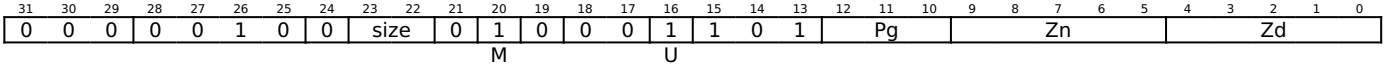
UXTB, UXTH, UXTW

Unsigned byte / halfword / word extend (predicated)

This instruction zero-extends the least-significant sub-element of each Active element of the source vector, and places the results in the corresponding elements of the destination vector. Inactive elements in the destination vector register remain unmodified or are set to zero, depending on whether merging or zeroing predication is selected.
It has encodings from 6 classes: Byte, merging , Byte, zeroing , Halfword, merging , Halfword, zeroing , Word, merging and Word, zeroing

Byte, merging

(FEAT_SVE || FEAT_SME)



Encoding

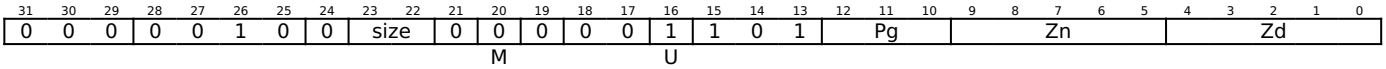
```
UXTB <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let merging : boolean = TRUE;
```

Byte, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

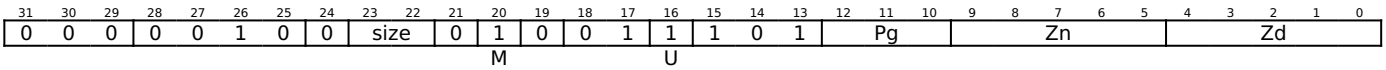
```
UXTB <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 8;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let merging : boolean = FALSE;
```

Halfword, merging

(FEAT_SVE || FEAT_SME)



Encoding

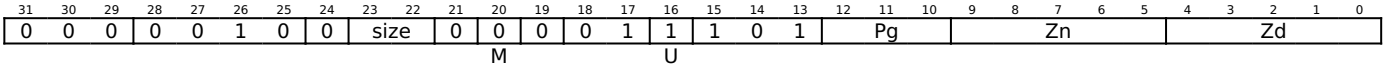
```
UXTH <Zd>.<T>, <Pg>/M, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let merging : boolean = TRUE;
```

Halfword, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

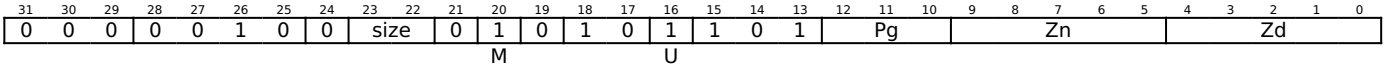
```
UXTH <Zd>.<T>, <Pg>/Z, <Zn>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size IN {'0x'} then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 16;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let merging : boolean = FALSE;
```

Word, merging

(FEAT_SVE || FEAT_SME)



Encoding

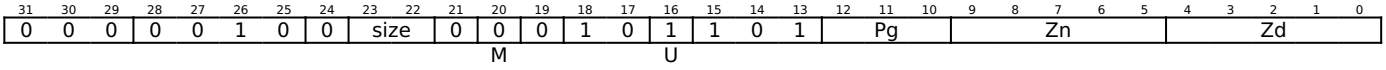
```
UXTW <Zd>.D, <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let merging : boolean = TRUE;
```

Word, zeroing

(FEAT_SVE2p2 || FEAT_SME2p2)



Encoding

```
UXTW <Zd>.D, <Pg>/Z, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p2) && !IsFeatureImplemented(FEAT_SME2p2) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let s_esize : integer{} = 32;
let g : integer = UInt(Pg);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let unsigned : boolean = TRUE;
let merging : boolean = FALSE;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> For the "Byte, merging" and "Byte, zeroing" variants: is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

For the "Halfword, merging" and "Halfword, zeroing" variants: is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	S
1	D

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = P{}(g);
let operand : bits(VL) = if AnyActiveElement{PL}(mask, esize) then Z{VL}(n) else Zeros{VL};
var result : bits(VL) = if merging then Z{VL}(d) else Zeros{VL};

for e = 0 to elements-1 do
    if ActivePredicateElement{PL}(mask, e, esize) then
        let element : bits(esize) = operand[e*:esize];
        result[e*:esize] = Extend{esize}(element[s_esize-1:0], unsigned);
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

The merging variant of this instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and the merging variant of this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX can be predicated or unpredicated.
- A predicated MOVPRFX must use the same governing predicate register as the merging variant of this instruction.
- A predicated MOVPRFX must use the larger of the destination element size and first source element size in the preferred disassembly of the merging variant of this instruction.
- The MOVPRFX must specify the same destination register as the merging variant of this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of the merging variant of this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UZP1, UZP2 (predicates)

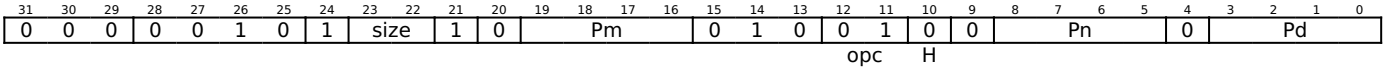
Concatenate even or odd elements from two predicates

This instruction concatenates adjacent even-numbered or odd-numbered elements from the first and second source predicates, and places the values in the destination predicate. This instruction is unpredicated.

It has encodings from 2 classes: Even and Odd

Even

(FEAT_SVE || FEAT_SME)



Encoding

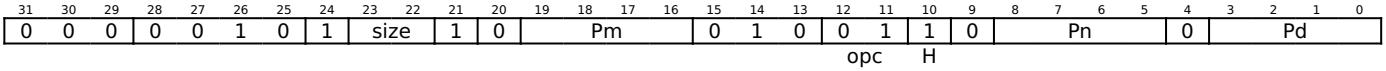
```
UZP1 <Pd>.<T>, <Pn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let part : integer = 0;
```

Odd

(FEAT_SVE || FEAT_SME)



Encoding

```
UZP2 <Pd>.<T>, <Pn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let part : integer = 1;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pn> Is the name of the first source scalable predicate register, encoded in the "Pn" field.

<Pm> Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let pairs : integer = VL DIV (esize * 2);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);

for p = 0 to pairs - 1 do
    result[p*:(esize DIV 8)] = operand1[(2*p+part)*:(esize DIV 8)];
end;

for p = 0 to pairs - 1 do
    result[(pairs+p)*:(esize DIV 8)] = operand2[(2*p+part)*:(esize DIV 8)];
end;

P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UZP1, UZP2 (vectors)

Concatenate even or odd elements from two vectors

This instruction concatenates adjacent even-numbered or odd-numbered elements from the first and second source vectors, and places the values in the destination vector. This instruction is unpredicated.

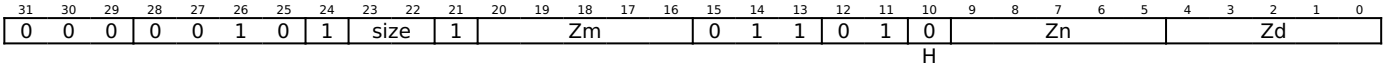
Note: UZP1 is equivalent to truncating and packing each element from two source vectors into a single destination vector with elements of half the size.

The Even (quadwords) and Odd (quadwords) variants require an Effective SVE vector length of at least 256 bits. ID_AA64ZFR0_EL1.F64MM or ID_AA64ZFR0_EL1.EltPerm indicates whether the Even (quadwords) and Odd (quadwords) variants are implemented. The Even (quadwords) and Odd (quadwords) variants are legal when executed in Streaming SVE mode if FEAT_SVE_PERM256 is implemented.

It has encodings from 4 classes: Even , Even (quadwords) , Odd and Odd (quadwords)

Even

(FEAT_SVE || FEAT_SME)



Encoding

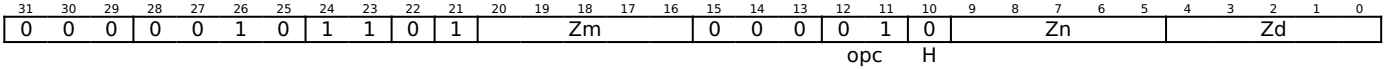
```
UZP1 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Even (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

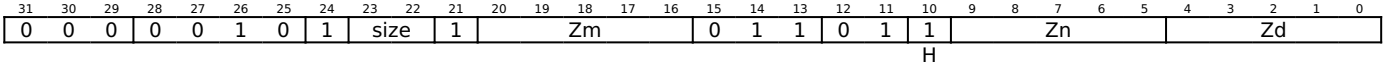
```
UZP1 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) then
    EndOfDecode(Decode_UNDEF);
end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Odd

(FEAT_SVE || FEAT_SME)



Encoding

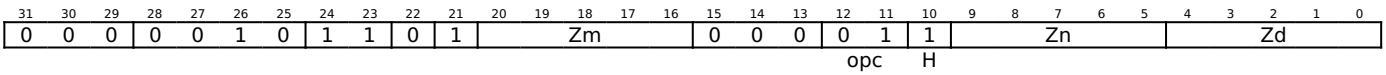
```
UZP2 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Odd (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

```
UZP2 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) then
    EndOfDecode(Decode_UNDEF);
end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if esize < 128 || IsFeatureImplemented(FEAT_SVE_PERM256) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
if VL < esize * 2 then EndOfDecode(Decode_UNDEF); end;
let pairs : integer = VL DIV (esize * 2);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Zeros{};

for p = 0 to pairs - 1 do
    result[p*:esize] = operand1[(2*p+part)*:esize];
end;

for p = 0 to pairs - 1 do
    result[(pairs+p)*:esize] = operand2[(2*p+part)*:esize];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

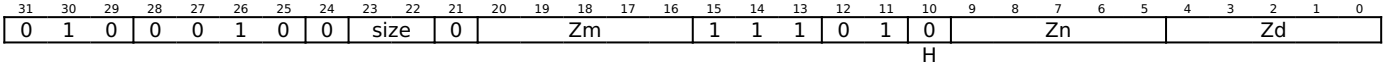
UZPQ1

Concatenate even elements within each pair of quadword vector segments

This instruction concatenates adjacent even-numbered elements from the corresponding 128-bit vector segments of the first and second source vectors, and places them in the corresponding elements of the destination vector segment. This instruction is unpredicated.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
UZPQ1 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let elements : integer = 128 DIV esize;
let pairs : integer = elements DIV 2;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for s = 0 to segments-1 do
    let soff : integer = s * elements;
    for p = 0 to pairs-1 do
        result[(soff + p)*:esize] = operand1[(soff + 2 * p + part)*:esize];
    end;

    for p = 0 to pairs-1 do
        result[(soff + pairs + p)*:esize] = operand2[(soff + 2 * p + part)*:esize];
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

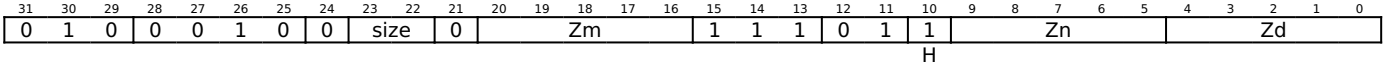
UZPQ2

Concatenate odd elements within each pair of quadword vector segments

This instruction concatenates adjacent odd-numbered elements from the corresponding 128-bit vector segments of the first and second source vectors, and places them in the corresponding elements of the destination vector segment. This instruction is unpredicated.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
UZPQ2 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let elements : integer = 128 DIV esize;
let pairs : integer = elements DIV 2;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for s = 0 to segments-1 do
    let soff : integer = s * elements;
    for p = 0 to pairs-1 do
        result[(soff + p)*:esize] = operand1[(soff + 2 * p + part)*:esize];
    end;

    for p = 0 to pairs-1 do
        result[(soff + pairs + p)*:esize] = operand2[(soff + 2 * p + part)*:esize];
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEGE (predicate as counter)

While decrementing signed scalar greater than or equal to scalar (predicate-as-counter)

This instruction generates a predicate for a group of two or four vectors that starting from the highest numbered element of the group is true while the decrementing value of the first, signed scalar operand is greater than or equal to the second scalar operand and false thereafter down to the lowest numbered element of the group.

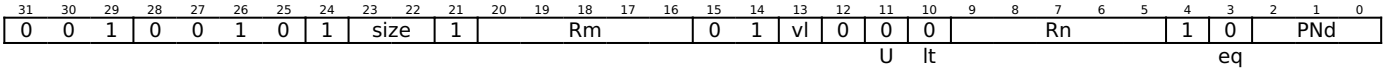
If the second scalar operand is equal to the minimum signed integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size.

The predicate result is placed in the predicate destination register using the predicate-as-counter encoding. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILEGE <PNd>.<T>, <Xn>, <Xm>, <vl>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt('1'::PNd);
let unsigned : boolean = FALSE;
let invert : boolean = TRUE;
let op : CmpOp = Cmp_GE;
let width : integer = 2 << UInt(vl);
```

Assembler Symbols

<PNd> Is the name of the destination scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNd" plus 8.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

<vl> Is the vl specifier, encoded in “vl”:

vl	<vl>
0	VLx2
1	VLx4

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = width * (VL DIV esize);
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
var count : integer = 0;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = elements-1 downto 0 do
    var cond : boolean;
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    if last then count = count + 1; end;
    operand1 = operand1 - 1;
end;

result = EncodePredCount{PL}(esize, elements, count, invert);
PSTATE.[N,Z,C,V] = PredCountTest(elements, count, invert);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEGE (predicate pair)

While decrementing signed scalar greater than or equal to scalar (pair of predicates)

This instruction generates a pair of predicates that starting from the highest numbered element of the pair is true while the decrementing value of the first, signed scalar operand is greater than or equal to the second scalar operand and false thereafter down to the lowest numbered element of the pair.

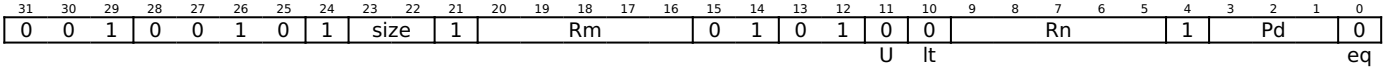
If the second scalar operand is equal to the minimum signed integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The lower-numbered elements are placed in the first predicate destination register, and the higher-numbered elements in the second predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILEGE { <Pd1>.<T>, <Pd2>.<T> }, <Xn>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d0 : integer = UInt(Pd::'0');
let d1 : integer = UInt(Pd::'1');
let unsigned : boolean = FALSE;
let op : CmpOp = Cmp_GE;
```

Assembler Symbols

<Pd1> Is the name of the first destination scalable predicate register, encoded as "Pd" times 2.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pd2> Is the name of the second destination scalable predicate register, encoded as "Pd" times 2 plus 1.

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL*2) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL*2);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = (elements*2)-1 downto 0 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 - 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL*2}(mask, result, esize);
P{PL}(d0) = result[PL-1:0];
P{PL}(d1) = result[PL*2-1:PL];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEGE (predicate)

While decrementing signed scalar greater than or equal to scalar

This instruction generates a predicate that starting from the highest numbered element is true while the decrementing value of the first, signed scalar operand is greater than or equal to the second scalar operand and false thereafter down to the lowest numbered element.

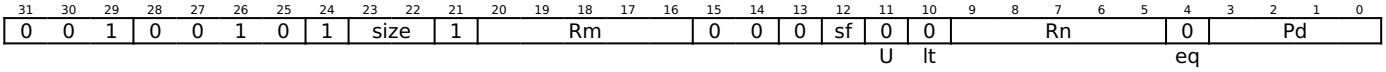
If the second scalar operand is equal to the minimum signed integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The predicate result is placed in the predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
WHILEGE <Pd>.<T>, <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 32 << UInt(sf);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Pd);
let unsigned : boolean = FALSE;
let op : CmpOp = Cmp_GE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “sf”:

sf	<R>
0	W
1	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = elements-1 downto 0 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 - 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEGT (predicate as counter)

While decrementing signed scalar greater than scalar (predicate-as-counter)

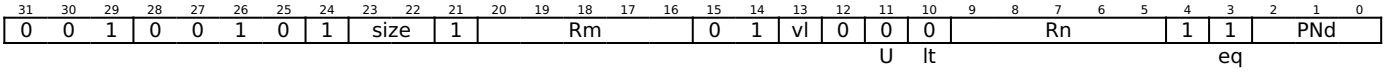
This instruction generates a predicate for a group of two or four vectors that starting from the highest numbered element of the group is true while the decrementing value of the first, signed scalar operand is greater than the second scalar operand and false thereafter down to the lowest numbered element of the group.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size.

The predicate result is placed in the predicate destination register using the predicate-as-counter encoding. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILEGT <PNd>.<T>, <Xn>, <Xm>, <vl>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt('1'::PNd);
let unsigned : boolean = FALSE;
let invert : boolean = TRUE;
let op : CmpOp = Cmp_GT;
let width : integer = 2 << UInt(vl);
```

Assembler Symbols

<PNd> Is the name of the destination scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNd" plus 8.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

<vl> Is the vl specifier, encoded in “vl”:

vl	<vl>
0	VLx2
1	VLx4

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = width * (VL DIV esize);
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
var count : integer = 0;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = elements-1 downto 0 do
    var cond : boolean;
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    if last then count = count + 1; end;
    operand1 = operand1 - 1;
end;

result = EncodePredCount{PL}(esize, elements, count, invert);
PSTATE.[N,Z,C,V] = PredCountTest(elements, count, invert);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEGT (predicate pair)

While decrementing signed scalar greater than scalar (pair of predicates)

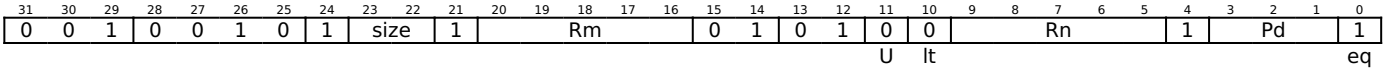
This instruction generates a pair of predicates that starting from the highest numbered element of the pair is true while the decrementing value of the first, signed scalar operand is greater than the second scalar operand and false thereafter down to the lowest numbered element of the pair.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The lower-numbered elements are placed in the first predicate destination register, and the higher-numbered elements in the second predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILEGT { <Pd1>.<T>, <Pd2>.<T> }, <Xn>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d0 : integer = UInt(Pd::'0');
let d1 : integer = UInt(Pd::'1');
let unsigned : boolean = FALSE;
let op : CmpOp = Cmp_GT;
```

Assembler Symbols

<Pd1> Is the name of the first destination scalable predicate register, encoded as "Pd" times 2.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pd2> Is the name of the second destination scalable predicate register, encoded as "Pd" times 2 plus 1.

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL*2) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL*2);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = (elements*2)-1 downto 0 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 - 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL*2}(mask, result, esize);
P{PL}(d0) = result[PL-1:0];
P{PL}(d1) = result[PL*2-1:PL];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEGT (predicate)

While decrementing signed scalar greater than scalar

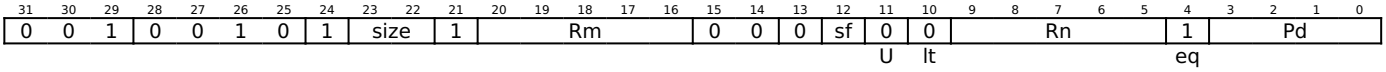
This instruction generates a predicate that starting from the highest numbered element is true while the decrementing value of the first, signed scalar operand is greater than the second scalar operand and false thereafter down to the lowest numbered element.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The predicate result is placed in the predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
WHILEGT <Pd>.<T>, <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 32 << UInt(sf);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Pd);
let unsigned : boolean = FALSE;
let op : CmpOp = Cmp_GT;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “sf”:

sf	<R>
0	W
1	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = elements-1 downto 0 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 - 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEHI (predicate as counter)

While decrementing unsigned scalar higher than scalar (predicate-as-counter)

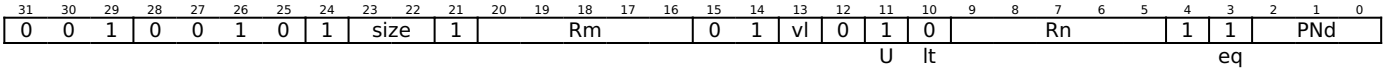
This instruction generates a predicate for a group of two or four vectors that starting from the highest numbered element of the group is true while the decrementing value of the first, unsigned scalar operand is higher than the second scalar operand and false thereafter down to the lowest numbered element of the group.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size.

The predicate result is placed in the predicate destination register using the predicate-as-counter encoding. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILEHI <PNd>.<T>, <Xn>, <Xm>, <vl>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt('1'::PNd);
let unsigned : boolean = TRUE;
let invert : boolean = TRUE;
let op : CmpOp = Cmp_GT;
let width : integer = 2 << UInt(vl);
```

Assembler Symbols

<PNd> Is the name of the destination scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNd" plus 8.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

<vl> Is the vl specifier, encoded in “vl”:

vl	<vl>
0	VLx2
1	VLx4

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = width * (VL DIV esize);
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
var count : integer = 0;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = elements-1 downto 0 do
    var cond : boolean;
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    if last then count = count + 1; end;
    operand1 = operand1 - 1;
end;

result = EncodePredCount{PL}(esize, elements, count, invert);
PSTATE.[N,Z,C,V] = PredCountTest(elements, count, invert);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEHI (predicate pair)

While decrementing unsigned scalar higher than scalar (pair of predicates)

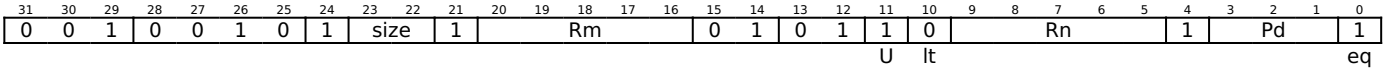
This instruction generates a pair of predicates that starting from the highest numbered element of the pair is true while the decrementing value of the first, unsigned scalar operand is higher than the second scalar operand and false thereafter down to the lowest numbered element of the pair.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The lower-numbered elements are placed in the first predicate destination register, and the higher-numbered elements in the second predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILEHI { <Pd1>.<T>, <Pd2>.<T> }, <Xn>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d0 : integer = UInt(Pd::'0');
let d1 : integer = UInt(Pd::'1');
let unsigned : boolean = TRUE;
let op : CmpOp = Cmp_GT;
```

Assembler Symbols

<Pd1> Is the name of the first destination scalable predicate register, encoded as "Pd" times 2.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pd2> Is the name of the second destination scalable predicate register, encoded as "Pd" times 2 plus 1.

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL*2) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL*2);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = (elements*2)-1 downto 0 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 - 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL*2}(mask, result, esize);
P{PL}(d0) = result[PL-1:0];
P{PL}(d1) = result[PL*2-1:PL];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEHI (predicate)

While decrementing unsigned scalar higher than scalar

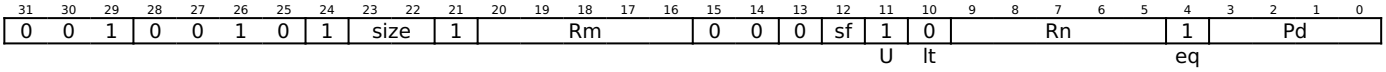
This instruction generates a predicate that starting from the highest numbered element is true while the decrementing value of the first, unsigned scalar operand is higher than the second scalar operand and false thereafter down to the lowest numbered element.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The predicate result is placed in the predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
WHILEHI <Pd>.<T>, <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 32 << UInt(sf);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Pd);
let unsigned : boolean = TRUE;
let op : CmpOp = Cmp_GT;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “sf”:

sf	<R>
0	W
1	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = elements-1 downto 0 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 - 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEHS (predicate as counter)

While decrementing unsigned scalar higher than or same as scalar (predicate-as-counter)

This instruction generates a predicate for a group of two or four vectors that starting from the highest numbered element of the group is true while the decrementing value of the first, unsigned scalar operand is higher than or the same as the second scalar operand and false thereafter down to the lowest numbered element of the group.

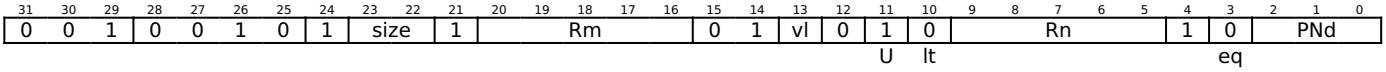
If the second scalar operand is equal to the minimum unsigned integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size.

The predicate result is placed in the predicate destination register using the predicate-as-counter encoding. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILEHS <PNd>.<T>, <Xn>, <Xm>, <vl>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt('1'::PNd);
let unsigned : boolean = TRUE;
let invert : boolean = TRUE;
let op : CmpOp = Cmp_GE;
let width : integer = 2 << UInt(vl);
```

Assembler Symbols

<PNd> Is the name of the destination scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNd" plus 8.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

<vl> Is the vl specifier, encoded in “vl”:

vl	<vl>
0	VLx2
1	VLx4

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = width * (VL DIV esize);
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
var count : integer = 0;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = elements-1 downto 0 do
    var cond : boolean;
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    if last then count = count + 1; end;
    operand1 = operand1 - 1;
end;

result = EncodePredCount{PL}(esize, elements, count, invert);
PSTATE.[N,Z,C,V] = PredCountTest(elements, count, invert);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEHS (predicate pair)

While decrementing unsigned scalar higher than or same as scalar (pair of predicates)

This instruction generates a pair of predicates that starting from the highest numbered element of the pair is true while the decrementing value of the first, unsigned scalar operand is higher than or the same as the second scalar operand and false thereafter down to the lowest numbered element of the pair.

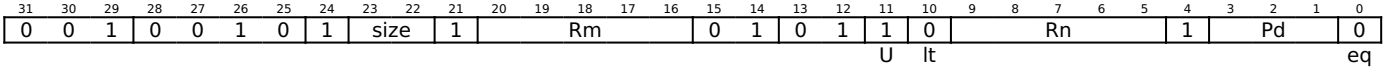
If the second scalar operand is equal to the minimum unsigned integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The lower-numbered elements are placed in the first predicate destination register, and the higher-numbered elements in the second predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILEHS { <Pd1>.<T>, <Pd2>.<T> }, <Xn>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d0 : integer = UInt(Pd::'0');
let d1 : integer = UInt(Pd::'1');
let unsigned : boolean = TRUE;
let op : CmpOp = Cmp_GE;
```

Assembler Symbols

<Pd1> Is the name of the first destination scalable predicate register, encoded as "Pd" times 2.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pd2> Is the name of the second destination scalable predicate register, encoded as "Pd" times 2 plus 1.

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL*2) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL*2);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = (elements*2)-1 downto 0 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 - 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL*2}(mask, result, esize);
P{PL}(d0) = result[PL-1:0];
P{PL}(d1) = result[PL*2-1:PL];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILEHS (predicate)

While decrementing unsigned scalar higher than or same as scalar

This instruction generates a predicate that starting from the highest numbered element is true while the decrementing value of the first, unsigned scalar operand is higher than or the same as the second scalar operand and false thereafter down to the lowest numbered element.

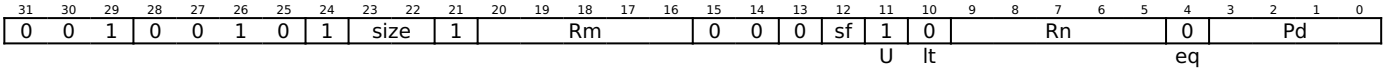
If the second scalar operand is equal to the minimum unsigned integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is decremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The predicate result is placed in the predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
WHILEHS <Pd>.<T>, <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 32 << UInt(sf);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Pd);
let unsigned : boolean = TRUE;
let op : CmpOp = Cmp_GE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “sf”:

sf	<R>
0	W
1	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = elements-1 downto 0 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_GT => cond = (element1 >  element2);
        when Cmp_GE => cond = (element1 >= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 - 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELE (predicate as counter)

While incrementing signed scalar less than or equal to scalar (predicate-as-counter)

This instruction generates a predicate for a group of two or four vectors that starting from the lowest numbered element of the group is true while the incrementing value of the first, signed scalar operand is less than or equal to the second scalar operand and false thereafter up to the highest numbered element of the group.

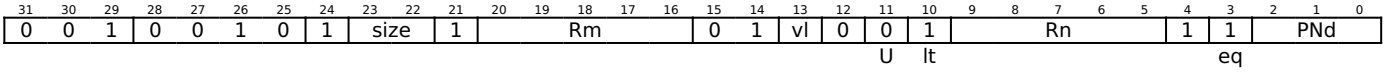
If the second scalar operand is equal to the maximum signed integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size.

The predicate result is placed in the predicate destination register using the predicate-as-counter encoding. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILELE <PNd>.<T>, <Xn>, <Xm>, <vl>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt('1'::PNd);
let unsigned : boolean = FALSE;
let invert : boolean = FALSE;
let op : CmpOp = Cmp_LE;
let width : integer = 2 << UInt(vl);
```

Assembler Symbols

<PNd> Is the name of the destination scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNd" plus 8.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

<vl> Is the vl specifier, encoded in “vl”:

vl	<vl>
0	VLx2
1	VLx4

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = width * (VL DIV esize);
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
var count : integer = 0;

let op2val : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to elements-1 do
    let op1val : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (op1val <  op2val);
        when Cmp_LE => cond = (op1val <= op2val);
    end;

    last = last && cond;
    if last then count = count + 1; end;
    operand1 = operand1 + 1;
end;

result = EncodePredCount{PL}(esize, elements, count, invert);
PSTATE.[N,Z,C,V] = PredCountTest(elements, count, invert);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELE (predicate pair)

While incrementing signed scalar less than or equal to scalar (pair of predicates)

This instruction generates a pair of predicates that starting from the lowest numbered element of the pair is true while the incrementing value of the first, signed scalar operand is less than or equal to the second scalar operand and false thereafter up to the highest numbered element of the pair.

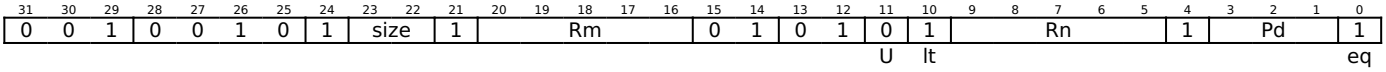
If the second scalar operand is equal to the maximum signed integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The lower-numbered elements are placed in the first predicate destination register, and the higher-numbered elements in the second predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILELE { <Pd1>.<T>, <Pd2>.<T> }, <Xn>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d0 : integer = UInt(Pd::'0');
let d1 : integer = UInt(Pd::'1');
let unsigned : boolean = FALSE;
let op : CmpOp = Cmp_LE;
```

Assembler Symbols

<Pd1> Is the name of the first destination scalable predicate register, encoded as "Pd" times 2.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pd2> Is the name of the second destination scalable predicate register, encoded as "Pd" times 2 plus 1.

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL*2) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL*2);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to (elements*2)-1 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (element1 <  element2);
        when Cmp_LE => cond = (element1 <= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 + 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL*2}(mask, result, esize);
P{PL}(d0) = result[PL-1:0];
P{PL}(d1) = result[PL*2-1:PL];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELE (predicate)

While incrementing signed scalar less than or equal to scalar

This instruction generates a predicate that starting from the lowest numbered element is true while the incrementing value of the first, signed scalar operand is less than or equal to the second scalar operand and false thereafter up to the highest numbered element.

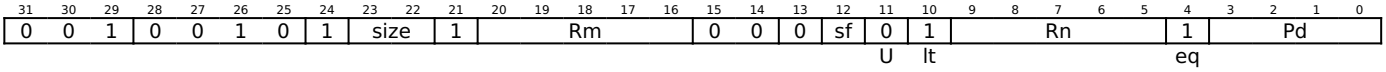
If the second scalar operand is equal to the maximum signed integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The predicate result is placed in the predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
WHILELE <Pd>.<T>, <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 32 << UInt(sf);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Pd);
let unsigned : boolean = FALSE;
let op : CmpOp = Cmp_LE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “sf”:

sf	<R>
0	W
1	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to elements-1 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (element1 <  element2);
        when Cmp_LE => cond = (element1 <= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 + 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELO (predicate as counter)

While incrementing unsigned scalar lower than scalar (predicate-as-counter)

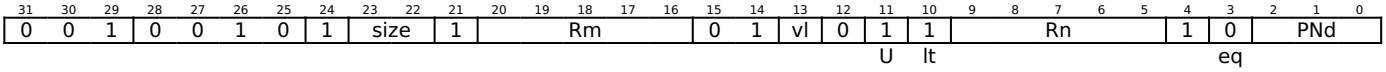
This instruction generates a predicate for a group of two or four vectors that starting from the lowest numbered element of the group is true while the incrementing value of the first, unsigned scalar operand is lower than the second scalar operand and false thereafter up to the highest numbered element of the group.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size.

The predicate result is placed in the predicate destination register using the predicate-as-counter encoding. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILELO <PNd>.<T>, <Xn>, <Xm>, <vl>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt('1'::PNd);
let unsigned : boolean = TRUE;
let invert : boolean = FALSE;
let op : CmpOp = Cmp_LT;
let width : integer = 2 << UInt(vl);
```

Assembler Symbols

<PNd> Is the name of the destination scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNd" plus 8.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

<vl> Is the vl specifier, encoded in “vl”:

vl	<vl>
0	VLx2
1	VLx4

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = width * (VL DIV esize);
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
var count : integer = 0;

let op2val : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to elements-1 do
    let op1val : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (op1val <  op2val);
        when Cmp_LE => cond = (op1val <= op2val);
    end;

    last = last && cond;
    if last then count = count + 1; end;
    operand1 = operand1 + 1;
end;

result = EncodePredCount{PL}(esize, elements, count, invert);
PSTATE.[N,Z,C,V] = PredCountTest(elements, count, invert);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELO (predicate pair)

While incrementing unsigned scalar lower than scalar (pair of predicates)

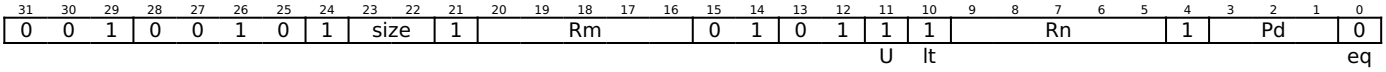
This instruction generates a pair of predicates that starting from the lowest numbered element of the pair is true while the incrementing value of the first, unsigned scalar operand is lower than the second scalar operand and false thereafter up to the highest numbered element of the pair.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The lower-numbered elements are placed in the first predicate destination register, and the higher-numbered elements in the second predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILELO { <Pd1>.<T>, <Pd2>.<T> }, <Xn>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d0 : integer = UInt(Pd::'0');
let d1 : integer = UInt(Pd::'1');
let unsigned : boolean = TRUE;
let op : CmpOp = Cmp_LT;
```

Assembler Symbols

<Pd1> Is the name of the first destination scalable predicate register, encoded as "Pd" times 2.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pd2> Is the name of the second destination scalable predicate register, encoded as "Pd" times 2 plus 1.

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL*2) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL*2);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to (elements*2)-1 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (element1 <  element2);
        when Cmp_LE => cond = (element1 <= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 + 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL*2}(mask, result, esize);
P{PL}(d0) = result[PL-1:0];
P{PL}(d1) = result[PL*2-1:PL];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELO (predicate)

While incrementing unsigned scalar lower than scalar

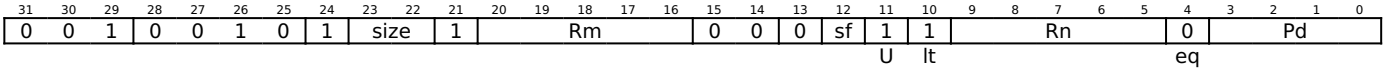
This instruction generates a predicate that starting from the lowest numbered element is true while the incrementing value of the first, unsigned scalar operand is lower than the second scalar operand and false thereafter up to the highest numbered element.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The predicate result is placed in the predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
WHILELO <Pd>.<T>, <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 32 << UInt(sf);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Pd);
let unsigned : boolean = TRUE;
let op : CmpOp = Cmp_LT;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “sf”:

sf	<R>
0	W
1	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to elements-1 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (element1 <  element2);
        when Cmp_LE => cond = (element1 <= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 + 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELS (predicate as counter)

While incrementing unsigned scalar lower than or same as scalar (predicate-as-counter)

This instruction generates a predicate for a group of two or four vectors that starting from the lowest numbered element of the group is true while the incrementing value of the first, unsigned scalar operand is lower than or the same as the second scalar operand and false thereafter up to the highest numbered element of the group.

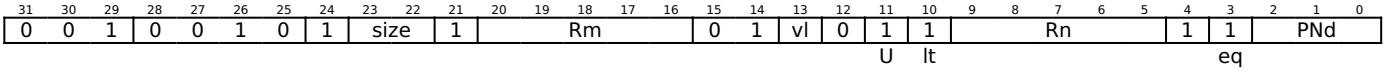
If the second scalar operand is equal to the maximum unsigned integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size.

The predicate result is placed in the predicate destination register using the predicate-as-counter encoding. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILELS <PNd>.<T>, <Xn>, <Xm>, <vl>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt('1'::PNd);
let unsigned : boolean = TRUE;
let invert : boolean = FALSE;
let op : CmpOp = Cmp_LE;
let width : integer = 2 << UInt(vl);
```

Assembler Symbols

<PNd> Is the name of the destination scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNd" plus 8.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

<vl> Is the vl specifier, encoded in “vl”:

vl	<vl>
0	VLx2
1	VLx4

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = width * (VL DIV esize);
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
var count : integer = 0;

let op2val : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to elements-1 do
    let op1val : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (op1val <  op2val);
        when Cmp_LE => cond = (op1val <= op2val);
    end;

    last = last && cond;
    if last then count = count + 1; end;
    operand1 = operand1 + 1;
end;

result = EncodePredCount{PL}(esize, elements, count, invert);
PSTATE.[N,Z,C,V] = PredCountTest(elements, count, invert);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELS (predicate pair)

While incrementing unsigned scalar lower than or same as scalar (pair of predicates)

This instruction generates a pair of predicates that starting from the lowest numbered element of the pair is true while the incrementing value of the first, unsigned scalar operand is lower than or the same as the second scalar operand and false thereafter up to the highest numbered element of the pair.

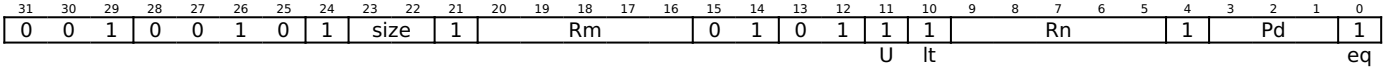
If the second scalar operand is equal to the maximum unsigned integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The lower-numbered elements are placed in the first predicate destination register, and the higher-numbered elements in the second predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILELS { <Pd1>.<T>, <Pd2>.<T> }, <Xn>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d0 : integer = UInt(Pd::'0');
let d1 : integer = UInt(Pd::'1');
let unsigned : boolean = TRUE;
let op : CmpOp = Cmp_LE;
```

Assembler Symbols

<Pd1> Is the name of the first destination scalable predicate register, encoded as "Pd" times 2.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pd2> Is the name of the second destination scalable predicate register, encoded as "Pd" times 2 plus 1.

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL*2) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL*2);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to (elements*2)-1 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (element1 <  element2);
        when Cmp_LE => cond = (element1 <= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 + 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL*2}(mask, result, esize);
P{PL}(d0) = result[PL-1:0];
P{PL}(d1) = result[PL*2-1:PL];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELS (predicate)

While incrementing unsigned scalar lower than or same as scalar

This instruction generates a predicate that starting from the lowest numbered element is true while the incrementing value of the first, unsigned scalar operand is lower than or the same as the second scalar operand and false thereafter up to the highest numbered element.

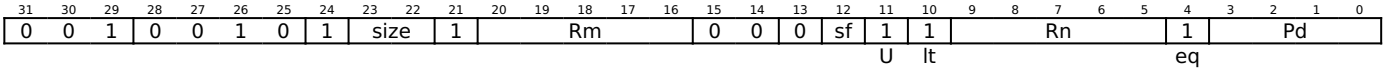
If the second scalar operand is equal to the maximum unsigned integer value, then a condition that includes an equality test can never fail and the result will be an all-true predicate.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The predicate result is placed in the predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
WHILELS <Pd>.<T>, <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 32 << UInt(sf);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Pd);
let unsigned : boolean = TRUE;
let op : CmpOp = Cmp_LE;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “sf”:

sf	<R>
0	W
1	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to elements-1 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (element1 <  element2);
        when Cmp_LE => cond = (element1 <= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 + 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELT (predicate as counter)

While incrementing signed scalar less than scalar (predicate-as-counter)

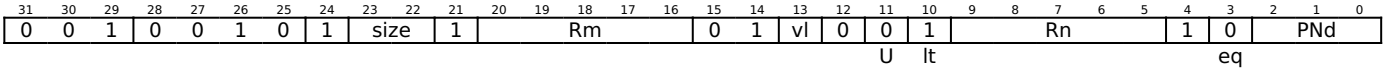
This instruction generates a predicate for a group of two or four vectors that starting from the lowest numbered element of the group is true while the incrementing value of the first, signed scalar operand is less than the second scalar operand and false thereafter up to the highest numbered element of the group.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size.

The predicate result is placed in the predicate destination register using the predicate-as-counter encoding. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILELT <PNd>.<T>, <Xn>, <Xm>, <vl>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt('1'::PNd);
let unsigned : boolean = FALSE;
let invert : boolean = FALSE;
let op : CmpOp = Cmp_LT;
let width : integer = 2 << UInt(vl);
```

Assembler Symbols

<PNd> Is the name of the destination scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNd" plus 8.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

<vl> Is the vl specifier, encoded in “vl”:

vl	<vl>
0	VLx2
1	VLx4

Operation

```
if IsFeatureImplemented(FEAT_SVE2p1) then
    CheckSVEEnabled();
else
    CheckStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = width * (VL DIV esize);
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
var count : integer = 0;

let op2val : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to elements-1 do
    let op1val : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (op1val <  op2val);
        when Cmp_LE => cond = (op1val <= op2val);
    end;

    last = last && cond;
    if last then count = count + 1; end;
    operand1 = operand1 + 1;
end;

result = EncodePredCount{PL}(esize, elements, count, invert);
PSTATE.[N,Z,C,V] = PredCountTest(elements, count, invert);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELT (predicate pair)

While incrementing signed scalar less than scalar (pair of predicates)

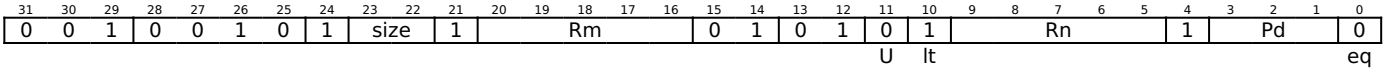
This instruction generates a pair of predicates that starting from the lowest numbered element of the pair is true while the incrementing value of the first, signed scalar operand is less than the second scalar operand and false thereafter up to the highest numbered element of the pair.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The lower-numbered elements are placed in the first predicate destination register, and the higher-numbered elements in the second predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SME2 || FEAT_SVE2p1)



Encoding

```
WHILELT { <Pd1>.<T>, <Pd2>.<T> }, <Xn>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) && !IsFeatureImplemented(FEAT_SVE2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 64;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d0 : integer = UInt(Pd::'0');
let d1 : integer = UInt(Pd::'1');
let unsigned : boolean = FALSE;
let op : CmpOp = Cmp_LT;
```

Assembler Symbols

<Pd1> Is the name of the first destination scalable predicate register, encoded as "Pd" times 2.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pd2> Is the name of the second destination scalable predicate register, encoded as "Pd" times 2 plus 1.

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL*2) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL*2);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to (elements*2)-1 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (element1 <  element2);
        when Cmp_LE => cond = (element1 <= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 + 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL*2}(mask, result, esize);
P{PL}(d0) = result[PL-1:0];
P{PL}(d1) = result[PL*2-1:PL];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WHILELT (predicate)

While incrementing signed scalar less than scalar

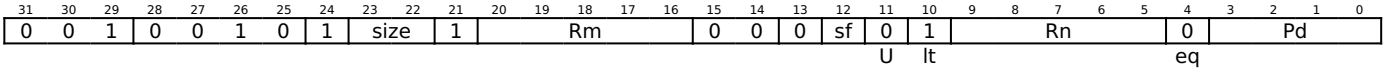
This instruction generates a predicate that starting from the lowest numbered element is true while the incrementing value of the first, signed scalar operand is less than the second scalar operand and false thereafter up to the highest numbered element.

The full width of the scalar operands is significant for the purposes of comparison, and the full width first operand is incremented by one for each destination predicate element, irrespective of the predicate result element size. The first general-purpose source register is not itself updated.

The predicate result is placed in the predicate destination register. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE

(FEAT_SVE || FEAT_SME)



Encoding

```
WHILELT <Pd>.<T>, <R><n>, <R><m>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let rsize : integer{} = 32 << UInt(sf);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Pd);
let unsigned : boolean = FALSE;
let op : CmpOp = Cmp_LT;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<R> Is a width specifier, encoded in “sf”:

sf	<R>
0	W
1	X

<n> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rn" field.

<m> Is the number [0-30] of the source general-purpose register or the name ZR (31), encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = Ones{};
var operand1 : bits(rsize) = X{}(n);
let operand2 : bits(rsize) = X{}(m);
var result : bits(PL);
var last : boolean = TRUE;
let psize : integer{} = esize DIV 8;

let element2 : integer = if unsigned then UInt(operand2) else SInt(operand2);
for e = 0 to elements-1 do
    let element1 : integer = if unsigned then UInt(operand1) else SInt(operand1);
    var cond : boolean;
    case op of
        when Cmp_LT => cond = (element1 <  element2);
        when Cmp_LE => cond = (element1 <= element2);
    end;

    last = last && cond;
    let pbit : bit = if last then '1' else '0';
    result[e*:psize] = ZeroExtend{psize}(pbit);
    operand1 = operand1 + 1;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

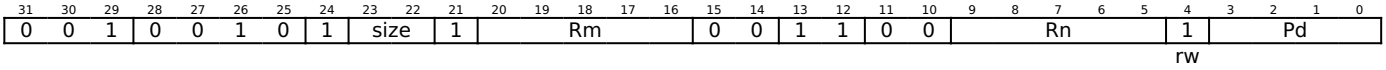
WHILERW

While free of read-after-write conflicts

This instruction checks two addresses for a conflict or overlap between address ranges of the form [addr,addr+VL÷8), where VL is the accessible vector length in bits, that could result in a loop-carried dependency through memory due to the use of these addresses by contiguous load and store instructions within the same iteration of a loop. Generate a predicate whose elements are true while the addresses cannot conflict within the same iteration, and false thereafter. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
WHILERW <Pd>.<T>, <Xn>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Pd);
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = Ones{};
let src1 : bits(64) = X{}(n);
let src2 : bits(64) = X{}(m);
let operand1 : integer = UInt(src1);
let operand2 : integer = UInt(src2);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

let diff : integer = Abs(operand2 - operand1) DIVRM (esize DIV 8);
for e = 0 to elements-1 do
    if diff == 0 || e < diff then
        result[e*:psize] = ZeroExtend{psize}('1');
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

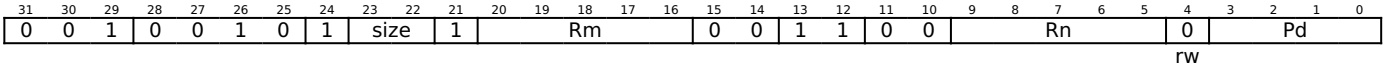
WHILEWR

While free of write-after-read/write conflicts

This instruction checks two addresses for a conflict or overlap between address ranges of the form [addr,addr+VL÷8), where VL is the accessible vector length in bits, that could result in a loop-carried dependency through memory due to the use of these addresses by contiguous load and store instructions within the same iteration of a loop. Generate a predicate whose elements are true while the addresses cannot conflict within the same iteration, and false thereafter. This instruction sets the First (N), None (Z), and !Last (C) condition flags based on the predicate result, and sets the V flag to zero.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
WHILEWR <Pd>.<T>, <Xn>, <Xm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let d : integer = UInt(Pd);
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Xn> Is the 64-bit name of the first source general-purpose register, encoded in the "Rn" field.

<Xm> Is the 64-bit name of the second source general-purpose register, encoded in the "Rm" field.

Operation

```
CheckSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mask : bits(PL) = Ones{};
let src1 : bits(64) = X{}(n);
let src2 : bits(64) = X{}(m);
let operand1 : integer = UInt(src1);
let operand2 : integer = UInt(src2);
var result : bits(PL);
let psize : integer{} = esize DIV 8;

let diff : integer = (operand2 - operand1) DIVRM (esize DIV 8);
for e = 0 to elements-1 do
    if diff <= 0 || e < diff then
        result[e*:psize] = ZeroExtend{psize}('1');
    else
        result[e*:psize] = ZeroExtend{psize}('0');
    end;
end;

PSTATE.[N,Z,C,V] = PredTest{PL}(mask, result, esize);
P{PL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

WRFFR

Write the first-fault register

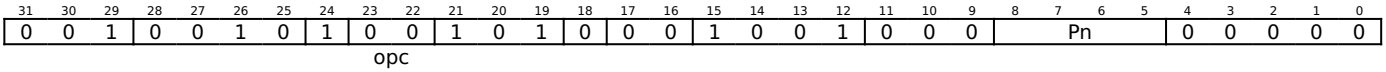
This instruction reads the source predicate register, and writes the value in the register to the first-fault register (FFR). This instruction is intended to restore a saved FFR and is not recommended for general use by applications.

This instruction requires that the source predicate contains a monotonic predicate value, in which starting from bit 0 there are zero or more 1 bits, followed only by 0 bits in any remaining bit positions. If the source is not a monotonic predicate value, then the resulting value in the FFR will be UNPREDICTABLE. It is not possible to generate a non-monotonic value in FFR when using SETFFR followed by first-fault or non-fault loads.

This instruction is illegal when executed in Streaming SVE mode, unless FEAT_SME_FA64 is implemented and enabled.

SVE

(FEAT_SVE)



Encoding

```
WRFFR <Pn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Pn);
```

Assembler Symbols

<Pn>	Is the name of the source scalable predicate register, encoded in the "Pn" field.

Operation

```
CheckNonStreamingSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let operand : bits(PL) = P{}(n);

let hsb : integer{} = HighestSetBit(operand);
if hsb < 0 || IsOnes(operand[hsb:0]) then
    FFR{PL}() = operand;
else // not a monotonic predicate
    FFR{PL}() = ARBITRARY : bits(PL);
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

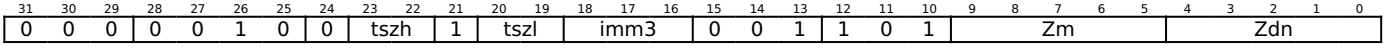
XAR

Bitwise exclusive-OR and rotate right by immediate

This instruction performs a bitwise exclusive-OR between the corresponding elements of the first source vector and the second source vector, and then rotates each result element right by an immediate amount. The final results are destructively placed in the corresponding elements of the destination and first source vector. This instruction is unpredicated.

SVE2

(FEAT_SVE2 || FEAT_SME)



Encoding

```
XAR <Zdn>.<T>, <Zdn>.<T>, <Zm>.<T>, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;

let tsize : bits(4) = tszh::tszl;
if tsize == '0000' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let m : integer = UInt(Zm);
let dn : integer = UInt(Zdn);
let rot : integer = 2 * esize - UInt(tsize::imm3);
```

Assembler Symbols

<Zdn> Is the name of the first source and destination scalable vector register, encoded in the "Zdn" field.

<T> Is the size specifier, encoded in “(tszh :: tszl)”:

tszh	tszl	<T>
00	00	RESERVED
00	01	B
00	1x	H
01	xx	S
1x	xx	D

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<const> Is the immediate shift amount, in the range 1 to number of bits per element, encoded in "tszh:tszl:imm3".

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let operand1 : bits(VL) = Z{}(dn);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for e = 0 to elements-1 do
    let element1 : bits(esize) = operand1[e*:esize];
    let element2 : bits(esize) = operand2[e*:esize];
    result[e*:esize] = ROR(element1 XOR element2, rot);
end;

Z{VL}(dn) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

This instruction might be immediately preceded in program order by a MOVPRFX instruction. The MOVPRFX must conform to all of the following requirements, otherwise the behavior of the MOVPRFX and this instruction is CONSTRAINED UNPREDICTABLE:

- The MOVPRFX must be unpredicated.
- The MOVPRFX must specify the same destination register as this instruction.
- The destination register must not refer to architectural register state referenced by any other source operand register of this instruction.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ZIP1, ZIP2 (predicates)

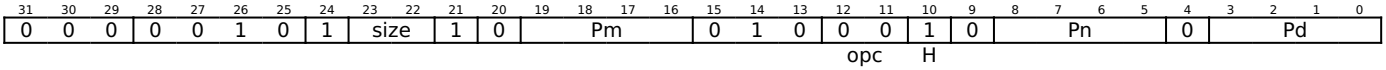
Interleave elements from two half predicates

This instruction interleaves alternating elements from the lowest or highest halves of the first and second source predicates, and places the values in the destination predicate. This instruction is unpredicated.

It has encodings from 2 classes: High halves and Low halves

High halves

(FEAT_SVE || FEAT_SME)



Encoding

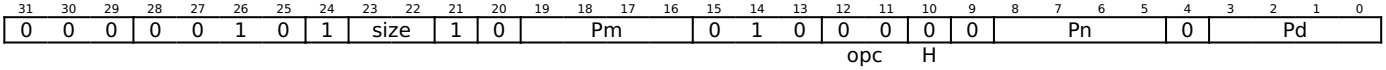
```
ZIP2 <Pd>.<T>, <Pn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let part : integer = 1;
```

Low halves

(FEAT_SVE || FEAT_SME)



Encoding

```
ZIP1 <Pd>.<T>, <Pn>.<T>, <Pm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Pn);
let m : integer = UInt(Pm);
let d : integer = UInt(Pd);
let part : integer = 0;
```

Assembler Symbols

<Pd> Is the name of the destination scalable predicate register, encoded in the "Pd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Pn> Is the name of the first source scalable predicate register, encoded in the "Pn" field.

<Pm> Is the name of the second source scalable predicate register, encoded in the "Pm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let pairs : integer = VL DIV (esize * 2);
let operand1 : bits(PL) = P{}(n);
let operand2 : bits(PL) = P{}(m);
var result : bits(PL);

let base : integer = part * pairs;
for p = 0 to pairs-1 do
    result[(2*p+0)*:(esize DIV 8)] = operand1[(base+p)*:(esize DIV 8)];
    result[(2*p+1)*:(esize DIV 8)] = operand2[(base+p)*:(esize DIV 8)];
end;

P{PL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ZIP1, ZIP2 (vectors)

Interleave elements from two half vectors

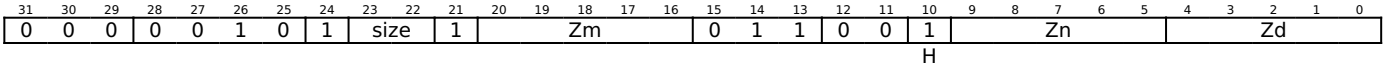
This instruction interleaves alternating elements from the lowest or highest halves of the first and second source vectors, and places the values in the destination vector. This instruction is unpredicated.

The High halves (quadwords) and Low halves (quadwords) variants require an Effective SVE vector length of at least 256 bits. ID_AA64ZFR0_EL1.F64MM or ID_AA64ZFR0_EL1.EltPerm indicates whether the High halves (quadwords) and Low halves (quadwords) variants are implemented. The High halves (quadwords) and Low halves (quadwords) variants are legal when executed in Streaming SVE mode if FEAT_SVE_PERM256 is implemented.

It has encodings from 4 classes: High halves , High halves (quadwords) , Low halves and Low halves (quadwords)

High halves

(FEAT_SVE || FEAT_SME)



Encoding

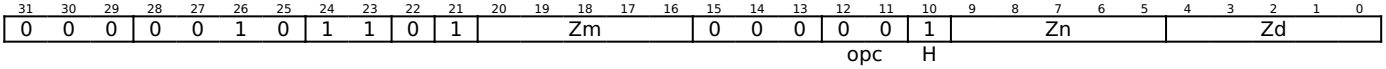
```
ZIP2 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

High halves (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

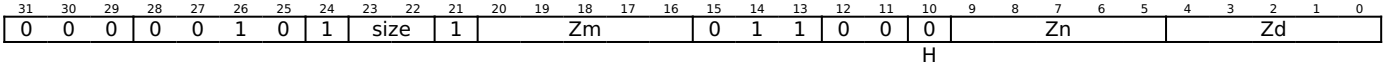
```
ZIP2 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) then
    EndOfDecode(Decode_UNDEF);
end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Low halves

(FEAT_SVE || FEAT_SME)



Encoding

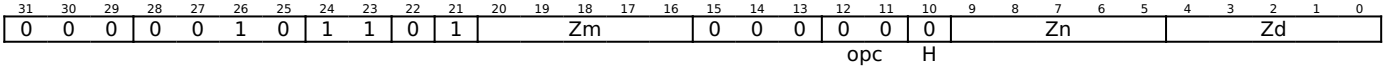
```
ZIP1 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE) && !IsFeatureImplemented(FEAT_SME) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Low halves (quadwords)

(FEAT_F64MM || FEAT_SVE_PERM256)



Encoding

```
ZIP1 <Zd>.Q, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_F64MM) && !IsFeatureImplemented(FEAT_SVE_PERM256) then
    EndOfDecode(Decode_UNDEF);
end;
if MaxImplementedVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
if esize < 128 || IsFeatureImplemented(FEAT_SVE_PERM256) then
    CheckSVEEnabled();
else
    CheckNonStreamingSVEEnabled();
end;
let VL : integer{} = CurrentVL();
if VL < esize * 2 then EndOfDecode(Decode_UNDEF); end;
let pairs : integer = VL DIV (esize * 2);
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL) = Zeros{};

let base : integer = part * pairs;
for p = 0 to pairs-1 do
    result[(2*p+0)*:esize] = operand1[(base+p)*:esize];
    result[(2*p+1)*:esize] = operand2[(base+p)*:esize];
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

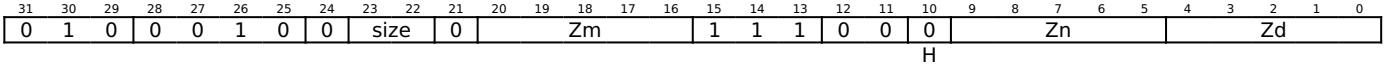
ZIPQ1

Interleave elements from low halves of each pair of quadword vector segments

This instruction interleaves alternating elements from low halves of the corresponding 128-bit vector segments of the first and second source vectors, and places them in the corresponding elements of the destination vector segment. This instruction is unpredicated.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ZIPQ1 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 0;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let elements : integer = 128 DIV esize;
let pairs : integer = elements DIV 2;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for s = 0 to segments-1 do
    let base : integer = s * elements + part * pairs;
    for p = 0 to pairs-1 do
        result[(s * elements + 2 * p + 0)*:esize] = operand1[(base + p)*:esize];
        result[(s * elements + 2 * p + 1)*:esize] = operand2[(base + p)*:esize];
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

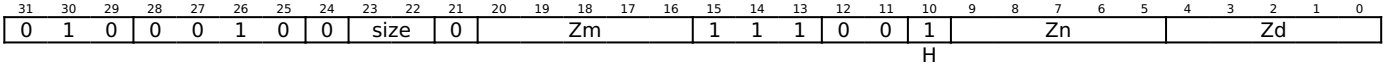
ZIPQ2

Interleave elements from high halves of each pair of quadword vector segments

This instruction interleaves alternating elements from high halves of the corresponding 128-bit vector segments of the first and second source vectors, and places them in the corresponding elements of the destination vector segment. This instruction is unpredicated.

SVE2

(FEAT_SVE2p1 || FEAT_SME2p1)



Encoding

```
ZIPQ2 <Zd>.<T>, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SVE2p1) && !IsFeatureImplemented(FEAT_SME2p1) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd);
let part : integer = 1;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckSVEEnabled();

let VL : integer{} = CurrentVL();
let segments : integer = VL DIV 128;
let elements : integer = 128 DIV esize;
let pairs : integer = elements DIV 2;
let operand1 : bits(VL) = Z{}(n);
let operand2 : bits(VL) = Z{}(m);
var result : bits(VL);

for s = 0 to segments-1 do
    let base : integer = s * elements + part * pairs;
    for p = 0 to pairs-1 do
        result[(s * elements + 2 * p + 0)*:esize] = operand1[(base + p)*:esize];
        result[(s * elements + 2 * p + 1)*:esize] = operand2[(base + p)*:esize];
    end;
end;

Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

A64 -- SME Instructions (alphabetic order)

ADD (to array, array and multiple vectors): Multi-vector accumulate to ZA array vectors.

ADD (to array, multiple and single vector): Multi-vector add by vector to ZA array vectors.

ADD (to array, multiple vectors): Multi-vector add to ZA array vectors.

ADD (to vector, multiple vectors): Multi-vector add by vector to multi-vector.

ADDHA: Add horizontally vector elements to ZA tile.

ADDSPL: Add multiple of Streaming SVE predicate register size to scalar register.

ADDSVL: Add multiple of Streaming SVE vector register size to scalar register.

ADDVA: Add vertically vector elements to ZA tile.

BF1CVT, BF2CVT: Multi-vector 8-bit floating-point convert to BFloat16.

BF1CVTL, BF2CVTL: Multi-vector 8-bit floating-point convert to deinterleaved BFloat16.

BFADD: Multi-vector BFloat16 accumulate to ZA array vectors.

BFCLAMP: Multi-vector BFloat16 clamp to minimum/maximum number.

BFCVT (BFloat16 to 8-bit floating-point): Multi-vector BFloat16 convert to 8-bit floating-point.

BFCVT (single-precision to BFloat16): Multi-vector single-precision convert to BFloat16.

BFCVTN: Multi-vector single-precision convert to interleaved BFloat16.

BFDOT (multiple and indexed vector): Multi-vector BFloat16 dot product by indexed element to single-precision.

BFDOT (multiple and single vector): Multi-vector BFloat16 dot product by vector to single-precision.

BFDOT (multiple vectors): Multi-vector BFloat16 dot product to single-precision.

BFMAX (multiple and single vector): Multi-vector BFloat16 maximum by vector.

BFMAX (multiple vectors): Multi-vector BFloat16 maximum.

BFMAXNM (multiple and single vector): Multi-vector BFloat16 maximum number by vector.

BFMAXNM (multiple vectors): Multi-vector BFloat16 maximum number.

BFMIN (multiple and single vector): Multi-vector BFloat16 minimum by vector.

BFMIN (multiple vectors): Multi-vector BFloat16 minimum.

BFMINNM (multiple and single vector): Multi-vector BFloat16 minimum number by vector.

BFMINNM (multiple vectors): Multi-vector BFloat16 minimum number.

BFMLA (multiple and indexed vector): Multi-vector BFloat16 fused multiply-add by indexed element.

BFMLA (multiple and single vector): Multi-vector BFloat16 fused multiply-add by vector.

BFMLA (multiple vectors): Multi-vector BFloat16 fused multiply-add.

BFMLAL (multiple and indexed vector): Multi-vector BFloat16 multiply-add by indexed element to single-precision.

BFMLAL (multiple and single vector): Multi-vector BFloat16 multiply-add by vector to single-precision.

BFMLAL (multiple vectors): Multi-vector BFloat16 multiply-add to single-precision.

BFMLS (multiple and indexed vector): Multi-vector BFloat16 fused multiply-subtract by indexed element.

BFMLS (multiple and single vector): Multi-vector BFloat16 fused multiply-subtract by vector.

BFMLS (multiple vectors): Multi-vector BFloat16 fused multiply-subtract.

BFMLSL (multiple and indexed vector): Multi-vector BFloat16 multiply-subtract by indexed element from single-precision.

BFMLSL (multiple and single vector): Multi-vector BFloat16 multiply-subtract by vector from single-precision.

BFMLSL (multiple vectors): Multi-vector BFloat16 multiply-subtract from single-precision.

BFMOP4A (non-widening): BFloat16 quarter-tile outer product, accumulating.

BFMOP4A (widening): BFloat16 quarter-tile sum of outer products to single-precision, accumulating.

BFMOP4S (non-widening): BFloat16 quarter-tile outer product, subtracting.

BFMOP4S (widening): BFloat16 quarter-tile sum of outer products to single-precision, subtracting.

BFMOPA (non-widening): BFloat16 outer product, accumulating.

BFMOPA (widening): BFloat16 sum of outer products to single-precision, accumulating.

BFMOPS (non-widening): BFloat16 outer product, subtracting.

BFMOPS (widening): BFloat16 sum of outer products to single-precision, subtracting.

BFMUL (multiple and single vector): Multi-vector BFloat16 multiply by vector.

BFMUL (multiple vectors): Multi-vector BFloat16 multiply.

BFSCALE (multiple and single vector): Multi-vector BFloat16 adjust exponent by vector.

BFSCALE (multiple vectors): Multi-vector BFloat16 adjust exponent.

BFSUB: Multi-vector BFloat16 subtract from ZA array vectors.

BFTMOPA (non-widening): BFloat16 sparse outer product, accumulating.

BFTMOPA (widening): BFloat16 sparse sum of outer products to single-precision, accumulating.

BFVDOT: Multi-vector BFloat16 vertical dot product by indexed element to single-precision.

BMOPA: Bitwise exclusive NOR population count outer product, accumulating.

BMOPS: Bitwise exclusive NOR population count outer product, subtracting.

F1CVT, F2CVT: Multi-vector 8-bit floating-point convert to half-precision.

F1CVTL, F2CVTL: Multi-vector 8-bit floating-point convert to deinterleaved half-precision.

FADD: Multi-vector floating-point accumulate to ZA array vectors.

FAMAX: Multi-vector floating-point absolute maximum.

FAMIN: Multi-vector floating-point absolute minimum.

FCLAMP: Multi-vector floating-point clamp to minimum/maximum number.

FCVT (narrowing, FP16 to FP8): Multi-vector half-precision convert to 8-bit floating-point.

FCVT (narrowing, FP32 to FP16): Multi-vector single-precision convert to half-precision.

FCVT (narrowing, FP32 to FP8): Multi-vector single-precision convert to 8-bit floating-point.

FCVT (widening): Multi-vector half-precision convert to single-precision.

FCVTL: Multi-vector half-precision convert to deinterleaved single-precision.

FCVTN (FP32 to FP16): Multi-vector single-precision convert to interleaved half-precision.

FCVTN (FP32 to FP8): Multi-vector single-precision convert to interleaved 8-bit floating-point.

FCVTZS: Multi-vector single-precision convert to signed 32-bit integer, rounding toward zero.

FCVTZU: Multi-vector single-precision convert to unsigned 32-bit integer, rounding toward zero.

FDOT (2-way, multiple and indexed vector, FP16 to FP32): Multi-vector half-precision dot product by indexed element to single-precision.

FDOT (2-way, multiple and indexed vector, FP8 to FP16): Multi-vector 8-bit floating-point dot product by indexed element to half-precision.

FDOT (2-way, multiple and single vector, FP16 to FP32): Multi-vector half-precision dot product by vector to single-precision.

FDOT (2-way, multiple and single vector, FP8 to FP16): Multi-vector 8-bit floating-point dot product by vector to half-precision.

FDOT (2-way, multiple vectors, FP16 to FP32): Multi-vector half-precision dot product to single-precision.

FDOT (2-way, multiple vectors, FP8 to FP16): Multi-vector 8-bit floating-point dot product to half-precision.

FDOT (4-way, multiple and indexed vector): Multi-vector 8-bit floating-point dot product by indexed element to single-precision.

FDOT (4-way, multiple and single vector): Multi-vector 8-bit floating-point dot product by vector to single-precision.

FDOT (4-way, multiple vectors): Multi-vector 8-bit floating-point dot product to single-precision.

FMAX (multiple and single vector): Multi-vector floating-point maximum by vector.

FMAX (multiple vectors): Multi-vector floating-point maximum.

FMAXNM (multiple and single vector): Multi-vector floating-point maximum number by vector.

FMAXNM (multiple vectors): Multi-vector floating-point maximum number.

FMIN (multiple and single vector): Multi-vector floating-point minimum by vector.

FMIN (multiple vectors): Multi-vector floating-point minimum.

FMINNM (multiple and single vector): Multi-vector floating-point minimum number by vector.

FMINNM (multiple vectors): Multi-vector floating-point minimum number.

FMLA (multiple and indexed vector): Multi-vector floating-point fused multiply-add by indexed element.

FMLA (multiple and single vector): Multi-vector floating-point fused multiply-add by vector.

FMLA (multiple vectors): Multi-vector floating-point fused multiply-add.

FMLAL (multiple and indexed vector, FP16 to FP32): Multi-vector half-precision multiply-add by indexed element to single-precision.

FMLAL (multiple and indexed vector, FP8 to FP16): Multi-vector 8-bit floating-point multiply-add by indexed element to half-precision.

FMLAL (multiple and single vector, FP16 to FP32): Multi-vector half-precision multiply-add by vector to single-precision.

FMLAL (multiple and single vector, FP8 to FP16): Multi-vector 8-bit floating-point multiply-add by vector to half-precision.

FMLAL (multiple vectors, FP16 to FP32): Multi-vector half-precision multiply-add to single-precision.

FMLAL (multiple vectors, FP8 to FP16): Multi-vector 8-bit floating-point multiply-add to half-precision.

FMLALL (multiple and indexed vector): Multi-vector 8-bit floating-point multiply-add by indexed element to single-precision.

FMLALL (multiple and single vector): Multi-vector 8-bit floating-point multiply-add by vector to single-precision.

FMLALL (multiple vectors): Multi-vector 8-bit floating-point multiply-add to single-precision.

FMLS (multiple and indexed vector): Multi-vector floating-point fused multiply-subtract by indexed element.

FMLS (multiple and single vector): Multi-vector floating-point fused multiply-subtract by vector.

FMLS (multiple vectors): Multi-vector floating-point fused multiply-subtract.

FMLSL (multiple and indexed vector): Multi-vector half-precision multiply-subtract by indexed element from single-precision.

FMLSL (multiple and single vector): Multi-vector half-precision multiply-subtract by vector from single-precision.

FMLSL (multiple vectors): Multi-vector half-precision multiply-subtract from single-precision.

FMOP4A (non-widening): Floating-point quarter-tile outer product, accumulating.

FMOP4A (widening, 2-way, FP16 to FP32): Half-precision quarter-tile sum of outer products to single-precision, accumulating.

FMOP4A (widening, 2-way, FP8 to FP16): 8-bit floating-point quarter-tile sum of outer products to half-precision, accumulating.

FMOP4A (widening, 4-way): 8-bit floating-point quarter-tile sum of outer products to single-precision, accumulating.

FMOP4S (non-widening): Floating-point quarter-tile outer product, subtracting.

FMOP4S (widening): Half-precision quarter-tile sum of outer products to single-precision, subtracting.

FMOPA (non-widening): Floating-point outer product, accumulating.

FMOPA (widening, 2-way, FP16 to FP32): Half-precision sum of outer products to single-precision, accumulating.

FMOPA (widening, 2-way, FP8 to FP16): 8-bit floating-point sum of outer products to half-precision, accumulating.

FMOPA (widening, 4-way): 8-bit floating-point sum of outer products to single-precision, accumulating.

FMOPS (non-widening): Floating-point outer product, subtracting.

FMOPS (widening): Half-precision sum of outer products to single-precision, subtracting.

FMUL (multiple and single vector): Multi-vector floating-point multiply by vector.

FMUL (multiple vectors): Multi-vector floating-point multiply.

FRINTA: Multi-vector single-precision round to integral value, to nearest with ties away from zero.

FRINTM: Multi-vector single-precision round to integral value, toward minus Infinity.

FRINTN: Multi-vector single-precision round to integral value, to nearest with ties to even.

FRINTP: Multi-vector single-precision round to integral value, toward plus Infinity.

FSCALE (multiple and single vector): Multi-vector floating-point adjust exponent by vector.

FSCALE (multiple vectors): Multi-vector floating-point adjust exponent.

FSUB: Multi-vector floating-point subtract from ZA array vectors.

FTMOPA (non-widening): Floating-point sparse outer product, accumulating.

FTMOPA (widening, 2-way, FP16 to FP32): Half-precision sparse sum of outer products to single-precision, accumulating.

FTMOPA (widening, 2-way, FP8 to FP16): 8-bit floating-point sparse sum of outer products to half-precision, accumulating.

FTMOPA (widening, 4-way): 8-bit floating-point sparse sum of outer products to single-precision, accumulating.

FVDOT (FP16 to FP32): Multi-vector half-precision vertical dot product by indexed element to single-precision.

FVDOT (FP8 to FP16): Multi-vector 8-bit floating-point vertical dot product by indexed element to half-precision.

FVDOTB: Multi-vector 8-bit floating-point vertical dot product by indexed element to single-precision (bottom).

FVDOTT: Multi-vector 8-bit floating-point vertical dot product by indexed element to single-precision (top).

LD1B (scalar plus immediate, strided registers): Contiguous load of bytes to multiple strided vectors (immediate index).

LD1B (scalar plus scalar, strided registers): Contiguous load of bytes to multiple strided vectors (scalar index).

LD1B (scalar plus scalar, tile slice): Contiguous load of bytes to 8-bit element ZA tile slice.

LD1D (scalar plus immediate, strided registers): Contiguous load of doublewords to multiple strided vectors (immediate index).

LD1D (scalar plus scalar, strided registers): Contiguous load of doublewords to multiple strided vectors (scalar index).

LD1D (scalar plus scalar, tile slice): Contiguous load of doublewords to 64-bit element ZA tile slice.

LD1H (scalar plus immediate, strided registers): Contiguous load of halfwords to multiple strided vectors (immediate index).

LD1H (scalar plus scalar, strided registers): Contiguous load of halfwords to multiple strided vectors (scalar index).

LD1H (scalar plus scalar, tile slice): Contiguous load of halfwords to 16-bit element ZA tile slice.

LD1Q: Contiguous load of quadwords to 128-bit element ZA tile slice.

LD1W (scalar plus immediate, strided registers): Contiguous load of words to multiple strided vectors (immediate index).

LD1W (scalar plus scalar, strided registers): Contiguous load of words to multiple strided vectors (scalar index).

LD1W (scalar plus scalar, tile slice): Contiguous load of words to 32-bit element ZA tile slice.

LDNT1B (scalar plus immediate, strided registers): Contiguous load non-temporal of bytes to multiple strided vectors (immediate index).

LDNT1B (scalar plus scalar, strided registers): Contiguous load non-temporal of bytes to multiple strided vectors (scalar index).

LDNT1D (scalar plus immediate, strided registers): Contiguous load non-temporal of doublewords to multiple strided vectors (immediate index).

LDNT1D (scalar plus scalar, strided registers): Contiguous load non-temporal of doublewords to multiple strided vectors (scalar index).

LDNT1H (scalar plus immediate, strided registers): Contiguous load non-temporal of halfwords to multiple strided vectors (immediate index).

LDNT1H (scalar plus scalar, strided registers): Contiguous load non-temporal of halfwords to multiple strided vectors (scalar index).

LDNT1W (scalar plus immediate, strided registers): Contiguous load non-temporal of words to multiple strided vectors (immediate index).

LDNT1W (scalar plus scalar, strided registers): Contiguous load non-temporal of words to multiple strided vectors (scalar index).

LDR (array vector): Load ZA array vector.

LDR (table): Load ZT0 register.

LUTI2 (four registers): Lookup table read with 2-bit indexes (four registers).

LUTI2 (single): Lookup table read with 2-bit indexes (single).

LUTI2 (two registers): Lookup table read with 2-bit indexes (two registers).

LUTI4 (four registers, 16-bit and 32-bit): Lookup table read with 4-bit indexes (four registers).

LUTI4 (four registers, 8-bit): Lookup table read with 4-bit indexes and 8-bit elements (four registers).

LUTI4 (single): Lookup table read with 4-bit indexes (single).

LUTI4 (two registers): Lookup table read with 4-bit indexes (two registers).

LUTI6 (table, four registers, 8-bit): Lookup table read with 6-bit indices (8-bit).

LUTI6 (table, single, 8-bit): Lookup table read with 6-bit indices (single).

LUTI6 (vector, 16-bit): Lookup table read with 6-bit indices (16-bit).

MOV (array to vector, four registers): Move four ZA single-vector groups to Z four-vector operand: an alias of MOVA (array to vector, four registers).

MOV (array to vector, two registers): Move two ZA single-vector groups to Z two-vector operand: an alias of MOVA (array to vector, two registers).

MOV (tile to vector, four registers): Move ZA four-slice operand to Z four-vector operand: an alias of MOVA (tile to vector, four registers).

MOV (tile to vector, single): Move ZA tile slice to Z vector: an alias of MOVA (tile to vector, single).

MOV (tile to vector, two registers): Move ZA two-slice operand to Z two-vector operand: an alias of MOVA (tile to vector, two registers).

MOV (vector to array, four registers): Move Z four-vector operand to four ZA single-vector groups: an alias of MOVA (vector to array, four registers).

MOV (vector to array, two registers): Move Z two-vector operand to two ZA single-vector groups: an alias of MOVA (vector to array, two registers).

MOV (vector to tile, four registers): Move Z four-vector operand to ZA four-slice operand: an alias of MOVA (vector to tile, four registers).

MOV (vector to tile, single): Move Z vector to ZA tile slice: an alias of MOVA (vector to tile, single).

MOV (vector to tile, two registers): Move Z two-vector operand to ZA two-slice operand: an alias of MOVA (vector to tile, two registers).

MOVA (array to vector, four registers): Move four ZA single-vector groups to Z four-vector operand.

MOVA (array to vector, two registers): Move two ZA single-vector groups to Z two-vector operand.

MOVA (tile to vector, four registers): Move ZA four-slice operand to Z four-vector operand.

MOVA (tile to vector, single): Move ZA tile slice to Z vector.

MOVA (tile to vector, two registers): Move ZA two-slice operand to Z two-vector operand.

MOVA (vector to array, four registers): Move Z four-vector operand to four ZA single-vector groups.

MOVA (vector to array, two registers): Move Z two-vector operand to two ZA single-vector groups.

MOVA (vector to tile, four registers): Move Z four-vector operand to ZA four-slice operand.

MOVA (vector to tile, single): Move Z vector to ZA tile slice.

MOVA (vector to tile, two registers): Move Z two-vector operand to ZA two-slice operand.

MOVAZ (array to vector, four registers): Move and zero four ZA single-vector groups to Z four-vector operand.

MOVAZ (array to vector, two registers): Move and zero two ZA single-vector groups to Z two-vector operand.

MOVAZ (tile to vector, four registers): Move and zero ZA four-slice operand to Z four-vector operand.

MOVAZ (tile to vector, single): Move and zero ZA tile slice to Z vector.

MOVAZ (tile to vector, two registers): Move and zero ZA two-slice operand to Z two-vector operand.

MOVT (scalar to table): Move 8 bytes from general-purpose register to ZT0.

MOVT (table to scalar): Move 8 bytes from ZT0 to general-purpose register.

MOVT (vector to table): Move vector register to ZT0.

RDSVL: Read multiple of Streaming SVE vector register size to scalar register.

SCLAMP: Multi-vector signed clamp to minimum/maximum.

SCVTF: Multi-vector signed 32-bit integer convert to single-precision.

SDOT (2-way, multiple and indexed vector): Multi-vector signed 16-bit integer dot product by indexed element to 32-bit integer.

SDOT (2-way, multiple and single vector): Multi-vector signed 16-bit integer dot product by vector to 32-bit integer.

SDOT (2-way, multiple vectors): Multi-vector signed 16-bit integer dot product to 32-bit integer.

SDOT (4-way, multiple and indexed vector): Multi-vector signed integer dot product by indexed element.

SDOT (4-way, multiple and single vector): Multi-vector signed integer dot product by vector.

SDOT (4-way, multiple vectors): Multi-vector signed integer dot product.

SEL: Multi-vector conditional select.

SMAX (multiple and single vector): Multi-vector signed maximum by vector.

SMAX (multiple vectors): Multi-vector signed maximum.

SMIN (multiple and single vector): Multi-vector signed minimum by vector.

SMIN (multiple vectors): Multi-vector signed minimum.

SMLAL (multiple and indexed vector): Multi-vector signed 16-bit integer multiply-add by indexed element to 32-bit integer.

SMLAL (multiple and single vector): Multi-vector signed 16-bit integer multiply-add by vector to 32-bit integer.

SMLAL (multiple vectors): Multi-vector signed 16-bit integer multiply-add to 32-bit integer.

SMLALL (multiple and indexed vector): Multi-vector signed integer multiply-add long long by indexed element.

SMLALL (multiple and single vector): Multi-vector signed integer multiply-add long long by vector.

SMLALL (multiple vectors): Multi-vector signed integer multiply-add long long.

SMLSL (multiple and indexed vector): Multi-vector signed 16-bit integer multiply-subtract by indexed element from 32-bit integer.

SMLSL (multiple and single vector): Multi-vector signed 16-bit integer multiply-subtract by vector from 32-bit integer.

SMLSL (multiple vectors): Multi-vector signed 16-bit integer multiply-subtract from 32-bit integer.

SMLSLL (multiple and indexed vector): Multi-vector signed integer multiply-subtract long long by indexed element.

SMLSLL (multiple and single vector): Multi-vector signed integer multiply-subtract long long by vector.

SMLSLL (multiple vectors): Multi-vector signed integer multiply-subtract long long.

SMOP4A (2-way): Signed 16-bit integer quarter-tile sum of outer products to 32-bit integer, accumulating.

SMOP4A (4-way): Signed integer quarter-tile sum of outer products, accumulating.

SMOP4S (2-way): Signed 16-bit integer quarter-tile sum of outer products to 32-bit integer, subtracting.

SMOP4S (4-way): Signed integer quarter-tile sum of outer products, subtracting.

SMOPA (2-way): Signed 16-bit integer sum of outer products to 32-bit integer, accumulating.

SMOPA (4-way): Signed integer sum of outer products, accumulating.

SMOPS (2-way): Signed 16-bit integer sum of outer products to 32-bit integer, subtracting.

SMOPS (4-way): Signed integer sum of outer products, subtracting.

SQCVT (four registers): Multi-vector signed saturating extract narrow.

SQCVT (two registers): Multi-vector signed 32-bit integer saturating extract narrow to 16-bit integer.

SQCVTN: Multi-vector signed saturating extract narrow to interleaved integer.

SQCVTU (four registers): Multi-vector signed saturating extract narrow to unsigned integer.

SQCVTU (two registers): Multi-vector signed 32-bit integer saturating extract narrow to unsigned 16-bit integer.

SQCVTUN: Multi-vector signed saturating extract narrow to interleaved unsigned integer.

SQDMULH (multiple and single vector): Multi-vector signed saturating doubling multiply high by vector.

SQDMULH (multiple vectors): Multi-vector signed saturating doubling multiply high.

SQRSHR (four registers): Multi-vector signed saturating rounding shift right narrow by immediate.

SQRSHR (two registers): Multi-vector signed 32-bit integer saturating rounding shift right narrow by immediate to 16-bit integer.

SQRSHRN: Multi-vector signed saturating rounding shift right narrow by immediate to interleaved integer.

SQRSHRU (four registers): Multi-vector signed saturating rounding shift right narrow by immediate to unsigned integer.

SQRSHRU (two registers): Multi-vector signed 32-bit integer saturating rounding shift right narrow by immediate to unsigned 16-bit integer.

SQRSHRUN: Multi-vector signed saturating rounding shift right narrow by immediate to interleaved unsigned integer.

SRSHL (multiple and single vector): Multi-vector signed rounding shift left by vector.

SRSHL (multiple vectors): Multi-vector signed rounding shift left.

ST1B (scalar plus immediate, strided registers): Contiguous store of bytes from multiple strided vectors (immediate index).

ST1B (scalar plus scalar, strided registers): Contiguous store of bytes from multiple strided vectors (scalar index).

ST1B (scalar plus scalar, tile slice): Contiguous store of bytes from 8-bit element ZA tile slice.

ST1D (scalar plus immediate, strided registers): Contiguous store of doublewords from multiple strided vectors (immediate index).

ST1D (scalar plus scalar, strided registers): Contiguous store of doublewords from multiple strided vectors (scalar index).

ST1D (scalar plus scalar, tile slice): Contiguous store of doublewords from 64-bit element ZA tile slice.

ST1H (scalar plus immediate, strided registers): Contiguous store of halfwords from multiple strided vectors (immediate index).

ST1H (scalar plus scalar, strided registers): Contiguous store of halfwords from multiple strided vectors (scalar index).

ST1H (scalar plus scalar, tile slice): Contiguous store of halfwords from 16-bit element ZA tile slice.

ST1Q: Contiguous store of quadwords from 128-bit element ZA tile slice.

ST1W (scalar plus immediate, strided registers): Contiguous store of words from multiple strided vectors (immediate index).

ST1W (scalar plus scalar, strided registers): Contiguous store of words from multiple strided vectors (scalar index).

ST1W (scalar plus scalar, tile slice): Contiguous store of words from 32-bit element ZA tile slice.

STMOPA (2-way): Signed 16-bit integer sparse sum of outer products to 32-bit integer, accumulating.

STMOPA (4-way): Signed 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

STNT1B (scalar plus immediate, strided registers): Contiguous store non-temporal of bytes from multiple strided vectors (immediate index).

STNT1B (scalar plus scalar, strided registers): Contiguous store non-temporal of bytes from multiple strided vectors (scalar index).

STNT1D (scalar plus immediate, strided registers): Contiguous store non-temporal of doublewords from multiple strided vectors (immediate index).

STNT1D (scalar plus scalar, strided registers): Contiguous store non-temporal of doublewords from multiple strided vectors (scalar index).

STNT1H (scalar plus immediate, strided registers): Contiguous store non-temporal of halfwords from multiple strided vectors (immediate index).

STNT1H (scalar plus scalar, strided registers): Contiguous store non-temporal of halfwords from multiple strided vectors (scalar index).

STNT1W (scalar plus immediate, strided registers): Contiguous store non-temporal of words from multiple strided vectors (immediate index).

STNT1W (scalar plus scalar, strided registers): Contiguous store non-temporal of words from multiple strided vectors (scalar index).

STR (array vector): Store ZA array vector.

STR (table): Store ZT0 register.

SUB (to array, array and multiple vectors): Multi-vector subtract from ZA array vectors.

SUB (to array, multiple and single vector): Multi-vector subtract by vector to ZA array vectors.

SUB (to array, multiple vectors): Multi-vector subtract to ZA array vectors.

SUDOT (4-way, multiple and indexed vector): Multi-vector signed by unsigned 8-bit integer dot product by indexed elements to 32-bit integer.

SUDOT (4-way, multiple and single vector): Multi-vector signed by unsigned 8-bit integer dot product by vector to 32-bit integer.

SUMLALL (multiple and indexed vector): Multi-vector signed by unsigned 8-bit integer multiply-add by indexed element to 32-bit integer.

SUMLALL (multiple and single vector): Multi-vector signed by unsigned 8-bit integer multiply-add by vector to 32-bit integer.

SUMOP4A: Signed by unsigned integer quarter-tile sum of outer products, accumulating.

SUMOP4S: Signed by unsigned integer quarter-tile sum of outer products, subtracting.

SUMOPA (4-way): Signed by unsigned integer sum of outer products, accumulating.

SUMOPS: Signed by unsigned integer sum of outer products, subtracting.

SUNPK: Unpack and sign-extend multi-vector elements.

SUTMOPA: Signed by unsigned 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

SUVDOT: Multi-vector signed by unsigned 8-bit integer vertical dot product by indexed element to 32-bit integer.

SVDOT (2-way): Multi-vector signed 16-bit integer vertical dot product by indexed element to 32-bit integer.

SVDOT (4-way): Multi-vector signed integer vertical dot product by indexed element.

UCLAMP: Multi-vector unsigned clamp to minimum/maximum.

UCVTF: Multi-vector unsigned 32-bit integer convert to single-precision.

UDOT (2-way, multiple and indexed vector): Multi-vector unsigned 16-bit integer dot product by indexed element to 32-bit integer.

UDOT (2-way, multiple and single vector): Multi-vector unsigned 16-bit integer dot product by vector to 32-bit integer.

UDOT (2-way, multiple vectors): Multi-vector unsigned 16-bit integer dot product to 32-bit integer.

UDOT (4-way, multiple and indexed vector): Multi-vector unsigned integer dot product by indexed element.

UDOT (4-way, multiple and single vector): Multi-vector unsigned integer dot product by vector.

UDOT (4-way, multiple vectors): Multi-vector unsigned integer dot product.

UMAX (multiple and single vector): Multi-vector unsigned maximum by vector.

UMAX (multiple vectors): Multi-vector unsigned maximum.

UMIN (multiple and single vector): Multi-vector unsigned minimum by vector.

UMIN (multiple vectors): Multi-vector unsigned minimum.

UMLAL (multiple and indexed vector): Multi-vector unsigned 16-bit integer multiply-add by indexed element to 32-bit integer.

UMLAL (multiple and single vector): Multi-vector unsigned 16-bit integer multiply-add by vector to 32-bit integer.

UMLAL (multiple vectors): Multi-vector unsigned 16-bit integer multiply-add to 32-bit integer.

UMLALL (multiple and indexed vector): Multi-vector unsigned integer multiply-add long long by indexed element.

UMLALL (multiple and single vector): Multi-vector unsigned integer multiply-add long long by vector.

UMLALL (multiple vectors): Multi-vector unsigned integer multiply-add long long.

UMLSL (multiple and indexed vector): Multi-vector unsigned 16-bit integer multiply-subtract by indexed element from 32-bit integer.

UMLSL (multiple and single vector): Multi-vector unsigned 16-bit integer multiply-subtract by vector from 32-bit integer.

UMLSL (multiple vectors): Multi-vector unsigned 16-bit integer multiply-subtract from 32-bit integer.

UMLSLL (multiple and indexed vector): Multi-vector unsigned integer multiply-subtract long long by indexed element.

UMLSLL (multiple and single vector): Multi-vector unsigned integer multiply-subtract long long by vector.

UMLSLL (multiple vectors): Multi-vector unsigned integer multiply-subtract long long.

UMOP4A (2-way): Unsigned 16-bit integer quarter-tile sum of outer products to 32-bit integer, accumulating.

UMOP4A (4-way): Unsigned integer quarter-tile sum of outer products, accumulating.

UMOP4S (2-way): Unsigned 16-bit integer quarter-tile sum of outer products to 32-bit integer, subtracting.

UMOP4S (4-way): Unsigned integer quarter-tile sum of outer products, subtracting.

UMOPA (2-way): Unsigned 16-bit integer sum of outer products to 32-bit integer, accumulating.

UMOPA (4-way): Unsigned integer sum of outer products, accumulating.

UMOPS (2-way): Unsigned 16-bit integer sum of outer products to 32-bit integer, subtracting.

UMOPS (4-way): Unsigned integer sum of outer products, subtracting.

UQCVT (four registers): Multi-vector unsigned saturating extract narrow.

UQCVT (two registers): Multi-vector unsigned 32-bit integer saturating extract narrow to 16-bit integer.

UQCVTN: Multi-vector unsigned saturating extract narrow to interleaved integer.

UQRSHR (four registers): Multi-vector unsigned saturating rounding shift right narrow by immediate.

UQRSHR (two registers): Multi-vector unsigned 32-bit integer saturating rounding shift right narrow by immediate to 16-bit integer.

UQRSHRN: Multi-vector unsigned saturating rounding shift right narrow by immediate to interleaved integer.

URSHL (multiple and single vector): Multi-vector unsigned rounding shift left by vector.

URSHL (multiple vectors): Multi-vector unsigned rounding shift left.

USDOT (4-way, multiple and indexed vector): Multi-vector unsigned by signed 8-bit integer dot product by indexed element to 32-bit integer.

USDOT (4-way, multiple and single vector): Multi-vector unsigned by signed 8-bit integer dot product by vector to 32-bit integer.

USDOT (4-way, multiple vectors): Multi-vector unsigned by signed 8-bit integer dot product to 32-bit integer.

USMLALL (multiple and indexed vector): Multi-vector unsigned by signed 8-bit integer multiply-add by indexed element to 32-bit integer.

USMLALL (multiple and single vector): Multi-vector unsigned by signed 8-bit integer multiply-add by vector to 32-bit integer.

USMLALL (multiple vectors): Multi-vector unsigned by signed 8-bit integer multiply-add to 32-bit integer.

USMOP4A: Unsigned by signed integer quarter-tile sum of outer products, accumulating.

USMOP4S: Unsigned by signed integer quarter-tile sum of outer products, subtracting.

USMOPA (4-way): Unsigned by signed integer sum of outer products, accumulating.

USMOPS: Unsigned by signed integer sum of outer products, subtracting.

USTMOPA: Unsigned by signed 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

USVDOT: Multi-vector unsigned by signed 8-bit integer vertical dot product by indexed element to 32-bit integer.

UTMOPA (2-way): Unsigned 16-bit integer sparse sum of outer products to 32-bit integer, accumulating.

UTMOPA (4-way): Unsigned 8-bit integer sparse sum of outer products to 32-bit integer, accumulating.

UUNPK: Unpack and zero-extend multi-vector elements.

UVDOT (2-way): Multi-vector unsigned 16-bit integer vertical dot product by indexed element to 32-bit integer.

UVDOT (4-way): Multi-vector unsigned integer vertical dot product by indexed element.

UZP (four registers): Concatenate elements from four vectors.

UZP (two registers): Concatenate elements from two vectors.

ZERO (double-vector): Zero ZA double-vector groups.

ZERO (quad-vector): Zero ZA quad-vector groups.

ZERO (single-vector): Zero ZA single-vector groups.

ZERO (table): Zero ZT0.

ZERO (tiles): Zero a list of 64-bit element ZA tiles.

ZIP (four registers): Interleave elements from four vectors.

ZIP (two registers): Interleave elements from two vectors.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

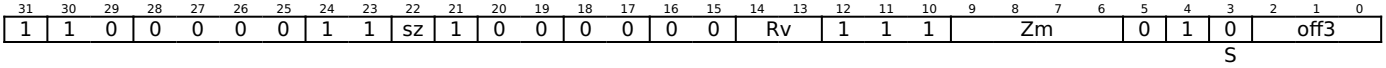
ADD (to array, array and multiple vectors)

Multi-vector accumulate to ZA array vectors

This instruction destructively adds all elements of the two or four source vectors to the corresponding elements of the ZA single-vector groups.
The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.
This instruction is unpredicated.
ID_AA64SMFR0_EL1.I16I64 indicates whether the 64-bit integer variant is implemented.
It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

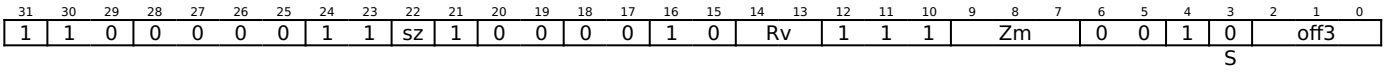
```
ADD ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
ADD ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zm1> For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zm4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = ZAvector{VL}(vec);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = element1 + element2;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADD (to array, multiple and single vector)

Multi-vector add by vector to ZA array vectors

This instruction adds all the corresponding elements of the second source vector and the two or four first source vectors and places the results in the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

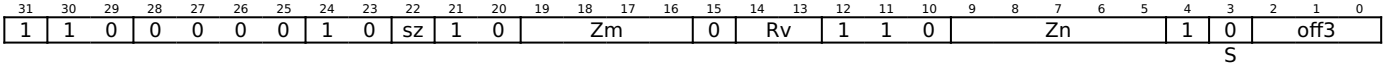
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 64-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

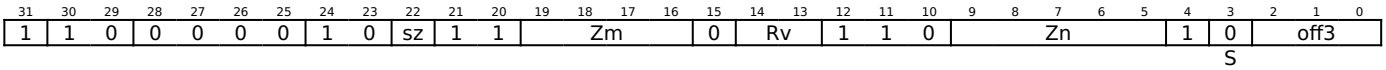
```
ADD ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<T>-<Zn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
ADD ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<T>-<Zn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = element1 + element2;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADD (to array, multiple vectors)

Multi-vector add to ZA array vectors

This instruction adds all the corresponding elements of the two or four second source vectors and first source vectors and places the results in the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

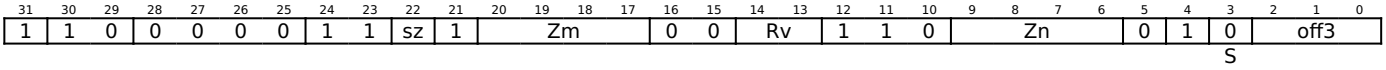
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 64-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

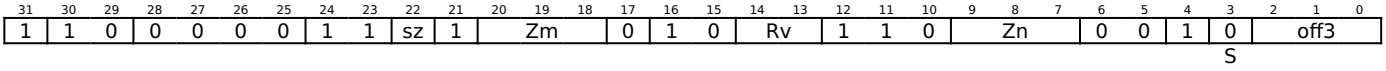
```
ADD ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<T>-<Zn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
ADD ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<T>-<Zn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = element1 + element2;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADD (to vector, multiple vectors)

Multi-vector add by vector to multi-vector

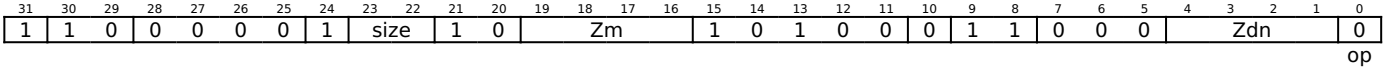
This instruction adds elements of the second source vector to the corresponding elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

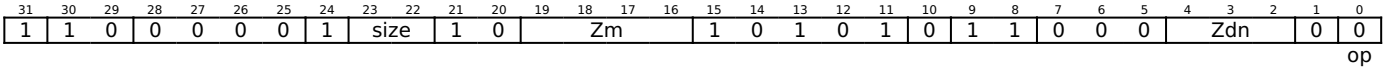
```
ADD { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
ADD { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = element1 + element2;
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDHA

Add horizontally vector elements to ZA tile

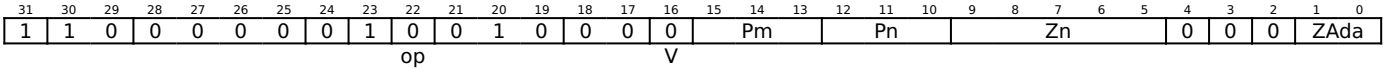
This instruction adds each element of the source vector to the corresponding Active element of each horizontal slice of a ZA tile. The tile elements are predicated by a pair of governing predicates. An element of a horizontal slice is considered Active if its corresponding element in the second governing predicate is TRUE and the element corresponding to its horizontal slice number in the first governing predicate is TRUE. Inactive elements in the destination tile remain unmodified.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 64-bit integer variant is implemented.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SME)



Encoding

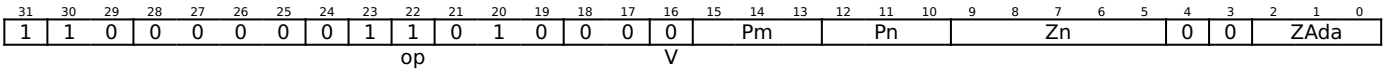
```
ADDHA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let da : integer = UInt(ZAda);
```

64-bit

(FEAT_SME_I16I64)



Encoding

```
ADDHA <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand_src : bits(VL) = Z{VL}(n);
let operand_acc : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);

for col = 0 to dim-1 do
    let element : bits(esize) = operand_src[col*:esize];
    for row = 0 to dim-1 do
        var res : bits(esize) = operand_acc[(row*dim+col)*:esize];
        if (ActivePredicateElement{PL}(mask1, row, esize) &&
              ActivePredicateElement{PL}(mask2, col, esize)) then
            res = res + element;
        end;
        result[(row*dim+col)*:esize] = res;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDSPL

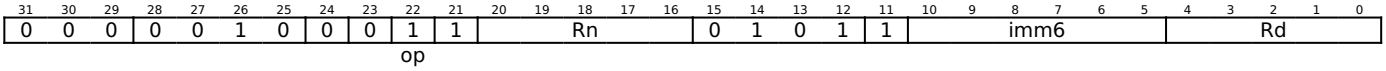
Add multiple of Streaming SVE predicate register size to scalar register

This instruction adds the Streaming SVE predicate register size in bytes multiplied by an immediate in the range -32 to 31 to the 64-bit source general-purpose register or current stack pointer and places the result in the 64-bit destination general-purpose register or current stack pointer.

This instruction does not require the PE to be in Streaming SVE mode.

SME

(FEAT_SME)



Encoding

```
ADDSPL <Xd|SP>, <Xn|SP>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let d : integer = UInt(Rd);
let imm : integer = SInt(imm6);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.
<imm>	Is the signed immediate operand, in the range -32 to 31, encoded in the "imm6" field.

Operation

```
CheckSMEEnabled();
let SVL : integer = CurrentSVL();
let len : integer = imm * (SVL DIV 64);
let operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let result : bits(64) = operand1 + len;

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDSVL

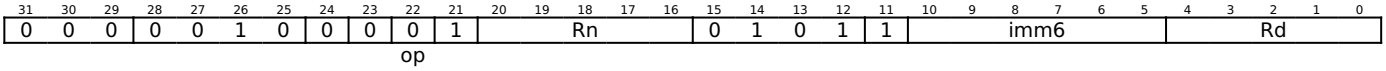
Add multiple of Streaming SVE vector register size to scalar register

This instruction adds the Streaming SVE vector register size in bytes multiplied by an immediate in the range -32 to 31 to the 64-bit source general-purpose register or current stack pointer, and places the result in the 64-bit destination general-purpose register or current stack pointer.

This instruction does not require the PE to be in Streaming SVE mode.

SME

(FEAT_SME)



Encoding

```
ADDSVL <Xd|SP>, <Xn|SP>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let d : integer = UInt(Rd);
let imm : integer = SInt(imm6);
```

Assembler Symbols

<Xd|SP>	Is the 64-bit name of the destination general-purpose register or stack pointer, encoded in the "Rd" field.
<Xn|SP>	Is the 64-bit name of the source general-purpose register or stack pointer, encoded in the "Rn" field.
<imm>	Is the signed immediate operand, in the range -32 to 31, encoded in the "imm6" field.

Operation

```
CheckSMEEnabled();
let SVL : integer = CurrentSVL();
let len : integer = imm * (SVL DIV 8);
let operand1 : bits(64) = if n == 31 then SP{64}() else X{64}(n);
let result : bits(64) = operand1 + len;

if d == 31 then
    SP{64}() = result;
else
    X{64}(d) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ADDVA

Add vertically vector elements to ZA tile

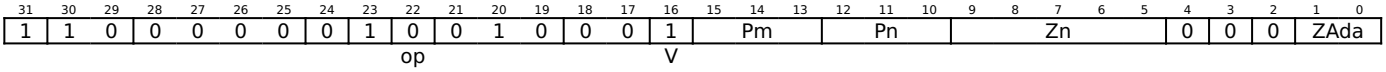
This instruction adds each element of the source vector to the corresponding Active element of each vertical slice of a ZA tile. The tile elements are predicated by a pair of governing predicates. An element of a vertical slice is considered Active if its corresponding element in the first governing predicate is TRUE and the element corresponding to its vertical slice number in the second governing predicate is TRUE. Inactive elements in the destination tile remain unmodified.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 64-bit integer variant is implemented.

It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SME)



Encoding

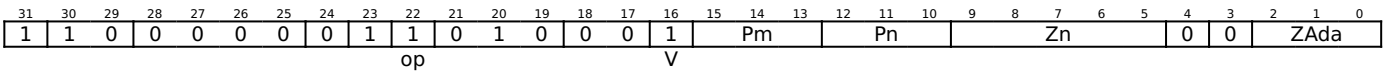
```
ADDVA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let da : integer = UInt(ZAda);
```

64-bit

(FEAT_SME_I16I64)



Encoding

```
ADDVA <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand_src : bits(VL) = Z{VL}(n);
let operand_acc : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);

for row = 0 to dim-1 do
    let element : bits(esize) = operand_src[row*:esize];
    for col = 0 to dim-1 do
        var res : bits(esize) = operand_acc[(row*dim+col)*:esize];
        if (ActivePredicateElement{PL}(mask1, row, esize) &&
              ActivePredicateElement{PL}(mask2, col, esize)) then
            res = res + element;
        end;
        result[(row*dim+col)*:esize] = res;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

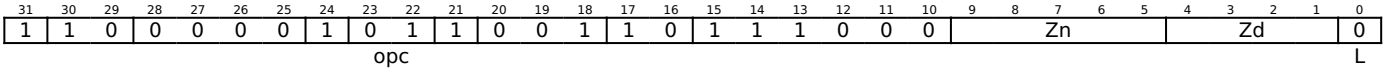
BF1CVT, BF2CVT

Multi-vector 8-bit floating-point convert to BFloat16

This instruction converts each 8-bit floating-point element of the source vector to BFloat16 while downscaling the value, and places the results in the corresponding 16-bit elements of the destination vectors. BF1CVT scales the values by $2^{-UInt(FPMR.LSCALE[5:0])}$. BF2CVT scales the values by $2^{-UInt(FPMR.LSCALE2[5:0])}$. The 8-bit floating-point encoding format for BF1CVT is selected by FPMR.F8S1. The 8-bit floating-point encoding format for BF2CVT is selected by FPMR.F8S2.
This instruction is unpredicated.
It has encodings from 2 classes: BF1CVT and BF2CVT

BF1CVT

(FEAT_SME2 && FEAT_FP8)



Encoding

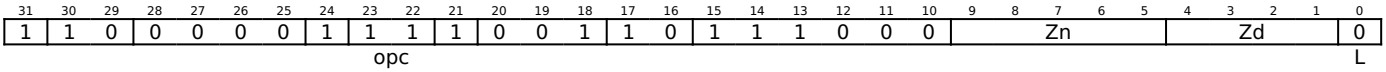
```
BF1CVT { <Zd1>.H-<Zd2>.H }, <Zn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
let issrc2 : boolean = FALSE;
```

BF2CVT

(FEAT_SME2 && FEAT_FP8)



Encoding

```
BF2CVT { <Zd1>.H-<Zd2>.H }, <Zn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
let issrc2 : boolean = TRUE;
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 8;
let operand : bits(VL) = Z{VL}(n);
var result : bits(2*VL);

for e = 0 to elements-1 do
    let element : bits(8) = operand[e*:8];
    result[e*:16] = FP8ConvertBF(element, issrc2, FPCR(), FPMR());
end;

Z{VL}(d+0) = result[VL-1:0];
Z{VL}(d+1) = result[2*VL-1:VL];
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

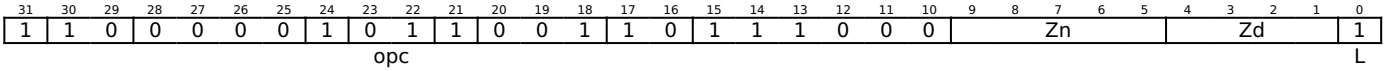
BF1CVTL, BF2CVTL

Multi-vector 8-bit floating-point convert to deinterleaved BFloat16

This instruction converts each 8-bit floating-point element of the source vector to BFloat16 while downscaling the value, and places the two-way deinterleaved results in the corresponding 16-bit elements of the destination vectors. BF1CVTL scales the values by $2^{-UInt(FPMR.LSCALE[5:0])}$. BF2CVTL scales the values by $2^{-UInt(FPMR.LSCALE2[5:0])}$. The 8-bit floating-point encoding format for BF1CVTL is selected by FPMR.F8S1. The 8-bit floating-point encoding format for BF2CVTL is selected by FPMR.F8S2.
This instruction is unpredicated.
It has encodings from 2 classes: BF1CVTL and BF2CVTL

BF1CVTL

(FEAT_SME2 && FEAT_FP8)



Encoding

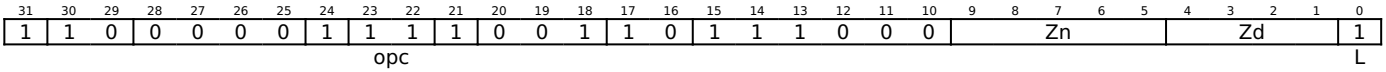
```
BF1CVTL { <Zd1>.H-<Zd2>.H }, <Zn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
let issrc2 : boolean = FALSE;
```

BF2CVTL

(FEAT_SME2 && FEAT_FP8)



Encoding

```
BF2CVTL { <Zd1>.H-<Zd2>.H }, <Zn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
let issrc2 : boolean = TRUE;
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV 16;
let operand : bits(VL) = Z{VL}(n);
var result1 : bits(VL);
var result2 : bits(VL);

for p = 0 to pairs-1 do
    let element1 : bits(8) = operand[(2*p + 0)*:8];
    let element2 : bits(8) = operand[(2*p + 1)*:8];
    result1[p*:16] = FP8ConvertBF(element1, issrc2, FPCR(), FPMR());
    result2[p*:16] = FP8ConvertBF(element2, issrc2, FPCR(), FPMR());
end;

Z{VL}(d+0) = result1;
Z{VL}(d+1) = result2;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFADD

Multi-vector BFloat16 accumulate to ZA array vectors

This instruction destructively adds all elements of the two or four source vectors to the corresponding BFloat16 elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.

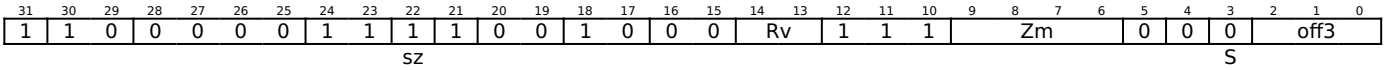
This instruction is unpredicated.

ID_AA64SMFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_B16B16)



Encoding

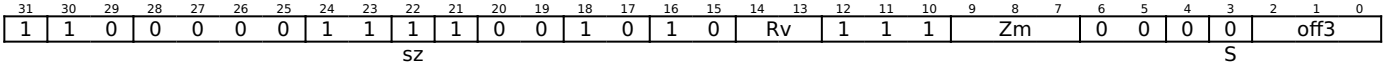
```
BFADD ZA.H[<Wv>, <offs>{, VGx2}], { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let m : integer{} = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_B16B16)



Encoding

```
BFADD ZA.H[<Wv>, <offs>{, VGx4}], { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let m : integer{} = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zm4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = ZAvector{VL}(vec);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFAdd_ZA(element1, element2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFCLAMP

Multi-vector BFloat16 clamp to minimum/maximum number

This instruction clamps each BFloat16 element in the two or four destination vectors to between the BFloat16 minimum value in the corresponding element of the first source vector and the BFloat16 maximum value in the corresponding element of the second source vector and destructively places the clamped results in the corresponding elements of the two or four destination vectors.

The behavior is as follows for each minimum number and maximum number operation:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

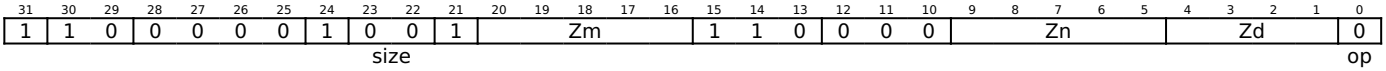
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

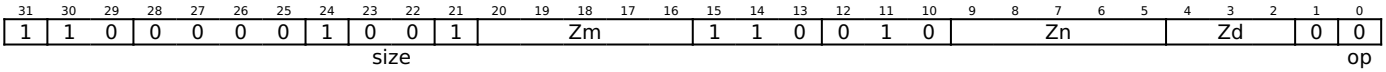
```
BFCLAMP { <Zd1>.H-<Zd2>.H }, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFCLAMP { <Zd1>.H-<Zd4>.H }, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = Z{VL}(d+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        let element3 : bits(16) = operand3[e*:16];
        let maxelement : bits(16) = BFMaxNum{16}(element1, element3, FPCR());
        results[[r]][e*:16] = BFMinNum{16}(maxelement, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

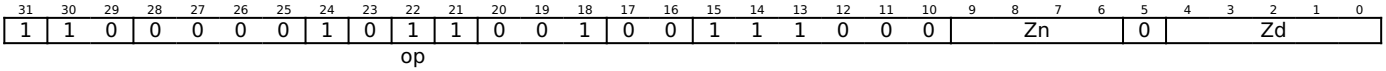
BFCVT (BFloat16 to 8-bit floating-point)

Multi-vector BFloat16 convert to 8-bit floating-point

This instruction converts each BFloat16 element of the two source vectors to 8-bit floating-point while scaling the value by $2^{SInt(FPMR.NSCALE)}$, and places the results in the half-width elements of the destination vector. The 8-bit floating-point encoding format is selected by FPMR.F8D.
This instruction is unpredicated.

SME2

(FEAT_SME2 && FEAT_FP8)



Encoding

```
BFCVT <Zd>.B, { <Zn1>.H-<Zn2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    let element2 : bits(16) = operand2[e*:16];
    result[(0*elements + e)*:8] = BFConvertFP8(element1, FPCR(), FPMR());
    result[(1*elements + e)*:8] = BFConvertFP8(element2, FPCR(), FPMR());
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFCVT (single-precision to BFloat16)

Multi-vector single-precision convert to BFloat16

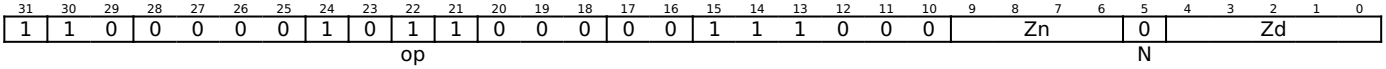
This instruction converts each element of the two source vectors from single-precision to BFloat16 floating-point, and places the results in the half-width destination elements.

This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
BFCVT <Zd>.H, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
for e = 0 to elements-1 do
    let element1 : bits(32) = operand1[e*:32];
    let element2 : bits(32) = operand2[e*:32];
    let res1 : bits(16) = FPConvertBF(element1, FPCR());
    let res2 : bits(16) = FPConvertBF(element2, FPCR());
    result[e*:16] = res1;
    result[(elements+e)*:16] = res2;
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFCVTN

Multi-vector single-precision convert to interleaved BFloat16

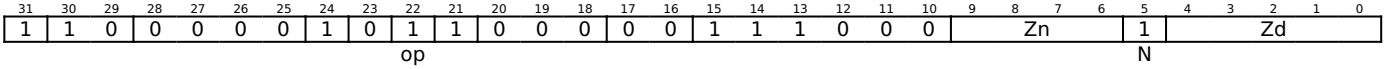
This instruction converts each element of the two source vectors from single-precision to BFloat16 floating-point, and places the two-way interleaved results in the half-width destination elements.

This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
BFCVTN <Zd>.H, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
for e = 0 to elements-1 do
    let element1 : bits(32) = operand1[e*:32];
    let element2 : bits(32) = operand2[e*:32];
    let res1 : bits(16) = FPConvertBF(element1, FPCR());
    let res2 : bits(16) = FPConvertBF(element2, FPCR());
    result[(2*e + 0)*:16] = res1;
    result[(2*e + 1)*:16] = res2;
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFDOT (multiple and indexed vector)

Multi-vector BFloat16 dot product by indexed element to single-precision

This instruction calculates the dot product of a pair of BFloat16 values held in the corresponding 32-bit elements of the two or four first source vectors and the indexed 32-bit element of the second source vector. The single-precision dot product results are destructively added to the corresponding single-precision elements of the ZA single-vector groups.

The BFloat16 pairs within the second source vector are specified using an immediate index that selects the same BFloat16 pair position within each 128-bit vector segment. The element index range is from 0 to 3.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

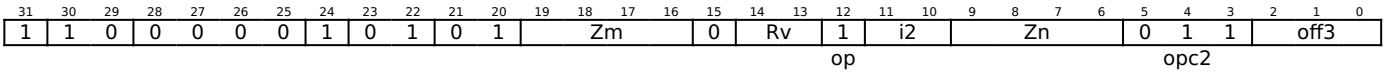
This instruction follows SME2 ZA-targeting BFloat16 numerical behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

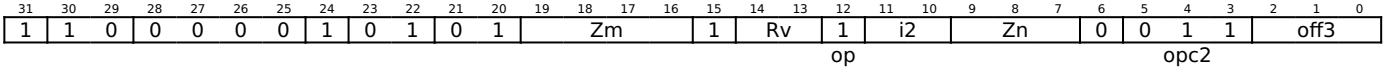
```
BFDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
BFDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<index> Is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let segmentsize : integer{} = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elt1_a : bits(hsize) = operand1[(2 * e + 0)*:hsize];
        let elt1_b : bits(hsize) = operand1[(2 * e + 1)*:hsize];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let elt2_a : bits(hsize) = operand2[(2 * s + 0)*:hsize];
        let elt2_b : bits(hsize) = operand2[(2 * s + 1)*:hsize];
        var sum : bits(esize) = operand3[e*:esize];
        sum = BFDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFDOT (multiple and single vector)

Multi-vector BFloat16 dot product by vector to single-precision

This instruction calculates the dot product of a pair of BFloat16 values held in the corresponding 32-bit elements of the two or four first source vectors and the second source vector. The single-precision dot product results are destructively added to the corresponding single-precision elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

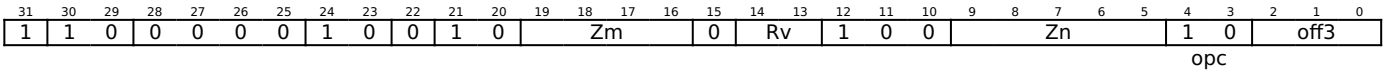
This instruction follows SME2 ZA-targeting BFloat16 numerical behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

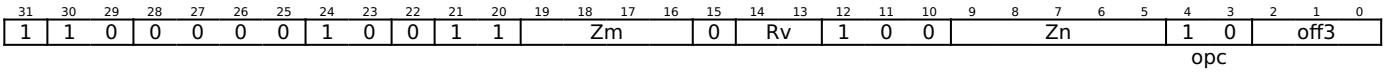
```
BFDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
BFDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elt1_a : bits(hsize) = operand1[(2 * e + 0)*:hsize];
        let elt1_b : bits(hsize) = operand1[(2 * e + 1)*:hsize];
        let elt2_a : bits(hsize) = operand2[(2 * e + 0)*:hsize];
        let elt2_b : bits(hsize) = operand2[(2 * e + 1)*:hsize];
        var sum : bits(esize) = operand3[e*:esize];
        sum = BFDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFDOT (multiple vectors)

Multi-vector BFloat16 dot product to single-precision

This instruction calculates the dot product of a pair of BFloat16 values held in the corresponding 32-bit elements of the two or four first and second source vectors. The single-precision dot product results are destructively added to the corresponding single-precision elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

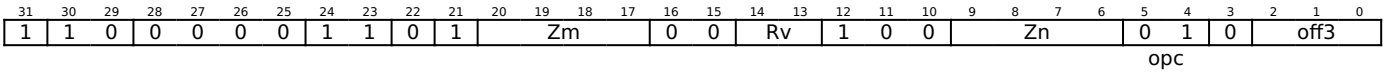
This instruction follows SME2 ZA-targeting BFloat16 numerical behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

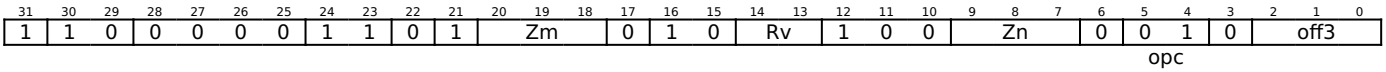
```
BFDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
BFDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elt1_a : bits(hsize) = operand1[(2 * e + 0)*:hsize];
        let elt1_b : bits(hsize) = operand1[(2 * e + 1)*:hsize];
        let elt2_a : bits(hsize) = operand2[(2 * e + 0)*:hsize];
        let elt2_b : bits(hsize) = operand2[(2 * e + 1)*:hsize];
        var sum : bits(esize) = operand3[e*:esize];
        sum = BFDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMAX (multiple and single vector)

Multi-vector BFloat16 maximum by vector

This instruction determines the maximum of BFloat16 elements of the second source vector and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

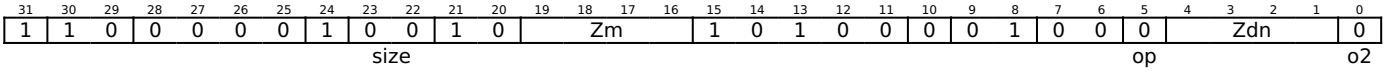
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

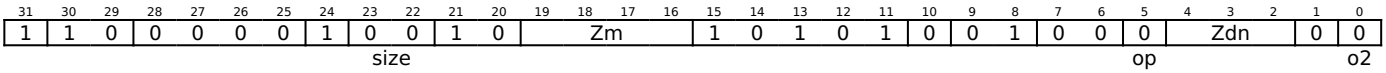
```
BFMAX { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMAX { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMax{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMAX (multiple vectors)

Multi-vector BFloat16 maximum

This instruction determines the maximum of BFloat16 elements of the two or four second source vectors and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

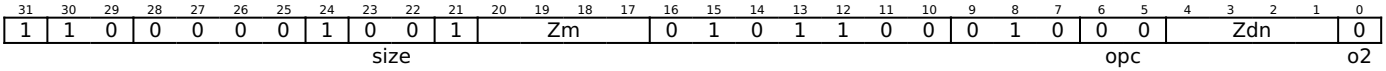
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

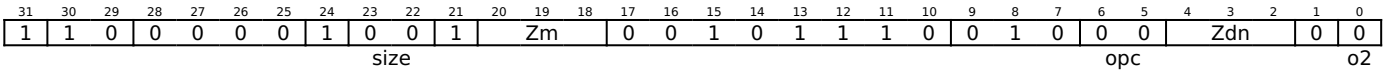
```
BFMAX { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMAX { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

	For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMax{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMAXNM (multiple and single vector)

Multi-vector BFloat16 maximum number by vector

This instruction determines the maximum number value of BFloat16 elements of the second source vector and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

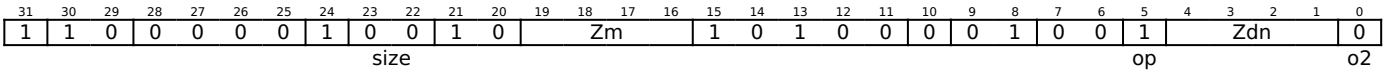
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

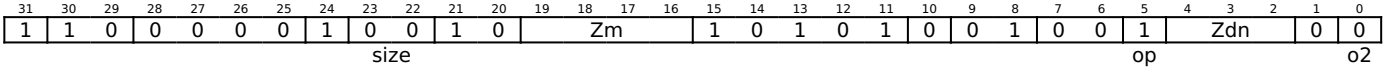
```
BFMAXNM { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMAXNM { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMaxNum{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMAXNM (multiple vectors)

Multi-vector BFloat16 maximum number

This instruction determines the maximum number value of BFloat16 elements of the two or four second source vectors and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

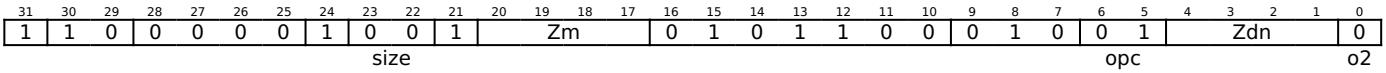
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

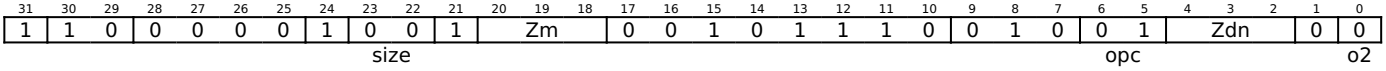
```
BFMAXNM { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMAXNM { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

	For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMaxNum{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMIN (multiple and single vector)

Multi-vector BFloat16 minimum by vector

This instruction determines the minimum of BFloat16 elements of the second source vector and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

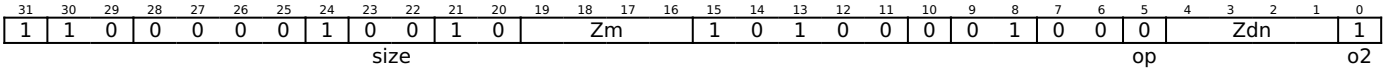
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

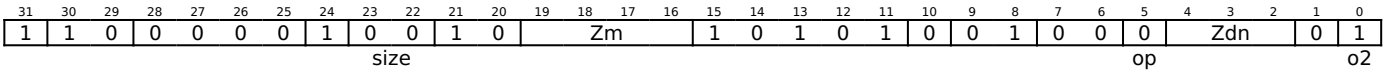
```
BFMIN { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMIN { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMin{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMIN (multiple vectors)

Multi-vector BFloat16 minimum

This instruction determines the minimum of BFloat16 elements of the two or four second source vectors and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

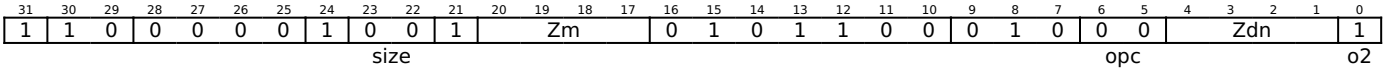
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

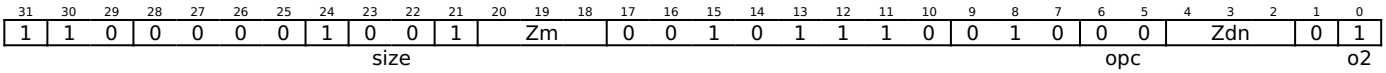
```
BFMIN { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMIN { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

	For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMin{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMINNM (multiple and single vector)

Multi-vector BFloat16 minimum number by vector

This instruction determines the minimum number value of BFloat16 elements of the second source vector and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

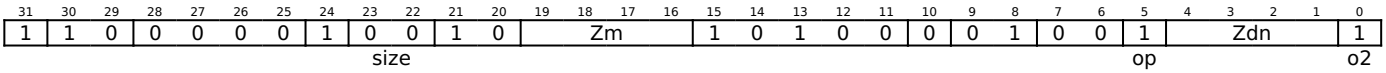
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

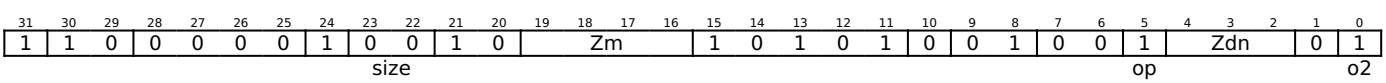
```
BFMINNM { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMINNM { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMinNum{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMINNM (multiple vectors)

Multi-vector BFloat16 minimum number

This instruction determines the minimum number value of BFloat16 elements of the two or four second source vectors and the corresponding BFloat16 elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

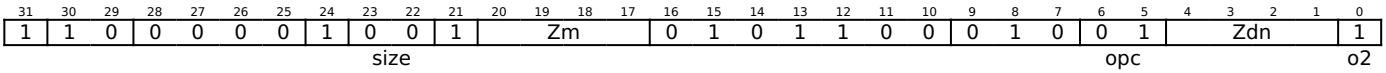
This instruction is unpredicated.

ID_AA64ZFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

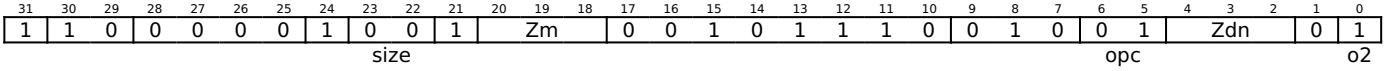
```
BFMINNM { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_B16B16)



Encoding

```
BFMINNM { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

	For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMinNum{16}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLA (multiple and indexed vector)

Multi-vector BFloat16 fused multiply-add by indexed element

This instruction multiplies the indexed element of the second source vector by the corresponding BFloat16 elements of the two or four first source vectors and destructively adds these values without intermediate rounding to the corresponding elements of the ZA single-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to 7.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.

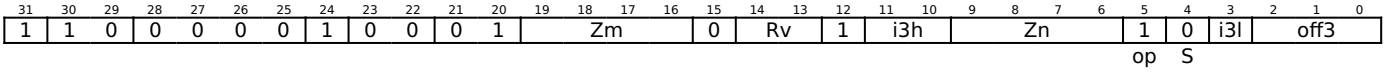
This instruction is unpredicated.

ID_AA64SMFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_B16B16)



Encoding

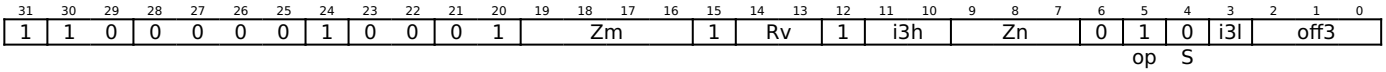
```
BFMLA ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_B16B16)



Encoding

```
BFMLA ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let segmentsize : integer{} = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = op1[e*:esize];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let elem2 : bits(esize) = op2[s*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = BFMulAdd_ZA(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLA (multiple and single vector)

Multi-vector BFloat16 fused multiply-add by vector

This instruction multiplies the corresponding BFloat16 elements of the two or four first source vector with the corresponding elements of the second source vector and destructively adds these values without intermediate rounding to the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.

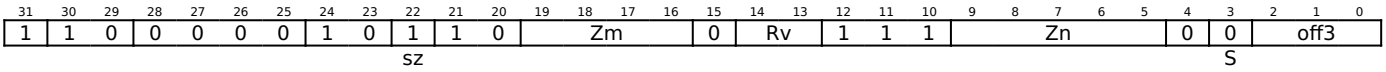
This instruction is unpredicated.

ID_AA64SMFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_B16B16)



Encoding

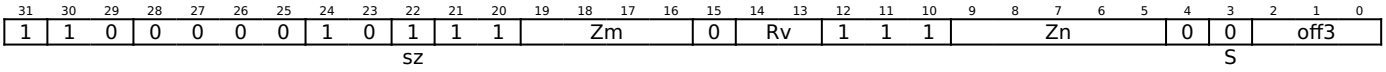
```
BFMLA ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_B16B16)



Encoding

```
BFMLA ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let op2 : bits(VL) = Z{VL}(m);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = op1[e*:esize];
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = BFMulAdd_ZA(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLA (multiple vectors)

Multi-vector BFloat16 fused multiply-add

This instruction multiplies the corresponding BFloat16 elements of the two or four first and second source vectors and destructively adds these values without intermediate rounding to the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.

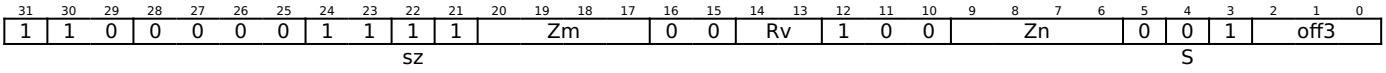
This instruction is unpredicated.

ID_AA64SMFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_B16B16)



Encoding

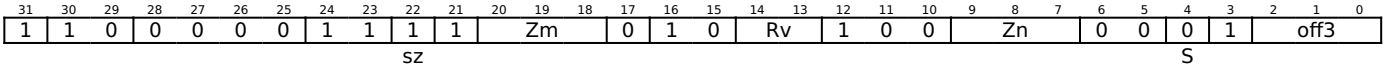
```
BFMLA ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_B16B16)



Encoding

```
BFMLA ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m+r);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = op1[e*:esize];
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = BFMulAdd_ZA(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLAL (multiple and indexed vector)

Multi-vector BFloat16 multiply-add by indexed element to single-precision

This instruction widens all 16-bit BFloat16 elements in the one, two, or four first source vectors and the indexed element of the second source vector to single-precision format, then multiplies the corresponding elements and destructively adds these values without intermediate rounding to the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The BFloat16 elements within the second source vector are specified using a 3-bit immediate index that selects the same element position within each 128-bit vector segment.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

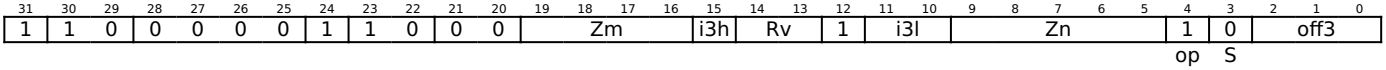
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

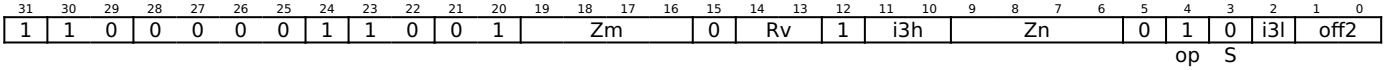
```
BFMLAL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

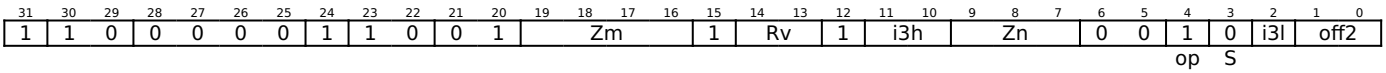
```
BFMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
BFMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let segmentsize : integer{} = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(hsize) = op1[(2 * e + i)*:hsize];
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 2 * segmentbase + index;
            let elem2 : bits(hsize) = op2[s*:hsize];
            let elem3 : bits(esize) = op3[e*:esize];
            result[e*:esize] = BFMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLAL (multiple and single vector)

Multi-vector BFloat16 multiply-add by vector to single-precision

This instruction widens all 16-bit BFloat16 elements in the one, two, or four first source vectors and the second source vector to single-precision format, then multiplies the corresponding elements and destructively adds these values without intermediate rounding to the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

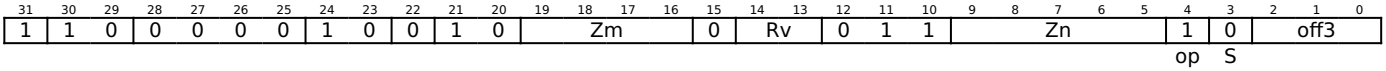
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

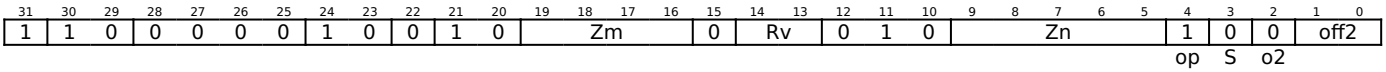
```
BFMLAL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let nreg : integer{} = 1;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

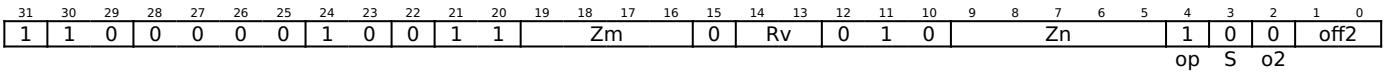
```
BFMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
BFMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let op2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(hsize) = op1[(2 * e + i)*:hsize];
            let elem2 : bits(hsize) = op2[(2 * e + i)*:hsize];
            let elem3 : bits(esize) = op3[e*:esize];
            result[e*:esize] = BFMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLAL (multiple vectors)

Multi-vector BFloat16 multiply-add to single-precision

This instruction widens all 16-bit BFloat16 elements in the two or four first and second source vectors to single-precision format, then multiplies the corresponding elements and destructively adds these values without intermediate rounding to the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The double-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

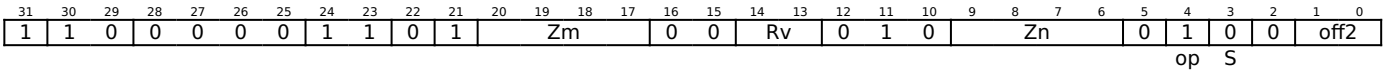
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA double-vectors and Four ZA double-vectors

Two ZA double-vectors

(FEAT_SME2)



Encoding

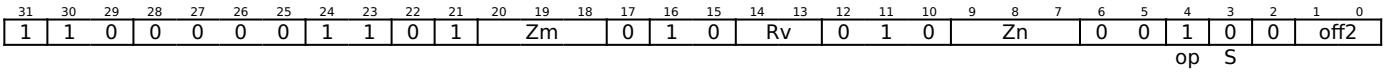
```
BFMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
BFMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "off2" field times 2.
<offs2>	Is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

	For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(hsize) = op1[(2 * e + i)*:hsize];
            let elem2 : bits(hsize) = op2[(2 * e + i)*:hsize];
            let elem3 : bits(esize) = op3[e*:esize];
            result[e*:esize] = BFMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLS (multiple and indexed vector)

Multi-vector BFloat16 fused multiply-subtract by indexed element

This instruction multiplies the indexed element of the second source vector by the corresponding BFloat16 elements of the two or four first source vectors and destructively subtracts these values without intermediate rounding from the corresponding elements of the ZA single-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to 7.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.

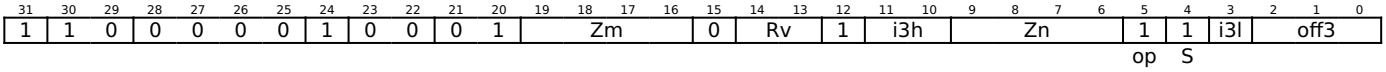
This instruction is unpredicated.

ID_AA64SMFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_B16B16)



Encoding

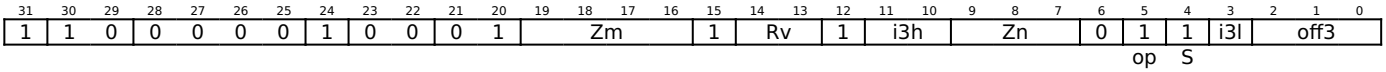
```
BFMLS ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_B16B16)



Encoding

```
BFMLS ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let segmentsize : integer{} = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = BFNeg(op1[e*:esize]);
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let elem2 : bits(esize) = op2[s*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = BFMulAdd_ZA(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLS (multiple and single vector)

Multi-vector BFloat16 fused multiply-subtract by vector

This instruction multiplies the corresponding BFloat16 elements of the two or four first source vector with the corresponding elements of the second source vector and destructively subtracts these values without intermediate rounding from the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.

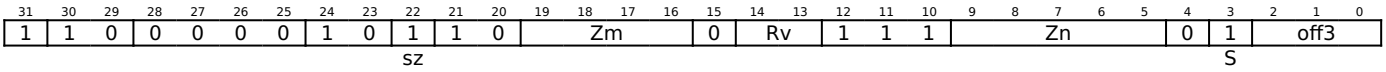
This instruction is unpredicated.

ID_AA64SMFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_B16B16)



Encoding

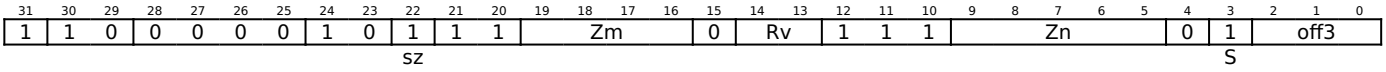
```
BFMLS ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_B16B16)



Encoding

```
BFMLS ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let op2 : bits(VL) = Z{VL}(m);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = BFNeg(op1[e*:esize]);
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = BFMulAdd_ZA(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLS (multiple vectors)

Multi-vector BFloat16 fused multiply-subtract

This instruction multiplies the corresponding BFloat16 elements of the two or four first and second source vectors and destructively subtracts these values without intermediate rounding from the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.

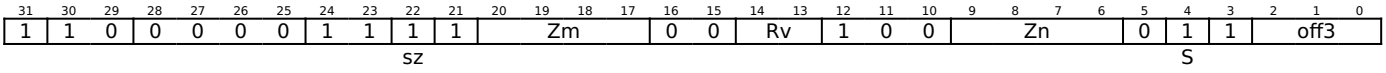
This instruction is unpredicated.

ID_AA64SMFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_B16B16)



Encoding

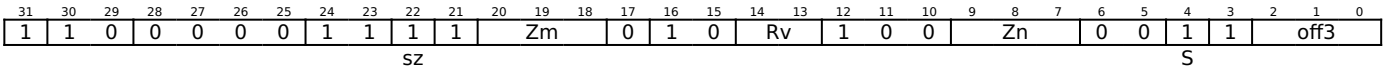
```
BFMLS ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_B16B16)



Encoding

```
BFMLS ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m+r);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = BFNeg(op1[e*:esize]);
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = BFMulAdd_ZA(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLSL (multiple and indexed vector)

Multi-vector BFloat16 multiply-subtract by indexed element from single-precision

This instruction widens all 16-bit BFloat16 elements in the one, two, or four first source vectors and the indexed element of the second source vector to single-precision format, then multiplies the corresponding elements and destructively subtracts these values without intermediate rounding from the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The BFloat16 elements within the second source vector are specified using a 3-bit immediate index that selects the same element position within each 128-bit vector segment.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

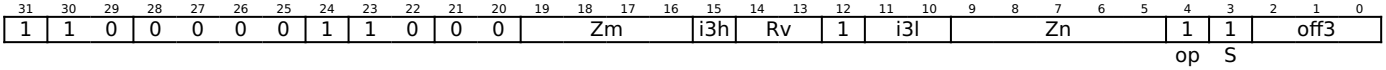
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

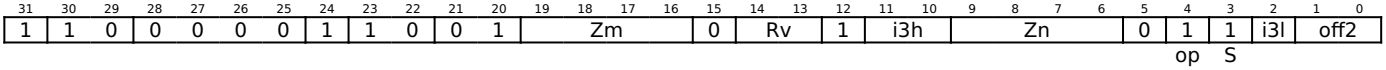
```
BFMLSL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

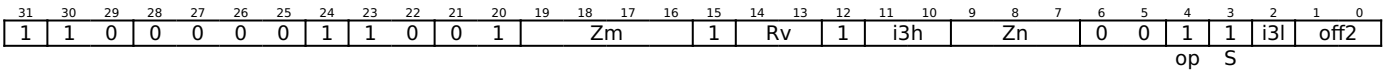
```
BFMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
BFMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let segmentsize : integer{} = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(hsize) = BFNeg(op1[(2 * e + i)*:hsize]);
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 2 * segmentbase + index;
            let elem2 : bits(hsize) = op2[s*:hsize];
            let elem3 : bits(esize) = op3[e*:esize];
            result[e*:esize] = BFMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLSL (multiple and single vector)

Multi-vector BFloat16 multiply-subtract by vector from single-precision

This instruction widens all 16-bit BFloat16 elements in the one, two, or four first source vectors and the second source vector to single-precision format, then multiplies the corresponding elements and destructively subtracts these values without intermediate rounding from the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

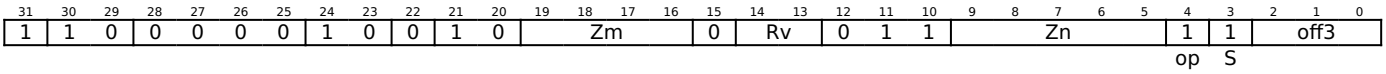
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

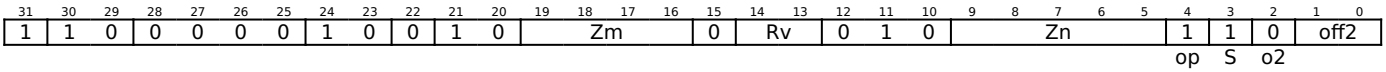
```
BFMLSL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let nreg : integer{} = 1;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

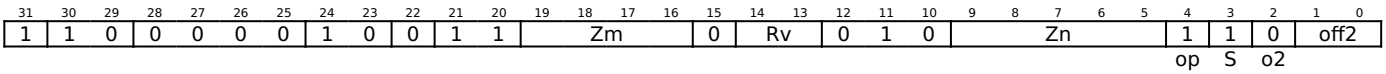
```
BFMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
BFMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let op2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(hsize) = BFNeg(op1[(2 * e + i)*:hsize]);
            let elem2 : bits(hsize) = op2[(2 * e + i)*:hsize];
            let elem3 : bits(esize) = op3[e*:esize];
            result[e*:esize] = BFMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMLSL (multiple vectors)

Multi-vector BFloat16 multiply-subtract from single-precision

This instruction widens all 16-bit BFloat16 elements in the two or four first and second source vectors to single-precision format, then multiplies the corresponding elements and destructively subtracts these values without intermediate rounding from the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The double-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

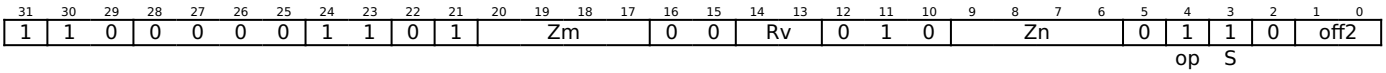
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA double-vectors and Four ZA double-vectors

Two ZA double-vectors

(FEAT_SME2)



Encoding

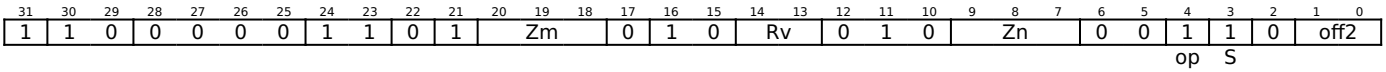
```
BFMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
BFMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "off2" field times 2.
<offs2>	Is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

	For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(hsize) = BFNeg(op1[(2 * e + i)*:hsize]);
            let elem2 : bits(hsize) = op2[(2 * e + i)*:hsize];
            let elem3 : bits(esize) = op3[e*:esize];
            result[e*:esize] = BFMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMOP4A (non-widening)

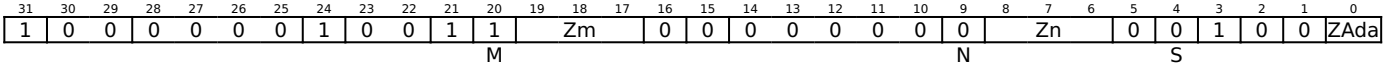
BFloat16 quarter-tile outer product, accumulating

This instruction generates four independent quarter-tile BFloat16 outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 16-bit element ZA tile.
Each of the quarter-tile outer products is generated by multiplying the $SVL_H \div 2 \times 1$ sub-matrix of BFloat16 values held in the half-vectors of the first source vectors by the $1 \times SVL_H \div 2$ sub-matrix of BFloat16 values held in the half-vectors of the second source vectors.

The resulting quarter-tile $SVL_H \div 2 \times SVL_H \div 2$ BFloat16 outer products are destructively added to the destination ZA tile. This is equivalent to performing a single multiply-accumulate to each of the destination tile elements.
This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.
This instruction is unpredicated.
It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_B16B16)



Encoding

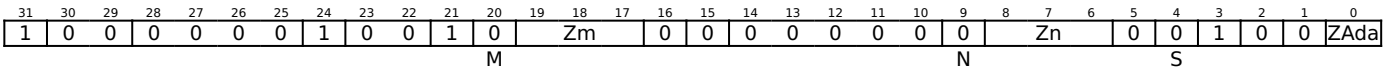
```
BFMOP4A <ZAda>.H, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Single vectors

(FEAT_SME_MOP4 && FEAT_SME_B16B16)



Encoding

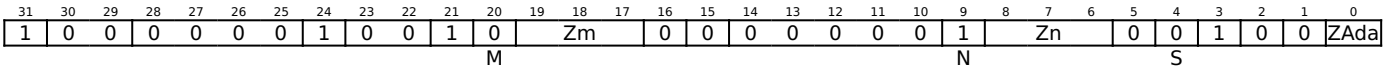
```
BFMOP4A <ZAda>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_B16B16)



Encoding

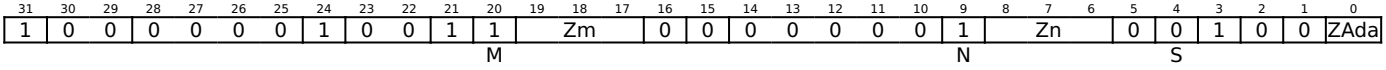
```
BFMOP4A <ZAda>.H, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_B16B16)



Encoding

```
BFMOP4A <ZAda>.H, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV 16;
let tilesize : integer{} = 4 * dim * dim * 16;
let op3 : bits(tilesize) = ZAtile{tilesize}(da, 16);
var result : bits(tilesize);

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{VL}(n + (nreg-1) * col_hv);
    let op2 : bits(VL) = Z{VL}(m + (mreg-1) * row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            let elem1 : bits(16) = op1[row_idx*:16];
            let elem2 : bits(16) = op2[col_idx*:16];
            let elem3 : bits(16) = op3[tile_idx*:16];
            result[tile_idx*:16] = BFMulAdd_ZA(elem3, elem1, elem2, FPCR());
        end;
    end;
end;
ZAtile{tilesize}(da, 16) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMOP4A (widening)

BFloat16 quarter-tile sum of outer products to single-precision, accumulating

This instruction generates four independent quarter-tile BFloat16 sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

Each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 2$ sub-matrix of BFloat16 values held in the half-vectors of the first source vectors by the $2 \times SVL_S \div 2$ sub-matrix of BFloat16 values held in the half-vectors of the second source vectors. Each 32-bit container of the first source vectors holds 2 elements of each row of a $SVL_S \div 2 \times 2$ sub-matrix. Similarly, each 32-bit container of the second source vectors holds 2 elements of each column of a $2 \times SVL_S \div 2$ sub-matrix.

The instruction widens the sub-matrices of BFloat16 values held in the first source vectors to single-precision values and multiplies them by the corresponding widened sub-matrices of BFloat16 values in the second source vectors to single-precision values. The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ single-precision sums of outer products are then destructively added to the single-precision destination tile. This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.

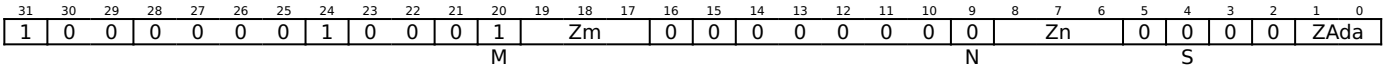
This instruction follows SME BFloat16 numerical behaviors.

This instruction is unpredicated.

It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4)



Encoding

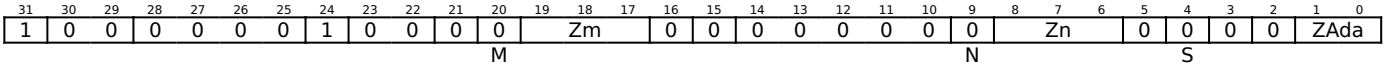
```
BFMOP4A <ZAda>.S, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Single vectors

(FEAT_SME_MOP4)



Encoding

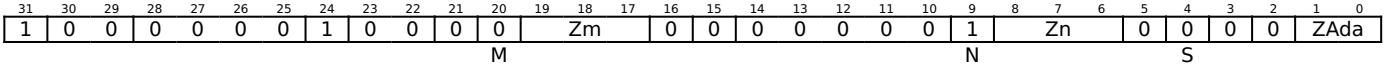
```
BFMOP4A <ZAda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple and single vectors

(FEAT_SME_MOP4)



Encoding

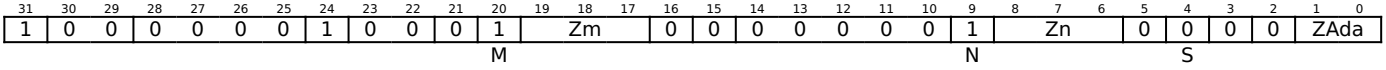
```
BFMOP4A <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple vectors

(FEAT_SME_MOP4)



Encoding

```
BFMOP4A <ZAda>.S, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV 32;
let tilesize : integer{} = 4 * dim * dim * 32;
let op3 : bits(tilesize) = ZAtile{tilesize}(da, 32);
var result : bits(tilesize);

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{VL}(n + (nreg-1) * col_hv);
    let op2 : bits(VL) = Z{VL}(m + (mreg-1) * row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            let sum : bits(32) = op3[tile_idx*:32];
            let erow_0 : bits(16) = op1[(2 * row_idx + 0)*:16];
            let erow_1 : bits(16) = op1[(2 * row_idx + 1)*:16];
            let ecol_0 : bits(16) = op2[(2 * col_idx + 0)*:16];
            let ecol_1 : bits(16) = op2[(2 * col_idx + 1)*:16];
            result[tile_idx*:32] = BFDotAdd(sum, erow_0, erow_1, ecol_0, ecol_1, FPCR());
        end;
    end;
end;
ZAtile{tilesize}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMOP4S (non-widening)

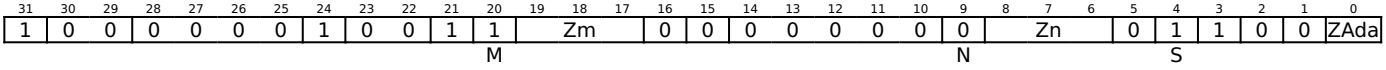
BFloat16 quarter-tile outer product, subtracting

This instruction generates four independent quarter-tile BFloat16 outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and subtracts the results from the corresponding elements of a 16-bit element ZA tile.
Each of the quarter-tile outer products is generated by multiplying the $SVL_H \div 2 \times 1$ sub-matrix of BFloat16 values held in the half-vectors of the first source vectors by the $1 \times SVL_H \div 2$ sub-matrix of BFloat16 values held in the half-vectors of the second source vectors.

The resulting quarter-tile $SVL_H \div 2 \times SVL_H \div 2$ BFloat16 outer products are destructively subtracted from the destination ZA tile. This is equivalent to performing a single multiply-subtract from each of the destination tile elements.
This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.
This instruction is unpredicated.
It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_B16B16)



Encoding

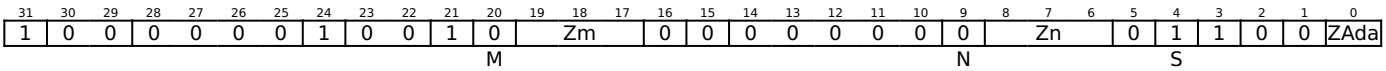
```
BFMOP4S <ZAda>.H, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Single vectors

(FEAT_SME_MOP4 && FEAT_SME_B16B16)



Encoding

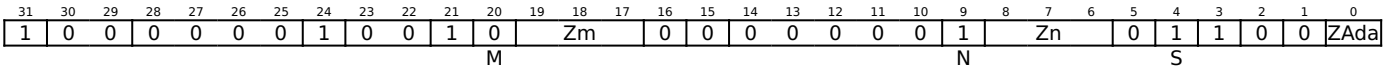
```
BFMOP4S <ZAda>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_B16B16)



Encoding

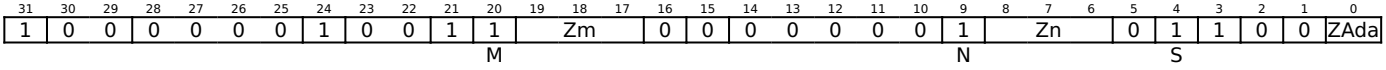
```
BFMOP4S <ZAda>.H, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_B16B16)



Encoding

```
BFMOP4S <ZAda>.H, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV 16;
let tilesize : integer{} = 4 * dim * dim * 16;
let op3 : bits(tilesize) = ZAtile{tilesize}(da, 16);
var result : bits(tilesize);

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{VL}(n + (nreg-1) * col_hv);
    let op2 : bits(VL) = Z{VL}(m + (mreg-1) * row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var elem1 : bits(16) = op1[row_idx*:16];
            let elem2 : bits(16) = op2[col_idx*:16];
            let elem3 : bits(16) = op3[tile_idx*:16];
            let honor_altfp : boolean = FALSE;   // Alternate handling ignored
            elem1 = BFNeg(elem1, honor_altfp);
            result[tile_idx*:16] = BFMulAdd_ZA(elem3, elem1, elem2, FPCR());
        end;
    end;
end;
ZAtile{tilesize}(da, 16) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMOP4S (widening)

BFloat16 quarter-tile sum of outer products to single-precision, subtracting

This instruction generates four independent quarter-tile BFloat16 sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and subtracts the results from the corresponding elements of a 32-bit element ZA tile.

Each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 2$ sub-matrix of BFloat16 values held in the half-vectors of the first source vectors by the $2 \times SVL_S \div 2$ sub-matrix of BFloat16 values held in the half-vectors of the second source vectors. Each 32-bit container of the first source vectors holds 2 elements of each row of a $SVL_S \div 2 \times 2$ sub-matrix. Similarly, each 32-bit container of the second source vectors holds 2 elements of each column of a $2 \times SVL_S \div 2$ sub-matrix.

The instruction widens the sub-matrices of BFloat16 values held in the first source vectors to single-precision values and multiplies them by the corresponding widened sub-matrices of BFloat16 values in the second source vectors to single-precision values. The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ single-precision sums of outer products are then destructively subtracted from the single-precision destination tile. This is equivalent to performing a 2-way dot product and subtract from each of the destination tile elements.

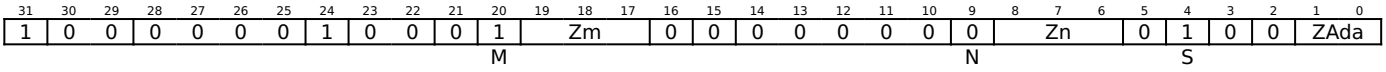
This instruction follows SME BFloat16 numerical behaviors.

This instruction is unpredicated.

It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4)



Encoding

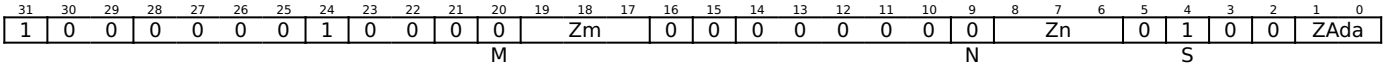
```
BFMOP4S <ZAda>.S, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Single vectors

(FEAT_SME_MOP4)



Encoding

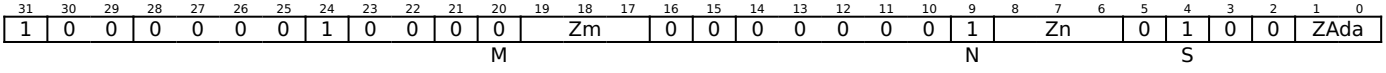
```
BFMOP4S <ZAda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple and single vectors

(FEAT_SME_MOP4)



Encoding

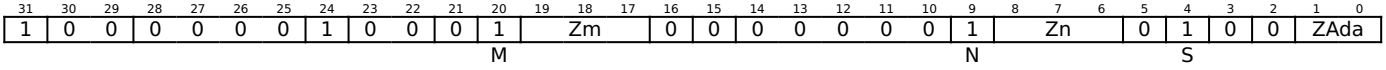
```
BFMOP4S <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple vectors

(FEAT_SME_MOP4)



Encoding

```
BFMOP4S <ZAda>.S, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV 32;
let tilesize : integer{} = 4 * dim * dim * 32;
let op3 : bits(tilesize) = ZAtile{tilesize}(da, 32);
var result : bits(tilesize);

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{VL}(n + (nreg-1) * col_hv);
    let op2 : bits(VL) = Z{VL}(m + (mreg-1) * row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            let sum : bits(32) = op3[tile_idx*:32];
            var erow_0 : bits(16) = op1[(2 * row_idx + 0)*:16];
            var erow_1 : bits(16) = op1[(2 * row_idx + 1)*:16];
            let ecol_0 : bits(16) = op2[(2 * col_idx + 0)*:16];
            let ecol_1 : bits(16) = op2[(2 * col_idx + 1)*:16];
            let honor_altfp : boolean = FALSE;   // Alternate handling ignored
            erow_0 = BFNeg(erow_0, honor_altfp);
            erow_1 = BFNeg(erow_1, honor_altfp);
            result[tile_idx*:32] = BFDotAdd(sum, erow_0, erow_1, ecol_0, ecol_1, FPCR());
        end;
    end;
end;
ZAtile{tilesize}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMOPA (non-widening)

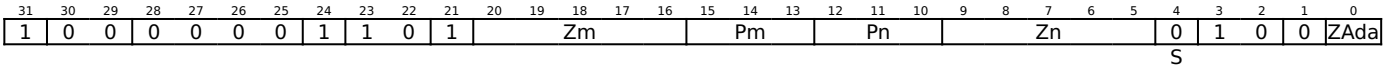
BFloat16 outer product, accumulating

This instruction works with a 16-bit element ZA tile.
This instruction generates an outer product of the first source vector and the second source vector. The first source is $SVL_H \times 1$ vector and the second source is $1 \times SVL_H$ vector.

Each source vector is independently predicated by a corresponding governing predicate. When either source vector element is Inactive the corresponding destination tile element remains unmodified.
The resulting outer product, $SVL_H \times SVL_H$, is then destructively added to the destination tile. This is equivalent to performing a single multiply-accumulate to each of the destination tile elements.
This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.
ID_AA64SMFR0_EL1.B16B16 indicates whether this instruction is implemented.

SME2

(FEAT_SME_B16B16)



Encoding

```
BFMOPA <ZAda>.H, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV 16;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let op1 : bits(VL) = Z{VL}(n);
let op2 : bits(VL) = Z{VL}(m);
let op3 : bits(dim*dim*16) = ZAtile{dim*dim*16}(da, 16);
var result : bits(dim*dim*16);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        let elem2 : bits(16) = op2[col*:16];
        let elem3 : bits(16) = op3[(row*dim+col)*:16];

        if (ActivePredicateElement{PL}(mask1, row, 16) &&
            ActivePredicateElement{PL}(mask2, col, 16)) then
            let elem1 : bits(16) = op1[row*:16];
            result[(row*dim+col)*:16] = BFMulAdd_ZA(elem3, elem1, elem2, FPCR());
        else
            result[(row*dim+col)*:16] = elem3;
        end;
    end;
end;

ZAtile{dim*dim*16}(da, 16) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMOPA (widening)

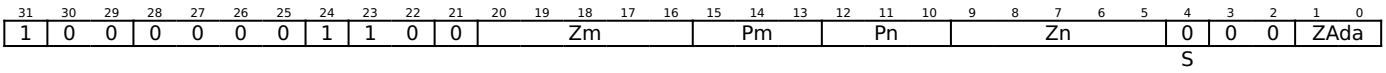
BFloat16 sum of outer products to single-precision, accumulating

This instruction works with a 32-bit element ZA tile.
This instruction multiplies the $SVL_S \times 2$ sub-matrix of BFloat16 values held in the first source vector by the $2 \times SVL_S$ sub-matrix of BFloat16 values in the second source vector.
Each source vector is independently predicated by a corresponding governing predicate. When a 16-bit source element is Inactive it is treated as having the value +0.0, but if both pairs of source vector elements that correspond to a 32-bit destination element contain Inactive elements, then the destination element remains unmodified.
The resulting $SVL_S \times SVL_S$ single-precision sum of outer products is then destructively added to the single-precision destination tile. This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.
Each 32-bit container of the first source vector holds 2 consecutive column elements of each row of a $SVL_S \times 2$ sub-matrix. Similarly, each 32-bit container of the second source vector holds 2 consecutive row elements of each column of a $2 \times SVL_S$ sub-matrix.

This instruction follows SME BFloat16 numerical behaviors.

SME

(FEAT_SME)



Encoding

```
BFMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV 32;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*32) = ZAtile{dim*dim*32}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        // determine row/col predicates
        let prow_0 : boolean = ActivePredicateElement{PL}(mask1, 2*row + 0, 16);
        let prow_1 : boolean = ActivePredicateElement{PL}(mask1, 2*row + 1, 16);
        let pcol_0 : boolean = ActivePredicateElement{PL}(mask2, 2*col + 0, 16);
        let pcol_1 : boolean = ActivePredicateElement{PL}(mask2, 2*col + 1, 16);

        var sum : bits(32) = operand3[(row*dim+col)*:32];
        if (prow_0 && pcol_0) || (prow_1 && pcol_1) then
            let erow_0 : bits(16) = if prow_0 then operand1[(2*row + 0)*:16]
                                              else FPZero{16}('0');
            let erow_1 : bits(16) = if prow_1 then operand1[(2*row + 1)*:16]
                                              else FPZero{16}('0');
            let ecol_0 : bits(16) = if pcol_0 then operand2[(2*col + 0)*:16]
                                              else FPZero{16}('0');
            let ecol_1 : bits(16) = if pcol_1 then operand2[(2*col + 1)*:16]
                                              else FPZero{16}('0');
            sum = BFDotAdd(sum, erow_0, erow_1, ecol_0, ecol_1, FPCR());
        end;
        result[(row*dim+col)*:32] = sum;
    end;
end;

ZAtile{dim*dim*32}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMOPS (non-widening)

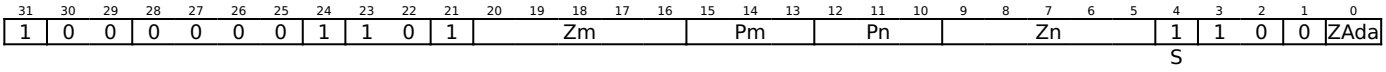
BFloat16 outer product, subtracting

This instruction works with a 16-bit element ZA tile.
This instruction generates an outer product of the first source vector and the second source vector. The first source is $SVL_H \times 1$ vector and the second source is $1 \times SVL_H$ vector.

Each source vector is independently predicated by a corresponding governing predicate. When either source vector element is Inactive the corresponding destination tile element remains unmodified.
The resulting outer product, $SVL_H \times SVL_H$, is then destructively subtracted from the destination tile. This is equivalent to performing a single multiply-subtract from each of the destination tile elements.
This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.
ID_AA64SMFR0_EL1.B16B16 indicates whether this instruction is implemented.

SME2

(FEAT_SME_B16B16)



Encoding

```
BFMOPS <ZAda>.H, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV 16;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let op1 : bits(VL) = Z{VL}(n);
let op2 : bits(VL) = Z{VL}(m);
let op3 : bits(dim*dim*16) = ZAtile{dim*dim*16}(da, 16);
var result : bits(dim*dim*16);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        let elem2 : bits(16) = op2[col*:16];
        let elem3 : bits(16) = op3[(row*dim+col)*:16];

        if (ActivePredicateElement{PL}(mask1, row, 16) &&
            ActivePredicateElement{PL}(mask2, col, 16)) then
            let elem1 : bits(16) = BFNeg(op1[row*:16]);
            result[(row*dim+col)*:16] = BFMulAdd_ZA(elem3, elem1, elem2, FPCR());
        else
            result[(row*dim+col)*:16] = elem3;
        end;
    end;
end;

ZAtile{dim*dim*16}(da, 16) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMOPS (widening)

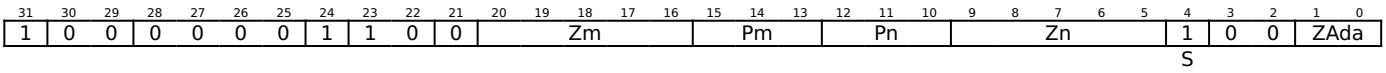
BFloat16 sum of outer products to single-precision, subtracting

This instruction works with a 32-bit element ZA tile.
This instruction multiplies the $SVL_S \times 2$ sub-matrix of BFloat16 values held in the first source vector by the $2 \times SVL_S$ sub-matrix of BFloat16 values in the second source vector.
Each source vector is independently predicated by a corresponding governing predicate. When a 16-bit source element is Inactive it is treated as having the value +0.0, but if both pairs of source vector elements that correspond to a 32-bit destination element contain Inactive elements, then the destination element remains unmodified.
The resulting $SVL_S \times SVL_S$ single-precision sum of outer products is then destructively subtracted from the single-precision destination tile. This is equivalent to performing a 2-way dot product and subtract from each of the destination tile elements.
Each 32-bit container of the first source vector holds 2 consecutive column elements of each row of a $SVL_S \times 2$ sub-matrix. Similarly, each 32-bit container of the second source vector holds 2 consecutive row elements of each column of a $2 \times SVL_S$ sub-matrix.

This instruction follows SME BFloat16 numerical behaviors.

SME

(FEAT_SME)



Encoding

```
BFMOPS <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV 32;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*32) = ZAtile{dim*dim*32}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        // determine row/col predicates
        let prow_0 : boolean = ActivePredicateElement{PL}(mask1, 2*row + 0, 16);
        let prow_1 : boolean = ActivePredicateElement{PL}(mask1, 2*row + 1, 16);
        let pcol_0 : boolean = ActivePredicateElement{PL}(mask2, 2*col + 0, 16);
        let pcol_1 : boolean = ActivePredicateElement{PL}(mask2, 2*col + 1, 16);

        var sum : bits(32) = operand3[(row*dim+col)*:32];
        if (prow_0 && pcol_0) || (prow_1 && pcol_1) then
            var erow_0 : bits(16) = if prow_0 then operand1[(2*row + 0)*:16]
                                              else FPZero{16}('0');
            var erow_1 : bits(16) = if prow_1 then operand1[(2*row + 1)*:16]
                                              else FPZero{16}('0');
            let ecol_0 : bits(16) = if pcol_0 then operand2[(2*col + 0)*:16]
                                              else FPZero{16}('0');
            let ecol_1 : bits(16) = if pcol_1 then operand2[(2*col + 1)*:16]
                                              else FPZero{16}('0');
            let honor_altfp : boolean = FALSE;   // Alternate handling ignored
            if prow_0 then erow_0 = BFNeg(erow_0, honor_altfp); end;
            if prow_1 then erow_1 = BFNeg(erow_1, honor_altfp); end;
            sum = BFDotAdd(sum, erow_0, erow_1, ecol_0, ecol_1, FPCR());
        end;
        result[(row*dim+col)*:32] = sum;
    end;
end;

ZAtile{dim*dim*32}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMUL (multiple and single vector)

Multi-vector BFloat16 multiply by vector

This instruction multiplies all the BFloat16 elements of the two or four first source vectors by the corresponding BFloat16 elements of the second source vector, and places the results in the corresponding elements of the two or four destination vectors.

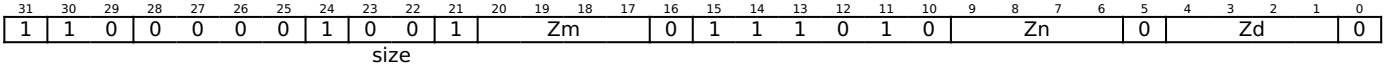
This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_BFSCALE)



Encoding

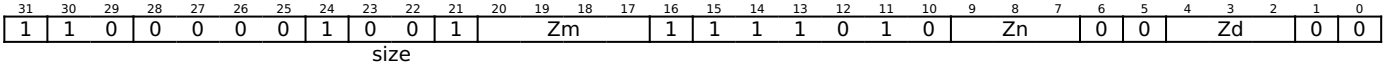
```
BFMUL { <Zd1>.H-<Zd2>.H }, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_BFSCALE) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let d : integer = UInt(Zd::'0');
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_BFSCALE)



Encoding

```
BFMUL { <Zd1>.H-<Zd4>.H }, { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_BFSCALE) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let d : integer = UInt(Zd::'00');
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn1>	For the "Two registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMul(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFMUL (multiple vectors)

Multi-vector BFloat16 multiply

This instruction multiplies all the BFloat16 elements of the two or four first source vectors by the corresponding BFloat16 elements of the two or four second source vectors, and places the results in the corresponding elements of the two or four destination vectors.

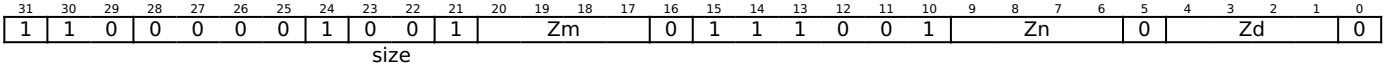
This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_BFSCALE)



Encoding

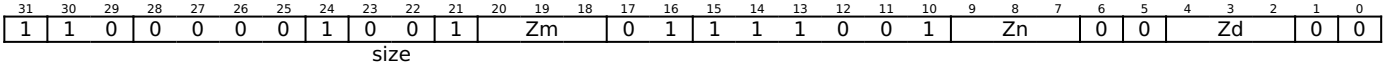
```
BFMUL { <Zd1>.H-<Zd2>.H }, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_BFSCALE) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let d : integer = UInt(Zd::'0');
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_BFSCALE)



Encoding

```
BFMUL { <Zd1>.H-<Zd4>.H }, { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_BFSCALE) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 16;
let d : integer = UInt(Zd::'00');
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn1>	For the "Two registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        results[[r]][e*:16] = BFMul(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFSCALE (multiple and single vector)

Multi-vector BFloat16 adjust exponent by vector

This instruction multiplies the BFloat16 elements of the two or four first source vectors by 2.0 to the power of the signed integer values in the corresponding elements of the second source vector and destructively places the results in the corresponding elements of the two or four first source vectors.

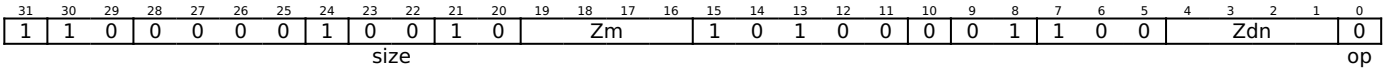
This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_BFSCALE)



Encoding

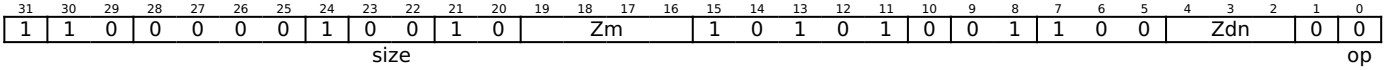
```
BFSCALE { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_BFSCALE) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_BFSCALE)



Encoding

```
BFSCALE { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_BFSCALE) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : integer = SInt(operand2[e*:16]);
        results[[r]][e*:16] = BFScale(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFSCALE (multiple vectors)

Multi-vector BFloat16 adjust exponent

This instruction multiplies the BFloat16 elements of the two or four first source vectors by 2.0 to the power of the signed integer values in the corresponding elements of the two or four second source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

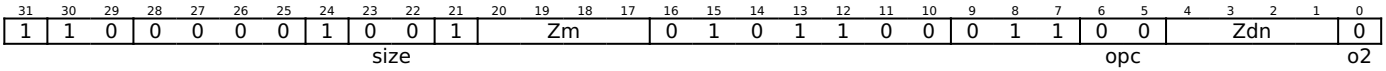
This instruction follows SME2 non-widening BFloat16 numerical behaviors corresponding to instructions that place their results in two or four SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_SVE_BFSCALE)



Encoding

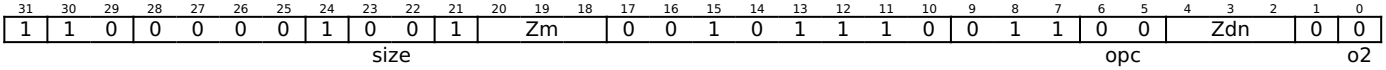
```
BFSCALE { <Zdn1>.H-<Zdn2>.H }, { <Zdn1>.H-<Zdn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_BFSCALE) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_SVE_BFSCALE)



Encoding

```
BFSCALE { <Zdn1>.H-<Zdn4>.H }, { <Zdn1>.H-<Zdn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_SVE_BFSCALE) then
    EndOfDecode(Decode_UNDEF);
end;
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.
<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : integer = SInt(operand2[e*:16]);
        results[[r]][e*:16] = BFScale(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFSUB

Multi-vector BFloat16 subtract from ZA array vectors

This instruction destructively subtracts all elements of the two or four source vectors from the corresponding BFloat16 elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.

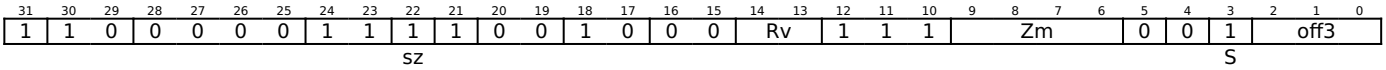
This instruction is unpredicated.

ID_AA64SMFR0_EL1.B16B16 indicates whether this instruction is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_B16B16)



Encoding

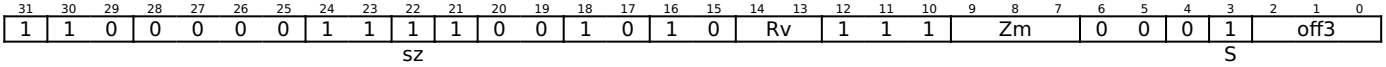
```
BFSUB ZA.H[<Wv>, <offs>{, VGx2}], { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let m : integer{} = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_B16B16)



Encoding

```
BFSUB ZA.H[<Wv>, <offs>{, VGx4}], { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_B16B16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let m : integer{} = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zm4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = ZAvector{VL}(vec);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(16) = operand1[e*:16];
        let element2 : bits(16) = operand2[e*:16];
        result[e*:16] = BFSub_ZA(element1, element2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFTMOPA (non-widening)

BFloat16 sparse outer product, accumulating

This instruction generates BFloat16 outer product by multiplying the 1-in-2 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the destination ZA tile elements.

The outer product is generated by multiplying the selected 1-in-2 BFloat16 value from each overlapping 16-bit containers of the two $SVL_H \times 1$ sub-matrices in the first source vectors by the BFloat16 value from the corresponding 16-bit container of the compressed $1 \times SVL_H$ sub-matrix in the second source vector.

The 1-in-2 BFloat16 value in the first source vectors is selected by 2-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the result. If both bits of the 2-bit control corresponding to the elements of the first source vectors are 1, only the element corresponding to the least significant bit is selected.

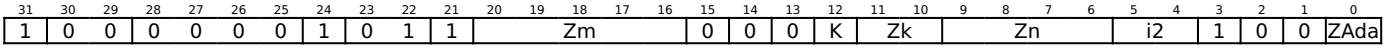
The instruction multiplies the selected elements of sub-matrices of BFloat16 values held in the first source vectors by the corresponding elements of sub-matrix of BFloat16 values in the second source vector. The resulting BFloat16 $SVL_H \times SVL_H$ outer product is then destructively added to the BFloat16 destination tile. This is equivalent to performing a single multiply-accumulate to each of the destination tile elements.

This instruction follows SME2 ZA-targeting non-widening BFloat16 numerical behaviors.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP && FEAT_SME_B16B16)



Encoding

```
BFTMOPA <ZAda>.H, { <Zn1>.H-<Zn2>.H }, <Zm>.H, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) || !IsFeatureImplemented(FEAT_SME_B16B16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 16;
let csize : integer{} = VL DIV 8;
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*16) = ZAtile{dim*dim*16}(da, 16);
var result : bits(dim*dim*16);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var i : integer = 0;
        var elem1 : bits(16) = BFZero{16}('0');
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{}(n+r);
            if i < 1 && ctrl[(2*col + r)*:1] == '1' then
                elem1 = op1[row*:16];
                i = i + 1;
            end;
        end;
        let elem2 : bits(16) = op2[col*:16];
        let sum : bits(16) = op4[(row*dim+col)*:16];
        result[(row*dim+col)*:16] = BFMulAdd_ZA(sum, elem1, elem2, FPCR());
    end;
end;
ZAtile{dim*dim*16}(da, 16) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFTMOPA (widening)

BFloat16 sparse sum of outer products to single-precision, accumulating

This instruction generates BFloat16 sum of outer products by multiplying the 2-in-4 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

The sum of outer products is generated by multiplying the selected 2-in-4 BFloat16 values from each overlapping 32-bit containers of the two $SVL_S \times 2$ sub-matrices in the first source vectors by the two BFloat16 values from the corresponding 32-bit container of the $2 \times SVL_S$ sub-matrix in the second source vector. The two selected elements from each overlapping 32-bit containers of the first source vectors correspond to 2-in-4 elements of rows of two $SVL_S \times 2$ sub-matrices. Each 32-bit container of the second source vector holds 2 elements of columns of a compressed $2 \times SVL_S$ sub-matrix.

The 2-in-4 BFloat16 values from overlapping 32-bit containers of the first source vectors are selected by 4-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the sum of products result. If more than two bits of the 4-bit control corresponding to 32-bit containers of the first source vectors are 1, only the elements corresponding to the least two significant bits are selected.

The instruction widens the selected elements of sub-matrices of BFloat16 values held in the first source vectors to single-precision values and multiplies them by the corresponding widened elements of sub-matrix of BFloat16 values in the second source vector to single-precision values. The resulting $SVL_S \times SVL_S$ single-precision sum of outer products is then destructively added to the single-precision destination tile. This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.

This instruction follows SME BFloat16 numerical behaviors.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	1	0	Zm					0	0	0	K	Zk		Zn				i2		0	0	ZAda	

Encoding

```
BFTMOPA <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 32;
let csize : integer{} = VL DIV 8;
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*32) = ZAtile{}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var i : integer = 0;
        var erow : array [[2]] of bits(16);
        erow[[0]] = BFZero{16}('0');
        erow[[1]] = BFZero{16}('0');
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{}(n+r);
            for e = 0 to 1 do
                if i < 2 && ctrl[(4*col + 2*r + e)*:1] == '1' then
                    erow[[i]] = op1[(2*row + e)*:16];
                    i = i + 1;
                end;
            end;
        end;
        var ecol : array [[2]] of bits(16);
        ecol[[0]] = op2[(2*col + 0)*:16];
        ecol[[1]] = op2[(2*col + 1)*:16];
        let sum : bits(32) = op4[(row*dim+col)*:32];
        result[(row*dim+col)*:32] = BFDotAdd(sum, erow[[0]], erow[[1]],
                                             ecol[[0]], ecol[[1]], FPCR());
    end;
end;
ZAtile{dim*dim*32}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

BFVDOT

Multi-vector BFloat16 vertical dot product by indexed element to single-precision

This instruction calculates the sum-of-products of each vertical pair of BFloat16 values in the corresponding elements of the two first source vectors with the pair of BFloat16 values in the indexed 32-bit group of the corresponding 128-bit segment of the second source vector. The single-precision sum-of-products are destructively added to the corresponding single-precision elements of the two ZA single-vector groups.

The BFloat16 pairs within the second source vector are specified using an immediate index that selects the same BFloat16 pair position within each 128-bit vector segment. The element index range is from 0 to 3.

The single-vector group within each half of the ZA array is selected by the sum of the vector select register and offset, modulo half the number of ZA array vectors.

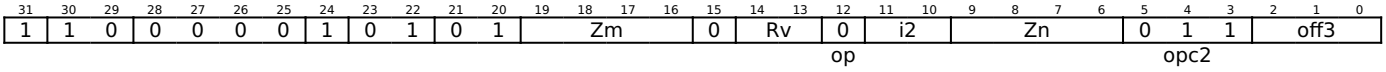
The vector group symbol VGx2 indicates that the ZA operand consists of two ZA single-vector groups. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME2 ZA-targeting BFloat16 numerical behaviors.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
BFVDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 2;
let segmentsize : integer{} = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to 1 do
    let operand1a : bits(VL) = Z{VL}(n);
    let operand1b : bits(VL) = Z{VL}(n+1);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let elt1_a : bits(hsize) = operand1a[(2 * e + r)*:hsize];
        let elt1_b : bits(hsize) = operand1b[(2 * e + r)*:hsize];
        let elt2_a : bits(hsize) = operand2[(2 * s + 0)*:hsize];
        let elt2_b : bits(hsize) = operand2[(2 * s + 1)*:hsize];
        var sum : bits(esize) = operand3[e*:esize];
        sum = BFDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

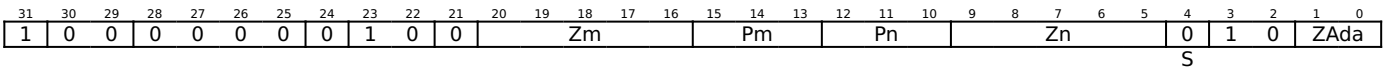
BMOPA

Bitwise exclusive NOR population count outer product, accumulating

This instruction works with 32-bit element ZA tile. This instruction generates an outer product of the first source $SVL_S \times 1$ vector and the second source $1 \times SVL_S$ vector. Each outer product element is obtained as population count of the bitwise XNOR result of the corresponding 32-bit elements of the first source vector and the second source vector. Each source vector is independently predicated by a corresponding governing predicate. When either source vector element is inactive the corresponding destination tile element remains unmodified. The resulting $SVL_S \times SVL_S$ product is then destructively added to the destination tile.

SME2

(FEAT_SME2)



Encoding

```
BMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.S, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);

for row = 0 to dim-1 do
    let element1 : bits(esize) = operand1[row*:esize];
    for col = 0 to dim-1 do
        let element2 : bits(esize) = operand2[col*:esize];
        let element3 : bits(esize) = operand3[(row*dim + col)*:esize];
        if (ActivePredicateElement{PL}(mask1, row, esize) &&
              ActivePredicateElement{PL}(mask2, col, esize)) then
            let res : integer = BitCount(NOT(element1 XOR element2));
            result[(row*dim + col)*:esize] = element3 + res;
        else
            result[(row*dim + col)*:esize] = element3;
        end;
    end;
end;
ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

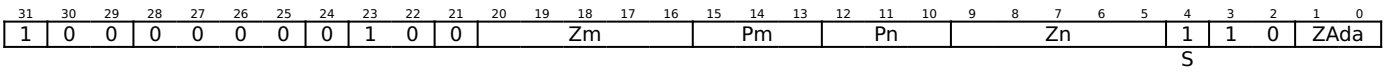
BMOPS

Bitwise exclusive NOR population count outer product, subtracting

This instruction works with 32-bit element ZA tile. This instruction generates an outer product of the first source $SVL_S \times 1$ vector and the second source $1 \times SVL_S$ vector. Each outer product element is obtained as population count of the bitwise XNOR result of the corresponding 32-bit elements of the first source vector and the second source vector. Each source vector is independently predicated by a corresponding governing predicate. When either source vector element is inactive the corresponding destination tile element remains unmodified. The resulting $SVL_S \times SVL_S$ product is then destructively subtracted from the destination tile.

SME2

(FEAT_SME2)



Encoding

```
BMOPS <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.S, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);

for row = 0 to dim-1 do
    let element1 : bits(esize) = operand1[row*:esize];
    for col = 0 to dim-1 do
        let element2 : bits(esize) = operand2[col*:esize];
        let element3 : bits(esize) = operand3[(row*dim + col)*:esize];
        if (ActivePredicateElement{PL}(mask1, row, esize) &&
              ActivePredicateElement{PL}(mask2, col, esize)) then
            let res : integer = BitCount(NOT(element1 XOR element2));
            result[(row*dim + col)*:esize] = element3 - res;
        else
            result[(row*dim + col)*:esize] = element3;
        end;
    end;
end;
ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

F1CVT, F2CVT

Multi-vector 8-bit floating-point convert to half-precision

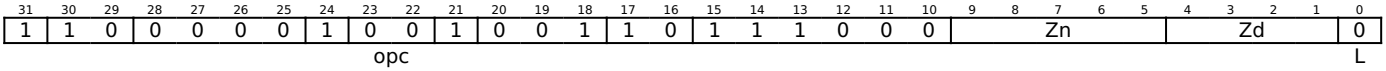
This instruction converts each 8-bit floating-point element of the source vector to half-precision while downscaling the value, and places the results in the corresponding 16-bit elements of the destination vectors. F1CVT scales the values by $2^{-UInt(FPMR.LSCALE[3:0])}$. F2CVT scales the values by $2^{-UInt(FPMR.LSCALE2[3:0])}$. The 8-bit floating-point encoding format for F1CVT is selected by FPMR.F8S1. The 8-bit floating-point encoding format for F2CVT is selected by FPMR.F8S2.

This instruction is unpredicated.

It has encodings from 2 classes: F1CVT and F2CVT

F1CVT

(FEAT_SME2 && FEAT_FP8)



Encoding

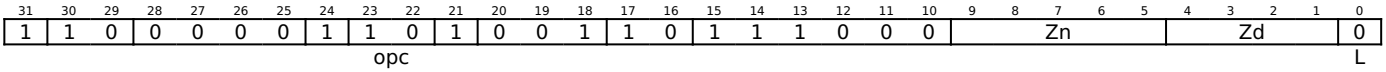
```
F1CVT { <Zd1>.H-<Zd2>.H }, <Zn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
let issrc2 : boolean = FALSE;
```

F2CVT

(FEAT_SME2 && FEAT_FP8)



Encoding

```
F2CVT { <Zd1>.H-<Zd2>.H }, <Zn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
let issrc2 : boolean = TRUE;
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 8;
let operand : bits(VL) = Z{VL}(n);
var result : bits(2*VL);

for e = 0 to elements-1 do
    let element : bits(8) = operand[e*:8];
    result[e*:16] = FP8ConvertFP(element, issrc2, FPCR(), FPMR());
end;

Z{VL}(d+0) = result[VL-1:0];
Z{VL}(d+1) = result[2*VL-1:VL];
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

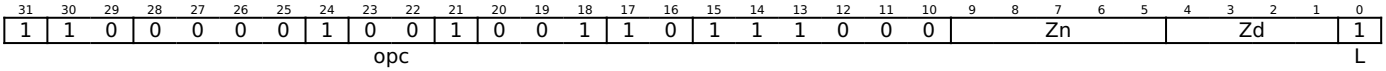
F1CVTL, F2CVTL

Multi-vector 8-bit floating-point convert to deinterleaved half-precision

This instruction converts each 8-bit floating-point element of the source vector to half-precision while downscaling the value, and places the two-way deinterleaved results in the corresponding 16-bit elements of the destination vectors. F1CVTL scales the values by $2^{-UInt(FPMR.LSCALE[3:0])}$. F2CVTL scales the values by $2^{-UInt(FPMR.LSCALE2[3:0])}$. The 8-bit floating-point encoding format for F1CVTL is selected by FPMR.F8S1. The 8-bit floating-point encoding format for F2CVTL is selected by FPMR.F8S2.
This instruction is unpredicated.
It has encodings from 2 classes: F1CVTL and F2CVTL

F1CVTL

(FEAT_SME2 && FEAT_FP8)



Encoding

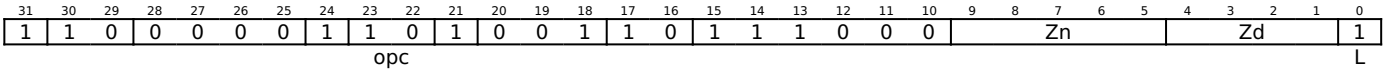
```
F1CVTL { <Zd1>.H-<Zd2>.H }, <Zn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
let issrc2 : boolean = FALSE;
```

F2CVTL

(FEAT_SME2 && FEAT_FP8)



Encoding

```
F2CVTL { <Zd1>.H-<Zd2>.H }, <Zn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
let issrc2 : boolean = TRUE;
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV 16;
let operand : bits(VL) = Z{VL}(n);
var result1 : bits(VL);
var result2 : bits(VL);

for p = 0 to pairs-1 do
    let element1 : bits(8) = operand[(2*p + 0)*:8];
    let element2 : bits(8) = operand[(2*p + 1)*:8];
    result1[p*:16] = FP8ConvertFP(element1, issrc2, FPCR(), FPMR());
    result2[p*:16] = FP8ConvertFP(element2, issrc2, FPCR(), FPMR());
end;

Z{VL}(d+0) = result1;
Z{VL}(d+1) = result2;
```

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FADD

Multi-vector floating-point accumulate to ZA array vectors

This instruction destructively adds all elements of the two or four source vectors to the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME ZA-targeting floating-point behaviors.

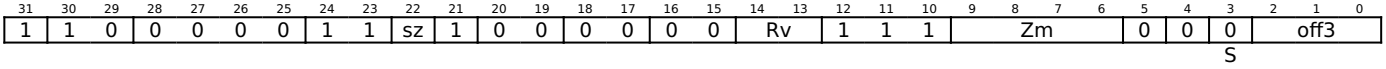
This instruction is unpredicated.

ID_AA64SMFR0_EL1.F64F64 indicates whether the double-precision variant is implemented, and ID_AA64SMFR0_EL1.F16F16 indicates whether the half-precision variant is implemented.

It has encodings from 4 classes: Two ZA single-vectors , Two ZA single-vectors of half-precision elements , Four ZA single-vectors and Four ZA single-vectors of half-precision elements

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

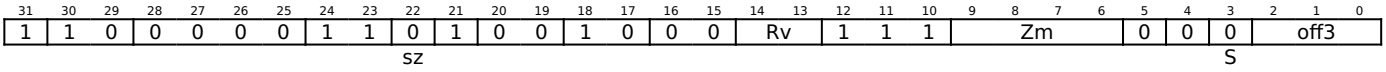
```
FADD ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let m : integer{} = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Two ZA single-vectors of half-precision elements

(FEAT_SME_F16F16 || FEAT_SME_F8F16)



Encoding

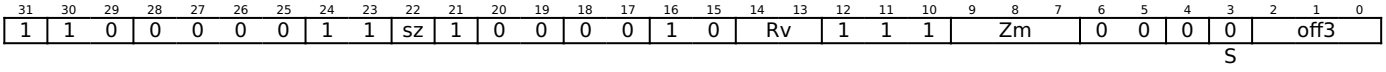
```
FADD ZA.H[<Wv>, <offs>{, VGx2}], { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) && !IsFeatureImplemented(FEAT_SME_F8F16) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let m : integer{} = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

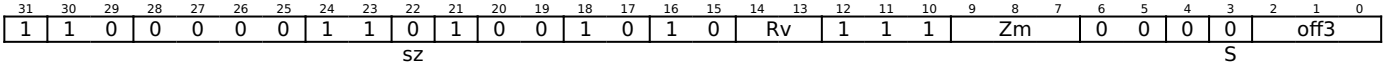
```
FADD ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let m : integer{} = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Four ZA single-vectors of half-precision elements

(FEAT_SME_F16F16 || FEAT_SME_F8F16)



Encoding

```
FADD ZA.H[<Wv>, <offs>{, VGx4}], { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) && !IsFeatureImplemented(FEAT_SME_F8F16) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let m : integer{} = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zm1> For the "Two ZA single-vectors of half-precision elements" and "Two ZA single-vectors" variants: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA single-vectors of half-precision elements" and "Four ZA single-vectors" variants: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zm4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = ZAvector{VL}(vec);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPAdd_ZA{esize}(element1, element2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FAMAX

Multi-vector floating-point absolute maximum

This instruction determines the maximum absolute value from floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

The behavior is as follows:

- Alternate floating-point behaviors do not apply, as if FPCR.AH is 0.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

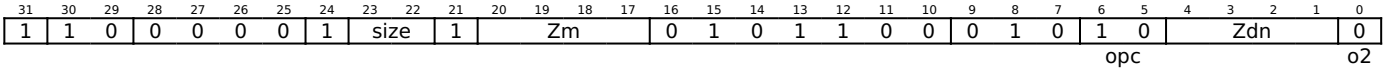
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_FAMINMAX)



Encoding

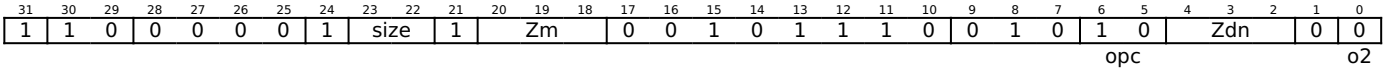
```
FAMAX { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_FAMINMAX)



Encoding

```
FAMAX { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPAbsMax{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FAMIN

Multi-vector floating-point absolute minimum

This instruction determines the minimum absolute value from floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

The behavior is as follows:

- Alternate floating-point behaviors do not apply, as if FPCR.AH is 0.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is the default NaN, with the sign bit set to 0.
- Denormalized inputs and results are never flushed to zero, as if FPCR.{FZ, FZ16, FIZ} are all 0.
- Denormalized inputs never generate an Input Denormal floating-point exception.

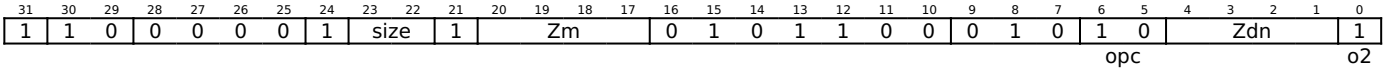
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_FAMINMAX)



Encoding

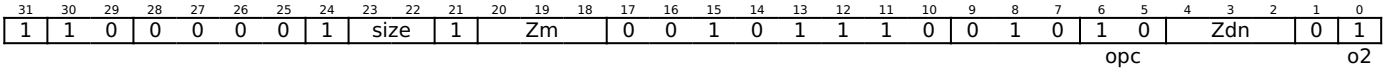
```
FAMIN { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_FAMINMAX)



Encoding

```
FAMIN { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FAMINMAX) then
    EndOfDecode(Decode_UNDEF);
end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPAbsMin{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCLAMP

Multi-vector floating-point clamp to minimum/maximum number

This instruction clamps each floating-point element in the two or four destination vectors to between the floating-point minimum value in the corresponding element of the first source vector and the floating-point maximum value in the corresponding element of the second source vector and destructively places the clamped results in the corresponding elements of the two or four destination vectors.

The behavior is as follows for each minimum number and maximum number operation:

- Negative zero compares less than positive zero.
- If one value is numeric and the other is a quiet NaN, the quiet NaN value is ignored and the result is the numeric value.
- When FPCR.DN is 0, if either value is a signaling NaN or if both values are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one value is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN value.
    - Otherwise, the result is a quiet NaN derived from the first NaN value.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN value.
- When FPCR.DN is 1, if either value is a signaling NaN or if both values are NaNs, the result is default NaN.

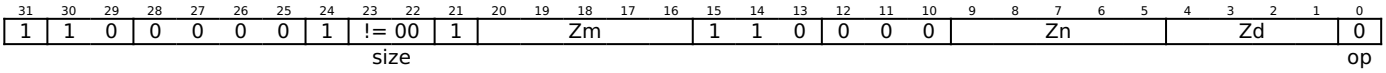
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

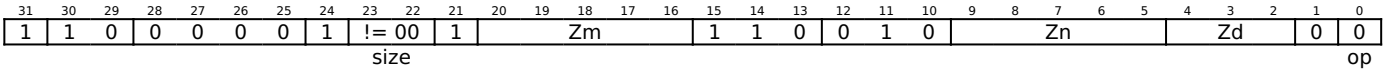
```
FCLAMP { <Zd1>.<T>-<Zd2>.<T> }, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FCLAMP { <Zd1>.<T>-<Zd4>.<T> }, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1> For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = Z{VL}(d+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        let element3 : bits(esize) = operand3[e*:esize];
        let maxelement : bits(esize) = FPMaxNum{esize}(element1, element3, FPCR());
        results[[r]][e*:esize] = FPMinNum{esize}(maxelement, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVT (narrowing, FP16 to FP8)

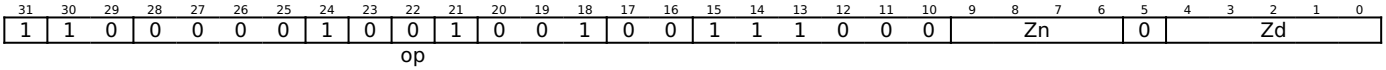
Multi-vector half-precision convert to 8-bit floating-point

This instruction converts each half-precision element of the two source vectors to 8-bit floating-point while scaling the value by $2^{\text{SInt(FPMR.NSCALE[4:0])}}$, and places the results in the half-width elements of the destination vector. The 8-bit floating-point encoding format is selected by FPMR.F8D.

This instruction is unpredicated.

SME2

(FEAT_SME2 && FEAT_FP8)



Encoding

```
FCVT <Zd>.B, { <Zn1>.H-<Zn2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
for e = 0 to elements-1 do
    let element1 : bits(16) = operand1[e*:16];
    let element2 : bits(16) = operand2[e*:16];
    result[(0*elements + e)*:8] = FPConvertFP8{8, 16}(element1, FPCR(), FPMR());
    result[(1*elements + e)*:8] = FPConvertFP8{8, 16}(element2, FPCR(), FPMR());
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVT (narrowing, FP32 to FP16)

Multi-vector single-precision convert to half-precision

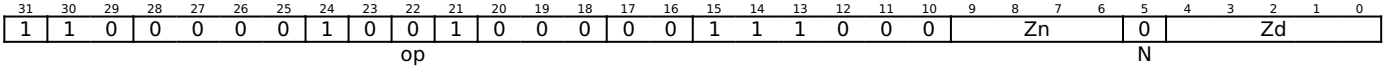
This instruction converts each element of the two source vectors from single-precision to half-precision floating-point, and places the results in the half-width destination elements.

This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
FCVT <Zd>.H, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
for e = 0 to elements-1 do
    let element1 : bits(32) = operand1[e*:32];
    let element2 : bits(32) = operand2[e*:32];
    let res1 : bits(16) = FPConvertSVE{16, 32}(element1, FPCR());
    let res2 : bits(16) = FPConvertSVE{16, 32}(element2, FPCR());
    result[e*:16] = res1;
    result[(elements+e)*:16] = res2;
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVT (narrowing, FP32 to FP8)

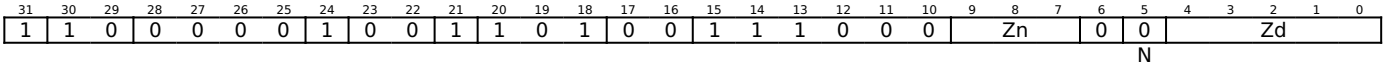
Multi-vector single-precision convert to 8-bit floating-point

This instruction converts each single-precision element of the four source vectors to 8-bit floating-point while scaling the value by $2^{SInt(FPMR.NSCALE)}$, and places the results in the quarter-width elements of the destination vector. The 8-bit floating-point encoding format is selected by FPMR.F8D.

This instruction is unpredicated.

SME2

(FEAT_SME2 && FEAT_FP8)



Encoding

```
FCVT <Zd>.B, { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
let operand3 : bits(VL) = Z{VL}(n+2);
let operand4 : bits(VL) = Z{VL}(n+3);
for e = 0 to elements-1 do
    let element1 : bits(32) = operand1[e*:32];
    let element2 : bits(32) = operand2[e*:32];
    let element3 : bits(32) = operand3[e*:32];
    let element4 : bits(32) = operand4[e*:32];
    result[(0*elements + e)*:8] = FPConvertFP8{8, 32}(element1, FPCR(), FPMR());
    result[(1*elements + e)*:8] = FPConvertFP8{8, 32}(element2, FPCR(), FPMR());
    result[(2*elements + e)*:8] = FPConvertFP8{8, 32}(element3, FPCR(), FPMR());
    result[(3*elements + e)*:8] = FPConvertFP8{8, 32}(element4, FPCR(), FPMR());
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVT (widening)

Multi-vector half-precision convert to single-precision

This instruction converts each element of the source vector from half-precision to single-precision floating-point, and places the results in the double-width destination elements of the destination vectors.

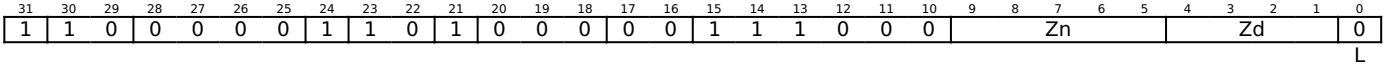
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

ID_AA64SMFR0_EL1.F16F16 indicates whether this instruction is implemented.

SME2

(FEAT_SME_F16F16)



Encoding

```
FCVT { <Zd1>.S-<Zd2>.S }, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let operand : bits(VL) = Z{VL}(n);
var result : bits(2*VL);

for e = 0 to elements-1 do
    let element : bits(16) = operand[e*:16];
    let res : bits(32) = FPConvertSVE{32, 16}(element, FPCR());
    result[e*:32] = res;
end;

Z{VL}(d+0) = result[VL-1:0];
Z{VL}(d+1) = result[2*VL-1:VL];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTL

Multi-vector half-precision convert to deinterleaved single-precision

This instruction converts each element of the source vector from half-precision to single-precision floating-point, and places the two-way deinterleaved results in the double-width destination elements of the destination vectors.

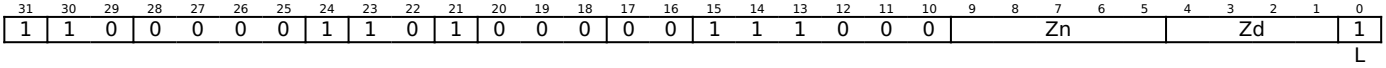
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

ID_AA64SMFR0_EL1.F16F16 indicates whether this instruction is implemented.

SME2

(FEAT_SME_F16F16)



Encoding

```
FCVTL { <Zd1>.S-<Zd2>.S }, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let pairs : integer = VL DIV 32;
let operand : bits(VL) = Z{VL}(n);
var result0 : bits(VL);
var result1 : bits(VL);

for p = 0 to pairs-1 do
    let element1 : bits(16) = operand[(2*p+0)*:16];
    let element2 : bits(16) = operand[(2*p+1)*:16];
    let res1 : bits(32) = FPConvertSVE{32, 16}(element1, FPCR());
    let res2 : bits(32) = FPConvertSVE{32, 16}(element2, FPCR());
    result0[p*:32] = res1;
    result1[p*:32] = res2;
end;

Z{VL}(d+0) = result0;
Z{VL}(d+1) = result1;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTN (FP32 to FP16)

Multi-vector single-precision convert to interleaved half-precision

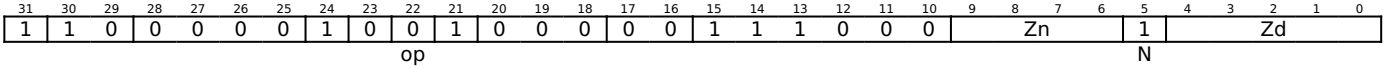
This instruction converts each element of the two source vectors from single-precision to half-precision floating-point, and places the two-way interleaved results in the half-width destination elements.

This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
FCVTN <Zd>.H, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
for e = 0 to elements-1 do
    let element1 : bits(32) = operand1[e*:32];
    let element2 : bits(32) = operand2[e*:32];
    let res1 : bits(16) = FPConvertSVE{16, 32}(element1, FPCR());
    let res2 : bits(16) = FPConvertSVE{16, 32}(element2, FPCR());
    result[(2*e + 0)*:16] = res1;
    result[(2*e + 1)*:16] = res2;
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTN (FP32 to FP8)

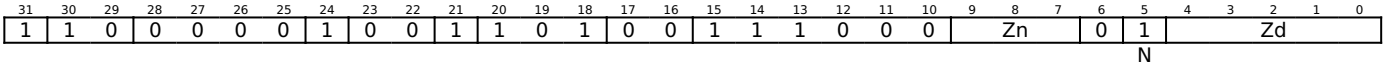
Multi-vector single-precision convert to interleaved 8-bit floating-point

This instruction converts each single-precision element of the four source vectors to 8-bit floating-point while scaling the value by $2^{SInt(FPMR.NSCALE)}$, and places the four-way interleaved results in the quarter-width elements of the destination vector. The 8-bit floating-point encoding format is selected by FPMR.F8D.

This instruction is unpredicated.

SME2

(FEAT_SME2 && FEAT_FP8)



Encoding

```
FCVTN <Zd>.B, { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var result : bits(VL);

let operand1 : bits(VL) = Z{VL}(n+0);
let operand2 : bits(VL) = Z{VL}(n+1);
let operand3 : bits(VL) = Z{VL}(n+2);
let operand4 : bits(VL) = Z{VL}(n+3);
for e = 0 to elements-1 do
    let element1 : bits(32) = operand1[e*:32];
    let element2 : bits(32) = operand2[e*:32];
    let element3 : bits(32) = operand3[e*:32];
    let element4 : bits(32) = operand4[e*:32];
    result[(4*e + 0)*:8] = FPConvertFP8{8, 32}(element1, FPCR(), FPMR());
    result[(4*e + 1)*:8] = FPConvertFP8{8, 32}(element2, FPCR(), FPMR());
    result[(4*e + 2)*:8] = FPConvertFP8{8, 32}(element3, FPCR(), FPMR());
    result[(4*e + 3)*:8] = FPConvertFP8{8, 32}(element4, FPCR(), FPMR());
end;
Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZS

Multi-vector single-precision convert to signed 32-bit integer, rounding toward zero

This instruction converts each element of the two or four source vectors from single-precision to the signed 32-bit integer nearer to zero, and places the results in the corresponding elements of the two or four destination vectors.

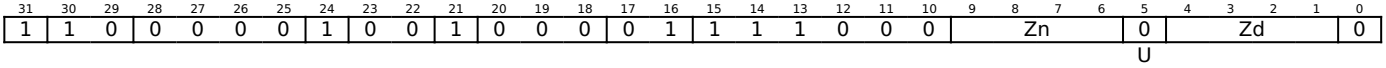
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

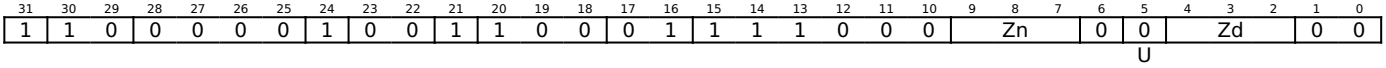
```
FCVTZS { <Zd1>.S-<Zd2>.S }, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
```

Four registers

(FEAT_SME2)



Encoding

```
FCVTZS { <Zd1>.S-<Zd4>.S }, { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn1>	For the "Two registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(32) = operand[e*:32];
        results[[r]][e*:32] = FPToFixed{32, 32}(element, 0, unsigned, FPCR(), rounding);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FCVTZU

Multi-vector single-precision convert to unsigned 32-bit integer, rounding toward zero

This instruction converts each element of the two or four source vectors from single-precision to the unsigned 32-bit integer nearer to zero, and places the results in the corresponding elements of the two or four destination vectors.

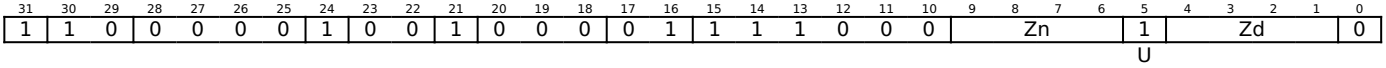
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

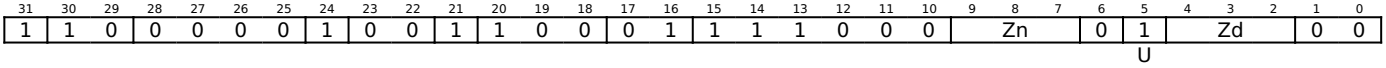
```
FCVTZU { <Zd1>.S-<Zd2>.S }, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
```

Four registers

(FEAT_SME2)



Encoding

```
FCVTZU { <Zd1>.S-<Zd4>.S }, { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRounding_ZERO;
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn1>	For the "Two registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(32) = operand[e*:32];
        results[[r]][e*:32] = FPToFixed{32, 32}(element, 0, unsigned, FPCR(), rounding);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (2-way, multiple and indexed vector, FP16 to FP32)

Multi-vector half-precision dot product by indexed element to single-precision

This instruction calculates the fused sum-of-products of a pair of half-precision values held in the corresponding 32-bit elements of the two or four first source vectors and the indexed 32-bit element of the second source vector, without intermediate rounding. The single-precision sum-of-products are destructively added to the corresponding single-precision elements of the ZA single-vector groups.

The half-precision pairs within the second source vector are specified using an immediate index that selects the same pair position within each 128-bit vector segment. The element index range is from 0 to 3.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

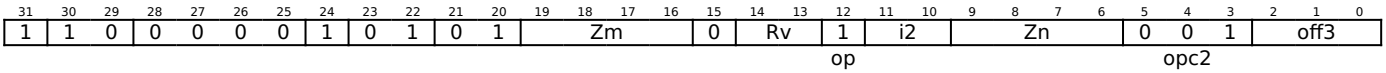
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

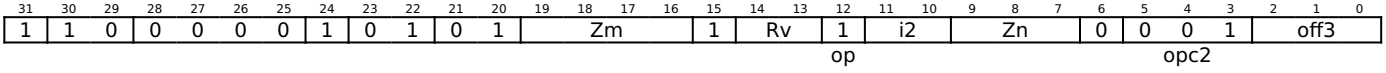
```
FDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
FDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<index> Is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let segmentsize : integer{} = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elt1_a : bits(hsize) = operand1[(2 * e + 0)*:hsize];
        let elt1_b : bits(hsize) = operand1[(2 * e + 1)*:hsize];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let elt2_a : bits(hsize) = operand2[(2 * s + 0)*:hsize];
        let elt2_b : bits(hsize) = operand2[(2 * s + 1)*:hsize];
        var sum : bits(esize) = operand3[e*:esize];
        sum = FPDotAdd_ZA(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (2-way, multiple and indexed vector, FP8 to FP16)

Multi-vector 8-bit floating-point dot product by indexed element to half-precision

This instruction calculates the fused sum-of-products of a pair of 8-bit floating-point values held in the corresponding 16-bit elements of the two or four first source vectors and the indexed 16-bit element of the second source vector. The half-precision sum-of-products are scaled by $2^{-UInt(FPMR.LSCALE[3:0])}$, before being destructively added without intermediate rounding to the corresponding half-precision elements of the ZA single-vector groups. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

The 8-bit floating-point pairs within the second source vector are specified using an immediate index that selects the same pair position within each 128-bit vector segment.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

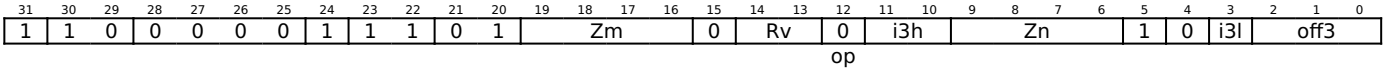
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_F8F16)



Encoding

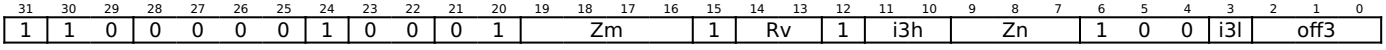
```
FDOT ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_F8F16)



Encoding

```
FDOT ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a pair of 8-bit elements within each 128-bit vector segment, in the range 0 to 7, encoded in the "i3h:i3l" fields.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 8;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let op1 : bits(16) = operand1[e*:16];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let op2 : bits(16) = operand2[s*:16];
        var sum : bits(16) = operand3[e*:16];
        sum = FP8DotAddFP{16, 16}(sum, op1, op2, FPCR(), FPMR());
        result[e*:16] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (2-way, multiple and single vector, FP16 to FP32)

Multi-vector half-precision dot product by vector to single-precision

This instruction calculates the fused sum-of-products of a pair of half-precision values held in the corresponding 32-bit elements of the two or four first source vectors and the second source vector, without intermediate rounding. The single-precision sum-of-products are destructively added to the corresponding single-precision elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

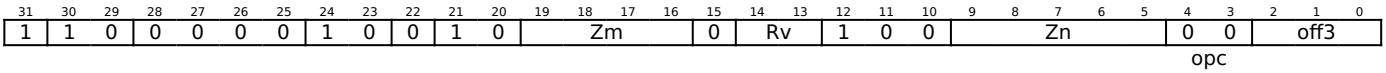
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

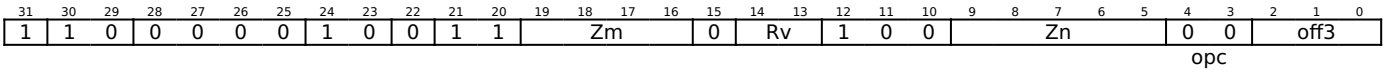
```
FDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
FDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elt1_a : bits(hsize) = operand1[(2 * e + 0)*:hsize];
        let elt1_b : bits(hsize) = operand1[(2 * e + 1)*:hsize];
        let elt2_a : bits(hsize) = operand2[(2 * e + 0)*:hsize];
        let elt2_b : bits(hsize) = operand2[(2 * e + 1)*:hsize];
        var sum : bits(esize) = operand3[e*:esize];
        sum = FPDotAdd_ZA(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (2-way, multiple and single vector, FP8 to FP16)

Multi-vector 8-bit floating-point dot product by vector to half-precision

This instruction calculates the fused sum-of-products of a pair of 8-bit floating-point values held in the corresponding 16-bit elements of the two or four first source vectors and the second source vector. The half-precision sum-of-products are scaled by $2^{-UInt(FPMR.LSCALE[3:0])}$, before being destructively added without intermediate rounding to the corresponding half-precision elements of the ZA single-vector groups. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

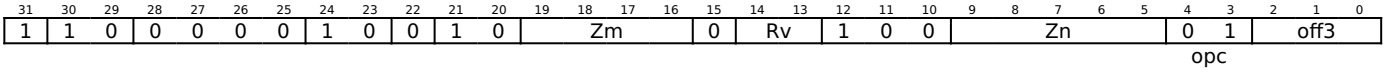
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_F8F16)



Encoding

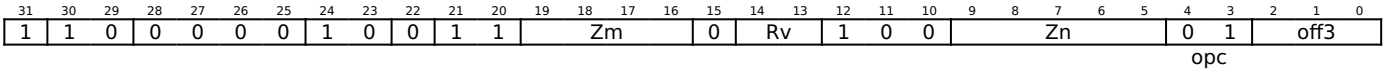
```
FDOT ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_F8F16)



Encoding

```
FDOT ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let op1 : bits(16) = operand1[e*:16];
        let op2 : bits(16) = operand2[e*:16];
        var sum : bits(16) = operand3[e*:16];
        sum = FP8DotAddFP{16, 16}(sum, op1, op2, FPCR(), FPMR());
        result[e*:16] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (2-way, multiple vectors, FP16 to FP32)

Multi-vector half-precision dot product to single-precision

This instruction calculates the fused sum-of-products of a pair of half-precision values held in the corresponding 32-bit elements of the two or four first and second source vectors, without intermediate rounding. The single-precision sum-of-products are destructively added to the corresponding single-precision elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

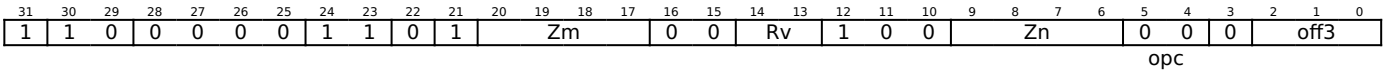
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

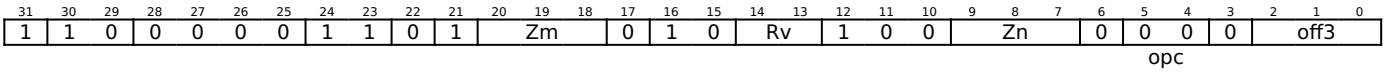
```
FDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
FDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elt1_a : bits(hsize) = operand1[(2 * e + 0)*:hsize];
        let elt1_b : bits(hsize) = operand1[(2 * e + 1)*:hsize];
        let elt2_a : bits(hsize) = operand2[(2 * e + 0)*:hsize];
        let elt2_b : bits(hsize) = operand2[(2 * e + 1)*:hsize];
        var sum : bits(esize) = operand3[e*:esize];
        sum = FPDotAdd_ZA(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (2-way, multiple vectors, FP8 to FP16)

Multi-vector 8-bit floating-point dot product to half-precision

This instruction calculates the fused sum-of-products of a pair of 8-bit floating-point values held in the corresponding 16-bit elements of the two or four first and second source vectors. The half-precision sum-of-products are scaled by $2^{-UInt(FPMR.LSCALE[3:0])}$, before being destructively added without intermediate rounding to the corresponding half-precision elements of the ZA single-vector groups. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

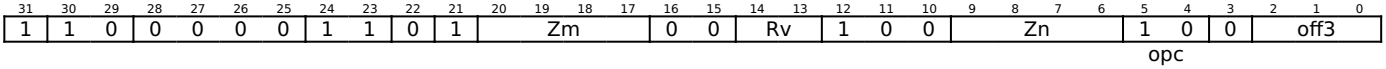
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_F8F16)



Encoding

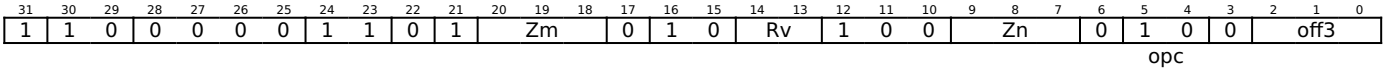
```
FDOT ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_F8F16)



Encoding

```
FDOT ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, { <Zm1>.B-<Zm4>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let op1 : bits(16) = operand1[e*:16];
        let op2 : bits(16) = operand2[e*:16];
        var sum : bits(16) = operand3[e*:16];
        sum = FP8DotAddFP{16, 16}(sum, op1, op2, FPCR(), FPMR());
        result[e*:16] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (4-way, multiple and indexed vector)

Multi-vector 8-bit floating-point dot product by indexed element to single-precision

This instruction calculates the fused sum-of-products of a group of four 8-bit floating-point values held in the corresponding 32-bit elements of the two or four first source vectors and the indexed 32-bit element of the second source vector. The single-precision sum-of-products are scaled by $2^{-UInt(FPMR.LSCALE)}$, before being destructively added without intermediate rounding to the corresponding single-precision elements of the ZA single-vector groups. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

The 8-bit floating-point groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

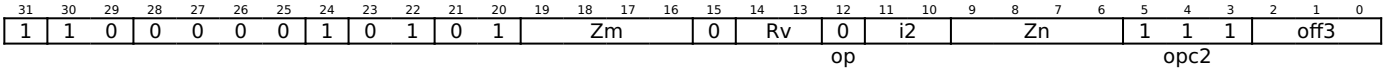
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_F8F32)



Encoding

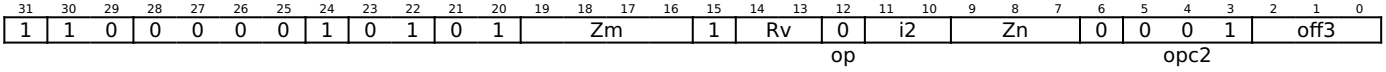
```
FDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_F8F32)



Encoding

```
FDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 4;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let op1 : bits(32) = operand1[e*:32];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let op2 : bits(32) = operand2[s*:32];
        var sum : bits(32) = operand3[e*:32];
        sum = FP8DotAddFP{32, 32}(sum, op1, op2, FPCR(), FPMR());
        result[e*:32] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (4-way, multiple and single vector)

Multi-vector 8-bit floating-point dot product by vector to single-precision

This instruction calculates the fused sum-of-products of a group of four 8-bit floating-point values held in the corresponding 32-bit elements of the two or four first source vectors and the second source vector. The single-precision sum-of-products are scaled by $2^{-\mathrm{UInt(FPMR.LSCALE)}}$, before being destructively added without intermediate rounding to the corresponding single-precision elements of the ZA single-vector groups. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

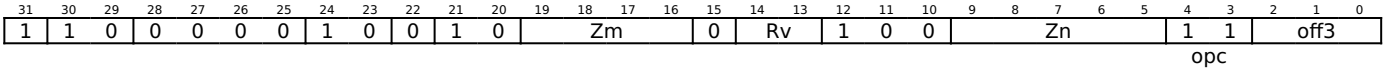
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_F8F32)



Encoding

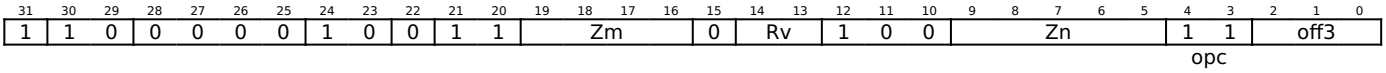
```
FDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_F8F32)



Encoding

```
FDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let op1 : bits(32) = operand1[e*:32];
        let op2 : bits(32) = operand2[e*:32];
        var sum : bits(32) = operand3[e*:32];
        sum = FP8DotAddFP{32, 32}(sum, op1, op2, FPCR(), FPMR());
        result[e*:32] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FDOT (4-way, multiple vectors)

Multi-vector 8-bit floating-point dot product to single-precision

This instruction calculates the fused sum-of-products of a group of four 8-bit floating-point values held in the corresponding 32-bit elements of the two or four first and second source vectors. The single-precision sum-of-products are scaled by $2^{-\mathrm{UInt(FPMR.LSCALE)}}$, before being destructively added without intermediate rounding to the corresponding single-precision elements of the ZA single-vector groups. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

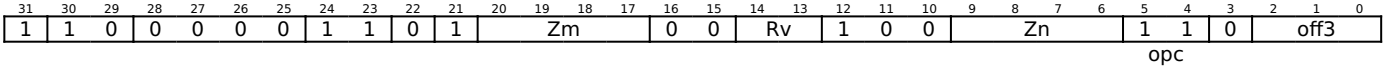
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME_F8F32)



Encoding

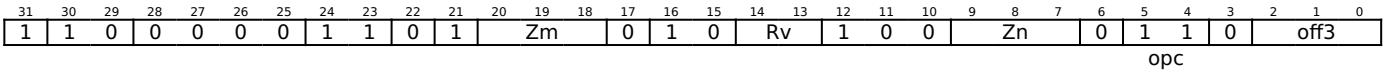
```
FDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME_F8F32)



Encoding

```
FDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, { <Zm1>.B-<Zm4>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let op1 : bits(32) = operand1[e*:32];
        let op2 : bits(32) = operand2[e*:32];
        var sum : bits(32) = operand3[e*:32];
        sum = FP8DotAddFP{32, 32}(sum, op1, op2, FPCR(), FPMR());
        result[e*:32] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAX (multiple and single vector)

Multi-vector floating-point maximum by vector

This instruction determines the maximum of floating-point elements of the second source vector and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

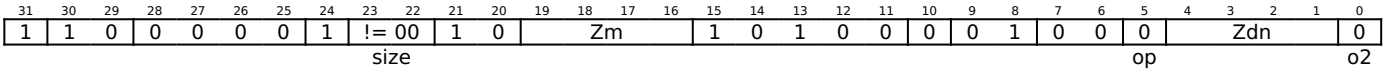
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

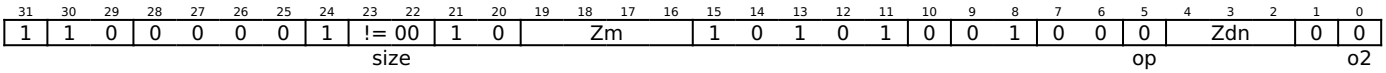
```
FMAX { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMAX { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMax{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAX (multiple vectors)

Multi-vector floating-point maximum

This instruction determines the maximum of floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

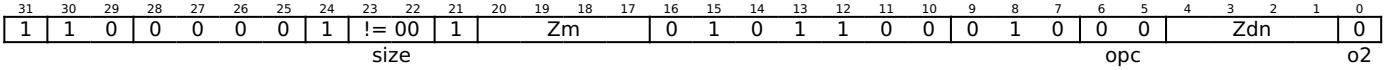
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

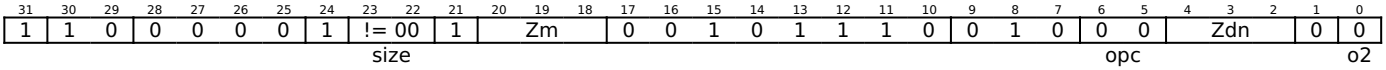
```
FMAX { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMAX { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMax{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNM (multiple and single vector)

Multi-vector floating-point maximum number by vector

This instruction determines the maximum number value of floating-point elements of the second source vector and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

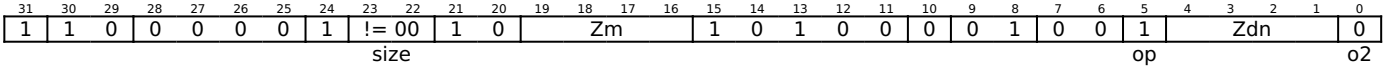
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

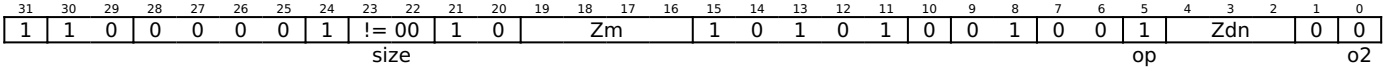
```
FMAXNM { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMAXNM { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMAXNM (multiple vectors)

Multi-vector floating-point maximum number

This instruction determines the maximum number value of floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

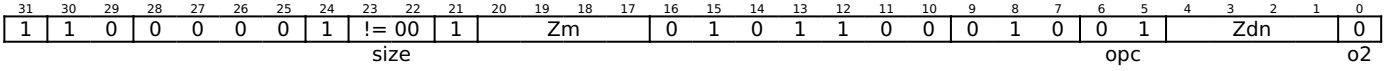
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

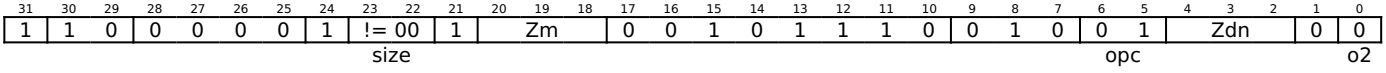
```
FMAXNM { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMAXNM { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMaxNum{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMIN (multiple and single vector)

Multi-vector floating-point minimum by vector

This instruction determines the minimum of floating-point elements of the second source vector and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

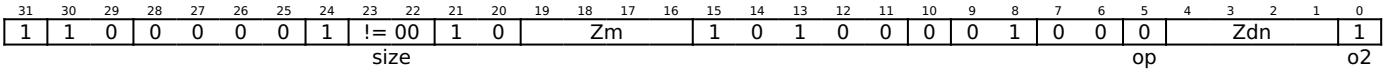
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

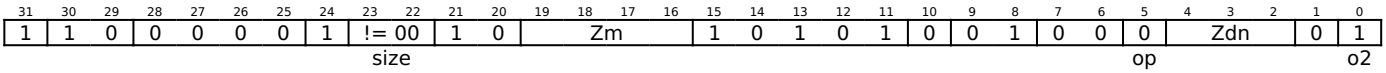
```
FMIN { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMIN { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMin{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMIN (multiple vectors)

Multi-vector floating-point minimum

This instruction determines the minimum of floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

When FPCR.AH is 0, the behavior is as follows:

- Negative zero compares less than positive zero.
- When FPCR.DN is 0, if either element is a NaN, the result is a quiet NaN derived as follows:
  - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
  - Otherwise, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a NaN, the result is default NaN.

When FPCR.AH is 1, the behavior is as follows:

- If both elements are zeros, regardless of the sign of either zero, the result is the second element.
- If either element is a NaN, regardless of the value of FPCR.DN, the result is the second element.

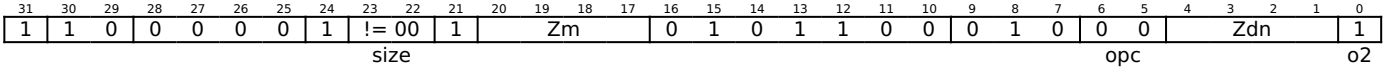
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

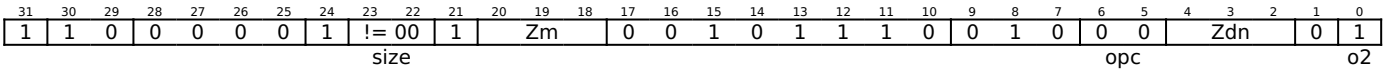
```
FMIN { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMIN { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMin{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNM (multiple and single vector)

Multi-vector floating-point minimum number by vector

This instruction determines the minimum number value of floating-point elements of the second source vector and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

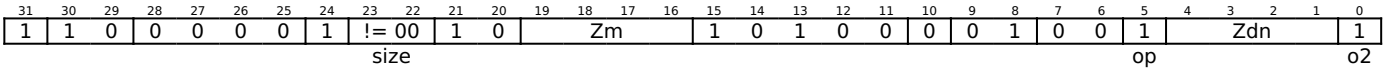
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

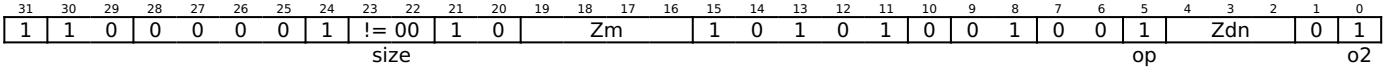
```
FMINNM { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMINNM { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMINNM (multiple vectors)

Multi-vector floating-point minimum number

This instruction determines the minimum number value of floating-point elements of the two or four second source vectors and the corresponding floating-point elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

The behavior is as follows:

- Negative zero compares less than positive zero.
- If one element is numeric and the other is a quiet NaN, the quiet NaN element is ignored and the result is the numeric element.
- When FPCR.DN is 0, if either element is a signaling NaN or if both elements are NaNs, the result is a quiet NaN derived as follows:
  - If FPCR.AH is 0:
    - If at least one element is a signaling NaN, the result is a quiet NaN derived from the first signaling NaN element.
    - Otherwise, the result is a quiet NaN derived from the first NaN element.
  - If FPCR.AH is 1, the result is a quiet NaN derived from the first NaN element.
- When FPCR.DN is 1, if either element is a signaling NaN or if both elements are NaNs, the result is default NaN.

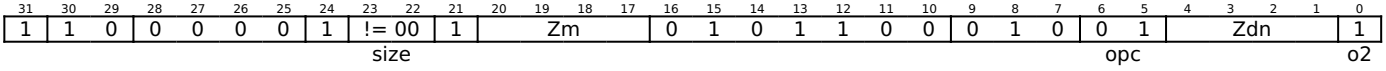
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

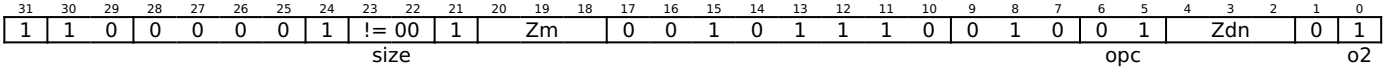
```
FMINNM { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
FMINNM { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2>	Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.
<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zdn4>	Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMinNum{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLA (multiple and indexed vector)

Multi-vector floating-point fused multiply-add by indexed element

This instruction multiplies the indexed element of the second source vector by the corresponding floating-point elements of the two or four first source vectors and destructively adds these values without intermediate rounding to the corresponding elements of the ZA single-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME ZA-targeting floating-point behaviors.

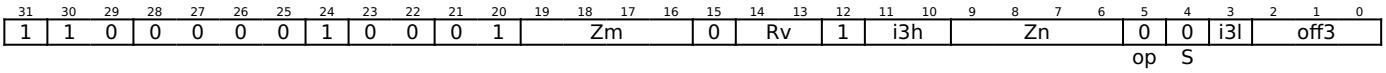
This instruction is unpredicated.

ID_AA64SMFR0_EL1.F64F64 indicates whether the double-precision variant is implemented, and ID_AA64SMFR0_EL1.F16F16 indicates whether the half-precision variant is implemented.

It has encodings from 6 classes: Two ZA single-vectors of half-precision elements , Two ZA single-vectors of single-precision elements , Two ZA single-vectors of double-precision elements , Four ZA single-vectors of half-precision elements , Four ZA single-vectors of single-precision elements and Four ZA single-vectors of double-precision elements

Two ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

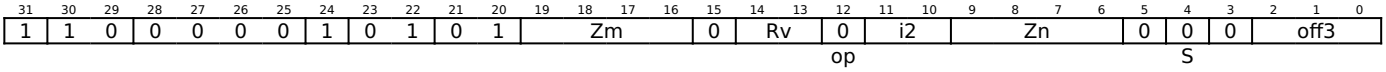
```
FMLA ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Two ZA single-vectors of single-precision elements

(FEAT_SME2)



Encoding

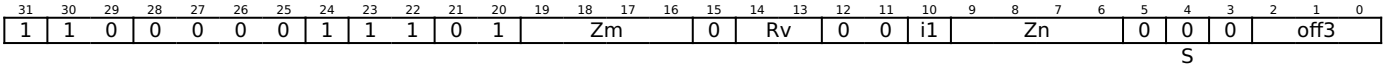
```
FMLA ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.S-<Zn2>.S }, <Zm>.S[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Two ZA single-vectors of double-precision elements

(FEAT_SME2 && FEAT_SME_F64F64)



Encoding

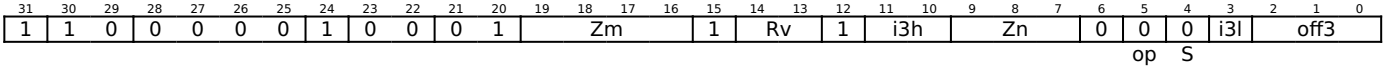
```
FMLA ZA.D[<Wv>, <offs>{, VGx2}], { <Zn1>.D-<Zn2>.D }, <Zm>.D[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_F64F64)) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 64;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i1);
let nreg : integer{} = 2;
```

Four ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

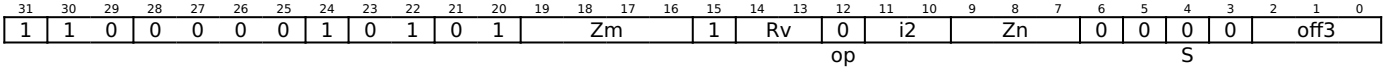
```
FMLA ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Four ZA single-vectors of single-precision elements

(FEAT_SME2)



Encoding

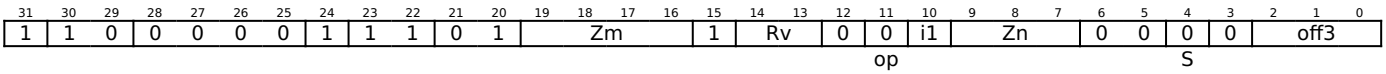
```
FMLA ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.S-<Zn4>.S }, <Zm>.S[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Four ZA single-vectors of double-precision elements

(FEAT_SME2 && FEAT_SME_F64F64)



Encoding

```
FMLA ZA.D[<Wv>, <offs>{, VGx4}], { <Zn1>.D-<Zn4>.D }, <Zm>.D[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_F64F64)) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 64;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i1);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors of double-precision elements", "Two ZA single-vectors of half-precision elements", and "Two ZA single-vectors of single-precision elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors of double-precision elements", "Four ZA single-vectors of half-precision elements", and "Four ZA single-vectors of single-precision elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "Four ZA single-vectors of half-precision elements" and "Two ZA single-vectors of half-precision elements" variants: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "Four ZA single-vectors of single-precision elements" and "Two ZA single-vectors of single-precision elements" variants: is the element index, in the range 0 to 3, encoded in the "i2" field. For the "Four ZA single-vectors of double-precision elements" and "Two ZA single-vectors of double-precision elements" variants: is the element index, in the range 0 to 1, encoded in the "i1" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let segmentsize : integer{} = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = op1[e*:esize];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let elem2 : bits(esize) = op2[s*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = FPMulAdd_ZA{esize}(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLA (multiple and single vector)

Multi-vector floating-point fused multiply-add by vector

This instruction multiplies the corresponding floating-point elements of the two or four first source vector with the corresponding elements of the second source vector and destructively adds these values without intermediate rounding to the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME ZA-targeting floating-point behaviors.

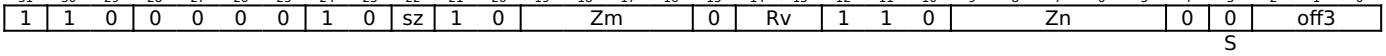
This instruction is unpredicated.

ID_AA64SMFR0_EL1.F64F64 indicates whether the double-precision variant is implemented, and ID_AA64SMFR0_EL1.F16F16 indicates whether the half-precision variant is implemented.

It has encodings from 4 classes: Two ZA single-vectors , Two ZA single-vectors of half-precision elements , Four ZA single-vectors and Four ZA single-vectors of half-precision elements

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

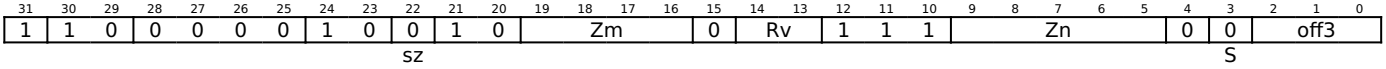
```
FMLA ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<T>-<Zn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Two ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

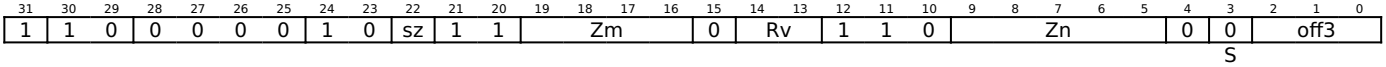
```
FMLA ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

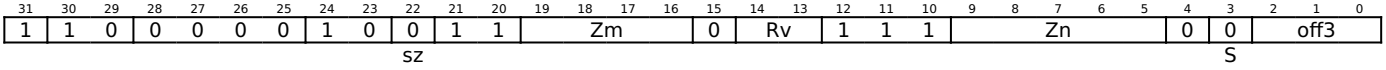
```
FMLA ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<T>-<Zn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Four ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

```
FMLA ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let op2 : bits(VL) = Z{VL}(m);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = op1[e*:esize];
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = FPMulAdd_ZA{esize}(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLA (multiple vectors)

Multi-vector floating-point fused multiply-add

This instruction multiplies the corresponding floating-point elements of the two or four first and second source vectors and destructively adds these values without intermediate rounding to the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME ZA-targeting floating-point behaviors.

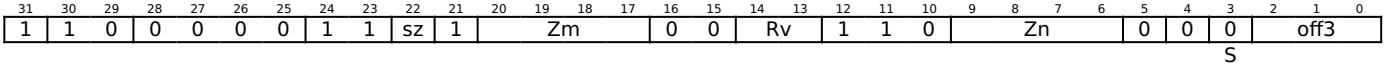
This instruction is unpredicated.

ID_AA64SMFR0_EL1.F64F64 indicates whether the double-precision variant is implemented, and ID_AA64SMFR0_EL1.F16F16 indicates whether the half-precision variant is implemented.

It has encodings from 4 classes: Two ZA single-vectors , Two ZA single-vectors of half-precision elements , Four ZA single-vectors and Four ZA single-vectors of half-precision elements

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

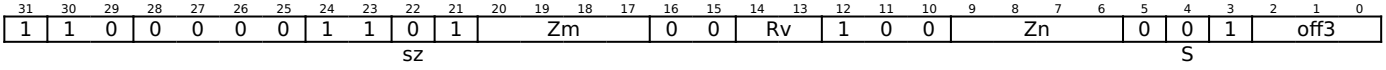
```
FMLA ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<T>-<Zn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Two ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

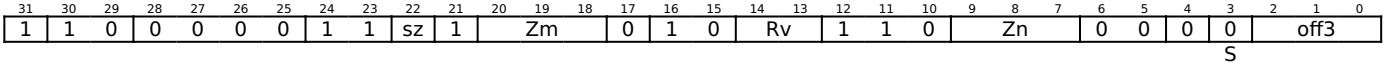
```
FMLA ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

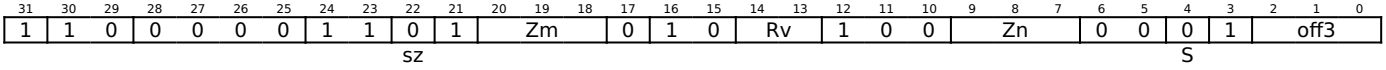
```
FMLA ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<T>-<Zn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Four ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

```
FMLA ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> For the "Two ZA single-vectors of half-precision elements" and "Two ZA single-vectors" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four ZA single-vectors of half-precision elements" and "Four ZA single-vectors" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1> For the "Two ZA single-vectors of half-precision elements" and "Two ZA single-vectors" variants: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA single-vectors of half-precision elements" and "Four ZA single-vectors" variants: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m+r);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = op1[e*:esize];
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = FPMulAdd_ZA{esize}(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLAL (multiple and indexed vector, FP16 to FP32)

Multi-vector half-precision multiply-add by indexed element to single-precision

This instruction widens all 16-bit half-precision elements in the one, two, or four first source vectors and the indexed element of the second source vector to single-precision format, then multiplies the corresponding elements and destructively adds these values without intermediate rounding to the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The half-precision elements within the second source vector are specified using a 3-bit immediate index that selects the same element position within each 128-bit vector segment.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

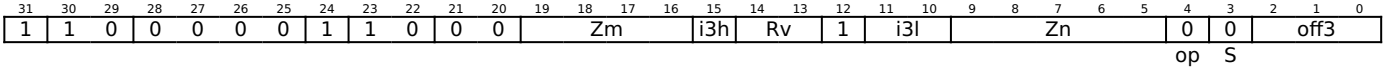
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

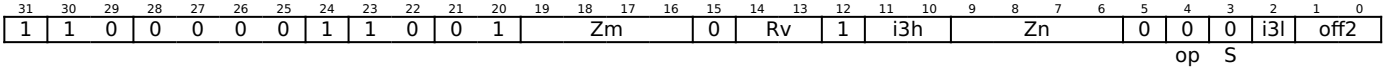
```
FMLAL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

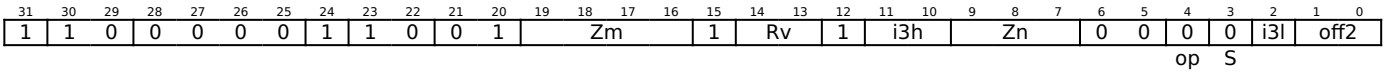
```
FMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
FMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 4;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 2 * segmentbase + index;
            let elem1 : bits(16) = op1[(2 * e + i)*:16];
            let elem2 : bits(16) = op2[s*:16];
            let elem3 : bits(32) = op3[e*:32];
            result[e*:32] = FPMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLAL (multiple and indexed vector, FP8 to FP16)

Multi-vector 8-bit floating-point multiply-add by indexed element to half-precision

This instruction widens all 8-bit floating-point elements in the one, two, or four first source vectors and the indexed element of the second source vector to half-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$ before being destructively added without intermediate rounding to the overlapping 16-bit half-precision elements of the ZA double-vector groups.

The 8-bit floating-point elements within the second source vector are specified using a 4-bit immediate index that selects the same element position within each 128-bit vector segment.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

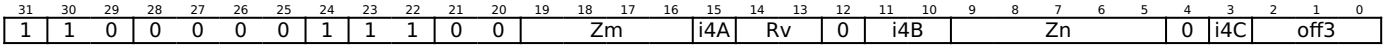
The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME_F8F16)



Encoding

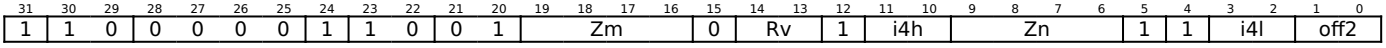
```
FMLAL ZA.H[<Wv>, <offs1>:<offs2>], <Zn>.B, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let index : integer = UInt(i4A::i4B::i4C);
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME_F8F16)



Encoding

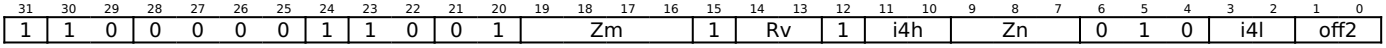
```
FMLAL ZA.H[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME_F8F16)



Encoding

```
FMLAL ZA.H[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "One ZA double-vector" variant: is the element index, in the range 0 to 15, encoded in the "i4A:i4B:i4C" fields. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the element index, in the range 0 to 15, encoded in the "i4h:i4l" fields.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 8;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 2 * segmentbase + index;
            let elem1 : bits(8) = op1[(2 * e + i)*:8];
            let elem2 : bits(8) = op2[s*:8];
            let elem3 : bits(16) = op3[e*:16];
            result[e*:16] = FP8MulAddFP{16}(elem3, elem1, elem2, FPCR(), FPMR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLAL (multiple and single vector, FP16 to FP32)

Multi-vector half-precision multiply-add by vector to single-precision

This instruction widens all 16-bit half-precision elements in the one, two, or four first source vectors and the second source vector to single-precision format, then multiplies the corresponding elements and destructively adds these values without intermediate rounding to the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

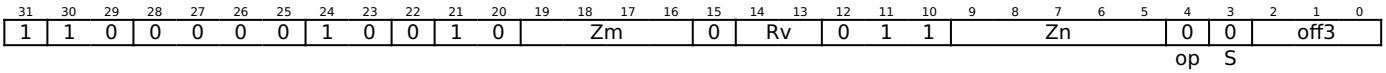
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

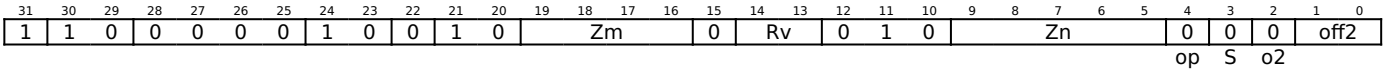
```
FMLAL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

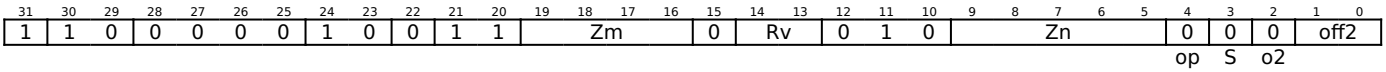
```
FMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
FMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let op2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(16) = op1[(2 * e + i)*:16];
            let elem2 : bits(16) = op2[(2 * e + i)*:16];
            let elem3 : bits(32) = op3[e*:32];
            result[e*:32] = FPMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLAL (multiple and single vector, FP8 to FP16)

Multi-vector 8-bit floating-point multiply-add by vector to half-precision

This instruction widens all 8-bit floating-point elements in the one, two, or four first source vectors and the second source vector to half-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$ before being destructively added without intermediate rounding to the overlapping 16-bit half-precision elements of the ZA double-vector groups.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

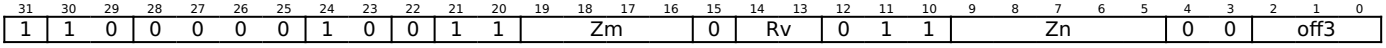
The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

It has encodings from 3 classes: [One ZA double-vector](#) , [Two ZA double-vectors](#) and [Four ZA double-vectors](#)

One ZA double-vector

(FEAT_SME_F8F16)



Encoding

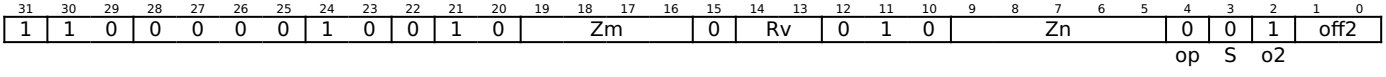
```
FMLAL ZA.H[<Wv>, <offs1>:<offs2>], <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME_F8F16)



Encoding

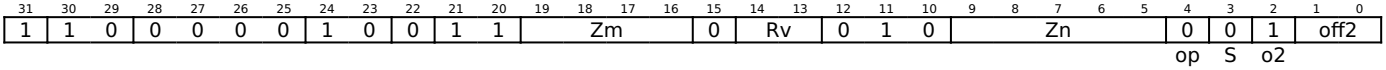
```
FMLAL ZA.H[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME_F8F16)



Encoding

```
FMLAL ZA.H[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let op2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(8) = op1[(2 * e + i)*:8];
            let elem2 : bits(8) = op2[(2 * e + i)*:8];
            let elem3 : bits(16) = op3[e*:16];
            result[e*:16] = FP8MulAddFP{16}(elem3, elem1, elem2, FPCR(), FPMR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLAL (multiple vectors, FP16 to FP32)

Multi-vector half-precision multiply-add to single-precision

This instruction widens all 16-bit half-precision elements in the two or four first and second source vectors to single-precision format, then multiplies the corresponding elements and destructively adds these values without intermediate rounding to the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The double-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

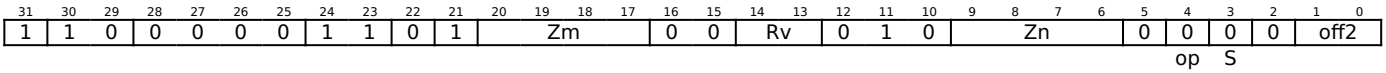
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA double-vectors and Four ZA double-vectors

Two ZA double-vectors

(FEAT_SME2)



Encoding

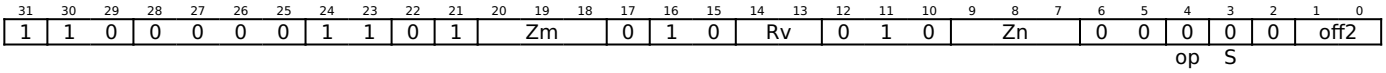
```
FMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
FMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "off2" field times 2.
<offs2>	Is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(16) = op1[(2 * e + i)*:16];
            let elem2 : bits(16) = op2[(2 * e + i)*:16];
            let elem3 : bits(32) = op3[e*:32];
            result[e*:32] = FPMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLAL (multiple vectors, FP8 to FP16)

Multi-vector 8-bit floating-point multiply-add to half-precision

This instruction widens all 8-bit floating-point elements in the two or four first and second source vectors to half-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$ before being destructively added without intermediate rounding to the overlapping 16-bit half-precision elements of the ZA double-vector groups.

The double-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

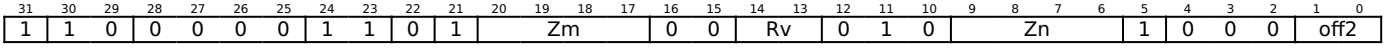
The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA double-vectors and Four ZA double-vectors

Two ZA double-vectors

(FEAT_SME_F8F16)



Encoding

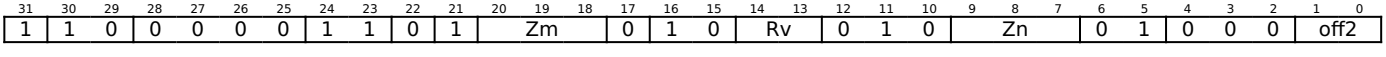
```
FMLAL ZA.H[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME_F8F16)



Encoding

```
FMLAL ZA.H[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.B-<Zn4>.B }, { <Zm1>.B-<Zm4>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "off2" field times 2.
<offs2>	Is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 16;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(8) = op1[(2 * e + i)*:8];
            let elem2 : bits(8) = op2[(2 * e + i)*:8];
            let elem3 : bits(16) = op3[e*:16];
            result[e*:16] = FP8MulAddFP{16}(elem3, elem1, elem2, FPCR(), FPMR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALL (multiple and indexed vector)

Multi-vector 8-bit floating-point multiply-add by indexed element to single-precision

This instruction widens all 8-bit floating-point elements in the one, two, or four first source vectors and the indexed element of the second source vector to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-\mathrm{UInt(FPMR.LSCALE)}}$ before being destructively added without intermediate rounding to the overlapping 32-bit single-precision elements of the ZA quad-vector groups.

The 8-bit floating-point elements within the second source vector are specified using a 4-bit immediate index that selects the same element position within each 128-bit vector segment.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

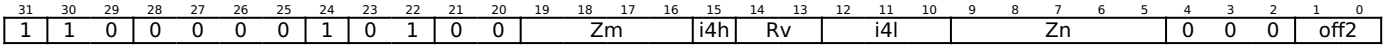
The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA quad-vector , Two ZA quad-vectors and Four ZA quad-vectors

One ZA quad-vector

(FEAT_SME_F8F32)



Encoding

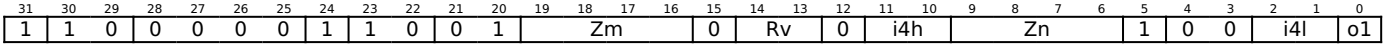
```
FMLALL ZA.S[<Wv>, <offs1>:<offs4>], <Zn>.B, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let esize : integer{} = 32;
let qsize : integer{} = esize DIV 4;
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 1;
```

Two ZA quad-vectors

(FEAT_SME_F8F32)



Encoding

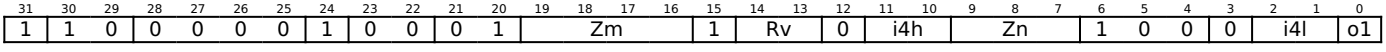
```
FMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let esize : integer{} = 32;
let qsize : integer{} = esize DIV 4;
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME_F8F32)



Encoding

```
FMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let esize : integer{} = 32;
let qsize : integer{} = esize DIV 4;
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA quad-vector" variant: is the first vector select offset, encoded as "off2" field times 4. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the first vector select offset, encoded as "o1" field times 4.
<offs4>	For the "One ZA quad-vector" variant: is the fourth vector select offset, encoded as "off2" field times 4 plus 3. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 15, encoded in the "i4h:i4l" fields.
<Zn1>	For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let segmentsize : integer = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 4 * segmentbase + index;
            let element1 : bits(qsize) = operand1[(4 * e + i)*:qsize];
            let element2 : bits(qsize) = operand2[s*:qsize];
            let element3 : bits(esize) = operand3[e*:esize];
            result[e*:esize] = FP8MulAddFP{32}(element3, element1, element2,
                                               FPCR(), FPMR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALL (multiple and single vector)

Multi-vector 8-bit floating-point multiply-add by vector to single-precision

This instruction widens all 8-bit floating-point elements in the one, two, or four first source vectors and the second source vector to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-UInt(FPMR.LSCALE)}$ before being destructively added without intermediate rounding to the overlapping 32-bit single-precision elements of the ZA quad-vector groups.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA quad-vector , Two ZA quad-vectors and Four ZA quad-vectors

One ZA quad-vector

(FEAT_SME_F8F32)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	Zm				0	Rv		0	0	1	Zn					0	0	0	off2	

Encoding

```
FMLALL ZA.S[<Wv>, <offs1>:<offs4>], <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let esize : integer{} = 32;
let qsize : integer{} = esize DIV 4;
let nreg : integer{} = 1;
```

Two ZA quad-vectors

(FEAT_SME_F8F32)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	0	Zm				0	Rv		0	0	0	Zn					0	0	0	1	o1

Encoding

```
FMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let esize : integer{} = 32;
let qsize : integer{} = esize DIV 4;
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME_F8F32)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	Zm				0	Rv		0	0	0	Zn					0	0	0	1	o1

Encoding

```
FMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let esize : integer{} = 32;
let qsize : integer{} = esize DIV 4;
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA quad-vector" variant: is the first vector select offset, encoded as "off2" field times 4. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the first vector select offset, encoded as "o1" field times 4.
<offs4>	For the "One ZA quad-vector" variant: is the fourth vector select offset, encoded as "off2" field times 4 plus 3. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let element1 : bits(qsize) = operand1[(4 * e + i)*:qsize];
            let element2 : bits(qsize) = operand2[(4 * e + i)*:qsize];
            let element3 : bits(esize) = operand3[e*:esize];
            result[e*:esize] = FP8MulAddFP{32}(element3, element1, element2,
                                               FPCR(), FPMR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLALL (multiple vectors)

Multi-vector 8-bit floating-point multiply-add to single-precision

This instruction widens all 8-bit floating-point elements in the two or four first and second source vectors to single-precision format and multiplies the corresponding elements. The intermediate products are scaled by $2^{-UInt(FPMR.LSCALE)}$ before being destructively added without intermediate rounding to the overlapping 32-bit single-precision elements of the ZA quad-vector groups.

The quad-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

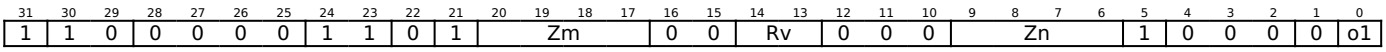
The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA quad-vectors and Four ZA quad-vectors

Two ZA quad-vectors

(FEAT_SME_F8F32)



Encoding

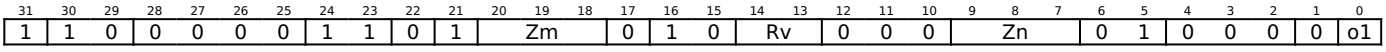
```
FMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(o1::'00');
let esize : integer{} = 32;
let qsize : integer{} = esize DIV 4;
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME_F8F32)



Encoding

```
FMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, { <Zm1>.B-<Zm4>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(o1::'00');
let esize : integer{} = 32;
let qsize : integer{} = esize DIV 4;
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "o1" field times 4.
<offs4>	Is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn1>	For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let element1 : bits(qsize) = operand1[(4 * e + i)*:qsize];
            let element2 : bits(qsize) = operand2[(4 * e + i)*:qsize];
            let element3 : bits(esize) = operand3[e*:esize];
            result[e*:esize] = FP8MulAddFP{32}(element3, element1, element2,
                                               FPCR(), FPMR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLS (multiple and indexed vector)

Multi-vector floating-point fused multiply-subtract by indexed element

This instruction multiplies the indexed element of the second source vector by the corresponding floating-point elements of the two or four first source vectors and destructively subtracts these values without intermediate rounding from the corresponding elements of the ZA single-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME ZA-targeting floating-point behaviors.

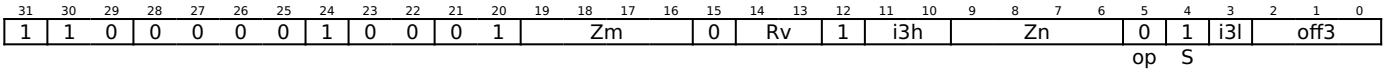
This instruction is unpredicated.

ID_AA64SMFR0_EL1.F64F64 indicates whether the double-precision variant is implemented, and ID_AA64SMFR0_EL1.F16F16 indicates whether the half-precision variant is implemented.

It has encodings from 6 classes: Two ZA single-vectors of half-precision elements , Two ZA single-vectors of single-precision elements , Two ZA single-vectors of double-precision elements , Four ZA single-vectors of half-precision elements , Four ZA single-vectors of single-precision elements and Four ZA single-vectors of double-precision elements

Two ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

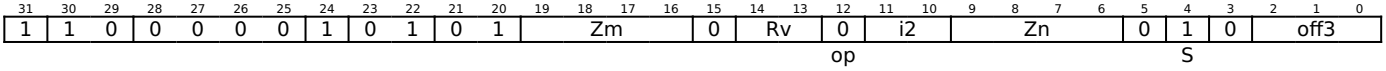
```
FMLS ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Two ZA single-vectors of single-precision elements

(FEAT_SME2)



Encoding

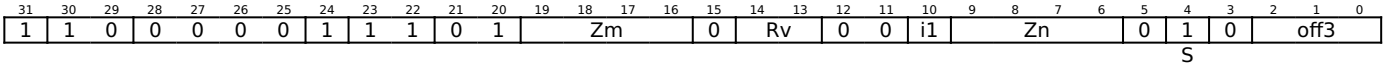
```
FMLS ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.S-<Zn2>.S }, <Zm>.S[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Two ZA single-vectors of double-precision elements

(FEAT_SME2 && FEAT_SME_F64F64)



Encoding

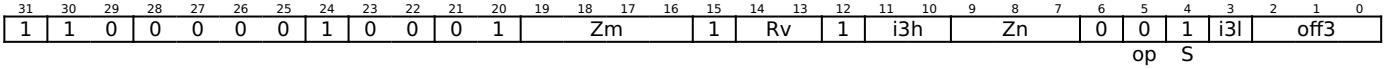
```
FMLS ZA.D[<Wv>, <offs>{, VGx2}], { <Zn1>.D-<Zn2>.D }, <Zm>.D[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_F64F64)) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 64;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i1);
let nreg : integer{} = 2;
```

Four ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

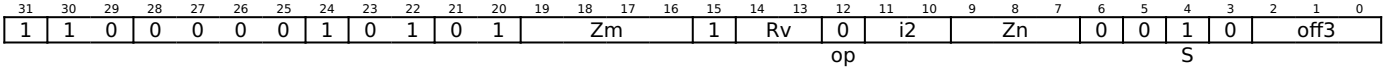
```
FMLS ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Four ZA single-vectors of single-precision elements

(FEAT_SME2)



Encoding

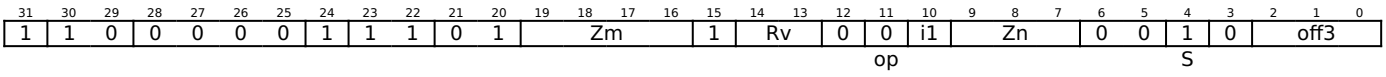
```
FMLS ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.S-<Zn4>.S }, <Zm>.S[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Four ZA single-vectors of double-precision elements

(FEAT_SME2 && FEAT_SME_F64F64)



Encoding

```
FMLS ZA.D[<Wv>, <offs>{, VGx4}], { <Zn1>.D-<Zn4>.D }, <Zm>.D[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_F64F64)) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 64;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i1);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors of double-precision elements", "Two ZA single-vectors of half-precision elements", and "Two ZA single-vectors of single-precision elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors of double-precision elements", "Four ZA single-vectors of half-precision elements", and "Four ZA single-vectors of single-precision elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "Four ZA single-vectors of half-precision elements" and "Two ZA single-vectors of half-precision elements" variants: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields. For the "Four ZA single-vectors of single-precision elements" and "Two ZA single-vectors of single-precision elements" variants: is the element index, in the range 0 to 3, encoded in the "i2" field. For the "Four ZA single-vectors of double-precision elements" and "Two ZA single-vectors of double-precision elements" variants: is the element index, in the range 0 to 1, encoded in the "i1" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let segmentsize : integer{} = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = FPNeg{esize}(op1[e*:esize], FPCR());
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let elem2 : bits(esize) = op2[s*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = FPMulAdd_ZA{esize}(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLS (multiple and single vector)

Multi-vector floating-point fused multiply-subtract by vector

This instruction multiplies the corresponding floating-point elements of the two or four first source vector with the corresponding elements of the second source vector and destructively subtracts these values without intermediate rounding from the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME ZA-targeting floating-point behaviors.

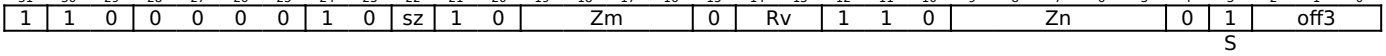
This instruction is unpredicated.

ID_AA64SMFR0_EL1.F64F64 indicates whether the double-precision variant is implemented, and ID_AA64SMFR0_EL1.F16F16 indicates whether the half-precision variant is implemented.

It has encodings from 4 classes: Two ZA single-vectors , Two ZA single-vectors of half-precision elements , Four ZA single-vectors and Four ZA single-vectors of half-precision elements

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

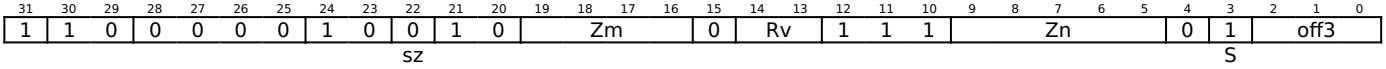
```
FMLS ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<T>-<Zn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Two ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

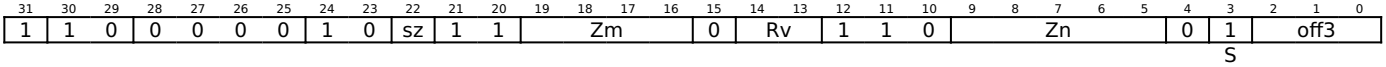
```
FMLS ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

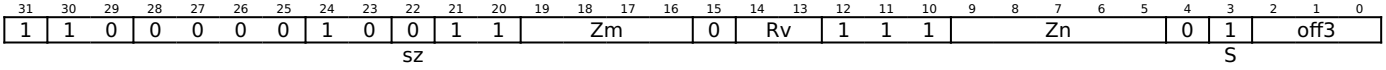
```
FMLS ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<T>-<Zn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Four ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

```
FMLS ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let op2 : bits(VL) = Z{VL}(m);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = FPNeg{esize}(op1[e*:esize], FPCR());
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = FPMulAdd_ZA{esize}(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLS (multiple vectors)

Multi-vector floating-point fused multiply-subtract

This instruction multiplies the corresponding floating-point elements of the two or four first and second source vectors and destructively subtracts these values without intermediate rounding from the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME ZA-targeting floating-point behaviors.

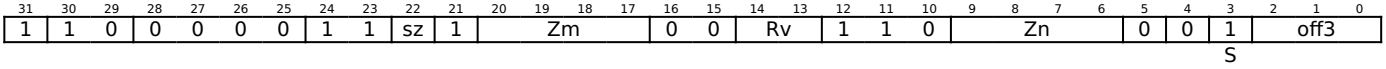
This instruction is unpredicated.

ID_AA64SMFR0_EL1.F64F64 indicates whether the double-precision variant is implemented, and ID_AA64SMFR0_EL1.F16F16 indicates whether the half-precision variant is implemented.

It has encodings from 4 classes: Two ZA single-vectors , Two ZA single-vectors of half-precision elements , Four ZA single-vectors and Four ZA single-vectors of half-precision elements

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

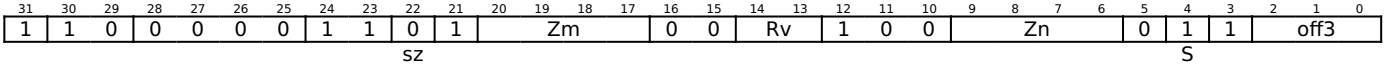
```
FMLS ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<T>-<Zn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Two ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

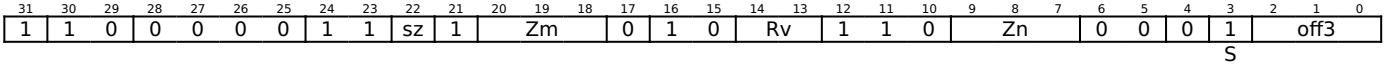
```
FMLS ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

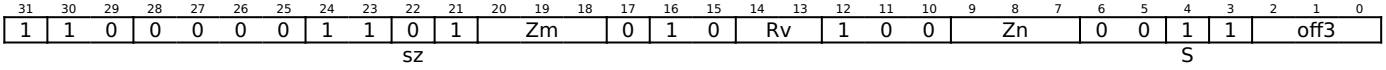
```
FMLS ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<T>-<Zn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Four ZA single-vectors of half-precision elements

(FEAT_SME_F16F16)



Encoding

```
FMLS ZA.H[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> For the "Two ZA single-vectors of half-precision elements" and "Two ZA single-vectors" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four ZA single-vectors of half-precision elements" and "Four ZA single-vectors" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1> For the "Two ZA single-vectors of half-precision elements" and "Two ZA single-vectors" variants: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA single-vectors of half-precision elements" and "Four ZA single-vectors" variants: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m+r);
    let op3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let elem1 : bits(esize) = FPNeg{esize}(op1[e*:esize], FPCR());
        let elem2 : bits(esize) = op2[e*:esize];
        let elem3 : bits(esize) = op3[e*:esize];
        result[e*:esize] = FPMulAdd_ZA{esize}(elem3, elem1, elem2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLSL (multiple and indexed vector)

Multi-vector half-precision multiply-subtract by indexed element from single-precision

This instruction widens all 16-bit half-precision elements in the one, two, or four first source vectors and the indexed element of the second source vector to single-precision format, then multiplies the corresponding elements and destructively subtracts these values without intermediate rounding from the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The half-precision elements within the second source vector are specified using a 3-bit immediate index that selects the same element position within each 128-bit vector segment.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

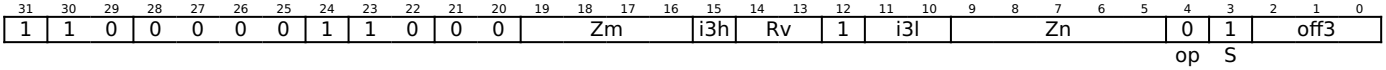
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

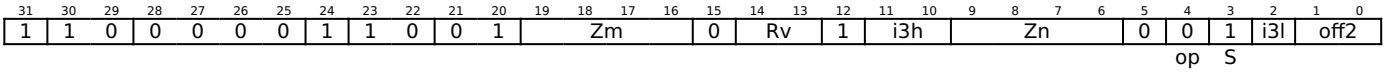
```
FMLSL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

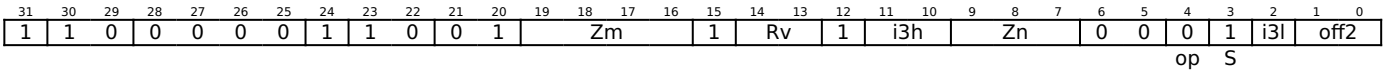
```
FMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
FMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 4;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 2 * segmentbase + index;
            let elem1 : bits(16) = FPNeg{16}(op1[(2 * e + i)*:16], FPCR());
            let elem2 : bits(16) = op2[s*:16];
            let elem3 : bits(32) = op3[e*:32];
            result[e*:32] = FPMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLSL (multiple and single vector)

Multi-vector half-precision multiply-subtract by vector from single-precision

This instruction widens all 16-bit half-precision elements in the one, two, or four first source vectors and the second source vector to single-precision format, then multiplies the corresponding elements and destructively subtracts these values without intermediate rounding from the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19–16	15	14–13	12	11	10	9–5	4	3	2–0
1	1	0	0	0	0	0	1	0	0	1	0	Zm	0	Rv	0	1	1	Zn	0	1	off3
																			op	S	

Encoding

```
FMLSL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19–16	15	14–13	12	11	10	9–5	4	3	2	1–0
1	1	0	0	0	0	0	1	0	0	1	0	Zm	0	Rv	0	1	0	Zn	0	1	0	off2
																			op	S	o2	

Encoding

```
FMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19–16	15	14–13	12	11	10	9–5	4	3	2	1–0
1	1	0	0	0	0	0	1	0	0	1	1	Zm	0	Rv	0	1	0	Zn	0	1	0	off2
																			op	S	o2	

Encoding

```
FMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let op2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(16) = FPNeg{16}(op1[(2 * e + i)*:16], FPCR());
            let elem2 : bits(16) = op2[(2 * e + i)*:16];
            let elem3 : bits(32) = op3[e*:32];
            result[e*:32] = FPMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMLSL (multiple vectors)

Multi-vector half-precision multiply-subtract from single-precision

This instruction widens all 16-bit half-precision elements in the two or four first and second source vectors to single-precision format, then multiplies the corresponding elements and destructively subtracts these values without intermediate rounding from the overlapping 32-bit single-precision elements of the ZA double-vector groups.

The double-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

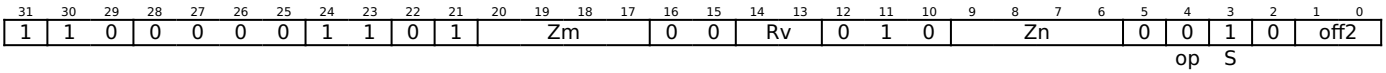
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA double-vectors and Four ZA double-vectors

Two ZA double-vectors

(FEAT_SME2)



Encoding

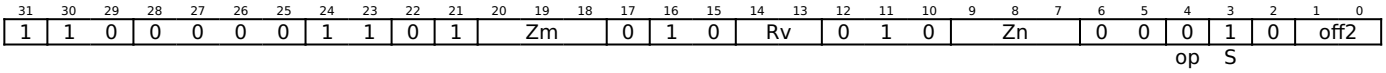
```
FMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
FMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "off2" field times 2.
<offs2>	Is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let op1 : bits(VL) = Z{VL}(n+r);
    let op2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 1 do
        let op3 : bits(VL) = ZAvector{VL}(vec + i);
        for e = 0 to elements-1 do
            let elem1 : bits(16) = FPNeg{16}(op1[(2 * e + i)*:16], FPCR());
            let elem2 : bits(16) = op2[(2 * e + i)*:16];
            let elem3 : bits(32) = op3[e*:32];
            result[e*:32] = FPMulAddH_ZA(elem3, elem1, elem2, FPCR());
        end;
        ZAvector{VL}(vec + i) = result;
    end;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOP4A (non-widening)

Floating-point quarter-tile outer product, accumulating

This instruction generates four independent quarter-tile floating-point outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 16-bit, 32-bit, or 64-bit element ZA tile.

In case of the half-precision variant, each of the quarter-tile outer products is generated by multiplying the $SVL_H \div 2 \times 1$ sub-matrix of half-precision values held in the half-vectors of the first source vectors by the $1 \times SVL_H \div 2$ sub-matrix of half-precision values held in the half-vectors of the second source vectors. In case of the single-precision variant, each of the quarter-tile outer products is generated by multiplying the $SVL_S \div 2 \times 1$ sub-matrix of single-precision values held in the half-vectors of the first source vectors by the $1 \times SVL_S \div 2$ sub-matrix of single-precision values held in the half-vectors of the second source vectors. In case of the double-precision variant, each of the quarter-tile outer products is generated by multiplying the $SVL_D \div 2 \times 1$ sub-matrix of double-precision values held in the half-vectors of the first source vectors by the $1 \times SVL_D \div 2$ sub-matrix of double-precision values held in the half-vectors of the second source vectors.

The resulting quarter-tile $SVL_H \div 2 \times SVL_H \div 2$ half-precision outer products in case of the half-precision variant, $SVL_S \div 2 \times SVL_S \div 2$ single-precision outer products in case of the single-precision variant, or $SVL_D \div 2 \times SVL_D \div 2$ double-precision outer products in case of the double-precision variant are destructively added to the destination ZA tile. This is equivalent to performing a single multiply-accumulate to each of the destination tile elements.

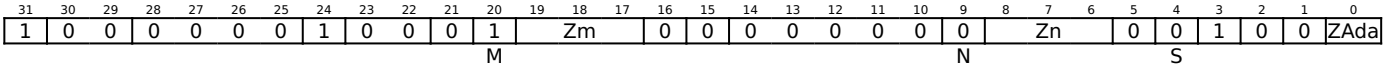
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 12 classes: Half-precision, single and multiple vectors , Half-precision, single vectors , Half-precision, multiple and single vectors , Half-precision, multiple vectors , Single-precision, single and multiple vectors , Single-precision, single vectors , Single-precision, multiple and single vectors , Single-precision, multiple vectors , Double-precision, single and multiple vectors , Double-precision, single vectors , Double-precision, multiple and single vectors and Double-precision, multiple vectors

Half-precision, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F16F16)



Encoding

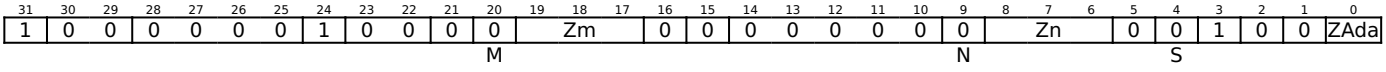
```
FMOP4A <ZAda>.H, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F16F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 16;
```

Half-precision, single vectors

(FEAT_SME_MOP4 && FEAT_SME_F16F16)



Encoding

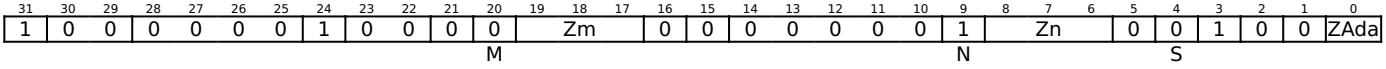
```
FMOP4A <ZAda>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F16F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 16;
```

Half-precision, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_F16F16)



Encoding

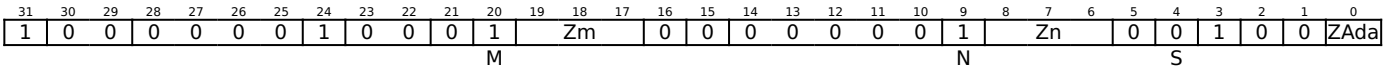
```
FMOP4A <ZAda>.H, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F16F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 16;
```

Half-precision, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F16F16)



Encoding

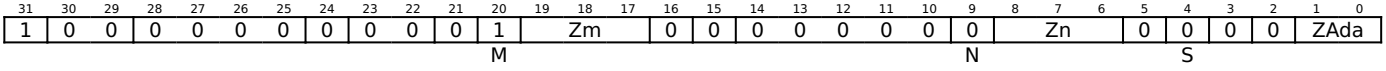
```
FMOP4A <ZAda>.H, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F16F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 16;
```

Single-precision, single and multiple vectors

(FEAT_SME_MOP4)



Encoding

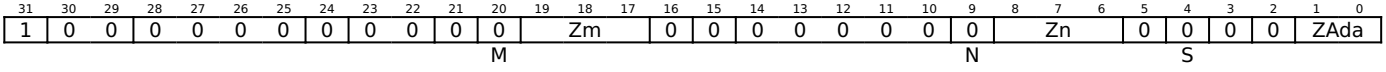
```
FMOP4A <ZAda>.S, <Zn>.S, { <Zm1>.S-<Zm2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 32;
```

Single-precision, single vectors

(FEAT_SME_MOP4)



Encoding

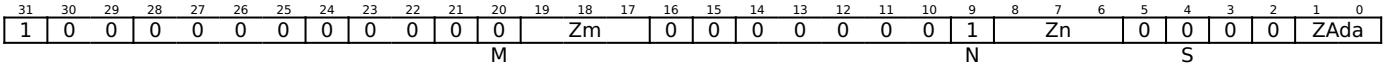
```
FMOP4A <ZAda>.S, <Zn>.S, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 32;
```

Single-precision, multiple and single vectors

(FEAT_SME_MOP4)



Encoding

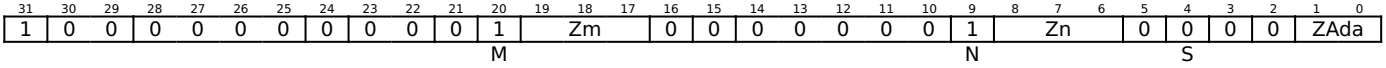
```
FMOP4A <ZAda>.S, { <Zn1>.S-<Zn2>.S }, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 32;
```

Single-precision, multiple vectors

(FEAT_SME_MOP4)



Encoding

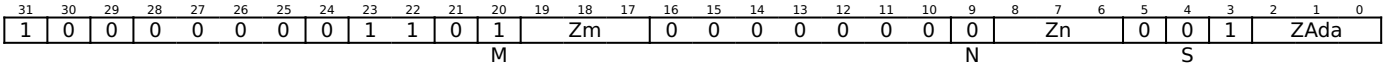
```
FMOP4A <ZAda>.S, { <Zn1>.S-<Zn2>.S }, { <Zm1>.S-<Zm2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 32;
```

Double-precision, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F64F64)



Encoding

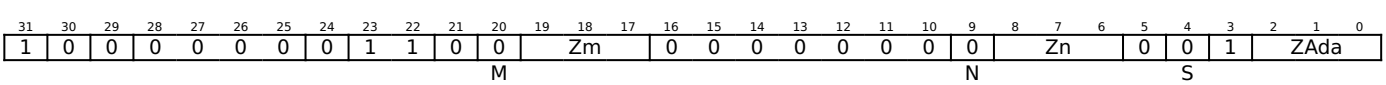
```
FMOP4A <ZAda>.D, <Zn>.D, { <Zm1>.D-<Zm2>.D }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F64F64) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 64;
```

Double-precision, single vectors

(FEAT_SME_MOP4 && FEAT_SME_F64F64)



Encoding

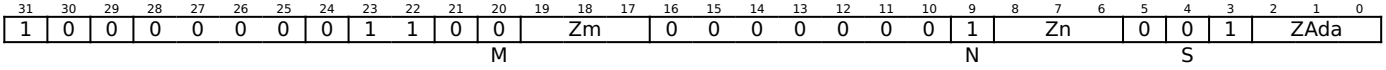
```
FMOP4A <ZAda>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F64F64) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 64;
```

Double-precision, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_F64F64)



Encoding

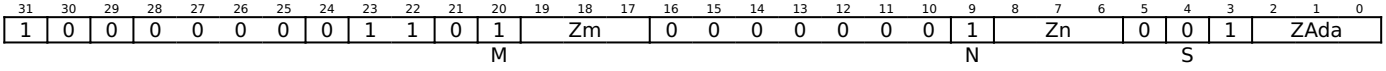
```
FMOP4A <ZAda>.D, { <Zn1>.D-<Zn2>.D }, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F64F64) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 64;
```

Double-precision, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F64F64)



Encoding

```
FMOP4A <ZAda>.D, { <Zn1>.D-<Zn2>.D }, { <Zm1>.D-<Zm2>.D }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F64F64) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 64;
```

Assembler Symbols

<ZAda>	For the "Half-precision, multiple and single vectors", "Half-precision, multiple vectors", "Half-precision, single and multiple vectors", and "Half-precision, single vectors" variants: is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field. For the "Single-precision, multiple and single vectors", "Single-precision, multiple vectors", "Single-precision, single and multiple vectors", and "Single-precision, single vectors" variants: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "Double-precision, multiple and single vectors", "Double-precision, multiple vectors", "Double-precision, single and multiple vectors", and "Double-precision, single vectors" variants: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.

<Zm> Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{tilesize}(da, esize);
var result : bits(tilesize);

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{VL}(n + (nreg-1) * col_hv);
    let op2 : bits(VL) = Z{VL}(m + (mreg-1) * row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            let elem1 : bits(esize) = op1[row_idx*:esize];
            let elem2 : bits(esize) = op2[col_idx*:esize];
            let elem3 : bits(esize) = op3[tile_idx*:esize];

            result[tile_idx*:esize] = FPMulAdd_ZA{esize}(elem3, elem1, elem2, FPCR());
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOP4A (widening, 2-way, FP16 to FP32)

Half-precision quarter-tile sum of outer products to single-precision, accumulating

This instruction generates four independent quarter-tile half-precision sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

Each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 2$ sub-matrix of half-precision values held in the half-vectors of the first source vectors by the $2 \times SVL_S \div 2$ sub-matrix of half-precision values held in the half-vectors of the second source vectors. Each 32-bit container of the first source vectors holds 2 elements of each row of a $SVL_S \div 2 \times 2$ sub-matrix. Similarly, each 32-bit container of the second source vectors holds 2 elements of each column of a $2 \times SVL_S \div 2$ sub-matrix.

The instruction widens the sub-matrices of half-precision values held in the first source vectors to single-precision values and multiplies them by the corresponding widened sub-matrices of half-precision values in the second source vectors to single-precision values. The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ single-precision sums of outer products are then destructively added to the single-precision destination tile. This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.

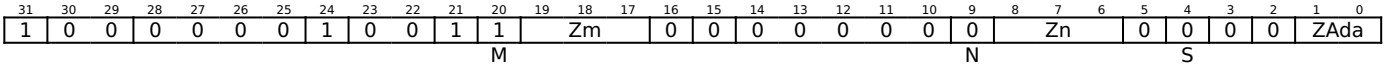
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4)



Encoding

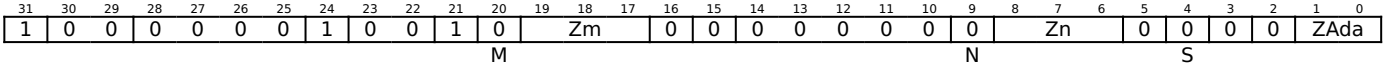
```
FMOP4A <ZAda>.S, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Single vectors

(FEAT_SME_MOP4)



Encoding

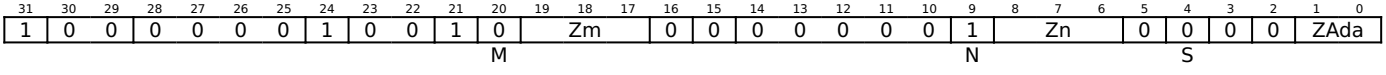
```
FMOP4A <ZAda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple and single vectors

(FEAT_SME_MOP4)



Encoding

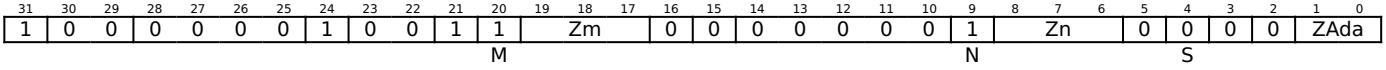
```
FMOP4A <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple vectors

(FEAT_SME_MOP4)



Encoding

```
FMOP4A <ZAda>.S, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV 32;
let tilesize : integer{} = 4 * dim * dim * 32;
let op3 : bits(tilesize) = ZAtile{tilesize}(da, 32);
var result : bits(tilesize);

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{VL}(n + (nreg-1) * col_hv);
    let op2 : bits(VL) = Z{VL}(m + (mreg-1) * row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            let sum : bits(32) = op3[tile_idx*:32];
            let erow_0 : bits(16) = op1[(2 * row_idx + 0)*:16];
            let erow_1 : bits(16) = op1[(2 * row_idx + 1)*:16];
            let ecol_0 : bits(16) = op2[(2 * col_idx + 0)*:16];
            let ecol_1 : bits(16) = op2[(2 * col_idx + 1)*:16];
            result[tile_idx*:32] = FPDotAdd_ZA(sum, erow_0, erow_1, ecol_0, ecol_1, FPCR());
        end;
    end;
end;
ZAtile{tilesize}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOP4A (widening, 2-way, FP8 to FP16)

8-bit floating-point quarter-tile sum of outer products to half-precision, accumulating

This instruction generates four independent quarter-tile 8-bit floating-point sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 16-bit element ZA tile.

Each of the quarter-tile sums of outer products is generated by multiplying the $SVL_H \div 2 \times 2$ sub-matrix of 8-bit floating-point values held in the half-vectors of the first source vectors by the $2 \times SVL_H \div 2$ sub-matrix of 8-bit floating-point values held in the half-vectors of the second source vectors. Each 16-bit container of the first source vectors holds 2 elements of each row of a $SVL_H \div 2 \times 2$ sub-matrix. Similarly, each 16-bit container of the second source vectors holds 2 elements of each column of a $2 \times SVL_H \div 2$ sub-matrix.

This instruction widens the sub-matrices of 8-bit floating-point values held in the first source vectors to half-precision values and multiplies them by the corresponding widened sub-matrices of 8-bit floating-point values in the second source vectors to half-precision values. The resulting quarter-tile $SVL_H \div 2 \times SVL_H \div 2$ half-precision sums of outer products are scaled by $2^{-UInt(FPMR.LSCALE[3:0])}$, before being destructively added to the half-precision destination tile. This is equivalent to performing a downscaled 2-way dot product and accumulate to each of the destination tile elements.

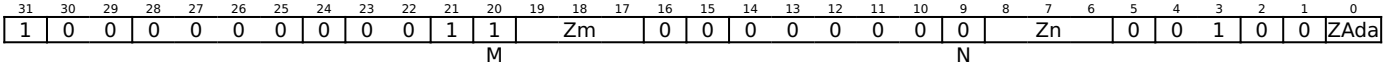
The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F8F16)



Encoding

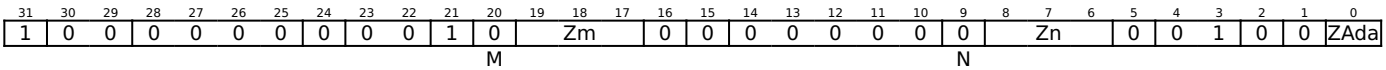
```
FMOP4A <ZAda>.H, <Zn>.B, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F8F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Single vectors

(FEAT_SME_MOP4 && FEAT_SME_F8F16)



Encoding

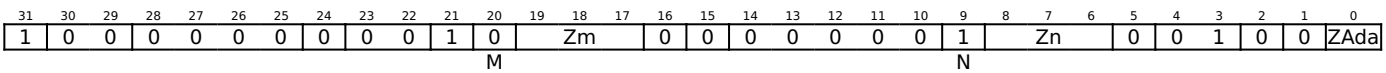
```
FMOP4A <ZAda>.H, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F8F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_F8F16)



Encoding

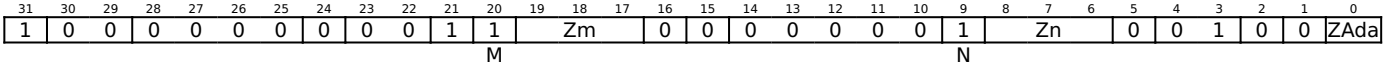
```
FMOP4A <ZAda>.H, { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F8F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F8F16)



Encoding

```
FMOP4A <ZAda>.H, { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F8F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV 16;
let tilesize : integer{} = 4 * dim * dim * 16;
let op3 : bits(tilesize) = ZAtile{tilesize}(da, 16);
var result : bits(tilesize);

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{VL}(n + (nreg-1) * col_hv);
    let op2 : bits(VL) = Z{VL}(m + (mreg-1) * row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(16) = op3[tile_idx*:16];
            var rowop : bits(16);
            var colop : bits(16);
            for i = 0 to 1 do
                rowop[i*:8] = op1[(2 * row_idx + i)*:8];
                colop[i*:8] = op2[(2 * col_idx + i)*:8];
            end;
            sum = FP8DotAddFP{16, 16}(sum, rowop, colop, FPCR(), FPMR());
            result[tile_idx*:16] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, 16) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOP4A (widening, 4-way)

8-bit floating-point quarter-tile sum of outer products to single-precision, accumulating

This instruction generates four independent quarter-tile 8-bit floating-point sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

Each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 4$ sub-matrix of 8-bit floating-point values held in the half-vectors of the first source vectors by the $4 \times SVL_S \div 2$ sub-matrix of 8-bit floating-point values held in the half-vectors of the second source vectors. Each 32-bit container of the first source vectors holds 4 elements of each row of a $SVL_S \div 2 \times 4$ sub-matrix. Similarly, each 32-bit container of the second source vectors holds 4 elements of each column of a $4 \times SVL_S \div 2$ sub-matrix.

This instruction widens the sub-matrices of 8-bit floating-point values held in the first source vectors to single-precision values and multiplies them by the corresponding widened sub-matrices of 8-bit floating-point values in the second source vectors to single-precision values. The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ single-precision sums of outer products are scaled by $2^{-UInt(FPMR.LSCALE)}$, before being destructively added to the single-precision destination tile. This is equivalent to performing a downscaled 4-way dot product and accumulate to each of the destination tile elements.

The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F8F32)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	1	1	Zm			0	0	0	0	0	0	0	0	Zn			0	0	0	0	ZAda	
											M											N									

Encoding

```
FMOP4A <ZAda>.S, <Zn>.B, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F8F32) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Single vectors

(FEAT_SME_MOP4 && FEAT_SME_F8F32)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	1	0	Zm			0	0	0	0	0	0	0	0	Zn			0	0	0	0	ZAda	
											M											N									

Encoding

```
FMOP4A <ZAda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F8F32) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_F8F32)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	1	0	Zm			0	0	0	0	0	0	0	1	Zn			0	0	0	0	ZAda	
											M											N									

Encoding

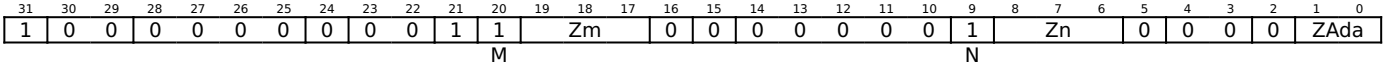
```
FMOP4A <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F8F32) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F8F32)



Encoding

```
FMOP4A <ZAda>.S, { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F8F32) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV 32;
let tilesize : integer{} = 4 * dim * dim * 32;
let op3 : bits(tilesize) = ZAtile{tilesize}(da, 32);
var result : bits(tilesize);

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{VL}(n + (nreg-1) * col_hv);
    let op2 : bits(VL) = Z{VL}(m + (mreg-1) * row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(32) = op3[tile_idx*:32];
            var rowop : bits(32);
            var colop : bits(32);
            for i = 0 to 3 do
                rowop[i*:8] = op1[(4 * row_idx + i)*:8];
                colop[i*:8] = op2[(4 * col_idx + i)*:8];
            end;
            sum = FP8DotAddFP{32, 32}(sum, rowop, colop, FPCR(), FPMR());
            result[tile_idx*:32] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOP4S (non-widening)

Floating-point quarter-tile outer product, subtracting

This instruction generates four independent quarter-tile floating-point outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and subtracts the results from the corresponding elements of a 16-bit, 32-bit, or 64-bit element ZA tile.

In case of the half-precision variant, each of the quarter-tile outer products is generated by multiplying the $SVL_H \div 2 \times 1$ sub-matrix of half-precision values held in the half-vectors of the first source vectors by the $1 \times SVL_H \div 2$ sub-matrix of half-precision values held in the half-vectors of the second source vectors. In case of the single-precision variant, each of the quarter-tile outer products is generated by multiplying the $SVL_S \div 2 \times 1$ sub-matrix of single-precision values held in the half-vectors of the first source vectors by the $1 \times SVL_S \div 2$ sub-matrix of single-precision values held in the half-vectors of the second source vectors. In case of the double-precision variant, each of the quarter-tile outer products is generated by multiplying the $SVL_D \div 2 \times 1$ sub-matrix of double-precision values held in the half-vectors of the first source vectors by the $1 \times SVL_D \div 2$ sub-matrix of double-precision values held in the half-vectors of the second source vectors.

The resulting quarter-tile $SVL_H \div 2 \times SVL_H \div 2$ half-precision outer products in case of the half-precision variant, $SVL_S \div 2 \times SVL_S \div 2$ single-precision outer products in case of the single-precision variant, or $SVL_D \div 2 \times SVL_D \div 2$ double-precision outer products in case of the double-precision variant are destructively subtracted from the destination ZA tile. This is equivalent to performing a single multiply-subtract from each of the destination tile elements.

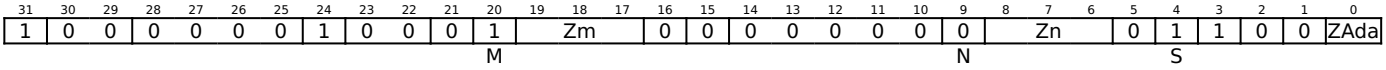
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 12 classes: Half-precision, single and multiple vectors , Half-precision, single vectors , Half-precision, multiple and single vectors , Half-precision, multiple vectors , Single-precision, single and multiple vectors , Single-precision, single vectors , Single-precision, multiple and single vectors , Single-precision, multiple vectors , Double-precision, single and multiple vectors , Double-precision, single vectors , Double-precision, multiple and single vectors and Double-precision, multiple vectors

Half-precision, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F16F16)



Encoding

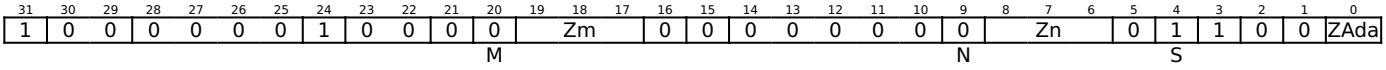
```
FMOP4S <ZAda>.H, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F16F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 16;
```

Half-precision, single vectors

(FEAT_SME_MOP4 && FEAT_SME_F16F16)



Encoding

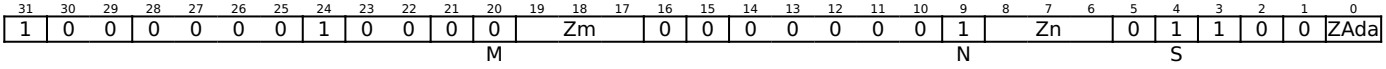
```
FMOP4S <ZAda>.H, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F16F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 16;
```

Half-precision, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_F16F16)



Encoding

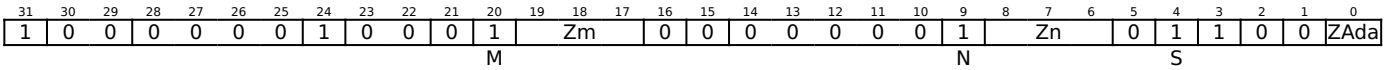
```
FMOP4S <ZAda>.H, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F16F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 16;
```

Half-precision, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F16F16)



Encoding

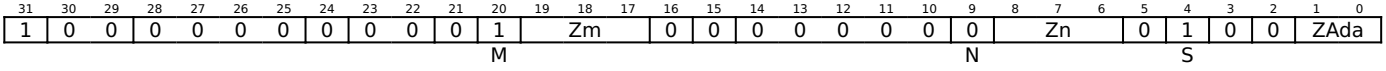
```
FMOP4S <ZAda>.H, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F16F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 16;
```

Single-precision, single and multiple vectors

(FEAT_SME_MOP4)



Encoding

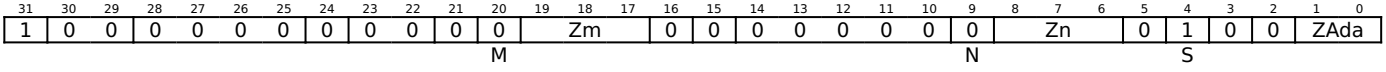
```
FMOP4S <ZAda>.S, <Zn>.S, { <Zm1>.S-<Zm2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 32;
```

Single-precision, single vectors

(FEAT_SME_MOP4)



Encoding

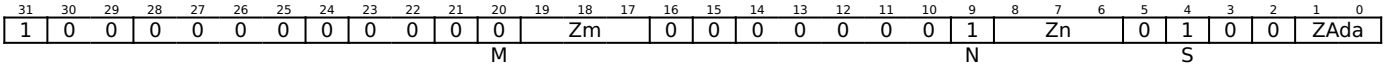
```
FMOP4S <ZAda>.S, <Zn>.S, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 32;
```

Single-precision, multiple and single vectors

(FEAT_SME_MOP4)



Encoding

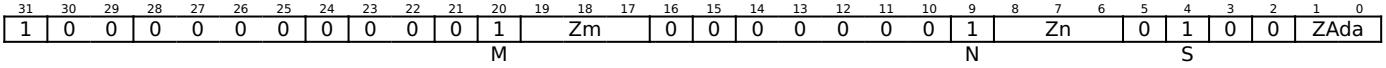
```
FMOP4S <ZAda>.S, { <Zn1>.S-<Zn2>.S }, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 32;
```

Single-precision, multiple vectors

(FEAT_SME_MOP4)



Encoding

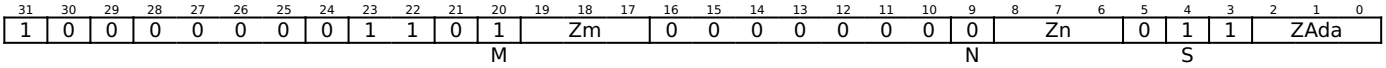
```
FMOP4S <ZAda>.S, { <Zn1>.S-<Zn2>.S }, { <Zm1>.S-<Zm2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 32;
```

Double-precision, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F64F64)



Encoding

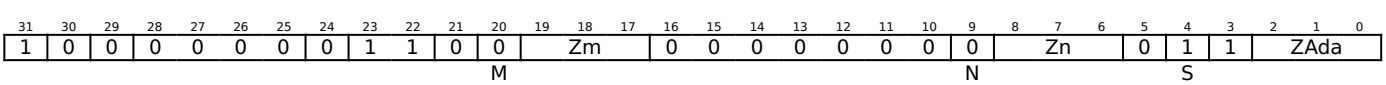
```
FMOP4S <ZAda>.D, <Zn>.D, { <Zm1>.D-<Zm2>.D }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F64F64) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 64;
```

Double-precision, single vectors

(FEAT_SME_MOP4 && FEAT_SME_F64F64)



Encoding

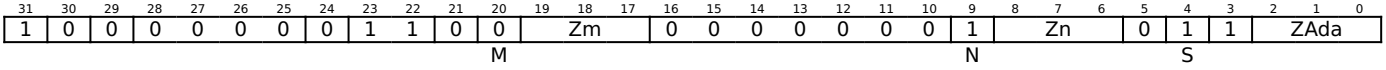
```
FMOP4S <ZAda>.D, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F64F64) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 64;
```

Double-precision, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_F64F64)



Encoding

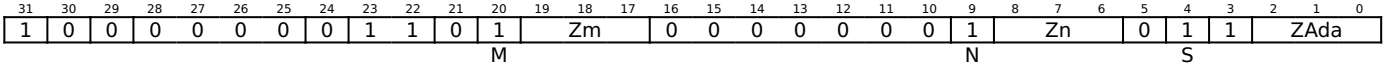
```
FMOP4S <ZAda>.D, { <Zn1>.D-<Zn2>.D }, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F64F64) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let esize : integer{} = 64;
```

Double-precision, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_F64F64)



Encoding

```
FMOP4S <ZAda>.D, { <Zn1>.D-<Zn2>.D }, { <Zm1>.D-<Zm2>.D }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_F64F64) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let esize : integer{} = 64;
```

Assembler Symbols

<ZAda>	For the "Half-precision, multiple and single vectors", "Half-precision, multiple vectors", "Half-precision, single and multiple vectors", and "Half-precision, single vectors" variants: is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field. For the "Single-precision, multiple and single vectors", "Single-precision, multiple vectors", "Single-precision, single and multiple vectors", and "Single-precision, single vectors" variants: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "Double-precision, multiple and single vectors", "Double-precision, multiple vectors", "Double-precision, single and multiple vectors", and "Double-precision, single vectors" variants: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.

<Zm> Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{tilesize}(da, esize);
var result : bits(tilesize);

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{VL}(n + (nreg-1) * col_hv);
    let op2 : bits(VL) = Z{VL}(m + (mreg-1) * row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var elem1 : bits(esize) = op1[row_idx*:esize];
            let elem2 : bits(esize) = op2[col_idx*:esize];
            let elem3 : bits(esize) = op3[tile_idx*:esize];

            elem1 = FPNeg{esize}(elem1, FPCR());
            result[tile_idx*:esize] = FPMulAdd_ZA{esize}(elem3, elem1, elem2, FPCR());
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOP4S (widening)

Half-precision quarter-tile sum of outer products to single-precision, subtracting

This instruction generates four independent quarter-tile half-precision sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and subtracts the results from the corresponding elements of a 32-bit element ZA tile.

Each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 2$ sub-matrix of half-precision values held in the half-vectors of the first source vectors by the $2 \times SVL_S \div 2$ sub-matrix of half-precision values held in the half-vectors of the second source vectors. Each 32-bit container of the first source vectors holds 2 elements of each row of a $SVL_S \div 2 \times 2$ sub-matrix. Similarly, each 32-bit container of the second source vectors holds 2 elements of each column of a $2 \times SVL_S \div 2$ sub-matrix.

The instruction widens the sub-matrices of half-precision values held in the first source vectors to single-precision values and multiplies them by the corresponding widened sub-matrices of half-precision values in the second source vectors to single-precision values. The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ single-precision sums of outer products are then destructively subtracted from the single-precision destination tile. This is equivalent to performing a 2-way dot product and subtract from each of the destination tile elements.

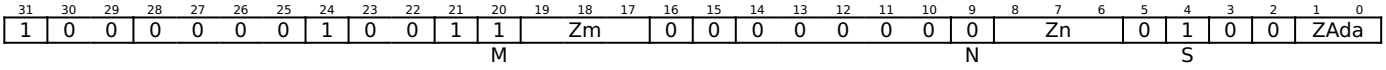
This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4)



Encoding

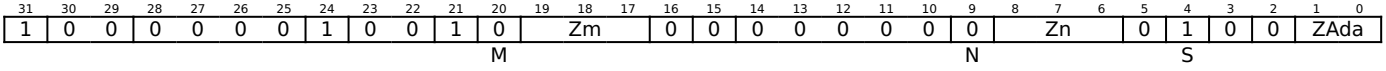
```
FMOP4S <ZAda>.S, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Single vectors

(FEAT_SME_MOP4)



Encoding

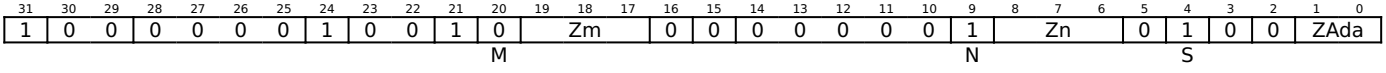
```
FMOP4S <ZAda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple and single vectors

(FEAT_SME_MOP4)



Encoding

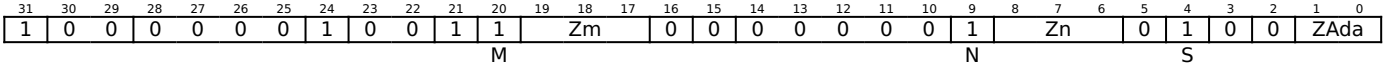
```
FMOP4S <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
```

Multiple vectors

(FEAT_SME_MOP4)



Encoding

```
FMOP4S <ZAda>.S, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV 32;
let tilesize : integer{} = 4 * dim * dim * 32;
let op3 : bits(tilesize) = ZAtile{tilesize}(da, 32);
var result : bits(tilesize);

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{VL}(n + (nreg-1) * col_hv);
    let op2 : bits(VL) = Z{VL}(m + (mreg-1) * row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            let sum : bits(32) = op3[tile_idx*:32];
            var erow_0 : bits(16) = op1[(2 * row_idx + 0)*:16];
            var erow_1 : bits(16) = op1[(2 * row_idx + 1)*:16];
            let ecol_0 : bits(16) = op2[(2 * col_idx + 0)*:16];
            let ecol_1 : bits(16) = op2[(2 * col_idx + 1)*:16];
            erow_0 = FPNeg{16}(erow_0, FPCR());
            erow_1 = FPNeg{16}(erow_1, FPCR());
            result[tile_idx*:32] = FPDotAdd_ZA(sum, erow_0, erow_1, ecol_0, ecol_1, FPCR());
        end;
    end;
end;
ZAtile{tilesize}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

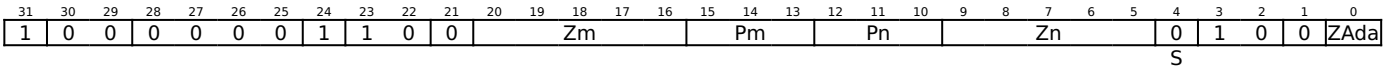
FMOPA (non-widening)

Floating-point outer product, accumulating

The half-precision variant works with a 16-bit element ZA tile.
The single-precision variant works with a 32-bit element ZA tile.
The double-precision variant works with a 64-bit element ZA tile.
This instruction generates an outer product of the first source vector and the second source vector. In case of the half-precision variant, the first source is $SVL_H \times 1$ vector and the second source is $1 \times SVL_H$ vector. In case of the single-precision variant, the first source is $SVL_S \times 1$ vector and the second source is $1 \times SVL_S$ vector. In case of the double-precision variant, the first source is $SVL_D \times 1$ vector and the second source is $1 \times SVL_D$ vector.
Each source vector is independently predicated by a corresponding governing predicate. When either source vector element is Inactive the corresponding destination tile element remains unmodified.
The resulting outer product, $SVL_H \times SVL_H$ in case of half-precision variant, $SVL_S \times SVL_S$ in case of single-precision variant or $SVL_D \times SVL_D$ in case of double-precision variant, is then destructively added to the destination tile. This is equivalent to performing a single multiply-accumulate to each of the destination tile elements.
This instruction follows SME ZA-targeting floating-point behaviors.
ID_AA64SMFR0_EL1.F64F64 indicates whether the double-precision variant is implemented, and ID_AA64SMFR0_EL1.F16F16 indicates whether the half-precision variant is implemented.
It has encodings from 3 classes: Half-precision , Single-precision and Double-precision

Half-precision

(FEAT_SME_F16F16)



Encoding

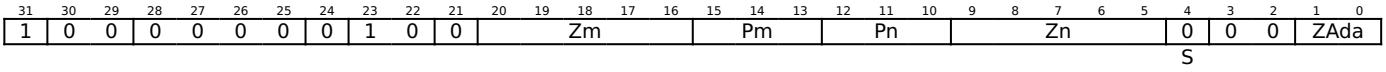
```
FMOPA <ZAda>.H, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Single-precision

(FEAT_SME)



Encoding

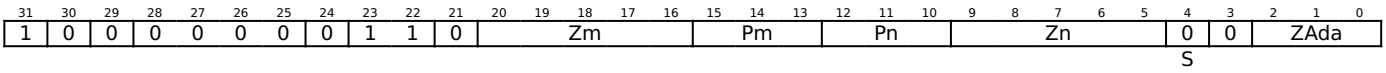
```
FMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.S, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Double-precision

(FEAT_SME_F64F64)



Encoding

```
FMOPA <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	For the "Half-precision" variant: is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field. For the "Single-precision" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "Double-precision" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let op1 : bits(VL) = Z{VL}(n);
let op2 : bits(VL) = Z{VL}(m);
let op3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        let elem2 : bits(esize) = op2[col*:esize];
        let elem3 : bits(esize) = op3[(row*dim+col)*:esize];

        if (ActivePredicateElement{PL}(mask1, row, esize) &&
              ActivePredicateElement{PL}(mask2, col, esize)) then
            let elem1 : bits(esize) = op1[row*:esize];
            result[(row*dim+col)*:esize] = FPMulAdd_ZA{esize}(elem3, elem1, elem2, FPCR());
        else
            result[(row*dim+col)*:esize] = elem3;
        end;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOPA (widening, 2-way, FP16 to FP32)

Half-precision sum of outer products to single-precision, accumulating

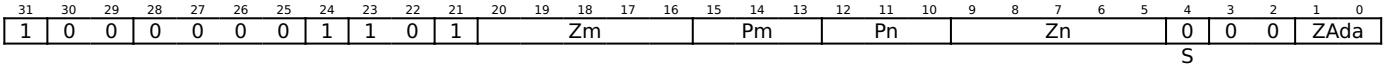
This instruction works with a 32-bit element ZA tile.
This instruction widens the $SVL_S \times 2$ sub-matrix of half-precision values held in the first source vector to single-precision values and multiplies it by the widened $2 \times SVL_S$ sub-matrix of half-precision values in the second source vector to single-precision values.

Each source vector is independently predicated by a corresponding governing predicate. When a 16-bit source element is Inactive it is treated as having the value +0.0, but if both pairs of source vector elements that correspond to a 32-bit destination element contain Inactive elements, then the destination element remains unmodified.
The resulting $SVL_S \times SVL_S$ single-precision sum of outer products is then destructively added to the single-precision destination tile. This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.
Each 32-bit container of the first source vector holds 2 consecutive column elements of each row of a $SVL_S \times 2$ sub-matrix. Similarly, each 32-bit container of the second source vector holds 2 consecutive row elements of each column of a $2 \times SVL_S$ sub-matrix.

This instruction follows SME ZA-targeting floating-point behaviors.

SME

(FEAT_SME)



Encoding

```
FMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV 32;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*32) = ZAtile{dim*dim*32}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        // determine row/col predicates
        let prow_0 : boolean = ActivePredicateElement{PL}(mask1, 2*row + 0, 16);
        let prow_1 : boolean = ActivePredicateElement{PL}(mask1, 2*row + 1, 16);
        let pcol_0 : boolean = ActivePredicateElement{PL}(mask2, 2*col + 0, 16);
        let pcol_1 : boolean = ActivePredicateElement{PL}(mask2, 2*col + 1, 16);

        var sum : bits(32) = operand3[(row*dim+col)*:32];
        if (prow_0 && pcol_0) || (prow_1 && pcol_1) then
            let erow_0 : bits(16) = if prow_0 then operand1[(2*row + 0)*:16]
                                              else FPZero{16}('0');
            let erow_1 : bits(16) = if prow_1 then operand1[(2*row + 1)*:16]
                                              else FPZero{16}('0');
            let ecol_0 : bits(16) = if pcol_0 then operand2[(2*col + 0)*:16]
                                              else FPZero{16}('0');
            let ecol_1 : bits(16) = if pcol_1 then operand2[(2*col + 1)*:16]
                                              else FPZero{16}('0');
            sum = FPDotAdd_ZA(sum, erow_0, erow_1, ecol_0, ecol_1, FPCR());
        end;
        result[(row*dim+col)*:32] = sum;
    end;
end;

ZAtile{dim*dim*32}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOPA (widening, 2-way, FP8 to FP16)

8-bit floating-point sum of outer products to half-precision, accumulating

This instruction works with a 16-bit element ZA tile.
This instruction widens the $SVL_H \times 2$ sub-matrix of 8-bit floating-point values held in the first source vector to half-precision values and multiplies it by the widened $2 \times SVL_H$ sub-matrix of 8-bit floating-point values in the second source vector to half-precision values.

Each source vector is independently predicated by a corresponding governing predicate. When a 8-bit source element is Inactive it is treated as having the value +0.0, but if both groups of source vector elements that correspond to a 16-bit destination element contain Inactive elements, then the destination element remains unmodified.

The resulting $SVL_H \times SVL_H$ half-precision sum of outer products are scaled by $2^{-UInt(FPMR.LSCALE[3:0])}$, before being destructively added to the half-precision destination tile. This is equivalent to performing a downscaled 2-way dot product and accumulate to each of the destination tile elements.

Each 16-bit container of the first source vector holds 2 consecutive column elements of each row of a $SVL_H \times 2$ sub-matrix. Similarly, each 16-bit container of the second source vector holds 2 consecutive row elements of each column of a $2 \times SVL_H$ sub-matrix.

The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

SME2

(FEAT_SME_F8F16)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	1	0	1	Zm					Pm			Pn			Zn					0	1	0	0	ZAda

Encoding

```
FMOPA <ZAda>.H, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV 16;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*16) = ZAtile{dim*dim*16}(da, 16);
var result : bits(dim*dim*16);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var prow : array [[2]] of boolean;
        var pcol : array [[2]] of boolean;
        var any_active : boolean = FALSE;
        for i = 0 to 1 do
            prow[[i]] = ActivePredicateElement{PL}(mask1, 2*row + i, 8);
            pcol[[i]] = ActivePredicateElement{PL}(mask2, 2*col + i, 8);
            any_active = any_active || (prow[[i]] && pcol[[i]]);
        end;

        if any_active then
            var sum : bits(16) = operand3[(row*dim+col)*:16];
            var rowop : bits(16) = Zeros{};
            var colop : bits(16) = Zeros{};
            for i = 0 to 1 do
                if prow[[i]] then
                    rowop[i*:8] = operand1[(2*row + i)*:8];
                end;
                if pcol[[i]] then
                    colop[i*:8] = operand2[(2*col + i)*:8];
                end;
            end;

            sum = FP8DotAddFP{16, 16}(sum, rowop, colop, FPCR(), FPMR());
            result[(row*dim+col)*:16] = sum;
        else
            result[(row*dim+col)*:16] = operand3[(row*dim+col)*:16];
        end;
    end;
end;

ZAtile{dim*dim*16}(da, 16) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOPA (widening, 4-way)

8-bit floating-point sum of outer products to single-precision, accumulating

This instruction works with a 32-bit element ZA tile.
This instruction widens the $SVL_S \times 4$ sub-matrix of 8-bit floating-point values held in the first source vector to single-precision values and multiplies it by the widened $4 \times SVL_S$ sub-matrix of 8-bit floating-point values in the second source vector to single-precision values.

Each source vector is independently predicated by a corresponding governing predicate. When a 8-bit source element is Inactive it is treated as having the value +0.0, but if both groups of source vector elements that correspond to a 32-bit destination element contain Inactive elements, then the destination element remains unmodified.

The resulting $SVL_S \times SVL_S$ single-precision sum of outer products are scaled by $2^{-UInt(FPMR.LSCALE)}$, before being destructively added to the single-precision destination tile. This is equivalent to performing a downscaled 4-way dot product and accumulate to each of the destination tile elements.
Each 32-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_S \times 4$ sub-matrix. Similarly, each 32-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_S$ sub-matrix.

The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

SME2

(FEAT_SME_F8F32)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	1	0	1	Zm					Pm			Pn			Zn					0	0	0	ZAda	

Encoding

```
FMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV 32;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*32) = ZAtile{dim*dim*32}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var prow : array [[4]] of boolean;
        var pcol : array [[4]] of boolean;
        var any_active : boolean = FALSE;
        for i = 0 to 3 do
            prow[[i]] = ActivePredicateElement{PL}(mask1, 4*row + i, 8);
            pcol[[i]] = ActivePredicateElement{PL}(mask2, 4*col + i, 8);
            any_active = any_active || (prow[[i]] && pcol[[i]]);
        end;

        if any_active then
            var sum : bits(32) = operand3[(row*dim+col)*:32];
            var rowop : bits(32) = Zeros{};
            var colop : bits(32) = Zeros{};
            for i = 0 to 3 do
                if prow[[i]] then
                    rowop[i*:8] = operand1[(4*row + i)*:8];
                end;
                if pcol[[i]] then
                    colop[i*:8] = operand2[(4*col + i)*:8];
                end;
            end;

            sum = FP8DotAddFP{32, 32}(sum, rowop, colop, FPCR(), FPMR());
            result[(row*dim+col)*:32] = sum;
        else
            result[(row*dim+col)*:32] = operand3[(row*dim+col)*:32];
        end;
    end;
end;

ZAtile{dim*dim*32}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOPS (non-widening)

Floating-point outer product, subtracting

The half-precision variant works with a 16-bit element ZA tile.
The single-precision variant works with a 32-bit element ZA tile.
The double-precision variant works with a 64-bit element ZA tile.
This instruction generates an outer product of the first source vector and the second source vector. In case of the half-precision variant, the first source is $SVL_H \times 1$ vector and the second source is $1 \times SVL_H$ vector. In case of the single-precision variant, the first source is $SVL_S \times 1$ vector and the second source is $1 \times SVL_S$ vector. In case of the double-precision variant, the first source is $SVL_D \times 1$ vector and the second source is $1 \times SVL_D$ vector.
Each source vector is independently predicated by a corresponding governing predicate. When either source vector element is Inactive the corresponding destination tile element remains unmodified.
The resulting outer product, $SVL_H \times SVL_H$ in case of half-precision variant, $SVL_S \times SVL_S$ in case of single-precision variant or $SVL_D \times SVL_D$ in case of double-precision variant, is then destructively subtracted from the destination tile. This is equivalent to performing a single multiply-subtract from each of the destination tile elements.
This instruction follows SME ZA-targeting floating-point behaviors.
ID_AA64SMFR0_EL1.F64F64 indicates whether the double-precision variant is implemented, and ID_AA64SMFR0_EL1.F16F16 indicates whether the half-precision variant is implemented.
It has encodings from 3 classes: Half-precision , Single-precision and Double-precision

Half-precision

(FEAT_SME_F16F16)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	1	0	0	Zm					Pm			Pn			Zn					1	1	0	0	ZAda
																											S				

Encoding

```
FMOPS <ZAda>.H, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Single-precision

(FEAT_SME)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	1	0	0	Zm					Pm			Pn			Zn					1	0	0	ZAda	
																											S				

Encoding

```
FMOPS <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.S, <Zm>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Double-precision

(FEAT_SME_F64F64)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	1	1	0	Zm					Pm			Pn			Zn					1	0	ZAda		
																											S				

Encoding

```
FMOPS <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.D, <Zm>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	For the "Half-precision" variant: is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field. For the "Single-precision" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "Double-precision" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let op1 : bits(VL) = Z{VL}(n);
let op2 : bits(VL) = Z{VL}(m);
let op3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        let elem2 : bits(esize) = op2[col*:esize];
        let elem3 : bits(esize) = op3[(row*dim+col)*:esize];

        if (ActivePredicateElement{PL}(mask1, row, esize) &&
              ActivePredicateElement{PL}(mask2, col, esize)) then
            let elem1 : bits(esize) = FPNeg{esize}(op1[row*:esize], FPCR());
            result[(row*dim+col)*:esize] = FPMulAdd_ZA{esize}(elem3, elem1, elem2, FPCR());
        else
            result[(row*dim+col)*:esize] = elem3;
        end;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMOPS (widening)

Half-precision sum of outer products to single-precision, subtracting

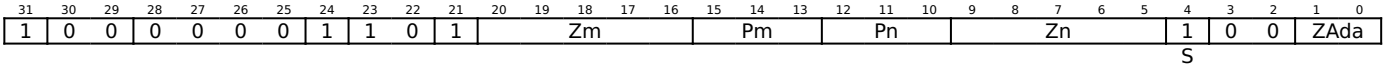
This instruction works with a 32-bit element ZA tile.
This instruction widens the $SVL_S \times 2$ sub-matrix of half-precision values held in the first source vector to single-precision values and multiplies it by the widened $2 \times SVL_S$ sub-matrix of half-precision values in the second source vector to single-precision values.

Each source vector is independently predicated by a corresponding governing predicate. When a 16-bit source element is Inactive it is treated as having the value +0.0, but if both pairs of source vector elements that correspond to a 32-bit destination element contain Inactive elements, then the destination element remains unmodified.
The resulting $SVL_S \times SVL_S$ single-precision sum of outer products is then destructively subtracted from the single-precision destination tile. This is equivalent to performing a 2-way dot product and subtract from each of the destination tile elements.
Each 32-bit container of the first source vector holds 2 consecutive column elements of each row of a $SVL_S \times 2$ sub-matrix. Similarly, each 32-bit container of the second source vector holds 2 consecutive row elements of each column of a $2 \times SVL_S$ sub-matrix.

This instruction follows SME ZA-targeting floating-point behaviors.

SME

(FEAT_SME)



Encoding

```
FMOPS <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV 32;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*32) = ZAtile{dim*dim*32}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        // determine row/col predicates
        let prow_0 : boolean = ActivePredicateElement{PL}(mask1, 2*row + 0, 16);
        let prow_1 : boolean = ActivePredicateElement{PL}(mask1, 2*row + 1, 16);
        let pcol_0 : boolean = ActivePredicateElement{PL}(mask2, 2*col + 0, 16);
        let pcol_1 : boolean = ActivePredicateElement{PL}(mask2, 2*col + 1, 16);

        var sum : bits(32) = operand3[(row*dim+col)*:32];
        if (prow_0 && pcol_0) || (prow_1 && pcol_1) then
            var erow_0 : bits(16) = if prow_0 then operand1[(2*row + 0)*:16]
                                              else FPZero{16}('0');
            var erow_1 : bits(16) = if prow_1 then operand1[(2*row + 1)*:16]
                                              else FPZero{16}('0');
            let ecol_0 : bits(16) = if pcol_0 then operand2[(2*col + 0)*:16]
                                              else FPZero{16}('0');
            let ecol_1 : bits(16) = if pcol_1 then operand2[(2*col + 1)*:16]
                                              else FPZero{16}('0');
            if prow_0 then erow_0 = FPNeg{16}(erow_0, FPCR()); end;
            if prow_1 then erow_1 = FPNeg{16}(erow_1, FPCR()); end;
            sum = FPDotAdd_ZA(sum, erow_0, erow_1, ecol_0, ecol_1, FPCR());
        end;
        result[(row*dim+col)*:32] = sum;
    end;
end;

ZAtile{dim*dim*32}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMUL (multiple and single vector)

Multi-vector floating-point multiply by vector

This instruction multiplies all the floating-point elements of the two or four first source vectors by the corresponding floating-point elements of the second source vector, and places the results in the corresponding elements of the two or four destination vectors.

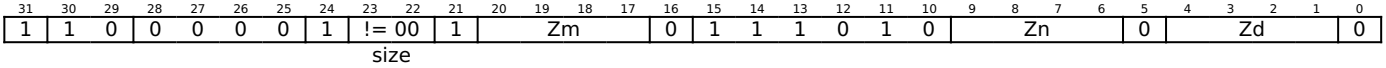
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2p2)



Encoding

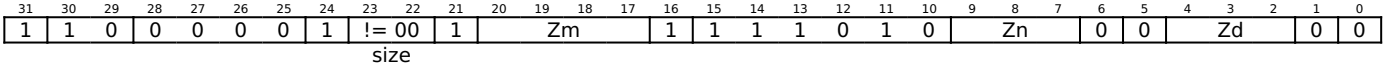
```
FMUL { <Zd1>.<T>-<Zd2>.<T> }, { <Zn1>.<T>-<Zn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt(Zd::'0');
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2p2)



Encoding

```
FMUL { <Zd1>.<T>-<Zd4>.<T> }, { <Zn1>.<T>-<Zn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt(Zd::'00');
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1> For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn1> For the "Two registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMul{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FMUL (multiple vectors)

Multi-vector floating-point multiply

This instruction multiplies all the floating-point elements of the two or four first source vectors by the corresponding floating-point elements of the two or four second source vectors, and places the results in the corresponding elements of the two or four destination vectors.

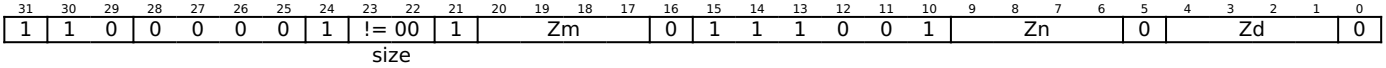
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2p2)



Encoding

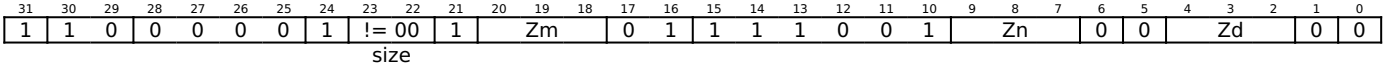
```
FMUL { <Zd1>.<T>-<Zd2>.<T> }, { <Zn1>.<T>-<Zn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt(Zd::'0');
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2p2)



Encoding

```
FMUL { <Zd1>.<T>-<Zd4>.<T> }, { <Zn1>.<T>-<Zn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let d : integer = UInt(Zd::'00');
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1> For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn1> For the "Two registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        results[[r]][e*:esize] = FPMul{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTA

Multi-vector single-precision round to integral value, to nearest with ties away from zero

This instruction rounds each single-precision element of the two or four source vectors to the nearest integral floating-point value, with ties rounding away from zero, and places the results in the corresponding elements of the two or four destination vectors.

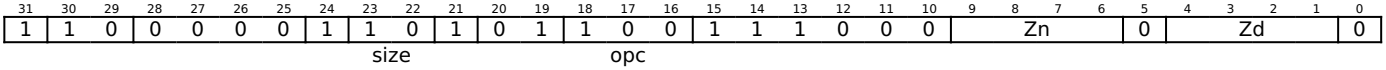
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

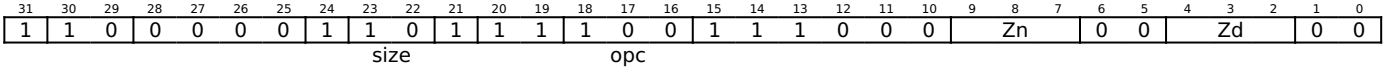
```
FRINTA { <Zd1>.S-<Zd2>.S }, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEAWAY;
```

Four registers

(FEAT_SME2)



Encoding

```
FRINTA { <Zd1>.S-<Zd4>.S }, { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEAWAY;
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn1>	For the "Two registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(32) = operand[e*:32];
        results[[r]][e*:32] = FPRoundInt{32}(element, FPCR(), rounding, exact);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTM

Multi-vector single-precision round to integral value, toward minus Infinity

This instruction rounds down each single-precision element of the two or four source vectors to an integral floating-point value, and places the results in the corresponding elements of the two or four destination vectors.

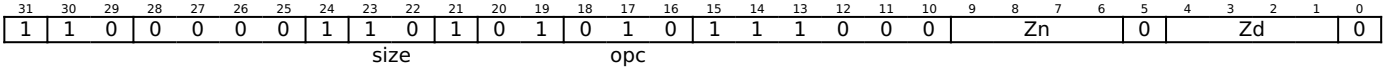
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

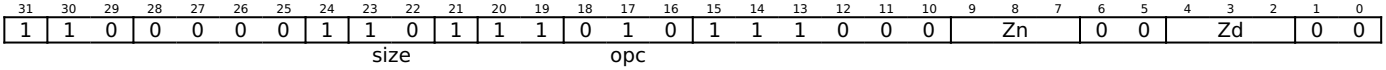
```
FRINTM { <Zd1>.S-<Zd2>.S }, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_NEGINF;
```

Four registers

(FEAT_SME2)



Encoding

```
FRINTM { <Zd1>.S-<Zd4>.S }, { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_NEGINF;
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn1>	For the "Two registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(32) = operand[e*:32];
        results[[r]][e*:32] = FPRoundInt{32}(element, FPCR(), rounding, exact);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTN

Multi-vector single-precision round to integral value, to nearest with ties to even

This instruction rounds each single-precision element of the two or four source vectors to the nearest integral floating-point value, with ties rounding to an even value, and places the results in the corresponding elements of the two or four destination vectors.

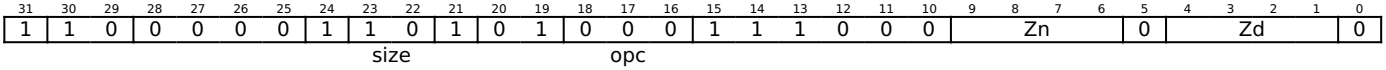
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

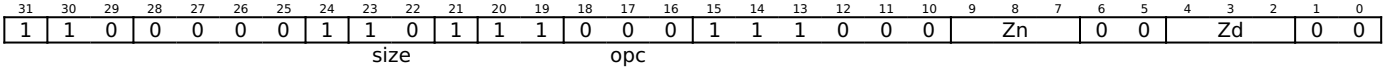
```
FRINTN { <Zd1>.S-<Zd2>.S }, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEEVEN;
```

Four registers

(FEAT_SME2)



Encoding

```
FRINTN { <Zd1>.S-<Zd4>.S }, { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_TIEEVEN;
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn1>	For the "Two registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(32) = operand[e*:32];
        results[[r]][e*:32] = FPRoundInt{32}(element, FPCR(), rounding, exact);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FRINTP

Multi-vector single-precision round to integral value, toward plus Infinity

This instruction rounds up each single-precision element of the two or four source vectors to an integral floating-point value, and places the results in the corresponding elements of the two or four destination vectors.

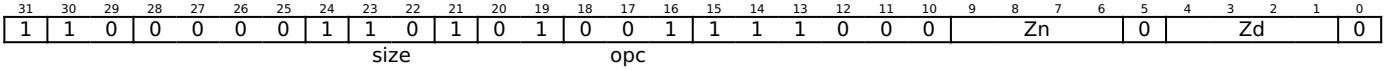
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

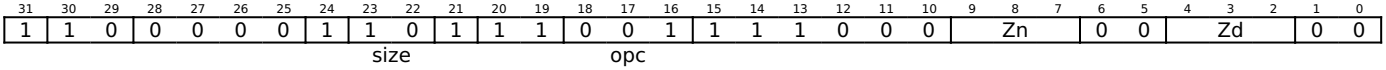
```
FRINTP { <Zd1>.S-<Zd2>.S }, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_POSINF;
```

Four registers

(FEAT_SME2)



Encoding

```
FRINTP { <Zd1>.S-<Zd4>.S }, { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
let exact : boolean = FALSE;
let rounding : FPRounding = FPRounding_POSINF;
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn1>	For the "Two registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(32) = operand[e*:32];
        results[[r]][e*:32] = FPRoundInt{32}(element, FPCR(), rounding, exact);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FSCALE (multiple and single vector)

Multi-vector floating-point adjust exponent by vector

This instruction multiplies the floating-point elements of the two or four first source vectors by 2.0 to the power of the signed integer values in the corresponding elements of the second source vector and destructively places the results in the corresponding elements of the two or four first source vectors.

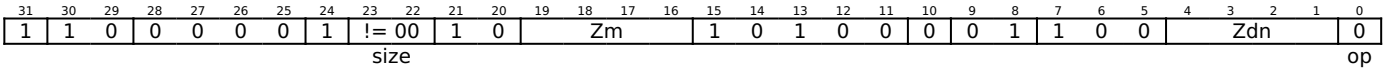
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_FP8)



Encoding

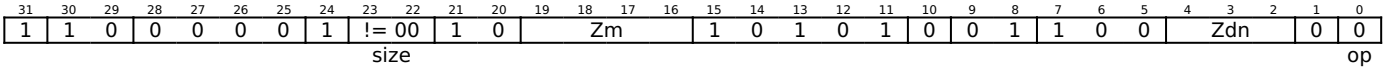
```
FSCALE { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_FP8)



Encoding

```
FSCALE { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : integer = SInt(operand2[e*:esize]);
        results[[r]][e*:esize] = FPScale{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FSCALE (multiple vectors)

Multi-vector floating-point adjust exponent

This instruction multiplies the floating-point elements of the two or four first source vectors by 2.0 to the power of the signed integer values in the corresponding elements of the two or four second source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

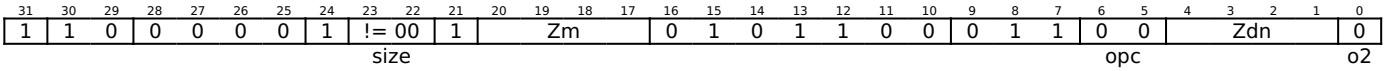
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2 && FEAT_FP8)



Encoding

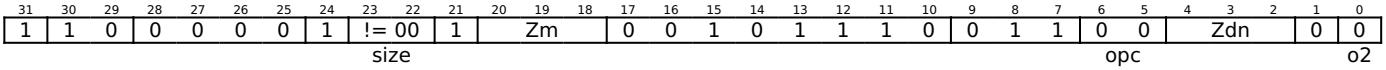
```
FSCALE { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2 && FEAT_FP8)



Encoding

```
FSCALE { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !IsFeatureImplemented(FEAT_FP8) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : integer = SInt(operand2[e*:esize]);
        results[[r]][e*:esize] = FPScale{esize}(element1, element2, FPCR());
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FSUB

Multi-vector floating-point subtract from ZA array vectors

This instruction destructively subtracts all elements of the two or four source vectors from the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME ZA-targeting floating-point behaviors.

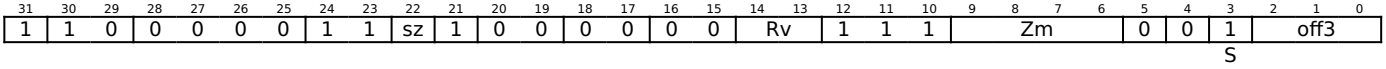
This instruction is unpredicated.

ID_AA64SMFR0_EL1.F64F64 indicates whether the double-precision variant is implemented, and ID_AA64SMFR0_EL1.F16F16 indicates whether the half-precision variant is implemented.

It has encodings from 4 classes: Two ZA single-vectors , Two ZA single-vectors of half-precision elements , Four ZA single-vectors and Four ZA single-vectors of half-precision elements

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

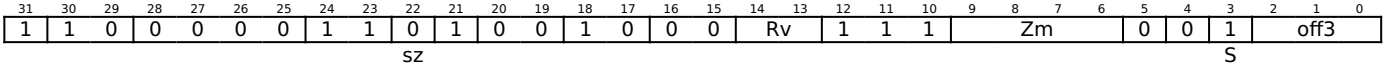
```
FSUB ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let m : integer{} = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Two ZA single-vectors of half-precision elements

(FEAT_SME_F16F16 || FEAT_SME_F8F16)



Encoding

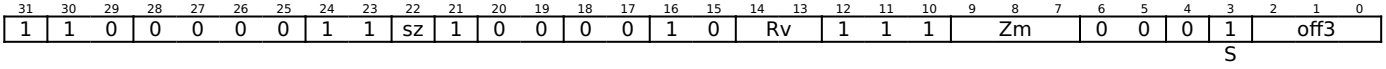
```
FSUB ZA.H[<Wv>, <offs>{, VGx2}], { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) && !IsFeatureImplemented(FEAT_SME_F8F16) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let m : integer{} = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64))



Encoding

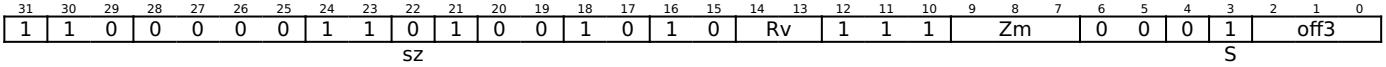
```
FSUB ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_F64F64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let m : integer{} = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Four ZA single-vectors of half-precision elements

(FEAT_SME_F16F16 || FEAT_SME_F8F16)



Encoding

```
FSUB ZA.H[<Wv>, <offs>{, VGx4}], { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F16F16) && !IsFeatureImplemented(FEAT_SME_F8F16) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 16;
let m : integer{} = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zm1> For the "Two ZA single-vectors of half-precision elements" and "Two ZA single-vectors" variants: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA single-vectors of half-precision elements" and "Four ZA single-vectors" variants: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zm4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = ZAvector{VL}(vec);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = FPSub_ZA{esize}(element1, element2, FPCR());
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FTMOPA (non-widening)

Floating-point sparse outer product, accumulating

This instruction generates floating-point outer product by multiplying the 1-in-2 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the destination ZA tile elements.

In case of the half-precision variant, the outer product is generated by multiplying the selected 1-in-2 half-precision value from each overlapping 16-bit containers of the two $SVL_H \times 1$ sub-matrices in the first source vectors by the half-precision value from the corresponding 16-bit container of the compressed $1 \times SVL_H$ sub-matrix in the second source vector. In case of the single-precision variant, the outer product is generated by multiplying the selected 1-in-2 single-precision value from each overlapping 32-bit containers of the two $SVL_S \times 1$ sub-matrices in the first source vectors by the single-precision value from the corresponding 32-bit container of the compressed $1 \times SVL_S$ sub-matrix in the second source vector.

The 1-in-2 floating-point value in the first source vectors is selected by 2-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the result. If both bits of the 2-bit control corresponding to the elements of the first source vectors are 1, only the element corresponding to the least significant bit is selected.

The instruction multiplies the selected elements of sub-matrices of floating-point values held in the first source vectors by the corresponding elements of sub-matrix of floating-point values in the second source vector. The resulting outer product, $SVL_H \times SVL_H$ in case of the half-precision variant or $SVL_S \times SVL_S$ in case of the single-precision variant, is then destructively added to the destination tile. This is equivalent to performing a single multiply-accumulate to each of the destination tile elements.

This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

It has encodings from 2 classes: Half-precision and Single-precision

Half-precision

(FEAT_SME_TMOP && FEAT_SME_F16F16)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	1	0	Zm					0	0	0	K	Zk		Zn				i2		1	0	0	ZAda

Encoding

```
FTMOPA <ZAda>.H, { <Zn1>.H-<Zn2>.H }, <Zm>.H, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) || !IsFeatureImplemented(FEAT_SME_F16F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
let esize : integer{} = 16;
```

Single-precision

(FEAT_SME_TMOP)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	1	0	Zm					0	0	0	K	Zk		Zn				i2		0	0	ZAda	

Encoding

```
FTMOPA <ZAda>.S, { <Zn1>.S-<Zn2>.S }, <Zm>.S, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
let esize : integer{} = 32;
```

Assembler Symbols

<ZAda>	For the "Half-precision" variant: is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field. For the "Single-precision" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV esize;
let csize : integer{} = (VL * 2) DIV esize;
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var i : integer = 0;
        var elem1 : bits(esize) = FPZero{esize}('0');
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{}(n+r);
            if i < 1 && ctrl[(2*col + r)*:1] == '1' then
                elem1 = op1[row*:esize];
                i = i + 1;
            end;
        end;
        let elem2 : bits(esize) = op2[col*:esize];
        let sum : bits(esize) = op4[(row*dim+col)*:esize];
        result[(row*dim+col)*:esize] = FPMulAdd_ZA{esize}(sum, elem1, elem2, FPCR());
    end;
end;
ZAtile{dim*dim*esize}(da, esize) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FTMOPA (widening, 2-way, FP16 to FP32)

Half-precision sparse sum of outer products to single-precision, accumulating

This instruction generates half-precision sum of outer products by multiplying the 2-in-4 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

The sum of outer products is generated by multiplying the selected 2-in-4 half-precision values from each overlapping 32-bit containers of the two $SVL_S \times 2$ sub-matrices in the first source vectors by the two half-precision values from the corresponding 32-bit container of the $2 \times SVL_S$ sub-matrix in the second source vector. The two selected elements from each overlapping 32-bit containers of the first source vectors correspond to 2-in-4 elements of rows of two $SVL_S \times 2$ sub-matrices. Each 32-bit container of the second source vector holds 2 elements of columns of a compressed $2 \times SVL_S$ sub-matrix.

The 2-in-4 half-precision values from overlapping 32-bit containers of the first source vectors are selected by 4-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the sum of products result. If more than two bits of the 4-bit control corresponding to 32-bit containers of the first source vectors are 1, only the elements corresponding to the least two significant bits are selected.

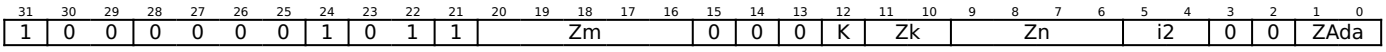
The instruction widens the selected elements of sub-matrices of half-precision values held in the first source vectors to single-precision values and multiplies them by the corresponding widened elements of sub-matrix of half-precision values in the second source vector to single-precision values. The resulting $SVL_S \times SVL_S$ single-precision sum of outer products is then destructively added to the single-precision destination tile. This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.

This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP)



Encoding

```
FTMOPA <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 32;
let csize : integer{} = VL DIV 8;
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*32) = ZAtile{}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var i : integer = 0;
        var erow : array [[2]] of bits(16);
        erow[[0]] = FPZero{16}('0');
        erow[[1]] = FPZero{16}('0');
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{}(n+r);
            for e = 0 to 1 do
                if i < 2 && ctrl[(4*col + 2*r + e)*:1] == '1' then
                    erow[[i]] = op1[(2*row + e)*:16];
                    i = i + 1;
                end;
            end;
        end;
        var ecol : array [[2]] of bits(16);
        ecol[[0]] = op2[(2*col + 0)*:16];
        ecol[[1]] = op2[(2*col + 1)*:16];
        let sum : bits(32) = op4[(row*dim+col)*:32];
        result[(row*dim+col)*:32] = FPDotAdd_ZA(sum, erow[[0]], erow[[1]],
                                                ecol[[0]], ecol[[1]], FPCR());
    end;
end;
ZAtile{dim*dim*32}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FTMOPA (widening, 2-way, FP8 to FP16)

8-bit floating-point sparse sum of outer products to half-precision, accumulating

This instruction generates 8-bit floating-point sum of outer products by multiplying the 2-in-4 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the corresponding elements of a 16-bit element ZA tile.

The sum of outer products is generated by multiplying the selected 2-in-4 8-bit floating-point values from each overlapping 16-bit containers of the two $SVL_H \times 2$ sub-matrices in the first source vectors by the two 8-bit floating point values from the corresponding 16-bit container of the $2 \times SVL_H$ sub-matrix in the second source vector. The two selected elements from each overlapping 16-bit containers of the first source vectors correspond to 2-in-4 elements of rows of two $SVL_H \times 2$ sub-matrices. Each 16-bit container of the second source vector holds 2 elements of columns of a compressed $2 \times SVL_H$ sub-matrix.

The 2-in-4 8-bit floating values from overlapping 16-bit containers of the first source vectors are selected by 4-bit control in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the sum of products result. If more than two bits of the 4-bit control corresponding to 16-bit containers of the first source vectors are 1, only the elements corresponding to the least two significant bits are selected.

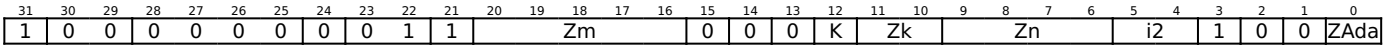
The instruction widens the selected elements of sub-matrices of 8-bit floating-point values held in the first source vectors to half-precision values and multiplies them by the corresponding widened elements of sub-matrix of 8-bit floating-point values in the second source vector to half-precision values. The resulting $SVL_H \times SVL_H$ half-precision sum of outer products is scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$, before being destructively added to the half-precision destination tile. This is equivalent to performing a downscaled 2-way dot product and accumulate to each of the destination tile elements.

The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP && FEAT_SME_F8F16)



Encoding

```
FTMOPA <ZAda>.H, { <Zn1>.B-<Zn2>.B }, <Zm>.B, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) || !IsFeatureImplemented(FEAT_SME_F8F16) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA1, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 16;
let csize : integer{} = VL DIV 4;
let op2 : bits(VL) = Z{VL}(m);
let op3 : bits(VL) = Z{VL}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*16) = ZAtile{dim*dim*16}(da, 16);
var result : bits(dim*dim*16);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var rowop : bits(16) = Zeros{};
        var colop : bits(16) = Zeros{};
        var i : integer = 0;
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{VL}(n+r);
            for e = 0 to 1 do
                if i < 2 && ctrl[(4*col + 2*r + e)*:1] == '1' then
                    rowop[i*:8] = op1[(2*row + e)*:8];
                    i = i + 1;
                end;
            end;
        end;
        for j = 0 to 1 do
            colop[j*:8] = op2[(2*col + j)*:8];
        end;
        let sum : bits(16) = op4[(row*dim+col)*:16];
        result[(row*dim+col)*:16] = FP8DotAddFP{16, 16}(sum, rowop, colop, FPCR(), FPMR());
    end;
end;
ZAtile{dim*dim*16}(da, 16) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FTMOPA (widening, 4-way)

8-bit floating-point sparse sum of outer products to single-precision, accumulating

This instruction generates 8-bit floating-point sum of outer products by multiplying the 2-in-4 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

The sum of outer products is generated by multiplying the selected two sets of 2-in-4 8-bit floating-point values from each overlapping 32-bit containers of the two $SVL_S \times 4$ sub-matrices in the first source vectors by the four 8-bit floating-point values from the corresponding 32-bit container of the $4 \times SVL_S$ sub-matrix in the second source vector. The four selected elements from the overlapping 32-bit containers of the first source vectors correspond to two sets of 2-in-4 elements of rows of two $SVL_S \times 4$ sub-matrices. Each 32-bit container of the second source vector holds 4 elements of columns of a compressed $4 \times SVL_S$ sub-matrix.

The two sets of 2-in-4 8-bit floating values from overlapping 32-bit containers of the first source vectors are selected by pairs of 4-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the sum of products result. If more than two bits of the 4-bit control corresponding to each 32-bit container of the first source vectors are 1, only the elements corresponding to the least two significant bits are selected.

The instruction widens the selected elements of sub-matrices of 8-bit floating-point values held in the first source vectors to single-precision values and multiplies them by the corresponding widened elements of sub-matrix of 8-bit floating-point values in the second source vector to single-precision values. The resulting $SVL_S \times SVL_S$ single-precision sum of outer products is scaled by $2^{-UInt(FPMR.LSCALE)}$, before being destructively added to the single-precision destination tile. This is equivalent to performing a downscaled 4-way dot product and accumulate to each of the destination tile elements.

The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP && FEAT_SME_F8F32)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	1	1	Zm					0	0	0	K	Zk		Zn				i2		0	0	ZAda	

Encoding

```
FTMOPA <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) || !IsFeatureImplemented(FEAT_SME_F8F32) then
    EndOfDecode(Decode_UNDEF);
end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 32;
let csize : integer{} = VL DIV 4;
let op2 : bits(VL) = Z{VL}(m);
let op3 : bits(VL) = Z{VL}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*32) = ZAtile{dim*dim*32}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var rowop : bits(32) = Zeros{};
        var colop : bits(32) = Zeros{};
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{VL}(n+r);
            var i : integer = 0;
            for e = 0 to 3 do
                if i < 2 && ctrl[(8*col + 4*r + e)*:1] == '1' then
                    rowop[(2*r + i)*:8] = op1[(4*row + e)*:8];
                    i = i + 1;
                end;
            end;
        end;
        for j = 0 to 3 do
            colop[j*:8] = op2[(4*col + j)*:8];
        end;
        let sum : bits(32) = op4[(row*dim+col)*:32];
        result[(row*dim+col)*:32] = FP8DotAddFP{32, 32}(sum, rowop, colop, FPCR(), FPMR());
    end;
end;
ZAtile{dim*dim*32}(da, 32) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FVDOT (FP16 to FP32)

Multi-vector half-precision vertical dot product by indexed element to single-precision

This instruction calculates the fused sum-of-products of each vertical pair of half-precision values in the corresponding elements of the two first source vectors with the pair of half-precision values in the indexed 32-bit group of the corresponding 128-bit segment of the second source vector, without intermediate rounding. The single-precision sum-of-products are destructively added to the corresponding single-precision elements of the two ZA single-vector groups.

The half-precision pairs within the second source vector are specified using an immediate index that selects the same pair position within each 128-bit vector segment. The element index range is from 0 to 3.

The single-vector group within each half of the ZA array is selected by the sum of the vector select register and offset, modulo half the number of ZA array vectors.

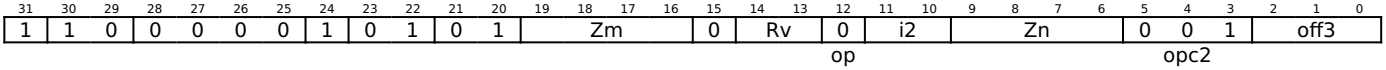
The vector group symbol VGx2 indicates that the ZA operand consists of two ZA single-vector groups. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction follows SME ZA-targeting floating-point behaviors.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
FVDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let esize : integer{} = 32;
let hsize : integer{} = esize DIV 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 2;
let segmentsize : integer{} = 128;
let eltspersegment : integer = segmentsize DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to 1 do
    let operand1a : bits(VL) = Z{VL}(n);
    let operand1b : bits(VL) = Z{VL}(n+1);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let elt1_a : bits(hsize) = operand1a[(2 * e + r)*:hsize];
        let elt1_b : bits(hsize) = operand1b[(2 * e + r)*:hsize];
        let elt2_a : bits(hsize) = operand2[(2 * s + 0)*:hsize];
        let elt2_b : bits(hsize) = operand2[(2 * s + 1)*:hsize];
        var sum : bits(esize) = operand3[e*:esize];
        sum = FPDotAdd_ZA(sum, elt1_a, elt1_b, elt2_a, elt2_b, FPCR());
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FVDOT (FP8 to FP16)

Multi-vector 8-bit floating-point vertical dot product by indexed element to half-precision

This instruction calculates the fused sum-of-products of each vertical pair of 8-bit floating-point values held in the corresponding elements of the two first source vectors with horizontal pair of 8-bit floating-point values in the indexed 16-bit group of the corresponding 128-bit segment of the second source vector. The half-precision sum-of-products are scaled by $2^{-\text{UInt(FPMR.LSCALE[3:0])}}$, before being destructively added without intermediate rounding to the corresponding half-precision elements of the two ZA single-vector groups. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

The 8-bit floating-point pairs within the second source vector are specified using an immediate index that selects the same pair position within each 128-bit vector segment.

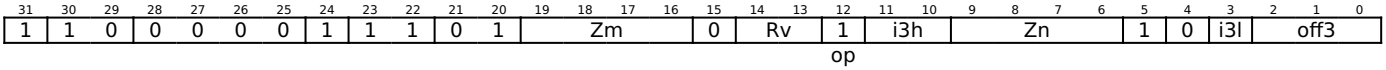
The single-vector group within each half of the ZA array is selected by the sum of the vector select register and offset, modulo half the number of ZA array vectors.

The vector group symbol VGx2 indicates that the ZA operand consists of two ZA single-vector groups. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

SME2

(FEAT_SME_F8F16)



Encoding

```
FVDOT ZA.H[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F16) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i3h::i3l);
let esize : integer{} = 16;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a pair of 8-bit elements within each 128-bit vector segment, in the range 0 to 7, encoded in the "i3h:i3l" fields.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 2;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to 1 do
    let operand1a : bits(VL) = Z{VL}(n+0);
    let operand1b : bits(VL) = Z{VL}(n+1);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var op1 : bits(16);
        op1[0+:8] = operand1a[(2 * e + r)*:8];
        op1[1*:8] = operand1b[(2 * e + r)*:8];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        let op2 : bits(16) = operand2[s*:16];
        var sum : bits(esize) = operand3[e*:esize];
        sum = FP8DotAddFP{16, 16}(sum, op1, op2, FPCR(), FPMR());
        result[e*:esize] = sum;
    end;

    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FVDOTB

Multi-vector 8-bit floating-point vertical dot product by indexed element to single-precision (bottom)

This instruction calculates the fused sum-of-products of each vertical pair of 8-bit floating-point values held in the corresponding elements of the two first source vectors with the lower-numbered horizontal pair of 8-bit floating-point values in the indexed 32-bit group of the corresponding 128-bit segment of the second source vector. The single-precision sum-of-products are scaled by $2^{-UInt(FPMR.LSCALE)}$, before being destructively added without intermediate rounding to the corresponding single-precision elements of the four ZA single-vector groups. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

The 8-bit floating-point groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment.

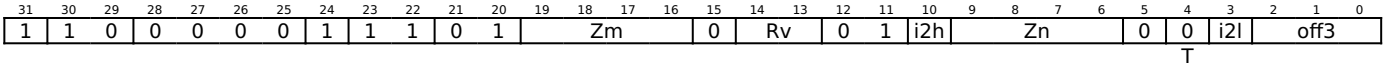
The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.

The vector group symbol VGx4 indicates that the ZA operand consists of four ZA single-vector groups.

This instruction is unpredicated.

SME2

(FEAT_SME_F8F32)



Encoding

```
FVDOTB ZA.S[<Wv>, <offs>, VGx4], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2h::i2l);
let esize : integer{} = 32;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 4;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to 3 do
    let operand1a : bits(VL) = Z{VL}(n+0);
    let operand1b : bits(VL) = Z{VL}(n+1);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var op1 : bits(16);
        op1[0+:8] = operand1a[(4 * e + r)*:8];
        op1[1*:8] = operand1b[(4 * e + r)*:8];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = 2 * (segmentbase + index);
        let op2 : bits(16) = operand2[s*:16];
        var sum : bits(esize) = operand3[e*:esize];
        sum = FP8DotAddFP{32, 16}(sum, op1, op2, FPCR(), FPMR());
        result[e*:esize] = sum;
    end;

    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

FVDOTT

Multi-vector 8-bit floating-point vertical dot product by indexed element to single-precision (top)

This instruction calculates the fused sum-of-products of each vertical pair of 8-bit floating-point values held in the corresponding elements of the two first source vectors with the higher-numbered horizontal pair of 8-bit floating-point values in the indexed 32-bit group of the corresponding 128-bit segment of the second source vector. The single-precision sum-of-products are scaled by $2^{-UInt(FPMR.LSCALE)}$, before being destructively added without intermediate rounding to the corresponding single-precision elements of the four ZA single-vector groups. The 8-bit floating-point encoding format for the elements of the first source vector and the second source vector is selected by FPMR.F8S1 and FPMR.F8S2 respectively.

The 8-bit floating-point groups within the second source vector are specified using an immediate index that selects the same group position within each 128-bit vector segment.

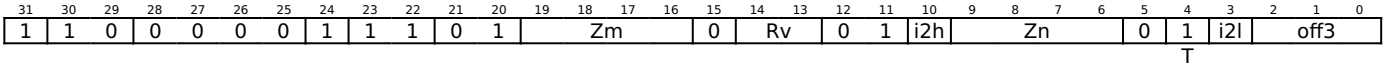
The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.

The vector group symbol VGx4 indicates that the ZA operand consists of four ZA single-vector groups.

This instruction is unpredicated.

SME2

(FEAT_SME_F8F32)



Encoding

```
FVDOTT ZA.S[<Wv>, <offs>, VGx4], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_F8F32) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2h::i2l);
let esize : integer{} = 32;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2h:i2l" fields.

Operation

```
CheckFPMREnabled();
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 4;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to 3 do
    let operand1a : bits(VL) = Z{VL}(n+0);
    let operand1b : bits(VL) = Z{VL}(n+1);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var op1 : bits(16);
        op1[0+:8] = operand1a[(4 * e + r)*:8];
        op1[1*:8] = operand1b[(4 * e + r)*:8];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = 2 * (segmentbase + index) + 1;
        let op2 : bits(16) = operand2[s*:16];
        var sum : bits(esize) = operand3[e*:esize];
        sum = FP8DotAddFP{32, 16}(sum, op1, op2, FPCR(), FPMR());
        result[e*:esize] = sum;
    end;

    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1B (scalar plus immediate, strided registers)

Contiguous load of bytes to multiple strided vectors (immediate index)

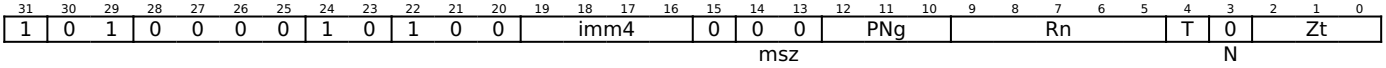
This instruction performs a contiguous load of unsigned bytes to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

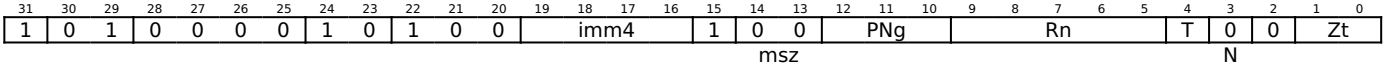
```
LD1B { <Zt1>.B, <Zt2>.B }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
LD1B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt".
	For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt".
	For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.
	For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1B (scalar plus scalar, strided registers)

Contiguous load of bytes to multiple strided vectors (scalar index)

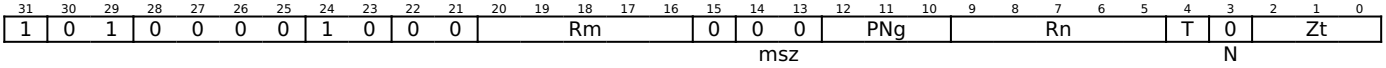
This instruction performs a contiguous load of unsigned bytes to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

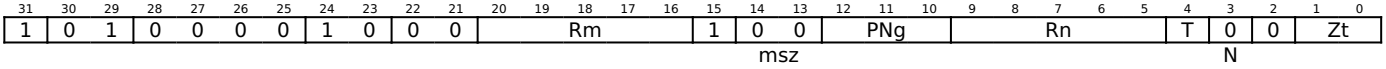
```
LD1B { <Zt1>.B, <Zt2>.B }, <PNg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 8;
```

Four registers

(FEAT_SME2)



Encoding

```
LD1B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <PNg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 8;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

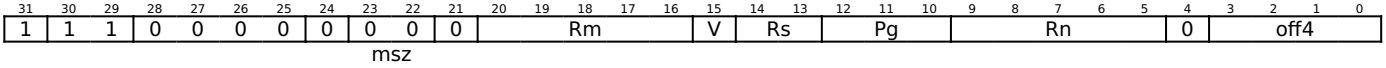
LD1B (scalar plus scalar, tile slice)

Contiguous load of bytes to 8-bit element ZA tile slice

This instruction performs a contiguous load of bytes to an 8-bit element ZA tile slice. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of 8-bit elements in a vector. The immediate offset is in the range 0 to 15. The memory address is generated by a 64-bit scalar base and an optional 64-bit scalar offset that is added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

SME

(FEAT_SME)



Encoding

```
LD1B { ZA0<HV>.B[<Ws>, <offs>] }, <Pg>/Z, [<Xn|SP>{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('0'::Pg);
let s : integer = UInt('011'::Rs);
let t : integer = 0;
let offset : integer = UInt(off4);
let esize : integer{} = 8;
let vertical : boolean = V == '1';
```

Assembler Symbols

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> Is the slice index offset, in the range 0 to 15, encoded in the "off4" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mask : bits(PL) = P{}(g);
var moffs : bits(64) = X{}(m);
let index : bits(32) = X{}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    if (AnyActiveElement{PL}(mask, esize) ||
          ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE)) then
        CheckSPAlignment();
    end;
    base = SP{64}();
else
    base = X{64}(n);
end;

for e = 0 to dim - 1 do
    addr = AddressAdd(base, UInt(moffs) * mbytes, accdesc);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    moffs = moffs + 1;
end;
ZAslice{VL}(t, esize, vertical, slice) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1D (scalar plus immediate, strided registers)

Contiguous load of doublewords to multiple strided vectors (immediate index)

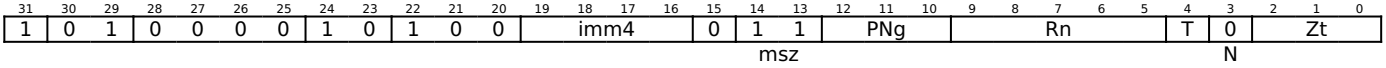
This instruction performs a contiguous load of unsigned doublewords to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

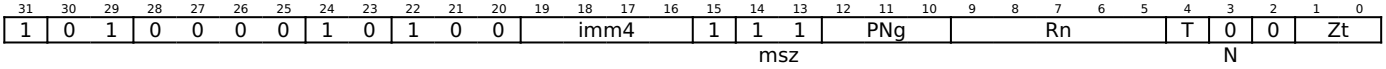
```
LD1D { <Zt1>.D, <Zt2>.D }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
LD1D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1D (scalar plus scalar, strided registers)

Contiguous load of doublewords to multiple strided vectors (scalar index)

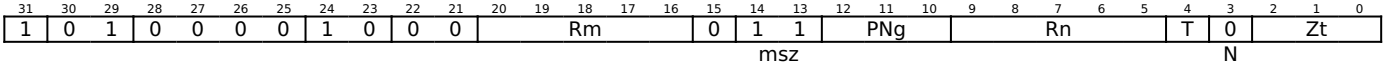
This instruction performs a contiguous load of unsigned doublewords to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

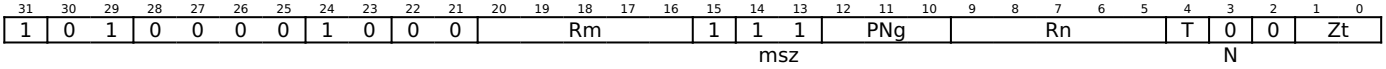
```
LD1D { <Zt1>.D, <Zt2>.D }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 64;
```

Four registers

(FEAT_SME2)



Encoding

```
LD1D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 64;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

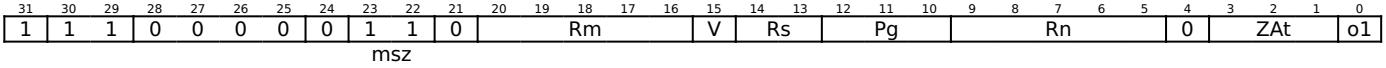
LD1D (scalar plus scalar, tile slice)

Contiguous load of doublewords to 64-bit element ZA tile slice

This instruction performs a contiguous load of doublewords to a 64-bit element ZA tile slice. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of 64-bit elements in a vector. The immediate offset is in the range 0 to 1. The memory address is generated by a 64-bit scalar base and an optional 64-bit scalar offset that is multiplied by 8 and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

SME

(FEAT_SME)



Encoding

```
LD1D { <ZAt><HV>.D[<Ws>, <offs>] }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #3}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('0'::Pg);
let s : integer = UInt('011'::Rs);
let t : integer = UInt(ZAt);
let offset : integer = UInt(o1);
let esize : integer{} = 64;
let vertical : boolean = V == '1';
```

Assembler Symbols

<ZAt> Is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAt" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> Is the slice index offset, in the range 0 to 1, encoded in the "o1" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mask : bits(PL) = P{}(g);
var moffs : bits(64) = X{}(m);
let index : bits(32) = X{}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    if (AnyActiveElement{PL}(mask, esize) ||
          ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE)) then
        CheckSPAlignment();
    end;
    base = SP{64}();
else
    base = X{64}(n);
end;

for e = 0 to dim - 1 do
    addr = AddressAdd(base, UInt(moffs) * mbytes, accdesc);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    moffs = moffs + 1;
end;
ZAslice{VL}(t, esize, vertical, slice) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1H (scalar plus immediate, strided registers)

Contiguous load of halfwords to multiple strided vectors (immediate index)

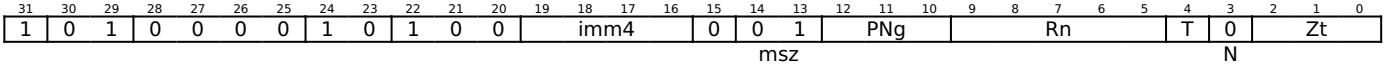
This instruction performs a contiguous load of unsigned halfwords to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

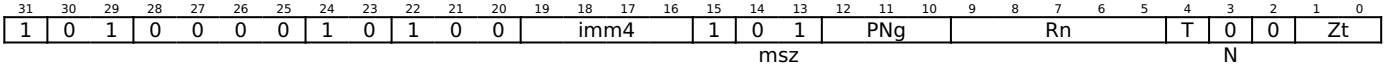
```
LD1H { <Zt1>.H, <Zt2>.H }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
LD1H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt".
	For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt".
	For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.
	For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1H (scalar plus scalar, strided registers)

Contiguous load of halfwords to multiple strided vectors (scalar index)

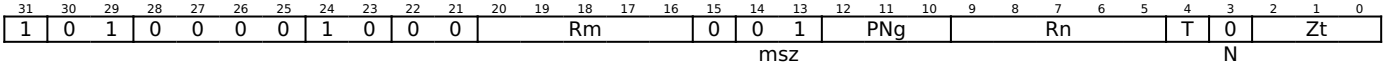
This instruction performs a contiguous load of unsigned halfwords to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

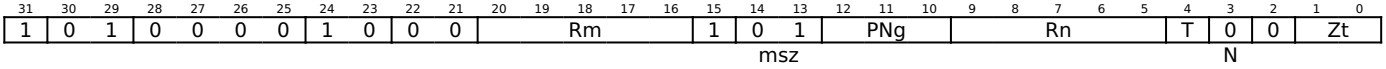
```
LD1H { <Zt1>.H, <Zt2>.H }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 16;
```

Four registers

(FEAT_SME2)



Encoding

```
LD1H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 16;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

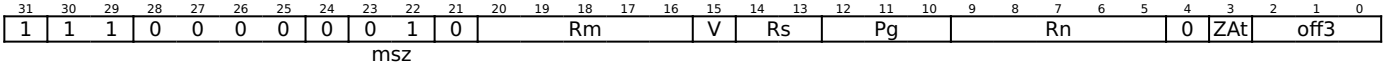
LD1H (scalar plus scalar, tile slice)

Contiguous load of halfwords to 16-bit element ZA tile slice

This instruction performs a contiguous load of halfwords to a 16-bit element ZA tile slice. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of 16-bit elements in a vector. The immediate offset is in the range 0 to 7. The memory address is generated by a 64-bit scalar base and an optional 64-bit scalar offset that is multiplied by 2 and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

SME

(FEAT_SME)



Encoding

```
LD1H { <ZAt><HV>.H[<Ws>, <offs>] }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #1}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('0'::Pg);
let s : integer = UInt('011'::Rs);
let t : integer = UInt(ZAt);
let offset : integer = UInt(off3);
let esize : integer{} = 16;
let vertical : boolean = V == '1';
```

Assembler Symbols

<ZAt> Is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAt" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> Is the slice index offset, in the range 0 to 7, encoded in the "off3" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mask : bits(PL) = P{}(g);
var moffs : bits(64) = X{}(m);
let index : bits(32) = X{}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    if (AnyActiveElement{PL}(mask, esize) ||
          ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE)) then
        CheckSPAlignment();
    end;
    base = SP{64}();
else
    base = X{64}(n);
end;

for e = 0 to dim - 1 do
    addr = AddressAdd(base, UInt(moffs) * mbytes, accdesc);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    moffs = moffs + 1;
end;
ZAslice{VL}(t, esize, vertical, slice) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

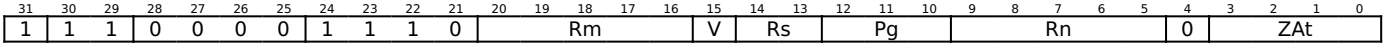
LD1Q

Contiguous load of quadwords to 128-bit element ZA tile slice

This instruction performs a contiguous load of quadwords to a 128-bit element ZA tile slice. The slice number in the tile is selected by the slice index register, modulo the number of 128-bit elements in a Streaming SVE vector. The memory address is generated by scalar base and optional scalar offset that is multiplied by 16 and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

SME

(FEAT_SME)



Encoding

```
LD1Q { <ZAt><HV>.Q[<Ws>, <offs>] }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #4}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('0'::Pg);
let s : integer = UInt('011'::Rs);
let t : integer = UInt(ZAt);
let offset : integer = 0;
let esize : integer{} = 128;
let vertical : boolean = V == '1';
```

Assembler Symbols

<ZAt> Is the name of the ZA tile ZA0-ZA15 to be accessed, encoded in the "ZAt" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> Is the slice index offset, with implicit value 0.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mask : bits(PL) = P{}(g);
var moffs : bits(64) = X{}(m);
let index : bits(32) = X{}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    if (AnyActiveElement{PL}(mask, esize) ||
          ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE)) then
        CheckSPAlignment();
    end;
    base = SP{64}();
else
    base = X{64}(n);
end;

for e = 0 to dim - 1 do
    addr = AddressAdd(base, UInt(moffs) * mbytes, accdesc);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    moffs = moffs + 1;
end;
ZAslice{VL}(t, esize, vertical, slice) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1W (scalar plus immediate, strided registers)

Contiguous load of words to multiple strided vectors (immediate index)

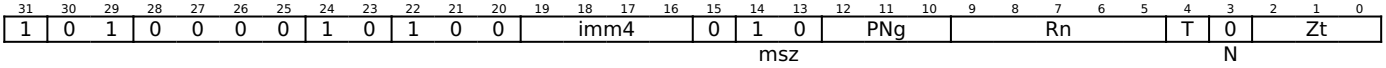
This instruction performs a contiguous load of unsigned words to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

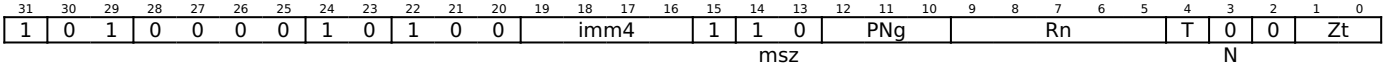
```
LD1W { <Zt1>.S, <Zt2>.S }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
LD1W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt".
	For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt".
	For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.
	For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LD1W (scalar plus scalar, strided registers)

Contiguous load of words to multiple strided vectors (scalar index)

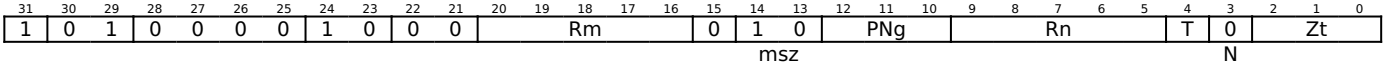
This instruction performs a contiguous load of unsigned words to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

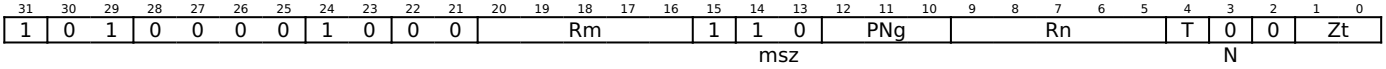
```
LD1W { <Zt1>.S, <Zt2>.S }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 32;
```

Four registers

(FEAT_SME2)



Encoding

```
LD1W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 32;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

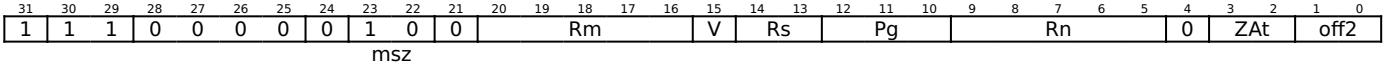
LD1W (scalar plus scalar, tile slice)

Contiguous load of words to 32-bit element ZA tile slice

This instruction performs a contiguous load of words to a 32-bit element ZA tile slice. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of 32-bit elements in a vector. The immediate offset is in the range 0 to 3. The memory address is generated by a 64-bit scalar base and an optional 64-bit scalar offset that is multiplied by 4 and added to the base address. Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

SME

(FEAT_SME)



Encoding

```
LD1W { <ZAt><HV>.S[<Ws>, <offs>] }, <Pg>/Z, [<Xn|SP>{, <Xm>, LSL #2}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('0'::Pg);
let s : integer = UInt('011'::Rs);
let t : integer = UInt(ZAt);
let offset : integer = UInt(off2);
let esize : integer{} = 32;
let vertical : boolean = V == '1';
```

Assembler Symbols

<ZAt> Is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAt" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> Is the slice index offset, in the range 0 to 3, encoded in the "off2" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mask : bits(PL) = P{}(g);
var moffs : bits(64) = X{}(m);
let index : bits(32) = X{}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var result : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    if (AnyActiveElement{PL}(mask, esize) ||
          ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE)) then
        CheckSPAlignment();
    end;
    base = SP{64}();
else
    base = X{64}(n);
end;

for e = 0 to dim - 1 do
    addr = AddressAdd(base, UInt(moffs) * mbytes, accdesc);
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = Mem{esize}(addr, accdesc);
    else
        result[e*:esize] = Zeros{esize};
    end;
    moffs = moffs + 1;
end;
ZAslice{VL}(t, esize, vertical, slice) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1B (scalar plus immediate, strided registers)

Contiguous load non-temporal of bytes to multiple strided vectors (immediate index)

This instruction performs a contiguous non-temporal load of bytes to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

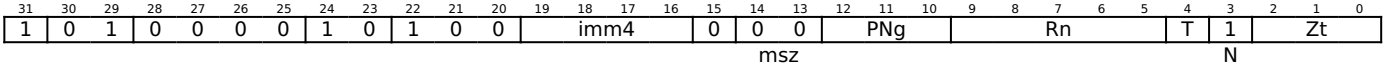
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

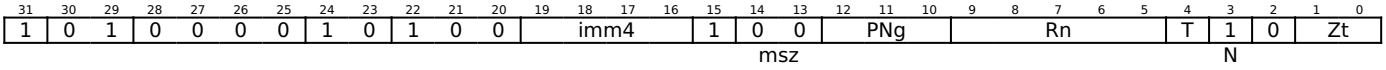
```
LDNT1B { <Zt1>.B, <Zt2>.B }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
LDNT1B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".

<Zt4> Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1B (scalar plus scalar, strided registers)

Contiguous load non-temporal of bytes to multiple strided vectors (scalar index)

This instruction performs a contiguous non-temporal load of bytes to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

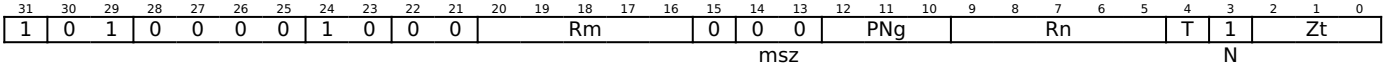
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

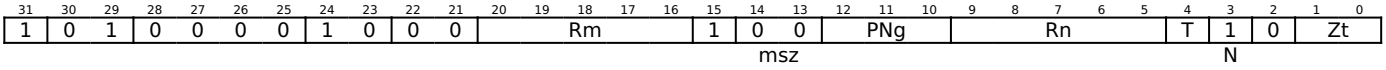
```
LDNT1B { <Zt1>.B, <Zt2>.B }, <PNg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 8;
```

Four registers

(FEAT_SME2)



Encoding

```
LDNT1B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <PNg>/Z, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 8;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1D (scalar plus immediate, strided registers)

Contiguous load non-temporal of doublewords to multiple strided vectors (immediate index)

This instruction performs a contiguous non-temporal load of doublewords to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

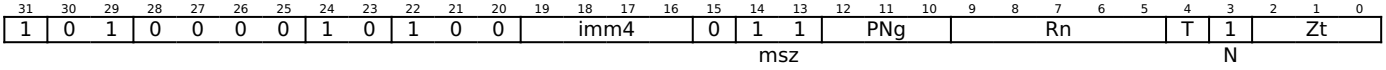
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

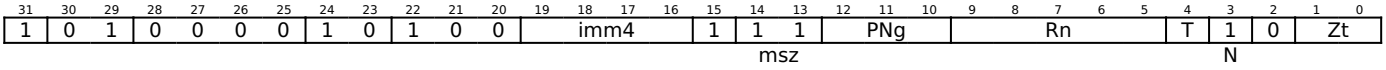
```
LDNT1D { <Zt1>.D, <Zt2>.D }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
LDNT1D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".

<Zt4> Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1D (scalar plus scalar, strided registers)

Contiguous load non-temporal of doublewords to multiple strided vectors (scalar index)

This instruction performs a contiguous non-temporal load of doublewords to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

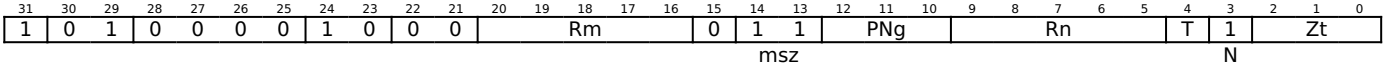
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

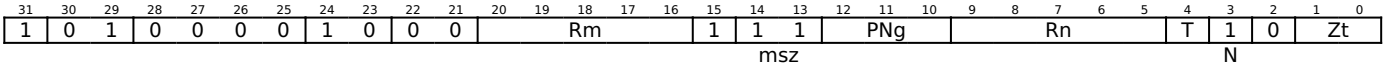
```
LDNT1D { <Zt1>.D, <Zt2>.D }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 64;
```

Four registers

(FEAT_SME2)



Encoding

```
LDNT1D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 64;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1H (scalar plus immediate, strided registers)

Contiguous load non-temporal of halfwords to multiple strided vectors (immediate index)

This instruction performs a contiguous non-temporal load of halfwords to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

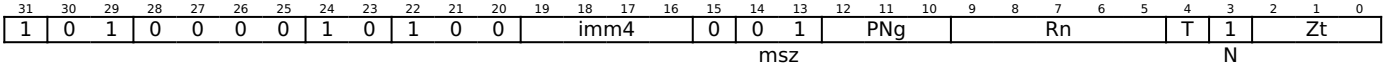
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

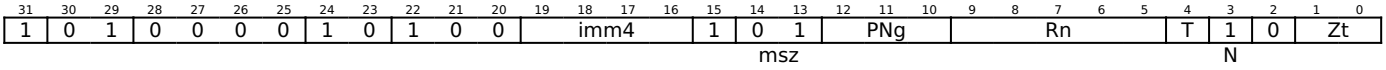
```
LDNT1H { <Zt1>.H, <Zt2>.H }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
LDNT1H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".

<Zt4> Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1H (scalar plus scalar, strided registers)

Contiguous load non-temporal of halfwords to multiple strided vectors (scalar index)

This instruction performs a contiguous non-temporal load of halfwords to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

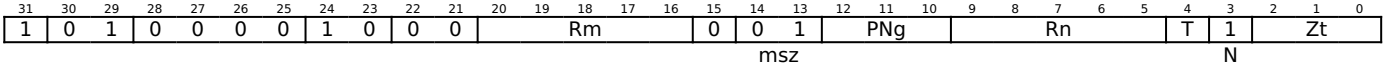
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

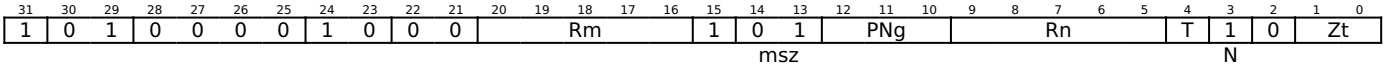
```
LDNT1H { <Zt1>.H, <Zt2>.H }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 16;
```

Four registers

(FEAT_SME2)



Encoding

```
LDNT1H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 16;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1W (scalar plus immediate, strided registers)

Contiguous load non-temporal of words to multiple strided vectors (immediate index)

This instruction performs a contiguous non-temporal load of words to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

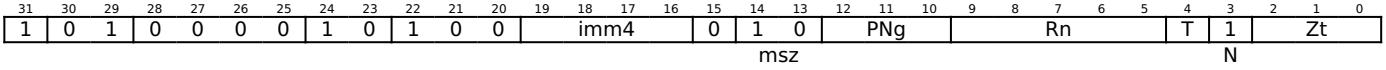
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

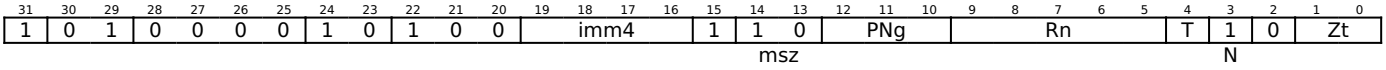
```
LDNT1W { <Zt1>.S, <Zt2>.S }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
LDNT1W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <PNg>/Z, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".

<Zt4> Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDNT1W (scalar plus scalar, strided registers)

Contiguous load non-temporal of words to multiple strided vectors (scalar index)

This instruction performs a contiguous non-temporal load of words to elements of two or four strided vector registers from the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

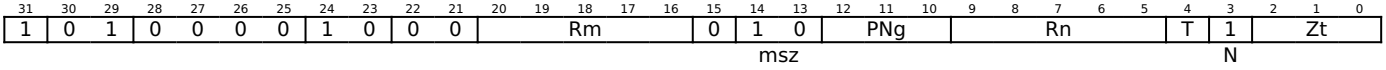
Inactive elements will not cause a read from Device memory or signal a fault, and are set to zero in the destination vector.

A non-temporal load is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

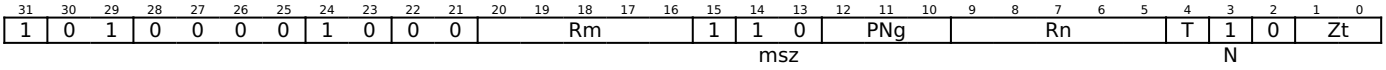
```
LDNT1W { <Zt1>.S, <Zt2>.S }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 32;
```

Four registers

(FEAT_SME2)



Encoding

```
LDNT1W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <PNg>/Z, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 32;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var values : array [[4]] of bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            values[[r]][e*:esize] = Mem{esize}(addr, accdesc);
        else
            values[[r]][e*:esize] = Zeros{esize};
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(transfer) = values[[r]];
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDR (array vector)

Load ZA array vector

This instruction performs a contiguous load of bytes to a ZA array vector from the memory address generated by a 64-bit scalar base plus an optional immediate offset multiplied by the current vector length in bytes. The ZA array vector is selected by the sum of the vector select register and the same immediate offset, modulo the number of bytes in a Streaming SVE vector.

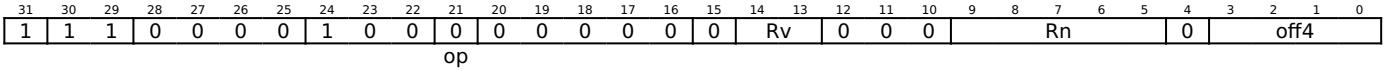
This instruction is unpredicated.

The load is performed as contiguous byte accesses, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then the base register must be aligned to 16 bytes.

This instruction does not require the PE to be in Streaming SVE mode, and it is expected that this instruction will not experience a significant slowdown due to contention with other PEs that are executing in Streaming SVE mode.

SME

(FEAT_SME)



Encoding

```
LDR ZA[<Wv>, <offs>], [<Xn|SP>{, #<offs>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let v : integer = UInt('011'::Rv);
let offset : integer = UInt(off4);
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W12-W15, encoded as "Rv" plus 12.
<offs>	Is the vector select offset and optional memory offset, in the range 0 to 15, defaulting to 0, encoded in the "off4" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckSMEAndZAEnabled();
let SVL : integer{} = CurrentSVL();
let dim : integer{} = SVL DIV 8;
var base : bits(64);
let moffs : integer = offset * dim;
var result : bits(SVL);
let vbase : bits(32) = X{32}(v);
let vec : integer = (UInt(vbase) + offset) MOD dim;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;
var addr : bits(64) = AddressAdd(base, moffs, accdesc);
let aligned : boolean = IsAlignedSize(base, 16);
if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;
for e = 0 to dim-1 do
    result[e*:8] = AArch64_MemSingle{8}(addr, accdesc, aligned);
    addr = AddressIncrement(addr, 1, accdesc);
end;

ZAvector{SVL}(vec) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LDR (table)

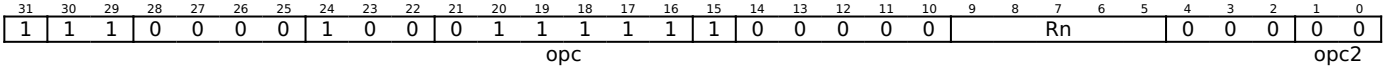
Load ZT0 register

This instruction loads the 64-byte ZT0 register from the memory address provided in the 64-bit scalar base register. This instruction is unpredicated. The load is performed as contiguous byte accesses, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then the base register must be aligned to 16 bytes.

This instruction does not require the PE to be in Streaming SVE mode, and it is expected that this instruction will not experience a significant slowdown due to contention with other PEs that are executing in Streaming SVE mode.

SME2

(FEAT_SME2)



Encoding

```
LDR ZT0, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let esize : integer{} = 8;
```

Assembler Symbols

<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckSMEEnabled();
CheckSMEZT0Enabled();
let tablesize : integer{} = 512;
let elements : integer = tablesize DIV esize;
var addr : bits(64);
var result : bits(tablesize);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_LOAD, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    CheckSPAlignment();
    addr = SP{64}();
else
    addr = X{64}(n);
end;
let aligned : boolean = IsAlignedSize(addr, 16);
if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;
for e = 0 to elements-1 do
    result[e*:esize] = AArch64_MemSingle{esize}(addr, accdesc, aligned);
    addr = AddressIncrement(addr, esize DIV 8, accdesc);
end;

ZT0{tablesize}() = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI2 (four registers)

Lookup table read with 2-bit indexes (four registers)

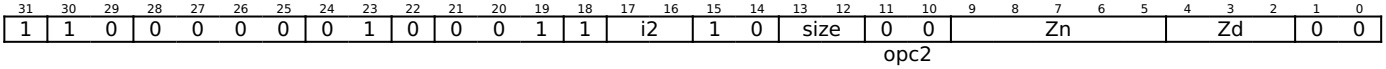
This instruction copies 8-bit, 16-bit or 32-bit elements from ZT0 to four destination vectors using packed 2-bit indices from a segment of the source vector register. A segment corresponds to a portion of the source vector that is consumed in order to fill the destination vector. The segment is selected by the vector segment index modulo the total number of segments.

This instruction is unpredicated.

It has encodings from 2 classes: Consecutive and Strided

Consecutive

(FEAT_SME2)



Encoding

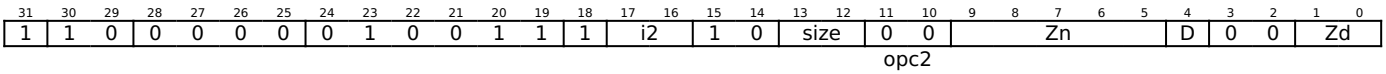
```
LUTI2 { <Zd1>.<T>-<Zd4>.<T> }, ZT0, <Zn>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 2;
let n : integer = UInt(Zn);
let dstride : integer = 1;
let d : integer = UInt(Zd::'00');
let imm : integer = UInt(i2);
let nreg : integer{} = 4;
```

Strided

(FEAT_SME2p1)



Encoding

```
LUTI2 { <Zd1>.<T>, <Zd2>.<T>, <Zd3>.<T>, <Zd4>.<T> }, ZT0, <Zn>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
if size == '10' || size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 2;
let n : integer = UInt(Zn);
let dstride : integer = 4;
let d : integer = UInt(D::'00'::Zd);
let imm : integer = UInt(i2);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1> For the "Consecutive" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

For the "Strided" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 of the destination multi-vector group, encoded as "D:'00':Zd".

<T> For the "Consecutive" variant: is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

For the "Strided" variant: is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	B
1	H

<Zd4>	For the "Consecutive" variant: is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3. For the "Strided" variant: is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 of the destination multi-vector group, encoded as "D:'11':Zd".
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.
<index>	Is the vector segment index, in the range 0 to 3, encoded in the "i2" field.
<Zd2>	Is the name of the second scalable vector register Z4-Z7 or Z20-Z23 of the destination multi-vector group, encoded as "D:'01':Zd".
<Zd3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 of the destination multi-vector group, encoded as "D:'10':Zd".

Operation

```
CheckStreamingSVEEnabled();
CheckSMEZT0Enabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let segments : integer = (esize DIV (isize * nreg)) as integer{1, 2, 4, 8, 16};
let segment : integer = imm MOD segments;
let indexes : bits(VL) = Z{VL}(n);
var dst : integer = d;
let table : bits(512) = ZT0{}();

for r = 0 to nreg-1 do
    let base : integer = (segment * nreg + r) * elements;
    var result : bits(VL);
    for e = 0 to elements-1 do
        let index : integer = UInt(indexes[(base+e)*:isize]);
        result[e*:esize] = table[index*:32][esize-1:0];
    end;
    Z{VL}(dst) = result;
    dst = dst + dstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI2 (single)

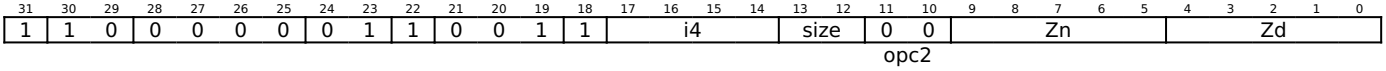
Lookup table read with 2-bit indexes (single)

This instruction copies 8-bit, 16-bit or 32-bit elements from ZT0 to one destination vector using packed 2-bit indices from a segment of the source vector register. A segment corresponds to a portion of the source vector that is consumed in order to fill the destination vector. The segment is selected by the vector segment index modulo the total number of segments.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
LUTI2 <Zd>.<T>, ZT0, <Zn>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 2;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let imm : integer = UInt(i4);
let nreg : integer{} = 1;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<index> Is the vector segment index, in the range 0 to 15, encoded in the "i4" field.

Operation

```
CheckStreamingSVEEnabled();
CheckSMEZT0Enabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let segments : integer = (esize DIV (isize * nreg)) as integer{1, 2, 4, 8, 16};
let segment : integer = imm MOD segments;
let indexes : bits(VL) = Z{VL}(n);
let dst : integer = d;
let table : bits(512) = ZT0{}();

for r = 0 to nreg-1 do
    let base : integer = (segment * nreg + r) * elements;
    var result : bits(VL);
    for e = 0 to elements-1 do
        let index : integer = UInt(indexes[(base+e)*:isize]);
        result[e*:esize] = table[index*:32][esize-1:0];
    end;
    Z{VL}(dst+r) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI2 (two registers)

Lookup table read with 2-bit indexes (two registers)

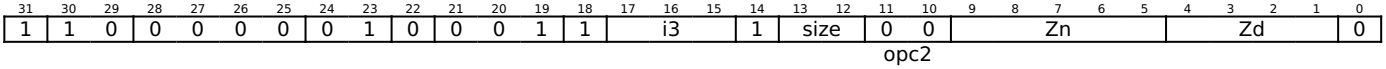
This instruction copies 8-bit, 16-bit or 32-bit elements from ZT0 to two destination vectors using packed 2-bit indices from a segment of the source vector register. A segment corresponds to a portion of the source vector that is consumed in order to fill the destination vector. The segment is selected by the vector segment index modulo the total number of segments.

This instruction is unpredicated.

It has encodings from 2 classes: Consecutive and Strided

Consecutive

(FEAT_SME2)



Encoding

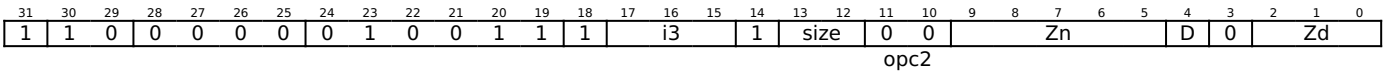
```
LUTI2 { <Zd1>.<T>-<Zd2>.<T> }, ZT0, <Zn>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 2;
let n : integer = UInt(Zn);
let dstride : integer = 1;
let d : integer = UInt(Zd::'0');
let imm : integer = UInt(i3);
let nreg : integer{} = 2;
```

Strided

(FEAT_SME2p1)



Encoding

```
LUTI2 { <Zd1>.<T>, <Zd2>.<T> }, ZT0, <Zn>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
if size == '10' || size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 2;
let n : integer = UInt(Zn);
let dstride : integer = 8;
let d : integer = UInt(D::'0'::Zd);
let imm : integer = UInt(i3);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zd1> For the "Consecutive" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Strided" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 of the destination multi-vector group, encoded as "D:'0':Zd".

<T> For the "Consecutive" variant: is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

For the "Strided" variant: is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	B
1	H

<Zd2>	For the "Consecutive" variant: is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1. For the "Strided" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 of the destination multi-vector group, encoded as "D:'1':Zd".
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.
<index>	Is the vector segment index, in the range 0 to 7, encoded in the "i3" field.

Operation

```
CheckStreamingSVEEnabled();
CheckSMEZT0Enabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let segments : integer = (esize DIV (isize * nreg)) as integer{1, 2, 4, 8, 16};
let segment : integer = imm MOD segments;
let indexes : bits(VL) = Z{VL}(n);
var dst : integer = d;
let table : bits(512) = ZT0{}();

for r = 0 to nreg-1 do
    let base : integer = (segment * nreg + r) * elements;
    var result : bits(VL);
    for e = 0 to elements-1 do
        let index : integer = UInt(indexes[(base+e)*:isize]);
        result[e*:esize] = table[index*:32][esize-1:0];
    end;
    Z{VL}(dst) = result;
    dst = dst + dstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI4 (four registers, 16-bit and 32-bit)

Lookup table read with 4-bit indexes (four registers)

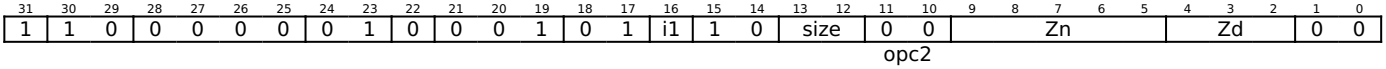
This instruction copies 16-bit or 32-bit elements from ZT0 to four destination vectors using packed 4-bit indices from a segment of the source vector register. A segment corresponds to a portion of the source vector that is consumed in order to fill the destination vector. The segment is selected by the vector segment index modulo the total number of segments.

This instruction is unpredicated.

It has encodings from 2 classes: Consecutive and Strided

Consecutive

(FEAT_SME2)



Encoding

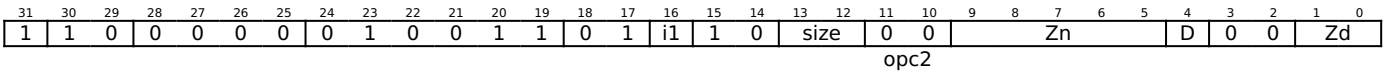
```
LUTI4 { <Zd1>.<T>-<Zd4>.<T> }, ZT0, <Zn>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '00' || size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 4;
let n : integer = UInt(Zn);
let dstride : integer = 1;
let d : integer = UInt(Zd::'00');
let imm : integer = UInt(i1);
let nreg : integer{} = 4;
```

Strided

(FEAT_SME2p1)



Encoding

```
LUTI4 { <Zd1>.H, <Zd2>.H, <Zd3>.H, <Zd4>.H }, ZT0, <Zn>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
if size != '01' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 4;
let n : integer = UInt(Zn);
let dstride : integer = 4;
let d : integer = UInt(D::'00'::Zd);
let imm : integer = UInt(i1);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1> For the "Consecutive" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

For the "Strided" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 of the destination multi-vector group, encoded as "D:'00':Zd".

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	RESERVED

<Zd4> For the "Consecutive" variant: is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

For the "Strided" variant: is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 of the destination multi-vector group, encoded as "D:'11':Zd".

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<index> Is the vector segment index, in the range 0 to 1, encoded in the "i1" field.

<Zd2> Is the name of the second scalable vector register Z4-Z7 or Z20-Z23 of the destination multi-vector group, encoded as "D:'01':Zd".

<Zd3> Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 of the destination multi-vector group, encoded as "D:'10':Zd".

Operation

```
CheckStreamingSVEEnabled();
CheckSMEZT0Enabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let segments : integer = (esize DIV (isize * nreg)) as integer{1, 2, 4, 8, 16};
let segment : integer = imm MOD segments;
let indexes : bits(VL) = Z{VL}(n);
var dst : integer = d;
let table : bits(512) = ZT0{}();

for r = 0 to nreg-1 do
    let base : integer = (segment * nreg + r) * elements;
    var result : bits(VL);
    for e = 0 to elements-1 do
        let index : integer = UInt(indexes[(base+e)*:isize]);
        result[e*:esize] = table[index*:32][esize-1:0];
    end;
    Z{VL}(dst) = result;
    dst = dst + dstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

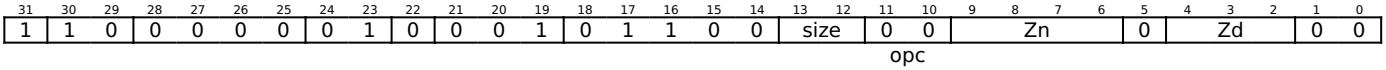
LUTI4 (four registers, 8-bit)

Lookup table read with 4-bit indexes and 8-bit elements (four registers)

This instruction copies 8-bit elements from ZT0 to four destination vectors using packed 4-bit indices in the two source vectors.
This instruction is unpredicated.
It has encodings from 2 classes: Consecutive and Strided

Consecutive

(FEAT_SME_LUTv2)



Encoding

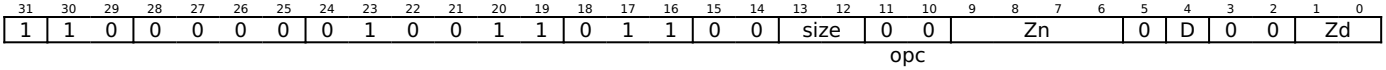
```
LUTI4 { <Zd1>.B-<Zd4>.B }, ZT0, { <Zn1>-<Zn2> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_LUTv2) then EndOfDecode(Decode_UNDEF); end;
if size != '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 4;
let n : integer = UInt(Zn::'0');
let dstride : integer = 1;
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
```

Strided

(FEAT_SME2p1 && FEAT_SME_LUTv2)



Encoding

```
LUTI4 { <Zd1>.B, <Zd2>.B, <Zd3>.B, <Zd4>.B }, ZT0, { <Zn1>-<Zn2> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) || !IsFeatureImplemented(FEAT_SME_LUTv2) then
    EndOfDecode(Decode_UNDEF);
end;
if size != '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 4;
let n : integer = UInt(Zn::'0');
let dstride : integer = 4;
let d : integer = UInt(D::'00'::Zd);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1>	For the "Consecutive" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4. For the "Strided" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 of the destination multi-vector group, encoded as "D:'00':Zd".
<Zd4>	For the "Consecutive" variant: is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3. For the "Strided" variant: is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 of the destination multi-vector group, encoded as "D:'11':Zd".
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zd2>	Is the name of the second scalable vector register Z4-Z7 or Z20-Z23 of the destination multi-vector group, encoded as "D:'01':Zd".
<Zd3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 of the destination multi-vector group, encoded as "D:'10':Zd".

Operation

```
CheckStreamingSVEEnabled();
CheckSMEZT0Enabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let indexes : bits(2 * VL) = Z{VL}(n+1) :: Z{VL}(n);
var dst : integer = d;
let table : bits(512) = ZT0{}();

for r = 0 to nreg-1 do
    let base : integer = r * elements;
    var result : bits(VL);
    for e = 0 to elements-1 do
        let index : integer = UInt(indexes[(base+e)*:isize]);
        result[e*:esize] = table[index*:32][esize-1:0];
    end;
    Z{VL}(dst) = result;
    dst = dst + dstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI4 (single)

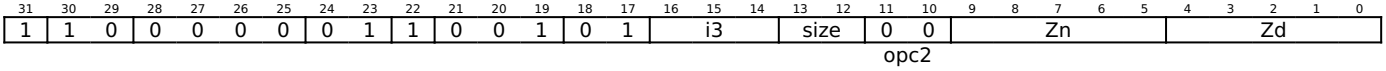
Lookup table read with 4-bit indexes (single)

This instruction copies 8-bit, 16-bit or 32-bit elements from ZT0 to one destination vector using packed 4-bit indices from a segment of the source vector register. A segment corresponds to a portion of the source vector that is consumed in order to fill the destination vector. The segment is selected by the vector segment index modulo the total number of segments.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
LUTI4 <Zd>.<T>, ZT0, <Zn>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 4;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
let imm : integer = UInt(i3);
let nreg : integer{} = 1;
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<index> Is the vector segment index, in the range 0 to 7, encoded in the "i3" field.

Operation

```
CheckStreamingSVEEnabled();
CheckSMEZT0Enabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let segments : integer = (esize DIV (isize * nreg)) as integer{1, 2, 4, 8, 16};
let segment : integer = imm MOD segments;
let indexes : bits(VL) = Z{VL}(n);
let dst : integer = d;
let table : bits(512) = ZT0{}();

for r = 0 to nreg-1 do
    let base : integer = (segment * nreg + r) * elements;
    var result : bits(VL);
    for e = 0 to elements-1 do
        let index : integer = UInt(indexes[(base+e)*:isize]);
        result[e*:esize] = table[index*:32][esize-1:0];
    end;
    Z{VL}(dst+r) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI4 (two registers)

Lookup table read with 4-bit indexes (two registers)

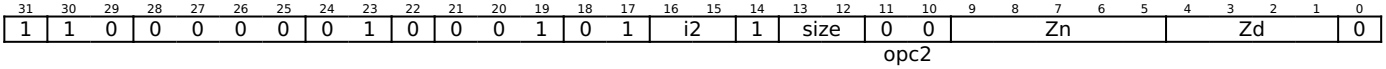
This instruction copies 8-bit, 16-bit or 32-bit elements from ZT0 to two destination vectors using packed 4-bit indices from a segment of the source vector register. A segment corresponds to a portion of the source vector that is consumed in order to fill the destination vector. The segment is selected by the vector segment index modulo the total number of segments.

This instruction is unpredicated.

It has encodings from 2 classes: Consecutive and Strided

Consecutive

(FEAT_SME2)



Encoding

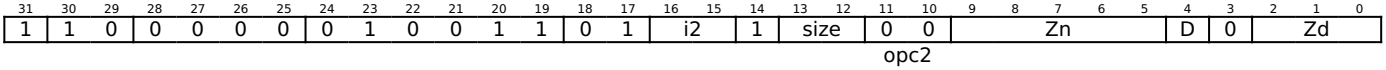
```
LUTI4 { <Zd1>.<T>-<Zd2>.<T> }, ZT0, <Zn>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 4;
let n : integer = UInt(Zn);
let dstride : integer = 1;
let d : integer = UInt(Zd::'0');
let imm : integer = UInt(i2);
let nreg : integer{} = 2;
```

Strided

(FEAT_SME2p1)



Encoding

```
LUTI4 { <Zd1>.<T>, <Zd2>.<T> }, ZT0, <Zn>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
if size == '10' || size == '11' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let isize : integer{} = 4;
let n : integer = UInt(Zn);
let dstride : integer = 8;
let d : integer = UInt(D::'0'::Zd);
let imm : integer = UInt(i2);
let nreg : integer{} = 2;
```

Assembler Symbols

<Zd1> For the "Consecutive" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Strided" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 of the destination multi-vector group, encoded as "D:'0':Zd".

<T> For the "Consecutive" variant: is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	RESERVED

For the "Strided" variant: is the size specifier, encoded in “size[0]”:

size[0]	<T>
0	B
1	H

<Zd2>	For the "Consecutive" variant: is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1. For the "Strided" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 of the destination multi-vector group, encoded as "D:'1':Zd".
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.
<index>	Is the vector segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEEnabled();
CheckSMEZT0Enabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let segments : integer = (esize DIV (isize * nreg)) as integer{1, 2, 4, 8, 16};
let segment : integer = imm MOD segments;
let indexes : bits(VL) = Z{VL}(n);
var dst : integer = d;
let table : bits(512) = ZT0{}();

for r = 0 to nreg-1 do
    let base : integer = (segment * nreg + r) * elements;
    var result : bits(VL);
    for e = 0 to elements-1 do
        let index : integer = UInt(indexes[(base+e)*:isize]);
        result[e*:esize] = table[index*:32][esize-1:0];
    end;
    Z{VL}(dst) = result;
    dst = dst + dstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI6 (table, four registers, 8-bit)

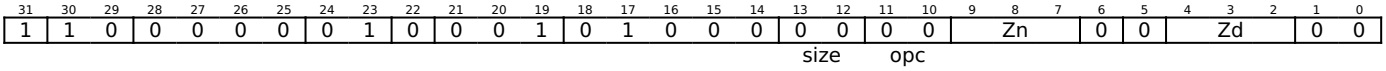
Lookup table read with 6-bit indices (8-bit)

This instruction copies indexed 8-bit elements from ZT0 to four destination vectors using packed 6-bit indices in the three source vectors. This instruction is unpredicated.

It has encodings from 2 classes: Consecutive and Strided

Consecutive

(FEAT_SME2p3)



Encoding

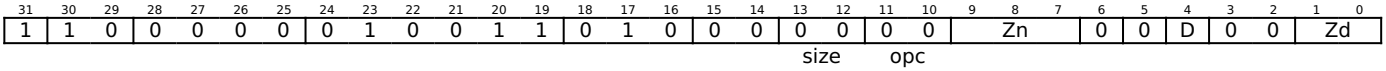
```
LUTI6 { <Zd1>.B-<Zd4>.B }, ZT0, { <Zn1>-<Zn3> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p3) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8;
let isize : integer{} = 6;
let n : integer = UInt('00'::Zn);
let dstride : integer = 1;
let d : integer = UInt(Zd::'00');
```

Strided

(FEAT_SME2p3)



Encoding

```
LUTI6 { <Zd1>.B, <Zd2>.B, <Zd3>.B, <Zd4>.B }, ZT0, { <Zn1>-<Zn3> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p3) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8;
let isize : integer{} = 6;
let n : integer = UInt('00'::Zn);
let dstride : integer = 4;
let d : integer = UInt(D::'00'::Zd);
```

Assembler Symbols

<Zd1>	For the "Consecutive" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4. For the "Strided" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 of the destination multi-vector group, encoded as "D:'00':Zd".
<Zd4>	For the "Consecutive" variant: is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3. For the "Strided" variant: is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 of the destination multi-vector group, encoded as "D:'11':Zd".
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group Z0-Z7, encoded in the "Zn" field.
<Zn3>	Is the name of the third scalable vector register of the source multi-vector group Z2-Z9, encoded as "Zn" plus 2.
<Zd2>	Is the name of the second scalable vector register Z4-Z7 or Z20-Z23 of the destination multi-vector group, encoded as "D:'01':Zd".
<Zd3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 of the destination multi-vector group, encoded as "D:'10':Zd".

Operation

```
CheckStreamingSVEEnabled();
CheckSMEZT0Enabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let indexes : bits(3 * VL) = Z{VL}(n+2) :: Z{VL}(n+1) :: Z{VL}(n);
var dst : integer = d;
let table : bits(512) = ZT0{}();
for r = 0 to 3 do
    let base : integer = r * elements;
    var result : bits(VL);
    for e = 0 to elements-1 do
        let index : integer = UInt(indexes[(base+e)*:isize]);
        result[e*:esize] = table[index*:esize];
    end;
    Z{VL}(dst) = result;
    dst = dst + dstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

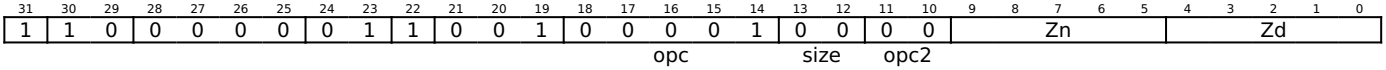
LUTI6 (table, single, 8-bit)

Lookup table read with 6-bit indices (single)

This instruction copies indexed 8-bit elements from ZT0 to the destination vector using packed 6-bit indices from the bottom three quarters of the source vector. This instruction is unpredicated.

SME2

(FEAT_SME2p3)



Encoding

```
LUTI6 <Zd>.B, ZT0, <Zn>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p3) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8;
let isize : integer{} = 6;
let n : integer = UInt(Zn);
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn>	Is the name of the source scalable vector register, encoded in the "Zn" field.

Operation

```
CheckStreamingSVEEnabled();
CheckSMEZT0Enabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let indexes : bits(VL) = Z{VL}(n); // Only 3/4 used
let table : bits(512) = ZT0{}();
var result : bits(VL);
for e = 0 to elements-1 do
    let index : integer = UInt(indexes[e*:isize]);
    result[e*:esize] = table[index*:esize];
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

LUTI6 (vector, 16-bit)

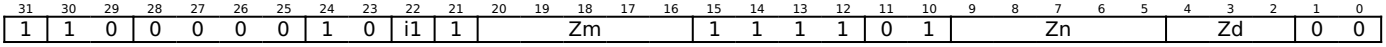
Lookup table read with 6-bit indices (16-bit)

This instruction copies indexed 16-bit elements from the low 512 bits of the two table vectors to four destination vectors using packed 6-bit indices in the low or high one and a half vector segment of the two source vectors. The source vectors segment consumed in order to fill the destination vectors is selected by the source vector segment index. This instruction is unpredicated. This instruction requires that the Effective SVE vector length is at least 512 bits.

It has encodings from 2 classes: Consecutive and Strided

Consecutive

(FEAT_SME2p3)



Encoding

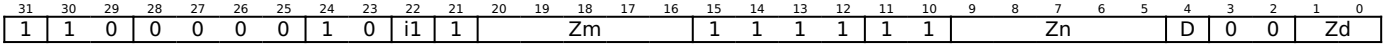
```
LUTI6 { <Zd1>.H-<Zd4>.H }, { <Zn1>.H, <Zn2>.H }, { <Zm1>-<Zm2> }[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p3) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 512 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let isize : integer{} = 6;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let dstride : integer = 1;
let d : integer = UInt(Zd::'00');
let select : integer = UInt(i1);
```

Strided

(FEAT_SME2p3)



Encoding

```
LUTI6 { <Zd1>.H, <Zd2>.H, <Zd3>.H, <Zd4>.H }, { <Zn1>.H, <Zn2>.H }, { <Zm1>-<Zm2> }[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p3) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 512 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let isize : integer{} = 6;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let dstride : integer = 4;
let d : integer = UInt(D::'00'::Zd);
let select : integer = UInt(i1);
```

Assembler Symbols

<Zd1>	For the "Consecutive" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4. For the "Strided" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 of the destination multi-vector group, encoded as "D:'00':Zd".
<Zd4>	For the "Consecutive" variant: is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3. For the "Strided" variant: is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 of the destination multi-vector group, encoded as "D:'11':Zd".
<Zn1>	Is the name of the first table vector register, encoded as "Zn".
<Zn2>	Is the name of the second table vector register, encoded as "Zn" plus 1 modulo 32.
<Zm1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm".
<Zm2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zm" plus 1 modulo 32.
<index>	Is the vector segment index, in the range 0 to 1, encoded in the "i1" field.
<Zd2>	Is the name of the second scalable vector register Z4-Z7 or Z20-Z23 of the destination multi-vector group, encoded as "D:'01':Zd".

<Zd3> Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 of the destination multi-vector group, encoded as "D:'10':Zd".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < 512 then EndOfDecode(Decode_UNDEF); end;
let elements : integer = VL DIV esize;
let operand : bits(2*VL) = Z{VL}((m+1) MOD 32) :: Z{VL}(m);
let idxsize : integer{} = VL + VL DIV 2;
let idxpos : integer{} = if select == 1 then VL DIV 2 else 0;
let indexes : bits(idxsize) = operand[idxpos+:idxsize];
let table : bits(1024) = Z{VL}((n+1) MOD 32)[511:0] :: Z{VL}(n)[511:0];
var dst : integer = d;
var result : bits(VL);

for r = 0 to 3 do
    let base : integer = r * elements;
    for e = 0 to elements-1 do
        let index : integer = UInt(indexes[(base+e)*:isize]);
        result[e*:esize] = table[index*:esize];
    end;
    Z{VL}(dst) = result;
    dst = dst + dstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (array to vector, four registers)

Move four ZA single-vector groups to Z four-vector operand

This instruction operates on four ZA single-vector groups.
The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.
The vector group symbol VGx4 indicates that the instruction operates on four ZA single-vector groups.
The preferred disassembly syntax uses a 64-bit element size, but an assembler should accept any element size if it is used consistently for all operands. The vector group symbol is preferred for disassembly, but optional in assembler source code.
This instruction is unpredicated.

This is an alias of MOVA (array to vector, four registers). This means:

- The encodings in this description are named to match the encodings of MOVA (array to vector, four registers).
- The description of MOVA (array to vector, four registers) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SME2

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	Rv		0	1	1	0	0	off3			Zd			0	0

Encoding

```
MOV { <Zd1>.D-<Zd4>.D }, ZA.D[<Wv>, <offs>{, VGx4}]
```

is equivalent to

```
MOVA { <Zd1>.D-<Zd4>.D }, ZA.D[<Wv>, <offs>{, VGx4}]
```

and is always the preferred disassembly.

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

Operation

The description of MOVA (array to vector, four registers) gives the operational pseudocode for this instruction.

Operational Information

The description of MOVA (array to vector, four registers) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (array to vector, two registers)

Move two ZA single-vector groups to Z two-vector operand

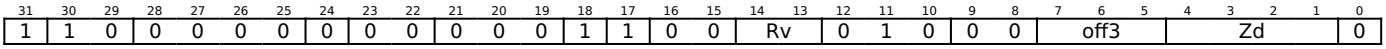
This instruction operates on two ZA single-vector groups.
The single-vector group within each half of the ZA array is selected by the sum of the vector select register and offset, modulo half the number of ZA array vectors.
The vector group symbol VGx2 indicates that the instruction operates on two ZA single-vector groups.
The preferred disassembly syntax uses a 64-bit element size, but an assembler should accept any element size if it is used consistently for all operands. The vector group symbol is preferred for disassembly, but optional in assembler source code.
This instruction is unpredicated.

This is an alias of MOVA (array to vector, two registers). This means:

- The encodings in this description are named to match the encodings of MOVA (array to vector, two registers).
- The description of MOVA (array to vector, two registers) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SME2

(FEAT_SME2)



Encoding

```
MOV { <Zd1>.D-<Zd2>.D }, ZA.D[<Wv>, <offs>{, VGx2}]
```

is equivalent to

```
MOVA { <Zd1>.D-<Zd2>.D }, ZA.D[<Wv>, <offs>{, VGx2}]
```

and is always the preferred disassembly.

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

Operation

The description of MOVA (array to vector, two registers) gives the operational pseudocode for this instruction.

Operational Information

The description of MOVA (array to vector, two registers) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (tile to vector, four registers)

Move ZA four-slice operand to Z four-vector operand

This instruction operates on a horizontal or vertical ZA four-slice operand within a ZA tile of the specified element size.
The first slice of the four-slice operand is selected by rounding down the sum of the slice index register and the immediate offset to the nearest lower multiple of 4, modulo the number of slices in the tile.
The immediate offset is a multiple of 4 in the range 0 to the number of elements in a 128-bit vector segment minus 4.
This instruction is unpredicated.

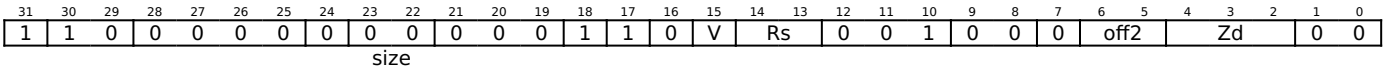
This is an alias of MOVA (tile to vector, four registers). This means:

- The encodings in this description are named to match the encodings of MOVA (tile to vector, four registers).
- The description of MOVA (tile to vector, four registers) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

It has encodings from 4 classes: 8-bit , 16-bit , 32-bit and 64-bit

8-bit

(FEAT_SME2)



Encoding

```
MOV { <Zd1>.B-<Zd4>.B }, ZA0<HV>.B[<Ws>, <offs1>:<offs4>]
```

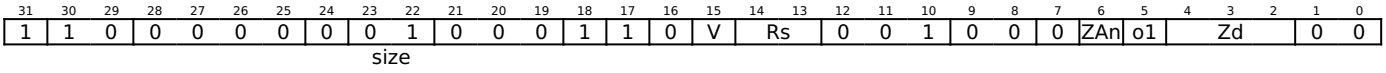
is equivalent to

```
MOVA { <Zd1>.B-<Zd4>.B }, ZA0<HV>.B[<Ws>, <offs1>:<offs4>]
```

and is always the preferred disassembly.

16-bit

(FEAT_SME2)



Encoding

```
MOV { <Zd1>.H-<Zd4>.H }, <ZAn><HV>.H[<Ws>, <offs1>:<offs4>]
```

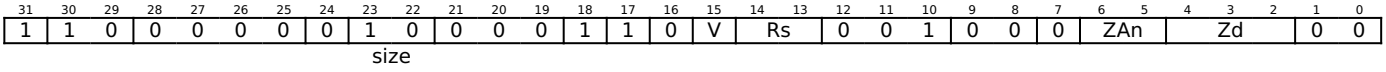
is equivalent to

```
MOVA { <Zd1>.H-<Zd4>.H }, <ZAn><HV>.H[<Ws>, <offs1>:<offs4>]
```

and is always the preferred disassembly.

32-bit

(FEAT_SME2)



Encoding

```
MOV { <Zd1>.S-<Zd4>.S }, <ZAn><HV>.S[<Ws>, <offs1>:<offs4>]
```

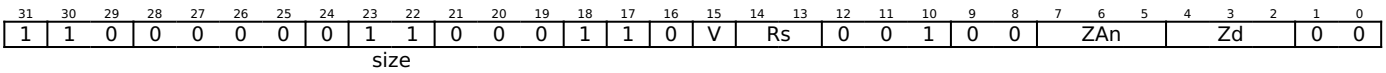
is equivalent to

```
MOVA { <Zd1>.S-<Zd4>.S }, <ZAn><HV>.S[<Ws>, <offs1>:<offs4>]
```

and is always the preferred disassembly.

64-bit

(FEAT_SME2)



Encoding

```
MOV { <Zd1>.D-<Zd4>.D }, <ZAn><HV>.D[<Ws>, <offs1>:<offs4>]
```

is equivalent to

```
MOVA { <Zd1>.D-<Zd4>.D }, <ZAn><HV>.D[<Ws>, <offs1>:<offs4>]
```

and is always the preferred disassembly.

Assembler Symbols

<Zd1> Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs1> For the "8-bit" variant: is the first slice index offset, encoded as "off2" field times 4.
For the "16-bit" variant: is the first slice index offset, encoded as "o1" field times 4.
For the "32-bit" and "64-bit" variants: is the first slice index offset, with implicit value 0.

<offs4> For the "8-bit" variant: is the fourth slice index offset, encoded as "off2" field times 4 plus 3.
For the "16-bit" variant: is the fourth slice index offset, encoded as "o1" field times 4 plus 3.
For the "32-bit" and "64-bit" variants: is the fourth slice index offset, with implicit value 3.

<ZAn> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAn" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAn" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAn" field.

Operation

The description of MOVA (tile to vector, four registers) gives the operational pseudocode for this instruction.

Operational Information

The description of MOVA (tile to vector, four registers) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (tile to vector, single)

Move ZA tile slice to Z vector

This instruction operates on individual horizontal or vertical slices within a named ZA tile of the specified element size. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of such elements in a vector. The immediate offset is in the range 0 to the number of elements in a 128-bit vector segment minus 1. Inactive elements in the destination vector remain unmodified.

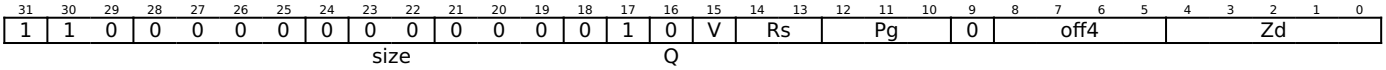
This is an alias of MOVA (tile to vector, single). This means:

- The encodings in this description are named to match the encodings of MOVA (tile to vector, single).
- The description of MOVA (tile to vector, single) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

It has encodings from 5 classes: 8-bit , 16-bit , 32-bit , 64-bit and 128-bit

8-bit

(FEAT_SME)



Encoding

```
MOV <Zd>.B, <Pg>/M, ZA0<HV>.B[<Ws>, <offs>]
```

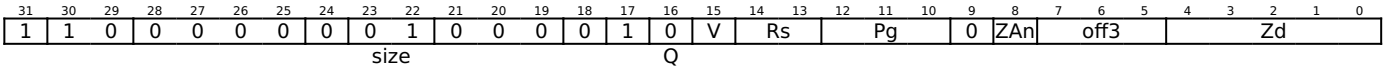
is equivalent to

```
MOVA <Zd>.B, <Pg>/M, ZA0<HV>.B[<Ws>, <offs>]
```

and is always the preferred disassembly.

16-bit

(FEAT_SME)



Encoding

```
MOV <Zd>.H, <Pg>/M, <ZAn><HV>.H[<Ws>, <offs>]
```

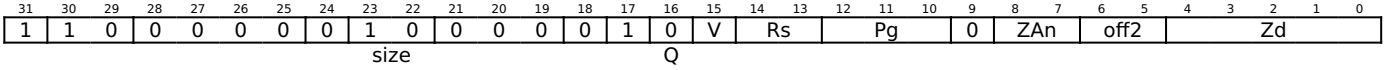
is equivalent to

```
MOVA <Zd>.H, <Pg>/M, <ZAn><HV>.H[<Ws>, <offs>]
```

and is always the preferred disassembly.

32-bit

(FEAT_SME)



Encoding

```
MOV <Zd>.S, <Pg>/M, <ZAn><HV>.S[<Ws>, <offs>]
```

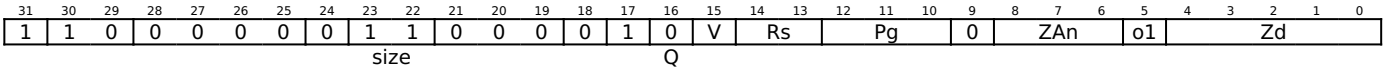
is equivalent to

```
MOVA <Zd>.S, <Pg>/M, <ZAn><HV>.S[<Ws>, <offs>]
```

and is always the preferred disassembly.

64-bit

(FEAT_SME)



Encoding

```
MOV <Zd>.D, <Pg>/M, <ZAn><HV>.D[<Ws>, <offs>]
```

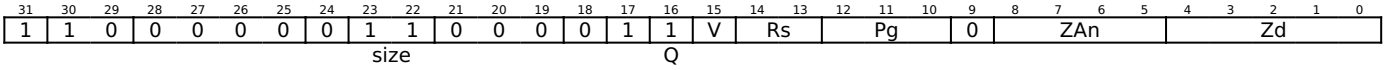
is equivalent to

```
MOVA <Zd>.D, <Pg>/M, <ZAn><HV>.D[<Ws>, <offs>]
```

and is always the preferred disassembly.

128-bit

(FEAT_SME)



Encoding

```
MOV <Zd>.Q, <Pg>/M, <ZAn><HV>.Q[<Ws>, <offs>]
```

is equivalent to

```
MOVA <Zd>.Q, <Pg>/M, <ZAn><HV>.Q[<Ws>, <offs>]
```

and is always the preferred disassembly.

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> For the "8-bit" variant: is the slice index offset, in the range 0 to 15, encoded in the "off4" field.
For the "16-bit" variant: is the slice index offset, in the range 0 to 7, encoded in the "off3" field.
For the "32-bit" variant: is the slice index offset, in the range 0 to 3, encoded in the "off2" field.
For the "64-bit" variant: is the slice index offset, in the range 0 to 1, encoded in the "o1" field.
For the "128-bit" variant: is the slice index offset, with implicit value 0.

<ZAn> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAn" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAn" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAn" field.
For the "128-bit" variant: is the name of the ZA tile ZA0-ZA15 to be accessed, encoded in the "ZAn" field.

Operation

The description of MOVA (tile to vector, single) gives the operational pseudocode for this instruction.

Operational Information

The description of MOVA (tile to vector, single) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (tile to vector, two registers)

Move ZA two-slice operand to Z two-vector operand

This instruction operates on a horizontal or vertical ZA two-slice operand within a ZA tile of the specified element size.
The first slice of the two-slice operand is selected by rounding down the sum of the slice index register and the immediate offset to the nearest lower multiple of 2, modulo the number of slices in the tile.
The immediate offset is a multiple of 2 in the range 0 to the number of elements in a 128-bit vector segment minus 2.
This instruction is unpredicated.

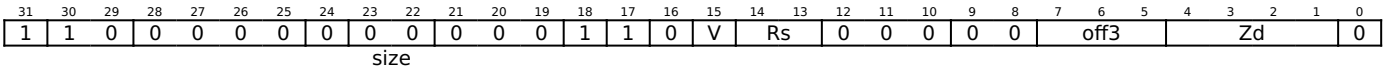
This is an alias of MOVA (tile to vector, two registers). This means:

- The encodings in this description are named to match the encodings of MOVA (tile to vector, two registers).
- The description of MOVA (tile to vector, two registers) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

It has encodings from 4 classes: 8-bit , 16-bit , 32-bit and 64-bit

8-bit

(FEAT_SME2)



Encoding

```
MOV { <Zd1>.B-<Zd2>.B }, ZA0<HV>.B[<Ws>, <offs1>:<offs2>]
```

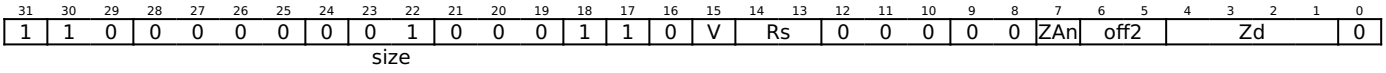
is equivalent to

```
MOVA { <Zd1>.B-<Zd2>.B }, ZA0<HV>.B[<Ws>, <offs1>:<offs2>]
```

and is always the preferred disassembly.

16-bit

(FEAT_SME2)



Encoding

```
MOV { <Zd1>.H-<Zd2>.H }, <ZAn><HV>.H[<Ws>, <offs1>:<offs2>]
```

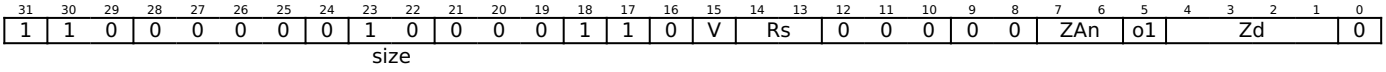
is equivalent to

```
MOVA { <Zd1>.H-<Zd2>.H }, <ZAn><HV>.H[<Ws>, <offs1>:<offs2>]
```

and is always the preferred disassembly.

32-bit

(FEAT_SME2)



Encoding

```
MOV { <Zd1>.S-<Zd2>.S }, <ZAn><HV>.S[<Ws>, <offs1>:<offs2>]
```

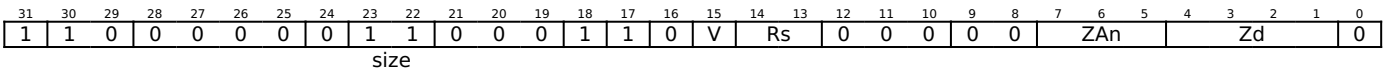
is equivalent to

```
MOVA { <Zd1>.S-<Zd2>.S }, <ZAn><HV>.S[<Ws>, <offs1>:<offs2>]
```

and is always the preferred disassembly.

64-bit

(FEAT_SME2)



Encoding

```
MOV { <Zd1>.D-<Zd2>.D }, <ZAn><HV>.D[<Ws>, <offs1>:<offs2>]
```

is equivalent to

```
MOVA { <Zd1>.D-<Zd2>.D }, <ZAn><HV>.D[<Ws>, <offs1>:<offs2>]
```

and is always the preferred disassembly.

Assembler Symbols

<Zd1> Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs1> For the "8-bit" variant: is the first slice index offset, encoded as "off3" field times 2.
For the "16-bit" variant: is the first slice index offset, encoded as "off2" field times 2.
For the "32-bit" variant: is the first slice index offset, encoded as "o1" field times 2.
For the "64-bit" variant: is the first slice index offset, with implicit value 0.

<offs2> For the "8-bit" variant: is the second slice index offset, encoded as "off3" field times 2 plus 1.
For the "16-bit" variant: is the second slice index offset, encoded as "off2" field times 2 plus 1.
For the "32-bit" variant: is the second slice index offset, encoded as "o1" field times 2 plus 1.
For the "64-bit" variant: is the second slice index offset, with implicit value 1.

<ZAn> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAn" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAn" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAn" field.

Operation

The description of MOVA (tile to vector, two registers) gives the operational pseudocode for this instruction.

Operational Information

The description of MOVA (tile to vector, two registers) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (vector to array, four registers)

Move Z four-vector operand to four ZA single-vector groups

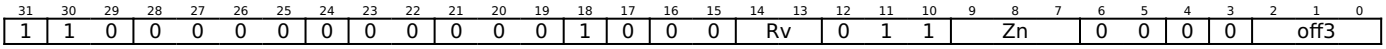
This instruction operates on four ZA single-vector groups.
The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.
The vector group symbol VGx4 indicates that the instruction operates on four ZA single-vector groups.
The preferred disassembly syntax uses a 64-bit element size, but an assembler should accept any element size if it is used consistently for all operands. The vector group symbol is preferred for disassembly, but optional in assembler source code.
This instruction is unpredicated.

This is an alias of MOVA (vector to array, four registers). This means:

- The encodings in this description are named to match the encodings of MOVA (vector to array, four registers).
- The description of MOVA (vector to array, four registers) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SME2

(FEAT_SME2)



Encoding

```
MOV ZA.D[<Wv>, <offs>{, VGx4}], { <Zn1>.D-<Zn4>.D }
```

is equivalent to

```
MOVA ZA.D[<Wv>, <offs>{, VGx4}], { <Zn1>.D-<Zn4>.D }
```

and is always the preferred disassembly.

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

The description of MOVA (vector to array, four registers) gives the operational pseudocode for this instruction.

Operational Information

The description of MOVA (vector to array, four registers) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (vector to array, two registers)

Move Z two-vector operand to two ZA single-vector groups

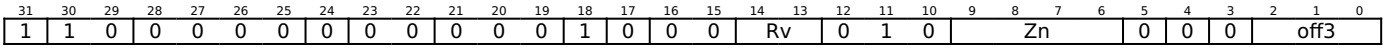
This instruction operates on two ZA single-vector groups.
The single-vector group within each half of the ZA array is selected by the sum of the vector select register and offset, modulo half the number of ZA array vectors.
The vector group symbol VGx2 indicates that the instruction operates on two ZA single-vector groups.
The preferred disassembly syntax uses a 64-bit element size, but an assembler should accept any element size if it is used consistently for all operands. The vector group symbol is preferred for disassembly, but optional in assembler source code.
This instruction is unpredicated.

This is an alias of MOVA (vector to array, two registers). This means:

- The encodings in this description are named to match the encodings of MOVA (vector to array, two registers).
- The description of MOVA (vector to array, two registers) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

SME2

(FEAT_SME2)



Encoding

```
MOV ZA.D[<Wv>, <offs>{, VGx2}], { <Zn1>.D-<Zn2>.D }
```

is equivalent to

```
MOVA ZA.D[<Wv>, <offs>{, VGx2}], { <Zn1>.D-<Zn2>.D }
```

and is always the preferred disassembly.

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

The description of MOVA (vector to array, two registers) gives the operational pseudocode for this instruction.

Operational Information

The description of MOVA (vector to array, two registers) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (vector to tile, four registers)

Move Z four-vector operand to ZA four-slice operand

This instruction operates on a horizontal or vertical ZA four-slice operand within a ZA tile of the specified element size.
The first slice of the four-slice operand is selected by rounding down the sum of the slice index register and the immediate offset to the nearest lower multiple of 4, modulo the number of slices in the tile.
The immediate offset is a multiple of 4 in the range 0 to the number of elements in a 128-bit vector segment minus 4.
This instruction is unpredicated.

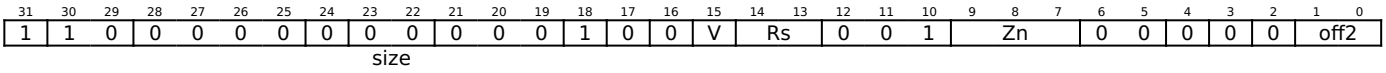
This is an alias of MOVA (vector to tile, four registers). This means:

- The encodings in this description are named to match the encodings of MOVA (vector to tile, four registers).
- The description of MOVA (vector to tile, four registers) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

It has encodings from 4 classes: 8-bit , 16-bit , 32-bit and 64-bit

8-bit

(FEAT_SME2)



Encoding

```
MOV ZA0<HV>.B[<Ws>, <offs1>:<offs4>], { <Zn1>.B-<Zn4>.B }
```

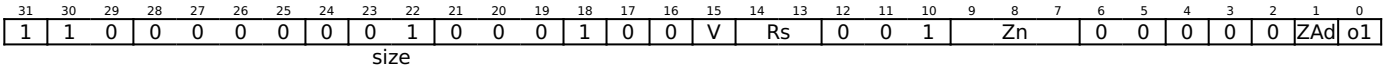
is equivalent to

```
MOVA ZA0<HV>.B[<Ws>, <offs1>:<offs4>], { <Zn1>.B-<Zn4>.B }
```

and is always the preferred disassembly.

16-bit

(FEAT_SME2)



Encoding

```
MOV <ZAd><HV>.H[<Ws>, <offs1>:<offs4>], { <Zn1>.H-<Zn4>.H }
```

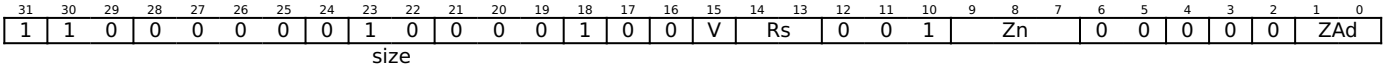
is equivalent to

```
MOVA <ZAd><HV>.H[<Ws>, <offs1>:<offs4>], { <Zn1>.H-<Zn4>.H }
```

and is always the preferred disassembly.

32-bit

(FEAT_SME2)



Encoding

```
MOV <ZAd><HV>.S[<Ws>, <offs1>:<offs4>], { <Zn1>.S-<Zn4>.S }
```

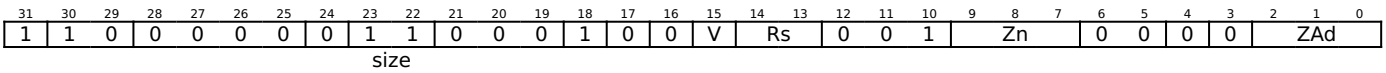
is equivalent to

```
MOVA <ZAd><HV>.S[<Ws>, <offs1>:<offs4>], { <Zn1>.S-<Zn4>.S }
```

and is always the preferred disassembly.

64-bit

(FEAT_SME2)



Encoding

```
MOV <ZAd><HV>.D[<Ws>, <offs1>:<offs4>], { <Zn1>.D-<Zn4>.D }
```

is equivalent to

```
MOVA <ZAd><HV>.D[<Ws>, <offs1>:<offs4>], { <Zn1>.D-<Zn4>.D }
```

and is always the preferred disassembly.

Assembler Symbols

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs1> For the "8-bit" variant: is the first slice index offset, encoded as "off2" field times 4.
For the "16-bit" variant: is the first slice index offset, encoded as "o1" field times 4.
For the "32-bit" and "64-bit" variants: is the first slice index offset, with implicit value 0.

<offs4> For the "8-bit" variant: is the fourth slice index offset, encoded as "off2" field times 4 plus 3.
For the "16-bit" variant: is the fourth slice index offset, encoded as "o1" field times 4 plus 3.
For the "32-bit" and "64-bit" variants: is the fourth slice index offset, with implicit value 3.

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

<ZAd> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAd" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAd" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAd" field.

Operation

The description of MOVA (vector to tile, four registers) gives the operational pseudocode for this instruction.

Operational Information

The description of MOVA (vector to tile, four registers) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (vector to tile, single)

Move Z vector to ZA tile slice

This instruction operates on individual horizontal or vertical slices within a named ZA tile of the specified element size. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of such elements in a vector. The immediate offset is in the range 0 to the number of elements in a 128-bit vector segment minus 1. Inactive elements in the destination slice remain unmodified.

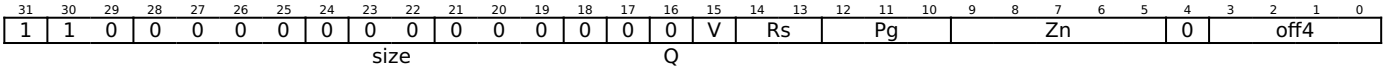
This is an alias of MOVA (vector to tile, single). This means:

- The encodings in this description are named to match the encodings of MOVA (vector to tile, single).
- The description of MOVA (vector to tile, single) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

It has encodings from 5 classes: 8-bit , 16-bit , 32-bit , 64-bit and 128-bit

8-bit

(FEAT_SME)



Encoding

```
MOV ZA0<HV>.B[<Ws>, <offs>], <Pg>/M, <Zn>.B
```

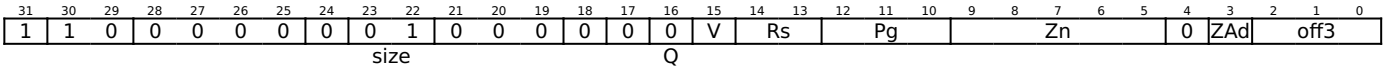
is equivalent to

```
MOVA ZA0<HV>.B[<Ws>, <offs>], <Pg>/M, <Zn>.B
```

and is always the preferred disassembly.

16-bit

(FEAT_SME)



Encoding

```
MOV <ZAd><HV>.H[<Ws>, <offs>], <Pg>/M, <Zn>.H
```

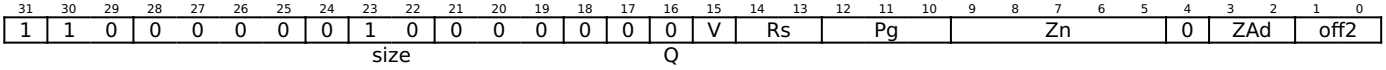
is equivalent to

```
MOVA <ZAd><HV>.H[<Ws>, <offs>], <Pg>/M, <Zn>.H
```

and is always the preferred disassembly.

32-bit

(FEAT_SME)



Encoding

```
MOV <ZAd><HV>.S[<Ws>, <offs>], <Pg>/M, <Zn>.S
```

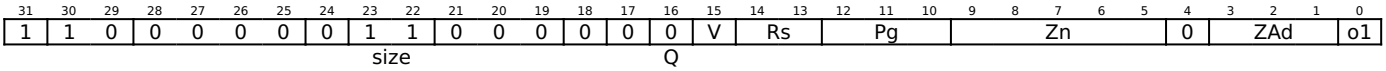
is equivalent to

```
MOVA <ZAd><HV>.S[<Ws>, <offs>], <Pg>/M, <Zn>.S
```

and is always the preferred disassembly.

64-bit

(FEAT_SME)



Encoding

```
MOV <ZAd><HV>.D[<Ws>, <offs>], <Pg>/M, <Zn>.D
```

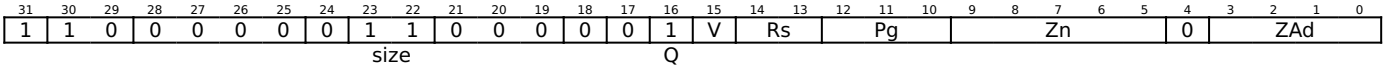
is equivalent to

```
MOVA <ZAd><HV>.D[<Ws>, <offs>], <Pg>/M, <Zn>.D
```

and is always the preferred disassembly.

128-bit

(FEAT_SME)



Encoding

```
MOV <ZAd><HV>.Q[<Ws>, <offs>], <Pg>/M, <Zn>.Q
```

is equivalent to

```
MOVA <ZAd><HV>.Q[<Ws>, <offs>], <Pg>/M, <Zn>.Q
```

and is always the preferred disassembly.

Assembler Symbols

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> For the "8-bit" variant: is the slice index offset, in the range 0 to 15, encoded in the "off4" field.
For the "16-bit" variant: is the slice index offset, in the range 0 to 7, encoded in the "off3" field.
For the "32-bit" variant: is the slice index offset, in the range 0 to 3, encoded in the "off2" field.
For the "64-bit" variant: is the slice index offset, in the range 0 to 1, encoded in the "o1" field.
For the "128-bit" variant: is the slice index offset, with implicit value 0.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<ZAd> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAd" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAd" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAd" field.
For the "128-bit" variant: is the name of the ZA tile ZA0-ZA15 to be accessed, encoded in the "ZAd" field.

Operation

The description of MOVA (vector to tile, single) gives the operational pseudocode for this instruction.

Operational Information

The description of MOVA (vector to tile, single) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOV (vector to tile, two registers)

Move Z two-vector operand to ZA two-slice operand

This instruction operates on a horizontal or vertical ZA two-slice operand within a ZA tile of the specified element size.
The first slice of the two-slice operand is selected by rounding down the sum of the slice index register and the immediate offset to the nearest lower multiple of 2, modulo the number of slices in the tile.
The immediate offset is a multiple of 2 in the range 0 to the number of elements in a 128-bit vector segment minus 2.
This instruction is unpredicated.

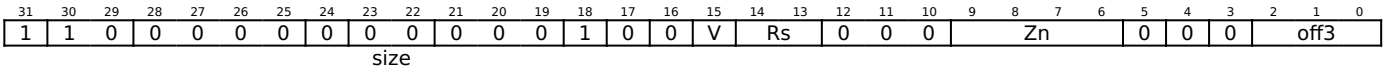
This is an alias of MOVA (vector to tile, two registers). This means:

- The encodings in this description are named to match the encodings of MOVA (vector to tile, two registers).
- The description of MOVA (vector to tile, two registers) gives the operational pseudocode, any CONSTRAINED UNPREDICTABLE behavior, and any operational information for this instruction.

It has encodings from 4 classes: 8-bit , 16-bit , 32-bit and 64-bit

8-bit

(FEAT_SME2)



Encoding

```
MOV ZA0<HV>.B[<Ws>, <offs1>:<offs2>], { <Zn1>.B-<Zn2>.B }
```

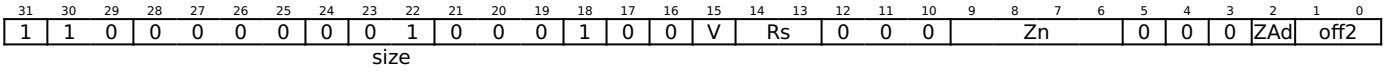
is equivalent to

```
MOVA ZA0<HV>.B[<Ws>, <offs1>:<offs2>], { <Zn1>.B-<Zn2>.B }
```

and is always the preferred disassembly.

16-bit

(FEAT_SME2)



Encoding

```
MOV <ZAd><HV>.H[<Ws>, <offs1>:<offs2>], { <Zn1>.H-<Zn2>.H }
```

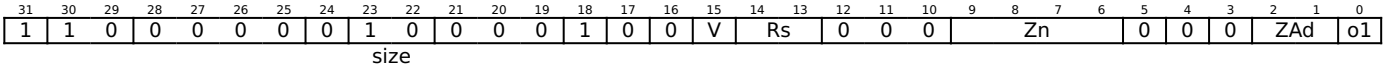
is equivalent to

```
MOVA <ZAd><HV>.H[<Ws>, <offs1>:<offs2>], { <Zn1>.H-<Zn2>.H }
```

and is always the preferred disassembly.

32-bit

(FEAT_SME2)



Encoding

```
MOV <ZAd><HV>.S[<Ws>, <offs1>:<offs2>], { <Zn1>.S-<Zn2>.S }
```

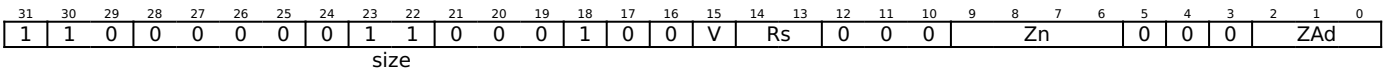
is equivalent to

```
MOVA <ZAd><HV>.S[<Ws>, <offs1>:<offs2>], { <Zn1>.S-<Zn2>.S }
```

and is always the preferred disassembly.

64-bit

(FEAT_SME2)



Encoding

```
MOV <ZAd><HV>.D[<Ws>, <offs1>:<offs2>], { <Zn1>.D-<Zn2>.D }
```

is equivalent to

```
MOVA <ZAd><HV>.D[<Ws>, <offs1>:<offs2>], { <Zn1>.D-<Zn2>.D }
```

and is always the preferred disassembly.

Assembler Symbols

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs1> For the "8-bit" variant: is the first slice index offset, encoded as "off3" field times 2.
For the "16-bit" variant: is the first slice index offset, encoded as "off2" field times 2.
For the "32-bit" variant: is the first slice index offset, encoded as "o1" field times 2.
For the "64-bit" variant: is the first slice index offset, with implicit value 0.

<offs2> For the "8-bit" variant: is the second slice index offset, encoded as "off3" field times 2 plus 1.
For the "16-bit" variant: is the second slice index offset, encoded as "off2" field times 2 plus 1.
For the "32-bit" variant: is the second slice index offset, encoded as "o1" field times 2 plus 1.
For the "64-bit" variant: is the second slice index offset, with implicit value 1.

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.

<Zn2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

<ZAd> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAd" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAd" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAd" field.

Operation

The description of MOVA (vector to tile, two registers) gives the operational pseudocode for this instruction.

Operational Information

The description of MOVA (vector to tile, two registers) gives the operational information for this instruction.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

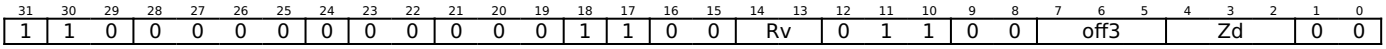
MOVA (array to vector, four registers)

Move four ZA single-vector groups to Z four-vector operand

This instruction operates on four ZA single-vector groups.
The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.
The vector group symbol VGx4 indicates that the instruction operates on four ZA single-vector groups.
The preferred disassembly syntax uses a 64-bit element size, but an assembler should accept any element size if it is used consistently for all operands. The vector group symbol is preferred for disassembly, but optional in assembler source code.
This instruction is unpredicated.
This instruction is used by the alias MOV (array to vector, four registers).

SME2

(FEAT_SME2)



Encoding

```
MOVA { <Zd1>.D-<Zd4>.D }, ZA.D[<Wv>, <offs>{, VGx4}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off3);
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
for r = 0 to nreg-1 do
    let result : bits(VL) = ZAvector{VL}(vec);
    Z{VL}(d + r) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

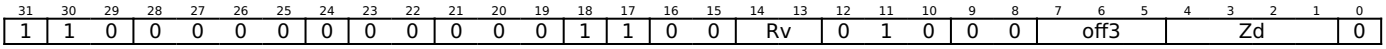
MOVA (array to vector, two registers)

Move two ZA single-vector groups to Z two-vector operand

This instruction operates on two ZA single-vector groups.
The single-vector group within each half of the ZA array is selected by the sum of the vector select register and offset, modulo half the number of ZA array vectors.
The vector group symbol VGx2 indicates that the instruction operates on two ZA single-vector groups.
The preferred disassembly syntax uses a 64-bit element size, but an assembler should accept any element size if it is used consistently for all operands. The vector group symbol is preferred for disassembly, but optional in assembler source code.
This instruction is unpredicated.
This instruction is used by the alias MOV (array to vector, two registers).

SME2

(FEAT_SME2)



Encoding

```
MOVA { <Zd1>.D-<Zd2>.D }, ZA.D[<Wv>, <offs>{, VGx2}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off3);
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
for r = 0 to nreg-1 do
    let result : bits(VL) = ZAvector{VL}(vec);
    Z{VL}(d + r) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

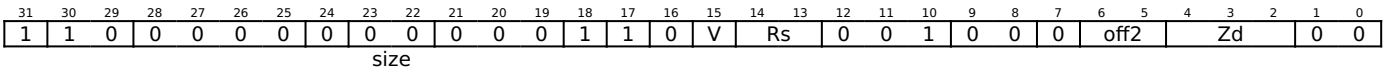
MOVA (tile to vector, four registers)

Move ZA four-slice operand to Z four-vector operand

This instruction operates on a horizontal or vertical ZA four-slice operand within a ZA tile of the specified element size.
The first slice of the four-slice operand is selected by rounding down the sum of the slice index register and the immediate offset to the nearest lower multiple of 4, modulo the number of slices in the tile.
The immediate offset is a multiple of 4 in the range 0 to the number of elements in a 128-bit vector segment minus 4.
This instruction is unpredicated.
This instruction is used by the alias MOV (tile to vector, four registers).
It has encodings from 4 classes: 8-bit , 16-bit , 32-bit and 64-bit

8-bit

(FEAT_SME2)



Encoding

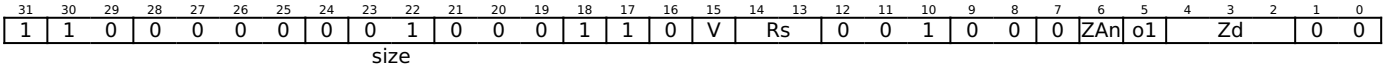
```
MOVA { <Zd1>.B-<Zd4>.B }, ZA0<HV>.B[<Ws>, <offs1>:<offs4>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 8;
let d : integer = UInt(Zd::'00');
let n : integer = 0;
let offset : integer = UInt(off2::'00');
let vertical : boolean = V == '1';
```

16-bit

(FEAT_SME2)



Encoding

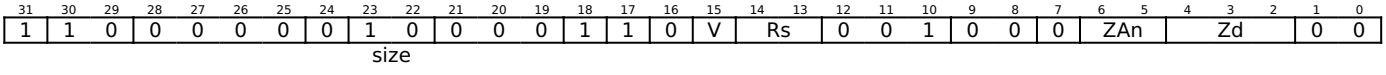
```
MOVA { <Zd1>.H-<Zd4>.H }, <ZAn><HV>.H[<Ws>, <offs1>:<offs4>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 16;
let d : integer = UInt(Zd::'00');
let n : integer = UInt(ZAn);
let offset : integer = UInt(o1::'00');
let vertical : boolean = V == '1';
```

32-bit

(FEAT_SME2)



Encoding

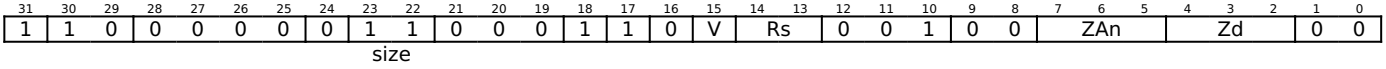
```
MOVA { <Zd1>.S-<Zd4>.S }, <ZAn><HV>.S[<Ws>, <offs1>:<offs4>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let d : integer = UInt(Zd::'00');
let n : integer = UInt(ZAn);
let offset : integer = 0;
let vertical : boolean = V == '1';
```

64-bit

(FEAT_SME2)



Encoding

```
MOVA { <Zd1>.D-<Zd4>.D }, <ZAn><HV>.D[<Ws>, <offs1>:<offs4>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 256 then
    EndOfDecode(Decode_UNDEF);
end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 64;
let d : integer = UInt(Zd::'00');
let n : integer = UInt(ZAn);
let offset : integer = 0;
let vertical : boolean = V == '1';
```

Assembler Symbols

<Zd1> Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs1> For the "8-bit" variant: is the first slice index offset, encoded as "off2" field times 4.
For the "16-bit" variant: is the first slice index offset, encoded as "o1" field times 4.
For the "32-bit" and "64-bit" variants: is the first slice index offset, with implicit value 0.

<offs4> For the "8-bit" variant: is the fourth slice index offset, encoded as "off2" field times 4 plus 3.
For the "16-bit" variant: is the fourth slice index offset, encoded as "o1" field times 4 plus 3.
For the "32-bit" and "64-bit" variants: is the fourth slice index offset, with implicit value 3.

<ZAn> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAn" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAn" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAn" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
if nreg == 4 && esize == 64 && VL < 256 then EndOfDecode(Decode_UNDEF); end;
let slices : integer = VL DIV esize;
let index : bits(32) = X{32}(s);
let slice : integer = ((UInt(index) - UInt(index) MOD nreg) + offset) MOD slices;
for r = 0 to nreg-1 do
    let result : bits(VL) = ZAslice{VL}(n, esize, vertical, slice + r);
    Z{VL}(d + r) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVA (tile to vector, single)

Move ZA tile slice to Z vector

This instruction operates on individual horizontal or vertical slices within a named ZA tile of the specified element size. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of such elements in a vector. The immediate offset is in the range 0 to the number of elements in a 128-bit vector segment minus 1.

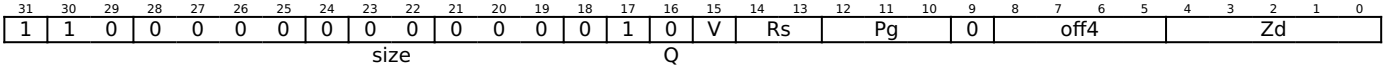
Inactive elements in the destination vector remain unmodified.

This instruction is used by the alias MOV (tile to vector, single).

It has encodings from 5 classes: 8-bit , 16-bit , 32-bit , 64-bit and 128-bit

8-bit

(FEAT_SME)



Encoding

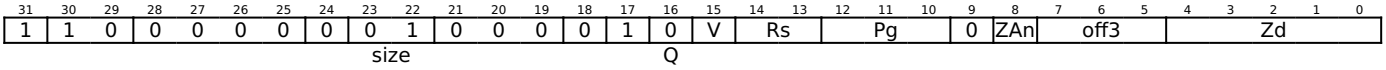
```
MOVA <Zd>.B, <Pg>/M, ZA0<HV>.B[<Ws>, <offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let s : integer = UInt('011'::Rs);
let n : integer = 0;
let offset : integer = UInt(off4);
let esize : integer{} = 8;
let d : integer = UInt(Zd);
let vertical : boolean = V == '1';
```

16-bit

(FEAT_SME)



Encoding

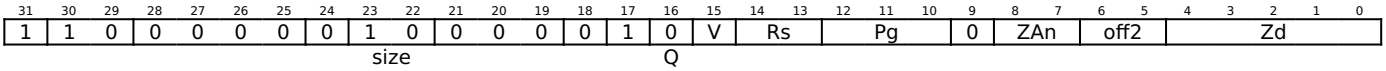
```
MOVA <Zd>.H, <Pg>/M, <ZAn><HV>.H[<Ws>, <offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let s : integer = UInt('011'::Rs);
let n : integer = UInt(ZAn);
let offset : integer = UInt(off3);
let esize : integer{} = 16;
let d : integer = UInt(Zd);
let vertical : boolean = V == '1';
```

32-bit

(FEAT_SME)



Encoding

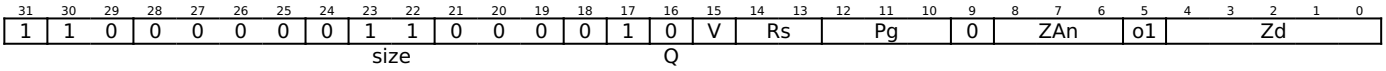
```
MOVA <Zd>.S, <Pg>/M, <ZAn><HV>.S[<Ws>, <offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let s : integer = UInt('011'::Rs);
let n : integer = UInt(ZAn);
let offset : integer = UInt(off2);
let esize : integer{} = 32;
let d : integer = UInt(Zd);
let vertical : boolean = V == '1';
```

64-bit

(FEAT_SME)



Encoding

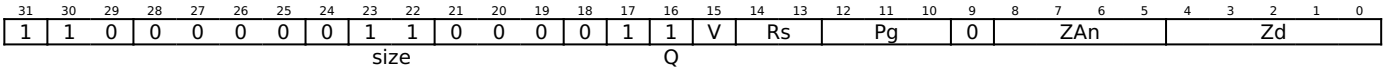
```
MOVA <Zd>.D, <Pg>/M, <ZAn><HV>.D[<Ws>, <offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let s : integer = UInt('011'::Rs);
let n : integer = UInt(ZAn);
let offset : integer = UInt(o1);
let esize : integer{} = 64;
let d : integer = UInt(Zd);
let vertical : boolean = V == '1';
```

128-bit

(FEAT_SME)



Encoding

```
MOVA <Zd>.Q, <Pg>/M, <ZAn><HV>.Q[<Ws>, <offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let s : integer = UInt('011'::Rs);
let n : integer = UInt(ZAn);
let offset : integer = 0;
let esize : integer{} = 128;
let d : integer = UInt(Zd);
let vertical : boolean = V == '1';
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> For the "8-bit" variant: is the slice index offset, in the range 0 to 15, encoded in the "off4" field.
For the "16-bit" variant: is the slice index offset, in the range 0 to 7, encoded in the "off3" field.
For the "32-bit" variant: is the slice index offset, in the range 0 to 3, encoded in the "off2" field.
For the "64-bit" variant: is the slice index offset, in the range 0 to 1, encoded in the "o1" field.
For the "128-bit" variant: is the slice index offset, with implicit value 0.

<ZAn> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAn" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAn" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAn" field.
For the "128-bit" variant: is the name of the ZA tile ZA0-ZA15 to be accessed, encoded in the "ZAn" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
let mask : bits(PL) = P{PL}(g);
let index : bits(32) = X{32}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
let operand : bits(VL) = ZAslice{VL}(n, esize, vertical, slice);
var result : bits(VL) = Z{VL}(d);

for e = 0 to dim-1 do
    let element : bits(esize) = operand[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = element;
    end;
end;
Z{VL}(d) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

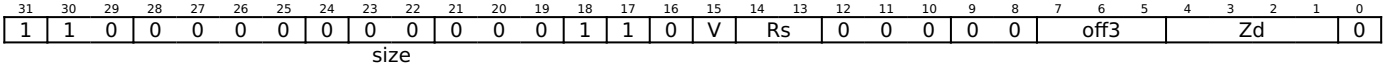
MOVA (tile to vector, two registers)

Move ZA two-slice operand to Z two-vector operand

This instruction operates on a horizontal or vertical ZA two-slice operand within a ZA tile of the specified element size.
The first slice of the two-slice operand is selected by rounding down the sum of the slice index register and the immediate offset to the nearest lower multiple of 2, modulo the number of slices in the tile.
The immediate offset is a multiple of 2 in the range 0 to the number of elements in a 128-bit vector segment minus 2.
This instruction is unpredicated.
This instruction is used by the alias MOV (tile to vector, two registers).
It has encodings from 4 classes: 8-bit , 16-bit , 32-bit and 64-bit

8-bit

(FEAT_SME2)



Encoding

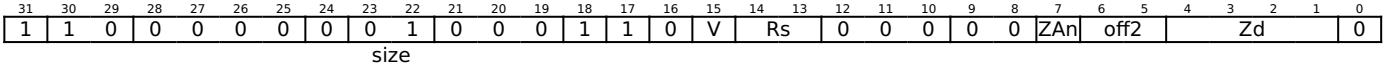
```
MOVA { <Zd1>.B-<Zd2>.B }, ZA0<HV>.B[<Ws>, <offs1>:<offs2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 8;
let d : integer = UInt(Zd::'0');
let n : integer = 0;
let offset : integer = UInt(off3::'0');
let vertical : boolean = V == '1';
```

16-bit

(FEAT_SME2)



Encoding

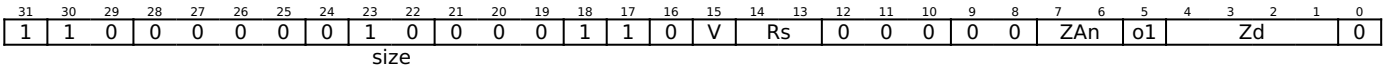
```
MOVA { <Zd1>.H-<Zd2>.H }, <ZAn><HV>.H[<Ws>, <offs1>:<offs2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 16;
let d : integer = UInt(Zd::'0');
let n : integer = UInt(ZAn);
let offset : integer = UInt(off2::'0');
let vertical : boolean = V == '1';
```

32-bit

(FEAT_SME2)



Encoding

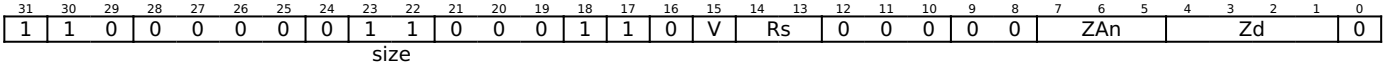
```
MOVA { <Zd1>.S-<Zd2>.S }, <ZAn><HV>.S[<Ws>, <offs1>:<offs2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let d : integer = UInt(Zd::'0');
let n : integer = UInt(ZAn);
let offset : integer = UInt(o1::'0');
let vertical : boolean = V == '1';
```

64-bit

(FEAT_SME2)



Encoding

```
MOVA { <Zd1>.D-<Zd2>.D }, <ZAn><HV>.D[<Ws>, <offs1>:<offs2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 64;
let d : integer = UInt(Zd::'0');
let n : integer = UInt(ZAn);
let offset : integer = 0;
let vertical : boolean = V == '1';
```

Assembler Symbols

<Zd1> Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs1> For the "8-bit" variant: is the first slice index offset, encoded as "off3" field times 2.
For the "16-bit" variant: is the first slice index offset, encoded as "off2" field times 2.
For the "32-bit" variant: is the first slice index offset, encoded as "o1" field times 2.
For the "64-bit" variant: is the first slice index offset, with implicit value 0.

<offs2> For the "8-bit" variant: is the second slice index offset, encoded as "off3" field times 2 plus 1.
For the "16-bit" variant: is the second slice index offset, encoded as "off2" field times 2 plus 1.
For the "32-bit" variant: is the second slice index offset, encoded as "o1" field times 2 plus 1.
For the "64-bit" variant: is the second slice index offset, with implicit value 1.

<ZAn> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAn" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAn" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAn" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
if nreg == 4 && esize == 64 && VL < 256 then EndOfDecode(Decode_UNDEF); end;
let slices : integer = VL DIV esize;
let index : bits(32) = X{32}(s);
let slice : integer = ((UInt(index) - UInt(index) MOD nreg) + offset) MOD slices;
for r = 0 to nreg-1 do
    let result : bits(VL) = ZAslice{VL}(n, esize, vertical, slice + r);
    Z{VL}(d + r) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

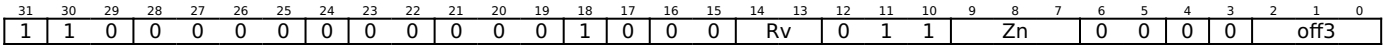
MOVA (vector to array, four registers)

Move Z four-vector operand to four ZA single-vector groups

This instruction operates on four ZA single-vector groups.
The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.
The vector group symbol VGx4 indicates that the instruction operates on four ZA single-vector groups.
The preferred disassembly syntax uses a 64-bit element size, but an assembler should accept any element size if it is used consistently for all operands. The vector group symbol is preferred for disassembly, but optional in assembler source code.
This instruction is unpredicated.
This instruction is used by the alias MOV (vector to array, four registers).

SME2

(FEAT_SME2)



Encoding

```
MOVA ZA.D[<Wv>, <offs>{, VGx4}], { <Zn1>.D-<Zn4>.D }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off3);
let n : integer = UInt(Zn::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
for r = 0 to nreg-1 do
    let result : bits(VL) = Z{VL}(n + r);
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

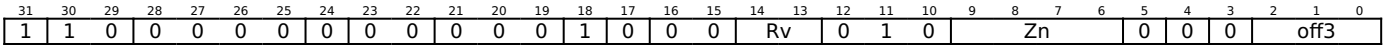
MOVA (vector to array, two registers)

Move Z two-vector operand to two ZA single-vector groups

This instruction operates on two ZA single-vector groups.
The single-vector group within each half of the ZA array is selected by the sum of the vector select register and offset, modulo half the number of ZA array vectors.
The vector group symbol VGx2 indicates that the instruction operates on two ZA single-vector groups.
The preferred disassembly syntax uses a 64-bit element size, but an assembler should accept any element size if it is used consistently for all operands. The vector group symbol is preferred for disassembly, but optional in assembler source code.
This instruction is unpredicated.
This instruction is used by the alias MOV (vector to array, two registers).

SME2

(FEAT_SME2)



Encoding

```
MOVA ZA.D[<Wv>, <offs>{, VGx2}], { <Zn1>.D-<Zn2>.D }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off3);
let n : integer = UInt(Zn::'0');
let nreg : integer{} = 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
for r = 0 to nreg-1 do
    let result : bits(VL) = Z{VL}(n + r);
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

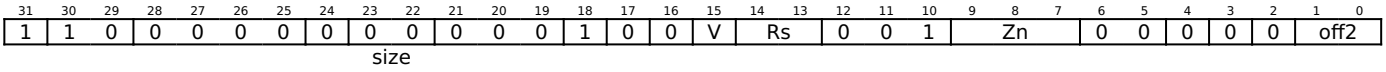
MOVA (vector to tile, four registers)

Move Z four-vector operand to ZA four-slice operand

This instruction operates on a horizontal or vertical ZA four-slice operand within a ZA tile of the specified element size.
The first slice of the four-slice operand is selected by rounding down the sum of the slice index register and the immediate offset to the nearest lower multiple of 4, modulo the number of slices in the tile.
The immediate offset is a multiple of 4 in the range 0 to the number of elements in a 128-bit vector segment minus 4.
This instruction is unpredicated.
This instruction is used by the alias MOV (vector to tile, four registers).
It has encodings from 4 classes: 8-bit , 16-bit , 32-bit and 64-bit

8-bit

(FEAT_SME2)



Encoding

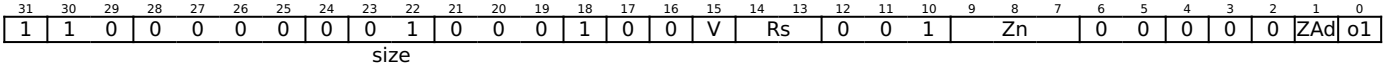
```
MOVA ZA0<HV>.B[<Ws>, <offs1>:<offs4>], { <Zn1>.B-<Zn4>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 8;
let n : integer = UInt(Zn::'00');
let d : integer = 0;
let offset : integer = UInt(off2::'00');
let vertical : boolean = V == '1';
```

16-bit

(FEAT_SME2)



Encoding

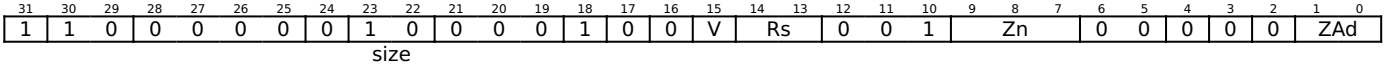
```
MOVA <ZAd><HV>.H[<Ws>, <offs1>:<offs4>], { <Zn1>.H-<Zn4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(ZAd);
let offset : integer = UInt(o1::'00');
let vertical : boolean = V == '1';
```

32-bit

(FEAT_SME2)



Encoding

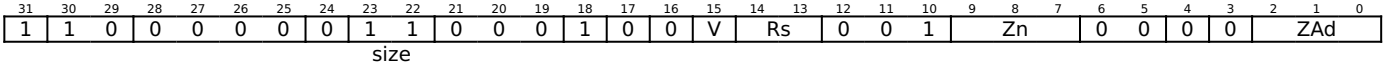
```
MOVA <ZAd><HV>.S[<Ws>, <offs1>:<offs4>], { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(ZAd);
let offset : integer = 0;
let vertical : boolean = V == '1';
```

64-bit

(FEAT_SME2)



Encoding

```
MOVA <ZAd><HV>.D[<Ws>, <offs1>:<offs4>], { <Zn1>.D-<Zn4>.D }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 256 then
    EndOfDecode(Decode_UNDEF);
end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 64;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(ZAd);
let offset : integer = 0;
let vertical : boolean = V == '1';
```

Assembler Symbols

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs1> For the "8-bit" variant: is the first slice index offset, encoded as "off2" field times 4.
For the "16-bit" variant: is the first slice index offset, encoded as "o1" field times 4.
For the "32-bit" and "64-bit" variants: is the first slice index offset, with implicit value 0.

<offs4> For the "8-bit" variant: is the fourth slice index offset, encoded as "off2" field times 4 plus 3.
For the "16-bit" variant: is the fourth slice index offset, encoded as "o1" field times 4 plus 3.
For the "32-bit" and "64-bit" variants: is the fourth slice index offset, with implicit value 3.

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

<ZAd> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAd" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAd" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAd" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
if nreg == 4 && esize == 64 && VL < 256 then EndOfDecode(Decode_UNDEF); end;
let slices : integer = VL DIV esize;
let index : bits(32) = X{32}(s);
let slice : integer = ((UInt(index) - UInt(index) MOD nreg) + offset) MOD slices;
for r = 0 to nreg-1 do
    let result : bits(VL) = Z{VL}(n + r);
    ZAslice{VL}(d, esize, vertical, slice + r) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVA (vector to tile, single)

Move Z vector to ZA tile slice

This instruction operates on individual horizontal or vertical slices within a named ZA tile of the specified element size. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of such elements in a vector. The immediate offset is in the range 0 to the number of elements in a 128-bit vector segment minus 1.

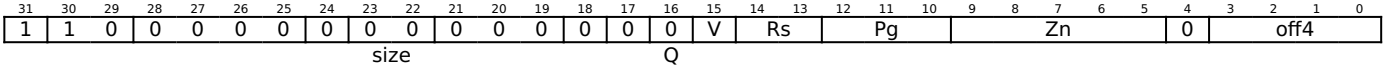
Inactive elements in the destination slice remain unmodified.

This instruction is used by the alias MOV (vector to tile, single).

It has encodings from 5 classes: 8-bit , 16-bit , 32-bit , 64-bit and 128-bit

8-bit

(FEAT_SME)



Encoding

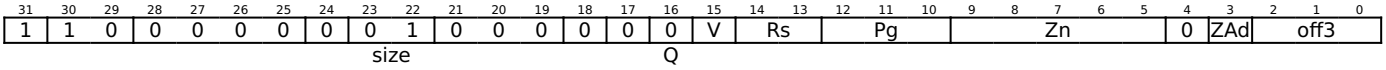
```
MOVA ZA0<HV>.B[<Ws>, <offs>], <Pg>/M, <Zn>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let s : integer = UInt('011'::Rs);
let n : integer = UInt(Zn);
let d : integer = 0;
let offset : integer = UInt(off4);
let esize : integer{} = 8;
let vertical : boolean = V == '1';
```

16-bit

(FEAT_SME)



Encoding

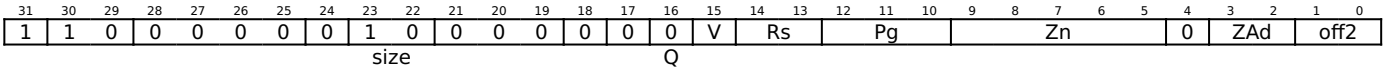
```
MOVA <ZAd><HV>.H[<Ws>, <offs>], <Pg>/M, <Zn>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let s : integer = UInt('011'::Rs);
let n : integer = UInt(Zn);
let d : integer = UInt(ZAd);
let offset : integer = UInt(off3);
let esize : integer{} = 16;
let vertical : boolean = V == '1';
```

32-bit

(FEAT_SME)



Encoding

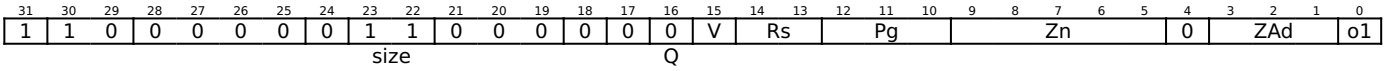
```
MOVA <ZAd><HV>.S[<Ws>, <offs>], <Pg>/M, <Zn>.S
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let s : integer = UInt('011'::Rs);
let n : integer = UInt(Zn);
let d : integer = UInt(ZAd);
let offset : integer = UInt(off2);
let esize : integer{} = 32;
let vertical : boolean = V == '1';
```

64-bit

(FEAT_SME)



Encoding

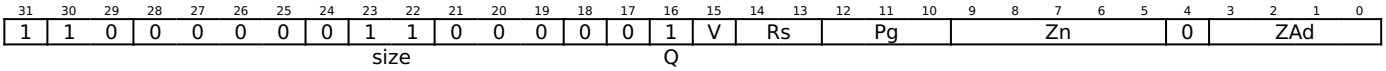
```
MOVA <ZAd><HV>.D[<Ws>, <offs>], <Pg>/M, <Zn>.D
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let s : integer = UInt('011'::Rs);
let n : integer = UInt(Zn);
let d : integer = UInt(ZAd);
let offset : integer = UInt(o1);
let esize : integer{} = 64;
let vertical : boolean = V == '1';
```

128-bit

(FEAT_SME)



Encoding

```
MOVA <ZAd><HV>.Q[<Ws>, <offs>], <Pg>/M, <Zn>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let g : integer = UInt(Pg);
let s : integer = UInt('011'::Rs);
let n : integer = UInt(Zn);
let d : integer = UInt(ZAd);
let offset : integer = 0;
let esize : integer{} = 128;
let vertical : boolean = V == '1';
```

Assembler Symbols

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> For the "8-bit" variant: is the slice index offset, in the range 0 to 15, encoded in the "off4" field.
For the "16-bit" variant: is the slice index offset, in the range 0 to 7, encoded in the "off3" field.
For the "32-bit" variant: is the slice index offset, in the range 0 to 3, encoded in the "off2" field.
For the "64-bit" variant: is the slice index offset, in the range 0 to 1, encoded in the "o1" field.
For the "128-bit" variant: is the slice index offset, with implicit value 0.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<ZAd> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAd" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAd" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAd" field.
For the "128-bit" variant: is the name of the ZA tile ZA0-ZA15 to be accessed, encoded in the "ZAd" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
let mask : bits(PL) = P{PL}(g);
let operand : bits(VL) = Z{VL}(n);
let index : bits(32) = X{32}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var result : bits(VL) = ZAslice{VL}(d, esize, vertical, slice);

for e = 0 to dim-1 do
    let element : bits(esize) = operand[e*:esize];
    if ActivePredicateElement{PL}(mask, e, esize) then
        result[e*:esize] = element;
    end;
end;

ZAslice{VL}(d, esize, vertical, slice) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

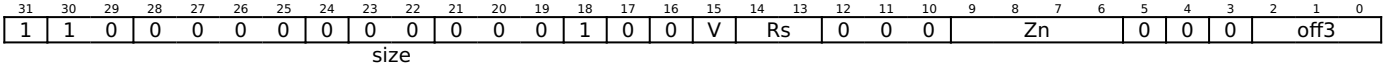
MOVA (vector to tile, two registers)

Move Z two-vector operand to ZA two-slice operand

This instruction operates on a horizontal or vertical ZA two-slice operand within a ZA tile of the specified element size.
The first slice of the two-slice operand is selected by rounding down the sum of the slice index register and the immediate offset to the nearest lower multiple of 2, modulo the number of slices in the tile.
The immediate offset is a multiple of 2 in the range 0 to the number of elements in a 128-bit vector segment minus 2.
This instruction is unpredicated.
This instruction is used by the alias MOV (vector to tile, two registers).
It has encodings from 4 classes: 8-bit , 16-bit , 32-bit and 64-bit

8-bit

(FEAT_SME2)



Encoding

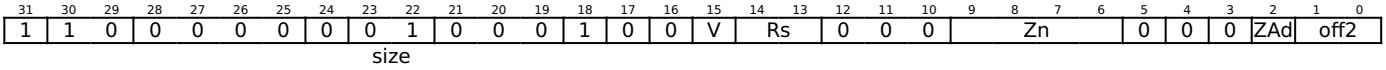
```
MOVA ZA0<HV>.B[<Ws>, <offs1>:<offs2>], { <Zn1>.B-<Zn2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 8;
let n : integer = UInt(Zn::'0');
let d : integer = 0;
let offset : integer = UInt(off3::'0');
let vertical : boolean = V == '1';
```

16-bit

(FEAT_SME2)



Encoding

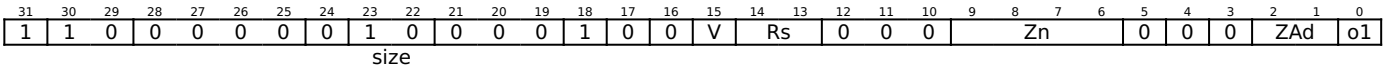
```
MOVA <ZAd><HV>.H[<Ws>, <offs1>:<offs2>], { <Zn1>.H-<Zn2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(ZAd);
let offset : integer = UInt(off2::'0');
let vertical : boolean = V == '1';
```

32-bit

(FEAT_SME2)



Encoding

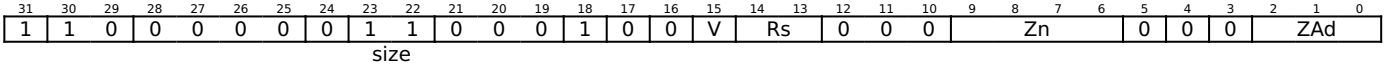
```
MOVA <ZAd><HV>.S[<Ws>, <offs1>:<offs2>], { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(ZAd);
let offset : integer = UInt(o1::'0');
let vertical : boolean = V == '1';
```

64-bit

(FEAT_SME2)



Encoding

```
MOVA <ZAd><HV>.D[<Ws>, <offs1>:<offs2>], { <Zn1>.D-<Zn2>.D }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 64;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(ZAd);
let offset : integer = 0;
let vertical : boolean = V == '1';
```

Assembler Symbols

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs1> For the "8-bit" variant: is the first slice index offset, encoded as "off3" field times 2.
For the "16-bit" variant: is the first slice index offset, encoded as "off2" field times 2.
For the "32-bit" variant: is the first slice index offset, encoded as "o1" field times 2.
For the "64-bit" variant: is the first slice index offset, with implicit value 0.

<offs2> For the "8-bit" variant: is the second slice index offset, encoded as "off3" field times 2 plus 1.
For the "16-bit" variant: is the second slice index offset, encoded as "off2" field times 2 plus 1.
For the "32-bit" variant: is the second slice index offset, encoded as "o1" field times 2 plus 1.
For the "64-bit" variant: is the second slice index offset, with implicit value 1.

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.

<Zn2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

<ZAd> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAd" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAd" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAd" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
if nreg == 4 && esize == 64 && VL < 256 then EndOfDecode(Decode_UNDEF); end;
let slices : integer = VL DIV esize;
let index : bits(32) = X{32}(s);
let slice : integer = ((UInt(index) - UInt(index) MOD nreg) + offset) MOD slices;
for r = 0 to nreg-1 do
    let result : bits(VL) = Z{VL}(n + r);
    ZAslice{VL}(d, esize, vertical, slice + r) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVAZ (array to vector, four registers)

Move and zero four ZA single-vector groups to Z four-vector operand

This instruction operates on four ZA single-vector groups. The ZA single-vector groups are zeroed after moving their contents to the destination vectors.

The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.

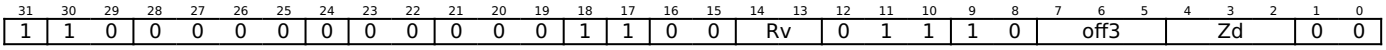
The vector group symbol VGx4 indicates that the instruction operates on four ZA single-vector groups.

The preferred disassembly syntax uses a 64-bit element size, but an assembler should accept any element size if it is used consistently for all operands. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

SME2

(FEAT_SME2p1)



Encoding

```
MOVAZ { <Zd1>.D-<Zd4>.D }, ZA.D[<Wv>, <offs>{, VGx4}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off3);
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
for r = 0 to nreg-1 do
    let result : bits(VL) = ZAvector{VL}(vec);
    ZAvector{VL}(vec) = Zeros{VL};
    Z{VL}(d + r) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVAZ (array to vector, two registers)

Move and zero two ZA single-vector groups to Z two-vector operand

This instruction operates on two ZA single-vector groups. The ZA single-vector groups are zeroed after moving their contents to the destination vectors.

The single-vector group within each half of the ZA array is selected by the sum of the vector select register and offset, modulo half the number of ZA array vectors.

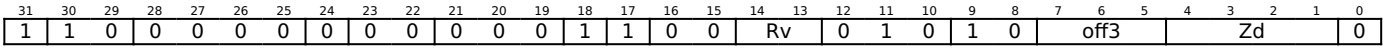
The vector group symbol VGx2 indicates that the instruction operates on two ZA single-vector groups.

The preferred disassembly syntax uses a 64-bit element size, but an assembler should accept any element size if it is used consistently for all operands. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

SME2

(FEAT_SME2p1)



Encoding

```
MOVAZ { <Zd1>.D-<Zd2>.D }, ZA.D[<Wv>, <offs>{, VGx2}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off3);
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
```

Assembler Symbols

<Zd1>	Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
for r = 0 to nreg-1 do
    let result : bits(VL) = ZAvector{VL}(vec);
    ZAvector{VL}(vec) = Zeros{VL};
    Z{VL}(d + r) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVAZ (tile to vector, four registers)

Move and zero ZA four-slice operand to Z four-vector operand

This instruction operates on a horizontal or vertical ZA four-slice operand within a ZA tile of the specified element size. The tile slices are zeroed after moving their contents to the destination vectors.

The first slice of the four-slice operand is selected by rounding down the sum of the slice index register and the immediate offset to the nearest lower multiple of 4, modulo the number of slices in the tile.

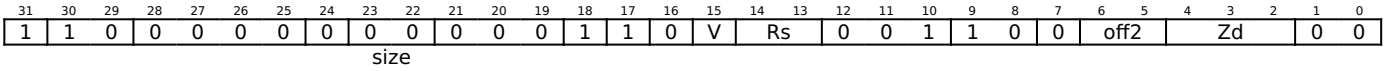
The immediate offset is a multiple of 4 in the range 0 to the number of elements in a 128-bit vector segment minus 4.

This instruction is unpredicated.

It has encodings from 4 classes: 8-bit , 16-bit , 32-bit and 64-bit

8-bit

(FEAT_SME2p1)



Encoding

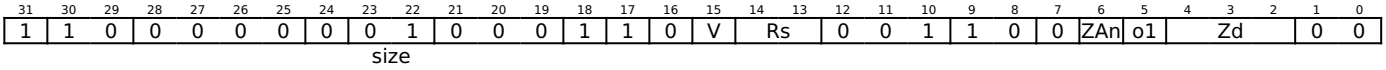
```
MOVAZ { <Zd1>.B-<Zd4>.B }, ZA0<HV>.B[<Ws>, <offs1>:<offs4>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 8;
let d : integer = UInt(Zd::'00');
let n : integer = 0;
let offset : integer = UInt(off2::'00');
let vertical : boolean = V == '1';
```

16-bit

(FEAT_SME2p1)



Encoding

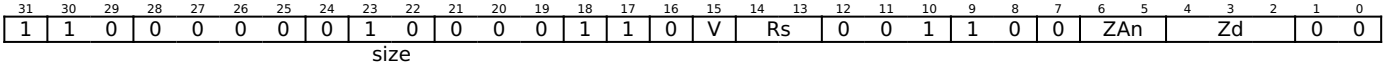
```
MOVAZ { <Zd1>.H-<Zd4>.H }, <ZAn><HV>.H[<Ws>, <offs1>:<offs4>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 16;
let d : integer = UInt(Zd::'00');
let n : integer = UInt(ZAn);
let offset : integer = UInt(o1::'00');
let vertical : boolean = V == '1';
```

32-bit

(FEAT_SME2p1)



Encoding

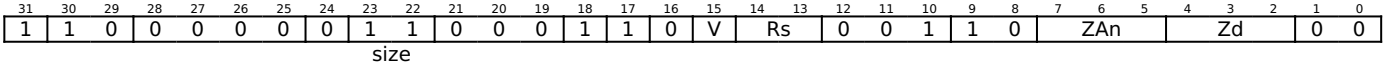
```
MOVAZ { <Zd1>.S-<Zd4>.S }, <ZAn><HV>.S[<Ws>, <offs1>:<offs4>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 32;
let d : integer = UInt(Zd::'00');
let n : integer = UInt(ZAn);
let offset : integer = 0;
let vertical : boolean = V == '1';
```

64-bit

(FEAT_SME2p1)



Encoding

```
MOVAZ { <Zd1>.D-<Zd4>.D }, <ZAn><HV>.D[<Ws>, <offs1>:<offs4>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 256 then
    EndOfDecode(Decode_UNDEF);
end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 4;
let esize : integer{} = 64;
let d : integer = UInt(Zd::'00');
let n : integer = UInt(ZAn);
let offset : integer = 0;
let vertical : boolean = V == '1';
```

Assembler Symbols

<Zd1> Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs1> For the "8-bit" variant: is the first slice index offset, encoded as "off2" field times 4.
For the "16-bit" variant: is the first slice index offset, encoded as "o1" field times 4.
For the "32-bit" and "64-bit" variants: is the first slice index offset, with implicit value 0.

<offs4> For the "8-bit" variant: is the fourth slice index offset, encoded as "off2" field times 4 plus 3.
For the "16-bit" variant: is the fourth slice index offset, encoded as "o1" field times 4 plus 3.
For the "32-bit" and "64-bit" variants: is the fourth slice index offset, with implicit value 3.

<ZAn> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAn" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAn" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAn" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
if nreg == 4 && esize == 64 && VL < 256 then EndOfDecode(Decode_UNDEF); end;
let slices : integer = VL DIV esize;
let index : bits(32) = X{32}(s);
let slice : integer = ((UInt(index) - UInt(index) MOD nreg) + offset) MOD slices;
for r = 0 to nreg-1 do
    let result : bits(VL) = ZAslice{VL}(n, esize, vertical, slice + r);
    ZAslice{VL}(n, esize, vertical, slice + r) = Zeros{VL};
    Z{VL}(d + r) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVAZ (tile to vector, single)

Move and zero ZA tile slice to Z vector

This instruction operates on a horizontal or vertical slice within a named ZA tile of the specified element size. The tile slice is zeroed after moving its contents to the destination vector.

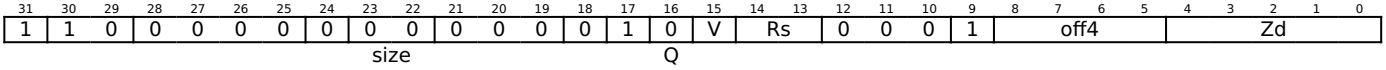
The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of such elements in a vector. The immediate offset is in the range 0 to the number of elements in a 128-bit vector segment minus 1.

This instruction is unpredicated.

It has encodings from 5 classes: 8-bit , 16-bit , 32-bit , 64-bit and 128-bit

8-bit

(FEAT_SME2p1)



Encoding

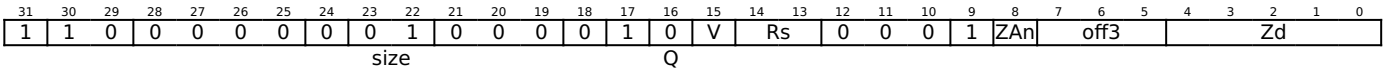
```
MOVAZ <Zd>.B, ZA0<HV>.B[<Ws>, <offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let n : integer = 0;
let offset : integer = UInt(off4);
let esize : integer{} = 8;
let d : integer = UInt(Zd);
let vertical : boolean = V == '1';
```

16-bit

(FEAT_SME2p1)



Encoding

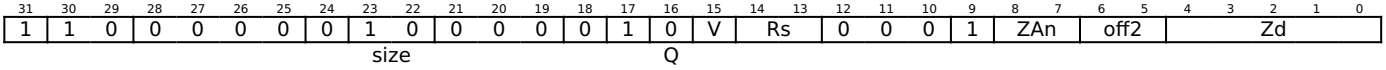
```
MOVAZ <Zd>.H, <ZAn><HV>.H[<Ws>, <offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let n : integer = UInt(ZAn);
let offset : integer = UInt(off3);
let esize : integer{} = 16;
let d : integer = UInt(Zd);
let vertical : boolean = V == '1';
```

32-bit

(FEAT_SME2p1)



Encoding

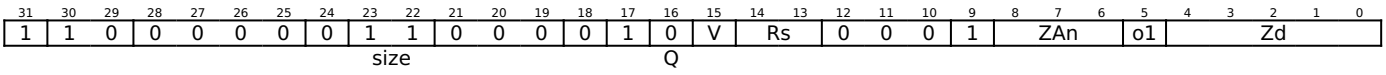
```
MOVAZ <Zd>.S, <ZAn><HV>.S[<Ws>, <offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let n : integer = UInt(ZAn);
let offset : integer = UInt(off2);
let esize : integer{} = 32;
let d : integer = UInt(Zd);
let vertical : boolean = V == '1';
```

64-bit

(FEAT_SME2p1)



Encoding

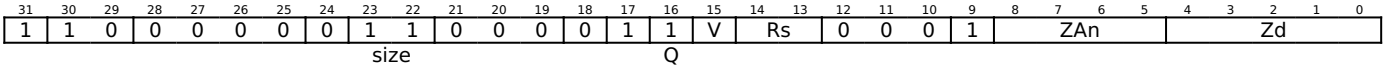
```
MOVAZ <Zd>.D, <ZAn><HV>.D[<Ws>, <offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let n : integer = UInt(ZAn);
let offset : integer = UInt(o1);
let esize : integer{} = 64;
let d : integer = UInt(Zd);
let vertical : boolean = V == '1';
```

128-bit

(FEAT_SME2p1)



Encoding

```
MOVAZ <Zd>.Q, <ZAn><HV>.Q[<Ws>, <offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let n : integer = UInt(ZAn);
let offset : integer = 0;
let esize : integer{} = 128;
let d : integer = UInt(Zd);
let vertical : boolean = V == '1';
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> For the "8-bit" variant: is the slice index offset, in the range 0 to 15, encoded in the "off4" field.
For the "16-bit" variant: is the slice index offset, in the range 0 to 7, encoded in the "off3" field.
For the "32-bit" variant: is the slice index offset, in the range 0 to 3, encoded in the "off2" field.
For the "64-bit" variant: is the slice index offset, in the range 0 to 1, encoded in the "o1" field.
For the "128-bit" variant: is the slice index offset, with implicit value 0.

<ZAn> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAn" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAn" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAn" field.
For the "128-bit" variant: is the name of the ZA tile ZA0-ZA15 to be accessed, encoded in the "ZAn" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer = VL DIV esize;
let index : bits(32) = X{32}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
let operand : bits(VL) = ZAslice{VL}(n, esize, vertical, slice);
ZAslice{VL}(n, esize, vertical, slice) = Zeros{VL};
Z{VL}(d) = operand;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVAZ (tile to vector, two registers)

Move and zero ZA two-slice operand to Z two-vector operand

This instruction operates on a horizontal or vertical ZA two-slice operand within a ZA tile of the specified element size. The tile slices are zeroed after moving their contents to the destination vectors.

The first slice of the two-slice operand is selected by rounding down the sum of the slice index register and the immediate offset to the nearest lower multiple of 2, modulo the number of slices in the tile.

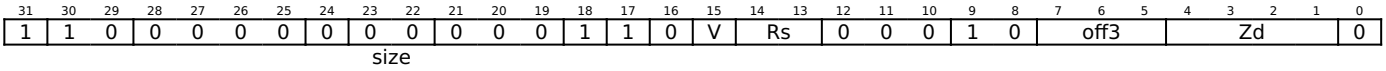
The immediate offset is a multiple of 2 in the range 0 to the number of elements in a 128-bit vector segment minus 2.

This instruction is unpredicated.

It has encodings from 4 classes: 8-bit , 16-bit , 32-bit and 64-bit

8-bit

(FEAT_SME2p1)



Encoding

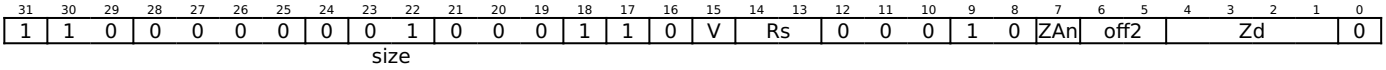
```
MOVAZ { <Zd1>.B-<Zd2>.B }, ZA0<HV>.B[<Ws>, <offs1>:<offs2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 8;
let d : integer = UInt(Zd::'0');
let n : integer = 0;
let offset : integer = UInt(off3::'0');
let vertical : boolean = V == '1';
```

16-bit

(FEAT_SME2p1)



Encoding

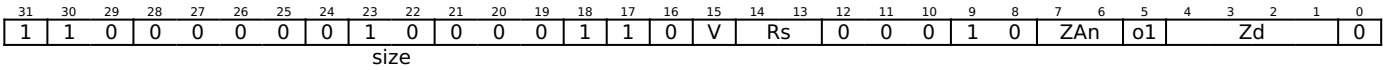
```
MOVAZ { <Zd1>.H-<Zd2>.H }, <ZAn><HV>.H[<Ws>, <offs1>:<offs2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 16;
let d : integer = UInt(Zd::'0');
let n : integer = UInt(ZAn);
let offset : integer = UInt(off2::'0');
let vertical : boolean = V == '1';
```

32-bit

(FEAT_SME2p1)



Encoding

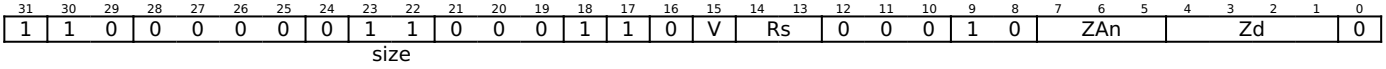
```
MOVAZ { <Zd1>.S-<Zd2>.S }, <ZAn><HV>.S[<Ws>, <offs1>:<offs2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 32;
let d : integer = UInt(Zd::'0');
let n : integer = UInt(ZAn);
let offset : integer = UInt(o1::'0');
let vertical : boolean = V == '1';
```

64-bit

(FEAT_SME2p1)



Encoding

```
MOVAZ { <Zd1>.D-<Zd2>.D }, <ZAn><HV>.D[<Ws>, <offs1>:<offs2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let s : integer = UInt('011'::Rs);
let nreg : integer{} = 2;
let esize : integer{} = 64;
let d : integer = UInt(Zd::'0');
let n : integer = UInt(ZAn);
let offset : integer = 0;
let vertical : boolean = V == '1';
```

Assembler Symbols

<Zd1> Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs1> For the "8-bit" variant: is the first slice index offset, encoded as "off3" field times 2.
For the "16-bit" variant: is the first slice index offset, encoded as "off2" field times 2.
For the "32-bit" variant: is the first slice index offset, encoded as "o1" field times 2.
For the "64-bit" variant: is the first slice index offset, with implicit value 0.

<offs2> For the "8-bit" variant: is the second slice index offset, encoded as "off3" field times 2 plus 1.
For the "16-bit" variant: is the second slice index offset, encoded as "off2" field times 2 plus 1.
For the "32-bit" variant: is the second slice index offset, encoded as "o1" field times 2 plus 1.
For the "64-bit" variant: is the second slice index offset, with implicit value 1.

<ZAn> For the "16-bit" variant: is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAn" field.
For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAn" field.
For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAn" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
if nreg == 4 && esize == 64 && VL < 256 then EndOfDecode(Decode_UNDEF); end;
let slices : integer = VL DIV esize;
let index : bits(32) = X{32}(s);
let slice : integer = ((UInt(index) - UInt(index) MOD nreg) + offset) MOD slices;
for r = 0 to nreg-1 do
    let result : bits(VL) = ZAslice{VL}(n, esize, vertical, slice + r);
    ZAslice{VL}(n, esize, vertical, slice + r) = Zeros{VL};
    Z{VL}(d + r) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

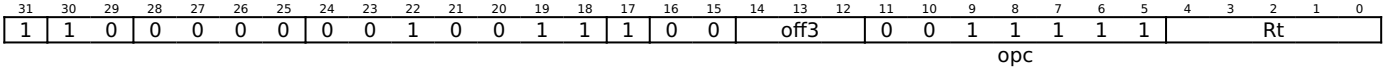
MOVT (scalar to table)

Move 8 bytes from general-purpose register to ZT0

This instruction moves 8 bytes to the ZT0 register at the byte offset specified by the immediate index from a general-purpose register. This instruction is UNDEFINED in Non-debug state.

SME2

(FEAT_SME2)



Encoding

```
MOVT ZT0[<offs>], <Xt>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !Halted() then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let offset : integer = UInt(off3);
```

Assembler Symbols

<offs>	Is the immediate byte offset, a multiple of 8 in the range of 0 to 56, encoded in the "off3" field as <offs>/8.
<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.

Operation

```
CheckSMEEnabled();
CheckSMEZT0Enabled();
var result : bits(512) = ZT0{512}();
result[offset*:64] = X{64}(t);
ZT0{512}() = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

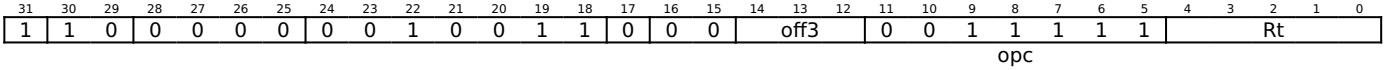
MOVT (table to scalar)

Move 8 bytes from ZT0 to general-purpose register

This instruction moves 8 bytes to a general-purpose register from the ZT0 register at the byte offset specified by the immediate index. This instruction is UNDEFINED in Non-debug state.

SME2

(FEAT_SME2)



Encoding

```
MOVT <Xt>, ZT0[<offs>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) || !Halted() then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Rt);
let offset : integer = UInt(off3);
```

Assembler Symbols

<Xt>	Is the 64-bit name of the general-purpose register to be transferred, encoded in the "Rt" field.
<offs>	Is the immediate byte offset, a multiple of 8 in the range of 0 to 56, encoded in the "off3" field as <offs>/8.

Operation

```
CheckSMEEnabled();
CheckSMEZT0Enabled();
let operand : bits(512) = ZT0{512}();
X{64}(t) = operand[offset*:64];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

MOVT (vector to table)

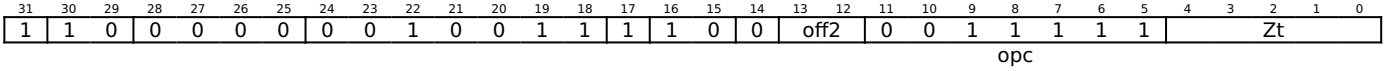
Move vector register to ZT0

This instruction copies the source vector register to ZT0 at the vector length offset specified by the immediate index. When the index is zero, the instruction writes zeros to the most significant (512-VL) bits of the ZT0 register. When the index is not zero, the unindexed portions of ZT0 remain unchanged.

This instruction is unpredicated.

SME2

(FEAT_SME_LUTv2)



Encoding

```
MOVT ZT0{[<offs>, MUL VL]}, <Zt>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_LUTv2) then EndOfDecode(Decode_UNDEF); end;
let t : integer = UInt(Zt);
let imm : integer = UInt(off2);
```

Assembler Symbols

<offs>	Is the vector length offset, in the range 0 to 3, defaulting to 0 when omitted, encoded in the "off2" field.
<Zt>	Is the name of the scalable vector register to be transferred, encoded in the "Zt" field.

Operation

```
CheckStreamingSVEEnabled();
CheckSMEZT0Enabled();
let VL : integer{} = CurrentVL();
let tsize : integer{} = (if VL <= 512 then VL else 512) as integer{128, 256, 512};
let offset : integer = imm MOD (512 DIV tsize);
var result : bits(512) = if imm == 0 then Zeros{512} else ZT0{512}();
result[offset*:tsize] = Z{VL}(t)[tsize-1:0];
ZT0{512}() = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

RDSVL

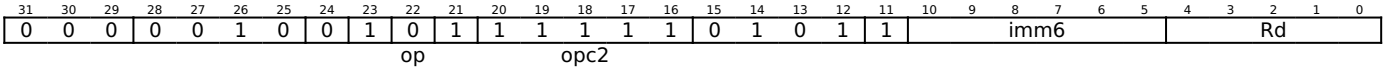
Read multiple of Streaming SVE vector register size to scalar register

This instruction multiplies the Streaming SVE vector register size in bytes by an immediate in the range -32 to 31 and places the result in the 64-bit destination general-purpose register.

This instruction does not require the PE to be in Streaming SVE mode.

SME

(FEAT_SME)



Encoding

```
RDSVL <Xd>, #<imm>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let d : integer = UInt(Rd);
let imm : integer = SInt(imm6);
```

Assembler Symbols

<Xd>	Is the 64-bit name of the destination general-purpose register, encoded in the "Rd" field.
<imm>	Is the signed immediate operand, in the range -32 to 31, encoded in the "imm6" field.

Operation

```
CheckSMEEnabled();
let SVL : integer = CurrentSVL();
let len : integer = imm * (SVL DIV 8);
X{64}(d) = len[63:0];
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SCLAMP

Multi-vector signed clamp to minimum/maximum

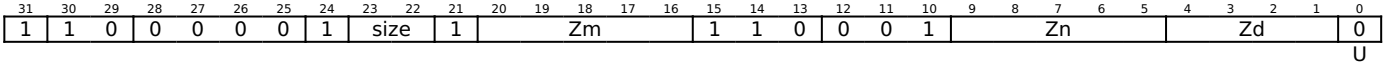
This instruction clamps each signed element in the two or four destination vectors to between the signed minimum value in the corresponding element of the first source vector and the signed maximum value in the corresponding element of the second source vector and destructively places the clamped results in the corresponding elements of the two or four destination vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

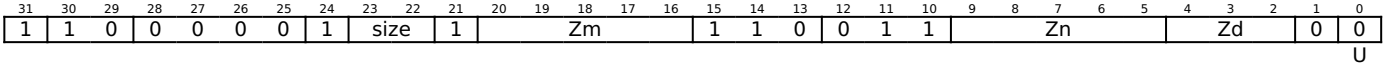
```
SCLAMP { <Zd1>.<T>-<Zd2>.<T> }, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
SCLAMP { <Zd1>.<T>-<Zd4>.<T> }, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1> For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = Z{VL}(d+r);
    for e = 0 to elements-1 do
        let element1 : integer = SInt(operand1[e*:esize]);
        let element2 : integer = SInt(operand2[e*:esize]);
        let element3 : integer = SInt(operand3[e*:esize]);
        let res : integer = Min(Max(element1, element3), element2);
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SCVTF

Multi-vector signed 32-bit integer convert to single-precision

This instruction converts each element of the two or four source vectors from signed 32-bit integer to single-precision, and places the results in the corresponding elements of the two or four destination vectors.

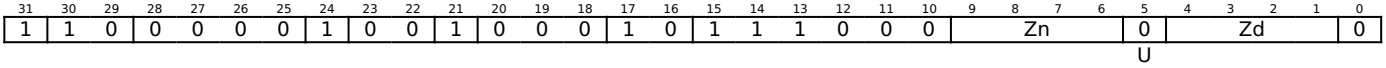
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

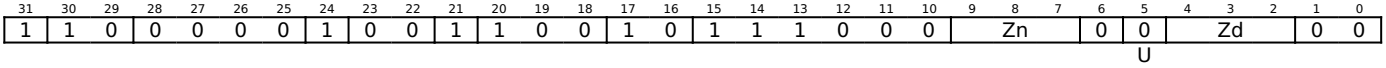
```
SCVTF { <Zd1>.S-<Zd2>.S }, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
```

Four registers

(FEAT_SME2)



Encoding

```
SCVTF { <Zd1>.S-<Zd4>.S }, { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
let unsigned : boolean = FALSE;
let rounding : FPRounding = FPRoundingMode(FPCR());
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn1>	For the "Two registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(32) = operand[e*:32];
        results[[r]][e*:32] = FixedToFP{32, 32}(element, 0, unsigned, FPCR(), rounding);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (2-way, multiple and indexed vector)

Multi-vector signed 16-bit integer dot product by indexed element to 32-bit integer

This instruction calculates the dot product of two signed 16-bit integer values held in each 32-bit element of the two or four first source vectors and two signed 16-bit integer values in the corresponding indexed 32-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to 3.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

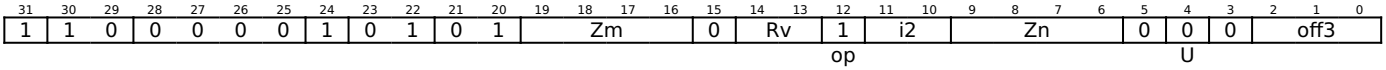
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

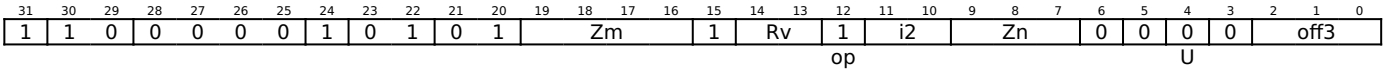
```
SDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
SDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        for i = 0 to 1 do
            let element1 : integer = SInt(operand1[(2 * e + i)*:(esize DIV 2)]);
            let element2 : integer = SInt(operand2[(2 * s + i)*:(esize DIV 2)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (2-way, multiple and single vector)

Multi-vector signed 16-bit integer dot product by vector to 32-bit integer

This instruction calculates the dot product of two signed 16-bit integer values held in each 32-bit element of the two or four first source vectors and two signed 16-bit integer values in the corresponding 32-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

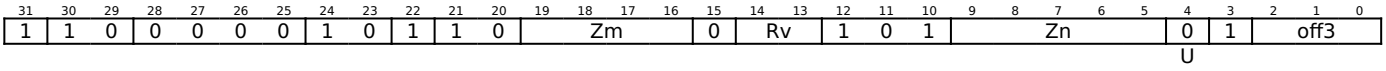
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

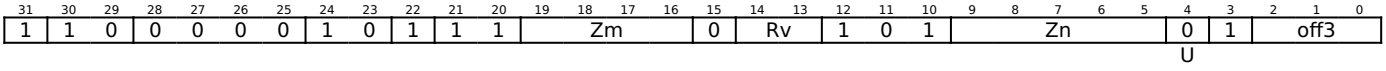
```
SDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
SDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 1 do
            let element1 : integer = SInt(operand1[(2 * e + i)*:(esize DIV 2)]);
            let element2 : integer = SInt(operand2[(2 * e + i)*:(esize DIV 2)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (2-way, multiple vectors)

Multi-vector signed 16-bit integer dot product to 32-bit integer

This instruction calculates the dot product of two signed 16-bit integer values held in each 32-bit element of the two or four first source vectors and two signed 16-bit integer values in the corresponding 32-bit element of the two or four second source vectors. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

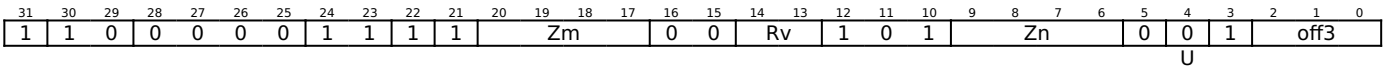
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

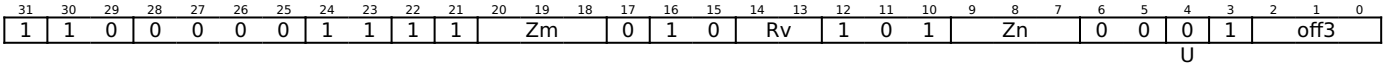
```
SDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
SDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 1 do
            let element1 : integer = SInt(operand1[(2 * e + i)*:(esize DIV 2)]);
            let element2 : integer = SInt(operand2[(2 * e + i)*:(esize DIV 2)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (4-way, multiple and indexed vector)

Multi-vector signed integer dot product by indexed element

This instruction calculates the dot product of four signed 8-bit or 16-bit integer values held in each 32-bit or 64-bit element of the two or four first source vectors and four signed 8-bit or 16-bit integer values in the corresponding indexed 32-bit or 64-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit or 64-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to one less than the number of groups per 128-bit segment.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

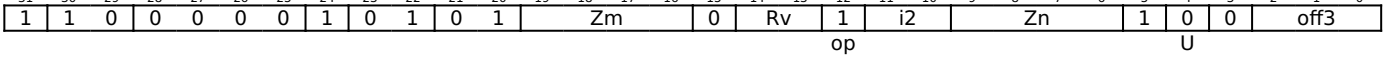
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 4 classes: Two ZA single-vectors of 32-bit elements , Two ZA single-vectors of 64-bit elements , Four ZA single-vectors of 32-bit elements and Four ZA single-vectors of 64-bit elements

Two ZA single-vectors of 32-bit elements

(FEAT_SME2)



Encoding

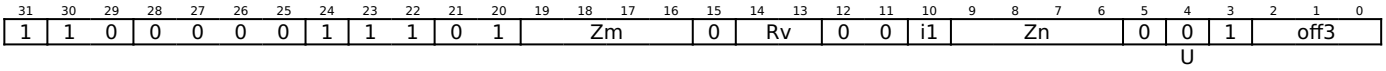
```
SDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Two ZA single-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

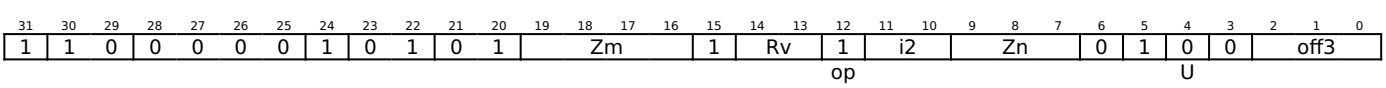
```
SDOT ZA.D[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 64;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i1);
let nreg : integer{} = 2;
```

Four ZA single-vectors of 32-bit elements

(FEAT_SME2)



Encoding

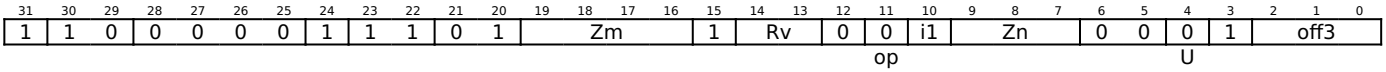
```
SDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Four ZA single-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

```
SDOT ZA.D[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 64;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i1);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors of 32-bit elements" and "Two ZA single-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors of 32-bit elements" and "Four ZA single-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "Four ZA single-vectors of 32-bit elements" and "Two ZA single-vectors of 32-bit elements" variants: is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field. For the "Four ZA single-vectors of 64-bit elements" and "Two ZA single-vectors of 64-bit elements" variants: is the immediate index of a group of four 16-bit values within each 128-bit vector segment, in the range 0 to 1, encoded in the "i1" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        for i = 0 to 3 do
            let element1 : integer = SInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4 * s + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (4-way, multiple and single vector)

Multi-vector signed integer dot product by vector

This instruction calculates the dot product of four signed 8-bit or 16-bit integer values held in each 32-bit or 64-bit element of the two or four first source vectors and four signed 8-bit or 16-bit integer values in the corresponding 32-bit or 64-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit or 64-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0	Zm				0	Rv		1	0	1	Zn					0	0	off3		
																											U				

Encoding

```
SDOT ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	1	Zm				0	Rv		1	0	1	Zn					0	0	off3		
																											U				

Encoding

```
SDOT ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let element1 : integer = SInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4 * e + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SDOT (4-way, multiple vectors)

Multi-vector signed integer dot product

This instruction calculates the dot product of four signed 8-bit or 16-bit integer values held in each 32-bit or 64-bit element of the two or four first source vectors and four signed 8-bit or 16-bit integer values in the corresponding 32-bit or 64-bit element of the two or four second source vectors. The widened dot product result is destructively added to the corresponding 32-bit or 64-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

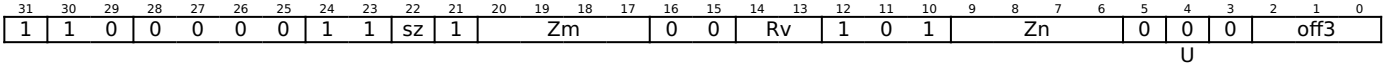
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

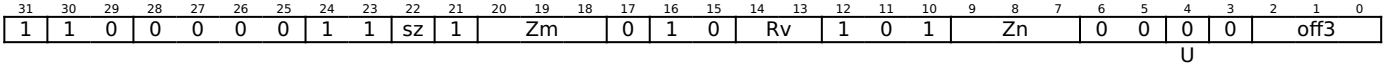
```
SDOT ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, { <Zm1>.<Tb>-<Zm2>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
SDOT ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, { <Zm1>.<Tb>-<Zm4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1> For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let element1 : integer = SInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4 * e + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SEL

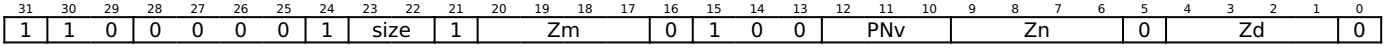
Multi-vector conditional select

This instruction selects consecutive elements from the two or four first source vectors where vector select predicate elements are true, and places them in the corresponding elements of the two or four destination vectors. Where vector select predicate elements are false, the corresponding consecutive elements are selected from the two or four second source vectors and placed in the remaining elements of the destination vectors.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

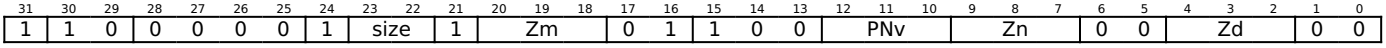
```
SEL { <Zd1>.<T>-<Zd2>.<T> }, <PNv>, { <Zn1>.<T>-<Zn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let d : integer = UInt(Zd::'0');
let v : integer = UInt('1'::PNv);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
SEL { <Zd1>.<T>-<Zd4>.<T> }, <PNv>, { <Zn1>.<T>-<Zn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let d : integer = UInt(Zd::'00');
let v : integer = UInt('1'::PNv);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1> For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<PNv> Is the name of the vector select predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNv" plus 8.

<Zn1> For the "Two registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1>	For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);
let pred : bits(PL) = P{PL}(v);
let mask : bits(PL*nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            results[[r]][e*:esize] = operand1[e*:esize];
        else
            results[[r]][e*:esize] = operand2[e*:esize];
        end;
    end;
end;
for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMAX (multiple and single vector)

Multi-vector signed maximum by vector

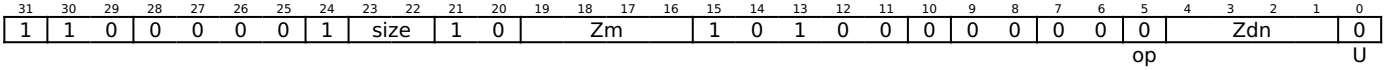
This instruction determines the signed maximum of elements of the second source vector and the corresponding elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

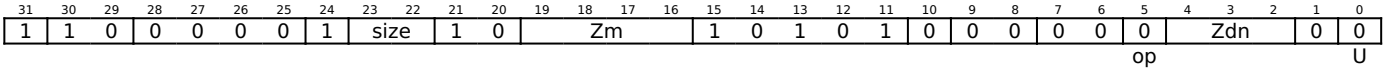
```
SMAX { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
let unsigned : boolean = FALSE;
```

Four registers

(FEAT_SME2)



Encoding

```
SMAX { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let op1elt : bits(esize) = operand1[e*:esize];
        let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
        let op2elt : bits(esize) = operand2[e*:esize];
        let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
        let res : integer = Max(element1, element2);
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMAX (multiple vectors)

Multi-vector signed maximum

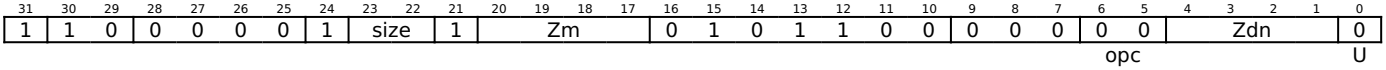
This instruction determines the signed maximum of elements of the two or four second source vectors and the corresponding elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

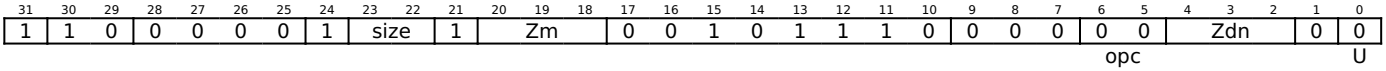
```
SMAX { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
let unsigned : boolean = FALSE;
```

Four registers

(FEAT_SME2)



Encoding

```
SMAX { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let op1elt : bits(esize) = operand1[e*:esize];
        let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
        let op2elt : bits(esize) = operand2[e*:esize];
        let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
        let res : integer = Max(element1, element2);
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMIN (multiple and single vector)

Multi-vector signed minimum by vector

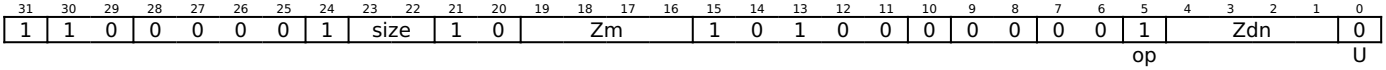
This instruction determines the signed minimum of elements of the second source vector and the corresponding elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

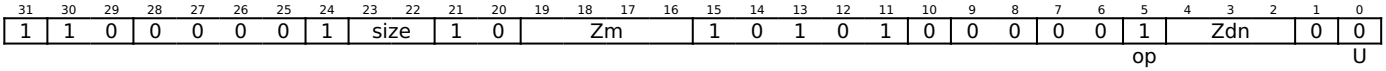
```
SMIN { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
let unsigned : boolean = FALSE;
```

Four registers

(FEAT_SME2)



Encoding

```
SMIN { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let op1elt : bits(esize) = operand1[e*:esize];
        let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
        let op2elt : bits(esize) = operand2[e*:esize];
        let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
        let res : integer = Min(element1, element2);
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMIN (multiple vectors)

Multi-vector signed minimum

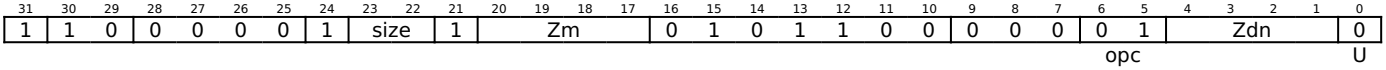
This instruction determines the signed minimum of elements of the two or four second source vectors and the corresponding elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

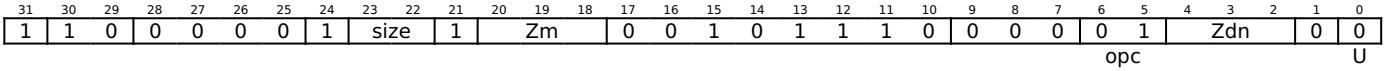
```
SMIN { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
let unsigned : boolean = FALSE;
```

Four registers

(FEAT_SME2)



Encoding

```
SMIN { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let op1elt : bits(esize) = operand1[e*:esize];
        let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
        let op2elt : bits(esize) = operand2[e*:esize];
        let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
        let res : integer = Min(element1, element2);
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLAL (multiple and indexed vector)

Multi-vector signed 16-bit integer multiply-add by indexed element to 32-bit integer

This instruction multiplies each signed 16-bit element in the one, two, or four first source vectors by each signed 16-bit indexed element of the second source vector, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA double-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to 7.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

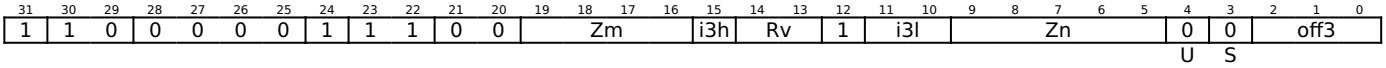
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

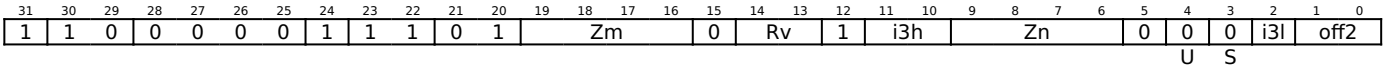
```
SMLAL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

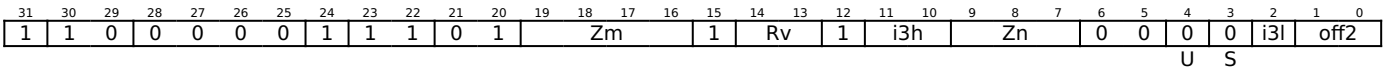
```
SMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
SMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 2 * segmentbase + index;
            let element1 : integer = SInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = SInt(operand2[s*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLAL (multiple and single vector)

Multi-vector signed 16-bit integer multiply-add by vector to 32-bit integer

This instruction multiplies each signed 16-bit element in the one, two, or four first source vectors by each signed 16-bit element in the second source vector, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA double-vector groups.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

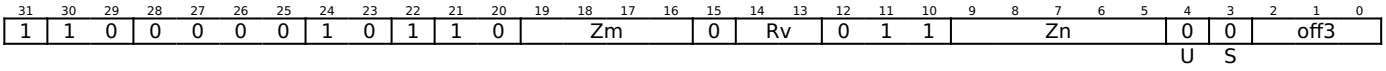
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

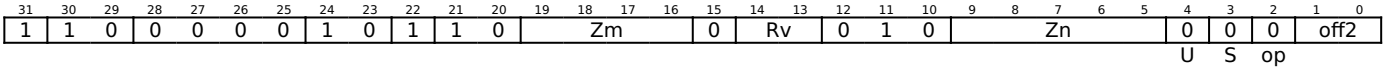
```
SMLAL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

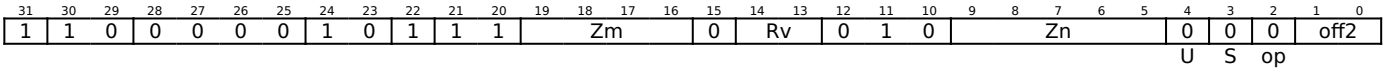
```
SMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
SMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = SInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = SInt(operand2[(2*e+i)*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLAL (multiple vectors)

Multi-vector signed 16-bit integer multiply-add to 32-bit integer

This instruction multiplies each signed 16-bit element in the two or four first source vectors by each signed 16-bit element in the two or four second source vectors, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA double-vector groups.

The double-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

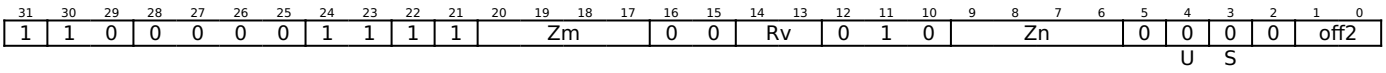
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA double-vectors and Four ZA double-vectors

Two ZA double-vectors

(FEAT_SME2)



Encoding

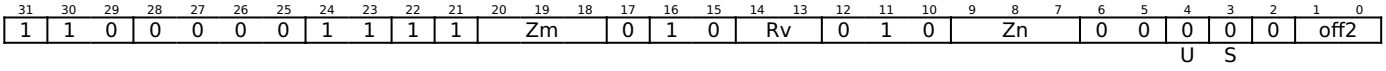
```
SMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
SMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "off2" field times 2.
<offs2>	Is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = SInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = SInt(operand2[(2*e+i)*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLALL (multiple and indexed vector)

Multi-vector signed integer multiply-add long long by indexed element

This instruction multiplies each signed 8-bit or 16-bit element in the one, two, or four first source vectors by each signed 8-bit or 16-bit indexed element of the second source vector, widens each product to 32 bits or 64 bits, and destructively adds these values to the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

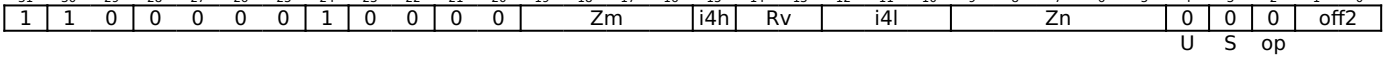
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 6 classes: One ZA quad-vector of 32-bit elements , One ZA quad-vector of 64-bit elements , Two ZA quad-vectors of 32-bit elements , Two ZA quad-vectors of 64-bit elements , Four ZA quad-vectors of 32-bit elements and Four ZA quad-vectors of 64-bit elements

One ZA quad-vector of 32-bit elements

(FEAT_SME2)



Encoding

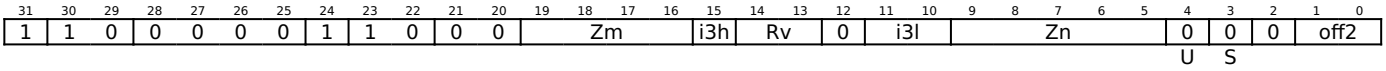
```
SMLALL ZA.S[<Wv>, <offs1>:<offs4>], <Zn>.B, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 1;
```

One ZA quad-vector of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

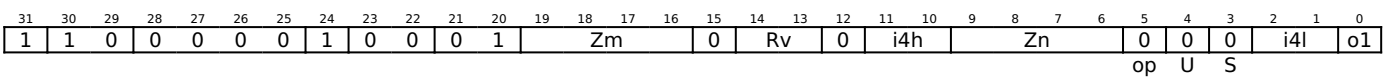
```
SMLALL ZA.D[<Wv>, <offs1>:<offs4>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
```

Two ZA quad-vectors of 32-bit elements

(FEAT_SME2)



Encoding

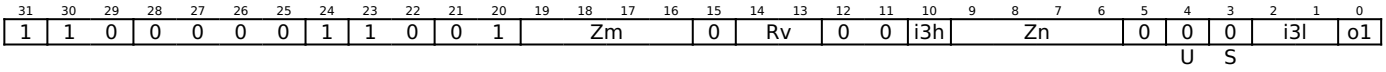
```
SMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 2;
```

Two ZA quad-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

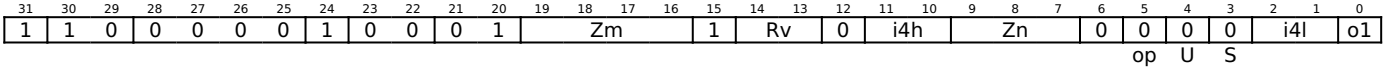
```
SMLALL ZA.D[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA quad-vectors of 32-bit elements

(FEAT_SME2)



Encoding

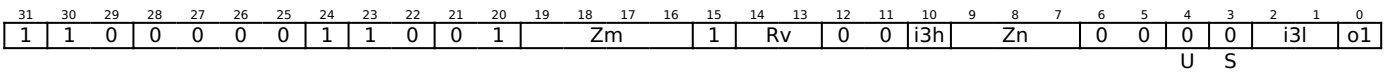
```
SMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 4;
```

Four ZA quad-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

```
SMLALL ZA.D[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA quad-vector of 32-bit elements" and "One ZA quad-vector of 64-bit elements" variants: is the first vector select offset, encoded as "off2" field times 4. For the "Four ZA quad-vectors of 32-bit elements", "Four ZA quad-vectors of 64-bit elements", "Two ZA quad-vectors of 32-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the first vector select offset, encoded as "o1" field times 4.
<offs4>	For the "One ZA quad-vector of 32-bit elements" and "One ZA quad-vector of 64-bit elements" variants: is the fourth vector select offset, encoded as "off2" field times 4 plus 3. For the "Four ZA quad-vectors of 32-bit elements", "Four ZA quad-vectors of 64-bit elements", "Two ZA quad-vectors of 32-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "Four ZA quad-vectors of 32-bit elements", "One ZA quad-vector of 32-bit elements", and "Two ZA quad-vectors of 32-bit elements" variants: is the element index, in the range 0 to 15, encoded in the "i4h:i4l" fields. For the "Four ZA quad-vectors of 64-bit elements", "One ZA quad-vector of 64-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA quad-vectors of 32-bit elements" and "Two ZA quad-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA quad-vectors of 32-bit elements" and "Four ZA quad-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 4 * segmentbase + index;
            let element1 : integer = SInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[s*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLALL (multiple and single vector)

Multi-vector signed integer multiply-add long long by vector

This instruction multiplies each signed 8-bit or 16-bit element in the one, two, or four first source vectors by each signed 8-bit or 16-bit element in the second source vector, widens each product to 32 bits or 64 bits, and destructively adds these values to the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

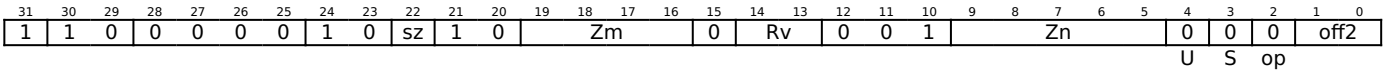
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 3 classes: One ZA quad-vector , Two ZA quad-vectors and Four ZA quad-vectors

One ZA quad-vector

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

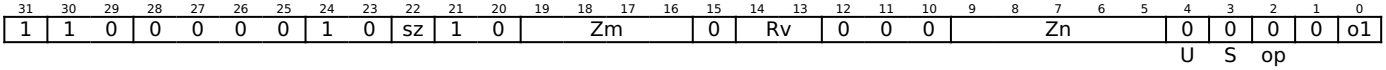
```
SMLALL ZA.<T>[<Wv>, <offs1>:<offs4>], <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let nreg : integer{} = 1;
```

Two ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

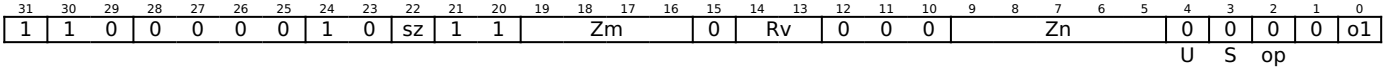
```
SMLALL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
SMLALL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs1> For the "One ZA quad-vector" variant: is the first vector select offset, encoded as "off2" field times 4.

For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the first vector select offset, encoded as "o1" field times 4.

<offs4> For the "One ZA quad-vector" variant: is the fourth vector select offset, encoded as "off2" field times 4 plus 3.

For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = SInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLALL (multiple vectors)

Multi-vector signed integer multiply-add long long

This instruction multiplies each signed 8-bit or 16-bit element in the two or four first source vectors by each signed 8-bit or 16-bit element in the one, two, or four second source vectors, widens each product to 32 bits or 64 bits, and destructively adds these values to the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The quad-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

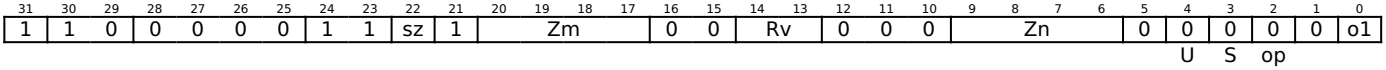
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA quad-vectors and Four ZA quad-vectors

Two ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

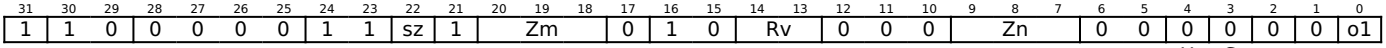
```
SMLALL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, { <Zm1>.<Tb>-<Zm2>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
SMLALL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, { <Zm1>.<Tb>-<Zm4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs1> Is the first vector select offset, encoded as "o1" field times 4.

<offs4> Is the fourth vector select offset, encoded as "o1" field times 4 plus 3.

<Zn1> For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1> For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = SInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLSL (multiple and indexed vector)

Multi-vector signed 16-bit integer multiply-subtract by indexed element from 32-bit integer

This instruction multiplies each signed 16-bit element in the one, two, or four first source vectors by each signed 16-bit indexed element of the second source vector, widens each product to 32 bits, and destructively subtracts these values from the corresponding 32-bit elements of the ZA double-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to 7.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

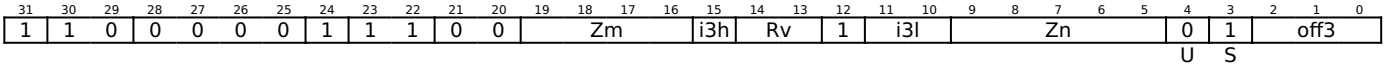
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

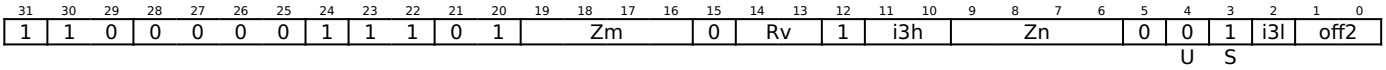
```
SMLSL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

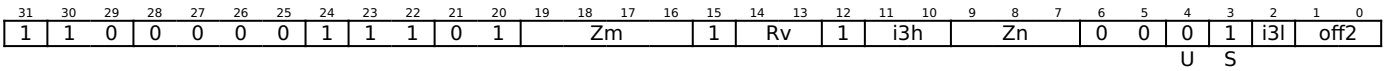
```
SMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
SMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 2 * segmentbase + index;
            let element1 : integer = SInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = SInt(operand2[s*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLSL (multiple and single vector)

Multi-vector signed 16-bit integer multiply-subtract by vector from 32-bit integer

This instruction multiplies each signed 16-bit element in the one, two, or four first source vectors by each signed 16-bit element in the second source vector, widens each product to 32 bits, and destructively subtracts these values from the corresponding 32-bit elements of the ZA double-vector groups.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

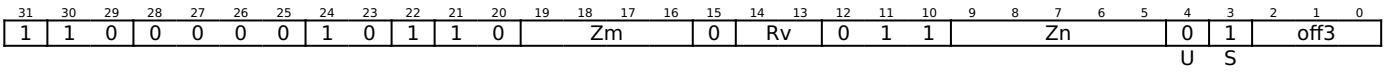
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

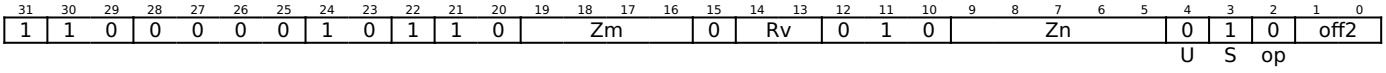
```
SMLSL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

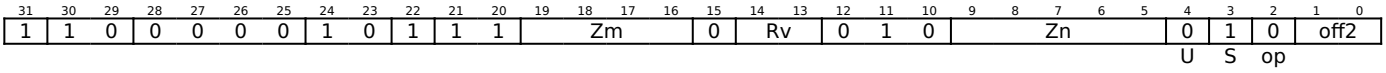
```
SMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
SMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = SInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = SInt(operand2[(2*e+i)*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLSL (multiple vectors)

Multi-vector signed 16-bit integer multiply-subtract from 32-bit integer

This instruction multiplies each signed 16-bit element in the two or four first source vectors by each signed 16-bit element in the two or four second source vectors, widens each product to 32 bits, and destructively subtracts these values from the corresponding 32-bit elements of the ZA double-vector groups.

The double-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

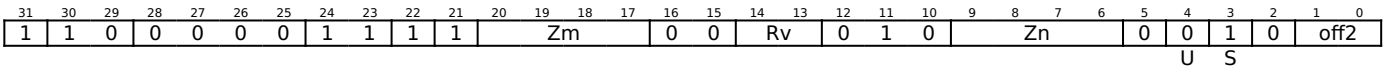
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA double-vectors and Four ZA double-vectors

Two ZA double-vectors

(FEAT_SME2)



Encoding

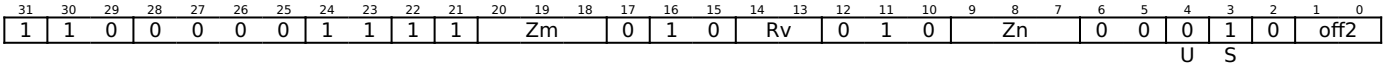
```
SMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
SMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "off2" field times 2.
<offs2>	Is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = SInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = SInt(operand2[(2*e+i)*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLSLL (multiple and indexed vector)

Multi-vector signed integer multiply-subtract long long by indexed element

This instruction multiplies each signed 8-bit or 16-bit element in the one, two, or four first source vectors by each signed 8-bit or 16-bit indexed element of the second source vector, widens each product to 32 bits or 64 bits, and destructively subtracts these values from the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

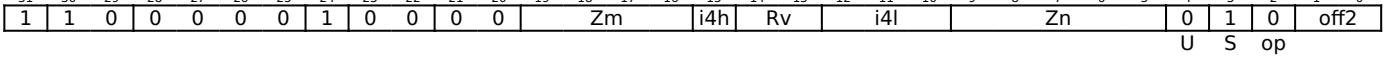
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 6 classes: One ZA quad-vector of 32-bit elements , One ZA quad-vector of 64-bit elements , Two ZA quad-vectors of 32-bit elements , Two ZA quad-vectors of 64-bit elements , Four ZA quad-vectors of 32-bit elements and Four ZA quad-vectors of 64-bit elements

One ZA quad-vector of 32-bit elements

(FEAT_SME2)



Encoding

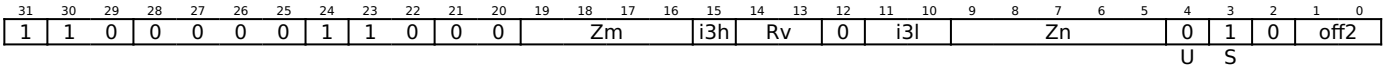
```
SMLSLL ZA.S[<Wv>, <offs1>:<offs4>], <Zn>.B, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 1;
```

One ZA quad-vector of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

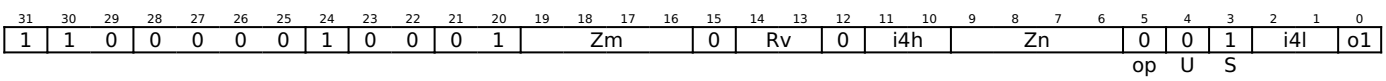
```
SMLSLL ZA.D[<Wv>, <offs1>:<offs4>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
```

Two ZA quad-vectors of 32-bit elements

(FEAT_SME2)



Encoding

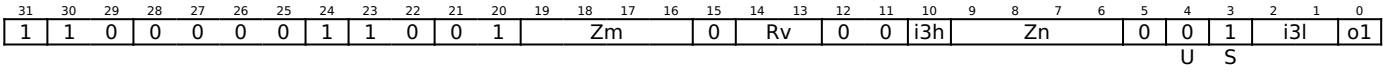
```
SMLSLL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 2;
```

Two ZA quad-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

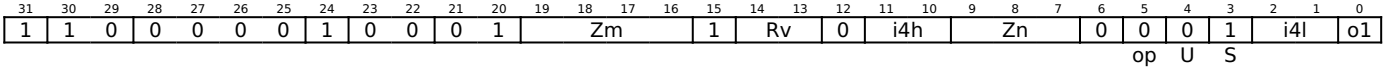
```
SMLSLL ZA.D[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA quad-vectors of 32-bit elements

(FEAT_SME2)



Encoding

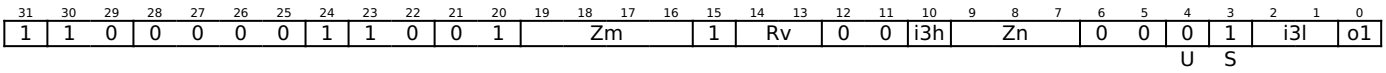
```
SMLSLL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 4;
```

Four ZA quad-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

```
SMLSLL ZA.D[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA quad-vector of 32-bit elements" and "One ZA quad-vector of 64-bit elements" variants: is the first vector select offset, encoded as "off2" field times 4. For the "Four ZA quad-vectors of 32-bit elements", "Four ZA quad-vectors of 64-bit elements", "Two ZA quad-vectors of 32-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the first vector select offset, encoded as "o1" field times 4.
<offs4>	For the "One ZA quad-vector of 32-bit elements" and "One ZA quad-vector of 64-bit elements" variants: is the fourth vector select offset, encoded as "off2" field times 4 plus 3. For the "Four ZA quad-vectors of 32-bit elements", "Four ZA quad-vectors of 64-bit elements", "Two ZA quad-vectors of 32-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "Four ZA quad-vectors of 32-bit elements", "One ZA quad-vector of 32-bit elements", and "Two ZA quad-vectors of 32-bit elements" variants: is the element index, in the range 0 to 15, encoded in the "i4h:i4l" fields. For the "Four ZA quad-vectors of 64-bit elements", "One ZA quad-vector of 64-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA quad-vectors of 32-bit elements" and "Two ZA quad-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA quad-vectors of 32-bit elements" and "Four ZA quad-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 4 * segmentbase + index;
            let element1 : integer = SInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[s*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLSLL (multiple and single vector)

Multi-vector signed integer multiply-subtract long long by vector

This instruction multiplies each signed 8-bit or 16-bit element in the one, two, or four first source vectors by each signed 8-bit or 16-bit element in the second source vector, widens each product to 32 bits or 64 bits, and destructively subtracts these values from the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

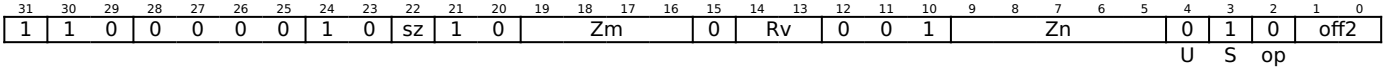
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 3 classes: One ZA quad-vector , Two ZA quad-vectors and Four ZA quad-vectors

One ZA quad-vector

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

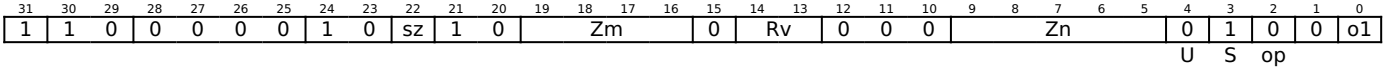
```
SMLSLL ZA.<T>[<Wv>, <offs1>:<offs4>], <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let nreg : integer{} = 1;
```

Two ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

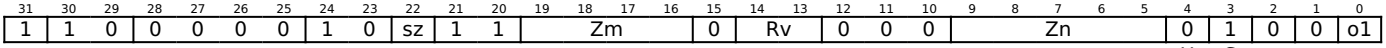
```
SMLSLL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
SMLSLL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs1> For the "One ZA quad-vector" variant: is the first vector select offset, encoded as "off2" field times 4.

For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the first vector select offset, encoded as "o1" field times 4.

<offs4> For the "One ZA quad-vector" variant: is the fourth vector select offset, encoded as "off2" field times 4 plus 3.

For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = SInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMLSLL (multiple vectors)

Multi-vector signed integer multiply-subtract long long

This instruction multiplies each signed 8-bit or 16-bit element in the two or four first source vectors by each signed 8-bit or 16-bit element in the one, two, or four second source vectors, widens each product to 32 bits or 64 bits, and destructively subtracts these values from the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The quad-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

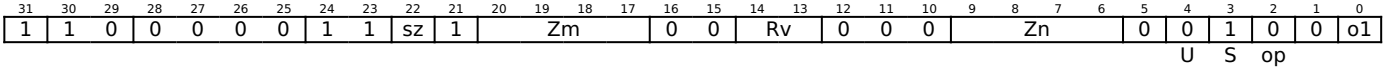
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA quad-vectors and Four ZA quad-vectors

Two ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

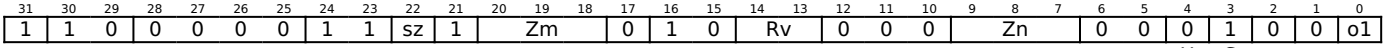
```
SMLSLL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, { <Zm1>.<Tb>-<Zm2>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
SMLSLL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, { <Zm1>.<Tb>-<Zm4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs1> Is the first vector select offset, encoded as "o1" field times 4.

<offs4> Is the fourth vector select offset, encoded as "o1" field times 4 plus 3.

<Zn1> For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1> For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = SInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

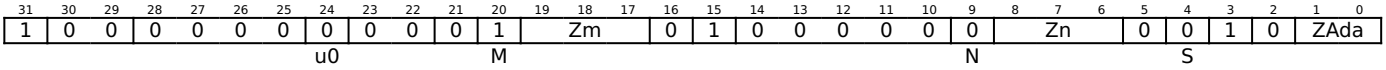
SMOP4A (2-way)

Signed 16-bit integer quarter-tile sum of outer products to 32-bit integer, accumulating

This instruction generates four independent quarter-tile signed integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 32-bit element ZA tile.
Each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 2$ sub-matrix of 16-bit signed values held in the half-vectors of the first source vectors by the $2 \times SVL_S \div 2$ sub-matrix of 16-bit signed values held in the half-vectors of the second source vectors. Each 32-bit container of the half-vectors in the first source vectors holds 2 elements of each row of a $SVL_S \div 2 \times 2$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 2 elements of each column of a $2 \times SVL_S \div 2$ sub-matrix. The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products are then destructively added to the 32-bit integer destination tile.
This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4)



Encoding

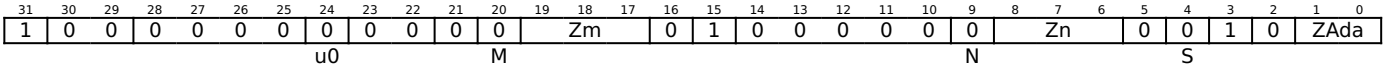
```
SMOP4A <ZAda>.S, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Single vectors

(FEAT_SME_MOP4)



Encoding

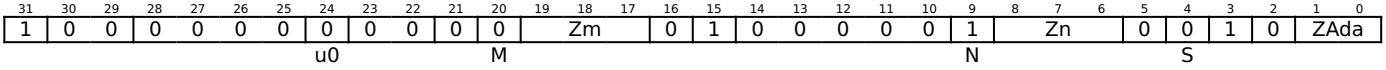
```
SMOP4A <ZAda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Multiple and single vectors

(FEAT_SME_MOP4)



Encoding

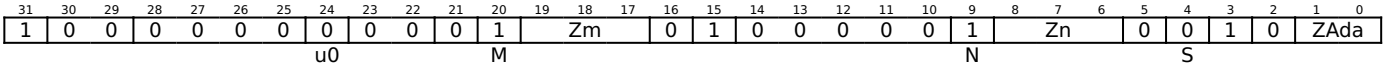
```
SMOP4A <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Multiple vectors

(FEAT_SME_MOP4)



Encoding

```
SMOP4A <ZAda>.S, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 1 do
                let op1elt : bits(esize DIV 2) = op1[(2*row_idx + k)*:(esize DIV 2)];
                let op2elt : bits(esize DIV 2) = op2[(2*col_idx + k)*:(esize DIV 2)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMOP4A (4-way)

Signed integer quarter-tile sum of outer products, accumulating

This instruction generates four independent quarter-tile signed integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 32-bit or 64-bit element ZA tile.
In case of the 8-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S÷2 \times 4$ sub-matrix of 8-bit signed values held in the half-vectors of the first source vectors by the $4 \times SVL_S÷2$ sub-matrix of 8-bit signed values held in the half-vectors of the second source vectors. Each 32-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_S÷2 \times 4$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_S÷2$ sub-matrix.
In case of the 16-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_D÷2 \times 4$ sub-matrix of 16-bit signed values held in the half-vectors of the first source vectors by the $4 \times SVL_D÷2$ sub-matrix of 16-bit signed values held in the half-vectors of the second source vectors. Each 64-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_D÷2 \times 4$ sub-matrix. Similarly, each 64-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_D÷2$ sub-matrix.
The resulting quarter-tile $SVL_S÷2 \times SVL_S÷2$ widened 32-bit integer sums of outer products in case of the 8-bit integer variant or $SVL_D÷2 \times SVL_D÷2$ widened 64-bit integer sums of outer products in case of the 16-bit integer variant are then destructively added to the 32-bit or 64-bit integer destination tile respectively.
This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 8 classes: 32-bit, single and multiple vectors , 32-bit, single vectors , 32-bit, multiple and single vectors , 32-bit, multiple vectors , 64-bit, single and multiple vectors , 64-bit, single vectors , 64-bit, multiple and single vectors and 64-bit, multiple vectors

32-bit, single and multiple vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	0	1	Zm			0	1	0	0	0	0	0	0	Zn			0	0	0	0	ZAda	
							u0			u1	M											N					S				

Encoding

```
SMOP4A <ZAda>.S, <Zn>.B, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

32-bit, single vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	0	0	Zm			0	1	0	0	0	0	0	0	Zn			0	0	0	0	ZAda	
							u0			u1	M											N					S				

Encoding

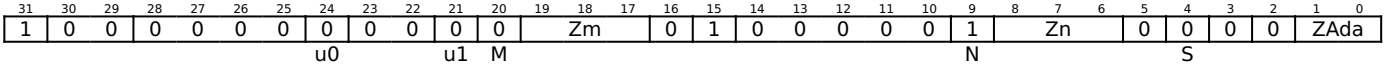
```
SMOP4A <ZAda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

32-bit, multiple and single vectors

(FEAT_SME_MOP4)



Encoding

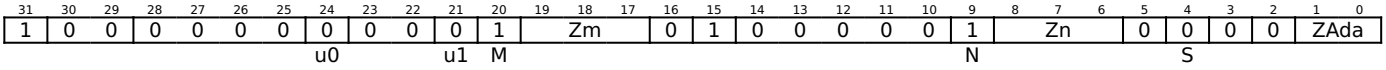
```
SMOP4A <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

32-bit, multiple vectors

(FEAT_SME_MOP4)



Encoding

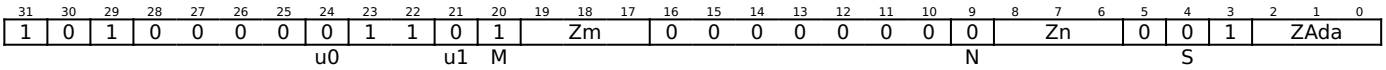
```
SMOP4A <ZAda>.S, { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

64-bit, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

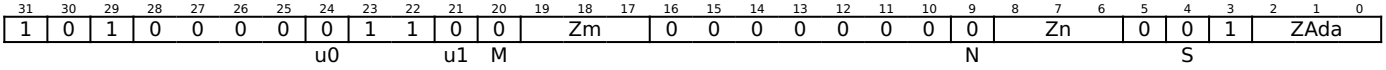
```
SMOP4A <ZAda>.D, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

64-bit, single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

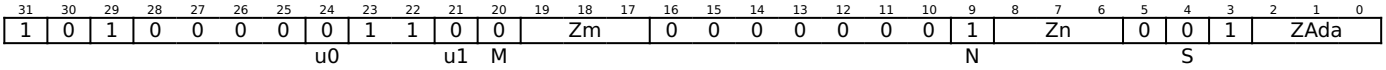
```
SMOP4A <ZAda>.D, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

64-bit, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

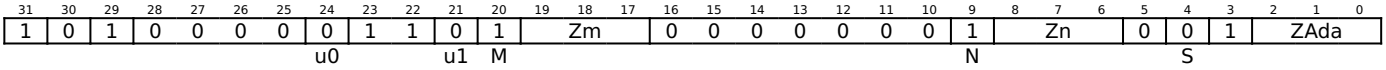
```
SMOP4A <ZAda>.D, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

64-bit, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

```
SMOP4A <ZAda>.D, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	For the "32-bit, multiple and single vectors", "32-bit, multiple vectors", "32-bit, single and multiple vectors", and "32-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit, multiple and single vectors", "64-bit, multiple vectors", "64-bit, single and multiple vectors", and "64-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.

<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 3 do
                let op1elt : bits(esize DIV 4) = op1[(4*row_idx + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = op2[(4*col_idx + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMOP4S (2-way)

Signed 16-bit integer quarter-tile sum of outer products to 32-bit integer, subtracting

This instruction generates four independent quarter-tile signed integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and subtracts the results from the corresponding elements of a 32-bit element ZA tile.

Each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 2$ sub-matrix of 16-bit signed values held in the half-vectors of the first source vectors by the $2 \times SVL_S \div 2$ sub-matrix of 16-bit signed values held in the half-vectors of the second source vectors. Each 32-bit container of the half-vectors in the first source vectors holds 2 elements of each row of a $SVL_S \div 2 \times 2$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 2 elements of each column of a $2 \times SVL_S \div 2$ sub-matrix. The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products are then destructively subtracted from the 32-bit integer destination tile.

This is equivalent to performing a 2-way dot product and subtract from each of the destination tile elements.

This instruction is unpredicated.

It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	0	1	Zm			0	1	0	0	0	0	0	0	Zn			0	1	1	0	ZAda	
							u0				M											N					S				

Encoding

```
SMOP4S <ZAda>.S, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Single vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	0	0	Zm			0	1	0	0	0	0	0	0	Zn			0	1	1	0	ZAda	
							u0				M											N					S				

Encoding

```
SMOP4S <ZAda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Multiple and single vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	0	0	Zm			0	1	0	0	0	0	0	1	Zn			0	1	1	0	ZAda	
							u0				M											N					S				

Encoding

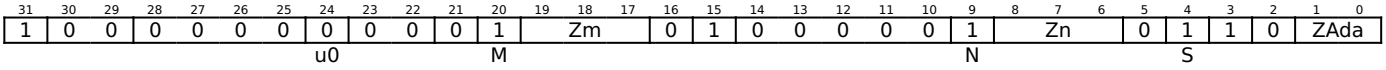
```
SMOP4S <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Multiple vectors

(FEAT_SME_MOP4)



Encoding

```
SMOP4S <ZAda>.S, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 1 do
                let op1elt : bits(esize DIV 2) = op1[(2*row_idx + k)*:(esize DIV 2)];
                let op2elt : bits(esize DIV 2) = op2[(2*col_idx + k)*:(esize DIV 2)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMOP4S (4-way)

Signed integer quarter-tile sum of outer products, subtracting

This instruction generates four independent quarter-tile signed integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and subtracts the results from the corresponding elements of a 32-bit or 64-bit element ZA tile.
In case of the 8-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 4$ sub-matrix of 8-bit signed values held in the half-vectors of the first source vectors by the $4 \times SVL_S \div 2$ sub-matrix of 8-bit signed values held in the half-vectors of the second source vectors. Each 32-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_S \div 2 \times 4$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_S \div 2$ sub-matrix.
In case of the 16-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_D \div 2 \times 4$ sub-matrix of 16-bit signed values held in the half-vectors of the first source vectors by the $4 \times SVL_D \div 2$ sub-matrix of 16-bit signed values held in the half-vectors of the second source vectors. Each 64-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_D \div 2 \times 4$ sub-matrix. Similarly, each 64-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_D \div 2$ sub-matrix.
The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products in case of the 8-bit integer variant or $SVL_D \div 2 \times SVL_D \div 2$ widened 64-bit integer sums of outer products in case of the 16-bit integer variant are then destructively subtracted from the 32-bit or 64-bit integer destination tile respectively.
This is equivalent to performing a 4-way dot product and subtract from each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 8 classes: 32-bit, single and multiple vectors , 32-bit, single vectors , 32-bit, multiple and single vectors , 32-bit, multiple vectors , 64-bit, single and multiple vectors , 64-bit, single vectors , 64-bit, multiple and single vectors and 64-bit, multiple vectors

32-bit, single and multiple vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19–18	17	16	15	14	13	12	11	10	9	8–7	6	5	4	3	2	1–0
1	0	0	0	0	0	0	0	0	0	0	1	Zm		0	1	0	0	0	0	0	0	Zn		0	1	0	0	ZAda
							u0			u1	M										N				S			

Encoding

```
SMOP4S <ZAda>.S, <Zn>.B, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

32-bit, single vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19–18	17	16	15	14	13	12	11	10	9	8–7	6	5	4	3	2	1–0
1	0	0	0	0	0	0	0	0	0	0	0	Zm		0	1	0	0	0	0	0	0	Zn		0	1	0	0	ZAda
							u0			u1	M										N				S			

Encoding

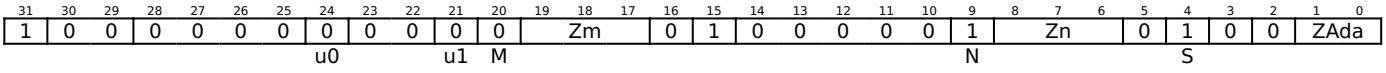
```
SMOP4S <ZAda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

32-bit, multiple and single vectors

(FEAT_SME_MOP4)



Encoding

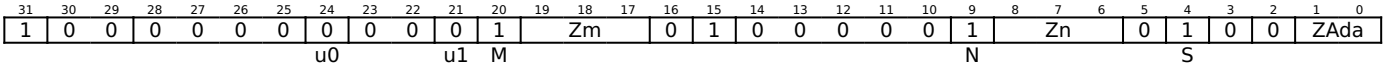
```
SMOP4S <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

32-bit, multiple vectors

(FEAT_SME_MOP4)



Encoding

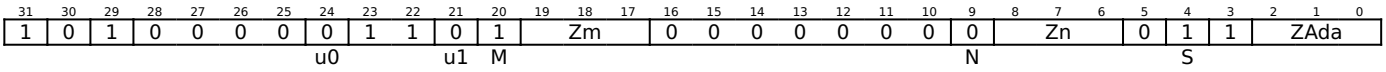
```
SMOP4S <ZAda>.S, { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

64-bit, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

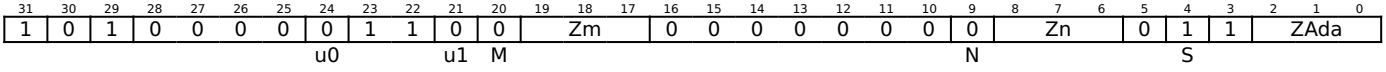
```
SMOP4S <ZAda>.D, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

64-bit, single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

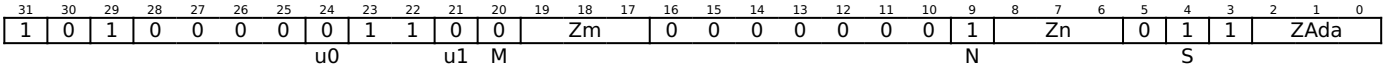
```
SMOP4S <ZAda>.D, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

64-bit, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

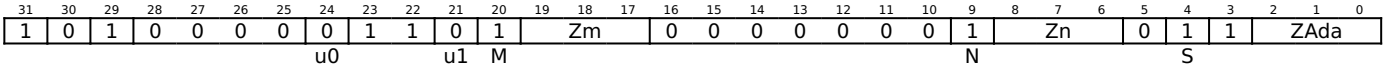
```
SMOP4S <ZAda>.D, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

64-bit, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

```
SMOP4S <ZAda>.D, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	For the "32-bit, multiple and single vectors", "32-bit, multiple vectors", "32-bit, single and multiple vectors", and "32-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit, multiple and single vectors", "64-bit, multiple vectors", "64-bit, single and multiple vectors", and "64-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.

<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 3 do
                let op1elt : bits(esize DIV 4) = op1[(4*row_idx + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = op2[(4*col_idx + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMOPA (2-way)

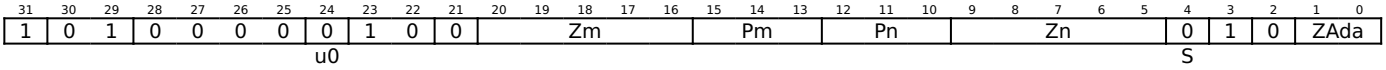
Signed 16-bit integer sum of outer products to 32-bit integer, accumulating

This instruction works with a 32-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. The first source holds $SVL_S \times 2$ sub-matrix of signed 16-bit integer values, and the second source holds $2 \times SVL_S$ sub-matrix of signed 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When a 16-bit source element is inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer sum of outer products is then destructively added to the 32-bit integer destination tile. This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.
Each 32-bit container of the first source vector holds 2 consecutive column elements of each row of a $SVL_S \times 2$ sub-matrix, and each 32-bit container of the second source vector holds 2 consecutive row elements of each column of a $2 \times SVL_S$ sub-matrix.

SME2

(FEAT_SME2)



Encoding

```
SMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 1 do
            if (ActivePredicateElement{PL}(mask1, 2*row + k, esize DIV 2) &&
                  ActivePredicateElement{PL}(mask2, 2*col + k, esize DIV 2)) then
                let op1elt : bits(esize DIV 2) = operand1[(2*row + k)*:(esize DIV 2)];
                let op2elt : bits(esize DIV 2) = operand2[(2*col + k)*:(esize DIV 2)];
                let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum + prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMOPA (4-way)

Signed integer sum of outer products, accumulating

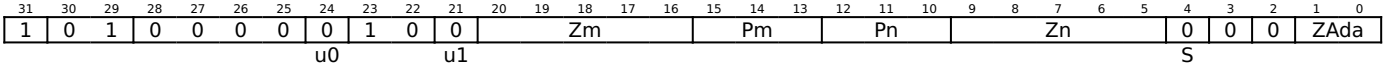
The 8-bit integer variant works with a 32-bit element ZA tile.
The 16-bit integer variant works with a 64-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. In case of the 8-bit integer variant, the first source holds $SVL_S \times 4$ sub-matrix of signed 8-bit integer values, and the second source holds $4 \times SVL_S$ sub-matrix of signed 8-bit integer values. In case of the 16-bit integer variant, the first source holds $SVL_D \times 4$ sub-matrix of signed 16-bit integer values, and the second source holds $4 \times SVL_D$ sub-matrix of signed 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When an 8-bit source element in case of 8-bit integer variant or a 16-bit source element in case of 16-bit integer variant is Inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer or $SVL_D \times SVL_D$ widened 64-bit integer sum of outer products is then destructively added to the 32-bit integer or 64-bit integer destination tile, respectively for 8-bit integer and 16-bit integer instruction variants. This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.
In case of the 8-bit integer variant, each 32-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_S \times 4$ sub-matrix, and each 32-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_S$ sub-matrix. In case of the 16-bit integer variant, each 64-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_D \times 4$ sub-matrix, and each 64-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_D$ sub-matrix.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SME)



Encoding

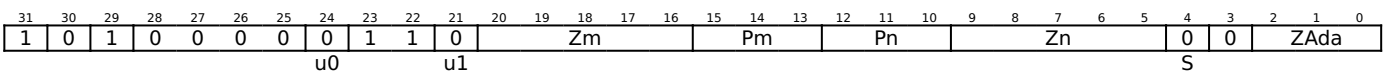
```
SMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

64-bit

(FEAT_SME_I16I64)



Encoding

```
SMOPA <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.

<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 3 do
            if (ActivePredicateElement{PL}(mask1, 4*row + k, esize DIV 4) &&
                  ActivePredicateElement{PL}(mask2, 4*col + k, esize DIV 4)) then
                let op1elt : bits(esize DIV 4) = operand1[(4*row + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = operand2[(4*col + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum + prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMOPS (2-way)

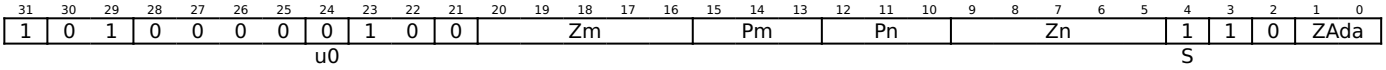
Signed 16-bit integer sum of outer products to 32-bit integer, subtracting

This instruction works with a 32-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. The first source holds $SVL_S \times 2$ sub-matrix of signed 16-bit integer values, and the second source holds $2 \times SVL_S$ sub-matrix of signed 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When a 16-bit source element is inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer sum of outer products is then destructively subtracted from the 32-bit integer destination tile.
This is equivalent to performing a 2-way dot product and subtract from each of the destination tile elements.
Each 32-bit container of the first source vector holds 2 consecutive column elements of each row of a $SVL_S \times 2$ sub-matrix, and each 32-bit container of the second source vector holds 2 consecutive row elements of each column of a $2 \times SVL_S$ sub-matrix.

SME2

(FEAT_SME2)



Encoding

```
SMOPS <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 1 do
            if (ActivePredicateElement{PL}(mask1, 2*row + k, esize DIV 2) &&
                  ActivePredicateElement{PL}(mask2, 2*col + k, esize DIV 2)) then
                let op1elt : bits(esize DIV 2) = operand1[(2*row + k)*:(esize DIV 2)];
                let op2elt : bits(esize DIV 2) = operand2[(2*col + k)*:(esize DIV 2)];
                let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum - prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SMOPS (4-way)

Signed integer sum of outer products, subtracting

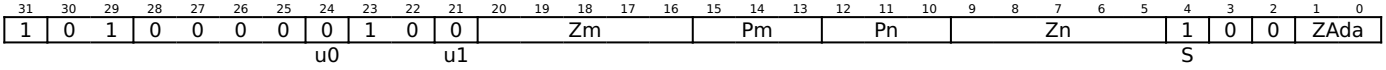
The 8-bit integer variant works with a 32-bit element ZA tile.
The 16-bit integer variant works with a 64-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. In case of the 8-bit integer variant, the first source holds $SVL_S \times 4$ sub-matrix of signed 8-bit integer values, and the second source holds $4 \times SVL_S$ sub-matrix of signed 8-bit integer values. In case of the 16-bit integer variant, the first source holds $SVL_D \times 4$ sub-matrix of signed 16-bit integer values, and the second source holds $4 \times SVL_D$ sub-matrix of signed 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When an 8-bit source element in case of 8-bit integer variant or a 16-bit source element in case of 16-bit integer variant is Inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer or $SVL_D \times SVL_D$ widened 64-bit integer sum of outer products is then destructively subtracted from the 32-bit integer or 64-bit integer destination tile, respectively for 8-bit integer and 16-bit integer instruction variants. This is equivalent to performing a 4-way dot product and subtract from each of the destination tile elements.
In case of the 8-bit integer variant, each 32-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_S \times 4$ sub-matrix, and each 32-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_S$ sub-matrix. In case of the 16-bit integer variant, each 64-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_D \times 4$ sub-matrix, and each 64-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_D$ sub-matrix.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SME)



Encoding

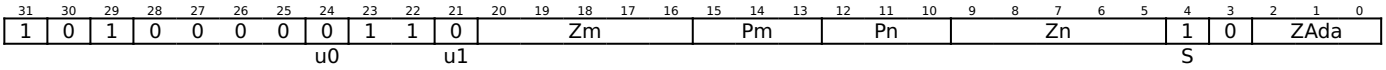
```
SMOPS <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

64-bit

(FEAT_SME_I16I64)



Encoding

```
SMOPS <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.

<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 3 do
            if (ActivePredicateElement{PL}(mask1, 4*row + k, esize DIV 4) &&
                  ActivePredicateElement{PL}(mask2, 4*col + k, esize DIV 4)) then
                let op1elt : bits(esize DIV 4) = operand1[(4*row + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = operand2[(4*col + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum - prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQCVT (four registers)

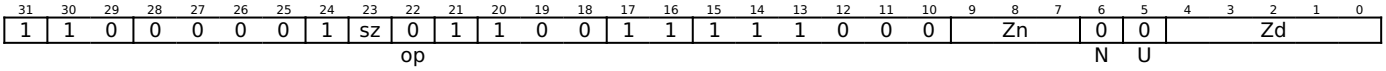
Multi-vector signed saturating extract narrow

This instruction saturates the signed integer value in each element of the four source vectors to quarter the original source element width, and places the results in the quarter-width destination elements.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQCVT <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(sz);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	B
1	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	S
1	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for r = 0 to 3 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : integer = SInt(operand[e*:(4 * esize)]);
        result[(r*elements + e)*:esize] = SignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQCVT (two registers)

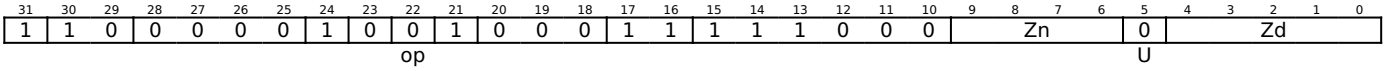
Multi-vector signed 32-bit integer saturating extract narrow to 16-bit integer

This instruction saturates the signed 32-bit integer value in each element of the two source vectors to half the original source element width, and places the 16-bit results in the half-width destination elements.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQCVT <Zd>.H, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for r = 0 to 1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : integer = SInt(operand[e*:(2 * esize)]);
        result[(r*elements + e)*:esize] = SignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQCVTN

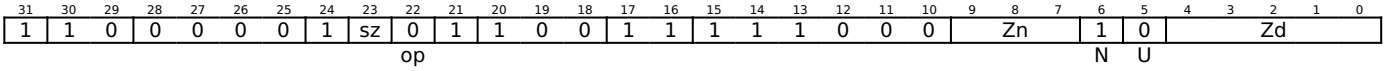
Multi-vector signed saturating extract narrow to interleaved integer

This instruction saturates the signed integer value in each element of the four source vectors to quarter the original source element width, and places the four-way interleaved results in the quarter-width destination elements.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQCVTN <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(sz);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	B
1	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	S
1	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 3 do
        let operand : bits(VL) = Z{VL}(n+i);
        let element : integer = SInt(operand[e*:(4 * esize)]);
        result[(4*e + i)*:esize] = SignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQCVTU (four registers)

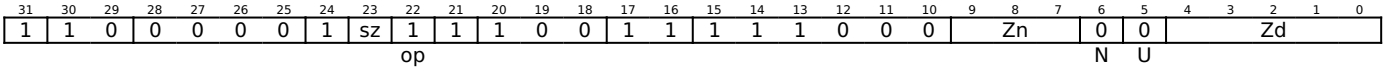
Multi-vector signed saturating extract narrow to unsigned integer

This instruction saturates the signed integer value in each element of the four source vectors to unsigned integer value that is quarter the original source element width, and places the results in the quarter-width destination elements.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQCVTU <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(sz);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	B
1	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	S
1	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for r = 0 to 3 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : integer = SInt(operand[e*:(4 * esize)]);
        result[(r*elements + e)*:esize] = UnsignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQCVTU (two registers)

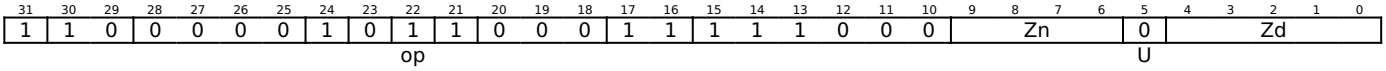
Multi-vector signed 32-bit integer saturating extract narrow to unsigned 16-bit integer

This instruction saturates the signed 32-bit integer value in each element of the two source vectors to unsigned integer value that is half the original source element width, and places the 16-bit results in the half-width destination elements.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQCVTU <Zd>.H, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for r = 0 to 1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : integer = SInt(operand[e*:(2 * esize)]);
        result[(r*elements + e)*:esize] = UnsignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQCVTUN

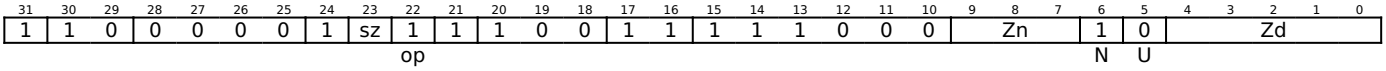
Multi-vector signed saturating extract narrow to interleaved unsigned integer

This instruction saturates the signed integer value in each element of the four source vectors to unsigned integer value that is quarter the original source element width, and places the four-way interleaved results in the quarter-width destination elements.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQCVTUN <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(sz);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	B
1	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	S
1	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 3 do
        let operand : bits(VL) = Z{VL}(n+i);
        let element : integer = SInt(operand[e*:(4 * esize)]);
        result[(4*e + i)*:esize] = UnsignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMULH (multiple and single vector)

Multi-vector signed saturating doubling multiply high by vector

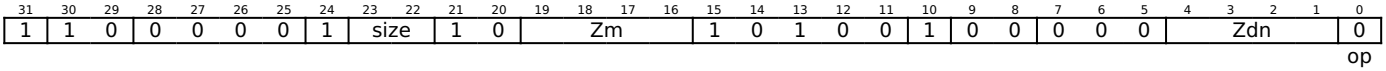
This instruction multiplies the signed elements of the two or four first source vectors by the corresponding elements of the second source vector, and then doubles the result and destructively places the most significant half of the result in the corresponding elements of the two or four first source vectors. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

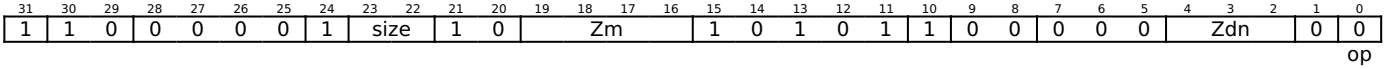
```
SQDMULH { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
SQDMULH { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : integer = SInt(operand1[e*:esize]);
        let element2 : integer = SInt(operand2[e*:esize]);
        let res : integer = 2 * element1 * element2;
        results[[r]][e*:esize] = SignedSat{esize}(res >> esize);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQDMULH (multiple vectors)

Multi-vector signed saturating doubling multiply high

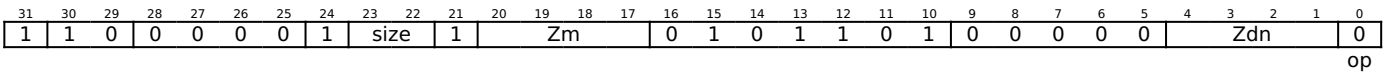
This instruction multiplies the corresponding signed elements of the two or four first and second source vectors, and then doubles the result and destructively places the most significant half of the result in the corresponding elements of the two or four first source vectors. Each result element is saturated to the N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

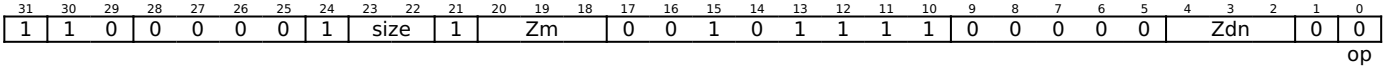
```
SQDMULH { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
SQDMULH { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : integer = SInt(operand1[e*:esize]);
        let element2 : integer = SInt(operand2[e*:esize]);
        let res : integer = 2 * element1 * element2;
        results[[r]][e*:esize] = SignedSat{esize}(res >> esize);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

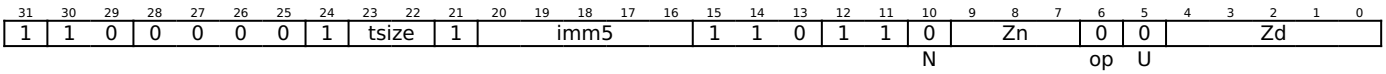
SQRSHR (four registers)

Multi-vector signed saturating rounding shift right narrow by immediate

This instruction shifts right by an immediate value the signed integer value in each element of the four source vectors, and places the rounded results in the quarter-width destination elements. Each result element is saturated to the quarter-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The immediate shift amount is an unsigned value in the range 1 to number of bits per source element.
This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQRSHR <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if tsize == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
let shift : integer = 8 * esize - UInt(tsize::imm5);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsize”:

tsize	<T>
00	RESERVED
01	B
1x	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “tsize”:

tsize	<Tb>
00	RESERVED
01	S
1x	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

<const> Is the immediate shift amount, in the range 1 to number of bits per source element, encoded in "tsize:imm5".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for r = 0 to 3 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(4 * esize) = operand[e*:(4 * esize)];
        let res : integer = (SInt(element) + 1 << (shift-1)) >> shift;
        result[(r*elements + e)*:esize] = SignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRSHR (two registers)

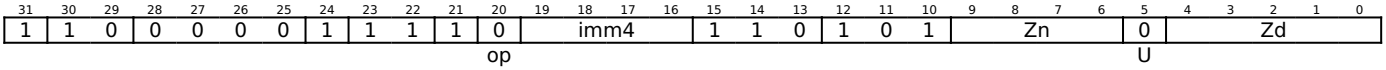
Multi-vector signed 32-bit integer saturating rounding shift right narrow by immediate to 16-bit integer

This instruction shifts right by an immediate value the signed integer value in each element of the two source vectors, and places the rounded results in the half-width destination elements. Each result element is saturated to the half-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The immediate shift amount is an unsigned value in the range 1 to 16.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQRSHR <Zd>.H, { <Zn1>.S-<Zn2>.S }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = esize - UInt(imm4);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<const>	Is the immediate shift amount, in the range 1 to 16, encoded as 16 minus "imm4".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for r = 0 to 1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(2 * esize) = operand[e*:(2 * esize)];
        let res : integer = (SInt(element) + 1 << (shift-1)) >> shift;
        result[(r*elements + e)*:esize] = SignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

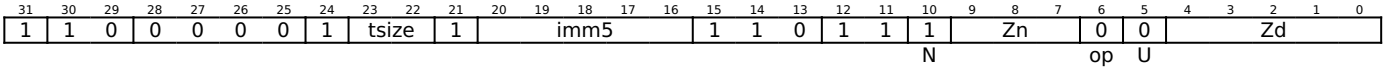
SQRSHRN

Multi-vector signed saturating rounding shift right narrow by immediate to interleaved integer

This instruction shifts right by an immediate value the signed integer value in each element of the four source vectors, and places the four-way interleaved rounded results in the quarter-width destination elements. Each result element is saturated to the quarter-width N-bit element's signed integer range $-2^{(N-1)}$ to $(2^{(N-1)})-1$. The immediate shift amount is an unsigned value in the range 1 to number of bits per source element.
This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQRSHRN <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if tsize == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
let shift : integer = 8 * esize - UInt(tsize::imm5);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsize”:

tsize	<T>
00	RESERVED
01	B
1x	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “tsize”:

tsize	<Tb>
00	RESERVED
01	S
1x	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

<const> Is the immediate shift amount, in the range 1 to number of bits per source element, encoded in "tsize:imm5".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 3 do
        let operand : bits(VL) = Z{VL}(n+i);
        let element : bits(4 * esize) = operand[e*:(4 * esize)];
        let res : integer = (SInt(element) + 1 << (shift-1)) >> shift;
        result[(4*e + i)*:esize] = SignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRSHRU (four registers)

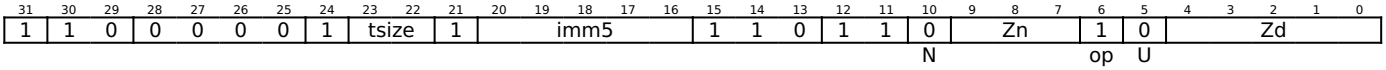
Multi-vector signed saturating rounding shift right narrow by immediate to unsigned integer

This instruction shifts right by an immediate value the signed integer value in each element of the four source vectors, and places the rounded results in the quarter-width destination elements. Each result element is saturated to the quarter-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per source element.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQRSHRU <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if tsize == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
let shift : integer = 8 * esize - UInt(tsize::imm5);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsize”:

tsize	<T>
00	RESERVED
01	B
1x	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “tsize”:

tsize	<Tb>
00	RESERVED
01	S
1x	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

<const> Is the immediate shift amount, in the range 1 to number of bits per source element, encoded in "tsize:imm5".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for r = 0 to 3 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(4 * esize) = operand[e*:(4 * esize)];
        let res : integer = (SInt(element) + 1 << (shift-1)) >> shift;
        result[(r*elements + e)*:esize] = UnsignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRSHRU (two registers)

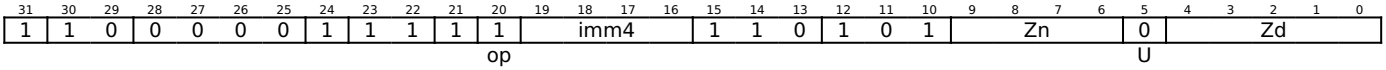
Multi-vector signed 32-bit integer saturating rounding shift right narrow by immediate to unsigned 16-bit integer

This instruction shifts right by an immediate value the signed integer value in each element of the two source vectors, and places the rounded results in the half-width destination elements. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to 16.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQRSHRU <Zd>.H, { <Zn1>.S-<Zn2>.S }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = esize - UInt(imm4);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<const>	Is the immediate shift amount, in the range 1 to 16, encoded as 16 minus "imm4".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for r = 0 to 1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(2 * esize) = operand[e*:(2 * esize)];
        let res : integer = (SInt(element) + 1 << (shift-1)) >> shift;
        result[(r*elements + e)*:esize] = UnsignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SQRSHRUN

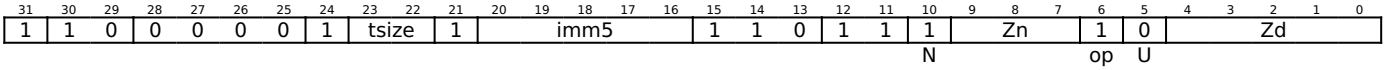
Multi-vector signed saturating rounding shift right narrow by immediate to interleaved unsigned integer

This instruction shifts right by an immediate value the signed integer value in each element of the four source vectors, and places the four-way interleaved rounded results in the quarter-width destination elements. Each result element is saturated to the quarter-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per source element.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SQRSHRUN <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if tsize == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
let shift : integer = 8 * esize - UInt(tsize::imm5);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsize”:

tsize	<T>
00	RESERVED
01	B
1x	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “tsize”:

tsize	<Tb>
00	RESERVED
01	S
1x	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

<const> Is the immediate shift amount, in the range 1 to number of bits per source element, encoded in "tsize:imm5".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 3 do
        let operand : bits(VL) = Z{VL}(n+i);
        let element : bits(4 * esize) = operand[e*:(4 * esize)];
        let res : integer = (SInt(element) + 1 << (shift-1)) >> shift;
        result[(4*e + i)*:esize] = UnsignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SRSHL (multiple and single vector)

Multi-vector signed rounding shift left by vector

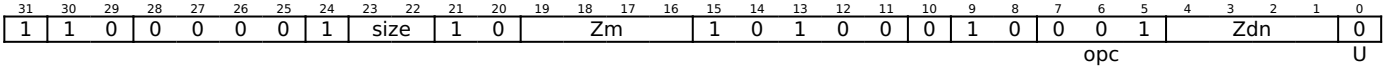
This instruction shifts the signed elements of the two or four first source vectors by the corresponding elements of the second source vector, and destructively places the rounded results in the corresponding elements of the two or four first source vectors. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

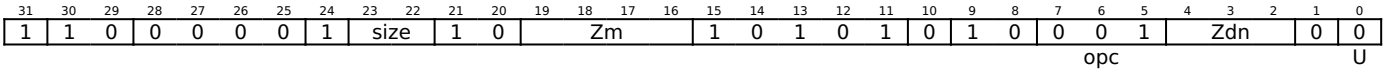
```
SRSHL { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
SRSHL { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element : integer = SInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + 1 << (shift - 1)) >> shift;
        end;
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SRSHL (multiple vectors)

Multi-vector signed rounding shift left

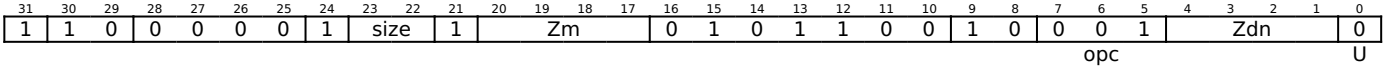
This instruction shifts the signed elements of the two or four first source vectors by the corresponding elements of the two or four second source vectors and destructively places the rounded results in the corresponding elements of the two or four first source vectors. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

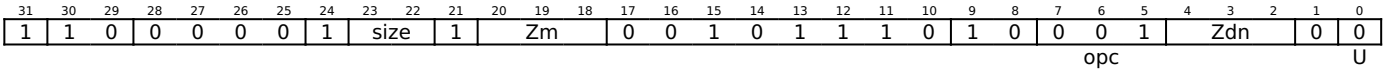
```
SRSHL { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
SRSHL { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element : integer = SInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + 1 << (shift - 1)) >> shift;
        end;
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1B (scalar plus immediate, strided registers)

Contiguous store of bytes from multiple strided vectors (immediate index)

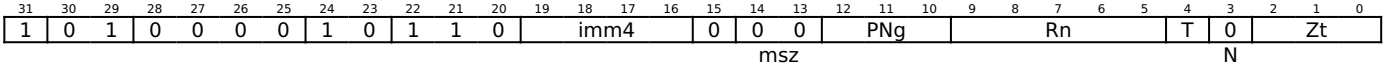
This instruction performs a contiguous store of bytes from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

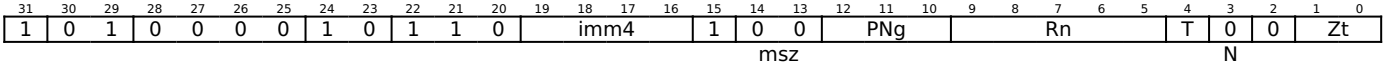
```
ST1B { <Zt1>.B, <Zt2>.B }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
ST1B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1B (scalar plus scalar, strided registers)

Contiguous store of bytes from multiple strided vectors (scalar index)

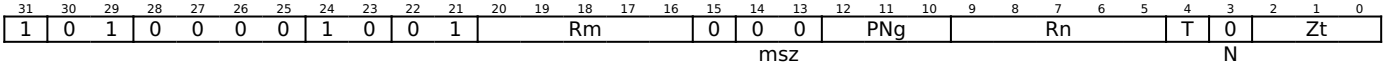
This instruction performs a contiguous store of bytes from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

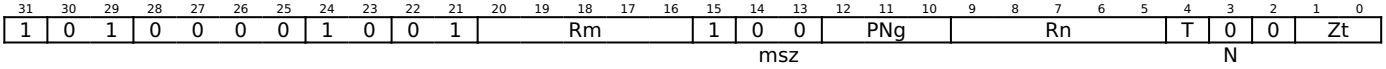
```
ST1B { <Zt1>.B, <Zt2>.B }, <PNg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 8;
```

Four registers

(FEAT_SME2)



Encoding

```
ST1B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <PNg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 8;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

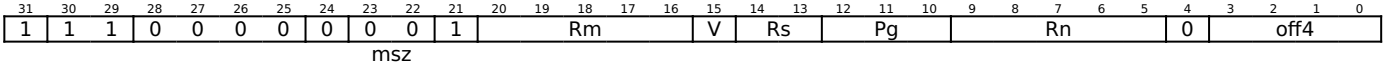
ST1B (scalar plus scalar, tile slice)

Contiguous store of bytes from 8-bit element ZA tile slice

This instruction performs a contiguous store of bytes from an 8-bit element ZA tile slice. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of 8-bit elements in a vector. The immediate offset is in the range 0 to 15. The memory address is generated by a 64-bit scalar base and an optional 64-bit scalar offset that is added to the base address. Inactive elements are not written to memory.

SME

(FEAT_SME)



Encoding

```
ST1B { ZA0<HV>.B[<Ws>, <offs>] }, <Pg>, [<Xn|SP>{, <Xm>}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('0'::Pg);
let s : integer = UInt('011'::Rs);
let t : integer = 0;
let offset : integer = UInt(off4);
let esize : integer{} = 8;
let vertical : boolean = V == '1';
```

Assembler Symbols

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> Is the slice index offset, in the range 0 to 15, encoded in the "off4" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mask : bits(PL) = P{}(g);
var moffs : bits(64) = X{}(m);
let index : bits(32) = X{}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var src : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    if (AnyActiveElement{PL}(mask, esize) ||
          ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE)) then
        CheckSPAlignment();
    end;
    base = SP{64}();
else
    base = X{64}(n);
end;

src = ZAslice{VL}(t, esize, vertical, slice);
for e = 0 to dim-1 do
    addr = AddressAdd(base, UInt(moffs) * mbytes, accdesc);
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    moffs = moffs + 1;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1D (scalar plus immediate, strided registers)

Contiguous store of doublewords from multiple strided vectors (immediate index)

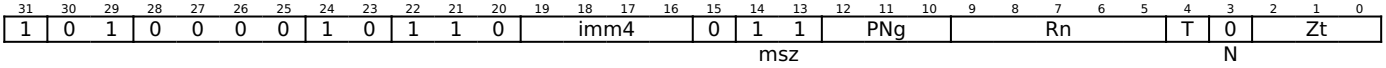
This instruction performs a contiguous store of doublewords from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

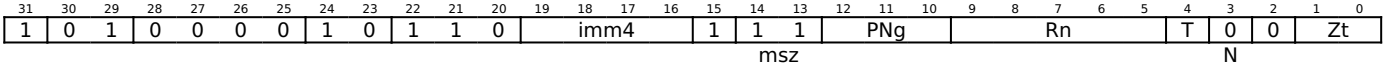
```
ST1D { <Zt1>.D, <Zt2>.D }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
ST1D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt".
	For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt".
	For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.
	For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1D (scalar plus scalar, strided registers)

Contiguous store of doublewords from multiple strided vectors (scalar index)

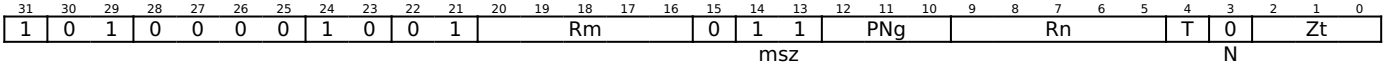
This instruction performs a contiguous store of doublewords from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

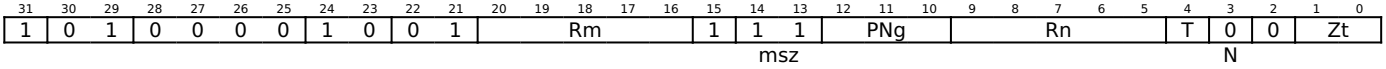
```
ST1D { <Zt1>.D, <Zt2>.D }, <PNg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 64;
```

Four registers

(FEAT_SME2)



Encoding

```
ST1D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <PNg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 64;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

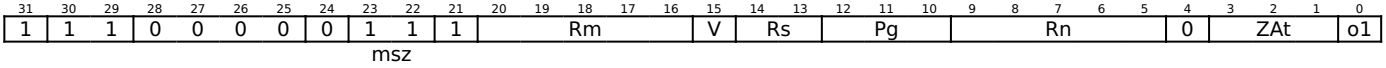
ST1D (scalar plus scalar, tile slice)

Contiguous store of doublewords from 64-bit element ZA tile slice

This instruction performs a contiguous store of doublewords from a 64-bit element ZA tile slice. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of 64-bit elements in a vector. The immediate offset is in the range 0 to 1. The memory address is generated by a 64-bit scalar base and an optional 64-bit scalar offset that is multiplied by 8 and added to the base address. Inactive elements are not written to memory.

SME

(FEAT_SME)



Encoding

```
ST1D { <ZAt><HV>.D[<Ws>, <offs>] }, <Pg>, [<Xn|SP>{, <Xm>, LSL #3}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('0'::Pg);
let s : integer = UInt('011'::Rs);
let t : integer = UInt(ZAt);
let offset : integer = UInt(o1);
let esize : integer{} = 64;
let vertical : boolean = V == '1';
```

Assembler Symbols

<ZAt> Is the name of the ZA tile ZA0-ZA7 to be accessed, encoded in the "ZAt" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> Is the slice index offset, in the range 0 to 1, encoded in the "o1" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mask : bits(PL) = P{}(g);
var moffs : bits(64) = X{}(m);
let index : bits(32) = X{}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var src : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    if (AnyActiveElement{PL}(mask, esize) ||
          ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE)) then
        CheckSPAlignment();
    end;
    base = SP{64}();
else
    base = X{64}(n);
end;

src = ZAslice{VL}(t, esize, vertical, slice);
for e = 0 to dim-1 do
    addr = AddressAdd(base, UInt(moffs) * mbytes, accdesc);
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    moffs = moffs + 1;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1H (scalar plus immediate, strided registers)

Contiguous store of halfwords from multiple strided vectors (immediate index)

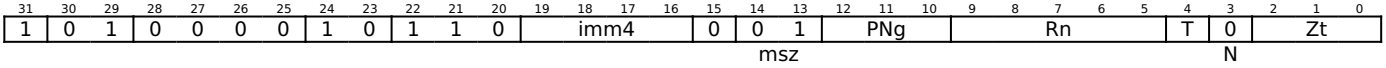
This instruction performs a contiguous store of halfwords from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

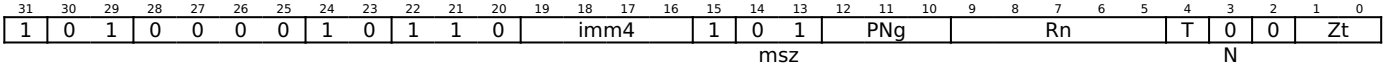
```
ST1H { <Zt1>.H, <Zt2>.H }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
ST1H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt".
	For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt".
	For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2.
	For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1H (scalar plus scalar, strided registers)

Contiguous store of halfwords from multiple strided vectors (scalar index)

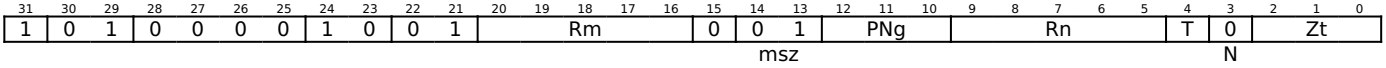
This instruction performs a contiguous store of halfwords from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

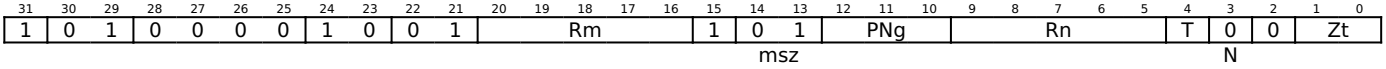
```
ST1H { <Zt1>.H, <Zt2>.H }, <PNg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 16;
```

Four registers

(FEAT_SME2)



Encoding

```
ST1H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <PNg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 16;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

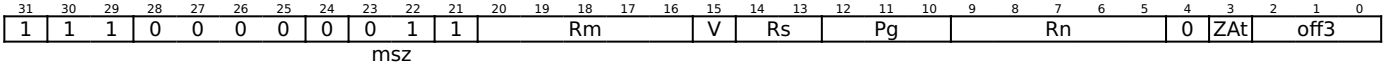
ST1H (scalar plus scalar, tile slice)

Contiguous store of halfwords from 16-bit element ZA tile slice

This instruction performs a contiguous store of halfwords from a 16-bit element ZA tile slice. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of 16-bit elements in a vector. The immediate offset is in the range 0 to 7. The memory address is generated by a 64-bit scalar base and an optional 64-bit scalar offset that is multiplied by 2 and added to the base address. Inactive elements are not written to memory.

SME

(FEAT_SME)



Encoding

```
ST1H { <ZAt><HV>.H[<Ws>, <offs>] }, <Pg>, [<Xn|SP>{, <Xm>, LSL #1}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('0'::Pg);
let s : integer = UInt('011'::Rs);
let t : integer = UInt(ZAt);
let offset : integer = UInt(off3);
let esize : integer{} = 16;
let vertical : boolean = V == '1';
```

Assembler Symbols

<ZAt> Is the name of the ZA tile ZA0-ZA1 to be accessed, encoded in the "ZAt" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> Is the slice index offset, in the range 0 to 7, encoded in the "off3" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mask : bits(PL) = P{}(g);
var moffs : bits(64) = X{}(m);
let index : bits(32) = X{}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var src : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    if (AnyActiveElement{PL}(mask, esize) ||
          ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE)) then
        CheckSPAlignment();
    end;
    base = SP{64}();
else
    base = X{64}(n);
end;

src = ZAslice{VL}(t, esize, vertical, slice);
for e = 0 to dim-1 do
    addr = AddressAdd(base, UInt(moffs) * mbytes, accdesc);
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    moffs = moffs + 1;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

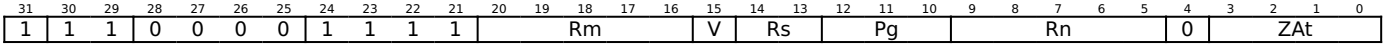
ST1Q

Contiguous store of quadwords from 128-bit element ZA tile slice

This instruction performs a contiguous store of quadwords from a 128-bit element ZA tile slice. The slice number in the tile is selected by the slice index register, modulo the number of 128-bit elements in a Streaming SVE vector. The memory address is generated by scalar base and optional scalar offset that is multiplied by 16 and added to the base address. Inactive elements are not written to memory.

SME

(FEAT_SME)



Encoding

```
ST1Q { <ZAt><HV>.Q[<Ws>, <offs>] }, <Pg>, [<Xn|SP>{, <Xm>, LSL #4}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('0'::Pg);
let s : integer = UInt('011'::Rs);
let t : integer = UInt(ZAt);
let offset : integer = 0;
let esize : integer{} = 128;
let vertical : boolean = V == '1';
```

Assembler Symbols

<ZAt> Is the name of the ZA tile ZA0-ZA15 to be accessed, encoded in the "ZAt" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> Is the slice index offset, with implicit value 0.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mask : bits(PL) = P{}(g);
var moffs : bits(64) = X{}(m);
let index : bits(32) = X{}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var src : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    if (AnyActiveElement{PL}(mask, esize) ||
          ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE)) then
        CheckSPAlignment();
    end;
    base = SP{64}();
else
    base = X{64}(n);
end;

src = ZAslice{VL}(t, esize, vertical, slice);
for e = 0 to dim-1 do
    addr = AddressAdd(base, UInt(moffs) * mbytes, accdesc);
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    moffs = moffs + 1;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1W (scalar plus immediate, strided registers)

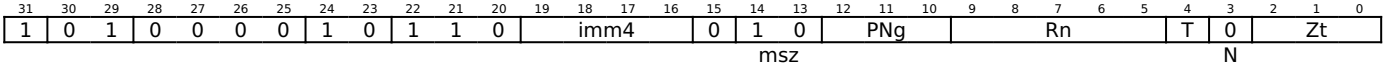
Contiguous store of words from multiple strided vectors (immediate index)

This instruction performs a contiguous store of words from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address. Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

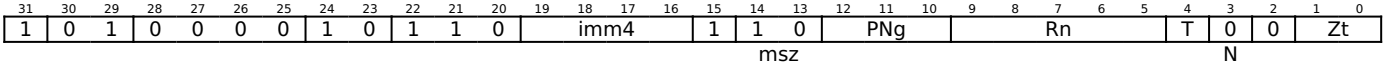
```
ST1W { <Zt1>.S, <Zt2>.S }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
ST1W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ST1W (scalar plus scalar, strided registers)

Contiguous store of words from multiple strided vectors (scalar index)

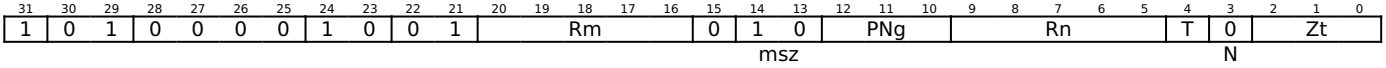
This instruction performs a contiguous store of words from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

Inactive elements are not written to memory.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

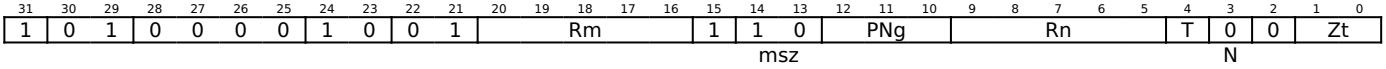
```
ST1W { <Zt1>.S, <Zt2>.S }, <PNg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 32;
```

Four registers

(FEAT_SME2)



Encoding

```
ST1W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <PNg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 32;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

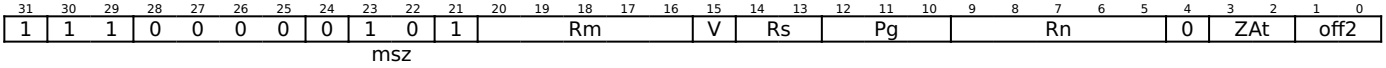
ST1W (scalar plus scalar, tile slice)

Contiguous store of words from 32-bit element ZA tile slice

This instruction performs a contiguous store of words from a 32-bit element ZA tile slice. The slice number within the tile is selected by the sum of the slice index register and immediate offset, modulo the number of 32-bit elements in a vector. The immediate offset is in the range 0 to 3. The memory address is generated by a 64-bit scalar base and an optional 64-bit scalar offset that is multiplied by 4 and added to the base address. Inactive elements are not written to memory.

SME

(FEAT_SME)



Encoding

```
ST1W { <ZAt><HV>.S[<Ws>, <offs>] }, <Pg>, [<Xn|SP>{, <Xm>, LSL #2}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('0'::Pg);
let s : integer = UInt('011'::Rs);
let t : integer = UInt(ZAt);
let offset : integer = UInt(off2);
let esize : integer{} = 32;
let vertical : boolean = V == '1';
```

Assembler Symbols

<ZAt> Is the name of the ZA tile ZA0-ZA3 to be accessed, encoded in the "ZAt" field.

<HV> Is the horizontal or vertical slice indicator, encoded in “V”:

V	<HV>
0	H
1	V

<Ws> Is the 32-bit name of the slice index register W12-W15, encoded as "Rs" plus 12.

<offs> Is the slice index offset, in the range 0 to 3, encoded in the "off2" field.

<Pg> Is the name of the governing scalable predicate register P0-P7, encoded in the "Pg" field.

<Xn|SP> Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

<Xm> Is the optional 64-bit name of the general-purpose offset register, defaulting to XZR, encoded in the "Rm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer = VL DIV esize;
var base : bits(64);
var addr : bits(64);
let mask : bits(PL) = P{}(g);
var moffs : bits(64) = X{}(m);
let index : bits(32) = X{}(s);
let slice : integer = (UInt(index) + offset) MOD dim;
var src : bits(VL);
let mbytes : integer{} = esize DIV 8;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if n == 31 then
    if (AnyActiveElement{PL}(mask, esize) ||
          ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE)) then
        CheckSPAlignment();
    end;
    base = SP{64}();
else
    base = X{64}(n);
end;

src = ZAslice{VL}(t, esize, vertical, slice);
for e = 0 to dim-1 do
    addr = AddressAdd(base, UInt(moffs) * mbytes, accdesc);
    if ActivePredicateElement{PL}(mask, e, esize) then
        Mem{esize}(addr, accdesc) = src[e*:esize];
    end;
    moffs = moffs + 1;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STMOPA (2-way)

Signed 16-bit integer sparse sum of outer products to 32-bit integer, accumulating

This instruction generates signed integer sum of outer products by multiplying the 2-in-4 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

The sum of outer products is generated by multiplying the selected 2-in-4 16-bit signed values from each overlapping 32-bit containers of the two $SVL_S \times 2$ sub-matrices in the first source vectors by the two 16-bit signed values from the corresponding 32-bit container of the $2 \times SVL_S$ sub-matrix in the second source vector. The two selected elements from each overlapping 32-bit containers of the first source vectors correspond to 2-in-4 elements of rows of two $SVL_S \times 2$ sub-matrices. Each 32-bit container of the second source vector holds 2 elements of columns of a compressed $2 \times SVL_S$ sub-matrix.

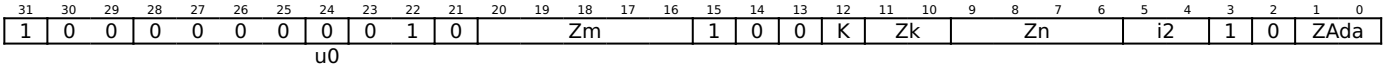
The 2-in-4 16-bit signed values from overlapping 32-bit containers of the first source vectors are selected by 4-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the sum of products result. If more than two bits of the 4-bit control corresponding to 32-bit containers of the first source vectors are 1, only the elements corresponding to the least two significant bits are selected.

The resulting $SVL_S \times SVL_S$ widened 32-bit integer sum of outer products is then destructively added to the 32-bit integer destination tile. This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP)



Encoding

```
STMOPA <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
let unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 32;
let csize : integer{} = VL DIV 8;
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*32) = ZAtile{}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var erow : array [[2]] of bits(16);
        var ecol : array [[2]] of bits(16);
        for j = 0 to 1 do
            erow[[j]] = Zeros{16};
            ecol[[j]] = op2[(2*col + j)*:16];
        end;
        var i : integer = 0;
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{}(n+r);
            for e = 0 to 1 do
                if i < 2 && ctrl[(4*col + 2*r + e)*:1] == '1' then
                    erow[[i]] = op1[(2*row + e)*:16];
                    i = i + 1;
                end;
            end;
        end;
        var sum : bits(32) = op4[(row*dim+col)*:32];
        for j = 0 to 1 do
            let erowval : integer = if unsigned then UInt(erow[[j]]) else SInt(erow[[j]]);
            let ecolval : integer = if unsigned then UInt(ecol[[j]]) else SInt(ecol[[j]]);
            sum = sum + erowval * ecolval;
        end;
        result[(row*dim+col)*:32] = sum;
    end;
end;
ZAtile{dim*dim*32}(da, 32) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STMOPA (4-way)

Signed 8-bit integer sparse sum of outer products to 32-bit integer, accumulating

This instruction generates signed integer sum of outer products by multiplying the 2-in-4 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

The sum of outer products is generated by multiplying the selected two sets of 2-in-4 8-bit signed values from each overlapping 32-bit containers of the two $SVL_S \times 4$ sub-matrices in the first source vectors by the four 8-bit signed values from the corresponding 32-bit container of the $4 \times SVL_S$ sub-matrix in the second source vector. The four selected elements from the overlapping 32-bit containers of the first source vectors correspond to two sets of 2-in-4 elements of rows of two $SVL_S \times 4$ sub-matrices. Each 32-bit container of the second source vector holds 4 elements of columns of a compressed $4 \times SVL_S$ sub-matrix.

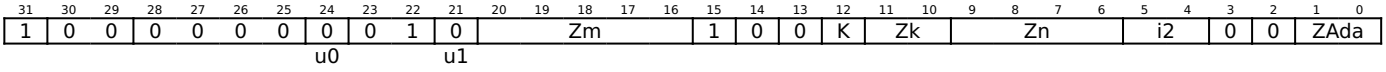
The two sets of 2-in-4 8-bit signed values from overlapping 32-bit containers of the first source vectors are selected by pairs of 4-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the sum of products result. If more than two bits of the 4-bit control corresponding to each 32-bit container of the first source vectors are 1, only the elements corresponding to the least two significant bits are selected.

The resulting $SVL_S \times SVL_S$ widened 32-bit integer sum of outer products is then destructively added to the 32-bit integer destination tile. This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP)



Encoding

```
STMOPA <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 32;
let csize : integer{} = VL DIV 4;
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*32) = ZAtile{}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var erow : array [[4]] of bits(8);
        var ecol : array [[4]] of bits(8);
        for j = 0 to 3 do
            erow[[j]] = Zeros{8};
            ecol[[j]] = op2[(4*col + j)*:8];
        end;
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{}(n+r);
            var i : integer = 0;
            for e = 0 to 3 do
                if i < 2 && ctrl[(8*col + 4*r + e)*:1] == '1' then
                    erow[[2*r + i]] = op1[(4*row + e)*:8];
                    i = i + 1;
                end;
            end;
        end;
        var sum : bits(32) = op4[(row*dim+col)*:32];
        for j = 0 to 3 do
            let erowval : integer = if op1_unsigned then UInt(erow[[j]]) else SInt(erow[[j]]);
            let ecolval : integer = if op2_unsigned then UInt(ecol[[j]]) else SInt(ecol[[j]]);
            sum = sum + erowval * ecolval;
        end;
        result[(row*dim+col)*:32] = sum;
    end;
end;
ZAtile{dim*dim*32}(da, 32) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1B (scalar plus immediate, strided registers)

Contiguous store non-temporal of bytes from multiple strided vectors (immediate index)

This instruction performs a contiguous non-temporal store of bytes from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

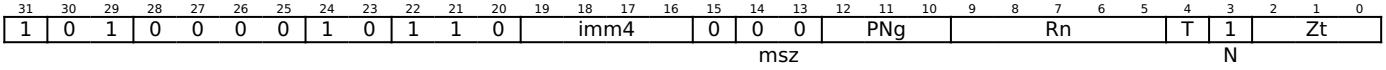
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

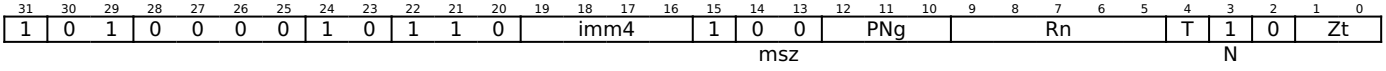
```
STNT1B { <Zt1>.B, <Zt2>.B }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
STNT1B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 8;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".

<Zt4> Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1B (scalar plus scalar, strided registers)

Contiguous store non-temporal of bytes from multiple strided vectors (scalar index)

This instruction performs a contiguous non-temporal store of bytes from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

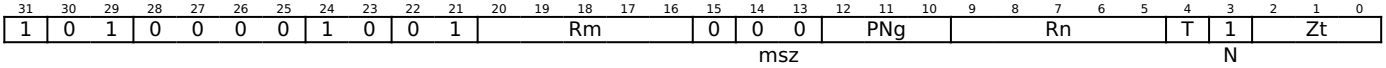
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

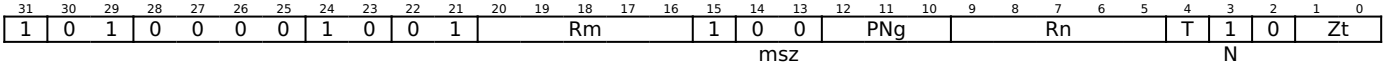
```
STNT1B { <Zt1>.B, <Zt2>.B }, <PNg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 8;
```

Four registers

(FEAT_SME2)



Encoding

```
STNT1B { <Zt1>.B, <Zt2>.B, <Zt3>.B, <Zt4>.B }, <PNg>, [<Xn|SP>, <Xm>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 8;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1D (scalar plus immediate, strided registers)

Contiguous store non-temporal of doublewords from multiple strided vectors (immediate index)

This instruction performs a contiguous non-temporal store of doublewords from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

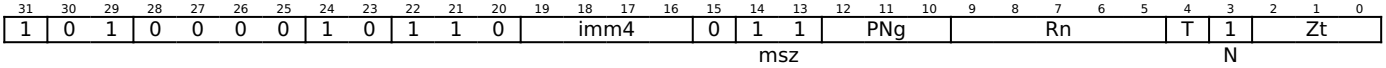
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

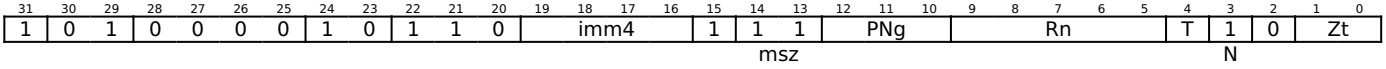
```
STNT1D { <Zt1>.D, <Zt2>.D }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
STNT1D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 64;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".

<Zt4> Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1D (scalar plus scalar, strided registers)

Contiguous store non-temporal of doublewords from multiple strided vectors (scalar index)

This instruction performs a contiguous non-temporal store of doublewords from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

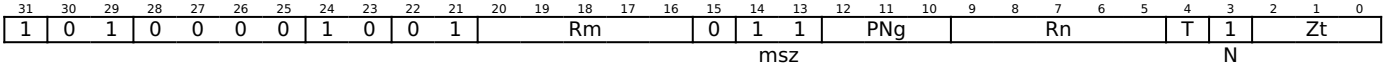
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

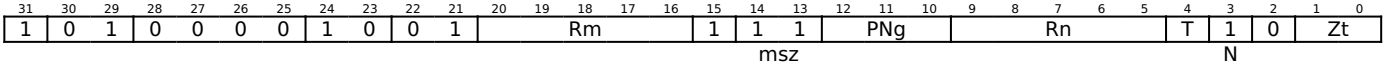
```
STNT1D { <Zt1>.D, <Zt2>.D }, <PNg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 64;
```

Four registers

(FEAT_SME2)



Encoding

```
STNT1D { <Zt1>.D, <Zt2>.D, <Zt3>.D, <Zt4>.D }, <PNg>, [<Xn|SP>, <Xm>, LSL #3]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 64;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1H (scalar plus immediate, strided registers)

Contiguous store non-temporal of halfwords from multiple strided vectors (immediate index)

This instruction performs a contiguous non-temporal store of halfwords from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

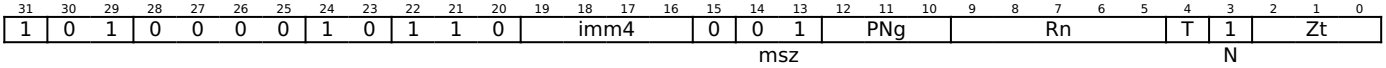
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

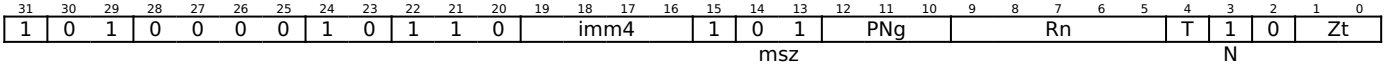
```
STNT1H { <Zt1>.H, <Zt2>.H }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
STNT1H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 16;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".

<Zt4> Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1H (scalar plus scalar, strided registers)

Contiguous store non-temporal of halfwords from multiple strided vectors (scalar index)

This instruction performs a contiguous non-temporal store of halfwords from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

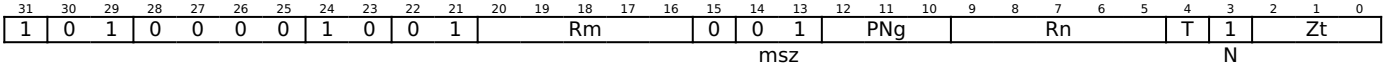
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

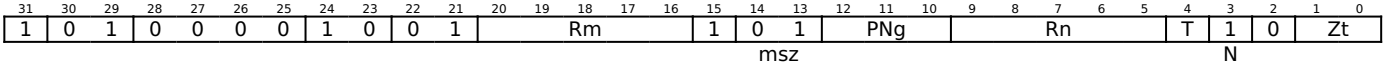
```
STNT1H { <Zt1>.H, <Zt2>.H }, <PNg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 16;
```

Four registers

(FEAT_SME2)



Encoding

```
STNT1H { <Zt1>.H, <Zt2>.H, <Zt3>.H, <Zt4>.H }, <PNg>, [<Xn|SP>, <Xm>, LSL #1]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 16;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1W (scalar plus immediate, strided registers)

Contiguous store non-temporal of words from multiple strided vectors (immediate index)

This instruction performs a contiguous non-temporal store of words from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and immediate index that is multiplied by the vector's in-memory size, irrespective of predication, and added to the base address.

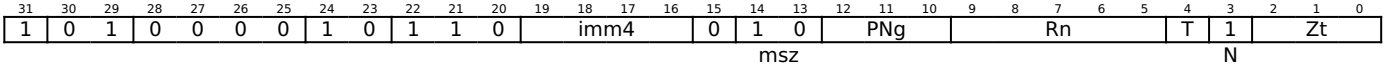
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

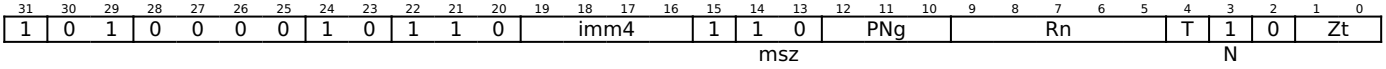
```
STNT1W { <Zt1>.S, <Zt2>.S }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Four registers

(FEAT_SME2)



Encoding

```
STNT1W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <PNg>, [<Xn|SP>{, #<imm>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 32;
let offset : integer = SInt(imm4);
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<imm>	For the "Two registers" variant: is the optional signed immediate vector offset, a multiple of 2 in the range -16 to 14, defaulting to 0, encoded as "imm4" times 2. For the "Four registers" variant: is the optional signed immediate vector offset, a multiple of 4 in the range -32 to 28, defaulting to 0, encoded as "imm4" times 4.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".

<Zt4> Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
addr = AddressAdd(base, offset * nreg * elements * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STNT1W (scalar plus scalar, strided registers)

Contiguous store non-temporal of words from multiple strided vectors (scalar index)

This instruction performs a contiguous non-temporal store of words from elements of two or four strided vector registers to the memory address generated by a 64-bit scalar base and scalar index that is added to the base address. After each element access the index value is incremented, but the index register is not updated.

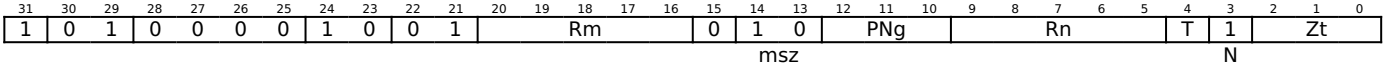
Inactive elements are not written to memory.

A non-temporal store is a hint to the system that this data is unlikely to be referenced again soon.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

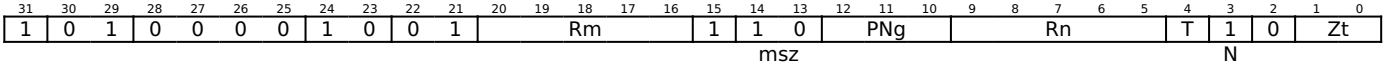
```
STNT1W { <Zt1>.S, <Zt2>.S }, <PNg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 2;
let tstride : integer = 8;
let t : integer = UInt(T::'0'::Zt);
let esize : integer{} = 32;
```

Four registers

(FEAT_SME2)



Encoding

```
STNT1W { <Zt1>.S, <Zt2>.S, <Zt3>.S, <Zt4>.S }, <PNg>, [<Xn|SP>, <Xm>, LSL #2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let m : integer = UInt(Rm);
let g : integer = UInt('1'::PNg);
let nreg : integer{} = 4;
let tstride : integer = 4;
let t : integer = UInt(T::'00'::Zt);
let esize : integer{} = 32;
```

Assembler Symbols

<Zt1>	For the "Two registers" variant: is the name of the first scalable vector register Z0-Z7 or Z16-Z23 to be transferred, encoded as "T:'0':Zt". For the "Four registers" variant: is the name of the first scalable vector register Z0-Z3 or Z16-Z19 to be transferred, encoded as "T:'00':Zt".
<Zt2>	For the "Two registers" variant: is the name of the second scalable vector register Z8-Z15 or Z24-Z31 to be transferred, encoded as "T:'1':Zt". For the "Four registers" variant: is the name of the second scalable vector register Z4-Z7 or Z20-Z23 to be transferred, encoded as "T:'01':Zt".
<PNg>	Is the name of the governing scalable predicate register PN8-PN15, with predicate-as-counter encoding, encoded as "PNg" plus 8.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.
<Xm>	Is the 64-bit name of the general-purpose offset register, encoded in the "Rm" field.
<Zt3>	Is the name of the third scalable vector register Z8-Z11 or Z24-Z27 to be transferred, encoded as "T:'10':Zt".
<Zt4>	Is the name of the fourth scalable vector register Z12-Z15 or Z28-Z31 to be transferred, encoded as "T:'11':Zt".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let elements : integer = VL DIV esize;
let mbytes : integer{} = esize DIV 8;
var offset : bits(64);
var base : bits(64);
var addr : bits(64);
let pred : bits(PL) = P{PL}(g);
let mask : bits(PL * nreg) = CounterToPredicate{PL*nreg}(pred[15:0]);
var src : bits(VL);
let contiguous : boolean = TRUE;
let nontemporal : boolean = TRUE;
var transfer : integer = t;
let tagchecked : boolean = TRUE;
let accdesc : AccessDescriptor = CreateAccDescSVE(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);

if !AnyActiveElement{PL*nreg}(mask, esize) then
    if n == 31 && ConstrainUnpredictableBool(Unpredictable_CHECKSPNONEACTIVE) then
        CheckSPAlignment();
    end;
else
    if n == 31 then CheckSPAlignment(); end;
end;

base = if n == 31 then SP{64}() else X{64}(n);
offset = X{64}(m);
addr = AddressAdd(base, UInt(offset) * mbytes, accdesc);

for r = 0 to nreg-1 do
    src = Z{VL}(transfer);
    for e = 0 to elements-1 do
        if ActivePredicateElement{PL*nreg}(mask, r * elements + e, esize) then
            Mem{esize}(addr, accdesc) = src[e*:esize];
        end;
        addr = AddressIncrement(addr, mbytes, accdesc);
    end;
    transfer = transfer + tstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

STR (array vector)

Store ZA array vector

This instruction performs a contiguous store of bytes from a ZA array vector to the memory address generated by a 64-bit scalar base plus an optional immediate offset multiplied by the current vector length in bytes. The ZA array vector is selected by the sum of the vector select register and the same immediate offset, modulo the number of bytes in a Streaming SVE vector.

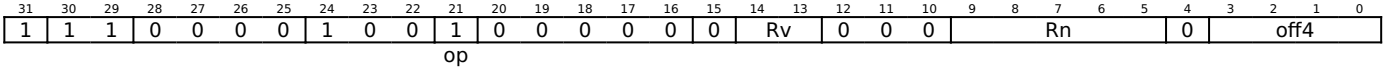
This instruction is unpredicated.

The store is performed as contiguous byte accesses, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then the base register must be aligned to 16 bytes.

This instruction does not require the PE to be in Streaming SVE mode, and it is expected that this instruction will not experience a significant slowdown due to contention with other PEs that are executing in Streaming SVE mode.

SME

(FEAT_SME)



Encoding

```
STR ZA[<Wv>, <offs>], [<Xn|SP>{, #<offs>, MUL VL}]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let v : integer = UInt('011'::Rv);
let offset : integer = UInt(off4);
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W12-W15, encoded as "Rv" plus 12.
<offs>	Is the vector select offset and optional memory offset, in the range 0 to 15, defaulting to 0, encoded in the "off4" field.
<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckSMEAndZAEnabled();
let SVL : integer{} = CurrentSVL();
let dim : integer{} = SVL DIV 8;
var base : bits(64);
let moffs : integer = offset * dim;
var src : bits(SVL);
let vbase : bits(32) = X{32}(v);
let vec : integer = (UInt(vbase) + offset) MOD dim;
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);
if n == 31 then
    CheckSPAlignment();
    base = SP{64}();
else
    base = X{64}(n);
end;
src = ZAvector{SVL}(vec);
var addr : bits(64) = AddressAdd(base, moffs, accdesc);
let aligned : boolean = IsAlignedSize(base, 16);
if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;
for e = 0 to dim-1 do
    AArch64_MemSingle{8}(addr, accdesc, aligned) = src[e*:8];
    addr = AddressIncrement(addr, 1, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

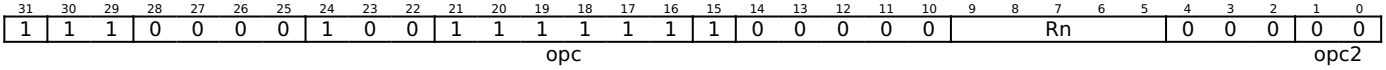
STR (table)

Store ZT0 register

This instruction stores the 64-byte ZT0 register to the memory address provided in the 64-bit scalar base register. This instruction is unpredicated.
The store is performed as contiguous byte accesses, with no endian conversion and no guarantee of single-copy atomicity larger than a byte. However, if alignment is checked, then the base register must be aligned to 16 bytes.
This instruction does not require the PE to be in Streaming SVE mode, and it is expected that this instruction will not experience a significant slowdown due to contention with other PEs that are executing in Streaming SVE mode.

SME2

(FEAT_SME2)



Encoding

```
STR ZT0, [<Xn|SP>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Rn);
let esize : integer{} = 8;
```

Assembler Symbols

<Xn|SP>	Is the 64-bit name of the general-purpose base register or stack pointer, encoded in the "Rn" field.

Operation

```
CheckSMEEnabled();
CheckSMEZT0Enabled();
let tablesize : integer{} = 512;
let elements : integer = tablesize DIV esize;
var addr : bits(64);
let table : bits(tablesize) = ZT0{tablesize}();
let contiguous : boolean = TRUE;
let nontemporal : boolean = FALSE;
let tagchecked : boolean = n != 31;
let accdesc : AccessDescriptor = CreateAccDescSME(MemOp_STORE, nontemporal, contiguous,
                                                  tagchecked);
if n == 31 then
    CheckSPAlignment();
    addr = SP{64}();
else
    addr = X{64}(n);
end;
let aligned : boolean = IsAlignedSize(addr, 16);
if !aligned && AlignmentEnforced() then
    let fault : FaultRecord = AlignmentFault(accdesc, addr);
    AArch64_Abort(fault);
end;
for e = 0 to elements-1 do
    AArch64_MemSingle{esize}(addr, accdesc, aligned) = table[e*:esize];
    addr = AddressIncrement(addr, esize DIV 8, accdesc);
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUB (to array, array and multiple vectors)

Multi-vector subtract from ZA array vectors

This instruction destructively subtracts all elements of the two or four source vectors from the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

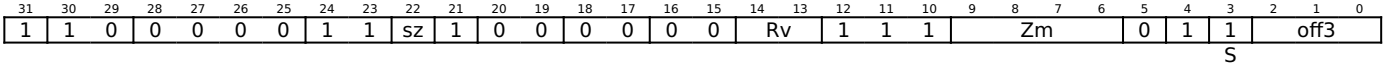
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 64-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

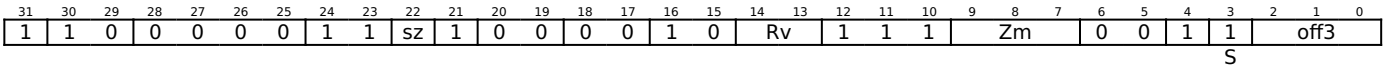
```
SUB ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
SUB ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zm1> For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zm4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = ZAvector{VL}(vec);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = element1 - element2;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUB (to array, multiple and single vector)

Multi-vector subtract by vector to ZA array vectors

This instruction subtracts all the corresponding elements of the second source vector from the two or four first source vectors and places the results in the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

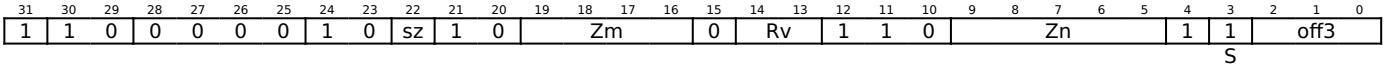
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 64-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

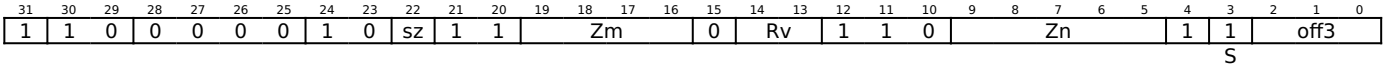
```
SUB ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<T>-<Zn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
SUB ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<T>-<Zn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = element1 - element2;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUB (to array, multiple vectors)

Multi-vector subtract to ZA array vectors

This instruction subtracts all the corresponding elements of the two or four second source vectors from first source vectors and places the results in the corresponding elements of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

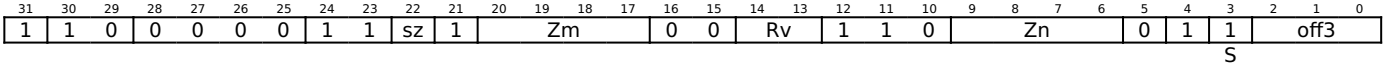
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 64-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

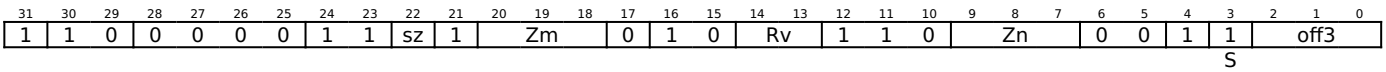
```
SUB ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<T>-<Zn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
SUB ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<T>-<Zn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm4>	Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element1 : bits(esize) = operand1[e*:esize];
        let element2 : bits(esize) = operand2[e*:esize];
        result[e*:esize] = element1 - element2;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUDOT (4-way, multiple and indexed vector)

Multi-vector signed by unsigned 8-bit integer dot product by indexed elements to 32-bit integer

This instruction calculates the dot product of four signed 8-bit integer values held in each 32-bit element of the two or four first source vectors and four unsigned 8-bit integer values in the corresponding indexed 32-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to 3.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

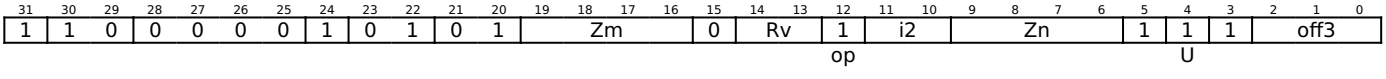
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

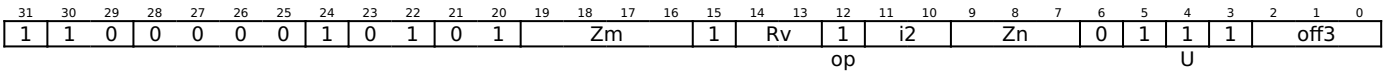
```
SUDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
SUDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        for i = 0 to 3 do
            let element1 : integer = SInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4 * s + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUDOT (4-way, multiple and single vector)

Multi-vector signed by unsigned 8-bit integer dot product by vector to 32-bit integer

This instruction calculates the dot product of four signed 8-bit integer values held in each 32-bit element of the two or four first source vectors and four unsigned 8-bit integer values in the corresponding 32-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

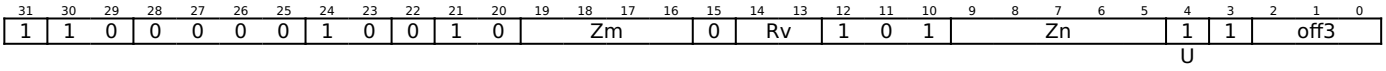
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

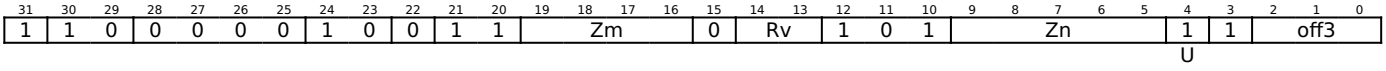
```
SUDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
SUDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let element1 : integer = SInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4 * e + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUMLALL (multiple and indexed vector)

Multi-vector signed by unsigned 8-bit integer multiply-add by indexed element to 32-bit integer

This instruction multiplies each signed 8-bit element in the one, two, or four first source vectors by each unsigned 8-bit indexed element of the second source vector, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA quad-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The element index range is from 0 to one less than the number of elements per 128-bit segment.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

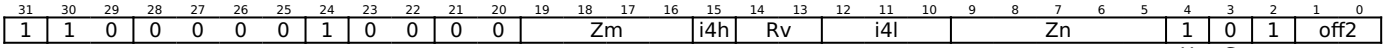
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA quad-vector , Two ZA quad-vectors and Four ZA quad-vectors

One ZA quad-vector

(FEAT_SME2)



Encoding

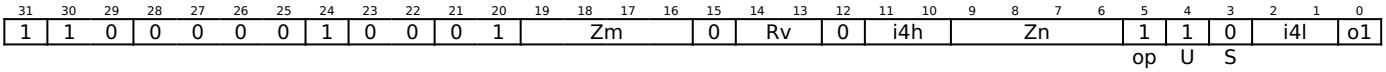
```
SUMLALL ZA.S[<Wv>, <offs1>:<offs4>], <Zn>.B, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 1;
```

Two ZA quad-vectors

(FEAT_SME2)



Encoding

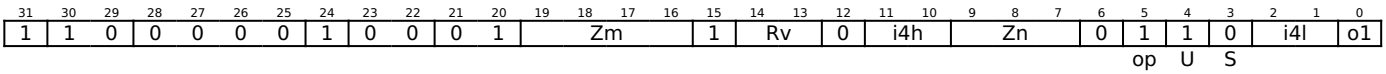
```
SUMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2)



Encoding

```
SUMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA quad-vector" variant: is the first vector select offset, encoded as "off2" field times 4. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the first vector select offset, encoded as "o1" field times 4.
<offs4>	For the "One ZA quad-vector" variant: is the fourth vector select offset, encoded as "off2" field times 4 plus 3. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 15, encoded in the "i4h:i4l" fields.
<Zn1>	For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 4 * segmentbase + index;
            let element1 : integer = SInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[s*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUMLALL (multiple and single vector)

Multi-vector signed by unsigned 8-bit integer multiply-add by vector to 32-bit integer

This instruction multiplies each signed 8-bit element in the two or four first source vectors by each unsigned 8-bit element in the second source vector, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA quad-vector groups.

The quad-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

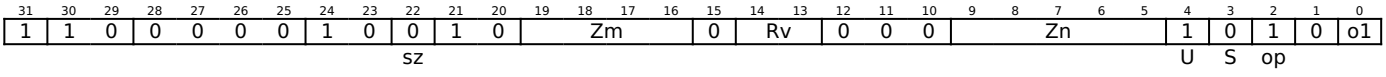
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA quad-vectors and Four ZA quad-vectors

Two ZA quad-vectors

(FEAT_SME2)



Encoding

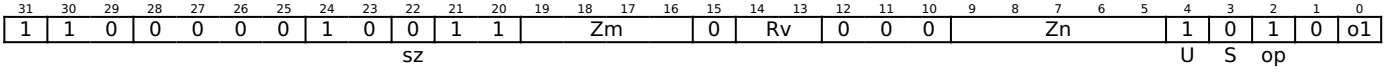
```
SUMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2)



Encoding

```
SUMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "o1" field times 4.
<offs4>	Is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = SInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUMOP4A

Signed by unsigned integer quarter-tile sum of outer products, accumulating

This instruction generates four independent quarter-tile signed by unsigned integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 32-bit or 64-bit element ZA tile.
In case of the 8-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 4$ sub-matrix of 8-bit signed values held in the half-vectors of the first source vectors by the $4 \times SVL_S \div 2$ sub-matrix of 8-bit unsigned values held in the half-vectors of the second source vectors. Each 32-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_S \div 2 \times 4$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_S \div 2$ sub-matrix.
In case of the 16-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_D \div 2 \times 4$ sub-matrix of 16-bit signed values held in the half-vectors of the first source vectors by the $4 \times SVL_D \div 2$ sub-matrix of 16-bit unsigned values held in the half-vectors of the second source vectors. Each 64-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_D \div 2 \times 4$ sub-matrix. Similarly, each 64-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_D \div 2$ sub-matrix.
The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products in case of the 8-bit integer variant or $SVL_D \div 2 \times SVL_D \div 2$ widened 64-bit integer sums of outer products in case of the 16-bit integer variant are then destructively added to the 32-bit or 64-bit integer destination tile respectively.
This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 8 classes: 32-bit, single and multiple vectors , 32-bit, single vectors , 32-bit, multiple and single vectors , 32-bit, multiple vectors , 64-bit, single and multiple vectors , 64-bit, single vectors , 64-bit, multiple and single vectors and 64-bit, multiple vectors

32-bit, single and multiple vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	1	1	Zm			0	1	0	0	0	0	0	0	Zn			0	0	0	0	ZAda	
							u0			u1	M											N					S				

Encoding

```
SUMOP4A <ZAda>.S, <Zn>.B, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

32-bit, single vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	1	0	Zm			0	1	0	0	0	0	0	0	Zn			0	0	0	0	ZAda	
							u0			u1	M											N					S				

Encoding

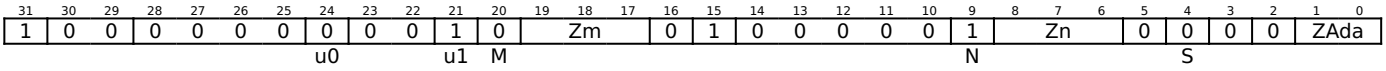
```
SUMOP4A <ZAda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

32-bit, multiple and single vectors

(FEAT_SME_MOP4)



Encoding

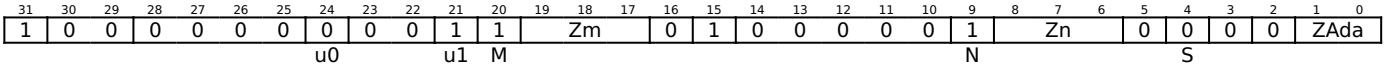
```
SUMOP4A <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

32-bit, multiple vectors

(FEAT_SME_MOP4)



Encoding

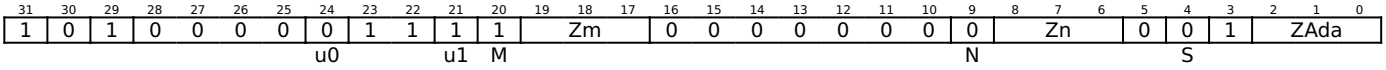
```
SUMOP4A <ZAda>.S, { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

64-bit, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

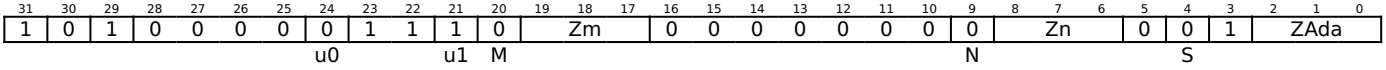
```
SUMOP4A <ZAda>.D, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

64-bit, single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

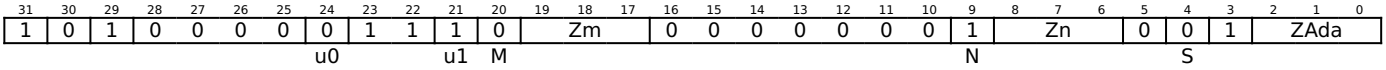
```
SUMOP4A <ZAda>.D, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

64-bit, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

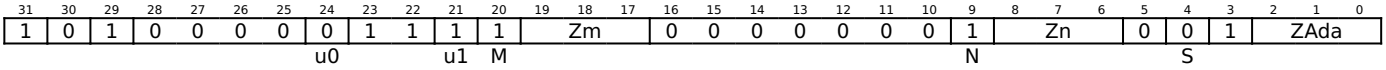
```
SUMOP4A <ZAda>.D, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

64-bit, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

```
SUMOP4A <ZAda>.D, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	For the "32-bit, multiple and single vectors", "32-bit, multiple vectors", "32-bit, single and multiple vectors", and "32-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit, multiple and single vectors", "64-bit, multiple vectors", "64-bit, single and multiple vectors", and "64-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.

<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 3 do
                let op1elt : bits(esize DIV 4) = op1[(4*row_idx + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = op2[(4*col_idx + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUMOP4S

Signed by unsigned integer quarter-tile sum of outer products, subtracting

This instruction generates four independent quarter-tile signed by unsigned integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and subtracts the results from the corresponding elements of a 32-bit or 64-bit element ZA tile.
In case of the 8-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 4$ sub-matrix of 8-bit signed values held in the half-vectors of the first source vectors by the $4 \times SVL_S \div 2$ sub-matrix of 8-bit unsigned values held in the half-vectors of the second source vectors. Each 32-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_S \div 2 \times 4$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_S \div 2$ sub-matrix.
In case of the 16-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_D \div 2 \times 4$ sub-matrix of 16-bit signed values held in the half-vectors of the first source vectors by the $4 \times SVL_D \div 2$ sub-matrix of 16-bit unsigned values held in the half-vectors of the second source vectors. Each 64-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_D \div 2 \times 4$ sub-matrix. Similarly, each 64-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_D \div 2$ sub-matrix.
The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products in case of the 8-bit integer variant or $SVL_D \div 2 \times SVL_D \div 2$ widened 64-bit integer sums of outer products in case of the 16-bit integer variant are then destructively subtracted from the 32-bit or 64-bit integer destination tile respectively.
This is equivalent to performing a 4-way dot product and subtract from each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 8 classes: 32-bit, single and multiple vectors , 32-bit, single vectors , 32-bit, multiple and single vectors , 32-bit, multiple vectors , 64-bit, single and multiple vectors , 64-bit, single vectors , 64-bit, multiple and single vectors and 64-bit, multiple vectors

32-bit, single and multiple vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	1	1	Zm			0	1	0	0	0	0	0	0	Zn			0	1	0	0	ZAda	
							u0			u1	M											N					S				

Encoding

```
SUMOP4S <ZAda>.S, <Zn>.B, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

32-bit, single vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	1	0	Zm			0	1	0	0	0	0	0	0	Zn			0	1	0	0	ZAda	
							u0			u1	M											N					S				

Encoding

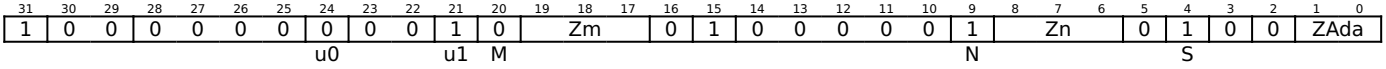
```
SUMOP4S <ZAda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

32-bit, multiple and single vectors

(FEAT_SME_MOP4)



Encoding

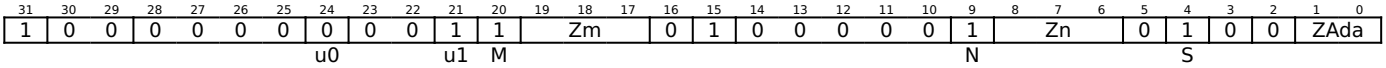
```
SUMOP4S <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

32-bit, multiple vectors

(FEAT_SME_MOP4)



Encoding

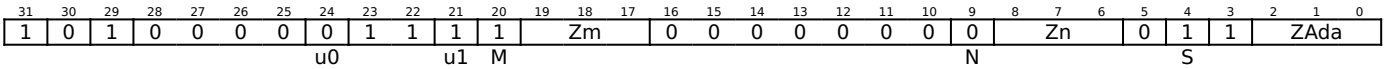
```
SUMOP4S <ZAda>.S, { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

64-bit, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

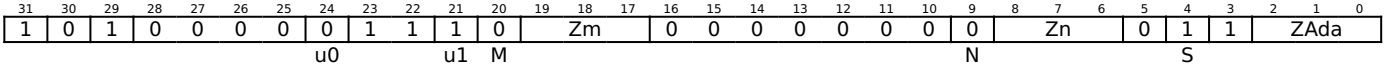
```
SUMOP4S <ZAda>.D, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

64-bit, single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

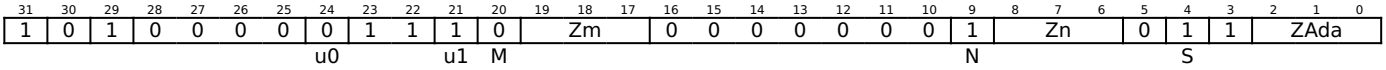
```
SUMOP4S <ZAda>.D, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

64-bit, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

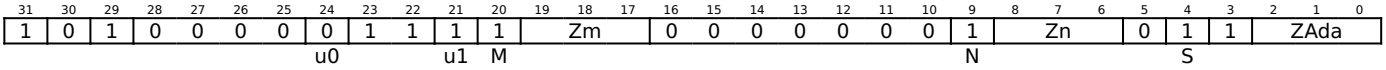
```
SUMOP4S <ZAda>.D, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

64-bit, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

```
SUMOP4S <ZAda>.D, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	For the "32-bit, multiple and single vectors", "32-bit, multiple vectors", "32-bit, single and multiple vectors", and "32-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit, multiple and single vectors", "64-bit, multiple vectors", "64-bit, single and multiple vectors", and "64-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.

<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 3 do
                let op1elt : bits(esize DIV 4) = op1[(4*row_idx + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = op2[(4*col_idx + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUMOPA (4-way)

Signed by unsigned integer sum of outer products, accumulating

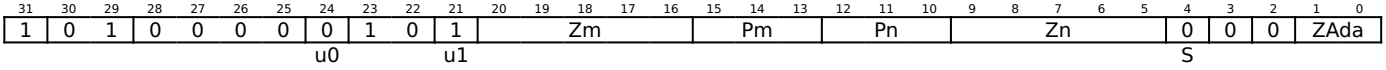
The 8-bit integer variant works with a 32-bit element ZA tile.
The 16-bit integer variant works with a 64-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. In case of the 8-bit integer variant, the first source holds $SVL_S \times 4$ sub-matrix of signed 8-bit integer values, and the second source holds $4 \times SVL_S$ sub-matrix of unsigned 8-bit integer values. In case of the 16-bit integer variant, the first source holds $SVL_D \times 4$ sub-matrix of signed 16-bit integer values, and the second source holds $4 \times SVL_D$ sub-matrix of unsigned 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When an 8-bit source element in case of 8-bit integer variant or a 16-bit source element in case of 16-bit integer variant is Inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer or $SVL_D \times SVL_D$ widened 64-bit integer sum of outer products is then destructively added to the 32-bit integer or 64-bit integer destination tile, respectively for 8-bit integer and 16-bit integer instruction variants. This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.
In case of the 8-bit integer variant, each 32-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_S \times 4$ sub-matrix, and each 32-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_S$ sub-matrix. In case of the 16-bit integer variant, each 64-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_D \times 4$ sub-matrix, and each 64-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_D$ sub-matrix.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SME)



Encoding

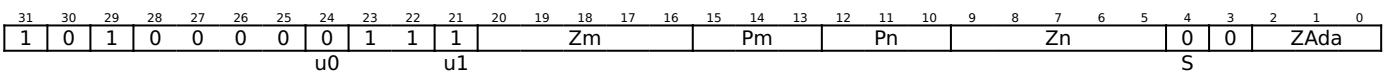
```
SUMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

64-bit

(FEAT_SME_I16I64)



Encoding

```
SUMOPA <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.

<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 3 do
            if (ActivePredicateElement{PL}(mask1, 4*row + k, esize DIV 4) &&
                  ActivePredicateElement{PL}(mask2, 4*col + k, esize DIV 4)) then
                let op1elt : bits(esize DIV 4) = operand1[(4*row + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = operand2[(4*col + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum + prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUMOPS

Signed by unsigned integer sum of outer products, subtracting

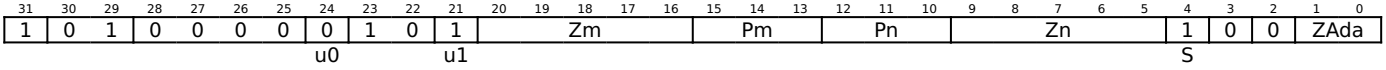
The 8-bit integer variant works with a 32-bit element ZA tile.
The 16-bit integer variant works with a 64-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. In case of the 8-bit integer variant, the first source holds $SVL_S \times 4$ sub-matrix of signed 8-bit integer values, and the second source holds $4 \times SVL_S$ sub-matrix of unsigned 8-bit integer values. In case of the 16-bit integer variant, the first source holds $SVL_D \times 4$ sub-matrix of signed 16-bit integer values, and the second source holds $4 \times SVL_D$ sub-matrix of unsigned 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When an 8-bit source element in case of 8-bit integer variant or a 16-bit source element in case of 16-bit integer variant is Inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer or $SVL_D \times SVL_D$ widened 64-bit integer sum of outer products is then destructively subtracted from the 32-bit integer or 64-bit integer destination tile, respectively for 8-bit integer and 16-bit integer instruction variants. This is equivalent to performing a 4-way dot product and subtract from each of the destination tile elements.
In case of the 8-bit integer variant, each 32-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_S \times 4$ sub-matrix, and each 32-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_S$ sub-matrix. In case of the 16-bit integer variant, each 64-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_D \times 4$ sub-matrix, and each 64-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_D$ sub-matrix.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SME)



Encoding

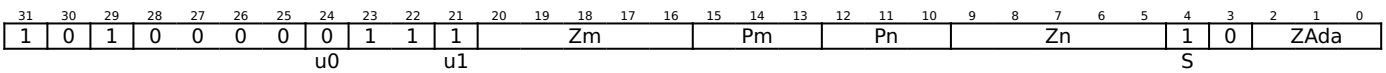
```
SUMOPS <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

64-bit

(FEAT_SME_I16I64)



Encoding

```
SUMOPS <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.

<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 3 do
            if (ActivePredicateElement{PL}(mask1, 4*row + k, esize DIV 4) &&
                  ActivePredicateElement{PL}(mask2, 4*col + k, esize DIV 4)) then
                let op1elt : bits(esize DIV 4) = operand1[(4*row + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = operand2[(4*col + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum - prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUNPK

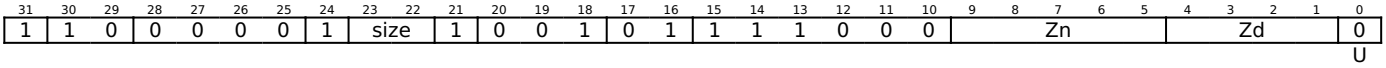
Unpack and sign-extend multi-vector elements

This instruction unpacks and sign-extends elements from one or two source vectors, and writes the sign-extended values into the double-width elements of two or four destination vectors. This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

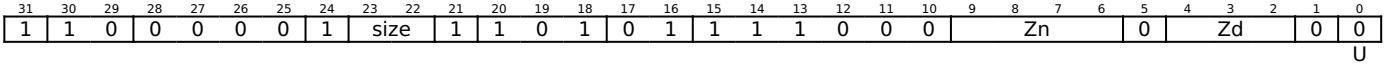
```
SUNPK { <Zd1>.<T>-<Zd2>.<T> }, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
let unsigned : boolean = FALSE;
```

Four registers

(FEAT_SME2)



Encoding

```
SUNPK { <Zd1>.<T>-<Zd4>.<T> }, { <Zn1>.<Tb>-<Zn2>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
let unsigned : boolean = FALSE;
```

Assembler Symbols

<Zd1> For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.

<Zn2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let hsize : integer{} = esize DIV 2;
let sreg : integer = nreg DIV 2;
var results : array [[4]] of bits(VL);

for r = 0 to sreg-1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for i = 0 to 1 do
        for e = 0 to elements-1 do
            let element : bits(hsize) = operand[(i*elements + e)*:hsize];
            results[[2*r+i]][e*:esize] = Extend{esize}(element, unsigned);
        end;
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUTMOPA

Signed by unsigned 8-bit integer sparse sum of outer products to 32-bit integer, accumulating

This instruction generates signed by unsigned integer sum of outer products by multiplying the 2-in-4 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

The sum of outer products is generated by multiplying the selected two sets of 2-in-4 8-bit signed values from each overlapping 32-bit containers of the two $SVL_S \times 4$ sub-matrices in the first source vectors by the four 8-bit unsigned values from the corresponding 32-bit container of the $4 \times SVL_S$ sub-matrix in the second source vector. The four selected elements from the overlapping 32-bit containers of the first source vectors correspond to two sets of 2-in-4 elements of rows of two $SVL_S \times 4$ sub-matrices. Each 32-bit container of the second source vector holds 4 elements of columns of a compressed $4 \times SVL_S$ sub-matrix.

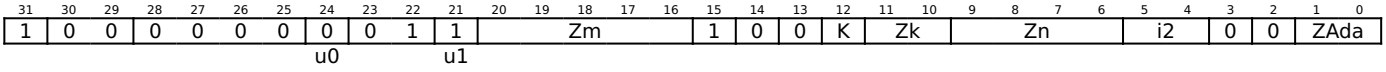
The two sets of 2-in-4 8-bit signed values from overlapping 32-bit containers of the first source vectors are selected by pairs of 4-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the sum of products result. If more than two bits of the 4-bit control corresponding to each 32-bit container of the first source vectors are 1, only the elements corresponding to the least two significant bits are selected.

The resulting $SVL_S \times SVL_S$ widened 32-bit integer sum of outer products is then destructively added to the 32-bit integer destination tile. This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP)



Encoding

```
SUTMOPA <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = FALSE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 32;
let csize : integer{} = VL DIV 4;
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*32) = ZAtile{}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var erow : array [[4]] of bits(8);
        var ecol : array [[4]] of bits(8);
        for j = 0 to 3 do
            erow[[j]] = Zeros{8};
            ecol[[j]] = op2[(4*col + j)*:8];
        end;
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{}(n+r);
            var i : integer = 0;
            for e = 0 to 3 do
                if i < 2 && ctrl[(8*col + 4*r + e)*:1] == '1' then
                    erow[[2*r + i]] = op1[(4*row + e)*:8];
                    i = i + 1;
                end;
            end;
        end;
        var sum : bits(32) = op4[(row*dim+col)*:32];
        for j = 0 to 3 do
            let erowval : integer = if op1_unsigned then UInt(erow[[j]]) else SInt(erow[[j]]);
            let ecolval : integer = if op2_unsigned then UInt(ecol[[j]]) else SInt(ecol[[j]]);
            sum = sum + erowval * ecolval;
        end;
        result[(row*dim+col)*:32] = sum;
    end;
end;
ZAtile{dim*dim*32}(da, 32) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SUVDOT

Multi-vector signed by unsigned 8-bit integer vertical dot product by indexed element to 32-bit integer

This instruction calculates the vertical dot product of the corresponding signed 8-bit elements from the four first source vectors and four unsigned 8-bit integer values in the corresponding indexed 32-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to 3.

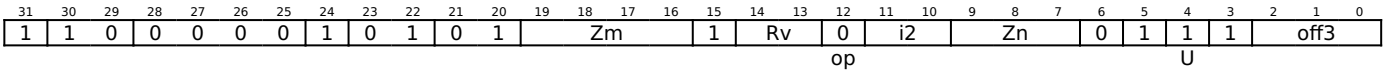
The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.

The vector group symbol VGx4 indicates that the ZA operand consists of four ZA single-vector groups. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SUVDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 4;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
let operand2 : bits(VL) = Z{VL}(m);
var result : bits(VL);

for r = 0 to 3 do
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let operand1 : bits(VL) = Z{VL}(n+i);
            let element1 : integer = SInt(operand1[(4 * e + r)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4 * s + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SVDOT (2-way)

Multi-vector signed 16-bit integer vertical dot product by indexed element to 32-bit integer

This instruction calculates the vertical dot product of the corresponding two signed 16-bit integer values held in the two first source vectors and two signed 16-bit integer values in the corresponding indexed 32-bit element of the second source vector. The widened dot product results are destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to 3.

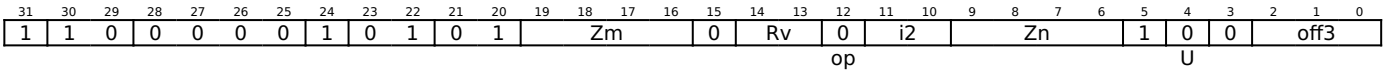
The single-vector group within each half of the ZA array is selected by the sum of the vector select register and offset, modulo half the number of ZA array vectors.

The vector group symbol VGx2 indicates that the ZA operand consists of two ZA single-vector groups. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
SVDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 2;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
let operand2 : bits(VL) = Z{VL}(m);
var result : bits(VL);

for r = 0 to 1 do
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 1 do
            let operand1 : bits(VL) = Z{VL}(n+i);
            let element1 : integer = SInt(operand1[(2 * e + r)*:(esize DIV 2)]);
            let element2 : integer = SInt(operand2[(2 * s + i)*:(esize DIV 2)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

SVDOT (4-way)

Multi-vector signed integer vertical dot product by indexed element

This instruction calculates the vertical dot product of the corresponding four signed 8-bit or 16-bit integer values held in the four first source vectors and four signed 8-bit or 16-bit integer values in the corresponding indexed 32-bit or 64-bit element of the second source vector. The widened dot product results are destructively added to the corresponding 32-bit or 64-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to one less than the number of groups per 128-bit segment.

The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.

The vector group symbol VGx4 indicates that the ZA operand consists of four ZA single-vector groups. The vector group symbol is preferred for disassembly, but optional in assembler source code.

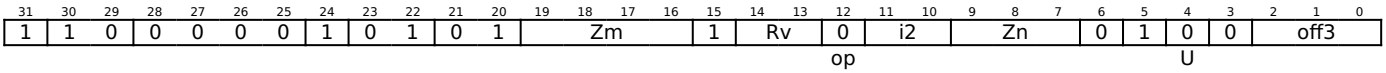
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 2 classes: 8-bit to 32-bit and 16-bit to 64-bit

8-bit to 32-bit

(FEAT_SME2)



Encoding

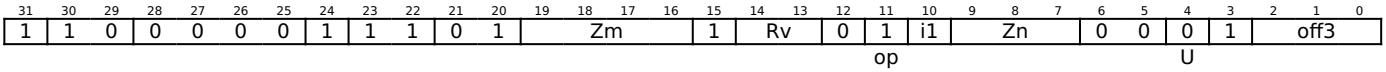
```
SVDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
```

16-bit to 64-bit

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

```
SVDOT ZA.D[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 64;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i1);
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "8-bit to 32-bit" variant: is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field. For the "16-bit to 64-bit" variant: is the immediate index of a group of four 16-bit values within each 128-bit vector segment, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 4;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
let operand2 : bits(VL) = Z{VL}(m);
var result : bits(VL);

for r = 0 to 3 do
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let operand1 : bits(VL) = Z{VL}(n+i);
            let element1 : integer = SInt(operand1[(4 * e + r)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4 * s + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UCLAMP

Multi-vector unsigned clamp to minimum/maximum

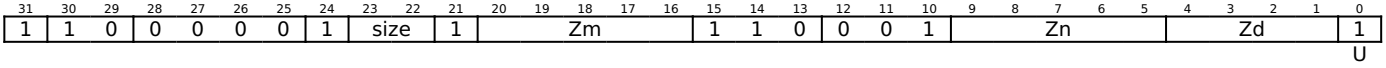
This instruction clamps each unsigned element in the two or four destination vectors to between the unsigned minimum value in the corresponding element of the first source vector and the unsigned maximum value in the corresponding element of the second source vector and destructively places the clamped results in the corresponding elements of the two or four destination vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

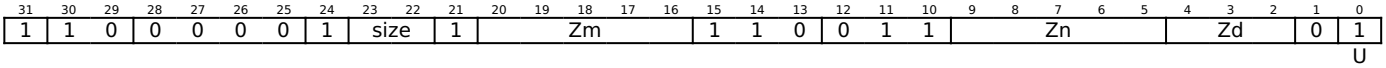
```
UCLAMP { <Zd1>.<T>-<Zd2>.<T> }, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
UCLAMP { <Zd1>.<T>-<Zd4>.<T> }, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zd1> For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = Z{VL}(d+r);
    for e = 0 to elements-1 do
        let element1 : integer = UInt(operand1[e*:esize]);
        let element2 : integer = UInt(operand2[e*:esize]);
        let element3 : integer = UInt(operand3[e*:esize]);
        let res : integer = Min(Max(element1, element3), element2);
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UCVTF

Multi-vector unsigned 32-bit integer convert to single-precision

This instruction converts each element of the two or four source vectors from unsigned 32-bit integer to single-precision, and places the results in the corresponding elements of the two or four destination vectors.

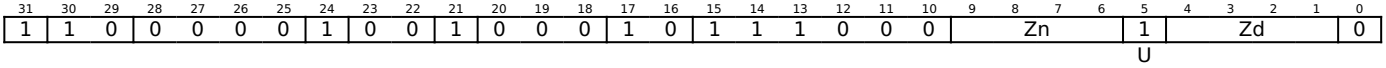
This instruction follows SME2 floating-point numerical behaviors corresponding to instructions that place their results in one or more SVE Z vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

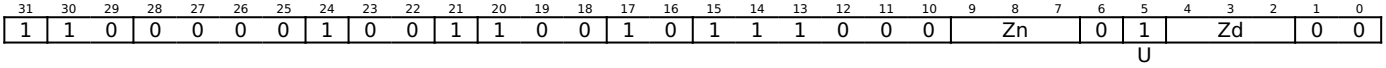
```
UCVTF { <Zd1>.S-<Zd2>.S }, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
```

Four registers

(FEAT_SME2)



Encoding

```
UCVTF { <Zd1>.S-<Zd4>.S }, { <Zn1>.S-<Zn4>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
let unsigned : boolean = TRUE;
let rounding : FPRounding = FPRoundingMode(FPCR());
```

Assembler Symbols

<Zd1>	For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.
<Zd2>	Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.
<Zn1>	For the "Two registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2. For the "Four registers" variant: is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zd4>	Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.
<Zn4>	Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV 32;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(32) = operand[e*:32];
        results[[r]][e*:32] = FixedToFP{32, 32}(element, 0, unsigned, FPCR(), rounding);
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (2-way, multiple and indexed vector)

Multi-vector unsigned 16-bit integer dot product by indexed element to 32-bit integer

This instruction calculates the dot product of two unsigned 16-bit integer values held in each 32-bit element of the two or four first source vectors and two unsigned 16-bit integer values in the corresponding indexed 32-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to 3.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

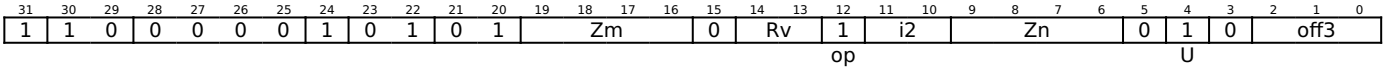
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

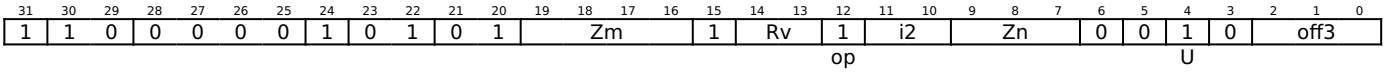
```
UDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
UDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        for i = 0 to 1 do
            let element1 : integer = UInt(operand1[(2 * e + i)*:(esize DIV 2)]);
            let element2 : integer = UInt(operand2[(2 * s + i)*:(esize DIV 2)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (2-way, multiple and single vector)

Multi-vector unsigned 16-bit integer dot product by vector to 32-bit integer

This instruction calculates the dot product of two unsigned 16-bit integer values held in each 32-bit element of the two or four first source vectors and two unsigned 16-bit integer values in the corresponding 32-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

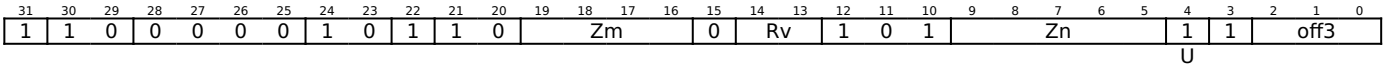
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

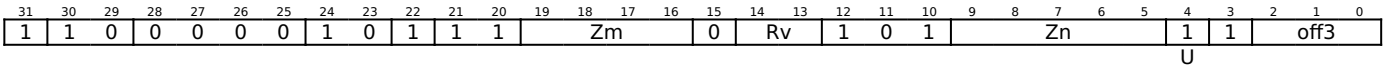
```
UDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
UDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 1 do
            let element1 : integer = UInt(operand1[(2 * e + i)*:(esize DIV 2)]);
            let element2 : integer = UInt(operand2[(2 * e + i)*:(esize DIV 2)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (2-way, multiple vectors)

Multi-vector unsigned 16-bit integer dot product to 32-bit integer

This instruction calculates the dot product of two unsigned 16-bit integer values held in each 32-bit element of the two or four first source vectors and two unsigned 16-bit integer values in the corresponding 32-bit element of the two or four second source vectors. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

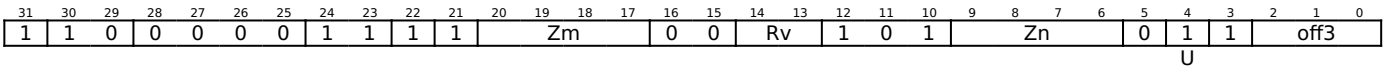
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

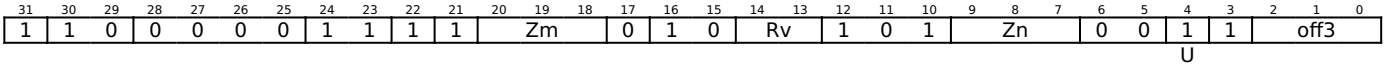
```
UDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
UDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 1 do
            let element1 : integer = UInt(operand1[(2 * e + i)*:(esize DIV 2)]);
            let element2 : integer = UInt(operand2[(2 * e + i)*:(esize DIV 2)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (4-way, multiple and indexed vector)

Multi-vector unsigned integer dot product by indexed element

This instruction calculates the dot product of four unsigned 8-bit or 16-bit integer values held in each 32-bit or 64-bit element of the two or four first source vectors and four unsigned 8-bit or 16-bit integer values in the corresponding indexed 32-bit or 64-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit or 64-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to one less than the number of groups per 128-bit segment.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

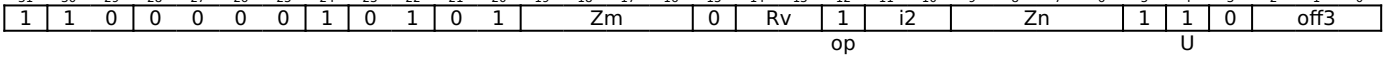
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 4 classes: Two ZA single-vectors of 32-bit elements , Two ZA single-vectors of 64-bit elements , Four ZA single-vectors of 32-bit elements and Four ZA single-vectors of 64-bit elements

Two ZA single-vectors of 32-bit elements

(FEAT_SME2)



Encoding

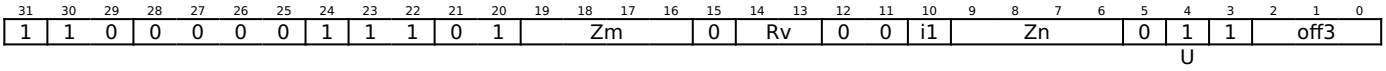
```
UDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Two ZA single-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

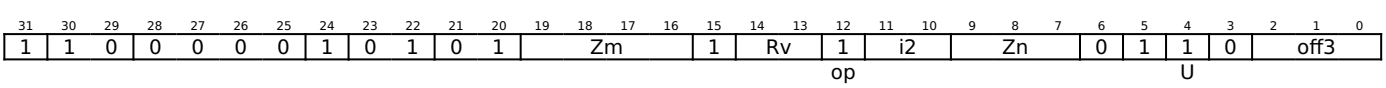
```
UDOT ZA.D[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 64;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i1);
let nreg : integer{} = 2;
```

Four ZA single-vectors of 32-bit elements

(FEAT_SME2)



Encoding

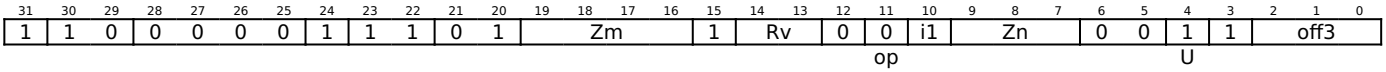
```
UDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Four ZA single-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

```
UDOT ZA.D[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 64;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i1);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors of 32-bit elements" and "Two ZA single-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors of 32-bit elements" and "Four ZA single-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "Four ZA single-vectors of 32-bit elements" and "Two ZA single-vectors of 32-bit elements" variants: is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field. For the "Four ZA single-vectors of 64-bit elements" and "Two ZA single-vectors of 64-bit elements" variants: is the immediate index of a group of four 16-bit values within each 128-bit vector segment, in the range 0 to 1, encoded in the "i1" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        for i = 0 to 3 do
            let element1 : integer = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4 * s + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (4-way, multiple and single vector)

Multi-vector unsigned integer dot product by vector

This instruction calculates the dot product of four unsigned 8-bit or 16-bit integer values held in each 32-bit or 64-bit element of the two or four first source vectors and four unsigned 8-bit or 16-bit integer values in the corresponding 32-bit or 64-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit or 64-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

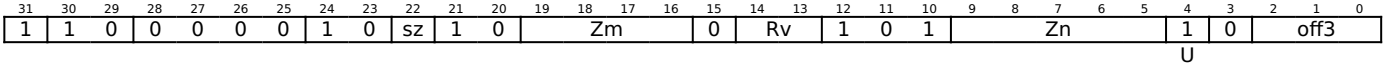
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

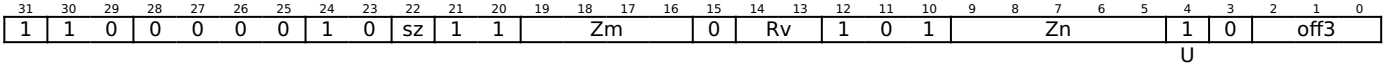
```
UDOT ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
UDOT ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let element1 : integer = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4 * e + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UDOT (4-way, multiple vectors)

Multi-vector unsigned integer dot product

This instruction calculates the dot product of four unsigned 8-bit or 16-bit integer values held in each 32-bit or 64-bit element of the two or four first source vectors and four unsigned 8-bit or 16-bit integer values in the corresponding 32-bit or 64-bit element of the two or four second source vectors. The widened dot product result is destructively added to the corresponding 32-bit or 64-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	Zm				0	0	Rv		1	0	1	Zn					0	1	0	off3	
																											U				

Encoding

```
UDOT ZA.<T>[<Wv>, <offs>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, { <Zm1>.<Tb>-<Zm2>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	sz	1	Zm				0	1	0	Rv		1	0	1	Zn				0	0	1	0	off3	
																												U				

Encoding

```
UDOT ZA.<T>[<Wv>, <offs>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, { <Zm1>.<Tb>-<Zm4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32 << UInt(sz);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs> Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

<Zn1> For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1> For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let element1 : integer = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4 * e + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMAX (multiple and single vector)

Multi-vector unsigned maximum by vector

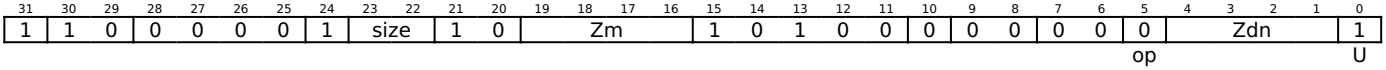
This instruction determines the unsigned maximum of elements of the second source vector and the corresponding elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

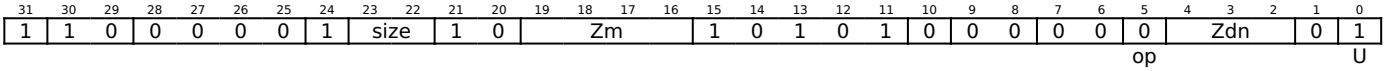
```
UMAX { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
let unsigned : boolean = TRUE;
```

Four registers

(FEAT_SME2)



Encoding

```
UMAX { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let op1elt : bits(esize) = operand1[e*:esize];
        let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
        let op2elt : bits(esize) = operand2[e*:esize];
        let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
        let res : integer = Max(element1, element2);
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMAX (multiple vectors)

Multi-vector unsigned maximum

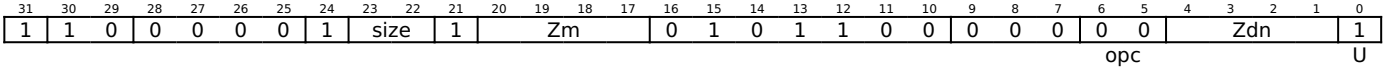
This instruction determines the unsigned maximum of elements of the two or four second source vectors and the corresponding elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

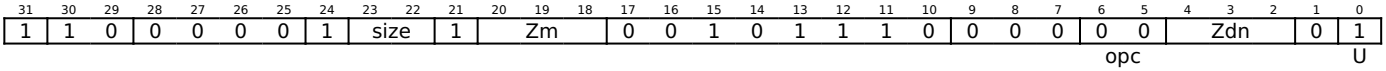
```
UMAX { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
let unsigned : boolean = TRUE;
```

Four registers

(FEAT_SME2)



Encoding

```
UMAX { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let op1elt : bits(esize) = operand1[e*:esize];
        let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
        let op2elt : bits(esize) = operand2[e*:esize];
        let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
        let res : integer = Max(element1, element2);
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMIN (multiple and single vector)

Multi-vector unsigned minimum by vector

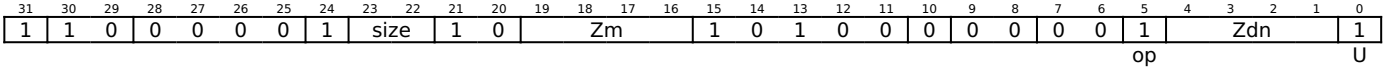
This instruction determines the unsigned minimum of elements of the second source vector and the corresponding elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

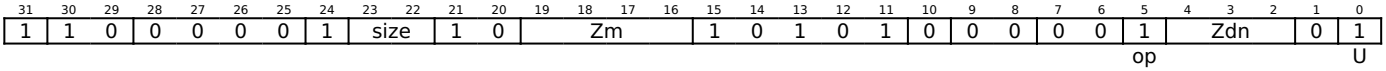
```
UMIN { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
let unsigned : boolean = TRUE;
```

Four registers

(FEAT_SME2)



Encoding

```
UMIN { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let op1elt : bits(esize) = operand1[e*:esize];
        let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
        let op2elt : bits(esize) = operand2[e*:esize];
        let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
        let res : integer = Min(element1, element2);
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMIN (multiple vectors)

Multi-vector unsigned minimum

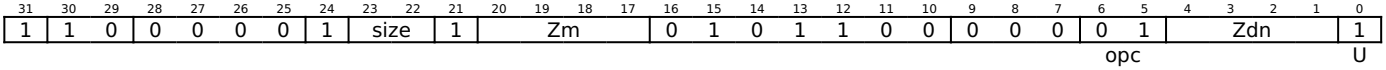
This instruction determines the unsigned minimum of elements of the two or four second source vectors and the corresponding elements of the two or four first source vectors and destructively places the results in the corresponding elements of the two or four first source vectors.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

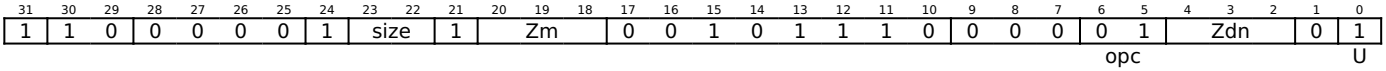
```
UMIN { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
let unsigned : boolean = TRUE;
```

Four registers

(FEAT_SME2)



Encoding

```
UMIN { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let op1elt : bits(esize) = operand1[e*:esize];
        let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
        let op2elt : bits(esize) = operand2[e*:esize];
        let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
        let res : integer = Min(element1, element2);
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLAL (multiple and indexed vector)

Multi-vector unsigned 16-bit integer multiply-add by indexed element to 32-bit integer

This instruction multiplies each unsigned 16-bit element in the one, two, or four first source vectors by each unsigned 16-bit indexed element of the second source vector, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA double-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to 7.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	0	Zm				i3h	Rv		1	i3l		Zn					1	0	off3		
																											U	S			

Encoding

```
UMLAL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	1	Zm				0	Rv		1	i3h		Zn				0	1	0	i3l	off2	
																											U	S			

Encoding

```
UMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	1	0	1	Zm				1	Rv		1	i3h		Zn			0	0	1	0	i3l	off2	
																											U	S			

Encoding

```
UMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 2 * segmentbase + index;
            let element1 : integer = UInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = UInt(operand2[s*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLAL (multiple and single vector)

Multi-vector unsigned 16-bit integer multiply-add by vector to 32-bit integer

This instruction multiplies each unsigned 16-bit element in the one, two, or four first source vectors by each unsigned 16-bit element in the second source vector, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA double-vector groups.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

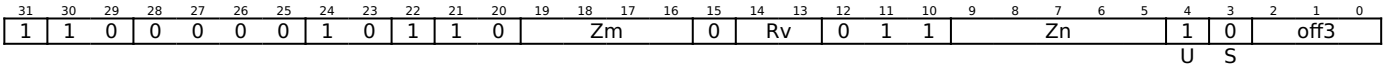
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

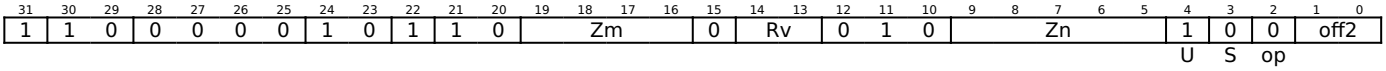
```
UMLAL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

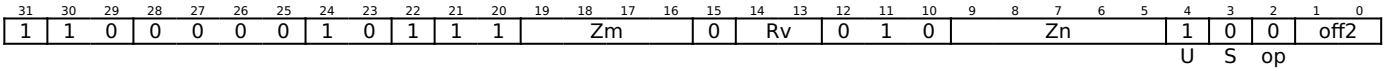
```
UMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
UMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = UInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = UInt(operand2[(2*e+i)*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLAL (multiple vectors)

Multi-vector unsigned 16-bit integer multiply-add to 32-bit integer

This instruction multiplies each unsigned 16-bit element in the two or four first source vectors by each unsigned 16-bit element in the two or four second source vectors, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA double-vector groups.

The double-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

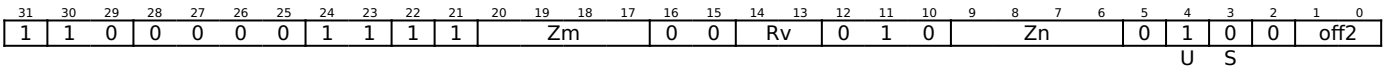
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA double-vectors and Four ZA double-vectors

Two ZA double-vectors

(FEAT_SME2)



Encoding

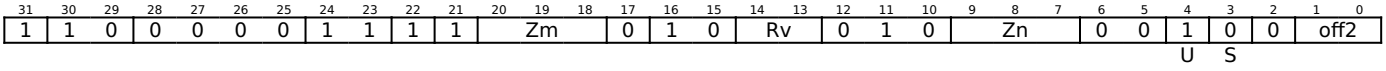
```
UMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
UMLAL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "off2" field times 2.
<offs2>	Is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = UInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = UInt(operand2[(2*e+i)*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLALL (multiple and indexed vector)

Multi-vector unsigned integer multiply-add long long by indexed element

This instruction multiplies each unsigned 8-bit or 16-bit element in the one, two, or four first source vectors by each unsigned 8-bit or 16-bit indexed element of the second source vector, widens each product to 32 bits or 64 bits, and destructively adds these values to the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

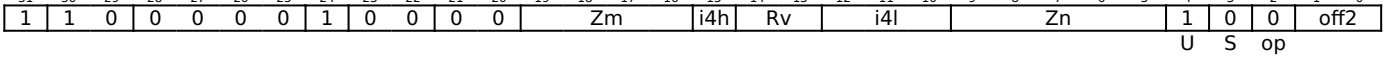
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 6 classes: One ZA quad-vector of 32-bit elements , One ZA quad-vector of 64-bit elements , Two ZA quad-vectors of 32-bit elements , Two ZA quad-vectors of 64-bit elements , Four ZA quad-vectors of 32-bit elements and Four ZA quad-vectors of 64-bit elements

One ZA quad-vector of 32-bit elements

(FEAT_SME2)



Encoding

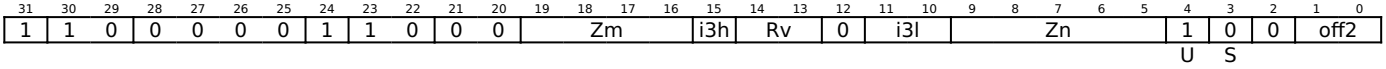
```
UMLALL ZA.S[<Wv>, <offs1>:<offs4>], <Zn>.B, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 1;
```

One ZA quad-vector of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

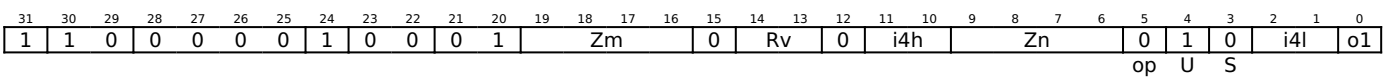
```
UMLALL ZA.D[<Wv>, <offs1>:<offs4>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
```

Two ZA quad-vectors of 32-bit elements

(FEAT_SME2)



Encoding

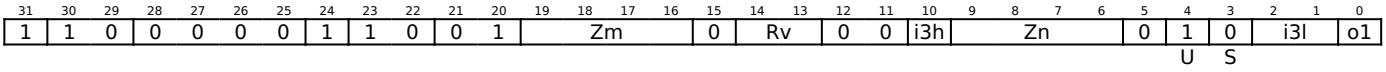
```
UMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 2;
```

Two ZA quad-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

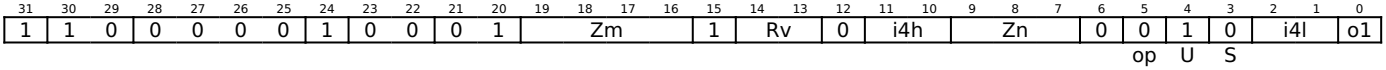
```
UMLALL ZA.D[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA quad-vectors of 32-bit elements

(FEAT_SME2)



Encoding

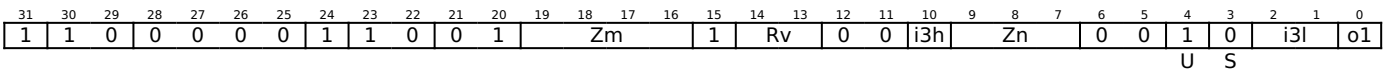
```
UMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 4;
```

Four ZA quad-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

```
UMLALL ZA.D[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA quad-vector of 32-bit elements" and "One ZA quad-vector of 64-bit elements" variants: is the first vector select offset, encoded as "off2" field times 4. For the "Four ZA quad-vectors of 32-bit elements", "Four ZA quad-vectors of 64-bit elements", "Two ZA quad-vectors of 32-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the first vector select offset, encoded as "o1" field times 4.
<offs4>	For the "One ZA quad-vector of 32-bit elements" and "One ZA quad-vector of 64-bit elements" variants: is the fourth vector select offset, encoded as "off2" field times 4 plus 3. For the "Four ZA quad-vectors of 32-bit elements", "Four ZA quad-vectors of 64-bit elements", "Two ZA quad-vectors of 32-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "Four ZA quad-vectors of 32-bit elements", "One ZA quad-vector of 32-bit elements", and "Two ZA quad-vectors of 32-bit elements" variants: is the element index, in the range 0 to 15, encoded in the "i4h:i4l" fields. For the "Four ZA quad-vectors of 64-bit elements", "One ZA quad-vector of 64-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA quad-vectors of 32-bit elements" and "Two ZA quad-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA quad-vectors of 32-bit elements" and "Four ZA quad-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 4 * segmentbase + index;
            let element1 : integer = UInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[s*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLALL (multiple and single vector)

Multi-vector unsigned integer multiply-add long long by vector

This instruction multiplies each unsigned 8-bit or 16-bit element in the one, two, or four first source vectors by each unsigned 8-bit or 16-bit element in the second source vector, widens each product to 32 bits or 64 bits, and destructively adds these values to the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 3 classes: One ZA quad-vector , Two ZA quad-vectors and Four ZA quad-vectors

One ZA quad-vector

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0	Zm				0	Rv		0	0	1	Zn					1	0	0	off2	
																											U	S	op		

Encoding

```
UMLALL ZA.<T>[<Wv>, <offs1>:<offs4>], <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let nreg : integer{} = 1;
```

Two ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0	Zm				0	Rv		0	0	0	Zn					1	0	0	0	o1
																											U	S	op		

Encoding

```
UMLALL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	1	Zm				0	Rv		0	0	0	Zn					1	0	0	0	o1
																											U	S	op		

Encoding

```
UMLALL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs1> For the "One ZA quad-vector" variant: is the first vector select offset, encoded as "off2" field times 4.

For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the first vector select offset, encoded as "o1" field times 4.

<offs4> For the "One ZA quad-vector" variant: is the fourth vector select offset, encoded as "off2" field times 4 plus 3.

For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = UInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLALL (multiple vectors)

Multi-vector unsigned integer multiply-add long long

This instruction multiplies each unsigned 8-bit or 16-bit element in the two or four first source vectors by each unsigned 8-bit or 16-bit element in the one, two, or four second source vectors, widens each product to 32 bits or 64 bits, and destructively adds these values to the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The quad-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

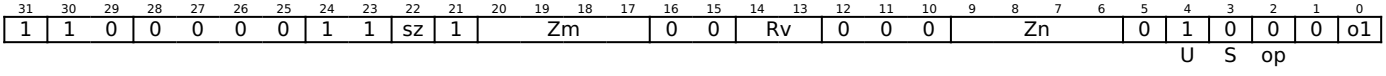
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA quad-vectors and Four ZA quad-vectors

Two ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

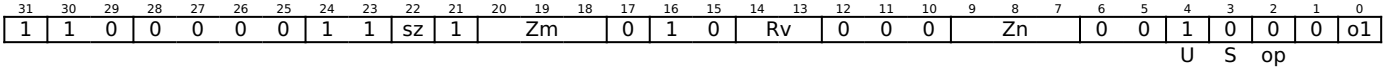
```
UMLALL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, { <Zm1>.<Tb>-<Zm2>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
UMLALL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, { <Zm1>.<Tb>-<Zm4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs1> Is the first vector select offset, encoded as "o1" field times 4.

<offs4> Is the fourth vector select offset, encoded as "o1" field times 4 plus 3.

<Zn1> For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1> For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = UInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLSL (multiple and indexed vector)

Multi-vector unsigned 16-bit integer multiply-subtract by indexed element from 32-bit integer

This instruction multiplies each unsigned 16-bit element in the one, two, or four first source vectors by each unsigned 16-bit indexed element of the second source vector, widens each product to 32 bits, and destructively subtracts these values from the corresponding 32-bit elements of the ZA double-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to 7.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

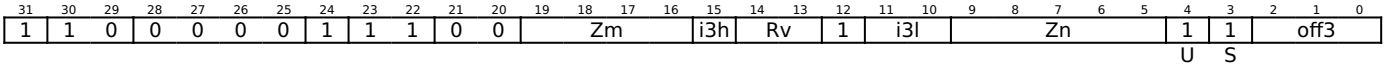
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

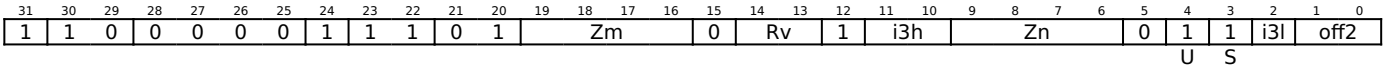
```
UMLSL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

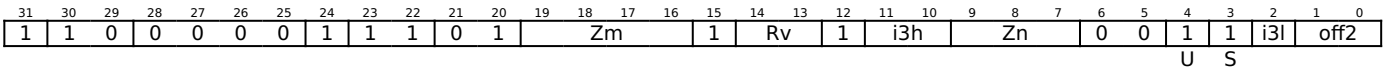
```
UMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
UMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 2 * segmentbase + index;
            let element1 : integer = UInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = UInt(operand2[s*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLSL (multiple and single vector)

Multi-vector unsigned 16-bit integer multiply-subtract by vector from 32-bit integer

This instruction multiplies each unsigned 16-bit element in the one, two, or four first source vectors by each unsigned 16-bit element in the second source vector, widens each product to 32 bits, and destructively subtracts these values from the corresponding 32-bit elements of the ZA double-vector groups.

The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

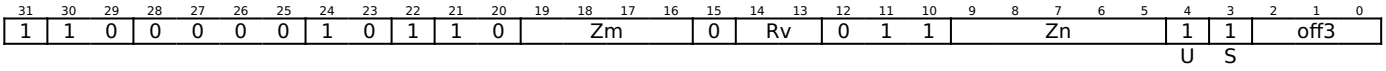
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2)



Encoding

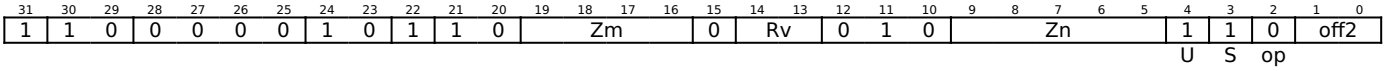
```
UMLSL ZA.S[<Wv>, <offs1>:<offs2>], <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3::'0');
let nreg : integer{} = 1;
```

Two ZA double-vectors

(FEAT_SME2)



Encoding

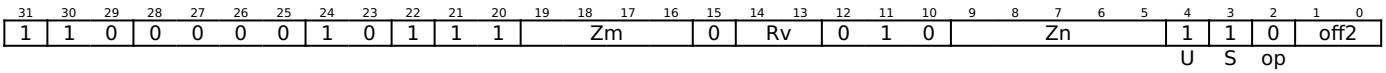
```
UMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
UMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = UInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = UInt(operand2[(2*e+i)*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLSL (multiple vectors)

Multi-vector unsigned 16-bit integer multiply-subtract from 32-bit integer

This instruction multiplies each unsigned 16-bit element in the two or four first source vectors by each unsigned 16-bit element in the two or four second source vectors, widens each product to 32 bits, and destructively subtracts these values from the corresponding 32-bit elements of the ZA double-vector groups.

The double-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

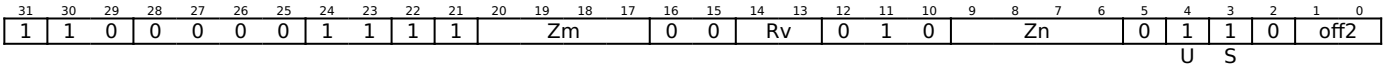
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA double-vectors and Four ZA double-vectors

Two ZA double-vectors

(FEAT_SME2)



Encoding

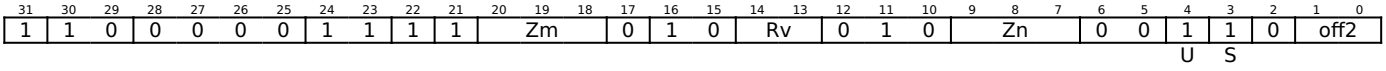
```
UMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx2}], { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 2;
```

Four ZA double-vectors

(FEAT_SME2)



Encoding

```
UMLSL ZA.S[<Wv>, <offs1>:<offs2>{, VGx4}], { <Zn1>.H-<Zn4>.H }, { <Zm1>.H-<Zm4>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off2::'0');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "off2" field times 2.
<offs2>	Is the second vector select offset, encoded as "off2" field times 2 plus 1.
<Zn1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA double-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 2;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 1 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = UInt(operand1[(2*e+i)*:(esize DIV 2)]);
            let element2 : integer = UInt(operand2[(2*e+i)*:(esize DIV 2)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLSLL (multiple and indexed vector)

Multi-vector unsigned integer multiply-subtract long long by indexed element

This instruction multiplies each unsigned 8-bit or 16-bit element in the one, two, or four first source vectors by each unsigned 8-bit or 16-bit indexed element of the second source vector, widens each product to 32 bits or 64 bits, and destructively subtracts these values from the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The index range is from 0 to one less than the number of elements per 128-bit segment.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

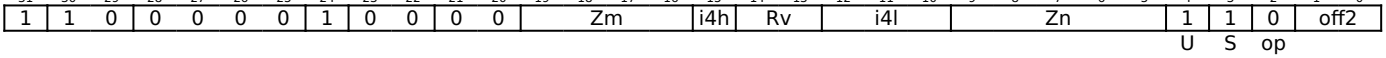
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 6 classes: One ZA quad-vector of 32-bit elements , One ZA quad-vector of 64-bit elements , Two ZA quad-vectors of 32-bit elements , Two ZA quad-vectors of 64-bit elements , Four ZA quad-vectors of 32-bit elements and Four ZA quad-vectors of 64-bit elements

One ZA quad-vector of 32-bit elements

(FEAT_SME2)



Encoding

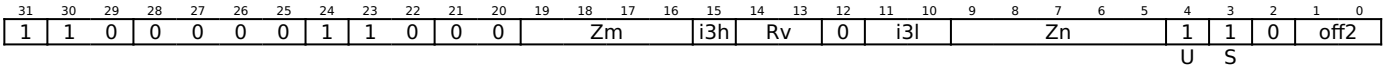
```
UMLSLL ZA.S[<Wv>, <offs1>:<offs4>], <Zn>.B, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 1;
```

One ZA quad-vector of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

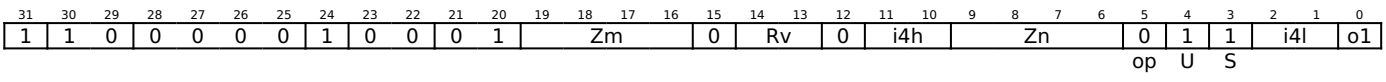
```
UMLSLL ZA.D[<Wv>, <offs1>:<offs4>], <Zn>.H, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 1;
```

Two ZA quad-vectors of 32-bit elements

(FEAT_SME2)



Encoding

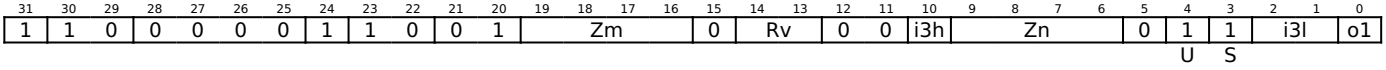
```
UMLSLL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 2;
```

Two ZA quad-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

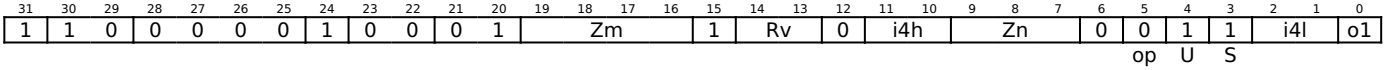
```
UMLSLL ZA.D[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 2;
```

Four ZA quad-vectors of 32-bit elements

(FEAT_SME2)



Encoding

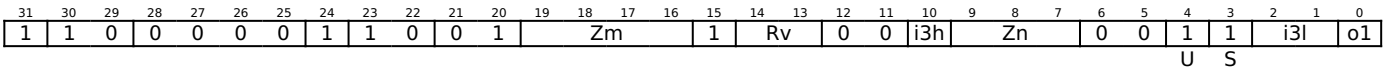
```
UMLSLL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 4;
```

Four ZA quad-vectors of 64-bit elements

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

```
UMLSLL ZA.D[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i3h::i3l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA quad-vector of 32-bit elements" and "One ZA quad-vector of 64-bit elements" variants: is the first vector select offset, encoded as "off2" field times 4. For the "Four ZA quad-vectors of 32-bit elements", "Four ZA quad-vectors of 64-bit elements", "Two ZA quad-vectors of 32-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the first vector select offset, encoded as "o1" field times 4.
<offs4>	For the "One ZA quad-vector of 32-bit elements" and "One ZA quad-vector of 64-bit elements" variants: is the fourth vector select offset, encoded as "off2" field times 4 plus 3. For the "Four ZA quad-vectors of 32-bit elements", "Four ZA quad-vectors of 64-bit elements", "Two ZA quad-vectors of 32-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "Four ZA quad-vectors of 32-bit elements", "One ZA quad-vector of 32-bit elements", and "Two ZA quad-vectors of 32-bit elements" variants: is the element index, in the range 0 to 15, encoded in the "i4h:i4l" fields. For the "Four ZA quad-vectors of 64-bit elements", "One ZA quad-vector of 64-bit elements", and "Two ZA quad-vectors of 64-bit elements" variants: is the element index, in the range 0 to 7, encoded in the "i3h:i3l" fields.
<Zn1>	For the "Two ZA quad-vectors of 32-bit elements" and "Two ZA quad-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA quad-vectors of 32-bit elements" and "Four ZA quad-vectors of 64-bit elements" variants: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 4 * segmentbase + index;
            let element1 : integer = UInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[s*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLSLL (multiple and single vector)

Multi-vector unsigned integer multiply-subtract long long by vector

This instruction multiplies each unsigned 8-bit or 16-bit element in the one, two, or four first source vectors by each unsigned 8-bit or 16-bit element in the second source vector, widens each product to 32 bits or 64 bits, and destructively subtracts these values from the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 3 classes: One ZA quad-vector , Two ZA quad-vectors and Four ZA quad-vectors

One ZA quad-vector

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0	Zm				0	Rv		0	0	1	Zn					1	1	0	off2	
																											U	S	op		

Encoding

```
UMLSLL ZA.<T>[<Wv>, <offs1>:<offs4>], <Zn>.<Tb>, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let nreg : integer{} = 1;
```

Two ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	0	Zm				0	Rv		0	0	0	Zn					1	1	0	0	o1
																											U	S	op		

Encoding

```
UMLSLL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	sz	1	1	Zm				0	Rv		0	0	0	Zn					1	1	0	0	o1
																											U	S	op		

Encoding

```
UMLSLL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, <Zm>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs1> For the "One ZA quad-vector" variant: is the first vector select offset, encoded as "off2" field times 4.

For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the first vector select offset, encoded as "o1" field times 4.

<offs4> For the "One ZA quad-vector" variant: is the fourth vector select offset, encoded as "off2" field times 4 plus 3.

For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zn1> Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = UInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMLSLL (multiple vectors)

Multi-vector unsigned integer multiply-subtract long long

This instruction multiplies each unsigned 8-bit or 16-bit element in the two or four first source vectors by each unsigned 8-bit or 16-bit element in the one, two, or four second source vectors, widens each product to 32 bits or 64 bits, and destructively subtracts these values from the corresponding 32-bit or 64-bit elements of the ZA quad-vector groups.

The quad-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

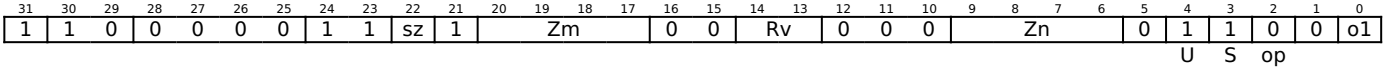
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 2 classes: Two ZA quad-vectors and Four ZA quad-vectors

Two ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

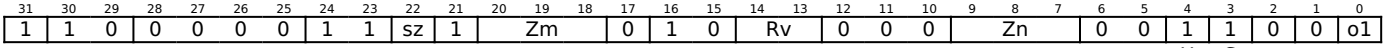
```
UMLSLL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.<Tb>-<Zn2>.<Tb> }, { <Zm1>.<Tb>-<Zm2>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64))



Encoding

```
UMLSLL ZA.<T>[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.<Tb>-<Zn4>.<Tb> }, { <Zm1>.<Tb>-<Zm4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if sz == '1' && !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 32 << UInt(sz);
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	S
1	D

<Wv> Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.

<offs1> Is the first vector select offset, encoded as "o1" field times 4.

<offs4> Is the fourth vector select offset, encoded as "o1" field times 4 plus 3.

<Zn1> For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.

For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	B
1	H

<Zn2> Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.

<Zm1> For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = UInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] - product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

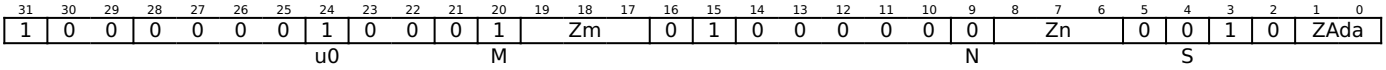
UMOP4A (2-way)

Unsigned 16-bit integer quarter-tile sum of outer products to 32-bit integer, accumulating

This instruction generates four independent quarter-tile unsigned integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 32-bit element ZA tile.
Each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 2$ sub-matrix of 16-bit unsigned values held in the half-vectors of the first source vectors by the $2 \times SVL_S \div 2$ sub-matrix of 16-bit unsigned values held in the half-vectors of the second source vectors.
Each 32-bit container of the half-vectors in the first source vectors holds 2 elements of each row of a $SVL_S \div 2 \times 2$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 2 elements of each column of a $2 \times SVL_S \div 2$ sub-matrix. The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products are then destructively added to the 32-bit integer destination tile.
This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4)



Encoding

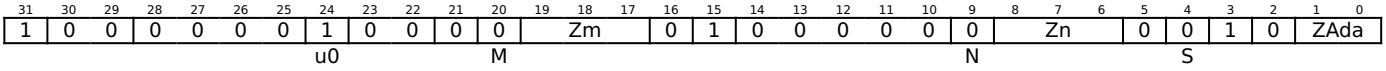
```
UMOP4A <ZAda>.S, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Single vectors

(FEAT_SME_MOP4)



Encoding

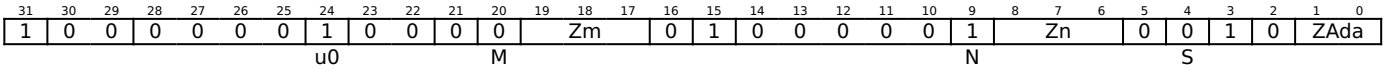
```
UMOP4A <ZAda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Multiple and single vectors

(FEAT_SME_MOP4)



Encoding

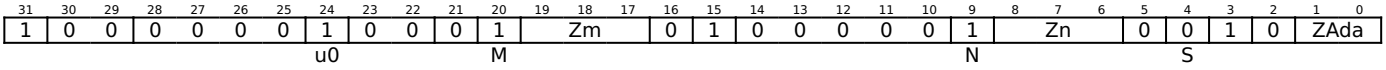
```
UMOP4A <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Multiple vectors

(FEAT_SME_MOP4)



Encoding

```
UMOP4A <ZAda>.S, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 1 do
                let op1elt : bits(esize DIV 2) = op1[(2*row_idx + k)*:(esize DIV 2)];
                let op2elt : bits(esize DIV 2) = op2[(2*col_idx + k)*:(esize DIV 2)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

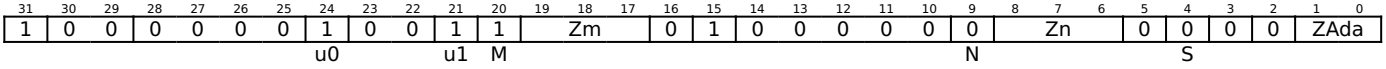
UMOP4A (4-way)

Unsigned integer quarter-tile sum of outer products, accumulating

This instruction generates four independent quarter-tile unsigned integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 32-bit or 64-bit element ZA tile.
In case of the 8-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 4$ sub-matrix of 8-bit unsigned values held in the half-vectors of the first source vectors by the $4 \times SVL_S \div 2$ sub-matrix of 8-bit unsigned values held in the half-vectors of the second source vectors. Each 32-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_S \div 2 \times 4$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_S \div 2$ sub-matrix.
In case of the 16-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_D \div 2 \times 4$ sub-matrix of 16-bit unsigned values held in the half-vectors of the first source vectors by the $4 \times SVL_D \div 2$ sub-matrix of 16-bit unsigned values held in the half-vectors of the second source vectors. Each 64-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_D \div 2 \times 4$ sub-matrix. Similarly, each 64-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_D \div 2$ sub-matrix.
The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products in case of the 8-bit integer variant or $SVL_D \div 2 \times SVL_D \div 2$ widened 64-bit integer sums of outer products in case of the 16-bit integer variant are then destructively added to the 32-bit or 64-bit integer destination tile respectively.
This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 8 classes: 32-bit, single and multiple vectors , 32-bit, single vectors , 32-bit, multiple and single vectors , 32-bit, multiple vectors , 64-bit, single and multiple vectors , 64-bit, single vectors , 64-bit, multiple and single vectors and 64-bit, multiple vectors

32-bit, single and multiple vectors

(FEAT_SME_MOP4)



Encoding

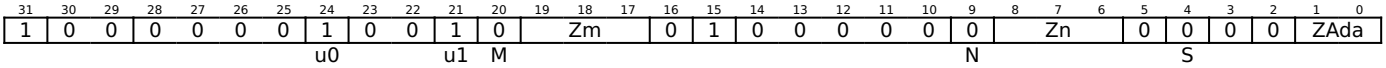
```
UMOP4A <ZAda>.S, <Zn>.B, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

32-bit, single vectors

(FEAT_SME_MOP4)



Encoding

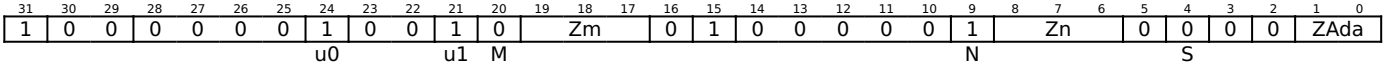
```
UMOP4A <ZAda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

32-bit, multiple and single vectors

(FEAT_SME_MOP4)



Encoding

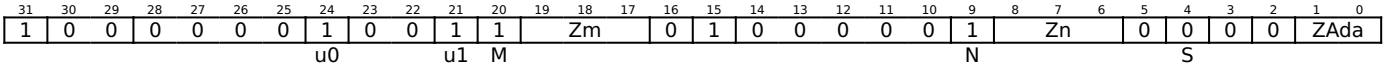
```
UMOP4A <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

32-bit, multiple vectors

(FEAT_SME_MOP4)



Encoding

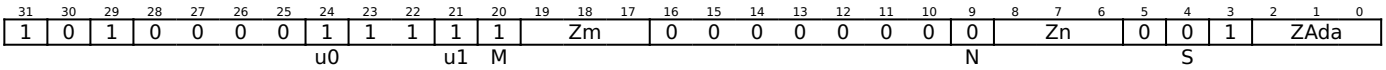
```
UMOP4A <ZAda>.S, { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

64-bit, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

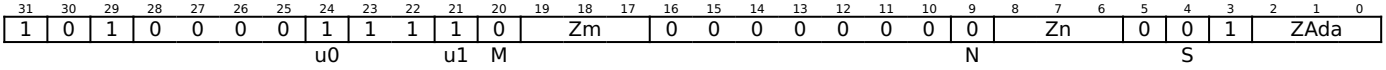
```
UMOP4A <ZAda>.D, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

64-bit, single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

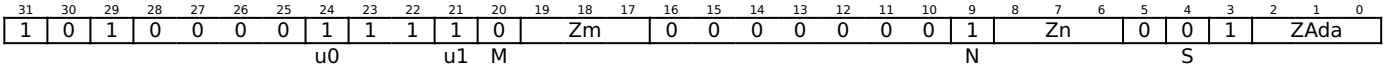
```
UMOP4A <ZAda>.D, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

64-bit, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

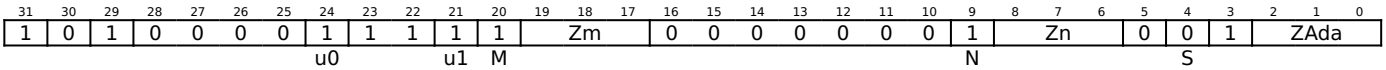
```
UMOP4A <ZAda>.D, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

64-bit, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

```
UMOP4A <ZAda>.D, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	For the "32-bit, multiple and single vectors", "32-bit, multiple vectors", "32-bit, single and multiple vectors", and "32-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit, multiple and single vectors", "64-bit, multiple vectors", "64-bit, single and multiple vectors", and "64-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.

<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 3 do
                let op1elt : bits(esize DIV 4) = op1[(4*row_idx + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = op2[(4*col_idx + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

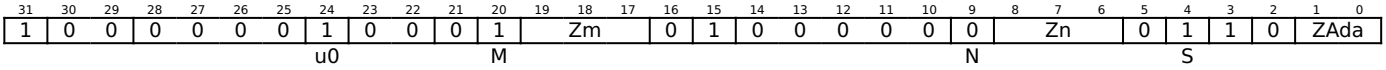
UMOP4S (2-way)

Unsigned 16-bit integer quarter-tile sum of outer products to 32-bit integer, subtracting

This instruction generates four independent quarter-tile unsigned integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and subtracts the results from the corresponding elements of a 32-bit element ZA tile.
Each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 2$ sub-matrix of 16-bit unsigned values held in the half-vectors of the first source vectors by the $2 \times SVL_S \div 2$ sub-matrix of 16-bit unsigned values held in the half-vectors of the second source vectors.
Each 32-bit container of the half-vectors in the first source vectors holds 2 elements of each row of a $SVL_S \div 2 \times 2$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 2 elements of each column of a $2 \times SVL_S \div 2$ sub-matrix. The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products are then destructively subtracted from the 32-bit integer destination tile.
This is equivalent to performing a 2-way dot product and subtract from each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 4 classes: Single and multiple vectors , Single vectors , Multiple and single vectors and Multiple vectors

Single and multiple vectors

(FEAT_SME_MOP4)



Encoding

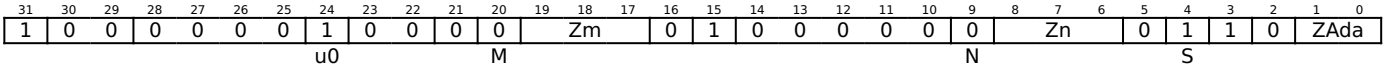
```
UMOP4S <ZAda>.S, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Single vectors

(FEAT_SME_MOP4)



Encoding

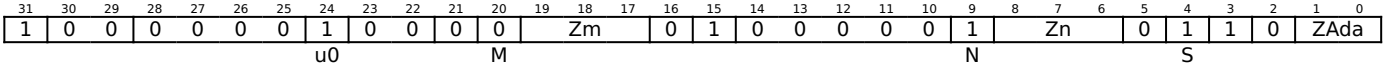
```
UMOP4S <ZAda>.S, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Multiple and single vectors

(FEAT_SME_MOP4)



Encoding

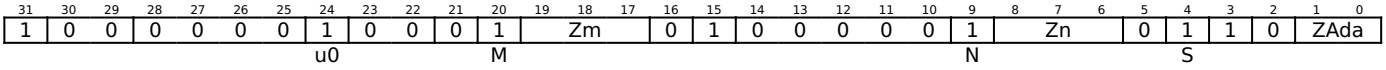
```
UMOP4S <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Multiple vectors

(FEAT_SME_MOP4)



Encoding

```
UMOP4S <ZAda>.S, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.
<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 1 do
                let op1elt : bits(esize DIV 2) = op1[(2*row_idx + k)*:(esize DIV 2)];
                let op2elt : bits(esize DIV 2) = op2[(2*col_idx + k)*:(esize DIV 2)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

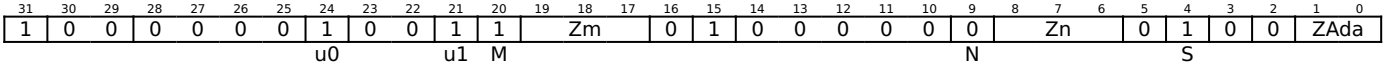
UMOP4S (4-way)

Unsigned integer quarter-tile sum of outer products, subtracting

This instruction generates four independent quarter-tile unsigned integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and subtracts the results from the corresponding elements of a 32-bit or 64-bit element ZA tile.
In case of the 8-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 4$ sub-matrix of 8-bit unsigned values held in the half-vectors of the first source vectors by the $4 \times SVL_S \div 2$ sub-matrix of 8-bit unsigned values held in the half-vectors of the second source vectors. Each 32-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_S \div 2 \times 4$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_S \div 2$ sub-matrix.
In case of the 16-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_D \div 2 \times 4$ sub-matrix of 16-bit unsigned values held in the half-vectors of the first source vectors by the $4 \times SVL_D \div 2$ sub-matrix of 16-bit unsigned values held in the half-vectors of the second source vectors. Each 64-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_D \div 2 \times 4$ sub-matrix. Similarly, each 64-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_D \div 2$ sub-matrix.
The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products in case of the 8-bit integer variant or $SVL_D \div 2 \times SVL_D \div 2$ widened 64-bit integer sums of outer products in case of the 16-bit integer variant are then destructively subtracted from the 32-bit or 64-bit integer destination tile respectively.
This is equivalent to performing a 4-way dot product and subtract from each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 8 classes: 32-bit, single and multiple vectors , 32-bit, single vectors , 32-bit, multiple and single vectors , 32-bit, multiple vectors , 64-bit, single and multiple vectors , 64-bit, single vectors , 64-bit, multiple and single vectors and 64-bit, multiple vectors

32-bit, single and multiple vectors

(FEAT_SME_MOP4)



Encoding

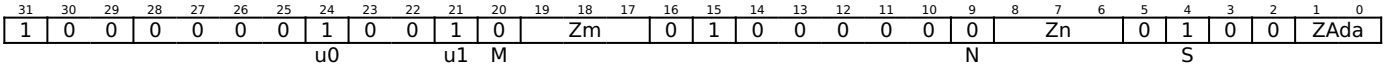
```
UMOP4S <ZAda>.S, <Zn>.B, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

32-bit, single vectors

(FEAT_SME_MOP4)



Encoding

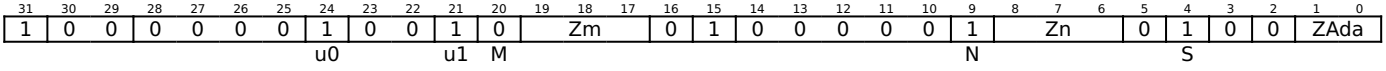
```
UMOP4S <ZAda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

32-bit, multiple and single vectors

(FEAT_SME_MOP4)



Encoding

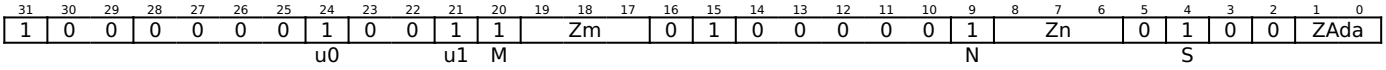
```
UMOP4S <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

32-bit, multiple vectors

(FEAT_SME_MOP4)



Encoding

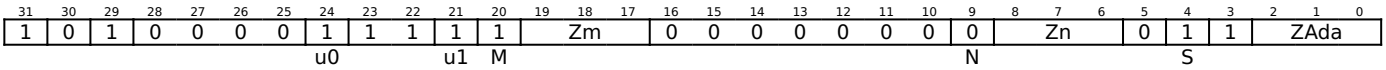
```
UMOP4S <ZAda>.S, { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

64-bit, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

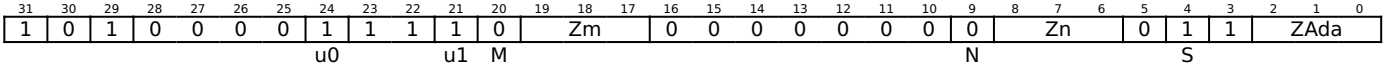
```
UMOP4S <ZAda>.D, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

64-bit, single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

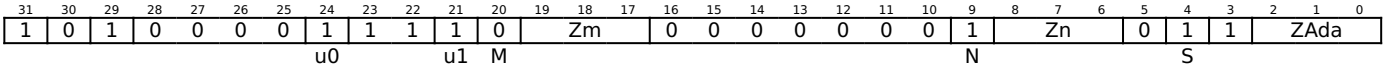
```
UMOP4S <ZAda>.D, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

64-bit, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

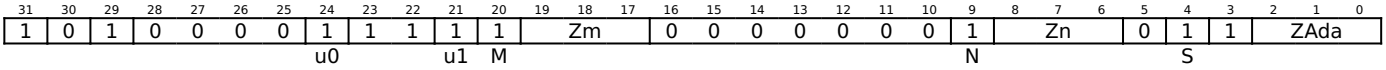
```
UMOP4S <ZAda>.D, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

64-bit, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

```
UMOP4S <ZAda>.D, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	For the "32-bit, multiple and single vectors", "32-bit, multiple vectors", "32-bit, single and multiple vectors", and "32-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit, multiple and single vectors", "64-bit, multiple vectors", "64-bit, single and multiple vectors", and "64-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.

<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 3 do
                let op1elt : bits(esize DIV 4) = op1[(4*row_idx + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = op2[(4*col_idx + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMOPA (2-way)

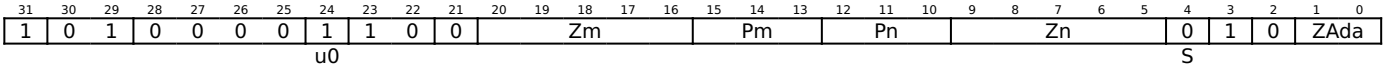
Unsigned 16-bit integer sum of outer products to 32-bit integer, accumulating

This instruction works with a 32-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. The first source holds $SVL_S \times 2$ sub-matrix of unsigned 16-bit integer values, and the second source holds $2 \times SVL_S$ sub-matrix of unsigned 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When a 16-bit source element is inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer sum of outer products is then destructively added to the 32-bit integer destination tile. This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.
Each 32-bit container of the first source vector holds 2 consecutive column elements of each row of a $SVL_S \times 2$ sub-matrix, and each 32-bit container of the second source vector holds 2 consecutive row elements of each column of a $2 \times SVL_S$ sub-matrix.

SME2

(FEAT_SME2)



Encoding

```
UMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 1 do
            if (ActivePredicateElement{PL}(mask1, 2*row + k, esize DIV 2) &&
                  ActivePredicateElement{PL}(mask2, 2*col + k, esize DIV 2)) then
                let op1elt : bits(esize DIV 2) = operand1[(2*row + k)*:(esize DIV 2)];
                let op2elt : bits(esize DIV 2) = operand2[(2*col + k)*:(esize DIV 2)];
                let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum + prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMOPA (4-way)

Unsigned integer sum of outer products, accumulating

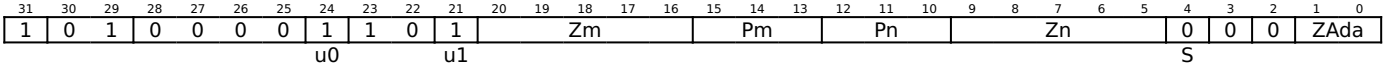
The 8-bit integer variant works with a 32-bit element ZA tile.
The 16-bit integer variant works with a 64-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. In case of the 8-bit integer variant, the first source holds $SVL_S \times 4$ sub-matrix of unsigned 8-bit integer values, and the second source holds $4 \times SVL_S$ sub-matrix of unsigned 8-bit integer values. In case of the 16-bit integer variant, the first source holds $SVL_D \times 4$ sub-matrix of unsigned 16-bit integer values, and the second source holds $4 \times SVL_D$ sub-matrix of unsigned 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When an 8-bit source element in case of 8-bit integer variant or a 16-bit source element in case of 16-bit integer variant is Inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer or $SVL_D \times SVL_D$ widened 64-bit integer sum of outer products is then destructively added to the 32-bit integer or 64-bit integer destination tile, respectively for 8-bit integer and 16-bit integer instruction variants. This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.
In case of the 8-bit integer variant, each 32-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_S \times 4$ sub-matrix, and each 32-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_S$ sub-matrix. In case of the 16-bit integer variant, each 64-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_D \times 4$ sub-matrix, and each 64-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_D$ sub-matrix.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SME)



Encoding

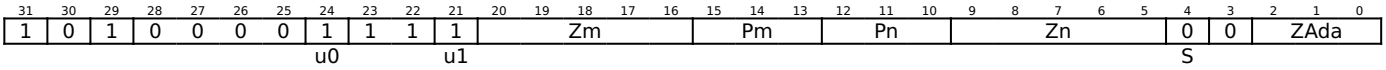
```
UMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

64-bit

(FEAT_SME_I16I64)



Encoding

```
UMOPA <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.

<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 3 do
            if (ActivePredicateElement{PL}(mask1, 4*row + k, esize DIV 4) &&
                  ActivePredicateElement{PL}(mask2, 4*col + k, esize DIV 4)) then
                let op1elt : bits(esize DIV 4) = operand1[(4*row + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = operand2[(4*col + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum + prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMOPS (2-way)

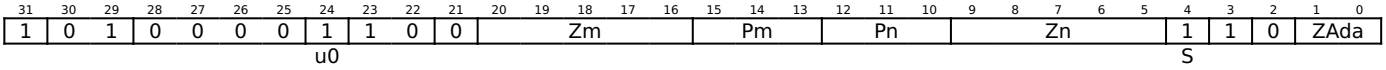
Unsigned 16-bit integer sum of outer products to 32-bit integer, subtracting

This instruction works with a 32-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. The first source holds $SVL_S \times 2$ sub-matrix of unsigned 16-bit integer values, and the second source holds $2 \times SVL_S$ sub-matrix of unsigned 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When a 16-bit source element is inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer sum of outer products is then destructively subtracted from the 32-bit integer destination tile.
This is equivalent to performing a 2-way dot product and subtract from each of the destination tile elements.
Each 32-bit container of the first source vector holds 2 consecutive column elements of each row of a $SVL_S \times 2$ sub-matrix, and each 32-bit container of the second source vector holds 2 consecutive row elements of each column of a $2 \times SVL_S$ sub-matrix.

SME2

(FEAT_SME2)



Encoding

```
UMOPS <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.
<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 1 do
            if (ActivePredicateElement{PL}(mask1, 2*row + k, esize DIV 2) &&
                  ActivePredicateElement{PL}(mask2, 2*col + k, esize DIV 2)) then
                let op1elt : bits(esize DIV 2) = operand1[(2*row + k)*:(esize DIV 2)];
                let op2elt : bits(esize DIV 2) = operand2[(2*col + k)*:(esize DIV 2)];
                let element1 : integer = if unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum - prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UMOPS (4-way)

Unsigned integer sum of outer products, subtracting

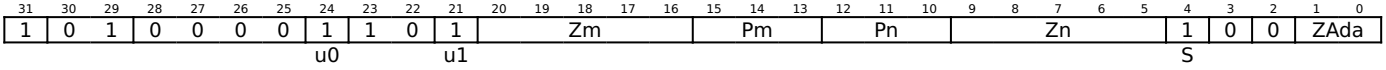
The 8-bit integer variant works with a 32-bit element ZA tile.
The 16-bit integer variant works with a 64-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. In case of the 8-bit integer variant, the first source holds $SVL_S \times 4$ sub-matrix of unsigned 8-bit integer values, and the second source holds $4 \times SVL_S$ sub-matrix of unsigned 8-bit integer values. In case of the 16-bit integer variant, the first source holds $SVL_D \times 4$ sub-matrix of unsigned 16-bit integer values, and the second source holds $4 \times SVL_D$ sub-matrix of unsigned 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When an 8-bit source element in case of 8-bit integer variant or a 16-bit source element in case of 16-bit integer variant is Inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer or $SVL_D \times SVL_D$ widened 64-bit integer sum of outer products is then destructively subtracted from the 32-bit integer or 64-bit integer destination tile, respectively for 8-bit integer and 16-bit integer instruction variants. This is equivalent to performing a 4-way dot product and subtract from each of the destination tile elements.
In case of the 8-bit integer variant, each 32-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_S \times 4$ sub-matrix, and each 32-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_S$ sub-matrix. In case of the 16-bit integer variant, each 64-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_D \times 4$ sub-matrix, and each 64-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_D$ sub-matrix.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SME)



Encoding

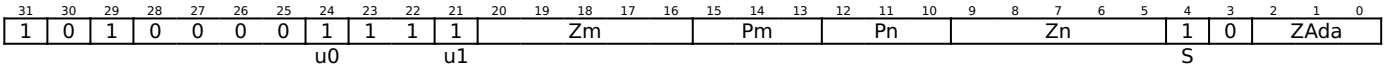
```
UMOPS <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

64-bit

(FEAT_SME_I16I64)



Encoding

```
UMOPS <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.

<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 3 do
            if (ActivePredicateElement{PL}(mask1, 4*row + k, esize DIV 4) &&
                  ActivePredicateElement{PL}(mask2, 4*col + k, esize DIV 4)) then
                let op1elt : bits(esize DIV 4) = operand1[(4*row + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = operand2[(4*col + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum - prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQCVT (four registers)

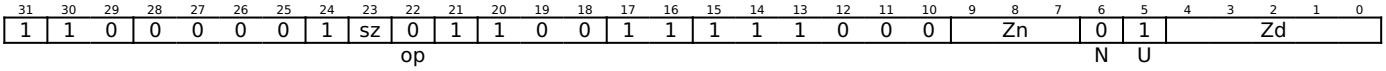
Multi-vector unsigned saturating extract narrow

This instruction saturates the unsigned integer value in each element of the four source vectors to quarter the original source element width, and places the results in the quarter-width destination elements.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
UQCVT <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(sz);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	B
1	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	S
1	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for r = 0 to 3 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : integer = UInt(operand[e*:(4 * esize)]);
        result[(r*elements + e)*:esize] = UnsignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQCVT (two registers)

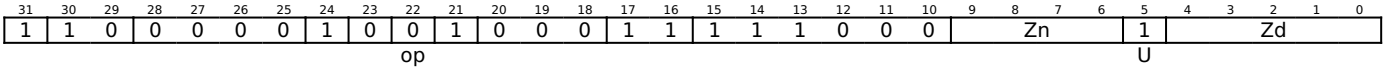
Multi-vector unsigned 32-bit integer saturating extract narrow to 16-bit integer

This instruction saturates the unsigned 32-bit integer value in each element of the two source vectors to half the original source element width, and places the 16-bit results in the half-width destination elements.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
UQCVT <Zd>.H, { <Zn1>.S-<Zn2>.S }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for r = 0 to 1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : integer = UInt(operand[e*:(2 * esize)]);
        result[(r*elements + e)*:esize] = UnsignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQCVTN

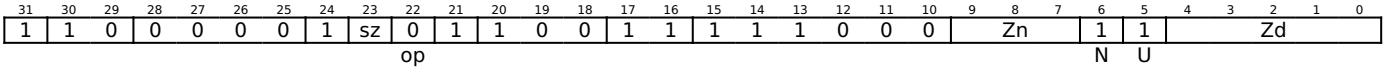
Multi-vector unsigned saturating extract narrow to interleaved integer

This instruction saturates the unsigned integer value in each element of the four source vectors to quarter the original source element width, and places the four-way interleaved results in the quarter-width destination elements.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
UQCVTN <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(sz);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “sz”:

sz	<T>
0	B
1	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “sz”:

sz	<Tb>
0	S
1	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 3 do
        let operand : bits(VL) = Z{VL}(n+i);
        let element : integer = UInt(operand[e*:(4 * esize)]);
        result[(4*e + i)*:esize] = UnsignedSat{esize}(element);
    end;
end;

Z{VL}(d) = result;
```


Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQRSHR (four registers)

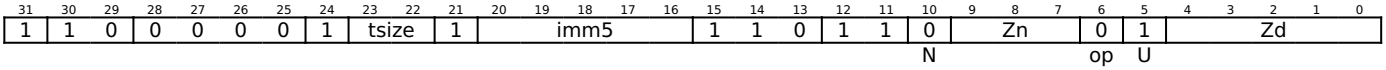
Multi-vector unsigned saturating rounding shift right narrow by immediate

This instruction shifts right by an immediate value the unsigned integer value in each element of the four source vectors, and places the rounded results in the quarter-width destination elements. Each result element is saturated to the quarter-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per source element.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
UQRSHR <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if tsize == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
let shift : integer = 8 * esize - UInt(tsize::imm5);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsize”:

tsize	<T>
00	RESERVED
01	B
1x	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “tsize”:

tsize	<Tb>
00	RESERVED
01	S
1x	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

<const> Is the immediate shift amount, in the range 1 to number of bits per source element, encoded in "tsize:imm5".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for r = 0 to 3 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(4 * esize) = operand[e*:(4 * esize)];
        let res : integer = (UInt(element) + 1 << (shift-1)) >> shift;
        result[(r*elements + e)*:esize] = UnsignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQRSHR (two registers)

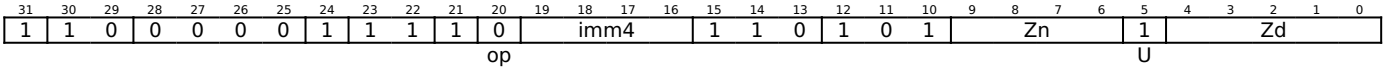
Multi-vector unsigned 32-bit integer saturating rounding shift right narrow by immediate to 16-bit integer

This instruction shifts right by an immediate value the unsigned integer value in each element of the two source vectors, and places the rounded results in the half-width destination elements. Each result element is saturated to the half-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to 16.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
UQRSHR <Zd>.H, { <Zn1>.S-<Zn2>.S }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 16;
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd);
let shift : integer = esize - UInt(imm4);
```

Assembler Symbols

<Zd>	Is the name of the destination scalable vector register, encoded in the "Zd" field.
<Zn1>	Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.
<const>	Is the immediate shift amount, in the range 1 to 16, encoded as 16 minus "imm4".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (2 * esize);
var result : bits(VL);

for r = 0 to 1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for e = 0 to elements-1 do
        let element : bits(2 * esize) = operand[e*:(2 * esize)];
        let res : integer = (UInt(element) + 1 << (shift-1)) >> shift;
        result[(r*elements + e)*:esize] = UnsignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UQRSHRN

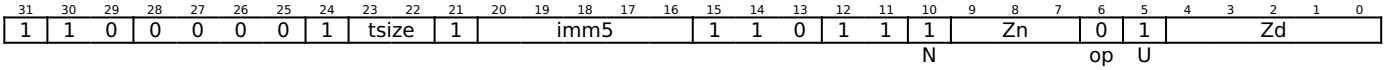
Multi-vector unsigned saturating rounding shift right narrow by immediate to interleaved integer

This instruction shifts right by an immediate value the unsigned integer value in each element of the four source vectors, and places the four-way interleaved rounded results in the quarter-width destination elements. Each result element is saturated to the quarter-width N-bit element's unsigned integer range 0 to (2^N)-1. The immediate shift amount is an unsigned value in the range 1 to number of bits per source element.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
UQRSHRN <Zd>.<T>, { <Zn1>.<Tb>-<Zn4>.<Tb> }, #<const>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if tsize == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << HighestSetBitNZ(tsize);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd);
let shift : integer = 8 * esize - UInt(tsize::imm5);
```

Assembler Symbols

<Zd> Is the name of the destination scalable vector register, encoded in the "Zd" field.

<T> Is the size specifier, encoded in “tsize”:

tsize	<T>
00	RESERVED
01	B
1x	H

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Tb> Is the size specifier, encoded in “tsize”:

tsize	<Tb>
00	RESERVED
01	S
1x	D

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

<const> Is the immediate shift amount, in the range 1 to number of bits per source element, encoded in "tsize:imm5".

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV (4 * esize);
var result : bits(VL);

for e = 0 to elements-1 do
    for i = 0 to 3 do
        let operand : bits(VL) = Z{VL}(n+i);
        let element : bits(4 * esize) = operand[e*:(4 * esize)];
        let res : integer = (UInt(element) + 1 << (shift-1)) >> shift;
        result[(4*e + i)*:esize] = UnsignedSat{esize}(res);
    end;
end;

Z{VL}(d) = result;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

URSHL (multiple and single vector)

Multi-vector unsigned rounding shift left by vector

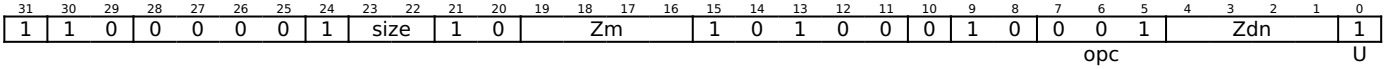
This instruction shifts the unsigned elements of the two or four first source vectors by the corresponding elements of the second source vector, and destructively places the rounded results in the corresponding elements of the two or four first source vectors. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

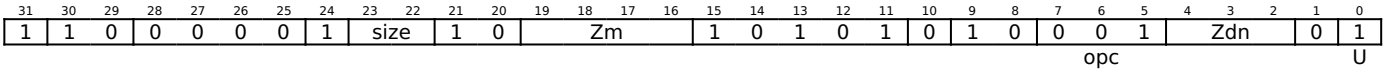
```
URSHL { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
URSHL { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt('0'::Zm);
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm> Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for e = 0 to elements-1 do
        let element : integer = UInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + 1 << (shift - 1)) >> shift;
        end;
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

URSHL (multiple vectors)

Multi-vector unsigned rounding shift left

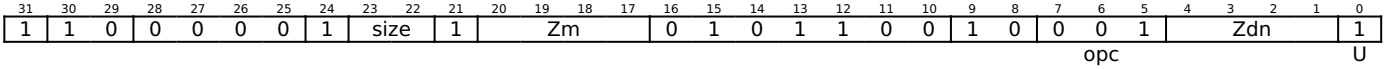
This instruction shifts the unsigned elements of the two or four first source vectors by the corresponding elements of the two or four second source vectors, and destructively places the rounded results in the corresponding elements of the two or four first source vectors. A positive shift amount performs a left shift, otherwise a right shift by the negated shift amount is performed.

This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

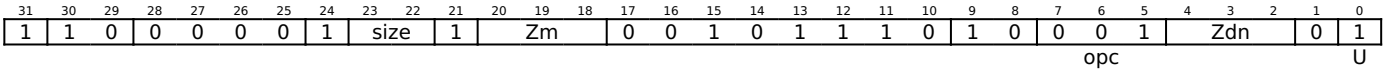
```
URSHL { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zdn1>.<T>-<Zdn2>.<T> }, { <Zm1>.<T>-<Zm2>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'0');
let m : integer = UInt(Zm::'0');
let nreg : integer{} = 2;
```

Four registers

(FEAT_SME2)



Encoding

```
URSHL { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zdn1>.<T>-<Zdn4>.<T> }, { <Zm1>.<T>-<Zm4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let dn : integer = UInt(Zdn::'00');
let m : integer = UInt(Zm::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Zdn1> For the "Two registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zdn2> Is the name of the second scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 2 plus 1.

<Zm1> For the "Two registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.

<Zm2> Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zdn4> Is the name of the fourth scalable vector register of the destination and first source multi-vector group, encoded as "Zdn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
var results : array [[4]] of bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(dn+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for e = 0 to elements-1 do
        let element : integer = UInt(operand1[e*:esize]);
        var shift : integer = ShiftSat(SInt(operand2[e*:esize]), esize);
        var res : integer;
        if shift >= 0 then
            res = element << shift;
        else
            shift = -shift;
            res = (element + 1 << (shift - 1)) >> shift;
        end;
        results[[r]][e*:esize] = res[esize-1:0];
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(dn+r) = results[[r]];
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USDOT (4-way, multiple and indexed vector)

Multi-vector unsigned by signed 8-bit integer dot product by indexed element to 32-bit integer

This instruction calculates the dot product of four unsigned 8-bit integer values held in each 32-bit element of the two or four first source vectors and four signed 8-bit integer values in the corresponding indexed 32-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to 3.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

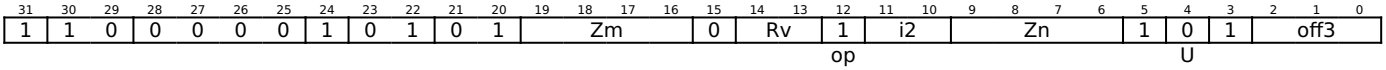
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

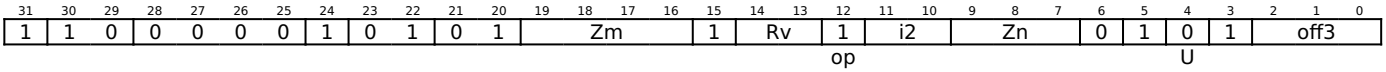
```
USDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
USDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        for i = 0 to 3 do
            let element1 : integer = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4 * s + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USDOT (4-way, multiple and single vector)

Multi-vector unsigned by signed 8-bit integer dot product by vector to 32-bit integer

This instruction calculates the dot product of four unsigned 8-bit integer values held in each 32-bit element of the two or four first source vectors and four signed 8-bit integer values in the corresponding 32-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

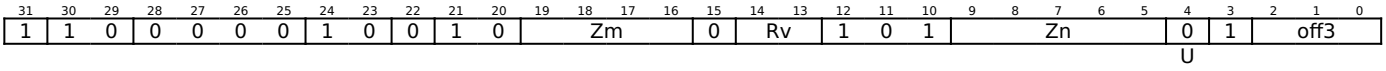
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)



Encoding

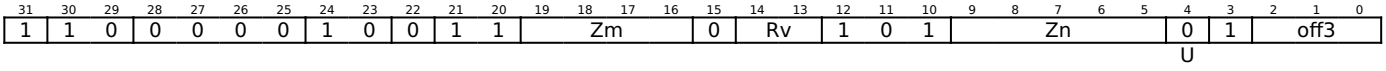
```
USDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)



Encoding

```
USDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let element1 : integer = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4 * e + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USDOT (4-way, multiple vectors)

Multi-vector unsigned by signed 8-bit integer dot product to 32-bit integer

This instruction calculates the dot product of four unsigned 8-bit integer values held in each 32-bit element of the two or four first source vectors and four signed 8-bit integer values in the corresponding 32-bit element of the two or four second source vectors. The widened dot product result is destructively added to corresponding 32-bit element of the ZA single-vector groups.

The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.

The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	1	Zm				0	0	Rv		1	0	1	Zn				0	0	1	off3		

Encoding

```
USDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(off3);
let nreg : integer{} = 2;
```

Four ZA single-vectors

(FEAT_SME2)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	1	Zm			0	1	0	Rv		1	0	1	Zn			0	0	0	1	off3		

Encoding

```
USDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, { <Zm1>.B-<Zm4>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(off3);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA single-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let element1 : integer = UInt(operand1[(4 * e + i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4 * e + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USMLALL (multiple and indexed vector)

Multi-vector unsigned by signed 8-bit integer multiply-add by indexed element to 32-bit integer

This instruction multiplies each unsigned 8-bit element in the one, two, or four first source vectors by each signed 8-bit indexed element of the second source vector, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA quad-vector groups.

The elements within the second source vector are specified using an immediate element index that selects the same element position within each 128-bit vector segment. The element index range is from 0 to one less than the number of elements per 128-bit segment.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

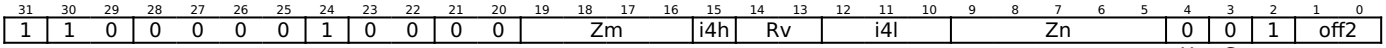
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA quad-vector , Two ZA quad-vectors and Four ZA quad-vectors

One ZA quad-vector

(FEAT_SME2)



Encoding

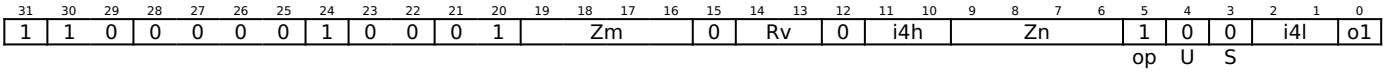
```
USMLALL ZA.S[<Wv>, <offs1>:<offs4>], <Zn>.B, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 1;
```

Two ZA quad-vectors

(FEAT_SME2)



Encoding

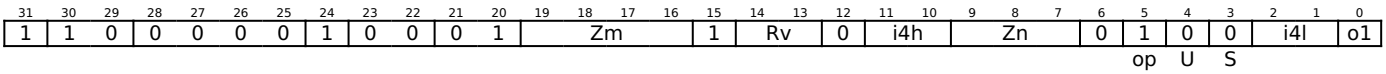
```
USMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2)



Encoding

```
USMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let index : integer = UInt(i4h::i4l);
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA quad-vector" variant: is the first vector select offset, encoded as "off2" field times 4. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the first vector select offset, encoded as "o1" field times 4.
<offs4>	For the "One ZA quad-vector" variant: is the fourth vector select offset, encoded as "off2" field times 4 plus 3. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the element index, in the range 0 to 15, encoded in the "i4h:i4l" fields.
<Zn1>	For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let segmentbase : integer = e - e MOD eltspersegment;
            let s : integer = 4 * segmentbase + index;
            let element1 : integer = UInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[s*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USMLALL (multiple and single vector)

Multi-vector unsigned by signed 8-bit integer multiply-add by vector to 32-bit integer

This instruction multiplies each unsigned 8-bit element in the one, two, or four first source vectors by each signed 8-bit element in the second source vector, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA quad-vector groups.

The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.

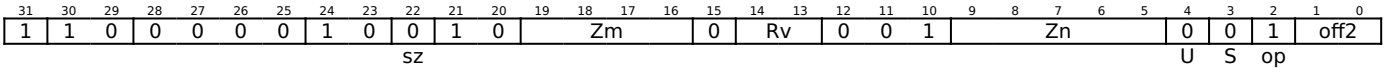
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 3 classes: One ZA quad-vector , Two ZA quad-vectors and Four ZA quad-vectors

One ZA quad-vector

(FEAT_SME2)



Encoding

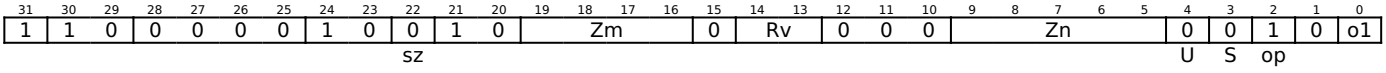
```
USMLALL ZA.S[<Wv>, <offs1>:<offs4>], <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off2::'00');
let nreg : integer{} = 1;
```

Two ZA quad-vectors

(FEAT_SME2)



Encoding

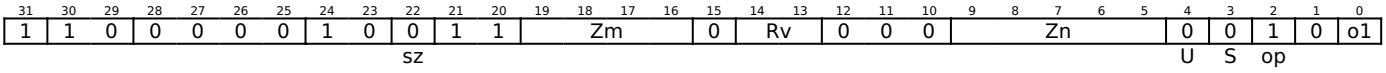
```
USMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2)



Encoding

```
USMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn);
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA quad-vector" variant: is the first vector select offset, encoded as "off2" field times 4. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the first vector select offset, encoded as "o1" field times 4.
<offs4>	For the "One ZA quad-vector" variant: is the fourth vector select offset, encoded as "off2" field times 4 plus 3. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn".
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" plus 1 modulo 32.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" plus 3 modulo 32.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}((n+r) MOD 32);
    let operand2 : bits(VL) = Z{VL}(m);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = UInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USMLALL (multiple vectors)

Multi-vector unsigned by signed 8-bit integer multiply-add to 32-bit integer

This instruction multiplies each unsigned 8-bit element in the two or four first source vectors by each signed 8-bit element in the two or four second source vectors, widens each product to 32 bits, and destructively adds these values to the corresponding 32-bit elements of the ZA quad-vector groups.

The quad-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo half or quarter the number of ZA array vectors.

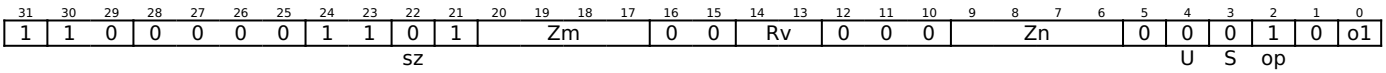
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

It has encodings from 2 classes: Two ZA quad-vectors and Four ZA quad-vectors

Two ZA quad-vectors

(FEAT_SME2)



Encoding

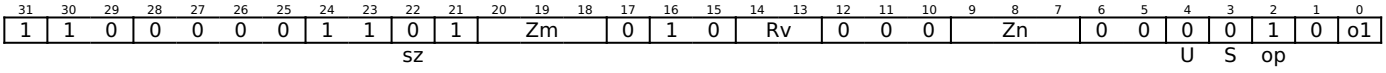
```
USMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx2}], { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm::'0');
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 2;
```

Four ZA quad-vectors

(FEAT_SME2)



Encoding

```
USMLALL ZA.S[<Wv>, <offs1>:<offs4>{, VGx4}], { <Zn1>.B-<Zn4>.B }, { <Zm1>.B-<Zm4>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let v : integer = UInt('010'::Rv);
let n : integer = UInt(Zn::'00');
let m : integer = UInt(Zm::'00');
let offset : integer = UInt(o1::'00');
let nreg : integer{} = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	Is the first vector select offset, encoded as "o1" field times 4.
<offs4>	Is the fourth vector select offset, encoded as "o1" field times 4 plus 3.
<Zn1>	For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2. For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm1>	For the "Two ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 2. For the "Four ZA quad-vectors" variant: is the name of the first scalable vector register of the second source multi-vector group, encoded as "Zm" times 4.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, encoded as "Zm" times 2 plus 1.

<Zn4> Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.

<Zm4> Is the name of the fourth scalable vector register of the second source multi-vector group, encoded as "Zm" times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV nreg;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
var result : bits(VL);
vec = vec - vec MOD 4;

for r = 0 to nreg-1 do
    let operand1 : bits(VL) = Z{VL}(n+r);
    let operand2 : bits(VL) = Z{VL}(m+r);
    for i = 0 to 3 do
        let operand3 : bits(VL) = ZAvector{VL}(vec+i);
        for e = 0 to elements-1 do
            let element1 : integer = UInt(operand1[(4*e+i)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4*e+i)*:(esize DIV 4)]);
            let product : bits(esize) = (element1 * element2)[esize-1:0];
            result[e*:esize] = operand3[e*:esize] + product;
        end;
        ZAvector{VL}(vec+i) = result;
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USMOP4A

Unsigned by signed integer quarter-tile sum of outer products, accumulating

This instruction generates four independent quarter-tile unsigned by signed integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and accumulates the results to the corresponding elements of a 32-bit or 64-bit element ZA tile.
In case of the 8-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 4$ sub-matrix of 8-bit unsigned values held in the half-vectors of the first source vectors by the $4 \times SVL_S \div 2$ sub-matrix of 8-bit signed values held in the half-vectors of the second source vectors. Each 32-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_S \div 2 \times 4$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_S \div 2$ sub-matrix.
In case of the 16-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_D \div 2 \times 4$ sub-matrix of 16-bit unsigned values held in the half-vectors of the first source vectors by the $4 \times SVL_D \div 2$ sub-matrix of 16-bit signed values held in the half-vectors of the second source vectors. Each 64-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_D \div 2 \times 4$ sub-matrix. Similarly, each 64-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_D \div 2$ sub-matrix.
The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products in case of the 8-bit integer variant or $SVL_D \div 2 \times SVL_D \div 2$ widened 64-bit integer sums of outer products in case of the 16-bit integer variant are then destructively added to the 32-bit or 64-bit integer destination tile respectively.
This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 8 classes: 32-bit, single and multiple vectors , 32-bit, single vectors , 32-bit, multiple and single vectors , 32-bit, multiple vectors , 64-bit, single and multiple vectors , 64-bit, single vectors , 64-bit, multiple and single vectors and 64-bit, multiple vectors

32-bit, single and multiple vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	0	1	Zm			0	1	0	0	0	0	0	0	Zn			0	0	0	0	ZAda	
							u0			u1	M											N					S				

Encoding

```
USMOP4A <ZAda>.S, <Zn>.B, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

32-bit, single vectors

(FEAT_SME_MOP4)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	0	0	Zm			0	1	0	0	0	0	0	0	Zn			0	0	0	0	ZAda	
							u0			u1	M											N					S				

Encoding

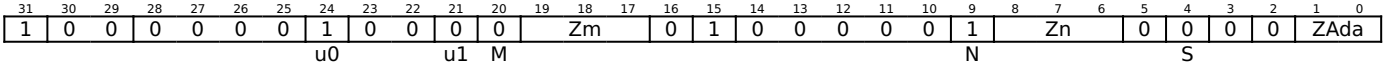
```
USMOP4A <ZAda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

32-bit, multiple and single vectors

(FEAT_SME_MOP4)



Encoding

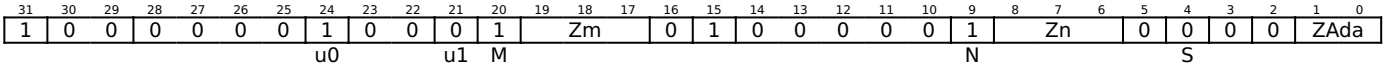
```
USMOP4A <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

32-bit, multiple vectors

(FEAT_SME_MOP4)



Encoding

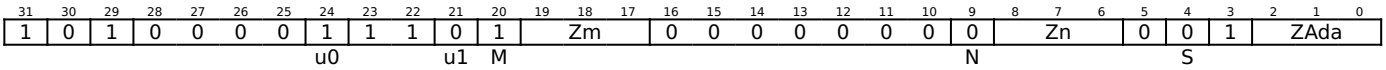
```
USMOP4A <ZAda>.S, { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

64-bit, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

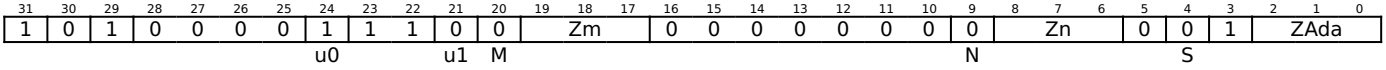
```
USMOP4A <ZAda>.D, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

64-bit, single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

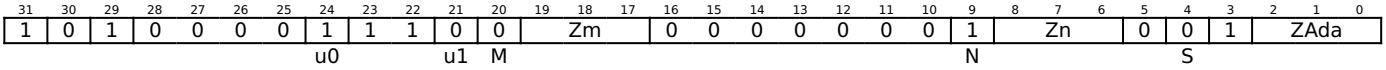
```
USMOP4A <ZAda>.D, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

64-bit, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

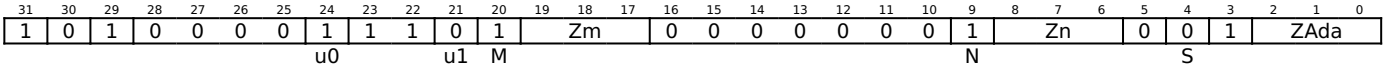
```
USMOP4A <ZAda>.D, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

64-bit, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

```
USMOP4A <ZAda>.D, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = FALSE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	For the "32-bit, multiple and single vectors", "32-bit, multiple vectors", "32-bit, single and multiple vectors", and "32-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit, multiple and single vectors", "64-bit, multiple vectors", "64-bit, single and multiple vectors", and "64-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.

<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 3 do
                let op1elt : bits(esize DIV 4) = op1[(4*row_idx + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = op2[(4*col_idx + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

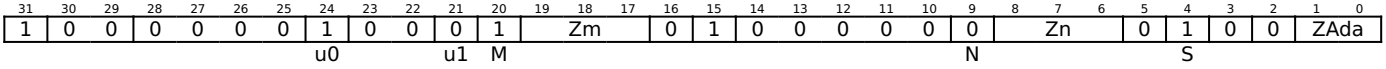
USMOP4S

Unsigned by signed integer quarter-tile sum of outer products, subtracting

This instruction generates four independent quarter-tile unsigned by signed integer sums of outer products from the sub-matrices in the half-vectors of the one or two first and second source vectors and subtracts the results from the corresponding elements of a 32-bit or 64-bit element ZA tile.
In case of the 8-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_S \div 2 \times 4$ sub-matrix of 8-bit unsigned values held in the half-vectors of the first source vectors by the $4 \times SVL_S \div 2$ sub-matrix of 8-bit signed values held in the half-vectors of the second source vectors. Each 32-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_S \div 2 \times 4$ sub-matrix. Similarly, each 32-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_S \div 2$ sub-matrix.
In case of the 16-bit integer variant, each of the quarter-tile sums of outer products is generated by multiplying the $SVL_D \div 2 \times 4$ sub-matrix of 16-bit unsigned values held in the half-vectors of the first source vectors by the $4 \times SVL_D \div 2$ sub-matrix of 16-bit signed values held in the half-vectors of the second source vectors. Each 64-bit container of the half-vectors in the first source vectors holds 4 elements of each row of a $SVL_D \div 2 \times 4$ sub-matrix. Similarly, each 64-bit container of the half-vectors in the second source vector holds 4 elements of each column of a $4 \times SVL_D \div 2$ sub-matrix.
The resulting quarter-tile $SVL_S \div 2 \times SVL_S \div 2$ widened 32-bit integer sums of outer products in case of the 8-bit integer variant or $SVL_D \div 2 \times SVL_D \div 2$ widened 64-bit integer sums of outer products in case of the 16-bit integer variant are then destructively subtracted from the 32-bit or 64-bit integer destination tile respectively.
This is equivalent to performing a 4-way dot product and subtract from each of the destination tile elements.
This instruction is unpredicated.
It has encodings from 8 classes: 32-bit, single and multiple vectors , 32-bit, single vectors , 32-bit, multiple and single vectors , 32-bit, multiple vectors , 64-bit, single and multiple vectors , 64-bit, single vectors , 64-bit, multiple and single vectors and 64-bit, multiple vectors

32-bit, single and multiple vectors

(FEAT_SME_MOP4)



Encoding

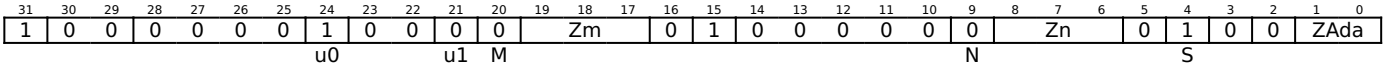
```
USMOP4S <ZAda>.S, <Zn>.B, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

32-bit, single vectors

(FEAT_SME_MOP4)



Encoding

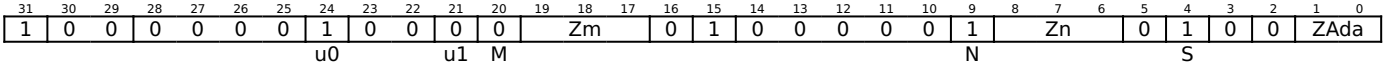
```
USMOP4S <ZAda>.S, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

32-bit, multiple and single vectors

(FEAT_SME_MOP4)



Encoding

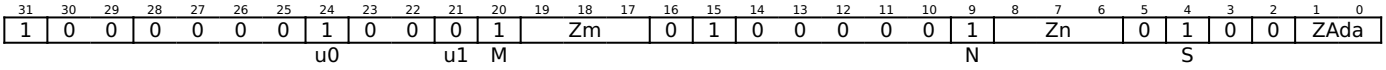
```
USMOP4S <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

32-bit, multiple vectors

(FEAT_SME_MOP4)



Encoding

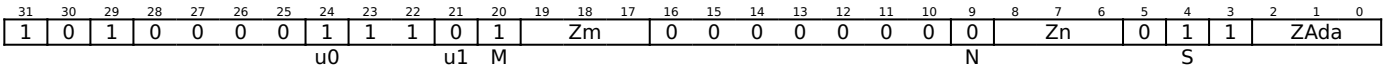
```
USMOP4S <ZAda>.S, { <Zn1>.B-<Zn2>.B }, { <Zm1>.B-<Zm2>.B }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

64-bit, single and multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

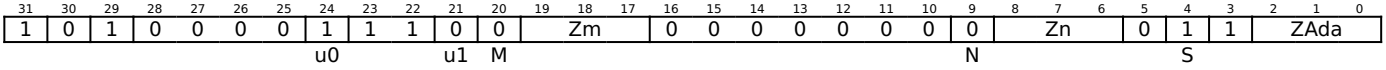
```
USMOP4S <ZAda>.D, <Zn>.H, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

64-bit, single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

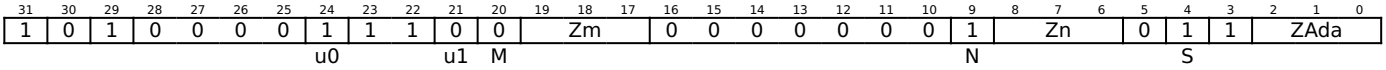
```
USMOP4S <ZAda>.D, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 1;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

64-bit, multiple and single vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

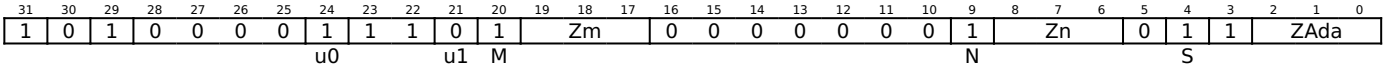
```
USMOP4S <ZAda>.D, { <Zn1>.H-<Zn2>.H }, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 1;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

64-bit, multiple vectors

(FEAT_SME_MOP4 && FEAT_SME_I16I64)



Encoding

```
USMOP4S <ZAda>.D, { <Zn1>.H-<Zn2>.H }, { <Zm1>.H-<Zm2>.H }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_MOP4) || !IsFeatureImplemented(FEAT_SME_I16I64) then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 64;
let n : integer = UInt('0'::Zn::'0');
let m : integer = UInt('1'::Zm::'0');
let nreg : integer{} = 2;
let mreg : integer = 2;
let da : integer = UInt(ZAda);
let sub_op : boolean = TRUE;
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	For the "32-bit, multiple and single vectors", "32-bit, multiple vectors", "32-bit, single and multiple vectors", and "32-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit, multiple and single vectors", "64-bit, multiple vectors", "64-bit, single and multiple vectors", and "64-bit, single vectors" variants: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Zn>	Is the name of the first source scalable vector register, registers in the range Z0-Z15, encoded as "Zn" times 2.
<Zm1>	Is the name of the first scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zm2>	Is the name of the second scalable vector register of the second source multi-vector group, in the range Z16-Z31, encoded as "Zm" times 2 plus 17.

<Zm>	Is the name of the second source scalable vector register, registers in the range Z16-Z31, encoded as "Zm" times 2 plus 16.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, in the range Z0-Z15, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let hvsize : integer{} = VL DIV 2;
let dim : integer{} = hvsize DIV esize;
let tilesize : integer{} = 4 * dim * dim * esize;
let op3 : bits(tilesize) = ZAtile{}(da, esize);
var result : bits(tilesize);
var prod : integer;

for outprod = 0 to 3 do
    let row_hv : integer = outprod DIVRM 2;
    let col_hv : integer = outprod MOD 2;
    let row_base : integer = row_hv * dim;
    let col_base : integer = col_hv * dim;

    let op1 : bits(VL) = Z{}(n + (nreg-1)*col_hv);
    let op2 : bits(VL) = Z{}(m + (mreg-1)*row_hv);

    for row = 0 to dim-1 do
        for col = 0 to dim-1 do
            let row_idx : integer = row_base + row;
            let col_idx : integer = col_base + col;
            let tile_idx : integer = row_idx * dim * 2 + col_idx;

            var sum : bits(esize) = op3[tile_idx*:esize];

            for k = 0 to 3 do
                let op1elt : bits(esize DIV 4) = op1[(4*row_idx + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = op2[(4*col_idx + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                if sub_op then prod = -prod; end;
                sum = sum + prod;
            end;
            result[tile_idx*:esize] = sum;
        end;
    end;
end;
ZAtile{tilesize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USMOPA (4-way)

Unsigned by signed integer sum of outer products, accumulating

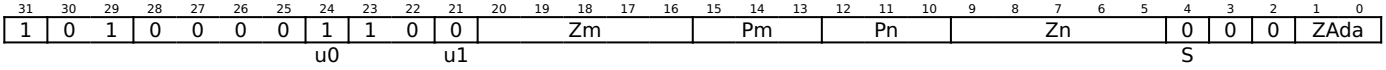
The 8-bit integer variant works with a 32-bit element ZA tile.
The 16-bit integer variant works with a 64-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. In case of the 8-bit integer variant, the first source holds $SVL_S \times 4$ sub-matrix of unsigned 8-bit integer values, and the second source holds $4 \times SVL_S$ sub-matrix of signed 8-bit integer values. In case of the 16-bit integer variant, the first source holds $SVL_D \times 4$ sub-matrix of unsigned 16-bit integer values, and the second source holds $4 \times SVL_D$ sub-matrix of signed 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When an 8-bit source element in case of 8-bit integer variant or a 16-bit source element in case of 16-bit integer variant is Inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer or $SVL_D \times SVL_D$ widened 64-bit integer sum of outer products is then destructively added to the 32-bit integer or 64-bit integer destination tile, respectively for 8-bit integer and 16-bit integer instruction variants. This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.
In case of the 8-bit integer variant, each 32-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_S \times 4$ sub-matrix, and each 32-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_S$ sub-matrix. In case of the 16-bit integer variant, each 64-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_D \times 4$ sub-matrix, and each 64-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_D$ sub-matrix.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SME)



Encoding

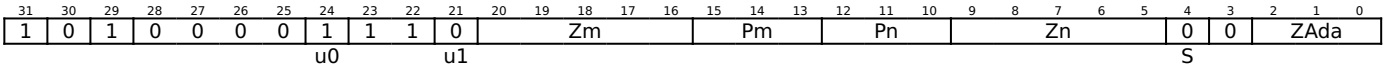
```
USMOPA <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

64-bit

(FEAT_SME_I16I64)



Encoding

```
USMOPA <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.

<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 3 do
            if (ActivePredicateElement{PL}(mask1, 4*row + k, esize DIV 4) &&
                  ActivePredicateElement{PL}(mask2, 4*col + k, esize DIV 4)) then
                let op1elt : bits(esize DIV 4) = operand1[(4*row + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = operand2[(4*col + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum + prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USMOPS

Unsigned by signed integer sum of outer products, subtracting

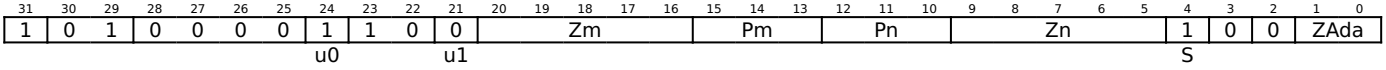
The 8-bit integer variant works with a 32-bit element ZA tile.
The 16-bit integer variant works with a 64-bit element ZA tile.
This instruction multiplies the sub-matrix in the first source vector by the sub-matrix in the second source vector. In case of the 8-bit integer variant, the first source holds $SVL_S \times 4$ sub-matrix of unsigned 8-bit integer values, and the second source holds $4 \times SVL_S$ sub-matrix of signed 8-bit integer values. In case of the 16-bit integer variant, the first source holds $SVL_D \times 4$ sub-matrix of unsigned 16-bit integer values, and the second source holds $4 \times SVL_D$ sub-matrix of signed 16-bit integer values.

Each source vector is independently predicated by a corresponding governing predicate. When an 8-bit source element in case of 8-bit integer variant or a 16-bit source element in case of 16-bit integer variant is Inactive, it is treated as having the value 0.
The resulting $SVL_S \times SVL_S$ widened 32-bit integer or $SVL_D \times SVL_D$ widened 64-bit integer sum of outer products is then destructively subtracted from the 32-bit integer or 64-bit integer destination tile, respectively for 8-bit integer and 16-bit integer instruction variants. This is equivalent to performing a 4-way dot product and subtract from each of the destination tile elements.
In case of the 8-bit integer variant, each 32-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_S \times 4$ sub-matrix, and each 32-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_S$ sub-matrix. In case of the 16-bit integer variant, each 64-bit container of the first source vector holds 4 consecutive column elements of each row of a $SVL_D \times 4$ sub-matrix, and each 64-bit container of the second source vector holds 4 consecutive row elements of each column of a $4 \times SVL_D$ sub-matrix.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.
It has encodings from 2 classes: 32-bit and 64-bit

32-bit

(FEAT_SME)



Encoding

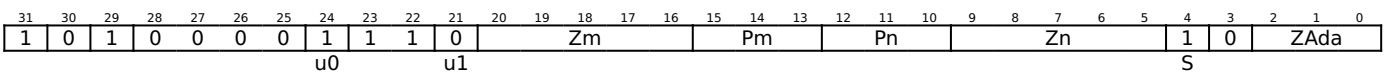
```
USMOPS <ZAda>.S, <Pn>/M, <Pm>/M, <Zn>.B, <Zm>.B
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 32;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

64-bit

(FEAT_SME_I16I64)



Encoding

```
USMOPS <ZAda>.D, <Pn>/M, <Pm>/M, <Zn>.H, <Zm>.H
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_I16I64) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 64;
let a : integer = UInt(Pn);
let b : integer = UInt(Pm);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	For the "32-bit" variant: is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field. For the "64-bit" variant: is the name of the ZA tile ZA0-ZA7, encoded in the "ZAda" field.
<Pn>	Is the name of the first governing scalable predicate register P0-P7, encoded in the "Pn" field.

<Pm>	Is the name of the second governing scalable predicate register P0-P7, encoded in the "Pm" field.
<Zn>	Is the name of the first source scalable vector register, encoded in the "Zn" field.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let PL : integer{} = VL DIV 8;
let dim : integer{} = VL DIV esize;
let mask1 : bits(PL) = P{PL}(a);
let mask2 : bits(PL) = P{PL}(b);
let operand1 : bits(VL) = Z{VL}(n);
let operand2 : bits(VL) = Z{VL}(m);
let operand3 : bits(dim*dim*esize) = ZAtile{dim*dim*esize}(da, esize);
var result : bits(dim*dim*esize);
var prod : integer;

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var sum : bits(esize) = operand3[(row*dim+col)*:esize];
        for k = 0 to 3 do
            if (ActivePredicateElement{PL}(mask1, 4*row + k, esize DIV 4) &&
                  ActivePredicateElement{PL}(mask2, 4*col + k, esize DIV 4)) then
                let op1elt : bits(esize DIV 4) = operand1[(4*row + k)*:(esize DIV 4)];
                let op2elt : bits(esize DIV 4) = operand2[(4*col + k)*:(esize DIV 4)];
                let element1 : integer = if op1_unsigned then UInt(op1elt) else SInt(op1elt);
                let element2 : integer = if op2_unsigned then UInt(op2elt) else SInt(op2elt);
                prod = element1 * element2;
                sum = sum - prod;
            end;
        end;
        result[(row*dim+col)*:esize] = sum;
    end;
end;

ZAtile{dim*dim*esize}(da, esize) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USTMOPA

Unsigned by signed 8-bit integer sparse sum of outer products to 32-bit integer, accumulating

This instruction generates unsigned by signed integer sum of outer products by multiplying the 2-in-4 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

The sum of outer products is generated by multiplying the selected two sets of 2-in-4 8-bit unsigned values from each overlapping 32-bit containers of the two $SVL_S \times 4$ sub-matrices in the first source vectors by the four 8-bit signed values from the corresponding 32-bit container of the $4 \times SVL_S$ sub-matrix in the second source vector. The four selected elements from the overlapping 32-bit containers of the first source vectors correspond to two sets of 2-in-4 elements of rows of two $SVL_S \times 4$ sub-matrices. Each 32-bit container of the second source vector holds 4 elements of columns of a compressed $4 \times SVL_S$ sub-matrix.

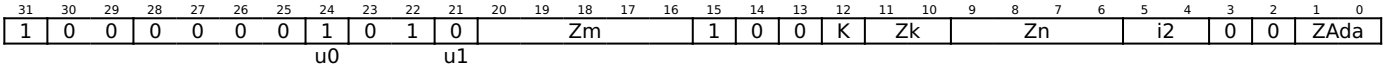
The two sets of 2-in-4 8-bit unsigned values from overlapping 32-bit containers of the first source vectors are selected by pairs of 4-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the sum of products result. If more than two bits of the 4-bit control corresponding to each 32-bit container of the first source vectors are 1, only the elements corresponding to the least two significant bits are selected.

The resulting $SVL_S \times SVL_S$ widened 32-bit integer sum of outer products is then destructively added to the 32-bit integer destination tile. This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP)



Encoding

```
USTMOPA <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = FALSE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 32;
let csize : integer{} = VL DIV 4;
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*32) = ZAtile{}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var erow : array [[4]] of bits(8);
        var ecol : array [[4]] of bits(8);
        for j = 0 to 3 do
            erow[[j]] = Zeros{8};
            ecol[[j]] = op2[(4*col + j)*:8];
        end;
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{}(n+r);
            var i : integer = 0;
            for e = 0 to 3 do
                if i < 2 && ctrl[(8*col + 4*r + e)*:1] == '1' then
                    erow[[2*r + i]] = op1[(4*row + e)*:8];
                    i = i + 1;
                end;
            end;
        end;
        var sum : bits(32) = op4[(row*dim+col)*:32];
        for j = 0 to 3 do
            let erowval : integer = if op1_unsigned then UInt(erow[[j]]) else SInt(erow[[j]]);
            let ecolval : integer = if op2_unsigned then UInt(ecol[[j]]) else SInt(ecol[[j]]);
            sum = sum + erowval * ecolval;
        end;
        result[(row*dim+col)*:32] = sum;
    end;
end;
ZAtile{dim*dim*32}(da, 32) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

USVDOT

Multi-vector unsigned by signed 8-bit integer vertical dot product by indexed element to 32-bit integer

This instruction calculates the vertical dot product of corresponding unsigned 8-bit elements from the four first source vectors and four signed 8-bit integer values in the corresponding indexed 32-bit element of the second source vector. The widened dot product result is destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to 3.

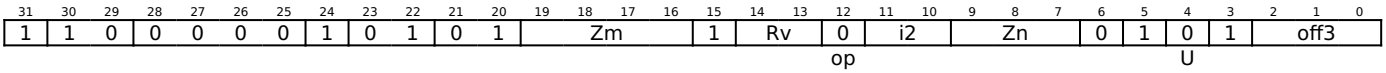
The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.

The vector group symbol VGx4 indicates that the ZA operand consists of four ZA single-vector groups. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
USVDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 4;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
let operand2 : bits(VL) = Z{VL}(m);
var result : bits(VL);

for r = 0 to 3 do
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let operand1 : bits(VL) = Z{VL}(n+i);
            let element1 : integer = UInt(operand1[(4 * e + r)*:(esize DIV 4)]);
            let element2 : integer = SInt(operand2[(4 * s + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UTMOPA (2-way)

Unsigned 16-bit integer sparse sum of outer products to 32-bit integer, accumulating

This instruction generates unsigned integer sum of outer products by multiplying the 2-in-4 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

The sum of outer products is generated by multiplying the selected 2-in-4 16-bit unsigned values from each overlapping 32-bit containers of the two $SVL_S \times 2$ sub-matrices in the first source vectors by the two 16-bit unsigned values from the corresponding 32-bit container of the $2 \times SVL_S$ sub-matrix in the second source vector. The two selected elements from each overlapping 32-bit containers of the first source vectors correspond to 2-in-4 elements of rows of two $SVL_S \times 2$ sub-matrices. Each 32-bit container of the second source vector holds 2 elements of columns of a compressed $2 \times SVL_S$ sub-matrix.

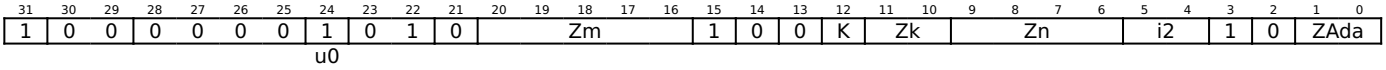
The 2-in-4 16-bit unsigned values from overlapping 32-bit containers of the first source vectors are selected by 4-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the sum of products result. If more than two bits of the 4-bit control corresponding to 32-bit containers of the first source vectors are 1, only the elements corresponding to the least two significant bits are selected.

The resulting $SVL_S \times SVL_S$ widened 32-bit integer sum of outer products is then destructively added to the 32-bit integer destination tile. This is equivalent to performing a 2-way dot product and accumulate to each of the destination tile elements.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP)



Encoding

```
UTMOPA <ZAda>.S, { <Zn1>.H-<Zn2>.H }, <Zm>.H, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
let unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 32;
let csize : integer{} = VL DIV 8;
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*32) = ZAtile{}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var erow : array [[2]] of bits(16);
        var ecol : array [[2]] of bits(16);
        for j = 0 to 1 do
            erow[[j]] = Zeros{16};
            ecol[[j]] = op2[(2*col + j)*:16];
        end;
        var i : integer = 0;
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{}(n+r);
            for e = 0 to 1 do
                if i < 2 && ctrl[(4*col + 2*r + e)*:1] == '1' then
                    erow[[i]] = op1[(2*row + e)*:16];
                    i = i + 1;
                end;
            end;
        end;
        var sum : bits(32) = op4[(row*dim+col)*:32];
        for j = 0 to 1 do
            let erowval : integer = if unsigned then UInt(erow[[j]]) else SInt(erow[[j]]);
            let ecolval : integer = if unsigned then UInt(ecol[[j]]) else SInt(ecol[[j]]);
            sum = sum + erowval * ecolval;
        end;
        result[(row*dim+col)*:32] = sum;
    end;
end;
ZAtile{dim*dim*32}(da, 32) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UTMOPA (4-way)

Unsigned 8-bit integer sparse sum of outer products to 32-bit integer, accumulating

This instruction generates unsigned integer sum of outer products by multiplying the 2-in-4 selected elements from the dense sub-matrices in the two first source vectors with the corresponding elements of the compressed sparse sub-matrix in the second source vector and accumulates the results to the corresponding elements of a 32-bit element ZA tile.

The sum of outer products is generated by multiplying the selected two sets of 2-in-4 8-bit unsigned values from each overlapping 32-bit containers of the two $SVL_S \times 4$ sub-matrices in the first source vectors by the four 8-bit unsigned values from the corresponding 32-bit container of the $4 \times SVL_S$ sub-matrix in the second source vector. The four selected elements from the overlapping 32-bit containers of the first source vectors correspond to two sets of 2-in-4 elements of rows of two $SVL_S \times 4$ sub-matrices. Each 32-bit container of the second source vector holds 4 elements of columns of a compressed $4 \times SVL_S$ sub-matrix.

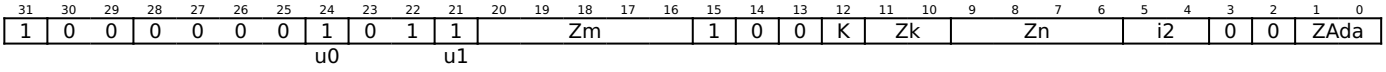
The two sets of 2-in-4 8-bit unsigned values from overlapping 32-bit containers of the first source vectors are selected by pairs of 4-bit controls in the indexed segment of the control vector register. If the control bit corresponding to an element in the first source vectors is 0, the element is discarded and does not contribute to the sum of products result. If more than two bits of the 4-bit control corresponding to each 32-bit container of the first source vectors are 1, only the elements corresponding to the least two significant bits are selected.

The resulting $SVL_S \times SVL_S$ widened 32-bit integer sum of outer products is then destructively added to the 32-bit integer destination tile. This is equivalent to performing a 4-way dot product and accumulate to each of the destination tile elements.

This instruction is unpredicated.

SME2

(FEAT_SME_TMOP)



Encoding

```
UTMOPA <ZAda>.S, { <Zn1>.B-<Zn2>.B }, <Zm>.B, <Zk>[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME_TMOP) then EndOfDecode(Decode_UNDEF); end;
let n : integer = UInt(Zn::'0');
let m : integer = UInt(Zm);
let k : integer = UInt('1'::K::'1'::Zk);
let index : integer = UInt(i2);
let da : integer = UInt(ZAda);
let op1_unsigned : boolean = TRUE;
let op2_unsigned : boolean = TRUE;
```

Assembler Symbols

<ZAda>	Is the name of the ZA tile ZA0-ZA3, encoded in the "ZAda" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register, encoded in the "Zm" field.
<Zk>	Is the name of the control vector register Z20-Z23 or Z28-Z31, encoded as "'1':K:'1':Zk".
<index>	Is the control segment index, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let dim : integer{} = VL DIV 32;
let csize : integer{} = VL DIV 4;
let op2 : bits(VL) = Z{}(m);
let op3 : bits(VL) = Z{}(k);
let ctrl : bits(csize) = op3[index*:csize];
let op4 : bits(dim*dim*32) = ZAtile{}(da, 32);
var result : bits(dim*dim*32);

for row = 0 to dim-1 do
    for col = 0 to dim-1 do
        var erow : array [[4]] of bits(8);
        var ecol : array [[4]] of bits(8);
        for j = 0 to 3 do
            erow[[j]] = Zeros{8};
            ecol[[j]] = op2[(4*col + j)*:8];
        end;
        for r = 0 to 1 do
            let op1 : bits(VL) = Z{}(n+r);
            var i : integer = 0;
            for e = 0 to 3 do
                if i < 2 && ctrl[(8*col + 4*r + e)*:1] == '1' then
                    erow[[2*r + i]] = op1[(4*row + e)*:8];
                    i = i + 1;
                end;
            end;
        end;
        var sum : bits(32) = op4[(row*dim+col)*:32];
        for j = 0 to 3 do
            let erowval : integer = if op1_unsigned then UInt(erow[[j]]) else SInt(erow[[j]]);
            let ecolval : integer = if op2_unsigned then UInt(ecol[[j]]) else SInt(ecol[[j]]);
            sum = sum + erowval * ecolval;
        end;
        result[(row*dim+col)*:32] = sum;
    end;
end;
ZAtile{dim*dim*32}(da, 32) = result;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UUNPK

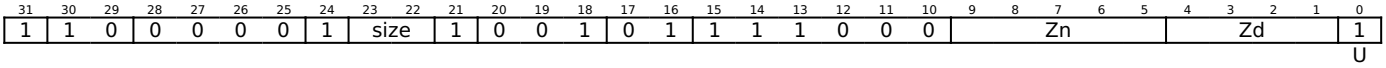
Unpack and zero-extend multi-vector elements

This instruction unpacks and zero-extends elements from one or two source vectors, and writes the zero-extended values into double width elements of the two or four destination vectors. This instruction is unpredicated.

It has encodings from 2 classes: Two registers and Four registers

Two registers

(FEAT_SME2)



Encoding

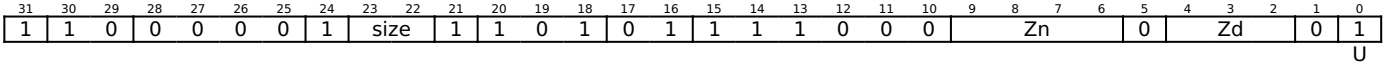
```
UUNPK { <Zd1>.<T>-<Zd2>.<T> }, <Zn>.<Tb>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let d : integer = UInt(Zd::'0');
let nreg : integer{} = 2;
let unsigned : boolean = TRUE;
```

Four registers

(FEAT_SME2)



Encoding

```
UUNPK { <Zd1>.<T>-<Zd4>.<T> }, { <Zn1>.<Tb>-<Zn2>.<Tb> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '00' then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn::'0');
let d : integer = UInt(Zd::'00');
let nreg : integer{} = 4;
let unsigned : boolean = TRUE;
```

Assembler Symbols

<Zd1> For the "Two registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

For the "Four registers" variant: is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	RESERVED
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn> Is the name of the source scalable vector register, encoded in the "Zn" field.

<Tb> Is the size specifier, encoded in “size”:

size	<Tb>
00	RESERVED
01	B
10	H
11	S

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 2.

<Zn2> Is the name of the second scalable vector register of the source multi-vector group, encoded as "Zn" times 2 plus 1.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let hsize : integer{} = esize DIV 2;
let sreg : integer = nreg DIV 2;
var results : array [[4]] of bits(VL);

for r = 0 to sreg-1 do
    let operand : bits(VL) = Z{VL}(n+r);
    for i = 0 to 1 do
        for e = 0 to elements-1 do
            let element : bits(hsize) = operand[(i*elements + e)*:hsize];
            results[[2*r+i]][e*:esize] = Extend{esize}(element, unsigned);
        end;
    end;
end;

for r = 0 to nreg-1 do
    Z{VL}(d+r) = results[[r]];
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UVDOT (2-way)

Multi-vector unsigned 16-bit integer vertical dot product by indexed element to 32-bit integer

This instruction calculates the vertical dot product of the corresponding two unsigned 16-bit integer values held in the two first source vectors and two unsigned 16-bit integer values in the corresponding indexed 32-bit element of the second source vector. The widened dot product results are destructively added to the corresponding 32-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to 3.

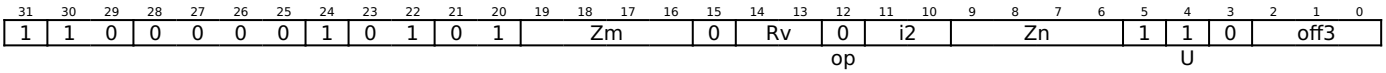
The single-vector group within each half of the ZA array is selected by the sum of the vector select register and offset, modulo half the number of ZA array vectors.

The vector group symbol VGx2 indicates that the ZA operand consists of two ZA single-vector groups. The vector group symbol is preferred for disassembly, but optional in assembler source code.

This instruction is unpredicated.

SME2

(FEAT_SME2)



Encoding

```
UVDOT ZA.S[<Wv>, <offs>{, VGx2}], { <Zn1>.H-<Zn2>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'0');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 2.
<Zn2>	Is the name of the second scalable vector register of the first source multi-vector group, encoded as "Zn" times 2 plus 1.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	Is the immediate index of a pair of 16-bit elements within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 2;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
let operand2 : bits(VL) = Z{VL}(m);
var result : bits(VL);

for r = 0 to 1 do
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 1 do
            let operand1 : bits(VL) = Z{VL}(n+i);
            let element1 : integer = UInt(operand1[(2 * e + r)*:(esize DIV 2)]);
            let element2 : integer = UInt(operand2[(2 * s + i)*:(esize DIV 2)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UVDOT (4-way)

Multi-vector unsigned integer vertical dot product by indexed element

This instruction calculates the vertical dot product of the corresponding four unsigned 8-bit or 16-bit integer values held in the four first source vectors and four unsigned 8-bit or 16-bit integer values in the corresponding indexed 32-bit or 64-bit element of the second source vector. The widened dot product results are destructively added to the corresponding 32-bit or 64-bit element of the ZA single-vector groups.

The groups within the second source vector are specified using an immediate element index that selects the same group position within each 128-bit vector segment. The index range is from 0 to one less than the number of groups per 128-bit segment.

The single-vector group within each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo quarter the number of ZA array vectors.

The vector group symbol VGx4 indicates that the ZA operand consists of four ZA single-vector groups. The vector group symbol is preferred for disassembly, but optional in assembler source code.

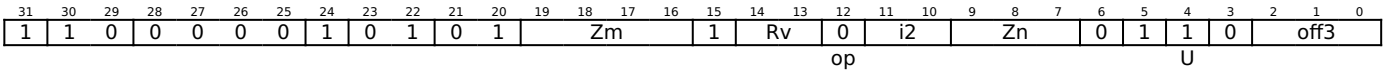
This instruction is unpredicated.

ID_AA64SMFR0_EL1.I16I64 indicates whether the 16-bit integer variant is implemented.

It has encodings from 2 classes: 8-bit to 32-bit and 16-bit to 64-bit

8-bit to 32-bit

(FEAT_SME2)



Encoding

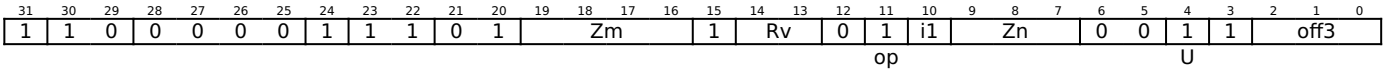
```
UVDOT ZA.S[<Wv>, <offs>{, VGx4}], { <Zn1>.B-<Zn4>.B }, <Zm>.B[<index>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 32;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i2);
```

16-bit to 64-bit

(FEAT_SME2 && FEAT_SME_I16I64)



Encoding

```
UVDOT ZA.D[<Wv>, <offs>{, VGx4}], { <Zn1>.H-<Zn4>.H }, <Zm>.H[<index>]
```

Decode for this encoding

```
if !(IsFeatureImplemented(FEAT_SME2) && IsFeatureImplemented(FEAT_SME_I16I64)) then
    EndOfDecode(Decode_UNDEF);
end;
let v : integer = UInt('010'::Rv);
let esize : integer{} = 64;
let n : integer = UInt(Zn::'00');
let m : integer = UInt('0'::Zm);
let offset : integer = UInt(off3);
let index : integer = UInt(i1);
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.
<Zn1>	Is the name of the first scalable vector register of the first source multi-vector group, encoded as "Zn" times 4.
<Zn4>	Is the name of the fourth scalable vector register of the first source multi-vector group, encoded as "Zn" times 4 plus 3.
<Zm>	Is the name of the second source scalable vector register Z0-Z15, encoded in the "Zm" field.
<index>	For the "8-bit to 32-bit" variant: is the immediate index of a group of four 8-bit values within each 128-bit vector segment, in the range 0 to 3, encoded in the "i2" field. For the "16-bit to 64-bit" variant: is the immediate index of a group of four 16-bit values within each 128-bit vector segment, in the range 0 to 1, encoded in the "i1" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let elements : integer = VL DIV esize;
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV 4;
let eltspersegment : integer = 128 DIV esize;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
let operand2 : bits(VL) = Z{VL}(m);
var result : bits(VL);

for r = 0 to 3 do
    let operand3 : bits(VL) = ZAvector{VL}(vec);
    for e = 0 to elements-1 do
        let segmentbase : integer = e - e MOD eltspersegment;
        let s : integer = segmentbase + index;
        var sum : bits(esize) = operand3[e*:esize];
        for i = 0 to 3 do
            let operand1 : bits(VL) = Z{VL}(n+i);
            let element1 : integer = UInt(operand1[(4 * e + r)*:(esize DIV 4)]);
            let element2 : integer = UInt(operand2[(4 * s + i)*:(esize DIV 4)]);
            sum = sum + element1 * element2;
        end;
        result[e*:esize] = sum;
    end;
    ZAvector{VL}(vec) = result;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UZP (four registers)

Concatenate elements from four vectors

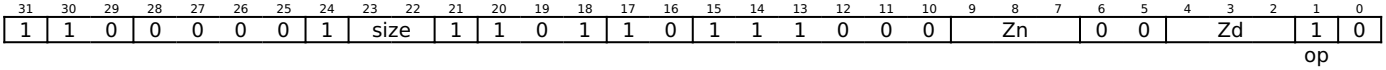
This instruction concatenates every fourth element from each of the four source vectors and places them in the corresponding elements of the four destination vectors.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit to 64-bit elements and 128-bit element

8-bit to 64-bit elements

(FEAT_SME2)



Encoding

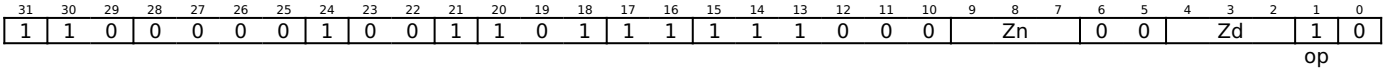
```
UZP { <Zd1>.<T>-<Zd4>.<T> }, { <Zn1>.<T>-<Zn4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '11' && MaxImplementedSVL() < 256 then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
```

128-bit element

(FEAT_SME2)



Encoding

```
UZP { <Zd1>.Q-<Zd4>.Q }, { <Zn1>.Q-<Zn4>.Q }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 512 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
```

Assembler Symbols

<Zd1> Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < esize * 4 then EndOfDecode(Decode_UNDEF); end;
let quads : integer = VL DIV (esize * 4);

var result0 : bits(VL);
var result1 : bits(VL);
var result2 : bits(VL);
var result3 : bits(VL);

for r = 0 to 3 do
    let operand : bits(VL) = Z{VL}(n+r);
    let base : integer = r * quads;
    for q = 0 to quads-1 do
        result0[(base+q)*:esize] = operand[(4*q+0)*:esize];
        result1[(base+q)*:esize] = operand[(4*q+1)*:esize];
        result2[(base+q)*:esize] = operand[(4*q+2)*:esize];
        result3[(base+q)*:esize] = operand[(4*q+3)*:esize];
    end;
end;

Z{VL}(d+0) = result0;
Z{VL}(d+1) = result1;
Z{VL}(d+2) = result2;
Z{VL}(d+3) = result3;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

UZP (two registers)

Concatenate elements from two vectors

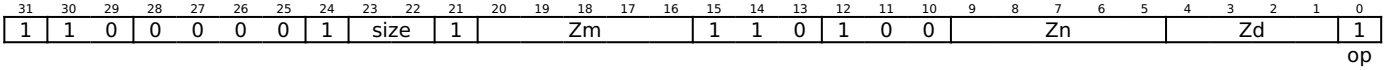
This instruction concatenates every second element from each of the first and second source vectors and places them in the corresponding elements of the two destination vectors.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit to 64-bit elements and 128-bit element

8-bit to 64-bit elements

(FEAT_SME2)



Encoding

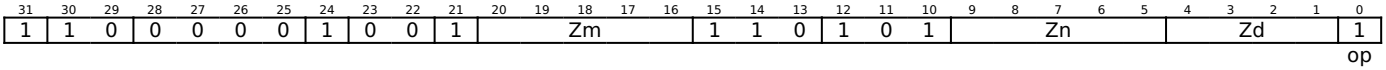
```
UZP { <Zd1>.<T>-<Zd2>.<T> }, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
```

128-bit element

(FEAT_SME2)



Encoding

```
UZP { <Zd1>.Q-<Zd2>.Q }, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
```

Assembler Symbols

<Zd1> Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < esize * 2 then EndOfDecode(Decode_UNDEF); end;
let pairs : integer = VL DIV (esize * 2);

var result0 : bits(VL);
var result1 : bits(VL);

for r = 0 to 1 do
    let base : integer = r * pairs;
    let operand : bits(VL) = if r == 0 then Z{VL}(n) else Z{VL}(m);
    for p = 0 to pairs-1 do
        result0[(base+p)*:esize] = operand[(2*p+0)*:esize];
        result1[(base+p)*:esize] = operand[(2*p+1)*:esize];
    end;
end;

Z{VL}(d+0) = result0;
Z{VL}(d+1) = result1;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

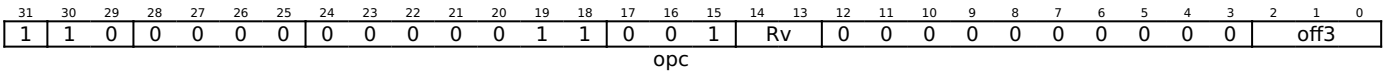
ZERO (double-vector)

Zero ZA double-vector groups

This instruction zeroes one, two, or four ZA double-vector groups.
The double-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA double-vector groups respectively.
It has encodings from 3 classes: One ZA double-vector , Two ZA double-vectors and Four ZA double-vectors

One ZA double-vector

(FEAT_SME2p1)



Encoding

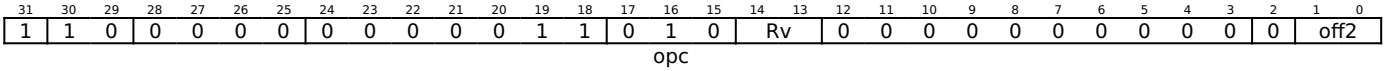
```
ZERO ZA.D[<Wv>, <offs1>:<offs2>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off3::'0');
let ngrp : integer = 1;
let nvec : integer = 2;
```

Two ZA double-vectors

(FEAT_SME2p1)



Encoding

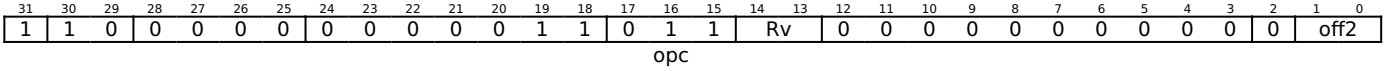
```
ZERO ZA.D[<Wv>, <offs1>:<offs2>, VGx2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off2::'0');
let ngrp : integer = 2;
let nvec : integer = 2;
```

Four ZA double-vectors

(FEAT_SME2p1)



Encoding

```
ZERO ZA.D[<Wv>, <offs1>:<offs2>, VGx4]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off2::'0');
let ngrp : integer = 4;
let nvec : integer = 2;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA double-vector" variant: is the first vector select offset, encoded as "off3" field times 2. For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the first vector select offset, encoded as "off2" field times 2.
<offs2>	For the "One ZA double-vector" variant: is the second vector select offset, encoded as "off3" field times 2 plus 1.

For the "Four ZA double-vectors" and "Two ZA double-vectors" variants: is the second vector select offset, encoded as "off2" field times 2 plus 1.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV ngrp;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
vec = vec - vec MOD nvec;
for r = 0 to ngrp-1 do
    for i = 0 to nvec-1 do
        ZAvector{VL}(vec + i) = Zeros{VL};
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

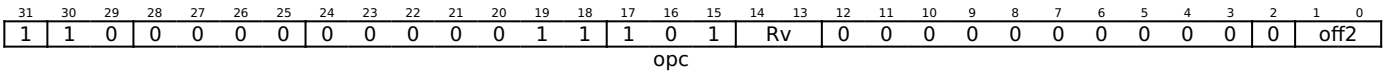
ZERO (quad-vector)

Zero ZA quad-vector groups

This instruction zeroes one, two, or four ZA quad-vector groups.
The quad-vector group within all of, each half of, or each quarter of the ZA array is selected by the sum of the vector select register and offset range, modulo all, half, or quarter the number of ZA array vectors.
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA quad-vector groups respectively.
It has encodings from 3 classes: One ZA quad-vector , Two ZA quad-vectors and Four ZA quad-vectors

One ZA quad-vector

(FEAT_SME2p1)



Encoding

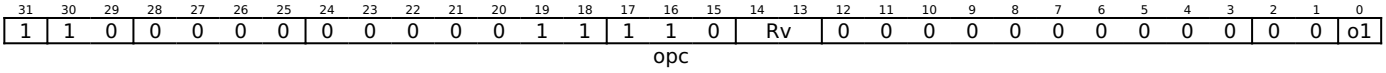
```
ZERO ZA.D[<Wv>, <offs1>:<offs4>]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off2::'00');
let ngrp : integer = 1;
let nvec : integer = 4;
```

Two ZA quad-vectors

(FEAT_SME2p1)



Encoding

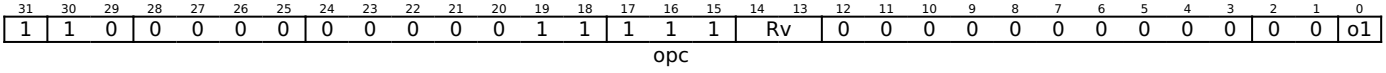
```
ZERO ZA.D[<Wv>, <offs1>:<offs4>, VGx2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(o1::'00');
let ngrp : integer = 2;
let nvec : integer = 4;
```

Four ZA quad-vectors

(FEAT_SME2p1)



Encoding

```
ZERO ZA.D[<Wv>, <offs1>:<offs4>, VGx4]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(o1::'00');
let ngrp : integer = 4;
let nvec : integer = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs1>	For the "One ZA quad-vector" variant: is the first vector select offset, encoded as "off2" field times 4. For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the first vector select offset, encoded as "o1" field times 4.
<offs4>	For the "One ZA quad-vector" variant: is the fourth vector select offset, encoded as "off2" field times 4 plus 3.

For the "Four ZA quad-vectors" and "Two ZA quad-vectors" variants: is the fourth vector select offset, encoded as "o1" field times 4 plus 3.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV ngrp;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
vec = vec - vec MOD nvec;
for r = 0 to ngrp-1 do
    for i = 0 to nvec-1 do
        ZAvector{VL}(vec + i) = Zeros{VL};
    end;
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

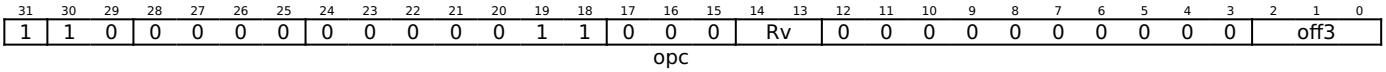
ZERO (single-vector)

Zero ZA single-vector groups

This instruction zeroes two or four ZA single-vector groups.
The single-vector group within each half of or each quarter of the ZA array is selected by the sum of the vector select register and offset, modulo half or quarter the number of ZA array vectors.
The vector group symbol, VGx2 or VGx4, indicates that the ZA operand consists of two or four ZA single-vector groups respectively.
It has encodings from 2 classes: Two ZA single-vectors and Four ZA single-vectors

Two ZA single-vectors

(FEAT_SME2p1)



Encoding

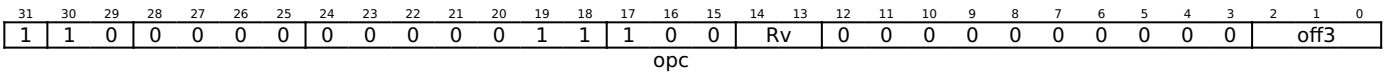
```
ZERO ZA.D[<Wv>, <offs>, VGx2]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off3);
let ngrp : integer = 2;
```

Four ZA single-vectors

(FEAT_SME2p1)



Encoding

```
ZERO ZA.D[<Wv>, <offs>, VGx4]
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2p1) then EndOfDecode(Decode_UNDEF); end;
let v : integer = UInt('010'::Rv);
let offset : integer = UInt(off3);
let ngrp : integer = 4;
```

Assembler Symbols

<Wv>	Is the 32-bit name of the vector select register W8-W11, encoded as "Rv" plus 8.
<offs>	Is the vector select offset, in the range 0 to 7, encoded in the "off3" field.

Operation

```
CheckStreamingSVEAndZAEnabled();
let VL : integer{} = CurrentVL();
let vectors : integer = VL DIV 8;
let vstride : integer = vectors DIV ngrp;
let vbase : bits(32) = X{32}(v);
var vec : integer = (UInt(vbase) + offset) MOD vstride;
for r = 0 to ngrp-1 do
    ZAvector{VL}(vec) = Zeros{VL};
    vec = vec + vstride;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

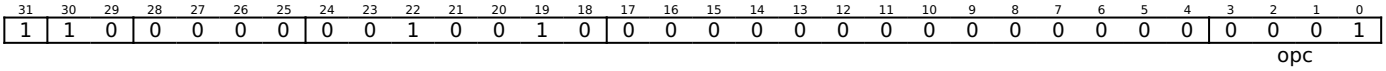
ZERO (table)

Zero ZT0

This instruction zeroes all bytes of the ZT0 register.
This instruction does not require the PE to be in Streaming SVE mode, and it is expected that this instruction will not experience a significant slowdown due to contention with other PEs that are executing in Streaming SVE mode.

SME2

(FEAT_SME2)



Encoding

```
ZERO { ZT0 }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
```

Operation

```
CheckSMEEnabled();
CheckSMEZT0Enabled();
ZT0{512}() = Zeros{512};
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ZERO (tiles)

Zero a list of 64-bit element ZA tiles

This instruction zeroes all bytes within each of the up to eight listed 64-bit element tiles named ZA0.D to ZA7.D, leaving the other 64-bit element tiles unmodified.

This instruction does not require the PE to be in Streaming SVE mode, and it is expected that this instruction will not experience a significant slowdown due to contention with other PEs that are executing in Streaming SVE mode.

For programmer convenience, an assembler must also accept the names of 32-bit, 16-bit, and 8-bit element tiles that are converted into the corresponding set of 64-bit element tiles.

In accordance with the architecturally defined mapping between different element size tiles:

- Zeroing the 8-bit element tile name ZA0.B, or the entire array name ZA, is equivalent to zeroing all eight 64-bit element tiles named ZA0.D to ZA7.D.
- Zeroing the 16-bit element tile name ZA0.H is equivalent to zeroing 64-bit element tiles named ZA0.D, ZA2.D, ZA4.D, and ZA6.D.
- Zeroing the 16-bit element tile name ZA1.H is equivalent to zeroing 64-bit element tiles named ZA1.D, ZA3.D, ZA5.D, and ZA7.D.
- Zeroing the 32-bit element tile name ZA0.S is equivalent to zeroing 64-bit element tiles named ZA0.D and ZA4.D.
- Zeroing the 32-bit element tile name ZA1.S is equivalent to zeroing 64-bit element tiles named ZA1.D and ZA5.D.
- Zeroing the 32-bit element tile name ZA2.S is equivalent to zeroing 64-bit element tiles named ZA2.D and ZA6.D.
- Zeroing the 32-bit element tile name ZA3.S is equivalent to zeroing 64-bit element tiles named ZA3.D and ZA7.D.

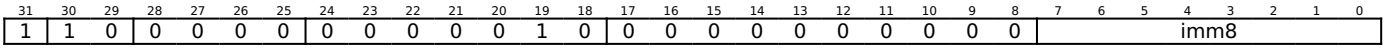
The preferred disassembly of this instruction uses the shortest list of tile names that represent the encoded immediate mask.

For example:

- An immediate that encodes 64-bit element tiles ZA0.D, ZA1.D, ZA4.D, and ZA5.D is disassembled as {ZA0.S, ZA1.S}.
- An immediate that encodes 64-bit element tiles ZA0.D, ZA2.D, ZA4.D, and ZA6.D is disassembled as {ZA0.H}.
- An all-ones immediate is disassembled as {ZA}.
- An all-zeros immediate is disassembled as an empty list { }.

SME

(FEAT_SME)



Encoding

```
ZERO { {<mask>} }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME) then EndOfDecode(Decode_UNDEF); end;
let mask : bits(8) = imm8;
let esize : integer{} = 64;
```

Assembler Symbols

<mask>	Is the optional list of up to eight 64-bit element tile names separated by commas, encoded in the "imm8" field.

Operation

```
CheckSMEAndZAEnabled();
let SVL : integer{} = CurrentSVL();
let dim : integer{} = SVL DIV esize;
let result : bits(dim*dim*esize) = Zeros{};

for i = 0 to 7 do
    if mask[i] == '1' then ZAtile{dim*dim*esize}(i, esize) = result; end;
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

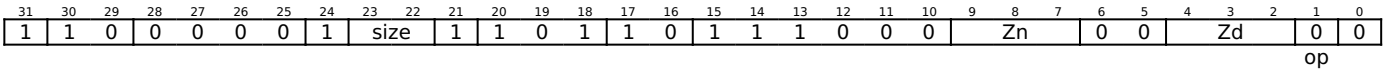
ZIP (four registers)

Interleave elements from four vectors

This instruction places the four-way interleaved elements from the four source vectors in the corresponding elements of the four destination vectors. This instruction is unpredicated.
It has encodings from 2 classes: 8-bit to 64-bit elements and 128-bit element

8-bit to 64-bit elements

(FEAT_SME2)



Encoding

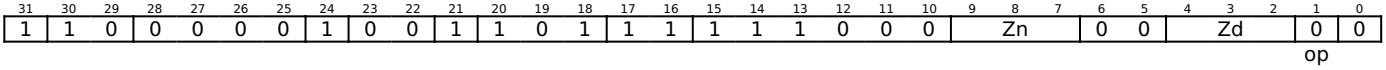
```
ZIP { <Zd1>.<T>-<Zd4>.<T> }, { <Zn1>.<T>-<Zn4>.<T> }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if size == '11' && MaxImplementedSVL() < 256 then
    EndOfDecode(Decode_UNDEF);
end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
```

128-bit element

(FEAT_SME2)



Encoding

```
ZIP { <Zd1>.Q-<Zd4>.Q }, { <Zn1>.Q-<Zn4>.Q }
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 512 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn::'00');
let d : integer = UInt(Zd::'00');
```

Assembler Symbols

<Zd1> Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 4.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zd4> Is the name of the fourth scalable vector register of the destination multi-vector group, encoded as "Zd" times 4 plus 3.

<Zn1> Is the name of the first scalable vector register of the source multi-vector group, encoded as "Zn" times 4.

<Zn4> Is the name of the fourth scalable vector register of the source multi-vector group, encoded as "Zn" times 4 plus 3.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < esize * 4 then EndOfDecode(Decode_UNDEF); end;
let quads : integer = VL DIV (esize * 4);

let operand0 : bits(VL) = Z{VL}(n);
let operand1 : bits(VL) = Z{VL}(n+1);
let operand2 : bits(VL) = Z{VL}(n+2);
let operand3 : bits(VL) = Z{VL}(n+3);
var result : bits(VL);

for r = 0 to 3 do
    let base : integer = r * quads;
    for q = 0 to quads-1 do
        result[(4*q+0)*:esize] = operand0[(base+q)*:esize];
        result[(4*q+1)*:esize] = operand1[(base+q)*:esize];
        result[(4*q+2)*:esize] = operand2[(base+q)*:esize];
        result[(4*q+3)*:esize] = operand3[(base+q)*:esize];
    end;
    Z{VL}(d+r) = result;
end;
```

Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

ZIP (two registers)

Interleave elements from two vectors

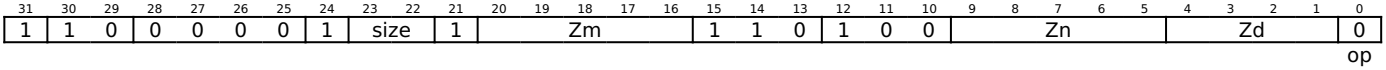
This instruction places the two-way interleaved elements from the first and second source vectors in the corresponding elements of the two destination vectors.

This instruction is unpredicated.

It has encodings from 2 classes: 8-bit to 64-bit elements and 128-bit element

8-bit to 64-bit elements

(FEAT_SME2)



Encoding

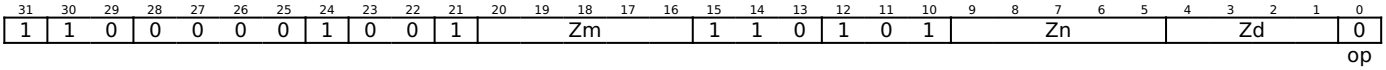
```
ZIP { <Zd1>.<T>-<Zd2>.<T> }, <Zn>.<T>, <Zm>.<T>
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 8 << UInt(size);
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
```

128-bit element

(FEAT_SME2)



Encoding

```
ZIP { <Zd1>.Q-<Zd2>.Q }, <Zn>.Q, <Zm>.Q
```

Decode for this encoding

```
if !IsFeatureImplemented(FEAT_SME2) then EndOfDecode(Decode_UNDEF); end;
if MaxImplementedSVL() < 256 then EndOfDecode(Decode_UNDEF); end;
let esize : integer{} = 128;
let n : integer = UInt(Zn);
let m : integer = UInt(Zm);
let d : integer = UInt(Zd::'0');
```

Assembler Symbols

<Zd1> Is the name of the first scalable vector register of the destination multi-vector group, encoded as "Zd" times 2.

<T> Is the size specifier, encoded in “size”:

size	<T>
00	B
01	H
10	S
11	D

<Zd2> Is the name of the second scalable vector register of the destination multi-vector group, encoded as "Zd" times 2 plus 1.

<Zn> Is the name of the first source scalable vector register, encoded in the "Zn" field.

<Zm> Is the name of the second source scalable vector register, encoded in the "Zm" field.

Operation

```
CheckStreamingSVEEnabled();
let VL : integer{} = CurrentVL();
if VL < esize * 2 then EndOfDecode(Decode_UNDEF); end;
let pairs : integer = VL DIV (esize * 2);

let operand0 : bits(VL) = Z{VL}(n);
let operand1 : bits(VL) = Z{VL}(m);
var result : bits(VL);

for r = 0 to 1 do
    let base : integer = r * pairs;
    for p = 0 to pairs-1 do
        result[(2*p+0)*:esize] = operand0[(base+p)*:esize];
        result[(2*p+1)*:esize] = operand1[(base+p)*:esize];
    end;
    Z{VL}(d+r) = result;
end;
```

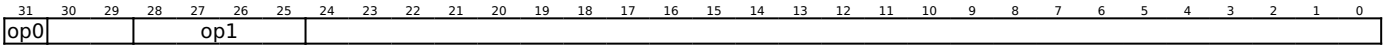
Operational information

This instruction is a data-independent-time instruction as described in *About PSTATE.DIT*.

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.

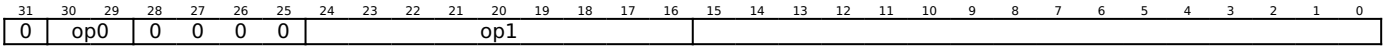
A64 instruction set encoding



Decode fields		Instruction details
op0	op1	
0	0000	Reserved
1	0000	SME encodings
	0010	SVE encodings
	00x1	UNALLOCATED
	100x	Data Processing -- Immediate
	101x	Branches, Exception Generating and System instructions
	x101	Data Processing -- Register
	x111	Data Processing -- Scalar Floating-Point and Advanced SIMD
	x1x0	Loads and Stores

Reserved

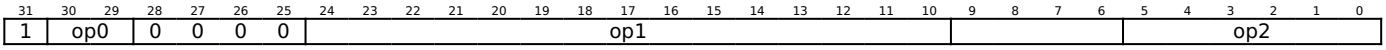
The encodings in this section are decoded from A64 instruction set encoding.



Decode fields		Instruction details
op0	op1	
00	000000000	UDF
00	!= 000000000	UNALLOCATED
!= 00		UNALLOCATED

SME encodings

The encodings in this section are decoded from A64 instruction set encoding.

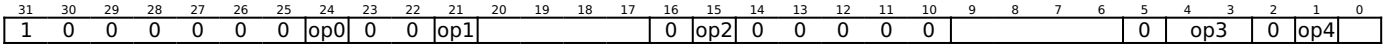


Decode fields			Instruction details
op0	op1	op2	
00	x00xxxxx0x00000	0xx0xx	SME2 Quarter Tile Outer Product - 16-bit and 32-bit
00	x00xxxxx0x00000	1xx0xx	UNALLOCATED
00	x00xxxxx0x00001	xxx0xx	UNALLOCATED
00	x00xxxxx0x0001x	xxx0xx	UNALLOCATED
00	x00xxxxx0x001xx	xxx0xx	UNALLOCATED
00	x00xxxxx0x01xxx	xxx0xx	UNALLOCATED
00	x00xxxxx1x0xxxx	xxx0xx	UNALLOCATED
00	x01xxxxxxx0xxxx	xxx0xx	SME2 Sparse Outer Product
00	x0xxxxxxxx0xxxx	xxx1xx	UNALLOCATED
00	x0xxxxxxxx1xxxx		UNALLOCATED
00	x10xxxxxxxxxxxx	xx00xx	SME FP Outer Product - 32 bit
00	x10xxxxxxxxxxxx	xx10xx	SME2 Outer Product - Misc
01	00xxxxxxxxxxxx		SME2 Multi-vector - Memory (Contiguous)
01	10xxxxxxxxxxxx		SME2 Multi-vector - Memory (Strided)
01	x10xxxxxxxxxxx	xxx0xx	SME Integer Outer Product - 32 bit
0x	x10xxxxxxxxxxx	xxx1xx	UNALLOCATED
0x	x11xxxxx0000000	0x1xxx	SME2 Quarter Tile Outer Product - 64-bit
0x	x11xxxxx0000000	1x1xxx	UNALLOCATED
0x	x11xxxxx0000001	xx1xxx	UNALLOCATED
0x	x11xxxxx000001x	xx1xxx	UNALLOCATED
0x	x11xxxxx00001xx	xx1xxx	UNALLOCATED
0x	x11xxxxx0001xxx	xx1xxx	UNALLOCATED
0x	x11xxxxx001xxxx	xx1xxx	UNALLOCATED
0x	x11xxxxx01xxxxx	xx1xxx	UNALLOCATED
0x	x11xxxxx1xxxxxx	xx1xxx	UNALLOCATED
0x	x11xxxxxxxxxxx	xx0xxx	SME Outer Product - 64 bit
10	0000010xxxxxxxx		SME zero array
10	0000011xxxxxxxx		SME2 Multiple Zero
10	0010010xxxxxxxx		SME2 zero lookup table
10	0010011xxxxxxxx		SME2 Move Lookup Table
10	00x011xxxxxxxxx		UNALLOCATED
10	010011xxxxxxxxx		SME2 Expand Lookup Table (Non-contiguous)
10	011011xxxxxxxxx		UNALLOCATED
10	01x001xxxxxxxxx		SME2 Expand Lookup Table (Contiguous)
10	0xx000x0xxxxxxx	x0xxxx	SME Move into Array
10	0xx000x0xxxxxxx	x1xxxx	UNALLOCATED
10	0xx000x1xxxxxxx		SME Move from Array
10	0xx010xxxxxxxxx	xx0xxx	SME Add Vector to Array
10	0xx010xxxxxxxxx	xx1xxx	UNALLOCATED
10	0xx1xxxxxxxxxx		UNALLOCATED
10	10x10xxxx0xxxxx		SME2 Multi-vector - Multiple and Single Array Vectors (Two registers)
10	10x11xxxx0xxxxx		SME2 Multi-vector - Multiple and Single Array Vectors (Four registers)
10	10x1xxxxx1111xx		SME2 Multi-vector - Lookup table with 6-bit indices
10	11x1xxxx00xxxxx		SME2 Multi-vector - Multiple Array Vectors (Two registers)
10	11x1xxxx10xxxxx		SME2 Multi-vector - Multiple Array Vectors (Four registers)

10	11x1xxxxx1111xx		UNALLOCATED
10	1xx00xxxxxxxxx		SME2 Multi-vector - Indexed (One register)
10	1xx01xxxx0xxxx		SME2 Multi-vector - Indexed (Two registers)
10	1xx01xxxx1xxxx		SME2 Multi-vector - Indexed (Four registers)
10	1xx10xxxx10100x		SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers)
10	1xx10xxxx10101x	xxxx0x	SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers)
10	1xx10xxxx10101x	xxxx1x	UNALLOCATED
10	1xx11xxxx1010xx		UNALLOCATED
10	1xx1xxx00101110	xxxx0x	SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers)
10	1xx1xxx00101111	xxxx0x	SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Four registers)
10	1xx1xxx1010111x	xxxx0x	UNALLOCATED
10	1xx1xxxx0101100		SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers)
10	1xx1xxxx0101101		SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Two registers)
10	1xx1xxxx010111x	xxxx1x	UNALLOCATED
10	1xx1xxxx11011xx		UNALLOCATED
10	1xx1xxxxx100xxx		SME2 Multi-vector - SVE Select
10	1xx1xxxxx110xxx		SME2 Multi-vector - SVE Constructive Binary
10	1xx1xxxxx111000		SME2 Multi-vector - SVE Constructive Unary
10	1xx1xxxxx111001	0xxxx0	SME2 Multi-vector - FP Multiply
10	1xx1xxxxx111001	!= 0xxxx0	UNALLOCATED
10	1xx1xxxxx111010	0xxxx0	SME2 Multiple and Single Vector - FP multiply
10	1xx1xxxxx111010	!= 0xxxx0	UNALLOCATED
10	1xx1xxxxx111011		UNALLOCATED
11			SME Memory

SME2 Quarter Tile Outer Product - 16-bit and 32-bit

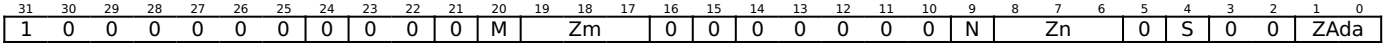
The encodings in this section are decoded from SME encodings.



Decode fields					Instruction details
op0	op1	op2	op3	op4	
0	0	0	x0		SME2 FP32 non-widening quarter tile outer product
0	0	0	x1	0	UNALLOCATED
0	1	0	00		SME2 FP8 to FP32 quarter tile outer product
0	1	0	01	0	SME2 FP8 to FP16 quarter tile outer product
0	1	0	10	1	UNALLOCATED
0	1	0	1x	0	UNALLOCATED
1	0	0	x0		SME2 BFloat16 to FP32 quarter tile outer product
1	0	0	x1	0	SME2 FP16 non-widening quarter tile outer product
1	1	0	x0		SME2 FP16 to FP32 quarter tile outer product
1	1	0	x1	0	SME2 BFloat16 non-widening quarter tile outer product
	0	1	x1		SME2 Int16 to Int32 quarter tile outer product
	1	1	x1		UNALLOCATED
		0	x1	1	UNALLOCATED
		1	x0		SME2 Int8 to Int32 quarter tile outer product

SME2 FP32 non-widening quarter tile outer product

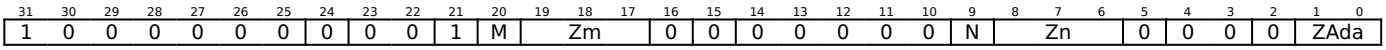
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.



Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	FMOP4A (non-widening) — Single-precision, single vectors	FEAT_SME_MOP4
0	0	1	FMOP4S (non-widening) — Single-precision, single vectors	FEAT_SME_MOP4
0	1	0	FMOP4A (non-widening) — Single-precision, multiple and single vectors	FEAT_SME_MOP4
0	1	1	FMOP4S (non-widening) — Single-precision, multiple and single vectors	FEAT_SME_MOP4
1	0	0	FMOP4A (non-widening) — Single-precision, single and multiple vectors	FEAT_SME_MOP4
1	0	1	FMOP4S (non-widening) — Single-precision, single and multiple vectors	FEAT_SME_MOP4
1	1	0	FMOP4A (non-widening) — Single-precision, multiple vectors	FEAT_SME_MOP4
1	1	1	FMOP4S (non-widening) — Single-precision, multiple vectors	FEAT_SME_MOP4

SME2 FP8 to FP32 quarter tile outer product

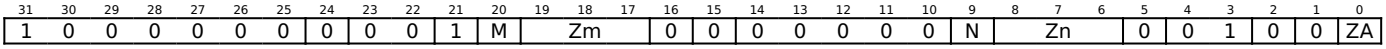
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.



Decode fields		Instruction Details	Condition
M	N		
0	0	FMOP4A (widening, 4-way) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32
0	1	FMOP4A (widening, 4-way) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32
1	0	FMOP4A (widening, 4-way) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32
1	1	FMOP4A (widening, 4-way) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F32

SME2 FP8 to FP16 quarter tile outer product

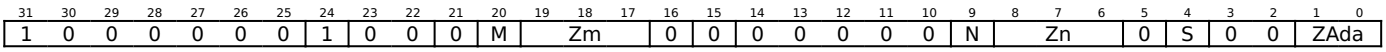
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.



Decode fields		Instruction Details	Condition
M	N		
0	0	FMOP4A (widening, 2-way, FP8 to FP16) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16
0	1	FMOP4A (widening, 2-way, FP8 to FP16) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16
1	0	FMOP4A (widening, 2-way, FP8 to FP16) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16
1	1	FMOP4A (widening, 2-way, FP8 to FP16) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F8F16

SME2 BFloat16 to FP32 quarter tile outer product

The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.



Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	BFMOP4A (widening) — Single vectors	FEAT_SME_MOP4
0	0	1	BFMOP4S (widening) — Single vectors	FEAT_SME_MOP4
0	1	0	BFMOP4A (widening) — Multiple and single vectors	FEAT_SME_MOP4
0	1	1	BFMOP4S (widening) — Multiple and single vectors	FEAT_SME_MOP4
1	0	0	BFMOP4A (widening) — Single and multiple vectors	FEAT_SME_MOP4
1	0	1	BFMOP4S (widening) — Single and multiple vectors	FEAT_SME_MOP4
1	1	0	BFMOP4A (widening) — Multiple vectors	FEAT_SME_MOP4
1	1	1	BFMOP4S (widening) — Multiple vectors	FEAT_SME_MOP4

SME2 FP16 non-widening quarter tile outer product

The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	0	M	Zm			0	0	0	0	0	0	0	N	Zn			0	S	1	0	0	ZA

Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	FMOP4A (non-widening) — Half-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
0	0	1	FMOP4S (non-widening) — Half-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
0	1	0	FMOP4A (non-widening) — Half-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
0	1	1	FMOP4S (non-widening) — Half-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	0	0	FMOP4A (non-widening) — Half-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	0	1	FMOP4S (non-widening) — Half-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	1	0	FMOP4A (non-widening) — Half-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16
1	1	1	FMOP4S (non-widening) — Half-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F16F16

SME2 FP16 to FP32 quarter tile outer product

The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	1	M	Zm			0	0	0	0	0	0	0	N	Zn			0	S	0	0	ZAda	

Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	FMOP4A (widening, 2-way, FP16 to FP32) — Single vectors	FEAT_SME_MOP4
0	0	1	FMOP4S (widening) — Single vectors	FEAT_SME_MOP4
0	1	0	FMOP4A (widening, 2-way, FP16 to FP32) — Multiple and single vectors	FEAT_SME_MOP4
0	1	1	FMOP4S (widening) — Multiple and single vectors	FEAT_SME_MOP4
1	0	0	FMOP4A (widening, 2-way, FP16 to FP32) — Single and multiple vectors	FEAT_SME_MOP4
1	0	1	FMOP4S (widening) — Single and multiple vectors	FEAT_SME_MOP4
1	1	0	FMOP4A (widening, 2-way, FP16 to FP32) — Multiple vectors	FEAT_SME_MOP4
1	1	1	FMOP4S (widening) — Multiple vectors	FEAT_SME_MOP4

SME2 BFloat16 non-widening quarter tile outer product

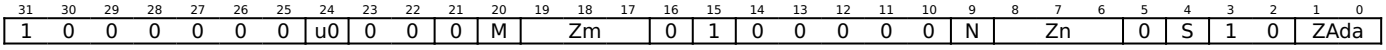
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	1	0	0	1	M	Zm			0	0	0	0	0	0	0	N	Zn			0	S	1	0	0	ZA

Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	BFMOP4A (non-widening) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
0	0	1	BFMOP4S (non-widening) — Single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
0	1	0	BFMOP4A (non-widening) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
0	1	1	BFMOP4S (non-widening) — Multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	0	0	BFMOP4A (non-widening) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	0	1	BFMOP4S (non-widening) — Single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	1	0	BFMOP4A (non-widening) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16
1	1	1	BFMOP4S (non-widening) — Multiple vectors	FEAT_SME_MOP4 && FEAT_SME_B16B16

SME2 Int16 to Int32 quarter tile outer product

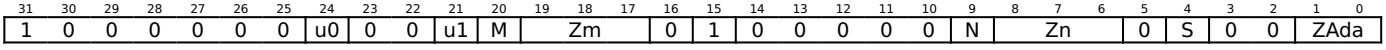
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.



Decode fields				Instruction Details	Condition
u0	M	N	S		
0	0	0	0	SMOP4A (2-way) — Single vectors	FEAT_SME_MOP4
0	0	0	1	SMOP4S (2-way) — Single vectors	FEAT_SME_MOP4
0	0	1	0	SMOP4A (2-way) — Multiple and single vectors	FEAT_SME_MOP4
0	0	1	1	SMOP4S (2-way) — Multiple and single vectors	FEAT_SME_MOP4
0	1	0	0	SMOP4A (2-way) — Single and multiple vectors	FEAT_SME_MOP4
0	1	0	1	SMOP4S (2-way) — Single and multiple vectors	FEAT_SME_MOP4
0	1	1	0	SMOP4A (2-way) — Multiple vectors	FEAT_SME_MOP4
0	1	1	1	SMOP4S (2-way) — Multiple vectors	FEAT_SME_MOP4
1	0	0	0	UMOP4A (2-way) — Single vectors	FEAT_SME_MOP4
1	0	0	1	UMOP4S (2-way) — Single vectors	FEAT_SME_MOP4
1	0	1	0	UMOP4A (2-way) — Multiple and single vectors	FEAT_SME_MOP4
1	0	1	1	UMOP4S (2-way) — Multiple and single vectors	FEAT_SME_MOP4
1	1	0	0	UMOP4A (2-way) — Single and multiple vectors	FEAT_SME_MOP4
1	1	0	1	UMOP4S (2-way) — Single and multiple vectors	FEAT_SME_MOP4
1	1	1	0	UMOP4A (2-way) — Multiple vectors	FEAT_SME_MOP4
1	1	1	1	UMOP4S (2-way) — Multiple vectors	FEAT_SME_MOP4

SME2 Int8 to Int32 quarter tile outer product

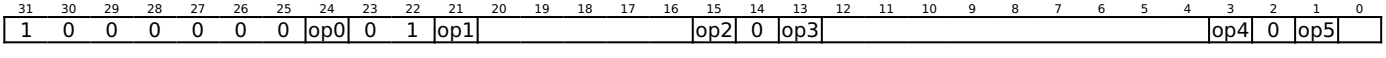
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 16-bit and 32-bit.



Decode fields					Instruction Details	Condition
u0	u1	M	N	S		
0	0	0	0	0	SMOP4A (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
0	0	0	0	1	SMOP4S (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
0	0	0	1	0	SMOP4A (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	0	0	1	1	SMOP4S (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	0	1	0	0	SMOP4A (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	0	1	0	1	SMOP4S (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	0	1	1	0	SMOP4A (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4
0	0	1	1	1	SMOP4S (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4
0	1	0	0	0	SUMOP4A — 32-bit, single vectors	FEAT_SME_MOP4
0	1	0	0	1	SUMOP4S — 32-bit, single vectors	FEAT_SME_MOP4
0	1	0	1	0	SUMOP4A — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	1	0	1	1	SUMOP4S — 32-bit, multiple and single vectors	FEAT_SME_MOP4
0	1	1	0	0	SUMOP4A — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	1	1	0	1	SUMOP4S — 32-bit, single and multiple vectors	FEAT_SME_MOP4
0	1	1	1	0	SUMOP4A — 32-bit, multiple vectors	FEAT_SME_MOP4
0	1	1	1	1	SUMOP4S — 32-bit, multiple vectors	FEAT_SME_MOP4
1	0	0	0	0	USMOP4A — 32-bit, single vectors	FEAT_SME_MOP4
1	0	0	0	1	USMOP4S — 32-bit, single vectors	FEAT_SME_MOP4
1	0	0	1	0	USMOP4A — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	0	0	1	1	USMOP4S — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	0	1	0	0	USMOP4A — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	0	1	0	1	USMOP4S — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	0	1	1	0	USMOP4A — 32-bit, multiple vectors	FEAT_SME_MOP4
1	0	1	1	1	USMOP4S — 32-bit, multiple vectors	FEAT_SME_MOP4
1	1	0	0	0	UMOP4A (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
1	1	0	0	1	UMOP4S (4-way) — 32-bit, single vectors	FEAT_SME_MOP4
1	1	0	1	0	UMOP4A (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	1	0	1	1	UMOP4S (4-way) — 32-bit, multiple and single vectors	FEAT_SME_MOP4
1	1	1	0	0	UMOP4A (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	1	1	0	1	UMOP4S (4-way) — 32-bit, single and multiple vectors	FEAT_SME_MOP4
1	1	1	1	0	UMOP4A (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4
1	1	1	1	1	UMOP4S (4-way) — 32-bit, multiple vectors	FEAT_SME_MOP4

SME2 Sparse Outer Product

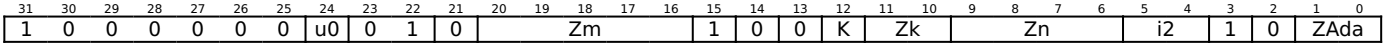
The encodings in this section are decoded from SME encodings.



Decode fields						Instruction details	Condition
op0	op1	op2	op3	op4	op5		
0	0	0	0	0		FTMOPA (non-widening) — Single-precision	FEAT_SME_TMOP
0	0	0	0	1	0	UNALLOCATED	-
0	1	0	0	0		FTMOPA (widening, 4-way)	FEAT_SME_TMOP && FEAT_SME_F8F32
0	1	0	0	1	0	FTMOPA (widening, 2-way, FP8 to FP16)	FEAT_SME_TMOP && FEAT_SME_F8F16
1	0	0	0	0		BFTMOPA (widening)	FEAT_SME_TMOP
1	0	0	0	1	0	FTMOPA (non-widening) — Half-precision	FEAT_SME_TMOP && FEAT_SME_F16F16
1	1	0	0	0		FTMOPA (widening, 2-way, FP16 to FP32)	FEAT_SME_TMOP
1	1	0	0	1	0	BFTMOPA (non-widening)	FEAT_SME_TMOP && FEAT_SME_B16B16
	0	1	0	1		SME2 Int16 to Int32 sparse outer product	-
	1	1	0	1		UNALLOCATED	-
		0	0	1	1	UNALLOCATED	-
		1	0	0		SME Int8 to Int32 sparse outer product	-
			1			UNALLOCATED	-

SME2 Int16 to Int32 sparse outer product

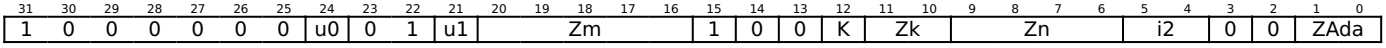
The encodings in this section are decoded from SME2 Sparse Outer Product.



Decode fields	Instruction Details	Condition
u0		
0	STMOPA (2-way)	FEAT_SME_TMOP
1	UTMOPA (2-way)	FEAT_SME_TMOP

SME Int8 to Int32 sparse outer product

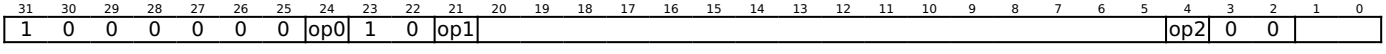
The encodings in this section are decoded from SME2 Sparse Outer Product.



Decode fields		Instruction Details	Condition
u0	u1		
0	0	STMOPA (4-way)	FEAT_SME_TMOP
0	1	SUTMOPA	FEAT_SME_TMOP
1	0	USTMOPA	FEAT_SME_TMOP
1	1	UTMOPA (4-way)	FEAT_SME_TMOP

SME FP Outer Product - 32 bit

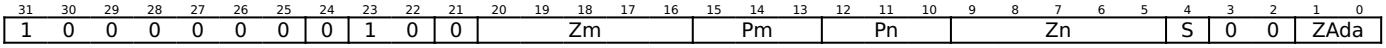
The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details	Condition
op0	op1	op2		
0	0		SME FP32 outer product	-
0	1	0	FMOPA (widening, 4-way)	FEAT_SME_F8F32
0	1	1	UNALLOCATED	-
1	0		SME BFloat16 widening outer product	-
1	1		SME FP16 widening outer product	-

SME FP32 outer product

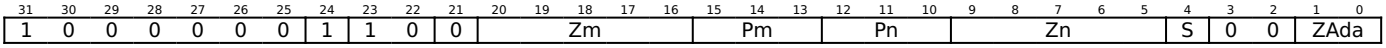
The encodings in this section are decoded from SME FP Outer Product - 32 bit.



Decode fields	Instruction Details	Condition
S		
0	FMOPA (non-widening)	FEAT_SME
1	FMOPS (non-widening)	FEAT_SME

SME BFloat16 widening outer product

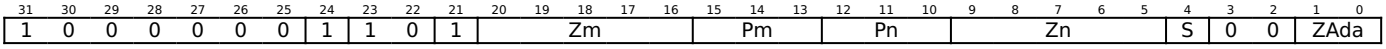
The encodings in this section are decoded from SME FP Outer Product - 32 bit.



Decode fields	Instruction Details	Condition
S		
0	BFMOPA (widening)	FEAT_SME
1	BFMOPS (widening)	FEAT_SME

SME FP16 widening outer product

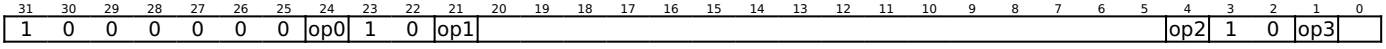
The encodings in this section are decoded from SME FP Outer Product - 32 bit.



Decode fields	Instruction Details	Condition
S		
0	FMOPA (widening, 2-way, FP16 to FP32)	FEAT_SME
1	FMOPS (widening)	FEAT_SME

SME2 Outer Product - Misc

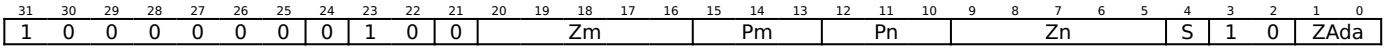
The encodings in this section are decoded from SME encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
0	0			SME2 32-bit binary outer product	-
0	1	0	0	FMOPA (widening, 2-way, FP8 to FP16)	FEAT_SME_F8F16
0	1	1	0	UNALLOCATED	-
0	1		1	UNALLOCATED	-
1	0		0	SME2 FP16 non-widening outer product	-
1	1		0	SME2 BFloat16 non-widening outer product	-
1			1	UNALLOCATED	-

SME2 32-bit binary outer product

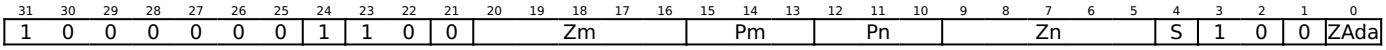
The encodings in this section are decoded from SME2 Outer Product - Misc.



Decode fields	Instruction Details	Condition
S		
0	BMOPA	FEAT_SME2
1	BMOPS	FEAT_SME2

SME2 FP16 non-widening outer product

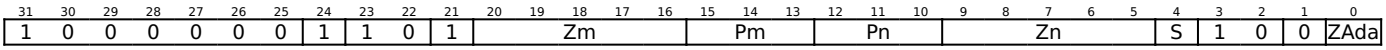
The encodings in this section are decoded from SME2 Outer Product - Misc.



Decode fields	Instruction Details	Condition
S		
0	FMOPA (non-widening)	FEAT_SME_F16F16
1	FMOPS (non-widening)	FEAT_SME_F16F16

SME2 BFloat16 non-widening outer product

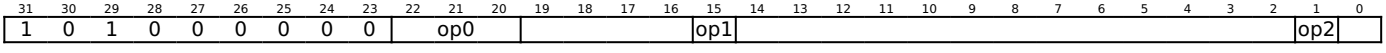
The encodings in this section are decoded from SME2 Outer Product - Misc.



Decode fields	Instruction Details	Condition
S		
0	BFMOPA (non-widening)	FEAT_SME_B16B16
1	BFMOPS (non-widening)	FEAT_SME_B16B16

SME2 Multi-vector - Memory (Contiguous)

The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
00x	0		SME2 multi-vec contiguous load (scalar plus scalar, two registers)
00x	1	0	SME2 multi-vec contiguous load (scalar plus scalar, four registers)
01x	0		SME2 multi-vec contiguous store (scalar plus scalar, two registers)
01x	1	0	SME2 multi-vec contiguous store (scalar plus scalar, four registers)
100	0		SME2 multi-vec contiguous load (scalar plus immediate, two registers)
100	1	0	SME2 multi-vec contiguous load (scalar plus immediate, four registers)
110	0		SME2 multi-vec contiguous store (scalar plus immediate, two registers)
110	1	0	SME2 multi-vec contiguous store (scalar plus immediate, four registers)
1x1			UNALLOCATED
!= 1x1	1	1	UNALLOCATED

SME2 multi-vec contiguous load (scalar plus scalar, two registers)

The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	0	0	Rm					0	msz		PNg			Rn					Zt				N

Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	LDNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	LD1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	LDNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	LD1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	LDNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	LD1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	LDNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous load (scalar plus scalar, four registers)

The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	0	0	Rm					1	msz		PNg			Rn					Zt			0	N

Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	LDNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	LD1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	LDNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	LD1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	LDNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	LD1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	LDNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus scalar, two registers)

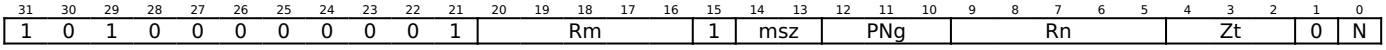
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	0	0	0	1	Rm					0	msz		PNg			Rn					Zt				N

Decode fields msz	N	Instruction Details	Condition
00	0	ST1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	STNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	ST1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	STNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	ST1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	STNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	ST1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	STNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus scalar, four registers)

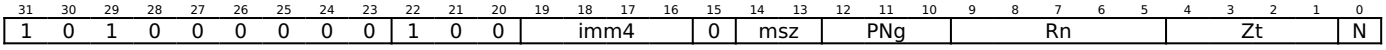
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	ST1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	STNT1B (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	ST1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	STNT1H (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	ST1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	STNT1W (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	ST1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	STNT1D (scalar plus scalar, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous load (scalar plus immediate, two registers)

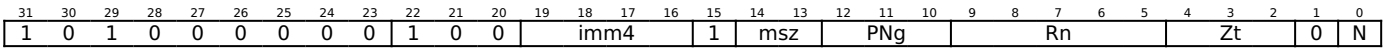
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	LD1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	LDNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	LD1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	LDNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	LD1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	LDNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	LD1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	LDNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous load (scalar plus immediate, four registers)

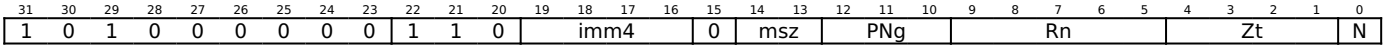
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	LD1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	LDNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	LD1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	LDNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	LD1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	LDNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	LD1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	LDNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus immediate, two registers)

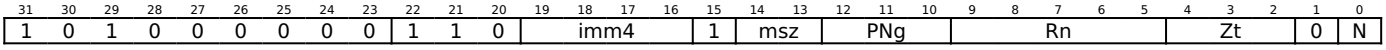
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	ST1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	STNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	ST1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	STNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	ST1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	STNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	ST1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	STNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 multi-vec contiguous store (scalar plus immediate, four registers)

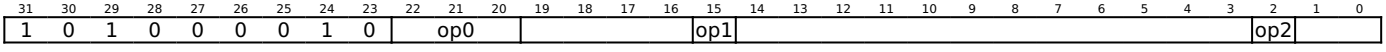
The encodings in this section are decoded from SME2 Multi-vector - Memory (Contiguous).



Decode fields msz	N	Instruction Details	Condition
00	0	ST1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
00	1	STNT1B (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	0	ST1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
01	1	STNT1H (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	0	ST1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
10	1	STNT1W (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	0	ST1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1
11	1	STNT1D (scalar plus immediate, consecutive registers)	FEAT_SME2 || FEAT_SVE2p1

SME2 Multi-vector - Memory (Strided)

The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
00x	0		SME2 multi-vec non-contiguous load (scalar plus scalar, two registers)
00x	1	0	SME2 multi-vec non-contiguous load (scalar plus scalar, four registers)
01x	0		SME2 multi-vec non-contiguous store (scalar plus scalar, two registers)
01x	1	0	SME2 multi-vec non-contiguous store (scalar plus scalar, four registers)
100	0		SME2 multi-vec non-contiguous load (scalar plus immediate, two registers)
100	1	0	SME2 multi-vec non-contiguous load (scalar plus immediate, four registers)
110	0		SME2 multi-vec non-contiguous store (scalar plus immediate, two registers)
110	1	0	SME2 multi-vec non-contiguous store (scalar plus immediate, four registers)
1x1			UNALLOCATED
!= 1x1	1	1	UNALLOCATED

SME2 multi-vec non-contiguous load (scalar plus scalar, two registers)

The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	0	Rm					0	msz		PNg			Rn					T	N	Zt		

Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous load (scalar plus scalar, four registers)

The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	0	Rm					1	msz		PNg			Rn					T	N	0	Zt	

Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus scalar, two registers)

The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	1	Rm					0	msz		PNg			Rn					T	N	Zt		

Decode fields		Instruction Details	Condition
msz	N		
00	0	ST1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus scalar, four registers)

The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	0	1	Rm					1	msz		PNg			Rn					T	N	0	Zt	

Decode fields		Instruction Details	Condition
msz	N		
00	0	ST1B (scalar plus scalar, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus scalar, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus scalar, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus scalar, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus scalar, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus scalar, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus scalar, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus scalar, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous load (scalar plus immediate, two registers)

The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	1	0	0	imm4				0	msz		PNg			Rn					T	N	Zt		

Decode fields		Instruction Details	Condition
msz	N		
00	0	LD1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous load (scalar plus immediate, four registers)

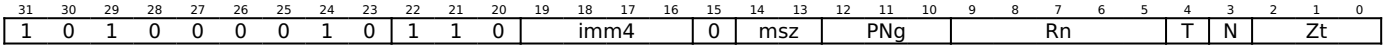
The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	1	0	1	0	0	imm4				1	msz		PNg			Rn					T	N	0	Zt	

Decode fields msz	N	Instruction Details	Condition
00	0	LD1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	LDNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	LD1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	LDNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	LD1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	LDNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	LD1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	LDNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus immediate, two registers)

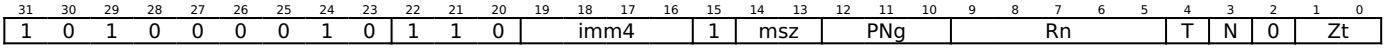
The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).



Decode fields msz	N	Instruction Details	Condition
00	0	ST1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME2 multi-vec non-contiguous store (scalar plus immediate, four registers)

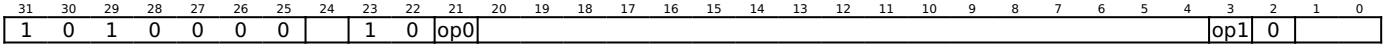
The encodings in this section are decoded from SME2 Multi-vector - Memory (Strided).



Decode fields msz	N	Instruction Details	Condition
00	0	ST1B (scalar plus immediate, strided registers)	FEAT_SME2
00	1	STNT1B (scalar plus immediate, strided registers)	FEAT_SME2
01	0	ST1H (scalar plus immediate, strided registers)	FEAT_SME2
01	1	STNT1H (scalar plus immediate, strided registers)	FEAT_SME2
10	0	ST1W (scalar plus immediate, strided registers)	FEAT_SME2
10	1	STNT1W (scalar plus immediate, strided registers)	FEAT_SME2
11	0	ST1D (scalar plus immediate, strided registers)	FEAT_SME2
11	1	STNT1D (scalar plus immediate, strided registers)	FEAT_SME2

SME Integer Outer Product - 32 bit

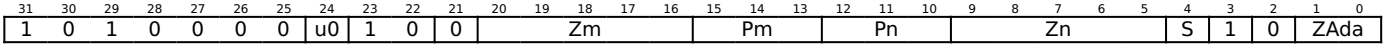
The encodings in this section are decoded from SME encodings.



Decode fields		Instruction details
op0	op1	
0	1	SME2 Int16 two-way outer product
1	1	UNALLOCATED
	0	SME Int8 outer product

SME2 Int16 two-way outer product

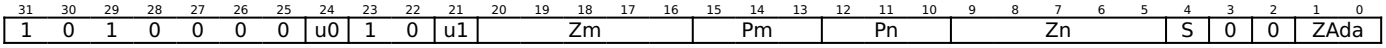
The encodings in this section are decoded from SME Integer Outer Product - 32 bit.



Decode fields		Instruction Details	Condition
u0	S		
0	0	SMOPA (2-way)	FEAT_SME2
0	1	SMOPS (2-way)	FEAT_SME2
1	0	UMOPA (2-way)	FEAT_SME2
1	1	UMOPS (2-way)	FEAT_SME2

SME Int8 outer product

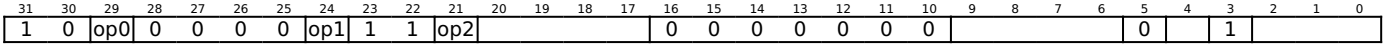
The encodings in this section are decoded from SME Integer Outer Product - 32 bit.



Decode fields			Instruction Details	Condition
u0	u1	S		
0	0	0	SMOPA (4-way)	FEAT_SME
0	0	1	SMOPS (4-way)	FEAT_SME
0	1	0	SUMOPA (4-way)	FEAT_SME
0	1	1	SUMOPS	FEAT_SME
1	0	0	USMOPA (4-way)	FEAT_SME
1	0	1	USMOPS	FEAT_SME
1	1	0	UMOPA (4-way)	FEAT_SME
1	1	1	UMOPS (4-way)	FEAT_SME

SME2 Quarter Tile Outer Product - 64-bit

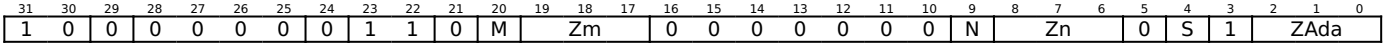
The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
0	0	0	SME2 FP64 non-widening quarter tile outer product
0	0	1	UNALLOCATED
0	1		UNALLOCATED
1			SME2 Int16 to Int64 quarter tile outer product

SME2 FP64 non-widening quarter tile outer product

The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 64-bit.



Decode fields			Instruction Details	Condition
M	N	S		
0	0	0	FMOP4A (non-widening) — Double-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
0	0	1	FMOP4S (non-widening) — Double-precision, single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
0	1	0	FMOP4A (non-widening) — Double-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
0	1	1	FMOP4S (non-widening) — Double-precision, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	0	0	FMOP4A (non-widening) — Double-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	0	1	FMOP4S (non-widening) — Double-precision, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	1	0	FMOP4A (non-widening) — Double-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64
1	1	1	FMOP4S (non-widening) — Double-precision, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_F64F64

SME2 Int16 to Int64 quarter tile outer product

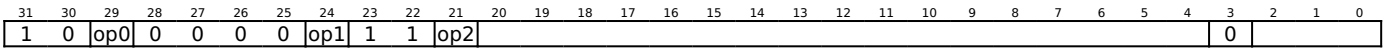
The encodings in this section are decoded from SME2 Quarter Tile Outer Product - 64-bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	0	0	u0	1	1	u1	M	Zm			0	0	0	0	0	0	0	N	Zn			0	S	1	ZAda		

Decode fields					Instruction Details	Condition
u0	u1	M	N	S		
0	0	0	0	0	SMOP4A (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	0	0	1	SMOP4S (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	0	1	0	SMOP4A (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	0	1	1	SMOP4S (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	0	0	SMOP4A (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	0	1	SMOP4S (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	1	0	SMOP4A (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	0	1	1	1	SMOP4S (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	0	0	SUMOP4A — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	0	1	SUMOP4S — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	1	0	SUMOP4A — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	0	1	1	SUMOP4S — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	0	0	SUMOP4A — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	0	1	SUMOP4S — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	1	0	SUMOP4A — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
0	1	1	1	1	SUMOP4S — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	0	0	USMOP4A — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	0	1	USMOP4S — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	1	0	USMOP4A — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	0	1	1	USMOP4S — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	0	0	USMOP4A — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	0	1	USMOP4S — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	1	0	USMOP4A — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	0	1	1	1	USMOP4S — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	0	0	UMOP4A (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	0	1	UMOP4S (4-way) — 64-bit, single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	1	0	UMOP4A (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	0	1	1	UMOP4S (4-way) — 64-bit, multiple and single vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	0	0	UMOP4A (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	0	1	UMOP4S (4-way) — 64-bit, single and multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	1	0	UMOP4A (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64
1	1	1	1	1	UMOP4S (4-way) — 64-bit, multiple vectors	FEAT_SME_MOP4 && FEAT_SME_I16I64

SME Outer Product - 64 bit

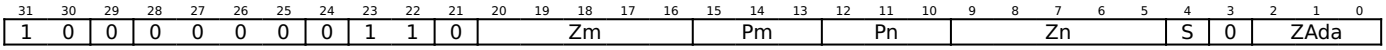
The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
0	0	0	SME FP64 outer product
0	0	1	UNALLOCATED
0	1		UNALLOCATED
1			SME Int16 outer product

SME FP64 outer product

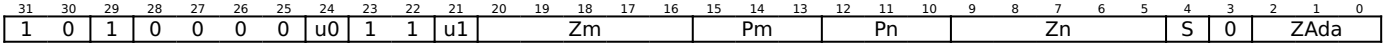
The encodings in this section are decoded from SME Outer Product - 64 bit.



Decode fields S	Instruction Details	Condition
0	FMOPA (non-widening)	FEAT_SME_F64F64
1	FMOPS (non-widening)	FEAT_SME_F64F64

SME Int16 outer product

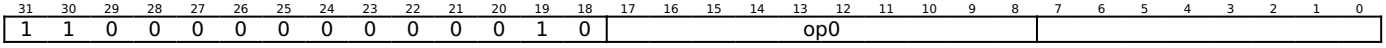
The encodings in this section are decoded from SME Outer Product - 64 bit.



Decode fields u0	u1	S	Instruction Details	Condition
0	0	0	SMOPA (4-way)	FEAT_SME_I16I64
0	0	1	SMOPS (4-way)	FEAT_SME_I16I64
0	1	0	SUMOPA (4-way)	FEAT_SME_I16I64
0	1	1	SUMOPS	FEAT_SME_I16I64
1	0	0	USMOPA (4-way)	FEAT_SME_I16I64
1	0	1	USMOPS	FEAT_SME_I16I64
1	1	0	UMOPA (4-way)	FEAT_SME_I16I64
1	1	1	UMOPS (4-way)	FEAT_SME_I16I64

SME zero array

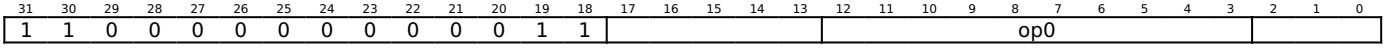
The encodings in this section are decoded from SME encodings.



Decode fields op0	Instruction details	Condition
0000000000	ZERO (tiles)	FEAT_SME
!= 0000000000	UNALLOCATED	-

SME2 Multiple Zero

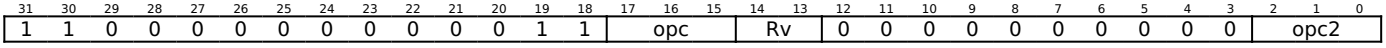
The encodings in this section are decoded from SME encodings.



Decode fields op0	Instruction details
0000000000	SME multiple vectors zero array
!= 0000000000	UNALLOCATED

SME multiple vectors zero array

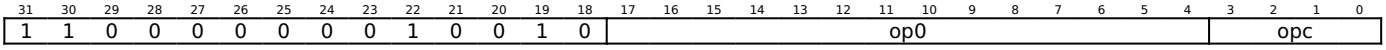
The encodings in this section are decoded from SME2 Multiple Zero.



Decode fields		Instruction Details	Condition
opc	opc2		
x1x	1xx	UNALLOCATED	-
000		ZERO (single-vector) — Two ZA single-vectors	FEAT_SME2p1
001		ZERO (double-vector) — One ZA double-vector	FEAT_SME2p1
010	0xx	ZERO (double-vector) — Two ZA double-vectors	FEAT_SME2p1
011	0xx	ZERO (double-vector) — Four ZA double-vectors	FEAT_SME2p1
100		ZERO (single-vector) — Four ZA single-vectors	FEAT_SME2p1
101	0xx	ZERO (quad-vector) — One ZA quad-vector	FEAT_SME2p1
101	1xx	UNALLOCATED	-
11x	01x	UNALLOCATED	-
110	00x	ZERO (quad-vector) — Two ZA quad-vectors	FEAT_SME2p1
111	00x	ZERO (quad-vector) — Four ZA quad-vectors	FEAT_SME2p1

SME2 zero lookup table

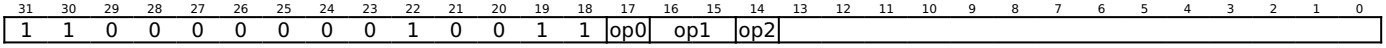
The encodings in this section are decoded from SME encodings.



Decode fields		Instruction Details	Condition
op0	opc		
00000000000000	0001	ZERO (table)	FEAT_SME2
00000000000000	!= 0001	UNALLOCATED	-
!= 00000000000000		UNALLOCATED	-

SME2 Move Lookup Table

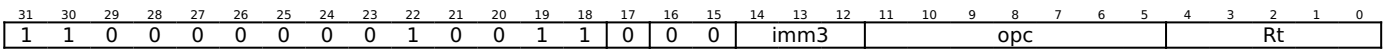
The encodings in this section are decoded from SME encodings.



Decode fields			Instruction details
op0	op1	op2	
0	00		SME2 move from lookup table
0	10		UNALLOCATED
1	00		SME2 move into lookup table
1	10	0	SME2 move vector to lookup table
1	10	1	UNALLOCATED
	x1		UNALLOCATED

SME2 move from lookup table

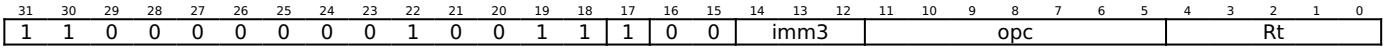
The encodings in this section are decoded from SME2 Move Lookup Table.



Decode fields	Instruction Details	Condition
opc		
0011111	MOVT (table to scalar)	FEAT_SME2
!= 0011111	UNALLOCATED	-

SME2 move into lookup table

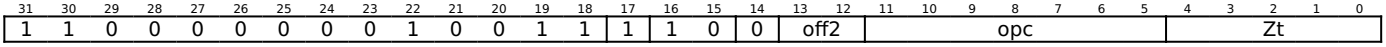
The encodings in this section are decoded from SME2 Move Lookup Table.



Decode fields opc	Instruction Details	Condition
0011111	MOVT (scalar to table)	FEAT_SME2
!= 0011111	UNALLOCATED	-

SME2 move vector to lookup table

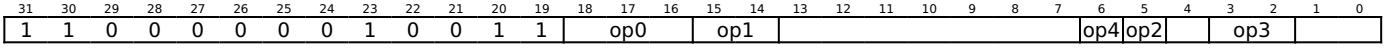
The encodings in this section are decoded from SME2 Move Lookup Table.



Decode fields opc	Instruction Details	Condition
0011111	MOVT (vector to table)	FEAT_SME_LUTv2
!= 0011111	UNALLOCATED	-

SME2 Expand Lookup Table (Non-contiguous)

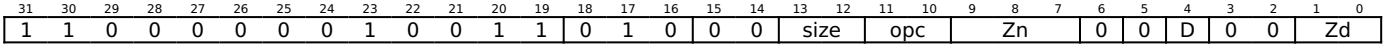
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op4	op2	op3	Instruction details
010	00	0	0	00	SME2 lookup table three source registers expand to four non-contiguous destination registers
010	00	1	0	00	UNALLOCATED
011	00		0	00	SME2 lookup table two source registers expand to four non-contiguous destination registers
01x	00		1	00	UNALLOCATED
	10			00	SME2 lookup table expand four non-contiguous registers
	x0			01	UNALLOCATED
	x1			0x	SME2 lookup table expand two non-contiguous registers
				1x	UNALLOCATED
!= 01x	00			00	UNALLOCATED

SME2 lookup table three source registers expand to four non-contiguous destination registers

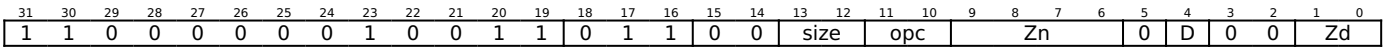
The encodings in this section are decoded from SME2 Expand Lookup Table (Non-contiguous).



Decode fields size	opc	Instruction Details	Condition
00	00	LUTI6 (table, four registers, 8-bit)	FEAT_SME2p3
00	!= 00	UNALLOCATED	-
!= 00		UNALLOCATED	-

SME2 lookup table two source registers expand to four non-contiguous destination registers

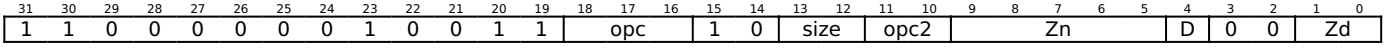
The encodings in this section are decoded from SME2 Expand Lookup Table (Non-contiguous).



Decode fields opc	Instruction Details	Condition
00	LUTI4 (four registers, 8-bit)	FEAT_SME2p1 && FEAT_SME_LUTv2
!= 00	UNALLOCATED	-

SME2 lookup table expand four non-contiguous registers

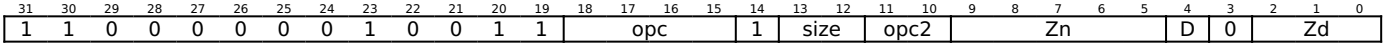
The encodings in this section are decoded from SME2 Expand Lookup Table (Non-contiguous).



Decode fields opc	Decode fields opc2	Instruction Details	Condition
	!= 00	UNALLOCATED	-
00x	00	UNALLOCATED	-
01x	00	LUTI4 (four registers, 16-bit and 32-bit)	FEAT_SME2p1
1xx	00	LUTI2 (four registers)	FEAT_SME2p1

SME2 lookup table expand two non-contiguous registers

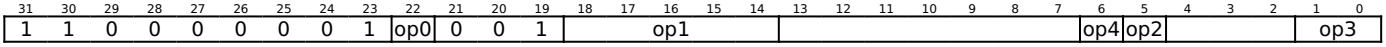
The encodings in this section are decoded from SME2 Expand Lookup Table (Non-contiguous).



Decode fields opc	Decode fields opc2	Instruction Details	Condition
	!= 00	UNALLOCATED	-
00xx	00	UNALLOCATED	-
01xx	00	LUTI4 (two registers)	FEAT_SME2p1
1xxx	00	LUTI2 (two registers)	FEAT_SME2p1

SME2 Expand Lookup Table (Contiguous)

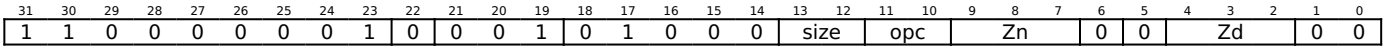
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op4	op2	op3	Instruction details
0	00x00			00	UNALLOCATED
0	01000	0	0	00	SME2 lookup table three source registers expand to four contiguous destination registers
0	01000	1	0	00	UNALLOCATED
0	01100		0	00	SME2 lookup table two source registers expand to four contiguous destination registers
0	01x00		1	00	UNALLOCATED
0	1xx00			00	UNALLOCATED
0	xxx10			00	SME2 lookup table expand four contiguous registers
0	xxxx0			10	UNALLOCATED
0	xxxx1			x0	SME2 lookup table expand two contiguous registers
0				x1	UNALLOCATED
1					SME2 lookup table expand one register

SME2 lookup table three source registers expand to four contiguous destination registers

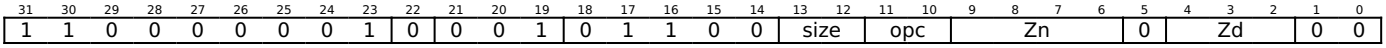
The encodings in this section are decoded from SME2 Expand Lookup Table (Contiguous).



Decode fields		Instruction Details	Condition
size	opc		
00	00	LUTI6 (table, four registers, 8-bit)	FEAT_SME2p3
00	!= 00	UNALLOCATED	-
!= 00		UNALLOCATED	-

SME2 lookup table two source registers expand to four contiguous destination registers

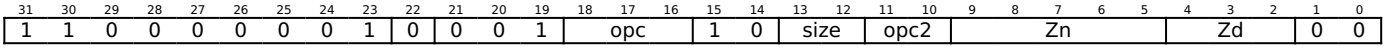
The encodings in this section are decoded from SME2 Expand Lookup Table (Contiguous).



Decode fields	Instruction Details	Condition
opc		
00	LUTI4 (four registers, 8-bit)	FEAT_SME_LUTv2
!= 00	UNALLOCATED	-

SME2 lookup table expand four contiguous registers

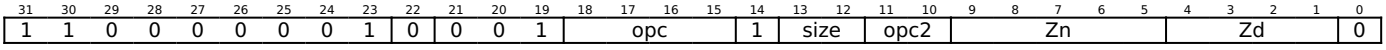
The encodings in this section are decoded from SME2 Expand Lookup Table (Contiguous).



Decode fields		Instruction Details	Condition
opc	opc2		
	!= 00	UNALLOCATED	-
00x	00	UNALLOCATED	-
01x	00	LUTI4 (four registers, 16-bit and 32-bit)	FEAT_SME2
1xx	00	LUTI2 (four registers)	FEAT_SME2

SME2 lookup table expand two contiguous registers

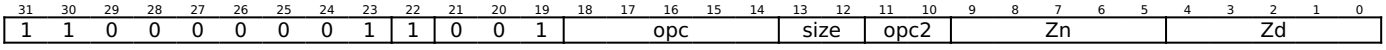
The encodings in this section are decoded from SME2 Expand Lookup Table (Contiguous).



Decode fields		Instruction Details	Condition
opc	opc2		
	!= 00	UNALLOCATED	-
00xx	00	UNALLOCATED	-
01xx	00	LUTI4 (two registers)	FEAT_SME2
1xxx	00	LUTI2 (two registers)	FEAT_SME2

SME2 lookup table expand one register

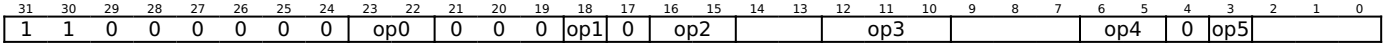
The encodings in this section are decoded from SME2 Expand Lookup Table (Contiguous).



Decode fields			Instruction Details	Condition
opc	size	opc2		
		!= 00	UNALLOCATED	-
00000		00	UNALLOCATED	-
00001	00	00	LUTI6 (table, single, 8-bit)	FEAT_SME2p3
00001	!= 00	00	UNALLOCATED	-
0001x		00	UNALLOCATED	-
001xx		00	UNALLOCATED	-
01xxx		00	LUTI4 (single)	FEAT_SME2
1xxxx		00	LUTI2 (single)	FEAT_SME2

SME Move into Array

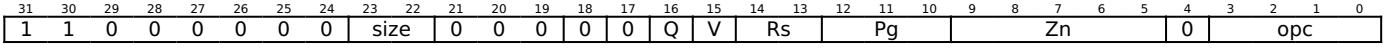
The encodings in this section are decoded from SME encodings.



Decode fields						Instruction details	Condition
op0	op1	op2	op3	op4	op5		
00	1	00	010	x0	0	MOVA (vector to array, two registers)	FEAT_SME2
00	1	00	011	00	0	MOVA (vector to array, four registers)	FEAT_SME2
00	1	00	011	10	0	UNALLOCATED	-
00	1	01	01x	x0	0	UNALLOCATED	-
	0					SME move vector to array	-
	1	0x	000	x0	0	SME2 move vector to tile, two registers	-
	1	0x	001	00	0	SME2 move vector to tile, four registers	-
	1	0x	001	10	0	UNALLOCATED	-
	1	0x	0xx	x0	1	UNALLOCATED	-
	1	0x	0xx	x1		UNALLOCATED	-
	1	0x	1xx			UNALLOCATED	-
	1	1x				UNALLOCATED	-
!= 00	1	0x	01x	x0	0	UNALLOCATED	-

SME move vector to array

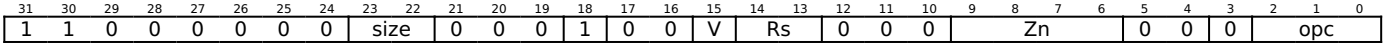
The encodings in this section are decoded from SME Move into Array.



Decode fields		Instruction Details	Condition
size	Q		
00	0	MOVA (vector to tile, single) — 8-bit	FEAT_SME
01	0	MOVA (vector to tile, single) — 16-bit	FEAT_SME
10	0	MOVA (vector to tile, single) — 32-bit	FEAT_SME
11	0	MOVA (vector to tile, single) — 64-bit	FEAT_SME
11	1	MOVA (vector to tile, single) — 128-bit	FEAT_SME
!= 11	1	UNALLOCATED	-

SME2 move vector to tile, two registers

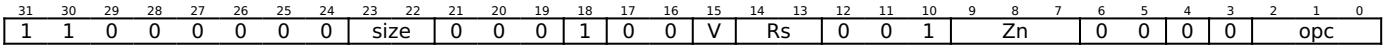
The encodings in this section are decoded from SME Move into Array.



Decode fields size	Instruction Details	Condition
00	MOVA (vector to tile, two registers) — 8-bit	FEAT_SME2
01	MOVA (vector to tile, two registers) — 16-bit	FEAT_SME2
10	MOVA (vector to tile, two registers) — 32-bit	FEAT_SME2
11	MOVA (vector to tile, two registers) — 64-bit	FEAT_SME2

SME2 move vector to tile, four registers

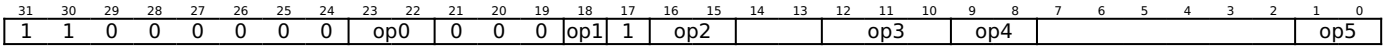
The encodings in this section are decoded from SME Move into Array.



Decode fields size	Decode fields opc	Instruction Details	Condition
00	0xx	MOVA (vector to tile, four registers) — 8-bit	FEAT_SME2
01	0xx	MOVA (vector to tile, four registers) — 16-bit	FEAT_SME2
10	0xx	MOVA (vector to tile, four registers) — 32-bit	FEAT_SME2
11		MOVA (vector to tile, four registers) — 64-bit	FEAT_SME2
!= 11	1xx	UNALLOCATED	-

SME Move from Array

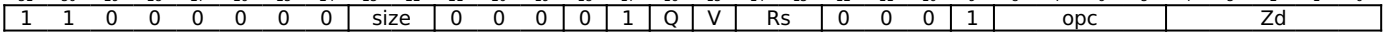
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	op3	op4	op5	Instruction details	Condition
00	1	00	010	00	x0	MOVA (array to vector, two registers)	FEAT_SME2
00	1	00	010	10	x0	MOVAZ (array to vector, two registers)	FEAT_SME2p1
00	1	00	011	00	00	MOVA (array to vector, four registers)	FEAT_SME2
00	1	00	011	10	00	MOVAZ (array to vector, four registers)	FEAT_SME2p1
00	1	00	011	x0	10	UNALLOCATED	-
00	1	01	01x	x0	x0	UNALLOCATED	-
	0		000	1x		SME zeroing move array to vector	-
	0			0x		SME move array to vector	-
	0		!= 000	1x		UNALLOCATED	-
	1	0x	000	00	x0	SME2 move tile to vector, two registers	-
	1	0x	000	10	x0	SME2 zeroing move tile to vector, two registers	-
	1	0x	001	00	00	SME2 move tile to vector, four registers	-
	1	0x	001	10	00	SME2 zeroing move tile to vector, four registers	-
	1	0x	001	x0	10	UNALLOCATED	-
	1	0x	0xx	x0	x1	UNALLOCATED	-
	1	0x	0xx	x1		UNALLOCATED	-
	1	0x	1xx			UNALLOCATED	-
	1	1x				UNALLOCATED	-
!= 00	1	0x	01x	x0	x0	UNALLOCATED	-

SME zeroing move array to vector

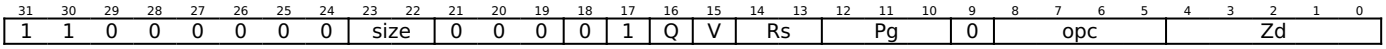
The encodings in this section are decoded from SME Move from Array.



Decode fields size	Q	Instruction Details	Condition
00	0	MOVAZ (tile to vector, single) — 8-bit	FEAT_SME2p1
01	0	MOVAZ (tile to vector, single) — 16-bit	FEAT_SME2p1
10	0	MOVAZ (tile to vector, single) — 32-bit	FEAT_SME2p1
11	0	MOVAZ (tile to vector, single) — 64-bit	FEAT_SME2p1
11	1	MOVAZ (tile to vector, single) — 128-bit	FEAT_SME2p1
!= 11	1	UNALLOCATED	-

SME move array to vector

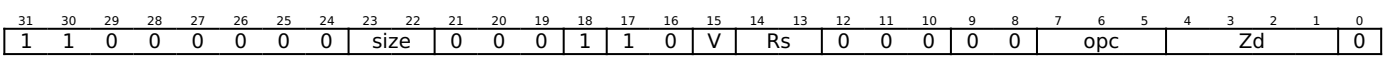
The encodings in this section are decoded from SME Move from Array.



Decode fields size	Q	Instruction Details	Condition
00	0	MOVA (tile to vector, single) — 8-bit	FEAT_SME
01	0	MOVA (tile to vector, single) — 16-bit	FEAT_SME
10	0	MOVA (tile to vector, single) — 32-bit	FEAT_SME
11	0	MOVA (tile to vector, single) — 64-bit	FEAT_SME
11	1	MOVA (tile to vector, single) — 128-bit	FEAT_SME
!= 11	1	UNALLOCATED	-

SME2 move tile to vector, two registers

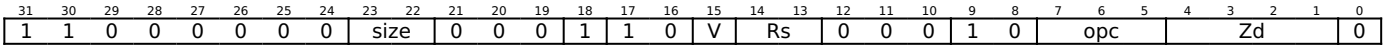
The encodings in this section are decoded from SME Move from Array.



Decode fields size	Instruction Details	Condition
00	MOVA (tile to vector, two registers) — 8-bit	FEAT_SME2
01	MOVA (tile to vector, two registers) — 16-bit	FEAT_SME2
10	MOVA (tile to vector, two registers) — 32-bit	FEAT_SME2
11	MOVA (tile to vector, two registers) — 64-bit	FEAT_SME2

SME2 zeroing move tile to vector, two registers

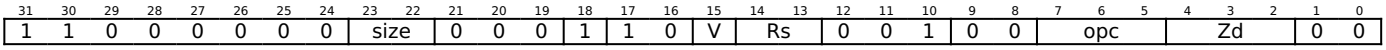
The encodings in this section are decoded from SME Move from Array.



Decode fields size	Instruction Details	Condition
00	MOVAZ (tile to vector, two registers) — 8-bit	FEAT_SME2p1
01	MOVAZ (tile to vector, two registers) — 16-bit	FEAT_SME2p1
10	MOVAZ (tile to vector, two registers) — 32-bit	FEAT_SME2p1
11	MOVAZ (tile to vector, two registers) — 64-bit	FEAT_SME2p1

SME2 move tile to vector, four registers

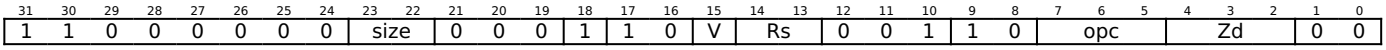
The encodings in this section are decoded from SME Move from Array.



Decode fields size	Decode fields opc	Instruction Details	Condition
00	0xx	MOVA (tile to vector, four registers) — 8-bit	FEAT_SME2
01	0xx	MOVA (tile to vector, four registers) — 16-bit	FEAT_SME2
10	0xx	MOVA (tile to vector, four registers) — 32-bit	FEAT_SME2
11		MOVA (tile to vector, four registers) — 64-bit	FEAT_SME2
!= 11	1xx	UNALLOCATED	-

SME2 zeroing move tile to vector, four registers

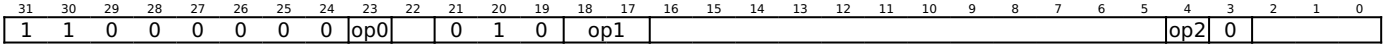
The encodings in this section are decoded from SME Move from Array.



Decode fields size	Decode fields opc	Instruction Details	Condition
00	0xx	MOVAZ (tile to vector, four registers) — 8-bit	FEAT_SME2p1
01	0xx	MOVAZ (tile to vector, four registers) — 16-bit	FEAT_SME2p1
10	0xx	MOVAZ (tile to vector, four registers) — 32-bit	FEAT_SME2p1
11		MOVAZ (tile to vector, four registers) — 64-bit	FEAT_SME2p1
!= 11	1xx	UNALLOCATED	-

SME Add Vector to Array

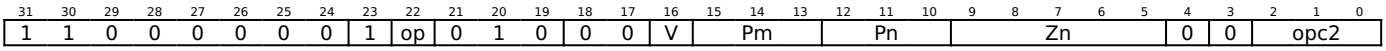
The encodings in this section are decoded from SME encodings.



Decode fields op0	Decode fields op1	Decode fields op2	Instruction details
0			UNALLOCATED
1	00	0	SME add vector to array
1	00	1	UNALLOCATED
1	!= 00		UNALLOCATED

SME add vector to array

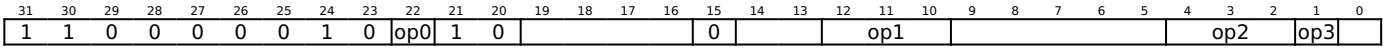
The encodings in this section are decoded from SME Add Vector to Array.



Decode fields op	Decode fields V	Decode fields opc2	Instruction Details	Condition
0		1xx	UNALLOCATED	-
0	0	0xx	ADDHA — 32-bit	FEAT_SME
0	1	0xx	ADDVA — 32-bit	FEAT_SME
1	0		ADDHA — 64-bit	FEAT_SME_I16I64
1	1		ADDVA — 64-bit	FEAT_SME_I16I64

SME2 Multi-vector - Multiple and Single Array Vectors (Two registers)

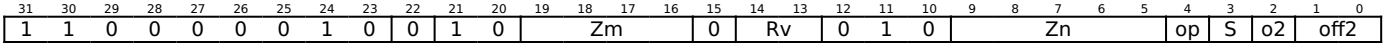
The encodings in this section are decoded from SME encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
0	000	000	1	FMLALL (multiple and single vector) — Two ZA quad-vectors	FEAT_SME_F8F32
0	000	!= 000	1	UNALLOCATED	-
0	010			SME2 single-multi long FMA two sources	-
0	011			SME2 multiple and single vector long FMA one source	-
0	100			SME2 single-multi FP dot product two registers	-
0	101	x1x		SME2 single-multi mixed dot product two registers	-
1	000		1	UNALLOCATED	-
1	010			SME2 single-multi long MLA two sources	-
1	011			SME2 multiple and single vector long MLA one source	-
1	100			UNALLOCATED	-
1	101	x1x		SME2 single-multi two-way dot product two registers	-
	000		0	SME2 single-multi long long MLA two sources	-
	001			SME2 multiple and single vector long long FMA one source	-
	101	x0x		SME2 single-multi four-way dot product two registers	-
	110	0xx		SME2 single-multi ternary FP two registers	-
	110	1xx		SME2 single-multi ternary int two registers	-
	111	0xx		SME2 single-multi ternary FP16 two registers	-
	111	1xx		UNALLOCATED	-

SME2 single-multi long FMA two sources

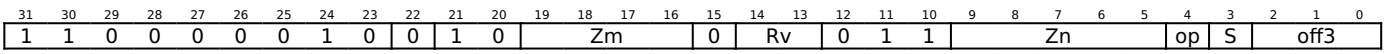
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields			Instruction Details	Condition
op	S	o2		
0	0	0	FMLAL (multiple and single vector, FP16 to FP32)	FEAT_SME2
0	0	1	FMLAL (multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
0	1	0	FMLSL (multiple and single vector)	FEAT_SME2
0	1	1	UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	BFMLAL (multiple and single vector)	FEAT_SME2
1	1	0	BFMLSL (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector long FMA one source

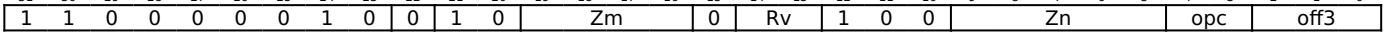
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields		Instruction Details	Condition
op	S		
0	0	FMLAL (multiple and single vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and single vector)	FEAT_SME2
1	0	BFMLAL (multiple and single vector)	FEAT_SME2
1	1	BFMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi FP dot product two registers

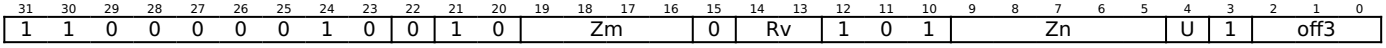
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields	Instruction Details	Condition
opc		
00	FDOT (2-way, multiple and single vector, FP16 to FP32)	FEAT_SME2
01	FDOT (2-way, multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
10	BFDOT (multiple and single vector)	FEAT_SME2
11	FDOT (4-way, multiple and single vector)	FEAT_SME_F8F32

SME2 single-multi mixed dot product two registers

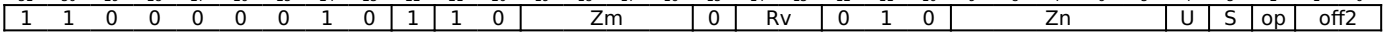
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields	Instruction Details	Condition
U		
0	USDOT (4-way, multiple and single vector)	FEAT_SME2
1	SUDOT (4-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long MLA two sources

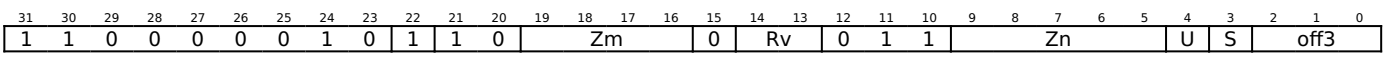
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields			Instruction Details	Condition
U	S	op		
		1	UNALLOCATED	-
0	0	0	SMLAL (multiple and single vector)	FEAT_SME2
0	1	0	SMLSL (multiple and single vector)	FEAT_SME2
1	0	0	UMLAL (multiple and single vector)	FEAT_SME2
1	1	0	UMLSL (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector long MLA one source

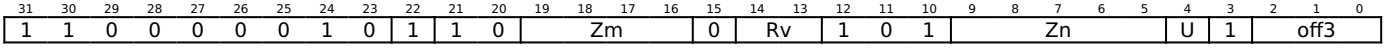
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields U	S	Instruction Details	Condition
0	0	SMLAL (multiple and single vector)	FEAT_SME2
0	1	SMLSL (multiple and single vector)	FEAT_SME2
1	0	UMLAL (multiple and single vector)	FEAT_SME2
1	1	UMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi two-way dot product two registers

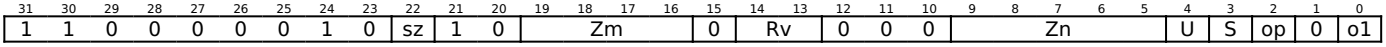
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields U	Instruction Details	Condition
0	SDOT (2-way, multiple and single vector)	FEAT_SME2
1	UDOT (2-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long long MLA two sources

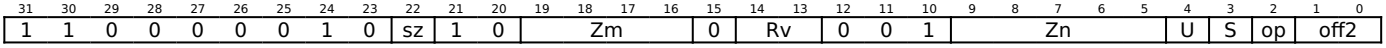
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields sz	U	S	op	Instruction Details	Condition
	0	0	0	SMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	0	1	0	SMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	0	0	UMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	1	0	UMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
0		1	1	UNALLOCATED	-
0	0	0	1	USMLALL (multiple and single vector)	FEAT_SME2
0	1	0	1	SUMLALL (multiple and single vector)	FEAT_SME2
1			1	UNALLOCATED	-

SME2 multiple and single vector long long FMA one source

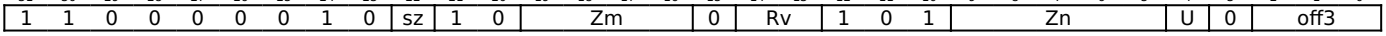
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields sz	U	S	op	Instruction Details	Condition
	0	0	0	SMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	0	1	0	SMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	0	0	UMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	1	0	UMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
0	0	0	1	USMLALL (multiple and single vector)	FEAT_SME2
0	0	1	1	UNALLOCATED	-
0	1		1	UNALLOCATED	-
1			1	UNALLOCATED	-

SME2 single-multi four-way dot product two registers

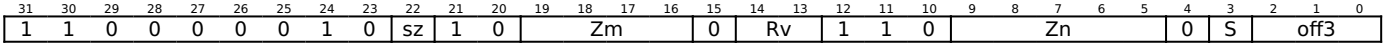
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields U	Instruction Details	Condition
0	SDOT (4-way, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	UDOT (4-way, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 single-multi ternary FP two registers

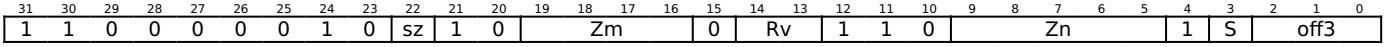
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields S	Instruction Details	Condition
0	FMLA (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FMLS (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 single-multi ternary int two registers

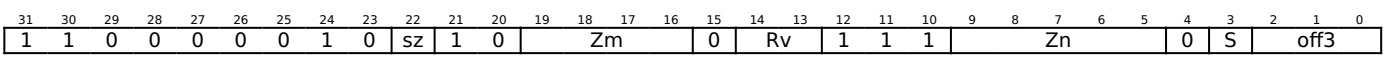
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields S	Instruction Details	Condition
0	ADD (to array, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 single-multi ternary FP16 two registers

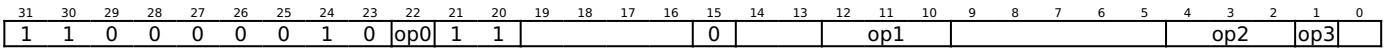
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Two registers).



Decode fields sz	Decode fields S	Instruction Details	Condition
0	0	FMLA (multiple and single vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and single vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and single vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and single vector)	FEAT_SME_B16B16

SME2 Multi-vector - Multiple and Single Array Vectors (Four registers)

The encodings in this section are decoded from SME encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
0	000	000	1	FMLALL (multiple and single vector) — Four ZA quad-vectors	FEAT_SME_F8F32
0	000	!= 000	1	UNALLOCATED	-
0	001	000		FMLALL (multiple and single vector) — One ZA quad-vector	FEAT_SME_F8F32
0	001	001		UNALLOCATED	-
0	010			SME2 single-multi long FMA four sources	-
0	011	00x		FMLAL (multiple and single vector, FP8 to FP16) — One ZA double-vector	FEAT_SME_F8F16
0	0x1	!= 00x		UNALLOCATED	-
0	100			SME2 single-multi FP dot product four registers	-
0	101	x1x		SME2 single-multi mixed dot product four registers	-
1	000		1	UNALLOCATED	-
1	010			SME2 single-multi long MLA four sources	-
1	0x1			UNALLOCATED	-
1	100			UNALLOCATED	-
1	101	x1x		SME2 single-multi two-way dot product four registers	-
	000		0	SME2 single-multi long long MLA four sources	-
	101	x0x		SME2 single-multi four-way dot product four registers	-
	110	0xx		SME2 single-multi ternary FP four registers	-
	110	1xx		SME2 single-multi ternary int four registers	-
	111	0xx		SME2 single-multi ternary FP16 four registers	-
	111	1xx		UNALLOCATED	-

SME2 single-multi long FMA four sources

The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	Zm				0	Rv		0	1	0	Zn					op	S	o2	off2	

Decode fields			Instruction Details	Condition
op	S	o2		
0	0	0	FMLAL (multiple and single vector, FP16 to FP32)	FEAT_SME2
0	0	1	FMLAL (multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
0	1	0	FMLSL (multiple and single vector)	FEAT_SME2
0	1	1	UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	BFMLAL (multiple and single vector)	FEAT_SME2
1	1	0	BFMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi FP dot product four registers

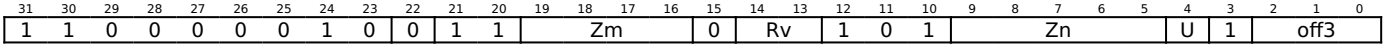
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	Zm				0	Rv		1	0	0	Zn					opc		off3		

Decode fields opc	Instruction Details	Condition
00	FDOT (2-way, multiple and single vector, FP16 to FP32)	FEAT_SME2
01	FDOT (2-way, multiple and single vector, FP8 to FP16)	FEAT_SME_F8F16
10	BFDOT (multiple and single vector)	FEAT_SME2
11	FDOT (4-way, multiple and single vector)	FEAT_SME_F8F32

SME2 single-multi mixed dot product four registers

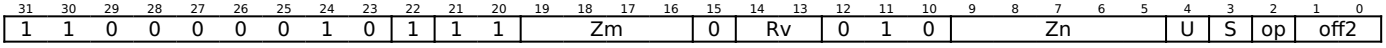
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields U	Instruction Details	Condition
0	USDOT (4-way, multiple and single vector)	FEAT_SME2
1	SUDOT (4-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long MLA four sources

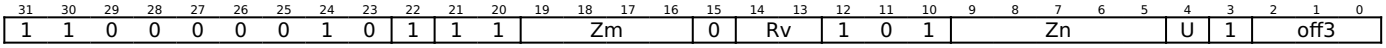
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields U	S	op	Instruction Details	Condition
		1	UNALLOCATED	-
0	0	0	SMLAL (multiple and single vector)	FEAT_SME2
0	1	0	SMLSL (multiple and single vector)	FEAT_SME2
1	0	0	UMLAL (multiple and single vector)	FEAT_SME2
1	1	0	UMLSL (multiple and single vector)	FEAT_SME2

SME2 single-multi two-way dot product four registers

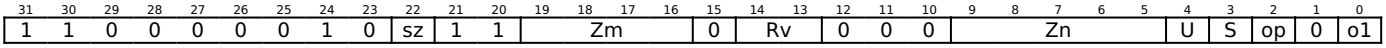
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields U	Instruction Details	Condition
0	SDOT (2-way, multiple and single vector)	FEAT_SME2
1	UDOT (2-way, multiple and single vector)	FEAT_SME2

SME2 single-multi long long MLA four sources

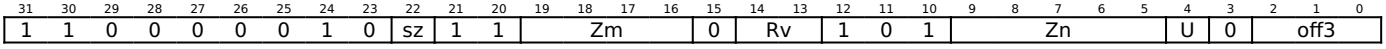
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields				Instruction Details	Condition
sz	U	S	op		
	0	0	0	SMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	0	1	0	SMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	0	0	UMLALL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	1	0	UMLSLL (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
0		1	1	UNALLOCATED	-
0	0	0	1	USMLALL (multiple and single vector)	FEAT_SME2
0	1	0	1	SUMLALL (multiple and single vector)	FEAT_SME2
1			1	UNALLOCATED	-

SME2 single-multi four-way dot product four registers

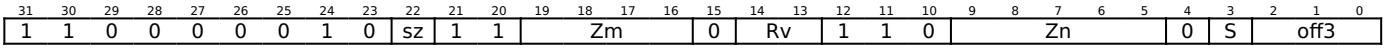
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields	Instruction Details	Condition
U		
0	SDOT (4-way, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	UDOT (4-way, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 single-multi ternary FP four registers

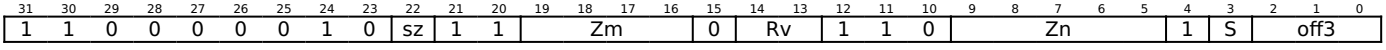
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields	Instruction Details	Condition
S		
0	FMLA (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FMLS (multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 single-multi ternary int four registers

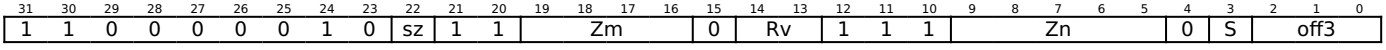
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields	Instruction Details	Condition
S		
0	ADD (to array, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, multiple and single vector)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 single-multi ternary FP16 four registers

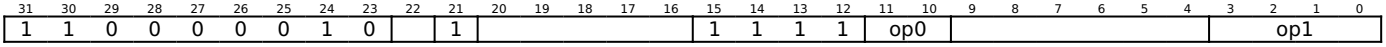
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single Array Vectors (Four registers).



Decode fields		Instruction Details	Condition
sz	S		
0	0	FMLA (multiple and single vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and single vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and single vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and single vector)	FEAT_SME_B16B16

SME2 Multi-vector - Lookup table with 6-bit indices

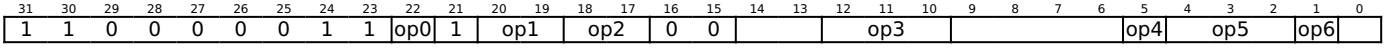
The encodings in this section are decoded from SME encodings.



Decode fields		Instruction details	Condition
op0	op1		
01	xx00	LUTI6 (vector, 16-bit) — Consecutive	FEAT_SME2p3
01	!= xx00	UNALLOCATED	-
11	00xx	LUTI6 (vector, 16-bit) — Strided	FEAT_SME2p3
11	!= 00xx	UNALLOCATED	-
x0		UNALLOCATED	-

SME2 Multi-vector - Multiple Array Vectors (Two registers)

The encodings in this section are decoded from SME encodings.



Decode fields							Instruction details	Condition
op0	op1	op2	op3	op4	op5	op6		
0			000	1	000	0	FMLALL (multiple vectors) — Two ZA quad-vectors	FEAT_SME_F8F32
0			000	1	001	0	UNALLOCATED	-
0			000	1	10x	0	UNALLOCATED	-
0			000	1	x0x	1	UNALLOCATED	-
0			010	0	xx0		SME2 multiple vectors long FMA two sources	-
0			010	1	000		FMLAL (multiple vectors, FP8 to FP16) — Two ZA double-vectors	FEAT_SME_F8F16
0			010	1	100		UNALLOCATED	-
0			010	1	x01		UNALLOCATED	-
0			100		x0x		SME2 multiple vectors FP dot product two registers	-
0			101	0	01x		USDOT (4-way, multiple vectors) — Two ZA single-vectors	FEAT_SME2
0			101	0	11x		UNALLOCATED	-
0			110	1	x0x		UNALLOCATED	-
0			xx0	1	x1x		UNALLOCATED	-
0			!= xx0	1			UNALLOCATED	-
1			010	0	xx0		SME2 multiple vectors long MLA two sources	-
1			100	0	x0x		UNALLOCATED	-
1			101	0	x1x		SME2 multiple vectors two-way dot product two registers	-
1				1			UNALLOCATED	-
	00	00	111	0	0xx		SME2 multiple vectors binary FP two registers	-
	00	00	111	0	1xx		SME2 multiple vectors binary int two registers	-
	00	10	111	0	0xx		SME2 multiple vectors binary FP16 two registers	-
	00	10	111	0	1xx		UNALLOCATED	-
	00	x1	111	0			UNALLOCATED	-
			000	0		0	SME2 multiple vectors long long MLA two sources	-
			000	0		1	UNALLOCATED	-
			010	0	xx1		UNALLOCATED	-
			0x1	0			UNALLOCATED	-
			100	0	x1x		SME2 multiple vectors ternary FP16 two registers	-
			101	0	x0x		SME2 multiple vectors four-way dot product two registers	-
			110	0	0xx		SME2 multiple vectors ternary FP two registers	-
			110	0	1xx		SME2 multiple vectors ternary int two registers	-
	!= 00		111	0			UNALLOCATED	-

SME2 multiple vectors long FMA two sources

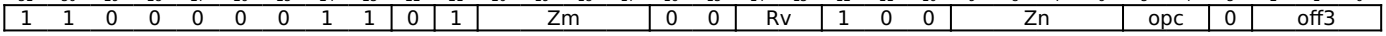
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	1	0	1	Zm				0	0	Rv		0	1	0	Zn				0	op	S	0	off2	

Decode fields		Instruction Details	Condition
op	S		
0	0	FMLAL (multiple vectors, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple vectors)	FEAT_SME2
1	0	BFMLAL (multiple vectors)	FEAT_SME2
1	1	BFMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors FP dot product two registers

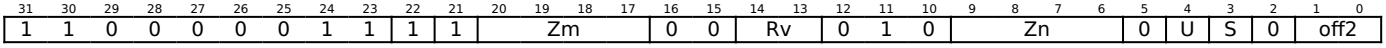
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields	Instruction Details	Condition
opc		
00	FDOT (2-way, multiple vectors, FP16 to FP32)	FEAT_SME2
01	BFDOT (multiple vectors)	FEAT_SME2
10	FDOT (2-way, multiple vectors, FP8 to FP16)	FEAT_SME_F8F16
11	FDOT (4-way, multiple vectors)	FEAT_SME_F8F32

SME2 multiple vectors long MLA two sources

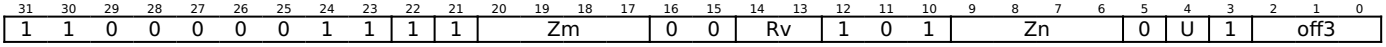
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields		Instruction Details	Condition
U	S		
0	0	SMLAL (multiple vectors)	FEAT_SME2
0	1	SMLSL (multiple vectors)	FEAT_SME2
1	0	UMLAL (multiple vectors)	FEAT_SME2
1	1	UMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors two-way dot product two registers

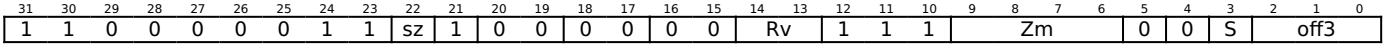
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields	Instruction Details	Condition
U		
0	SDOT (2-way, multiple vectors)	FEAT_SME2
1	UDOT (2-way, multiple vectors)	FEAT_SME2

SME2 multiple vectors binary FP two registers

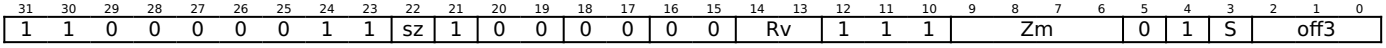
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields	Instruction Details	Condition
S		
0	FADD	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FSUB	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 multiple vectors binary int two registers

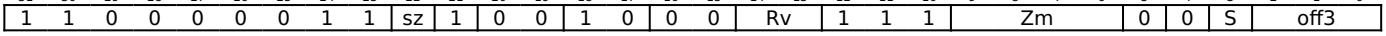
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields S	Instruction Details	Condition
0	ADD (to array, array and multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, array and multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 multiple vectors binary FP16 two registers

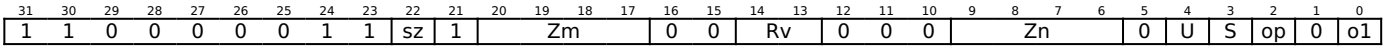
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields sz	S	Instruction Details	Condition
0	0	FADD	FEAT_SME_F16F16 || FEAT_SME_F8F16
0	1	FSUB	FEAT_SME_F16F16 || FEAT_SME_F8F16
1	0	BFADD	FEAT_SME_B16B16
1	1	BFSUB	FEAT_SME_B16B16

SME2 multiple vectors long long MLA two sources

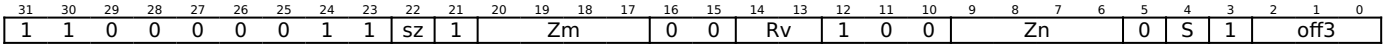
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields sz	U	S	op	Instruction Details	Condition
	0	0	0	SMLALL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	0	1	0	SMLSLL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	0	0	UMLALL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	1	0	UMLSLL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
0	0	0	1	USMLALL (multiple vectors)	FEAT_SME2
0	0	1	1	UNALLOCATED	-
0	1		1	UNALLOCATED	-
1			1	UNALLOCATED	-

SME2 multiple vectors ternary FP16 two registers

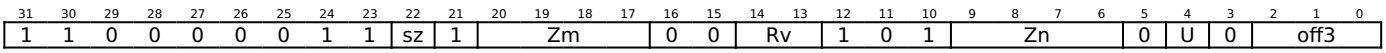
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields sz	S	Instruction Details	Condition
0	0	FMLA (multiple vectors)	FEAT_SME_F16F16
0	1	FMLS (multiple vectors)	FEAT_SME_F16F16
1	0	BFMLA (multiple vectors)	FEAT_SME_B16B16
1	1	BFMLS (multiple vectors)	FEAT_SME_B16B16

SME2 multiple vectors four-way dot product two registers

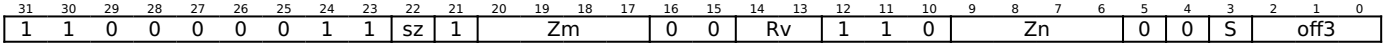
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields U	Instruction Details	Condition
0	SDOT (4-way, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	UDOT (4-way, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 multiple vectors ternary FP two registers

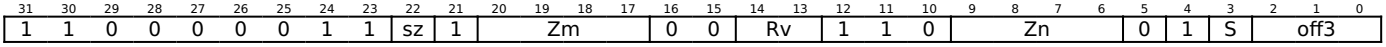
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields S	Instruction Details	Condition
0	FMLA (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FMLS (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 multiple vectors ternary int two registers

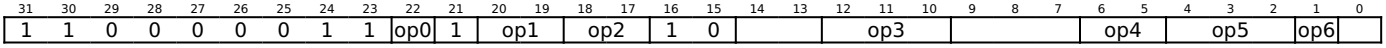
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Two registers).



Decode fields S	Instruction Details	Condition
0	ADD (to array, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 Multi-vector - Multiple Array Vectors (Four registers)

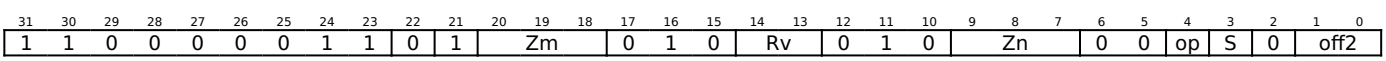
The encodings in this section are decoded from SME encodings.



Decode fields							Instruction details	Condition
op0	op1	op2	op3	op4	op5	op6		
0		x0	000	01	000	0	FMLALL (multiple vectors) — Four ZA quad-vectors	FEAT_SME_F8F32
0		x0	000	01	001	0	UNALLOCATED	-
0		x0	000	01	10x	0	UNALLOCATED	-
0		x0	000	01	x0x	1	UNALLOCATED	-
0		x0	010	00	xx0		SME2 multiple vectors long FMA four sources	-
0		x0	010	01	000		FMLAL (multiple vectors, FP8 to FP16) — Four ZA double-vectors	FEAT_SME_F8F16
0		x0	010	01	100		UNALLOCATED	-
0		x0	010	01	x01		UNALLOCATED	-
0		x0	100	0x	x0x		SME2 multiple vectors FP dot product four registers	-
0		x0	101	00	01x		USDOT (4-way, multiple vectors) — Four ZA single-vectors	FEAT_SME2
0		x0	101	00	11x		UNALLOCATED	-
0		x0	110	01	x0x		UNALLOCATED	-
0		x0	xx0	01	x1x		UNALLOCATED	-
0		x0	!= xx0	01			UNALLOCATED	-
1		x0	010	00	xx0		SME2 multiple vectors long MLA four sources	-
1		x0	100	00	x0x		UNALLOCATED	-
1		x0	101	00	x1x		SME2 multiple vectors two-way dot product four registers	-
1		x0		01			UNALLOCATED	-
	00	00	111	00	0xx		SME2 multiple vectors binary FP four registers	-
	00	00	111	00	1xx		SME2 multiple vectors binary int four registers	-
	00	10	111	00	0xx		SME2 multiple vectors binary FP16 four registers	-
	00	10	111	00	1xx		UNALLOCATED	-
		x0	000	00		0	SME2 multiple vectors long long MLA four sources	-
		x0	000	00		1	UNALLOCATED	-
		x0	010	00	xx1		UNALLOCATED	-
		x0	0x1	00			UNALLOCATED	-
		x0	100	00	x1x		SME2 multiple vectors ternary FP16 four registers	-
		x0	101	00	x0x		SME2 multiple vectors four-way dot product four registers	-
		x0	110	00	0xx		SME2 multiple vectors ternary FP four registers	-
		x0	110	00	1xx		SME2 multiple vectors ternary int four registers	-
		x0		1x			UNALLOCATED	-
		x1					UNALLOCATED	-
	!= 00	x0	111	00			UNALLOCATED	-

SME2 multiple vectors long FMA four sources

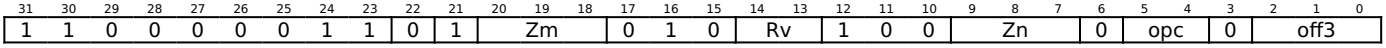
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields		Instruction Details	Condition
op	S		
0	0	FMLAL (multiple vectors, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple vectors)	FEAT_SME2
1	0	BFMLAL (multiple vectors)	FEAT_SME2
1	1	BFMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors FP dot product four registers

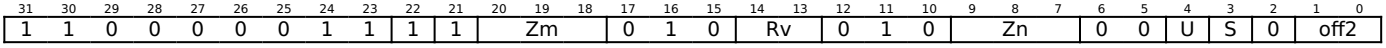
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields	Instruction Details	Condition
opc		
00	FDOT (2-way, multiple vectors, FP16 to FP32)	FEAT_SME2
01	BFDOT (multiple vectors)	FEAT_SME2
10	FDOT (2-way, multiple vectors, FP8 to FP16)	FEAT_SME_F8F16
11	FDOT (4-way, multiple vectors)	FEAT_SME_F8F32

SME2 multiple vectors long MLA four sources

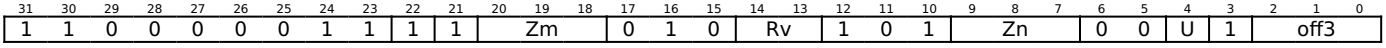
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields		Instruction Details	Condition
U	S		
0	0	SMLAL (multiple vectors)	FEAT_SME2
0	1	SMLSL (multiple vectors)	FEAT_SME2
1	0	UMLAL (multiple vectors)	FEAT_SME2
1	1	UMLSL (multiple vectors)	FEAT_SME2

SME2 multiple vectors two-way dot product four registers

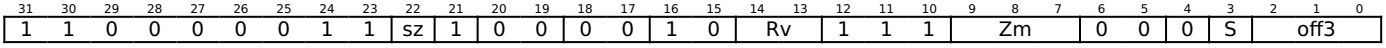
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields	Instruction Details	Condition
U		
0	SDOT (2-way, multiple vectors)	FEAT_SME2
1	UDOT (2-way, multiple vectors)	FEAT_SME2

SME2 multiple vectors binary FP four registers

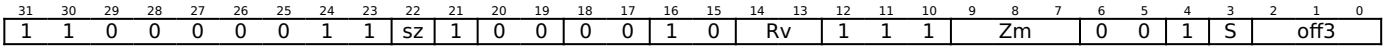
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields	Instruction Details	Condition
S		
0	FADD	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FSUB	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 multiple vectors binary int four registers

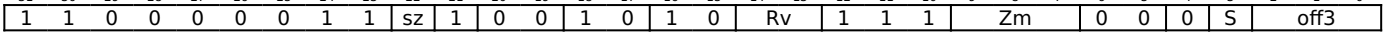
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields S	Instruction Details	Condition
0	ADD (to array, array and multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, array and multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 multiple vectors binary FP16 four registers

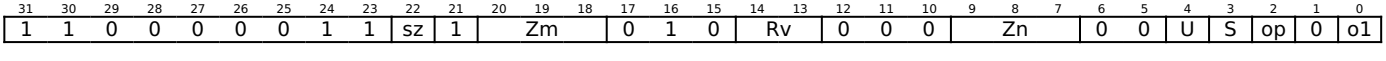
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields sz	S	Instruction Details	Condition
0	0	FADD	FEAT_SME_F16F16 || FEAT_SME_F8F16
0	1	FSUB	FEAT_SME_F16F16 || FEAT_SME_F8F16
1	0	BFADD	FEAT_SME_B16B16
1	1	BFSUB	FEAT_SME_B16B16

SME2 multiple vectors long long MLA four sources

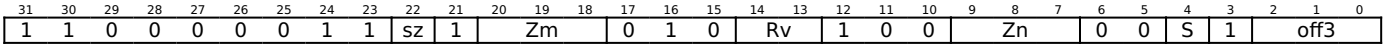
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields sz	U	S	op	Instruction Details	Condition
	0	0	0	SMLALL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	0	1	0	SMLSLL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	0	0	UMLALL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
	1	1	0	UMLSLL (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
0	0	0	1	USMLALL (multiple vectors)	FEAT_SME2
0	0	1	1	UNALLOCATED	-
0	1		1	UNALLOCATED	-
1			1	UNALLOCATED	-

SME2 multiple vectors ternary FP16 four registers

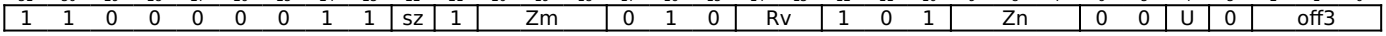
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields sz	S	Instruction Details	Condition
0	0	FMLA (multiple vectors)	FEAT_SME_F16F16
0	1	FMLS (multiple vectors)	FEAT_SME_F16F16
1	0	BFMLA (multiple vectors)	FEAT_SME_B16B16
1	1	BFMLS (multiple vectors)	FEAT_SME_B16B16

SME2 multiple vectors four-way dot product four registers

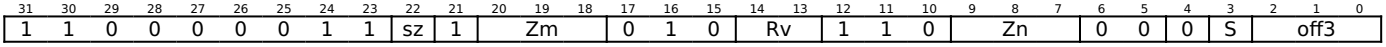
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields U	Instruction Details	Condition
0	SDOT (4-way, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	UDOT (4-way, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 multiple vectors ternary FP four registers

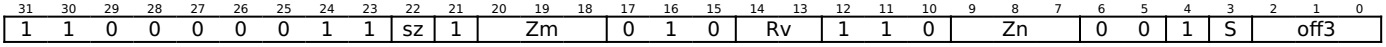
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields S	Instruction Details	Condition
0	FMLA (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)
1	FMLS (multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_F64F64)

SME2 multiple vectors ternary int four registers

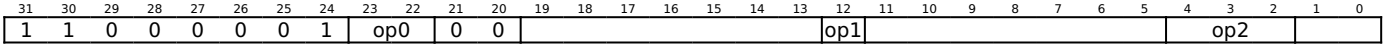
The encodings in this section are decoded from SME2 Multi-vector - Multiple Array Vectors (Four registers).



Decode fields S	Instruction Details	Condition
0	ADD (to array, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)
1	SUB (to array, multiple vectors)	FEAT_SME2 && (sz == '0' || FEAT_SME_I16I64)

SME2 Multi-vector - Indexed (One register)

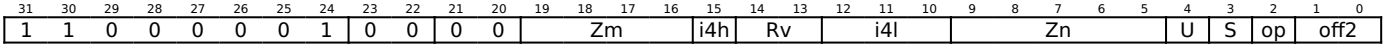
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	Instruction details	Condition
00			SME2 multi-vec indexed long long MLA one source 32-bit	-
01		000	FMLALL (multiple and indexed vector) — One ZA quad-vector	FEAT_SME_F8F32
01		!= 000	UNALLOCATED	-
10	0	xx0	SME2 multi-vec indexed long long MLA one source 64-bit	-
10	0	xx1	UNALLOCATED	-
10	1		SME2 multi-vec indexed long FMA one source	-
11	0	0xx	FMLAL (multiple and indexed vector, FP8 to FP16) — One ZA double-vector	FEAT_SME_F8F16
11	0	1xx	UNALLOCATED	-
11	1		SME2 multi-vec indexed long MLA one source	-

SME2 multi-vec indexed long long MLA one source 32-bit

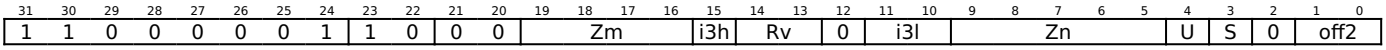
The encodings in this section are decoded from SME2 Multi-vector - Indexed (One register).



Decode fields U	S	op	Instruction Details	Condition
	1	1	UNALLOCATED	-
0	0	0	SMLALL (multiple and indexed vector)	FEAT_SME2
0	0	1	USMLALL (multiple and indexed vector)	FEAT_SME2
0	1	0	SMLSLL (multiple and indexed vector)	FEAT_SME2
1	0	0	UMLALL (multiple and indexed vector)	FEAT_SME2
1	0	1	SUMLALL (multiple and indexed vector)	FEAT_SME2
1	1	0	UMLSLL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long long MLA one source 64-bit

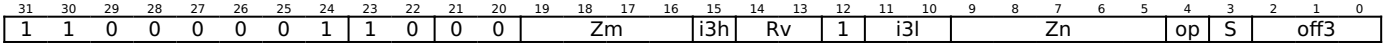
The encodings in this section are decoded from SME2 Multi-vector - Indexed (One register).



Decode fields U	S	Instruction Details	Condition
0	0	SMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	0	UMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long FMA one source

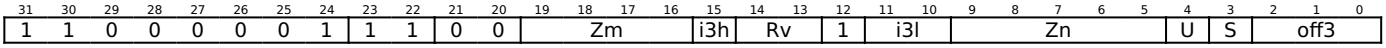
The encodings in this section are decoded from SME2 Multi-vector - Indexed (One register).



Decode fields op	S	Instruction Details	Condition
0	0	FMLAL (multiple and indexed vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and indexed vector)	FEAT_SME2
1	0	BFMLAL (multiple and indexed vector)	FEAT_SME2
1	1	BFMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long MLA one source

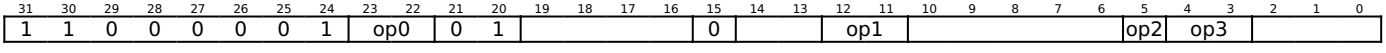
The encodings in this section are decoded from SME2 Multi-vector - Indexed (One register).



Decode fields U	S	Instruction Details	Condition
0	0	SMLAL (multiple and indexed vector)	FEAT_SME2
0	1	SMLSL (multiple and indexed vector)	FEAT_SME2
1	0	UMLAL (multiple and indexed vector)	FEAT_SME2
1	1	UMLSL (multiple and indexed vector)	FEAT_SME2

SME2 Multi-vector - Indexed (Two registers)

The encodings in this section are decoded from SME encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
00	0x			SME2 multi-vec indexed long long MLA two sources 32-bit	-
00	1x			SME2 multi-vec ternary indexed two registers 16-bit	-
01				SME2 multi-vec ternary indexed two registers 32-bit	-
10	00	0		SME2 multi-vec indexed long long MLA two sources 64-bit	-
10	01	0		UNALLOCATED	-
10	0x	1	00	FMLALL (multiple and indexed vector) — Two ZA quad-vectors	FEAT_SME_F8F32
10	0x	1	!= 00	UNALLOCATED	-
10	1x	0		SME2 multi-vec indexed long FMA two sources	-
10	1x	1	0x	UNALLOCATED	-
10	1x	1	1x	FMLAL (multiple and indexed vector, FP8 to FP16) — Two ZA double-vectors	FEAT_SME_F8F16
11	00	0		SME2 multi-vec ternary indexed two registers 64-bit	-
11	01	0		SME2 multi-vec indexed FP8 two-way vertical dot product to single-precision two registers	-
11	1x	0		SME2 multi-vec indexed long MLA two sources	-
11		1	0x	SME2 multi-vec indexed FP8 two-way dot product to FP16 two registers	-
11		1	1x	UNALLOCATED	-

SME2 multi-vec indexed long long MLA two sources 32-bit

The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	1	Zm				0	Rv			0	i4h	Zn				op	U	S	i4l		o1

Decode fields			Instruction Details	Condition
op	U	S		
0	0	0	SMLALL (multiple and indexed vector)	FEAT_SME2
0	0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2
0	1	0	UMLALL (multiple and indexed vector)	FEAT_SME2
0	1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2
1		1	UNALLOCATED	-
1	0	0	USMLALL (multiple and indexed vector)	FEAT_SME2
1	1	0	SUMLALL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed two registers 16-bit

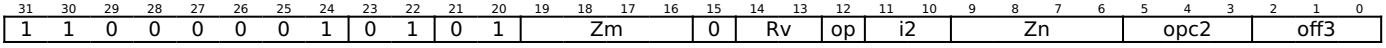
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	1	Zm				0	Rv			1	i3h	Zn				op	S	i3l		off3	

Decode fields		Instruction Details	Condition
op	S		
0	0	FMLA (multiple and indexed vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and indexed vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and indexed vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and indexed vector)	FEAT_SME_B16B16

SME2 multi-vec ternary indexed two registers 32-bit

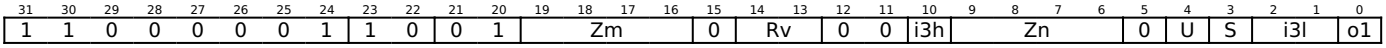
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields op	opc2	Instruction Details	Condition
0	000	FMLA (multiple and indexed vector)	FEAT_SME2
0	001	FVDOT (FP16 to FP32)	FEAT_SME2
0	010	FMLS (multiple and indexed vector)	FEAT_SME2
0	011	BFVDOT	FEAT_SME2
0	100	SVDOT (2-way)	FEAT_SME2
0	101	UNALLOCATED	-
0	110	UVDOT (2-way)	FEAT_SME2
0	111	FDOT (4-way, multiple and indexed vector)	FEAT_SME_F8F32
1	000	SDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	001	FDOT (2-way, multiple and indexed vector, FP16 to FP32)	FEAT_SME2
1	010	UDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	011	BFDOT (multiple and indexed vector)	FEAT_SME2
1	100	SDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	101	USDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	110	UDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	111	SUDOT (4-way, multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long long MLA two sources 64-bit

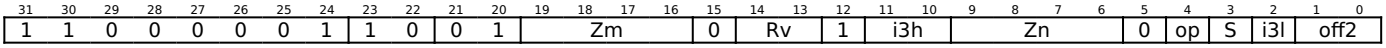
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields U	S	Instruction Details	Condition
0	0	SMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	0	UMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long FMA two sources

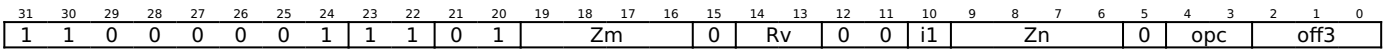
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields op	S	Instruction Details	Condition
0	0	FMLAL (multiple and indexed vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and indexed vector)	FEAT_SME2
1	0	BFMLAL (multiple and indexed vector)	FEAT_SME2
1	1	BFMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed two registers 64-bit

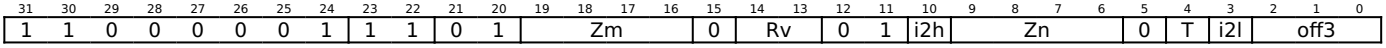
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields opc	Instruction Details	Condition
00	FMLA (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
01	SDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
10	FMLS (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
11	UDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed FP8 two-way vertical dot product to single-precision two registers

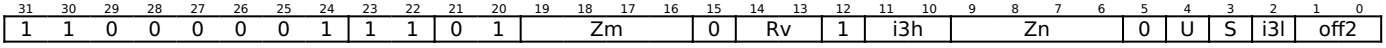
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields T	Instruction Details	Condition
0	FVDOTB	FEAT_SME_F8F32
1	FVDOTT	FEAT_SME_F8F32

SME2 multi-vec indexed long MLA two sources

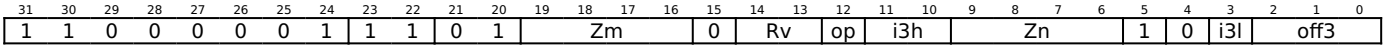
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields U	Decode fields S	Instruction Details	Condition
0	0	SMLAL (multiple and indexed vector)	FEAT_SME2
0	1	SMLSL (multiple and indexed vector)	FEAT_SME2
1	0	UMLAL (multiple and indexed vector)	FEAT_SME2
1	1	UMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed FP8 two-way dot product to FP16 two registers

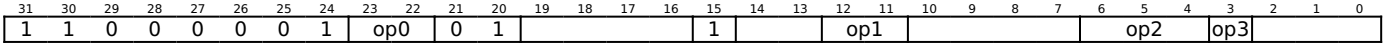
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Two registers).



Decode fields op	Instruction Details	Condition
0	FDOT (2-way, multiple and indexed vector, FP8 to FP16)	FEAT_SME_F8F16
1	FVDOT (FP8 to FP16)	FEAT_SME_F8F16

SME2 Multi-vector - Indexed (Four registers)

The encodings in this section are decoded from SME encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
00	0x	0xx		SME2 multi-vec indexed long long MLA four sources 32-bit	-
00	0x	100	0	FMLALL (multiple and indexed vector) — Four ZA quad-vectors	FEAT_SME_F8F32
00	0x	100	1	UNALLOCATED	-
00	1x	0xx		SME2 multi-vec ternary indexed four registers 16-bit	-
00	1x	100		FDOT (2-way, multiple and indexed vector, FP8 to FP16) — Four ZA single-vectors	FEAT_SME_F8F16
00		101		UNALLOCATED	-
00		11x		UNALLOCATED	-
01		0xx		SME2 multi-vec ternary indexed four registers 32-bit	-
10	00	00x		SME2 multi-vec indexed long long MLA four sources 64-bit	-
10	01	00x		UNALLOCATED	-
10	0x	01x		UNALLOCATED	-
10	1x	00x		SME2 multi-vec indexed long FMA four sources	-
10	1x	010		FMLAL (multiple and indexed vector, FP8 to FP16) — Four ZA double-vectors	FEAT_SME_F8F16
10	1x	011		UNALLOCATED	-
11	0x	00x		SME2 multi-vec ternary indexed four registers 64-bit	-
11	1x	00x		SME2 multi-vec indexed long MLA four sources	-
11		01x		UNALLOCATED	-
!= 00		1xx		UNALLOCATED	-

SME2 multi-vec indexed long long MLA four sources 32-bit

The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	1	Zm				1	Rv		0	i4h		Zn			0	op	U	S	i4l		o1

Decode fields			Instruction Details	Condition
op	U	S		
0	0	0	SMLALL (multiple and indexed vector)	FEAT_SME2
0	0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2
0	1	0	UMLALL (multiple and indexed vector)	FEAT_SME2
0	1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2
1		1	UNALLOCATED	-
1	0	0	USMLALL (multiple and indexed vector)	FEAT_SME2
1	1	0	SUMLALL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed four registers 16-bit

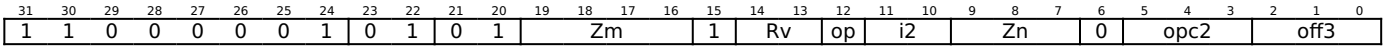
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	0	1	Zm				1	Rv		1	i3h		Zn			0	op	S	i3l		off3	

Decode fields		Instruction Details	Condition
op	S		
0	0	FMLA (multiple and indexed vector)	FEAT_SME_F16F16
0	1	FMLS (multiple and indexed vector)	FEAT_SME_F16F16
1	0	BFMLA (multiple and indexed vector)	FEAT_SME_B16B16
1	1	BFMLS (multiple and indexed vector)	FEAT_SME_B16B16

SME2 multi-vec ternary indexed four registers 32-bit

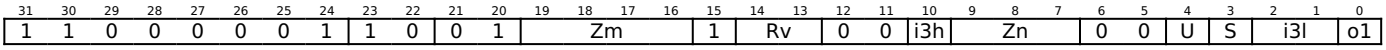
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields op	Decode fields opc2	Instruction Details	Condition
0	000	FMLA (multiple and indexed vector)	FEAT_SME2
0	001	FDOT (4-way, multiple and indexed vector)	FEAT_SME_F8F32
0	010	FMLS (multiple and indexed vector)	FEAT_SME2
0	011	UNALLOCATED	-
0	100	SVDOT (4-way)	FEAT_SME2
0	101	USVDOT	FEAT_SME2
0	110	UVDOT (4-way)	FEAT_SME2
0	111	SUVDOT	FEAT_SME2
1	000	SDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	001	FDOT (2-way, multiple and indexed vector, FP16 to FP32)	FEAT_SME2
1	010	UDOT (2-way, multiple and indexed vector)	FEAT_SME2
1	011	BFDOT (multiple and indexed vector)	FEAT_SME2
1	100	SDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	101	USDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	110	UDOT (4-way, multiple and indexed vector)	FEAT_SME2
1	111	SUDOT (4-way, multiple and indexed vector)	FEAT_SME2

SME2 multi-vec indexed long long MLA four sources 64-bit

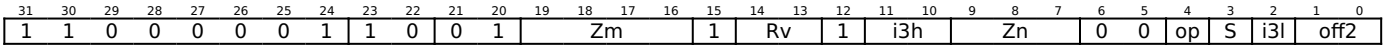
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields U	Decode fields S	Instruction Details	Condition
0	0	SMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	1	SMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	0	UMLALL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	1	UMLSLL (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long FMA four sources

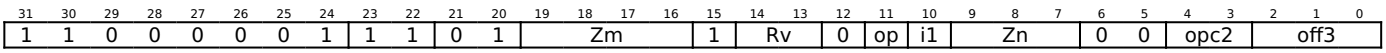
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields op	Decode fields S	Instruction Details	Condition
0	0	FMLAL (multiple and indexed vector, FP16 to FP32)	FEAT_SME2
0	1	FMLSL (multiple and indexed vector)	FEAT_SME2
1	0	BFMLAL (multiple and indexed vector)	FEAT_SME2
1	1	BFMLSL (multiple and indexed vector)	FEAT_SME2

SME2 multi-vec ternary indexed four registers 64-bit

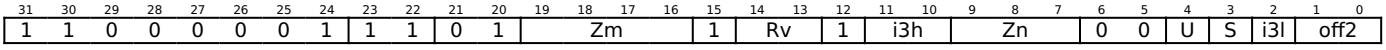
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields op	Decode fields opc2	Instruction Details	Condition
0	00	FMLA (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
0	01	SDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
0	10	FMLS (multiple and indexed vector)	FEAT_SME2 && FEAT_SME_F64F64
0	11	UDOT (4-way, multiple and indexed vector)	FEAT_SME2 && FEAT_SME_I16I64
1	x0	UNALLOCATED	-
1	01	SVDOT (4-way)	FEAT_SME2 && FEAT_SME_I16I64
1	11	UVDOT (4-way)	FEAT_SME2 && FEAT_SME_I16I64

SME2 multi-vec indexed long MLA four sources

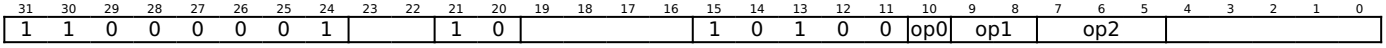
The encodings in this section are decoded from SME2 Multi-vector - Indexed (Four registers).



Decode fields U	Decode fields S	Instruction Details	Condition
0	0	SMLAL (multiple and indexed vector)	FEAT_SME2
0	1	SMLSL (multiple and indexed vector)	FEAT_SME2
1	0	UMLAL (multiple and indexed vector)	FEAT_SME2
1	1	UMLSL (multiple and indexed vector)	FEAT_SME2

SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers)

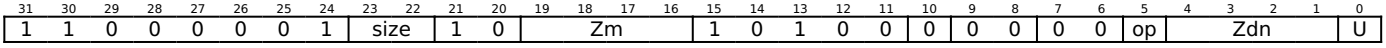
The encodings in this section are decoded from SME encodings.



Decode fields op0	Decode fields op1	Decode fields op2	Instruction details
0	00	00x	SME2 single-multi int min/max two registers
0	01	00x	SME2 single-multi FP min/max two registers
0	01	100	SME2 multiple and single vector FSCALE two registers
0	10		SME2 single-multi shift two registers
0	11	000	SME2 single-multi add two registers
0	11	001	UNALLOCATED
0	11	10x	UNALLOCATED
0	!= 1x	101	UNALLOCATED
0	!= 10	x1x	UNALLOCATED
1	00	000	SME2 single-multi signed saturating doubling multiply high two registers
1	00	!= x00	UNALLOCATED
1	!= 00		UNALLOCATED
	00	100	UNALLOCATED

SME2 single-multi int min/max two registers

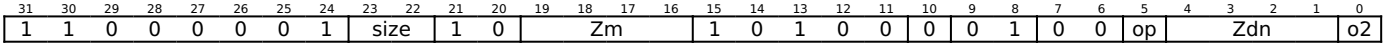
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields		Instruction Details	Condition
op	U		
0	0	SMAX (multiple and single vector)	FEAT_SME2
0	1	UMAX (multiple and single vector)	FEAT_SME2
1	0	SMIN (multiple and single vector)	FEAT_SME2
1	1	UMIN (multiple and single vector)	FEAT_SME2

SME2 single-multi FP min/max two registers

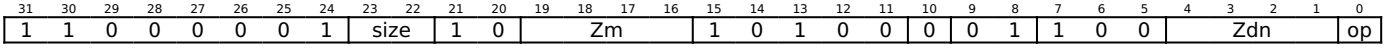
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields			Instruction Details	Condition
size	op	o2		
00	0	0	BFMAX (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	0	1	BFMIN (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	0	BFMAXNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	1	BFMINNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	0	FMAX (multiple and single vector)	FEAT_SME2
!= 00	0	1	FMIN (multiple and single vector)	FEAT_SME2
!= 00	1	0	FMAXNM (multiple and single vector)	FEAT_SME2
!= 00	1	1	FMINNM (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector FSCALE two registers

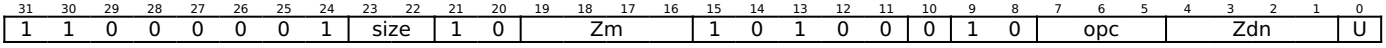
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields		Instruction Details	Condition
size	op		
	1	UNALLOCATED	-
00	0	BFSCALE (multiple and single vector)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	0	FSCALE (multiple and single vector)	FEAT_SME2 && FEAT_FP8

SME2 single-multi shift two registers

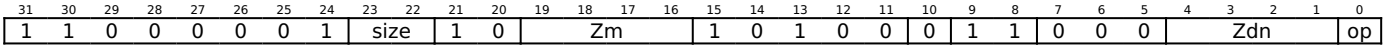
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields		Instruction Details	Condition
opc	U		
001	0	SRSHL (multiple and single vector)	FEAT_SME2
001	1	URSHL (multiple and single vector)	FEAT_SME2
!= 001		UNALLOCATED	-

SME2 single-multi add two registers

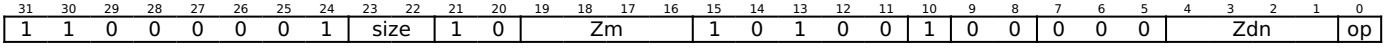
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields op	Instruction Details	Condition
0	ADD (to vector, multiple vectors)	FEAT_SME2
1	UNALLOCATED	-

SME2 single-multi signed saturating doubling multiply high two registers

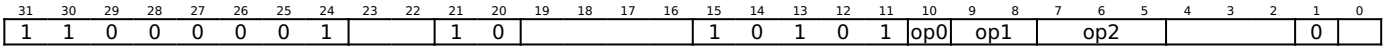
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Two registers).



Decode fields op	Instruction Details	Condition
0	SQDMULH (multiple and single vector)	FEAT_SME2
1	UNALLOCATED	-

SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers)

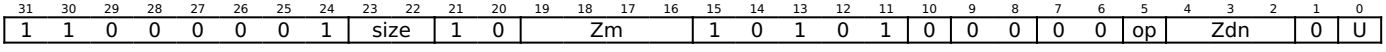
The encodings in this section are decoded from SME encodings.



Decode fields op0	Decode fields op1	Decode fields op2	Instruction details
0	00	00x	SME2 single-multi int min/max four registers
0	01	00x	SME2 single-multi FP min/max four registers
0	01	100	SME2 multiple and single vector FSCALE four registers
0	10		SME2 single-multi shift four registers
0	11	000	SME2 single-multi add four registers
0	11	001	UNALLOCATED
0	11	10x	UNALLOCATED
0	!= 1x	101	UNALLOCATED
0	!= 10	x1x	UNALLOCATED
1	00	000	SME2 single-multi signed saturating doubling multiply high four registers
1	00	!= x00	UNALLOCATED
1	!= 00		UNALLOCATED
	00	100	UNALLOCATED

SME2 single-multi int min/max four registers

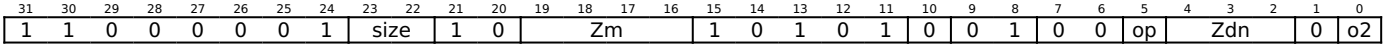
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields op	Decode fields U	Instruction Details	Condition
0	0	SMAX (multiple and single vector)	FEAT_SME2
0	1	UMAX (multiple and single vector)	FEAT_SME2
1	0	SMIN (multiple and single vector)	FEAT_SME2
1	1	UMIN (multiple and single vector)	FEAT_SME2

SME2 single-multi FP min/max four registers

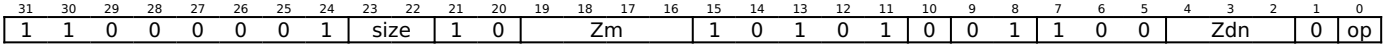
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields size	op	o2	Instruction Details	Condition
00	0	0	BFMAX (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	0	1	BFMIN (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	0	BFMAXNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
00	1	1	BFMINNM (multiple and single vector)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	0	FMAX (multiple and single vector)	FEAT_SME2
!= 00	0	1	FMIN (multiple and single vector)	FEAT_SME2
!= 00	1	0	FMAXNM (multiple and single vector)	FEAT_SME2
!= 00	1	1	FMINNM (multiple and single vector)	FEAT_SME2

SME2 multiple and single vector FSCALE four registers

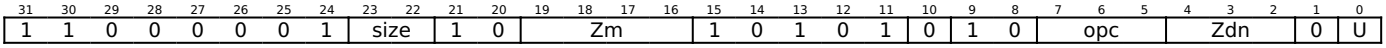
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields size	op	Instruction Details	Condition
	1	UNALLOCATED	-
00	0	BFSCALE (multiple and single vector)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	0	FSCALE (multiple and single vector)	FEAT_SME2 && FEAT_FP8

SME2 single-multi shift four registers

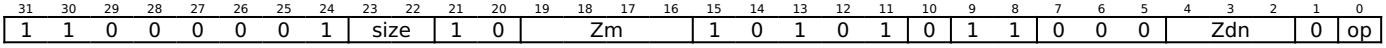
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields opc	U	Instruction Details	Condition
001	0	SRSHL (multiple and single vector)	FEAT_SME2
001	1	URSHL (multiple and single vector)	FEAT_SME2
!= 001		UNALLOCATED	-

SME2 single-multi add four registers

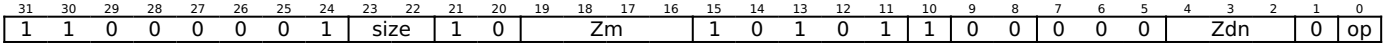
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields op	Instruction Details	Condition
0	ADD (to vector, multiple vectors)	FEAT_SME2
1	UNALLOCATED	-

SME2 single-multi signed saturating doubling multiply high four registers

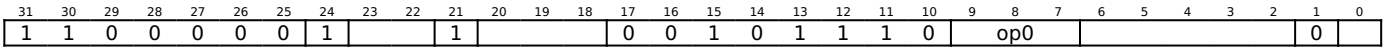
The encodings in this section are decoded from SME2 Multi-vector - Multiple and Single SVE Destructive (Four registers).



Decode fields op	Instruction Details	Condition
0	SQDMULH (multiple and single vector)	FEAT_SME2
1	UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers)

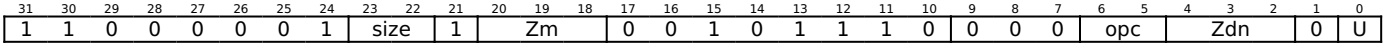
The encodings in this section are decoded from SME encodings.



Decode fields op0	Instruction details
000	SME2 multiple vectors int min/max four registers
001	UNALLOCATED
010	SME2 multiple vectors FP min/max four registers
011	SME2 multiple vectors FSCALE four registers
10x	SME2 multiple vectors shift four registers
11x	UNALLOCATED

SME2 multiple vectors int min/max four registers

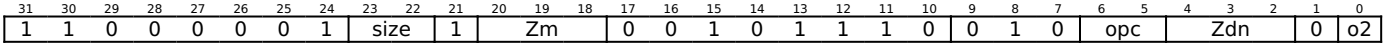
The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers).



Decode fields opc	U	Instruction Details	Condition
00	0	SMAX (multiple vectors)	FEAT_SME2
00	1	UMAX (multiple vectors)	FEAT_SME2
01	0	SMIN (multiple vectors)	FEAT_SME2
01	1	UMIN (multiple vectors)	FEAT_SME2
1x		UNALLOCATED	-

SME2 multiple vectors FP min/max four registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers).



Decode fields			Instruction Details	Condition
size	opc	o2		
	10	0	FAMAX	FEAT_SME2 && FEAT_FAMINMAX
	10	1	FAMIN	FEAT_SME2 && FEAT_FAMINMAX
	11		UNALLOCATED	-
00	00	0	BFMAX (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	00	1	BFMIN (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	01	0	BFMAXNM (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	01	1	BFMINNM (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	00	0	FMAX (multiple vectors)	FEAT_SME2
!= 00	00	1	FMIN (multiple vectors)	FEAT_SME2
!= 00	01	0	FMAXNM (multiple vectors)	FEAT_SME2
!= 00	01	1	FMINNM (multiple vectors)	FEAT_SME2

SME2 multiple vectors FSCALE four registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size		1	Zm			0	0	1	0	1	1	1	0	0	1	1	opc		Zdn			0	o2

Decode fields			Instruction Details	Condition
size	opc	o2		
	00	1	UNALLOCATED	-
	!= 00		UNALLOCATED	-
00	00	0	BFSCALE (multiple vectors)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	00	0	FSCALE (multiple vectors)	FEAT_SME2 && FEAT_FP8

SME2 multiple vectors shift four registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Four registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size		1	Zm			0	0	1	0	1	1	1	0	1	0	opc			Zdn			0	U

Decode fields		Instruction Details	Condition
opc	U		
001	0	SRSHL (multiple vectors)	FEAT_SME2
001	1	URSHL (multiple vectors)	FEAT_SME2
!= 001		UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Four registers)

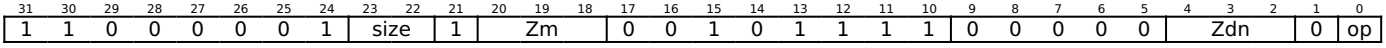
The encodings in this section are decoded from SME encodings.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1				0	0	1	0	1	1	1	1	op0								0	

Decode fields	Instruction details
op0	
00000	SME2 multi-vector signed saturating doubling multiply high four registers
!= 00000	UNALLOCATED

SME2 multi-vector signed saturating doubling multiply high four registers

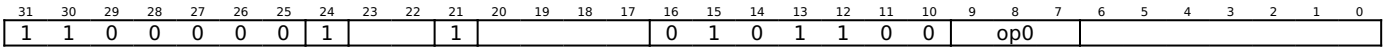
The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Four registers).



Decode fields op	Instruction Details	Condition
0	SQDMULH (multiple vectors)	FEAT_SME2
1	UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers)

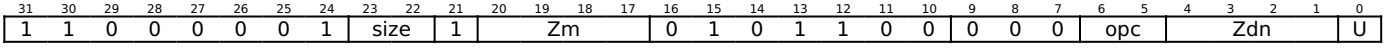
The encodings in this section are decoded from SME encodings.



Decode fields op0	Instruction details
000	SME2 multiple vectors int min/max two registers
001	UNALLOCATED
010	SME2 multiple vectors FP min/max two registers
011	SME2 multiple vectors FSCALE two registers
10x	SME2 multiple vectors shift two registers
11x	UNALLOCATED

SME2 multiple vectors int min/max two registers

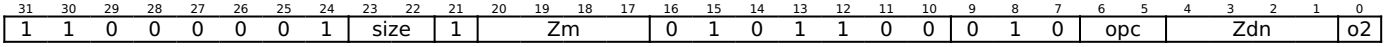
The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers).



Decode fields opc	U	Instruction Details	Condition
00	0	SMAX (multiple vectors)	FEAT_SME2
00	1	UMAX (multiple vectors)	FEAT_SME2
01	0	SMIN (multiple vectors)	FEAT_SME2
01	1	UMIN (multiple vectors)	FEAT_SME2
1x		UNALLOCATED	-

SME2 multiple vectors FP min/max two registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers).



Decode fields			Instruction Details	Condition
size	opc	o2		
	10	0	FAMAX	FEAT_SME2 && FEAT_FAMINMAX
	10	1	FAMIN	FEAT_SME2 && FEAT_FAMINMAX
	11		UNALLOCATED	-
00	00	0	BFMAX (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	00	1	BFMIN (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	01	0	BFMAXNM (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
00	01	1	BFMINNM (multiple vectors)	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	00	0	FMAX (multiple vectors)	FEAT_SME2
!= 00	00	1	FMIN (multiple vectors)	FEAT_SME2
!= 00	01	0	FMAXNM (multiple vectors)	FEAT_SME2
!= 00	01	1	FMINNM (multiple vectors)	FEAT_SME2

SME2 multiple vectors FSCALE two registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size		1	Zm				0	1	0	1	1	0	0	0	1	1	opc		Zdn					o2

Decode fields			Instruction Details	Condition
size	opc	o2		
	00	1	UNALLOCATED	-
	!= 00		UNALLOCATED	-
00	00	0	BFSCALE (multiple vectors)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	00	0	FSCALE (multiple vectors)	FEAT_SME2 && FEAT_FP8

SME2 multiple vectors shift two registers

The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Destructive (Two registers).

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	size		1	Zm				0	1	0	1	1	0	0	1	0	opc			Zdn					U

Decode fields		Instruction Details	Condition
opc	U		
001	0	SRSHL (multiple vectors)	FEAT_SME2
001	1	URSHL (multiple vectors)	FEAT_SME2
!= 001		UNALLOCATED	-

SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Two registers)

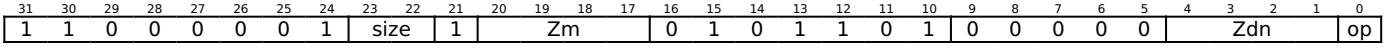
The encodings in this section are decoded from SME encodings.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1			1					0	1	0	1	1	0	1	op0									

Decode fields	Instruction details
op0	
00000	SME2 multi-vector signed saturating doubling multiply high two registers
!= 00000	UNALLOCATED

SME2 multi-vector signed saturating doubling multiply high two registers

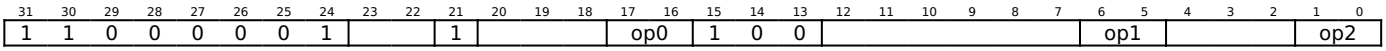
The encodings in this section are decoded from SME2 Multi-vector - Multiple Vectors SVE Saturating Multiply (Two registers).



Decode fields op	Instruction Details	Condition
0	SQDMULH (multiple vectors)	FEAT_SME2
1	UNALLOCATED	-

SME2 Multi-vector - SVE Select

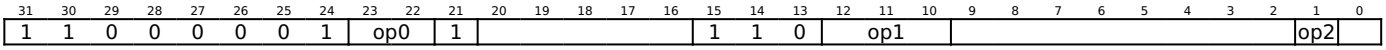
The encodings in this section are decoded from SME encodings.



Decode fields op0	Decode fields op1	Decode fields op2	Instruction details	Condition
01	00	00	SEL — Four registers	FEAT_SME2
01	00	10	UNALLOCATED	-
01	10	x0	UNALLOCATED	-
11	x0	x0	UNALLOCATED	-
x0	x0	x0	SEL — Two registers	FEAT_SME2
	x0	x1	UNALLOCATED	-
	x1		UNALLOCATED	-

SME2 Multi-vector - SVE Constructive Binary

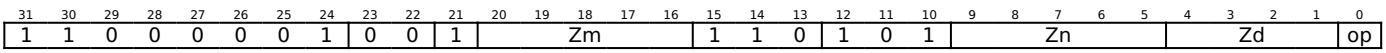
The encodings in this section are decoded from SME encodings.



Decode fields op0	Decode fields op1	Decode fields op2	Instruction details
00	101		SME2 multi-vec quadwords ZIP two registers
01	101		UNALLOCATED
10	101		UNALLOCATED
11	101		SME2 multi-vec saturating shift right narrow two registers
	000		SME2 multi-vec FCLAMP two registers
	001		SME2 multi-vec CLAMP two registers
	010	0	SME2 multi-vec FCLAMP four registers
	011	0	SME2 multi-vec CLAMP four registers
	01x	1	UNALLOCATED
	100		SME2 multi-vec ZIP two registers
	11x		SME2 multi-vec saturating shift right narrow four registers

SME2 multi-vec quadwords ZIP two registers

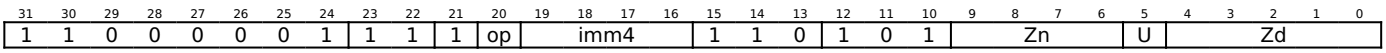
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields op	Instruction Details	Condition
0	ZIP (two registers)	FEAT_SME2
1	UZP (two registers)	FEAT_SME2

SME2 multi-vec saturating shift right narrow two registers

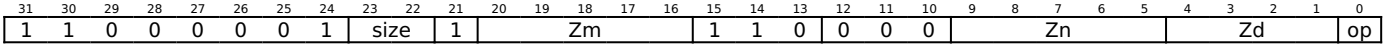
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields op	U	Instruction Details	Condition
0	0	SQRSHR (two registers)	FEAT_SME2
0	1	UQRSHR (two registers)	FEAT_SME2
1	0	SQRSHRU (two registers)	FEAT_SME2
1	1	UNALLOCATED	-

SME2 multi-vec FCLAMP two registers

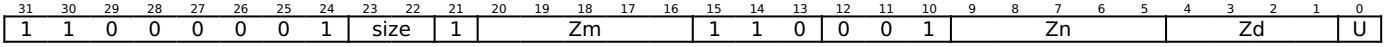
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields size	op	Instruction Details	Condition
	1	UNALLOCATED	-
00	0	BFCLAMP	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	FCLAMP	FEAT_SME2

SME2 multi-vec CLAMP two registers

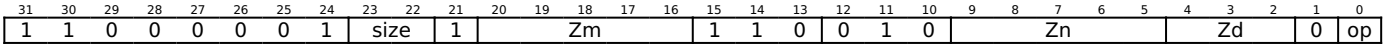
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields U	Instruction Details	Condition
0	SCLAMP	FEAT_SME2
1	UCLAMP	FEAT_SME2

SME2 multi-vec FCLAMP four registers

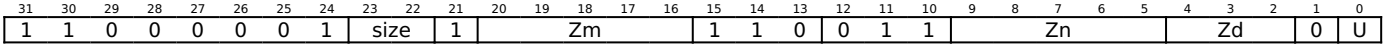
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields size	op	Instruction Details	Condition
	1	UNALLOCATED	-
00	0	BFCLAMP	FEAT_SME2 && FEAT_SVE_B16B16
!= 00	0	FCLAMP	FEAT_SME2

SME2 multi-vec CLAMP four registers

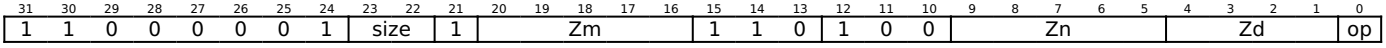
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields U	Instruction Details	Condition
0	SCLAMP	FEAT_SME2
1	UCLAMP	FEAT_SME2

SME2 multi-vec ZIP two registers

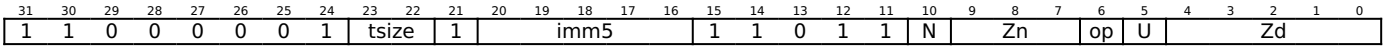
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields op	Instruction Details	Condition
0	ZIP (two registers)	FEAT_SME2
1	UZP (two registers)	FEAT_SME2

SME2 multi-vec saturating shift right narrow four registers

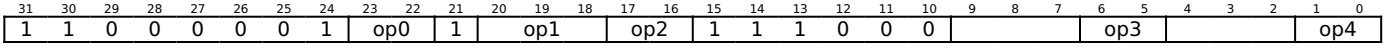
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Binary.



Decode fields N	Decode fields op	Decode fields U	Instruction Details	Condition
	1	1	UNALLOCATED	-
0	0	0	SQRSHR (four registers)	FEAT_SME2
0	0	1	UQRSHR (four registers)	FEAT_SME2
0	1	0	SQRSHRU (four registers)	FEAT_SME2
1	0	0	SQRSHRN	FEAT_SME2
1	0	1	UQRSHRN	FEAT_SME2
1	1	0	SQRSHRUN	FEAT_SME2

SME2 Multi-vector - SVE Constructive Unary

The encodings in this section are decoded from SME encodings.



Decode fields					Instruction details
op0	op1	op2	op3	op4	
00	000	01		x0	SME2 multi-vec FP to int convert two registers
00	000	10		x0	SME2 multi-vec int to FP two registers
00	001	11			UNALLOCATED
00	100	01	0x	00	SME2 multi-vec FP to int convert four registers
00	100	10	0x	00	SME2 multi-vec int to FP four registers
00	101	00	0x		SME2 multi-vec FP8 down convert four registers
00	101	11	00	x0	SME2 multi-vec quadwords ZIP four registers
01	000	01		x0	UNALLOCATED
01	001	11			UNALLOCATED
0x	000	00			SME2 multi-vec FP down convert two registers
0x	000	11			SME2 multi-vec int down convert two registers
0x	001	00	x0		SME2 multi-vec FP8 down convert two registers
0x	001	00	x1		UNALLOCATED
10	000	00			SME2 multi-vec convert two registers
10	001	00	1x		UNALLOCATED
10	x01	00	0x		UNALLOCATED
11	00x	00			UNALLOCATED
1x	000	11		x1	UNALLOCATED
1x	000	x1		x0	UNALLOCATED
1x	001	11			UNALLOCATED
x0	100	00	0x	0x	UNALLOCATED
	000	01		x1	UNALLOCATED
	000	10		x1	UNALLOCATED
	001	01			SME2 multi-vec unpack two registers
	001	10			SME2 multi-vec FP8 up convert two registers
	01x		x0	x0	SME2 multi-vec FRINT two registers
	100	01	0x	01	UNALLOCATED
	100	01	1x		UNALLOCATED
	100	10	0x	01	UNALLOCATED
	100	11			SME2 multi-vec int down convert four registers
	100	!= 11	0x	1x	UNALLOCATED
	101	01	x0	0x	SME2 multi-vec unpack four registers
	101	01	x0	1x	UNALLOCATED
	101	01	x1		UNALLOCATED
	101	10	00	x0	SME2 multi-vec ZIP four registers
	101	11	1x		UNALLOCATED
	101	1x	00	x1	UNALLOCATED
	101	1x	01		UNALLOCATED
	10x	x0	1x		UNALLOCATED
	11x		00	00	SME2 multi-vec FRINT four registers
	11x		00	10	UNALLOCATED
	11x		10	x0	UNALLOCATED
	x1x		x0	x1	UNALLOCATED
	x1x		x1		UNALLOCATED
!= x0	101	00	0x	1x	UNALLOCATED

!= x0	10x	00	0x	0x	UNALLOCATED
!= 00	000	10		x0	UNALLOCATED
!= 00	100	01	0x	00	UNALLOCATED
!= 00	100	10	0x	00	UNALLOCATED
!= 00	101	11	00	x0	UNALLOCATED

SME2 multi-vec FP to int convert two registers

The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	0	0	0	0	1	1	1	1	0	0	0	Zn				U	Zd				0

Decode fields U	Instruction Details	Condition
0	FCVTZS	FEAT_SME2
1	FCVTZU	FEAT_SME2

SME2 multi-vec int to FP two registers

The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	0	0	0	1	0	1	1	1	0	0	0	Zn				U	Zd				0

Decode fields U	Instruction Details	Condition
0	SCVTF	FEAT_SME2
1	UCVTF	FEAT_SME2

SME2 multi-vec FP to int convert four registers

The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	0	0	0	1	1	1	1	0	0	0	Zn			0	U	Zd			0	0

Decode fields U	Instruction Details	Condition
0	FCVTZS	FEAT_SME2
1	FCVTZU	FEAT_SME2

SME2 multi-vec int to FP four registers

The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	0	0	1	0	1	1	1	0	0	0	Zn			0	U	Zd			0	0

Decode fields U	Instruction Details	Condition
0	SCVTF	FEAT_SME2
1	UCVTF	FEAT_SME2

SME2 multi-vec FP8 down convert four registers

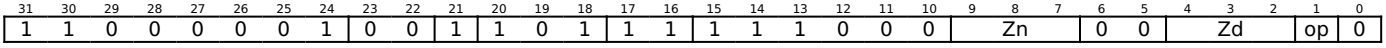
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	1	0	0	1	1	0	1	0	0	1	1	1	0	0	0	Zn			0	N	Zd				

Decode fields N	Instruction Details	Condition
0	FCVT (narrowing, FP32 to FP8)	FEAT_SME2 && FEAT_FP8
1	FCVTN (FP32 to FP8)	FEAT_SME2 && FEAT_FP8

SME2 multi-vec quadwords ZIP four registers

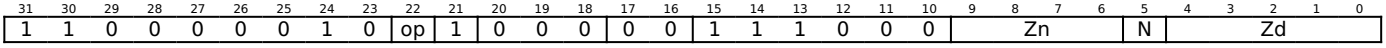
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields op	Instruction Details	Condition
0	ZIP (four registers)	FEAT_SME2
1	UZP (four registers)	FEAT_SME2

SME2 multi-vec FP down convert two registers

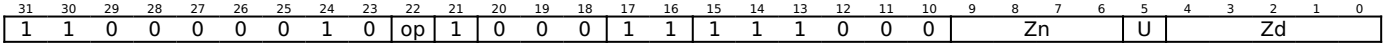
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields op	N	Instruction Details	Condition
0	0	FCVT (narrowing, FP32 to FP16)	FEAT_SME2
0	1	FCVTN (FP32 to FP16)	FEAT_SME2
1	0	BFCVT (single-precision to BFloat16)	FEAT_SME2
1	1	BFCVTN	FEAT_SME2

SME2 multi-vec int down convert two registers

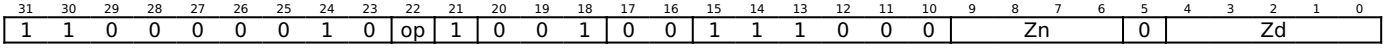
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields op	U	Instruction Details	Condition
0	0	SQCVT (two registers)	FEAT_SME2
0	1	UQCVT (two registers)	FEAT_SME2
1	0	SQCVTU (two registers)	FEAT_SME2
1	1	UNALLOCATED	-

SME2 multi-vec FP8 down convert two registers

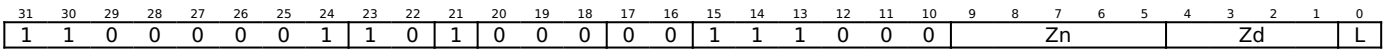
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields op	Instruction Details	Condition
0	FCVT (narrowing, FP16 to FP8)	FEAT_SME2 && FEAT_FP8
1	BFCVT (BFloat16 to 8-bit floating-point)	FEAT_SME2 && FEAT_FP8

SME2 multi-vec convert two registers

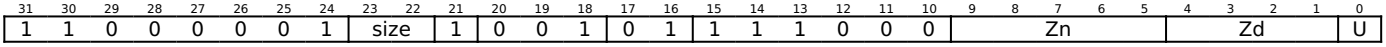
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields L	Instruction Details	Condition
0	FCVT (widening)	FEAT_SME_F16F16
1	FCVTL	FEAT_SME_F16F16

SME2 multi-vec unpack two registers

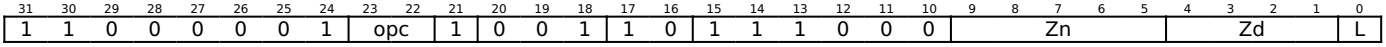
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields U	Instruction Details	Condition
0	SUNPK	FEAT_SME2
1	UUNPK	FEAT_SME2

SME2 multi-vec FP8 up convert two registers

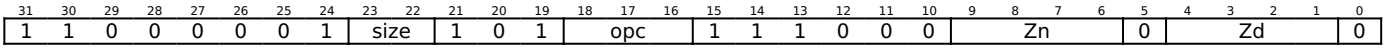
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields opc	L	Instruction Details	Condition
00	0	F1CVT, F2CVT — F1CVT	FEAT_SME2 && FEAT_FP8
00	1	F1CVTL, F2CVTL — F1CVTL	FEAT_SME2 && FEAT_FP8
01	0	BF1CVT, BF2CVT — BF1CVT	FEAT_SME2 && FEAT_FP8
01	1	BF1CVTL, BF2CVTL — BF1CVTL	FEAT_SME2 && FEAT_FP8
10	0	F1CVT, F2CVT — F2CVT	FEAT_SME2 && FEAT_FP8
10	1	F1CVTL, F2CVTL — F2CVTL	FEAT_SME2 && FEAT_FP8
11	0	BF1CVT, BF2CVT — BF2CVT	FEAT_SME2 && FEAT_FP8
11	1	BF1CVTL, BF2CVTL — BF2CVTL	FEAT_SME2 && FEAT_FP8

SME2 multi-vec FRINT two registers

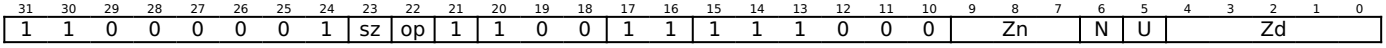
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields size	opc	Instruction Details	Condition
10	000	FRINTN	FEAT_SME2
10	001	FRINTP	FEAT_SME2
10	010	FRINTM	FEAT_SME2
10	011	UNALLOCATED	-
10	100	FRINTA	FEAT_SME2
10	101	UNALLOCATED	-
10	11x	UNALLOCATED	-
!= 10		UNALLOCATED	-

SME2 multi-vec int down convert four registers

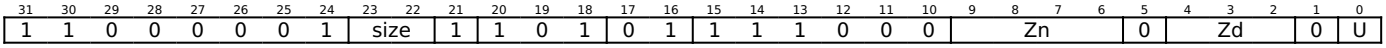
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields			Instruction Details	Condition
op	N	U		
0	0	0	SQCVT (four registers)	FEAT_SME2
0	0	1	UQCVT (four registers)	FEAT_SME2
0	1	0	SQCVTN	FEAT_SME2
0	1	1	UQCVTN	FEAT_SME2
1		1	UNALLOCATED	-
1	0	0	SQCVTU (four registers)	FEAT_SME2
1	1	0	SQCVTUN	FEAT_SME2

SME2 multi-vec unpack four registers

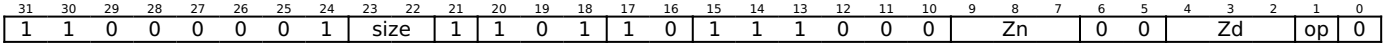
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields		Instruction Details	Condition
U			
0		SUNPK	FEAT_SME2
1		UUNPK	FEAT_SME2

SME2 multi-vec ZIP four registers

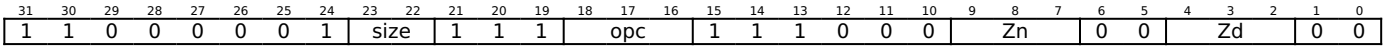
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields		Instruction Details	Condition
op			
0		ZIP (four registers)	FEAT_SME2
1		UZP (four registers)	FEAT_SME2

SME2 multi-vec FRINT four registers

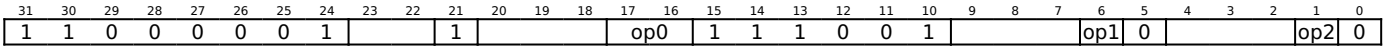
The encodings in this section are decoded from SME2 Multi-vector - SVE Constructive Unary.



Decode fields		Instruction Details	Condition
size	opc		
10	000	FRINTN	FEAT_SME2
10	001	FRINTP	FEAT_SME2
10	010	FRINTM	FEAT_SME2
10	011	UNALLOCATED	-
10	100	FRINTA	FEAT_SME2
10	101	UNALLOCATED	-
10	11x	UNALLOCATED	-
!= 10		UNALLOCATED	-

SME2 Multi-vector - FP Multiply

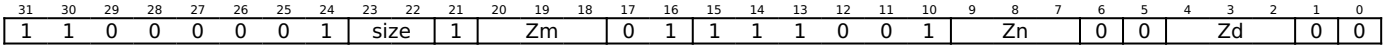
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	Instruction details
01	0	0	SME2 multi-vec FP multiply (four registers)
01	0	1	UNALLOCATED
01	1		UNALLOCATED
11			UNALLOCATED
x0			SME2 multi-vec FP multiply (two registers)

SME2 multi-vec FP multiply (four registers)

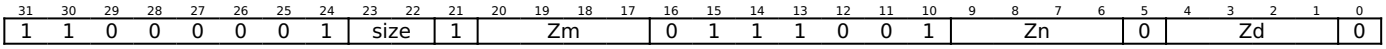
The encodings in this section are decoded from SME2 Multi-vector - FP Multiply.



Decode fields size	Instruction Details	Condition
00	BFMUL (multiple vectors)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	FMUL (multiple vectors)	FEAT_SME2p2

SME2 multi-vec FP multiply (two registers)

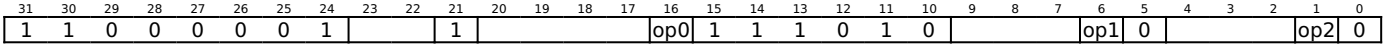
The encodings in this section are decoded from SME2 Multi-vector - FP Multiply.



Decode fields size	Instruction Details	Condition
00	BFMUL (multiple vectors)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	FMUL (multiple vectors)	FEAT_SME2p2

SME2 Multiple and Single Vector - FP multiply

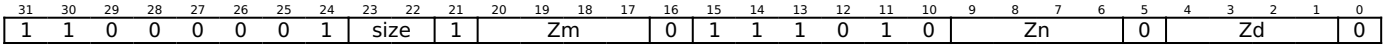
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	Instruction details
0			SME2 multiple and single vector FP multiply (two registers)
1	0	0	SME2 multiple and single vector FP multiply (four registers)
1	0	1	UNALLOCATED
1	1		UNALLOCATED

SME2 multiple and single vector FP multiply (two registers)

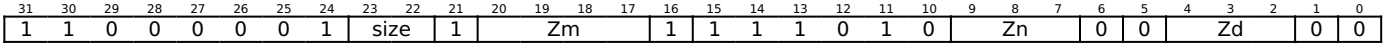
The encodings in this section are decoded from SME2 Multiple and Single Vector - FP multiply.



Decode fields size	Instruction Details	Condition
00	BFMUL (multiple and single vector)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	FMUL (multiple and single vector)	FEAT_SME2p2

SME2 multiple and single vector FP multiply (four registers)

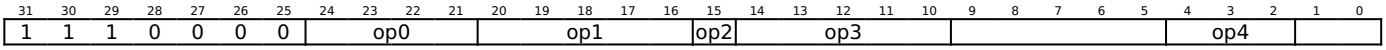
The encodings in this section are decoded from SME2 Multiple and Single Vector - FP multiply.



Decode fields size	Instruction Details	Condition
00	BFMUL (multiple and single vector)	FEAT_SME2 && FEAT_SVE_BFSCALE
!= 00	FMUL (multiple and single vector)	FEAT_SME2p2

SME Memory

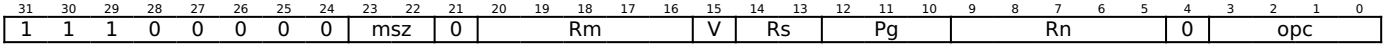
The encodings in this section are decoded from SME encodings.



Decode fields op0	op1	op2	op3	op4	Instruction details	Condition
0xx0				0xx	SME load array vector (elements)	-
0xx1				0xx	SME store array vector (elements)	-
100x	00000	0	xx000	0xx	SME save and restore array	-
100x		1	00000	000	SME2 lookup table load/store	-
100x		1	00000	001	UNALLOCATED	-
100x		1	00000	01x	UNALLOCATED	-
100x		1	01000	0xx	UNALLOCATED	-
100x		1	1x000	0xx	UNALLOCATED	-
100x			!= xx000	0xx	UNALLOCATED	-
100x	!= 00000	0	xx000	0xx	UNALLOCATED	-
101x				0xx	UNALLOCATED	-
110x				0xx	UNALLOCATED	-
1110				0xx	LD1Q	FEAT_SME
1111				0xx	ST1Q	FEAT_SME
				1xx	UNALLOCATED	-

SME load array vector (elements)

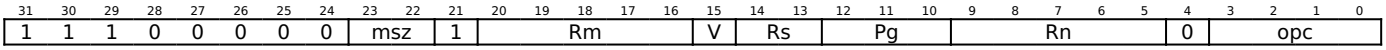
The encodings in this section are decoded from SME Memory.



Decode fields msz	Instruction Details	Condition
00	LD1B (scalar plus scalar, tile slice)	FEAT_SME
01	LD1H (scalar plus scalar, tile slice)	FEAT_SME
10	LD1W (scalar plus scalar, tile slice)	FEAT_SME
11	LD1D (scalar plus scalar, tile slice)	FEAT_SME

SME store array vector (elements)

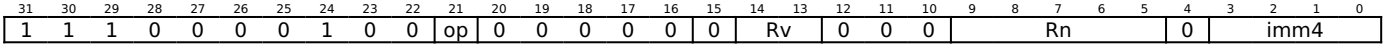
The encodings in this section are decoded from SME Memory.



Decode fields msz	Instruction Details	Condition
00	ST1B (scalar plus scalar, tile slice)	FEAT_SME
01	ST1H (scalar plus scalar, tile slice)	FEAT_SME
10	ST1W (scalar plus scalar, tile slice)	FEAT_SME
11	ST1D (scalar plus scalar, tile slice)	FEAT_SME

SME save and restore array

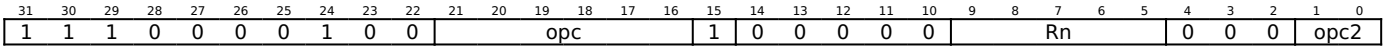
The encodings in this section are decoded from SME Memory.



Decode fields op	Instruction Details	Condition
0	LDR (array vector)	FEAT_SME
1	STR (array vector)	FEAT_SME

SME2 lookup table load/store

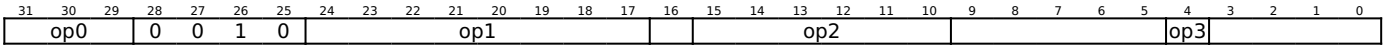
The encodings in this section are decoded from SME Memory.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
x11111	!= 00	UNALLOCATED	-
!= x11111		UNALLOCATED	-
011111	00	LDR (table)	FEAT_SME2
111111	00	STR (table)	FEAT_SME2

SVE encodings

The encodings in this section are decoded from A64 instruction set encoding.



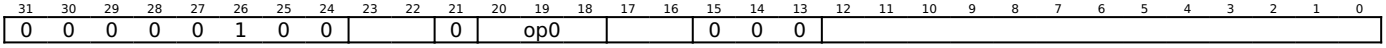
Decode fields				Instruction details	Condition
op0	op1	op2	op3		
000	0xx0xxxx	000xxx		SVE Integer Binary Arithmetic - Predicated	-
000	0xx0xxxx	001xxx		SVE Integer Reduction	-
000	0xx0xxxx	100xxx		SVE Bitwise Shift - Predicated	-
000	0xx0xxxx	101xxx		SVE Integer Unary Arithmetic - Predicated	-
000	0xx0xxxx	x1xxxx		SVE Integer Multiply-Add - Predicated	-
000	0xx1xxxx	001xxx		SVE Bitwise Logical - Unpredicated	-
000	0xx1xxxx	0100xx		SVE Index Generation	-
000	0xx1xxxx	0101xx		SVE Stack Allocation	-
000	0xx1xxxx	011xxx		SVE2 Integer Arithmetic - Unpredicated	-
000	0xx1xxxx	100xxx		SVE Bitwise Shift - Unpredicated	-
000	0xx1xxxx	1011xx		SVE Integer Misc - Unpredicated	-
000	0xx1xxxx	11xxxx		SVE Element Count	-
000	10x1xxxx	000xxx		SVE Permute Vector - Extract	-
000	11x1xxxx	000xxx		SVE Permute Vector - Segments	-
000	1xx00xxx			SVE Bitwise Immediate	-
000	1xx01xxx			SVE Integer Wide Immediate - Predicated	-
000	1xx1xxxx	001001		SVE Permute Vector - One Source Quadwords	-
000	1xx1xxxx	001110		SVE Permute Vector - Unpredicated	-
000	1xx1xxxx	001111		UNALLOCATED	-
000	1xx1xxxx	010xxx		SVE Permute Predicate	-
000	1xx1xxxx	10xxxx		SVE Permute Vector - Predicated	-
001	0xx0xxxx			SVE Integer Compare - Vectors	-
001	1xx00xxx	11xxxx		SVE Propagate Break	-
001	1xx01xxx	01xxxx		SVE Partition Break	-
001	1xx01xxx	11xxxx		SVE Predicate Misc	-
001	1xx100xx	10xxxx		SVE Predicate Count	-
001	1xx101xx	1000xx		SVE Inc/Dec by Predicate Count	-
001	1xx101xx	1001xx		SVE Write FFR	-
001	1xx101xx	101xxx		UNALLOCATED	-
001	1xx11xxx	10xxxx		UNALLOCATED	-
001	1xx1xxxx	00xxxx		SVE Integer Compare - Scalars	-
001	1xx1xxxx	01xxxx	1	SVE Scalar Integer Compare - Predicate-as-counter	-
001	1xx1xxxx	11xxxx		SVE Integer Wide Immediate - Unpredicated	-
010	0x10xxxx	11001x		UNALLOCATED	-
010	0xx0xxxx	0xxxxx		SVE Integer Multiply-Add - Unpredicated	-
010	0xx0xxxx	10xxxx		SVE2 Integer - Predicated	-
010	0xx1xxxx			SVE Multiply - Indexed	-
010	1xx0xxxx	0xxxxx		SVE2 Widening Integer Arithmetic	-
010	1xx0xxxx	10xxxx		SVE Misc	-
010	1xx0xxxx	11xxxx		SVE2 Accumulate	-
010	1xx1xxxx	0xxxxx		SVE2 Narrowing	-
010	1xx1xxxx	101xxx		SVE2 Histogram (Segment) and Lookup Table	-
010	1xx1xxxx	111xxx		SVE2 Crypto Extensions	-
011	0x01xxxx	0111xx		UNALLOCATED	-
011	0x01xxxx	10xx10		SVE2 FP8 widening multiply-add	-

011	0x01xxxx	10xx11		UNALLOCATED	-
011	0x01xxxx	11x1xx		UNALLOCATED	-
011	0x11xxxx	10xx1x		UNALLOCATED	-
011	0x11xxxx	x1x1xx		UNALLOCATED	-
011	0xx00001	100xxx		UNALLOCATED	-
011	0xx0010x	100xxx		UNALLOCATED	-
011	0xx00x0x	11xxxx		UNALLOCATED	-
011	0xx00x1x	1xxxxx		UNALLOCATED	-
011	0xx010xx	11xxxx		UNALLOCATED	-
011	0xx011xx	1xxxxx		SVE2 floating-point unary operations - zeroing predicated	-
011	0xx1xxxx	001011		UNALLOCATED	-
011	0xx1xxxx	0011xx		UNALLOCATED	-
011	0xx1xxxx	01x0xx		SVE floating-point widening multiply-add - indexed	-
011	0xx1xxxx	10x00x		SVE floating-point widening multiply-add	-
011	0xx1xxxx	10x10x		UNALLOCATED	-
011	0xx1xxxx	11101x		UNALLOCATED	-
011	1xx001xx	0010xx		UNALLOCATED	-
011	1xx001xx	0011xx		SVE floating-point unary operations - unpredicated	-
011	1xx010xx	001xxx		SVE floating-point compare - with zero	-
011	1xx011xx	001xxx		SVE floating-point accumulating reduction	-
011	1xx0xxxx	100xxx		SVE floating-point arithmetic - predicated	-
011	1xx0xxxx	101xxx		SVE floating-point unary operations - merging predicated	-
011	1xx1xxxx			SVE floating-point multiply-add	-
100				SVE Memory - 32-bit Gather and Unsized Contiguous	-
101				SVE Memory - Contiguous Load	-
110				SVE Memory - 64-bit Gather	-
111		001xxx		SVE Memory - Non-temporal and Quadword Scatter Store	-
111		011xxx		SVE Memory - Non-temporal and Multi-register Contiguous Store	-
111		0x0xxx		SVE Memory - Contiguous Store and Unsized Contiguous	-
111		101xxx		SVE Memory - Scatter	-
111		111xxx		SVE Memory - Contiguous Store with Immediate Offset	-
111		1x0xxx		SVE Memory - Scatter with Optional Sign Extend	-
000	0xx1xxxx	000xxx		SVE integer add/subtract vectors (unpredicated)	-
000	0xx1xxxx	1010xx		SVE address generation	-
000	1xx1xxxx	001000		DUP (indexed)	FEAT_SVE || FEAT_SME
000	1xx1xxxx	00101x		SVE table lookup (three sources)	-
000	1xx1xxxx	001100		TBL — Single register table	FEAT_SVE || FEAT_SME
000	1xx1xxxx	001101		TBXQ	FEAT_SVE2p1 || FEAT_SME2p1
000	1xx1xxxx	011xxx		SVE permute vector elements	-
000	1xx1xxxx	11xxxx		SEL (vectors)	FEAT_SVE || FEAT_SME
001	0xx1xxxx			SVE integer compare with unsigned immediate	-
001	1xx00xxx	01xxxx		SVE predicate logical operations	-
001	1xx0xxxx	x0xxxx		SVE integer compare with signed immediate	-
001	1xx1xxxx	01xxxx	0	SVE broadcast predicate element	-
010	0000xxxx	11001x		SVE two-way dot product	-

010	0100xxxx	11001x		SVE two-way dot product (indexed)	-
010	0xx0xxxx	11000x		SVE integer clamp	-
010	0xx0xxxx	1101x0		SVE2 multiply-add (checked pointer)	-
010	0xx0xxxx	1101x1		SVE2 absolute difference and accumulate long	-
010	0xx0xxxx	111xxx		SVE permute vector elements (quadwords)	-
010	1xx1xxxx	100xxx		SVE2 character match	-
010	1xx1xxxx	110xxx		HISTCNT	FEAT_SVE2
011	00x1xxxx	111000		SVE2 FP8 matrix multiply-accumulate	-
011	01x1xxxx	111000		SVE2 floating-point non-widening matrix-multiply accumulate	-
011	0x01xxxx	0101xx		SVE2 FP8 multiply-add long (indexed)	-
011	0xx00000	100xxx		FCADD	FEAT_SVE || FEAT_SME
011	0xx0000x	101xxx		SVE floating-point convert (top, predicated)	-
011	0xx0010x	101xxx		SVE floating-point convert precision odd elements	-
011	0xx010xx	100xxx		SVE2 floating-point pairwise operations	-
011	0xx010xx	101xxx		SVE floating-point recursive reduction (quadwords)	-
011	0xx0xxxx	0xxxxx		FCMLA (vectors)	FEAT_SVE || FEAT_SME
011	0xx1xxxx	0000xx		SVE floating-point multiply-add (indexed)	-
011	0xx1xxxx	0001xx		SVE floating-point complex multiply-add (indexed)	-
011	0xx1xxxx	001001		SVE FP clamp	-
011	0xx1xxxx	0010x0		SVE floating-point multiply (indexed)	-
011	0xx1xxxx	1100xx		SVE2 FP8 multiply-add long long (indexed)	-
011	0xx1xxxx	111001		SVE floating-point matrix multiply accumulate	-
011	1xx000xx	001xxx		SVE floating-point recursive reduction	-
011	1xx0xxxx	000xxx		SVE floating-point arithmetic (unpredicated)	-
011	1xx0xxxx	x1xxxx		SVE floating-point compare vectors	-

SVE Integer Binary Arithmetic - Predicated

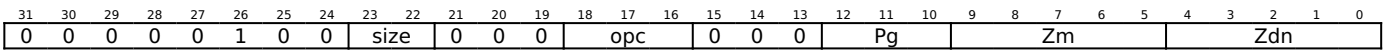
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00x	SVE integer add/subtract vectors (predicated)
01x	SVE integer min/max/difference (predicated)
100	SVE integer multiply vectors (predicated)
101	SVE integer divide vectors (predicated)
11x	SVE bitwise logical operations (predicated)

SVE integer add/subtract vectors (predicated)

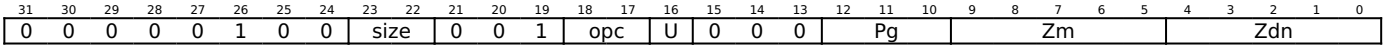
The encodings in this section are decoded from SVE Integer Binary Arithmetic - Predicated.



Decode fields		Instruction Details	Condition
size	opc		
	000	ADD (vectors, predicated)	FEAT_SVE || FEAT_SME
	001	SUB (vectors, predicated)	FEAT_SVE || FEAT_SME
	010	UNALLOCATED	-
	011	SUBR (vectors)	FEAT_SVE || FEAT_SME
11	100	ADDPT (predicated)	FEAT_SVE && FEAT_CPA
11	101	SUBPT (predicated)	FEAT_SVE && FEAT_CPA
11	11x	UNALLOCATED	-
!= 11	1xx	UNALLOCATED	-

SVE integer min/max/difference (predicated)

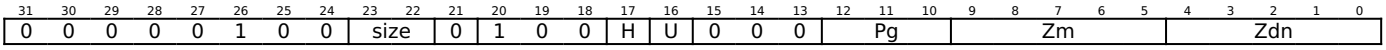
The encodings in this section are decoded from SVE Integer Binary Arithmetic - Predicated.



Decode fields		Instruction Details	Condition
opc	U		
00	0	SMAX (vectors)	FEAT_SVE || FEAT_SME
00	1	UMAX (vectors)	FEAT_SVE || FEAT_SME
01	0	SMIN (vectors)	FEAT_SVE || FEAT_SME
01	1	UMIN (vectors)	FEAT_SVE || FEAT_SME
10	0	SABD	FEAT_SVE || FEAT_SME
10	1	UABD	FEAT_SVE || FEAT_SME
11		UNALLOCATED	-

SVE integer multiply vectors (predicated)

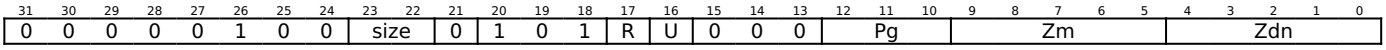
The encodings in this section are decoded from SVE Integer Binary Arithmetic - Predicated.



Decode fields		Instruction Details	Condition
H	U		
0	0	MUL (vectors, predicated)	FEAT_SVE || FEAT_SME
0	1	UNALLOCATED	-
1	0	SMULH (predicated)	FEAT_SVE || FEAT_SME
1	1	UMULH (predicated)	FEAT_SVE || FEAT_SME

SVE integer divide vectors (predicated)

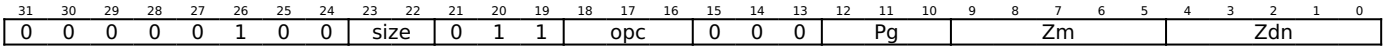
The encodings in this section are decoded from SVE Integer Binary Arithmetic - Predicated.



Decode fields		Instruction Details	Condition
R	U		
0	0	SDIV	FEAT_SVE || FEAT_SME
0	1	UDIV	FEAT_SVE || FEAT_SME
1	0	SDIVR	FEAT_SVE || FEAT_SME
1	1	UDIVR	FEAT_SVE || FEAT_SME

SVE bitwise logical operations (predicated)

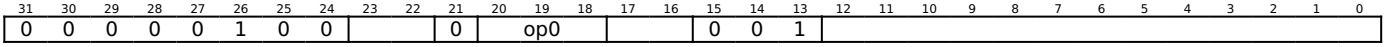
The encodings in this section are decoded from SVE Integer Binary Arithmetic - Predicated.



Decode fields opc	Instruction Details	Condition
000	ORR (vectors, predicated)	FEAT_SVE || FEAT_SME
001	EOR (vectors, predicated)	FEAT_SVE || FEAT_SME
010	AND (vectors, predicated)	FEAT_SVE || FEAT_SME
011	BIC (vectors, predicated)	FEAT_SVE || FEAT_SME
1xx	UNALLOCATED	-

SVE Integer Reduction

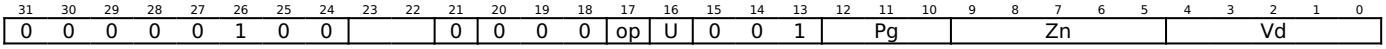
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
000	SVE integer add reduction (predicated)
001	SVE integer add reduction (quadwords)
010	SVE integer min/max reduction (predicated)
011	SVE integer min/max reduction (quadwords)
10x	SVE constructive prefix (predicated)
110	SVE bitwise logical reduction (predicated)
111	SVE bitwise logical reduction (quadwords)

SVE integer add reduction (predicated)

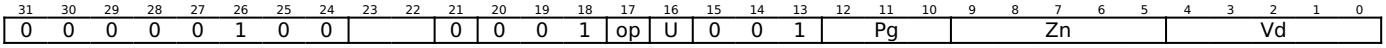
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields op	U	Instruction Details	Condition
0	0	SADDV	FEAT_SVE || FEAT_SME
0	1	UADDV	FEAT_SVE || FEAT_SME
1		UNALLOCATED	-

SVE integer add reduction (quadwords)

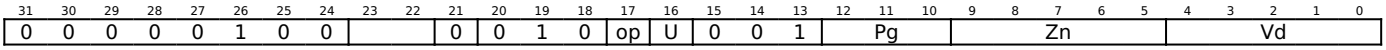
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields op	U	Instruction Details	Condition
0	0	UNALLOCATED	-
0	1	ADDQV	FEAT_SVE2p1 || FEAT_SME2p1
1		UNALLOCATED	-

SVE integer min/max reduction (predicated)

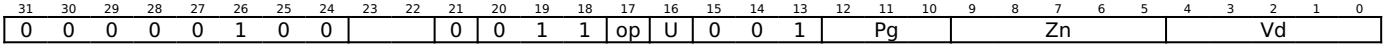
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields		Instruction Details	Condition
op	U		
0	0	SMAXV	FEAT_SVE || FEAT_SME
0	1	UMAXV	FEAT_SVE || FEAT_SME
1	0	SMINV	FEAT_SVE || FEAT_SME
1	1	UMINV	FEAT_SVE || FEAT_SME

SVE integer min/max reduction (quadwords)

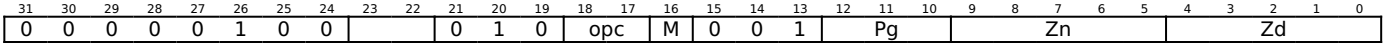
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields		Instruction Details	Condition
op	U		
0	0	SMAXQV	FEAT_SVE2p1 || FEAT_SME2p1
0	1	UMAXQV	FEAT_SVE2p1 || FEAT_SME2p1
1	0	SMINQV	FEAT_SVE2p1 || FEAT_SME2p1
1	1	UMINQV	FEAT_SVE2p1 || FEAT_SME2p1

SVE constructive prefix (predicated)

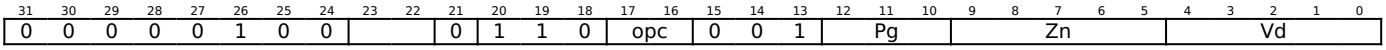
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields	Instruction Details	Condition
opc		
00	MOVPRFX (predicated)	FEAT_SVE || FEAT_SME
!= 00	UNALLOCATED	-

SVE bitwise logical reduction (predicated)

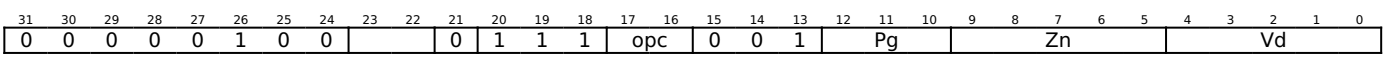
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields	Instruction Details	Condition
opc		
00	ORV	FEAT_SVE || FEAT_SME
01	EORV	FEAT_SVE || FEAT_SME
10	ANDV	FEAT_SVE || FEAT_SME
11	UNALLOCATED	-

SVE bitwise logical reduction (quadwords)

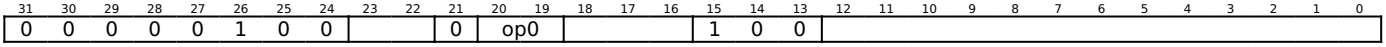
The encodings in this section are decoded from SVE Integer Reduction.



Decode fields opc	Instruction Details	Condition
00	ORQV	FEAT_SVE2p1 || FEAT_SME2p1
01	EORQV	FEAT_SVE2p1 || FEAT_SME2p1
10	ANDQV	FEAT_SVE2p1 || FEAT_SME2p1
11	UNALLOCATED	-

SVE Bitwise Shift - Predicated

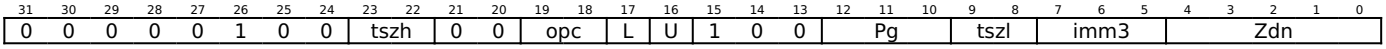
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0x	SVE bitwise shift by immediate (predicated)
10	SVE bitwise shift by vector (predicated)
11	SVE bitwise shift by wide elements (predicated)

SVE bitwise shift by immediate (predicated)

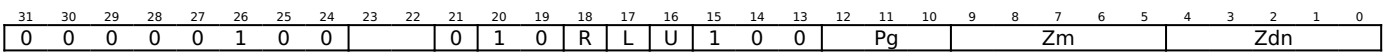
The encodings in this section are decoded from SVE Bitwise Shift - Predicated.



Decode fields opc	L	U	Instruction Details	Condition
00	0	0	ASR (immediate, predicated)	FEAT_SVE || FEAT_SME
00	0	1	LSR (immediate, predicated)	FEAT_SVE || FEAT_SME
00	1	0	UNALLOCATED	-
00	1	1	LSL (immediate, predicated)	FEAT_SVE || FEAT_SME
01	0	0	ASRD	FEAT_SVE || FEAT_SME
01	0	1	UNALLOCATED	-
01	1	0	SQSHL (immediate)	FEAT_SVE2 || FEAT_SME
01	1	1	UQSHL (immediate)	FEAT_SVE2 || FEAT_SME
10			UNALLOCATED	-
11	0	0	SRSHR	FEAT_SVE2 || FEAT_SME
11	0	1	URSHR	FEAT_SVE2 || FEAT_SME
11	1	0	UNALLOCATED	-
11	1	1	SQSHLU	FEAT_SVE2 || FEAT_SME

SVE bitwise shift by vector (predicated)

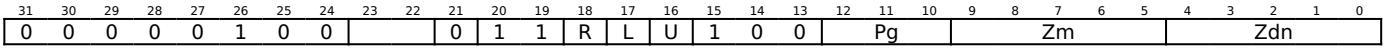
The encodings in this section are decoded from SVE Bitwise Shift - Predicated.



Decode fields R	L	U	Instruction Details	Condition
	1	0	UNALLOCATED	-
0	0	0	ASR (vectors)	FEAT_SVE || FEAT_SME
0	0	1	LSR (vectors)	FEAT_SVE || FEAT_SME
0	1	1	LSL (vectors)	FEAT_SVE || FEAT_SME
1	0	0	ASRR	FEAT_SVE || FEAT_SME
1	0	1	LSRR	FEAT_SVE || FEAT_SME
1	1	1	LSLR	FEAT_SVE || FEAT_SME

SVE bitwise shift by wide elements (predicated)

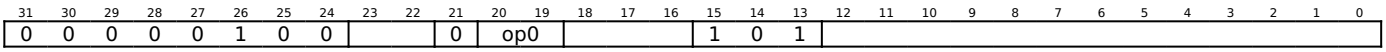
The encodings in this section are decoded from SVE Bitwise Shift - Predicated.



Decode fields R	L	U	Instruction Details	Condition
0	0	0	ASR (wide elements, predicated)	FEAT_SVE || FEAT_SME
0	0	1	LSR (wide elements, predicated)	FEAT_SVE || FEAT_SME
0	1	0	UNALLOCATED	-
0	1	1	LSL (wide elements, predicated)	FEAT_SVE || FEAT_SME
1			UNALLOCATED	-

SVE Integer Unary Arithmetic - Predicated

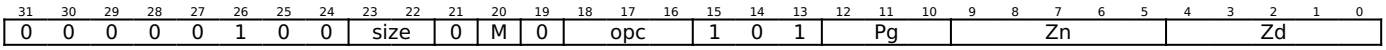
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
x0	SVE integer unary operations (predicated)
x1	SVE bitwise unary operations (predicated)

SVE integer unary operations (predicated)

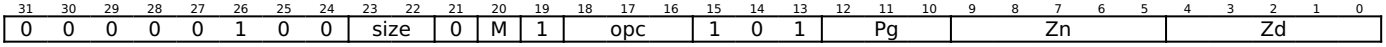
The encodings in this section are decoded from SVE Integer Unary Arithmetic - Predicated.



Decode fields M	opc	Instruction Details	Condition
0	000	SXTB, SXTH, SXTW — Byte, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	001	UXTB, UXTH, UXTW — Byte, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	010	SXTB, SXTH, SXTW — Halfword, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	011	UXTB, UXTH, UXTW — Halfword, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	100	SXTB, SXTH, SXTW — Word, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	101	UXTB, UXTH, UXTW — Word, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	110	ABS — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	111	NEG — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	000	SXTB, SXTH, SXTW — Byte, merging	FEAT_SVE || FEAT_SME
1	001	UXTB, UXTH, UXTW — Byte, merging	FEAT_SVE || FEAT_SME
1	010	SXTB, SXTH, SXTW — Halfword, merging	FEAT_SVE || FEAT_SME
1	011	UXTB, UXTH, UXTW — Halfword, merging	FEAT_SVE || FEAT_SME
1	100	SXTB, SXTH, SXTW — Word, merging	FEAT_SVE || FEAT_SME
1	101	UXTB, UXTH, UXTW — Word, merging	FEAT_SVE || FEAT_SME
1	110	ABS — Merging	FEAT_SVE || FEAT_SME
1	111	NEG — Merging	FEAT_SVE || FEAT_SME

SVE bitwise unary operations (predicated)

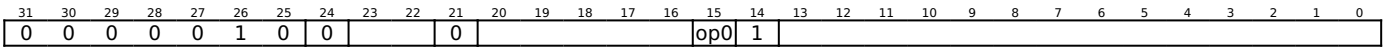
The encodings in this section are decoded from SVE Integer Unary Arithmetic - Predicated.



Decode fields M	opc	Instruction Details	Condition
	111	UNALLOCATED	-
0	000	CLS — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	001	CLZ — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	010	CNT — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	011	CNOT — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	100	FABS — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	101	FNEG — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	110	NOT (vector) — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	000	CLS — Merging	FEAT_SVE || FEAT_SME
1	001	CLZ — Merging	FEAT_SVE || FEAT_SME
1	010	CNT — Merging	FEAT_SVE || FEAT_SME
1	011	CNOT — Merging	FEAT_SVE || FEAT_SME
1	100	FABS — Merging	FEAT_SVE || FEAT_SME
1	101	FNEG — Merging	FEAT_SVE || FEAT_SME
1	110	NOT (vector) — Merging	FEAT_SVE || FEAT_SME

SVE Integer Multiply-Add - Predicated

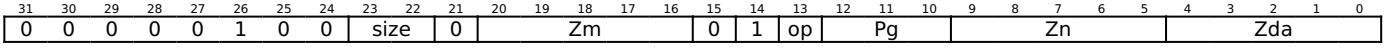
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0	SVE integer multiply-accumulate writing addend (predicated)
1	SVE integer multiply-add writing multiplicand (predicated)

SVE integer multiply-accumulate writing addend (predicated)

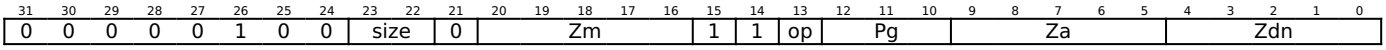
The encodings in this section are decoded from SVE Integer Multiply-Add - Predicated.



Decode fields op	Instruction Details	Condition
0	MLA (vectors)	FEAT_SVE || FEAT_SME
1	MLS (vectors)	FEAT_SVE || FEAT_SME

SVE integer multiply-add writing multiplicand (predicated)

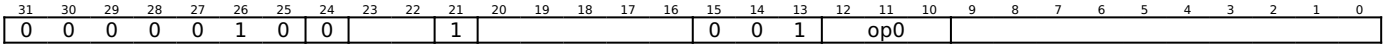
The encodings in this section are decoded from SVE Integer Multiply-Add - Predicated.



Decode fields op	Instruction Details	Condition
0	MAD	FEAT_SVE || FEAT_SME
1	MSB	FEAT_SVE || FEAT_SME

SVE Bitwise Logical - Unpredicated

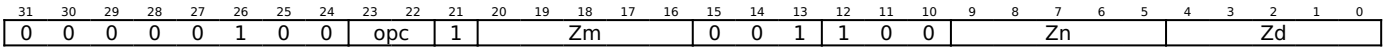
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details	Condition
0xx	UNALLOCATED	-
100	SVE bitwise logical operations (unpredicated)	-
101	XAR	FEAT_SVE2 || FEAT_SME
11x	SVE2 bitwise ternary operations	-

SVE bitwise logical operations (unpredicated)

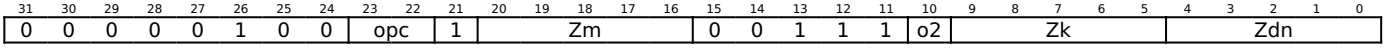
The encodings in this section are decoded from SVE Bitwise Logical - Unpredicated.



Decode fields opc	Instruction Details	Condition
00	AND (vectors, unpredicated)	FEAT_SVE || FEAT_SME
01	ORR (vectors, unpredicated)	FEAT_SVE || FEAT_SME
10	EOR (vectors, unpredicated)	FEAT_SVE || FEAT_SME
11	BIC (vectors, unpredicated)	FEAT_SVE || FEAT_SME

SVE2 bitwise ternary operations

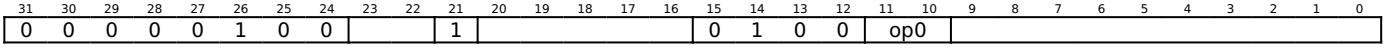
The encodings in this section are decoded from SVE Bitwise Logical - Unpredicated.



Decode fields opc	o2	Instruction Details	Condition
00	0	EOR3	FEAT_SVE2 || FEAT_SME
00	1	BSL	FEAT_SVE2 || FEAT_SME
01	0	BCAX	FEAT_SVE2 || FEAT_SME
01	1	BSL1N	FEAT_SVE2 || FEAT_SME
1x	0	UNALLOCATED	-
10	1	BSL2N	FEAT_SVE2 || FEAT_SME
11	1	NBSL	FEAT_SVE2 || FEAT_SME

SVE Index Generation

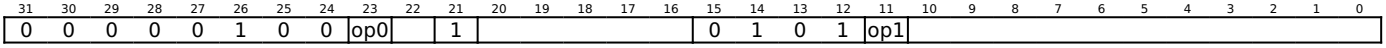
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details	Condition
00	INDEX (immediates)	FEAT_SVE || FEAT_SME
01	INDEX (scalar, immediate)	FEAT_SVE || FEAT_SME
10	INDEX (immediate, scalar)	FEAT_SVE || FEAT_SME
11	INDEX (scalars)	FEAT_SVE || FEAT_SME

SVE Stack Allocation

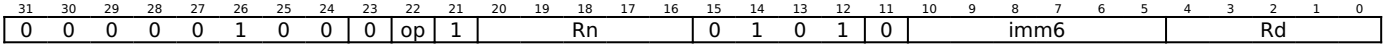
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
0	0	SVE stack frame adjustment
0	1	Streaming SVE stack frame adjustment
1	0	SVE stack frame size
1	1	Streaming SVE stack frame size

SVE stack frame adjustment

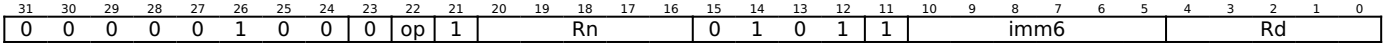
The encodings in this section are decoded from SVE Stack Allocation.



Decode fields op	Instruction Details	Condition
0	ADDVL	FEAT_SVE || FEAT_SME
1	ADDPL	FEAT_SVE || FEAT_SME

Streaming SVE stack frame adjustment

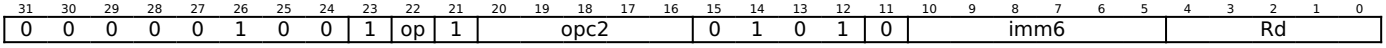
The encodings in this section are decoded from SVE Stack Allocation.



Decode fields op	Instruction Details	Condition
0	ADDSVL	FEAT_SME
1	ADDSPL	FEAT_SME

SVE stack frame size

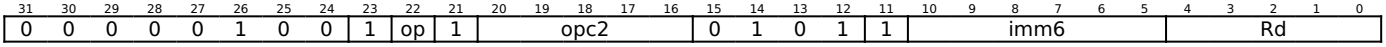
The encodings in this section are decoded from SVE Stack Allocation.



Decode fields op	Decode fields opc2	Instruction Details	Condition
0	11111	RDVL	FEAT_SVE || FEAT_SME
0	!= 11111	UNALLOCATED	-
1		UNALLOCATED	-

Streaming SVE stack frame size

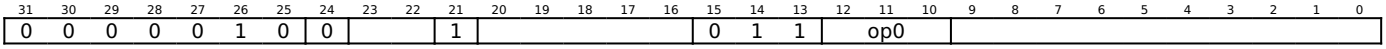
The encodings in this section are decoded from SVE Stack Allocation.



Decode fields op	Decode fields opc2	Instruction Details	Condition
0	11111	RDSVL	FEAT_SME
0	!= 11111	UNALLOCATED	-
1		UNALLOCATED	-

SVE2 Integer Arithmetic - Unpredicated

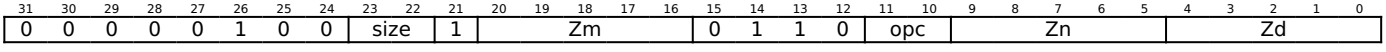
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details	Condition
0xx	SVE2 integer multiply vectors (unpredicated)	-
10x	SVE2 signed saturating doubling multiply high (unpredicated)	-
110	ADDQP	FEAT_SVE2p3 || FEAT_SME2p3
111	ADDSUBP	FEAT_SVE2p3 || FEAT_SME2p3

SVE2 integer multiply vectors (unpredicated)

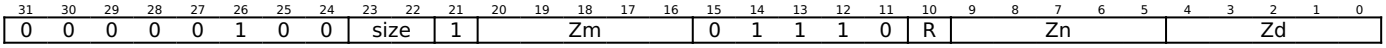
The encodings in this section are decoded from SVE2 Integer Arithmetic - Unpredicated.



Decode fields size	opc	Instruction Details	Condition
	00	MUL (vectors, unpredicated)	FEAT_SVE2 || FEAT_SME
	10	SMULH (unpredicated)	FEAT_SVE2 || FEAT_SME
	11	UMULH (unpredicated)	FEAT_SVE2 || FEAT_SME
00	01	PMUL	FEAT_SVE2 || FEAT_SME
!= 00	01	UNALLOCATED	-

SVE2 signed saturating doubling multiply high (unpredicated)

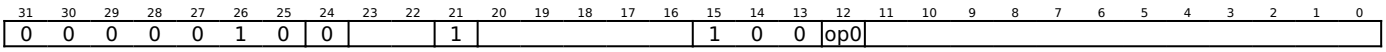
The encodings in this section are decoded from SVE2 Integer Arithmetic - Unpredicated.



Decode fields R	Instruction Details	Condition
0	SQDMULH (vectors)	FEAT_SVE2 || FEAT_SME
1	SQRDMULH (vectors)	FEAT_SVE2 || FEAT_SME

SVE Bitwise Shift - Unpredicated

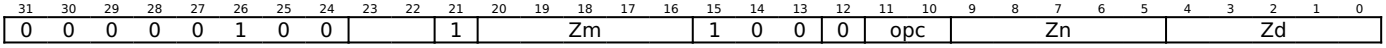
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0	SVE bitwise shift by wide elements (unpredicated)
1	SVE bitwise shift by immediate (unpredicated)

SVE bitwise shift by wide elements (unpredicated)

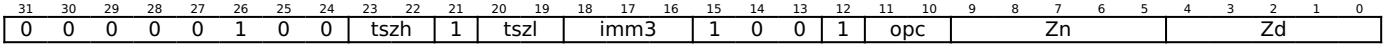
The encodings in this section are decoded from SVE Bitwise Shift - Unpredicated.



Decode fields opc	Instruction Details	Condition
00	ASR (wide elements, unpredicated)	FEAT_SVE || FEAT_SME
01	LSR (wide elements, unpredicated)	FEAT_SVE || FEAT_SME
10	UNALLOCATED	-
11	LSL (wide elements, unpredicated)	FEAT_SVE || FEAT_SME

SVE bitwise shift by immediate (unpredicated)

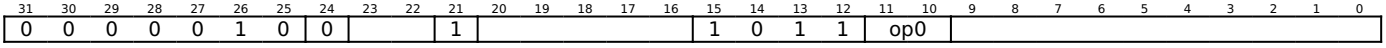
The encodings in this section are decoded from SVE Bitwise Shift - Unpredicated.



Decode fields opc	Instruction Details	Condition
00	ASR (immediate, unpredicated)	FEAT_SVE || FEAT_SME
01	LSR (immediate, unpredicated)	FEAT_SVE || FEAT_SME
10	UNALLOCATED	-
11	LSL (immediate, unpredicated)	FEAT_SVE || FEAT_SME

SVE Integer Misc - Unpredicated

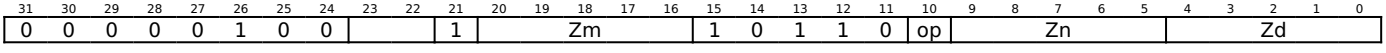
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0x	SVE floating-point trig select coefficient
10	SVE floating-point exponential accelerator
11	SVE constructive prefix (unpredicated)

SVE floating-point trig select coefficient

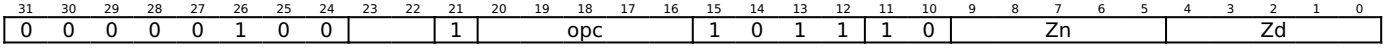
The encodings in this section are decoded from SVE Integer Misc - Unpredicated.



Decode fields op	Instruction Details	Condition
0	FTSSEL	FEAT_SVE
1	UNALLOCATED	-

SVE floating-point exponential accelerator

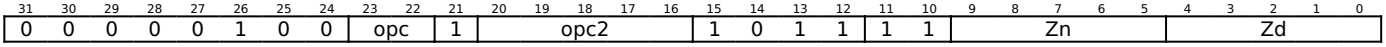
The encodings in this section are decoded from SVE Integer Misc - Unpredicated.



Decode fields opc	Instruction Details	Condition
00000	FEXPA	FEAT_SVE || FEAT_SSVE_FEXPA
!= 00000	UNALLOCATED	-

SVE constructive prefix (unpredicated)

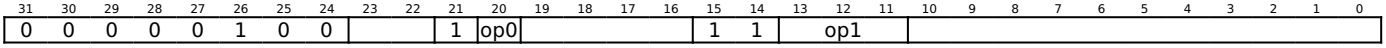
The encodings in this section are decoded from SVE Integer Misc - Unpredicated.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
00	00000	MOVPRFX (unpredicated)	FEAT_SVE || FEAT_SME
00	!= 00000	UNALLOCATED	-
!= 00		UNALLOCATED	-

SVE Element Count

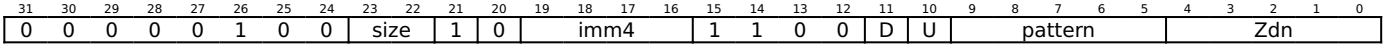
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Decode fields op1	Instruction details
0	00x	SVE saturating inc/dec vector by element count
0	100	SVE element count
0	101	UNALLOCATED
1	000	SVE inc/dec vector by element count
1	100	SVE inc/dec register by element count
1	x01	UNALLOCATED
	01x	UNALLOCATED
	11x	SVE saturating inc/dec register by element count

SVE saturating inc/dec vector by element count

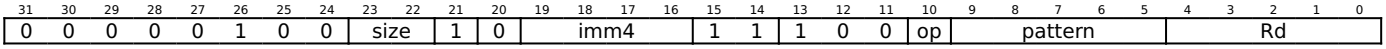
The encodings in this section are decoded from SVE Element Count.



Decode fields size	Decode fields D	Decode fields U	Instruction Details	Condition
00			UNALLOCATED	-
01	0	0	SQINCH (vector)	FEAT_SVE || FEAT_SME
01	0	1	UQINCH (vector)	FEAT_SVE || FEAT_SME
01	1	0	SQDECH (vector)	FEAT_SVE || FEAT_SME
01	1	1	UQDECH (vector)	FEAT_SVE || FEAT_SME
10	0	0	SQINCW (vector)	FEAT_SVE || FEAT_SME
10	0	1	UQINCW (vector)	FEAT_SVE || FEAT_SME
10	1	0	SQDECW (vector)	FEAT_SVE || FEAT_SME
10	1	1	UQDECW (vector)	FEAT_SVE || FEAT_SME
11	0	0	SQINCD (vector)	FEAT_SVE || FEAT_SME
11	0	1	UQINCD (vector)	FEAT_SVE || FEAT_SME
11	1	0	SQDECD (vector)	FEAT_SVE || FEAT_SME
11	1	1	UQDECD (vector)	FEAT_SVE || FEAT_SME

SVE element count

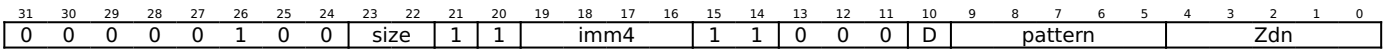
The encodings in this section are decoded from SVE Element Count.



Decode fields size	Decode fields op	Instruction Details	Condition
	1	UNALLOCATED	-
00	0	CNTB, CNTD, CNTH, CNTW — Byte	FEAT_SVE || FEAT_SME
01	0	CNTB, CNTD, CNTH, CNTW — Halfword	FEAT_SVE || FEAT_SME
10	0	CNTB, CNTD, CNTH, CNTW — Word	FEAT_SVE || FEAT_SME
11	0	CNTB, CNTD, CNTH, CNTW — Doubleword	FEAT_SVE || FEAT_SME

SVE inc/dec vector by element count

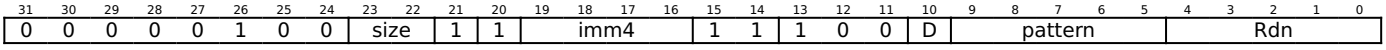
The encodings in this section are decoded from SVE Element Count.



Decode fields size	D	Instruction Details	Condition
00		UNALLOCATED	-
01	0	INCD, INCH, INCW (vector) — Halfword	FEAT_SVE || FEAT_SME
01	1	DECD, DECH, DECW (vector) — Halfword	FEAT_SVE || FEAT_SME
10	0	INCD, INCH, INCW (vector) — Word	FEAT_SVE || FEAT_SME
10	1	DECD, DECH, DECW (vector) — Word	FEAT_SVE || FEAT_SME
11	0	INCD, INCH, INCW (vector) — Doubleword	FEAT_SVE || FEAT_SME
11	1	DECD, DECH, DECW (vector) — Doubleword	FEAT_SVE || FEAT_SME

SVE inc/dec register by element count

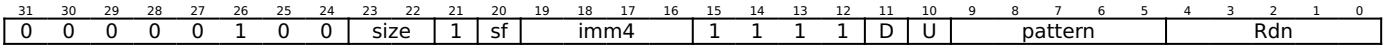
The encodings in this section are decoded from SVE Element Count.



Decode fields size	D	Instruction Details	Condition
00	0	INCB, INCD, INCH, INCW (scalar) — Byte	FEAT_SVE || FEAT_SME
00	1	DECB, DECD, DECH, DECW (scalar) — Byte	FEAT_SVE || FEAT_SME
01	0	INCB, INCD, INCH, INCW (scalar) — Halfword	FEAT_SVE || FEAT_SME
01	1	DECB, DECD, DECH, DECW (scalar) — Halfword	FEAT_SVE || FEAT_SME
10	0	INCB, INCD, INCH, INCW (scalar) — Word	FEAT_SVE || FEAT_SME
10	1	DECB, DECD, DECH, DECW (scalar) — Word	FEAT_SVE || FEAT_SME
11	0	INCB, INCD, INCH, INCW (scalar) — Doubleword	FEAT_SVE || FEAT_SME
11	1	DECB, DECD, DECH, DECW (scalar) — Doubleword	FEAT_SVE || FEAT_SME

SVE saturating inc/dec register by element count

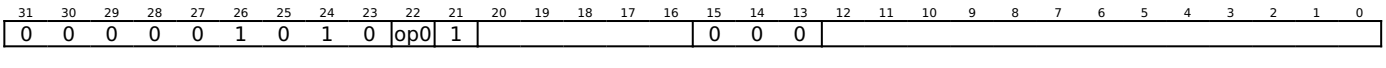
The encodings in this section are decoded from SVE Element Count.



Decode fields				Instruction Details	Condition
size	sf	D	U		
00	0	0	0	SQINCB — 32-bit	FEAT_SVE || FEAT_SME
00	0	0	1	UQINCB — 32-bit	FEAT_SVE || FEAT_SME
00	0	1	0	SQDECB — 32-bit	FEAT_SVE || FEAT_SME
00	0	1	1	UQDECB — 32-bit	FEAT_SVE || FEAT_SME
00	1	0	0	SQINCB — 64-bit	FEAT_SVE || FEAT_SME
00	1	0	1	UQINCB — 64-bit	FEAT_SVE || FEAT_SME
00	1	1	0	SQDECB — 64-bit	FEAT_SVE || FEAT_SME
00	1	1	1	UQDECB — 64-bit	FEAT_SVE || FEAT_SME
01	0	0	0	SQINCH (scalar) — 32-bit	FEAT_SVE || FEAT_SME
01	0	0	1	UQINCH (scalar) — 32-bit	FEAT_SVE || FEAT_SME
01	0	1	0	SQDECH (scalar) — 32-bit	FEAT_SVE || FEAT_SME
01	0	1	1	UQDECH (scalar) — 32-bit	FEAT_SVE || FEAT_SME
01	1	0	0	SQINCH (scalar) — 64-bit	FEAT_SVE || FEAT_SME
01	1	0	1	UQINCH (scalar) — 64-bit	FEAT_SVE || FEAT_SME
01	1	1	0	SQDECH (scalar) — 64-bit	FEAT_SVE || FEAT_SME
01	1	1	1	UQDECH (scalar) — 64-bit	FEAT_SVE || FEAT_SME
10	0	0	0	SQINCW (scalar) — 32-bit	FEAT_SVE || FEAT_SME
10	0	0	1	UQINCW (scalar) — 32-bit	FEAT_SVE || FEAT_SME
10	0	1	0	SQDECW (scalar) — 32-bit	FEAT_SVE || FEAT_SME
10	0	1	1	UQDECW (scalar) — 32-bit	FEAT_SVE || FEAT_SME
10	1	0	0	SQINCW (scalar) — 64-bit	FEAT_SVE || FEAT_SME
10	1	0	1	UQINCW (scalar) — 64-bit	FEAT_SVE || FEAT_SME
10	1	1	0	SQDECW (scalar) — 64-bit	FEAT_SVE || FEAT_SME
10	1	1	1	UQDECW (scalar) — 64-bit	FEAT_SVE || FEAT_SME
11	0	0	0	SQINCD (scalar) — 32-bit	FEAT_SVE || FEAT_SME
11	0	0	1	UQINCD (scalar) — 32-bit	FEAT_SVE || FEAT_SME
11	0	1	0	SQDECD (scalar) — 32-bit	FEAT_SVE || FEAT_SME
11	0	1	1	UQDECD (scalar) — 32-bit	FEAT_SVE || FEAT_SME
11	1	0	0	SQINCD (scalar) — 64-bit	FEAT_SVE || FEAT_SME
11	1	0	1	UQINCD (scalar) — 64-bit	FEAT_SVE || FEAT_SME
11	1	1	0	SQDECD (scalar) — 64-bit	FEAT_SVE || FEAT_SME
11	1	1	1	UQDECD (scalar) — 64-bit	FEAT_SVE || FEAT_SME

SVE Permute Vector - Extract

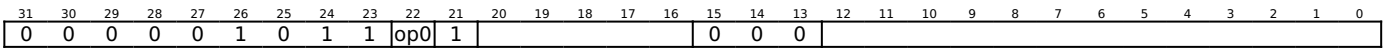
The encodings in this section are decoded from SVE encodings.



Decode fields	Instruction details	Condition
op0		
0	EXT — Destructive	FEAT_SVE || FEAT_SME
1	EXT — Constructive	FEAT_SVE2 || FEAT_SME

SVE Permute Vector - Segments

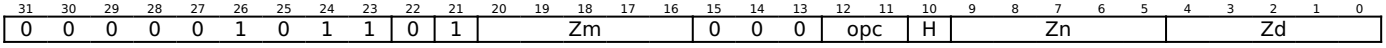
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0	SVE permute vector segments
1	UNALLOCATED

SVE permute vector segments

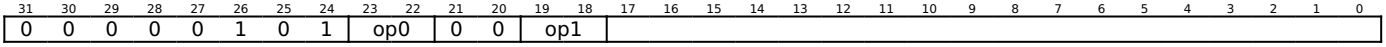
The encodings in this section are decoded from SVE Permute Vector - Segments.



Decode fields opc	H	Instruction Details	Condition
00	0	ZIP1, ZIP2 (vectors) — Low halves (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256
00	1	ZIP1, ZIP2 (vectors) — High halves (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256
01	0	UZP1, UZP2 (vectors) — Even (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256
01	1	UZP1, UZP2 (vectors) — Odd (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256
10		UNALLOCATED	-
11	0	TRN1, TRN2 (vectors) — Even (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256
11	1	TRN1, TRN2 (vectors) — Odd (quadwords)	FEAT_F64MM || FEAT_SVE_PERM256

SVE Bitwise Immediate

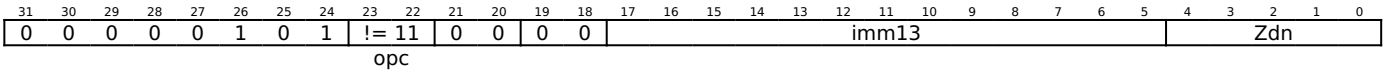
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details	Condition
11	00	DUPM	FEAT_SVE || FEAT_SME
	!= 00	UNALLOCATED	-
!= 11	00	SVE bitwise logical with immediate (unpredicated)	-

SVE bitwise logical with immediate (unpredicated)

The encodings in this section are decoded from SVE Bitwise Immediate.

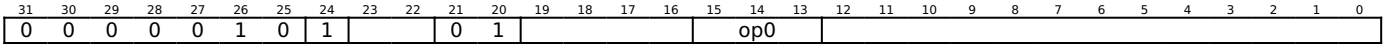


The following constraints also apply to this encoding: opc != '11'

Decode fields opc	Instruction Details	Condition
00	ORR (immediate)	FEAT_SVE || FEAT_SME
01	EOR (immediate)	FEAT_SVE || FEAT_SME
10	AND (immediate)	FEAT_SVE || FEAT_SME

SVE Integer Wide Immediate - Predicated

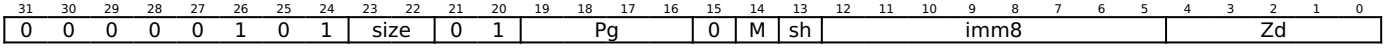
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details	Condition
0xx	SVE copy integer immediate (predicated)	-
10x	UNALLOCATED	-
110	FCPY	FEAT_SVE || FEAT_SME
111	UNALLOCATED	-

SVE copy integer immediate (predicated)

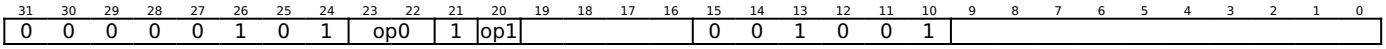
The encodings in this section are decoded from SVE Integer Wide Immediate - Predicated.



Decode fields M	Instruction Details	Condition
0	CPY (immediate, zeroing)	FEAT_SVE || FEAT_SME
1	CPY (immediate, merging)	FEAT_SVE || FEAT_SME

SVE Permute Vector - One Source Quadwords

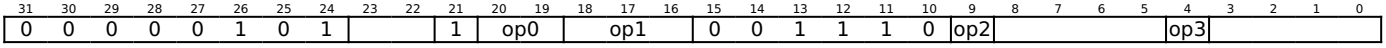
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Decode fields op1	Instruction details	Condition
00		DUPQ	FEAT_SVE2p1 || FEAT_SME2p1
01	0	EXTQ	FEAT_SVE2p1 || FEAT_SME2p1
01	1	UNALLOCATED	-
1x		UNALLOCATED	-

SVE Permute Vector - Unpredicated

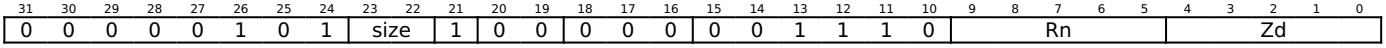
The encodings in this section are decoded from SVE encodings.



op0	Decode fields op1	op2	op3	Instruction details	Condition
00	000			SVE broadcast general register	-
00	100			INSR (scalar)	FEAT_SVE || FEAT_SME
00	!= x00			UNALLOCATED	-
01	xx0		0	SVE move predicate from vector	-
01	xx0		1	UNALLOCATED	-
01	xx1	0		SVE move predicate into vector	-
01	xx1	1		UNALLOCATED	-
10	0xx			SVE unpack vector elements	-
10	100			INSR (SIMD&FP scalar)	FEAT_SVE || FEAT_SME
10	101			UNALLOCATED	-
10	11x			UNALLOCATED	-
11	000			REV (vector)	FEAT_SVE || FEAT_SME
11	!= 000			UNALLOCATED	-

SVE broadcast general register

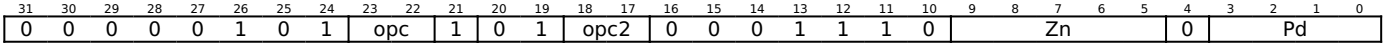
The encodings in this section are decoded from SVE Permute Vector - Unpredicated.



Decode fields size	Instruction Details	Condition
11	DUP (scalar) — 64-bit	FEAT_SVE || FEAT_SME
!= 11	DUP (scalar) — 32-bit	FEAT_SVE || FEAT_SME

SVE move predicate from vector

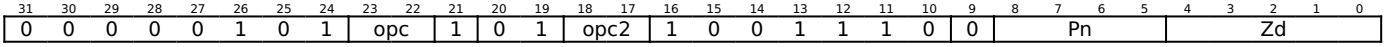
The encodings in this section are decoded from SVE Permute Vector - Unpredicated.



Decode fields opc	opc2	Instruction Details	Condition
00	00	UNALLOCATED	-
00	01	PMOV (to predicate) — Byte	FEAT_SVE2p1 || FEAT_SME2p1
00	1x	PMOV (to predicate) — Halfword	FEAT_SVE2p1 || FEAT_SME2p1
01		PMOV (to predicate) — Word	FEAT_SVE2p1 || FEAT_SME2p1
1x		PMOV (to predicate) — Doubleword	FEAT_SVE2p1 || FEAT_SME2p1

SVE move predicate into vector

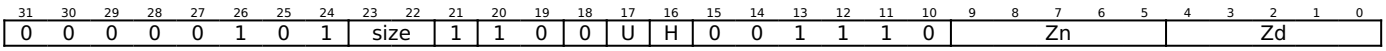
The encodings in this section are decoded from SVE Permute Vector - Unpredicated.



Decode fields opc	opc2	Instruction Details	Condition
00	00	UNALLOCATED	-
00	01	PMOV (to vector) — Byte	FEAT_SVE2p1 || FEAT_SME2p1
00	1x	PMOV (to vector) — Halfword	FEAT_SVE2p1 || FEAT_SME2p1
01		PMOV (to vector) — Word	FEAT_SVE2p1 || FEAT_SME2p1
1x		PMOV (to vector) — Doubleword	FEAT_SVE2p1 || FEAT_SME2p1

SVE unpack vector elements

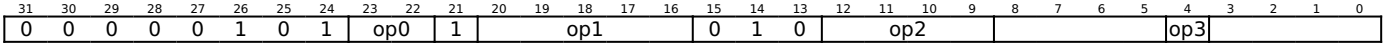
The encodings in this section are decoded from SVE Permute Vector - Unpredicated.



Decode fields U	H	Instruction Details	Condition
0	0	SUNPKHI, SUNPKLO — Low half	FEAT_SVE || FEAT_SME
0	1	SUNPKHI, SUNPKLO — High half	FEAT_SVE || FEAT_SME
1	0	UUNPKHI, UUNPKLO — Low half	FEAT_SVE || FEAT_SME
1	1	UUNPKHI, UUNPKLO — High half	FEAT_SVE || FEAT_SME

SVE Permute Predicate

The encodings in this section are decoded from SVE encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
00	1000x	0000	0	SVE unpack predicate elements	-
	0xxxx	xxx0	0	SVE permute predicate elements	-
	10100	0000	0	REV (predicate)	FEAT_SVE || FEAT_SME
	10101	0000	0	UNALLOCATED	-
	10x0x	0010	0	UNALLOCATED	-
	10x0x	01x0	0	UNALLOCATED	-
	10x0x	1xx0	0	UNALLOCATED	-
	10x1x	xxx0	0	UNALLOCATED	-
	11xxx	xxx0	0	UNALLOCATED	-
		xxx0	1	UNALLOCATED	-
		xxx1		UNALLOCATED	-
!= 00	1000x	0000	0	UNALLOCATED	-

SVE unpack predicate elements

The encodings in this section are decoded from SVE Permute Predicate.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	0	0	1	1	0	0	0	H	0	1	0	0	0	0	0	Pn				0	Pd			

Decode fields	Instruction Details	Condition
H		
0	PUNPKHI, PUNPKLO — Low half	FEAT_SVE || FEAT_SME
1	PUNPKHI, PUNPKLO — High half	FEAT_SVE || FEAT_SME

SVE permute predicate elements

The encodings in this section are decoded from SVE Permute Predicate.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size		1	0	Pm				0	1	0	opc		H	0	Pn				0	Pd			

Decode fields		Instruction Details	Condition
opc	H		
00	0	ZIP1, ZIP2 (predicates) — Low halves	FEAT_SVE || FEAT_SME
00	1	ZIP1, ZIP2 (predicates) — High halves	FEAT_SVE || FEAT_SME
01	0	UZP1, UZP2 (predicates) — Even	FEAT_SVE || FEAT_SME
01	1	UZP1, UZP2 (predicates) — Odd	FEAT_SVE || FEAT_SME
10	0	TRN1, TRN2 (predicates) — Even	FEAT_SVE || FEAT_SME
10	1	TRN1, TRN2 (predicates) — Odd	FEAT_SVE || FEAT_SME
11		UNALLOCATED	-

SVE Permute Vector - Predicated

The encodings in this section are decoded from SVE encodings.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1			1	op0	op1			op2	1	0	op3													

Decode fields				Instruction details	Condition
op0	op1	op2	op3		
0	000	0	0	CPY (SIMD&FP scalar)	FEAT_SVE || FEAT_SME
0	000	1	0	SVE compress Active elements	-
0	000		1	SVE extract element to general register	-
0	001	1	1	UNALLOCATED	-
0	001		0	SVE extract element to SIMD&FP scalar register	-
0	01x			SVE reverse within elements	-
0	100	0	1	SVE copy general register to vector (predicated)	-
0	100		0	SVE conditionally broadcast element to vector	-
0	101		0	SVE conditionally extract element to SIMD&FP scalar	-
0	110	0	0	SPLICE — Destructive	FEAT_SVE || FEAT_SME
0	110	0	1	UNALLOCATED	-
0	110	1	0	SPLICE — Constructive	FEAT_SVE2 || FEAT_SME
0	111	0		SVE reverse doublewords	-
0	111	1	0	UNALLOCATED	-
0	1xx	1	1	UNALLOCATED	-
0	x01	0	1	UNALLOCATED	-
1	000	0	0	UNALLOCATED	-
1	000	1	0	EXPAND	FEAT_SVE2p2 || FEAT_SME2p2
1	000		1	SVE conditionally extract element to general register	-
1	!= 000			UNALLOCATED	-

SVE compress Active elements

The encodings in this section are decoded from SVE Permute Vector - Predicated.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size		1	0	0	0	0	1	1	0	0	Pg			Zn					Zd				

Decode fields	Instruction Details	Condition
size		
0x	COMPACT — Byte and halfword	FEAT_SVE2p2 || FEAT_SME2p2
1x	COMPACT — Word and doubleword	FEAT_SVE || FEAT_SME2p2

SVE extract element to general register

The encodings in this section are decoded from SVE Permute Vector - Predicated.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size		1	0	0	0	0	B	1	0	1	Pg			Zn					Rd				

Decode fields	Instruction Details	Condition
B		
0	LASTA (scalar)	FEAT_SVE || FEAT_SME
1	LASTB (scalar)	FEAT_SVE || FEAT_SME

SVE extract element to SIMD&FP scalar register

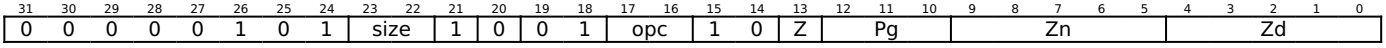
The encodings in this section are decoded from SVE Permute Vector - Predicated.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	1	size		1	0	0	0	1	B	1	0	0	Pg			Zn					Vd				

Decode fields B	Instruction Details	Condition
0	LASTA (SIMD&FP scalar)	FEAT_SVE || FEAT_SME
1	LASTB (SIMD&FP scalar)	FEAT_SVE || FEAT_SME

SVE reverse within elements

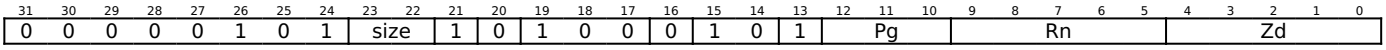
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields opc	Decode fields Z	Instruction Details	Condition
00	0	REVB, REVH, REVW — Byte, merging	FEAT_SVE || FEAT_SME
00	1	REVB, REVH, REVW — Byte, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	0	REVB, REVH, REVW — Halfword, merging	FEAT_SVE || FEAT_SME
01	1	REVB, REVH, REVW — Halfword, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	0	REVB, REVH, REVW — Word, merging	FEAT_SVE || FEAT_SME
10	1	REVB, REVH, REVW — Word, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	0	RBIT — Merging	FEAT_SVE || FEAT_SME
11	1	RBIT — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2

SVE copy general register to vector (predicated)

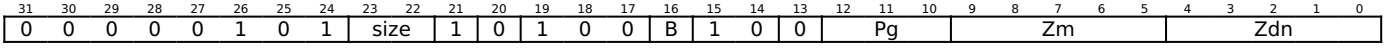
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields size	Instruction Details	Condition
11	CPY (scalar) — 64-bit	FEAT_SVE || FEAT_SME
!= 11	CPY (scalar) — 32-bit	FEAT_SVE || FEAT_SME

SVE conditionally broadcast element to vector

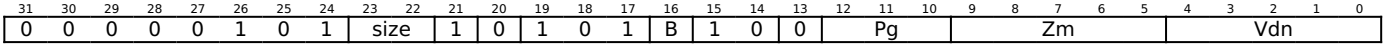
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields B	Instruction Details	Condition
0	CLASTA (vectors)	FEAT_SVE || FEAT_SME
1	CLASTB (vectors)	FEAT_SVE || FEAT_SME

SVE conditionally extract element to SIMD&FP scalar

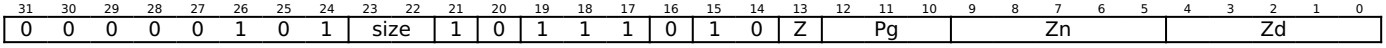
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields B	Instruction Details	Condition
0	CLASTA (SIMD&FP scalar)	FEAT_SVE || FEAT_SME
1	CLASTB (SIMD&FP scalar)	FEAT_SVE || FEAT_SME

SVE reverse doublewords

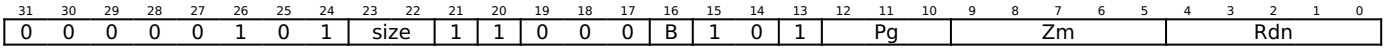
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields size	Z	Instruction Details	Condition
00	0	REVD — Merging	FEAT_SME || FEAT_SVE2p1
00	1	REVD — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
!= 00		UNALLOCATED	-

SVE conditionally extract element to general register

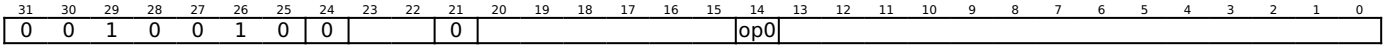
The encodings in this section are decoded from SVE Permute Vector - Predicated.



Decode fields B	Instruction Details	Condition
0	CLASTA (scalar)	FEAT_SVE || FEAT_SME
1	CLASTB (scalar)	FEAT_SVE || FEAT_SME

SVE Integer Compare - Vectors

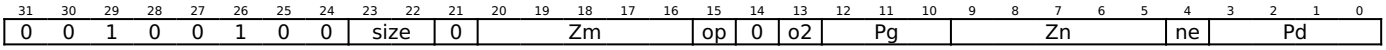
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0	SVE integer compare vectors
1	SVE integer compare with wide elements

SVE integer compare vectors

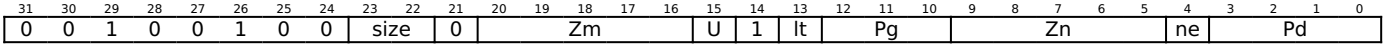
The encodings in this section are decoded from SVE Integer Compare - Vectors.



Decode fields op	o2	ne	Instruction Details	Condition
0	0	0	CMP<cc> (vectors) — Higher or same	FEAT_SVE || FEAT_SME
0	0	1	CMP<cc> (vectors) — Higher	FEAT_SVE || FEAT_SME
0	1	0	CMP<cc> (wide elements) — Equal	FEAT_SVE || FEAT_SME
0	1	1	CMP<cc> (wide elements) — Not equal	FEAT_SVE || FEAT_SME
1	0	0	CMP<cc> (vectors) — Greater than or equal	FEAT_SVE || FEAT_SME
1	0	1	CMP<cc> (vectors) — Greater than	FEAT_SVE || FEAT_SME
1	1	0	CMP<cc> (vectors) — Equal	FEAT_SVE || FEAT_SME
1	1	1	CMP<cc> (vectors) — Not equal	FEAT_SVE || FEAT_SME

SVE integer compare with wide elements

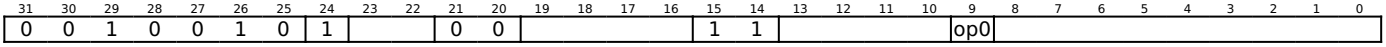
The encodings in this section are decoded from SVE Integer Compare - Vectors.



Decode fields U	lt	ne	Instruction Details	Condition
0	0	0	CMP<cc> (wide elements) — Greater than or equal	FEAT_SVE || FEAT_SME
0	0	1	CMP<cc> (wide elements) — Greater than	FEAT_SVE || FEAT_SME
0	1	0	CMP<cc> (wide elements) — Less than	FEAT_SVE || FEAT_SME
0	1	1	CMP<cc> (wide elements) — Less than or equal	FEAT_SVE || FEAT_SME
1	0	0	CMP<cc> (wide elements) — Higher or same	FEAT_SVE || FEAT_SME
1	0	1	CMP<cc> (wide elements) — Higher	FEAT_SVE || FEAT_SME
1	1	0	CMP<cc> (wide elements) — Lower	FEAT_SVE || FEAT_SME
1	1	1	CMP<cc> (wide elements) — Lower or same	FEAT_SVE || FEAT_SME

SVE Propagate Break

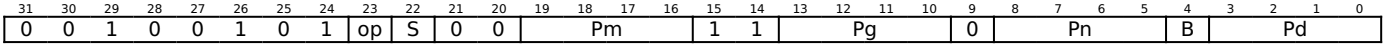
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0	SVE propagate break from previous partition
1	UNALLOCATED

SVE propagate break from previous partition

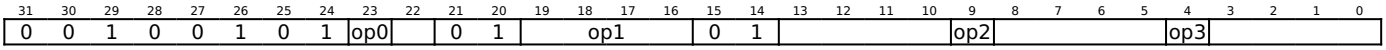
The encodings in this section are decoded from SVE Propagate Break.



Decode fields op	S	B	Instruction Details	Condition
0	0	0	BRKPA	FEAT_SVE || FEAT_SME
0	0	1	BRKPB	FEAT_SVE || FEAT_SME
0	1	0	BRKPAS	FEAT_SVE || FEAT_SME
0	1	1	BRKPBS	FEAT_SVE || FEAT_SME
1			UNALLOCATED	-

SVE Partition Break

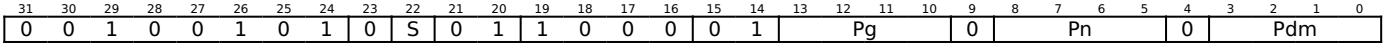
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	Instruction details
0	1000	0	0	SVE propagate break to next partition
0	1000	0	1	UNALLOCATED
1	1000	0		UNALLOCATED
	0000	0		SVE partition break condition
	x000	1		UNALLOCATED
	!= x000			UNALLOCATED

SVE propagate break to next partition

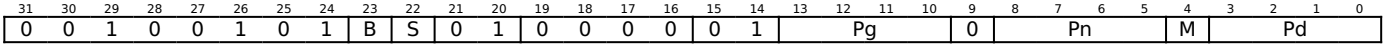
The encodings in this section are decoded from SVE Partition Break.



Decode fields S	Instruction Details	Condition
0	BRKN	FEAT_SVE || FEAT_SME
1	BRKNS	FEAT_SVE || FEAT_SME

SVE partition break condition

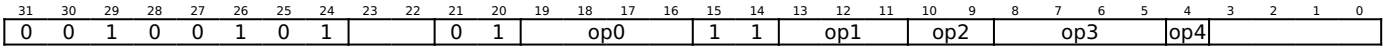
The encodings in this section are decoded from SVE Partition Break.



Decode fields B	S	M	Instruction Details	Condition
	1	1	UNALLOCATED	-
0	0		BRKA	FEAT_SVE || FEAT_SME
0	1	0	BRKAS	FEAT_SVE || FEAT_SME
1	0		BRKB	FEAT_SVE || FEAT_SME
1	1	0	BRKBS	FEAT_SVE || FEAT_SME

SVE Predicate Misc

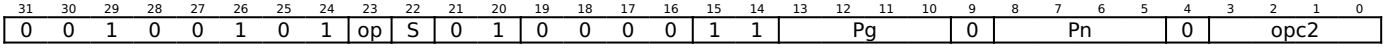
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	op4	Instruction details	Condition
0000		x0		0	SVE predicate test	-
0000		x1		0	UNALLOCATED	-
0001				0	UNALLOCATED	-
1000	000	00		0	SVE predicate first active	-
1000	000	10		0	UNALLOCATED	-
1000	100	10	0000	0	SVE predicate zero	-
1000	100	10	!= 0000	0	UNALLOCATED	-
1000	110	00		0	SVE predicate read from FFR (predicated)	-
1001	000	00		0	UNALLOCATED	-
1001	000	10		0	PNEXT	FEAT_SVE || FEAT_SME
1001	100	10		0	UNALLOCATED	-
1001	110	00	0000	0	SVE predicate read from FFR (unpredicated)	-
1001	110	00	!= 0000	0	UNALLOCATED	-
100x	010	x0		0	UNALLOCATED	-
100x	0x0	x1		0	UNALLOCATED	-
100x	100	0x		0	SVE predicate initialize	-
100x	100	11		0	UNALLOCATED	-
100x	110	!= 00		0	UNALLOCATED	-
100x	xx1			0	UNALLOCATED	-
x00x				1	UNALLOCATED	-
!= x00x					UNALLOCATED	-

SVE predicate test

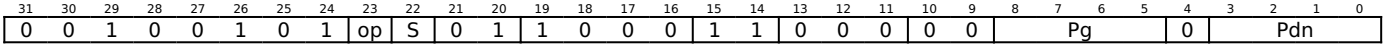
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields			Instruction Details	Condition
op	S	opc2		
0	0		UNALLOCATED	-
0	1	0000	PTEST	FEAT_SVE || FEAT_SME
0	1	!= 0000	UNALLOCATED	-
1			UNALLOCATED	-

SVE predicate first active

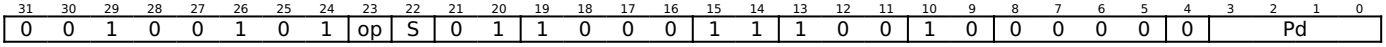
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields		Instruction Details	Condition
op	S		
0	0	UNALLOCATED	-
0	1	PFIRST	FEAT_SVE || FEAT_SME
1		UNALLOCATED	-

SVE predicate zero

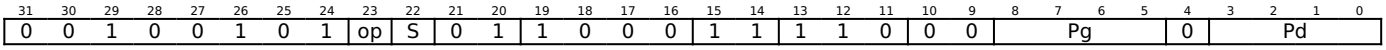
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields		Instruction Details	Condition
op	S		
0	0	PFALSE	FEAT_SVE || FEAT_SME
0	1	UNALLOCATED	-
1		UNALLOCATED	-

SVE predicate read from FFR (predicated)

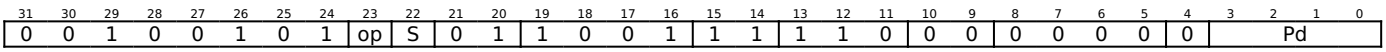
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields		Instruction Details	Condition
op	S		
0	0	RDFFR (predicated)	FEAT_SVE
0	1	RDFFRS	FEAT_SVE
1		UNALLOCATED	-

SVE predicate read from FFR (unpredicated)

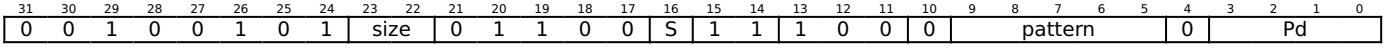
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields op	S	Instruction Details	Condition
0	0	RDFFR (unpredicated)	FEAT_SVE
0	1	UNALLOCATED	-
1		UNALLOCATED	-

SVE predicate initialize

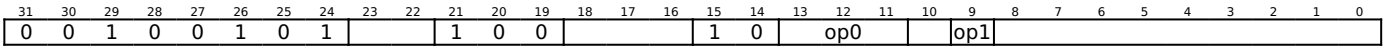
The encodings in this section are decoded from SVE Predicate Misc.



Decode fields S	Instruction Details	Condition
0	PTRUE (predicate)	FEAT_SVE || FEAT_SME
1	PTRUES	FEAT_SVE || FEAT_SME

SVE Predicate Count

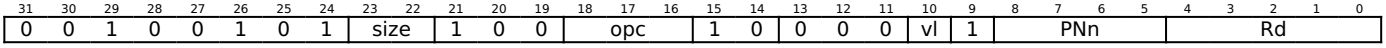
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
000	1	SVE predicate count (predicate-as-counter)
	0	SVE predicate count
!= 000	1	UNALLOCATED

SVE predicate count (predicate-as-counter)

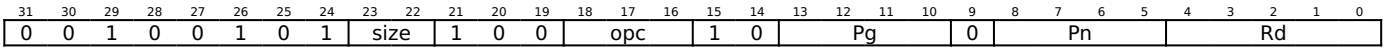
The encodings in this section are decoded from SVE Predicate Count.



Decode fields opc	Instruction Details	Condition
000	CNTP (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
!= 000	UNALLOCATED	-

SVE predicate count

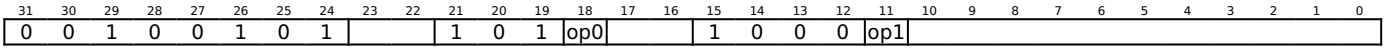
The encodings in this section are decoded from SVE Predicate Count.



Decode fields opc	Instruction Details	Condition
000	CNTP (predicate)	FEAT_SVE || FEAT_SME
001	FIRSTP	FEAT_SVE2p2 || FEAT_SME2p2
010	LASTP	FEAT_SVE2p2 || FEAT_SME2p2
011	UNALLOCATED	-
1xx	UNALLOCATED	-

SVE Inc/Dec by Predicate Count

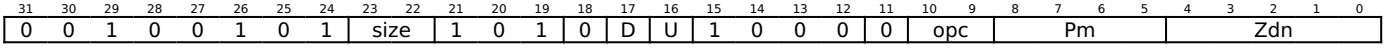
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Decode fields op1	Instruction details
0	0	SVE saturating inc/dec vector by predicate count
0	1	SVE saturating inc/dec register by predicate count
1	0	SVE inc/dec vector by predicate count
1	1	SVE inc/dec register by predicate count

SVE saturating inc/dec vector by predicate count

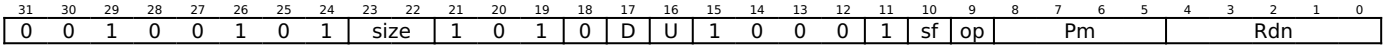
The encodings in this section are decoded from SVE Inc/Dec by Predicate Count.



Decode fields D	Decode fields U	Decode fields opc	Instruction Details	Condition
		!= 00	UNALLOCATED	-
0	0	00	SQINCP (vector)	FEAT_SVE || FEAT_SME
0	1	00	UQINCP (vector)	FEAT_SVE || FEAT_SME
1	0	00	SQDECP (vector)	FEAT_SVE || FEAT_SME
1	1	00	UQDECP (vector)	FEAT_SVE || FEAT_SME

SVE saturating inc/dec register by predicate count

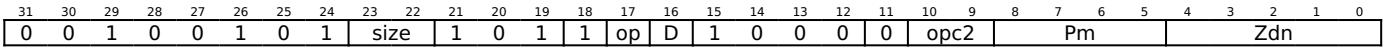
The encodings in this section are decoded from SVE Inc/Dec by Predicate Count.



Decode fields D	Decode fields U	Decode fields sf	Decode fields op	Instruction Details	Condition
			1	UNALLOCATED	-
0	0	0	0	SQINCP (scalar) — 32-bit	FEAT_SVE || FEAT_SME
0	0	1	0	SQINCP (scalar) — 64-bit	FEAT_SVE || FEAT_SME
0	1	0	0	UQINCP (scalar) — 32-bit	FEAT_SVE || FEAT_SME
0	1	1	0	UQINCP (scalar) — 64-bit	FEAT_SVE || FEAT_SME
1	0	0	0	SQDECP (scalar) — 32-bit	FEAT_SVE || FEAT_SME
1	0	1	0	SQDECP (scalar) — 64-bit	FEAT_SVE || FEAT_SME
1	1	0	0	UQDECP (scalar) — 32-bit	FEAT_SVE || FEAT_SME
1	1	1	0	UQDECP (scalar) — 64-bit	FEAT_SVE || FEAT_SME

SVE inc/dec vector by predicate count

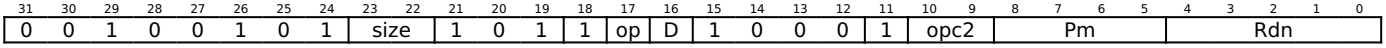
The encodings in this section are decoded from SVE Inc/Dec by Predicate Count.



Decode fields			Instruction Details	Condition
op	D	opc2		
0		!= 00	UNALLOCATED	-
0	0	00	INCP (vector)	FEAT_SVE || FEAT_SME
0	1	00	DECP (vector)	FEAT_SVE || FEAT_SME
1			UNALLOCATED	-

SVE inc/dec register by predicate count

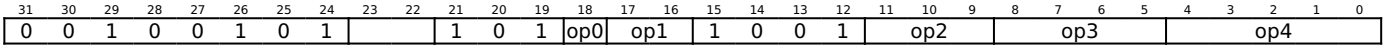
The encodings in this section are decoded from SVE Inc/Dec by Predicate Count.



Decode fields			Instruction Details	Condition
op	D	opc2		
0		!= 00	UNALLOCATED	-
0	0	00	INCP (scalar)	FEAT_SVE || FEAT_SME
0	1	00	DECP (scalar)	FEAT_SVE || FEAT_SME
1			UNALLOCATED	-

SVE Write FFR

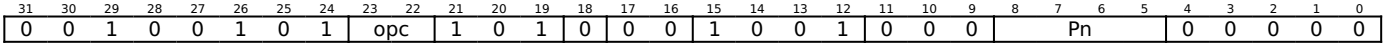
The encodings in this section are decoded from SVE encodings.



Decode fields					Instruction details
op0	op1	op2	op3	op4	
0	00	000		00000	SVE FFR write from predicate
1	00	000	0000	00000	SVE FFR initialize
1	00	000	!= 0000	00000	UNALLOCATED
	00	000		!= 00000	UNALLOCATED
	00	!= 000			UNALLOCATED
	!= 00				UNALLOCATED

SVE FFR write from predicate

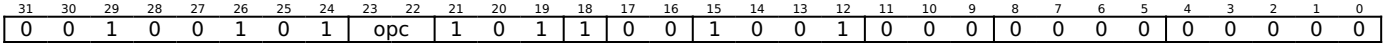
The encodings in this section are decoded from SVE Write FFR.



Decode fields	Instruction Details	Condition
opc		
00	WRFFR	FEAT_SVE
!= 00	UNALLOCATED	-

SVE FFR initialize

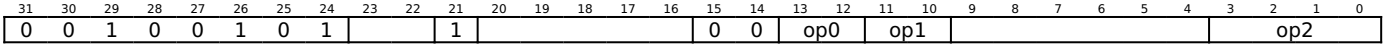
The encodings in this section are decoded from SVE Write FFR.



Decode fields opc	Instruction Details	Condition
00	SETFFR	FEAT_SVE
!= 00	UNALLOCATED	-

SVE Integer Compare - Scalars

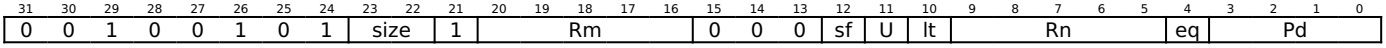
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	Instruction details
0x			SVE integer compare scalar count and limit
10	00	0000	SVE conditionally terminate scalars
10	00	!= 0000	UNALLOCATED
11	00		SVE pointer conflict compare
1x	!= 00		UNALLOCATED

SVE integer compare scalar count and limit

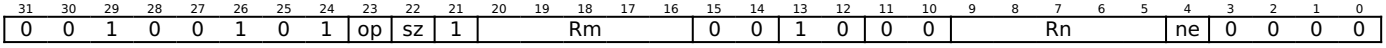
The encodings in this section are decoded from SVE Integer Compare - Scalars.



Decode fields U	lt	eq	Instruction Details	Condition
0	0	0	WHILEGE (predicate)	FEAT_SVE2 || FEAT_SME
0	0	1	WHILEGT (predicate)	FEAT_SVE2 || FEAT_SME
0	1	0	WHILELT (predicate)	FEAT_SVE || FEAT_SME
0	1	1	WHILELE (predicate)	FEAT_SVE || FEAT_SME
1	0	0	WHILEHS (predicate)	FEAT_SVE2 || FEAT_SME
1	0	1	WHILEHI (predicate)	FEAT_SVE2 || FEAT_SME
1	1	0	WHILELO (predicate)	FEAT_SVE || FEAT_SME
1	1	1	WHILELS (predicate)	FEAT_SVE || FEAT_SME

SVE conditionally terminate scalars

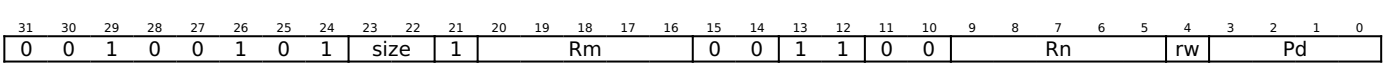
The encodings in this section are decoded from SVE Integer Compare - Scalars.



Decode fields op	ne	Instruction Details	Condition
0		UNALLOCATED	-
1	0	CTERMEQ, CTERMNE — Equal	FEAT_SVE || FEAT_SME
1	1	CTERMEQ, CTERMNE — Not equal	FEAT_SVE || FEAT_SME

SVE pointer conflict compare

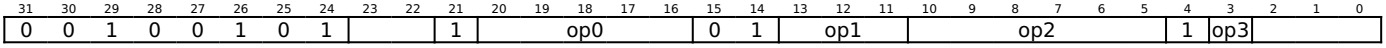
The encodings in this section are decoded from SVE Integer Compare - Scalars.



Decode fields rw	Instruction Details	Condition
0	WHILEWR	FEAT_SVE2 || FEAT_SME
1	WHILERW	FEAT_SVE2 || FEAT_SME

SVE Scalar Integer Compare - Predicate-as-counter

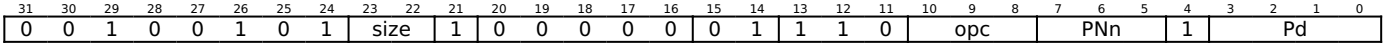
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	Instruction details	Condition
00000	110			SVE extract mask predicate from predicate-as-counter	-
00000	111	000000	0	PTRUE (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
00000	111	000000	1	UNALLOCATED	-
00000	111	!= 000000		UNALLOCATED	-
	01x			SVE integer compare scalar count and limit (predicate pair)	-
	x0x			SVE integer compare scalar count and limit (predicate-as-counter)	-
!= 00000	11x			UNALLOCATED	-

SVE extract mask predicate from predicate-as-counter

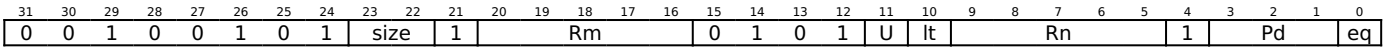
The encodings in this section are decoded from SVE Scalar Integer Compare - Predicate-as-counter.



Decode fields opc	Instruction Details	Condition
0xx	PEXT (predicate)	FEAT_SME2 || FEAT_SVE2p1
10x	PEXT (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
11x	UNALLOCATED	-

SVE integer compare scalar count and limit (predicate pair)

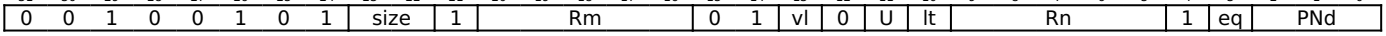
The encodings in this section are decoded from SVE Scalar Integer Compare - Predicate-as-counter.



Decode fields U	lt	eq	Instruction Details	Condition
0	0	0	WHILEGE (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
0	0	1	WHILEGT (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
0	1	0	WHILELT (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
0	1	1	WHILELE (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
1	0	0	WHILEHS (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
1	0	1	WHILEHI (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
1	1	0	WHILELO (predicate pair)	FEAT_SME2 || FEAT_SVE2p1
1	1	1	WHILELS (predicate pair)	FEAT_SME2 || FEAT_SVE2p1

SVE integer compare scalar count and limit (predicate-as-counter)

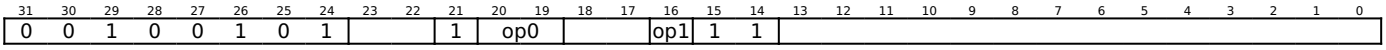
The encodings in this section are decoded from SVE Scalar Integer Compare - Predicate-as-counter.



Decode fields U	lt	eq	Instruction Details	Condition
0	0	0	WHILEGE (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
0	0	1	WHILEGT (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
0	1	0	WHILELT (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
0	1	1	WHILELE (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
1	0	0	WHILEHS (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
1	0	1	WHILEHI (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
1	1	0	WHILELO (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1
1	1	1	WHILELS (predicate as counter)	FEAT_SME2 || FEAT_SVE2p1

SVE Integer Wide Immediate - Unpredicated

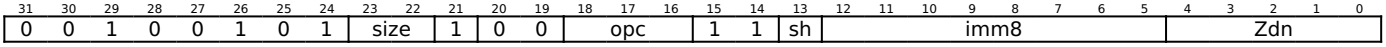
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
00		SVE integer add/subtract immediate (unpredicated)
01		SVE integer min/max immediate (unpredicated)
10		SVE integer multiply immediate (unpredicated)
11	0	SVE broadcast integer immediate (unpredicated)
11	1	SVE broadcast floating-point immediate (unpredicated)

SVE integer add/subtract immediate (unpredicated)

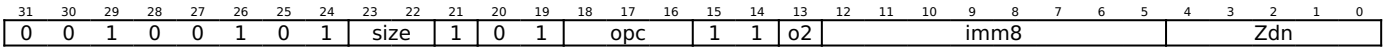
The encodings in this section are decoded from SVE Integer Wide Immediate - Unpredicated.



Decode fields opc	Instruction Details	Condition
000	ADD (immediate)	FEAT_SVE || FEAT_SME
001	SUB (immediate)	FEAT_SVE || FEAT_SME
010	UNALLOCATED	-
011	SUBR (immediate)	FEAT_SVE || FEAT_SME
100	SQADD (immediate)	FEAT_SVE || FEAT_SME
101	UQADD (immediate)	FEAT_SVE || FEAT_SME
110	SQSUB (immediate)	FEAT_SVE || FEAT_SME
111	UQSUB (immediate)	FEAT_SVE || FEAT_SME

SVE integer min/max immediate (unpredicated)

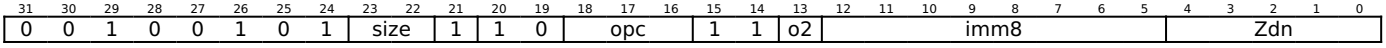
The encodings in this section are decoded from SVE Integer Wide Immediate - Unpredicated.



Decode fields		Instruction Details	Condition
opc	o2		
0xx	1	UNALLOCATED	-
000	0	SMAX (immediate)	FEAT_SVE || FEAT_SME
001	0	UMAX (immediate)	FEAT_SVE || FEAT_SME
010	0	SMIN (immediate)	FEAT_SVE || FEAT_SME
011	0	UMIN (immediate)	FEAT_SVE || FEAT_SME
1xx		UNALLOCATED	-

SVE integer multiply immediate (unpredicated)

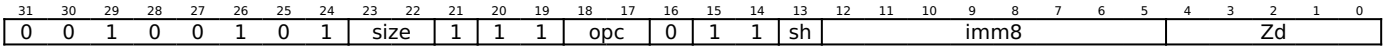
The encodings in this section are decoded from SVE Integer Wide Immediate - Unpredicated.



Decode fields		Instruction Details	Condition
opc	o2		
000	0	MUL (immediate)	FEAT_SVE || FEAT_SME
000	1	UNALLOCATED	-
!= 000		UNALLOCATED	-

SVE broadcast integer immediate (unpredicated)

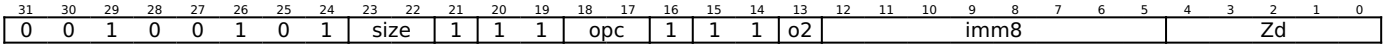
The encodings in this section are decoded from SVE Integer Wide Immediate - Unpredicated.



Decode fields	Instruction Details	Condition
opc		
00	DUP (immediate)	FEAT_SVE || FEAT_SME
!= 00	UNALLOCATED	-

SVE broadcast floating-point immediate (unpredicated)

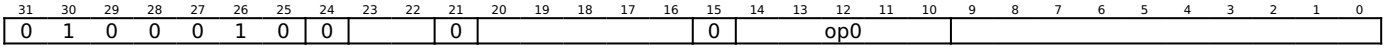
The encodings in this section are decoded from SVE Integer Wide Immediate - Unpredicated.



Decode fields		Instruction Details	Condition
opc	o2		
00	0	FDUP	FEAT_SVE || FEAT_SME
00	1	UNALLOCATED	-
!= 00		UNALLOCATED	-

SVE Integer Multiply-Add - Unpredicated

The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details	Condition
0000x	SVE integer dot product (unpredicated)	-
0001x	SVE2 saturating multiply-add interleaved long	-
001xx	CDOT (vectors)	FEAT_SVE2 || FEAT_SME
01xxx	SVE2 complex integer multiply-add	-
10xxx	SVE2 integer multiply-add long	-
110xx	SVE2 saturating multiply-add long	-
1110x	SVE2 saturating multiply-add high	-
11110	SVE mixed sign dot product	-
11111	UNALLOCATED	-

SVE integer dot product (unpredicated)

The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size		0	Zm					0	0	0	0	0	U	Zn					Zda				

Decode fields size	U	Instruction Details	Condition
00		UNALLOCATED	-
01	0	SDOT (2-way, vectors)	FEAT_SVE2p3 || FEAT_SME2p3
01	1	UDOT (2-way, vectors)	FEAT_SVE2p3 || FEAT_SME2p3
1x	0	SDOT (4-way, vectors)	FEAT_SVE || FEAT_SME
1x	1	UDOT (4-way, vectors)	FEAT_SVE || FEAT_SME

SVE2 saturating multiply-add interleaved long

The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size		0	Zm					0	0	0	0	1	S	Zn					Zda				

Decode fields S	Instruction Details	Condition
0	SQDMLALBT	FEAT_SVE2 || FEAT_SME
1	SQDMLSLBT	FEAT_SVE2 || FEAT_SME

SVE2 complex integer multiply-add

The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size		0	Zm					0	0	1	op	rot		Zn					Zda				

Decode fields op	Instruction Details	Condition
0	CMLA (vectors)	FEAT_SVE2 || FEAT_SME
1	SQRDCMLAH (vectors)	FEAT_SVE2 || FEAT_SME

SVE2 integer multiply-add long

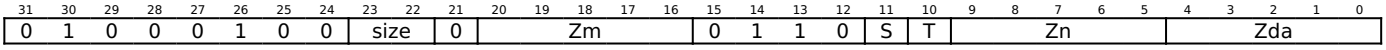
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size		0	Zm					0	1	0	S	U	T	Zn					Zda				

Decode fields S	U	T	Instruction Details	Condition
0	0	0	SMLALB (vectors)	FEAT_SVE2 || FEAT_SME
0	0	1	SMLALT (vectors)	FEAT_SVE2 || FEAT_SME
0	1	0	UMLALB (vectors)	FEAT_SVE2 || FEAT_SME
0	1	1	UMLALT (vectors)	FEAT_SVE2 || FEAT_SME
1	0	0	SMLSLB (vectors)	FEAT_SVE2 || FEAT_SME
1	0	1	SMLSLT (vectors)	FEAT_SVE2 || FEAT_SME
1	1	0	UMLSLB (vectors)	FEAT_SVE2 || FEAT_SME
1	1	1	UMLSLT (vectors)	FEAT_SVE2 || FEAT_SME

SVE2 saturating multiply-add long

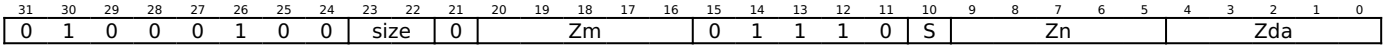
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.



Decode fields S	T	Instruction Details	Condition
0	0	SQDMLALB (vectors)	FEAT_SVE2 || FEAT_SME
0	1	SQDMLALT (vectors)	FEAT_SVE2 || FEAT_SME
1	0	SQDMLSLB (vectors)	FEAT_SVE2 || FEAT_SME
1	1	SQDMLSLT (vectors)	FEAT_SVE2 || FEAT_SME

SVE2 saturating multiply-add high

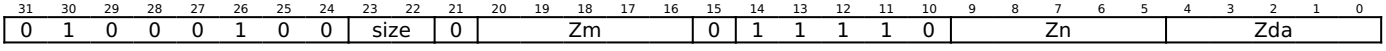
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.



Decode fields S	Instruction Details	Condition
0	SQRDMLAH (vectors)	FEAT_SVE2 || FEAT_SME
1	SQRDMLSH (vectors)	FEAT_SVE2 || FEAT_SME

SVE mixed sign dot product

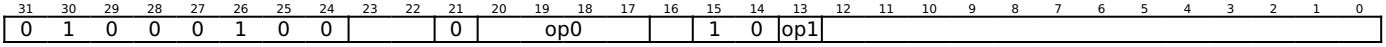
The encodings in this section are decoded from SVE Integer Multiply-Add - Unpredicated.



Decode fields size	Instruction Details	Condition
10	USDOT (vectors)	(FEAT_SVE && FEAT_I8MM) || (FEAT_SME && FEAT_I8MM)
!= 10	UNALLOCATED	-

SVE2 Integer - Predicated

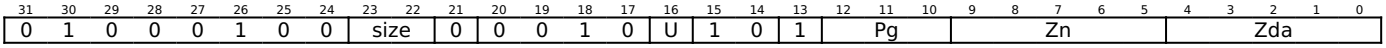
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
0010	1	SVE2 integer pairwise add and accumulate long
0011	1	UNALLOCATED
011x	1	UNALLOCATED
0x0x	1	SVE2 integer unary operations (predicated)
0xxx	0	SVE2 saturating/rounding bitwise shift left (predicated)
10xx	0	SVE2 integer halving add/subtract (predicated)
10xx	1	SVE2 integer pairwise arithmetic
11xx	0	SVE2 saturating add/subtract
11xx	1	UNALLOCATED

SVE2 integer pairwise add and accumulate long

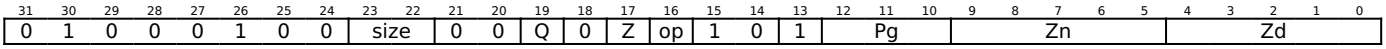
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields U	Instruction Details	Condition
0	SADALP	FEAT_SVE2 || FEAT_SME
1	UADALP	FEAT_SVE2 || FEAT_SME

SVE2 integer unary operations (predicated)

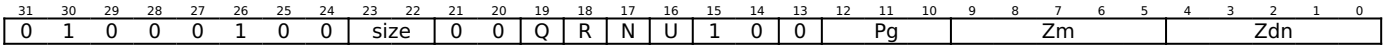
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields Q	Z	op	Instruction Details	Condition
0	0	0	URECPE — Merging	FEAT_SVE2 || FEAT_SME
0	0	1	URSQRTE — Merging	FEAT_SVE2 || FEAT_SME
0	1	0	URECPE — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	1	1	URSQRTE — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	0	0	SQABS — Merging	FEAT_SVE2 || FEAT_SME
1	0	1	SQNEG — Merging	FEAT_SVE2 || FEAT_SME
1	1	0	SQABS — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	1	1	SQNEG — Zeroing	FEAT_SVE2p2 || FEAT_SME2p2

SVE2 saturating/rounding bitwise shift left (predicated)

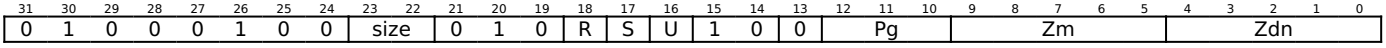
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields				Instruction Details	Condition
Q	R	N	U		
0		0		UNALLOCATED	-
0	0	1	0	SRSHL	FEAT_SVE2 || FEAT_SME
0	0	1	1	URSHL	FEAT_SVE2 || FEAT_SME
0	1	1	0	SRSHLR	FEAT_SVE2 || FEAT_SME
0	1	1	1	URSHLR	FEAT_SVE2 || FEAT_SME
1	0	0	0	SQSHL (vectors)	FEAT_SVE2 || FEAT_SME
1	0	0	1	UQSHL (vectors)	FEAT_SVE2 || FEAT_SME
1	0	1	0	SQRSHL	FEAT_SVE2 || FEAT_SME
1	0	1	1	UQRSHL	FEAT_SVE2 || FEAT_SME
1	1	0	0	SQSHLR	FEAT_SVE2 || FEAT_SME
1	1	0	1	UQSHLR	FEAT_SVE2 || FEAT_SME
1	1	1	0	SQRSHLR	FEAT_SVE2 || FEAT_SME
1	1	1	1	UQRSHLR	FEAT_SVE2 || FEAT_SME

SVE2 integer halving add/subtract (predicated)

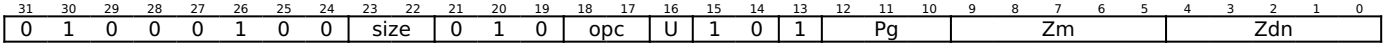
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields			Instruction Details	Condition
R	S	U		
0	0	0	SHADD	FEAT_SVE2 || FEAT_SME
0	0	1	UHADD	FEAT_SVE2 || FEAT_SME
0	1	0	SHSUB	FEAT_SVE2 || FEAT_SME
0	1	1	UHSUB	FEAT_SVE2 || FEAT_SME
1	0	0	SRHADD	FEAT_SVE2 || FEAT_SME
1	0	1	URHADD	FEAT_SVE2 || FEAT_SME
1	1	0	SHSUBR	FEAT_SVE2 || FEAT_SME
1	1	1	UHSUBR	FEAT_SVE2 || FEAT_SME

SVE2 integer pairwise arithmetic

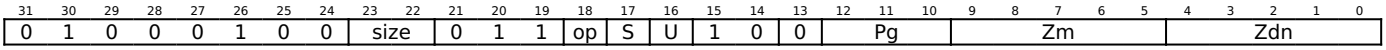
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields		Instruction Details	Condition
opc	U		
00	0	SUBP	FEAT_SVE2p3 || FEAT_SME2p3
00	1	ADDP	FEAT_SVE2 || FEAT_SME
01		UNALLOCATED	-
10	0	SMAXP	FEAT_SVE2 || FEAT_SME
10	1	UMAXP	FEAT_SVE2 || FEAT_SME
11	0	SMINP	FEAT_SVE2 || FEAT_SME
11	1	UMINP	FEAT_SVE2 || FEAT_SME

SVE2 saturating add/subtract

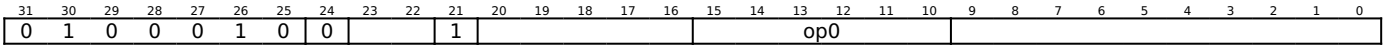
The encodings in this section are decoded from SVE2 Integer - Predicated.



Decode fields			Instruction Details	Condition
op	S	U		
0	0	0	SQADD (vectors, predicated)	FEAT_SVE2 || FEAT_SME
0	0	1	UQADD (vectors, predicated)	FEAT_SVE2 || FEAT_SME
0	1	0	SQSUB (vectors, predicated)	FEAT_SVE2 || FEAT_SME
0	1	1	UQSUB (vectors, predicated)	FEAT_SVE2 || FEAT_SME
1	0	0	SUQADD	FEAT_SVE2 || FEAT_SME
1	0	1	USQADD	FEAT_SVE2 || FEAT_SME
1	1	0	SQSUBR	FEAT_SVE2 || FEAT_SME
1	1	1	UQSUBR	FEAT_SVE2 || FEAT_SME

SVE Multiply - Indexed

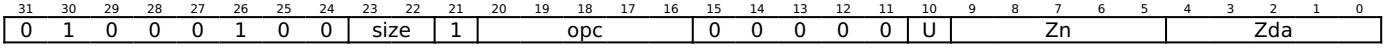
The encodings in this section are decoded from SVE encodings.



Decode fields	Instruction details
op0	
00000x	SVE integer dot product (indexed)
00001x	SVE2 integer multiply-add (indexed)
00010x	SVE2 saturating multiply-add high (indexed)
00011x	SVE mixed sign dot product (indexed)
001xxx	SVE2 saturating multiply-add (indexed)
0100xx	SVE2 complex integer dot product (indexed)
0101xx	UNALLOCATED
0110xx	SVE2 complex integer multiply-add (indexed)
0111xx	SVE2 complex saturating multiply-add (indexed)
10xxxx	SVE2 integer multiply-add long (indexed)
110xxx	SVE2 integer multiply long (indexed)
1110xx	SVE2 saturating multiply (indexed)
11110x	SVE2 saturating multiply high (indexed)
111110	SVE2 integer multiply (indexed)
111111	UNALLOCATED

SVE integer dot product (indexed)

The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields		Instruction Details	Condition
size	U		
0x	0	SDOT (2-way, indexed)	FEAT_SVE2p3 || FEAT_SME2p3
0x	1	UDOT (2-way, indexed)	FEAT_SVE2p3 || FEAT_SME2p3
10	0	SDOT (4-way, indexed) — 8-bit to 32-bit	FEAT_SVE || FEAT_SME
10	1	UDOT (4-way, indexed) — 8-bit to 32-bit	FEAT_SVE || FEAT_SME
11	0	SDOT (4-way, indexed) — 16-bit to 64-bit	FEAT_SVE || FEAT_SME
11	1	UDOT (4-way, indexed) — 16-bit to 64-bit	FEAT_SVE || FEAT_SME

SVE2 integer multiply-add (indexed)

The encodings in this section are decoded from SVE Multiply - Indexed.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size		1	opc					0	0	0	0	1	S	Zn					Zda				

Decode fields		Instruction Details	Condition
size	S		
0x	0	MLA (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
0x	1	MLS (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
10	0	MLA (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	MLS (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	MLA (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	MLS (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 saturating multiply-add high (indexed)

The encodings in this section are decoded from SVE Multiply - Indexed.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size		1	opc					0	0	0	1	0	S	Zn					Zda				

Decode fields		Instruction Details	Condition
size	S		
0x	0	SQRDMLAH (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
0x	1	SQRDMLSH (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
10	0	SQRDMLAH (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	SQRDMLSH (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	SQRDMLAH (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	SQRDMLSH (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE mixed sign dot product (indexed)

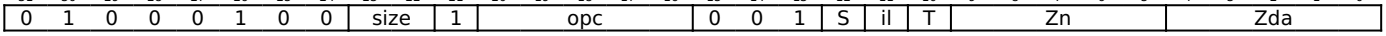
The encodings in this section are decoded from SVE Multiply - Indexed.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	0	size		1	opc					0	0	0	1	1	U	Zn					Zda				

Decode fields		Instruction Details	Condition
size	U		
10	0	USDOT (indexed)	(FEAT_SVE && FEAT_I8MM) || (FEAT_SME && FEAT_I8MM)
10	1	SUDOT	(FEAT_SVE && FEAT_I8MM) || (FEAT_SME && FEAT_I8MM)
!= 10		UNALLOCATED	-

SVE2 saturating multiply-add (indexed)

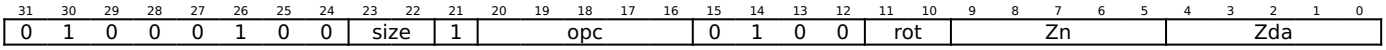
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	S	T	Instruction Details	Condition
0x			UNALLOCATED	-
10	0	0	SQDMLALB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	0	1	SQDMLALT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	0	SQDMLSLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	1	SQDMLSLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	0	SQDMLALB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	0	1	SQDMLALT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	0	SQDMLSLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	1	SQDMLSLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 complex integer dot product (indexed)

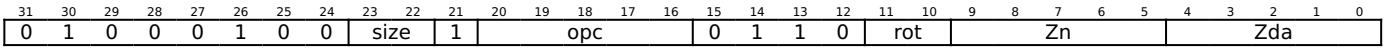
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	Instruction Details	Condition
0x	UNALLOCATED	-
10	CDOT (indexed) — 8-bit to 32-bit	FEAT_SVE2 || FEAT_SME
11	CDOT (indexed) — 16-bit to 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 complex integer multiply-add (indexed)

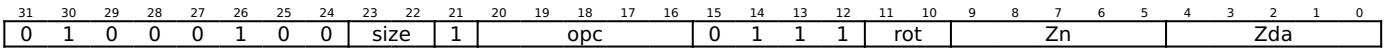
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	Instruction Details	Condition
0x	UNALLOCATED	-
10	CMLA (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
11	CMLA (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME

SVE2 complex saturating multiply-add (indexed)

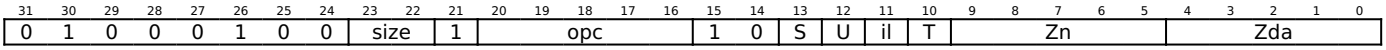
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	Instruction Details	Condition
0x	UNALLOCATED	-
10	SQRDCMLAH (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
11	SQRDCMLAH (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME

SVE2 integer multiply-add long (indexed)

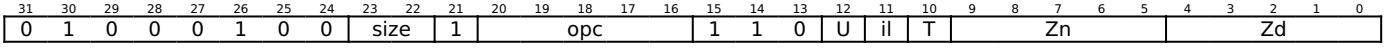
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields				Instruction Details	Condition
size	S	U	T		
0x				UNALLOCATED	-
10	0	0	0	SMLALB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	0	0	1	SMLALT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	0	1	0	UMLALB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	0	1	1	UMLALT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	0	0	SMLSLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	0	1	SMLSLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	1	0	UMLSLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	1	1	UMLSLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	0	0	SMLALB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	0	0	1	SMLALT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	0	1	0	UMLALB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	0	1	1	UMLALT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	0	0	SMLSLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	0	1	SMLSLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	1	0	UMLSLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	1	1	UMLSLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 integer multiply long (indexed)

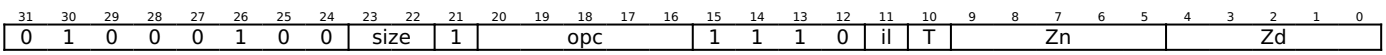
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields			Instruction Details	Condition
size	U	T		
0x			UNALLOCATED	-
10	0	0	SMULLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	0	1	SMULLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	0	UMULLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	1	UMULLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	0	SMULLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	0	1	SMULLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	0	UMULLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	1	UMULLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 saturating multiply (indexed)

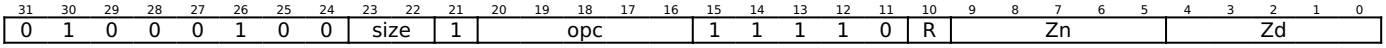
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	T	Instruction Details	Condition
0x		UNALLOCATED	-
10	0	SQDMULLB (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	SQDMULLT (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	SQDMULLB (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	SQDMULLT (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 saturating multiply high (indexed)

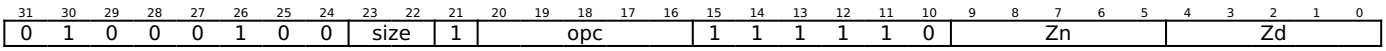
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	R	Instruction Details	Condition
0x	0	SQDMULH (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
0x	1	SQRDMULH (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
10	0	SQDMULH (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
10	1	SQRDMULH (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	0	SQDMULH (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME
11	1	SQRDMULH (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 integer multiply (indexed)

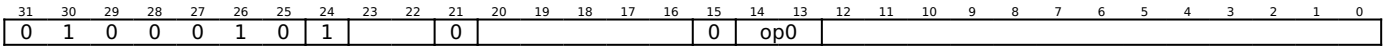
The encodings in this section are decoded from SVE Multiply - Indexed.



Decode fields size	Instruction Details	Condition
0x	MUL (indexed) — 16-bit	FEAT_SVE2 || FEAT_SME
10	MUL (indexed) — 32-bit	FEAT_SVE2 || FEAT_SME
11	MUL (indexed) — 64-bit	FEAT_SVE2 || FEAT_SME

SVE2 Widening Integer Arithmetic

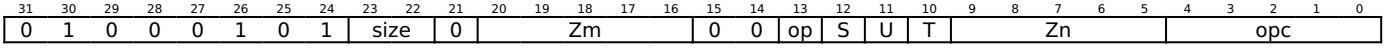
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0x	SVE2 integer add/subtract long
10	SVE2 integer add/subtract wide
11	SVE2 integer multiply long

SVE2 integer add/subtract long

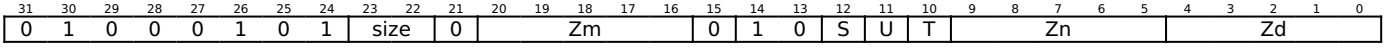
The encodings in this section are decoded from SVE2 Widening Integer Arithmetic.



Decode fields				Instruction Details	Condition
op	S	U	T		
0	0	0	0	SADDLB	FEAT_SVE2 || FEAT_SME
0	0	0	1	SADDLT	FEAT_SVE2 || FEAT_SME
0	0	1	0	UADDLB	FEAT_SVE2 || FEAT_SME
0	0	1	1	UADDLT	FEAT_SVE2 || FEAT_SME
0	1	0	0	SSUBLB	FEAT_SVE2 || FEAT_SME
0	1	0	1	SSUBLT	FEAT_SVE2 || FEAT_SME
0	1	1	0	USUBLB	FEAT_SVE2 || FEAT_SME
0	1	1	1	USUBLT	FEAT_SVE2 || FEAT_SME
1	0			UNALLOCATED	-
1	1	0	0	SABDLB	FEAT_SVE2 || FEAT_SME
1	1	0	1	SABDLT	FEAT_SVE2 || FEAT_SME
1	1	1	0	UABDLB	FEAT_SVE2 || FEAT_SME
1	1	1	1	UABDLT	FEAT_SVE2 || FEAT_SME

SVE2 integer add/subtract wide

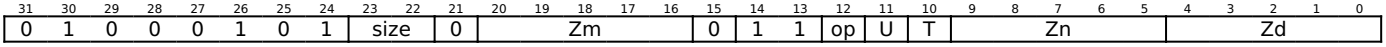
The encodings in this section are decoded from SVE2 Widening Integer Arithmetic.



Decode fields			Instruction Details	Condition
S	U	T		
0	0	0	SADDWB	FEAT_SVE2 || FEAT_SME
0	0	1	SADDWT	FEAT_SVE2 || FEAT_SME
0	1	0	UADDWB	FEAT_SVE2 || FEAT_SME
0	1	1	UADDWT	FEAT_SVE2 || FEAT_SME
1	0	0	SSUBWB	FEAT_SVE2 || FEAT_SME
1	0	1	SSUBWT	FEAT_SVE2 || FEAT_SME
1	1	0	USUBWB	FEAT_SVE2 || FEAT_SME
1	1	1	USUBWT	FEAT_SVE2 || FEAT_SME

SVE2 integer multiply long

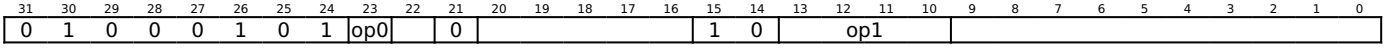
The encodings in this section are decoded from SVE2 Widening Integer Arithmetic.



Decode fields				Instruction Details	Condition
size	op	U	T		
	0	0	0	SQDMULLB (vectors)	FEAT_SVE2 || FEAT_SME
	0	0	1	SQDMULLT (vectors)	FEAT_SVE2 || FEAT_SME
	1	0	0	SMULLB (vectors)	FEAT_SVE2 || FEAT_SME
	1	0	1	SMULLT (vectors)	FEAT_SVE2 || FEAT_SME
	1	1	0	UMULLB (vectors)	FEAT_SVE2 || FEAT_SME
	1	1	1	UMULLT (vectors)	FEAT_SVE2 || FEAT_SME
00	0	1	0	PMULLB — 128-bit element	FEAT_SVE_PMULL128
00	0	1	1	PMULLT — 128-bit element	FEAT_SVE_PMULL128
!= 00	0	1	0	PMULLB — 16-bit or 64-bit elements	FEAT_SVE2 || FEAT_SME
!= 00	0	1	1	PMULLT — 16-bit or 64-bit elements	FEAT_SVE2 || FEAT_SME

SVE Misc

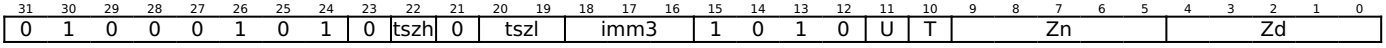
The encodings in this section are decoded from SVE encodings.



Decode fields		Instruction details
op0	op1	
0	10xx	SVE2 bitwise shift left long
1	10xx	UNALLOCATED
	00xx	SVE2 integer add/subtract interleaved long
	010x	SVE2 bitwise exclusive-OR interleaved
	0110	SVE integer matrix multiply accumulate
	0111	UNALLOCATED
	11xx	SVE2 bitwise permute

SVE2 bitwise shift left long

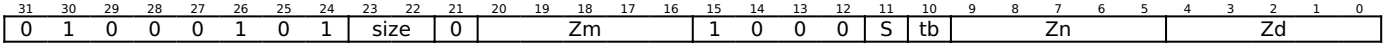
The encodings in this section are decoded from SVE Misc.



Decode fields		Instruction Details	Condition
U	T		
0	0	SSHLLB	FEAT_SVE2 || FEAT_SME
0	1	SSHLLT	FEAT_SVE2 || FEAT_SME
1	0	USHLLB	FEAT_SVE2 || FEAT_SME
1	1	USHLLT	FEAT_SVE2 || FEAT_SME

SVE2 integer add/subtract interleaved long

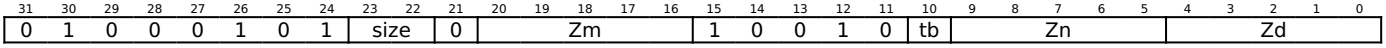
The encodings in this section are decoded from SVE Misc.



Decode fields S	tb	Instruction Details	Condition
0	0	SADDLBT	FEAT_SVE2 || FEAT_SME
0	1	UNALLOCATED	-
1	0	SSUBLBT	FEAT_SVE2 || FEAT_SME
1	1	SSUBLTB	FEAT_SVE2 || FEAT_SME

SVE2 bitwise exclusive-OR interleaved

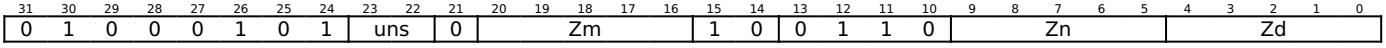
The encodings in this section are decoded from SVE Misc.



Decode fields tb	Instruction Details	Condition
0	EORBT	FEAT_SVE2 || FEAT_SME
1	EORTB	FEAT_SVE2 || FEAT_SME

SVE integer matrix multiply accumulate

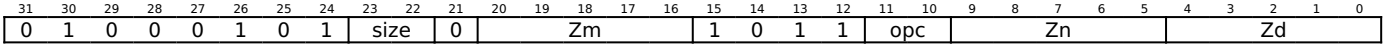
The encodings in this section are decoded from SVE Misc.



Decode fields uns	Instruction Details	Condition
00	SMMLA	FEAT_SVE && FEAT_I8MM
01	UNALLOCATED	-
10	USMMLA	FEAT_SVE && FEAT_I8MM
11	UMMLA	FEAT_SVE && FEAT_I8MM

SVE2 bitwise permute

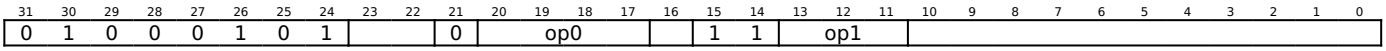
The encodings in this section are decoded from SVE Misc.



Decode fields opc	Instruction Details	Condition
00	BEXT	FEAT_SVE_BitPerm
01	BDEP	FEAT_SVE_BitPerm
10	BGRP	FEAT_SVE_BitPerm
11	UNALLOCATED	-

SVE2 Accumulate

The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
0000	011	SVE2 complex integer add
	00x	SVE2 integer absolute difference and accumulate long
	010	SVE2 integer add/subtract long with carry
	10x	SVE2 bitwise shift right and accumulate
	110	SVE2 bitwise shift and insert
	111	SVE2 integer absolute difference and accumulate
!= 0000	011	UNALLOCATED

SVE2 complex integer add

The encodings in this section are decoded from SVE2 Accumulate.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size		0	0	0	0	0	op	1	1	0	1	1	rot	Zm					Zdn				

Decode fields op	Instruction Details	Condition
0	CADD	FEAT_SVE2 || FEAT_SME
1	SQCADD	FEAT_SVE2 || FEAT_SME

SVE2 integer absolute difference and accumulate long

The encodings in this section are decoded from SVE2 Accumulate.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size		0	Zm					1	1	0	0	U	T	Zn					Zda				

Decode fields U	T	Instruction Details	Condition
0	0	SABALB	FEAT_SVE2 || FEAT_SME
0	1	SABALT	FEAT_SVE2 || FEAT_SME
1	0	UABALB	FEAT_SVE2 || FEAT_SME
1	1	UABALT	FEAT_SVE2 || FEAT_SME

SVE2 integer add/subtract long with carry

The encodings in this section are decoded from SVE2 Accumulate.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	size		0	Zm					1	1	0	1	0	T	Zn					Zda				

Decode fields size	T	Instruction Details	Condition
0x	0	ADCLB	FEAT_SVE2 || FEAT_SME
0x	1	ADCLT	FEAT_SVE2 || FEAT_SME
1x	0	SBCLB	FEAT_SVE2 || FEAT_SME
1x	1	SBCLT	FEAT_SVE2 || FEAT_SME

SVE2 bitwise shift right and accumulate

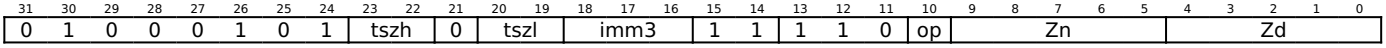
The encodings in this section are decoded from SVE2 Accumulate.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	1	0	1	tszh		0	tszl		imm3			1	1	1	0	R	U	Zn					Zda				

Decode fields R	U	Instruction Details	Condition
0	0	SSRA	FEAT_SVE2 || FEAT_SME
0	1	USRA	FEAT_SVE2 || FEAT_SME
1	0	SRSRA	FEAT_SVE2 || FEAT_SME
1	1	URSRA	FEAT_SVE2 || FEAT_SME

SVE2 bitwise shift and insert

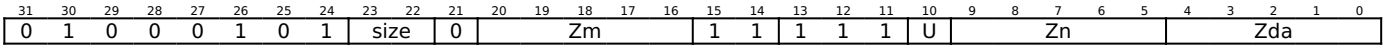
The encodings in this section are decoded from SVE2 Accumulate.



Decode fields op	Instruction Details	Condition
0	SRI	FEAT_SVE2 || FEAT_SME
1	SLI	FEAT_SVE2 || FEAT_SME

SVE2 integer absolute difference and accumulate

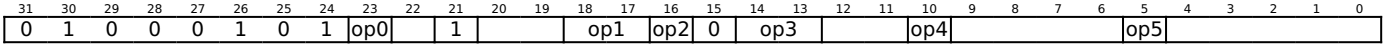
The encodings in this section are decoded from SVE2 Accumulate.



Decode fields U	Instruction Details	Condition
0	SABA	FEAT_SVE2 || FEAT_SME
1	UABA	FEAT_SVE2 || FEAT_SME

SVE2 Narrowing

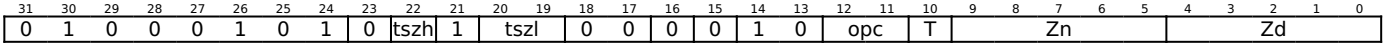
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	op4	op5	Instruction details
0	00	0	10			SVE2 saturating extract narrow
0	00	1	10	0	0	SME2 multi-vec extract narrow
0	00	1	10	0	1	UNALLOCATED
0	00	1	10	1		UNALLOCATED
0			0x			SVE2 bitwise shift right narrow
0	!= 00		10			UNALLOCATED
1			0x	0	0	SME2 multi-vec shift narrow
1			0x	0	1	UNALLOCATED
1			0x	1		UNALLOCATED
1			10			UNALLOCATED
			11			SVE2 integer add/subtract narrow high part

SVE2 saturating extract narrow

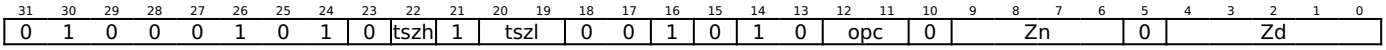
The encodings in this section are decoded from SVE2 Narrowing.



Decode fields		Instruction Details	Condition
opc	T		
00	0	SQXTNB	FEAT_SVE2 || FEAT_SME
00	1	SQXTNT	FEAT_SVE2 || FEAT_SME
01	0	UQXTNB	FEAT_SVE2 || FEAT_SME
01	1	UQXTNT	FEAT_SVE2 || FEAT_SME
10	0	SQXTUNB	FEAT_SVE2 || FEAT_SME
10	1	SQXTUNT	FEAT_SVE2 || FEAT_SME
11		UNALLOCATED	-

SME2 multi-vec extract narrow

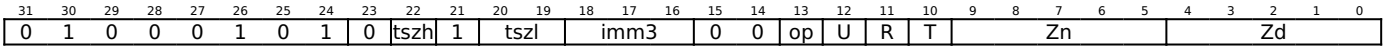
The encodings in this section are decoded from SVE2 Narrowing.



Decode fields			Instruction Details	Condition
tszh	tszl	opc		
0	10	00	SQCVTN	FEAT_SME2 || FEAT_SVE2p1
0	10	01	UQCVTN	FEAT_SME2 || FEAT_SVE2p1
0	10	10	SQCVTUN	FEAT_SME2 || FEAT_SVE2p1
0	10	11	UNALLOCATED	-
0	!= 10		UNALLOCATED	-
1			UNALLOCATED	-

SVE2 bitwise shift right narrow

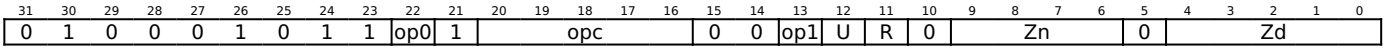
The encodings in this section are decoded from SVE2 Narrowing.



Decode fields				Instruction Details	Condition
op	U	R	T		
0	0	0	0	SQSHRUNB	FEAT_SVE2 || FEAT_SME
0	0	0	1	SQSHRUNT	FEAT_SVE2 || FEAT_SME
0	0	1	0	SQRSHRUNB	FEAT_SVE2 || FEAT_SME
0	0	1	1	SQRSHRUNT	FEAT_SVE2 || FEAT_SME
0	1	0	0	SHRNB	FEAT_SVE2 || FEAT_SME
0	1	0	1	SHRNT	FEAT_SVE2 || FEAT_SME
0	1	1	0	RSHRNB	FEAT_SVE2 || FEAT_SME
0	1	1	1	RSHRNT	FEAT_SVE2 || FEAT_SME
1	0	0	0	SQSHRNB	FEAT_SVE2 || FEAT_SME
1	0	0	1	SQSHRNT	FEAT_SVE2 || FEAT_SME
1	0	1	0	SQRSHRNB	FEAT_SVE2 || FEAT_SME
1	0	1	1	SQRSHRNT	FEAT_SVE2 || FEAT_SME
1	1	0	0	UQSHRNB	FEAT_SVE2 || FEAT_SME
1	1	0	1	UQSHRNT	FEAT_SVE2 || FEAT_SME
1	1	1	0	UQRSHRNB	FEAT_SVE2 || FEAT_SME
1	1	1	1	UQRSHRNT	FEAT_SVE2 || FEAT_SME

SME2 multi-vec shift narrow

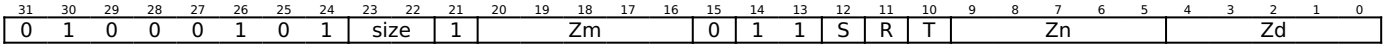
The encodings in this section are decoded from SVE2 Narrowing.



Decode fields					Instruction Details	Condition
op0	opc	op1	U	R		
0		0	0	0	SQSHRN	FEAT_SVE2p3 || FEAT_SME2p3
0		0	1	0	UQSHRN	FEAT_SVE2p3 || FEAT_SME2p3
0		1	0	0	SQSHRUN	FEAT_SVE2p3 || FEAT_SME2p3
0		1	1	0	UNALLOCATED	-
0	00xxx			1	UNALLOCATED	-
0	01xxx	0	0	1	SQRSHRUN — 8-bit	FEAT_SVE2p3 || FEAT_SME2p3
0	01xxx	1	0	1	SQRSHRN — 8-bit	FEAT_SVE2p3 || FEAT_SME2p3
0	01xxx	1	1	1	UQRSHRN — 8-bit	FEAT_SVE2p3 || FEAT_SME2p3
0	1xxxx	0	0	1	SQRSHRUN — 16-bit	FEAT_SME2 || FEAT_SVE2p1
0	1xxxx	1	0	1	SQRSHRN — 16-bit	FEAT_SME2 || FEAT_SVE2p1
0	1xxxx	1	1	1	UQRSHRN — 16-bit	FEAT_SME2 || FEAT_SVE2p1
0	!= 00xxx	0	1	1	UNALLOCATED	-
1					UNALLOCATED	-

SVE2 integer add/subtract narrow high part

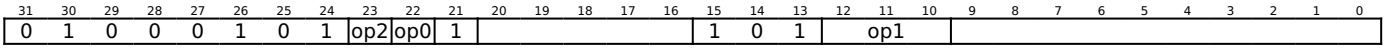
The encodings in this section are decoded from SVE2 Narrowing.



Decode fields			Instruction Details	Condition
S	R	T		
0	0	0	ADDHNB	FEAT_SVE2 || FEAT_SME
0	0	1	ADDHNT	FEAT_SVE2 || FEAT_SME
0	1	0	RADDHNB	FEAT_SVE2 || FEAT_SME
0	1	1	RADDHNT	FEAT_SVE2 || FEAT_SME
1	0	0	SUBHNB	FEAT_SVE2 || FEAT_SME
1	0	1	SUBHNT	FEAT_SVE2 || FEAT_SME
1	1	0	RSUBHNB	FEAT_SVE2 || FEAT_SME
1	1	1	RSUBHNT	FEAT_SVE2 || FEAT_SME

SVE2 Histogram (Segment) and Lookup Table

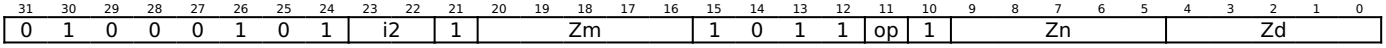
The encodings in this section are decoded from SVE encodings.



Decode fields op2	op0	op1	Instruction details	Condition
0	0	001	UNALLOCATED	-
0	0	011	LUTI6 (8-bit)	FEAT_SVE2p3
1	0	0x1	UNALLOCATED	-
	1	001	LUTI4 (8-bit and 16-bit) — Byte, single register table	(FEAT_SVE2 && FEAT_LUT) || (FEAT_SME2 && FEAT_LUT)
	1	011	LUTI6 (16-bit)	FEAT_SVE2p3 || FEAT_SME2p3
		000	HISTSEG	FEAT_SVE2
		100	LUTI2 (8-bit and 16-bit) — Byte	(FEAT_SVE2 && FEAT_LUT) || (FEAT_SME2 && FEAT_LUT)
		1x1	SVE2 lookup table with 4-bit indices and 16-bit element size	-
		x10	LUTI2 (8-bit and 16-bit) — Halfword	(FEAT_SVE2 && FEAT_LUT) || (FEAT_SME2 && FEAT_LUT)

SVE2 lookup table with 4-bit indices and 16-bit element size

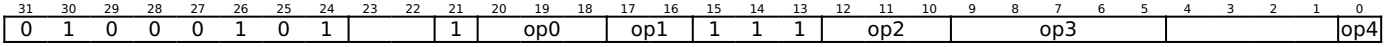
The encodings in this section are decoded from SVE2 Histogram (Segment) and Lookup Table.



Decode fields op	Instruction Details	Condition
0	LUTI4 (8-bit and 16-bit) — Halfword, two register table	(FEAT_SVE2 && FEAT_LUT) || (FEAT_SME2 && FEAT_LUT)
1	LUTI4 (8-bit and 16-bit) — Halfword, single register table	(FEAT_SVE2 && FEAT_LUT) || (FEAT_SME2 && FEAT_LUT)

SVE2 Crypto Extensions

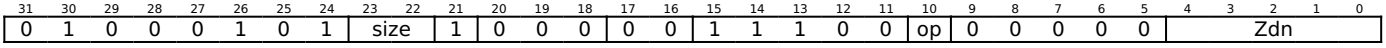
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	op4	Instruction details
000	00	00x	00000		SVE2 crypto unary operations
000	00	00x	!= 00000		UNALLOCATED
000	01	00x			UNALLOCATED
000	1x	00x			SVE2 crypto destructive binary operations
xx0	1x	01x			SVE2 multi-vector AES single round (two registers)
xx1	1x	01x			SVE2 multi-vector AES single round (four registers)
		10x			SVE2 crypto constructive binary operations
		110		0	SVE2 Multi-vector polynomial multiply long
		111		0	SVE2 Multi-vector polynomial multiply long and accumulate vectors
		11x		1	UNALLOCATED
	!= 1x	01x			UNALLOCATED
!= 000		00x			UNALLOCATED

SVE2 crypto unary operations

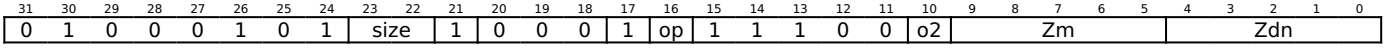
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields		Instruction Details	Condition
size	op		
00	0	AESMC	FEAT_SVE_AES
00	1	AESIMC	FEAT_SVE_AES
!= 00		UNALLOCATED	-

SVE2 crypto destructive binary operations

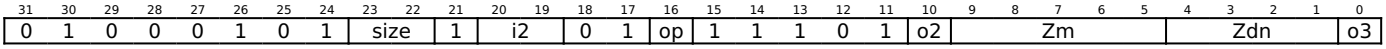
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields			Instruction Details	Condition
size	op	o2		
00	0	0	AESE (vectors)	FEAT_SVE_AES
00	0	1	AESD (vectors)	FEAT_SVE_AES
00	1	0	SM4E	FEAT_SVE_SM4
00	1	1	UNALLOCATED	-
!= 00			UNALLOCATED	-

SVE2 multi-vector AES single round (two registers)

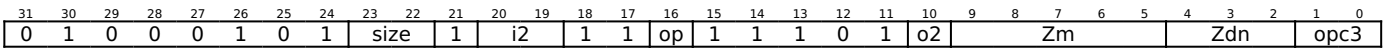
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields				Instruction Details	Condition
size	op	o2	o3		
00			1	UNALLOCATED	-
00	0	0	0	AESE (indexed)	FEAT_SVE_AES2
00	0	1	0	AESD (indexed)	FEAT_SVE_AES2
00	1	0	0	AESEMC	FEAT_SVE_AES2
00	1	1	0	AESDIMC	FEAT_SVE_AES2
!= 00				UNALLOCATED	-

SVE2 multi-vector AES single round (four registers)

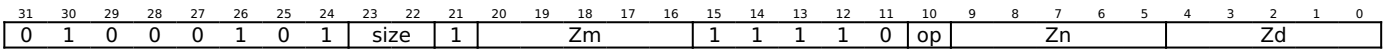
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields				Instruction Details	Condition
size	op	o2	opc3		
00			!= 00	UNALLOCATED	-
00	0	0	00	AESE (indexed)	FEAT_SVE_AES2
00	0	1	00	AESD (indexed)	FEAT_SVE_AES2
00	1	0	00	AESEMC	FEAT_SVE_AES2
00	1	1	00	AESDIMC	FEAT_SVE_AES2
!= 00				UNALLOCATED	-

SVE2 crypto constructive binary operations

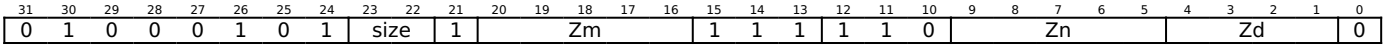
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields size	op	Instruction Details	Condition
00	0	SM4EKEY	FEAT_SVE_SM4
00	1	RAX1	FEAT_SVE_SHA3
!= 00		UNALLOCATED	-

SVE2 Multi-vector polynomial multiply long

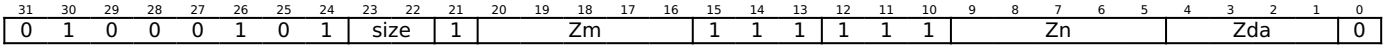
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields size	Instruction Details	Condition
00	PMULL	FEAT_SVE_AES2
!= 00	UNALLOCATED	-

SVE2 Multi-vector polynomial multiply long and accumulate vectors

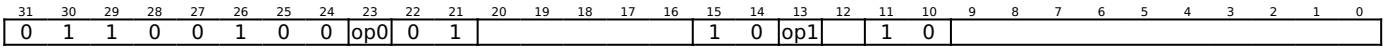
The encodings in this section are decoded from SVE2 Crypto Extensions.



Decode fields size	Instruction Details	Condition
00	PMLAL	FEAT_SVE_AES2
!= 00	UNALLOCATED	-

SVE2 FP8 widening multiply-add

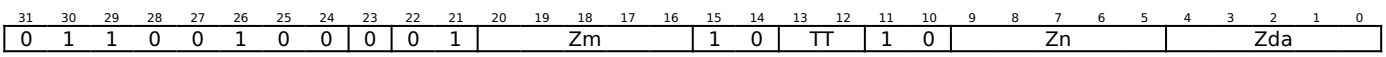
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
0		SVE2 FP8 multiply-add long long
1	0	SVE2 FP8 multiply-add long
1	1	UNALLOCATED

SVE2 FP8 multiply-add long long

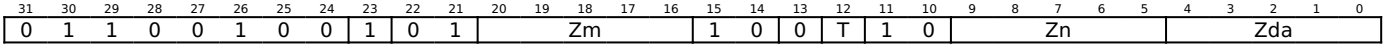
The encodings in this section are decoded from SVE2 FP8 widening multiply-add.



Decode fields TT	Instruction Details	Condition
00	FMLALLBB (vectors)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
01	FMLALLBT (vectors)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
10	FMLALLTB (vectors)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
11	FMLALLTT (vectors)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)

SVE2 FP8 multiply-add long

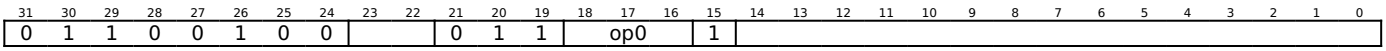
The encodings in this section are decoded from SVE2 FP8 widening multiply-add.



Decode fields T	Instruction Details	Condition
0	FMLALB (vectors, FP8 to FP16)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
1	FMLALT (vectors, FP8 to FP16)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)

SVE2 floating-point unary operations - zeroing predicated

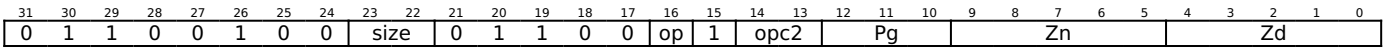
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00x	Floating-point round to integral value (predicated)
010	Floating-point convert (predicated)
011	Floating-point square root (predicated)
10x	Floating-point round and convert from integer (predicated)
11x	Floating-point log and convert to integer

Floating-point round to integral value (predicated)

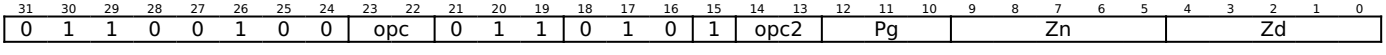
The encodings in this section are decoded from SVE2 floating-point unary operations - zeroing predicated.



Decode fields op	opc2	Instruction Details	Condition
0	00	FRINT<r> — Nearest with ties to even, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	01	FRINT<r> — Toward plus infinity, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	10	FRINT<r> — Toward minus infinity, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
0	11	FRINT<r> — Toward zero, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	00	FRINT<r> — Nearest with ties to away, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	01	UNALLOCATED	-
1	10	FRINT<r> — Current mode signaling inexact, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
1	11	FRINT<r> — Current mode, zeroing	FEAT_SVE2p2 || FEAT_SME2p2

Floating-point convert (predicated)

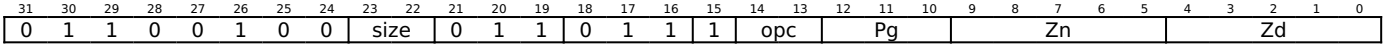
The encodings in this section are decoded from SVE2 floating-point unary operations - zeroing predicated.



Decode fields opc	opc2	Instruction Details	Condition
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTX	FEAT_SVE2p2 || FEAT_SME2p2
01		UNALLOCATED	-
10	00	FCVT — Single-precision to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	01	FCVT — Half-precision to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	10	BFCVT	FEAT_SVE2p2 || FEAT_SME2p2
11	00	FCVT — Double-precision to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	01	FCVT — Half-precision to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	10	FCVT — Double-precision to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	11	FCVT — Single-precision to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2

Floating-point square root (predicated)

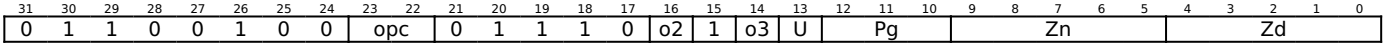
The encodings in this section are decoded from SVE2 floating-point unary operations - zeroing predicated.



Decode fields opc	Instruction Details	Condition
00	FRECPX	FEAT_SVE2p2 || FEAT_SME2p2
01	FSQRT	FEAT_SVE2p2 || FEAT_SME2p2
1x	UNALLOCATED	-

Floating-point round and convert from integer (predicated)

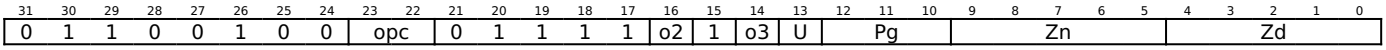
The encodings in this section are decoded from SVE2 floating-point unary operations - zeroing predicated.



Decode fields				Instruction Details	Condition
opc	o2	o3	U		
00	0		0	FRINT32Z	FEAT_SVE2p2 || FEAT_SME2p2
00	0		1	FRINT32X	FEAT_SVE2p2 || FEAT_SME2p2
00	1		0	FRINT64Z	FEAT_SVE2p2 || FEAT_SME2p2
00	1		1	FRINT64X	FEAT_SVE2p2 || FEAT_SME2p2
01	0	0		UNALLOCATED	-
01	0	1	0	SCVTF (predicated) — 16-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	0	1	1	UCVTF (predicated) — 16-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	0	0	SCVTF (predicated) — 32-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	0	1	UCVTF (predicated) — 32-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	1	0	SCVTF (predicated) — 64-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	1	1	UCVTF (predicated) — 64-bit to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	0			UNALLOCATED	-
10	1	0	0	SCVTF (predicated) — 32-bit to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	1	0	1	UCVTF (predicated) — 32-bit to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	1	1		UNALLOCATED	-
11	0	0	0	SCVTF (predicated) — 32-bit to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	0	0	1	UCVTF (predicated) — 32-bit to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	0	1		UNALLOCATED	-
11	1	0	0	SCVTF (predicated) — 64-bit to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	0	1	UCVTF (predicated) — 64-bit to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	1	0	SCVTF (predicated) — 64-bit to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	1	1	UCVTF (predicated) — 64-bit to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2

Floating-point log and convert to integer

The encodings in this section are decoded from SVE2 floating-point unary operations - zeroing predicated.



Decode fields				Instruction Details	Condition
opc	o2	o3	U		
00	0			FLOGB	FEAT_SVE2p2 || FEAT_SME2p2
00	1			UNALLOCATED	-
01	0	0		UNALLOCATED	-
01	0	1	0	FCVTZS — Half-precision to 16-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	0	1	1	FCVTZU — Half-precision to 16-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	0	0	FCVTZS — Half-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	0	1	FCVTZU — Half-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	1	0	FCVTZS — Half-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
01	1	1	1	FCVTZU — Half-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	0			UNALLOCATED	-
10	1	0	0	FCVTZS — Single-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	1	0	1	FCVTZU — Single-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	1	1		UNALLOCATED	-
11	0	0	0	FCVTZS — Double-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	0	0	1	FCVTZU — Double-precision to 32-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	0	1		UNALLOCATED	-
11	1	0	0	FCVTZS — Single-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	0	1	FCVTZU — Single-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	1	0	FCVTZS — Double-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	1	1	1	FCVTZU — Double-precision to 64-bit, zeroing	FEAT_SVE2p2 || FEAT_SME2p2

SVE floating-point widening multiply-add - indexed

The encodings in this section are decoded from SVE encodings.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	op0		1						0	1	op1	0												

Decode fields		Instruction details
op0	op1	
0	0	SVE BFloat16 floating-point dot product (indexed)
0	1	UNALLOCATED
1		SVE floating-point multiply-add long (indexed)

SVE BFloat16 floating-point dot product (indexed)

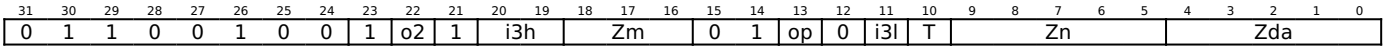
The encodings in this section are decoded from SVE floating-point widening multiply-add - indexed.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	0	0	0	op	1	i2			Zm		0	1	0	0	opc2			Zn					Zda			

Decode fields		Instruction Details	Condition
op	opc2		
0	x1	FDOT (2-way, indexed, FP8 to FP16)	FEAT_SSVE_FP8DOT2 || (FEAT_SVE2 && FEAT_FP8DOT2)
0	00	FDOT (2-way, indexed, FP16 to FP32)	FEAT_SME2 || FEAT_SVE2p1
0	10	UNALLOCATED	-
1	00	BFDOT (indexed)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	01	FDOT (4-way, indexed)	FEAT_SSVE_FP8DOT4 || (FEAT_SVE2 && FEAT_FP8DOT4)
1	1x	UNALLOCATED	-

SVE floating-point multiply-add long (indexed)

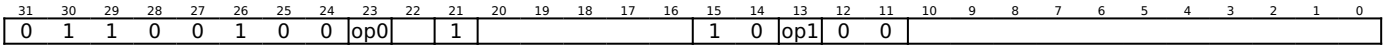
The encodings in this section are decoded from SVE floating-point widening multiply-add - indexed.



Decode fields			Instruction Details	Condition
o2	op	T		
0	0	0	FMLALB (indexed, FP16 to FP32)	FEAT_SVE2 || FEAT_SME
0	0	1	FMLALT (indexed, FP16 to FP32)	FEAT_SVE2 || FEAT_SME
0	1	0	FMLSLB (indexed)	FEAT_SVE2 || FEAT_SME
0	1	1	FMLSLT (indexed)	FEAT_SVE2 || FEAT_SME
1	0	0	BFMLALB (indexed)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	0	1	BFMLALT (indexed)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	1	0	BFMLSLB (indexed)	FEAT_SME2 || FEAT_SVE2p1
1	1	1	BFMLSLT (indexed)	FEAT_SME2 || FEAT_SVE2p1

SVE floating-point widening multiply-add

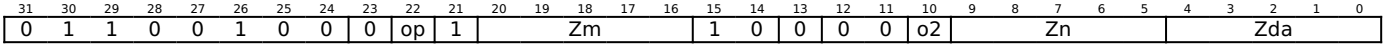
The encodings in this section are decoded from SVE encodings.



Decode fields		Instruction details
op0	op1	
0	0	SVE BFloat16 floating-point dot product
0	1	UNALLOCATED
1		SVE floating-point multiply-add long

SVE BFloat16 floating-point dot product

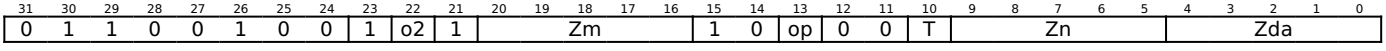
The encodings in this section are decoded from SVE floating-point widening multiply-add.



Decode fields		Instruction Details	Condition
op	o2		
0	0	FDOT (2-way, vectors, FP16 to FP32)	FEAT_SME2 || FEAT_SVE2p1
0	1	FDOT (2-way, vectors, FP8 to FP16)	FEAT_SSVE_FP8DOT2 || (FEAT_SVE2 && FEAT_FP8DOT2)
1	0	BFDOT (vectors)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	1	FDOT (4-way, vectors)	FEAT_SSVE_FP8DOT4 || (FEAT_SVE2 && FEAT_FP8DOT4)

SVE floating-point multiply-add long

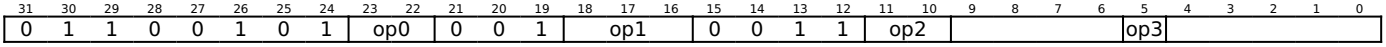
The encodings in this section are decoded from SVE floating-point widening multiply-add.



Decode fields			Instruction Details	Condition
o2	op	T		
0	0	0	FMLALB (vectors, FP16 to FP32)	FEAT_SVE2 || FEAT_SME
0	0	1	FMLALT (vectors, FP16 to FP32)	FEAT_SVE2 || FEAT_SME
0	1	0	FMLSLB (vectors)	FEAT_SVE2 || FEAT_SME
0	1	1	FMLSLT (vectors)	FEAT_SVE2 || FEAT_SME
1	0	0	BFMLALB (vectors)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	0	1	BFMLALT (vectors)	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
1	1	0	BFMLSLB (vectors)	FEAT_SME2 || FEAT_SVE2p1
1	1	1	BFMLSLT (vectors)	FEAT_SME2 || FEAT_SVE2p1

SVE floating-point unary operations - unpredicated

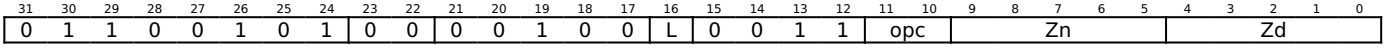
The encodings in this section are decoded from SVE encodings.



Decode fields				Instruction details
op0	op1	op2	op3	
00	00x			SVE2 FP8 upconverts
00	010		0	SVE2 FP8 downconverts
00	010		1	UNALLOCATED
00	011			UNALLOCATED
	100			SVE2 Int to FP upconverts
	101	0x	0	SVE2 FP to Int downconverts
	101	0x	1	UNALLOCATED
	101	1x		UNALLOCATED
	11x	00		SVE floating-point reciprocal estimate (unpredicated)
	11x	!= 00		UNALLOCATED
!= 00	0xx			UNALLOCATED

SVE2 FP8 upconverts

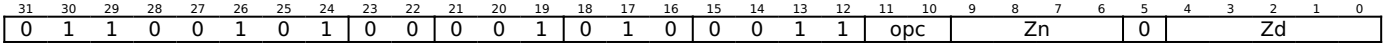
The encodings in this section are decoded from SVE floating-point unary operations - unpredicated.



Decode fields		Instruction Details	Condition
L	opc		
0	00	F1CVT, F2CVT — F1CVT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
0	01	F1CVT, F2CVT — F2CVT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
0	10	BF1CVT, BF2CVT — BF1CVT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
0	11	BF1CVT, BF2CVT — BF2CVT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
1	00	F1CVTLT, F2CVTLT — F1CVTLT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
1	01	F1CVTLT, F2CVTLT — F2CVTLT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
1	10	BF1CVTLT, BF2CVTLT — BF1CVTLT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
1	11	BF1CVTLT, BF2CVTLT — BF2CVTLT	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)

SVE2 FP8 downconverts

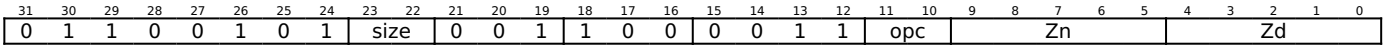
The encodings in this section are decoded from SVE floating-point unary operations - unpredicated.



Decode fields opc	Instruction Details	Condition
00	FCVTN	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
01	FCVTNB	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
10	BFCVTN	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)
11	FCVTNT (unpredicated)	(FEAT_SVE2 && FEAT_FP8) || (FEAT_SME2 && FEAT_FP8)

SVE2 Int to FP upconverts

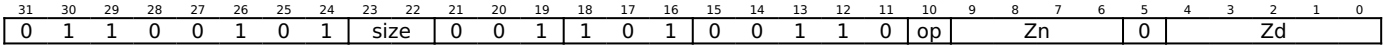
The encodings in this section are decoded from SVE floating-point unary operations - unpredicated.



Decode fields opc	Instruction Details	Condition
00	SCVTF (bottom, unpredicated)	FEAT_SVE2p3 || FEAT_SME2p3
01	UCVTF (bottom, unpredicated)	FEAT_SVE2p3 || FEAT_SME2p3
10	SCVTFLT	FEAT_SVE2p3 || FEAT_SME2p3
11	UCVTFLT	FEAT_SVE2p3 || FEAT_SME2p3

SVE2 FP to Int downconverts

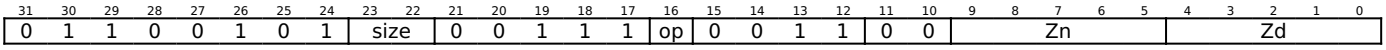
The encodings in this section are decoded from SVE floating-point unary operations - unpredicated.



Decode fields op	Instruction Details	Condition
0	FCVTZSN	FEAT_SVE2p3 || FEAT_SME2p3
1	FCVTZUN	FEAT_SVE2p3 || FEAT_SME2p3

SVE floating-point reciprocal estimate (unpredicated)

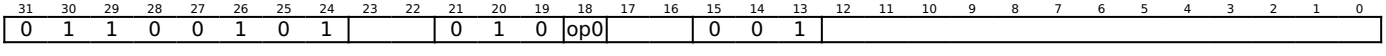
The encodings in this section are decoded from SVE floating-point unary operations - unpredicated.



Decode fields op	Instruction Details	Condition
0	FRECPE	FEAT_SVE || FEAT_SME
1	FRSQRTE	FEAT_SVE || FEAT_SME

SVE floating-point compare - with zero

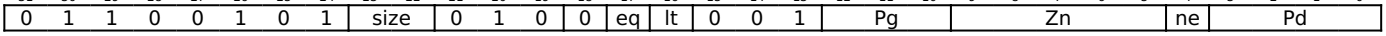
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
0	SVE floating-point compare with zero
1	UNALLOCATED

SVE floating-point compare with zero

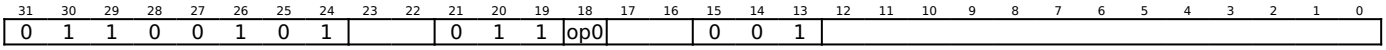
The encodings in this section are decoded from SVE floating-point compare - with zero.



Decode fields			Instruction Details	Condition
eq	lt	ne		
0	0	0	FCM<cc> (zero) — Greater than or equal	FEAT_SVE || FEAT_SME
0	0	1	FCM<cc> (zero) — Greater than	FEAT_SVE || FEAT_SME
0	1	0	FCM<cc> (zero) — Less than	FEAT_SVE || FEAT_SME
0	1	1	FCM<cc> (zero) — Less than or equal	FEAT_SVE || FEAT_SME
1		1	UNALLOCATED	-
1	0	0	FCM<cc> (zero) — Equal	FEAT_SVE || FEAT_SME
1	1	0	FCM<cc> (zero) — Not equal	FEAT_SVE || FEAT_SME

SVE floating-point accumulating reduction

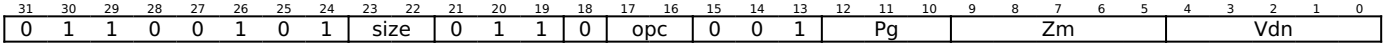
The encodings in this section are decoded from SVE encodings.



Decode fields	Instruction details
op0	
0	SVE floating-point serial reduction (predicated)
1	UNALLOCATED

SVE floating-point serial reduction (predicated)

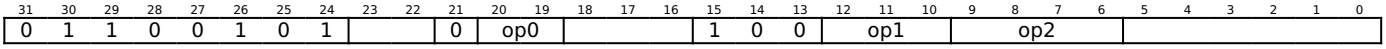
The encodings in this section are decoded from SVE floating-point accumulating reduction.



Decode fields	Instruction Details	Condition
opc		
00	FADDA	FEAT_SVE
!= 00	UNALLOCATED	-

SVE floating-point arithmetic - predicated

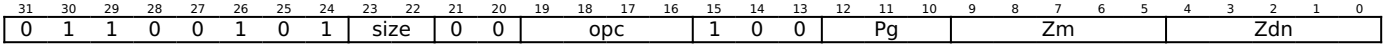
The encodings in this section are decoded from SVE encodings.



Decode fields			Instruction details	Condition
op0	op1	op2		
0x			SVE floating-point arithmetic (predicated)	-
10	000		FTMAD	FEAT_SVE
10	!= 000		UNALLOCATED	-
11		0000	SVE floating-point arithmetic with immediate (predicated)	-
11		!= 0000	UNALLOCATED	-

SVE floating-point arithmetic (predicated)

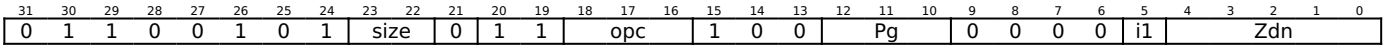
The encodings in this section are decoded from SVE floating-point arithmetic - predicated.



Decode fields size	Decode fields opc	Instruction Details	Condition
	0011	FSUBR (vectors)	FEAT_SVE || FEAT_SME
	1000	FABD	FEAT_SVE || FEAT_SME
	1010	FMULX	FEAT_SVE || FEAT_SME
	1011	UNALLOCATED	-
	1100	FDIVR	FEAT_SVE || FEAT_SME
	1101	FDIV	FEAT_SVE || FEAT_SME
	1110	FAMAX	(FEAT_SVE2 && FEAT_FAMINMAX) || (FEAT_SME2 && FEAT_FAMINMAX)
	1111	FAMIN	(FEAT_SVE2 && FEAT_FAMINMAX) || (FEAT_SME2 && FEAT_FAMINMAX)
00	0000	BFADD (predicated)	FEAT_SVE_B16B16
00	0001	BFSUB (predicated)	FEAT_SVE_B16B16
00	0010	BFMUL (vectors, predicated)	FEAT_SVE_B16B16
00	0100	BFMAXNM	FEAT_SVE_B16B16
00	0101	BFMINNM	FEAT_SVE_B16B16
00	0110	BFMAX	FEAT_SVE_B16B16
00	0111	BFMIN	FEAT_SVE_B16B16
00	1001	BFSCALE	FEAT_SVE_BFSCALE
!= 00	0000	FADD (vectors, predicated)	FEAT_SVE || FEAT_SME
!= 00	0001	FSUB (vectors, predicated)	FEAT_SVE || FEAT_SME
!= 00	0010	FMUL (vectors, predicated)	FEAT_SVE || FEAT_SME
!= 00	0100	FMAXNM (vectors)	FEAT_SVE || FEAT_SME
!= 00	0101	FMINNM (vectors)	FEAT_SVE || FEAT_SME
!= 00	0110	FMAX (vectors)	FEAT_SVE || FEAT_SME
!= 00	0111	FMIN (vectors)	FEAT_SVE || FEAT_SME
!= 00	1001	FSCALE	FEAT_SVE || FEAT_SME

SVE floating-point arithmetic with immediate (predicated)

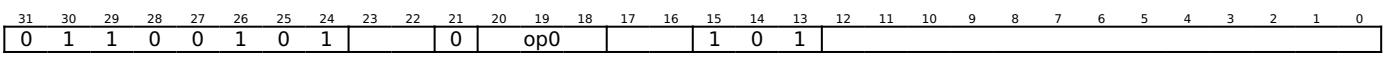
The encodings in this section are decoded from SVE floating-point arithmetic - predicated.



Decode fields opc	Instruction Details	Condition
000	FADD (immediate)	FEAT_SVE || FEAT_SME
001	FSUB (immediate)	FEAT_SVE || FEAT_SME
010	FMUL (immediate)	FEAT_SVE || FEAT_SME
011	FSUBR (immediate)	FEAT_SVE || FEAT_SME
100	FMAXNM (immediate)	FEAT_SVE || FEAT_SME
101	FMINNM (immediate)	FEAT_SVE || FEAT_SME
110	FMAX (immediate)	FEAT_SVE || FEAT_SME
111	FMIN (immediate)	FEAT_SVE || FEAT_SME

SVE floating-point unary operations - merging predicated

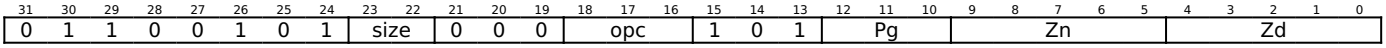
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00x	SVE floating-point round to integral value
010	SVE floating-point convert precision
011	SVE floating-point unary operations
10x	SVE integer convert to floating-point
11x	SVE floating-point convert to integer

SVE floating-point round to integral value

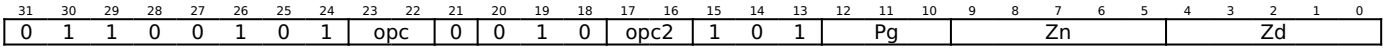
The encodings in this section are decoded from SVE floating-point unary operations - merging predicated.



Decode fields opc	Instruction Details	Condition
000	FRINT<r> — Nearest with ties to even, merging	FEAT_SVE || FEAT_SME
001	FRINT<r> — Toward plus infinity, merging	FEAT_SVE || FEAT_SME
010	FRINT<r> — Toward minus infinity, merging	FEAT_SVE || FEAT_SME
011	FRINT<r> — Toward zero, merging	FEAT_SVE || FEAT_SME
100	FRINT<r> — Nearest with ties to away, merging	FEAT_SVE || FEAT_SME
101	UNALLOCATED	-
110	FRINT<r> — Current mode signaling inexact, merging	FEAT_SVE || FEAT_SME
111	FRINT<r> — Current mode, merging	FEAT_SVE || FEAT_SME

SVE floating-point convert precision

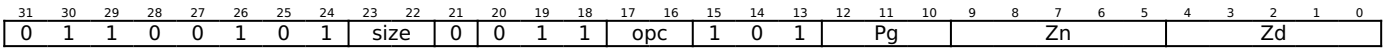
The encodings in this section are decoded from SVE floating-point unary operations - merging predicated.



Decode fields opc	opc2	Instruction Details	Condition
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTX	FEAT_SVE2 || FEAT_SME
01		UNALLOCATED	-
10	00	FCVT — Single-precision to half-precision, merging	FEAT_SVE || FEAT_SME
10	01	FCVT — Half-precision to single-precision, merging	FEAT_SVE || FEAT_SME
10	10	BFCVT	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
11	00	FCVT — Double-precision to half-precision, merging	FEAT_SVE || FEAT_SME
11	01	FCVT — Half-precision to double-precision, merging	FEAT_SVE || FEAT_SME
11	10	FCVT — Double-precision to single-precision, merging	FEAT_SVE || FEAT_SME
11	11	FCVT — Single-precision to double-precision, merging	FEAT_SVE || FEAT_SME

SVE floating-point unary operations

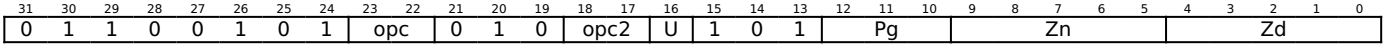
The encodings in this section are decoded from SVE floating-point unary operations - merging predicated.



Decode fields	Instruction Details	Condition
opc		
00	FRECPX	FEAT_SVE || FEAT_SME
01	FSQRT	FEAT_SVE || FEAT_SME
1x	UNALLOCATED	-

SVE integer convert to floating-point

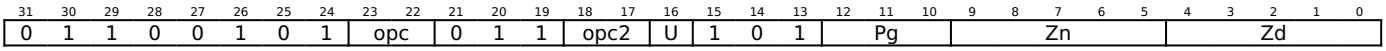
The encodings in this section are decoded from SVE floating-point unary operations - merging predicated.



Decode fields			Instruction Details	Condition
opc	opc2	U		
00	0x	0	FRINT32Z	FEAT_SVE2p2 || FEAT_SME2p2
00	0x	1	FRINT32X	FEAT_SVE2p2 || FEAT_SME2p2
00	1x	0	FRINT64Z	FEAT_SVE2p2 || FEAT_SME2p2
00	1x	1	FRINT64X	FEAT_SVE2p2 || FEAT_SME2p2
01	00		UNALLOCATED	-
01	01	0	SCVTF (predicated) — 16-bit to half-precision, merging	FEAT_SVE || FEAT_SME
01	01	1	UCVTF (predicated) — 16-bit to half-precision, merging	FEAT_SVE || FEAT_SME
01	10	0	SCVTF (predicated) — 32-bit to half-precision, merging	FEAT_SVE || FEAT_SME
01	10	1	UCVTF (predicated) — 32-bit to half-precision, merging	FEAT_SVE || FEAT_SME
01	11	0	SCVTF (predicated) — 64-bit to half-precision, merging	FEAT_SVE || FEAT_SME
01	11	1	UCVTF (predicated) — 64-bit to half-precision, merging	FEAT_SVE || FEAT_SME
10	10	0	SCVTF (predicated) — 32-bit to single-precision, merging	FEAT_SVE || FEAT_SME
10	10	1	UCVTF (predicated) — 32-bit to single-precision, merging	FEAT_SVE || FEAT_SME
10	!= 10		UNALLOCATED	-
11	00	0	SCVTF (predicated) — 32-bit to double-precision, merging	FEAT_SVE || FEAT_SME
11	00	1	UCVTF (predicated) — 32-bit to double-precision, merging	FEAT_SVE || FEAT_SME
11	01		UNALLOCATED	-
11	10	0	SCVTF (predicated) — 64-bit to single-precision, merging	FEAT_SVE || FEAT_SME
11	10	1	UCVTF (predicated) — 64-bit to single-precision, merging	FEAT_SVE || FEAT_SME
11	11	0	SCVTF (predicated) — 64-bit to double-precision, merging	FEAT_SVE || FEAT_SME
11	11	1	UCVTF (predicated) — 64-bit to double-precision, merging	FEAT_SVE || FEAT_SME

SVE floating-point convert to integer

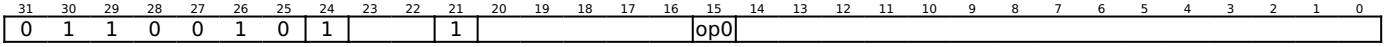
The encodings in this section are decoded from SVE floating-point unary operations - merging predicated.



Decode fields			Instruction Details	Condition
opc	opc2	U		
00		0	FLOGB	FEAT_SVE2 || FEAT_SME
00		1	UNALLOCATED	-
01	00		UNALLOCATED	-
01	01	0	FCVTZS — Half-precision to 16-bit, merging	FEAT_SVE || FEAT_SME
01	01	1	FCVTZU — Half-precision to 16-bit, merging	FEAT_SVE || FEAT_SME
01	10	0	FCVTZS — Half-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
01	10	1	FCVTZU — Half-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
01	11	0	FCVTZS — Half-precision to 64-bit, merging	FEAT_SVE || FEAT_SME
01	11	1	FCVTZU — Half-precision to 64-bit, merging	FEAT_SVE || FEAT_SME
10	10	0	FCVTZS — Single-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
10	10	1	FCVTZU — Single-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
10	!= 10		UNALLOCATED	-
11	00	0	FCVTZS — Double-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
11	00	1	FCVTZU — Double-precision to 32-bit, merging	FEAT_SVE || FEAT_SME
11	01		UNALLOCATED	-
11	10	0	FCVTZS — Single-precision to 64-bit, merging	FEAT_SVE || FEAT_SME
11	10	1	FCVTZU — Single-precision to 64-bit, merging	FEAT_SVE || FEAT_SME
11	11	0	FCVTZS — Double-precision to 64-bit, merging	FEAT_SVE || FEAT_SME
11	11	1	FCVTZU — Double-precision to 64-bit, merging	FEAT_SVE || FEAT_SME

SVE floating-point multiply-add

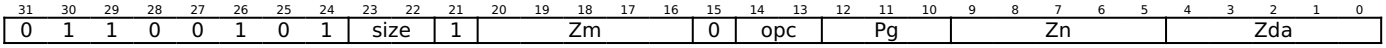
The encodings in this section are decoded from SVE encodings.



Decode fields	Instruction details
op0	
0	SVE floating-point multiply-accumulate writing addend
1	SVE floating-point multiply-accumulate writing multiplicand

SVE floating-point multiply-accumulate writing addend

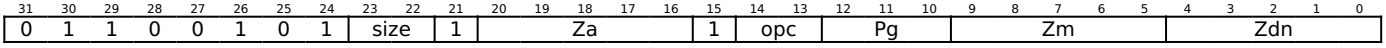
The encodings in this section are decoded from SVE floating-point multiply-add.



Decode fields		Instruction Details	Condition
size	opc		
00	00	BFMLA (vectors)	FEAT_SVE_B16B16
00	01	BFMLS (vectors)	FEAT_SVE_B16B16
00	1x	UNALLOCATED	-
!= 00	00	FMLA (vectors)	FEAT_SVE || FEAT_SME
!= 00	01	FMLS (vectors)	FEAT_SVE || FEAT_SME
!= 00	10	FNMLA	FEAT_SVE || FEAT_SME
!= 00	11	FNMLS	FEAT_SVE || FEAT_SME

SVE floating-point multiply-accumulate writing multiplicand

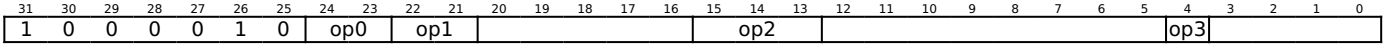
The encodings in this section are decoded from SVE floating-point multiply-add.



Decode fields opc	Instruction Details	Condition
00	FMAD	FEAT_SVE || FEAT_SME
01	FMSB	FEAT_SVE || FEAT_SME
10	FNMAD	FEAT_SVE || FEAT_SME
11	FNMSB	FEAT_SVE || FEAT_SME

SVE Memory - 32-bit Gather and Unsized Contiguous

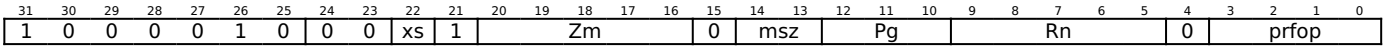
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	Instruction details	Condition
00	x1	0xx	0	SVE 32-bit gather prefetch (scalar plus 32-bit scaled offsets)	-
00	x1	0xx	1	UNALLOCATED	-
01	x1	0xx		SVE 32-bit gather load halfwords (scalar plus 32-bit scaled offsets)	-
10	x1	0xx		SVE 32-bit gather load words (scalar plus 32-bit scaled offsets)	-
11	00	001	0	UNALLOCATED	-
11	00	011		LDAPRO	FEAT_SVE_LRCPC
11	01	011	1	UNALLOCATED	-
11	01	0x1	0	UNALLOCATED	-
11	0x	000	0	LDR (predicate)	FEAT_SVE || FEAT_SME
11	0x	00x	1	UNALLOCATED	-
11	0x	010		LDR (vector)	FEAT_SVE || FEAT_SME
11	1x	0xx	0	SVE contiguous prefetch (scalar plus immediate)	-
11	1x	0xx	1	UNALLOCATED	-
	00	10x		SVE2 32-bit gather non-temporal load (vector plus scalar)	-
	00	110	0	SVE contiguous prefetch (scalar plus scalar)	-
	00	111	0	SVE 32-bit gather prefetch (vector plus immediate)	-
	00	11x	1	UNALLOCATED	-
	01	1xx		SVE 32-bit gather load (vector plus immediate)	-
	1x	1xx		SVE load and broadcast element	-
!= 11	x0	0xx		SVE 32-bit gather load (scalar plus 32-bit unscaled offsets)	-

SVE 32-bit gather prefetch (scalar plus 32-bit scaled offsets)

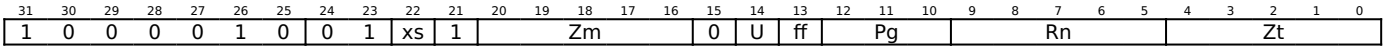
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields msz	Instruction Details	Condition
00	PRFB (scalar plus vector)	FEAT_SVE
01	PRFH (scalar plus vector)	FEAT_SVE
10	PRFW (scalar plus vector)	FEAT_SVE
11	PRFD (scalar plus vector)	FEAT_SVE

SVE 32-bit gather load halfwords (scalar plus 32-bit scaled offsets)

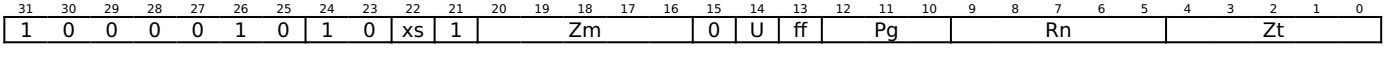
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields U	ff	Instruction Details	Condition
0	0	LD1SH (scalar plus vector)	FEAT_SVE
0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
1	0	LD1H (scalar plus vector)	FEAT_SVE
1	1	LDFF1H (scalar plus vector)	FEAT_SVE

SVE 32-bit gather load words (scalar plus 32-bit scaled offsets)

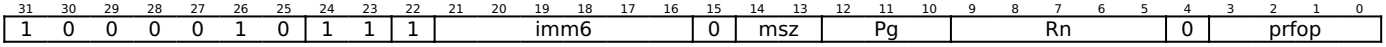
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields U	ff	Instruction Details	Condition
0		UNALLOCATED	-
1	0	LD1W (scalar plus vector)	FEAT_SVE
1	1	LDFF1W (scalar plus vector)	FEAT_SVE

SVE contiguous prefetch (scalar plus immediate)

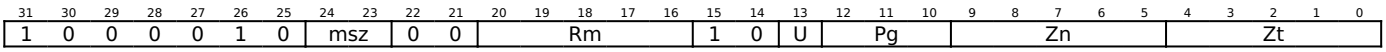
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields msz	Instruction Details	Condition
00	PRFB (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	PRFH (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	PRFW (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	PRFD (scalar plus immediate)	FEAT_SVE || FEAT_SME

SVE2 32-bit gather non-temporal load (vector plus scalar)

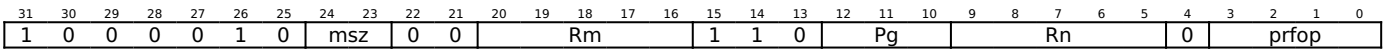
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields msz	U	Instruction Details	Condition
00	0	LDNT1SB	FEAT_SVE2
00	1	LDNT1B (vector plus scalar)	FEAT_SVE2
01	0	LDNT1SH	FEAT_SVE2
01	1	LDNT1H (vector plus scalar)	FEAT_SVE2
10	0	UNALLOCATED	-
10	1	LDNT1W (vector plus scalar)	FEAT_SVE2
11		UNALLOCATED	-

SVE contiguous prefetch (scalar plus scalar)

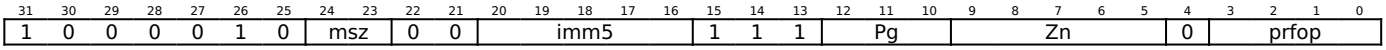
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields msz	Decode fields Rm	Instruction Details	Condition
	11111	UNALLOCATED	-
00	!= 11111	PRFB (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	!= 11111	PRFH (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	!= 11111	PRFW (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	!= 11111	PRFD (scalar plus scalar)	FEAT_SVE || FEAT_SME

SVE 32-bit gather prefetch (vector plus immediate)

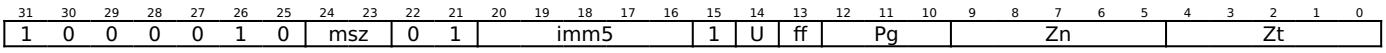
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields msz	Instruction Details	Condition
00	PRFB (vector plus immediate)	FEAT_SVE
01	PRFH (vector plus immediate)	FEAT_SVE
10	PRFW (vector plus immediate)	FEAT_SVE
11	PRFD (vector plus immediate)	FEAT_SVE

SVE 32-bit gather load (vector plus immediate)

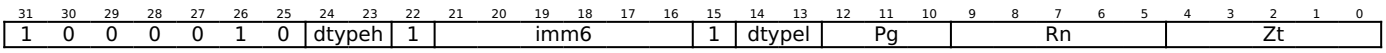
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields msz	Decode fields U	Decode fields ff	Instruction Details	Condition
00	0	0	LD1SB (vector plus immediate)	FEAT_SVE
00	0	1	LDFF1SB (vector plus immediate)	FEAT_SVE
00	1	0	LD1B (vector plus immediate)	FEAT_SVE
00	1	1	LDFF1B (vector plus immediate)	FEAT_SVE
01	0	0	LD1SH (vector plus immediate)	FEAT_SVE
01	0	1	LDFF1SH (vector plus immediate)	FEAT_SVE
01	1	0	LD1H (vector plus immediate)	FEAT_SVE
01	1	1	LDFF1H (vector plus immediate)	FEAT_SVE
10	0		UNALLOCATED	-
10	1	0	LD1W (vector plus immediate)	FEAT_SVE
10	1	1	LDFF1W (vector plus immediate)	FEAT_SVE
11			UNALLOCATED	-

SVE load and broadcast element

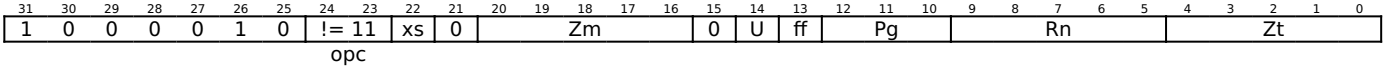
The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.



Decode fields		Instruction Details	Condition
dtypeh	dtypel		
00	00	LD1RB — 8-bit element	FEAT_SVE || FEAT_SME
00	01	LD1RB — 16-bit element	FEAT_SVE || FEAT_SME
00	10	LD1RB — 32-bit element	FEAT_SVE || FEAT_SME
00	11	LD1RB — 64-bit element	FEAT_SVE || FEAT_SME
01	00	LD1RSW	FEAT_SVE || FEAT_SME
01	01	LD1RH — 16-bit element	FEAT_SVE || FEAT_SME
01	10	LD1RH — 32-bit element	FEAT_SVE || FEAT_SME
01	11	LD1RH — 64-bit element	FEAT_SVE || FEAT_SME
10	00	LD1RSH — 64-bit element	FEAT_SVE || FEAT_SME
10	01	LD1RSH — 32-bit element	FEAT_SVE || FEAT_SME
10	10	LD1RW — 32-bit element	FEAT_SVE || FEAT_SME
10	11	LD1RW — 64-bit element	FEAT_SVE || FEAT_SME
11	00	LD1RSB — 64-bit element	FEAT_SVE || FEAT_SME
11	01	LD1RSB — 32-bit element	FEAT_SVE || FEAT_SME
11	10	LD1RSB — 16-bit element	FEAT_SVE || FEAT_SME
11	11	LD1RD	FEAT_SVE || FEAT_SME

SVE 32-bit gather load (scalar plus 32-bit unscaled offsets)

The encodings in this section are decoded from SVE Memory - 32-bit Gather and Unsized Contiguous.

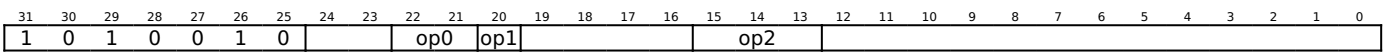


The following constraints also apply to this encoding: opc != '11'

Decode fields			Instruction Details	Condition
opc	U	ff		
00	0	0	LD1SB (scalar plus vector)	FEAT_SVE
00	0	1	LDFF1SB (scalar plus vector)	FEAT_SVE
00	1	0	LD1B (scalar plus vector)	FEAT_SVE
00	1	1	LDFF1B (scalar plus vector)	FEAT_SVE
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0		UNALLOCATED	-
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE

SVE Memory - Contiguous Load

The encodings in this section are decoded from SVE encodings.



Decode fields			Instruction details
op0	op1	op2	
00	0	111	SVE contiguous non-temporal load (scalar plus immediate)
00	1	001	SVE contiguous load (quadwords, scalar plus immediate)
00	1	111	SVE load multiple structures (quadwords, scalar plus immediate)
00		100	SVE contiguous load (quadwords, scalar plus scalar)
00		110	SVE contiguous non-temporal load (scalar plus scalar)
01		100	SVE load multiple structures (quadwords, scalar plus scalar)
1x		100	UNALLOCATED
	0	001	SVE load and broadcast quadword (scalar plus immediate)
	0	101	SVE contiguous load (scalar plus immediate)
	1	101	SVE contiguous non-fault load (scalar plus immediate)
		000	SVE load and broadcast quadword (scalar plus scalar)
		010	SVE contiguous load (scalar plus scalar)
		011	SVE contiguous first-fault load (scalar plus scalar)
!= 00	0	111	SVE load multiple structures (scalar plus immediate)
!= 00	1	001	UNALLOCATED
!= 00	1	111	UNALLOCATED
!= 00		110	SVE load multiple structures (scalar plus scalar)

SVE contiguous non-temporal load (scalar plus immediate)

The encodings in this section are decoded from SVE Memory - Contiguous Load.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	msz		0	0	0	imm4				1	1	1	Pg			Rn						Zt			

Decode fields	Instruction Details	Condition
msz		
00	LDNT1B (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
01	LDNT1H (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
10	LDNT1W (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
11	LDNT1D (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME

SVE contiguous load (quadwords, scalar plus immediate)

The encodings in this section are decoded from SVE Memory - Contiguous Load.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	dtype		0	0	1	imm4				0	0	1	Pg			Rn						Zt			

Decode fields	Instruction Details	Condition
dtype		
0x	UNALLOCATED	-
10	LD1W (scalar plus immediate, single register)	FEAT_SVE2p1
11	LD1D (scalar plus immediate, single register)	FEAT_SVE2p1

SVE load multiple structures (quadwords, scalar plus immediate)

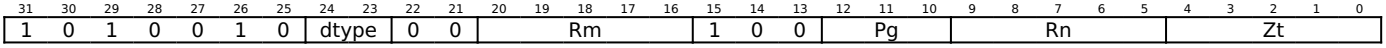
The encodings in this section are decoded from SVE Memory - Contiguous Load.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	0	num		0	0	1	imm4				1	1	1	Pg			Rn						Zt			

Decode fields num	Instruction Details	Condition
00	UNALLOCATED	-
01	LD2Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1
10	LD3Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1
11	LD4Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1

SVE contiguous load (quadwords, scalar plus scalar)

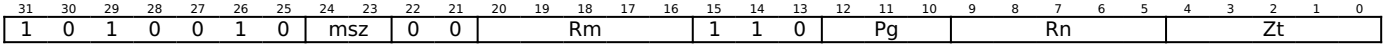
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields dtype	Decode fields Rm	Instruction Details	Condition
0x		UNALLOCATED	-
1x	11111	UNALLOCATED	-
10	!= 11111	LD1W (scalar plus scalar, single register)	FEAT_SVE2p1
11	!= 11111	LD1D (scalar plus scalar, single register)	FEAT_SVE2p1

SVE contiguous non-temporal load (scalar plus scalar)

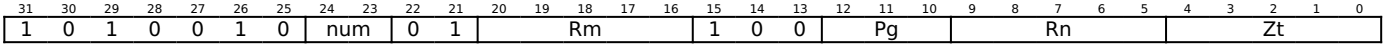
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields msz	Decode fields Rm	Instruction Details	Condition
	11111	UNALLOCATED	-
00	!= 11111	LDNT1B (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
01	!= 11111	LDNT1H (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
10	!= 11111	LDNT1W (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
11	!= 11111	LDNT1D (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME

SVE load multiple structures (quadwords, scalar plus scalar)

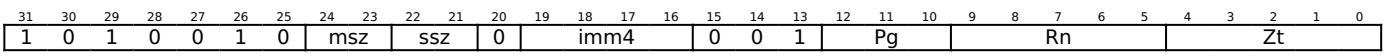
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields num	Decode fields Rm	Instruction Details	Condition
00		UNALLOCATED	-
01	!= 11111	LD2Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
10	!= 11111	LD3Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
11	!= 11111	LD4Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
!= 00	11111	UNALLOCATED	-

SVE load and broadcast quadword (scalar plus immediate)

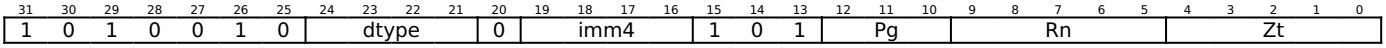
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields msz	ssz	Instruction Details	Condition
	1x	UNALLOCATED	-
00	00	LD1RQB (scalar plus immediate)	FEAT_SVE || FEAT_SME
00	01	LD1ROB (scalar plus immediate)	FEAT_F64MM
01	00	LD1RQH (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	01	LD1ROH (scalar plus immediate)	FEAT_F64MM
10	00	LD1RQW (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	01	LD1ROW (scalar plus immediate)	FEAT_F64MM
11	00	LD1RQD (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	01	LD1ROD (scalar plus immediate)	FEAT_F64MM

SVE contiguous load (scalar plus immediate)

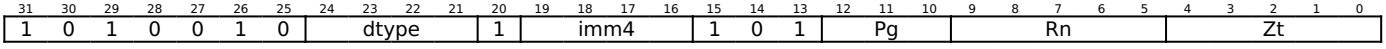
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields dtype	Instruction Details	Condition
0000	LD1B (scalar plus immediate, single register) — 8-bit element	FEAT_SVE || FEAT_SME
0001	LD1B (scalar plus immediate, single register) — 16-bit element	FEAT_SVE || FEAT_SME
0010	LD1B (scalar plus immediate, single register) — 32-bit element	FEAT_SVE || FEAT_SME
0011	LD1B (scalar plus immediate, single register) — 64-bit element	FEAT_SVE || FEAT_SME
0100	LD1SW (scalar plus immediate)	FEAT_SVE || FEAT_SME
0101	LD1H (scalar plus immediate, single register) — 16-bit element	FEAT_SVE || FEAT_SME
0110	LD1H (scalar plus immediate, single register) — 32-bit element	FEAT_SVE || FEAT_SME
0111	LD1H (scalar plus immediate, single register) — 64-bit element	FEAT_SVE || FEAT_SME
1000	LD1SH (scalar plus immediate) — 64-bit element	FEAT_SVE || FEAT_SME
1001	LD1SH (scalar plus immediate) — 32-bit element	FEAT_SVE || FEAT_SME
1010	LD1W (scalar plus immediate, single register) — 32-bit element	FEAT_SVE || FEAT_SME
1011	LD1W (scalar plus immediate, single register) — 64-bit element	FEAT_SVE || FEAT_SME
1100	LD1SB (scalar plus immediate) — 64-bit element	FEAT_SVE || FEAT_SME
1101	LD1SB (scalar plus immediate) — 32-bit element	FEAT_SVE || FEAT_SME
1110	LD1SB (scalar plus immediate) — 16-bit element	FEAT_SVE || FEAT_SME
1111	LD1D (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME

SVE contiguous non-fault load (scalar plus immediate)

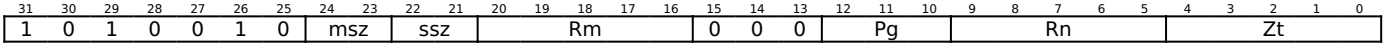
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields dtype	Instruction Details	Condition
0000	LDNF1B — 8-bit element	FEAT_SVE
0001	LDNF1B — 16-bit element	FEAT_SVE
0010	LDNF1B — 32-bit element	FEAT_SVE
0011	LDNF1B — 64-bit element	FEAT_SVE
0100	LDNF1SW	FEAT_SVE
0101	LDNF1H — 16-bit element	FEAT_SVE
0110	LDNF1H — 32-bit element	FEAT_SVE
0111	LDNF1H — 64-bit element	FEAT_SVE
1000	LDNF1SH — 64-bit element	FEAT_SVE
1001	LDNF1SH — 32-bit element	FEAT_SVE
1010	LDNF1W — 32-bit element	FEAT_SVE
1011	LDNF1W — 64-bit element	FEAT_SVE
1100	LDNF1SB — 64-bit element	FEAT_SVE
1101	LDNF1SB — 32-bit element	FEAT_SVE
1110	LDNF1SB — 16-bit element	FEAT_SVE
1111	LDNF1D	FEAT_SVE

SVE load and broadcast quadword (scalar plus scalar)

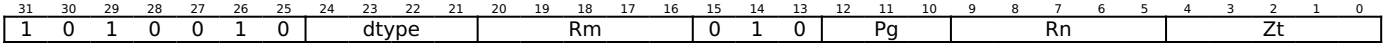
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields msz	ssz	Rm	Instruction Details	Condition
	0x	11111	UNALLOCATED	-
	1x		UNALLOCATED	-
00	00	!= 11111	LD1RQB (scalar plus scalar)	FEAT_SVE || FEAT_SME
00	01	!= 11111	LD1ROB (scalar plus scalar)	FEAT_F64MM
01	00	!= 11111	LD1RQH (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	01	!= 11111	LD1ROH (scalar plus scalar)	FEAT_F64MM
10	00	!= 11111	LD1RQW (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	01	!= 11111	LD1ROW (scalar plus scalar)	FEAT_F64MM
11	00	!= 11111	LD1RQD (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	01	!= 11111	LD1ROD (scalar plus scalar)	FEAT_F64MM

SVE contiguous load (scalar plus scalar)

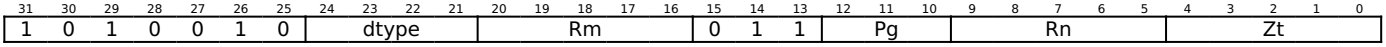
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields dtype	Rm	Instruction Details	Condition
	11111	UNALLOCATED	-
0000	!= 11111	LD1B (scalar plus scalar, single register) — 8-bit element	FEAT_SVE || FEAT_SME
0001	!= 11111	LD1B (scalar plus scalar, single register) — 16-bit element	FEAT_SVE || FEAT_SME
0010	!= 11111	LD1B (scalar plus scalar, single register) — 32-bit element	FEAT_SVE || FEAT_SME
0011	!= 11111	LD1B (scalar plus scalar, single register) — 64-bit element	FEAT_SVE || FEAT_SME
0100	!= 11111	LD1SW (scalar plus scalar)	FEAT_SVE || FEAT_SME
0101	!= 11111	LD1H (scalar plus scalar, single register) — 16-bit element	FEAT_SVE || FEAT_SME
0110	!= 11111	LD1H (scalar plus scalar, single register) — 32-bit element	FEAT_SVE || FEAT_SME
0111	!= 11111	LD1H (scalar plus scalar, single register) — 64-bit element	FEAT_SVE || FEAT_SME
1000	!= 11111	LD1SH (scalar plus scalar) — 64-bit element	FEAT_SVE || FEAT_SME
1001	!= 11111	LD1SH (scalar plus scalar) — 32-bit element	FEAT_SVE || FEAT_SME
1010	!= 11111	LD1W (scalar plus scalar, single register) — 32-bit element	FEAT_SVE || FEAT_SME
1011	!= 11111	LD1W (scalar plus scalar, single register) — 64-bit element	FEAT_SVE || FEAT_SME
1100	!= 11111	LD1SB (scalar plus scalar) — 64-bit element	FEAT_SVE || FEAT_SME
1101	!= 11111	LD1SB (scalar plus scalar) — 32-bit element	FEAT_SVE || FEAT_SME
1110	!= 11111	LD1SB (scalar plus scalar) — 16-bit element	FEAT_SVE || FEAT_SME
1111	!= 11111	LD1D (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME

SVE contiguous first-fault load (scalar plus scalar)

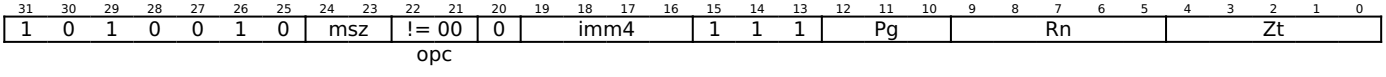
The encodings in this section are decoded from SVE Memory - Contiguous Load.



Decode fields dtype	Instruction Details	Condition
0000	LDFF1B (scalar plus scalar) — 8-bit element	FEAT_SVE
0001	LDFF1B (scalar plus scalar) — 16-bit element	FEAT_SVE
0010	LDFF1B (scalar plus scalar) — 32-bit element	FEAT_SVE
0011	LDFF1B (scalar plus scalar) — 64-bit element	FEAT_SVE
0100	LDFF1SW (scalar plus scalar)	FEAT_SVE
0101	LDFF1H (scalar plus scalar) — 16-bit element	FEAT_SVE
0110	LDFF1H (scalar plus scalar) — 32-bit element	FEAT_SVE
0111	LDFF1H (scalar plus scalar) — 64-bit element	FEAT_SVE
1000	LDFF1SH (scalar plus scalar) — 64-bit element	FEAT_SVE
1001	LDFF1SH (scalar plus scalar) — 32-bit element	FEAT_SVE
1010	LDFF1W (scalar plus scalar) — 32-bit element	FEAT_SVE
1011	LDFF1W (scalar plus scalar) — 64-bit element	FEAT_SVE
1100	LDFF1SB (scalar plus scalar) — 64-bit element	FEAT_SVE
1101	LDFF1SB (scalar plus scalar) — 32-bit element	FEAT_SVE
1110	LDFF1SB (scalar plus scalar) — 16-bit element	FEAT_SVE
1111	LDFF1D (scalar plus scalar)	FEAT_SVE

SVE load multiple structures (scalar plus immediate)

The encodings in this section are decoded from SVE Memory - Contiguous Load.

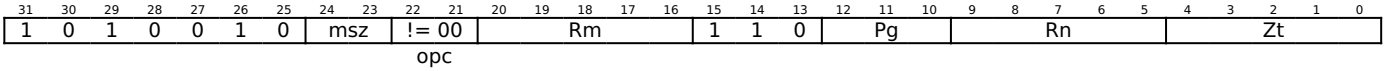


The following constraints also apply to this encoding: opc != '00'

Decode fields msz	Decode fields opc	Instruction Details	Condition
00	01	LD2B (scalar plus immediate)	FEAT_SVE || FEAT_SME
00	10	LD3B (scalar plus immediate)	FEAT_SVE || FEAT_SME
00	11	LD4B (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	01	LD2H (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	10	LD3H (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	11	LD4H (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	01	LD2W (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	10	LD3W (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	11	LD4W (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	01	LD2D (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	10	LD3D (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	11	LD4D (scalar plus immediate)	FEAT_SVE || FEAT_SME

SVE load multiple structures (scalar plus scalar)

The encodings in this section are decoded from SVE Memory - Contiguous Load.

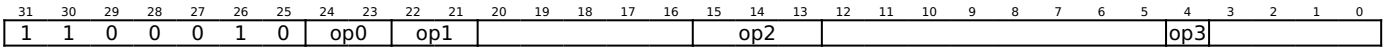


The following constraints also apply to this encoding: opc != '00'

msz	opc	Rm	Instruction Details	Condition
	!= 00	11111	UNALLOCATED	-
00	01	!= 11111	LD2B (scalar plus scalar)	FEAT_SVE || FEAT_SME
00	10	!= 11111	LD3B (scalar plus scalar)	FEAT_SVE || FEAT_SME
00	11	!= 11111	LD4B (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	01	!= 11111	LD2H (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	10	!= 11111	LD3H (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	11	!= 11111	LD4H (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	01	!= 11111	LD2W (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	10	!= 11111	LD3W (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	11	!= 11111	LD4W (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	01	!= 11111	LD2D (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	10	!= 11111	LD3D (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	11	!= 11111	LD4D (scalar plus scalar)	FEAT_SVE || FEAT_SME

SVE Memory - 64-bit Gather

The encodings in this section are decoded from SVE encodings.



Decode fields				Instruction details	Condition
op0	op1	op2	op3		
00	00	101		LD1Q	FEAT_SVE2p1
00	01	0xx	1	UNALLOCATED	-
00	11	1xx	0	SVE 64-bit gather prefetch (scalar plus 64-bit scaled offsets)	-
00	11		1	UNALLOCATED	-
00	x1	0xx	0	SVE 64-bit gather prefetch (scalar plus unpacked 32-bit scaled offsets)	-
	00	111	0	SVE 64-bit gather prefetch (vector plus immediate)	-
	00	111	1	UNALLOCATED	-
	00	1x0		SVE2 64-bit gather non-temporal load (vector plus scalar)	-
	01	1xx		SVE 64-bit gather load (vector plus immediate)	-
	10	1xx		SVE 64-bit gather load (scalar plus 64-bit unscaled offsets)	-
	x0	0xx		SVE 64-bit gather load (scalar plus unpacked 32-bit unscaled offsets)	-
!= 00	00	101		UNALLOCATED	-
!= 00	11	1xx		SVE 64-bit gather load (scalar plus 64-bit scaled offsets)	-
!= 00	x1	0xx		SVE 64-bit gather load (scalar plus 32-bit unpacked scaled offsets)	-

SVE 64-bit gather prefetch (scalar plus 64-bit scaled offsets)

The encodings in this section are decoded from SVE Memory - 64-bit Gather.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	0	0	1	1	Zm					1	msz		Pg			Rn					0	prfop			

Decode fields	Instruction Details	Condition
msz		
00	PRFB (scalar plus vector)	FEAT_SVE
01	PRFH (scalar plus vector)	FEAT_SVE
10	PRFW (scalar plus vector)	FEAT_SVE
11	PRFD (scalar plus vector)	FEAT_SVE

SVE 64-bit gather prefetch (scalar plus unpacked 32-bit scaled offsets)

The encodings in this section are decoded from SVE Memory - 64-bit Gather.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	0	0	xs	1	Zm					0	msz		Pg			Rn					0	prfop			

Decode fields	Instruction Details	Condition
msz		
00	PRFB (scalar plus vector)	FEAT_SVE
01	PRFH (scalar plus vector)	FEAT_SVE
10	PRFW (scalar plus vector)	FEAT_SVE
11	PRFD (scalar plus vector)	FEAT_SVE

SVE 64-bit gather prefetch (vector plus immediate)

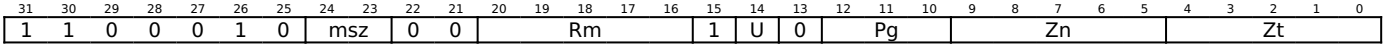
The encodings in this section are decoded from SVE Memory - 64-bit Gather.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	1	0	msz		0	0	imm5					1	1	1	Pg			Zn					0	prfop			

Decode fields msz	Instruction Details	Condition
00	PRFB (vector plus immediate)	FEAT_SVE
01	PRFH (vector plus immediate)	FEAT_SVE
10	PRFW (vector plus immediate)	FEAT_SVE
11	PRFD (vector plus immediate)	FEAT_SVE

SVE2 64-bit gather non-temporal load (vector plus scalar)

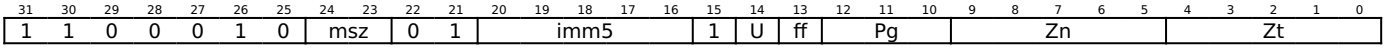
The encodings in this section are decoded from SVE Memory - 64-bit Gather.



Decode fields msz	U	Instruction Details	Condition
00	0	LDNT1SB	FEAT_SVE2
00	1	LDNT1B (vector plus scalar)	FEAT_SVE2
01	0	LDNT1SH	FEAT_SVE2
01	1	LDNT1H (vector plus scalar)	FEAT_SVE2
10	0	LDNT1SW	FEAT_SVE2
10	1	LDNT1W (vector plus scalar)	FEAT_SVE2
11	0	UNALLOCATED	-
11	1	LDNT1D (vector plus scalar)	FEAT_SVE2

SVE 64-bit gather load (vector plus immediate)

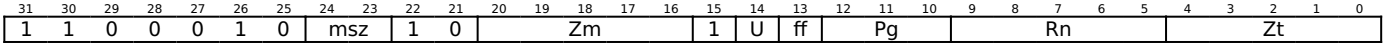
The encodings in this section are decoded from SVE Memory - 64-bit Gather.



Decode fields msz	U	ff	Instruction Details	Condition
00	0	0	LD1SB (vector plus immediate)	FEAT_SVE
00	0	1	LDFF1SB (vector plus immediate)	FEAT_SVE
00	1	0	LD1B (vector plus immediate)	FEAT_SVE
00	1	1	LDFF1B (vector plus immediate)	FEAT_SVE
01	0	0	LD1SH (vector plus immediate)	FEAT_SVE
01	0	1	LDFF1SH (vector plus immediate)	FEAT_SVE
01	1	0	LD1H (vector plus immediate)	FEAT_SVE
01	1	1	LDFF1H (vector plus immediate)	FEAT_SVE
10	0	0	LD1SW (vector plus immediate)	FEAT_SVE
10	0	1	LDFF1SW (vector plus immediate)	FEAT_SVE
10	1	0	LD1W (vector plus immediate)	FEAT_SVE
10	1	1	LDFF1W (vector plus immediate)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (vector plus immediate)	FEAT_SVE
11	1	1	LDFF1D (vector plus immediate)	FEAT_SVE

SVE 64-bit gather load (scalar plus 64-bit unscaled offsets)

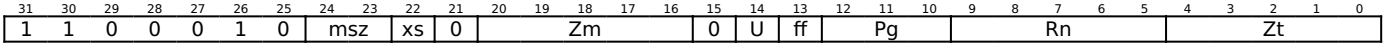
The encodings in this section are decoded from SVE Memory - 64-bit Gather.



Decode fields msz	U	ff	Instruction Details	Condition
00	0	0	LD1SB (scalar plus vector)	FEAT_SVE
00	0	1	LDFF1SB (scalar plus vector)	FEAT_SVE
00	1	0	LD1B (scalar plus vector)	FEAT_SVE
00	1	1	LDFF1B (scalar plus vector)	FEAT_SVE
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE 64-bit gather load (scalar plus unpacked 32-bit unscaled offsets)

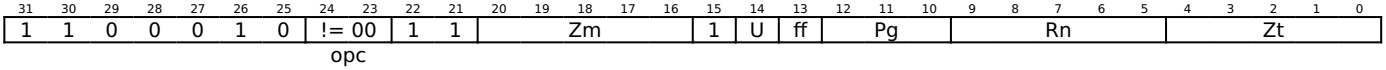
The encodings in this section are decoded from SVE Memory - 64-bit Gather.



Decode fields msz	U	ff	Instruction Details	Condition
00	0	0	LD1SB (scalar plus vector)	FEAT_SVE
00	0	1	LDFF1SB (scalar plus vector)	FEAT_SVE
00	1	0	LD1B (scalar plus vector)	FEAT_SVE
00	1	1	LDFF1B (scalar plus vector)	FEAT_SVE
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE 64-bit gather load (scalar plus 64-bit scaled offsets)

The encodings in this section are decoded from SVE Memory - 64-bit Gather.

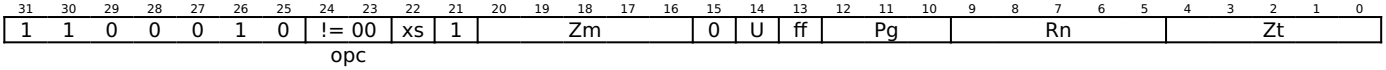


The following constraints also apply to this encoding: opc != '00'

Decode fields			Instruction Details	Condition
opc	U	ff		
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE 64-bit gather load (scalar plus 32-bit unpacked scaled offsets)

The encodings in this section are decoded from SVE Memory - 64-bit Gather.

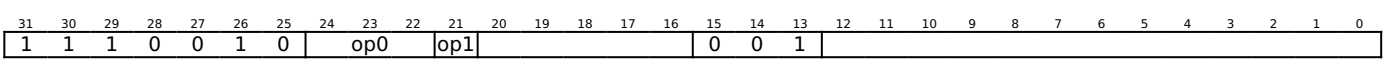


The following constraints also apply to this encoding: opc != '00'

Decode fields			Instruction Details	Condition
opc	U	ff		
01	0	0	LD1SH (scalar plus vector)	FEAT_SVE
01	0	1	LDFF1SH (scalar plus vector)	FEAT_SVE
01	1	0	LD1H (scalar plus vector)	FEAT_SVE
01	1	1	LDFF1H (scalar plus vector)	FEAT_SVE
10	0	0	LD1SW (scalar plus vector)	FEAT_SVE
10	0	1	LDFF1SW (scalar plus vector)	FEAT_SVE
10	1	0	LD1W (scalar plus vector)	FEAT_SVE
10	1	1	LDFF1W (scalar plus vector)	FEAT_SVE
11	0		UNALLOCATED	-
11	1	0	LD1D (scalar plus vector)	FEAT_SVE
11	1	1	LDFF1D (scalar plus vector)	FEAT_SVE

SVE Memory - Non-temporal and Quadword Scatter Store

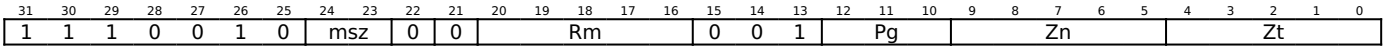
The encodings in this section are decoded from SVE encodings.



Decode fields		Instruction details	Condition
op0	op1		
000	1	ST1Q	FEAT_SVE2p1
xx0	0	SVE2 64-bit scatter non-temporal store (vector plus scalar)	-
xx1	0	SVE2 32-bit scatter non-temporal store (vector plus scalar)	-
!= 000	1	UNALLOCATED	-

SVE2 64-bit scatter non-temporal store (vector plus scalar)

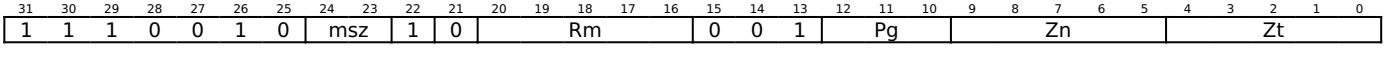
The encodings in this section are decoded from SVE Memory - Non-temporal and Quadword Scatter Store.



Decode fields msz	Instruction Details	Condition
00	STNT1B (vector plus scalar)	FEAT_SVE2
01	STNT1H (vector plus scalar)	FEAT_SVE2
10	STNT1W (vector plus scalar)	FEAT_SVE2
11	STNT1D (vector plus scalar)	FEAT_SVE2

SVE2 32-bit scatter non-temporal store (vector plus scalar)

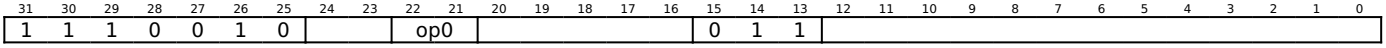
The encodings in this section are decoded from SVE Memory - Non-temporal and Quadword Scatter Store.



Decode fields msz	Instruction Details	Condition
00	STNT1B (vector plus scalar)	FEAT_SVE2
01	STNT1H (vector plus scalar)	FEAT_SVE2
10	STNT1W (vector plus scalar)	FEAT_SVE2
11	UNALLOCATED	-

SVE Memory - Non-temporal and Multi-register Contiguous Store

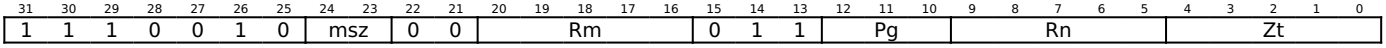
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00	SVE contiguous non-temporal store (scalar plus scalar)
!= 00	SVE store multiple structures (scalar plus scalar)

SVE contiguous non-temporal store (scalar plus scalar)

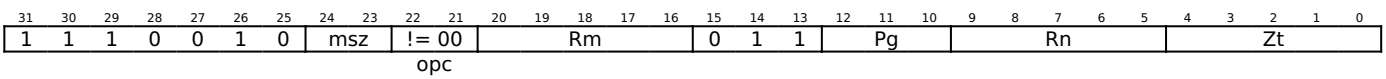
The encodings in this section are decoded from SVE Memory - Non-temporal and Multi-register Contiguous Store.



Decode fields msz	Decode fields Rm	Instruction Details	Condition
	11111	UNALLOCATED	-
00	!= 11111	STNT1B (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
01	!= 11111	STNT1H (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
10	!= 11111	STNT1W (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
11	!= 11111	STNT1D (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME

SVE store multiple structures (scalar plus scalar)

The encodings in this section are decoded from SVE Memory - Non-temporal and Multi-register Contiguous Store.

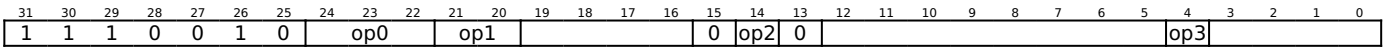


The following constraints also apply to this encoding: opc != '00'

Decode fields msz	opc	Rm	Instruction Details	Condition
	!= 00	11111	UNALLOCATED	-
00	01	!= 11111	ST2B (scalar plus scalar)	FEAT_SVE || FEAT_SME
00	10	!= 11111	ST3B (scalar plus scalar)	FEAT_SVE || FEAT_SME
00	11	!= 11111	ST4B (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	01	!= 11111	ST2H (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	10	!= 11111	ST3H (scalar plus scalar)	FEAT_SVE || FEAT_SME
01	11	!= 11111	ST4H (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	01	!= 11111	ST2W (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	10	!= 11111	ST3W (scalar plus scalar)	FEAT_SVE || FEAT_SME
10	11	!= 11111	ST4W (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	01	!= 11111	ST2D (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	10	!= 11111	ST3D (scalar plus scalar)	FEAT_SVE || FEAT_SME
11	11	!= 11111	ST4D (scalar plus scalar)	FEAT_SVE || FEAT_SME

SVE Memory - Contiguous Store and Unsized Contiguous

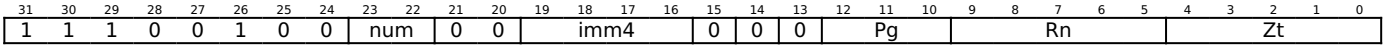
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	op2	op3	Instruction details	Condition
0xx	00	0		SVE store multiple structures (quadwords, scalar plus immediate)	-
0xx	01	0		UNALLOCATED	-
0xx	1x	0		SVE store multiple structures (quadwords, scalar plus scalar)	-
10x		0		UNALLOCATED	-
110		0	0	STR (predicate)	FEAT_SVE || FEAT_SME
110		0	1	UNALLOCATED	-
110		1		STR (vector)	FEAT_SVE || FEAT_SME
111	0x	0		STLRO	FEAT_SVE_LRCPC
111	1x	0		UNALLOCATED	-
!= 110		1		SVE contiguous store (scalar plus scalar)	-

SVE store multiple structures (quadwords, scalar plus immediate)

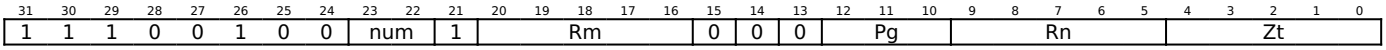
The encodings in this section are decoded from SVE Memory - Contiguous Store and Unsized Contiguous.



Decode fields num	Instruction Details	Condition
00	UNALLOCATED	-
01	ST2Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1
10	ST3Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1
11	ST4Q (scalar plus immediate)	FEAT_SVE2p1 || FEAT_SME2p1

SVE store multiple structures (quadwords, scalar plus scalar)

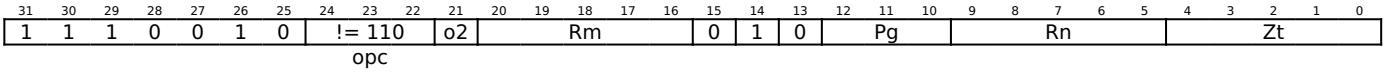
The encodings in this section are decoded from SVE Memory - Contiguous Store and Unsized Contiguous.



Decode fields num	Rm	Instruction Details	Condition
00		UNALLOCATED	-
01	!= 11111	ST2Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
10	!= 11111	ST3Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
11	!= 11111	ST4Q (scalar plus scalar)	FEAT_SVE2p1 || FEAT_SME2p1
!= 00	11111	UNALLOCATED	-

SVE contiguous store (scalar plus scalar)

The encodings in this section are decoded from SVE Memory - Contiguous Store and Unsized Contiguous.

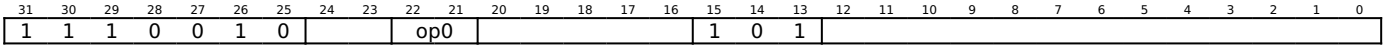


The following constraints also apply to this encoding: opc != '110'

Decode fields opc	o2	Rm	Instruction Details	Condition
0xx		11111	UNALLOCATED	-
00x		!= 11111	ST1B (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
01x		!= 11111	ST1H (scalar plus scalar, single register)	FEAT_SVE || FEAT_SME
10x	0	11111	UNALLOCATED	-
100	0	!= 11111	ST1W (scalar plus scalar, single register) — SVE2	FEAT_SVE2p1
100	1		UNALLOCATED	-
101		!= 11111	ST1W (scalar plus scalar, single register) — SVE	FEAT_SVE || FEAT_SME
101	1	11111	UNALLOCATED	-
111		11111	UNALLOCATED	-
111	0	!= 11111	ST1D (scalar plus scalar, single register) — SVE2	FEAT_SVE2p1
111	1	!= 11111	ST1D (scalar plus scalar, single register) — SVE	FEAT_SVE || FEAT_SME

SVE Memory - Scatter

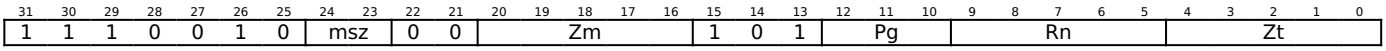
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00	SVE 64-bit scatter store (scalar plus 64-bit unscaled offsets)
01	SVE 64-bit scatter store (scalar plus 64-bit scaled offsets)
10	SVE 64-bit scatter store (vector plus immediate)
11	SVE 32-bit scatter store (vector plus immediate)

SVE 64-bit scatter store (scalar plus 64-bit unscaled offsets)

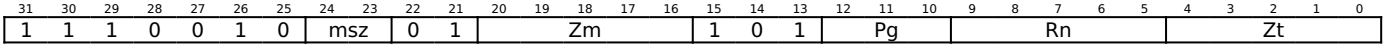
The encodings in this section are decoded from SVE Memory - Scatter.



Decode fields msz	Instruction Details	Condition
00	ST1B (scalar plus vector)	FEAT_SVE
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 64-bit scatter store (scalar plus 64-bit scaled offsets)

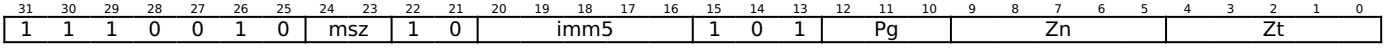
The encodings in this section are decoded from SVE Memory - Scatter.



Decode fields msz	Instruction Details	Condition
00	UNALLOCATED	-
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 64-bit scatter store (vector plus immediate)

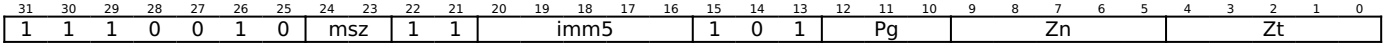
The encodings in this section are decoded from SVE Memory - Scatter.



Decode fields msz	Instruction Details	Condition
00	ST1B (vector plus immediate)	FEAT_SVE
01	ST1H (vector plus immediate)	FEAT_SVE
10	ST1W (vector plus immediate)	FEAT_SVE
11	ST1D (vector plus immediate)	FEAT_SVE

SVE 32-bit scatter store (vector plus immediate)

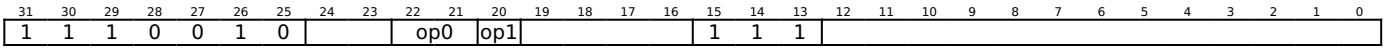
The encodings in this section are decoded from SVE Memory - Scatter.



Decode fields msz	Instruction Details	Condition
00	ST1B (vector plus immediate)	FEAT_SVE
01	ST1H (vector plus immediate)	FEAT_SVE
10	ST1W (vector plus immediate)	FEAT_SVE
11	UNALLOCATED	-

SVE Memory - Contiguous Store with Immediate Offset

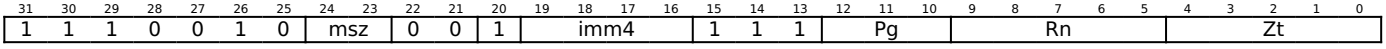
The encodings in this section are decoded from SVE encodings.



Decode fields op0	op1	Instruction details
00	1	SVE contiguous non-temporal store (scalar plus immediate)
	0	SVE contiguous store (scalar plus immediate)
!= 00	1	SVE store multiple structures (scalar plus immediate)

SVE contiguous non-temporal store (scalar plus immediate)

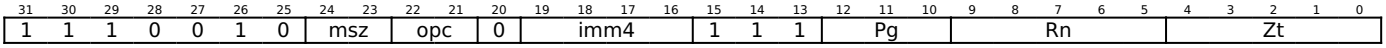
The encodings in this section are decoded from SVE Memory - Contiguous Store with Immediate Offset.



Decode fields msz	Instruction Details	Condition
00	STNT1B (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
01	STNT1H (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
10	STNT1W (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
11	STNT1D (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME

SVE contiguous store (scalar plus immediate)

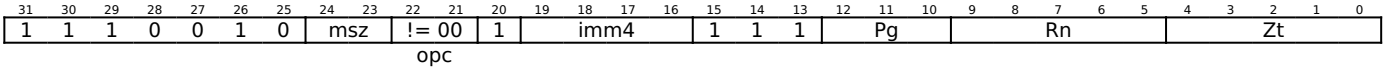
The encodings in this section are decoded from SVE Memory - Contiguous Store with Immediate Offset.



Decode fields msz	opc	Instruction Details	Condition
00		ST1B (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
01		ST1H (scalar plus immediate, single register)	FEAT_SVE || FEAT_SME
10	00	ST1W (scalar plus immediate, single register) — SVE2	FEAT_SVE2p1
10	01	UNALLOCATED	-
10	1x	ST1W (scalar plus immediate, single register) — SVE	FEAT_SVE || FEAT_SME
11	0x	UNALLOCATED	-
11	10	ST1D (scalar plus immediate, single register) — SVE2	FEAT_SVE2p1
11	11	ST1D (scalar plus immediate, single register) — SVE	FEAT_SVE || FEAT_SME

SVE store multiple structures (scalar plus immediate)

The encodings in this section are decoded from SVE Memory - Contiguous Store with Immediate Offset.

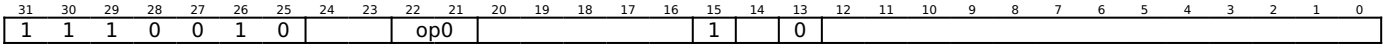


The following constraints also apply to this encoding: opc != '00'

Decode fields msz	opc	Instruction Details	Condition
00	01	ST2B (scalar plus immediate)	FEAT_SVE || FEAT_SME
00	10	ST3B (scalar plus immediate)	FEAT_SVE || FEAT_SME
00	11	ST4B (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	01	ST2H (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	10	ST3H (scalar plus immediate)	FEAT_SVE || FEAT_SME
01	11	ST4H (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	01	ST2W (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	10	ST3W (scalar plus immediate)	FEAT_SVE || FEAT_SME
10	11	ST4W (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	01	ST2D (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	10	ST3D (scalar plus immediate)	FEAT_SVE || FEAT_SME
11	11	ST4D (scalar plus immediate)	FEAT_SVE || FEAT_SME

SVE Memory - Scatter with Optional Sign Extend

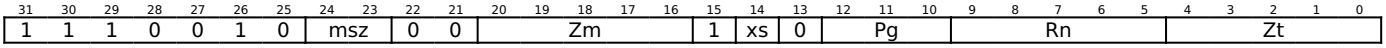
The encodings in this section are decoded from SVE encodings.



Decode fields op0	Instruction details
00	SVE 64-bit scatter store (scalar plus unpacked 32-bit unscaled offsets)
01	SVE 64-bit scatter store (scalar plus unpacked 32-bit scaled offsets)
10	SVE 32-bit scatter store (scalar plus 32-bit unscaled offsets)
11	SVE 32-bit scatter store (scalar plus 32-bit scaled offsets)

SVE 64-bit scatter store (scalar plus unpacked 32-bit unscaled offsets)

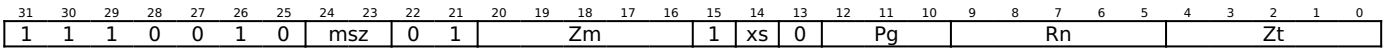
The encodings in this section are decoded from SVE Memory - Scatter with Optional Sign Extend.



Decode fields msz	Instruction Details	Condition
00	ST1B (scalar plus vector)	FEAT_SVE
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 64-bit scatter store (scalar plus unpacked 32-bit scaled offsets)

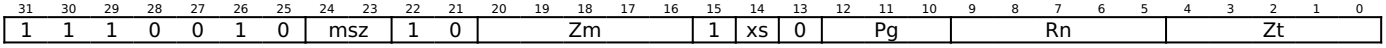
The encodings in this section are decoded from SVE Memory - Scatter with Optional Sign Extend.



Decode fields msz	Instruction Details	Condition
00	UNALLOCATED	-
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	ST1D (scalar plus vector)	FEAT_SVE

SVE 32-bit scatter store (scalar plus 32-bit unscaled offsets)

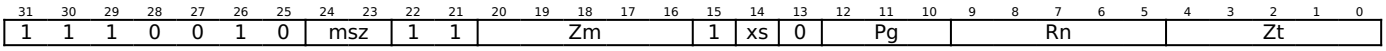
The encodings in this section are decoded from SVE Memory - Scatter with Optional Sign Extend.



Decode fields msz	Instruction Details	Condition
00	ST1B (scalar plus vector)	FEAT_SVE
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	UNALLOCATED	-

SVE 32-bit scatter store (scalar plus 32-bit scaled offsets)

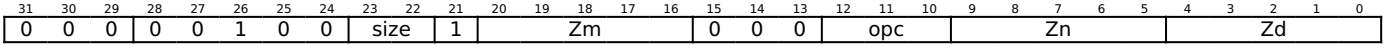
The encodings in this section are decoded from SVE Memory - Scatter with Optional Sign Extend.



Decode fields msz	Instruction Details	Condition
00	UNALLOCATED	-
01	ST1H (scalar plus vector)	FEAT_SVE
10	ST1W (scalar plus vector)	FEAT_SVE
11	UNALLOCATED	-

SVE integer add/subtract vectors (unpredicated)

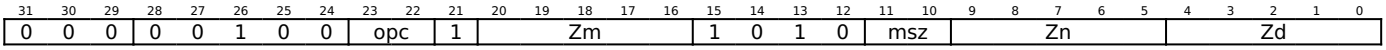
The encodings in this section are decoded from SVE encodings.



Decode fields size	Decode fields opc	Instruction Details	Condition
	000	ADD (vectors, unpredicated)	FEAT_SVE || FEAT_SME
	001	SUB (vectors, unpredicated)	FEAT_SVE || FEAT_SME
	100	SQADD (vectors, unpredicated)	FEAT_SVE || FEAT_SME
	101	UQADD (vectors, unpredicated)	FEAT_SVE || FEAT_SME
	110	SQSUB (vectors, unpredicated)	FEAT_SVE || FEAT_SME
	111	UQSUB (vectors, unpredicated)	FEAT_SVE || FEAT_SME
11	010	ADDPT (unpredicated)	FEAT_SVE && FEAT_CPA
11	011	SUBPT (unpredicated)	FEAT_SVE && FEAT_CPA
!= 11	01x	UNALLOCATED	-

SVE address generation

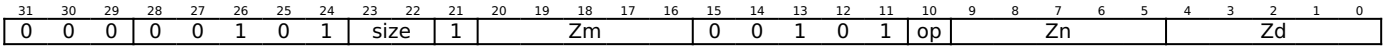
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
00	ADR — Unpacked 32-bit signed offsets	FEAT_SVE
01	ADR — Unpacked 32-bit unsigned offsets	FEAT_SVE
1x	ADR — Packed offsets	FEAT_SVE

SVE table lookup (three sources)

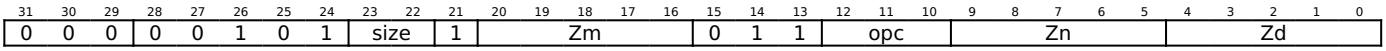
The encodings in this section are decoded from SVE encodings.



Decode fields op	Instruction Details	Condition
0	TBL	FEAT_SVE2 || FEAT_SME
1	TBX	FEAT_SVE2 || FEAT_SME

SVE permute vector elements

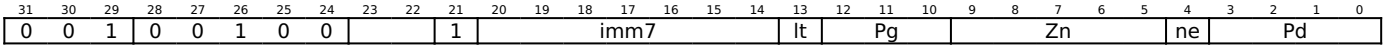
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
000	ZIP1, ZIP2 (vectors) — Low halves	FEAT_SVE || FEAT_SME
001	ZIP1, ZIP2 (vectors) — High halves	FEAT_SVE || FEAT_SME
010	UZP1, UZP2 (vectors) — Even	FEAT_SVE || FEAT_SME
011	UZP1, UZP2 (vectors) — Odd	FEAT_SVE || FEAT_SME
100	TRN1, TRN2 (vectors) — Even	FEAT_SVE || FEAT_SME
101	TRN1, TRN2 (vectors) — Odd	FEAT_SVE || FEAT_SME
11x	UNALLOCATED	-

SVE integer compare with unsigned immediate

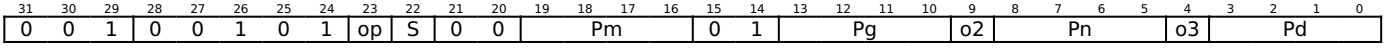
The encodings in this section are decoded from SVE encodings.



Decode fields lt	ne	Instruction Details	Condition
0	0	CMP<cc> (immediate) — Higher or same	FEAT_SVE || FEAT_SME
0	1	CMP<cc> (immediate) — Higher	FEAT_SVE || FEAT_SME
1	0	CMP<cc> (immediate) — Lower	FEAT_SVE || FEAT_SME
1	1	CMP<cc> (immediate) — Lower or same	FEAT_SVE || FEAT_SME

SVE predicate logical operations

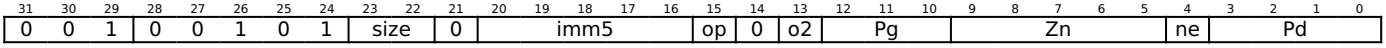
The encodings in this section are decoded from SVE encodings.



Decode fields op	S	o2	o3	Instruction Details	Condition
0	0	0	0	AND (predicates)	FEAT_SVE || FEAT_SME
0	0	0	1	BIC (predicates)	FEAT_SVE || FEAT_SME
0	0	1	0	EOR (predicates)	FEAT_SVE || FEAT_SME
0	0	1	1	SEL (predicates)	FEAT_SVE || FEAT_SME
0	1	0	0	ANDS	FEAT_SVE || FEAT_SME
0	1	0	1	BICS	FEAT_SVE || FEAT_SME
0	1	1	0	EORS	FEAT_SVE || FEAT_SME
0	1	1	1	UNALLOCATED	-
1	0	0	0	ORR (predicates)	FEAT_SVE || FEAT_SME
1	0	0	1	ORN (predicates)	FEAT_SVE || FEAT_SME
1	0	1	0	NOR	FEAT_SVE || FEAT_SME
1	0	1	1	NAND	FEAT_SVE || FEAT_SME
1	1	0	0	ORRS	FEAT_SVE || FEAT_SME
1	1	0	1	ORNS	FEAT_SVE || FEAT_SME
1	1	1	0	NORS	FEAT_SVE || FEAT_SME
1	1	1	1	NANDS	FEAT_SVE || FEAT_SME

SVE integer compare with signed immediate

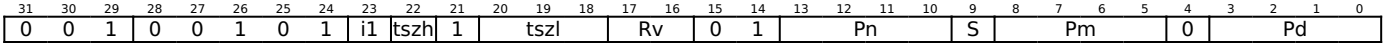
The encodings in this section are decoded from SVE encodings.



Decode fields op	o2	ne	Instruction Details	Condition
0	0	0	CMP<cc> (immediate) — Greater than or equal	FEAT_SVE || FEAT_SME
0	0	1	CMP<cc> (immediate) — Greater than	FEAT_SVE || FEAT_SME
0	1	0	CMP<cc> (immediate) — Less than	FEAT_SVE || FEAT_SME
0	1	1	CMP<cc> (immediate) — Less than or equal	FEAT_SVE || FEAT_SME
1	0	0	CMP<cc> (immediate) — Equal	FEAT_SVE || FEAT_SME
1	0	1	CMP<cc> (immediate) — Not equal	FEAT_SVE || FEAT_SME
1	1		UNALLOCATED	-

SVE broadcast predicate element

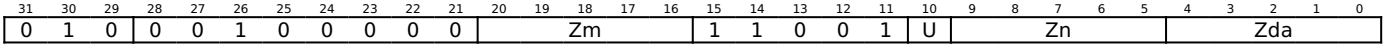
The encodings in this section are decoded from SVE encodings.



Decode fields S	Instruction Details	Condition
0	PSEL	FEAT_SME || FEAT_SVE2p1
1	UNALLOCATED	-

SVE two-way dot product

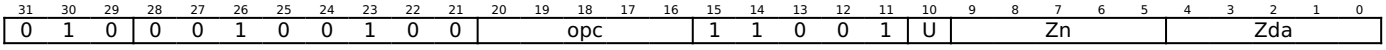
The encodings in this section are decoded from SVE encodings.



Decode fields U	Instruction Details	Condition
0	SDOT (2-way, vectors)	FEAT_SME2 || FEAT_SVE2p1
1	UDOT (2-way, vectors)	FEAT_SME2 || FEAT_SVE2p1

SVE two-way dot product (indexed)

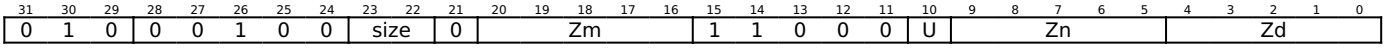
The encodings in this section are decoded from SVE encodings.



Decode fields U	Instruction Details	Condition
0	SDOT (2-way, indexed)	FEAT_SME2 || FEAT_SVE2p1
1	UDOT (2-way, indexed)	FEAT_SME2 || FEAT_SVE2p1

SVE integer clamp

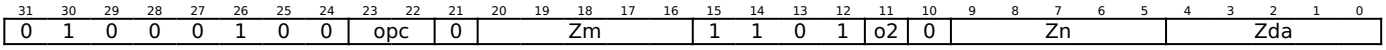
The encodings in this section are decoded from SVE encodings.



Decode fields U	Instruction Details	Condition
0	SCLAMP	FEAT_SME || FEAT_SVE2p1
1	UCLAMP	FEAT_SME || FEAT_SVE2p1

SVE2 multiply-add (checked pointer)

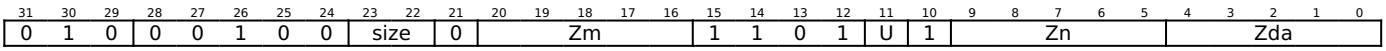
The encodings in this section are decoded from SVE encodings.



Decode fields opc	o2	Instruction Details	Condition
11	0	MLAPT	FEAT_SVE && FEAT_CPA
11	1	MADPT	FEAT_SVE && FEAT_CPA
!= 11		UNALLOCATED	-

SVE2 absolute difference and accumulate long

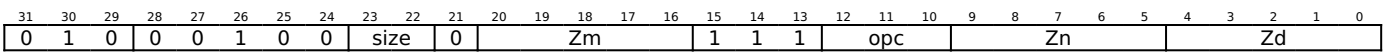
The encodings in this section are decoded from SVE encodings.



Decode fields U	Instruction Details	Condition
0	SABAL	FEAT_SVE2p3 || FEAT_SME2p3
1	UABAL	FEAT_SVE2p3 || FEAT_SME2p3

SVE permute vector elements (quadwords)

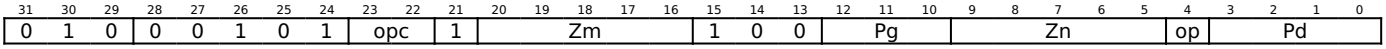
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
000	ZIPQ1	FEAT_SVE2p1 || FEAT_SME2p1
001	ZIPQ2	FEAT_SVE2p1 || FEAT_SME2p1
010	UZPQ1	FEAT_SVE2p1 || FEAT_SME2p1
011	UZPQ2	FEAT_SVE2p1 || FEAT_SME2p1
10x	UNALLOCATED	-
110	TBLQ	FEAT_SVE2p1 || FEAT_SME2p1
111	UNALLOCATED	-

SVE2 character match

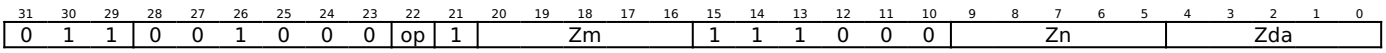
The encodings in this section are decoded from SVE encodings.



Decode fields opc	op	Instruction Details	Condition
0x	0	MATCH	FEAT_SVE2
0x	1	NMATCH	FEAT_SVE2
1x		UNALLOCATED	-

SVE2 FP8 matrix multiply-accumulate

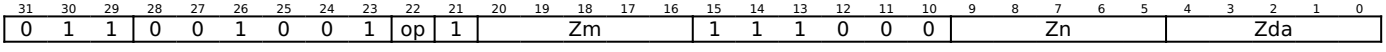
The encodings in this section are decoded from SVE encodings.



Decode fields op	Instruction Details	Condition
0	FMMLA (widening, FP8 to FP32)	FEAT_SVE2 && FEAT_F8F32MM
1	FMMLA (widening, FP8 to FP16)	FEAT_SVE2 && FEAT_F8F16MM

SVE2 floating-point non-widening matrix-multiply accumulate

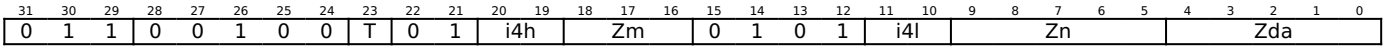
The encodings in this section are decoded from SVE encodings.



Decode fields op	Instruction Details	Condition
0	FMMLA (non-widening)	FEAT_SVE2p2 && FEAT_F16MM
1	BFMMLA (non-widening)	FEAT_SVE_B16MM

SVE2 FP8 multiply-add long (indexed)

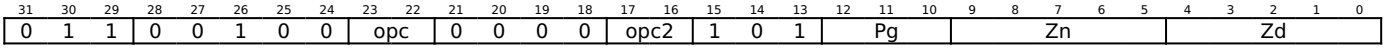
The encodings in this section are decoded from SVE encodings.



Decode fields T	Instruction Details	Condition
0	FMLALB (indexed, FP8 to FP16)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
1	FMLALT (indexed, FP8 to FP16)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)

SVE floating-point convert (top, predicated)

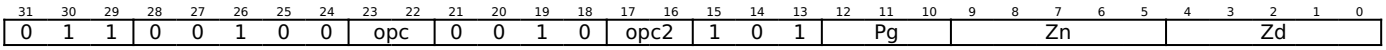
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTXNT	FEAT_SVE2p2 || FEAT_SME2p2
01		UNALLOCATED	-
10	00	FCVTNT (predicated) — Single-precision to half-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	01	FCVTLT — Half-precision to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
10	10	BFCVTNT	FEAT_SVE2p2 || FEAT_SME2p2
11	0x	UNALLOCATED	-
11	10	FCVTNT (predicated) — Double-precision to single-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2
11	11	FCVTLT — Single-precision to double-precision, zeroing	FEAT_SVE2p2 || FEAT_SME2p2

SVE floating-point convert precision odd elements

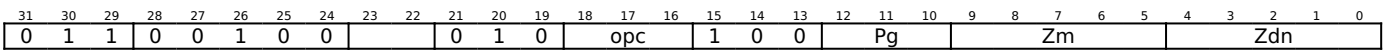
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Decode fields opc2	Instruction Details	Condition
x0	11	UNALLOCATED	-
00	0x	UNALLOCATED	-
00	10	FCVTXNT	FEAT_SVE2 || FEAT_SME
01		UNALLOCATED	-
10	00	FCVTNT (predicated) — Single-precision to half-precision, merging	FEAT_SVE2 || FEAT_SME
10	01	FCVTLT — Half-precision to single-precision, merging	FEAT_SVE2 || FEAT_SME
10	10	BFCVTNT	(FEAT_SVE && FEAT_BF16) || (FEAT_SME && FEAT_BF16)
11	0x	UNALLOCATED	-
11	10	FCVTNT (predicated) — Double-precision to single-precision, merging	FEAT_SVE2 || FEAT_SME
11	11	FCVTLT — Single-precision to double-precision, merging	FEAT_SVE2 || FEAT_SME

SVE2 floating-point pairwise operations

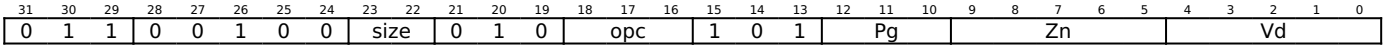
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
000	FADDP	FEAT_SVE2 || FEAT_SME
001	UNALLOCATED	-
01x	UNALLOCATED	-
100	FMAXNMP	FEAT_SVE2 || FEAT_SME
101	FMINNMP	FEAT_SVE2 || FEAT_SME
110	FMAXP	FEAT_SVE2 || FEAT_SME
111	FMINP	FEAT_SVE2 || FEAT_SME

SVE floating-point recursive reduction (quadwords)

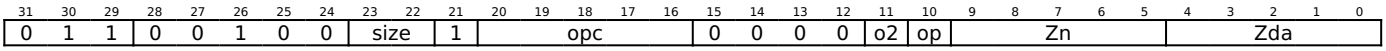
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
000	FADDQV	FEAT_SVE2p1 || FEAT_SME2p1
001	UNALLOCATED	-
01x	UNALLOCATED	-
100	FMAXNMQV	FEAT_SVE2p1 || FEAT_SME2p1
101	FMINNMQV	FEAT_SVE2p1 || FEAT_SME2p1
110	FMAXQV	FEAT_SVE2p1 || FEAT_SME2p1
111	FMINQV	FEAT_SVE2p1 || FEAT_SME2p1

SVE floating-point multiply-add (indexed)

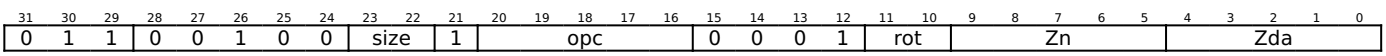
The encodings in this section are decoded from SVE encodings.



Decode fields size	o2	op	Instruction Details	Condition
0x	0	0	FMLA (indexed) — Half-precision	FEAT_SVE || FEAT_SME
0x	0	1	FMLS (indexed) — Half-precision	FEAT_SVE || FEAT_SME
0x	1	0	BFMLA (indexed)	FEAT_SVE_B16B16
0x	1	1	BFMLS (indexed)	FEAT_SVE_B16B16
1x	1		UNALLOCATED	-
10	0	0	FMLA (indexed) — Single-precision	FEAT_SVE || FEAT_SME
10	0	1	FMLS (indexed) — Single-precision	FEAT_SVE || FEAT_SME
11	0	0	FMLA (indexed) — Double-precision	FEAT_SVE || FEAT_SME
11	0	1	FMLS (indexed) — Double-precision	FEAT_SVE || FEAT_SME

SVE floating-point complex multiply-add (indexed)

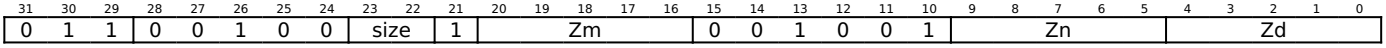
The encodings in this section are decoded from SVE encodings.



Decode fields size	Instruction Details	Condition
0x	UNALLOCATED	-
10	FCMLA (indexed) — Half-precision	FEAT_SVE || FEAT_SME
11	FCMLA (indexed) — Single-precision	FEAT_SVE || FEAT_SME

SVE FP clamp

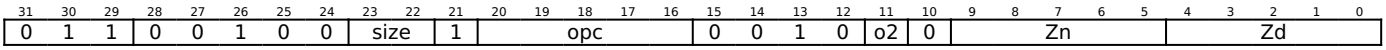
The encodings in this section are decoded from SVE encodings.



Decode fields size	Instruction Details	Condition
00	BFCLAMP	FEAT_SVE_B16B16
!= 00	FCLAMP	FEAT_SME2 || FEAT_SVE2p1

SVE floating-point multiply (indexed)

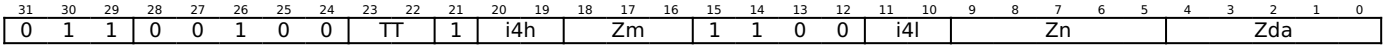
The encodings in this section are decoded from SVE encodings.



Decode fields size	o2	Instruction Details	Condition
0x	0	FMUL (indexed) — Half-precision	FEAT_SVE || FEAT_SME
0x	1	BFMUL (indexed)	FEAT_SVE_B16B16
1x	1	UNALLOCATED	-
10	0	FMUL (indexed) — Single-precision	FEAT_SVE || FEAT_SME
11	0	FMUL (indexed) — Double-precision	FEAT_SVE || FEAT_SME

SVE2 FP8 multiply-add long long (indexed)

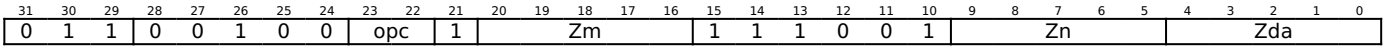
The encodings in this section are decoded from SVE encodings.



Decode fields TT	Instruction Details	Condition
00	FMLALLBB (indexed)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
01	FMLALLBT (indexed)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
10	FMLALLTB (indexed)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)
11	FMLALLTT (indexed)	FEAT_SSVE_FP8FMA || (FEAT_SVE2 && FEAT_FP8FMA)

SVE floating-point matrix multiply accumulate

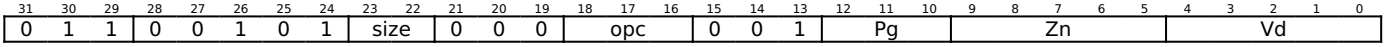
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
00	FMMLA (widening, FP16 to FP32)	FEAT_SVE_F16F32MM
01	BFMMLA (widening)	FEAT_SVE && FEAT_BF16
10	FMMLA (non-widening) — 32-bit element	FEAT_F32MM
11	FMMLA (non-widening) — 64-bit element	FEAT_F64MM

SVE floating-point recursive reduction

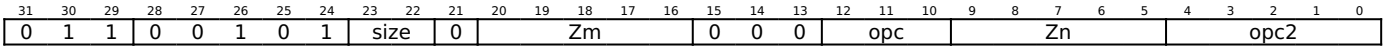
The encodings in this section are decoded from SVE encodings.



Decode fields opc	Instruction Details	Condition
000	FADDV	FEAT_SVE || FEAT_SME
001	UNALLOCATED	-
01x	UNALLOCATED	-
100	FMAXNMV	FEAT_SVE || FEAT_SME
101	FMINNMV	FEAT_SVE || FEAT_SME
110	FMAXV	FEAT_SVE || FEAT_SME
111	FMINV	FEAT_SVE || FEAT_SME

SVE floating-point arithmetic (unpredicated)

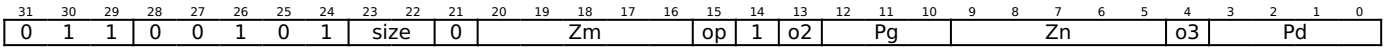
The encodings in this section are decoded from SVE encodings.



Decode fields size	opc	Instruction Details	Condition
	011	FTSMUL	FEAT_SVE
	10x	UNALLOCATED	-
	110	FRECPS	FEAT_SVE || FEAT_SME
	111	FRSQRTS	FEAT_SVE || FEAT_SME
00	000	BFADD (unpredicated)	FEAT_SVE_B16B16
00	001	BFSUB (unpredicated)	FEAT_SVE_B16B16
00	010	BFMUL (vectors, unpredicated)	FEAT_SVE_B16B16
!= 00	000	FADD (vectors, unpredicated)	FEAT_SVE || FEAT_SME
!= 00	001	FSUB (vectors, unpredicated)	FEAT_SVE || FEAT_SME
!= 00	010	FMUL (vectors, unpredicated)	FEAT_SVE || FEAT_SME

SVE floating-point compare vectors

The encodings in this section are decoded from SVE encodings.



Decode fields			Instruction Details	Condition
op	o2	o3		
0	0	0	FCM<cc> (vectors) — Greater than or equal	FEAT_SVE || FEAT_SME
0	0	1	FCM<cc> (vectors) — Greater than	FEAT_SVE || FEAT_SME
0	1	0	FCM<cc> (vectors) — Equal	FEAT_SVE || FEAT_SME
0	1	1	FCM<cc> (vectors) — Not equal	FEAT_SVE || FEAT_SME
1	0	0	FCM<cc> (vectors) — Unordered	FEAT_SVE || FEAT_SME
1	0	1	FAC<cc> — Greater than or equal	FEAT_SVE || FEAT_SME
1	1	0	UNALLOCATED	-
1	1	1	FAC<cc> — Greater than	FEAT_SVE || FEAT_SME

Data Processing -- Immediate

The encodings in this section are decoded from A64 instruction set encoding.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	op0		1	0	0	op1																									

Decode fields		Instruction details
op0	op1	
11	111x	Data-processing (1 source immediate)
	00xx	PC-rel. addressing
	010x	Add/subtract (immediate)
	0110	Add/subtract (immediate, with tags)
	0111	Min/max (immediate)
	100x	Logical (immediate)
	101x	Move wide (immediate)
	110x	Bitfield
!= 11	111x	Extract

Data-processing (1 source immediate)

The encodings in this section are decoded from Data Processing -- Immediate.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	1	1	1	0	0	1	1	1	opc		imm16																Rd				

Decode fields			Instruction Details	Condition
sf	opc	Rd		
0			UNALLOCATED	-
1	0x	!= 11111	UNALLOCATED	-
1	00	11111	AUTIASPPC	FEAT_PAuth_LR
1	01	11111	AUTIBSPPC	FEAT_PAuth_LR
1	1x		UNALLOCATED	-

PC-rel. addressing

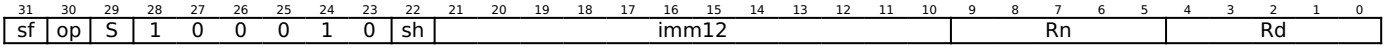
The encodings in this section are decoded from Data Processing -- Immediate.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
op	immlo		1	0	0	0	0	immhi																			Rd				

Decode fields	Instruction Details
op	
0	ADR
1	ADRP

Add/subtract (immediate)

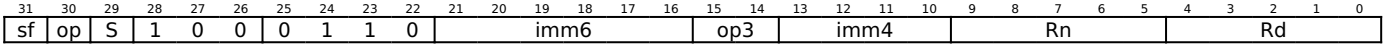
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details
sf	op	S	
0	0	0	ADD (immediate) — 32-bit
0	0	1	ADDS (immediate) — 32-bit
0	1	0	SUB (immediate) — 32-bit
0	1	1	SUBS (immediate) — 32-bit
1	0	0	ADD (immediate) — 64-bit
1	0	1	ADDS (immediate) — 64-bit
1	1	0	SUB (immediate) — 64-bit
1	1	1	SUBS (immediate) — 64-bit

Add/subtract (immediate, with tags)

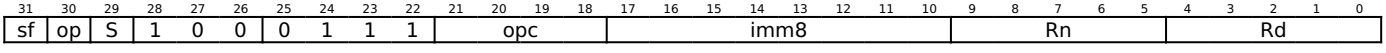
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details	Condition
sf	op	S		
0			UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	ADDG	FEAT_MTE
1	1	0	SUBG	FEAT_MTE

Min/max (immediate)

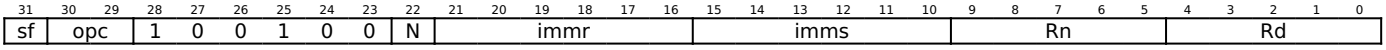
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields				Instruction Details	Condition
sf	op	S	opc		
	0	0	!= 00xx	UNALLOCATED	-
	0	1		UNALLOCATED	-
	1			UNALLOCATED	-
0	0	0	0000	SMAX (immediate) — 32-bit	FEAT_CSSC
0	0	0	0001	UMAX (immediate) — 32-bit	FEAT_CSSC
0	0	0	0010	SMIN (immediate) — 32-bit	FEAT_CSSC
0	0	0	0011	UMIN (immediate) — 32-bit	FEAT_CSSC
1	0	0	0000	SMAX (immediate) — 64-bit	FEAT_CSSC
1	0	0	0001	UMAX (immediate) — 64-bit	FEAT_CSSC
1	0	0	0010	SMIN (immediate) — 64-bit	FEAT_CSSC
1	0	0	0011	UMIN (immediate) — 64-bit	FEAT_CSSC

Logical (immediate)

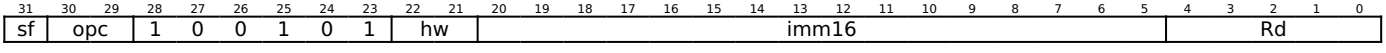
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details
sf	opc	N	
0		1	UNALLOCATED
0	00	0	AND (immediate) — 32-bit
0	01	0	ORR (immediate) — 32-bit
0	10	0	EOR (immediate) — 32-bit
0	11	0	ANDS (immediate) — 32-bit
1	00		AND (immediate) — 64-bit
1	01		ORR (immediate) — 64-bit
1	10		EOR (immediate) — 64-bit
1	11		ANDS (immediate) — 64-bit

Move wide (immediate)

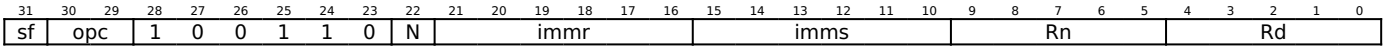
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details
sf	opc	hw	
	01	0x	UNALLOCATED
0		1x	UNALLOCATED
0	00	0x	MOVN — 32-bit
0	10	0x	MOVZ — 32-bit
0	11	0x	MOVK — 32-bit
1	00		MOVN — 64-bit
1	01	1x	UNALLOCATED
1	10		MOVZ — 64-bit
1	11		MOVK — 64-bit

Bitfield

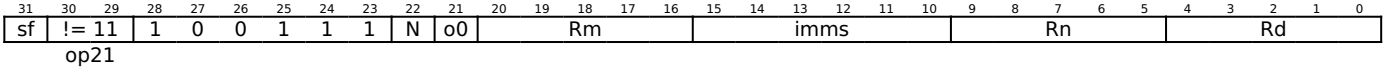
The encodings in this section are decoded from Data Processing -- Immediate.



Decode fields			Instruction Details
sf	opc	N	
0		1	UNALLOCATED
0	00	0	SBFM — 32-bit
0	01	0	BFM — 32-bit
0	10	0	UBFM — 32-bit
0	11	0	UNALLOCATED
1		0	UNALLOCATED
1	00	1	SBFM — 64-bit
1	01	1	BFM — 64-bit
1	10	1	UBFM — 64-bit
1	11	1	UNALLOCATED

Extract

The encodings in this section are decoded from Data Processing -- Immediate.

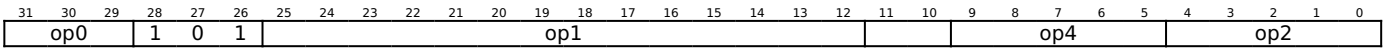


The following constraints also apply to this encoding: op21 != '11'

Decode fields					Instruction Details
sf	op21	N	o0	imms	
	00		1		UNALLOCATED
	01				UNALLOCATED
	10				UNALLOCATED
0	00	0	0	0xxxxx	EXTR — 32-bit
0	00	0	0	1xxxxx	UNALLOCATED
0	00	1	0		UNALLOCATED
1	00	0	0		UNALLOCATED
1	00	1	0		EXTR — 64-bit

Branches, Exception Generating and System instructions

The encodings in this section are decoded from A64 instruction set encoding.

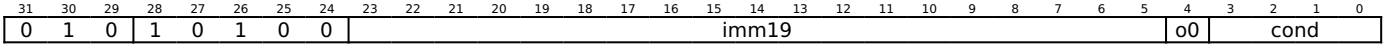


Decode fields				Instruction details	Condition
op0	op1	op4	op2		
010	00xxxxxxxxxxx			Conditional branch (immediate)	-
010	01xxxxxxxxxxx			Miscellaneous branch (immediate)	-
010	1xxxxxxxxxxxx			UNALLOCATED	-
011	00xxxxxxx1xxx			Compare bytes/halfwords in registers and branch	-
110	00xxxxxxxxxxx			Exception generation	-
110	010000000x0010			HINTE	FEAT_HINTE
110	010000000x00x1			UNALLOCATED	-
110	01000000100010			HINTE	FEAT_HINTE
110	010000001000x1			UNALLOCATED	-
110	01000000110001			System instructions with register argument	-
110	01000000110010		11111	Hints	-
110	01000000110010		!= 11111	HINTE	FEAT_HINTE
110	01000000110011			Barriers	-
110	01000001xx0010			HINTE	FEAT_HINTE
110	01000001xx00x1			UNALLOCATED	-
110	0100000xxx0100			PSTATE	-
110	0100000xxx0101			UNALLOCATED	-
110	0100000xxx011x			UNALLOCATED	-
110	0100100xxx0010			HINTE	FEAT_HINTE
110	0100100xxx00x1			UNALLOCATED	-
110	0100100xxx01xx			UNALLOCATED	-
110	0100x00xxx0000			UNALLOCATED	-
110	0100x00xxx1xxx			UNALLOCATED	-
110	0100x01xxxxxxx			System instructions	-
110	0100x1xxxxxxxx			System register move	-
110	0101x00xxxxxxx			UNALLOCATED	-
110	0101x01xxxxxxx			System pair instructions	-
110	0101x1xxxxxxxx			System register pair move	-
110	011xxxxxxxxxx			UNALLOCATED	-
110	1xxxxxxxxxxxx			Unconditional branch (register)	-
111	00xxxxxxx1xxx			UNALLOCATED	-
x00				Unconditional branch (immediate)	-
x01	0xxxxxxxxxxxx			Compare and branch (immediate)	-
x01	1xxxxxxxxxxxx			Test and branch (immediate)	-
x11	00xxxxxxx00xx			Compare registers and branch	-
x11	00xxxxxxx01xx			UNALLOCATED	-
x11	01xxxxxxxx0xx			Compare register with immediate and branch	-
x11	01xxxxxxxx1xx			UNALLOCATED	-
x11	10xxxxx0000000	1001x		Test bit and fault	-
x11	10xxxxx0010000	1001x		UNALLOCATED	-
x11	10xxxxx01x0000	1001x		UNALLOCATED	-
x11	10xxxxx1xx0000	1001x		UNALLOCATED	-
x11	10xxxxxxxx0000	0xxxx		Compare and fault (immediate)	-
x11	10xxxxxxxx0000	10000		Compare and fault (register)	-
x11	10xxxxxxxx0000	10001		UNALLOCATED	-

x11	10xxxxxxxx0000	101xx		UNALLOCATED	-
x11	10xxxxxxxx0000	11xxx		UNALLOCATED	-
x11	10xxxxxxxx0100			UNALLOCATED	-
x11	10xxxxxxxx0x01			UNALLOCATED	-
x11	10xxxxxxxx0x1x			UNALLOCATED	-
x11	10xxxxxxxx1xxx			UNALLOCATED	-
x11	11xxxxxxxxxxxx			UNALLOCATED	-

Conditional branch (immediate)

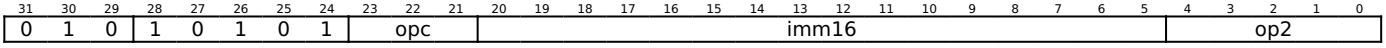
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields o0	Instruction Details	Condition
0	B.cond	-
1	BC.cond	FEAT_HBC

Miscellaneous branch (immediate)

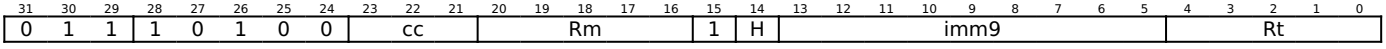
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields opc	op2	Instruction Details	Condition
00x	!= 11111	UNALLOCATED	-
000	11111	RETAASPPC, RETABSPPC — RETAASPPC	FEAT_PAuth_LR
001	11111	RETAASPPC, RETABSPPC — RETABSPPC	FEAT_PAuth_LR
!= 00x		UNALLOCATED	-

Compare bytes/halfwords in registers and branch

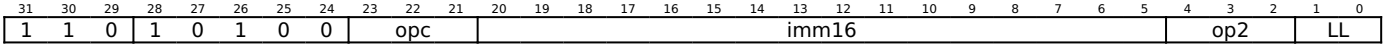
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields cc	H	Instruction Details	Condition
000	0	CBB<cc> — Greater than	FEAT_CMPBR
000	1	CBH<cc> — Greater than	FEAT_CMPBR
001	0	CBB<cc> — Greater than or equal	FEAT_CMPBR
001	1	CBH<cc> — Greater than or equal	FEAT_CMPBR
010	0	CBB<cc> — Higher	FEAT_CMPBR
010	1	CBH<cc> — Higher	FEAT_CMPBR
011	0	CBB<cc> — Higher or same	FEAT_CMPBR
011	1	CBH<cc> — Higher or same	FEAT_CMPBR
10x		UNALLOCATED	-
110	0	CBB<cc> — Equal	FEAT_CMPBR
110	1	CBH<cc> — Equal	FEAT_CMPBR
111	0	CBB<cc> — Not equal	FEAT_CMPBR
111	1	CBH<cc> — Not equal	FEAT_CMPBR

Exception generation

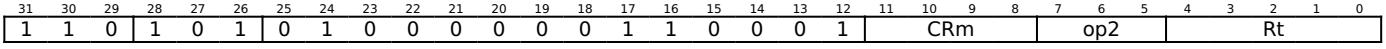
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields			Instruction Details	Condition
opc	op2	LL		
x01	001		UNALLOCATED	-
x01	01x		UNALLOCATED	-
x11	0xx		UNALLOCATED	-
000	000	00	UNALLOCATED	-
000	000	01	SVC	-
000	000	10	HVC	-
000	000	11	SMC	-
000	!= 000		UNALLOCATED	-
001	000	00	BRK	-
001	000	!= 00	UNALLOCATED	-
001	1xx		FLT.cond	FEAT_CFLT
010		!= 00	UNALLOCATED	-
010	000	00	HLT	-
010	!= 000	00	UNALLOCATED	-
011	1xx		UNALLOCATED	-
1x0			UNALLOCATED	-
1x1	1xx		UNALLOCATED	-
101	000	00	UNALLOCATED	-
101	000	01	DCPS1	-
101	000	10	DCPS2	-
101	000	11	DCPS3	-

System instructions with register argument

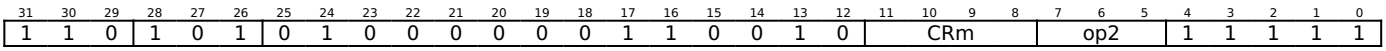
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields		Instruction Details	Condition
CRm	op2		
0000	000	WFET	FEAT_WFxT
0000	001	WFIT	FEAT_WFxT
0000	!= 00x	UNALLOCATED	-
!= 0000		UNALLOCATED	-

Hints

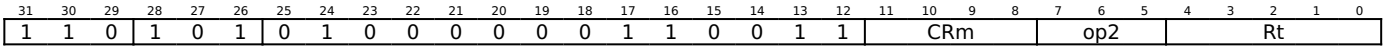
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields		Instruction Details	Condition
CRm	op2		
		HINT	-
0000	000	NOP	-
0000	001	YIELD	-
0000	010	WFE	-
0000	011	WFI	-
0000	100	SEV	-
0000	101	SEVL	-
0000	110	DGH	FEAT_DGH
0000	111	XPACD, XPACI, XPACLRI	FEAT_PAuth
0001	000	PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIA1716	FEAT_PAuth
0001	010	PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIB1716	FEAT_PAuth
0001	100	AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIA1716	FEAT_PAuth
0001	110	AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIB1716	FEAT_PAuth
0010	000	ESB	FEAT_RAS
0010	001	PSB	FEAT_SPE
0010	010	TSB	FEAT_TRF
0010	011	GCSB	FEAT_GCS
0010	100	CSDB	-
0010	110	CLRBHB	FEAT_CLRBHB
0011	000	PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIAZ	FEAT_PAuth
0011	001	PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIASP	FEAT_PAuth
0011	010	PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIBZ	FEAT_PAuth
0011	011	PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIBSP	FEAT_PAuth
0011	100	AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIAZ	FEAT_PAuth
0011	101	AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIASP	FEAT_PAuth
0011	110	AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIBZ	FEAT_PAuth
0011	111	AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIBSP	FEAT_PAuth
0100	xx0	BTI	FEAT_BTI
0100	111	PACM	FEAT_PAuth_LR
0101	000	CHKFEAT	FEAT_CHK
0110	00x	STSHH	FEAT_PCDPHINT
0110	01x	SHUH	FEAT_CMH
0110	100	STCPH	FEAT_CMH
0110	11x	SRLS	FEAT_RLCS
0111	000	SLBND	FEAT_RLCS

Barriers

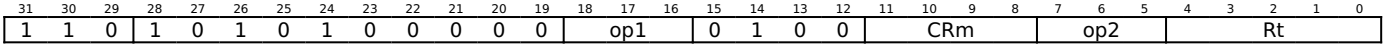
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields			Instruction Details	Condition
CRm	op2	Rt		
		!= 11111	UNALLOCATED	-
	000	11111	UNALLOCATED	-
	010	11111	CLREX	-
	100	11111	DSB — Memory barrier	-
	101	11111	DMB	-
	110	11111	ISB	-
	111	11111	SB	FEAT_SB
xx10	001	11111	DSB — Memory nXS barrier	FEAT_XS
xx10	011	11111	UNALLOCATED	-
!= xx10	0x1	11111	UNALLOCATED	-

PSTATE

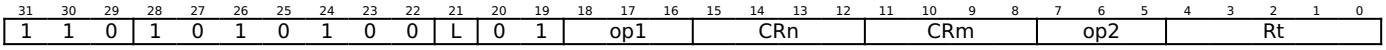
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields			Instruction Details	Condition
op1	op2	Rt		
		!= 11111	UNALLOCATED	-
000	000	11111	CFINV	FEAT_FlagM
000	001	11111	XAFLAG	FEAT_FlagM2
000	010	11111	AXFLAG	FEAT_FlagM2
000	011	11111	MSR (immediate)	-
000	1xx	11111	MSR (immediate)	-
!= 000		11111	MSR (immediate)	-

System instructions

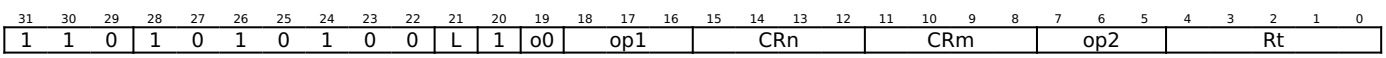
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields	Instruction Details
L	
0	SYS
1	SYSL

System register move

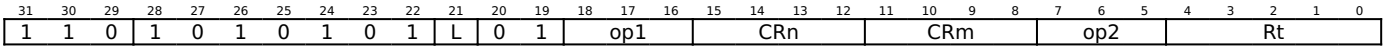
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields	Instruction Details
L	
0	MSR (register)
1	MRS

System pair instructions

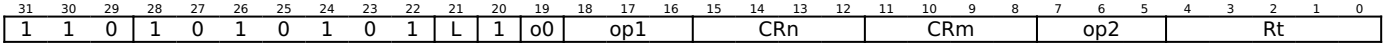
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields L	Instruction Details	Condition
0	SYSP	FEAT_SYSINSTR128
1	UNALLOCATED	-

System register pair move

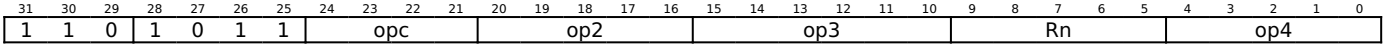
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields L	Instruction Details	Condition
0	MSRR	FEAT_SYSREG128
1	MRRS	FEAT_SYSREG128

Unconditional branch (register)

The encodings in this section are decoded from Branches, Exception Generating and System instructions.

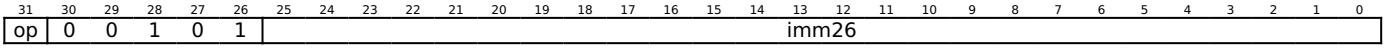


Decode fields					Instruction Details	Condition
opc	op2	op3	Rn	op4		
	11111	!= 0000xx			UNALLOCATED	-
	!= 11111				UNALLOCATED	-
00x0	11111	000000		!= 00000	UNALLOCATED	-
00x0	11111	000001			UNALLOCATED	-
000x	11111	00001x		!= 11111	UNALLOCATED	-
0000	11111	000000		00000	BR	-
0000	11111	000010		11111	BRAA, BRAAZ, BRAB, BRABZ — Key A, zero modifier	FEAT_PAuth
0000	11111	000011		11111	BRAA, BRAAZ, BRAB, BRABZ — Key B, zero modifier	FEAT_PAuth
0001	11111	000000		00000	BLR	-
0001	11111	000000		!= 00000	UNALLOCATED	-
0001	11111	000001			UNALLOCATED	-
0001	11111	000010		11111	BLRAA, BLRAAZ, BLRAB, BLRABZ — Key A, zero modifier	FEAT_PAuth
0001	11111	000011		11111	BLRAA, BLRAAZ, BLRAB, BLRABZ — Key B, zero modifier	FEAT_PAuth
0010	11111	000000		00000	RET	-
0010	11111	00001x	!= 11111		UNALLOCATED	-
0010	11111	000010	11111	11111	RETAA, RETAB — RETAA	FEAT_PAuth
0010	11111	000010	11111	!= 11111	RETAASPPCR, RETABSPPCR — RETAASPPCR	FEAT_PAuth_LR
0010	11111	000011	11111	11111	RETAA, RETAB — RETAB	FEAT_PAuth
0010	11111	000011	11111	!= 11111	RETAASPPCR, RETABSPPCR — RETABSPPCR	FEAT_PAuth_LR
0011	11111	0000xx			UNALLOCATED	-
010x	11111	0000xx	!= 11111		UNALLOCATED	-
010x	11111	000000	11111	!= 00000	UNALLOCATED	-
010x	11111	000001	11111		UNALLOCATED	-
010x	11111	00001x	11111	xxxx0	UNALLOCATED	-
0100	11111	000000	11111	00000	ERET	-
0100	11111	00001x	11111	0xxx1	UNALLOCATED	-
0100	11111	00001x	11111	10xx1	UNALLOCATED	-
0100	11111	00001x	11111	110x1	UNALLOCATED	-
0100	11111	00001x	11111	11101	UNALLOCATED	-
0100	11111	000010	11111	11111	ERETAA, ERETAB — ERETAA	FEAT_PAuth
0100	11111	000011	11111	11111	ERETAA, ERETAB — ERETAB	FEAT_PAuth
0101	11111	000000	11111	00000	DRPS	-
0101	11111	00001x	11111	xxxx1	UNALLOCATED	-
011x	11111	0000xx			UNALLOCATED	-

Decode fields					Instruction Details	Condition
opc	op2	op3	Rn	op4		
100x	11111	00000x			UNALLOCATED	-
1000	11111	000010			BRAA, BRAAZ, BRAB, BRABZ — Key A, register modifier	FEAT_PAuth
1000	11111	000011			BRAA, BRAAZ, BRAB, BRABZ — Key B, register modifier	FEAT_PAuth
1001	11111	000010			BLRAA, BLRAAZ, BLRAB, BLRABZ — Key A, register modifier	FEAT_PAuth
1001	11111	000011			BLRAA, BLRAAZ, BLRAB, BLRABZ — Key B, register modifier	FEAT_PAuth
101x	11111	0000xx			UNALLOCATED	-
11xx	11111	0000xx			UNALLOCATED	-

Unconditional branch (immediate)

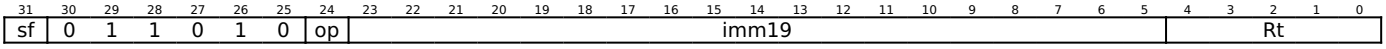
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields	Instruction Details
op	
0	B
1	BL

Compare and branch (immediate)

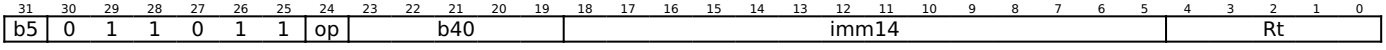
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields		Instruction Details
sf	op	
0	0	CBZ — 32-bit
0	1	CBNZ — 32-bit
1	0	CBZ — 64-bit
1	1	CBNZ — 64-bit

Test and branch (immediate)

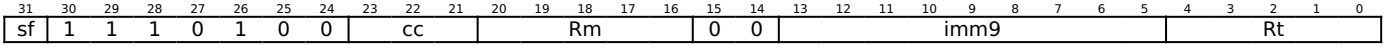
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields	Instruction Details
op	
0	TBZ
1	TBNZ

Compare registers and branch

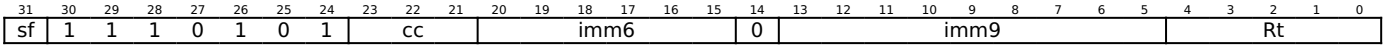
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields sf	cc	Instruction Details	Condition
	10x	UNALLOCATED	-
0	000	CB<cc> (register) — 32-bit greater than	FEAT_CMPBR
0	001	CB<cc> (register) — 32-bit greater than or equal	FEAT_CMPBR
0	010	CB<cc> (register) — 32-bit higher	FEAT_CMPBR
0	011	CB<cc> (register) — 32-bit higher or same	FEAT_CMPBR
0	110	CB<cc> (register) — 32-bit equal	FEAT_CMPBR
0	111	CB<cc> (register) — 32-bit not equal	FEAT_CMPBR
1	000	CB<cc> (register) — 64-bit greater than	FEAT_CMPBR
1	001	CB<cc> (register) — 64-bit greater than or equal	FEAT_CMPBR
1	010	CB<cc> (register) — 64-bit higher	FEAT_CMPBR
1	011	CB<cc> (register) — 64-bit higher or same	FEAT_CMPBR
1	110	CB<cc> (register) — 64-bit equal	FEAT_CMPBR
1	111	CB<cc> (register) — 64-bit not equal	FEAT_CMPBR

Compare register with immediate and branch

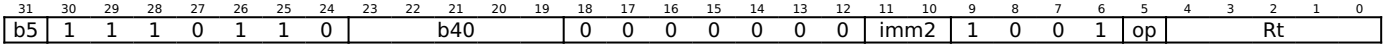
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields sf	cc	Instruction Details	Condition
	10x	UNALLOCATED	-
0	000	CB<cc> (immediate) — 32-bit greater than	FEAT_CMPBR
0	001	CB<cc> (immediate) — 32-bit less than	FEAT_CMPBR
0	010	CB<cc> (immediate) — 32-bit higher	FEAT_CMPBR
0	011	CB<cc> (immediate) — 32-bit lower	FEAT_CMPBR
0	110	CB<cc> (immediate) — 32-bit equal	FEAT_CMPBR
0	111	CB<cc> (immediate) — 32-bit not equal	FEAT_CMPBR
1	000	CB<cc> (immediate) — 64-bit greater than	FEAT_CMPBR
1	001	CB<cc> (immediate) — 64-bit less than	FEAT_CMPBR
1	010	CB<cc> (immediate) — 64-bit higher	FEAT_CMPBR
1	011	CB<cc> (immediate) — 64-bit lower	FEAT_CMPBR
1	110	CB<cc> (immediate) — 64-bit equal	FEAT_CMPBR
1	111	CB<cc> (immediate) — 64-bit not equal	FEAT_CMPBR

Test bit and fault

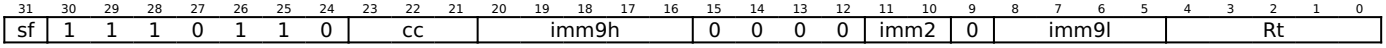
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields op	Instruction Details	Condition
0	TFLTZ	FEAT_CFLT
1	TFLTNZ	FEAT_CFLT

Compare and fault (immediate)

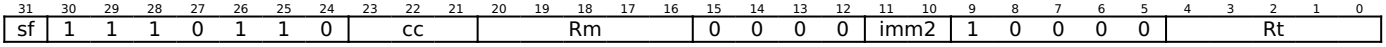
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields sf	Decode fields cc	Instruction Details	Condition
	11x	UNALLOCATED	-
0	000	CFLT<cc> (immediate) — 32-bit greater than	FEAT_CFLT
0	001	CFLT<cc> (immediate) — 32-bit less than	FEAT_CFLT
0	010	CFLT<cc> (immediate) — 32-bit higher	FEAT_CFLT
0	011	CFLT<cc> (immediate) — 32-bit lower	FEAT_CFLT
0	100	CFLT<cc> (immediate) — 32-bit equal	FEAT_CFLT
0	101	CFLT<cc> (immediate) — 32-bit not equal	FEAT_CFLT
1	000	CFLT<cc> (immediate) — 64-bit greater than	FEAT_CFLT
1	001	CFLT<cc> (immediate) — 64-bit less than	FEAT_CFLT
1	010	CFLT<cc> (immediate) — 64-bit higher	FEAT_CFLT
1	011	CFLT<cc> (immediate) — 64-bit lower	FEAT_CFLT
1	100	CFLT<cc> (immediate) — 64-bit equal	FEAT_CFLT
1	101	CFLT<cc> (immediate) — 64-bit not equal	FEAT_CFLT

Compare and fault (register)

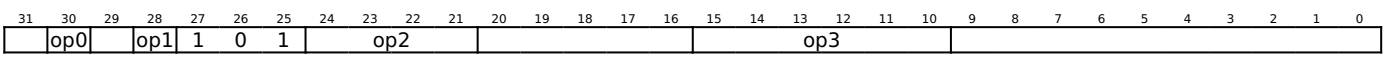
The encodings in this section are decoded from Branches, Exception Generating and System instructions.



Decode fields sf	Decode fields cc	Instruction Details	Condition
	10x	UNALLOCATED	-
0	000	CFLT<cc> (register) — 32-bit greater than	FEAT_CFLT
0	001	CFLT<cc> (register) — 32-bit greater than or equal	FEAT_CFLT
0	010	CFLT<cc> (register) — 32-bit higher	FEAT_CFLT
0	011	CFLT<cc> (register) — 32-bit higher or same	FEAT_CFLT
0	110	CFLT<cc> (register) — 32-bit equal	FEAT_CFLT
0	111	CFLT<cc> (register) — 32-bit not equal	FEAT_CFLT
1	000	CFLT<cc> (register) — 64-bit greater than	FEAT_CFLT
1	001	CFLT<cc> (register) — 64-bit greater than or equal	FEAT_CFLT
1	010	CFLT<cc> (register) — 64-bit higher	FEAT_CFLT
1	011	CFLT<cc> (register) — 64-bit higher or same	FEAT_CFLT
1	110	CFLT<cc> (register) — 64-bit equal	FEAT_CFLT
1	111	CFLT<cc> (register) — 64-bit not equal	FEAT_CFLT

Data Processing -- Register

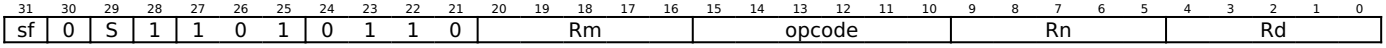
The encodings in this section are decoded from A64 instruction set encoding.



Decode fields				Instruction details
op0	op1	op2	op3	
0	1	0110		Data-processing (2 source)
1	1	0110		Data-processing (1 source)
	0	0xxx		Logical (shifted register)
	0	1xx0		Add/subtract (shifted register)
	0	1xx1		Add/subtract (extended register)
	1	0000	000000	Add/subtract (with carry)
	1	0000	001xxx	Add/subtract (checked pointer)
	1	0000	011xxx	UNALLOCATED
	1	0000	100000	UNALLOCATED
	1	0000	1x1xxx	UNALLOCATED
	1	0000	x00001	Rotate right into flags
	1	0000	x0010x	UNALLOCATED
	1	0000	x10x0x	UNALLOCATED
	1	0000	xx0010	Evaluate into flags
	1	0000	xx0011	UNALLOCATED
	1	0000	xx011x	UNALLOCATED
	1	0010	xxxx0x	Conditional compare (register)
	1	0010	xxxx1x	Conditional compare (immediate)
	1	0100		Conditional select
	1	0xx1		UNALLOCATED
	1	1xxx		Data-processing (3 source)

Data-processing (2 source)

The encodings in this section are decoded from Data Processing -- Register.

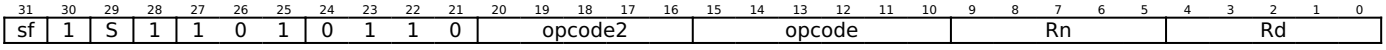


Decode fields			Instruction Details	Condition
sf	S	opcode		
		1xxxxx	UNALLOCATED	-
	0	00011x	UNALLOCATED	-
	1	00001x	UNALLOCATED	-
	1	0001xx	UNALLOCATED	-
	1	001xxx	UNALLOCATED	-
	1	01xxxx	UNALLOCATED	-
0		00000x	UNALLOCATED	-
0	0	0x11xx	UNALLOCATED	-
0	0	000010	UDIV (quotient) — 32-bit	-
0	0	000011	SDIV (quotient) — 32-bit	-
0	0	00010x	UNALLOCATED	-
0	0	001000	LSLV — 32-bit	-
0	0	001001	LSRV — 32-bit	-
0	0	001010	ASRV — 32-bit	-
0	0	001011	RORV — 32-bit	-
0	0	010x11	UNALLOCATED	-
0	0	010000	CRC32B, CRC32H, CRC32W, CRC32X — CRC32B	FEAT_CRC32
0	0	010001	CRC32B, CRC32H, CRC32W, CRC32X — CRC32H	FEAT_CRC32
0	0	010010	CRC32B, CRC32H, CRC32W, CRC32X — CRC32W	FEAT_CRC32
0	0	010100	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CB	FEAT_CRC32
0	0	010101	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CH	FEAT_CRC32
0	0	010110	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CW	FEAT_CRC32
0	0	011000	SMAX (register) — 32-bit	FEAT_CSSC
0	0	011001	UMAX (register) — 32-bit	FEAT_CSSC
0	0	011010	SMIN (register) — 32-bit	FEAT_CSSC
0	0	011011	UMIN (register) — 32-bit	FEAT_CSSC
1		000001	UNALLOCATED	-
1	0	000000	SUBP	FEAT_MTE
1	0	000010	UDIV (quotient) — 64-bit	-
1	0	000011	SDIV (quotient) — 64-bit	-
1	0	000100	IRG	FEAT_MTE
1	0	000101	GMI	FEAT_MTE
1	0	001000	LSLV — 64-bit	-
1	0	001001	LSRV — 64-bit	-
1	0	001010	ASRV — 64-bit	-
1	0	001011	RORV — 64-bit	-
1	0	001100	PACGA	FEAT_PAuth
1	0	001101	UNALLOCATED	-
1	0	00111x	UNALLOCATED	-
1	0	010xx0	UNALLOCATED	-
1	0	010001	UNALLOCATED	-
1	0	010011	CRC32B, CRC32H, CRC32W, CRC32X — CRC32X	FEAT_CRC32
1	0	010101	UNALLOCATED	-
1	0	010111	CRC32CB, CRC32CH, CRC32CW, CRC32CX — CRC32CX	FEAT_CRC32
1	0	011000	SMAX (register) — 64-bit	FEAT_CSSC

Decode fields			Instruction Details	Condition
sf	S	opcode		
1	0	011001	UMAX (register) — 64-bit	FEAT_CSSC
1	0	011010	SMIN (register) — 64-bit	FEAT_CSSC
1	0	011011	UMIN (register) — 64-bit	FEAT_CSSC
1	0	0111xx	UNALLOCATED	-
1	1	000000	SUBPS	FEAT_MTE

Data-processing (1 source)

The encodings in this section are decoded from Data Processing -- Register.

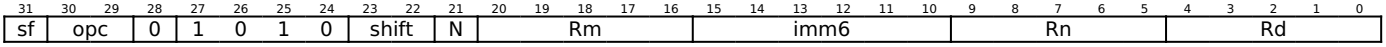


Decode fields						Instruction Details	Condition
sf	S	opcode2	opcode	Rn	Rd		
	0	00000	001001			UNALLOCATED	-
	0	00000	00101x			UNALLOCATED	-
	0	00000	0011xx			UNALLOCATED	-
	0	00000	01xxxx			UNALLOCATED	-
	0	00000	1xxxxx			UNALLOCATED	-
	0	!= 0000x				UNALLOCATED	-
	1					UNALLOCATED	-
0	0	00000	000000			RBIT — 32-bit	-
0	0	00000	000001			REV16 — 32-bit	-
0	0	00000	000010			REV — 32-bit	-
0	0	00000	000011			UNALLOCATED	-
0	0	00000	000100			CLZ — 32-bit	-
0	0	00000	000101			CLS — 32-bit	-
0	0	00000	000110			CTZ — 32-bit	FEAT_CSSC
0	0	00000	000111			CNT — 32-bit	FEAT_CSSC
0	0	00000	001000			ABS — 32-bit	FEAT_CSSC
0	0	00001				UNALLOCATED	-
1	0	00000	000000			RBIT — 64-bit	-
1	0	00000	000001			REV16 — 64-bit	-
1	0	00000	000010			REV32	-
1	0	00000	000011			REV — 64-bit	-
1	0	00000	000100			CLZ — 64-bit	-
1	0	00000	000101			CLS — 64-bit	-
1	0	00000	000110			CTZ — 64-bit	FEAT_CSSC
1	0	00000	000111			CNT — 64-bit	FEAT_CSSC
1	0	00000	001000			ABS — 64-bit	FEAT_CSSC
1	0	00001	000000			PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIA	FEAT_PAuth
1	0	00001	000001			PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIB	FEAT_PAuth
1	0	00001	000010			PACDA, PACDZA — PACDA	FEAT_PAuth
1	0	00001	000011			PACDB, PACDZB — PACDB	FEAT_PAuth
1	0	00001	000100			AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIA	FEAT_PAuth
1	0	00001	000101			AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIB	FEAT_PAuth
1	0	00001	000110			AUTDA, AUTDZA — AUTDA	FEAT_PAuth
1	0	00001	000111			AUTDB, AUTDZB — AUTDB	FEAT_PAuth
1	0	00001	001xxx	!= 11111		UNALLOCATED	-
1	0	00001	001000	11111		PACIA, PACIA1716, PACIASP, PACIAZ, PACIZA — PACIZA	FEAT_PAuth
1	0	00001	001001	11111		PACIB, PACIB1716, PACIBSP, PACIBZ, PACIZB — PACIZB	FEAT_PAuth
1	0	00001	001010	11111		PACDA, PACDZA — PACDZA	FEAT_PAuth
1	0	00001	001011	11111		PACDB, PACDZB — PACDZB	FEAT_PAuth
1	0	00001	001100	11111		AUTIA, AUTIA1716, AUTIASP, AUTIAZ, AUTIZA — AUTIZA	FEAT_PAuth

Decode fields						Instruction Details	Condition
sf	S	opcode2	opcode	Rn	Rd		
1	0	00001	001101	11111		AUTIB, AUTIB1716, AUTIBSP, AUTIBZ, AUTIZB — AUTIZB	FEAT_PAuth
1	0	00001	001110	11111		AUTDA, AUTDZA — AUTDZA	FEAT_PAuth
1	0	00001	001111	11111		AUTDB, AUTDZB — AUTDZB	FEAT_PAuth
1	0	00001	01000x	!= 11111		UNALLOCATED	-
1	0	00001	010000	11111		XPACD, XPACI, XPACLRI — XPACI	FEAT_PAuth
1	0	00001	010001	11111		XPACD, XPACI, XPACLRI — XPACD	FEAT_PAuth
1	0	00001	01001x			UNALLOCATED	-
1	0	00001	0101xx			UNALLOCATED	-
1	0	00001	011xxx			UNALLOCATED	-
1	0	00001	10xxxx		!= 11110	UNALLOCATED	-
1	0	00001	1000xx	!= 11111	11110	UNALLOCATED	-
1	0	00001	100000	11111	11110	PACNBIASPPC	FEAT_PAuth_LR
1	0	00001	100001	11111	11110	PACNBIBSPPC	FEAT_PAuth_LR
1	0	00001	100010	11111	11110	PACIA171615	FEAT_PAuth_LR
1	0	00001	100011	11111	11110	PACIB171615	FEAT_PAuth_LR
1	0	00001	100100		11110	AUTIASPPCR	FEAT_PAuth_LR
1	0	00001	100101		11110	AUTIBSPPCR	FEAT_PAuth_LR
1	0	00001	10011x		11110	UNALLOCATED	-
1	0	00001	101xxx	!= 11111	11110	UNALLOCATED	-
1	0	00001	101000	11111	11110	PACIASPPC	FEAT_PAuth_LR
1	0	00001	101001	11111	11110	PACIBSPPC	FEAT_PAuth_LR
1	0	00001	10101x	11111	11110	UNALLOCATED	-
1	0	00001	10110x	11111	11110	UNALLOCATED	-
1	0	00001	101110	11111	11110	AUTIA171615	FEAT_PAuth_LR
1	0	00001	101111	11111	11110	AUTIB171615	FEAT_PAuth_LR
1	0	00001	11xxxx			UNALLOCATED	-

Logical (shifted register)

The encodings in this section are decoded from Data Processing -- Register.



Decode fields			Instruction Details
sf	opc	N	
0	00	0	AND (shifted register) — 32-bit
0	00	1	BIC (shifted register) — 32-bit
0	01	0	ORR (shifted register) — 32-bit
0	01	1	ORN (shifted register) — 32-bit
0	10	0	EOR (shifted register) — 32-bit
0	10	1	EON (shifted register) — 32-bit
0	11	0	ANDS (shifted register) — 32-bit
0	11	1	BICS (shifted register) — 32-bit
1	00	0	AND (shifted register) — 64-bit
1	00	1	BIC (shifted register) — 64-bit
1	01	0	ORR (shifted register) — 64-bit
1	01	1	ORN (shifted register) — 64-bit
1	10	0	EOR (shifted register) — 64-bit
1	10	1	EON (shifted register) — 64-bit
1	11	0	ANDS (shifted register) — 64-bit
1	11	1	BICS (shifted register) — 64-bit

Add/subtract (shifted register)

The encodings in this section are decoded from Data Processing -- Register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	0	1	0	1	1	shift		0	Rm					imm6						Rn					Rd				

Decode fields			Instruction Details
sf	op	S	
0	0	0	ADD (shifted register) — 32-bit
0	0	1	ADDS (shifted register) — 32-bit
0	1	0	SUB (shifted register) — 32-bit
0	1	1	SUBS (shifted register) — 32-bit
1	0	0	ADD (shifted register) — 64-bit
1	0	1	ADDS (shifted register) — 64-bit
1	1	0	SUB (shifted register) — 64-bit
1	1	1	SUBS (shifted register) — 64-bit

Add/subtract (extended register)

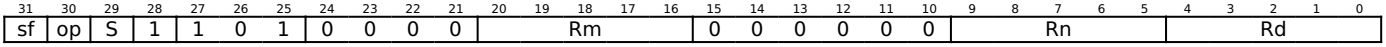
The encodings in this section are decoded from Data Processing -- Register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	0	1	0	1	1	opt		1	Rm					option			imm3			Rn					Rd				

Decode fields				Instruction Details
sf	op	S	opt	
			!= 00	UNALLOCATED
0	0	0	00	ADD (extended register) — 32-bit
0	0	1	00	ADDS (extended register) — 32-bit
0	1	0	00	SUB (extended register) — 32-bit
0	1	1	00	SUBS (extended register) — 32-bit
1	0	0	00	ADD (extended register) — 64-bit
1	0	1	00	ADDS (extended register) — 64-bit
1	1	0	00	SUB (extended register) — 64-bit
1	1	1	00	SUBS (extended register) — 64-bit

Add/subtract (with carry)

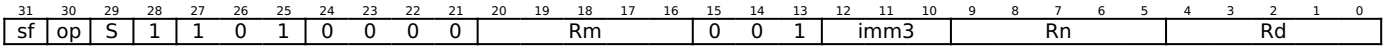
The encodings in this section are decoded from Data Processing -- Register.



Decode fields			Instruction Details
sf	op	S	
0	0	0	ADC — 32-bit
0	0	1	ADCS — 32-bit
0	1	0	SBC — 32-bit
0	1	1	SBCS — 32-bit
1	0	0	ADC — 64-bit
1	0	1	ADCS — 64-bit
1	1	0	SBC — 64-bit
1	1	1	SBCS — 64-bit

Add/subtract (checked pointer)

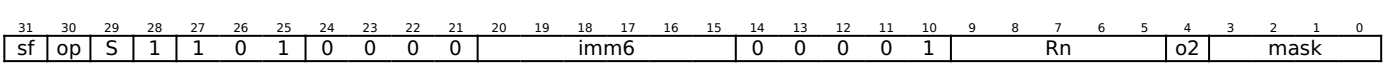
The encodings in this section are decoded from Data Processing -- Register.



Decode fields			Instruction Details	Condition
sf	op	S		
0			UNALLOCATED	-
1		1	UNALLOCATED	-
1	0	0	ADDPT	FEAT_CPA
1	1	0	SUBPT	FEAT_CPA

Rotate right into flags

The encodings in this section are decoded from Data Processing -- Register.



Decode fields				Instruction Details	Condition
sf	op	S	o2		
0				UNALLOCATED	-
1	0	0		UNALLOCATED	-
1	0	1	0	RMIF	FEAT_FlagM
1	0	1	1	UNALLOCATED	-
1	1			UNALLOCATED	-

Evaluate into flags

The encodings in this section are decoded from Data Processing -- Register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	0	0	0	opcode2						sz	0	0	1	0	Rn						o3	mask		

Decode fields							Instruction Details	Condition
sf	op	S	opcode2	sz	o3	mask		
0	0	0					UNALLOCATED	-
0	0	1	000000		0	!= 1101	UNALLOCATED	-
0	0	1	000000		1		UNALLOCATED	-
0	0	1	000000	0	0	1101	SETF8, SETF16 — SETF8	FEAT_FlagM
0	0	1	000000	1	0	1101	SETF8, SETF16 — SETF16	FEAT_FlagM
0	0	1	!= 000000				UNALLOCATED	-
0	1						UNALLOCATED	-
1							UNALLOCATED	-

Conditional compare (register)

The encodings in this section are decoded from Data Processing -- Register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	0	1	0	Rm					cond				0	o2	Rn					o3	nzcv			

Decode fields					Instruction Details
sf	op	S	o2	o3	
		0			UNALLOCATED
		1	0	1	UNALLOCATED
		1	1		UNALLOCATED
0	0	1	0	0	CCMN (register) — 32-bit
0	1	1	0	0	CCMP (register) — 32-bit
1	0	1	0	0	CCMN (register) — 64-bit
1	1	1	0	0	CCMP (register) — 64-bit

Conditional compare (immediate)

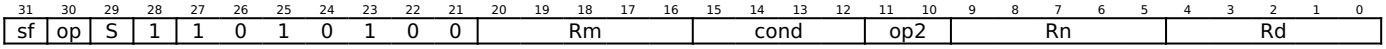
The encodings in this section are decoded from Data Processing -- Register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sf	op	S	1	1	0	1	0	0	1	0	imm5					cond				1	o2	Rn					o3	nzcv			

Decode fields					Instruction Details
sf	op	S	o2	o3	
		0			UNALLOCATED
		1	0	1	UNALLOCATED
		1	1		UNALLOCATED
0	0	1	0	0	CCMN (immediate) — 32-bit
0	1	1	0	0	CCMP (immediate) — 32-bit
1	0	1	0	0	CCMN (immediate) — 64-bit
1	1	1	0	0	CCMP (immediate) — 64-bit

Conditional select

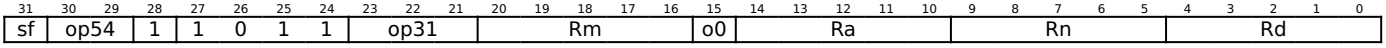
The encodings in this section are decoded from Data Processing -- Register.



Decode fields				Instruction Details
sf	op	S	op2	
		0	1x	UNALLOCATED
		1		UNALLOCATED
0	0	0	00	CSEL — 32-bit
0	0	0	01	CSINC — 32-bit
0	1	0	00	CSINV — 32-bit
0	1	0	01	CSNEG — 32-bit
1	0	0	00	CSEL — 64-bit
1	0	0	01	CSINC — 64-bit
1	1	0	00	CSINV — 64-bit
1	1	0	01	CSNEG — 64-bit

Data-processing (3 source)

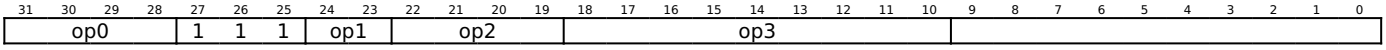
The encodings in this section are decoded from Data Processing -- Register.



Decode fields				Instruction Details	Condition
sf	op54	op31	o0		
	00	100		UNALLOCATED	-
	!= 00			UNALLOCATED	-
0	00	!= x00		UNALLOCATED	-
0	00	000	0	MADD — 32-bit	-
0	00	000	1	MSUB — 32-bit	-
1	00	x10	1	UNALLOCATED	-
1	00	000	0	MADD — 64-bit	-
1	00	000	1	MSUB — 64-bit	-
1	00	001	0	SMADDL	-
1	00	001	1	SMSUBL	-
1	00	010	0	SMULH	-
1	00	011	0	MADDPT	FEAT_CPA
1	00	011	1	MSUBPT	FEAT_CPA
1	00	101	0	UMADDL	-
1	00	101	1	UMSUBL	-
1	00	110	0	UMULH	-
1	00	111		UNALLOCATED	-

Data Processing -- Scalar Floating-Point and Advanced SIMD

The encodings in this section are decoded from A64 instruction set encoding.

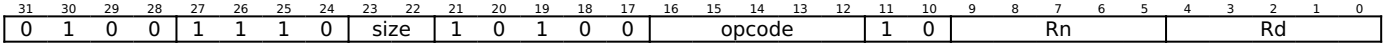


Decode fields				Instruction details	Condition
op0	op1	op2	op3		
00x0	0x	x101	00xxxxx10	UNALLOCATED	-
00x0	11		xxxxxxxx1	UNALLOCATED	-
0100	0x	x101	00xxxxx10	Cryptographic AES	-
0101	0x	x0xx	xxx0xxx00	Cryptographic three-register SHA	-
0101	0x	x0xx	xxx0xxx10	UNALLOCATED	-
0101	0x	x0xx	xxx1xxxx0	UNALLOCATED	-
0101	0x	x101	00xxxxx10	Cryptographic two-register SHA	-
0111	0x	x0xx	xxxxxxxx0	UNALLOCATED	-
011x	0x	x101	00xxxxx10	UNALLOCATED	-
01x1	00	00xx	xxx0xxxx1	Advanced SIMD scalar copy	-
01x1	01	00xx	xxx0xxxx1	UNALLOCATED	-
01x1	0x	0111	00xxxxx10	UNALLOCATED	-
01x1	0x	10xx	xxx00xxx1	Advanced SIMD scalar three same FP16	-
01x1	0x	10xx	xxx01xxx1	UNALLOCATED	-
01x1	0x	1111	00xxxxx10	Advanced SIMD scalar two-register miscellaneous FP16	-
01x1	0x	x0xx	xxx1xxxx1	Advanced SIMD scalar three same extra	-
01x1	0x	x100	00xxxxx10	Advanced SIMD scalar two-register miscellaneous	-
01x1	0x	x110	00xxxxx10	Advanced SIMD scalar pairwise	-
01x1	0x	x1xx	01xxxxx10	UNALLOCATED	-
01x1	0x	x1xx	1xxxxxx10	UNALLOCATED	-
01x1	0x	x1xx	xxxxxxx00	Advanced SIMD scalar three different	-
01x1	0x	x1xx	xxxxxxxx1	Advanced SIMD scalar three same	-
01x1	10		xxxxxxxx1	Advanced SIMD scalar shift by immediate	-
01x1	1x		xxxxxxxx0	Advanced SIMD scalar x indexed element	-
01xx	11		xxxxxxxx1	UNALLOCATED	-
0x00	0x	x0xx	xxx0xxx00	Advanced SIMD table lookup	-
0x00	0x	x0xx	xxx0xxx10	Advanced SIMD permute	-
0x10	0x	x0xx	xxx0xxxx0	Advanced SIMD extract	-
0xx0	00	00xx	xxx0xxxx1	Advanced SIMD copy	-
0xx0	01	00xx	xxx0xxxx1	UNALLOCATED	-
0xx0	0x	0111	00xxxxx10	UNALLOCATED	-
0xx0	0x	10xx	xxx00xxx1	Advanced SIMD three same (FP16)	-
0xx0	0x	10xx	xxx01xxx1	UNALLOCATED	-
0xx0	0x	1111	00xxxxx10	Advanced SIMD two-register miscellaneous (FP16)	-
0xx0	0x	x0xx	xxx1xxxx0	UNALLOCATED	-
0xx0	0x	x0xx	xxx1xxxx1	Advanced SIMD three-register extension	-
0xx0	0x	x100	00xxxxx10	Advanced SIMD two-register miscellaneous	-
0xx0	0x	x110	00xxxxx10	Advanced SIMD across lanes	-
0xx0	0x	x1xx	01xxxxx10	UNALLOCATED	-
0xx0	0x	x1xx	1xxxxxx10	UNALLOCATED	-
0xx0	0x	x1xx	xxxxxxx00	Advanced SIMD three different	-
0xx0	0x	x1xx	xxxxxxxx1	Advanced SIMD three same	-
0xx0	10	0000	xxxxxxxx1	Advanced SIMD modified immediate	-
0xx0	10	!= 0000	xxxxxxxx1	Advanced SIMD shift by immediate	-
0xx0	1x		xxxxxxxx0	Advanced SIMD vector x indexed element	-

10x0				UNALLOCATED	-
1100	00	0xxx	xxx1xxxxx	UNALLOCATED	-
1100	00	10xx	xxx10xxxx	Cryptographic three-register, imm2	-
1100	00	10xx	xxx11xxxx	UNALLOCATED	-
1100	00	11xx	xxx1x00xx	Cryptographic three-register SHA 512	-
1100	00	11xx	xxx1x01xx	UNALLOCATED	-
1100	00	11xx	xxx1x1xxx	UNALLOCATED	-
1100	00		xxx0xxxxx	Cryptographic four-register	-
1100	01	00xx		XAR	FEAT_SHA3
1100	01	1000	0001000xx	Cryptographic two-register SHA 512	-
1100	01	1000	!= 0001000xx	UNALLOCATED	-
1100	01	1001		UNALLOCATED	-
1100	01	101x		UNALLOCATED	-
1100	01	x1xx		UNALLOCATED	-
1100	1x			UNALLOCATED	-
1110				UNALLOCATED	-
11x1				UNALLOCATED	-
x0x1	0x	x0xx		Conversion between floating-point and fixed-point	-
x0x1	0x	x1xx	xxx000000	Conversion between floating-point and integer	-
x0x1	0x	x1xx	xxx100000	UNALLOCATED	-
x0x1	0x	x1xx	xxxx10000	Floating-point data-processing (1 source)	-
x0x1	0x	x1xx	xxxxx1000	Floating-point compare	-
x0x1	0x	x1xx	xxxxxx100	Floating-point immediate	-
x0x1	0x	x1xx	xxxxxxx01	Floating-point conditional compare	-
x0x1	0x	x1xx	xxxxxxx10	Floating-point data-processing (2 source)	-
x0x1	0x	x1xx	xxxxxxx11	Floating-point conditional select	-
x0x1	1x			Floating-point data-processing (3 source)	-

Cryptographic AES

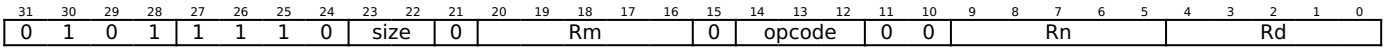
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
size	opcode		
00	00100	AESE	FEAT_AES
00	00101	AESD	FEAT_AES
00	00110	AESMC	FEAT_AES
00	00111	AESIMC	FEAT_AES
00	!= 001xx	UNALLOCATED	-
!= 00		UNALLOCATED	-

Cryptographic three-register SHA

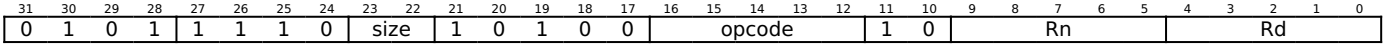
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
size	opcode		
00	000	SHA1C	FEAT_SHA1
00	001	SHA1P	FEAT_SHA1
00	010	SHA1M	FEAT_SHA1
00	011	SHA1SU0	FEAT_SHA1
00	100	SHA256H	FEAT_SHA256
00	101	SHA256H2	FEAT_SHA256
00	110	SHA256SU1	FEAT_SHA256
00	111	UNALLOCATED	-
!= 00		UNALLOCATED	-

Cryptographic two-register SHA

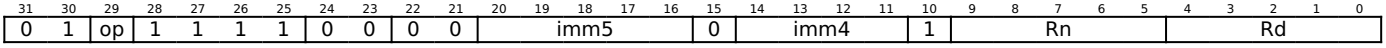
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
size	opcode		
00	00000	SHA1H	FEAT_SHA1
00	00001	SHA1SU1	FEAT_SHA1
00	00010	SHA256SU0	FEAT_SHA256
00	00011	UNALLOCATED	-
00	001xx	UNALLOCATED	-
00	01xxx	UNALLOCATED	-
00	1xxxx	UNALLOCATED	-
!= 00		UNALLOCATED	-

Advanced SIMD scalar copy

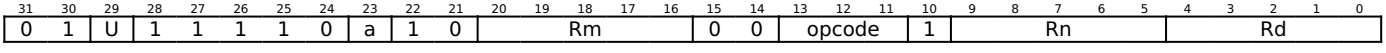
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
op	imm4		
0	0000	DUP (element)	FEAT_AdvSIMD
0	!= 0000	UNALLOCATED	-
1		UNALLOCATED	-

Advanced SIMD scalar three same FP16

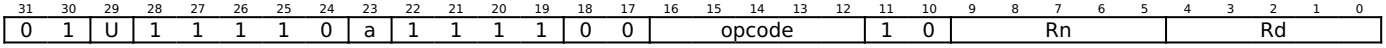
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	a	opcode		
		00x	UNALLOCATED	-
	0	010	UNALLOCATED	-
0		101	UNALLOCATED	-
0	0	011	FMULX	FEAT_AdvSIMD && FEAT_FP16
0	0	100	FCMEQ (register)	FEAT_AdvSIMD && FEAT_FP16
0	0	110	UNALLOCATED	-
0	0	111	FRECPS	FEAT_AdvSIMD && FEAT_FP16
0	1	x10	UNALLOCATED	-
0	1	011	UNALLOCATED	-
0	1	100	UNALLOCATED	-
0	1	111	FRSQRTS	FEAT_AdvSIMD && FEAT_FP16
1		011	UNALLOCATED	-
1		11x	UNALLOCATED	-
1	0	100	FCMGE (register)	FEAT_AdvSIMD && FEAT_FP16
1	0	101	FACGE	FEAT_AdvSIMD && FEAT_FP16
1	1	010	FABD	FEAT_AdvSIMD && FEAT_FP16
1	1	100	FCMGT (register)	FEAT_AdvSIMD && FEAT_FP16
1	1	101	FACGT	FEAT_AdvSIMD && FEAT_FP16

Advanced SIMD scalar two-register miscellaneous FP16

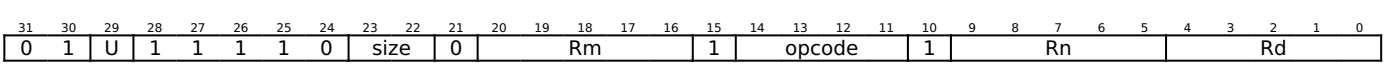
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	a	opcode		
		x0xxx	UNALLOCATED	-
		1100x	UNALLOCATED	-
	0	01xxx	UNALLOCATED	-
	1	010xx	UNALLOCATED	-
	1	11100	UNALLOCATED	-
0	0	11010	FCVTNS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11011	FCVTMS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11100	FCVTAS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11101	SCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	0	1111x	UNALLOCATED	-
0	1	01100	FCMGT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01101	FCMEQ (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01110	FCMLT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01111	UNALLOCATED	-
0	1	11010	FCVTPS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	1	11101	FRECPE	FEAT_AdvSIMD && FEAT_FP16
0	1	11110	UNALLOCATED	-
0	1	11111	FRECPX	FEAT_AdvSIMD && FEAT_FP16
1		1111x	UNALLOCATED	-
1	0	11010	FCVTNU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11011	FCVTMU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11100	FCVTAU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11101	UCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	01100	FCMGE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	01101	FCMLE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	0111x	UNALLOCATED	-
1	1	11010	FCVTPU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	11101	FRSQRTE	FEAT_AdvSIMD && FEAT_FP16

Advanced SIMD scalar three same extra

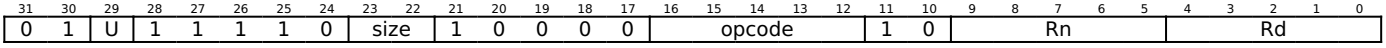
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
U	opcode		
0		UNALLOCATED	-
1	0000	SQRDMLAH (vector)	FEAT_RDM
1	0001	SQRDMLSH (vector)	FEAT_RDM
1	!= 000x	UNALLOCATED	-

Advanced SIMD scalar two-register miscellaneous

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

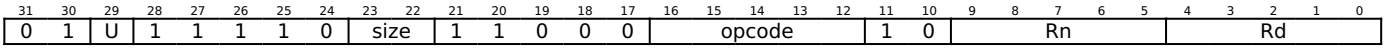

31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1 0
0 1 U 1 1 1 1 0 size 1 0 0 0 0 opcode 1 0 Rn Rd

Decode fields			Instruction Details	Condition
U	size	opcode		
		00xx0	UNALLOCATED	-
		00x01	UNALLOCATED	-
		1x000	UNALLOCATED	-
		10xx1	UNALLOCATED	-
		11001	UNALLOCATED	-
	0x	01xxx	UNALLOCATED	-
	0x	1111x	UNALLOCATED	-
	1x	01111	UNALLOCATED	-
	1x	11100	UNALLOCATED	-
	10	010x1	UNALLOCATED	-
	10	01000	UNALLOCATED	-
0		00011	SUQADD	FEAT_AdvSIMD
0		00111	SQABS	FEAT_AdvSIMD
0		10010	UNALLOCATED	-
0		10100	SQXTN, SQXTN2	FEAT_AdvSIMD
0		10110	UNALLOCATED	-
0	0x	11010	FCVTNS (vector)	FEAT_AdvSIMD
0	0x	11011	FCVTMS (vector)	FEAT_AdvSIMD
0	0x	11100	FCVTAS (vector)	FEAT_AdvSIMD
0	0x	11101	SCVTF (vector, integer)	FEAT_AdvSIMD
0	1x	01100	FCMGT (zero)	FEAT_AdvSIMD
0	1x	01101	FCMEQ (zero)	FEAT_AdvSIMD
0	1x	01110	FCMLT (zero)	FEAT_AdvSIMD
0	1x	11010	FCVTPS (vector)	FEAT_AdvSIMD
0	1x	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD
0	1x	11101	FRECPE	FEAT_AdvSIMD
0	1x	11110	UNALLOCATED	-
0	1x	11111	FRECPX	FEAT_AdvSIMD
0	10	01010	UNALLOCATED	-
0	11	01000	CMGT (zero)	FEAT_AdvSIMD
0	11	01001	CMEQ (zero)	FEAT_AdvSIMD
0	11	01010	CMLT (zero)	FEAT_AdvSIMD
0	11	01011	ABS	FEAT_AdvSIMD
1		00011	USQADD	FEAT_AdvSIMD
1		00111	SQNEG	FEAT_AdvSIMD
1		10010	SQXTUN, SQXTUN2	FEAT_AdvSIMD
1		10100	UQXTN, UQXTN2	FEAT_AdvSIMD
1	0x	11010	FCVTNU (vector)	FEAT_AdvSIMD
1	0x	11011	FCVTMU (vector)	FEAT_AdvSIMD
1	0x	11100	FCVTAU (vector)	FEAT_AdvSIMD
1	0x	11101	UCVTF (vector, integer)	FEAT_AdvSIMD
1	00	10110	UNALLOCATED	-
1	01	10110	FCVTXN, FCVTXN2	FEAT_AdvSIMD
1	1x	01x10	UNALLOCATED	-
1	1x	01100	FCMGE (zero)	FEAT_AdvSIMD

Decode fields			Instruction Details	Condition
U	size	opcode		
1	1x	01101	FCMLE (zero)	FEAT_AdvSIMD
1	1x	1x110	UNALLOCATED	-
1	1x	11010	FCVTPU (vector)	FEAT_AdvSIMD
1	1x	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD
1	1x	11101	FRSQRTE	FEAT_AdvSIMD
1	1x	11111	UNALLOCATED	-
1	11	01000	CMGE (zero)	FEAT_AdvSIMD
1	11	01001	CMLE (zero)	FEAT_AdvSIMD
1	11	01011	NEG (vector)	FEAT_AdvSIMD

Advanced SIMD scalar pairwise

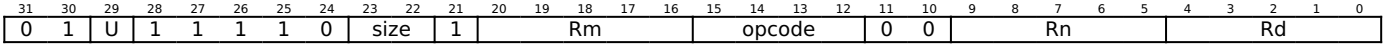
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	size	opcode		
		x0xxx	UNALLOCATED	-
		01110	UNALLOCATED	-
		111xx	UNALLOCATED	-
	10	01101	UNALLOCATED	-
0	x1	011x1	UNALLOCATED	-
0	x1	01100	UNALLOCATED	-
0	00	01100	FMAXNMP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	00	01101	FADDP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	00	01111	FMAXP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	10	01100	FMINNMP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	10	01111	FMINP (scalar) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
0	11	x100x	UNALLOCATED	-
0	11	x1010	UNALLOCATED	-
0	11	01011	UNALLOCATED	-
0	11	11011	ADDP (scalar)	FEAT_AdvSIMD
0	!= 11	x10xx	UNALLOCATED	-
1		x10xx	UNALLOCATED	-
1	0x	01100	FMAXNMP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	0x	01101	FADDP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	0x	01111	FMAXP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	1x	01100	FMINNMP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	1x	01111	FMINP (scalar) — Single-precision and double-precision	FEAT_AdvSIMD
1	11	01101	UNALLOCATED	-

Advanced SIMD scalar three different

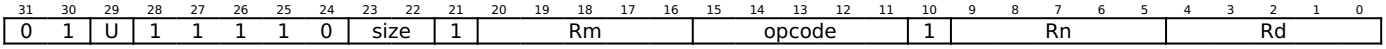
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
U	opcode		
0	0xxx	UNALLOCATED	-
0	1xx0	UNALLOCATED	-
0	1001	SQDMLAL, SQDMLAL2 (vector)	FEAT_AdvSIMD
0	1011	SQDMLSL, SQDMLSL2 (vector)	FEAT_AdvSIMD
0	1101	SQDMULL, SQDMULL2 (vector)	FEAT_AdvSIMD
0	1111	UNALLOCATED	-
1		UNALLOCATED	-

Advanced SIMD scalar three same

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

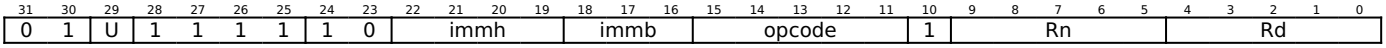


Decode fields			Instruction Details	Condition
U	size	opcode		
		x0011	UNALLOCATED	-
		011x1	UNALLOCATED	-
		11001	UNALLOCATED	-
		11110	UNALLOCATED	-
	x0	0x1x0	UNALLOCATED	-
	x1	00100	UNALLOCATED	-
	x1	011x0	UNALLOCATED	-
	0x	xx0x0	UNALLOCATED	-
	0x	10x01	UNALLOCATED	-
	01	00110	UNALLOCATED	-
	1x	10010	UNALLOCATED	-
	1x	11000	UNALLOCATED	-
	10	0x0x0	UNALLOCATED	-
	10	10x0x	UNALLOCATED	-
	11	00000	UNALLOCATED	-
	11	00010	UNALLOCATED	-
	11	10101	UNALLOCATED	-
	!= 10	10100	UNALLOCATED	-
	!= 11	00111	UNALLOCATED	-
0		00001	SQADD	FEAT_AdvSIMD
0		00101	SQSUB	FEAT_AdvSIMD
0		01001	SQSHL (register)	FEAT_AdvSIMD
0		01011	SQRSHL	FEAT_AdvSIMD
0		10110	SQDMULH (vector)	FEAT_AdvSIMD
0		10111	UNALLOCATED	-
0		11101	UNALLOCATED	-
0	0x	11011	FMULX	FEAT_AdvSIMD
0	0x	11100	FCMEQ (register)	FEAT_AdvSIMD
0	0x	11111	FRECPS	FEAT_AdvSIMD
0	1x	11010	UNALLOCATED	-
0	1x	11011	UNALLOCATED	-
0	1x	11100	UNALLOCATED	-
0	1x	11111	FRSQRTS	FEAT_AdvSIMD
0	11	00110	CMGT (register)	FEAT_AdvSIMD
0	11	00111	CMGE (register)	FEAT_AdvSIMD
0	11	01000	SSHL	FEAT_AdvSIMD
0	11	01010	SRSHL	FEAT_AdvSIMD
0	11	10000	ADD (vector)	FEAT_AdvSIMD
0	11	10001	CMTST	FEAT_AdvSIMD
1		00001	UQADD	FEAT_AdvSIMD
1		00101	UQSUB	FEAT_AdvSIMD
1		01001	UQSHL (register)	FEAT_AdvSIMD
1		01011	UQRSHL	FEAT_AdvSIMD
1		1x111	UNALLOCATED	-
1		10110	SQRDMULH (vector)	FEAT_AdvSIMD

Decode fields			Instruction Details	Condition
U	size	opcode		
1		11011	UNALLOCATED	-
1	0x	11100	FCMGE (register)	FEAT_AdvSIMD
1	0x	11101	FACGE	FEAT_AdvSIMD
1	1x	11010	FABD	FEAT_AdvSIMD
1	1x	11100	FCMGT (register)	FEAT_AdvSIMD
1	1x	11101	FACGT	FEAT_AdvSIMD
1	11	00110	CMHI (register)	FEAT_AdvSIMD
1	11	00111	CMHS (register)	FEAT_AdvSIMD
1	11	01000	USHL	FEAT_AdvSIMD
1	11	01010	URSHL	FEAT_AdvSIMD
1	11	10000	SUB (vector)	FEAT_AdvSIMD
1	11	10001	CMEQ (register)	FEAT_AdvSIMD

Advanced SIMD scalar shift by immediate

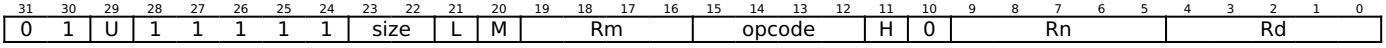
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	immh	opcode		
		0xxx1	UNALLOCATED	-
		101xx	UNALLOCATED	-
		11101	UNALLOCATED	-
	0xxx	x10x0	UNALLOCATED	-
	0xxx	00xx0	UNALLOCATED	-
	0xxx	110x1	UNALLOCATED	-
	0000	x1110	UNALLOCATED	-
	0000	1001x	UNALLOCATED	-
	0000	11100	UNALLOCATED	-
	0000	11111	UNALLOCATED	-
	1xxx	110xx	UNALLOCATED	-
	!= 0000	11110	UNALLOCATED	-
0		01100	UNALLOCATED	-
0		1000x	UNALLOCATED	-
0	1xxx	00000	SSHR	FEAT_AdvSIMD
0	1xxx	00010	SSRA	FEAT_AdvSIMD
0	1xxx	00100	SRSHR	FEAT_AdvSIMD
0	1xxx	00110	SRSRA	FEAT_AdvSIMD
0	1xxx	01000	UNALLOCATED	-
0	1xxx	01010	SHL	FEAT_AdvSIMD
0	!= 0000	01110	SQSHL (immediate)	FEAT_AdvSIMD
0	!= 0000	10010	SQSHRN, SQSHRN2	FEAT_AdvSIMD
0	!= 0000	10011	SQRSHRN, SQRSHRN2	FEAT_AdvSIMD
0	!= 0000	11100	SCVTF (vector, fixed-point)	FEAT_AdvSIMD
0	!= 0000	11111	FCVTZS (vector, fixed-point)	FEAT_AdvSIMD
1	0000	01100	UNALLOCATED	-
1	0000	1000x	UNALLOCATED	-
1	1xxx	00000	USHR	FEAT_AdvSIMD
1	1xxx	00010	USRA	FEAT_AdvSIMD
1	1xxx	00100	URSHR	FEAT_AdvSIMD
1	1xxx	00110	URSRA	FEAT_AdvSIMD
1	1xxx	01000	SRI	FEAT_AdvSIMD
1	1xxx	01010	SLI	FEAT_AdvSIMD
1	!= 0000	01100	SQSHLU	FEAT_AdvSIMD
1	!= 0000	01110	UQSHL (immediate)	FEAT_AdvSIMD
1	!= 0000	10000	SQSHRUN, SQSHRUN2	FEAT_AdvSIMD
1	!= 0000	10001	SQRSHRUN, SQRSHRUN2	FEAT_AdvSIMD
1	!= 0000	10010	UQSHRN, UQSHRN2	FEAT_AdvSIMD
1	!= 0000	10011	UQRSHRN, UQRSHRN2	FEAT_AdvSIMD
1	!= 0000	11100	UCVTF (vector, fixed-point)	FEAT_AdvSIMD
1	!= 0000	11111	FCVTZU (vector, fixed-point)	FEAT_AdvSIMD

Advanced SIMD scalar x indexed element

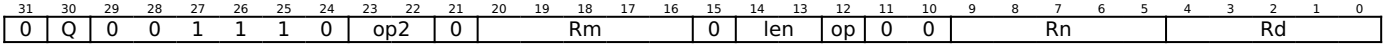
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	size	opcode		
	01	1001	UNALLOCATED	-
0		0xx0	UNALLOCATED	-
0		0011	SQDMLAL, SQDMLAL2 (by element)	FEAT_AdvSIMD
0		0111	SQDMLSL, SQDMLSL2 (by element)	FEAT_AdvSIMD
0		10x0	UNALLOCATED	-
0		1011	SQDMULL, SQDMULL2 (by element)	FEAT_AdvSIMD
0		1100	SQDMULH (by element)	FEAT_AdvSIMD
0		1101	SQRDMULH (by element)	FEAT_AdvSIMD
0		111x	UNALLOCATED	-
0	00	0001	FMLA (by element) — Scalar, half-precision	FEAT_AdvSIMD && FEAT_FP16
0	00	0101	FMLS (by element) — Scalar, half-precision	FEAT_AdvSIMD && FEAT_FP16
0	00	1001	FMUL (by element) — Scalar, half-precision	FEAT_AdvSIMD && FEAT_FP16
0	01	0x01	UNALLOCATED	-
0	1x	0001	FMLA (by element) — Scalar, single-precision and double-precision	FEAT_AdvSIMD
0	1x	0101	FMLS (by element) — Scalar, single-precision and double-precision	FEAT_AdvSIMD
0	1x	1001	FMUL (by element) — Scalar, single-precision and double-precision	FEAT_AdvSIMD
1		0xxx	UNALLOCATED	-
1		1xx0	UNALLOCATED	-
1		1011	UNALLOCATED	-
1		1101	SQRDMLAH (by element)	FEAT_RDM
1		1111	SQRDMLSH (by element)	FEAT_RDM
1	00	1001	FMULX (by element) — Scalar, half-precision	FEAT_AdvSIMD && FEAT_FP16
1	1x	1001	FMULX (by element) — Scalar, single-precision and double-precision	FEAT_AdvSIMD

Advanced SIMD table lookup

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Q	Decode fields op2	len	op	Instruction Details	Condition
	00	00	0	TBL — Single register table	FEAT_AdvSIMD
	00	00	1	TBX — Single register table	FEAT_AdvSIMD
	00	01	0	TBL — Two register table	FEAT_AdvSIMD
	00	01	1	TBX — Two register table	FEAT_AdvSIMD
	00	10	0	TBL — Three register table	FEAT_AdvSIMD
	00	10	1	TBX — Three register table	FEAT_AdvSIMD
	00	11	0	TBL — Four register table	FEAT_AdvSIMD
	00	11	1	TBX — Four register table	FEAT_AdvSIMD
0	!= 00			UNALLOCATED	-
1	01		1	LUTI4 — Halfword	FEAT_AdvSIMD && FEAT_LUT
1	01	x0	0	UNALLOCATED	-
1	01	x1	0	LUTI4 — Byte	FEAT_AdvSIMD && FEAT_LUT
1	10		0	UNALLOCATED	-
1	10		1	LUTI2 — Byte	FEAT_AdvSIMD && FEAT_LUT
1	11			LUTI2 — Halfword	FEAT_AdvSIMD && FEAT_LUT

Advanced SIMD permute

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	1	0	size		0	Rm					0	opcode			1	0	Rn					Rd				

Decode fields opcode	Instruction Details	Condition
x00	UNALLOCATED	-
001	UZP1	FEAT_AdvSIMD
010	TRN1	FEAT_AdvSIMD
011	ZIP1	FEAT_AdvSIMD
101	UZP2	FEAT_AdvSIMD
110	TRN2	FEAT_AdvSIMD
111	ZIP2	FEAT_AdvSIMD

Advanced SIMD extract

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	1	0	1	1	1	0	op2		0	Rm					0	imm4				0	Rn					Rd				

Decode fields op2	Instruction Details	Condition
00	EXT	FEAT_AdvSIMD
!= 00	UNALLOCATED	-

Advanced SIMD copy

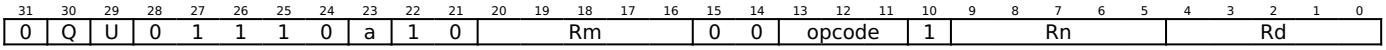
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	op	0	1	1	1	0	0	0	0	imm5					0	imm4				1	Rn					Rd				

Q	op	Decode fields imm5	imm4	Instruction Details	Condition
	0		0000	DUP (element)	FEAT_AdvSIMD
	0		0001	DUP (general)	FEAT_AdvSIMD
	0		01x0	UNALLOCATED	-
	0		1xxx	UNALLOCATED	-
0	0		001x	UNALLOCATED	-
0	0		0101	SMOV — 32-bit	FEAT_AdvSIMD
0	0		0111	UMOV — 32-bit	FEAT_AdvSIMD
0	1			UNALLOCATED	-
1	0		0010	UNALLOCATED	-
1	0		0011	INS (general)	FEAT_AdvSIMD
1	0		0101	SMOV — 64-bit	FEAT_AdvSIMD
1	0	x1000	0111	UMOV — 64-bit	FEAT_AdvSIMD
1	0	!= x1000	0111	UNALLOCATED	-
1	1			INS (element)	FEAT_AdvSIMD

Advanced SIMD three same (FP16)

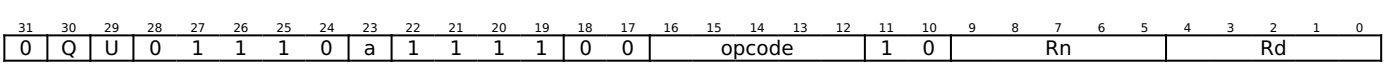
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	a	opcode		
0	0	000	FMAXNM (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	001	FMLA (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	010	FADD (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	011	FMULX	FEAT_AdvSIMD && FEAT_FP16
0	0	100	FCMEQ (register)	FEAT_AdvSIMD && FEAT_FP16
0	0	101	UNALLOCATED	-
0	0	110	FMAX (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	111	FRECPS	FEAT_AdvSIMD && FEAT_FP16
0	1	000	FMINNM (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	001	FMLS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	010	FSUB (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	011	FAMAX	FEAT_AdvSIMD && FEAT_FAMINMAX
0	1	10x	UNALLOCATED	-
0	1	110	FMIN (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	111	FRSQRTS	FEAT_AdvSIMD && FEAT_FP16
1		001	UNALLOCATED	-
1	0	000	FMAXNMP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	010	FADDP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	011	FMUL (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	100	FCMGE (register)	FEAT_AdvSIMD && FEAT_FP16
1	0	101	FACGE	FEAT_AdvSIMD && FEAT_FP16
1	0	110	FMAXP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	111	FDIV (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	000	FMINNMP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	010	FABD	FEAT_AdvSIMD && FEAT_FP16
1	1	011	FAMIN	FEAT_AdvSIMD && FEAT_FAMINMAX
1	1	100	FCMGT (register)	FEAT_AdvSIMD && FEAT_FP16
1	1	101	FACGT	FEAT_AdvSIMD && FEAT_FP16
1	1	110	FMINP (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	111	FSCALE	FEAT_FP8

Advanced SIMD two-register miscellaneous (FP16)

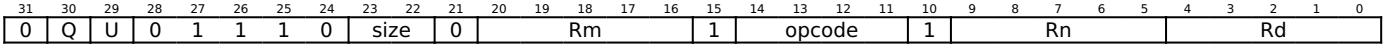
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
U	a	opcode		
		x0xxx	UNALLOCATED	-
	0	01xxx	UNALLOCATED	-
	0	1111x	UNALLOCATED	-
	1	010xx	UNALLOCATED	-
	1	11100	UNALLOCATED	-
0	0	11000	FRINTN (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11001	FRINTM (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11010	FCVTNS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11011	FCVTMS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11100	FCVTAS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	0	11101	SCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	1	01100	FCMGT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01101	FCMEQ (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01110	FCMLT (zero)	FEAT_AdvSIMD && FEAT_FP16
0	1	01111	FABS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11000	FRINTP (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11001	FRINTZ (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11010	FCVTPS (vector)	FEAT_AdvSIMD && FEAT_FP16
0	1	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
0	1	11101	FRECPE	FEAT_AdvSIMD && FEAT_FP16
0	1	1111x	UNALLOCATED	-
1	0	11000	FRINTA (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11001	FRINTX (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11010	FCVTNU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11011	FCVTMU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11100	FCVTAU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	0	11101	UCVTF (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	x1110	UNALLOCATED	-
1	1	01100	FCMGE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	01101	FCMLE (zero)	FEAT_AdvSIMD && FEAT_FP16
1	1	01111	FNEG (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11000	UNALLOCATED	-
1	1	11001	FRINTI (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11010	FCVTPU (vector)	FEAT_AdvSIMD && FEAT_FP16
1	1	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD && FEAT_FP16
1	1	11101	FRSQRTE	FEAT_AdvSIMD && FEAT_FP16
1	1	11111	FSQRT (vector)	FEAT_AdvSIMD && FEAT_FP16

Advanced SIMD three-register extension

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

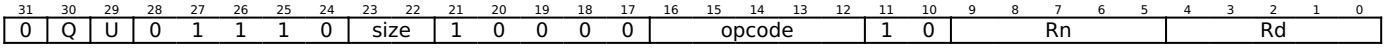


Decode fields				Instruction Details	Condition
Q	U	size	opcode		
	0		0000	UNALLOCATED	-
	0		0010	SDOT (vector)	FEAT_DotProd
	0	x0	0001	UNALLOCATED	-
	0	x0	1001	UNALLOCATED	-
	0	x0	1011	UNALLOCATED	-
	0	x1	x0x1	UNALLOCATED	-
	0	0x	1010	UNALLOCATED	-
	0	0x	1100	UNALLOCATED	-
	0	00	0011	UNALLOCATED	-
	0	00	1110	FCVTN, FCVTN2 (single-precision to 8-bit floating-point)	FEAT_FP8
	0	00	1111	FDOT (8-bit floating-point to single-precision, vector)	FEAT_FP8DOT4
	0	01	1110	FCVTN (half-precision to 8-bit floating-point)	FEAT_FP8
	0	01	1111	FDOT (8-bit floating-point to half-precision, vector)	FEAT_FP8DOT2
	0	1x	1xx0	UNALLOCATED	-
	0	10	0011	USDOT (vector)	FEAT_I8MM
	0	10	1111	FDOT (half-precision to single-precision, vector)	FEAT_F16F32DOT
	1		0000	SQRDMLAH (vector)	FEAT_RDM
	1		0001	SQRDMLSH (vector)	FEAT_RDM
	1		0010	UDOT (vector)	FEAT_DotProd
	1		0011	UNALLOCATED	-
	1		10xx	FCMLA	FEAT_FCMA
	1		11x0	FCADD	FEAT_FCMA
	1	x0	1111	UNALLOCATED	-
	1	01	1111	BFDOT (vector)	FEAT_BF16
	1	11	1111	BFMLALB, BFMLALT (vector)	FEAT_BF16
0			01xx	UNALLOCATED	-
0			1101	UNALLOCATED	-
0	0	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLBB	FEAT_FP8FMA
0	0	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLBT	FEAT_FP8FMA
0	0	11	1111	FMLALB, FMLALT (vector) — FMLALB	FEAT_FP8FMA
1		10	011x	UNALLOCATED	-
1		!= 10	01xx	UNALLOCATED	-
1	0	x0	1101	UNALLOCATED	-
1	0	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLTB	FEAT_FP8FMA
1	0	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (vector) — FMLALLTT	FEAT_FP8FMA
1	0	01	1101	FMMLA (widening, half-precision to single-precision)	FEAT_F16F32MM
1	0	10	0100	SMMLA (vector)	FEAT_I8MM
1	0	10	0101	USMMLA (vector)	FEAT_I8MM
1	0	11	1101	FMMLA (non-widening)	FEAT_F16MM
1	0	11	1111	FMLALB, FMLALT (vector) — FMLALT	FEAT_FP8FMA
1	1	00	1101	FMMLA (widening, 8-bit floating-point to half-precision)	FEAT_F8F16MM
1	1	01	1101	BFMMLA (widening)	FEAT_BF16
1	1	10	0100	UMMLA (vector)	FEAT_I8MM
1	1	10	0101	UNALLOCATED	-
1	1	10	1101	FMMLA (widening, 8-bit floating-point to single-precision)	FEAT_F8F32MM

Decode fields				Instruction Details	Condition
Q	U	size	opcode		
1	1	11	1101	UNALLOCATED	-

Advanced SIMD two-register miscellaneous

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

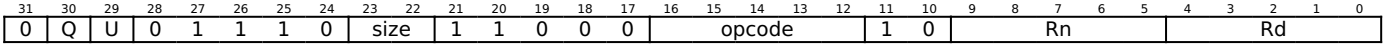


Decode fields			Instruction Details	Condition
U	size	opcode		
		1000x	UNALLOCATED	-
		10101	UNALLOCATED	-
	0x	011xx	UNALLOCATED	-
	10	11110	UNALLOCATED	-
	11	1x110	UNALLOCATED	-
0		00000	REV64	FEAT_AdvSIMD
0		00001	REV16 (vector)	FEAT_AdvSIMD
0		00010	SADDLP	FEAT_AdvSIMD
0		00011	SUQADD	FEAT_AdvSIMD
0		00100	CLS (vector)	FEAT_AdvSIMD
0		00101	CNT	FEAT_AdvSIMD
0		00110	SADALP	FEAT_AdvSIMD
0		00111	SQABS	FEAT_AdvSIMD
0		01000	CMGT (zero)	FEAT_AdvSIMD
0		01001	CMEQ (zero)	FEAT_AdvSIMD
0		01010	CMLT (zero)	FEAT_AdvSIMD
0		01011	ABS	FEAT_AdvSIMD
0		10010	XTN, XTN2	FEAT_AdvSIMD
0		10011	UNALLOCATED	-
0		10100	SQXTN, SQXTN2	FEAT_AdvSIMD
0	0x	10110	FCVTN, FCVTN2 (double to single-precision, single to half-precision)	FEAT_AdvSIMD
0	0x	10111	FCVTL, FCVTL2	FEAT_AdvSIMD
0	0x	11000	FRINTN (vector)	FEAT_AdvSIMD
0	0x	11001	FRINTM (vector)	FEAT_AdvSIMD
0	0x	11010	FCVTNS (vector)	FEAT_AdvSIMD
0	0x	11011	FCVTMS (vector)	FEAT_AdvSIMD
0	0x	11100	FCVTAS (vector)	FEAT_AdvSIMD
0	0x	11101	SCVTF (vector, integer)	FEAT_AdvSIMD
0	0x	11110	FRINT32Z (vector)	FEAT_FRINTTS
0	0x	11111	FRINT64Z (vector)	FEAT_FRINTTS
0	1x	01100	FCMGT (zero)	FEAT_AdvSIMD
0	1x	01101	FCMEQ (zero)	FEAT_AdvSIMD
0	1x	01110	FCMLT (zero)	FEAT_AdvSIMD
0	1x	01111	FABS (vector)	FEAT_AdvSIMD
0	1x	1x111	UNALLOCATED	-
0	1x	11000	FRINTP (vector)	FEAT_AdvSIMD
0	1x	11001	FRINTZ (vector)	FEAT_AdvSIMD
0	1x	11010	FCVTPS (vector)	FEAT_AdvSIMD
0	1x	11011	FCVTZS (vector, integer)	FEAT_AdvSIMD
0	1x	11100	URECPE	FEAT_AdvSIMD
0	1x	11101	FRECPE	FEAT_AdvSIMD
0	10	10110	BFCVTN, BFCVTN2	FEAT_BF16
1		00000	REV32 (vector)	FEAT_AdvSIMD
1		00010	UADDLP	FEAT_AdvSIMD
1		00011	USQADD	FEAT_AdvSIMD

Decode fields			Instruction Details	Condition
U	size	opcode		
1		00100	CLZ (vector)	FEAT_AdvSIMD
1		00110	UADALP	FEAT_AdvSIMD
1		00111	SQNEG	FEAT_AdvSIMD
1		01000	CMGE (zero)	FEAT_AdvSIMD
1		01001	CMLE (zero)	FEAT_AdvSIMD
1		01010	UNALLOCATED	-
1		01011	NEG (vector)	FEAT_AdvSIMD
1		10010	SQXTUN, SQXTUN2	FEAT_AdvSIMD
1		10011	SHLL, SHLL2	FEAT_AdvSIMD
1		10100	UQXTN, UQXTN2	FEAT_AdvSIMD
1	x0	10110	UNALLOCATED	-
1	0x	00001	UNALLOCATED	-
1	0x	11000	FRINTA (vector)	FEAT_AdvSIMD
1	0x	11001	FRINTX (vector)	FEAT_AdvSIMD
1	0x	11010	FCVTNU (vector)	FEAT_AdvSIMD
1	0x	11011	FCVTMU (vector)	FEAT_AdvSIMD
1	0x	11100	FCVTAU (vector)	FEAT_AdvSIMD
1	0x	11101	UCVTF (vector, integer)	FEAT_AdvSIMD
1	0x	11110	FRINT32X (vector)	FEAT_FRINTTS
1	0x	11111	FRINT64X (vector)	FEAT_FRINTTS
1	00	00101	NOT	FEAT_AdvSIMD
1	00	10111	F1CVTL, F1CVTL2, F2CVTL, F2CVTL2 — F1CVTL{2}	FEAT_FP8
1	01	00101	RBIT (vector)	FEAT_AdvSIMD
1	01	10110	FCVTXN, FCVTXN2	FEAT_AdvSIMD
1	01	10111	F1CVTL, F1CVTL2, F2CVTL, F2CVTL2 — F2CVTL{2}	FEAT_FP8
1	1x	00x01	UNALLOCATED	-
1	1x	01100	FCMGE (zero)	FEAT_AdvSIMD
1	1x	01101	FCMLE (zero)	FEAT_AdvSIMD
1	1x	01110	UNALLOCATED	-
1	1x	01111	FNEG (vector)	FEAT_AdvSIMD
1	1x	11000	UNALLOCATED	-
1	1x	11001	FRINTI (vector)	FEAT_AdvSIMD
1	1x	11010	FCVTPU (vector)	FEAT_AdvSIMD
1	1x	11011	FCVTZU (vector, integer)	FEAT_AdvSIMD
1	1x	11100	URSQRTE	FEAT_AdvSIMD
1	1x	11101	FRSQRTE	FEAT_AdvSIMD
1	1x	11111	FSQRT (vector)	FEAT_AdvSIMD
1	10	10111	BF1CVTL, BF1CVTL2, BF2CVTL, BF2CVTL2 — BF1CVTL{2}	FEAT_FP8
1	11	10111	BF1CVTL, BF1CVTL2, BF2CVTL, BF2CVTL2 — BF2CVTL{2}	FEAT_FP8

Advanced SIMD across lanes

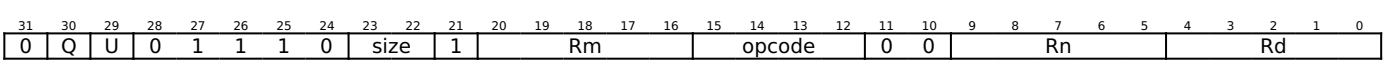
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields				Instruction Details	Condition
Q	U	size	opcode		
			x100x	UNALLOCATED	-
			000x0	UNALLOCATED	-
			00001	UNALLOCATED	-
			01011	UNALLOCATED	-
			100xx	UNALLOCATED	-
		x0	001xx	UNALLOCATED	-
		x0	01101	UNALLOCATED	-
		x0	101xx	UNALLOCATED	-
		x0	111xx	UNALLOCATED	-
		x1	xx1xx	UNALLOCATED	-
	0		00011	SADDLV	FEAT_AdvSIMD
	0		01010	SMAXV	FEAT_AdvSIMD
	0		11010	SMINV	FEAT_AdvSIMD
	0		11011	ADDV	FEAT_AdvSIMD
	0	x0	01110	UNALLOCATED	-
	0	00	01100	FMAXNMV — Half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	01111	FMAXV — Half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	10	01100	FMINNMV — Half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	10	01111	FMINV — Half-precision	FEAT_AdvSIMD && FEAT_FP16
	1		00011	UADDLV	FEAT_AdvSIMD
	1		01010	UMAXV	FEAT_AdvSIMD
	1		11010	UMINV	FEAT_AdvSIMD
	1		11011	UNALLOCATED	-
0	1	x0	011x0	UNALLOCATED	-
0	1	x0	01111	UNALLOCATED	-
1	1	x0	01110	UNALLOCATED	-
1	1	00	01100	FMAXNMV — Single-precision	FEAT_AdvSIMD
1	1	00	01111	FMAXV — Single-precision	FEAT_AdvSIMD
1	1	10	01100	FMINNMV — Single-precision	FEAT_AdvSIMD
1	1	10	01111	FMINV — Single-precision	FEAT_AdvSIMD

Advanced SIMD three different

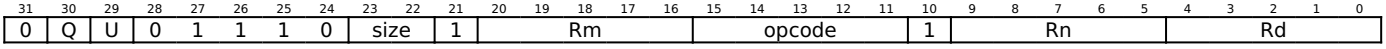
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields		Instruction Details	Condition
U	opcode		
0	0000	SADDL, SADDL2	FEAT_AdvSIMD
0	0001	SADDW, SADDW2	FEAT_AdvSIMD
0	0010	SSUBL, SSUBL2	FEAT_AdvSIMD
0	0011	SSUBW, SSUBW2	FEAT_AdvSIMD
0	0100	ADDHN, ADDHN2	FEAT_AdvSIMD
0	0101	SABAL, SABAL2	FEAT_AdvSIMD
0	0110	SUBHN, SUBHN2	FEAT_AdvSIMD
0	0111	SABDL, SABDL2	FEAT_AdvSIMD
0	1000	SMLAL, SMLAL2 (vector)	FEAT_AdvSIMD
0	1001	SQDMLAL, SQDMLAL2 (vector)	FEAT_AdvSIMD
0	1010	SMLSL, SMLSL2 (vector)	FEAT_AdvSIMD
0	1011	SQDMLSL, SQDMLSL2 (vector)	FEAT_AdvSIMD
0	1100	SMULL, SMULL2 (vector)	FEAT_AdvSIMD
0	1101	SQDMULL, SQDMULL2 (vector)	FEAT_AdvSIMD
0	1110	PMULL, PMULL2	FEAT_AdvSIMD
0	1111	UNALLOCATED	-
1	0000	UADDL, UADDL2	FEAT_AdvSIMD
1	0001	UADDW, UADDW2	FEAT_AdvSIMD
1	0010	USUBL, USUBL2	FEAT_AdvSIMD
1	0011	USUBW, USUBW2	FEAT_AdvSIMD
1	0100	RADDHN, RADDHN2	FEAT_AdvSIMD
1	0101	UABAL, UABAL2	FEAT_AdvSIMD
1	0110	RSUBHN, RSUBHN2	FEAT_AdvSIMD
1	0111	UABDL, UABDL2	FEAT_AdvSIMD
1	1xx1	UNALLOCATED	-
1	1000	UMLAL, UMLAL2 (vector)	FEAT_AdvSIMD
1	1010	UMLSL, UMLSL2 (vector)	FEAT_AdvSIMD
1	1100	UMULL, UMULL2 (vector)	FEAT_AdvSIMD
1	1110	UNALLOCATED	-

Advanced SIMD three same

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

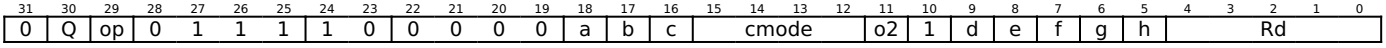


Decode fields			Instruction Details	Condition
U	size	opcode		
0		00000	SHADD	FEAT_AdvSIMD
0		00001	SQADD	FEAT_AdvSIMD
0		00010	SRHADD	FEAT_AdvSIMD
0		00100	SHSUB	FEAT_AdvSIMD
0		00101	SQSUB	FEAT_AdvSIMD
0		00110	CMGT (register)	FEAT_AdvSIMD
0		00111	CMGE (register)	FEAT_AdvSIMD
0		01000	SSHL	FEAT_AdvSIMD
0		01001	SQSHL (register)	FEAT_AdvSIMD
0		01010	SRSHL	FEAT_AdvSIMD
0		01011	SQRSHL	FEAT_AdvSIMD
0		01100	SMAX	FEAT_AdvSIMD
0		01101	SMIN	FEAT_AdvSIMD
0		01110	SABD	FEAT_AdvSIMD
0		01111	SABA	FEAT_AdvSIMD
0		10000	ADD (vector)	FEAT_AdvSIMD
0		10001	CMTST	FEAT_AdvSIMD
0		10010	MLA (vector)	FEAT_AdvSIMD
0		10011	MUL (vector)	FEAT_AdvSIMD
0		10100	SMAXP	FEAT_AdvSIMD
0		10101	SMINP	FEAT_AdvSIMD
0		10110	SQDMULH (vector)	FEAT_AdvSIMD
0		10111	ADDP (vector)	FEAT_AdvSIMD
0	x1	11101	UNALLOCATED	-
0	0x	11000	FMAXNM (vector)	FEAT_AdvSIMD
0	0x	11001	FMLA (vector)	FEAT_AdvSIMD
0	0x	11010	FADD (vector)	FEAT_AdvSIMD
0	0x	11011	FMULX	FEAT_AdvSIMD
0	0x	11100	FCMEQ (register)	FEAT_AdvSIMD
0	0x	11110	FMAX (vector)	FEAT_AdvSIMD
0	0x	11111	FRECPS	FEAT_AdvSIMD
0	00	00011	AND (vector)	FEAT_AdvSIMD
0	00	11101	FMLAL, FMLAL2 (vector) — FMLAL	FEAT_FHM
0	01	00011	BIC (vector, register)	FEAT_AdvSIMD
0	1x	11000	FMINNM (vector)	FEAT_AdvSIMD
0	1x	11001	FMLS (vector)	FEAT_AdvSIMD
0	1x	11010	FSUB (vector)	FEAT_AdvSIMD
0	1x	11011	FAMAX	FEAT_AdvSIMD && FEAT_FAMINMAX
0	1x	11100	UNALLOCATED	-
0	1x	11110	FMIN (vector)	FEAT_AdvSIMD
0	1x	11111	FRSQRTS	FEAT_AdvSIMD
0	10	00011	ORR (vector, register)	FEAT_AdvSIMD
0	10	11101	FMLSL, FMLSL2 (vector) — FMLSL	FEAT_FHM
0	11	00011	ORN (vector)	FEAT_AdvSIMD
1		00000	UHADD	FEAT_AdvSIMD

Decode fields			Instruction Details	Condition
U	size	opcode		
1		00001	UQADD	FEAT_AdvSIMD
1		00010	URHADD	FEAT_AdvSIMD
1		00100	UHSUB	FEAT_AdvSIMD
1		00101	UQSUB	FEAT_AdvSIMD
1		00110	CMHI (register)	FEAT_AdvSIMD
1		00111	CMHS (register)	FEAT_AdvSIMD
1		01000	USHL	FEAT_AdvSIMD
1		01001	UQSHL (register)	FEAT_AdvSIMD
1		01010	URSHL	FEAT_AdvSIMD
1		01011	UQRSHL	FEAT_AdvSIMD
1		01100	UMAX	FEAT_AdvSIMD
1		01101	UMIN	FEAT_AdvSIMD
1		01110	UABD	FEAT_AdvSIMD
1		01111	UABA	FEAT_AdvSIMD
1		10000	SUB (vector)	FEAT_AdvSIMD
1		10001	CMEQ (register)	FEAT_AdvSIMD
1		10010	MLS (vector)	FEAT_AdvSIMD
1		10011	PMUL	FEAT_AdvSIMD
1		10100	UMAXP	FEAT_AdvSIMD
1		10101	UMINP	FEAT_AdvSIMD
1		10110	SQRDMULH (vector)	FEAT_AdvSIMD
1		10111	UNALLOCATED	-
1	x1	11001	UNALLOCATED	-
1	0x	11000	FMAXNMP (vector)	FEAT_AdvSIMD
1	0x	11010	FADDP (vector)	FEAT_AdvSIMD
1	0x	11011	FMUL (vector)	FEAT_AdvSIMD
1	0x	11100	FCMGE (register)	FEAT_AdvSIMD
1	0x	11101	FACGE	FEAT_AdvSIMD
1	0x	11110	FMAXP (vector)	FEAT_AdvSIMD
1	0x	11111	FDIV (vector)	FEAT_AdvSIMD
1	00	00011	EOR (vector)	FEAT_AdvSIMD
1	00	11001	FMLAL, FMLAL2 (vector) — FMLAL2	FEAT_FHM
1	01	00011	BSL	FEAT_AdvSIMD
1	1x	11000	FMINNMP (vector)	FEAT_AdvSIMD
1	1x	11010	FABD	FEAT_AdvSIMD
1	1x	11011	FAMIN	FEAT_AdvSIMD && FEAT_FAMINMAX
1	1x	11100	FCMGT (register)	FEAT_AdvSIMD
1	1x	11101	FACGT	FEAT_AdvSIMD
1	1x	11110	FMINP (vector)	FEAT_AdvSIMD
1	1x	11111	FSCALE	FEAT_FP8
1	10	00011	BIT	FEAT_AdvSIMD
1	10	11001	FMLSL, FMLSL2 (vector) — FMLSL2	FEAT_FHM
1	11	00011	BIF	FEAT_AdvSIMD

Advanced SIMD modified immediate

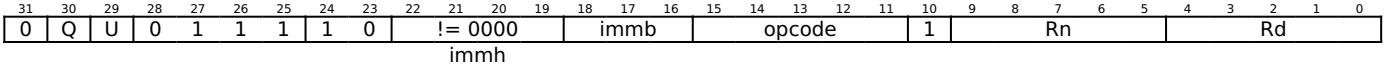
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields				Instruction Details	Condition
Q	op	cmode	o2		
	0	0xx0	0	MOVI — 32-bit shifted immediate	FEAT_AdvSIMD
	0	0xx1	0	ORR (vector, immediate) — 32-bit	FEAT_AdvSIMD
	0	10x0	0	MOVI — 16-bit shifted immediate	FEAT_AdvSIMD
	0	10x1	0	ORR (vector, immediate) — 16-bit	FEAT_AdvSIMD
	0	110x	0	MOVI — 32-bit shifting ones	FEAT_AdvSIMD
	0	1110	0	MOVI — 8-bit	FEAT_AdvSIMD
	0	1111	0	FMOV (vector, immediate) — Single-precision	FEAT_AdvSIMD
	0	1111	1	FMOV (vector, immediate) — Half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	!= 1111	1	UNALLOCATED	-
	1		1	UNALLOCATED	-
	1	0xx0	0	MVNI — 32-bit shifted immediate	FEAT_AdvSIMD
	1	0xx1	0	BIC (vector, immediate) — 32-bit	FEAT_AdvSIMD
	1	10x0	0	MVNI — 16-bit shifted immediate	FEAT_AdvSIMD
	1	10x1	0	BIC (vector, immediate) — 16-bit	FEAT_AdvSIMD
	1	110x	0	MVNI — 32-bit shifting ones	FEAT_AdvSIMD
0	1	1110	0	MOVI — 64-bit scalar	FEAT_AdvSIMD
0	1	1111	0	UNALLOCATED	-
1	1	1110	0	MOVI — 64-bit vector	FEAT_AdvSIMD
1	1	1111	0	FMOV (vector, immediate) — Double-precision	FEAT_AdvSIMD

Advanced SIMD shift by immediate

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



The following constraints also apply to this encoding: immh != '0000'

Decode fields			Instruction Details	Condition
U	immh	opcode		
	!= 0000	0xxx1	UNALLOCATED	-
	!= 0000	1x110	UNALLOCATED	-
	!= 0000	101x1	UNALLOCATED	-
	!= 0000	110xx	UNALLOCATED	-
	!= 0000	11101	UNALLOCATED	-
0	!= 0000	00000	SSHR	FEAT_AdvSIMD
0	!= 0000	00010	SSRA	FEAT_AdvSIMD
0	!= 0000	00100	SRSHR	FEAT_AdvSIMD
0	!= 0000	00110	SRSRA	FEAT_AdvSIMD
0	!= 0000	01x00	UNALLOCATED	-
0	!= 0000	01010	SHL	FEAT_AdvSIMD
0	!= 0000	01110	SQSHL (immediate)	FEAT_AdvSIMD
0	!= 0000	10000	SHRN, SHRN2	FEAT_AdvSIMD
0	!= 0000	10001	RSHRN, RSHRN2	FEAT_AdvSIMD
0	!= 0000	10010	SQSHRN, SQSHRN2	FEAT_AdvSIMD
0	!= 0000	10011	SQRSHRN, SQRSHRN2	FEAT_AdvSIMD
0	!= 0000	10100	SSHLL, SSHLL2	FEAT_AdvSIMD
0	!= 0000	11100	SCVTF (vector, fixed-point)	FEAT_AdvSIMD
0	!= 0000	11111	FCVTZS (vector, fixed-point)	FEAT_AdvSIMD
1	!= 0000	00000	USHR	FEAT_AdvSIMD
1	!= 0000	00010	USRA	FEAT_AdvSIMD
1	!= 0000	00100	URSHR	FEAT_AdvSIMD
1	!= 0000	00110	URSRA	FEAT_AdvSIMD
1	!= 0000	01000	SRI	FEAT_AdvSIMD
1	!= 0000	01010	SLI	FEAT_AdvSIMD
1	!= 0000	01100	SQSHLU	FEAT_AdvSIMD
1	!= 0000	01110	UQSHL (immediate)	FEAT_AdvSIMD
1	!= 0000	10000	SQSHRUN, SQSHRUN2	FEAT_AdvSIMD
1	!= 0000	10001	SQRSHRUN, SQRSHRUN2	FEAT_AdvSIMD
1	!= 0000	10010	UQSHRN, UQSHRN2	FEAT_AdvSIMD
1	!= 0000	10011	UQRSHRN, UQRSHRN2	FEAT_AdvSIMD
1	!= 0000	10100	USHLL, USHLL2	FEAT_AdvSIMD
1	!= 0000	11100	UCVTF (vector, fixed-point)	FEAT_AdvSIMD
1	!= 0000	11111	FCVTZU (vector, fixed-point)	FEAT_AdvSIMD

Advanced SIMD vector x indexed element

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	U	0	1	1	1	1	size		L	M	Rm				opcode				H	0	Rn						Rd			

Decode fields				Instruction Details	Condition
Q	U	size	opcode		
	0		0010	SMLAL, SMLAL2 (by element)	FEAT_AdvSIMD
	0		0011	SQDMLAL, SQDMLAL2 (by element)	FEAT_AdvSIMD
	0		0110	SMLSL, SMLSL2 (by element)	FEAT_AdvSIMD
	0		0111	SQDMLSL, SQDMLSL2 (by element)	FEAT_AdvSIMD
	0		1000	MUL (by element)	FEAT_AdvSIMD
	0		1010	SMULL, SMULL2 (by element)	FEAT_AdvSIMD
	0		1011	SQDMULL, SQDMULL2 (by element)	FEAT_AdvSIMD
	0		1100	SQDMULH (by element)	FEAT_AdvSIMD
	0		1101	SQRDMULH (by element)	FEAT_AdvSIMD
	0		1110	SDOT (by element)	FEAT_DotProd
	0	00	0000	FDOT (8-bit floating-point to single-precision, by element)	FEAT_FP8DOT4
	0	00	0001	FMLA (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	0101	FMLS (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	1001	FMUL (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	0	00	1111	SUDOT (by element)	FEAT_I8MM
	0	01	0x01	UNALLOCATED	-
	0	01	0000	FDOT (8-bit floating-point to half-precision, by element)	FEAT_FP8DOT2
	0	01	1001	FDOT (half-precision to single-precision, by element)	FEAT_F16F32DOT
	0	01	1111	BFDOT (by element)	FEAT_BF16
	0	1x	0001	FMLA (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	0	1x	0101	FMLS (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	0	1x	1001	FMUL (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	0	10	0000	FMLAL, FMLAL2 (by element) — FMLAL	FEAT_FHM
	0	10	0100	FMLSL, FMLSL2 (by element) — FMLSL	FEAT_FHM
	0	10	1111	USDOT (by element)	FEAT_I8MM
	0	11	1111	BFMLALB, BFMLALT (by element)	FEAT_BF16
	0	!= 10	0100	UNALLOCATED	-
	1		0xx1	FCMLA (by element)	FEAT_FCMA
	1		0000	MLA (by element)	FEAT_AdvSIMD
	1		0010	UMLAL, UMLAL2 (by element)	FEAT_AdvSIMD
	1		0100	MLS (by element)	FEAT_AdvSIMD
	1		0110	UMLSL, UMLSL2 (by element)	FEAT_AdvSIMD
	1		1010	UMULL, UMULL2 (by element)	FEAT_AdvSIMD
	1		1011	UNALLOCATED	-
	1		1101	SQRDMLAH (by element)	FEAT_RDM
	1		1110	UDOT (by element)	FEAT_DotProd
	1		1111	SQRDMLSH (by element)	FEAT_RDM
	1	0x	1100	UNALLOCATED	-
	1	00	1001	FMULX (by element) — Vector, half-precision	FEAT_AdvSIMD && FEAT_FP16
	1	01	1001	UNALLOCATED	-
	1	1x	1001	FMULX (by element) — Vector, single-precision and double-precision	FEAT_AdvSIMD
	1	10	1000	FMLAL, FMLAL2 (by element) — FMLAL2	FEAT_FHM

Decode fields				Instruction Details	Condition
Q	U	size	opcode		
	1	10	1100	FMLSL, FMLSL2 (by element) — FMLSL2	FEAT_FHM
	1	11	1x00	UNALLOCATED	-
0	0	11	0000	FMLALB, FMLALT (by element) — FMLALB	FEAT_FP8FMA
0	1	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLBB	FEAT_FP8FMA
0	1	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLBT	FEAT_FP8FMA
1	0	11	0000	FMLALB, FMLALT (by element) — FMLALT	FEAT_FP8FMA
1	1	00	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLTB	FEAT_FP8FMA
1	1	01	1000	FMLALLBB, FMLALLBT, FMLALLTB, FMLALLTT (by element) — FMLALLTT	FEAT_FP8FMA

Cryptographic three-register, imm2

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	1	1	0	0	1	0	Rm					1	0	imm2		opcode		Rn					Rd				

Decode fields	Instruction Details	Condition
opcode		
00	SM3TT1A	FEAT_SM3
01	SM3TT1B	FEAT_SM3
10	SM3TT2A	FEAT_SM3
11	SM3TT2B	FEAT_SM3

Cryptographic three-register SHA 512

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	1	1	0	0	1	1	Rm					1	O	0	0	opcode		Rn					Rd				

Decode fields		Instruction Details	Condition
O	opcode		
0	00	SHA512H	FEAT_SHA512
0	01	SHA512H2	FEAT_SHA512
0	10	SHA512SU1	FEAT_SHA512
0	11	RAX1	FEAT_SHA3
1	00	SM3PARTW1	FEAT_SM3
1	01	SM3PARTW2	FEAT_SM3
1	10	SM4EKEY	FEAT_SM4
1	11	UNALLOCATED	-

Cryptographic four-register

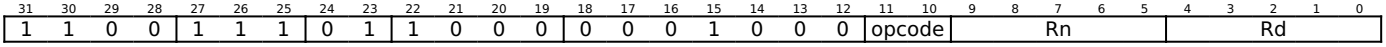
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	1	1	1	0	0	Op0		Rm					0	Ra					Rn					Rd				

Decode fields Op0	Instruction Details	Condition
00	EOR3	FEAT_SHA3
01	BCAX	FEAT_SHA3
10	SM3SS1	FEAT_SM3
11	UNALLOCATED	-

Cryptographic two-register SHA 512

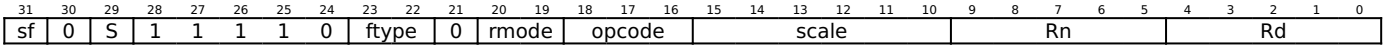
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields opcode	Instruction Details	Condition
00	SHA512SU0	FEAT_SHA512
01	SM4E	FEAT_SM4
1x	UNALLOCATED	-

Conversion between floating-point and fixed-point

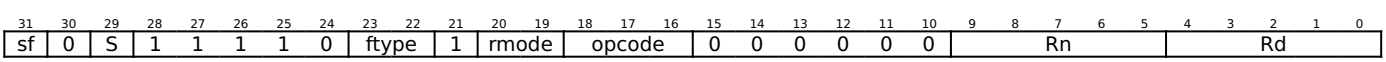
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields					Instruction Details	Condition
sf	S	ftype	rmode	opcode		
	0			1xx	UNALLOCATED	-
	0	10		0xx	UNALLOCATED	-
	0	!= 10	x1	01x	UNALLOCATED	-
	0	!= 10	0x	00x	UNALLOCATED	-
	0	!= 10	10	0xx	UNALLOCATED	-
	1				UNALLOCATED	-
0	0	00	00	010	SCVTF (scalar, fixed-point) — 32-bit to single-precision	FEAT_FP
0	0	00	00	011	UCVTF (scalar, fixed-point) — 32-bit to single-precision	FEAT_FP
0	0	00	11	000	FCVTZS (scalar, fixed-point) — Single-precision to 32-bit	FEAT_FP
0	0	00	11	001	FCVTZU (scalar, fixed-point) — Single-precision to 32-bit	FEAT_FP
0	0	01	00	010	SCVTF (scalar, fixed-point) — 32-bit to double-precision	FEAT_FP
0	0	01	00	011	UCVTF (scalar, fixed-point) — 32-bit to double-precision	FEAT_FP
0	0	01	11	000	FCVTZS (scalar, fixed-point) — Double-precision to 32-bit	FEAT_FP
0	0	01	11	001	FCVTZU (scalar, fixed-point) — Double-precision to 32-bit	FEAT_FP
0	0	11	00	010	SCVTF (scalar, fixed-point) — 32-bit to half-precision	FEAT_FP16
0	0	11	00	011	UCVTF (scalar, fixed-point) — 32-bit to half-precision	FEAT_FP16
0	0	11	11	000	FCVTZS (scalar, fixed-point) — Half-precision to 32-bit	FEAT_FP16
0	0	11	11	001	FCVTZU (scalar, fixed-point) — Half-precision to 32-bit	FEAT_FP16
1	0	00	00	010	SCVTF (scalar, fixed-point) — 64-bit to single-precision	FEAT_FP
1	0	00	00	011	UCVTF (scalar, fixed-point) — 64-bit to single-precision	FEAT_FP
1	0	00	11	000	FCVTZS (scalar, fixed-point) — Single-precision to 64-bit	FEAT_FP
1	0	00	11	001	FCVTZU (scalar, fixed-point) — Single-precision to 64-bit	FEAT_FP
1	0	01	00	010	SCVTF (scalar, fixed-point) — 64-bit to double-precision	FEAT_FP
1	0	01	00	011	UCVTF (scalar, fixed-point) — 64-bit to double-precision	FEAT_FP
1	0	01	11	000	FCVTZS (scalar, fixed-point) — Double-precision to 64-bit	FEAT_FP
1	0	01	11	001	FCVTZU (scalar, fixed-point) — Double-precision to 64-bit	FEAT_FP
1	0	11	00	010	SCVTF (scalar, fixed-point) — 64-bit to half-precision	FEAT_FP16
1	0	11	00	011	UCVTF (scalar, fixed-point) — 64-bit to half-precision	FEAT_FP16
1	0	11	11	000	FCVTZS (scalar, fixed-point) — Half-precision to 64-bit	FEAT_FP16
1	0	11	11	001	FCVTZU (scalar, fixed-point) — Half-precision to 64-bit	FEAT_FP16

Conversion between floating-point and integer

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields					Instruction Details	Condition
sf	S	ftype	rmode	opcode		
	0	10	x0		UNALLOCATED	-
	0	10	x1	!= 11x	UNALLOCATED	-
	0	11	x1	11x	UNALLOCATED	-
	0	!= 10	01	10x	UNALLOCATED	-
	1				UNALLOCATED	-
0	0	x0	01	11x	UNALLOCATED	-
0	0	00	00	000	FCVTNS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	001	FCVTNU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	010	SCVTF (scalar, integer) — 32-bit to single-precision	FEAT_FP
0	0	00	00	011	UCVTF (scalar, integer) — 32-bit to single-precision	FEAT_FP
0	0	00	00	100	FCVTAS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	101	FCVTAU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	110	FMOV (general) — Single-precision to 32-bit	FEAT_FP
0	0	00	00	111	FMOV (general) — 32-bit to single-precision	FEAT_FP
0	0	00	01	000	FCVTPS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	01	001	FCVTPU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	1x	1xx	UNALLOCATED	-
0	0	00	10	000	FCVTMS (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	10	001	FCVTMU (scalar) — Single-precision to 32-bit	FEAT_FP
0	0	00	11	000	FCVTZS (scalar, integer) — Single-precision to 32-bit	FEAT_FP
0	0	00	11	001	FCVTZU (scalar, integer) — Single-precision to 32-bit	FEAT_FP
0	0	00	!= 00	01x	UNALLOCATED	-
0	0	01	0x	11x	UNALLOCATED	-
0	0	01	00	000	FCVTNS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	00	001	FCVTNU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	00	010	SCVTF (scalar, integer) — 32-bit to double-precision	FEAT_FP
0	0	01	00	011	UCVTF (scalar, integer) — 32-bit to double-precision	FEAT_FP
0	0	01	00	100	FCVTAS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	00	101	FCVTAU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	01	000	FCVTPS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	01	001	FCVTPU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	01	010	FCVTNS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	01	011	FCVTNU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	000	FCVTMS (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	10	001	FCVTMU (scalar) — Double-precision to 32-bit	FEAT_FP
0	0	01	10	010	FCVTPS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	011	FCVTPU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	100	FCVTMS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	101	FCVTMU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	110	FCVTZS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	10	111	FCVTZU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	11	000	FCVTZS (scalar, integer) — Double-precision to 32-bit	FEAT_FP
0	0	01	11	001	FCVTZU (scalar, integer) — Double-precision to 32-bit	FEAT_FP
0	0	01	11	010	FCVTAS (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT
0	0	01	11	011	FCVTAU (scalar SIMD&FP) — Double-precision to 32-bit	FEAT_FPRCVT

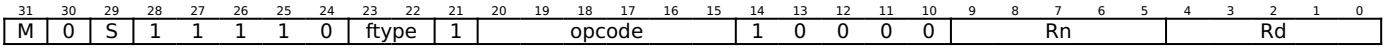
Decode fields					Instruction Details	Condition
sf	S	ftype	rmode	opcode		
0	0	01	11	100	SCVTF (scalar SIMD&FP) — 32-bit to double-precision	FEAT_FPRCVT
0	0	01	11	101	UCVTF (scalar SIMD&FP) — 32-bit to double-precision	FEAT_FPRCVT
0	0	01	11	110	FJCVTZS	FEAT_JSCVT
0	0	01	11	111	UNALLOCATED	-
0	0	10	11	11x	UNALLOCATED	-
0	0	11	00	000	FCVTNS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	001	FCVTNU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	010	SCVTF (scalar, integer) — 32-bit to half-precision	FEAT_FP16
0	0	11	00	011	UCVTF (scalar, integer) — 32-bit to half-precision	FEAT_FP16
0	0	11	00	100	FCVTAS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	101	FCVTAU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	110	FMOV (general) — Half-precision to 32-bit	FEAT_FP16
0	0	11	00	111	FMOV (general) — 32-bit to half-precision	FEAT_FP16
0	0	11	01	000	FCVTPS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	01	001	FCVTPU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	01	010	FCVTNS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	01	011	FCVTNU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	000	FCVTMS (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	10	001	FCVTMU (scalar) — Half-precision to 32-bit	FEAT_FP16
0	0	11	10	010	FCVTPS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	011	FCVTPU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	100	FCVTMS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	101	FCVTMU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	110	FCVTZS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	10	111	FCVTZU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	11	000	FCVTZS (scalar, integer) — Half-precision to 32-bit	FEAT_FP16
0	0	11	11	001	FCVTZU (scalar, integer) — Half-precision to 32-bit	FEAT_FP16
0	0	11	11	010	FCVTAS (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	11	011	FCVTAU (scalar SIMD&FP) — Half-precision to 32-bit	FEAT_FPRCVT
0	0	11	11	100	SCVTF (scalar SIMD&FP) — 32-bit to half-precision	FEAT_FPRCVT
0	0	11	11	101	UCVTF (scalar SIMD&FP) — 32-bit to half-precision	FEAT_FPRCVT
1	0	x0	11	11x	UNALLOCATED	-
1	0	00	0x	11x	UNALLOCATED	-
1	0	00	00	000	FCVTNS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	00	001	FCVTNU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	00	010	SCVTF (scalar, integer) — 64-bit to single-precision	FEAT_FP
1	0	00	00	011	UCVTF (scalar, integer) — 64-bit to single-precision	FEAT_FP
1	0	00	00	100	FCVTAS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	00	101	FCVTAU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	01	000	FCVTPS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	01	001	FCVTPU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	01	010	FCVTNS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	01	011	FCVTNU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	000	FCVTMS (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	10	001	FCVTMU (scalar) — Single-precision to 64-bit	FEAT_FP
1	0	00	10	010	FCVTPS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT

Decode fields					Instruction Details	Condition
sf	S	ftype	rmode	opcode		
1	0	00	10	011	FCVTPU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	100	FCVTMS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	101	FCVTMU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	110	FCVTZS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	10	111	FCVTZU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	11	000	FCVTZS (scalar, integer) — Single-precision to 64-bit	FEAT_FP
1	0	00	11	001	FCVTZU (scalar, integer) — Single-precision to 64-bit	FEAT_FP
1	0	00	11	010	FCVTAS (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	11	011	FCVTAU (scalar SIMD&FP) — Single-precision to 64-bit	FEAT_FPRCVT
1	0	00	11	100	SCVTF (scalar SIMD&FP) — 64-bit to single-precision	FEAT_FPRCVT
1	0	00	11	101	UCVTF (scalar SIMD&FP) — 64-bit to single-precision	FEAT_FPRCVT
1	0	01	00	000	FCVTNS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	001	FCVTNU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	010	SCVTF (scalar, integer) — 64-bit to double-precision	FEAT_FP
1	0	01	00	011	UCVTF (scalar, integer) — 64-bit to double-precision	FEAT_FP
1	0	01	00	100	FCVTAS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	101	FCVTAU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	110	FMOV (general) — Double-precision to 64-bit	FEAT_FP
1	0	01	00	111	FMOV (general) — 64-bit to double-precision	FEAT_FP
1	0	01	01	x1x	UNALLOCATED	-
1	0	01	01	000	FCVTPS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	01	001	FCVTPU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	1x	!= 00x	UNALLOCATED	-
1	0	01	10	000	FCVTMS (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	10	001	FCVTMU (scalar) — Double-precision to 64-bit	FEAT_FP
1	0	01	11	000	FCVTZS (scalar, integer) — Double-precision to 64-bit	FEAT_FP
1	0	01	11	001	FCVTZU (scalar, integer) — Double-precision to 64-bit	FEAT_FP
1	0	10	01	110	FMOV (general) — Top half of 128-bit to 64-bit	FEAT_FP
1	0	10	01	111	FMOV (general) — 64-bit to top half of 128-bit	FEAT_FP
1	0	11	00	000	FCVTNS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	001	FCVTNU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	010	SCVTF (scalar, integer) — 64-bit to half-precision	FEAT_FP16
1	0	11	00	011	UCVTF (scalar, integer) — 64-bit to half-precision	FEAT_FP16
1	0	11	00	100	FCVTAS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	101	FCVTAU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	110	FMOV (general) — Half-precision to 64-bit	FEAT_FP16
1	0	11	00	111	FMOV (general) — 64-bit to half-precision	FEAT_FP16
1	0	11	01	000	FCVTPS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	01	001	FCVTPU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	01	010	FCVTNS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	01	011	FCVTNU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	000	FCVTMS (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	10	001	FCVTMU (scalar) — Half-precision to 64-bit	FEAT_FP16
1	0	11	10	010	FCVTPS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	011	FCVTPU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	100	FCVTMS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT

Decode fields					Instruction Details	Condition
sf	S	ftype	rmode	opcode		
1	0	11	10	101	FCVTMU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	110	FCVTZS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	10	111	FCVTZU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	11	000	FCVTZS (scalar, integer) — Half-precision to 64-bit	FEAT_FP16
1	0	11	11	001	FCVTZU (scalar, integer) — Half-precision to 64-bit	FEAT_FP16
1	0	11	11	010	FCVTAS (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	11	011	FCVTAU (scalar SIMD&FP) — Half-precision to 64-bit	FEAT_FPRCVT
1	0	11	11	100	SCVTF (scalar SIMD&FP) — 64-bit to half-precision	FEAT_FPRCVT
1	0	11	11	101	UCVTF (scalar SIMD&FP) — 64-bit to half-precision	FEAT_FPRCVT

Floating-point data-processing (1 source)

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields				Instruction Details	Condition
M	S	ftype	opcode		
0	0		1xxxxx	UNALLOCATED	-
0	0	0x	0101xx	UNALLOCATED	-
0	0	0x	011xxx	UNALLOCATED	-
0	0	00	000000	FMOV (register) — Single-precision	FEAT_FP
0	0	00	000001	FABS (scalar) — Single-precision	FEAT_FP
0	0	00	000010	FNEG (scalar) — Single-precision	FEAT_FP
0	0	00	000011	FSQRT (scalar) — Single-precision	FEAT_FP
0	0	00	0001x0	UNALLOCATED	-
0	0	00	000101	FCVT — Single-precision to double-precision	FEAT_FP
0	0	00	000111	FCVT — Single-precision to half-precision	FEAT_FP
0	0	00	001000	FRINTN (scalar) — Single-precision	FEAT_FP
0	0	00	001001	FRINTP (scalar) — Single-precision	FEAT_FP
0	0	00	001010	FRINTM (scalar) — Single-precision	FEAT_FP
0	0	00	001011	FRINTZ (scalar) — Single-precision	FEAT_FP
0	0	00	001100	FRINTA (scalar) — Single-precision	FEAT_FP
0	0	00	001110	FRINTX (scalar) — Single-precision	FEAT_FP
0	0	00	001111	FRINTI (scalar) — Single-precision	FEAT_FP
0	0	00	010000	FRINT32Z (scalar) — Single-precision	FEAT_FRINTTS
0	0	00	010001	FRINT32X (scalar) — Single-precision	FEAT_FRINTTS
0	0	00	010010	FRINT64Z (scalar) — Single-precision	FEAT_FRINTTS
0	0	00	010011	FRINT64X (scalar) — Single-precision	FEAT_FRINTTS
0	0	01	000000	FMOV (register) — Double-precision	FEAT_FP
0	0	01	000001	FABS (scalar) — Double-precision	FEAT_FP
0	0	01	000010	FNEG (scalar) — Double-precision	FEAT_FP
0	0	01	000011	FSQRT (scalar) — Double-precision	FEAT_FP
0	0	01	000100	FCVT — Double-precision to single-precision	FEAT_FP
0	0	01	000101	UNALLOCATED	-
0	0	01	000110	BFCVT	FEAT_BF16
0	0	01	000111	FCVT — Double-precision to half-precision	FEAT_FP
0	0	01	001000	FRINTN (scalar) — Double-precision	FEAT_FP
0	0	01	001001	FRINTP (scalar) — Double-precision	FEAT_FP
0	0	01	001010	FRINTM (scalar) — Double-precision	FEAT_FP
0	0	01	001011	FRINTZ (scalar) — Double-precision	FEAT_FP
0	0	01	001100	FRINTA (scalar) — Double-precision	FEAT_FP
0	0	01	001110	FRINTX (scalar) — Double-precision	FEAT_FP
0	0	01	001111	FRINTI (scalar) — Double-precision	FEAT_FP
0	0	01	010000	FRINT32Z (scalar) — Double-precision	FEAT_FRINTTS
0	0	01	010001	FRINT32X (scalar) — Double-precision	FEAT_FRINTTS
0	0	01	010010	FRINT64Z (scalar) — Double-precision	FEAT_FRINTTS
0	0	01	010011	FRINT64X (scalar) — Double-precision	FEAT_FRINTTS
0	0	10	0xxxxx	UNALLOCATED	-
0	0	11	000000	FMOV (register) — Half-precision	FEAT_FP16
0	0	11	000001	FABS (scalar) — Half-precision	FEAT_FP16
0	0	11	000010	FNEG (scalar) — Half-precision	FEAT_FP16
0	0	11	000011	FSQRT (scalar) — Half-precision	FEAT_FP16

Decode fields				Instruction Details	Condition
M	S	ftype	opcode		
0	0	11	000100	FCVT — Half-precision to single-precision	FEAT_FP
0	0	11	000101	FCVT — Half-precision to double-precision	FEAT_FP
0	0	11	00011x	UNALLOCATED	-
0	0	11	001000	FRINTN (scalar) — Half-precision	FEAT_FP16
0	0	11	001001	FRINTP (scalar) — Half-precision	FEAT_FP16
0	0	11	001010	FRINTM (scalar) — Half-precision	FEAT_FP16
0	0	11	001011	FRINTZ (scalar) — Half-precision	FEAT_FP16
0	0	11	001100	FRINTA (scalar) — Half-precision	FEAT_FP16
0	0	11	001110	FRINTX (scalar) — Half-precision	FEAT_FP16
0	0	11	001111	FRINTI (scalar) — Half-precision	FEAT_FP16
0	0	11	01xxxx	UNALLOCATED	-
0	0	!= 10	001101	UNALLOCATED	-
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point compare

The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	0	ftype		1	Rm					op		1	0	0	0	Rn					opcode2				

Decode fields					Instruction Details	Condition
M	S	ftype	op	opcode2		
0	0		00	!= xx000	UNALLOCATED	-
0	0		!= 00		UNALLOCATED	-
0	0	00	00	00000	FCMP — Single-precision	FEAT_FP
0	0	00	00	01000	FCMP — Single-precision, zero	FEAT_FP
0	0	00	00	10000	FCMPE — Single-precision	FEAT_FP
0	0	00	00	11000	FCMPE — Single-precision, zero	FEAT_FP
0	0	01	00	00000	FCMP — Double-precision	FEAT_FP
0	0	01	00	01000	FCMP — Double-precision, zero	FEAT_FP
0	0	01	00	10000	FCMPE — Double-precision	FEAT_FP
0	0	01	00	11000	FCMPE — Double-precision, zero	FEAT_FP
0	0	10	00	xx000	UNALLOCATED	-
0	0	11	00	00000	FCMP — Half-precision	FEAT_FP16
0	0	11	00	01000	FCMP — Half-precision, zero	FEAT_FP16
0	0	11	00	10000	FCMPE — Half-precision	FEAT_FP16
0	0	11	00	11000	FCMPE — Half-precision, zero	FEAT_FP16
0	1				UNALLOCATED	-
1					UNALLOCATED	-

Floating-point immediate

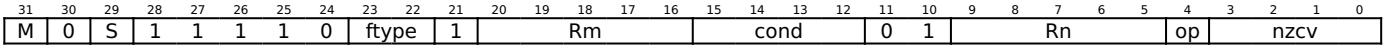
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
M	0	S	1	1	1	1	0	ftype		1	imm8								1	0	0	imm5					Rd				

Decode fields				Instruction Details	Condition
M	S	ftype	imm5		
0	0		!= 00000	UNALLOCATED	-
0	0	00	00000	FMOV (scalar, immediate) — Single-precision	FEAT_FP
0	0	01	00000	FMOV (scalar, immediate) — Double-precision	FEAT_FP
0	0	10	00000	UNALLOCATED	-
0	0	11	00000	FMOV (scalar, immediate) — Half-precision	FEAT_FP16
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point conditional compare

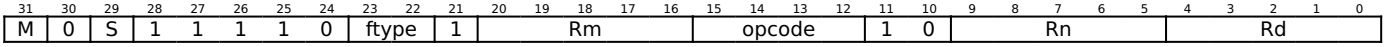
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields				Instruction Details	Condition
M	S	ftype	op		
0	0	00	0	FCCMP — Single-precision	FEAT_FP
0	0	00	1	FCCMPE — Single-precision	FEAT_FP
0	0	01	0	FCCMP — Double-precision	FEAT_FP
0	0	01	1	FCCMPE — Double-precision	FEAT_FP
0	0	10		UNALLOCATED	-
0	0	11	0	FCCMP — Half-precision	FEAT_FP16
0	0	11	1	FCCMPE — Half-precision	FEAT_FP16
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point data-processing (2 source)

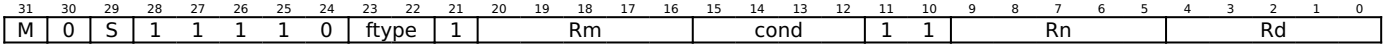
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields				Instruction Details	Condition
M	S	ftype	opcode		
0	0	00	0000	FMUL (scalar) — Single-precision	FEAT_FP
0	0	00	0001	FDIV (scalar) — Single-precision	FEAT_FP
0	0	00	0010	FADD (scalar) — Single-precision	FEAT_FP
0	0	00	0011	FSUB (scalar) — Single-precision	FEAT_FP
0	0	00	0100	FMAX (scalar) — Single-precision	FEAT_FP
0	0	00	0101	FMIN (scalar) — Single-precision	FEAT_FP
0	0	00	0110	FMAXNM (scalar) — Single-precision	FEAT_FP
0	0	00	0111	FMINNM (scalar) — Single-precision	FEAT_FP
0	0	00	1000	FNMUL (scalar) — Single-precision	FEAT_FP
0	0	01	0000	FMUL (scalar) — Double-precision	FEAT_FP
0	0	01	0001	FDIV (scalar) — Double-precision	FEAT_FP
0	0	01	0010	FADD (scalar) — Double-precision	FEAT_FP
0	0	01	0011	FSUB (scalar) — Double-precision	FEAT_FP
0	0	01	0100	FMAX (scalar) — Double-precision	FEAT_FP
0	0	01	0101	FMIN (scalar) — Double-precision	FEAT_FP
0	0	01	0110	FMAXNM (scalar) — Double-precision	FEAT_FP
0	0	01	0111	FMINNM (scalar) — Double-precision	FEAT_FP
0	0	01	1000	FNMUL (scalar) — Double-precision	FEAT_FP
0	0	10		UNALLOCATED	-
0	0	11	0000	FMUL (scalar) — Half-precision	FEAT_FP16
0	0	11	0001	FDIV (scalar) — Half-precision	FEAT_FP16
0	0	11	0010	FADD (scalar) — Half-precision	FEAT_FP16
0	0	11	0011	FSUB (scalar) — Half-precision	FEAT_FP16
0	0	11	0100	FMAX (scalar) — Half-precision	FEAT_FP16
0	0	11	0101	FMIN (scalar) — Half-precision	FEAT_FP16
0	0	11	0110	FMAXNM (scalar) — Half-precision	FEAT_FP16
0	0	11	0111	FMINNM (scalar) — Half-precision	FEAT_FP16
0	0	11	1000	FNMUL (scalar) — Half-precision	FEAT_FP16
0	0	!= 10	1001	UNALLOCATED	-
0	0	!= 10	101x	UNALLOCATED	-
0	0	!= 10	11xx	UNALLOCATED	-
0	1			UNALLOCATED	-
1				UNALLOCATED	-

Floating-point conditional select

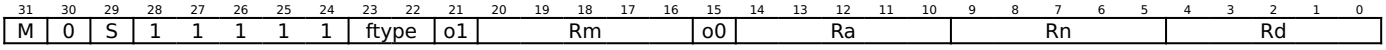
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields			Instruction Details	Condition
M	S	ftype		
0	0	00	FCSEL — Single-precision	FEAT_FP
0	0	01	FCSEL — Double-precision	FEAT_FP
0	0	10	UNALLOCATED	-
0	0	11	FCSEL — Half-precision	FEAT_FP16
0	1		UNALLOCATED	-
1			UNALLOCATED	-

Floating-point data-processing (3 source)

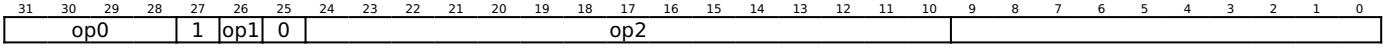
The encodings in this section are decoded from Data Processing -- Scalar Floating-Point and Advanced SIMD.



Decode fields					Instruction Details	Condition
M	S	ftype	o1	o0		
0	0	00	0	0	FMADD — Single-precision	FEAT_FP
0	0	00	0	1	FMSUB — Single-precision	FEAT_FP
0	0	00	1	0	FNMADD — Single-precision	FEAT_FP
0	0	00	1	1	FNMSUB — Single-precision	FEAT_FP
0	0	01	0	0	FMADD — Double-precision	FEAT_FP
0	0	01	0	1	FMSUB — Double-precision	FEAT_FP
0	0	01	1	0	FNMADD — Double-precision	FEAT_FP
0	0	01	1	1	FNMSUB — Double-precision	FEAT_FP
0	0	10			UNALLOCATED	-
0	0	11	0	0	FMADD — Half-precision	FEAT_FP16
0	0	11	0	1	FMSUB — Half-precision	FEAT_FP16
0	0	11	1	0	FNMADD — Half-precision	FEAT_FP16
0	0	11	1	1	FNMSUB — Half-precision	FEAT_FP16
0	1				UNALLOCATED	-
1					UNALLOCATED	-

Loads and Stores

The encodings in this section are decoded from A64 instruction set encoding.

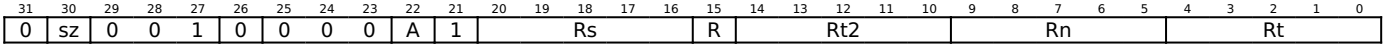


Decode fields			Instruction details
op0	op1	op2	
0000	0	1xx1xxxxxxxxxx	UNALLOCATED
0001	1	1xx1xxxxxxxxxx	UNALLOCATED
0x00	0	00x1xxxxxxxxxx	Compare and swap pair
0x00	0	10x0xxxxxxxxxx	UNALLOCATED
0x00	0	11x0xxxxxxxxxx	Compare and swap pair (unprivileged)
0x00	1	00x000000xxxxxx	Advanced SIMD load/store multiple structures
0x00	1	00x000001xxxxxx	UNALLOCATED
0x00	1	01x0xxxxxxxxxx	Advanced SIMD load/store multiple structures (post-indexed)
0x00	1	0xx1xxxxxxxxxx	UNALLOCATED
0x00	1	10x10001xxxxxxx	UNALLOCATED
0x00	1	10x1001xxxxxxxx	UNALLOCATED
0x00	1	10x101xxxxxxxxx	UNALLOCATED
0x00	1	10x11xxxxxxxxxx	UNALLOCATED
0x00	1	10xx0000xxxxxxx	Advanced SIMD load/store single structure
0x00	1	11xxxxxxxxxxxxx	Advanced SIMD load/store single structure (post-indexed)
0x00	1	x0x00001xxxxxxx	UNALLOCATED
0x00	1	x0x0001xxxxxxxx	UNALLOCATED
0x00	1	x0x001xxxxxxxxx	UNALLOCATED
0x00	1	x0x01xxxxxxxxxx	UNALLOCATED
0x01	0	1xx0xxxxxxxxx11	UNALLOCATED
0x01	0	1xx1xxxxx000010	RCW compare and swap
0x01	0	1xx1xxxxx000011	RCW compare and swap pair
0x01	0	1xx1xxxxx00011x	UNALLOCATED
0x01	0	1xx1xxxxx001x1x	UNALLOCATED
0x01	0	1xx1xxxxx01xx1x	UNALLOCATED
0x01	0	1xx1xxxxx1xxx1x	UNALLOCATED
0x01	0	1xx1xxxxxxxxx00	128-bit atomic memory operations
0x01	0	1xx1xxxxxxxxx01	Atomic memory operations (unprivileged)
1001	0	1xx0xxxxxxxxx11	UNALLOCATED
1001	1	1xx1xxxxxxxxxx	UNALLOCATED
100x	0	1xx1xxxxxxxxxx	UNALLOCATED
1101	0	1000111110xxx11	GCS load/store
1101	0	1100111110xxx11	UNALLOCATED
1101	0	1x000xxxxxxxx11	UNALLOCATED
1101	0	1x0010xxxxxxx11	UNALLOCATED
1101	0	1x00110xxxxxx11	UNALLOCATED
1101	0	1x001110xxxxx11	UNALLOCATED
1101	0	1x0011110xxxx11	UNALLOCATED
1101	0	1x00111111xxx11	UNALLOCATED
1101	0	1x10xxxxxxxx11	UNALLOCATED
1101	0	1xx1xxxxxxxxxx	Load/store memory tags
1x00	0	00x1xxxxxxxxxx	Load/store exclusive pair
1x00	0	10x0xxxxxxxxxx	Load/store exclusive register (unprivileged)
1x00	0	11x0xxxxxxxxxx	Compare and swap (unprivileged)
1x00	1		UNALLOCATED

x100	0	1xx1xxxxxxxxxx	UNALLOCATED
x101	1	1xx1xxxxxxxxxx	UNALLOCATED
xx00	0	00x0xxxxxxxxxx	Load/store exclusive register
xx00	0	01x0xxxxxxxxxx	Load/store ordered
xx00	0	01x1xxxxxxxxxx	Compare and swap
xx01	0	10x0xxxxxxxxx10	Load/store ordered register pair
xx01	0	11x000000000010	Load/store ordered (writeback)
xx01	0	11x000000000110	UNALLOCATED
xx01	0	11x000000001x10	UNALLOCATED
xx01	0	11x00000001xx10	UNALLOCATED
xx01	0	11x0000001xxx10	UNALLOCATED
xx01	0	11x000001xxxx10	UNALLOCATED
xx01	0	11x00001xxxxx10	UNALLOCATED
xx01	0	11x0001xxxxxx10	UNALLOCATED
xx01	0	11x001xxxxxx10	UNALLOCATED
xx01	0	11x01xxxxxxx10	UNALLOCATED
xx01	0	1xx0xxxxxxxx00	Load/store ordered (unscaled immediate)
xx01	1	1xx0xxxxxxxx00	UNALLOCATED
xx01	1	1xx0xxxxxxxx10	Load/store ordered (SIMD&FP)
xx01	1	1xx0xxxxxxxx11	UNALLOCATED
xx01		0xxxxxxxxxxxxx	Load register (literal)
xx01		1xx0xxxxxxxx01	Memory Copy and Memory Set
xx10		00xxxxxxxxxxxx	Load/store no-allocate pair (offset)
xx10		01xxxxxxxxxxxx	Load/store register pair (post-indexed)
xx10		10xxxxxxxxxxxx	Load/store register pair (offset)
xx10		11xxxxxxxxxxxx	Load/store register pair (pre-indexed)
xx11		0xx0xxxxxxxx00	Load/store register (unscaled immediate)
xx11		0xx0xxxxxxxx01	Load/store register (immediate post-indexed)
xx11		0xx0xxxxxxxx10	Load/store register (unprivileged)
xx11		0xx0xxxxxxxx11	Load/store register (immediate pre-indexed)
xx11		0xx1xxxxxxxx00	Atomic memory operations
xx11		0xx1xxxxxxxx10	Load/store register (register offset)
xx11		0xx1xxxxxxxxx1	Load/store register (pac)
xx11		1xxxxxxxxxxxxx	Load/store register (unsigned immediate)

Compare and swap pair

The encodings in this section are decoded from Loads and Stores.



Decode fields				Instruction Details	Condition
sz	A	R	Rt2		
			!= 11111	UNALLOCATED	-
0	0	0	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASP	FEAT_LSE
0	0	1	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASPL	FEAT_LSE
0	1	0	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASPA	FEAT_LSE
0	1	1	11111	CASP, CASPA, CASPAL, CASPL — 32-bit CASPAL	FEAT_LSE
1	0	0	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASP	FEAT_LSE
1	0	1	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASPL	FEAT_LSE
1	1	0	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASPA	FEAT_LSE
1	1	1	11111	CASP, CASPA, CASPAL, CASPL — 64-bit CASPAL	FEAT_LSE

Compare and swap pair (unprivileged)

The encodings in this section are decoded from Loads and Stores.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	sz	0	0	1	0	0	1	1	A	0	Rs					R	Rt2					Rn					Rt				

Decode fields				Instruction Details	Condition
sz	A	R	Rt2		
0				UNALLOCATED	-
1			!= 11111	UNALLOCATED	-
1	0	0	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPT	FEAT_LSUI
1	0	1	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPLT	FEAT_LSUI
1	1	0	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPAT	FEAT_LSUI
1	1	1	11111	CASPT, CASPAT, CASPALT, CASPLT — CASPALT	FEAT_LSUI

Advanced SIMD load/store multiple structures

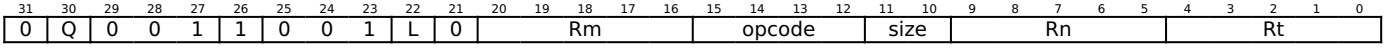
The encodings in this section are decoded from Loads and Stores.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	0	0	0	L	0	0	0	0	0	0	opcode				size		Rn					Rt				

Decode fields		Instruction Details	Condition
L	opcode		
	x0x1	UNALLOCATED	-
	0101	UNALLOCATED	-
	11xx	UNALLOCATED	-
0	0000	ST4 (multiple structures)	FEAT_AdvSIMD
0	0010	ST1 (multiple structures) — Four registers	FEAT_AdvSIMD
0	0100	ST3 (multiple structures)	FEAT_AdvSIMD
0	0110	ST1 (multiple structures) — Three registers	FEAT_AdvSIMD
0	0111	ST1 (multiple structures) — One register	FEAT_AdvSIMD
0	1000	ST2 (multiple structures)	FEAT_AdvSIMD
0	1010	ST1 (multiple structures) — Two registers	FEAT_AdvSIMD
1	0000	LD4 (multiple structures)	FEAT_AdvSIMD
1	0010	LD1 (multiple structures) — Four registers	FEAT_AdvSIMD
1	0100	LD3 (multiple structures)	FEAT_AdvSIMD
1	0110	LD1 (multiple structures) — Three registers	FEAT_AdvSIMD
1	0111	LD1 (multiple structures) — One register	FEAT_AdvSIMD
1	1000	LD2 (multiple structures)	FEAT_AdvSIMD
1	1010	LD1 (multiple structures) — Two registers	FEAT_AdvSIMD

Advanced SIMD load/store multiple structures (post-indexed)

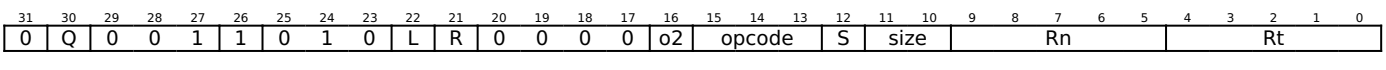
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
L	Rm	opcode		
		x0x1	UNALLOCATED	-
		0101	UNALLOCATED	-
		11xx	UNALLOCATED	-
0	11111	0000	ST4 (multiple structures) — Immediate offset	FEAT_AdvSIMD
0	11111	0010	ST1 (multiple structures) — Four registers, immediate offset	FEAT_AdvSIMD
0	11111	0100	ST3 (multiple structures) — Immediate offset	FEAT_AdvSIMD
0	11111	0110	ST1 (multiple structures) — Three registers, immediate offset	FEAT_AdvSIMD
0	11111	0111	ST1 (multiple structures) — One register, immediate offset	FEAT_AdvSIMD
0	11111	1000	ST2 (multiple structures) — Immediate offset	FEAT_AdvSIMD
0	11111	1010	ST1 (multiple structures) — Two registers, immediate offset	FEAT_AdvSIMD
0	!= 11111	0000	ST4 (multiple structures) — Register offset	FEAT_AdvSIMD
0	!= 11111	0010	ST1 (multiple structures) — Four registers, register offset	FEAT_AdvSIMD
0	!= 11111	0100	ST3 (multiple structures) — Register offset	FEAT_AdvSIMD
0	!= 11111	0110	ST1 (multiple structures) — Three registers, register offset	FEAT_AdvSIMD
0	!= 11111	0111	ST1 (multiple structures) — One register, register offset	FEAT_AdvSIMD
0	!= 11111	1000	ST2 (multiple structures) — Register offset	FEAT_AdvSIMD
0	!= 11111	1010	ST1 (multiple structures) — Two registers, register offset	FEAT_AdvSIMD
1	11111	0000	LD4 (multiple structures) — Immediate offset	FEAT_AdvSIMD
1	11111	0010	LD1 (multiple structures) — Four registers, immediate offset	FEAT_AdvSIMD
1	11111	0100	LD3 (multiple structures) — Immediate offset	FEAT_AdvSIMD
1	11111	0110	LD1 (multiple structures) — Three registers, immediate offset	FEAT_AdvSIMD
1	11111	0111	LD1 (multiple structures) — One register, immediate offset	FEAT_AdvSIMD
1	11111	1000	LD2 (multiple structures) — Immediate offset	FEAT_AdvSIMD
1	11111	1010	LD1 (multiple structures) — Two registers, immediate offset	FEAT_AdvSIMD
1	!= 11111	0000	LD4 (multiple structures) — Register offset	FEAT_AdvSIMD
1	!= 11111	0010	LD1 (multiple structures) — Four registers, register offset	FEAT_AdvSIMD
1	!= 11111	0100	LD3 (multiple structures) — Register offset	FEAT_AdvSIMD
1	!= 11111	0110	LD1 (multiple structures) — Three registers, register offset	FEAT_AdvSIMD
1	!= 11111	0111	LD1 (multiple structures) — One register, register offset	FEAT_AdvSIMD
1	!= 11111	1000	LD2 (multiple structures) — Register offset	FEAT_AdvSIMD
1	!= 11111	1010	LD1 (multiple structures) — Two registers, register offset	FEAT_AdvSIMD

Advanced SIMD load/store single structure

The encodings in this section are decoded from Loads and Stores.



Decode fields						Instruction Details	Condition
L	R	o2	opcode	S	size		
		0	01x		x1	UNALLOCATED	-
		0	10x		1x	UNALLOCATED	-
		0	10x	1	01	UNALLOCATED	-
	0	1	100		!= 01	UNALLOCATED	-
	0	1	100	1	01	UNALLOCATED	-
	0	1	!= 100			UNALLOCATED	-
	1	1				UNALLOCATED	-
0		0	11x			UNALLOCATED	-
0	0	0	000			ST1 (single structure) — 8-bit	FEAT_AdvSIMD
0	0	0	001			ST3 (single structure) — 8-bit	FEAT_AdvSIMD
0	0	0	010		x0	ST1 (single structure) — 16-bit	FEAT_AdvSIMD
0	0	0	011		x0	ST3 (single structure) — 16-bit	FEAT_AdvSIMD
0	0	0	100		00	ST1 (single structure) — 32-bit	FEAT_AdvSIMD
0	0	0	100	0	01	ST1 (single structure) — 64-bit	FEAT_AdvSIMD
0	0	0	101		00	ST3 (single structure) — 32-bit	FEAT_AdvSIMD
0	0	0	101	0	01	ST3 (single structure) — 64-bit	FEAT_AdvSIMD
0	0	1	100	0	01	STL1 (SIMD&FP)	FEAT_AdvSIMD && FEAT_LRCPC3
0	1	0	000			ST2 (single structure) — 8-bit	FEAT_AdvSIMD
0	1	0	001			ST4 (single structure) — 8-bit	FEAT_AdvSIMD
0	1	0	010		x0	ST2 (single structure) — 16-bit	FEAT_AdvSIMD
0	1	0	011		x0	ST4 (single structure) — 16-bit	FEAT_AdvSIMD
0	1	0	100		00	ST2 (single structure) — 32-bit	FEAT_AdvSIMD
0	1	0	100	0	01	ST2 (single structure) — 64-bit	FEAT_AdvSIMD
0	1	0	101		00	ST4 (single structure) — 32-bit	FEAT_AdvSIMD
0	1	0	101	0	01	ST4 (single structure) — 64-bit	FEAT_AdvSIMD
1		0	11x	1		UNALLOCATED	-
1	0	0	000			LD1 (single structure) — 8-bit	FEAT_AdvSIMD
1	0	0	001			LD3 (single structure) — 8-bit	FEAT_AdvSIMD
1	0	0	010		x0	LD1 (single structure) — 16-bit	FEAT_AdvSIMD
1	0	0	011		x0	LD3 (single structure) — 16-bit	FEAT_AdvSIMD
1	0	0	100		00	LD1 (single structure) — 32-bit	FEAT_AdvSIMD
1	0	0	100	0	01	LD1 (single structure) — 64-bit	FEAT_AdvSIMD
1	0	0	101		00	LD3 (single structure) — 32-bit	FEAT_AdvSIMD
1	0	0	101	0	01	LD3 (single structure) — 64-bit	FEAT_AdvSIMD
1	0	0	110	0		LD1R	FEAT_AdvSIMD
1	0	0	111	0		LD3R	FEAT_AdvSIMD
1	0	1	100	0	01	LDAP1 (SIMD&FP)	FEAT_AdvSIMD && FEAT_LRCPC3
1	1	0	000			LD2 (single structure) — 8-bit	FEAT_AdvSIMD
1	1	0	001			LD4 (single structure) — 8-bit	FEAT_AdvSIMD
1	1	0	010		x0	LD2 (single structure) — 16-bit	FEAT_AdvSIMD
1	1	0	011		x0	LD4 (single structure) — 16-bit	FEAT_AdvSIMD
1	1	0	100		00	LD2 (single structure) — 32-bit	FEAT_AdvSIMD
1	1	0	100	0	01	LD2 (single structure) — 64-bit	FEAT_AdvSIMD
1	1	0	101		00	LD4 (single structure) — 32-bit	FEAT_AdvSIMD
1	1	0	101	0	01	LD4 (single structure) — 64-bit	FEAT_AdvSIMD

Decode fields						Instruction Details	Condition
L	R	o2	opcode	S	size		
1	1	0	110	0		LD2R	FEAT_AdvSIMD
1	1	0	111	0		LD4R	FEAT_AdvSIMD

Advanced SIMD load/store single structure (post-indexed)

The encodings in this section are decoded from Loads and Stores.

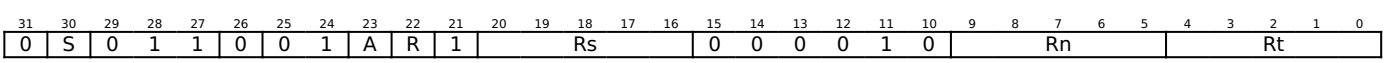
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	Q	0	0	1	1	0	1	1	L	R	Rm					opcode			S	size		Rn					Rt				

Decode fields						Instruction Details	Condition
L	R	Rm	opcode	S	size		
			01x		x1	UNALLOCATED	-
			10x		1x	UNALLOCATED	-
			10x	1	01	UNALLOCATED	-
0			11x			UNALLOCATED	-
0	0	11111	000			ST1 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	001			ST3 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	010		x0	ST1 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	011		x0	ST3 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	100		00	ST1 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	100	0	01	ST1 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	101		00	ST3 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	0	11111	101	0	01	ST3 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	0	!= 11111	000			ST1 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	001			ST3 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	010		x0	ST1 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	011		x0	ST3 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	100		00	ST1 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	100	0	01	ST1 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	101		00	ST3 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	0	!= 11111	101	0	01	ST3 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
0	1	11111	000			ST2 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	001			ST4 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	010		x0	ST2 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	011		x0	ST4 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	100		00	ST2 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	100	0	01	ST2 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	101		00	ST4 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
0	1	11111	101	0	01	ST4 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
0	1	!= 11111	000			ST2 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	001			ST4 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	010		x0	ST2 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	011		x0	ST4 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	100		00	ST2 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	100	0	01	ST2 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	101		00	ST4 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
0	1	!= 11111	101	0	01	ST4 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1			11x	1		UNALLOCATED	-
1	0	11111	000			LD1 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	001			LD3 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	010		x0	LD1 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	011		x0	LD3 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	100		00	LD1 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	100	0	01	LD1 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	101		00	LD3 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	0	11111	101	0	01	LD3 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD

Decode fields						Instruction Details	Condition
L	R	Rm	opcode	S	size		
1	0	11111	110	0		LD1R — Immediate offset	FEAT_AdvSIMD
1	0	11111	111	0		LD3R — Immediate offset	FEAT_AdvSIMD
1	0	!= 11111	000			LD1 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	001			LD3 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	010		x0	LD1 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	011		x0	LD3 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	100		00	LD1 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	100	0	01	LD1 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	101		00	LD3 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	101	0	01	LD3 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	0	!= 11111	110	0		LD1R — Register offset	FEAT_AdvSIMD
1	0	!= 11111	111	0		LD3R — Register offset	FEAT_AdvSIMD
1	1	11111	000			LD2 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	001			LD4 (single structure) — 8-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	010		x0	LD2 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	011		x0	LD4 (single structure) — 16-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	100		00	LD2 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	100	0	01	LD2 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	101		00	LD4 (single structure) — 32-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	101	0	01	LD4 (single structure) — 64-bit, immediate offset	FEAT_AdvSIMD
1	1	11111	110	0		LD2R — Immediate offset	FEAT_AdvSIMD
1	1	11111	111	0		LD4R — Immediate offset	FEAT_AdvSIMD
1	1	!= 11111	000			LD2 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	001			LD4 (single structure) — 8-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	010		x0	LD2 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	011		x0	LD4 (single structure) — 16-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	100		00	LD2 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	100	0	01	LD2 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	101		00	LD4 (single structure) — 32-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	101	0	01	LD4 (single structure) — 64-bit, register offset	FEAT_AdvSIMD
1	1	!= 11111	110	0		LD2R — Register offset	FEAT_AdvSIMD
1	1	!= 11111	111	0		LD4R — Register offset	FEAT_AdvSIMD

RCW compare and swap

The encodings in this section are decoded from Loads and Stores.



Decode fields S	A	R	Instruction Details	Condition
0	0	0	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCAS	FEAT_THE
0	0	1	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCASL	FEAT_THE
0	1	0	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCASA	FEAT_THE
0	1	1	RCWCAS, RCWCASA, RCWCASAL, RCWCASL — RCWCASAL	FEAT_THE
1	0	0	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCAS	FEAT_THE
1	0	1	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCASL	FEAT_THE
1	1	0	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCASA	FEAT_THE
1	1	1	RCWSCAS, RCWSCASA, RCWSCASAL, RCWSCASL — RCWSCASAL	FEAT_THE

RCW compare and swap pair

The encodings in this section are decoded from Loads and Stores.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	S	0	1	1	0	0	1	A	R	1	Rs					0	0	0	0	1	1	Rn					Rt				

Decode fields S	A	R	Instruction Details	Condition
0	0	0	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASP	FEAT_D128 && FEAT_THE
0	0	1	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASPL	FEAT_D128 && FEAT_THE
0	1	0	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASPA	FEAT_D128 && FEAT_THE
0	1	1	RCWCASP, RCWCASPA, RCWCASPAL, RCWCASPL — RCWCASPAL	FEAT_D128 && FEAT_THE
1	0	0	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASP	FEAT_D128 && FEAT_THE
1	0	1	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASPL	FEAT_D128 && FEAT_THE
1	1	0	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASPA	FEAT_D128 && FEAT_THE
1	1	1	RCWSCASP, RCWSCASPA, RCWSCASPAL, RCWSCASPL — RCWSCASPAL	FEAT_D128 && FEAT_THE

128-bit atomic memory operations

The encodings in this section are decoded from Loads and Stores.

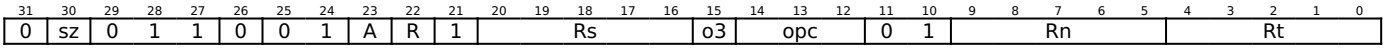
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	S	0	1	1	0	0	1	A	R	1	Rt2					o3	opc			0	0	Rn					Rt				

Decode fields					Instruction Details	Condition
S	A	R	o3	opc		
				1xx	UNALLOCATED	-
0			0	0x0	UNALLOCATED	-
0	0	0	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRP	FEAT_LSE128
0	0	0	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETP	FEAT_LSE128
0	0	0	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPP	FEAT_LSE128
0	0	0	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRP	FEAT_D128 && FEAT_THE
0	0	0	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPP	FEAT_D128 && FEAT_THE
0	0	0	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETP	FEAT_D128 && FEAT_THE
0	0	1	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRPL	FEAT_LSE128
0	0	1	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETPL	FEAT_LSE128
0	0	1	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPPL	FEAT_LSE128
0	0	1	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRPL	FEAT_D128 && FEAT_THE
0	0	1	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPPL	FEAT_D128 && FEAT_THE
0	0	1	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETPL	FEAT_D128 && FEAT_THE
0	1	0	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRPA	FEAT_LSE128
0	1	0	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETPA	FEAT_LSE128
0	1	0	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPPA	FEAT_LSE128
0	1	0	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRPA	FEAT_D128 && FEAT_THE
0	1	0	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPPA	FEAT_D128 && FEAT_THE
0	1	0	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETPA	FEAT_D128 && FEAT_THE
0	1	1	0	001	LDCLRP, LDCLRPA, LDCLRPAL, LDCLRPL — LDCLRPAL	FEAT_LSE128
0	1	1	0	011	LDSETP, LDSETPA, LDSETPAL, LDSETPL — LDSETPAL	FEAT_LSE128
0	1	1	1	000	SWPP, SWPPA, SWPPAL, SWPPL — SWPPAL	FEAT_LSE128
0	1	1	1	001	RCWCLRP, RCWCLRPA, RCWCLRPAL, RCWCLRPL — RCWCLRPAL	FEAT_D128 && FEAT_THE
0	1	1	1	010	RCWSWPP, RCWSWPPA, RCWSWPPAL, RCWSWPPL — RCWSWPPAL	FEAT_D128 && FEAT_THE
0	1	1	1	011	RCWSETP, RCWSETPA, RCWSETPAL, RCWSETPL — RCWSETPAL	FEAT_D128 && FEAT_THE
1			0	0xx	UNALLOCATED	-
1			1	000	UNALLOCATED	-
1	0	0	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRP	FEAT_D128 && FEAT_THE
1	0	0	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPP	FEAT_D128 && FEAT_THE
1	0	0	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETP	FEAT_D128 && FEAT_THE
1	0	1	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRPL	FEAT_D128 && FEAT_THE
1	0	1	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPPL	FEAT_D128 && FEAT_THE
1	0	1	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETPL	FEAT_D128 && FEAT_THE
1	1	0	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRPA	FEAT_D128 && FEAT_THE

Decode fields					Instruction Details	Condition
S	A	R	o3	opc		
1	1	0	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPPA	FEAT_D128 && FEAT_THE
1	1	0	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETPA	FEAT_D128 && FEAT_THE
1	1	1	1	001	RCWSCLRP, RCWSCLRPA, RCWSCLRPAL, RCWSCLRPL — RCWSCLRPAL	FEAT_D128 && FEAT_THE
1	1	1	1	010	RCWSSWPP, RCWSSWPPA, RCWSSWPPAL, RCWSSWPPL — RCWSSWPPAL	FEAT_D128 && FEAT_THE
1	1	1	1	011	RCWSSETP, RCWSSETPA, RCWSSETPAL, RCWSSETPL — RCWSSETPAL	FEAT_D128 && FEAT_THE

Atomic memory operations (unprivileged)

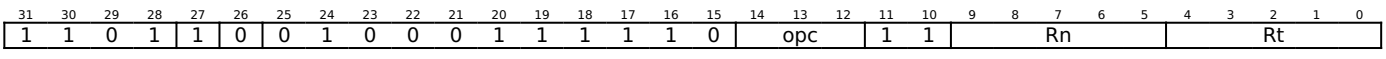
The encodings in this section are decoded from Loads and Stores.



Decode fields					Instruction Details	Condition
sz	A	R	o3	opc		
				010	UNALLOCATED	-
				1xx	UNALLOCATED	-
			1	0x1	UNALLOCATED	-
0	0	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit no memory ordering	FEAT_LSUI
0	0	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit no memory ordering	FEAT_LSUI
0	0	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit no memory ordering	FEAT_LSUI
0	0	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPT	FEAT_LSUI
0	0	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit release	FEAT_LSUI
0	0	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit release	FEAT_LSUI
0	0	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit release	FEAT_LSUI
0	0	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPTL	FEAT_LSUI
0	1	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit acquire	FEAT_LSUI
0	1	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit acquire	FEAT_LSUI
0	1	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit acquire	FEAT_LSUI
0	1	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPTA	FEAT_LSUI
0	1	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 32-bit acquire-release	FEAT_LSUI
0	1	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 32-bit acquire-release	FEAT_LSUI
0	1	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 32-bit acquire-release	FEAT_LSUI
0	1	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 32-bit SWPTAL	FEAT_LSUI
1	0	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit no memory ordering	FEAT_LSUI
1	0	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit no memory ordering	FEAT_LSUI
1	0	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit no memory ordering	FEAT_LSUI
1	0	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPT	FEAT_LSUI
1	0	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit release	FEAT_LSUI
1	0	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit release	FEAT_LSUI
1	0	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit release	FEAT_LSUI
1	0	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPTL	FEAT_LSUI
1	1	0	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit acquire	FEAT_LSUI
1	1	0	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit acquire	FEAT_LSUI
1	1	0	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit acquire	FEAT_LSUI
1	1	0	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPTA	FEAT_LSUI
1	1	1	0	000	LDTADD, LDTADDA, LDTADDAL, LDTADDL — 64-bit acquire-release	FEAT_LSUI
1	1	1	0	001	LDTCLR, LDTCLRA, LDTCLRAL, LDTCLRL — 64-bit acquire-release	FEAT_LSUI
1	1	1	0	011	LDTSET, LDTSETA, LDTSETAL, LDTSETL — 64-bit acquire-release	FEAT_LSUI
1	1	1	1	000	SWPT, SWPTA, SWPTAL, SWPTL — 64-bit SWPTAL	FEAT_LSUI

GCS load/store

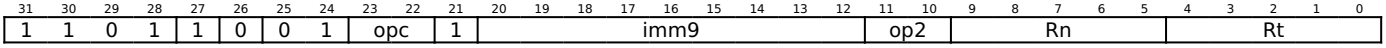
The encodings in this section are decoded from Loads and Stores.



Decode fields	Instruction Details	Condition
opc		
000	GCSSTR	FEAT_GCS
001	GCSSTTR	FEAT_GCS
!= 00x	UNALLOCATED	-

Load/store memory tags

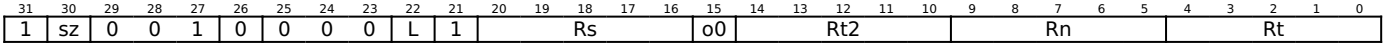
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
opc	imm9	op2		
00		01	STG — Post-index	FEAT_MTE
00		10	STG — Signed offset	FEAT_MTE
00		11	STG — Pre-index	FEAT_MTE
00	000000000	00	STZGM	FEAT_MTE2
01		00	LDG	FEAT_MTE
01		01	STZG — Post-index	FEAT_MTE
01		10	STZG — Signed offset	FEAT_MTE
01		11	STZG — Pre-index	FEAT_MTE
10		01	ST2G — Post-index	FEAT_MTE
10		10	ST2G — Signed offset	FEAT_MTE
10		11	ST2G — Pre-index	FEAT_MTE
10	000000000	00	STGM	FEAT_MTE2
11		01	STZ2G — Post-index	FEAT_MTE
11		10	STZ2G — Signed offset	FEAT_MTE
11		11	STZ2G — Pre-index	FEAT_MTE
11	000000000	00	LDGM	FEAT_MTE2
!= 01	!= 000000000	00	UNALLOCATED	-

Load/store exclusive pair

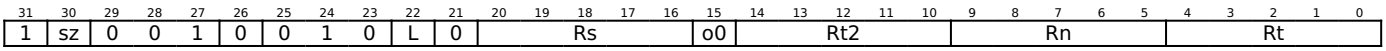
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details
sz	L	o0	
0	0	0	STXP — 32-bit
0	0	1	STLXP — 32-bit
0	1	0	LDXP — 32-bit
0	1	1	LDAXP — 32-bit
1	0	0	STXP — 64-bit
1	0	1	STLXP — 64-bit
1	1	0	LDXP — 64-bit
1	1	1	LDAXP — 64-bit

Load/store exclusive register (unprivileged)

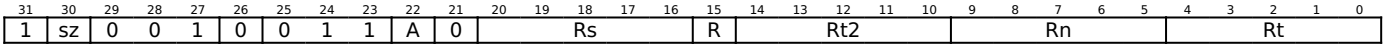
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
sz	L	o0		
0	0	0	STTXR — 32-bit	FEAT_LSUI
0	0	1	STLTXR — 32-bit	FEAT_LSUI
0	1	0	LDTXR — 32-bit	FEAT_LSUI
0	1	1	LDATXR — 32-bit	FEAT_LSUI
1	0	0	STTXR — 64-bit	FEAT_LSUI
1	0	1	STLTXR — 64-bit	FEAT_LSUI
1	1	0	LDTXR — 64-bit	FEAT_LSUI
1	1	1	LDATXR — 64-bit	FEAT_LSUI

Compare and swap (unprivileged)

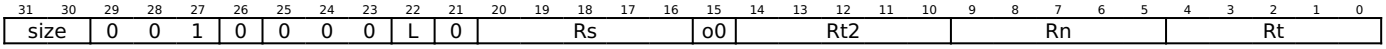
The encodings in this section are decoded from Loads and Stores.



Decode fields				Instruction Details	Condition
sz	A	R	Rt2		
0				UNALLOCATED	-
1			!= 11111	UNALLOCATED	-
1	0	0	11111	CAST, CASAT, CASALT, CASLT — CAST	FEAT_LSUI
1	0	1	11111	CAST, CASAT, CASALT, CASLT — CASLT	FEAT_LSUI
1	1	0	11111	CAST, CASAT, CASALT, CASLT — CASAT	FEAT_LSUI
1	1	1	11111	CAST, CASAT, CASALT, CASLT — CASALT	FEAT_LSUI

Load/store exclusive register

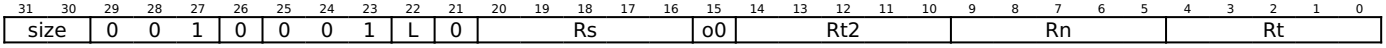
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details
size	L	o0	
00	0	0	STXRB
00	0	1	STLXRB
00	1	0	LDXRB
00	1	1	LDAXRB
01	0	0	STXRH
01	0	1	STLXRH
01	1	0	LDXRH
01	1	1	LDAXRH
10	0	0	STXR — 32-bit
10	0	1	STLXR — 32-bit
10	1	0	LDXR — 32-bit
10	1	1	LDAXR — 32-bit
11	0	0	STXR — 64-bit
11	0	1	STLXR — 64-bit
11	1	0	LDXR — 64-bit
11	1	1	LDAXR — 64-bit

Load/store ordered

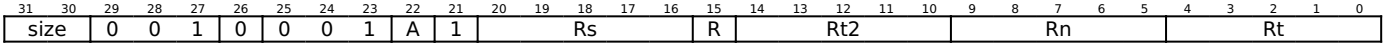
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	L	o0		
00	0	0	STLLRB	FEAT_LOR
00	0	1	STLRB	-
00	1	0	LDLARB	FEAT_LOR
00	1	1	LDARB	-
01	0	0	STLLRH	FEAT_LOR
01	0	1	STLRH	-
01	1	0	LDLARH	FEAT_LOR
01	1	1	LDARH	-
10	0	0	STLLR — 32-bit	FEAT_LOR
10	0	1	STLR — 32-bit	-
10	1	0	LDLAR — 32-bit	FEAT_LOR
10	1	1	LDAR — 32-bit	-
11	0	0	STLLR — 64-bit	FEAT_LOR
11	0	1	STLR — 64-bit	-
11	1	0	LDLAR — 64-bit	FEAT_LOR
11	1	1	LDAR — 64-bit	-

Compare and swap

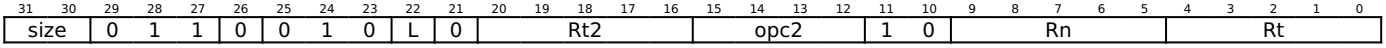
The encodings in this section are decoded from Loads and Stores.



Decode fields				Instruction Details	Condition
size	A	R	Rt2		
			!= 11111	UNALLOCATED	-
00	0	0	11111	CASB, CASAB, CASALB, CASLB — CASB	FEAT_LSE
00	0	1	11111	CASB, CASAB, CASALB, CASLB — CASLB	FEAT_LSE
00	1	0	11111	CASB, CASAB, CASALB, CASLB — CASAB	FEAT_LSE
00	1	1	11111	CASB, CASAB, CASALB, CASLB — CASALB	FEAT_LSE
01	0	0	11111	CASH, CASAH, CASALH, CASLH — CASH	FEAT_LSE
01	0	1	11111	CASH, CASAH, CASALH, CASLH — CASLH	FEAT_LSE
01	1	0	11111	CASH, CASAH, CASALH, CASLH — CASAH	FEAT_LSE
01	1	1	11111	CASH, CASAH, CASALH, CASLH — CASALH	FEAT_LSE
10	0	0	11111	CAS, CASA, CASAL, CASL — 32-bit CAS	FEAT_LSE
10	0	1	11111	CAS, CASA, CASAL, CASL — 32-bit CASL	FEAT_LSE
10	1	0	11111	CAS, CASA, CASAL, CASL — 32-bit CASA	FEAT_LSE
10	1	1	11111	CAS, CASA, CASAL, CASL — 32-bit CASAL	FEAT_LSE
11	0	0	11111	CAS, CASA, CASAL, CASL — 64-bit CAS	FEAT_LSE
11	0	1	11111	CAS, CASA, CASAL, CASL — 64-bit CASL	FEAT_LSE
11	1	0	11111	CAS, CASA, CASAL, CASL — 64-bit CASA	FEAT_LSE
11	1	1	11111	CAS, CASA, CASAL, CASL — 64-bit CASAL	FEAT_LSE

Load/store ordered register pair

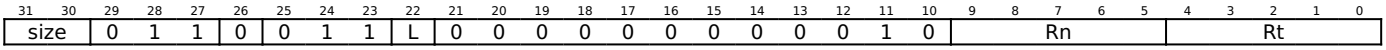
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	L	opc2		
0x			UNALLOCATED	-
1x		1xxx	UNALLOCATED	-
1x	0	001x	UNALLOCATED	-
1x	1	001x	UNALLOCATED	-
10		01xx	UNALLOCATED	-
10	0	0000	STILP — 32-bit pre-index	FEAT_LRCPC3
10	0	0001	STILP — 32-bit	FEAT_LRCPC3
10	1	0000	LDIAPP — 32-bit post-index	FEAT_LRCPC3
10	1	0001	LDIAPP — 32-bit	FEAT_LRCPC3
11		0100	UNALLOCATED	-
11	0	0000	STILP — 64-bit pre-index	FEAT_LRCPC3
11	0	0001	STILP — 64-bit	FEAT_LRCPC3
11	0	0101	STLP	FEAT_LSCP
11	0	011x	UNALLOCATED	-
11	1	0000	LDIAPP — 64-bit post-index	FEAT_LRCPC3
11	1	0001	LDIAPP — 64-bit	FEAT_LRCPC3
11	1	0101	LDAP	FEAT_LSCP
11	1	0110	UNALLOCATED	-
11	1	0111	LDAPP	FEAT_LSCP

Load/store ordered (writeback)

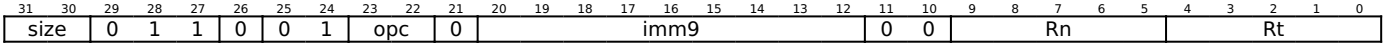
The encodings in this section are decoded from Loads and Stores.



Decode fields		Instruction Details	Condition
size	L		
0x		UNALLOCATED	-
10	0	STLR — 32-bit	FEAT_LRCPC3
10	1	LDAPR — 32-bit	FEAT_LRCPC3
11	0	STLR — 64-bit	FEAT_LRCPC3
11	1	LDAPR — 64-bit	FEAT_LRCPC3

Load/store ordered (unscaled immediate)

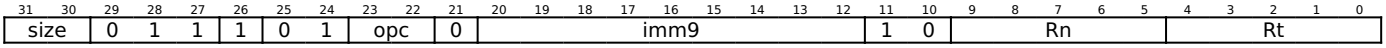
The encodings in this section are decoded from Loads and Stores.



Decode fields		Instruction Details	Condition
size	opc		
00	00	STLURB	FEAT_LRCPC2
00	01	LDAPURB	FEAT_LRCPC2
00	10	LDAPURSB — 64-bit	FEAT_LRCPC2
00	11	LDAPURSB — 32-bit	FEAT_LRCPC2
01	00	STLURH	FEAT_LRCPC2
01	01	LDAPURH	FEAT_LRCPC2
01	10	LDAPURSH — 64-bit	FEAT_LRCPC2
01	11	LDAPURSH — 32-bit	FEAT_LRCPC2
10	00	STLUR — 32-bit	FEAT_LRCPC2
10	01	LDAPUR — 32-bit	FEAT_LRCPC2
10	10	LDAPURSW	FEAT_LRCPC2
10	11	UNALLOCATED	-
11	00	STLUR — 64-bit	FEAT_LRCPC2
11	01	LDAPUR — 64-bit	FEAT_LRCPC2
11	1x	UNALLOCATED	-

Load/store ordered (SIMD&FP)

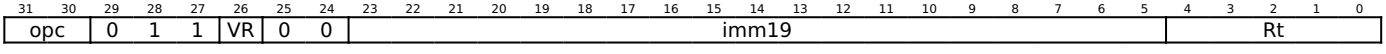
The encodings in this section are decoded from Loads and Stores.



Decode fields		Instruction Details	Condition
size	opc		
00	00	STLUR (SIMD&FP) — 8-bit	FEAT_FP && FEAT_LRCPC3
00	01	LDAPUR (SIMD&FP) — 8-bit	FEAT_FP && FEAT_LRCPC3
00	10	STLUR (SIMD&FP) — 128-bit	FEAT_FP && FEAT_LRCPC3
00	11	LDAPUR (SIMD&FP) — 128-bit	FEAT_FP && FEAT_LRCPC3
01	00	STLUR (SIMD&FP) — 16-bit	FEAT_FP && FEAT_LRCPC3
01	01	LDAPUR (SIMD&FP) — 16-bit	FEAT_FP && FEAT_LRCPC3
10	00	STLUR (SIMD&FP) — 32-bit	FEAT_FP && FEAT_LRCPC3
10	01	LDAPUR (SIMD&FP) — 32-bit	FEAT_FP && FEAT_LRCPC3
11	00	STLUR (SIMD&FP) — 64-bit	FEAT_FP && FEAT_LRCPC3
11	01	LDAPUR (SIMD&FP) — 64-bit	FEAT_FP && FEAT_LRCPC3
!= 00	1x	UNALLOCATED	-

Load register (literal)

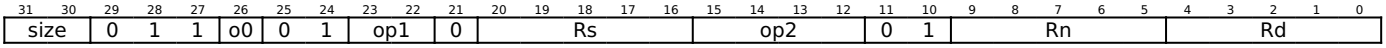
The encodings in this section are decoded from Loads and Stores.



Decode fields		Instruction Details	Condition
opc	VR		
00	0	LDR (literal) — 32-bit	-
00	1	LDR (literal, SIMD&FP) — 32-bit	FEAT_FP
01	0	LDR (literal) — 64-bit	-
01	1	LDR (literal, SIMD&FP) — 64-bit	FEAT_FP
10	0	LDRSW (literal)	-
10	1	LDR (literal, SIMD&FP) — 128-bit	FEAT_FP
11	0	PRFM (literal)	-
11	1	UNALLOCATED	-

Memory Copy and Memory Set

The encodings in this section are decoded from Loads and Stores.



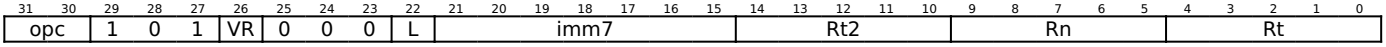
Decode fields			Instruction Details	Condition
o0	op1	op2		
	11	11xx	UNALLOCATED	-
0	00	0000	CPYFP, CPYFM, CPYFE — Prologue	FEAT_MOPS
0	00	0001	CPYFPWT, CPYFMWT, CPYFEWT — Prologue	FEAT_MOPS
0	00	0010	CPYFPRT, CPYFMRT, CPYFERT — Prologue	FEAT_MOPS
0	00	0011	CPYFPT, CPYFMT, CPYFET — Prologue	FEAT_MOPS
0	00	0100	CPYFPWN, CPYFMWN, CPYFEWN — Prologue	FEAT_MOPS
0	00	0101	CPYFPWTWN, CPYFMWTWN, CPYFEWTWN — Prologue	FEAT_MOPS
0	00	0110	CPYFPRTWN, CPYFMRTWN, CPYFERTWN — Prologue	FEAT_MOPS
0	00	0111	CPYFPTWN, CPYFMTWN, CPYFETWN — Prologue	FEAT_MOPS
0	00	1000	CPYFPRN, CPYFMRN, CPYFERN — Prologue	FEAT_MOPS
0	00	1001	CPYFPWTRN, CPYFMWTRN, CPYFEWTRN — Prologue	FEAT_MOPS
0	00	1010	CPYFPRTRN, CPYFMRTRN, CPYFERTRN — Prologue	FEAT_MOPS
0	00	1011	CPYFPTRN, CPYFMTRN, CPYFETRN — Prologue	FEAT_MOPS
0	00	1100	CPYFPN, CPYFMN, CPYFEN — Prologue	FEAT_MOPS
0	00	1101	CPYFPWTN, CPYFMWTN, CPYFEWTN — Prologue	FEAT_MOPS
0	00	1110	CPYFPRTN, CPYFMRTN, CPYFERTN — Prologue	FEAT_MOPS
0	00	1111	CPYFPTN, CPYFMTN, CPYFETN — Prologue	FEAT_MOPS
0	01	0000	CPYFP, CPYFM, CPYFE — Main	FEAT_MOPS
0	01	0001	CPYFPWT, CPYFMWT, CPYFEWT — Main	FEAT_MOPS
0	01	0010	CPYFPRT, CPYFMRT, CPYFERT — Main	FEAT_MOPS
0	01	0011	CPYFPT, CPYFMT, CPYFET — Main	FEAT_MOPS
0	01	0100	CPYFPWN, CPYFMWN, CPYFEWN — Main	FEAT_MOPS
0	01	0101	CPYFPWTWN, CPYFMWTWN, CPYFEWTWN — Main	FEAT_MOPS
0	01	0110	CPYFPRTWN, CPYFMRTWN, CPYFERTWN — Main	FEAT_MOPS
0	01	0111	CPYFPTWN, CPYFMTWN, CPYFETWN — Main	FEAT_MOPS
0	01	1000	CPYFPRN, CPYFMRN, CPYFERN — Main	FEAT_MOPS
0	01	1001	CPYFPWTRN, CPYFMWTRN, CPYFEWTRN — Main	FEAT_MOPS
0	01	1010	CPYFPRTRN, CPYFMRTRN, CPYFERTRN — Main	FEAT_MOPS
0	01	1011	CPYFPTRN, CPYFMTRN, CPYFETRN — Main	FEAT_MOPS
0	01	1100	CPYFPN, CPYFMN, CPYFEN — Main	FEAT_MOPS
0	01	1101	CPYFPWTN, CPYFMWTN, CPYFEWTN — Main	FEAT_MOPS
0	01	1110	CPYFPRTN, CPYFMRTN, CPYFERTN — Main	FEAT_MOPS
0	01	1111	CPYFPTN, CPYFMTN, CPYFETN — Main	FEAT_MOPS
0	10	0000	CPYFP, CPYFM, CPYFE — Epilogue	FEAT_MOPS
0	10	0001	CPYFPWT, CPYFMWT, CPYFEWT — Epilogue	FEAT_MOPS
0	10	0010	CPYFPRT, CPYFMRT, CPYFERT — Epilogue	FEAT_MOPS
0	10	0011	CPYFPT, CPYFMT, CPYFET — Epilogue	FEAT_MOPS
0	10	0100	CPYFPWN, CPYFMWN, CPYFEWN — Epilogue	FEAT_MOPS
0	10	0101	CPYFPWTWN, CPYFMWTWN, CPYFEWTWN — Epilogue	FEAT_MOPS
0	10	0110	CPYFPRTWN, CPYFMRTWN, CPYFERTWN — Epilogue	FEAT_MOPS
0	10	0111	CPYFPTWN, CPYFMTWN, CPYFETWN — Epilogue	FEAT_MOPS
0	10	1000	CPYFPRN, CPYFMRN, CPYFERN — Epilogue	FEAT_MOPS
0	10	1001	CPYFPWTRN, CPYFMWTRN, CPYFEWTRN — Epilogue	FEAT_MOPS
0	10	1010	CPYFPRTRN, CPYFMRTRN, CPYFERTRN — Epilogue	FEAT_MOPS
0	10	1011	CPYFPTRN, CPYFMTRN, CPYFETRN — Epilogue	FEAT_MOPS

Decode fields			Instruction Details	Condition
o0	op1	op2		
0	10	1100	CPYFPN, CPYFMN, CPYFEN — Epilogue	FEAT_MOPS
0	10	1101	CPYFPWTN, CPYFMWTN, CPYFEWTN — Epilogue	FEAT_MOPS
0	10	1110	CPYFPRTN, CPYFMRTN, CPYFERTN — Epilogue	FEAT_MOPS
0	10	1111	CPYFPTN, CPYFMTN, CPYFETN — Epilogue	FEAT_MOPS
0	11	0000	SETP, SETM, SETE — Prologue	FEAT_MOPS
0	11	0001	SETPT, SETMT, SETET — Prologue	FEAT_MOPS
0	11	0010	SETPN, SETMN, SETEN — Prologue	FEAT_MOPS
0	11	0011	SETPTN, SETMTN, SETETN — Prologue	FEAT_MOPS
0	11	0100	SETP, SETM, SETE — Main	FEAT_MOPS
0	11	0101	SETPT, SETMT, SETET — Main	FEAT_MOPS
0	11	0110	SETPN, SETMN, SETEN — Main	FEAT_MOPS
0	11	0111	SETPTN, SETMTN, SETETN — Main	FEAT_MOPS
0	11	1000	SETP, SETM, SETE — Epilogue	FEAT_MOPS
0	11	1001	SETPT, SETMT, SETET — Epilogue	FEAT_MOPS
0	11	1010	SETPN, SETMN, SETEN — Epilogue	FEAT_MOPS
0	11	1011	SETPTN, SETMTN, SETETN — Epilogue	FEAT_MOPS
1	00	0000	CPYP, CPYM, CPYE — Prologue	FEAT_MOPS
1	00	0001	CPYPWT, CPYMWT, CPYEWT — Prologue	FEAT_MOPS
1	00	0010	CPYPRT, CPYMRT, CPYERT — Prologue	FEAT_MOPS
1	00	0011	CPYPT, CPYMT, CPYET — Prologue	FEAT_MOPS
1	00	0100	CPYPWN, CPYMWN, CPYEWN — Prologue	FEAT_MOPS
1	00	0101	CPYPWTWN, CPYMWTWN, CPYEWTWN — Prologue	FEAT_MOPS
1	00	0110	CPYPRTWN, CPYMRTWN, CPYERTWN — Prologue	FEAT_MOPS
1	00	0111	CPYPTWN, CPYMTWN, CPYETWN — Prologue	FEAT_MOPS
1	00	1000	CPYPRN, CPYMRN, CPYERN — Prologue	FEAT_MOPS
1	00	1001	CPYPWTRN, CPYMWTRN, CPYEWTRN — Prologue	FEAT_MOPS
1	00	1010	CPYPRTRN, CPYMRTRN, CPYERTRN — Prologue	FEAT_MOPS
1	00	1011	CPYPTRN, CPYMTRN, CPYETRN — Prologue	FEAT_MOPS
1	00	1100	CPYPN, CPYMN, CPYEN — Prologue	FEAT_MOPS
1	00	1101	CPYPWTN, CPYMWTN, CPYEWTN — Prologue	FEAT_MOPS
1	00	1110	CPYPRTN, CPYMRTN, CPYERTN — Prologue	FEAT_MOPS
1	00	1111	CPYPTN, CPYMTN, CPYETN — Prologue	FEAT_MOPS
1	01	0000	CPYP, CPYM, CPYE — Main	FEAT_MOPS
1	01	0001	CPYPWT, CPYMWT, CPYEWT — Main	FEAT_MOPS
1	01	0010	CPYPRT, CPYMRT, CPYERT — Main	FEAT_MOPS
1	01	0011	CPYPT, CPYMT, CPYET — Main	FEAT_MOPS
1	01	0100	CPYPWN, CPYMWN, CPYEWN — Main	FEAT_MOPS
1	01	0101	CPYPWTWN, CPYMWTWN, CPYEWTWN — Main	FEAT_MOPS
1	01	0110	CPYPRTWN, CPYMRTWN, CPYERTWN — Main	FEAT_MOPS
1	01	0111	CPYPTWN, CPYMTWN, CPYETWN — Main	FEAT_MOPS
1	01	1000	CPYPRN, CPYMRN, CPYERN — Main	FEAT_MOPS
1	01	1001	CPYPWTRN, CPYMWTRN, CPYEWTRN — Main	FEAT_MOPS
1	01	1010	CPYPRTRN, CPYMRTRN, CPYERTRN — Main	FEAT_MOPS
1	01	1011	CPYPTRN, CPYMTRN, CPYETRN — Main	FEAT_MOPS
1	01	1100	CPYPN, CPYMN, CPYEN — Main	FEAT_MOPS
1	01	1101	CPYPWTN, CPYMWTN, CPYEWTN — Main	FEAT_MOPS

Decode fields			Instruction Details	Condition
o0	op1	op2		
1	01	1110	CPYPRTN, CPYMRTN, CPYERTN — Main	FEAT_MOPS
1	01	1111	CPYPTN, CPYMTN, CPYETN — Main	FEAT_MOPS
1	10	0000	CPYP, CPYM, CPYE — Epilogue	FEAT_MOPS
1	10	0001	CPYPWT, CPYMWT, CPYEWT — Epilogue	FEAT_MOPS
1	10	0010	CPYPRT, CPYMRT, CPYERT — Epilogue	FEAT_MOPS
1	10	0011	CPYPT, CPYMT, CPYET — Epilogue	FEAT_MOPS
1	10	0100	CPYPWN, CPYMWN, CPYEWN — Epilogue	FEAT_MOPS
1	10	0101	CPYPWTWN, CPYMWTWN, CPYEWTWN — Epilogue	FEAT_MOPS
1	10	0110	CPYPRTWN, CPYMRTWN, CPYERTWN — Epilogue	FEAT_MOPS
1	10	0111	CPYPTWN, CPYMTWN, CPYETWN — Epilogue	FEAT_MOPS
1	10	1000	CPYPRN, CPYMRN, CPYERN — Epilogue	FEAT_MOPS
1	10	1001	CPYPWTRN, CPYMWTRN, CPYEWTRN — Epilogue	FEAT_MOPS
1	10	1010	CPYPRTRN, CPYMRTRN, CPYERTRN — Epilogue	FEAT_MOPS
1	10	1011	CPYPTRN, CPYMTRN, CPYETRN — Epilogue	FEAT_MOPS
1	10	1100	CPYPN, CPYMN, CPYEN — Epilogue	FEAT_MOPS
1	10	1101	CPYPWTN, CPYMWTN, CPYEWTN — Epilogue	FEAT_MOPS
1	10	1110	CPYPRTN, CPYMRTN, CPYERTN — Epilogue	FEAT_MOPS
1	10	1111	CPYPTN, CPYMTN, CPYETN — Epilogue	FEAT_MOPS
1	11	0000	SETGP, SETGM, SETGE — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0001	SETGPT, SETGMT, SETGET — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0010	SETGPN, SETGMN, SETGEN — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0011	SETGPTN, SETGMTN, SETGETN — Prologue	FEAT_MOPS && FEAT_MTE
1	11	0100	SETGP, SETGM, SETGE — Main	FEAT_MOPS && FEAT_MTE
1	11	0101	SETGPT, SETGMT, SETGET — Main	FEAT_MOPS && FEAT_MTE
1	11	0110	SETGPN, SETGMN, SETGEN — Main	FEAT_MOPS && FEAT_MTE
1	11	0111	SETGPTN, SETGMTN, SETGETN — Main	FEAT_MOPS && FEAT_MTE
1	11	1000	SETGP, SETGM, SETGE — Epilogue	FEAT_MOPS && FEAT_MTE
1	11	1001	SETGPT, SETGMT, SETGET — Epilogue	FEAT_MOPS && FEAT_MTE
1	11	1010	SETGPN, SETGMN, SETGEN — Epilogue	FEAT_MOPS && FEAT_MTE
1	11	1011	SETGPTN, SETGMTN, SETGETN — Epilogue	FEAT_MOPS && FEAT_MTE

Load/store no-allocate pair (offset)

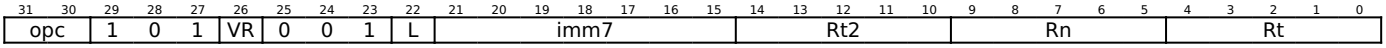
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
opc	VR	L		
00	0	0	STNP — 32-bit	-
00	0	1	LDNP — 32-bit	-
00	1	0	STNP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDNP (SIMD&FP) — 32-bit	FEAT_FP
01	0		UNALLOCATED	-
01	1	0	STNP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDNP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STNP — 64-bit	-
10	0	1	LDNP — 64-bit	-
10	1	0	STNP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDNP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTNP	FEAT_LSUI
11	0	1	LDTNP	FEAT_LSUI
11	1	0	STTNP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTNP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register pair (post-indexed)

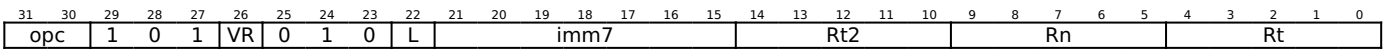
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
opc	VR	L		
00	0	0	STP — 32-bit	-
00	0	1	LDP — 32-bit	-
00	1	0	STP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDP (SIMD&FP) — 32-bit	FEAT_FP
01	0	0	STGP	FEAT_MTE
01	0	1	LDPSW	-
01	1	0	STP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STP — 64-bit	-
10	0	1	LDP — 64-bit	-
10	1	0	STP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTP	FEAT_LSUI
11	0	1	LDTP	FEAT_LSUI
11	1	0	STTP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register pair (offset)

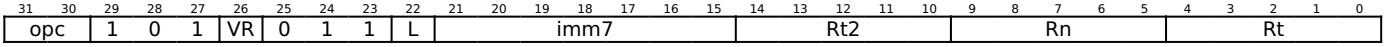
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
opc	VR	L		
00	0	0	STP — 32-bit	-
00	0	1	LDP — 32-bit	-
00	1	0	STP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDP (SIMD&FP) — 32-bit	FEAT_FP
01	0	0	STGP	FEAT_MTE
01	0	1	LDPSW	-
01	1	0	STP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STP — 64-bit	-
10	0	1	LDP — 64-bit	-
10	1	0	STP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTP	FEAT_LSUI
11	0	1	LDTP	FEAT_LSUI
11	1	0	STTP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register pair (pre-indexed)

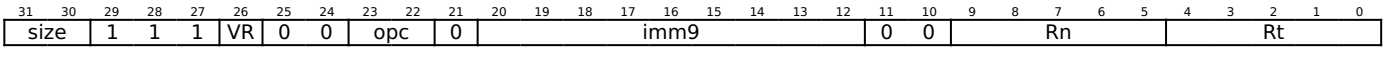
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
opc	VR	L		
00	0	0	STP — 32-bit	-
00	0	1	LDP — 32-bit	-
00	1	0	STP (SIMD&FP) — 32-bit	FEAT_FP
00	1	1	LDP (SIMD&FP) — 32-bit	FEAT_FP
01	0	0	STGP	FEAT_MTE
01	0	1	LDPSW	-
01	1	0	STP (SIMD&FP) — 64-bit	FEAT_FP
01	1	1	LDP (SIMD&FP) — 64-bit	FEAT_FP
10	0	0	STP — 64-bit	-
10	0	1	LDP — 64-bit	-
10	1	0	STP (SIMD&FP) — 128-bit	FEAT_FP
10	1	1	LDP (SIMD&FP) — 128-bit	FEAT_FP
11	0	0	STTP	FEAT_LSUI
11	0	1	LDTP	FEAT_LSUI
11	1	0	STTP (SIMD&FP)	FEAT_FP && FEAT_LSUI
11	1	1	LDTP (SIMD&FP)	FEAT_FP && FEAT_LSUI

Load/store register (unscaled immediate)

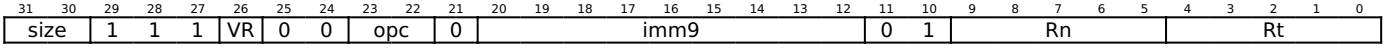
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	VR	opc		
00	0	00	STURB	-
00	0	01	LDURB	-
00	0	10	LDURSB — 64-bit	-
00	0	11	LDURSB — 32-bit	-
00	1	00	STUR (SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDUR (SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STUR (SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDUR (SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STURH	-
01	0	01	LDURH	-
01	0	10	LDURSH — 64-bit	-
01	0	11	LDURSH — 32-bit	-
01	1	00	STUR (SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDUR (SIMD&FP) — 16-bit	FEAT_FP
1x	0	11	UNALLOCATED	-
10	0	00	STUR — 32-bit	-
10	0	01	LDUR — 32-bit	-
10	0	10	LDURSW	-
10	1	00	STUR (SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDUR (SIMD&FP) — 32-bit	FEAT_FP
11	0	00	STUR — 64-bit	-
11	0	01	LDUR — 64-bit	-
11	0	10	PRFUM	-
11	1	00	STUR (SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDUR (SIMD&FP) — 64-bit	FEAT_FP
!= 00	1	1x	UNALLOCATED	-

Load/store register (immediate post-indexed)

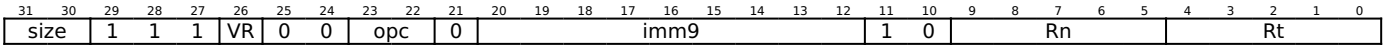
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	VR	opc		
00	0	00	STRB (immediate)	-
00	0	01	LDRB (immediate)	-
00	0	10	LDRSB (immediate) — 64-bit	-
00	0	11	LDRSB (immediate) — 32-bit	-
00	1	00	STR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STR (immediate, SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDR (immediate, SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STRH (immediate)	-
01	0	01	LDRH (immediate)	-
01	0	10	LDRSH (immediate) — 64-bit	-
01	0	11	LDRSH (immediate) — 32-bit	-
01	1	00	STR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	1x	UNALLOCATED	-
10	0	00	STR (immediate) — 32-bit	-
10	0	01	LDR (immediate) — 32-bit	-
10	0	10	LDRSW (immediate)	-
10	0	11	UNALLOCATED	-
10	1	00	STR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	1x	UNALLOCATED	-
11		1x	UNALLOCATED	-
11	0	00	STR (immediate) — 64-bit	-
11	0	01	LDR (immediate) — 64-bit	-
11	1	00	STR (immediate, SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDR (immediate, SIMD&FP) — 64-bit	FEAT_FP

Load/store register (unprivileged)

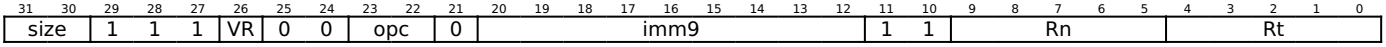
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details
size	VR	opc	
	1		UNALLOCATED
00	0	00	STTRB
00	0	01	LDTRB
00	0	10	LDTRSB — 64-bit
00	0	11	LDTRSB — 32-bit
01	0	00	STTRH
01	0	01	LDTRH
01	0	10	LDTRSH — 64-bit
01	0	11	LDTRSH — 32-bit
10	0	00	STTR — 32-bit
10	0	01	LDTR — 32-bit
10	0	10	LDTRSW
10	0	11	UNALLOCATED
11	0	00	STTR — 64-bit
11	0	01	LDTR — 64-bit
11	0	1x	UNALLOCATED

Load/store register (immediate pre-indexed)

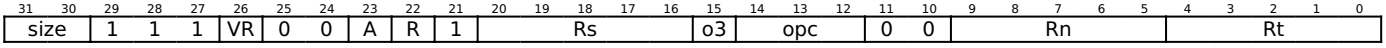
The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	VR	opc		
00	0	00	STRB (immediate)	-
00	0	01	LDRB (immediate)	-
00	0	10	LDRSB (immediate) — 64-bit	-
00	0	11	LDRSB (immediate) — 32-bit	-
00	1	00	STR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STR (immediate, SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDR (immediate, SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STRH (immediate)	-
01	0	01	LDRH (immediate)	-
01	0	10	LDRSH (immediate) — 64-bit	-
01	0	11	LDRSH (immediate) — 32-bit	-
01	1	00	STR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	1x	UNALLOCATED	-
10	0	00	STR (immediate) — 32-bit	-
10	0	01	LDR (immediate) — 32-bit	-
10	0	10	LDRSW (immediate)	-
10	0	11	UNALLOCATED	-
10	1	00	STR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	1x	UNALLOCATED	-
11		1x	UNALLOCATED	-
11	0	00	STR (immediate) — 64-bit	-
11	0	01	LDR (immediate) — 64-bit	-
11	1	00	STR (immediate, SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDR (immediate, SIMD&FP) — 64-bit	FEAT_FP

Atomic memory operations

The encodings in this section are decoded from Loads and Stores.



Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
	0		1		1	10x		UNALLOCATED	-
	0	0	0		1	100		UNALLOCATED	-
	0	0	0		1	11x		UNALLOCATED	-
	0	0	1		1	11x		UNALLOCATED	-
	1				0	001		UNALLOCATED	-
	1				0	01x		UNALLOCATED	-
	1	0			1		!= 11111	UNALLOCATED	-
	1	0			1	001	11111	UNALLOCATED	-
	1	0			1	01x	11111	UNALLOCATED	-
	1	1			1			UNALLOCATED	-
0x	0	1	0		1	11x		UNALLOCATED	-
0x	0	1	1		1	11x		UNALLOCATED	-
00	0	0	0		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — No memory ordering	FEAT_LSE
00	0	0	0		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — No memory ordering	FEAT_LSE
00	0	0	0		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — No memory ordering	FEAT_LSE
00	0	0	0		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — No memory ordering	FEAT_LSE
00	0	0	0		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — No memory ordering	FEAT_LSE
00	0	0	0		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — No memory ordering	FEAT_LSE
00	0	0	0		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — No memory ordering	FEAT_LSE
00	0	0	0		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — No memory ordering	FEAT_LSE
00	0	0	0		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPB	FEAT_LSE
00	0	0	0		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLR	FEAT_THE
00	0	0	0		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWP	FEAT_THE
00	0	0	0		1	011		RCWSET, RCWSETA, RCWSETAL, RCWSETL — RCWSET	FEAT_THE
00	0	0	1		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — Release	FEAT_LSE
00	0	0	1		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — Release	FEAT_LSE
00	0	0	1		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — Release	FEAT_LSE
00	0	0	1		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — Release	FEAT_LSE
00	0	0	1		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — Release	FEAT_LSE
00	0	0	1		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — Release	FEAT_LSE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
00	0	0	1		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — Release	FEAT_LSE
00	0	0	1		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — Release	FEAT_LSE
00	0	0	1		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPLB	FEAT_LSE
00	0	0	1		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLRL	FEAT_THE
00	0	0	1		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWPL	FEAT_THE
00	0	0	1		1	011		RCWSET, RCWSETA, RCWSETAL, RCWSETL — RCWSETL	FEAT_THE
00	0	1	0		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — Acquire	FEAT_LSE
00	0	1	0		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — Acquire	FEAT_LSE
00	0	1	0		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — Acquire	FEAT_LSE
00	0	1	0		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — Acquire	FEAT_LSE
00	0	1	0		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — Acquire	FEAT_LSE
00	0	1	0		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — Acquire	FEAT_LSE
00	0	1	0		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — Acquire	FEAT_LSE
00	0	1	0		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — Acquire	FEAT_LSE
00	0	1	0		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPAB	FEAT_LSE
00	0	1	0		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLRA	FEAT_THE
00	0	1	0		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWPA	FEAT_THE
00	0	1	0		1	011		RCWSET, RCWSETA, RCWSETAL, RCWSETL — RCWSETA	FEAT_THE
00	0	1	0		1	100		LDAPRB	FEAT_LRCPC
00	0	1	1		0	000		LDADDB, LDADDAB, LDADDALB, LDADDLB — Acquire-release	FEAT_LSE
00	0	1	1		0	001		LDCLRB, LDCLRAB, LDCLRALB, LDCLRLB — Acquire-release	FEAT_LSE
00	0	1	1		0	010		LDEORB, LDEORAB, LDEORALB, LDEORLB — Acquire-release	FEAT_LSE
00	0	1	1		0	011		LDSETB, LDSETAB, LDSETALB, LDSETLB — Acquire-release	FEAT_LSE
00	0	1	1		0	100		LDSMAXB, LDSMAXAB, LDSMAXALB, LDSMAXLB — Acquire-release	FEAT_LSE
00	0	1	1		0	101		LDSMINB, LDSMINAB, LDSMINALB, LDSMINLB — Acquire-release	FEAT_LSE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
00	0	1	1		0	110		LDUMAXB, LDUMAXAB, LDUMAXALB, LDUMAXLB — Acquire-release	FEAT_LSE
00	0	1	1		0	111		LDUMINB, LDUMINAB, LDUMINALB, LDUMINLB — Acquire-release	FEAT_LSE
00	0	1	1		1	000		SWPB, SWPAB, SWPALB, SWPLB — SWPALB	FEAT_LSE
00	0	1	1		1	001		RCWCLR, RCWCLRA, RCWCLRAL, RCWCLRL — RCWCLRAL	FEAT_THE
00	0	1	1		1	010		RCWSWP, RCWSWPA, RCWSWPAL, RCWSWPL — RCWSWPAL	FEAT_THE
00	0	1	1		1	011		RCWSET, RCWSETA, RCWSETAL, RCWSETL — RCWSETAL	FEAT_THE
00	1	0	0		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — No memory ordering	FEAT_LSFE
00	1	0	0		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — No memory ordering	FEAT_LSFE
00	1	0	0		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — No memory ordering	FEAT_LSFE
00	1	0	0		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — No memory ordering	FEAT_LSFE
00	1	0	0		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — No memory ordering	FEAT_LSFE
00	1	0	0		1	000	11111	STBFADD, STBFADDL — No memory ordering	FEAT_LSFE
00	1	0	0		1	100	11111	STBFMAX, STBFMAXL — No memory ordering	FEAT_LSFE
00	1	0	0		1	101	11111	STBFMIN, STBFMINL — No memory ordering	FEAT_LSFE
00	1	0	0		1	110	11111	STBFMAXNM, STBFMAXNML — No memory ordering	FEAT_LSFE
00	1	0	0		1	111	11111	STBFMINNM, STBFMINNML — No memory ordering	FEAT_LSFE
00	1	0	1		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — Release	FEAT_LSFE
00	1	0	1		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — Release	FEAT_LSFE
00	1	0	1		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — Release	FEAT_LSFE
00	1	0	1		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — Release	FEAT_LSFE
00	1	0	1		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — Release	FEAT_LSFE
00	1	0	1		1	000	11111	STBFADD, STBFADDL — Release	FEAT_LSFE
00	1	0	1		1	100	11111	STBFMAX, STBFMAXL — Release	FEAT_LSFE
00	1	0	1		1	101	11111	STBFMIN, STBFMINL — Release	FEAT_LSFE
00	1	0	1		1	110	11111	STBFMAXNM, STBFMAXNML — Release	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
00	1	0	1		1	111	11111	STBFMINNM, STBFMINNML — Release	FEAT_LSFE
00	1	1	0		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — Acquire	FEAT_LSFE
00	1	1	0		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — Acquire	FEAT_LSFE
00	1	1	0		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — Acquire	FEAT_LSFE
00	1	1	0		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — Acquire	FEAT_LSFE
00	1	1	0		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — Acquire	FEAT_LSFE
00	1	1	1		0	000		LDBFADD, LDBFADDA, LDBFADDAL, LDBFADDL — Acquire-release	FEAT_LSFE
00	1	1	1		0	100		LDBFMAX, LDBFMAXA, LDBFMAXAL, LDBFMAXL — Acquire-release	FEAT_LSFE
00	1	1	1		0	101		LDBFMIN, LDBFMINA, LDBFMINAL, LDBFMINL — Acquire-release	FEAT_LSFE
00	1	1	1		0	110		LDBFMAXNM, LDBFMAXNMA, LDBFMAXNMAL, LDBFMAXNML — Acquire-release	FEAT_LSFE
00	1	1	1		0	111		LDBFMINNM, LDBFMINNMA, LDBFMINNMAL, LDBFMINNML — Acquire-release	FEAT_LSFE
01	0	0	0		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — No memory ordering	FEAT_LSE
01	0	0	0		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — No memory ordering	FEAT_LSE
01	0	0	0		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — No memory ordering	FEAT_LSE
01	0	0	0		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — No memory ordering	FEAT_LSE
01	0	0	0		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — No memory ordering	FEAT_LSE
01	0	0	0		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — No memory ordering	FEAT_LSE
01	0	0	0		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — No memory ordering	FEAT_LSE
01	0	0	0		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — No memory ordering	FEAT_LSE
01	0	0	0		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPH	FEAT_LSE
01	0	0	0		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLR	FEAT_THE
01	0	0	0		1	010		RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL — RCWSSWP	FEAT_THE
01	0	0	0		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSET	FEAT_THE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
01	0	0	1		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — Release	FEAT_LSE
01	0	0	1		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — Release	FEAT_LSE
01	0	0	1		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — Release	FEAT_LSE
01	0	0	1		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — Release	FEAT_LSE
01	0	0	1		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — Release	FEAT_LSE
01	0	0	1		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — Release	FEAT_LSE
01	0	0	1		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — Release	FEAT_LSE
01	0	0	1		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — Release	FEAT_LSE
01	0	0	1		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPLH	FEAT_LSE
01	0	0	1		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLRL	FEAT_THE
01	0	0	1		1	010		RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL — RCWSSWPL	FEAT_THE
01	0	0	1		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSETL	FEAT_THE
01	0	1	0		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — Acquire	FEAT_LSE
01	0	1	0		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — Acquire	FEAT_LSE
01	0	1	0		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — Acquire	FEAT_LSE
01	0	1	0		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — Acquire	FEAT_LSE
01	0	1	0		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — Acquire	FEAT_LSE
01	0	1	0		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — Acquire	FEAT_LSE
01	0	1	0		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — Acquire	FEAT_LSE
01	0	1	0		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — Acquire	FEAT_LSE
01	0	1	0		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPAH	FEAT_LSE
01	0	1	0		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLRA	FEAT_THE
01	0	1	0		1	010		RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL — RCWSSWPA	FEAT_THE
01	0	1	0		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSETA	FEAT_THE
01	0	1	0		1	100		LDAPRH	FEAT_LRCPC

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
01	0	1	1		0	000		LDADDH, LDADDAH, LDADDALH, LDADDLH — Acquire-release	FEAT_LSE
01	0	1	1		0	001		LDCLRH, LDCLRAH, LDCLRALH, LDCLRLH — Acquire-release	FEAT_LSE
01	0	1	1		0	010		LDEORH, LDEORAH, LDEORALH, LDEORLH — Acquire-release	FEAT_LSE
01	0	1	1		0	011		LDSETH, LDSETAH, LDSETALH, LDSETLH — Acquire-release	FEAT_LSE
01	0	1	1		0	100		LDSMAXH, LDSMAXAH, LDSMAXALH, LDSMAXLH — Acquire-release	FEAT_LSE
01	0	1	1		0	101		LDSMINH, LDSMINAH, LDSMINALH, LDSMINLH — Acquire-release	FEAT_LSE
01	0	1	1		0	110		LDUMAXH, LDUMAXAH, LDUMAXALH, LDUMAXLH — Acquire-release	FEAT_LSE
01	0	1	1		0	111		LDUMINH, LDUMINAH, LDUMINALH, LDUMINLH — Acquire-release	FEAT_LSE
01	0	1	1		1	000		SWPH, SWPAH, SWPALH, SWPLH — SWPALH	FEAT_LSE
01	0	1	1		1	001		RCWSCLR, RCWSCLRA, RCWSCLRAL, RCWSCLRL — RCWSCLRAL	FEAT_THE
01	0	1	1		1	010		RCWSSWP, RCWSSWPA, RCWSSWPAL, RCWSSWPL — RCWSSWPAL	FEAT_THE
01	0	1	1		1	011		RCWSSET, RCWSSETA, RCWSSETAL, RCWSSETL — RCWSSETAL	FEAT_THE
01	1	0	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	000	11111	STFADD, STFADDL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	100	11111	STFMAX, STFMAXL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	101	11111	STFMIN, STFMINL — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	110	11111	STFMAXNM, STFMAXNML — Half-precision no memory ordering	FEAT_LSFE
01	1	0	0		1	111	11111	STFMINNM, STFMINNML — Half-precision no memory ordering	FEAT_LSFE
01	1	0	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision release	FEAT_LSFE
01	1	0	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision release	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
01	1	0	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision release	FEAT_LSFE
01	1	0	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision release	FEAT_LSFE
01	1	0	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision release	FEAT_LSFE
01	1	0	1		1	000	11111	STFADD, STFADDL — Half-precision release	FEAT_LSFE
01	1	0	1		1	100	11111	STFMAX, STFMAXL — Half-precision release	FEAT_LSFE
01	1	0	1		1	101	11111	STFMIN, STFMINL — Half-precision release	FEAT_LSFE
01	1	0	1		1	110	11111	STFMAXNM, STFMAXNML — Half-precision release	FEAT_LSFE
01	1	0	1		1	111	11111	STFMINNM, STFMINNML — Half-precision release	FEAT_LSFE
01	1	1	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision acquire	FEAT_LSFE
01	1	1	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision acquire	FEAT_LSFE
01	1	1	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Half-precision acquire-release	FEAT_LSFE
01	1	1	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Half-precision acquire-release	FEAT_LSFE
1x	0	1			1	x1x		UNALLOCATED	-
10	0				1	001		UNALLOCATED	-
10	0	0			1	010		UNALLOCATED	-
10	0	0			1	011		UNALLOCATED	-
10	0	0	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit no memory ordering	FEAT_LSE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
10	0	0	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit no memory ordering	FEAT_LSE
10	0	0	0		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWP	FEAT_LSE
10	0	0	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit release	FEAT_LSE
10	0	0	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit release	FEAT_LSE
10	0	0	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit release	FEAT_LSE
10	0	0	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit release	FEAT_LSE
10	0	0	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit release	FEAT_LSE
10	0	0	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit release	FEAT_LSE
10	0	0	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit release	FEAT_LSE
10	0	0	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit release	FEAT_LSE
10	0	0	1		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWPL	FEAT_LSE
10	0	1	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit acquire	FEAT_LSE
10	0	1	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit acquire	FEAT_LSE
10	0	1	0		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWPA	FEAT_LSE
10	0	1	0		1	100		LDAPR — 32-bit	FEAT_LRCPC
10	0	1	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 32-bit acquire-release	FEAT_LSE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
10	0	1	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 32-bit acquire-release	FEAT_LSE
10	0	1	1		1	000		SWP, SWPA, SWPAL, SWPL — 32-bit SWPAL	FEAT_LSE
10	1	0	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	000	11111	STFADD, STFADDL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	100	11111	STFMAX, STFMAXL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	101	11111	STFMIN, STFMINL — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	110	11111	STFMAXNM, STFMAXNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	0		1	111	11111	STFMINNM, STFMINNML — Single-precision no memory ordering	FEAT_LSFE
10	1	0	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision release	FEAT_LSFE
10	1	0	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision release	FEAT_LSFE
10	1	0	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision release	FEAT_LSFE
10	1	0	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision release	FEAT_LSFE
10	1	0	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision release	FEAT_LSFE
10	1	0	1		1	000	11111	STFADD, STFADDL — Single-precision release	FEAT_LSFE
10	1	0	1		1	100	11111	STFMAX, STFMAXL — Single-precision release	FEAT_LSFE
10	1	0	1		1	101	11111	STFMIN, STFMINL — Single-precision release	FEAT_LSFE
10	1	0	1		1	110	11111	STFMAXNM, STFMAXNML — Single-precision release	FEAT_LSFE
10	1	0	1		1	111	11111	STFMINNM, STFMINNML — Single-precision release	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
10	1	1	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision acquire	FEAT_LSFE
10	1	1	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision acquire	FEAT_LSFE
10	1	1	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Single-precision acquire-release	FEAT_LSFE
10	1	1	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Single-precision acquire-release	FEAT_LSFE
10	1	1	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Single-precision acquire-release	FEAT_LSFE
10	1	1	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Single-precision acquire-release	FEAT_LSFE
10	1	1	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Single-precision acquire-release	FEAT_LSFE
11	0		0	!= 11111	1	101		UNALLOCATED	-
11	0	0		!= 11111	1	001		UNALLOCATED	-
11	0	0	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit no memory ordering	FEAT_LSE
11	0	0	0		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWP	FEAT_LSE
11	0	0	0		1	010		ST64BV0	FEAT_LS64_ACCDATA
11	0	0	0		1	011		ST64BV	FEAT_LS64_V
11	0	0	0	11111	1	001		ST64B	FEAT_LS64

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
11	0	0	0	11111	1	101		LD64B	FEAT_LS64
11	0	0	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit release	FEAT_LSE
11	0	0	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit release	FEAT_LSE
11	0	0	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit release	FEAT_LSE
11	0	0	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit release	FEAT_LSE
11	0	0	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit release	FEAT_LSE
11	0	0	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit release	FEAT_LSE
11	0	0	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit release	FEAT_LSE
11	0	0	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit release	FEAT_LSE
11	0	0	1		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWPL	FEAT_LSE
11	0	0	1		1	010		STL64BV0	FEAT_LS64_ACCDATA && FEAT_LSC64B
11	0	0	1		1	011		STL64BV	FEAT_LS64_V && FEAT_LSC64B
11	0	0	1	11111	1	001		STL64B	FEAT_LS64 && FEAT_LSC64B
11	0	1			1	001		UNALLOCATED	-
11	0	1	0		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit acquire	FEAT_LSE
11	0	1	0		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit acquire	FEAT_LSE
11	0	1	0		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWPA	FEAT_LSE
11	0	1	0		1	100		LDAPR — 64-bit	FEAT_LRCPC
11	0	1	0	11111	1	101		LDA64B	FEAT_LS64 && FEAT_LSC64B
11	0	1	1		0	000		LDADD, LDADDA, LDADDAL, LDADDL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	001		LDCLR, LDCLRA, LDCLRAL, LDCLRL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	010		LDEOR, LDEORA, LDEORAL, LDEORL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	011		LDSET, LDSETA, LDSETAL, LDSETL — 64-bit acquire-release	FEAT_LSE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
11	0	1	1		0	100		LDSMAX, LDSMAXA, LDSMAXAL, LDSMAXL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	101		LDSMIN, LDSMINA, LDSMINAL, LDSMINL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	110		LDUMAX, LDUMAXA, LDUMAXAL, LDUMAXL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		0	111		LDUMIN, LDUMINA, LDUMINAL, LDUMINL — 64-bit acquire-release	FEAT_LSE
11	0	1	1		1	000		SWP, SWPA, SWPAL, SWPL — 64-bit SWPAL	FEAT_LSE
11	1	0	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	000	11111	STFADD, STFADDL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	100	11111	STFMAX, STFMAXL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	101	11111	STFMIN, STFMINL — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	110	11111	STFMAXNM, STFMAXNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	0		1	111	11111	STFMINNM, STFMINNML — Double-precision no memory ordering	FEAT_LSFE
11	1	0	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision release	FEAT_LSFE
11	1	0	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision release	FEAT_LSFE
11	1	0	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision release	FEAT_LSFE
11	1	0	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision release	FEAT_LSFE
11	1	0	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision release	FEAT_LSFE
11	1	0	1		1	000	11111	STFADD, STFADDL — Double-precision release	FEAT_LSFE
11	1	0	1		1	100	11111	STFMAX, STFMAXL — Double-precision release	FEAT_LSFE
11	1	0	1		1	101	11111	STFMIN, STFMINL — Double-precision release	FEAT_LSFE
11	1	0	1		1	110	11111	STFMAXNM, STFMAXNML — Double-precision release	FEAT_LSFE
11	1	0	1		1	111	11111	STFMINNM, STFMINNML — Double-precision release	FEAT_LSFE

Decode fields								Instruction Details	Condition
size	VR	A	R	Rs	o3	opc	Rt		
11	1	1	0		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision acquire	FEAT_LSFE
11	1	1	0		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision acquire	FEAT_LSFE
11	1	1	1		0	000		LDFADD, LDFADDA, LDFADDAL, LDFADDL — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	100		LDFMAX, LDFMAXA, LDFMAXAL, LDFMAXL — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	101		LDFMIN, LDFMINA, LDFMINAL, LDFMINL — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	110		LDFMAXNM, LDFMAXNMA, LDFMAXNMAL, LDFMAXNML — Double-precision acquire-release	FEAT_LSFE
11	1	1	1		0	111		LDFMINNM, LDFMINNMA, LDFMINNMAL, LDFMINNML — Double-precision acquire-release	FEAT_LSFE
!= 11	0		0		1	101		UNALLOCATED	-

Load/store register (register offset)

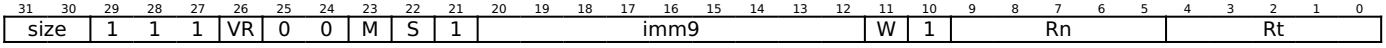
The encodings in this section are decoded from Loads and Stores.

31 30	29 28 27	26	25 24	23 22	21	20 19 18 17 16	15 14 13	12	11 10	9 8 7 6 5	4 3 2 1 0
size	1 1 1	VR	0 0	opc	1	Rm	option	S	1 0	Rn	Rt

Decode fields					Instruction Details	Condition
size	VR	opc	option	Rt		
00	0	00	011		STRB (register) — Shifted register	-
00	0	00	!= 011		STRB (register) — Extended register	-
00	0	01	011		LDRB (register) — Shifted register	-
00	0	01	!= 011		LDRB (register) — Extended register	-
00	0	10	011		LDRSB (register) — 64-bit with shifted register offset	-
00	0	10	!= 011		LDRSB (register) — 64-bit with extended register offset	-
00	0	11	011		LDRSB (register) — 32-bit with shifted register offset	-
00	0	11	!= 011		LDRSB (register) — 32-bit with extended register offset	-
00	1	00	011		STR (register, SIMD&FP) — 8-bit with shifted register offset	FEAT_FP
00	1	00	!= 011		STR (register, SIMD&FP) — 8-bit with extended register offset	FEAT_FP
00	1	01	011		LDR (register, SIMD&FP) — 8-bit with shifted register offset	FEAT_FP
00	1	01	!= 011		LDR (register, SIMD&FP) — 8-bit with extended register offset	FEAT_FP
00	1	10			STR (register, SIMD&FP) — 128-bit	FEAT_FP
00	1	11			LDR (register, SIMD&FP) — 128-bit	FEAT_FP
01	0	00			STRH (register)	-
01	0	01			LDRH (register)	-
01	0	10			LDRSH (register) — 64-bit	-
01	0	11			LDRSH (register) — 32-bit	-
01	1	00			STR (register, SIMD&FP) — 16-bit	FEAT_FP
01	1	01			LDR (register, SIMD&FP) — 16-bit	FEAT_FP
1x	0	11			UNALLOCATED	-
10	0	00			STR (register) — 32-bit	-
10	0	01			LDR (register) — 32-bit	-
10	0	10			LDRSW (register)	-
10	1	00			STR (register, SIMD&FP) — 32-bit	FEAT_FP
10	1	01			LDR (register, SIMD&FP) — 32-bit	FEAT_FP
11	0	00			STR (register) — 64-bit	-
11	0	01			LDR (register) — 64-bit	-
11	0	10	x0x		UNALLOCATED	-
11	0	10	x1x	11xxx	RPRFM	FEAT_RPRFM
11	0	10	x1x	!= 11xxx	PRFM (register)	-
11	1	00			STR (register, SIMD&FP) — 64-bit	FEAT_FP
11	1	01			LDR (register, SIMD&FP) — 64-bit	FEAT_FP
!= 00	1	1x			UNALLOCATED	-

Load/store register (pac)

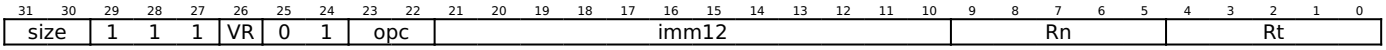
The encodings in this section are decoded from Loads and Stores.



Decode fields				Instruction Details	Condition
size	VR	M	W		
11	0	0	0	LDRAA, LDRAB — Key A, offset	FEAT_PAuth
11	0	0	1	LDRAA, LDRAB — Key A, pre-indexed	FEAT_PAuth
11	0	1	0	LDRAA, LDRAB — Key B, offset	FEAT_PAuth
11	0	1	1	LDRAA, LDRAB — Key B, pre-indexed	FEAT_PAuth
11	1			UNALLOCATED	-
!= 11				UNALLOCATED	-

Load/store register (unsigned immediate)

The encodings in this section are decoded from Loads and Stores.



Decode fields			Instruction Details	Condition
size	VR	opc		
00	0	00	STRB (immediate)	-
00	0	01	LDRB (immediate)	-
00	0	10	LDRSB (immediate) — 64-bit	-
00	0	11	LDRSB (immediate) — 32-bit	-
00	1	00	STR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	01	LDR (immediate, SIMD&FP) — 8-bit	FEAT_FP
00	1	10	STR (immediate, SIMD&FP) — 128-bit	FEAT_FP
00	1	11	LDR (immediate, SIMD&FP) — 128-bit	FEAT_FP
01	0	00	STRH (immediate)	-
01	0	01	LDRH (immediate)	-
01	0	10	LDRSH (immediate) — 64-bit	-
01	0	11	LDRSH (immediate) — 32-bit	-
01	1	00	STR (immediate, SIMD&FP) — 16-bit	FEAT_FP
01	1	01	LDR (immediate, SIMD&FP) — 16-bit	FEAT_FP
1x	0	11	UNALLOCATED	-
10	0	00	STR (immediate) — 32-bit	-
10	0	01	LDR (immediate) — 32-bit	-
10	0	10	LDRSW (immediate)	-
10	1	00	STR (immediate, SIMD&FP) — 32-bit	FEAT_FP
10	1	01	LDR (immediate, SIMD&FP) — 32-bit	FEAT_FP
11	0	00	STR (immediate) — 64-bit	-
11	0	01	LDR (immediate) — 64-bit	-
11	0	10	PRFM (immediate)	-
11	1	00	STR (immediate, SIMD&FP) — 64-bit	FEAT_FP
11	1	01	LDR (immediate, SIMD&FP) — 64-bit	FEAT_FP
!= 00	1	1x	UNALLOCATED	-

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved.

Shared Pseudocode Functions

This page displays common pseudocode functions shared by many pages

Pseudocode

Library pseudocode for aarch32/at/AArch32_AT

```
// AArch32_AT()
// ============
// Perform address translation as per AT instructions.

func AArch32_AT(vaddress : bits(32), stage_in : TranslationStage, el : bits(2), ataccess : ATAccess)
begin
    var stage : TranslationStage = stage_in;
    var ss : SecurityState;
    var regime : Regime;
    var eae : boolean;

    // ATS1Hx instructions
    if el == EL2 then
        regime = Regime_EL2;
        eae = TRUE;
        ss = SS_NonSecure;

    // ATS1Cxx instructions
    elsif stage == TranslationStage_1 || (stage == TranslationStage_12 && !HaveEL(EL2)) then
        stage = TranslationStage_1;
        ss = SecurityStateAtEL(PSTATE.EL);
        regime = if ss == SS_Secure && ELUsingAArch32(EL3) then Regime_EL30 else Regime_EL10;
        eae = TTBCR().EAE == '1';

    // ATS12NSOxx instructions
    else
        regime = Regime_EL10;
        eae = (if HaveEL(EL3) && ELUsingAArch32(EL3) then TTBCR_NS().EAE == '1'
                                                     else TTBCR().EAE == '1');
        ss = SS_NonSecure;
    end;

    var addrdesc : AddressDescriptor;
    var sdftype : SDFType;
    let aligned : boolean = TRUE;
    var supersection : bit = '0';

    let accdesc : AccessDescriptor = CreateAccDescAT(ss, el, ataccess);

    // Prepare fault fields in case a fault is detected
    var fault : FaultRecord = NoFault(accdesc, ZeroExtend{64}(vaddress));

    if eae then
        (fault, addrdesc) = AArch32_S1TranslateLD(fault, regime, vaddress, aligned, accdesc);
    else
        (fault, addrdesc, sdftype) = AArch32_S1TranslateSD(fault, regime, vaddress, aligned,
                                                           accdesc);
        supersection = if sdftype == SDFType_Supersection then '1' else '0';
    end;

    // ATS12NSOxx instructions
    if stage == TranslationStage_12 && fault.statuscode == Fault_None then
        (fault, addrdesc) = AArch32_S2Translate(fault, addrdesc, aligned, accdesc);
    end;

    if fault.statuscode != Fault_None then
        // Take exception on External abort or when a fault occurs on translation table walk
        if IsExternalAbort(fault) || (PSTATE.EL == EL1 && EL2Enabled() && fault.s2fs1walk) then
            PAR() = ARBITRARY : bits(64);
            AArch32_Abort(fault);
        end;
    end;

    addrdesc.fault = fault;

    if (eae || (stage == TranslationStage_12 && (HCR().VM == '1' || HCR().DC == '1')) ||
          (stage == TranslationStage_1 && el != EL2 && PSTATE.EL == EL2)) then
        AArch32_EncodePARLD(addrdesc, ss);
    else
        AArch32_EncodePARSD(addrdesc, supersection, ss);
    end;
    return;
end;
```

Library pseudocode for aarch32/at/AArch32_EncodePARLD

```
// AArch32_EncodePARLD()
// =====================
// Returns 64-bit format PAR on address translation instruction.

func AArch32_EncodePARLD(addrdesc : AddressDescriptor, ss : SecurityState)
begin
    PAR()[63:0] = Zeros{64};
    if !IsFault(addrdesc) then
        var ns : bit;
        if ss == SS_NonSecure then
            ns = ARBITRARY : bit;
        elsif addrdesc.paddress.paspace == PAS_Secure then
            ns = '0';
        else
            ns = '1';
        end;
        PAR().F      = '0';
        PAR().F0_64.SH     = ReportedPARShareability(PAREncodeShareability(addrdesc.mematttrs));
        PAR().NS     = ns;
        PAR()[10]    = ImpDefBit("Non-Faulting PAR");               // IMPDEF
        PAR().LPAE   = '1';
        PAR().F0_64.PA     = addrdesc.paddress.address[39:12];
        PAR().ATTR   = ReportedPARAttrs(EncodePARAttrs(addrdesc.mematttrs));
    else
        PAR().F      = '1';
        PAR().FST    = AArch32_PARFaultStatusLD(addrdesc.fault);
        PAR().S2WLK  = if addrdesc.fault.s2fs1walk then '1' else '0';
        PAR().FSTAGE = if addrdesc.fault.secondstage then '1' else '0';
        PAR().LPAE   = '1';
        PAR()[63:48] = ImpDefBits{16}("Faulting PAR");             // IMPDEF
    end;
    return;
end;
```

Library pseudocode for aarch32/at/AArch32_EncodePARSD

```
// AArch32_EncodePARSD()
// =====================
// Returns 32-bit format PAR on address translation instruction.

func AArch32_EncodePARSD(addrdesc_in : AddressDescriptor, supersection : bit, ss : SecurityState)
begin
    PAR()[31:0] = Zeros{32};
    var addrdesc : AddressDescriptor = addrdesc_in;
    if !IsFault(addrdesc) then
        if (addrdesc.mematttrs.memtype == MemType_Device ||
                (addrdesc.mematttrs.inner.attrs == MemAttr_NC &&
                 addrdesc.mematttrs.outer.attrs == MemAttr_NC)) then
            addrdesc.mematttrs.shareability = Shareability_OSH;
        end;
        var ns : bit;
        if ss == SS_NonSecure then
            ns = ARBITRARY : bit;
        elsif addrdesc.paddress.paspace == PAS_Secure then
            ns = '0';
        else
            ns = '1';
        end;
        let sh : bits(2) = ReportedPARShareability(PAREncodeShareability(addrdesc.mematttrs));
        PAR().F      = '0';
        PAR().SS     = supersection;
        PAR().Outer  = AArch32_ReportedOuterAttrs(AArch32_PAROuterAttrs(addrdesc.mematttrs));
        PAR().Inner  = AArch32_ReportedInnerAttrs(AArch32_PARInnerAttrs(addrdesc.mematttrs));
        PAR().F0_32.SH     = sh[1];
        PAR()[8]     = ImpDefBit("Non-Faulting PAR");               // IMPDEF
        PAR().NS     = ns;
        PAR().NOS    = sh[0];
        PAR().LPAE   = '0';
        PAR().F0_32.PA     = addrdesc.paddress.address[31:12];
    else
        PAR().F      = '1';
        PAR().FST    = AArch32_PARFaultStatusSD(addrdesc.fault);
        PAR().LPAE   = '0';
        PAR()[31:16] = ImpDefBits{16}("Faulting PAR");             // IMPDEF
    end;
    return;
end;
```

Library pseudocode for aarch32/at/AArch32_PARFaultStatusLD

```
// AArch32_PARFaultStatusLD()
// ==========================
// Fault status field decoding of 64-bit PAR

func AArch32_PARFaultStatusLD(fault : FaultRecord) => bits(6)
begin
    var syndrome : bits(6);
    if fault.statuscode == Fault_Domain then
        // Report Domain fault
        assert fault.level IN {1,2};
        syndrome[1:0] = if fault.level == 1 then '01' else '10';
        syndrome[5:2] = '1111';
    else
        syndrome = EncodeLDFSC(fault.statuscode, fault.level);
    end;
    return syndrome;
end;
```

Library pseudocode for aarch32/at/AArch32_PARFaultStatusSD

```
// AArch32_PARFaultStatusSD()
// ==========================
// Fault status field decoding of 32-bit PAR.

func AArch32_PARFaultStatusSD(fault : FaultRecord) => bits(6)
begin
    var syndrome : bits(6);

    syndrome[5] = if IsExternalAbort(fault) then fault.extflag else '0';
    syndrome[4:0] = EncodeSDFSC(fault.statuscode, fault.level);
    return syndrome;
end;
```

Library pseudocode for aarch32/at/AArch32_PARInnerAttrs

```
// AArch32_PARInnerAttrs()
// =======================
// Convert orthogonal attributes and hints to 32-bit PAR Inner field.

func AArch32_PARInnerAttrs(mematTrs : MemoryAttributes) => bits(3)
begin
    var result : bits(3);

    if mematTrs.memtype == MemType_Device then
        if mematTrs.device == DeviceType_nGnRnE then
            result = '001'; // Non-cacheable
        elsif mematTrs.device == DeviceType_nGnRE then
            result = '011'; // Non-cacheable
        end;
    else
        let inner : MemAttrHints = mematTrs.inner;
        if inner.attrs == MemAttr_NC then
            result = '000'; // Non-cacheable
        elsif inner.attrs == MemAttr_WB && inner.hints[0] == '1' then
            result = '101'; // Write-Back, Write-Allocate
        elsif inner.attrs == MemAttr_WT then
            result = '110'; // Write-Through
        elsif inner.attrs == MemAttr_WB && inner.hints[0] == '0' then
            result = '111'; // Write-Back, no Write-Allocate
        end;
    end;
    return result;
end;
```

Library pseudocode for aarch32/at/AArch32_PAROuterAttrs

```
// AArch32_PAROuterAttrs()
// ======================
// Convert orthogonal attributes and hints to 32-bit PAR Outer field.

func AArch32_PAROuterAttrs(mematTrs : MemoryAttributes) => bits(2)
begin
    var result : bits(2);

    if mematTrs.memtype == MemType_Device then
        result = ARBITRARY : bits(2);
    else
        let outer : MemAttrHints = mematTrs.outer;
        if outer.attrs == MemAttr_NC then
            result = '00'; // Non-cacheable
        elsif outer.attrs == MemAttr_WB && outer.hints[0] == '1' then
            result = '01'; // Write-Back, Write-Allocate
        elsif outer.attrs == MemAttr_WT && outer.hints[0] == '0' then
            result = '10'; // Write-Through, no Write-Allocate
        elsif outer.attrs == MemAttr_WB && outer.hints[0] == '0' then
            result = '11'; // Write-Back, no Write-Allocate
        end;
    end;
    return result;
end;
```

Library pseudocode for aarch32/at/AArch32_ReportedInnerAttrs

```
// AArch32_ReportedInnerAttrs()
// ============================
// The value returned in this field can be the resulting attribute, as determined by any permitted
// implementation choices and any applicable configuration bits, instead of the value that appears
// in the translation table descriptor.

impdef func AArch32_ReportedInnerAttrs(attrs : bits(3)) => bits(3)
begin
    return attrs;
end;
```

Library pseudocode for aarch32/at/AArch32_ReportedOuterAttrs

```
// AArch32_ReportedOuterAttrs()
// ============================
// The value returned in this field can be the resulting attribute, as determined by any permitted
// implementation choices and any applicable configuration bits, instead of the value that appears
// in the translation table descriptor.

impdef func AArch32_ReportedOuterAttrs(attrs : bits(2)) => bits(2)
begin
    return attrs;
end;
```

Library pseudocode for aarch32/at/GetPAR_F

```
// GetPAR_F()
// ==========
// Query the PAR.F field.

func GetPAR_F() => bit
begin
    return PAR().F;
end;
```

Library pseudocode for aarch32/at/GetPAR_LPAE

```
// GetPAR_LPAE()
// =============
// Query the PAR.LPAE field.

func GetPAR_LPAE() => bit
begin
    return PAR().LPAE;
end;
```

Library pseudocode for aarch32/dc/AArch32_CanTrapDC

```
// AArch32_CanTrapDC()
// ===================
// Determines whether the execution of the DC instruction can be trapped.

func AArch32_CanTrapDC(cacheop : CacheOp, opscope : CacheOpScope) => boolean
begin
    return (!AArch32_TreatDCAsNOP(cacheop, opscope) || ImpDefBool(
            "When DC is treated as NOP, data cache maintenance operations are trapped"));
end;
```

Library pseudocode for aarch32/dc/AArch32_DC

```
// AArch32_DC()
// ============
// Perform Data Cache Operation.

func AArch32_DC(regval : bits(32), cacheop : CacheOp, opscope : CacheOpScope)
begin
    var cache : CacheRecord;

    cache.acctype   = AccessType_DC;
    cache.cacheop   = cacheop;
    cache.opscope   = opscope;
    cache.cachetype = CacheType_Data;
    cache.security  = SecurityStateAtEL(PSTATE.EL);

    if opscope == CacheOpScope_SetWay then
        cache.shareability = Shareability_NSH;
        cache.(setnum, waynum, level) = DecodeSW(ZeroExtend{64}(regval),
                                                 CacheType_Data);

        if (cacheop == CacheOp_Invalidate && PSTATE.EL == EL1 && EL2Enabled() &&
              ((!ELUsingAArch32(EL2) && (HCR_EL2().SWIO == '1' || HCR_EL2().[DC,VM] != '00')) ||
                  (ELUsingAArch32(EL2) && (HCR().SWIO == '1' || HCR().[DC,VM] != '00')))) then
            cache.cacheop = CacheOp_CleanInvalidate;
        end;
        CACHE_OP(cache);
        return;
    end;

    if EL2Enabled() then
        if PSTATE.EL IN {EL0, EL1} then
            cache.is_vmid_valid = TRUE;
            cache.vmid          = VMID();
        else
            cache.is_vmid_valid = FALSE;
        end;
    else
        cache.is_vmid_valid = FALSE;
    end;

    if PSTATE.EL == EL0 then
        cache.is_asid_valid = TRUE;
        cache.asid          = ASID();
    else
        cache.is_asid_valid = FALSE;
    end;

    var vaddress : bits(32) = regval;

    var size : integer = 0;        // by default no watchpoint address
    if cacheop == CacheOp_Invalidate then
        size     = DataCacheWatchpointSize();
        vaddress = AlignDownSize{32}(regval, size as AddressSize);
    end;

    cache.vaddress = ZeroExtend{64}(vaddress);

    let aligned : boolean               = TRUE;
    let accdesc : AccessDescriptor      = CreateAccDescDC(cache);
    var memaddrdesc : AddressDescriptor = AArch32_TranslateAddress(vaddress, accdesc,
                                                                   aligned, size);
    if IsFault(memaddrdesc) then
        memaddrdesc.fault.vaddress = ZeroExtend{64}(regval);
        AArch32_Abort(memaddrdesc.fault);
    end;

    cache.paddress = memaddrdesc.paddress;
    if opscope == CacheOpScope_PoC then
        cache.shareability = memaddrdesc.mematttrs.shareability;
    else
        cache.shareability = Shareability_NSH;
    end;

    if (cacheop == CacheOp_Invalidate && PSTATE.EL == EL1 && EL2Enabled() &&
          ((!ELUsingAArch32(EL2) && HCR_EL2().[DC,VM] != '00') ||
             (ELUsingAArch32(EL2) && HCR().[DC,VM] != '00'))) then
        cache.cacheop = CacheOp_CleanInvalidate;
    end;

    CACHE_OP(cache);
    return;
end;
```

Library pseudocode for aarch32/dc/AArch32_TreatDCAsNOP

```
// AArch32_TreatDCAsNOP()
// ======================
// Determines whether the execution of the DC instruction is treated as a NOP.

func AArch32_TreatDCAsNOP(cacheop : CacheOp, opscope : CacheOpScope) => boolean
begin
    // DC to PoU: IMPLEMENTATION DEFINED - treated as NOP if LoUU and LoUIS are 0
    if opscope == CacheOpScope_PoU && CLIDR().LoUU == '000' && CLIDR().LoUIS == '000' then
        return ImpDefBool("DC to PoU is treated as a NOP");
    end;

    // DC to PoC: IMPLEMENTATION DEFINED - treated as NOP if LoC is 0
    if opscope == CacheOpScope_PoC && CLIDR().LoC == '000' then
        return ImpDefBool("DC to PoC is treated as a NOP");
    end;

    return FALSE;
end;
```

Library pseudocode for aarch32/debug/VCRMatch/AArch32_VCRMatch

```
// AArch32_VCRMatch()
// ==================

func AArch32_VCRMatch(vaddress : bits(32)) => boolean
begin

    var match : boolean;
    if UsingAArch32() && ELUsingAArch32(EL1) && PSTATE.EL != EL2 then
        // Each bit position in this string corresponds to a bit in DBGVCR and an exception vector.
        var match_word : bits(32) = Zeros{};

        let ss : SecurityState = CurrentSecurityState();
        if vaddress[31:5] == ExcVectorBase()[31:5] then
            if HaveEL(EL3) && ss == SS_NonSecure then
                match_word[UInt(vaddress[4:2]) + 24] = '1';     // Non-secure vectors
            else
                match_word[UInt(vaddress[4:2]) + 0] = '1';      // Secure vectors (or no EL3)
            end;
        end;
        if (HaveEL(EL3) && ELUsingAArch32(EL3) && vaddress[31:5] == MVBAR()[31:5] &&
            ss == SS_Secure) then
            match_word[UInt(vaddress[4:2]) + 8] = '1';          // Monitor vectors
        end;
        // Mask out bits not corresponding to vectors.
        var mask : bits(32);
        if !HaveEL(EL3) then
            mask = '00000000'::'00000000'::'00000000'::'11011110'; // DBGVCR[31:8] are RES0
        elsif !ELUsingAArch32(EL3) then
            mask = '11011110'::'00000000'::'00000000'::'11011110'; // DBGVCR[15:8] are RES0
        else
            mask = '11011110'::'00000000'::'11011100'::'11011110';
        end;

        match_word = match_word AND DBGVCR() AND mask;
        match = !IsZero(match_word);

        // Check for UNPREDICTABLE case - match on Prefetch Abort and Data Abort vectors
        if !IsZero(match_word[28:27,12:11,4:3]) && DebugTarget() == PSTATE.EL then
            match = ConstrainUnpredictableBool(Unpredictable_VCMATCHDAPA);
        end;

        if !IsZero(vaddress[1:0]) && match then
            match = ConstrainUnpredictableBool(Unpredictable_VCMATCHHALF);
        end;
    else
        match = FALSE;
    end;

    return match;
end;
```

Library pseudocode for aarch32/debug/authentication/AArch32_SelfHostedSecurePrivilegedInvasiveDebugEnabled

```
// AArch32_SelfHostedSecurePrivilegedInvasiveDebugEnabled()
// ========================================================

func AArch32_SelfHostedSecurePrivilegedInvasiveDebugEnabled() => boolean
begin
    // The definition of this function is IMPLEMENTATION DEFINED.
    // In the recommended interface, AArch32_SelfHostedSecurePrivilegedInvasiveDebugEnabled returns
    // the state of the (DBGEN AND SPIDEN) signal.
    if !HaveEL(EL3) && NonSecureOnlyImplementation() then return FALSE; end;
    return DBGEN == HIGH && SPIDEN == HIGH;
end;
```

Library pseudocode for aarch32/debug/breakpoint/AArch32_BreakpointMatch

```
// AArch32_BreakpointMatch()
// =========================
// Breakpoint matching in an AArch32 translation regime.

func AArch32_BreakpointMatch(n : integer, vaddress : bits(32), accdesc : AccessDescriptor,
                             size : integer) => BreakpointInfo
begin
    assert ELUsingAArch32(S1TranslationRegime());
    assert n < NumBreakpointsImplemented();

    var brkptinfo : BreakpointInfo;
    let enabled : boolean    = DBGBCR(n).E == '1';
    let isbreakpnt : boolean = TRUE;
    let linked : boolean     = DBGBCR(n).BT == '0x01';
    let linked_to : boolean  = FALSE;
    let linked_n : integer{} = UInt(DBGBCR(n).LBN);

    let state_match : boolean = AArch32_StateMatch(DBGBCR(n).SSC, DBGBCR(n).HMC, DBGBCR(n).PMC,
                                                   linked, linked_n, isbreakpnt,  accdesc);
    var (value_match, value_mismatch) = AArch32_BreakpointValueMatch(n,vaddress, linked_to);

    if size == 4 then                    // Check second halfword
        // If the breakpoint address and BAS of an Address breakpoint match the address of the
        // second halfword of an instruction, but not the address of the first halfword, it is
        // CONSTRAINED UNPREDICTABLE whether this breakpoint generates a Breakpoint debug
        // event.
        let (match_i, mismatch_i) = AArch32_BreakpointValueMatch(n, vaddress + 2, linked_to);

        if !value_match && match_i then
            value_match = ConstrainUnpredictableBool(Unpredictable_BPMATCHHALF);
        end;

        if value_mismatch && !mismatch_i then
            value_mismatch = ConstrainUnpredictableBool(Unpredictable_BPMISMATCHHALF);
        end;
    end;

    if vaddress[1] == '1' && DBGBCR(n).BAS == '1111' then
        // The above notwithstanding, if DBGBCR(n).BAS == '1111', then it is CONSTRAINED
        // UNPREDICTABLE whether or not a Breakpoint debug event is generated for an instruction
        // at the address DBGBVR(n)+2.
        if value_match then
            value_match = ConstrainUnpredictableBool(Unpredictable_BPMATCHHALF);
        end;

        if !value_mismatch then
            value_mismatch = ConstrainUnpredictableBool(Unpredictable_BPMISMATCHHALF);
        end;
    end;

    brkptinfo.match    = value_match && state_match && enabled;
    brkptinfo.mismatch = value_mismatch && state_match && enabled;

    return brkptinfo;
end;
```

Library pseudocode for aarch32/debug/breakpoint/AArch32_BreakpointValueMatch

```
// AArch32_BreakpointValueMatch()
// ==============================
// The first result is whether an Address Match or Context breakpoint is programmed on the
// instruction at "address". The second result is whether an Address Mismatch breakpoint is
// programmed on the instruction, that is, whether the instruction should be stepped.

func AArch32_BreakpointValueMatch(n_in : integer, vaddress : bits(32),
                                  linked_to : boolean) => (boolean, boolean)
begin

    // "n" is the identity of the breakpoint unit to match against.
    // "vaddress" is the current instruction address, ignored if linked_to is TRUE and for Context
    //   matching breakpoints.
    // "linked_to" is TRUE if this is a call from StateMatch for linking.
    var n : integer = n_in;
    var c : Constraint;

    // If a non-existent breakpoint then it is CONSTRAINED UNPREDICTABLE whether this gives
    // no match or the breakpoint is mapped to another UNKNOWN implemented breakpoint.
    if n >= NumBreakpointsImplemented() then
        (c, n) = ConstrainUnpredictableInteger(0, NumBreakpointsImplemented() - 1,
                                               Unpredictable_BPNOTIMPL);
        assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};
        if c == Constraint_DISABLED then return (FALSE, FALSE); end;
    end;

    // If this breakpoint is not enabled, it cannot generate a match.
    // (This could also happen on a call from StateMatch for linking).
    if DBGBCR(n).E == '0' then return (FALSE, FALSE); end;

    var dbgtype : bits(4) = DBGBCR(n).BT;

    (c, dbgtype) = AArch32_ReservedBreakpointType(n, dbgtype);
    if c == Constraint_DISABLED then return (FALSE, FALSE); end;
    // Otherwise the dbgtype value returned by AArch32_ReservedBreakpointType is valid.

    // Determine what to compare against.
    let match_addr : boolean      = (dbgtype == '0x0x');
    let mismatch : boolean        = (dbgtype == '010x');
    let match_vmid : boolean      = (dbgtype == '10xx');
    let match_cid1 : boolean      = (dbgtype == 'xx1x');
    let match_cid2 : boolean      = (dbgtype == '11xx');
    let linking_enabled : boolean = (dbgtype == 'xxx1');

    // If called from StateMatch, is CONSTRAINED UNPREDICTABLE if the
    // breakpoint is not programmed with linking enabled.
    if linked_to && !linking_enabled then
        if !ConstrainUnpredictableBool(Unpredictable_BPLINKINGDISABLED) then
            return (FALSE, FALSE);
        end;
    end;

    // If called from BreakpointMatch return FALSE for Linked context ID and/or VMID matches.
    if !linked_to && linking_enabled && !match_addr then
        return (FALSE, FALSE);
    end;

    var bvr_match : boolean  = FALSE;
    var bxvr_match : boolean = FALSE;

    // Do the comparison.
    if match_addr then
        let byte : integer = UInt(vaddress[1:0]);
        assert byte IN {0,2};                     // "vaddress" is halfword aligned

        let byte_select_match : boolean = (DBGBCR(n).BAS[byte] == '1');
        bvr_match = (vaddress[31:2] == DBGBVR(n)[31:2]) && byte_select_match;

    elsif match_cid1 then
        bvr_match = (PSTATE.EL != EL2 && CONTEXTIDR() == DBGBVR(n)[31:0]);
    end;

    if match_vmid then
        let vmid : bits(NUM_VMIDBITS) = VMID();
        var bvr_vmid : bits(NUM_VMIDBITS);

        if ELUsingAArch32(EL2) then
            bvr_vmid = ZeroExtend{NUM_VMIDBITS}(DBGBXVR(n)[7:0]);
        elsif !IsFeatureImplemented(FEAT_VMID16) || VTCR_EL2().VS == '0' then
            bvr_vmid = ZeroExtend{NUM_VMIDBITS}(DBGBXVR(n)[7:0]);
        else
            bvr_vmid = DBGBXVR(n)[15:0];
        end;

        bxvr_match = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() && vmid == bvr_vmid);

    elsif match_cid2 then
        bxvr_match = (PSTATE.EL != EL3 && EL2Enabled() && !ELUsingAArch32(EL2) &&
                      DBGBXVR(n)[31:0] == CONTEXTIDR_EL2()[31:0]);
```

```
    end;

    let bvr_match_valid : boolean  = (match_addr || match_cid1);
    let bxvr_match_valid : boolean = (match_vmid || match_cid2);

    let match : boolean = (!bxvr_match_valid || bxvr_match) && (!bvr_match_valid || bvr_match);

    return (match && !mismatch, !match && mismatch);
end;
```

Library pseudocode for aarch32/debug/breakpoint/AArch32_ReservedBreakpointType

```
// AArch32_ReservedBreakpointType()
// ================================
// Checks if the given DBGBCR(n).BT value is reserved and will generate Constrained Unpredictable
// behavior, otherwise returns Constraint_NONE.

func AArch32_ReservedBreakpointType(n : integer, bt_in : bits(4)) => (Constraint, bits(4))
begin
    var bt : bits(4)       = bt_in;
    var reserved : boolean = FALSE;
    let context_aware : boolean = IsContextAwareBreakpoint(n);

    // Address mismatch
    if bt == '010x' && HaltOnBreakpointOrWatchpoint() then
        reserved = TRUE;
    end;

    // Context matching
    if bt != '0x0x' && !context_aware then
        reserved = TRUE;
    end;

    // EL2 extension
    if bt == '1xxx' && !HaveEL(EL2) then
        reserved = TRUE;
    end;

    // Context matching
    if (bt IN {'011x','11xx'} && !IsFeatureImplemented(FEAT_VHE) &&
          !IsFeatureImplemented(FEAT_Debugv8p2)) then
        reserved = TRUE;
    end;

    if reserved then
        var c : Constraint;
        (c, bt) = ConstrainUnpredictableBits{4}(Unpredictable_RESBPTYPE);
        assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};
        if c == Constraint_DISABLED then
            return (c, ARBITRARY : bits(4));
        // Otherwise the value returned by ConstrainUnpredictableBits must be a not-reserved value
        end;
    end;

    return (Constraint_NONE, bt);
end;
```

Library pseudocode for aarch32/debug/breakpoint/AArch32_StateMatch

```
// AArch32_StateMatch()
// ====================
// Determine whether a breakpoint or watchpoint is enabled in the current mode and state.

func AArch32_StateMatch(ssc_in : bits(2),  hmc_in : bit, pxc_in : bits(2), linked_in : boolean,
                        linked_n_in : integer, isbreakpnt : boolean,
                        accdesc : AccessDescriptor) => boolean
begin

    // "ssc_in","hmc_in","pxc_in" are the control fields from the DBGBCR(n) or DBGWCR(n) register.
    // "linked_in" is TRUE if this is a linked breakpoint/watchpoint type.
    // "linked_n_in" is the linked breakpoint number from the DBGBCR(n) or DBGWCR(n) register.
    // "isbreakpnt" is TRUE for breakpoints, FALSE for watchpoints.
    // "accdesc" describes the properties of the access being matched.
    var hmc : bit          = hmc_in;
    var ssc : bits(2)      = ssc_in;
    var pxc : bits(2)      = pxc_in;
    var linked : boolean   = linked_in;
    var linked_n : integer = linked_n_in;

    // If parameters are set to a reserved type, behaves as either disabled or a defined type
    var c : Constraint;
    // SSCE value discarded as there is no SSCE bit in AArch32_
    (c, ssc, -, hmc, pxc) = CheckValidStateMatch(ssc, '0', hmc, pxc, isbreakpnt);
    if c == Constraint_DISABLED then return FALSE; end;
    // Otherwise the hmc,ssc,pxc values are either valid or the values returned by
    // CheckValidStateMatch are valid.

    let pl2_match : boolean = HaveEL(EL2) && ((hmc == '1' && (ssc::pxc != '1000')) || ssc == '11');
    let pl1_match : boolean = pxc[0] == '1';
    let pl0_match : boolean = pxc[1] == '1';
    let ssu_match : boolean = isbreakpnt && hmc == '0' && pxc == '00' && ssc != '11';

    var priv_match : boolean;
    if ssu_match then
        priv_match = PSTATE.M IN {M32_User,M32_Svc,M32_System};
    else
        case accdesc.el of
            when EL3 => priv_match = pl1_match;           // EL3 and EL1 are both PL1
            when EL2 => priv_match = pl2_match;
            when EL1 => priv_match = pl1_match;
            when EL0 => priv_match = pl0_match;
        end;
    end;

    // Security state match
    var ss_match : boolean;
    case ssc of
        when '00' => ss_match = TRUE;                                    // Both
        when '01' => ss_match = accdesc.ss == SS_NonSecure;              // Non-secure only
        when '10' => ss_match = accdesc.ss == SS_Secure;                 // Secure only
        when '11' => ss_match = (hmc == '1' || accdesc.ss == SS_Secure); // HMC=1 -> Both,
                                                                         // HMC=0 -> Secure only
    end;

    var linked_match : boolean = FALSE;

    if linked then
        // "linked_n" must be an enabled context-aware breakpoint unit.
        // If it is not context-aware then it is CONSTRAINED UNPREDICTABLE whether
        // this gives no match, gives a match without linking, or linked_n is mapped to some
        // UNKNOWN breakpoint that is context-aware.
        if !IsContextAwareBreakpoint(linked_n) then
            let (first_ctx_cmp, last_ctx_cmp) : (integer, integer) = ContextAwareBreakpointRange();
            (c, linked_n) = ConstrainUnpredictableInteger(first_ctx_cmp, last_ctx_cmp,
                                                          Unpredictable_BPNOTCTXCMP);
            assert c IN {Constraint_DISABLED, Constraint_NONE, Constraint_UNKNOWN};

            case c of
                when Constraint_DISABLED =>  return FALSE;      // Disabled
                when Constraint_NONE =>      linked = FALSE;    // No linking
                // Otherwise ConstrainUnpredictableInteger returned a context-aware breakpoint
            end;
        end;

        let vaddress : bits(32) = ARBITRARY : bits(32);
        let linked_to : boolean = TRUE;
        (linked_match,-) = AArch32_BreakpointValueMatch(linked_n, vaddress, linked_to);
    end;

    return priv_match && ss_match && (!linked || linked_match);
end;
```

Library pseudocode for aarch32/debug/enables/AArch32_GenerateDebugExceptions

```
// AArch32_GenerateDebugExceptions()
// =================================

func AArch32_GenerateDebugExceptions() => boolean
begin
    let ss : SecurityState = CurrentSecurityState();
    return AArch32_GenerateDebugExceptionsFrom(PSTATE.EL, ss);
end;
```

Library pseudocode for aarch32/debug/enables/AArch32_GenerateDebugExceptionsFrom

```
// AArch32_GenerateDebugExceptionsFrom()
// =====================================

func AArch32_GenerateDebugExceptionsFrom(from_el : bits(2), from_state : SecurityState) => boolean
begin
    if !ELUsingAArch32(DebugTargetFrom(from_state)) then
        let mask : bit = '0';    // No PSTATE.D in AArch32 state
        return AArch64_GenerateDebugExceptionsFrom(from_el, from_state, mask);
    end;
    if DBGOSLSR().OSLK == '1' || DoubleLockStatus() || Halted() then
        return FALSE;
    end;

    var enabled : boolean;
    if HaveEL(EL3) && from_state == SS_Secure then
        assert from_el != EL2;    // Secure EL2 always uses AArch64
        if IsSecureEL2Enabled() then
            // Implies that EL3 and EL2 both using AArch64
            enabled = MDCR_EL3().SDD == '0';
        else
            let spd : bits(2) = if ELUsingAArch32(EL3) then SDCR().SPD else MDCR_EL3().SPD32;
            if spd[1] == '1' then
                enabled = spd[0] == '1';
            else
                // SPD == 0b01 is reserved, but behaves the same as 0b00.
                enabled = AArch32_SelfHostedSecurePrivilegedInvasiveDebugEnabled();
            end;
        end;
        if from_el == EL0 then enabled = enabled || SDER().SUIDEN == '1'; end;
    else
        enabled = from_el != EL2;
    end;

    return enabled;
end;
```

Library pseudocode for aarch32/debug/pmu/AArch32_IncrementCycleCounter

```
// AArch32_IncrementCycleCounter()
// ===============================
// Increment the cycle counter and possibly set overflow bits.

func AArch32_IncrementCycleCounter()
begin
    if !CountPMUEvents(CYCLE_COUNTER_ID) then return; end;
    var d : bit = PMCR().D;   // Check divide-by-64
    var lc : bit = PMCR().LC;
    // Effective value of 'D' bit is 0 when Effective value of LC is '1'
    if lc == '1' then d = '0'; end;
    if d == '1' && !HasElapsed64Cycles() then return; end;

    let old_value : integer = UInt(PMCCNTR());
    let new_value : integer = old_value + 1;
    PMCCNTR() = new_value[63:0];

    let ovflw : integer{} = if lc == '1' then 64 else 32;

    if old_value[64:ovflw] != new_value[64:ovflw] then
        PMOVSSET().C = '1';
    end;

    return;
end;
```

Library pseudocode for aarch32/debug/pmu/AArch32_IncrementEventCounter

```
// AArch32_IncrementEventCounter()
// ===============================
// Increment the specified event counter 'idx' by the specified amount 'increment'.
// 'Vm' is the value event counter 'idx-1' is being incremented by if 'idx' is odd,
// zero otherwise.
// Returns the amount the counter was incremented by.

func AArch32_IncrementEventCounter(idx : integer, increment_in : integer, Vm : integer) => integer
begin
    if HaveAArch64() then
        // Force the counter to be incremented as a 64-bit counter.
        return AArch64_IncrementEventCounter(idx, increment_in, Vm);
    end;

    // In this model, event counters in an AArch32-only implementation are 32 bits and
    // the LP bits are RES0 in this model, even if FEAT_PMUv3p5 is implemented.
    var old_value : integer;
    var new_value : integer;

    old_value = UInt(PMEVCNTR(idx));
    let increment : integer = PMUCountValue(idx, increment_in, Vm);
    new_value = old_value + increment;

    PMEVCNTR(idx) = new_value[31:0];
    let ovflw : integer{} = 32;
    if old_value[64:ovflw] != new_value[64:ovflw] then
        PMOVSSET()[idx] = '1';
        // Check for the CHAIN event from an even counter
        if (idx[0] == '0' && idx + 1 < NUM_PMU_COUNTERS &&
              (GetPMUCounterRange(idx) == GetPMUCounterRange(idx+1) ||
                 ConstrainUnpredictableBool(Unpredictable_COUNT_CHAIN))) then
            // If PMU counters idx and idx+1 are not in same range,
            // it is CONSTRAINED UNPREDICTABLE if CHAIN event is counted
            PMUEvent(PMU_EVENT_CHAIN, 1, idx + 1);
        end;
    end;

    return increment;
end;
```

Library pseudocode for aarch32/debug/pmu/AArch32_PMUCycle

```
// AArch32_PMUCycle()
// ==================
// Called at the end of each cycle to increment event counters and
// check for PMU overflow. In pseudocode, a cycle ends after the
// execution of the operational pseudocode.

func AArch32_PMUCycle()
begin
    if HaveAArch64() then
        AArch64_PMUCycle();
        return;
    end;

    if !IsFeatureImplemented(FEAT_PMUv3) then
        return;
    end;
    if ((!IsWFIsleep && !IsWFEsleep) || (!IsFeatureImplemented(FEAT_PMUv3p9) &&
                                         ImpDefBool("PMU counts cycles in WFI or WFE state"))) then
        PMUEvent(PMU_EVENT_CPU_CYCLES);
    end;
    let counters : integer = NUM_PMU_COUNTERS;
    var Vm : integer = 0;
    if counters != 0 then
        for idx = 0 to counters - 1 do
            if CountPMUEvents(idx) then
                let accumulated : integer = PMUEventAccumulator[[idx]];
                if (idx MOD 2) == 0 then Vm = 0; end;
                Vm = AArch32_IncrementEventCounter(idx, accumulated, Vm);
            end;
            PMUEventAccumulator[[idx]] = 0;
        end;
    end;
    AArch32_IncrementCycleCounter();
    CheckForPMUOverflow();
end;
```

Library pseudocode for aarch32/debug/takeexceptiondbg/AArch32_EnterHypModeInDebugState

```
// AArch32_EnterHypModeInDebugState()
// ==================================
// Take an exception in Debug state to Hyp mode.

noreturn func AArch32_EnterHypModeInDebugState(except : ExceptionRecord)
begin
    SynchronizeContext();
    assert HaveEL(EL2) && CurrentSecurityState() == SS_NonSecure && ELUsingAArch32(EL2);

    AArch32_ReportHypEntry(except);
    AArch32_WriteMode(M32_Hyp);
    SPSR_curr() = ARBITRARY : bits(32);
    ELR_hyp()   = ARBITRARY : bits(32);
    // In Debug state, the PE always execute T32 instructions when in AArch32 state, and
    // PSTATE.[SS,A,I,F] are not observable so behave as UNKNOWN.
    PSTATE.T = '1';                             // PSTATE.J is RES0
    PSTATE.[SS,A,I,F] = ARBITRARY : bits(4);
    DLR()   = ARBITRARY : bits(32);
    DSPSR() = ARBITRARY : bits(32);
    if IsFeatureImplemented(FEAT_Debugv8p9) then
        DSPSR2() = ARBITRARY : bits(32);
    end;
    PSTATE.E  = HSCTLR().EE;
    PSTATE.IL = '0';
    PSTATE.IT = '00000000';
    if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = ARBITRARY : bit; end;
    EDSCR().ERR = '1';
    UpdateEDSCRFields();

    EndOfInstruction(EndOfInstructionReason_Exception);
end;
```

Library pseudocode for aarch32/debug/takeexceptiondbg/AArch32_EnterModeInDebugState

```
// AArch32_EnterModeInDebugState()
// ===============================
// Take an exception in Debug state to a mode other than Monitor and Hyp mode.

noreturn func AArch32_EnterModeInDebugState(target_mode : bits(5))
begin
    SynchronizeContext();
    assert ELUsingAArch32(EL1) && PSTATE.EL != EL2;

    if PSTATE.M == M32_Monitor then SCR().NS = '0'; end;
    AArch32_WriteMode(target_mode);
    SPSR_curr() = ARBITRARY : bits(32);
    R(14)       = ARBITRARY : bits(32);
    // In Debug state, the PE always execute T32 instructions when in AArch32 state, and
    // PSTATE.[SS,A,I,F] are not observable so behave as UNKNOWN.
    PSTATE.T = '1';                             // PSTATE.J is RES0
    PSTATE.[SS,A,I,F] = ARBITRARY : bits(4);
    DLR() = ARBITRARY : bits(32);
    DSPSR() = ARBITRARY : bits(32);
    if IsFeatureImplemented(FEAT_Debugv8p9) then
        DSPSR2() = ARBITRARY : bits(32);
    end;
    PSTATE.E  = SCTLR().EE;
    PSTATE.IL = '0';
    PSTATE.IT = '00000000';
    if IsFeatureImplemented(FEAT_PAN) && SCTLR().SPAN == '0' then PSTATE.PAN = '1'; end;
    if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = ARBITRARY : bit; end;
    EDSCR().ERR = '1';
    UpdateEDSCRFields();                        // Update EDSCR PE state flags.

    EndOfInstruction(EndOfInstructionReason_Exception);
end;
```

Library pseudocode for aarch32/debug/takeexceptiondbg/AArch32_EnterMonitorModeInDebugState

```
// AArch32_EnterMonitorModeInDebugState()
// ======================================
// Take an exception in Debug state to Monitor mode.

func AArch32_EnterMonitorModeInDebugState()
begin
    SynchronizeContext();
    assert HaveEL(EL3) && ELUsingAArch32(EL3);
    let from_secure : boolean = CurrentSecurityState() == SS_Secure;
    if PSTATE.M == M32_Monitor then SCR().NS = '0'; end;
    AArch32_WriteMode(M32_Monitor);
    SPSR_curr() = ARBITRARY : bits(32);
    R(14) = ARBITRARY : bits(32);
    // In Debug state, the PE always execute T32 instructions when in AArch32 state, and
    // PSTATE.[SS,A,I,F] are not observable so behave as UNKNOWN.
    PSTATE.T = '1';                             // PSTATE.J is RES0
    PSTATE.[SS,A,I,F] = ARBITRARY : bits(4);
    PSTATE.E = SCTLR().EE;
    PSTATE.IL = '0';
    PSTATE.IT = '00000000';
    if IsFeatureImplemented(FEAT_PAN) then
        if !from_secure then
            PSTATE.PAN = '0';
        elsif SCTLR().SPAN == '0' then
            PSTATE.PAN = '1';
        end;
    end;
    if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = ARBITRARY : bit; end;
    DLR() = ARBITRARY : bits(32);
    DSPSR() = ARBITRARY : bits(32);
    if IsFeatureImplemented(FEAT_Debugv8p9) then
        DSPSR2() = ARBITRARY : bits(32);
    end;
    EDSCR().ERR = '1';
    UpdateEDSCRFields();                        // Update EDSCR PE state flags.

    EndOfInstruction(EndOfInstructionReason_Exception);
end;
```

Library pseudocode for aarch32/debug/watchpoint/AArch32_WatchpointByteMatch

```
// AArch32_WatchpointByteMatch()
// ============================

func AArch32_WatchpointByteMatch(n : integer, vaddress : bits(32)) => boolean
begin
    let dbgtop : integer{} = 31;
    let cmpbottom : integer{} = if DBGWVR(n)[2] == '1' then 2 else 3; // Word or doubleword
    var bottom = cmpbottom;
    let select : integer{} = UInt(vaddress[cmpbottom-1:0]);
    var byte_select_match : boolean = (DBGWCR(n).BAS[select] != '0');
    var mask : integer{} = UInt(DBGWCR(n).MASK);

    // If DBGWCR(n).MASK is a nonzero value and DBGWCR(n).BAS is not set to '11111111', or
    // DBGWCR(n).BAS specifies a non-contiguous set of bytes behavior is CONSTRAINED
    // UNPREDICTABLE.
    if mask > 0 && !IsOnes(DBGWCR(n).BAS) then
        byte_select_match = ConstrainUnpredictableBool(Unpredictable_WPMASKANDBAS);
    else
        let LSB : bits(8) = (DBGWCR(n).BAS AND NOT(DBGWCR(n).BAS - 1));
        let MSB : bits(8) = (DBGWCR(n).BAS + LSB);
        if !IsZero(MSB AND (MSB - 1)) then          // Not contiguous
            byte_select_match = ConstrainUnpredictableBool(Unpredictable_WPBASCONTIGUOUS);
            bottom = 3;                             // For the whole doubleword
        end;
    end;

    // If the address mask is set to a reserved value, the behavior is CONSTRAINED UNPREDICTABLE.
    if mask > 0 && mask <= 2 then
        var c : Constraint;
        var unpred_mask : integer;
        (c, unpred_mask) = ConstrainUnpredictableInteger(3, 31, Unpredictable_RESWPMASK);
        assert c IN {Constraint_DISABLED, Constraint_NONE, Constraint_UNKNOWN};
        case c of
            when Constraint_DISABLED => return FALSE; // Disabled
            when Constraint_NONE     => mask = 0;     // No masking
            // Otherwise the value returned by ConstrainUnpredictableInteger is a not-reserved value
            otherwise                => mask = unpred_mask as integer{3..31};
        end;
    end;

    let cmpmsb : integer{}    = dbgtop;
    let cmplsb : integer{}    = if mask > bottom then mask else bottom;
    let bottombit : integer{} = bottom;
    var WVR_match : boolean   = (vaddress[cmpmsb:cmplsb] == DBGWVR(n)[cmpmsb:cmplsb]);
    if mask > bottom then
        // If masked bits of DBGWVR(n) are not zero, the behavior is CONSTRAINED UNPREDICTABLE.
        if WVR_match && !IsZero(DBGWVR(n)[cmplsb-1:bottombit]) then
            WVR_match = ConstrainUnpredictableBool(Unpredictable_WPMASKEDBITS);
        end;
    end;

    return (WVR_match && byte_select_match);
end;
```

Library pseudocode for aarch32/debug/watchpoint/AArch32_WatchpointMatch

```
// AArch32_WatchpointMatch()
// =========================
// Watchpoint matching in an AArch32 translation regime.

func AArch32_WatchpointMatch(n : integer, vaddress : bits(32), size : integer,
                             accdesc : AccessDescriptor) => WatchpointInfo
begin
    assert ELUsingAArch32(S1TranslationRegime());
    assert n < NumWatchpointsImplemented();

    let enabled : boolean     = DBGWCR(n).E == '1';
    let linked : boolean      = DBGWCR(n).WT == '1';
    let isbreakpnt : boolean  = FALSE;
    let linked_n : integer{}  = UInt(DBGWCR_EL1(n).LBN);
    let state_match : boolean = AArch32_StateMatch(DBGWCR(n).SSC, DBGWCR(n).HMC, DBGWCR(n).PAC,
                                                   linked, linked_n, isbreakpnt, accdesc);

    var watchptinfo : WatchpointInfo;
    var ls_match : boolean;
    case DBGWCR(n).LSC[1:0] of
        when '00' => ls_match = FALSE;
        when '01' => ls_match = accdesc.read;
        when '10' => ls_match = accdesc.write || accdesc.acctype == AccessType_DC;
        when '11' => ls_match = TRUE;
    end;

    var value_match : boolean = FALSE;
    watchptinfo.vaddress = ZeroExtend{64}(vaddress);
    for byte = 0 to size - 1 do
        if (!value_match &&
              !AddressInNaturallyAlignedBlock(watchptinfo.vaddress,
                                              ZeroExtend{64}(vaddress + byte))) then
            // Watchpoint should report an address which is in
            // the naturally aligned block of the matched address.
            watchptinfo.vaddress = ZeroExtend{64}(vaddress + byte);
        end;
        value_match = value_match || AArch32_WatchpointByteMatch(n, vaddress + byte);
    end;

    watchptinfo.watchpt_num = n;
    watchptinfo.value_match = value_match && state_match && ls_match && enabled;
    return watchptinfo;
end;
```

Library pseudocode for aarch32/exceptions/aborts/AArch32_Abort

```
// AArch32_Abort()
// ===============
// Abort and Debug exception handling in an AArch32 translation regime.

func AArch32_Abort(fault : FaultRecord)
begin
    // Check if routed to AArch64 state
    var route_to_aarch64 : boolean = ((IsExternalAbort(fault) &&
                                      !ELUsingAArch32(SyncExternalAbortTarget(fault))) ||
                                      (PSTATE.EL == EL0 && !ELUsingAArch32(EL1)));

    if !route_to_aarch64 && EL2Enabled() && !ELUsingAArch32(EL2) then
        route_to_aarch64 = (HCR_EL2().TGE == '1' || IsSecondStage(fault) ||
                            (IsDebugException(fault) && MDCR_EL2().TDE == '1'));
    end;

    if route_to_aarch64 then
        AArch64_Abort(fault);
    end;
    if fault.accessdesc.acctype == AccessType_IFETCH then
        AArch32_TakePrefetchAbortException(fault);
    else
        AArch32_TakeDataAbortException(fault);
    end;
end;
```

Library pseudocode for aarch32/exceptions/aborts/AArch32_AbortSyndrome

```
// AArch32_AbortSyndrome()
// =======================
// Creates an exception syndrome record for Abort exceptions
// taken to Hyp mode
// from an AArch32 translation regime.
func AArch32_AbortSyndrome(exceptype : Exception, fault : FaultRecord,
                           target_el : bits(2)) => ExceptionRecord
begin
    var except : ExceptionRecord = ExceptionSyndrome(exceptype);

    except.syndrome.iss = AArch32_FaultSyndrome(exceptype, fault);
    if exceptype == Exception_Watchpoint then
        except.vaddress = fault.watchptinfo.vaddress;
    else
        except.vaddress = ZeroExtend{64}(fault.vaddress);
    end;

    if IPAValid(fault) then
        except.ipavalid  = TRUE;
        except.NS        = if fault.ipaddress.paspace == PAS_NonSecure then '1' else '0';
        except.ipaddress = ZeroExtend{NUM_PABITS}(fault.ipaddress.address);
    else
        except.ipavalid = FALSE;
    end;

    return except;
end;
```

Library pseudocode for aarch32/exceptions/aborts/AArch32_CheckPCAlignment

```
// AArch32_CheckPCAlignment()
// ==========================

func AArch32_CheckPCAlignment()
begin
    let pc : bits(32) = ThisInstrAddr{}();

    if (CurrentInstrSet() == InstrSet_A32 && pc[1] == '1') || pc[0] == '1' then
        if AArch32_GeneralExceptionsToAArch64() then AArch64_PCAlignmentFault(); end;
        let accdesc : AccessDescriptor = CreateAccDescIFetch();
        var fault : FaultRecord = NoFault(accdesc, ZeroExtend{64}(pc));
        // Generate an Alignment fault Prefetch Abort exception
        fault.statuscode = Fault_Alignment;
        AArch32_Abort(fault);
    end;
end;
```

Library pseudocode for aarch32/exceptions/aborts/AArch32_CommonFaultStatus

```
// AArch32_CommonFaultStatus()
// ===========================
// Return the common part of the fault status on reporting a Data
// or Prefetch Abort.

func AArch32_CommonFaultStatus(fault : FaultRecord, long_format : boolean) => bits(32)
begin
    var target : bits(32) = Zeros{};
    if IsFeatureImplemented(FEAT_RAS) && IsAsyncAbort(fault) then
        let errstate : ErrorState = PEErrorState(fault);
        target[15:14] = AArch32_EncodeAsyncErrorSyndrome(errstate);   // AET
    end;
    if IsExternalAbort(fault) then target[12] = fault.extflag; end;   // ExT
    target[9] = if long_format then '1' else '0';                     // LPAE
    if long_format then                                               // Long-descriptor format
        target[5:0]    = EncodeLDFSC(fault.statuscode, fault.level);  // STATUS
    else                                                              // Short-descriptor format
        target[10,3:0] = EncodeSDFSC(fault.statuscode, fault.level);  // FS
    end;
    return target;
end;
```

Library pseudocode for aarch32/exceptions/aborts/AArch32_ReportDataAbort

```
// AArch32_ReportDataAbort()
// =========================
// Report syndrome information for aborts taken to modes other than Hyp mode.

func AArch32_ReportDataAbort(route_to_monitor : boolean, fault : FaultRecord)
begin
    var long_format : boolean;
    if route_to_monitor && CurrentSecurityState() != SS_Secure then
        long_format = ((TTBCR_S().EAE == '1') ||
                       (IsExternalSyncAbort(fault) && ((PSTATE.EL == EL2 || TTBCR().EAE == '1') ||
                        (fault.secondstage && ImpDefBool(
                                              "Report abort using Long-descriptor format")))));
    else
        long_format = TTBCR().EAE == '1';
    end;
    var syndrome : bits(32) = AArch32_CommonFaultStatus(fault, long_format);

    // bits of syndrome that are not common to I and D side
    if fault.accessdesc.acctype IN {AccessType_DC, AccessType_IC, AccessType_AT} then
        syndrome[13] = '1';                             // CM
        syndrome[11] = '1';                             // WnR
    else
        syndrome[11] = if fault.write then '1' else '0'; // WnR
    end;

    if !long_format then
        syndrome[7:4] = fault.domain;                   // Domain
    end;

    if fault.accessdesc.acctype == AccessType_IC then
        var i_syndrome : bits(32);
        if (!long_format &&
            ImpDefBool("Report I-cache maintenance fault in IFSR")) then
            i_syndrome = syndrome;
            syndrome[10,3:0] = EncodeSDFSC(Fault_ICacheMaint, 1);
        else
            i_syndrome = ARBITRARY : bits(32);
        end;
        if route_to_monitor then
            IFSR_S() = i_syndrome;
        else
            IFSR() = i_syndrome;
        end;
    end;

    if route_to_monitor then
        DFSR_S() = syndrome;
        DFAR_S() = fault.vaddress[31:0];
    else
        DFSR()   = syndrome;
        DFAR()   = fault.vaddress[31:0];
    end;
end;
```

Library pseudocode for aarch32/exceptions/aborts/AArch32_ReportPrefetchAbort

```
// AArch32_ReportPrefetchAbort()
// =============================
// Report syndrome information for aborts taken to modes other than Hyp mode.

func AArch32_ReportPrefetchAbort(route_to_monitor : boolean, fault : FaultRecord)
begin
    // The encoding used in the IFSR can be Long-descriptor format or Short-descriptor format.
    // Normally, the current translation table format determines the format. For an abort from
    // Non-secure state to Monitor mode, the IFSR uses the Long-descriptor format if any of the
    // following applies:
    // * The Secure TTBCR.EAE is set to 1.
    // * It is taken from Hyp mode.
    // * It is taken from EL1 or EL0, and the Non-secure TTBCR.EAE is set to 1.
    var long_format : boolean = FALSE;
    if route_to_monitor && CurrentSecurityState() != SS_Secure then
        long_format = TTBCR_S().EAE == '1' || PSTATE.EL == EL2 || TTBCR().EAE == '1';
    else
        long_format = TTBCR().EAE == '1';
    end;

    let fsr : bits(32) = AArch32_CommonFaultStatus(fault, long_format);
    if route_to_monitor then
        IFSR_S() = fsr;
        IFAR_S() = fault.vaddress[31:0];
    else
        IFSR() = fsr;
        IFAR() = fault.vaddress[31:0];
    end;
    return;
end;
```

Library pseudocode for aarch32/exceptions/aborts/AArch32_TakeDataAbortException

```
// AArch32_TakeDataAbortException()
// ================================

func AArch32_TakeDataAbortException(fault : FaultRecord)
begin

    var sea_target : bits(2);
    if IsExternalAbort(fault) then
        sea_target = SyncExternalAbortTarget(fault);
    else
        sea_target = ARBITRARY : bits(2);
    end;

    let route_to_monitor : boolean = IsExternalAbort(fault) && sea_target == EL3;
    let route_to_hyp : boolean     = (EL2Enabled() && PSTATE.EL IN {EL0, EL1} &&
                                      (HCR().TGE == '1' ||
                                      (IsExternalAbort(fault) && sea_target == EL2) ||
                                      (IsDebugException(fault) && HDCR().TDE == '1') ||
                                       IsSecondStage(fault)));

    let preferred_exception_return : bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x10;
    let lr_offset : integer = 8;

    if IsDebugException(fault) then DBGDSCRext().MOE = fault.debugmoe; end;
    if route_to_monitor then
        AArch32_ReportDataAbort(route_to_monitor, fault);
        AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
    elsif PSTATE.EL == EL2 || route_to_hyp then
        let except : ExceptionRecord = AArch32_AbortSyndrome(Exception_DataAbort, fault, EL2);
        if PSTATE.EL == EL2 then
            AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
        else
            AArch32_EnterHypMode(except, preferred_exception_return, 0x14);
        end;
    else
        AArch32_ReportDataAbort(route_to_monitor, fault);
        AArch32_EnterMode(M32_Abort, preferred_exception_return, lr_offset, vect_offset);
    end;
end;
```

Library pseudocode for aarch32/exceptions/aborts/AArch32_TakePrefetchAbortException

```
// AArch32_TakePrefetchAbortException()
// ====================================

func AArch32_TakePrefetchAbortException(fault : FaultRecord)
begin

    var sea_target : bits(2);
    if IsExternalAbort(fault) then
        sea_target = SyncExternalAbortTarget(fault);
    else
        sea_target = ARBITRARY : bits(2);
    end;

    let route_to_monitor : boolean = IsExternalAbort(fault) && sea_target == EL3;
    let route_to_hyp : boolean = (EL2Enabled() && PSTATE.EL IN {EL0, EL1} &&
                                  (HCR().TGE == '1' ||
                                  (IsExternalAbort(fault) && sea_target == EL2) ||
                                  (IsDebugException(fault) && HDCR().TDE == '1') ||
                                   IsSecondStage(fault)));

    var except : ExceptionRecord;
    let preferred_exception_return : bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0C;
    let lr_offset : integer = 4;

    if IsDebugException(fault) then DBGDSCRext().MOE = fault.debugmoe; end;
    if route_to_monitor then
        AArch32_ReportPrefetchAbort(route_to_monitor, fault);
        AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
    elsif PSTATE.EL == EL2 || route_to_hyp then
        if fault.statuscode == Fault_Alignment then             // PC Alignment fault
            except = ExceptionSyndrome(Exception_PCAlignment);
            except.vaddress = ThisInstrAddr{64}();
        else
            except = AArch32_AbortSyndrome(Exception_InstructionAbort, fault, EL2);
        end;
        if PSTATE.EL == EL2 then
            AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
        else
            AArch32_EnterHypMode(except, preferred_exception_return, 0x14);
        end;
    else
        AArch32_ReportPrefetchAbort(route_to_monitor, fault);
        AArch32_EnterMode(M32_Abort, preferred_exception_return, lr_offset, vect_offset);
    end;
end;
```

Library pseudocode for aarch32/exceptions/async/AArch32_TakePhysicalFIQException

```
// AArch32_TakePhysicalFIQException()
// ==================================

func AArch32_TakePhysicalFIQException()
begin

    // Check if routed to AArch64 state
    var route_to_aarch64 : boolean = PSTATE.EL == EL0 && !ELUsingAArch32(EL1);
    if !route_to_aarch64 && EL2Enabled() && !ELUsingAArch32(EL2) then
        route_to_aarch64 = HCR_EL2().TGE == '1' || (HCR_EL2().FMO == '1' && !IsInHost());
    end;

    if !route_to_aarch64 then
        route_to_aarch64 = EffectiveSCR_EL3_FIQ() == '1';
    end;

    if route_to_aarch64 then AArch64_TakePhysicalFIQException(); end;
    let route_to_monitor : boolean = HaveEL(EL3) && SCR().FIQ == '1';
    let route_to_hyp : boolean = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
                                  (HCR().TGE == '1' || HCR().FMO == '1'));
    let preferred_exception_return : bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x1C;
    let lr_offset : integer   = 4;
    if route_to_monitor then
        AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
    elsif PSTATE.EL == EL2 || route_to_hyp then
        let except : ExceptionRecord = ExceptionSyndrome(Exception_FIQ);
        AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
    else
        AArch32_EnterMode(M32_FIQ, preferred_exception_return, lr_offset, vect_offset);
    end;
end;
```

Library pseudocode for aarch32/exceptions/async/AArch32_TakePhysicalIRQException

```
// AArch32_TakePhysicalIRQException()
// ==================================
// Take an enabled physical IRQ exception.

func AArch32_TakePhysicalIRQException()
begin

    // Check if routed to AArch64 state
    var route_to_aarch64 : boolean = PSTATE.EL == EL0 && !ELUsingAArch32(EL1);
    if !route_to_aarch64 && EL2Enabled() && !ELUsingAArch32(EL2) then
        route_to_aarch64 = HCR_EL2().TGE == '1' || (HCR_EL2().IMO == '1' && !IsInHost());
    end;
    if !route_to_aarch64 then
        route_to_aarch64 = EffectiveSCR_EL3_IRQ() == '1';
    end;

    if route_to_aarch64 then AArch64_TakePhysicalIRQException(); end;

    let route_to_monitor : boolean = HaveEL(EL3) && SCR().IRQ == '1';
    let route_to_hyp : boolean = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
                                  (HCR().TGE == '1' || HCR().IMO == '1'));
    let preferred_exception_return : bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x18;
    let lr_offset : integer   = 4;
    if route_to_monitor then
        AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
    elsif PSTATE.EL == EL2 || route_to_hyp then
        let except : ExceptionRecord = ExceptionSyndrome(Exception_IRQ);
        AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
    else
        AArch32_EnterMode(M32_IRQ, preferred_exception_return, lr_offset, vect_offset);
    end;
end;
```

Library pseudocode for aarch32/exceptions/async/AArch32_TakePhysicalSErrorException

```
// AArch32_TakePhysicalSErrorException()
// =====================================

func AArch32_TakePhysicalSErrorException(implicit_esb : boolean)
begin
    var masked : boolean;
    var target_el : bits(2);

    (masked, target_el) = PhysicalSErrorTarget();
    assert !masked;

    // Check if routed to AArch64 state
    if !ELUsingAArch32(target_el) then
        AArch64_TakePhysicalSErrorException(implicit_esb);
    end;

    let preferred_exception_return : bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x10;
    let fault : FaultRecord   = GetPendingPhysicalSError();
    let lr_offset : integer   = 8;

    let except : ExceptionRecord   = AArch32_AbortSyndrome(Exception_DataAbort, fault, target_el);
    let route_to_monitor : boolean = (target_el == EL3);

    if IsSErrorEdgeTriggered() then
        ClearPendingPhysicalSError();
    end;

    case target_el of
        when EL3 =>
            AArch32_ReportDataAbort(route_to_monitor, fault);
            AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
        when EL2 =>
            if PSTATE.EL == EL2 then
                AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
            else
                AArch32_EnterHypMode(except, preferred_exception_return, 0x14);
            end;
        when EL1 =>
            AArch32_ReportDataAbort(route_to_monitor, fault);
            AArch32_EnterMode(M32_Abort, preferred_exception_return, lr_offset, vect_offset);
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for aarch32/exceptions/async/AArch32_TakeVirtualFIQException

```
// AArch32_TakeVirtualFIQException()
// =================================

func AArch32_TakeVirtualFIQException()
begin
    assert PSTATE.EL IN {EL0, EL1} && EL2Enabled();
    if ELUsingAArch32(EL2) then  // Virtual IRQ enabled if TGE==0 and FMO==1
        assert HCR().TGE == '0' && HCR().FMO == '1';
    else
        assert HCR_EL2().TGE == '0' && HCR_EL2().FMO == '1';
    end;
    // Check if routed to AArch64 state
    if PSTATE.EL == EL0 && !ELUsingAArch32(EL1) then AArch64_TakeVirtualFIQException(); end;

    let preferred_exception_return : bits(32)= ThisInstrAddr{}();
    let vect_offset : integer = 0x1C;
    let lr_offset : integer   = 4;

    AArch32_EnterMode(M32_FIQ, preferred_exception_return, lr_offset, vect_offset);
end;
```

Library pseudocode for aarch32/exceptions/async/AArch32_TakeVirtualIRQException

```
// AArch32_TakeVirtualIRQException()
// =================================

func AArch32_TakeVirtualIRQException()
begin
    assert PSTATE.EL IN {EL0, EL1} && EL2Enabled();

    if ELUsingAArch32(EL2) then   // Virtual IRQs enabled if TGE==0 and IMO==1
        assert HCR().TGE == '0' && HCR().IMO == '1';
    else
        assert HCR_EL2().TGE == '0' && HCR_EL2().IMO == '1';
    end;

    // Check if routed to AArch64 state
    if PSTATE.EL == EL0 && !ELUsingAArch32(EL1) then AArch64_TakeVirtualIRQException(); end;

    let preferred_exception_return :  bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x18;
    let lr_offset : integer   = 4;

    AArch32_EnterMode(M32_IRQ, preferred_exception_return, lr_offset, vect_offset);
end;
```

Library pseudocode for aarch32/exceptions/async/AArch32_TakeVirtualSErrorException

```
// AArch32_TakeVirtualSErrorException()
// ====================================

func AArch32_TakeVirtualSErrorException()
begin

    assert PSTATE.EL IN {EL0, EL1} && EL2Enabled();
    if ELUsingAArch32(EL2) then   // Virtual SError enabled if TGE==0 and AMO==1
        assert HCR().TGE == '0' && HCR().AMO == '1';
    else
        assert HCR_EL2().TGE == '0' && HCR_EL2().AMO == '1';
    end;
    // Check if routed to AArch64 state
    if PSTATE.EL == EL0 && !ELUsingAArch32(EL1) then AArch64_TakeVirtualSErrorException(); end;
    let route_to_monitor : boolean = FALSE;

    let preferred_exception_return : bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x10;
    let lr_offset : integer   = 8;

    let vaddress : bits(32)   = ARBITRARY : bits(32);
    let parity : boolean      = FALSE;
    let fault : Fault         = Fault_AsyncExternal;
    let level : integer       = ARBITRARY : integer;
    var fsr : bits(32)        = Zeros{};
    if IsFeatureImplemented(FEAT_RAS) then
        if ELUsingAArch32(EL2) then
            fsr[15:14] =  VDFSR().AET;
            fsr[12] =  VDFSR().ExT;
        else
            fsr[15:14] =  VSESR_EL2().AET;
            fsr[12] =  VSESR_EL2().ExT;
        end;
    else
        fsr[12] = ImpDefBit("Virtual External abort type");
    end;
    if TTBCR().EAE == '1' then // Long-descriptor format
        fsr[9] = '1';
        fsr[5:0] = EncodeLDFSC(fault, level);
    else                       // Short-descriptor format
        fsr[9] = '0';
        fsr[10,3:0] = EncodeSDFSC(fault, level);
    end;
    DFSR() = fsr;
    DFAR() = ARBITRARY : bits(32);
    ClearPendingVirtualSError();
    AArch32_EnterMode(M32_Abort, preferred_exception_return, lr_offset, vect_offset);
end;
```

Library pseudocode for aarch32/exceptions/debug/AArch32_SoftwareBreakpoint

```
// AArch32_SoftwareBreakpoint()
// ============================

func AArch32_SoftwareBreakpoint(immediate : bits(16))
begin

    if (EL2Enabled() && !ELUsingAArch32(EL2) &&
        (HCR_EL2().TGE == '1' || MDCR_EL2().TDE == '1')) || !ELUsingAArch32(EL1) then
        AArch64_SoftwareBreakpoint(immediate);
    end;

    let accdesc : AccessDescriptor = CreateAccDescIFetch();
    var fault : FaultRecord = NoFault(accdesc, ARBITRARY : bits(64));

    fault.statuscode = Fault_Debug;
    fault.debugmoe   = DebugException_BKPT;

    AArch32_Abort(fault);
end;
```

Library pseudocode for aarch32/exceptions/debug/DebugException

```
// DebugException
// ==============
// Reason codes for debug exceptions, taken to AArch32

constant DebugException_Breakpoint : bits(4)  = '0001';
constant DebugException_BKPT : bits(4)        = '0011';
constant DebugException_VectorCatch : bits(4) = '0101';
constant DebugException_Watchpoint : bits(4)  = '1010';
```

Library pseudocode for aarch32/exceptions/exceptions/AArch32_CheckAdvSIMDOrFPRegisterTraps

```
// AArch32_CheckAdvSIMDOrFPRegisterTraps()
// =======================================
// Check if an instruction that accesses an Advanced SIMD and
// floating-point System register is trapped by an appropriate HCR.TIDx
// ID group trap control.

func AArch32_CheckAdvSIMDOrFPRegisterTraps(reg : bits(4))
begin
    if PSTATE.EL == EL1 && EL2Enabled() then
        let tid0 : bit = if ELUsingAArch32(EL2) then HCR().TID0 else HCR_EL2().TID0;
        let tid3 : bit = if ELUsingAArch32(EL2) then HCR().TID3 else HCR_EL2().TID3;
        if ((tid0 == '1' && reg == '0000') ||                           // FPSID
              (tid3 == '1' && reg IN {'0101', '0110', '0111'})) then    // MVFRx
            if ELUsingAArch32(EL2) then
                AArch32_SystemAccessTrap(M32_Hyp, 0x8);
            else
                AArch64_AArch32SystemAccessTrap(EL2, 0x8);
            end;
        end;
    end;
end;
```

Library pseudocode for aarch32/exceptions/exceptions/AArch32_ExceptionClass

```
// AArch32_ExceptionClass()
// ========================
// Returns the Exception Class and Instruction Length fields to be reported in HSR

func AArch32_ExceptionClass(exceptype : Exception) => (integer,bit)
begin
    var il_is_valid : boolean = TRUE;
    var ec : integer;
    case exceptype of
        when Exception_Uncategorized =>        ec = 0x00; il_is_valid = FALSE;
        when Exception_WFxTrap =>              ec = 0x01;
        when Exception_CP15RTTrap =>           ec = 0x03;
        when Exception_CP15RRTTrap =>          ec = 0x04;
        when Exception_CP14RTTrap =>           ec = 0x05;
        when Exception_CP14DTTrap =>           ec = 0x06;
        when Exception_AdvSIMDFPAccessTrap =>  ec = 0x07;
        when Exception_FPIDTrap =>             ec = 0x08;
        when Exception_CP14RRTTrap =>          ec = 0x0C;
        when Exception_IllegalState =>         ec = 0x0E; il_is_valid = FALSE;
        when Exception_SupervisorCall =>       ec = 0x11;
        when Exception_HypervisorCall =>       ec = 0x12;
        when Exception_MonitorCall =>          ec = 0x13;
        when Exception_InstructionAbort =>
            ec = if PSTATE.EL == EL2 then 0x21 else 0x20;
            il_is_valid = FALSE;
        when Exception_PCAlignment =>          ec = 0x22; il_is_valid = FALSE;
        when Exception_DataAbort =>            ec = if PSTATE.EL == EL2 then 0x25 else 0x24;
        when Exception_FPTrappedException =>   ec = 0x28;
        when Exception_Profiling =>            ec = 0x3D;
        otherwise =>                           unreachable;
    end;

    var il : bit;
    if il_is_valid then
        il = if ThisInstrLength() == 32 then '1' else '0';
    else
        il = '1';
    end;

    return (ec,il);
end;
```

Library pseudocode for aarch32/exceptions/exceptions/AArch32_GeneralExceptionsToAArch64

```
// AArch32_GeneralExceptionsToAArch64()
// ====================================
// Returns TRUE if exceptions normally routed to EL1 are being handled at an Exception
// level using AArch64, because either EL1 is using AArch64 or TGE is in force and EL2
// is using AArch64.

func AArch32_GeneralExceptionsToAArch64() => boolean
begin
    return ((PSTATE.EL == EL0 && !ELUsingAArch32(EL1)) ||
            (EL2Enabled() && !ELUsingAArch32(EL2) && HCR_EL2().TGE == '1'));
end;
```

Library pseudocode for aarch32/exceptions/exceptions/AArch32_ReportHypEntry

```
// AArch32_ReportHypEntry()
// ========================
// Report syndrome information to Hyp mode registers.

func AArch32_ReportHypEntry(except : ExceptionRecord)
begin

    let exceptype : Exception = except.exceptype;

    var (ec,il) : (integer, bit) = AArch32_ExceptionClass(exceptype);
    let iss  : bits(25)= except.syndrome.iss;
    let iss2 : bits(24)= except.syndrome.iss2;

    // IL is not valid for Data Abort exceptions without valid instruction syndrome information
    if ec IN {0x24,0x25} && iss[24] == '0' then
        il = '1';
    end;

    HSR() = ec[5:0]::il::iss;

    if exceptype IN {Exception_InstructionAbort, Exception_PCAlignment} then
        HIFAR() = except.vaddress[31:0];
        HDFAR() = ARBITRARY : bits(32);
    elsif exceptype == Exception_DataAbort then
        HIFAR() = ARBITRARY : bits(32);
        HDFAR() = except.vaddress[31:0];
    end;

    if except.ipavalid then
        HPFAR()[31:4] = except.ipaddress[39:12];
    else
        HPFAR()[31:4] = ARBITRARY : bits(28);
    end;

    return;
end;
```

Library pseudocode for aarch32/exceptions/exceptions/AArch32_ResetControlRegisters

```
// AArch32_ResetControlRegisters()
// ===============================
// Resets System registers and memory-mapped control registers that have architecturally-defined
// reset values to those values.

impdef func AArch32_ResetControlRegisters(cold_reset : boolean)
begin
    return;
end;
```

Library pseudocode for aarch32/exceptions/exceptions/AArch32_TakeReset

```
// AArch32_TakeReset()
// ===================
// Reset into AArch32 state

func AArch32_TakeReset(cold_reset : boolean)
begin
    assert !HaveAArch64();

    // Enter the highest implemented Exception level in AArch32 state
    if HaveEL(EL3) then
        AArch32_WriteMode(M32_Svc);
        SCR().NS = '0';                                 // Secure state
    elsif HaveEL(EL2) then
        AArch32_WriteMode(M32_Hyp);
    else
        AArch32_WriteMode(M32_Svc);
    end;

    // Reset System registers in the coproc=0b111x encoding space
    // and other system components
    AArch32_ResetControlRegisters(cold_reset);
    FPEXC().EN = '0';

    // Reset all other PSTATE fields, including instruction set and endianness according to the
    // SCTLR values produced by the above call to ResetControlRegisters()
    PSTATE.[A,I,F] = '111';             // All asynchronous exceptions masked
    PSTATE.IT = '00000000';             // IT block state reset
    if HaveEL(EL2) && !HaveEL(EL3) then
        PSTATE.T = HSCTLR().TE;           // Instruction set: TE=0:A32, TE=1:T32. PSTATE.J is RES0.
        PSTATE.E = HSCTLR().EE;           // Endianness: EE=0: little-endian, EE=1: big-endian.
    else
        PSTATE.T = SCTLR().TE;            // Instruction set: TE=0:A32, TE=1:T32. PSTATE.J is RES0.
        PSTATE.E = SCTLR().EE;            // Endianness: EE=0: little-endian, EE=1: big-endian.
    end;
    PSTATE.IL = '0';                    // Clear Illegal Execution state bit

    // All registers, bits and fields not reset by the above pseudocode or by the BranchTo() call
    // below are UNKNOWN bitstrings after reset. In particular, the return information registers
    // R14 or ELR_hyp and SPSR have UNKNOWN values, so that it
    // is impossible to return from a reset in an architecturally defined way.
    AArch32_ResetGeneralRegisters();
    if IsFeatureImplemented(FEAT_SME) || IsFeatureImplemented(FEAT_SVE) then
        ResetSVERegisters();
    else
        AArch32_ResetSIMDFPRegisters();
    end;
    AArch32_ResetSpecialRegisters();
    ResetExternalDebugRegisters(cold_reset);


    var rv : bits(32);                      // IMPLEMENTATION DEFINED reset vector
    if HaveEL(EL3) then
        if MVBAR()[0] == '1' then           // Reset vector in MVBAR
            rv = MVBAR()[31:1]::'0';
        else
            rv = ImpDefBits{32}("reset vector address");
        end;
    else
        rv = RVBAR()[31:1]::'0';
    end;

    // The reset vector must be correctly aligned
    assert rv[0] == '0' && (PSTATE.T == '1' || rv[1] == '0');

    let branch_conditional : boolean = FALSE;
    EDPRSR().R = '0';                 // Leaving Reset State.
    BranchTo{32}(rv, BranchType_RESET, branch_conditional);
end;
```

Library pseudocode for aarch32/exceptions/exceptions/ExcVectorBase

```
// ExcVectorBase()
// ===============

func ExcVectorBase() => bits(32)
begin
    if SCTLR().V == '1' then  // Hivecs selected, base = 0xFFFF0000
        return Ones{16}::Zeros{16};
    else
        return VBAR()[31:5]::Zeros{5};
    end;
end;
```

Library pseudocode for aarch32/exceptions/ieeefp/AArch32_FPTrappedException

```
// AArch32_FPTrappedException()
// ============================

func AArch32_FPTrappedException(accumulated_exceptions : bits(8))
begin
    if AArch32_GeneralExceptionsToAArch64() then
        let is_ase : boolean = FALSE;
        let element : integer = 0;
        AArch64_FPTrappedException(is_ase, accumulated_exceptions);
    end;
    FPEXC().DEX    = '1';
    FPEXC().TFV    = '1';
    FPEXC()[7,4:0] = accumulated_exceptions[7,4:0];                  // IDF,IXF,UFF,OFF,DZF,IOF
    FPEXC()[10:8]  = '111';                                          // VECITR is RES1

    AArch32_TakeUndefInstrException();
end;
```

Library pseudocode for aarch32/exceptions/syscalls/AArch32_CallHypervisor

```
// AArch32_CallHypervisor()
// ========================
// Performs a HVC call

func AArch32_CallHypervisor(immediate : bits(16))
begin
    assert HaveEL(EL2);

    if !ELUsingAArch32(EL2) then
        AArch64_CallHypervisor(immediate);
    else
        AArch32_TakeHVCException(immediate);
    end;
end;
```

Library pseudocode for aarch32/exceptions/syscalls/AArch32_CallSupervisor

```
// AArch32_CallSupervisor()
// ========================
// Calls the Supervisor

func AArch32_CallSupervisor(immediate_in : bits(16))
begin
    var immediate : bits(16) = immediate_in;
    if CurrentCond() != '1110' then
        immediate = ARBITRARY : bits(16);
    end;
    if AArch32_GeneralExceptionsToAArch64() then
        AArch64_CallSupervisor(immediate);
    else
        AArch32_TakeSVCException(immediate);
    end;
end;
```

Library pseudocode for aarch32/exceptions/syscalls/AArch32_TakeHVCException

```
// AArch32_TakeHVCException()
// ==========================

func AArch32_TakeHVCException(immediate : bits(16))
begin
    assert HaveEL(EL2) && ELUsingAArch32(EL2);

    AArch32_ITAdvance();
    SSAdvance();
    let preferred_exception_return : bits(32) = NextInstrAddr{}();
    let vect_offset : integer{} = 0x08;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_HypervisorCall);
    except.syndrome.iss[15:0] = immediate;

    if PSTATE.EL == EL2 then
        AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
    else
        AArch32_EnterHypMode(except, preferred_exception_return, 0x14);
    end;
end;
```

Library pseudocode for aarch32/exceptions/syscalls/AArch32_TakeSMCException

```
// AArch32_TakeSMCException()
// ==========================

func AArch32_TakeSMCException()
begin
    assert HaveEL(EL3) && ELUsingAArch32(EL3);
    AArch32_ITAdvance();
    HSAdvance();
    SSAdvance();
    let preferred_exception_return : bits(32) = NextInstrAddr{}();
    let vect_offset : integer = 0x08;
    let lr_offset : integer = 0;
    AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
end;
```

Library pseudocode for aarch32/exceptions/syscalls/AArch32_TakeSVCException

```
// AArch32_TakeSVCException()
// ==========================

func AArch32_TakeSVCException(immediate : bits(16))
begin

    AArch32_ITAdvance();
    SSAdvance();
    let route_to_hyp : boolean = PSTATE.EL == EL0 && EL2Enabled() && HCR().TGE == '1';

    let preferred_exception_return : bits(32) = NextInstrAddr{}();
    let vect_offset : integer{} = 0x08;
    let lr_offset : integer{} = 0;

    if PSTATE.EL == EL2 || route_to_hyp then
        var except : ExceptionRecord = ExceptionSyndrome(Exception_SupervisorCall);
        except.syndrome.iss[15:0] = immediate;
        if PSTATE.EL == EL2 then
            AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
        else
            AArch32_EnterHypMode(except, preferred_exception_return, 0x14);
        end;
    else
        AArch32_EnterMode(M32_Svc, preferred_exception_return, lr_offset, vect_offset);
    end;
end;
```

Library pseudocode for aarch32/exceptions/takeexception/AArch32_EnterHypMode

```
// AArch32_EnterHypMode()
// ======================
// Take an exception to Hyp mode.

noreturn func AArch32_EnterHypMode(except : ExceptionRecord, preferred_exception_return : bits(32),
                                   vect_offset : integer)
begin
    SynchronizeContext();
    assert HaveEL(EL2) && CurrentSecurityState() == SS_NonSecure && ELUsingAArch32(EL2);

    if Halted() then
        AArch32_EnterHypModeInDebugState(except);
    end;
    let spsr : bits(32) = GetPSRFromPSTATE{}(AArch32_NonDebugState);
    if ! except.exceptype IN {Exception_IRQ, Exception_FIQ} then
        AArch32_ReportHypEntry(except);
    end;
    AArch32_WriteMode(M32_Hyp);
    SPSR_curr() = spsr;
    ELR_hyp() = preferred_exception_return;
    PSTATE.T  = HSCTLR().TE;                        // PSTATE.J is RES0
    PSTATE.SS = '0';
    if !HaveEL(EL3) then
        PSTATE.A = '1';
        PSTATE.I = '1';
        PSTATE.F = '1';
    else
        if ELUsingAArch32(EL3) then
            if SCR().EA  == '0' then PSTATE.A = '1'; end;
            if SCR().IRQ == '0' then PSTATE.I = '1'; end;
            if SCR().FIQ == '0' then PSTATE.F = '1'; end;
        else
            if SCR_EL3().EA  == '0' then PSTATE.A = '1'; end;
            if EffectiveSCR_EL3_IRQ() == '0' then PSTATE.I = '1'; end;
            if EffectiveSCR_EL3_FIQ() == '0' then PSTATE.F = '1'; end;
        end;
    end;
    PSTATE.E  = HSCTLR().EE;
    PSTATE.IL = '0';
    PSTATE.IT = '00000000';
    if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = HSCTLR().DSSBS; end;
    let branch_conditional : boolean = FALSE;
    BranchTo{32}(HVBAR()[31:5]::vect_offset[4:0], BranchType_EXCEPTION, branch_conditional);

    CheckExceptionCatch(TRUE);                  // Check for debug event on exception entry

    EndOfInstruction(EndOfInstructionReason_Exception);
end;
```

Library pseudocode for aarch32/exceptions/takeexception/AArch32_EnterMode

```
// AArch32_EnterMode()
// ===================
// Take an exception to a mode other than Monitor and Hyp mode.

noreturn func AArch32_EnterMode(target_mode : bits(5),
                                preferred_exception_return : bits(32),
                                lr_offset : integer,
                                vect_offset : integer)
begin
    SynchronizeContext();
    assert ELUsingAArch32(EL1) && PSTATE.EL != EL2;

    if Halted() then
        AArch32_EnterModeInDebugState(target_mode);
    end;
    let spsr : bits(32) = GetPSRFromPSTATE{}(AArch32_NonDebugState);
    if PSTATE.M == M32_Monitor then SCR().NS = '0'; end;
    AArch32_WriteMode(target_mode);
    SPSR_curr() = spsr;
    R(14) = preferred_exception_return + lr_offset;
    PSTATE.T = SCTLR().TE;                          // PSTATE.J is RES0
    PSTATE.SS = '0';
    if target_mode == M32_FIQ then
        PSTATE.[A,I,F] = '111';
    elsif target_mode IN {M32_Abort, M32_IRQ} then
        PSTATE.[A,I] = '11';
    else
        PSTATE.I = '1';
    end;
    PSTATE.E = SCTLR().EE;
    PSTATE.IL = '0';
    PSTATE.IT = '00000000';
    if IsFeatureImplemented(FEAT_PAN) && SCTLR().SPAN == '0' then PSTATE.PAN = '1'; end;
    if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = SCTLR().DSSBS; end;
    let branch_conditional : boolean = FALSE;
    BranchTo{32}(ExcVectorBase()[31:5]::vect_offset[4:0], BranchType_EXCEPTION, branch_conditional);

    CheckExceptionCatch(TRUE);                  // Check for debug event on exception entry

    EndOfInstruction(EndOfInstructionReason_Exception);
end;
```

Library pseudocode for aarch32/exceptions/takeexception/AArch32_EnterMonitorMode

```
// AArch32_EnterMonitorMode()
// ==========================
// Take an exception to Monitor mode.

func AArch32_EnterMonitorMode(preferred_exception_return : bits(32), lr_offset : integer,
                              vect_offset : integer)
begin
    SynchronizeContext();
    assert HaveEL(EL3) && ELUsingAArch32(EL3);
    let from_secure : boolean = CurrentSecurityState() == SS_Secure;
    if Halted() then
        AArch32_EnterMonitorModeInDebugState();
    end;
    let spsr : bits(32) = GetPSRFromPSTATE{}(AArch32_NonDebugState);
    if PSTATE.M == M32_Monitor then SCR().NS = '0'; end;
    AArch32_WriteMode(M32_Monitor);
    SPSR_curr() = spsr;
    R(14) = preferred_exception_return + lr_offset;
    PSTATE.T       = SCTLR().TE;                    // PSTATE.J is RES0
    PSTATE.SS      = '0';
    PSTATE.[A,I,F] = '111';
    PSTATE.E       = SCTLR().EE;
    PSTATE.IL      = '0';
    PSTATE.IT      = '00000000';
    if IsFeatureImplemented(FEAT_PAN) then
        if !from_secure then
            PSTATE.PAN = '0';
        elsif SCTLR().SPAN == '0' then
            PSTATE.PAN = '1';
        end;
    end;
    if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = SCTLR().DSSBS; end;
    let branch_conditional : boolean = FALSE;
    BranchTo{32}(MVBAR()[31:5]::vect_offset[4:0], BranchType_EXCEPTION, branch_conditional);

    CheckExceptionCatch(TRUE);                  // Check for debug event on exception entry

    EndOfInstruction(EndOfInstructionReason_Exception);
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_CheckAdvSIMDOrFPEnabled

```
// AArch32_CheckAdvSIMDOrFPEnabled()
// =================================
// Check against CPACR, FPEXC, HCPTR, NSACR, and CPTR_EL3.

func AArch32_CheckAdvSIMDOrFPEnabled(fpexc_check_in : boolean, advsimd : boolean)
begin
    var fpexc_check : boolean = fpexc_check_in;
    if PSTATE.EL == EL0 && !ELUsingAArch32(EL1) then
        // When executing at EL0 using AArch32, if EL1 is using AArch64 then the Effective value of
        // FPEXC.EN is 1. This includes when EL2 is using AArch64 and enabled in the current
        // Security state, HCR_EL2.TGE is 1, and the Effective value of HCR_EL2.RW is 1.
        AArch64_CheckFPAdvSIMDEnabled();
    else
        var cpacr_asedis = CPACR().ASEDIS;
        var cpacr_cp10 = CPACR().cp10;

        if HaveEL(EL3) && ELUsingAArch32(EL3) && CurrentSecurityState() == SS_NonSecure then
            // Check if access disabled in NSACR
            if NSACR().NSASEDIS == '1' then cpacr_asedis = '1'; end;
            if NSACR().cp10 == '0' then cpacr_cp10 = '00'; end;
        end;

        if PSTATE.EL != EL2 then
            // Check if Advanced SIMD disabled in CPACR
            if advsimd && cpacr_asedis == '1' then AArch32_Undefined(); end;

            // Check if access disabled in CPACR
            var disabled : boolean;
            case cpacr_cp10 of
                when '00' => disabled = TRUE;
                when '01' => disabled = PSTATE.EL == EL0;
                when '10' => disabled = ConstrainUnpredictableBool(Unpredictable_RESCPACR);
                when '11' => disabled = FALSE;
            end;
            if disabled then AArch32_Undefined(); end;
        end;

        // If required, check FPEXC enabled bit.
        if (fpexc_check && PSTATE.EL == EL0 && EL2Enabled() && !ELUsingAArch32(EL2) &&
              HCR_EL2().TGE == '1') then
            // When executing at EL0 using AArch32, if EL2 is using AArch64 and enabled in the
            // current Security state, HCR_EL2.TGE is 1, and the Effective value of HCR_EL2.RW is 0
            // then it is IMPLEMENTATION DEFINED whether the Effective value of FPEXC.EN is 1
            // or the value of FPEXC32_EL2.EN.
            fpexc_check = ImpDefBool("Use FPEXC32_EL2.EN value when {TGE,RW} == {1,0}");
        end;

        if fpexc_check && FPEXC().EN == '0' then
            AArch32_Undefined();
        end;

        AArch32_CheckFPAdvSIMDTrap(advsimd);    // Also check against HCPTR and CPTR_EL3
    end;
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_CheckFPAdvSIMDTrap

```
// AArch32_CheckFPAdvSIMDTrap()
// ============================
// Check against CPTR_EL2 and CPTR_EL3.

func AArch32_CheckFPAdvSIMDTrap(advsimd : boolean)
begin
    if EL2Enabled() && !ELUsingAArch32(EL2) then
        AArch64_CheckFPAdvSIMDTrap();
    else
        if (HaveEL(EL3) && !ELUsingAArch32(EL3) &&
            CPTR_EL3().TFP == '1' && EL3SDDUndefPriority()) then
            Undefined();
        end;

        let ss : SecurityState = CurrentSecurityState();
        if HaveEL(EL2) && ss != SS_Secure then
            var hcptr_tase = HCPTR().TASE;
            var hcptr_cp10 = HCPTR().TCP10;

            if HaveEL(EL3) && ELUsingAArch32(EL3) then
                // Check if access disabled in NSACR
                if NSACR().NSASEDIS == '1' then hcptr_tase = '1'; end;
                if NSACR().cp10 == '0' then hcptr_cp10 = '1'; end;
            end;

            // Check if access disabled in HCPTR
            if (advsimd && hcptr_tase == '1') || hcptr_cp10 == '1' then
                var except : ExceptionRecord = ExceptionSyndrome(Exception_AdvSIMDFPAccessTrap);
                except.syndrome.iss[24:20] = ConditionSyndrome();

                if advsimd then
                    except.syndrome.iss[5] = '1';
                else
                    except.syndrome.iss[5]   = '0';
                    except.syndrome.iss[3:0] = '1010';         // coproc field, always 0xA
                end;

                if PSTATE.EL == EL2 then
                    AArch32_TakeUndefInstrException(except);
                else
                    AArch32_TakeHypTrapException(except);
                end;
            end;
        end;

        if HaveEL(EL3) && !ELUsingAArch32(EL3) then
            // Check if access disabled in CPTR_EL3
            if CPTR_EL3().TFP == '1' then
                if EL3SDDUndef() then
                    Undefined();
                else
                    AArch64_AdvSIMDFPAccessTrap(EL3);
                end;
            end;
        end;
    end;
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_CheckForSMCUndefOrTrap

```
// AArch32_CheckForSMCUndefOrTrap()
// ================================
// Check for UNDEFINED or trap on SMC instruction

func AArch32_CheckForSMCUndefOrTrap()
begin
    if !HaveEL(EL3) || PSTATE.EL == EL0 then
        Undefined();
    end;

    if EL2Enabled() && !ELUsingAArch32(EL2) then
        AArch64_CheckForSMCUndefOrTrap(Zeros{16});
    else
        let route_to_hyp : boolean = EL2Enabled() && PSTATE.EL == EL1 && HCR().TSC == '1';
        if route_to_hyp then
            let except : ExceptionRecord = ExceptionSyndrome(Exception_MonitorCall);
            AArch32_TakeHypTrapException(except);
        end;
    end;
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_CheckForSVCTrap

```
// AArch32_CheckForSVCTrap()
// =========================
// Check for trap on SVC instruction

func AArch32_CheckForSVCTrap(immediate : bits(16))
begin
    if IsFeatureImplemented(FEAT_FGT) then
        let route_to_el2 : boolean = (PSTATE.EL == EL0 && !ELUsingAArch32(EL1) && EL2Enabled() &&
                                      HFGITR_EL2().SVC_EL0 == '1' &&
                                      (!IsInHost() && (!HaveEL(EL3) || SCR_EL3().FGTEn == '1')));

        if route_to_el2 then
            var except : ExceptionRecord = ExceptionSyndrome(Exception_SupervisorCall);
            except.syndrome.iss[15:0] = immediate;
            except.trappedsyscallinst = TRUE;
            let preferred_exception_return : bits(64) = ThisInstrAddr{}();
            let vect_offset : integer = 0x0;

            AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
        end;
    end;
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_CheckForWFxTrap

```
// AArch32_CheckForWFxTrap()
// =========================
// Check for trap on WFE or WFI instruction

func AArch32_CheckForWFxTrap(target_el : bits(2), wfxtype : WFxType)
begin
    assert HaveEL(target_el);

    // Check for routing to AArch64
    if !ELUsingAArch32(target_el) then
        var trap : boolean;
        let is_wfe : boolean = wfxtype == WFxType_WFE;
        case target_el of
            when EL1 =>
                trap = (if is_wfe then SCTLR_ELx().nTWE else SCTLR_ELx().nTWI) == '0';
            when EL2 =>
                trap = (if is_wfe then HCR_EL2().TWE else HCR_EL2().TWI) == '1';
            when EL3 =>
                trap = (if is_wfe then SCR_EL3().TWE else SCR_EL3().TWI) == '1';
        end;

        if trap then
            if target_el == EL3 && EL3SDDUndef() then
                Undefined();
            else
                AArch64_WFxTrap(wfxtype, target_el);
            end;
        end;
        return;
    end;

    let is_wfe : boolean = wfxtype == WFxType_WFE;
    var trap : boolean;
    case target_el of
        when EL1 =>
            trap = (if is_wfe then SCTLR().nTWE else SCTLR().nTWI) == '0';
        when EL2 =>
            trap = (if is_wfe then HCR().TWE else HCR().TWI) == '1';
        when EL3 =>
            trap = (if is_wfe then SCR().TWE else SCR().TWI) == '1';
    end;

    if trap then
        if target_el == EL1 && EL2Enabled() && !ELUsingAArch32(EL2) && HCR_EL2().TGE == '1' then
            AArch64_WFxTrap(wfxtype, target_el);
        end;

        if target_el == EL3 && !EL3SDDUndef() then
            AArch32_TakeMonitorTrapException();
        elsif target_el == EL2 then
            var except : ExceptionRecord = ExceptionSyndrome(Exception_WFxTrap);
            except.syndrome.iss[24:20]   = ConditionSyndrome();
            except.syndrome.iss[0]       = if wfxtype == WFxType_WFI then '0' else '1';
            AArch32_TakeHypTrapException(except);
        else
            AArch32_TakeUndefInstrException();
        end;
    end;
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_CheckITEnabled

```
// AArch32_CheckITEnabled()
// ========================
// Check whether the T32 IT instruction is disabled.

func AArch32_CheckITEnabled(mask : bits(4))
begin
    var it_disabled : bit;

    if PSTATE.EL == EL2 then
        it_disabled = HSCTLR().ITD;
    else
        it_disabled = (if ELUsingAArch32(EL1) then SCTLR().ITD else SCTLR_ELx().ITD);
    end;
    if it_disabled == '1' then
        if mask != '1000' then Undefined(); end;

        let accdesc : AccessDescriptor = CreateAccDescIFetch();
        let aligned = TRUE;
        // Otherwise whether the IT block is allowed depends on hw1 of the next instruction.
        let next_instr = AArch32_MemSingle{16}(NextInstrAddr{32}(), accdesc, aligned);

        if next_instr IN {'11xxxxxxxxxxxxxx', '1011xxxxxxxxxxxx', '10100xxxxxxxxxxx',
                          '01001xxxxxxxxxxx', '010001xxx1111xxx', '010001xx1xxxx111'} then
            // It is IMPLEMENTATION DEFINED whether the Undefined Instruction exception is
            // taken on the IT instruction or the next instruction. This is not reflected in
            // the pseudocode, which always takes the exception on the IT instruction. This
            // also does not take into account cases where the next instruction is UNPREDICTABLE.
            Undefined();
        end;
    end;

    return;
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_CheckIllegalState

```
// AArch32_CheckIllegalState()
// ===========================
// Check PSTATE.IL bit and generate Illegal Execution state exception if set.

func AArch32_CheckIllegalState()
begin
    if AArch32_GeneralExceptionsToAArch64() then
        AArch64_CheckIllegalState();
    elsif PSTATE.IL == '1' then
        let route_to_hyp = PSTATE.EL == EL0 && EL2Enabled() && HCR().TGE == '1';

        let preferred_exception_return : bits(32) = ThisInstrAddr{}();
        let vect_offset : integer = 0x04;

        if PSTATE.EL == EL2 || route_to_hyp then
            let except : ExceptionRecord = ExceptionSyndrome(Exception_IllegalState);
            if PSTATE.EL == EL2 then
                AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
            else
                AArch32_EnterHypMode(except, preferred_exception_return, 0x14);
            end;
        else
            AArch32_TakeUndefInstrException();
        end;
    end;
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_CheckSETENDEnabled

```
// AArch32_CheckSETENDEnabled()
// ============================
// Check whether the AArch32 SETEND instruction is disabled.

func AArch32_CheckSETENDEnabled()
begin
    var setend_disabled : bit;
    if PSTATE.EL == EL2 then
        setend_disabled = HSCTLR().SED;
    else
        setend_disabled = (if ELUsingAArch32(EL1) then SCTLR().SED else SCTLR_ELx().SED);
    end;
    if setend_disabled == '1' then
        Undefined();
    end;

    return;
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_SystemAccessTrap

```
// AArch32_SystemAccessTrap()
// ==========================
// Trapped AArch32 System register access.

func AArch32_SystemAccessTrap(mode : bits(5), ec : integer)
begin
    let (valid, target_el) = ELFromM32(mode);
    assert valid && HaveEL(target_el) && target_el != EL0 && UInt(target_el) >= UInt(PSTATE.EL);

    if target_el == EL2 then
        let except : ExceptionRecord = AArch32_SystemAccessTrapSyndrome(ThisInstr(), ec);
        AArch32_TakeHypTrapException(except);
    else
        AArch32_TakeUndefInstrException();
    end;
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_SystemAccessTrapSyndrome

```
// AArch32_SystemAccessTrapSyndrome()
// ==================================
// Returns the syndrome information for traps on AArch32 MCR, MCRR, MRC, MRRC, and VMRS,
// VMSR instructions, other than traps that are due to HCPTR or CPACR.

func AArch32_SystemAccessTrapSyndrome(instr : bits(32), ec : integer) => ExceptionRecord
begin
    var except : ExceptionRecord;

    case ec of
        when 0x0  =>   except = ExceptionSyndrome(Exception_Uncategorized);
        when 0x3  =>   except = ExceptionSyndrome(Exception_CP15RTTrap);
        when 0x4  =>   except = ExceptionSyndrome(Exception_CP15RRTTrap);
        when 0x5  =>   except = ExceptionSyndrome(Exception_CP14RTTrap);
        when 0x6  =>   except = ExceptionSyndrome(Exception_CP14DTTrap);
        when 0x7  =>   except = ExceptionSyndrome(Exception_AdvSIMDFPAccessTrap);
        when 0x8  =>   except = ExceptionSyndrome(Exception_FPIDTrap);
        when 0xC  =>   except = ExceptionSyndrome(Exception_CP14RRTTrap);
        otherwise =>   unreachable;
    end;

    var iss : bits(20) = Zeros{};

    if except.exceptype == Exception_Uncategorized then
        return except;
    elsif except.exceptype IN {Exception_FPIDTrap, Exception_CP14RTTrap,
                               Exception_CP15RTTrap} then
        // Trapped MRC/MCR, VMRS on FPSID
        iss[13:10] = instr[19:16];       // CRn, Reg in case of VMRS
        iss[8:5]   = instr[15:12];       // Rt
        iss[9]     = '0';                // RES0

        if except.exceptype != Exception_FPIDTrap then    // When trap is not for VMRS
            iss[19:17] = instr[7:5];     // opc2
            iss[16:14] = instr[23:21];   // opc1
            iss[4:1]   = instr[3:0];     // CRm
        else //VMRS Access
            iss[19:17] = '000';          // opc2 - Hardcoded for VMRS
            iss[16:14] = '111';          // opc1 - Hardcoded for VMRS
            iss[4:1]   = '0000';         // CRm  - Hardcoded for VMRS
        end;
    elsif except.exceptype IN {Exception_CP14RRTTrap, Exception_AdvSIMDFPAccessTrap,
                               Exception_CP15RRTTrap} then
        // Trapped MRRC/MCRR, VMRS/VMSR
        iss[19:16] = instr[7:4];         // opc1
        iss[13:10] = instr[19:16];       // Rt2
        iss[8:5]   = instr[15:12];       // Rt
        iss[4:1]   = instr[3:0];         // CRm
    elsif except.exceptype == Exception_CP14DTTrap then
        // Trapped LDC/STC
        iss[19:12] = instr[7:0];         // imm8
        iss[4]     = instr[23];          // U
        iss[2:1]   = instr[24,21];       // P, W
        if instr[19:16] == '1111' then   // Rn==15, LDC(Literal addressing)/STC
            iss[8:5] = ARBITRARY : bits(4);
            iss[3]   = '1';
        end;
    end;
    iss[0] = instr[20];                  // Direction

    except.syndrome.iss[24:20] = ConditionSyndrome();
    except.syndrome.iss[19:0]  = iss;

    return except;
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_TakeHypTrapException

```
// AArch32_TakeHypTrapException()
// ==============================
// Exceptions routed to Hyp mode as a Hyp Trap exception.

func AArch32_TakeHypTrapException(ec : integer)
begin
    let except : ExceptionRecord = AArch32_SystemAccessTrapSyndrome(ThisInstr(), ec);
    AArch32_TakeHypTrapException(except);
end;

// AArch32_TakeHypTrapException()
// ==============================
// Exceptions routed to Hyp mode as a Hyp Trap exception.

func AArch32_TakeHypTrapException(except : ExceptionRecord)
begin
    assert HaveEL(EL2) && CurrentSecurityState() == SS_NonSecure && ELUsingAArch32(EL2);

    let preferred_exception_return : bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x14;

    AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_TakeMonitorTrapException

```
// AArch32_TakeMonitorTrapException()
// ==================================
// Exceptions routed to Monitor mode as a Monitor Trap exception.

func AArch32_TakeMonitorTrapException()
begin
    assert HaveEL(EL3) && ELUsingAArch32(EL3);

    let preferred_exception_return :  bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x04;
    let lr_offset : integer = if CurrentInstrSet() == InstrSet_A32 then 4 else 2;
    AArch32_EnterMonitorMode(preferred_exception_return, lr_offset, vect_offset);
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_TakeUndefInstrException

```
// AArch32_TakeUndefInstrException()
// =================================

noreturn func AArch32_TakeUndefInstrException(except : ExceptionRecord)
begin

    let route_to_hyp : boolean = PSTATE.EL == EL0 && EL2Enabled() && HCR().TGE == '1';
    let preferred_exception_return : bits(32) = ThisInstrAddr{}();
    let vect_offset : integer = 0x04;
    let lr_offset : integer = if CurrentInstrSet() == InstrSet_A32 then 4 else 2;

    if PSTATE.EL == EL2 then
        AArch32_EnterHypMode(except, preferred_exception_return, vect_offset);
    elsif route_to_hyp then
        AArch32_EnterHypMode(except, preferred_exception_return, 0x14);
    else
        AArch32_EnterMode(M32_Undef, preferred_exception_return, lr_offset, vect_offset);
    end;
end;

// AArch32_TakeUndefInstrException()
// =================================

noreturn func AArch32_TakeUndefInstrException()
begin
    let except : ExceptionRecord = ExceptionSyndrome(Exception_Uncategorized);
    AArch32_TakeUndefInstrException(except);
end;
```

Library pseudocode for aarch32/exceptions/traps/AArch32_Undefined

```
// AArch32_Undefined()
// ===================

noreturn func AArch32_Undefined()
begin

    if AArch32_GeneralExceptionsToAArch64() then AArch64_Undefined(); end;
    AArch32_TakeUndefInstrException();
end;
```

Library pseudocode for aarch32/functions/aborts/AArch32_DomainValid

```
// AArch32_DomainValid()
// =====================
// Returns TRUE if the Domain is valid for a Short-descriptor translation scheme.

func AArch32_DomainValid(statuscode : Fault, level : integer) => boolean
begin
    assert statuscode != Fault_None;

    case statuscode of
        when Fault_Domain =>
            return TRUE;
        when Fault_Translation,Fault_AccessFlag, Fault_SyncExternalOnWalk, Fault_SyncParityOnWalk =>
            return level == 2;
        otherwise =>
            return FALSE;
    end;
end;
```

Library pseudocode for aarch32/functions/aborts/AArch32_FaultSyndrome

```
// AArch32_FaultSyndrome()
// =======================
// Creates an exception syndrome value and updates the virtual address for Abort and Watchpoint
// exceptions taken to AArch32 Hyp mode.

func AArch32_FaultSyndrome(exceptype : Exception, fault : FaultRecord) => bits(25)
begin
    assert fault.statuscode != Fault_None;

    var isstype : IssType;
    isstype.iss  = Zeros{25};

    let d_side : boolean = exceptype == Exception_DataAbort;
    if IsFeatureImplemented(FEAT_RAS) && IsAsyncAbort(fault) then
        let errstate : ErrorState = PEErrorState(fault);
        isstype.iss[11:10] = AArch32_EncodeAsyncErrorSyndrome(errstate); // AET
    end;

    if d_side then

        if AArch32_InstructionSyndromeValid(fault) then
            isstype.iss[24:14] = LSInstructionSyndrome();
        end;

        if fault.accessdesc.acctype IN {AccessType_DC, AccessType_IC, AccessType_AT} then
            isstype.iss[8] = '1';
        end;

        if fault.accessdesc.acctype IN {AccessType_DC, AccessType_IC, AccessType_AT} then
            isstype.iss[6] = '1';
        elsif fault.statuscode == Fault_Exclusive then
            isstype.iss[6] = ARBITRARY : bit;
        else
            isstype.iss[6] = if fault.write then '1' else '0';
        end;

    end;
    if IsExternalAbort(fault) then isstype.iss[9] = fault.extflag; end;
    isstype.iss[7] = if fault.s2fs1walk then '1' else '0';
    isstype.iss[5:0] = EncodeLDFSC(fault.statuscode, fault.level);

    return isstype.iss;
end;
```

Library pseudocode for aarch32/functions/aborts/AArch32_InstructionSyndromeValid

```
// AArch32_InstructionSyndromeValid()
// ==================================
// Returns TRUE if ESR_ELx.ISV is '1' for the given Fault.

func AArch32_InstructionSyndromeValid(fault : FaultRecord) => boolean
begin
    if IsSecondStage(fault) && !fault.s2fs1walk then
        return (!IsExternalSyncAbort(fault) ||
               (!IsFeatureImplemented(FEAT_RAS) && IsExternalAbortOnWalk(fault) &&
                  ImpDefBool("ISV on second stage translation table walk")));
    end;

    return FALSE;
end;
```

Library pseudocode for aarch32/functions/aborts/EncodeSDFSC

```
// EncodeSDFSC()
// =============
// Function that gives the Short-descriptor FSR code for different types of Fault

func EncodeSDFSC(statuscode : Fault, level : integer) => bits(5)
begin
    var result : bits(5);
    case statuscode of
        when Fault_AccessFlag =>
            assert level IN {1,2};
            result = if level == 1 then '00011' else '00110';
        when Fault_Alignment =>
            result = '00001';
        when Fault_Permission =>
            assert level IN {1,2};
            result = if level == 1 then '01101' else '01111';
        when Fault_Domain =>
            assert level IN {1,2};
            result = if level == 1 then '01001' else '01011';
        when Fault_Translation =>
            assert level IN {1,2};
            result = if level == 1 then '00101' else '00111';
        when Fault_SyncExternal =>
            result = '01000';
        when Fault_SyncExternalOnWalk =>
            assert level IN {1,2};
            result = if level == 1 then '01100' else '01110';
        when Fault_SyncParity =>
            result = '11001';
        when Fault_SyncParityOnWalk =>
            assert level IN {1,2};
            result = if level == 1 then '11100' else '11110';
        when Fault_AsyncParity =>
            result = '11000';
        when Fault_AsyncExternal =>
            result = '10110';
        when Fault_Debug =>
            result = '00010';
        when Fault_TLBConflict =>
            result = '10000';
        when Fault_Lockdown =>
            result = '10100';   // IMPLEMENTATION DEFINED
        when Fault_Exclusive =>
            result = '10101';   // IMPLEMENTATION DEFINED
        when Fault_ICacheMaint =>
            result = '00100';
        otherwise =>
            unreachable;
    end;

    return result;
end;
```

Library pseudocode for aarch32/functions/common/A32ExpandImm

```
// A32ExpandImm()
// ==============

func A32ExpandImm(imm12 : bits(12)) => bits(32)
begin
    // PSTATE.C argument to following function call does not affect the imm32 result.
    let (imm32, -) = A32ExpandImm_C(imm12, PSTATE.C);

    return imm32;
end;
```

Library pseudocode for aarch32/functions/common/A32ExpandImm_C

```
// A32ExpandImm_C()
// ================

func A32ExpandImm_C(imm12 : bits(12), carry_in : bit) => (bits(32), bit)
begin
    let unrotated_value : bits(32) = ZeroExtend{}(imm12[7:0]);
    let (imm32, carry_out) : (bits(32), bit) = Shift_C{32}(unrotated_value, SRType_ROR,
                                                           2*UInt(imm12[11:8]), carry_in);

    return (imm32, carry_out);
end;
```

Library pseudocode for aarch32/functions/common/DecodeImmShift

```
// DecodeImmShift()
// ================

func DecodeImmShift(srtype : bits(2), imm5 : bits(5)) => (SRType, integer{0..32})
begin
    var shift_t : SRType;
    var shift_n : integer{0..32};
    case srtype of
        when '00' =>
            shift_t = SRType_LSL;  shift_n = UInt(imm5);
        when '01' =>
            shift_t = SRType_LSR;  shift_n = if imm5 == '00000' then 32 else UInt(imm5);
        when '10' =>
            shift_t = SRType_ASR;  shift_n = if imm5 == '00000' then 32 else UInt(imm5);
        when '11' =>
            if imm5 == '00000' then
                shift_t = SRType_RRX;  shift_n = 1;
            else
                shift_t = SRType_ROR;  shift_n = UInt(imm5);
            end;
    end;

    return (shift_t, shift_n);
end;
```

Library pseudocode for aarch32/functions/common/DecodeRegShift

```
// DecodeRegShift()
// ================

func DecodeRegShift(srtype : bits(2)) => SRType
begin
    var shift_t : SRType;
    case srtype of
        when '00' =>  shift_t = SRType_LSL;
        when '01' =>  shift_t = SRType_LSR;
        when '10' =>  shift_t = SRType_ASR;
        when '11' =>  shift_t = SRType_ROR;
    end;
    return shift_t;
end;
```

Library pseudocode for aarch32/functions/common/RRX

```
// RRX()
// =====

func RRX{N}(x : bits(N), carry_in : bit) => bits(N)
begin
    let (result, -) = RRX_C{N}(x, carry_in);
    return result;
end;
```

Library pseudocode for aarch32/functions/common/RRX_C

```
// RRX_C()
// =======

func RRX_C{N}(x : bits(N), carry_in : bit) => (bits(N), bit)
begin
    let result : bits(N) = carry_in :: x[N-1:1];
    let carry_out : bit = x[0];
    return (result, carry_out);
end;
```

Library pseudocode for aarch32/functions/common/SRType

```
// SRType
// ======

type SRType of enumeration {SRType_LSL, SRType_LSR, SRType_ASR, SRType_ROR, SRType_RRX};
```

Library pseudocode for aarch32/functions/common/Shift

```
// Shift()
// =======

func Shift{N}(value : bits(N), srtype : SRType, amount : integer, carry_in : bit) => bits(N)
begin
    let (result, -) = Shift_C{N}(value, srtype, amount, carry_in);
    return result;
end;
```

Library pseudocode for aarch32/functions/common/Shift_C

```
// Shift_C()
// =========

func Shift_C{N}(value : bits(N), srtype : SRType, amount : integer,
                carry_in : bit) => (bits(N), bit)
begin
    assert !(srtype == SRType_RRX && amount != 1);

    var result : bits(N);
    var carry_out : bit;
    if amount == 0 then
        (result, carry_out) = (value, carry_in);
    else
        case srtype of
            when SRType_LSL =>
                (result, carry_out) = LSL_C(value, amount);
            when SRType_LSR =>
                (result, carry_out) = LSR_C(value, amount);
            when SRType_ASR =>
                (result, carry_out) = ASR_C(value, amount);
            when SRType_ROR =>
                (result, carry_out) = ROR_C(value, amount);
            when SRType_RRX =>
                (result, carry_out) = RRX_C{N}(value, carry_in);
        end;
    end;

    return (result, carry_out);
end;
```

Library pseudocode for aarch32/functions/common/T32ExpandImm

```
// T32ExpandImm()
// ==============

func T32ExpandImm(imm12 : bits(12)) => bits(32)
begin
    // PSTATE.C argument to following function call does not affect the imm32 result.
    let (imm32, -) = T32ExpandImm_C(imm12, PSTATE.C);

    return imm32;
end;
```

Library pseudocode for aarch32/functions/common/T32ExpandImm_C

```
// T32ExpandImm_C()
// ================

func T32ExpandImm_C(imm12 : bits(12), carry_in : bit) => (bits(32), bit)
begin
    var imm32 : bits(32);
    var carry_out : bit;
    if imm12[11:10] == '00' then
        case imm12[9:8] of
            when '00' =>
                imm32 = ZeroExtend{32}(imm12[7:0]);
            when '01' =>
                imm32 = '00000000' :: imm12[7:0] :: '00000000' :: imm12[7:0];
            when '10' =>
                imm32 = imm12[7:0] :: '00000000' :: imm12[7:0] :: '00000000';
            when '11' =>
                imm32 = imm12[7:0] :: imm12[7:0] :: imm12[7:0] :: imm12[7:0];
        end;
        carry_out = carry_in;
    else
        let unrotated_value : bits(32) = ZeroExtend{}('1'::imm12[6:0]);
        (imm32, carry_out) = ROR_C(unrotated_value, UInt(imm12[11:7]));
    end;

    return (imm32, carry_out);
end;
```

Library pseudocode for aarch32/functions/common/VBitOps

```
// VBitOps
// =======

type VBitOps of enumeration {VBitOps_VBIF, VBitOps_VBIT, VBitOps_VBSL};
```

Library pseudocode for aarch32/functions/common/VCGEType

```
// VCGEType
// ========

type VCGEType of enumeration {VCGEType_signed, VCGEType_unsigned, VCGEType_fp};
```

Library pseudocode for aarch32/functions/common/VCGTtype

```
// VCGTtype
// ========

type VCGTtype of enumeration {VCGTtype_signed, VCGTtype_unsigned, VCGTtype_fp};
```

Library pseudocode for aarch32/functions/common/VFPNegMul

```
// VFPNegMul
// =========

type VFPNegMul of enumeration {VFPNegMul_VNMLA, VFPNegMul_VNMLS, VFPNegMul_VNMUL};
```

Library pseudocode for aarch32/functions/coproc/AArch32_CheckCP15InstrCoarseTraps

```
// AArch32_CheckCP15InstrCoarseTraps()
// ===================================
// Check for coarse-grained traps to System registers in the
// coproc=0b1111 encoding space by HSTR and HCR.

func AArch32_CheckCP15InstrCoarseTraps(CRn : integer, nreg : integer, CRm : integer)
begin
    if PSTATE.EL == EL0 && (!ELUsingAArch32(EL1) ||
          (EL2Enabled() && !ELUsingAArch32(EL2))) then
        AArch64_CheckCP15InstrCoarseTraps(CRn, nreg, CRm);
    end;

    let trapped_encoding = ((CRn == 9  && CRm IN {0,1,2,    5,6,7,8   }) ||
                            (CRn == 10 && CRm IN {0,1,    4,      8   }) ||
                            (CRn == 11 && CRm IN {0,1,2,3,4,5,6,7,8,15}));

    // Check for coarse-grained Hyp traps
    if PSTATE.EL IN {EL0, EL1} && EL2Enabled() then
        let major : integer = if nreg == 1 then CRn else CRm;
        // Check for MCR, MRC, MCRR, and MRRC disabled by HSTR().CRn/HSTR().CRm
        // and MRC and MCR disabled by HCR.TIDCP.
        if ((! major IN {4,14} && HSTR()[major] == '1') ||
              (HCR().TIDCP == '1' && nreg == 1 && trapped_encoding)) then
            if (PSTATE.EL == EL0 &&
                  ImpDefBool("UNDEF unallocated CP15 access at EL0")) then
                Undefined();
            end;
            if ELUsingAArch32(EL2) then
                AArch32_SystemAccessTrap(M32_Hyp, 0x3);
            else
                AArch64_AArch32SystemAccessTrap(EL2, 0x3);
            end;
        end;
    end;
end;
```

Library pseudocode for aarch32/functions/exclusive/AArch32_ExclusiveMonitorsPass

```
// AArch32_ExclusiveMonitorsPass()
// ===============================
// Return TRUE if the Exclusives monitors for the current PE include all of the addresses
// associated with the virtual address region of size bytes starting at address.
// The immediately following memory write must be to the same addresses.

// It is IMPLEMENTATION DEFINED whether the detection of memory aborts happens
// before or after the check on the local Exclusives monitor. As a result a failure
// of the local monitor can occur on some implementations even if the memory
// access would give a memory abort.


func AArch32_ExclusiveMonitorsPass(address : bits(32),
                                   size : integer{1, 2, 4, 8, 16, 32}) => boolean
begin
    let acqrel : boolean = FALSE;
    let privileged : boolean = PSTATE.EL != EL0;
    let tagchecked : boolean = FALSE;
    let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel,
                                                         tagchecked, privileged);
    let aligned : boolean = IsAlignedSize(address, size);

    if !aligned then
        let fault : FaultRecord = AlignmentFault(accdesc, ZeroExtend{64}(address));
        AArch32_Abort(fault);
    end;

    if !AArch32_IsExclusiveVA(address, ProcessorID(), size) then
        return FALSE;
    end;

    let memaddrdesc : AddressDescriptor = AArch32_TranslateAddress(address, accdesc, aligned, size);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        AArch32_Abort(memaddrdesc.fault);
    end;

    var passed : boolean = IsExclusiveLocal(memaddrdesc.paddress, ProcessorID(), size);
    ClearExclusiveLocal(ProcessorID());

    if passed && memaddrdesc.memattrs.shareability != Shareability_NSH then
        passed = IsExclusiveGlobal(memaddrdesc.paddress, ProcessorID(), size);
    end;

    return passed;
end;
```

Library pseudocode for aarch32/functions/exclusive/AArch32_IsExclusiveVA

```
// AArch32_IsExclusiveVA()
// =======================
// An optional IMPLEMENTATION DEFINED test for an exclusive access to a virtual
// address region of size bytes starting at address.
//
// It is permitted (but not required) for this function to return FALSE and
// cause a store exclusive to fail if the virtual address region is not
// totally included within the region recorded by MarkExclusiveVA().
//
// It is always safe to return TRUE which will check the physical address only.

impdef func AArch32_IsExclusiveVA(address : bits(32),
                                  processorid : integer,
                                  size : integer) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch32/functions/exclusive/AArch32_MarkExclusiveVA

```
// AArch32_MarkExclusiveVA()
// =========================
// Optionally record an exclusive access to the virtual address region of size bytes
// starting at address for processorid.

func AArch32_MarkExclusiveVA(address : bits(32), processorid : integer, size : integer)
begin
    return;
end;
```

Library pseudocode for aarch32/functions/exclusive/AArch32_SetExclusiveMonitors

```
// AArch32_SetExclusiveMonitors()
// ==============================
// Sets the Exclusives monitors for the current PE to record the addresses associated
// with the virtual address region of size bytes starting at address.

func AArch32_SetExclusiveMonitors(address : bits(32), size : integer{1, 2, 4, 8, 16, 32})
begin
    let acqrel : boolean = FALSE;
    let privileged : boolean = PSTATE.EL != EL0;
    let tagchecked : boolean = FALSE;
    let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel,
                                                         tagchecked, privileged);
    let aligned : boolean = IsAlignedSize(address, size);

    if !aligned then
        let fault : FaultRecord = AlignmentFault(accdesc, ZeroExtend{64}(address));
        AArch32_Abort(fault);
    end;

    let memaddrdesc : AddressDescriptor = AArch32_TranslateAddress(address, accdesc, aligned, size);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return;
    end;

    if memaddrdesc.memattrs.shareability != Shareability_NSH then
        MarkExclusiveGlobal(memaddrdesc.paddress, ProcessorID(), size);
    end;

    MarkExclusiveLocal(memaddrdesc.paddress, ProcessorID(), size);

    AArch32_MarkExclusiveVA(address, ProcessorID(), size);
end;
```

Library pseudocode for aarch32/functions/float/CheckAdvSIMDEnabled

```
// CheckAdvSIMDEnabled()
// =====================

func CheckAdvSIMDEnabled()
begin
    let fpexc_check : boolean = TRUE;
    let advsimd : boolean = TRUE;

    AArch32_CheckAdvSIMDOrFPEnabled(fpexc_check, advsimd);
    // Return from CheckAdvSIMDOrFPEnabled() occurs only if Advanced SIMD access is permitted

    // Make temporary copy of D registers
    // _Dclone[] is used as input data for instruction pseudocode
    for i = 0 to 31 do
        _Dclone[[i]] = D(i);
    end;

    return;
end;
```

Library pseudocode for aarch32/functions/float/CheckAdvSIMDOrVFPEnabled

```
// CheckAdvSIMDOrVFPEnabled()
// ==========================

func CheckAdvSIMDOrVFPEnabled(include_fpexc_check : boolean, advsimd : boolean)
begin
    AArch32_CheckAdvSIMDOrFPEnabled(include_fpexc_check, advsimd);
    // Return from CheckAdvSIMDOrFPEnabled() occurs only if VFP access is permitted
    return;
end;
```

Library pseudocode for aarch32/functions/float/CheckCryptoEnabled32

```
// CheckCryptoEnabled32()
// ======================

func CheckCryptoEnabled32()
begin
    CheckAdvSIMDEnabled();
    // Return from CheckAdvSIMDEnabled() occurs only if access is permitted
    return;
end;
```

Library pseudocode for aarch32/functions/float/CheckVFPEnabled

```
// CheckVFPEnabled()
// =================

func CheckVFPEnabled(include_fpexc_check : boolean)
begin
    let advsimd : boolean = FALSE;
    AArch32_CheckAdvSIMDOrFPEnabled(include_fpexc_check, advsimd);
    // Return from CheckAdvSIMDOrFPEnabled() occurs only if VFP access is permitted
    return;
end;
```

Library pseudocode for aarch32/functions/float/FPHalvedSub

```
// FPHalvedSub()
// =============

func FPHalvedSub{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    let rounding : FPRounding = FPRoundingMode(fpcr);
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);
    var (done,result) : (boolean, bits(N)) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr);
    if !done then
        let inf1 : boolean  = (type1 == FPType_Infinity);
        let inf2 : boolean  = (type2 == FPType_Infinity);
        let zero1 : boolean = (type1 == FPType_Zero);
        let zero2 : boolean = (type2 == FPType_Zero);
        if inf1 && inf2 && sign1 == sign2 then
            result = FPDefaultNaN{N}(fpcr);
            FPProcessException(FPExc_InvalidOp, fpcr);
        elsif (inf1 && sign1 == '0') || (inf2 && sign2 == '1') then
            result = FPInfinity{N}('0');
        elsif (inf1 && sign1 == '1') || (inf2 && sign2 == '0') then
            result = FPInfinity{N}('1');
        elsif zero1 && zero2 && sign1 != sign2 then
            result = FPZero{N}(sign1);
        else
            let result_value : real = (value1 - value2) / 2.0;
            if result_value == 0.0 then  // Sign of exact zero result depends on rounding mode
                let result_sign : bit = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{N}(result_sign);
            else
                result = FPRound{N}(result_value, fpcr);
            end;
        end;
    end;
    return result;
end;
```

Library pseudocode for aarch32/functions/float/FPRSqrtStep

```
// FPRSqrtStep()
// =============

func FPRSqrtStep{N}(op1 : bits(N), op2 : bits(N)) => bits(N)
begin
    assert N IN {16,32};
    let fpcr : FPCR_Type = StandardFPCR();
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);
    var (done,result) : (boolean, bits(N)) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr);
    if !done then
        let inf1 : boolean  = (type1 == FPType_Infinity);
        let inf2 : boolean  = (type2 == FPType_Infinity);
        let zero1 : boolean = (type1 == FPType_Zero);
        let zero2 : boolean = (type2 == FPType_Zero);
        var product : bits(N);
        if (inf1 && zero2) || (zero1 && inf2) then
            product = FPZero{N}('0');
        else
            product = FPMul{N}(op1, op2, fpcr);
        end;
        let three : bits(N) = FPThree{}('0');
        result = FPHalvedSub{N}(three, product, fpcr);
    end;
    return result;
end;
```

Library pseudocode for aarch32/functions/float/FPRecipStep

```
// FPRecipStep()
// =============

func FPRecipStep{N}(op1 : bits(N), op2 : bits(N)) => bits(N)
begin
    assert N IN {16,32};
    let fpcr : FPCR_Type = StandardFPCR();
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);
    var (done,result) : (boolean, bits(N)) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr);
    if !done then
        let inf1 : boolean  = (type1 == FPType_Infinity);
        let inf2 : boolean  = (type2 == FPType_Infinity);
        let zero1 : boolean = (type1 == FPType_Zero);
        let zero2 : boolean = (type2 == FPType_Zero);
        var product : bits(N);
        if (inf1 && zero2) || (zero1 && inf2) then
            product = FPZero{N}('0');
        else
            product = FPMul{N}(op1, op2, fpcr);
        end;
        let two : bits(N) = FPTwo{}('0');
        result = FPSub{N}(two, product, fpcr);
    end;
    return result;
end;
```

Library pseudocode for aarch32/functions/float/StandardFPCR

```
// StandardFPCR()
// ==============

func StandardFPCR() => FPCR_Type
begin
    let value : bits(32) = ('00000' :: FPSCR().AHP :: '110000' ::
                            FPSCR().FZ16 :: '0000000000000000000');
    return ZeroExtend{64}(value);
end;
```

Library pseudocode for aarch32/functions/memory/AArch32_MemSingle

```
// AArch32_MemSingle
// =================

accessor AArch32_MemSingle{size : integer{8, 16, 32, 64}}(address : bits(32),
                                                          accdesc : AccessDescriptor,
                                                          aligned : boolean) <=> value : bits(size)
begin
    // Perform an atomic, little-endian read of 'size' bits.
    getter
        let bytes : integer{} = size DIV 8;
        var value : bits(size);
        var memaddrdesc : AddressDescriptor;
        var memstatus : PhysMemRetStatus;

        (value, memaddrdesc, memstatus) = AArch32_MemSingleRead{size}(address, accdesc, aligned);

        // Check for a fault from translation or the output of translation.
        if IsFault(memaddrdesc) then
            AArch32_Abort(memaddrdesc.fault);
        end;

        // Check for external aborts.
        if IsFault(memstatus) then
            HandleExternalAbort(memstatus, accdesc.write, memaddrdesc, bytes, accdesc);
        end;

        return value;
    end;

    // Perform an atomic, little-endian write of 'size' bits.
    setter
        let bytes : integer{} = size DIV 8;
        var memaddrdesc : AddressDescriptor;
        var memstatus : PhysMemRetStatus;

        (memaddrdesc, memstatus) = AArch32_MemSingleWrite{size}(address, accdesc, aligned, value);

        // Check for a fault from translation or the output of translation.
        if IsFault(memaddrdesc) then
            AArch32_Abort(memaddrdesc.fault);
        end;

        // Check for external aborts.
        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, bytes, accdesc);
        end;

        return;
    end;
end;
```

Library pseudocode for aarch32/functions/memory/AArch32_MemSingleRead

```
// AArch32_MemSingleRead()
// =======================
// Perform an atomic, little-endian read of 'size' bits.

func AArch32_MemSingleRead{size : integer{8, 16, 32, 64}}(address : bits(32),
                                                          accdesc_in : AccessDescriptor,
                                                          aligned : boolean
                                                          ) => (bits(size), AddressDescriptor,
                                                                PhysMemRetStatus)
begin
    let bytes : integer{}            = size DIV 8;
    var value : bits(size)           = ARBITRARY : bits(size);
    var memstatus : PhysMemRetStatus = ARBITRARY : PhysMemRetStatus;
    var accdesc : AccessDescriptor = accdesc_in;
    assert IsAlignedSize(address, bytes);

    var memaddrdesc : AddressDescriptor;
    memaddrdesc = AArch32_TranslateAddress(address, accdesc, aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (value, memaddrdesc, memstatus);
    end;

    // Memory array access
    (memstatus, value) = PhysMemRead{size}(memaddrdesc, accdesc);
    if IsFault(memstatus) then
        return (value, memaddrdesc, memstatus);
    end;

    if accdesc.acctype == AccessType_IFETCH then
        if ELUsingAArch32(S1TranslationRegime()) then
            memaddrdesc.fault = AArch32_CheckDebug(address, accdesc, bytes);
        else
            memaddrdesc.fault = AArch64_CheckDebug(ZeroExtend{64}(address), accdesc, bytes);
        end;
    end;

    return (value, memaddrdesc, memstatus);
end;
```

Library pseudocode for aarch32/functions/memory/AArch32_MemSingleWrite

```
// AArch32_MemSingleWrite()
// ========================
// Perform an atomic, little-endian write of 'size' bits.

func AArch32_MemSingleWrite{size : integer{8, 16, 32, 64}}(address : bits(32),
                                                           accdesc_in : AccessDescriptor,
                                                           aligned : boolean,
                                                           value : bits(size)
                                                          ) => (AddressDescriptor, PhysMemRetStatus)
begin
    let bytes : integer{} = size DIV 8;
    var accdesc : AccessDescriptor = accdesc_in;
    assert IsAlignedSize(address, bytes);

    var memaddrdesc : AddressDescriptor;
    var memstatus : PhysMemRetStatus = ARBITRARY : PhysMemRetStatus;
    memaddrdesc = AArch32_TranslateAddress(address, accdesc, aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (memaddrdesc, memstatus);
    end;

    // Effect on exclusives
    if memaddrdesc.memattrs.shareability != Shareability_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), bytes);
    end;

    memstatus = PhysMemWrite{size}(memaddrdesc, accdesc, value);
    if IsFault(memstatus) then
        return (memaddrdesc, memstatus);
    end;

    return (memaddrdesc, memstatus);
end;
```

Library pseudocode for aarch32/functions/memory/AArch32_UnalignedAccessFaults

```
// AArch32_UnalignedAccessFaults()
// ===============================
// Determine whether the unaligned access generates an Alignment fault

func AArch32_UnalignedAccessFaults(accdesc : AccessDescriptor) => boolean
begin
    return (AlignmentEnforced() ||
            accdesc.a32lsmd     ||
            accdesc.exclusive   ||
            accdesc.acqsc       ||
            accdesc.relsc);
end;
```

Library pseudocode for aarch32/functions/memory/Hint_PreloadData

```
// Hint_PreloadData()
// ==================
// Perform a preload data hint.

impdef func Hint_PreloadData(address : bits(32))
begin
    return;
end;
```

Library pseudocode for aarch32/functions/memory/Hint_PreloadDataForWrite

```
// Hint_PreloadDataForWrite()
// ==========================
// Perform a preload data hint with a probability that the use will be for a write.

impdef func Hint_PreloadDataForWrite(address : bits(32))
begin
    return;
end;
```

Library pseudocode for aarch32/functions/memory/Hint_PreloadInstr

```
// Hint_PreloadInstr()
// ===================
// Perform a preload instruction hint.

impdef func Hint_PreloadInstr(address : bits(32))
begin
    return;
end;
```

Library pseudocode for aarch32/functions/memory/MemA

```
// MemA - accessor
// ===============

accessor MemA{size : integer{8, 16, 32, 64}}(address : bits(32)) <=> value : bits(size)
begin
    getter
        let acqrel : boolean = FALSE;
        let privileged : boolean = PSTATE.EL != EL0;
        let tagchecked : boolean = FALSE;
        let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_LOAD, acqrel, tagchecked,
                                                             privileged);
        return Mem_with_type{size}(address, accdesc);
    end;

    setter
        let acqrel : boolean = FALSE;
        let privileged : boolean = PSTATE.EL != EL0;
        let tagchecked : boolean = FALSE;
        let accdesc : AccessDescriptor = CreateAccDescExLDST(MemOp_STORE, acqrel, tagchecked,
                                                             privileged);
        Mem_with_type{size}(address, accdesc) = value;
    end;
end;
```

Library pseudocode for aarch32/functions/memory/MemO

```
// MemO
// ====

accessor MemO{size : integer{8, 16, 32, 64} }(address : bits(32)) <=> value : bits(size)
begin
    getter
        let acquire : boolean = TRUE;
        let tagchecked : boolean = FALSE;
        let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_LOAD, tagchecked, acquire);
        return Mem_with_type{size}(address, accdesc);
    end;

    setter
        let acquire : boolean = FALSE;
        let tagchecked : boolean = FALSE;
        let accdesc : AccessDescriptor = CreateAccDescAcqRel(MemOp_STORE, tagchecked, acquire);
        Mem_with_type{size}(address, accdesc) = value;
    end;
end;
```

Library pseudocode for aarch32/functions/memory/MemS

```
// MemS - accessor
// ===============

accessor MemS{size : integer{8, 16, 32, 64}}(address : bits(32)) <=> value : bits(size)
begin
    // Memory accessor for streaming load multiple instructions
    getter
        let accdesc : AccessDescriptor = CreateAccDescA32LSMD(MemOp_LOAD);
        return Mem_with_type{size}(address, accdesc);
    end;

    // Memory accessor for streaming store multiple instructions
    setter
        let accdesc : AccessDescriptor = CreateAccDescA32LSMD(MemOp_STORE);
        Mem_with_type{size}(address, accdesc) = value;
    end;
end;
```

Library pseudocode for aarch32/functions/memory/MemU

```
// MemU - accessor
// ===============

accessor MemU{size : integer{8, 16, 32, 64} }(address : bits(32) ) <=> value: bits(size)
begin
    getter
        let nontemporal : boolean = FALSE;
        let privileged : boolean = PSTATE.EL != EL0;
        let tagchecked : boolean = FALSE;
        let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                          tagchecked);
        return Mem_with_type{size}(address, accdesc);
    end;

    setter
        let nontemporal : boolean = FALSE;
        let privileged : boolean  = PSTATE.EL != EL0;
        let tagchecked : boolean  = FALSE;
        let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                          tagchecked);
        Mem_with_type{size}(address, accdesc) = value;
    end;
end;
```

Library pseudocode for aarch32/functions/memory/MemU_unpriv

```
// MemU_unpriv
// ===========

accessor MemU_unpriv{size : integer{8, 16, 32, 64}}(address : bits(32)) <=> value : bits(size)
begin
    getter
        let nontemporal : boolean = FALSE;
        let privileged : boolean  = FALSE;
        let tagchecked : boolean  = FALSE;
        let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_LOAD, nontemporal, privileged,
                                                          tagchecked);
        return Mem_with_type{size}(address, accdesc);
    end;

    setter
        let nontemporal : boolean = FALSE;
        let privileged : boolean  = FALSE;
        let tagchecked : boolean  = FALSE;
        let accdesc : AccessDescriptor = CreateAccDescGPR(MemOp_STORE, nontemporal, privileged,
                                                          tagchecked);
        Mem_with_type{size}(address, accdesc) = value;
    end;
end;
```

Library pseudocode for aarch32/functions/memory/Mem_with_type

```
// Mem_with_type
// =============

accessor Mem_with_type{size : integer{8, 16, 32, 64}}(address : bits(32),
                                                      accdesc_in : AccessDescriptor
                                                     ) <=> value_in : bits(size)
begin
    // Perform a read of 'size' bits. The access byte order is reversed for a big-endian access.
    // Instruction fetches would call AArch32_MemSingle directly.
    getter
        let bytes : integer{} = size DIV 8;
        var accdesc : AccessDescriptor = accdesc_in;
        var value : bits(size);

        // Check alignment on size of element accessed, not overall access size
        var aligned : boolean;
        if accdesc.ispair then
            let half : integer{} = (size DIV 2) as integer{32};
            aligned = IsAlignedSize(address, half DIV 8);
        else
            aligned = IsAlignedSize(address, bytes);
        end;

        if !aligned && AArch32_UnalignedAccessFaults(accdesc) then
            let fault : FaultRecord = AlignmentFault(accdesc, ZeroExtend{64}(address));
            AArch32_Abort(fault);
        end;
        if aligned then
            value = AArch32_MemSingle{size}(address, accdesc, aligned);
        else
            assert bytes > 1;
            value[7:0] = AArch32_MemSingle{8}(address, accdesc, aligned);

            // For subsequent bytes, if they cross to a new translation page which assigns
            // Device memory type, it is CONSTRAINED UNPREDICTABLE whether an unaligned access
            // will generate an Alignment Fault.
            let c : Constraint = ConstrainUnpredictable(Unpredictable_DEVPAGE2);
            assert c IN {Constraint_FAULT, Constraint_NONE};
            if c == Constraint_NONE then aligned = TRUE; end;

            for i = 1 to bytes-1 do
                value[i*:8] = AArch32_MemSingle{8}(address+i, accdesc, aligned);
            end;
        end;

        if BigEndian(accdesc.acctype) then
            value = BigEndianReverse{size}(value);
        end;

        return value;
    end;

    // Perform a write of 'size' bits. The byte order is reversed for a big-endian access.
    setter
        let bytes : integer{}          = size DIV 8;
        var value :  bits(size)        = value_in;
        var accdesc : AccessDescriptor = accdesc_in;

        // Check alignment on size of element accessed, not overall access size
        var aligned : boolean;
        if accdesc.ispair then
            let half : integer{} = (size DIV 2) as integer{32};
            aligned = IsAlignedSize(address, half DIV 8);
        else
            aligned = IsAlignedSize(address, bytes);
        end;

        if !aligned && AArch32_UnalignedAccessFaults(accdesc) then
            let fault : FaultRecord = AlignmentFault(accdesc, ZeroExtend{64}(address));
            AArch32_Abort(fault);
        end;

        if BigEndian(accdesc.acctype) then
            value = BigEndianReverse{size}(value);
        end;
        if aligned then
            AArch32_MemSingle{size}(address, accdesc, aligned) = value;
        else
            assert bytes > 1;
            AArch32_MemSingle{8}(address, accdesc, aligned) = value[7:0];

            // For subsequent bytes, if they cross to a new translation page which assigns
            // Device memory type, it is CONSTRAINED UNPREDICTABLE whether an unaligned access
            // will generate an Alignment Fault.

            let c : Constraint = ConstrainUnpredictable(Unpredictable_DEVPAGE2);
            assert c IN {Constraint_FAULT, Constraint_NONE};
            if c == Constraint_NONE then aligned = TRUE; end;
```

```
            for i = 1 to bytes-1 do
                AArch32_MemSingle{8}(address+i, accdesc, aligned) = value[i*:8];
            end;
        end;
        return;
    end;
end;
```

Library pseudocode for aarch32/functions/ras/AArch32_ESBOperation

```
// AArch32_ESBOperation()
// ======================
// Perform the AArch32 ESB operation for ESB executed in AArch32 state.

func AArch32_ESBOperation()
begin
    var masked : boolean;
    var target_el : bits(2);

    (masked, target_el) = PhysicalSErrorTarget();

    // Check if routed to AArch64 state
    if !masked && !ELUsingAArch32(target_el) then
        AArch64_ESBOperation();
        return;
    end;

    // Check for a masked Physical SError pending that can be synchronized
    // by an Error synchronization event.
    if masked && IsSynchronizablePhysicalSErrorPending() then
        var syndrome : bits(32) = Zeros{};
        syndrome[31] = '1';  // A
        syndrome[15:0] = AArch32_PhysicalSErrorSyndrome();
        DISR() = syndrome;
        ClearPendingPhysicalSError();
    end;

    return;
end;
```

Library pseudocode for aarch32/functions/ras/AArch32_EncodeAsyncErrorSyndrome

```
// AArch32_EncodeAsyncErrorSyndrome()
// ==================================
// Return the encoding for specified  ErrorState for an SError exception taken
// to AArch32 state.

func AArch32_EncodeAsyncErrorSyndrome(errorstate : ErrorState) => bits(2)
begin
    case errorstate of
        when ErrorState_UC =>   return '00';
        when ErrorState_UEU =>  return '01';
        when ErrorState_UEO =>  return '10';
        when ErrorState_UER =>  return '11';
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for aarch32/functions/ras/AArch32_PhysicalSErrorSyndrome

```
// AArch32_PhysicalSErrorSyndrome()
// ================================
// Generate SError syndrome.

func AArch32_PhysicalSErrorSyndrome() => bits(16)
begin
    var syndrome : bits(32) = Zeros{};
    let fault : FaultRecord = GetPendingPhysicalSError();
    if PSTATE.EL == EL2 then
        let errstate : ErrorState = PEErrorState(fault);
        syndrome[11:10] = AArch32_EncodeAsyncErrorSyndrome(errstate);   // AET
        syndrome[9]     = fault.extflag;                                // EA
        syndrome[5:0]   = '010001';                                     // DFSC
    else
        let long_format : boolean = TTBCR().EAE == '1';
        syndrome = AArch32_CommonFaultStatus(fault, long_format);
    end;
    return syndrome[15:0];
end;
```

Library pseudocode for aarch32/functions/ras/AArch32_vESBOperation

```
// AArch32_vESBOperation()
// =======================
// Perform the ESB operation for virtual SError interrupts executed in AArch32 state.
// If FEAT_E3DSE is implemented and there is no unmasked virtual SError exception
// pending, then AArch64_dESBOperation() is called to perform the AArch64 ESB operation
// for a pending delegated SError exception.

func AArch32_vESBOperation()
begin
    assert PSTATE.EL IN {EL0, EL1} && EL2Enabled();

    // Check for EL2 using AArch64 state
    if !ELUsingAArch32(EL2) then
        AArch64_vESBOperation();
        return;
    end;

    // If physical SError interrupts are routed to Hyp mode, and TGE is not set, then a virtual
    // SError interrupt might be pending.
    let vsei_pending : boolean = IsVirtualSErrorPending() && HCR().TGE == '0' && HCR().AMO == '1';
    let vsei_masked : boolean  = (PSTATE.A == '1' || Halted() ||
                                  ExternalDebugInterruptsDisabled(EL1));

    // Check for a masked virtual SError pending
    if vsei_pending && vsei_masked then
        var syndrome : bits(32) = Zeros{};
        syndrome[31]    = '1';           // A
        syndrome[15:14] = VDFSR()[15:14];  // AET
        syndrome[12]    = VDFSR()[12];     // ExT
        syndrome[9]     = TTBCR().EAE;     // LPAE
        if TTBCR().EAE == '1' then         // Long-descriptor format
            syndrome[5:0]    = '010001'; // STATUS
        else                               // Short-descriptor format
            syndrome[10,3:0] = '10110';  // FS
        end;
        VDISR() = syndrome;
        ClearPendingVirtualSError();
    elsif IsFeatureImplemented(FEAT_E3DSE) && !ELUsingAArch32(EL3) then
        AArch64_dESBOperation();
    end;

    return;
end;
```

Library pseudocode for aarch32/functions/registers/AArch32_ResetGeneralRegisters

```
// AArch32_ResetGeneralRegisters()
// ===============================

func AArch32_ResetGeneralRegisters()
begin

    for i = 0 to 7 do
        R(i) = ARBITRARY : bits(32);
    end;
    for i = 8 to 12 do
        Rmode(i, M32_User) = ARBITRARY : bits(32);
        Rmode(i, M32_FIQ)  = ARBITRARY : bits(32);
    end;
    if HaveEL(EL2) then Rmode(13, M32_Hyp) = ARBITRARY : bits(32); end;   // No R14_hyp
    for i = 13 to 14 do
        Rmode(i, M32_User)  = ARBITRARY : bits(32);
        Rmode(i, M32_FIQ)   = ARBITRARY : bits(32);
        Rmode(i, M32_IRQ)   = ARBITRARY : bits(32);
        Rmode(i, M32_Svc)   = ARBITRARY : bits(32);
        Rmode(i, M32_Abort) = ARBITRARY : bits(32);
        Rmode(i, M32_Undef) = ARBITRARY : bits(32);
        if HaveEL(EL3) then Rmode(i, M32_Monitor) = ARBITRARY : bits(32); end;
    end;

    return;
end;
```

Library pseudocode for aarch32/functions/registers/AArch32_ResetSIMDFPRegisters

```
// AArch32_ResetSIMDFPRegisters()
// ==============================

func AArch32_ResetSIMDFPRegisters()
begin

    for i = 0 to 15 do
        Q(i) = ARBITRARY : bits(128);
    end;

    return;
end;
```

Library pseudocode for aarch32/functions/registers/AArch32_ResetSpecialRegisters

```
// AArch32_ResetSpecialRegisters()
// ===============================

func AArch32_ResetSpecialRegisters()
begin

    // AArch32 special registers
    SPSR_fiq()[31:0] = ARBITRARY : bits(32);
    SPSR_irq()[31:0] = ARBITRARY : bits(32);
    SPSR_svc()[31:0] = ARBITRARY : bits(32);
    SPSR_abt()[31:0] = ARBITRARY : bits(32);
    SPSR_und()[31:0] = ARBITRARY : bits(32);
    if HaveEL(EL2) then
        SPSR_hyp() = ARBITRARY : bits(32);
        ELR_hyp()  = ARBITRARY : bits(32);
    end;
    if HaveEL(EL3) then
        SPSR_mon() = ARBITRARY : bits(32);
    end;

    // External debug special registers
    DLR()   = ARBITRARY : bits(32);
    DSPSR() = ARBITRARY : bits(32);

    return;
end;
```

Library pseudocode for aarch32/functions/registers/AArch32_ResetSystemRegisters

```
// AArch32_ResetSystemRegisters()
// ==============================

impdef func AArch32_ResetSystemRegisters(cold_reset : boolean)
begin
    return;
end;
```

Library pseudocode for aarch32/functions/registers/ALUExceptionReturn

```
// ALUExceptionReturn()
// ====================

func ALUExceptionReturn(address : bits(32))
begin
    if PSTATE.EL == EL2 then
        Undefined();
    elsif PSTATE.M IN {M32_User,M32_System} then
        let c : Constraint = ConstrainUnpredictable(Unpredictable_ALUEXCEPTIONRETURN);
        assert c IN {Constraint_UNDEF, Constraint_NOP};
        case c of
            when Constraint_UNDEF =>
                Undefined();
            when Constraint_NOP =>
                ExecuteAsNOP();
        end;
    else
        AArch32_ExceptionReturn(address, SPSR_curr());
    end;
end;
```

Library pseudocode for aarch32/functions/registers/ALUWritePC

```
// ALUWritePC()
// ============

func ALUWritePC(address : bits(32))
begin
    if CurrentInstrSet() == InstrSet_A32 then
        BXWritePC(address, BranchType_INDIR);
    else
        BranchWritePC(address, BranchType_INDIR);
    end;
end;
```

Library pseudocode for aarch32/functions/registers/BXWritePC

```
// BXWritePC()
// ===========

func BXWritePC(address_in : bits(32), branch_type : BranchType)
begin
    var address : bits(32) = address_in;
    if address[0] == '1' then
        SelectInstrSet(InstrSet_T32);
        address[0] = '0';
    else
        SelectInstrSet(InstrSet_A32);
        // For branches to an unaligned PC counter in A32 state, the PE takes the branch
        // and does one of:
        // * Forces the address to be aligned
        // * Leaves the PC unaligned, meaning the target generates a PC Alignment fault.
        if address[1] == '1' && ConstrainUnpredictableBool(Unpredictable_A32FORCEALIGNPC) then
            address[1] = '0';
        end;
    end;
    let branch_conditional : boolean = CurrentCond() != '111x';
    BranchTo{32}(address, branch_type, branch_conditional);
end;
```

Library pseudocode for aarch32/functions/registers/BranchWritePC

```
// BranchWritePC()
// ===============

func BranchWritePC(address_in : bits(32), branch_type : BranchType)
begin
    var address : bits(32) = address_in;
    if CurrentInstrSet() == InstrSet_A32 then
        address[1:0] = '00';
    else
        address[0] = '0';
    end;
    let branch_conditional : boolean = CurrentCond() != '111x';
    BranchTo{32}(address, branch_type, branch_conditional);
end;
```

Library pseudocode for aarch32/functions/registers/CBWritePC

```
// CBWritePC()
// ===========
// Takes a branch from a CBNZ/CBZ instruction.

func CBWritePC(address_in : bits(32))
begin
    var address : bits(32) = address_in;
    assert CurrentInstrSet() == InstrSet_T32;
    address[0] = '0';
    let branch_conditional : boolean = TRUE;
    BranchTo{32}(address, BranchType_DIR, branch_conditional);
end;
```

Library pseudocode for aarch32/functions/registers/D

```
// D - accessor
// ============

accessor D(n : integer) <=> value : bits(64)
begin
    getter
        assert n >= 0 && n <= 31;
        let vreg : bits(128) = V{}(n DIVRM 2);
        return vreg[(n MOD 2)*:64];
    end;
    setter
        assert n >= 0 && n <= 31;
        var vreg : bits(128) = V{}(n DIVRM 2);
        vreg[(n MOD 2)*:64] = value;
        V{128}(n DIVRM 2) = vreg;
    end;
end;
```

Library pseudocode for aarch32/functions/registers/Din

```
// Din
// ===
// Return the initial value of D input

func Din(n : integer) => bits(64)
begin
    assert n >= 0 && n <= 31;
    return _Dclone[[n]];
end;
```

Library pseudocode for aarch32/functions/registers/H

```
// H - accessor
// ============

accessor H(n : integer) <=> value : bits(16)
begin
    getter
        assert n >= 0 && n <= 31;
        return S(n)[15:0];
    end;
    setter
        S(n) = ZeroExtend{32}(value);
    end;
end;
```

Library pseudocode for aarch32/functions/registers/LR

```
// LR - accessor
// =============

accessor LR() <=> value : bits(32)
begin
    getter
        return R(14);
    end;
    setter
        R(14) = value;
    end;
end;
```

Library pseudocode for aarch32/functions/registers/LoadWritePC

```
// LoadWritePC()
// =============

func LoadWritePC(address : bits(32))
begin
    BXWritePC(address, BranchType_INDIR);
end;
```

Library pseudocode for aarch32/functions/registers/LookUpRIndex

```
// LookUpRIndex()
// ==============

func LookUpRIndex(n : integer, mode : bits(5)) => integer
begin
    assert n >= 0 && n <= 14;

    var result : integer;
    case n of  // Select  index by mode:     usr fiq irq svc abt und hyp
        when 8    =>  result = RBankSelect(mode,  8, 24,  8,  8,  8,  8);
        when 9    =>  result = RBankSelect(mode,  9, 25,  9,  9,  9,  9);
        when 10   =>  result = RBankSelect(mode, 10, 26, 10, 10, 10, 10);
        when 11   =>  result = RBankSelect(mode, 11, 27, 11, 11, 11, 11);
        when 12   =>  result = RBankSelect(mode, 12, 28, 12, 12, 12, 12);
        when 13   =>  result = RBankSelect(mode, 13, 29, 17, 19, 21, 15);
        when 14   =>  result = RBankSelect(mode, 14, 30, 16, 18, 20, 14);
        otherwise =>  result = n;
    end;

    return result;
end;
```

Library pseudocode for aarch32/functions/registers/Monitor

```
// Monitor mode registers
// ======================
// The Monitor mode registers do not map to X registers, so must be defined separately

var SP_mon : bits(32);

var LR_mon : bits(32);
```

Library pseudocode for aarch32/functions/registers/PC32

```
// PC32
// ====
// Read 32-bit program counter

func PC32() => bits(32)
begin
    return R(15);               // This includes the offset from AArch32 state
end;
```

Library pseudocode for aarch32/functions/registers/PCStoreValue

```
// PCStoreValue()
// ==============

func PCStoreValue() => bits(32)
begin
    // This function returns the PC value. On architecture versions before Armv7, it
    // is permitted to instead return PC+4, provided it does so consistently. It is
    // used only to describe A32 instructions, so it returns the address of the current
    // instruction plus 8 (normally) or 12 (when the alternative is permitted).
    return PC32();
end;
```

Library pseudocode for aarch32/functions/registers/Q

```
// Q - accessor
// ============

accessor Q(n : integer) <=> value : bits(128)
begin
    getter
        assert n >= 0 && n <= 15;
        return V{128}(n);
    end;
    setter
        assert n >= 0 && n <= 15;
        V{128}(n) = value;
    end;
end;
```

Library pseudocode for aarch32/functions/registers/Qin

```
// Qin
// ===
// Return the initial value of Q input

func Qin(n : integer) => bits(128)
begin
    assert n >= 0 && n <= 15;
    return Din(2*n+1)::Din(2*n);
end;
```

Library pseudocode for aarch32/functions/registers/R

```
// R - accessor
// ============

accessor R(n : integer) <=> value : bits(32)
begin
    getter
        if n == 15 then
            let offset : integer{} = (if CurrentInstrSet() == InstrSet_A32 then 8 else 4);
            return _PC[31:0] + offset;
        else
            return Rmode(n, PSTATE.M);
        end;
    end;
    setter
        Rmode(n, PSTATE.M) = value;
        return;
    end;
end;

// R - accessor
// ============

accessor R(lr : integer, hr : integer) <=> value : bits(64)
begin
    getter
        return R(hr) :: R(lr);
    end;

    setter
        R(lr) = value[0+:32];
        R(hr) = value[32+:32];
    end;
end;
```

Library pseudocode for aarch32/functions/registers/RBankSelect

```
// RBankSelect()
// =============

func RBankSelect(mode : bits(5), usr : integer, fiq : integer, irq : integer,
                 svc : integer, abt : integer, und : integer, hyp : integer) => integer
begin

    var result : integer;
    case mode of
        when M32_User   => result = usr;  // User mode
        when M32_FIQ    => result = fiq;  // FIQ mode
        when M32_IRQ    => result = irq;  // IRQ mode
        when M32_Svc    => result = svc;  // Supervisor mode
        when M32_Abort  => result = abt;  // Abort mode
        when M32_Hyp    => result = hyp;  // Hyp mode
        when M32_Undef  => result = und;  // Undefined mode
        when M32_System => result = usr;  // System mode uses User mode registers
        otherwise       => unreachable;   // Monitor mode
    end;

    return result;
end;
```

Library pseudocode for aarch32/functions/registers/ReadAnyAllocatedRegister

```
// ReadAnyAllocatedRegister()
// ==========================

impdef func ReadAnyAllocatedRegister() => bits(32)
begin
    return Zeros{32};
end;
```

Library pseudocode for aarch32/functions/registers/ReadAnyAllocatedSPSR

```
// ReadAnyAllocatedSPSR()
// ======================

impdef func ReadAnyAllocatedSPSR() => bits(32)
begin
    return Zeros{32};
end;
```

Library pseudocode for aarch32/functions/registers/Rmode

```
// Rmode - accessor
// ================

accessor Rmode(n : integer, mode : bits(5)) <=> value : bits(32)
begin
    getter
        assert n >= 0 && n <= 14;

        // Check for attempted use of Monitor mode in Non-secure state.
        if CurrentSecurityState() != SS_Secure then assert mode != M32_Monitor; end;
        assert !BadMode(mode);

        if mode == M32_Monitor then
            if n == 13 then return SP_mon;
            elsif n == 14 then return LR_mon;
            else return _R[[n]][31:0];
            end;
        else
            return _R[[LookUpRIndex(n, mode)]][31:0];
        end;
    end;

    setter
        assert n >= 0 && n <= 14;

        // Check for attempted use of Monitor mode in Non-secure state.
        if CurrentSecurityState() != SS_Secure then assert mode != M32_Monitor; end;
        assert !BadMode(mode);

        if mode == M32_Monitor then
            if n == 13 then SP_mon = value;
            elsif n == 14 then LR_mon = value;
            else _R[[n]][31:0] = value;
            end;
        else
            // It is CONSTRAINED UNPREDICTABLE whether the upper 32 bits of the X
            // register are unchanged or set to zero. This is also tested for on
            // exception entry, as this applies to all AArch32 registers.
            if HaveAArch64() && ConstrainUnpredictableBool(Unpredictable_ZEROUPPER) then
                _R[[LookUpRIndex(n, mode)]] = ZeroExtend{64}(value);
            else
                _R[[LookUpRIndex(n, mode)]][31:0] = value;
            end;
        end;
    end;
end;
```

Library pseudocode for aarch32/functions/registers/S

```
// S - accessor
// ============

accessor S(n : integer) <=> value : bits(32)
begin
    getter
        assert n >= 0 && n <= 31;
        let vreg : bits(128) = V{}(n DIVRM 4);
        return vreg[(n MOD 4)*:32];
    end;

    setter
        assert n >= 0 && n <= 31;
        var vreg : bits(128) = V{}(n DIVRM 4);
        vreg[(n MOD 4)*:32] = value;
        V{128}(n DIVRM 4) = vreg;
    end;
end;
```

Library pseudocode for aarch32/functions/registers/WriteAnyAllocatedRegister

```
// WriteAnyAllocatedRegister()
// ===========================

impdef func WriteAnyAllocatedRegister(value : bits(32))
begin
    return;
end;
```

Library pseudocode for aarch32/functions/registers/WriteAnyAllocatedSPSR

```
// WriteAnyAllocatedSPSR()
// =======================

impdef func WriteAnyAllocatedSPSR(value : bits(32))
begin
    return;
end;
```

Library pseudocode for aarch32/functions/registers/_Dclone

```
// _Dclone[]
// =========
// Clone the 64-bit Advanced SIMD and VFP extension register bank for use as input to
// instruction pseudocode, to avoid read-after-write for Advanced SIMD and VFP operations.

var _Dclone : array [[32]] of bits(64);
```

Library pseudocode for aarch32/functions/system/AArch32_ExceptionReturn

```
// AArch32_ExceptionReturn()
// =========================

func AArch32_ExceptionReturn(new_pc_in : bits(32), spsr : bits(32))
begin
    var new_pc : bits(32) = new_pc_in;
    SynchronizeContext();
    // Attempts to change to an illegal mode or state will invoke the Illegal Execution state
    // mechanism
    SetPSTATEFromPSR{32}(spsr);
    ClearExclusiveLocal(ProcessorID());
    SendEventLocal();

    if PSTATE.IL == '1' then
        // If the exception return is illegal, PC[1:0] are UNKNOWN
        new_pc[1:0] = ARBITRARY : bits(2);
    else
        // LR[1:0] or LR[0] are treated as being 0, depending on the target instruction set state
        if PSTATE.T == '1' then
            new_pc[0] = '0';                // T32
        else
            new_pc[1:0] = '00';             // A32
        end;
    end;

    let branch_conditional : boolean = CurrentCond() != '111x';
    BranchTo{32}(new_pc, BranchType_ERET, branch_conditional);
    CheckExceptionCatch(FALSE);             // Check for debug event on exception return
end;
```

Library pseudocode for aarch32/functions/system/AArch32_ExecutingCP10or11Instr

```
// AArch32_ExecutingCP10or11Instr()
// ================================

func AArch32_ExecutingCP10or11Instr() => boolean
begin
    let instr : bits(32) =  ThisInstr();
    let instr_set : InstrSet = CurrentInstrSet();
    assert instr_set IN {InstrSet_A32, InstrSet_T32};

    if instr_set == InstrSet_A32 then
        return ((instr[27:24] == '1110' || instr[27:25] == '110') && instr[11:8] == '101x');
    else // InstrSet_T32
        return (instr[31:28] == '111x' && (instr[27:24] == '1110' || instr[27:25] == '110') &&
                instr[11:8] == '101x');
    end;
end;
```

Library pseudocode for aarch32/functions/system/AArch32_ITAdvance

```
// AArch32_ITAdvance()
// ===================

func AArch32_ITAdvance()
begin
    if PSTATE.IT[2:0] == '000' then
        PSTATE.IT = '00000000';
    else
        PSTATE.IT[4:0] = LSL(PSTATE.IT[4:0], 1);
    end;
    return;
end;
```

Library pseudocode for aarch32/functions/system/AArch32_InterruptPending

```
// AArch32_InterruptPending()
// ==========================
// Returns TRUE if there are any pending physical or virtual interrupts, and FALSE otherwise.

func AArch32_InterruptPending() => boolean
begin
    if !ELUsingAArch32(EL2) then
        return AArch64_InterruptPending();
    end;

    let (irq_pending, -) = IRQPending();
    let (fiq_pending, -) = FIQPending();
    let pending_physical_interrupt : boolean = (irq_pending || fiq_pending ||
                                                IsPhysicalSErrorPending());

    var pending_virtual_interrupt : boolean = FALSE;
    if EL2Enabled() && PSTATE.EL IN {EL0, EL1} && HCR().TGE == '0' then
        let virq_pending : boolean = (HCR().IMO == '1' && (VirtualIRQPending() || HCR().VI == '1'));
        let vfiq_pending : boolean = (HCR().FMO == '1' && (VirtualFIQPending() || HCR().VF == '1'));
        let vsei_pending : boolean = (HCR().AMO == '1' &&
                                      (IsVirtualSErrorPending() || HCR().VA == '1'));

        pending_virtual_interrupt = vsei_pending || virq_pending || vfiq_pending;
    end;

    return pending_physical_interrupt || pending_virtual_interrupt;
end;
```

Library pseudocode for aarch32/functions/system/AArch32_SysRegRead

```
// AArch32_SysRegRead()
// ====================
// Read from a 32-bit AArch32 System register and write the register's contents to R[t].

impdef func AArch32_SysRegRead(cp_num : integer, instr : bits(32), t : integer)
begin
    let (-, el) = ELFromM32(PSTATE.M);
    let opc1 = instr[21+:3];
    let CRn  = instr[16+:4];
    let CRm  = instr[0+:4];
    let opc2 = instr[5+:3];
end;
```

Library pseudocode for aarch32/functions/system/AArch32_SysRegRead64

```
// AArch32_SysRegRead64()
// ======================
// Read from a 64-bit AArch32 System register and write the register's contents to R[t] and R[t2].

impdef func AArch32_SysRegRead64(cp_num : integer, instr : bits(32), t : integer, t2 : integer)
begin
    let (-, el) = ELFromM32(PSTATE.M);
    let opc1 = instr[4+:4];
    let CRm  = instr[0+:4];
end;
```

Library pseudocode for aarch32/functions/system/AArch32_SysRegReadCanWriteAPSR

```
// AArch32_SysRegReadCanWriteAPSR()
// ================================
// Determines whether the AArch32 System register read instruction can write to APSR flags.

func AArch32_SysRegReadCanWriteAPSR(cp_num : integer, instr : bits(32)) => boolean
begin
    assert UsingAArch32();
    assert (cp_num IN {14,15});
    assert cp_num == UInt(instr[11:8]);

    let opc1 : integer{} = UInt(instr[23:21]);
    let opc2 : integer{} = UInt(instr[7:5]);
    let CRn : integer{}  = UInt(instr[19:16]);
    let CRm : integer{}  = UInt(instr[3:0]);

    if cp_num == 14 && opc1 == 0 && CRn == 0 && CRm == 1 && opc2 == 0 then // DBGDSCRint
        return TRUE;
    end;

    return FALSE;
end;
```

Library pseudocode for aarch32/functions/system/AArch32_SysRegWrite

```
// AArch32_SysRegWrite()
// =====================
// Read the contents of R[t] and write to a 32-bit AArch32 System register.

impdef func AArch32_SysRegWrite(cp_num : integer, instr : bits(32), t : integer)
begin
    let (-, el) = ELFromM32(PSTATE.M);
    let opc1 = instr[21+:3];
    let CRn  = instr[16+:4];
    let CRm  = instr[ 0+:4];
    let opc2 = instr[ 5+:3];
end;
```

Library pseudocode for aarch32/functions/system/AArch32_SysRegWrite64

```
// AArch32_SysRegWrite64()
// =======================
// Read the contents of R[t] and R[t2] and write to a 64-bit AArch32 System register.

impdef func AArch32_SysRegWrite64(cp_num : integer,
                                  instr : bits(32),
                                  t : integer,
                                  t2 : integer)
begin
    let (-, el) = ELFromM32(PSTATE.M);
    let opc1 = instr[4+:4];
    let CRm  = instr[0+:4];
end;
```

Library pseudocode for aarch32/functions/system/AArch32_SysRegWriteM

```
// AArch32_SysRegWriteM()
// ======================
// Read a value from a virtual address and write it to an AArch32 System register.

impdef func AArch32_SysRegWriteM(cp_num : integer, instr : bits(32), address : bits(32))
begin
    let CRd : bits(4) = instr[15:12];
end;
```

Library pseudocode for aarch32/functions/system/AArch32_WriteMode

```
// AArch32_WriteMode()
// ===================
// Function for dealing with writes to PSTATE.M from AArch32 state only.
// This ensures that PSTATE.EL and PSTATE.SP are always valid.

func AArch32_WriteMode(mode : bits(5))
begin
    let (valid,el) : (boolean, bits(2)) = ELFromM32(mode);
    assert valid;
    PSTATE.M   = mode;
    PSTATE.EL  = el;
    PSTATE.nRW = '1';
    PSTATE.SP  = (if mode IN {M32_User,M32_System} then '0' else '1');
    return;
end;
```

Library pseudocode for aarch32/functions/system/AArch32_WriteModeByInstr

```
// AArch32_WriteModeByInstr()
// ==========================
// Function for dealing with writes to PSTATE.M from an AArch32 instruction, and ensuring that
// illegal state changes are correctly flagged in PSTATE.IL.

func AArch32_WriteModeByInstr(mode : bits(5))
begin
    var (valid,el) : (boolean, bits(2)) = ELFromM32(mode);

    // 'valid' is set to FALSE if' mode' is invalid for this implementation or the current value
    // of SCR.NS/SCR_EL3.NS. Additionally, it is illegal for an instruction to write 'mode' to
    // PSTATE.EL if it would result in any of:
    // * A change to a mode that would cause entry to a higher Exception level.
    if UInt(el) > UInt(PSTATE.EL) then
        valid = FALSE;
    end;

    // * A change to or from Hyp mode.
    if (PSTATE.M == M32_Hyp || mode == M32_Hyp) && PSTATE.M != mode then
        valid = FALSE;
    end;
    // * When EL2 is implemented, the value of HCR.TGE is '1', a change to a Non-secure EL1 mode.
    if (PSTATE.M == M32_Monitor && HaveEL(EL2) && el == EL1 && SCR().NS == '1'
            && HCR().TGE == '1') then
        valid = FALSE;
    end;

    if !valid then
        PSTATE.IL = '1';
    else
        AArch32_WriteMode(mode);
    end;
end;
```

Library pseudocode for aarch32/functions/system/BPIALL

```
// BPIALL()
// ========
// Invalidate all entries from branch predictors.

impdef func BPIALL()
begin
    return;
end;
```

Library pseudocode for aarch32/functions/system/BPIALLIS

```
// BPIALLIS()
// ==========
// Invalidate all entries from branch predictors, Inner Shareable.

impdef func BPIALLIS()
begin
    return;
end;
```

Library pseudocode for aarch32/functions/system/BPIMVA

```
// BPIMVA()
// ========
// Invalidate the virtual address from the branch predictors.

impdef func BPIMVA(val : bits(32))
begin
    return;
end;
```

Library pseudocode for aarch32/functions/system/BadMode

```
// BadMode()
// =========

readonly func BadMode(mode : bits(5)) => boolean
begin
    // Return TRUE if 'mode' encodes a mode that is not valid for this implementation
    var valid : boolean;
    case mode of
        when M32_Monitor =>
            valid = HaveAArch32EL(EL3);
        when M32_Hyp =>
            valid = HaveAArch32EL(EL2);
        when M32_FIQ, M32_IRQ, M32_Svc, M32_Abort, M32_Undef, M32_System =>
            // If EL3 is implemented and using AArch32, then these modes are EL3 modes in Secure
            // state, and EL1 modes in Non-secure state. If EL3 is not implemented or is using
            // AArch64, then these modes are EL1 modes.
            // Therefore it is sufficient to test this implementation supports EL1 using AArch32
            valid = HaveAArch32EL(EL1);
        when M32_User =>
            valid = HaveAArch32EL(EL0);
        otherwise =>
            valid = FALSE;           // Passed an illegal mode value
    end;
    return !valid;
end;
```

Library pseudocode for aarch32/functions/system/BankedRegisterAccessValid

```
// BankedRegisterAccessValid()
// ===========================
// Checks for MRS (Banked register) or MSR (Banked register) accesses to registers
// other than the SPSRs that are invalid. This includes ELR_hyp accesses.

func BankedRegisterAccessValid(SYSm : bits(5), mode : bits(5)) => (boolean, Constraint)
begin
    var valid : boolean = TRUE;
    var c : Constraint = Constraint_NONE;
    var banked_unpred : boolean = FALSE;
    case SYSm of
        when '000xx', '00100' =>                          // R8_usr to R12_usr
            banked_unpred = mode != M32_FIQ;
        when '00101' =>                                   // SP_usr
            banked_unpred = mode == M32_System;
        when '00110' =>                                   // LR_usr
            banked_unpred = mode IN {M32_Hyp,M32_System};
        when '010xx', '0110x', '01110' =>                 // R8_fiq to R12_fiq, SP_fiq, LR_fiq
            banked_unpred = mode == M32_FIQ;
        when '1000x' =>                                   // LR_irq, SP_irq
            banked_unpred = mode == M32_IRQ;
        when '1001x' =>                                   // LR_svc, SP_svc
            banked_unpred = mode == M32_Svc;
        when '1010x' =>                                   // LR_abt, SP_abt
            banked_unpred = mode == M32_Abort;
        when '1011x' =>                                   // LR_und, SP_und
            banked_unpred = mode == M32_Undef;
        when '1110x' =>                                   // LR_mon, SP_mon
            banked_unpred = (!HaveEL(EL3) || CurrentSecurityState() != SS_Secure ||
                             mode == M32_Monitor);
        when '11110' =>                                   // ELR_hyp, only from Monitor or Hyp mode
            banked_unpred = !HaveEL(EL2) || ! mode IN {M32_Monitor,M32_Hyp};
        when '11111' =>                                   // SP_hyp, only from Monitor mode
            banked_unpred = !HaveEL(EL2) || mode != M32_Monitor;
        otherwise =>
            valid = FALSE;
            c     = ConstrainUnpredictable(Unpredictable_UnimplementedRegister);
            assert c IN {Constraint_UNDEF, Constraint_NOP, Constraint_ANYREG};
    end;

    if banked_unpred then
        valid = FALSE;
        c     = ConstrainUnpredictable(Unpredictable_BankedRegister);
        assert c IN {Constraint_UNDEF, Constraint_NOP, Constraint_UNKNOWN};
    end;

    return (valid, c);
end;
```

Library pseudocode for aarch32/functions/system/CP15DMB

```
// CP15DMB()
// =========
// Legacy AArch32 system instruction form of DMB SY.

func CP15DMB()
begin
    DataMemoryBarrier(MBReqTypes_All);
    return;
end;
```

Library pseudocode for aarch32/functions/system/CP15DSB

```
// CP15DSB()
// =========
// Legacy AArch32 system instruction form of DSB SY.

func CP15DSB()
begin
    DataSynchronizationBarrier(MBMaintenanceScope_OuterShareable, MBReqTypes_All, FALSE);
    return;
end;
```

Library pseudocode for aarch32/functions/system/CP15ISB

```
// CP15ISB()
// =========
// Legacy AArch32 system instruction form of ISB.

func CP15ISB()
begin
    InstructionSynchronizationBarrier();
    return;
end;
```

Library pseudocode for aarch32/functions/system/CPSRWriteByInstr

```
// CPSRWriteByInstr()
// ==================
// Update PSTATE.[N,Z,C,V,Q,GR,E,A,I,F,M] from a CPSR value written by an MSR instruction.

func CPSRWriteByInstr(value : bits(32), bytemask : bits(4))
begin
    let privileged : boolean = PSTATE.EL != EL0;        // PSTATE.[A,I,F,M] are not writable at EL0

    // Write PSTATE from 'value', ignoring bytes masked by 'bytemask'
    if bytemask[3] == '1' then
        PSTATE.[N,Z,C,V,Q] = value[31:27];
        // Bits [26:24] are ignored
    end;

    if bytemask[2] == '1' then
        if IsFeatureImplemented(FEAT_SSBS) then
            PSTATE.SSBS = value[23];
        end;
        if privileged then
            PSTATE.PAN = value[22];
        end;
        if IsFeatureImplemented(FEAT_DIT) then
            PSTATE.DIT = value[21];
        end;
        // Bit [20] is RES0
        PSTATE.GE = value[19:16];
    end;

    if bytemask[1] == '1' then
        // Bits [15:10] are RES0
        PSTATE.E = value[9];                    // PSTATE.E is writable at EL0
        if privileged then
            PSTATE.A = value[8];
        end;
    end;

    if bytemask[0] == '1' then
        if privileged then
            PSTATE.[I,F] = value[7:6];
            // Bit [5] is RES0
            // AArch32_WriteModeByInstr() sets PSTATE.IL to 1 if this is an illegal mode change.
            AArch32_WriteModeByInstr(value[4:0]);
        end;
    end;
    return;
end;
```

Library pseudocode for aarch32/functions/system/ConditionPassed

```
// ConditionPassed()
// =================

func ConditionPassed() => boolean
begin
    return ConditionHolds(CurrentCond());
end;
```

Library pseudocode for aarch32/functions/system/CurrentCond

```
// CurrentCond()
// =============
// Returns the condition code that applies to the currently executing instruction

impdef func CurrentCond() => bits(4)
begin
    return Zeros{4};
end;
```

Library pseudocode for aarch32/functions/system/InITBlock

```
// InITBlock()
// ===========

func InITBlock() => boolean
begin
    if CurrentInstrSet() == InstrSet_T32 then
        return PSTATE.IT[3:0] != '0000';
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch32/functions/system/LastInITBlock

```
// LastInITBlock()
// ===============

func LastInITBlock() => boolean
begin
    return (PSTATE.IT[3:0] == '1000');
end;
```

Library pseudocode for aarch32/functions/system/SPSRWriteByInstr

```
// SPSRWriteByInstr()
// ==================

func SPSRWriteByInstr(value : bits(32), bytemask : bits(4))
begin
    var new_spsr : bits(32) = SPSR_curr();

    if bytemask[3] == '1' then
        new_spsr[31:24] = value[31:24];  // N, Z, C, V, Q flags, IT[1:0],J bits
    end;

    if bytemask[2] == '1' then
        new_spsr[23:16] = value[23:16];  // IL bit, GE[3:0] flags
    end;

    if bytemask[1] == '1' then
        new_spsr[15:8] = value[15:8];    // IT[7:2] bits, E bit, A interrupt mask
    end;

    if bytemask[0] == '1' then
        new_spsr[7:0] = value[7:0];      // I, F interrupt masks, T bit, Mode bits
    end;

    SPSR_curr() = new_spsr;              // UNPREDICTABLE if User or System mode

    return;
end;
```

Library pseudocode for aarch32/functions/system/SPSRaccessValid

```
// SPSRaccessValid()
// =================
// Checks for MRS (Banked register) or MSR (Banked register) accesses to the SPSRs
// that are UNPREDICTABLE

func SPSRaccessValid(SYSm : bits(5), mode : bits(5)) => (boolean, Constraint)
begin
    var valid : boolean = TRUE;
    var c : Constraint = Constraint_NONE;
    var banked_unpred : boolean = FALSE;
    case SYSm of
        when '01110' =>                                                 // SPSR_fiq
            banked_unpred = mode == M32_FIQ;
        when '10000' =>                                                 // SPSR_irq
            banked_unpred = mode == M32_IRQ;
        when '10010' =>                                                 // SPSR_svc
            banked_unpred = mode == M32_Svc;
        when '10100' =>                                                 // SPSR_abt
            banked_unpred = mode == M32_Abort;
        when '10110' =>                                                 // SPSR_und
            banked_unpred = mode == M32_Undef;
        when '11100' =>                                                 // SPSR_mon
            banked_unpred = (!HaveEL(EL3) || mode == M32_Monitor ||
                             CurrentSecurityState() != SS_Secure);
        when '11110' =>                                                 // SPSR_hyp
            banked_unpred = !HaveEL(EL2) || mode != M32_Monitor;
        otherwise =>
            valid = FALSE;
            c = ConstrainUnpredictable(Unpredictable_UnimplementedRegister);
            assert c IN {Constraint_UNDEF, Constraint_NOP, Constraint_ANYREG};
    end;

    if banked_unpred then
        valid = FALSE;
        c     = ConstrainUnpredictable(Unpredictable_BankedRegister);
        assert c IN {Constraint_UNDEF, Constraint_NOP, Constraint_UNKNOWN};
    end;

    return (valid, c);
end;
```

Library pseudocode for aarch32/functions/system/SelectInstrSet

```
// SelectInstrSet()
// ================

func SelectInstrSet(iset : InstrSet)
begin
    assert CurrentInstrSet() IN {InstrSet_A32, InstrSet_T32};
    assert iset IN {InstrSet_A32, InstrSet_T32};

    PSTATE.T = if iset == InstrSet_A32 then '0' else '1';

    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_DTLBI_ALL

```
// AArch32_DTLBI_ALL()
// ===================
// Invalidate all data TLB entries for the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast domain.
// Invalidation applies to all applicable stage 1 and stage 2 entries.

func AArch32_DTLBI_ALL(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                       broadcast_in : Broadcast, attr_in : TLBIMemAttr)
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_DALL;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = TLBILevel_Any;
    r.attr         = attr;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_DTLBI_ASID

```
// AArch32_DTLBI_ASID()
// ====================
// Invalidate all data TLB stage 1 entries matching the indicated VMID (where regime supports)
// and ASID in the parameter Rt in the indicated translation regime with the
// indicated security state for all TLBs within the indicated broadcast domain.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_DTLBI_ASID(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                        broadcast_in : Broadcast, attr_in : TLBIMemAttr, Rt : bits(32))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_DASID;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = TLBILevel_Any;
    r.attr         = attr;
    r.asid         = Zeros{8} :: Rt[7:0];

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_DTLBI_VA

```
// AArch32_DTLBI_VA()
// ==================
// Invalidate by VA all stage 1 data TLB entries in the indicated broadcast domain
// matching the indicated VMID and ASID (where regime supports VMID, ASID) in the indicated regime
// with the indicated security state.
// ASID, VA and related parameters are derived from Rt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_DTLBI_VA(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                      broadcast_in : Broadcast, level : TLBILevel,  attr_in : TLBIMemAttr,
                      Rt : bits(32))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_DVA;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.asid         = Zeros{8} :: Rt[7:0];
    r.address      = Zeros{32} :: Rt[31:12] :: Zeros{12};

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_EffectiveBroadcast

```
// AArch32_EffectiveBroadcast()
// ============================

func AArch32_EffectiveBroadcast(broadcast_in : Broadcast) => Broadcast
begin
    var broadcast : Broadcast = broadcast_in;

    if PSTATE.EL != EL1 then return broadcast; end;

    if (broadcast == Broadcast_NSH && ((ELUsingAArch32(EL2) && HCR().FB == '1') ||
          (!ELUsingAArch32(EL2) && HCR_EL2().FB == '1'))) then
        broadcast = Broadcast_ForcedISH;
    end;

    return broadcast;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_ITLBI_ALL

```
// AArch32_ITLBI_ALL()
// ===================
// Invalidate all instruction TLB entries for the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast domain.
// Invalidation applies to all applicable stage 1 and stage 2 entries.

func AArch32_ITLBI_ALL(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                       broadcast_in : Broadcast, attr_in : TLBIMemAttr)
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_IALL;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = TLBILevel_Any;
    r.attr         = attr;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_ITLBI_ASID

```
// AArch32_ITLBI_ASID()
// ====================
// Invalidate all instruction TLB stage 1 entries matching the indicated VMID
// (where regime supports) and ASID in the parameter Rt in the indicated translation
// regime with the indicated security state for all TLBs within the indicated broadcast domain.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_ITLBI_ASID(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                        broadcast_in : Broadcast, attr_in : TLBIMemAttr, Rt : bits(32))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_IASID;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = TLBILevel_Any;
    r.attr         = attr;
    r.asid         = Zeros{8} :: Rt[7:0];

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_ITLBI_VA

```
// AArch32_ITLBI_VA()
// ==================
// Invalidate by VA all stage 1 instruction TLB entries in the indicated broadcast domain
// matching the indicated VMID and ASID (where regime supports VMID, ASID) in the indicated regime
// with the indicated security state.
// ASID, VA and related parameters are derived from Rt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_ITLBI_VA(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                      broadcast_in : Broadcast, level : TLBILevel,  attr_in : TLBIMemAttr,
                      Rt : bits(32))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_IVA;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.asid         = Zeros{8} :: Rt[7:0];
    r.address      = Zeros{32} :: Rt[31:12] :: Zeros{12};

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_ALL

```
// AArch32_TLBI_ALL()
// ==================
// Invalidate all entries for the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast domain.
// Invalidation applies to all applicable stage 1 and stage 2 entries.

func AArch32_TLBI_ALL(security : SecurityState, regime : Regime,
                      broadcast : Broadcast, attr : TLBIMemAttr)
begin
    assert PSTATE.EL IN {EL3, EL2};

    var r : TLBIRecord;
    r.op           = TLBIOp_ALL;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.level        = TLBILevel_Any;
    r.attr         = attr;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_ASID

```
// AArch32_TLBI_ASID()
// ===================
// Invalidate all stage 1 entries matching the indicated VMID (where regime supports)
// and ASID in the parameter Rt in the indicated translation regime with the
// indicated security state for all TLBs within the indicated broadcast domain.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_TLBI_ASID(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                       broadcast_in : Broadcast, attr_in : TLBIMemAttr, Rt : bits(32))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op          = TLBIOp_ASID;
    r.from_aarch64 = FALSE;
    r.security    = security;
    r.regime      = regime;
    r.vmid        = vmid;
    r.use_vmid    = UseVMID(regime);
    r.level       = TLBILevel_Any;
    r.attr        = attr;
    r.asid        = Zeros{8} :: Rt[7:0];

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_IPAS2

```
// AArch32_TLBI_IPAS2()
// ====================
// Invalidate by IPA all stage 2 only TLB entries in the indicated broadcast
// domain matching the indicated VMID in the indicated regime with the indicated security state.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// IPA and related parameters of the are derived from Rt.

func AArch32_TLBI_IPAS2(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                        broadcast : Broadcast, level : TLBILevel, attr : TLBIMemAttr, Rt : bits(32))
begin
    assert PSTATE.EL IN {EL3, EL2};
    assert security == SS_NonSecure;

    var r : TLBIRecord;
    r.op          = TLBIOp_IPAS2;
    r.from_aarch64 = FALSE;
    r.security    = security;
    r.regime      = regime;
    r.vmid        = vmid;
    r.use_vmid    = TRUE;
    r.level       = level;
    r.attr        = attr;
    r.address     = Zeros{24} :: Rt[27:0] :: Zeros{12};
    r.ipaspace    = PAS_NonSecure;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_VA

```
// AArch32_TLBI_VA()
// =================
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID and ASID (where regime supports VMID, ASID) in the indicated regime
// with the indicated security state.
// ASID, VA and related parameters are derived from Rt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_TLBI_VA(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                     broadcast_in : Broadcast, level : TLBILevel,  attr_in : TLBIMemAttr,
                     Rt : bits(32))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_VA;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.asid         = Zeros{8} :: Rt[7:0];
    r.address      = Zeros{32} :: Rt[31:12] :: Zeros{12};

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_VAA

```
// AArch32_TLBI_VAA()
// ==================
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID (where regime supports VMID) and all ASID in the indicated regime
// with the indicated security state.
// VA and related parameters are derived from Rt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch32_TLBI_VAA(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                      broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
                      Rt : bits(32))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_VAA;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.address      = Zeros{32} :: Rt[31:12] :: Zeros{12};

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_VMALL

```
// AArch32_TLBI_VMALL()
// ====================
// Invalidate all stage 1 entries for the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast
// domain that match the indicated VMID (where applicable).
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.
// Note: stage 2 only entries are not in the scope of this operation.

func AArch32_TLBI_VMALL(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                        broadcast_in : Broadcast, attr_in : TLBIMemAttr)
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var attr : TLBIMemAttr = attr_in;

    let broadcast : Broadcast = AArch32_EffectiveBroadcast(broadcast_in);
    if ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_VMALL;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.level        = TLBILevel_Any;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.attr         = attr;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/tlbi/AArch32_TLBI_VMALLS12

```
// AArch32_TLBI_VMALLS12()
// =======================
// Invalidate all stage 1 and stage 2 entries for the indicated translation
// regime with the indicated security state for all TLBs within the indicated
// broadcast domain that match the indicated VMID.

func AArch32_TLBI_VMALLS12(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                           broadcast : Broadcast, attr : TLBIMemAttr)
begin
    assert PSTATE.EL IN {EL3, EL2};

    var r : TLBIRecord;
    r.op           = TLBIOp_VMALLS12;
    r.from_aarch64 = FALSE;
    r.security     = security;
    r.regime       = regime;
    r.level        = TLBILevel_Any;
    r.vmid         = vmid;
    r.use_vmid     = TRUE;
    r.attr         = attr;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch32/functions/v6simd/Sat

```
// Sat()
// =====

func Sat{N}(i : integer, unsigned : boolean) => bits(N)
begin
    let result : bits(N) = if unsigned then UnsignedSat{N}(i) else SignedSat{N}(i);
    return result;
end;
```

Library pseudocode for aarch32/ic/AArch32_CanTrapIC

```
// AArch32_CanTrapIC()
// ===================
// Determines whether the execution of the IC instruction can be trapped.

func AArch32_CanTrapIC(cacheop : CacheOp, opscope : CacheOpScope) => boolean
begin
    return (!AArch32_TreatICAsNOP(cacheop, opscope) || ImpDefBool(
            "When IC is treated as NOP, data cache maintenance operations are trapped"));
end;
```

Library pseudocode for aarch32/ic/AArch32_IC

```
// AArch32_IC()
// ============
// Perform Instruction Cache Operation.

func AArch32_IC(opscope : CacheOpScope)
begin
    let regval : bits(32) = ARBITRARY : bits(32);
    AArch32_IC(regval, opscope);
end;

// AArch32_IC()
// ============
// Perform Instruction Cache Operation.

func AArch32_IC(regval : bits(32), opscope : CacheOpScope)
begin
    var cache : CacheRecord;

    cache.acctype   = AccessType_IC;
    cache.cachetype = CacheType_Instruction;
    cache.cacheop   = CacheOp_Invalidate;
    cache.opscope   = opscope;
    cache.security  = SecurityStateAtEL(PSTATE.EL);

    if opscope IN {CacheOpScope_ALLU, CacheOpScope_ALLUIS} then
        if (opscope == CacheOpScope_ALLUIS ||
              AArch32_EffectiveBroadcast(Broadcast_NSH) == Broadcast_ForcedISH) then
            cache.shareability = Shareability_ISH;
        else
            cache.shareability = Shareability_NSH;
        end;
        cache.regval = ZeroExtend{64}(regval);
        CACHE_OP(cache);
    else
        assert opscope == CacheOpScope_PoU;

        if EL2Enabled() then
            if PSTATE.EL IN {EL0, EL1} then
                cache.is_vmid_valid = TRUE;
                cache.vmid          = VMID();
            else
                cache.is_vmid_valid = FALSE;
            end;
        else
            cache.is_vmid_valid = FALSE;
        end;

        if PSTATE.EL == EL0 then
            cache.is_asid_valid = TRUE;
            cache.asid          = ASID();
        else
            cache.is_asid_valid = FALSE;
        end;

        cache.shareability = Shareability_NSH;
        cache.vaddress     = ZeroExtend{64}(regval);

        let size : integer    = 0;
        let aligned : boolean = TRUE;
        let accdesc : AccessDescriptor = CreateAccDescIC(cache);
        let memaddrdesc : AddressDescriptor = AArch32_TranslateAddress(regval, accdesc, aligned,
                                                                       size);
        if IsFault(memaddrdesc) then
            AArch32_Abort(memaddrdesc.fault);
        end;

        cache.paddress = memaddrdesc.paddress;
        CACHE_OP(cache);
    end;
    return;
end;
```

Library pseudocode for aarch32/ic/AArch32_TreatICAsNOP

```
// AArch32_TreatICAsNOP()
// ======================
// Determines whether the execution of the IC instruction is treated as a NOP.

func AArch32_TreatICAsNOP(cacheop : CacheOp, opscope : CacheOpScope) => boolean
begin
    if CTR().DIC == '1' then
        return ImpDefBool("IC is treated as NOP");
    end;

    return FALSE;
end;
```

Library pseudocode for aarch32/predictionrestrict/AArch32_RestrictPrediction

```
// AArch32_RestrictPrediction()
// ============================
// Clear all predictions in the context.

func AArch32_RestrictPrediction(val : bits(32), restriction : RestrictType)
begin

    var c : ExecutionCntxt;
    let target_el : bits(2) = val[25:24];

    // If the target EL is not implemented or the instruction is executed at an
    // EL lower than the specified level, the instruction is treated as a NOP.
    if !HaveEL(target_el) || UInt(target_el) > UInt(PSTATE.EL) then ExecuteAsNOP(); end;

    let ns : bit  = val[26];
    let nse : bit = ARBITRARY : bit;
    let ss : SecurityState = TargetSecurityState(ns, nse);

    c.security  = ss;
    c.target_el = target_el;

    if EL2Enabled() then
        if PSTATE.EL IN {EL0, EL1} then
            c.is_vmid_valid = TRUE;
            c.all_vmid      = FALSE;
            c.vmid          = VMID();

        elsif target_el IN {EL0, EL1} then
            c.is_vmid_valid = TRUE;
            c.all_vmid      = val[27] == '1';
            c.vmid          = ZeroExtend{NUM_VMIDBITS}(val[23:16]);// Only valid if  val[27] == '0'
        else
            c.is_vmid_valid = FALSE;
        end;
    else
        c.is_vmid_valid = FALSE;
    end;

    if PSTATE.EL == EL0 then
        c.is_asid_valid = TRUE;
        c.all_asid      = FALSE;
        c.asid          = ASID();

    elsif target_el == EL0 then
        c.is_asid_valid = TRUE;
        c.all_asid      = val[8] == '1';
        c.asid          = ZeroExtend{NUM_ASIDBITS}(val[7:0]);      // Only valid if  val[8] == '0'

    else
        c.is_asid_valid = FALSE;
    end;

    c.restriction = restriction;
    RESTRICT_PREDICTIONS(c);
end;
```

Library pseudocode for aarch32/translation/attrs/AArch32_DefaultTEXDecode

```
// AArch32_DefaultTEXDecode()
// ==========================
// Apply short-descriptor format memory region attributes, without TEX remap

func AArch32_DefaultTEXDecode(TEX_in : bits(3), c_in : bit, b_in : bit, s : bit) => MemoryAttributes
begin
    var mematTrs : MemoryAttributes;
    var TEX : bits(3) = TEX_in;
    var c : bit = c_in;
    var b : bit = b_in;

    // Reserved values map to allocated values
    if (TEX == '001' && c::b == '01') || (TEX == '010' && c::b != '00') || TEX == '011' then
        var texcb : bits(5);
        (-, texcb) = ConstrainUnpredictableBits{5}(Unpredictable_RESTEXCB);
        TEX = texcb[4:2];  c = texcb[1];  b = texcb[0];
    end;

    // Distinction between Inner Shareable and Outer Shareable is not supported in this format
    // A memory region is either Non-shareable or Outer Shareable
    case TEX::c::b of
        when '00000' =>
            // Device-nGnRnE
            mematTrs.memtype      = MemType_Device;
            mematTrs.device       = DeviceType_nGnRnE;
            mematTrs.shareability = Shareability_OSH;
        when '00001', '01000' =>
            // Device-nGnRE
            mematTrs.memtype      = MemType_Device;
            mematTrs.device       = DeviceType_nGnRE;
            mematTrs.shareability = Shareability_OSH;
        when '00010' =>
            // Write-through Read allocate
            mematTrs.memtype      = MemType_Normal;
            mematTrs.inner.attrs  = MemAttr_WT;
            mematTrs.inner.hints  = MemHint_RA;
            mematTrs.outer.attrs  = MemAttr_WT;
            mematTrs.outer.hints  = MemHint_RA;
            mematTrs.shareability = if s == '1' then Shareability_OSH else Shareability_NSH;
        when '00011' =>
            // Write-back Read allocate
            mematTrs.memtype      = MemType_Normal;
            mematTrs.inner.attrs  = MemAttr_WB;
            mematTrs.inner.hints  = MemHint_RA;
            mematTrs.outer.attrs  = MemAttr_WB;
            mematTrs.outer.hints  = MemHint_RA;
            mematTrs.shareability = if s == '1' then Shareability_OSH else Shareability_NSH;
        when '00100' =>
            // Non-cacheable
            mematTrs.memtype      = MemType_Normal;
            mematTrs.inner.attrs  = MemAttr_NC;
            mematTrs.outer.attrs  = MemAttr_NC;
            mematTrs.shareability = Shareability_OSH;
        when '00110' =>
            mematTrs = ImpDefMemoryAttributes("IMPLEMENTATION_DEFINED");
        when '00111' =>
            // Write-back Read and Write allocate
            mematTrs.memtype      = MemType_Normal;
            mematTrs.inner.attrs  = MemAttr_WB;
            mematTrs.inner.hints  = MemHint_RWA;
            mematTrs.outer.attrs  = MemAttr_WB;
            mematTrs.outer.hints  = MemHint_RWA;
            mematTrs.shareability = if s == '1' then Shareability_OSH else Shareability_NSH;
        when '1xxxx' =>
            // Cacheable, TEX[1:0] = Outer attrs, {c,b} = Inner attrs
            mematTrs.memtype = MemType_Normal;
            mematTrs.inner   = DecodeSDFAttr(c::b);
            mematTrs.outer   = DecodeSDFAttr(TEX[1:0]);

            if mematTrs.inner.attrs == MemAttr_NC && mematTrs.outer.attrs == MemAttr_NC then
                mematTrs.shareability = Shareability_OSH;
            else
                mematTrs.shareability = if s == '1' then Shareability_OSH else Shareability_NSH;
            end;
        otherwise =>
            // Reserved, handled above
            unreachable;
    end;

    // The Transient hint is not supported in this format
    mematTrs.inner.transient = FALSE;
    mematTrs.outer.transient = FALSE;
    mematTrs.tags            = MemTag_Untagged;

    if mematTrs.inner.attrs == MemAttr_WB && mematTrs.outer.attrs == MemAttr_WB then
        mematTrs.xs = '0';
    else
        mematTrs.xs = '1';
    end;
```

```
    return memattrs;
end;
```

Library pseudocode for aarch32/translation/attrs/AArch32_MAIRAttr

```
// AArch32_MAIRAttr()
// ==================
// Retrieve the memory attribute encoding indexed in the given MAIR

func AArch32_MAIRAttr(index : integer, mair : bits(64)) => bits(8)
begin
    assert (index < 8);
    return mair[index*:8];
end;
```

Library pseudocode for aarch32/translation/attrs/AArch32_RemappedTEXDecode

```
// AArch32_RemappedTEXDecode()
// ===========================
// Apply short-descriptor format memory region attributes, with TEX remap

func AArch32_RemappedTEXDecode(regime : Regime, TEX : bits(3),
                               c : bit, b : bit, s : bit) => MemoryAttributes
begin

    var mematTrs : MemoryAttributes;
    var prrr : PRRR_Type;
    var nmrr : NMRR_Type;

    let region : integer{} = UInt(TEX[0]::c::b);   // TEX[2:1] are ignored in this mapping scheme
    if region == 6 then
        return ImpDefMemoryAttributes("IMPLEMENTATION_DEFINED");
    end;

    if regime == Regime_EL30 then
        prrr = PRRR_S();
        nmrr = NMRR_S();
    elsif HaveEL(EL3) && ELUsingAArch32(EL3) then
        prrr = PRRR_NS();
        nmrr = NMRR_NS();
    else
        prrr = PRRR();
        nmrr = NMRR();
    end;

    let base : integer{} = 2 * region;
    var attrfield : bits(2) = prrr[region*:2];

    if attrfield == '11' then      // Reserved, maps to allocated value
        (-, attrfield) = ConstrainUnpredictableBits{2}(Unpredictable_RESPRRR);
    end;

    case attrfield of
        when '00' =>                  // Device-nGnRnE
            mematTrs.memtype      = MemType_Device;
            mematTrs.device       = DeviceType_nGnRnE;
            mematTrs.shareability = Shareability_OSH;
        when '01' =>                  // Device-nGnRE
            mematTrs.memtype      = MemType_Device;
            mematTrs.device       = DeviceType_nGnRE;
            mematTrs.shareability = Shareability_OSH;
        when '10' =>
            let NSn : bit  = if s == '0' then prrr.NS0 else prrr.NS1;
            let NOSm : bit = prrr[region+24] AND NSn;
            let IRn : bits(2)  = nmrr[base+1:base];
            let ORn : bits(2)  = nmrr[base+17:base+16];

            mematTrs.memtype = MemType_Normal;
            mematTrs.inner   = DecodeSDFAttr(IRn);
            mematTrs.outer   = DecodeSDFAttr(ORn);
            if mematTrs.inner.attrs == MemAttr_NC && mematTrs.outer.attrs == MemAttr_NC then
                mematTrs.shareability = Shareability_OSH;
            else
                let sh : bits(2) = NSn::NOSm;
                mematTrs.shareability = DecodeShareability(sh);
            end;
        when '11' =>
            unreachable;
    end;

    // The Transient hint is not supported in this format
    mematTrs.inner.transient = FALSE;
    mematTrs.outer.transient = FALSE;
    mematTrs.tags            = MemTag_Untagged;

    if mematTrs.inner.attrs == MemAttr_WB && mematTrs.outer.attrs == MemAttr_WB then
        mematTrs.xs = '0';
    else
        mematTrs.xs = '1';
    end;

    return mematTrs;
end;
```

Library pseudocode for aarch32/translation/attrs/ImpDefMemoryAttributes

```
// ImpDefMemoryAttributes()
// ========================
// Return an IMPLEMENTATION DEFINED set of memory attributes identified by the string parameter.

impdef func ImpDefMemoryAttributes(x : string) => MemoryAttributes
begin
    return ARBITRARY : MemoryAttributes;
end;
```

Library pseudocode for aarch32/translation/debug/AArch32_CheckBreakpoint

```
// AArch32_CheckBreakpoint()
// =========================
// Called before executing the instruction of length "size" bytes at "vaddress" in an AArch32
// translation regime, when either debug exceptions are enabled, or halting debug is enabled
// and halting is allowed.

func AArch32_CheckBreakpoint(fault_in : FaultRecord, vaddress : bits(32),
                             accdesc : AccessDescriptor, size : integer) => FaultRecord
begin
    assert ELUsingAArch32(S1TranslationRegime());
    assert size IN {2,4};

    var fault : FaultRecord = fault_in;
    var match : boolean = FALSE;
    var mismatch : boolean = FALSE;
    var brkptinfo : BreakpointInfo;

    for i = 0 to NumBreakpointsImplemented() - 1 do
        brkptinfo = AArch32_BreakpointMatch(i, vaddress, accdesc, size);
        match = match || brkptinfo.match;
        mismatch = mismatch || brkptinfo.mismatch;
    end;

    if match && HaltOnBreakpointOrWatchpoint() then
        let reason : bits(6) = DebugHalt_Breakpoint;
        Halt(reason);
    elsif (match || mismatch) then
        fault.statuscode = Fault_Debug;
        fault.debugmoe   = DebugException_Breakpoint;
        fault.vaddress   = ZeroExtend{64}(vaddress);
    end;

    return fault;
end;
```

Library pseudocode for aarch32/translation/debug/AArch32_CheckDebug

```
// AArch32_CheckDebug()
// ====================
// Called on each access to check for a debug exception or entry to Debug state.

func AArch32_CheckDebug(vaddress : bits(32), accdesc : AccessDescriptor,
                        size : integer) => FaultRecord
begin

    var fault : FaultRecord = NoFault(accdesc, ZeroExtend{64}(vaddress));

    let d_side : boolean = IsWatchpointableAccess(accdesc);
    let i_side : boolean = (accdesc.acctype == AccessType_IFETCH);
    let generate_exception : boolean = (AArch32_GenerateDebugExceptions() &&
                                        DBGDSCRext().MDBGen == '1');
    let halt : boolean = HaltOnBreakpointOrWatchpoint();
    // Relative priority of Vector Catch and Breakpoint exceptions not defined in the architecture
    let vector_catch_first : boolean = ConstrainUnpredictableBool(Unpredictable_BPVECTORCATCHPRI);

    if i_side && vector_catch_first && generate_exception then
        fault = AArch32_CheckVectorCatch(fault, vaddress, size);
    end;

    if fault.statuscode == Fault_None && (generate_exception || halt) then
        if d_side then
            fault = AArch32_CheckWatchpoint(fault, vaddress, accdesc, size);
        elsif i_side then
            fault = AArch32_CheckBreakpoint(fault, vaddress, accdesc, size);
        end;
    end;

    if fault.statuscode == Fault_None && i_side && !vector_catch_first && generate_exception then
        return AArch32_CheckVectorCatch(fault, vaddress, size);
    end;

    return fault;
end;
```

Library pseudocode for aarch32/translation/debug/AArch32_CheckVectorCatch

```
// AArch32_CheckVectorCatch()
// ==========================
// Called before executing the instruction of length "size" bytes at "vaddress" in an AArch32
// translation regime, when debug exceptions are enabled.

func AArch32_CheckVectorCatch(fault_in : FaultRecord, vaddress : bits(32),
                              size : integer) => FaultRecord
begin
    assert ELUsingAArch32(S1TranslationRegime());

    var fault : FaultRecord = fault_in;
    var match : boolean = AArch32_VCRMatch(vaddress);
    if size == 4 && !match && AArch32_VCRMatch(vaddress + 2) then
        match = ConstrainUnpredictableBool(Unpredictable_VCMATCHHALF);
    end;

    if match then
        fault.statuscode = Fault_Debug;
        fault.debugmoe   = DebugException_VectorCatch;
    end;

    return fault;
end;
```

Library pseudocode for aarch32/translation/debug/AArch32_CheckWatchpoint

```
// AArch32_CheckWatchpoint()
// =========================
// Called before accessing the memory location of "size" bytes at "address",
// when either debug exceptions are enabled for the access, or halting debug
// is enabled and halting is allowed.

func AArch32_CheckWatchpoint(fault_in : FaultRecord, vaddress : bits(32),
                             accdesc : AccessDescriptor, size : integer) => FaultRecord
begin
    assert ELUsingAArch32(S1TranslationRegime());
    var fault : FaultRecord = fault_in;
    var match : boolean = FALSE;

    if !IsWatchpointableAccess(accdesc) then
        return fault;
    end;

    for i = 0 to NumWatchpointsImplemented() - 1 do
        let watchptinfo : WatchpointInfo = AArch32_WatchpointMatch(i, vaddress, size, accdesc);
        match = match || watchptinfo.value_match;
    end;

    if match && HaltOnBreakpointOrWatchpoint() then
        let reason : bits(6) = DebugHalt_Watchpoint;
        EDWAR()  = ZeroExtend{64}(vaddress);
        Halt(reason);
    elsif match then
        fault.statuscode = Fault_Debug;
        fault.debugmoe   = DebugException_Watchpoint;
        fault.vaddress   = ZeroExtend{64}(vaddress);
    end;

    return fault;
end;
```

Library pseudocode for aarch32/translation/faults/AArch32_IPAIsOutOfRange

```
// AArch32_IPAIsOutOfRange()
// =========================
// Check intermediate physical address bits not resolved by translation are ZERO

func AArch32_IPAIsOutOfRange(walkparams : S2TTWParams, ipa : bits(40)) => boolean
begin
    // Input Address size
    let iasize : integer{} = AArch32_S2IASize(walkparams.t0sz);

    return iasize < 40 && !IsZero(ipa[39:iasize]);
end;
```

Library pseudocode for aarch32/translation/faults/AArch32_S1HasAlignmentFaultDueToMemType

```
// AArch32_S1HasAlignmentFaultDueToMemType()
// =========================================
// Returns whether stage 1 output fails alignment requirement on data accesses to memory type

func AArch32_S1HasAlignmentFaultDueToMemType(accdesc : AccessDescriptor, aligned : boolean,
                                             ntlsmd : bit, memattrs : MemoryAttributes) => boolean
begin
    if memattrs.memtype != MemType_Device then
        return FALSE;
    elsif accdesc.a32lsmd && ntlsmd == '0' then
        return memattrs.device != DeviceType_GRE;
    elsif accdesc.acctype == AccessType_DCZero then
        return TRUE;
    elsif !aligned then
        return !(ImpDefBool("Device location supports unaligned access"));
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch32/translation/faults/AArch32_S1LDHasPermissionsFault

```
// AArch32_S1LDHasPermissionsFault()
// =================================
// Returns whether an access using stage 1 long-descriptor translation
// violates permissions of target memory

func AArch32_S1LDHasPermissionsFault(regime : Regime, walkparams : S1TTWParams, perms : Permissions,
                                     memtype : MemType, paspace : PASpace,
                                     accdesc : AccessDescriptor) => boolean
begin
    var r,  w,  x : bit;
    var pr, pw : bit;
    var ur, uw : bit;
    var xn : bit;
    if HasUnprivileged(regime) then
        // Apply leaf permissions
        case perms.ap[2:1] of
            when '00' => (pr,pw,ur,uw) = ('1','1','0','0'); // R/W at PL1 only
            when '01' => (pr,pw,ur,uw) = ('1','1','1','1'); // R/W at any PL
            when '10' => (pr,pw,ur,uw) = ('1','0','0','0'); // RO at PL1 only
            when '11' => (pr,pw,ur,uw) = ('1','0','1','0'); // RO at any PL
        end;

        // Apply hierarchical permissions
        case perms.ap_table of
            when '00' => (pr,pw,ur,uw) = ( pr, pw, ur, uw); // No effect
            when '01' => (pr,pw,ur,uw) = ( pr, pw,'0','0'); // Privileged access
            when '10' => (pr,pw,ur,uw) = ( pr,'0', ur,'0'); // Read-only
            when '11' => (pr,pw,ur,uw) = ( pr,'0','0','0'); // Read-only, privileged access
        end;

        xn   = perms.xn OR perms.xn_table;
        let pxn : bit  = perms.pxn OR perms.pxn_table;

        let ux : bit = ur AND NOT(xn OR (uw AND walkparams.wxn));
        let px : bit = pr AND NOT(xn OR pxn OR (pw AND walkparams.wxn) OR (uw AND walkparams.uwxn));

        if IsFeatureImplemented(FEAT_PAN) && accdesc.pan then
            let pan : bit = PSTATE.PAN AND (ur OR uw);
            pr  = pr AND NOT(pan);
            pw  = pw AND NOT(pan);
        end;

        (r,w,x) = if accdesc.el == EL0 then (ur,uw,ux) else (pr,pw,px);

        // Prevent execution from Non-secure space by PE in Secure state if SIF is set
        if accdesc.ss == SS_Secure && paspace == PAS_NonSecure then
            x = x AND NOT(walkparams.sif);
        end;
    else
        // Apply leaf permissions
        case perms.ap[2] of
            when '0' => (r,w) = ('1','1'); // No effect
            when '1' => (r,w) = ('1','0'); // Read-only
        end;

        // Apply hierarchical permissions
        case perms.ap_table[1] of
            when '0' => (r,w) = ( r, w );  // No effect
            when '1' => (r,w) = ( r, '0'); // Read-only
        end;

        xn = perms.xn OR perms.xn_table;
        x  = NOT(xn OR (w AND walkparams.wxn));
    end;

    if accdesc.acctype == AccessType_IFETCH then
        let constraint : Constraint = ConstrainUnpredictable(Unpredictable_INSTRDEVICE);
        if constraint == Constraint_FAULT && memtype == MemType_Device then
            return TRUE;
        else
            return x == '0';
        end;
    elsif accdesc.acctype IN {AccessType_IC, AccessType_DC} then
        return FALSE;
    elsif accdesc.write then
        return w == '0';
    else
        return r == '0';
    end;
end;
```

Library pseudocode for aarch32/translation/faults/AArch32_S1SDHasPermissionsFault

```
// AArch32_S1SDHasPermissionsFault()
// =================================
// Returns whether an access using stage 1 short-descriptor translation
// violates permissions of target memory

func AArch32_S1SDHasPermissionsFault(regime : Regime, perms_in : Permissions, memtype : MemType,
                                     paspace : PASpace, accdesc : AccessDescriptor) => boolean
begin
    var perms : Permissions = perms_in;
    var pr, pw : bit;
    var ur, uw : bit;
    var sctlr : SCTLR_Type;
    if regime == Regime_EL30 then
        sctlr = SCTLR_S();
    elsif HaveEL(EL3) && ELUsingAArch32(EL3) then
        sctlr = SCTLR_NS();
    else
        sctlr = SCTLR();
    end;

    if sctlr.AFE == '0' then
        // Map Reserved encoding '100'
        if perms.ap == '100' then
            perms.ap = ImpDefBits{3}("Reserved short descriptor AP encoding");
        end;

        case perms.ap of
            when '000' => (pr,pw,ur,uw) = ('0','0','0','0'); // No access
            when '001' => (pr,pw,ur,uw) = ('1','1','0','0'); // R/W at PL1 only
            when '010' => (pr,pw,ur,uw) = ('1','1','1','0'); // R/W at PL1, RO at PL0
            when '011' => (pr,pw,ur,uw) = ('1','1','1','1'); // R/W at any PL
            //   '100' is reserved
            when '101' => (pr,pw,ur,uw) = ('1','0','0','0'); // RO at PL1 only
            when '110' => (pr,pw,ur,uw) = ('1','0','1','0'); // RO at any PL (deprecated)
            when '111' => (pr,pw,ur,uw) = ('1','0','1','0'); // RO at any PL
        end;
    else // Simplified access permissions model
        case perms.ap[2:1] of
            when '00' => (pr,pw,ur,uw) = ('1','1','0','0'); // R/W at PL1 only
            when '01' => (pr,pw,ur,uw) = ('1','1','1','1'); // R/W at any PL
            when '10' => (pr,pw,ur,uw) = ('1','0','0','0'); // RO at PL1 only
            when '11' => (pr,pw,ur,uw) = ('1','0','1','0'); // RO at any PL
        end;
    end;

    let ux : bit = ur AND NOT(perms.xn OR (uw AND sctlr.WXN));
    let px : bit = pr AND NOT(perms.xn OR perms.pxn OR (pw AND sctlr.WXN) OR (uw AND sctlr.UWXN));

    if IsFeatureImplemented(FEAT_PAN) && accdesc.pan then
        let pan : bit = PSTATE.PAN AND (ur OR uw);
        pr  = pr AND NOT(pan);
        pw  = pw AND NOT(pan);
    end;

    var (r,w,x) : (bit, bit, bit) = if accdesc.el == EL0 then (ur,uw,ux) else (pr,pw,px);

    // Prevent execution from Non-secure space by PE in Secure state if SIF is set
    if accdesc.ss == SS_Secure && paspace == PAS_NonSecure then
        x = x AND NOT(if ELUsingAArch32(EL3) then SCR().SIF else SCR_EL3().SIF);
    end;

    if accdesc.acctype == AccessType_IFETCH then
        if (memtype == MemType_Device &&
                ConstrainUnpredictable(Unpredictable_INSTRDEVICE) == Constraint_FAULT) then
            return TRUE;
        else
            return x == '0';
        end;
    elsif accdesc.acctype IN {AccessType_IC, AccessType_DC} then
        return FALSE;
    elsif accdesc.write then
        return w == '0';
    else
        return r == '0';
    end;
end;
```

Library pseudocode for aarch32/translation/faults/AArch32_S2HasAlignmentFaultDueToMemType

```
// AArch32_S2HasAlignmentFaultDueToMemType()
// =========================================
// Returns whether stage 2 output fails alignment requirement on data accesses due to memory type

func AArch32_S2HasAlignmentFaultDueToMemType(accdesc : AccessDescriptor, aligned : boolean,
                                             memattrs : MemoryAttributes) => boolean
begin

    if memattrs.memtype != MemType_Device then
        return FALSE;
    elsif accdesc.acctype == AccessType_DCZero then
        return TRUE;
    elsif !aligned then
        return !(ImpDefBool("Device location supports unaligned access"));
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch32/translation/faults/AArch32_S2HasPermissionsFault

```
// AArch32_S2HasPermissionsFault()
// ===============================
// Returns whether stage 2 access violates permissions of target memory

func AArch32_S2HasPermissionsFault(walkparams : S2TTWParams, perms : Permissions, memtype : MemType,
                                   accdesc : AccessDescriptor) => boolean
begin
    let r : bit = perms.s2ap[0];
    let w : bit = perms.s2ap[1];
    var x : bit;
    var ux : bit;
    var px : bit;
    if IsFeatureImplemented(FEAT_XNX) then
        case perms.s2xn::perms.s2xnx of
            when '00' =>  (px, ux) = ( r, r );
            when '01' =>  (px, ux) = ('0', r );
            when '10' =>  (px, ux) = ('0', '0');
            when '11' =>  (px, ux) = ( r, '0');
        end;

        x = if accdesc.el == EL0 then ux else px;
    else
        x = r AND NOT(perms.s2xn);
    end;

    if accdesc.acctype == AccessType_TTW then
        return (walkparams.ptw == '1' && memtype == MemType_Device) || r == '0';

    elsif accdesc.acctype == AccessType_IFETCH then
        let constraint : Constraint = ConstrainUnpredictable(Unpredictable_INSTRDEVICE);
        return (constraint == Constraint_FAULT && memtype == MemType_Device) || x == '0';

    elsif accdesc.acctype IN {AccessType_IC, AccessType_DC} then
        return FALSE;

    elsif accdesc.write then
        return w == '0';

    else
        return r == '0';
    end;
end;
```

Library pseudocode for aarch32/translation/faults/AArch32_S2InconsistentSL

```
// AArch32_S2InconsistentSL()
// ==========================
// Detect inconsistent configuration of stage 2 T0SZ and SL fields

func AArch32_S2InconsistentSL(walkparams : S2TTWParams) => boolean
begin
    let startlevel : integer  = AArch32_S2StartLevel(walkparams.sl0);
    let levels : integer      = FINAL_LEVEL - startlevel;
    let granulebits : AddressSize = TGxGranuleBits(walkparams.tgx);
    let stride : integer{}      = granulebits - 3;

    // Input address size must at least be large enough to be resolved from the start level
    let sl_min_iasize : integer = (
        levels * stride // Bits resolved by table walk, except initial level
        + granulebits   // Bits directly mapped to output address
        + 1);           // At least 1 more bit to be decoded by initial level

    // Can accommodate 1 more stride in the level + concatenation of up to 2^4 tables
    let sl_max_iasize : integer = sl_min_iasize + (stride-1) + 4;
    // Configured Input Address size
    let iasize : AddressSize        = AArch32_S2IASize(walkparams.t0sz);

    return iasize < sl_min_iasize || iasize > sl_max_iasize;
end;
```

Library pseudocode for aarch32/translation/faults/AArch32_VAIsOutOfRange

```
// AArch32_VAIsOutOfRange()
// ========================
// Check virtual address bits not resolved by translation are identical
// and of accepted value

func AArch32_VAIsOutOfRange(regime : Regime, walkparams : S1TTWParams, va : bits(32)) => boolean
begin
    if regime == Regime_EL2 then
        // Input Address size
        let iasize : integer{} = AArch32_S1IASize(walkparams.t0sz);
        return walkparams.t0sz != '000' && !IsZero(va[31:iasize]);
    elsif walkparams.t1sz != '000' && walkparams.t0sz != '000' then
        // Lower range Input Address size
        let lo_iasize : integer{} = AArch32_S1IASize(walkparams.t0sz);
        // Upper range Input Address size
        let up_iasize : integer{} = AArch32_S1IASize(walkparams.t1sz);
        return !IsZero(va[31:lo_iasize]) && !IsOnes(va[31:up_iasize]);
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch32/translation/tlbcontext/AArch32_GetS1TLBContext

```
// AArch32_GetS1TLBContext()
// =========================
// Gather translation context for accesses with VA to match against TLB entries

func AArch32_GetS1TLBContext(regime : Regime, ss : SecurityState, va : bits(32)) => TLBContext
begin
    var tlbcontext : TLBContext;

    case regime of
        when Regime_EL2  => tlbcontext = AArch32_TLBContextEL2(va);
        when Regime_EL10 => tlbcontext = AArch32_TLBContextEL10(ss, va);
        when Regime_EL30 => tlbcontext = AArch32_TLBContextEL30(va);
    end;

    tlbcontext.includes_s1 = TRUE;
    // The following may be amended for EL1&0 Regime if caching of stage 2 is successful
    tlbcontext.includes_s2 = FALSE;
    return tlbcontext;
end;
```

Library pseudocode for aarch32/translation/tlbcontext/AArch32_GetS2TLBContext

```
// AArch32_GetS2TLBContext()
// =========================
// Gather translation context for accesses with IPA to match against TLB entries

func AArch32_GetS2TLBContext(ipa : FullAddress) => TLBContext
begin
    assert ipa.paspace == PAS_NonSecure;

    var tlbcontext : TLBContext;

    tlbcontext.ss          = SS_NonSecure;
    tlbcontext.regime      = Regime_EL10;
    tlbcontext.ipaspace    = ipa.paspace;
    tlbcontext.vmid        = ZeroExtend{NUM_VMIDBITS}(VTTBR().VMID);
    tlbcontext.tg          = TGx_4KB;
    tlbcontext.includes_s1 = FALSE;
    tlbcontext.includes_s2 = TRUE;
    tlbcontext.ia          = ZeroExtend{64}(ipa.address);
    tlbcontext.cnp         = if IsFeatureImplemented(FEAT_TTCNP) then VTTBR().CnP else '0';

    return tlbcontext;
end;
```

Library pseudocode for aarch32/translation/tlbcontext/AArch32_TLBContextEL10

```
// AArch32_TLBContextEL10()
// ========================
// Gather translation context for accesses under EL10 regime
// (PL10 when EL3 is A64) to match against TLB entries

func AArch32_TLBContextEL10(ss : SecurityState, va : bits(32)) => TLBContext
begin
    var tlbcontext : TLBContext;
    var ttbcr : TTBCR_Type;
    var ttbr0 : TTBR0_Type;
    var ttbr1 : TTBR1_Type;
    var contextidr : CONTEXTIDR_Type;

    if HaveEL(EL3) && ELUsingAArch32(EL3) then
        ttbcr      = TTBCR_NS();
        ttbr0      = TTBR0_NS();
        ttbr1      = TTBR1_NS();
        contextidr = CONTEXTIDR_NS();
    else
        ttbcr      = TTBCR();
        ttbr0      = TTBR0();
        ttbr1      = TTBR1();
        contextidr = CONTEXTIDR();
    end;

    tlbcontext.ss     = ss;
    tlbcontext.regime = Regime_EL10;

    if AArch32_EL2Enabled(ss) then
        tlbcontext.vmid = ZeroExtend{NUM_VMIDBITS}(VTTBR().VMID);
    end;

    if ttbcr.EAE == '1' then
        if ttbcr.A1 == '0' then
            tlbcontext.asid = ZeroExtend{NUM_ASIDBITS}(ttbr0.ASID);
        else
            tlbcontext.asid = ZeroExtend{NUM_ASIDBITS}(ttbr1.ASID);
        end;
    else
        tlbcontext.asid = ZeroExtend{NUM_ASIDBITS}(contextidr.ASID);
    end;

    tlbcontext.tg = TGx_4KB;
    tlbcontext.ia = ZeroExtend{64}(va);

    if IsFeatureImplemented(FEAT_TTCNP) && ttbcr.EAE == '1' then
        if AArch32_GetVARange(va, ttbcr.T0SZ, ttbcr.T1SZ) == VARange_LOWER then
            tlbcontext.cnp = ttbr0.CnP;
        else
            tlbcontext.cnp = ttbr1.CnP;
        end;
    else
        tlbcontext.cnp = '0';
    end;

    return tlbcontext;
end;
```

Library pseudocode for aarch32/translation/tlbcontext/AArch32_TLBContextEL2

```
// AArch32_TLBContextEL2()
// =======================
// Gather translation context for accesses under EL2 regime to match against TLB entries

func AArch32_TLBContextEL2(va : bits(32)) => TLBContext
begin
    var tlbcontext : TLBContext;

    tlbcontext.ss     = SS_NonSecure;
    tlbcontext.regime = Regime_EL2;
    tlbcontext.ia     = ZeroExtend{64}(va);
    tlbcontext.tg     = TGx_4KB;
    tlbcontext.cnp    = if IsFeatureImplemented(FEAT_TTCNP) then HTTBR().CnP else '0';

    return tlbcontext;
end;
```

Library pseudocode for aarch32/translation/tlbcontext/AArch32_TLBContextEL30

```
// AArch32_TLBContextEL30()
// ========================
// Gather translation context for accesses under EL30 regime
// (PL10 in Secure state and EL3 is A32) to match against TLB entries

func AArch32_TLBContextEL30(va : bits(32)) => TLBContext
begin
    var tlbcontext : TLBContext;

    tlbcontext.ss     = SS_Secure;
    tlbcontext.regime = Regime_EL30;
    if TTBCR_S().EAE == '1' then
        tlbcontext.asid = ZeroExtend{NUM_ASIDBITS}(if TTBCR_S().A1 == '0' then TTBR0_S().ASID
                                                                          else TTBR1_S().ASID);
    else
        tlbcontext.asid = ZeroExtend{NUM_ASIDBITS}(CONTEXTIDR_S().ASID);
    end;

    tlbcontext.tg = TGx_4KB;
    tlbcontext.ia = ZeroExtend{64}(va);

    if IsFeatureImplemented(FEAT_TTCNP) && TTBCR_S().EAE == '1' then
        if AArch32_GetVARange(va, TTBCR_S().T0SZ, TTBCR_S().T1SZ) == VARange_LOWER then
            tlbcontext.cnp = TTBR0_S().CnP;
        else
            tlbcontext.cnp = TTBR1_S().CnP;
        end;
    else
        tlbcontext.cnp = '0';
    end;

    return tlbcontext;
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_EL2Enabled

```
// AArch32_EL2Enabled()
// ====================
// When using VMSAv8-32, returns whether EL2 is enabled for the given Security State.

func AArch32_EL2Enabled(ss : SecurityState) => boolean
begin
    assert ELUsingAArch32(EL1);
    if ss == SS_Secure then
        if !(HaveEL(EL2) && IsFeatureImplemented(FEAT_SEL2)) then
            return FALSE;
        elsif HaveEL(EL3) then
            return SCR_EL3().EEL2 == '1';
        else
            return SecureOnlyImplementation();
        end;
    else
        return HaveEL(EL2);
    end;
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_FullTranslate

```
// AArch32_FullTranslate()
// =======================
// Perform address translation as specified by VMSA-A32

func AArch32_FullTranslate(va : bits(32), accdesc : AccessDescriptor,
                           aligned : boolean) => AddressDescriptor
begin

    // Prepare fault fields in case a fault is detected
    var fault : FaultRecord = NoFault(accdesc, ZeroExtend{64}(va));
    let regime : Regime = TranslationRegime(accdesc.el);

    // First Stage Translation
    var ipa : AddressDescriptor;
    if regime == Regime_EL2 || TTBCR().EAE == '1' then
        (fault, ipa)    = AArch32_S1TranslateLD(fault, regime, va, aligned, accdesc);
    else
        (fault, ipa, -) = AArch32_S1TranslateSD(fault, regime, va, aligned, accdesc);
    end;

    if fault.statuscode != Fault_None then
        return CreateFaultyAddressDescriptor(fault);
    end;

    if regime == Regime_EL10 && EL2Enabled() then
        ipa.vaddress = ZeroExtend{64}(va);
        var pa : AddressDescriptor;
        (fault, pa) = AArch32_S2Translate(fault, ipa, aligned, accdesc);

        if fault.statuscode != Fault_None then
            return CreateFaultyAddressDescriptor(fault);
        else
            return pa;
        end;
    else
        return ipa;
    end;
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_OutputDomain

```
// AArch32_OutputDomain()
// ======================
// Determine the domain the translated output address

func AArch32_OutputDomain(regime : Regime, domain : bits(4)) => bits(2)
begin
    var Dn : bits(2);
    if regime == Regime_EL30 then
        Dn = DACR_S()[UInt(domain)*:2];
    elsif HaveEL(EL3) && ELUsingAArch32(EL3) then
        Dn = DACR_NS()[UInt(domain)*:2];
    else
        Dn = DACR()[UInt(domain)*:2];
    end;

    if Dn == '10' then
        // Reserved value maps to an allocated value
        (-, Dn) = ConstrainUnpredictableBits{2}(Unpredictable_RESDACR);
    end;

    return Dn;
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_S1DisabledOutput

```
// AArch32_S1DisabledOutput()
// ==========================
// Flat map the VA to IPA/PA, depending on the regime, assigning default memory attributes

func AArch32_S1DisabledOutput(fault_in : FaultRecord, regime : Regime,
                              va : bits(32), aligned : boolean,
                              accdesc : AccessDescriptor) => (FaultRecord, AddressDescriptor)
begin

    var fault : FaultRecord = fault_in;
    // No memory page is guarded when stage 1 address translation is disabled
    SetInGuardedPage(FALSE);

    var mematTrs : MemoryAttributes;
    var default_cacheable : bit;
    if regime == Regime_EL10 && AArch32_EL2Enabled(accdesc.ss) then
        if ELStateUsingAArch32(EL2, accdesc.ss == SS_Secure) then
            default_cacheable = HCR().DC;
        else
            default_cacheable = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().DC else '0';
        end;
    else
        default_cacheable = '0';
    end;

    if default_cacheable == '1' then
        // Use default cacheable settings
        mematTrs.memtype      = MemType_Normal;
        mematTrs.inner.attrs  = MemAttr_WB;
        mematTrs.inner.hints  = MemHint_RWA;
        mematTrs.outer.attrs  = MemAttr_WB;
        mematTrs.outer.hints  = MemHint_RWA;
        mematTrs.shareability = Shareability_NSH;
        mematTrs.tags     = MemTag_Untagged;
        mematTrs.xs           = '0';
    elsif accdesc.acctype == AccessType_IFETCH then
        mematTrs.memtype      = MemType_Normal;
        mematTrs.shareability = Shareability_OSH;
        mematTrs.tags         = MemTag_Untagged;
        if AArch32_S1ICacheEnabled(regime) then
            mematTrs.inner.attrs = MemAttr_WT;
            mematTrs.inner.hints = MemHint_RA;
            mematTrs.outer.attrs = MemAttr_WT;
            mematTrs.outer.hints = MemHint_RA;
        else
            mematTrs.inner.attrs = MemAttr_NC;
            mematTrs.outer.attrs = MemAttr_NC;
        end;
        mematTrs.xs           = '1';
    else
        // Treat memory region as Device
        mematTrs.memtype      = MemType_Device;
        mematTrs.device       = DeviceType_nGnRnE;
        mematTrs.shareability = Shareability_OSH;
        mematTrs.tags         = MemTag_Untagged;
        mematTrs.xs           = '1';
    end;

    var ntlsmd : bit;
    if IsFeatureImplemented(FEAT_LSMAOC) then
        case regime of
            when Regime_EL30 => ntlsmd = SCTLR_S().nTLSMD;
            when Regime_EL2  => ntlsmd = HSCTLR().nTLSMD;
            when Regime_EL10 =>
                if HaveEL(EL3) && ELUsingAArch32(EL3) then
                    ntlsmd = SCTLR_NS().nTLSMD;
                else
                    ntlsmd = SCTLR().nTLSMD;
                end;
        end;
    else
        ntlsmd = '1';
    end;

    if AArch32_S1HasAlignmentFaultDueToMemType(accdesc, aligned, ntlsmd, mematTrs) then
        fault.statuscode  = Fault_Alignment;
        return (fault, ARBITRARY : AddressDescriptor);
    elsif (accdesc.exclusive &&
           !(regime == Regime_EL10 && EL2Enabled() &&
             ((!ELUsingAArch32(EL2) && IsFeatureImplemented(FEAT_HCRMAC) && HCR_EL2().DC == '1') ||
                (ELUsingAArch32(EL2) && HCR().DC == '1'))) &&
           ConstrainUnpredictableBool(Unpredictable_Atomic_MMU_IMPDEF_FAULT)) then
        fault.statuscode = Fault_Exclusive;
    end;

    var oa : FullAddress;
    oa.address = ZeroExtend{NUM_PABITS}(va);
    oa.paspace = if accdesc.ss == SS_Secure then PAS_Secure else PAS_NonSecure;
    let ipa : AddressDescriptor = CreateAddressDescriptor(ZeroExtend{64}(va),
```

```
                                                        oa,
                                                        memattrs,
                                                        accdesc);

    return (fault, ipa);
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_S1Enabled

```
// AArch32_S1Enabled()
// ===================
// Returns whether stage 1 translation is enabled for the active translation regime

func AArch32_S1Enabled(regime : Regime, ss : SecurityState) => boolean
begin
    if regime == Regime_EL2 then
        return HSCTLR().M == '1';
    elsif regime == Regime_EL30 then
        return SCTLR_S().M == '1';
    elsif !AArch32_EL2Enabled(ss) then
        return (if HaveEL(EL3) && ELUsingAArch32(EL3) then SCTLR_NS().M else SCTLR().M) == '1';
    elsif ELStateUsingAArch32(EL2, ss == SS_Secure) then
        if HaveEL(EL3) && ELUsingAArch32(EL3) then
            return HCR().[TGE,DC] == '00' && SCTLR_NS().M == '1';
        else
            return HCR().[TGE,DC] == '00' && SCTLR().M == '1';
        end;
    else
        return EL2Enabled() && HCR_EL2().[TGE,DC] == '00' && SCTLR().M == '1';
    end;
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_S1TranslateLD

```
// AArch32_S1TranslateLD()
// =======================
// Perform a stage 1 translation using long-descriptor format mapping VA to IPA/PA
// depending on the regime

func AArch32_S1TranslateLD(fault_in : FaultRecord, regime : Regime,
                           va : bits(32), aligned : boolean,
                           accdesc : AccessDescriptor) => (FaultRecord, AddressDescriptor)
begin
    var fault : FaultRecord = fault_in;

    if !AArch32_S1Enabled(regime, accdesc.ss) then
        return AArch32_S1DisabledOutput(fault, regime, va, aligned, accdesc);
    end;

    let walkparams : S1TTWParams = AArch32_GetS1TTWParams(regime, va);

    if AArch32_VAIsOutOfRange(regime, walkparams, va) then
        fault.level      = 1;
        fault.statuscode = Fault_Translation;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    var walkstate : TTWState;
    (fault, walkstate) = AArch32_S1WalkLD(fault, regime, walkparams, accdesc, va);

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    SetInGuardedPage(FALSE);  // AArch32-VMSA does not guard any pages

    if AArch32_S1HasAlignmentFaultDueToMemType(accdesc, aligned, walkparams.ntlsmd,
                                               walkstate.mematTrs) then
        fault.statuscode = Fault_Alignment;
    elsif AArch32_S1LDHasPermissionsFault(regime, walkparams,
                                          walkstate.permissions,
                                          walkstate.mematTrs.memtype,
                                          walkstate.baseaddress.paspace,
                                          accdesc) then
        fault.statuscode = Fault_Permission;
    end;

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    var mematTrs : MemoryAttributes;
    if ((accdesc.acctype == AccessType_IFETCH &&
            (walkstate.mematTrs.memtype == MemType_Device || !AArch32_S1ICacheEnabled(regime))) ||
        (accdesc.acctype != AccessType_IFETCH &&
             walkstate.mematTrs.memtype == MemType_Normal && !AArch32_S1DCacheEnabled(regime))) then
        // Treat memory attributes as Normal Non-Cacheable
        mematTrs = NormalNCMemAttr();
        mematTrs.xs = walkstate.mematTrs.xs;
    else
        mematTrs = walkstate.mematTrs;
    end;

    // Shareability value of stage 1 translation subject to stage 2 is IMPLEMENTATION DEFINED
    // to be either effective value or descriptor value
    if (regime == Regime_EL10 && AArch32_EL2Enabled(accdesc.ss) &&
        (if ELStateUsingAArch32(EL2, accdesc.ss==SS_Secure) then HCR().VM
                                                            else HCR_EL2().VM) == '1' &&
        !(ImpDefBool("Apply effective shareability at stage 1"))) then
        mematTrs.shareability = walkstate.mematTrs.shareability;
    else
        mematTrs.shareability = EffectiveShareability(mematTrs);
    end;

    // Output Address
    let oa : FullAddress = Stage1OA(ZeroExtend{64}(va), walkparams, walkstate);
    let ipa : AddressDescriptor = CreateAddressDescriptor(ZeroExtend{64}(va),
                                                          oa, mematTrs, accdesc);

    return (fault, ipa);
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_S1TranslateSD

```
// AArch32_S1TranslateSD()
// =======================
// Perform a stage 1 translation using short-descriptor format mapping VA to IPA/PA
// depending on the regime

func AArch32_S1TranslateSD(fault_in : FaultRecord, regime : Regime,
                           va : bits(32), aligned : boolean,
                           accdesc : AccessDescriptor) => (FaultRecord, AddressDescriptor, SDFType)
begin

    var fault : FaultRecord = fault_in;

    if !AArch32_S1Enabled(regime, accdesc.ss) then
        var ipa : AddressDescriptor;
        (fault, ipa) = AArch32_S1DisabledOutput(fault, regime, va, aligned, accdesc);
        return (fault, ipa, ARBITRARY : SDFType);
    end;

    var walkstate : TTWState;
    (fault, walkstate) = AArch32_S1WalkSD(fault, regime, accdesc, va);

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : SDFType);
    end;

    let domain : bits(2) = AArch32_OutputDomain(regime, walkstate.domain);
    SetInGuardedPage(FALSE);  // AArch32-VMSA does not guard any pages

    var ntlsmd : bit;
    if IsFeatureImplemented(FEAT_LSMAOC) then
        case regime of
            when Regime_EL30 => ntlsmd = SCTLR_S().nTLSMD;
            when Regime_EL10 =>
                if HaveEL(EL3) && ELUsingAArch32(EL3) then
                    ntlsmd = SCTLR_NS().nTLSMD;
                else
                    ntlsmd = SCTLR().nTLSMD;
                end;
        end;
    else
        ntlsmd = '1';
    end;

    if AArch32_S1HasAlignmentFaultDueToMemType(accdesc, aligned, ntlsmd, walkstate.mematTrs) then
        fault.statuscode = Fault_Alignment;
    elsif (! accdesc.acctype IN {AccessType_IC, AccessType_DC} &&
           domain == Domain_NoAccess) then
        fault.statuscode = Fault_Domain;
    elsif domain == Domain_Client then
        if AArch32_S1SDHasPermissionsFault(regime, walkstate.permissions,
                                           walkstate.mematTrs.memtype,
                                           walkstate.baseaddress.paspace,
                                           accdesc) then
            fault.statuscode = Fault_Permission;
        end;
    end;

    if fault.statuscode != Fault_None then
        fault.domain = walkstate.domain;
        return (fault, ARBITRARY : AddressDescriptor, walkstate.sdftype);
    end;

    var mematTrs : MemoryAttributes;
    if ((accdesc.acctype == AccessType_IFETCH &&
         (walkstate.mematTrs.memtype == MemType_Device || !AArch32_S1ICacheEnabled(regime))) ||
        (accdesc.acctype != AccessType_IFETCH &&
         walkstate.mematTrs.memtype == MemType_Normal && !AArch32_S1DCacheEnabled(regime))) then
        // Treat memory attributes as Normal Non-Cacheable
        mematTrs = NormalNCMemAttr();
        mematTrs.xs = walkstate.mematTrs.xs;
    else
        mematTrs = walkstate.mematTrs;
    end;
    // Shareability value of stage 1 translation subject to stage 2 is IMPLEMENTATION DEFINED
    // to be either effective value or descriptor value
    if (regime == Regime_EL10 && AArch32_EL2Enabled(accdesc.ss) &&
        (if ELStateUsingAArch32(EL2, accdesc.ss==SS_Secure) then HCR().VM
                                                            else HCR_EL2().VM) == '1' &&
        !(ImpDefBool("Apply effective shareability at stage 1"))) then
        mematTrs.shareability = walkstate.mematTrs.shareability;
    else
        mematTrs.shareability = EffectiveShareability(mematTrs);
    end;
    // Output Address
    let oa : FullAddress = AArch32_SDStageOA(walkstate.baseaddress, va, walkstate.sdftype);
    let ipa : AddressDescriptor = CreateAddressDescriptor(ZeroExtend{64}(va),
                                                          oa, mematTrs, accdesc);
```

```
    return (fault, ipa, walkstate.sdftype);
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_S2Translate

```
// AArch32_S2Translate()
// =====================
// Perform a stage 2 translation mapping an IPA to a PA

func AArch32_S2Translate(fault_in : FaultRecord, ipa : AddressDescriptor,
                         aligned : boolean, accdesc : AccessDescriptor
                        ) => (FaultRecord, AddressDescriptor)
begin

    var fault : FaultRecord = fault_in;
    assert IsZero(ipa.paddress.address[NUM_PABITS-1:40]);
    if !ELStateUsingAArch32(EL2, accdesc.ss == SS_Secure) then
        let s1aarch64 : boolean = FALSE;
        return AArch64_S2Translate(fault, ipa, s1aarch64, aligned, accdesc);
    end;
    // Prepare fault fields in case a fault is detected
    fault.statuscode  = Fault_None;
    fault.secondstage = TRUE;
    fault.s2fs1walk   = accdesc.acctype == AccessType_TTW;
    fault.ipaddress   = ipa.paddress;

    let walkparams : S2TTWParams = AArch32_GetS2TTWParams();

    if walkparams.vm == '0' then
        // Stage 2 is disabled
        return (fault, ipa);
    end;

    if AArch32_IPAIsOutOfRange(walkparams, ipa.paddress.address[39:0]) then
        fault.statuscode = Fault_Translation;
        fault.level      = 1;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    var walkstate : TTWState;
    (fault, walkstate) = AArch32_S2Walk(fault, walkparams, accdesc, ipa);

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if AArch32_S2HasAlignmentFaultDueToMemType(accdesc, aligned, walkstate.mematttrs) then
        fault.statuscode = Fault_Alignment;
    elsif AArch32_S2HasPermissionsFault(walkparams,
                                        walkstate.permissions,
                                        walkstate.mematttrs.memtype,
                                        accdesc) then
        fault.statuscode = Fault_Permission;
    end;
    var s2_mematttrs : MemoryAttributes;
    if ((accdesc.acctype == AccessType_TTW &&
         walkstate.mematttrs.memtype == MemType_Device) ||
        (accdesc.acctype == AccessType_IFETCH &&
            (walkstate.mematttrs.memtype == MemType_Device || HCR2().ID == '1')) ||
        (accdesc.acctype != AccessType_IFETCH &&
             walkstate.mematttrs.memtype == MemType_Normal && HCR2().CD == '1')) then
        // Treat memory attributes as Normal Non-Cacheable
        s2_mematttrs = NormalNCMemAttr();
        s2_mematttrs.xs = walkstate.mematttrs.xs;
    else
        s2_mematttrs = walkstate.mematttrs;
    end;

    let s2aarch64 : boolean = FALSE;
    let mematttrs : MemoryAttributes = S2CombineS1MemAttrs(ipa.mematttrs, s2_mematttrs, s2aarch64);
    // Output Address
    let oa : FullAddress = Stage2OA(ipa.paddress.address, walkparams, walkstate);
    let pa : AddressDescriptor = CreateAddressDescriptor(ipa.vaddress, oa, mematttrs, accdesc);

    return (fault, pa);
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_SDStageOA

```
// AArch32_SDStageOA()
// ===================
// Given the final walk state of a short-descriptor translation walk,
// map the untranslated input address bits to the base output address

func AArch32_SDStageOA(baseaddress : FullAddress, va : bits(32), sdftype : SDFType) => FullAddress
begin
    let tsize : integer{} = SDFSize(sdftype);

    // Output Address
    var oa : FullAddress;
    oa.address = baseaddress.address[NUM_PABITS-1:tsize] :: va[tsize-1:0];
    oa.paspace = baseaddress.paspace;
    return oa;
end;
```

Library pseudocode for aarch32/translation/translation/AArch32_TranslateAddress

```
// AArch32_TranslateAddress()
// ==========================
// Main entry point for translating an address

func AArch32_TranslateAddress(va : bits(32), accdesc : AccessDescriptor,
                              aligned : boolean, size : integer) => AddressDescriptor
begin

    let regime : Regime = TranslationRegime(PSTATE.EL);
    if !RegimeUsingAArch32(regime) then
        return AArch64_TranslateAddress(ZeroExtend{64}(va), accdesc, aligned, size);
    end;

    var result : AddressDescriptor = AArch32_FullTranslate(va, accdesc, aligned);

    if !IsFault(result) && accdesc.acctype != AccessType_IFETCH then
        // For an instruction fetch, CheckDebug will be called
        // after the instruction is read from memory
        result.fault = AArch32_CheckDebug(va, accdesc, size);
    end;

    // Update virtual address for abort functions
    result.vaddress = ZeroExtend{64}(va);

    return result;
end;
```

Library pseudocode for aarch32/translation/translation/SDFSize

```
// SDFSize()
// =========
// Returns the short-descriptor format translation granule size

func SDFSize(sdftype : SDFType) => AddressSize
begin
    case sdftype of
        when SDFType_SmallPage =>    return 12;
        when SDFType_LargePage =>    return 16;
        when SDFType_Section =>      return 20;
        when SDFType_Supersection => return 24;
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for aarch32/translation/walk/AArch32_DecodeDescriptorTypeLD

```
// AArch32_DecodeDescriptorTypeLD()
// ================================
// Determine whether the long-descriptor is a page, block or table

func AArch32_DecodeDescriptorTypeLD(descriptor : bits(64), level : integer) => DescriptorType
begin
    if descriptor[1:0] == '11' && level == FINAL_LEVEL then
        return DescriptorType_Leaf;
    elsif descriptor[1:0] == '11' then
        return DescriptorType_Table;
    elsif descriptor[1:0] == '01' && level != FINAL_LEVEL then
        return DescriptorType_Leaf;
    else
        return DescriptorType_Invalid;
    end;
end;
```

Library pseudocode for aarch32/translation/walk/AArch32_DecodeDescriptorTypeSD

```
// AArch32_DecodeDescriptorTypeSD()
// ================================
// Determine the type of the short-descriptor

func AArch32_DecodeDescriptorTypeSD(descriptor : bits(32), level : integer) => SDFType
begin
    if level == 1 && descriptor[1:0] == '01' then
        return SDFType_Table;
    elsif level == 1 && descriptor[18,1] == '01' then
        return SDFType_Section;
    elsif level == 1 && descriptor[18,1] == '11' then
        return SDFType_Supersection;
    elsif level == 2 && descriptor[1:0] == '01' then
        return SDFType_LargePage;
    elsif level == 2 && descriptor[1:0] == '1x' then
        return SDFType_SmallPage;
    else
        return SDFType_Invalid;
    end;
end;
```

Library pseudocode for aarch32/translation/walk/AArch32_S1IASize

```
// AArch32_S1IASize()
// ==================
// Retrieve the number of bits containing the input address for stage 1 translation

func AArch32_S1IASize(txsz : bits(3)) => AddressSize
begin
    return 32 - UInt(txsz);
end;
```

Library pseudocode for aarch32/translation/walk/AArch32_S1WalkLD

```
// AArch32_S1WalkLD()
// ==================
// Traverse stage 1 translation tables in long format to obtain the final descriptor

func AArch32_S1WalkLD(fault_in : FaultRecord, regime : Regime,
                      walkparams : S1TTWParams, accdesc : AccessDescriptor,
                      va : bits(32)) => (FaultRecord, TTWState)
begin
    var fault : FaultRecord = fault_in;
    var txsz : bits(3);
    var ttbr : bits(64);
    var epd : bit;
    var varange : VARange;
    if regime == Regime_EL2 then
        ttbr = HTTBR();
        txsz = walkparams.t0sz;
        varange = VARange_LOWER;
    else
        varange = AArch32_GetVARange(va, walkparams.t0sz, walkparams.t1sz);
        var ttbr0 : bits(64);
        var ttbr1 : bits(64);
        var ttbcr : TTBCR_Type;
        if regime == Regime_EL30 then
            ttbcr = TTBCR_S();
            ttbr0 = TTBR0_S();
            ttbr1 = TTBR1_S();
        elsif HaveEL(EL3) && ELUsingAArch32(EL3) then
            ttbcr = TTBCR_NS();
            ttbr0 = TTBR0_NS();
            ttbr1 = TTBR1_NS();
        else
            ttbcr = TTBCR();
            ttbr0 = TTBR0();
            ttbr1 = TTBR1();
        end;

        assert ttbcr.EAE == '1';
        if varange == VARange_LOWER then
            txsz = walkparams.t0sz;
            ttbr = ttbr0;
            epd  = ttbcr.EPD0;
        else
            txsz = walkparams.t1sz;
            ttbr = ttbr1;
            epd  = ttbcr.EPD1;
        end;
    end;

    if regime != Regime_EL2 && epd == '1' then
        fault.level      = 1;
        fault.statuscode = Fault_Translation;
        return (fault, ARBITRARY : TTWState);
    end;

    // Input Address size
    let iasize : AddressSize      = AArch32_S1IASize(txsz);
    let granulebits : integer{} = TGxGranuleBits(walkparams.tgx) as integer{12, 14, 16};
    let stride : integer{}      = granulebits - 3;
    let startlevel : integer  = FINAL_LEVEL - (((iasize-1) - granulebits) DIVRM stride);
    let levels : integer      = FINAL_LEVEL - startlevel;

    if !IsZero(ttbr[47:40]) then
        fault.statuscode = Fault_AddressSize;
        fault.level      = 0;
        return (fault, ARBITRARY : TTWState);
    end;

    var baseaddress : FullAddress;
    let baselsb = ((iasize - (levels*stride + granulebits)) + 3) as AddressSize;
    baseaddress.paspace = if accdesc.ss == SS_Secure then PAS_Secure else PAS_NonSecure;
    baseaddress.address = ZeroExtend{NUM_PABITS}(ttbr[39:baselsb]::Zeros{baselsb});

    var walkstate : TTWState;
    walkstate.baseaddress = baseaddress;
    walkstate.level       = startlevel;
    walkstate.istable     = TRUE;
    // In regimes that support global and non-global translations, translation
    // table entries from lookup levels other than the final level of lookup
    // are treated as being non-global
    walkstate.nG          = if HasUnprivileged(regime) then '1' else '0';
    walkstate.memattrs    = WalkMemAttrs(walkparams.sh, walkparams.irgn, walkparams.orgn);
    walkstate.permissions.ap_table  = '00';
    walkstate.permissions.xn_table  = '0';
    walkstate.permissions.pxn_table = '0';

    var descriptor : bits(64);
    var walkaddress : AddressDescriptor;

    walkaddress.vaddress = ZeroExtend{64}(va);
```

```
if !AArch32_S1DCacheEnabled(regime) then
    walkaddress.memattrs = NormalNCMemAttr();
    walkaddress.memattrs.xs = walkstate.memattrs.xs;
else
    walkaddress.memattrs = walkstate.memattrs;
end;

// Shareability value of stage 1 translation subject to stage 2 is IMPLEMENTATION DEFINED
// to be either effective value or descriptor value
if (regime == Regime_EL10 && AArch32_EL2Enabled(accdesc.ss) &&
    (if ELStateUsingAArch32(EL2, accdesc.ss==SS_Secure) then HCR().VM
                                                        else HCR_EL2().VM) == '1' &&
    !(ImpDefBool("Apply effective shareability at stage 1"))) then
    walkaddress.memattrs.shareability = walkstate.memattrs.shareability;
else
    walkaddress.memattrs.shareability = EffectiveShareability(walkaddress.memattrs);
end;

var desctype : DescriptorType;
var msb_residual : integer = iasize - 1;
repeat
    fault.level = walkstate.level;
    let indexlsb = ((FINAL_LEVEL - walkstate.level)*stride + granulebits) as AddressSize;
    let indexmsb = msb_residual as AddressSize;
    let index : bits(40) = ZeroExtend{}(va[indexmsb:indexlsb]::'000');

    walkaddress.paddress.address = (walkstate.baseaddress.address OR
                                    ZeroExtend{NUM_PABITS}(index));
    walkaddress.paddress.paspace = walkstate.baseaddress.paspace;

    let toplevel : boolean = walkstate.level == startlevel;
    let walkaccess : AccessDescriptor = CreateAccDescS1TTW(toplevel, varange, accdesc);
    // If there are two stages of translation, then the first stage table walk addresses
    // are themselves subject to translation
    if regime == Regime_EL10 && AArch32_EL2Enabled(accdesc.ss) then
        let s2aligned : boolean = TRUE;
        let (s2fault, s2walkaddress) = AArch32_S2Translate(fault, walkaddress, s2aligned,
                                                           walkaccess);
        // Check for a fault on the stage 2 walk
        if s2fault.statuscode != Fault_None then
            return (s2fault, ARBITRARY : TTWState);
        end;

        (fault, descriptor) = FetchDescriptor{64}(walkparams.ee, s2walkaddress, walkaccess,
                                                  fault);
    else
        (fault, descriptor) = FetchDescriptor{64}(walkparams.ee, walkaddress, walkaccess,
                                                  fault);
    end;

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : TTWState);
    end;

    desctype = AArch32_DecodeDescriptorTypeLD(descriptor, walkstate.level);

    case desctype of
        when DescriptorType_Table =>
            if !IsZero(descriptor[47:40]) then
                fault.statuscode = Fault_AddressSize;
                return (fault, ARBITRARY : TTWState);
            end;

            walkstate.baseaddress.address = ZeroExtend{NUM_PABITS}(descriptor[39:12]::
                                                                   Zeros{12});
            if walkstate.baseaddress.paspace == PAS_Secure && descriptor[63] == '1' then
                walkstate.baseaddress.paspace = PAS_NonSecure;
            end;
            if walkparams.hpd == '0' then
                walkstate.permissions.xn_table  = (walkstate.permissions.xn_table  OR
                                                   descriptor[60]);
                walkstate.permissions.ap_table  = (walkstate.permissions.ap_table  OR
                                                   descriptor[62:61]);
                walkstate.permissions.pxn_table = (walkstate.permissions.pxn_table OR
                                                   descriptor[59]);
            end;
            walkstate.level = walkstate.level + 1;
            msb_residual = indexlsb - 1;

        when DescriptorType_Invalid =>
            fault.statuscode = Fault_Translation;
            return (fault, ARBITRARY : TTWState);

        when DescriptorType_Leaf =>
            walkstate.istable = FALSE;
    end;

until desctype == DescriptorType_Leaf looplimit 3;
```

```
    // Check the output address is inside the supported range
    if !IsZero(descriptor[47:40]) then
        fault.statuscode = Fault_AddressSize;
        return (fault, ARBITRARY : TTWState);
    end;

    // Check the access flag
    if descriptor[10] == '0' then
        fault.statuscode = Fault_AccessFlag;
        return (fault, ARBITRARY : TTWState);
    end;

    walkstate.permissions.xn  = descriptor[54];
    walkstate.permissions.pxn = descriptor[53];
    walkstate.permissions.ap  = descriptor[7:6]::'1';
    walkstate.contiguous      = descriptor[52];
    if regime == Regime_EL2 then
        // All EL2 regime accesses are treated as Global
        walkstate.nG = '0';
    elsif accdesc.ss == SS_Secure && walkstate.baseaddress.paspace == PAS_NonSecure then
        // When a PE is using the Long-descriptor translation table format,
        // and is in Secure state, a translation must be treated as non-global,
        // regardless of the value of the nG bit,
        // if NSTable is set to 1 at any level of the translation table walk.
        walkstate.nG = '1';
    else
        walkstate.nG = descriptor[11];
    end;

    let indexlsb = ((FINAL_LEVEL - walkstate.level)*stride + granulebits) as AddressSize;
    walkstate.baseaddress.address = ZeroExtend{NUM_PABITS}(descriptor[39:indexlsb]::
                                                           Zeros{indexlsb});
    if walkstate.baseaddress.paspace == PAS_Secure && descriptor[5] == '1' then
        walkstate.baseaddress.paspace = PAS_NonSecure;
    end;

    let memattr : bits(3) = descriptor[4:2];
    let sh : bits(2)      = descriptor[9:8];
    let attr : bits(8)    = AArch32_MAIRAttr(UInt(memattr), walkparams.mair);
    let s1aarch64 : boolean = FALSE;
    walkstate.memattrs = S1DecodeMemAttrs(attr, sh, s1aarch64);

    return (fault, walkstate);
end;
```

Library pseudocode for aarch32/translation/walk/AArch32_S1WalkSD

```
// AArch32_S1WalkSD()
// ==================
// Traverse stage 1 translation tables in short format to obtain the final descriptor

func AArch32_S1WalkSD(fault_in : FaultRecord, regime : Regime,
                      accdesc : AccessDescriptor, va : bits(32)) => (FaultRecord, TTWState)
begin
    var fault : FaultRecord = fault_in;
    var sctlr : SCTLR_Type;
    var ttbcr : TTBCR_Type;
    var ttbr0 : TTBR0_Type;
    var ttbr1 : TTBR1_Type;
    // Determine correct translation control registers to use.
    if regime == Regime_EL30 then
        sctlr = SCTLR_S();
        ttbcr = TTBCR_S();
        ttbr0 = TTBR0_S();
        ttbr1 = TTBR1_S();
    elsif HaveEL(EL3) && ELUsingAArch32(EL3) then
        sctlr = SCTLR_NS();
        ttbcr = TTBCR_NS();
        ttbr0 = TTBR0_NS();
        ttbr1 = TTBR1_NS();
    else
        sctlr = SCTLR();
        ttbcr = TTBCR();
        ttbr0 = TTBR0();
        ttbr1 = TTBR1();
    end;

    assert ttbcr.EAE == '0';
    let ee : bit  = sctlr.EE;
    let afe : bit = sctlr.AFE;
    let tre : bit = sctlr.TRE;
    let ttbcr_n : integer{} = UInt(ttbcr.N);
    let varange : VARange = (if ttbcr_n == 0 || IsZero(va[31:(32-ttbcr_n)]) then VARange_LOWER
                             else VARange_UPPER);
    let n : integer{} = if varange == VARange_LOWER then ttbcr_n else 0;
    var ttb : bits(32);
    var pd : bits(1);
    var irgn : bits(2);
    var rgn : bits(2);
    var s : bits(1);
    var nos : bits(1);
    if varange == VARange_LOWER then
        ttb  = ttbr0.TTB0::Zeros{7};
        pd   = ttbcr.PD0;
        irgn = ttbr0.IRGN;
        rgn  = ttbr0.RGN;
        s    = ttbr0.S;
        nos  = ttbr0.NOS;
    else
        ttb  = ttbr1.TTB1::Zeros{7};
        pd   = ttbcr.PD1;
        irgn = ttbr1.IRGN;
        rgn  = ttbr1.RGN;
        s    = ttbr1.S;
        nos  = ttbr1.NOS;
    end;

    // Check if Translation table walk disabled for translations with this Base register.
    if pd == '1' then
        fault.level      = 1;
        fault.statuscode = Fault_Translation;
        return (fault, ARBITRARY : TTWState);
    end;

    var baseaddress : FullAddress;
    baseaddress.paspace = if accdesc.ss == SS_Secure then PAS_Secure else PAS_NonSecure;
    baseaddress.address = ZeroExtend{NUM_PABITS}(ttb[31:14-n]::Zeros{14-n});

    let startlevel : integer = 1;
    var walkstate : TTWState;
    walkstate.baseaddress = baseaddress;
    // In regimes that support global and non-global translations, translation
    // table entries from lookup levels other than the final level of lookup
    // are treated as being non-global. Translations in Short-Descriptor Format
    // always support global & non-global translations.
    walkstate.nG          = '1';
    walkstate.memattrs    = WalkMemAttrs(s::nos, irgn, rgn);
    walkstate.level       = startlevel;
    walkstate.istable     = TRUE;

    var domain : bits(4);
    var descriptor : bits(32);
    var walkaddress : AddressDescriptor;

    walkaddress.vaddress = ZeroExtend{64}(va);
```

```
if !AArch32_S1DCacheEnabled(regime) then
    walkaddress.memattrs = NormalNCMemAttr();
    walkaddress.memattrs.xs = walkstate.memattrs.xs;
else
    walkaddress.memattrs = walkstate.memattrs;
end;

// Shareability value of stage 1 translation subject to stage 2 is IMPLEMENTATION DEFINED
// to be either effective value or descriptor value
if (regime == Regime_EL10 && AArch32_EL2Enabled(accdesc.ss) &&
    (if ELStateUsingAArch32(EL2, accdesc.ss==SS_Secure) then HCR().VM
                                                        else HCR_EL2().VM) == '1' &&
    !(ImpDefBool("Apply effective shareability at stage 1"))) then
    walkaddress.memattrs.shareability = walkstate.memattrs.shareability;
else
    walkaddress.memattrs.shareability = EffectiveShareability(walkaddress.memattrs);
end;

var nG : bit;
var ns : bit;
var pxn : bit;
var ap : bits(3);
var tex : bits(3);
var c : bit;
var b : bit;
var xn : bit;
repeat
    fault.level = walkstate.level;

    var index : bits(32);
    if walkstate.level == 1 then
        index = ZeroExtend{32}(va[31-n:20]::'00');
    else
        index = ZeroExtend{32}(va[19:12]::'00');
    end;

    walkaddress.paddress.address = (walkstate.baseaddress.address OR
                                    ZeroExtend{NUM_PABITS}(index));
    walkaddress.paddress.paspace = walkstate.baseaddress.paspace;

    let toplevel : boolean = walkstate.level == startlevel;
    let walkaccess : AccessDescriptor = CreateAccDescS1TTW(toplevel, varange, accdesc);
    if regime == Regime_EL10 && AArch32_EL2Enabled(accdesc.ss) then
        let s2aligned : boolean = TRUE;
        let (s2fault, s2walkaddress) = AArch32_S2Translate(fault, walkaddress, s2aligned,
                                                           walkaccess);

        if s2fault.statuscode != Fault_None then
            return (s2fault, ARBITRARY : TTWState);
        end;

        (fault, descriptor) = FetchDescriptor{32}(ee, s2walkaddress, walkaccess, fault);
    else
        (fault, descriptor) = FetchDescriptor{32}(ee, walkaddress, walkaccess, fault);
    end;

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : TTWState);
    end;

    walkstate.sdftype = AArch32_DecodeDescriptorTypeSD(descriptor, walkstate.level);

    case walkstate.sdftype of
        when SDFType_Invalid =>
            fault.domain     = domain;
            fault.statuscode = Fault_Translation;
            return (fault, ARBITRARY : TTWState);

        when SDFType_Table =>
            domain = descriptor[8:5];
            ns     = descriptor[3];
            pxn    = descriptor[2];

            walkstate.baseaddress.address = ZeroExtend{NUM_PABITS}(descriptor[31:10]::
                                                                   Zeros{10});
            walkstate.level = 2;

        when SDFType_SmallPage =>
            nG  = descriptor[11];
            s   = descriptor[10];
            ap  = descriptor[9,5:4];
            tex = descriptor[8:6];
            c   = descriptor[3];
            b   = descriptor[2];
            xn  = descriptor[0];

            walkstate.baseaddress.address = ZeroExtend{NUM_PABITS}(descriptor[31:12]::
                                                                   Zeros{12});
            walkstate.istable = FALSE;
```

```
            when SDFType_LargePage =>
                xn  = descriptor[15];
                tex = descriptor[14:12];
                nG  = descriptor[11];
                s   = descriptor[10];
                ap  = descriptor[9,5:4];
                c   = descriptor[3];
                b   = descriptor[2];

                walkstate.baseaddress.address = ZeroExtend{NUM_PABITS}(descriptor[31:16]::
                                                                       Zeros{16});
                walkstate.istable = FALSE;

            when SDFType_Section =>
                ns     = descriptor[19];
                nG     = descriptor[17];
                s      = descriptor[16];
                ap     = descriptor[15,11:10];
                tex    = descriptor[14:12];
                domain = descriptor[8:5];
                xn     = descriptor[4];
                c      = descriptor[3];
                b      = descriptor[2];
                pxn    = descriptor[0];

                walkstate.baseaddress.address = ZeroExtend{NUM_PABITS}(descriptor[31:20]::
                                                                       Zeros{20});
                walkstate.istable = FALSE;

            when SDFType_Supersection =>
                ns     = descriptor[19];
                nG     = descriptor[17];
                s      = descriptor[16];
                ap     = descriptor[15,11:10];
                tex    = descriptor[14:12];
                xn     = descriptor[4];
                c      = descriptor[3];
                b      = descriptor[2];
                pxn    = descriptor[0];
                domain = '0000';

                walkstate.baseaddress.address = ZeroExtend{NUM_PABITS}(descriptor[8:5,23:20,31:24]::
                                                                       Zeros{24});

                walkstate.istable = FALSE;
        end;

    until walkstate.sdftype != SDFType_Table looplimit 2;

    if afe == '1' && ap[0] == '0' then
        fault.domain     = domain;
        fault.statuscode = Fault_AccessFlag;
        return (fault, ARBITRARY : TTWState);
    end;

    // Decode the TEX, C, B and S bits to produce target memory attributes
    if tre == '1' then
        walkstate.memattrs = AArch32_RemappedTEXDecode(regime, tex, c, b, s);
    elsif RemapRegsHaveResetValues() then
        walkstate.memattrs = AArch32_DefaultTEXDecode(tex, c, b, s);
    else
        walkstate.memattrs = ImpDefMemoryAttributes("IMPLEMENTATION_DEFINED");
    end;

    walkstate.permissions.ap  = ap;
    walkstate.permissions.xn  = xn;
    walkstate.permissions.pxn = pxn;
    walkstate.domain = domain;
    walkstate.nG     = nG;

    if accdesc.ss == SS_Secure && ns == '0' then
        walkstate.baseaddress.paspace = PAS_Secure;
    else
        walkstate.baseaddress.paspace = PAS_NonSecure;
    end;


    return (fault, walkstate);
end;
```

Library pseudocode for aarch32/translation/walk/AArch32_S2IASize

```
// AArch32_S2IASize()
// ==================
// Retrieve the number of bits containing the input address for stage 2 translation

func AArch32_S2IASize(t0sz : bits(4)) => AddressSize
begin
    return 32 - SInt(t0sz);
end;
```

Library pseudocode for aarch32/translation/walk/AArch32_S2StartLevel

```
// AArch32_S2StartLevel()
// ======================
// Determine the initial lookup level when performing a stage 2 translation
// table walk

func AArch32_S2StartLevel(sl0 : bits(2)) => integer
begin
    return 2 - UInt(sl0);
end;
```

Library pseudocode for aarch32/translation/walk/AArch32_S2Walk

```
// AArch32_S2Walk()
// ================
// Traverse stage 2 translation tables in long format to obtain the final descriptor

func AArch32_S2Walk(fault_in : FaultRecord, walkparams : S2TTWParams,
                    accdesc : AccessDescriptor, ipa : AddressDescriptor) => (FaultRecord, TTWState)
begin
    var fault : FaultRecord = fault_in;

    if walkparams.sl0 == '1x' || AArch32_S2InconsistentSL(walkparams) then
        fault.statuscode = Fault_Translation;
        fault.level      = 1;
        return (fault, ARBITRARY : TTWState);
    end;

    // Input Address size
    let iasize : AddressSize      = AArch32_S2IASize(walkparams.t0sz);
    let startlevel : integer  = AArch32_S2StartLevel(walkparams.sl0);
    let levels : integer      = FINAL_LEVEL - startlevel;
    let granulebits : AddressSize = TGxGranuleBits(walkparams.tgx);
    let stride : integer{}      = granulebits - 3;

    if !IsZero(VTTBR()[47:40]) then
        fault.statuscode = Fault_AddressSize;
        fault.level      = 0;
        return (fault, ARBITRARY : TTWState);
    end;

    var baseaddress : FullAddress;
    let baselsb = ((iasize - (levels*stride + granulebits)) + 3) as AddressSize;
    baseaddress.paspace = PAS_NonSecure;
    baseaddress.address = ZeroExtend{NUM_PABITS}(VTTBR()[39:baselsb]::Zeros{baselsb});

    var walkstate : TTWState;
    walkstate.baseaddress = baseaddress;
    walkstate.level       = startlevel;
    walkstate.istable     = TRUE;
    walkstate.memattrs    = WalkMemAttrs(walkparams.sh, walkparams.irgn,
                                         walkparams.orgn);

    var descriptor : bits(64);
    let walkaccess : AccessDescriptor = CreateAccDescS2TTW(accdesc);
    var walkaddress : AddressDescriptor;

    walkaddress.vaddress = ipa.vaddress;

    if HCR2().CD == '1' then
        walkaddress.memattrs = NormalNCMemAttr();
        walkaddress.memattrs.xs = walkstate.memattrs.xs;
    else
        walkaddress.memattrs = walkstate.memattrs;
    end;

    walkaddress.memattrs.shareability = EffectiveShareability(walkaddress.memattrs);

    var msb_residual : integer = iasize - 1;
    var desctype : DescriptorType;
    repeat
        fault.level = walkstate.level;

        let indexlsb = ((FINAL_LEVEL - walkstate.level)*stride + granulebits) as AddressSize;
        let indexmsb = msb_residual as AddressSize;
        let index : bits(40) = ZeroExtend{}(ipa.paddress.address[indexmsb:indexlsb]::'000');

        walkaddress.paddress.address = (walkstate.baseaddress.address OR
                                        ZeroExtend{NUM_PABITS}(index));
        walkaddress.paddress.paspace = walkstate.baseaddress.paspace;

        (fault, descriptor) = FetchDescriptor{64}(walkparams.ee, walkaddress, walkaccess, fault);

        if fault.statuscode != Fault_None then
            return (fault, ARBITRARY : TTWState);
        end;

        desctype = AArch32_DecodeDescriptorTypeLD(descriptor, walkstate.level);

        case desctype of
            when DescriptorType_Table =>
                if !IsZero(descriptor[47:40]) then
                    fault.statuscode = Fault_AddressSize;
                    return (fault, ARBITRARY : TTWState);
                end;

                walkstate.baseaddress.address = ZeroExtend{NUM_PABITS}(descriptor[39:12]::
                                                                       Zeros{12});
                walkstate.level = walkstate.level + 1;
                msb_residual = indexlsb - 1;

            when DescriptorType_Invalid =>
```

```
                fault.statuscode = Fault_Translation;
                return (fault, ARBITRARY : TTWState);

            when DescriptorType_Leaf =>
                walkstate.istable = FALSE;
        end;

    until desctype == DescriptorType_Leaf looplimit 3;

    // Check the output address is inside the supported range
    if !IsZero(descriptor[47:40]) then
        fault.statuscode = Fault_AddressSize;
        return (fault, ARBITRARY : TTWState);
    end;

    // Check the access flag
    if descriptor[10] == '0' then
        fault.statuscode = Fault_AccessFlag;
        return (fault, ARBITRARY : TTWState);
    end;

    // Unpack the descriptor into address and upper and lower block attributes
    let indexlsb = ((FINAL_LEVEL - walkstate.level)*stride + granulebits) as AddressSize;
    walkstate.baseaddress.address = ZeroExtend{NUM_PABITS}(descriptor[39:indexlsb]::
                                                           Zeros{indexlsb});

    walkstate.permissions.s2ap = descriptor[7:6];
    walkstate.permissions.s2xn = descriptor[54];
    if IsFeatureImplemented(FEAT_XNX) then
        walkstate.permissions.s2xnx = descriptor[53];
    else
        walkstate.permissions.s2xnx = '0';
    end;

    let memattr : bits(4) = descriptor[5:2];
    let sh : bits(2)      = descriptor[9:8];
    let s2aarch64 : boolean = FALSE;
    walkstate.memattrs    = S2DecodeMemAttrs(memattr, sh, s2aarch64);
    walkstate.contiguous = descriptor[52];


    return (fault, walkstate);
end;
```

Library pseudocode for aarch32/translation/walk/RemapRegsHaveResetValues

```
// RemapRegsHaveResetValues()
// ==========================

impdef func RemapRegsHaveResetValues() => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch32/translation/walkparams/AArch32_GetS1TTWParams

```
// AArch32_GetS1TTWParams()
// ========================
// Returns stage 1 translation table walk parameters from respective controlling
// System registers.

func AArch32_GetS1TTWParams(regime : Regime, va : bits(32)) => S1TTWParams
begin
    var walkparams : S1TTWParams;

    case regime of
        when Regime_EL2  => walkparams = AArch32_S1TTWParamsEL2();
        when Regime_EL10 => walkparams = AArch32_S1TTWParamsEL10(va);
        when Regime_EL30 => walkparams = AArch32_S1TTWParamsEL30(va);
    end;

    return walkparams;
end;
```

Library pseudocode for aarch32/translation/walkparams/AArch32_GetS2TTWParams

```
// AArch32_GetS2TTWParams()
// ========================
// Gather walk parameters for stage 2 translation

func AArch32_GetS2TTWParams() => S2TTWParams
begin
    var walkparams : S2TTWParams;

    walkparams.tgx  = TGx_4KB;
    walkparams.s    = VTCR().S;
    walkparams.t0sz = VTCR().T0SZ;
    walkparams.sl0  = VTCR().SL0;
    walkparams.irgn = VTCR().IRGN0;
    walkparams.orgn = VTCR().ORGN0;
    walkparams.sh   = VTCR().SH0;
    walkparams.ee   = HSCTLR().EE;
    walkparams.ptw  = HCR().PTW;
    walkparams.vm   = HCR().VM OR HCR().DC;

    // VTCR.S must match VTCR.T0SZ[3]
    if walkparams.s != walkparams.t0sz[3] then
        (-, walkparams.t0sz) = ConstrainUnpredictableBits{4}(Unpredictable_RESVTCRS);
    end;

    return walkparams;
end;
```

Library pseudocode for aarch32/translation/walkparams/AArch32_GetVARange

```
// AArch32_GetVARange()
// ====================
// Select the translation base address for stage 1 long-descriptor walks

func AArch32_GetVARange(va : bits(32), t0sz : bits(3), t1sz : bits(3)) => VARange
begin
    // Lower range Input Address size
    let lo_iasize : AddressSize = AArch32_S1IASize(t0sz);
    // Upper range Input Address size
    let up_iasize : AddressSize = AArch32_S1IASize(t1sz);

    if t1sz == '000' && t0sz == '000' then
        return VARange_LOWER;
    elsif t1sz == '000' then
        return if IsZero(va[31:lo_iasize]) then VARange_LOWER else VARange_UPPER;
    elsif t0sz == '000' then
        return if IsOnes(va[31:up_iasize]) then VARange_UPPER else VARange_LOWER;
    elsif IsZero(va[31:lo_iasize]) then
        return VARange_LOWER;
    elsif IsOnes(va[31:up_iasize]) then
        return VARange_UPPER;
    else
        // Will be reported as a Translation Fault
        return ARBITRARY : VARange;
    end;
end;
```

Library pseudocode for aarch32/translation/walkparams/AArch32_S1DCacheEnabled

```
// AArch32_S1DCacheEnabled()
// =========================
// Determine cacheability of stage 1 data accesses

func AArch32_S1DCacheEnabled(regime : Regime) => boolean
begin
    case regime of
        when Regime_EL30 => return SCTLR_S().C == '1';
        when Regime_EL2  => return HSCTLR().C == '1';
        when Regime_EL10 =>
            return (if HaveEL(EL3) && ELUsingAArch32(EL3) then SCTLR_NS().C else SCTLR().C) == '1';
    end;
end;
```

Library pseudocode for aarch32/translation/walkparams/AArch32_S1ICacheEnabled

```
// AArch32_S1ICacheEnabled()
// =========================
// Determine cacheability of stage 1 instruction fetches

func AArch32_S1ICacheEnabled(regime : Regime) => boolean
begin
    case regime of
        when Regime_EL30 => return SCTLR_S().I == '1';
        when Regime_EL2  => return HSCTLR().I == '1';
        when Regime_EL10 =>
            return (if HaveEL(EL3) && ELUsingAArch32(EL3) then SCTLR_NS().I else SCTLR().I) == '1';
    end;
end;
```

Library pseudocode for aarch32/translation/walkparams/AArch32_S1TTWParamsEL10

```
// AArch32_S1TTWParamsEL10()
// =========================
// Gather stage 1 translation table walk parameters for EL1&0 regime
// (with EL2 enabled or disabled).

func AArch32_S1TTWParamsEL10(va : bits(32)) => S1TTWParams
begin
    var mair : bits(64);
    var sif : bit;
    var ttbcr : TTBCR_Type;
    var ttbcr2 : TTBCR2_Type;
    var sctlr : SCTLR_Type;

    if ELUsingAArch32(EL3) then
        ttbcr  = TTBCR_NS();
        ttbcr2 = TTBCR2_NS();
        sctlr  = SCTLR_NS();
        mair   = MAIR1_NS()::MAIR0_NS();
        sif    = SCR().SIF;
    else
        ttbcr  = TTBCR();
        ttbcr2 = TTBCR2();
        sctlr  = SCTLR();
        mair   = MAIR1()::MAIR0();
        sif    = if HaveEL(EL3) then SCR_EL3().SIF else '0';
    end;

    assert ttbcr.EAE == '1';
    var walkparams : S1TTWParams;

    walkparams.t0sz   = ttbcr.T0SZ;
    walkparams.t1sz   = ttbcr.T1SZ;
    walkparams.ee     = sctlr.EE;
    walkparams.wxn    = sctlr.WXN;
    walkparams.uwxn   = sctlr.UWXN;
    walkparams.ntlsmd = if IsFeatureImplemented(FEAT_LSMAOC) then sctlr.nTLSMD else '1';
    walkparams.mair   = mair;
    walkparams.sif    = sif;

    let varange : VARange = AArch32_GetVARange(va, walkparams.t0sz, walkparams.t1sz);
    if varange == VARange_LOWER then
        walkparams.sh   = ttbcr.SH0;
        walkparams.irgn = ttbcr.IRGN0;
        walkparams.orgn = ttbcr.ORGN0;
        walkparams.hpd  = (if IsFeatureImplemented(FEAT_AA32HPD) then ttbcr.T2E AND ttbcr2.HPD0
                           else '0');
    else
        walkparams.sh   = ttbcr.SH1;
        walkparams.irgn = ttbcr.IRGN1;
        walkparams.orgn = ttbcr.ORGN1;
        walkparams.hpd  = (if IsFeatureImplemented(FEAT_AA32HPD) then ttbcr.T2E AND ttbcr2.HPD1
                           else '0');
    end;

    return walkparams;
end;
```

Library pseudocode for aarch32/translation/walkparams/AArch32_S1TTWParamsEL2

```
// AArch32_S1TTWParamsEL2()
// ========================
// Gather stage 1 translation table walk parameters for EL2 regime

func AArch32_S1TTWParamsEL2() => S1TTWParams
begin
    var walkparams : S1TTWParams;

    walkparams.tgx  = TGx_4KB;
    walkparams.t0sz = HTCR().T0SZ;
    walkparams.irgn = HTCR().IRGN0;
    walkparams.orgn = HTCR().ORGN0;
    walkparams.sh   = HTCR().SH0;
    walkparams.hpd  = if IsFeatureImplemented(FEAT_AA32HPD) then HTCR().HPD else '0';
    walkparams.ee   = HSCTLR().EE;
    walkparams.wxn  = HSCTLR().WXN;
    if IsFeatureImplemented(FEAT_LSMAOC) then
        walkparams.ntlsmd = HSCTLR().nTLSMD;
    else
        walkparams.ntlsmd = '1';
    end;

    walkparams.mair = HMAIR1()::HMAIR0();

    return walkparams;
end;
```

Library pseudocode for aarch32/translation/walkparams/AArch32_S1TTWParamsEL30

```
// AArch32_S1TTWParamsEL30()
// =========================
// Gather stage 1 translation table walk parameters for EL3&0 regime

func AArch32_S1TTWParamsEL30(va : bits(32)) => S1TTWParams
begin
    assert TTBCR_S().EAE == '1';
    var walkparams : S1TTWParams;

    walkparams.t0sz   = TTBCR_S().T0SZ;
    walkparams.t1sz   = TTBCR_S().T1SZ;
    walkparams.ee     = SCTLR_S().EE;
    walkparams.wxn    = SCTLR_S().WXN;
    walkparams.uwxn   = SCTLR_S().UWXN;
    walkparams.ntlsmd = if IsFeatureImplemented(FEAT_LSMAOC) then SCTLR_S().nTLSMD else '1';
    walkparams.mair   = MAIR1_S()::MAIR0_S();
    walkparams.sif    = SCR().SIF;

    let varange : VARange = AArch32_GetVARange(va, walkparams.t0sz, walkparams.t1sz);
    if varange == VARange_LOWER then
        walkparams.sh   = TTBCR_S().SH0;
        walkparams.irgn = TTBCR_S().IRGN0;
        walkparams.orgn = TTBCR_S().ORGN0;
        walkparams.hpd  = (if IsFeatureImplemented(FEAT_AA32HPD)
                           then TTBCR_S().T2E AND TTBCR2_S().HPD0
                           else '0');
    else
        walkparams.sh   = TTBCR_S().SH1;
        walkparams.irgn = TTBCR_S().IRGN1;
        walkparams.orgn = TTBCR_S().ORGN1;
        walkparams.hpd  = (if IsFeatureImplemented(FEAT_AA32HPD)
                           then TTBCR_S().T2E AND TTBCR2_S().HPD1
                           else '0');
    end;

    return walkparams;
end;
```

Library pseudocode for aarch64/debug/brbe

```
// Contents of the Branch Record Buffer
//=====================================

var Records_SRC : array [[64]] of BRBSRC_EL1_Type;
var Records_TGT : array [[64]] of BRBTGT_EL1_Type;
var Records_INF : array [[64]] of BRBINF_EL1_Type;



// Getter functions for branch records
//====================================
// Functions used by MRS instructions that access branch records

func BRBSRC_EL1(n : integer) => BRBSRC_EL1_Type
begin
    assert n IN {0..31};
    let record_index : integer = UInt(BRBFCR_EL1().BANK::n[4:0]);
    if record_index < GetBRBENumRecords() then
        return Records_SRC[[record_index]];
    else
        return Zeros{64};
    end;
end;

func BRBTGT_EL1(n : integer) => BRBTGT_EL1_Type
begin
    assert n IN {0..31};
    let record_index : integer = UInt(BRBFCR_EL1().BANK::n[4:0]);
    if record_index < GetBRBENumRecords() then
        return Records_TGT[[record_index]];
    else
        return Zeros{64};
    end;
end;

func BRBINF_EL1(n : integer) => BRBINF_EL1_Type
begin
    assert n IN {0..31};
    let record_index : integer = UInt(BRBFCR_EL1().BANK::n[4:0]);
    if record_index < GetBRBENumRecords() then
        return Records_INF[[record_index]];
    else
        return Zeros{64};
    end;
end;
```

Library pseudocode for aarch64/debug/brbe/BRBCycleCountingEnabled

```
// BRBCycleCountingEnabled()
// =========================
// Returns TRUE if the recording of cycle counts is allowed,
// FALSE otherwise.

func BRBCycleCountingEnabled() => boolean
begin
    if HaveEL(EL2) && BRBCR_EL2().CC == '0' then return FALSE; end;
    if BRBCR_EL1().CC == '0' then return FALSE; end;
    return TRUE;
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEBranch

```
// BRBEBranch()
// ============
// Called to write branch record for the following branches when BRBE is active:
// direct branches,
// indirect branches,
// direct branches with link,
// indirect branches with link,
// returns from subroutines.

func BRBEBranch(br_type : BranchType, cond : boolean, target_address : bits(64))
begin
    if BranchRecordAllowed(PSTATE.EL) && FilterBranchRecord(br_type, cond) then
        var branch_type : bits(6);
        case br_type of
            when BranchType_DIR =>
                branch_type = if cond then '001000' else '000000';
            when BranchType_INDIR   => branch_type = '000001';
            when BranchType_DIRCALL => branch_type = '000010';
            when BranchType_INDCALL => branch_type = '000011';
            when BranchType_RET     => branch_type = '000101';
            otherwise               => unreachable;
        end;

        var ccu : bit;
        var cc : bits(14);
        (ccu, cc) = BranchEncCycleCount();
        let el : bits(2) = PSTATE.EL;
        let mispredict : bit = (if BRBEMispredictAllowed() && BranchMispredict() then '1'
                                else '0');

        UpdateBranchRecordBuffer(ccu, cc, branch_type, el, mispredict,
                                 '11', PC64(), target_address);

        PMUEvent(PMU_EVENT_BRB_FILTRATE);
    end;

    return;
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEBranchOnISB

```
// BRBEBranchOnISB()
// =================
// Returns TRUE if ISBs generate Branch records, and FALSE otherwise.

func BRBEBranchOnISB() => boolean
begin
    return ImpDefBool("ISB generates Branch records");
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEDebugStateEntry

```
// BRBEDebugStateEntry()
// =====================
// Called to write Debug state entry branch record when BRBE is active.

func BRBEDebugStateEntry(source_address : bits(64))
begin
    if BranchRecordAllowed(PSTATE.EL) then
        let branch_type : bits(6) = '100001';
        var ccu : bit;
        var cc : bits(14);
        (ccu, cc)             = BranchEncCycleCount();
        let el : bits(2)      = '00';
        let mispredict : bit = '0';

        // Debug state is a prohibited region, therefore target_address=0
        UpdateBranchRecordBuffer(ccu, cc, branch_type, el, mispredict,
                                 '10', source_address, Zeros{64});

        PMUEvent(PMU_EVENT_BRB_FILTRATE);
    end;

    return;
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEDebugStateExit

```
// BRBEDebugStateExit()
// ====================
// Called to write Debug state exit branch record when BRBE is active.

func BRBEDebugStateExit(target_address : bits(64))
begin
    if BranchRecordAllowed(PSTATE.EL) then
        // Debug state is a prohibited region, therefore ccu=1, cc=0, source_address=0
        let branch_type : bits(6) = '111001';
        let ccu : bit             = '1';
        let cc : bits(14)         = Zeros{14};
        let el : bits(2)          = PSTATE.EL;
        let mispredict : bit      = '0';

        UpdateBranchRecordBuffer(ccu, cc, branch_type, el, mispredict,
                                 '01', Zeros{64}, target_address);

        PMUEvent(PMU_EVENT_BRB_FILTRATE);
    end;

    return;
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEException

```
// BRBEException()
// ===============
// Called to write exception branch record when BRBE is active.

func BRBEException(erec : ExceptionRecord, source_valid : boolean,
                   source_address_in : bits(64),
                   target_address_in : bits(64), target_el : bits(2),
                   trappedsyscallinst : boolean)
begin
    var target_address : bits(64) = target_address_in;
    let except : Exception = erec.exceptype;
    let iss : bits(25)     = erec.syndrome.iss;
    case target_el of
        when EL3 =>
            if !IsFeatureImplemented(FEAT_BRBEv1p1) || (MDCR_EL3().E3BREC == MDCR_EL3().E3BREW) then
                return;
            end;
        when EL2 => if BRBCR_EL2().EXCEPTION == '0' then return; end;
        when EL1 => if BRBCR_EL1().EXCEPTION == '0' then return; end;
    end;

    let target_valid : boolean = BranchRecordAllowed(target_el);

    if source_valid || target_valid then
        var branch_type : bits(6);
        case except of
            when Exception_Uncategorized         => branch_type = '100011'; // Trap
            when Exception_WFxTrap               => branch_type = '100011'; // Trap
            when Exception_CP15RTTrap            => branch_type = '100011'; // Trap
            when Exception_CP15RRTTrap           => branch_type = '100011'; // Trap
            when Exception_CP14RTTrap            => branch_type = '100011'; // Trap
            when Exception_CP14DTTrap            => branch_type = '100011'; // Trap
            when Exception_AdvSIMDFPAccessTrap   => branch_type = '100011'; // Trap
            when Exception_FPIDTrap              => branch_type = '100011'; // Trap
            when Exception_PACTrap               => branch_type = '100011'; // Trap
            when Exception_CP14RRTTrap           => branch_type = '100011'; // Trap
            when Exception_OtherTrap             => branch_type = '100011'; // Trap
            when Exception_BranchTarget          => branch_type = '101011'; // Inst Fault
            when Exception_IllegalState          => branch_type = '100011'; // Trap
            when Exception_SupervisorCall        =>
                if !trappedsyscallinst then      branch_type = '100010';    // Call
                else                             branch_type = '100011';    // Trap
                end;
            when Exception_HypervisorCall        => branch_type = '100010'; // Call
            when Exception_MonitorCall           =>
                if !trappedsyscallinst then      branch_type = '100010';    // Call
                else                             branch_type = '100011';    // Trap
                end;
            when Exception_SystemRegisterTrap    => branch_type = '100011'; // Trap
            when Exception_SystemRegister128Trap => branch_type = '100011'; // Trap
            when Exception_SVEAccessTrap         => branch_type = '100011'; // Trap
            when Exception_SMEAccessTrap         => branch_type = '100011'; // Trap
            when Exception_ERetTrap              => branch_type = '100011'; // Trap
            when Exception_PACFail               => branch_type = '101100'; // Data Fault
            when Exception_InstructionAbort      => branch_type = '101011'; // Inst Fault
            when Exception_PCAlignment           => branch_type = '101010'; // Alignment
            when Exception_DataAbort             => branch_type = '101100'; // Data Fault
            when Exception_NV2DataAbort          => branch_type = '101100'; // Data Fault
            when Exception_SPAlignment           => branch_type = '101010'; // Alignment
            when Exception_FPTrappedException    => branch_type = '100011'; // Trap
            when Exception_SError                => branch_type = '100100'; // System Error
            when Exception_Breakpoint            => branch_type = '100110'; // Inst debug
            when Exception_SoftwareStep          => branch_type = '100110'; // Inst debug
            when Exception_Watchpoint            => branch_type = '100111'; // Data debug
            when Exception_NV2Watchpoint         => branch_type = '100111'; // Data debug
            when Exception_SoftwareBreakpoint    => branch_type = '100110'; // Inst debug
            when Exception_IRQ                   => branch_type = '101110'; // IRQ
            when Exception_FIQ                   => branch_type = '101111'; // FIQ
            when Exception_MemCpyMemSet          => branch_type = '100011'; // Trap
            when Exception_GCSFail               =>
                if iss[23:20] == '0000' then        branch_type = '101100'; // Data Fault
                elsif iss[23:20] == '0001' then     branch_type = '101011'; // Inst Fault
                elsif iss[23:20] == '0010' then     branch_type = '100011'; // Trap
                else                                unreachable;
                end;
            when Exception_Profiling             => branch_type = '100110'; // Inst debug
            when Exception_GPC                   =>
                if iss[20] == '1' then              branch_type = '101011'; // Inst fault
                else                                branch_type = '101100'; // Data fault
                end;
            when Exception_ConditionalFault      => branch_type = '100110'; // Inst debug
            otherwise                            => unreachable;
        end;

        var ccu : bit;
        var cc : bits(14);
        (ccu, cc) = BranchEncCycleCount();
        let el : bits(2) = if target_valid then target_el else '00';
        let mispredict : bit = '0';
```

```
        let sv : bit = if source_valid then '1' else '0';
        let tv : bit = if target_valid then '1' else '0';
        let source_address : bits(64) = if source_valid then source_address_in else Zeros{64};

        if !target_valid then
            target_address = Zeros{64};
        else
            target_address = AArch64_BranchAddr(target_address, target_el);
        end;

        UpdateBranchRecordBuffer(ccu, cc, branch_type, el, mispredict,
                                 sv::tv, source_address, target_address);

        PMUEvent(PMU_EVENT_BRB_FILTRATE);
    end;

    return;
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEExceptionReturn

```
// BRBEExceptionReturn()
// =====================
// Called to write exception return branch record when BRBE is active.

func BRBEExceptionReturn(target_address_in : bits(64), source_el : bits(2),
                         source_valid : boolean, source_address_in : bits(64))
begin
    var target_address : bits(64) = target_address_in;
    case source_el of
        when EL3 =>
            if !IsFeatureImplemented(FEAT_BRBEv1p1) || (MDCR_EL3().E3BREC == MDCR_EL3().E3BREW) then
                return;
            end;
        when EL2 => if BRBCR_EL2().ERTN == '0' then return; end;
        when EL1 => if BRBCR_EL1().ERTN == '0' then return; end;
    end;

    let target_valid : boolean = BranchRecordAllowed(PSTATE.EL);

    if source_valid || target_valid then
        let branch_type : bits(6) = '000111';
        var ccu : bit;
        var cc : bits(14);
        (ccu, cc) = BranchEncCycleCount();
        let el : bits(2)     = if target_valid then PSTATE.EL else '00';
        let mispredict : bit = (if source_valid && BRBEMispredictAllowed() &&
                                   BranchMispredict() then '1' else '0');
        let sv : bit         = if source_valid then '1' else '0';
        let tv : bit         = if target_valid then '1' else '0';
        let source_address : bits(64) = if source_valid then source_address_in else Zeros{64};
        if !target_valid then
            target_address = Zeros{64};
        end;

        UpdateBranchRecordBuffer(ccu, cc, branch_type, el, mispredict,
                                 sv::tv, source_address, target_address);

        PMUEvent(PMU_EVENT_BRB_FILTRATE);
    end;

    return;
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEFreeze

```
// BRBEFreeze()
// ============
// Generates BRBE freeze event.

func BRBEFreeze()
begin
    BRBFCR_EL1().PAUSED = '1';
    BRBTS_EL1() = GetTimestamp(BRBETimeStamp());
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEISB

```
// BRBEISB()
// =========
// Handles ISB instruction for BRBE.

func BRBEISB()
begin
    let branch_conditional : boolean = FALSE;
    BRBEBranch(BranchType_DIR, branch_conditional, PC64() + 4);
end;
```

Library pseudocode for aarch64/debug/brbe/BRBEMispredictAllowed

```
// BRBEMispredictAllowed()
// =======================
// Returns TRUE if the recording of branch misprediction is allowed,
// FALSE otherwise.

func BRBEMispredictAllowed() => boolean
begin
    if HaveEL(EL2) && BRBCR_EL2().MPRED == '0' then return FALSE; end;
    if BRBCR_EL1().MPRED == '0' then return FALSE; end;
    return TRUE;
end;
```

Library pseudocode for aarch64/debug/brbe/BRBETimeStamp

```
// BRBETimeStamp()
// ===============
// Returns captured timestamp.

func BRBETimeStamp() => TimeStamp
begin
    if HaveEL(EL2) then
        var TS_el2 : bits(2) = BRBCR_EL2().TS;
        if !IsFeatureImplemented(FEAT_ECV) && TS_el2 == '10' then
            // Reserved value
            (-, TS_el2) = ConstrainUnpredictableBits{2}(Unpredictable_EL2TIMESTAMP);
        end;
        case TS_el2 of
            when '00' =>
                // Falls out to check BRBCR_EL1.TS
                pass;
            when '01' =>
                return TimeStamp_Virtual;
            when '10' =>
                assert IsFeatureImplemented(FEAT_ECV); // Otherwise ConstrainUnpredictableBits
                                                       // removes this case
                return TimeStamp_OffsetPhysical;
            when '11' =>
                return TimeStamp_Physical;
        end;
    end;

    var TS_el1 : bits(2) = BRBCR_EL1().TS;
    if TS_el1 == '00' || (!IsFeatureImplemented(FEAT_ECV) && TS_el1 == '10') then
        // Reserved value
        (-, TS_el1) = ConstrainUnpredictableBits{2}(Unpredictable_EL1TIMESTAMP);
    end;
    case TS_el1 of
        when '01' =>
            return TimeStamp_Virtual;
        when '10' =>
            return TimeStamp_OffsetPhysical;
        when '11' =>
            return TimeStamp_Physical;
        otherwise =>
            unreachable;           // ConstrainUnpredictableBits removes this case
    end;
end;
```

Library pseudocode for aarch64/debug/brbe/BRB_IALL

```
// BRB_IALL()
// ==========
// Called to perform invalidation of branch records

func BRB_IALL()
begin
    let nbrberecords : integer = GetBRBENumRecords();
    for i = 0 to nbrberecords - 1 do
        Records_SRC[[i]] = Zeros{64};
        Records_TGT[[i]] = Zeros{64};
        Records_INF[[i]] = Zeros{64};
    end;
end;
```

Library pseudocode for aarch64/debug/brbe/BRB_INJ

```
// BRB_INJ()
// =========
// Called to perform manual injection of branch records.

func BRB_INJ()
begin
    UpdateBranchRecordBuffer(BRBINFINJ_EL1().CCU, BRBINFINJ_EL1().CC,
                             BRBINFINJ_EL1().TYPE, BRBINFINJ_EL1().EL,
                             BRBINFINJ_EL1().MPRED, BRBINFINJ_EL1().VALID, BRBSRCINJ_EL1().ADDRESS,
                             BRBTGTINJ_EL1().ADDRESS);
    BRBINFINJ_EL1() = ARBITRARY : bits(64);
    BRBSRCINJ_EL1() = ARBITRARY : bits(64);
    BRBTGTINJ_EL1() = ARBITRARY : bits(64);

    if ConstrainUnpredictableBool(Unpredictable_BRBFILTRATE) then
        PMUEvent(PMU_EVENT_BRB_FILTRATE);
    end;
end;
```

Library pseudocode for aarch64/debug/brbe/BranchEncCycleCount

```
// BranchEncCycleCount()
// =====================

// The first return result is '1' if either of the following is true, and '0' otherwise:
// - This is the first Branch record after the PE exited a Prohibited Region.
// - This is the first Branch record after cycle counting has been enabled.
// If the first return is '0', the second return result is the encoded cycle count
// since the last branch.
// The format of this field uses a mantissa and exponent to express the cycle count value.
//  - bits[7:0] indicate the mantissa M.
//  - bits[13:8] indicate the exponent E.
// The cycle count is expressed using the following function:
//   cycle_count = (if IsZero(E) then UInt(M) else UInt('1'::M::Zeros{UInt(E)-1}))
// A value of all ones in both the mantissa and exponent indicates the cycle count value
// exceeded the size of the cycle counter.
// If the cycle count is not known, the second return result is zero.

func BranchEncCycleCount() => (bit, bits(14))
begin
    var cc : integer = BranchRawCycleCount();
    if !BRBCycleCountingEnabled() || FirstBranchAfterProhibited() then
        return ('1', Zeros{14});
    end;

    // The format of this field uses a mantissa and exponent to express the cycle count value.
    //  - bits[7:0] indicate the mantissa M.
    //  - bits[13:8] indicate the exponent E.
    // The cycle count is expressed using the following function:
    //   cycle_count = (if IsZero(E) then UInt(M) else UInt('1'::M::Zeros{UInt(E)-1}))
    // A value of all ones in both the mantissa and exponent indicates the cycle count value
    // exceeded the size of the cycle counter.

    var E : bits(6);
    var M : bits(8);

    if cc < 2^8 then
        E = Zeros{6};
        M = cc[7:0];
    elsif cc >= 2^20 then
        E = Ones{6};
        M = Ones{8};
    else
        E = 1[5:0];
        while cc >= 2^9 looplimit 11 do
            E = E + 1;
            cc = cc DIVRM 2;
        end;
        M = cc[7:0];
    end;

    return ('0', E::M);
end;
```

Library pseudocode for aarch64/debug/brbe/BranchMispredict

```
// BranchMispredict()
// ==================
// Returns TRUE if the branch being executed was mispredicted, FALSE otherwise.

impdef func BranchMispredict() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/debug/brbe/BranchRawCycleCount

```
// BranchRawCycleCount()
// =====================
// If the cycle count is known, the return result is the cycle count since the last branch.

impdef func BranchRawCycleCount() => integer
begin
    return 0;
end;
```

Library pseudocode for aarch64/debug/brbe/BranchRecordAllowed

```
// BranchRecordAllowed()
// =====================
// Returns TRUE if branch recording is allowed, FALSE otherwise.

func BranchRecordAllowed(el : bits(2)) => boolean
begin
    if ELUsingAArch32(el) then
        return FALSE;
    end;

    if BRBFCR_EL1().PAUSED == '1' then
        return FALSE;
    end;

    if el == EL3 && IsFeatureImplemented(FEAT_BRBEv1p1) then
        return (MDCR_EL3().E3BREC != MDCR_EL3().E3BREW);
    end;

    if HaveEL(EL3) && (MDCR_EL3().SBRBE == '00' ||
        (CurrentSecurityState() == SS_Secure && MDCR_EL3().SBRBE == '01')) then
        return FALSE;
    end;

    case el of
        when EL3 =>  return FALSE;                // FEAT_BRBEv1p1 not implemented
        when EL2 =>  return BRBCR_EL2().E2BRE == '1';
        when EL1 =>  return BRBCR_EL1().E1BRE == '1';
        when EL0 =>
            if EL2Enabled() && HCR_EL2().TGE == '1' then
                return BRBCR_EL2().E0HBRE == '1';
            else
                return BRBCR_EL1().E0BRE == '1';
            end;
    end;
end;
```

Library pseudocode for aarch64/debug/brbe/FilterBranchRecord

```
// FilterBranchRecord()
// ====================
// Returns TRUE if the branch record is not filtered out, FALSE otherwise.

func FilterBranchRecord(br : BranchType, cond : boolean) => boolean
begin
    case br of
        when BranchType_DIRCALL =>
            return BRBFCR_EL1().DIRCALL != BRBFCR_EL1().EnI;
        when BranchType_INDCALL =>
            return BRBFCR_EL1().INDCALL != BRBFCR_EL1().EnI;
        when BranchType_RET =>
            return BRBFCR_EL1().RTN != BRBFCR_EL1().EnI;
        when BranchType_DIR =>
            if cond then
                return BRBFCR_EL1().CONDDIR != BRBFCR_EL1().EnI;
            else
                return BRBFCR_EL1().DIRECT != BRBFCR_EL1().EnI;
            end;
        when BranchType_INDIR =>
            return BRBFCR_EL1().INDIRECT != BRBFCR_EL1().EnI;
        otherwise =>  unreachable;
    end;
    return FALSE;
end;
```

Library pseudocode for aarch64/debug/brbe/FirstBranchAfterProhibited

```
// FirstBranchAfterProhibited()
// ============================
// Returns TRUE if branch recorded is the first branch after a prohibited region,
// FALSE otherwise.

impdef func FirstBranchAfterProhibited() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/debug/brbe/GetBRBENumRecords

```
// GetBRBENumRecords()
// ===================
// Returns the number of branch records implemented.

func GetBRBENumRecords() => integer
begin
    assert UInt(BRBIDR0_EL1().NUMREC) IN {0x08, 0x10, 0x20, 0x40};
    return ImpDefInt("Number of BRBE Branch records");
end;
```

Library pseudocode for aarch64/debug/brbe/ShouldBRBEFreeze

```
// ShouldBRBEFreeze()
// ==================
// Returns TRUE if the BRBE freeze event conditions have been met, and FALSE otherwise.

func ShouldBRBEFreeze() => boolean
begin
    if !BranchRecordAllowed(PSTATE.EL) then
        return FALSE;
    end;

    var include_r1 : boolean;
    var include_r2 : boolean;
    let include_r3 : boolean = FALSE;

    if HaveEL(EL2) then
        include_r1 = (BRBCR_EL1().FZP == '1');
        include_r2 = (BRBCR_EL2().FZP == '1');
    else
        include_r1 = TRUE;
        include_r2 = TRUE;
    end;

    return CheckPMUOverflowCondition(PMUOverflowCondition_BRBEFreeze,
                                     include_r1, include_r2, include_r3);
end;
```

Library pseudocode for aarch64/debug/brbe/UpdateBranchRecordBuffer

```
// UpdateBranchRecordBuffer()
// ==========================
// Add a new Branch record to the buffer.

func UpdateBranchRecordBuffer(ccu : bit, cc : bits(14), branch_type : bits(6), el : bits(2),
                              mispredict : bit, valid : bits(2),
                              source_address : bits(64), target_address : bits(64))
begin
    // Shift the Branch Records in the buffer
    let nbrberecords : integer = GetBRBENumRecords();
    for i = nbrberecords - 1 downto 1 do
        Records_SRC[[i]] = Records_SRC[[i - 1]];
        Records_TGT[[i]] = Records_TGT[[i - 1]];
        Records_INF[[i]] = Records_INF[[i - 1]];
    end;

    Records_INF[[0]].CCU   = ccu;
    Records_INF[[0]].CC    = cc;

    Records_INF[[0]].EL    = el;
    Records_INF[[0]].VALID = valid;
    Records_INF[[0]].MPRED = mispredict;
    Records_INF[[0]].TYPE  = branch_type;

    Records_SRC[[0]]       = source_address;
    Records_TGT[[0]]       = target_address;

    return;
end;
```

Library pseudocode for aarch64/debug/breakpoint/AArch64_BreakpointMatch

```
// AArch64_BreakpointMatch()
// =========================
// Breakpoint matching in an AArch64 translation regime.
// Returns BreakpointInfo structure that contains breakpoint type, a boolean to indicate the
// type of breakpoint, matched address and whether the breakpoint is active and matched
// successfully. For Address Mismatch breakpoints, the returned boolean is the inverted result.
func AArch64_BreakpointMatch(n : integer, vaddress : bits(64), accdesc : AccessDescriptor,
                             size : integer) => BreakpointInfo
begin
    assert !ELUsingAArch32(S1TranslationRegime());
    assert n < NumBreakpointsImplemented();
    var brkptinfo : BreakpointInfo;

    let linking_enabled : boolean = (DBGBCR_EL1(n).BT IN {'0x11', '1xx1'} ||
                                     (IsFeatureImplemented(FEAT_ABLE) && DBGBCR_EL1(n).BT2 == '1'));

    // A breakpoint that has linking enabled does not generate debug events in isolation
    if linking_enabled then
        brkptinfo.bptype = BreakpointType_Inactive;
        brkptinfo.match = FALSE;
        return brkptinfo;
    end;

    let enabled : boolean    = IsBreakpointEnabled(n);
    let linked : boolean     = DBGBCR_EL1(n).BT == '0x01';
    let isbreakpnt : boolean = TRUE;
    let linked_to : boolean  = FALSE;
    let from_linking_enabled : boolean = FALSE;
    let lbnx : bits(2) = if IsFeatureImplemented(FEAT_Debugv8p9) then DBGBCR_EL1(n).LBNX else '00';
    let linked_n : integer{} = UInt(lbnx :: DBGBCR_EL1(n).LBN);
    let ssce : bit = if IsFeatureImplemented(FEAT_RME) then DBGBCR_EL1(n).SSCE else '0';
    let state_match = AArch64_StateMatch(DBGBCR_EL1(n).SSC, ssce, DBGBCR_EL1(n).HMC,
                                         DBGBCR_EL1(n).PMC, linked, linked_n, isbreakpnt,
                                         vaddress, accdesc);

    var (bp_type, value_match) = AArch64_BreakpointValueMatch(n, vaddress, linked_to, isbreakpnt,
                                                              from_linking_enabled);

    if HaveAArch32() && size == 4 then                    // Check second halfword
        // If the breakpoint address and BAS of an Address breakpoint match the address of the
        // second halfword of an instruction, but not the address of the first halfword, it is
        // CONSTRAINED UNPREDICTABLE whether or not this breakpoint generates a Breakpoint debug
        // event.
        let (-, match_i) = AArch64_BreakpointValueMatch(n, vaddress + 2, linked_to, isbreakpnt,
                                                        from_linking_enabled);
        if !value_match && match_i then
            value_match = ConstrainUnpredictableBool(Unpredictable_BPMATCHHALF);
        end;
    end;

    if vaddress[1] == '1' && DBGBCR_EL1(n).BAS == '1111' then
        // The above notwithstanding, if DBGBCR_EL1(n).BAS == '1111', then it is CONSTRAINED
        // UNPREDICTABLE whether or not a Breakpoint debug event is generated for an instruction
        // at the address DBGBVR_EL1(n)+2.
        if value_match then
            value_match = ConstrainUnpredictableBool(Unpredictable_BPMATCHHALF);
        end;
    end;

    if !(state_match && enabled) then
        brkptinfo.bptype = BreakpointType_Inactive;
        brkptinfo.match = FALSE;
    else
        brkptinfo.bptype = bp_type;
        brkptinfo.match = value_match;
    end;
    return brkptinfo;
end;
```

Library pseudocode for aarch64/debug/breakpoint/AArch64_BreakpointValueMatch

```
// AArch64_BreakpointValueMatch()
// ==============================
// Returns breakpoint type to indicate the type of breakpoint and a boolean to indicate
// whether the breakpoint matched successfully. For Address Mismatch breakpoints, the
// returned boolean is the inverted result. If the breakpoint type return value is Inactive,
// then the boolean result is FALSE.

func AArch64_BreakpointValueMatch(n_in : integer, vaddress : bits(64),
                                  linked_to : boolean, isbreakpnt : boolean,
                                  from_linking_enabled : boolean
                                 ) => (BreakpointType, boolean) recurselimit 2
begin

    // "n_in" is the identity of the breakpoint unit to match against.
    // "vaddress" is the current instruction address, ignored if linked_to is TRUE and for Context
    //   matching breakpoints.
    // "linked_to" is TRUE if this is a call from StateMatch for linking.
    // "isbreakpnt"  TRUE is this is a call from BreakpointMatch or from StateMatch for a
    // linked breakpoint or from BreakpointValueMatch for a linked breakpoint with linking enabled.
    // "from_linking_enabled" is TRUE if this is a call from BreakpointValueMatch for a linked
    // breakpoint with linking enabled.
    var n : integer = n_in;
    var c : Constraint;
    var dbgtype : bits(5);

    // If a non-existent breakpoint then it is CONSTRAINED UNPREDICTABLE whether this gives
    // no match or the breakpoint is mapped to another UNKNOWN implemented breakpoint.
    if n >= NumBreakpointsImplemented() then
        (c, n) = ConstrainUnpredictableInteger(0, NumBreakpointsImplemented() - 1,
                                               Unpredictable_BPNOTIMPL);
        assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};
        if c == Constraint_DISABLED then return (BreakpointType_Inactive, FALSE); end;
    end;

    // If this breakpoint is not enabled, it cannot generate a match.
    // (This could also happen on a call from StateMatch for linking).
    if !IsBreakpointEnabled(n) then return (BreakpointType_Inactive, FALSE); end;

    // If BT is set to a reserved type, behaves either as disabled or as a not-reserved type.
    if IsFeatureImplemented(FEAT_ABLE) then
        dbgtype = DBGBCR_EL1(n).[BT2,BT];
    else
        dbgtype = '0' :: DBGBCR_EL1(n).BT;
    end;

    (c, dbgtype) = AArch64_ReservedBreakpointType(n, dbgtype);
    if c == Constraint_DISABLED then return (BreakpointType_Inactive, FALSE); end;
    // Otherwise the value returned by ConstrainUnpredictableBits must be a not-reserved value

    // Determine what to compare against.
    let match_addr : boolean      = (dbgtype == 'x0x0x');
    let mismatch : boolean        = (dbgtype == 'x010x');
    let match_vmid : boolean      = (dbgtype == 'x10xx');
    let match_cid : boolean       = (dbgtype == 'x001x');
    let match_cid1 : boolean      = (dbgtype IN {'x101x', 'xx11x'});
    let match_cid2 : boolean      = (dbgtype == 'x11xx');
    let linking_enabled : boolean = (dbgtype IN {'xxx11', 'x1xx1', '1xxxx'});

    // If this is a call from StateMatch, return FALSE if the breakpoint is not
    // programmed with linking enabled.
    if linked_to && !linking_enabled then
        return (BreakpointType_Inactive, FALSE);
    end;

    // If called from BreakpointMatch return FALSE for breakpoint with linking enabled.
    if !linked_to && linking_enabled then
        return (BreakpointType_Inactive, FALSE);
    end;

    let linked : boolean = (dbgtype == 'x0x01');
    if from_linking_enabled then  // A breakpoint with linking enabled has called this function.
        assert linked_to && isbreakpnt;
        if linked then
            // A breakpoint with linking enabled is linked to a linked breakpoint. This is
            // architecturally UNPREDICTABLE, but treated as disabled in the pseudo code to
            // avoid potential recursion in BreakpointValueMatch().
            return (BreakpointType_Inactive, FALSE);
        end;
    end;

    // If a linked breakpoint is linked to an address matching breakpoint,
    // the behavior is CONSTRAINED UNPREDICTABLE.
    if linked_to && match_addr && isbreakpnt then
        if !ConstrainUnpredictableBool(Unpredictable_BPLINKEDADDRMATCH) then
            return (BreakpointType_Inactive, FALSE);
        end;
    end;

    // A breakpoint programmed for address mismatch does not match in AArch32 state.
```

```
    if mismatch && UsingAArch32() then
        return (BreakpointType_Inactive, FALSE);
    end;

    var bvr_match : boolean  = FALSE;
    var bxvr_match : boolean = FALSE;
    var bp_type : BreakpointType;
    var mask : integer{0..31};

    if IsFeatureImplemented(FEAT_BWE) then
        mask = UInt(DBGBCR_EL1(n).MASK);

        // If the mask is set to a reserved value, the behavior is CONSTRAINED UNPREDICTABLE.
        if mask IN {1, 2} then
            var unpred_mask : integer;
            (c, unpred_mask) = ConstrainUnpredictableInteger(3, 31, Unpredictable_RESBPMASK);
            assert c IN {Constraint_DISABLED, Constraint_NONE, Constraint_UNKNOWN};
            case c of
                when Constraint_DISABLED => return (BreakpointType_Inactive, FALSE);  // Disabled
                when Constraint_NONE =>     mask = 0;                                 // No masking
                // Otherwise the value returned by ConstrainUnpredictableBits must
                // be a not-reserved value.
                otherwise => mask = unpred_mask as integer{3..31};
            end;
        end;

        if mask != 0 then
            // When DBGBCR_EL1(n).MASK is a valid nonzero value, the behavior is
            // CONSTRAINED UNPREDICTABLE if any of the following are true:
            //     - DBGBCR_EL1(n).[BT2,BT] is programmed for a Context matching breakpoint.
            //     - DBGBCR_EL1(n).BAS is not '1111' and AArch32 is supported at EL0.
            if ((match_cid || match_cid1 || match_cid2) ||
                (DBGBCR_EL1(n).BAS != '1111' && HaveAArch32())) then
                if !ConstrainUnpredictableBool(Unpredictable_BPMASK) then
                    return (BreakpointType_Inactive, FALSE);
                end;
            end;
        else
            // A stand-alone mismatch of a single address is not supported.
            if mismatch then
                return (BreakpointType_Inactive, FALSE);
            end;
        end;

    else
        mask = 0;
    end;

    // Do the comparison.
    if match_addr then
        var byte_select_match : boolean;
        let byte : integer = UInt(vaddress[1:0]);

        if HaveAArch32() then
            // T32 instructions can be executed at EL0 in an AArch64 translation regime.
            assert byte IN {0,2};                       // "vaddress" is halfword aligned
            byte_select_match = (DBGBCR_EL1(n).BAS[byte] == '1');
        else
            assert byte == 0;                           // "vaddress" is word aligned
            byte_select_match = TRUE;                   // DBGBCR_EL1(n).BAS[byte] is RES1
        end;

        // When FEAT_LVA3 is not implemented, if the DBGBVR_EL1(n).RESS field bits are not a
        // sign extension of the MSB of DBGBVR_EL1(n).VA, it is UNPREDICTABLE whether they
        // appear to be included in the match.
        // If 'vaddress' is outside of the current virtual address space, then the access
        // generates a Translation fault.
        let dbgtop : AddressSize = DebugAddrTop();
        let unpredictable_ress : boolean = (dbgtop < 55 && !IsOnes(DBGBVR_EL1(n)[63:dbgtop]) &&
                                            !IsZero(DBGBVR_EL1(n)[63:dbgtop]) &&
                                       ConstrainUnpredictableBool(Unpredictable_DBGxVR_RESS));
        let cmpmsb : integer{} = if unpredictable_ress then 63 else dbgtop;
        let cmplsb : integer{} = if mask > 2 then mask else 2;
        bvr_match = ((vaddress[cmpmsb:cmplsb] == DBGBVR_EL1(n)[cmpmsb:cmplsb]) &&
                     byte_select_match);
        if mask > 2 then
            // If masked bits of DBGBVR_EL1(n) are not zero, the behavior
            // is CONSTRAINED UNPREDICTABLE.
            let masktop : integer{} = mask - 1;
            if bvr_match && !IsZero(DBGBVR_EL1(n)[masktop:2]) then
                bvr_match = ConstrainUnpredictableBool(Unpredictable_BPMASKEDBITS);
            end;
        end;

    elsif match_cid then
        if IsInHost() then
            bvr_match = (CONTEXTIDR_EL2()[31:0] == DBGBVR_EL1(n)[31:0]);
        else
            bvr_match = (PSTATE.EL IN {EL0, EL1} && CONTEXTIDR_EL1()[31:0] == DBGBVR_EL1(n)[31:0]);
```

```
        end;

    elsif match_cid1 then
        bvr_match = (PSTATE.EL IN {EL0, EL1} && !IsInHost() &&
                     CONTEXTIDR_EL1()[31:0] == DBGBVR_EL1(n)[31:0]);
    end;

    if match_vmid then
        let vmid : bits(NUM_VMIDBITS) = VMID();
        var bvr_vmid : bits(NUM_VMIDBITS);

        if !IsFeatureImplemented(FEAT_VMID16) || VTCR_EL2().VS == '0' then
            bvr_vmid = ZeroExtend{NUM_VMIDBITS}(DBGBVR_EL1(n)[39:32]);
        else
            bvr_vmid = DBGBVR_EL1(n)[47:32];
        end;

        bxvr_match = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() && !IsInHost() && vmid == bvr_vmid);

    elsif match_cid2 then
        bxvr_match = (PSTATE.EL != EL3 && EL2Enabled() &&
                      DBGBVR_EL1(n)[63:32] == CONTEXTIDR_EL2()[31:0]);
    end;

    let bvr_match_valid : boolean  = (match_addr || match_cid || match_cid1);
    let bxvr_match_valid : boolean = (match_vmid || match_cid2);

    var value_match : boolean = ((!bxvr_match_valid || bxvr_match) &&
                                 (!bvr_match_valid || bvr_match));

    // A watchpoint might be linked to a linked address matching breakpoint with linking enabled,
    // which is in turn linked to a context matching breakpoint.
    if linked_to && linked then
        // If called from StateMatch and breakpoint is a linked breakpoint then it must be a
        // watchpoint that is linked to an address matching breakpoint which is linked to a
        // context matching breakpoint.
        assert !isbreakpnt && match_addr && IsFeatureImplemented(FEAT_ABLE);
        let lbnx : bits(2) = (if IsFeatureImplemented(FEAT_Debugv8p9) then DBGBCR_EL1(n).LBNX
                              else '00');
        let linked_linked_n : integer{} = UInt(lbnx :: DBGBCR_EL1(n).LBN);
        var linked_value_match : boolean;
        let linked_vaddress = ARBITRARY : bits(64);
        let linked_linked_to = TRUE;
        let linked_isbreakpnt = TRUE;
        let linked_from_linking_enabled = TRUE;
        (bp_type, linked_value_match) = AArch64_BreakpointValueMatch(linked_linked_n,
                                                                     linked_vaddress,
                                                                     linked_linked_to,
                                                                     linked_isbreakpnt,
                                                                     linked_from_linking_enabled);
        value_match = value_match && linked_value_match;
    end;

    if match_addr && !mismatch then
        bp_type = BreakpointType_AddrMatch;
    elsif match_addr && mismatch then
        bp_type = BreakpointType_AddrMismatch;
    elsif match_vmid || match_cid || match_cid1 || match_cid2 then
        bp_type = BreakpointType_CtxtMatch;
    else
        unreachable;
    end;

    return (bp_type, value_match);
end;
```

Library pseudocode for aarch64/debug/breakpoint/AArch64_ReservedBreakpointType

```
// AArch64_ReservedBreakpointType()
// ================================
// Checks if the given DBGBCR_EL1(n).[BT2,BT] values are reserved and will
// generate Constrained Unpredictable behavior, otherwise returns Constraint_NONE.

func AArch64_ReservedBreakpointType(n : integer, bt_in : bits(5)) => (Constraint, bits(5))
begin
    var bt : bits(5)            = bt_in;
    var reserved : boolean      = FALSE;
    let context_aware : boolean = IsContextAwareBreakpoint(n);

    if bt[4] == '0' then
        // Context matching
        if bt != '00x0x' && !context_aware then
            reserved = TRUE;
        end;

        // EL2 extension
        if bt == '01xxx' && !HaveEL(EL2) then
            reserved = TRUE;
        end;

        // Context matching
        if (bt IN {'0011x','011xx'} && !IsFeatureImplemented(FEAT_VHE) &&
              !IsFeatureImplemented(FEAT_Debugv8p2)) then
            reserved = TRUE;
        end;

        // Reserved
        if bt == '0010x' && !IsFeatureImplemented(FEAT_BWE) && !HaveAArch32EL(EL1) then
            reserved = TRUE;
        end;
    else
        // Reserved
        if bt != '10x0x' then
            reserved = TRUE;
        end;
    end;

    if reserved then
        var c : Constraint;
        (c, bt) = ConstrainUnpredictableBits{5}(Unpredictable_RESBPTYPE);
        assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};
        if c == Constraint_DISABLED then
            return (c, ARBITRARY : bits(5));
        // Otherwise the value returned by ConstrainUnpredictableBits must be a not-reserved value
        end;
    end;

    return (Constraint_NONE, bt);
end;
```

Library pseudocode for aarch64/debug/breakpoint/AArch64_StateMatch

```
// AArch64_StateMatch()
// ====================
// Determine whether a breakpoint or watchpoint is enabled in the current mode and state.

func AArch64_StateMatch(ssc_in : bits(2), ssce_in : bit, hmc_in : bit,
                        pxc_in : bits(2), linked_in : boolean, linked_n_in : integer,
                        isbreakpnt : boolean, vaddress : bits(64),
                        accdesc : AccessDescriptor) => boolean
begin
    if !IsFeatureImplemented(FEAT_RME) then assert ssce_in == '0'; end;

    // "ssc_in","ssce_in","hmc_in","pxc_in" are the control fields from
    // the DBGBCR_EL1(n) or DBGWCR_EL1(n) register.
    // "linked_in" is TRUE if this is a linked breakpoint/watchpoint type.
    // "linked_n_in" is the linked breakpoint number from the DBGBCR_EL1(n) or
    // DBGWCR_EL1(n) register.
    // "isbreakpnt" is TRUE for breakpoints, FALSE for watchpoints.
    // "vaddress" is the program counter for a linked watchpoint or the same value passed to
    // AArch64_CheckBreakpoint for a linked breakpoint.
    // "accdesc" describes the properties of the access being matched.
    var ssc : bits(2)      = ssc_in;
    var ssce : bit         = ssce_in;
    var hmc : bit          = hmc_in;
    var pxc : bits(2)      = pxc_in;
    let linked : boolean   = linked_in;
    let linked_n : integer = linked_n_in;

    // If parameters are set to a reserved type, behaves as either disabled or a defined type
    var c : Constraint;
    (c, ssc, ssce, hmc, pxc) = CheckValidStateMatch(ssc, ssce, hmc, pxc, isbreakpnt);
    if c == Constraint_DISABLED then return FALSE; end;
    // Otherwise the hmc,ssc,ssce,pxc values are either valid or the values returned by
    // CheckValidStateMatch are valid.

    let EL3_match : boolean = HaveEL(EL3) && hmc == '1' && ssc[0] == '0';
    let EL2_match : boolean = HaveEL(EL2) && ((hmc == '1' && (ssc::pxc != '1000')) || ssc == '11');
    let EL1_match : boolean = pxc[0] == '1';
    let EL0_match : boolean = pxc[1] == '1';

    var priv_match : boolean;
    case accdesc.el of
        when EL3 =>  priv_match = EL3_match;
        when EL2 =>  priv_match = EL2_match;
        when EL1 =>  priv_match = EL1_match;
        when EL0 =>  priv_match = EL0_match;
    end;

    // Security state match
    var ss_match : boolean;
    case ssce::ssc of
        when '000' => ss_match = hmc == '1' || accdesc.ss != SS_Root;
        when '001' => ss_match = accdesc.ss == SS_NonSecure;
        when '010' => ss_match = (hmc == '1' && accdesc.ss == SS_Root) || accdesc.ss == SS_Secure;
        when '011' => ss_match = (hmc == '1' && accdesc.ss != SS_Root) || accdesc.ss == SS_Secure;
        when '101' => ss_match = accdesc.ss == SS_Realm;
    end;

    var linked_match : boolean = FALSE;

    if linked then
        // "linked_n" must be an enabled context-aware breakpoint unit. If it is not context-aware
        // then it is CONSTRAINED UNPREDICTABLE whether this gives no match, gives a match without
        // linking, or linked_n is mapped to some UNKNOWN breakpoint that is context-aware.
        if (!IsContextAwareBreakpoint(linked_n) &&
            (isbreakpnt || !IsAddressLinkingBreakpoint(linked_n))) then
            return ConstrainUnpredictableBool(Unpredictable_BPNOTLINKING);
        end;
    end;

    if linked then
        let linked_to : boolean = TRUE;
        var bp_type : BreakpointType;
        let from_linking_enabled : boolean = FALSE;
        (bp_type, linked_match) = AArch64_BreakpointValueMatch(linked_n, vaddress,
                                                               linked_to, isbreakpnt,
                                                               from_linking_enabled);
        if bp_type == BreakpointType_AddrMismatch then
            linked_match = !linked_match;
        end;
    end;

    return priv_match && ss_match && (!linked || linked_match);
end;
```

Library pseudocode for aarch64/debug/breakpoint/DebugAddrTop

```
// DebugAddrTop()
// ==============
// Returns the value for the top bit used in Breakpoint and Watchpoint address comparisons.

func DebugAddrTop() => AddressSize
begin
    if IsFeatureImplemented(FEAT_LVA3) then
        return 55;
    elsif IsFeatureImplemented(FEAT_LVA) then
        return 52;
    else
        return 48;
    end;
end;
```

Library pseudocode for aarch64/debug/breakpoint/EffectiveMDSELR_EL1_BANK

```
// EffectiveMDSELR_EL1_BANK()
// ==========================
// Return the effective value of MDSELR_EL1.BANK.

func EffectiveMDSELR_EL1_BANK() => bits(2)
begin
    // If 16 or fewer breakpoints and 16 or fewer watchpoints are implemented,
    // then the field is RES0.
    let num_bp : integer = NumBreakpointsImplemented();
    let num_wp : integer = NumWatchpointsImplemented();
    if num_bp <= 16 && num_wp <= 16 then
        return '00';
    end;

    // At EL3, the Effective value of this field is zero if MDCR_EL3.EBWE is 0.
    // At EL2, the Effective value is zero if the Effective value of MDCR_EL2.EBWE is 0.
    // That is, if either MDCR_EL3.EBWE is 0 or MDCR_EL2.EBWE is 0.
    // At EL1, the Effective value is zero if the Effective value of MDSCR_EL2.EMBWE is 0.
    // That is, if any of MDCR_EL3.EBWE, MDCR_EL2.EBWE, or MDSCR_EL1.EMBWE is 0.
    if ((HaveEL(EL3) && MDCR_EL3().EBWE == '0') ||
          (PSTATE.EL != EL3 && EL2Enabled() && MDCR_EL2().EBWE == '0') ||
          (PSTATE.EL == EL1 && MDSCR_EL1().EMBWE == '0')) then
        return '00';
    end;

    var bank : bits(2) = MDSELR_EL1().BANK;

    // Values are reserved depending on the number of breakpoints or watchpoints
    // implemented.
    if ((bank == '11' && num_bp <= 48 && num_wp <= 48) ||
          (bank == '10' && num_bp <= 32 && num_wp <= 32)) then
        // Reserved value
        (-, bank) = ConstrainUnpredictableBits{2}(Unpredictable_RESMDSELR);
        // The value returned by ConstrainUnpredictableBits must be a not-reserved value
    end;

    return bank;
end;
```

Library pseudocode for aarch64/debug/breakpoint/IsAddressLinkingBreakpoint

```
// IsAddressLinkingBreakpoint()
// ============================
// Returns TRUE if DBGBCR_EL1(n) supports being configured as an Address matching breakpoint with
// linking enabled.

func IsAddressLinkingBreakpoint(n : integer) => boolean
begin
    return n < NumAddressLinkingBreakpointsImplemented();
end;
```

Library pseudocode for aarch64/debug/breakpoint/IsBreakpointEnabled

```
// IsBreakpointEnabled()
// =====================
// Returns TRUE if the effective value of DBGBCR_EL1(n).E is '1', and FALSE otherwise.

func IsBreakpointEnabled(n : integer) => boolean
begin
    if (n > 15 &&
          ((!HaltOnBreakpointOrWatchpoint() && !SelfHostedExtendedBPWPEnabled()) ||
          (HaltOnBreakpointOrWatchpoint() && EDSCR2().EHBWE == '0'))) then
        return FALSE;
    end;

    return DBGBCR_EL1(n).E == '1';
end;
```

Library pseudocode for aarch64/debug/breakpoint/NumAddressLinkingBreakpointsImplemented

```
// NumAddressLinkingBreakpointsImplemented()
// =========================================
// Returns the number of breakpoints that support being configured as Address matching breakpoint
// with linking enabled, as described by ID_AA64DFR1_EL1().ABL_CMPs.

func NumAddressLinkingBreakpointsImplemented() => integer
begin
    if !IsFeatureImplemented(FEAT_ABLE) then
        return 0;
    else
        return ImpDefInt("Number of address-linking breakpoints");
    end;
end;
```

Library pseudocode for aarch64/debug/breakpoint/SelfHostedExtendedBPWPEnabled

```
// SelfHostedExtendedBPWPEnabled()
// ===============================
// Returns TRUE if the extended breakpoints and watchpoints are enabled, and FALSE otherwise
// from a self-hosted debug perspective.

func SelfHostedExtendedBPWPEnabled() => boolean
begin
    if NumBreakpointsImplemented() <= 16 && NumWatchpointsImplemented() <= 16 then
        return FALSE;
    end;

    if ((HaveEL(EL3) && MDCR_EL3().EBWE == '0') ||
          (EL2Enabled() && MDCR_EL2().EBWE == '0')) then
        return FALSE;
    end;

    return MDSCR_EL1().EMBWE == '1';
end;
```

Library pseudocode for aarch64/debug/ebep/CheckForPMUException

```
// CheckForPMUException()
// ======================
// Take a PMU exception if enabled, permitted, and unmasked.

func CheckForPMUException()
begin
    var enabled : boolean;
    var target_el : bits(2);
    var pmu_exception : boolean;
    var synchronous : boolean;
    (enabled, target_el) = PMUExceptionEnabled();
    if !enabled || PMUExceptionMasked(target_el, PSTATE.EL, PSTATE.PM) then
        pmu_exception = FALSE;
    else
        let include_r1 : boolean = TRUE;
        let include_r2 : boolean = TRUE;
        let include_r3 : boolean = TRUE;
        pmu_exception = CheckPMUOverflowCondition(PMUOverflowCondition_PMUException,
                                                  include_r1, include_r2, include_r3);
        synchronous = FALSE;
    end;
    if pmu_exception then
        let fsc : bits(5) = '00000';
        TakeProfilingException(target_el, fsc, synchronous);
    end;
end;
```

Library pseudocode for aarch64/debug/ebep/EffectivePMEE

```
// EffectivePMEE()
// ===============
// Return the Effective value of PMEE from MDCR_ELx or PMECR_EL1, handling reserved encodings:
// '10' (MDCR_ELx), '10' and '01' (PMECR_EL1).

func EffectivePMEE(el : bits(2), for_interrupt : boolean) => bits(2)
begin
    var val : bits(2);
    case el of
        when EL1  => val = PMECR_EL1().PMEE;
        when EL2  => val = MDCR_EL2().PMEE;
        when EL3  => val = MDCR_EL3().PMEE;
        otherwise => unreachable;
    end;

    if (val == '10' || (el == EL1 && val == '01')) then
        var c : Constraint;
        (c, val) = ConstrainUnpredictableBits{2}(Unpredictable_RESPMEE);
        assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};
        if c == Constraint_DISABLED then
            val = if for_interrupt then '11' else '00';
            // Otherwise the value returned by ConstrainUnpredictableBits must be
            // a non-reserved value
        end;
    end;
    return val;
end;
```

Library pseudocode for aarch64/debug/ebep/PMUExceptionEnabled

```
// PMUExceptionEnabled()
// =====================
// The first return value is TRUE if the PMU exception is enabled, and FALSE otherwise.
// The second return value is the target Exception level for an enabled PMU exception.

func PMUExceptionEnabled() => (boolean, bits(2))
begin

    if !IsFeatureImplemented(FEAT_EBEP) then
        return (FALSE, ARBITRARY : bits(2));
    end;

    var enabled : boolean;
    var target : bits(2) = ARBITRARY : bits(2);
    let for_interrupt : boolean = FALSE;
    let pmee_el3 : bits(2) = EffectivePMEE(EL3, for_interrupt);
    let pmee_el2 : bits(2) = EffectivePMEE(EL2, for_interrupt);
    let pmee_el1 : bits(2) = EffectivePMEE(EL1, for_interrupt);

    if HaveEL(EL3) && pmee_el3 != '01' then
        enabled = pmee_el3 == '11';
        if enabled then target = EL3; end;

    elsif EL2Enabled() && pmee_el2 != '01' then
        enabled = pmee_el2 == '11';
        if enabled then target = EL2; end;

    else
        enabled = pmee_el1 == '11';
        if enabled then
            target = if EL2Enabled() && HCR_EL2().TGE == '1' then EL2 else EL1;
        end;
    end;

    return (enabled, target);
end;
```

Library pseudocode for aarch64/debug/ebep/PMUExceptionMasked

```
// PMUExceptionMasked()
// ====================
// Return TRUE if the PMU Exception is masked at the specified target Exception level
// relative to the specified source Exception level, and by the value of PSTATE.PM,
// and FALSE otherwise.

func PMUExceptionMasked(target_el : bits(2), from_el : bits(2), pm : bit) => boolean
begin
    assert IsFeatureImplemented(FEAT_EBEP);
    let for_interrupt : boolean = FALSE;

    if Halted() then
        return TRUE;
    elsif UInt(target_el) < UInt(from_el) then
        return TRUE;
    elsif (from_el == EL2 && target_el == EL2 &&
           EffectivePMEE(EL2, for_interrupt) != '11') then
        return TRUE;
    elsif target_el == from_el && (PMECR_EL1().KPME == '0' || pm == '1') then
        return TRUE;
    end;

    return FALSE;
end;
```

Library pseudocode for aarch64/debug/ebep/PMUInterruptEnabled

```
// PMUInterruptEnabled()
// =====================
// Return TRUE if the PMU interrupt request (PMUIRQ) is enabled, FALSE otherwise.

func PMUInterruptEnabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_EBEP) then
        return TRUE;
    end;

    var enabled : boolean;
    let for_interrupt : boolean = TRUE;
    let pmee_el3 : bits(2) = EffectivePMEE(EL3, for_interrupt);
    let pmee_el2 : bits(2) = EffectivePMEE(EL2, for_interrupt);
    let pmee_el1 : bits(2) = EffectivePMEE(EL1, for_interrupt);

    if HaveEL(EL3) && pmee_el3 != '01' then
        enabled = pmee_el3 == '00';

    elsif EL2Enabled() && pmee_el2 != '01' then
        enabled = pmee_el2 == '00';

    else
        enabled = pmee_el1 == '00';
    end;

    return enabled;
end;
```

Library pseudocode for aarch64/debug/enables/AArch64_GenerateDebugExceptions

```
// AArch64_GenerateDebugExceptions()
// =================================

func AArch64_GenerateDebugExceptions() => boolean
begin
    let ss : SecurityState = CurrentSecurityState();
    return AArch64_GenerateDebugExceptionsFrom(PSTATE.EL, ss, PSTATE.D);
end;
```

Library pseudocode for aarch64/debug/enables/AArch64_GenerateDebugExceptionsFrom

```
// AArch64_GenerateDebugExceptionsFrom()
// =====================================

func AArch64_GenerateDebugExceptionsFrom(from_el : bits(2), from_state : SecurityState,
                                         mask : bit) => boolean
begin

    if OSLSR_EL1().OSLK == '1' || DoubleLockStatus() || Halted() then
        return FALSE;
    end;

    let route_to_el2 : boolean = (HaveEL(EL2) &&
                                  (from_state != SS_Secure || IsSecureEL2Enabled()) &&
                                  (HCR_EL2().TGE == '1' || MDCR_EL2().TDE == '1'));
    let target : bits(2) = (if route_to_el2 then EL2 else EL1);
    var enabled : boolean;
    if HaveEL(EL3) && from_state == SS_Secure then
        enabled = MDCR_EL3().SDD == '0';
        if from_el == EL0 && ELUsingAArch32(EL1) then
            enabled = enabled || SDER32_EL3().SUIDEN == '1';
        end;
    else
        enabled = TRUE;
    end;

    if from_el == target then
        enabled = enabled && MDSCR_EL1().KDE == '1' && mask == '0';
    else
        enabled = enabled && UInt(target) > UInt(from_el);
    end;

    return enabled;
end;
```

Library pseudocode for aarch64/debug/ite/AArch64_TRCIT

```
// AArch64_TRCIT()
// ===============
// Determines whether an Instrumentation trace packet should
// be generated and then generates an instrumentation trace packet
// containing the value of the register passed as an argument

func AArch64_TRCIT(Xt : bits(64))
begin
    let ss = CurrentSecurityState();
    if TraceInstrumentationAllowed(ss, PSTATE.EL) then
        TraceInstrumentation(Xt);
    end;
end;
```

Library pseudocode for aarch64/debug/ite/TraceInstrumentation

```
// TraceInstrumentation()
// ======================
// Generates an instrumentation trace packet
// containing the value of the register passed as an argument

impdef func TraceInstrumentation(Xt : bits(64))
begin
    return;
end;
```

Library pseudocode for aarch64/debug/pmu/AArch64_IncrementCycleCounter

```
// AArch64_IncrementCycleCounter()
// ===============================
// Increment the cycle counter and possibly set overflow bits.

func AArch64_IncrementCycleCounter()
begin
    if !CountPMUEvents(CYCLE_COUNTER_ID) then return; end;
    var d : bit = PMCR_EL0().D;   // Check divide-by-64
    var lc : bit = PMCR_EL0().LC;
    var lc_enabled : boolean;
    (lc_enabled, -) = PMUExceptionEnabled();
    lc = if lc_enabled then '1' else lc;
    // Effective value of 'D' bit is 0 when Effective value of LC is '1'
    if lc == '1' then d = '0'; end;
    if d == '1' && !HasElapsed64Cycles() then return; end;

    let old_value : integer = UInt(PMCCNTR_EL0());
    let new_value : integer = old_value + 1;
    PMCCNTR_EL0() = new_value[63:0];

    let ovflw : integer{} = if HaveAArch32() && lc == '0' then 32 else 64;

    if old_value[64:ovflw] != new_value[64:ovflw] then
        PMOVSSET_EL0().C = '1';
    end;

    return;
end;
```

Library pseudocode for aarch64/debug/pmu/AArch64_IncrementEventCounter

```
// AArch64_IncrementEventCounter()
// ===============================
// Increment the specified event counter 'idx' by the specified amount 'increment'.
// 'Vm' is the value event counter 'idx-1' is being incremented by if 'idx' is odd,
// zero otherwise.
// Returns the amount the counter was incremented by.

func AArch64_IncrementEventCounter(idx : integer, increment_in : integer, Vm : integer) => integer
begin
    var old_value : integer;
    var new_value : integer;

    old_value = UInt(PMEVCNTR_EL0(idx));
    let increment : integer = PMUCountValue(idx, increment_in, Vm);
    new_value = old_value + increment;

    var lp : bit;
    if IsFeatureImplemented(FEAT_PMUv3p5) then
        PMEVCNTR_EL0(idx) = new_value[63:0];
        var pmuexception_enabled : boolean;
        (pmuexception_enabled, -) = PMUExceptionEnabled();
        if pmuexception_enabled then
            lp = '1';
        else
            case GetPMUCounterRange(idx) of
                when PMUCounterRange_R1 =>
                    lp = PMCR_EL0().LP;
                when PMUCounterRange_R2 =>
                    lp = MDCR_EL2().HLP;
                when PMUCounterRange_R3 =>
                    lp = '1';
                otherwise =>
                    unreachable;
            end;
        end;
    else
        lp = '0';
        PMEVCNTR_EL0(idx) = ZeroExtend{64}(new_value[31:0]);
    end;
    let ovflw : integer{} = if lp == '1' then 64 else 32;
    if old_value[64:ovflw] != new_value[64:ovflw] then
        PMOVSSET_EL0()[idx] = '1';
        // Check for the CHAIN event from an even counter
        if (idx[0] == '0' && idx + 1 < NUM_PMU_COUNTERS && lp == '0' &&
              (GetPMUCounterRange(idx) == GetPMUCounterRange(idx+1) ||
                 ConstrainUnpredictableBool(Unpredictable_COUNT_CHAIN))) then
            // If PMU counters idx and idx+1 are not in same range,
            // it is CONSTRAINED UNPREDICTABLE if CHAIN event is counted
            PMUEvent(PMU_EVENT_CHAIN, 1, idx + 1);
        end;
    end;

    return increment;
end;
```

Library pseudocode for aarch64/debug/pmu/AArch64_PMUCycle

```
// AArch64_PMUCycle()
// ==================
// Called at the end of each cycle to increment event counters and
// check for PMU overflow. In pseudocode, a cycle ends after the
// execution of the operational pseudocode.

func AArch64_PMUCycle()
begin
    if !IsFeatureImplemented(FEAT_PMUv3) then
        return;
    end;
    if ((!IsWFIsleep && !IsWFEsleep) || (!IsFeatureImplemented(FEAT_PMUv3p9) &&
                                         ImpDefBool("PMU counts cycles in WFI or WFE state"))) then
        PMUEvent(PMU_EVENT_CPU_CYCLES);
    end;
    let counters : integer = NUM_PMU_COUNTERS;
    var Vm : integer = 0;
    if counters != 0 then
        for idx = 0 to counters - 1 do
            if CountPMUEvents(idx) then
                let accumulated : integer = PMUEventAccumulator[[idx]];
                if (idx MOD 2) == 0 then Vm = 0; end;
                Vm = AArch64_IncrementEventCounter(idx, accumulated, Vm);
            end;
            PMUEventAccumulator[[idx]] = 0;
        end;
    end;
    AArch64_IncrementCycleCounter();
    CheckForPMUOverflow();
end;
```

Library pseudocode for aarch64/debug/profilingexception/TakeProfilingException

```
// TakeProfilingException()
// ========================
// Takes a Profiling exception.

func TakeProfilingException(target_el : bits(2), fsc : bits(5), synchronous : boolean)
begin
    var except : ExceptionRecord = ExceptionSyndrome(Exception_Profiling);
    except.syndrome.iss[5:1] = fsc;
    if synchronous then
        except.syndrome.iss[0] = '1';
    end;

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/CheckForSPEException

```
// CheckForSPEException()
// ======================
// Take an SPE Profiling exception if pending, permitted, and unmasked.

func CheckForSPEException()
begin
    if !IsFeatureImplemented(FEAT_SPE_EXC) then
        return;
    end;

    if Halted() || Restarting() then
        return;
    end;

    var route_to_el3 : boolean = FALSE;
    var route_to_el2 : boolean = FALSE;
    var route_to_el1 : boolean = FALSE;

    if HaveEL(EL3) && MDCR_EL3().PMSEE == '1x' then
        let pending : boolean = PMBSR_EL3().S == '1';
        let masked : boolean = PSTATE.EL == EL3;
        route_to_el3 = pending && !masked;
    end;

    var owning_ss : SecurityState;
    var owning_el : bits(2);
    (owning_ss, owning_el) = ProfilingBufferOwner();
    let in_owning_ss : boolean = IsCurrentSecurityState(owning_ss);

    if EffectivePMSCR_EL2_EE() == '1x' then
        let pending : boolean = PMBSR_EL2().S == '1';
        let masked : boolean = (!in_owning_ss || PSTATE.EL == EL3 ||
                                (PSTATE.EL == EL2 && (PMSCR_EL2().EE != '11' ||
                                 PMSCR_EL2().KE == '0' || PSTATE.PM == '1'))
                                );
        route_to_el2 = pending && !masked;
    end;

    if EffectivePMSCR_EL1_EE() == '11' then
        let pending : boolean = PMBSR_EL1().S == '1';
        let masked : boolean = (!in_owning_ss || PSTATE.EL IN {EL3, EL2} ||
                                (PSTATE.EL == EL1 && (PMSCR_EL1().KE == '0' || PSTATE.PM == '1'))
                                );
        if EffectiveTGE() == '1' then
            route_to_el2 = route_to_el2 || (pending && !masked);
        else
            route_to_el1 = pending && !masked;
        end;
    end;

    let fsc : bits(5) = '00001';        // SPE exception
    let synchronous : boolean = FALSE;

    // The relative priorities of the following checks is IMPLEMENTATION DEFINED
    if route_to_el3 then
        TakeProfilingException(EL3, fsc, synchronous);
    end;
    if route_to_el2 then
        TakeProfilingException(EL2, fsc, synchronous);
    end;
    if route_to_el1 then
        TakeProfilingException(EL1, fsc, synchronous);
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/CheckMDCR_EL3_NSPBTrap

```
// CheckMDCR_EL3_NSPBTrap()
// ========================
// Check if the register access is trappable by MDCR_EL3.[NSPBE, NSPB]

func CheckMDCR_EL3_NSPBTrap() => boolean
begin
    var state_bits : bits(3);
    var reserved : boolean;
    (state_bits, reserved) = EffectiveMDCR_EL3_NSPB();
    return ((reserved && ConstrainUnpredictableBool(Unpredictable_RESERVEDNSxB_Trap)) ||
             state_bits[0] == '0' || state_bits[1] != EffectiveSCR_EL3_NS() ||
             (IsFeatureImplemented(FEAT_RME) && state_bits[2] != EffectiveSCR_EL3_NSE()));
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/CheckPSBTrap

```
// CheckPSBTrap()
// ==============
// Checks whether PSB should trap to EL2.

func CheckPSBTrap()
begin
    if (IsFeatureImplemented(FEAT_FGT) && IsFeatureImplemented(FEAT_SPEv1p5) &&
          PSTATE.EL IN {EL0, EL1} && EL2Enabled() && !IsInHost() &&
          (!HaveEL(EL3) || SCR_EL3().FGTEn == '1') && HFGITR_EL2().PSBCSYNC == '1') then
        let target_el : bits(2) = EL2;
        let iss : bits(25)      = 0x3[24:0];
        AArch64_OtherInstrTrap(target_el, iss);
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/CollectContextIDR1

```
// CollectContextIDR1()
// ====================
func CollectContextIDR1() => boolean
begin
    if !StatisticalProfilingEnabled() then return FALSE; end;
    if PSTATE.EL == EL2 then return FALSE; end;
    if EL2Enabled() && HCR_EL2().TGE == '1' then return FALSE; end;
    return PMSCR_EL1().CX == '1';
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/CollectContextIDR2

```
// CollectContextIDR2()
// ====================

func CollectContextIDR2() => boolean
begin
    if !StatisticalProfilingEnabled() then return FALSE; end;
    if !EL2Enabled() then return FALSE; end;
    return PMSCR_EL2().CX == '1';
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/CollectPhysicalAddress

```
// CollectPhysicalAddress()
// ========================

func CollectPhysicalAddress() => boolean
begin
    if !StatisticalProfilingEnabled() then return FALSE; end;
    let (owning_ss, owning_el) : (SecurityState, bits(2)) = ProfilingBufferOwner();
    if HaveEL(EL2) && (owning_ss != SS_Secure || IsSecureEL2Enabled()) then
        return PMSCR_EL2().PA == '1' && (owning_el == EL2 || PMSCR_EL1().PA == '1');
    else
        return PMSCR_EL1().PA == '1';
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/CollectTimeStamp

```
// CollectTimeStamp()
// ==================
func CollectTimeStamp() => TimeStamp
begin
    if !StatisticalProfilingEnabled() then return TimeStamp_None; end;
    let (-, owning_el) = ProfilingBufferOwner();

    if owning_el == EL2 then
        if PMSCR_EL2().TS == '0' then return TimeStamp_None; end;
    else
        if PMSCR_EL1().TS == '0' then return TimeStamp_None; end;
    end;

    var PCT_el1 : bits(2);
    if !IsFeatureImplemented(FEAT_ECV) then
        PCT_el1 = '0'::PMSCR_EL1().PCT[0];                          // PCT[1] is RES0
    else
        PCT_el1 = PMSCR_EL1().PCT;
        if PCT_el1 == '10' then
            // Reserved value
            (-, PCT_el1) = ConstrainUnpredictableBits{2}(Unpredictable_PMSCR_PCT);
        end;
    end;
    if EL2Enabled() then
        var PCT_el2 : bits(2);
        if !IsFeatureImplemented(FEAT_ECV) then
            PCT_el2 = '0'::PMSCR_EL2().PCT[0];                      // PCT[1] is RES0
        else
            PCT_el2 = PMSCR_EL2().PCT;
            if PCT_el2 == '10' then
                // Reserved value
                (-, PCT_el2) = ConstrainUnpredictableBits{2}(Unpredictable_PMSCR_PCT);
            end;
        end;
        case PCT_el2 of
            when '00' =>
                return if IsInHost() then TimeStamp_Physical else TimeStamp_Virtual;
            when '01' =>
                if owning_el == EL2 then return TimeStamp_Physical; end;
            when '11' =>
                assert IsFeatureImplemented(FEAT_ECV);          // FEAT_ECV must be implemented
                if owning_el == EL1 && PCT_el1 == '00' then
                    return if IsInHost() then TimeStamp_Physical else TimeStamp_Virtual;
                else
                    return TimeStamp_OffsetPhysical;
                end;
            otherwise =>
                unreachable;
        end;
    end;

    case PCT_el1 of
        when '00' => return if IsInHost() then TimeStamp_Physical else TimeStamp_Virtual;
        when '01' => return TimeStamp_Physical;
        when '11' =>
            assert IsFeatureImplemented(FEAT_ECV);              // FEAT_ECV must be implemented
            return TimeStamp_OffsetPhysical;
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/DefaultSPEEvent

```
// DefaultSPEEvent()
// =================
// Return the target ELx for an indirect write to PMBSR_ELx for an Other buffer management
// event or anything other than a buffer management event.

func DefaultSPEEvent() => bits(2)
begin
    return ReportSPEEvent(Zeros{6}, ARBITRARY : bits(6));
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/EffectiveMDCR_EL3_NSPB

```
// EffectiveMDCR_EL3_NSPB()
// ========================
// Return the Effective value of MDCR_EL3.[NSPBE, NSPB] field and whether it is a reserved value.

readonly func EffectiveMDCR_EL3_NSPB() => (bits(3), boolean)
begin
    var state_bits : bits(3);
    var reserved : boolean = FALSE;

    if IsFeatureImplemented(FEAT_RME) then
        state_bits = MDCR_EL3().[NSPBE, NSPB];
        if state_bits == '10x' || (!IsFeatureImplemented(FEAT_Secure) && state_bits == '00x') then
            // Reserved value
            reserved = TRUE;
            (-, state_bits) = ConstrainUnpredictableBits{3}(Unpredictable_RESERVEDNSxB);
        end;
    else
        state_bits = '0' :: MDCR_EL3().NSPB;
    end;

    return (state_bits, reserved);
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/EffectivePMBLIMITR_EL1_nVM

```
// EffectivePMBLIMITR_EL1_nVM()
// ============================

readonly func EffectivePMBLIMITR_EL1_nVM() => bit
begin
    if !IsFeatureImplemented(FEAT_SPE_nVM) then
        return '0';
    end;
    if HaveEL(EL2) then
        let (owning_ss, owning_el) : (SecurityState, bits(2)) = ProfilingBufferOwner();
        if ((owning_ss != SS_Secure || IsSecureEL2Enabled()) && owning_el == EL1 &&
              PMSCR_EL2().EnVM == '0') then
            return '0';
        end;
    end;
    return PMBLIMITR_EL1().nVM;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/EffectivePMSCR_EL1_EE

```
// EffectivePMSCR_EL1_EE()
// =======================
// Return the Effective value of PMSCR_EL1.EE for the purpose of controlling the
// SPE Profiling exception.

func EffectivePMSCR_EL1_EE() => bits(2)
begin
    if EffectivePMSCR_EL2_EE() == '00' then
        return '00';
    end;

    var ee : bits(2) = PMSCR_EL1().EE;
    if ee IN {'01', '10'} then            // Reserved value
        if IsFeatureImplemented(FEAT_NV) then
            ee[0] = ee[1];
        else
            var c : Constraint;
            (c, ee) = ConstrainUnpredictableBits{2}(Unpredictable_RESPMSEE);
            assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};
            if c == Constraint_DISABLED then
                ee = '00';
            end;
            // Otherwise the value returned by ConstrainUnpredictableBits must be
            // a non-reserved value
        end;
    end;

    return ee;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/EffectivePMSCR_EL2_EE

```
// EffectivePMSCR_EL2_EE()
// =======================
// Return the Effective value of PMSCR_EL2.EE.

readonly func EffectivePMSCR_EL2_EE() => bits(2)
begin
    if !IsFeatureImplemented(FEAT_SPE_EXC) then
        return '00';
    end;

    if HaveEL(EL3) && MDCR_EL3().PMSEE == '00' then
        return '00';
    end;

    let check_el2 : boolean = (HaveEL(EL2) &&
                               (EffectiveSCR_EL3_NS() == '1' || IsSecureEL2Enabled()));
    return if check_el2 then PMSCR_EL2().EE else '01';
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/GetPMBSR_EL1_FSC

```
// GetPMBSR_EL1_FSC()
// ==================
// Query the PMBSR_EL1.FSC field.

func GetPMBSR_EL1_FSC() => bits(6)
begin
    var FSC : bits(6);

    FSC = PMBSR_EL1()[5:0];
    return FSC;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/GetPMBSR_EL2_FSC

```
// GetPMBSR_EL2_FSC()
// ==================
// Query the PMBSR_EL2.FSC field.

func GetPMBSR_EL2_FSC() => bits(6)
begin
    var FSC : bits(6);

    FSC = PMBSR_EL2()[5:0];
    return FSC;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/GetPMBSR_EL3_FSC

```
// GetPMBSR_EL3_FSC()
// ==================
// Query the PMBSR_EL3.FSC field.

func GetPMBSR_EL3_FSC() => bits(6)
begin
    var FSC : bits(6);

    FSC = PMBSR_EL3()[5:0];
    return FSC;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/OtherSPEManagementEvent

```
// OtherSPEManagementEvent()
// =========================
// Report an Other buffer management event, with the status code 'bsc'

func OtherSPEManagementEvent(bsc : bits(6))
begin
    let target_el : bits(2) = DefaultSPEEvent();
    if PMBSR_EL(target_el).S == '0' then
        PMBSR_EL(target_el).S   = '1';       // Assert interrupt or exception
        PMBSR_EL(target_el).EC  = '000000';  // Other buffer management event
        PMBSR_EL(target_el).MSS = ZeroExtend{16}(bsc);
        PMBSR_EL(target_el).MSS2 = Zeros{24};
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/PMBSR_EL

```
// PMBSR_EL - accessor
// ==================

accessor PMBSR_EL(el : bits(2)) <=> value : PMBSRType
begin
    getter
        var r : bits(64);
        case el of
            when EL1  => r = PMBSR_EL1();
            when EL2  => r = PMBSR_EL2();
            when EL3  => r = PMBSR_EL3();
            otherwise => unreachable;
        end;
        return r;
    end;
    setter
        let r : bits(64) = value;
        case el of
            when EL1  => PMBSR_EL1() = r;
            when EL2  => PMBSR_EL2() = r;
            when EL3  => PMBSR_EL3() = r;
            otherwise => unreachable;
        end;
        return;
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/ProfilingBufferEnabled

```
// ProfilingBufferEnabled()
// ========================

readonly func ProfilingBufferEnabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_SPE) then
        return FALSE;
    end;

    let (-, owning_el) : (SecurityState, bits(2)) = ProfilingBufferOwner();

    return !ELUsingAArch32(owning_el) && PMBLIMITR_EL1().E == '1';
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/ProfilingBufferOwner

```
// ProfilingBufferOwner()
// ======================

readonly func ProfilingBufferOwner() => (SecurityState, bits(2))
begin
    var owning_ss : SecurityState;

    var state_bits : bits(3);
    if HaveEL(EL3) then
        (state_bits, -) = EffectiveMDCR_EL3_NSPB();
    else
        state_bits = if SecureOnlyImplementation() then '001' else '011';
    end;

    case state_bits of
        when '00x' => owning_ss = SS_Secure;
        when '01x' => owning_ss = SS_NonSecure;
        when '11x' => owning_ss = SS_Realm;
    end;

    var owning_el : bits(2);
    if HaveEL(EL2) && (owning_ss != SS_Secure || IsSecureEL2Enabled()) then
        owning_el = if MDCR_EL2().E2PB == '00' then EL2 else EL1;
    else
        owning_el = EL1;
    end;

    return (owning_ss, owning_el);
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/ProfilingSynchronizationBarrier

```
// ProfilingSynchronizationBarrier()
// =================================
// Barrier to ensure that all existing profiling data has been formatted, and profiling buffer
// addresses have been translated such that writes to the profiling buffer have been initiated.
// A following DSB completes when writes to the profiling buffer have completed.

impdef func ProfilingSynchronizationBarrier()
begin
    return;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/ReportSPEEvent

```
// ReportSPEEvent()
// ================
// Return the target ELx for an indirect write to PMBSR_ELx.
// When the indirect write is due to a buffer management event:
// 'ec_bits' is the Event Class for the management event.
// 'fsc_bits' is the Fault Status Code when this is a fault, ignored otherwise.
// Otherwise, 'ec_bits' should be Zeros().

func ReportSPEEvent(ec_bits : bits(6), fsc_bits : bits(6)) => bits(2)
begin
    var target_el : bits(2);
    var route_to_el3 : boolean = FALSE;
    var route_to_el2 : boolean = FALSE;

    if IsFeatureImplemented(FEAT_SPE_EXC) then
        if ec_bits == '011111' then                     // implementation defined fault
            return ImpDefBits{2}("target_el for SPE ImpDef Fault");
        end;

        let s1fault : boolean = (ec_bits == '100100');   // Stage 1 fault
        let s2fault : boolean = (ec_bits == '100101');   // Stage 2 fault

        var gpcfault, gpfault : boolean;
        if IsFeatureImplemented(FEAT_RME) then
            // Granule Protection Check fault, other than GPF. That is, a GPT address size fault,
            // GPT walk fault, or synchronous External abort on GPT fetch.
            gpcfault = (ec_bits == '011110');
            // Other Granule Protection Fault, reported as Stage 1 or Stage 2 fault.
            gpfault  =  ((s1fault || s2fault) && fsc_bits IN {'10001x', '1001xx', '101000'});
        else
            gpcfault = FALSE;
            gpfault  = FALSE;
        end;
        let sync_ext_abort : boolean = ((s1fault || s2fault) &&
                                        fsc_bits IN {'010000', '01001x', '0101xx', '011011'});

        var owning_ss : SecurityState;
        var owning_el : bits(2);
        (owning_ss, owning_el) = ProfilingBufferOwner();

        if HaveEL(EL3) && MDCR_EL3().PMSEE == '1x' then
            route_to_el3 = (MDCR_EL3().PMSEE == '11' ||
                            gpcfault || (gpfault && SCR_EL3().GPF == '1') ||
                            (sync_ext_abort && EffectiveEA() == '1'));
        end;

        if EffectivePMSCR_EL2_EE() == '1x' then
            route_to_el2 = (PMSCR_EL2().EE == '11' || (s1fault && owning_el == EL2) || s2fault ||
                            gpcfault || (gpfault && HCR_EL2().GPF == '1') ||
                            (sync_ext_abort && EffectiveHCR_TEA() == '1'));
        end;
    end;

    if route_to_el3 then
        target_el = EL3;
    elsif route_to_el2 then
        target_el = EL2;
    else
        target_el = EL1;
    end;

    return target_el;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPE

```
// SPE Implementation Constants
// ============================

constant SPEMaxAddrs : integer{}      = 32;
constant SPEMaxCounters : integer{}   = 32;
constant SPEMaxRecordSize : integer{} = 2048;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEAddByteToRecord

```
// SPEAddByteToRecord()
// ====================
// Add one byte to a record and increase size property appropriately.

func SPEAddByteToRecord(b : bits(8))
begin
    assert SPERecordSize < SPEMaxRecordSize;
    SPERecordData[[SPERecordSize]] = b;
    SPERecordSize = SPERecordSize + 1;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEAddPacketToRecord

```
// SPEAddPacketToRecord()
// ======================
// Add passed header and payload data to the record.
// Payload must be a multiple of 8.

func SPEAddPacketToRecord{N}(header_hi : bits(2), header_lo : bits(4),
                             payload : bits(N))
begin
    assert N MOD 8 == 0;
    var sz : bits(2);
    case N of
        when 8  => sz = '00';
        when 16 => sz = '01';
        when 32 => sz = '10';
        when 64 => sz = '11';
        otherwise => unreachable;
    end;

    let header : bits(8) = header_hi::sz::header_lo;
    SPEAddByteToRecord(header);
    for i = 0 to (N DIV 8)-1 do
        SPEAddByteToRecord(payload[i*:8]);
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEAddressEnabled

```
// SPEAddressEnabled()
// ===================
// Checks if an address packet is enabled and its data can be recorded.

func SPEAddressEnabled(address_index : integer) => boolean
begin
    var address_packet_enabled: boolean;
    let have_pbtfilt : boolean = IsFeatureImplemented(FEAT_SPE_PBTFILT);

    case address_index of
        when SPEAddrPosPrevBranchTarget =>
            address_packet_enabled = !have_pbtfilt || PMSFCR_EL1().nPBT == '0';
        otherwise =>
            address_packet_enabled = TRUE;
    end;

    return address_packet_enabled;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEBranch

```
// SPEBranch()
// ===========
// Called on every branch if SPE is present. Maintains previous branch target
// and branch related SPE functionality.

func SPEBranch{N}(target : bits(N), branch_type : BranchType, conditional : boolean,
                  taken_flag : boolean)
begin
    let is_isb : boolean = FALSE;
    SPEBranch{N}(target, branch_type, conditional, taken_flag, is_isb);
end;

func SPEBranch{N}(target : bits(N), branch_type : BranchType, conditional : boolean,
                  taken_flag : boolean, is_isb : boolean)
begin
    // If the PE implements branch prediction, data about (mis)prediction is collected
    // through the PMU events.

    var collect_prev_br : boolean;
    let collect_prev_br_eret : boolean = ImpDefBool("SPE prev br on eret");
    let collect_prev_br_exception : boolean = (ImpDefBool(
                                                "SPE prev br on exception"));
    let collect_prev_br_isb : boolean = ImpDefBool("SPE prev br on isb");
    case branch_type of
        when BranchType_EXCEPTION => collect_prev_br = collect_prev_br_exception;
        when BranchType_ERET      => collect_prev_br = collect_prev_br_eret;
        otherwise                 => collect_prev_br = !is_isb || collect_prev_br_isb;
    end;

    // Implements previous branch target functionality
    if (taken_flag && IsFeatureImplemented(FEAT_SPE_PBT) && StatisticalProfilingEnabled() &&
            collect_prev_br) then

        if SPESampleInFlight then
            let previous_target : bits(64) = SPESamplePreviousBranchAddress;
            SPESampleAddress[[SPEAddrPosPrevBranchTarget]][63:0] = previous_target[63:0];
            let previous_branch_valid : boolean = SPESamplePreviousBranchAddressValid;
            SPESampleAddressValid[[SPEAddrPosPrevBranchTarget]] = previous_branch_valid;
        end;
        // Save the target address for it to be added to a future record.
        SPESamplePreviousBranchAddress[55:0] = target[55:0];

        var ns : bit;
        var nse : bit;
        case CurrentSecurityState() of
            when SS_Secure =>
                ns = '0';
                nse = '0';
            when SS_NonSecure =>
                ns = '1';
                nse = '0';
            when SS_Realm =>
                ns  = '1';
                nse = '1';
            otherwise => unreachable;
        end;

        SPESamplePreviousBranchAddress[63] = ns;
        SPESamplePreviousBranchAddress[60] = nse;
        SPESamplePreviousBranchAddress[62:61] = PSTATE.EL;
        SPESamplePreviousBranchAddressValid   = TRUE;
    end;

    if !StatisticalProfilingEnabled() then
        if taken_flag then
            // Invalidate previous branch address, if profiling is disabled
            // or prohibited.
            SPESamplePreviousBranchAddressValid = FALSE;
        end;
        return;
    end;

    if SPESampleInFlight then
        SPESampleOpAttr.branch_is_direct = branch_type IN {BranchType_DIR, BranchType_DIRCALL};
        SPESampleOpAttr.branch_has_link  = branch_type IN {BranchType_DIRCALL, BranchType_INDCALL};
        SPESampleOpAttr.procedure_return = branch_type == BranchType_RET;
        SPESampleOpAttr.op_type          = SPEOpType_Branch;
        SPESampleOpAttr.is_conditional   = conditional;
        SPESampleOpAttr.cond_pass        = taken_flag;

        // Save the target address.
        if taken_flag then
            var ns : bit;
            var nse : bit;
            case CurrentSecurityState() of
                when SS_Secure =>
                    ns = '0';
                    nse = '0';
                when SS_NonSecure =>
```

```
                    ns = '1';
                    nse = '0';
                when SS_Realm =>
                    ns  = '1';
                    nse = '1';
                otherwise => unreachable;
            end;
            let el : bits(2) = PSTATE.EL;

            SPESampleAddress[[SPEAddrPosBranchTarget]][55:0]  = target[55:0];
            SPESampleAddress[[SPEAddrPosBranchTarget]][63:56] = ns::el::nse::Zeros{4};
            SPESampleAddressValid[[SPEAddrPosBranchTarget]]   = TRUE;
        end;
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEBufferIsFull

```
// SPEBufferIsFull()
// =================
// Return true if another full size sample record would not fit in the
// profiling buffer.

func SPEBufferIsFull() => boolean
begin
    let write_pointer_limit : integer   = UInt(PMBLIMITR_EL1().LIMIT::Zeros{12});
    let current_write_pointer : integer = UInt(PMBPTR_EL1());
    let record_max_size : integer       = 1 << UInt(PMSIDR_EL1().MaxSize);
    return current_write_pointer > (write_pointer_limit - record_max_size);
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPECollectRecord

```
// SPECollectRecord()
// ==================
// Returns TRUE if the sampled class of instructions or operations, as
// determined by PMSFCR_EL1, are recorded and FALSE otherwise.

func SPECollectRecord(events : bits(64), total_latency : integer, optype : SPEOpType) => boolean
begin
    // Filtering by events
    var is_rejected_event : boolean;
    var is_rejected_nevent : boolean;
    (is_rejected_event, is_rejected_nevent) = SPEFilterByEvents(events);

    // Filtering by type
    let is_rejected_type : boolean = SPEFilterByType(SPESampleOpAttr);

    // Filtering by latency
    let is_rejected_latency : boolean = SPEFilterByLatency(total_latency);

    let is_rejected_data_source : boolean = SPEFilterByDataSource(optype);

    var return_value : boolean;
    return_value = !(is_rejected_nevent || is_rejected_event ||
                     is_rejected_type || is_rejected_latency ||
                     is_rejected_data_source);

    if return_value then
        PMUEvent(PMU_EVENT_SAMPLE_FILTRATE);
    end;

    return return_value;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPECompleteSample

```
// SPECompleteSample()
// ===================
// Called to complete the sampling process.

func SPECompleteSample()
begin
    assert SPESampleInFlight;

    PMUEvent(PMU_EVENT_SAMPLE_FEED);

    // Stop any pending counters
    for counter_index = 0 to (SPEMaxCounters - 1) do
        if SPESampleCounterPending[[counter_index]] then
            SPEStopCounter(counter_index);
        end;
    end;

    // Record any IMPLEMENTATION DEFINED events
    let impdef_events : bits(64) = ImpDefBits{}("SPE EVENTS");
    SPESampleEvents[63:48] = impdef_events[63:48];
    // The number of bits available for IMPLEMENTATION DEFINED events
    // is reduced by FEAT_SPEv1p4
    if !IsFeatureImplemented(FEAT_SPEv1p4) then
        SPESampleEvents[31:24] = impdef_events[31:24];
    end;
    SPESampleEvents[15:12] = impdef_events[15:12];
    // Bit 24 encodes whether the sample was collected in Streaming SVE mode.
    if IsFeatureImplemented(FEAT_SPE_SME) then
        SPESampleEvents[24] = PSTATE.SM;
    end;
    SPESampleEvents[6] = if (SPESampleOpAttr.is_conditional &&
                             !SPESampleOpAttr.cond_pass) then '1' else '0';

    var collect_record : boolean;
    collect_record = SPECollectRecord(SPESampleEvents,
                                      SPESampleCounter[[SPECounterPosTotalLatency]],
                                      SPESampleOpAttr.op_type);
    var discard : boolean = FALSE;
    if IsFeatureImplemented(FEAT_SPEv1p2) then
        discard = PMBLIMITR_EL1().FM == '10';
    end;
    if collect_record && !discard then
        SPEConstructRecord();
        if SPEBufferIsFull() then
            let bsc : bits(6) = '000001';     // Buffer full event
            OtherSPEManagementEvent(bsc);
            PMUEvent(PMU_EVENT_SAMPLE_BUFFER_FULL);
        end;
    end;

    SPESampleInFlight = FALSE;

    SPEResetSampleStorage();
end;

// Order of addresses in sample storage and architectural index values
constant SPEAddrPosPCVirtual : integer        = 0;
constant SPEAddrPosBranchTarget : integer     = 1;
constant SPEAddrPosDataVirtual : integer      = 2;
constant SPEAddrPosDataPhysical : integer     = 3;
constant SPEAddrPosPrevBranchTarget : integer = 4;
// Order of counters in sample storage and architectural index values
constant SPECounterPosTotalLatency : integer       = 0;
constant SPECounterPosIssueLatency : integer       = 1;
constant SPECounterPosTranslationLatency : integer = 2;
constant SPECounterPosAltIssueLatency : integer    = 4;
// Sample in-flight flag
var SPESampleInFlight : boolean = FALSE;

// Globally declared variables which store data about the sample.

// Context storage
var SPESampleContextEL1 : bits(32);
var SPESampleContextEL1Valid : boolean;
var SPESampleContextEL2 : bits(32);
var SPESampleContextEL2Valid : boolean;

// Counter storage
var SPESampleCounter : array [[SPEMaxCounters]] of integer;
var SPESampleCounterValid : array [[SPEMaxCounters]] of boolean;
var SPESampleCounterPending : array [[SPEMaxCounters]] of boolean;

// Address storage
var SPESampleAddress : array [[SPEMaxAddrs]] of bits(64);
var SPESampleAddressValid : array [[SPEMaxAddrs]] of boolean;
var SPESamplePreviousBranchAddress : bits(64);
var SPESamplePreviousBranchAddressValid : boolean;

// Data source storage
```

```
var SPESampleDataSource : bits(16);
var SPESampleDataSourceValid : boolean;

// OpAttr storage
var SPESampleOpAttr : SPEOpAttr;

// Timestamp storage
var SPESampleTimestamp : bits(64);
var SPESampleTimestampValid : boolean;

// Event storage
var SPESampleEvents : bits(64);
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEConstructClass

```
// SPEConstructClass()
// ===================
// Constructs the encodings for the class and subclass fields.

func SPEConstructClass() => (bits(2), bits(8))
begin
    var op_class : bits(2);
    var op_subclass : bits(8);
    let cond : bit     = if SPESampleOpAttr.is_conditional then '1' else '0';
    let fp : bit       = if SPESampleOpAttr.is_floating_point then '1' else '0';
    let ldst : bit     = if SPESampleOpAttr.op_type == SPEOpType_Store then '1' else '0';
    let ar : bit       = if SPESampleOpAttr.is_acquire_release then '1' else '0';
    let excl : bit     = if SPESampleOpAttr.is_exclusive then '1' else '0';
    let at : bit       = if SPESampleOpAttr.at then '1' else '0';
    let indirect : bit = if SPESampleOpAttr.branch_is_direct then '0' else '1';
    let pred : bit = if SPESampleOpAttr.is_predicated then '1' else '0';
    let sg : bit   = if SPESampleOpAttr.is_gather_scatter then '1' else '0';
    let evl : bits(3) = SPESampleOpAttr.evl;
    let simd : bit = if SPESampleOpAttr.is_simd then '1' else '0';
    let ets : bits(4) = SPESampleOpAttr.ets;
    // Since this implementation of SPE samples instruction instead of micro-operations, a
    // Branch with link or Procedure return instruction will never be recorded as a "Load/store,
    // GCS" format Operation Type packet. Therefore the COMMON bit is hard-wired to '1'.
    let common : bit = '1';
    if SPESampleOpAttr.op_type == SPEOpType_Other then
        op_class = '00';
        op_subclass = Zeros{5}::simd::fp::cond;
    elsif SPESampleOpAttr.op_type == SPEOpType_OtherSVE then
        op_class = '00';
        op_subclass = '0'::evl::'1'::pred::fp::'0';
    elsif SPESampleOpAttr.op_type == SPEOpType_OtherSME then
        op_class = '00';
        op_subclass = '1'::ets[3:1]::'1'::ets[0]::fp::'0';
    elsif SPESampleOpAttr.op_type == SPEOpType_Branch then
        op_class = '10';
        var cr : bits(2) = '00';
        var gcs : bit = '0';
        if IsFeatureImplemented(FEAT_SPE_CRR) then
            if SPESampleOpAttr.branch_has_link then
                cr = '01';
            elsif SPESampleOpAttr.procedure_return then
                cr = '10';
            else
                cr = '11';
            end;
        end;
        if IsFeatureImplemented(FEAT_GCS) then
            if (SPESampleOpAttr.ldst_type == SPELDSTType_GCS &&
                (SPESampleOpAttr.branch_has_link || SPESampleOpAttr.procedure_return)) then
                gcs = '1';
            end;
        end;
        op_subclass = Zeros{3}::cr::gcs::indirect::cond;
    elsif SPESampleOpAttr.op_type IN {SPEOpType_Load, SPEOpType_Store, SPEOpType_LoadAtomic} then
        op_class = '01';
        case SPESampleOpAttr.ldst_type of
            when SPELDSTType_NV2         => op_subclass = '0011000'::ldst;
            when SPELDSTType_Extended    => op_subclass = '000'::ar::excl::at::'1'::ldst;
            when SPELDSTType_General     => op_subclass = Zeros{7}::ldst;
            when SPELDSTType_SIMDFP      => op_subclass = '0000010'::ldst;
            when SPELDSTType_SVESME      => op_subclass = sg::evl::'1'::pred::'0'::ldst;
            when SPELDSTType_Unspecified => op_subclass = '0001000'::ldst;
            when SPELDSTType_Tags        => op_subclass = '0001010'::ldst;
            when SPELDSTType_MemCopy     => op_subclass = '0010000'::ldst;
            when SPELDSTType_MemSet      => op_subclass = '00100101';
            when SPELDSTType_GCS         => op_subclass = '01000'::common::'0'::ldst;
            when SPELDSTType_GCSSS2      => unreachable;
                // GCSSS2 is converted to GCS, should not appear here
            otherwise                    => unreachable;
        end;
    else
        unreachable;
    end;
    return (op_class, op_subclass);
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEConstructRecord

```
// SPEConstructRecord()
// ====================
// Create new record and populate it with packets using sample storage data.
// This is an example implementation, packets may appear in
// any order as long as the record ends with an End or Timestamp packet.

func SPEConstructRecord()
begin
    // Empty the record.
    SPEEmptyRecord();

    var op_class : bits(2);
    var op_subclass : bits(8);
    // Add operation details
    (op_class, op_subclass) = SPEConstructClass();
    SPEAddPacketToRecord{8}('01', '10'::op_class, op_subclass);

    // Add contextEL1 if available
    if SPESampleContextEL1Valid then
        SPEAddPacketToRecord{32}('01', '0100', SPESampleContextEL1);
    end;

     // Add contextEL2 if available
    if SPESampleContextEL2Valid then
        SPEAddPacketToRecord{32}('01', '0101', SPESampleContextEL2);
    end;

    // Add valid counters
    for counter_index = 0 to (SPEMaxCounters - 1) do
        if SPESampleCounterValid[[counter_index]] then
            if counter_index >= 8 then
                // Need extended format
                SPEAddByteToRecord('001000'::counter_index[4:3]);
            end;
            // Check for overflow
            let large_counters : boolean = ImpDefBool("SPE 16bit counters");
            if large_counters then
                if SPESampleCounter[[counter_index]] > 0xFFFF then
                    SPESampleCounter[[counter_index]] = 0xFFFF;
                end;
            else
                if SPESampleCounter[[counter_index]] > 0xFFF then
                    SPESampleCounter[[counter_index]] = 0xFFF;
                end;
            end;

            // Add byte0 for short format (byte1 for extended format)
            SPEAddPacketToRecord{16}('10', '1'::counter_index[2:0],
                                     SPESampleCounter[[counter_index]][15:0]);
        end;
    end;

    // Add valid addresses
    if IsFeatureImplemented(FEAT_SPE_PBT) then
        // Under some conditions, it is IMPLEMENTATION_DEFINED whether the
        // previous branch packet is present.
        let include_prev_br : boolean = ImpDefBool("SPE get prev br if not br");
        if SPESampleOpAttr.op_type != SPEOpType_Branch && !include_prev_br then
            SPESampleAddressValid[[SPEAddrPosPrevBranchTarget]] = FALSE;
        end;
    end;

    // Data Virtual address should not be collected if this was an NV2 access and Statistical
    // Profiling is disabled at EL2.
    if !StatisticalProfilingEnabled(EL2) && SPESampleOpAttr.ldst_type == SPELDSTType_NV2 then
        SPESampleAddressValid[[SPEAddrPosDataVirtual]] = FALSE;
    end;

    for address_index = 0 to (SPEMaxAddrs - 1) do
        if SPESampleAddressValid[[address_index]] && SPEAddressEnabled(address_index) then
            if address_index >= 8 then
                // Need extended format
                SPEAddByteToRecord('001000'::address_index[4:3]);
            end;
            SPEAddPacketToRecord{64}('10', '0'::address_index[2:0],
                                     SPESampleAddress[[address_index]]);
        end;
    end;

    // Add Data Source
    if SPESampleDataSourceValid then
        let ds_payload_size : integer{} = SPEGetDataSourcePayloadSize();
        SPEAddPacketToRecord{8 * ds_payload_size}('01', '0011',
                                                  SPESampleDataSource[8*ds_payload_size-1:0]);
    end;

    // Add events
    // Get size of payload in bytes.
    let payload_size : integer{} = SPEGetEventsPayloadSize();
```

```
    SPEAddPacketToRecord{8 * payload_size}('01', '0010', SPESampleEvents[8*payload_size-1:0]);

    // Add Timestamp to end the record if one is available.
    // Otherwise end with an End packet.
    if SPESampleTimestampValid then
        SPEAddPacketToRecord{64}('01', '0001', SPESampleTimestamp);
    else
        SPEAddByteToRecord('00000001');
    end;

    // Add padding
    while SPERecordSize MOD (1<<UInt(PMBIDR_EL1().Align)) != 0 looplimit 2048 do
        SPEAddByteToRecord(Zeros{8});
    end;
    SPEWriteToBuffer();
    CTI_SignalEvent(CrossTriggerIn_SPESample);
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPECycle

```
// SPECycle()
// ==========
// Function called at the end of every cycle. Responsible for asserting interrupts
// and advancing counters.

func SPECycle()
begin
    if !IsFeatureImplemented(FEAT_SPE) then
        return;
    end;

    // Increment pending counters
    if SPESampleInFlight then
        for i = 0 to (SPEMaxCounters - 1) do
            if SPESampleCounterPending[[i]] then
                SPESampleCounter[[i]] = SPESampleCounter[[i]] + 1;
            end;
        end;
    end;

    // Assert PMBIRQ if appropriate.
    if SPEInterruptEnabled() && PMBSR_EL1().S == '1' then
        SetInterruptRequestLevel(InterruptID_PMBIRQ, HIGH);
    else
        SetInterruptRequestLevel(InterruptID_PMBIRQ, LOW);
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEEmptyRecord

```
// SPEEmptyRecord()
// ================
// Reset record data.

func SPEEmptyRecord()
begin
    SPERecordSize = 0;
    for i = 0 to (SPEMaxRecordSize - 1) do
        SPERecordData[[i]] = Zeros{8};
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEEncodeETS

```
// SPEEncodeETS()
// ==============
// Encodes an integer tile size length into the ets field for the SPE operation type packet.

func SPEEncodeETS(size : integer) => bits(4)
begin
    var ets : bits(4);
    if size <= 128 then
        ets = '0000';
    elsif size <= 256 then
        ets = '0001';
    elsif size <= 512 then
        ets = '0010';
    elsif size <= 1024 then
        ets = '0011';
    elsif size <= 2048 then
        ets = '0100';
    elsif size <= 4096 then
        ets = '0101';
    elsif size <= 8192 then
        ets = '0110';
    elsif size <= 16384 then
        ets = '0111';
    elsif size <= 32768 then
        ets = '1000';
    elsif size <= 65536 then
        ets = '1001';
    elsif size <= 131072 then
        ets = '1010';
    elsif size <= 262144 then
        ets = '1011';
    else
        ets = '1111';
    end;
    return ets;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEEncodeEVL

```
// SPEEncodeEVL()
// ==============
// Encodes an integer vector length into the evl field for the SPE operation type packet.

func SPEEncodeEVL(vl : integer) => bits(3)
begin
    var evl : bits(3);
    if vl <= 32 then
        evl = '000';
    elsif vl <= 64 then
        evl = '001';
    elsif vl <= 128 then
        evl = '010';
    elsif vl <= 256 then
        evl = '011';
    elsif vl <= 512 then
        evl = '100';
    elsif vl <= 1024 then
        evl = '101';
    elsif vl <= 2048 then
        evl = '110';
    else
        if IsFeatureImplemented(FEAT_SPE_SME) then
            evl = '111';
        else
            unreachable;
        end;
    end;
    return evl;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEEvent

```
// SPEEvent()
// ==========
// Called when a PMU event is generated by the sampled structure.
// Sets appropriate bit in SPESampleStorage.events.

func SPEEvent(pmuevent : bits(16))
begin
    if SPESampleInFlight then
        case pmuevent of
            when PMU_EVENT_DSNP_HIT_RD =>
                if IsFeatureImplemented(FEAT_SPEv1p4) then
                    SPESampleEvents[23] = '1';                    // Snoop hit
                end;
            when {PMU_EVENT_L1D_LFB_HIT_RD, PMU_EVENT_L2D_LFB_HIT_RD,
                  PMU_EVENT_L3D_LFB_HIT_RD, PMU_EVENT_LL_LFB_HIT_RD} =>
                if IsFeatureImplemented(FEAT_SPEv1p4) then
                    SPESampleEvents[22] = '1';                    // Recent fetch
                end;
            when {PMU_EVENT_L1D_CACHE_HITM_RD, PMU_EVENT_L2D_CACHE_HITM_RD,
                  PMU_EVENT_L3D_CACHE_HITM_RD, PMU_EVENT_LL_CACHE_HITM_RD} =>
                if IsFeatureImplemented(FEAT_SPEv1p4) then
                    SPESampleEvents[21] = '1';                    // Modified
                end;
            when PMU_EVENT_L2D_CACHE_LMISS_RD =>
                if IsFeatureImplemented(FEAT_SPEv1p4) then
                    SPESampleEvents[20] = '1';                    // Level 2 cache miss
                end;
            when PMU_EVENT_L2D_CACHE_RD =>
                if IsFeatureImplemented(FEAT_SPEv1p4) then
                    SPESampleEvents[19] = '1';                    // Level 2 cache access
                end;
            when PMU_EVENT_SVE_PRED_EMPTY_SPEC =>
                if IsFeatureImplemented(FEAT_SPEv1p1) then
                    SPESampleEvents[18] = '1';                    // Empty predicate
                end;
            when PMU_EVENT_SVE_PRED_NOT_FULL_SPEC =>
                if IsFeatureImplemented(FEAT_SPEv1p1) then
                    SPESampleEvents[17] = '1';                    // Partial predicate
                end;
            when PMU_EVENT_LDST_ALIGN_LAT =>
                if IsFeatureImplemented(FEAT_SPEv1p1) then
                    SPESampleEvents[11] = '1';                    // Misaligned
                end;
            when PMU_EVENT_REMOTE_ACCESS =>
                SPESampleEvents[10] = '1';                        // Remote access
            when PMU_EVENT_LL_CACHE_MISS =>
                SPESampleEvents[9]  = '1';                        // Last level cache miss
            when PMU_EVENT_LL_CACHE =>
                SPESampleEvents[8]  = '1';                        // Last level cache access
            when PMU_EVENT_BR_MIS_PRED =>
                SPESampleEvents[7]  = '1';                        // Mispredicted
            when PMU_EVENT_BR_MIS_PRED_RETIRED =>
                SPESampleEvents[7]  = '1';                        // Not taken
            when PMU_EVENT_DTLB_WALK =>
                SPESampleEvents[5]  = '1';                        // TLB walk
            when PMU_EVENT_L1D_TLB =>
                SPESampleEvents[4]  = '1';                        // TLB access
            when PMU_EVENT_L1D_CACHE_REFILL =>
                if !IsFeatureImplemented(FEAT_SPEv1p4) then
                    SPESampleEvents[3] = '1';                     // Level 1 cache refill
                end;
            when PMU_EVENT_L1D_CACHE_LMISS_RD =>
                if IsFeatureImplemented(FEAT_SPEv1p4) then
                    SPESampleEvents[3] = '1';                     // Level 1 cache miss
                end;
            when PMU_EVENT_L1D_CACHE =>
                SPESampleEvents[2]  = '1';                        // Level 1 cache access
            when PMU_EVENT_INST_RETIRED =>
                SPESampleEvents[1]  = '1';                        // Retired
            when PMU_EVENT_EXC_TAKEN =>
                SPESampleEvents[0]  = '1';                        // Exception
            otherwise =>
                return;
        end;
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEFilterByDataSource

```
// SPEFilterByDataSource()
// =======================
// Carry out filtering by data source.

func SPEFilterByDataSource(optype : SPEOpType) => boolean
begin
    // Filtering by Data Source
    var is_rejected_data_source : boolean = FALSE;
    if (IsFeatureImplemented(FEAT_SPE_FDS) && SPESampleDataSourceValid &&
         (optype IN {SPEOpType_Load, SPEOpType_LoadAtomic})) then
        let data_source : bits(16) = SPESampleDataSource;
        let index : integer        = UInt(data_source[5:0]);
        let is_ds : boolean        = PMSDSFR_EL1()[index] == '1';
        if is_ds then PMUEvent(PMU_EVENT_SAMPLE_FEED_DS); end;
        if PMSFCR_EL1().FDS == '1' then
            // Filtering by Data Source is enabled
            is_rejected_data_source = !is_ds;
        end;
    end;
    return is_rejected_data_source;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEFilterByEvents

```
// SPEFilterByEvents()
// ===================
// Carries out filtering by the event bits.

func SPEFilterByEvents(events : bits(64)) => (boolean, boolean)
begin
    // "mask" defines which Events packet bits are checked by the filter
    var mask : bits(64) = Zeros{};
    let impdef_mask : bits(64) = ImpDefBits{}("SPE mask");

    mask[63:48] = impdef_mask[63:48];
    if IsFeatureImplemented(FEAT_SPE_SME) && IsFeatureImplemented(FEAT_SPEv1p4) then
        mask[25:24] = '11';                 // Streaming mode, SMCU
    end;
    if IsFeatureImplemented(FEAT_SPEv1p4) then
        mask[23:19] = '11111';              // Snoop hit, Recent fetch, Modified,
                                            // Level 2 cache miss, Level 2 cache access
    else
        mask[31:24] = impdef_mask[31:24];
    end;
    if IsFeatureImplemented(FEAT_SPEv1p1) && IsFeatureImplemented(FEAT_SVE) then
        mask[18:17] = '11';                 // Predicates
    end;
    mask[15:12] = impdef_mask[15:12];
    if IsFeatureImplemented(FEAT_SPEv1p1) then
        mask[11] = '1';                     // Data alignment
    end;
    if IsFeatureImplemented(FEAT_SPEv1p4) then
        mask[10:8] = '111';                 // Remote access, Last level cache access,
                                            // Last level cache miss
    else
        mask[10:8] = impdef_mask[10:8];
    end;
    mask[7] = '1';                          // Mispredicted
    if IsFeatureImplemented(FEAT_SPE_FnE) then
        mask[6] = '1';                      // Not taken
    end;
    mask[5,3,1] = '111';                    // TLB walk, Level 1 cache miss, Retired
    if IsFeatureImplemented(FEAT_SPEv1p4) then
        mask[4,2] = '11';                   // TLB access, Level 1 cache access
    else
        mask[4,2] = impdef_mask[4,2];
    end;

    let e : bits(64) = events AND mask;

    // Filtering by event
    let evfr : bits(64) = PMSEVFR_EL1() AND mask;
    var is_rejected_event : boolean = FALSE;
    let is_evt : boolean = IsZero(NOT(e) AND evfr);
    if PMSFCR_EL1().FE == '1' then
        // Filtering by event is enabled
        if !IsZero(evfr) then
            // Not a CONSTRAINED UNPREDICTABLE case
            is_rejected_event = !is_evt;
        else
            is_rejected_event = ConstrainUnpredictableBool(Unpredictable_BADPMSFCR);
        end;
    end;

    // Filtering by inverse event
    var is_rejected_nevent : boolean = FALSE;
    var is_nevt : boolean;
    if IsFeatureImplemented(FEAT_SPE_FnE) then
        let nevfr : bits(64) = PMSNEVFR_EL1() AND mask;
        is_nevt = IsZero(e AND nevfr);
        if PMSFCR_EL1().FnE == '1' then
            // Inverse filtering by event is enabled
            if !IsZero(nevfr) then
                // Not a CONSTRAINED UNPREDICTABLE case
                is_rejected_nevent = !is_nevt;
            else
                is_rejected_nevent = ConstrainUnpredictableBool(Unpredictable_BADPMSFCR);
            end;
        end;
    else
        is_nevt = TRUE; // not implemented
    end;
    if is_evt && is_nevt then
        PMUEvent(PMU_EVENT_SAMPLE_FEED_EVENT);
    end;
    if (IsFeatureImplemented(FEAT_SPE_FnE) && PMSFCR_EL1().[FnE,FE]== '11' &&
          !IsZero(PMSEVFR_EL1() AND PMSNEVFR_EL1() AND mask)) then
        // CONSTRAINED UNPREDICTABLE case due to combination of filter and inverse filter
        is_rejected_nevent = ConstrainUnpredictableBool(Unpredictable_BADPMSFCR);
        is_rejected_event = ConstrainUnpredictableBool(Unpredictable_BADPMSFCR);
    end;
```

```
    return (is_rejected_event, is_rejected_nevent);
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEFilterByLatency

```
// SPEFilterByLatency()
// ====================
// Carries out filtering by latency.

func SPEFilterByLatency(total_latency : integer) => boolean
begin

    let is_lat : boolean = (total_latency >= UInt(PMSLATFR_EL1().MINLAT));
    if is_lat then PMUEvent(PMU_EVENT_SAMPLE_FEED_LAT); end;

    var is_rejected_latency : boolean = FALSE;
    if PMSFCR_EL1().FL == '1' then
        // Filtering by latency is enabled
        if !IsZero(PMSLATFR_EL1().MINLAT) then
            // Not a CONSTRAINED UNPREDICTABLE case
            is_rejected_latency = is_rejected_latency || !is_lat;
        else
            is_rejected_latency = ConstrainUnpredictableBool(Unpredictable_BADPMSFCR);
        end;
    end;

    return is_rejected_latency;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEFilterByType

```
// SPEFilterByType()
// =================
// Returns TRUE if the operation is to be discarded because of its OpAttrs, and FALSE otherwise.

func SPEFilterByType(opattr : SPEOpAttr) => boolean
begin
    // Bit positions in the PMSFCR_EL1().[TYPE, TYPEm] fields
    let B : integer    = 0;
    let LD : integer   = 1;
    let ST : integer   = 2;
    let FP : integer   = 3;
    let SIMD : integer = 4;

    var flags : bits(5) = Zeros{};
    var ctrl : bits(5)  = Zeros{};
    // With GCS, Branch Link and Procedure Return instructions write and read the GCS.
    // BL and RET instructions with GCS enabled call SPESampleLoadStore() before
    // branch/packet construction, setting ldst_type to indicate a GCS access.
    let is_gcs_ldst : boolean = (opattr.op_type == SPEOpType_Branch &&
                                 opattr.ldst_type == SPELDSTType_GCS);
    let is_load : boolean     = (opattr.op_type IN {SPEOpType_Load, SPEOpType_LoadAtomic} ||
                                 (is_gcs_ldst && opattr.procedure_return));
    let is_store : boolean    = (opattr.op_type IN {SPEOpType_Store, SPEOpType_LoadAtomic} ||
                                 (is_gcs_ldst && opattr.branch_has_link));
    flags[B]    = (if opattr.op_type == SPEOpType_Branch then '1' else '0');
    flags[LD]   = (if is_load then '1' else '0');
    flags[ST]   = (if is_store then '1' else '0');
    flags[FP]   = (if opattr.is_floating_point then '1' else '0');
    flags[SIMD] = (if opattr.is_simd then '1' else '0');
    ctrl[2:0]   = PMSFCR_EL1().TYPE[2:0];
    var mask : bits(5) = Zeros{};
    if IsFeatureImplemented(FEAT_SPE_EFT) then
        ctrl[4:3] = PMSFCR_EL1().TYPE[4:3];
        mask[4:0] = PMSFCR_EL1().TYPEm;
    end;

    let ctrl_or : bits(5)  = (ctrl AND (NOT mask));
    let ctrl_and : bits(5) = (ctrl AND mask);

    let is_op : boolean = ((IsZero(ctrl_or) || !IsZero(flags AND ctrl_or)) &&
                           ((flags AND mask) == ctrl_and));

    if flags[B]    == '1' then PMUEvent(PMU_EVENT_SAMPLE_FEED_BR); end;
    if flags[LD]   == '1' then PMUEvent(PMU_EVENT_SAMPLE_FEED_LD); end;
    if flags[ST]   == '1' then PMUEvent(PMU_EVENT_SAMPLE_FEED_ST); end;
    if flags[FP]   == '1' then PMUEvent(PMU_EVENT_SAMPLE_FEED_FP); end;
    if flags[SIMD] == '1' then PMUEvent(PMU_EVENT_SAMPLE_FEED_SIMD); end;

    var is_rejected_type : boolean = FALSE;
    if PMSFCR_EL1().FT == '1' then
        // Filtering by type is enabled
        if IsFeatureImplemented(FEAT_SPE_EFT) || !IsZero(ctrl) then
            is_rejected_type = !is_op;
        else
            is_rejected_type = ConstrainUnpredictableBool(Unpredictable_BADPMSFCR);
        end;
    end;
    if is_op && (!IsZero(ctrl) || !IsZero(mask)) then PMUEvent(PMU_EVENT_SAMPLE_FEED_OP); end;

    return is_rejected_type;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEFreezeOnEvent

```
// SPEFreezeOnEvent()
// ==================
// Returns TRUE if PMU event counter idx should be frozen due to an SPE event, and FALSE otherwise.

func SPEFreezeOnEvent(idx : integer) => boolean
begin
    let counters : integer = NUM_PMU_COUNTERS;
    assert (idx >= 0 &&
            (idx < counters || idx == CYCLE_COUNTER_ID ||
             (idx == INSTRUCTION_COUNTER_ID && IsFeatureImplemented(FEAT_PMUv3_ICNTR))));

    if !IsFeatureImplemented(FEAT_SPEv1p2) || !IsFeatureImplemented(FEAT_PMUv3p7) then
        return FALSE;
    end;
    if idx == CYCLE_COUNTER_ID && !IsFeatureImplemented(FEAT_SPE_DPFZS) then
        // FZS does not affect the cycle counter when FEAT_SPE_DPFZS is not implemented
        return FALSE;
    end;

    let freeze_on_event : boolean = PMBLIMITR_EL1().[E,PMFZ] == '11';
    let stopped : boolean = SPEProfilingStopped();

    case GetPMUCounterRange(idx) of
        when PMUCounterRange_R1 =>
            return freeze_on_event && stopped && PMCR_EL0().FZS == '1';
        when PMUCounterRange_R2 =>
            return freeze_on_event && stopped && MDCR_EL2().HPMFZS == '1';
        otherwise =>
            return FALSE;
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEGetDataSource

```
// SPEGetDataSource()
// ==================
// Returns a tuple indicating the data source for an access passed to SPESampleLoadStore, and
// whether the data source is valid.

impdef func SPEGetDataSource(is_load : boolean,
                             accdesc : AccessDescriptor,
                             addrdesc : AddressDescriptor) => (boolean, bits(16))
begin
    return (TRUE, Zeros{16});
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEGetDataSourcePayloadSize

```
// SPEGetDataSourcePayloadSize()
// =============================
// Returns the size of the Data Source payload in bytes.

func SPEGetDataSourcePayloadSize() => integer{1..2}
begin
    return ImpDefInt("SPE Data Source packet payload size") as integer{1..2};
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEGetEventsPayloadSize

```
// SPEGetEventsPayloadSize()
// =========================
// Returns the size in bytes of the Events packet payload as an integer.

func SPEGetEventsPayloadSize() => integer{1..4}
begin
    return ImpDefInt("SPE Events packet payload size") as integer{1..4};
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEGetRandomBoolean

```
// SPEGetRandomBoolean()
// =====================
// Returns a random or pseudo-random boolean value.

impdef func SPEGetRandomBoolean() => boolean
begin
    return SPEGetRandomInterval()[0] == '1';
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEGetRandomInterval

```
// SPEGetRandomInterval()
// ======================
// Returns a random or pseudo-random byte for resetting COUNT or ECOUNT.

impdef func SPEGetRandomInterval() => bits(8)
begin
    return _PC[0+:8];
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEISB

```
// SPEISB()
// ========
// Called by ISB instruction, correctly calls SPEBranch to save previous branches.

func SPEISB()
begin
    let address : bits(64)           = PC64() + 4;
    let branch_type : BranchType     = BranchType_DIR;
    let branch_conditional : boolean = FALSE;
    let taken : boolean              = FALSE;
    let is_isb : boolean             = TRUE;

    SPEBranch{64}(address, branch_type, branch_conditional, taken, is_isb);
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEInterruptEnabled

```
// SPEInterruptEnabled()
// =====================
// Return TRUE if the SPE interrupt request (PMBIRQ) is enabled, FALSE otherwise.

func SPEInterruptEnabled() => boolean
begin
    return EffectivePMSCR_EL1_EE() == '00';
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPELDSTType

```
// SPELDSTType
// ===========
// Type of a load or store operation.

type SPELDSTType of enumeration {
    SPELDSTType_NV2,
    SPELDSTType_Extended,
    SPELDSTType_General,
    SPELDSTType_SIMDFP,
    SPELDSTType_SVESME,
    SPELDSTType_Tags,
    SPELDSTType_MemCopy,
    SPELDSTType_MemSet,
    SPELDSTType_GCS,
    SPELDSTType_GCSSS2,
    SPELDSTType_Unspecified
};
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEMultiAccessSample

```
// SPEMultiAccessSample()
// ======================
// Called by instructions which make at least one store and one load access, where the configuration
// of the operation type filter affects which access is sampled.

func SPEMultiAccessSample() => boolean
begin
    // If loads or stores are filtered out, the other should be recorded.
    // If neither or both are filtered out, pick one in an unbiased way.

    // Are loads allowed by filter?
    let loads_pass_filter : boolean  = PMSFCR_EL1().FT == '1' && PMSFCR_EL1().TYPE.LD == '1';
    // Are stores allowed by filter?
    let stores_pass_filter : boolean = PMSFCR_EL1().FT == '1' && PMSFCR_EL1().TYPE.ST == '1';

    var record_load : boolean;
    if loads_pass_filter && !stores_pass_filter then
        // Only loads pass filter
        record_load = TRUE;
    elsif !loads_pass_filter && stores_pass_filter then
        // Only stores pass filter
        record_load = FALSE;
    else
        // Both loads and stores pass the filter or neither pass the filter.
        // Pick between the load or the store access (pseudo-)randomly.
        record_load = SPEGetRandomBoolean();
    end;
    return record_load;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEOpAttr

```
// SPEOpAttr
// =========
// Attributes of sampled operation filtered by SPECollectRecord().

type SPEOpAttr of record {
    op_type : SPEOpType,
    ldst_type : SPELDSTType,
    branch_is_direct : boolean,
    branch_has_link : boolean,
    procedure_return : boolean,
    is_conditional : boolean,
    is_floating_point : boolean,
    is_simd : boolean,
    cond_pass : boolean,
    at : boolean,
    is_acquire_release : boolean,
    is_predicated : boolean,
    evl : bits(3),
    is_gather_scatter : boolean,
    is_exclusive : boolean,
    ets : bits(4),
    addr_valid : boolean
};
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEOpType

```
// SPEOpType
// =========
// Types of operation filtered by SPECollectRecord().

type SPEOpType of enumeration {
    SPEOpType_Load,        // Any memory-read operation other than atomics, compare-and-swap,
                           // and swap
    SPEOpType_Store,       // Any memory-write operation, including atomics without return
    SPEOpType_LoadAtomic,  // Atomics with return, compare-and-swap and swap
    SPEOpType_Branch,      // Software write to the PC
    SPEOpType_OtherSVE,    // Other SVE operation
    SPEOpType_OtherSME,    // Other SME operation
    SPEOpType_Other,       // Any other class of operation
    SPEOpType_Invalid
};
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEProfilingStopped

```
// SPEProfilingStopped()
// =====================

readonly func SPEProfilingStopped() => boolean
begin
    var stopped : boolean = (PMBSR_EL1().S == '1');
    if IsFeatureImplemented(FEAT_SPE_EXC) then
        if HaveEL(EL3) && MDCR_EL3().PMSEE == '1x' then
            stopped = stopped || (PMBSR_EL3().S == '1');
        end;
        if EffectivePMSCR_EL2_EE() == '1x' then
            stopped = stopped || (PMBSR_EL2().S == '1');
        end;
    end;
    return stopped;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEResetSampleCounter

```
// SPEResetSampleCounter()
// =======================
// Reset PMSICR_EL1.Counter

func SPEResetSampleCounter()
begin
    PMSICR_EL1().COUNT[31:8] = PMSIRR_EL1().INTERVAL;
    if PMSIRR_EL1().RND == '1' && !IsFeatureImplemented(FEAT_SPE_ERnd) then
        PMSICR_EL1().COUNT[7:0] = SPEGetRandomInterval();
    else
        PMSICR_EL1().COUNT[7:0] = Zeros{8};
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEResetSampleStorage

```
// SPEResetSampleStorage()
// =======================
// Reset all variables inside sample storage.

func SPEResetSampleStorage()
begin
    // Context values
    SPESampleContextEL1      = Zeros{32};
    SPESampleContextEL1Valid = FALSE;
    SPESampleContextEL2      = Zeros{32};
    SPESampleContextEL2Valid = FALSE;

    // Counter values
    for i = 0 to (SPEMaxCounters - 1) do
        SPESampleCounter[[i]] = 0;
        SPESampleCounterValid[[i]] = FALSE;
        SPESampleCounterPending[[i]] = FALSE;
    end;

    // Address values
    for i = 0 to (SPEMaxAddrs - 1) do
        SPESampleAddressValid[[i]] = FALSE;
        SPESampleAddress[[i]] = Zeros{64};
    end;

    // Data source values
    SPESampleDataSource      = Zeros{16};
    SPESampleDataSourceValid = FALSE;

    // Timestamp values
    SPESampleTimestamp      = Zeros{64};
    SPESampleTimestampValid = FALSE;

    // Event values
    SPESampleEvents = Zeros{64};

    // Operation attributes
    SPESampleOpAttr.op_type = SPEOpType_Invalid;
    SPESampleOpAttr.ldst_type          = SPELDSTType_Unspecified;
    SPESampleOpAttr.branch_is_direct   = FALSE;
    SPESampleOpAttr.branch_has_link    = FALSE;
    SPESampleOpAttr.procedure_return   = FALSE;
    SPESampleOpAttr.is_conditional     = FALSE;
    SPESampleOpAttr.is_floating_point  = FALSE;
    SPESampleOpAttr.is_simd            = FALSE;
    SPESampleOpAttr.cond_pass          = FALSE;
    SPESampleOpAttr.at                 = FALSE;
    SPESampleOpAttr.is_acquire_release = FALSE;
    SPESampleOpAttr.is_exclusive       = FALSE;
    SPESampleOpAttr.is_predicated      = FALSE;
    SPESampleOpAttr.evl                = '000';
    SPESampleOpAttr.is_gather_scatter  = FALSE;
    SPESampleOpAttr.addr_valid         = FALSE;
end;

var SPERecordData : array [[SPEMaxRecordSize]] of bits(8);
var SPERecordSize : integer;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleAddAddressPCVirtual

```
// SPESampleAddAddressPCVirtual()
// ==============================
// Save the current PC address to sample storage.

func SPESampleAddAddressPCVirtual()
begin
    var ns : bit;
    var nse : bit;
    case CurrentSecurityState() of
        when SS_Secure =>
            ns = '0';
            nse = '0';
        when SS_NonSecure =>
            ns = '1';
            nse = '0';
        when SS_Realm =>
            ns = '1';
            nse = '1';
        otherwise => unreachable;
    end;
    let el : bits(2) = PSTATE.EL;

    let pc : bits(64) = ThisInstrAddr{}();
    SPESampleAddress[[SPEAddrPosPCVirtual]][55:0] = pc[55:0];
    SPESampleAddress[[SPEAddrPosPCVirtual]][63:56] = ns::el::nse::Zeros{4};
    SPESampleAddressValid[[SPEAddrPosPCVirtual]]   = TRUE;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleAddContext

```
// SPESampleAddContext()
// =====================
// Save contexts to sample storage if appropriate.

func SPESampleAddContext()
begin
    if CollectContextIDR1() then
        SPESampleContextEL1      = CONTEXTIDR_EL1()[31:0];
        SPESampleContextEL1Valid = TRUE;
    end;
    if CollectContextIDR2() then
        SPESampleContextEL2      = CONTEXTIDR_EL2()[31:0];
        SPESampleContextEL2Valid = TRUE;
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleAddTimeStamp

```
// SPESampleAddTimeStamp()
// =======================
// Save the appropriate type of timestamp to sample storage.

func SPESampleAddTimeStamp()
begin
    let timestamp : TimeStamp = CollectTimeStamp();
    case timestamp of
        when TimeStamp_None =>
            SPESampleTimestampValid = FALSE;
        otherwise =>
            SPESampleTimestampValid = TRUE;
            SPESampleTimestamp = GetTimestamp(timestamp);
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleCollision

```
// SPESampleCollision()
// ====================
// Called when there is an SPE sample collision.

func SPESampleCollision()
begin
    // Sample collision with the previous sample
    PMUEvent(PMU_EVENT_SAMPLE_COLLISION);
    let target_el : bits(2)  = DefaultSPEEvent();
    PMBSR_EL(target_el).COLL = '1';
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleExtendedLoadStore

```
// SPESampleExtendedLoadStore()
// ============================
// Sets the subclass of the operation type packet for
// extended load/store operations.

func SPESampleExtendedLoadStore(ar : boolean, excl : boolean, at : boolean, is_load : boolean)
begin
    SPESampleOpAttr.is_acquire_release = ar;
    SPESampleOpAttr.is_exclusive       = excl;
    SPESampleOpAttr.ldst_type          = SPELDSTType_Extended;
    SPESampleOpAttr.at                 = at;
    if is_load then
        if at then
            SPESampleOpAttr.op_type = SPEOpType_LoadAtomic;
        else
            SPESampleOpAttr.op_type = SPEOpType_Load;
        end;
    else
        SPESampleOpAttr.op_type = SPEOpType_Store;
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleGCSSS2

```
// SPESampleGCSSS2()
// =================
// Sets the subclass of the operation type packet for GCSSS2 load/store operations.

func SPESampleGCSSS2()
begin
    // GCSSS2 does a read and a write.
    let record_load : boolean = SPEMultiAccessSample();
    SPESampleOpAttr.op_type   = if record_load then SPEOpType_Load else SPEOpType_Store;
    SPESampleOpAttr.ldst_type = SPELDSTType_GCSSS2;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleGeneralPurposeLoadStore

```
// SPESampleGeneralPurposeLoadStore()
// ==================================
// Sets the subclass of the operation type packet for general
// purpose load/store operations.

func SPESampleGeneralPurposeLoadStore(is_load : boolean)
begin
    SPESampleOpAttr.ldst_type = SPELDSTType_General;
    SPESampleOpAttr.op_type = if is_load then SPEOpType_Load else SPEOpType_Store;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleLoadStore

```
// SPESampleLoadStore()
// ====================
// Called if a sample is in flight when writing or reading memory,
// indicating that the operation being sampled is in the Load, Store or atomic category.

func SPESampleLoadStore(is_load : boolean, accdesc : AccessDescriptor,
                        addrdesc : AddressDescriptor)
begin
    // Check if this access type is suitable to be sampled.
    // This implementation of SPE always samples the first access made by a suitable instruction.
    // FEAT_MOPS instructions are an exception, where the first load or first store access may be
    // selected based on the configuration of the sample filters.
    if accdesc.acctype IN {AccessType_SPE,
                           AccessType_IFETCH,
                           AccessType_TRBE,
                           AccessType_DC,
                           AccessType_TTW,
                           AccessType_AT} then
        return;
    end;

    var sample_access : boolean = FALSE;
    // For FEAT_MOPS and FEAT_GCS GCSSS2 instructions which perform both loads and stores, the
    // filter configuration will influence which part of the access is chosen to be sampled.
    if (SPESampleOpAttr.ldst_type IN
          {SPELDSTType_MemCopy, SPELDSTType_MemSet, SPELDSTType_GCSSS2}) then
        // SPEMultiAccessSample() will have been called before this function,
        // and chooses whether to sample a load or a store.
        var sample_load : boolean;
        sample_load = SPESampleOpAttr.op_type IN {SPEOpType_Load, SPEOpType_LoadAtomic};

        // If no valid data has been collected, and this operation is acceptable for sampling.
        if !SPESampleOpAttr.addr_valid && (is_load == sample_load) then
            sample_access = TRUE;
        end;
    else
        if !SPESampleOpAttr.addr_valid then
            sample_access = TRUE;
        end;
    end;

    if sample_access then
        // Data access virtual address
        SPESetDataVirtualAddress(addrdesc.vaddress);

        // Data access physical address
        if CollectPhysicalAddress() then
            SPESetDataPhysicalAddress(addrdesc, accdesc);
        end;

        SPESampleOpAttr.addr_valid = TRUE;
    end;

    if SPESampleOpAttr.op_type == SPEOpType_Invalid then
        // Set as unspecified load/store by default, instructions will overwrite this if it does not
        // apply to them.
        SPESampleOpAttr.op_type = if is_load then SPEOpType_Load else SPEOpType_Store;

        if accdesc.acctype == AccessType_NV2 then
            // NV2 register load/store
            SPESampleOpAttr.ldst_type = SPELDSTType_NV2;
        end;
    end;

    // Set SPELDSTType to GCS for all GCS instruction, overwriting type GCSSS2.
    // After selection of which operation of a GCSSS2 instruction to sample, GCSSS2 is treated the
    // same as other GCS instructions.
    if accdesc.acctype == AccessType_GCS then
        SPESampleOpAttr.ldst_type = SPELDSTType_GCS;
        // If the GCS access is from a BL or RET, this will get overwritten to SPEOpType_Branch.
        SPESampleOpAttr.op_type = if is_load then SPEOpType_Load else SPEOpType_Store;
    end;
    (SPESampleDataSourceValid, SPESampleDataSource) = SPEGetDataSource(is_load, accdesc, addrdesc);
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleMemCopy

```
// SPESampleMemCopy()
// ==================
// Sets the subclass of the operation type packet for Memory Copy load/store
// operations.

func SPESampleMemCopy()
begin
    // MemCopy does a read and a write.
    let record_load : boolean = SPEMultiAccessSample();
    SPESampleOpAttr.op_type   = if record_load then SPEOpType_Load else SPEOpType_Store;
    SPESampleOpAttr.ldst_type = SPELDSTType_MemCopy;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleMemSet

```
// SPESampleMemSet()
// =================
// Callback used by memory set instructions to pass data back to the SPU.

func SPESampleMemSet()
begin
    SPESampleOpAttr.op_type   = SPEOpType_Store;
    SPESampleOpAttr.ldst_type = SPELDSTType_MemSet;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleOnSharedResource

```
// SPESampleOnSharedResource()
// ===========================
// Called when the sampled instruction is executed on an SMCU or other shared resource, sets bit 25
// of the events packet to 0b1.
func SPESampleOnSharedResource()
begin
    SPESampleEvents[25] = '1';
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleOpOther

```
// SPESampleOpOther()
// ==================
// Add other operation to sample storage.

func SPESampleOpOther(conditional : boolean, cond_pass : boolean,
                      is_fp : boolean, is_simd : boolean)
begin
    SPESampleOpAttr.is_simd = is_simd;
    SPESampleOpOther(conditional, cond_pass, is_fp);
end;

func SPESampleOpOther(conditional : boolean, cond_pass : boolean, is_fp : boolean)
begin
    SPESampleOpAttr.cond_pass = cond_pass;
    SPESampleOpAttr.is_floating_point = is_fp;
    SPESampleOpOther(conditional);
end;

func SPESampleOpOther(conditional : boolean)
begin
    SPESampleOpAttr.is_conditional = conditional;
    SPESampleOpAttr.op_type = SPEOpType_Other;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleOpSMEArrayOther

```
// SPESampleOpSMEArrayOther()
// ==========================
// Sets the subclass of the operation type packet to Other, SME array.

func SPESampleOpSMEArrayOther(floating_point : boolean, size : integer)
begin
    // If the sampled effective vector or tile size is not a power of two, or is less than 128 bits,
    // the value is rounded up before it is encoded in the ets field.

    SPESampleOpAttr.is_floating_point = floating_point;
    SPESampleOpAttr.ets               = SPEEncodeETS(size);
    SPESampleOpAttr.op_type           = SPEOpType_OtherSME;
    SPESampleOpAttr.is_simd           = TRUE;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleOpSVEOther

```
// SPESampleOpSVEOther()
// =====================
// Callback used by SVE, Other operations to pass data back to the SPU.

func SPESampleOpSVEOther(vl : integer, predicated : boolean, floating_point : boolean)
begin
    SPESampleOpAttr.is_predicated     = predicated;
    SPESampleOpAttr.is_floating_point = floating_point;
    SPESampleOpAttr.evl               = SPEEncodeEVL(vl);
    SPESampleOpAttr.op_type           = SPEOpType_OtherSVE;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleOpSVESMELoadStore

```
// SPESampleOpSVESMELoadStore()
// ============================
// Callback used by SVE or SME loads and stores to pass data to SPE.

func SPESampleOpSVESMELoadStore(is_gather_scatter : boolean, vl : integer, predicated : boolean,
                                is_load : boolean)
begin
    SPESampleOpAttr.is_gather_scatter = is_gather_scatter;
    SPESampleOpAttr.is_predicated     = predicated;
    SPESampleOpAttr.evl               = SPEEncodeEVL(vl);
    assert SPESampleOpAttr.evl != '111';
    SPESampleOpAttr.op_type = if is_load then SPEOpType_Load else SPEOpType_Store;
    SPESampleOpAttr.ldst_type = SPELDSTType_SVESME;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESampleSIMDFPLoadStore

```
// SPESampleSIMDFPLoadStore()
// ==========================
// Sets the subclass of the operation type packet for SIMD & FP
// load store operations.

func SPESampleSIMDFPLoadStore(is_load : boolean, scalar : boolean)
begin
    SPESampleOpAttr.ldst_type = SPELDSTType_SIMDFP;
    SPESampleOpAttr.op_type   = if is_load then SPEOpType_Load else SPEOpType_Store;
    SPESampleOpAttr.is_simd   = !scalar;
    // Scalar operations in SIMD&FP are treated as floating point.
    SPESampleOpAttr.is_floating_point = scalar;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESetDataPhysicalAddress

```
// SPESetDataPhysicalAddress()
// ===========================
// Called from SampleLoadStore() to save data physical packet.

func SPESetDataPhysicalAddress(addrdesc : AddressDescriptor, accdesc : AccessDescriptor)
begin
    var ns : bit;
    var nse : bit;
    var ch : bit;
    var lat : bits(4);
    case addrdesc.paddress.paspace of
        when PAS_Secure =>
            ns = '0';
            nse = '0';
        when PAS_NonSecure =>
            ns = '1';
            nse = '0';
        when PAS_Realm =>
            ns  = '1';
            nse = '1';
        otherwise => unreachable;
    end;

    if IsFeatureImplemented(FEAT_MTE2) then
        if accdesc.tagchecked then
            ch = '1';
            lat = AArch64_LogicalAddressTag(addrdesc.vaddress);
        else
            ch = '0';
            // If the access is Unchecked, this is an IMPLEMENTATION_DEFINED choice
            // between 0b0000 and the Logical Address Tag
            var zero_unchecked : boolean;
            zero_unchecked = ImpDefBool("SPE PAT for tag unchecked access zero");
            if !zero_unchecked then
                lat = AArch64_LogicalAddressTag(addrdesc.vaddress);
            else
                lat = Zeros{4};
            end;
        end;
    else
        ch = '0';
        lat = '0000';
    end;

    let paddr : bits(NUM_PABITS) = addrdesc.paddress.address;
    SPESampleAddress[[SPEAddrPosDataPhysical]][55:0] = ZeroExtend{56}(paddr);
    SPESampleAddress[[SPEAddrPosDataPhysical]][63:56] = ns::ch::'0'::nse::lat;
    SPESampleAddressValid[[SPEAddrPosDataPhysical]] = TRUE;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPESetDataVirtualAddress

```
// SPESetDataVirtualAddress()
// ==========================
// Called from SampleLoadStore() to save data virtual packet.
// Also used by exclusive load/stores to save virtual addresses if exclusive monitor is lost
// before a read/write is completed.

func SPESetDataVirtualAddress(vaddress : bits(64))
begin
    var tbi : bit;
    tbi = EffectiveTBI(vaddress, FALSE, PSTATE.EL);
    var non_tbi_is_zeros : boolean;
    non_tbi_is_zeros = ImpDefBool("SPE non-tbi tag is zero");
    if tbi == '1' || !non_tbi_is_zeros then
        SPESampleAddress[[SPEAddrPosDataVirtual]][63:0]  = vaddress[63:0];
    else
        SPESampleAddress[[SPEAddrPosDataVirtual]][55:0]  = vaddress[55:0];
        SPESampleAddress[[SPEAddrPosDataVirtual]][63:56] = Zeros{8};
    end;
    SPESampleAddressValid[[SPEAddrPosDataVirtual]]       = TRUE;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEStartCounter

```
// SPEStartCounter()
// =================
// Enables incrementing of the counter at the passed index when SPECycle is called.

func SPEStartCounter(counter_index : integer)
begin
    assert counter_index < SPEMaxCounters;
    SPESampleCounterPending[[counter_index]] = TRUE;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEStartSample

```
// SPEStartSample()
// ================
// Called to start the sampling process. Returns TRUE if a new sample is to be collected, and
// FALSE otherwise.

func SPEStartSample() => boolean
begin
    if !StatisticalProfilingEnabled() then
        return FALSE;
    end;
    PMUEvent(PMU_EVENT_SAMPLE_POP);
    if !SPEToCollectSample() then
        return FALSE;
    end;

    if SPESampleInFlight then
        SPESampleCollision();
        return FALSE;
    end;

    SPESampleInFlight = TRUE;

    SPEStartCounter(SPECounterPosTotalLatency);
    SPEStartCounter(SPECounterPosIssueLatency);
    SPESampleAddAddressPCVirtual();

    // Many operations are type other and not conditional, can save footprint
    // and overhead by having this as the default and not calling SPESampleOpOther
    // if conditional == FALSE
    SPESampleOpOther(FALSE);

    SPESampleAddContext();

    // Timestamp may be collected at any point in the sampling operation.
    // Collecting prior to execution is one possible choice.
    // This choice is IMPLEMENTATION DEFINED.
    SPESampleAddTimeStamp();

    return TRUE;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEStopCounter

```
// SPEStopCounter()
// ================
// Disables incrementing of the counter at the passed index when SPECycle is called.

func SPEStopCounter(counter_index : integer)
begin
    SPESampleCounterValid[[counter_index]] = TRUE;
    SPESampleCounterPending[[counter_index]] = FALSE;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEToCollectSample

```
// SPEToCollectSample()
// ====================
// Returns TRUE if the instruction which is about the executed should be sampled, and FALSE
// otherwise.

func SPEToCollectSample() => boolean
begin
    if IsZero(PMSICR_EL1().COUNT) then
        SPEResetSampleCounter();
    else
        PMSICR_EL1().COUNT = PMSICR_EL1().COUNT - 1;
        if IsZero(PMSICR_EL1().COUNT) then
            if PMSIRR_EL1().RND == '1' && IsFeatureImplemented(FEAT_SPE_ERnd) then
                PMSICR_EL1().ECOUNT = SPEGetRandomInterval();
                return IsZero(PMSICR_EL1().ECOUNT);
            else
                return TRUE;
            end;
        end;
    end;
    if (PMSIRR_EL1().RND == '1' && IsFeatureImplemented(FEAT_SPE_ERnd) &&
          !IsZero(PMSICR_EL1().ECOUNT)) then
        PMSICR_EL1().ECOUNT = PMSICR_EL1().ECOUNT - 1;
        if IsZero(PMSICR_EL1().ECOUNT) then
            return TRUE;
        end;
    end;
    return FALSE;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/SPEWriteToBuffer

```
// SPEWriteToBuffer()
// ==================
// Write the active record to the Profiling Buffer.

func SPEWriteToBuffer()
begin
    assert ProfilingBufferEnabled() && !SPEProfilingStopped();

    // Check alignment
    let align : integer{} = UInt(PMBIDR_EL1().Align);
    let aligned : boolean = IsAlignedP2(PMBPTR_EL1().PTR, align);

    let ttw_abort_as_fault : boolean = (ImpDefBool(
                                        "Report SPE ExtAbort on TTW as fault"));

    var owning_ss : SecurityState;
    var owning_el : bits(2);
    (owning_ss, owning_el) = ProfilingBufferOwner();
    let accdesc : AccessDescriptor = CreateAccDescSPE(owning_ss, owning_el);

    let start_vaddr : bits(64) = PMBPTR_EL1();
    for i = 0 to SPERecordSize - 1 do
        // If a previous write did not cause an issue
        if !SPEProfilingStopped() then
            let address : bits(64) = PMBPTR_EL1();
            var memstatus : PhysMemRetStatus;
            var addrdesc : AddressDescriptor;
            (memstatus, addrdesc) = DebugMemWrite(address, accdesc, aligned,
                                                  SPERecordData[[i]]);

            let fault : FaultRecord = addrdesc.fault;
            let ttw_abort : boolean = fault.statuscode IN {Fault_SyncExternalOnWalk,
                                                           Fault_SyncParityOnWalk};

            if IsFault(fault.statuscode) && (!ttw_abort || ttw_abort_as_fault) then
                DebugWriteFault(address, fault);
            elsif IsFault(memstatus) || (ttw_abort && !ttw_abort_as_fault) then
                DebugWriteExternalAbort(memstatus, addrdesc, start_vaddr);
            end;

            // Move pointer if no Buffer Management Event has been caused.
            if !SPEProfilingStopped() then
                PMBPTR_EL1() = PMBPTR_EL1() + 1;
            end;
        end;
    end;
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/StatisticalProfilingEnabled

```
// StatisticalProfilingEnabled()
// =============================
// Return TRUE if Statistical Profiling is Enabled in the current EL, FALSE otherwise.

func StatisticalProfilingEnabled() => boolean
begin
    return StatisticalProfilingEnabled(PSTATE.EL);
end;

// StatisticalProfilingEnabled()
// =============================
// Return TRUE if Statistical Profiling is Enabled in the specified EL, FALSE otherwise.

func StatisticalProfilingEnabled(el : bits(2)) => boolean
begin
    if (!IsFeatureImplemented(FEAT_SPE) || UsingAArch32() ||
          !ProfilingBufferEnabled() || SPEProfilingStopped() || Halted()) then
        return FALSE;
    end;

    let tge_set : boolean = EL2Enabled() && HCR_EL2().TGE == '1';
    let (owning_ss, owning_el) : (SecurityState, bits(2)) = ProfilingBufferOwner();
    if (UInt(owning_el) <  UInt(el) || (tge_set && owning_el == EL1) ||
          owning_ss != SecurityStateAtEL(el))  then
        return FALSE;
    end;
    var spe_bit : bit;
    case el of
        when EL3 =>  unreachable;
        when EL2 =>  spe_bit = PMSCR_EL2().E2SPE;
        when EL1 =>  spe_bit = PMSCR_EL1().E1SPE;
        when EL0 =>  spe_bit = (if tge_set then PMSCR_EL2().E0HSPE else PMSCR_EL1().E0SPE);
    end;

    return spe_bit == '1';
end;
```

Library pseudocode for aarch64/debug/statisticalprofiling/TimeStamp

```
// TimeStamp
// =========

type TimeStamp of enumeration {
    TimeStamp_None,              // No timestamp
    TimeStamp_CoreSight,         // CoreSight time (IMPLEMENTATION DEFINED)
    TimeStamp_Physical,          // Physical counter value with no offset
    TimeStamp_OffsetPhysical,    // Physical counter value minus CNTPOFF_EL2
    TimeStamp_Virtual  };        // Physical counter value minus CNTVOFF_EL2
```

Library pseudocode for aarch64/debug/takeexceptiondbg/AArch64_TakeExceptionInDebugState

```
// AArch64_TakeExceptionInDebugState()
// ===================================
// Take an exception in Debug state to an Exception level using AArch64.

noreturn func AArch64_TakeExceptionInDebugState(target_el : bits(2), exception_in : ExceptionRecord)
begin
    assert HaveEL(target_el) && !ELUsingAArch32(target_el) && UInt(target_el) >= UInt(PSTATE.EL);
    assert target_el != EL3 || EDSCR().SDD == '0';
    var except : ExceptionRecord = exception_in;

    if !IsFeatureImplemented(FEAT_ExS) || SCTLR_EL(target_el).EIS == '1' then
        // Synchronize the context, including Instruction Fetch Barrier effect
        SynchronizeContext();
    end;

    // If coming from AArch32 state, the top parts of the X() registers might be set to zero
    let from_32 : boolean = UsingAArch32();
    if from_32 then AArch64_MaybeZeroRegisterUppers(); end;
    if from_32 && IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then
        ResetSVEState();
    else
        MaybeZeroSVEUppers(target_el);
    end;

    AArch64_ReportException(except, target_el);

    if IsFeatureImplemented(FEAT_GCS) then
        PSTATE.EXLOCK = '0';  // Effective value of GCSCR_ELx.EXLOCKEN is 0 in Debug state
    end;

    PSTATE.EL  = target_el;
    PSTATE.nRW = '0';
    PSTATE.SP  = '1';

    SPSR_ELx() = ARBITRARY : bits(64);
    ELR_ELx() = ARBITRARY : bits(64);
    // PSTATE.[SS,D,A,I,F] are not observable and ignored in Debug state, so behave as if UNKNOWN.
    PSTATE.[SS,D,A,I,F] = ARBITRARY : bits(5);
    if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = ARBITRARY : bit; end;
    if (IsFeatureImplemented(FEAT_EBEP) || IsFeatureImplemented(FEAT_SPE_EXC) ||
          IsFeatureImplemented(FEAT_TRBE_EXC)) then
        PSTATE.PM = ARBITRARY : bit;
    end;
    if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = '1'; end;
    PSTATE.IL = '0';
    if IsFeatureImplemented(FEAT_UAO) then PSTATE.UAO = '0'; end;
    if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    if (IsFeatureImplemented(FEAT_PAN) && (PSTATE.EL == EL1 ||
          (PSTATE.EL == EL2 && ELIsInHost(EL0))) &&
          SCTLR_ELx().SPAN == '0') then
        PSTATE.PAN = '1';
    end;
    if from_32 then                             // Coming from AArch32
        PSTATE.IT = '00000000';
        PSTATE.T = '0';                         // PSTATE.J is RES0
    end;
    if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = '00'; end;
    DLR_EL0() = ARBITRARY : bits(64);
    DSPSR_EL0() = ARBITRARY : bits(64);

    EDSCR().ERR = '1';
    UpdateEDSCRFields();                        // Update EDSCR PE state flags.

    EndOfInstruction(EndOfInstructionReason_Exception);
end;
```

Library pseudocode for aarch64/debug/watchpoint/AArch64_WatchpointByteMatch

```
// AArch64_WatchpointByteMatch()
// =============================

func AArch64_WatchpointByteMatch(n : integer, vaddress : bits(64)) => boolean
begin
    let dbgtop : AddressSize        = DebugAddrTop();
    let cmpbottom : integer{}       = if DBGWVR_EL1(n)[2] == '1' then 2 else 3;// Word or doubleword
    var bottom : integer{}          = cmpbottom;
    let select : integer{}          = UInt(vaddress[cmpbottom-1:0]);
    var byte_select_match : boolean = (DBGWCR_EL1(n).BAS[select] != '0');
    var mask : integer{}            = UInt(DBGWCR_EL1(n).MASK);

    // If DBGWCR_EL1(n).MASK is a nonzero value and DBGWCR_EL1(n).BAS is not set to '11111111', or
    // DBGWCR_EL1(n).BAS specifies a non-contiguous set of bytes behavior is CONSTRAINED
    // UNPREDICTABLE.
    if mask > 0 && !IsOnes(DBGWCR_EL1(n).BAS) then
        byte_select_match = ConstrainUnpredictableBool(Unpredictable_WPMASKANDBAS);
    else
        let LSB : bits(8) = (DBGWCR_EL1(n).BAS AND NOT(DBGWCR_EL1(n).BAS - 1));
        let MSB : bits(8) = (DBGWCR_EL1(n).BAS + LSB);
        if !IsZero(MSB AND (MSB - 1)) then          // Not contiguous
            byte_select_match = ConstrainUnpredictableBool(Unpredictable_WPBASCONTIGUOUS);
            bottom            = 3;                  // For the whole doubleword
        end;
    end;

    // If the address mask is set to a reserved value, the behavior is CONSTRAINED UNPREDICTABLE.
    if mask > 0 && mask <= 2 then
        var c : Constraint;
        var unpred_mask : integer;
        (c, unpred_mask) = ConstrainUnpredictableInteger(3, 31, Unpredictable_RESWPMASK);
        assert c IN {Constraint_DISABLED, Constraint_NONE, Constraint_UNKNOWN};
        case c of
            when Constraint_DISABLED => return FALSE; // Disabled
            when Constraint_NONE     => mask = 0;     // No masking
            // Otherwise the value returned by ConstrainUnpredictableInteger is a not-reserved value
            otherwise                => mask = unpred_mask as integer{3..31};
        end;
    end;

    // When FEAT_LVA3 is not implemented, if the DBGWVR_EL1(n).RESS field bits are not a
    // sign extension of the MSB of DBGWVR_EL1(n).VA, it is UNPREDICTABLE whether they
    // appear to be included in the match.
    let unpredictable_ress : boolean = (dbgtop < 55 && !IsOnes(DBGWVR_EL1(n)[63:dbgtop]) &&
                                        !IsZero(DBGWVR_EL1(n)[63:dbgtop]) &&
                                        ConstrainUnpredictableBool(Unpredictable_DBGxVR_RESS));
    let cmpmsb : integer{}    = if unpredictable_ress then 63 else dbgtop;
    let cmplsb : integer{}    = if mask > bottom then mask else bottom;
    let bottombit : integer{} = bottom;
    var WVR_match : boolean   = (vaddress[cmpmsb:cmplsb] == DBGWVR_EL1(n)[cmpmsb:cmplsb]);
    if mask > bottom then
        // If masked bits of DBGWVR_EL1(n) are not zero, the behavior is CONSTRAINED UNPREDICTABLE.
        if WVR_match && !IsZero(DBGWVR_EL1(n)[cmplsb-1:bottombit]) then
            WVR_match = ConstrainUnpredictableBool(Unpredictable_WPMASKEDBITS);
        end;
    end;

    return (WVR_match && byte_select_match);
end;
```

Library pseudocode for aarch64/debug/watchpoint/AArch64_WatchpointMatch

```
// AArch64_WatchpointMatch()
// =========================
// Watchpoint matching in an AArch64 translation regime.

func AArch64_WatchpointMatch(n : integer, vaddress : bits(64), size : integer,
                             accdesc : AccessDescriptor) => WatchpointInfo
begin
    assert !ELUsingAArch32(S1TranslationRegime());
    assert n < NumWatchpointsImplemented();

    let enabled : boolean     = IsWatchpointEnabled(n);
    let linked : boolean      = DBGWCR_EL1(n).WT == '1';
    let isbreakpnt : boolean  = FALSE;
    let lbnx : bits(2) = if IsFeatureImplemented(FEAT_Debugv8p9) then DBGWCR_EL1(n).LBNX else '00';
    let linked_n : integer{}  = UInt(lbnx :: DBGWCR_EL1(n).LBN);
    let ssce : bit            = if IsFeatureImplemented(FEAT_RME) then DBGWCR_EL1(n).SSCE else '0';
    let mismatch : boolean    = IsFeatureImplemented(FEAT_BWE2) && DBGWCR_EL1(n).WT2 == '1';
    let state_match : boolean = AArch64_StateMatch(DBGWCR_EL1(n).SSC, ssce, DBGWCR_EL1(n).HMC,
                                                   DBGWCR_EL1(n).PAC,
                                                   linked, linked_n, isbreakpnt, PC64(), accdesc);

    var watchptinfo : WatchpointInfo;
    var ls_match : boolean;
    case DBGWCR_EL1(n).LSC[1:0] of
        when '00' => ls_match = FALSE;
        when '01' => ls_match = accdesc.read;
        when '10' => ls_match = accdesc.write || accdesc.acctype == AccessType_DC;
        when '11' => ls_match = TRUE;
    end;

    var value_match : boolean = FALSE;
    watchptinfo.vaddress = vaddress;
    for byte = 0 to size - 1 do
        if (!value_match &&
              !AddressInNaturallyAlignedBlock(watchptinfo.vaddress, vaddress + byte)) then
            // Watchpoint should report an address which is in
            // the naturally aligned block of the matched address.
            watchptinfo.vaddress = vaddress + byte;
        end;
        value_match = value_match || AArch64_WatchpointByteMatch(n, vaddress + byte);
    end;

    watchptinfo.watchpt_num = n;
    watchptinfo.value_match = value_match;
    if !(state_match && ls_match && enabled) then
        watchptinfo.wptype = WatchpointType_Inactive;
        watchptinfo.value_match = FALSE;
    elsif mismatch then
        watchptinfo.wptype = WatchpointType_AddrMismatch;
    else
        watchptinfo.wptype = WatchpointType_AddrMatch;
    end;
    return watchptinfo;
end;
```

Library pseudocode for aarch64/debug/watchpoint/IsWatchpointEnabled

```
// IsWatchpointEnabled()
// =====================
// Returns TRUE if the effective value of DBGWCR_EL1(n).E is '1', and FALSE otherwise.

func IsWatchpointEnabled(n : integer) => boolean
begin
    if (n > 15 &&
          ((!HaltOnBreakpointOrWatchpoint() && !SelfHostedExtendedBPWPEnabled()) ||
           (HaltOnBreakpointOrWatchpoint() && EDSCR2().EHBWE == '0'))) then
        return FALSE;
    end;
    return DBGWCR_EL1(n).E == '1';
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_Abort

```
// AArch64_Abort()
// ===============
// Abort and Debug exception handling in an AArch64 translation regime.

func AArch64_Abort(fault : FaultRecord)
begin
    if IsDebugException(fault) then
        if fault.accessdesc.acctype == AccessType_IFETCH then
            if UsingAArch32() && fault.debugmoe == DebugException_VectorCatch then
                AArch64_VectorCatchException(fault);
            else
                AArch64_BreakpointException(fault);
            end;
        else
            AArch64_WatchpointException(fault);
        end;
    elsif fault.gpcf.gpf != GPCF_None && ReportAsGPCException(fault) then
        TakeGPCException(fault);
    elsif fault.statuscode == Fault_TagCheck then
        AArch64_RaiseTagCheckFault(fault);
    elsif IsExternalAbort(fault) && !IsExternalSyncAbort(fault) then
        PendSErrorInterrupt(fault);
    elsif fault.accessdesc.acctype == AccessType_IFETCH then
        AArch64_InstructionAbort(fault);
    else
        AArch64_DataAbort(fault);
    end;
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_AbortSyndrome

```
// AArch64_AbortSyndrome()
// =======================
// Creates an exception syndrome record for Abort and Watchpoint exceptions
// from an AArch64 translation regime.

func AArch64_AbortSyndrome(exceptype : Exception, fault : FaultRecord,
                           target_el : bits(2)) => ExceptionRecord
begin
    var except : ExceptionRecord = ExceptionSyndrome(exceptype);

    except.syndrome = AArch64_FaultSyndrome(exceptype, fault, target_el);
    if exceptype IN {Exception_NV2Watchpoint, Exception_Watchpoint} then
        except.vaddress = fault.watchptinfo.vaddress;
    elsif fault.statuscode == Fault_TagCheck then
        if IsFeatureImplemented(FEAT_MTE_TAGGED_FAR) then
            except.vaddress = ZeroExtend{64}(fault.vaddress);
        else
            except.vaddress = ARBITRARY : bits(4) :: fault.vaddress[59:0];
        end;
    else
        except.vaddress = ZeroExtend{64}(fault.vaddress);
    end;

    if IPAValid(fault) then
        except.ipavalid = TRUE;
        except.NS = if fault.ipaddress.paspace == PAS_NonSecure then '1' else '0';
        except.ipaddress = fault.ipaddress.address;
    else
        except.ipavalid = FALSE;
    end;

    if except.syndrome.iss[24,14] == '01' then
        except.pavalid  = TRUE;
        except.paddress = fault.paddress;
    else
        except.pavalid  = FALSE;
        except.paddress = ARBITRARY : FullAddress;
    end;

    return except;
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_CheckPCAlignment

```
// AArch64_CheckPCAlignment()
// ==========================

func AArch64_CheckPCAlignment()
begin
    let pc : bits(64) = ThisInstrAddr{}();

    if pc[1:0] != '00' then
        AArch64_PCAlignmentFault();
    end;
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_DataAbort

```
// AArch64_DataAbort()
// ===================

func AArch64_DataAbort(fault : FaultRecord)
begin
    var target_el : bits(2);
    if IsExternalAbort(fault) then
        target_el = SyncExternalAbortTarget(fault);
    else
        let route_to_el2 = (EL2Enabled() && PSTATE.EL IN {EL0, EL1} &&
                            (HCR_EL2().TGE == '1' ||
                            (IsFeatureImplemented(FEAT_RME) && fault.gpcf.gpf != GPCF_None &&
                             HCR_EL2().GPF == '1') ||
                             (IsFeatureImplemented(FEAT_NV2) &&
                              fault.accessdesc.acctype == AccessType_NV2) ||
                             IsSecondStage(fault)));

        if PSTATE.EL == EL3 then
            target_el = EL3;
        elsif PSTATE.EL == EL2 || route_to_el2 then
            target_el = EL2;
        else
            target_el = EL1;
        end;
    end;

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let route_to_serr : boolean = (IsExternalAbort(fault) &&
                                   AArch64_RouteToSErrorOffset(target_el));
    let vect_offset : integer = if route_to_serr then 0x180 else 0x0;

    var except : ExceptionRecord;
    if IsFeatureImplemented(FEAT_NV2) && fault.accessdesc.acctype == AccessType_NV2 then
        except = AArch64_AbortSyndrome(Exception_NV2DataAbort, fault, target_el);
    else
        except = AArch64_AbortSyndrome(Exception_DataAbort, fault, target_el);
    end;
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_EffectiveTCF

```
// AArch64_EffectiveTCF()
// ======================
// Indicate if a Tag Check Fault should cause a synchronous exception,
// be asynchronously accumulated, or have no effect on the PE.

func AArch64_EffectiveTCF(el : bits(2), read : boolean) => TCFType
begin
    var tcf : bits(2);

    let regime : Regime = TranslationRegime(el);

    case regime of
        when Regime_EL3 =>  tcf = SCTLR_EL3().TCF;
        when Regime_EL2 =>  tcf = SCTLR_EL2().TCF;
        when Regime_EL20 => tcf = if el == EL0 then SCTLR_EL2().TCF0 else SCTLR_EL2().TCF;
        when Regime_EL10 => tcf = if el == EL0 then SCTLR_EL1().TCF0 else SCTLR_EL1().TCF;
        otherwise =>        unreachable;
    end;

    if tcf == '11' then        // Reserved value
        if !IsFeatureImplemented(FEAT_MTE_ASYM_FAULT) then
            (-,tcf) = ConstrainUnpredictableBits{2}(Unpredictable_RESTCF);
        end;
    end;

    case tcf of
        when '00' =>   // Tag Check Faults have no effect on the PE
            return TCFType_Ignore;
        when '01' =>   // Tag Check Faults cause a synchronous exception
            return TCFType_Sync;
        when '10' =>
            if IsFeatureImplemented(FEAT_MTE_ASYNC) then
                // If asynchronous faults are implemented,
                // Tag Check Faults are asynchronously accumulated
                return TCFType_Async;
            else
                // Otherwise, Tag Check Faults have no effect on the PE
                return TCFType_Ignore;
            end;
        when '11' =>
            if IsFeatureImplemented(FEAT_MTE_ASYM_FAULT) then
                // Tag Check Faults cause a synchronous exception on reads or on
                // a read/write access, and are asynchronously accumulated on writes
                if read then
                    return TCFType_Sync;
                else
                    return TCFType_Async;
                end;
            else
                // Otherwise, Tag Check Faults have no effect on the PE
                return TCFType_Ignore;
            end;
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_InstructionAbort

```
// AArch64_InstructionAbort()
// ==========================

func AArch64_InstructionAbort(fault : FaultRecord)
begin
    // External aborts on instruction fetch must be taken synchronously
    if IsFeatureImplemented(FEAT_DoubleFault) then
        assert fault.statuscode != Fault_AsyncExternal;
    end;

    var target_el : bits(2);
    if IsExternalAbort(fault) then
        target_el = SyncExternalAbortTarget(fault);
    else
        let route_to_el2 : boolean = (EL2Enabled() && PSTATE.EL IN {EL0, EL1} &&
                                      (HCR_EL2().TGE == '1' ||
                                      (IsFeatureImplemented(FEAT_RME) &&
                                       fault.gpcf.gpf != GPCF_None &&
                                       HCR_EL2().GPF == '1') ||
                                       IsSecondStage(fault)));

        if PSTATE.EL == EL3 then
            target_el = EL3;
        elsif PSTATE.EL == EL2 || route_to_el2 then
            target_el = EL2;
        else
            target_el = EL1;
        end;
    end;

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    var vect_offset : integer;

    if IsExternalAbort(fault) && AArch64_RouteToSErrorOffset(target_el) then
        vect_offset = 0x180;
    else
        vect_offset = 0x0;
    end;

    let except : ExceptionRecord = AArch64_AbortSyndrome(Exception_InstructionAbort, fault,
                                                         target_el);
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_PCAlignmentFault

```
// AArch64_PCAlignmentFault()
// ==========================
// Called on unaligned program counter in AArch64 state.

func AArch64_PCAlignmentFault()
begin

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_PCAlignment);
    except.vaddress = ThisInstrAddr{64}();
    var target_el : bits(2) = EL1;
    if UInt(PSTATE.EL) > UInt(EL1) then
        target_el = PSTATE.EL;
    elsif EL2Enabled() && HCR_EL2().TGE == '1' then
        target_el = EL2;
    end;
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_RaiseTagCheckFault

```
// AArch64_RaiseTagCheckFault()
// ============================
// Raise a Tag Check Fault exception.

func AArch64_RaiseTagCheckFault(fault : FaultRecord)
begin
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;
    var target_el : bits(2) = EL1;
    if UInt(PSTATE.EL) > UInt(EL1) then
        target_el = PSTATE.EL;
    elsif PSTATE.EL == EL0 && EL2Enabled() && HCR_EL2().TGE == '1' then
        target_el = EL2;
    end;

    let except : ExceptionRecord = AArch64_AbortSyndrome(Exception_DataAbort, fault, target_el);
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_ReportTagCheckFault

```
// AArch64_ReportTagCheckFault()
// =============================
// Records a Tag Check Fault exception into the appropriate TFSR_ELx.

func AArch64_ReportTagCheckFault(el : bits(2), ttbr : bit)
begin
    case el of
        when EL3 => assert ttbr == '0'; TFSR_EL3().TF0   = '1';
        when EL2 => if ttbr == '0' then TFSR_EL2().TF0   = '1'; else TFSR_EL2().TF1   = '1'; end;
        when EL1 => if ttbr == '0' then TFSR_EL1().TF0   = '1'; else TFSR_EL1().TF1   = '1'; end;
        when EL0 => if ttbr == '0' then TFSRE0_EL1().TF0 = '1'; else TFSRE0_EL1().TF1 = '1'; end;
    end;
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_RouteToSErrorOffset

```
// AArch64_RouteToSErrorOffset()
// =============================
// Returns TRUE if synchronous External abort exceptions are taken to the
// appropriate SError vector offset, and FALSE otherwise.

func AArch64_RouteToSErrorOffset(target_el : bits(2)) => boolean
begin
    if !IsFeatureImplemented(FEAT_DoubleFault) then return FALSE; end;

    var ease_bit : bit;
    case target_el of
        when EL3 =>
            ease_bit = SCR_EL3().EASE;
        when EL2 =>
            if IsFeatureImplemented(FEAT_DoubleFault2) && IsSCTLR2EL2Enabled() then
                ease_bit = SCTLR2_EL2().EASE;
            else
                ease_bit = '0';
            end;
        when EL1 =>
            if IsFeatureImplemented(FEAT_DoubleFault2) && IsSCTLR2EL1Enabled() then
                ease_bit = SCTLR2_EL1().EASE;
            else
                ease_bit = '0';
            end;
    end;
    return (ease_bit == '1');
end;
```

Library pseudocode for aarch64/exceptions/aborts/AArch64_SPAlignmentFault

```
// AArch64_SPAlignmentFault()
// ==========================
// Called on an unaligned stack pointer in AArch64 state.

func AArch64_SPAlignmentFault()
begin

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    let except : ExceptionRecord = ExceptionSyndrome(Exception_SPAlignment);

    var target_el : bits(2) = EL1;
    if UInt(PSTATE.EL) > UInt(EL1) then
        target_el = PSTATE.EL;
    elsif EL2Enabled() && HCR_EL2().TGE == '1' then
        target_el = EL2;
    end;
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/aborts/BranchTargetException

```
// BranchTargetException()
// =======================
// Generate branch target exception.

func AArch64_BranchTargetException(vaddress : bits(52))
begin
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_BranchTarget);
    except.syndrome.iss[1:0]   = PSTATE.BTYPE;
    except.syndrome.iss[24:2]  = Zeros{23};         // RES0

    var target_el : bits(2) = EL1;
    if UInt(PSTATE.EL) > UInt(EL1) then
        target_el = PSTATE.EL;
    elsif PSTATE.EL == EL0 && EL2Enabled() && HCR_EL2().TGE == '1' then
        target_el = EL2;
    end;
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/aborts/TCFType

```
// TCFType
// =======

type TCFType of enumeration { TCFType_Sync, TCFType_Async, TCFType_Ignore };
```

Library pseudocode for aarch64/exceptions/aborts/TakeGPCException

```
// TakeGPCException()
// ==================
// Report Granule Protection Exception faults

func TakeGPCException(fault : FaultRecord)
begin
    assert IsFeatureImplemented(FEAT_RME);

    var except : ExceptionRecord;

    except.exceptype = Exception_GPC;
    except.vaddress  = ZeroExtend{64}(fault.vaddress);
    except.paddress  = fault.paddress;
    except.pavalid   = TRUE;

    if IPAValid(fault) then
        except.ipavalid  = TRUE;
        except.NS        = if fault.ipaddress.paspace == PAS_NonSecure then '1' else '0';
        except.ipaddress = fault.ipaddress.address;
    else
        except.ipavalid  = FALSE;
    end;

    except.syndrome.iss2[11] = if fault.hdbssf then '1' else '0';   // HDBSSF
    if fault.accessdesc.acctype == AccessType_GCS then
        except.syndrome.iss2[8] = '1'; //GCS
    end;

    // Populate the fields grouped in ISS
    except.syndrome.iss[24:22] = Zeros{3}; // RES0
    except.syndrome.iss[21]    = if fault.gpcfs2walk then '1' else '0';  // S2PTW
    if fault.accessdesc.acctype == AccessType_IFETCH then
        except.syndrome.iss[20] = '1';     // InD
    else
        except.syndrome.iss[20] = '0';     // InD
    end;
    except.syndrome.iss[19:14] = EncodeGPCSC(fault.gpcf); // GPCSC
    if IsFeatureImplemented(FEAT_NV2) && fault.accessdesc.acctype == AccessType_NV2 then
        except.syndrome.iss[13] = '1';     // VNCR
    else
        except.syndrome.iss[13] = '0';     // VNCR
    end;
    except.syndrome.iss[12:11]  = '00';     // RES0
    except.syndrome.iss[10:9]   = '00';     // RES0

    if fault.accessdesc.acctype IN {AccessType_DC, AccessType_IC, AccessType_AT} then
        except.syndrome.iss[8] = '1'; // CM
    else
        except.syndrome.iss[8] = '0'; // CM
    end;

    except.syndrome.iss[7] = if fault.s2fs1walk then '1' else '0'; // S1PTW

    if fault.accessdesc.acctype IN {AccessType_DC, AccessType_IC, AccessType_AT} then
        except.syndrome.iss[6] = '1';                               // WnR
    elsif fault.statuscode IN {Fault_UnsupportedAtomicHWUpdate, Fault_Exclusive} then
        except.syndrome.iss[6] = ARBITRARY : bit;                   // WnR
    elsif fault.accessdesc.atomicop && IsExternalAbort(fault) then
        except.syndrome.iss[6] = ARBITRARY : bit;                   // WnR
    else
        except.syndrome.iss[6] = if fault.write then '1' else '0'; // WnR
    end;

    except.syndrome.iss[5:0] = EncodeLDFSC(fault.statuscode, fault.level); // xFSC

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let target_el : bits(2) = EL3;

    var vect_offset : integer;
    if IsExternalAbort(fault) && AArch64_RouteToSErrorOffset(target_el) then
        vect_offset = 0x180;
    else
        vect_offset = 0x0;
    end;

    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/async/AArch64_TakeDelegatedSErrorException

```
// AArch64_TakeDelegatedSErrorException()
// ======================================

func AArch64_TakeDelegatedSErrorException()
begin
    assert (IsFeatureImplemented(FEAT_E3DSE) && PSTATE.EL != EL3 &&
            EffectiveSCR_EL3_EnDSE() == '1' && SCR_EL3().DSE == '1');

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset = 0x180;
    var except : ExceptionRecord = ExceptionSyndrome(Exception_SError);

    var target_el : bits(2);
    var dsei_masked : boolean;
    (dsei_masked, target_el)  = AArch64_DelegatedSErrorTarget();
    assert !dsei_masked;
    except.syndrome.iss[24]   = VSESR_EL3().IDS;
    except.syndrome.iss[23:0] = VSESR_EL3().ISS;
    ClearPendingDelegatedSError();

    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/async/AArch64_TakePhysicalFIQException

```
// AArch64_TakePhysicalFIQException()
// ==================================

func AArch64_TakePhysicalFIQException()
begin

    let route_to_el3 : boolean = EffectiveSCR_EL3_FIQ() == '1';
    let route_to_el2 : boolean = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
                                  (HCR_EL2().TGE == '1' || HCR_EL2().FMO == '1'));
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x100;
    let except : ExceptionRecord = ExceptionSyndrome(Exception_FIQ);

    if route_to_el3 then
        AArch64_TakeException(EL3, except, preferred_exception_return, vect_offset);
    elsif PSTATE.EL == EL2 || route_to_el2 then
        assert PSTATE.EL != EL3;
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        assert PSTATE.EL IN {EL0, EL1};
        AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/async/AArch64_TakePhysicalIRQException

```
// AArch64_TakePhysicalIRQException()
// ==================================
// Take an enabled physical IRQ exception.

func AArch64_TakePhysicalIRQException()
begin

    let route_to_el3 : boolean = EffectiveSCR_EL3_IRQ() == '1';
    let route_to_el2 : boolean = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
                                  (HCR_EL2().TGE == '1' || HCR_EL2().IMO == '1'));
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x80;

    let except : ExceptionRecord = ExceptionSyndrome(Exception_IRQ);

    if route_to_el3 then
        AArch64_TakeException(EL3, except, preferred_exception_return, vect_offset);
    elsif PSTATE.EL == EL2 || route_to_el2 then
        assert PSTATE.EL != EL3;
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        assert PSTATE.EL IN {EL0, EL1};
        AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/async/AArch64_TakePhysicalSErrorException

```
// AArch64_TakePhysicalSErrorException()
// =====================================

func AArch64_TakePhysicalSErrorException(implicit_esb : boolean)
begin
    var masked : boolean;
    var target_el : bits(2);

    (masked, target_el) = PhysicalSErrorTarget();
    assert !masked;

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer    = 0x180;
    let fault : FaultRecord      = GetPendingPhysicalSError();
    var except : ExceptionRecord = ExceptionSyndrome(Exception_SError);
    let is_esb : boolean         = FALSE;
    let syndrome : bits(25)      = AArch64_PhysicalSErrorSyndrome(is_esb, implicit_esb);

    if IsSErrorEdgeTriggered() then
        ClearPendingPhysicalSError();
    end;

    if except.syndrome.iss[14] == '1' then
        except.pavalid  = TRUE;
        except.paddress = fault.paddress;
    else
        except.pavalid  = FALSE;
        except.paddress = ARBITRARY : FullAddress;
    end;
    except.syndrome.iss = syndrome;
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/async/AArch64_TakeVirtualFIQException

```
// AArch64_TakeVirtualFIQException()
// =================================

func AArch64_TakeVirtualFIQException()
begin
    assert PSTATE.EL IN {EL0, EL1} && EL2Enabled();
    // Virtual IRQ enabled if TGE==0 and FMO==1
    assert HCR_EL2().TGE == '0' && HCR_EL2().FMO == '1';

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x100;

    let except : ExceptionRecord = ExceptionSyndrome(Exception_FIQ);

    AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/async/AArch64_TakeVirtualIRQException

```
// AArch64_TakeVirtualIRQException()
// =================================

func AArch64_TakeVirtualIRQException()
begin
    assert PSTATE.EL IN {EL0, EL1} && EL2Enabled();
    // Virtual IRQ enabled if TGE==0 and IMO==1
    assert HCR_EL2().TGE == '0' && HCR_EL2().IMO == '1';

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x80;

    let except : ExceptionRecord = ExceptionSyndrome(Exception_IRQ);

    AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/async/AArch64_TakeVirtualSErrorException

```
// AArch64_TakeVirtualSErrorException()
// ====================================

func AArch64_TakeVirtualSErrorException()
begin

    assert PSTATE.EL IN {EL0, EL1} && EL2Enabled();
    // Virtual SError enabled if TGE==0 and AMO==1 or TMEA==1
    assert (HCR_EL2().TGE == '0' &&
            (HCR_EL2().AMO == '1' || (IsHCRXEL2Enabled() && HCRX_EL2().TMEA == '1')));

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer                 = 0x180;
    var except : ExceptionRecord              = ExceptionSyndrome(Exception_SError);

    if IsFeatureImplemented(FEAT_RAS) then
        except.syndrome.iss[24]   = VSESR_EL2().IDS;
        except.syndrome.iss[23:0] = VSESR_EL2().ISS;
    else
        let syndrome : bits(25)       = ImpDefBits{}("Virtual SError syndrome");
        let impdef_syndrome : boolean = syndrome[24] == '1';
        if impdef_syndrome then except.syndrome.iss = syndrome; end;
    end;

    ClearPendingVirtualSError();
    AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/debug/AArch64_BreakpointException

```
// AArch64_BreakpointException()
// =============================

func AArch64_BreakpointException(fault : FaultRecord)
begin
    assert PSTATE.EL != EL3;

    let route_to_el2 : boolean = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
                                  (HCR_EL2().TGE == '1' || MDCR_EL2().TDE == '1'));

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    var target_el : bits(2);
    let vect_offset : integer = 0x0;
    target_el = if (PSTATE.EL == EL2 || route_to_el2) then EL2 else EL1;

    let vaddress : bits(64)      = ARBITRARY : bits(64);
    let except : ExceptionRecord = AArch64_AbortSyndrome(Exception_Breakpoint, fault, target_el);
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/debug/AArch64_SoftwareBreakpoint

```
// AArch64_SoftwareBreakpoint()
// ============================

func AArch64_SoftwareBreakpoint(immediate : bits(16))
begin

    let route_to_el2 : boolean = (PSTATE.EL IN {EL0, EL1} &&
                                  EL2Enabled() && (HCR_EL2().TGE == '1' || MDCR_EL2().TDE == '1'));

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_SoftwareBreakpoint);
    except.syndrome.iss[15:0]    = immediate;

    if UInt(PSTATE.EL) > UInt(EL1) then
        AArch64_TakeException(PSTATE.EL, except, preferred_exception_return, vect_offset);
    elsif route_to_el2 then
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/debug/AArch64_SoftwareStepException

```
// AArch64_SoftwareStepException()
// ===============================

func AArch64_SoftwareStepException()
begin
    assert PSTATE.EL != EL3;

    let route_to_el2 : boolean = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
                                  (HCR_EL2().TGE == '1' || MDCR_EL2().TDE == '1'));

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_SoftwareStep);
    except.syndrome.iss[5:0] = '100010';                // IFSC = Debug Exception

    if PSTATE.EL == EL2 || route_to_el2 then
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/debug/AArch64_VectorCatchException

```
// AArch64_VectorCatchException()
// ==============================
// Vector Catch taken from EL0 or EL1 to EL2. This can only be called when debug exceptions are
// being routed to EL2, as Vector Catch is a legacy debug event.

func AArch64_VectorCatchException(fault : FaultRecord)
begin
    assert PSTATE.EL != EL2;
    assert EL2Enabled() && (HCR_EL2().TGE == '1' || MDCR_EL2().TDE == '1');

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    let vaddress : bits(64)      = ARBITRARY : bits(64);
    let except : ExceptionRecord = AArch64_AbortSyndrome(Exception_VectorCatch, fault, EL2);

    AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/debug/AArch64_WatchpointException

```
// AArch64_WatchpointException()
// =============================

func AArch64_WatchpointException(fault : FaultRecord)
begin
    assert PSTATE.EL != EL3;

    let route_to_el2 : boolean = (PSTATE.EL IN {EL0, EL1} && EL2Enabled() &&
                                  (HCR_EL2().TGE == '1' || MDCR_EL2().TDE == '1'));

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    var target_el : bits(2);
    let vect_offset : integer = 0x0;
    target_el = if (PSTATE.EL == EL2 || route_to_el2) then EL2 else EL1;

    var except : ExceptionRecord;
    if IsFeatureImplemented(FEAT_NV2) && fault.accessdesc.acctype == AccessType_NV2 then
        except = AArch64_AbortSyndrome(Exception_NV2Watchpoint, fault, target_el);
    else
        except = AArch64_AbortSyndrome(Exception_Watchpoint, fault, target_el);
    end;
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/exceptions/AArch64_ExceptionClass

```
// AArch64_ExceptionClass()
// ========================
// Returns the Exception Class and Instruction Length fields to be reported in ESR

func AArch64_ExceptionClass(exceptype : Exception, target_el : bits(2)) => (integer,bit)
begin

    var il_is_valid : boolean = TRUE;
    let from_32 : boolean = UsingAArch32();
    var ec : integer;
    case exceptype of
        when Exception_Uncategorized         =>  ec = 0x00; il_is_valid = FALSE;
        when Exception_WFxTrap               =>  ec = 0x01;
        when Exception_CP15RTTrap            =>  ec = 0x03; assert from_32;
        when Exception_CP15RRTTrap           =>  ec = 0x04; assert from_32;
        when Exception_CP14RTTrap            =>  ec = 0x05; assert from_32;
        when Exception_CP14DTTrap            =>  ec = 0x06; assert from_32;
        when Exception_AdvSIMDFPAccessTrap   =>  ec = 0x07;
        when Exception_FPIDTrap              =>  ec = 0x08;
        when Exception_PACTrap               =>  ec = 0x09;
        when Exception_OtherTrap             =>  ec = 0x0A;
        when Exception_GPC                   =>  ec = 0x1E;
        when Exception_ConditionalFault      =>  ec = 0x0B;
        when Exception_CP14RRTTrap           =>  ec = 0x0C; assert from_32;
        when Exception_BranchTarget          =>  ec = 0x0D;
        when Exception_IllegalState          =>  ec = 0x0E; il_is_valid = FALSE;
        when Exception_SupervisorCall        =>  ec = if from_32 then 0x11 else 0x15;
        when Exception_HypervisorCall        =>  ec = if from_32 then 0x12 else 0x16;
        when Exception_MonitorCall           =>  ec = if from_32 then 0x13 else 0x17;
        when Exception_SystemRegisterTrap    =>  ec = 0x18; assert !from_32;
        when Exception_SystemRegister128Trap =>  ec = 0x14; assert !from_32;
        when Exception_SVEAccessTrap         =>  ec = 0x19; assert !from_32;
        when Exception_ERetTrap              =>  ec = 0x1A; assert !from_32;
        when Exception_PACFail               =>  ec = 0x1C; assert !from_32;
        when Exception_SMEAccessTrap         =>  ec = 0x1D; assert !from_32;
        when Exception_InstructionAbort      =>
            ec = if target_el == PSTATE.EL then 0x21 else 0x20;
            il_is_valid = FALSE;
        when Exception_PCAlignment           =>  ec = 0x22; il_is_valid = FALSE;
        when Exception_DataAbort             =>  ec = if target_el == PSTATE.EL then 0x25 else 0x24;
        when Exception_NV2DataAbort          =>  ec = 0x25;
        when Exception_SPAlignment           =>  ec = 0x26; il_is_valid = FALSE; assert !from_32;
        when Exception_MemCpyMemSet          =>  ec = 0x27;
        when Exception_GCSFail               =>  ec = 0x2D; assert !from_32;
        when Exception_FPTrappedException    =>  ec = if from_32 then 0x28 else 0x2C;
        when Exception_SError                =>  ec = 0x2F; il_is_valid = FALSE;
        when Exception_Breakpoint =>
            ec = if target_el == PSTATE.EL then 0x31 else 0x30;
            il_is_valid = FALSE;
        when Exception_SoftwareStep =>
            ec = if target_el == PSTATE.EL then 0x33 else 0x32;
            il_is_valid = FALSE;
        when Exception_Watchpoint =>
            ec = if target_el == PSTATE.EL then 0x35 else 0x34;
            il_is_valid = FALSE;
        when Exception_NV2Watchpoint         =>  ec = 0x35; il_is_valid = FALSE;
        when Exception_SoftwareBreakpoint    =>  ec = if from_32 then 0x38 else 0x3C;
        when Exception_VectorCatch           =>  ec = 0x3A; il_is_valid = FALSE; assert from_32;
        when Exception_Profiling             =>  ec = 0x3D;
        otherwise                            =>  unreachable;
    end;

    var il : bit;
    if il_is_valid then
        il = if ThisInstrLength() == 32 then '1' else '0';
    else
        il = '1';
    end;
    assert from_32 || il == '1';            // AArch64 instructions always 32-bit

    return (ec,il);
end;
```

Library pseudocode for aarch64/exceptions/exceptions/AArch64_ReportException

```
// AArch64_ReportException()
// =========================
// Report syndrome information for exception taken to AArch64 state.

func AArch64_ReportException(except : ExceptionRecord, target_el : bits(2))
begin

    let exceptype : Exception = except.exceptype;

    var (ec,il) = AArch64_ExceptionClass(exceptype, target_el);
    let iss : bits(25)  = except.syndrome.iss;
    let iss2 : bits(24) = except.syndrome.iss2;

    // IL is not valid for Data Abort exceptions without valid instruction syndrome information
    if ec IN {0x24,0x25} && iss[24] == '0' then
        il = '1';
    end;

    ESR_EL(target_el) = (Zeros{8}  ::   // [63:56]
                         iss2      ::   // [55:32]
                         ec[5:0]   ::   // [31:26]
                         il        ::   // [25]
                         iss);          // [24:0]

    if exceptype IN {
        Exception_InstructionAbort,
        Exception_PCAlignment,
        Exception_DataAbort,
        Exception_NV2DataAbort,
        Exception_NV2Watchpoint,
        Exception_GPC,
        Exception_Watchpoint
    } then
        FAR_EL(target_el) = except.vaddress;
    else
        FAR_EL(target_el) = ARBITRARY : bits(64);
    end;

    if except.ipavalid then
        HPFAR_EL2()[NUM_PABITS-9:4] = except.ipaddress[NUM_PABITS-1:12];
        if IsSecureEL2Enabled() && CurrentSecurityState() == SS_Secure then
            HPFAR_EL2().NS = except.NS;
        else
            HPFAR_EL2().NS = '0';
        end;
    elsif target_el == EL2 then
        HPFAR_EL2()[NUM_PABITS-9:4] = ARBITRARY : bits(NUM_PABITS-12);
    end;

    if except.pavalid then
        var faultaddr : bits(64) = ZeroExtend{}(except.paddress.address);
        if IsFeatureImplemented(FEAT_RME) then
            case except.paddress.paspace of
                when PAS_Secure =>     faultaddr[63:62] = '00';
                when PAS_NonSecure =>  faultaddr[63:62] = '10';
                when PAS_Root =>       faultaddr[63:62] = '01';
                when PAS_Realm =>      faultaddr[63:62] = '11';
            end;
            if exceptype == Exception_GPC then
                faultaddr[11:0] = Zeros{12};
            end;
        else
            faultaddr[63] = if except.paddress.paspace == PAS_NonSecure then '1' else '0';
        end;
        PFAR_EL(target_el) = faultaddr;
    elsif (IsFeatureImplemented(FEAT_PFAR) ||
           (IsFeatureImplemented(FEAT_RME) && target_el == EL3)) then
        PFAR_EL(target_el) = ARBITRARY : bits(64);
    end;
    return;
end;
```

Library pseudocode for aarch64/exceptions/exceptions/AArch64_ResetControlRegisters

```
// AArch64_ResetControlRegisters()
// ===============================
// Resets System registers and memory-mapped control registers that have architecturally-defined
// reset values to those values.

impdef func AArch64_ResetControlRegisters(cold_reset : boolean)
begin
    return;
end;
```

Library pseudocode for aarch64/exceptions/exceptions/AArch64_TakeReset

```
// AArch64_TakeReset()
// ===================
// Reset into AArch64 state

func AArch64_TakeReset(cold_reset : boolean)
begin
    assert HaveAArch64();

    // Enter the highest implemented Exception level in AArch64 state
    PSTATE.nRW = '0';
    if HaveEL(EL3) then
        PSTATE.EL = EL3;
    elsif HaveEL(EL2) then
        PSTATE.EL = EL2;
    else
        PSTATE.EL = EL1;
    end;

    // Reset System registers and other system components
    AArch64_ResetControlRegisters(cold_reset);

    // Reset all other PSTATE fields
    PSTATE.SP = '1';                    // Select stack pointer
    PSTATE.[D,A,I,F]  = '1111';         // All asynchronous exceptions masked
    PSTATE.SS = '0';                    // Clear software step bit
    PSTATE.DIT = '0';                   // PSTATE.DIT is reset to 0 when resetting into AArch64
    if IsFeatureImplemented(FEAT_PAuth_LR) then
        PSTATE.PACM = '0';              // PAC modifier
    end;
    if IsFeatureImplemented(FEAT_SME) then
        PSTATE.[SM,ZA] = '00';          // Disable Streaming SVE mode & ZA storage
        ResetSMEState('0');
    end;
    if IsFeatureImplemented(FEAT_SSBS) then
        PSTATE.SSBS = ImpDefBit("PSTATE.SSBS bit at reset");
    end;
    if IsFeatureImplemented(FEAT_GCS) then
        PSTATE.EXLOCK = '0';            // PSTATE.EXLOCK is reset to 0 when resetting into AArch64
    end;
    if IsFeatureImplemented(FEAT_UINJ) then
        PSTATE.UINJ = '0';              // PSTATE.UINJ is reset to 0 when resetting into AArch64
    end;
    PSTATE.IL = '0';                    // Clear Illegal Execution state bit

    // All registers, bits and fields not reset by the above pseudocode or by the BranchTo() call
    // below are UNKNOWN bitstrings after reset. In particular, the return information registers
    // ELR_ELx and SPSR_ELx have UNKNOWN values, so that it
    // is impossible to return from a reset in an architecturally defined way.
    AArch64_ResetGeneralRegisters();
    if IsFeatureImplemented(FEAT_SME) || IsFeatureImplemented(FEAT_SVE) then
        ResetSVERegisters();
    else
        AArch64_ResetSIMDFPRegisters();
    end;
    AArch64_ResetSpecialRegisters();
    ResetExternalDebugRegisters(cold_reset);

    var rv : bits(64);                      // IMPLEMENTATION DEFINED reset vector

    if HaveEL(EL3) then
        rv = RVBAR_EL3();
    elsif HaveEL(EL2) then
        rv = RVBAR_EL2();
    else
        rv = RVBAR_EL1();
    end;

    // The reset vector must be correctly aligned
    let pamax : AddressSize = AArch64_PAMax();
    assert IsZero(rv[63:pamax]) && IsZero(rv[1:0]);

    let branch_conditional : boolean = FALSE;
    EDPRSR().R = '0';                   // Leaving Reset State.
    BranchTo{64}(rv, BranchType_RESET, branch_conditional);
end;
```

Library pseudocode for aarch64/exceptions/exceptions/GetESR_ELx_DFSC

```
// GetESR_ELx_DFSC()
// =================
// Query the ESR_ELx to get the DFSC field.

func GetESR_ELx_DFSC(el : bits(2)) =>  bits(6)
begin
    let ec : bits(6) = ESR_EL(el).EC;
    assert ec IN {'10010x', '101111', '11010x'};

    return ESR_EL(el)[5:0];
end;
```

Library pseudocode for aarch64/exceptions/exceptions/GetESR_ELx_ExType

```
// GetESR_ELx_ExType()
// ===================
// Query the ESR_ELx to get the ExType field.

func GetESR_ELx_ExType(el : bits(2)) =>  bits(4)
begin
    let ec : bits(6) = ESR_EL(el).EC;
    assert ec == '101101';

    return ESR_EL(el)[23:20];
end;
```

Library pseudocode for aarch64/exceptions/exceptions/GetESR_ELx_IFSC

```
// GetESR_ELx_IFSC()
// =================
// Query the ESR_ELx to get the IFSC field.

func GetESR_ELx_IFSC(el : bits(2)) =>  bits(6)
begin
    let ec : bits(6) = ESR_EL(el).EC;
    assert ec IN {'1100xx', '10000x'};

    return ESR_EL(el)[5:0];
end;
```

Library pseudocode for aarch64/exceptions/ieeefp/AArch64_FPTrappedException

```
// AArch64_FPTrappedException()
// ============================

func AArch64_FPTrappedException(is_ase : boolean, accumulated_exceptions : bits(8))
begin
    var except : ExceptionRecord = ExceptionSyndrome(Exception_FPTrappedException);
    if is_ase then
        if ImpDefBool("vector instructions set TFV to 1") then
            except.syndrome.iss[23] = '1';                          // TFV
        else
            except.syndrome.iss[23] = '0';                          // TFV
        end;
    else
        except.syndrome.iss[23]     = '1';                          // TFV
    end;
    except.syndrome.iss[10:8] = ARBITRARY : bits(3);                // VECITR
    if except.syndrome.iss[23] == '1' then
        except.syndrome.iss[7,4:0] = accumulated_exceptions[7,4:0]; // IDF, IXF, UFF, OFF, DZF, IOF
    else
        except.syndrome.iss[7,4:0] = ARBITRARY : bits(6);
    end;

    let route_to_el2 : boolean = PSTATE.EL == EL0 && EL2Enabled() && HCR_EL2().TGE == '1';

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    if UInt(PSTATE.EL) > UInt(EL1) then
        AArch64_TakeException(PSTATE.EL, except, preferred_exception_return, vect_offset);
    elsif route_to_el2 then
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/syscalls/AArch64_CallHypervisor

```
// AArch64_CallHypervisor()
// ========================
// Performs a HVC call

func AArch64_CallHypervisor(immediate : bits(16))
begin
    assert HaveEL(EL2);

    if UsingAArch32() then AArch32_ITAdvance(); end;
    SSAdvance();
    let preferred_exception_return : bits(64) = NextInstrAddr{}();
    let vect_offset : integer{} = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_HypervisorCall);
    except.syndrome.iss[15:0] = immediate;

    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    if PSTATE.EL == EL3 then
        AArch64_TakeException(EL3, except, preferred_exception_return, vect_offset);
    else
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/syscalls/AArch64_CallSecureMonitor

```
// AArch64_CallSecureMonitor()
// ===========================

func AArch64_CallSecureMonitor(immediate : bits(16))
begin
    assert HaveEL(EL3) && !ELUsingAArch32(EL3);
    if UsingAArch32() then AArch32_ITAdvance(); end;
    HSAdvance();
    SSAdvance();
    let preferred_exception_return : bits(64) = NextInstrAddr{}();
    let vect_offset : integer{} = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_MonitorCall);
    except.syndrome.iss[15:0] = immediate;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    AArch64_TakeException(EL3, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/syscalls/AArch64_CallSupervisor

```
// AArch64_CallSupervisor()
// ========================
// Calls the Supervisor

func AArch64_CallSupervisor(immediate : bits(16))
begin
    if UsingAArch32() then AArch32_ITAdvance(); end;
    SSAdvance();
    let route_to_el2 : boolean = PSTATE.EL == EL0 && EL2Enabled() && HCR_EL2().TGE == '1';

    let preferred_exception_return : bits(64) = NextInstrAddr{}();
    let vect_offset : integer{} = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_SupervisorCall);
    except.syndrome.iss[15:0] = immediate;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    if UInt(PSTATE.EL) > UInt(EL1) then
        AArch64_TakeException(PSTATE.EL, except, preferred_exception_return, vect_offset);
    elsif route_to_el2 then
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/takeexception/AArch64_TakeException

```
// AArch64_TakeException()
// =======================
// Take an exception to an Exception level using AArch64.

noreturn func AArch64_TakeException(target_el : bits(2), exception_in : ExceptionRecord,
                                    preferred_exception_return : bits(64), vect_offset_in : integer)
begin
    assert HaveEL(target_el) && !ELUsingAArch32(target_el) && UInt(target_el) >= UInt(PSTATE.EL);
    if Halted() then
        AArch64_TakeExceptionInDebugState(target_el, exception_in);
    end;
    var except : ExceptionRecord = exception_in;
    var sync_errors : boolean;
    var iesb_req : boolean;
    if IsFeatureImplemented(FEAT_IESB) then
        sync_errors = EffectiveIESB(target_el) == '1' && except.exceptype != Exception_SError;
        if sync_errors && InsertIESBBeforeException(except.exceptype, target_el) then
            SynchronizeErrors();
            iesb_req = FALSE;
            sync_errors = FALSE;
            TakeUnmaskedPhysicalSErrorInterrupts(iesb_req);
        end;
    else
        sync_errors = FALSE;
    end;

    var brbe_source_allowed : boolean = FALSE;
    var brbe_source_address : bits(64) = Zeros{};
    if IsFeatureImplemented(FEAT_BRBE) then
        brbe_source_allowed = BranchRecordAllowed(PSTATE.EL);
        brbe_source_address = preferred_exception_return;
    end;

    if !IsFeatureImplemented(FEAT_ExS) || SCTLR_EL(target_el).EIS == '1' then
        // Synchronize the context, including Instruction Fetch Barrier effect
        SynchronizeContext();
    elsif !(except.exceptype == Exception_SoftwareBreakpoint ||
            (except.exceptype IN {Exception_SupervisorCall,
                                  Exception_HypervisorCall,
                                  Exception_MonitorCall} &&
             !except.trappedsyscallinst)) then
        InstructionFetchBarrier();
    end;

    // If coming from AArch32 state, the top parts of the X() registers might be set to zero
    let from_32 : boolean = UsingAArch32();
    if from_32 then AArch64_MaybeZeroRegisterUppers(); end;
    if from_32 && IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then
        ResetSVEState();
    else
        MaybeZeroSVEUppers(target_el);
    end;

    var vect_offset : integer = vect_offset_in;
    if UInt(target_el) > UInt(PSTATE.EL) then
        var lower_32 : boolean;
        if target_el == EL3 then
            if EL2Enabled() then
                lower_32 = ELUsingAArch32(EL2);
            else
                lower_32 = ELUsingAArch32(EL1);
            end;
        elsif IsInHost() && PSTATE.EL == EL0 && target_el == EL2 then
            lower_32 = ELUsingAArch32(EL0);
        else
            lower_32 = ELUsingAArch32(target_el - 1);
        end;
        vect_offset = vect_offset + (if lower_32 then 0x600 else 0x400);

    elsif PSTATE.SP == '1' then
        vect_offset = vect_offset + 0x200;
    end;

    assert vect_offset < 2048;
    let vect_base : bits(64) = VBAR_EL(target_el)[63:11]::Zeros{11};
    let target_vector : bits(64) = vect_base + vect_offset;

    var spsr : bits(64) = GetPSRFromPSTATE{}(AArch64_NonDebugState);

    if PSTATE.EL == EL1 && target_el == EL1 && EL2Enabled() then
        if EffectiveHCR_EL2_NVx() IN {'x01', '111'} then
            spsr[3:2] = '10';
        end;
    end;

    if IsFeatureImplemented(FEAT_BTI) && !UsingAArch32() then
        var zero_btype : boolean;
        // SPSR_ELx[].BTYPE is only guaranteed valid for these exception types
        if except.exceptype IN {Exception_SError, Exception_IRQ, Exception_FIQ,
```

```
                              Exception_SoftwareStep, Exception_PCAlignment,
                              Exception_InstructionAbort, Exception_Breakpoint,
                              Exception_VectorCatch, Exception_SoftwareBreakpoint,
                              Exception_IllegalState, Exception_BranchTarget} then
            zero_btype = FALSE;
        else
            zero_btype = ConstrainUnpredictableBool(Unpredictable_ZEROBTYPE);
        end;
        if zero_btype then spsr[11:10] = '00'; end;
    end;

    if (IsFeatureImplemented(FEAT_NV2) &&
          except.exceptype == Exception_NV2DataAbort && target_el == EL3) then
        // External aborts are configured to be taken to EL3
        except.exceptype = Exception_DataAbort;
    end;
    if ! except.exceptype IN {Exception_IRQ, Exception_FIQ} then
        AArch64_ReportException(except, target_el);
    end;

    if IsFeatureImplemented(FEAT_BRBE) then
        BRBEException(except, brbe_source_allowed, brbe_source_address,
                      target_vector, target_el, except.trappedsyscallinst);
    end;

    if IsFeatureImplemented(FEAT_GCS) then
        if PSTATE.EL == target_el then
            if GetCurrentEXLOCKEN() then
                PSTATE.EXLOCK = '1';
            else
                PSTATE.EXLOCK = '0';
            end;
        else
            PSTATE.EXLOCK = '0';
        end;
    end;

    PSTATE.EL = target_el;
    PSTATE.nRW = '0';
    PSTATE.SP = '1';

    SPSR_ELx() = spsr;
    ELR_ELx() = preferred_exception_return;
    PSTATE.SS = '0';
    if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = SCTLR_ELx().DSSBS; end;
    if (IsFeatureImplemented(FEAT_EBEP) || IsFeatureImplemented(FEAT_SPE_EXC) ||
          IsFeatureImplemented(FEAT_TRBE_EXC)) then
        PSTATE.PM = '1';
    end;
    if IsFeatureImplemented(FEAT_NMI) then
        PSTATE.ALLINT = NOT SCTLR_ELx().SPINTMASK;
    end;
    PSTATE.[D,A,I,F] = '1111';
    if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = '1'; end;
    PSTATE.IL = '0';
    if IsFeatureImplemented(FEAT_UAO) then PSTATE.UAO = '0'; end;
    if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    if IsFeatureImplemented(FEAT_PAN) && SCTLR_ELx().SPAN == '0' then
        if PSTATE.EL == EL1 then
            PSTATE.PAN = '1';
        elsif PSTATE.EL == EL2 && ELIsInHost(EL0) then
            PSTATE.PAN = '1';
        elsif (PSTATE.EL == EL2 && ELIsInHost(EL2) &&
                (IsFeatureImplemented(FEAT_NV3) ||
                 ImpDefBool("Set PAN as 1 when E2H,TGE is '10'"))) then
            PSTATE.PAN = '1';
        end;
    end;
    if from_32 then                             // Coming from AArch32
        PSTATE.IT = '00000000';
        PSTATE.T = '0';                         // PSTATE.J is RES0
    end;
    if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = '00'; end;
    let branch_conditional : boolean = FALSE;
    BranchTo{64}(target_vector, BranchType_EXCEPTION, branch_conditional);
    CheckExceptionCatch(TRUE);                  // Check for debug event on exception entry

    if sync_errors then
        SynchronizeErrors();
        iesb_req = TRUE;
        TakeUnmaskedPhysicalSErrorInterrupts(iesb_req);
    end;

    EndOfInstruction(EndOfInstructionReason_Exception);
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_AArch32SystemAccessTrap

```
// AArch64_AArch32SystemAccessTrap()
// =================================
// Trapped AArch32 System register access.

func AArch64_AArch32SystemAccessTrap(target_el : bits(2), ec : integer)
begin
    assert HaveEL(target_el) && target_el != EL0 && UInt(target_el) >= UInt(PSTATE.EL);

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    let except : ExceptionRecord = AArch64_AArch32SystemAccessTrapSyndrome(ThisInstr(), ec);
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_AArch32SystemAccessTrapSyndrome

```
// AArch64_AArch32SystemAccessTrapSyndrome()
// =========================================
// Returns the syndrome information for traps on AArch32 MCR, MCRR, MRC, MRRC, and VMRS,
// VMSR instructions, other than traps that are due to HCPTR or CPACR.

func AArch64_AArch32SystemAccessTrapSyndrome(instr : bits(32), ec : integer) => ExceptionRecord
begin
    var except : ExceptionRecord;

    case ec of
        when 0x0 =>    except = ExceptionSyndrome(Exception_Uncategorized);
        when 0x3 =>    except = ExceptionSyndrome(Exception_CP15RTTrap);
        when 0x4 =>    except = ExceptionSyndrome(Exception_CP15RRTTrap);
        when 0x5 =>    except = ExceptionSyndrome(Exception_CP14RTTrap);
        when 0x6 =>    except = ExceptionSyndrome(Exception_CP14DTTrap);
        when 0x7 =>    except = ExceptionSyndrome(Exception_AdvSIMDFPAccessTrap);
        when 0x8 =>    except = ExceptionSyndrome(Exception_FPIDTrap);
        when 0xC =>    except = ExceptionSyndrome(Exception_CP14RRTTrap);
        otherwise =>   unreachable;
    end;

    var iss : bits(20) = Zeros{};

    if except.exceptype == Exception_Uncategorized then
        return except;
    elsif except.exceptype IN {Exception_FPIDTrap, Exception_CP14RTTrap,
                               Exception_CP15RTTrap} then
        // Trapped MRC/MCR, VMRS on FPSID
        if except.exceptype != Exception_FPIDTrap then    // When trap is not for VMRS
            iss[19:17] = instr[7:5];           // opc2
            iss[16:14] = instr[23:21];         // opc1
            iss[13:10] = instr[19:16];         // CRn
            iss[4:1]   = instr[3:0];           // CRm
        else
            iss[19:17] = '000';
            iss[16:14] = '111';
            iss[13:10] = instr[19:16];         // reg
            iss[4:1]   = '0000';
        end;

        if instr[20] == '1' && instr[15:12] == '1111' then    // MRC, Rt==15
            iss[9:5] = '11111';
        elsif instr[20] == '0' && instr[15:12] == '1111' then // MCR, Rt==15
            iss[9:5] = ARBITRARY : bits(5);
        else
            iss[9:5] = LookUpRIndex(UInt(instr[15:12]), PSTATE.M)[4:0];
        end;
    elsif except.exceptype IN {Exception_CP14RRTTrap, Exception_AdvSIMDFPAccessTrap,
                               Exception_CP15RRTTrap} then
        // Trapped MRRC/MCRR, VMRS/VMSR
        iss[19:16] = instr[7:4];           // opc1
        if instr[19:16] == '1111' then     // Rt2==15
            iss[14:10] = ARBITRARY : bits(5);
        else
            iss[14:10] = LookUpRIndex(UInt(instr[19:16]), PSTATE.M)[4:0];
        end;

        if instr[15:12] == '1111' then     // Rt==15
            iss[9:5] = ARBITRARY : bits(5);
        else
            iss[9:5] = LookUpRIndex(UInt(instr[15:12]), PSTATE.M)[4:0];
        end;
        iss[4:1]   = instr[3:0];           // CRm
    elsif except.exceptype == Exception_CP14DTTrap then
        // Trapped LDC/STC
        iss[19:12] = instr[7:0];           // imm8
        iss[4]     = instr[23];            // U
        iss[2:1]   = instr[24,21];         // P,W
        if instr[19:16] == '1111' then     // Rn==15, LDC(Literal addressing)/STC
            iss[9:5] = ARBITRARY : bits(5);
            iss[3]   = '1';
        end;
    end;
    iss[0] = instr[20];                    // Direction

    except.syndrome.iss[24:20] = ConditionSyndrome();
    except.syndrome.iss[19:0]  = iss;

    return except;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_AArch32SystemAccessTraptoEL1orEL2

```
// AArch64_AArch32SystemAccessTraptoEL1orEL2()
// ===========================================
// Trapped access to AArch32 System register or system instruction from EL0.
// The trap is taken to EL1, or to EL2 if HCR_EL2().TGE is 1.

func AArch64_AArch32SystemAccessTraptoEL1orEL2(ec : integer)
begin
    assert PSTATE.EL == EL0;
    let target_el : bits(2) = if EL2Enabled() && HCR_EL2().TGE == '1' then EL2 else EL1;

    AArch64_AArch32SystemAccessTrap(target_el, ec);
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_AdvSIMDFPAccessTrap

```
// AArch64_AdvSIMDFPAccessTrap()
// =============================
// Trapped access to Advanced SIMD or FP registers due to CPACR.

func AArch64_AdvSIMDFPAccessTrap(target_el : bits(2))
begin
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    var except : ExceptionRecord;
    let vect_offset : integer = 0x0;

    let route_to_el2 : boolean = (target_el == EL1 && EL2Enabled() && HCR_EL2().TGE == '1');

    if route_to_el2 then
        except = ExceptionSyndrome(Exception_Uncategorized);
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        except = ExceptionSyndrome(Exception_AdvSIMDFPAccessTrap);
        except.syndrome.iss[24:20] = ConditionSyndrome();
        AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
    end;

    return;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_CheckCP15InstrCoarseTraps

```
// AArch64_CheckCP15InstrCoarseTraps()
// ===================================
// Check for coarse-grained AArch32 traps to System registers in the
// coproc=0b1111 encoding space by HSTR_EL2, HCR_EL2, and SCTLR_ELx.

func AArch64_CheckCP15InstrCoarseTraps(CRn : integer, nreg : integer, CRm : integer)
begin
    let trapped_encoding : boolean = ((CRn == 9  && CRm IN {0,1,2,    5,6,7,8   }) ||
                                      (CRn == 10 && CRm IN {0,1,    4,      8   }) ||
                                      (CRn == 11 && CRm IN {0,1,2,3,4,5,6,7,8,15}));

    // Check for MRC and MCR disabled by SCTLR_EL1.TIDCP.
    if (IsFeatureImplemented(FEAT_TIDCP1) && PSTATE.EL == EL0 && !IsInHost() &&
            !ELUsingAArch32(EL1) && SCTLR_EL1().TIDCP == '1' && trapped_encoding) then
        if EL2Enabled() && HCR_EL2().TGE == '1' then
            AArch64_AArch32SystemAccessTrap(EL2, 0x3);
        else
            AArch64_AArch32SystemAccessTrap(EL1, 0x3);
        end;
    end;

    // Check for coarse-grained Hyp traps
    if PSTATE.EL IN {EL0, EL1} && EL2Enabled() then
        // Check for MRC and MCR disabled by SCTLR_EL2.TIDCP.
        if (IsFeatureImplemented(FEAT_TIDCP1) && PSTATE.EL == EL0 && IsInHost() &&
              SCTLR_EL2().TIDCP == '1' && trapped_encoding) then
            AArch64_AArch32SystemAccessTrap(EL2, 0x3);
        end;

        let major : integer = if nreg == 1 then CRn else CRm;
        // Check for MCR, MRC, MCRR, and MRRC disabled by HSTR_EL2().CRn/HSTR().CRm
        // and MRC and MCR disabled by HCR_EL2.TIDCP.
        if ((!IsInHost() && ! major IN {4,14} && HSTR_EL2()[major] == '1') ||
                (HCR_EL2().TIDCP == '1' && nreg == 1 && trapped_encoding)) then
            if (PSTATE.EL == EL0 &&
                    ImpDefBool("UNDEF unallocated CP15 access at EL0")) then
                Undefined();
            end;
            AArch64_AArch32SystemAccessTrap(EL2, 0x3);
        end;
    end;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_CheckFPAdvSIMDEnabled

```
// AArch64_CheckFPAdvSIMDEnabled()
// ===============================

func AArch64_CheckFPAdvSIMDEnabled()
begin
    AArch64_CheckFPEnabled();
    // Check for illegal use of Advanced
    // SIMD in Streaming SVE Mode
    if IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' && !IsFullA64Enabled() then
        SMEAccessTrap(SMEExceptionType_Streaming, PSTATE.EL);
    end;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_CheckFPAdvSIMDTrap

```
// AArch64_CheckFPAdvSIMDTrap()
// ============================
// Check against CPTR_EL2 and CPTR_EL3.

func AArch64_CheckFPAdvSIMDTrap()
begin
    if HaveEL(EL3) && CPTR_EL3().TFP == '1' && EL3SDDUndefPriority() then
        Undefined();
    end;

    if PSTATE.EL IN {EL0, EL1, EL2} && EL2Enabled() then
        // Check if access disabled in CPTR_EL2
        if ELIsInHost(EL2) then
            var disabled : boolean;
            case CPTR_EL2().FPEN of
                when 'x0' => disabled = TRUE;
                when '01' => disabled = PSTATE.EL == EL0 && HCR_EL2().TGE == '1';
                when '11' => disabled = FALSE;
            end;
            if disabled then AArch64_AdvSIMDFPAccessTrap(EL2); end;
        else
            if CPTR_EL2().TFP == '1' then AArch64_AdvSIMDFPAccessTrap(EL2); end;
        end;
    end;

    if HaveEL(EL3) then
        // Check if access disabled in CPTR_EL3
        if CPTR_EL3().TFP == '1' then
            if EL3SDDUndef() then
                Undefined();
            else
                AArch64_AdvSIMDFPAccessTrap(EL3);
            end;
        end;
    end;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_CheckFPEnabled

```
// AArch64_CheckFPEnabled()
// ========================
// Check against CPACR

func AArch64_CheckFPEnabled()
begin
    if PSTATE.EL IN {EL0, EL1} && !IsInHost() then
        // Check if access disabled in CPACR_EL1
        var disabled : boolean;
        case CPACR_EL1().FPEN of
            when 'x0' => disabled = TRUE;
            when '01' => disabled = PSTATE.EL == EL0;
            when '11' => disabled = FALSE;
        end;
        if disabled then AArch64_AdvSIMDFPAccessTrap(EL1); end;
    end;

    AArch64_CheckFPAdvSIMDTrap();               // Also check against CPTR_EL2 and CPTR_EL3
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_CheckForERetTrap

```
// AArch64_CheckForERetTrap()
// ==========================
// Check for trap on ERET, ERETAA, ERETAB instruction

func AArch64_CheckForERetTrap(eret_with_pac : boolean, pac_uses_key_a : boolean)
begin
    if PSTATE.EL == EL1 && EL2Enabled() then
        var route_to_el2 : boolean = FALSE;

        // Check for a fine-grained trap by the hypervisor.
        if (IsFeatureImplemented(FEAT_FGT) && (!HaveEL(EL3) || SCR_EL3().FGTEn == '1') &&
              HFGITR_EL2().ERET == '1') then
            route_to_el2 = TRUE;
        // Check for a trap by the Effective value of the HCR_EL2.NV bit
        elsif EffectiveHCR_EL2_NVx()[0] == '1' then
            // Check for a trap by NVHCR_EL2.TGE based on the SPSR_EL1().M[3:2] value
            if EffectiveHCRX_EL2_NVTGE() == '1' then
                route_to_el2 = NVHCR_EL2().TGE == '0' && SPSR_EL1()[3:2] == '0x';
            else
                route_to_el2 = TRUE;
            end;
        end;

        if route_to_el2 then
            let preferred_exception_return : bits(64) = ThisInstrAddr{}();
            let vect_offset : integer = 0x0;
            var except : ExceptionRecord = ExceptionSyndrome(Exception_ERetTrap);

            if !eret_with_pac then                              // ERET
                except.syndrome.iss[1] = '0';
                except.syndrome.iss[0] = '0';                   // RES0
            else
                except.syndrome.iss[1] = '1';
                except.syndrome.iss[0] = if pac_uses_key_a then '0' else '1';
            end;

            AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
        end;
    end;

    return;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_CheckForSMCUndefOrTrap

```
// AArch64_CheckForSMCUndefOrTrap()
// ================================
// Check for UNDEFINED or trap on SMC instruction

func AArch64_CheckForSMCUndefOrTrap(imm : bits(16))
begin
    if PSTATE.EL == EL0 then Undefined(); end;
    if (!(PSTATE.EL == EL1 && EL2Enabled() && HCR_EL2().TSC == '1') &&
        HaveEL(EL3) && SCR_EL3().SMD == '1') then
        Undefined();
    end;
    var route_to_el2 : boolean = FALSE;
    if !HaveEL(EL3) then
        if (PSTATE.EL == EL1 && EL2Enabled() && HCR_EL2().TSC == '1' &&
              (EffectiveHCR_EL2_NVx() == 'xx1' ||
              (ImpDefBool("Trap SMC execution at EL1 to EL2")))) then
            route_to_el2 = TRUE;
        else
            Undefined();
        end;
    else
        route_to_el2 = PSTATE.EL == EL1 && EL2Enabled() && HCR_EL2().TSC == '1';
    end;
    if route_to_el2 then
        let preferred_exception_return : bits(64) = ThisInstrAddr{}();
        let vect_offset : integer = 0x0;
        var except : ExceptionRecord = ExceptionSyndrome(Exception_MonitorCall);
        except.syndrome.iss[15:0] = imm;
        except.trappedsyscallinst = TRUE;
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_CheckForSVCTrap

```
// AArch64_CheckForSVCTrap()
// =========================
// Check for trap on SVC instruction

func AArch64_CheckForSVCTrap(immediate : bits(16))
begin
    if IsFeatureImplemented(FEAT_FGT) then
        var route_to_el2 : boolean = FALSE;
        if PSTATE.EL == EL0 then
            route_to_el2 = (!UsingAArch32() && !ELUsingAArch32(EL1) &&
                            EL2Enabled() && HFGITR_EL2().SVC_EL0 == '1' &&
                            (!IsInHost() && (!HaveEL(EL3) || SCR_EL3().FGTEn == '1')));

        elsif PSTATE.EL == EL1 then
            route_to_el2 = (EL2Enabled() && HFGITR_EL2().SVC_EL1 == '1' &&
                            (!HaveEL(EL3) || SCR_EL3().FGTEn == '1'));
        end;

        if route_to_el2 then
            var except : ExceptionRecord = ExceptionSyndrome(Exception_SupervisorCall);
            except.syndrome.iss[15:0] = immediate;
            except.trappedsyscallinst = TRUE;
            let preferred_exception_return : bits(64) = ThisInstrAddr{}();
            let vect_offset : integer = 0x0;

            AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
        end;
    end;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_CheckForWFxTrap

```
// AArch64_CheckForWFxTrap()
// =========================
// Checks for a trap on a WFE, WFET, WFI or WFIT instruction.

func AArch64_CheckForWFxTrap(wfxtype : WFxType) => (boolean, bits(2))
begin
    let is_wfe : boolean = wfxtype IN {WFxType_WFE, WFxType_WFET};
    var target_el : bits(2);
    var trap : boolean = FALSE;

    if HaveEL(EL3) && EL3SDDUndefPriority() && PSTATE.EL != EL3 then
        // Check for traps described by the Secure Monitor.
        // If the trap is enabled, the instruction will be UNDEFINED because EDSCR.SDD is 1.
        trap      = (if is_wfe then SCR_EL3().TWE else SCR_EL3().TWI) == '1';
        target_el = EL3;
    end;

    if !trap && PSTATE.EL == EL0 then
        // Check for traps described by the OS which may be EL1 or EL2.
        trap      = (if is_wfe then SCTLR_ELx().nTWE else SCTLR_ELx().nTWI) == '0';
        target_el = EL1;
    end;

    if !trap && PSTATE.EL IN {EL0, EL1} && EL2Enabled() && !IsInHost() then
        // Check for traps described by the Hypervisor.
        trap      = (if is_wfe then HCR_EL2().TWE else HCR_EL2().TWI) == '1';
        target_el = EL2;
    end;

    if !trap && HaveEL(EL3) && PSTATE.EL != EL3 then
        // Check for traps described by the Secure Monitor.
        trap      = (if is_wfe then SCR_EL3().TWE else SCR_EL3().TWI) == '1';
        target_el = EL3;
    end;
    return (trap, target_el);
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_CheckIllegalState

```
// AArch64_CheckIllegalState()
// ===========================
// Check PSTATE.IL bit and generate Illegal Execution state exception if set.

func AArch64_CheckIllegalState()
begin
    if PSTATE.IL == '1' then
        let route_to_el2 : boolean = PSTATE.EL == EL0 && EL2Enabled() && HCR_EL2().TGE == '1';

        let preferred_exception_return : bits(64) = ThisInstrAddr{}();
        let vect_offset : integer = 0x0;

        let except : ExceptionRecord = ExceptionSyndrome(Exception_IllegalState);

        if UInt(PSTATE.EL) > UInt(EL1) then
            AArch64_TakeException(PSTATE.EL, except, preferred_exception_return, vect_offset);
        elsif route_to_el2 then
            AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
        else
            AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
        end;
    end;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_MonitorModeTrap

```
// AArch64_MonitorModeTrap()
// =========================
// Trapped use of Monitor mode features in a Secure EL1 AArch32 mode

func AArch64_MonitorModeTrap()
begin
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer                 = 0x0;

    let except : ExceptionRecord              = ExceptionSyndrome(Exception_Uncategorized);

    if IsSecureEL2Enabled() then
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    end;
    AArch64_TakeException(EL3, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_OtherInstrTrap

```
// AArch64_OtherInstrTrap()
// ========================
// Trapped access to the following instructions:
// - LD64B
// - ST64B
// - ST64BV
// - ST64BV0
// - PSB
// - TSB

func AArch64_OtherInstrTrap(target_el : bits(2), iss : bits(25))
begin
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_OtherTrap);
    except.syndrome.iss = iss;
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_SystemAccessTrap

```
// AArch64_SystemAccessTrap()
// ==========================
// Trapped access to AArch64 System register or system instruction.

func AArch64_SystemAccessTrap(target_el : bits(2), ec : integer)
begin
    assert HaveEL(target_el) && target_el != EL0 && UInt(target_el) >= UInt(PSTATE.EL);

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    let except : ExceptionRecord = AArch64_SystemAccessTrapSyndrome(ThisInstr(), ec);
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_SystemAccessTrapSyndrome

```
// AArch64_SystemAccessTrapSyndrome()
// ==================================
// Returns the syndrome information for traps on AArch64 MSR/MRS instructions.

func AArch64_SystemAccessTrapSyndrome(instr_in : bits(32), ec : integer) => ExceptionRecord
begin
    var except : ExceptionRecord;
    var instr : bits(32) = instr_in;
    case ec of
        when 0x0 =>                     // Trapped access due to unknown reason.
            except = ExceptionSyndrome(Exception_Uncategorized);
        when 0x7 =>                     // Trapped access to SVE, Advance SIMD&FP System register.
            except = ExceptionSyndrome(Exception_AdvSIMDFPAccessTrap);
            except.syndrome.iss[24:20] = ConditionSyndrome();
        when 0x14 =>                    // Trapped access to 128-bit System register or
                                        // 128-bit System instruction.
            except = ExceptionSyndrome(Exception_SystemRegister128Trap);
            instr = ThisInstr();
            except.syndrome.iss[21:20] = instr[20:19]; // Op0
            except.syndrome.iss[19:17] = instr[7:5];   // Op2
            except.syndrome.iss[16:14] = instr[18:16]; // Op1
            except.syndrome.iss[13:10] = instr[15:12]; // CRn
            except.syndrome.iss[9:6]   = instr[4:1];   // Rt
            except.syndrome.iss[4:1]   = instr[11:8];  // CRm
            except.syndrome.iss[0]     = instr[21];    // Direction
        when 0x18 =>                    // Trapped access to System register or system instruction.
            except = ExceptionSyndrome(Exception_SystemRegisterTrap);
            instr = ThisInstr();
            except.syndrome.iss[21:20] = instr[20:19]; // Op0
            except.syndrome.iss[19:17] = instr[7:5];   // Op2
            except.syndrome.iss[16:14] = instr[18:16]; // Op1
            except.syndrome.iss[13:10] = instr[15:12]; // CRn
            except.syndrome.iss[9:5]   = instr[4:0];   // Rt
            except.syndrome.iss[4:1]   = instr[11:8];  // CRm
            except.syndrome.iss[0]     = instr[21];    // Direction
        when 0x19 =>                       // Trapped access to SVE System register
            except = ExceptionSyndrome(Exception_SVEAccessTrap);
        when 0x1D =>                       // Trapped access to SME System register
            except = ExceptionSyndrome(Exception_SMEAccessTrap);
        otherwise =>
            unreachable;
    end;

    return except;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_SystemAccessTraptoEL1orEL2

```
// AArch64_SystemAccessTraptoEL1orEL2()
// ====================================
// Trapped access to AArch64 System register or system instruction from EL0.
// The trap is taken to EL1, or to EL2 if HCR_EL2().TGE is 1.

func AArch64_SystemAccessTraptoEL1orEL2(ec : integer)
begin
    assert PSTATE.EL == EL0;
    let target_el : bits(2) = if EL2Enabled() && HCR_EL2().TGE == '1' then EL2 else EL1;

    AArch64_SystemAccessTrap(target_el, ec);
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_Undefined

```
// AArch64_Undefined()
// ===================

noreturn func AArch64_Undefined()
begin

    let route_to_el2 : boolean = PSTATE.EL == EL0 && EL2Enabled() && HCR_EL2().TGE == '1';
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    let except : ExceptionRecord = ExceptionSyndrome(Exception_Uncategorized);

    if UInt(PSTATE.EL) > UInt(EL1) then
        AArch64_TakeException(PSTATE.EL, except, preferred_exception_return, vect_offset);
    elsif route_to_el2 then
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/traps/AArch64_WFxTrap

```
// AArch64_WFxTrap()
// =================
// Generate an exception for a trapped WFE, WFI, WFET or WFIT instruction.

func AArch64_WFxTrap(wfxtype : WFxType, target_el : bits(2))
begin
    assert UInt(target_el) > UInt(PSTATE.EL);
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;
    var except : ExceptionRecord = ExceptionSyndrome(Exception_WFxTrap);
    except.syndrome.iss[24:20] = ConditionSyndrome();
    case wfxtype of
        when WFxType_WFI =>
            except.syndrome.iss[1:0] = '00';
        when WFxType_WFE =>
            except.syndrome.iss[1:0] = '01';
        when WFxType_WFIT =>
            except.syndrome.iss[1:0] = '10';
            except.syndrome.iss[2]   = '1';   // Register field is valid
            except.syndrome.iss[9:5] = ThisInstr()[4:0];
        when WFxType_WFET =>
            except.syndrome.iss[1:0] = '11';
            except.syndrome.iss[2]   = '1';   // Register field is valid
            except.syndrome.iss[9:5] = ThisInstr()[4:0];
    end;

    if target_el == EL1 && EL2Enabled() && HCR_EL2().TGE == '1' then
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/exceptions/traps/CheckLDST64BEnabled

```
// CheckLDST64BEnabled()
// =====================
// Checks for trap on ST64B, LD64B, STL64B, and LDA64B instructions

func CheckLDST64BEnabled(acqrel : boolean)
begin
    var trap : boolean = FALSE;
    let iss : bits(25) = if acqrel then 0x8[24:0] else 0x2[24:0];
    var target_el : bits(2);

    if PSTATE.EL == EL0 then
        if !IsInHost() then
            trap = SCTLR_EL1().EnALS == '0';
            target_el = if EL2Enabled() && HCR_EL2().TGE == '1' then EL2 else EL1;
        else
            trap = SCTLR_EL2().EnALS == '0';
            target_el = EL2;
        end;
    else
        target_el = EL1;
    end;

    if !trap && EL2Enabled() && ((PSTATE.EL == EL0 && !IsInHost()) || PSTATE.EL == EL1) then
        trap = !IsHCRXEL2Enabled() || HCRX_EL2().EnALS == '0';
        target_el = EL2;
    end;

    if trap then AArch64_OtherInstrTrap(target_el, iss); end;
end;
```

Library pseudocode for aarch64/exceptions/traps/CheckST64BV0Enabled

```
// CheckST64BV0Enabled()
// ====================
// Checks for trap on ST64BV0 and STL64BV0 instructions

func CheckST64BV0Enabled(acqrel : boolean)
begin
    var trap : boolean = FALSE;
    let iss : bits(25) = if acqrel then 0x7[24:0] else 0x1[24:0];
    var target_el : bits(2);

    if PSTATE.EL != EL3 && HaveEL(EL3) && SCR_EL3().EnAS0 == '0' && EL3SDDUndefPriority() then
        Undefined();
    end;

    if PSTATE.EL == EL0 then
        if !IsInHost() then
            trap = SCTLR_EL1().EnAS0 == '0';
            target_el = if EL2Enabled() && HCR_EL2().TGE == '1' then EL2 else EL1;
        else
            trap = SCTLR_EL2().EnAS0 == '0';
            target_el = EL2;
        end;
    end;

    if !trap && EL2Enabled() && ((PSTATE.EL == EL0 && !IsInHost()) || PSTATE.EL == EL1) then
        trap = !IsHCRXEL2Enabled() || HCRX_EL2().EnAS0 == '0';
        target_el = EL2;
    end;

    if !trap && PSTATE.EL != EL3 then
        trap = HaveEL(EL3) && SCR_EL3().EnAS0 == '0';
        target_el = EL3;
    end;

    if trap then
        if target_el == EL3 && EL3SDDUndef() then
            Undefined();
        else
            AArch64_OtherInstrTrap(target_el, iss);
        end;
    end;
end;
```

Library pseudocode for aarch64/exceptions/traps/CheckST64BVEnabled

```
// CheckST64BVEnabled()
// ====================
// Checks for trap on ST64BV and STL64BV instructions

func CheckST64BVEnabled(acqrel : boolean)
begin
    var trap : boolean = FALSE;
    let iss : bits(25) = if acqrel then 0x6[24:0] else 0x0[24:0];
    var target_el : bits(2);

    if PSTATE.EL == EL0 then
        if !IsInHost() then
            trap = SCTLR_EL1().EnASR == '0';
            target_el = if EL2Enabled() && HCR_EL2().TGE == '1' then EL2 else EL1;
        else
            trap = SCTLR_EL2().EnASR == '0';
            target_el = EL2;
        end;
    end;

    if !trap && EL2Enabled() && ((PSTATE.EL == EL0 && !IsInHost()) || PSTATE.EL == EL1) then
        trap = !IsHCRXEL2Enabled() || HCRX_EL2().EnASR == '0';
        target_el = EL2;
    end;

    if trap then AArch64_OtherInstrTrap(target_el, iss); end;
end;
```

Library pseudocode for aarch64/exceptions/traps/WFETrapDelay

```
// WFETrapDelay()
// ==============
// Returns TRUE when delay in trap to WFE is enabled with value to amount of delay,
// FALSE otherwise.

func WFETrapDelay(target_el : bits(2)) => (boolean, integer)
begin
    var delay_enabled : boolean;
    var delay : integer;
    case target_el of
        when EL1 =>
            if !IsInHost() then
                delay_enabled = SCTLR_EL1().TWEDEn == '1';
                delay         = 1 << (UInt(SCTLR_EL1().TWEDEL) + 8);
            else
                delay_enabled = SCTLR_EL2().TWEDEn == '1';
                delay         = 1 << (UInt(SCTLR_EL2().TWEDEL) + 8);
            end;
        when EL2 =>
            assert EL2Enabled();
            delay_enabled = HCR_EL2().TWEDEn == '1';
            delay         = 1 << (UInt(HCR_EL2().TWEDEL) + 8);
        when EL3 =>
            delay_enabled = SCR_EL3().TWEDEn == '1';
            delay         = 1 << (UInt(SCR_EL3().TWEDEL) + 8);
    end;
    return (delay_enabled, delay);
end;
```

Library pseudocode for aarch64/exceptions/traps/WaitForEventUntilDelay

```
// WaitForEventUntilDelay()
// ========================
// Returns TRUE if WaitForEvent() returns before WFE trap delay expires, including potentially
// some IMPLEMENTATION SPECIFIC additional delay, FALSE otherwise.

impdef func WaitForEventUntilDelay(delay_enabled : boolean, delay : integer) => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/functions/aborts/AArch64_FaultSyndrome

```
// AArch64_FaultSyndrome()
// =======================
// Creates an exception syndrome value and updates the virtual address for Abort and Watchpoint
// exceptions taken to an Exception level using AArch64.

func AArch64_FaultSyndrome(exceptype : Exception, fault : FaultRecord,
                           target_el : bits(2)) => IssType
begin
    assert fault.statuscode != Fault_None;

    var isstype : IssType;
    isstype.iss  = Zeros{25};
    isstype.iss2 = Zeros{24};

    let d_side : boolean = exceptype IN {Exception_DataAbort, Exception_NV2DataAbort,
                                         Exception_Watchpoint, Exception_NV2Watchpoint};
    if IsFeatureImplemented(FEAT_RAS) && fault.statuscode == Fault_SyncExternal then
        let errstate : ErrorState = PEErrorState(fault);
        isstype.iss[12:11] = AArch64_EncodeSyncErrorSyndrome(errstate);  // SET
    end;

    if d_side then
        if (IsFeatureImplemented(FEAT_EAESR) && exceptype == Exception_DataAbort &&
              !fault.s2fs1walk && ! fault.accessdesc.acctype IN {AccessType_DC,
                                                                 AccessType_IC,
                                                                 AccessType_AT,
                                                                 AccessType_NV2}) then
            // LST2 field
            if fault.accessdesc.atomicop then
                isstype.iss2[15:13] = '001';    // Data Abort is from an Atomic instruction
            elsif (fault.accessdesc.acctype == AccessType_DCZero &&
                     fault.accessdesc.cachetype IN {CacheType_Data, CacheType_Data_Tag}) then
                isstype.iss2[15:13] = '010';    // Data Abort is from a DC ZVA or DC GZVA
            end;

            // Unpriv field
            if (fault.accessdesc.el == EL0 && PSTATE.EL != EL0 &&
                  fault.accessdesc.acctype IN {AccessType_GPR,
                                               AccessType_GCS,
                                               AccessType_ASIMD}) then
                isstype.iss2[12] = '1';         // Data Abort is from an unprivileged load or store
            end;
        end;

        if fault.accessdesc.acctype == AccessType_GCS then
            isstype.iss2[8] = '1';
        end;
        if exceptype IN {Exception_Watchpoint, Exception_NV2Watchpoint} then
            isstype.iss[23:0] = WatchpointRelatedSyndrome(fault);
        end;

        if AArch64_InstructionSyndromeValid(fault) then
            if fault.accessdesc.ls64 then
                (isstype.iss2, isstype.iss[24:14]) = LS64InstructionSyndrome();
            else
                isstype.iss[24:14] = LSInstructionSyndrome();
            end;
        end;

        if IsFeatureImplemented(FEAT_NV2) && fault.accessdesc.acctype == AccessType_NV2 then
            isstype.iss[13] = '1';   // Fault is generated by use of VNCR_EL2
        end;

        if (IsFeatureImplemented(FEAT_LS64) &&
              fault.statuscode IN {Fault_AccessFlag, Fault_Translation, Fault_Permission}) then
            isstype.iss[12:11] = GetLoadStoreType();
        end;

        if fault.accessdesc.acctype IN {AccessType_DC, AccessType_IC, AccessType_AT} then
            isstype.iss[8] = '1';
        end;

        if fault.accessdesc.acctype IN {AccessType_DC, AccessType_IC, AccessType_AT} then
            isstype.iss[6] = '1';
        elsif fault.statuscode IN {Fault_UnsupportedAtomicHWUpdate, Fault_Exclusive} then
            isstype.iss[6] = ARBITRARY : bit;
        elsif fault.accessdesc.atomicop && IsExternalAbort(fault) then
            isstype.iss[6] = ARBITRARY : bit;
        else
            isstype.iss[6] = if fault.write then '1' else '0';
        end;
        isstype.iss2[9] = if fault.tagaccess then '1' else '0';
        if exceptype IN {Exception_DataAbort, Exception_NV2DataAbort} then
            isstype.iss2[5] = if fault.dirtybit then '1' else '0';
            isstype.iss2[6] = if fault.overlay then '1' else '0';
            if isstype.iss[24] == '0' then
                isstype.iss[21] = if fault.toplevel then '1' else '0';
            end;
            isstype.iss2[7] = if fault.assuredonly then '1' else '0';
```

```
            isstype.iss2[10] = if fault.s1tagnotdata then '1' else '0';
        end;

    else
        if fault.accessdesc.acctype == AccessType_IFETCH then
            isstype.iss2[5] = if fault.dirtybit then '1' else '0';
            isstype.iss[21] = if fault.toplevel then '1' else '0';
            isstype.iss2[7] = if fault.assuredonly then '1' else '0';
            isstype.iss2[6] = if fault.overlay then '1' else '0';
        end;
    end;
    if fault.hdbssf then
        isstype.iss2[11] = '1';
    end;

    if (IsFeatureImplemented(FEAT_PFAR) && IsExternalSyncAbort(fault) &&
          !(EL2Enabled() &&
              (HCR_EL2().VM == '1' || (IsFeatureImplemented(FEAT_HCRMAC) && HCR_EL2().DC == '1')) &&
              target_el == EL1)) then
        isstype.iss[14] = if IsPFARValid(fault) then '1' else '0';
    end;

    if IsExternalAbort(fault) then isstype.iss[9] = fault.extflag; end;
    isstype.iss[7] = if fault.s2fs1walk then '1' else '0';
    isstype.iss[5:0] = EncodeLDFSC(fault.statuscode, fault.level);

    return isstype;
end;
```

Library pseudocode for aarch64/functions/aborts/AArch64_InstructionSyndromeValid

```
// AArch64_InstructionSyndromeValid()
// ==================================
// Returns TRUE if ESR_ELx.ISV is '1' for the given Fault.

readonly func AArch64_InstructionSyndromeValid(fault : FaultRecord) => boolean
begin
    if !fault.accessdesc.lowestaddress then
        return FALSE;
    end;

    if fault.s2fs1walk then
        return FALSE;
    end;

    if IsFeatureImplemented(FEAT_RAS) && IsExternalSyncAbort(fault) then
        return FALSE;
    end;

    if fault.accessdesc.ls64 then
        return fault.statuscode IN {Fault_AccessFlag, Fault_Translation, Fault_Permission};
    end;

    if fault.accessdesc.acctype == AccessType_GCS then
        return FALSE;
    end;

    if (IsFeatureImplemented(FEAT_EAESR) && !IsSecondStage(fault) &&
          fault.statuscode IN {Fault_AccessFlag, Fault_Translation, Fault_Permission}) then
        return TRUE;
    end;

    if IsSecondStage(fault) then
        return (!IsExternalAbortOnWalk(fault) ||
                ImpDefBool("ISV on second stage translation table walk"));
    end;

    return FALSE;
end;
```

Library pseudocode for aarch64/functions/aborts/EncodeGPCSC

```
// EncodeGPCSC()
// =============
// Function that gives the GPCSC code for types of GPT Fault

func EncodeGPCSC(gpcf : GPCFRecord) => bits(6)
begin
    assert gpcf.level IN {0,1};

    case gpcf.gpf of
        when GPCF_AddressSize => return '00000'::gpcf.level[0];
        when GPCF_Walk =>        return '00010'::gpcf.level[0];
        when GPCF_Fail =>        return '00110'::gpcf.level[0];
        when GPCF_EABT =>        return '01010'::gpcf.level[0];
    end;
end;
```

Library pseudocode for aarch64/functions/aborts/LS64InstructionSyndrome

```
// LS64InstructionSyndrome()
// =========================
// Returns the syndrome information and LST for a Data Abort by a
// ST64B, ST64BV, ST64BV0, or LD64B instruction. The syndrome information
// includes the ISS2, extended syndrome field.

impdef func LS64InstructionSyndrome() => (bits(24), bits(11))
begin
    return (Zeros{24}, LSInstructionSyndrome());
end;
```

Library pseudocode for aarch64/functions/aborts/WatchpointFARNotPrecise

```
// WatchpointFARNotPrecise()
// =========================
// Returns TRUE If the lowest watchpointed address that is higher than or equal to the address
// recorded in EDWAR might not have been accessed by the instruction, other than the CONSTRAINED
// UNPREDICTABLE condition of watchpoint matching a range of addresses with lowest address 16 bytes
// rounded down and upper address rounded up to nearest 16 byte multiple,
// FALSE otherwise.

impdef func WatchpointFARNotPrecise(fault : FaultRecord) => boolean
begin
    if IsFeatureImplemented(FEAT_SVE) || IsFeatureImplemented(FEAT_SME) then
        return fault.watchptinfo.maybe_false_match;
    end;
    return FALSE;
end;
```

Library pseudocode for aarch64/functions/apas/AArch64_APAS

```
// AArch64_APAS()
// ==============
// Decode Xt and perform an APAS operation for the decoded record.

func AArch64_APAS(Xt : bits(64))
begin
    var apas : APASRecord;
    let nse2 : bit = '0';
    apas.paspace = DecodePASpace(nse2, Xt[62], Xt[63]);

    apas.pa = Xt[NUM_PABITS-1:6] :: '000000';
    apas.target_attributes = Xt[2:0];

    if AArch64_LocationSupportsAPAS(apas) then
        APAS_OP(apas);
    end;
end;
```

Library pseudocode for aarch64/functions/apas/AArch64_LocationSupportsAPAS

```
// AArch64_LocationSupportsAPAS()
// ==============================
// Returns TRUE if the given memory location supports the APAS instruction.

impdef func AArch64_LocationSupportsAPAS(apas : APASRecord) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/apas/APASRecord

```
// APASRecord
// ==========
// Details related to an APAS operation.

type APASRecord of record {
    pa : bits(NUM_PABITS),
    paspace : PASpace,
    target_attributes : bits(3)
};
```

Library pseudocode for aarch64/functions/apas/APAS_OP

```
// APAS_OP()
// =========
// Sets the PA Space of the address in the APASRecord to the target PA space. If the location
// does not support the APAS instruction or cannot be associated with the indicated PASpace,
// then the instruction has no effect on the location and does not generate an External abort.

impdef func APAS_OP(apas : APASRecord)
begin
    return;
end;
```

Library pseudocode for aarch64/functions/at/AArch64_AT

```
// AArch64_AT()
// ============
// Perform address translation as per AT instructions.

func AArch64_AT(address : bits(64), stage : TranslationStage, el_in : bits(2), ataccess : ATAccess)
begin
    var el : bits(2) = el_in;
    // For stage 1 translation, when HCR_EL2().[E2H, TGE] is {1,1} and requested EL is EL1,
    // the EL2&0 translation regime is used.
    if ELIsInHost(EL0) && el == EL1 && stage == TranslationStage_1 then
        el = EL2;
    end;

    let ss : SecurityState = SecurityStateAtEL(el);

    let accdesc : AccessDescriptor = CreateAccDescAT(ss, el, ataccess);
    let aligned : boolean = TRUE;

    var fault : FaultRecord = NoFault(accdesc, address);
    let regime : Regime = TranslationRegime(el);

    var addrdesc : AddressDescriptor;
    if (el == EL0 && ELUsingAArch32(EL1)) || (el != EL0 && ELUsingAArch32(el)) then
        if regime == Regime_EL2 || TTBCR().EAE == '1' then
            (fault, addrdesc) = AArch32_S1TranslateLD(fault, regime, address[31:0], aligned,
                                                      accdesc);
        else
            (fault, addrdesc, -) = AArch32_S1TranslateSD(fault, regime, address[31:0], aligned,
                                                         accdesc);
        end;
    else
        let size : integer = 1;
        (fault, addrdesc) = AArch64_S1Translate(fault, regime, address, size, aligned, accdesc);
    end;

    if stage == TranslationStage_12 && fault.statuscode == Fault_None then
        let s1aarch64 : boolean = TRUE;
        if ELUsingAArch32(EL1) && regime == Regime_EL10 && EL2Enabled() then
            addrdesc.vaddress = ZeroExtend{64}(address);
            (fault, addrdesc) = AArch32_S2Translate(fault, addrdesc, aligned, accdesc);
        elsif regime == Regime_EL10 && EL2Enabled() then
            (fault, addrdesc) = AArch64_S2Translate(fault, addrdesc, s1aarch64, aligned, accdesc);
        end;
    end;

    let is_ATS1Ex : boolean = stage != TranslationStage_12;
    if fault.statuscode != Fault_None then
        addrdesc = CreateFaultyAddressDescriptor(fault);
        // Take an exception on:
        // * A Synchronous External abort occurs on translation table walk
        // * A stage 2 fault occurs on a stage 1 walk
        // * A GPC Exception (FEAT_RME)
        // * A GPF from ATS1E{1,0}* when executed from EL1 and HCR_EL2.GPF == '1' (FEAT_RME)
        if (IsExternalAbort(fault) ||
              (PSTATE.EL == EL1 && fault.s2fs1walk) ||
               (IsFeatureImplemented(FEAT_RME) && fault.gpcf.gpf != GPCF_None && (
                ReportAsGPCException(fault) ||
                (EL2Enabled() && HCR_EL2().GPF == '1' && PSTATE.EL == EL1 && el IN {EL1, EL0} &&
                 is_ATS1Ex)
                ))) then
            if IsFeatureImplemented(FEAT_D128) then
                PAR_EL1()       = ARBITRARY : bits(128);
            else
                PAR_EL1()[63:0] = ARBITRARY : bits(64);
            end;
            AArch64_Abort(addrdesc.fault);
        end;
    end;

    AArch64_EncodePAR(regime, el, is_ATS1Ex, addrdesc);
    return;
end;
```

Library pseudocode for aarch64/functions/at/AArch64_EncodePAR

```
// AArch64_EncodePAR()
// ===================
// Encode PAR register with result of translation.

func AArch64_EncodePAR(regime : Regime, el : bits(2), is_ATS1Ex : boolean,
                       addrdesc : AddressDescriptor)
begin
    let paspace : PASpace = addrdesc.paddress.paspace;
    if IsFeatureImplemented(FEAT_D128) then
        PAR_EL1() = Zeros{128};
        if AArch64_isPARFormatD128(regime, is_ATS1Ex) then
            PAR_EL1().D128 = '1';
        else
            PAR_EL1().D128 = '0';
        end;
    else
        PAR_EL1()[63:0] = Zeros{64};
    end;

    if !IsFault(addrdesc) then
        PAR_EL1().F = '0';
        if IsFeatureImplemented(FEAT_RME) then
            if regime == Regime_EL3 then
                case paspace of
                    when PAS_Secure =>     PAR_EL1().[NSE,NS] = '00';
                    when PAS_NonSecure =>  PAR_EL1().[NSE,NS] = '01';
                    when PAS_Root =>       PAR_EL1().[NSE,NS] = '10';
                    when PAS_Realm =>      PAR_EL1().[NSE,NS] = '11';
                end;

            elsif SecurityStateForRegime(regime) == SS_Secure  then
                PAR_EL1().NSE = ARBITRARY : bit;
                PAR_EL1().NS = if paspace == PAS_Secure then '0' else '1';

            elsif SecurityStateForRegime(regime) == SS_Realm then
                if regime == Regime_EL10 && is_ATS1Ex then
                    PAR_EL1().NSE = ARBITRARY : bit;
                    PAR_EL1().NS  = ARBITRARY : bit;
                else
                    PAR_EL1().NSE = ARBITRARY : bit;
                    PAR_EL1().NS  = if paspace == PAS_Realm then '0' else '1';
                end;

            else
                PAR_EL1().NSE = ARBITRARY : bit;
                PAR_EL1().NS  = ARBITRARY : bit;
            end;
        else
            PAR_EL1()[11] = '1'; // RES1
            if SecurityStateForRegime(regime) == SS_Secure then
                PAR_EL1().NS = if paspace == PAS_Secure then '0' else '1';
            else
                PAR_EL1().NS = ARBITRARY : bit;
            end;
        end;
        PAR_EL1().SH   = ReportedPARShareability(PAREncodeShareability(addrdesc.memattrs));
        if IsFeatureImplemented(FEAT_D128) && PAR_EL1().D128 == '1' then
            PAR_EL1()[119:76] = addrdesc.paddress.address[55:12];
        else
            PAR_EL1()[55:12] = addrdesc.paddress.address[55:12];
        end;
        PAR_EL1().ATTR = ReportedPARAttrs(EncodePARAttrs(addrdesc.memattrs));
        PAR_EL1()[10] = ImpDefBit("Non-Faulting PAR");
    else
        PAR_EL1().F   = '1';
        PAR_EL1().DirtyBit    = if addrdesc.fault.dirtybit then '1' else '0';
        PAR_EL1().Overlay     = if addrdesc.fault.overlay then '1' else '0';
        PAR_EL1().TopLevel    = if addrdesc.fault.toplevel then '1' else '0';
        PAR_EL1().AssuredOnly = if addrdesc.fault.assuredonly then '1' else '0';
        PAR_EL1().FST = AArch64_PARFaultStatus(addrdesc.fault);
        PAR_EL1().PTW = if addrdesc.fault.s2fs1walk then '1' else '0';
        PAR_EL1().S   = if addrdesc.fault.secondstage then '1' else '0';
        PAR_EL1()[11] = '1'; // RES1
        PAR_EL1()[63:48] = ImpDefBits{16}("Faulting PAR");
    end;
    return;
end;
```

Library pseudocode for aarch64/functions/at/AArch64_PARFaultStatus

```
// AArch64_PARFaultStatus()
// ========================
// Fault status field decoding of 64-bit PAR.

func AArch64_PARFaultStatus(fault : FaultRecord) => bits(6)
begin
    var fst : bits(6);

    if fault.statuscode == Fault_Domain then
        // Report Domain fault
        assert fault.level IN {1,2};
        fst[1:0] = if fault.level == 1 then '01' else '10';
        fst[5:2] = '1111';
    else
        fst = EncodeLDFSC(fault.statuscode, fault.level);
    end;
    return fst;
end;
```

Library pseudocode for aarch64/functions/at/AArch64_isPARFormatD128

```
// AArch64_isPARFormatD128()
// =========================
// Check if last stage of translation uses VMSAv9-128.
// Last stage of translation is stage 2 if enabled, else it is stage 1.

func AArch64_isPARFormatD128(regime : Regime, is_ATS1Ex : boolean) => boolean
begin
    var isPARFormatD128 : boolean;
    // Regime_EL2 does not support VMSAv9-128
    if regime == Regime_EL2 || !IsFeatureImplemented(FEAT_D128) then
        isPARFormatD128 = FALSE;
    else
        isPARFormatD128 = FALSE;
        case regime of
            when Regime_EL3 =>
                isPARFormatD128 = TCR_EL3().D128 == '1';
            when Regime_EL20 =>
                isPARFormatD128 = IsTCR2EL2Enabled() && TCR2_EL2().D128 == '1';
            when Regime_EL10 =>
                if is_ATS1Ex || !EL2Enabled() || HCR_EL2().[VM,DC] == '00' then
                    isPARFormatD128 = IsTCR2EL1Enabled() && TCR2_EL1().D128 == '1';
                else
                    isPARFormatD128 = VTCR_EL2().D128 == '1';
                end;
        end;
    end;

    return isPARFormatD128;
end;
```

Library pseudocode for aarch64/functions/at/GetPAR_EL1_D128

```
// GetPAR_EL1_D128()
// =================
// Query the PAR_EL1.D128 field

func GetPAR_EL1_D128() => bit
begin
    return if IsFeatureImplemented(FEAT_D128) then PAR_EL1().D128 else '0';
end;
```

Library pseudocode for aarch64/functions/at/GetPAR_EL1_F

```
// GetPAR_EL1_F()
// ==============
// Query the PAR_EL1.F field.

func GetPAR_EL1_F() => bit
begin
    var F : bit;

    F = PAR_EL1().F;
    return F;
end;
```

Library pseudocode for aarch64/functions/barrierop/MemBarrierOp

```
// MemBarrierOp
// ============
// Memory barrier instruction types.

type MemBarrierOp of enumeration {
                                  MemBarrierOp_DSB,     // Data Synchronization Barrier
                                  MemBarrierOp_DMB,     // Data Memory Barrier
                                  MemBarrierOp_ISB,     // Instruction Synchronization Barrier
                                  MemBarrierOp_SSBB,    // Speculative Synchronization Barrier to VA
                                  MemBarrierOp_PSSBB,   // Speculative Synchronization Barrier to PA
                                  MemBarrierOp_SB       // Speculation Barrier
                              };
```

Library pseudocode for aarch64/functions/bfxpreferred/BFXPreferred

```
// BFXPreferred()
// ==============
//
// Return TRUE if UBFX or SBFX is the preferred disassembly of a
// UBFM or SBFM bitfield instruction. Must exclude more specific
// aliases UBFIZ, SBFIZ, UXT[BH], SXT[BHW], LSL, LSR and ASR.

func BFXPreferred(sf : bit, uns : bit, imms : bits(6), immr : bits(6)) => boolean
begin

    // must not match UBFIZ/SBFIX alias
    if UInt(imms) < UInt(immr) then
        return FALSE;
    end;

    // must not match LSR/ASR/LSL alias (imms == 31 or 63)
    if imms == sf::'11111' then
        return FALSE;
    end;

    // must not match UXTx/SXTx alias
    if immr == '000000' then
        // must not match 32-bit UXT[BH] or SXT[BH]
        if sf == '0' && imms IN {'000111', '001111'} then
            return FALSE;
        end;
        // must not match 64-bit SXT[BHW]
        if sf::uns == '10' && imms IN {'000111', '001111', '011111'} then
            return FALSE;
        end;
    end;

    // must be UBFX/SBFX alias
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/bitmasks/AltDecodeBitMasks

```
// AltDecodeBitMasks()
// ===================
// Alternative but logically equivalent implementation of DecodeBitMasks() that
// uses simpler primitives to compute tmask and wmask.

func AltDecodeBitMasks{M}(immN : bit, imms : bits(6), immr : bits(6),
                          immediate : boolean) => (bits(M), bits(M))
begin
    var tmask, wmask : bits(64);
    var tmask_and, wmask_and : bits(6);
    var tmask_or, wmask_or : bits(6);
    var levels : bits(6);

    // Compute log2 of element size
    // 2^len must be in range [2, M]
    if immN::NOT(imms) == '000000x' then Undefined(); end;
    let len : integer{} = HighestSetBitNZ(immN::NOT(imms));
    assert 2 <= (2^len) && (2^len) <= M;

    // Determine s, r and s - r parameters
    levels = ZeroExtend{6}(Ones{len});

    // For logical immediates an all-ones value of s is reserved
    // since it would generate a useless all-ones result (many times)
    if immediate && (imms AND levels) == levels then Undefined(); end;

    let s : integer{} = UInt(imms AND levels);
    let r : integer{} = UInt(immr AND levels);
    let diff : integer{} = s - r;    // 6-bit subtract with borrow

    // Compute "top mask"
    tmask_and = diff[5:0] OR NOT(levels);
    tmask_or  = diff[5:0] AND levels;

    tmask = Ones{64};
    tmask = ((tmask
              AND Replicate{64, 2}(Replicate{1}(tmask_and[0]) :: Ones{1}))
               OR  Replicate{64, 2}(Zeros{1} :: Replicate{1}(tmask_or[0])));
    // optimization of first step:
    // tmask = Replicate{64}(tmask_and[0] :: '1');
    tmask = ((tmask
              AND Replicate{64, 4}(Replicate{2}(tmask_and[1]) :: Ones{2}))
               OR  Replicate{64, 4}(Zeros{2} :: Replicate{2}(tmask_or[1])));
    tmask = ((tmask
              AND Replicate{64, 8}(Replicate{4}(tmask_and[2]) :: Ones{4}))
               OR  Replicate{64, 8}(Zeros{4} :: Replicate{4}(tmask_or[2])));
    tmask = ((tmask
              AND Replicate{64, 16}(Replicate{8}(tmask_and[3]) :: Ones{8}))
               OR  Replicate{64, 16}(Zeros{8} :: Replicate{8}(tmask_or[3])));
    tmask = ((tmask
              AND Replicate{64, 32}(Replicate{16}(tmask_and[4]) :: Ones{16}))
               OR  Replicate{64, 32}(Zeros{16} :: Replicate{16}(tmask_or[4])));
    tmask = ((tmask
              AND Replicate{64, 64}(Replicate{32}(tmask_and[5]) :: Ones{32}))
               OR  Replicate{64, 64}(Zeros{32} :: Replicate{32}(tmask_or[5])));

    // Compute "wraparound mask"
    wmask_and = immr OR NOT(levels);
    wmask_or  = immr AND levels;

    wmask = Zeros{64};
    wmask = ((wmask
              AND Replicate{64, 2}(Ones{1} :: Replicate{1}(wmask_and[0])))
               OR  Replicate{64, 2}(Replicate{1}(wmask_or[0]) :: Zeros{1}));
    // wmask = Replicate{64}(wmask_or[0] :: '0');
    wmask = ((wmask
              AND Replicate{64, 4}(Ones{2} :: Replicate{2}(wmask_and[1])))
               OR  Replicate{64, 4}(Replicate{2}(wmask_or[1]) :: Zeros{2}));
    wmask = ((wmask
              AND Replicate{64, 8}(Ones{4} :: Replicate{4}(wmask_and[2])))
               OR  Replicate{64, 8}(Replicate{4}(wmask_or[2]) :: Zeros{4}));
    wmask = ((wmask
              AND Replicate{64, 16}(Ones{8} :: Replicate{8}(wmask_and[3])))
               OR  Replicate{64, 16}(Replicate{8}(wmask_or[3]) :: Zeros{8}));
    wmask = ((wmask
              AND Replicate{64, 32}(Ones{16} :: Replicate{16}(wmask_and[4])))
               OR  Replicate{64, 32}(Replicate{16}(wmask_or[4]) :: Zeros{16}));
    wmask = ((wmask
              AND Replicate{64, 64}(Ones{32} :: Replicate{32}(wmask_and[5])))
               OR  Replicate{64, 64}(Replicate{32}(wmask_or[5]) :: Zeros{32}));

    if diff[6] != '0' then // borrow from s - r
        wmask = wmask AND tmask;
    else
        wmask = wmask OR tmask;
    end;

    return (wmask[M-1:0], tmask[M-1:0]);
end;
```

Library pseudocode for aarch64/functions/bitmasks/DecodeBitMasks

```
// DecodeBitMasks()
// ================
// Decode AArch64 bitfield and logical immediate masks which use a similar encoding structure

func DecodeBitMasks{M}(immN : bit, imms : bits(6), immr : bits(6),
                       immediate : boolean) => (bits(M), bits(M))
begin
    var levels : bits(6);

    // Compute log2 of element size
    // 2^len must be in range [2, M]
    if immN::NOT(imms) == '000000x' then Undefined(); end;
    let len : integer{} = HighestSetBitNZ(immN::NOT(imms));
    assert 2 <= (2^len) && (2^len) <= M;

    // Determine s, r and s - r parameters
    levels = ZeroExtend{6}(Ones{len});

    // For logical immediates an all-ones value of s is reserved
    // since it would generate a useless all-ones result (many times)
    if immediate && (imms AND levels) == levels then Undefined(); end;

    let s : integer{} = UInt(imms AND levels);
    let r : integer{} = UInt(immr AND levels);
    let diff : integer{} = s - r;    // 6-bit subtract with borrow

    let esize : integer{} = 1 << len;
    let d : integer{}     = UInt(diff[len-1:0]);
    let welem : bits(esize) = ZeroExtend{}(Ones{s + 1});
    let telem : bits(esize) = ZeroExtend{}(Ones{d + 1});
    let wmask : bits(M) = Replicate{}(ROR(welem, r));
    let tmask : bits(M) = Replicate{}(telem);
    return (wmask, tmask);
end;
```

Library pseudocode for aarch64/functions/cache/AArch64_DataMemZero

```
// AArch64_DataMemZero()
// =====================
// Write Zero to data memory.

func AArch64_DataMemZero(regval : bits(64), vaddress : bits(64),
                         accdesc_in : AccessDescriptor, size : integer)
begin
    var accdesc : AccessDescriptor = accdesc_in;

    // If the instruction targets tags as a payload, confer with system register configuration
    // which may override this.
    if accdesc.tagaccess then
        accdesc.tagaccess = IsMTEEnabled(accdesc.el);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(vaddress, accdesc);
    end;

    let aligned : boolean = TRUE;
    var memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(vaddress,
                                                                   accdesc, aligned, size);

    if IsFault(memaddrdesc) then
        if !IsDebugException(memaddrdesc.fault) then
            memaddrdesc.fault.vaddress = regval;
        end;
        AArch64_Abort(memaddrdesc.fault);
    end;

    for i = 0 to size-1 do
        if accdesc.tagchecked then
            let ltag : bits(4)      = AArch64_LogicalAddressTag(vaddress);
            let bytes : integer = 1;
            let readcheck : boolean = FALSE;
            var fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck,
                                                       bytes, ltag);
            if fault.statuscode != Fault_None then
                if ImpDefBool(
                      "DC_ZVA tag fault reported with lowest faulting address") then
                    AArch64_Abort(fault);
                else
                    fault.vaddress = regval;
                    AArch64_Abort(fault);
                end;
            end;
        end;
        let memstatus : PhysMemRetStatus = PhysMemWrite{8}(memaddrdesc, accdesc, Zeros{8});
        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, 1, accdesc);
        end;

        memaddrdesc.paddress.address = memaddrdesc.paddress.address + 1;
    end;
    return;
end;
```

Library pseudocode for aarch64/functions/cache/AArch64_WriteTagMem

```
// AArch64_WriteTagMem()
// =====================
// Write to tag memory.

func AArch64_WriteTagMem(regval : bits(64), cachetype : CacheType,
                         accdesc_in : AccessDescriptor, size : integer)
begin
    assert accdesc_in.tagaccess;

    var accdesc : AccessDescriptor = accdesc_in;
    let vaddress : bits(64) = AlignDownSize{64}(regval, size as integer{16..2048});
    let count : integer     = size >> LOG2_TAG_GRANULE;
    var tag : bits(4);
    if cachetype == CacheType_TagZero then
        tag = '0000';
    else
        tag = AArch64_AllocationTagFromAddress(vaddress);
    end;
    let aligned : boolean   = IsAlignedSize{64}(vaddress, TAG_GRANULE);
    assert aligned;

    accdesc.tagaccess = IsMTEEnabled(accdesc.el);

    var (memtagtype, memaddrdesc) = AArch64_TranslateTagAddress(vaddress, accdesc, aligned, size);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        if IsDebugException(memaddrdesc.fault) then
            AArch64_Abort(memaddrdesc.fault);
        else
            memaddrdesc.fault.vaddress = regval;
            AArch64_Abort(memaddrdesc.fault);
        end;
    end;

    if !accdesc.tagaccess || memtagtype != MemTag_AllocationTagged then
        return;
    end;

    var memstatus : PhysMemRetStatus;

    for i = 0 to count-1 do
        memstatus = PhysMemTagWrite(memaddrdesc, accdesc, tag);
        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, 1, accdesc);
        end;

        memaddrdesc.paddress.address = memaddrdesc.paddress.address + TAG_GRANULE;
        memaddrdesc.vaddress         = memaddrdesc.vaddress + TAG_GRANULE;
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/cflt/AArch64_ConditionalFault

```
// AArch64_ConditionalFault()
// ==========================
// Generate a Conditional fault.

noreturn func AArch64_ConditionalFault(isflt : boolean, immediate : bits(16))
begin
    assert !UsingAArch32();
    var target_el : bits(2);
    if PSTATE.EL == EL0 then
        target_el = if EffectiveTGE() == '1' then EL2 else EL1;
    else
        target_el = PSTATE.EL;
    end;
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer{} = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_ConditionalFault);
    except.syndrome.iss[24]      = if isflt then '1' else '0';
    except.syndrome.iss[15:0]    = immediate;

    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/functions/cflt/CheckCFLTEnabled

```
// CheckCFLTEnabled()
// ==================
// Check for EL0 access to a conditional fault instruction

func CheckCFLTEnabled()
begin
    if PSTATE.EL == EL0 && !IsInHost() && !(IsSCTLR2EL1Enabled() && SCTLR2_EL1().EnCF0 == '1') then
        Undefined();
    end;

    if PSTATE.EL == EL0 && IsInHost() && !(IsSCTLR2EL2Enabled() && SCTLR2_EL2().EnCF0 == '1') then
        Undefined();
    end;
end;
```

Library pseudocode for aarch64/functions/compareop/CompareOp

```
// CompareOp
// =========
// Vector compare instruction types.

type CompareOp of enumeration {CompareOp_GT, CompareOp_GE, CompareOp_EQ,
                               CompareOp_LE, CompareOp_LT};
```

Library pseudocode for aarch64/functions/countop/CountOp

```
// CountOp
// =======
// Bit counting instruction types.

type CountOp of enumeration {CountOp_CLZ, CountOp_CLS, CountOp_CNT};
```

Library pseudocode for aarch64/functions/cpa/EffectiveCPTA

```
// EffectiveCPTA()
// ===============
// Returns the CPTA bit applied to Checked Pointer Arithmetic for Addition in the given EL.

func EffectiveCPTA(el : bits(2)) => bit
begin
    if !IsFeatureImplemented(FEAT_CPA2) then
        return '0';
    end;

    if Halted() then
        return '0';
    end;

    var cpta : bits(1);
    let regime : Regime = TranslationRegime(el);

    case regime of
        when Regime_EL3 =>
            cpta = SCTLR2_EL3().CPTA;
        when Regime_EL2 =>
            if IsSCTLR2EL2Enabled() then
                cpta = SCTLR2_EL2().CPTA;
            else
                cpta = '0';
            end;
        when Regime_EL20 =>
            if IsSCTLR2EL2Enabled() then
                cpta = if el == EL0 then SCTLR2_EL2().CPTA0 else SCTLR2_EL2().CPTA;
            else
                cpta = '0';
            end;
        when Regime_EL10 =>
            if IsSCTLR2EL1Enabled() then
                cpta = if el == EL0 then SCTLR2_EL1().CPTA0 else SCTLR2_EL1().CPTA;
            else
                cpta = '0';
            end;
        otherwise =>
            unreachable;
    end;

    return cpta;
end;
```

Library pseudocode for aarch64/functions/cpa/EffectiveCPTM

```
// EffectiveCPTM()
// ===============
// Returns the CPTM bit applied to Checked Pointer Arithmetic for Multiplication in the given EL.

func EffectiveCPTM(el : bits(2)) => bit
begin
    if !IsFeatureImplemented(FEAT_CPA2) then
        return '0';
    end;

    if EffectiveCPTA(el) == '0' then
        return '0';
    end;

    if Halted() then
        return '0';
    end;

    var cptm : bits(1);
    let regime : Regime = TranslationRegime(el);

    case regime of
        when Regime_EL3 =>
            cptm = SCTLR2_EL3().CPTM;
        when Regime_EL2 =>
            if IsSCTLR2EL2Enabled() then
                cptm = SCTLR2_EL2().CPTM;
            else
                cptm = '0';
            end;
        when Regime_EL20 =>
            if IsSCTLR2EL2Enabled() then
                cptm = if el == EL0 then SCTLR2_EL2().CPTM0 else SCTLR2_EL2().CPTM;
            else
                cptm = '0';
            end;
        when Regime_EL10 =>
            if IsSCTLR2EL1Enabled() then
                cptm = if el == EL0 then SCTLR2_EL1().CPTM0 else SCTLR2_EL1().CPTM;
            else
                cptm = '0';
            end;
        otherwise =>
            unreachable;
    end;

    return cptm;
end;
```

Library pseudocode for aarch64/functions/cpa/PointerAddCheck

```
// PointerAddCheck()
// =================
// Apply Checked Pointer Arithmetic for addition.

func PointerAddCheck(result : bits(64), base : bits(64)) => bits(64)
begin
    return PointerCheckAtEL(PSTATE.EL, result, base, FALSE);
end;
```

Library pseudocode for aarch64/functions/cpa/PointerAddCheckAtEL

```
// PointerAddCheckAtEL()
// =====================
// Apply Checked Pointer Arithmetic for addition at the specified EL.

func PointerAddCheckAtEL(el : bits(2), result : bits(64), base : bits(64)) => bits(64)
begin
    return PointerCheckAtEL(el, result, base, FALSE);
end;
```

Library pseudocode for aarch64/functions/cpa/PointerCheckAtEL

```
// PointerCheckAtEL()
// ==================
// Apply Checked Pointer Arithmetic at the specified EL.

func PointerCheckAtEL(el : bits(2), result : bits(64), base : bits(64),
                      cptm_detected : boolean) => bits(64)
begin
    var rv : bits(64)  = result;

    let previous_detection : boolean = (base[55] != base[54]);
    let cpta_detected : boolean = (result[63:56] != base[63:56] || previous_detection);

    if ((cpta_detected && EffectiveCPTA(el) == '1') ||
          (cptm_detected && EffectiveCPTM(el) == '1')) then
        rv[63:55] = base[63:55];
        rv[54] = NOT(rv[55]);
    end;

    return rv;
end;
```

Library pseudocode for aarch64/functions/cpa/PointerMultiplyAddCheck

```
// PointerMultiplyAddCheck()
// =========================
// Apply Checked Pointer Arithmetic for multiplication.

func PointerMultiplyAddCheck(result : bits(64), base : bits(64),
                             cptm_detected : boolean) => bits(64)
begin
    return PointerCheckAtEL(PSTATE.EL, result, base, cptm_detected);
end;
```

Library pseudocode for aarch64/functions/d128/IsD128Enabled

```
// IsD128Enabled()
// ===============
// Returns true if 128-bit page descriptor is enabled

func IsD128Enabled(el : bits(2)) => boolean
begin
    var d128enabled : boolean;
    if IsFeatureImplemented(FEAT_D128) then
        case el of
            when EL0 =>
                if !ELIsInHost(EL0) then
                    d128enabled = IsTCR2EL1Enabled() && TCR2_EL1().D128 == '1';
                else
                    d128enabled = IsTCR2EL2Enabled() && TCR2_EL2().D128 == '1';
                end;
            when EL1 =>
                d128enabled = IsTCR2EL1Enabled() && TCR2_EL1().D128 == '1';
            when EL2 =>
                d128enabled = IsTCR2EL2Enabled() && IsInHost() && TCR2_EL2().D128 == '1';
            when EL3 =>
                d128enabled = TCR_EL3().D128 == '1';
        end;
    else
        d128enabled = FALSE;
    end;

    return d128enabled;
end;
```

Library pseudocode for aarch64/functions/dc/AArch64_CanTrapDC

```
// AArch64_CanTrapDC()
// ===================
// Determines whether the execution of the DC instruction can be trapped.

func AArch64_CanTrapDC(cachetype : CacheType, cacheop : CacheOp, opscope : CacheOpScope) => boolean
begin
    return (!AArch64_TreatDCAsNOP(cachetype, cacheop, opscope) || ImpDefBool(
             "When DC is treated as NOP, data cache maintenance operations are trapped"));
end;
```

Library pseudocode for aarch64/functions/dc/AArch64_DC

```
// AArch64_DC()
// ============
// Perform Data Cache Operation.

func AArch64_DC(regval : bits(64), cachetype : CacheType,
                cacheop : CacheOp, opscope_in : CacheOpScope)
begin
    var opscope : CacheOpScope = opscope_in;
    var cache : CacheRecord;

    cache.acctype   = AccessType_DC;
    cache.cachetype = cachetype;
    cache.cacheop   = cacheop;
    cache.opscope   = opscope;

    if opscope == CacheOpScope_SetWay then
        let ss : SecurityState = SecurityStateAtEL(PSTATE.EL);
        cache.cpas = CPASAtSecurityState(ss);
        cache.shareability = Shareability_NSH;
        cache.(setnum, waynum, level) = DecodeSW(regval, cachetype);
        if (cacheop == CacheOp_Invalidate && PSTATE.EL == EL1 && EL2Enabled() &&
          (HCR_EL2().SWIO == '1' || HCR_EL2().[DC,VM] != '00')) then
            cache.cacheop = CacheOp_CleanInvalidate;
        end;

        CACHE_OP(cache);
        return;
    end;

    if EL2Enabled() && !IsInHost() then
        if PSTATE.EL IN {EL0, EL1} then
            cache.is_vmid_valid = TRUE;
            cache.vmid          = VMID();
        else
            cache.is_vmid_valid = FALSE;
        end;
    else
        cache.is_vmid_valid = FALSE;
    end;

    if PSTATE.EL == EL0 then
        cache.is_asid_valid = TRUE;
        cache.asid          = ASID();
    else
        cache.is_asid_valid = FALSE;
    end;

    if (opscope == CacheOpScope_PoPS &&
          !ImpDefBool("Memory system supports PoPS")) then
        opscope = CacheOpScope_PoC;
    end;
    if (opscope == CacheOpScope_PoDP &&
          !ImpDefBool("Memory system supports PoDP")) then
        opscope = CacheOpScope_PoP;
    end;
    if (opscope == CacheOpScope_PoP &&
          !ImpDefBool("Memory system supports PoP")) then
        opscope = CacheOpScope_PoC;
    end;
    var vaddress : bits(64) = regval;

    var size : integer = 0;        // by default no watchpoint address
    if cacheop == CacheOp_Invalidate then
        size = DataCacheWatchpointSize();
        vaddress = AlignDownSize{64}(regval, size as AddressSize);
    end;

    if opscope == CacheOpScope_PoE then
        cache.shareability     = Shareability_OSH;
        cache.paddress.address = regval[NUM_PABITS-1:0];
        let nse2 : bit = if IsFeatureImplemented(FEAT_RME_GDI) then regval[61] else '0';
        cache.paddress.paspace = DecodePASpace(nse2, regval[62], regval[63]);
        cache.cpas             = CPASAtPAS(cache.paddress.paspace);

        // If a Reserved encoding is selected, the instruction is permitted to be treated as a NOP.
        if cache.paddress.paspace != PAS_Realm then
            ExecuteAsNOP();
        end;

        if ImpDefBool("Apply granule protection check on DC to PoE") then
            var memaddrdesc : AddressDescriptor;
            let accdesc : AccessDescriptor = CreateAccDescDC(cache);
            memaddrdesc.paddress     = cache.paddress;
            memaddrdesc.fault.gpcf   = GranuleProtectionCheck(memaddrdesc, accdesc);

            if memaddrdesc.fault.gpcf.gpf != GPCF_None then
                memaddrdesc.fault.statuscode = Fault_GPCFOnOutput;
                memaddrdesc.fault.paddress   = memaddrdesc.paddress;
                memaddrdesc.fault.vaddress   = ARBITRARY : bits(64);
```

```
                AArch64_Abort(memaddrdesc.fault);
            end;
        end;
    elsif opscope == CacheOpScope_PoPA then
        cache.shareability = Shareability_OSH;
        cache.paddress.address = regval[NUM_PABITS-1:0];
        let nse2 : bit = if IsFeatureImplemented(FEAT_RME_GDI) then regval[61] else '0';
        cache.paddress.paspace = DecodePASpace(nse2, regval[62], regval[63]);

        // If {NSE2, NSE, NS} is reserved, then no cache entries are required
        // to be cleaned or invalidated.
        if cache.paddress.paspace IN {PAS_NA6, PAS_NA7} then
            ExecuteAsNOP();
        end;
        cache.cpas = CPASAtPAS(cache.paddress.paspace);
    else
        cache.vaddress  = vaddress;
        let aligned : boolean = TRUE;
        let accdesc : AccessDescriptor = CreateAccDescDC(cache);
        var memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(vaddress, accdesc,
                                                                       aligned, size);
        if IsFault(memaddrdesc) then
            memaddrdesc.fault.vaddress = regval;
            AArch64_Abort(memaddrdesc.fault);
        end;

        cache.paddress   = memaddrdesc.paddress;
        cache.cpas       = CPASAtPAS(memaddrdesc.paddress.paspace);

        if (opscope IN {
                        CacheOpScope_PoDP,
                        CacheOpScope_PoP,
                        CacheOpScope_PoC,
                        CacheOpScope_PoU
                    }) then
            cache.shareability = memaddrdesc.mematttrs.shareability;
        else
            cache.shareability = Shareability_NSH;
        end;
    end;

    if (cacheop == CacheOp_Invalidate && PSTATE.EL == EL1 && EL2Enabled() &&
            HCR_EL2().[DC,VM] != '00') then
        cache.cacheop = CacheOp_CleanInvalidate;
    end;

    // If Secure state is not implemented, but RME is, the instruction acts as a NOP
    if cache.cpas == CPAS_Secure && !HaveSecureState() then
        return;
    end;

    CACHE_OP(cache);
    return;
end;
```

Library pseudocode for aarch64/functions/dc/AArch64_MemZero

```
// AArch64_MemZero()
// =================

func AArch64_MemZero(regval : bits(64), cachetype : CacheType)
begin
    let size : integer{} = (if cachetype IN {CacheType_TagWrite, CacheType_TagZero}
                            then 4*(2^(UInt(DCZID_EL0().TBS)))
                            else 4*(2^(UInt(DCZID_EL0().BS))));
    assert size <= MAX_ZERO_BLOCK_SIZE;
    if IsFeatureImplemented(FEAT_MTE2) then
        assert size >= TAG_GRANULE;
    end;
    let vaddress : bits(64) = AlignDownSize{}(regval, size);

    let accdesc : AccessDescriptor = CreateAccDescDCZero(cachetype);

    if cachetype != CacheType_Data then
        AArch64_WriteTagMem(regval, cachetype, accdesc, size);
    end;

    if cachetype IN {CacheType_Data, CacheType_Data_Tag} then
        AArch64_DataMemZero(regval, vaddress, accdesc, size);
    end;
    return;
end;
```

Library pseudocode for aarch64/functions/dc/AArch64_TreatDCAsNOP

```
// AArch64_TreatDCAsNOP()
// ======================
// Determines whether the execution of the DC instruction is treated as a NOP.

func AArch64_TreatDCAsNOP(cachetype : CacheType, cacheop : CacheOp,
                          opscope : CacheOpScope) => boolean
begin
    // DC to PoU: IMPLEMENTATION DEFINED - treated as NOP if LoUU and LoUIS are 0
    if opscope == CacheOpScope_PoU && CLIDR_EL1().LoUU == '000' && CLIDR_EL1().LoUIS == '000' then
        return ImpDefBool("DC to PoU is treated as a NOP");
    end;

    // DC to PoC: IMPLEMENTATION DEFINED - treated as NOP if LoC is 0
    if opscope == CacheOpScope_PoC && CLIDR_EL1().LoC == '000' then
        return ImpDefBool("DC to PoC is treated as a NOP");
    end;

    // DC to PoP: IMPLEMENTATION DEFINED - treated as NOP if LoC is 0 and PoP unsupported
    if (opscope == CacheOpScope_PoP && CLIDR_EL1().LoC == '000' &&
          !ImpDefBool("Memory system supports PoP")) then
        return ImpDefBool("DC to PoP is treated as a NOP");
    end;

    // DC to PoDP: IMPLEMENTATION DEFINED - treated as NOP if LoC is 0 and PoP/PoDP unsupported
    if (opscope == CacheOpScope_PoDP && CLIDR_EL1().LoC == '000' &&
          !ImpDefBool("Memory system supports PoP") &&
          !ImpDefBool("Memory system supports PoDP")) then
        return ImpDefBool("DC to PoDP is treated as a NOP");
    end;

    // DC operating on Allocation tags: IMPLEMENTATION DEFINED - treated as NOP
    // if FEAT_MTE is implemented and FEAT_MTE2 is not implemented
    if cachetype == CacheType_Tag && !IsFeatureImplemented(FEAT_MTE2) then
        return ImpDefBool("DC on Allocation tags without FEAT_MTE2 is treated as a NOP");
    end;

    return FALSE;
end;
```

Library pseudocode for aarch64/functions/dc/MemZero

```
// MemZero block size
// ==================

constant MAX_ZERO_BLOCK_SIZE : integer = 2048;
```

Library pseudocode for aarch64/functions/eret/AArch64_ExceptionReturn

```
// AArch64_ExceptionReturn()
// =========================

func AArch64_ExceptionReturn(new_pc_in : bits(64), spsr : bits(64))
begin
    var new_pc : bits(64) = new_pc_in;

    if IsFeatureImplemented(FEAT_IESB) then
        var sync_errors : boolean = SCTLR_ELx().IESB == '1';
        if IsFeatureImplemented(FEAT_DoubleFault) then
            sync_errors = sync_errors || (SCR_EL3().[EA,NMEA] == '11' && PSTATE.EL == EL3);
        end;
        if sync_errors then
            SynchronizeErrors();
            // If FEAT_IESBv1p1 is implemented and an implicit error synchronization barrier
            // on exception return generates an SError exception, the resulting ESR_ELx.IESB is
            // set to 0. Otherwise, the value is an IMPLEMENTATION DEFINED choice of 0 or 1.
            let iesb_req : boolean = (!IsFeatureImplemented(FEAT_IESBv1p1) && ImpDefBool(
                     "ESR_ELx.IESB is 1 after Implicit Error Synchronization on exception return"));
            TakeUnmaskedPhysicalSErrorInterrupts(iesb_req);
        end;
    end;

    var brbe_source_allowed : boolean = FALSE;
    var brbe_source_address : bits(64) = Zeros{};
    if IsFeatureImplemented(FEAT_BRBE) then
        brbe_source_allowed = BranchRecordAllowed(PSTATE.EL);
        brbe_source_address = PC64();
    end;


    if !IsFeatureImplemented(FEAT_ExS) || SCTLR_ELx().EOS == '1' then
        SynchronizeContext();
    end;

    // Attempts to change to an illegal state will invoke the Illegal Execution state mechanism
    let source_el : bits(2) = PSTATE.EL;
    let illegal_psr_state : boolean = IllegalExceptionReturn{64}(spsr);
    SetPSTATEFromPSR{64}(spsr, illegal_psr_state);
    ClearExclusiveLocal(ProcessorID());
    SendEventLocal();

    if illegal_psr_state && spsr[4] == '1' then
        // If the exception return is illegal, PC[63:32,1:0] are UNKNOWN
        new_pc[63:32] = ARBITRARY : bits(32);
        new_pc[1:0] = ARBITRARY : bits(2);
    elsif UsingAArch32() then               // Return to AArch32
        // ELR_ELx[1:0] or ELR_ELx[0] are treated as being 0, depending on the
        // target instruction set state
        if PSTATE.T == '1' then
            new_pc[0] = '0';                // T32
        else
            new_pc[1:0] = '00';             // A32
        end;
    else                                    // Return to AArch64
        // ELR_ELx[63:56] might include a tag
        new_pc = AArch64_BranchAddr(new_pc, PSTATE.EL);
    end;

    if IsFeatureImplemented(FEAT_BRBE) then
        BRBEExceptionReturn(new_pc, source_el,
                            brbe_source_allowed, brbe_source_address);
    end;


    if UsingAArch32() then
        if IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then ResetSVEState(); end;

        // 32 most significant bits are ignored.
        let branch_conditional : boolean = FALSE;
        BranchTo{32}(new_pc[31:0], BranchType_ERET, branch_conditional);
    else
        BranchToAddr{64}(new_pc, BranchType_ERET);
    end;

    CheckExceptionCatch(FALSE);             // Check for debug event on exception return
end;
```

Library pseudocode for aarch64/functions/exclusive/AArch64_ExclusiveMonitorsPass

```
// AArch64_ExclusiveMonitorsPass()
// ===============================
// Return TRUE if the Exclusives monitors for the current PE include all of the addresses
// associated with the virtual address region of size bytes starting at address.
// The immediately following memory write must be to the same addresses.

// It is IMPLEMENTATION DEFINED whether the detection of memory aborts happens
// before or after the check on the local Exclusives monitor. As a result a failure
// of the local monitor can occur on some implementations even if the memory
// access would give a memory abort.


func AArch64_ExclusiveMonitorsPass(address : bits(64), size : integer{1, 2, 4, 8, 16, 32},
                                   accdesc : AccessDescriptor) => boolean
begin
    let aligned : boolean = IsAlignedSize(address, size);

    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, size) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    if !AArch64_IsExclusiveVA(address, ProcessorID(), size) then
        return FALSE;
    end;

    let memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(address, accdesc, aligned, size);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    var passed : boolean = IsExclusiveLocal(memaddrdesc.paddress, ProcessorID(), size);
    ClearExclusiveLocal(ProcessorID());

    if passed && memaddrdesc.memattrs.shareability != Shareability_NSH then
        passed = IsExclusiveGlobal(memaddrdesc.paddress, ProcessorID(), size);
    end;

    return passed;
end;
```

Library pseudocode for aarch64/functions/exclusive/AArch64_IsExclusiveVA

```
// AArch64_IsExclusiveVA()
// =======================
// An optional IMPLEMENTATION DEFINED test for an exclusive access to a virtual
// address region of size bytes starting at address.
//
// It is permitted (but not required) for this function to return FALSE and
// cause a store exclusive to fail if the virtual address region is not
// totally included within the region recorded by MarkExclusiveVA().
//
// It is always safe to return TRUE which will check the physical address only.


impdef func AArch64_IsExclusiveVA(address : bits(64),
                                  processorid : integer,
                                  size : integer) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/exclusive/AArch64_MarkExclusiveVA

```
// AArch64_MarkExclusiveVA()
// =========================
// Optionally record an exclusive access to the virtual address region of size bytes
// starting at address for processorid.

func AArch64_MarkExclusiveVA(address : bits(64), processorid : integer, size : integer)
begin
    return;
end;
```

Library pseudocode for aarch64/functions/exclusive/AArch64_SetExclusiveMonitors

```
// AArch64_SetExclusiveMonitors()
// ==============================
// Sets the Exclusives monitors for the current PE to record the addresses associated
// with the virtual address region of size bytes starting at address.

func AArch64_SetExclusiveMonitors(address : bits(64), size : integer{1, 2, 4, 8, 16, 32},
                                  accdesc : AccessDescriptor)
begin
    let aligned : boolean = IsAlignedSize(address, size);

    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, size) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    let memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(address, accdesc, aligned, size);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return;
    end;

    if memaddrdesc.memattrs.shareability != Shareability_NSH then
        MarkExclusiveGlobal(memaddrdesc.paddress, ProcessorID(), size);
    end;

    MarkExclusiveLocal(memaddrdesc.paddress, ProcessorID(), size);

    AArch64_MarkExclusiveVA(address, ProcessorID(), size);
end;
```

Library pseudocode for aarch64/functions/extendreg/DecodeRegExtend

```
// DecodeRegExtend()
// =================
// Decode a register extension option

func DecodeRegExtend(op : bits(3)) => ExtendType
begin
    case op of
        when '000' => return ExtendType_UXTB;
        when '001' => return ExtendType_UXTH;
        when '010' => return ExtendType_UXTW;
        when '011' => return ExtendType_UXTX;
        when '100' => return ExtendType_SXTB;
        when '101' => return ExtendType_SXTH;
        when '110' => return ExtendType_SXTW;
        when '111' => return ExtendType_SXTX;
    end;
end;
```

Library pseudocode for aarch64/functions/extendreg/ExtendReg

```
// ExtendReg()
// ===========
// Perform a register extension and shift

func ExtendReg{N}(reg : integer, exttype : ExtendType, shift : integer{0..4}) => bits(N)
begin
    let val : bits(N) = X{}(reg);
    var unsigned : boolean;
    var len : ESize;

    case exttype of
        when ExtendType_SXTB => unsigned = FALSE; len = 8;
        when ExtendType_SXTH => unsigned = FALSE; len = 16;
        when ExtendType_SXTW => unsigned = FALSE; len = 32;
        when ExtendType_SXTX => unsigned = FALSE; len = 64;
        when ExtendType_UXTB => unsigned = TRUE;  len = 8;
        when ExtendType_UXTH => unsigned = TRUE;  len = 16;
        when ExtendType_UXTW => unsigned = TRUE;  len = 32;
        when ExtendType_UXTX => unsigned = TRUE;  len = 64;
    end;

    // Sign or zero extend bottom LEN bits of register and shift left by SHIFT
    let nbits : integer{} = Min(len, N) as integer{8..N};
    let extval : bits(N) = Extend{}(val[nbits-1:0], unsigned);
    return LSL(extval, shift);
end;
```

Library pseudocode for aarch64/functions/extendreg/ExtendType

```
// ExtendType
// ==========
// AArch64 register extend and shift.

type ExtendType of enumeration {ExtendType_SXTB, ExtendType_SXTH, ExtendType_SXTW, ExtendType_SXTX,
                                ExtendType_UXTB, ExtendType_UXTH, ExtendType_UXTW, ExtendType_UXTX};
```

Library pseudocode for aarch64/functions/fpconvop/FPConvOp

```
// FPConvOp
// ========
// Floating-point convert/move instruction types.

type FPConvOp of enumeration {FPConvOp_CVT_FtoI, FPConvOp_CVT_ItoF,
                              FPConvOp_MOV_FtoI, FPConvOp_MOV_ItoF,
                              FPConvOp_CVT_FtoI_JS
};
```

Library pseudocode for aarch64/functions/fpmaxminop/FPMaxMinOp

```
// FPMaxMinOp
// ==========
// Floating-point min/max instruction types.

type FPMaxMinOp of enumeration {FPMaxMinOp_MAX, FPMaxMinOp_MIN,
                                FPMaxMinOp_MAXNUM, FPMaxMinOp_MINNUM};
```

Library pseudocode for aarch64/functions/fpmr/CheckFPMREnabled

```
// CheckFPMREnabled()
// ==================
// Check for Undefined instruction exception on indirect FPMR accesses.

func CheckFPMREnabled()
begin
    assert IsFeatureImplemented(FEAT_FPMR);

    if PSTATE.EL == EL0 then
        if !IsInHost() then
            if SCTLR_EL1().EnFPM == '0' then Undefined(); end;
        else
            if SCTLR_EL2().EnFPM == '0' then Undefined(); end;
        end;
    end;

    if PSTATE.EL IN {EL0, EL1} && EL2Enabled() && !IsInHost() then
        if !IsHCRXEL2Enabled() || HCRX_EL2().EnFPM == '0' then Undefined(); end;
    end;

    if PSTATE.EL != EL3 && HaveEL(EL3) then
        if SCR_EL3().EnFPM == '0' then Undefined(); end;
    end;
end;
```

Library pseudocode for aarch64/functions/fpscale/FPClampScale

```
// FPClampScale()
// ==============

func FPClampScale{N}(op : bits(N), scale_in : integer) => integer
begin
    assert N IN {16,32,64};
    var E : integer;
    var exp : integer;
    case N of
        when 16 =>
            E = 5;
            exp = UInt(op[14:10]);
        when 32 =>
            E = 8;
            exp = UInt(op[30:23]);
        when 64 =>
            E = 11;
            exp = UInt(op[62:52]);
        otherwise => unreachable;
    end;

    let F : integer = N - (E + 1);
    let emax : integer = (1 << E) - 1;
    let min_scale : integer = -(F + 1);
    let max_scale : integer = emax + (F + 1);

    let scale : integer = Max(min_scale - exp, Min(scale_in, max_scale - exp));
    return scale;
end;
```

Library pseudocode for aarch64/functions/fpscale/FPScale

```
// FPScale()
// =========

func FPScale{N}(op : bits(N), scale : integer, fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var result : bits(N);
    let (fptype,sign,value) : (FPType, bit, real) = FPUnpack{N}(op, fpcr);

    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        result = FPProcessNaN{N}(fptype, op, fpcr);
    elsif fptype == FPType_Zero then
        result = FPZero{N}(sign);
    elsif fptype == FPType_Infinity then
        result = FPInfinity{N}(sign);
    else
        let clamped_scale : integer = FPClampScale{N}(op, scale);
        result = FPRound{N}(value * (2.0^clamped_scale), fpcr);
        FPProcessDenorm(fptype, N, fpcr);
    end;
    return result;
end;
```

Library pseudocode for aarch64/functions/fpunaryop/FPUnaryOp

```
// FPUnaryOp
// =========
// Floating-point unary instruction types.

type FPUnaryOp of enumeration {FPUnaryOp_ABS, FPUnaryOp_MOV,
                               FPUnaryOp_NEG, FPUnaryOp_SQRT};
```

Library pseudocode for aarch64/functions/fusedrstep/FPRSqrtStepFused

```
// FPRSqrtStepFused()
// ==================

func FPRSqrtStepFused{N}(op1_in : bits(N), op2 : bits(N), fpcr_in : FPCR_Type) => bits(N)
begin
    assert N IN {16, 32, 64};
    var fpcr : FPCR_Type = fpcr_in;
    var result : bits(N);
    var op1 : bits(N) = op1_in;
    var done : boolean;
    op1 = FPNeg{N}(op1, fpcr);
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && fpcr.AH == '1';
    let fpexc : boolean = !altfp;                    // Generate no floating-point exceptions
    if altfp then fpcr.[FIZ,FZ] = '11'; end;         // Flush denormal input and output to zero
    if altfp then fpcr.RMode = '00'; end;            // Use RNE rounding mode

    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr, fpexc);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr, fpexc);
    (done,result) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr, fpexc);
    let rounding : FPRounding = FPRoundingMode(fpcr);

    if !done then
        let inf1 : boolean = (type1 == FPType_Infinity);
        let inf2 : boolean = (type2 == FPType_Infinity);
        let zero1 : boolean = (type1 == FPType_Zero);
        let zero2 : boolean = (type2 == FPType_Zero);

        if (inf1 && zero2) || (zero1 && inf2) then
            result = FPOnePointFive{N}('0');
        elsif inf1 || inf2 then
            result = FPInfinity{N}(sign1 XOR sign2);
        else
            // Fully fused multiply-add and halve
            let result_value : real = (3.0 + (value1 * value2)) / 2.0;
            if result_value == 0.0 then
                // Sign of exact zero result depends on rounding mode
                let sign : bit = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{N}(sign);
            else
                result = FPRound{N}(result_value, fpcr, rounding, fpexc);
            end;
        end;
    end;

    return result;
end;
```

Library pseudocode for aarch64/functions/fusedrstep/FPRecipStepFused

```
// FPRecipStepFused()
// ==================

func FPRecipStepFused{N}(op1_in : bits(N), op2 : bits(N), fpcr_in : FPCR_Type) => bits(N)
begin
    assert N IN {16, 32, 64};
    var fpcr : FPCR_Type = fpcr_in;
    var op1 : bits(N)    = op1_in;
    var result : bits(N);
    var done : boolean;
    op1 = FPNeg{N}(op1, fpcr);

    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && fpcr.AH == '1';
    let fpexc : boolean = !altfp;                    // Generate no floating-point exceptions
    if altfp then fpcr.[FIZ,FZ] = '11'; end;         // Flush denormal input and output to zero
    if altfp then fpcr.RMode    = '00'; end;         // Use RNE rounding mode

    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr, fpexc);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr, fpexc);
    (done,result) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr, fpexc);
    let rounding : FPRounding = FPRoundingMode(fpcr);

    if !done then
        let inf1 : boolean  = (type1 == FPType_Infinity);
        let inf2 : boolean  = (type2 == FPType_Infinity);
        let zero1 : boolean = (type1 == FPType_Zero);
        let zero2 : boolean = (type2 == FPType_Zero);

        if (inf1 && zero2) || (zero1 && inf2) then
            result = FPTwo{N}('0');
        elsif inf1 || inf2 then
            result = FPInfinity{N}(sign1 XOR sign2);
        else
            // Fully fused multiply-add
            let result_value : real = 2.0 + (value1 * value2);
            if result_value == 0.0 then
                // Sign of exact zero result depends on rounding mode
                let sign : bit = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{N}(sign);
            else
                result = FPRound{N}(result_value, fpcr, rounding, fpexc);
            end;
        end;
    end;

    return result;
end;
```

Library pseudocode for aarch64/functions/gcs/AddGCSExRecord

```
// AddGCSExRecord()
// ================
// Generates and then writes an exception record to the
// current Guarded Control Stack.

func AddGCSExRecord(elr : bits(64), spsr : bits(64), lr : bits(64))
begin
    var ptr : bits(64);
    let privileged : boolean = PSTATE.EL != EL0;
    let accdesc : AccessDescriptor = CreateAccDescGCS(MemOp_STORE, privileged);

    ptr = GetCurrentGCSPointer();

    // Store the record
    Mem{64}(ptr-8, accdesc) = lr;
    Mem{64}(ptr-16, accdesc) = spsr;
    Mem{64}(ptr-24, accdesc) = elr;
    Mem{64}(ptr-32, accdesc) = Zeros{60}::'1001';

    // Decrement the pointer value
    ptr = ptr - 32;

    SetCurrentGCSPointer(ptr);
    return;
end;
```

Library pseudocode for aarch64/functions/gcs/AddGCSRecord

```
// AddGCSRecord()
// ==============
// Generates and then writes a record to the current Guarded
// control stack.

func AddGCSRecord(vaddress : bits(64))
begin
    var ptr : bits(64);
    let privileged : boolean = PSTATE.EL != EL0;
    let accdesc : AccessDescriptor = CreateAccDescGCS(MemOp_STORE, privileged);

    ptr = GetCurrentGCSPointer();

    // Store the record
    Mem{64}(ptr-8, accdesc) = vaddress;

    // Decrement the pointer value
    ptr = ptr - 8;

    SetCurrentGCSPointer(ptr);
    return;
end;
```

Library pseudocode for aarch64/functions/gcs/CheckGCSExRecord

```
// CheckGCSExRecord()
// ==================
// Validates the provided values against the top entry of the
// current Guarded Control Stack.

func CheckGCSExRecord(elr : bits(64), spsr : bits(64), lr : bits(64), gcsinst_type : GCSInstruction)
begin
    var ptr : bits(64);
    let privileged : boolean = PSTATE.EL != EL0;
    let accdesc : AccessDescriptor = CreateAccDescGCS(MemOp_LOAD, privileged);
    ptr = GetCurrentGCSPointer();

    // Check the lowest doubleword is correctly formatted
    let recorded_first_dword : bits(64) = Mem{64}(ptr, accdesc);
    if recorded_first_dword != Zeros{60}::'1001' then
        GCSDataCheckException(gcsinst_type);
    end;

    // Check the ELR matches the recorded value
    let recorded_elr : bits(64) = Mem{64}(ptr+8, accdesc);
    if recorded_elr != elr then
        GCSDataCheckException(gcsinst_type);
    end;

    // Check the SPSR matches the recorded value
    let recorded_spsr : bits(64) = Mem{64}(ptr+16, accdesc);
    if recorded_spsr != spsr then
        GCSDataCheckException(gcsinst_type);
    end;

    // Check the LR matches the recorded value
    let recorded_lr : bits(64) = Mem{64}(ptr+24, accdesc);
    if recorded_lr != lr then
        GCSDataCheckException(gcsinst_type);
    end;

    // Increment the pointer value
    ptr = ptr + 32;

    SetCurrentGCSPointer(ptr);
    return;
end;
```

Library pseudocode for aarch64/functions/gcs/CheckGCSSTREnabled

```
// CheckGCSSTREnabled()
// ====================
// Trap GCSSTR or GCSSTTR instruction if trapping is enabled.

func CheckGCSSTREnabled()
begin
    case PSTATE.EL of
        when EL0 =>
            if GCSCRE0_EL1().STREn == '0' then
                if EL2Enabled() && HCR_EL2().TGE == '1' then
                    GCSSTRTrapException(EL2);
                else
                    GCSSTRTrapException(EL1);
                end;
            end;
        when EL1 =>
            if GCSCR_EL1().STREn == '0' then
                GCSSTRTrapException(EL1);
            elsif (EL2Enabled() && (!HaveEL(EL3) || SCR_EL3().FGTEn == '1') &&
                    HFGITR_EL2().nGCSSTR_EL1 == '0') then
                GCSSTRTrapException(EL2);
            end;
        when EL2 =>
            if GCSCR_EL2().STREn == '0' then
                GCSSTRTrapException(EL2);
            end;
        when EL3 =>
            if GCSCR_EL3().STREn == '0' then
                GCSSTRTrapException(EL3);
            end;
    end;
    return;
end;
```

Library pseudocode for aarch64/functions/gcs/EXLOCKException

```
// EXLOCKException()
// =================
// Handle an EXLOCK exception condition.

func EXLOCKException()
begin
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_GCSFail);
    except.syndrome.iss[24] = '0';
    except.syndrome.iss[23:20] = '0001';
    except.syndrome.iss[19:0] = Zeros{20};
    AArch64_TakeException(PSTATE.EL, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/functions/gcs/GCSDataCheckException

```
// GCSDataCheckException()
// ======================
// Handle a GCS data check fault condition.

func GCSDataCheckException(gcsinst_type : GCSInstruction)
begin
    var target_el : bits(2);
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;
    var rn_unknown : boolean  = FALSE;
    var is_ret : boolean      = FALSE;
    var is_reta : boolean     = FALSE;

    if PSTATE.EL == EL0 then
        target_el = if (EL2Enabled() && HCR_EL2().TGE == '1') then EL2 else EL1;
    else
        target_el = PSTATE.EL;
    end;
    var except : ExceptionRecord = ExceptionSyndrome(Exception_GCSFail);
    case gcsinst_type of
        when GCSInstType_PRET =>
            except.syndrome.iss[4:0] = '00000';
            is_ret = TRUE;
        when GCSInstType_POPM =>
            except.syndrome.iss[4:0] = '00001';
        when GCSInstType_PRETAA =>
            except.syndrome.iss[4:0] = '00010';
            is_reta = TRUE;
        when GCSInstType_PRETAB =>
            except.syndrome.iss[4:0] = '00011';
            is_reta = TRUE;
        when GCSInstType_SS1 =>
            except.syndrome.iss[4:0] = '00100';
        when GCSInstType_SS2 =>
            except.syndrome.iss[4:0] = '00101';
            rn_unknown = TRUE;
        when GCSInstType_POPCX =>
            rn_unknown = TRUE;
            except.syndrome.iss[4:0] = '01000';
        when GCSInstType_POPX =>
            except.syndrome.iss[4:0] = '01001';
    end;
    if rn_unknown == TRUE then
        except.syndrome.iss[9:5] = ARBITRARY : bits(5);
    elsif is_ret == TRUE then
        except.syndrome.iss[9:5] = ThisInstr()[9:5];
    elsif is_reta == TRUE then
        except.syndrome.iss[9:5] = '11110';
    else
        except.syndrome.iss[9:5] = ThisInstr()[4:0];
    end;
    except.syndrome.iss[24:10] = Zeros{15};
    except.vaddress = ARBITRARY : bits(64);
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/functions/gcs/GCSEnabled

```
// GCSEnabled()
// ============
// Returns TRUE if the Guarded Control Stack is enabled at
// the provided Exception level.

func GCSEnabled(el : bits(2)) => boolean
begin
    if UsingAArch32() then
        return FALSE;
    end;

    if HaveEL(EL3) && el != EL3 && SCR_EL3().GCSEn == '0' then
        return FALSE;
    end;

    if (el IN {EL0, EL1} && EL2Enabled() && !ELIsInHost(EL0) &&
        (!IsHCRXEL2Enabled() || HCRX_EL2().GCSEn == '0')) then
        return FALSE;
    end;

    return GCSPCRSelected(el);
end;
```

Library pseudocode for aarch64/functions/gcs/GCSInstruction

```
// GCSInstruction
// ==============

type GCSInstruction of enumeration {
    GCSInstType_PRET,    // Procedure return without Pointer authentication
    GCSInstType_POPM,    // GCSPOPM instruction
    GCSInstType_PRETAA,  // Procedure return with Pointer authentication that used key A
    GCSInstType_PRETAB,  // Procedure return with Pointer authentication that used key B
    GCSInstType_SS1,     // GCSSS1 instruction
    GCSInstType_SS2,     // GCSSS2 instruction
    GCSInstType_POPCX,   // GCSPOPCX instruction
    GCSInstType_POPX     // GCSPOPX instruction
};
```

Library pseudocode for aarch64/functions/gcs/GCSPCREnabled

```
// GCSPCREnabled()
// ===============
// Returns TRUE if the Guarded Control Stack is PCR enabled
// at the provided Exception level.

func GCSPCREnabled(el : bits(2)) => boolean
begin
    return GCSPCRSelected(el) && GCSEnabled(el);
end;
```

Library pseudocode for aarch64/functions/gcs/GCSPCRSelected

```
// GCSPCRSelected()
// ================
// Returns TRUE if the Guarded Control Stack is PCR selected
// at the provided Exception level.

func GCSPCRSelected(el : bits(2)) => boolean
begin
    case el of
        when EL0 => return GCSCRE0_EL1().PCRSEL == '1';
        when EL1 => return GCSCR_EL1().PCRSEL == '1';
        when EL2 => return GCSCR_EL2().PCRSEL == '1';
        when EL3 => return GCSCR_EL3().PCRSEL == '1';
    end;
    unreachable;
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/gcs/GCSPOPCX

```
// GCSPOPCX()
// ==========
// Called to pop and compare a GCS exception return record.

func GCSPOPCX()
begin
    let spsr : bits(64) = SPSR_ELx();
    CheckGCSExRecord(ELR_ELx(), spsr, X{64}(30), GCSInstType_POPCX);
    PSTATE.EXLOCK = if GetCurrentEXLOCKEN() then '1' else '0';
    return;
end;
```

Library pseudocode for aarch64/functions/gcs/GCSPOPM

```
// GCSPOPM()
// =========
// Called to pop a GCS procedure return record.

func GCSPOPM() => bits(64)
begin
    var ptr : bits(64);
    let privileged : boolean = PSTATE.EL != EL0;
    let accdesc : AccessDescriptor = CreateAccDescGCS(MemOp_LOAD, privileged);

    ptr = GetCurrentGCSPointer();
    let gcs_entry : bits(64) = Mem{64}(ptr, accdesc);

    if gcs_entry[1:0] != '00' then
        GCSDataCheckException(GCSInstType_POPM);
    end;

    ptr = ptr + 8;
    SetCurrentGCSPointer(ptr);
    return gcs_entry;
end;
```

Library pseudocode for aarch64/functions/gcs/GCSPOPX

```
// GCSPOPX()
// =========
// Called to pop a GCS exception return record.

func GCSPOPX()
begin
    var ptr : bits(64);
    let privileged : boolean = PSTATE.EL != EL0;
    let accdesc : AccessDescriptor = CreateAccDescGCS(MemOp_LOAD, privileged);
    ptr = GetCurrentGCSPointer();

    // Check the lowest doubleword is correctly formatted
    let recorded_first_dword : bits(64) = Mem{64}(ptr, accdesc);
    if recorded_first_dword != Zeros{60}::'1001' then
        GCSDataCheckException(GCSInstType_POPX);
    end;

    // Ignore these loaded values, however they might have
    // faulted which is why we load them anyway
    let recorded_elr : bits(64) = Mem{64}(ptr+8, accdesc);
    let recorded_spsr : bits(64) = Mem{64}(ptr+16, accdesc);
    let recorded_lr : bits(64) = Mem{64}(ptr+24, accdesc);

    // Increment the pointer value
    ptr = ptr + 32;

    SetCurrentGCSPointer(ptr);
    return;
end;
```

Library pseudocode for aarch64/functions/gcs/GCSPUSHM

```
// GCSPUSHM()
// ==========
// Called to push a GCS procedure return record.

func GCSPUSHM(value : bits(64))
begin
    AddGCSRecord(value);
    return;
end;
```

Library pseudocode for aarch64/functions/gcs/GCSPUSHX

```
// GCSPUSHX()
// ==========
// Called to push a GCS exception return record.

func GCSPUSHX()
begin
    let spsr : bits(64) = SPSR_ELx();
    AddGCSExRecord(ELR_ELx(), spsr, X{64}(30));
    PSTATE.EXLOCK = '0';
    return;
end;
```

Library pseudocode for aarch64/functions/gcs/GCSReturnValueCheckEnabled

```
// GCSReturnValueCheckEnabled()
// ============================
// Returns TRUE if the Guarded Control Stack has return value
// checking enabled at the current Exception level.

func GCSReturnValueCheckEnabled(el : bits(2)) => boolean
begin
    if UsingAArch32() then
        return FALSE;
    end;
    case el of
        when EL0 => return GCSCRE0_EL1().RVCHKEN == '1';
        when EL1 => return GCSCR_EL1().RVCHKEN == '1';
        when EL2 => return GCSCR_EL2().RVCHKEN == '1';
        when EL3 => return GCSCR_EL3().RVCHKEN == '1';
    end;
end;
```

Library pseudocode for aarch64/functions/gcs/GCSSS1

```
// GCSSS1()
// ========
// Operational pseudocode for GCSSS1 instruction.

func GCSSS1(incoming_pointer : bits(64))
begin
    var outgoing_pointer, cmpoperand, operand, data : bits(64);
    let privileged : boolean = PSTATE.EL != EL0;
    let accdesc : AccessDescriptor = CreateAccDescGCSSS1(privileged);
    outgoing_pointer = GetCurrentGCSPointer();
    // Valid GCS cap record is expected
    cmpoperand = incoming_pointer[63:12]::'000000000001';
    // In-progress GCS cap record should be stored if the comparison is successful
    operand = outgoing_pointer[63:3]::'101';

    data = MemAtomic{64}(incoming_pointer, cmpoperand, operand, accdesc);
    if data == cmpoperand then
        SetCurrentGCSPointer(incoming_pointer[63:3]::'000');
    else
        GCSDataCheckException(GCSInstType_SS1);
    end;
    return;
end;
```

Library pseudocode for aarch64/functions/gcs/GCSSS2

```
// GCSSS2()
// ========
// Operational pseudocode for GCSSS2 instruction.

func GCSSS2() => bits(64)
begin
    var outgoing_pointer, incoming_pointer, outgoing_value : bits(64);
    let privileged : boolean = PSTATE.EL != EL0;
    let accdesc_ld : AccessDescriptor = CreateAccDescGCS(MemOp_LOAD, privileged);
    let accdesc_st : AccessDescriptor = CreateAccDescGCS(MemOp_STORE, privileged);
    incoming_pointer = GetCurrentGCSPointer();
    outgoing_value = Mem{64}(incoming_pointer, accdesc_ld);

    if outgoing_value[2:0] == '101' then  //in_progress token
        outgoing_pointer[63:3] = (outgoing_value[63:3]) - 1;
        outgoing_pointer[2:0] = '000';
        outgoing_value = outgoing_pointer[63:12]::'000000000001';
        Mem{64}(outgoing_pointer, accdesc_st) = outgoing_value;
        SetCurrentGCSPointer(incoming_pointer + 8);
        GCSSynchronizationBarrier();
    else
        GCSDataCheckException(GCSInstType_SS2);
    end;
    return outgoing_pointer;
end;
```

Library pseudocode for aarch64/functions/gcs/GCSSTRTrapException

```
// GCSSTRTrapException()
// =====================
// Handle a trap on GCSSTR instruction condition.

func GCSSTRTrapException(target_el : bits(2))
begin
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_GCSFail);
    except.syndrome.iss[24]    = '0';
    except.syndrome.iss[23:20] = '0010';
    except.syndrome.iss[19:15] = '00000';
    except.syndrome.iss[14:10] = ThisInstr()[9:5];
    except.syndrome.iss[9:5]   = ThisInstr()[4:0];
    except.syndrome.iss[4:0]   = '00000';
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/functions/gcs/GCSSynchronizationBarrier

```
// GCSSynchronizationBarrier()
// ===========================
// Barrier instruction that synchronizes GCS accesses
// in relation to other load and store accesses

impdef func GCSSynchronizationBarrier()
begin
    return;
end;
```

Library pseudocode for aarch64/functions/gcs/GetCurrentEXLOCKEN

```
// GetCurrentEXLOCKEN()
// ====================

readonly func GetCurrentEXLOCKEN() => boolean
begin
    if Halted() || Restarting() then
        return FALSE;
    end;

    case PSTATE.EL of
        when EL0 =>
            unreachable;
        when EL1 =>
            return GCSCR_EL1().EXLOCKEN == '1';
        when EL2 =>
            return GCSCR_EL2().EXLOCKEN == '1';
        when EL3 =>
            return GCSCR_EL3().EXLOCKEN == '1';
    end;
end;
```

Library pseudocode for aarch64/functions/gcs/GetCurrentGCSPointer

```
// GetCurrentGCSPointer()
// ======================
// Returns the value of the current GCS pointer register.

func GetCurrentGCSPointer() => bits(64)
begin
    var ptr : bits(64);

    case PSTATE.EL of
        when EL0 =>
            ptr = GCSPR_EL0().PTR::'000';
        when EL1 =>
            ptr = GCSPR_EL1().PTR::'000';
        when EL2 =>
            ptr = GCSPR_EL2().PTR::'000';
        when EL3 =>
            ptr = GCSPR_EL3().PTR::'000';
    end;
    return ptr;
end;
```

Library pseudocode for aarch64/functions/gcs/LoadCheckGCSRecord

```
// LoadCheckGCSRecord()
// ====================
// Validates the provided address against the top entry of the
// current Guarded Control Stack.

func LoadCheckGCSRecord(vaddress : bits(64), gcsinst_type : GCSInstruction) => bits(64)
begin
    var ptr : bits(64);
    var recorded_va : bits(64);
    let privileged : boolean = PSTATE.EL != EL0;
    let accdesc : AccessDescriptor = CreateAccDescGCS(MemOp_LOAD, privileged);

    ptr = GetCurrentGCSPointer();
    recorded_va = Mem{64}(ptr, accdesc);
    if GCSReturnValueCheckEnabled(PSTATE.EL) && (recorded_va != vaddress) then
        GCSDataCheckException(gcsinst_type);
    end;

    return recorded_va;
end;
```

Library pseudocode for aarch64/functions/gcs/SetCurrentGCSPointer

```
// SetCurrentGCSPointer()
// ======================
// Writes a value to the current GCS pointer register.

func SetCurrentGCSPointer(ptr : bits(64))
begin
    case PSTATE.EL of
        when EL0 =>
            GCSPR_EL0().PTR = ptr[63:3];
        when EL1 =>
            GCSPR_EL1().PTR = ptr[63:3];
        when EL2 =>
            GCSPR_EL2().PTR = ptr[63:3];
        when EL3 =>
            GCSPR_EL3().PTR = ptr[63:3];
    end;
    return;
end;
```

Library pseudocode for aarch64/functions/gic/GetCurrentDomainPPIMask

```
// GetCurrentDomainPPIMask()
// =========================
// Returns a mask of PPIs assigned to the current domain.

func GetCurrentDomainPPIMask(n : integer) => bits(64)
begin
    var mask : bits(64) = Zeros{64};
    for x = 0 to 63 do
        if IsPPIAssignedToCurrentDomain((n * 64) + x) then
            mask[x] = '1';
        end;
    end;
    return mask;
end;
```

Library pseudocode for aarch64/functions/gic/GetICCPendingWritablePPIMask

```
// GetICCPendingWritablePPIMask()
// ==============================
// Returns the mask of PPIs writable through ICC_PPI_*PENDR_EL1(n).

func GetICCPendingWritablePPIMask(n : integer) => bits(64)
begin
    return GetICCWritablePPIMask(n) AND NOT ICC_PPI_HMR_EL1(n);
end;
```

Library pseudocode for aarch64/functions/gic/GetICCWritablePPIMask

```
// GetICCWritablePPIMask()
// =======================
// Returns the mask of PPIs writable through ICC_PPI_*_EL1(n).

func GetICCWritablePPIMask(n : integer) => bits(64)
begin
    var mask : bits(64) = GetImplementedPPIMask(n);
    if PSTATE.EL IN {EL2, EL1} then
        mask = mask AND GetCurrentDomainPPIMask(n);
    end;
    return mask;
end;
```

Library pseudocode for aarch64/functions/gic/GetICVPendingWritablePPIMask

```
// GetICVPendingWritablePPIMask()
// ==============================
// Returns the mask of PPIs writable through ICV_PPI_*PENDR_EL1(n).

func GetICVPendingWritablePPIMask(n : integer) => bits(64)
begin
    return GetImplementedPPIMask(n) AND NOT ICV_PPI_HMR_EL1(n);
end;
```

Library pseudocode for aarch64/functions/gic/GetImplementedPPIMask

```
// GetImplementedPPIMask()
// =======================
// Returns a mask of implemented PPIs for ICC_PPI_*_EL1(n) and ICV_PPI_*_EL1(n).

func GetImplementedPPIMask(n : integer) => bits(64)
begin
    var mask : bits(64) = Zeros{64};
    for x = 0 to 63 do
        if IsPPIImplemented((n * 64) + x) then
            mask[x] = '1';
        end;
    end;
    return mask;
end;
```

Library pseudocode for aarch64/functions/gic/Write_ICC_PPI_CACTIVER_EL1

```
// Write_ICC_PPI_CACTIVER_EL1()
// ============================
// System register write to ICC_PPI_CACTIVER_EL1(n).

func Write_ICC_PPI_CACTIVER_EL1(n : integer, value_in : bits(64))
begin
    ICC_PPI_SACTIVER_EL1(n) = (ICC_PPI_SACTIVER_EL1(n)
                               AND NOT (value_in AND GetICCWritablePPIMask(n)));
end;
```

Library pseudocode for aarch64/functions/gic/Write_ICC_PPI_CPENDR_EL1

```
// Write_ICC_PPI_CPENDR_EL1()
// ==========================
// System register write to ICC_PPI_CPENDR_EL1(n).

func Write_ICC_PPI_CPENDR_EL1(n : integer, value_in : bits(64))
begin
    ICC_PPI_SPENDR_EL1(n) = (ICC_PPI_SPENDR_EL1(n)
                             AND NOT (value_in AND GetICCPendingWritablePPIMask(n)));
end;
```

Library pseudocode for aarch64/functions/gic/Write_ICC_PPI_SACTIVER_EL1

```
// Write_ICC_PPI_SACTIVER_EL1()
// ============================
// System register write to ICC_PPI_SACTIVER_EL1(n).

func Write_ICC_PPI_SACTIVER_EL1(n : integer, value_in : bits(64))
begin
    ICC_PPI_SACTIVER_EL1(n) = ICC_PPI_SACTIVER_EL1(n) OR (value_in AND GetICCWritablePPIMask(n));
end;
```

Library pseudocode for aarch64/functions/gic/Write_ICC_PPI_SPENDR_EL1

```
// Write_ICC_PPI_SPENDR_EL1()
// ==========================
// System register write to ICC_PPI_SPENDR_EL1(n).

func Write_ICC_PPI_SPENDR_EL1(n : integer, value_in : bits(64))
begin
    ICC_PPI_SPENDR_EL1(n) = ICC_PPI_SPENDR_EL1(n) OR (value_in AND GetICCPendingWritablePPIMask(n));
end;
```

Library pseudocode for aarch64/functions/gic/Write_ICV_PPI_CACTIVER_EL1

```
// Write_ICV_PPI_CACTIVER_EL1()
// ============================
// System register write to ICV_PPI_CACTIVER_EL1(n).

func Write_ICV_PPI_CACTIVER_EL1(n : integer, value_in : bits(64))
begin
    ICH_PPI_ACTIVER_EL2(n) = ICH_PPI_ACTIVER_EL2(n) AND NOT (value_in AND GetImplementedPPIMask(n));
end;
```

Library pseudocode for aarch64/functions/gic/Write_ICV_PPI_CPENDR_EL1

```
// Write_ICV_PPI_CPENDR_EL1()
// ==========================
// System register write to ICV_PPI_CPENDR_EL1(n).

func Write_ICV_PPI_CPENDR_EL1(n : integer, value_in : bits(64))
begin
    ICH_PPI_PENDR_EL2(n) =
        ICH_PPI_PENDR_EL2(n) AND NOT (value_in AND GetICVPendingWritablePPIMask(n));
end;
```

Library pseudocode for aarch64/functions/gic/Write_ICV_PPI_SACTIVER_EL1

```
// Write_ICV_PPI_SACTIVER_EL1()
// ============================
// System register write to ICV_PPI_SACTIVER_EL1(n).

func Write_ICV_PPI_SACTIVER_EL1(n : integer, value_in : bits(64))
begin
    ICH_PPI_ACTIVER_EL2(n) = ICH_PPI_ACTIVER_EL2(n) OR (value_in AND GetImplementedPPIMask(n));
end;
```

Library pseudocode for aarch64/functions/gic/Write_ICV_PPI_SPENDR_EL1

```
// Write_ICV_PPI_SPENDR_EL1()
// ==========================
// System register write to ICV_PPI_SPENDR_EL1(n).

func Write_ICV_PPI_SPENDR_EL1(n : integer, value_in : bits(64))
begin
    ICH_PPI_PENDR_EL2(n) = ICH_PPI_PENDR_EL2(n) OR (value_in AND GetICVPendingWritablePPIMask(n));
end;
```

Library pseudocode for aarch64/functions/hacdbs

```
constant HACDBS_ERR_REASON_STRUCTF : bits(2) = '01';
constant HACDBS_ERR_REASON_IPAF : bits(2)    = '10';
constant HACDBS_ERR_REASON_IPHACF : bits(2)  = '11';
```

Library pseudocode for aarch64/functions/hacdbs/IsHACDBSIRQAsserted

```
// IsHACDBSIRQAsserted()
// =====================
// Returns TRUE if HACDBSIRQ is asserted, and FALSE otherwise.

impdef func IsHACDBSIRQAsserted() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/functions/hacdbs/ProcessHACDBSEntry

```
// ProcessHACDBSEntry()
// ====================
// Process a single entry from the HACDBS.

func ProcessHACDBSEntry()
begin
    if !IsFeatureImplemented(FEAT_HACDBS) then return; end;

    if (PSTATE.EL == EL3 || (HaveEL(EL3) && SCR_EL3().HACDBSEn == '0') ||
          HACDBSBR_EL2().EN == '0') then
        SetInterruptRequestLevel(InterruptID_HACDBSIRQ, LOW);
        return;
    end;

    if HCR_EL2().VM == '0' then return; end;

    if (UInt(HACDBSCONS_EL2().INDEX) >= (2 ^ (UInt(HACDBSBR_EL2().SZ) + 12)) DIVRM 8 ||
          HACDBSCONS_EL2().ERR_REASON != '00') then
        SetInterruptRequestLevel(InterruptID_HACDBSIRQ, HIGH);
        return;
    elsif IsHACDBSIRQAsserted() then
        SetInterruptRequestLevel(InterruptID_HACDBSIRQ, LOW);
    end;

    let hacdbs_size : integer{} = UInt(HACDBSBR_EL2().SZ);
    var baddr : bits(NUM_PABITS) = HACDBSBR_EL2().BADDR[NUM_PABITS-13 : 0] :: Zeros{12};
    baddr[11 + hacdbs_size : 12] = Zeros{hacdbs_size};

    var accdesc : AccessDescriptor = CreateAccDescHACDBS();

    var addrdesc : AddressDescriptor;
    addrdesc.paddress.address = baddr + (8 * UInt(HACDBSCONS_EL2().INDEX));
    let nse2 : bit = '0';     // NSE2 has the Effective value of 0 within a PE.
    addrdesc.paddress.paspace = DecodePASpace(nse2, EffectiveSCR_EL3_NSE(), EffectiveSCR_EL3_NS());

    // Accesses to the HACDBS use the same memory attributes as used for stage 2 translation walks.
    addrdesc.memattrs = WalkMemAttrs(VTCR_EL2().SH0, VTCR_EL2().IRGN0, VTCR_EL2().ORGN0);
    let emec : bit = (if IsFeatureImplemented(FEAT_MEC) &&
                         IsSCTLR2EL2Enabled() then SCTLR2_EL2().EMEC else '0');
    addrdesc.mecid = AArch64_S2TTWalkMECID(emec, accdesc.ss);

    var fault : FaultRecord = NoFault(accdesc);

    if IsFeatureImplemented(FEAT_RME) then
        fault.gpcf = GranuleProtectionCheck(addrdesc, accdesc);

        if fault.gpcf.gpf != GPCF_None then
            HACDBSCONS_EL2().ERR_REASON = HACDBS_ERR_REASON_STRUCTF;
            return;
        end;
    end;

    var memstatus : PhysMemRetStatus;
    var hacdbs_entry : bits(64);
    (memstatus, hacdbs_entry) = PhysMemRead{64}(addrdesc, accdesc);

    if IsFault(memstatus) then
        HACDBSCONS_EL2().ERR_REASON = HACDBS_ERR_REASON_STRUCTF;
        return;
    end;

    if BigEndian(accdesc.acctype) then
        hacdbs_entry = BigEndianReverse{64}(hacdbs_entry);
    end;

    // If the Valid field is clear, do not perform any cleaning operation
    // and increment HACDBSCONS_EL2.INDEX.
    if hacdbs_entry[0] == '0' then
        HACDBSCONS_EL2().INDEX = HACDBSCONS_EL2().INDEX + 1;
        return;
    end;

    accdesc = CreateAccDescTTEUpdate(accdesc);
    var ipa : AddressDescriptor;
    ipa.paddress.address = hacdbs_entry[NUM_PABITS-1:12] :: Zeros{12};

    let nsipa : bit       = hacdbs_entry[11];
    let paspace : PASpace = DecodePASpace(nse2, EffectiveSCR_EL3_NSE(), EffectiveSCR_EL3_NS());
    ipa.paddress.paspace = (if accdesc.ss == SS_Secure && nsipa == '1' then PAS_NonSecure
                                                                       else paspace);

    let s1aarch64 : boolean = TRUE;
    let walkparams : S2TTWParams = AArch64_GetS2TTWParams(accdesc.ss, ipa.paddress.paspace,
                                                          s1aarch64);

    var descpaddr : AddressDescriptor;
    var walkstate : TTWState;
    var descriptor : bits(128);
    if walkparams.d128 == '1' then
```

```
        (fault, descpaddr, walkstate, descriptor) = AArch64_S2Walk{128}(fault, ipa, walkparams,
                                                                       accdesc);
    else
        (fault, descpaddr, walkstate, descriptor[63:0]) = AArch64_S2Walk{64}(fault, ipa, walkparams,
                                                                            accdesc);
    end;

    // If the Access flag on the Block or Page descriptor is set to 0, this does not generate
    // an Access flag fault and the PE can still perform the cleaning operation on that descriptor.
    if fault.statuscode == Fault_AccessFlag then
        fault.statuscode = Fault_None;
    elsif fault.statuscode != Fault_None then
        HACDBSCONS_EL2().ERR_REASON = HACDBS_ERR_REASON_IPAF;
        return;
    end;

    let hacdbs_level : integer = SInt(hacdbs_entry[3:1]);
    if walkstate.level != hacdbs_level || walkstate.contiguous == '1' then
        HACDBSCONS_EL2().ERR_REASON = HACDBS_ERR_REASON_IPHACF;
        return;
    end;

    // For the purpose of cleaning HACDBS entries, it is not required that HW update of dirty bit
    // is enabled for a descriptor to be qualified as writable-clean or writable-dirty.

    // Check if the descriptor is neither writable-clean nor writable-dirty.
    if walkparams.s2pie == '1' then
        let perms : S2AccessControls = AArch64_S2ComputePermissions(walkstate.permissions,
                                                                    walkparams, accdesc);
        if perms.w == '0' && perms.w_mmu == '0' then
            HACDBSCONS_EL2().ERR_REASON = HACDBS_ERR_REASON_IPHACF;
            return;
        end;

    // If DBM is 0, the descriptor is not writable-clean or writable-dirty.
    elsif descriptor[51] == '0' then
        HACDBSCONS_EL2().ERR_REASON = HACDBS_ERR_REASON_IPHACF;
        return;
    end;

    // If the descriptor is writable-clean, do not perform any cleaning
    // operation and increment HACDBSCONS_EL2.INDEX.
    if descriptor[7] == '0' then
        HACDBSCONS_EL2().INDEX = HACDBSCONS_EL2().INDEX + 1;
        return;
    end;

    var new_descriptor : bits(128) = descriptor;
    new_descriptor[7] = '0';

    let descaccess : AccessDescriptor = CreateAccDescTTEUpdate(accdesc);
    if walkparams.d128 == '1' then
        (fault, -) = AArch64_MemSwapTableDesc{128}(fault, descriptor, new_descriptor, walkparams.ee,
                                                   descaccess, descpaddr);
    else
        (fault, -) = AArch64_MemSwapTableDesc{64}(fault, descriptor[63:0], new_descriptor[63:0],
                                                  walkparams.ee, descaccess, descpaddr);
    end;

    if fault.statuscode != Fault_None then
        HACDBSCONS_EL2().ERR_REASON = HACDBS_ERR_REASON_IPAF;
    else
        HACDBSCONS_EL2().INDEX = HACDBSCONS_EL2().INDEX + 1;
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/ic/AArch64_CanTrapIC

```
// AArch64_CanTrapIC()
// ===================
// Determines whether the execution of the IC instruction can be trapped.

func AArch64_CanTrapIC(cachetype : CacheType, cacheop : CacheOp, opscope : CacheOpScope) => boolean
begin
    return (!AArch64_TreatICAsNOP(cachetype, cacheop, opscope) || ImpDefBool(
            "When IC is treated as NOP, data cache maintenance operations are trapped"));
end;
```

Library pseudocode for aarch64/functions/ic/AArch64_EffectiveOpScope

```
// AArch64_EffectiveOpScope()
// ==========================

func AArch64_EffectiveOpScope(opscope_in : CacheOpScope) => CacheOpScope
begin
    var opscope : CacheOpScope = opscope_in;

    if PSTATE.EL != EL1 then return opscope; end;

    if IsFeatureImplemented(FEAT_TLBID) then
        let fnb : bit = if IsHCRXEL2Enabled() then HCRX_EL2().FNB else '0';
        case fnb::HCR_EL2().FB of
            when '00' =>
                pass;
            when '01' =>
                if opscope == CacheOpScope_ALLU then
                    // The operation is broadcast within the Inner Shareable shareability domain.
                    opscope = CacheOpScope_ALLUIS;
                end;
            when '10' =>
                if opscope == CacheOpScope_ALLUIS then
                    // The operation only affects the PE on which the instruction is executed.
                    opscope = CacheOpScope_ALLU;
                end;
            when '11' =>
                if opscope == CacheOpScope_ALLUIS then
                    // The operation is broadcast to PEs which would share TLB entries with the
                    // current PE if CnP were 1.
                    opscope = CacheOpScope_ALLUIS_CnP;
                end;
        end;
    else
        if opscope == CacheOpScope_ALLU && EL2Enabled() && HCR_EL2().FB == '1' then
            opscope = CacheOpScope_ALLUIS;
        end;
    end;

    return opscope;
end;
```

Library pseudocode for aarch64/functions/ic/AArch64_IC

```
// AArch64_IC()
// ============
// Perform Instruction Cache Operation.

func AArch64_IC(opscope : CacheOpScope)
begin
    let regval : bits(64) = ARBITRARY : bits(64);
    AArch64_IC(regval, opscope);
end;

// AArch64_IC()
// ============
// Perform Instruction Cache Operation.

func AArch64_IC(regval : bits(64), opscope_in : CacheOpScope)
begin
    var cache : CacheRecord;
    var opscope : CacheOpScope = opscope_in;

    cache.acctype   = AccessType_IC;
    cache.cachetype = CacheType_Instruction;
    cache.cacheop   = CacheOp_Invalidate;
    cache.opscope   = opscope;

    if opscope IN {CacheOpScope_ALLU,
                   CacheOpScope_ALLUIS_CnP,
                   CacheOpScope_ALLUIS} then
        let ss : SecurityState = SecurityStateAtEL(PSTATE.EL);
        cache.cpas = CPASAtSecurityState(ss);
        opscope = AArch64_EffectiveOpScope(opscope);

        case opscope of
            when CacheOpScope_ALLU =>
                cache.shareability = Shareability_NSH;
            when CacheOpScope_ALLUIS =>
                cache.shareability = Shareability_ISH;
                // When executed at EL1, if the Effective value of HCRX_EL2.VTLBIDEn is '1',
                // then the operation is broadcast to the set of PEs selected by VTLBID0_EL2.TD0.
                // Otherwise, the operation is broadcast to all PEs in the Shareability domain.
                cache.domains = TLBIDomains(Broadcast_ISH, Zeros{16});
            when CacheOpScope_ALLUIS_CnP =>
                cache.shareability = Shareability_ISH;
                cache.domains = DEFAULT_TLBI_DOMAIN;
            otherwise =>
                unreachable;
        end;

        cache.regval = regval;
        CACHE_OP(cache);
    else
        assert opscope == CacheOpScope_PoU;

        if EL2Enabled() && !IsInHost() then
            if PSTATE.EL IN {EL0, EL1} then
                cache.is_vmid_valid = TRUE;
                cache.vmid          = VMID();
            else
                cache.is_vmid_valid = FALSE;
            end;
        else
            cache.is_vmid_valid = FALSE;
        end;

        if PSTATE.EL == EL0 then
            cache.is_asid_valid = TRUE;
            cache.asid          = ASID();
        else
            cache.is_asid_valid = FALSE;
        end;

        let vaddress : bits(64) = regval;
        cache.vaddress = regval;

        let accdesc : AccessDescriptor = CreateAccDescIC(cache);
        let aligned : boolean = TRUE;
        let size : integer = 0;
        var memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(vaddress, accdesc,
                                                                       aligned, size);

        if IsFault(memaddrdesc) then
            memaddrdesc.fault.vaddress = regval;
            AArch64_Abort(memaddrdesc.fault);
        end;

        cache.cpas         = CPASAtPAS(memaddrdesc.paddress.paspace);
        cache.paddress     = memaddrdesc.paddress;
        cache.shareability = memaddrdesc.memattrs.shareability;
        if memaddrdesc.memattrs.shareability == Shareability_OSH then
            cache.shareability = Shareability_ISH;
```

```
        end;

        if memaddrdesc.memattrs.shareability != Shareability_NSH then
            // If the Effective value of HCRX_EL2().VTLBIDEn is '1', the invalidation is broadcast
            // to the set of PEs in the Inner shareable domain selected by VTLBID0_EL2().TD0 when
            // executed at EL1, or at EL0 when HCR_EL2().TGE is 0.
            cache.domains = TLBIDomains(Broadcast_ISH, Zeros{16});
        end;

        CACHE_OP(cache);
    end;
    return;
end;
```

Library pseudocode for aarch64/functions/ic/AArch64_TreatICAsNOP

```
// AArch64_TreatICAsNOP()
// ======================
// Determines whether the execution of the IC instruction is treated as a NOP.

func AArch64_TreatICAsNOP(cachetype : CacheType, cacheop : CacheOp,
                          opscope : CacheOpScope) => boolean
begin
    if CTR_EL0().DIC == '1' then
        return ImpDefBool("IC is treated as NOP");
    end;

    return FALSE;
end;
```

Library pseudocode for aarch64/functions/iesb/EIESB

```
// EIESB
// =====
// Implementation options for the ID_AA64MMFR4_EL1.EIESB exception-entry behavior.

type EIESB of enumeration {
     EIESB_None,                        // 0b0000
     EIESB_Before_EL3_Conditional,      // 0b0001
     EIESB_Before_All_Conditional,      // 0b0010
     EIESB_Before_All_NonSError,        // 0b0011
     EIESB_After_EL3_Conditional        // 0b1111
 };
```

Library pseudocode for aarch64/functions/iesb/GetEIESB

```
// GetEIESB()
// ==========
// Returns the EIESB implementation level.

impdef func GetEIESB() => EIESB
begin
    return EIESB_Before_All_NonSError;
end;
```

Library pseudocode for aarch64/functions/iesb/InsertIESBBeforeException

```
// InsertIESBBeforeException()
// ===========================
// Returns whether an implicit Error synchronization event is inserted before
// taking an exception, according to the EIESB implementation behavior.
//
// If the Effective value of SCTLR_ELx.IESB is 1 when an exception is generated
// to ELx, any pending Unrecoverable SError interrupt must be taken before
// executing any instructions in the exception handler. Depending on the EIESB
// implementation behavior, this can be before the branch to the exception
// handler is made.

func InsertIESBBeforeException(exceptype : Exception, target_el : bits(2)) => boolean
begin
    // Exclude SError exceptions.
    if exceptype == Exception_SError then
        return FALSE;
    end;

    case GetEIESB() of
        when {EIESB_After_EL3_Conditional,
              EIESB_None} =>
            // Might insert after taking an exception to EL3, unless the exception is also an SError
            // exception.
            // Or behavior is not described.
            return ImpDefBool("Has Implicit Error Synchronization Barrier before Exception");

        when EIESB_Before_EL3_Conditional =>
            // Insert before taking an exception to EL3,
            // unless the exception is also an SError exception.
            if target_el == EL3 && EffectiveEA() == '1' && EffectiveNMEA(EL3) == '1' then
                return TRUE;
            end;

        when EIESB_Before_All_Conditional =>
            // As EIESB_Before_EL3_Conditional. Also insert before taking exceptions to EL1/EL2
            // when the architectural conditions apply.
            if target_el == EL3 && EffectiveEA() == '1' && EffectiveNMEA(EL3) == '1' then
                return TRUE;

            elsif target_el IN {EL1, EL2} then
                if EffectiveNMEA(target_el) == '1' then
                    return TRUE;
                end;
            end;

        when EIESB_Before_All_NonSError =>
            // Insert before taking all exceptions other than SError exceptions,
            // regardless of the architectural conditions.
            return TRUE;
    end;
    return ImpDefBool("Has Implicit Error Synchronization Barrier before Exception");
end;
```

Library pseudocode for aarch64/functions/immediateop/ImmediateOp

```
// ImmediateOp
// ===========
// Vector logical immediate instruction types.

type ImmediateOp of enumeration {ImmediateOp_MOVI, ImmediateOp_MVNI,
                                 ImmediateOp_ORR, ImmediateOp_BIC};
```

Library pseudocode for aarch64/functions/logicalop/LogicalOp

```
// LogicalOp
// =========
// Logical instruction types.

type LogicalOp of enumeration {LogicalOp_AND, LogicalOp_EOR, LogicalOp_ORR};
```

Library pseudocode for aarch64/functions/mec

```
constant DEFAULT_MECID : bits(16) = Zeros{};
```

Library pseudocode for aarch64/functions/mec/AArch64_S1AMECFault

```
// AArch64_S1AMECFault()
// =====================
// Returns TRUE if a Translation fault should occur for Realm EL2 and Realm EL2&0
// stage 1 translated addresses to Realm PA space.

func AArch64_S1AMECFault{N}(walkparams : S1TTWParams, paspace : PASpace, regime : Regime,
                            descriptor : bits(N)) => boolean
begin
    assert N IN {64,128};
    let descriptor_amec : bit = (if walkparams.d128 == '1' then descriptor[108]
                                 else descriptor[63]);

    return (walkparams.[emec,amec] == '10' &&
            regime IN {Regime_EL2, Regime_EL20} &&
            paspace == PAS_Realm &&
            descriptor_amec == '1');
end;
```

Library pseudocode for aarch64/functions/mec/AArch64_S1DisabledOutputMECID

```
// AArch64_S1DisabledOutputMECID()
// ===============================
// Returns the output MECID when stage 1 address translation is disabled.

func AArch64_S1DisabledOutputMECID(walkparams : S1TTWParams, regime : Regime,
                                   paspace : PASpace) => bits(16)
begin
    if walkparams.emec == '0' then
        return DEFAULT_MECID;
    end;

    if regime IN !{Regime_EL2, Regime_EL20, Regime_EL10} then
        return DEFAULT_MECID;
    end;

    if paspace != PAS_Realm then
        return DEFAULT_MECID;
    end;

    if EffectiveSCR_EL3_MECEn() == '0' then
        return DEFAULT_MECID;
    end;

    if regime == Regime_EL10 then
        return VMECID_P_EL2().MECID;
    else
        return MECID_P0_EL2().MECID;
    end;
end;
```

Library pseudocode for aarch64/functions/mec/AArch64_S1OutputMECID

```
// AArch64_S1OutputMECID()
// =======================
// Returns the output MECID when stage 1 address translation is enabled.

func AArch64_S1OutputMECID{N}(walkparams : S1TTWParams, regime : Regime, varange : VARange,
                              paspace : PASpace, descriptor : bits(N)) => bits(16)
begin
    assert N IN {64,128};

    if walkparams.emec == '0' then
        return DEFAULT_MECID;
    end;

    if paspace != PAS_Realm then
        return DEFAULT_MECID;
    end;

    if regime != Regime_EL3 && EffectiveSCR_EL3_MECEn() == '0' then
        return DEFAULT_MECID;
    end;

    let descriptor_amec : bit = (if walkparams.d128 == '1' then descriptor[108]
                                 else descriptor[63]);
    case regime of
        when Regime_EL3 =>
            return MECID_RL_A_EL3().MECID;
        when Regime_EL2 =>
            if descriptor_amec == '0' then
                return MECID_P0_EL2().MECID;
            else
                return MECID_A0_EL2().MECID;
            end;
        when Regime_EL20 =>
            if varange == VARange_LOWER then
                if descriptor_amec == '0' then
                    return MECID_P0_EL2().MECID;
                else
                    return MECID_A0_EL2().MECID;
                end;
            else
                if descriptor_amec == '0' then
                    return MECID_P1_EL2().MECID;
                else
                    return MECID_A1_EL2().MECID;
                end;
            end;
        // Stage 2 translation might later override the MECID according to AMEC configuration.
        when Regime_EL10 =>
            return VMECID_P_EL2().MECID;
    end;
end;
```

Library pseudocode for aarch64/functions/mec/AArch64_S1TTWalkMECID

```
// AArch64_S1TTWalkMECID()
// =======================
// Returns the associated MECID for the stage 1 translation table walk of the given
// translation regime and Security state.

func AArch64_S1TTWalkMECID(emec : bit, regime : Regime, ss : SecurityState) => bits(16)
begin
    if emec == '0' then
        return DEFAULT_MECID;
    end;

    if ss != SS_Realm then
        return DEFAULT_MECID;
    end;

    if EffectiveSCR_EL3_MECEn() == '0' then
        return DEFAULT_MECID;
    end;

    case regime of
        when Regime_EL2 =>
            return MECID_P0_EL2().MECID;
        when Regime_EL20 =>
            if TCR_EL2().A1 == '0' then
                return MECID_P1_EL2().MECID;
            else
                return MECID_P0_EL2().MECID;
            end;
        // Stage 2 translation for a stage 1 walk might later override the
        // MECID according to AMEC configuration.
        when Regime_EL10 =>
            return VMECID_P_EL2().MECID;
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for aarch64/functions/mec/AArch64_S2OutputMECID

```
// AArch64_S2OutputMECID()
// =======================
// Returns the output MECID for stage 2 address translation.

func AArch64_S2OutputMECID{N}(walkparams : S2TTWParams, paspace : PASpace,
                              descriptor : bits(N)) => bits(16)
begin
    assert N IN {64,128};

    if walkparams.emec == '0' then
        return DEFAULT_MECID;
    end;

    if paspace != PAS_Realm then
        return DEFAULT_MECID;
    end;

    if EffectiveSCR_EL3_MECEn() == '0' then
        return DEFAULT_MECID;
    end;

    let descriptor_amec : bit = (if walkparams.d128 == '1' then descriptor[108]
                                 else descriptor[63]);
    if descriptor_amec == '0' then
        return VMECID_P_EL2().MECID;
    else
        return VMECID_A_EL2().MECID;
    end;
end;
```

Library pseudocode for aarch64/functions/mec/AArch64_S2TTWalkMECID

```
// AArch64_S2TTWalkMECID()
// =======================
// Returns the associated MECID for the stage 2 translation table walk of the
// given Security state.

func AArch64_S2TTWalkMECID(emec : bit, ss : SecurityState) => bits(16)
begin
    if emec == '0' then
        return DEFAULT_MECID;
    end;

    if ss != SS_Realm then
        return DEFAULT_MECID;
    end;

    if EffectiveSCR_EL3_MECEn() == '0' then
        return DEFAULT_MECID;
    end;

    //Stage 2 translation might later override the MECID according to AMEC configuration
    return VMECID_P_EL2().MECID;
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_AccessIsTagChecked

```
// AArch64_AccessIsTagChecked()
// ============================
// TRUE if a given access is tag-checked, FALSE otherwise.

readonly func AArch64_AccessIsTagChecked(vaddr : bits(64), accdesc : AccessDescriptor) => boolean
begin
    assert accdesc.tagchecked;

    if UsingAArch32() then
        return FALSE;
    end;

    if !IsMTEEnabled(accdesc.el) then
        return FALSE;
    end;

    if PSTATE.TCO == '1' then
        return FALSE;
    end;

    if (Halted() && EDSCR().MA == '1' &&
          ConstrainUnpredictableBool(Unpredictable_NODTRTAGCHK)) then
        return FALSE;
    end;

    if (IsFeatureImplemented(FEAT_MTE_STORE_ONLY) && !accdesc.write &&
          StoreOnlyTagCheckingEnabled(accdesc.el)) then
        return FALSE;
    end;

    let is_instr : boolean = FALSE;
    if (EffectiveMTX(vaddr, is_instr, PSTATE.EL) == '0' &&
          EffectiveTBI(vaddr, is_instr, PSTATE.EL) == '0') then
        return FALSE;
    end;

    if (EffectiveTCMA(vaddr, PSTATE.EL) == '1' &&
          (vaddr[59:55] == '00000' || vaddr[59:55] == '11111')) then
        return FALSE;
    end;

    return TRUE;
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_AddressWithAllocationTag

```
// AArch64_AddressWithAllocationTag()
// ==================================
// Generate a 64-bit value containing a Logical Address Tag from a 64-bit
// virtual address and an Allocation Tag.

func AArch64_AddressWithAllocationTag(address : bits(64), allocation_tag : bits(4)) => bits(64)
begin
    return address[63:60]::allocation_tag::address[55:0];
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_AllocationTagCheck

```
// AArch64_AllocationTagCheck()
// ============================
// Performs an Allocation Tag Check operation for a memory access and
// returns whether the check passed.

func AArch64_AllocationTagCheck(memaddrdesc : AddressDescriptor, accdesc : AccessDescriptor,
                                ltag : bits(4)) => FaultRecord
begin
    var fault : FaultRecord = NoFault(accdesc, memaddrdesc.vaddress);
    if memaddrdesc.memattrs.tags == MemTag_AllocationTagged then
        var memstatus : PhysMemRetStatus;
        var readtag : bits(4);

        // Physical tagging needs no further translation, use the data PA to read the tag
        (memstatus, readtag) = PhysMemTagRead(memaddrdesc, accdesc);
        if IsFault(memstatus) then
            let iswrite : boolean = FALSE;
            return ExternalFault(memstatus, iswrite, memaddrdesc, 1, accdesc);
        end;

        if ltag != readtag then
            fault.statuscode = Fault_TagCheck;
        end;
    end;

    return fault;
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_AllocationTagFromAddress

```
// AArch64_AllocationTagFromAddress()
// ==================================
// Generate an Allocation Tag from a 64-bit value containing a Logical Address Tag.

func AArch64_AllocationTagFromAddress(tagged_address : bits(64)) => bits(4)
begin
    return tagged_address[59:56];
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_CanonicalTagCheck

```
// AArch64_CanonicalTagCheck()
// ===========================
// Performs a Canonical Tag Check operation for a memory access and
// returns whether the check passed.

func AArch64_CanonicalTagCheck(memaddrdesc : AddressDescriptor, ltag : bits(4)) => boolean
begin
    let expected_tag : bits(4) = if memaddrdesc.vaddress[55] == '0' then '0000' else '1111';
    return ltag == expected_tag;
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_CheckTag

```
// AArch64_CheckTag()
// ==================
// Performs a Tag Check operation for a memory access and returns a FaultRecord indicating if
// the check passed. If Tag Check Faults are asynchronously accumulated, a Tag Check Fault
// exception is recorded in TFSR_ELx.

func AArch64_CheckTag(memaddrdesc_in : AddressDescriptor, accdesc : AccessDescriptor,
                      readcheck : boolean, size : integer, ltag : bits(4)) => FaultRecord
begin
    var memaddrdesc : AddressDescriptor = memaddrdesc_in;
    // NoFault() will set fault.write to FALSE to accesses that perform both a read and a write.
    var fault : FaultRecord             = NoFault(accdesc, memaddrdesc.vaddress);
    let granules : integer              = Max(size DIVRM TAG_GRANULE, 1);
    let forcesync : boolean             = accdesc.nonfault || (accdesc.firstfault && !accdesc.first);
    let tcf : TCFType                   = AArch64_EffectiveTCF(accdesc.el, readcheck);

    for i = 0 to granules - 1 do
        case memaddrdesc.memattrs.tags of
            when MemTag_AllocationTagged =>
                fault = AArch64_AllocationTagCheck(memaddrdesc, accdesc, ltag);
                if fault.statuscode == Fault_TagCheck then
                    if tcf == TCFType_Sync || forcesync then
                        fault.statuscode = Fault_TagCheck;
                        return fault;
                    elsif tcf == TCFType_Async then
                        AArch64_ReportTagCheckFault(accdesc.el, memaddrdesc.vaddress[55]);
                        fault.statuscode = Fault_None;
                        return fault;
                    else
                        // Tag Check Faults have no effect on the PE.
                        fault.statuscode = Fault_None;
                    end;
                elsif fault.statuscode != Fault_None then
                    return fault;
                end;
            when MemTag_CanonicallyTagged =>
                if !AArch64_CanonicalTagCheck(memaddrdesc, ltag) then
                    if tcf == TCFType_Sync || forcesync then
                        fault.statuscode = Fault_TagCheck;
                        return fault;
                    elsif tcf == TCFType_Async then
                        AArch64_ReportTagCheckFault(accdesc.el, memaddrdesc.vaddress[55]);
                        return fault;
                    else
                        // Tag Check Faults have no effect on the PE.
                        pass;
                    end;
                end;
            when MemTag_Untagged =>
                pass;
            otherwise =>
                unreachable;
        end;

        memaddrdesc.paddress.address = memaddrdesc.paddress.address + TAG_GRANULE;
        memaddrdesc.vaddress         = memaddrdesc.vaddress + TAG_GRANULE;
    end;

    return fault;
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_IsUnprivAccessPriv

```
// AArch64_IsUnprivAccessPriv()
// ============================
// Returns TRUE if an unprivileged access is privileged, and FALSE otherwise.

func AArch64_IsUnprivAccessPriv() => boolean
begin
    var privileged : boolean;

    case PSTATE.EL of
        when EL0 =>
            privileged = FALSE;
        when EL1 => privileged = EffectiveHCR_EL2_NVx()[1:0] == '11';
        when EL2 => privileged = !ELIsInHost(EL0);
        when EL3 =>
            privileged = TRUE;
    end;

    if IsFeatureImplemented(FEAT_UAO) && PSTATE.UAO == '1' then
        privileged = PSTATE.EL != EL0;
    end;

    return privileged;
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_LogicalAddressTag

```
// AArch64_LogicalAddressTag()
// ===========================
// Extract the Logical Address Tag from an address

func AArch64_LogicalAddressTag(vaddr : bits(64)) => bits(4)
begin
    return vaddr[59:56];
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_MemSingle

```
// AArch64_MemSingle - accessor
// ============================

accessor AArch64_MemSingle{size : integer{8, 16, 32, 64, 128, 256}}(address : bits(64),
                                                                    accdesc : AccessDescriptor,
                                                                    aligned : boolean
                                                                   ) <=> value : bits(size)
begin
    // Perform an atomic, little-endian read of 'size' bits.
    getter
        let bytes : integer{} = size DIV 8;
        var value : bits(size);
        var memaddrdesc : AddressDescriptor;
        var memstatus : PhysMemRetStatus;

        (value, memaddrdesc, memstatus) = AArch64_MemSingleRead{size}(address, accdesc, aligned);

        // Check for a fault from translation or the output of translation.
        if IsFault(memaddrdesc) then
            AArch64_Abort(memaddrdesc.fault);
        end;

        // Check for external aborts.
        if IsFault(memstatus) then
            HandleExternalAbort(memstatus, accdesc.write, memaddrdesc, bytes, accdesc);
        end;

        return value;
    end;

    // Perform an atomic, little-endian write of 'size' bits.
    setter
        let bytes : integer{} = size DIV 8;
        var memaddrdesc : AddressDescriptor;
        var memstatus : PhysMemRetStatus;

        (memaddrdesc, memstatus) = AArch64_MemSingleWrite{size}(address, accdesc, aligned, value);

        // Check for a fault from translation or the output of translation.
        if IsFault(memaddrdesc) then
            AArch64_Abort(memaddrdesc.fault);
        end;

        // Check for external aborts.
        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, bytes, accdesc);
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_MemSingleRead

```
// AArch64_MemSingleRead()
// =======================
// Perform an atomic, little-endian read of 'size' bits.

func AArch64_MemSingleRead{size : integer{8, 16, 32, 64, 128, 256}}(address : bits(64),
                                                                    accdesc_in : AccessDescriptor,
                                                                    aligned : boolean
                                                                   ) => (bits(size),
                                                                         AddressDescriptor,
                                                                         PhysMemRetStatus)
begin
    var value : bits(size) = ARBITRARY : bits(size);
    var memstatus : PhysMemRetStatus = ARBITRARY : PhysMemRetStatus;
    let bytes : integer{} = size DIV 8;
    memstatus.statuscode = Fault_None;
    var accdesc : AccessDescriptor = accdesc_in;
    if IsFeatureImplemented(FEAT_LSE2) then
        assert ((IsFeatureImplemented(FEAT_LS64WB) &&
                 bytes == 32 && accdesc.acctype == AccessType_ASIMD) ||
                 (IsFeatureImplemented(FEAT_SVE_LRCPC) && bytes == 32) ||
                 AllInSCAGranule(address, bytes));
    else
        assert IsAlignedSize(address, bytes);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    var memaddrdesc : AddressDescriptor;
    memaddrdesc = AArch64_TranslateAddress(address, accdesc, aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (value, memaddrdesc, memstatus);
    end;

    // Memory array access
    if accdesc.tagchecked then
        let ltag : bits(4) = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = TRUE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            memaddrdesc.fault = fault;
            return (value, memaddrdesc, memstatus);
        end;
    end;

    if accdesc.acctype != AccessType_IFETCH && SPESampleInFlight then
        let is_load : boolean = TRUE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;
    var atomic : boolean;
    if IsWBShareable(memaddrdesc.memattrs) then
        atomic = TRUE;
    elsif accdesc.exclusive then
        atomic = TRUE;
    elsif (accdesc.acctype == AccessType_SVE && accdesc.predicated &&
           bytes == 8 && IsAlignedSize(address, 8)) then
        // An SVE predicated load of a 128-bit element that is 64-bit aligned
        // is treated as a pair of 64-bit single-copy atomic accesses.
        // This is one of the 64-bit single-copy atomic access.
        atomic = TRUE;
    elsif aligned then
        atomic = !accdesc.ispair;
    else
        // Misaligned accesses within a single-copy atomic granule but
        // not Normal Cacheable Writeback are Atomic
        atomic = ImpDefBool("FEAT_LSE2: access is atomic");
    end;

    if atomic then
        (memstatus, value) = PhysMemRead{size}(memaddrdesc, accdesc);

    elsif accdesc.acctype == AccessType_ASIMD && bytes == 32 && accdesc.ispair then
        // A 32 byte LDP (SIMD&FP) that does not target Normal Inner Write-Back, Outer
        // Write-Back cacheable, Shareable memory is treated as four 8 byte atomic accesses.
        // As this access was not split in Mem(), it must be aligned to 32 bytes.
        assert IsAlignedSize(address, 32);
        accdesc.ispair = FALSE;
        for i = 0 to 3 do
            if !IsFault(memstatus) then     // Do not continue past a fault
                (memstatus, value[i*64+:64]) = PhysMemRead{64}(memaddrdesc, accdesc);
            end;

            if !IsFault(memstatus) then
                memaddrdesc.paddress.address = memaddrdesc.paddress.address + 8;
```

```
                memaddrdesc.vaddress         = memaddrdesc.vaddress + 8;
            end;
        end;

    elsif aligned && accdesc.ispair then
        let half : integer{} = (size DIV 2) as integer{32, 64, 128};
        (memstatus, value[0*:half]) = PhysMemRead{half}(memaddrdesc, accdesc);

        if !IsFault(memstatus) then
            memaddrdesc.paddress.address = memaddrdesc.paddress.address + (half DIV 8);
            memaddrdesc.vaddress         = memaddrdesc.vaddress + (half DIV 8);
            (memstatus, value[1*:half]) = PhysMemRead{half}(memaddrdesc, accdesc);
        end;

    else
        for i = 0 to bytes-1 do
            if !IsFault(memstatus) then     // Do not continue past a fault
                (memstatus, value[i*:8]) = PhysMemRead{8}(memaddrdesc, accdesc);
            end;

            if !IsFault(memstatus) then
                memaddrdesc.paddress.address = memaddrdesc.paddress.address + 1;
                memaddrdesc.vaddress         = memaddrdesc.vaddress + 1;
            end;

        end;
    end;

    if IsFault(memstatus) then
        return (value, memaddrdesc, memstatus);
    end;

    if ((accdesc.acctype != AccessType_IFETCH &&
           ImpDefBool(
               "GPF, GPC, Data SyncExternal Aborts, TagChecks prioritized over Watchpoints")) ||
          (accdesc.acctype == AccessType_IFETCH &&
           ImpDefBool("Instruction SyncExternal Aborts prioritized over Breakpoints"))) then
        memaddrdesc.fault = AArch64_CheckDebug(address, accdesc, bytes);
    end;

    return (value, memaddrdesc, memstatus);
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_MemSingleWrite

```
// AArch64_MemSingleWrite()
// ========================
// Perform an atomic, little-endian write of 'size' bits.

func AArch64_MemSingleWrite{size : integer{8, 16, 32, 64, 128, 256}}(address : bits(64),
                                                                     accdesc_in : AccessDescriptor,
                                                                     aligned : boolean,
                                                                     value : bits(size)
                                                                 ) => (AddressDescriptor, PhysMemRetStatus)
begin
    var accdesc : AccessDescriptor = accdesc_in;
    let bytes : integer{} = size DIV 8;
    if IsFeatureImplemented(FEAT_LSE2) then
        assert ((IsFeatureImplemented(FEAT_LS64WB) &&
                 bytes == 32 && accdesc.acctype == AccessType_ASIMD) ||
                 (IsFeatureImplemented(FEAT_SVE_LRCPC) && bytes == 32) ||
                 AllInSCAGranule(address, bytes));
    else
        assert IsAlignedSize(address, bytes);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    var memaddrdesc : AddressDescriptor;
    var memstatus : PhysMemRetStatus = ARBITRARY : PhysMemRetStatus;
    memaddrdesc = AArch64_TranslateAddress(address, accdesc, aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (memaddrdesc, memstatus);
    end;

    // Effect on exclusives
    if memaddrdesc.mematttrs.shareability != Shareability_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), bytes);
    end;

    if accdesc.tagchecked then
        let ltag : bits(4) = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = FALSE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            memaddrdesc.fault = fault;
            return (memaddrdesc, memstatus);
        end;
    end;

    if SPESampleInFlight then
        let is_load : boolean = FALSE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    var atomic : boolean;
    if IsWBShareable(memaddrdesc.mematttrs) then
        atomic = TRUE;
    elsif accdesc.exclusive then
        atomic = TRUE;
    elsif (accdesc.acctype == AccessType_SVE && accdesc.predicated &&
           bytes == 8 && IsAlignedSize(address, 8)) then
        // An SVE predicated load of a 128-bit element that is 64-bit aligned
        // is treated as a pair of 64-bit single-copy atomic accesses.
        // This is one of the 64-bit single-copy atomic access.
        atomic = TRUE;
    elsif aligned then
        atomic = !accdesc.ispair;
    else
        // Misaligned accesses within a single-copy atomic granule but
        // not Normal Cacheable Writeback are Atomic
        atomic = ImpDefBool("FEAT_LSE2: access is atomic");
    end;
    if atomic then
        memstatus = PhysMemWrite{size}(memaddrdesc, accdesc, value);
        if IsFault(memstatus) then
            return (memaddrdesc, memstatus);
        end;

    elsif accdesc.acctype == AccessType_ASIMD && bytes == 32 && accdesc.ispair then
        // A 32 byte STP (SIMD&FP) that does not target Normal Inner Write-Back, Outer
        // Write-Back cacheable, Shareable memory is treated as four 8 byte atomic accesses.
        // As this access was not split in Mem(), it must be aligned to 32 bytes.
        assert IsAlignedSize(address, 32);
        accdesc.ispair = FALSE;
        for i = 0 to 3 do
            memstatus = PhysMemWrite{64}(memaddrdesc, accdesc, value[64*i+:64]);
            if IsFault(memstatus) then
```

```
                return (memaddrdesc, memstatus);
            end;
            memaddrdesc.paddress.address = memaddrdesc.paddress.address + 8;
            memaddrdesc.vaddress         = memaddrdesc.vaddress + 8;
        end;

    elsif aligned && accdesc.ispair then
        let half : integer{} = (size DIV 2) as integer{32, 64};
        memstatus = PhysMemWrite{half}(memaddrdesc, accdesc, value[0*:half]);
        if IsFault(memstatus) then
            return (memaddrdesc, memstatus);
        end;

        memaddrdesc.paddress.address = memaddrdesc.paddress.address + (half DIV 8);
        memaddrdesc.vaddress         = memaddrdesc.vaddress + (half DIV 8);
        memstatus = PhysMemWrite{half}(memaddrdesc, accdesc, value[1*:half]);
        if IsFault(memstatus) then
            return (memaddrdesc, memstatus);
        end;

    else
        for i = 0 to bytes-1 do
            memstatus = PhysMemWrite{8}(memaddrdesc, accdesc, value[i*:8]);
            if IsFault(memstatus) then
                return (memaddrdesc, memstatus);
            end;
            memaddrdesc.paddress.address = memaddrdesc.paddress.address + 1;
            memaddrdesc.vaddress         = memaddrdesc.vaddress + 1;
        end;
    end;

    if (accdesc.acctype != AccessType_IFETCH &&
          ImpDefBool(
              "GPF, GPC, Data SyncExternal Aborts, TagChecks prioritized over Watchpoints")) then
        memaddrdesc.fault = AArch64_CheckDebug(address, accdesc, bytes);
    end;

    return (memaddrdesc, memstatus);
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_MemTag

```
// AArch64_MemTag - accessor
// =========================

accessor AArch64_MemTag(address : bits(64), accdesc : AccessDescriptor) <=> value : bits(4)
begin
    // Load an Allocation Tag from memory.
    getter
        var value : bits(4);
        var memaddrdesc : AddressDescriptor;
        var memstatus : PhysMemRetStatus;
        (value, memaddrdesc, memstatus) = AArch64_MemTagRead(address, accdesc);

        if IsFault(memaddrdesc) then
            AArch64_Abort(memaddrdesc.fault);
        end;

        if IsFault(memstatus) then
            HandleExternalReadAbort(memstatus, memaddrdesc, 1, accdesc);
        end;

        return value;
    end;

    // Store an Allocation Tag to memory.
    setter
        var memaddrdesc : AddressDescriptor;
        var memstatus : PhysMemRetStatus;
        (memaddrdesc, memstatus) = AArch64_MemTagWrite(address, accdesc, value);

        if IsFault(memaddrdesc) then
            AArch64_Abort(memaddrdesc.fault);
        end;

        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, 1, accdesc);
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_MemTagRead

```
// AArch64_MemTagRead()
// ====================
// Load an Allocation Tag from memory.

func AArch64_MemTagRead(address : bits(64),
                        accdesc_in : AccessDescriptor
                        ) => (bits(4), AddressDescriptor, PhysMemRetStatus)
begin
    assert accdesc_in.tagaccess;

    var accdesc : AccessDescriptor      = accdesc_in;
    var tag : bits(4)                   = ARBITRARY : bits(4);
    var memaddrdesc : AddressDescriptor = ARBITRARY : AddressDescriptor;
    var memtagtype : MemTagType         = ARBITRARY : MemTagType;
    var memstatus : PhysMemRetStatus    = ARBITRARY : PhysMemRetStatus;

    let aligned : boolean = TRUE;

    accdesc.tagaccess = IsMTEEnabled(accdesc.el);

    (memtagtype, memaddrdesc) = AArch64_TranslateTagAddress(address, accdesc, aligned, TAG_GRANULE);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (tag, memaddrdesc, memstatus);
    end;

    if accdesc.tagaccess && memtagtype == MemTag_AllocationTagged then
        (memstatus, tag) = PhysMemTagRead(memaddrdesc, accdesc);
        return (tag, memaddrdesc, memstatus);
    elsif (IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS) &&
           accdesc.tagaccess && memtagtype == MemTag_CanonicallyTagged) then
        tag = if address[55] == '0' then '0000' else '1111';
        return (tag, memaddrdesc, memstatus);
    else
        // Otherwise read the tag as zero
        tag = '0000';
        return (tag, memaddrdesc, memstatus);
    end;
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_MemTagWrite

```
// AArch64_MemTagWrite()
// =====================
// Store an Allocation Tag to memory.

func AArch64_MemTagWrite(address : bits(64),
                         accdesc_in : AccessDescriptor,
                         value : bits(4)) => (AddressDescriptor, PhysMemRetStatus)
begin
    assert accdesc_in.tagaccess;

    var accdesc : AccessDescriptor      = accdesc_in;
    var memaddrdesc : AddressDescriptor = ARBITRARY : AddressDescriptor;
    var memtagtype : MemTagType         = ARBITRARY : MemTagType;
    var memstatus : PhysMemRetStatus    = ARBITRARY : PhysMemRetStatus;

    let aligned : boolean = IsAlignedSize(address, TAG_GRANULE);

    // Stores of allocation tags must be aligned
    if !aligned then
        memaddrdesc.fault = AlignmentFault(accdesc, address);
        return (memaddrdesc, memstatus);
    end;

    accdesc.tagaccess = IsMTEEnabled(accdesc.el);

    (memtagtype, memaddrdesc) = AArch64_TranslateTagAddress(address, accdesc, aligned, TAG_GRANULE);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (memaddrdesc, memstatus);
    end;

    if accdesc.tagaccess && memtagtype == MemTag_AllocationTagged then
        memstatus = PhysMemTagWrite(memaddrdesc, accdesc, value);
    end;
    return (memaddrdesc, memstatus);
end;
```

Library pseudocode for aarch64/functions/memory/AArch64_UnalignedAccessFaults

```
// AArch64_UnalignedAccessFaults()
// ===============================
// Determine whether the unaligned access generates an Alignment fault

func AArch64_UnalignedAccessFaults(accdesc : AccessDescriptor,
                                   address : bits(64),
                                   size : integer) => boolean
begin
    if AlignmentEnforced() then
        return TRUE;
    elsif accdesc.acctype == AccessType_GCS then
        return TRUE;
    elsif accdesc.rcw then
        return TRUE;
    elsif accdesc.ls64 then
        return TRUE;
    elsif (accdesc.exclusive || accdesc.atomicop)  then
        return (!IsFeatureImplemented(FEAT_LSE2) ||
                !AllInSCAGranule(address, size));
    elsif (accdesc.acqsc || accdesc.acqpc || accdesc.relsc) then
        // If nAA is 0, the address accessed by each register in the pair
        // must lie within a single 16-byte aligned block
        if accdesc.ispair then
            return  (SCTLR_ELx().nAA == '0' &&
                       (!AllInAlignedQuantity{64}(address, size DIV 2, 16) ||
                        !AllInAlignedQuantity{64}(address + size DIV 2, size DIV 2, 16)));
        // Octaword load/store are 32 bytes aligned
        elsif IsFeatureImplemented(FEAT_SVE_LRCPC) && (size == 32) then
            return  (SCTLR_ELx().nAA == '0' &&
                       !AllInAlignedQuantity{64}(address, size, 32));
        else
            return  (SCTLR_ELx().nAA == '0' &&
                       !AllInAlignedQuantity{64}(address, size, 16));
        end;
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/functions/memory/AddressSupportsLS64

```
// AddressSupportsLS64()
// =====================
// Returns TRUE if the 64-byte block following the given address supports the
// LD64B and ST64B instructions, and FALSE otherwise.

impdef func AddressSupportsLS64(paddress : bits(NUM_PABITS)) => boolean
begin
    return ImpDefBool("Memory location supports ST64B and LD64B");
end;
```

Library pseudocode for aarch64/functions/memory/AllInSCAGranule

```
// AllInSCAGranule()
// =================
// When FEAT_LSE2 is implemented, for some memory accesses if all bytes of the access lie within
// the single-copy atomicity granule corresponding to CTR_EL0.LSE2Size and satisfy additional
// requirements, then the access is guaranteed to be single-copy atomic.

readonly func AllInSCAGranule(address : bits(64), size : integer) => boolean
begin
    if NUM_SCA_GRANULE_BYTES == 0 then
        // The single-copy atomicity granule size is unbounded.
        return TRUE;
    end;
    return AllInAlignedQuantity{64}(address, size, NUM_SCA_GRANULE_BYTES);
end;
```

Library pseudocode for aarch64/functions/memory/CASCompare

```
// CASCompare()
// ===========
// Performs a comparison for CAS

func CASCompare{N}(oldvalue : bits(N),
                   comparevalue : bits(N),
                   newvalue : bits(N)) => (bits(N), boolean, bits(N))
begin
    var memresult : bits(N);
    var cmpfail : boolean;
    var regresult : bits(N) = oldvalue;

    if oldvalue == comparevalue then
        cmpfail = FALSE;
        memresult = newvalue;
        if !ConstrainUnpredictableBool(Unpredictable_CASRETURNOLDVALUE) then
            regresult = comparevalue;
        end;
    else
        cmpfail = TRUE;
        memresult = oldvalue;
    end;

    return (memresult, cmpfail, regresult);
end;
```

Library pseudocode for aarch64/functions/memory/CheckSPAlignment

```
// CheckSPAlignment()
// ==================
// Check correct stack pointer alignment for AArch64 state.

func CheckSPAlignment()
begin
    let sp : bits(64) = SP{}();
    var stack_align_check : boolean;
    if PSTATE.EL == EL0 then
        stack_align_check = (SCTLR_ELx().SA0 != '0');
    else
        stack_align_check = (SCTLR_ELx().SA != '0');
    end;

    if stack_align_check && sp != AlignDownSize(sp, 16) then
        AArch64_SPAlignmentFault();
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/memory/IsConventionalMemory

```
// IsConventionalMemory()
// ======================
// Returns TRUE if the memory location is in Conventional memory, and FALSE otherwise.

impdef func IsConventionalMemory(addrdesc : AddressDescriptor) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/memory/Mem

```
// Mem - accessor
// ==============

accessor Mem{size : integer{8, 16, 32, 64, 128, 256}}(address : bits(64),
                                                      accdesc_in : AccessDescriptor
                                                    ) <=>  value_in : bits(size)
begin
    // Perform a read of 'size' bits. The access byte order is reversed for a big-endian access.
    // Instruction fetches would call AArch64_MemSingle directly.
    getter
        var accdesc : AccessDescriptor = accdesc_in;
        let bytes : integer{} = size DIV 8;
        var value : bits(size);

        // Check alignment on size of element accessed, not overall access size
        var aligned : boolean;
        if accdesc.ispair && !accdesc.exclusive then
            let half : integer{} = (size DIV 2) as integer{32, 64, 128};
            aligned = IsAlignedSize(address, half DIV 8);
        else
            aligned = IsAlignedSize(address, bytes);
        end;

        if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, bytes) then
            let fault : FaultRecord = AlignmentFault(accdesc, address);
            AArch64_Abort(fault);
        end;
        if accdesc.acctype == AccessType_ASIMD && bytes == 16 && IsAlignedSize(address, 8) then
            // If 128-bit SIMD&FP ordered access are treated as a pair of
            // 64-bit single-copy atomic accesses, then these single copy atomic
            // access can be observed in any order.
            let half : integer{} = (size DIV 2) as integer{64};
            let highaddress : bits(64) = AddressIncrement(address, half DIV 8, accdesc);
            value[0*:half] = AArch64_MemSingle{half}(address, accdesc, aligned);
            value[1*:half] = AArch64_MemSingle{half}(highaddress, accdesc, aligned);
        elsif (IsFeatureImplemented(FEAT_LS64WB) && accdesc.acctype == AccessType_ASIMD &&
                bytes == 32 && accdesc.ispair && IsAlignedSize(address, 32)) then
            value = AArch64_MemSingle{size}(address, accdesc, aligned);
        elsif accdesc.acctype == AccessType_ASIMD && bytes == 32 && IsAlignedSize(address, 8) then
            // If a 32 byte LDP (SIMD&FP) access is not aligned to 32 bytes but aligned to
            // 8 bytes, it is treated as four 8 byte single-copy atomic accesses.
            accdesc.ispair = FALSE;
            aligned        = TRUE;
            for i = 0 to 3 do
                let blockaddress : bits(64) = AddressIncrement(address, i*8, accdesc);
                value[64*i+:64] = AArch64_MemSingle{64}(blockaddress, accdesc, aligned);
            end;
        elsif (IsFeatureImplemented(FEAT_LSE2) &&
            AllInSCAGranule(address, bytes)) then
            value = AArch64_MemSingle{size}(address, accdesc, aligned);
        elsif ((aligned && accdesc.ispair) ||
                  (accdesc.acctype == AccessType_SVE && accdesc.predicated &&
                     bytes == 16 && IsAlignedSize(address, 8))) then
            // Either: an aligned pair access, OR
            // an SVE predicated load of a 128-bit element that is 64-bit aligned,
            // which is treated as two 64-bit single-copy atomic accesses.
            accdesc.ispair = FALSE;
            let half : integer{} = (size DIV 2) as integer{32, 64, 128};
            let highaddress : bits(64) = AddressIncrement(address, half DIV 8, accdesc);
            if IsFeatureImplemented(FEAT_LRCPC3) && accdesc.highestaddressfirst then
                value[1*:half] = AArch64_MemSingle{half}(highaddress, accdesc, aligned);
                value[0*:half] = AArch64_MemSingle{half}(address, accdesc, aligned);
            else
                value[0*:half] = AArch64_MemSingle{half}(address, accdesc, aligned);
                value[1*:half] = AArch64_MemSingle{half}(highaddress, accdesc, aligned);
            end;
        elsif aligned then
            value = AArch64_MemSingle{size}(address, accdesc, aligned);
        else
            assert bytes > 1;
            if (IsFeatureImplemented(FEAT_LRCPC3) && accdesc.ispair &&
                    accdesc.highestaddressfirst) then
                let half : integer{} = (size DIV 2) as integer{32, 64, 128};
                var lowhalf, highhalf : bits(half);

                for i = 0 to (half DIV 8)-1 do
                    let byteaddress : bits(64) = AddressIncrement(address,
                                                                  (half DIV 8) + i, accdesc);
                    // Individual byte access can be observed in any order
                    highhalf[i*:8] = AArch64_MemSingle{8}(byteaddress, accdesc, aligned);
                end;
                for i = 0 to (half DIV 8)-1 do
                    let byteaddress : bits(64) = AddressIncrement(address, i, accdesc);
                    // Individual byte access can be observed in any order
                    lowhalf[i*:8] = AArch64_MemSingle{8}(byteaddress, accdesc, aligned);
                end;

                value = highhalf::lowhalf;
```

```
        else
            value[7:0] = AArch64_MemSingle{8}(address, accdesc, aligned);
            accdesc.lowestaddress = FALSE;

            // For subsequent bytes, if they cross to a new translation page which assigns
            // Device memory type, it is CONSTRAINED UNPREDICTABLE whether an unaligned access
            // will generate an Alignment Fault.
            let c : Constraint = ConstrainUnpredictable(Unpredictable_DEVPAGE2);
            assert c IN {Constraint_FAULT, Constraint_NONE};
            if c == Constraint_NONE then aligned = TRUE; end;

            for i = 1 to bytes-1 do
                let byteaddress : bits(64) = AddressIncrement(address, i, accdesc);
                value[i*:8] = AArch64_MemSingle{8}(byteaddress, accdesc, aligned);
            end;
        end;
    end;

    if BigEndian(accdesc.acctype) then
        value = BigEndianReverse{size}(value);
    end;

    return value;
end;

// Perform a write of 'size' bits. The byte order is reversed for a big-endian access.
setter
    var value : bits(size) = value_in;
    let bytes : integer{} = size DIV 8;
    var accdesc : AccessDescriptor = accdesc_in;

    // Check alignment on size of element accessed, not overall access size
    var aligned : boolean;
    if accdesc.ispair && !accdesc.exclusive then
        let half : integer{} = (size DIV 2) as integer{32, 64, 128};
        aligned = IsAlignedSize(address, half DIV 8);
    else
        aligned = IsAlignedSize(address, bytes);
    end;
    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, bytes) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    if BigEndian(accdesc.acctype) then
        value = BigEndianReverse{size}(value);
    end;
    if accdesc.acctype == AccessType_ASIMD && bytes == 16 && IsAlignedSize(address, 8) then
        let half : integer{} = (size DIV 2) as integer{64};
        // 128-bit SIMD&FP stores are treated as a pair of 64-bit single-copy atomic accesses
        // 64-bit aligned.
        let highaddress : bits(64) = AddressIncrement(address, half DIV 8, accdesc);
        AArch64_MemSingle{half}(address, accdesc, aligned) = value[0+:half];
        AArch64_MemSingle{half}(highaddress, accdesc, aligned) = value[half+:half];
    elsif (IsFeatureImplemented(FEAT_LS64WB) && accdesc.acctype == AccessType_ASIMD &&
            bytes == 32 && accdesc.ispair && IsAlignedSize(address, 32)) then
        AArch64_MemSingle{size}(address, accdesc, aligned) = value;
    elsif accdesc.acctype == AccessType_ASIMD && bytes == 32 && IsAlignedSize(address, 8) then
        // If a 32 byte STP (SIMD&FP) access is not aligned to 32 bytes but aligned to
        // 8 bytes, it is treated as four 8 byte single-copy atomic accesses.
        accdesc.ispair = FALSE;
        aligned        = TRUE;
        for i = 0 to 3 do
            let blockaddress : bits(64) = AddressIncrement(address, i*8, accdesc);
            AArch64_MemSingle{64}(blockaddress, accdesc, aligned) = value[64*i+:64];
        end;
    elsif (IsFeatureImplemented(FEAT_LSE2) &&
        AllInSCAGranule(address, bytes)) then
        AArch64_MemSingle{size}(address, accdesc, aligned) = value;
    elsif ((aligned && accdesc.ispair) ||
              (accdesc.acctype == AccessType_SVE && accdesc.predicated &&
                bytes == 16 && IsAlignedSize(address, 8))) then
        // Either: an aligned pair access, OR
        // an SVE predicated load of a 128-bit element that is 64-bit aligned,
        // which is treated as two 64-bit single-copy atomic accesses.
        let half : integer{} = (size DIV 2) as integer{32, 64, 128};
        accdesc.ispair = FALSE;
        let highaddress : bits(64) = AddressIncrement(address, half DIV 8, accdesc);
        if IsFeatureImplemented(FEAT_LRCPC3) && accdesc.highestaddressfirst then
            AArch64_MemSingle{half}(highaddress, accdesc, aligned) = value[half+:half];
            AArch64_MemSingle{half}(address, accdesc, aligned) = value[0+:half];
        else
            AArch64_MemSingle{half}(address, accdesc, aligned) = value[0+:half];
            AArch64_MemSingle{half}(highaddress, accdesc, aligned) = value[half+:half];
        end;
    elsif aligned then
        AArch64_MemSingle{size}(address, accdesc, aligned) = value;
    else
        assert bytes > 1;
```

```
            if (IsFeatureImplemented(FEAT_LRCPC3) && accdesc.ispair &&
                    accdesc.highestaddressfirst) then
                let half : integer{} = (size DIV 2) as integer{32, 64};
                var lowhalf, highhalf : bits(half);
                (highhalf, lowhalf) = (value[half+:half], value[0+:half]);
                for i = 0 to (half DIV 8)-1 do
                    let byteaddress : bits(64) = AddressIncrement(address,
                                                                  (half DIV 8) + i, accdesc);
                    // Individual byte access can be observed in any order
                    AArch64_MemSingle{8}(byteaddress, accdesc, aligned) = highhalf[i*:8];
                end;
                for i = 0 to (half DIV 8)-1 do
                    let byteaddress : bits(64) = AddressIncrement(address,
                                                                  (half DIV 8) + i, accdesc);
                    // Individual byte access can be observed in any order,
                    // but implies observability of highhalf
                    AArch64_MemSingle{8}(byteaddress, accdesc, aligned) = lowhalf[i*:8];
                end;
            else
                AArch64_MemSingle{8}(address, accdesc, aligned) = value[7:0];
                accdesc.lowestaddress = FALSE;

                // For subsequent bytes, if they cross to a new translation page which assigns
                // Device memory type, it is CONSTRAINED UNPREDICTABLE whether an unaligned access
                // will generate an Alignment Fault.

                let c : Constraint = ConstrainUnpredictable(Unpredictable_DEVPAGE2);
                assert c IN {Constraint_FAULT, Constraint_NONE};
                if c == Constraint_NONE then aligned = TRUE; end;

                for i = 1 to bytes-1 do
                    let byteaddress : bits(64) = AddressIncrement(address, i, accdesc);
                    AArch64_MemSingle{8}(byteaddress, accdesc, aligned) = value[i*:8];
                end;
            end;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/memory/MemAtomic

```
// MemAtomic()
// ===========
// Performs load and store memory operations for a given virtual address.

func MemAtomic{size : integer{8, 16, 32, 64, 128}}(address : bits(64),
                                                   cmpoperand : bits(size),
                                                   operand : bits(size),
                                                   accdesc_in : AccessDescriptor
                                                  ) => bits(size)
begin
    assert accdesc_in.atomicop;

    let bytes : integer{} = (size DIV 8) as integer{1, 2, 4, 8, 16};

    var newvalue : bits(size);
    var oldvalue : bits(size);
    var accdesc : AccessDescriptor = accdesc_in;
    let aligned : boolean = IsAlignedSize(address, bytes);

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, bytes) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    // MMU or MPU lookup
    let memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(address, accdesc,
                                                                   aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if (!IsWBShareable(memaddrdesc.mematttrs) &&
            ConstrainUnpredictableBool(Unpredictable_Atomic_NOP)) then
        return ARBITRARY : bits(size);
    end;

    // Effect on exclusives
    if memaddrdesc.mematttrs.shareability != Shareability_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), bytes);
    end;

    // For Store-only Tag checking, the tag check is performed on the store.
    if (accdesc.tagchecked &&
          (!IsFeatureImplemented(FEAT_MTE_STORE_ONLY) ||
           !StoreOnlyTagCheckingEnabled(accdesc.el))) then
        let ltag : bits(4) = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = TRUE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            AArch64_Abort(fault);
        end;
    end;

    // All observers in the shareability domain observe the following load and store atomically.
    var memstatus : PhysMemRetStatus;
    (memstatus, oldvalue) = PhysMemRead{size}(memaddrdesc, accdesc);
    // Depending on the memory type of the physical address, the access might generate
    // either a synchronous External abort or an SError exception
    // among other CONSTRAINED UNPREDICTABLE choices.

    if IsFault(memstatus) then
        HandleExternalReadAbort(memstatus, memaddrdesc, bytes, accdesc);
    end;
    if BigEndian(accdesc.acctype) then
        oldvalue = BigEndianReverse{size}(oldvalue);
    end;

    var cmpfail : boolean = FALSE;
    var retvalue : bits(size) = oldvalue;
    if accdesc.acctype == AccessType_FP then
        newvalue = MemAtomicFP{size}(accdesc.modop, oldvalue, operand);
    else
        (newvalue, cmpfail, retvalue) = MemAtomicInt{size}(accdesc.modop, oldvalue,
                                                           operand, cmpoperand);
    end;

    let requirewrite : boolean = (!cmpfail ||
                                  ConstrainUnpredictableBool(Unpredictable_WRITEFAILEDCAS));

    if IsFeatureImplemented(FEAT_MTE_STORE_ONLY) && StoreOnlyTagCheckingEnabled(accdesc.el) then
        // If the compare on a Tag checked CAS fails and no write is performed, then it is
```

```
        // CONSTRAINED UNPREDICTABLE whether the Tag check is performed.
        if accdesc.tagchecked && !requirewrite then
            accdesc.tagchecked = ConstrainUnpredictableBool(Unpredictable_STRONLYTAGCHECKEDCAS);
        end;

        if accdesc.tagchecked then
            let ltag : bits(4) = AArch64_LogicalAddressTag(address);
            let readcheck : boolean = FALSE;
            var fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck,
                                                       bytes, ltag);
            if fault.statuscode != Fault_None then
                // For a synchronous Tag Check Fault due to FEAT_MTE_STORE_ONLY, set WnR.
                fault.write = TRUE;
                AArch64_Abort(fault);
            end;
        end;
    end;

    if requirewrite then
        if BigEndian(accdesc.acctype) then
            newvalue = BigEndianReverse{size}(newvalue);
        end;
        memstatus = PhysMemWrite{size}(memaddrdesc, accdesc, newvalue);
        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, bytes, accdesc);
        end;
    end;

    if SPESampleInFlight then
        let is_load : boolean = FALSE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    // Load operations return the old (pre-operation) value.
    // Compare and Swap operations return the old (pre-operation) value. For a successful CAS,
    // this might be the value from the compare operand or from memory.
    return retvalue;
end;
```

Library pseudocode for aarch64/functions/memory/MemAtomicFP

```
// MemAtomicFP()
// =============
// Performs FP Atomic operation

func MemAtomicFP{N}(modop : MemAtomicOp, op1 : bits(N), op2 : bits(N)) => bits(N)
begin
    var fpcr : FPCR_Type = FPCR();
    let altfp : boolean  = FALSE;
    let fpexc : boolean  = FALSE;
    fpcr.[AH,DN]   = '01';
    fpcr.FZ        = fpcr.FZ OR fpcr.FIZ; // Flush denormalized input and output to zero
                                          // if either bit is set.
    var result : bits(N);

    case modop of
        when MemAtomicOp_FPADD =>    result = FPAdd{N}(op1, op2, fpcr, fpexc);
        when MemAtomicOp_FPMAX =>    result = FPMax{N}(op1, op2, fpcr, altfp, fpexc);
        when MemAtomicOp_FPMIN =>    result = FPMin{N}(op1, op2, fpcr, altfp, fpexc);
        when MemAtomicOp_FPMAXNM =>  result = FPMaxNum{N}(op1, op2, fpcr, fpexc);
        when MemAtomicOp_FPMINNM =>  result = FPMinNum{N}(op1, op2, fpcr, fpexc);
        when MemAtomicOp_BFADD =>    result = BFAdd{N}(op1, op2, fpcr, fpexc);
        when MemAtomicOp_BFMAX =>    result = BFMax{N}(op1, op2, fpcr, altfp, fpexc);
        when MemAtomicOp_BFMIN =>    result = BFMin{N}(op1, op2, fpcr, altfp, fpexc);
        when MemAtomicOp_BFMAXNM =>  result = BFMaxNum{N}(op1, op2, fpcr, fpexc);
        when MemAtomicOp_BFMINNM =>  result = BFMinNum{N}(op1, op2, fpcr, fpexc);
    end;
    return result;
end;
```

Library pseudocode for aarch64/functions/memory/MemAtomicInt

```
// MemAtomicInt()
// ==============
// Performs Integer Atomic operation

func MemAtomicInt{N}(modop : MemAtomicOp, op1 : bits(N), op2 : bits(N), cmpop : bits(N)
                     ) => (bits(N), boolean, bits(N))
begin
    var result : bits(N);
    var cmpfail : boolean = FALSE;
    var retvalue : bits(N) = op1;

    case modop of
        when MemAtomicOp_ADD =>      result = op1 + op2;
        when MemAtomicOp_BIC =>      result = op1 AND NOT(op2);
        when MemAtomicOp_EOR =>      result = op1 XOR op2;
        when MemAtomicOp_ORR =>      result = op1 OR op2;
        when MemAtomicOp_SMAX =>     result = Max(SInt(op1), SInt(op2))[N-1:0];
        when MemAtomicOp_SMIN =>     result = Min(SInt(op1), SInt(op2))[N-1:0];
        when MemAtomicOp_UMAX =>     result = Max(UInt(op1), UInt(op2))[N-1:0];
        when MemAtomicOp_UMIN =>     result = Min(UInt(op1), UInt(op2))[N-1:0];
        when MemAtomicOp_SWP =>      result = op2;
        when MemAtomicOp_CAS =>      (result, cmpfail, retvalue) = CASCompare{N}(op1, cmpop, op2);
        when MemAtomicOp_GCSSS1 =>   (result, cmpfail, retvalue) = CASCompare{N}(op1, cmpop, op2);
    end;
    return (result, cmpfail, retvalue);
end;
```

Library pseudocode for aarch64/functions/memory/MemAtomicRCW

```
// MemAtomicRCW()
// ==============
// Perform a single-copy-atomic access with Read-Check-Write operation

func MemAtomicRCW{size : integer{64, 128}}(address : bits(64), cmpoperand : bits(size),
                                           operand : bits(size), accdesc_in : AccessDescriptor
                                    ) => (bits(4), bits(size))
begin
    assert accdesc_in.atomicop;
    assert accdesc_in.rcw;

    let bytes : integer{} = (size DIV 8) as integer{8, 16};

    var nzcv : bits(4);
    var oldvalue : bits(size);
    var newvalue : bits(size);
    var accdesc : AccessDescriptor = accdesc_in;
    let aligned : boolean = IsAlignedSize(address, bytes);

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, bytes) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    // MMU or MPU lookup
    let memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(address, accdesc,
                                                                   aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    if (!IsWBShareable(memaddrdesc.memattrs) &&
            ConstrainUnpredictableBool(Unpredictable_Atomic_NOP)) then
        return (ARBITRARY : bits(4), ARBITRARY : bits(size));
    end;

    // Effect on exclusives
    if memaddrdesc.memattrs.shareability != Shareability_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), bytes);
    end;

    // For Store-only Tag checking, the tag check is performed on the store.
    if (accdesc.tagchecked &&
          (!IsFeatureImplemented(FEAT_MTE_STORE_ONLY) ||
           !StoreOnlyTagCheckingEnabled(accdesc.el))) then
        let ltag : bits(4) = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = TRUE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            AArch64_Abort(fault);
        end;
    end;

    // All observers in the shareability domain observe the following load and store atomically.
    var memstatus : PhysMemRetStatus;
    (memstatus, oldvalue) = PhysMemRead{size}(memaddrdesc, accdesc);
    // Depending on the memory type of the physical address, the access might generate
    // either a synchronous External abort or an SError exception
    // among other CONSTRAINED UNPREDICTABLE choices.

    if IsFault(memstatus) then
        HandleExternalReadAbort(memstatus, memaddrdesc, bytes, accdesc);
    end;
    if BigEndian(accdesc.acctype) then
        oldvalue = BigEndianReverse{size}(oldvalue);
    end;

    var cmpfail : boolean = FALSE;
    var retvalue : bits(size) = oldvalue;
    case accdesc.modop of
        when MemAtomicOp_BIC => newvalue = oldvalue AND NOT(operand);
        when MemAtomicOp_ORR => newvalue = oldvalue OR operand;
        when MemAtomicOp_SWP => newvalue = operand;
        when MemAtomicOp_CAS =>
            (newvalue, cmpfail, retvalue) = CASCompare{size}(oldvalue, cmpoperand, operand);
    end;

    if cmpfail then
        nzcv = '1010'; // N = 1 indicates compare failure
    else
        nzcv = RCWCheck{size}(retvalue, newvalue, accdesc.rcws);
```

```
    end;

    // If RCWCheck() passes, it returns nzcv == '0010'
    let requirewrite : boolean = (nzcv == '0010' ||
                                  ConstrainUnpredictableBool(Unpredictable_WRITEFAILEDRCWCAS));

    if IsFeatureImplemented(FEAT_MTE_STORE_ONLY) && StoreOnlyTagCheckingEnabled(accdesc.el) then
        // If the RCW(S) checks, or if the compare on a Tag checked RCW(S)CAS fails and no write is
        // performed, then it is CONSTRAINED UNPREDICTABLE whether the Tag check is performed.
        if accdesc.tagchecked && !requirewrite then
            accdesc.tagchecked = ConstrainUnpredictableBool(Unpredictable_STRONLYTAGCHECKEDRCWSCAS);
        end;

        if accdesc.tagchecked then
            let ltag : bits(4) = AArch64_LogicalAddressTag(address);
            let readcheck : boolean = FALSE;
            var fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck,
                                                       bytes, ltag);
            if fault.statuscode != Fault_None then
                // For a synchronous Tag Check Fault due to FEAT_MTE_STORE_ONLY, set WnR.
                fault.write = TRUE;
                AArch64_Abort(fault);
            end;
        end;
    end;

    if requirewrite then
        if BigEndian(accdesc.acctype) then
            newvalue = BigEndianReverse{size}(newvalue);
        end;

        memstatus = PhysMemWrite{size}(memaddrdesc, accdesc, newvalue);

        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, bytes, accdesc);
        end;
    end;

    if SPESampleInFlight then
        let is_load : boolean = TRUE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    // Load operations return the old (pre-operation) value.
    // Compare and Swap operations return the old (pre-operation) value. For a successful CAS,
    // this might be the value from the compare operand or from memory.
    return (nzcv, retvalue);
end;
```

Library pseudocode for aarch64/functions/memory/MemLoad64B

```
// MemLoad64B()
// ============
// Performs an atomic 64-byte read from a given virtual address.

func MemLoad64B(address : bits(64), accdesc_in : AccessDescriptor) => bits(512)
begin
    let size : integer{} = 512;
    let bytes : integer{} = size DIV 8;
    var data : bits(size);
    var accdesc : AccessDescriptor = accdesc_in;
    let aligned : boolean = IsAlignedSize(address, bytes);

    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, bytes) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    var memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(address, accdesc,
                                                                   aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    // Effect on exclusives
    if memaddrdesc.mematttrs.shareability != Shareability_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), bytes);
    end;

    if accdesc.tagchecked then
        let ltag : bits(4)      = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = TRUE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            AArch64_Abort(fault);
        end;
    end;

    var byte_atomic : boolean = FALSE;
    if ((memaddrdesc.mematttrs.memtype == MemType_Device ||
          (memaddrdesc.mematttrs.inner.attrs == MemAttr_NC &&
             memaddrdesc.mematttrs.outer.attrs == MemAttr_NC)) &&
          !AddressSupportsLS64(memaddrdesc.paddress.address)) then
        let c : Constraint = ConstrainUnpredictable(Unpredictable_LS64UNSUPPORTED);
        assert c IN {Constraint_LIMITED_ATOMICITY, Constraint_FAULT};
        if c == Constraint_FAULT then
            // Generate a stage 1 Data Abort reported using the DFSC code of 110101.
            let fault : FaultRecord = ExclusiveFault(accdesc, address);
            AArch64_Abort(fault);
        else
            byte_atomic = TRUE;
        end;
    elsif IsWBShareable(memaddrdesc.mematttrs) && !IsConventionalMemory(memaddrdesc) then
        if ImpDefBool("LD64B faults to iWBoWB non-Conventional memory") then
            // Generate a Data Abort reported using the DFSC code of 110101.
            let fault : FaultRecord = ExclusiveFault(accdesc, address);
            AArch64_Abort(fault);
        else
            byte_atomic = TRUE;
        end;
    end;

    if SPESampleInFlight then
        let is_load : boolean = TRUE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    var memstatus : PhysMemRetStatus;
    if byte_atomic then
        // Accesses are not single-copy atomic above the byte level.
        for i = 0 to bytes-1 do
            (memstatus, data[i*:8]) = PhysMemRead{8}(memaddrdesc, accdesc);
            if IsFault(memstatus) then
                HandleExternalReadAbort(memstatus, memaddrdesc, 1, accdesc);
            end;
            memaddrdesc.paddress.address = memaddrdesc.paddress.address + 1;
            memaddrdesc.vaddress         = memaddrdesc.vaddress + 1;
        end;
    else
        (memstatus, data) = PhysMemRead{size}(memaddrdesc, accdesc);
        if IsFault(memstatus) then
            HandleExternalReadAbort(memstatus, memaddrdesc, bytes, accdesc);
```

```
        end;
    end;

    return data;
end;
```

Library pseudocode for aarch64/functions/memory/MemStore64B

```
// MemStore64B()
// =============
// Performs an atomic 64-byte store to a given virtual address. Function does
// not return the status of the store.

func MemStore64B(address : bits(64), value : bits(512), accdesc_in : AccessDescriptor)
begin
    let size : integer{} = 512;
    let bytes : integer{} = size DIV 8;
    var accdesc : AccessDescriptor = accdesc_in;
    let aligned : boolean = IsAlignedSize(address, bytes);

    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, bytes) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    var memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(address, accdesc,
                                                                   aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    // Effect on exclusives
    if memaddrdesc.mematttrs.shareability != Shareability_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), 64);
    end;

    if accdesc.tagchecked then
        let ltag : bits(4)      = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = FALSE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            AArch64_Abort(fault);
        end;
    end;

    var byte_atomic : boolean = FALSE;
    if ((memaddrdesc.mematttrs.memtype == MemType_Device ||
          (memaddrdesc.mematttrs.inner.attrs == MemAttr_NC &&
             memaddrdesc.mematttrs.outer.attrs == MemAttr_NC)) &&
          !AddressSupportsLS64(memaddrdesc.paddress.address)) then
        let c : Constraint = ConstrainUnpredictable(Unpredictable_LS64UNSUPPORTED);
        assert c IN {Constraint_LIMITED_ATOMICITY, Constraint_FAULT};
        if c == Constraint_FAULT then
            // Generate a Data Abort reported using the DFSC code of 110101.
            let fault : FaultRecord = ExclusiveFault(accdesc, address);
            AArch64_Abort(fault);
        else
            byte_atomic = TRUE;
        end;
    elsif IsWBShareable(memaddrdesc.mematttrs) && !IsConventionalMemory(memaddrdesc) then
        if ImpDefBool("ST64B faults to iWBoWB non-Conventional memory") then
            // Generate a Data Abort reported using the DFSC code of 110101.
            let fault : FaultRecord = ExclusiveFault(accdesc, address);
            AArch64_Abort(fault);
        else
            byte_atomic = TRUE;
        end;
    end;

    if SPESampleInFlight then
        let is_load : boolean = FALSE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    var memstatus : PhysMemRetStatus;
    if byte_atomic then
        // Accesses are not single-copy atomic above the byte level.
        for i = 0 to bytes-1 do
            memstatus = PhysMemWrite{8}(memaddrdesc, accdesc, value[i*:8]);
            if IsFault(memstatus) then
                HandleExternalWriteAbort(memstatus, memaddrdesc, 1, accdesc);
            end;
            memaddrdesc.paddress.address = memaddrdesc.paddress.address + 1;
            memaddrdesc.vaddress         = memaddrdesc.vaddress + 1;
        end;
    else
        memstatus = PhysMemWrite{size}(memaddrdesc, accdesc, value);
        if IsFault(memstatus) then
            HandleExternalWriteAbort(memstatus, memaddrdesc, bytes, accdesc);
```

```
        end;
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/memory/MemStore64BWithRet

```
// MemStore64BWithRet()
// ====================
// Performs an atomic 64-byte store to a given virtual address returning
// the status value of the operation.

func MemStore64BWithRet(address : bits(64), value : bits(512),
                        accdesc_in : AccessDescriptor) => bits(64)
begin
    let size : integer{} = 512;
    let bytes : integer{} = size DIV 8;
    var accdesc : AccessDescriptor = accdesc_in;
    let aligned : boolean = IsAlignedSize(address, bytes);

    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, bytes) then
        let fault : FaultRecord = AlignmentFault(accdesc, address);
        AArch64_Abort(fault);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    let memaddrdesc : AddressDescriptor = AArch64_TranslateAddress(address, accdesc,
                                                                   aligned, bytes);

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        AArch64_Abort(memaddrdesc.fault);
    end;

    // Effect on exclusives
    if memaddrdesc.mematttrs.shareability != Shareability_NSH then
        ClearExclusiveByAddress(memaddrdesc.paddress, ProcessorID(), 64);
    end;

    if accdesc.tagchecked then
        let ltag : bits(4)      = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = FALSE;
        let fault : FaultRecord = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            AArch64_Abort(fault);
        end;
    end;

    var memstatus : PhysMemRetStatus;
    memstatus = PhysMemWrite{size}(memaddrdesc, accdesc, value);

    // If an access generated by ST64BV or ST64BV0 fails solely because the memory location
    // does not support 64-byte access, then memstatus does not indicate a fault,
    if IsFault(memstatus) then
        HandleExternalWriteAbort(memstatus, memaddrdesc, bytes, accdesc);
    end;

    if SPESampleInFlight then
        let is_load : boolean = FALSE;
        SPESampleLoadStore(is_load, accdesc, memaddrdesc);
    end;

    return memstatus.store64bstatus;
end;
```

Library pseudocode for aarch64/functions/memory/MemStore64BWithRetStatus

```
// MemStore64BWithRetStatus()
// ==========================
// Generates the return status of memory write with ST64BV or ST64BV0
// instructions. The status indicates if the operation succeeded, failed,
// or was not supported at this memory location.

impdef func MemStore64BWithRetStatus() => bits(64)
begin
    return Zeros{64};
end;
```

Library pseudocode for aarch64/functions/memory/NVMem

```
// NVMem - accessor
// ================

accessor NVMem(offset : integer) <=> value : bits(64)
begin
    // This function is the load memory access for the transformed System register read access
    // when Enhanced Nested Virtualization is enabled with HCR_EL2.NV2 = 1.
    // The address for the load memory access is calculated using
    // the formula SignExtend{64}(VNCR_EL2.BADDR :: Offset[11:0]) where,
    //  * VNCR_EL2.BADDR holds the base address of the memory location, and
    //  * Offset is the unique offset value defined architecturally for each System register that
    //    supports transformation of register access to memory access.
    getter
        assert offset > 0 && offset MOD 8 == 0;
        let directread : boolean = FALSE;
        let baddr : bits(64)     = EffectiveBADDR(VNCR_EL2().BADDR :: Zeros{12}, directread);
        let address : bits(64)   = baddr + offset;
        let accdesc : AccessDescriptor = CreateAccDescNV2(MemOp_LOAD);
        return Mem{64}(address, accdesc);
    end;

    // This function is the store memory access for the transformed System register write access
    // when Enhanced Nested Virtualization is enabled with HCR_EL2.NV2 = 1.
    // The address for the store memory access is calculated using
    // the formula SignExtend{64}(VNCR_EL2.BADDR :: Offset[11:0]) where,
    //  * VNCR_EL2.BADDR holds the base address of the memory location, and
    //  * Offset is the unique offset value defined architecturally for each System register that
    //    supports transformation of register access to memory access.
    setter
        assert offset > 0 && offset MOD 8 == 0;
        let directread : boolean = FALSE;
        let baddr : bits(64)     = EffectiveBADDR(VNCR_EL2().BADDR :: Zeros{12}, directread);
        let address : bits(64)   = baddr + offset;
        let accdesc : AccessDescriptor = CreateAccDescNV2(MemOp_STORE);
        Mem{64}(address, accdesc) = value;
        return;
    end;
end;

accessor NVMem128(offset : integer) <=> value : bits(128)
begin
    getter
        assert offset > 0 && offset MOD 16 == 0;
        let directread : boolean = FALSE;
        let baddr : bits(64)     = EffectiveBADDR(VNCR_EL2().BADDR :: Zeros{12}, directread);
        let address : bits(64)   = baddr + offset;
        let accdesc : AccessDescriptor = CreateAccDescNV2(MemOp_LOAD);
        return Mem{128}(address, accdesc);
    end;

    setter
        assert offset > 0 && offset MOD 16 == 0;
        let directread : boolean = FALSE;
        let baddr : bits(64)     = EffectiveBADDR(VNCR_EL2().BADDR :: Zeros{12}, directread);
        let address : bits(64)   = baddr + offset;
        let accdesc : AccessDescriptor = CreateAccDescNV2(MemOp_STORE);
        Mem{128}(address, accdesc) = value;
    end;
end;
```

Library pseudocode for aarch64/functions/memory/PhysMemTagRead

```
// PhysMemTagRead()
// ================
// This is the hardware operation which perform a single-copy atomic,
// Allocation Tag granule aligned, memory access from the tag in PA space.
//
// The function address the array using desc.paddress which supplies:
// * A 52-bit physical address
// * A single NS bit to select between Secure and Non-secure parts of the array.
//
// The accdesc descriptor describes the access type: normal, exclusive, ordered, streaming,
// etc and other parameters required to access the physical memory or for setting syndrome
// register in the event of an External abort.

impdef func PhysMemTagRead(desc : AddressDescriptor,
                           accdesc : AccessDescriptor) => (PhysMemRetStatus, bits(4))
begin
    return (ARBITRARY : PhysMemRetStatus, Zeros{4});
end;
```

Library pseudocode for aarch64/functions/memory/PhysMemTagWrite

```
// PhysMemTagWrite()
// =================
// This is the hardware operation which perform a single-copy atomic,
// Allocation Tag granule aligned, memory access to the tag in PA space.
//
// The function address the array using desc.paddress which supplies:
// * A 52-bit physical address
// * A single NS bit to select between Secure and Non-secure parts of the array.
//
// The accdesc descriptor describes the access type: normal, exclusive, ordered, streaming,
// etc and other parameters required to access the physical memory or for setting syndrome
// register in the event of an External abort.

impdef func PhysMemTagWrite(desc : AddressDescriptor,
                            accdesc : AccessDescriptor,
                            value : bits(4)) => PhysMemRetStatus
begin
    return ARBITRARY : PhysMemRetStatus;
end;
```

Library pseudocode for aarch64/functions/memory/StoreOnlyTagCheckingEnabled

```
// StoreOnlyTagCheckingEnabled()
// =============================
// Returns TRUE if loads executed at the given Exception level are Tag unchecked.

readonly func StoreOnlyTagCheckingEnabled(el : bits(2)) => boolean
begin
    assert IsFeatureImplemented(FEAT_MTE_STORE_ONLY);
    var tcso : bit;

    case el of
        when EL0 =>
            if !ELIsInHost(el) then
                tcso = SCTLR_EL1().TCSO0;
            else
                tcso = SCTLR_EL2().TCSO0;
            end;
        when EL1 =>
            tcso = SCTLR_EL1().TCSO;
        when EL2 =>
            tcso = SCTLR_EL2().TCSO;
        otherwise =>
            tcso = SCTLR_EL3().TCSO;
    end;

    return tcso == '1';
end;
```

Library pseudocode for aarch64/functions/mops/ArchMaxMOPSBlockSize

```
// ArchMaxMOPSBlockSize
// ====================
// Maximum number of bytes CPY/SET instructions can use

constant ArchMaxMOPSBlockSize : integer{} = 0x7FFF_FFFF_FFFF_FFFF;
```

Library pseudocode for aarch64/functions/mops/ArchMaxMOPSCPYSize

```
// ArchMaxMOPSCPYSize
// ==================
// Maximum number of bytes CPY instructions can use

constant ArchMaxMOPSCPYSize : integer{} = 0x007F_FFFF_FFFF_FFFF;
```

Library pseudocode for aarch64/functions/mops/ArchMaxMOPSSETGSize

```
// ArchMaxMOPSSETGSize
// ===================
// Maximum number of bytes SETG instructions can use

constant ArchMaxMOPSSETGSize : integer{} = 0x7FFF_FFFF_FFFF_FFF0;
```

Library pseudocode for aarch64/functions/mops/CPYFOptionA

```
// CPYFOptionA()
// =============
// Returns TRUE if the implementation uses Option A for the
// CPYF* instructions, and FALSE otherwise.

func CPYFOptionA() => boolean
begin
    return ImpDefBool("CPYF* instructions use Option A");
end;
```

Library pseudocode for aarch64/functions/mops/CPYOptionA

```
// CPYOptionA()
// ============
// Returns TRUE if the implementation uses Option A for the
// CPY* instructions, and FALSE otherwise.

func CPYOptionA() => boolean
begin
    return ImpDefBool("CPY* instructions use Option A");
end;
```

Library pseudocode for aarch64/functions/mops/CPYParams

```
// CPYParams
// =========

type CPYParams of record {
    stage : MOPSStage,
    implements_option_a : boolean,
    forward : boolean,
    cpysize : integer,
    stagecpysize : integer,
    toaddress : bits(64),
    fromaddress : bits(64),
    nzcv : bits(4),
    n : integer,
    d : integer,
    s : integer
};
```

Library pseudocode for aarch64/functions/mops/CPYPostSizeChoice

```
// CPYPostSizeChoice()
// ===================
// Returns the size of the copy that is performed by the CPYE* instructions for this
// implementation given the parameters of the destination, source and size of the copy.

impdef func CPYPostSizeChoice(memcpy : CPYParams) => integer
begin
    return ARBITRARY : integer;
end;
```

Library pseudocode for aarch64/functions/mops/CPYPreSizeChoice

```
// CPYPreSizeChoice()
// ==================
// Returns the size of the copy that is performed by the CPYP* instructions for this
// implementation given the parameters of the destination, source and size of the copy.

impdef func CPYPreSizeChoice(memcpy : CPYParams) => integer
begin
    return ARBITRARY : integer;
end;
```

Library pseudocode for aarch64/functions/mops/CPYSizeChoice

```
// CPYSizeChoice()
// ===============
// Returns the size of the block this performed for an iteration of the copy given the
// parameters of the destination, source and size of the copy.

impdef func CPYSizeChoice(memcpy : CPYParams) => MOPSBlockSize
begin
    return ARBITRARY : MOPSBlockSize;
end;
```

Library pseudocode for aarch64/functions/mops/CheckCPYConstrainedUnpredictable

```
// CheckCPYConstrainedUnpredictable()
// ==================================
// Check for CONSTRAINED UNPREDICTABLE behavior in the CPY* and CPYF* instructions.

func CheckCPYConstrainedUnpredictable(n : integer, d : integer, s : integer)
begin
    if (s == n || s == d || n == d) then
        let c : Constraint = ConstrainUnpredictable(Unpredictable_MOPSOVERLAP);
        assert c IN {Constraint_UNDEF, Constraint_NOP};
        case c of
            when Constraint_UNDEF =>      Undefined();
            when Constraint_NOP =>        ExecuteAsNOP();
        end;
    end;

    if (d == 31 || s == 31 || n == 31) then
        let c : Constraint = ConstrainUnpredictable(Unpredictable_MOPS_R31);
        assert c IN {Constraint_UNDEF, Constraint_NOP};
        case c of
            when Constraint_UNDEF =>      Undefined();
            when Constraint_NOP =>        ExecuteAsNOP();
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/mops/CheckMOPSEnabled

```
// CheckMOPSEnabled()
// ==================
// Check for EL0 and EL1 access to the CPY* and SET* instructions.

func CheckMOPSEnabled()
begin
    if (PSTATE.EL IN {EL0, EL1} && EL2Enabled() && !ELIsInHost(EL0) &&
        (!IsHCRXEL2Enabled() || HCRX_EL2().MSCEn == '0')) then
        Undefined();
    end;

    if PSTATE.EL == EL0 && !IsInHost() && SCTLR_EL1().MSCEn == '0' then
        Undefined();
    end;

    if PSTATE.EL == EL0 && IsInHost() && SCTLR_EL2().MSCEn == '0' then
        Undefined();
    end;
end;
```

Library pseudocode for aarch64/functions/mops/CheckMemCpyParams

```
// CheckMemCpyParams()
// ===================
// Check if the parameters to a CPY* or CPYF* instruction are consistent with the
// PE state and well-formed.

func CheckMemCpyParams(memcpy : CPYParams, options : bits(4))
begin
    let from_epilogue : boolean = memcpy.stage == MOPSStage_Epilogue;

    // Check if this version is consistent with the state of the call.
    if ((memcpy.stagecpysize != 0 || MemStageCpyZeroSizeCheck()) &&
          (memcpy.cpysize != 0 || MemCpyZeroSizeCheck())) then
        let using_option_a : boolean = memcpy.nzcv[1] == '0';
        if memcpy.implements_option_a != using_option_a then
            let formatoption : bits(2) = '1':: (if memcpy.implements_option_a then '1' else '0');
            MismatchedMemCpyException(memcpy, options, formatoption);
        end;
    end;

    // Check if the parameters to this instruction are valid.
    if memcpy.stage == MOPSStage_Main then
        if MemCpyParametersIllformedM(memcpy) then
            let formatoption : bits(2) = '0':: (if memcpy.implements_option_a then '1' else '0');
            MismatchedMemCpyException(memcpy, options, formatoption);
        end;
    else
        let postsize : integer = CPYPostSizeChoice(memcpy);
        if memcpy.cpysize != postsize || MemCpyParametersIllformedE(memcpy) then
            let formatoption : bits(2) = '0':: (if memcpy.implements_option_a then '1' else '0');
            MismatchedMemCpyException(memcpy, options, formatoption);
        end;
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/mops/CheckMemSetParams

```
// CheckMemSetParams()
// ===================
// Check if the parameters to a SET* or SETG* instruction are consistent with the
// PE state and well-formed.

func CheckMemSetParams(memset : SETParams, options : bits(2))
begin
    let from_epilogue : boolean = memset.stage == MOPSStage_Epilogue;

    // Check if this version is consistent with the state of the call.
    if ((memset.stagesetsize != 0 || MemStageSetZeroSizeCheck()) &&
          (memset.setsize != 0 || MemSetZeroSizeCheck())) then
        let using_option_a : boolean = memset.nzcv[1] == '0';
        if memset.implements_option_a != using_option_a then
            let formatoption : bits(2) = '1':: (if memset.implements_option_a then '1' else '0');
            MismatchedMemSetException(memset, options, formatoption);
        end;
    end;

    // Check if the parameters to this instruction are valid.
    if memset.stage == MOPSStage_Main then
        if MemSetParametersIllformedM(memset) then
            let formatoption : bits(2) = '0':: (if memset.implements_option_a then '1' else '0');
            MismatchedMemSetException(memset, options, formatoption);
        end;
    else
        let postsize : integer = SETPostSizeChoice(memset);
        if memset.setsize != postsize || MemSetParametersIllformedE(memset) then
            let formatoption : bits(2) = '0':: (if memset.implements_option_a then '1' else '0');
            MismatchedMemSetException(memset, options, formatoption);
        end;
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/mops/CheckSETConstrainedUnpredictable

```
// CheckSETConstrainedUnpredictable()
// ==================================
// Check for CONSTRAINED UNPREDICTABLE behavior in the SET* and SETG* instructions.

func CheckSETConstrainedUnpredictable(memset : SETParams)
begin
    if (memset.s == memset.n || memset.s == memset.d || memset.n == memset.d) then
        let c : Constraint = ConstrainUnpredictable(Unpredictable_MOPSOVERLAP);
        assert c IN {Constraint_UNDEF, Constraint_NOP};
        case c of
            when Constraint_UNDEF => Undefined();
            when Constraint_NOP   => ExecuteAsNOP();
        end;
    end;

    if (memset.d == 31 || memset.n == 31) then
        let c : Constraint = ConstrainUnpredictable(Unpredictable_MOPS_R31);
        assert c IN {Constraint_UNDEF, Constraint_NOP};
        case c of
            when Constraint_UNDEF => Undefined();
            when Constraint_NOP   => ExecuteAsNOP();
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/mops/IsMemCpyForward

```
// IsMemCpyForward()
// =================
// Returns TRUE if in a memcpy of size cpysize bytes from the source address fromaddress
// to destination address toaddress is done in the forward direction on this implementation.

func IsMemCpyForward(memcpy : CPYParams) => boolean
begin
    var forward : boolean;

    // Check for overlapping cases
    if ((UInt(memcpy.fromaddress[55:0]) > UInt(memcpy.toaddress[55:0])) &&
          (UInt(memcpy.fromaddress[55:0]) < UInt(ZeroExtend{64}(memcpy.toaddress[55:0]) +
             memcpy.cpysize))) then
        forward = TRUE;

    elsif ((UInt(memcpy.fromaddress[55:0]) < UInt(memcpy.toaddress[55:0])) &&
             (UInt(ZeroExtend{64}(memcpy.fromaddress[55:0]) + memcpy.cpysize) >
                UInt(memcpy.toaddress[55:0]))) then
        forward = FALSE;

    // Non-overlapping case
    else
        forward = ImpDefBool("CPY in the forward direction");
    end;

    return forward;
end;
```

Library pseudocode for aarch64/functions/mops/MOPSBlockSize

```
// MOPSBlockSize
// =================

type MOPSBlockSize of integer{0..MaxMOPSBlockSize};
```

Library pseudocode for aarch64/functions/mops/MOPSStage

```
// MOPSStage
// =========

type MOPSStage of enumeration { MOPSStage_Prologue, MOPSStage_Main, MOPSStage_Epilogue };
```

Library pseudocode for aarch64/functions/mops/MaxMOPSBlockSize

```
// MaxMOPSBlockSize
// ================
// Maximum number of bytes CPY/SET instructions can use

config MaxMOPSBlockSize : integer{1..ArchMaxMOPSBlockSize} = 64;
```

Library pseudocode for aarch64/functions/mops/MemCpyBytes

```
// MemCpyBytes()
// =============
// Copies 'bytes' bytes of memory from fromaddress to toaddress.
// The integer return parameter indicates the number of bytes copied. The boolean return parameter
// indicates if a Fault or Abort occurred on the write. The AddressDescriptor and PhysMemRetStatus
// parameters contain Fault or Abort information for the caller to handle.

func MemCpyBytes(toaddress : bits(64),
                 fromaddress : bits(64),
                 forward : boolean,
                 bytes : MOPSBlockSize,
                 raccdesc : AccessDescriptor,
                 waccdesc : AccessDescriptor
                ) => (integer, boolean, AddressDescriptor, PhysMemRetStatus)
begin
    var rmemaddrdesc : AddressDescriptor;                // AddressDescriptor for reads
    var rmemstatus : PhysMemRetStatus;                  // PhysMemRetStatus  for writes
    rmemaddrdesc.fault    = NoFault();
    rmemstatus.statuscode = Fault_None;

    var wmemaddrdesc : AddressDescriptor;                // AddressDescriptor for writes
    var wmemstatus : PhysMemRetStatus;                  // PhysMemRetStatus  for writes
    wmemaddrdesc.fault    = NoFault();
    wmemstatus.statuscode = Fault_None;

    var value : bits(8*bytes);
    let aligned : boolean = TRUE;

    if forward then
        var read : integer  = 0;                         // Bytes read
        var write : integer = 0;                         // Bytes written

        // Read until all bytes are read or until a fault is encountered.
        while (read < bytes && !IsFault(rmemaddrdesc) && !IsFault(rmemstatus))
                                          looplimit MaxMOPSBlockSize do
            (value[8 * read +:8], rmemaddrdesc, rmemstatus) = AArch64_MemSingleRead{8}(
                                                                fromaddress + read,
                                                                raccdesc, aligned);
            read = read + 1;
        end;

        // Ensure no UNKNOWN data is written.
        if IsFault(rmemaddrdesc) || IsFault(rmemstatus) then
            read = read - 1;
        end;

        // Write all bytes that were read or until a fault is encountered.
        while write < read && !IsFault(wmemaddrdesc) && !IsFault(wmemstatus)
                                     looplimit MaxMOPSBlockSize  do
            (wmemaddrdesc, wmemstatus) = AArch64_MemSingleWrite{8}(toaddress + write,
                                                                   waccdesc, aligned,
                                                                   value[8 * write +:8]);
            write = write + 1;
        end;

        // Check all bytes were written.
        if IsFault(wmemaddrdesc) || IsFault(wmemstatus) then
            let fault_on_write : boolean = TRUE;
            return (write - 1, fault_on_write, wmemaddrdesc, wmemstatus);
        end;

        // Check all bytes were read.
        if IsFault(rmemaddrdesc) || IsFault(rmemstatus) then
            let fault_on_write : boolean = FALSE;
            return (read, fault_on_write, rmemaddrdesc, rmemstatus);
        end;

    else
        var read : integer  = bytes;                     // Bytes to read
        var write : integer = bytes;                     // Bytes to write

        // Read until all bytes are read or until a fault is encountered.
        while read > 0 && !IsFault(rmemaddrdesc) && !IsFault(rmemstatus)
                                          looplimit MaxMOPSBlockSize do
            read = read - 1;
            (value[8 * read +:8], rmemaddrdesc, rmemstatus) = AArch64_MemSingleRead{8}(
                                                                fromaddress + read,
                                                                raccdesc, aligned);
        end;

        // Ensure no UNKNOWN data is written.
        if IsFault(rmemaddrdesc) || IsFault(rmemstatus) then
            read = read + 1;
        end;

        // Write all bytes that were read or until a fault is encountered.
        while write > read && !IsFault(wmemaddrdesc) && !IsFault(wmemstatus)
                                        looplimit MaxMOPSBlockSize do
            write = write - 1;
```

```
            (wmemaddrdesc, wmemstatus) = AArch64_MemSingleWrite{8}(toaddress + write,
                                                                   waccdesc, aligned,
                                                                   value[8 * write +:8]);
        end;

        // Check all bytes were written.
        if IsFault(wmemaddrdesc) || IsFault(wmemstatus) then
            let fault_on_write : boolean = TRUE;
            return (bytes - (write + 1), fault_on_write, wmemaddrdesc, wmemstatus);
        end;

        // Check all bytes were read.
        if IsFault(rmemaddrdesc) || IsFault(rmemstatus) then
            let fault_on_write : boolean = FALSE;
            return (bytes - read, fault_on_write, rmemaddrdesc, rmemstatus);
        end;
    end;

    // Return any AddressDescriptor and PhysMemRetStatus.
    return (bytes, FALSE, wmemaddrdesc, wmemstatus);
end;
```

Library pseudocode for aarch64/functions/mops/MemCpyParametersIllformedE

```
// MemCpyParametersIllformedE()
// ============================
// Returns TRUE if the inputs are not well formed (in terms of their size and/or alignment)
// for a CPYE* instruction for this implementation given the parameters of the destination,
// source and size of the copy.

impdef func MemCpyParametersIllformedE(memcpy : CPYParams) => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/functions/mops/MemCpyParametersIllformedM

```
// MemCpyParametersIllformedM()
// ============================
// Returns TRUE if the inputs are not well formed (in terms of their size and/or alignment)
// for a CPYM* instruction for this implementation given the parameters of the destination,
// source and size of the copy.

impdef func MemCpyParametersIllformedM(memcpy : CPYParams) => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/functions/mops/MemCpyStageSize

```
// MemCpyStageSize()
// =================
// Returns the number of bytes copied by the given stage of a CPY* or CPYF* instruction.

func MemCpyStageSize(memcpy : CPYParams) => integer
begin
    var stagecpysize : integer;

    if memcpy.stage == MOPSStage_Prologue then
        // IMP DEF selection of the amount covered by pre-processing.
        stagecpysize = CPYPreSizeChoice(memcpy);
        assert stagecpysize == 0 || (stagecpysize < 0) == (memcpy.cpysize < 0);

        if memcpy.cpysize > 0 then
            assert stagecpysize <= memcpy.cpysize;
        else
            assert stagecpysize >= memcpy.cpysize;
        end;

    else
        let postsize : integer = CPYPostSizeChoice(memcpy);
        assert postsize == 0 || (postsize < 0) == (memcpy.cpysize < 0);

        if memcpy.stage == MOPSStage_Main then
            stagecpysize = memcpy.cpysize - postsize;
        else
            stagecpysize = postsize;
        end;
    end;

    return stagecpysize;
end;
```

Library pseudocode for aarch64/functions/mops/MemCpyZeroSizeCheck

```
// MemCpyZeroSizeCheck()
// =====================
// Returns TRUE if the implementation option is checked on a copy of size zero remaining.

func MemCpyZeroSizeCheck() => boolean
begin
    return ImpDefBool("Implementation option is checked with a cpysize of 0");
end;
```

Library pseudocode for aarch64/functions/mops/MemSetBytes

```
// MemSetBytes()
// =============
// Writes a byte of data to the given address 'bytes' times.
// The integer return parameter indicates the number of bytes set. The AddressDescriptor and
// PhysMemRetStatus parameters contain Fault or Abort information for the caller to handle.

func MemSetBytes(toaddress : bits(64), data : bits(8),
                 bytes : MOPSBlockSize,
                 accdesc : AccessDescriptor) => (integer, AddressDescriptor, PhysMemRetStatus)
begin
    var memaddrdesc : AddressDescriptor;
    var memstatus : PhysMemRetStatus;
    memaddrdesc.fault     = NoFault();
    memstatus.statuscode = Fault_None;

    let aligned : boolean = TRUE;
    var write : integer   = 0;                              // Bytes written

    // Write until all bytes are written or a fault is encountered.
    while write < bytes && !IsFault(memaddrdesc) && !IsFault(memstatus)
                                            looplimit MaxMOPSBlockSize do
        (memaddrdesc, memstatus) = AArch64_MemSingleWrite{8}(toaddress + write, accdesc,
                                                             aligned, data);
        write = write + 1;
    end;

    // Check all bytes were written.
    if IsFault(memaddrdesc) || IsFault(memstatus) then
        return (write - 1, memaddrdesc, memstatus);
    end;

    return (bytes, memaddrdesc, memstatus);
end;
```

Library pseudocode for aarch64/functions/mops/MemSetParametersIllformedE

```
// MemSetParametersIllformedE()
// ============================
// Returns TRUE if the inputs are not well formed (in terms of their size and/or
// alignment) for a SETE* or SETGE* instruction for this implementation given the
// parameters of the destination and size of the set.

impdef func MemSetParametersIllformedE(memset : SETParams) => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/functions/mops/MemSetParametersIllformedM

```
// MemSetParametersIllformedM()
// ============================
// Returns TRUE if the inputs are not well formed (in terms of their size and/or
// alignment) for a SETM* or SETGM* instruction for this implementation given the
// parameters of the destination and size of the copy.

impdef func MemSetParametersIllformedM(memset : SETParams) => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/functions/mops/MemSetStageSize

```
// MemSetStageSize()
// =================
// Returns the number of bytes set by the given stage of a SET* or SETG* instruction.

func MemSetStageSize(memset : SETParams) => integer
begin
    var stagesetsize : integer;

    if memset.stage == MOPSStage_Prologue then
        // IMP DEF selection of the amount covered by pre-processing.
        stagesetsize = SETPreSizeChoice(memset);
        assert stagesetsize == 0 || (stagesetsize < 0) == (memset.setsize < 0);

        if memset.is_setg then assert stagesetsize[3:0] == '0000'; end;

        if memset.setsize > 0 then
            assert stagesetsize <= memset.setsize;
        else
            assert stagesetsize >= memset.setsize;
        end;

    else
        let postsize : integer = SETPostSizeChoice(memset);
        assert postsize == 0 || (postsize < 0) == (memset.setsize < 0);
        if memset.is_setg then assert postsize[3:0] == '0000'; end;

        if memset.stage == MOPSStage_Main then
            stagesetsize = memset.setsize - postsize;
        else
            stagesetsize = postsize;
        end;
    end;

    return stagesetsize;
end;
```

Library pseudocode for aarch64/functions/mops/MemSetTags

```
// MemSetTags()
// ============
// Write Allocation Tags for each Tag Granule in 'size'.
// The integer return parameter indicates the number of Tag Granules written. The
// AddressDescriptor and PhysMemRetStatus parameters contain Fault or Abort information for
// the caller to handle.

func MemSetTags(toaddress : bits(64), tag : bits(4),
                size : integer, accdesc : AccessDescriptor
            ) => (integer, AddressDescriptor, PhysMemRetStatus)
begin
    assert IsAlignedSize{64}(toaddress, TAG_GRANULE) && size MOD TAG_GRANULE == 0;

    var memaddrdesc : AddressDescriptor;
    var memstatus : PhysMemRetStatus;
    memaddrdesc.fault     = NoFault();
    memstatus.statuscode  = Fault_None;
    var tagstep : integer = size DIV TAG_GRANULE;

    // Write until all tags are written or a fault is encountered.
    while tagstep > 0 && !IsFault(memaddrdesc) && !IsFault(memstatus)
                                              looplimit ArchMaxMOPSSETGSize DIV TAG_GRANULE do
        let tagaddr : bits(64) = toaddress + (tagstep - 1) * TAG_GRANULE;
        (memaddrdesc, memstatus) = AArch64_MemTagWrite(tagaddr, accdesc, tag);
        tagstep = tagstep - 1;
    end;

    if IsFault(memaddrdesc) || IsFault(memstatus) then
        tagstep = tagstep + 1;
    end;

    let tags_written : integer = (size DIV TAG_GRANULE) - tagstep;
    return (tags_written, memaddrdesc, memstatus);
end;
```

Library pseudocode for aarch64/functions/mops/MemSetZeroSizeCheck

```
// MemSetZeroSizeCheck()
// =====================
// Returns TRUE if the implementation option is checked on a set of size zero remaining.

func MemSetZeroSizeCheck() => boolean
begin
    return ImpDefBool("Implementation option is checked with a setsize of 0");
end;
```

Library pseudocode for aarch64/functions/mops/MemStageCpyZeroSizeCheck

```
// MemStageCpyZeroSizeCheck()
// ==========================
// Returns TRUE if the implementation option is checked on a stage copy of size zero remaining.

func MemStageCpyZeroSizeCheck() => boolean
begin
    return (ImpDefBool(
            "Implementation option is checked with a stage cpysize of 0"));
end;
```

Library pseudocode for aarch64/functions/mops/MemStageSetZeroSizeCheck

```
// MemStageSetZeroSizeCheck()
// ==========================
// Returns TRUE if the implementation option is checked on a stage set of size zero remaining.

func MemStageSetZeroSizeCheck() => boolean
begin
    return (ImpDefBool(
            "Implementation option is checked with a stage setsize of 0"));
end;
```

Library pseudocode for aarch64/functions/mops/MismatchedCpySetTargetEL

```
// MismatchedCpySetTargetEL()
// ==========================
// Return the target exception level for an Exception_MemCpyMemSet.

func MismatchedCpySetTargetEL() => bits(2)
begin
    var target_el : bits(2);

    if UInt(PSTATE.EL) > UInt(EL1) then
        target_el = PSTATE.EL;
    elsif PSTATE.EL == EL0 && EL2Enabled() && HCR_EL2().TGE == '1' then
        target_el = EL2;
    elsif (PSTATE.EL == EL1 && EL2Enabled() &&
        IsHCRXEL2Enabled() && HCRX_EL2().MCE2 == '1') then
        target_el = EL2;
    else
        target_el = EL1;
    end;

    return target_el;
end;
```

Library pseudocode for aarch64/functions/mops/MismatchedMemCpyException

```
// MismatchedMemCpyException()
// ===========================
// Generates an exception for a CPY* instruction if the version
// is inconsistent with the state of the call.

func MismatchedMemCpyException(memcpy : CPYParams, options : bits(4), formatoption : bits(2))
begin
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer                 = 0x0;
    let target_el : bits(2)                   = MismatchedCpySetTargetEL();

    var except : ExceptionRecord = ExceptionSyndrome(Exception_MemCpyMemSet);
    except.syndrome.iss[24]    = '0';
    except.syndrome.iss[23]    = '0';
    except.syndrome.iss[22:19] = options;
    except.syndrome.iss[18]    = if memcpy.stage == MOPSStage_Epilogue then '1' else '0';
    except.syndrome.iss[17:16] = formatoption;
    // exception.syndrome[15] is RES0.
    except.syndrome.iss[14:10] = memcpy.d[4:0];
    except.syndrome.iss[9:5]   = memcpy.s[4:0];
    except.syndrome.iss[4:0]   = memcpy.n[4:0];

    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/functions/mops/MismatchedMemSetException

```
// MismatchedMemSetException()
// ===========================
// Generates an exception for a SET* instruction if the version
// is inconsistent with the state of the call.

func MismatchedMemSetException(memset : SETParams, options : bits(2), formatoption : bits(2))
begin
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;
    let target_el : bits(2) = MismatchedCpySetTargetEL();

    var except : ExceptionRecord = ExceptionSyndrome(Exception_MemCpyMemSet);
    except.syndrome.iss[24]    = '1';
    except.syndrome.iss[23]    = if memset.is_setg then '1' else '0';
    // exception.syndrome[22:21] is RES0.
    except.syndrome.iss[20:19] = options;
    except.syndrome.iss[18]    = if memset.stage == MOPSStage_Epilogue then '1' else '0';
    except.syndrome.iss[17:16] = formatoption;
    // exception.syndrome[15] is RES0.
    except.syndrome.iss[14:10] = memset.d[4:0];
    except.syndrome.iss[9:5]   = memset.s[4:0];
    except.syndrome.iss[4:0]   = memset.n[4:0];

    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/functions/mops/SETGOptionA

```
// SETGOptionA()
// =============
// Returns TRUE if the implementation uses Option A for the
// SETG* instructions, and FALSE otherwise.

func SETGOptionA() => boolean
begin
    return ImpDefBool("SETG* instructions use Option A");
end;
```

Library pseudocode for aarch64/functions/mops/SETOptionA

```
// SETOptionA()
// ============
// Returns TRUE if the implementation uses Option A for the
// SET* instructions, and FALSE otherwise.

func SETOptionA() => boolean
begin
    return ImpDefBool("SET* instructions use Option A");
end;
```

Library pseudocode for aarch64/functions/mops/SETParams

```
// SETParams
// =========

type SETParams of record {
    stage : MOPSStage,
    implements_option_a : boolean,
    is_setg : boolean,
    setsize : integer,
    stagesetsize : integer,
    toaddress : bits(64),
    nzcv : bits(4),
    n : integer,
    d : integer,
    s : integer
};
```

Library pseudocode for aarch64/functions/mops/SETPostSizeChoice

```
// SETPostSizeChoice()
// ===================
// Returns the size of the set that is performed by the SETE* or SETGE* instructions
// for this implementation, given the parameters of the destination and size of the set.

impdef func SETPostSizeChoice(memset : SETParams) => integer
begin
    return ARBITRARY : integer;
end;
```

Library pseudocode for aarch64/functions/mops/SETPreSizeChoice

```
// SETPreSizeChoice()
// ==================
// Returns the size of the set that is performed by the SETP* or SETGP* instructions
// for this implementation, given the parameters of the destination and size of the set.

impdef func SETPreSizeChoice(memset : SETParams) => integer
begin
    return ARBITRARY : integer;
end;
```

Library pseudocode for aarch64/functions/mops/SETSizeChoice

```
// SETSizeChoice()
// ===============
// Returns the size of the block this performed for an iteration of the set given
// the parameters of the destination and size of the set. The size of the block
// is an integer multiple of alignsize.

impdef func SETSizeChoice(memset : SETParams, alignsize : integer) => MOPSBlockSize
begin
    return ARBITRARY : MOPSBlockSize;
end;
```

Library pseudocode for aarch64/functions/mops/UpdateCpyRegisters

```
// UpdateCpyRegisters()
// ====================
// Performs updates to the X[n], X[d], and X[s] registers, as appropriate, for the CPY* and CPYF*
// instructions. When fault is TRUE, the values correspond to the first element not copied,
// such that a return to the instruction will enable a resumption of the copy.

func UpdateCpyRegisters(memcpy : CPYParams, fault : boolean, copied : integer)
begin
    if fault then
        if memcpy.stage == MOPSStage_Prologue then
            // Undo any formatting of the input parameters performed in the prologue.
            if memcpy.implements_option_a then
                if memcpy.forward then
                    // cpysize is negative.
                    let cpysize : integer  = memcpy.cpysize + copied;
                    X{64}(memcpy.n)  = (0 - cpysize)[63:0];
                    X{64}(memcpy.d)  = memcpy.toaddress   + cpysize;
                    X{64}(memcpy.s)  = memcpy.fromaddress + cpysize;

                else
                    X{64}(memcpy.n) = (memcpy.cpysize - copied)[63:0];
                end;

            else
                if memcpy.forward then
                    X{64}(memcpy.n) = (memcpy.cpysize - copied)[63:0];
                    X{64}(memcpy.d) = memcpy.toaddress   + copied;
                    X{64}(memcpy.s) = memcpy.fromaddress + copied;

                else
                    X{64}(memcpy.n)  = (memcpy.cpysize - copied)[63:0];
                end;
            end;

        else
            if memcpy.implements_option_a then
                if memcpy.forward then
                    X{64}(memcpy.n) = (memcpy.cpysize + copied)[63:0];
                else
                    X{64}(memcpy.n) = (memcpy.cpysize - copied)[63:0];
                end;

            else
                X{64}(memcpy.n) = (memcpy.cpysize - copied)[63:0];

                if memcpy.forward then
                    X{64}(memcpy.d) = memcpy.toaddress   + copied;
                    X{64}(memcpy.s) = memcpy.fromaddress + copied;
                else
                    X{64}(memcpy.d) = memcpy.toaddress   - copied;
                    X{64}(memcpy.s) = memcpy.fromaddress - copied;
                end;
            end;
        end;
    else
        X{64}(memcpy.n) = memcpy.cpysize[63:0];
        if memcpy.stage == MOPSStage_Prologue || !memcpy.implements_option_a then
            X{64}(memcpy.d) = memcpy.toaddress;
            X{64}(memcpy.s) = memcpy.fromaddress;
        end;
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/mops/UpdateSetRegisters

```
// UpdateSetRegisters()
// ====================
// Performs updates to the X[n] and X[d] registers, as appropriate, for the SET* and SETG*
// instructions. When fault is TRUE, the values correspond to the first element not set, such
// that a return to the instruction will enable a resumption of the memory set.

func UpdateSetRegisters(memset : SETParams, fault : boolean, memory_set : integer)
begin
    if fault then
        // Undo any formatting of the input parameters performed in the prologue.
        if memset.stage == MOPSStage_Prologue then
            if memset.implements_option_a then
                // setsize is negative.
                let setsize : integer = memset.setsize   + memory_set;
                X{64}(memset.n) = (0 - setsize)[63:0];
                X{64}(memset.d) = memset.toaddress + setsize;
            else
                X{64}(memset.n) = (memset.setsize - memory_set)[63:0];
                X{64}(memset.d) = memset.toaddress + memory_set;
            end;

        else
            if memset.implements_option_a then
                X{64}(memset.n) = (memset.setsize + memory_set)[63:0];
            else
                X{64}(memset.n) = (memset.setsize - memory_set)[63:0];
                X{64}(memset.d) = memset.toaddress + memory_set;
            end;
        end;
    else
        X{64}(memset.n) = memset.setsize[63:0];
        if memset.stage == MOPSStage_Prologue || !memset.implements_option_a then
            X{64}(memset.d) = memset.toaddress;
        end;
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/movewideop/MoveWideOp

```
// MoveWideOp
// ==========
// Move wide 16-bit immediate instruction types.

type MoveWideOp of enumeration {MoveWideOp_N, MoveWideOp_Z, MoveWideOp_K};
```

Library pseudocode for aarch64/functions/movwpreferred/MoveWidePreferred

```
// MoveWidePreferred()
// ===================
//
// Return TRUE if a bitmask immediate encoding would generate an immediate
// value that could also be represented by a single MOVZ or MOVN instruction.
// Used as a condition for the preferred MOV<-ORR alias.

func MoveWidePreferred(sf : bit, immN : bit, imms : bits(6), immr : bits(6)) => boolean
begin
    let s : integer = UInt(imms);
    let r : integer = UInt(immr);
    let width : integer = if sf == '1' then 64 else 32;

    // element size must equal total immediate size
    if sf == '1' && (immN::imms) != '1xxxxxx' then
        return FALSE;
    end;
    if sf == '0' && (immN::imms) != '00xxxxx' then
        return FALSE;
    end;

    // for MOVZ must contain no more than 16 ones
    if s < 16 then
        // ones must not span halfword boundary when rotated
        return (-r MOD 16) <= (15 - s);
    end;

    // for MOVN must contain no more than 16 zeros
    if s >= width - 15 then
        // zeros must not span halfword boundary when rotated
        return (r MOD 16) <= (s - (width - 15));
    end;

    return FALSE;
end;
```

Library pseudocode for aarch64/functions/pac/addpac/AddPAC

```
// AddPAC()
// ========
// Calculates the pointer authentication code for a 64-bit quantity and then
// inserts that into pointer authentication code field of that 64-bit quantity.

func AddPAC(ptr : bits(64), modifier : bits(64), K : bits(128), data : boolean) => bits(64)
begin
    let use_modifier2 : boolean = FALSE;
    return InsertPAC(ptr, modifier, Zeros{64}, use_modifier2, K, data);
end;
```

Library pseudocode for aarch64/functions/pac/addpac/AddPAC2

```
// AddPAC2()
// =========
// Calculates the pointer authentication code for a 64-bit quantity and then
// inserts that into pointer authentication code field of that 64-bit quantity.

func AddPAC2(ptr : bits(64), modifier1 : bits(64), modifier2 : bits(64),
             K : bits(128), data : boolean) => bits(64)
begin
    let use_modifier2 : boolean = TRUE;
    return InsertPAC(ptr, modifier1, modifier2, use_modifier2, K, data);
end;
```

Library pseudocode for aarch64/functions/pac/addpac/InsertPAC

```
// InsertPAC()
// ===========
// Calculates the pointer authentication code for a 64-bit quantity and then
// inserts that into pointer authentication code field of that 64-bit quantity.

func InsertPAC(ptr : bits(64), modifier : bits(64), modifier2 : bits(64), use_modifier2 : boolean,
               K : bits(128), data : boolean) => bits(64)
begin
    var PAC : bits(64);
    var result : bits(64);
    var ext_ptr : bits(64);
    var extfield : bits(64);
    var selbit : bit;
    let tbi : boolean = EffectiveTBI(ptr, !data, PSTATE.EL) == '1';
    let mtx : boolean = (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) &&
                         EffectiveMTX(ptr, !data, PSTATE.EL) == '1');
    let top_bit : integer{} = if tbi then 55 else 63;
    let EL3_using_lva3 : boolean = (IsFeatureImplemented(FEAT_LVA3) &&
                                    TranslationRegime(PSTATE.EL) == Regime_EL3 &&
                                    AArch64_IASize(TCR_EL3().T0SZ) > 52);
    let is_VA_56bit : boolean = (TranslationRegime(PSTATE.EL) == Regime_EL3 &&
                                 AArch64_IASize(TCR_EL3().T0SZ) == 56);

    // If tagged pointers are in use for a regime with two TTBRs, use bit[55] of
    // the pointer to select between upper and lower ranges, and preserve this.
    // This handles the awkward case where there is apparently no correct choice between
    // the upper and lower address range - ie an addr of 1xxxxxxx0... with TBI0=0 and TBI1=1
    // and 0xxxxxxx1 with TBI1=0 and TBI0=1:
    if PtrHasUpperAndLowerAddRanges() then
        assert S1TranslationRegime() IN {EL1, EL2};
        if S1TranslationRegime() == EL1 then
            // EL1 translation regime registers
            if data then
                if TCR_EL1().TBI1 == '1' || TCR_EL1().TBI0 == '1' then
                    selbit = ptr[55];
                else
                    selbit = ptr[63];
                end;
            else
                if ((TCR_EL1().TBI1 == '1' && TCR_EL1().TBID1 == '0') ||
                    (TCR_EL1().TBI0 == '1' && TCR_EL1().TBID0 == '0')) then
                    selbit = ptr[55];
                else
                    selbit = ptr[63];
                end;
            end;
        else
            // EL2 translation regime registers
            if data then
                if TCR_EL2().TBI1 == '1' || TCR_EL2().TBI0 == '1' then
                    selbit = ptr[55];
                else
                    selbit = ptr[63];
                end;
            else
                if ((TCR_EL2().TBI1 == '1' && TCR_EL2().TBID1 == '0') ||
                    (TCR_EL2().TBI0 == '1' && TCR_EL2().TBID0 == '0')) then
                    selbit = ptr[55];
                else
                    selbit = ptr[63];
                end;
            end;
        end;
    else
        selbit = if tbi then ptr[55] else ptr[63];
    end;

    if IsFeatureImplemented(FEAT_PAuth2) && IsFeatureImplemented(FEAT_CONSTPACFIELD) then
        selbit = ptr[55];
    end;
    let bottom_PAC_bit : AddressSize = CalculateBottomPACBit(selbit);

    if EL3_using_lva3 then
        extfield = Replicate{64}('0');
    else
        extfield = Replicate{64}(selbit);
    end;

    // Compute the pointer authentication code for a ptr with good extension bits
    if tbi then
        if bottom_PAC_bit <= 55 then
            ext_ptr = (ptr[63:56] ::
                   extfield[55:bottom_PAC_bit] :: ptr[bottom_PAC_bit-1:0]);
        else
            ext_ptr = ptr[63:56] :: ptr[55:0];
        end;
    elsif mtx then
        if bottom_PAC_bit <= 55 then
            ext_ptr = (extfield[63:60] :: ptr[59:56] ::
```

```
                      extfield[55:bottom_PAC_bit] :: ptr[bottom_PAC_bit-1:0]);
        else
            ext_ptr = extfield[63:60] :: ptr[59:56] :: ptr[55:0];
        end;
    else
        ext_ptr =  extfield[63:bottom_PAC_bit] :: ptr[bottom_PAC_bit-1:0];
    end;

    if use_modifier2 then
        assert IsFeatureImplemented(FEAT_PAuth_LR);
        PAC = ComputePAC2(ext_ptr, modifier, modifier2, K[127:64], K[63:0]);
    else
        PAC = ComputePAC(ext_ptr, modifier, K[127:64], K[63:0]);
    end;

    if !IsFeatureImplemented(FEAT_PAuth2) then
        // If FEAT_PAuth2 is not implemented, the PAC is corrupted if the pointer does not have
        // a canonical VA.
        assert bottom_PAC_bit <= 52;
        if !IsZero(ptr[top_bit:bottom_PAC_bit]) && !IsOnes(ptr[top_bit:bottom_PAC_bit]) then
            PAC[top_bit-1] = NOT(PAC[top_bit-1]);
        end;
    end;

    // Preserve the determination between upper and lower address at bit[55] and insert PAC into
    // bits that are not used for the address or the tag(s).
    if !IsFeatureImplemented(FEAT_PAuth2) then
        assert (bottom_PAC_bit <= 52);
        if tbi then
            result = ptr[63:56]::selbit::PAC[54:bottom_PAC_bit]::ptr[bottom_PAC_bit-1:0];
        else
            result = PAC[63:56]::selbit::PAC[54:bottom_PAC_bit]::ptr[bottom_PAC_bit-1:0];
        end;
    else
        var bit55 : bit;
        if EL3_using_lva3 then
            // Bit 55 is an address bit (when VA size is 56-bits) or
            // used to store PAC (when VA size is less than 56-bits)
            if is_VA_56bit then
                bit55 = ptr[55];
            else
                bit55 = ptr[55] XOR PAC[55];
            end;
        else
            bit55 = selbit;
        end;
        if tbi then
            if bottom_PAC_bit < 55 then
                result = (ptr[63:56]                                :: bit55 ::
                          (ptr[54:bottom_PAC_bit] XOR PAC[54:bottom_PAC_bit]) ::
                          ptr[bottom_PAC_bit-1:0]);
            else
                result = (ptr[63:56] :: bit55 :: ptr[54:0]);
            end;
        elsif mtx then
            if bottom_PAC_bit < 55 then
                result = ((ptr[63:60] XOR PAC[63:60]) :: ptr[59:56] :: bit55 ::
                          (ptr[54:bottom_PAC_bit] XOR PAC[54:bottom_PAC_bit]) ::
                          ptr[bottom_PAC_bit-1:0]);
            else
                result = ((ptr[63:60] XOR PAC[63:60]) :: ptr[59:56] :: bit55 ::
                           ptr[54:0]);
            end;
        else
            if bottom_PAC_bit < 55 then
                result = ((ptr[63:56] XOR PAC[63:56])               :: bit55 ::
                          (ptr[54:bottom_PAC_bit] XOR PAC[54:bottom_PAC_bit]) ::
                          ptr[bottom_PAC_bit-1:0]);
            else
                result = ((ptr[63:56] XOR PAC[63:56])               :: bit55 ::
                           ptr[54:0]);
            end;
        end;
    end;
    return result;
end;
```

Library pseudocode for aarch64/functions/pac/addpacda/AddPACDA

```
// AddPACDA()
// ==========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with a pointer authentication code, where the pointer authentication
// code is derived using a cryptographic algorithm as a combination of x, y and the
// APDAKey_EL1.

func AddPACDA(x : bits(64), y : bits(64)) => bits(64)
begin
    let APDAKey_EL1 : bits(128) = APDAKeyHi_EL1()[63:0] :: APDAKeyLo_EL1()[63:0];
    if !IsAPDAKeyEnabled() then
        return x;
    else
        return AddPAC(x, y, APDAKey_EL1, TRUE);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/addpacdb/AddPACDB

```
// AddPACDB()
// ==========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with a pointer authentication code, where the pointer authentication
// code is derived using a cryptographic algorithm as a combination of x, y and the
// APDBKey_EL1.

func AddPACDB(x : bits(64), y : bits(64)) => bits(64)
begin
    let APDBKey_EL1 : bits(128) = APDBKeyHi_EL1()[63:0] :: APDBKeyLo_EL1()[63:0];
    if !IsAPDBKeyEnabled() then
        return x;
    else
        return AddPAC(x, y, APDBKey_EL1, TRUE);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/addpacga/AddPACGA

```
// AddPACGA()
// ==========
// Returns a 64-bit value where the lower 32 bits are 0, and the upper 32 bits contain
// a 32-bit pointer authentication code which is derived using a cryptographic
// algorithm as a combination of x, y and the APGAKey_EL1.

func AddPACGA(x : bits(64), y : bits(64)) => bits(64)
begin
    var TrapEL2 : boolean;
    let APGAKey_EL1 : bits(128) = APGAKeyHi_EL1()[63:0] :: APGAKeyLo_EL1()[63:0];

    var TrapEL3 : boolean;
    case PSTATE.EL of
        when EL0 =>
            TrapEL2 = EL2Enabled() && HCR_EL2().API == '0' && !IsInHost();
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL1 =>
            TrapEL2 = EL2Enabled() && HCR_EL2().API == '0';
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL2 =>
            TrapEL2 = FALSE;
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL3 =>
            TrapEL2 = FALSE;
            TrapEL3 = FALSE;
    end;

    if TrapEL3 && EL3SDDUndefPriority() then
        Undefined();
    elsif TrapEL2 then
        TrapPACUse(EL2);
    elsif TrapEL3 then
        if EL3SDDUndef() then
            Undefined();
        else
            TrapPACUse(EL3);
        end;
    else
        return ComputePAC(x, y, APGAKey_EL1[127:64], APGAKey_EL1[63:0])[63:32]::Zeros{32};
    end;
end;
```

Library pseudocode for aarch64/functions/pac/addpacia/AddPACIA

```
// AddPACIA()
// ==========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with a pointer authentication code, where the pointer authentication
// code is derived using a cryptographic algorithm as a combination of x, y, and the
// APIAKey_EL1.

func AddPACIA(x : bits(64), y : bits(64)) => bits(64)
begin
    let APIAKey_EL1 : bits(128) = APIAKeyHi_EL1()[63:0]::APIAKeyLo_EL1()[63:0];
    if !IsAPIAKeyEnabled() then
        return x;
    else
        return AddPAC(x, y, APIAKey_EL1, FALSE);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/addpacia/AddPACIA2

```
// AddPACIA2()
// ===========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with a pointer authentication code, where the pointer authentication
// code is derived using a cryptographic algorithm as a combination of x, y, z, and
// the APIAKey_EL1.

func AddPACIA2(x : bits(64), y : bits(64), z : bits(64)) => bits(64)
begin
    let APIAKey_EL1 : bits(128) = APIAKeyHi_EL1()[63:0]::APIAKeyLo_EL1()[63:0];
    if !IsAPIAKeyEnabled() then
        return x;
    else
        return AddPAC2(x, y, z, APIAKey_EL1, FALSE);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/addpacib/AddPACIB

```
// AddPACIB()
// ==========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with a pointer authentication code, where the pointer authentication
// code is derived using a cryptographic algorithm as a combination of x, y and the
// APIBKey_EL1.

func AddPACIB(x : bits(64), y : bits(64)) => bits(64)
begin
    let APIBKey_EL1 : bits(128) = APIBKeyHi_EL1()[63:0] :: APIBKeyLo_EL1()[63:0];
    if !IsAPIBKeyEnabled() then
        return x;
    else
        return AddPAC(x, y, APIBKey_EL1, FALSE);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/addpacib/AddPACIB2

```
// AddPACIB2()
// ===========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with a pointer authentication code, where the pointer authentication
// code is derived using a cryptographic algorithm as a combination of x, y, z, and
// the APIBKey_EL1.

func AddPACIB2(x : bits(64), y : bits(64), z : bits(64)) => bits(64)
begin
    let APIBKey_EL1 : bits(128) = APIBKeyHi_EL1()[63:0] :: APIBKeyLo_EL1()[63:0];
    if !IsAPIBKeyEnabled() then
        return x;
    else
        return AddPAC2(x, y, z, APIBKey_EL1, FALSE);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/auth/AArch64_PACFailException

```
// AArch64_PACFailException()
// ==========================
// Generates a PAC Fail Exception

func AArch64_PACFailException(syndrome : bits(2))
begin
    let route_to_el2 : boolean = PSTATE.EL == EL0 && EL2Enabled() && HCR_EL2().TGE == '1';
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer = 0x0;

    var except : ExceptionRecord = ExceptionSyndrome(Exception_PACFail);
    except.syndrome.iss[1:0]     = syndrome;
    except.syndrome.iss[24:2]    = Zeros{23};               // RES0

    if UInt(PSTATE.EL) > UInt(EL0) then
        AArch64_TakeException(PSTATE.EL, except, preferred_exception_return, vect_offset);
    elsif route_to_el2 then
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        AArch64_TakeException(EL1, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/auth/Auth

```
// Auth()
// ======
// Restores the upper bits of the address to be all zeros or all ones (based on the
// value of bit[55]) and computes and checks the pointer authentication code. If the
// check passes, then the restored address is returned. If the check fails, the
// second-top and third-top bits of the extension bits in the pointer authentication code
// field are corrupted to ensure that accessing the address will give a translation fault.

func Auth(ptr : bits(64), modifier : bits(64), K : bits(128), data : boolean, key_number : bit,
          is_combined : boolean) => bits(64)
begin
    let use_modifier2 : boolean = FALSE;
    return Authenticate(ptr, modifier, Zeros{64}, use_modifier2,
                        K, data, key_number, is_combined);
end;
```

Library pseudocode for aarch64/functions/pac/auth/Auth2

```
// Auth2()
// =======
// Restores the upper bits of the address to be all zeros or all ones (based on the
// value of bit[55]) and computes and checks the pointer authentication code. If the
// check passes, then the restored address is returned. If the check fails, the
// second-top and third-top bits of the extension bits in the pointer authentication code
// field are corrupted to ensure that accessing the address will give a translation fault.

func Auth2(ptr : bits(64), modifier1 : bits(64), modifier2 : bits(64), K : bits(128),
           data : boolean, key_number : bit, is_combined : boolean) => bits(64)
begin
    let use_modifier2 : boolean = TRUE;
    return Authenticate(ptr, modifier1, modifier2, use_modifier2, K, data, key_number, is_combined);
end;
```

Library pseudocode for aarch64/functions/pac/auth/Authenticate

```
// Authenticate()
// ==============
// Restores the upper bits of the address to be all zeros or all ones (based on the
// value of bit[55]) and computes and checks the pointer authentication code. If the
// check passes, then the restored address is returned. If the check fails, the
// second-top and third-top bits of the extension bits in the pointer authentication code
// field are corrupted to ensure that accessing the address will give a translation fault.

func Authenticate(ptr : bits(64), modifier : bits(64), modifier2 : bits(64),
                  use_modifier2 : boolean, K : bits(128), data : boolean, key_number : bit,
                  is_combined : boolean) => bits(64)
begin
    var PAC : bits(64);
    var result : bits(64);
    var original_ptr : bits(64);
    var error_code : bits(2);
    var extfield : bits(64);

    // Reconstruct the extension field used of adding the PAC to the pointer
    let tbi : boolean = EffectiveTBI(ptr, !data, PSTATE.EL) == '1';
    let mtx : boolean = (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) &&
                         EffectiveMTX(ptr, !data, PSTATE.EL) == '1');
    let bottom_PAC_bit : AddressSize = CalculateBottomPACBit(ptr[55]);
    let EL3_using_lva3 : boolean = (IsFeatureImplemented(FEAT_LVA3) &&
                                    TranslationRegime(PSTATE.EL) == Regime_EL3 &&
                                    AArch64_IASize(TCR_EL3().T0SZ) > 52);
    let is_VA_56bit : boolean = (TranslationRegime(PSTATE.EL) == Regime_EL3 &&
                                 AArch64_IASize(TCR_EL3().T0SZ) == 56);
    if EL3_using_lva3 then
        extfield = Replicate{64}('0');
    else
        extfield = Replicate{64}(ptr[55]);
    end;

    if tbi then
        if bottom_PAC_bit <= 55 then
            original_ptr = (ptr[63:56] ::
                            extfield[55:bottom_PAC_bit] :: ptr[bottom_PAC_bit-1:0]);
        else
            original_ptr = ptr[63:56] :: ptr[55:0];
        end;
    elsif mtx then
        if bottom_PAC_bit <= 55 then
            original_ptr = (extfield[63:60] :: ptr[59:56] ::
                            extfield[55:bottom_PAC_bit] :: ptr[bottom_PAC_bit-1:0]);
        else
            original_ptr = extfield[63:60] :: ptr[59:56] :: ptr[55:0];
        end;
    else
        original_ptr =  extfield[63:bottom_PAC_bit] :: ptr[bottom_PAC_bit-1:0];
    end;

    if use_modifier2 then
        assert IsFeatureImplemented(FEAT_PAuth_LR);
        PAC = ComputePAC2(original_ptr, modifier, modifier2, K[127:64], K[63:0]);
    else
        PAC = ComputePAC(original_ptr, modifier, K[127:64], K[63:0]);
    end;
    // Check pointer authentication code
    if tbi then
        if !IsFeatureImplemented(FEAT_PAuth2) then
            assert (bottom_PAC_bit <= 52);
            if PAC[54:bottom_PAC_bit] == ptr[54:bottom_PAC_bit] then
                result = original_ptr;
            else
                error_code = key_number::NOT(key_number);
                result = original_ptr[63:55]::error_code::original_ptr[52:0];
            end;
        else
            result = ptr;
            if EL3_using_lva3 && !is_VA_56bit then
                result[55] = result[55] XOR PAC[55];
            end;
            if (bottom_PAC_bit < 55) then
                result[54:bottom_PAC_bit] = result[54:bottom_PAC_bit] XOR PAC[54:bottom_PAC_bit];
            end;
            if (IsFeatureImplemented(FEAT_FPACCOMBINE) ||
                  (IsFeatureImplemented(FEAT_FPAC) && !is_combined)) then
                if (EL3_using_lva3 && !is_VA_56bit && !IsZero(result[55:bottom_PAC_bit])) then
                    error_code = (if data then '1' else '0')::key_number;
                    AArch64_PACFailException(error_code);
                elsif (!EL3_using_lva3 && (bottom_PAC_bit < 55) &&
                         result[54:bottom_PAC_bit] !=
                         Replicate{55-bottom_PAC_bit}(result[55])) then
                    error_code = (if data then '1' else '0')::key_number;
                    AArch64_PACFailException(error_code);
                end;
            end;
        end;
```

```
    elsif mtx then
        assert IsFeatureImplemented(FEAT_PAuth2);
        result = ptr;
        if EL3_using_lva3 && !is_VA_56bit then
            result[55] = result[55] XOR PAC[55];
        end;
        if (bottom_PAC_bit < 55) then
            result[54:bottom_PAC_bit] = result[54:bottom_PAC_bit] XOR PAC[54:bottom_PAC_bit];
        end;
        result[63:60] = result[63:60] XOR PAC[63:60];
        if (IsFeatureImplemented(FEAT_FPACCOMBINE) ||
              (IsFeatureImplemented(FEAT_FPAC) && !is_combined)) then
            if (EL3_using_lva3 && !is_VA_56bit &&
                  (!IsZero(result[55:bottom_PAC_bit]) || !IsZero(result[63:60]))) then
                error_code = (if data then '1' else '0')::key_number;
                AArch64_PACFailException(error_code);
            elsif (!EL3_using_lva3 && (bottom_PAC_bit < 55) &&
                     (((result[54:bottom_PAC_bit] !=
                        Replicate{55-bottom_PAC_bit}(result[55])) ||
                       (result[63:60] != Replicate{4}(result[55])))) then
                error_code = (if data then '1' else '0')::key_number;
                AArch64_PACFailException(error_code);
            end;
        end;
    else
        if !IsFeatureImplemented(FEAT_PAuth2) then
            assert (bottom_PAC_bit <= 52);
            if PAC[54:bottom_PAC_bit] == ptr[54:bottom_PAC_bit] && PAC[63:56] == ptr[63:56] then
                result = original_ptr;
            else
                error_code = key_number::NOT(key_number);
                result     = original_ptr[63]::error_code::original_ptr[60:0];
            end;
        else
            result = ptr;
            if EL3_using_lva3 && !is_VA_56bit then
                result[55] = result[55] XOR PAC[55];
            end;
            if bottom_PAC_bit < 55 then
                result[54:bottom_PAC_bit] = result[54:bottom_PAC_bit] XOR PAC[54:bottom_PAC_bit];
            end;
            result[63:56] = result[63:56] XOR PAC[63:56];
            if (IsFeatureImplemented(FEAT_FPACCOMBINE) ||
                  (IsFeatureImplemented(FEAT_FPAC) && !is_combined)) then
                if (EL3_using_lva3 && !IsZero(result[63:bottom_PAC_bit])) then
                    error_code = (if data then '1' else '0')::key_number;
                    AArch64_PACFailException(error_code);
                elsif (!EL3_using_lva3 &&
                         result[63:bottom_PAC_bit] !=
                         Replicate{64-bottom_PAC_bit}(result[55])) then
                    error_code = (if data then '1' else '0')::key_number;
                    AArch64_PACFailException(error_code);
                end;
            end;
        end;
    end;
    return result;
end;
```

Library pseudocode for aarch64/functions/pac/authda/AuthDA

```
// AuthDA()
// ========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with the extension of the address bits. The instruction checks a pointer
// authentication code in the pointer authentication code field bits of x, using the same
// algorithm and key as AddPACDA().

func AuthDA(x : bits(64), y : bits(64), is_combined : boolean) => bits(64)
begin
    let APDAKey_EL1 : bits(128) = APDAKeyHi_EL1()[63:0] :: APDAKeyLo_EL1()[63:0];
    if !IsAPDAKeyEnabled() then
        return x;
    else
        return Auth(x, y, APDAKey_EL1, TRUE, '0', is_combined);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/authdb/AuthDB

```
// AuthDB()
// ========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with the extension of the address bits. The instruction checks a
// pointer authentication code in the pointer authentication code field bits of x, using
// the same algorithm and key as AddPACDB().

func AuthDB(x : bits(64), y : bits(64), is_combined : boolean) => bits(64)
begin
    let APDBKey_EL1 : bits(128) = APDBKeyHi_EL1()[63:0] :: APDBKeyLo_EL1()[63:0];
    if !IsAPDBKeyEnabled() then
        return x;
    else
        return Auth(x, y, APDBKey_EL1, TRUE, '1', is_combined);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/authia/AuthIA

```
// AuthIA()
// ========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with the extension of the address bits. The instruction checks a pointer
// authentication code in the pointer authentication code field bits of x, using the same
// algorithm and key as AddPACIA().

func AuthIA(x : bits(64), y : bits(64), is_combined : boolean) => bits(64)
begin
    let APIAKey_EL1 : bits(128) = APIAKeyHi_EL1()[63:0] :: APIAKeyLo_EL1()[63:0];
    if !IsAPIAKeyEnabled() then
        return x;
    else
        return Auth(x, y, APIAKey_EL1, FALSE, '0', is_combined);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/authia/AuthIA2

```
// AuthIA2()
// =========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with the extension of the address bits. The instruction checks a pointer
// authentication code in the pointer authentication code field bits of x, using the same
// algorithm and key as AddPACIA2().

func AuthIA2(x : bits(64), y : bits(64), z : bits(64), is_combined : boolean) => bits(64)
begin
    let APIAKey_EL1 : bits(128) = APIAKeyHi_EL1()[63:0] :: APIAKeyLo_EL1()[63:0];
    if !IsAPIAKeyEnabled() then
        return x;
    else
        return Auth2(x, y, z, APIAKey_EL1, FALSE, '0', is_combined);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/authib/AuthIB

```
// AuthIB()
// ========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with the extension of the address bits. The instruction checks a pointer
// authentication code in the pointer authentication code field bits of x, using the same
// algorithm and key as AddPACIB().

func AuthIB(x : bits(64), y : bits(64), is_combined : boolean) => bits(64)
begin
    let APIBKey_EL1 : bits(128) = APIBKeyHi_EL1()[63:0] :: APIBKeyLo_EL1()[63:0];
    if !IsAPIBKeyEnabled() then
        return x;
    else
        return Auth(x, y, APIBKey_EL1, FALSE, '1', is_combined);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/authib/AuthIB2

```
// AuthIB2()
// =========
// Returns a 64-bit value containing x, but replacing the pointer authentication code
// field bits with the extension of the address bits. The instruction checks a pointer
// authentication code in the pointer authentication code field bits of x, using the same
// algorithm and key as AddPACIB2().

func AuthIB2(x : bits(64), y : bits(64), z : bits(64), is_combined : boolean) => bits(64)
begin
    let APIBKey_EL1 : bits(128) = APIBKeyHi_EL1()[63:0] :: APIBKeyLo_EL1()[63:0];
    if !IsAPIBKeyEnabled() then
        return x;
    else
        return Auth2(x, y, z, APIBKey_EL1, FALSE, '1', is_combined);
    end;
end;
```

Library pseudocode for aarch64/functions/pac/calcbottompacbit/AArch64_PACEffectiveTxSZ

```
// AArch64_PACEffectiveTxSZ()
// ==========================
// Compute the effective value for TxSZ used to determine the placement of the PAC field

readonly func AArch64_PACEffectiveTxSZ(regime : Regime, walkparams : S1TTWParams) => bits(6)
begin
    let s1maxtxsz : integer = AArch64_MaxTxSZ(walkparams.tgx);
    let s1mintxsz : integer = AArch64_S1MinTxSZ(regime, walkparams);

    if AArch64_S1TxSZFaults(regime, walkparams) then
        if ConstrainUnpredictable(Unpredictable_RESTnSZ) == Constraint_FORCE then
            if UInt(walkparams.txsz) < s1mintxsz then
                return s1mintxsz[5:0];
            end;
            if UInt(walkparams.txsz) > s1maxtxsz then
                return s1maxtxsz[5:0];
            end;
        end;
    elsif UInt(walkparams.txsz) < s1mintxsz then
        return s1mintxsz[5:0];
    elsif UInt(walkparams.txsz) > s1maxtxsz then
        return s1maxtxsz[5:0];
    end;

    return walkparams.txsz;
end;
```

Library pseudocode for aarch64/functions/pac/calcbottompacbit/CalculateBottomPACBit

```
// CalculateBottomPACBit()
// =======================

readonly func CalculateBottomPACBit(top_bit : bit) => AddressSize
begin
    var regime : Regime;
    var walkparams : S1TTWParams;
    var bottom_PAC_bit : AddressSize;

    regime = TranslationRegime(PSTATE.EL);
    let ss : SecurityState = CurrentSecurityState();
    walkparams             = AArch64_GetS1TTWParams(regime, PSTATE.EL, ss, Replicate{64}(top_bit));
    bottom_PAC_bit         = ((64 - UInt(AArch64_PACEffectiveTxSZ(regime, walkparams)))
                              as AddressSize);

    return bottom_PAC_bit;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/ComputePAC

```
// ComputePAC()
// ============

func ComputePAC(data : bits(64), modifier : bits(64), key0 : bits(64), key1 : bits(64)) => bits(64)
begin
    if IsFeatureImplemented(FEAT_PACIMP) then
        return ComputePACIMPDEF(data, modifier, key0, key1);
    end;
    if IsFeatureImplemented(FEAT_PACQARMA3) then
        let isqarma3 : boolean = TRUE;
        return ComputePACQARMA(data, modifier, key0, key1, isqarma3);
    end;
    if IsFeatureImplemented(FEAT_PACQARMA5) then
        let isqarma3 : boolean = FALSE;
        return ComputePACQARMA(data, modifier, key0, key1, isqarma3);
    end;
    unreachable;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/ComputePAC2

```
// ComputePAC2()
// =============

func ComputePAC2(data : bits(64), modifier1 : bits(64), modifier2 : bits(64),
                 key0 : bits(64), key1 : bits(64)) => bits(64)
begin
    if IsFeatureImplemented(FEAT_PACIMP) then
        return ComputePAC2IMPDEF(data, modifier1, modifier2, key0, key1);
    end;
    if IsFeatureImplemented(FEAT_PACQARMA3) then
        let isqarma3 : boolean = TRUE;
        return ComputePAC2QARMA(data, modifier1, modifier2, key0, key1, isqarma3);
    end;
    if IsFeatureImplemented(FEAT_PACQARMA5) then
        let isqarma3 : boolean = FALSE;
        return ComputePAC2QARMA(data, modifier1, modifier2, key0, key1, isqarma3);
    end;
    unreachable;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/ComputePAC2IMPDEF

```
// ComputePAC2IMPDEF()
// ===================
// Compute IMPLEMENTATION DEFINED cryptographic algorithm to be used for PAC calculation.

impdef func ComputePAC2IMPDEF(data : bits(64),
                              modifier1 : bits(64),
                              modifier2 : bits(64),
                              key0 : bits(64),
                              key1 : bits(64)) => bits(64)
begin
    Unimplemented();
end;
```

Library pseudocode for aarch64/functions/pac/computepac/ComputePAC2QARMA

```
// ComputePAC2QARMA()
// ==================

func ComputePAC2QARMA(data : bits(64),
                      modifier1 : bits(64),
                      modifier2 : bits(64),
                      key0 : bits(64),
                      key1 : bits(64),
                      isqarma3 : boolean) => bits(64)
begin
    let concat_modifiers : bits(64) = modifier2[36:5]::modifier1[35:4];
    return ComputePACQARMA(data, concat_modifiers, key0, key1, isqarma3);
end;
```

Library pseudocode for aarch64/functions/pac/computepac/ComputePACIMPDEF

```
// ComputePACIMPDEF()
// ==================
// Compute IMPLEMENTATION DEFINED cryptographic algorithm to be used for PAC calculation.

impdef func ComputePACIMPDEF(data : bits(64), modifier : bits(64),
                             key0 : bits(64), key1 : bits(64)) => bits(64)
begin
    Unimplemented();
end;
```

Library pseudocode for aarch64/functions/pac/computepac/ComputePACQARMA

```
// ComputePACQARMA()
// =================
// Compute QARMA3 or QARMA5 cryptographic algorithm for PAC calculation

func ComputePACQARMA(data : bits(64),
                     modifier : bits(64),
                     key0 : bits(64),
                     key1 : bits(64),
                     isqarma3 : boolean) => bits(64)
begin
    var workingval : bits(64);
    var runningmod : bits(64);
    var roundkey : bits(64);
    var modk0 : bits(64);
    let Alpha : bits(64) = 0xC0AC29B7C97C50DD[63:0];

    var iterations : integer;
    RC[[0]] = 0x0000000000000000[63:0];
    RC[[1]] = 0x13198A2E03707344[63:0];
    RC[[2]] = 0xA4093822299F31D0[63:0];

    if isqarma3 then
        iterations = 2;
    else // QARMA5
        iterations = 4;
        RC[[3]] = 0x082EFA98EC4E6C89[63:0];
        RC[[4]] = 0x452821E638D01377[63:0];
    end;

    modk0 = key0[0]::key0[63:2]::(key0[63] XOR key0[1]);
    runningmod = modifier;
    workingval = data XOR key0;

    for i = 0 to iterations do
        roundkey   = key1 XOR runningmod;
        workingval = workingval XOR roundkey;
        workingval = workingval XOR RC[[i]];
        if i > 0 then
            workingval = PACCellShuffle(workingval);
            workingval = PACMult(workingval);
        end;
        if isqarma3 then
            workingval = PACSub1(workingval);
        else
            workingval = PACSub(workingval);
        end;
        runningmod = TweakShuffle(runningmod[63:0]);
    end;
    roundkey   = modk0 XOR runningmod;
    workingval = workingval XOR roundkey;
    workingval = PACCellShuffle(workingval);
    workingval = PACMult(workingval);
    if isqarma3 then
        workingval = PACSub1(workingval);
    else
        workingval = PACSub(workingval);
    end;
    workingval = PACCellShuffle(workingval);
    workingval = PACMult(workingval);
    workingval = key1 XOR workingval;
    workingval = PACCellInvShuffle(workingval);
    if isqarma3 then
        workingval = PACSub1(workingval);
    else
        workingval = PACInvSub(workingval);
    end;
    workingval = PACMult(workingval);
    workingval = PACCellInvShuffle(workingval);
    workingval = workingval XOR key0;
    workingval = workingval XOR runningmod;
    for i = 0 to iterations do
        if isqarma3 then
            workingval = PACSub1(workingval);
        else
            workingval = PACInvSub(workingval);
        end;
        if i < iterations then
            workingval = PACMult(workingval);
            workingval = PACCellInvShuffle(workingval);
        end;
        runningmod = TweakInvShuffle(runningmod[63:0]);
        roundkey = key1 XOR runningmod;
        workingval = workingval XOR RC[[iterations-i]];
        workingval = workingval XOR roundkey;
        workingval = workingval XOR Alpha;
    end;
    workingval = workingval XOR modk0;
```

```
    return workingval;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/PACCellInvShuffle

```
// PACCellInvShuffle()
// ===================

func PACCellInvShuffle(indata : bits(64)) => bits(64)
begin
    var outdata : bits(64);
    outdata[3:0]   = indata[15:12];
    outdata[7:4]   = indata[27:24];
    outdata[11:8]  = indata[51:48];
    outdata[15:12] = indata[39:36];
    outdata[19:16] = indata[59:56];
    outdata[23:20] = indata[47:44];
    outdata[27:24] = indata[7:4];
    outdata[31:28] = indata[19:16];
    outdata[35:32] = indata[35:32];
    outdata[39:36] = indata[55:52];
    outdata[43:40] = indata[31:28];
    outdata[47:44] = indata[11:8];
    outdata[51:48] = indata[23:20];
    outdata[55:52] = indata[3:0];
    outdata[59:56] = indata[43:40];
    outdata[63:60] = indata[63:60];
    return outdata;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/PACCellShuffle

```
// PACCellShuffle()
// ================

func PACCellShuffle(indata : bits(64)) => bits(64)
begin
    var outdata : bits(64);
    outdata[3:0]   = indata[55:52];
    outdata[7:4]   = indata[27:24];
    outdata[11:8]  = indata[47:44];
    outdata[15:12] = indata[3:0];
    outdata[19:16] = indata[31:28];
    outdata[23:20] = indata[51:48];
    outdata[27:24] = indata[7:4];
    outdata[31:28] = indata[43:40];
    outdata[35:32] = indata[35:32];
    outdata[39:36] = indata[15:12];
    outdata[43:40] = indata[59:56];
    outdata[47:44] = indata[23:20];
    outdata[51:48] = indata[11:8];
    outdata[55:52] = indata[39:36];
    outdata[59:56] = indata[19:16];
    outdata[63:60] = indata[63:60];
    return outdata;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/PACInvSub

```
// PACInvSub()
// ===========

func PACInvSub(Tinput : bits(64)) => bits(64)
begin
    // This is a 4-bit substitution from the PRINCE-family cipher
    var Toutput : bits(64);
    for i = 0 to 15 do
        case Tinput[i*:4] of
            when '0000' =>  Toutput[i*:4] = '0101';
            when '0001' =>  Toutput[i*:4] = '1110';
            when '0010' =>  Toutput[i*:4] = '1101';
            when '0011' =>  Toutput[i*:4] = '1000';
            when '0100' =>  Toutput[i*:4] = '1010';
            when '0101' =>  Toutput[i*:4] = '1011';
            when '0110' =>  Toutput[i*:4] = '0001';
            when '0111' =>  Toutput[i*:4] = '1001';
            when '1000' =>  Toutput[i*:4] = '0010';
            when '1001' =>  Toutput[i*:4] = '0110';
            when '1010' =>  Toutput[i*:4] = '1111';
            when '1011' =>  Toutput[i*:4] = '0000';
            when '1100' =>  Toutput[i*:4] = '0100';
            when '1101' =>  Toutput[i*:4] = '1100';
            when '1110' =>  Toutput[i*:4] = '0111';
            when '1111' =>  Toutput[i*:4] = '0011';
        end;
    end;
    return Toutput;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/PACMult

```
// PACMult()
// =========

func PACMult(Sinput : bits(64)) => bits(64)
begin
    var t0 : bits(4);
    var t1 : bits(4);
    var t2 : bits(4);
    var t3 : bits(4);
    var Soutput : bits(64);

    for i = 0 to 3 do
        t0[3:0] = ROL(Sinput[((i+8))*:4], 1) XOR ROL(Sinput[((i+4))*:4], 2);
        t0[3:0] = t0[3:0] XOR ROL(Sinput[i*:4], 1);
        t1[3:0] = ROL(Sinput[((i+12))*:4], 1) XOR ROL(Sinput[((i+4))*:4], 1);
        t1[3:0] = t1[3:0] XOR ROL(Sinput[i*:4], 2);
        t2[3:0] = ROL(Sinput[((i+12))*:4], 2) XOR ROL(Sinput[((i+8))*:4], 1);
        t2[3:0] = t2[3:0] XOR ROL(Sinput[i*:4], 1);
        t3[3:0] = ROL(Sinput[((i+12))*:4], 1) XOR ROL(Sinput[((i+8))*:4], 2);
        t3[3:0] = t3[3:0] XOR ROL(Sinput[((i+4))*:4], 1);
        Soutput[i*:4] = t3[3:0];
        Soutput[((i+4))*:4] = t2[3:0];
        Soutput[((i+8))*:4] = t1[3:0];
        Soutput[((i+12))*:4] = t0[3:0];
    end;
    return Soutput;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/PACSub

```
// PACSub()
// ========

func PACSub(Tinput : bits(64)) => bits(64)
begin
    // This is a 4-bit substitution from the PRINCE-family cipher
    var Toutput : bits(64);
    for i = 0 to 15 do
        case Tinput[i*:4] of
            when '0000' =>  Toutput[i*:4] = '1011';
            when '0001' =>  Toutput[i*:4] = '0110';
            when '0010' =>  Toutput[i*:4] = '1000';
            when '0011' =>  Toutput[i*:4] = '1111';
            when '0100' =>  Toutput[i*:4] = '1100';
            when '0101' =>  Toutput[i*:4] = '0000';
            when '0110' =>  Toutput[i*:4] = '1001';
            when '0111' =>  Toutput[i*:4] = '1110';
            when '1000' =>  Toutput[i*:4] = '0011';
            when '1001' =>  Toutput[i*:4] = '0111';
            when '1010' =>  Toutput[i*:4] = '0100';
            when '1011' =>  Toutput[i*:4] = '0101';
            when '1100' =>  Toutput[i*:4] = '1101';
            when '1101' =>  Toutput[i*:4] = '0010';
            when '1110' =>  Toutput[i*:4] = '0001';
            when '1111' =>  Toutput[i*:4] = '1010';
        end;
    end;
    return Toutput;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/PacSub1

```
// PacSub1()
// =========

func PACSub1(Tinput : bits(64)) => bits(64)
begin
    // This is a 4-bit substitution from Qarma sigma1
    var Toutput : bits(64);
    for i = 0 to 15 do
        case Tinput[i*:4] of
            when '0000' => Toutput[i*:4] = '1010';
            when '0001' => Toutput[i*:4] = '1101';
            when '0010' => Toutput[i*:4] = '1110';
            when '0011' => Toutput[i*:4] = '0110';
            when '0100' => Toutput[i*:4] = '1111';
            when '0101' => Toutput[i*:4] = '0111';
            when '0110' => Toutput[i*:4] = '0011';
            when '0111' => Toutput[i*:4] = '0101';
            when '1000' => Toutput[i*:4] = '1001';
            when '1001' => Toutput[i*:4] = '1000';
            when '1010' => Toutput[i*:4] = '0000';
            when '1011' => Toutput[i*:4] = '1100';
            when '1100' => Toutput[i*:4] = '1011';
            when '1101' => Toutput[i*:4] = '0001';
            when '1110' => Toutput[i*:4] = '0010';
            when '1111' => Toutput[i*:4] = '0100';
        end;
    end;
    return Toutput;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/RC

```
// RC[]
// ====

var RC : array [[5]] of bits(64);
```

Library pseudocode for aarch64/functions/pac/computepac/TweakCellInvRot

```
// TweakCellInvRot()
// =================

func TweakCellInvRot(incell : bits(4)) => bits(4)
begin
    var outcell : bits(4);
    outcell[3] = incell[2];
    outcell[2] = incell[1];
    outcell[1] = incell[0];
    outcell[0] = incell[0] XOR incell[3];
    return outcell;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/TweakCellRot

```
// TweakCellRot()
// ==============

func TweakCellRot(incell : bits(4)) => bits(4)
begin
    var outcell : bits(4);
    outcell[3] = incell[0] XOR incell[1];
    outcell[2] = incell[3];
    outcell[1] = incell[2];
    outcell[0] = incell[1];
    return outcell;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/TweakInvShuffle

```
// TweakInvShuffle()
// =================

func TweakInvShuffle(indata : bits(64)) => bits(64)
begin
    var outdata : bits(64);
    outdata[3:0]   = TweakCellInvRot(indata[51:48]);
    outdata[7:4]   = indata[55:52];
    outdata[11:8]  = indata[23:20];
    outdata[15:12] = indata[27:24];
    outdata[19:16] = indata[3:0];
    outdata[23:20] = indata[7:4];
    outdata[27:24] = TweakCellInvRot(indata[11:8]);
    outdata[31:28] = indata[15:12];
    outdata[35:32] = TweakCellInvRot(indata[31:28]);
    outdata[39:36] = TweakCellInvRot(indata[63:60]);
    outdata[43:40] = TweakCellInvRot(indata[59:56]);
    outdata[47:44] = TweakCellInvRot(indata[19:16]);
    outdata[51:48] = indata[35:32];
    outdata[55:52] = indata[39:36];
    outdata[59:56] = indata[43:40];
    outdata[63:60] = TweakCellInvRot(indata[47:44]);
    return outdata;
end;
```

Library pseudocode for aarch64/functions/pac/computepac/TweakShuffle

```
// TweakShuffle()
// ==============

func TweakShuffle(indata : bits(64)) => bits(64)
begin
    var outdata : bits(64);
    outdata[3:0]   = indata[19:16];
    outdata[7:4]   = indata[23:20];
    outdata[11:8]  = TweakCellRot(indata[27:24]);
    outdata[15:12] = indata[31:28];
    outdata[19:16] = TweakCellRot(indata[47:44]);
    outdata[23:20] = indata[11:8];
    outdata[27:24] = indata[15:12];
    outdata[31:28] = TweakCellRot(indata[35:32]);
    outdata[35:32] = indata[51:48];
    outdata[39:36] = indata[55:52];
    outdata[43:40] = indata[59:56];
    outdata[47:44] = TweakCellRot(indata[63:60]);
    outdata[51:48] = TweakCellRot(indata[3:0]);
    outdata[55:52] = indata[7:4];
    outdata[59:56] = TweakCellRot(indata[43:40]);
    outdata[63:60] = TweakCellRot(indata[39:36]);
    return outdata;
end;
```

Library pseudocode for aarch64/functions/pac/pac/IsAPDAKeyEnabled

```
// IsAPDAKeyEnabled()
// ==================
// Returns TRUE if authentication using the APDAKey_EL1 key is enabled.
// Otherwise, depending on the state of the PE, generate a trap, or return FALSE.

func IsAPDAKeyEnabled() => boolean
begin
    var TrapEL2 : boolean;
    var TrapEL3 : boolean;
    var Enable : bits(1);

    case PSTATE.EL of
        when EL0 =>
            let IsEL1Regime : boolean = S1TranslationRegime() == EL1;
            Enable = if IsEL1Regime then SCTLR_EL1().EnDA else SCTLR_EL2().EnDA;
            if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
                var enda2 : bit;
                if IsEL1Regime then
                    enda2 = if IsSCTLR2EL1Enabled() then SCTLR2_EL1().EnDA2 else '0';
                else
                    enda2 = if IsSCTLR2EL2Enabled() then SCTLR2_EL2().EnDA2 else '0';
                end;
                Enable = Enable XOR enda2;
            end;
            TrapEL2 = EL2Enabled() && HCR_EL2().API == '0' && !IsInHost();
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL1 =>
            Enable  = SCTLR_EL1().EnDA;
            TrapEL2 = EL2Enabled() && HCR_EL2().API == '0';
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL2 =>
            Enable  = SCTLR_EL2().EnDA;
            TrapEL2 = FALSE;
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL3 =>
            Enable  = SCTLR_EL3().EnDA;
            TrapEL2 = FALSE;
            TrapEL3 = FALSE;
    end;

    if Enable == '0' then
        return FALSE;
    elsif TrapEL3 && EL3SDDUndefPriority() then
        Undefined();
    elsif TrapEL2 then
        TrapPACUse(EL2);
    elsif TrapEL3 then
        if EL3SDDUndef() then
            Undefined();
        else
            TrapPACUse(EL3);
        end;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/functions/pac/pac/IsAPDBKeyEnabled

```
// IsAPDBKeyEnabled()
// ==================
// Returns TRUE if authentication using the APDBKey_EL1 key is enabled.
// Otherwise, depending on the state of the PE, generate a trap, or return FALSE.

func IsAPDBKeyEnabled() => boolean
begin
    var TrapEL2 : boolean;
    var TrapEL3 : boolean;
    var Enable : bits(1);

    case PSTATE.EL of
        when EL0 =>
            let IsEL1Regime : boolean = S1TranslationRegime() == EL1;
            Enable = if IsEL1Regime then SCTLR_EL1().EnDB else SCTLR_EL2().EnDB;
            if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
                var endb2 : bit;
                if IsEL1Regime then
                    endb2 = if IsSCTLR2EL1Enabled() then SCTLR2_EL1().EnDB2 else '0';
                else
                    endb2 = if IsSCTLR2EL2Enabled() then SCTLR2_EL2().EnDB2 else '0';
                end;
                Enable = Enable XOR endb2;
            end;
            TrapEL2 = EL2Enabled() && HCR_EL2().API == '0' && !IsInHost();
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL1 =>
            Enable  = SCTLR_EL1().EnDB;
            TrapEL2 = EL2Enabled() && HCR_EL2().API == '0';
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL2 =>
            Enable  = SCTLR_EL2().EnDB;
            TrapEL2 = FALSE;
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL3 =>
            Enable  = SCTLR_EL3().EnDB;
            TrapEL2 = FALSE;
            TrapEL3 = FALSE;
    end;

    if Enable == '0' then
        return FALSE;
    elsif TrapEL3 && EL3SDDUndefPriority() then
        Undefined();
    elsif TrapEL2 then
        TrapPACUse(EL2);
    elsif TrapEL3 then
        if EL3SDDUndef() then
            Undefined();
        else
            TrapPACUse(EL3);
        end;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/functions/pac/pac/IsAPIAKeyEnabled

```
// IsAPIAKeyEnabled()
// ==================
// Returns TRUE if authentication using the APIAKey_EL1 key is enabled.
// Otherwise, depending on the state of the PE, generate a trap, or return FALSE.

func IsAPIAKeyEnabled() => boolean
begin
    var TrapEL2 : boolean;
    var TrapEL3 : boolean;
    var Enable : bits(1);

    case PSTATE.EL of
        when EL0 =>
            let IsEL1Regime : boolean = S1TranslationRegime() == EL1;
            Enable = if IsEL1Regime then SCTLR_EL1().EnIA else SCTLR_EL2().EnIA;
            if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
                var enia2 : bit;
                if IsEL1Regime then
                    enia2 = if IsSCTLR2EL1Enabled() then SCTLR2_EL1().EnIA2 else '0';
                else
                    enia2 = if IsSCTLR2EL2Enabled() then SCTLR2_EL2().EnIA2 else '0';
                end;
                Enable = Enable XOR enia2;
            end;
            TrapEL2 = EL2Enabled() && HCR_EL2().API == '0' && !IsInHost();
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL1 =>
            Enable  = SCTLR_EL1().EnIA;
            TrapEL2 = EL2Enabled() && HCR_EL2().API == '0';
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL2 =>
            Enable  = SCTLR_EL2().EnIA;
            TrapEL2 = FALSE;
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL3 =>
            Enable  = SCTLR_EL3().EnIA;
            TrapEL2 = FALSE;
            TrapEL3 = FALSE;
    end;

    if Enable == '0' then
        return FALSE;
    elsif TrapEL3 && EL3SDDUndefPriority() then
        Undefined();
    elsif TrapEL2 then
        TrapPACUse(EL2);
    elsif TrapEL3 then
        if EL3SDDUndef() then
            Undefined();
        else
            TrapPACUse(EL3);
        end;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/functions/pac/pac/IsAPIBKeyEnabled

```
// IsAPIBKeyEnabled()
// ==================
// Returns TRUE if authentication using the APIBKey_EL1 key is enabled.
// Otherwise, depending on the state of the PE, generate a trap, or return FALSE.

func IsAPIBKeyEnabled() => boolean
begin
    var TrapEL2 : boolean;
    var TrapEL3 : boolean;
    var Enable : bits(1);

    case PSTATE.EL of
        when EL0 =>
            let IsEL1Regime : boolean = S1TranslationRegime() == EL1;
            Enable = if IsEL1Regime then SCTLR_EL1().EnIB else SCTLR_EL2().EnIB;
            if IsFeatureImplemented(FEAT_PAuth_EnhCtl) then
                var enib2 : bit;
                if IsEL1Regime then
                    enib2 = if IsSCTLR2EL1Enabled() then SCTLR2_EL1().EnIB2 else '0';
                else
                    enib2 = if IsSCTLR2EL2Enabled() then SCTLR2_EL2().EnIB2 else '0';
                end;
                Enable = Enable XOR enib2;
            end;
            TrapEL2 = EL2Enabled() && HCR_EL2().API == '0' && !IsInHost();
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL1 =>
            Enable  = SCTLR_EL1().EnIB;
            TrapEL2 = EL2Enabled() && HCR_EL2().API == '0';
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL2 =>
            Enable  = SCTLR_EL2().EnIB;
            TrapEL2 = FALSE;
            TrapEL3 = HaveEL(EL3) && SCR_EL3().API == '0';
        when EL3 =>
            Enable  = SCTLR_EL3().EnIB;
            TrapEL2 = FALSE;
            TrapEL3 = FALSE;
    end;

    if Enable == '0' then
        return FALSE;
    elsif TrapEL3 && EL3SDDUndefPriority() then
        Undefined();
    elsif TrapEL2 then
        TrapPACUse(EL2);
    elsif TrapEL3 then
        if EL3SDDUndef() then
            Undefined();
        else
            TrapPACUse(EL3);
        end;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/functions/pac/pac/IsPACMEnabled

```
// IsPACMEnabled()
// ===============
// Returns TRUE if the effects of the PACM instruction are enabled, otherwise FALSE.

func IsPACMEnabled() => boolean
begin
    assert IsFeatureImplemented(FEAT_PAuth) && IsFeatureImplemented(FEAT_PAuth_LR);

    if IsTrivialPACMImplementation() then
        return FALSE;
    end;

    var enabled : boolean;

    // EL2 could force the behavior at EL1 and EL0 to NOP.
    if PSTATE.EL IN {EL0, EL1} && EL2Enabled() && !IsInHost() then
        enabled = IsHCRXEL2Enabled() && HCRX_EL2().PACMEn == '1';
    else
        enabled = TRUE;
    end;

    // Otherwise, the SCTLR2_ELx bit determines the behavior.
    if enabled then
        var enpacm_bit : bit;
        case PSTATE.EL of
            when EL3 =>
                enpacm_bit = SCTLR2_EL3().EnPACM;
            when EL2 =>
                enpacm_bit = if IsSCTLR2EL2Enabled() then SCTLR2_EL2().EnPACM else '0';
            when EL1 =>
                enpacm_bit = if IsSCTLR2EL1Enabled() then SCTLR2_EL1().EnPACM else '0';
            when EL0 =>
                if IsInHost() then
                    enpacm_bit = if IsSCTLR2EL2Enabled() then SCTLR2_EL2().EnPACM0 else '0';
                else
                    enpacm_bit = if IsSCTLR2EL1Enabled() then SCTLR2_EL1().EnPACM0 else '0';
                end;
        end;
        enabled = enpacm_bit == '1';
    end;

    return enabled;
end;
```

Library pseudocode for aarch64/functions/pac/pac/IsTrivialPACMImplementation

```
// IsTrivialPACMImplementation()
// =============================
// Returns TRUE if the PE has a trivial implementation of PACM.

func IsTrivialPACMImplementation() => boolean
begin
    return (IsFeatureImplemented(FEAT_PACIMP) &&
              ImpDefBool("Trivial PSTATE.PACM implementation"));
end;
```

Library pseudocode for aarch64/functions/pac/pac/PtrHasUpperAndLowerAddRanges

```
// PtrHasUpperAndLowerAddRanges()
// ==============================
// Returns TRUE if the pointer has upper and lower address ranges, FALSE otherwise.

func PtrHasUpperAndLowerAddRanges() => boolean
begin
    let regime : Regime = TranslationRegime(PSTATE.EL);

    return HasUnprivileged(regime);
end;
```

Library pseudocode for aarch64/functions/pac/strip/Strip

```
// Strip()
// =======
// Strip() returns a 64-bit value containing A, but replacing the pointer authentication
// code field bits with the extension of the address bits. This can apply to either
// instructions or data, where, as the use of tagged pointers is distinct, it might be
// handled differently.

readonly func Strip(A : bits(64), data : boolean) => bits(64)
begin
    var original_ptr : bits(64);
    var extfield : bits(64);
    let tbi : boolean = EffectiveTBI(A, !data, PSTATE.EL) == '1';
    let mtx : boolean = (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) &&
                         EffectiveMTX(A, !data, PSTATE.EL) == '1');
    let bottom_PAC_bit : AddressSize = CalculateBottomPACBit(A[55]);
    let EL3_using_lva3 : boolean = (IsFeatureImplemented(FEAT_LVA3) &&
                                    TranslationRegime(PSTATE.EL) == Regime_EL3 &&
                                    AArch64_IASize(TCR_EL3().T0SZ) > 52);
    if EL3_using_lva3 then
        extfield = Replicate{64}('0');
    else
        extfield = Replicate{64}(A[55]);
    end;

    if tbi then
        if (bottom_PAC_bit <= 55) then
            original_ptr = (A[63:56] ::
                            extfield[55:bottom_PAC_bit] :: A[bottom_PAC_bit-1:0]);
        else
            original_ptr = A[63:56] :: A[55:0];
        end;
    elsif mtx then
        if (bottom_PAC_bit <= 55) then
            original_ptr = (extfield[63:60] :: A[59:56] ::
                            extfield[55:bottom_PAC_bit] :: A[bottom_PAC_bit-1:0]);
        else
            original_ptr = extfield[63:60] :: A[59:56] :: A[55:0];
        end;
    else
        original_ptr =  extfield[63:bottom_PAC_bit] :: A[bottom_PAC_bit-1:0];
    end;

    return original_ptr;
end;
```

Library pseudocode for aarch64/functions/pac/trappacuse/TrapPACUse

```
// TrapPACUse()
// ============
// Used for the trapping of the pointer authentication functions by higher exception
// levels.

noreturn func TrapPACUse(target_el : bits(2))
begin
    assert HaveEL(target_el) && target_el !=  EL0 && UInt(target_el) >= UInt(PSTATE.EL);

    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    var except : ExceptionRecord;
    let vect_offset : integer = 0;
    except = ExceptionSyndrome(Exception_PACTrap);
    AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
end;
```

Library pseudocode for aarch64/functions/predictionrestrict/AArch64_RestrictPrediction

```
// AArch64_RestrictPrediction()
// ============================
// Clear all predictions in the context.

func AArch64_RestrictPrediction(val : bits(64), restriction : RestrictType)
begin

    var c : ExecutionCntxt;
    let target_el : bits(2)    = val[25:24];

    // If the target EL is not implemented or the instruction is executed at an
    // EL lower than the specified level, the instruction is treated as a NOP.
    if !HaveEL(target_el) || UInt(target_el) > UInt(PSTATE.EL) then ExecuteAsNOP(); end;

    let ns : bit  = val[26];
    let nse : bit = val[27];
    let ss : SecurityState = TargetSecurityState(ns, nse);

    // If the combination of Security state and Exception level is not implemented,
    // the instruction is treated as a NOP.
    if ss == SS_Root && target_el != EL3 then ExecuteAsNOP(); end;
    if !IsFeatureImplemented(FEAT_RME) && target_el == EL3 && ss != SS_Secure then
        ExecuteAsNOP();
    end;

    c.security  = ss;
    c.target_el = target_el;

    if EL2Enabled() then
        if (PSTATE.EL == EL0 && !IsInHost()) || PSTATE.EL == EL1 then
            c.is_vmid_valid = TRUE;
            c.all_vmid      = FALSE;
            c.vmid          = VMID();

        elsif (target_el == EL0 && !ELIsInHost(target_el)) || target_el == EL1 then
            c.is_vmid_valid = TRUE;
            c.all_vmid      = val[48] == '1';
            c.vmid          = val[47:32];       // Only valid if  val[48] == '0';

        else
            c.is_vmid_valid = FALSE;
        end;
    else
        c.is_vmid_valid = FALSE;
    end;

    if PSTATE.EL == EL0 then
        c.is_asid_valid = TRUE;
        c.all_asid      = FALSE;
        c.asid          = ASID();

    elsif target_el == EL0 then
        c.is_asid_valid = TRUE;
        c.all_asid      = val[16] == '1';
        c.asid          = val[15:0];            // Only valid if  val[16] == '0';

    else
        c.is_asid_valid = FALSE;
    end;

    c.software_context = Zeros{64};
    if IsFeatureImplemented(FEAT_CSV2_2) then
        case target_el of
            when EL3 => c.software_context = SCXTNUM_EL3();
            when EL2 => c.software_context = SCXTNUM_EL2();
            when EL1 => c.software_context = SCXTNUM_EL1();
            when EL0 => c.software_context = SCXTNUM_EL0();
        end;
    end;

    c.restriction = restriction;
    RESTRICT_PREDICTIONS(c);
end;
```

Library pseudocode for aarch64/functions/prefetch/Prefetch

```
// Prefetch()
// ==========

// Decode and execute the prefetch hint on ADDRESS specified by PRFOP

func Prefetch(address : bits(64), prfop : bits(5), supports_ir : boolean)
begin
    var hint : PrefetchHint;
    var target : integer;
    var stream : boolean;

    case prfop of
        when '00xxx'                   => hint = Prefetch_READ;      // PLD: prefetch for load
        when '01xxx'                   => hint = Prefetch_EXEC;      // PLI: preload instructions
        when '10xxx'                   => hint = Prefetch_WRITE;     // PST: prepare for store
        when '11000' where supports_ir => hint = Prefetch_IR;        // IR: intent to read
        otherwise                      => return;                    // unallocated hint
    end;

    // If <type> is IR, then <target> and <policy> are not specified and Rt[2:0] are '000'.
    if hint != Prefetch_IR then
        target = UInt(prfop[2:1]);                // target cache level
        stream = (prfop[0] != '0');               // streaming (non-temporal)
    else
        target = ARBITRARY: integer;
        stream = ARBITRARY: boolean;
    end;
    Hint_Prefetch(address, hint, target, stream);
    return;
end;
```

Library pseudocode for aarch64/functions/pstatefield/PSTATEField

```
// PSTATEField
// ===========
// MSR (immediate) instruction destinations.

type PSTATEField of enumeration {
                         PSTATEField_DAIFSet, PSTATEField_DAIFClr,
                         PSTATEField_PAN, // Armv8.1
                         PSTATEField_UAO, // Armv8.2
                         PSTATEField_DIT, // Armv8.4
                         PSTATEField_SSBS,
                         PSTATEField_TCO, // Armv8.5
                         PSTATEField_SVCRSM,
                         PSTATEField_SVCRZA,
                         PSTATEField_SVCRSMZA,
                         PSTATEField_ALLINT,
                         PSTATEField_PM,
                         PSTATEField_SP
                         };
```

Library pseudocode for aarch64/functions/ras/AArch64_DelegatedSErrorTarget

```
// AArch64_DelegatedSErrorTarget()
// ===============================
// Returns whether a delegated SError exception pended by SCR_EL3.VSE is masked,
// and the target Exception level of the delegated SError exception.

func AArch64_DelegatedSErrorTarget() => (boolean, bits(2))
begin
    assert IsFeatureImplemented(FEAT_E3DSE);
    if Halted() || PSTATE.EL == EL3 then
        return (TRUE, ARBITRARY : bits(2));
    end;

    let effective_amo : bit = EffectiveHCR_AMO();
    let effective_tge : bit = EffectiveTGE();

    // The exception is masked by software.
    var masked : boolean;
    case PSTATE.EL of
        when EL2 =>
            masked = ((effective_tge == '0' && effective_amo == '0') || PSTATE.A == '1');
        when EL1, EL0 =>
            masked = (effective_amo == '0' && PSTATE.A == '1');
        otherwise =>
            unreachable;
    end;

    // The exception might be disabled debug in the Security state indicated by
    // SCR_EL3().[NS, NSE] by external debug.
    let intdis : boolean = ExternalDebugInterruptsDisabled(EL1);

    var target_el : bits(2) = ARBITRARY : bits(2);
    if EL2Enabled() && effective_amo == '1' && !intdis && PSTATE.EL IN {EL0, EL1} then
        target_el = EL2;
        masked = FALSE;

    elsif (EffectiveHCRX_EL2_TMEA() == '1' && !intdis &&
             ((PSTATE.EL == EL1 && PSTATE.A == '1') ||
              (PSTATE.EL == EL0 && masked && !IsInHost()))) then
        target_el = EL2;
        masked = FALSE;

    elsif PSTATE.EL == EL2 || IsInHost() then
        if !masked then target_el = EL2; end;

    else
        assert (PSTATE.EL == EL1 || (PSTATE.EL == EL0 && !IsInHost()));
        if !masked then target_el = EL1; end;
    end;

    // External debug might disable the delegated exception for the target Exception level.
    if ExternalDebugInterruptsDisabled(target_el) then
        masked = TRUE;
        target_el = ARBITRARY : bits(2);
    end;

    return (masked, target_el);
end;
```

Library pseudocode for aarch64/functions/ras/AArch64_ESBOperation

```
// AArch64_ESBOperation()
// ======================
// Perform the AArch64 ESB operation, either for ESB executed in AArch64 state, or for
// ESB in AArch32 state when SError interrupts are routed to an Exception level using
// AArch64

func AArch64_ESBOperation()
begin
    var target_el : bits(2);
    var masked : boolean;

    (masked, target_el) = PhysicalSErrorTarget();

    // Check for a masked Physical SError pending that can be synchronized
    // by an Error synchronization event.
    if masked && IsSynchronizablePhysicalSErrorPending() then
        // This function might be called for an interprocessing case, and INTdis is masking
        // the SError interrupt.
        if ELUsingAArch32(S1TranslationRegime()) then
            var syndrome : bits(32) = Zeros{};
            syndrome[31]      = '1';    // A
            syndrome[15:0]    = AArch32_PhysicalSErrorSyndrome();
            DISR()            = syndrome;
        else
            let implicit_esb : boolean = FALSE;
            let is_esb : boolean       = TRUE;
            var syndrome : bits(64)    = Zeros{};

            syndrome[31]   = '1';       // A
            syndrome[24:0] = AArch64_PhysicalSErrorSyndrome(is_esb, implicit_esb);
            DISR_EL1()     = syndrome;
        end;
        ClearPendingPhysicalSError();   // Set ISR_EL1.A to 0
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/ras/AArch64_EncodeAsyncErrorSyndrome

```
// AArch64_EncodeAsyncErrorSyndrome()
// ==================================
// Return the encoding for specified ErrorState for an SError exception taken
// to AArch64 state.

func AArch64_EncodeAsyncErrorSyndrome(errorstate : ErrorState) => bits(3)
begin
    case errorstate of
        when ErrorState_UC  => return '000';
        when ErrorState_UEU => return '001';
        when ErrorState_UEO => return '010';
        when ErrorState_UER => return '011';
        when ErrorState_CE  => return '110';
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for aarch64/functions/ras/AArch64_EncodeSyncErrorSyndrome

```
// AArch64_EncodeSyncErrorSyndrome()
// =================================
// Return the encoding for specified ErrorState for a synchronous Abort
// exception taken to AArch64 state.

func AArch64_EncodeSyncErrorSyndrome(errorstate : ErrorState) => bits(2)
begin
    case errorstate of
        when ErrorState_UC  => return '10';
        when ErrorState_UEU => return '10';    // UEU is reported as UC
        when ErrorState_UEO => return '11';
        when ErrorState_UER => return '00';
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for aarch64/functions/ras/AArch64_PhysicalSErrorSyndrome

```
// AArch64_PhysicalSErrorSyndrome()
// ================================
// Generate SError syndrome.

func AArch64_PhysicalSErrorSyndrome(is_esb : boolean, implicit_esb : boolean) => bits(25)
begin
    var syndrome : bits(25) = Zeros{};
    var target_el : bits(2);
    (-, target_el) = PhysicalSErrorTarget();

    if ReportErrorAsUncategorized() then
        syndrome = Zeros{25};
    elsif ReportErrorAsIMPDEF() then
        syndrome[24]   = '1';                                          // IDS
        syndrome[23:0] = ImpDefBits{24}("IMPDEF ErrorState");
    else
        let fault : FaultRecord = GetPendingPhysicalSError();
        let errorstate : ErrorState = PEErrorState(fault);
        syndrome[24]    = '0';                                         // IDS
        if (!is_esb && IsFeatureImplemented(FEAT_PFAR) &&
              !(EL2Enabled() &&
              (HCR_EL2().VM == '1' || (IsFeatureImplemented(FEAT_HCRMAC) && HCR_EL2().DC == '1')) &&
              target_el == EL1)) then
            syndrome[14] = if IsPFARValid(fault) then '1' else '0';
        end;
        syndrome[13]    = if implicit_esb then '1' else '0';           // IESB
        syndrome[12:10] = AArch64_EncodeAsyncErrorSyndrome(errorstate);   // AET
        syndrome[9]     = fault.extflag;                               // EA
        syndrome[5:0]   = '010001';                                    // DFSC
    end;

    return syndrome;
end;
```

Library pseudocode for aarch64/functions/ras/AArch64_dESBOperation

```
// AArch64_dESBOperation()
// =======================
// Perform the AArch64 ESB operation for a pending delegated SError exception.

func AArch64_dESBOperation()
begin
    assert (IsFeatureImplemented(FEAT_E3DSE) && !ELUsingAArch32(EL3) && PSTATE.EL != EL3);
    // When FEAT_E3DSE is implemented, SCR_EL3.DSE might inject a delegated SError exception.
    var dsei_pending, dsei_masked : boolean;
    dsei_pending = EffectiveSCR_EL3_EnDSE() == '1' && SCR_EL3().DSE == '1';
    (dsei_masked, -) = AArch64_DelegatedSErrorTarget();
    if dsei_pending && dsei_masked then
        var target : bits(64) = Zeros{};
        target[31] = '1';                  // A
        target[24:0] = VSESR_EL3()[24:0];
        VDISR_EL3() = target;
        ClearPendingDelegatedSError();
    end;
    return;
end;
```

Library pseudocode for aarch64/functions/ras/AArch64_vESBOperation

```
// AArch64_vESBOperation()
// =======================
// Perform the AArch64 ESB operation for an unmasked pending virtual SError exception.
// If FEAT_E3DSE is implemented and there is no unmasked virtual SError exception
// pending, then AArch64_dESBOperation() is called to perform the AArch64 ESB operation
// for a pending delegated SError exception.

func AArch64_vESBOperation()
begin
    assert PSTATE.EL IN {EL0, EL1} && EL2Enabled() && !ELUsingAArch32(EL2);

    // If physical SError exceptions are routed to EL2, and TGE is not set, then a virtual
    // SError exception might be pending.
    let vsei_pending : boolean = (IsVirtualSErrorPending() && EffectiveTGE() == '0' &&
                                  (EffectiveHCR_AMO() == '1' || EffectiveHCRX_EL2_TMEA() == '1'));
    let vsei_masked : boolean = PSTATE.A == '1' || Halted() || ExternalDebugInterruptsDisabled(EL1);

    // Check for a masked virtual SError pending
    if vsei_pending && vsei_masked then
        // This function might be called for the interprocessing case, and INTdis is masking
        // the virtual SError exception.
        if ELUsingAArch32(EL1) then
            var target : bits(32) = Zeros{};
            target[31]    = '1';              // A
            target[15:14] = VDFSR()[15:14];   // AET
            target[12]    = VDFSR()[12];      // ExT
            target[9]     = TTBCR().EAE;      // LPAE
            if TTBCR().EAE == '1' then        // Long-descriptor format
                target[5:0]    = '010001';    // STATUS
            else                              // Short-descriptor format
                target[10,3:0] = '10110';     // FS
            end;
            VDISR() = target;
        else
            var target : bits(64) = Zeros{};
            target[31]   = '1';               // A
            target[24:0] = VSESR_EL2()[24:0];
            VDISR_EL2() = target;
        end;
        ClearPendingVirtualSError();
    elsif IsFeatureImplemented(FEAT_E3DSE) then
        AArch64_dESBOperation();
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/ras/CheckEnADERRApplies

```
// CheckEnADERRApplies()
// =====================
// Returns TRUE if SCTLR2_ELx().EnADERR applies to the given memory read access.

func CheckEnADERRApplies(accdesc : AccessDescriptor) => boolean
begin
     var imp_specific_access : boolean = FALSE;

    case accdesc.acctype of
        when AccessType_ASIMD  => imp_specific_access = TRUE;
        when AccessType_SVE    => imp_specific_access = TRUE;
        when AccessType_SME    => imp_specific_access = TRUE;
        when AccessType_GPR    =>
            imp_specific_access = FALSE;
            imp_specific_access = imp_specific_access || accdesc.atomicop;
            imp_specific_access = imp_specific_access || accdesc.rcw || accdesc.rcws;
            imp_specific_access = imp_specific_access || (accdesc.ls64 && accdesc.withstatus);

        when AccessType_NV2    => imp_specific_access = TRUE;
        otherwise              => imp_specific_access = FALSE;
    end;

    if imp_specific_access then
        return ImpDefBool("EnADERR applies to given memory read AccessType");
    end;
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/ras/CheckEnANERRApplies

```
// CheckEnANERRApplies()
// =====================
// Returns TRUE if SCTLR2_ELx().EnANERR applies to the given memory read access.

func CheckEnANERRApplies(accdesc : AccessDescriptor, desc : AddressDescriptor) => boolean
begin
    var imp_specific_access : boolean = FALSE;

    case accdesc.acctype of
        when AccessType_GPR =>
            imp_specific_access = FALSE;
            imp_specific_access = imp_specific_access || accdesc.atomicop;
            imp_specific_access = imp_specific_access || accdesc.rcw || accdesc.rcws;
            imp_specific_access = imp_specific_access || (accdesc.ls64 && accdesc.withstatus);
        when {AccessType_ASIMD, AccessType_SVE} =>
            imp_specific_access = IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1';
        when AccessType_SME =>
            imp_specific_access = TRUE;
        when AccessType_NV2 =>
            imp_specific_access = !IsWBShareable(desc.memattrs);
        otherwise =>
            imp_specific_access = FALSE;
    end;

    if imp_specific_access then
        return ImpDefBool("EnANERR applies to given memory read AccessType");
    end;
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/ras/EffectiveHCRX_EL2_EnSDERR

```
// EffectiveHCRX_EL2_EnSDERR()
// ===========================
// Return the Effective value of HCRX_EL2().EnSDERR

func EffectiveHCRX_EL2_EnSDERR() => bit
begin
    if !IsHCRXEL2Enabled() then return '0'; end;
    if ELIsInHost(EL0) then return '0'; end;

     if (IsFeatureImplemented(FEAT_ANERR) && GetADERRErrorMode() == ErrSignalMode_TiedToPeer &&
           HCRX_EL2().[EnSDERR, EnSNERR] == '10') then
        return '0';
    end;
    return HCRX_EL2().EnSDERR;
end;
```

Library pseudocode for aarch64/functions/ras/EffectiveHCRX_EL2_EnSNERR

```
// EffectiveHCRX_EL2_EnSNERR()
// ===========================
// Return the Effective value of HCRX_EL2().EnSNERR

func EffectiveHCRX_EL2_EnSNERR() => bit
begin
    if !IsHCRXEL2Enabled() then return '0'; end;
    if ELIsInHost(EL0) then return '0'; end;

    if (IsFeatureImplemented(FEAT_ADERR) && GetADERRErrorMode() == ErrSignalMode_TiedToPeer &&
          HCRX_EL2().[EnSDERR, EnSNERR] == '01') then
        return '0';
    end;
    return HCRX_EL2().EnSNERR;
end;
```

Library pseudocode for aarch64/functions/ras/ErrSignalMode

```
// ErrSignalMode
// =============
// Implementation options for the ANERR/ADERR synchronous error behavior.

type ErrSignalMode of enumeration {
    ErrSignalMode_NotImplemented,
    ErrSignalMode_EnSelfOnly,        // EnANERR-only / EnADERR-only
    ErrSignalMode_TiedToPeer,        // ANERR tied to ADERR, or vice versa
    ErrSignalMode_EnxNERRIndependent
};
```

Library pseudocode for aarch64/functions/ras/FirstRecordOfNode

```
// FirstRecordOfNode()
// ===================
// Return the first record in the node that contains the record n.

func FirstRecordOfNode(n : integer) => integer
begin
    for q = n downto 0 do
        if IsFirstRecordOfNode(q) then return q; end;
    end;
    unreachable;
end;
```

Library pseudocode for aarch64/functions/ras/GetADERRErrorMode

```
// GetADERRErrorMode()
// ===================
// Returns the ADERR implementation level

impdef func GetADERRErrorMode() => ErrSignalMode
begin
    return ErrSignalMode_EnxNERRIndependent;
end;
```

Library pseudocode for aarch64/functions/ras/GetANERRErrorMode

```
// GetANERRErrorMode()
// ===================
// Returns the ANERR implementation level

impdef func GetANERRErrorMode() => ErrSignalMode
begin
    return ErrSignalMode_EnxNERRIndependent;
end;
```

Library pseudocode for aarch64/functions/ras/IsCommonFaultInjectionImplemented

```
// IsCommonFaultInjectionImplemented()
// ===================================
// Check if the Common Fault Injection Model Extension is implemented by the node that owns this
// error record.

impdef func IsCommonFaultInjectionImplemented(n : integer) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/ras/IsCountableErrorsRecorded

```
// IsCountableErrorsRecorded()
// ===========================
// Check whether Error record n records countable errors.

impdef func IsCountableErrorsRecorded(n : integer) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/ras/IsDeviceMemoryReadErrorSynchronous

```
// IsDeviceMemoryReadErrorSynchronous()
// ====================================
// Returns TRUE when the Device memory read is synchronous in the current Exception level.

func IsDeviceMemoryReadErrorSynchronous(accdesc : AccessDescriptor, desc : AddressDescriptor)
                                        => boolean
begin
    if !IsFeatureImplemented(FEAT_ADERR) || !CheckEnADERRApplies(accdesc) then
        return ImpDefBool("Device read error sync when EnADERR not implemented or does not apply");
    end;
    case S1TranslationRegime(accdesc.el) of
        when EL3 =>
            if ((!IsFeatureImplemented(FEAT_ANERR) ||
                 GetADERRErrorMode() != ErrSignalMode_TiedToPeer ||
                 SCTLR2_EL3().EnANERR == '0') && SCTLR2_EL3().EnADERR == '0') then
                return TRUE;
            end;

        when EL2 =>
            if !IsSCTLR2EL2Enabled() then
                if ImpDefBool("SCTLR2_ELx.EnADERR resets to 0 at highest ELx") then
                    return TRUE;
                end;
            elsif ((!IsFeatureImplemented(FEAT_ANERR) ||
                    GetADERRErrorMode() != ErrSignalMode_TiedToPeer ||
                    SCTLR2_EL2().EnANERR == '0') && SCTLR2_EL2().EnADERR == '0') then
                return TRUE;
            end;

        when EL1 =>
            if EffectiveHCRX_EL2_EnSDERR() == '1' then
                return TRUE;
            end;
            if !IsSCTLR2EL1Enabled() then
                if ImpDefBool("SCTLR2_ELx.EnADERR resets to 0 at highest ELx") then
                    return TRUE;
                end;
            elsif ((!IsFeatureImplemented(FEAT_ANERR) ||
                    GetADERRErrorMode() != ErrSignalMode_TiedToPeer ||
                    SCTLR2_EL1().EnANERR == '0') &&
                   SCTLR2_EL1().EnADERR == '0') then
                return TRUE;
            end;
        otherwise => unreachable;
    end;
    return ImpDefBool("Device read error sync when the Effective value of EnADERR is 1");
end;
```

Library pseudocode for aarch64/functions/ras/IsErrorAddressIncluded

```
// IsErrorAddressIncluded()
// ========================
// Check whether Error record n includes an address associated with an error.

impdef func IsErrorAddressIncluded(n : integer) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/ras/IsErrorRecordImplemented

```
// IsErrorRecordImplemented()
// ==========================
// Is the error record n implemented

readonly impdef func IsErrorRecordImplemented(n : integer) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/ras/IsFirstRecordOfNode

```
// IsFirstRecordOfNode()
// =====================
// Check if the record q is the first error record in its node.

readonly impdef func IsFirstRecordOfNode(q : integer) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/ras/IsNormalMemoryReadErrorSynchronous

```
// IsNormalMemoryReadErrorSynchronous()
// ====================================
// Returns TRUE when error signaled on a load from Normal memory is taken synchronously.

func IsNormalMemoryReadErrorSynchronous(accdesc : AccessDescriptor, desc : AddressDescriptor)
                                        => boolean
begin
    if !IsFeatureImplemented(FEAT_ANERR) || !CheckEnANERRApplies(accdesc, desc) then
        return ImpDefBool("Normal read error sync when EnANERR not implemented or does not apply");
    end;
    case S1TranslationRegime(accdesc.el) of
        when EL3 =>
            if ((!IsFeatureImplemented(FEAT_ADERR) ||
                  GetANERRErrorMode() != ErrSignalMode_TiedToPeer ||
                  SCTLR2_EL3().EnADERR == '0') && SCTLR2_EL3().EnANERR == '0') then
                return TRUE;
            end;

        when EL2 =>
            if !IsSCTLR2EL2Enabled() then
                if ImpDefBool("SCTLR2_ELx.EnANERR resets to 0 at highest ELx") then
                    return TRUE;
                end;
            elsif ((!IsFeatureImplemented(FEAT_ADERR) ||
                    GetANERRErrorMode() != ErrSignalMode_TiedToPeer ||
                    SCTLR2_EL2().EnADERR == '0') && SCTLR2_EL2().EnANERR == '0') then
                return TRUE;
            end;

        when EL1 =>
            if EffectiveHCRX_EL2_EnSNERR() == '1' then
                return TRUE;
            end;
            if !IsSCTLR2EL1Enabled() then
                if ImpDefBool("SCTLR2_ELx.EnANERR resets to 0 at highest ELx") then
                    return TRUE;
                end;
            elsif ((!IsFeatureImplemented(FEAT_ADERR) ||
                    GetANERRErrorMode() != ErrSignalMode_TiedToPeer ||
                    SCTLR2_EL1().EnADERR == '0') &&
                   SCTLR2_EL1().EnANERR == '0') then
                return TRUE;
            end;
        otherwise => unreachable;
    end;
    return ImpDefBool("Normal read error sync when the Effective value of EnANERR is 1");
end;
```

Library pseudocode for aarch64/functions/ras/IsPFARValid

```
// IsPFARValid()
// =============
// Determine if faulting PA can be reported.

impdef func IsPFARValid(fault : FaultRecord) => boolean
begin
    return !IsAsyncAbort(fault);
end;
```

Library pseudocode for aarch64/functions/ras/IsSPMUCounterImplemented

```
// IsSPMUCounterImplemented()
// ==========================
// Does the System PMU s implement the counter n.

impdef func IsSPMUCounterImplemented(s : integer, n : integer) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for aarch64/functions/rcw

```
constant RCW64_PROTECTED_BIT : integer = 52;
constant RCW128_PROTECTED_BIT : integer = 114;
```

Library pseudocode for aarch64/functions/rcw/ProtectionEnabled

```
// ProtectionEnabled()
// ===================
// Returns TRUE if the ProtectedBit is
// enabled in the current Exception level.

func ProtectionEnabled(el : bits(2)) => boolean
begin
    assert HaveEL(el);
    let regime : bits(2) = S1TranslationRegime(el);
    assert(!ELUsingAArch32(regime));
    if (!IsD128Enabled(el)) then
        case regime of
            when EL1 =>
                return IsTCR2EL1Enabled() && TCR2_EL1().PnCH == '1';
            when EL2 =>
                return IsTCR2EL2Enabled() && TCR2_EL2().PnCH == '1';
            when EL3 =>
                return TCR_EL3().PnCH == '1';
        end;
    else
        return TRUE;
    end;
    return FALSE;
end;
```

Library pseudocode for aarch64/functions/rcw/RCWCheck

```
// RCWCheck()
// ==========
// Returns nzcv based on : if the new value for RCW/RCWS instructions satisfy RCW and/or RCWS checks
// Z is set to 1 if RCW checks fail
// C is set to 0 if RCWS checks fail

func RCWCheck{N}(old : bits(N), new : bits(N), soft : boolean) => bits(4)
begin
    assert N IN {64,128};
    let protectedbit : integer = if N == 128 then RCW128_PROTECTED_BIT else RCW64_PROTECTED_BIT;
    var rcw_fail : boolean        = FALSE;
    var rcws_fail : boolean       = FALSE;
    var rcw_state_fail : boolean  = FALSE;
    var rcws_state_fail : boolean = FALSE;
    var rcw_mask_fail : boolean   = FALSE;
    var rcws_mask_fail : boolean  = FALSE;

    //Effective RCWMask calculation
    var rcwmask : bits(N) = RCWMASK_EL1()[N-1:0];
    if N == 64 then
        rcwmask[49:18] = Replicate{32}(rcwmask[17]);
        rcwmask[0] = '0';
    else
        rcwmask[55:17] = Replicate{39}(rcwmask[16]);
        rcwmask[126:125,120:119,107:101,90:56,1:0] = Zeros{48};
    end;

    //Effective RCWSMask calculation
    var rcwsoftmask : bits(N) = RCWSMASK_EL1()[N-1:0];
    if N == 64 then
        rcwsoftmask[49:18] = Replicate{32}(rcwsoftmask[17]);
        rcwsoftmask[0] = '0';
        if(ProtectionEnabled(PSTATE.EL)) then
            rcwsoftmask[52] = '0';
        end;
    else
        rcwsoftmask[55:17] = Replicate{39}(rcwsoftmask[16]);
        rcwsoftmask[126:125,120:119,107:101,90:56,1:0] = Zeros{48};
        rcwsoftmask[114] = '0';
    end;

    //RCW Checks
    //State Check
    if (ProtectionEnabled(PSTATE.EL)) then
        if old[protectedbit] == '1' then
            rcw_state_fail = new[protectedbit,0] != old[protectedbit,0];
        elsif old[protectedbit] == '0' then
            rcw_state_fail = new[protectedbit] != old[protectedbit];
        end;
    end;

    //Mask Check
    if (ProtectionEnabled(PSTATE.EL)) then
        if old[protectedbit,0] == '11' then
            rcw_mask_fail = !IsZero((new XOR old) AND NOT(rcwmask));
        end;
    end;

    //RCWS Checks
    if soft then
        //State Check
        if old[0] == '1' then
            rcws_state_fail = new[0] != old[0];
        elsif (!ProtectionEnabled(PSTATE.EL) ||
              (ProtectionEnabled(PSTATE.EL) && old[protectedbit] == '0')) then
            rcws_state_fail = new[0] != old[0] ;
        end;
        //Mask Check
        if old[0] == '1' then
            rcws_mask_fail = !IsZero((new XOR old) AND NOT(rcwsoftmask));
        end;
    end;

    rcw_fail = rcw_state_fail  || rcw_mask_fail ;
    rcws_fail = rcws_state_fail || rcws_mask_fail;

    let n : bit = '0';
    let z : bit = if rcw_fail then '1' else '0';
    let c : bit = if rcws_fail then '0' else '1';
    let v : bit = '0';
    return n::z::c::v;
end;
```

Library pseudocode for aarch64/functions/reduceop/FPReduce

```
// FPReduce()
// ==========
// Perform the floating-point operation 'op' on pairs of elements from the input vector,
// reducing the vector to a scalar result.

func FPReduce{esize, N}(op : ReduceOp, input : bits(N),
                        fpcr : FPCR_Type) => bits(esize) recurselimit 8
begin
    var hi : bits(esize);
    var lo : bits(esize);
    var result : bits(esize);
    let half : integer{} = N DIV 2;

    if N == esize then
        return input[esize-1:0];
    end;

    hi = FPReduce{esize, (N - N DIV 2)}(op, input[N-1:half], fpcr);
    lo = FPReduce{esize, N DIV 2}(op, input[half-1:0], fpcr);
    case op of
        when ReduceOp_FMINNUM =>
            result = FPMinNum{esize}(lo, hi, fpcr);
        when ReduceOp_FMAXNUM =>
            result = FPMaxNum{esize}(lo, hi, fpcr);
        when ReduceOp_FMIN =>
            result = FPMin{esize}(lo, hi, fpcr);
        when ReduceOp_FMAX =>
            result = FPMax{esize}(lo, hi, fpcr);
        when ReduceOp_FADD =>
            result = FPAdd{esize}(lo, hi, fpcr);
    end;

    return result;
end;
```

Library pseudocode for aarch64/functions/reduceop/IntReduce

```
// IntReduce()
// ===========
// Perform the integer operation 'op' on pairs of elements from the input vector,
// reducing the vector to a scalar result.

func IntReduce{esize, N}(op : ReduceOp, input : bits(N)) => bits(esize) recurselimit 8
begin
    var hi : bits(esize);
    var lo : bits(esize);
    var result : bits(esize);
    let half : integer{} = N DIV 2;
    if N == esize then
        return input[esize-1:0];
    end;

    hi = IntReduce{esize, (N - N DIV 2)}(op, input[N-1:half]);
    lo = IntReduce{esize, N DIV 2}(op, input[half-1:0]);
    case op of
        when ReduceOp_ADD =>
            result = lo + hi;
    end;

    return result;
end;
```

Library pseudocode for aarch64/functions/reduceop/ReduceOp

```
// ReduceOp
// ========
// Vector reduce instruction types.

type ReduceOp of enumeration {ReduceOp_FMINNUM, ReduceOp_FMAXNUM,
                              ReduceOp_FMIN, ReduceOp_FMAX,
                              ReduceOp_FADD, ReduceOp_ADD};
```

Library pseudocode for aarch64/functions/registers/AArch64_MaybeZeroRegisterUppers

```
// AArch64_MaybeZeroRegisterUppers()
// =================================
// On taking an exception to  AArch64 from AArch32, it is CONSTRAINED UNPREDICTABLE whether the top
// 32 bits of registers visible at any lower Exception level using AArch32 are set to zero.

func AArch64_MaybeZeroRegisterUppers()
begin
    assert UsingAArch32();         // Always called from AArch32 state before entering AArch64 state

    var first : integer;
    var last : integer;
    var include_R15 : boolean;
    if PSTATE.EL == EL0 && !ELUsingAArch32(EL1) then
        first = 0;  last = 14;  include_R15 = FALSE;
    elsif PSTATE.EL IN {EL0, EL1} && EL2Enabled() && !ELUsingAArch32(EL2) then
        first = 0;  last = 30;  include_R15 = FALSE;
    else
        first = 0;  last = 30;  include_R15 = TRUE;
    end;

    for n = first to last do
        if (n != 15 || include_R15) && ConstrainUnpredictableBool(Unpredictable_ZEROUPPER) then
            _R[[n]][63:32] = Zeros{32};
        end;
    end;

    return;
end;
```

Library pseudocode for aarch64/functions/registers/AArch64_ResetGeneralRegisters

```
// AArch64_ResetGeneralRegisters()
// ===============================

func AArch64_ResetGeneralRegisters()
begin

    for i = 0 to 30 do
        X{64}(i) = ARBITRARY : bits(64);
    end;
end;
```

Library pseudocode for aarch64/functions/registers/AArch64_ResetSIMDFPRegisters

```
// AArch64_ResetSIMDFPRegisters()
// ==============================

func AArch64_ResetSIMDFPRegisters()
begin

    for i = 0 to 31 do
        V{128}(i) = ARBITRARY : bits(128);
    end;
end;
```

Library pseudocode for aarch64/functions/registers/AArch64_ResetSpecialRegisters

```
// AArch64_ResetSpecialRegisters()
// ===============================

func AArch64_ResetSpecialRegisters()
begin

    // AArch64 special registers
    SP_EL0()        = ARBITRARY : bits(64);
    SP_EL1()        = ARBITRARY : bits(64);

    SPSR_EL1()      = ARBITRARY : bits(64);
    ELR_EL1()       = ARBITRARY : bits(64);
    if HaveEL(EL2) then
        SP_EL2()    = ARBITRARY : bits(64);
        SPSR_EL2()  = ARBITRARY : bits(64);
        ELR_EL2()   = ARBITRARY : bits(64);
    end;


    // AArch32 special registers that are not architecturally mapped to AArch64 registers
    if HaveAArch32EL(EL1) then
        SPSR_fiq()[31:0] = ARBITRARY : bits(32);
        SPSR_irq()[31:0] = ARBITRARY : bits(32);
        SPSR_abt()[31:0] = ARBITRARY : bits(32);
        SPSR_und()[31:0] = ARBITRARY : bits(32);
    end;

    // External debug special registers
    DLR_EL0()    = ARBITRARY : bits(64);
    DSPSR_EL0()  = ARBITRARY : bits(64);

    return;
end;
```

Library pseudocode for aarch64/functions/registers/AArch64_ResetSystemRegisters

```
// AArch64_ResetSystemRegisters()
// ==============================

impdef func AArch64_ResetSystemRegisters(cold_reset : boolean)
begin
    return;
end;
```

Library pseudocode for aarch64/functions/registers/PC64

```
// PC64 - getter
// =============
// Read 64-bit program counter.

readonly func PC64() => bits(64)
begin
    return _PC;
end;
```

Library pseudocode for aarch64/functions/registers/SP

```
// SP - accessor
// =============

accessor SP{width}() <=> value : bits(width)
begin
    // Write a 32-bit or 64-bit value to the current stack pointer.
    setter
        assert width IN {64, 32};
        if PSTATE.SP == '0' then
            SP_EL0() = ZeroExtend{64}(value);
        else
            case PSTATE.EL of
                when EL0 =>  SP_EL0() = ZeroExtend{64}(value);
                when EL1 =>  SP_EL1() = ZeroExtend{64}(value);
                when EL2 =>  SP_EL2() = ZeroExtend{64}(value);
                when EL3 =>  SP_EL3() = ZeroExtend{64}(value);
            end;
        end;
    end;

    // Read the least-significant 32 or 64 bits from the current stack pointer.
    getter
        assert width IN {64, 32};
        if PSTATE.SP == '0' then
            return SP_EL0()[width-1:0];
        else
            case PSTATE.EL of
                when EL0 =>  return SP_EL0()[width-1:0];
                when EL1 =>  return SP_EL1()[width-1:0];
                when EL2 =>  return SP_EL2()[width-1:0];
                when EL3 =>  return SP_EL3()[width-1:0];
            end;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/registers/X

```
// X - accessor
// ============

accessor X{width}(n : integer) <=> value : bits(width)
begin
    // Write a 32-bit or 64-bit value to a general-purpose register.
    setter
        assert n >= 0 && n <= 31;
        assert width IN {32,64};
        if n != 31 then
            _R[[n]] = ZeroExtend{64}(value);
        end;
        return;
    end;

    // Read the least-significant 8, 16, 32, or 64 bits from a general-purpose register.
    readonly getter
        assert n >= 0 && n <= 31;
        let rw : integer{} = width as integer{8, 16, 32, 64};
        if n != 31 then
            return _R[[n]][rw-1:0];
        else
            return Zeros{rw};
        end;
    end;
end;

// X - accessor
// ============

accessor X{width}(lr : integer, hr : integer) <=> value : bits(width)
begin
    // Write a 128-bit value to two general-purpose registers.
    setter
        assert width == 128;
        let half : integer{} = width DIV 2;
        X{half}(lr)          = value[0+:half];
        X{half}(hr)          = value[half+:half];
        return;
    end;

    // Read 128 bits from two separate general-purpose registers.
    readonly getter
        assert width == 128;
        let half : integer{} = width DIV 2;
        return X{half}(hr) :: X{half}(lr);
    end;
end;
```

Library pseudocode for aarch64/functions/shiftreg/DecodeShift

```
// DecodeShift()
// =============
// Decode shift encodings

func DecodeShift(op : bits(2)) => ShiftType
begin
    case op of
        when '00' =>  return ShiftType_LSL;
        when '01' =>  return ShiftType_LSR;
        when '10' =>  return ShiftType_ASR;
        when '11' =>  return ShiftType_ROR;
    end;
end;
```

Library pseudocode for aarch64/functions/shiftreg/ShiftReg

```
// ShiftReg()
// ==========
// Perform shift of a register operand

func ShiftReg{N}(reg : integer, shiftype : ShiftType, amount : integer) => bits(N)
begin
    var result : bits(N) = X{}(reg);
    case shiftype of
        when ShiftType_LSL => result = LSL(result, amount);
        when ShiftType_LSR => result = LSR(result, amount);
        when ShiftType_ASR => result = ASR(result, amount);
        when ShiftType_ROR => result = ROR(result, amount);
    end;
    return result;
end;
```

Library pseudocode for aarch64/functions/shiftreg/ShiftType

```
// ShiftType
// =========
// AArch64 register shifts.

type ShiftType of enumeration {ShiftType_LSL, ShiftType_LSR, ShiftType_ASR, ShiftType_ROR};
```

Library pseudocode for aarch64/functions/sme/CounterToPredicate

```
// CounterToPredicate()
// ====================

func CounterToPredicate{width}(pred : bits(16)) => bits(width)
begin
    var count : integer;
    var esize : ESize;
    var elements : integer;
    let VL : VecLen  = CurrentVL();
    let PL : PredLen = VL DIV 8;
    let maxbit : integer{} = FloorLog2(PL * 4) as integer{0..14};
    var result : bits(PL*4);
    let invert : boolean = pred[15] == '1';

    assert width == PL || width == PL*2 || width == PL*3 || width == PL*4;

    case pred[3:0] of
        when '0000' =>
            return Zeros{width};
        when 'xxx1' =>
            count = UInt(pred[maxbit:1]);
            esize = 8;
        when 'xx10' =>
            count = UInt(pred[maxbit:2]);
            esize = 16;
        when 'x100' =>
            count = UInt(pred[maxbit:3]);
            esize = 32;
        when '1000' =>
            count = UInt(pred[maxbit:4]);
            esize = 64;
    end;

    elements = (VL * 4) DIV esize;
    result = Zeros{PL*4};
    let psize : integer{} = esize DIV 8;
    for e = 0 to elements-1 do
        var pbit : bit = if e < count then '1' else '0';
        if invert then
            pbit = NOT(pbit);
        end;
        result[e*:psize] = ZeroExtend{psize}(pbit);
    end;

    return result[width-1:0];
end;
```

Library pseudocode for aarch64/functions/sme/EncodePredCount

```
// EncodePredCount()
// =================

func EncodePredCount{width}(esize : ESize, elements : integer,
                            count_in : integer, invert_in : boolean) => bits(width)
begin
    var count : integer  = count_in;
    var invert : boolean = invert_in;
    let PL : PredLen = CurrentVL() DIV 8;
    assert width == PL;
    assert count >=0 && count <= elements;
    var pred : bits(16);

    if count == 0 then
        return Zeros{width};
    end;

    if invert then
        count = elements - count;
    elsif count == elements then
        count = 0;
        invert = TRUE;
    end;

    let inv : bit = (if invert then '1' else '0');
    case esize of
        when 8 =>  pred = inv :: count[13:0] ::    '1';
        when 16 => pred = inv :: count[12:0] ::   '10';
        when 32 => pred = inv :: count[11:0] ::  '100';
        when 64 => pred = inv :: count[10:0] :: '1000';
    end;

    return ZeroExtend{width}(pred);
end;
```

Library pseudocode for aarch64/functions/sme/Lookup

```
// Lookup Table
// ============

var _ZT0 : bits(ZT0_LEN);
```

Library pseudocode for aarch64/functions/sme/PredCountTest

```
// PredCountTest()
// ===============

func PredCountTest(elements : integer, count : integer, invert : boolean) => bits(4)
begin
    var n, z, c, v : bit;
    z = (if count == 0 then '1' else '0');              // none active
    if !invert then
        n = (if count != 0 then '1' else '0');          // first active
        c = (if count == elements then '0' else '1');   // NOT last active
    else
        n = (if count == elements then '1' else '0');   // first active
        c = (if count != 0 then '0' else '1');          // NOT last active
    end;
    v = '0';

    return n::z::c::v;
end;
```

Library pseudocode for aarch64/functions/sme/System

```
// System Registers
// ================

var _ZA : array [[256]] of bits(MAX_VL);
```

Library pseudocode for aarch64/functions/sme/ZAhslice

```
// ZAhslice - accessor
// ===================

accessor ZAhslice{width}(tile : integer, esize : ESize, slice : integer) <=> value : bits(width)
begin
    getter
        let tiles : integer = esize DIV 8;
        assert tile >= 0 && tile < tiles;
        let slices : integer = CurrentSVL() DIV esize;
        assert slice >= 0 && slice < slices;

        return ZAvector{width}(tile + slice * tiles);
    end;
    setter
        let tiles : integer = esize DIV 8;
        assert tile >= 0 && tile < tiles;
        let slices : integer = CurrentSVL() DIV esize;
        assert slice >= 0 && slice < slices;

        ZAvector{width}(tile + slice * tiles) = value;
    end;
end;
```

Library pseudocode for aarch64/functions/sme/ZAslice

```
// ZAslice - accessor
// ==================

accessor ZAslice{width}(tile : integer, esize : ESize,
                        vertical : boolean, slice : integer) <=> value : bits(width)
begin
    getter
        var result : bits(width);

        if vertical then
            result = ZAvslice{width}(tile, esize, slice);
        else
            result = ZAhslice{width}(tile, esize, slice);
        end;

        return result;
    end;
    setter
        if vertical then
            ZAvslice{width}(tile, esize, slice) = value;
        else
            ZAhslice{width}(tile, esize, slice) = value;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sme/ZAtile

```
// ZAtile - accessor
// =================

accessor ZAtile{width}(tile : integer, esize : ESize) <=> value : bits(width)
begin
    getter
        let SVL : VecLen = CurrentSVL();
        let slices : integer = SVL DIV esize;
        assert width == SVL * slices;
        var result : bits(width);

        for slice = 0 to slices-1 do
            result[slice*:SVL] = ZAhslice{SVL}(tile, esize, slice);
        end;

        return result;
    end;
    setter
        let SVL : VecLen = CurrentSVL();
        let slices : integer = SVL DIV esize;
        assert width == SVL * slices;

        for slice = 0 to slices-1 do
            ZAhslice{SVL}(tile, esize, slice) = value[slice*:SVL];
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sme/ZAvector

```
// ZAvector - accessor
// ===================

accessor ZAvector{width}(index : integer) <=> value : bits(width)
begin
    getter
        assert width == CurrentSVL();
        assert index >= 0 && index < (width DIV 8);

        return _ZA[[index]][width-1:0];
    end;

    setter
        assert width == CurrentSVL();
        assert index >= 0 && index < (width DIV 8);

        if ConstrainUnpredictableBool(Unpredictable_SMEZEROUPPER) then
            _ZA[[index]] = ZeroExtend{MAX_VL}(value);
        else
            _ZA[[index]][width-1:0] = value;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sme/ZAvslice

```
// ZAvslice - accessor
// ===================

accessor ZAvslice{width}(tile : integer, esize : ESize, slice : integer) <=> value : bits(width)
begin
    getter
        let slices : integer = CurrentSVL() DIV esize;
        var result : bits(width);

        for s = 0 to slices-1 do
            let hslice : bits(width) = ZAhslice{}(tile, esize, s);
            result[s*:esize] = hslice[slice*:esize];
        end;

        return result;
    end;

    setter
        let slices : integer = CurrentSVL() DIV esize;

        for s = 0 to slices-1 do
            var hslice : bits(width) = ZAhslice{}(tile, esize, s);
            hslice[slice*:esize] = value[s*:esize];
            ZAhslice{width}(tile, esize, s) = hslice;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sme/ZT0

```
// ZT0 - accessor
// ==============

accessor ZT0{width}() <=> value : bits(width)
begin
    getter
        assert width == ZT0_LEN;
        return _ZT0[width-1:0];
    end;
    setter
        assert width == ZT0_LEN;
        _ZT0[width-1:0] = value;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/AArch32_IsFPEnabled

```
// AArch32_IsFPEnabled()
// =====================
// Returns TRUE if access to the SIMD&FP instructions or System registers are
// enabled at the target exception level in AArch32 state and FALSE otherwise.

func AArch32_IsFPEnabled(el : bits(2)) => boolean
begin
    if el == EL0 && !ELUsingAArch32(EL1) then
        return AArch64_IsFPEnabled(el);
    end;

    if HaveEL(EL3) && ELUsingAArch32(EL3) && CurrentSecurityState() == SS_NonSecure then
        // Check if access disabled in NSACR
        if NSACR().cp10 == '0' then return FALSE; end;
    end;

    if el IN {EL0, EL1} then
        // Check if access disabled in CPACR
        var disabled : boolean;
        case CPACR().cp10 of
            when '00' => disabled = TRUE;
            when '01' => disabled = el == EL0;
            when '10' => disabled = ConstrainUnpredictableBool(Unpredictable_RESCPACR);
            when '11' => disabled = FALSE;
        end;
        if disabled then return FALSE; end;
    end;

    if el IN {EL0, EL1, EL2} && EL2Enabled() then
        if !ELUsingAArch32(EL2) then
            return AArch64_IsFPEnabled(EL2);
        end;
        if HCPTR().TCP10 == '1' then return FALSE; end;
    end;

    if HaveEL(EL3) && !ELUsingAArch32(EL3) then
        // Check if access disabled in CPTR_EL3
        if CPTR_EL3().TFP == '1' then return FALSE; end;
    end;

    return TRUE;
end;
```

Library pseudocode for aarch64/functions/sve/AArch64_IsFPEnabled

```
// AArch64_IsFPEnabled()
// =====================
// Returns TRUE if access to the SIMD&FP instructions or System registers are
// enabled at the target exception level in AArch64 state and FALSE otherwise.

func AArch64_IsFPEnabled(el : bits(2)) => boolean
begin
    // Check if access disabled in CPACR_EL1
    if el IN {EL0, EL1} && !IsInHost() then
        // Check SIMD&FP at EL0/EL1
        var disabled : boolean;
        case CPACR_EL1().FPEN of
            when 'x0' => disabled = TRUE;
            when '01' => disabled = el == EL0;
            when '11' => disabled = FALSE;
        end;
        if disabled then return FALSE; end;
    end;

    // Check if access disabled in CPTR_EL2
    if el IN {EL0, EL1, EL2} && EL2Enabled() then
        if ELIsInHost(EL2) then
            var disabled : boolean;
            case CPTR_EL2().FPEN of
                when 'x0' => disabled = TRUE;
                when '01' => disabled = el == EL0 && HCR_EL2().TGE == '1';
                when '11' => disabled = FALSE;
            end;
            if disabled then return FALSE; end;
        else
            if CPTR_EL2().TFP == '1' then return FALSE; end;
        end;
    end;

    // Check if access disabled in CPTR_EL3
    if HaveEL(EL3) then
        if CPTR_EL3().TFP == '1' then return FALSE; end;
    end;

    return TRUE;
end;
```

Library pseudocode for aarch64/functions/sve/ActivePredicateElement

```
// ActivePredicateElement()
// ========================
// Returns TRUE if the predicate bit is 1 and FALSE otherwise

func ActivePredicateElement{N}(pred : bits(N), e : integer, esize : integer) => boolean
begin
    assert esize IN {8, 16, 32, 64, 128};
    let n : integer = e * (esize DIV 8);
    assert n >= 0 && n < N;
    return pred[n] == '1';
end;
```

Library pseudocode for aarch64/functions/sve/AllElementsActive

```
// AllElementsActive()
// ===================
// Return TRUE if all the elements are active in the mask. Otherwise,
// return FALSE.

func AllElementsActive{N}(mask : bits(N), esize : integer) => boolean
begin
    let elements : integer = N DIV (esize DIV 8);
    var active : integer = 0;
    for e = 0 to elements-1 do
        if ActivePredicateElement{N}(mask, e, esize) then active = active + 1; end;
    end;
    return active == elements;
end;
```

Library pseudocode for aarch64/functions/sve/AnyActiveElement

```
// AnyActiveElement()
// ==================
// Return TRUE if there is at least one active element in mask. Otherwise,
// return FALSE.

func AnyActiveElement{N}(mask : bits(N), esize : integer) => boolean
begin
    return LastActiveElement{N}(mask, esize) >= 0;
end;
```

Library pseudocode for aarch64/functions/sve/BitDeposit

```
// BitDeposit()
// ============
// Deposit the least significant bits from DATA into result positions
// selected by nonzero bits in MASK, setting other result bits to zero.

func BitDeposit{N}(data : bits(N), mask : bits(N)) => bits(N)
begin
    var res : bits(N) = Zeros{};
    var db : integer = 0;
    for rb = 0 to N-1 do
        if mask[rb] == '1' then
            res[rb] = data[db];
            db      = db + 1;
        end;
    end;
    return res;
end;
```

Library pseudocode for aarch64/functions/sve/BitExtract

```
// BitExtract()
// ============
// Extract and pack DATA bits selected by the nonzero bits in MASK into
// the least significant result bits, setting other result bits to zero.

func BitExtract{N}(data : bits(N), mask : bits(N)) => bits(N)
begin
    var res : bits(N) = Zeros{};
    var rb : integer = 0;
    for db = 0 to N-1 do
        if mask[db] == '1' then
            res[rb] = data[db];
            rb      = rb + 1;
        end;
    end;
    return res;
end;
```

Library pseudocode for aarch64/functions/sve/BitGroup

```
// BitGroup()
// ==========
// Extract and pack DATA bits selected by the nonzero bits in MASK into
// the least significant result bits, and pack unselected bits into the
// most significant result bits.

func BitGroup{N}(data : bits(N), mask : bits(N)) => bits(N)
begin
    var res : bits(N);
    var rb : integer = 0;

    // compress masked bits to right
    for db = 0 to N-1 do
        if mask[db] == '1' then
            res[rb] = data[db];
            rb      = rb + 1;
        end;
    end;
    // compress unmasked bits to left
    for db = 0 to N-1 do
        if mask[db] == '0' then
            res[rb] = data[db];
            rb      = rb + 1;
        end;
    end;
    return res;
end;
```

Library pseudocode for aarch64/functions/sve/CheckNonStreamingSVEEnabled

```
// CheckNonStreamingSVEEnabled()
// =============================
// Checks for traps on SVE instructions that are not legal when executed in Streaming mode.

func CheckNonStreamingSVEEnabled()
begin
    CheckSVEEnabled();

    if IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' && !IsFullA64Enabled() then
        SMEAccessTrap(SMEExceptionType_Streaming, PSTATE.EL);
    end;
end;
```

Library pseudocode for aarch64/functions/sve/CheckOriginalSVEEnabled

```
// CheckOriginalSVEEnabled()
// =========================
// Checks for traps on SVE instructions and instructions that access SVE System
// registers.

func CheckOriginalSVEEnabled()
begin
    assert IsFeatureImplemented(FEAT_SVE);
    var disabled : boolean;

    if (HaveEL(EL3) && (CPTR_EL3().EZ == '0' || CPTR_EL3().TFP == '1') &&
          EL3SDDUndefPriority()) then
        Undefined();
    end;

    // Check if access disabled in CPACR_EL1
    if PSTATE.EL IN {EL0, EL1} && !IsInHost() then
        // Check SVE at EL0/EL1
        case CPACR_EL1().ZEN of
            when 'x0' => disabled = TRUE;
            when '01' => disabled = PSTATE.EL == EL0;
            when '11' => disabled = FALSE;
        end;
        if disabled then SVEAccessTrap(EL1); end;

        // Check SIMD&FP at EL0/EL1
        case CPACR_EL1().FPEN of
            when 'x0' => disabled = TRUE;
            when '01' => disabled = PSTATE.EL == EL0;
            when '11' => disabled = FALSE;
        end;
        if disabled then AArch64_AdvSIMDFPAccessTrap(EL1); end;
    end;

    // Check if access disabled in CPTR_EL2
    if PSTATE.EL IN {EL0, EL1, EL2} && EL2Enabled() then
        if ELIsInHost(EL2) then
            // Check SVE at EL2
            case CPTR_EL2().ZEN of
                when 'x0' => disabled = TRUE;
                when '01' => disabled = PSTATE.EL == EL0 && HCR_EL2().TGE == '1';
                when '11' => disabled = FALSE;
            end;
            if disabled then SVEAccessTrap(EL2); end;

            // Check SIMD&FP at EL2
            case CPTR_EL2().FPEN of
                when 'x0' => disabled = TRUE;
                when '01' => disabled = PSTATE.EL == EL0 && HCR_EL2().TGE == '1';
                when '11' => disabled = FALSE;
            end;
            if disabled then AArch64_AdvSIMDFPAccessTrap(EL2); end;
        else
            if CPTR_EL2().TZ == '1'  then SVEAccessTrap(EL2); end;
            if CPTR_EL2().TFP == '1' then AArch64_AdvSIMDFPAccessTrap(EL2); end;
        end;
    end;

    // Check if access disabled in CPTR_EL3
    if HaveEL(EL3) then
        if CPTR_EL3().EZ == '0' then
            if EL3SDDUndef() then Undefined(); end;
            SVEAccessTrap(EL3);
        end;

        if CPTR_EL3().TFP == '1' then
            if EL3SDDUndef() then Undefined(); end;
            AArch64_AdvSIMDFPAccessTrap(EL3);
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/CheckSMEAccess

```
// CheckSMEAccess()
// ================
// Check that access to SME System registers is enabled.

func CheckSMEAccess()
begin
    var disabled : boolean;

    if HaveEL(EL3) && CPTR_EL3().ESM == '0' && EL3SDDUndefPriority() then
        Undefined();
    end;

    // Check if access disabled in CPACR_EL1
    if PSTATE.EL IN {EL0, EL1} && !IsInHost() then
        // Check SME at EL0/EL1
        case CPACR_EL1().SMEN of
            when 'x0' => disabled = TRUE;
            when '01' => disabled = EffectivelyAtEL0NotInHost();
            when '11' => disabled = FALSE;
        end;
        if disabled then SMEAccessTrap(SMEExceptionType_AccessTrap, EL1); end;
    end;

    if PSTATE.EL IN {EL0, EL1, EL2} && EL2Enabled() then
        if ELIsInHost(EL2) then
            // Check SME at EL2
            case CPTR_EL2().SMEN of
                when 'x0' => disabled = TRUE;
                when '01' => disabled = EffectivelyAtEL0InHost();
                when '11' => disabled = FALSE;
            end;
            if disabled then SMEAccessTrap(SMEExceptionType_AccessTrap, EL2); end;
        else
            if CPTR_EL2().TSM == '1' then SMEAccessTrap(SMEExceptionType_AccessTrap, EL2); end;
        end;
    end;

    // Check if access disabled in CPTR_EL3
    if HaveEL(EL3) then
        if CPTR_EL3().ESM == '0' then
            if EL3SDDUndef() then Undefined(); end;
            SMEAccessTrap(SMEExceptionType_AccessTrap, EL3);
        end;
    end;

end;
```

Library pseudocode for aarch64/functions/sve/CheckSMEAndZAEnabled

```
// CheckSMEAndZAEnabled()
// ======================

func CheckSMEAndZAEnabled()
begin
    CheckSMEEnabled();

    if PSTATE.ZA == '0' then
        SMEAccessTrap(SMEExceptionType_InactiveZA, PSTATE.EL);
    end;
end;
```

Library pseudocode for aarch64/functions/sve/CheckSMEEnabled

```
// CheckSMEEnabled()
// =================

func CheckSMEEnabled()
begin
    var disabled : boolean;

    if HaveEL(EL3) && CPTR_EL3().[ESM,TFP] != '10' && EL3SDDUndefPriority() then
        Undefined();
    end;

    // Check if access disabled in CPACR_EL1
    if PSTATE.EL IN {EL0, EL1} && !IsInHost() then
        // Check SME at EL0/EL1
        case CPACR_EL1().SMEN of
            when 'x0' => disabled = TRUE;
            when '01' => disabled = PSTATE.EL == EL0;
            when '11' => disabled = FALSE;
        end;
        if disabled then SMEAccessTrap(SMEExceptionType_AccessTrap, EL1); end;

        // Check SIMD&FP at EL0/EL1
        case CPACR_EL1().FPEN of
            when 'x0' => disabled = TRUE;
            when '01' => disabled = PSTATE.EL == EL0;
            when '11' => disabled = FALSE;
        end;
        if disabled then AArch64_AdvSIMDFPAccessTrap(EL1); end;
    end;

    if PSTATE.EL IN {EL0, EL1, EL2} && EL2Enabled() then
        if ELIsInHost(EL2) then
            // Check SME at EL2
            case CPTR_EL2().SMEN of
                when 'x0' => disabled = TRUE;
                when '01' => disabled = PSTATE.EL == EL0 && HCR_EL2().TGE == '1';
                when '11' => disabled = FALSE;
            end;
            if disabled then SMEAccessTrap(SMEExceptionType_AccessTrap, EL2); end;

            // Check SIMD&FP at EL2
            case CPTR_EL2().FPEN of
                when 'x0' => disabled = TRUE;
                when '01' => disabled = PSTATE.EL == EL0 && HCR_EL2().TGE == '1';
                when '11' => disabled = FALSE;
            end;
            if disabled then AArch64_AdvSIMDFPAccessTrap(EL2); end;
        else
            if CPTR_EL2().TSM == '1' then SMEAccessTrap(SMEExceptionType_AccessTrap, EL2); end;
            if CPTR_EL2().TFP == '1' then AArch64_AdvSIMDFPAccessTrap(EL2); end;
        end;
    end;

    // Check if access disabled in CPTR_EL3
    if HaveEL(EL3) then
        if CPTR_EL3().ESM == '0' then
            if EL3SDDUndef() then Undefined(); end;
            SMEAccessTrap(SMEExceptionType_AccessTrap, EL3);
        end;

        if CPTR_EL3().TFP == '1' then
            if EL3SDDUndef() then Undefined(); end;
            AArch64_AdvSIMDFPAccessTrap(EL3);
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/CheckSMEZT0Enabled

```
// CheckSMEZT0Enabled()
// ===================
// Checks for ZT0 enabled.

func CheckSMEZT0Enabled()
begin
    if HaveEL(EL3) && SMCR_EL3().EZT0 == '0' && EL3SDDUndefPriority() then
        Undefined();
    end;

    // Check if ZA and ZT0 are inactive in PSTATE
    if PSTATE.ZA == '0' then
        SMEAccessTrap(SMEExceptionType_InactiveZA, PSTATE.EL);
    end;

    // Check if EL0/EL1 accesses to ZT0 are disabled in SMCR_EL1
    if PSTATE.EL IN {EL0, EL1} && !IsInHost() then
        if SMCR_EL1().EZT0 == '0' then
            SMEAccessTrap(SMEExceptionType_InaccessibleZT0, EL1);
        end;
    end;

    // Check if EL0/EL1/EL2 accesses to ZT0 are disabled in SMCR_EL2
    if PSTATE.EL IN {EL0, EL1, EL2} && EL2Enabled() then
        if SMCR_EL2().EZT0 == '0' then
            SMEAccessTrap(SMEExceptionType_InaccessibleZT0, EL2);
        end;
    end;

    // Check if all accesses to ZT0 are disabled in SMCR_EL3
    if HaveEL(EL3) then
        if SMCR_EL3().EZT0 == '0' then
            if EL3SDDUndef() then Undefined(); end;
            SMEAccessTrap(SMEExceptionType_InaccessibleZT0, EL3);
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/CheckSVEEnabled

```
// CheckSVEEnabled()
// =================
// Checks for traps on SVE instructions and instructions that
// access SVE System registers.

func CheckSVEEnabled()
begin
    if IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then
        CheckSMEEnabled();
    elsif IsFeatureImplemented(FEAT_SME) && !IsFeatureImplemented(FEAT_SVE) then
        CheckStreamingSVEEnabled();
    else
        CheckOriginalSVEEnabled();
    end;
end;
```

Library pseudocode for aarch64/functions/sve/CheckStreamingSVEAndZAEnabled

```
// CheckStreamingSVEAndZAEnabled()
// ===============================

func CheckStreamingSVEAndZAEnabled()
begin
    CheckStreamingSVEEnabled();

    if PSTATE.ZA == '0' then
        SMEAccessTrap(SMEExceptionType_InactiveZA, PSTATE.EL);
    end;
end;
```

Library pseudocode for aarch64/functions/sve/CheckStreamingSVEEnabled

```
// CheckStreamingSVEEnabled()
// ==========================

func CheckStreamingSVEEnabled()
begin
    CheckSMEEnabled();

    if PSTATE.SM == '0' then
        SMEAccessTrap(SMEExceptionType_NotStreaming, PSTATE.EL);
    end;
end;
```

Library pseudocode for aarch64/functions/sve/CmpOp

```
// CmpOp
// =====

type CmpOp of enumeration { Cmp_EQ, Cmp_NE, Cmp_GE, Cmp_GT, Cmp_LT, Cmp_LE, Cmp_UN };
```

Library pseudocode for aarch64/functions/sve/CurrentNSVL

```
// CurrentNSVL
// ===========
// Non-Streaming VL

readonly func CurrentNSVL() => VecLen
begin
    var vl : integer;

    if PSTATE.EL == EL1 || (PSTATE.EL == EL0 && !IsInHost()) then
        vl = UInt(ZCR_EL1().LEN);
    end;

    if PSTATE.EL == EL2 || (PSTATE.EL == EL0 && IsInHost()) then
        vl = UInt(ZCR_EL2().LEN);
    elsif PSTATE.EL IN {EL0, EL1} && EL2Enabled() then
        vl = Min(vl, UInt(ZCR_EL2().LEN));
    end;

    if PSTATE.EL == EL3 then
        vl = UInt(ZCR_EL3().LEN);
    elsif HaveEL(EL3) then
        vl = Min(vl, UInt(ZCR_EL3().LEN));
    end;

    return ImplementedSVEVectorLength((vl + 1) * 128);
end;
```

Library pseudocode for aarch64/functions/sve/CurrentSVL

```
// CurrentSVL
// ==========
// Streaming SVL

readonly func CurrentSVL() => VecLen
begin
    var vl : integer;

    if PSTATE.EL == EL1 || (PSTATE.EL == EL0 && !IsInHost()) then
        vl = UInt(SMCR_EL1().LEN);
    end;

    if PSTATE.EL == EL2 || (PSTATE.EL == EL0 && IsInHost()) then
        vl = UInt(SMCR_EL2().LEN);
    elsif PSTATE.EL IN {EL0, EL1} && EL2Enabled() then
        vl = Min(vl, UInt(SMCR_EL2().LEN));
    end;

    if PSTATE.EL == EL3 then
        vl = UInt(SMCR_EL3().LEN);
    elsif HaveEL(EL3) then
        vl = Min(vl, UInt(SMCR_EL3().LEN));
    end;

    return ImplementedSMEVectorLength((vl + 1) * 128);
end;
```

Library pseudocode for aarch64/functions/sve/CurrentVL

```
// CurrentVL
// =========

readonly func CurrentVL() => VecLen
begin
    return (if IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then CurrentSVL()
                                                                  else CurrentNSVL());
end;
```

Library pseudocode for aarch64/functions/sve/DecodePredCount

```
// DecodePredCount()
// =================

func DecodePredCount(bitpattern : bits(5), esize : integer) => integer
begin
    let elements : integer = CurrentVL() DIV esize;
    var numElem : integer;
    case bitpattern of
        when '00000' => numElem = FloorPow2(elements);
        when '00001' => numElem = if elements >= 1 then 1 else 0;
        when '00010' => numElem = if elements >= 2 then 2 else 0;
        when '00011' => numElem = if elements >= 3 then 3 else 0;
        when '00100' => numElem = if elements >= 4 then 4 else 0;
        when '00101' => numElem = if elements >= 5 then 5 else 0;
        when '00110' => numElem = if elements >= 6 then 6 else 0;
        when '00111' => numElem = if elements >= 7 then 7 else 0;
        when '01000' => numElem = if elements >= 8 then 8 else 0;
        when '01001' => numElem = if elements >= 16 then 16 else 0;
        when '01010' => numElem = if elements >= 32 then 32 else 0;
        when '01011' => numElem = if elements >= 64 then 64 else 0;
        when '01100' => numElem = if elements >= 128 then 128 else 0;
        when '01101' => numElem = if elements >= 256 then 256 else 0;
        when '11101' => numElem = elements - (elements MOD 4);
        when '11110' => numElem = elements - (elements MOD 3);
        when '11111' => numElem = elements;
        otherwise    => numElem = 0;
    end;
    return numElem;
end;
```

Library pseudocode for aarch64/functions/sve/ElemFFR

```
// ElemFFR - accessor
// ==================

accessor ElemFFR(e : integer, esize : ESize) <=> value : bit
begin
    getter
        return PredicateElement{MAX_PL}(_FFR, e, esize);
    end;
    setter
        let psize : integer{} = esize DIV 8;
        _FFR[e*:psize] = ZeroExtend{psize}(value);
    end;
end;
```

Library pseudocode for aarch64/functions/sve/FFR

```
// FFR - accessor
// ==============

accessor FFR{width}() <=> value : bits(width)
begin
    getter
        assert width == CurrentVL() DIV 8;
        return _FFR[width-1:0];
    end;

    setter
        assert width == CurrentVL() DIV 8;
        if ConstrainUnpredictableBool(Unpredictable_SVEZEROUPPER) then
            _FFR = ZeroExtend{MAX_PL}(value);
        else
            _FFR[width-1:0] = value;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/FPCompareNE

```
// FPCompareNE()
// =============

func FPCompareNE{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => boolean
begin
    assert N IN {16,32,64};
    var result : boolean;
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);
    let op1_nan : boolean = type1 IN {FPType_SNaN, FPType_QNaN};
    let op2_nan : boolean = type2 IN {FPType_SNaN, FPType_QNaN};

    if op1_nan || op2_nan then
        result = TRUE;
        if type1 == FPType_SNaN || type2 == FPType_SNaN then
            FPProcessException(FPExc_InvalidOp, fpcr);
        end;
    else // All non-NaN cases can be evaluated on the values produced by FPUnpack()
        result = (value1 != value2);
        FPProcessDenorms(type1, type2, N, fpcr);
    end;
    return result;
end;
```

Library pseudocode for aarch64/functions/sve/FPCompareUN

```
// FPCompareUN()
// =============

func FPCompareUN{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => boolean
begin
    assert N IN {16,32,64};
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);

    if type1 == FPType_SNaN || type2 == FPType_SNaN then
        FPProcessException(FPExc_InvalidOp, fpcr);
    end;

    let result : boolean = (type1 IN {FPType_SNaN, FPType_QNaN} ||
                            type2 IN {FPType_SNaN, FPType_QNaN});
    if !result then
        FPProcessDenorms(type1, type2, N, fpcr);
    end;

    return result;
end;
```

Library pseudocode for aarch64/functions/sve/FPConvertSVE

```
// FPConvertSVE()
// ==============

func FPConvertSVE{M, N}(op : bits(N), fpcr_in : FPCR_Type, rounding : FPRounding) => bits(M)
begin
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.AHP = '0';
    return FPConvert{M, N}(op, fpcr, rounding);
end;

// FPConvertSVE()
// ==============

func FPConvertSVE{M, N}(op : bits(N), fpcr_in : FPCR_Type) => bits(M)
begin
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.AHP = '0';
    return FPConvert{M, N}(op, fpcr, FPRoundingMode(fpcr));
end;
```

Library pseudocode for aarch64/functions/sve/FPExpA

```
// FPExpA()
// ========

func FPExpA{N}(op : bits(N)) => bits(N)
begin
    assert N IN {16,32,64};
    var result : bits(N);
    var coeff : bits(N);
    let idx : integer = if N == 16 then UInt(op[4:0]) else UInt(op[5:0]);
    coeff = FPExpCoefficient{N}(idx);
    if N == 16 then
        result[15:0] = '0'::op[9:5]::coeff[9:0];
    elsif N == 32 then
        result[31:0] = '0'::op[13:6]::coeff[22:0];
    else // N == 64
        result[63:0] = '0'::op[16:6]::coeff[51:0];
    end;

    return result;
end;
```

Library pseudocode for aarch64/functions/sve/FPExpCoefficient

```
// FPExpCoefficient()
// ==================

func FPExpCoefficient{N}(index : integer) => bits(N)
begin
    assert N IN {16,32,64};
    var result : integer;

    if N == 16 then
        case index of
            when  0 => result = 0x000;
            when  1 => result = 0x016;
            when  2 => result = 0x02d;
            when  3 => result = 0x045;
            when  4 => result = 0x05d;
            when  5 => result = 0x075;
            when  6 => result = 0x08e;
            when  7 => result = 0x0a8;
            when  8 => result = 0x0c2;
            when  9 => result = 0x0dc;
            when 10 => result = 0x0f8;
            when 11 => result = 0x114;
            when 12 => result = 0x130;
            when 13 => result = 0x14d;
            when 14 => result = 0x16b;
            when 15 => result = 0x189;
            when 16 => result = 0x1a8;
            when 17 => result = 0x1c8;
            when 18 => result = 0x1e8;
            when 19 => result = 0x209;
            when 20 => result = 0x22b;
            when 21 => result = 0x24e;
            when 22 => result = 0x271;
            when 23 => result = 0x295;
            when 24 => result = 0x2ba;
            when 25 => result = 0x2e0;
            when 26 => result = 0x306;
            when 27 => result = 0x32e;
            when 28 => result = 0x356;
            when 29 => result = 0x37f;
            when 30 => result = 0x3a9;
            when 31 => result = 0x3d4;
        end;

    elsif N == 32 then
        case index of
            when  0 => result = 0x000000;
            when  1 => result = 0x0164d2;
            when  2 => result = 0x02cd87;
            when  3 => result = 0x043a29;
            when  4 => result = 0x05aac3;
            when  5 => result = 0x071f62;
            when  6 => result = 0x08980f;
            when  7 => result = 0x0a14d5;
            when  8 => result = 0x0b95c2;
            when  9 => result = 0x0d1adf;
            when 10 => result = 0x0ea43a;
            when 11 => result = 0x1031dc;
            when 12 => result = 0x11c3d3;
            when 13 => result = 0x135a2b;
            when 14 => result = 0x14f4f0;
            when 15 => result = 0x16942d;
            when 16 => result = 0x1837f0;
            when 17 => result = 0x19e046;
            when 18 => result = 0x1b8d3a;
            when 19 => result = 0x1d3eda;
            when 20 => result = 0x1ef532;
            when 21 => result = 0x20b051;
            when 22 => result = 0x227043;
            when 23 => result = 0x243516;
            when 24 => result = 0x25fed7;
            when 25 => result = 0x27cd94;
            when 26 => result = 0x29a15b;
            when 27 => result = 0x2b7a3a;
            when 28 => result = 0x2d583f;
            when 29 => result = 0x2f3b79;
            when 30 => result = 0x3123f6;
            when 31 => result = 0x3311c4;
            when 32 => result = 0x3504f3;
            when 33 => result = 0x36fd92;
            when 34 => result = 0x38fbaf;
            when 35 => result = 0x3aff5b;
            when 36 => result = 0x3d08a4;
            when 37 => result = 0x3f179a;
            when 38 => result = 0x412c4d;
            when 39 => result = 0x4346cd;
            when 40 => result = 0x45672a;
            when 41 => result = 0x478d75;
            when 42 => result = 0x49b9be;
```

```
            when 43 => result = 0x4bec15;
            when 44 => result = 0x4e248c;
            when 45 => result = 0x506334;
            when 46 => result = 0x52a81e;
            when 47 => result = 0x54f35b;
            when 48 => result = 0x5744fd;
            when 49 => result = 0x599d16;
            when 50 => result = 0x5bfbb8;
            when 51 => result = 0x5e60f5;
            when 52 => result = 0x60ccdf;
            when 53 => result = 0x633f89;
            when 54 => result = 0x65b907;
            when 55 => result = 0x68396a;
            when 56 => result = 0x6ac0c7;
            when 57 => result = 0x6d4f30;
            when 58 => result = 0x6fe4ba;
            when 59 => result = 0x728177;
            when 60 => result = 0x75257d;
            when 61 => result = 0x77d0df;
            when 62 => result = 0x7a83b3;
            when 63 => result = 0x7d3e0c;
        end;

    else // N == 64
        case index of
            when  0 => result = 0x0000000000000;
            when  1 => result = 0x02C9A3E778061;
            when  2 => result = 0x059B0D3158574;
            when  3 => result = 0x0874518759BC8;
            when  4 => result = 0x0B5586CF9890F;
            when  5 => result = 0x0E3EC32D3D1A2;
            when  6 => result = 0x11301D0125B51;
            when  7 => result = 0x1429AAEA92DE0;
            when  8 => result = 0x172B83C7D517B;
            when  9 => result = 0x1A35BEB6FCB75;
            when 10 => result = 0x1D4873168B9AA;
            when 11 => result = 0x2063B88628CD6;
            when 12 => result = 0x2387A6E756238;
            when 13 => result = 0x26B4565E27CDD;
            when 14 => result = 0x29E9DF51FDEE1;
            when 15 => result = 0x2D285A6E4030B;
            when 16 => result = 0x306FE0A31B715;
            when 17 => result = 0x33C08B26416FF;
            when 18 => result = 0x371A7373AA9CB;
            when 19 => result = 0x3A7DB34E59FF7;
            when 20 => result = 0x3DEA64C123422;
            when 21 => result = 0x4160A21F72E2A;
            when 22 => result = 0x44E086061892D;
            when 23 => result = 0x486A2B5C13CD0;
            when 24 => result = 0x4BFDAD5362A27;
            when 25 => result = 0x4F9B2769D2CA7;
            when 26 => result = 0x5342B569D4F82;
            when 27 => result = 0x56F4736B527DA;
            when 28 => result = 0x5AB07DD485429;
            when 29 => result = 0x5E76F15AD2148;
            when 30 => result = 0x6247EB03A5585;
            when 31 => result = 0x6623882552225;
            when 32 => result = 0x6A09E667F3BCD;
            when 33 => result = 0x6DFB23C651A2F;
            when 34 => result = 0x71F75E8EC5F74;
            when 35 => result = 0x75FEB564267C9;
            when 36 => result = 0x7A11473EB0187;
            when 37 => result = 0x7E2F336CF4E62;
            when 38 => result = 0x82589994CCE13;
            when 39 => result = 0x868D99B4492ED;
            when 40 => result = 0x8ACE5422AA0DB;
            when 41 => result = 0x8F1AE99157736;
            when 42 => result = 0x93737B0CDC5E5;
            when 43 => result = 0x97D829FDE4E50;
            when 44 => result = 0x9C49182A3F090;
            when 45 => result = 0xA0C667B5DE565;
            when 46 => result = 0xA5503B23E255D;
            when 47 => result = 0xA9E6B5579FDBF;
            when 48 => result = 0xAE89F995AD3AD;
            when 49 => result = 0xB33A2B84F15FB;
            when 50 => result = 0xB7F76F2FB5E47;
            when 51 => result = 0xBCC1E904BC1D2;
            when 52 => result = 0xC199BDD85529C;
            when 53 => result = 0xC67F12E57D14B;
            when 54 => result = 0xCB720DCEF9069;
            when 55 => result = 0xD072D4A07897C;
            when 56 => result = 0xD5818DCFBA487;
            when 57 => result = 0xDA9E603DB3285;
            when 58 => result = 0xDFC97337B9B5F;
            when 59 => result = 0xE502EE78B3FF6;
            when 60 => result = 0xEA4AFA2A490DA;
            when 61 => result = 0xEFA1BEE615A27;
            when 62 => result = 0xF50765B6E4540;
            when 63 => result = 0xFA7C1819E90D8;
```

```
        end;
    end;

    return result[N-1:0];
end;
```

Library pseudocode for aarch64/functions/sve/FPLogB

```
// FPLogB()
// ========

func FPLogB{N}(op : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var result : integer;
    var (fptype,sign,value) : (FPType, bit, real) = FPUnpack{N}(op, fpcr);

    if fptype == FPType_SNaN || fptype == FPType_QNaN || fptype == FPType_Zero then
        FPProcessException(FPExc_InvalidOp, fpcr);
        result = -(2^(N-1));            // MinInt, 100..00
    elsif fptype == FPType_Infinity then
        result = 2^(N-1) - 1;           // MaxInt, 011..11
    else
        // FPUnpack has already scaled a subnormal input
        value = Abs(value);
        (value, result) = NormalizeReal(value);

        FPProcessDenorm(fptype, N, fpcr);
    end;
    return result[N-1:0];
end;
```

Library pseudocode for aarch64/functions/sve/FPOne

```
// FPOne()
// =======

func FPOne{N}(sign : bit) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{}  = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{}  = N - (E + 1);
    let exp : bits(E)  = '0'::Ones{E-1};
    let frac : bits(F) = Zeros{};
    return sign :: exp :: frac;
end;
```

Library pseudocode for aarch64/functions/sve/FPPointFive

```
// FPPointFive()
// =============

func FPPointFive{N}(sign : bit) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{}  = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{}  = N - (E + 1);
    let exp : bits(E)  = '0'::Ones{E-2}::'0';
    let frac : bits(F) = Zeros{};
    return sign :: exp :: frac;
end;
```

Library pseudocode for aarch64/functions/sve/FPReducePredicated

```
// FPReducePredicated()
// ====================
func FPReducePredicated{esize, N, M}(op : ReduceOp, input : bits(N), mask : bits(M),
                                     identity : bits(esize), fpcr : FPCR_Type) => bits(esize)
begin
    assert(N == M * 8);
    assert IsPow2(N);
    var operand : bits(N);
    let elements : integer = N DIV esize;

    for e = 0 to elements-1 do
        if e * esize < N && ActivePredicateElement{M}(mask, e, esize) then
            operand[e*:esize] = input[e*:esize];
        else
            operand[e*:esize] = identity;
        end;
    end;

    return FPReduce{esize, N}(op, operand, fpcr);
end;
```

Library pseudocode for aarch64/functions/sve/FPTrigMAdd

```
// FPTrigMAdd()
// ============

func FPTrigMAdd{N}(x_in : integer, op1 : bits(N), op2_in : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var coeff : bits(N);
    var op2 : bits(N) = op2_in;
    var x : integer = x_in;
    assert x >= 0;
    assert x < 8;

    if op2[N-1] == '1' then
        x = x + 8;
    end;

    coeff = FPTrigMAddCoefficient{N}(x);
    // Safer to use EffectiveFPCR() in case the input fpcr argument
    // is modified as opposed to actual value of FPCR

    op2                   = FPAbs{N}(op2, EffectiveFPCR());
    let result : bits(N)  = FPMulAdd{}(coeff, op1, op2, fpcr);
    return result;
end;
```

Library pseudocode for aarch64/functions/sve/FPTrigMAddCoefficient

```
// FPTrigMAddCoefficient()
// =======================

func FPTrigMAddCoefficient{N}(index : integer) => bits(N)
begin
    assert N IN {16,32,64};
    var result : integer;

    if N == 16 then
        case index of
            when  0 => result = 0x3c00;
            when  1 => result = 0xb155;
            when  2 => result = 0x2030;
            when  3 => result = 0x0000;
            when  4 => result = 0x0000;
            when  5 => result = 0x0000;
            when  6 => result = 0x0000;
            when  7 => result = 0x0000;
            when  8 => result = 0x3c00;
            when  9 => result = 0xb800;
            when 10 => result = 0x293a;
            when 11 => result = 0x0000;
            when 12 => result = 0x0000;
            when 13 => result = 0x0000;
            when 14 => result = 0x0000;
            when 15 => result = 0x0000;
        end;
    elsif N == 32 then
        case index of
            when  0 => result = 0x3f800000;
            when  1 => result = 0xbe2aaaab;
            when  2 => result = 0x3c088886;
            when  3 => result = 0xb95008b9;
            when  4 => result = 0x36369d6d;
            when  5 => result = 0x00000000;
            when  6 => result = 0x00000000;
            when  7 => result = 0x00000000;
            when  8 => result = 0x3f800000;
            when  9 => result = 0xbf000000;
            when 10 => result = 0x3d2aaaa6;
            when 11 => result = 0xbab60705;
            when 12 => result = 0x37cd37cc;
            when 13 => result = 0x00000000;
            when 14 => result = 0x00000000;
            when 15 => result = 0x00000000;
        end;
    else // N == 64
        case index of
            when  0 => result = 0x3ff0000000000000;
            when  1 => result = 0xbfc5555555555543;
            when  2 => result = 0x3f8111111110f30c;
            when  3 => result = 0xbf2a01a019b92fc6;
            when  4 => result = 0x3ec71de351f3d22b;
            when  5 => result = 0xbe5ae5e2b60f7b91;
            when  6 => result = 0x3de5d8408868552f;
            when  7 => result = 0x0000000000000000;
            when  8 => result = 0x3ff0000000000000;
            when  9 => result = 0xbfe0000000000000;
            when 10 => result = 0x3fa5555555555536;
            when 11 => result = 0xbf56c16c16c13a0b;
            when 12 => result = 0x3efa01a019b1e8d8;
            when 13 => result = 0xbe927e4f7282f468;
            when 14 => result = 0x3e21ee96d2641b13;
            when 15 => result = 0xbda8f76380fbb401;
        end;
    end;

    return result[N-1:0];
end;
```

Library pseudocode for aarch64/functions/sve/FPTrigSMul

```
// FPTrigSMul()
// ============

func FPTrigSMul{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var result : bits(N) = FPMul{}(op1, op1, fpcr);
    let fpexc = FALSE;
    let (fptype, sign, value) : (FPType, bit, real) = FPUnpack{N}(result, fpcr, fpexc);

    if ! fptype IN {FPType_QNaN, FPType_SNaN} then
        result[N-1] = op2[0];
    end;

    return result;
end;
```

Library pseudocode for aarch64/functions/sve/FPTrigSSel

```
// FPTrigSSel()
// ============

func FPTrigSSel{N}(op1 : bits(N), op2 : bits(N)) => bits(N)
begin
    assert N IN {16,32,64};
    var result : bits(N);

    if op2[0] == '1' then
        result = FPOne{N}(op2[1]);
    elsif op2[1] == '1' then
        result = FPNeg{N}(op1, EffectiveFPCR());
    else
        result = op1;
    end;

    return result;
end;
```

Library pseudocode for aarch64/functions/sve/FirstActive

```
// FirstActive()
// =============

func FirstActive{N}(mask : bits(N), x : bits(N), esize : integer) => bit
begin
    let elements : integer = N DIV (esize DIV 8);
    for e = 0 to elements-1 do
        if ActivePredicateElement{N}(mask, e, esize) then
            return PredicateElement{N}(x, e, esize);
        end;
    end;
    return '0';
end;
```

Library pseudocode for aarch64/functions/sve/HaveSVE2FP8DOT2

```
// HaveSVE2FP8DOT2()
// =================
// Returns TRUE if SVE2 FP8 dot product to half-precision instructions
// are implemented, FALSE otherwise.

func HaveSVE2FP8DOT2() => boolean
begin
    return ((IsFeatureImplemented(FEAT_SVE2) && IsFeatureImplemented(FEAT_FP8DOT2)) ||
            IsFeatureImplemented(FEAT_SSVE_FP8DOT2));
end;
```

Library pseudocode for aarch64/functions/sve/HaveSVE2FP8DOT4

```
// HaveSVE2FP8DOT4()
// =================
// Returns TRUE if SVE2 FP8 dot product to single-precision instructions
// are implemented, FALSE otherwise.

func HaveSVE2FP8DOT4() => boolean
begin
    return ((IsFeatureImplemented(FEAT_SVE2) && IsFeatureImplemented(FEAT_FP8DOT4)) ||
            IsFeatureImplemented(FEAT_SSVE_FP8DOT4));
end;
```

Library pseudocode for aarch64/functions/sve/HaveSVE2FP8FMA

```
// HaveSVE2FP8FMA()
// ================
// Returns TRUE if SVE2 FP8 multiply-accumulate to half-precision and single-precision
// instructions are implemented, FALSE otherwise.

func HaveSVE2FP8FMA() => boolean
begin
    return ((IsFeatureImplemented(FEAT_SVE2) && IsFeatureImplemented(FEAT_FP8FMA)) ||
            IsFeatureImplemented(FEAT_SSVE_FP8FMA));
end;
```

Library pseudocode for aarch64/functions/sve/ImplementedSMEVectorLength

```
// ImplementedSMEVectorLength()
// ============================
// Reduce SVE/SME vector length to a supported value (power of two)

readonly func ImplementedSMEVectorLength(nbits_in : integer) => VecLen
begin
    let maxbits : VecLen = MaxImplementedSVL();
    assert 128 <= maxbits && maxbits <= 2048 && IsPow2(maxbits);
    var nbits : integer = Min(nbits_in, maxbits);
    assert 128 <= nbits && nbits <= 2048 && AlignDownSize(nbits, 128) == nbits;

    // Search for a supported power-of-two VL less than or equal to nbits
    while nbits > 128 && !SupportedPowerTwoSVL(nbits) looplimit 7 do
        nbits = nbits - 128;
    end;

    // Return the smallest supported power-of-two VL
    while nbits < maxbits && !SupportedPowerTwoSVL(nbits) looplimit 7 do
        nbits = nbits * 2;
    end;

    return nbits as VecLen;
end;
```

Library pseudocode for aarch64/functions/sve/ImplementedSVEVectorLength

```
// ImplementedSVEVectorLength()
// ============================
// Reduce SVE vector length to a supported value (power of two)

readonly func ImplementedSVEVectorLength(nbits_in : integer) => VecLen
begin
    let maxbits : integer = MaxImplementedVL();
    assert 128 <= maxbits && maxbits <= 2048 && IsPow2(maxbits);
    var nbits : integer = Min(nbits_in, maxbits);
    assert 128 <= nbits && nbits <= 2048 && AlignDownSize(nbits, 128) == nbits;

    while !IsPow2(nbits) looplimit 7 do
        nbits = nbits - 128;
    end;
    return nbits as VecLen;
end;
```

Library pseudocode for aarch64/functions/sve/InStreamingMode

```
// InStreamingMode()
// =================

func InStreamingMode() => boolean
begin
    return IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1';
end;
```

Library pseudocode for aarch64/functions/sve/IsFPEnabled

```
// IsFPEnabled()
// =============
// Returns TRUE if accesses to the Advanced SIMD and floating-point
// registers are enabled at the target exception level in the current
// execution state and FALSE otherwise.

func IsFPEnabled(el : bits(2)) => boolean
begin
    if ELUsingAArch32(el) then
        return AArch32_IsFPEnabled(el);
    else
        return AArch64_IsFPEnabled(el);
    end;
end;
```

Library pseudocode for aarch64/functions/sve/IsFullA64Enabled

```
// IsFullA64Enabled()
// ==================
// Returns TRUE if full A64 is enabled in Streaming mode and FALSE otherwise.

func IsFullA64Enabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_SME_FA64) then return FALSE; end;

    // Check if full A64 disabled in SMCR_EL1
    if PSTATE.EL IN {EL0, EL1} && !IsInHost() then
        // Check full A64 at EL0/EL1
        if SMCR_EL1().FA64 == '0' then return FALSE; end;
    end;

    // Check if full A64 disabled in SMCR_EL2
    if PSTATE.EL IN {EL0, EL1, EL2} && EL2Enabled() then
        if SMCR_EL2().FA64 == '0' then return FALSE; end;
    end;

    // Check if full A64 disabled in SMCR_EL3
    if HaveEL(EL3) then
        if SMCR_EL3().FA64 == '0' then return FALSE; end;
    end;

    return TRUE;
end;
```

Library pseudocode for aarch64/functions/sve/IsOriginalSVEEnabled

```
// IsOriginalSVEEnabled()
// ======================
// Returns TRUE if access to SVE functionality is enabled at the target
// exception level and FALSE otherwise.

func IsOriginalSVEEnabled(el : bits(2)) => boolean
begin
    var disabled : boolean;
    if ELUsingAArch32(el) then
        return FALSE;
    end;

    // Check if access disabled in CPACR_EL1
    if el IN {EL0, EL1} && !IsInHost() then
        // Check SVE at EL0/EL1
        case CPACR_EL1().ZEN of
            when 'x0' => disabled = TRUE;
            when '01' => disabled = el == EL0;
            when '11' => disabled = FALSE;
        end;
        if disabled then return FALSE; end;
    end;

    // Check if access disabled in CPTR_EL2
    if el IN {EL0, EL1, EL2} && EL2Enabled() then
        if ELIsInHost(EL2) then
            case CPTR_EL2().ZEN of
                when 'x0' => disabled = TRUE;
                when '01' => disabled = el == EL0 && HCR_EL2().TGE == '1';
                when '11' => disabled = FALSE;
            end;
            if disabled then return FALSE; end;
        else
            if CPTR_EL2().TZ == '1' then return FALSE; end;
        end;
    end;

    // Check if access disabled in CPTR_EL3
    if HaveEL(EL3) then
        if CPTR_EL3().EZ == '0' then return FALSE; end;
    end;

    return TRUE;
end;
```

Library pseudocode for aarch64/functions/sve/IsSMEEnabled

```
// IsSMEEnabled()
// ==============
// Returns TRUE if access to SME functionality is enabled at the target
// exception level and FALSE otherwise.

func IsSMEEnabled(el : bits(2)) => boolean
begin
    var disabled : boolean;
    if ELUsingAArch32(el) then
        return FALSE;
    end;

    // Check if access disabled in CPACR_EL1
    if el IN {EL0, EL1} && !IsInHost() then
        // Check SME at EL0/EL1
        case CPACR_EL1().SMEN of
            when 'x0' => disabled = TRUE;
            when '01' => disabled = el == EL0;
            when '11' => disabled = FALSE;
        end;
        if disabled then return FALSE; end;
    end;

    // Check if access disabled in CPTR_EL2
    if el IN {EL0, EL1, EL2} && EL2Enabled() then
        if ELIsInHost(EL2) then
            case CPTR_EL2().SMEN of
                when 'x0' => disabled = TRUE;
                when '01' => disabled = el == EL0 && HCR_EL2().TGE == '1';
                when '11' => disabled = FALSE;
            end;
            if disabled then return FALSE; end;
        else
            if CPTR_EL2().TSM == '1' then return FALSE; end;
        end;
    end;

    // Check if access disabled in CPTR_EL3
    if HaveEL(EL3) then
        if CPTR_EL3().ESM == '0' then return FALSE; end;
    end;

    return TRUE;
end;
```

Library pseudocode for aarch64/functions/sve/IsSVEEnabled

```
// IsSVEEnabled()
// ==============
// Returns TRUE if access to SVE registers is enabled at the target exception
// level and FALSE otherwise.

func IsSVEEnabled(el : bits(2)) => boolean
begin
    if IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then
        return IsSMEEnabled(el);
    elsif IsFeatureImplemented(FEAT_SVE) then
        return IsOriginalSVEEnabled(el);
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/LastActive

```
// LastActive()
// ============

func LastActive{N}(mask : bits(N), x : bits(N), esize : integer) => bit
begin
    let elements : integer = N DIV (esize DIV 8);
    for e = elements-1 downto 0 do
        if ActivePredicateElement{N}(mask, e, esize) then
            return PredicateElement{N}(x, e, esize);
        end;
    end;
    return '0';
end;
```

Library pseudocode for aarch64/functions/sve/LastActiveElement

```
// LastActiveElement()
// ===================

func LastActiveElement{N}(mask : bits(N), esize : integer) => integer
begin
    let elements : integer = N DIV (esize DIV 8);
    for e = elements-1 downto 0 do
        if ActivePredicateElement{N}(mask, e, esize) then return e; end;
    end;
    return -1;
end;
```

Library pseudocode for aarch64/functions/sve/MaxImplementedAnyVL

```
// MaxImplementedAnyVL()
// =====================

readonly func MaxImplementedAnyVL() => integer
begin
    if IsFeatureImplemented(FEAT_SME) && IsFeatureImplemented(FEAT_SVE) then
        return Max(MaxImplementedVL(), MaxImplementedSVL());
    end;
    if IsFeatureImplemented(FEAT_SME) then
        return MaxImplementedSVL();
    end;
    return MaxImplementedVL();
end;
```

Library pseudocode for aarch64/functions/sve/MaxImplementedSVL

```
// MaxImplementedSVL()
// ===================

readonly func MaxImplementedSVL() => VecLen
begin
    return ImpDefInt("Max implemented SVL") as VecLen;
end;
```

Library pseudocode for aarch64/functions/sve/MaxImplementedVL

```
// MaxImplementedVL()
// ==================

readonly func MaxImplementedVL() => integer
begin
    let max_vl : integer = ImpDefInt("Max implemented VL");
    if IsFeatureImplemented(FEAT_SME_FA64) then
        let max_svl : integer = ImpDefInt("Max implemented SVL");
        return Max(max_vl, max_svl);
    end;

    return max_vl;
end;
```

Library pseudocode for aarch64/functions/sve/MaybeZeroSVEUppers

```
// MaybeZeroSVEUppers()
// ====================

func MaybeZeroSVEUppers(target_el : bits(2))
begin
    var lower_enabled : boolean;

    if UInt(target_el) <= UInt(PSTATE.EL) || !IsSVEEnabled(target_el) then
        return;
    end;

    if target_el == EL3 then
        if EL2Enabled() then
            lower_enabled = IsFPEnabled(EL2);
        else
            lower_enabled = IsFPEnabled(EL1);
        end;
    elsif target_el == EL2 then
        assert EL2Enabled() && !ELUsingAArch32(EL2);
        if HCR_EL2().TGE == '0' then
            lower_enabled = IsFPEnabled(EL1);
        else
            lower_enabled = IsFPEnabled(EL0);
        end;
    else
        assert target_el == EL1 && !ELUsingAArch32(EL1);
        lower_enabled = IsFPEnabled(EL0);
    end;

    if lower_enabled then
        let VL : integer{} = if IsSVEEnabled(PSTATE.EL) then CurrentVL() else 128;
        let PL : integer{} = VL DIV 8;
        for n = 0 to 31 do
            if ConstrainUnpredictableBool(Unpredictable_SVEZEROUPPER) then
                _Z[[n]] = ZeroExtend{MAX_VL}(_Z[[n]][VL-1:0]);
            end;
        end;
        for n = 0 to 15 do
            if ConstrainUnpredictableBool(Unpredictable_SVEZEROUPPER) then
                _P[[n]] = ZeroExtend{MAX_PL}(_P[[n]][PL-1:0]);
            end;
        end;
        if ConstrainUnpredictableBool(Unpredictable_SVEZEROUPPER) then
            _FFR = ZeroExtend{MAX_PL}(_FFR[PL-1:0]);
        end;
        if IsFeatureImplemented(FEAT_SME) && PSTATE.ZA == '1' then
            let SVL : integer{}          = CurrentSVL();
            let accessiblevecs : integer = SVL DIV 8;
            let allvecs : integer        = MaxImplementedSVL() DIV 8;

            for n = 0 to accessiblevecs - 1 do
                if ConstrainUnpredictableBool(Unpredictable_SMEZEROUPPER) then
                    _ZA[[n]] = ZeroExtend{MAX_VL}(_ZA[[n]][SVL-1:0]);
                end;
            end;
            for n = accessiblevecs to allvecs - 1 do
                if ConstrainUnpredictableBool(Unpredictable_SMEZEROUPPER) then
                    _ZA[[n]] = Zeros{MAX_VL};
                end;
            end;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/MemNF

```
// MemNF
// =====

func MemNF{size : integer{8, 16, 32, 64, 128}}(address : bits(64),
                                               accdesc : AccessDescriptor
                                              ) => (bits(size), boolean)
begin
    let bytes : integer{} = size DIV 8;
    var value : bits(size);
    var bad : boolean;

    var aligned : boolean = IsAlignedSize{64}(address, bytes);

    if !aligned && AArch64_UnalignedAccessFaults(accdesc, address, bytes) then
        return (ARBITRARY : bits(size), TRUE);
    end;

    let atomic : boolean = (aligned ||
                            (IsFeatureImplemented(FEAT_LSE2) && AllInSCAGranule(address, bytes)));

    if !atomic then
        (value[7:0], bad) = MemSingleNF{8}(address, accdesc, aligned);

        if bad then
            return (ARBITRARY : bits(size), TRUE);
        end;

        // For subsequent bytes, if they cross to a new translation page which assigns
        // Device memory type, it is CONSTRAINED UNPREDICTABLE whether an unaligned access
        // will generate an Alignment Fault.
        if !aligned then
            let c : Constraint = ConstrainUnpredictable(Unpredictable_DEVPAGE2);
            assert c IN {Constraint_FAULT, Constraint_NONE};
            if c == Constraint_NONE then aligned = TRUE; end;
        end;

        for i = 1 to bytes-1 do
            (value[i*:8], bad) = MemSingleNF{8}(address+i, accdesc, aligned);

            if bad then
                return (ARBITRARY : bits(size), TRUE);
            end;
        end;
    else
        (value, bad) = MemSingleNF{size}(address, accdesc, aligned);
        if bad then
            return (ARBITRARY : bits(size), TRUE);
        end;
    end;

    if BigEndian(accdesc.acctype) then
        value = BigEndianReverse{size}(value);
    end;

    return (value, FALSE);
end;
```

Library pseudocode for aarch64/functions/sve/MemSingleNF

```
// MemSingleNF
// ===========

func MemSingleNF{size : integer{8, 16, 32, 64, 128}}(address : bits(64),
                                                     accdesc_in : AccessDescriptor,
                                                     aligned : boolean) => (bits(size), boolean)
begin
    assert accdesc_in.acctype == AccessType_SVE;
    assert accdesc_in.nonfault || (accdesc_in.firstfault && !accdesc_in.first);

    let bytes : integer{} = size DIV 8;
    var value : bits(size);
    var memaddrdesc : AddressDescriptor;
    var memstatus : PhysMemRetStatus;
    var accdesc : AccessDescriptor = accdesc_in;
    var fault : FaultRecord = NoFault(accdesc, address);

    // Implementation may suppress NF load for any reason
    if ConstrainUnpredictableBool(Unpredictable_NONFAULT) then
        return (ARBITRARY : bits(size), TRUE);
    end;

    // If the instruction encoding permits tag checking, confer with system register configuration
    // which may override this.
    if accdesc.tagchecked then
        accdesc.tagchecked = AArch64_AccessIsTagChecked(address, accdesc);
    end;

    // MMU or MPU
    memaddrdesc = AArch64_TranslateAddress(address, accdesc, aligned, bytes);

    // Non-fault and first-fault loads from non-Conventional Device memory must not be performed
    // externally. For Conventional memory, it is IMPLEMENTATION DEFINED.
    if memaddrdesc.memattrs.memtype == MemType_Device then
        if (!IsConventionalMemory(memaddrdesc) ||
              ImpDefBool("SVE NF/FF load from Device Conventional memory is suppressed")) then
            return (ARBITRARY : bits(size), TRUE);
        end;
    end;

    // Check for aborts or debug exceptions
    if IsFault(memaddrdesc) then
        return (ARBITRARY : bits(size), TRUE);
    end;

    if accdesc.tagchecked then
        let ltag : bits(4) = AArch64_LogicalAddressTag(address);
        let readcheck : boolean = TRUE;
        fault = AArch64_CheckTag(memaddrdesc, accdesc, readcheck, bytes, ltag);
        if fault.statuscode != Fault_None then
            return (ARBITRARY : bits(size), TRUE);
        end;
    end;

    (memstatus, value) = PhysMemRead{size}(memaddrdesc, accdesc);
    if IsFault(memstatus) then
        let iswrite : boolean = FALSE;
        if IsExternalAbortTakenSynchronously(memstatus, iswrite, memaddrdesc, bytes, accdesc) then
            return (ARBITRARY : bits(size), TRUE);
        end;
        fault.merrorstate = memstatus.merrorstate;
        fault.extflag     = memstatus.extflag;
        fault.statuscode = memstatus.statuscode;
        PendSErrorInterrupt(fault);
    end;

    return (value, FALSE);
end;
```

Library pseudocode for aarch64/functions/sve/NoneActive

```
// NoneActive()
// ============

func NoneActive{N}(mask : bits(N), x : bits(N), esize : integer) => bit
begin
    let elements : integer = N DIV (esize DIV 8);
    for e = 0 to elements-1 do
        if ActivePredicateElement{N}(mask, e, esize) && ActivePredicateElement{N}(x, e, esize) then
            return '0';
        end;
    end;
    return '1';
end;
```

Library pseudocode for aarch64/functions/sve/P

```
// P - accessor
// ============

accessor P{width}(n : integer) <=> value : bits(width)
begin
    getter
        assert n >= 0 && n <= 31;
        assert width == CurrentVL() DIV 8;
        return _P[[n]][width-1:0];
    end;

    setter
        assert n >= 0 && n <= 31;
        assert width == CurrentVL() DIV 8;
        if ConstrainUnpredictableBool(Unpredictable_SVEZEROUPPER) then
            _P[[n]] = ZeroExtend{MAX_PL}(value);
        else
            _P[[n]][width-1:0] = value;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/PredLen

```
// PredLen
// =======

type PredLen of integer{16, 32, 64, 128, 256};
```

Library pseudocode for aarch64/functions/sve/PredTest

```
// PredTest()
// ==========

func PredTest{N}(mask : bits(N), result : bits(N), esize : integer) => bits(4)
begin
    let n : bit = FirstActive{N}(mask, result, esize);
    let z : bit = NoneActive{N}(mask, result, esize);
    let c : bit = NOT LastActive{N}(mask, result, esize);
    let v : bit = '0';
    return n::z::c::v;
end;
```

Library pseudocode for aarch64/functions/sve/PredicateElement

```
// PredicateElement()
// ==================
// Returns the predicate bit

func PredicateElement{N}(pred : bits(N), e : integer, esize : integer) => bit
begin
    assert esize IN {8, 16, 32, 64, 128};
    let n : integer = e * (esize DIV 8);
    assert n >= 0 && n < N;
    return pred[n];
end;
```

Library pseudocode for aarch64/functions/sve/ResetSMEState

```
// ResetSMEState()
// ===============

func ResetSMEState(newenable : bit)
begin
    let vectors : integer = MAX_VL DIV 8;
    if newenable == '1' then
        for n = 0 to vectors - 1 do
            _ZA[[n]] = Zeros{MAX_VL};
        end;
        if IsFeatureImplemented(FEAT_SME2) then
            _ZT0 = Zeros{ZT0_LEN};
        end;
    else
        for n = 0 to vectors - 1 do
            _ZA[[n]] = ARBITRARY : bits(MAX_VL);
        end;
        if IsFeatureImplemented(FEAT_SME2) then
            _ZT0 = ARBITRARY : bits(ZT0_LEN);
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/ResetSVERegisters

```
// ResetSVERegisters()
// ===================

func ResetSVERegisters()
begin
    for n = 0 to 31 do
        _Z[[n]] = ARBITRARY : bits(MAX_VL);
    end;
    for n = 0 to 15 do
        _P[[n]] = ARBITRARY : bits(MAX_PL);
    end;
    _FFR = ARBITRARY : bits(MAX_PL);
end;
```

Library pseudocode for aarch64/functions/sve/ResetSVEState

```
// ResetSVEState()
// ===============

func ResetSVEState()
begin
    for n = 0 to 31 do
        _Z[[n]] = Zeros{MAX_VL};
    end;
    for n = 0 to 15 do
        _P[[n]] = Zeros{MAX_PL};
    end;
    _FFR = Zeros{MAX_PL};
    FPSR() = ZeroExtend{64}(0x0800009f[31:0]);
    FPMR() = Zeros{64};
end;
```

Library pseudocode for aarch64/functions/sve/SMEAccessTrap

```
// SMEAccessTrap()
// ===============
// Trapped access to SME registers due to CPACR_EL1, CPTR_EL2, or CPTR_EL3.

func SMEAccessTrap(etype : SMEExceptionType, target_el_in : bits(2))
begin
    var target_el : bits(2) = target_el_in;
    assert UInt(target_el) >= UInt(PSTATE.EL);
    if target_el == EL0 then
        target_el = EL1;
    end;
    let route_to_el2 : boolean = (PSTATE.EL == EL0 && target_el == EL1 &&
                                  EL2Enabled() && HCR_EL2().TGE == '1');

    var except : ExceptionRecord              = ExceptionSyndrome(Exception_SMEAccessTrap);
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer                 = 0x0;

    case etype of
        when SMEExceptionType_AccessTrap =>
            except.syndrome.iss[2:0] = '000';
        when SMEExceptionType_Streaming =>
            except.syndrome.iss[2:0] = '001';
        when SMEExceptionType_NotStreaming =>
            except.syndrome.iss[2:0] = '010';
        when SMEExceptionType_InactiveZA =>
            except.syndrome.iss[2:0] = '011';
        when SMEExceptionType_InaccessibleZT0 =>
            except.syndrome.iss[2:0] = '100';
    end;

    if route_to_el2 then
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/functions/sve/SMEExceptionType

```
// SMEExceptionType
// ================
type SMEExceptionType of enumeration {
    SMEExceptionType_AccessTrap,        // SME functionality trapped or disabled
    SMEExceptionType_Streaming,         // Illegal instruction in Streaming SVE mode
    SMEExceptionType_NotStreaming,      // Illegal instruction not in Streaming SVE mode
    SMEExceptionType_InactiveZA,        // Illegal instruction when ZA is inactive
    SMEExceptionType_InaccessibleZT0,   // Access to ZT0 is disabled
};
```

Library pseudocode for aarch64/functions/sve/SVCR

```
// SVCR
// ====

accessor SVCR() <=> value : SVCR_Type
begin
    // Returns PSTATE.[ZA, SM]
    getter
        let value : SVCR_Type = Zeros{62} :: PSTATE.ZA :: PSTATE.SM;
        return value;
    end;

    // Sets PSTATE.[ZA, SM]
    setter
        SetPSTATE_SM(value[0]);
        SetPSTATE_ZA(value[1]);
        return;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/SVEAccessTrap

```
// SVEAccessTrap()
// ===============
// Trapped access to SVE registers due to CPACR_EL1, CPTR_EL2, or CPTR_EL3.

func SVEAccessTrap(target_el : bits(2))
begin
    assert UInt(target_el) >= UInt(PSTATE.EL) && target_el != EL0 && HaveEL(target_el);
    let route_to_el2 : boolean = target_el == EL1 && EL2Enabled() && HCR_EL2().TGE == '1';

    let except : ExceptionRecord              = ExceptionSyndrome(Exception_SVEAccessTrap);
    let preferred_exception_return : bits(64) = ThisInstrAddr{}();
    let vect_offset : integer{}               = 0x0;

    if route_to_el2 then
        AArch64_TakeException(EL2, except, preferred_exception_return, vect_offset);
    else
        AArch64_TakeException(target_el, except, preferred_exception_return, vect_offset);
    end;
end;
```

Library pseudocode for aarch64/functions/sve/SVEMoveMaskPreferred

```
// SVEMoveMaskPreferred()
// ======================
// Return FALSE if a bitmask immediate encoding would generate an immediate
// value that could also be represented by a single DUP instruction.
// Used as a condition for the preferred MOV<-DUPM alias.

func SVEMoveMaskPreferred(imm13 : bits(13)) => boolean
begin
    var imm : bits(64);
    (imm, -) = DecodeBitMasks{64}(imm13[12], imm13[5:0], imm13[11:6], TRUE);

    // Check for 8 bit immediates
    if !IsZero(imm[7:0]) then
        // Check for 'ffffffffffffffxy' or '00000000000000xy'
        if IsZero(imm[63:7]) || IsOnes(imm[63:7]) then
            return FALSE;
        end;

        // Check for 'ffffffxyffffffxy' or '000000xy000000xy'
        if imm[63:32] == imm[31:0] && (IsZero(imm[31:7]) || IsOnes(imm[31:7])) then
            return FALSE;
        end;

        // Check for 'ffxyffxyffxyffxy' or '00xy00xy00xy00xy'
        if (imm[63:32] == imm[31:0] && imm[31:16] == imm[15:0] &&
              (IsZero(imm[15:7]) || IsOnes(imm[15:7]))) then
            return FALSE;
        end;

        // Check for 'xyxyxyxyxyxyxyxy'
        if imm[63:32] == imm[31:0] && imm[31:16] == imm[15:0] && (imm[15:8] == imm[7:0]) then
            return FALSE;
        end;

    // Check for 16 bit immediates
    else
        // Check for 'ffffffffffffxy00' or '000000000000xy00'
        if IsZero(imm[63:15]) || IsOnes(imm[63:15]) then
            return FALSE;
        end;

        // Check for 'ffffxy00ffffxy00' or '0000xy000000xy00'
        if imm[63:32] == imm[31:0] && (IsZero(imm[31:15]) || IsOnes(imm[31:15])) then
            return FALSE;
        end;

        // Check for 'xy00xy00xy00xy00'
        if imm[63:32] == imm[31:0] && imm[31:16] == imm[15:0] then
            return FALSE;
        end;
    end;

    return TRUE;
end;
```

Library pseudocode for aarch64/functions/sve/SetPSTATE_SM

```
// SetPSTATE_SM()
// ==============

func SetPSTATE_SM(value : bit)
begin
    if PSTATE.SM != value then
        ResetSVEState();
        PSTATE.SM = value;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/SetPSTATE_ZA

```
// SetPSTATE_ZA()
// ==============

func SetPSTATE_ZA(value : bit)
begin
    if PSTATE.ZA != value then
        ResetSMEState(value);
        PSTATE.ZA = value;
    end;
end;
```

Library pseudocode for aarch64/functions/sve/SupportedPowerTwoSVL

```
// SupportedPowerTwoSVL()
// ======================
// Return an IMPLEMENTATION DEFINED specific value
// returns TRUE if SVL is supported and is a power of two, FALSE otherwise

readonly impdef func SupportedPowerTwoSVL(nbits : integer) => boolean
begin
    return IsPow2(nbits) && (nbits <= MAX_VL);
end;
```

Library pseudocode for aarch64/functions/sve/System

```
// System Registers
// ================

constant MAX_VL : VecLen = 2048;
constant MAX_PL : PredLen = 256;
constant ZT0_LEN : integer{} = 512;
var _Z : array [[32]] of bits(MAX_VL);
var _P : array [[16]] of bits(MAX_PL);
var _FFR : bits(MAX_PL);
```

Library pseudocode for aarch64/functions/sve/VecLen

```
// VecLen
// ======

type VecLen of integer{128, 256, 512, 1024, 2048};
```

Library pseudocode for aarch64/functions/sve/Z

```
// Z - accessor
// ============

accessor Z{width}(n : integer) <=> value : bits(width)
begin
    getter
        assert n >= 0 && n <= 31;
        assert width == CurrentVL();
        return _Z[[n]][width-1:0];
    end;
    setter
        assert n >= 0 && n <= 31;
        assert width == CurrentVL();
        if ConstrainUnpredictableBool(Unpredictable_SVEZEROUPPER) then
            _Z[[n]] = ZeroExtend{MAX_VL}(value);
        else
            _Z[[n]][width-1:0] = value;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/syshintop/SystemHintOp

```
// SystemHintOp
// ============
// System Hint instruction types.

type SystemHintOp of enumeration {
    SystemHintOp_NOP,
    SystemHintOp_YIELD,
    SystemHintOp_WFE,
    SystemHintOp_WFI,
    SystemHintOp_SEV,
    SystemHintOp_SEVL,
    SystemHintOp_DGH,
    SystemHintOp_ESB,
    SystemHintOp_PSB,
    SystemHintOp_TSB,
    SystemHintOp_BTI,
    SystemHintOp_WFET,
    SystemHintOp_WFIT,
    SystemHintOp_CLRBHB,
    SystemHintOp_GCSB,
    SystemHintOp_CHKFEAT,
    SystemHintOp_STSHH,
    SystemHintOp_SHUH,
    SystemHintOp_STCPH,
    SystemHintOp_SLBND,
    SystemHintOp_SRLS,
    SystemHintOp_CSDB
};
```

Library pseudocode for aarch64/functions/syslop/SysLOp

```
// SysLOp()
// ========

func SysLOp(op1 : bits(3), CRn : bits(4), CRm : bits(4), op2 : bits(3)) => SystemLOp
begin
    case op1::CRn::CRm::op2 of
        when '000 1100 0011 000' => return Sysl_GICR;   // CDIA
        when '000 1100 0011 001' => return Sysl_GICR;   // CDNMIA
        otherwise                => return Sysl_SYSL;
    end;
end;
```

Library pseudocode for aarch64/functions/syslop/SystemLOp

```
// SystemLOp
// =========
// System instruction with result types.

type SystemLOp of enumeration {
    Sysl_GICR,
    Sysl_SYSL
};
```

Library pseudocode for aarch64/functions/sysop/SysOp

```
// SysOp()
// =======

func SysOp(op1 : bits(3), CRn : bits(4), CRm : bits(4), op2 : bits(3)) => SystemOp
begin
    case op1::CRn::CRm::op2 of
        when '000 0111 1000 000' => return Sys_AT;    // S1E1R
        when '000 0111 1000 001' => return Sys_AT;    // S1E1W
        when '000 0111 1000 010' => return Sys_AT;    // S1E0R
        when '000 0111 1000 011' => return Sys_AT;    // S1E0W
        when '000 0111 1001 000' => return Sys_AT;    // S1E1RP
        when '000 0111 1001 001' => return Sys_AT;    // S1E1WP
        when '000 0111 1001 010' => return Sys_AT;    // S1E1A
        when '100 0111 1000 000' => return Sys_AT;    // S1E2R
        when '100 0111 1000 001' => return Sys_AT;    // S1E2W
        when '100 0111 1001 010' => return Sys_AT;    // S1E2A
        when '100 0111 1000 100' => return Sys_AT;    // S12E1R
        when '100 0111 1000 101' => return Sys_AT;    // S12E1W
        when '100 0111 1000 110' => return Sys_AT;    // S12E0R
        when '100 0111 1000 111' => return Sys_AT;    // S12E0W
        when '110 0111 1000 000' => return Sys_AT;    // S1E3R
        when '110 0111 1000 001' => return Sys_AT;    // S1E3W
        when '110 0111 1001 010' => return Sys_AT;    // S1E3A
        when '001 0111 0010 100' => return Sys_BRB;   // IALL
        when '001 0111 0010 101' => return Sys_BRB;   // INJ
        when '000 0111 0110 001' => return Sys_DC;    // IVAC
        when '000 0111 0110 010' => return Sys_DC;    // ISW
        when '000 0111 0110 011' => return Sys_DC;    // IGVAC
        when '000 0111 0110 100' => return Sys_DC;    // IGSW
        when '000 0111 0110 101' => return Sys_DC;    // IGDVAC
        when '000 0111 0110 110' => return Sys_DC;    // IGDSW
        when '000 0111 1010 010' => return Sys_DC;    // CSW
        when '000 0111 1010 100' => return Sys_DC;    // CGSW
        when '000 0111 1010 110' => return Sys_DC;    // CGDSW
        when '000 0111 1110 010' => return Sys_DC;    // CISW
        when '000 0111 1110 100' => return Sys_DC;    // CIGSW
        when '000 0111 1110 110' => return Sys_DC;    // CIGDSW
        when '011 0111 0100 001' => return Sys_DC;    // ZVA
        when '011 0111 0100 011' => return Sys_DC;    // GVA
        when '011 0111 0100 100' => return Sys_DC;    // GZVA
        when '011 0111 1010 001' => return Sys_DC;    // CVAC
        when '011 0111 1010 011' => return Sys_DC;    // CGVAC
        when '011 0111 1010 101' => return Sys_DC;    // CGDVAC
        when '011 0111 1011 001' => return Sys_DC;    // CVAU
        when '011 0111 1100 001' => return Sys_DC;    // CVAP
        when '011 0111 1100 011' => return Sys_DC;    // CGVAP
        when '011 0111 1100 101' => return Sys_DC;    // CGDVAP
        when '011 0111 1101 001' => return Sys_DC;    // CVADP
        when '011 0111 1101 011' => return Sys_DC;    // CGVADP
        when '011 0111 1101 101' => return Sys_DC;    // CGDVADP
        when '011 0111 1110 001' => return Sys_DC;    // CIVAC
        when '011 0111 1110 011' => return Sys_DC;    // CIGVAC
        when '011 0111 1110 101' => return Sys_DC;    // CIGDVAC
        when '100 0111 1110 000' => return Sys_DC;    // CIPAE
        when '100 0111 1110 111' => return Sys_DC;    // CIGDPAE
        when '110 0111 1110 001' => return Sys_DC;    // CIPAPA
        when '110 0111 1110 101' => return Sys_DC;    // CIGDPAPA
        when '000 0111 1111 001' => return Sys_DC;    // CIVAPS
        when '000 0111 1111 101' => return Sys_DC;    // CIGDVAPS
        when '000 0111 0001 000' => return Sys_IC;    // IALLUIS
        when '000 0111 0101 000' => return Sys_IC;    // IALLU
        when '011 0111 0101 001' => return Sys_IC;    // IVAU
        when '000 1000 0001 000' => return Sys_TLBI;  // VMALLE1OS
        when '000 1000 0001 001' => return Sys_TLBI;  // VAE1OS
        when '000 1000 0001 010' => return Sys_TLBI;  // ASIDE1OS
        when '000 1000 0001 011' => return Sys_TLBI;  // VAAE1OS
        when '000 1000 0001 101' => return Sys_TLBI;  // VALE1OS
        when '000 1000 0001 111' => return Sys_TLBI;  // VAALE1OS
        when '000 1000 0010 001' => return Sys_TLBI;  // RVAE1IS
        when '000 1000 0010 011' => return Sys_TLBI;  // RVAAE1IS
        when '000 1000 0010 101' => return Sys_TLBI;  // RVALE1IS
        when '000 1000 0010 111' => return Sys_TLBI;  // RVAALE1IS
        when '000 1000 0011 000' => return Sys_TLBI;  // VMALLE1IS
        when '000 1000 0011 001' => return Sys_TLBI;  // VAE1IS
        when '000 1000 0011 010' => return Sys_TLBI;  // ASIDE1IS
        when '000 1000 0011 011' => return Sys_TLBI;  // VAAE1IS
        when '000 1000 0011 101' => return Sys_TLBI;  // VALE1IS
        when '000 1000 0011 111' => return Sys_TLBI;  // VAALE1IS
        when '000 1000 0101 001' => return Sys_TLBI;  // RVAE1OS
        when '000 1000 0101 011' => return Sys_TLBI;  // RVAAE1OS
        when '000 1000 0101 101' => return Sys_TLBI;  // RVALE1OS
        when '000 1000 0101 111' => return Sys_TLBI;  // RVAALE1OS
        when '000 1000 0110 001' => return Sys_TLBI;  // RVAE1
        when '000 1000 0110 011' => return Sys_TLBI;  // RVAAE1
        when '000 1000 0110 101' => return Sys_TLBI;  // RVALE1
        when '000 1000 0110 111' => return Sys_TLBI;  // RVAALE1
        when '000 1000 0111 000' => return Sys_TLBI;  // VMALLE1
        when '000 1000 0111 001' => return Sys_TLBI;  // VAE1
        when '000 1000 0111 010' => return Sys_TLBI;  // ASIDE1
```

```
        when '000 1000 0111 011' => return Sys_TLBI;  // VAAE1
        when '000 1000 0111 101' => return Sys_TLBI;  // VALE1
        when '000 1000 0111 111' => return Sys_TLBI;  // VAALE1
        when '000 1001 0001 000' => return Sys_TLBI;  // VMALLE1OSNXS
        when '000 1001 0001 001' => return Sys_TLBI;  // VAE1OSNXS
        when '000 1001 0001 010' => return Sys_TLBI;  // ASIDE1OSNXS
        when '000 1001 0001 011' => return Sys_TLBI;  // VAAE1OSNXS
        when '000 1001 0001 101' => return Sys_TLBI;  // VALE1OSNXS
        when '000 1001 0001 111' => return Sys_TLBI;  // VAALE1OSNXS
        when '000 1001 0010 001' => return Sys_TLBI;  // RVAE1ISNXS
        when '000 1001 0010 011' => return Sys_TLBI;  // RVAAE1ISNXS
        when '000 1001 0010 101' => return Sys_TLBI;  // RVALE1ISNXS
        when '000 1001 0010 111' => return Sys_TLBI;  // RVAALE1ISNXS
        when '000 1001 0011 000' => return Sys_TLBI;  // VMALLE1ISNXS
        when '000 1001 0011 001' => return Sys_TLBI;  // VAE1ISNXS
        when '000 1001 0011 010' => return Sys_TLBI;  // ASIDE1ISNXS
        when '000 1001 0011 011' => return Sys_TLBI;  // VAAE1ISNXS
        when '000 1001 0011 101' => return Sys_TLBI;  // VALE1ISNXS
        when '000 1001 0011 111' => return Sys_TLBI;  // VAALE1ISNXS
        when '000 1001 0101 001' => return Sys_TLBI;  // RVAE1OSNXS
        when '000 1001 0101 011' => return Sys_TLBI;  // RVAAE1OSNXS
        when '000 1001 0101 101' => return Sys_TLBI;  // RVALE1OSNXS
        when '000 1001 0101 111' => return Sys_TLBI;  // RVAALE1OSNXS
        when '000 1001 0110 001' => return Sys_TLBI;  // RVAE1NXS
        when '000 1001 0110 011' => return Sys_TLBI;  // RVAAE1NXS
        when '000 1001 0110 101' => return Sys_TLBI;  // RVALE1NXS
        when '000 1001 0110 111' => return Sys_TLBI;  // RVAALE1NXS
        when '000 1001 0111 000' => return Sys_TLBI;  // VMALLE1NXS
        when '000 1001 0111 001' => return Sys_TLBI;  // VAE1NXS
        when '000 1001 0111 010' => return Sys_TLBI;  // ASIDE1NXS
        when '000 1001 0111 011' => return Sys_TLBI;  // VAAE1NXS
        when '000 1001 0111 101' => return Sys_TLBI;  // VALE1NXS
        when '000 1001 0111 111' => return Sys_TLBI;  // VAALE1NXS
        when '100 1000 0000 001' => return Sys_TLBI;  // IPAS2E1IS
        when '100 1000 0000 010' => return Sys_TLBI;  // RIPAS2E1IS
        when '100 1000 0000 101' => return Sys_TLBI;  // IPAS2LE1IS
        when '100 1000 0000 110' => return Sys_TLBI;  // RIPAS2LE1IS
        when '100 1000 0001 000' => return Sys_TLBI;  // ALLE2OS
        when '100 1000 0001 001' => return Sys_TLBI;  // VAE2OS
        when '100 1000 0001 100' => return Sys_TLBI;  // ALLE1OS
        when '100 1000 0001 101' => return Sys_TLBI;  // VALE2OS
        when '100 1000 0001 110' => return Sys_TLBI;  // VMALLS12E1OS
        when '100 1000 0010 001' => return Sys_TLBI;  // RVAE2IS
        when '100 1000 0010 101' => return Sys_TLBI;  // RVALE2IS
        when '100 1000 0011 000' => return Sys_TLBI;  // ALLE2IS
        when '100 1000 0011 001' => return Sys_TLBI;  // VAE2IS
        when '100 1000 0011 100' => return Sys_TLBI;  // ALLE1IS
        when '100 1000 0011 101' => return Sys_TLBI;  // VALE2IS
        when '100 1000 0011 110' => return Sys_TLBI;  // VMALLS12E1IS
        when '100 1000 0100 000' => return Sys_TLBI;  // IPAS2E1OS
        when '100 1000 0100 001' => return Sys_TLBI;  // IPAS2E1
        when '100 1000 0100 010' => return Sys_TLBI;  // RIPAS2E1
        when '100 1000 0100 011' => return Sys_TLBI;  // RIPAS2E1OS
        when '100 1000 0100 100' => return Sys_TLBI;  // IPAS2LE1OS
        when '100 1000 0100 101' => return Sys_TLBI;  // IPAS2LE1
        when '100 1000 0100 110' => return Sys_TLBI;  // RIPAS2LE1
        when '100 1000 0100 111' => return Sys_TLBI;  // RIPAS2LE1OS
        when '100 1000 0101 001' => return Sys_TLBI;  // RVAE2OS
        when '100 1000 0101 101' => return Sys_TLBI;  // RVALE2OS
        when '100 1000 0110 001' => return Sys_TLBI;  // RVAE2
        when '100 1000 0110 101' => return Sys_TLBI;  // RVALE2
        when '100 1000 0111 000' => return Sys_TLBI;  // ALLE2
        when '100 1000 0111 001' => return Sys_TLBI;  // VAE2
        when '100 1000 0111 100' => return Sys_TLBI;  // ALLE1
        when '100 1000 0111 101' => return Sys_TLBI;  // VALE2
        when '100 1000 0111 110' => return Sys_TLBI;  // VMALLS12E1
        when '100 1001 0000 001' => return Sys_TLBI;  // IPAS2E1ISNXS
        when '100 1001 0000 010' => return Sys_TLBI;  // RIPAS2E1ISNXS
        when '100 1001 0000 101' => return Sys_TLBI;  // IPAS2LE1ISNXS
        when '100 1001 0000 110' => return Sys_TLBI;  // RIPAS2LE1ISNXS
        when '100 1001 0001 000' => return Sys_TLBI;  // ALLE2OSNXS
        when '100 1001 0001 001' => return Sys_TLBI;  // VAE2OSNXS
        when '100 1001 0001 100' => return Sys_TLBI;  // ALLE1OSNXS
        when '100 1001 0001 101' => return Sys_TLBI;  // VALE2OSNXS
        when '100 1001 0001 110' => return Sys_TLBI;  // VMALLS12E1OSNXS
        when '100 1001 0010 001' => return Sys_TLBI;  // RVAE2ISNXS
        when '100 1001 0010 101' => return Sys_TLBI;  // RVALE2ISNXS
        when '100 1001 0011 000' => return Sys_TLBI;  // ALLE2ISNXS
        when '100 1001 0011 001' => return Sys_TLBI;  // VAE2ISNXS
        when '100 1001 0011 100' => return Sys_TLBI;  // ALLE1ISNXS
        when '100 1001 0011 101' => return Sys_TLBI;  // VALE2ISNXS
        when '100 1001 0011 110' => return Sys_TLBI;  // VMALLS12E1ISNXS
        when '100 1001 0100 000' => return Sys_TLBI;  // IPAS2E1OSNXS
        when '100 1001 0100 001' => return Sys_TLBI;  // IPAS2E1NXS
        when '100 1001 0100 010' => return Sys_TLBI;  // RIPAS2E1NXS
        when '100 1001 0100 011' => return Sys_TLBI;  // RIPAS2E1OSNXS
        when '100 1001 0100 100' => return Sys_TLBI;  // IPAS2LE1OSNXS
        when '100 1001 0100 101' => return Sys_TLBI;  // IPAS2LE1NXS
        when '100 1001 0100 110' => return Sys_TLBI;  // RIPAS2LE1NXS
```

```
        when '100 1001 0100 111' => return Sys_TLBI;  // RIPAS2LE1OSNXS
        when '100 1001 0101 001' => return Sys_TLBI;  // RVAE2OSNXS
        when '100 1001 0101 101' => return Sys_TLBI;  // RVALE2OSNXS
        when '100 1001 0110 001' => return Sys_TLBI;  // RVAE2NXS
        when '100 1001 0110 101' => return Sys_TLBI;  // RVALE2NXS
        when '100 1001 0111 000' => return Sys_TLBI;  // ALLE2NXS
        when '100 1001 0111 001' => return Sys_TLBI;  // VAE2NXS
        when '100 1001 0111 100' => return Sys_TLBI;  // ALLE1NXS
        when '100 1001 0111 101' => return Sys_TLBI;  // VALE2NXS
        when '100 1001 0111 110' => return Sys_TLBI;  // VMALLS12E1NXS
        when '110 1000 0001 000' => return Sys_TLBI;  // ALLE3OS
        when '110 1000 0001 001' => return Sys_TLBI;  // VAE3OS
        when '110 1000 0001 100' => return Sys_TLBI;  // PAALLOS
        when '110 1000 0001 101' => return Sys_TLBI;  // VALE3OS
        when '110 1000 0010 001' => return Sys_TLBI;  // RVAE3IS
        when '110 1000 0010 101' => return Sys_TLBI;  // RVALE3IS
        when '110 1000 0011 000' => return Sys_TLBI;  // ALLE3IS
        when '110 1000 0011 001' => return Sys_TLBI;  // VAE3IS
        when '110 1000 0011 101' => return Sys_TLBI;  // VALE3IS
        when '110 1000 0100 011' => return Sys_TLBI;  // RPAOS
        when '110 1000 0100 111' => return Sys_TLBI;  // RPALOS
        when '110 1000 0101 001' => return Sys_TLBI;  // RVAE3OS
        when '110 1000 0101 101' => return Sys_TLBI;  // RVALE3OS
        when '110 1000 0110 001' => return Sys_TLBI;  // RVAE3
        when '110 1000 0110 101' => return Sys_TLBI;  // RVALE3
        when '110 1000 0111 000' => return Sys_TLBI;  // ALLE3
        when '110 1000 0111 001' => return Sys_TLBI;  // VAE3
        when '110 1000 0111 100' => return Sys_TLBI;  // PAALL
        when '110 1000 0111 101' => return Sys_TLBI;  // VALE3
        when '110 1001 0001 000' => return Sys_TLBI;  // ALLE3OSNXS
        when '110 1001 0001 001' => return Sys_TLBI;  // VAE3OSNXS
        when '110 1001 0001 101' => return Sys_TLBI;  // VALE3OSNXS
        when '110 1001 0010 001' => return Sys_TLBI;  // RVAE3ISNXS
        when '110 1001 0010 101' => return Sys_TLBI;  // RVALE3ISNXS
        when '110 1001 0011 000' => return Sys_TLBI;  // ALLE3ISNXS
        when '110 1001 0011 001' => return Sys_TLBI;  // VAE3ISNXS
        when '110 1001 0011 101' => return Sys_TLBI;  // VALE3ISNXS
        when '110 1001 0101 001' => return Sys_TLBI;  // RVAE3OSNXS
        when '110 1001 0101 101' => return Sys_TLBI;  // RVALE3OSNXS
        when '110 1001 0110 001' => return Sys_TLBI;  // RVAE3NXS
        when '110 1001 0110 101' => return Sys_TLBI;  // RVALE3NXS
        when '110 1001 0111 000' => return Sys_TLBI;  // ALLE3NXS
        when '110 1001 0111 001' => return Sys_TLBI;  // VAE3NXS
        when '110 1001 0111 101' => return Sys_TLBI;  // VALE3NXS
        when '000 1100 0000 000' => return Sys_GSB;   // SYS
        when '000 1100 0000 001' => return Sys_GSB;   // ACK
        when '000 1100 0001 000' => return Sys_GIC;   // CDDIS
        when '000 1100 0001 001' => return Sys_GIC;   // CDEN
        when '000 1100 0001 010' => return Sys_GIC;   // CDPRI
        when '000 1100 0001 011' => return Sys_GIC;   // CDAFF
        when '000 1100 0001 100' => return Sys_GIC;   // CDPEND
        when '000 1100 0001 101' => return Sys_GIC;   // CDRCFG
        when '000 1100 0001 111' => return Sys_GIC;   // CDEOI
        when '000 1100 0010 000' => return Sys_GIC;   // CDDI
        when '000 1100 0010 001' => return Sys_GIC;   // CDHM
        when '100 1100 0001 000' => return Sys_GIC;   // VDDIS
        when '100 1100 0001 001' => return Sys_GIC;   // VDEN
        when '100 1100 0001 010' => return Sys_GIC;   // VDPRI
        when '100 1100 0001 011' => return Sys_GIC;   // VDAFF
        when '100 1100 0001 100' => return Sys_GIC;   // VDPEND
        when '100 1100 0001 101' => return Sys_GIC;   // VDRCFG
        when '100 1100 0010 000' => return Sys_GIC;   // VDDI
        when '100 1100 0010 001' => return Sys_GIC;   // VDHM
        when '110 1100 0001 000' => return Sys_GIC;   // LDDIS
        when '110 1100 0001 001' => return Sys_GIC;   // LDEN
        when '110 1100 0001 010' => return Sys_GIC;   // LDPRI
        when '110 1100 0001 011' => return Sys_GIC;   // LDAFF
        when '110 1100 0001 100' => return Sys_GIC;   // LDPEND
        when '110 1100 0001 101' => return Sys_GIC;   // LDRCFG
        when '110 1100 0010 000' => return Sys_GIC;   // LDDI
        when '110 1100 0010 001' => return Sys_GIC;   // LDHM
        otherwise =>                return Sys_SYS;
    end;
end;
```

Library pseudocode for aarch64/functions/sysop/SystemOp

```
// SystemOp
// ========
// System instruction types.

type SystemOp of enumeration {
    Sys_AT,
    Sys_BRB,
    Sys_DC,
    Sys_GIC,
    Sys_GSB,
    Sys_IC,
    Sys_TLBI,
    Sys_SYS
};
```

Library pseudocode for aarch64/functions/sysop_128/SysOp128

```
// SysOp128()
// ==========

func SysOp128(op1 : bits(3), CRn : bits(4), CRm : bits(4), op2 : bits(3)) => SystemOp128
begin
    case op1::CRn::CRm::op2 of
        when '000 1000 0001 001' => return Sys_TLBIP;  // VAE1OS
        when '000 1000 0001 011' => return Sys_TLBIP;  // VAAE1OS
        when '000 1000 0001 101' => return Sys_TLBIP;  // VALE1OS
        when '000 1000 0001 111' => return Sys_TLBIP;  // VAALE1OS
        when '000 1000 0011 001' => return Sys_TLBIP;  // VAE1IS
        when '000 1000 0011 011' => return Sys_TLBIP;  // VAAE1IS
        when '000 1000 0011 101' => return Sys_TLBIP;  // VALE1IS
        when '000 1000 0011 111' => return Sys_TLBIP;  // VAALE1IS
        when '000 1000 0111 001' => return Sys_TLBIP;  // VAE1
        when '000 1000 0111 011' => return Sys_TLBIP;  // VAAE1
        when '000 1000 0111 101' => return Sys_TLBIP;  // VALE1
        when '000 1000 0111 111' => return Sys_TLBIP;  // VAALE1
        when '000 1001 0001 001' => return Sys_TLBIP;  // VAE1OSNXS
        when '000 1001 0001 011' => return Sys_TLBIP;  // VAAE1OSNXS
        when '000 1001 0001 101' => return Sys_TLBIP;  // VALE1OSNXS
        when '000 1001 0001 111' => return Sys_TLBIP;  // VAALE1OSNXS
        when '000 1001 0011 001' => return Sys_TLBIP;  // VAE1ISNXS
        when '000 1001 0011 011' => return Sys_TLBIP;  // VAAE1ISNXS
        when '000 1001 0011 101' => return Sys_TLBIP;  // VALE1ISNXS
        when '000 1001 0011 111' => return Sys_TLBIP;  // VAALE1ISNXS
        when '000 1001 0111 001' => return Sys_TLBIP;  // VAE1NXS
        when '000 1001 0111 011' => return Sys_TLBIP;  // VAAE1NXS
        when '000 1001 0111 101' => return Sys_TLBIP;  // VALE1NXS
        when '000 1001 0111 111' => return Sys_TLBIP;  // VAALE1NXS
        when '100 1000 0001 001' => return Sys_TLBIP;  // VAE2OS
        when '100 1000 0001 101' => return Sys_TLBIP;  // VALE2OS
        when '100 1000 0011 001' => return Sys_TLBIP;  // VAE2IS
        when '100 1000 0011 101' => return Sys_TLBIP;  // VALE2IS
        when '100 1000 0111 001' => return Sys_TLBIP;  // VAE2
        when '100 1000 0111 101' => return Sys_TLBIP;  // VALE2
        when '100 1001 0001 001' => return Sys_TLBIP;  // VAE2OSNXS
        when '100 1001 0001 101' => return Sys_TLBIP;  // VALE2OSNXS
        when '100 1001 0011 001' => return Sys_TLBIP;  // VAE2ISNXS
        when '100 1001 0011 101' => return Sys_TLBIP;  // VALE2ISNXS
        when '100 1001 0111 001' => return Sys_TLBIP;  // VAE2NXS
        when '100 1001 0111 101' => return Sys_TLBIP;  // VALE2NXS
        when '110 1000 0001 001' => return Sys_TLBIP;  // VAE3OS
        when '110 1000 0001 101' => return Sys_TLBIP;  // VALE3OS
        when '110 1000 0011 001' => return Sys_TLBIP;  // VAE3IS
        when '110 1000 0011 101' => return Sys_TLBIP;  // VALE3IS
        when '110 1000 0111 001' => return Sys_TLBIP;  // VAE3
        when '110 1000 0111 101' => return Sys_TLBIP;  // VALE3
        when '110 1001 0001 001' => return Sys_TLBIP;  // VAE3OSNXS
        when '110 1001 0001 101' => return Sys_TLBIP;  // VALE3OSNXS
        when '110 1001 0011 001' => return Sys_TLBIP;  // VAE3ISNXS
        when '110 1001 0011 101' => return Sys_TLBIP;  // VALE3ISNXS
        when '110 1001 0111 001' => return Sys_TLBIP;  // VAE3NXS
        when '110 1001 0111 101' => return Sys_TLBIP;  // VALE3NXS
        when '100 1000 0000 001' => return Sys_TLBIP;  // IPAS2E1IS
        when '100 1000 0000 101' => return Sys_TLBIP;  // IPAS2LE1IS
        when '100 1000 0100 000' => return Sys_TLBIP;  // IPAS2E1OS
        when '100 1000 0100 001' => return Sys_TLBIP;  // IPAS2E1
        when '100 1000 0100 100' => return Sys_TLBIP;  // IPAS2LE1OS
        when '100 1000 0100 101' => return Sys_TLBIP;  // IPAS2LE1
        when '100 1001 0000 001' => return Sys_TLBIP;  // IPAS2E1ISNXS
        when '100 1001 0000 101' => return Sys_TLBIP;  // IPAS2LE1ISNXS
        when '100 1001 0100 000' => return Sys_TLBIP;  // IPAS2E1OSNXS
        when '100 1001 0100 001' => return Sys_TLBIP;  // IPAS2E1NXS
        when '100 1001 0100 100' => return Sys_TLBIP;  // IPAS2LE1OSNXS
        when '100 1001 0100 101' => return Sys_TLBIP;  // IPAS2LE1NXS
        when '000 1000 0010 001' => return Sys_TLBIP;  // RVAE1IS
        when '000 1000 0010 011' => return Sys_TLBIP;  // RVAAE1IS
        when '000 1000 0010 101' => return Sys_TLBIP;  // RVALE1IS
        when '000 1000 0010 111' => return Sys_TLBIP;  // RVAALE1IS
        when '000 1000 0101 001' => return Sys_TLBIP;  // RVAE1OS
        when '000 1000 0101 011' => return Sys_TLBIP;  // RVAAE1OS
        when '000 1000 0101 101' => return Sys_TLBIP;  // RVALE1OS
        when '000 1000 0101 111' => return Sys_TLBIP;  // RVAALE1OS
        when '000 1000 0110 001' => return Sys_TLBIP;  // RVAE1
        when '000 1000 0110 011' => return Sys_TLBIP;  // RVAAE1
        when '000 1000 0110 101' => return Sys_TLBIP;  // RVALE1
        when '000 1000 0110 111' => return Sys_TLBIP;  // RVAALE1
        when '000 1001 0010 001' => return Sys_TLBIP;  // RVAE1ISNXS
        when '000 1001 0010 011' => return Sys_TLBIP;  // RVAAE1ISNXS
        when '000 1001 0010 101' => return Sys_TLBIP;  // RVALE1ISNXS
        when '000 1001 0010 111' => return Sys_TLBIP;  // RVAALE1ISNXS
        when '000 1001 0101 001' => return Sys_TLBIP;  // RVAE1OSNXS
        when '000 1001 0101 011' => return Sys_TLBIP;  // RVAAE1OSNXS
        when '000 1001 0101 101' => return Sys_TLBIP;  // RVALE1OSNXS
        when '000 1001 0101 111' => return Sys_TLBIP;  // RVAALE1OSNXS
        when '000 1001 0110 001' => return Sys_TLBIP;  // RVAE1NXS
        when '000 1001 0110 011' => return Sys_TLBIP;  // RVAAE1NXS
        when '000 1001 0110 101' => return Sys_TLBIP;  // RVALE1NXS
```

```
        when '000 1001 0110 111' => return Sys_TLBIP;  // RVAALE1NXS
        when '100 1000 0010 001' => return Sys_TLBIP;  // RVAE2IS
        when '100 1000 0010 101' => return Sys_TLBIP;  // RVALE2IS
        when '100 1000 0101 001' => return Sys_TLBIP;  // RVAE2OS
        when '100 1000 0101 101' => return Sys_TLBIP;  // RVALE2OS
        when '100 1000 0110 001' => return Sys_TLBIP;  // RVAE2
        when '100 1000 0110 101' => return Sys_TLBIP;  // RVALE2
        when '100 1001 0010 001' => return Sys_TLBIP;  // RVAE2ISNXS
        when '100 1001 0010 101' => return Sys_TLBIP;  // RVALE2ISNXS
        when '100 1001 0101 001' => return Sys_TLBIP;  // RVAE2OSNXS
        when '100 1001 0101 101' => return Sys_TLBIP;  // RVALE2OSNXS
        when '100 1001 0110 001' => return Sys_TLBIP;  // RVAE2NXS
        when '100 1001 0110 101' => return Sys_TLBIP;  // RVALE2NXS
        when '110 1000 0010 001' => return Sys_TLBIP;  // RVAE3IS
        when '110 1000 0010 101' => return Sys_TLBIP;  // RVALE3IS
        when '110 1000 0101 001' => return Sys_TLBIP;  // RVAE3OS
        when '110 1000 0101 101' => return Sys_TLBIP;  // RVALE3OS
        when '110 1000 0110 001' => return Sys_TLBIP;  // RVAE3
        when '110 1000 0110 101' => return Sys_TLBIP;  // RVALE3
        when '110 1001 0010 001' => return Sys_TLBIP;  // RVAE3ISNXS
        when '110 1001 0010 101' => return Sys_TLBIP;  // RVALE3ISNXS
        when '110 1001 0101 001' => return Sys_TLBIP;  // RVAE3OSNXS
        when '110 1001 0101 101' => return Sys_TLBIP;  // RVALE3OSNXS
        when '110 1001 0110 001' => return Sys_TLBIP;  // RVAE3NXS
        when '110 1001 0110 101' => return Sys_TLBIP;  // RVALE3NXS
        when '100 1000 0000 010' => return Sys_TLBIP;  // RIPAS2E1IS
        when '100 1000 0000 110' => return Sys_TLBIP;  // RIPAS2LE1IS
        when '100 1000 0100 010' => return Sys_TLBIP;  // RIPAS2E1
        when '100 1000 0100 011' => return Sys_TLBIP;  // RIPAS2E1OS
        when '100 1000 0100 110' => return Sys_TLBIP;  // RIPAS2LE1
        when '100 1000 0100 111' => return Sys_TLBIP;  // RIPAS2LE1OS
        when '100 1001 0000 010' => return Sys_TLBIP;  // RIPAS2E1ISNXS
        when '100 1001 0000 110' => return Sys_TLBIP;  // RIPAS2LE1ISNXS
        when '100 1001 0100 010' => return Sys_TLBIP;  // RIPAS2E1NXS
        when '100 1001 0100 011' => return Sys_TLBIP;  // RIPAS2E1OSNXS
        when '100 1001 0100 110' => return Sys_TLBIP;  // RIPAS2LE1NXS
        when '100 1001 0100 111' => return Sys_TLBIP;  // RIPAS2LE1OSNXS
        otherwise                => return Sys_SYSP;
    end;
end;
```

Library pseudocode for aarch64/functions/sysop_128/SystemOp128

```
// SystemOp128()
// =============
// System instruction types.

type SystemOp128 of enumeration {Sys_TLBIP, Sys_SYSP};
```

Library pseudocode for aarch64/functions/sysregisters/ELR_EL

```
// ELR_EL - accessor
// =================

accessor ELR_EL(el : bits(2)) <=> value : bits(64)
begin
    getter
        var r : bits(64);
        case el of
            when EL1 =>  r = ELR_EL1();
            when EL2 =>  r = ELR_EL2();
            when EL3 =>  r = ELR_EL3();
            otherwise => unreachable;
        end;
        return r;
    end;
    setter
        let r : bits(64) = value;
        case el of
            when EL1 =>  ELR_EL1() = r;
            when EL2 =>  ELR_EL2() = r;
            when EL3 =>  ELR_EL3() = r;
            otherwise => unreachable;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/ELR_ELx

```
// ELR_ELx - accessor
// ==================

accessor ELR_ELx() <=> value : bits(64)
begin
    getter
        assert PSTATE.EL != EL0;
        return ELR_EL(PSTATE.EL);
    end;
    setter
        assert PSTATE.EL != EL0;
        ELR_EL(PSTATE.EL) = value;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/ESR_EL

```
// ESR_EL - accessor
// =================

accessor ESR_EL(regime : bits(2)) <=> value : ESRType
begin
    readonly getter
        var r : bits(64);
        case regime of
            when EL1 =>  r = ESR_EL1();
            when EL2 =>  r = ESR_EL2();
            when EL3 =>  r = ESR_EL3();
            otherwise => unreachable;
        end;
        return r;
    end;
    setter
        let r : bits(64) = value;
        case regime of
            when EL1 =>  ESR_EL1() = r;
            when EL2 =>  ESR_EL2() = r;
            when EL3 =>  ESR_EL3() = r;
            otherwise => unreachable;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/ESR_ELx

```
// ESR_ELx - accessor
// ==================

accessor ESR_ELx() <=> value : ESRType
begin
    readonly getter
        return ESR_EL(S1TranslationRegime());
    end;

    setter
        ESR_EL(S1TranslationRegime()) = value;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/FAR_EL

```
// FAR_EL - accessor
// =================

accessor FAR_EL(regime : bits(2)) <=> value : bits(64)
begin
    readonly getter
        var r : bits(64);
        case regime of
            when EL1 =>  r = FAR_EL1();
            when EL2 =>  r = FAR_EL2();
            when EL3 =>  r = FAR_EL3();
            otherwise => unreachable;
        end;
        return r;
    end;
    setter
        let r : bits(64) = value;
        case regime of
            when EL1 =>  FAR_EL1() = r;
            when EL2 =>  FAR_EL2() = r;
            when EL3 =>  FAR_EL3() = r;
            otherwise => unreachable;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/FAR_ELx

```
// FAR_ELx - accessor
// ==================

accessor FAR_ELx() <=> value : bits(64)
begin
    readonly getter
        return FAR_EL(S1TranslationRegime());
    end;

    setter
        FAR_EL(S1TranslationRegime()) = value;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/GetSPSR_EL1_M4

```
// GetSPSR_EL1_M4()
// ================
// Query the SPSR_EL1.M[4] field.

func GetSPSR_EL1_M4() => bit
begin
    return SPSR_EL1().M[4];
end;
```

Library pseudocode for aarch64/functions/sysregisters/GetSPSR_EL2_M4

```
// GetSPSR_EL2_M4()
// ================
// Query the SPSR_EL2.M[4] field.

func GetSPSR_EL2_M4() => bit
begin
    return SPSR_EL2().M[4];
end;
```

Library pseudocode for aarch64/functions/sysregisters/GetSPSR_EL3_M4

```
// GetSPSR_EL3_M4()
// ================
// Query the SPSR_EL3.M[4] field.

func GetSPSR_EL3_M4() => bit
begin
    return SPSR_EL3().M[4];
end;
```

Library pseudocode for aarch64/functions/sysregisters/PFAR_EL

```
// PFAR_EL - accessor
// ==================

accessor PFAR_EL(regime : bits(2)) <=> value : bits(64)
begin
    getter
        assert (IsFeatureImplemented(FEAT_PFAR) ||
                (regime == EL3 && IsFeatureImplemented(FEAT_RME)));
        var r : bits(64);
        case regime of
            when EL1 => r = PFAR_EL1();
            when EL2 => r = PFAR_EL2();
            when EL3 => r = MFAR_EL3();
            otherwise => unreachable;
        end;
        return r;
    end;
    setter
        let r : bits(64) = value;
        assert (IsFeatureImplemented(FEAT_PFAR) ||
                  (IsFeatureImplemented(FEAT_RME) && regime == EL3));
        case regime of
            when EL1 => PFAR_EL1() = r;
            when EL2 => PFAR_EL2() = r;
            when EL3 => MFAR_EL3() = r;
            otherwise => unreachable;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/PFAR_ELx

```
// PFAR_ELx - accessor
// ===================

accessor PFAR_ELx() <=> value : bits(64)
begin
    getter
        return PFAR_EL(S1TranslationRegime());
    end;

    setter
        PFAR_EL(S1TranslationRegime()) = value;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/SCTLR_EL

```
// SCTLR_EL - accessor
// ===================

accessor SCTLR_EL(regime : bits(2)) <=> value : SCTLRType
begin
    getter
        var r : bits(64);
        case regime of
            when EL1 =>  r = SCTLR_EL1();
            when EL2 =>  r = SCTLR_EL2();
            when EL3 =>  r = SCTLR_EL3();
            otherwise => unreachable;
        end;
        return r;
    end;
    setter
        let r : bits(64) = value;
        case regime of
            when EL1 => SCTLR_EL1() = r;
            when EL2 => SCTLR_EL2() = r;
            when EL3 => SCTLR_EL3() = r;
            otherwise => unreachable;
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/SCTLR_ELx

```
// SCTLR_ELx - accessor
// ===================

accessor SCTLR_ELx() <=> value : SCTLRType
begin
    getter
        return SCTLR_EL(S1TranslationRegime());
    end;
    setter
        SCTLR_EL(S1TranslationRegime()) = value;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/VBAR_EL

```
// VBAR_EL - accessor
// ==================

accessor VBAR_EL(regime : bits(2)) <=> value : bits(64)
begin
    getter
        var r : bits(64);
        case regime of
            when EL1 =>  r = VBAR_EL1();
            when EL2 =>  r = VBAR_EL2();
            when EL3 =>  r = VBAR_EL3();
            otherwise => unreachable;
        end;
        return r;
    end;
    setter
        pass;
    end;
end;
```

Library pseudocode for aarch64/functions/sysregisters/VBAR_ELx

```
// VBAR_ELx - accessor
// ===================

accessor VBAR_ELx() <=> value : bits(64)
begin
    getter
        return VBAR_EL(S1TranslationRegime());
    end;
    setter
        VBAR_EL(S1TranslationRegime())[63:0] = value;
    end;
end;
```

Library pseudocode for aarch64/functions/system/AArch64_CheckDAIFAccess

```
// AArch64_CheckDAIFAccess()
// =========================
// Check that an AArch64 MSR/MRS access to the DAIF flags is permitted.

func AArch64_CheckDAIFAccess(field : PSTATEField)
begin
    if field IN {PSTATEField_DAIFSet, PSTATEField_DAIFClr} then
        if (EffectivelyAtEL0InHost() ||
              (EffectivelyAtEL0NotInHost() && SCTLR_EL1().UMA == '0')) then
            AArch64_SystemAccessTraptoEL1orEL2(0x18);
        end;
    end;
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ChooseEIRGNonExcludedTag

```
// AArch64_ChooseEIRGNonExcludedTag()
// ==================================
// Return a tag derived from SEED and EIRGSTATE, excluding any tags in the
// given mask.

func AArch64_ChooseEIRGNonExcludedTag(exclude : bits(16)) => bits(4)
begin
    assert GCR_EL1().RRND == '0';

    if IsOnes(exclude) then
        return '0000';
    end;

    var bins : array[[16]] of bits(4);

    var N : integer = 0;
    for i = 0 to 15 do
        if exclude[i] == '0' then
            bins[[N]] = i[3:0];
            N = N + 1;
        end;
    end;

    let seed : bits(16) = RGSR_EL1().RRND0.SEED[15:0];
    var eirgstate : integer = UInt(RGSR_EL1().TAG);

    eirgstate = eirgstate + 1;
    if eirgstate >= N then
        eirgstate = 0;
    end;

    let lbin : integer = (eirgstate + (N * UInt(seed)) >> 16) MOD N;

    RGSR_EL1().TAG = eirgstate[3:0];

    let top : bit = seed[5] XOR seed[3] XOR seed[2] XOR seed[0];
    RGSR_EL1().RRND0.SEED[15:0] = top::seed[15:1];

    return bins[[lbin]];
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ChooseNonExcludedTag

```
// AArch64_ChooseNonExcludedTag()
// ==============================
// Return a tag derived from the start and the offset values, excluding
// any tags in the given mask.

func AArch64_ChooseNonExcludedTag(tag_in : bits(4), offset_in : bits(4),
                                  exclude : bits(16)) => bits(4)
begin
    var tag : bits(4) = tag_in;
    var offset : bits(4) = offset_in;

    if IsOnes(exclude) then
        return '0000';
    end;

    if offset == '0000' then
        while exclude[UInt(tag)] == '1' looplimit 16 do
            tag = tag + '0001';
        end;
    end;

    while offset != '0000' looplimit 16 do
        offset = offset - '0001';
        tag = tag + '0001';
        while exclude[UInt(tag)] == '1' looplimit 16 do
            tag = tag + '0001';
        end;
    end;

    return tag;
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ChooseNonExcludedTagOrZero

```
// AArch64_ChooseNonExcludedTagOrZero()
// ====================================
// Return a tag derived from the start and the offset values, excluding any
// tags in the given mask, or zero if Allocation Tag access is not enabled.

func AArch64_ChooseNonExcludedTagOrZero(tag : bits(4), offset : bits(4),
                                        exclude : bits(16)) => bits(4)
begin
    if IsMTEEnabled(PSTATE.EL) then
        return AArch64_ChooseNonExcludedTag(tag, offset, exclude);
    else
        return '0000';
    end;
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ChooseTagOrZero

```
// AArch64_ChooseTagOrZero()
// =========================
// Return a tag, excluding any tags in the given mask, , or zero if Allocation
// Tag access is not enabled.

func AArch64_ChooseTagOrZero(exclude : bits(16)) => bits(4)
begin
    if IsMTEEnabled(PSTATE.EL) then
        if GCR_EL1().RRND == '1' then
            if IsOnes(exclude) then
                return '0000';
            else
                return ChooseRandomNonExcludedTag(exclude);
            end;
        else
            if IsFeatureImplemented(FEAT_MTE_EIRG) then
                return AArch64_ChooseEIRGNonExcludedTag(exclude);
            end;
            let start_tag : bits(4) = RGSR_EL1().TAG;
            let offset : bits(4)    = AArch64_RandomTag();

            RGSR_EL1().TAG = AArch64_ChooseNonExcludedTag(start_tag, offset, exclude);

            return RGSR_EL1().TAG;
        end;
    else
        return '0000';
    end;
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ExecutingERETInstr

```
// AArch64_ExecutingERETInstr()
// ============================
// Returns TRUE if current instruction is ERET.

func AArch64_ExecutingERETInstr() => boolean
begin
    let instr : bits(32) = ThisInstr();
    return instr[31:12] == '11010110100111110000';
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ImpDefSysInstr

```
// AArch64_ImpDefSysInstr()
// ========================
// Execute an implementation-defined system instruction with write (source operand).

impdef func AArch64_ImpDefSysInstr(op0 : bits(2), op1 : bits(3), crn : bits(4),
                                   crm : bits(4), op2 : bits(3), t : integer)
begin
    Undefined();
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ImpDefSysInstr128

```
// AArch64_ImpDefSysInstr128()
// ===========================
// Execute an implementation-defined system instruction with write (128-bit source operand).

impdef func AArch64_ImpDefSysInstr128(op0 : bits(2), op1 : bits(3), crn : bits(4),
                                      crm : bits(4), op2 : bits(3),
                                      t : integer, t2 : integer)
begin
    Undefined();
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ImpDefSysInstrWithResult

```
// AArch64_ImpDefSysInstrWithResult()
// ==================================
// Execute an implementation-defined system instruction with read (result operand).

impdef func AArch64_ImpDefSysInstrWithResult(op0 : bits(2),
                                             op1 : bits(3),
                                             crn : bits(4),
                                             crm : bits(4),
                                             op2 : bits(3),
                                             t : integer)
begin
    Undefined();
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ImpDefSysRegRead

```
// AArch64_ImpDefSysRegRead()
// ==========================
// Read from an implementation-defined System register and write the contents of the register
// to X[t].

impdef func AArch64_ImpDefSysRegRead(op0 : bits(2), op1 : bits(3),
                                     crn : bits(4), crm : bits(4),
                                     op2 : bits(3), t : integer)
begin
    Undefined();
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ImpDefSysRegRead128

```
// AArch64_ImpDefSysRegRead128()
// =============================
// Read from an 128-bit implementation-defined System register
// and write the contents of the register to X[t], X[t+1].

impdef func AArch64_ImpDefSysRegRead128(op0 : bits(2),
                                        op1 : bits(3), crn : bits(4),
                                        crm : bits(4), op2 : bits(3),
                                        t : integer, t2 : integer)
begin
    Undefined();
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ImpDefSysRegWrite

```
// AArch64_ImpDefSysRegWrite()
// ===========================
// Write to an implementation-defined System register.

impdef func AArch64_ImpDefSysRegWrite(op0 : bits(2), op1 : bits(3), crn : bits(4),
                                      crm : bits(4), op2 : bits(3), t : integer)
begin
    Undefined();
end;
```

Library pseudocode for aarch64/functions/system/AArch64_ImpDefSysRegWrite128

```
// AArch64_ImpDefSysRegWrite128()
// ==============================
// Write the contents of X[t], X[t+1] to an 128-bit implementation-defined System register.

impdef func AArch64_ImpDefSysRegWrite128(op0 : bits(2), op1 : bits(3), crn : bits(4),
                                         crm : bits(4), op2 : bits(3),
                                         t : integer, t2 : integer)
begin
    Undefined();
end;
```

Library pseudocode for aarch64/functions/system/AArch64_InterruptPending

```
// AArch64_InterruptPending()
// ==========================
// Returns TRUE if there are any pending physical, virtual, or delegated
// interrupts, and FALSE otherwise.

func AArch64_InterruptPending() => boolean
begin
    let (irq_pending, -) = IRQPending();
    let (fiq_pending, -) = FIQPending();
    let pending_physical_interrupt : boolean = (irq_pending || fiq_pending ||
                                                IsPhysicalSErrorPending());

    var pending_virtual_interrupt : boolean = FALSE;
    if EL2Enabled() && PSTATE.EL IN {EL0, EL1} && HCR_EL2().TGE == '0' then
        let virq_pending : boolean = (HCR_EL2().IMO == '1' && (VirtualIRQPending() ||
                                      HCR_EL2().VI == '1'));
        let vfiq_pending : boolean = (HCR_EL2().FMO == '1' && (VirtualFIQPending() ||
                                      HCR_EL2().VF == '1'));
        let vsei_pending : boolean = ((HCR_EL2().AMO == '1' ||
                                     (IsFeatureImplemented(FEAT_DoubleFault2) &&
                                      IsHCRXEL2Enabled() && HCRX_EL2().TMEA == '1')) &&
                                     (IsVirtualSErrorPending() || HCR_EL2().VSE == '1'));

        pending_virtual_interrupt = vsei_pending || virq_pending || vfiq_pending;
    end;

    let pending_delegated_interrupt : boolean = (PSTATE.EL != EL3 &&
                                                 EffectiveSCR_EL3_EnDSE() == '1' &&
                                                 SCR_EL3().DSE == '1');

    return pending_physical_interrupt || pending_virtual_interrupt || pending_delegated_interrupt;
end;
```

Library pseudocode for aarch64/functions/system/AArch64_NextRandomTagBit

```
// AArch64_NextRandomTagBit()
// ==========================
// Generate a random bit suitable for generating a random Allocation Tag.

func AArch64_NextRandomTagBit() => bit
begin
    assert GCR_EL1().RRND == '0';
    let lfsr : bits(16) = RGSR_EL1().RRND0.SEED[15:0];
    let top : bit = lfsr[5] XOR lfsr[3] XOR lfsr[2] XOR lfsr[0];
    RGSR_EL1().RRND0.SEED[15:0] = top::lfsr[15:1];
    return top;
end;
```

Library pseudocode for aarch64/functions/system/AArch64_RandomTag

```
// AArch64_RandomTag()
// ===================
// Generate a random Allocation Tag.

func AArch64_RandomTag() => bits(4)
begin
    var tag : bits(4);
    for i = 0 to 3 do
        tag[i] = AArch64_NextRandomTagBit();
    end;
    return tag;
end;
```

Library pseudocode for aarch64/functions/system/AArch64_SysInstr

```
// AArch64_SysInstr()
// ==================
// Execute a system instruction with write (source operand).

impdef func AArch64_SysInstr(op0 : bits(2), op1 : bits(3),
                             crn : bits(4), crm : bits(4), op2 : bits(3), t : integer)
begin
    Unimplemented();
end;
```

Library pseudocode for aarch64/functions/system/AArch64_SysInstrWithResult

```
// AArch64_SysInstrWithResult()
// ============================
// Execute a system instruction with read (result operand).
// Writes the result of the instruction to X[t].

impdef func AArch64_SysInstrWithResult(op0 : bits(2), op1 : bits(3),
                                       crn : bits(4), crm : bits(4),
                                       op2 : bits(3), t : integer)
begin
    Unimplemented();
end;
```

Library pseudocode for aarch64/functions/system/AArch64_SysRegRead

```
// AArch64_SysRegRead()
// ====================
// Read from a System register and write the contents of the register to X[t].

impdef func AArch64_SysRegRead(op0 : bits(2), op1 : bits(3), crn : bits(4),
                               crm : bits(4), op2 : bits(3), t : integer)
begin
    Unimplemented();
end;
```

Library pseudocode for aarch64/functions/system/AArch64_SysRegWrite

```
// AArch64_SysRegWrite()
// =====================
// Write to a System register.

impdef func AArch64_SysRegWrite(op0 : bits(2), op1 : bits(3), crn : bits(4),
                                crm : bits(4), op2 : bits(3), t : integer)
begin
    Unimplemented();
end;
```

Library pseudocode for aarch64/functions/system/BTypeCompatible

```
// BTypeCompatible
// ===============
// Records the branch target compatibility.
// Returns TRUE if the branch target is compatible with PSTATE.BTYPE, else FALSE.

var BTypeCompatible : boolean;
```

Library pseudocode for aarch64/functions/system/BTypeCompatible_BTI

```
// BTypeCompatible_BTI
// ===================
// This function determines whether a given hint encoding is compatible with the current value of
// PSTATE.BTYPE. A value of TRUE here indicates a valid Branch Target Identification instruction.

func BTypeCompatible_BTI(hintcode : bits(2)) => boolean
begin
    case hintcode of
        when '00' =>
            return PSTATE.BTYPE == '00';
        when '01' =>
            return PSTATE.BTYPE != '11';
        when '10' =>
            return PSTATE.BTYPE != '10';
        when '11' =>
            return TRUE;
    end;
end;
```

Library pseudocode for aarch64/functions/system/BTypeCompatible_PAC

```
// BTypeCompatible_PAC()
// =====================
// Returns TRUE if PACIxSP or PACIxSPPC instructions are implicitly compatible with PSTATE.BTYPE,
// FALSE otherwise.

func BTypeCompatible_PAC(pacinst : PACInstType) => boolean
begin
    // For only PACIxSP instructions, check if BTI behavior should be disabled
    if IsFeatureImplemented(FEAT_PAuth_EnhCtl) && pacinst == PACIxSP then
        var btd : bit;
        case PSTATE.EL of
            when EL0 =>
                if ELIsInHost(EL0) then
                    btd = if IsSCTLR2EL2Enabled() then SCTLR2_EL2().BTD0 else '0';
                else
                    btd = if IsSCTLR2EL1Enabled() then SCTLR2_EL1().BTD0 else '0';
                end;
            when EL1 => btd = if IsSCTLR2EL1Enabled() then SCTLR2_EL1().BTD else '0';
            when EL2 => btd = if IsSCTLR2EL2Enabled() then SCTLR2_EL2().BTD else '0';
            when EL3 => btd = SCTLR2_EL3().BTD;
        end;
        // When BTI compatibility is disabled,
        // these instructions are not compatible when PSTATE.BTYPE is non-zero
        if btd == '1' then return IsZero(PSTATE.BTYPE); end;
    end;

    if PSTATE.BTYPE != '11' then
        return TRUE;
    else
        let index : integer = if PSTATE.EL == EL0 then 35 else 36;
        return SCTLR_ELx()[index] == '0';
    end;
end;
```

Library pseudocode for aarch64/functions/system/BTypeCompatible_PACIXSP

```
// BTypeCompatible_PACIXSP()
// =========================
// Returns TRUE if PACIASP or PACIBSP instructions are implicitly compatible with PSTATE.BTYPE,
// FALSE otherwise.

func BTypeCompatible_PACIXSP() => boolean
begin
    let pacinst : PACInstType = PACIxSP;
    return BTypeCompatible_PAC(pacinst);
end;
```

Library pseudocode for aarch64/functions/system/BTypeNext

```
// BTypeNext
// =========
// Updated every cycle with a value that depends upon the instruction being executed. Assigned to
// PSTATE.BTYPE at the end of each cycle and then cleared to zero. Allows SPSR save/restore of
// BTYPE for the current instruction being executed.

var BTypeNext : bits(2);
```

Library pseudocode for aarch64/functions/system/ChooseRandomNonExcludedTag

```
// ChooseRandomNonExcludedTag()
// ============================
// The ChooseRandomNonExcludedTag function is used when GCR_EL1.RRND == '1' to generate random
// Allocation Tags.
//
// The resulting Allocation Tag is selected from the set [0,15], excluding any Allocation Tag where
// exclude[tag_value] == 1. If 'exclude' is all Ones, the returned Allocation Tag is '0000'.
//
// This function is permitted to generate a non-deterministic selection from the set of non-excluded
// Allocation Tags. A reasonable implementation should select a tag from a uniform distribution and
// avoid common pitfalls such as modulo bias.
//
// This function can read RGSR_EL1 and/or write RGSR_EL1 to an IMPLEMENTATION DEFINED value.
// If it is not capable of writing RGSR_EL1.SEED[15:0] to zero from a previous nonzero
// RGSR_EL1.SEED value, it is IMPLEMENTATION DEFINED whether the randomness is significantly
// impacted if RGSR_EL1.SEED[15:0] is set to zero.

impdef func ChooseRandomNonExcludedTag(exclude_in : bits(16)) => bits(4)
begin
    return ARBITRARY : bits(4);
end;
```

Library pseudocode for aarch64/functions/system/EffectiveBADDR

```
// EffectiveBADDR()
// ================
// Check if the given VA, held in a register BADDR field, is RESS and apply
// CONSTRAINED UNPREDICTABLE behavior if it is not.

func EffectiveBADDR(baddr_in : bits(64), directread : boolean) => bits(64)
begin
    var baddr : bits(64) = baddr_in;
    // Determine top bit position based on feature support.
    let n : integer{} = (if IsFeatureImplemented(FEAT_LVA3) then 57
                         else (if IsFeatureImplemented(FEAT_LVA) then 53
                         else 49));

    // Check if the upper bits of the base address form a valid sign extension.
    if baddr[63:n] != Replicate{64-n}(baddr[n-1]) then
        let c = ConstrainUnpredictable(Unpredictable_BADDR_RESS);
        assert c IN {Constraint_RESS, Constraint_ALLRESS, Constraint_FAULT};
        if c == Constraint_ALLRESS || (!directread && c == Constraint_RESS) then
            baddr[63:n] = Replicate{64-n}(baddr[n-1]);
        end;
        // If c == Constraint_FAULT, baddr will cause an Address Size fault when used.
    end;

    return baddr;
end;
```

Library pseudocode for aarch64/functions/system/EffectiveDIT

```
// EffectiveDIT()
// ==============
// Returns the Effective value of PSTATE.DIT.

readonly func EffectiveDIT() => bit
begin
    if !IsFeatureImplemented(FEAT_FDIT) then
        return PSTATE.DIT;
    end;

    if (PSTATE.EL == EL0 &&
          ((!ELIsInHost(EL0) && IsSCTLR2EL1Enabled() && SCTLR2_EL1().FDIT == '1') ||
           ( ELIsInHost(EL0) && IsSCTLR2EL2Enabled() && SCTLR2_EL2().FDIT == '1'))) then
        return '1';
    end;

    if (((PSTATE.EL == EL0 && !ELIsInHost(EL0)) || PSTATE.EL == EL1) &&
          IsHCRXEL2Enabled() && HCRX_EL2().FDIT == '1') then
        return '1';
    end;

    if PSTATE.EL != EL3 && SCR2_EL3().FDIT == '1' then
        return '1';
    end;

    return PSTATE.DIT;
end;
```

Library pseudocode for aarch64/functions/system/EffectiveHCRX_EL2_NVTGE

```
// EffectiveHCRX_EL2_NVTGE()
// =========================
// Returns the Effective value of HCRX_EL2.NVTGE.

readonly func EffectiveHCRX_EL2_NVTGE() => bit
begin
    if !IsFeatureImplemented(FEAT_NV3) then
        return '0';
    end;

    if HaveEL(EL3) && SCR2_EL3().NV3En == '0' then
        return '0';
    end;

    if EffectiveHCR_EL2_NVx() != '101' then
        return '0';
    end;

    if !IsHCRXEL2Enabled() then
        return '0';
    end;

    return HCRX_EL2().NVTGE;
end;
```

Library pseudocode for aarch64/functions/system/EffectiveICC_SRE_EL2_Enable

```
// EffectiveICC_SRE_EL2_Enable()
// =============================
// Returns the Effective value of ICC_SRE_EL2.Enable

func EffectiveICC_SRE_EL2_Enable() => bit
begin
    if IsFeatureImplemented(FEAT_GCIE) then
        return '1';
    end;
    if (ImpDefBool("Treat ICC_SRE_EL2.SRE as RAO/WI") &&
          ImpDefBool("Treat ICC_SRE_EL2.Enable as RAO/WI")) then
        return '1';
    end;
    if ICC_SRE_EL2().SRE == '0' then
        return '1';
    end;

    return ICC_SRE_EL2().Enable;
end;
```

Library pseudocode for aarch64/functions/system/EffectiveICC_SRE_EL3_Enable

```
// EffectiveICC_SRE_EL3_Enable()
// =============================
// Returns the Effective value of ICC_SRE_EL3.Enable

func EffectiveICC_SRE_EL3_Enable() => bit
begin
    if IsFeatureImplemented(FEAT_GCIE) && HaveEL(EL2) then
        return '1';
    end;

    return ICC_SRE_EL3().Enable;
end;
```

Library pseudocode for aarch64/functions/system/InGuardedPage

```
// InGuardedPage
// =============
// Records whether the currently fetched instruction was retrieved from a guarded page, will be
// TRUE if the GP bit in the page or block descriptor for the current instruction fetch was equal
// to one.

var InGuardedPage : boolean;
```

Library pseudocode for aarch64/functions/system/IsGMEnabledAtEL0

```
// IsGMEnabledAtEL0()
// ==================

func IsGMEnabledAtEL0() => boolean
begin
    if IsFeatureImplemented(FEAT_MTE_GM0) then
        if IsInHost() then
            return IsSCTLR2EL2Enabled() && SCTLR2_EL2().GM0 == '1';
        else
            return IsSCTLR2EL1Enabled() && SCTLR2_EL1().GM0 == '1';
        end;
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/functions/system/IsHCRXEL2Enabled

```
// IsHCRXEL2Enabled()
// ==================
// Returns TRUE if access to HCRX_EL2 register is enabled, and FALSE otherwise.
// Indirect read of HCRX_EL2 returns 0 when access is not enabled.

readonly func IsHCRXEL2Enabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_HCX) then return FALSE; end;
    if ELUsingAArch32(EL2) then return FALSE; end;
    if HaveEL(EL3) && SCR_EL3().HXEn == '0' then
        return FALSE;
    end;

    return EL2Enabled();
end;
```

Library pseudocode for aarch64/functions/system/IsMTEEnabled

```
// IsMTEEnabled()
// ==============
// Returns TRUE if the currently selected MTE mechanism is enabled, and FALSE otherwise.

readonly func IsMTEEnabled(el : bits(2)) => boolean
begin
    let regime : Regime = TranslationRegime(el);
    var selected : boolean = FALSE;

    if IsPMTESelected(el) then
        if HaveEL(EL3) && SCR_EL3().ATA == '0' && el IN {EL0, EL1, EL2} then
            return FALSE;
        end;

        if EL2Enabled() && !ELIsInHost(EL0) && HCR_EL2().ATA == '0' && el IN {EL0, EL1} then
            return FALSE;
        end;

        selected = TRUE;
    end;

    if selected then
        case regime of
            when Regime_EL3 =>
                return SCTLR_EL3().ATA == '1';
            when Regime_EL2 =>
                return SCTLR_EL2().ATA == '1';
            when Regime_EL20 =>
                return if el == EL0 then SCTLR_EL2().ATA0 == '1' else SCTLR_EL2().ATA == '1';
            when Regime_EL10 =>
                return if el == EL0 then SCTLR_EL1().ATA0 == '1' else SCTLR_EL1().ATA == '1';
            otherwise => unreachable;
        end;
    end;

    return FALSE;
end;
```

Library pseudocode for aarch64/functions/system/IsPMTESelected

```
// IsPMTESelected()
// ================
// Returns TRUE if Physical MTE is selected for the translation regime
// associated with the EL, and FALSE otherwise

readonly func IsPMTESelected(el : bits(2)) => boolean
begin
    if !IsFeatureImplemented(FEAT_MTE2) then
        return FALSE;
    end;

    return TRUE;
end;
```

Library pseudocode for aarch64/functions/system/IsSCTLR2EL1Enabled

```
// IsSCTLR2EL1Enabled()
// ====================
// Returns TRUE if access to SCTLR2_EL1 register is enabled, and FALSE otherwise.
// Indirect read of SCTLR2_EL1 returns 0 when access is not enabled.

readonly func IsSCTLR2EL1Enabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_SCTLR2) then return FALSE; end;
    if HaveEL(EL3) && SCR_EL3().SCTLR2En == '0' then
        return FALSE;
    elsif (EL2Enabled() && (!IsHCRXEL2Enabled() || HCRX_EL2().SCTLR2En == '0')) then
        return FALSE;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/functions/system/IsSCTLR2EL2Enabled

```
// IsSCTLR2EL2Enabled()
// ===================
// Returns TRUE if access to SCTLR2_EL2 register is enabled, and FALSE otherwise.
// Indirect read of SCTLR2_EL2 returns 0 when access is not enabled.

readonly func IsSCTLR2EL2Enabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_SCTLR2) then return FALSE; end;
    if HaveEL(EL3) && SCR_EL3().SCTLR2En == '0' then
        return FALSE;
    end;

    return EL2Enabled();
end;
```

Library pseudocode for aarch64/functions/system/IsSystemRegisterMaskingEnabled

```
// IsSystemRegisterMaskingEnabled()
// ================================
// Returns TRUE if access to the MASK registers introduced by FEAT_SRMASK is enabled and the
// values in the register are not treated as 0.

func IsSystemRegisterMaskingEnabled(el : bits(2)) => boolean
begin
    assert IsFeatureImplemented(FEAT_SRMASK) && el IN {EL1, EL2};

    if HaveEL(EL3) && SCR_EL3().SRMASKEn == '0' then
        return FALSE;
    end;

    return el == EL2 || !EL2Enabled() || (IsHCRXEL2Enabled() && HCRX_EL2().SRMASKEn == '1');
end;
```

Library pseudocode for aarch64/functions/system/IsTCR2EL1Enabled

```
// IsTCR2EL1Enabled()
// ==================
// Returns TRUE if access to TCR2_EL1 register is enabled, and FALSE otherwise.
// Indirect read of TCR2_EL1 returns 0 when access is not enabled.

readonly func IsTCR2EL1Enabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_TCR2) then return FALSE; end;
    if HaveEL(EL3) && SCR_EL3().TCR2En == '0' then
        return FALSE;
    elsif (EL2Enabled() && (!IsHCRXEL2Enabled() || HCRX_EL2().TCR2En == '0')) then
        return FALSE;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/functions/system/IsTCR2EL2Enabled

```
// IsTCR2EL2Enabled()
// ==================
// Returns TRUE if access to TCR2_EL2 register is enabled, and FALSE otherwise.
// Indirect read of TCR2_EL2 returns 0 when access is not enabled.

readonly func IsTCR2EL2Enabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_TCR2) then return FALSE; end;
    if HaveEL(EL3) && SCR_EL3().TCR2En == '0' then
        return FALSE;
    end;

    return EL2Enabled();
end;
```

Library pseudocode for aarch64/functions/system/PACInstType

```
// PACInstType
// ===========
type PACInstType of enumeration {
    PACIxSP,
    PACIxSPPC,
};
```

Library pseudocode for aarch64/functions/system/SetBTypeCompatible

```
// SetBTypeCompatible()
// ====================
// Sets the value of BTypeCompatible global variable used by BTI

func SetBTypeCompatible(x : boolean)
begin
    BTypeCompatible = x;
end;
```

Library pseudocode for aarch64/functions/system/SetBTypeNext

```
// SetBTypeNext()
// ==============
// Set the value of BTypeNext global variable used by BTI

func SetBTypeNext(x : bits(2))
begin
    BTypeNext = x;
end;
```

Library pseudocode for aarch64/functions/system/SetInGuardedPage

```
// SetInGuardedPage()
// ==================
// Global state updated to denote if memory access is from a guarded page.

func SetInGuardedPage(guardedpage : boolean)
begin
    InGuardedPage = guardedpage;
end;
```

Library pseudocode for aarch64/functions/system128/AArch64_SysInstr128

```
// AArch64_SysInstr128()
// =====================
// Execute a system instruction with write (2 64-bit source operands).

impdef func AArch64_SysInstr128(op0 : bits(2), op1 : bits(3), crn : bits(4),
                                crm : bits(4), op2 : bits(3), t : integer, t2 : integer)
begin
    Unimplemented();
end;
```

Library pseudocode for aarch64/functions/system128/AArch64_SysRegRead128

```
// AArch64_SysRegRead128()
// =======================
// Read from a 128-bit System register and write the contents of the register to X[t] and X[t2].

impdef func AArch64_SysRegRead128(op0 : bits(2), op1 : bits(3),
                                  crn : bits(4), crm : bits(4),
                                  op2 : bits(3), t : integer, t2 : integer)
begin
    Unimplemented();
end;
```

Library pseudocode for aarch64/functions/system128/AArch64_SysRegWrite128

```
// AArch64_SysRegWrite128()
// ========================
// Read the contents of X[t] and X[t2] and write the contents to a 128-bit System register.

impdef func AArch64_SysRegWrite128(op0 : bits(2), op1 : bits(3),
                                   crn : bits(4), crm : bits(4),
                                   op2 : bits(3), t : integer, t2 : integer)
begin
    Unimplemented();
end;
```

Library pseudocode for aarch64/functions/tlbi

```
constant DEFAULT_TLBI_DOMAIN : bits(16) = Zeros{};   // Broadcast to all agents in the
                                                     // corresponding broadcast domain
```

Library pseudocode for aarch64/functions/tlbi/AArch64_EffectiveBroadcast

```
// AArch64_EffectiveBroadcast()
// ============================

func AArch64_EffectiveBroadcast(broadcast_in : Broadcast) => Broadcast
begin
    var broadcast : Broadcast = broadcast_in;

    if PSTATE.EL != EL1 then return broadcast; end;

    if IsFeatureImplemented(FEAT_TLBID) then
        let fnb : bit = if IsHCRXEL2Enabled() then HCRX_EL2().FNB else '0';
        case fnb::HCR_EL2().FB of
            when '00' =>
                pass;
            when '01' =>
                if broadcast == Broadcast_NSH then
                    // The operation is broadcast within the Inner Shareable shareability domain.
                    broadcast = Broadcast_ForcedISH;
                end;
            when '10' =>
                if broadcast == Broadcast_ISH then
                    // The operation only affects the PE on which the instruction is executed.
                    broadcast = Broadcast_NSH;
                end;
            when '11' =>
                if broadcast == Broadcast_ISH then
                    // The operation is broadcast to PEs which would share TLB entries with the
                    // current PE if CnP were 1.
                    broadcast = Broadcast_ISH_CnP;
                end;
        end;
    else
        if broadcast == Broadcast_NSH && EL2Enabled() && HCR_EL2().FB == '1' then
            broadcast = Broadcast_ForcedISH;
        end;
    end;

    return broadcast;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_IPAS2

```
// AArch64_TLBIP_IPAS2()
// =====================
// Invalidate by IPA all stage 2 only TLB entries in the indicated broadcast
// domain matching the indicated VMID in the indicated regime with the indicated security state.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// IPA and related parameters of the are derived from Xt.

func AArch64_TLBIP_IPAS2(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                         broadcast_in : Broadcast, level : TLBILevel, attr : TLBIMemAttr,
                         Xt : bits(128))
begin
    assert PSTATE.EL IN {EL3, EL2};
    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op           = TLBIOp_IPAS2;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = TRUE;
    r.level        = level;
    r.attr         = attr;
    r.ttl          = Xt[47:44];
    r.address      = ZeroExtend{64}(Xt[107:64] :: Zeros{12});
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64      = TRUE;
        r.d128     = r.ttl == '00xx';
    else
        r.d64      = r.ttl == '00xx';
        r.d128     = TRUE;
    end;

    case security of
        when SS_NonSecure =>
            r.ipaspace = PAS_NonSecure;
        when SS_Secure =>
            r.ipaspace = if Xt[63] == '1' then PAS_NonSecure else PAS_Secure;
        when SS_Realm =>
            r.ipaspace = PAS_Realm;
        otherwise =>
            // Root security state does not have stage 2 translation
            unreachable;
    end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_RIPAS2

```
// AArch64_TLBIP_RIPAS2()
// ======================
// Range invalidate by IPA all stage 2 only TLB entries in the indicated
// broadcast domain matching the indicated VMID in the indicated regime with the indicated
// security state.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// The range of IPA and related parameters of the are derived from Xt.

func AArch64_TLBIP_RIPAS2(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                          broadcast_in : Broadcast, level : TLBILevel, attr : TLBIMemAttr,
                          Xt : bits(128))
begin
    assert PSTATE.EL IN {EL3, EL2};
    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op           = TLBIOp_RIPAS2;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = TRUE;
    r.level        = level;
    r.attr         = attr;
    r.ttl[1:0]     = Xt[38:37];
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64      = TRUE;
        r.d128     = r.ttl[1:0] == '00';
    else
        r.d64      = r.ttl[1:0] == '00';
        r.d128     = TRUE;
    end;

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIPRange(regime, Xt);

    if !valid then return; end;

    case security of
        when SS_NonSecure =>
            r.ipaspace = PAS_NonSecure;
        when SS_Secure =>
            r.ipaspace = if Xt[63] == '1' then PAS_NonSecure else PAS_Secure;
        when SS_Realm =>
            r.ipaspace = PAS_Realm;
        otherwise =>
            // Root security state does not have stage 2 translation
            unreachable;
    end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_RVA

```
// AArch64_TLBIP_RVA()
// ===================
// Range invalidate by VA range all stage 1 TLB entries in the indicated
// broadcast domain matching the indicated VMID and ASID (where regime
// supports VMID, ASID) in the indicated regime with the indicated security state.
// ASID, and range related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBIP_RVA(security : SecurityState, regime_in : Regime, vmid_in : bits(NUM_VMIDBITS),
                       broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
                       Xt : bits(128))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast   = AArch64_EffectiveBroadcast(broadcast_in);
    var attr      : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime   = regime_in;
    var vmid   : bits(NUM_VMIDBITS) = vmid_in;

    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid   = VMID_NONE;
    elsif regime == Regime_EL2 && ELIsInHost(EL2) then
        regime = Regime_EL20;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_RVA;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.asid         = Xt[63:48];
    r.ttl[1:0]     = Xt[38:37];
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64      = TRUE;
        r.d128     = r.ttl[1:0] == '00';
    else
        r.d64      = r.ttl[1:0] == '00';
        r.d128     = TRUE;
    end;

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIPRange(regime, Xt);

    if !valid then return; end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_RVAA

```
// AArch64_TLBIP_RVAA()
// ====================
// Range invalidate by VA range all stage 1 TLB entries in the indicated
// broadcast domain matching the indicated VMID (where regimesupports VMID)
// and all ASID in the indicated regime with the indicated security state.
// VA range related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBIP_RVAA(security : SecurityState, regime_in : Regime, vmid_in : bits(NUM_VMIDBITS),
                        broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
                        Xt : bits(128))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast   = AArch64_EffectiveBroadcast(broadcast_in);
    var attr      : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime   = regime_in;
    var vmid   : bits(NUM_VMIDBITS) = vmid_in;

    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid   = VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_RVAA;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.ttl[1:0]     = Xt[38:37];
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64      = TRUE;
        r.d128     = r.ttl[1:0] == '00';
    else
        r.d64      = r.ttl[1:0] == '00';
        r.d128     = TRUE;
    end;

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIPRange(regime, Xt);

    if !valid then return; end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_VA

```
// AArch64_TLBIP_VA()
// ==================
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID and ASID (where regime supports VMID, ASID) in the indicated regime
// with the indicated security state.
// ASID, VA and related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBIP_VA(security : SecurityState, regime_in : Regime, vmid_in : bits(NUM_VMIDBITS),
                      broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
                      Xt : bits(128))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast   = AArch64_EffectiveBroadcast(broadcast_in);
    var attr      : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime   = regime_in;
    var vmid   : bits(NUM_VMIDBITS) = vmid_in;

    if regime == Regime_EL10 && ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid   = VMID_NONE;
    elsif regime == Regime_EL2 && ELIsInHost(EL2) then
        regime = Regime_EL20;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_VA;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.asid         = Xt[63:48];
    r.ttl          = Xt[47:44];
    r.address      = ZeroExtend{64}(Xt[107:64] :: Zeros{12});
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64      = TRUE;
        r.d128     = r.ttl == '00xx';
    else
        r.d64      = r.ttl == '00xx';
        r.d128     = TRUE;
    end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBIP_VAA

```
// AArch64_TLBIP_VAA()
// ===================
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID (where regime supports VMID) and all ASID in the indicated regime
// with the indicated security state.
// VA and related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBIP_VAA(security : SecurityState, regime_in : Regime, vmid_in : bits(NUM_VMIDBITS),
                       broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
                       Xt : bits(128))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast   = AArch64_EffectiveBroadcast(broadcast_in);
    var attr      : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime   = regime_in;
    var vmid   : bits(NUM_VMIDBITS) = vmid_in;
    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid   = VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_VAA;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.ttl          = Xt[47:44];
    r.address      = ZeroExtend{64}(Xt[107:64] :: Zeros{12});
    if IsFeatureImplemented(FEAT_TLBID) && Xt[32] == '1' then
        r.d64      = TRUE;
        r.d128     = r.ttl == '00xx';
    else
        r.d64      = r.ttl == '00xx';
        r.d128     = TRUE;
    end;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_ALL

```
// AArch64_TLBI_ALL()
// ==================
// Invalidate all entries for the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast domain.
// Invalidation applies to all applicable stage 1 and stage 2 entries.

func AArch64_TLBI_ALL(security : SecurityState, regime : Regime,
                      broadcast_in : Broadcast, attr : TLBIMemAttr, Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2};
    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op           = TLBIOp_ALL;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.level        = TLBILevel_Any;
    r.attr         = attr;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_ASID

```
// AArch64_TLBI_ASID()
// ===================
// Invalidate all stage 1 entries matching the indicated VMID (where regime supports)
// and ASID in the parameter Xt in the indicated translation regime with the
// indicated security state for all TLBs within the indicated broadcast domain.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBI_ASID(security : SecurityState, regime_in : Regime, vmid_in : bits(NUM_VMIDBITS),
                       broadcast_in : Broadcast, attr_in : TLBIMemAttr,
                       Xt :  bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast    = AArch64_EffectiveBroadcast(broadcast_in);
    var attr      : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime   = regime_in;
    var vmid   : bits(NUM_VMIDBITS) = vmid_in;

    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid   = VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_ASID;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = TLBILevel_Any;
    r.attr         = attr;
    r.asid         = Xt[63:48];

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_IPAS2

```
// AArch64_TLBI_IPAS2()
// ====================
// Invalidate by IPA all stage 2 only TLB entries in the indicated broadcast
// domain matching the indicated VMID in the indicated regime with the indicated security state.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// IPA and related parameters of the are derived from Xt.

func AArch64_TLBI_IPAS2(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                        broadcast_in : Broadcast, level : TLBILevel, attr : TLBIMemAttr,
                        Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2};
    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op           = TLBIOp_IPAS2;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = TRUE;
    r.level        = level;
    r.attr         = attr;
    r.ttl          = Xt[47:44];
    r.address      = ZeroExtend{64}(Xt[39:0] :: Zeros{12});
    r.d64          = TRUE;
    r.d128         = r.ttl == '00xx';

    case security of
        when SS_NonSecure =>
            r.ipaspace = PAS_NonSecure;
        when SS_Secure =>
            r.ipaspace = if Xt[63] == '1' then PAS_NonSecure else PAS_Secure;
        when SS_Realm =>
            r.ipaspace = PAS_Realm;
        otherwise =>
            // Root security state does not have stage 2 translation
            unreachable;
    end;

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH() then
        broadcast = Broadcast_OSHnISH;
    end;
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_PAALL

```
// AArch64_TLBI_PAALL()
// ====================
// TLB Invalidate ALL GPT Information.
// Invalidates cached copies of GPT entries from TLBs in the indicated
// Shareability domain.
// The invalidation applies to all TLB entries containing GPT information.

func AArch64_TLBI_PAALL(broadcast : Broadcast)
begin
    assert IsFeatureImplemented(FEAT_RME) && PSTATE.EL == EL3;

    var r : TLBIRecord;

    // r.security and r.regime do not apply for TLBI by PA operations
    r.op    = TLBIOp_PAALL;
    r.level = TLBILevel_Any;
    r.attr  = TLBI_AllAttr;

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_RIPAS2

```
// AArch64_TLBI_RIPAS2()
// =====================
// Range invalidate by IPA all stage 2 only TLB entries in the indicated
// broadcast domain matching the indicated VMID in the indicated regime with the indicated
// security state.
// Note: stage 1 and stage 2 combined entries are not in the scope of this operation.
// The range of IPA and related parameters of the are derived from Xt.

func AArch64_TLBI_RIPAS2(security : SecurityState, regime : Regime, vmid : bits(NUM_VMIDBITS),
                         broadcast_in : Broadcast, level : TLBILevel, attr : TLBIMemAttr,
                         Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2};
    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op           = TLBIOp_RIPAS2;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = TRUE;
    r.level        = level;
    r.attr         = attr;
    r.ttl[1:0]     = Xt[38:37];
    r.d64          = TRUE;
    r.d128         = r.ttl[1:0] == '00';

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIRange(regime, Xt);

    if !valid then return; end;

    case security of
        when SS_NonSecure =>
            r.ipaspace = PAS_NonSecure;
        when SS_Secure =>
            r.ipaspace = if Xt[63] == '1' then PAS_NonSecure else PAS_Secure;
        when SS_Realm =>
            r.ipaspace = PAS_Realm;
        otherwise =>
            // Root security state does not have stage 2 translation
            unreachable;
    end;

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH() then
        broadcast = Broadcast_OSHnISH;
    end;
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_RPA

```
// AArch64_TLBI_RPA()
// ==================
// TLB Range Invalidate GPT Information by PA.
// Invalidates cached copies of GPT entries from TLBs in the indicated
// Shareability domain.
// The invalidation applies to TLB entries containing GPT information relating
// to the indicated physical address range.
// When the indicated level is
//     TLBILevel_Any  : this applies to TLB entries containing GPT information
//                      from all levels of the GPT walk
//     TLBILevel_Last : this applies to TLB entries containing GPT information
//                      from the last level of the GPT walk

func AArch64_TLBI_RPA(level : TLBILevel, Xt : bits(64), broadcast : Broadcast)
begin
    assert IsFeatureImplemented(FEAT_RME) && PSTATE.EL == EL3;

    var r : TLBIRecord;
    var range_bits : AddressSize;
    var p : AddressSize;

    // r.security and r.regime do not apply for TLBI by PA operations
    r.op    = TLBIOp_RPA;
    r.level = level;
    r.attr  = TLBI_AllAttr;

    // SIZE field
    case Xt[47:44] of
        when '0000' => range_bits = 12; // 4KB
        when '0001' => range_bits = 14; // 16KB
        when '0010' => range_bits = 16; // 64KB
        when '0011' => range_bits = 21; // 2MB
        when '0100' => range_bits = 25; // 32MB
        when '0101' => range_bits = 29; // 512MB
        when '0110' => range_bits = 30; // 1GB
        when '0111' => range_bits = 34; // 16GB
        when '1000' => range_bits = 36; // 64GB
        when '1001' => range_bits = 39; // 512GB
        otherwise =>   return;  // Reserved encoding, no TLB entries are required to be invalidated
    end;

    // If SIZE selects a range smaller than PGS, then PGS is used instead
    case DecodePGS(GPCCR_EL3().PGS) of
        when PGS_4KB =>  p = 12;
        when PGS_16KB => p = 14;
        when PGS_64KB => p = 16;
        otherwise => return;    // Reserved encoding, no TLB entries are required to be invalidated
    end;

    if range_bits < p then
        range_bits = p;
    end;

    var BaseADDR : bits(NUM_PABITS) = Zeros{};
    case GPCCR_EL3().PGS of
        when '00' => BaseADDR[51:12] = Xt[39:0];   // 4KB
        when '10' => BaseADDR[51:14] = Xt[39:2];   // 16KB
        when '01' => BaseADDR[51:16] = Xt[39:4];   // 64KB
    end;

    if IsFeatureImplemented(FEAT_D128) && AArch64_PAMax() == 56 then
        BaseADDR[55:52] = Xt[43:40]; // Extension to Address
    end;

    // The calculation here automatically aligns BaseADDR to the size of
    // the region specified in SIZE. However, the architecture does not
    // require this alignment and if BaseADDR is not aligned to the region
    // specified by SIZE then no entries are required to be invalidated.
    let range_pbits : integer{} = range_bits;
    let start_addr : bits(NUM_PABITS) = BaseADDR AND NOT ZeroExtend{NUM_PABITS}(Ones{range_pbits});
    let end_addr : bits(NUM_PABITS)   = start_addr + ZeroExtend{NUM_PABITS}(Ones{range_pbits});

    // PASpace is not considered in TLBI by PA operations
    r.address     = ZeroExtend{64}(start_addr);
    r.end_address = ZeroExtend{64}(end_addr);

    TLBI(r);
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_RVA

```
// AArch64_TLBI_RVA()
// ==================
// Range invalidate by VA range all stage 1 TLB entries in the indicated
// broadcast domain matching the indicated VMID and ASID (where regime
// supports VMID, ASID) in the indicated regime with the indicated security state.
// ASID, and range related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBI_RVA(security : SecurityState, regime_in : Regime, vmid_in : bits(NUM_VMIDBITS),
                      broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
                      Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast   = AArch64_EffectiveBroadcast(broadcast_in);
    var attr      : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime   = regime_in;
    var vmid   : bits(NUM_VMIDBITS) = vmid_in;

    if regime == Regime_EL10 && ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid   = VMID_NONE;
    elsif regime == Regime_EL2 && ELIsInHost(EL2) then
        regime = Regime_EL20;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_RVA;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.asid         = Xt[63:48];
    r.ttl[1:0]     = Xt[38:37];
    r.d64          = TRUE;
    r.d128         = r.ttl[1:0] == '00';

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIRange(regime, Xt);

    if !valid then return; end;

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH() then
        broadcast = Broadcast_OSHnISH;
    end;
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_RVAA

```
// AArch64_TLBI_RVAA()
// ===================
// Range invalidate by VA range all stage 1 TLB entries in the indicated
// broadcast domain matching the indicated VMID (where regimesupports VMID)
// and all ASID in the indicated regime with the indicated security state.
// VA range related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBI_RVAA(security : SecurityState, regime_in : Regime, vmid_in : bits(NUM_VMIDBITS),
                       broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
                       Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast   = AArch64_EffectiveBroadcast(broadcast_in);
    var attr      : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime   = regime_in;
    var vmid   : bits(NUM_VMIDBITS) = vmid_in;

    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid   = VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_RVAA;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.ttl[1:0]     = Xt[38:37];
    r.d64          = TRUE;
    r.d128         = r.ttl[1:0] == '00';

    var valid : boolean;
    (valid, r.tg, r.address, r.end_address) = TLBIRange(regime, Xt);

    if !valid then return; end;

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH() then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;
    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_VA

```
// AArch64_TLBI_VA()
// =================
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID and ASID (where regime supports VMID, ASID) in the indicated regime
// with the indicated security state.
// ASID, VA and related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBI_VA(security : SecurityState, regime_in : Regime, vmid_in : bits(NUM_VMIDBITS),
                     broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
                     Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast   = AArch64_EffectiveBroadcast(broadcast_in);
    var attr      : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime   = regime_in;
    var vmid   : bits(NUM_VMIDBITS) = vmid_in;
    if regime == Regime_EL10 && ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid   = VMID_NONE;
    elsif regime == Regime_EL2 && ELIsInHost(EL2) then
        regime = Regime_EL20;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_VA;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.asid         = Xt[63:48];
    r.ttl          = Xt[47:44];
    r.address      = ZeroExtend{64}(Xt[43:0] :: Zeros{12});
    r.d64          = TRUE;
    r.d128         = r.ttl == '00xx';

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH() then
        broadcast = Broadcast_OSHnISH;
    end;
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_VAA

```
// AArch64_TLBI_VAA()
// ==================
// Invalidate by VA all stage 1 TLB entries in the indicated broadcast domain
// matching the indicated VMID (where regime supports VMID) and all ASID in the indicated regime
// with the indicated security state.
// VA and related parameters are derived from Xt.
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.

func AArch64_TLBI_VAA(security : SecurityState, regime_in : Regime, vmid_in : bits(NUM_VMIDBITS),
                      broadcast_in : Broadcast, level : TLBILevel, attr_in : TLBIMemAttr,
                      Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast   = AArch64_EffectiveBroadcast(broadcast_in);
    var attr      : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime   = regime_in;
    var vmid   : bits(NUM_VMIDBITS) = vmid_in;
    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid   = VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_VAA;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.level        = level;
    r.attr         = attr;
    r.ttl          = Xt[47:44];
    r.address      = ZeroExtend{64}(Xt[43:0] :: Zeros{12});
    r.d64          = TRUE;
    r.d128         = r.ttl == '00xx';

    TLBI(r);
    if broadcast == Broadcast_OSH && IsBroadcast_OSHnISH() then
        broadcast = Broadcast_OSHnISH;
    end;
    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_VMALL

```
// AArch64_TLBI_VMALL()
// ====================
// Invalidate all stage 1 entries for the indicated translation regime with the
// the indicated security state for all TLBs within the indicated broadcast
// domain that match the indicated VMID (where applicable).
// Note: stage 1 and stage 2 combined entries are in the scope of this operation.
// Note: stage 2 only entries are not in the scope of this operation.

func AArch64_TLBI_VMALL(security : SecurityState, regime_in : Regime,
                        vmid_in : bits(NUM_VMIDBITS), broadcast_in : Broadcast,
                        attr_in : TLBIMemAttr, Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2, EL1};
    var broadcast : Broadcast   = AArch64_EffectiveBroadcast(broadcast_in);
    var attr      : TLBIMemAttr = attr_in;

    if attr == TLBI_AllAttr && ExcludeXS() then
        attr = TLBI_ExcludeXS;
    end;

    var regime : Regime   = regime_in;
    var vmid   : bits(NUM_VMIDBITS) = vmid_in;

    if ELIsInHost(EL0) then
        regime = Regime_EL20;
        vmid   = VMID_NONE;
    end;

    var r : TLBIRecord;
    r.op           = TLBIOp_VMALL;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.level        = TLBILevel_Any;
    r.vmid         = vmid;
    r.use_vmid     = UseVMID(regime);
    r.attr         = attr;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_VMALLS12

```
// AArch64_TLBI_VMALLS12()
// =======================
// Invalidate all stage 1 and stage 2 entries for the indicated translation
// regime with the indicated security state for all TLBs within the indicated
// broadcast domain that match the indicated VMID.

func AArch64_TLBI_VMALLS12(security : SecurityState, regime : Regime,
                           vmid : bits(NUM_VMIDBITS), broadcast_in : Broadcast,
                           attr : TLBIMemAttr, Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2};
    var broadcast : Broadcast = AArch64_EffectiveBroadcast(broadcast_in);

    var r : TLBIRecord;
    r.op           = TLBIOp_VMALLS12;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.level        = TLBILevel_Any;
    r.vmid         = vmid;
    r.use_vmid     = TRUE;
    r.attr         = attr;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/AArch64_TLBI_VMALLWS2

```
// AArch64_TLBI_VMALLWS2()
// =======================
// Remove stage 2 dirty state from entries for the indicated translation regime
// with the indicated security state for all TLBs within the indicated broadcast
// domain that match the indicated VMID.

func AArch64_TLBI_VMALLWS2(security : SecurityState, regime : Regime, vmid : bits(16),
                           broadcast_in : Broadcast, attr : TLBIMemAttr,
                           Xt : bits(64))
begin
    assert PSTATE.EL IN {EL3, EL2};
    assert regime == Regime_EL10;

    if security == SS_Secure && HaveEL(EL3) && SCR_EL3().EEL2 == '0' then
        return;
    end;

    var broadcast : Broadcast = broadcast_in;

    var r : TLBIRecord;
    r.op           = TLBIOp_VMALLWS2;
    r.from_aarch64 = TRUE;
    r.security     = security;
    r.regime       = regime;
    r.level        = TLBILevel_Any;
    r.vmid         = vmid;
    r.use_vmid     = TRUE;
    r.attr         = attr;

    TLBI(r);
    let domains : bits(16) = TLBIDomains(broadcast, Xt[15:0]);
    if (broadcast == Broadcast_OSH &&
          (IsBroadcast_OSHnISH() || OSHDomainExceedsNIS(domains))) then
        broadcast = Broadcast_OSHnISH;
    end;

    if broadcast != Broadcast_NSH then BroadcastTLBI(broadcast, r, domains); end;

    return;
end;
```

Library pseudocode for aarch64/functions/tlbi/DecodeTLBITG

```
// DecodeTLBITG()
// ==============
// Decode translation granule size in TLBI range instructions

func DecodeTLBITG(tg : bits(2)) => TGx
begin
    case tg of
        when '01' =>   return TGx_4KB;
        when '10' =>   return TGx_16KB;
        when '11' =>   return TGx_64KB;
    end;
end;
```

Library pseudocode for aarch64/functions/tlbi/GPTTLBIMatch

```
// GPTTLBIMatch()
// ==============
// Determine whether the GPT TLB entry lies within the scope of invalidation

func GPTTLBIMatch(tlbi : TLBIRecord, gpt_entry : GPTEntry) => boolean
begin
    assert tlbi.op IN {TLBIOp_RPA, TLBIOp_PAALL};

    var match : boolean;
    let entry_size_mask : bits(64)     = ZeroExtend{}(Ones{gpt_entry.size});
    let entry_end_address : bits(64)   = (ZeroExtend{64}(gpt_entry.pa[NUM_PABITS-1:0] OR
                                             entry_size_mask[NUM_PABITS-1:0]));
    let entry_start_address : bits(64) = (ZeroExtend{64}(gpt_entry.pa[NUM_PABITS-1:0] AND NOT
                                             entry_size_mask[NUM_PABITS-1:0]));

    case tlbi.op of
        when TLBIOp_RPA =>
            match = ((UInt(tlbi.address[NUM_PABITS-1:0]) <=
                        UInt(entry_end_address[NUM_PABITS-1:0])) &&
                     (UInt(tlbi.end_address[NUM_PABITS-1:0]) >
                        UInt(entry_start_address[NUM_PABITS-1:0])) &&
                     (tlbi.level == TLBILevel_Any || !gpt_entry.istable));
        when TLBIOp_PAALL =>
            match = TRUE;
    end;

    return match;
end;
```

Library pseudocode for aarch64/functions/tlbi/HasLargeAddress

```
// HasLargeAddress()
// =================
// Returns TRUE if the regime is configured for 52 bit addresses, FALSE otherwise.

func HasLargeAddress(regime : Regime) => boolean
begin
    if !IsFeatureImplemented(FEAT_LPA2) then
        return FALSE;
    end;
    case regime of
        when Regime_EL3 =>
            return TCR_EL3().DS == '1';
        when Regime_EL2 =>
            return TCR_EL2().E2H0.DS == '1';
        when Regime_EL20 =>
            return TCR_EL2().E2H1.DS == '1';
        when Regime_EL10 =>
            return TCR_EL1().DS == '1';
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for aarch64/functions/tlbi/IsBroadcast_OSHnISH

```
// IsBroadcast_OSHnISH()
// =====================
// Returns TRUE if a TLBI *OS* instruction affects TLBs in the Outer shareability domain,
// but not in the Inner shareability domain.

readonly func IsBroadcast_OSHnISH() => boolean
begin
    case PSTATE.EL of
        when EL3  => return FALSE;
        when EL2  => return IsSCTLR2EL2Enabled() && SCTLR2_EL2().TLBOSNIS == '1';
        when EL1  => return IsSCTLR2EL1Enabled() && SCTLR2_EL1().TLBOSNIS == '1';
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for aarch64/functions/tlbi/OSHDomainExceedsNIS

```
// OSHDomainExceedsNIS()
// =====================
// Returns TRUE if a TLBI *OS* instruction affects TLBs in the Outer shareability domain,
// but not in the Inner shareability domain.

readonly func OSHDomainExceedsNIS(domains : bits(16)) => boolean
begin
    // If the TLBID value exceeds the supported value in TLBIDIDR_EL1.NIS, then no TLBs in the
    // Inner Shareable shareability domain are guaranteed to be affected by the TLBI operation.
    let nis : integer{} = UInt(TLBIDIDR_EL1().NIS);
    return nis < 16 && !IsZero(domains[15:nis]);
end;
```

Library pseudocode for aarch64/functions/tlbi/ResTLBIRTTL

```
// ResTLBIRTTL()
// =============
// Determine whether the TTL field in TLBI instructions that do apply
// to a range of addresses contains a reserved value

func ResTLBIRTTL(tg : bits(2), ttl : bits(2)) => boolean
begin
    case ttl of
        when '00' => return TRUE;
        when '01' => return DecodeTLBITG(tg) == TGx_16KB && !IsFeatureImplemented(FEAT_LPA2);
        otherwise => return FALSE;
    end;
end;
```

Library pseudocode for aarch64/functions/tlbi/ResTLBITTL

```
// ResTLBITTL()
// ============
// Determine whether the TTL field in TLBI instructions that do not apply
// to a range of addresses contains a reserved value

func ResTLBITTL(ttl : bits(4)) => boolean
begin
    case ttl of
        when '00xx' => return TRUE;
        when '0100' => return !IsFeatureImplemented(FEAT_LPA2);
        when '1000' => return TRUE;
        when '1001' => return !IsFeatureImplemented(FEAT_LPA2);
        when '1100' => return TRUE;
        otherwise =>   return FALSE;
    end;
end;
```

Library pseudocode for aarch64/functions/tlbi/TGBits

```
// TGBits()
// ========
// Return the number of least-significant address bits within a single Translation Granule.

func TGBits(tg : bits(2)) => AddressSize
begin
    case tg of
        when '01' => return 12; // 4KB
        when '10' => return 14; // 16KB
        when '11' => return 16; // 64KB
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for aarch64/functions/tlbi/TLBIDomains

```
// TLBIDomains()
// =============
// Determine the domains the TLBI operation is broadcast to

readonly func TLBIDomains(broadcast_in : Broadcast, tlbid_in : bits(16)) => bits(16)
begin
    assert broadcast_in != Broadcast_OSHnISH;

    var broadcast : Broadcast = broadcast_in;
    var tlbid : bits(16)      = tlbid_in;

    if !IsFeatureImplemented(FEAT_TLBID)        then return DEFAULT_TLBI_DOMAIN; end;
    if broadcast == Broadcast_NSH               then return DEFAULT_TLBI_DOMAIN; end;
    if broadcast == Broadcast_ISH_CnP           then return DEFAULT_TLBI_DOMAIN; end;
    if PSTATE.EL == EL3                         then return DEFAULT_TLBI_DOMAIN; end;
    if PSTATE.EL == EL0 && HCR_EL2().TGE == '1' then return DEFAULT_TLBI_DOMAIN; end;

    let nos : integer{} = UInt(TLBIDIDR_EL1().NOS);
    assert nos <= 16;
    if broadcast == Broadcast_OSH && nos == 0 then
        return DEFAULT_TLBI_DOMAIN;
    end;

    let nis : integer{} = UInt(TLBIDIDR_EL1().NIS);
    assert nis <= 16;
    if broadcast == Broadcast_ISH && nis == 0 then
        return DEFAULT_TLBI_DOMAIN;
    end;

    if PSTATE.EL == EL2 then
        if broadcast == Broadcast_OSH then
            return ZeroExtend{16}(tlbid[nos-1:0]);
        else    // Broadcast_ISH
            return ZeroExtend{16}(tlbid[nis-1:0]);
        end;
    end;

    assert PSTATE.EL == EL1 || (PSTATE.EL == EL0 && HCR_EL2().TGE == '0');

    if broadcast == Broadcast_ForcedISH then
        // For a TLBI instruction without an IS or OS qualifier executed at EL1, if HCR_EL2.FB
        // is 1 then the process for a TLBI IS instruction is followed as though TLBID is zero.
        broadcast = Broadcast_ISH;
        tlbid     = Zeros{16};
    end;

    assert broadcast IN {Broadcast_ISH, Broadcast_OSH};

    // If the Effective value of VTLBID(OS)En is 0 the TLBID field is not transformed.
    if ((HaveEL(EL3) && SCR_EL3().VTLBIDEn == '0') || !IsHCRXEL2Enabled() ||
          (broadcast == Broadcast_OSH && HCRX_EL2().VTLBIDOSEn == '0') ||
          (broadcast == Broadcast_ISH && HCRX_EL2().VTLBIDEn == '0')) then
        if broadcast == Broadcast_OSH then
            return ZeroExtend{16}(tlbid[nos-1:0]);
        else    // Broadcast_ISH
            return ZeroExtend{16}(tlbid[nis-1:0]);
        end;
    end;

    if broadcast == Broadcast_OSH then
        let nvos : integer{} = UInt(TLBIDIDR_EL1().NVOS);
        assert (nos <= 8 && nvos <= 5) || (nos <= 16 && nvos <= 4);

        // If NVOS is 0, the TLBID field is ignored as only VTLBIDOS[n]_EL2.TD0 is implemented.
        let d : integer{} = if nvos == 0 then 0 else UInt(tlbid[nvos-1:0]);
        let s : integer{} = if nos <= 8 then 8 else 4;
        let n : integer = d DIVRM s;
        let m : integer = d MOD s;

        var vtlbid : bits(16);
        if nos <= 8 then
            vtlbid = ZeroExtend{16}(VTLBIDOS_EL2(n)[8*m+:8]);
        else
            vtlbid = VTLBIDOS_EL2(n)[16*m+:16];
        end;

        return ZeroExtend{16}(vtlbid[nos-1:0]);
    else    // Broadcast_ISH
        let nvis : integer{} = UInt(TLBIDIDR_EL1().NVIS);
        assert (nis <= 8 && nvis <= 5) || (nis <= 16 && nvis <= 4);

        // If NVIS is 0, the TLBID field is ignored as only VTLBID[n]_EL2.TD0 is implemented.
        let d : integer{} = if nvis == 0 then 0 else UInt(tlbid[nvis-1:0]);
        let s : integer{} = if nis <= 8 then 8 else 4;
        let n : integer = d DIVRM s;
        let m : integer = d MOD s;

        var vtlbid : bits(16);
        if nis <= 8 then
```

```
            vtlbid = ZeroExtend{16}(VTLBID_EL2(n)[8*m+:8]);
        else
            vtlbid = VTLBID_EL2(n)[16*m+:16];
        end;

        return ZeroExtend{16}(vtlbid[nis-1:0]);
    end;
end;
```

Library pseudocode for aarch64/functions/tlbi/TLBIMatch

```
// TLBIMatch()
// ===========
// Determine whether the TLB entry lies within the scope of invalidation

func TLBIMatch(tlbi : TLBIRecord, tlb_entry : TLBRecord) => boolean
begin
    var match : boolean;
    let entry_block_mask : bits(64)    = ZeroExtend{}(Ones{tlb_entry.blocksize});
    var entry_end_address : bits(64)   = tlb_entry.context.ia OR entry_block_mask;
    var entry_start_address : bits(64) = tlb_entry.context.ia AND NOT entry_block_mask;
    case tlbi.op of
        when TLBIOp_DALL, TLBIOp_IALL =>
            match = (tlbi.security == tlb_entry.context.ss &&
                     tlbi.regime   == tlb_entry.context.regime &&
                     tlbi.use_vmid == tlb_entry.context.use_vmid &&
                     (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid));
        when TLBIOp_DASID, TLBIOp_IASID =>
            match = (tlb_entry.context.includes_s1 &&
                     tlbi.security == tlb_entry.context.ss &&
                     tlbi.regime   == tlb_entry.context.regime &&
                     tlbi.use_vmid == tlb_entry.context.use_vmid &&
                     (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid) &&
                     (UseASID(tlb_entry.context) && tlb_entry.context.nG == '1' &&
                       tlbi.asid  == tlb_entry.context.asid));
        when TLBIOp_DVA, TLBIOp_IVA =>
            var regime_match : boolean;
            var context_match : boolean;
            var address_match : boolean;
            var level_match : boolean;
            regime_match = (tlb_entry.context.includes_s1 &&
                            tlbi.security == tlb_entry.context.ss &&
                            tlbi.regime   == tlb_entry.context.regime);
            context_match = (tlbi.use_vmid == tlb_entry.context.use_vmid &&
                             (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid) &&
                             (!UseASID(tlb_entry.context) || tlbi.asid == tlb_entry.context.asid ||
                                tlb_entry.context.nG == '0'));
            let addr_lsb : integer{} = tlb_entry.blocksize;
            address_match = tlbi.address[55:addr_lsb] == tlb_entry.context.ia[55:addr_lsb];
            level_match = (tlbi.level == TLBILevel_Any || !tlb_entry.walkstate.istable);
            match = regime_match && context_match  && address_match  && level_match;
        when TLBIOp_ALL =>
            let relax_regime : boolean = (tlbi.from_aarch64 &&
                            tlbi.regime IN {Regime_EL20, Regime_EL2} &&
                            tlb_entry.context.regime IN {Regime_EL20, Regime_EL2});
            match = (tlbi.security == tlb_entry.context.ss &&
                     (tlbi.regime  == tlb_entry.context.regime || relax_regime));
        when TLBIOp_ASID =>
            match = (tlb_entry.context.includes_s1 &&
                     tlbi.security == tlb_entry.context.ss &&
                     tlbi.regime   == tlb_entry.context.regime &&
                     tlbi.use_vmid == tlb_entry.context.use_vmid &&
                     (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid) &&
                     (UseASID(tlb_entry.context) && tlb_entry.context.nG == '1' &&
                       tlbi.asid  == tlb_entry.context.asid));
        when TLBIOp_IPAS2, TLBIPOp_IPAS2 =>
            let addr_lsb : integer{} = tlb_entry.blocksize;
            match = (!tlb_entry.context.includes_s1 && tlb_entry.context.includes_s2 &&
                     tlbi.security == tlb_entry.context.ss &&
                     tlbi.regime   == tlb_entry.context.regime &&
                     (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid) &&
                     tlbi.ipaspace == tlb_entry.context.ipaspace &&
                     (tlbi.address[NUM_PABITS-1:addr_lsb] ==
                        tlb_entry.context.ia[NUM_PABITS-1:addr_lsb]) &&
                     (!tlbi.from_aarch64 || ResTLBITTL(tlbi.ttl) || (
                         DecodeTLBITG(tlbi.ttl[3:2]) == tlb_entry.context.tg &&
                         UInt(tlbi.ttl[1:0]) == tlb_entry.walkstate.level)
                     ) &&
                     ((tlbi.d128  && tlb_entry.context.isd128) ||
                      (tlbi.d64  && !tlb_entry.context.isd128) ||
                      (tlbi.d64 && tlbi.d128)) &&
                     (tlbi.level == TLBILevel_Any || !tlb_entry.walkstate.istable));
        when TLBIOp_VAA, TLBIPOp_VAA =>
            let addr_lsb : integer{} = tlb_entry.blocksize;
            match = (tlb_entry.context.includes_s1 &&
                     tlbi.security == tlb_entry.context.ss &&
                     tlbi.regime   == tlb_entry.context.regime &&
                     tlbi.use_vmid == tlb_entry.context.use_vmid &&
                     (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid) &&
                     tlbi.address[55:addr_lsb] == tlb_entry.context.ia[55:addr_lsb] &&
                     (!tlbi.from_aarch64 || ResTLBITTL(tlbi.ttl) || (
                         DecodeTLBITG(tlbi.ttl[3:2]) == tlb_entry.context.tg &&
                         UInt(tlbi.ttl[1:0]) == tlb_entry.walkstate.level)
                     ) &&
                     ((tlbi.d128  && tlb_entry.context.isd128) ||
                      (tlbi.d64  && !tlb_entry.context.isd128) ||
                      (tlbi.d64 && tlbi.d128)) &&
                     (tlbi.level == TLBILevel_Any || !tlb_entry.walkstate.istable));
        when TLBIOp_VA, TLBIPOp_VA =>
            let addr_lsb : integer{} = tlb_entry.blocksize;
```

```
        match = (tlb_entry.context.includes_s1 &&
                 tlbi.security == tlb_entry.context.ss &&
                 tlbi.regime   == tlb_entry.context.regime &&
                 tlbi.use_vmid == tlb_entry.context.use_vmid &&
                 (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid) &&
                 (!UseASID(tlb_entry.context) || tlbi.asid == tlb_entry.context.asid ||
                    tlb_entry.context.nG == '0') &&
                 tlbi.address[55:addr_lsb] == tlb_entry.context.ia[55:addr_lsb] &&
                 (!tlbi.from_aarch64 || ResTLBITTL(tlbi.ttl) || (
                     DecodeTLBITG(tlbi.ttl[3:2]) == tlb_entry.context.tg &&
                     UInt(tlbi.ttl[1:0]) == tlb_entry.walkstate.level)
                 ) &&
                 ((tlbi.d128  && tlb_entry.context.isd128) ||
                  (tlbi.d64  && !tlb_entry.context.isd128) ||
                  (tlbi.d64 && tlbi.d128)) &&
                 (tlbi.level == TLBILevel_Any || !tlb_entry.walkstate.istable));
    when TLBIOp_VMALL =>
        match = (tlb_entry.context.includes_s1 &&
                 tlbi.security == tlb_entry.context.ss &&
                 tlbi.regime   == tlb_entry.context.regime &&
                 tlbi.use_vmid == tlb_entry.context.use_vmid &&
                 (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid));
    when TLBIOp_VMALLS12 =>
        match = (tlbi.security == tlb_entry.context.ss &&
                 tlbi.regime   == tlb_entry.context.regime &&
                 (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid));
    when TLBIOp_RIPAS2, TLBIPOp_RIPAS2 =>
        match = (!tlb_entry.context.includes_s1 && tlb_entry.context.includes_s2 &&
                 tlbi.security == tlb_entry.context.ss &&
                 tlbi.regime   == tlb_entry.context.regime &&
                 (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid) &&
                 tlbi.ipaspace == tlb_entry.context.ipaspace &&
                 (tlbi.tg != '00' && DecodeTLBITG(tlbi.tg) == tlb_entry.context.tg) &&
                 (!tlbi.from_aarch64 || ResTLBIRTTL(tlbi.tg, tlbi.ttl[1:0]) ||
                    UInt(tlbi.ttl[1:0]) == tlb_entry.walkstate.level) &&
                 ((tlbi.d128  && tlb_entry.context.isd128) ||
                  (tlbi.d64  && !tlb_entry.context.isd128) ||
                  (tlbi.d64 && tlbi.d128)) &&
                 (UInt(tlbi.address[NUM_PABITS-1:0]) <=
                    UInt(entry_end_address[NUM_PABITS-1:0])) &&
                 (UInt(tlbi.end_address[NUM_PABITS-1:0]) >
                    UInt(entry_start_address[NUM_PABITS-1:0])));
    when TLBIOp_RVAA, TLBIPOp_RVAA =>
        match = (tlb_entry.context.includes_s1 &&
                 tlbi.security == tlb_entry.context.ss &&
                 tlbi.regime   == tlb_entry.context.regime &&
                 tlbi.use_vmid == tlb_entry.context.use_vmid &&
                 (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid) &&
                 (tlbi.tg != '00' && DecodeTLBITG(tlbi.tg) == tlb_entry.context.tg) &&
                 (!tlbi.from_aarch64 || ResTLBIRTTL(tlbi.tg, tlbi.ttl[1:0]) ||
                    UInt(tlbi.ttl[1:0]) == tlb_entry.walkstate.level) &&
                 ((tlbi.d128  && tlb_entry.context.isd128) ||
                  (tlbi.d64  && !tlb_entry.context.isd128) ||
                  (tlbi.d64 && tlbi.d128)) &&
                 UInt(tlbi.address[55:0]) <= UInt(entry_end_address[55:0]) &&
                 UInt(tlbi.end_address[55:0]) > UInt(entry_start_address[55:0]));
    when TLBIOp_RVA, TLBIPOp_RVA =>
        match = (tlb_entry.context.includes_s1 &&
                 tlbi.security == tlb_entry.context.ss &&
                 tlbi.regime   == tlb_entry.context.regime &&
                 tlbi.use_vmid == tlb_entry.context.use_vmid &&
                 (!tlb_entry.context.use_vmid || tlbi.vmid == tlb_entry.context.vmid) &&
                 (!UseASID(tlb_entry.context) || tlbi.asid == tlb_entry.context.asid ||
                    tlb_entry.context.nG == '0') &&
                 (tlbi.tg != '00' && DecodeTLBITG(tlbi.tg) == tlb_entry.context.tg) &&
                 (!tlbi.from_aarch64 || ResTLBIRTTL(tlbi.tg, tlbi.ttl[1:0]) ||
                    UInt(tlbi.ttl[1:0]) == tlb_entry.walkstate.level) &&
                 ((tlbi.d128  && tlb_entry.context.isd128) ||
                  (tlbi.d64  && !tlb_entry.context.isd128) ||
                  (tlbi.d64 && tlbi.d128)) &&
                 UInt(tlbi.address[55:0]) <= UInt(entry_end_address[55:0]) &&
                 UInt(tlbi.end_address[55:0]) > UInt(entry_start_address[55:0]));
    when TLBIOp_RPA =>
        let addr = tlb_entry.walkstate.baseaddress.address;
        entry_end_address[NUM_PABITS-1:0] = (addr[NUM_PABITS-1:0] OR
                                             entry_block_mask[NUM_PABITS-1:0]);
        entry_start_address[NUM_PABITS-1:0] = (addr[NUM_PABITS-1:0] AND
                                               NOT entry_block_mask[NUM_PABITS-1:0]);
        match = (tlb_entry.context.includes_gpt &&
                 (UInt(tlbi.address[NUM_PABITS-1:0]) <=
                    UInt(entry_end_address[NUM_PABITS-1:0])) &&
                 (UInt(tlbi.end_address[NUM_PABITS-1:0]) >
                    UInt(entry_start_address[NUM_PABITS-1:0])));
    when TLBIOp_PAALL =>
        match = tlb_entry.context.includes_gpt;
end;

if tlbi.attr == TLBI_ExcludeXS && tlb_entry.context.xs == '1' then
    match = FALSE;
```

```
    end;

    return match;
end;
```

Library pseudocode for aarch64/functions/tlbi/TLBIPRange

```
// TLBIPRange()
// ============
// Extract the input address range information from encoded Xt.

func TLBIPRange(regime : Regime, Xt : bits(128)) => (boolean, bits(2), bits(64), bits(64))
begin
    let valid : boolean = TRUE;
    var start_address : bits(64) = Zeros{};
    var end_address : bits(64)   = Zeros{};

    let tg : bits(2)    = Xt[47:46];
    let scale : integer = UInt(Xt[45:44]);
    let num : integer   = UInt(Xt[43:39]);

    if tg == '00' then
        return (FALSE, tg, start_address, end_address);
    end;

    let tg_bits : AddressSize = TGBits(tg);
    // The more-significant bits of the start_address is not updated,
    // as they are not used when performing address matching in TLB
    start_address[55:tg_bits] = Xt[107:64+(tg_bits-12)];

    let range : integer = (num+1) << (5*scale + 1 + tg_bits);
    end_address   = start_address + range[63:0];

    if end_address[55] != start_address[55] then
        // overflow, saturate it
        end_address = Replicate{9}(start_address[55]) :: Ones{55};
    end;

    return (valid, tg, start_address, end_address);
end;
```

Library pseudocode for aarch64/functions/tlbi/TLBIRange

```
// TLBIRange()
// ===========
// Extract the input address range information from encoded Xt.

func TLBIRange(regime : Regime, Xt : bits(64)) => (boolean, bits(2), bits(64), bits(64))
begin
    let valid : boolean = TRUE;
    var start_address : bits(64) = Zeros{};
    var end_address : bits(64)   = Zeros{};

    let tg : bits(2)    = Xt[47:46];
    let scale : integer = UInt(Xt[45:44]);
    let num : integer   = UInt(Xt[43:39]);
    var tg_bits : integer;

    if tg == '00' then
        return (FALSE, tg, start_address, end_address);
    end;

    case tg of
        when '01' => // 4KB
            tg_bits = 12;
            if HasLargeAddress(regime) then
                start_address[52:16] = Xt[36:0];
                start_address[63:53] = Replicate{11}(Xt[36]);
            else
                start_address[48:12] = Xt[36:0];
                start_address[63:49] = Replicate{15}(Xt[36]);
            end;
        when '10' => // 16KB
            tg_bits = 14;
            if HasLargeAddress(regime) then
                start_address[52:16] = Xt[36:0];
                start_address[63:53] = Replicate{11}(Xt[36]);
            else
                start_address[50:14] = Xt[36:0];
                start_address[63:51] = Replicate{13}(Xt[36]);
            end;
        when '11' => // 64KB
            tg_bits = 16;
            start_address[52:16] = Xt[36:0];
            start_address[63:53] = Replicate{11}(Xt[36]);
        otherwise =>
            unreachable;
    end;

    let range : integer = (num+1) << (5*scale + 1 + tg_bits);
    end_address   = start_address + range[63:0];

    if IsFeatureImplemented(FEAT_LVA3) && end_address[56] != start_address[56] then
        // overflow, saturate it
        end_address = Replicate{8}(start_address[56]) :: Ones{56};
    elsif end_address[52] != start_address[52] then
        // overflow, saturate it
        end_address = Replicate{12}(start_address[52]) :: Ones{52};
    end;

    return (valid, tg, start_address, end_address);
end;
```

Library pseudocode for aarch64/functions/vbitop/VBitOp

```
// VBitOp
// ======
// Vector bit select instruction types.

type VBitOp of enumeration {VBitOp_VBIF, VBitOp_VBIT, VBitOp_VBSL, VBitOp_VEOR};
```

Library pseudocode for aarch64/gic/gcie/AArch64_GIC

```
// AArch64_GIC()
// =============
// Perform GIC operation as per GIC instructions.

impdef func AArch64_GIC(Xt : bits(64), instrdomain : GICInstrDomain, instruction : GICInstr)
begin
    return;
end;
```

Library pseudocode for aarch64/gic/gcie/AArch64_GICR

```
// AArch64_GICR()
// ==============
// Perform GICR operation as per GICR instructions.

impdef func AArch64_GICR(instrdomain : GICInstrDomain, instruction : GICInstr) => bits(64)
begin
    return Zeros{64};
end;
```

Library pseudocode for aarch64/gic/gcie/AArch64_GSB

```
// AArch64_GSB()
// =============
// Perform GSB operation as per GSB instructions.

impdef func AArch64_GSB(instruction : GICInstr)
begin
    return;
end;
```

Library pseudocode for aarch64/gic/gcie/CurrentPhysicalGICDomain

```
// CurrentPhysicalGICDomain()
// ==========================
// Determines the physical interrupt domain the core is operating in

func CurrentPhysicalGICDomain() => GICDomain
begin
    // Return Current Interrupt Domain
    if PSTATE.EL == EL3 then
        return GICDomain_EL3;
    else
        let state_bits : bits(2) = EffectiveSCR_EL3_NSE() :: EffectiveSCR_EL3_NS();
        case state_bits of
            when '00' => return GICDomain_S;
            when '01' => return GICDomain_NS;
            when '10' => unreachable;
            when '11' => return GICDomain_RL;
        end;
    end;
end;
```

Library pseudocode for aarch64/gic/gcie/GICDomain

```
// GICDomain
// =========

type GICDomain of enumeration {GICDomain_S, GICDomain_NS, GICDomain_RL, GICDomain_EL3, GICDomain_V};
```

Library pseudocode for aarch64/gic/gcie/GICInstr

```
// GICInstr
// ========

type GICInstr of enumeration {GICInstr_AFF, GICInstr_DI, GICInstr_DIS,GICInstr_EN, GICInstr_EOI,
                              GICInstr_PEND, GICInstr_PRI, GICInstr_HM, GICInstr_RCFG,
                              GICInstr_IA, GICInstr_NMIA, GICInstr_ACK, GICInstr_SYS};
```

Library pseudocode for aarch64/gic/gcie/GICInstrDomain

```
// GICInstrDomain
// ==============

type GICInstrDomain of enumeration {GICInstrDomain_CD, GICInstrDomain_LD, GICInstrDomain_VD};
```

Library pseudocode for aarch64/gic/gcie/HavePreemptiveDomain

```
// HavePreemptiveDomain()
// ======================

impdef func HavePreemptiveDomain() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/gic/gcie/IsGICDomainImplemented

```
// IsGICDomainImplemented()
// ========================
// This function returns if the PE implements the domain passed as an argument.

impdef func IsGICDomainImplemented(dom : GICDomain) => boolean
begin
    case dom of
        when GICDomain_S   => return HaveSecureState();
        when GICDomain_NS  => return !SecureOnlyImplementation();
        when GICDomain_RL  => return IsFeatureImplemented(FEAT_RME);
        when GICDomain_EL3 => return HaveEL(EL3);
        otherwise          => unreachable;
    end;
end;
```

Library pseudocode for aarch64/gic/gcie/IsPPIAssignedToCurrentDomain

```
// IsPPIAssignedToCurrentDomain()
// ==============================

impdef func IsPPIAssignedToCurrentDomain(n : integer) => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/gic/gcie/IsPPIImplemented

```
// IsPPIImplemented()
// ==================

impdef func IsPPIImplemented(n : integer) => boolean
begin
    return FALSE;
end;
```

Library pseudocode for aarch64/translation/attrs/AArch64_MAIRAttr

```
// AArch64_MAIRAttr()
// ==================
// Retrieve the memory attribute encoding indexed in the given MAIR

func AArch64_MAIRAttr(index : integer, mair2 : MAIRType, mair : MAIRType) => bits(8)
begin
    assert (index < 8 || (IsFeatureImplemented(FEAT_AIE) && (index < 16)));
    if (index > 7) then
        return mair2[(index-8)*:8]; // Read from LSB at MAIR2
    else
        return mair[index*:8];
    end;
end;
```

Library pseudocode for aarch64/translation/debug/AArch64_CheckBreakpoint

```
// AArch64_CheckBreakpoint()
// =========================
// Called before executing the instruction of length "size" bytes at "vaddress" in an AArch64
// translation regime, when either debug exceptions are enabled, or halting debug is enabled
// and halting is allowed.

func AArch64_CheckBreakpoint(fault_in : FaultRecord, vaddress : bits(64),
                             accdesc : AccessDescriptor, size : integer) => FaultRecord
begin
    assert !ELUsingAArch32(S1TranslationRegime());
    assert (UsingAArch32() && size IN {2,4}) || size == 4;

    var fault : FaultRecord = fault_in;
    var match : boolean     = FALSE;
    var addr_match_bp : boolean     = FALSE; // Default assumption that all address match
                                             // breakpoints are inactive or disabled.
    var addr_mismatch_bp : boolean  = FALSE; // Default assumption that all address mismatch
                                             // breakpoints are inactive or disabled.
    var addr_match : boolean        = FALSE;
    var addr_mismatch : boolean     = TRUE;  // Default assumption that the given virtual address
                                             // is outside the range of all address mismatch
                                             // breakpoints.
    var ctxt_match : boolean        = FALSE;

    for i = 0 to NumBreakpointsImplemented() - 1 do
        let brkptinfo : BreakpointInfo = AArch64_BreakpointMatch(i, vaddress, accdesc, size);
        if brkptinfo.bptype == BreakpointType_AddrMatch then
            addr_match_bp = TRUE;
            addr_match = addr_match || brkptinfo.match;
        elsif brkptinfo.bptype == BreakpointType_AddrMismatch then
            addr_mismatch_bp = TRUE;
            addr_mismatch = addr_mismatch && !brkptinfo.match;
        elsif brkptinfo.bptype == BreakpointType_CtxtMatch then
            ctxt_match = ctxt_match || brkptinfo.match;
        end;
    end;
    if addr_match_bp && addr_mismatch_bp then
        match = addr_match && addr_mismatch;
    else
        match = (addr_match_bp && addr_match) || (addr_mismatch_bp && addr_mismatch);
    end;

    match = match || ctxt_match;

    if match then
        fault.statuscode = Fault_Debug;
        fault.vaddress   = vaddress;
        if HaltOnBreakpointOrWatchpoint() then
            let reason : bits(6) = DebugHalt_Breakpoint;
            Halt(reason);
        end;
    end;

    return fault;
end;
```

Library pseudocode for aarch64/translation/debug/AArch64_CheckDebug

```
// AArch64_CheckDebug()
// ====================
// Called on each access to check for a debug exception or entry to Debug state.

func AArch64_CheckDebug(vaddress : bits(64), accdesc : AccessDescriptor,
                        size : integer) => FaultRecord
begin

    var fault : FaultRecord = NoFault(accdesc, vaddress);
    var generate_exception : boolean;

    let d_side : boolean = IsWatchpointableAccess(accdesc);
    let i_side : boolean = (accdesc.acctype == AccessType_IFETCH);
    if accdesc.acctype == AccessType_NV2 then
        let mask : bit = '0';
        let ss : SecurityState = CurrentSecurityState();
        generate_exception = (AArch64_GenerateDebugExceptionsFrom(EL2, ss, mask) &&
                              MDSCR_EL1().MDE == '1');
    else
        generate_exception = AArch64_GenerateDebugExceptions() && MDSCR_EL1().MDE == '1';
    end;
    let halt : boolean = HaltOnBreakpointOrWatchpoint();

    if generate_exception || halt then
        if d_side then
            fault = AArch64_CheckWatchpoint(fault, vaddress, accdesc, size);
        elsif i_side then
            fault = AArch64_CheckBreakpoint(fault, vaddress, accdesc, size);
        end;
    end;

    return fault;
end;
```

Library pseudocode for aarch64/translation/debug/AArch64_CheckWatchpoint

```
// AArch64_CheckWatchpoint()
// =========================
// Called before accessing the memory location of "size" bytes at "address",
// when either debug exceptions are enabled for the access, or halting debug
// is enabled and halting is allowed.

func AArch64_CheckWatchpoint(fault_in : FaultRecord, vaddress_in : bits(64),
                             accdesc : AccessDescriptor, size_in : integer) => FaultRecord
begin
    assert !ELUsingAArch32(S1TranslationRegime());
    var fault : FaultRecord          = fault_in;
    var fault_match : FaultRecord    = fault_in;
    var fault_mismatch : FaultRecord = fault_in;
    var vaddress : bits(64)          = vaddress_in;
    var size : integer               = size_in;
    var rounded_match : boolean      = FALSE;
    let original_vaddress : bits(64) = vaddress;
    let original_size : integer = size;
    var addr_match_wp : boolean      = FALSE;// Default assumption that all address
                                             //  match watchpoints are inactive or disabled.
    var addr_mismatch_wp : boolean   = FALSE; // Default assumption that all address mismatch
                                              // watchpoints are inactive or disabled.
    var addr_match : boolean         = FALSE;
    var addr_mismatch : boolean      = TRUE;// Default assumption that the given virtual address is
                                           // outside the range of all address mismatch watchpoints

    // For memory accesses of below type
    // - Contiguous SVE access
    // - SME access
    // - SIMD&FP access when the PE is in Streaming SVE mode
    // each call to this function is such that:
    // - the lowest accessed address is rounded down to the nearest multiple of 16 bytes
    // - the highest accessed address is rounded up to the nearest multiple of 16 bytes
    // Since the WPF field is set if the implementation does rounding, regardless of true or
    // false match, it would be acceptable to return TRUE for either/both of the first and last
    // access.
    if IsRelaxedWatchpointAccess(accdesc) then
        var upper_vaddress : integer = UInt(original_vaddress) + original_size;
        if ConstrainUnpredictableBool(Unpredictable_16BYTEROUNDEDDOWNACCESS) then
            vaddress = AlignDownSize{64}(vaddress, 16);
            rounded_match = TRUE;
        end;
        if ConstrainUnpredictableBool(Unpredictable_16BYTEROUNDEDUPACCESS) then
            upper_vaddress = AlignUpSize(upper_vaddress,16);
            rounded_match = TRUE;
        end;
        size = upper_vaddress - UInt(vaddress);
    end;

    for i = 0 to NumWatchpointsImplemented() - 1 do
        let watchptinfo : WatchpointInfo = AArch64_WatchpointMatch(i, vaddress, size, accdesc);
        if watchptinfo.wptype == WatchpointType_AddrMatch then
            addr_match_wp = TRUE;
            addr_match = addr_match || watchptinfo.value_match;
            if watchptinfo.value_match then
                fault_match.statuscode = Fault_Debug;
                if DBGWCR_EL1(i).LSC[0] == '1' && accdesc.read then
                    fault_match.write = FALSE;
                elsif DBGWCR_EL1(i).LSC[1] == '1' && accdesc.write then
                    fault_match.write = TRUE;
                end;
                fault_match.watchptinfo = watchptinfo;
            end;
        elsif watchptinfo.wptype == WatchpointType_AddrMismatch then
            addr_mismatch_wp = TRUE;
            addr_mismatch = addr_mismatch && !watchptinfo.value_match;
            if !watchptinfo.value_match then
                fault_mismatch.statuscode = Fault_Debug;
                if DBGWCR_EL1(i).LSC[0] == '1' && accdesc.read then
                    fault_mismatch.write = FALSE;
                elsif DBGWCR_EL1(i).LSC[1] == '1' && accdesc.write then
                    fault_mismatch.write = TRUE;
                end;
                fault_mismatch.watchptinfo = watchptinfo;
            end;
        end;
    end;
    if ((addr_match_wp && addr_mismatch_wp && addr_match && addr_mismatch) ||
        (addr_match_wp && !addr_mismatch_wp && addr_match)) then
        fault = fault_match;
    elsif !addr_match_wp && addr_mismatch_wp && addr_mismatch then
        fault = fault_mismatch;
    end;
    fault.vaddress = vaddress;
    fault.watchptinfo.maybe_false_match = rounded_match;
    if (fault.statuscode == Fault_Debug && HaltOnBreakpointOrWatchpoint() &&
            !accdesc.nonfault && !(accdesc.firstfault && !accdesc.first)) then
        let reason : bits(6) = DebugHalt_Watchpoint;
        EDWAR() = fault.vaddress;
```

```
        let is_async : boolean = FALSE;
        Halt(reason, is_async, fault);
    end;
    return fault;
end;
```

Library pseudocode for aarch64/translation/hdbss/AppendToHDBSS

```
// AppendToHDBSS()
// ===============
// Appends an entry to the HDBSS when the dirty state of a stage 2 descriptor is updated
// from writable-clean to writable-dirty by hardware.

func AppendToHDBSS(fault_in : FaultRecord, ipa_in : FullAddress, accdesc : AccessDescriptor,
                   walkparams : S2TTWParams, level : integer) => FaultRecord
begin
    assert CanAppendToHDBSS();

    var fault : FaultRecord  = fault_in;
    var ipa : FullAddress    = ipa_in;
    let hdbss_size : integer{} = UInt(HDBSSBR_EL2().SZ) as integer{0..9};

    var hdbss_addrdesc : AddressDescriptor;

    var baddr : bits(NUM_PABITS) = HDBSSBR_EL2().BADDR[NUM_PABITS-13 : 0] :: Zeros{12};
    baddr[11 + hdbss_size : 12]  = Zeros{hdbss_size};

    hdbss_addrdesc.paddress.address = baddr + (8 * UInt(HDBSSPROD_EL2().INDEX));
    let nse2 : bit = '0';     // NSE2 has the Effective value of 0 within a PE.
    hdbss_addrdesc.paddress.paspace = DecodePASpace(nse2, EffectiveSCR_EL3_NSE(),
                                                    EffectiveSCR_EL3_NS());

    // Accesses to the HDBSS use the same memory attributes as used for stage 2 translation walks.
    hdbss_addrdesc.memattrs         = WalkMemAttrs(walkparams.sh, walkparams.irgn, walkparams.orgn);
    let hdbss_access : AccessDescriptor = CreateAccDescHDBSS(accdesc);
    hdbss_addrdesc.mecid            = AArch64_S2TTWalkMECID(walkparams.emec, accdesc.ss);

    if IsFeatureImplemented(FEAT_RME) then
        fault.gpcf = GranuleProtectionCheck(hdbss_addrdesc, hdbss_access);

        if fault.gpcf.gpf != GPCF_None then
            if (ImpDefBool(
                  "GPC fault on HDBSSS write reported in HDBSSPROD_EL2")) then
                HDBSSPROD_EL2().FSC = '101000';
            else
                fault.statuscode = Fault_GPCFOnWalk;
                fault.paddress   = hdbss_addrdesc.paddress;
                fault.level      = level;
                fault.gpcfs2walk = TRUE;
                fault.hdbssf     = TRUE;
            end;

            return fault;
        end;
    end;

    // The reported IPA must be aligned to the size of the translation.
    let lsb : AddressSize = TranslationSize(walkparams.d128, walkparams.tgx, level);
    ipa.address           = ipa.address[NUM_PABITS-1:lsb] :: Zeros{lsb};
    var hdbss_entry : bits(64) = CreateHDBSSEntry(ipa, hdbss_access.ss, level);

    if walkparams.ee == '1' then
        hdbss_entry = BigEndianReverse{64}(hdbss_entry);
    end;

    let memstatus : PhysMemRetStatus = PhysMemWrite{64}(hdbss_addrdesc, hdbss_access,
                                                        hdbss_entry);

    if IsFault(memstatus) then
        if (ImpDefBool(
              "External Abort on HDBSS write reported in HDBSSPROD_EL2")) then
            HDBSSPROD_EL2().FSC = '010000';
        else
            let iswrite : boolean = TRUE;
            fault = HandleExternalTTWAbort(memstatus, iswrite, hdbss_addrdesc,
                                           hdbss_access, 8, fault);
            fault.level  = level;
            fault.hdbssf = TRUE;
        end;
    else
        HDBSSPROD_EL2().INDEX = HDBSSPROD_EL2().INDEX + 1;
    end;

    return fault;
end;
```

Library pseudocode for aarch64/translation/hdbss/CanAppendToHDBSS

```
// CanAppendToHDBSS()
// ==================
// Return TRUE if HDBSS can be appended.

readonly func CanAppendToHDBSS() => boolean
begin
    if !IsFeatureImplemented(FEAT_HDBSS) then
        return FALSE;
    end;
    assert EL2Enabled();
    // The PE cannot append entries to the HDBSS if HDBSSPROD_EL2.FSC is
    // any other value than 0b000000, or HDBSS buffer is full.

    if ((UInt(HDBSSPROD_EL2().INDEX) >= ((2 ^ (UInt(HDBSSBR_EL2().SZ) + 12)) DIV 8)) ||
         (HDBSSPROD_EL2().FSC != '000000')) then
        return FALSE;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/translation/hdbss/CreateHDBSSEntry

```
// CreateHDBSSEntry()
// ==================
// Returns a HDBSS entry.

func CreateHDBSSEntry(ipa : FullAddress, ss : SecurityState, level : integer) => bits(64)
begin
    let ns_ipa : bit = if ss == SS_Secure && ipa.paspace == PAS_NonSecure then '1' else '0';
    return ZeroExtend{64}(ipa.address[NUM_PABITS-1:12] ::
                          ns_ipa :: Zeros{7} :: level[2:0] :: '1');
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_IASize

```
// AArch64_IASize()
// ================
// Retrieve the number of bits containing the input address

readonly func AArch64_IASize(txsz : bits(6)) => AddressSize
begin
    return (64 - UInt(txsz)) as AddressSize;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_NextTableBase

```
// AArch64_NextTableBase()
// =======================
// Extract the address embedded in a table descriptor pointing to the base of
// the next level table of descriptors

func AArch64_NextTableBase{N}(descriptor : bits(N), d128 : bit, skl : bits(2),
                              ds : bit, tgx : TGx) => bits(NUM_PABITS)
begin
    var tablebase : bits(NUM_PABITS) = Zeros{};
    let granulebits : AddressSize = TGxGranuleBits(tgx);
    var tablesize : integer;

    if d128 == '1' then
        let descsizelog2 : integer = 4;
        let stride : integer = granulebits - descsizelog2;
        tablesize = stride*(1 + UInt(skl)) + descsizelog2;
    else
        tablesize = granulebits;
    end;

    case tgx of
        when TGx_4KB =>  tablebase[47:12] = descriptor[47:12];
        when TGx_16KB => tablebase[47:14] = descriptor[47:14];
        when TGx_64KB => tablebase[47:16] = descriptor[47:16];
    end;

    tablebase = AlignDownP2{NUM_PABITS}(tablebase, tablesize as integer{0..NUM_PABITS});

    if d128 == '1' then
        tablebase[55:48] = descriptor[55:48];
    elsif tgx == TGx_64KB && (AArch64_PAMax() >= 52 ||
            ImpDefBool("descriptor[15:12] for 64KB granule are OA[51:48]")) then
        tablebase[51:48] = descriptor[15:12];
    elsif ds == '1' then
        tablebase[51:48] = descriptor[9:8]::descriptor[49:48];
    end;

    return tablebase;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_PhysicalAddressSize

```
// AArch64_PhysicalAddressSize()
// =============================
// Retrieve the number of bits bounding the physical address

func AArch64_PhysicalAddressSize(d128 : bit, ds : bit, encoded_ps : bits(3),
                                 tgx : TGx) => AddressSize
begin
    var ps : integer;
    var max_ps : integer;

    case encoded_ps of
        when '000' =>  ps = 32;
        when '001' =>  ps = 36;
        when '010' =>  ps = 40;
        when '011' =>  ps = 42;
        when '100' =>  ps = 44;
        when '101' =>  ps = 48;
        when '110' =>  ps = 52;
        when '111' =>  ps = 56;
    end;

    if d128 == '1' then
        max_ps = AArch64_PAMax();
    elsif IsFeatureImplemented(FEAT_LPA) && (tgx == TGx_64KB || ds == '1') then
        max_ps = Min(52, AArch64_PAMax());
    else
        max_ps = Min(48, AArch64_PAMax());
    end;

    return Min(ps, max_ps) as AddressSize;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S1LeafBase

```
// AArch64_S1LeafBase()
// ====================
// Extract the address embedded in a block and page descriptor pointing to the
// base of a memory block

func AArch64_S1LeafBase{N}(descriptor : bits(N), walkparams : S1TTWParams,
                           level : integer) => bits(NUM_PABITS)
begin
    var leafbase : bits(NUM_PABITS) = Zeros{};

    let granulebits : AddressSize  = TGxGranuleBits(walkparams.tgx);
    let descsizelog2 : integer{}   = if walkparams.d128 == '1' then 4 else 3;
    let stride : integer = granulebits - descsizelog2;
    let leafsize : integer = granulebits + stride * (FINAL_LEVEL - level);

    leafbase[47:0] = AlignDownP2{48}(descriptor[47:0], leafsize as integer{0..48});

    if walkparams.d128 == '1' then
        leafbase[55:48] = descriptor[55:48];
    elsif walkparams.tgx == TGx_64KB && (AArch64_PAMax() >= 52 ||
            ImpDefBool("descriptor[15:12] for 64KB granule are OA[51:48]")) then
        leafbase[51:48] = descriptor[15:12];
    elsif walkparams.ds == '1' then
        leafbase[51:48] = descriptor[9:8,49:48];
    end;

    return leafbase;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S1SLTTEntryAddress

```
// AArch64_S1SLTTEntryAddress()
// ============================
// Compute the first stage 1 translation table descriptor address within the
// table pointed to by the base at the start level

func AArch64_S1SLTTEntryAddress(level : integer, walkparams : S1TTWParams,
                                ia : bits(64), tablebase : FullAddress) => FullAddress
begin
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let nstride : integer = 1;
    let start_level_lookup : boolean = TRUE;
    let (lsb, msb) = AArch64_TTEntryAddressBounds(level, walkparams.txsz, walkparams.tgx,
                                                  descsizelog2, start_level_lookup, nstride);
    let index : bits(NUM_PABITS) = ZeroExtend{NUM_PABITS}(ia[msb:lsb]::Zeros{descsizelog2});

    var descaddress : FullAddress;
    descaddress.address = tablebase.address OR index;
    descaddress.paspace = tablebase.paspace;

    return descaddress;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S1StartLevel

```
// AArch64_S1StartLevel()
// ======================
// Compute the initial lookup level when performing a stage 1 translation
// table walk

func AArch64_S1StartLevel(walkparams : S1TTWParams) => integer
begin
    // Input Address size
    let iasize : AddressSize       = AArch64_IASize(walkparams.txsz);
    let granulebits : AddressSize  = TGxGranuleBits(walkparams.tgx);
    let descsizelog2 : integer = if walkparams.d128 == '1' then 4 else 3;
    let stride : integer       = granulebits - descsizelog2;
    var s1startlevel : integer = FINAL_LEVEL - (((iasize-1) - granulebits) DIVRM stride);
    if walkparams.d128 == '1' then
        s1startlevel = s1startlevel + UInt(walkparams.skl);
    end;
    return s1startlevel;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S1TTBaseAddress

```
// AArch64_S1TTBaseAddress()
// =========================
// Retrieve the PA/IPA pointing to the base of the initial translation table of stage 1

func AArch64_S1TTBaseAddress{N}(walkparams : S1TTWParams,
                                regime : Regime, ttbr : bits(N)) => bits(NUM_PABITS)
begin
    var tablebase : bits(NUM_PABITS) = Zeros{};

    // Input Address size
    let iasize : AddressSize      = AArch64_IASize(walkparams.txsz);
    let granulebits : AddressSize = TGxGranuleBits(walkparams.tgx);
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let stride : integer{}    = granulebits - descsizelog2;
    let startlevel : integer  = AArch64_S1StartLevel(walkparams);
    let levels : integer      = FINAL_LEVEL - startlevel;

    // Base address is aligned to size of the initial translation table in bytes
    var tsize : integer = (iasize - (levels*stride + granulebits)) + descsizelog2;

    if walkparams.d128 == '1' then
        tsize = Max(tsize, 5);
        if regime == Regime_EL3 then
            tablebase[55:5] = ttbr[55:5];
        else
            tablebase[55:5] = ttbr[87:80]::ttbr[47:5];
        end;
    elsif walkparams.ds == '1' || (walkparams.tgx == TGx_64KB && walkparams.ps == '110' &&
            (IsFeatureImplemented(FEAT_LPA) ||
             ImpDefBool("BADDR expresses 52 bits for 64KB granule"))) then
        tsize = Max(tsize, 6);
        tablebase[51:6] = ttbr[5:2]::ttbr[47:6];
    else
        tablebase[47:1] = ttbr[47:1];
    end;
    tablebase = AlignDownP2{NUM_PABITS}(tablebase,tsize as integer{0..NUM_PABITS});
    return tablebase;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S1TTEntryAddress

```
// AArch64_S1TTEntryAddress()
// ==========================
// Compute translation table descriptor address within the table pointed to by
// the table base

func AArch64_S1TTEntryAddress{N}(level : integer, walkparams : S1TTWParams, skl : bits(2),
                                 ia : bits(64), tablebase : FullAddress,
                                 descriptor : bits(N)) => FullAddress
begin
    let nstride : integer = if walkparams.d128 == '1' then UInt(skl) + 1 else 1;
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let start_level_lookup : boolean = FALSE;
    let (lsb, msb) = AArch64_TTEntryAddressBounds(level, walkparams.txsz, walkparams.tgx,
                                                  descsizelog2, start_level_lookup, nstride);
    let index : bits(NUM_PABITS) = ZeroExtend{NUM_PABITS}(ia[msb:lsb]::Zeros{descsizelog2});

    var descaddress : FullAddress;
    descaddress.address = tablebase.address OR index;
    descaddress.paspace = tablebase.paspace;

    return descaddress;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S2LeafBase

```
// AArch64_S2LeafBase()
// ====================
// Extract the address embedded in a block and page descriptor pointing to the
// base of a memory block

func AArch64_S2LeafBase{N}(descriptor : bits(N), walkparams : S2TTWParams,
                           level : integer) => bits(NUM_PABITS)
begin
    var leafbase : bits(NUM_PABITS) = Zeros{};

    let granulebits : AddressSize  = TGxGranuleBits(walkparams.tgx);
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let stride : integer   = granulebits - descsizelog2;
    let leafsize : integer = granulebits + stride * (FINAL_LEVEL - level);

    leafbase[47:0] = AlignDownP2{48}(descriptor[47:0], leafsize as integer{0..48});

    if walkparams.d128 == '1' then
        leafbase[55:48] = descriptor[55:48];
    elsif walkparams.tgx == TGx_64KB && (AArch64_PAMax() >= 52 ||
            (ImpDefBool(
               "descriptor[15:12] for 64KB granule are OA[51:48]"))) then
        leafbase[51:48] = descriptor[15:12];
    elsif walkparams.ds == '1' then
        leafbase[51:48] = descriptor[9:8]::descriptor[49:48];
    end;

    return leafbase;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S2SLTTEntryAddress

```
// AArch64_S2SLTTEntryAddress()
// ============================
// Compute the first stage 2 translation table descriptor address within the
// table pointed to by the base at the start level

func AArch64_S2SLTTEntryAddress(walkparams : S2TTWParams, ipa : bits(NUM_PABITS),
                                tablebase : FullAddress) => FullAddress
begin
    let startlevel   = AArch64_S2StartLevel(walkparams);
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let nstride : integer = 1;
    let start_level_lookup : boolean = TRUE;
    let (lsb, msb) = AArch64_TTEntryAddressBounds(startlevel, walkparams.txsz, walkparams.tgx,
                                                  descsizelog2, start_level_lookup, nstride);
    let index : bits(NUM_PABITS) = ZeroExtend{NUM_PABITS}(ipa[msb:lsb]::Zeros{descsizelog2});

    var descaddress : FullAddress;
    descaddress.address = tablebase.address OR index;
    descaddress.paspace = tablebase.paspace;

    return descaddress;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S2StartLevel

```
// AArch64_S2StartLevel()
// ======================
// Determine the initial lookup level when performing a stage 2 translation
// table walk

func AArch64_S2StartLevel(walkparams : S2TTWParams) => integer
begin
    if walkparams.d128 == '1' then
        let iasize       : AddressSize = AArch64_IASize(walkparams.txsz);
        let granulebits  : AddressSize = TGxGranuleBits(walkparams.tgx);
        let descsizelog2 : integer{} = 4;
        let stride : integer = granulebits - descsizelog2;
        var s2startlevel : integer = FINAL_LEVEL - (((iasize-1) - granulebits) DIVRM stride);
        s2startlevel = s2startlevel + UInt(walkparams.skl);

        return s2startlevel;
    end;

    case walkparams.tgx of
        when TGx_4KB =>
            case walkparams.sl2::walkparams.sl0 of
                when '000' => return 2;
                when '001' => return 1;
                when '010' => return 0;
                when '011' => return 3;
                when '100' => return -1;
            end;
        when TGx_16KB =>
            case walkparams.sl0 of
                when '00' => return 3;
                when '01' => return 2;
                when '10' => return 1;
                when '11' => return 0;
            end;
        when TGx_64KB =>
            case walkparams.sl0 of
                when '00' => return 3;
                when '01' => return 2;
                when '10' => return 1;
            end;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S2TTBaseAddress

```
// AArch64_S2TTBaseAddress()
// =========================
// Retrieve the PA/IPA pointing to the base of the initial translation table of stage 2

func AArch64_S2TTBaseAddress{N}(walkparams : S2TTWParams, paspace : PASpace,
                                ttbr : bits(N)) => bits(NUM_PABITS)
begin
    var tablebase : bits(NUM_PABITS) = Zeros{};

    // Input Address size
    let iasize : AddressSize      = AArch64_IASize(walkparams.txsz);
    let granulebits : AddressSize = TGxGranuleBits(walkparams.tgx);
    let descsizelog2 : integer{}  = if walkparams.d128 == '1' then 4 else 3;
    let stride : integer{}        = granulebits - descsizelog2;
    let startlevel : integer      = AArch64_S2StartLevel(walkparams);
    let levels : integer          = FINAL_LEVEL - startlevel;

    // Base address is aligned to size of the initial translation table in bytes
    var tsize : integer = (iasize - (levels*stride + granulebits)) + descsizelog2;

    if walkparams.d128 == '1' then
        tsize = Max(tsize, 5);
        if paspace == PAS_Secure then
            tablebase[55:5] = ttbr[55:5];
        else
            tablebase[55:5] = ttbr[87:80]::ttbr[47:5];
        end;
    elsif walkparams.ds == '1' || (walkparams.tgx == TGx_64KB && walkparams.ps == '110' &&
            (IsFeatureImplemented(FEAT_LPA) ||
             ImpDefBool("BADDR expresses 52 bits for 64KB granule"))) then
        tsize = Max(tsize, 6);
        tablebase[51:6] = ttbr[5:2]::ttbr[47:6];
    else
        tablebase[47:1] = ttbr[47:1];
    end;
    tablebase = AlignDownP2{NUM_PABITS}(tablebase, tsize as integer{0..NUM_PABITS});
    return tablebase;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_S2TTEntryAddress

```
// AArch64_S2TTEntryAddress()
// ==========================
// Compute translation table descriptor address within the table pointed to by
// the table base

func AArch64_S2TTEntryAddress(level : integer, walkparams : S2TTWParams, skl :  bits(2),
                              ipa :  bits(NUM_PABITS), tablebase : FullAddress) => FullAddress
begin
    let nstride : integer = if walkparams.d128 == '1' then UInt(skl) + 1 else 1;
    let descsizelog2 : integer{} = if walkparams.d128 == '1' then 4 else 3;
    let start_level_lookup : boolean = FALSE;
    let (lsb, msb) = AArch64_TTEntryAddressBounds(level, walkparams.txsz, walkparams.tgx,
                                                  descsizelog2, start_level_lookup, nstride);
    let index : bits(NUM_PABITS) = ZeroExtend{NUM_PABITS}(ipa[msb:lsb]::Zeros{descsizelog2});

    var descaddress : FullAddress;
    descaddress.address = tablebase.address OR index;
    descaddress.paspace = tablebase.paspace;

    return descaddress;
end;
```

Library pseudocode for aarch64/translation/vmsa_addrcalc/AArch64_TTEntryAddressBounds

```
// AArch64_TTEntryAddressBounds()
// ==============================
// Calculate the input-address bounds used to index a translation table entry

func AArch64_TTEntryAddressBounds(level : integer, txsz : bits(6), tgx : TGx,
                                  descsizelog2 : integer{3,4}, start_level_lookup : boolean,
                                  nstride : integer) => (AddressSize, AddressSize)
begin
    let iasize : AddressSize      = AArch64_IASize(txsz);
    let granulebits : AddressSize = TGxGranuleBits(tgx);
    let stride : integer{}        = granulebits - descsizelog2;
    let levels : integer          = FINAL_LEVEL - level;

    let lsb : AddressSize = (levels * stride + granulebits) as AddressSize;
    var msb : AddressSize;
    if start_level_lookup then
        msb = (iasize - 1) as AddressSize;
    else
        msb = ((lsb + (stride * nstride)) - 1) as AddressSize;
    end;

    return(lsb, msb);
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_AddrTop

```
// AArch64_AddrTop()
// =================
// Get the top bit position of the virtual address.
// Bits above are not accounted as part of the translation process.

readonly func AArch64_AddrTop(tbid : bit, acctype : AccessType, tbi : bit) => AddressSize
begin
    if tbid == '1' && acctype == AccessType_IFETCH then
        return 63;
    end;

    if tbi == '1' then
        return 55;
    else
        return 63;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_IPAIsOutOfRange

```
// AArch64_IPAIsOutOfRange()
// =========================
// Check bits not resolved by translation are ZERO

func AArch64_IPAIsOutOfRange(ipa : bits(NUM_PABITS), walkparams : S2TTWParams) => boolean
begin
    //Input Address size
    let iasize : integer{} = AArch64_IASize(walkparams.txsz);

    if iasize < NUM_PABITS then
        return !IsZero(ipa[NUM_PABITS-1:iasize]);
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_OAOutOfRange

```
// AArch64_OAOutOfRange()
// ======================
// Returns whether output address is expressed in the configured size number of bits

func AArch64_OAOutOfRange(address : bits(NUM_PABITS), d128 : bit,
                          ds : bit, ps : bits(3), tgx : TGx) => boolean
begin
    // Output Address size
    let oasize : integer{} = AArch64_PhysicalAddressSize(d128, ds, ps, tgx);

    if oasize < NUM_PABITS then
        return !IsZero(address[NUM_PABITS-1:oasize]);
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_PermissionsOverlaysApplied

```
// AArch64_PermissionsOverlaysApplied()
// ====================================
// Returns TRUE if Permission overlays are applied for the given access type.

func AArch64_PermissionsOverlaysApplied(acctype : AccessType) => boolean
begin
    case acctype of
        when AccessType_SPE         => return FALSE;
        when AccessType_TRBE        => return FALSE;
        otherwise                   => return TRUE;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1CheckPermissions

```
// AArch64_S1CheckPermissions()
// ============================
// Checks whether stage 1 access violates permissions of target memory
// and returns a fault record

func AArch64_S1CheckPermissions(fault_in : FaultRecord, va : bits(64), size : integer,
                                regime : Regime, walkstate : TTWState, walkparams : S1TTWParams,
                                accdesc : AccessDescriptor) => FaultRecord
begin
    var fault : FaultRecord = fault_in;
    let permissions : Permissions = walkstate.permissions;
    var s1perms : S1AccessControls = AArch64_S1ComputePermissions(regime, walkstate,
                                                                  walkparams, accdesc);


    if accdesc.acctype == AccessType_IFETCH then
        // Flag the access is from a guarded page
        SetInGuardedPage(walkstate.guardedpage == '1' && s1perms.x == '1');

        if s1perms.overlay && s1perms.ox == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;
        elsif (walkstate.memattrs.memtype == MemType_Device &&
               ConstrainUnpredictable(Unpredictable_INSTRDEVICE) == Constraint_FAULT) then
            fault.statuscode = Fault_Permission;
        elsif s1perms.x == '0' then
            fault.statuscode = Fault_Permission;
        end;
    elsif accdesc.acctype == AccessType_DC then
        if accdesc.cacheop == CacheOp_Invalidate then
            if s1perms.overlay && s1perms.ow == '0' then
                fault.statuscode = Fault_Permission;
                fault.overlay    = TRUE;
            elsif s1perms.w == '0' then
                fault.statuscode = Fault_Permission;
            elsif (walkparams.hd != '1' && walkparams.pie == '1' &&
                   permissions.ndirty == '1') then
                fault.statuscode = Fault_Permission;
                fault.dirtybit   = TRUE;
            end;
        // DC from privileged context which clean cannot generate a Permission fault
        elsif accdesc.el == EL0 then
            if s1perms.overlay && s1perms.or == '0' then
                fault.statuscode = Fault_Permission;
                fault.overlay    = TRUE;
            elsif (walkparams.cmow == '1' &&
                   accdesc.cacheop == CacheOp_CleanInvalidate &&
                   s1perms.overlay && s1perms.ow == '0') then
                fault.statuscode = Fault_Permission;
                fault.overlay    = TRUE;
            elsif s1perms.r == '0' then
                fault.statuscode = Fault_Permission;
            elsif (walkparams.cmow == '1' &&
                   accdesc.cacheop == CacheOp_CleanInvalidate &&
                   s1perms.w == '0') then
                fault.statuscode = Fault_Permission;
            elsif (walkparams.cmow == '1' && walkparams.hd != '1' && walkparams.pie == '1' &&
                   permissions.ndirty == '1' && accdesc.cacheop == CacheOp_CleanInvalidate) then
                fault.statuscode = Fault_Permission;
                fault.dirtybit   = TRUE;
            end;
        end;
    elsif accdesc.acctype == AccessType_IC then
        // IC from privileged context cannot generate Permission fault
        if accdesc.el == EL0 then
            if (s1perms.overlay && s1perms.or == '0' &&
                ImpDefBool("Permission fault on EL0 IC_IVAU execution")) then
                fault.statuscode = Fault_Permission;
                fault.overlay    = TRUE;
            elsif walkparams.cmow == '1' && s1perms.overlay && s1perms.ow == '0' then
                fault.statuscode = Fault_Permission;
                fault.overlay    = TRUE;
            elsif (s1perms.r == '0' &&
                   ImpDefBool("Permission fault on EL0 IC_IVAU execution")) then
                fault.statuscode = Fault_Permission;
            elsif walkparams.cmow == '1' && s1perms.w == '0' then
                fault.statuscode = Fault_Permission;
            elsif (walkparams.cmow == '1' && walkparams.hd != '1' && walkparams.pie == '1' &&
                   permissions.ndirty == '1') then
                fault.statuscode = Fault_Permission;
                fault.dirtybit   = TRUE;
            end;
        end;
    elsif IsFeatureImplemented(FEAT_GCS) && accdesc.acctype == AccessType_GCS then
        if s1perms.gcs == '0' then
            fault.statuscode = Fault_Permission;
        elsif accdesc.write && walkparams.hd != '1' && permissions.ndirty == '1' then
            fault.statuscode = Fault_Permission;
            fault.dirtybit   = TRUE;
```

```
            fault.write      = TRUE;
        end;
    elsif accdesc.read && s1perms.overlay && s1perms.or == '0' then
        fault.statuscode = Fault_Permission;
        fault.overlay    = TRUE;
        fault.write      = FALSE;
    elsif accdesc.write && s1perms.overlay && s1perms.ow == '0' then
        fault.statuscode = Fault_Permission;
        fault.overlay    = TRUE;
        fault.write      = TRUE;
    elsif accdesc.read && s1perms.r == '0' then
        fault.statuscode = Fault_Permission;
        fault.write      = FALSE;
    elsif accdesc.write && s1perms.w == '0' then
        fault.statuscode = Fault_Permission;
        fault.write      = TRUE;
    elsif (accdesc.write && accdesc.tagaccess &&
           walkstate.memattrs.tags == MemTag_CanonicallyTagged) then
        fault.statuscode   = Fault_Permission;
        fault.write        = TRUE;
        fault.s1tagnotdata = TRUE;
    elsif (accdesc.write && walkparams.hd != '1' && walkparams.pie == '1' &&
           permissions.ndirty == '1') then
        fault.statuscode = Fault_Permission;
        fault.dirtybit   = TRUE;
        fault.write      = TRUE;
    end;

    return fault;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1ComputePermissions

```
// AArch64_S1ComputePermissions()
// ==============================
// Computes the overall stage 1 permissions

func AArch64_S1ComputePermissions(regime : Regime, walkstate : TTWState,
                                  walkparams : S1TTWParams,
                                  accdesc : AccessDescriptor) => S1AccessControls
begin
    let permissions : Permissions = walkstate.permissions;
    var s1perms : S1AccessControls;

    if walkparams.pie == '1' then
        s1perms = AArch64_S1IndirectBasePermissions(regime, walkstate, walkparams, accdesc);
    else
        s1perms = AArch64_S1DirectBasePermissions(regime, walkstate, walkparams, accdesc);
    end;

    if accdesc.el == EL0 && !AArch64_S1E0POEnabled(regime, walkparams.nv1) then
        s1perms.overlay = FALSE;
    elsif accdesc.el != EL0 && !AArch64_S1POEnabled(regime) then
        s1perms.overlay = FALSE;
    end;

    if s1perms.overlay then
        let s1overlay_perms : S1AccessControls = AArch64_S1OverlayPermissions(regime,
                                                                              walkstate,
                                                                              accdesc);
        s1perms.or = s1overlay_perms.or;
        s1perms.ow = s1overlay_perms.ow;
        s1perms.ox = s1overlay_perms.ox;
    end;

    if s1perms.overlay then
        // If WXN and the overlay X permission is present, the overlay W permission is removed.
        s1perms.ow = s1perms.ow AND NOT(s1perms.wxn AND s1perms.ox);
    else
        // If WXN and the W and X permissions are present, the X permission is removed.
        // The W and X permissions are factored into the WXN computation.
        s1perms.x  = s1perms.x AND NOT(s1perms.wxn);
    end;

    return s1perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1ContiguousBitFaults

```
// AArch64_S1ContiguousBitFaults()
// ===============================
// If contiguous bit is set, returns whether the translation size exceeds the
// stage 1 input address size and if the implementation generates a fault.

func AArch64_S1ContiguousBitFaults(walkparams : S1TTWParams, walkstate : TTWState) => boolean
begin
    // Input Address size
    let iasize : AddressSize = AArch64_IASize(walkparams.txsz);
    // Translation size
    let tsize : integer = TranslationSize(walkparams.d128, walkparams.tgx, walkstate.level);
    let csize : integer = ContiguousSize(walkparams.d128, walkparams.tgx, walkstate);

    return (tsize + csize > iasize &&
            ImpDefBool("Translation fault on misprogrammed contiguous bit"));
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1DirectBasePermissions

```
// AArch64_S1DirectBasePermissions()
// =================================
// Computes the stage 1 direct base permissions

func AArch64_S1DirectBasePermissions(regime : Regime, walkstate : TTWState,
                                     walkparams : S1TTWParams,
                                     accdesc : AccessDescriptor) => S1AccessControls
begin
    var r,  w,  x,  wxn  : bit;
    var pr, pw, px, pwxn : bit;
    var ur, uw, ux, uwxn : bit;
    var permissions : Permissions = walkstate.permissions;
    var s1perms : S1AccessControls;
    // Descriptors marked with DBM set have the effective value of AP[2] cleared.
    // This implies no Permission faults caused by lack of write permissions are
    // reported, and the Dirty bit can be set.
    if permissions.dbm == '1' && walkparams.hd == '1' then
        permissions.ap[2] = '0';
    end;

    if HasUnprivileged(regime) then
        // Apply leaf permissions
        case permissions.ap[2:1] of
            when '00' => (pr,pw,ur,uw) = ('1','1','0','0'); // Privileged access
            when '01' => (pr,pw,ur,uw) = ('1','1','1','1'); // No effect
            when '10' => (pr,pw,ur,uw) = ('1','0','0','0'); // Read-only, privileged access
            when '11' => (pr,pw,ur,uw) = ('1','0','1','0'); // Read-only
        end;

        // Apply hierarchical permissions
        case permissions.ap_table of
            when '00' => (pr,pw,ur,uw) = (pr, pw, ur, uw);  // No effect
            when '01' => (pr,pw,ur,uw) = (pr, pw,'0','0');  // Privileged access
            when '10' => (pr,pw,ur,uw) = (pr,'0', ur,'0');  // Read-only
            when '11' => (pr,pw,ur,uw) = (pr,'0','0','0');  // Read-only, privileged access
        end;

        // Locations writable by unprivileged cannot be executed by privileged
        px = NOT(permissions.pxn OR permissions.pxn_table OR uw);
        ux = NOT(permissions.uxn OR permissions.uxn_table);
        pwxn = walkparams.wxn AND pw AND px;
        uwxn = walkparams.wxn AND uw AND ux;

        if (IsFeatureImplemented(FEAT_PAN) && accdesc.pan && !(regime == Regime_EL10 &&
              walkparams.nv1 == '1')) then
            if (ImpDefBool("SCR_EL3.SIF affects EPAN") && accdesc.ss == SS_Secure &&
                  walkstate.baseaddress.paspace == PAS_NonSecure && walkparams.sif == '1') then
                ux = '0';
            end;

            if (ImpDefBool("Realm EL2&0 regime affects EPAN") && accdesc.ss == SS_Realm &&
                  regime == Regime_EL20 && walkstate.baseaddress.paspace != PAS_Realm) then
                ux = '0';
            end;

            let pan : bit = PSTATE.PAN AND (ur OR uw OR (walkparams.epan AND ux));
            pr = pr AND NOT(pan);
            pw = pw AND NOT(pan);
        end;

    else
        // Apply leaf permissions
        case permissions.ap[2] of
            when '0' => (pr,pw) = ('1','1'); // No effect
            when '1' => (pr,pw) = ('1','0'); // Read-only
        end;

        // Apply hierarchical permissions
        case permissions.ap_table[1] of
            when '0' => (pr,pw) = (pr, pw);  // No effect
            when '1' => (pr,pw) = (pr,'0');  // Read-only
        end;

        px   = NOT(permissions.xn OR permissions.xn_table);
        pwxn = walkparams.wxn AND pw AND px;
    end;

    (r,w,x,wxn) = if accdesc.el == EL0 then (ur,uw,ux,uwxn) else (pr,pw,px,pwxn);

    // Prevent execution from Non-secure space by PE in secure state if SIF is set
    if accdesc.ss == SS_Secure && walkstate.baseaddress.paspace == PAS_NonSecure then
        x = x AND NOT(walkparams.sif);
    end;
    // Prevent execution from non-Root space by Root
    if accdesc.ss == SS_Root && walkstate.baseaddress.paspace != PAS_Root then
        x = '0';
    end;
    // Prevent execution from non-Realm space by Realm EL2 and Realm EL2&0
    if (accdesc.ss == SS_Realm && regime IN {Regime_EL2, Regime_EL20} &&
```

```
            walkstate.baseaddress.paspace != PAS_Realm) then
        x = '0';
    end;


    s1perms.r   = r;
    s1perms.w   = w;
    s1perms.x   = x;
    s1perms.gcs = '0';
    s1perms.wxn = wxn;
    s1perms.overlay = AArch64_PermissionsOverlaysApplied(accdesc.acctype);

    return s1perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1HasAlignmentFaultDueToMemType

```
// AArch64_S1HasAlignmentFaultDueToMemType()
// =========================================
// Returns whether stage 1 output fails alignment requirement on data accesses due to memory type

func AArch64_S1HasAlignmentFaultDueToMemType(regime : Regime, accdesc : AccessDescriptor,
                                             aligned : boolean, ntlsmd : bit,
                                             mematttrs : MemoryAttributes) => boolean
begin
    if accdesc.exclusive || accdesc.atomicop || accdesc.acqsc || accdesc.acqpc || accdesc.relsc then
        if (!aligned && !(IsWBShareable(mematttrs) && AArch64_S1DCacheEnabled(regime)) &&
              ConstrainUnpredictableBool(Unpredictable_LSE2_ALIGNMENT_FAULT)) then
            return TRUE;
        end;
    end;

    if mematttrs.memtype != MemType_Device then
        return FALSE;
    elsif ((accdesc.acctype == AccessType_DCZero &&
              accdesc.cachetype IN {CacheType_Tag, CacheType_TagWrite, CacheType_TagZero}) ||
             accdesc.stzgm) then
        return ConstrainUnpredictable(Unpredictable_DEVICETAGSTORE) == Constraint_FAULT;
    elsif accdesc.a32lsmd && ntlsmd == '0' then
        return mematttrs.device != DeviceType_GRE;
    elsif accdesc.acctype == AccessType_DCZero then
        return TRUE;
    elsif !aligned then
        return !(ImpDefBool("Device location supports unaligned access"));
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1IndirectBasePermissions

```
// AArch64_S1IndirectBasePermissions()
// ===================================
// Computes the stage 1 indirect base permissions

func AArch64_S1IndirectBasePermissions(regime : Regime, walkstate : TTWState,
                                       walkparams : S1TTWParams,
                                       accdesc : AccessDescriptor) => S1AccessControls
begin

    var r,  w,  x,  gcs,  wxn,   overlay : bit;
    var pr, pw, px, pgcs, pwxn, p_overlay : bit;
    var ur, uw, ux, ugcs, uwxn, u_overlay : bit;
    let permissions : Permissions = walkstate.permissions;
    var s1perms : S1AccessControls;

    // Apply privileged indirect permissions
    case permissions.ppi of
        when '0000' => (pr,pw,px,pgcs) = ('0','0','0','0'); // No access
        when '0001' => (pr,pw,px,pgcs) = ('1','0','0','0'); // Privileged read
        when '0010' => (pr,pw,px,pgcs) = ('0','0','1','0'); // Privileged execute
        when '0011' => (pr,pw,px,pgcs) = ('1','0','1','0'); // Privileged read and execute
        when '0100' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
        when '0101' => (pr,pw,px,pgcs) = ('1','1','0','0'); // Privileged read and write
        when '0110' => (pr,pw,px,pgcs) = ('1','1','1','0'); // Privileged read, write and execute
        when '0111' => (pr,pw,px,pgcs) = ('1','1','1','0'); // Privileged read, write and execute
        when '1000' => (pr,pw,px,pgcs) = ('1','0','0','0'); // Privileged read
        when '1001' =>
            if IsFeatureImplemented(FEAT_GCS) then
                (pr,pw,px,pgcs) = ('1','0','0','1');     // Privileged read and gcs
            else
                (pr,pw,px,pgcs) = ('0','0','0','0');     // Reserved
            end;
        when '1010' => (pr,pw,px,pgcs) = ('1','0','1','0'); // Privileged read and execute
        when '1011' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
        when '1100' => (pr,pw,px,pgcs) = ('1','1','0','0'); // Privileged read and write
        when '1101' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
        when '1110' => (pr,pw,px,pgcs) = ('1','1','1','0'); // Privileged read, write and execute
        when '1111' => (pr,pw,px,pgcs) = ('0','0','0','0'); // Reserved
    end;

    p_overlay = NOT(permissions.ppi[3]);
    pwxn = if permissions.ppi == '0110' then '1' else '0';
    var reserved_upi : boolean = FALSE;

    if HasUnprivileged(regime) then
        // Apply unprivileged indirect permissions
        case permissions.upi of
            when '0000' => (ur,uw,ux,ugcs) = ('0','0','0','0'); // No access
            when '0001' => (ur,uw,ux,ugcs) = ('1','0','0','0'); // Unprivileged read
            when '0010' => (ur,uw,ux,ugcs) = ('0','0','1','0'); // Unprivileged execute
            when '0011' => (ur,uw,ux,ugcs) = ('1','0','1','0'); // Unprivileged read and execute
            when '0100' =>
                (ur,uw,ux,ugcs) = ('0','0','0','0');         // Reserved
                reserved_upi    = TRUE;
            when '0101' => (ur,uw,ux,ugcs)=('1','1','0','0');// Unprivileged read and write
            when '0110' => (ur,uw,ux,ugcs)=('1','1','1','0');// Unprivileged read, write and execute
            when '0111' => (ur,uw,ux,ugcs)=('1','1','1','0');// Unprivileged read, write and execute
            when '1000' => (ur,uw,ux,ugcs)=('1','0','0','0');// Unprivileged read
            when '1001' =>
                if IsFeatureImplemented(FEAT_GCS) then
                    (ur,uw,ux,ugcs) = ('1','0','0','1');     // Unprivileged read and gcs
                else
                    (ur,uw,ux,ugcs) = ('0','0','0','0');     // Reserved
                    reserved_upi    = TRUE;
                end;
            when '1010' => (ur,uw,ux,ugcs)=('1','0','1','0');// Unprivileged read and execute
            when '1011' =>
                (ur,uw,ux,ugcs) = ('0','0','0','0');          // Reserved
                reserved_upi    = TRUE;
            when '1100' => (ur,uw,ux,ugcs) =('1','1','0','0');// Unprivileged read and write
            when '1101' =>
                (ur,uw,ux,ugcs) =('0','0','0','0');           // Reserved
                reserved_upi    = TRUE;
            when '1110' => (ur,uw,ux,ugcs) =('1','1','1','0');// Unprivileged read, write
                                                               // and execute
            when '1111' =>
                (ur,uw,ux,ugcs) = ('0','0','0','0');          // Reserved
                reserved_upi    = TRUE;

        end;

        u_overlay = NOT(permissions.upi[3]);
        uwxn = if permissions.upi == '0110' then '1' else '0';

        // If the decoded permissions has either px or pgcs along with either uw or ugcs,
        // then all effective Stage 1 Base Permissions are set to 0
        if ((px == '1' || pgcs == '1') && (uw == '1' || ugcs == '1')) then
            (pr,pw,px,pgcs) = ('0','0','0','0');
            (ur,uw,ux,ugcs) = ('0','0','0','0');
```

```
        end;
    end;

    // Prevent execution from Non-secure space by PE in secure state
    if accdesc.ss == SS_Secure && walkstate.baseaddress.paspace == PAS_NonSecure then
        pgcs = '0';
        if HasUnprivileged(regime) then ugcs = '0'; end;

        if walkparams.sif == '1' then
            px = '0';
            if HasUnprivileged(regime) then ux = '0'; end;
        end;
    end;

    // Prevent execution from non-Root space by Root
    if accdesc.ss == SS_Root && walkstate.baseaddress.paspace != PAS_Root then
        (px, pgcs)   = ('0','0');
        if HasUnprivileged(regime) then
            (ux, ugcs)   = ('0','0');
        end;
    end;

    // Prevent execution from non-Realm space by Realm EL2 and Realm EL2&0
    if (accdesc.ss == SS_Realm && regime IN {Regime_EL2, Regime_EL20} &&
          walkstate.baseaddress.paspace != PAS_Realm) then
        (px, pgcs)   = ('0','0');
        if HasUnprivileged(regime) then
            (ux, ugcs)   = ('0','0');
        end;
    end;

    if (HasUnprivileged(regime) && IsFeatureImplemented(FEAT_PAN) && accdesc.pan &&
          !(regime == Regime_EL10 && walkparams.nv1 == '1') &&
          PSTATE.PAN == '1' && !IsZero(permissions.upi)) then
        if reserved_upi then
            if ImpDefBool("PAN applies when S1UnprivBasePerm is reserved") then
                (pr,pw) = ('0','0');
            end;
        elsif (!IsZero(ur::uw::ux::ugcs) ||
                 ImpDefBool("PAN applies when S1UnprivBasePerms are removed")) then
            (pr,pw) = ('0','0');
        end;
    end;

    if accdesc.el == EL0 then
        (r,w,x,gcs,wxn,overlay) = (ur,uw,ux,ugcs,uwxn,u_overlay);
    else
        (r,w,x,gcs,wxn,overlay) = (pr,pw,px,pgcs,pwxn,p_overlay);
    end;

    s1perms.r       = r;
    s1perms.w       = w;
    s1perms.x       = x;
    s1perms.gcs     = gcs;
    s1perms.wxn     = wxn;
    s1perms.overlay = AArch64_PermissionsOverlaysApplied(accdesc.acctype) && overlay == '1';

    return s1perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1OAOutOfRange

```
// AArch64_S1OAOutOfRange()
// ========================
// Returns whether stage 1 output address is expressed in the configured size number of bits

func AArch64_S1OAOutOfRange(address : bits(NUM_PABITS), walkparams : S1TTWParams) => boolean
begin
    return AArch64_OAOutOfRange(address, walkparams.d128, walkparams.ds, walkparams.ps,
                                walkparams.tgx);
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1OverlayPermissions

```
// AArch64_S1OverlayPermissions()
// ==============================
// Computes the stage 1 overlay permissions

func AArch64_S1OverlayPermissions(regime : Regime, walkstate : TTWState,
                                  accdesc : AccessDescriptor) => S1AccessControls
begin

    var r,  w,  x : bit;
    var pr, pw, px : bit;
    var ur, uw, ux : bit;
    let permissions : Permissions = walkstate.permissions;
    var s1overlay_perms : S1AccessControls;

    let por : S1PORType = AArch64_S1POR(regime);
    let bit_index : integer{} = 4 * UInt(permissions.po_index);

    let ppo : bits(4) = por[bit_index+3:bit_index];

    // Apply privileged overlay permissions
    case ppo of
        when '0000' => (pr,pw,px) = ('0','0','0'); // No access
        when '0001' => (pr,pw,px) = ('1','0','0'); // Privileged read
        when '0010' => (pr,pw,px) = ('0','0','1'); // Privileged execute
        when '0011' => (pr,pw,px) = ('1','0','1'); // Privileged read and execute
        when '0100' => (pr,pw,px) = ('0','1','0'); // Privileged write
        when '0101' => (pr,pw,px) = ('1','1','0'); // Privileged read and write
        when '0110' => (pr,pw,px) = ('0','1','1'); // Privileged write and execute
        when '0111' => (pr,pw,px) = ('1','1','1'); // Privileged read, write and execute
        when '1xxx' => (pr,pw,px) = ('0','0','0'); // Reserved
    end;

    if HasUnprivileged(regime) then
        var upo : bits(4) = '0000';
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
               ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            upo = POR_EL0()[bit_index+3:bit_index];
        end;

        // Apply unprivileged overlay permissions
        case upo of
            when '0000' => (ur,uw,ux) = ('0','0','0'); // No access
            when '0001' => (ur,uw,ux) = ('1','0','0'); // Unprivileged read
            when '0010' => (ur,uw,ux) = ('0','0','1'); // Unprivileged execute
            when '0011' => (ur,uw,ux) = ('1','0','1'); // Unprivileged read and execute
            when '0100' => (ur,uw,ux) = ('0','1','0'); // Unprivileged write
            when '0101' => (ur,uw,ux) = ('1','1','0'); // Unprivileged read and write
            when '0110' => (ur,uw,ux) = ('0','1','1'); // Unprivileged write and execute
            when '0111' => (ur,uw,ux) = ('1','1','1'); // Unprivileged read, write and execute
            when '1xxx' => (ur,uw,ux) = ('0','0','0'); // Reserved
        end;
    end;

    (r,w,x) = if accdesc.el == EL0 then (ur,uw,ux) else (pr,pw,px);

    s1overlay_perms.or = r;
    s1overlay_perms.ow = w;
    s1overlay_perms.ox = x;

    return s1overlay_perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S1TxSZFaults

```
// AArch64_S1TxSZFaults()
// ======================
// Detect whether configuration of stage 1 TxSZ field generates a fault

readonly func AArch64_S1TxSZFaults(regime : Regime, walkparams : S1TTWParams) => boolean
begin
    let mintxsz : integer = AArch64_S1MinTxSZ(regime, walkparams);
    let maxtxsz : integer = AArch64_MaxTxSZ(walkparams.tgx);

    if UInt(walkparams.txsz) < mintxsz then
        return (IsFeatureImplemented(FEAT_LVA) ||
                ImpDefBool("Fault on TxSZ value below minimum"));
    end;
    if UInt(walkparams.txsz) > maxtxsz then
        return ImpDefBool("Fault on TxSZ value above maximum");
    end;

    return FALSE;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2CheckPermissions

```
// AArch64_S2CheckPermissions()
// ============================
// Verifies memory access with available permissions.

func AArch64_S2CheckPermissions(fault_in : FaultRecord, walkstate : TTWState,
                                walkparams : S2TTWParams, ipa : AddressDescriptor,
                                accdesc : AccessDescriptor) => (FaultRecord, boolean)
begin
    let memtype : MemType = walkstate.memattrs.memtype;
    let permissions : Permissions = walkstate.permissions;
    var fault : FaultRecord = fault_in;
    let s2perms : S2AccessControls = AArch64_S2ComputePermissions(permissions, walkparams,
                                                                  accdesc);

    var r,  w : bit;
    var or, ow : bit;

    if accdesc.acctype == AccessType_TTW then
        r = s2perms.r_mmu;
        w = s2perms.w_mmu;
        or = s2perms.or_mmu;
        ow = s2perms.ow_mmu;
    elsif accdesc.rcw then
        r = s2perms.r_rcw;
        w = s2perms.w_rcw;
        or = s2perms.or_rcw;
        ow = s2perms.ow_rcw;
    else
        r = s2perms.r;
        w = s2perms.w;
        or = s2perms.or;
        ow = s2perms.ow;
    end;

    if accdesc.acctype == AccessType_TTW then
        if (accdesc.toplevel && accdesc.varange == VARange_LOWER &&
              ((walkparams.tl0 == '1' && s2perms.toplevel0 == '0') ||
              (walkparams.tl1 == '1' && s2perms.[toplevel1,toplevel0] == '10'))) then
            fault.statuscode = Fault_Permission;
            fault.toplevel   = TRUE;
        elsif (accdesc.toplevel && accdesc.varange == VARange_UPPER &&
                ((walkparams.tl1 == '1' && s2perms.toplevel1 == '0') ||
                (walkparams.tl0 == '1' && s2perms.[toplevel1,toplevel0] == '01'))) then
            fault.statuscode = Fault_Permission;
            fault.toplevel   = TRUE;
        // Stage 2 Permission fault due to AssuredOnly check
        elsif (walkstate.s2assuredonly == '1' && !ipa.s1assured) then
                fault.statuscode  = Fault_Permission;
                fault.assuredonly = TRUE;

        elsif s2perms.overlay && or == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;
        elsif accdesc.write && s2perms.overlay && ow == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;

        elsif walkparams.ptw == '1' && memtype == MemType_Device then
            fault.statuscode = Fault_Permission;
        // Prevent translation table walks in Non-secure space by Realm state
        elsif accdesc.ss == SS_Realm && walkstate.baseaddress.paspace != PAS_Realm then
            fault.statuscode = Fault_Permission;
        elsif r == '0' then
            fault.statuscode = Fault_Permission;
        elsif accdesc.write && w == '0' then
            fault.statuscode = Fault_Permission;
            fault.hdbssf = walkparams.hdbss == '1' && !CanAppendToHDBSS() && permissions.dbm == '1';
        elsif (accdesc.write && walkparams.s2pie == '1' && permissions.s2dirty == '0' &&
                 (walkparams.hd != '1' ||
                 (accdesc.acctype != AccessType_AT &&
                  walkparams.hdbss == '1' && !CanAppendToHDBSS()))) then
            fault.statuscode = Fault_Permission;
            fault.dirtybit   = TRUE;
            fault.hdbssf = walkparams.hdbss == '1' && !CanAppendToHDBSS();

        end;

    // Stage 2 Permission fault due to AssuredOnly check
    elsif ((walkstate.s2assuredonly == '1' && !ipa.s1assured) ||
            (walkstate.s2assuredonly != '1' && IsFeatureImplemented(FEAT_GCS) &&
             VTCR_EL2().GCSH == '1' && accdesc.acctype == AccessType_GCS && accdesc.el != EL0)) then
        fault.statuscode  = Fault_Permission;
        fault.assuredonly = TRUE;

    elsif accdesc.acctype == AccessType_IFETCH then
        if s2perms.overlay && s2perms.ox == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;
        elsif (memtype == MemType_Device &&
```

```
            ConstrainUnpredictable(Unpredictable_INSTRDEVICE) == Constraint_FAULT) then
        fault.statuscode = Fault_Permission;

    // Prevent execution from Non-secure space by Realm state
    elsif accdesc.ss == SS_Realm && walkstate.baseaddress.paspace != PAS_Realm then
        fault.statuscode = Fault_Permission;
    elsif s2perms.x == '0' then
        fault.statuscode = Fault_Permission;
    end;

elsif accdesc.acctype == AccessType_DC then
    if accdesc.cacheop == CacheOp_Invalidate then
        if !ELUsingAArch32(EL1) && s2perms.overlay && ow == '0' then
            fault.statuscode = Fault_Permission;
            fault.overlay    = TRUE;
        elsif !ELUsingAArch32(EL1) && w == '0' then
            fault.statuscode = Fault_Permission;
        elsif (walkparams.hd != '1' && walkparams.s2pie == '1' &&
                  permissions.s2dirty == '0') then
            fault.statuscode = Fault_Permission;
            fault.dirtybit   = TRUE;
        end;
    elsif !ELUsingAArch32(EL1) && accdesc.el == EL0 && s2perms.overlay && or == '0' then
        fault.statuscode = Fault_Permission;
        fault.overlay    = TRUE;
    elsif (walkparams.cmow == '1' && accdesc.cacheop == CacheOp_CleanInvalidate &&
            s2perms.overlay && ow == '0') then
        fault.statuscode = Fault_Permission;
        fault.overlay    = TRUE;
    elsif !ELUsingAArch32(EL1) && accdesc.el == EL0 && r == '0' then
        fault.statuscode = Fault_Permission;
    elsif (walkparams.cmow == '1' && accdesc.cacheop == CacheOp_CleanInvalidate &&
            w == '0') then
        fault.statuscode = Fault_Permission;
    elsif (walkparams.cmow == '1' && accdesc.cacheop == CacheOp_CleanInvalidate &&
             walkparams.hd != '1' && walkparams.s2pie == '1' && permissions.s2dirty == '0') then
        fault.statuscode = Fault_Permission;
        fault.dirtybit   = TRUE;
    end;

elsif accdesc.acctype == AccessType_IC then
    if (!ELUsingAArch32(EL1) && accdesc.el == EL0 && s2perms.overlay && or == '0' &&
            ImpDefBool("Permission fault on EL0 IC_IVAU execution")) then
        fault.statuscode = Fault_Permission;
        fault.overlay    = TRUE;
    elsif walkparams.cmow == '1' && s2perms.overlay && ow == '0' then
        fault.statuscode = Fault_Permission;
        fault.overlay    = TRUE;
    elsif (!ELUsingAArch32(EL1) && accdesc.el == EL0 &&  r == '0' &&
          ImpDefBool("Permission fault on EL0 IC_IVAU execution")) then
        fault.statuscode = Fault_Permission;
    elsif walkparams.cmow == '1' && w == '0' then
        fault.statuscode = Fault_Permission;
    elsif (walkparams.cmow == '1' && walkparams.hd != '1' && walkparams.s2pie == '1' &&
             permissions.s2dirty == '0') then
        fault.statuscode = Fault_Permission;
        fault.dirtybit   = TRUE;
    end;

elsif accdesc.read && s2perms.overlay && or == '0' then
    fault.statuscode = Fault_Permission;
    fault.overlay    = TRUE;
    fault.write      = FALSE;
elsif accdesc.write && s2perms.overlay && ow == '0' then
    fault.statuscode = Fault_Permission;
    fault.overlay    = TRUE;
    fault.write      = TRUE;
elsif accdesc.read && r == '0' then
    fault.statuscode = Fault_Permission;
    fault.write      = FALSE;
elsif accdesc.write && w == '0' then
    fault.statuscode = Fault_Permission;
    fault.write      = TRUE;
    fault.hdbssf     = walkparams.hdbss == '1' && !CanAppendToHDBSS() && permissions.dbm == '1';
elsif (IsFeatureImplemented(FEAT_MTE_PERM) &&
         ((accdesc.tagchecked &&
           AArch64_EffectiveTCF(accdesc.el, accdesc.read) != TCFType_Ignore) ||
          accdesc.tagaccess) &&
         ipa.memattrs.tags == MemTag_AllocationTagged &&
         permissions.s2tag_na == '1' && S2DCacheEnabled()) then
    fault.statuscode = Fault_Permission;
    fault.tagaccess  = TRUE;
    fault.write      = accdesc.tagaccess && accdesc.write;
elsif (accdesc.write &&
         (walkparams.hd != '1' ||
         (walkparams.hdbss == '1' && accdesc.acctype != AccessType_AT &&
            !CanAppendToHDBSS())) &&
          walkparams.s2pie == '1' && permissions.s2dirty == '0') then
    fault.statuscode = Fault_Permission;
```

```
        fault.dirtybit   = TRUE;
        fault.write      = TRUE;
        fault.hdbssf     = (walkparams.hdbss == '1' && accdesc.acctype != AccessType_AT &&
                            !CanAppendToHDBSS());
    end;
    // MRO* allows only RCW and MMU writes
    var mro : boolean;
    if s2perms.overlay then
        mro = (s2perms.[w,w_rcw,w_mmu] AND s2perms.[ow,ow_rcw,ow_mmu]) == '011';
    else
        mro = s2perms.[w,w_rcw,w_mmu] == '011';
    end;

    return (fault, mro);
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2ComputePermissions

```
// AArch64_S2ComputePermissions()
// ==============================
// Compute the overall stage 2 permissions.

func AArch64_S2ComputePermissions(permissions : Permissions, walkparams : S2TTWParams,
                                  accdesc : AccessDescriptor) => S2AccessControls
begin

    var s2perms : S2AccessControls;

    if walkparams.s2pie == '1' then
        s2perms = AArch64_S2IndirectBasePermissions(permissions, accdesc);
        s2perms.overlay = IsFeatureImplemented(FEAT_S2POE) && VTCR_EL2().S2POE == '1';
        if s2perms.overlay then
            let s2overlay_perms : S2AccessControls = AArch64_S2OverlayPermissions(permissions,
                                                                                  accdesc);
            s2perms.or          = s2overlay_perms.or;
            s2perms.ow          = s2overlay_perms.ow;
            s2perms.ox          = s2overlay_perms.ox;
            s2perms.or_rcw      = s2overlay_perms.or_rcw;
            s2perms.ow_rcw      = s2overlay_perms.ow_rcw;
            s2perms.or_mmu      = s2overlay_perms.or_mmu;
            s2perms.ow_mmu      = s2overlay_perms.ow_mmu;

            // Toplevel is applicable only when the effective S2 permissions is MRO
            if ((s2perms.[w,w_rcw,w_mmu] AND s2perms.[ow,ow_rcw,ow_mmu]) == '011') then
                s2perms.toplevel0 = s2perms.toplevel0 OR s2overlay_perms.toplevel0;
                s2perms.toplevel1 = s2perms.toplevel1 OR s2overlay_perms.toplevel1;

            else
                s2perms.toplevel0 = '0';
                s2perms.toplevel1 = '0';
            end;
        end;
    else
        s2perms = AArch64_S2DirectBasePermissions(permissions, accdesc, walkparams);
    end;

    return s2perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2ContiguousBitFaults

```
// AArch64_S2ContiguousBitFaults()
// ===============================
// If contiguous bit is set, returns whether the translation size exceeds the
// stage 2 input address size and if the implementation generates a fault.

func AArch64_S2ContiguousBitFaults(walkparams : S2TTWParams, walkstate : TTWState) => boolean
begin
    // Input Address size
    let iasize : AddressSize = AArch64_IASize(walkparams.txsz);
    // Translation size
    let tsize : integer = TranslationSize(walkparams.d128, walkparams.tgx, walkstate.level);
    let csize : integer = ContiguousSize(walkparams.d128, walkparams.tgx, walkstate);

    return (tsize + csize > iasize &&
            ImpDefBool("Translation fault on misprogrammed contiguous bit"));
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2DirectBasePermissions

```
// AArch64_S2DirectBasePermissions()
// =================================
// Computes the stage 2 direct base permissions.

func AArch64_S2DirectBasePermissions(permissions : Permissions,
                                     accdesc : AccessDescriptor,
                                     walkparams : S2TTWParams) => S2AccessControls
begin
    var s2perms : S2AccessControls;
    var w : bit;
    let r : bit = permissions.s2ap[0];
    if permissions.s2ap[1] == '1' then
        w = '1';
    // Descriptors marked with DBM set have the effective value of S2AP[1] set.
    // This implies no Permission faults caused by lack of write permissions are
    // reported, and the Dirty bit can be set.
    elsif permissions.dbm == '1' && walkparams.hd == '1' then
        // An update occurs here, conditional to being able to append to HDBSS
        if (walkparams.hdbss == '1' &&
              accdesc.acctype IN !{AccessType_DC, AccessType_IC, AccessType_AT}) then
            w = if CanAppendToHDBSS() then '1' else '0';
        else
            w = '1';
        end;
    else
        w = '0';
    end;

    var px, ux : bit;
    case (permissions.s2xn::permissions.s2xnx) of
        when '00' => (px,ux) = ('1','1');
        when '01' => (px,ux) = ('0','1');
        when '10' => (px,ux) = ('0','0');
        when '11' => (px,ux) = ('1','0');
    end;

    let x : bit = if accdesc.el == EL0 then ux else px;
    s2perms.r = r;
    s2perms.w = w;
    s2perms.x = x;
    s2perms.r_rcw = r;
    s2perms.w_rcw = w;
    s2perms.r_mmu = r;
    s2perms.w_mmu = w;
    s2perms.toplevel0 = '0';
    s2perms.toplevel1 = '0';
    s2perms.overlay = FALSE;

    return s2perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2HasAlignmentFaultDueToMemType

```
// AArch64_S2HasAlignmentFaultDueToMemType()
// =========================================
// Returns whether stage 2 output fails alignment requirement on data accesses due to memory type

func AArch64_S2HasAlignmentFaultDueToMemType(accdesc : AccessDescriptor, aligned : boolean,
                                             memattrs : MemoryAttributes) => boolean
begin

    if accdesc.exclusive || accdesc.atomicop || accdesc.acqsc || accdesc.acqpc || accdesc.relsc then
        if (!aligned && !(IsWBShareable(memattrs) && S2DCacheEnabled()) &&
                ConstrainUnpredictableBool(Unpredictable_LSE2_ALIGNMENT_FAULT)) then
            return TRUE;
        end;
    end;

    if memattrs.memtype != MemType_Device then
        return FALSE;
    elsif ((accdesc.acctype == AccessType_DCZero &&
              accdesc.cachetype IN {CacheType_Tag, CacheType_TagWrite, CacheType_TagZero}) ||
             accdesc.stzgm) then
        return ConstrainUnpredictable(Unpredictable_DEVICETAGSTORE) == Constraint_FAULT;
    elsif accdesc.acctype == AccessType_DCZero then
        return TRUE;
    elsif !aligned then
        return !(ImpDefBool("Device location supports unaligned access"));
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2InconsistentSL

```
// AArch64_S2InconsistentSL()
// ==========================
// Detect inconsistent configuration of stage 2 TxSZ and SL fields

func AArch64_S2InconsistentSL(walkparams : S2TTWParams) => boolean
begin
    let startlevel : integer      = AArch64_S2StartLevel(walkparams);
    let levels : integer          = FINAL_LEVEL - startlevel;
    let granulebits : AddressSize = TGxGranuleBits(walkparams.tgx);
    let descsizelog2 : integer{}  = 3;
    let stride : integer{}        = granulebits - descsizelog2;

    // Input address size must at least be large enough to be resolved from the start level
    let sl_min_iasize : integer = (
        levels * stride // Bits resolved by table walk, except initial level
        + granulebits   // Bits directly mapped to output address
        + 1);           // At least 1 more bit to be decoded by initial level

    // Can accommodate 1 more stride in the level + concatenation of up to 2^4 tables
    let sl_max_iasize : integer = sl_min_iasize + (stride-1) + 4;
    // Configured Input Address size
    let iasize : AddressSize    = AArch64_IASize(walkparams.txsz);

    return iasize < sl_min_iasize || iasize > sl_max_iasize;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2IndirectBasePermissions

```
// AArch64_S2IndirectBasePermissions()
// ===================================
// Computes the stage 2 indirect base permissions.

func AArch64_S2IndirectBasePermissions(permissions : Permissions,
                                       accdesc : AccessDescriptor) => S2AccessControls
begin
    var r, w : bit;
    var r_rcw, w_rcw : bit;
    var r_mmu, w_mmu : bit;
    var px, ux : bit;
    var toplevel0, toplevel1 : bit;
    var s2perms : S2AccessControls;

    let s2pi : bits(4) = permissions.s2pi;
    case s2pi of
        when '0000' => (r,w,px,ux,w_rcw,w_mmu) = ('0','0','0','0','0','0');  // No Access
        when '0001' => (r,w,px,ux,w_rcw,w_mmu) = ('0','0','0','0','0','0');  // Reserved
        when '0010' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','0','1','1');  // MRO
        when '0011' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','0','1','1');  // MRO-TL1
        when '0100' => (r,w,px,ux,w_rcw,w_mmu) = ('0','1','0','0','0','0');  // Write Only
        when '0101' => (r,w,px,ux,w_rcw,w_mmu) = ('0','0','0','0','0','0');  // Reserved
        when '0110' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','0','1','1');  // MRO-TL0
        when '0111' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','0','1','1');  // MRO-TL01
        when '1000' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','0','0','0');  // Read Only
        when '1001' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','1','0','0');  // Read, Unpriv Execute
        when '1010' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','1','0','0','0');  // Read, Priv Execute
        when '1011' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','1','1','0','0');  // Read, All Execute
        when '1100' => (r,w,px,ux,w_rcw,w_mmu) = ('1','1','0','0','1','1');  // RW
        when '1101' => (r,w,px,ux,w_rcw,w_mmu) = ('1','1','0','1','1','1');  // RW, Unpriv Execute
        when '1110' => (r,w,px,ux,w_rcw,w_mmu) = ('1','1','1','0','1','1');  // RW, Priv Execute
        when '1111' => (r,w,px,ux,w_rcw,w_mmu) = ('1','1','1','1','1','1');  // RW, All Execute
    end;

    let x : bit = if accdesc.el == EL0 then ux else px;

    // RCW and MMU read permissions.
    (r_rcw, r_mmu) = (r, r);

    // Stage 2 Top Level Permission Attributes.
    case s2pi of
        when '0110' => (toplevel0,toplevel1) = ('1','0');
        when '0011' => (toplevel0,toplevel1) = ('0','1');
        when '0111' => (toplevel0,toplevel1) = ('1','1');
        otherwise =>   (toplevel0,toplevel1) = ('0','0');
    end;

    s2perms.r = r;
    s2perms.w = w;
    s2perms.x = x;
    s2perms.r_rcw = r_rcw;
    s2perms.r_mmu = r_mmu;
    s2perms.w_rcw = w_rcw;
    s2perms.w_mmu = w_mmu;
    s2perms.toplevel0 = toplevel0;
    s2perms.toplevel1 = toplevel1;

    return s2perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2InvalidSL

```
// AArch64_S2InvalidSL()
// =====================
// Detect invalid configuration of SL field

func AArch64_S2InvalidSL(walkparams : S2TTWParams) => boolean
begin
    case walkparams.tgx of
        when TGx_4KB =>
            case walkparams.sl2::walkparams.sl0 of
                when '1x1' => return TRUE;
                when '11x' => return TRUE;
                when '100' => return AArch64_PAMax() < 52;
                when '010' => return AArch64_PAMax() < 44;
                when '011' => return !IsFeatureImplemented(FEAT_TTST);
                otherwise  =>  return FALSE;
            end;
        when TGx_16KB =>
            case walkparams.sl0 of
                when '11' => return walkparams.ds == '0' || AArch64_PAMax() < 52;
                when '10' => return AArch64_PAMax() < 42;
                otherwise =>  return FALSE;
            end;
        when TGx_64KB =>
            case walkparams.sl0 of
                when '11' =>  return TRUE;
                when '10' =>  return AArch64_PAMax() < 44;
                otherwise =>  return FALSE;
            end;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2OAOutOfRange

```
// AArch64_S2OAOutOfRange()
// ========================
// Returns whether stage 2 output address is expressed in the configured size number of bits

func AArch64_S2OAOutOfRange(address : bits(NUM_PABITS), walkparams : S2TTWParams) => boolean
begin
    return AArch64_OAOutOfRange(address, walkparams.d128, walkparams.ds, walkparams.ps,
                                walkparams.tgx);
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2OverlayPermissions

```
// AArch64_S2OverlayPermissions()
// ==============================
// Computes the stage 2 overlay permissions.

func AArch64_S2OverlayPermissions(permissions : Permissions,
                                  accdesc : AccessDescriptor) => S2AccessControls
begin
    var r, w : bit;
    var r_rcw, w_rcw : bit;
    var r_mmu, w_mmu : bit;
    var px, ux : bit;
    var toplevel0, toplevel1 : bit;
    var s2overlay_perms : S2AccessControls;

    let index : integer{} = 4 * UInt(permissions.s2po_index);
    var s2po : bits(4) = '0000';
    if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
          ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
        s2po = S2POR_EL1()[index+3:index];
    end;
    case s2po of
        when '0000' => (r,w,px,ux,w_rcw,w_mmu) = ('0','0','0','0','0','0');  // No Access
        when '0001' => (r,w,px,ux,w_rcw,w_mmu) = ('0','0','0','0','0','0');  // Reserved
        when '0010' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','0','1','1');  // MRO
        when '0011' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','0','1','1');  // MRO-TL1
        when '0100' => (r,w,px,ux,w_rcw,w_mmu) = ('0','1','0','0','0','0');  // Write Only
        when '0101' => (r,w,px,ux,w_rcw,w_mmu) = ('0','0','0','0','0','0');  // Reserved
        when '0110' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','0','1','1');  // MRO-TL0
        when '0111' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','0','1','1');  // MRO-TL01
        when '1000' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','0','0','0');  // Read Only
        when '1001' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','0','1','0','0');  // Read, Unpriv Execute
        when '1010' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','1','0','0','0');  // Read, Priv Execute
        when '1011' => (r,w,px,ux,w_rcw,w_mmu) = ('1','0','1','1','0','0');  // Read, All Execute
        when '1100' => (r,w,px,ux,w_rcw,w_mmu) = ('1','1','0','0','1','1');  // RW
        when '1101' => (r,w,px,ux,w_rcw,w_mmu) = ('1','1','0','1','1','1');  // RW, Unpriv Execute
        when '1110' => (r,w,px,ux,w_rcw,w_mmu) = ('1','1','1','0','1','1');  // RW, Priv Execute
        when '1111' => (r,w,px,ux,w_rcw,w_mmu) = ('1','1','1','1','1','1');  // RW, All Execute
    end;

    let x : bit = if accdesc.el == EL0 then ux else px;

    // RCW and MMU read permissions.
    (r_rcw, r_mmu) = (r, r);

    // Stage 2 Top Level Permission Attributes.
    case s2po of
        when '0110' => (toplevel0,toplevel1) = ('1','0');
        when '0011' => (toplevel0,toplevel1) = ('0','1');
        when '0111' => (toplevel0,toplevel1) = ('1','1');
        otherwise =>   (toplevel0,toplevel1) = ('0','0');
    end;

    s2overlay_perms.or = r;
    s2overlay_perms.ow = w;
    s2overlay_perms.ox = x;
    s2overlay_perms.or_rcw = r_rcw;
    s2overlay_perms.ow_rcw = w_rcw;
    s2overlay_perms.or_mmu = r_mmu;
    s2overlay_perms.ow_mmu = w_mmu;
    s2overlay_perms.toplevel0 = toplevel0;
    s2overlay_perms.toplevel1 = toplevel1;

    return s2overlay_perms;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_S2TxSZFaults

```
// AArch64_S2TxSZFaults()
// ======================
// Detect whether configuration of stage 2 TxSZ field generates a fault

func AArch64_S2TxSZFaults(walkparams : S2TTWParams, s1aarch64 : boolean) => boolean
begin
    let mintxsz : integer = AArch64_S2MinTxSZ(walkparams, s1aarch64);
    let maxtxsz : integer = AArch64_MaxTxSZ(walkparams.tgx);

    if UInt(walkparams.txsz) < mintxsz then
        return (IsFeatureImplemented(FEAT_LPA) ||
                ImpDefBool("Fault on TxSZ value below minimum"));
    end;
    if UInt(walkparams.txsz) > maxtxsz then
        return ImpDefBool("Fault on TxSZ value above maximum");
    end;

    return FALSE;
end;
```

Library pseudocode for aarch64/translation/vmsa_faults/AArch64_VAIsOutOfRange

```
// AArch64_VAIsOutOfRange()
// ========================
// Check bits not resolved by translation are identical and of accepted value

readonly func AArch64_VAIsOutOfRange(va_in : bits(64), acctype : AccessType,
                            regime : Regime, walkparams : S1TTWParams) => boolean
begin
    var va : bits(64) = va_in;

    let addrtop : AddressSize = AArch64_AddrTop(walkparams.tbid, acctype, walkparams.tbi);

    // If the VA has a Logical Address Tag then the bits holding the Logical Address Tag are
    // ignored when checking if the address is out of range.
    if walkparams.mtx == '1' && acctype != AccessType_IFETCH then
        va[59:56] = if AArch64_GetVARange(va) == VARange_UPPER then '1111' else '0000';
    end;

    // Input Address size
    let iasize : integer{} = AArch64_IASize(walkparams.txsz);

    // The min value of TxSZ can be 8, with LVA3 implemented.
    // If TxSZ is set to 8 iasize becomes 64 - 8 = 56
    // If tbi is also set, addrtop becomes 55
    // Then the return statements check va[56:55]
    // The check here is to guard against this corner case.
    if addrtop < iasize then
        return FALSE;
    end;

    if HasUnprivileged(regime) then
        if AArch64_GetVARange(va) == VARange_LOWER then
            return !IsZero(va[addrtop:iasize]);
        else
            return !IsOnes(va[addrtop:iasize]);
        end;
    else
        return !IsZero(va[addrtop:iasize]);
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_memattr/AArch64_S2ApplyFWBMemAttrs

```
// AArch64_S2ApplyFWBMemAttrs()
// ============================
// Apply stage 2 forced Write-Back on stage 1 memory attributes.

func AArch64_S2ApplyFWBMemAttrs{N}(s1_memattrs : MemoryAttributes, walkparams : S2TTWParams,
                                   descriptor : bits(N)) => MemoryAttributes
begin
    var memattrs : MemoryAttributes;
    let s2_attr : bits(4) = descriptor[5:2];
    let s2_sh : bits(2)   = if walkparams.ds == '1' then walkparams.sh else descriptor[9:8];
    let s2_fnxs : bit     = descriptor[11];

    if s2_attr[2] == '0' then          // S2 Device, S1 any
        let s2_device : DeviceType = DecodeDevice(s2_attr[1:0]);
        memattrs.memtype = MemType_Device;
        if s1_memattrs.memtype == MemType_Device then
            memattrs.device = S2CombineS1Device(s1_memattrs.device, s2_device);
        else
            memattrs.device = s2_device;
        end;

        memattrs.xs = s1_memattrs.xs;

    elsif s2_attr[1:0] == '11' then    // S2 attr = S1 attr
        memattrs = s1_memattrs;

    elsif s2_attr[1:0] == '10' then    // Force writeback
        memattrs.memtype = MemType_Normal;
        memattrs.inner.attrs = MemAttr_WB;
        memattrs.outer.attrs = MemAttr_WB;

        if (s1_memattrs.memtype == MemType_Normal &&
                s1_memattrs.inner.attrs != MemAttr_NC) then
            memattrs.inner.hints     = s1_memattrs.inner.hints;
            memattrs.inner.transient = s1_memattrs.inner.transient;
        else
            memattrs.inner.hints     = MemHint_RWA;
            memattrs.inner.transient = FALSE;
        end;

        if (s1_memattrs.memtype == MemType_Normal &&
                s1_memattrs.outer.attrs != MemAttr_NC) then
            memattrs.outer.hints     = s1_memattrs.outer.hints;
            memattrs.outer.transient = s1_memattrs.outer.transient;
        else
            memattrs.outer.hints     = MemHint_RWA;
            memattrs.outer.transient = FALSE;
        end;

        memattrs.xs = '0';

    else                               // Non-cacheable unless S1 is device
        if s1_memattrs.memtype == MemType_Device then
            memattrs = s1_memattrs;
        else
            var cacheability_attr : MemAttrHints;
            cacheability_attr.attrs = MemAttr_NC;

            memattrs.memtype = MemType_Normal;
            memattrs.inner   = cacheability_attr;
            memattrs.outer   = cacheability_attr;

            memattrs.xs = s1_memattrs.xs;
        end;
    end;

    let s2_shareability : Shareability = DecodeShareability(s2_sh);
    memattrs.shareability = S2CombineS1Shareability(s1_memattrs.shareability, s2_shareability);
    memattrs.tags         = S2MemTagType(memattrs, s1_memattrs.tags);
    memattrs.notagaccess  = (s2_attr[3:1] == '111' && memattrs.tags == MemTag_AllocationTagged);

    if s2_fnxs == '1' then
        memattrs.xs = '0';
    end;

    memattrs.shareability = EffectiveShareability(memattrs);
    return memattrs;
end;
```

Library pseudocode for aarch64/translation/vmsa_tlbcontext/AArch64_GetS1TLBContext

```
// AArch64_GetS1TLBContext()
// =========================
// Gather translation context for accesses with VA to match against TLB entries

func AArch64_GetS1TLBContext(regime : Regime, ss : SecurityState,
                             va : bits(64), tg : TGx) => TLBContext
begin
    var tlbcontext : TLBContext;

    case regime of
        when Regime_EL3 =>  tlbcontext = AArch64_TLBContextEL3(ss, va, tg);
        when Regime_EL2 =>  tlbcontext = AArch64_TLBContextEL2(ss, va, tg);
        when Regime_EL20 => tlbcontext = AArch64_TLBContextEL20(ss, va, tg);
        when Regime_EL10 => tlbcontext = AArch64_TLBContextEL10(ss, va, tg);
        otherwise =>
            unreachable;

    end;

    tlbcontext.includes_s1  = TRUE;
    // The following may be amended for EL1&0 Regime if caching of stage 2 is successful
    tlbcontext.includes_s2  = FALSE;
    tlbcontext.use_vmid     = UseVMID(regime);
    // The following may be amended if Granule Protection Check passes
    tlbcontext.includes_gpt = FALSE;
    return tlbcontext;
end;
```

Library pseudocode for aarch64/translation/vmsa_tlbcontext/AArch64_GetS2TLBContext

```
// AArch64_GetS2TLBContext()
// =========================
// Gather translation context for accesses with IPA to match against TLB entries

func AArch64_GetS2TLBContext(ss : SecurityState, ipa : FullAddress, tg : TGx) => TLBContext
begin
    assert EL2Enabled();

    var tlbcontext : TLBContext;

    tlbcontext.ss       = ss;
    tlbcontext.regime   = Regime_EL10;
    tlbcontext.ipaspace = ipa.paspace;
    tlbcontext.vmid     = VMID();
    tlbcontext.tg       = tg;
    tlbcontext.ia       = ZeroExtend{64}(ipa.address);
    if IsFeatureImplemented(FEAT_TTCNP) then
        tlbcontext.cnp = if ipa.paspace == PAS_Secure then VSTTBR_EL2().CnP else VTTBR_EL2().CnP;
    else
        tlbcontext.cnp = '0';
    end;


    tlbcontext.includes_s1  = FALSE;
    tlbcontext.includes_s2  = TRUE;
    tlbcontext.use_vmid     = TRUE;
    // This may be amended if Granule Protection Check passes
    tlbcontext.includes_gpt = FALSE;
    return tlbcontext;
end;
```

Library pseudocode for aarch64/translation/vmsa_tlbcontext/AArch64_TLBContextEL10

```
// AArch64_TLBContextEL10()
// ========================
// Gather translation context for accesses under EL10 regime to match against TLB entries

func AArch64_TLBContextEL10(ss : SecurityState, va : bits(64), tg : TGx) => TLBContext
begin
    var tlbcontext : TLBContext;

    tlbcontext.ss     = ss;
    tlbcontext.regime = Regime_EL10;
    tlbcontext.vmid   = VMID();

    if IsFeatureImplemented(FEAT_ASID2) && IsTCR2EL1Enabled() && TCR2_EL1().A2 == '1' then
        let varange : VARange = AArch64_GetVARange(va);
        tlbcontext.asid = if varange == VARange_LOWER then TTBR0_EL1().ASID else TTBR1_EL1().ASID;
    else
        tlbcontext.asid = if TCR_EL1().A1 == '0' then TTBR0_EL1().ASID else TTBR1_EL1().ASID;
    end;

    if TCR_EL1().AS == '0' then
        tlbcontext.asid[15:8] = Zeros{8};
    end;
    tlbcontext.tg     = tg;
    tlbcontext.ia     = va;

    if IsFeatureImplemented(FEAT_TTCNP) then
        if AArch64_GetVARange(va) == VARange_LOWER then
            tlbcontext.cnp = TTBR0_EL1().CnP;
        else
            tlbcontext.cnp = TTBR1_EL1().CnP;
        end;
    else
        tlbcontext.cnp = '0';
    end;

    return tlbcontext;
end;
```

Library pseudocode for aarch64/translation/vmsa_tlbcontext/AArch64_TLBContextEL2

```
// AArch64_TLBContextEL2()
// =======================
// Gather translation context for accesses under EL2 regime to match against TLB entries

func AArch64_TLBContextEL2(ss : SecurityState, va : bits(64), tg : TGx) => TLBContext
begin
    var tlbcontext : TLBContext;

    tlbcontext.ss     = ss;
    tlbcontext.regime = Regime_EL2;
    tlbcontext.tg     = tg;
    tlbcontext.ia     = va;
    tlbcontext.cnp    = if IsFeatureImplemented(FEAT_TTCNP) then TTBR0_EL2().CnP else '0';

    return tlbcontext;
end;
```

Library pseudocode for aarch64/translation/vmsa_tlbcontext/AArch64_TLBContextEL20

```
// AArch64_TLBContextEL20()
// ========================
// Gather translation context for accesses under EL20 regime to match against TLB entries

func AArch64_TLBContextEL20(ss : SecurityState, va : bits(64), tg : TGx) => TLBContext
begin
    var tlbcontext : TLBContext;

    tlbcontext.ss     = ss;
    tlbcontext.regime = Regime_EL20;

    if IsFeatureImplemented(FEAT_ASID2) && IsTCR2EL2Enabled() && TCR2_EL2().A2 == '1' then
        let varange : VARange = AArch64_GetVARange(va);
        tlbcontext.asid = if varange == VARange_LOWER then TTBR0_EL2().ASID else TTBR1_EL2().ASID;
    else
        tlbcontext.asid = if TCR_EL2().A1 == '0' then TTBR0_EL2().ASID else TTBR1_EL2().ASID;
    end;

    if TCR_EL2().AS == '0' then
        tlbcontext.asid[15:8] = Zeros{8};
    end;
    tlbcontext.tg     = tg;
    tlbcontext.ia     = va;

    if IsFeatureImplemented(FEAT_TTCNP) then
        if AArch64_GetVARange(va) == VARange_LOWER then
            tlbcontext.cnp = TTBR0_EL2().CnP;
        else
            tlbcontext.cnp = TTBR1_EL2().CnP;
        end;
    else
        tlbcontext.cnp = '0';
    end;

    return tlbcontext;
end;
```

Library pseudocode for aarch64/translation/vmsa_tlbcontext/AArch64_TLBContextEL3

```
// AArch64_TLBContextEL3()
// =======================
// Gather translation context for accesses under EL3 regime to match against TLB entries

func AArch64_TLBContextEL3(ss : SecurityState, va : bits(64), tg : TGx) => TLBContext
begin
    var tlbcontext : TLBContext;

    tlbcontext.ss     = ss;
    tlbcontext.regime = Regime_EL3;
    tlbcontext.tg     = tg;
    tlbcontext.ia     = va;
    tlbcontext.cnp    = if IsFeatureImplemented(FEAT_TTCNP) then TTBR0_EL3().CnP else '0';

    return tlbcontext;
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_FullTranslate

```
// AArch64_FullTranslate()
// =======================
// Address translation as specified by VMSA
// Alignment check NOT due to memory type is expected to be done before translation


func AArch64_FullTranslate(va : bits(64), size : integer, accdesc : AccessDescriptor,
                           aligned : boolean) => AddressDescriptor
begin
    let regime : Regime = TranslationRegime(accdesc.el);
    var fault : FaultRecord = NoFault(accdesc, va);

    var ipa : AddressDescriptor;
    (fault, ipa) = AArch64_S1Translate(fault, regime, va, size, aligned, accdesc);

    if fault.statuscode != Fault_None then
        return CreateFaultyAddressDescriptor(fault);
    end;

    if accdesc.ss == SS_Realm then
        assert EL2Enabled();
    end;
    if regime == Regime_EL10 && EL2Enabled() then
        let s1aarch64 : boolean = TRUE;
        var pa : AddressDescriptor;
        (fault, pa) = AArch64_S2Translate(fault, ipa, s1aarch64, aligned, accdesc);

        if fault.statuscode != Fault_None then
            return CreateFaultyAddressDescriptor(fault);
        else
            return pa;
        end;
    else
        return ipa;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_MemSwapTableDesc

```
// AArch64_MemSwapTableDesc()
// ==========================
// Perform HW update of table descriptor as an atomic operation

func AArch64_MemSwapTableDesc{N : integer{64, 128}}(fault_in : FaultRecord, prev_desc : bits(N),
                                                    new_desc : bits(N), ee : bit,
                                                    descaccess : AccessDescriptor,
                                                    descpaddr : AddressDescriptor
                                                   ) => (FaultRecord, bits(N))
begin
    assert descaccess.acctype == AccessType_TTW;
    var fault : FaultRecord = fault_in;
    var iswrite : boolean;

    if IsFeatureImplemented(FEAT_RME) then
        fault.gpcf = GranuleProtectionCheck(descpaddr, descaccess);
        if fault.gpcf.gpf != GPCF_None then
            fault.statuscode = Fault_GPCFOnWalk;
            fault.paddress   = descpaddr.paddress;
            fault.gpcfs2walk = fault.secondstage;
            return (fault, ARBITRARY : bits(N));
        end;
    end;

    // All observers in the shareability domain observe the
    // following memory read and write accesses atomically.
    var mem_desc : bits(N);
    var memstatus : PhysMemRetStatus;
    (memstatus, mem_desc) = PhysMemRead{N}(descpaddr, descaccess);

    if ee == '1' then
        mem_desc = BigEndianReverse{N}(mem_desc);
    end;

    if IsFault(memstatus) then
        iswrite = FALSE;
        fault = HandleExternalTTWAbort(memstatus, iswrite, descpaddr, descaccess, N DIV 8, fault);
        if IsFault(fault.statuscode) then
            return (fault, ARBITRARY : bits(N));
        end;
    end;

    if mem_desc == prev_desc  then
        let ordered_new_desc : bits(N) = (if ee == '1' then BigEndianReverse{N}(new_desc)
                                                       else new_desc);
        memstatus = PhysMemWrite{N}(descpaddr, descaccess, ordered_new_desc);

        if IsFault(memstatus) then
            iswrite = TRUE;
            fault = HandleExternalTTWAbort(memstatus, iswrite, descpaddr, descaccess, N DIV 8,
                                           fault);

            if IsFault(fault.statuscode) then
                return (fault, ARBITRARY : bits(N));
            end;
        end;

        // Reflect what is now in memory (in little endian format)
        mem_desc = new_desc;
    end;

    return (fault, mem_desc);
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_S1DisabledOutput

```
// AArch64_S1DisabledOutput()
// ==========================
// Map the VA to IPA/PA and assign default memory attributes

func AArch64_S1DisabledOutput(fault_in : FaultRecord, regime : Regime,
                              va_in :  bits(64), accdesc : AccessDescriptor,
                              aligned : boolean) => (FaultRecord, AddressDescriptor)
begin
    var va : bits(64) = va_in;
    let walkparams : S1TTWParams = AArch64_GetS1TTWParams(regime, accdesc.el, accdesc.ss, va);
    var fault : FaultRecord = fault_in;

    if accdesc.acctype == AccessType_IFETCH then
        // No memory page is guarded when stage 1 address translation is disabled
        SetInGuardedPage(FALSE);
    end;

    // Output Address
    var oa : FullAddress;
    oa.address = va[NUM_PABITS-1:0];
    case accdesc.ss of
        when SS_Secure =>    oa.paspace = PAS_Secure;
        when SS_NonSecure => oa.paspace = PAS_NonSecure;
        when SS_Root =>      oa.paspace = PAS_Root;
        when SS_Realm =>     oa.paspace = PAS_Realm;
    end;

    var memattrs : MemoryAttributes;
    if regime == Regime_EL10 && EL2Enabled() && walkparams.dc == '1' then
        var default_cacheability : MemAttrHints;
        default_cacheability.attrs     = MemAttr_WB;
        default_cacheability.hints     = MemHint_RWA;
        default_cacheability.transient = FALSE;

        memattrs.memtype      = MemType_Normal;
        memattrs.outer        = default_cacheability;
        memattrs.inner        = default_cacheability;
        memattrs.shareability = Shareability_NSH;
        if walkparams.dct == '1' then
            memattrs.tags     = MemTag_AllocationTagged;
        elsif IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS) && walkparams.mtx == '1' then
            memattrs.tags     = MemTag_CanonicallyTagged;
        else
            memattrs.tags     = MemTag_Untagged;
        end;
        memattrs.xs           = '0';
    elsif accdesc.acctype == AccessType_IFETCH then
        var i_cache_attr : MemAttrHints;
        if AArch64_S1ICacheEnabled(regime) then
            i_cache_attr.attrs     = MemAttr_WT;
            i_cache_attr.hints     = MemHint_RA;
            i_cache_attr.transient = FALSE;
        else
            i_cache_attr.attrs     = MemAttr_NC;
        end;

        memattrs.memtype      = MemType_Normal;
        memattrs.outer        = i_cache_attr;
        memattrs.inner        = i_cache_attr;
        memattrs.shareability = Shareability_OSH;
        memattrs.tags         = MemTag_Untagged;
        memattrs.xs           = '1';
    elsif accdesc.acctype == AccessType_SPE && EffectivePMBLIMITR_EL1_nVM() == '1' then
        memattrs = S1DecodeMemAttrs(PMBMAR_EL1().Attr, PMBMAR_EL1().SH, TRUE,
                                    walkparams, accdesc.acctype);
    elsif accdesc.acctype == AccessType_TRBE && EffectiveTRBLIMITR_EL1_nVM() == '1' then
        memattrs = S1DecodeMemAttrs(TRBMAR_EL1().Attr, TRBMAR_EL1().SH, TRUE,
                                    walkparams, accdesc.acctype);
    else
        memattrs.memtype      = MemType_Device;
        memattrs.device       = DeviceType_nGnRnE;
        memattrs.shareability = Shareability_OSH;
        if IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS) && walkparams.mtx == '1' then
            memattrs.tags = MemTag_CanonicallyTagged;
        else
            memattrs.tags = MemTag_Untagged;
        end;
        memattrs.xs           = '1';
    end;
    memattrs.notagaccess = FALSE;

    if walkparams.mtx == '1' && walkparams.tbi == '0' && accdesc.acctype != AccessType_IFETCH then
        // For the purpose of the checks in this function, the MTE tag bits are ignored.
        va[59:56] = if HasUnprivileged(regime) then Replicate{4}(va[55]) else '0000';
    end;

    fault.level = 0;
    let addrtop : AddressSize = AArch64_AddrTop(walkparams.tbid, accdesc.acctype,
                                                walkparams.tbi);
```

```
    let pamax : AddressSize = AArch64_PAMax();

    let oamax : AddressSize = AArch64_OAMax();
    var addrchecked : boolean = FALSE;

    if accdesc.acctype == AccessType_SPE && EffectivePMBLIMITR_EL1_nVM() == '1' then
        addrchecked = TRUE;
        if oamax > pamax && !IsZero(va[(oamax-1):pamax])then
            fault.statuscode = Fault_AddressSize;
        elsif !IsZero(va[63:oamax]) then
            if ConstrainUnpredictableBool(Unpredictable_PMBPTR_Fault_AddressSize)then
                fault.statuscode = Fault_AddressSize;
            else
                oa.address = ZeroExtend{NUM_PABITS}(oa.address[oamax-1:0]);
            end;
        end;
    end;

    if accdesc.acctype == AccessType_TRBE && EffectiveTRBLIMITR_EL1_nVM() == '1' then
        addrchecked = TRUE;
        if oamax > pamax && !IsZero(va[(oamax-1):pamax])then
            fault.statuscode = Fault_AddressSize;
        elsif !IsZero(va[63:oamax]) then
            if ConstrainUnpredictableBool(Unpredictable_TRBPTR_Fault_AddressSize)then
                fault.statuscode = Fault_AddressSize;
            else
                oa.address = ZeroExtend{NUM_PABITS}(oa.address[oamax-1:0]);
            end;
        end;
    end;

    if fault.statuscode == Fault_None then
        if !addrchecked && addrtop > pamax && !IsZero(va[addrtop:pamax]) then
            fault.statuscode = Fault_AddressSize;
        elsif AArch64_S1HasAlignmentFaultDueToMemType(regime, accdesc, aligned, walkparams.ntlsmd,
                                                      memattrs) then
            fault.statuscode = Fault_Alignment;
        elsif ((accdesc.exclusive || accdesc.atomicop) &&
                !(regime == Regime_EL10 && EL2Enabled() &&
                    IsFeatureImplemented(FEAT_HCRMAC) && HCR_EL2().DC == '1') &&
                ConstrainUnpredictableBool(Unpredictable_Atomic_MMU_IMPDEF_FAULT)) then
            fault.statuscode = Fault_Exclusive;
        end;
    end;

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor);
    else
        var ipa : AddressDescriptor = CreateAddressDescriptor(va_in, oa, memattrs, accdesc);
        ipa.mecid = AArch64_S1DisabledOutputMECID(walkparams, regime, ipa.paddress.paspace);
        return (fault, ipa);
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_S1Translate

```
// AArch64_S1Translate()
// =====================
// Translate VA to IPA/PA depending on the regime

func AArch64_S1Translate(fault_in : FaultRecord, regime : Regime,
                         va : bits(64), size : integer, aligned : boolean,
                         accdesc : AccessDescriptor) => (FaultRecord, AddressDescriptor)
                         recurselimit Unbounded_DescriptorUpdate
begin
    var fault : FaultRecord = fault_in;
    // Prepare fault fields in case a fault is detected
    fault.secondstage = FALSE;
    fault.s2fs1walk   = FALSE;

    if !AArch64_S1Enabled(regime, accdesc.acctype) then
        return AArch64_S1DisabledOutput(fault, regime, va, accdesc, aligned);
    end;

    var walkparams : S1TTWParams = AArch64_GetS1TTWParams(regime, accdesc.el, accdesc.ss, va);
    let s1mintxsz : integer = AArch64_S1MinTxSZ(regime, walkparams);
    let s1maxtxsz : integer = AArch64_MaxTxSZ(walkparams.tgx);
    if AArch64_S1TxSZFaults(regime, walkparams) then
        fault.statuscode = Fault_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    elsif UInt(walkparams.txsz) < s1mintxsz then
        walkparams.txsz = s1mintxsz[5:0];
    elsif UInt(walkparams.txsz) > s1maxtxsz then
        walkparams.txsz = s1maxtxsz[5:0];
    end;

    if AArch64_VAIsOutOfRange(va, accdesc.acctype, regime, walkparams) then
        fault.statuscode = Fault_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if accdesc.el == EL0 && walkparams.e0pd == '1' then
        fault.statuscode = Fault_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if (IsFeatureImplemented(FEAT_SVE) && accdesc.el == EL0 && walkparams.nfd == '1' &&
          ((accdesc.nonfault && accdesc.contiguous) ||
           (accdesc.firstfault && !accdesc.first && !accdesc.contiguous))) then
        fault.statuscode = Fault_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    var descipaddr : AddressDescriptor;
    var walkstate : TTWState;
    var descriptor : bits(128);
    if walkparams.d128 == '1' then
        (fault, descipaddr, walkstate, descriptor) = AArch64_S1Walk{128}(fault, walkparams, va,
                                                                        regime, accdesc);
    else
        (fault, descipaddr, walkstate, descriptor[63:0]) = AArch64_S1Walk{64}(fault, walkparams, va,
                                                                             regime, accdesc);
        descriptor[127:64] = Zeros{64};
    end;
    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor);
    end;
    if AArch64_S1HasAlignmentFaultDueToMemType(regime, accdesc, aligned, walkparams.ntlsmd,
                                               walkstate.memattrs) then
        fault.statuscode = Fault_Alignment;
    end;

    let fault_perm : FaultRecord = AArch64_S1CheckPermissions(fault, va, size, regime,
                                                              walkstate, walkparams, accdesc);

    var mem_desc : bits(128);
    var new_desc : bits(128) = descriptor;

    if AArch64_SetAccessFlag(walkparams.ha, accdesc, fault, fault_perm) then
        // Set descriptor AF bit
        new_desc[10] = '1';
    end;

    // If HW update of dirty bit is enabled, the walk state permissions
    // will already reflect a configuration permitting writes.
    // The update of the descriptor occurs only if the descriptor bits in
    // memory do not reflect that and the access instigates a write.

    if AArch64_SetDirtyState(walkparams.hd, (walkparams.pie OR descriptor[51]),
                             accdesc, fault, fault_perm) then
        // Clear descriptor AP[2]/nDirty bit permitting stage 1 writes
```

```
    new_desc[7] = '0';
end;

if fault.statuscode == Fault_None && fault_perm.statuscode != Fault_None then
    fault = fault_perm;
end;

// Either the access flag was clear or AP[2]/nDirty was set
if new_desc != descriptor then
    if (!IsWBShareable(descipaddr.memattrs) &&
          ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
        fault.statuscode = Fault_UnsupportedAtomicHWUpdate;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    var descpaddr : AddressDescriptor;
    let descaccess : AccessDescriptor = CreateAccDescTTEUpdate(accdesc);

    if regime == Regime_EL10 && EL2Enabled() then
        var s2fault : FaultRecord;
        let s1aarch64 : boolean = TRUE;
        let s2aligned : boolean = TRUE;
        (s2fault, descpaddr) = AArch64_S2Translate(fault, descipaddr, s1aarch64, s2aligned,
                                                   descaccess);
        if s2fault.statuscode != Fault_None then
            return (s2fault, ARBITRARY : AddressDescriptor);
        end;

    else
        descpaddr = descipaddr;
    end;
    if walkparams.d128 == '1' then
        (fault, mem_desc) = AArch64_MemSwapTableDesc{128}(fault, descriptor,
                                                          new_desc, walkparams.ee,
                                                          descaccess, descpaddr);
    else
        (fault, mem_desc[63:0]) = AArch64_MemSwapTableDesc{64}(fault, descriptor[63:0],
                                                               new_desc[63:0], walkparams.ee,
                                                               descaccess, descpaddr);
        mem_desc[127:64] = Zeros{64};
    end;

    if fault.statuscode != Fault_None then
        if (accdesc.acctype == AccessType_AT &&
                !(ImpDefBool("AT reports the HW update fault"))) then
            // Mask the fault
            fault.statuscode = Fault_None;
        else
            return (fault, ARBITRARY : AddressDescriptor);
        end;
    elsif new_desc != descriptor && mem_desc != new_desc then
        // HW update of Dirty state or AF was not successful due to the descriptor being updated
        // not matching the descriptor used for translation. Due to this, the walk is restarted.
        return AArch64_S1Translate(fault_in, regime, va, size, aligned, accdesc);
    end;
end;

if fault.statuscode != Fault_None then
    return (fault, ARBITRARY : AddressDescriptor);
end;

// Output Address
let oa : FullAddress = Stage1OA(va, walkparams, walkstate);

var memattrs : MemoryAttributes;
if AArch64_S1TreatAsNormalNC(walkstate, regime, accdesc) then
   // Treat memory attributes as Normal Non-Cacheable
    memattrs = NormalNCMemAttr();
    memattrs.xs = walkstate.memattrs.xs;

    // The effect of SCTLR_ELx.C when '0' is Constrained UNPREDICTABLE on the Tagged attribute
    // when the memory region is Allocation Tagged using Physical MTE.
    if (IsPMTESelected(accdesc.el) && walkstate.memattrs.tags == MemTag_AllocationTagged &&
          ConstrainUnpredictableBool(Unpredictable_S1CTAGGED)) then
        memattrs.tags = MemTag_AllocationTagged;
    // SCTLR_ELx.C has no effect on whether the memory region is treated as Canonically Tagged.
    elsif (IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS) &&
             walkstate.memattrs.tags == MemTag_CanonicallyTagged) then
        memattrs.tags = MemTag_CanonicallyTagged;
    end;
else
    memattrs = walkstate.memattrs;
end;

// Shareability value of stage 1 translation subject to stage 2 is IMPLEMENTATION DEFINED
// to be either effective value or descriptor value
if (regime == Regime_EL10 && EL2Enabled() && HCR_EL2().VM == '1' &&
        !(ImpDefBool("Apply effective shareability at stage 1"))) then
    memattrs.shareability = walkstate.memattrs.shareability;
```

```
    else
        mattrs.shareability = EffectiveShareability(mattrs);
    end;

    var ipa : AddressDescriptor = CreateAddressDescriptor(va, oa, mattrs, accdesc);
    ipa.s1assured = walkstate.s1assured;
    let varange : VARange = AArch64_GetVARange(va);
    ipa.mecid = AArch64_S1OutputMECID{128}(walkparams, regime, varange, ipa.paddress.paspace,
                                           descriptor);

    if (accdesc.atomicop && !IsWBShareable(mattrs) &&
          ConstrainUnpredictableBool(Unpredictable_Atomic_MMU_IMPDEF_FAULT)) then
        fault.statuscode = Fault_Exclusive;
        return (fault, ipa);
    end;

    if accdesc.ls64 && mattrs.memtype == MemType_Normal then
        if IsFeatureImplemented(FEAT_LS64WB) && !accdesc.withstatus then
            if (!IsWBShareable(mattrs) &&
                  !(mattrs.inner.attrs == MemAttr_NC &&
                    mattrs.outer.attrs == MemAttr_NC) &&
                  (ImpDefBool(
                     "LD64B or ST64B faults to cacheable non-iWBoWB memory"))) then
                fault.statuscode = Fault_Exclusive;
                return (fault, ipa);
            end;
        elsif !(mattrs.inner.attrs == MemAttr_NC && mattrs.outer.attrs == MemAttr_NC) then
            fault.statuscode = Fault_Exclusive;
            return (fault, ipa);
        end;
    end;

    return (fault, ipa);
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_S1TreatAsNormalNC

```
// AArch64_S1TreatAsNormalNC()
// ===========================
// Returns TRUE if stage 1 memory attributes should be treated as Normal Non-Cacheable

readonly func AArch64_S1TreatAsNormalNC(walkstate : TTWState, regime : Regime,
                                        accdesc : AccessDescriptor) => boolean
begin
    return ((accdesc.acctype == AccessType_IFETCH &&
               (walkstate.mattrs.memtype == MemType_Device || !AArch64_S1ICacheEnabled(regime))) ||
            (accdesc.acctype != AccessType_IFETCH &&
               !AArch64_S1DCacheEnabled(regime) && walkstate.mattrs.memtype == MemType_Normal));
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_S2Translate

```
// AArch64_S2Translate()
// =====================
// Translate stage 1 IPA to PA and combine memory attributes

func AArch64_S2Translate(fault_in : FaultRecord, ipa : AddressDescriptor,
                         s1aarch64 : boolean, aligned : boolean,
                         accdesc : AccessDescriptor) => (FaultRecord, AddressDescriptor)
                         recurselimit Unbounded_DescriptorUpdate
begin
    var walkparams : S2TTWParams = AArch64_GetS2TTWParams(accdesc.ss, ipa.paddress.paspace,
                                                          s1aarch64);
    var fault : FaultRecord = fault_in;
    var s2fs1mro : boolean;
    // Prepare fault fields in case a fault is detected
    fault.statuscode = Fault_None; // Ignore any faults from stage 1
    fault.dirtybit     = FALSE;
    fault.overlay      = FALSE;
    fault.tagaccess    = FALSE;
    fault.s1tagnotdata = FALSE;
    fault.secondstage  = TRUE;
    fault.s2fs1walk    = accdesc.acctype == AccessType_TTW;
    fault.ipaddress    = ipa.paddress;

    if walkparams.vm != '1' then
        // Stage 2 translation is disabled
        return (fault, ipa);
    end;

    let s2mintxsz : integer = AArch64_S2MinTxSZ(walkparams, s1aarch64);
    let s2maxtxsz : integer = AArch64_MaxTxSZ(walkparams.tgx);
    if AArch64_S2TxSZFaults(walkparams, s1aarch64) then
        fault.statuscode = Fault_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    elsif UInt(walkparams.txsz) < s2mintxsz then
        walkparams.txsz = s2mintxsz[5:0];
    elsif UInt(walkparams.txsz) > s2maxtxsz then
        walkparams.txsz = s2maxtxsz[5:0];
    end;

    if (walkparams.d128 == '0' &&
        (AArch64_S2InvalidSL(walkparams) || AArch64_S2InconsistentSL(walkparams))) then
        fault.statuscode = Fault_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if AArch64_IPAIsOutOfRange(ipa.paddress.address, walkparams) then
        fault.statuscode = Fault_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    var descpaddr : AddressDescriptor;
    var walkstate : TTWState;
    var descriptor : bits(128);
    if walkparams.d128 == '1' then
        (fault, descpaddr, walkstate, descriptor) = AArch64_S2Walk{128}(fault, ipa, walkparams,
                                                                       accdesc);
    else
        (fault, descpaddr, walkstate, descriptor[63:0]) = AArch64_S2Walk{64}(fault, ipa, walkparams,
                                                                            accdesc);
        descriptor[127:64] = Zeros{64};
    end;
    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if AArch64_S2HasAlignmentFaultDueToMemType(accdesc, aligned, walkstate.memattrs) then
        fault.statuscode = Fault_Alignment;
    end;

    var fault_perm : FaultRecord;
    (fault_perm, s2fs1mro) = AArch64_S2CheckPermissions(fault, walkstate, walkparams, ipa, accdesc);

    var mem_desc : bits(128);
    var new_desc : bits(128) = descriptor;

    if AArch64_SetAccessFlag(walkparams.ha, accdesc, fault, fault_perm) then
        // Set descriptor AF bit
        new_desc[10] = '1';
    end;

    // If HW update of dirty bit is enabled, the walk state permissions
    // will already reflect a configuration permitting writes.
    // The update of the descriptor occurs only if the descriptor bits in
    // memory do not reflect that and the access instigates a write.

    if AArch64_SetDirtyState(walkparams.hd, (walkparams.s2pie OR descriptor[51]),
```

```
                          accdesc, fault, fault_perm) then
    // Set descriptor S2AP[1]/Dirty bit permitting stage 2 writes
    new_desc[7] = '1';
end;

if fault.statuscode == Fault_None && fault_perm.statuscode != Fault_None then
    fault = fault_perm;
end;

// Either the access flag was clear or S2AP[1]/Dirty was clear
if new_desc != descriptor then
    if (!IsWBShareable(descpaddr.memattrs) &&
          ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
        fault.statuscode = Fault_UnsupportedAtomicHWUpdate;
        return (fault, ARBITRARY : AddressDescriptor);
    end;

    if walkparams.hdbss == '1' && descriptor[7] == '0' && new_desc[7] == '1' then
        fault = AppendToHDBSS(fault, ipa.paddress, accdesc, walkparams, walkstate.level);
    end;

    // If an error, other than a synchronous External abort, occurred on the HDBSS update,
    // stage 2 hardware update of dirty state is not permitted.
    if (!(walkparams.hdbss == '1' && descriptor[7] != new_desc[7]) ||
          (HDBSSPROD_EL2().FSC != '101000' &&
            (!fault.hdbssf || IsExternalAbort(fault.statuscode)))) then
        let descaccess : AccessDescriptor = CreateAccDescTTEUpdate(accdesc);
        if walkparams.d128 == '1' then
            (fault, mem_desc) = AArch64_MemSwapTableDesc{128}(fault, descriptor, new_desc,
                                                              walkparams.ee, descaccess,
                                                              descpaddr);
        else
            (fault, mem_desc[63:0]) = AArch64_MemSwapTableDesc{64}(fault, descriptor[63:0],
                                                                   new_desc[63:0], walkparams.ee,
                                                                   descaccess, descpaddr);
            mem_desc[127:64] = Zeros{64};
        end;
    end;

    if fault.statuscode != Fault_None then
        if (accdesc.acctype == AccessType_AT &&
                !(ImpDefBool("AT reports the HW update fault"))) then
            // Mask the fault
            fault.statuscode = Fault_None;
        else
            return (fault, ARBITRARY : AddressDescriptor);
        end;
    elsif new_desc != descriptor && mem_desc != new_desc then
        // HW update of Dirty state or AF was not successful due to the descriptor being updated
        // not matching the descriptor used for translation. Due to this, the walk is restarted.
        return AArch64_S2Translate(fault_in, ipa, s1aarch64, aligned, accdesc);
    end;
end;

if fault.statuscode != Fault_None then
    return (fault, ARBITRARY : AddressDescriptor);
end;

// Output Address
let oa : FullAddress = Stage2OA(ipa.paddress.address, walkparams, walkstate);
var s2_mematttrs : MemoryAttributes;

if AArch64_S2TreatAsNormalNC(walkstate, walkparams, accdesc) then
    // Treat memory attributes as Normal Non-Cacheable
    s2_mematttrs = NormalNCMemAttr();
    s2_mematttrs.xs = walkstate.mematttrs.xs;
    if walkstate.mematttrs.tags == MemTag_CanonicallyTagged then
        s2_mematttrs.tags = MemTag_CanonicallyTagged;
    end;
else
    s2_mematttrs = walkstate.mematttrs;
end;

let s2aarch64 : boolean = TRUE;
var mematttrs : MemoryAttributes;
if walkparams.fwb == '0' then
    mematttrs = S2CombineS1MemAttrs(ipa.mematttrs, s2_mematttrs, s2aarch64);
else
    mematttrs = s2_mematttrs;
end;

var pa : AddressDescriptor = CreateAddressDescriptor(ipa.vaddress, oa, mematttrs, accdesc);
pa.s2fs1mro = s2fs1mro;
pa.mecid = AArch64_S2OutputMECID{128}(walkparams, pa.paddress.paspace, descriptor);

if (accdesc.acctype == AccessType_TTW && accdesc.atomicop && !IsWBShareable(pa.mematttrs) &&
      ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
    fault.statuscode = Fault_UnsupportedAtomicHWUpdate;
    return (fault, pa);
```

```
    end;

    if (accdesc.acctype != AccessType_TTW && accdesc.atomicop && !IsWBShareable(pa.mattrs) &&
          ConstrainUnpredictableBool(Unpredictable_Atomic_MMU_IMPDEF_FAULT)) then
        fault.statuscode = Fault_Exclusive;
        return (fault, pa);
    end;

    if accdesc.ls64 && s2_mattrs.memtype == MemType_Normal then
        if IsFeatureImplemented(FEAT_LS64WB) && !accdesc.withstatus then
            if (!IsWBShareable(s2_mattrs) &&
                  !(s2_mattrs.inner.attrs == MemAttr_NC &&
                    s2_mattrs.outer.attrs == MemAttr_NC) &&
                  (ImpDefBool(
                     "LD64B or ST64B faults to cacheable non-iWBoWB memory"))) then
                fault.statuscode = Fault_Exclusive;
                return (fault, pa);
            end;
        elsif !(s2_mattrs.inner.attrs == MemAttr_NC && s2_mattrs.outer.attrs == MemAttr_NC) then
            fault.statuscode = Fault_Exclusive;
            return (fault, pa);
        end;
    end;

    return (fault, pa);
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_S2TreatAsNormalNC

```
// AArch64_S2TreatAsNormalNC()
// ===========================
// Returns TRUE if stage 2 memory attributes should be treated as Normal Non-cacheable

readonly func AArch64_S2TreatAsNormalNC(walkstate : TTWState, walkparams : S2TTWParams,
                                        accdesc : AccessDescriptor) => boolean
begin
    return ((accdesc.acctype == AccessType_TTW &&
               walkstate.mattrs.memtype == MemType_Device && walkparams.ptw == '0') ||
            (accdesc.acctype == AccessType_IFETCH &&
               (walkstate.mattrs.memtype == MemType_Device || !S2ICacheEnabled())) ||
            (accdesc.acctype != AccessType_IFETCH &&
               walkstate.mattrs.memtype == MemType_Normal && !S2DCacheEnabled()));
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_SetAccessFlag

```
// AArch64_SetAccessFlag()
// =======================
// Determine whether the access flag could be set by HW given the fault status

func AArch64_SetAccessFlag(ha : bit, accdesc : AccessDescriptor, fault : FaultRecord,
                           fault_perm : FaultRecord) => boolean
begin
    if ha == '0' || !AArch64_SettingAccessFlagPermitted(fault, fault_perm) then
        return FALSE;
    elsif accdesc.acctype == AccessType_AT then
        return ImpDefBool("AT updates AF");
    elsif accdesc.acctype IN {AccessType_DC, AccessType_IC} then
        return ImpDefBool("Generate access flag fault on IC/DC operations");
    else
        // Set descriptor AF bit
        return TRUE;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_SetDirtyState

```
// AArch64_SetDirtyState()
// =======================
// Determine whether dirty state is required to be updated by HW given the fault status

func AArch64_SetDirtyState(hd : bits(1), dbm : bits(1),
                           accdesc : AccessDescriptor,
                           fault : FaultRecord,
                           fault_perm : FaultRecord) => boolean
begin
    if hd == '0' then
        return FALSE;
    elsif !AArch64_SettingDirtyStatePermitted(fault, fault_perm) then
        return FALSE;
    elsif accdesc.acctype IN {AccessType_AT, AccessType_IC, AccessType_DC} then
        return FALSE;
    elsif !accdesc.write then
        return FALSE;
    else
        return dbm == '1';
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_SettingAccessFlagPermitted

```
// AArch64_SettingAccessFlagPermitted()
// ====================================
// Determine whether the access flag could be set by HW given the fault status

func AArch64_SettingAccessFlagPermitted(fault : FaultRecord, fault_perm : FaultRecord) => boolean
begin
    if fault.statuscode == Fault_None && fault_perm.statuscode == Fault_None then
        return TRUE;
    elsif fault.statuscode == Fault_Alignment || fault_perm.statuscode == Fault_Permission then
        return ConstrainUnpredictableBool(Unpredictable_AFUPDATE);
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_SettingDirtyStatePermitted

```
// AArch64_SettingDirtyStatePermitted()
// ====================================
// Determine whether the dirty state could be set by HW given the fault status

func AArch64_SettingDirtyStatePermitted(fault : FaultRecord, fault_perm : FaultRecord) => boolean
begin
    if fault_perm.statuscode != Fault_None then
        return FALSE;
    elsif fault.statuscode == Fault_None then
        return TRUE;
    elsif fault.statuscode == Fault_Alignment then
        return ConstrainUnpredictableBool(Unpredictable_DBUPDATE);
    else
        return FALSE;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_TranslateAddress

```
// AArch64_TranslateAddress()
// ==========================
// Main entry point for translating an address

func AArch64_TranslateAddress(va : bits(64), accdesc : AccessDescriptor,
                              aligned : boolean, size : integer) => AddressDescriptor
begin
    if (SPESampleInFlight &&
        !(accdesc.acctype IN {AccessType_IFETCH, AccessType_SPE, AccessType_TRBE})) then
        SPEStartCounter(SPECounterPosTranslationLatency);
    end;

    var result : AddressDescriptor = AArch64_FullTranslate(va, size, accdesc, aligned);

    if (!IsFault(result) && accdesc.acctype != AccessType_IFETCH &&
          !ImpDefBool(
              "GPF, GPC, Data SyncExternal Aborts, TagChecks prioritized over Watchpoints")) then
        result.fault = AArch64_CheckDebug(va, accdesc, size);
    end;

    if (IsFeatureImplemented(FEAT_RME) && !IsFault(result) &&
          IsGranuleProtectionCheckedAccess(accdesc)) then
        result.fault.gpcf = GranuleProtectionCheck(result, accdesc);

        if result.fault.gpcf.gpf != GPCF_None then
            result.fault.statuscode = Fault_GPCFOnOutput;
            result.fault.paddress   = result.paddress;
            result.fault.vaddress   = result.vaddress;
        end;
    end;

    if (!IsFault(result) && accdesc.acctype == AccessType_IFETCH &&
          !ImpDefBool("Instruction SyncExternal Aborts prioritized over Breakpoints")) then
        result.fault = AArch64_CheckDebug(va, accdesc, size);
    end;

    if SPESampleInFlight &&
        !(accdesc.acctype IN {AccessType_IFETCH, AccessType_TRBE, AccessType_SPE}) then
        SPEStopCounter(SPECounterPosTranslationLatency);
    end;

    // Update virtual address for abort functions
    result.vaddress = ZeroExtend{64}(va);

    return result;
end;
```

Library pseudocode for aarch64/translation/vmsa_translation/AArch64_TranslateTagAddress

```
// AArch64_TranslateTagAddress()
// =============================
// Translate an address for accessing an Allocation Tag.

func AArch64_TranslateTagAddress(va : bits(64),
                                 accdesc_in : AccessDescriptor,
                                 aligned : boolean,
                                 size : integer) => (MemTagType, AddressDescriptor)
begin
    assert !accdesc_in.tagchecked;
    var accdesc : AccessDescriptor = accdesc_in;
    let taddrdesc : AddressDescriptor = AArch64_TranslateAddress(va, accdesc, aligned, size);

    return (taddrdesc.memattrs.tags, taddrdesc);
end;
```

Library pseudocode for aarch64/translation/vmsa_ttentry/AArch64_BlockDescSupported

```
// AArch64_BlockDescSupported()
// ============================
// Determine whether a block descriptor is valid for the given granule size
// and level

func AArch64_BlockDescSupported(d128 : bit, ds : bit, tgx : TGx, level : integer) => boolean
begin
    case tgx of
        when TGx_4KB => return ((level == 0 && (ds == '1' || d128 == '1')) ||
                                level == 1 ||
                                level == 2);
        when TGx_16KB => return ((level == 1 && (ds == '1' || d128 == '1')) ||
                                   level == 2);
        when TGx_64KB => return ((level == 1 && (d128 == '1' || AArch64_PAMax() >= 52)) ||
                                   level == 2);
    end;
    return FALSE;
end;
```

Library pseudocode for aarch64/translation/vmsa_ttentry/AArch64_ContiguousBit

```
// AArch64_ContiguousBit()
// =======================
// Get the value of the contiguous bit

func AArch64_ContiguousBit{N}(tgx : TGx, d128 : bit, level : integer, descriptor : bits(N)) => bit
begin
    if ImpDefBool("Treat Contiguous bit as 0") then
        return '0';
    end;

    if d128 == '1' then
        if (tgx == TGx_64KB && level == 1) || (tgx == TGx_4KB && level == 0) then
            return '0'; // RES0
        else
            return descriptor[111];
        end;
    end;
    // When using TGx 64KB then the Contiguous bit is RES0 for Block
    // descriptors at level 1. For VMSAv8-64, level 1 Block descriptors
    // can exist only if FEAT_LPA is implemented.

    if tgx == TGx_64KB && level == 1 then
        return '0'; // RES0
    end;

    // When the Effective value of TCR_ELx.DS is '1',
    // the Contiguous bit is RES0 for all the following:
    //      * For TGx 4KB, Block descriptors at level 0
    //      * For TGx 16KB, Block descriptors at level 1
    // For VMSAv8-64, the above Block descriptors at the specified
    // levels can exist only if the Effective value of TCR_ELx.DS is 1.

    if tgx == TGx_16KB && level == 1 then
        return '0'; // RES0
    end;

    if tgx == TGx_4KB  && level == 0 then
        return '0'; // RES0
    end;

    return descriptor[52];
end;
```

Library pseudocode for aarch64/translation/vmsa_ttentry/AArch64_DecodeDescriptorType

```
// AArch64_DecodeDescriptorType()
// ==============================
// Determine whether the descriptor is a page, block or table

func AArch64_DecodeDescriptorType{N}(descriptor : bits(N), d128 : bit, ds : bit,
                                     tgx : TGx, level : integer) => DescriptorType
begin
    if descriptor[0] == '0' then
        return DescriptorType_Invalid;
    elsif d128 == '1' then
        let skl : bits(2) = descriptor[110:109];
        if tgx IN {TGx_16KB, TGx_64KB} && UInt(skl) == 3 then
            return DescriptorType_Invalid;
        end;

        let effective_level : integer = level + UInt(skl);
        if effective_level > FINAL_LEVEL then
            return DescriptorType_Invalid;
        elsif effective_level == FINAL_LEVEL then
            return DescriptorType_Leaf;
        else
            return DescriptorType_Table;
        end;
    else
        if descriptor[1] == '1' then
            if level == FINAL_LEVEL then
                return DescriptorType_Leaf;
            else
                return DescriptorType_Table;
            end;
        elsif descriptor[1] == '0' then
            if AArch64_BlockDescSupported(d128, ds, tgx, level) then
                return DescriptorType_Leaf;
            else
                return DescriptorType_Invalid;
            end;
        end;
        unreachable;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_ttentry/AArch64_S1ApplyOutputPerms

```
// AArch64_S1ApplyOutputPerms()
// ============================
// Apply output permissions encoded in stage 1 page/block descriptors

func AArch64_S1ApplyOutputPerms{N}(permissions_in : Permissions, descriptor : bits(N),
                                   regime : Regime, walkparams : S1TTWParams) => Permissions
begin
    var permissions : Permissions = permissions_in;

    var pi_index : bits (4);
    if walkparams.pie == '1' then
        if walkparams.d128 == '1' then
            pi_index = descriptor[118:115];
        else
            pi_index = descriptor[54:53]::descriptor[51]::descriptor[6];
        end;
        permissions.ppi    = walkparams.pir[UInt(pi_index)*:4];
        permissions.upi    = walkparams.pire0[UInt(pi_index)*:4];
        permissions.ndirty = descriptor[7];
    else
        if regime == Regime_EL10 && EL2Enabled() && walkparams.nv1 == '1' then
            permissions.ap[2:1] = descriptor[7]::'0';
            permissions.pxn     = descriptor[54];
        elsif HasUnprivileged(regime) then
            permissions.ap[2:1] = descriptor[7:6];
            permissions.uxn     = descriptor[54];
            permissions.pxn     = descriptor[53];
        else
            permissions.ap[2:1] = descriptor[7]::'1';
            permissions.xn      = descriptor[54];
        end;
        permissions.dbm = descriptor[51];
    end;
    if IsFeatureImplemented(FEAT_S1POE) then
        if walkparams.d128 == '1' then
            permissions.po_index[3:0] = descriptor[124:121];
        else
            permissions.po_index[3:0] = '0'::descriptor[62:60];
        end;
    end;

    return permissions;
end;
```

Library pseudocode for aarch64/translation/vmsa_ttentry/AArch64_S1ApplyTablePerms

```
// AArch64_S1ApplyTablePerms()
// ===========================
// Apply hierarchical permissions encoded in stage 1 table descriptors

func AArch64_S1ApplyTablePerms(permissions_in : Permissions, descriptor : bits(64),
                               regime : Regime, walkparams : S1TTWParams) => Permissions
begin
    var permissions : Permissions = permissions_in;
    var ap_table : bits(2);
    var pxn_table : bit;
    var uxn_table : bit;
    var xn_table : bit;
    if regime == Regime_EL10 && EL2Enabled() && walkparams.nv1 == '1' then
        ap_table  = descriptor[62]::'0';
        pxn_table = descriptor[60];
        permissions.ap_table  = permissions.ap_table  OR ap_table;
        permissions.pxn_table = permissions.pxn_table OR pxn_table;

    elsif HasUnprivileged(regime) then
        ap_table  = descriptor[62:61];
        uxn_table = descriptor[60];
        pxn_table = descriptor[59];
        permissions.ap_table  = permissions.ap_table  OR ap_table;
        permissions.uxn_table = permissions.uxn_table OR uxn_table;
        permissions.pxn_table = permissions.pxn_table OR pxn_table;
    else
        ap_table = descriptor[62]::'0';
        xn_table = descriptor[60];
        permissions.ap_table = permissions.ap_table OR ap_table;
        permissions.xn_table = permissions.xn_table OR xn_table;
    end;

    return permissions;
end;
```

Library pseudocode for aarch64/translation/vmsa_ttentry/AArch64_S2ApplyOutputPerms

```
// AArch64_S2ApplyOutputPerms()
// ============================
// Apply output permissions encoded in stage 2 page/block descriptors

func AArch64_S2ApplyOutputPerms{N}(descriptor : bits(N), walkparams : S2TTWParams) => Permissions
begin
    var permissions : Permissions;
    var s2pi_index : bits(4);
    if walkparams.s2pie == '1' then
        if walkparams.d128 == '1' then
            s2pi_index = descriptor[118:115];
        else
            s2pi_index = descriptor[54:53,51,6];
        end;
        permissions.s2pi = walkparams.s2pir[UInt(s2pi_index)*:4];
        permissions.s2dirty = descriptor[7];
    else
        permissions.s2ap = descriptor[7:6];
        if walkparams.d128 == '1' then
            permissions.s2xn = descriptor[118];
        else
            permissions.s2xn = descriptor[54];
        end;

        if IsFeatureImplemented(FEAT_XNX) then
            if walkparams.d128 == '1' then
                permissions.s2xnx = descriptor[117];
            else
                permissions.s2xnx = descriptor[53];
            end;
        else
            permissions.s2xnx = '0';
        end;

        permissions.dbm = descriptor[51];
    end;
    if IsFeatureImplemented(FEAT_S2POE) then
        if walkparams.d128 == '1' then
            permissions.s2po_index = descriptor[124:121];
        else
            permissions.s2po_index = descriptor[62:59];
        end;
    end;
    return permissions;
end;
```

Library pseudocode for aarch64/translation/vmsa_ttentry/AArch64_nTFaults

```
// AArch64_nTFaults()
// ==================
// Identify whether the nT bit in a block or table descriptor is effectively set
// causing a translation fault

func AArch64_nTFaults{N}(d128 : bit, descriptor : bits(N)) => boolean
begin
    if !IsFeatureImplemented(FEAT_BBML1) then
        return FALSE;
    end;
    let nT : bit = if d128 == '1' then descriptor[6] else descriptor[16];

    return nT == '1' && ImpDefBool("nT bit causes Translation Fault");
end;
```

Library pseudocode for aarch64/translation/vmsa_walk/AArch64_S1InitialTTWState

```
// AArch64_S1InitialTTWState()
// ===========================
// Set properties of first access to translation tables in stage 1

func AArch64_S1InitialTTWState(walkparams : S1TTWParams, va : bits(64), regime : Regime,
                               ss : SecurityState) => TTWState
begin
    var walkstate : TTWState;
    var tablebase : FullAddress;
    var permissions : Permissions;
    var ttbr : bits(128);

    ttbr              = AArch64_S1TTBR(regime, va);
    case ss of
        when SS_Secure =>    tablebase.paspace = PAS_Secure;
        when SS_NonSecure => tablebase.paspace = PAS_NonSecure;
        when SS_Root =>      tablebase.paspace = PAS_Root;
        when SS_Realm =>     tablebase.paspace = PAS_Realm;
    end;

    tablebase.address = AArch64_S1TTBaseAddress{128}(walkparams, regime, ttbr);

    permissions.ap_table = '00';
    if HasUnprivileged(regime) then
        permissions.uxn_table = '0';
        permissions.pxn_table = '0';
    else
        permissions.xn_table  = '0';
    end;

    walkstate.baseaddress = tablebase;
    walkstate.level       = AArch64_S1StartLevel(walkparams);
    walkstate.istable     = TRUE;
    // In regimes that support global and non-global translations, translation
    // table entries from lookup levels other than the final level of lookup
    // are treated as being non-global
    walkstate.nG          = if HasUnprivileged(regime) then '1' else '0';
    walkstate.memattrs    = WalkMemAttrs(walkparams.sh, walkparams.irgn, walkparams.orgn);
    walkstate.permissions = permissions;
    if regime == Regime_EL10 && EL2Enabled() && HCR_EL2().VM == '1' then
        if ((AArch64_GetVARange(va) == VARange_LOWER && VTCR_EL2().TL0 == '1') ||
            (AArch64_GetVARange(va) == VARange_UPPER && VTCR_EL2().TL1 == '1')) then
            walkstate.s1assured = TRUE;
        else
            walkstate.s1assured = FALSE;
        end;
    else
        walkstate.s1assured = FALSE;
    end;
    walkstate.disch = walkparams.disch;

    return walkstate;
end;
```

Library pseudocode for aarch64/translation/vmsa_walk/AArch64_S1NextWalkStateLeaf

```
// AArch64_S1NextWalkStateLeaf()
// =============================
// Decode stage 1 page or block descriptor as output to this stage of translation

func AArch64_S1NextWalkStateLeaf{N}(currentstate : TTWState, s2fs1mro : boolean, regime : Regime,
                                    accdesc : AccessDescriptor, walkparams : S1TTWParams,
                                    descriptor : bits(N)) => TTWState
begin
    var nextstate : TTWState;
    var baseaddress : FullAddress;
    baseaddress.address = AArch64_S1LeafBase{N}(descriptor, walkparams, currentstate.level);

    if currentstate.baseaddress.paspace == PAS_Secure then
        // Determine PA space of the block from NS bit
        let ns : bit = if walkparams.d128 == '1' then descriptor[127] else descriptor[5];
        baseaddress.paspace = if ns == '0' then PAS_Secure else PAS_NonSecure;
    elsif currentstate.baseaddress.paspace == PAS_Root then
        // Determine PA space of the block from NSE and NS bits
        let ns : bit = if walkparams.d128 == '1' then descriptor[127] else descriptor[5];
        let nse : bit = descriptor[11];
        let nse2 : bit = '0';     // NSE2 has the Effective value of 0 within a PE.
        baseaddress.paspace = DecodePASpace(nse2, nse, ns);

        // If Secure state is not implemented, but RME is,
        // force Secure space accesses to Non-secure space
        if baseaddress.paspace == PAS_Secure && !HaveSecureState() then
            baseaddress.paspace = PAS_NonSecure;
        end;

    elsif (currentstate.baseaddress.paspace == PAS_Realm &&
           regime IN {Regime_EL2, Regime_EL20}) then
        // Realm EL2 and EL2&0 regimes have a stage 1 NS bit
        let ns : bit = if walkparams.d128 == '1' then descriptor[127] else descriptor[5];
        baseaddress.paspace = if ns == '0' then PAS_Realm else PAS_NonSecure;
    elsif currentstate.baseaddress.paspace == PAS_Realm then
        // Realm EL1&0 regime does not have a stage 1 NS bit
        baseaddress.paspace = PAS_Realm;
    else
        baseaddress.paspace = PAS_NonSecure;
    end;

    nextstate.istable     = FALSE;
    nextstate.level       = currentstate.level;
    nextstate.baseaddress = baseaddress;

    var attrindx : bits(4);
    if walkparams.aie == '1' then
        if walkparams.d128 == '1' then
            attrindx = descriptor[5:2];
        else
            attrindx = descriptor[59,4:2];
        end;
    else
        attrindx = '0'::descriptor[4:2];
    end;

    var sh : bits(2);
    if walkparams.d128 == '1' then
        sh = descriptor[9:8];
    elsif walkparams.ds == '1' then
        sh = walkparams.sh;
    else
        sh = descriptor[9:8];
    end;
    let attr : bits(8) = AArch64_MAIRAttr(UInt(attrindx), walkparams.mair2, walkparams.mair);
    let s1aarch64 : boolean = TRUE;

    nextstate.memattrs = S1DecodeMemAttrs(attr, sh, s1aarch64, walkparams, accdesc.acctype);
    nextstate.permissions = AArch64_S1ApplyOutputPerms{N}(currentstate.permissions,
                                                          descriptor, regime, walkparams);
    var protectedbit : bit;
    if walkparams.d128 == '1' then
        protectedbit = descriptor[114];
    else
        protectedbit = if walkparams.pnch == '1' then descriptor[52] else '0';
    end;
    if (currentstate.s1assured && s2fs1mro && protectedbit == '1') then
        nextstate.s1assured = TRUE;
    else
        nextstate.s1assured = FALSE;
    end;

    if walkparams.pnch == '1' || currentstate.disch == '1' then
        nextstate.contiguous = '0';
    else
        nextstate.contiguous = AArch64_ContiguousBit{N}(walkparams.tgx, walkparams.d128,
                                                        currentstate.level, descriptor);
    end;
    if !HasUnprivileged(regime) then
```

```
        nextstate.nG = '0';
    elsif accdesc.ss == SS_Secure && currentstate.baseaddress.paspace == PAS_NonSecure then
        // In Secure state, a translation must be treated as non-global,
        // regardless of the value of the nG bit,
        // if NSTable is set to 1 at any level of the translation table walk
        nextstate.nG = '1';
    elsif walkparams.fng == '1' then
        // Translations are treated as non-global regardless of the value of the nG bit.
        nextstate.nG = '1';
    elsif (regime == Regime_EL10 && EL2Enabled() && HCR_EL2().VM == '1' &&
            (walkparams.d128 == '1' || walkparams.pnch == '1') &&
            !nextstate.s1assured && walkparams.fngna == '1') then
        // Translations are treated as non-global regardless of the value of the nG bit.
        nextstate.nG = '1';
    else
        nextstate.nG = descriptor[11];
    end;

    if walkparams.d128 == '1' then
        nextstate.guardedpage = descriptor[113];
    else
        nextstate.guardedpage = descriptor[50];
    end;

    return nextstate;
end;
```

Library pseudocode for aarch64/translation/vmsa_walk/AArch64_S1NextWalkStateTable

```
// AArch64_S1NextWalkStateTable()
// ==============================
// Decode stage 1 table descriptor to transition to the next level

func AArch64_S1NextWalkStateTable{N}(currentstate : TTWState, s2fs1mro : boolean, regime : Regime,
                                     walkparams : S1TTWParams, descriptor : bits(N)) => TTWState
begin
    var nextstate : TTWState;
    var tablebase : FullAddress;
    let skl : bits(2) = if walkparams.d128 == '1' then descriptor[110:109] else '00';
    tablebase.address = AArch64_NextTableBase{N}(descriptor, walkparams.d128, skl, walkparams.ds,
                                                 walkparams.tgx);

    if currentstate.baseaddress.paspace == PAS_Secure then
        // Determine PA space of the next table from NSTable bit
        var nstable : bit;
        nstable = if walkparams.d128 == '1' then descriptor[127] else descriptor[63];
        tablebase.paspace = if nstable == '0' then PAS_Secure else PAS_NonSecure;
    else
        // Otherwise bit 63 is RES0 and there is no NSTable bit
        tablebase.paspace = currentstate.baseaddress.paspace;
    end;

    nextstate.istable     = TRUE;
    nextstate.nG          = currentstate.nG;
    if walkparams.d128 == '1' then
        nextstate.level   = currentstate.level + UInt(skl) + 1;
    else
        nextstate.level   = currentstate.level + 1;
    end;
    nextstate.baseaddress = tablebase;
    nextstate.memattrs    = currentstate.memattrs;
    if walkparams.hpd == '0' && walkparams.pie == '0' then
        nextstate.permissions = AArch64_S1ApplyTablePerms(currentstate.permissions,
                                                          descriptor[63:0], regime, walkparams);
    else
        nextstate.permissions = currentstate.permissions;
    end;
    var protectedbit : bit;
    if walkparams.d128 == '1' then
        protectedbit = descriptor[114];
    else
        protectedbit = if walkparams.pnch == '1' then descriptor[52] else '0';
    end;
    if (currentstate.s1assured && s2fs1mro && protectedbit == '1') then
        nextstate.s1assured = TRUE;
    else
        nextstate.s1assured = FALSE;
    end;
    nextstate.disch = if walkparams.d128 == '1' then descriptor[112] else '0';

    return nextstate;
end;
```

Library pseudocode for aarch64/translation/vmsa_walk/AArch64_S1Walk

```
// AArch64_S1Walk()
// ================
// Traverse stage 1 translation tables obtaining the final descriptor
// as well as the address leading to that descriptor

func AArch64_S1Walk{N : integer{64, 128}}(fault_in : FaultRecord,
                                          walkparams : S1TTWParams,
                                          va : bits(64), regime : Regime,
                                          accdesc : AccessDescriptor
                                         ) => (FaultRecord, AddressDescriptor, TTWState, bits(N))
begin
    var fault : FaultRecord = fault_in;
    var aligned : boolean;

    if HasUnprivileged(regime) && AArch64_S1EPD(regime, va) == '1' then
        fault.statuscode = Fault_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    var walkstate : TTWState = AArch64_S1InitialTTWState(walkparams, va, regime, accdesc.ss);
    let startlevel : integer = walkstate.level;

    if startlevel > 3 then
        fault.statuscode = Fault_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    var descriptor : bits(N);
    var walkaddress : AddressDescriptor;
    var skl : bits(2) = '00';
    walkaddress.vaddress = va;
    walkaddress.mecid = AArch64_S1TTWalkMECID(walkparams.emec, regime, accdesc.ss);

    if !AArch64_S1DCacheEnabled(regime) then
        walkaddress.mematttrs = NormalNCMemAttr();
        walkaddress.mematttrs.xs = walkstate.mematttrs.xs;
    else
        walkaddress.mematttrs = walkstate.mematttrs;
    end;

    // Shareability value of stage 1 translation subject to stage 2 is IMPLEMENTATION DEFINED
    // to be either effective value or descriptor value
    if (regime == Regime_EL10 && EL2Enabled() && HCR_EL2().VM == '1' &&
            !(ImpDefBool("Apply effective shareability at stage 1"))) then
        walkaddress.mematttrs.shareability = walkstate.mematttrs.shareability;
    else
        walkaddress.mematttrs.shareability = EffectiveShareability(walkaddress.mematttrs);
    end;

    var s2fs1mro : boolean = FALSE;

    var desctype : DescriptorType;
    var descaddress : FullAddress = AArch64_S1SLTTEntryAddress(walkstate.level, walkparams, va,
                                                               walkstate.baseaddress);

    // Detect Address Size Fault by Descriptor Address
    if AArch64_S1OAOutOfRange(descaddress.address, walkparams) then
        fault.statuscode = Fault_AddressSize;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    repeat
        fault.level = walkstate.level;
        walkaddress.paddress = descaddress;
        walkaddress.s1assured = walkstate.s1assured;

        let toplevel : boolean = walkstate.level == startlevel;
        let varange : VARange  = AArch64_GetVARange(va);
        let walkaccess : AccessDescriptor = CreateAccDescS1TTW(toplevel, varange, accdesc);
        var s2fault : FaultRecord;
        var s2walkaddress : AddressDescriptor;
        if regime == Regime_EL10 && EL2Enabled() then
            let s1aarch64 : boolean = TRUE;
            aligned   = TRUE;
            (s2fault, s2walkaddress) = AArch64_S2Translate(fault, walkaddress, s1aarch64, aligned,
                                                           walkaccess);

            if s2fault.statuscode != Fault_None then
                return (s2fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                        ARBITRARY : bits(N));
            end;

            s2fs1mro = s2walkaddress.s2fs1mro;
            (fault, descriptor) = FetchDescriptor{N}(walkparams.ee, s2walkaddress, walkaccess,
                                                     fault);
        else
```

```
        (fault, descriptor) = FetchDescriptor{N}(walkparams.ee, walkaddress, walkaccess,
                                                 fault);
    end;

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                ARBITRARY : bits(N));
    end;

    var new_descriptor : bits(N);
    repeat
        new_descriptor = descriptor;
        desctype = AArch64_DecodeDescriptorType{N}(descriptor, walkparams.d128, walkparams.ds,
                                                   walkparams.tgx, walkstate.level);
        case desctype of
            when DescriptorType_Table =>
                walkstate = AArch64_S1NextWalkStateTable{N}(walkstate, s2fs1mro,
                                                            regime, walkparams, descriptor);
                skl = if walkparams.d128 == '1' then descriptor[110:109] else '00';
                descaddress = AArch64_S1TTEntryAddress{N}(walkstate.level, walkparams, skl, va,
                                                          walkstate.baseaddress, descriptor);

                // Detect Address Size Fault by Descriptor Address
                if AArch64_S1OAOutOfRange(descaddress.address, walkparams) then
                    fault.statuscode = Fault_AddressSize;
                    return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                            ARBITRARY : bits(N));
                end;

                if walkparams.haft == '1' then
                    new_descriptor[10] = '1';
                end;
                if (walkparams.d128 == '1' && skl != '00' &&
                      AArch64_nTFaults{N}(walkparams.d128, descriptor)) then
                    fault.statuscode = Fault_Translation;
                    return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                            ARBITRARY : bits(N));
                end;
            when DescriptorType_Leaf =>
                walkstate = AArch64_S1NextWalkStateLeaf{N}(walkstate, s2fs1mro, regime, accdesc,
                                                           walkparams, descriptor);
            when DescriptorType_Invalid =>
                fault.statuscode = Fault_Translation;
                return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                        ARBITRARY : bits(N));
            otherwise =>
                unreachable;
        end;

        if new_descriptor != descriptor then

            if (!IsWBShareable(walkaddress.memattrs) &&
                  ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
                fault.statuscode = Fault_UnsupportedAtomicHWUpdate;
                return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                        ARBITRARY : bits(N));
            end;

            var descpaddr : AddressDescriptor;
            let descaccess : AccessDescriptor = CreateAccDescTTEUpdate(accdesc);
            if regime == Regime_EL10 && EL2Enabled() then
                let s1aarch64 : boolean = TRUE;
                aligned   = TRUE;
                (s2fault, descpaddr) = AArch64_S2Translate(fault, walkaddress,
                                                           s1aarch64, aligned,
                                                           descaccess);

                if s2fault.statuscode != Fault_None then
                    return (s2fault, ARBITRARY : AddressDescriptor,
                            ARBITRARY : TTWState, ARBITRARY : bits(N));
                end;
            else
                descpaddr = walkaddress;
            end;

            (fault, descriptor) = AArch64_MemSwapTableDesc{N}(fault, descriptor, new_descriptor,
                                                              walkparams.ee, descaccess,
                                                              descpaddr);
            if fault.statuscode != Fault_None then
                return (fault, ARBITRARY : AddressDescriptor,
                        ARBITRARY : TTWState, ARBITRARY : bits(N));
            end;
        end;
    until new_descriptor == descriptor looplimit Unbounded_DescriptorUpdate;
until desctype == DescriptorType_Leaf looplimit MAX_WALK_LEVELS;

let oa : FullAddress = Stage1OA(va, walkparams, walkstate);

if walkstate.contiguous == '1' && AArch64_S1ContiguousBitFaults(walkparams, walkstate) then
```

```
        fault.statuscode = Fault_Translation;
    elsif walkstate.level < FINAL_LEVEL && AArch64_nTFaults{N}(walkparams.d128, descriptor) then
        fault.statuscode = Fault_Translation;
    elsif AArch64_S1AMECFault{N}(walkparams, walkstate.baseaddress.paspace, regime, descriptor) then
        fault.statuscode = Fault_Translation;
    // Detect Address Size Fault by final output
    elsif AArch64_S1OAOutOfRange(oa.address, walkparams) then
        fault.statuscode = Fault_AddressSize;
    // Check descriptor AF bit
    elsif (descriptor[10] == '0' && walkparams.ha == '0' &&
           (!accdesc.acctype IN {AccessType_DC, AccessType_IC} ||
            ImpDefBool("Generate access flag fault on IC/DC operations"))) then
        fault.statuscode = Fault_AccessFlag;
    end;

    if fault.statuscode != Fault_None then
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    return (fault, walkaddress, walkstate, descriptor);
end;
```

Library pseudocode for aarch64/translation/vmsa_walk/AArch64_S2InitialTTWState

```
// AArch64_S2InitialTTWState()
// ===========================
// Set properties of first access to translation tables in stage 2

func AArch64_S2InitialTTWState(ss : SecurityState, walkparams : S2TTWParams) => TTWState
begin
    var walkstate : TTWState;
    var tablebase : FullAddress;
    var ttbr : bits(128);

    ttbr = ZeroExtend{128}(VTTBR_EL2());
    case ss of
        when SS_NonSecure => tablebase.paspace = PAS_NonSecure;
        when SS_Realm     => tablebase.paspace = PAS_Realm;
    end;
    tablebase.address = AArch64_S2TTBaseAddress{128}(walkparams, tablebase.paspace, ttbr);

    walkstate.baseaddress = tablebase;
    walkstate.level       = AArch64_S2StartLevel(walkparams);
    walkstate.istable     = TRUE;
    walkstate.memattrs    = WalkMemAttrs(walkparams.sh, walkparams.irgn, walkparams.orgn);

    return walkstate;
end;
```

Library pseudocode for aarch64/translation/vmsa_walk/AArch64_S2NextWalkStateLeaf

```
// AArch64_S2NextWalkStateLeaf()
// =============================
// Decode stage 2 page or block descriptor as output to this stage of translation

func AArch64_S2NextWalkStateLeaf{N}(currentstate : TTWState, ss : SecurityState,
                                    walkparams : S2TTWParams, ipa : AddressDescriptor,
                                    descriptor : bits(N)) => TTWState
begin
    var nextstate : TTWState;
    var baseaddress : FullAddress;

    if ss == SS_Secure then
        baseaddress.paspace = AArch64_SS2OutputPASpace(walkparams, ipa.paddress.paspace);
    elsif ss == SS_Realm then
        var ns : bit;
        ns = if walkparams.d128 == '1' then descriptor[127] else descriptor[55];
        baseaddress.paspace = if ns == '1' then PAS_NonSecure else PAS_Realm;
    else
        baseaddress.paspace = PAS_NonSecure;
    end;
    baseaddress.address   = AArch64_S2LeafBase{N}(descriptor, walkparams, currentstate.level);

    nextstate.istable     = FALSE;
    nextstate.level       = currentstate.level;
    nextstate.baseaddress = baseaddress;
    nextstate.permissions = AArch64_S2ApplyOutputPerms{N}(descriptor, walkparams);

    let s2_attr : bits(4) = descriptor[5:2];
    let s2_sh : bits(2)   = if walkparams.ds == '1' then walkparams.sh else descriptor[9:8];
    let s2_fnxs : bit     = descriptor[11];
    if walkparams.fwb == '1' then
        nextstate.memattrs = AArch64_S2ApplyFWBMemAttrs{N}(ipa.memattrs, walkparams, descriptor);
        if s2_attr[3:1] == '111' then
            nextstate.permissions.s2tag_na = '1';
        else
            nextstate.permissions.s2tag_na = '0';
        end;
    else
        let s2aarch64 : boolean = TRUE;
        nextstate.memattrs = S2DecodeMemAttrs(s2_attr, s2_sh, s2aarch64);
        // FnXS is used later to mask the XS value from stage 1
        nextstate.memattrs.xs = NOT s2_fnxs;
        if s2_attr == '0100' then
            nextstate.permissions.s2tag_na = '1';
        else
            nextstate.permissions.s2tag_na = '0';
        end;
    end;
    nextstate.contiguous = AArch64_ContiguousBit{N}(walkparams.tgx, walkparams.d128,
                                                    currentstate.level, descriptor);
    if walkparams.d128 == '1' then
        nextstate.s2assuredonly = descriptor[114];
    else
        nextstate.s2assuredonly = if walkparams.assuredonly == '1' then descriptor[58] else '0';
    end;

    return nextstate;
end;
```

Library pseudocode for aarch64/translation/vmsa_walk/AArch64_S2NextWalkStateTable

```
// AArch64_S2NextWalkStateTable()
// ==============================
// Decode stage 2 table descriptor to transition to the next level

func AArch64_S2NextWalkStateTable{N}(currentstate : TTWState, walkparams : S2TTWParams,
                                     descriptor : bits(N)) => TTWState
begin
    var nextstate : TTWState;
    var tablebase : FullAddress;
    let skl : bits(2) = if walkparams.d128 == '1' then descriptor[110:109] else '00';
    tablebase.address = AArch64_NextTableBase{N}(descriptor, walkparams.d128, skl, walkparams.ds,
                                                 walkparams.tgx);

    tablebase.paspace = currentstate.baseaddress.paspace;

    nextstate.istable     = TRUE;
    if walkparams.d128 == '1' then
        nextstate.level   = currentstate.level + UInt(skl) + 1;
    else
        nextstate.level   = currentstate.level + 1;
    end;
    nextstate.baseaddress = tablebase;
    nextstate.memattrs    = currentstate.memattrs;

    return nextstate;
end;
```

Library pseudocode for aarch64/translation/vmsa_walk/AArch64_S2Walk

```
// AArch64_S2Walk()
// ================
// Traverse stage 2 translation tables obtaining the final descriptor
// as well as the address leading to that descriptor

func AArch64_S2Walk{N : integer{64, 128}}(fault_in : FaultRecord,
                                          ipa : AddressDescriptor,
                                          walkparams : S2TTWParams,
                                          accdesc : AccessDescriptor) => (FaultRecord,
                                                                          AddressDescriptor,
                                                                          TTWState,
                                                                          bits(N))
begin

    var fault : FaultRecord = fault_in;

    var walkstate : TTWState;
    if accdesc.ss == SS_Secure then
        walkstate = AArch64_SS2InitialTTWState(walkparams, ipa.paddress.paspace);
    else
        walkstate = AArch64_S2InitialTTWState(accdesc.ss, walkparams);
    end;
    let startlevel : integer = walkstate.level;

    if startlevel > 3 then
        fault.statuscode = Fault_Translation;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    var descriptor : bits(N);
    let walkaccess : AccessDescriptor = CreateAccDescS2TTW(accdesc);
    var walkaddress : AddressDescriptor;
    var skl : bits(2) = '00';

    walkaddress.vaddress = ipa.vaddress;
    walkaddress.mecid = AArch64_S2TTWalkMECID(walkparams.emec, accdesc.ss);

    if !S2DCacheEnabled() then
        walkaddress.mematTrs = NormalNCMemAttr();
        walkaddress.mematTrs.xs = walkstate.mematTrs.xs;
    else
        walkaddress.mematTrs = walkstate.mematTrs;
    end;

    walkaddress.mematTrs.shareability = EffectiveShareability(walkaddress.mematTrs);

    var desctype : DescriptorType;

    // Initial lookup might index into concatenated tables
    var descaddress : FullAddress = AArch64_S2SLTTEntryAddress(walkparams, ipa.paddress.address,
                                                               walkstate.baseaddress);

    // Detect Address Size Fault by Descriptor Address
    if AArch64_S2OAOutOfRange(descaddress.address, walkparams) then
        fault.statuscode = Fault_AddressSize;
        fault.level      = 0;
        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState, ARBITRARY : bits(N));
    end;

    repeat
        fault.level = walkstate.level;
        walkaddress.paddress = descaddress;
        (fault, descriptor) = FetchDescriptor{N}(walkparams.ee, walkaddress, walkaccess, fault);

        if fault.statuscode != Fault_None then
            return (fault, ARBITRARY : AddressDescriptor,
                    ARBITRARY : TTWState, ARBITRARY : bits(N));
        end;

        var new_descriptor : bits(N);
        repeat
            new_descriptor = descriptor;
            desctype = AArch64_DecodeDescriptorType{N}(descriptor, walkparams.d128, walkparams.ds,
                                                       walkparams.tgx, walkstate.level);
            case desctype of
                when DescriptorType_Table =>
                    walkstate = AArch64_S2NextWalkStateTable{N}(walkstate, walkparams, descriptor);
                    skl = if walkparams.d128 == '1' then descriptor[110:109] else '00';
                    descaddress = AArch64_S2TTEntryAddress(walkstate.level, walkparams, skl,
                                                           ipa.paddress.address,
                                                           walkstate.baseaddress);

                    // Detect Address Size Fault by table descriptor
                    if AArch64_S2OAOutOfRange(descaddress.address, walkparams) then
                        fault.statuscode = Fault_AddressSize;
                        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                                ARBITRARY : bits(N));
                    end;
```

```
                    if walkparams.haft == '1' then
                        new_descriptor[10] = '1';
                    end;

                    if (walkparams.d128 == '1' && skl != '00' &&
                          AArch64_nTFaults{N}(walkparams.d128, descriptor)) then
                        fault.statuscode = Fault_Translation;
                        return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                                ARBITRARY : bits(N));
                    end;

                when DescriptorType_Leaf =>
                    walkstate = AArch64_S2NextWalkStateLeaf{N}(walkstate, accdesc.ss, walkparams,
                                                               ipa, descriptor);
                when DescriptorType_Invalid =>
                    fault.statuscode = Fault_Translation;
                    return (fault, ARBITRARY : AddressDescriptor,
                            ARBITRARY : TTWState, ARBITRARY : bits(N));

                otherwise =>
                    unreachable;
            end;

            if new_descriptor != descriptor then

                if (!IsWBShareable(walkaddress.memattrs) &&
                      ConstrainUnpredictableBool(Unpredictable_Unsupported_Atomic_HW_Update)) then
                    fault.statuscode = Fault_UnsupportedAtomicHWUpdate;
                    return (fault, ARBITRARY : AddressDescriptor, ARBITRARY : TTWState,
                            ARBITRARY : bits(N));
                end;

                let descaccess : AccessDescriptor = CreateAccDescTTEUpdate(accdesc);
                (fault, descriptor) = AArch64_MemSwapTableDesc{N}(fault, descriptor, new_descriptor,
                                                                  walkparams.ee, descaccess,
                                                                  walkaddress);
                if fault.statuscode != Fault_None then
                    return (fault, ARBITRARY : AddressDescriptor,
                            ARBITRARY : TTWState, ARBITRARY : bits(N));
                end;
            end;
        until new_descriptor == descriptor looplimit Unbounded_DescriptorUpdate;
    until desctype == DescriptorType_Leaf looplimit MAX_WALK_LEVELS;

    let oa : FullAddress = Stage2OA(ipa.paddress.address, walkparams, walkstate);

    if walkstate.contiguous == '1' && AArch64_S2ContiguousBitFaults(walkparams, walkstate) then
        fault.statuscode = Fault_Translation;
    elsif walkstate.level < FINAL_LEVEL && AArch64_nTFaults{N}(walkparams.d128, descriptor) then
        fault.statuscode = Fault_Translation;
    // Detect Address Size Fault by final output
    elsif AArch64_S2OAOutOfRange(oa.address, walkparams) then
        fault.statuscode = Fault_AddressSize;
    // Check descriptor AF bit
    elsif (descriptor[10] == '0' && walkparams.ha == '0' &&
            (!accdesc.acctype IN {AccessType_DC, AccessType_IC} ||
             ImpDefBool("Generate access flag fault on IC/DC operations"))) then
        fault.statuscode = Fault_AccessFlag;
    end;

    return (fault, walkaddress, walkstate, descriptor);
end;
```

Library pseudocode for aarch64/translation/vmsa_walk/AArch64_SS2InitialTTWState

```
// AArch64_SS2InitialTTWState()
// ============================
// Set properties of first access to translation tables in Secure stage 2

func AArch64_SS2InitialTTWState(walkparams : S2TTWParams, ipaspace : PASpace) => TTWState
begin
    var walkstate : TTWState;
    var tablebase : FullAddress;
    var ttbr : bits(128);

    if ipaspace == PAS_Secure then
        ttbr = ZeroExtend{128}(VSTTBR_EL2());
    else
        ttbr = ZeroExtend{128}(VTTBR_EL2());
    end;

    if ipaspace == PAS_Secure then
        if walkparams.sw == '0' then
            tablebase.paspace = PAS_Secure;
        else
            tablebase.paspace = PAS_NonSecure;
        end;
    else
        if walkparams.nsw == '0' then
            tablebase.paspace = PAS_Secure;
        else
            tablebase.paspace = PAS_NonSecure;
        end;
    end;
    tablebase.address = AArch64_S2TTBaseAddress{128}(walkparams, tablebase.paspace, ttbr);

    walkstate.baseaddress = tablebase;
    walkstate.level       = AArch64_S2StartLevel(walkparams);
    walkstate.istable     = TRUE;
    walkstate.memattrs    = WalkMemAttrs(walkparams.sh, walkparams.irgn, walkparams.orgn);

    return walkstate;
end;
```

Library pseudocode for aarch64/translation/vmsa_walk/AArch64_SS2OutputPASpace

```
// AArch64_SS2OutputPASpace()
// ==========================
// Assign PA Space to output of Secure stage 2 translation

func AArch64_SS2OutputPASpace(walkparams : S2TTWParams, ipaspace : PASpace) => PASpace
begin
    if ipaspace == PAS_Secure then
        if walkparams.[sw,sa] == '00' then
            return PAS_Secure;
        else
            return PAS_NonSecure;
        end;
    else
        if walkparams.[sw,sa,nsw,nsa] == '0000' then
            return PAS_Secure;
        else
            return PAS_NonSecure;
        end;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_GetS1TTWParams

```
// AArch64_GetS1TTWParams()
// ========================
// Returns stage 1 translation table walk parameters from respective controlling
// System registers.


readonly func AArch64_GetS1TTWParams(regime : Regime, el : bits(2),
                           ss : SecurityState, va : bits(64)) => S1TTWParams
begin
    var walkparams : S1TTWParams;

    let varange : VARange = AArch64_GetVARange(va);

    case regime of
        when Regime_EL3 =>  walkparams = AArch64_S1TTWParamsEL3();
        when Regime_EL2 =>  walkparams = AArch64_S1TTWParamsEL2(ss);
        when Regime_EL20 => walkparams = AArch64_S1TTWParamsEL20(el, ss, varange);
        when Regime_EL10 => walkparams = AArch64_S1TTWParamsEL10(el, varange);
    end;

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_GetS2TTWParams

```
// AArch64_GetS2TTWParams()
// ========================
// Gather walk parameters for stage 2 translation

func AArch64_GetS2TTWParams(ss : SecurityState, ipaspace : PASpace,
                           s1aarch64 : boolean) => S2TTWParams
begin
    var walkparams : S2TTWParams;

    if ss == SS_NonSecure then
        walkparams = AArch64_NSS2TTWParams(s1aarch64);
    elsif IsFeatureImplemented(FEAT_SEL2) && ss == SS_Secure then
        walkparams = AArch64_SS2TTWParams(ipaspace, s1aarch64);
    elsif ss == SS_Realm then
        walkparams = AArch64_RLS2TTWParams(s1aarch64);
    else
        unreachable;
    end;

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_GetVARange

```
// AArch64_GetVARange()
// ====================
// Determines if the VA that is to be translated lies in LOWER or UPPER address range.

readonly func AArch64_GetVARange(va : bits(64)) => VARange
begin
    if va[55] == '0' then
        return VARange_LOWER;
    else
        return VARange_UPPER;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_HaveS1TG

```
// AArch64_HaveS1TG()
// ==================
// Determine whether the given translation granule is supported for stage 1

readonly func AArch64_HaveS1TG(tgx : TGx) => boolean
begin
    case tgx of
        when TGx_4KB =>  return IsFeatureImplemented(FEAT_TGran4K);
        when TGx_16KB => return IsFeatureImplemented(FEAT_TGran16K);
        when TGx_64KB => return IsFeatureImplemented(FEAT_TGran64K);
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_HaveS2TG

```
// AArch64_HaveS2TG()
// ==================
// Determine whether the given translation granule is supported for stage 2

func AArch64_HaveS2TG(tgx : TGx) => boolean
begin
    assert HaveEL(EL2);

    if IsFeatureImplemented(FEAT_GTG) then
        case tgx of
            when TGx_4KB =>  return IsFeatureImplemented(FEAT_S2TGran4K);
            when TGx_16KB => return IsFeatureImplemented(FEAT_S2TGran16K);
            when TGx_64KB => return IsFeatureImplemented(FEAT_S2TGran64K);
        end;
    else
        return AArch64_HaveS1TG(tgx);
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_MaxTxSZ

```
// AArch64_MaxTxSZ()
// =================
// Retrieve the maximum value of TxSZ indicating minimum input address size for both
// stages of translation

readonly func AArch64_MaxTxSZ(tgx : TGx) => integer
begin
    if IsFeatureImplemented(FEAT_TTST) then
        case tgx of
            when TGx_4KB =>   return 48;
            when TGx_16KB =>  return 48;
            when TGx_64KB =>  return 47;
        end;
    end;

    return 39;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_NSS2TTWParams

```
// AArch64_NSS2TTWParams()
// =======================
// Gather walk parameters specific for Non-secure stage 2 translation

func AArch64_NSS2TTWParams(s1aarch64 : boolean) => S2TTWParams
begin
    var walkparams : S2TTWParams;

    if IsFeatureImplemented(FEAT_HCRMAC) then
        walkparams.vm = HCR_EL2().VM OR HCR_EL2().DC;
    else
        walkparams.vm = HCR_EL2().VM;
    end;
    walkparams.tgx  = AArch64_S2DecodeTG0(VTCR_EL2().TG0);
    walkparams.txsz = VTCR_EL2().T0SZ;
    walkparams.ps   = VTCR_EL2().PS;
    walkparams.irgn = VTCR_EL2().IRGN0;
    walkparams.orgn = VTCR_EL2().ORGN0;
    walkparams.sh   = VTCR_EL2().SH0;
    walkparams.ee   = SCTLR_EL2().EE;
    walkparams.d128 = if IsFeatureImplemented(FEAT_D128)  then VTCR_EL2().D128 else '0';
    if walkparams.d128 == '1' then
        walkparams.skl = VTTBR_EL2().SKL;
    else
        walkparams.sl0 = VTCR_EL2().SL0;
    end;

    if HCR_EL2().TGE == '0' then
        walkparams.ptw = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().PTW else '1';
    else
        walkparams.ptw = '0';
    end;
    if IsFeatureImplemented(FEAT_S2FWB) then
        walkparams.fwb = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().FWB else '1';
    else
        walkparams.fwb = '0';
    end;
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then VTCR_EL2().HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = VTCR_EL2().HD;
    else
        walkparams.hd = '0';
    end;
    if walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) then
        walkparams.ds = VTCR_EL2().DS;
    else
        walkparams.ds = '0';
    end;
    if walkparams.tgx == TGx_4KB && IsFeatureImplemented(FEAT_LPA2) then
        walkparams.sl2 = VTCR_EL2().SL2 AND VTCR_EL2().DS;
    else
        walkparams.sl2 = '0';
    end;
    walkparams.cmow = (if IsFeatureImplemented(FEAT_CMOW) && IsHCRXEL2Enabled() then HCRX_EL2().CMOW
                       else '0');
    if walkparams.d128 == '1' then
        walkparams.s2pie = '1';
    else
        walkparams.s2pie = if IsFeatureImplemented(FEAT_S2PIE) then VTCR_EL2().S2PIE else '0';
    end;
    if IsFeatureImplemented(FEAT_S2PIE) then
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
              ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            walkparams.s2pir = S2PIR_EL2() as S2PIRType;
        else
            walkparams.s2pir = Zeros{64} as S2PIRType;
        end;
    end;
    if IsFeatureImplemented(FEAT_THE) && walkparams.d128 != '1' then
        walkparams.assuredonly = VTCR_EL2().AssuredOnly;
    else
        walkparams.assuredonly = '0';
    end;
    walkparams.tl0   = if IsFeatureImplemented(FEAT_THE) then VTCR_EL2().TL0 else '0';
    walkparams.tl1   = if IsFeatureImplemented(FEAT_THE) then VTCR_EL2().TL1 else '0';
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' then
        walkparams.haft = VTCR_EL2().HAFT;
    else
        walkparams.haft = '0';
    end;
    if (IsFeatureImplemented(FEAT_HDBSS) && walkparams.hd == '1' &&
          (!HaveEL(EL3) || SCR_EL3().HDBSSEn == '1')) then
        walkparams.hdbss = VTCR_EL2().HDBSS;
    else
        walkparams.hdbss = '0';
    end;

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_OAMax

```
// AArch64_OAMax()
// ===============
// Returns the maximum number of bits capable of representing physical address
// based on the implemented features.

func AArch64_OAMax() => AddressSize
begin
    if IsFeatureImplemented(FEAT_D128) then return 56; end;
    if IsFeatureImplemented(FEAT_LPA2) then return 52; end;
    if IsFeatureImplemented(FEAT_LPA) then return 52; end;
    return 48;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_PAMax

```
// AArch64_PAMax()
// ===============
// Returns the IMPLEMENTATION DEFINED maximum number of bits capable of representing
// physical address for this PE

readonly func AArch64_PAMax() => AddressSize
begin
    return ImpDefInt("Maximum Physical Address Size") as AddressSize;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_RLS2TTWParams

```
// AArch64_RLS2TTWParams()
// =======================
// Gather walk parameters specific for Realm stage 2 translation

func AArch64_RLS2TTWParams(s1aarch64 : boolean) => S2TTWParams
begin
    // Realm stage 2 walk parameters are similar to Non-secure
    var walkparams : S2TTWParams = AArch64_NSS2TTWParams(s1aarch64);
    walkparams.emec = (if IsFeatureImplemented(FEAT_MEC) &&
                       IsSCTLR2EL2Enabled() then SCTLR2_EL2().EMEC else '0');
    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1DCacheEnabled

```
// AArch64_S1DCacheEnabled()
// =========================
// Determine cacheability of stage 1 data accesses

readonly func AArch64_S1DCacheEnabled(regime : Regime) => boolean
begin
    case regime of
        when Regime_EL3 =>  return SCTLR_EL3().C == '1';
        when Regime_EL2 =>  return SCTLR_EL2().C == '1';
        when Regime_EL20 => return SCTLR_EL2().C == '1';
        when Regime_EL10 => return SCTLR_EL1().C == '1';
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1DecodeTG0

```
// AArch64_S1DecodeTG0()
// =====================
// Decode stage 1 granule size configuration bits TG0

readonly func AArch64_S1DecodeTG0(tg0_in : bits(2)) => TGx
begin
    var tg0 : bits(2) = tg0_in;
    var tgx : TGx;

    if tg0 == '11' then
        tg0 = ImpDefBits{2}("TG0 encoded granule size");
    end;

    case tg0 of
        when '00' =>   tgx = TGx_4KB;
        when '01' =>   tgx = TGx_64KB;
        when '10' =>   tgx = TGx_16KB;
    end;

    if !AArch64_HaveS1TG(tgx) then
        case ImpDefBits{2}("TG0 encoded granule size") of
            when '00' =>   tgx = TGx_4KB;
            when '01' =>   tgx = TGx_64KB;
            when '10' =>   tgx = TGx_16KB;
        end;
    end;

    return tgx;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1DecodeTG1

```
// AArch64_S1DecodeTG1()
// =====================
// Decode stage 1 granule size configuration bits TG1

readonly func AArch64_S1DecodeTG1(tg1_in : bits(2)) => TGx
begin
    var tg1 : bits(2) = tg1_in;
    var tgx : TGx;

    if tg1 == '00' then
        tg1 = ImpDefBits{2}("TG1 encoded granule size");
    end;

    case tg1 of
        when '10' =>   tgx = TGx_4KB;
        when '11' =>   tgx = TGx_64KB;
        when '01' =>   tgx = TGx_16KB;
    end;

    if !AArch64_HaveS1TG(tgx) then
        case ImpDefBits{2}("TG1 encoded granule size") of
            when '10' =>   tgx = TGx_4KB;
            when '11' =>   tgx = TGx_64KB;
            when '01' =>   tgx = TGx_16KB;
        end;
    end;

    return tgx;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1E0POEnabled

```
// AArch64_S1E0POEnabled()
// =======================
// Determine whether stage 1 unprivileged permission overlay is enabled

readonly func AArch64_S1E0POEnabled(regime : Regime, nv1 : bit) => boolean
begin
    assert HasUnprivileged(regime);

    if !IsFeatureImplemented(FEAT_S1POE) then
        return FALSE;
    end;

    case regime of
        when Regime_EL20 => return IsTCR2EL2Enabled() && TCR2_EL2().E0POE == '1';
        when Regime_EL10 => return IsTCR2EL1Enabled() && nv1 == '0' && TCR2_EL1().E0POE == '1';
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1EPD

```
// AArch64_S1EPD()
// ===============
// Determine whether stage 1 translation table walk is allowed for the VA range

func AArch64_S1EPD(regime : Regime, va : bits(64)) => bit
begin
    assert HasUnprivileged(regime);
    let varange : VARange = AArch64_GetVARange(va);

    case regime of
        when Regime_EL20 => return (if varange == VARange_LOWER then TCR_EL2().EPD0
                                                                else TCR_EL2().EPD1);
        when Regime_EL10 => return (if varange == VARange_LOWER then TCR_EL1().EPD0
                                                                else TCR_EL1().EPD1);
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1Enabled

```
// AArch64_S1Enabled()
// ===================
// Determine if stage 1 is enabled for the access type for this translation regime

readonly func AArch64_S1Enabled(regime : Regime, acctype : AccessType) => boolean
begin
    if acctype == AccessType_TRBE && EffectiveTRBLIMITR_EL1_nVM()  == '1' then
        return FALSE;
    end;
    if acctype == AccessType_SPE && EffectivePMBLIMITR_EL1_nVM() == '1' then
        return FALSE;
    end;
    case regime of
        when Regime_EL3 =>  return SCTLR_EL3().M == '1';
        when Regime_EL2 =>  return SCTLR_EL2().M == '1';
        when Regime_EL20 => return SCTLR_EL2().M == '1';
        when Regime_EL10 => return ((!EL2Enabled() || HCR_EL2().[DC,TGE] == '00') &&
                                    SCTLR_EL1().M == '1');
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1ICacheEnabled

```
// AArch64_S1ICacheEnabled()
// =========================
// Determine cacheability of stage 1 instruction fetches

readonly func AArch64_S1ICacheEnabled(regime : Regime) => boolean
begin
    case regime of
        when Regime_EL3 =>  return SCTLR_EL3().I == '1';
        when Regime_EL2 =>  return SCTLR_EL2().I == '1';
        when Regime_EL20 => return SCTLR_EL2().I == '1';
        when Regime_EL10 => return SCTLR_EL1().I == '1';
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1MinTxSZ

```
// AArch64_S1MinTxSZ()
// ===================
// Retrieve the minimum value of TxSZ indicating maximum input address size for stage 1

readonly func AArch64_S1MinTxSZ(regime : Regime, walkparams : S1TTWParams) => integer
begin
    if IsFeatureImplemented(FEAT_LVA3) then
        if walkparams.d128 == '1' then
            if HasUnprivileged(regime) then
                return 9;
            else
                return 8;
            end;
        elsif walkparams.tgx == TGx_64KB || walkparams.ds == '1' then
            return 12;
        else
            return 16;
        end;
    end;
    if IsFeatureImplemented(FEAT_LVA) then
        if walkparams.tgx == TGx_64KB || walkparams.ds == '1' || walkparams.d128 == '1' then
            return 12;
        else
            return 16;
        end;
    end;

    return 16;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1POEnabled

```
// AArch64_S1POEnabled()
// =====================
// Determine whether stage 1 privileged permission overlay is enabled

readonly func AArch64_S1POEnabled(regime : Regime) => boolean
begin
    if !IsFeatureImplemented(FEAT_S1POE) then
        return FALSE;
    end;

    case regime of
        when Regime_EL3 =>  return TCR_EL3().POE == '1';
        when Regime_EL2 =>  return IsTCR2EL2Enabled() && TCR2_EL2().POE == '1';
        when Regime_EL20 => return IsTCR2EL2Enabled() && TCR2_EL2().POE == '1';
        when Regime_EL10 => return IsTCR2EL1Enabled() && TCR2_EL1().POE == '1';
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1POR

```
// AArch64_S1POR()
// ===============
// Identify stage 1 permissions overlay register for the acting translation regime

func AArch64_S1POR(regime : Regime) => S1PORType
begin
    if (HaveEL(EL3) && SCR_EL3().PIEn == '0' && regime != Regime_EL3 &&
          ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
        return Zeros{64} as S1PORType;
    end;

    case regime of
        when Regime_EL3 =>  return POR_EL3() as S1PORType;
        when Regime_EL2 =>  return POR_EL2() as S1PORType;
        when Regime_EL20 => return POR_EL2() as S1PORType;
        when Regime_EL10 => return POR_EL1() as S1PORType;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1TTBR

```
// AArch64_S1TTBR()
// ================
// Identify stage 1 table base register for the acting translation regime

func AArch64_S1TTBR(regime : Regime, va : bits(64)) => bits(128)
begin
    let varange : VARange = AArch64_GetVARange(va);

    case regime of
        when Regime_EL3 =>  return ZeroExtend{128}(TTBR0_EL3());
        when Regime_EL2 =>  return ZeroExtend{128}(TTBR0_EL2());
        when Regime_EL20 =>
            if varange == VARange_LOWER then
                return ZeroExtend{128}(TTBR0_EL2());
            else
                return ZeroExtend{128}(TTBR1_EL2());
            end;
        when Regime_EL10 =>
            if varange == VARange_LOWER then
                return ZeroExtend{128}(TTBR0_EL1());
            else
                return ZeroExtend{128}(TTBR1_EL1());
            end;
    end;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1TTWParamsEL10

```
// AArch64_S1TTWParamsEL10()
// =========================
// Gather stage 1 translation table walk parameters for EL1&0 regime
// (with EL2 enabled or disabled)

readonly func AArch64_S1TTWParamsEL10(el : bits(2), varange : VARange) => S1TTWParams
begin
    var walkparams : S1TTWParams;

    if IsFeatureImplemented(FEAT_D128) && IsTCR2EL1Enabled() then
        walkparams.d128 = TCR2_EL1().D128;
    else
        walkparams.d128 = '0';
    end;
    let nvs : bits(3) = EffectiveHCR_EL2_NVx();
    walkparams.nv1 = nvs[1] AND nvs[0];

    if IsFeatureImplemented(FEAT_AIE) then
        walkparams.mair2 = MAIR2_EL1() as MAIRType;
    end;
    walkparams.aie  = (if IsFeatureImplemented(FEAT_AIE) && IsTCR2EL1Enabled() then TCR2_EL1().AIE
                       else '0');
    if walkparams.d128 == '1' then
        walkparams.pie = '1';
    else
        walkparams.pie = (if IsFeatureImplemented(FEAT_S1PIE) &&
                          IsTCR2EL1Enabled() then TCR2_EL1().PIE else '0');
    end;
    if IsFeatureImplemented(FEAT_S1PIE) then
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
               ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            walkparams.pir = PIR_EL1() as S1PIRType;
            if walkparams.nv1 == '1' then
                walkparams.pire0 = Zeros{64} as S1PIRType;
            else
                walkparams.pire0 = PIRE0_EL1() as S1PIRType;
            end;
        else
            walkparams.pir   = Zeros{64} as S1PIRType;
            walkparams.pire0 = Zeros{64} as S1PIRType;
        end;
    end;

    if varange == VARange_LOWER then
        walkparams.tgx   = AArch64_S1DecodeTG0(TCR_EL1().TG0);
        walkparams.txsz  = TCR_EL1().T0SZ;
        walkparams.irgn  = TCR_EL1().IRGN0;
        walkparams.orgn  = TCR_EL1().ORGN0;
        walkparams.sh    = TCR_EL1().SH0;
        walkparams.tbi   = TCR_EL1().TBI0;

        walkparams.nfd   = if IsFeatureImplemented(FEAT_SVE)   then TCR_EL1().NFD0  else '0';
        walkparams.tbid  = if IsFeatureImplemented(FEAT_PAuth) then TCR_EL1().TBID0 else '0';
        walkparams.e0pd  = if IsFeatureImplemented(FEAT_E0PD)  then TCR_EL1().E0PD0 else '0';
        walkparams.hpd   = if IsFeatureImplemented(FEAT_HPDS)  then TCR_EL1().HPD0  else '0';
        if walkparams.hpd == '0' then
            if walkparams.aie == '1' then walkparams.hpd = '1'; end;
            if walkparams.pie == '1' then walkparams.hpd = '1'; end;
            if (AArch64_S1POEnabled(Regime_EL10) ||
                AArch64_S1E0POEnabled(Regime_EL10, walkparams.nv1)) then walkparams.hpd = '1';
            end;
        end;
        if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
            IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
            walkparams.mtx  = TCR_EL1().MTX0;
        else
            walkparams.mtx  = '0';
        end;
        walkparams.skl   = if walkparams.d128 == '1' then TTBR0_EL1().SKL else '00';
        walkparams.disch = if walkparams.d128 == '1' then TCR2_EL1().DisCH0 else '0';
        if IsFeatureImplemented(FEAT_ASID2) && IsTCR2EL1Enabled() then
            walkparams.fng = TCR2_EL1().FNG0;
        else
            walkparams.fng = '0';
        end;
        if IsFeatureImplemented(FEAT_THE) && IsTCR2EL1Enabled() then
            walkparams.fngna = TCR2_EL1().FNGNA0;
        else
            walkparams.fngna = '0';
        end;
    else
        walkparams.tgx   = AArch64_S1DecodeTG1(TCR_EL1().TG1);
        walkparams.txsz  = TCR_EL1().T1SZ;
        walkparams.irgn  = TCR_EL1().IRGN1;
        walkparams.orgn  = TCR_EL1().ORGN1;
        walkparams.sh    = TCR_EL1().SH1;
        walkparams.tbi   = TCR_EL1().TBI1;

        walkparams.nfd   = if IsFeatureImplemented(FEAT_SVE)   then TCR_EL1().NFD1  else '0';
```

```
        walkparams.tbid  = if IsFeatureImplemented(FEAT_PAuth) then TCR_EL1().TBID1 else '0';
        walkparams.e0pd  = if IsFeatureImplemented(FEAT_E0PD)  then TCR_EL1().E0PD1 else '0';
        walkparams.hpd   = if IsFeatureImplemented(FEAT_HPDS)  then TCR_EL1().HPD1  else '0';
        if walkparams.hpd == '0' then
            if walkparams.aie == '1' then walkparams.hpd = '1'; end;
            if walkparams.pie == '1' then walkparams.hpd = '1'; end;
            if (AArch64_S1POEnabled(Regime_EL10) ||
                  AArch64_S1E0POEnabled(Regime_EL10, walkparams.nv1)) then walkparams.hpd = '1';
            end;
        end;
        if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
              IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
            walkparams.mtx  = TCR_EL1().MTX1;
        else
            walkparams.mtx  = '0';
        end;
        walkparams.skl   = if walkparams.d128 == '1' then TTBR1_EL1().SKL else '00';
        walkparams.disch = if walkparams.d128 == '1' then TCR2_EL1().DisCH1 else '0';
        if IsFeatureImplemented(FEAT_ASID2) && IsTCR2EL1Enabled() then
            walkparams.fng = TCR2_EL1().FNG1;
        else
            walkparams.fng = '0';
        end;
        if IsFeatureImplemented(FEAT_THE) && IsTCR2EL1Enabled() then
            walkparams.fngna = TCR2_EL1().FNGNA1;
        else
            walkparams.fngna = '0';
        end;
    end;

    walkparams.mair = MAIR_EL1() as MAIRType;
    walkparams.wxn  = SCTLR_EL1().WXN;
    walkparams.ps   = TCR_EL1().IPS;
    walkparams.ee   = SCTLR_EL1().EE;
    if (HaveEL(EL3) && (!IsFeatureImplemented(FEAT_RME) || IsFeatureImplemented(FEAT_SEL2))) then
        walkparams.sif = SCR_EL3().SIF;
    else
        walkparams.sif = '0';
    end;

    if EL2Enabled() then
        walkparams.dc  = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().DC else '0';
        if IsFeatureImplemented(FEAT_MTE2) && IsFeatureImplemented(FEAT_HCRMAC) then
            walkparams.dct = HCR_EL2().DCT;
        else
            walkparams.dct = '0';
        end;
    end;

    if IsFeatureImplemented(FEAT_LSMAOC) then
        walkparams.ntlsmd = SCTLR_EL1().nTLSMD;
    else
        walkparams.ntlsmd = '1';
    end;

    walkparams.cmow = if IsFeatureImplemented(FEAT_CMOW)   then SCTLR_EL1().CMOW else '0';
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then TCR_EL1().HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = TCR_EL1().HD;
    else
        walkparams.hd = '0';
    end;
    if (walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) &&
          walkparams.d128 == '0') then
        walkparams.ds = TCR_EL1().DS;
    else
        walkparams.ds = '0';
    end;
    if IsFeatureImplemented(FEAT_PAN3) then
        walkparams.epan = if walkparams.pie == '0' then SCTLR_EL1().EPAN else '1';
    else
        walkparams.epan = '0';
    end;
    if IsFeatureImplemented(FEAT_THE) && walkparams.d128 == '0' && IsTCR2EL1Enabled() then
        walkparams.pnch = TCR2_EL1().PnCH;
    else
        walkparams.pnch = '0';
    end;
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' && IsTCR2EL1Enabled() then
        walkparams.haft = TCR2_EL1().HAFT;
    else
        walkparams.haft = '0';
    end;
    walkparams.emec = (if IsFeatureImplemented(FEAT_MEC) &&
                       IsSCTLR2EL2Enabled() then SCTLR2_EL2().EMEC else '0');

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1TTWParamsEL2

```
// AArch64_S1TTWParamsEL2()
// ========================
// Gather stage 1 translation table walk parameters for EL2 regime

readonly func AArch64_S1TTWParamsEL2(ss : SecurityState) => S1TTWParams
begin
    var walkparams : S1TTWParams;

    walkparams.tgx  = AArch64_S1DecodeTG0(TCR_EL2().TG0);
    walkparams.txsz = TCR_EL2().T0SZ;
    walkparams.ps   = TCR_EL2().PS;
    walkparams.irgn = TCR_EL2().IRGN0;
    walkparams.orgn = TCR_EL2().ORGN0;
    walkparams.sh   = TCR_EL2().SH0;
    walkparams.tbi  = TCR_EL2().TBI;
    walkparams.mair = MAIR_EL2() as MAIRType;

    walkparams.pie = (if IsFeatureImplemented(FEAT_S1PIE) && IsTCR2EL2Enabled() then TCR2_EL2().PIE
                      else '0');

    if IsFeatureImplemented(FEAT_S1PIE) then
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
               ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            walkparams.pir = PIR_EL2() as S1PIRType;
        else
            walkparams.pir = Zeros{64} as S1PIRType;
        end;
    end;
    if IsFeatureImplemented(FEAT_AIE) then
        walkparams.mair2 = MAIR2_EL2() as MAIRType;
    end;
    walkparams.aie  = (if IsFeatureImplemented(FEAT_AIE) && IsTCR2EL2Enabled() then TCR2_EL2().AIE
                       else '0');
    walkparams.wxn  = SCTLR_EL2().WXN;
    walkparams.ee   = SCTLR_EL2().EE;
    if (HaveEL(EL3) && (!IsFeatureImplemented(FEAT_RME) || IsFeatureImplemented(FEAT_SEL2))) then
        walkparams.sif = SCR_EL3().SIF;
    else
        walkparams.sif = '0';
    end;

    walkparams.tbid = if IsFeatureImplemented(FEAT_PAuth)  then TCR_EL2().TBID else '0';
    walkparams.hpd  = if IsFeatureImplemented(FEAT_HPDS)   then TCR_EL2().HPD  else '0';
    if walkparams.hpd == '0' then
        if walkparams.aie == '1' then walkparams.hpd = '1'; end;
        if walkparams.pie == '1' then walkparams.hpd = '1'; end;
        if AArch64_S1POEnabled(Regime_EL2) then walkparams.hpd = '1'; end;
    end;
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then TCR_EL2().E2H0.HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = TCR_EL2().E2H0.HD;
    else
        walkparams.hd = '0';
    end;
    if walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) then
        walkparams.ds = TCR_EL2().E2H0.DS;
    else
        walkparams.ds = '0';
    end;
    if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
          IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
        walkparams.mtx  = TCR_EL2().MTX;
    else
        walkparams.mtx  = '0';
    end;
    walkparams.pnch = (if IsFeatureImplemented(FEAT_THE) && IsTCR2EL2Enabled() then TCR2_EL2().PnCH
                       else '0');
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' && IsTCR2EL2Enabled() then
        walkparams.haft = TCR2_EL2().HAFT;
    else
        walkparams.haft = '0';
    end;
    walkparams.emec = (if IsFeatureImplemented(FEAT_MEC) &&
                       IsSCTLR2EL2Enabled() then SCTLR2_EL2().EMEC else '0');
    if IsFeatureImplemented(FEAT_MEC) && ss == SS_Realm && IsTCR2EL2Enabled() then
        walkparams.amec = TCR2_EL2().AMEC0;
    else
        walkparams.amec = '0';
    end;

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1TTWParamsEL20

```
// AArch64_S1TTWParamsEL20()
// =========================
// Gather stage 1 translation table walk parameters for EL2&0 regime

readonly func AArch64_S1TTWParamsEL20(el : bits(2), ss : SecurityState,
                                      varange : VARange) => S1TTWParams
begin
    var walkparams : S1TTWParams;

    if IsFeatureImplemented(FEAT_D128) && IsTCR2EL2Enabled() then
        walkparams.d128 = TCR2_EL2().D128;
    else
        walkparams.d128 = '0';
    end;

    if walkparams.d128 == '1' then
        walkparams.pie = '1';
    else
        walkparams.pie = (if IsFeatureImplemented(FEAT_S1PIE) &&
                          IsTCR2EL2Enabled() then TCR2_EL2().PIE else '0');
    end;
    if IsFeatureImplemented(FEAT_S1PIE) then
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
               ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            walkparams.pir   = PIR_EL2() as S1PIRType;
            walkparams.pire0 = PIRE0_EL2() as S1PIRType;
        else
            walkparams.pir   = Zeros{64} as S1PIRType;
            walkparams.pire0 = Zeros{64} as S1PIRType;
        end;
    end;

    if IsFeatureImplemented(FEAT_AIE) then
        walkparams.mair2 = MAIR2_EL2() as MAIRType;
    end;
    walkparams.aie  = (if IsFeatureImplemented(FEAT_AIE) && IsTCR2EL2Enabled() then TCR2_EL2().AIE
                       else '0');
    if varange == VARange_LOWER then
        walkparams.tgx   = AArch64_S1DecodeTG0(TCR_EL2().TG0);
        walkparams.txsz  = TCR_EL2().T0SZ;
        walkparams.irgn  = TCR_EL2().IRGN0;
        walkparams.orgn  = TCR_EL2().ORGN0;
        walkparams.sh    = TCR_EL2().SH0;
        walkparams.tbi   = TCR_EL2().TBI0;

        walkparams.nfd   = if IsFeatureImplemented(FEAT_SVE)   then TCR_EL2().NFD0  else '0';
        walkparams.tbid  = if IsFeatureImplemented(FEAT_PAuth) then TCR_EL2().TBID0 else '0';
        walkparams.e0pd  = if IsFeatureImplemented(FEAT_E0PD)  then TCR_EL2().E0PD0 else '0';
        walkparams.hpd   = if IsFeatureImplemented(FEAT_HPDS)  then TCR_EL2().HPD0  else '0';
        if walkparams.hpd == '0' then
            if walkparams.aie == '1' then walkparams.hpd = '1'; end;
            if walkparams.pie == '1' then walkparams.hpd = '1'; end;
            if AArch64_S1POEnabled(Regime_EL20) || AArch64_S1E0POEnabled(Regime_EL20, '0') then
                walkparams.hpd = '1';
            end;
        end;
        if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
              IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
            walkparams.mtx  = TCR_EL2().MTX0;
        else
            walkparams.mtx  = '0';
        end;
        walkparams.skl   = if walkparams.d128 == '1' then TTBR0_EL2().SKL else '00';
        walkparams.disch = if walkparams.d128 == '1' then TCR2_EL2().DisCH0 else '0';
        if IsFeatureImplemented(FEAT_ASID2) && IsTCR2EL2Enabled() then
            walkparams.fng = TCR2_EL2().FNG0;
        else
            walkparams.fng = '0';
        end;
    else
        walkparams.tgx   = AArch64_S1DecodeTG1(TCR_EL2().TG1);
        walkparams.txsz  = TCR_EL2().T1SZ;
        walkparams.irgn  = TCR_EL2().IRGN1;
        walkparams.orgn  = TCR_EL2().ORGN1;
        walkparams.sh    = TCR_EL2().SH1;
        walkparams.tbi   = TCR_EL2().TBI1;

        walkparams.nfd   = if IsFeatureImplemented(FEAT_SVE)   then TCR_EL2().NFD1  else '0';
        walkparams.tbid  = if IsFeatureImplemented(FEAT_PAuth) then TCR_EL2().TBID1 else '0';
        walkparams.e0pd  = if IsFeatureImplemented(FEAT_E0PD)  then TCR_EL2().E0PD1 else '0';
        walkparams.hpd   = if IsFeatureImplemented(FEAT_HPDS)  then TCR_EL2().HPD1  else '0';
        if walkparams.hpd == '0' then
            if walkparams.aie == '1' then walkparams.hpd = '1'; end;
            if walkparams.pie == '1' then walkparams.hpd = '1'; end;
            if AArch64_S1POEnabled(Regime_EL20) || AArch64_S1E0POEnabled(Regime_EL20, '0') then
                walkparams.hpd = '1';
            end;
        end;
        if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
```

```
              IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
            walkparams.mtx  = TCR_EL2().MTX1;
        else
            walkparams.mtx  = '0';
        end;
        walkparams.skl   = if walkparams.d128 == '1' then TTBR1_EL2().SKL else '00';
        walkparams.disch = if walkparams.d128 == '1' then TCR2_EL2().DisCH1 else '0';
        if IsFeatureImplemented(FEAT_ASID2) && IsTCR2EL2Enabled() then
            walkparams.fng = TCR2_EL2().FNG1;
        else
            walkparams.fng = '0';
        end;
    end;

    walkparams.mair = MAIR_EL2() as MAIRType;
    walkparams.wxn  = SCTLR_EL2().WXN;
    walkparams.ps   = TCR_EL2().IPS;
    walkparams.ee   = SCTLR_EL2().EE;
    if (HaveEL(EL3) && (!IsFeatureImplemented(FEAT_RME) || IsFeatureImplemented(FEAT_SEL2))) then
        walkparams.sif = SCR_EL3().SIF;
    else
        walkparams.sif = '0';
    end;

    if IsFeatureImplemented(FEAT_LSMAOC) then
        walkparams.ntlsmd = SCTLR_EL2().nTLSMD;
    else
        walkparams.ntlsmd = '1';
    end;

    walkparams.cmow = if IsFeatureImplemented(FEAT_CMOW)   then SCTLR_EL2().CMOW else '0';
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then TCR_EL2().E2H1.HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = TCR_EL2().E2H1.HD;
    else
        walkparams.hd = '0';
    end;
    if (walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) &&
          walkparams.d128 == '0') then
        walkparams.ds = TCR_EL2().E2H1.DS;
    else
        walkparams.ds = '0';
    end;
    if IsFeatureImplemented(FEAT_PAN3) then
        walkparams.epan = if walkparams.pie == '0' then SCTLR_EL2().EPAN else '1';
    else
        walkparams.epan = '0';
    end;
    if IsFeatureImplemented(FEAT_THE) && walkparams.d128 == '0' && IsTCR2EL2Enabled() then
        walkparams.pnch = TCR2_EL2().PnCH;
    else
        walkparams.pnch = '0';
    end;
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' && IsTCR2EL2Enabled() then
        walkparams.haft = TCR2_EL2().HAFT;
    else
        walkparams.haft = '0';
    end;
    walkparams.emec = (if IsFeatureImplemented(FEAT_MEC) && IsSCTLR2EL2Enabled()
                       then SCTLR2_EL2().EMEC else '0');
    if IsFeatureImplemented(FEAT_MEC) && ss == SS_Realm && IsTCR2EL2Enabled() then
        walkparams.amec = if varange == VARange_LOWER then TCR2_EL2().AMEC0 else TCR2_EL2().AMEC1;
    else
        walkparams.amec = '0';
    end;

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S1TTWParamsEL3

```
// AArch64_S1TTWParamsEL3()
// ========================
// Gather stage 1 translation table walk parameters for EL3 regime

readonly func AArch64_S1TTWParamsEL3() => S1TTWParams
begin
    var walkparams : S1TTWParams;

    walkparams.tgx  = AArch64_S1DecodeTG0(TCR_EL3().TG0);
    walkparams.txsz = TCR_EL3().T0SZ;
    walkparams.ps   = TCR_EL3().PS;
    walkparams.irgn = TCR_EL3().IRGN0;
    walkparams.orgn = TCR_EL3().ORGN0;
    walkparams.sh   = TCR_EL3().SH0;
    walkparams.tbi  = TCR_EL3().TBI;
    walkparams.mair = MAIR_EL3() as MAIRType;
    walkparams.d128  = if IsFeatureImplemented(FEAT_D128)  then TCR_EL3().D128 else '0';
    walkparams.skl   = if walkparams.d128 == '1' then TTBR0_EL3().SKL else '00';
    walkparams.disch = if walkparams.d128 == '1' then TCR_EL3().DisCH0 else '0';

    if walkparams.d128 == '1' then
        walkparams.pie = '1';
    else
        walkparams.pie = if IsFeatureImplemented(FEAT_S1PIE) then TCR_EL3().PIE else '0';
    end;
    if IsFeatureImplemented(FEAT_S1PIE) then
        walkparams.pir = PIR_EL3() as S1PIRType;
    end;

    if IsFeatureImplemented(FEAT_AIE) then
        walkparams.mair2 = MAIR2_EL3() as MAIRType;
    end;
    walkparams.aie  = if IsFeatureImplemented(FEAT_AIE) then TCR_EL3().AIE else '0';
    walkparams.wxn  = SCTLR_EL3().WXN;
    walkparams.ee   = SCTLR_EL3().EE;
    walkparams.sif = (if !IsFeatureImplemented(FEAT_RME) || IsFeatureImplemented(FEAT_SEL2)
                      then SCR_EL3().SIF else '0');

    walkparams.tbid = if IsFeatureImplemented(FEAT_PAuth)  then TCR_EL3().TBID else '0';
    walkparams.hpd  = if IsFeatureImplemented(FEAT_HPDS)   then TCR_EL3().HPD else '0';
    if walkparams.hpd == '0' then
        if walkparams.aie == '1' then walkparams.hpd = '1'; end;
        if walkparams.pie == '1' then walkparams.hpd = '1'; end;
        if AArch64_S1POEnabled(Regime_EL3) then walkparams.hpd = '1'; end;
    end;
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then TCR_EL3().HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = TCR_EL3().HD;
    else
        walkparams.hd = '0';
    end;
    if (walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) &&
          walkparams.d128 == '0') then
        walkparams.ds = TCR_EL3().DS;
    else
        walkparams.ds = '0';
    end;
    if (IsFeatureImplemented(FEAT_MTE_NO_ADDRESS_TAGS) ||
          IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS)) then
        walkparams.mtx  = TCR_EL3().MTX;
    else
        walkparams.mtx  = '0';
    end;
    if IsFeatureImplemented(FEAT_THE) && walkparams.d128 == '0' then
        walkparams.pnch = TCR_EL3().PnCH;
    else
        walkparams.pnch = '0';
    end;
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' then
        walkparams.haft = TCR_EL3().HAFT;
    else
        walkparams.haft = '0';
    end;
    walkparams.emec = if IsFeatureImplemented(FEAT_MEC) then SCTLR2_EL3().EMEC else '0';

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S2DecodeTG0

```
// AArch64_S2DecodeTG0()
// =====================
// Decode stage 2 granule size configuration bits TG0

func AArch64_S2DecodeTG0(tg0_in : bits(2)) => TGx
begin
    var tg0 : bits(2) = tg0_in;
    var tgx : TGx;

    if tg0 == '11' then
        tg0 = ImpDefBits{2}("TG0 encoded granule size");
    end;

    case tg0 of
        when '00' =>   tgx = TGx_4KB;
        when '01' =>   tgx = TGx_64KB;
        when '10' =>   tgx = TGx_16KB;
    end;

    if !AArch64_HaveS2TG(tgx) then
        case ImpDefBits{2}("TG0 encoded granule size") of
            when '00' =>   tgx = TGx_4KB;
            when '01' =>   tgx = TGx_64KB;
            when '10' =>   tgx = TGx_16KB;
        end;
    end;

    return tgx;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_S2MinTxSZ

```
// AArch64_S2MinTxSZ()
// ===================
// Retrieve the minimum value of TxSZ indicating maximum input address size for stage 2

func AArch64_S2MinTxSZ(walkparams : S2TTWParams, s1aarch64 : boolean) => integer
begin
    var ips : integer;

    if AArch64_PAMax() == 56 then
        if walkparams.d128 == '1' then
            ips = 56;
        elsif walkparams.tgx == TGx_64KB || walkparams.ds == '1' then
            ips = 52;
        else
            ips = 48;
        end;
    elsif AArch64_PAMax() == 52 then
        if walkparams.tgx == TGx_64KB || walkparams.ds == '1' then
            ips = 52;
        else
            ips = 48;
        end;
    else
        ips = AArch64_PAMax();
    end;

    var min_txsz : integer = 64 - ips;
    if !s1aarch64 then
        // EL1 is AArch32
        min_txsz = Min(min_txsz, 24);
    end;

    return min_txsz;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/AArch64_SS2TTWParams

```
// AArch64_SS2TTWParams()
// ======================
// Gather walk parameters specific for secure stage 2 translation

func AArch64_SS2TTWParams(ipaspace : PASpace, s1aarch64 : boolean) => S2TTWParams
begin
    var walkparams : S2TTWParams;

    walkparams.d128 = if IsFeatureImplemented(FEAT_D128) then VTCR_EL2().D128 else '0';
    if ipaspace == PAS_Secure then
        walkparams.tgx  = AArch64_S2DecodeTG0(VSTCR_EL2().TG0);
        walkparams.txsz = VSTCR_EL2().T0SZ;
        if walkparams.d128 == '1' then
            walkparams.skl = VSTTBR_EL2().SKL;
        else
            walkparams.sl0 = VSTCR_EL2().SL0;
        end;
        if walkparams.tgx == TGx_4KB && IsFeatureImplemented(FEAT_LPA2) then
            walkparams.sl2 = VSTCR_EL2().SL2 AND VTCR_EL2().DS;
        else
            walkparams.sl2 = '0';
        end;
    elsif ipaspace == PAS_NonSecure then
        walkparams.tgx  = AArch64_S2DecodeTG0(VTCR_EL2().TG0);
        walkparams.txsz = VTCR_EL2().T0SZ;
        if walkparams.d128 == '1' then
            walkparams.skl = VTTBR_EL2().SKL;
        else
            walkparams.sl0 = VTCR_EL2().SL0;
        end;
        if walkparams.tgx == TGx_4KB && IsFeatureImplemented(FEAT_LPA2) then
            walkparams.sl2 = VTCR_EL2().SL2 AND VTCR_EL2().DS;
        else
            walkparams.sl2 = '0';
        end;
    else
        unreachable;
    end;

    walkparams.sw   = VSTCR_EL2().SW;
    walkparams.nsw  = VTCR_EL2().NSW;
    walkparams.sa   = VSTCR_EL2().SA;
    walkparams.nsa  = VTCR_EL2().NSA;
    if IsFeatureImplemented(FEAT_HCRMAC) then
        walkparams.vm  = HCR_EL2().VM OR HCR_EL2().DC;
    else
        walkparams.vm  = HCR_EL2().VM;
    end;
    walkparams.ps   = VTCR_EL2().PS;
    walkparams.irgn = VTCR_EL2().IRGN0;
    walkparams.orgn = VTCR_EL2().ORGN0;
    walkparams.sh   = VTCR_EL2().SH0;
    walkparams.ee   = SCTLR_EL2().EE;

    if HCR_EL2().TGE == '0' then
        walkparams.ptw = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().PTW else '1';
    else
        walkparams.ptw = '0';
    end;
    if IsFeatureImplemented(FEAT_S2FWB) then
        walkparams.fwb = if IsFeatureImplemented(FEAT_HCRMAC) then HCR_EL2().FWB else '1';
    else
        walkparams.fwb = '0';
    end;
    walkparams.ha = if IsFeatureImplemented(FEAT_HAF) then VTCR_EL2().HA else '0';
    if IsFeatureImplemented(FEAT_HAFDBS) && walkparams.ha == '1' then
        walkparams.hd = VTCR_EL2().HD;
    else
        walkparams.hd = '0';
    end;
    if walkparams.tgx IN {TGx_4KB, TGx_16KB} && IsFeatureImplemented(FEAT_LPA2) then
        walkparams.ds = VTCR_EL2().DS;
    else
        walkparams.ds = '0';
    end;
    walkparams.cmow = (if IsFeatureImplemented(FEAT_CMOW) && IsHCRXEL2Enabled() then HCRX_EL2().CMOW
                       else '0');
    if walkparams.d128 == '1' then
        walkparams.s2pie = '1';
    else
        walkparams.s2pie = if IsFeatureImplemented(FEAT_S2PIE) then VTCR_EL2().S2PIE else '0';
    end;
    if IsFeatureImplemented(FEAT_S2PIE) then
        if !(HaveEL(EL3) && SCR_EL3().PIEn == '0' &&
               ImpDefBool("SCR_EL3.PIEn forces PIR_ELx or POR_ELx to zero")) then
            walkparams.s2pir = S2PIR_EL2() as S2PIRType;
        else
            walkparams.s2pir = Zeros{64} as S2PIRType;
        end;
```

```
    end;
    if IsFeatureImplemented(FEAT_THE) && walkparams.d128 != '1' then
        walkparams.assuredonly = VTCR_EL2().AssuredOnly;
    else
        walkparams.assuredonly = '0';
    end;
    walkparams.tl0   = if IsFeatureImplemented(FEAT_THE) then VTCR_EL2().TL0 else '0';
    walkparams.tl1   = if IsFeatureImplemented(FEAT_THE) then VTCR_EL2().TL1 else '0';
    if IsFeatureImplemented(FEAT_HAFT) && walkparams.ha == '1' then
        walkparams.haft = VTCR_EL2().HAFT;
    else
        walkparams.haft = '0';
    end;
    walkparams.emec = '0';
    if (IsFeatureImplemented(FEAT_HDBSS) && walkparams.hd == '1' &&
          (!HaveEL(EL3) || SCR_EL3().HDBSSEn == '1')) then
        walkparams.hdbss = VTCR_EL2().HDBSS;
    else
        walkparams.hdbss = '0';
    end;

    return walkparams;
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/S2DCacheEnabled

```
// S2DCacheEnabled()
// =================
// Returns TRUE if Stage 2 Data access cacheability is enabled

readonly func S2DCacheEnabled() => boolean
begin
    return !IsFeatureImplemented(FEAT_HCRMAC) || HCR_EL2().CD == '0';
end;
```

Library pseudocode for aarch64/translation/vmsa_walkparams/S2ICacheEnabled

```
// S2ICacheEnabled()
// =================
// Returns TRUE if Stage 2 Instruction access cacheability is enabled

readonly func S2ICacheEnabled() => boolean
begin
    return !IsFeatureImplemented(FEAT_HCRMAC) || HCR_EL2().ID == '0';
end;
```

Library pseudocode for shared/debug/ClearStickyErrors/ClearStickyErrors

```
// ClearStickyErrors()
// ===================

func ClearStickyErrors()
begin
    EDSCR().TXU = '0';            // Clear TX underrun flag
    EDSCR().RXO = '0';            // Clear RX overrun flag

    if Halted() then            // in Debug state
        EDSCR().ITO = '0';        // Clear ITR overrun flag
    end;

    // If halted and the ITR is not empty then it is UNPREDICTABLE whether the EDSCR.ERR is cleared.
    // The UNPREDICTABLE behavior also affects the instructions in flight, but this is not described
    // in the pseudocode.
    if (Halted() && EDSCR().ITE == '0' &&
          ConstrainUnpredictableBool(Unpredictable_CLEARERRITEZERO)) then
        return;
    end;
    EDSCR().ERR = '0';            // Clear cumulative error flag

    return;
end;
```

Library pseudocode for shared/debug/DebugTarget/DebugTarget

```
// DebugTarget()
// =============
// Returns the debug exception target Exception level

func DebugTarget() => bits(2)
begin
    let ss : SecurityState = CurrentSecurityState();
    return DebugTargetFrom(ss);
end;
```

Library pseudocode for shared/debug/DebugTarget/DebugTargetFrom

```
// DebugTargetFrom()
// =================

func DebugTargetFrom(from_state : SecurityState) => bits(2)
begin
    var route_to_el2 : boolean;
    if HaveEL(EL2) && (from_state != SS_Secure ||
        (IsFeatureImplemented(FEAT_SEL2) && (!HaveEL(EL3) || SCR_EL3().EEL2 == '1'))) then
        if ELUsingAArch32(EL2) then
            route_to_el2 = (HDCR().TDE == '1' || HCR().TGE == '1');
        else
            route_to_el2 = (MDCR_EL2().TDE == '1' || HCR_EL2().TGE == '1');
        end;
    else
        route_to_el2 = FALSE;
    end;

    var target : bits(2);
    if route_to_el2 then
        target = EL2;
    elsif HaveEL(EL3) && !HaveAArch64() && from_state == SS_Secure then
        target = EL3;
    else
        target = EL1;
    end;

    return target;
end;
```

Library pseudocode for shared/debug/DoubleLockStatus/DoubleLockStatus

```
// DoubleLockStatus()
// ==================
// Returns the state of the OS Double Lock.
//    FALSE if OSDLR_EL1.DLK == 0 or DBGPRCR_EL1.CORENPDRQ == 1 or the PE is in Debug state.
//    TRUE if OSDLR_EL1.DLK == 1 and DBGPRCR_EL1.CORENPDRQ == 0 and the PE is in Non-debug state.

func DoubleLockStatus() => boolean
begin
    if !IsFeatureImplemented(FEAT_DoubleLock) then
        return FALSE;
    end;
    if ELUsingAArch32(EL1) then
        return DBGOSDLR().DLK == '1' && DBGPRCR().CORENPDRQ == '0' && !Halted();
    else
        return OSDLR_EL1().DLK == '1' && DBGPRCR_EL1().CORENPDRQ == '0' && !Halted();
    end;
end;
```

Library pseudocode for shared/debug/OSLockStatus/OSLockStatus

```
// OSLockStatus()
// ==============
// Returns the state of the OS Lock.

readonly func OSLockStatus() => boolean
begin
    return (if ELUsingAArch32(EL1) then DBGOSLSR().OSLK else OSLSR_EL1().OSLK) == '1';
end;
```

Library pseudocode for shared/debug/SoftwareLockStatus/Component

```
// Component
// =========
// Component Types.

type Component of enumeration {
        Component_AMU,
        Component_ETE,
        Component_TRBE,
        Component_RAS,
        Component_GIC,
        Component_PMU,
        Component_Debug,
        Component_CTI
};
```

Library pseudocode for shared/debug/SoftwareLockStatus/GetAccessComponent

```
// GetAccessComponent()
// ====================
// Determines the component by decoding the physical address in the address descriptor.

impdef func GetAccessComponent(addrdesc : AddressDescriptor) => Component
begin
    return Component_Debug;
end;
```

Library pseudocode for shared/debug/SoftwareLockStatus/HaveSoftwareLock

```
// HaveSoftwareLock()
// ==================
// Returns TRUE if Software Lock is implemented.

func HaveSoftwareLock(component : Component) => boolean
begin
    if IsFeatureImplemented(FEAT_Debugv8p4) then
        return FALSE;
    end;
    if IsFeatureImplemented(FEAT_DoPD) && component != Component_CTI then
        return FALSE;
    end;
    case component of
        when Component_ETE =>
            return ImpDefBool("ETE has Software Lock");
        when Component_Debug =>
            return ImpDefBool("Debug has Software Lock");
        when Component_PMU =>
            return ImpDefBool("PMU has Software Lock");
        when Component_CTI =>
            return ImpDefBool("CTI has Software Lock");
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/debug/SoftwareLockStatus/SoftwareLockStatus

```
// SoftwareLockStatus()
// ====================
// Returns the state of the Software Lock.

func SoftwareLockStatus(addrdesc : AddressDescriptor) => boolean
begin
    let component : Component = GetAccessComponent(addrdesc);
    if !HaveSoftwareLock(component) then
        return FALSE;
    end;
    case component of
        when Component_ETE   => return TRCLSR().SLK == '1';
        when Component_Debug => return EDLSR().SLK == '1';
        when Component_PMU   => return PMLSR().SLK == '1';
        when Component_CTI   => return CTILSR().SLK == '1';
        otherwise            => return FALSE;
    end;
end;
```

Library pseudocode for shared/debug/amu/G1ActivityMonitorImplementedMask

```
// G1ActivityMonitorImplementedMask()
// ==================================
// Return the implementation-defined G1 activity monitor mask.

func G1ActivityMonitorImplementedMask() => bits(16)
begin
    assert NUM_AMU_CG1_MONITORS <= 16;
    var mask : bits(16) = Zeros{16};

    for i = 0 to (NUM_AMU_CG1_MONITORS-1) do
        mask[i] = if IsG1ActivityMonitorImplemented(i) then '1' else '0';
    end;
    return mask;
end;
```

Library pseudocode for shared/debug/amu/IsG1ActivityMonitorImplemented

```
// IsG1ActivityMonitorImplemented()
// ================================
// Returns TRUE if a G1 activity monitor is implemented for the counter
// and FALSE otherwise.

impdef func IsG1ActivityMonitorImplemented(i : integer) => boolean
begin
    case i of
        when 0 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 0"));
        when 1 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 1"));
        when 2 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 2"));
        when 3 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 3"));
        when 4 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 4"));
        when 5 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 5"));
        when 6 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 6"));
        when 7 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 7"));
        when 8 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 8"));
        when 9 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 9"));
        when 10 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 10"));
        when 11 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 11"));
        when 12 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 12"));
        when 13 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 13"));
        when 14 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 14"));
        when 15 =>
            return (ImpDefBool(
                    "G1 activity monitor is implemented for counter 15"));
        otherwise =>
            return FALSE;
    end;
end;
```

Library pseudocode for shared/debug/amu/IsG1ActivityMonitorOffsetImplemented

```
// IsG1ActivityMonitorOffsetImplemented()
// ======================================
// Returns TRUE if a G1 activity monitor offset is implemented for the counter,
// and FALSE otherwise.

impdef func IsG1ActivityMonitorOffsetImplemented(i : integer) => boolean
begin
    case i of
        when 0 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 0"));
        when 1 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 1"));
        when 2 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 2"));
        when 3 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 3"));
        when 4 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 4"));
        when 5 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 5"));
        when 6 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 6"));
        when 7 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 7"));
        when 8 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 8"));
        when 9 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 9"));
        when 10 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 10"));
        when 11 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 11"));
        when 12 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 12"));
        when 13 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 13"));
        when 14 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 14"));
        when 15 =>
            return (ImpDefBool(
                    "G1 activity monitor offset is implemented for counter 15"));
        otherwise =>
            return FALSE;
    end;
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENCLR0

```
// Write_AMCNTENCLR0()
// ===================
// System register write to AMCNTENCLR0.

func Write_AMCNTENCLR0(value_in : bits(32))
begin
    AMCNTENCLR0().P = AMCNTENCLR0().P AND NOT value_in[0 +: NUM_AMU_CG0_MONITORS];
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENCLR0_EL0

```
// Write_AMCNTENCLR0_EL0()
// =======================
// System register write to AMCNTENCLR0_EL0.

func Write_AMCNTENCLR0_EL0(value_in : bits(64))
begin
    AMCNTENCLR0_EL0().P = AMCNTENCLR0_EL0().P AND NOT value_in[0 +: NUM_AMU_CG0_MONITORS];
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENCLR1

```
// Write_AMCNTENCLR1()
// ===================
// System register write to AMCNTENCLR1.

func Write_AMCNTENCLR1(value_in : bits(32))
begin
    assert NUM_AMU_CG1_MONITORS <= 16;
    let implemented : bits(16) = G1ActivityMonitorImplementedMask();

    AMCNTENCLR1().P = (AMCNTENCLR1().P AND NOT
                       (ZeroExtend{16}(value_in[0 +: NUM_AMU_CG1_MONITORS]) AND implemented));
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENCLR1_EL0

```
// Write_AMCNTENCLR1_EL0()
// =======================
// System register write to AMCNTENCLR1_EL0.

func Write_AMCNTENCLR1_EL0(value_in : bits(64))
begin
    assert NUM_AMU_CG1_MONITORS <= 16;
    let implemented : bits(16) = G1ActivityMonitorImplementedMask();

    AMCNTENCLR1_EL0().P = (AMCNTENCLR1_EL0().P AND NOT
                           (ZeroExtend{16}(value_in[0 +: NUM_AMU_CG1_MONITORS]) AND implemented));
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENSET0

```
// Write_AMCNTENSET0()
// ===================
// System register write to AMCNTENSET0.

func Write_AMCNTENSET0(value_in : bits(32))
begin
    AMCNTENSET0().P = AMCNTENSET0().P OR value_in[0 +: NUM_AMU_CG0_MONITORS];
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENSET0_EL0

```
// Write_AMCNTENSET0_EL0()
// =======================
// System register write to AMCNTENSET0_EL0.

func Write_AMCNTENSET0_EL0(value_in : bits(64))
begin
    AMCNTENSET0_EL0().P = AMCNTENSET0_EL0().P OR value_in[0 +: NUM_AMU_CG0_MONITORS];
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENSET1

```
// Write_AMCNTENSET1()
// ===================
// System register write to AMCNTENSET1.

func Write_AMCNTENSET1(value_in : bits(32))
begin
    assert NUM_AMU_CG1_MONITORS <= 16;
    let implemented : bits(16) = G1ActivityMonitorImplementedMask();

    AMCNTENSET1().P = (AMCNTENSET1().P OR
                       (ZeroExtend{16}(value_in[0 +: NUM_AMU_CG1_MONITORS]) AND implemented));
end;
```

Library pseudocode for shared/debug/amu/Write_AMCNTENSET1_EL0

```
// Write_AMCNTENSET1_EL0()
// =======================
// System register write to AMCNTENSET1_EL0.

func Write_AMCNTENSET1_EL0(value_in : bits(64))
begin
    assert NUM_AMU_CG1_MONITORS <= 16;
    let implemented : bits(16) = G1ActivityMonitorImplementedMask();

    AMCNTENSET1_EL0().P = (AMCNTENSET1_EL0().P OR
                           (ZeroExtend{16}(value_in[0 +: NUM_AMU_CG1_MONITORS]) AND implemented));
end;
```

Library pseudocode for shared/debug/authentication/AllowExternalDebugAccess

```
// AllowExternalDebugAccess()
// ==========================
// Returns TRUE if an external debug interface access to the External DBGBVR<n>_EL1,
// DBGBCR<n>_EL1, DBGWVR<n>_EL1, DBGWCR<n>_EL1 registers, and, from Armv8.2, the
// OSLAR_EL1 register is allowed for the access. Returns FALSE otherwise.

func AllowExternalDebugAccess(addrdesc : AddressDescriptor) => boolean
begin
    // The access may also be subject to OS Lock, power-down, etc.
    if IsFeatureImplemented(FEAT_RME) then
        case MDCR_EL3().[EDADE,EDAD] of
            when '00' => return TRUE;
            when '01' => return addrdesc.paddress.paspace IN {PAS_Root, PAS_Secure};
            when '10' => return addrdesc.paddress.paspace IN {PAS_Root, PAS_Realm};
            when '11' => return addrdesc.paddress.paspace == PAS_Root;
        end;
    end;

    if IsFeatureImplemented(FEAT_Debugv8p4) then
        if addrdesc.paddress.paspace == PAS_Secure then return TRUE; end;
    else
        if !ExternalInvasiveDebugEnabled() then return FALSE; end;
        if ExternalSecureInvasiveDebugEnabled() then return TRUE; end;
    end;

    if HaveEL(EL3) then
        let EDAD_bit : bit = if ELUsingAArch32(EL3) then SDCR().EDAD else MDCR_EL3().EDAD;
        return EDAD_bit == '0';
    else
        return NonSecureOnlyImplementation();
    end;
end;
```

Library pseudocode for shared/debug/authentication/AllowExternalPMSSAccess

```
// AllowExternalPMSSAccess()
// =========================
// Returns TRUE if an external debug interface access to the PMU Snapshot
// registers is allowed for the given Security state, FALSE otherwise.

func AllowExternalPMSSAccess(addrdesc : AddressDescriptor) => boolean
begin
    assert IsFeatureImplemented(FEAT_PMUv3_SS) && HaveAArch64();
    // FEAT_Debugv8p4 is always implemented when FEAT_PMUv3_SS is implemented.
    assert IsFeatureImplemented(FEAT_Debugv8p4);

    // The access may also be subject to the OS Double Lock, power-down, etc.
    var epmssad : bits(2) = if HaveEL(EL3) then MDCR_EL3().EPMSSAD else '11';

    // Check for reserved values
    if !IsFeatureImplemented(FEAT_RME) && epmssad IN {'01','10'} then
        (-, epmssad) = ConstrainUnpredictableBits{2}(Unpredictable_RESEPMSSAD);
        // The value returned by ConstrainUnpredictableBits() must be a
        // non-reserved value
        assert epmssad IN {'00','11'};
    end;

    case epmssad of
        when '00' =>
            if IsFeatureImplemented(FEAT_RME) then
                return addrdesc.paddress.paspace == PAS_Root;
            else
                return addrdesc.paddress.paspace == PAS_Secure;
            end;
        when '01' =>
            assert IsFeatureImplemented(FEAT_RME);
            return addrdesc.paddress.paspace IN {PAS_Root, PAS_Realm};
        when '10' =>
            assert IsFeatureImplemented(FEAT_RME);
            return addrdesc.paddress.paspace IN {PAS_Root, PAS_Secure};
        when '11' =>
            return TRUE;
    end;
end;
```

Library pseudocode for shared/debug/authentication/AllowExternalPMUAccess

```
// AllowExternalPMUAccess()
// ========================
// Returns TRUE if an external debug interface access to the PMU registers is
// allowed for the given Security state, FALSE otherwise.

func AllowExternalPMUAccess(addrdesc : AddressDescriptor) => boolean
begin
    // The access may also be subject to OS Lock, power-down, etc.
    if IsFeatureImplemented(FEAT_RME) then
        case MDCR_EL3().[EPMADE,EPMAD] of
            when '00' => return TRUE;
            when '01' => return addrdesc.paddress.paspace IN {PAS_Root, PAS_Secure};
            when '10' => return addrdesc.paddress.paspace IN {PAS_Root, PAS_Realm};
            when '11' => return addrdesc.paddress.paspace == PAS_Root;
        end;
    end;

    if IsFeatureImplemented(FEAT_Debugv8p4) then
        if addrdesc.paddress.paspace == PAS_Secure then return TRUE; end;
    else
        if !ExternalInvasiveDebugEnabled() then return FALSE; end;
        if ExternalSecureInvasiveDebugEnabled() then return TRUE; end;
    end;

    if HaveEL(EL3) then
        let EPMAD_bit : bit = if ELUsingAArch32(EL3) then SDCR().EPMAD else MDCR_EL3().EPMAD;
        return EPMAD_bit == '0';
    else
        return NonSecureOnlyImplementation();
    end;
end;
```

Library pseudocode for shared/debug/authentication/AllowExternalTraceAccess

```
// AllowExternalTraceAccess()
// ==========================
// Returns TRUE if an external Trace access to the Trace registers is allowed for the
// given PAS, FALSE otherwise.

func AllowExternalTraceAccess(addrdesc : AddressDescriptor) => boolean
begin
    // The access may also be subject to OS lock, power-down, etc.
    if !IsFeatureImplemented(FEAT_TRBE) then return TRUE; end;
    assert IsFeatureImplemented(FEAT_Debugv8p4);
    if IsFeatureImplemented(FEAT_RME) then
        case MDCR_EL3().[ETADE,ETAD] of
            when '00' => return TRUE;
            when '01' => return addrdesc.paddress.paspace IN {PAS_Root, PAS_Secure};
            when '10' => return addrdesc.paddress.paspace IN {PAS_Root, PAS_Realm};
            when '11' => return addrdesc.paddress.paspace == PAS_Root;
        end;
    end;

    if addrdesc.paddress.paspace == PAS_Secure then return TRUE; end;
    if HaveEL(EL3) then
        // External Trace access is not supported for EL3 using AArch32
        assert !ELUsingAArch32(EL3);
        return MDCR_EL3().ETAD == '0';
    else
        return NonSecureOnlyImplementation();
    end;
end;
```

Library pseudocode for shared/debug/authentication/AllowRASMemoryMappedAccess

```
// AllowRASMemoryMappedAccess()
// ============================
// Returns TRUE if read and/or write access to RAS memory-mapped registers is
// permitted for the given access.

func AllowRASMemoryMappedAccess(addrdesc : AddressDescriptor, iswrite : boolean) => boolean
begin
    // An access from the most secure implemented PAS is always permitted.
    if IsMostSecureAccess(addrdesc) then
        return TRUE;
    end;

    if IsFeatureImplemented(FEAT_RASSA_ACR) && IsFeatureImplemented(FEAT_RASSAv2) then
        var ra : bits(2);
        let implements_rl_and_s : boolean = (IsFeatureImplemented(FEAT_RME) &&
                                             ImpDefBool(
              "the error record group allows configuration of Secure and Realm register accesses"));
        if IsAccessRealm(addrdesc) then
            ra = if implements_rl_and_s then ERRACR().RLRA else ERRACR().NSRA;
        elsif IsAccessSecure(addrdesc) then
            ra = if implements_rl_and_s then ERRACR().SRA else ERRACR().NSRA;
        elsif IsAccessNonSecure(addrdesc) then
            ra = ERRACR().NSRA;
        else
            unreachable;
        end;
        // '10' is reserved
        if ra == '10' then (-, ra) = ConstrainUnpredictableBits{2}(Unpredictable_RES_RA); end;
        // Interpret configured permissions to a boolean for the requested access
        case ra of
            when '00' => return FALSE;          // RAZ/WI => no read or write
            when '01' => return !iswrite;       // RO => reads allowed, writes not
            when '11' => return TRUE;           // RW => both allowed
            otherwise => unreachable;
        end;
    end;
    return ImpDefBool("error records are accessible to all PAS");
end;
```

Library pseudocode for shared/debug/authentication/Debug

```
// Debug authentication signals
// ============================

var DBGEN : Signal;
var NIDEN : Signal;
var SPIDEN : Signal;
var SPNIDEN : Signal;
var RLPIDEN : Signal;
var RTPIDEN : Signal;
```

Library pseudocode for shared/debug/authentication/ExternalInvasiveDebugEnabled

```
// ExternalInvasiveDebugEnabled()
// ==============================
// The definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the DBGEN signal.

readonly func ExternalInvasiveDebugEnabled() => boolean
begin
    return DBGEN == HIGH;
end;
```

Library pseudocode for shared/debug/authentication/ExternalNoninvasiveDebugAllowed

```
// ExternalNoninvasiveDebugAllowed()
// =================================
// Returns TRUE if Trace and PC Sample-based Profiling are allowed

func ExternalNoninvasiveDebugAllowed() => boolean
begin
    return ExternalNoninvasiveDebugAllowed(PSTATE.EL);
end;

// ExternalNoninvasiveDebugAllowed()
// =================================

func ExternalNoninvasiveDebugAllowed(el : bits(2)) => boolean
begin
    if !ExternalNoninvasiveDebugEnabled() then return FALSE; end;
    let ss : SecurityState = SecurityStateAtEL(el);

    if ((ELUsingAArch32(EL3) || ELUsingAArch32(EL1)) && el == EL0 &&
        ss == SS_Secure && SDER().SUNIDEN == '1') then
        return TRUE;
    end;

    case ss of
        when SS_NonSecure => return TRUE;
        when SS_Secure =>    return ExternalSecureNoninvasiveDebugEnabled();
        when SS_Realm =>     return ExternalRealmNoninvasiveDebugEnabled();
        when SS_Root =>      return ExternalRootNoninvasiveDebugEnabled();
    end;
end;
```

Library pseudocode for shared/debug/authentication/ExternalNoninvasiveDebugEnabled

```
// ExternalNoninvasiveDebugEnabled()
// =================================
// This function returns TRUE if the FEAT_Debugv8p4 is implemented.
// Otherwise, this function is IMPLEMENTATION DEFINED, and, in the
// recommended interface, ExternalNoninvasiveDebugEnabled returns
// the state of the (DBGEN OR NIDEN) signal.

readonly func ExternalNoninvasiveDebugEnabled() => boolean
begin
    return (IsFeatureImplemented(FEAT_Debugv8p4) || ExternalInvasiveDebugEnabled() ||
            NIDEN == HIGH);
end;
```

Library pseudocode for shared/debug/authentication/ExternalRealmInvasiveDebugEnabled

```
// ExternalRealmInvasiveDebugEnabled()
// ===================================
// The definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the
// (DBGEN AND RLPIDEN) signal.

readonly func ExternalRealmInvasiveDebugEnabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_RME) then return FALSE; end;
    return ExternalInvasiveDebugEnabled() && RLPIDEN == HIGH;
end;
```

Library pseudocode for shared/debug/authentication/ExternalRealmNoninvasiveDebugEnabled

```
// ExternalRealmNoninvasiveDebugEnabled()
// ======================================
// The definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the
// (DBGEN AND RLPIDEN) signal.

readonly func ExternalRealmNoninvasiveDebugEnabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_RME) then return FALSE; end;
    return ExternalRealmInvasiveDebugEnabled();
end;
```

Library pseudocode for shared/debug/authentication/ExternalRootInvasiveDebugEnabled

```
// ExternalRootInvasiveDebugEnabled()
// ==================================
// The definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the
// (DBGEN AND RLPIDEN AND RTPIDEN AND SPIDEN) signal when FEAT_SEL2 is implemented
// and the (DBGEN AND RLPIDEN AND RTPIDEN) signal when FEAT_SEL2 is not implemented.

readonly func ExternalRootInvasiveDebugEnabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_RME) then return FALSE; end;
    return (ExternalInvasiveDebugEnabled() &&
            (!IsFeatureImplemented(FEAT_SEL2) || ExternalSecureInvasiveDebugEnabled()) &&
            ExternalRealmInvasiveDebugEnabled() &&
            RTPIDEN == HIGH);
end;
```

Library pseudocode for shared/debug/authentication/ExternalRootNoninvasiveDebugEnabled

```
// ExternalRootNoninvasiveDebugEnabled()
// =====================================
// The definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the
// (DBGEN AND RLPIDEN AND SPIDEN AND RTPIDEN) signal.

func ExternalRootNoninvasiveDebugEnabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_RME) then return FALSE; end;
    return ExternalRootInvasiveDebugEnabled();
end;
```

Library pseudocode for shared/debug/authentication/ExternalSecureInvasiveDebugEnabled

```
// ExternalSecureInvasiveDebugEnabled()
// ====================================
// The definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the (DBGEN AND SPIDEN) signal.
// CoreSight allows asserting SPIDEN without also asserting DBGEN, but this is not recommended.

readonly func ExternalSecureInvasiveDebugEnabled() => boolean
begin
    if !HaveSecureState() then
        return IsFeatureImplemented(FEAT_RME) && RTPIDEN == HIGH;
    end;
    return ExternalInvasiveDebugEnabled() && SPIDEN == HIGH;
end;
```

Library pseudocode for shared/debug/authentication/ExternalSecureNoninvasiveDebugEnabled

```
// ExternalSecureNoninvasiveDebugEnabled()
// =======================================
// This function returns the value of ExternalSecureInvasiveDebugEnabled() when FEAT_Debugv8p4
// is implemented. Otherwise, the definition of this function is IMPLEMENTATION DEFINED.
// In the recommended interface, this function returns the state of the (DBGEN OR NIDEN) AND
// (SPIDEN OR SPNIDEN) signal.

readonly func ExternalSecureNoninvasiveDebugEnabled() => boolean
begin
    if !HaveSecureState() then return FALSE; end;
    if !IsFeatureImplemented(FEAT_Debugv8p4) then
        return (ExternalNoninvasiveDebugEnabled() &&
                (SPIDEN == HIGH || SPNIDEN == HIGH));
    else
        return ExternalSecureInvasiveDebugEnabled();
    end;
end;
```

Library pseudocode for shared/debug/authentication/InvasiveDebugPermittedPAS

```
// InvasiveDebugPermittedPAS()
// ===========================
// Returns TRUE if the invasive debug of the configured PASpace is permitted by
// the authentication interface, and FALSE otherwise.

readonly func InvasiveDebugPermittedPAS(pas : PASpace) => boolean
begin
    case pas of
        when PAS_Secure =>    return ExternalSecureInvasiveDebugEnabled();
        when PAS_NonSecure => return ExternalInvasiveDebugEnabled();
        when PAS_Root =>      return ExternalRootInvasiveDebugEnabled();
        when PAS_Realm =>     return ExternalRealmInvasiveDebugEnabled();
        otherwise =>          return FALSE;
    end;
end;
```

Library pseudocode for shared/debug/authentication/IsAccessNonSecure

```
// IsAccessNonSecure()
// ===================
// Returns TRUE when an access is Non-Secure

func IsAccessNonSecure(addrdesc : AddressDescriptor) => boolean
begin
    return addrdesc.paddress.paspace == PAS_NonSecure;
end;
```

Library pseudocode for shared/debug/authentication/IsAccessRealm

```
// IsAccessRealm()
// ===============
// Returns TRUE when an access is Realm

func IsAccessRealm(addrdesc : AddressDescriptor) => boolean
begin
    return addrdesc.paddress.paspace == PAS_Realm;
end;
```

Library pseudocode for shared/debug/authentication/IsAccessRoot

```
// IsAccessRoot()
// ==============
// Returns TRUE when an access is Root

func IsAccessRoot(addrdesc : AddressDescriptor) => boolean
begin
    return addrdesc.paddress.paspace == PAS_Root;
end;
```

Library pseudocode for shared/debug/authentication/IsAccessSecure

```
// IsAccessSecure()
// ================
// Returns TRUE when an access is Secure

func IsAccessSecure(addrdesc : AddressDescriptor) => boolean
begin
    return addrdesc.paddress.paspace == PAS_Secure;
end;
```

Library pseudocode for shared/debug/authentication/IsCorePowered

```
// IsCorePowered()
// ===============
// Returns TRUE if the Core power domain is powered on, FALSE otherwise.

impdef func IsCorePowered() => boolean
begin
    return TRUE;
end;
```

Library pseudocode for shared/debug/authentication/IsMostSecureAccess

```
// IsMostSecureAccess()
// ====================
// Returns TRUE if the security state of an access is the most secure PAS.

func IsMostSecureAccess(addrdesc : AddressDescriptor) => boolean
begin
    if IsFeatureImplemented(FEAT_RME) then
        return addrdesc.paddress.paspace == PAS_Root;
    elsif HaveEL(EL3) || SecureOnlyImplementation() then
        return addrdesc.paddress.paspace == PAS_Secure;
    else
        assert addrdesc.paddress.paspace == PAS_NonSecure;
        return TRUE;
    end;
end;
```

Library pseudocode for shared/debug/authentication/IsPASValid

```
// IsPASValid()
// ============
// Returns TRUE if the given value of 'pas' is not reserved, and FALSE otherwise.

func IsPASValid(pas : bits(2)) => boolean
begin
    case pas of
        when '00' => return IsFeatureImplemented(FEAT_Secure);
        when '01' => return TRUE;
        when '10' => return IsFeatureImplemented(FEAT_RME);
        when '11' => return IsFeatureImplemented(FEAT_RME);
    end;
end;
```

Library pseudocode for shared/debug/breakpoint/BreakpointInfo

```
// BreakpointInfo
// ==============
// Breakpoint related fields.

type BreakpointInfo of record {
    bptype : BreakpointType, // Type of breakpoint matched
    match : boolean,         // breakpoint match
    mismatch : boolean       // breakpoint mismatch
};
```

Library pseudocode for shared/debug/breakpoint/BreakpointType

```
// BreakpointType
// ==============

type BreakpointType of enumeration {
    BreakpointType_Inactive,     // Breakpoint inactive or disabled
    BreakpointType_AddrMatch,    // Address Match breakpoint
    BreakpointType_AddrMismatch, // Address Mismatch breakpoint
    BreakpointType_CtxtMatch  }; // Context matching breakpoint
```

Library pseudocode for shared/debug/breakpoint/CheckValidStateMatch

```
// CheckValidStateMatch()
// ======================
// Checks for an invalid state match that will generate Constrained
// Unpredictable behavior, otherwise returns Constraint_NONE.

func CheckValidStateMatch(ssc : bits(2), ssce : bit,
                          hmc : bit, pxc : bits(2),
                          isbreakpnt : boolean) => (Constraint, bits(2), bit, bit, bits(2))
begin
    if !IsFeatureImplemented(FEAT_RME) then assert ssce == '0'; end;
    var reserved : boolean = FALSE;

    // Values that are not allocated in any architecture version
    case hmc::ssce::ssc::pxc of
        when '0 0 11 10' => reserved = TRUE;
        when '0 1 01 00' => reserved = TRUE;
        when '1 0 00 x0' => reserved = TRUE;
        when '1 0 01 10' => reserved = TRUE;
        when '1 0 1x 10' => reserved = TRUE;
        when '1 1 01 10' => reserved = TRUE;
        when 'x 1 x0 xx' => reserved = TRUE;
        when 'x 1 11 xx' => reserved = TRUE;
        otherwise => reserved = FALSE;
    end;

    // Reserved when any combination has HMC = 0, SSCE = 0, and SSC = 0b10.
    if !IsFeatureImplemented(FEAT_Secure) && hmc::ssce::ssc == '0 0 10' then
        reserved = TRUE;
    end;

    // Reserved when any combination has HMC = 0 and SSC = 0b01,
    // and when HMC = 1, SSC = 0b10, and PxC = 0b00.
    if (!IsFeatureImplemented(FEAT_RME) && !IsFeatureImplemented(FEAT_Secure) &&
          (hmc::ssc == '0 01' || hmc::ssc::pxc == '1 10 00')) then
        reserved = TRUE;
    end;

    // Reserved when SSC is either 0b01 or 0b10, when EL3 is not implemented and
    // EL2 is implemented.
    if !HaveEL(EL3) && HaveEL(EL2) && ssc IN {'01','10'} then
        reserved = TRUE;
    end;

    // Reserved when either HMC = 1 or SSC is not 0b00, when both EL3 and
    // EL2 are not implemented.
    if !HaveEL(EL3) && !HaveEL(EL2) && (hmc == '1' || ssc != '00') then
        reserved = TRUE;
    end;

    // Reserved when HMC = 1, SSC = 0b11, and PxC = 0b00, when EL2 is not implemented.
    if !HaveEL(EL2) && hmc::ssc::pxc == '1 11 00' then
        reserved = TRUE;
    end;

    // Reserved when SSC = 0b11, except when HMC = 1, SSCE = 0, SSC = 0b11, and PxC = 0b00, when
    // Secure EL2 is not implemented.
    if (!IsFeatureImplemented(FEAT_SEL2) && ssc == '11' &&
          hmc::ssce::ssc::pxc != '1 0 11 00') then
        reserved = TRUE;
    end;

    // Reserved when HMC = 1, SSCE = 0, SSC = 0b01 and PxC = 0b00, when EL3 is not implemented OR
    // Secure EL2 is not implemented.
    if ((!HaveEL(EL3) || !IsFeatureImplemented(FEAT_SEL2)) &&
          hmc::ssce::ssc::pxc == '1 0 01 00') then
        reserved = TRUE;
    end;

    // Reserved when HMC = 1, SSC = 0b10, and PxC = 0b00, when AArch64 is not implemented.
    if !HaveAArch64() && hmc::ssc::pxc == '1 10 00' then
        reserved = TRUE;
    end;


    // 'Usr/Sys/Svc' encoding is valid only for AArch32 EL1 breakpoints.
    // Otherwise reserved.
    if (!HaveAArch32EL(EL1) || !isbreakpnt) && hmc::pxc == '000' && ssc != '11' then
        reserved = TRUE;
    end;

    if reserved then
        // If parameters are set to a reserved type, behaves as either disabled or a defined type
        var c : Constraint;
        var unpred_state_bits : bits(6);
        var unpred_hmc  : bit;
        var unpred_ssc  : bits(2);
        var unpred_ssce : bit;
        var unpred_pxc  : bits(2);
```

```
        (c, unpred_state_bits) = ConstrainUnpredictableBits{6}(Unpredictable_RESBPWPCTRL);
        assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};

        if c == Constraint_DISABLED then
            unpred_hmc  = ARBITRARY : bit;
            unpred_ssc  = ARBITRARY : bits(2);
            unpred_ssce = ARBITRARY : bit;
            unpred_pxc  = ARBITRARY : bits(2);
            return (c, unpred_ssc, unpred_ssce, unpred_hmc, unpred_pxc);
        else
            unpred_hmc  = unpred_state_bits[5];
            unpred_ssc  = unpred_state_bits[4:3];
            unpred_ssce = unpred_state_bits[2];
            unpred_pxc  = unpred_state_bits[1:0];
            return (Constraint_NONE, unpred_ssc, unpred_ssce, unpred_hmc, unpred_pxc);
        end;
    end;

    return (Constraint_NONE, ssc, ssce, hmc, pxc);
end;
```

Library pseudocode for shared/debug/breakpoint/ContextAwareBreakpointRange

```
// ContextAwareBreakpointRange()
// =============================
// Returns two numbers indicating the index of the first and last context-aware breakpoint.

func ContextAwareBreakpointRange() => (integer, integer)
begin
    let b : integer = NumBreakpointsImplemented();
    let c : integer = NumContextAwareBreakpointsImplemented();

    if b <= 16 then
        return (b - c, b - 1);
    elsif c <= 16 then
        return (16 - c, 15);
    else
        return (0, c - 1);
    end;
end;
```

Library pseudocode for shared/debug/breakpoint/IsContextAwareBreakpoint

```
// IsContextAwareBreakpoint()
// ==========================
// Returns TRUE if DBGBCR_EL1[n] is a context-aware breakpoint.

func IsContextAwareBreakpoint(n : integer) => boolean
begin
    let (lower, upper) : (integer, integer) = ContextAwareBreakpointRange();
    return n >= lower && n <= upper;
end;
```

Library pseudocode for shared/debug/breakpoint/NumBreakpointsImplemented

```
// NumBreakpointsImplemented()
// ===========================
// Returns the number of breakpoints implemented.

readonly func NumBreakpointsImplemented() => integer
begin
    return ImpDefInt("Number of breakpoints");
end;
```

Library pseudocode for shared/debug/breakpoint/NumContextAwareBreakpointsImplemented

```
// NumContextAwareBreakpointsImplemented()
// =======================================
// Returns the number of context-aware breakpoints implemented.

readonly func NumContextAwareBreakpointsImplemented() => integer
begin
    return ImpDefInt("Number of context-aware breakpoints");
end;
```

Library pseudocode for shared/debug/breakpoint/NumWatchpointsImplemented

```
// NumWatchpointsImplemented()
// ===========================
// Returns the number of watchpoints implemented.

readonly func NumWatchpointsImplemented() => integer
begin
    return ImpDefInt("Number of watchpoints");
end;
```

Library pseudocode for shared/debug/claim

```
var DBGClaimTags : bits(8);
```

Library pseudocode for shared/debug/claim/Write_DBGCLAIMCLR

```
// Write_DBGCLAIMCLR()
// ===================
// System register write to DBGCLAIMCLR.

func Write_DBGCLAIMCLR(value_in : bits(32))
begin
    DBGClaimTags = DBGClaimTags AND NOT value_in[7:0];
end;
```

Library pseudocode for shared/debug/claim/Write_DBGCLAIMCLR_EL1

```
// Write_DBGCLAIMCLR_EL1()
// =======================
// System register write to DBGCLAIMCLR_EL1.

func Write_DBGCLAIMCLR_EL1(value_in : bits(64))
begin
    DBGClaimTags = DBGClaimTags AND NOT value_in[7:0];
end;
```

Library pseudocode for shared/debug/claim/Write_DBGCLAIMSET

```
// Write_DBGCLAIMSET()
// ===================
// System register write to DBGCLAIMSET.

func Write_DBGCLAIMSET(value_in : bits(32))
begin
    DBGClaimTags = DBGClaimTags OR value_in[7:0];
end;
```

Library pseudocode for shared/debug/claim/Write_DBGCLAIMSET_EL1

```
// Write_DBGCLAIMSET_EL1()
// =======================
// System register write to DBGCLAIMSET_EL1.

func Write_DBGCLAIMSET_EL1(value_in : bits(64))
begin
    DBGClaimTags = DBGClaimTags OR value_in[7:0];
end;
```

Library pseudocode for shared/debug/cti/CTI_ProcessEvent

```
// CTI_ProcessEvent()
// ==================
// Process a discrete event on a Cross Trigger output event trigger.

impdef func CTI_ProcessEvent(id : CrossTriggerOut)
begin
    return;
end;
```

Library pseudocode for shared/debug/cti/CTI_SetEventLevel

```
// CTI_SetEventLevel()
// ===================
// Set a Cross Trigger multi-cycle input event trigger to the specified level.

impdef func CTI_SetEventLevel(id : CrossTriggerIn, level : Signal)
begin
    return;
end;
```

Library pseudocode for shared/debug/cti/CTI_SignalEvent

```
// CTI_SignalEvent()
// =================
// Signal a discrete event on a Cross Trigger input event trigger.

impdef func CTI_SignalEvent(id : CrossTriggerIn)
begin
    return;
end;
```

Library pseudocode for shared/debug/cti/CrossTrigger

```
// CrossTrigger
// ============

type CrossTriggerOut of enumeration {CrossTriggerOut_DebugRequest, CrossTriggerOut_RestartRequest,
                                     CrossTriggerOut_IRQ,          CrossTriggerOut_RSVD3,
                                     CrossTriggerOut_TraceExtIn0,  CrossTriggerOut_TraceExtIn1,
                                     CrossTriggerOut_TraceExtIn2,  CrossTriggerOut_TraceExtIn3};

type CrossTriggerIn of enumeration {CrossTriggerIn_CrossHalt,     CrossTriggerIn_PMUOverflow,
                                    CrossTriggerIn_SPESample,     CrossTriggerIn_RSVD3,
                                    CrossTriggerIn_TraceExtOut0,  CrossTriggerIn_TraceExtOut1,
                                    CrossTriggerIn_TraceExtOut2,  CrossTriggerIn_TraceExtOut3,
                                    CrossTriggerIn_TRBEStop,      CrossTriggerIn_TRBEMgmt,
                                    CrossTriggerIn_TRBEWrap};

var CTIClaimTags : bits(32);
```

Library pseudocode for shared/debug/dccanditr/CheckForDCCInterrupts

```
// CheckForDCCInterrupts()
// =======================

func CheckForDCCInterrupts()
begin
    let commrx : boolean = (EDSCR().RXfull == '1');
    let commtx : boolean = (EDSCR().TXfull == '0');

    // COMMRX and COMMTX support is optional and not recommended for new designs.
    // SetInterruptRequestLevel(InterruptID_COMMRX, if commrx then HIGH else LOW);
    // SetInterruptRequestLevel(InterruptID_COMMTX, if commtx then HIGH else LOW);

    // The value to be driven onto the common COMMIRQ signal.
    var commirq : boolean;
    if ELUsingAArch32(EL1) then
        commirq = ((commrx && DBGDCCINT().RX == '1') ||
                   (commtx && DBGDCCINT().TX == '1'));
    else
        commirq = ((commrx && MDCCINT_EL1().RX == '1') ||
                   (commtx && MDCCINT_EL1().TX == '1'));
    end;
    SetInterruptRequestLevel(InterruptID_COMMIRQ, if commirq then HIGH else LOW);

    return;
end;
```

Library pseudocode for shared/debug/dccanditr/DTR

```
// DTR
// ===

var DTRRX : bits(32);
var DTRTX : bits(32);
```

Library pseudocode for shared/debug/dccanditr/Read_DBGDTRRX_EL0

```
// Read_DBGDTRRX_EL0()
// ===================
// Called on reads of debug register 0x080.

func Read_DBGDTRRX_EL0(memory_mapped : boolean) => bits(32)
begin
    return DTRRX;
end;
```

Library pseudocode for shared/debug/dccanditr/Read_DBGDTRTX_EL0

```
// Read_DBGDTRTX_EL0()
// ===================
// Called on reads of debug register 0x08C.

func Read_DBGDTRTX_EL0(memory_mapped : boolean) => bits(32)
begin
    let underrun : boolean = (EDSCR().TXfull == '0' ||
                              (Halted() && EDSCR().MA == '1' && EDSCR().ITE == '0'));
    let value : bits(32)   = if underrun then ARBITRARY : bits(32) else DTRTX;

    if EDSCR().ERR == '1' then return value; end;       // Error flag set: no side-effects

    if underrun then
        EDSCR().TXU = '1';  EDSCR().ERR = '1';          // Underrun condition: block side-effects
        return value;                                   // Return UNKNOWN
    end;

    EDSCR().TXfull = '0';
    if Halted() && EDSCR().MA == '1' then
        EDSCR().ITE = '0';                              // See comments in Write_EDITR()

        if !UsingAArch32() then
            ExecuteA64(0xB8404401[31:0]);               // A64 "LDR W1,[X0],#4"
        else
            ExecuteT32(0xF850[15:0] /*hw1*/, 0x1B04[15:0] /*hw2*/);      // T32 "LDR R1,[R0],#4"
        end;
        // If the load aborts, the Data Abort exception is taken and EDSCR.ERR is set to 1
        if EDSCR().ERR == '1' then
            EDSCR().TXfull = ARBITRARY : bit;
            DBGDTRTX_EL0() = ARBITRARY : bits(64);
        else

            if !UsingAArch32() then
                ExecuteA64(0xD5130501[31:0]);           // A64 "MSR DBGDTRTX_EL0,X1"
            else
                ExecuteT32(0xEE00[15:0] /*hw1*/, 0x1E15[15:0] /*hw2*/);  // T32 "MSR DBGDTRTXint,R1"
            end;
            // "MSR DBGDTRTX_EL0,X1" calls Write_DBGDTR_EL0() which sets TXfull.
            assert EDSCR().TXfull == '1';
        end;
        if !UsingAArch32() then
            X{64}(1) = ARBITRARY : bits(64);
        else
            R(1) = ARBITRARY : bits(32);
        end;
        EDSCR().ITE = '1';                                // See comments in Write_EDITR()
    end;

    return value;
end;
```

Library pseudocode for shared/debug/dccanditr/Read_DBGDTR_EL0

```
// Read_DBGDTR_EL0()
// =================
// System register reads of DBGDTR_EL0, DBGDTRRX_EL0 (AArch64) and DBGDTRRXint (AArch32)

func Read_DBGDTR_EL0{N}() => bits(N)
begin
    // For MRS <Rt>,DBGDTRTX_EL0  N=32, X[t]=Zeros(32):result
    // For MRS <Xt>,DBGDTR_EL0    N=64, X[t]=result
    assert N IN {32,64};
    var result : bits(N);
    if EDSCR().RXfull == '0' then
        result = ARBITRARY : bits(N);
    else
        // On a 64-bit read, implement a half-duplex channel
        // NOTE: the word order is reversed on reads with regards to writes
        if N == 64 then result[63:32] = DTRTX; end;
        result[31:0] = DTRRX;
    end;
    EDSCR().RXfull = '0';
    return result;
end;
```

Library pseudocode for shared/debug/dccanditr/Write_DBGDTRRX_EL0

```
// Write_DBGDTRRX_EL0()
// ====================
// Called on writes to debug register 0x080.

func Write_DBGDTRRX_EL0(memory_mapped : boolean, value : bits(32))
begin
    if EDSCR().ERR == '1' then return; end;                // Error flag set: ignore write

    if EDSCR().RXfull == '1' || (Halted() && EDSCR().MA == '1' && EDSCR().ITE == '0') then
        EDSCR().RXO = '1';  EDSCR().ERR = '1';              // Overrun condition: ignore write
        return;
    end;

    EDSCR().RXfull = '1';
    DTRRX          = value;

    if Halted() && EDSCR().MA == '1' then
        EDSCR().ITE = '0';                                 // See comments in Write_EDITR()
        if !UsingAArch32() then
            ExecuteA64(0xD5330501[31:0]);                 // A64 "MRS X1,DBGDTRRX_EL0"
            ExecuteA64(0xB8004401[31:0]);                 // A64 "STR W1,[X0],#4"
            X{64}(1) = ARBITRARY : bits(64);
        else
            ExecuteT32(0xEE10[15:0] /*hw1*/, 0x1E15[15:0] /*hw2*/);  // T32 "MRS R1,DBGDTRRXint"
            ExecuteT32(0xF840[15:0] /*hw1*/, 0x1B04[15:0] /*hw2*/);  // T32 "STR R1,[R0],#4"
            R(1) = ARBITRARY : bits(32);
        end;
        // If the store aborts, the Data Abort exception is taken and EDSCR.ERR is set to 1
        if EDSCR().ERR == '1' then
            EDSCR().RXfull = ARBITRARY : bit;
            DBGDTRRX_EL0() = ARBITRARY : bits(64);
        else
            // "MRS X1,DBGDTRRX_EL0" calls Read_DBGDTR_EL0() which clears RXfull.
            assert EDSCR().RXfull == '0';
        end;

        EDSCR().ITE = '1';                                 // See comments in Write_EDITR()
    end;
    return;
end;
```

Library pseudocode for shared/debug/dccanditr/Write_DBGDTRTX_EL0

```
// Write_DBGDTRTX_EL0()
// ====================
// Called on writes to debug register 0x08C.

func Write_DBGDTRTX_EL0(memory_mapped : boolean, value : bits(32))
begin
    DTRTX = value;
    return;
end;
```

Library pseudocode for shared/debug/dccanditr/Write_DBGDTR_EL0

```
// Write_DBGDTR_EL0()
// ==================
// System register writes to DBGDTR_EL0, DBGDTRTX_EL0 (AArch64) and DBGDTRTXint (AArch32)

func Write_DBGDTR_EL0{N}(value_in : bits(N))
begin
    var value : bits(N) = value_in;
    // For MSR DBGDTRTX_EL0,<Rt>  N=32, value=X[t]<31:0>, X[t]<63:32> is ignored
    // For MSR DBGDTR_EL0,<Xt>    N=64, value=X[t]<63:0>
    assert N IN {32,64};
    if EDSCR().TXfull == '1' then
        value = ARBITRARY : bits(N);
    end;
    // On a 64-bit write, implement a half-duplex channel
    if N == 64 then DTRRX = value[63:32]; end;
    DTRTX = value[31:0];        // 32-bit or 64-bit write
    EDSCR().TXfull = '1';
    return;
end;
```

Library pseudocode for shared/debug/dccanditr/Write_EDITR

```
// Write_EDITR()
// =============
// Called on writes to debug register 0x084.

func Write_EDITR(memory_mapped : boolean, value : bits(32))
begin
    if EDSCR().ERR == '1' then return; end;         // Error flag set: ignore write

    if !Halted() then return; end;                  // Non-debug state: ignore write

    if EDSCR().ITE == '0' || EDSCR().MA == '1' then
        EDSCR().ITO = '1';  EDSCR().ERR = '1';      // Overrun condition: block write
        return;
    end;

    // ITE indicates whether the PE is ready to accept another instruction; the PE
    // may support multiple outstanding instructions. Unlike the "InstrCompl" flag in [v7A] there
    // is no indication that the pipeline is empty (all instructions have completed). In this
    // pseudocode, the assumption is that only one instruction can be executed at a time,
    // meaning ITE acts like "InstrCompl".
    EDSCR().ITE = '0';

    if !UsingAArch32() then
        ExecuteA64(value);
    else
        ExecuteT32(value[15:0]/*hw1*/, value[31:16] /*hw2*/);
    end;

    EDSCR().ITE = '1';

    return;
end;
```

Library pseudocode for shared/debug/halting/DCPSInstruction

```
// DCPSInstruction()
// =================
// Operation of the DCPS instruction in Debug state

func DCPSInstruction(target_el : bits(2))
begin

    SynchronizeContext();

    var handle_el : bits(2);
    case target_el of
        when EL1 =>
            if PSTATE.EL == EL2 || (PSTATE.EL == EL3 && !UsingAArch32()) then
                handle_el = PSTATE.EL;
            elsif EL2Enabled() && HCR_EL2().TGE == '1' then
                Undefined();
            else
                handle_el = EL1;
            end;
        when EL2 =>
            if !HaveEL(EL2) then
                Undefined();
            elsif PSTATE.EL == EL3 && !UsingAArch32() then
                handle_el = EL3;
            elsif !IsSecureEL2Enabled() && CurrentSecurityState() == SS_Secure then
                Undefined();
            else
                handle_el = EL2;
            end;
        when EL3 =>
            if EDSCR().SDD == '1' || !HaveEL(EL3) then
                Undefined();
            else
                handle_el = EL3;
            end;
        otherwise =>
            unreachable;
    end;

    let from_secure : boolean = CurrentSecurityState() == SS_Secure;
    if ELUsingAArch32(handle_el) then
        if PSTATE.M == M32_Monitor then SCR().NS = '0'; end;
        assert UsingAArch32();                  // Cannot move from AArch64 to AArch32
        case handle_el of
            when EL1 =>
                AArch32_WriteMode(M32_Svc);
                if IsFeatureImplemented(FEAT_PAN) && SCTLR().SPAN == '0' then
                    PSTATE.PAN = '1';
                end;
            when EL2 =>  AArch32_WriteMode(M32_Hyp);
            when EL3 =>
                AArch32_WriteMode(M32_Monitor);
                if IsFeatureImplemented(FEAT_PAN) then
                    if !from_secure then
                        PSTATE.PAN = '0';
                    elsif SCTLR().SPAN == '0' then
                        PSTATE.PAN = '1';
                    end;
                end;
        end;
        if handle_el == EL2 then
            ELR_hyp() = ARBITRARY : bits(32);  HSR() = ARBITRARY : bits(32);
        else
            LR() = ARBITRARY : bits(32);
        end;
        SPSR_curr() = ARBITRARY : bits(32);
        if handle_el == EL2 then
            PSTATE.E = HSCTLR().EE;
        else
            PSTATE.E = SCTLR().EE;
        end;
        DLR() = ARBITRARY : bits(32);  DSPSR() = ARBITRARY : bits(32);

    else                                        // Targeting AArch64
        let from_32 : boolean = UsingAArch32();
        if from_32 then AArch64_MaybeZeroRegisterUppers(); end;
        if from_32 && IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' then
            ResetSVEState();
        else
            MaybeZeroSVEUppers(target_el);
        end;
        PSTATE.nRW = '0';  PSTATE.SP = '1';  PSTATE.EL = handle_el;
        if IsFeatureImplemented(FEAT_PAN) && ((handle_el == EL1 && SCTLR_EL1().SPAN == '0') ||
                                              (handle_el == EL2 && ELIsInHost(EL0) &&
                                               SCTLR_EL2().SPAN == '0')) then
            PSTATE.PAN = '1';
        end;
        ELR_ELx() = ARBITRARY : bits(64);
        SPSR_ELx() = ARBITRARY : bits(64);
```

```
        ESR_ELx() = ARBITRARY : bits(64);
        DLR_EL0() = ARBITRARY : bits(64);  DSPSR_EL0() = ARBITRARY : bits(64);
        if IsFeatureImplemented(FEAT_UAO) then PSTATE.UAO = '0'; end;
        if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = '1'; end;
        if IsFeatureImplemented(FEAT_GCS) then PSTATE.EXLOCK = '0'; end;
    end;
    if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
    UpdateEDSCRFields();                          // Update EDSCR PE state flags
    return;
end;
```

Library pseudocode for shared/debug/halting/DRPSInstruction

```
// DRPSInstruction()
// =================
// Operation of the A64 DRPS and T32 ERET instructions in Debug state

func DRPSInstruction()
begin
    SynchronizeContext();

    DebugRestorePSR();

    return;
end;
```

Library pseudocode for shared/debug/halting/DebugHalt

```
// DebugHalt
// =========
// Reason codes for entry to Debug state

constant DebugHalt_Breakpoint : bits(6)      = '000111';
constant DebugHalt_EDBGRQ : bits(6)          = '010011';
constant DebugHalt_Step_Normal : bits(6)     = '011011';
constant DebugHalt_Step_Exclusive : bits(6)  = '011111';
constant DebugHalt_OSUnlockCatch : bits(6)   = '100011';
constant DebugHalt_ResetCatch : bits(6)      = '100111';
constant DebugHalt_Watchpoint : bits(6)      = '101011';
constant DebugHalt_HaltInstruction : bits(6) = '101111';
constant DebugHalt_SoftwareAccess : bits(6)  = '110011';
constant DebugHalt_ExceptionCatch : bits(6)  = '110111';
constant DebugHalt_Step_NoSyndrome : bits(6) = '111011';
```

Library pseudocode for shared/debug/halting/DebugRestorePSR

```
// DebugRestorePSR()
// =================

func DebugRestorePSR()
begin
    // PSTATE.[N,Z,C,V,Q,GE,SS,D,A,I,F] are not observable and ignored in Debug state, so
    // behave as if UNKNOWN.
    if UsingAArch32() then
        let spsr : bits(32) = SPSR_curr();
        SetPSTATEFromPSR{32}(spsr);
        PSTATE.[N,Z,C,V,Q,GE,SS,A,I,F] = ARBITRARY : bits(13);
        //  In AArch32, all instructions are T32 and unconditional.
        PSTATE.IT = '00000000';  PSTATE.T = '1';        // PSTATE.J is RES0
        DLR() = ARBITRARY : bits(32);  DSPSR() = ARBITRARY : bits(32);
    else
        let spsr : bits(64) = SPSR_ELx();
        SetPSTATEFromPSR{64}(spsr);
        PSTATE.[N,Z,C,V,SS,D,A,I,F] = ARBITRARY : bits(9);
        DLR_EL0() = ARBITRARY : bits(64);  DSPSR_EL0() = ARBITRARY : bits(64);
    end;
    UpdateEDSCRFields();                              // Update EDSCR PE state flags
end;
```

Library pseudocode for shared/debug/halting/DisableITRAndResumeInstructionPrefetch

```
// DisableITRAndResumeInstructionPrefetch()
// ========================================

impdef func DisableITRAndResumeInstructionPrefetch()
begin
    return;
end;
```

Library pseudocode for shared/debug/halting/ExecuteA64

```
// ExecuteA64()
// ============
// Execute an A64 instruction in Debug state.

impdef func ExecuteA64(instr : bits(32))
begin
    Unimplemented();
end;
```

Library pseudocode for shared/debug/halting/ExecuteT32

```
// ExecuteT32()
// ============
// Execute a T32 instruction in Debug state.

impdef func ExecuteT32(hw1 : bits(16), hw2 : bits(16))
begin
    Unimplemented();
end;
```

Library pseudocode for shared/debug/halting/ExitDebugState

```
// ExitDebugState()
// ================

func ExitDebugState()
begin
    assert Halted();
    SynchronizeContext();

    // Although EDSCR.STATUS signals that the PE is restarting, debuggers must use EDPRSR.SDR to
    // detect that the PE has restarted.
    EDSCR().STATUS = '000001';                          // Signal restarting
    // Clear any pending Halting debug events
    if IsFeatureImplemented(FEAT_Debugv8p8) then
        EDESR()[3:0] = '0000';
    else
        EDESR()[2:0] = '000';
    end;

    var new_pc : bits(64);
    var spsr : bits(64);

    if UsingAArch32() then
        new_pc = ZeroExtend{64}(DLR());
        if IsFeatureImplemented(FEAT_Debugv8p9) then
            spsr = DSPSR2() :: DSPSR();
        else
            spsr = ZeroExtend{64}(DSPSR());
        end;
    else
        new_pc = DLR_EL0();
        spsr = DSPSR_EL0();
    end;

    let illegal_psr_state : boolean = IllegalExceptionReturn{64}(spsr);
    // If this is an illegal return, SetPSTATEFromPSR() will set PSTATE.IL.
    SetPSTATEFromPSR{64}(spsr); // Can update privileged bits, even at EL0

    let branch_conditional : boolean = FALSE;
    if UsingAArch32() then
        if ConstrainUnpredictableBool(Unpredictable_RESTARTALIGNPC) then new_pc[0] = '0'; end;
        // AArch32 branch
        BranchTo{32}(new_pc[31:0], BranchType_DBGEXIT, branch_conditional);
    else
        // If targeting AArch32 then PC[63:32,1:0] might be set to UNKNOWN.
        if illegal_psr_state && spsr[4] == '1' then
            new_pc[63:32] = ARBITRARY : bits(32);
            new_pc[1:0] = ARBITRARY : bits(2);
        end;
        if IsFeatureImplemented(FEAT_BRBE) then
            BRBEDebugStateExit(AArch64_BranchAddr(new_pc,PSTATE.EL));
        end;
        // A type of branch that is never predicted
        BranchTo{64}(new_pc, BranchType_DBGEXIT, branch_conditional);
    end;

     // Atomically signal restarted
    EDSCR().STATUS = '000010';
    EDPRSR().SDR = '1';
    // End of atomically signal
    EDPRSR().HALTED = '0';
    UpdateEDSCRFields();                                // Stop signaling PE state
    DisableITRAndResumeInstructionPrefetch();

    return;
end;
```

Library pseudocode for shared/debug/halting/Halt

```
// Halt()
// ======

func Halt(reason : bits(6))
begin
    let is_async : boolean = FALSE;
    let fault : FaultRecord = NoFault();
    Halt(reason, is_async, fault);
end;

// Halt()
// ======

func Halt(reason : bits(6), is_async : boolean, fault : FaultRecord)
begin
    CTI_SignalEvent(CrossTriggerIn_CrossHalt);  // Trigger other cores to halt

    let preferred_restart_address : bits(64) = ThisInstrAddr{}();
    var spsr : bits(64) = GetPSRFromPSTATE{64}(DebugState);

    if (IsFeatureImplemented(FEAT_BTI) && !is_async &&
          ! reason IN {DebugHalt_Step_Normal, DebugHalt_Step_Exclusive,
                       DebugHalt_Step_NoSyndrome, DebugHalt_Breakpoint,
                       DebugHalt_HaltInstruction} &&
          ConstrainUnpredictableBool(Unpredictable_ZEROBTYPE)) then
        spsr[11:10] = '00';
    end;

    if UsingAArch32() then
        DLR() = preferred_restart_address[31:0];
        DSPSR() = spsr[31:0];
        if IsFeatureImplemented(FEAT_Debugv8p9) then
            DSPSR2() = spsr[63:32];
        end;
    else
        DLR_EL0() = preferred_restart_address;
        DSPSR_EL0() = spsr;
    end;
    EDSCR().ITE = '1';
    EDSCR().ITO = '0';
    if IsFeatureImplemented(FEAT_RME) then
        if PSTATE.EL == EL3 then
            EDSCR().SDD = '0';
        else
            EDSCR().SDD = if ExternalRootInvasiveDebugEnabled() then '0' else '1';
        end;
    elsif CurrentSecurityState() == SS_Secure then
        EDSCR().SDD = '0';                          // If entered in Secure state, allow debug
    elsif HaveEL(EL3) then
        EDSCR().SDD = if ExternalSecureInvasiveDebugEnabled() then '0' else '1';
    else
        EDSCR().SDD = '1';                          // Otherwise EDSCR.SDD is RES1
    end;
    EDSCR().MA = '0';

    // In Debug state:
    // * PSTATE.[SS,SSBS,D,A,I,F] are not observable and ignored so behave-as-if UNKNOWN.
    // * PSTATE.[N,Z,C,V,Q,GE,E,M,nRW,EL,SP,DIT] are also not observable, but since these
    //     are not changed on exception entry, this function also leaves them unchanged.
    // * PSTATE.[IT,T] are ignored.
    // * PSTATE.IL is ignored and behave-as-if 0.
    // * PSTATE.BTYPE is ignored and behave-as-if 0.
    // * PSTATE.TCO is set 1.
    // * PSTATE.PACM is ignored and behave-as-if 0.
    // * PSTATE.[UAO,PAN] are observable and not changed on entry into Debug state.
    // * PSTATE.UINJ is set to 0.

    if UsingAArch32() then
        PSTATE.[IT,SS,SSBS,A,I,F,T] = ARBITRARY : bits(14);
    else
        PSTATE.[SS,SSBS,D,A,I,F]    = ARBITRARY : bits(6);
    end;

    if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = '1'; end;
    if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = '00'; end;
    if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = '0'; end;
    PSTATE.IL = '0';
    if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
    if IsFeatureImplemented(FEAT_BRBE) then
        BRBEDebugStateEntry(preferred_restart_address);
    end;
    StopInstructionPrefetchAndEnableITR();
    // atomic write
    EDSCR().STATUS = reason;
    EDPRSR().HALTED = '1';
    // end of atomic write
    UpdateEDSCRFields();                            // Update EDSCR PE state flags.
    if IsFeatureImplemented(FEAT_EDHSR) then
        UpdateEDHSR(reason, fault);                 // Update EDHSR fields.
```

```
    end;
    if !is_async then EndOfInstruction(EndOfInstructionReason_Exception); end;
    return;
end;
```

Library pseudocode for shared/debug/halting/HaltOnBreakpointOrWatchpoint

```
// HaltOnBreakpointOrWatchpoint()
// ==============================
// Returns TRUE if the Breakpoint and Watchpoint debug events should be considered for Debug
// state entry, FALSE if they should be considered for a debug exception.

func HaltOnBreakpointOrWatchpoint() => boolean
begin
    return HaltingAllowed() && EDSCR().HDE == '1' && OSLSR_EL1().OSLK == '0';
end;
```

Library pseudocode for shared/debug/halting/Halted

```
// Halted()
// ========

readonly func Halted() => boolean
begin
    return ! EDSCR().STATUS IN {'000001', '000010'};                    // Halted
end;
```

Library pseudocode for shared/debug/halting/HaltingAllowed

```
// HaltingAllowed()
// ================
// Returns TRUE if halting is currently allowed, FALSE if halting is prohibited.

func HaltingAllowed() => boolean
begin
    if Halted() || DoubleLockStatus() then
        return FALSE;
    end;
    let ss : SecurityState = CurrentSecurityState();
    case ss of
        when SS_NonSecure => return ExternalInvasiveDebugEnabled();
        when SS_Secure =>    return ExternalSecureInvasiveDebugEnabled();
        when SS_Root =>      return ExternalRootInvasiveDebugEnabled();
        when SS_Realm =>     return ExternalRealmInvasiveDebugEnabled();
    end;
end;
```

Library pseudocode for shared/debug/halting/Restarting

```
// Restarting()
// ============

readonly func Restarting() => boolean
begin
    return EDSCR().STATUS == '000001';                                  // Restarting
end;
```

Library pseudocode for shared/debug/halting/StopInstructionPrefetchAndEnableITR

```
// StopInstructionPrefetchAndEnableITR()
// =====================================

impdef func StopInstructionPrefetchAndEnableITR()
begin
    return;
end;
```

Library pseudocode for shared/debug/halting/UpdateDbgAuthStatus

```
// UpdateDbgAuthStatus()
// =====================
// Provides information about the state of the
// IMPLEMENTATION DEFINED authentication interface for debug.

func UpdateDbgAuthStatus()
begin
    var nsid, nsnid : bits(2);
    var sid, snid : bits(2);
    var rlid, rtid : bits(2);
    if SecureOnlyImplementation() then
        nsid = '00';
    elsif ExternalInvasiveDebugEnabled() then
        nsid = '11';          // Non-secure Invasive debug implemented and enabled.
    else
        nsid = '10';          // Non-secure Invasive debug implemented and disabled.
    end;

    if SecureOnlyImplementation() then
        nsnid = '00';
    elsif ExternalNoninvasiveDebugEnabled() then
        nsnid = '11';         // Non-secure Non-Invasive debug implemented and enabled.
    else
        nsnid = '10';         // Non-secure Non-Invasive debug implemented and disabled.
    end;

    if !HaveSecureState() then
        sid = '00';
    elsif ExternalSecureInvasiveDebugEnabled() then
        sid = '11';           // Secure Invasive debug implemented and enabled.
    else
        sid = '10';           // Secure Invasive debug implemented and disabled.
    end;

    if !HaveSecureState() then
        snid = '00';
    elsif ExternalSecureNoninvasiveDebugEnabled() then
        snid = '11';          // Secure Non-Invasive debug implemented and enabled.
    else
        snid = '10';          // Secure Non-Invasive debug implemented and disabled.
    end;

    if !IsFeatureImplemented(FEAT_RME) then
        rlid = '00';
    elsif ExternalRealmInvasiveDebugEnabled() then
        rlid = '11';          // Realm Invasive debug implemented and enabled.
    else
        rlid = '10';          // Realm Invasive debug implemented and disabled.
    end;

    if !IsFeatureImplemented(FEAT_RME) then
        rtid = '00';
    elsif ExternalRootInvasiveDebugEnabled() then
        rtid = '11';          // Root Invasive debug implemented and enabled.
    else
        rtid = '10';          // Root Invasive debug implemented and disabled.
    end;

    DBGAUTHSTATUS_EL1().NSID  = nsid;
    DBGAUTHSTATUS_EL1().NSNID = nsnid;
    DBGAUTHSTATUS_EL1().SID   = sid;
    DBGAUTHSTATUS_EL1().SNID  = snid;
    DBGAUTHSTATUS_EL1().RLID  = rlid;
    DBGAUTHSTATUS_EL1().RLNID = rlid;    // Field has the same value as DBGAUTHSTATUS_EL1.RLID.
    DBGAUTHSTATUS_EL1().RTID  = rtid;
    DBGAUTHSTATUS_EL1().RTNID = rtid;    // Field has the same value as DBGAUTHSTATUS_EL1.RTID.
    return;
end;
```

Library pseudocode for shared/debug/halting/UpdateEDHSR

```
// UpdateEDHSR()
// =============
// Update EDHSR watchpoint related fields.

func UpdateEDHSR(reason : bits(6), fault : FaultRecord)
begin
    var syndrome : bits(64) = Zeros{};
    if reason == DebugHalt_Watchpoint then
        if IsFeatureImplemented(FEAT_GCS) && fault.accessdesc.acctype == AccessType_GCS then
            syndrome[40] = '1';                         // GCS
        end;
        syndrome[23:0] = WatchpointRelatedSyndrome(fault);
        if IsFeatureImplemented(FEAT_Debugv8p9) then
            if fault.write then syndrome[6] = '1'; end; // WnR
            if fault.accessdesc.acctype IN {AccessType_DC, AccessType_IC, AccessType_AT} then
                syndrome[8] = '1';                      // CM
            end;
            if IsFeatureImplemented(FEAT_NV2) && fault.accessdesc.acctype == AccessType_NV2 then
                syndrome[13] = '1';                     // VNCR
            end;
        end;
    else
        syndrome = ARBITRARY : bits(64);
    end;

    EDHSR() = syndrome;
end;
```

Library pseudocode for shared/debug/halting/UpdateEDSCRFields

```
// UpdateEDSCRFields()
// ===================
// Update EDSCR PE state fields

func UpdateEDSCRFields()
begin
    if !Halted() then
        EDSCR().EL = '00';
        if IsFeatureImplemented(FEAT_RME) then
           // SDD bit.
            EDSCR().SDD = if ExternalRootInvasiveDebugEnabled() then '0' else '1';
            EDSCR().[NSE,NS] = ARBITRARY : bits(2);
        else
            // SDD bit.
            EDSCR().SDD = if ExternalSecureInvasiveDebugEnabled() then '0' else '1';
            EDSCR().NS = ARBITRARY : bit;
        end;

        EDSCR().RW = '1111';
    else
        EDSCR().EL = PSTATE.EL;
        // SError Pending.
        if EL2Enabled() && HCR_EL2().[AMO,TGE] == '10' && PSTATE.EL IN {EL0,EL1} then
            EDSCR().A = if IsVirtualSErrorPending() then '1' else '0';
        elsif (EffectiveSCR_EL3_EnDSE() == '1' && PSTATE.EL IN {EL0,EL1,EL2}) then
            EDSCR().A = if IsDelegatedSErrorPending() then '1' else '0';
        else
            EDSCR().A = if IsPhysicalSErrorPending() then '1' else '0';
        end;

        let ss : SecurityState = CurrentSecurityState();
        if IsFeatureImplemented(FEAT_RME) then
            case ss of
                when SS_Secure =>    EDSCR().[NSE,NS] = '00';
                when SS_NonSecure => EDSCR().[NSE,NS] = '01';
                when SS_Root =>      EDSCR().[NSE,NS] = '10';
                when SS_Realm =>     EDSCR().[NSE,NS] = '11';
            end;
        else
            EDSCR().NS = if ss == SS_Secure then '0' else '1';
        end;

        var RW : bits(4);
        RW[1] = if ELUsingAArch32(EL1) then '0' else '1';
        if PSTATE.EL != EL0 then
            RW[0] = RW[1];
        else
            RW[0] = if UsingAArch32() then '0' else '1';
        end;
        if !EL2Enabled() then
            RW[2] = RW[1];
        else
            RW[2] = if ELUsingAArch32(EL2) then '0' else '1';
        end;
        if !HaveEL(EL3) then
            RW[3] = RW[2];
        else
            RW[3] = if ELUsingAArch32(EL3) then '0' else '1';
        end;

        // The least-significant bits of EDSCR.RW are UNKNOWN if any higher EL is using AArch32.
        if RW[3] == '0' then RW[2:0] = ARBITRARY : bits(3);
        elsif RW[2] == '0' then RW[1:0] = ARBITRARY : bits(2);
        elsif RW[1] == '0' then RW[0] = ARBITRARY : bit; end;
        EDSCR().RW = RW;
    end;
    return;
end;
```

Library pseudocode for shared/debug/haltingevents

```
var EDBGRQ : Signal;  // Input Signal in external debug interface
```

Library pseudocode for shared/debug/haltingevents/CheckEDBGRQ

```
// CheckEDBGRQ()
// ============
// Checks IMPLEMENTATION DEFINED EDBGRQ Input Signal of external debug interface.
// This is an example of an IMPLEMENTATION DEFINED source of External Debug Request debug events.

func CheckEDBGRQ()
begin
    if EDBGRQ == HIGH then
        ExternalDebugRequest();
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckExceptionCatch

```
// CheckExceptionCatch()
// =====================
// Check whether an Exception Catch debug event is set on the current Exception level

func CheckExceptionCatch(exception_entry : boolean)
begin
    // Called after an exception entry or exit, that is, such that the Security state
    // and PSTATE.EL are correct for the exception target. When FEAT_Debugv8p2
    // is not implemented, this function might also be called at any time.
    let ss : SecurityState = SecurityStateAtEL(PSTATE.EL);
    var base : integer;

    case ss of
        when SS_Secure    => base = 0;
        when SS_NonSecure => base = 4;
        when SS_Realm     => base = 16;
        when SS_Root      => base = 0;
    end;
    if HaltingAllowed() then
        var halt : boolean;
        if IsFeatureImplemented(FEAT_Debugv8p2) then
            let exception_exit : boolean = !exception_entry;
            let increment : integer{} = if ss == SS_Realm then 4 else 8;
            let ctrl : bits(2) = (EDECCR()[UInt(PSTATE.EL) + base + increment]::
                                  EDECCR()[UInt(PSTATE.EL) + base]);
            case ctrl of
                when '00' =>  halt = FALSE;
                when '01' =>  halt = TRUE;
                when '10' =>  halt = (exception_exit == TRUE);
                when '11' =>  halt = (exception_entry == TRUE);
            end;
        else
            halt = (EDECCR()[UInt(PSTATE.EL) + base] == '1');
        end;

        if halt then
            if IsFeatureImplemented(FEAT_Debugv8p8) && exception_entry then
                EDESR().EC = '1';
            else
                Halt(DebugHalt_ExceptionCatch);
            end;
        end;
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckExternalDebugRequestEvents

```
// CheckExternalDebugRequestEvents()
// =================================
// Checks for all External Debug Request debug events.

func CheckExternalDebugRequestEvents()
begin
    CheckTRBEHalt();
    CheckPMUHalt();
    CheckEDBGRQ();
    return;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckHaltingStep

```
// CheckHaltingStep()
// ==================
// Check whether EDESR.SS has been set by Halting Step

func CheckHaltingStep(is_async : boolean)
begin
    let step_enabled : boolean = EDECR().SS == '1' && HaltingAllowed();
    let active_pending : boolean = step_enabled && EDESR().SS == '1';
    if active_pending then
        let fault : FaultRecord = NoFault();
        Halt(DebugHalt_Step_NoSyndrome, is_async, fault);
    end;
    if step_enabled then ShouldAdvanceHS = TRUE; end;
    return;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckOSUnlockCatch

```
// CheckOSUnlockCatch()
// ====================
// Called on unlocking the OS Lock to pend an OS Unlock Catch debug event

func CheckOSUnlockCatch()
begin
    if ((IsFeatureImplemented(FEAT_DoPD) && CTIDEVCTL().OSUCE == '1') ||
          (!IsFeatureImplemented(FEAT_DoPD) && EDECR().OSUCE == '1')) then
        if !Halted() then EDESR().OSUC = '1'; end;
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckPMUHalt

```
// CheckPMUHalt()
// ==============

func CheckPMUHalt()
begin
    if !IsFeatureImplemented(FEAT_Debugv8p9) || !IsFeatureImplemented(FEAT_PMUv3p9) then
        return;
    end;

    // The request remains set until the condition is cleared.
    // For example, an interrupt handler or cross-triggered event handler clears
    // the overflow status flag by writing to PMOVSCLR_EL0

    let include_r1 : boolean = TRUE;
    let include_r2 : boolean = TRUE;
    let include_r3 : boolean = TRUE;

    let pmuhalt : boolean = CheckPMUOverflowCondition(PMUOverflowCondition_EDBGRQ,
                                                      include_r1, include_r2, include_r3);
    if pmuhalt && EDECR().PME == '1' then
        ExternalDebugRequest();
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckPendingExceptionCatch

```
// CheckPendingExceptionCatch()
// ============================
// Check whether EDESR.EC has been set by an Exception Catch debug event.

func CheckPendingExceptionCatch(is_async : boolean)
begin
    if IsFeatureImplemented(FEAT_Debugv8p8) && HaltingAllowed() && EDESR().EC == '1' then
        let fault : FaultRecord = NoFault();
        Halt(DebugHalt_ExceptionCatch, is_async, fault);
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckPendingOSUnlockCatch

```
// CheckPendingOSUnlockCatch()
// ===========================
// Check whether EDESR.OSUC has been set by an OS Unlock Catch debug event

func CheckPendingOSUnlockCatch()
begin
    if HaltingAllowed() && EDESR().OSUC == '1' then
        let is_async : boolean = TRUE;
        let fault : FaultRecord = NoFault();
        Halt(DebugHalt_OSUnlockCatch, is_async, fault);
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckPendingResetCatch

```
// CheckPendingResetCatch()
// ========================
// Check whether EDESR.RC has been set by a Reset Catch debug event

func CheckPendingResetCatch()
begin
    if HaltingAllowed() && EDESR().RC == '1' then
        let is_async : boolean = TRUE;
        let fault : FaultRecord = NoFault();
        Halt(DebugHalt_ResetCatch, is_async, fault);
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckResetCatch

```
// CheckResetCatch()
// =================
// Called after reset

func CheckResetCatch()
begin
    if ((IsFeatureImplemented(FEAT_DoPD) && CTIDEVCTL().RCE == '1') ||
          (!IsFeatureImplemented(FEAT_DoPD) && EDECR().RCE == '1')) then
        EDESR().RC = '1';
        // If halting is allowed then halt immediately
        if HaltingAllowed() then Halt(DebugHalt_ResetCatch); end;
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckSoftwareAccessToDebugRegisters

```
// CheckSoftwareAccessToDebugRegisters()
// =====================================
// Check for access to Breakpoint and Watchpoint registers.

func CheckSoftwareAccessToDebugRegisters()
begin
    let os_lock : bit = (if ELUsingAArch32(EL1) then DBGOSLSR().OSLK else OSLSR_EL1().OSLK);
    if HaltingAllowed() && EDSCR().TDA == '1' && os_lock == '0' then
        Halt(DebugHalt_SoftwareAccess);
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/CheckTRBEHalt

```
// CheckTRBEHalt()
// ===============

func CheckTRBEHalt()
begin
    if !IsFeatureImplemented(FEAT_Debugv8p9) || !IsFeatureImplemented(FEAT_TRBE_EXT) then
        return;
    end;

    if TraceBufferEnabled() && TRBSR_EL1().IRQ == '1' && EDECR().TRBE == '1' then
        ExternalDebugRequest();
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/ExternalDebugRequest

```
// ExternalDebugRequest()
// ======================

func ExternalDebugRequest()
begin
    if HaltingAllowed() then
        let is_async : boolean = TRUE;
        let fault : FaultRecord = NoFault();
        Halt(DebugHalt_EDBGRQ, is_async, fault);
    // Otherwise the CTI continues to assert the debug request until it is taken.
    end;
end;
```

Library pseudocode for shared/debug/haltingevents/HSAdvance

```
// HSAdvance()
// ===========
// Advance the Halting Step State Machine

func HSAdvance()
begin
    if !ShouldAdvanceHS then return; end;
    let step_enabled : boolean = EDECR().SS == '1' && HaltingAllowed();
    let active_not_pending : boolean = step_enabled && EDESR().SS == '0';
    if active_not_pending then EDESR().SS = '1'; end;  // set as pending.
    ShouldAdvanceHS = FALSE;
    return;
end;
```

Library pseudocode for shared/debug/interrupts/ExternalDebugInterruptsDisabled

```
// ExternalDebugInterruptsDisabled()
// =================================
// Determine whether EDSCR disables interrupts routed to 'target'.

func ExternalDebugInterruptsDisabled(target : bits(2)) => boolean
begin
    var int_dis : boolean;
    let ss : SecurityState = SecurityStateAtEL(target);
    if IsFeatureImplemented(FEAT_Debugv8p4) then
        if EDSCR().INTdis[0] == '1' then
            case ss of
                when SS_NonSecure => int_dis = ExternalInvasiveDebugEnabled();
                when SS_Secure =>    int_dis = ExternalSecureInvasiveDebugEnabled();
                when SS_Realm =>     int_dis = ExternalRealmInvasiveDebugEnabled();
                when SS_Root =>      int_dis = ExternalRootInvasiveDebugEnabled();
            end;
        else
            int_dis = FALSE;
        end;
    else
        case target of
            when EL3 =>
                int_dis = (EDSCR().INTdis == '11' && ExternalSecureInvasiveDebugEnabled());
            when EL2 =>
                int_dis = (EDSCR().INTdis == '1x' && ExternalInvasiveDebugEnabled());
            when EL1 =>
                if ss == SS_Secure then
                    int_dis = (EDSCR().INTdis == '1x' && ExternalSecureInvasiveDebugEnabled());
                else
                    int_dis = (EDSCR().INTdis != '00' && ExternalInvasiveDebugEnabled());
                end;
        end;
    end;
    return int_dis;
end;
```

Library pseudocode for shared/debug/pmu

```
var PMUEventAccumulator : array [[31]] of integer;   // Accumulates PMU events for a cycle
var PMULastThresholdValue : array [[31]] of boolean; // A record of the threshold result for each
                                                     // event counter on the previous cycle

// Constant used in PMU functions to represent actions on the cycle counter.
constant CYCLE_COUNTER_ID : integer = 31;
// Constant used in PMU functions to represent actions on the instruction counter.
constant INSTRUCTION_COUNTER_ID : integer = 32;
```

Library pseudocode for shared/debug/pmu/ApplyPMUZeroMask

```
// ApplyPMUZeroMask()
// ==================
// Zero the set of counters specified by the mask in 'masked_val'.
// 'masked_val' is expected to already be masked for the permitted counters for the current
// access type before this function is called.

func ApplyPMUZeroMask(masked_val : bits(64))
begin
    for idx = 0 to 63 do
        if masked_val[idx] == '1' then
            case idx of
                when INSTRUCTION_COUNTER_ID =>
                    PMICNTR_EL0() = Zeros{64};
                when CYCLE_COUNTER_ID =>
                    if !HaveAArch64() then
                        PMCCNTR() = Zeros{64};
                    else
                        PMCCNTR_EL0() = Zeros{64};
                    end;
                otherwise =>
                    if !HaveAArch64() then
                        PMEVCNTR(idx) = Zeros{32};
                    elsif IsFeatureImplemented(FEAT_PMUv3p5) then
                        PMEVCNTR_EL0(idx) = Zeros{64};
                    else
                        PMEVCNTR_EL0(idx)[31:0] = Zeros{32};
                    end;
            end;
        end;
    end;

    return;
end;
```

Library pseudocode for shared/debug/pmu/CheckForPMUOverflow

```
// CheckForPMUOverflow()
// =====================
// Called before each instruction is executed.
// If a PMU event counter has overflowed, this function might do any of:
//  - Signal a Performance Monitors overflow interrupt request.
//  - Signal a CTI Performance Monitors overflow event.
//  - Generate an External Debug Request debug event.
//  - Generate a BRBE freeze event.

func CheckForPMUOverflow()
begin
    let include_r1 : boolean = TRUE;
    let include_r2 : boolean = TRUE;
    let include_r3 : boolean = TRUE;

    let enabled : boolean = PMUInterruptEnabled();
    let pmuirq : boolean = CheckPMUOverflowCondition(PMUOverflowCondition_IRQ,
                                                     include_r1, include_r2, include_r3);

    SetInterruptRequestLevel(InterruptID_PMUIRQ,
                             if enabled && pmuirq then HIGH else LOW);
    CTI_SetEventLevel(CrossTriggerIn_PMUOverflow,
                      if pmuirq then HIGH else LOW);

    if ShouldBRBEFreeze() then
        BRBEFreeze();
    end;

    return;
end;
```

Library pseudocode for shared/debug/pmu/CheckPMUOverflowCondition

```
// CheckPMUOverflowCondition()
// ===========================
// Checks for PMU overflow under certain parameter conditions described by 'reason'.
// If 'include_r1' is TRUE, then check counters in the range [0..(HPMN-1)], CCNTR
//     and ICNTR, unless excluded by 'reason'.
// If 'include_r2' is TRUE, then check counters in the range [HPMN..(EPMN-1)].
// If 'include_r3' is TRUE, then check counters in the range [EPMN..(N-1)].

func CheckPMUOverflowCondition(reason : PMUOverflowCondition,
                               include_r1 : boolean, include_r2 : boolean, include_r3 : boolean
                              ) => boolean
begin
    // 'reason' is decoded into a further set of parameters:
    // If 'check_e' is TRUE, then check the applicable one of PMCR_EL0.E and MDCR_EL2.HPME.
    // If 'check_inten' is TRUE, then check the applicable PMINTENCLR_EL1 bit.
    // If 'exclude_cyc' is TRUE, then CCNTR is NOT checked.
    var check_e : boolean;
    var check_inten : boolean;
    var exclude_cyc : boolean;
    case reason of
        when PMUOverflowCondition_PMUException =>
            check_e      = TRUE;
            check_inten  = TRUE;
            exclude_cyc  = FALSE;
        when PMUOverflowCondition_BRBEFreeze =>
            check_e      = FALSE;
            check_inten  = FALSE;
            exclude_cyc  = TRUE;
        when PMUOverflowCondition_Freeze =>
            check_e      = FALSE;
            check_inten  = FALSE;
            exclude_cyc  = TRUE;
        when PMUOverflowCondition_IRQ, PMUOverflowCondition_EDBGRQ =>
            check_e      = TRUE;
            check_inten  = TRUE;
            exclude_cyc  = FALSE;
        otherwise =>
            unreachable;
    end;

    var ovsf : bits(64);

    if HaveAArch64() then
        ovsf = PMOVSSET_EL0();
        ovsf[63:33] = Zeros{31};
        if !IsFeatureImplemented(FEAT_PMUv3_ICNTR) then
            ovsf[INSTRUCTION_COUNTER_ID] = '0';
        end;
    else
        ovsf = ZeroExtend{64}(PMOVSSET());
    end;

    let counters : integer{} = NUM_PMU_COUNTERS;
    // Remove unimplemented counters - these fields are RES0
    if counters < 31 then
        ovsf[30:counters] = Zeros{31-counters};
    end;

    for idx = 0 to counters - 1 do
        var global_en : bit;
        case GetPMUCounterRange(idx) of
            when PMUCounterRange_R1 =>
                global_en = if HaveAArch64() then PMCR_EL0().E else PMCR().E;
                if !include_r1 then
                    ovsf[idx] = '0';
                end;
            when PMUCounterRange_R2 =>
                global_en = if HaveAArch64() then MDCR_EL2().HPME else HDCR().HPME;
                if !include_r2 then
                    ovsf[idx] = '0';
                end;
            when PMUCounterRange_R3 =>
                global_en = PMCCR().EPME;
                if !include_r3 then
                    ovsf[idx] = '0';
                end;
            otherwise =>
                unreachable;
        end;
        if check_e then
            ovsf[idx] = ovsf[idx] AND global_en;
        end;
    end;

    // Cycle counter
    if exclude_cyc || !include_r1 then
        ovsf[CYCLE_COUNTER_ID] = '0';
    end;
```

```
    if check_e then
        ovsf[CYCLE_COUNTER_ID] = ovsf[CYCLE_COUNTER_ID] AND PMCR_EL0().E;
    end;

    // Instruction counter
    if HaveAArch64() && IsFeatureImplemented(FEAT_PMUv3_ICNTR) then
        if !include_r1 then
            ovsf[INSTRUCTION_COUNTER_ID] = '0';
        end;
        if check_e then
            ovsf[INSTRUCTION_COUNTER_ID] = ovsf[INSTRUCTION_COUNTER_ID] AND PMCR_EL0().E;
        end;
    end;

    if check_inten then
        let inten : bits(64) = (if HaveAArch64() then PMINTENCLR_EL1()
                                else ZeroExtend{64}(PMINTENCLR()));
        ovsf = ovsf AND inten;
    end;

    return !IsZero(ovsf);
end;
```

Library pseudocode for shared/debug/pmu/CountPMUEvents

```
// CountPMUEvents()
// ================
// Return TRUE if counter "idx" should count its event.
// For the cycle counter, idx == CYCLE_COUNTER_ID (31).
// For the instruction counter, idx == INSTRUCTION_COUNTER_ID (32).

func CountPMUEvents(idx : integer) => boolean
begin
    let counters : integer = NUM_PMU_COUNTERS;
    assert (idx == CYCLE_COUNTER_ID || idx < counters ||
            (idx == INSTRUCTION_COUNTER_ID && IsFeatureImplemented(FEAT_PMUv3_ICNTR)));

    var debug : boolean;
    var enabled : boolean;
    var prohibited : boolean;
    var filtered : boolean;
    var frozen : boolean;

    // Event counting is disabled in Debug state
    debug = Halted();

    // Software can reserve some counters
    let counter_range : PMUCounterRange = GetPMUCounterRange(idx);
    var ss : SecurityState = CurrentSecurityState();

    // Main enable controls
    var global_en : bit;
    var counter_en : bit;
    case counter_range of
        when PMUCounterRange_R1 =>
            global_en = if HaveAArch64() then PMCR_EL0().E else PMCR().E;
        when PMUCounterRange_R2 =>
            global_en = if HaveAArch64() then MDCR_EL2().HPME else HDCR().HPME;
        when PMUCounterRange_R3 =>
            assert IsFeatureImplemented(FEAT_PMUv3_EXTPMN);
            global_en = PMCCR().EPME;
        otherwise =>
            unreachable;
    end;

    case idx of
        when INSTRUCTION_COUNTER_ID =>
            assert HaveAArch64();
            counter_en = PMCNTENSET_EL0().F0;
        when CYCLE_COUNTER_ID =>
            counter_en = if HaveAArch64() then PMCNTENSET_EL0().C else PMCNTENSET().C;
        otherwise =>
            counter_en = if HaveAArch64() then PMCNTENSET_EL0()[idx] else PMCNTENSET()[idx];
    end;

    enabled = global_en == '1' && counter_en == '1';

    // Event counting is allowed unless it is prohibited by any rule below
    prohibited = FALSE;

    // Event counting in Secure state or at EL3 is prohibited if all of:
    // * EL3 is implemented
    // * One of the following is true:
    //   - EL3 is using AArch64, MDCR_EL3.SPME == 0, and either:
    //     - FEAT_PMUv3p7 is not implemented
    //     - MDCR_EL3.MPMX == 0
    //   - EL3 is using AArch32 and SDCR.SPME == 0
    // * Either not executing at EL0 using AArch32, or one of the following is true:
    //     - EL3 is using AArch32 and SDER.SUNIDEN == 0
    //     - EL3 is using AArch64, EL1 is using AArch32, and SDER32_EL3.SUNIDEN == 0
    // * PMNx is not reserved for use by the external interface
    if (HaveEL(EL3) && (ss == SS_Secure || PSTATE.EL == EL3) &&
          counter_range != PMUCounterRange_R3) then
        if !ELUsingAArch32(EL3) then
            prohibited = (MDCR_EL3().SPME == '0' &&
                          (!IsFeatureImplemented(FEAT_PMUv3p7) || MDCR_EL3().MPMX == '0'));
        else
            prohibited = SDCR().SPME == '0';
        end;

        if prohibited && PSTATE.EL == EL0 then
            if ELUsingAArch32(EL3) then
                prohibited = SDER().SUNIDEN == '0';
            elsif ELUsingAArch32(EL1) then
                prohibited = SDER32_EL3().SUNIDEN == '0';
            end;
        end;
    end;

    // Event counting at EL3 is prohibited if all of:
    // * FEAT_PMUv3p7 is implemented
    // * EL3 is using AArch64
    // * One of the following is true:
    //   - MDCR_EL3.SPME == 0
```

```
//   - PMNx is not reserved for EL2
// * MDCR_EL3.MPMX == 1
// * PMNx is not reserved for use by the external interface
if (!prohibited && IsFeatureImplemented(FEAT_PMUv3p7) && PSTATE.EL == EL3 &&
      HaveAArch64() && counter_range != PMUCounterRange_R3) then
    prohibited = (MDCR_EL3().MPMX == '1' &&
                  (MDCR_EL3().SPME == '0' || counter_range == PMUCounterRange_R1));
end;

// Event counting at EL2 is prohibited if all of:
// * FEAT_PMUv3p1 is implemented
// * PMNx is not reserved for EL2 or the external interface
// * EL2 is using AArch64 and MDCR_EL2.HPMD == 1, or EL2 is using AArch32 and HDCR.HPMD == 1
if (!prohibited && PSTATE.EL == EL2 && IsFeatureImplemented(FEAT_PMUv3p1) &&
      counter_range == PMUCounterRange_R1) then
    let hpmd : bit = if HaveAArch64() then MDCR_EL2().HPMD else HDCR().HPMD;
    prohibited = hpmd == '1';
end;

// The IMPLEMENTATION DEFINED authentication interface might override software
if prohibited && !IsFeatureImplemented(FEAT_Debugv8p2) then
    prohibited = !ExternalSecureNoninvasiveDebugEnabled();
end;

// If FEAT_PMUv3p7 is implemented, event counting can be frozen
if IsFeatureImplemented(FEAT_PMUv3p7) then
    var fz : bit;
    case counter_range of
        when PMUCounterRange_R1 =>
            fz = if HaveAArch64() then PMCR_EL0().FZO else PMCR().FZO;
        when PMUCounterRange_R2 =>
            fz = if HaveAArch64() then MDCR_EL2().HPMFZO else HDCR().HPMFZO;
        when PMUCounterRange_R3 =>
            fz = '0';
        otherwise =>
            unreachable;
    end;
    frozen = (fz == '1') && ShouldPMUFreeze(counter_range);
    frozen = frozen || SPEFreezeOnEvent(idx);
else
    frozen = FALSE;
end;

// PMCR_EL0.DP or PMCR.DP disables the cycle counter when event counting is prohibited
// or frozen
if (prohibited || frozen) && idx == CYCLE_COUNTER_ID then
    let dp : bit = if HaveAArch64() then PMCR_EL0().DP else PMCR().DP;
    enabled = enabled && dp == '0';
    // Otherwise whether event counting is prohibited or frozen does not affect the cycle
    // counter
    prohibited = FALSE;
    frozen = FALSE;
end;

// If FEAT_PMUv3p5 is implemented, cycle counting can be prohibited.
// This is not overridden by PMCR_EL0.DP.
if IsFeatureImplemented(FEAT_PMUv3p5) && idx == CYCLE_COUNTER_ID then
    if HaveEL(EL3) && (ss == SS_Secure || PSTATE.EL == EL3) then
        let sccd = if HaveAArch64() then MDCR_EL3().SCCD else SDCR().SCCD;
        if sccd == '1' then
            prohibited = TRUE;
        end;
    end;

    if PSTATE.EL == EL2 then
        let hccd = if HaveAArch64() then MDCR_EL2().HCCD else HDCR().HCCD;
        if hccd == '1' then
            prohibited = TRUE;
        end;
    end;
end;

// If FEAT_PMUv3p7 is implemented, cycle counting an be prohibited at EL3.
// This is not overridden by PMCR_EL0.DP.
if IsFeatureImplemented(FEAT_PMUv3p7) && idx == CYCLE_COUNTER_ID then
    if PSTATE.EL == EL3 && HaveAArch64() && MDCR_EL3().MCCD == '1' then
        prohibited = TRUE;
    end;
end;

// Event counting can be filtered by the {P, U, NSK, NSU, NSH, M, SH, RLK, RLU, RLH} bits
var filter : bits(32);
case idx of
    when INSTRUCTION_COUNTER_ID =>
        filter = PMICFILTR_EL0()[31:0];
    when CYCLE_COUNTER_ID =>
        filter = if HaveAArch64() then PMCCFILTR_EL0()[31:0] else PMCCFILTR();
    otherwise =>
        filter = if HaveAArch64() then PMEVTYPER_EL0(idx)[31:0] else PMEVTYPER(idx);
```

```
    end;

    let p : bit   = filter[31];
    let u : bit   = filter[30];
    let nsk : bit = if HaveEL(EL3) then filter[29] else '0';
    let nsu : bit = if HaveEL(EL3) then filter[28] else '0';
    let nsh : bit = if HaveEL(EL2) then filter[27] else '0';
    let m : bit   = if HaveEL(EL3) && HaveAArch64() then filter[26] else '0';
    let sh : bit  = if HaveEL(EL3) && IsFeatureImplemented(FEAT_SEL2) then filter[24] else '0';
    let rlk : bit = if IsFeatureImplemented(FEAT_RME) then filter[22] else '0';
    let rlu : bit = if IsFeatureImplemented(FEAT_RME) then filter[21] else '0';
    let rlh : bit = if IsFeatureImplemented(FEAT_RME) then filter[20] else '0';

    ss = CurrentSecurityState();
    case PSTATE.EL of
        when EL0 =>
            case ss of
                when SS_NonSecure => filtered = u != nsu;
                when SS_Secure =>    filtered = u == '1';
                when SS_Realm =>     filtered = u != rlu;
            end;
        when EL1 =>
            case ss of
                when SS_NonSecure => filtered = p != nsk;
                when SS_Secure =>    filtered = p == '1';
                when SS_Realm =>     filtered = p != rlk;
            end;
        when EL2 =>
            case ss of
                when SS_NonSecure => filtered = nsh == '0';
                when SS_Secure =>    filtered = nsh == sh;
                when SS_Realm =>     filtered = nsh == rlh;
            end;
        when EL3 =>
            if HaveAArch64() then
                filtered = m != p;
            else
                filtered = p == '1';
            end;
    end;

    if IsFeatureImplemented(FEAT_PMUv3_SME) then
        let is_streaming_mode : boolean = PSTATE.SM == '1';
        var vs : bits(2);
        case idx of
            when INSTRUCTION_COUNTER_ID =>
                vs = PMICFILTR_EL0().VS;
            when CYCLE_COUNTER_ID =>
                vs = PMCCFILTR_EL0().VS;
            otherwise =>
                vs = PMEVTYPER_EL0(idx).VS;
        end;

        var streaming_mode_filtered : boolean;
        if vs == '11' then
            streaming_mode_filtered = ConstrainUnpredictableBool(Unpredictable_RES_PMU_VS);
        else
            streaming_mode_filtered =  ((is_streaming_mode && vs[0] == '1') ||
                                        (!is_streaming_mode && vs[1] == '1'));
        end;

        filtered = filtered || streaming_mode_filtered;
    end;

    return !debug && enabled && !prohibited && !filtered && !frozen;
end;
```

Library pseudocode for shared/debug/pmu/EffectiveEPMN

```
// EffectiveEPMN()
// ===============
// Returns the Effective value of PMCCR.EPMN.

func EffectiveEPMN() => bits(5)
begin
    let counters : integer = NUM_PMU_COUNTERS;
    var epmn_bits : bits(5);

    if IsFeatureImplemented(FEAT_PMUv3_EXTPMN) then
        epmn_bits = PMCCR().EPMN;
        if UInt(epmn_bits) > counters then
            (-, epmn_bits) = ConstrainUnpredictableBits{5}(Unpredictable_RES_EPMN);
        end;
    else
        epmn_bits = counters[4:0];
    end;

    return epmn_bits;
end;
```

Library pseudocode for shared/debug/pmu/EffectiveHPMN

```
// EffectiveHPMN()
// ===============
// Returns the Effective value of MDCR_EL2.HPMN or HDCR.HPMN.

func EffectiveHPMN() => bits(5)
begin
    let counters : integer = UInt(EffectiveEPMN());
    var hpmn_bits : bits(5);

    if HaveEL(EL2) then     // Software can reserve some event counters for EL2
        hpmn_bits = if HaveAArch64() then MDCR_EL2().HPMN else HDCR().HPMN;

        // When FEAT_PMUv3_EXTPMN is implemented, out of range values are capped.
        if UInt(hpmn_bits) > counters && IsFeatureImplemented(FEAT_PMUv3_EXTPMN) then
            hpmn_bits = counters[4:0];
        end;

        if (UInt(hpmn_bits) > counters ||
              (!IsFeatureImplemented(FEAT_HPMN0) && IsZero(hpmn_bits))) then
            (-, hpmn_bits) = ConstrainUnpredictableBits{5}(Unpredictable_RES_HPMN);
        end;
    else
        hpmn_bits = counters[4:0];
    end;

    return hpmn_bits;
end;
```

Library pseudocode for shared/debug/pmu/ExtRegGetPMUAccessMask

```
// ExtRegGetPMUAccessMask()
// ========================
// Return a mask of the PMU counters accessible via external PMU access.

func ExtRegGetPMUAccessMask(addrdesc : AddressDescriptor) => bits(64)
begin
    var mask : bits(64) = Zeros{};

    if IsFeatureImplemented(FEAT_PMUv3_ICNTR) then
        mask[INSTRUCTION_COUNTER_ID] = '1';
    end;

    mask[CYCLE_COUNTER_ID] = '1';

    let counters : integer{} = (if (IsFeatureImplemented(FEAT_PMUv3_EXTPMN) &&
                                    !IsMostSecureAccess(addrdesc)) then UInt(EffectiveEPMN())
                                else NUM_PMU_COUNTERS);

    if counters > 0 then
        mask[counters-1:0] = Ones{counters};
    end;

    return mask;
end;
```

Library pseudocode for shared/debug/pmu/ExtRegZeroCycleCounter

```
// ExtRegZeroCycleCounter()
// ========================
// Called on an external PMU write to PMCR_EL0 that writes '1' to PMCR_EL0.C.

func ExtRegZeroCycleCounter(addrdesc : AddressDescriptor)
begin
    var mask : bits(64) = Zeros{};
    mask[CYCLE_COUNTER_ID] = '1';
    ExtRegZeroPMUCounters(mask, addrdesc);
end;
```

Library pseudocode for shared/debug/pmu/ExtRegZeroEventCounters

```
// ExtRegZeroEventCounters()
// =========================
// Zero all the event counters.
// Called on an external PMU write to PMCR_EL0 that writes '1' to PMCR_EL0.P.

func ExtRegZeroEventCounters(addrdesc : AddressDescriptor)
begin
    ExtRegZeroPMUCounters(Zeros{33} :: Ones{31}, addrdesc);
end;
```

Library pseudocode for shared/debug/pmu/ExtRegZeroPMUCounters

```
// ExtRegZeroPMUCounters()
// =======================
// Zero set of counters specified by the mask in 'val', for external PMU access.

func ExtRegZeroPMUCounters(val : bits(64), addrdesc : AddressDescriptor)
begin
    let masked_val : bits(64) = val AND ExtRegGetPMUAccessMask(addrdesc);
    ApplyPMUZeroMask(masked_val);
    return;
end;
```

Library pseudocode for shared/debug/pmu/GetNumEventCountersAccessible

```
// GetNumEventCountersAccessible()
// ===============================
// Return the number of event counters that can be accessed at the current Exception level.

func GetNumEventCountersAccessible() => integer{0..NUM_PMU_COUNTERS}
begin
    var n : integer;

    // Software can reserve some counters for EL2
    if PSTATE.EL IN {EL1, EL0} && EL2Enabled() then
        n = UInt(EffectiveHPMN());
    else
        n = UInt(EffectiveEPMN());
    end;

    return n as integer{0..NUM_PMU_COUNTERS};
end;
```

Library pseudocode for shared/debug/pmu/GetNumEventCountersSelfHosted

```
// GetNumEventCountersSelfHosted()
// ===============================
// Return the number of event counters that can be accessed by the Self-hosted software.

func GetNumEventCountersSelfHosted() => integer
begin
    if IsFeatureImplemented(FEAT_PMUv3_EXTPMN) then
        return UInt(EffectiveEPMN());
    else
        return NUM_PMU_COUNTERS;
    end;
end;
```

Library pseudocode for shared/debug/pmu/GetPMUCounterRange

```
// GetPMUCounterRange()
// ====================
// Returns the range that a counter is currently in.

func GetPMUCounterRange(n : integer) => PMUCounterRange
begin

    let counters : integer = NUM_PMU_COUNTERS;
    let epmn : integer = UInt(EffectiveEPMN());
    let hpmn : integer = UInt(EffectiveHPMN());

    if n < hpmn then
        return PMUCounterRange_R1;
    elsif n < epmn then
        return PMUCounterRange_R2;
    elsif n < counters then
        assert IsFeatureImplemented(FEAT_PMUv3_EXTPMN);
        return PMUCounterRange_R3;
    elsif n == CYCLE_COUNTER_ID then
        return PMUCounterRange_R1;
    elsif n == INSTRUCTION_COUNTER_ID then
        assert IsFeatureImplemented(FEAT_PMUv3_ICNTR);
        return PMUCounterRange_R1;
    else
        unreachable;
    end;
end;
```

Library pseudocode for shared/debug/pmu/GetPMUReadMask

```
// GetPMUReadMask()
// ================
// Return a mask of the PMU counters that can be read at the current
// Exception level.
// This mask masks reads from PMCNTENSET_EL0, PMCNTENCLR_EL0, PMINTENSET_EL1,
// PMINTENCLR_EL1, PMOVSSET_EL0, and PMOVSCLR_EL0.

func GetPMUReadMask() => bits(64)
begin
    var mask : bits(64) = SysRegGetPMUAccessMask();

    // Additional PMICNTR_EL0 accessibility checks. PMICNTR_EL0 controls read-as-zero
    // if a read of PMICFILTR_EL0 would be trapped to a higher Exception level.
    if IsFeatureImplemented(FEAT_PMUv3_ICNTR) && mask[INSTRUCTION_COUNTER_ID] == '1' then
        // Check for trap to EL3.
        if HaveEL(EL3) && PSTATE.EL != EL3 && MDCR_EL3().EnPM2 == '0' then
            mask[INSTRUCTION_COUNTER_ID] = '0';
        end;

        // Check for trap to EL2.
        if EL2Enabled() && PSTATE.EL IN {EL0, EL1} && !ELIsInHost(EL0) then
            // If FEAT_PMUv3_ICNTR and EL2 are implemented, then so is FEAT_FGT2.
            assert IsFeatureImplemented(FEAT_FGT2);
            if ((HaveEL(EL3) && SCR_EL3().FGTEn2 == '0') ||
                  HDFGRTR2_EL2().nPMICFILTR_EL0 == '0') then
                mask[INSTRUCTION_COUNTER_ID] = '0';
            end;
        end;
    end;

    // Traps on other counters do not affect those counters' controls in the same way.

    return mask;
end;
```

Library pseudocode for shared/debug/pmu/GetPMUWriteMask

```
// GetPMUWriteMask()
// =================
// Return a mask of the PMU counters writable at the current Exception level.
// This mask masks writes to PMCNTENSET_EL0, PMCNTENCLR_EL0, PMINTENSET_EL1,
// PMINTENCLR_EL1, PMOVSSET_EL0, PMOVSCLR_EL0, and PMZR_EL0.
// 'write_counter' is TRUE for a write to PMZR_EL0, when the counter is being
// updated, and FALSE for other cases when the controls are being updated.

func GetPMUWriteMask(write_counter : boolean) => bits(64)
begin
    var mask : bits(64) = SysRegGetPMUAccessMask();

    // Check EL0 ignore write conditions
    if (IsFeatureImplemented(FEAT_PMUv3p9) && !ELUsingAArch32(EL1) &&
          PSTATE.EL == EL0 && PMUSERENR_EL0().UEN == '1') then
        if (IsFeatureImplemented(FEAT_PMUv3_ICNTR) &&
              PMUSERENR_EL0().IR == '1') then          // PMICNTR_EL0 read-only
            mask[INSTRUCTION_COUNTER_ID] = '0';
        end;
        if PMUSERENR_EL0().CR == '1' then              // PMCCNTR_EL0 read-only
            mask[CYCLE_COUNTER_ID] = '0';
        end;
        if PMUSERENR_EL0().ER == '1' then              // PMEVCNTR[n]_EL0 read-only
            mask[30:0] = Zeros{31};
        end;
    end;

    // Additional PMICNTR_EL0 accessibility checks. PMICNTR_EL0 controls ignore writes
    // if a write of PMICFILTR_EL0 would be trapped to a higher Exception level.
    // Indirect writes to PMICNTR_EL0 (through PMZR_EL0) are ignored if a write of
    // PMICNTR_EL0 would be trapped to a higher Exception level.
    if IsFeatureImplemented(FEAT_PMUv3_ICNTR) && mask[INSTRUCTION_COUNTER_ID] == '1' then
        // Check for trap to EL3.
        if HaveEL(EL3) && PSTATE.EL != EL3 && MDCR_EL3().EnPM2 == '0' then
            mask[INSTRUCTION_COUNTER_ID] = '0';
        end;

        // Check for trap to EL2.
        if EL2Enabled() && PSTATE.EL IN {EL0, EL1} && !ELIsInHost(EL0) then
            // If FEAT_PMUv3_ICNTR and EL2 are implemented, then so is FEAT_FGT2.
            assert IsFeatureImplemented(FEAT_FGT2);
            let fgt_bit : bit = (if write_counter then HDFGWTR2_EL2().nPMICNTR_EL0
                                 else HDFGWTR2_EL2().nPMICFILTR_EL0);
            if (HaveEL(EL3) && SCR_EL3().FGTEn2 == '0') || fgt_bit == '0' then
                mask[INSTRUCTION_COUNTER_ID] = '0';
            end;
        end;
    end;

    // Traps on other counters do not affect those counters' controls in the same way.

    return mask;
end;
```

Library pseudocode for shared/debug/pmu/HasElapsed64Cycles

```
// HasElapsed64Cycles()
// ====================
// Returns TRUE if 64 cycles have elapsed between the last count, and FALSE otherwise.

impdef func HasElapsed64Cycles() => boolean
begin
    return TRUE;
end;
```

Library pseudocode for shared/debug/pmu/IncrementInstructionCounter

```
// IncrementInstructionCounter()
// =============================
// Increment the instruction counter and possibly set overflow bits.

func IncrementInstructionCounter(increment : integer)
begin
    if CountPMUEvents(INSTRUCTION_COUNTER_ID) then
        let old_value : integer = UInt(PMICNTR_EL0());
        let new_value : integer = old_value + increment;
        PMICNTR_EL0()           = new_value[63:0];

        // The effective value of PMCR_EL0.LP is '1' for the instruction counter
        if old_value[64] != new_value[64] then
            PMOVSSET_EL0().F0 = '1';
        end;
    end;
    return;
end;
```

Library pseudocode for shared/debug/pmu/IsRange3Counter

```
// IsRange3Counter()
// =================
// Returns TRUE if the counter is in the third range.

func IsRange3Counter(n : integer) => boolean
begin
    return PMUCounterRange_R3 == GetPMUCounterRange(n);
end;
```

Library pseudocode for shared/debug/pmu/PMUCaptureEvent

```
// PMUCaptureEvent()
// =================
// If permitted and enabled, generate a PMU snapshot Capture event.

func PMUCaptureEvent()
begin
    assert HaveEL(EL3) && IsFeatureImplemented(FEAT_PMUv3_SS) && HaveAArch64();
    let debug_state : boolean = Halted();

    if !PMUCaptureEventAllowed() then
        // Indicate a Capture event completed, unsuccessfully
        PMSSCR_EL1().[NC,SS] = '10';
        return;
    end;
    let counters : integer = NUM_PMU_COUNTERS;
    for idx = 0 to counters - 1 do
        PMEVCNTSVR_EL1(idx) = PMEVCNTR_EL0(idx) as PMEVCNTSVR_EL1_Type;
    end;
    PMCCNTSVR_EL1() = PMCCNTR_EL0() as PMCCNTSVR_EL1_Type;

    if IsFeatureImplemented(FEAT_PMUv3_ICNTR) then
        PMICNTSVR_EL1() = PMICNTR_EL0() as PMICNTSVR_EL1_Type;
    end;

    if IsFeatureImplemented(FEAT_PCSRv8p9) && PMPCSCTL().SS == '1' then
        if pc_sample.valid && !debug_state then
            SetPCSRActive();
            SetPCSample();
        else
            PMPCSR()[31:0] = Ones{32};
        end;
    end;

    if (IsFeatureImplemented(FEAT_BRBE) && BranchRecordAllowed(PSTATE.EL) &&
          BRBCR_EL1().FZPSS == '1' && (!HaveEL(EL2) || BRBCR_EL2().FZPSS == '1')) then
        BRBEFreeze();
    end;

    // Indicate a successful Capture event
    PMSSCR_EL1().[NC,SS] = '00';
    if !debug_state || ConstrainUnpredictableBool(Unpredictable_PMUSNAPSHOTEVENT) then
        PMUEvent(PMU_EVENT_PMU_SNAPSHOT);
    end;

    return;
end;
```

Library pseudocode for shared/debug/pmu/PMUCaptureEventAllowed

```
// PMUCaptureEventAllowed()
// ========================
// Returns TRUE if PMU Capture events are allowed, and FALSE otherwise.

func PMUCaptureEventAllowed() => boolean
begin
    if !IsFeatureImplemented(FEAT_PMUv3_SS) || !HaveAArch64() then
        return FALSE;
    end;

    if !PMUCaptureEventEnabled() || OSLockStatus() then
        return FALSE;
    elsif HaveEL(EL3) && MDCR_EL3().PMSSE != '01' then
        return MDCR_EL3().PMSSE == '11';
    elsif HaveEL(EL2) && MDCR_EL2().PMSSE != '01' then
        return MDCR_EL2().PMSSE == '11';
    else
        var pmsse_el1 : bits(2) = PMECR_EL1().SSE;
        if pmsse_el1 == '01' then            // Reserved value
            var c : Constraint;
            (c, pmsse_el1) = ConstrainUnpredictableBits{2}(Unpredictable_RESPMSSE);
            assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};
            if c == Constraint_DISABLED then pmsse_el1 = '00'; end;
            // Otherwise the value returned by ConstrainUnpredictableBits must be
            // a non-reserved value
        end;
        return pmsse_el1 == '11';
    end;
end;
```

Library pseudocode for shared/debug/pmu/PMUCaptureEventEnabled

```
// PMUCaptureEventEnabled()
// ========================
// Returns TRUE if PMU Capture events are enabled, and FALSE otherwise.

func PMUCaptureEventEnabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_PMUv3_SS) || !HaveAArch64() then
        return FALSE;
    end;
    if HaveEL(EL3) && MDCR_EL3().PMSSE != '01' then
        return MDCR_EL3().PMSSE == '1x';
    elsif HaveEL(EL2) && ELUsingAArch32(EL2) then
        return FALSE;
    elsif HaveEL(EL2) && MDCR_EL2().PMSSE != '01' then
        return MDCR_EL2().PMSSE == '1x';
    elsif ELUsingAArch32(EL1) then
        return FALSE;
    else
        var pmsse_el1 : bits(2) = PMECR_EL1().SSE;
        if pmsse_el1 == '01' then            // Reserved value
            var c : Constraint;
            (c, pmsse_el1) = ConstrainUnpredictableBits{2}(Unpredictable_RESPMSSE);
            assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};
            if c == Constraint_DISABLED then pmsse_el1 = '00'; end;
        end;
            // Otherwise the value returned by ConstrainUnpredictableBits must be
            // a non-reserved value
        return pmsse_el1 == '1x';
    end;
end;
```

Library pseudocode for shared/debug/pmu/PMUCountValue

```
// PMUCountValue()
// ===============
// Implements the PMU threshold function, if implemented.
// Returns the value to increment event counter 'n' by.
// 'Vb' is the base value of the event that event counter 'n' is configured to count.
// 'Vm' is the value to increment event counter 'n-1' by if 'n' is odd, zero otherwise.

func PMUCountValue(n : integer, Vb : integer, Vm : integer) => integer
begin
    assert (n MOD 2) == 1 || Vm == 0;
    assert n < NUM_PMU_COUNTERS;

    if !IsFeatureImplemented(FEAT_PMUv3_TH) || !HaveAArch64() then
        return Vb;
    end;

    let TH : integer = UInt(PMEVTYPER_EL0(n).TH);

    // Control register fields
    var tc : bits(3) = PMEVTYPER_EL0(n).TC;
    var te : bit = '0';
    if IsFeatureImplemented(FEAT_PMUv3_EDGE) then
        te = PMEVTYPER_EL0(n).TE;
    end;
    var tlc : bits(2) = '00';
    if IsFeatureImplemented(FEAT_PMUv3_TH2) && (n MOD 2) == 1 then
        tlc = PMEVTYPER_EL0(n).TLC;
    end;

    // Check for reserved cases
    var c : Constraint;
    (c, tc, te, tlc) = ReservedPMUThreshold(n, tc, te, tlc);
    if c == Constraint_DISABLED then
        return Vb;
    end;
    // Otherwise the values returned by ReservedPMUThreshold must be defined values

    // Check if disabled. Note that this function will return the value of Vb when
    // the control register fields are all zero, even without this check.
    if tc == '000' && TH == 0 && te == '0' && tlc == '00' then
        return Vb;
    end;

    // Threshold condition
    var Ct : boolean;
    case tc[2:1] of
        when '00' => Ct = (Vb != TH);        // Disabled or not-equal
        when '01' => Ct = (Vb == TH);        // Equals
        when '10' => Ct = (Vb >= TH);        // Greater-than-or-equal
        when '11' => Ct = (Vb <  TH);        // Less-than
    end;

    var Vn : integer;
    if te == '1' then
        // Edge condition
        let Cp : boolean = PMULastThresholdValue[[n]];
        var Ce : boolean;
        var Ve : integer;
        case tc[1:0] of
            when '10' =>  Ce = (Cp != Ct);   // Both edges
            when 'x1' =>  Ce = (!Cp && Ct);  // Single edge
            otherwise =>  unreachable;     // Covered by ReservedPMUThreshold
        end;
        case tlc of
            when '00' =>  Ve = (if Ce then 1 else 0);
            when '10' =>  Ve = (if Ce then Vm else 0);
            otherwise =>  unreachable;     // Covered by ReservedPMUThreshold
        end;
        Vn = Ve;
    else
        // Threshold condition
        var Vt : integer;
        case tc[0]::tlc of
            when '0 00' =>  Vt = (if Ct then Vb else 0);
            when '0 01' =>  Vt = (if Ct then Vb else Vm);
            when '0 10' =>  Vt = (if Ct then Vm else 0);
            when '1 00' =>  Vt = (if Ct then 1 else 0);
            when '1 01' =>  Vt = (if Ct then 1 else Vm);
            otherwise =>    unreachable;   // Covered by ReservedPMUThreshold
        end;
        Vn = Vt;
    end;

    PMULastThresholdValue[[n]] = Ct;

    return Vn;
end;
```

Library pseudocode for shared/debug/pmu/PMUCounterRange

```
// PMUCounterRange
// ===============
// Enumerates the ranges to which an event counter belongs to.

type PMUCounterRange of enumeration {
    PMUCounterRange_R1,
    PMUCounterRange_R2,
    PMUCounterRange_R3
    };
```

Library pseudocode for shared/debug/pmu/PMUEvent

```
// PMUEvent()
// ==========
// Generate a PMU event. By default, increment by 1.
func PMUEvent(pmuevent : bits(16))
begin
    PMUEvent(pmuevent, 1);
end;

// PMUEvent()
// ==========
// Accumulate a PMU Event.

func PMUEvent(pmuevent : bits(16), increment : integer)
begin
    let counters : integer = NUM_PMU_COUNTERS;
    if counters != 0 then
        for idx = 0 to counters - 1 do
            PMUEvent(pmuevent, increment, idx);
        end;
    end;

    if (HaveAArch64() && IsFeatureImplemented(FEAT_PMUv3_ICNTR) &&
          pmuevent == PMU_EVENT_INST_RETIRED) then
        IncrementInstructionCounter(increment);
    end;

end;

// PMUEvent()
// ==========
// Accumulate a PMU Event for a specific event counter.

func PMUEvent(pmuevent : bits(16), increment : integer, idx : integer)
begin
    if !IsFeatureImplemented(FEAT_PMUv3) then
        return;
    end;
    if UsingAArch32() then
        if PMEVTYPER(idx).evtCount == pmuevent then
            PMUEventAccumulator[[idx]] = PMUEventAccumulator[[idx]] + increment;
        end;
    else
        if PMEVTYPER_EL0(idx).evtCount == pmuevent then
            PMUEventAccumulator[[idx]] = PMUEventAccumulator[[idx]] + increment;
        end;
    end;
end;
```

Library pseudocode for shared/debug/pmu/PMUOverflowCondition

```
// PMUOverflowCondition()
// ======================
// Enumerates the reasons for which the PMU overflow condition is evaluated.

type PMUOverflowCondition of enumeration {
    PMUOverflowCondition_PMUException,
    PMUOverflowCondition_BRBEFreeze,
    PMUOverflowCondition_Freeze,
    PMUOverflowCondition_IRQ,
    PMUOverflowCondition_EDBGRQ
};
```

Library pseudocode for shared/debug/pmu/PMUSwIncrement

```
// PMUSwIncrement()
// ================
// Generate PMU Events on a System register write to PMSWINC

func PMUSwIncrement(sw_incr_in : bits(64))
begin

    var sw_incr : bits(64)   = sw_incr_in;
    var mask : bits(31)      = Zeros{};
    let counters : integer{} = GetNumEventCountersAccessible();
    if counters > 0 then
        mask[counters-1:0] = Ones{counters};
    end;

    if (IsFeatureImplemented(FEAT_PMUv3p9) && !ELUsingAArch32(EL1) &&
          PSTATE.EL == EL0 && PMUSERENR_EL0().[UEN,SW] == '10') then
        mask = mask AND PMUACR_EL1()[30:0];
    end;

    sw_incr = sw_incr AND ZeroExtend{64}(mask);
    for idx = 0 to 30 do
        if sw_incr[idx] == '1' then
            PMUEvent(PMU_EVENT_SW_INCR, 1, idx);
        end;
    end;

    return;
end;
```

Library pseudocode for shared/debug/pmu/ReservedPMUThreshold

```
// ReservedPMUThreshold()
// ======================
// Checks if the given PMEVTYPER_EL1().[TH,TE,TLC] values are reserved and will
// generate Constrained Unpredictable behavior, otherwise return Constraint_NONE.

func ReservedPMUThreshold(n : integer, tc_in : bits(3),
                          te_in : bit, tlc_in : bits(2)) => (Constraint, bits(3), bit, bits(2))
begin
    var tc : bits(3) = tc_in;
    var te : bit = te_in;
    var tlc : bits(2) = tlc_in;

    var reserved : boolean = FALSE;

    if IsFeatureImplemented(FEAT_PMUv3_EDGE) then
        if te == '1' && tc[1:0] == '00' then      // Edge condition
            reserved = TRUE;
        end;
    else
        te = '0';                                 // Control is RES0
    end;

    if IsFeatureImplemented(FEAT_PMUv3_TH2) && (n MOD 2) == 1 then
        if tlc == '11' then                       // Reserved value
            reserved = TRUE;
        end;
        if te == '1' then                         // Edge condition
            if tlc == '01' then
                reserved = TRUE;
            end;
        else                                      // Threshold condition
            if tc[0] == '1' && tlc == '10' then
                reserved = TRUE;
            end;
        end;
    else
        tlc = '00';                               // Controls are RES0
    end;

    var c : Constraint = Constraint_NONE;
    if reserved then
        var unpred_reserved_bits : bits(6);
        (c, unpred_reserved_bits) = ConstrainUnpredictableBits{6}(Unpredictable_RESTC);
        tc  = unpred_reserved_bits[5:3];
        te  = unpred_reserved_bits[2];
        tlc = unpred_reserved_bits[1:0];
    end;

    return (c, tc, te, tlc);
end;
```

Library pseudocode for shared/debug/pmu/SMEPMUEventPredicate

```
// SMEPMUEventPredicate()
// ======================
// Call the relevant PMU predication events based on the SME instruction properties.

func SMEPMUEventPredicate{N}(mask1 : bits(N), mask2 : bits(N), esize : integer)
begin
    PMUEvent(PMU_EVENT_SVE_PRED_SPEC);
    PMUEvent(PMU_EVENT_SME_PRED2_SPEC);
    if AllElementsActive{N}(mask1, esize) && AllElementsActive{N}(mask2, esize) then
        PMUEvent(PMU_EVENT_SME_PRED2_FULL_SPEC);
    else
        PMUEvent(PMU_EVENT_SME_PRED2_NOT_FULL_SPEC);
        if !AnyActiveElement{N}(mask1, esize) && !AnyActiveElement{N}(mask2, esize) then
            PMUEvent(PMU_EVENT_SME_PRED2_EMPTY_SPEC);
        else
            PMUEvent(PMU_EVENT_SME_PRED2_PARTIAL_SPEC);
        end;
    end;
end;
```

Library pseudocode for shared/debug/pmu/SVEPMUEventPredicate

```
// SVEPMUEventPredicate()
// ======================
// Call the relevant PMU predication events based on the SVE instruction properties.

func SVEPMUEventPredicate{N}(mask : bits(N), esize : integer)
begin
    PMUEvent(PMU_EVENT_SVE_PRED_SPEC);
    if AllElementsActive{N}(mask, esize) then
        PMUEvent(PMU_EVENT_SVE_PRED_FULL_SPEC);
    else
        PMUEvent(PMU_EVENT_SVE_PRED_NOT_FULL_SPEC);
        if !AnyActiveElement{N}(mask, esize) then
            PMUEvent(PMU_EVENT_SVE_PRED_EMPTY_SPEC);
        else
            PMUEvent(PMU_EVENT_SVE_PRED_PARTIAL_SPEC);
        end;
    end;
end;
```

Library pseudocode for shared/debug/pmu/ShouldPMUFreeze

```
// ShouldPMUFreeze()
// =================

func ShouldPMUFreeze(r : PMUCounterRange) => boolean
begin
    let include_r1 : boolean = (r == PMUCounterRange_R1);
    let include_r2 : boolean = (r == PMUCounterRange_R2);
    let include_r3 : boolean = FALSE;

    if r == PMUCounterRange_R3 then
        return FALSE;
    end;

    let overflow : boolean = CheckPMUOverflowCondition(PMUOverflowCondition_Freeze,
                                                       include_r1, include_r2, include_r3);
    return overflow;
end;
```

Library pseudocode for shared/debug/pmu/SysRegGetPMUAccessMask

```
// SysRegGetPMUAccessMask()
// ========================
// Return a mask of the PMU counters accessible at the current Exception level
// for a System register access

func SysRegGetPMUAccessMask() => bits(64)
begin
    var mask : bits(64) = Zeros{};

    // PMICNTR_EL0 is only accessible at EL0 using AArch64 when PMUSERENR_EL0.UEN is 1.
    if IsFeatureImplemented(FEAT_PMUv3_ICNTR) && !UsingAArch32() then
        assert IsFeatureImplemented(FEAT_PMUv3p9);
        if PSTATE.EL != EL0 || PMUSERENR_EL0().UEN == '1' then
            mask[INSTRUCTION_COUNTER_ID] = '1';
        end;
    end;

    // PMCCNTR_EL0 is always implemented and accessible
    mask[CYCLE_COUNTER_ID] = '1';

    // PMEVCNTR_EL0(n)
    let counters : integer{} = GetNumEventCountersAccessible();
    if counters > 0 then
        mask[counters-1:0] = Ones{counters};
    end;

    // Check EL0 ignore access conditions
    if (IsFeatureImplemented(FEAT_PMUv3p9) && !ELUsingAArch32(EL1) &&
          PSTATE.EL == EL0 && PMUSERENR_EL0().UEN == '1') then
        mask = mask AND PMUACR_EL1();  // User access control
    end;

    return mask;
end;
```

Library pseudocode for shared/debug/pmu/SysRegZeroCycleCounter

```
// SysRegZeroCycleCounter()
// ========================
// Called on a System register write to PMCR_EL0 or PMCR that writes '1' to PMCR_EL0.C or PMCR.C.

func SysRegZeroCycleCounter()
begin
    var mask : bits(64) = Zeros{};
    mask[CYCLE_COUNTER_ID] = '1';
    SysRegZeroPMUCounters(mask);
end;
```

Library pseudocode for shared/debug/pmu/SysRegZeroEventCounters

```
// SysRegZeroEventCounters()
// =========================
// Zero all the event counters.
// Called on a System register write to PMCR_EL0 or PMCR that writes '1' to PMCR_EL0.P or PMCR.P.

func SysRegZeroEventCounters()
begin
    // Although SysRegZeroPMUCounters implements the functionality for PMUACR_EL1
    // that is part of FEAT_PMUv3p9, it should be noted that writes to
    // PMCR_EL0 are not allowed at EL0 when PMUSERENR_EL0.UEN is 1, meaning
    // it is not relevant in this case.
    SysRegZeroPMUCounters(Zeros{33} :: Ones{31});
end;
```

Library pseudocode for shared/debug/pmu/SysRegZeroPMUCounters

```
// SysRegZeroPMUCounters()
// =======================
// Zero set of counters specified by the mask in 'val', for System register access.
// For a System register write to PMZR_EL0, 'val' is the value passed in X<t>.

func SysRegZeroPMUCounters(val : bits(64))
begin
    let masked_val : bits(64) = val AND GetPMUWriteMask(TRUE);
    ApplyPMUZeroMask(masked_val);
    return;
end;
```

Library pseudocode for shared/debug/pmu/Write_PMCNTENCLR

```
// Write_PMCNTENCLR()
// ==================
// System register write to PMCNTENCLR.

func Write_PMCNTENCLR(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMCNTENCLR() = PMCNTENCLR() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMCNTENCLR_EL0

```
// Write_PMCNTENCLR_EL0()
// ======================
// System register write to PMCNTENCLR_EL0.

func Write_PMCNTENCLR_EL0(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMCNTENCLR_EL0() = PMCNTENCLR_EL0() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMCNTENSET

```
// Write_PMCNTENSET()
// ==================
// System register write to PMCNTENSET.

func Write_PMCNTENSET(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMCNTENSET() = PMCNTENSET() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMCNTENSET_EL0

```
// Write_PMCNTENSET_EL0()
// ======================
// System register write to PMCNTENSET_EL0.

func Write_PMCNTENSET_EL0(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMCNTENSET_EL0() = PMCNTENSET_EL0() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMINTENCLR

```
// Write_PMINTENCLR()
// ==================
// System register write to PMINTENCLR.

func Write_PMINTENCLR(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMINTENCLR() = PMINTENCLR() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMINTENCLR_EL1

```
// Write_PMINTENCLR_EL1()
// ======================
// System register write to PMINTENCLR_EL1.

func Write_PMINTENCLR_EL1(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMINTENCLR_EL1() = PMINTENCLR_EL1() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMINTENSET

```
// Write_PMINTENSET()
// ==================
// System register write to PMINTENSET.

func Write_PMINTENSET(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMINTENSET() = PMINTENSET() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMINTENSET_EL1

```
// Write_PMINTENSET_EL1()
// ======================
// System register write to PMINTENSET_EL1.

func Write_PMINTENSET_EL1(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMINTENSET_EL1() = PMINTENSET_EL1() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMOVSCLR_EL0

```
// Write_PMOVSCLR_EL0()
// ====================
// System register write to PMOVSCLR_EL0.

func Write_PMOVSCLR_EL0(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMOVSCLR_EL0() = PMOVSCLR_EL0() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMOVSR

```
// Write_PMOVSR()
// ==============
// System register write to PMOVSR.

func Write_PMOVSR(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMOVSR() = PMOVSR() AND NOT (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMOVSSET

```
// Write_PMOVSSET()
// ================
// System register write to PMOVSSET.

func Write_PMOVSSET(value_in : bits(32))
begin
    let write_mask : bits(32) = GetPMUWriteMask(FALSE)[31:0];

    PMOVSSET() = PMOVSSET() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/pmu/Write_PMOVSSET_EL0

```
// Write_PMOVSSET_EL0()
// ====================
// System register write to PMOVSSET_EL0.

func Write_PMOVSSET_EL0(value_in : bits(64))
begin
    let write_mask : bits(64) = GetPMUWriteMask(FALSE);

    PMOVSSET_EL0() = PMOVSSET_EL0() OR (value_in AND write_mask);
end;
```

Library pseudocode for shared/debug/samplebasedprofiling/CreatePCSample

```
// CreatePCSample()
// ================

func CreatePCSample()
begin
    // In a simple sequential execution of the program, CreatePCSample is executed each time the PE
    // executes an instruction that can be sampled. An implementation is not constrained such that
    // reads of EDPCSRlo return the current values of PC, etc.
    if PCSRSuspended() then return; end;
    pc_sample.valid = ExternalNoninvasiveDebugAllowed() && !Halted();
    pc_sample.pc = ThisInstrAddr{64}();
    pc_sample.el = PSTATE.EL;
    pc_sample.rw = if UsingAArch32() then '0' else '1';
    pc_sample.ss = CurrentSecurityState();
    pc_sample.contextidr = if ELUsingAArch32(EL1) then CONTEXTIDR() else CONTEXTIDR_EL1()[31:0];
    pc_sample.has_el2 = PSTATE.EL != EL3 && EL2Enabled();

    if pc_sample.has_el2 then
        pc_sample.vmid = VMID();
        if ((IsFeatureImplemented(FEAT_VHE) || IsFeatureImplemented(FEAT_Debugv8p2)) &&
              !ELUsingAArch32(EL2)) then
            pc_sample.contextidr_el2 = CONTEXTIDR_EL2()[31:0];
        else
            pc_sample.contextidr_el2 = ARBITRARY : bits(32);
        end;
        pc_sample.el0h = PSTATE.EL == EL0 && IsInHost();
    end;
    return;
end;
```

Library pseudocode for shared/debug/samplebasedprofiling/PCSRSuspended

```
// PCSRSuspended()
// ===============
// Returns TRUE if PC Sample-based Profiling is suspended, and FALSE otherwise.

func PCSRSuspended() => boolean
begin
    if IsFeatureImplemented(FEAT_PMUv3_SS) && PMPCSCTL().SS == '1' then
        return FALSE;
    end;
    if IsFeatureImplemented(FEAT_PCSRv8p9) && PMPCSCTL().IMP == '1' then
        return PMPCSCTL().EN == '0';
    end;
    return ImpDefBool("PCSR is suspended");
end;
```

Library pseudocode for shared/debug/samplebasedprofiling/PCSample

```
// PCSample
// ========

type PCSample of record {
    valid : boolean,
    pc : bits(64),
    el : bits(2),
    rw : bit,
    ss : SecurityState,
    has_el2 : boolean,
    contextidr : bits(32),
    contextidr_el2 : bits(32),
    el0h : boolean,
    vmid : bits(NUM_VMIDBITS)
};


var pc_sample : PCSample;
```

Library pseudocode for shared/debug/samplebasedprofiling/Read_EDPCSRlo

```
// Read_EDPCSRlo()
// ===============

func Read_EDPCSRlo(memory_mapped : boolean) => bits(32)
begin
    // The Software lock is OPTIONAL.
    let update : boolean = !memory_mapped || EDLSR().SLK == '0';// Software locked: no side-effects
    var sample : bits(32);
    if pc_sample.valid then
        sample = pc_sample.pc[31:0];
        if update then
            if IsFeatureImplemented(FEAT_VHE) && EDSCR().SC2 == '1' then
                EDPCSR().SC2_1.EDPCSRhi  = (if pc_sample.rw == '0' then Zeros{24}
                                            else pc_sample.pc[55:32]);
                EDPCSR().EL              = pc_sample.el;
                EDPCSR().NS              = (if pc_sample.ss == SS_Secure then '0' else '1');
            else
                EDPCSR().SC2_0.EDPCSRhi  = (if pc_sample.rw == '0' then Zeros{32}
                                            else pc_sample.pc[63:32]);
            end;
            EDCIDSR() = pc_sample.contextidr;
            if ((IsFeatureImplemented(FEAT_VHE) || IsFeatureImplemented(FEAT_Debugv8p2)) &&
                  EDSCR().SC2 == '1') then
                EDVIDSR() = (if pc_sample.has_el2 then pc_sample.contextidr_el2
                             else ARBITRARY : bits(32));
            else
                EDVIDSR().VMID = (if pc_sample.has_el2 && pc_sample.el IN {EL1,EL0}
                                  then pc_sample.vmid else Zeros{NUM_VMIDBITS});
                EDVIDSR().NS = (if pc_sample.ss == SS_Secure then '0' else '1');
                EDVIDSR().E2 = (if pc_sample.el == EL2 then '1' else '0');
                EDVIDSR().E3 = (if pc_sample.el == EL3 then '1' else '0') AND pc_sample.rw;
                // The conditions for setting HV are not specified if PCSRhi is zero.
                // An example implementation may be "pc_sample.rw".
                EDVIDSR().HV = (if !IsZero(EDPCSR().SC2_0.EDPCSRhi) then '1'
                                else ImpDefBit("0 or 1"));
            end;
        end;
    else
        sample = Ones{32};
        if update then
            EDPCSR()[63:32] = ARBITRARY : bits(32);
            EDCIDSR() = ARBITRARY : bits(32);
            EDVIDSR() = ARBITRARY : bits(32);
        end;
    end;

    return sample;
end;
```

Library pseudocode for shared/debug/samplebasedprofiling/Read_PMPCSR

```
// Read_PMPCSR()
// =============

func Read_PMPCSR(memory_mapped : boolean) => bits(64)
begin
    // The Software lock is OPTIONAL.
    var update : boolean = !memory_mapped || PMLSR().SLK == '0'; // Software locked: no side-effects

    if IsFeatureImplemented(FEAT_PCSRv8p9) && update then
        if IsFeatureImplemented(FEAT_PMUv3_SS) && PMPCSCTL().SS == '1' then
            update = FALSE;
        elsif PMPCSCTL().[IMP,EN] == '10' || (PMPCSCTL().IMP == '0' && PCSRSuspended()) then
            pc_sample.valid = FALSE;
            SetPCSRActive();
        end;
    end;

    if pc_sample.valid then
        if update then SetPCSample(); end;
        return PMPCSR();
    else
        if update then SetPCSRUnknown(); end;
        return (ARBITRARY : bits(32) :: Ones{32});
    end;
end;
```

Library pseudocode for shared/debug/samplebasedprofiling/SetPCSRActive

```
// SetPCSRActive()
// ===============
// Sets PC Sample-based Profiling to active state.

func SetPCSRActive()
begin
    if PMPCSCTL().IMP == '1' then
        PMPCSCTL().EN = '1';
    // If PMPCSCTL.IMP reads as `0b0`, then PMPCSCTL.EN is RES0, and it is
    // IMPLEMENTATION DEFINED whether PSCR is suspended or active at reset.
    end;
end;
```

Library pseudocode for shared/debug/samplebasedprofiling/SetPCSRUnknown

```
// SetPCSRUnknown()
// ================
// Sets the PC sample registers to UNKNOWN values because PC sampling
// is prohibited.

func SetPCSRUnknown()
begin
    PMPCSR()[31:0]  = Ones{32};
    PMPCSR()[55:32] = ARBITRARY : bits(24);
    PMPCSR().EL     = ARBITRARY : bits(2);
    PMPCSR().NS     = ARBITRARY : bit;

    if IsFeatureImplemented(FEAT_PMUv3_EXT64) then
        PMCCIDSR()      = ARBITRARY : bits(64);
        PMVCIDSR().VMID = ARBITRARY : bits(NUM_VMIDBITS);
    end;
    if IsFeatureImplemented(FEAT_PMUv3_EXT32) then
        PMCID1SR()      = ARBITRARY : bits(32);
        PMCID2SR()      = ARBITRARY : bits(32);
        PMVIDSR().VMID  = ARBITRARY : bits(NUM_VMIDBITS);
    end;

    return;
end;
```

Library pseudocode for shared/debug/samplebasedprofiling/SetPCSample

```
// SetPCSample()
// =============
// Sets the PC sample registers to the appropriate sample values.

func SetPCSample()
begin
    PMPCSR()[31:0] = pc_sample.pc[31:0];
    PMPCSR()[55:32] = (if pc_sample.rw == '0' then Zeros{24} else pc_sample.pc[55:32]);
    PMPCSR().EL = pc_sample.el;
    if IsFeatureImplemented(FEAT_RME) then
        case pc_sample.ss of
            when SS_Secure =>
                PMPCSR().NSE = '0'; PMPCSR().NS = '0';
            when SS_NonSecure =>
                PMPCSR().NSE = '0'; PMPCSR().NS = '1';
            when SS_Root =>
                PMPCSR().NSE = '1'; PMPCSR().NS = '0';
            when SS_Realm =>
                PMPCSR().NSE = '1'; PMPCSR().NS = '1';
        end;
    else
        PMPCSR().NS = (if pc_sample.ss == SS_Secure then '0' else '1');
    end;
    let contextidr_el2 : bits(32) = (if pc_sample.has_el2 then
                                        pc_sample.contextidr_el2 else ARBITRARY : bits(32));
    var vmid : bits(NUM_VMIDBITS) = ARBITRARY : bits(NUM_VMIDBITS);
    if pc_sample.has_el2 && pc_sample.el IN {EL1,EL0} && !pc_sample.el0h then
        vmid = pc_sample.vmid;
    end;
    if IsFeatureImplemented(FEAT_PMUv3_EXT64) then
        PMCCIDSR() = contextidr_el2::pc_sample.contextidr;
        PMVCIDSR().VMID = vmid;
    end;
    if IsFeatureImplemented(FEAT_PMUv3_EXT32) then
        PMCID1SR() = pc_sample.contextidr;
        PMCID2SR() = contextidr_el2;
        PMVIDSR().VMID = vmid;
    end;

    return;
end;
```

Library pseudocode for shared/debug/softwarestep/CheckSoftwareStep

```
// CheckSoftwareStep()
// ===================
// Take a Software Step exception if in the active-pending state

func CheckSoftwareStep()
begin

    // Other self-hosted debug functions will call AArch32_GenerateDebugExceptions() if called from
    // AArch32 state. However, because Software Step is only active when the debug target Exception
    // level is using AArch64, CheckSoftwareStep only calls AArch64_GenerateDebugExceptions().
    let step_enabled : boolean = (!ELUsingAArch32(DebugTarget()) &&
                                  AArch64_GenerateDebugExceptions() &&
                                  MDSCR_EL1().SS == '1');
    let active_pending : boolean = step_enabled && PSTATE.SS == '0';   // active-pending
    if active_pending then
        AArch64_SoftwareStepException();
    end;
    ShouldAdvanceSS = TRUE;
    return;
end;
```

Library pseudocode for shared/debug/softwarestep/DebugExceptionReturnSS

```
// DebugExceptionReturnSS()
// ========================
// Returns value to write to PSTATE.SS on an exception return or Debug state exit.

func DebugExceptionReturnSS{N}(spsr : bits(N)) => bit
begin
    assert Halted() || Restarting() ||  PSTATE.EL != EL0;

    var enabled_at_source : boolean;
    if Restarting() then
        enabled_at_source = FALSE;
    elsif UsingAArch32() then
        enabled_at_source = AArch32_GenerateDebugExceptions();
    else
        enabled_at_source = AArch64_GenerateDebugExceptions();
    end;
    var valid : boolean;
    var dest_el : bits(2);
    if IllegalExceptionReturn{N}(spsr) then
        dest_el = PSTATE.EL;
    else
        (valid, dest_el) = ELFromSPSR{N}(spsr);  assert valid;
    end;

    let dest_ss : SecurityState = SecurityStateAtEL(dest_el);
    var enabled_at_dest : boolean;
    let dest_using_32 : boolean = (if dest_el == EL0 then spsr[4] == '1'
                                                     else ELUsingAArch32(dest_el));
    if dest_using_32 then
        enabled_at_dest = AArch32_GenerateDebugExceptionsFrom(dest_el, dest_ss);
    else
        let mask : bit = spsr[9];
        enabled_at_dest = AArch64_GenerateDebugExceptionsFrom(dest_el, dest_ss, mask);
    end;

    let ELd : bits(2) = DebugTargetFrom(dest_ss);
    var SS_bit : bit;
    if !ELUsingAArch32(ELd) && MDSCR_EL1().SS == '1' && !enabled_at_source && enabled_at_dest then
        SS_bit = spsr[21];
    else
        SS_bit = '0';
    end;
    return SS_bit;
end;
```

Library pseudocode for shared/debug/softwarestep/SSAdvance

```
// SSAdvance()
// ===========
// Advance the Software Step state machine.

func SSAdvance()
begin

    // A simpler implementation of this function just clears PSTATE.SS to zero regardless of the
    // current Software Step state machine. However, this check is made to illustrate that the
    // PE only needs to consider advancing the state machine from the active-not-pending
    // state.
    if !ShouldAdvanceSS then return; end;
    let target : bits(2) = DebugTarget();
    let step_enabled : boolean = !ELUsingAArch32(target) && MDSCR_EL1().SS == '1';
    let active_not_pending : boolean = step_enabled && PSTATE.SS == '1';
    if active_not_pending then PSTATE.SS = '0'; end;
    ShouldAdvanceSS = FALSE;
    return;
end;
```

Library pseudocode for shared/debug/softwarestep/SoftwareStepOpEnabled

```
// SoftwareStepOpEnabled()
// =======================
// Returns a boolean indicating if execution from MDSTEPOP_EL1 is enabled.

func SoftwareStepOpEnabled() => boolean
begin

    if !IsFeatureImplemented(FEAT_STEP2) || UsingAArch32() then
        return FALSE;
    end;

    let step_enabled = AArch64_GenerateDebugExceptions() && MDSCR_EL1().SS == '1';
    let active_not_pending = step_enabled && PSTATE.SS == '1';
    let stepop = (MDSCR_EL1().EnSTEPOP == '1' &&
                  (!HaveEL(EL3) || MDCR_EL3().EnSTEPOP == '1') &&
                  (!EL2Enabled() || MDCR_EL2().EnSTEPOP == '1'));
    return active_not_pending && stepop;
end;
```

Library pseudocode for shared/debug/watchpoint/DataCacheWatchpointSize

```
// DataCacheWatchpointSize()
// =========================
// Return the IMPLEMENTATION DEFINED data cache watchpoint size
func DataCacheWatchpointSize() => integer
begin
    let size : integer = ImpDefInt("Data Cache Invalidate Watchpoint Size");
    assert IsPow2(size) && size >= 2^(UInt(CTR_EL0().DminLine) + 2) && size <= 2048;
    return size;
end;
```

Library pseudocode for shared/debug/watchpoint/WatchpointInfo

```
// WatchpointInfo
// ==============
// Watchpoint related fields.

type WatchpointInfo of record {
    wptype : WatchpointType,     // Type of watchpoint matched
    maybe_false_match : boolean, // Watchpoint matches rounded range
    watchpt_num : integer,       // Matching watchpoint number
    value_match : boolean,       // Watchpoint match
    vaddress : bits(64)          // Matching Virtual Address
};
```

Library pseudocode for shared/debug/watchpoint/WatchpointType

```
// WatchpointType
// ==============

type WatchpointType of enumeration {
    WatchpointType_Inactive,      // Watchpoint inactive or disabled
    WatchpointType_AddrMatch,     // Address Match watchpoint
    WatchpointType_AddrMismatch   // Address Mismatch watchpoint
};
```

Library pseudocode for shared/exceptions/aborts/EffectiveHCRX_EL2_TMEA

```
// EffectiveHCRX_EL2_TMEA()
// ========================
// Return the Effective value of HCRX_EL2.TMEA.

readonly func EffectiveHCRX_EL2_TMEA() => bit
begin
    if (IsFeatureImplemented(FEAT_DoubleFault2) && EL2Enabled() &&
          !ELUsingAArch32(EL2) && IsHCRXEL2Enabled()) then
        return HCRX_EL2().TMEA;
    else
        return '0';
    end;
end;
```

Library pseudocode for shared/exceptions/aborts/EffectiveHCR_AMO

```
// EffectiveHCR_AMO()
// ==================
// Return the Effective value of HCR_EL2.AMO.

readonly func EffectiveHCR_AMO() => bit
begin
    if EffectiveTGE() == '1' then
        return (if ELUsingAArch32(EL2) || EffectiveHCR_EL2_E2H() == '0' then '1' else '0');
    elsif EL2Enabled() then
        return (if ELUsingAArch32(EL2) then HCR().AMO else HCR_EL2().AMO);
    else
        return '0';
    end;
end;
```

Library pseudocode for shared/exceptions/aborts/EffectiveHCR_TEA

```
// EffectiveHCR_TEA()
// ==================
// Return the Effective value of HCR_EL2.TEA.

func EffectiveHCR_TEA() => bit
begin
    if EL2Enabled() && IsFeatureImplemented(FEAT_RAS) then
        return (if ELUsingAArch32(EL2) then HCR2().TEA else HCR_EL2().TEA);
    else
        return '0';
    end;
end;
```

Library pseudocode for shared/exceptions/aborts/EffectiveNMEA

```
// EffectiveNMEA()
// ===============
// Returns the Effective value of SCR_EL3.NMEA or SCTLR2_ELx.NMEA.

func EffectiveNMEA(el : bits(2)) => bit
begin
    if IsFeatureImplemented(FEAT_DoubleFault2) then
        if el == EL3 && !ELUsingAArch32(EL3) then
            return EffectiveSCR_EL3_NMEA();
        elsif (el == EL2 || (el == EL0 && ELIsInHost(el))) && !ELUsingAArch32(EL2) then
            return (if IsSCTLR2EL2Enabled() then SCTLR2_EL2().NMEA else '0');
        elsif (el == EL1 || (el == EL0 && !ELIsInHost(el))) && !ELUsingAArch32(EL1) then
            return (if IsSCTLR2EL1Enabled() then SCTLR2_EL1().NMEA else '0');
        else
            return '0';
        end;
    end;
    if IsFeatureImplemented(FEAT_DoubleFault) && el == EL3 && !ELUsingAArch32(EL3) then
        return EffectiveSCR_EL3_NMEA();
    else
        return '0';
    end;
end;
```

Library pseudocode for shared/exceptions/aborts/PhysicalSErrorTarget

```
// PhysicalSErrorTarget()
// ======================
// Returns a tuple of whether SError exception can be taken and, if so, the
// target Exception level.
// If EL3 is implemented and using AArch32, then a target Exception level of
// EL1 means Abort mode, and EL3 means Monitor mode, including in Secure
// state when Abort mode is part of EL3.

func PhysicalSErrorTarget() => (boolean, bits(2))
begin
    if Halted() then
        return (TRUE, ARBITRARY : bits(2));
    end;

    let effective_ea : bit = EffectiveEA();
    let effective_amo : bit = EffectiveHCR_AMO();
    let effective_tge : bit = EffectiveTGE();
    let effective_nmea : bit = EffectiveNMEA(PSTATE.EL);

    // When EL3 is implemented and using AArch32, the SCR.AW bit can allow PSTATE.A
    // to mask SError exceptions in Non-secure state when SCR.EA is 1 and the Effective
    // value of HCR.AMO is 0.
    var effective_aw : bit;
    if (ELUsingAArch32(EL3) && effective_ea == '1' &&
          CurrentSecurityState() == SS_NonSecure && effective_amo == '0') then
        effective_aw = SCR().AW;
    else
        effective_aw = '0';
    end;

    // The exception is masked by software.
    var masked : boolean;
    case PSTATE.EL of
        when EL3 =>
            masked = (!UsingAArch32() && effective_ea == '0') || PSTATE.A == '1';
        when EL2 =>
            masked = ((effective_ea == '0' || effective_aw == '1') &&
                      ((!UsingAArch32() && effective_tge == '0' && effective_amo == '0') ||
                       PSTATE.A == '1'));
        when EL1, EL0 =>
            masked = ((effective_ea == '0' || effective_aw == '1') &&
                      effective_amo == '0' && PSTATE.A == '1');
    end;

    // When FEAT_DoubleFault or FEAT_DoubleFault2 is implemented, the mask might be overridden.
    masked = (masked && effective_nmea == '0');

    // External debug might disable the exception in the current Security state.
    // This is not relevant at EL3.
    let intdis : boolean = PSTATE.EL != EL3 && ExternalDebugInterruptsDisabled(EL1);

    var target_el : bits(2) = ARBITRARY : bits(2);
    if effective_ea == '1' || (PSTATE.EL == EL3 && !ELUsingAArch32(EL3)) then
        if !masked then target_el = EL3; end;

    elsif EL2Enabled() && effective_amo == '1' && !intdis && PSTATE.EL IN {EL0, EL1} then
        target_el = EL2;
        masked = FALSE;

    elsif (EffectiveHCRX_EL2_TMEA() == '1' && !intdis &&
             ((PSTATE.EL == EL1 && PSTATE.A == '1') ||
              (PSTATE.EL == EL0 && masked && !IsInHost()))) then
        target_el = EL2;
        masked = FALSE;

    elsif (EffectiveSCR_EL3_TMEA() == '1' &&
             ((PSTATE.EL IN {EL2, EL1} && PSTATE.A == '1') ||
             (PSTATE.EL IN {EL2, EL0} && masked) || intdis)) then
        target_el = EL3;
        masked = FALSE;

    elsif PSTATE.EL == EL2 || IsInHost() then
        if !masked then target_el = EL2; end;

    else
        assert (PSTATE.EL == EL1 ||
                  (PSTATE.EL == EL3 && ELUsingAArch32(EL3)) ||
                  (PSTATE.EL == EL0 && !IsInHost()));
        if !masked then target_el = EL1; end;
    end;

    // External debug might disable the exception for the target Exception level.
    if !masked && ExternalDebugInterruptsDisabled(target_el) then
        masked = TRUE;
        target_el = ARBITRARY : bits(2);
    end;

    return (masked, target_el);
end;
```

Library pseudocode for shared/exceptions/aborts/SyncExternalAbortTarget

```
// SyncExternalAbortTarget()
// =========================
// Returns the target Exception level for a Synchronous External Data or
// Instruction or Prefetch Abort.
// If EL3 is implemented and using AArch32, then a target Exception level of
// EL1 means Abort mode, and EL3 means Monitor mode, including in Secure
// state when Abort mode is part of EL3.

func SyncExternalAbortTarget(fault : FaultRecord) => bits(2)
begin
    let effective_ea : bit = EffectiveEA();
    let effective_tea : bit = EffectiveHCR_TEA();
    let effective_tge : bit = EffectiveTGE();

    var target_el : bits(2);
    if effective_ea == '1' || (PSTATE.EL == EL3 && !ELUsingAArch32(EL3)) then
        target_el = EL3;

    elsif (EL2Enabled() && PSTATE.EL IN {EL1, EL0} &&
           (effective_tea == '1' || IsSecondStage(fault) ||
            fault.accessdesc.acctype == AccessType_NV2 ||
            (PSTATE.EL == EL0 && effective_tge == '1'))) then
        target_el = EL2;

    elsif EffectiveHCRX_EL2_TMEA() == '1' && PSTATE.A == '1' && PSTATE.EL == EL1 then
        target_el = EL2;

    elsif EffectiveSCR_EL3_TMEA() == '1' && PSTATE.A == '1' && PSTATE.EL IN {EL1, EL2} then
        target_el = EL3;

    else
        assert PSTATE.EL != EL3 || ELUsingAArch32(EL3);
        target_el = (if PSTATE.EL == EL2 then EL2 else EL1);
    end;

    return target_el;
end;
```

Library pseudocode for shared/exceptions/exceptions/ConditionSyndrome

```
// ConditionSyndrome()
// ===================
// Return CV and COND fields of instruction syndrome

func ConditionSyndrome() => bits(5)
begin

    var syndrome : bits(5);

    if UsingAArch32() then
        let cond = CurrentCond();
        if PSTATE.T == '0' then             // A32
            syndrome[4] = '1';
            // A conditional A32 instruction that is known to pass its condition code check
            // can be presented either with COND set to 0xE, the value for unconditional, or
            // the COND value held in the instruction.
            if ConditionHolds(cond) && ConstrainUnpredictableBool(Unpredictable_ESRCONDPASS) then
                syndrome[3:0] = '1110';
            else
                syndrome[3:0] = cond;
            end;
        else                                // T32
            // When a T32 instruction is trapped, it is IMPLEMENTATION DEFINED whether:
            //  * CV set to 0 and COND is set to an UNKNOWN value
            //  * CV set to 1 and COND is set to the condition code for the condition that
            //    applied to the instruction.
            if ImpDefBool("Condition valid for trapped T32") then
                syndrome[4] = '1';
                syndrome[3:0] = cond;
            else
                syndrome[4] = '0';
                syndrome[3:0] = ARBITRARY : bits(4);
            end;
        end;
    else
        syndrome[4] = '1';
        syndrome[3:0] = '1110';
    end;
    return syndrome;
end;
```

Library pseudocode for shared/exceptions/exceptions/Exception

```
// Exception
// =========
// Classes of exception.

type Exception of enumeration {
        Exception_Uncategorized,        // Uncategorized or unknown reason
        Exception_WFxTrap,              // Trapped WFI or WFE instruction
        Exception_CP15RTTrap,           // Trapped AArch32 MCR or MRC access, coproc=0b111
        Exception_CP15RRTTrap,          // Trapped AArch32 MCRR or MRRC access, coproc=0b1111
        Exception_CP14RTTrap,           // Trapped AArch32 MCR or MRC access, coproc=0b1110
        Exception_CP14DTTrap,           // Trapped AArch32 LDC or STC access, coproc=0b1110
        Exception_CP14RRTTrap,          // Trapped AArch32 MRRC access, coproc=0b1110
        Exception_AdvSIMDFPAccessTrap,  // HCPTR-trapped access to SIMD or FP
        Exception_FPIDTrap,             // Trapped access to SIMD or FP ID register
        Exception_OtherTrap,            // Trapped access to other instructions
        // Trapped BXJ instruction not supported in Armv8
        Exception_PACTrap,               // Trapped invalid PAC use
        Exception_IllegalState,          // Illegal Execution state
        Exception_SupervisorCall,        // Supervisor Call
        Exception_HypervisorCall,        // Hypervisor Call
        Exception_MonitorCall,           // Monitor Call or Trapped SMC instruction
        Exception_SystemRegisterTrap,    // Trapped MRS or MSR System register access
        Exception_ERetTrap,              // Trapped invalid ERET use
        Exception_InstructionAbort,      // Instruction Abort or Prefetch Abort
        Exception_PCAlignment,           // PC alignment fault
        Exception_DataAbort,             // Data Abort
        Exception_NV2DataAbort,          // Data abort at EL1 reported as being from EL2
        Exception_PACFail,               // PAC Authentication failure
        Exception_SPAlignment,           // SP alignment fault
        Exception_FPTrappedException,    // IEEE trapped FP exception
        Exception_SError,                // SError interrupt
        Exception_Breakpoint,            // (Hardware) Breakpoint
        Exception_SoftwareStep,          // Software Step
        Exception_Watchpoint,            // Watchpoint
        Exception_NV2Watchpoint,         // Watchpoint at EL1 reported as being from EL2
        Exception_SoftwareBreakpoint,    // Software Breakpoint Instruction
        Exception_VectorCatch,           // AArch32 Vector Catch
        Exception_IRQ,                   // IRQ interrupt
        Exception_SVEAccessTrap,         // HCPTR trapped access to SVE
        Exception_SMEAccessTrap,         // HCPTR trapped access to SME
        Exception_GPC,                   // Granule protection check
        Exception_BranchTarget,          // Branch Target Identification
        Exception_MemCpyMemSet,          // Exception from a CPY* or SET* instruction
        Exception_GCSFail,               // GCS Exceptions
        Exception_Profiling,             // Profiling exception
        Exception_SystemRegister128Trap, // Trapped MRRS or MSRR System register or SYSP access
        Exception_ConditionalFault,      // Exception due to a CFLT instruction
        Exception_FIQ};                  // FIQ interrupt
```

Library pseudocode for shared/exceptions/exceptions/ExceptionRecord

```
// ExceptionRecord
// ===============

type ExceptionRecord of record {
    exceptype : Exception,           // Exception class
    syndrome : IssType,              // Syndrome record
    paddress : FullAddress,          // Physical fault address
    vaddress : bits(64),             // Virtual fault address
    ipavalid : boolean,              // Validity of Intermediate Physical fault address
    pavalid : boolean,               // Validity of Physical fault address
    NS : bit,                        // Intermediate Physical fault address space
    ipaddress : bits(NUM_PABITS),            // Intermediate Physical fault address
    trappedsyscallinst : boolean};   // Trapped SVC or SMC instruction
```

Library pseudocode for shared/exceptions/exceptions/ExceptionSyndrome

```
// ExceptionSyndrome()
// ===================
// Return a blank exception syndrome record for an exception of the given type.

func ExceptionSyndrome(exceptype : Exception) => ExceptionRecord
begin

    var r : ExceptionRecord;

    r.exceptype      = exceptype;

    // Initialize all other fields
    r.syndrome.iss   = Zeros{25};
    r.syndrome.iss2  = Zeros{24};
    r.vaddress       = Zeros{64};
    r.ipavalid       = FALSE;
    r.pavalid        = FALSE;
    r.NS             = '0';
    r.ipaddress      = Zeros{NUM_PABITS};
    r.paddress.paspace = ARBITRARY : PASpace;
    r.paddress.address = ARBITRARY : bits(NUM_PABITS);
    r.trappedsyscallinst = FALSE;
    return r;
end;
```

Library pseudocode for shared/exceptions/traps/Undefined

```
// Undefined()
// ===========

noreturn func Undefined()
begin
    if UsingAArch32() then
        AArch32_Undefined();
    else
        AArch64_Undefined();
    end;
end;
```

Library pseudocode for shared/functions/aborts/EncodeLDFSC

```
// EncodeLDFSC()
// =============
// Function that gives the Long-descriptor FSC code for types of Fault

func EncodeLDFSC(statuscode : Fault, level : integer) => bits(6)
begin
    var result : bits(6);

    // 128-bit descriptors will start from level -2 for 4KB to resolve bits IA[55:51]
    if level == -2 then
        assert IsFeatureImplemented(FEAT_D128);
        case statuscode of
            when Fault_AddressSize =>          result = '101100';
            when Fault_Translation =>          result = '101010';
            when Fault_SyncExternalOnWalk =>   result = '010010';
            when Fault_SyncParityOnWalk =>
                result = '011010';
                assert !IsFeatureImplemented(FEAT_RAS);
            when Fault_GPCFOnWalk =>           result = '100010';
            otherwise =>                       unreachable;
        end;
        return result;
    end;

    if level == -1 then
        assert IsFeatureImplemented(FEAT_LPA2);
        case statuscode of
            when Fault_AddressSize =>          result = '101001';
            when Fault_Translation =>          result = '101011';
            when Fault_SyncExternalOnWalk =>   result = '010011';
            when Fault_SyncParityOnWalk =>
                result = '011011';
                assert !IsFeatureImplemented(FEAT_RAS);
            when Fault_GPCFOnWalk =>           result = '100011';
            otherwise =>                       unreachable;
        end;

        return result;
    end;
    case statuscode of
        when Fault_AddressSize =>          result = '0000'::level[1:0]; assert level IN {0,1,2,3};
        when Fault_AccessFlag =>           result = '0010'::level[1:0]; assert level IN {0,1,2,3};
        when Fault_Permission =>           result = '0011'::level[1:0]; assert level IN {0,1,2,3};
        when Fault_Translation   =>        result = '0001'::level[1:0]; assert level IN {0,1,2,3};
        when Fault_AsyncExternal =>        result = '010001'; assert UsingAArch32();
        when Fault_SyncExternal  =>        result = '010000';
        when Fault_SyncExternalOnWalk =>   result = '0101'::level[1:0]; assert level IN {0,1,2,3};
        when Fault_SyncParity    =>        result = '011000';
        when Fault_SyncParityOnWalk =>     result = '0111'::level[1:0]; assert level IN {0,1,2,3};
        when Fault_AsyncParity   =>        result = '011001';
        when Fault_TagCheck =>             result = '010001'; assert IsFeatureImplemented(FEAT_MTE2);
        when Fault_Alignment =>            result = '100001';
        when Fault_Debug =>                result = '100010';
        when Fault_GPCFOnWalk =>           result = '1001'::level[1:0]; assert level IN {0,1,2,3};
        when Fault_GPCFOnOutput =>         result = '101000';
        when Fault_TLBConflict =>          result = '110000';
        when Fault_UnsupportedAtomicHWUpdate =>
                                           result = '110001';
        when Fault_Lockdown =>             result = '110100';  // IMPLEMENTATION DEFINED
        when Fault_Exclusive =>            result = '110101';  // IMPLEMENTATION DEFINED
        otherwise =>                       unreachable;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/aborts/IPAValid

```
// IPAValid()
// ==========
// Return TRUE if the IPA is reported for the abort

readonly func IPAValid(fault : FaultRecord) => boolean
begin
    assert fault.statuscode != Fault_None;

    if fault.gpcf.gpf != GPCF_None then
        return fault.secondstage;
    elsif fault.s2fs1walk then
        return fault.statuscode IN {
            Fault_AccessFlag,
            Fault_Permission,
            Fault_Translation,
            Fault_AddressSize
        };
    elsif fault.secondstage then
        return fault.statuscode IN {
            Fault_AccessFlag,
            Fault_Translation,
            Fault_AddressSize
        } || (IsFeatureImplemented(FEAT_EAESR) && fault.statuscode == Fault_Permission);
    else
        return FALSE;
    end;
end;
```

Library pseudocode for shared/functions/aborts/IsAsyncAbort

```
// IsAsyncAbort()
// ==============
// Returns TRUE if the abort currently being processed is an asynchronous abort, and FALSE
// otherwise.

func IsAsyncAbort(statuscode : Fault) => boolean
begin
    assert statuscode != Fault_None;

    return (statuscode IN {Fault_AsyncExternal, Fault_AsyncParity});
end;

// IsAsyncAbort()
// ==============

func IsAsyncAbort(fault : FaultRecord) => boolean
begin
    return IsAsyncAbort(fault.statuscode);
end;
```

Library pseudocode for shared/functions/aborts/IsDebugException

```
// IsDebugException()
// ==================

pure func IsDebugException(fault : FaultRecord) => boolean
begin
    assert fault.statuscode != Fault_None;
    return fault.statuscode == Fault_Debug;
end;
```

Library pseudocode for shared/functions/aborts/IsExternalAbort

```
// IsExternalAbort()
// =================
// Returns TRUE if the abort currently being processed is an External abort and FALSE otherwise.

readonly func IsExternalAbort(statuscode : Fault) => boolean
begin
    assert statuscode != Fault_None;

    return (statuscode IN {
              Fault_SyncExternal,
              Fault_SyncParity,
              Fault_SyncExternalOnWalk,
              Fault_SyncParityOnWalk,
              Fault_AsyncExternal,
              Fault_AsyncParity
    });
end;

// IsExternalAbort()
// =================

readonly func IsExternalAbort(fault : FaultRecord) => boolean
begin
    return IsExternalAbort(fault.statuscode) || fault.gpcf.gpf == GPCF_EABT;
end;
```

Library pseudocode for shared/functions/aborts/IsExternalAbortOnWalk

```
// IsExternalAbortOnWalk()
// =======================

pure func IsExternalAbortOnWalk(fault : FaultRecord) => boolean
begin
    assert fault.statuscode != Fault_None;
    return fault.statuscode IN {Fault_SyncExternalOnWalk, Fault_SyncParityOnWalk};
end;
```

Library pseudocode for shared/functions/aborts/IsExternalSyncAbort

```
// IsExternalSyncAbort()
// =====================
// Returns TRUE if the abort currently being processed is an external
// synchronous abort and FALSE otherwise.

readonly func IsExternalSyncAbort(statuscode : Fault) => boolean
begin
    assert statuscode != Fault_None;
    if IsFeatureImplemented(FEAT_RAS) then
        assert ! statuscode IN {Fault_SyncParity, Fault_SyncParityOnWalk};
    end;

    return (statuscode IN {
              Fault_SyncExternal,
              Fault_SyncParity,
              Fault_SyncExternalOnWalk,
              Fault_SyncParityOnWalk
    });
end;

// IsExternalSyncAbort()
// =====================

readonly func IsExternalSyncAbort(fault : FaultRecord) => boolean
begin
    return IsExternalSyncAbort(fault.statuscode) || fault.gpcf.gpf == GPCF_EABT;
end;
```

Library pseudocode for shared/functions/aborts/IsFault

```
// IsFault()
// =========
// Return TRUE if a fault is associated with an address descriptor

pure func IsFault(addrdesc : AddressDescriptor) => boolean
begin
    return addrdesc.fault.statuscode != Fault_None;
end;

// IsFault()
// =========
// Return TRUE if a fault is associated with a memory access.

pure func IsFault(fault : Fault) => boolean
begin
    return fault != Fault_None;
end;

// IsFault()
// =========
// Return TRUE if a fault is associated with status returned by memory.

pure func IsFault(retstatus : PhysMemRetStatus) => boolean
begin
    return retstatus.statuscode != Fault_None;
end;
```

Library pseudocode for shared/functions/aborts/IsSErrorInterrupt

```
// IsSErrorInterrupt()
// ===================
// Returns TRUE if the abort currently being processed is an SError interrupt, and FALSE
// otherwise.

func IsSErrorInterrupt(statuscode : Fault) => boolean
begin
    assert statuscode != Fault_None;

    return (statuscode IN {Fault_AsyncExternal, Fault_AsyncParity});
end;

// IsSErrorInterrupt()
// ===================

func IsSErrorInterrupt(fault : FaultRecord) => boolean
begin
    return IsSErrorInterrupt(fault.statuscode);
end;


// Add a specific type of return value for FaultSyndrome
type IssType of record {
    iss : bits(25),
    iss2 : bits(24)
};
```

Library pseudocode for shared/functions/aborts/IsSecondStage

```
// IsSecondStage()
// ===============

pure func IsSecondStage(fault : FaultRecord) => boolean
begin
    assert fault.statuscode != Fault_None;

    return fault.secondstage;
end;
```

Library pseudocode for shared/functions/aborts/LSInstructionSyndrome

```
// LSInstructionSyndrome()
// =======================
// Returns the extended syndrome information for a second stage fault.
//  [10]  - Syndrome valid bit. The syndrome is valid only for certain types of access instruction.
//  [9:8] - Access size.
//  [7]   - Sign extended (for loads).
//  [6:2] - Transfer register.
//  [1]   - Transfer register is 64-bit.
//  [0]   - Instruction has acquire/release semantics.

impdef func LSInstructionSyndrome() => bits(11)
begin
    return ARBITRARY : bits(11);
end;
```

Library pseudocode for shared/functions/aborts/ReportAsGPCException

```
// ReportAsGPCException()
// ======================
// Determine whether the given GPCF is reported as a Granule Protection Check Exception
// rather than a Data or Instruction Abort

func ReportAsGPCException(fault : FaultRecord) => boolean
begin
    assert IsFeatureImplemented(FEAT_RME);
    assert fault.statuscode IN {Fault_GPCFOnWalk, Fault_GPCFOnOutput};
    assert fault.gpcf.gpf != GPCF_None;

    if Halted() && EDSCR().SDD == '1' then
        return FALSE;
    end;

    case fault.gpcf.gpf of
        when GPCF_Walk =>        return TRUE;
        when GPCF_AddressSize => return TRUE;
        when GPCF_EABT =>        return TRUE;
        when GPCF_Fail =>        return SCR_EL3().GPF == '1' && PSTATE.EL != EL3;
    end;
end;
```

Library pseudocode for shared/functions/cache/CACHE_OP

```
// CACHE_OP()
// ==========
// Performs Cache maintenance operations as per CacheRecord.

impdef func CACHE_OP(cache : CacheRecord)
begin
    return;
end;
```

Library pseudocode for shared/functions/cache/CPASAtPAS

```
// CPASAtPAS()
// ===========
// Get cache PA space for given PA space.

readonly func CPASAtPAS(pas : PASpace) => CachePASpace
begin
    case pas of
        when PAS_NonSecure =>
            return CPAS_NonSecure;
        when PAS_Secure =>
            return CPAS_Secure;
        when PAS_Root =>
            return CPAS_Root;
        when PAS_Realm =>
            return CPAS_Realm;
        when PAS_SystemAgent =>
            return CPAS_SystemAgent;
        when PAS_NonSecureProtected =>
            return CPAS_NonSecureProtected;
        when PAS_NA6 =>
            return CPAS_NA6;
        when PAS_NA7 =>
            return CPAS_NA7;
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/functions/cache/CPASAtSecurityState

```
// CPASAtSecurityState()
// ====================
// Get cache PA space for given security state.

func CPASAtSecurityState(ss : SecurityState) => CachePASpace
begin
    case ss of
        when SS_NonSecure =>
            return CPAS_NonSecure;
        when SS_Secure =>
            return CPAS_SecureNonSecure;
        when SS_Root =>
            return CPAS_Any;
        when SS_Realm =>
            return CPAS_RealmNonSecure;
    end;
end;
```

Library pseudocode for shared/functions/cache/CacheRecord

```
// CacheRecord
// ===========
// Details related to a cache operation.

type CacheRecord of record {
    acctype       : AccessType,        // Access type
    cacheop       : CacheOp,           // Cache operation
    opscope       : CacheOpScope,      // Cache operation type
    cachetype     : CacheType,         // Cache type
    regval        : bits(64),
    paddress      : FullAddress,
    vaddress      : bits(64),          // For VA operations
    setnum        : integer,           // For SW operations
    waynum        : integer,           // For SW operations
    level         : integer,           // For SW operations
    shareability  : Shareability,
    is_vmid_valid : boolean,           // is vmid valid for current context
    vmid          : bits(NUM_VMIDBITS),
    is_asid_valid : boolean,           // is asid valid for current context
    asid          : bits(NUM_ASIDBITS),
    security      : SecurityState,
    // For inner shareable IC operations, the TLBI Domains the operation is broadcast to
    domains : bits(16),
    // For cache operations to full cache or by setnum/waynum
    // For operations by address, PA space in paddress
    cpas : CachePASpace
};
```

Library pseudocode for shared/functions/cache/DecodeSW

```
// DecodeSW()
// ==========
// Decode input value into setnum, waynum and level for SW instructions.

func DecodeSW(regval : bits(64), cachetype : CacheType) => (integer, integer, integer)
begin
    let level : integer = UInt(regval[3:1]);
    let (numsets, associativity, linesize) = GetCacheInfo(level, cachetype);
    // For the given level and cachetype, get the number of sets, associativity and
    // cache line size in terms of actual bytes.

    let max_waybits : integer{} = if IsFeatureImplemented(FEAT_CCIDX) then 21 else 10;
    let max_setbits : integer{} = if IsFeatureImplemented(FEAT_CCIDX) then 24 else 15;

    let waybits : integer{} = CeilLog2(associativity) as integer{0..max_waybits};
    let setbits : integer{} = CeilLog2(numsets) as integer{0..max_setbits};
    let linebits : integer{} = FloorLog2(linesize) as integer{4..11};

    let waynum : integer = if associativity == 1 then 0 else UInt(regval[31:32-waybits]);
    let setnum : integer = if numsets == 1 then 0 else UInt(regval[linebits +: setbits]);

    return (setnum, waynum, level);
end;
```

Library pseudocode for shared/functions/cache/GetCacheInfo

```
// GetCacheInfo()
// ==============
// Returns numsets, associativity & linesize in terms of actual bytes.

impdef func GetCacheInfo(level : integer, cachetype : CacheType) => (integer, integer, integer)
begin
    let numsets : integer = ImpDefInt("Numsets for DC SW instructions");
    let associativity : integer = ImpDefInt("Associativity for DC SW instructions");
    let linesize : integer = ImpDefInt("Linesize for DC SW instructions");
    return (numsets, associativity, linesize);
end;
```

Library pseudocode for shared/functions/common/IsZeroBit

```
// IsZeroBit()
// ===========

pure func IsZeroBit{N}(x : bits(N)) => bit
begin
    return if IsZero(x) then '1' else '0';
end;
```

Library pseudocode for shared/functions/common/NormalizeReal

```
// NormalizeReal
// =============
// Normalizes x to the form 1.xxx... x 2^y and returns (mantissa, exponent)

pure func NormalizeReal(x : real) => (real, integer)
begin
    let exponent : integer = ILog2(x);
    let mantissa : real = x / (2.0 ^ exponent);
    return (mantissa, exponent);
end;
```

Library pseudocode for shared/functions/common/RShr

```
// RShr()
// ======
// Shift integer value right with rounding

pure func RShr(value : integer, shift : integer, round : boolean) => integer
begin
    assert shift > 0;
    if round then
        return (value + (1 << (shift - 1))) >> shift;
    else
        return value >> shift;
    end;
end;
```

Library pseudocode for shared/functions/common/Reverse

```
// Reverse()
// =========
// Reverse subwords of M bits in an N-bit word

pure func Reverse{N}(word : bits(N), M : integer) => bits(N)
begin
    assert N MOD M == 0;
    var result : bits(N);
    let swsize : integer{} = M as integer{1..N};
    let sw : integer = N DIV swsize;
    for s = 0 to sw-1 do
        result[((sw - 1) - s)*:swsize] = word[s*:swsize];
    end;
    return result;
end;
```

Library pseudocode for shared/functions/common/Signal

```
// Signal
// ======
// Available signal types

type Signal of enumeration {LOW, HIGH};
```

Library pseudocode for shared/functions/counters

```
var PhysicalCount : bits(88);

var IsLocalTimeoutEventPending : boolean;
var LocalTimeoutVal : bits(64);           // Value to compare against the Virtual Counter Timer
                                          // to generate the local timeout event.
```

Library pseudocode for shared/functions/counters/AArch32_CheckTimerConditions

```
// AArch32_CheckTimerConditions()
// ==============================
// Checking timer conditions for all A32 timer registers

func AArch32_CheckTimerConditions()
begin
    var status : boolean;
    var offset : bits(64);
    offset = Zeros{64};
    assert !HaveAArch64();
    if HaveEL(EL3) then
        if CNTP_CTL_S().ENABLE == '1' then
            status = IsTimerConditionMet(offset, CNTP_CVAL_S(),
                                         CNTP_CTL_S().IMASK, InterruptID_CNTPS);
            CNTP_CTL_S().ISTATUS = if status then '1' else '0';
        end;

        if CNTP_CTL_NS().ENABLE == '1' then
            status = IsTimerConditionMet(offset, CNTP_CVAL_NS(),
                                         CNTP_CTL_NS().IMASK, InterruptID_CNTP);
            CNTP_CTL_NS().ISTATUS = if status then '1' else '0';
        end;
    else
        if CNTP_CTL().ENABLE == '1' then
            status = IsTimerConditionMet(offset, CNTP_CVAL(),
                                         CNTP_CTL().IMASK, InterruptID_CNTP);
            CNTP_CTL().ISTATUS = if status then '1' else '0';
        end;
    end;
    if HaveEL(EL2) && CNTHP_CTL().ENABLE == '1' then
        status = IsTimerConditionMet(offset, CNTHP_CVAL(),
                                     CNTHP_CTL().IMASK, InterruptID_CNTHP);
        CNTHP_CTL().ISTATUS = if status then '1' else '0';
    end;
    if CNTV_CTL_EL0().ENABLE == '1' then
        status = IsTimerConditionMet(CNTVOFF_EL2(), CNTV_CVAL_EL0(),
                                     CNTV_CTL_EL0().IMASK, InterruptID_CNTV);
        CNTV_CTL_EL0().ISTATUS = if status then '1' else '0';
    end;
    return;
end;
```

Library pseudocode for shared/functions/counters/AArch64_CheckTimerConditions

```
// AArch64_CheckTimerConditions()
// ==============================
// Checking timer conditions for all A64 timer registers

func AArch64_CheckTimerConditions()
begin
    var status : boolean;
    var offset : bits(64);
    var imask : bit;
    let ss : SecurityState = CurrentSecurityState();
    if (IsFeatureImplemented(FEAT_ECV_POFF) && EL2Enabled() && !ELIsInHost(EL0) &&
          CNTHCTL_EL2().ECV == '1' && SCR_EL3().ECVEn == '1') then
        offset = CNTPOFF_EL2();
    else
        offset = Zeros{64};
    end;
    if CNTP_CTL_EL0().ENABLE == '1' then
        imask = CNTP_CTL_EL0().IMASK;
        if (IsFeatureImplemented(FEAT_RME) && ss IN {SS_Root, SS_Realm} &&
              CNTHCTL_EL2().CNTPMASK == '1') then
            imask = '1';
        end;
        status = IsTimerConditionMet(offset, CNTP_CVAL_EL0(),
                                     imask, InterruptID_CNTP);
        CNTP_CTL_EL0().ISTATUS = if status then '1' else '0';
    end;
    if ((HaveEL(EL3) || (HaveEL(EL2) && !IsFeatureImplemented(FEAT_SEL2))) &&
       CNTHP_CTL_EL2().ENABLE == '1') then
        status = IsTimerConditionMet(Zeros{64}, CNTHP_CVAL_EL2(),
                                     CNTHP_CTL_EL2().IMASK, InterruptID_CNTHP);
        CNTHP_CTL_EL2().ISTATUS = if status then '1' else '0';
    end;
    if HaveEL(EL2) && IsFeatureImplemented(FEAT_SEL2) && CNTHPS_CTL_EL2().ENABLE == '1' then
        status = IsTimerConditionMet(Zeros{64}, CNTHPS_CVAL_EL2(),
                                     CNTHPS_CTL_EL2().IMASK, InterruptID_CNTHPS);
        CNTHPS_CTL_EL2().ISTATUS = if status then '1' else '0';
    end;

    if CNTPS_CTL_EL1().ENABLE == '1' then
        status = IsTimerConditionMet(Zeros{64}, CNTPS_CVAL_EL1(),
                                     CNTPS_CTL_EL1().IMASK, InterruptID_CNTPS);
        CNTPS_CTL_EL1().ISTATUS = if status then '1' else '0';
    end;

    if CNTV_CTL_EL0().ENABLE == '1' then
        imask = CNTV_CTL_EL0().IMASK;
        if (IsFeatureImplemented(FEAT_RME) && ss IN {SS_Root, SS_Realm} &&
              CNTHCTL_EL2().CNTVMASK == '1') then
            imask = '1';
        end;
        status = IsTimerConditionMet(CNTVOFF_EL2(), CNTV_CVAL_EL0(),
                                     imask, InterruptID_CNTV);
        CNTV_CTL_EL0().ISTATUS = if status then '1' else '0';
    end;

    if ((IsFeatureImplemented(FEAT_VHE) && (HaveEL(EL3) || !IsFeatureImplemented(FEAT_SEL2))) &&
          CNTHV_CTL_EL2().ENABLE == '1') then
        status = IsTimerConditionMet(Zeros{64}, CNTHV_CVAL_EL2(),
                                     CNTHV_CTL_EL2().IMASK, InterruptID_CNTHV);
        CNTHV_CTL_EL2().ISTATUS = if status then '1' else '0';
    end;

    if ((IsFeatureImplemented(FEAT_SEL2) && IsFeatureImplemented(FEAT_VHE)) &&
          CNTHVS_CTL_EL2().ENABLE == '1') then
        status = IsTimerConditionMet(Zeros{64}, CNTHVS_CVAL_EL2(),
                                     CNTHVS_CTL_EL2().IMASK, InterruptID_CNTHVS);
        CNTHVS_CTL_EL2().ISTATUS = if status then '1' else '0';
    end;
    return;
end;
```

Library pseudocode for shared/functions/counters/CNTHCTL_EL2_VHE

```
// CNTHCTL_EL2_VHE()
// =================
// In the case where EL2 accesses the CNTKCTL_EL1 register, and the access
// is redirected to CNTHCTL_EL2 as a result of HCR_EL2.E2H being 1,
// then the bits of CNTHCTL_EL2 that are RES0 in CNTKCTL_EL1 are
// treated as being UNKNOWN. This function applies the UNKNOWN behavior.

func CNTHCTL_EL2_VHE(original_value : bits(64)) => bits(64)
begin
    assert PSTATE.EL == EL2;
    assert HCR_EL2().E2H == '1';

    var return_value : bits(64) = original_value;
    if !IsFeatureImplemented(FEAT_NV2p1) then
        return_value[19:18] = ARBITRARY : bits(2);
        return_value[16:10] = ARBITRARY : bits(7);
    end;
    return return_value;
end;
```

Library pseudocode for shared/functions/counters/GenericCounterTick

```
// GenericCounterTick()
// ====================
// Increments PhysicalCount value for every clock tick.

func GenericCounterTick()
begin
    var prev_physical_count : bits(64);
    if CNTCR().EN == '0' then
        if !HaveAArch64() then
            AArch32_CheckTimerConditions();
        else
            AArch64_CheckTimerConditions();
        end;
        return;
    end;
    prev_physical_count = PhysicalCountInt();
    if IsFeatureImplemented(FEAT_CNTSC) && CNTCR().SCEN == '1' then
        PhysicalCount = PhysicalCount + ZeroExtend{88}(CNTSCR());
    else
        PhysicalCount[87:24] = PhysicalCount[87:24] + 1;
    end;
    if !HaveAArch64() then
        AArch32_CheckTimerConditions();
    else
        AArch64_CheckTimerConditions();
    end;
    TestEventCNTP(prev_physical_count, PhysicalCountInt());
    TestEventCNTV(prev_physical_count, PhysicalCountInt());
    return;
end;
```

Library pseudocode for shared/functions/counters/IsTimerConditionMet

```
// IsTimerConditionMet()
// =====================

func IsTimerConditionMet(offset : bits(64), compare_value : bits(64),
                         imask : bits(1), intid : InterruptID) => boolean
begin
    var condition_met : boolean;
    var level : Signal;
    condition_met = (UInt(PhysicalCountInt() - offset) - UInt(compare_value)) >= 0;
    level = if condition_met && imask == '0' then HIGH else LOW;
    SetInterruptRequestLevel(intid, level);
    return condition_met;
end;
```

Library pseudocode for shared/functions/counters/SetEventRegister

```
// SetEventRegister()
// ==================
// Sets the Event Register of this PE

func SetEventRegister()
begin
    EventRegister = '1';
    return;
end;
```

Library pseudocode for shared/functions/counters/TestEventCNTP

```
// TestEventCNTP()
// ===============
// Generate Event stream from the physical counter

func TestEventCNTP(prev_physical_count : bits(64), current_physical_count : bits(64))
begin
    var n : integer;
    if CNTHCTL_EL2().EVNTEN == '1' then
        n = UInt(CNTHCTL_EL2().EVNTI);
        if IsFeatureImplemented(FEAT_ECV) && CNTHCTL_EL2().EVNTIS == '1' then
            n = n + 8;
        end;
        if CNTHCTL_EL2().EVNTDIR == '0' then
            if prev_physical_count[n] == '0' && current_physical_count[n] == '1' then
                SetEventRegister();
            end;
        else
            if prev_physical_count[n] == '1' && current_physical_count[n] == '0' then
                SetEventRegister();
            end;
        end;
    end;
    return;
end;
```

Library pseudocode for shared/functions/counters/TestEventCNTV

```
// TestEventCNTV()
// ===============
// Generate Event stream from the virtual counter

func TestEventCNTV(prev_physical_count : bits(64), current_physical_count : bits(64))
begin
    var offset : bits(64);
    var samplebit, previousbit : bit;
    var n : integer;
    if (EffectiveHCR_EL2_E2H()::EffectiveTGE() != '11' &&
          CNTKCTL_EL1().EVNTEN == '1') then
        n = UInt(CNTKCTL_EL1().EVNTI);
        if IsFeatureImplemented(FEAT_ECV) && CNTKCTL_EL1().EVNTIS == '1' then
            n = n + 8;
        end;
        offset      = if HaveEL(EL2) then CNTVOFF_EL2() else Zeros{64};

        samplebit   = (current_physical_count - offset)[n];
        previousbit = (prev_physical_count - offset)[n];
        if CNTKCTL_EL1().EVNTDIR == '0' then
            if previousbit == '0' && samplebit == '1' then SetEventRegister(); end;
        else
            if previousbit == '1' && samplebit == '0' then SetEventRegister(); end;
        end;
    end;
    return;
end;
```

Library pseudocode for shared/functions/counters/VirtualCounterTimer

```
// VirtualCounterTimer()
// =====================
// Returns the Counter-Timer Virtual Count value, the value is as read by CurrentEL to CNTVCT_EL0.

func VirtualCounterTimer() => bits(64)
begin
    var cntvct : bits(64);

    if PSTATE.EL != EL3 then
        if HaveEL(EL2) && !ELIsInHost(PSTATE.EL) then
            cntvct = PhysicalCountInt() - CNTVOFF_EL2();
        else
            cntvct = PhysicalCountInt();
        end;
    else
        if HaveEL(EL2) && !ELUsingAArch32(EL2) then
            cntvct = PhysicalCountInt() - CNTVOFF_EL2();
        elsif HaveEL(EL2) && ELUsingAArch32(EL2) then
            cntvct = PhysicalCountInt() - CNTVOFF();
        else
            cntvct = PhysicalCountInt();
        end;
    end;

    return cntvct;
end;
```

Library pseudocode for shared/functions/crc/BitReverse

```
// BitReverse()
// ============

func BitReverse{N}(data : bits(N)) => bits(N)
begin
    var result : bits(N);
    for i = 0 to N-1 do
        result[(N-i)-1] = data[i];
    end;
    return result;
end;
```

Library pseudocode for shared/functions/crc/Poly32Mod2

```
// Poly32Mod2()
// ============

// Poly32Mod2 on a bitstring does a polynomial Modulus over {0,1} operation

func Poly32Mod2{N}(data_in : bits(N), poly : bits(32)) => bits(32)
begin
    assert N > 32;
    var data : bits(N) = data_in;
    for i = N-1 downto 32 do
        if data[i] == '1' then
            data[i-1:0] = data[i-1:0] XOR (poly::Zeros{i-32});
        end;
    end;
    return data[31:0];
end;
```

Library pseudocode for shared/functions/crypto/AESInvMixColumns

```
// AESInvMixColumns()
// ==================
// Transformation in the Inverse Cipher that is the inverse of AESMixColumns.

func AESInvMixColumns(op : bits (128)) => bits(128)
begin
    let in0 : bits(4*8) = op[ 96+:8] :: op[ 64+:8] :: op[ 32+:8] :: op[  0+:8];
    let in1 : bits(4*8) = op[104+:8] :: op[ 72+:8] :: op[ 40+:8] :: op[  8+:8];
    let in2 : bits(4*8) = op[112+:8] :: op[ 80+:8] :: op[ 48+:8] :: op[ 16+:8];
    let in3 : bits(4*8) = op[120+:8] :: op[ 88+:8] :: op[ 56+:8] :: op[ 24+:8];

    var out0 : bits(4*8);
    var out1 : bits(4*8);
    var out2 : bits(4*8);
    var out3 : bits(4*8);

    for c = 0 to 3 do
        out0[c*8+:8] =  (FFmul0E(in0[c*8+:8]) XOR FFmul0B(in1[c*8+:8]) XOR FFmul0D(in2[c*8+:8]) XOR
                         FFmul09(in3[c*8+:8]));
        out1[c*8+:8] =  (FFmul09(in0[c*8+:8]) XOR FFmul0E(in1[c*8+:8]) XOR FFmul0B(in2[c*8+:8]) XOR
                         FFmul0D(in3[c*8+:8]));
        out2[c*8+:8] =  (FFmul0D(in0[c*8+:8]) XOR FFmul09(in1[c*8+:8]) XOR FFmul0E(in2[c*8+:8]) XOR
                         FFmul0B(in3[c*8+:8]));
        out3[c*8+:8] =  (FFmul0B(in0[c*8+:8]) XOR FFmul0D(in1[c*8+:8]) XOR FFmul09(in2[c*8+:8]) XOR
                         FFmul0E(in3[c*8+:8]));
    end;

    return (
        out3[3*8+:8] :: out2[3*8+:8] :: out1[3*8+:8] :: out0[3*8+:8] ::
        out3[2*8+:8] :: out2[2*8+:8] :: out1[2*8+:8] :: out0[2*8+:8] ::
        out3[1*8+:8] :: out2[1*8+:8] :: out1[1*8+:8] :: out0[1*8+:8] ::
        out3[0*8+:8] :: out2[0*8+:8] :: out1[0*8+:8] :: out0[0*8+:8]
    );
end;
```

Library pseudocode for shared/functions/crypto/AESInvShiftRows

```
// AESInvShiftRows()
// =================
// Transformation in the Inverse Cipher that is inverse of AESShiftRows.

func AESInvShiftRows(op : bits(128)) => bits(128)
begin
    return (
        op[ 31: 24] :: op[ 55: 48] :: op[ 79: 72] :: op[103: 96] ::
        op[127:120] :: op[ 23: 16] :: op[ 47: 40] :: op[ 71: 64] ::
        op[ 95: 88] :: op[119:112] :: op[ 15:  8] :: op[ 39: 32] ::
        op[ 63: 56] :: op[ 87: 80] :: op[111:104] :: op[  7:  0]
    );
end;
```

Library pseudocode for shared/functions/crypto/AESInvSubBytes

```
// AESInvSubBytes()
// ================
// Transformation in the Inverse Cipher that is the inverse of AESSubBytes.

func AESInvSubBytes(op : bits(128)) => bits(128)
begin
    // Inverse S-box values
    let GF2_inv : bits(16*16*8) = (
        /*       F E D C B A 9 8 7 6 5 4 3 2 1 0       */
        /*F*/ 0x7d0c2155631469e126d677ba7e042b17[127:0] ::
        /*E*/ 0x619953833cbbebc8b0f52aae4d3be0a0[127:0] ::
        /*D*/ 0xef9cc9939f7ae52d0d4ab519a97f5160[127:0] ::
        /*C*/ 0x5fec8027591012b131c7078833a8dd1f[127:0] ::
        /*B*/ 0xf45acd78fec0db9a2079d2c64b3e56fc[127:0] ::
        /*A*/ 0x1bbe18aa0e62b76f89c5291d711af147[127:0] ::
        /*9*/ 0x6edf751ce837f9e28535ade72274ac96[127:0] ::
        /*8*/ 0x73e6b4f0cecff297eadc674f4111913a[127:0] ::
        /*7*/ 0x6b8a130103bdafc1020f3fca8f1e2cd0[127:0] ::
        /*6*/ 0x0645b3b80558e4f70ad3bc8c00abd890[127:0] ::
        /*5*/ 0x849d8da75746155edab9edfd5048706c[127:0] ::
        /*4*/ 0x92b6655dcc5ca4d41698688664f6f872[127:0] ::
        /*3*/ 0x25d18b6d49a25b76b224d92866a12e08[127:0] ::
        /*2*/ 0x4ec3fa420b954cee3d23c2a632947b54[127:0] ::
        /*1*/ 0xcbe9dec444438e3487ff2f9b8239e37c[127:0] ::
        /*0*/ 0xfbd7f3819ea340bf38a53630d56a0952[127:0]
    );
    var out : bits(128);
    for i = 0 to 15 do
        out[i*8+:8] = GF2_inv[UInt(op[i*8+:8])*8+:8];
    end;
    return out;
end;
```

Library pseudocode for shared/functions/crypto/AESMixColumns

```
// AESMixColumns()
// ===============
// Transformation in the Cipher that takes all of the columns of the
// State and mixes their data (independently of one another) to
// produce new columns.

func AESMixColumns(op : bits (128)) => bits(128)
begin
    let in0 : bits(4*8) = op[ 96+:8] :: op[ 64+:8] :: op[ 32+:8] :: op[  0+:8];
    let in1 : bits(4*8) = op[104+:8] :: op[ 72+:8] :: op[ 40+:8] :: op[  8+:8];
    let in2 : bits(4*8) = op[112+:8] :: op[ 80+:8] :: op[ 48+:8] :: op[ 16+:8];
    let in3 : bits(4*8) = op[120+:8] :: op[ 88+:8] :: op[ 56+:8] :: op[ 24+:8];

    var out0 : bits(4*8);
    var out1 : bits(4*8);
    var out2 : bits(4*8);
    var out3 : bits(4*8);

    for c = 0 to 3 do
        out0[c*8+:8] = (FFmul02(in0[c*8+:8]) XOR FFmul03(in1[c*8+:8]) XOR
                                in2[c*8+:8]  XOR         in3[c*8+:8]);
        out1[c*8+:8] = (FFmul02(in1[c*8+:8])  XOR FFmul03(in2[c*8+:8]) XOR
                               in3[c*8+:8]   XOR         in0[c*8+:8]);
        out2[c*8+:8] = (FFmul02(in2[c*8+:8])  XOR FFmul03(in3[c*8+:8]) XOR
                               in0[c*8+:8]   XOR         in1[c*8+:8]);
        out3[c*8+:8] = (FFmul02(in3[c*8+:8])  XOR FFmul03(in0[c*8+:8]) XOR
                               in1[c*8+:8]   XOR         in2[c*8+:8]);
    end;

    return (
        out3[3*8+:8] :: out2[3*8+:8] :: out1[3*8+:8] :: out0[3*8+:8] ::
        out3[2*8+:8] :: out2[2*8+:8] :: out1[2*8+:8] :: out0[2*8+:8] ::
        out3[1*8+:8] :: out2[1*8+:8] :: out1[1*8+:8] :: out0[1*8+:8] ::
        out3[0*8+:8] :: out2[0*8+:8] :: out1[0*8+:8] :: out0[0*8+:8]
    );
end;
```

Library pseudocode for shared/functions/crypto/AESShiftRows

```
// AESShiftRows()
// ==============
// Transformation in the Cipher that processes the State by cyclically
// shifting the last three rows of the State by different offsets.

func AESShiftRows(op : bits(128)) => bits(128)
begin
    return (
        op[ 95: 88] :: op[ 55: 48] :: op[ 15:  8] :: op[103: 96] ::
        op[ 63: 56] :: op[ 23: 16] :: op[111:104] :: op[ 71: 64] ::
        op[ 31: 24] :: op[119:112] :: op[ 79: 72] :: op[ 39: 32] ::
        op[127:120] :: op[ 87: 80] :: op[ 47: 40] :: op[  7:  0]
    );
end;
```

Library pseudocode for shared/functions/crypto/AESSubBytes

```
// AESSubBytes()
// =============
// Transformation in the Cipher that processes the State using a nonlinear
// byte substitution table (S-box) that operates on each of the State bytes
// independently.

func AESSubBytes(op : bits(128)) => bits(128)
begin
    // S-box values
    let GF2 : bits(16*16*8) = (
        /*       F E D C B A 9 8 7 6 5 4 3 2 1 0       */
        /*F*/ 0x16bb54b00f2d99416842e6bf0d89a18c[127:0] ::
        /*E*/ 0xdf2855cee9871e9b948ed9691198f8e1[127:0] ::
        /*D*/ 0x9e1dc186b95735610ef6034866b53e70[127:0] ::
        /*C*/ 0x8a8bbd4b1f74dde8c6b4a61c2e2578ba[127:0] ::
        /*B*/ 0x08ae7a65eaf4566ca94ed58d6d37c8e7[127:0] ::
        /*A*/ 0x79e4959162acd3c25c2406490a3a32e0[127:0] ::
        /*9*/ 0xdb0b5ede14b8ee4688902a22dc4f8160[127:0] ::
        /*8*/ 0x73195d643d7ea7c41744975fec130ccd[127:0] ::
        /*7*/ 0xd2f3ff1021dab6bcf5389d928f40a351[127:0] ::
        /*6*/ 0xa89f3c507f02f94585334d43fbaaefd0[127:0] ::
        /*5*/ 0xcf584c4a39becb6a5bb1fc20ed00d153[127:0] ::
        /*4*/ 0x842fe329b3d63b52a05a6e1b1a2c8309[127:0] ::
        /*3*/ 0x75b227ebe28012079a059618c323c704[127:0] ::
        /*2*/ 0x1531d871f1e5a534ccf73f362693fdb7[127:0] ::
        /*1*/ 0xc072a49cafa2d4adf04759fa7dc982ca[127:0] ::
        /*0*/ 0x76abd7fe2b670130c56f6bf27b777c63[127:0]
    );
    var out : bits(128);
    for i = 0 to 15 do
        out[i*8+:8] = GF2[UInt(op[i*8+:8])*8+:8];
    end;
    return out;
end;
```

Library pseudocode for shared/functions/crypto/FFmul02

```
// FFmul02()
// =========

func FFmul02(b : bits(8)) => bits(8)
begin
    let FFmul_02 : bits(256*8) = (
        /*       F E D C B A 9 8 7 6 5 4 3 2 1 0       */
        /*F*/ 0xE5E7E1E3EDEFE9EBF5F7F1F3FDFFF9FB[127:0] ::
        /*E*/ 0xC5C7C1C3CDCFC9CBD5D7D1D3DDDFD9DB[127:0] ::
        /*D*/ 0xA5A7A1A3ADAFA9ABB5B7B1B3BDBFB9BB[127:0] ::
        /*C*/ 0x858781838D8F898B959791939D9F999B[127:0] ::
        /*B*/ 0x656761636D6F696B757771737D7F797B[127:0] ::
        /*A*/ 0x454741434D4F494B555751535D5F595B[127:0] ::
        /*9*/ 0x252721232D2F292B353731333D3F393B[127:0] ::
        /*8*/ 0x050701030D0F090B151711131D1F191B[127:0] ::
        /*7*/ 0xFEFCFAF8F6F4F2F0EEECEAE8E6E4E2E0[127:0] ::
        /*6*/ 0xDEDCDAD8D6D4D2D0CECCCAC8C6C4C2C0[127:0] ::
        /*5*/ 0xBEBCBAB8B6B4B2B0AEACAAA8A6A4A2A0[127:0] ::
        /*4*/ 0x9E9C9A98969492908E8C8A8886848280[127:0] ::
        /*3*/ 0x7E7C7A787674727 06E6C6A6866646260[127:0] ::
        /*2*/ 0x5E5C5A58565452504E4C4A4846444240[127:0] ::
        /*1*/ 0x3E3C3A38363432302E2C2A2826242220[127:0] ::
        /*0*/ 0x1E1C1A18161412100E0C0A0806040200[127:0]
    );
    return FFmul_02[UInt(b)*8+:8];
end;
```

Library pseudocode for shared/functions/crypto/FFmul03

```
// FFmul03()
// =========

func FFmul03(b : bits(8)) => bits(8)
begin
    let FFmul_03 : bits(256*8) = (
        /*       F E D C B A 9 8 7 6 5 4 3 2 1 0       */
        /*F*/ 0x1A191C1F16151013020104070E0D080B[127:0] ::
        /*E*/ 0x2A292C2F26252023323134373E3D383B[127:0] ::
        /*D*/ 0x7A797C7F76757073626164676E6D686B[127:0] ::
        /*C*/ 0x4A494C4F46454043525154575E5D585B[127:0] ::
        /*B*/ 0xDAD9DCDFD6D5D0D3C2C1C4C7CECDC8CB[127:0] ::
        /*A*/ 0xEAE9ECEFE6E5E0E3F2F1F4F7FEFDF8FB[127:0] ::
        /*9*/ 0xBAB9BCBFB6B5B0B3A2A1A4A7AEADA8AB[127:0] ::
        /*8*/ 0x8A898C8F86858083929194979E9D989B[127:0] ::
        /*7*/ 0x818287848D8E8B88999A9F9C95969390[127:0] ::
        /*6*/ 0xB1B2B7B4BDBEBBB8A9AAAFACA5A6A3A0[127:0] ::
        /*5*/ 0xE1E2E7E4EDEEEBE8F9FAFFFCF5F6F3F0[127:0] ::
        /*4*/ 0xD1D2D7D4DDDEDBD8C9CACFCCC5C6C3C0[127:0] ::
        /*3*/ 0x414247444D4E4B48595A5F5C55565350[127:0] ::
        /*2*/ 0x717277747D7E7B78696A6F6C65666360[127:0] ::
        /*1*/ 0x212227242D2E2B28393A3F3C35363330[127:0] ::
        /*0*/ 0x111217141D1E1B18090A0F0C05060300[127:0]
    );
    return FFmul_03[UInt(b)*8+:8];
end;
```

Library pseudocode for shared/functions/crypto/FFmul09

```
// FFmul09()
// =========

func FFmul09(b : bits(8)) => bits(8)
begin
    let FFmul_09 : bits(256*8) = (
        /*       F E D C B A 9 8 7 6 5 4 3 2 1 0       */
        /*F*/ 0x464F545D626B70790E071C152A233831[127:0] ::
        /*E*/ 0xD6DFC4CDF2FBE0E99E978C85BAB3A8A1[127:0] ::
        /*D*/ 0x7D746F6659504B42353C272E1118030A[127:0] ::
        /*C*/ 0xEDE4FFF6C9C0DBD2A5ACB7BE8188939A[127:0] ::
        /*B*/ 0x3039222B141D060F78716A635C554E47[127:0] ::
        /*A*/ 0xA0A9B2BB848D969FE8E1FAF3CCC5DED7[127:0] ::
        /*9*/ 0x0B0219102F263D34434A5158676E757C[127:0] ::
        /*8*/ 0x9B928980BFB6ADA4D3DAC1C8F7FEE5EC[127:0] ::
        /*7*/ 0xAAA3B8B18E879C95E2EBF0F9C6CFD4DD[127:0] ::
        /*6*/ 0x3A3328211E170C05727B6069565F444D[127:0] ::
        /*5*/ 0x9198838AB5BCA7AED9D0CBC2FDF4EFE6[127:0] ::
        /*4*/ 0x0108131A252C373E49405B526D647F76[127:0] ::
        /*3*/ 0xDCD5CEC7F8F1EAE3949D868FB0B9A2AB[127:0] ::
        /*2*/ 0x4C455E5768617A73040D161F2029323B[127:0] ::
        /*1*/ 0xE7EEF5FCC3CAD1D8AFA6BDB48B829990[127:0] ::
        /*0*/ 0x777E656C535A41483F362D241B120900[127:0]
    );
    return FFmul_09[UInt(b)*8+:8];
end;
```

Library pseudocode for shared/functions/crypto/FFmul0B

```
// FFmul0B()
// =========

func FFmul0B(b : bits(8)) => bits(8)
begin
    let FFmul_0B : bits(256*8) = (
        /*       F E D C B A 9 8 7 6 5 4 3 2 1 0       */
        /*F*/ 0xA3A8B5BE8F849992FBF0EDE6D7DCC1CA[127:0] ::
        /*E*/ 0x1318050E3F3429224B405D56676C717A[127:0] ::
        /*D*/ 0xD8D3CEC5F4FFE2E9808B969DACA7BAB1[127:0] ::
        /*C*/ 0x68637E75444F5259303B262D1C170A01[127:0] ::
        /*B*/ 0x555E434879726F640D061B10212A373C[127:0] ::
        /*A*/ 0xE5EEF3F8C9C2DFD4BDB6ABA0919A878C[127:0] ::
        /*9*/ 0x2E2538330209141F767D606B5A514C47[127:0] ::
        /*8*/ 0x9E958883B2B9A4AFC6CDD0DBEAE1FCF7[127:0] ::
        /*7*/ 0x545F424978736E650C071A11202B363D[127:0] ::
        /*6*/ 0xE4EFF2F9C8C3DED5BCB7AAA1909B868D[127:0] ::
        /*5*/ 0x2F2439320308151E777C616A5B504D46[127:0] ::
        /*4*/ 0x9F948982B3B8A5AEC7CCD1DAEBE0FDF6[127:0] ::
        /*3*/ 0xA2A9B4BF8E859893FAF1ECE7D6DDC0CB[127:0] ::
        /*2*/ 0x1219040F3E3528234A415C57666D707B[127:0] ::
        /*1*/ 0xD9D2CFC4F5FEE3E8818A979CADA6BBB0[127:0] ::
        /*0*/ 0x69627F74454E5358313A272C1D160B00[127:0]
    );
    return FFmul_0B[UInt(b)*8+:8];
end;
```

Library pseudocode for shared/functions/crypto/FFmul0D

```
// FFmul0D()
// =========

func FFmul0D(b : bits(8)) => bits(8)
begin
    let FFmul_0D : bits(256*8) = (
        /*       F E D C B A 9 8 7 6 5 4 3 2 1 0       */
        /*F*/ 0x979A8D80A3AEB9B4FFF2E5E8CBC6D1DC[127:0] ::
        /*E*/ 0x474A5D50737E69642F2235381B16010C[127:0] ::
        /*D*/ 0x2C21363B1815020F44495E53707D6A67[127:0] ::
        /*C*/ 0xFCF1E6EBC8C5D2DF94998E83A0ADBAB7[127:0] ::
        /*B*/ 0xFAF7E0EDCEC3D4D9929F8885A6ABBCB1[127:0] ::
        /*A*/ 0x2A27303D1E130409424F5855767B6C61[127:0] ::
        /*9*/ 0x414C5B5675786F622924333E1D10070A[127:0] ::
        /*8*/ 0x919C8B86A5A8BFB2F9F4E3EECDC0D7DA[127:0] ::
        /*7*/ 0x4D40575A7974636E25283F32111C0B06[127:0] ::
        /*6*/ 0x9D90878AA9A4B3BEF5F8EFE2C1CCDBD6[127:0] ::
        /*5*/ 0xF6FBECE1C2CFD8D59E938489AAA7B0BD[127:0] ::
        /*4*/ 0x262B3C31121F08054E4354597A77606D[127:0] ::
        /*3*/ 0x202D3A3714190E034845525F7C71666B[127:0] ::
        /*2*/ 0xF0FDEAE7C4C9DED39895828FACA1B6BB[127:0] ::
        /*1*/ 0x9B96818CAFA2B5B8F3FEE9E4C7CADDD0[127:0] ::
        /*0*/ 0x4B46515C7F726568232E3934171A0D00[127:0]
    );
    return FFmul_0D[UInt(b)*8+:8];
end;
```

Library pseudocode for shared/functions/crypto/FFmul0E

```
// FFmul0E()
// =========

func FFmul0E(b : bits(8)) => bits(8)
begin
    let FFmul_0E : bits(256*8) = (
        /*       F E D C B A 9 8 7 6 5 4 3 2 1 0       */
        /*F*/ 0x8D83919FB5BBA9A7FDF3E1EFC5CBD9D7[127:0] ::
        /*E*/ 0x6D63717F555B49471D13010F252B3937[127:0] ::
        /*D*/ 0x56584A446E60727C26283A341E10020C[127:0] ::
        /*C*/ 0xB6B8AAA48E80929CC6C8DAD4FEF0E2EC[127:0] ::
        /*B*/ 0x202E3C321816040A505E4C426866747A[127:0] ::
        /*A*/ 0xC0CEDCD2F8F6E4EAB0BEACA28886949A[127:0] ::
        /*9*/ 0xFBF5E7E9C3CDDFD18B859799B3BDAFA1[127:0] ::
        /*8*/ 0x1B150709232D3F316B657779535D4F41[127:0] ::
        /*7*/ 0xCCC2D0DEF4FAE8E6BCB2A0AE848A9896[127:0] ::
        /*6*/ 0x2C22303E141A08065C52404E646A7876[127:0] ::
        /*5*/ 0x17190B052F21333D67697B755F51434D[127:0] ::
        /*4*/ 0xF7F9EBE5CFC1D3DD87899B95BFB1A3AD[127:0] ::
        /*3*/ 0x616F7D735957454B111F0D032927353B[127:0] ::
        /*2*/ 0x818F9D93B9B7A5ABF1FFEDE3C9C7D5DB[127:0] ::
        /*1*/ 0xBAB4A6A8828C9E90CAC4D6D8F2FCEEE0[127:0] ::
        /*0*/ 0x5A544648626C7E702A243638121C0E00[127:0]
    );
    return FFmul_0E[UInt(b)*8+:8];
end;
```

Library pseudocode for shared/functions/crypto/SHA256hash

```
// SHA256hash()
// ============

func SHA256hash(x_in : bits (128), y_in : bits(128), w : bits(128),
                part1 : boolean) => bits(128)
begin
    var chs, maj, t : bits(32);
    var x : bits(128) = x_in;
    var y : bits(128) = y_in;

    for e = 0 to 3 do
        chs = SHAchoose(y[31:0], y[63:32], y[95:64]);
        maj = SHAmajority{32}(x[31:0], x[63:32], x[95:64]);
        t   = y[127:96] + SHA256hashSIGMA1(y[31:0]) + chs + w[e*:32];
        x[127:96] = t + x[127:96];
        y[127:96] = t + SHA256hashSIGMA0(x[31:0]) + maj;
        let yx : bits(256) = ROL(y :: x, 32);
        (y, x) = (yx[128+:128], yx[0+:128]);
    end;
    return (if part1 then x else y);
end;
```

Library pseudocode for shared/functions/crypto/SHA256hashSIGMA0

```
// SHA256hashSIGMA0()
// ==================

func SHA256hashSIGMA0(x : bits(32)) => bits(32)
begin
    return ROR(x, 2) XOR ROR(x, 13) XOR ROR(x, 22);
end;
```

Library pseudocode for shared/functions/crypto/SHA256hashSIGMA1

```
// SHA256hashSIGMA1()
// ==================

func SHA256hashSIGMA1(x : bits(32)) => bits(32)
begin
    return ROR(x, 6) XOR ROR(x, 11) XOR ROR(x, 25);
end;
```

Library pseudocode for shared/functions/crypto/SHAchoose

```
// SHAchoose()
// ===========
func SHAchoose(x : bits(32), y : bits(32), z : bits(32)) => bits(32)
begin
    return (((y XOR z) AND x) XOR z);
end;
```

Library pseudocode for shared/functions/crypto/SHAmajority

```
// SHAmajority()
// =============

func SHAmajority{N}(x : bits(N), y : bits(N), z : bits(N)) => bits(N)
begin
    assert N IN {32, 64};
    return ((x AND y) OR ((x OR y) AND z));
end;
```

Library pseudocode for shared/functions/crypto/SHAparity

```
// SHAparity()
// ===========

func SHAparity(x : bits(32), y : bits(32), z : bits(32)) => bits(32)
begin
    return (x XOR y XOR z);
end;
```

Library pseudocode for shared/functions/crypto/Sbox

```
// Sbox()
// ======
// Used in SM4E crypto instruction

func Sbox(sboxin : bits(8)) => bits(8)
begin
    var sboxout : bits(8);
    let sboxstring : bits(2048) = (
        /*       F E D C B A 9 8 7 6 5 4 3 2 1 0       */
        /*F*/ 0xd690e9fecce13db716b614c228fb2c05[127:0] ::
        /*E*/ 0x2b679a762abe04c3aa44132649860699[127:0] ::
        /*D*/ 0x9c4250f491ef987a33540b43edcfac62[127:0] ::
        /*C*/ 0xe4b31ca9c908e89580df94fa758f3fa6[127:0] ::
        /*B*/ 0x4707a7fcf37317ba83593c19e6854fa8[127:0] ::
        /*A*/ 0x686b81b27164da8bf8eb0f4b70569d35[127:0] ::
        /*9*/ 0x1e240e5e6358d1a225227c3b01217887[127:0] ::
        /*8*/ 0xd40046579fd327524c3602e7a0c4c89e[127:0] ::
        /*7*/ 0xeabf8ad240c738b5a3f7f2cef96115a1[127:0] ::
        /*6*/ 0xe0ae5da49b341a55ad933230f58cb1e3[127:0] ::
        /*5*/ 0x1df6e22e8266ca60c02923ab0d534e6f[127:0] ::
        /*4*/ 0xd5db3745defd8e2f03ff6a726d6c5b51[127:0] ::
        /*3*/ 0x8d1baf92bbddbc7f11d95c411f105ad8[127:0] ::
        /*2*/ 0x0ac13188a5cd7bbd2d74d012b8e5b4b0[127:0] ::
        /*1*/ 0x8969974a0c96777e65b9f109c56ec684[127:0] ::
        /*0*/ 0x18f07dec3adc4d2079ee5f3ed7cb3948[127:0]
    );
    let sboxindex : integer = 255 - UInt(sboxin);
    sboxout = sboxstring[sboxindex*:8];
    return sboxout;
end;
```

Library pseudocode for shared/functions/decode/DecodeType

```
// DecodeType
// ==========

type DecodeType of enumeration {
    Decode_UNDEF,
    Decode_NOP,
    Decode_OK
};
```

Library pseudocode for shared/functions/decode/EndOfDecode

```
// EndOfDecode()
// =============
// This function is invoked to end the Decode phase and performs Branch target Checks
// before taking any UNDEFINED exceptions, NOPs, or continuing to execute.

func EndOfDecode(reason : DecodeType)
begin
    if IsFeatureImplemented(FEAT_BTI) && !UsingAArch32() then
        BranchTargetCheck();
    end;
    case reason of
        when Decode_NOP   => ExecuteAsNOP();
        when Decode_UNDEF => Undefined();
        when Decode_OK    => pass;          // Continue to execute.
    end;
    return;
end;
```

Library pseudocode for shared/functions/exclusive/ClearExclusiveByAddress

```
// ClearExclusiveByAddress()
// =========================
// Clear the global Exclusives monitors for all PEs EXCEPT processorid if they
// record any part of the physical address region of size bytes starting at paddress.
// It is IMPLEMENTATION DEFINED whether the global Exclusives monitor for processorid
// is also cleared if it records any part of the address region.

impdef func ClearExclusiveByAddress(paddress : FullAddress,
                                    processorid : integer,
                                    size : integer)
begin
    return;
end;
```

Library pseudocode for shared/functions/exclusive/ClearExclusiveLocal

```
// ClearExclusiveLocal()
// =====================
// Clear the local Exclusives monitor for the specified processorid.

impdef func ClearExclusiveLocal(processorid : integer)
begin
    return;
end;
```

Library pseudocode for shared/functions/exclusive/ExclusiveMonitorsStatus

```
// ExclusiveMonitorsStatus()
// =========================
// Returns '0' to indicate success if the last memory write by this PE was to
// the same physical address region endorsed by ExclusiveMonitorsPass().
// Returns '1' to indicate failure if address translation resulted in a different
// physical address.

impdef func ExclusiveMonitorsStatus() => bit
begin
    return '0';
end;
```

Library pseudocode for shared/functions/exclusive/IsExclusiveGlobal

```
// IsExclusiveGlobal()
// ===================
// Return TRUE if the global Exclusives monitor for processorid includes all of
// the physical address region of size bytes starting at paddress.


impdef func IsExclusiveGlobal(paddress : FullAddress,
                              processorid : integer,
                              size : integer) => boolean
begin
    return TRUE;
end;
```

Library pseudocode for shared/functions/exclusive/IsExclusiveLocal

```
// IsExclusiveLocal()
// ==================
// Return TRUE if the local Exclusives monitor for processorid includes all of
// the physical address region of size bytes starting at paddress.

impdef func IsExclusiveLocal(paddress : FullAddress, processorid : integer,
                             size : integer) => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/exclusive/MarkExclusiveGlobal

```
// MarkExclusiveGlobal()
// =====================
// Record the physical address region of size bytes starting at paddress in
// the global Exclusives monitor for processorid.

impdef func MarkExclusiveGlobal(paddress : FullAddress,
                                processorid : integer,
                                size : integer)
begin
    return;
end;
```

Library pseudocode for shared/functions/exclusive/MarkExclusiveLocal

```
// MarkExclusiveLocal()
// ====================
// Record the physical address region of size bytes starting at paddress in
// the local Exclusives monitor for processorid.

impdef func MarkExclusiveLocal(paddress : FullAddress, processorid : integer, size : integer)
begin
    return;
end;
```

Library pseudocode for shared/functions/exclusive/ProcessorID

```
// ProcessorID()
// =============
// Return the ID of the currently executing PE.

impdef func ProcessorID() => integer
begin
    if HaveAArch64() then
        return UInt(MPIDR_EL1().Aff3 :: MPIDR_EL1().Aff2 :: MPIDR_EL1().Aff1 :: MPIDR_EL1().Aff0);
    else
        return UInt(MPIDR().Aff2 :: MPIDR().Aff1 :: MPIDR().Aff0);
    end;
end;
```

Library pseudocode for shared/functions/externalaborts/ActionRequired

```
// ActionRequired()
// ================
// Return an implementation specific value:
// returns TRUE if action is required, FALSE otherwise.

impdef func ActionRequired() => boolean
begin
    return TRUE;
end;
```

Library pseudocode for shared/functions/externalaborts/ClearPendingDelegatedSError

```
// ClearPendingDelegatedSError()
// =============================
// Clear a pending delegated SError interrupt.

func ClearPendingDelegatedSError()
begin
    assert IsFeatureImplemented(FEAT_E3DSE);
    SCR_EL3().DSE = '0';
end;
```

Library pseudocode for shared/functions/externalaborts/ClearPendingPhysicalSError

```
// ClearPendingPhysicalSError()
// ============================
// Clear a pending physical SError interrupt.

impdef func ClearPendingPhysicalSError()
begin
    return;
end;
```

Library pseudocode for shared/functions/externalaborts/ClearPendingVirtualSError

```
// ClearPendingVirtualSError()
// ===========================
// Clear a pending virtual SError interrupt.

func ClearPendingVirtualSError()
begin
    if ELUsingAArch32(EL2) then
        HCR().VA      = '0';
    else
        HCR_EL2().VSE = '0';
    end;
end;
```

Library pseudocode for shared/functions/externalaborts/ErrorIsContained

```
// ErrorIsContained()
// ==================
// Return an implementation specific value:
// TRUE if Error is contained by the PE, FALSE otherwise.

impdef func ErrorIsContained() => boolean
begin
    return TRUE;
end;
```

Library pseudocode for shared/functions/externalaborts/ErrorIsSynchronized

```
// ErrorIsSynchronized()
// =====================
// Return an implementation specific value:
// returns TRUE if Error is synchronized by any synchronization event
// FALSE otherwise.

impdef func ErrorIsSynchronized() => boolean
begin
    return TRUE;
end;
```

Library pseudocode for shared/functions/externalaborts/ExtAbortToAArch64

```
// ExtAbortToAArch64()
// ===================
// Returns TRUE if synchronous exception is being taken to an Exception level using AArch64_

func ExtAbortToAArch64(fault : FaultRecord) => boolean
begin
    assert IsExternalSyncAbort(fault.statuscode);

    return !ELUsingAArch32(SyncExternalAbortTarget(fault));
end;
```

Library pseudocode for shared/functions/externalaborts/ExternalAbort

```
// ExternalAbort()
// ===============

func ExternalAbort(fault : FaultRecord) recurselimit 1
begin
    if IsExternalSyncAbort(fault) then
        if UsingAArch32() then
            AArch32_Abort(fault);
        else
            AArch64_Abort(fault);
        end;

    else
        PendSErrorInterrupt(fault);
    end;
end;
```

Library pseudocode for shared/functions/externalaborts/ExternalFault

```
// ExternalFault()
// ===============
// Return a fault recording indicating a fault for a Synchronous/Asynchronous External abort.

func ExternalFault(memretstatus : PhysMemRetStatus, iswrite : boolean,
                   memaddrdesc : AddressDescriptor, size : integer,
                   accdesc : AccessDescriptor) => FaultRecord
begin
    assert (memretstatus.statuscode IN {Fault_SyncExternal, Fault_AsyncExternal} ||
           (!IsFeatureImplemented(FEAT_RAS) && memretstatus.statuscode IN {Fault_SyncParity,
                                                                           Fault_AsyncParity}));

    var fault : FaultRecord = NoFault(accdesc, memaddrdesc.vaddress);
    fault.statuscode = memretstatus.statuscode;
    fault.write      = iswrite;
    fault.extflag    = memretstatus.extflag;
    // It is implementation specific whether External aborts signaled
    // in-band synchronously are taken synchronously or asynchronously
    if (IsExternalSyncAbort(fault) &&
          ((IsFeatureImplemented(FEAT_RASv2) && ExtAbortToAArch64(fault) &&
            PEErrorState(fault) IN {ErrorState_UC, ErrorState_UEU}) ||
           !IsExternalAbortTakenSynchronously(memretstatus, iswrite, memaddrdesc,
                                              size, accdesc))) then
        if fault.statuscode == Fault_SyncParity then
            fault.statuscode = Fault_AsyncParity;
        else
            fault.statuscode = Fault_AsyncExternal;
        end;
    end;

    if IsFeatureImplemented(FEAT_RAS) then
        fault.merrorstate = memretstatus.merrorstate;
    end;
    fault.paddress = memaddrdesc.paddress;
    return fault;
end;
```

Library pseudocode for shared/functions/externalaborts/FaultIsCorrected

```
// FaultIsCorrected()
// ==================
// Return an implementation specific value:
// TRUE if fault is corrected by the PE, FALSE otherwise.

readonly impdef func FaultIsCorrected() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/externalaborts/GetPendingPhysicalSError

```
// GetPendingPhysicalSError()
// ==========================
// Returns the FaultRecord containing details of pending Physical SError
// interrupt.

impdef func GetPendingPhysicalSError() => FaultRecord
begin
    return ARBITRARY : FaultRecord;
end;
```

Library pseudocode for shared/functions/externalaborts/HandleExternalAbort

```
// HandleExternalAbort()
// =====================
// Takes a Synchronous/Asynchronous abort based on fault.

func HandleExternalAbort(memretstatus : PhysMemRetStatus, iswrite : boolean,
                         memaddrdesc : AddressDescriptor, size : integer,
                         accdesc : AccessDescriptor)
begin
    assert (memretstatus.statuscode IN {Fault_SyncExternal, Fault_AsyncExternal} ||
           (!IsFeatureImplemented(FEAT_RAS) && memretstatus.statuscode IN {Fault_SyncParity,
                                                                           Fault_AsyncParity}));

    let fault : FaultRecord = ExternalFault(memretstatus, iswrite, memaddrdesc, size, accdesc);
    ExternalAbort(fault);
end;
```

Library pseudocode for shared/functions/externalaborts/HandleExternalReadAbort

```
// HandleExternalReadAbort()
// =========================
// Wrapper function for HandleExternalAbort function in case of an External
// Abort on memory read.

func HandleExternalReadAbort(memstatus : PhysMemRetStatus, memaddrdesc : AddressDescriptor,
                             size : integer, accdesc : AccessDescriptor)
begin
    let iswrite : boolean = FALSE;
    HandleExternalAbort(memstatus, iswrite, memaddrdesc, size, accdesc);
end;
```

Library pseudocode for shared/functions/externalaborts/HandleExternalTTWAbort

```
// HandleExternalTTWAbort()
// ========================
// Take Asynchronous abort or update FaultRecord for Translation Table Walk
// based on PhysMemRetStatus.

func HandleExternalTTWAbort(memretstatus : PhysMemRetStatus, iswrite : boolean,
                            memaddrdesc : AddressDescriptor,
                            accdesc : AccessDescriptor, size : integer,
                            input_fault : FaultRecord) => FaultRecord
begin
    var output_fault : FaultRecord = input_fault;
    output_fault.extflag    = memretstatus.extflag;
    output_fault.statuscode = memretstatus.statuscode;
    if (IsExternalSyncAbort(output_fault) &&
          ((IsFeatureImplemented(FEAT_RASv2) && ExtAbortToAArch64(output_fault) &&
            PEErrorState(output_fault) IN {ErrorState_UC, ErrorState_UEU}) ||
           !IsExternalAbortTakenSynchronously(memretstatus, iswrite, memaddrdesc,
                                              size, accdesc))) then
        if output_fault.statuscode == Fault_SyncParity then
            output_fault.statuscode = Fault_AsyncParity;
        else
            output_fault.statuscode = Fault_AsyncExternal;
        end;
    end;

    // If a synchronous fault is on a translation table walk, then update the fault type.
    if IsExternalSyncAbort(output_fault) then
        if output_fault.statuscode == Fault_SyncParity then
            output_fault.statuscode = Fault_SyncParityOnWalk;
        else
            output_fault.statuscode = Fault_SyncExternalOnWalk;
        end;
    end;
    if IsFeatureImplemented(FEAT_RAS) then
        output_fault.merrorstate = memretstatus.merrorstate;
    end;
    if !IsExternalSyncAbort(output_fault) then
        PendSErrorInterrupt(output_fault);
        output_fault.statuscode = Fault_None;
    end;
    output_fault.paddress = memaddrdesc.paddress;
    return output_fault;
end;
```

Library pseudocode for shared/functions/externalaborts/HandleExternalWriteAbort

```
// HandleExternalWriteAbort()
// ==========================
// Wrapper function for HandleExternalAbort function in case of an External
// Abort on memory write.

func HandleExternalWriteAbort(memstatus : PhysMemRetStatus, memaddrdesc : AddressDescriptor,
                              size : integer, accdesc : AccessDescriptor)
begin
    let iswrite : boolean = TRUE;
    HandleExternalAbort(memstatus, iswrite, memaddrdesc, size, accdesc);
end;
```

Library pseudocode for shared/functions/externalaborts/IsDelegatedSErrorPending

```
// IsDelegatedSErrorPending()
// ==========================
// Return TRUE if a delegated SError interrupt is pending.

func IsDelegatedSErrorPending() => boolean
begin
    return SCR_EL3().DSE == '1';
end;
```

Library pseudocode for shared/functions/externalaborts/IsExternalAbortTakenSynchronously

```
// IsExternalAbortTakenSynchronously()
// ===================================
// Return an implementation specific value:
// TRUE if the fault returned for the access can be taken synchronously,
// FALSE otherwise.
//
// This might vary between accesses, for example depending on the error type
// or memory type being accessed.
// External aborts on data accesses and translation table walks on data accesses
// can be either synchronous or asynchronous.
//
// When FEAT_DoubleFault is not implemented, External aborts on instruction
// fetches and translation table walks on instruction fetches can be either
// synchronous or asynchronous.
// When FEAT_DoubleFault is implemented, all External abort exceptions on
// instruction fetches and translation table walks on instruction fetches
// must be synchronous.


impdef func IsExternalAbortTakenSynchronously(memstatus : PhysMemRetStatus, iswrite : boolean,
                                              desc : AddressDescriptor, size : integer,
                                              accdesc : AccessDescriptor) => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/externalaborts/IsPhysicalSErrorPending

```
// IsPhysicalSErrorPending()
// =========================
// Returns TRUE if a physical SError interrupt is pending.

readonly impdef func IsPhysicalSErrorPending() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/externalaborts/IsSErrorEdgeTriggered

```
// IsSErrorEdgeTriggered()
// =======================
// Returns TRUE if the physical SError interrupt is edge-triggered
// and FALSE otherwise.

func IsSErrorEdgeTriggered() => boolean
begin
    if IsFeatureImplemented(FEAT_DoubleFault) then
        return TRUE;
    else
        return ImpDefBool("Edge-triggered SError");
    end;
end;
```

Library pseudocode for shared/functions/externalaborts/IsSynchronizablePhysicalSErrorPending

```
// IsSynchronizablePhysicalSErrorPending()
// =======================================
// Returns TRUE if a synchronizable physical SError interrupt is pending.

impdef func IsSynchronizablePhysicalSErrorPending() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/externalaborts/IsVirtualSErrorPending

```
// IsVirtualSErrorPending()
// ========================
// Return TRUE if a virtual SError interrupt is pending.

readonly func IsVirtualSErrorPending() => boolean
begin
    if ELUsingAArch32(EL2) then
        return HCR().VA == '1';
    else
        return HCR_EL2().VSE == '1';
    end;
end;
```

Library pseudocode for shared/functions/externalaborts/PEErrorState

```
// PEErrorState()
// ==============
// Returns the error state of the PE on taking an error exception:
// The PE error state reported to software through the exception syndrome also
// depends on how the exception is taken, and so might differ from the value
// returned from this function.

func PEErrorState(fault : FaultRecord) => ErrorState
begin
    assert !FaultIsCorrected();
    if (!ErrorIsContained() ||
        (!ErrorIsSynchronized() && !StateIsRecoverable()) ||
         ReportErrorAsUC()) then
        return ErrorState_UC;
    end;

    if !StateIsRecoverable() || ReportErrorAsUEU() then
        return ErrorState_UEU;
    end;

    if ActionRequired() || ReportErrorAsUER() then
        return ErrorState_UER;
    end;

    return ErrorState_UEO;
end;
```

Library pseudocode for shared/functions/externalaborts/PendSErrorInterrupt

```
// PendSErrorInterrupt()
// =====================
// Pend the SError Interrupt.

impdef func PendSErrorInterrupt(fault : FaultRecord)
begin
    return;
end;
```

Library pseudocode for shared/functions/externalaborts/ReportErrorAsIMPDEF

```
// ReportErrorAsIMPDEF()
// =====================
// Return an implementation specific value:
// returns TRUE if Error is IMPDEF, FALSE otherwise.

impdef func ReportErrorAsIMPDEF() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/externalaborts/ReportErrorAsUC

```
// ReportErrorAsUC()
// =================
// Return an implementation specific value:
// returns TRUE if Error is Uncontainable, FALSE otherwise.

impdef func ReportErrorAsUC() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/externalaborts/ReportErrorAsUER

```
// ReportErrorAsUER()
// ==================
// Return an implementation specific value:
// returns TRUE if Error is Recoverable, FALSE otherwise.

impdef func ReportErrorAsUER() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/externalaborts/ReportErrorAsUEU

```
// ReportErrorAsUEU()
// ==================
// Return an implementation specific value:
// returns TRUE if Error is Unrecoverable, FALSE otherwise.

impdef func ReportErrorAsUEU() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/externalaborts/ReportErrorAsUncategorized

```
// ReportErrorAsUncategorized()
// ============================
// Return an implementation specific value:
// returns TRUE if Error is uncategorized, FALSE otherwise.

impdef func ReportErrorAsUncategorized() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/externalaborts/StateIsRecoverable

```
// StateIsRecoverable()
// ====================
// Return an implementation specific value:
// returns TRUE is PE State is unrecoverable else FALSE.

impdef func StateIsRecoverable() => boolean
begin
    return TRUE;
end;
```

Library pseudocode for shared/functions/float/bfloat/BFAdd

```
// BFAdd()
// =======
// Non-widening BFloat16 addition used by SVE2 instructions.

func BFAdd{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let fpexc : boolean = TRUE;
    return BFAdd{N}(op1, op2, fpcr, fpexc);
end;

// BFAdd()
// =======
// Non-widening BFloat16 addition following computational behaviors
// corresponding to instructions that read and write BFloat16 values.
// Calculates op1 + op2.
// The 'fpcr' argument supplies the FPCR control bits.

func BFAdd{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type, fpexc : boolean) => bits(N)
begin
    assert N == 16;
    let rounding : FPRounding = FPRoundingMode(fpcr);
    var done : boolean;
    var result : bits(2*N);

    let op1_s : bits(2*N) = op1 :: Zeros{N};
    let op2_s : bits(2*N) = op2 :: Zeros{N};
    let (type1,sign1,value1) = FPUnpack{2*N}(op1_s, fpcr, fpexc);
    let (type2,sign2,value2) = FPUnpack{2*N}(op2_s, fpcr, fpexc);

    (done,result) = FPProcessNaNs{2*N}(type1, type2, op1_s, op2_s, fpcr, fpexc);

    if !done then
        let inf1  = (type1 == FPType_Infinity);
        let inf2  = (type2 == FPType_Infinity);
        let zero1 = (type1 == FPType_Zero);
        let zero2 = (type2 == FPType_Zero);

        if inf1 && inf2 && sign1 == NOT(sign2) then
            result = FPDefaultNaN{2*N}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        elsif (inf1 && sign1 == '0') || (inf2 && sign2 == '0') then
            result = FPInfinity{2*N}('0');
        elsif (inf1 && sign1 == '1') || (inf2 && sign2 == '1') then
            result = FPInfinity{2*N}('1');
        elsif zero1 && zero2 && sign1 == sign2 then
            result = FPZero{2*N}(sign1);
        else
            let result_value : real = value1 + value2;
            if result_value == 0.0 then  // Sign of exact zero result depends on rounding mode
                let result_sign : bit = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{2*N}(result_sign);
            else
                result = FPRoundBF{2*N}(result_value, fpcr, rounding, fpexc);
            end;
        end;

        if fpexc then FPProcessDenorms(type1, type2, 2*N, fpcr); end;
    end;


    return result[2*N-1:N];
end;
```

Library pseudocode for shared/functions/float/bfloat/BFAdd_ZA

```
// BFAdd_ZA()
// ==========
// Non-widening BFloat16 addition used by SME2 ZA-targeting instructions.

func BFAdd_ZA(op1 : bits(16), op2 : bits(16), fpcr_in : FPCR_Type) => bits(16)
begin
    let fpexc : boolean = FALSE;
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.DN = '1';          // Generate default NaN values
    return BFAdd{16}(op1, op2, fpcr, fpexc);
end;
```

Library pseudocode for shared/functions/float/bfloat/BFClampScale

```
// BFClampScale()
// ==============

func BFClampScale(op : bits(16), scale_in : integer) => integer
begin
    let E : integer = 8;
    let F : integer = 7;
    let exp : integer = UInt(op[14:7]);
    let emax : integer = (1 << E) - 1;
    let min_scale : integer = -(F + 1);
    let max_scale : integer = emax + (F + 1);

    let scale : integer = Max(min_scale - exp, Min(scale_in, max_scale - exp));
    return scale;
end;
```

Library pseudocode for shared/functions/float/bfloat/BFDotAdd

```
// BFDotAdd()
// ==========
// BFloat16 2-way dot-product and add to single-precision
// result = addend + op1_a*op2_a + op1_b*op2_b

func BFDotAdd(addend : bits(32), op1_a : bits(16), op1_b : bits(16),
              op2_a : bits(16), op2_b : bits(16), fpcr_in : FPCR_Type) => bits(32)
begin
    var fpcr : FPCR_Type = fpcr_in;
    var prod : bits(32);

    var result : bits(32);
    if !IsFeatureImplemented(FEAT_EBF16) || fpcr.EBF == '0' then   // Standard BFloat16 behaviors
        prod   = FPAdd_BF16(BFMulH(op1_a, op2_a, fpcr), BFMulH(op1_b, op2_b, fpcr), fpcr);
        result = FPAdd_BF16(addend, prod, fpcr);
    else                                                           // Extended BFloat16 behaviors
        let isbfloat16 : boolean = TRUE;
        let fpexc : boolean = FALSE;    // Do not generate floating-point exceptions
        fpcr.DN = '1';                  // Generate default NaN values
        prod    = FPDot(op1_a, op1_b, op2_a, op2_b, fpcr, isbfloat16, fpexc);
        result  = FPAdd{32}(addend, prod, fpcr, fpexc);
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/bfloat/BFInfinity

```
// BFInfinity()
// ============

func BFInfinity{N}(sign : bit) => bits(N)
begin
    assert N == 16;
    let E : integer{} = 8;
    let F : integer{} = N - (E + 1);
    return sign :: Ones{E} :: Zeros{F};
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMatMulAdd

```
// BFMatMulAdd()
// =============
// BFloat16 matrix multiply and add to same precision matrix
// result[2, 2] = addend[2, 2] + (op1[2, 2] * op2[2, 2])

func BFMatMulAdd(addend : bits(64), op1 : bits(64), op2 : bits(64), fpcr : FPCR_Type) => bits(64)
begin

    var result : bits(64);

    for i = 0 to 1 do
        for j = 0 to 1 do
            let sum : bits(16)    = addend[(2*i + j)*:16];
            let prod0 : bits(16)  = BFMul(op1[(2*i + 0)*:16],
                                          op2[(2*j + 0)*:16], fpcr);
            let prod1 : bits(16)  = BFMul(op1[(2*i + 1)*:16],
                                          op2[(2*j + 1)*:16], fpcr);
            result[(2*i + j)*:16] = BFAdd{16}(sum, BFAdd{16}(prod0, prod1, fpcr), fpcr);
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMatMulAddH

```
// BFMatMulAddH()
// ==============
// BFloat16 matrix multiply and add to single-precision matrix
// result[2, 2] = addend[2, 2] + (op1[2, 4] * op2[4, 2])

func BFMatMulAddH(addend : bits(128), op1 : bits(128),
                  op2 : bits(128), fpcr : FPCR_Type) => bits(128)
begin

    var result : bits(128);
    var sum : bits(32);

    for i = 0 to 1 do
        for j = 0 to 1 do
            sum = addend[(2*i + j)*:32];
            for k = 0 to 1 do
                let elt1_a : bits(16) = op1[(4*i + 2*k + 0)*:16];
                let elt1_b : bits(16) = op1[(4*i + 2*k + 1)*:16];
                let elt2_a : bits(16) = op2[(4*j + 2*k + 0)*:16];
                let elt2_b : bits(16) = op2[(4*j + 2*k + 1)*:16];
                sum = BFDotAdd(sum, elt1_a, elt1_b, elt2_a, elt2_b, fpcr);
            end;
            result[(2*i + j)*:32] = sum;
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMax

```
// BFMax()
// =======
// BFloat16 maximum.

func BFMax{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let fpexc : boolean = TRUE;
    return BFMax{N}(op1, op2, fpcr, altfp, fpexc);
end;

// BFMax()
// =======
// BFloat16 maximum.

func BFMax{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type, altfp : boolean) => bits(N)
begin
    let fpexc : boolean = TRUE;
    return BFMax{N}(op1, op2, fpcr, altfp, fpexc);
end;

// BFMax()
// =======
// BFloat16 maximum following computational behaviors
// corresponding to instructions that read and write BFloat16 values.
// Compare op1 and op2 and return the larger value after rounding.
// The 'fpcr' argument supplies the FPCR control bits and 'altfp' determines
// if the function should use alternative floating-point behavior.

func BFMax{N}(op1 : bits(N), op2 : bits(N), fpcr_in : FPCR_Type,
              altfp : boolean, fpexc : boolean) => bits(N)
begin
    assert N == 16;
    var fpcr : FPCR_Type = fpcr_in;
    let rounding : FPRounding = FPRoundingMode(fpcr);
    var done : boolean;
    var result : bits(2*N);

    let op1_s : bits(2*N) = op1 :: Zeros{N};
    let op2_s : bits(2*N) = op2 :: Zeros{N};
    let (type1,sign1,value1) = FPUnpack{2*N}(op1_s, fpcr, fpexc);
    let (type2,sign2,value2) = FPUnpack{2*N}(op2_s, fpcr, fpexc);

    if altfp && type1 == FPType_Zero && type2 == FPType_Zero && sign1 != sign2 then
        // Alternate handling of zeros with differing sign
        return BFZero{N}(sign2);
    elsif altfp && (type1 IN {FPType_SNaN, FPType_QNaN} || type2 IN {FPType_SNaN, FPType_QNaN}) then
        // Alternate handling of NaN inputs
        if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        return (if type2 == FPType_Zero then BFZero{N}(sign2) else op2);
    end;

    (done,result) = FPProcessNaNs{2*N}(type1, type2, op1_s, op2_s, fpcr, fpexc);
    if !done then
        var fptype : FPType;
        var sign : bit;
        var value : real;
        if value1 > value2 then
            (fptype,sign,value) = (type1,sign1,value1);
        else
            (fptype,sign,value) = (type2,sign2,value2);
        end;
        if fptype == FPType_Infinity then
            result = FPInfinity{2*N}(sign);
        elsif fptype == FPType_Zero then
            sign = sign1 AND sign2;              // Use most positive sign
            result = FPZero{2*N}(sign);
        else
            if altfp then    // Denormal output is not flushed to zero
                fpcr.FZ = '0';
            end;
            result = FPRoundBF{2*N}(value, fpcr, rounding, fpexc);
        end;

        if fpexc then FPProcessDenorms(type1, type2, 2*N, fpcr); end;
    end;

    return result[2*N-1:N];
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMaxNum

```
// BFMaxNum()
// ==========

func BFMaxNum{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let fpexc : boolean = TRUE;
    return BFMaxNum{N}(op1, op2, fpcr, fpexc);
end;

// BFMaxNum()
// ==========
// BFloat16 maximum number following computational behaviors corresponding
// to instructions that read and write BFloat16 values.
// Compare op1 and op2 and return the larger number operand after rounding.
// The 'fpcr' argument supplies the FPCR control bits.

func BFMaxNum{N}(op1_in : bits(N), op2_in : bits(N), fpcr : FPCR_Type, fpexc : boolean) => bits(N)
begin
    assert N == 16;
    let isbfloat16 : boolean = TRUE;
    var op1 : bits(N)   = op1_in;
    var op2 : bits(N)   = op2_in;
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    var result : bits(N);

    let (type1,-,-) = FPUnpackBase{N}(op1, fpcr, fpexc, isbfloat16);
    let (type2,-,-) = FPUnpackBase{N}(op2, fpcr, fpexc, isbfloat16);

    let type1_nan : boolean = type1 IN {FPType_QNaN, FPType_SNaN};
    let type2_nan : boolean = type2 IN {FPType_QNaN, FPType_SNaN};

    if !(altfp && type1_nan && type2_nan) then
        // Treat a single quiet-NaN as -Infinity.
        if type1 == FPType_QNaN && type2 != FPType_QNaN then
            op1 = BFInfinity{N}('1');
        elsif type1 != FPType_QNaN && type2 == FPType_QNaN then
            op2 = BFInfinity{N}('1');
        end;
    end;

    let altfmaxfmin : boolean = FALSE;    // Do not use alternate NaN handling
    result = BFMax{N}(op1, op2, fpcr, altfmaxfmin, fpexc);

    return result;
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMin

```
// BFMin()
// =======
// BFloat16 minimum.

func BFMin{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let fpexc : boolean = TRUE;
    return BFMin{N}(op1, op2, fpcr, altfp, fpexc);
end;

// BFMin()
// =======
// BFloat16 minimum.

func BFMin{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type, altfp : boolean) => bits(N)
begin
    let fpexc : boolean = TRUE;
    return BFMin{N}(op1, op2, fpcr, altfp, fpexc);
end;

// BFMin()
// =======
// BFloat16 minimum following computational behaviors
// corresponding to instructions that read and write BFloat16 values.
// Compare op1 and op2 and return the smaller value after rounding.
// The 'fpcr' argument supplies the FPCR control bits and 'altfp' determines
// if the function should use alternative floating-point behavior.

func BFMin{N}(op1 : bits(N), op2 : bits(N), fpcr_in : FPCR_Type,
              altfp : boolean, fpexc : boolean) => bits(N)
begin
    assert N == 16;
    var fpcr : FPCR_Type = fpcr_in;
    let rounding : FPRounding = FPRoundingMode(fpcr);
    var done : boolean;
    var result : bits(2*N);

    let op1_s : bits(2*N) = op1 :: Zeros{N};
    let op2_s : bits(2*N) = op2 :: Zeros{N};
    let (type1,sign1,value1) = FPUnpack{2*N}(op1_s, fpcr, fpexc);
    let (type2,sign2,value2) = FPUnpack{2*N}(op2_s, fpcr, fpexc);

    if altfp && type1 == FPType_Zero && type2 == FPType_Zero && sign1 != sign2 then
        // Alternate handling of zeros with differing sign
        return BFZero{N}(sign2);
    elsif altfp && (type1 IN {FPType_SNaN, FPType_QNaN} || type2 IN {FPType_SNaN, FPType_QNaN}) then
        // Alternate handling of NaN inputs
        if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        return (if type2 == FPType_Zero then BFZero{N}(sign2) else op2);
    end;

    (done,result) = FPProcessNaNs{2*N}(type1, type2, op1_s, op2_s, fpcr, fpexc);
    if !done then
        var fptype : FPType;
        var sign : bit;
        var value : real;
        if value1 < value2 then
            (fptype,sign,value) = (type1,sign1,value1);
        else
            (fptype,sign,value) = (type2,sign2,value2);
        end;
        if fptype == FPType_Infinity then
            result = FPInfinity{2*N}(sign);
        elsif fptype == FPType_Zero then
            sign = sign1 OR sign2;              // Use most negative sign
            result = FPZero{2*N}(sign);
        else
            if altfp then                       // Denormal output is not flushed to zero
                fpcr.FZ = '0';
            end;
            result = FPRoundBF{2*N}(value, fpcr, rounding, fpexc);
        end;

        if fpexc then FPProcessDenorms(type1, type2, 2*N, fpcr); end;
    end;

    return result[2*N-1:N];
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMinNum

```
// BFMinNum()
// ==========

func BFMinNum{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let fpexc : boolean = TRUE;
    return BFMinNum{N}(op1, op2, fpcr, fpexc);
end;

// BFMinNum()
// ==========
// BFloat16 minimum number following computational behaviors corresponding
// to instructions that read and write BFloat16 values.
// Compare op1 and op2 and return the smaller number operand after rounding.
// The 'fpcr' argument supplies the FPCR control bits.

func BFMinNum{N}(op1_in : bits(N), op2_in : bits(N), fpcr : FPCR_Type, fpexc : boolean) => bits(N)
begin
    assert N == 16;
    let isbfloat16 : boolean = TRUE;
    var op1 : bits(N)   = op1_in;
    var op2 : bits(N)   = op2_in;
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    var result : bits(N);

    let (type1,-,-) = FPUnpackBase{N}(op1, fpcr, fpexc, isbfloat16);
    let (type2,-,-) = FPUnpackBase{N}(op2, fpcr, fpexc, isbfloat16);

    let type1_nan : boolean = type1 IN {FPType_QNaN, FPType_SNaN};
    let type2_nan : boolean = type2 IN {FPType_QNaN, FPType_SNaN};

    if !(altfp && type1_nan && type2_nan) then
        // Treat a single quiet-NaN as +Infinity.
        if type1 == FPType_QNaN && type2 != FPType_QNaN then
            op1 = BFInfinity{N}('0');
        elsif type1 != FPType_QNaN && type2 == FPType_QNaN then
            op2 = BFInfinity{N}('0');
        end;
    end;

    let altfmaxfmin : boolean = FALSE;    // Do not use alternate NaN handling
    result = BFMin{N}(op1, op2, fpcr, altfmaxfmin, fpexc);

    return result;
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMul

```
// BFMul()
// =======
// Non-widening BFloat16 multiply used by SVE2 instructions.

func BFMul(op1 : bits(16), op2 : bits(16), fpcr : FPCR_Type) => bits(16)
begin
    let fpexc : boolean = TRUE;
    return BFMul(op1, op2, fpcr, fpexc);
end;

// BFMul()
// =======
// Non-widening BFloat16 multiply following computational behaviors
// corresponding to instructions that read and write BFloat16 values.
// Calculates op1 * op2.
// The 'fpcr' argument supplies the FPCR control bits.

func BFMul(op1 : bits(16), op2 : bits(16), fpcr : FPCR_Type, fpexc : boolean) => bits(16)
begin
    let rounding : FPRounding = FPRoundingMode(fpcr);
    var done : boolean;
    var result : bits(32);

    let op1_s : bits(32) = op1 :: Zeros{16};
    let op2_s : bits(32) = op2 :: Zeros{16};
    let (type1,sign1,value1) = FPUnpack{32}(op1_s, fpcr, fpexc);
    let (type2,sign2,value2) = FPUnpack{32}(op2_s, fpcr, fpexc);

    (done,result) = FPProcessNaNs{32}(type1, type2, op1_s, op2_s, fpcr, fpexc);

    if !done then
        let inf1  = (type1 == FPType_Infinity);
        let inf2  = (type2 == FPType_Infinity);
        let zero1 = (type1 == FPType_Zero);
        let zero2 = (type2 == FPType_Zero);

        if (inf1 && zero2) || (zero1 && inf2) then
            result = FPDefaultNaN{32}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        elsif inf1 || inf2 then
            result = FPInfinity{32}(sign1 XOR sign2);
        elsif zero1 || zero2 then
            result = FPZero{32}(sign1 XOR sign2);
        else
            result = FPRoundBF{32}(value1*value2, fpcr, rounding, fpexc);
        end;

        if fpexc then FPProcessDenorms(type1, type2, 32, fpcr); end;
    end;

    return result[31:16];
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMulAdd

```
// BFMulAdd()
// ==========
// Non-widening BFloat16 fused multiply-add used by SVE2 instructions.

func BFMulAdd(addend : bits(16), op1 : bits(16), op2 : bits(16), fpcr : FPCR_Type) => bits(16)
begin
    let fpexc : boolean = TRUE;
    return BFMulAdd(addend, op1, op2, fpcr, fpexc);
end;

// BFMulAdd()
// ==========
// Non-widening BFloat16 fused multiply-add following computational behaviors
// corresponding to instructions that read and write BFloat16 values.
// Calculates addend + op1*op2 with a single rounding.
// The 'fpcr' argument supplies the FPCR control bits.

func BFMulAdd(addend : bits(16), op1 : bits(16), op2 : bits(16),
              fpcr : FPCR_Type, fpexc : boolean) => bits(16)
begin
    let rounding : FPRounding = FPRoundingMode(fpcr);
    var done : boolean;
    var result : bits(32);

    let addend_s : bits(32)  = addend :: Zeros{16};
    let op1_s : bits(32)     = op1 :: Zeros{16};
    let op2_s : bits(32)     = op2 :: Zeros{16};
    let (typeA,signA,valueA) = FPUnpack{32}(addend_s, fpcr, fpexc);
    let (type1,sign1,value1) = FPUnpack{32}(op1_s, fpcr, fpexc);
    let (type2,sign2,value2) = FPUnpack{32}(op2_s, fpcr, fpexc);

    let inf1  = (type1 == FPType_Infinity);
    let inf2  = (type2 == FPType_Infinity);
    let zero1 = (type1 == FPType_Zero);
    let zero2 = (type2 == FPType_Zero);

    (done,result) = FPProcessNaNs3{32}(typeA, type1, type2, addend_s, op1_s, op2_s, fpcr, fpexc);

    if !(IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1') then
        if typeA == FPType_QNaN && ((inf1 && zero2) || (zero1 && inf2)) then
            result = FPDefaultNaN{32}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        end;
    end;

    if !done then
        let infA  = (typeA == FPType_Infinity);
        let zeroA = (typeA == FPType_Zero);

        // Determine sign and type product will have if it does not cause an
        // Invalid Operation.
        let signP = sign1 XOR sign2;
        let infP  = inf1 || inf2;
        let zeroP = zero1 || zero2;

        // Non SNaN-generated Invalid Operation cases are multiplies of zero
        // by infinity and additions of opposite-signed infinities.
        let invalidop = (inf1 && zero2) || (zero1 && inf2) || (infA && infP && signA != signP);

        if invalidop then
            result = FPDefaultNaN{32}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;

        // Other cases involving infinities produce an infinity of the same sign.
        elsif (infA && signA == '0') || (infP && signP == '0') then
            result = FPInfinity{32}('0');
        elsif (infA && signA == '1') || (infP && signP == '1') then
            result = FPInfinity{32}('1');

        // Cases where the result is exactly zero and its sign is not determined by the
        // rounding mode are additions of same-signed zeros.
        elsif zeroA && zeroP && signA == signP then
            result = FPZero{32}(signA);

        // Otherwise calculate numerical result and round it.
        else
            let result_value : real = valueA + (value1 * value2);
            if result_value == 0.0 then  // Sign of exact zero result depends on rounding mode
                let result_sign : bit = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{32}(result_sign);
            else
                result = FPRoundBF{32}(result_value, fpcr, rounding, fpexc);
            end;
        end;

        if !invalidop && fpexc then
            FPProcessDenorms3(typeA, type1, type2, 32, fpcr);
        end;
    end;
```

```
    return result[31:16];
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMulAddH

```
// BFMulAddH()
// ===========
// Used by BFMLALB, BFMLALT, BFMLSLB and BFMLSLT instructions.

func BFMulAddH(addend : bits(32), op1 : bits(16), op2 : bits(16), fpcr_in : FPCR_Type) => bits(32)
begin
    let value1 : bits(32) = op1 :: Zeros{16};
    let value2 : bits(32) = op2 :: Zeros{16};
    var fpcr : FPCR_Type  = fpcr_in;
    let altfp : boolean   = IsFeatureImplemented(FEAT_AFP) && fpcr.AH == '1';
    // When using alternative floating-point behavior, do not generate floating-point exceptions
    let fpexc : boolean   = !altfp;
    if altfp then fpcr.[FIZ,FZ] = '11'; end;                          // Flush denormal input and
                                                                      // output to zero
    if altfp then fpcr.RMode    = '00'; end;                          // Use RNE rounding mode
    return FPMulAdd{32}(addend, value1, value2, fpcr, fpexc);
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMulAddH_ZA

```
// BFMulAddH_ZA()
// ==============
// Used by SME2 ZA-targeting BFMLAL and BFMLSL instructions.

func BFMulAddH_ZA(addend : bits(32), op1 : bits(16), op2 : bits(16), fpcr : FPCR_Type) => bits(32)
begin
    let value1 : bits(32) = op1 :: Zeros{16};
    let value2 : bits(32) = op2 :: Zeros{16};
    return FPMulAdd_ZA{32}(addend, value1, value2, fpcr);
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMulAdd_ZA

```
// BFMulAdd_ZA()
// =============
// Non-widening BFloat16 fused multiply-add used by SME2 ZA-targeting instructions.

func BFMulAdd_ZA(addend : bits(16), op1 : bits(16), op2 : bits(16), fpcr_in : FPCR_Type) => bits(16)
begin
    let fpexc : boolean  = FALSE;
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.DN = '1';          // Generate default NaN values
    return BFMulAdd(addend, op1, op2, fpcr, fpexc);
end;
```

Library pseudocode for shared/functions/float/bfloat/BFMulH

```
// BFMulH()
// ========
// BFloat16 widening multiply to single-precision following BFloat16
// computation behaviors.

func BFMulH(op1 : bits(16), op2 : bits(16), fpcr : FPCR_Type) => bits(32)
begin
    var result : bits(32);

    let (type1,sign1,value1) = BFUnpack{16}(op1);
    let (type2,sign2,value2) = BFUnpack{16}(op2);
    if type1 == FPType_QNaN || type2 == FPType_QNaN then
        result = FPDefaultNaN{32}(fpcr);
    else
        let inf1  = (type1 == FPType_Infinity);
        let inf2  = (type2 == FPType_Infinity);
        let zero1 = (type1 == FPType_Zero);
        let zero2 = (type2 == FPType_Zero);
        if (inf1 && zero2) || (zero1 && inf2) then
            result = FPDefaultNaN{32}(fpcr);
        elsif inf1 || inf2 then
            result = FPInfinity{32}(sign1 XOR sign2);
        elsif zero1 || zero2 then
            result = FPZero{32}(sign1 XOR sign2);
        else
            result = BFRound(value1*value2);
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/bfloat/BFNeg

```
// BFNeg()
// =======

func BFNeg(op : bits(16)) => bits(16)
begin
    let honor_altfp : boolean = TRUE;    // Honor alternate handling
    return BFNeg(op, honor_altfp);
end;

// BFNeg()
// =======

func BFNeg(op : bits(16), honor_altfp : boolean) => bits(16)
begin
    if honor_altfp && !UsingAArch32() && IsFeatureImplemented(FEAT_AFP) then
        if FPCR().AH == '1' then
            let fpexc : boolean = FALSE;
            let isbfloat16 : boolean = TRUE;
            let (fptype, -, -) = FPUnpackBase{16}(op, FPCR(), fpexc, isbfloat16);
            if fptype IN {FPType_SNaN, FPType_QNaN} then
                return op;        // When FPCR().AH=1, sign of NaN has no consequence
            end;
        end;
    end;
    return NOT(op[15]) :: op[14:0];
end;
```

Library pseudocode for shared/functions/float/bfloat/BFRound

```
// BFRound()
// =========
// Converts a real number OP into a single-precision value using the
// Round to Odd rounding mode and following BFloat16 computation behaviors.

func BFRound(op : real) => bits(32)
begin
    assert op != 0.0;
    var result : bits(32);

    // Format parameters - minimum exponent, numbers of exponent and fraction bits.
    let minimum_exp : integer = -126;  let E : integer{} = 8;  let F : integer{} = 23;

    // Split value into sign, unrounded mantissa and exponent.
    var sign : bit;
    var exponent : integer;
    var mantissa : real;
    if op < 0.0 then
        sign = '1';  mantissa = -op;
    else
        sign = '0';  mantissa = op;
    end;

    (mantissa, exponent) = NormalizeReal(mantissa);
    // Fixed Flush-to-zero.
    if exponent < minimum_exp then
        return FPZero{32}(sign);
    end;

    // Start creating the exponent value for the result. Start by biasing the actual exponent
    // so that the minimum exponent becomes 1, lower values 0 (indicating possible underflow).
    let biased_exp : integer = Max((exponent - minimum_exp) + 1, 0);
    if biased_exp == 0 then mantissa = mantissa / 2.0^(minimum_exp - exponent); end;

    // Get the unrounded mantissa as an integer, and the "units in last place" rounding error.
    // < 2.0^F if biased_exp == 0, >= 2.0^F if not
    var int_mant : integer = RoundDown(mantissa * 2.0^F);
    let error : real = mantissa * 2.0^F - Real(int_mant);

    // Round to Odd
    if error != 0.0 && int_mant[0] == '0' then
        int_mant = int_mant + 1;
    end;

    // Deal with overflow and generate result.
    if biased_exp >= 2^E - 1 then
        result = FPInfinity{32}(sign);      // Overflows generate appropriately-signed Infinity
    else
        result = sign :: biased_exp[E-1:0] :: int_mant[F-1:0];
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/bfloat/BFScale

```
// BFScale()
// =========
// Scales BFloat16 operand by 2.0 to the power of the signed integer value.

func BFScale(op : bits(16), scale : integer, fpcr : FPCR_Type) => bits(16)
begin
    var result : bits(32);

    let op_s : bits(32) = op :: Zeros{16};
    let (fptype,sign,value) = FPUnpack{32}(op_s, fpcr);

    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        result = FPProcessNaN{32}(fptype, op_s, fpcr);
    elsif fptype == FPType_Zero then
        result = FPZero{32}(sign);
    elsif fptype == FPType_Infinity then
        result = FPInfinity{32}(sign);
    else
        let rounding : FPRounding = FPRoundingMode(fpcr);
        let fpexc : boolean = TRUE;
        let clamped_scale : integer = BFClampScale(op, scale);
        result = FPRoundBF{32}(value * (2.0^clamped_scale), fpcr, rounding, fpexc);
        if fpexc then FPProcessDenorm(fptype, 32, fpcr); end;
    end;

    return result[31:16];
end;
```

Library pseudocode for shared/functions/float/bfloat/BFSub

```
// BFSub()
// =======
// Non-widening BFloat16 subtraction used by SVE2 instructions.

func BFSub(op1 : bits(16), op2 : bits(16), fpcr : FPCR_Type) => bits(16)
begin
    let fpexc : boolean = TRUE;
    return BFSub(op1, op2, fpcr, fpexc);
end;

// BFSub()
// =======
// Non-widening BFloat16 subtraction following computational behaviors
// corresponding to instructions that read and write BFloat16 values.
// Calculates op1 - op2.
// The 'fpcr' argument supplies the FPCR control bits.

func BFSub(op1 : bits(16), op2 : bits(16), fpcr : FPCR_Type, fpexc : boolean) => bits(16)
begin
    let rounding : FPRounding = FPRoundingMode(fpcr);
    var done : boolean;
    var result : bits(32);

    let op1_s : bits(32) = op1 :: Zeros{16};
    let op2_s : bits(32) = op2 :: Zeros{16};
    let (type1,sign1,value1) = FPUnpack{32}(op1_s, fpcr, fpexc);
    let (type2,sign2,value2) = FPUnpack{32}(op2_s, fpcr, fpexc);

    (done,result) = FPProcessNaNs{32}(type1, type2, op1_s, op2_s, fpcr, fpexc);

    if !done then
        let inf1  = (type1 == FPType_Infinity);
        let inf2  = (type2 == FPType_Infinity);
        let zero1 = (type1 == FPType_Zero);
        let zero2 = (type2 == FPType_Zero);

        if inf1 && inf2 && sign1 == sign2 then
            result = FPDefaultNaN{32}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        elsif (inf1 && sign1 == '0') || (inf2 && sign2 == '1') then
            result = FPInfinity{32}('0');
        elsif (inf1 && sign1 == '1') || (inf2 && sign2 == '0') then
            result = FPInfinity{32}('1');
        elsif zero1 && zero2 && sign1 == NOT(sign2) then
            result = FPZero{32}(sign1);
        else
            let result_value : real = value1 - value2;
            if result_value == 0.0 then  // Sign of exact zero result depends on rounding mode
                let result_sign : bit = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{32}(result_sign);
            else
                result = FPRoundBF{32}(result_value, fpcr, rounding, fpexc);
            end;
        end;

        if fpexc then FPProcessDenorms(type1, type2, 32, fpcr); end;
    end;

    return result[31:16];
end;
```

Library pseudocode for shared/functions/float/bfloat/BFSub_ZA

```
// BFSub_ZA()
// ==========
// Non-widening BFloat16 subtraction used by SME2 ZA-targeting instructions.

func BFSub_ZA(op1 : bits(16), op2 : bits(16), fpcr_in : FPCR_Type) => bits(16)
begin
    let fpexc : boolean = FALSE;
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.DN = '1';           // Generate default NaN values
    return BFSub(op1, op2, fpcr, fpexc);
end;
```

Library pseudocode for shared/functions/float/bfloat/BFUnpack

```
// BFUnpack()
// ==========
// Unpacks a BFloat16 or single-precision value into its type,
// sign bit and real number that it represents.
// The real number result has the correct sign for numbers and infinities,
// is very large in magnitude for infinities, and is 0.0 for NaNs.
// (These values are chosen to simplify the description of
// comparisons and conversions.)

func BFUnpack{N}(fpval : bits(N)) => (FPType, bit, real)
begin
    assert N IN {16,32};

    var sign : bit;
    var exp : bits(8);
    var frac : bits(23);
    if N == 16 then
        sign   = fpval[15];
        exp    = fpval[14:7];
        frac   = fpval[6:0] :: Zeros{16};
    else  // N == 32
        sign   = fpval[31];
        exp    = fpval[30:23];
        frac   = fpval[22:0];
    end;

    var fptype : FPType;
    var value : real;
    if IsZero(exp) then
        fptype = FPType_Zero;  value = 0.0;    // Fixed Flush to Zero
    elsif IsOnes(exp) then
        if IsZero(frac) then
            fptype = FPType_Infinity;  value = 2.0^1000000;
        else    // no SNaN for BF16 arithmetic
            fptype = FPType_QNaN; value = 0.0;
        end;
    else
        fptype = FPType_Nonzero;
        value = 2.0^(UInt(exp)-127) * (1.0 + Real(UInt(frac)) * 2.0^-23);
    end;

    if sign == '1' then value = -value; end;

    return (fptype, sign, value);
end;
```

Library pseudocode for shared/functions/float/bfloat/BFZero

```
// BFZero()
// ========

func BFZero{N}(sign : bit) => bits(N)
begin
    assert N == 16;
    let E : integer{} = 8;
    let F : integer{} = N - (E + 1);
    return sign :: Zeros{E} :: Zeros{F};
end;
```

Library pseudocode for shared/functions/float/bfloat/FPAdd_BF16

```
// FPAdd_BF16()
// ============
// Single-precision add following BFloat16 computation behaviors.

func FPAdd_BF16(op1 : bits(32), op2 : bits(32), fpcr : FPCR_Type) => bits(32)
begin
    var result : bits(32);

    let (type1,sign1,value1) = BFUnpack{32}(op1);
    let (type2,sign2,value2) = BFUnpack{32}(op2);
    if type1 == FPType_QNaN || type2 == FPType_QNaN then
        result = FPDefaultNaN{32}(fpcr);
    else
        let inf1  = (type1 == FPType_Infinity);
        let inf2  = (type2 == FPType_Infinity);
        let zero1 = (type1 == FPType_Zero);
        let zero2 = (type2 == FPType_Zero);
        if inf1 && inf2 && sign1 == NOT(sign2) then
            result = FPDefaultNaN{32}(fpcr);
        elsif (inf1 && sign1 == '0') || (inf2 && sign2 == '0') then
            result = FPInfinity{32}('0');
        elsif (inf1 && sign1 == '1') || (inf2 && sign2 == '1') then
            result = FPInfinity{32}('1');
        elsif zero1 && zero2 && sign1 == sign2 then
            result = FPZero{32}(sign1);
        else
            let result_value : real = value1 + value2;
            if result_value == 0.0 then
                result = FPZero{32}('0');    // Positive sign when Round to Odd
            else
                result = BFRound(result_value);
            end;
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/bfloat/FPConvertBF

```
// FPConvertBF()
// =============
// Converts a single-precision OP to BFloat16 value using the
// Round to Nearest Even rounding mode when executed from AArch64 state and
// FPCR.AH == '1', otherwise rounding is controlled by FPCR/FPSCR.

func FPConvertBF(op : bits(32), fpcr_in : FPCR_Type, rounding_in : FPRounding) => bits(16)
begin
    let halfsize : integer{}  = 16;
    var fpcr : FPCR_Type      = fpcr_in;
    var rounding : FPRounding = rounding_in;
    var result : bits(32);                                  // BF16 value in top 16 bits
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let fpexc : boolean = !altfp;                          // Generate no floating-point exceptions
    if altfp then fpcr.[FIZ,FZ] = '11'; end;               // Flush denormal input and output to zero
    if altfp then rounding = FPRounding_TIEEVEN; end;  // Use RNE rounding mode

    // Unpack floating-point operand, with always flush-to-zero if fpcr.AH == '1'.
    let (fptype,sign,value) = FPUnpack{32}(op, fpcr, fpexc);

    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        if fpcr.DN == '1' then
            result = FPDefaultNaN{32}(fpcr);
        else
            result = FPConvertNaN{32, 32}(op);
        end;
        if fptype == FPType_SNaN then
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        end;
    elsif fptype == FPType_Infinity then
        result = FPInfinity{32}(sign);
    elsif fptype == FPType_Zero then
        result = FPZero{32}(sign);
    else
        result = FPRoundBF{32}(value, fpcr, rounding, fpexc);
    end;

    // Returns correctly rounded BF16 value from top 16 bits
    return result[(2*halfsize)-1:halfsize];
end;

// FPConvertBF()
// =============
// Converts a single-precision operand to BFloat16 value.

func FPConvertBF(op : bits(32), fpcr : FPCR_Type) => bits(16)
begin
    return FPConvertBF(op, fpcr, FPRoundingMode(fpcr));
end;
```

Library pseudocode for shared/functions/float/bfloat/FPRoundBF

```
// FPRoundBF()
// ===========
// Converts a real number OP into a BFloat16 value using the supplied
// rounding mode RMODE. The 'fpexc' argument controls the generation of
// floating-point exceptions.

func FPRoundBF{N}(op : real, fpcr : FPCR_Type, rounding : FPRounding, fpexc : boolean) => bits(N)
begin
    assert N == 32;
    let isbfloat16 : boolean = TRUE;
    return FPRoundBase{N}(op, fpcr, rounding, isbfloat16, fpexc);
end;
```

Library pseudocode for shared/functions/float/fixedtofp/FixedToFP

```
// FixedToFP()
// ===========

// Convert M-bit fixed point 'op' with FBITS fractional bits to
// N-bit precision floating point, controlled by UNSIGNED and ROUNDING.

func FixedToFP{N, M}(op : bits(M), fbits : integer, unsigned : boolean, fpcr : FPCR_Type,
                     rounding : FPRounding) => bits(N)
begin
    assert N IN {16,32,64};
    assert M IN {8,16,32,64};
    var result : bits(N);
    assert fbits >= 0;
    assert rounding != FPRounding_ODD;

    // Correct signed-ness
    let int_operand : integer{} = if unsigned then UInt(op) else SInt(op);

    // Scale by fractional bits and generate a real value
    let real_operand : real = Real(int_operand) / 2.0^fbits;

    if real_operand == 0.0 then
        result = FPZero{N}('0');
    else
        result = FPRound{N}(real_operand, fpcr, rounding);
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fp8float/BFConvertFP8

```
// BFConvertFP8()
// ==============
// Converts a BFloat16 OP to FP8 value.

func BFConvertFP8(op_in : bits(16), fpcr : FPCR_Type, fpmr : FPMR_Type) => bits(8)
begin
    let op : bits(32) = op_in :: Zeros{16};
    return FPConvertFP8{8, 32}(op, fpcr, fpmr);
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8Bits

```
// FP8Bits()
// =========
// Returns the minimum exponent, numbers of exponent and fraction bits.

func FP8Bits(fp8type : FP8Type) => FPBitsType
begin
    var minimum_exp : integer;
    var F : integer{2..3};
    if fp8type == FP8Type_OFP8_E4M3 then
        minimum_exp = -6;  F = 3;
    else  // fp8type == FP8Type_OFP8_E5M2
        minimum_exp = -14;  F = 2;
    end;

    return (F, minimum_exp);
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8ConvertBF

```
// FP8ConvertBF()
// ==============
// Converts an FP8 operand to BFloat16 value.

func FP8ConvertBF(op : bits(8), issrc2 : boolean, fpcr : FPCR_Type, fpmr : FPMR_Type) => bits(16)
begin
    let isbfloat16 : boolean = TRUE;
    let result : bits(32) = FP8ConvertFP{}(op, issrc2, fpcr, fpmr, isbfloat16);
    return result[16+:16];
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8ConvertFP

```
// FP8ConvertFP()
// ==============
// Converts an FP8 operand to half-precision value.

func FP8ConvertFP(op : bits(8), issrc2 : boolean, fpcr : FPCR_Type, fpmr : FPMR_Type) => bits(16)
begin
    let isbfloat16 : boolean = FALSE;
    return FP8ConvertFP{16}(op, issrc2, fpcr, fpmr, isbfloat16);
end;

// FP8ConvertFP()
// ==============
// Converts an FP8 operand to half-precision or BFloat16 value.
// The downscaling factor in FPMR.LSCALE or FPMR.LSCALE2 is applied to
// the value before rounding.

func FP8ConvertFP{M}(op : bits(8), issrc2 : boolean, fpcr_in : FPCR_Type, fpmr : FPMR_Type,
                     isbfloat16 : boolean) => bits(M)
begin
    assert M IN {16,32};
    var result : bits(M);

    let fpexc : boolean = TRUE;
    var fpcr : FPCR_Type = fpcr_in;
    // Do not flush denormal inputs and outputs to zero.
    // Do not support alternative half-precision format.
    fpcr.[FIZ,FZ,FZ16,AHP] = '0000';
    let rounding = FPRounding_TIEEVEN;
    let fp8type : FP8Type = (if issrc2 then FP8DecodeType(fpmr.F8S2)
                                       else FP8DecodeType(fpmr.F8S1));

    let (fptype,sign,value) = FP8Unpack{8}(op, fp8type);

    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        result = FPDefaultNaN{M}(fpcr);
        if fptype == FPType_SNaN then
            FPProcessException(FPExc_InvalidOp, fpcr);
        end;
    elsif fptype == FPType_Infinity then
        result = FPInfinity{M}(sign);
    elsif fptype == FPType_Zero then
        result = FPZero{M}(sign);
    else
        var dscale : integer;
        if issrc2 then
            dscale = (if M == 16 then UInt(fpmr.LSCALE2[3:0])
                      else UInt(fpmr.LSCALE2[:6]));
        else
            dscale = (if M == 16 then UInt(fpmr.LSCALE[3:0])
                       else UInt(fpmr.LSCALE[:6]));
        end;
        let result_value : real = value * (2.0^-dscale);
        result = FPRoundBase{M}(result_value, fpcr, rounding, isbfloat16, fpexc);
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8DecodeType

```
// FP8DecodeType()
// ===============
// Decode the FP8 format encoded in F8S1, F8S2 or F8D field in FPMR

func FP8DecodeType(f8format : bits(3)) => FP8Type
begin
    case f8format of
        when '000' => return FP8Type_OFP8_E5M2;
        when '001' => return FP8Type_OFP8_E4M3;
        otherwise =>  return FP8Type_UNSUPPORTED;
    end;
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8DefaultNaN

```
// FP8DefaultNaN()
// ===============

func FP8DefaultNaN{N}(fp8type : FP8Type, fpcr : FPCR_Type) => bits(N)
begin
    assert N == 8;
    assert fp8type IN {FP8Type_OFP8_E5M2, FP8Type_OFP8_E4M3};
    let sign : bit = if IsFeatureImplemented(FEAT_AFP) then fpcr.AH else '0';
    let E : integer{} = if fp8type == FP8Type_OFP8_E4M3 then 4 else 5;
    let F : integer{} = N - (E + 1);
    var exp : bits(E);
    var frac : bits(F);

    case fp8type of
        when FP8Type_OFP8_E4M3 =>
            exp  = Ones{E};
            frac = Ones{F};
        when FP8Type_OFP8_E5M2 =>
            exp  = Ones{E};
            frac = '1'::Zeros{F-1};
    end;

    return sign :: exp :: frac;
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8DotAddFP

```
// FP8DotAddFP()
// =============

func FP8DotAddFP{M, N}(addend : bits(M), op1 : bits(N), op2 : bits(N),
                       fpcr : FPCR_Type, fpmr : FPMR_Type) => bits(M)
begin
    let E : integer{} = (N DIV 8) as integer{1, 2, 4, 8};
    return FP8DotAddFP{M, N}(addend, op1, op2, E, fpcr, fpmr);
end;

// FP8DotAddFP()
// ==============
// Calculates result of "E"-way 8-bit floating-point dot-product with scaling
// and addition to half-precision or single-precision value without
// intermediate rounding.
// c = round(c + 2^-S*(a1*b1+..+aE*bE))
// The 8-bit floating-point format for op1 is determined by FPMR.F8S1
// and the one for op2 by FPMR.F8S2. The scaling factor in FPMR.LSCALE
// is applied to the sum-of-products before adding to the addend and rounding.

func FP8DotAddFP{M, N}(addend : bits(M), op1 : bits(N), op2 : bits(N), E : integer{1, 2, 4, 8},
                       fpcr_in : FPCR_Type, fpmr : FPMR_Type) => bits(M)
begin
    assert M IN {16,32};
    assert N IN {2*M, M, M DIV 2, M DIV 4};
    var fpcr : FPCR_Type = fpcr_in;
    var result : bits(M);

    fpcr.[FIZ,FZ,FZ16] = '000';        // Do not flush denormal inputs and outputs to zero
    fpcr.DN = '1';
    let rounding = FPRounding_TIEEVEN;

    let fp8type1 : FP8Type = FP8DecodeType(fpmr.F8S1);
    let fp8type2 : FP8Type = FP8DecodeType(fpmr.F8S2);

    var type1 : array[[E]] of FPType;
    var type2 : array[[E]] of FPType;
    var sign1 : array[[E]] of bit;
    var sign2 : array[[E]] of bit;
    var value1 : array[[E]] of real;
    var value2 : array[[E]] of real;
    var inf1 : array[[E]] of boolean;
    var inf2 : array[[E]] of boolean;
    var zero1 : array[[E]] of boolean;
    var zero2 : array[[E]] of boolean;

    let fpexc : boolean = FALSE;
    let (typeA,signA,valueA) = FPUnpack{M}(addend, fpcr, fpexc);
    let infA = (typeA == FPType_Infinity);   let zeroA = (typeA == FPType_Zero);
    var any_nan : boolean = typeA IN {FPType_SNaN, FPType_QNaN};
    for i = 0 to E-1 do
        (type1[[i]], sign1[[i]], value1[[i]]) = FP8Unpack{N DIV E}(op1[i*:(N DIV E)], fp8type1);
        (type2[[i]], sign2[[i]], value2[[i]]) = FP8Unpack{N DIV E}(op2[i*:(N DIV E)], fp8type2);
        inf1[[i]] = (type1[[i]] == FPType_Infinity); zero1[[i]] = (type1[[i]] == FPType_Zero);
        inf2[[i]] = (type2[[i]] == FPType_Infinity); zero2[[i]] = (type2[[i]] == FPType_Zero);
        any_nan = (any_nan || type1[[i]] IN {FPType_SNaN, FPType_QNaN} ||
                   type2[[i]] IN {FPType_SNaN, FPType_QNaN});
    end;

    if any_nan then
        result = FPDefaultNaN{M}(fpcr);
    else
        // Determine sign and type products will have if it does not cause an Invalid
        // Operation.
        var signP : array [[E]] of bit;
        var infP : array [[E]] of boolean;
        var zeroP : array [[E]] of boolean;
        for i = 0 to E-1 do
            signP[[i]] = sign1[[i]] XOR sign2[[i]];
            infP[[i]]  = inf1[[i]] || inf2[[i]];
            zeroP[[i]] = zero1[[i]] || zero2[[i]];
        end;

        // Detect non-numeric results of dot product and accumulate
        var posInfR : boolean = (infA && signA == '0');
        var negInfR : boolean = (infA && signA == '1');
        var zeroR : boolean = zeroA;
        var invalidop : boolean = FALSE;
        for i = 0 to E-1 do
            // Result is infinity if any input is infinity
            posInfR  = posInfR || (infP[[i]] && signP[[i]] == '0');
            negInfR  = negInfR || (infP[[i]] && signP[[i]] == '1');
            // Result is zero if the addend and the products are zeros of the same sign
            zeroR    = zeroR && zeroP[[i]] && (signA == signP[[i]]);
            // Non SNaN-generated Invalid Operation cases are multiplies of zero
            // by infinity and additions of opposite-signed infinities.
            invalidop = (invalidop || (inf1[[i]] && zero2[[i]]) || (zero1[[i]] && inf2[[i]]) ||
                         (infA && infP[[i]] && (signA != signP[[i]])));
            for j = i+1 to E-1 do
```

```
                invalidop = invalidop || (infP[[i]] && infP[[j]] && (signP[[i]] != signP[[j]]));
            end;
        end;

        if invalidop then
            result = FPDefaultNaN{M}(fpcr);

        // Other cases involving infinities produce an infinity of the same sign.
        elsif posInfR then
            result = FPInfinity{M}('0');
        elsif negInfR then
            result = FPInfinity{M}('1');

        // Cases where the result is exactly zero and its sign is not determined by the
        // rounding mode are additions of same-signed zeros.
        elsif zeroR then
            result = FPZero{M}(signA);

        // Otherwise calculate numerical value and round it.
        else
            // Apply scaling to sum-of-product
            let dscale : integer = if M == 32 then UInt(fpmr.LSCALE) else UInt(fpmr.LSCALE[3:0]);

            var dp_value : real = value1[[0]] * value2[[0]];
            for i = 1 to E-1 do
                dp_value = dp_value + value1[[i]] * value2[[i]];
            end;

            let result_value : real = valueA + dp_value * (2.0^-dscale);
            if result_value == 0.0 then  // Sign of exact zero result is '0' for RNE rounding mode
                result = FPZero{M}('0');
            else
                let satoflo : boolean = (fpmr.OSM == '1');
                result = FPRound_FP8{M}(result_value, fpcr, rounding, satoflo);
            end;
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8Infinity

```
// FP8Infinity()
// =============

func FP8Infinity{N}(fp8type : FP8Type, sign : bit) => bits(N)
begin
    assert N == 8;
    assert fp8type IN {FP8Type_OFP8_E5M2, FP8Type_OFP8_E4M3};
    let E : integer{} = if fp8type == FP8Type_OFP8_E4M3 then 4 else 5;
    let F : integer{} = N - (E + 1);
    var exp : bits(E);
    var frac : bits(F);

    case fp8type of
        when FP8Type_OFP8_E4M3 =>
            exp  = Ones{E};
            frac = Ones{F};
        when FP8Type_OFP8_E5M2 =>
            exp  = Ones{E};
            frac = Zeros{F};
    end;

    return sign :: exp :: frac;
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8MatMulAddFP

```
// FP8MatMulAddFP()
// ================
// 8-bit floating-point matrix multiply with scaling and add to half-precision
// or single-precision matrix.
// result[2, 2] = addend[2, 2] + (op1[2, E] * op2[E, 2])

func FP8MatMulAddFP{N}(addend : bits(N), op1 : bits(N), op2 : bits(N), E : integer{4, 8},
                       fpcr : FPCR_Type, fpmr : FPMR_Type) => bits(N)
begin
    assert N IN {64, 128};
    assert N == E*16;
    let M : integer{} = N DIV 4;
    var result : bits(N);

    for i = 0 to 1 do
        for j = 0 to 1 do
            let elt1 : bits(2*M) = op1[i*:(2*M)];
            let elt2 : bits(2*M) = op2[j*:(2*M)];
            let sum : bits(M) = addend[(2*i + j)*:M];
            result[(2*i + j)*:M] = FP8DotAddFP{M, N DIV 2}(sum, elt1, elt2, E, fpcr, fpmr);
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8MaxNormal

```
// FP8MaxNormal()
// ==============

func FP8MaxNormal{N}(fp8type : FP8Type, sign : bit) => bits(N)
begin
    assert N == 8;
    assert fp8type IN {FP8Type_OFP8_E5M2, FP8Type_OFP8_E4M3};
    let E : integer{} = if fp8type == FP8Type_OFP8_E4M3 then 4 else 5;
    let F : integer{} = N - (E + 1);
    var exp : bits(E);
    var frac : bits(F);

    case fp8type of
        when FP8Type_OFP8_E4M3 =>
            exp  = Ones{E};
            frac = Ones{F-1}::'0';
        when FP8Type_OFP8_E5M2 =>
            exp  = Ones{E-1}::'0';
            frac = Ones{F};
    end;

    return sign :: exp :: frac;
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8MulAddFP

```
// FP8MulAddFP()
// =============

func FP8MulAddFP{M}(addend : bits(M), op1 : bits(8), op2 : bits(8), fpcr : FPCR_Type,
                    fpmr : FPMR_Type) => bits(M)
begin
    assert M IN {16,32};
    let E : integer{} = 1;
    return FP8DotAddFP{M, 8}(addend, op1, op2, E, fpcr, fpmr);
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8Round

```
// FP8Round()
// ==========
// Used by FP8 downconvert instructions which observe FPMR.OSC
// to convert a real number OP into an FP8 value.

func FP8Round{N}(op : real, fp8type : FP8Type, fpcr : FPCR_Type, fpmr : FPMR_Type) => bits(N)
begin
    assert N == 8;
    assert fp8type IN {FP8Type_OFP8_E5M2, FP8Type_OFP8_E4M3};
    assert op != 0.0;
    var result : bits(N);

    // Format parameters - minimum exponent, numbers of exponent and fraction bits.
    let (F, minimum_exp) = FP8Bits(fp8type);
    let E : integer{} = (N - F) - 1;

    // Split value into sign, unrounded mantissa and exponent.
    var sign : bit;
    var exponent : integer;
    var mantissa : real;
    if op < 0.0 then
        sign = '1';  mantissa = -op;
    else
        sign = '0';  mantissa = op;
    end;

    (mantissa, exponent) = NormalizeReal(mantissa);
    // When TRUE, detection of underflow occurs after rounding.
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && fpcr.AH == '1';

    var biased_exp_unconstrained : integer = (exponent - minimum_exp) + 1;
    var int_mant_unconstrained : integer= RoundDown(mantissa * 2.0^F);
    let error_unconstrained : real = mantissa * 2.0^F - Real(int_mant_unconstrained);

    // Start creating the exponent value for the result. Start by biasing the actual exponent
    // so that the minimum exponent becomes 1, lower values 0 (indicating possible underflow).
    var biased_exp : integer = Max((exponent - minimum_exp) + 1, 0);
    if biased_exp == 0 then mantissa = mantissa / 2.0^(minimum_exp - exponent); end;

    // Get the unrounded mantissa as an integer, and the "units in last place" rounding error.
    // < 2.0^F if biased_exp == 0, >= 2.0^F if not
    var int_mant : integer = RoundDown(mantissa * 2.0^F);
    var error : real = mantissa * 2.0^F - Real(int_mant);

    let trapped_UF : boolean = fpcr.UFE == '1' && (!InStreamingMode() || IsFullA64Enabled());

    var round_up_unconstrained : boolean;
    var round_up : boolean;

    if altfp then
        // Round to Nearest Even
        round_up_unconstrained = (error_unconstrained > 0.5 ||
                  (error_unconstrained == 0.5 && int_mant_unconstrained[0] == '1'));
        round_up = (error > 0.5 || (error == 0.5 && int_mant[0] == '1'));

        if round_up_unconstrained then
            int_mant_unconstrained = int_mant_unconstrained + 1;
            if int_mant_unconstrained == 2^(F+1) then    // Rounded up to next exponent
                biased_exp_unconstrained = biased_exp_unconstrained + 1;
                int_mant_unconstrained   = int_mant_unconstrained DIV 2;
            end;
        end;

        // Follow alternate floating-point behavior of underflow after rounding
        if (biased_exp_unconstrained < 1 && int_mant_unconstrained != 0 &&
            (error != 0.0 || trapped_UF)) then
            FPProcessException(FPExc_Underflow, fpcr);
        end;
    else // altfp == FALSE
        // Underflow occurs if exponent is too small before rounding, and result is inexact or
        // the Underflow exception is trapped. This applies before rounding if FPCR.AH != '1'.
        if biased_exp == 0 && (error != 0.0 || trapped_UF) then
            FPProcessException(FPExc_Underflow, fpcr);
        end;

        // Round to Nearest Even
        round_up = (error > 0.5 || (error == 0.5 && int_mant[0] == '1'));
    end;

    if round_up then
        int_mant = int_mant + 1;
        if int_mant == 2^F then      // Rounded up from denormalized to normalized
            biased_exp = 1;
        end;
        if int_mant == 2^(F+1) then  // Rounded up to next exponent
            biased_exp = biased_exp + 1;
            int_mant = int_mant DIV 2;
        end;
    end;
```

```
    // Deal with overflow and generate result.
    var overflow : boolean;
    case fp8type of
        when FP8Type_OFP8_E4M3 =>
            overflow = biased_exp >= 2^E || (biased_exp == 2^E - 1 && int_mant == 2^(F+1) - 1);
        when FP8Type_OFP8_E5M2 =>
            overflow = biased_exp >= 2^E - 1;
    end;

    if overflow then
        result = (if fpmr.OSC == '0' then FP8Infinity{N}(fp8type, sign)
                  else FP8MaxNormal{N}(fp8type, sign));
        // Flag Overflow exception regardless of FPMR.OSC
        FPProcessException(FPExc_Overflow, fpcr);
        error = 1.0;  // Ensure that an Inexact exception occurs
    else
        result = sign :: biased_exp[E-1:0] :: int_mant[F-1:0];
    end;

    // Deal with Inexact exception.
    if error != 0.0 then
        FPProcessException(FPExc_Inexact, fpcr);
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8Type

```
// FP8Type
// =======

type FP8Type of enumeration {FP8Type_OFP8_E5M2, FP8Type_OFP8_E4M3, FP8Type_UNSUPPORTED};
```

Library pseudocode for shared/functions/float/fp8float/FP8Unpack

```
// FP8Unpack()
// ===========
// Unpacks an FP8 value into its type, sign bit and real number that
// it represents.

func FP8Unpack{N}(fpval : bits(N), fp8type : FP8Type) => (FPType, bit, real)
begin
    assert N == 8;
    let E : integer{} = if fp8type == FP8Type_OFP8_E4M3 then 4 else 5;
    let F : integer{} = N - (E + 1);

    let sign : bit = fpval[N-1];
    let exp : bits(E) = fpval[(E+F)-1:F];
    let frac : bits(F) = fpval[F-1:0];

    var value : real;
    var fptype : FPType;

    if fp8type == FP8Type_OFP8_E4M3 then
        if IsZero(exp) then
            if IsZero(frac) then
                fptype = FPType_Zero;  value = 0.0;
            else
                fptype = FPType_Denormal;  value = 2.0^-6 * (Real(UInt(frac)) * 2.0^-3);
            end;
        elsif IsOnes(exp) && IsOnes(frac) then
            fptype = FPType_SNaN;
            value = 0.0;
        else
            fptype = FPType_Nonzero;
            value = 2.0^(UInt(exp)-7) * (1.0 + Real(UInt(frac)) * 2.0^-3);
        end;

    elsif fp8type == FP8Type_OFP8_E5M2 then
        if IsZero(exp) then
            if IsZero(frac) then
                fptype = FPType_Zero;  value = 0.0;
            else
                fptype = FPType_Denormal;  value = 2.0^-14 * (Real(UInt(frac)) * 2.0^-2);
            end;
        elsif IsOnes(exp) then
            if IsZero(frac) then
                fptype = FPType_Infinity;  value = 2.0^1000000;
            else
                fptype = if frac[1] == '1' then FPType_QNaN else FPType_SNaN;
                value = 0.0;
            end;
        else
            fptype = FPType_Nonzero;
            value = 2.0^(UInt(exp)-15) * (1.0 + Real(UInt(frac)) * 2.0^-2);
        end;

    else // fp8type == FP8Type_UNSUPPORTED
        fptype = FPType_SNaN;
        value = 0.0;
    end;

    if sign == '1' then value = -value; end;

    return (fptype, sign, value);
end;
```

Library pseudocode for shared/functions/float/fp8float/FP8Zero

```
// FP8Zero()
// =========

func FP8Zero{N}(fp8type : FP8Type, sign : bit) => bits(N)
begin
    assert N == 8;
    assert fp8type IN {FP8Type_OFP8_E5M2, FP8Type_OFP8_E4M3};
    let E : integer{} = if fp8type == FP8Type_OFP8_E4M3 then 4 else 5;
    let F : integer{} = N - (E + 1);
    return sign :: Zeros{E} :: Zeros{F};
end;
```

Library pseudocode for shared/functions/float/fp8float/FPConvertFP8

```
// FPConvertFP8()
// ==============
// Converts a half-precision or single-precision OP to FP8 value.
// The scaling factor in FPMR.NSCALE is applied to the value before rounding.

func FPConvertFP8{M, N}(op : bits(N), fpcr_in : FPCR_Type, fpmr : FPMR_Type) => bits(M)
begin
    assert N IN {16,32} && M == 8;
    var result : bits(M);

    let fpexc : boolean = TRUE;
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.[FIZ,FZ,FZ16] = '000';    // Do not flush denormal inputs and outputs to zero
    let fp8type : FP8Type = FP8DecodeType(fpmr.F8D);

    let (fptype,sign,value) = FPUnpack{N}(op, fpcr, fpexc);

    if fp8type == FP8Type_UNSUPPORTED then
        result = Ones{M};
        FPProcessException(FPExc_InvalidOp, fpcr);
    elsif fptype == FPType_SNaN || fptype == FPType_QNaN then
        result = FP8DefaultNaN{M}(fp8type, fpcr);     // Always generate default NaN as result
        if fptype == FPType_SNaN then
            FPProcessException(FPExc_InvalidOp, fpcr);
        end;
    elsif fptype == FPType_Infinity then
        result = (if fpmr.OSC == '0' then FP8Infinity{M}(fp8type, sign)
                  else FP8MaxNormal{M}(fp8type, sign));
    elsif fptype == FPType_Zero then
        result = FP8Zero{M}(fp8type, sign);
    else
        let scale : integer = if N == 16 then SInt(fpmr.NSCALE[4:0]) else SInt(fpmr.NSCALE);
        let result_value : real = value * (2.0^scale);
        result = FP8Round{M}(result_value, fp8type, fpcr, fpmr);
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpabs/FPAbs

```
// FPAbs()
// =======

func FPAbs{N}(op : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    if !UsingAArch32() && IsFeatureImplemented(FEAT_AFP) then
        if fpcr.AH == '1' then
            let (fptype, -, -) = FPUnpack{N}(op, fpcr, FALSE);
            if fptype IN {FPType_SNaN, FPType_QNaN} then
                return op;       // When fpcr.AH=1, sign of NaN has no consequence
            end;
        end;
    end;
    return '0' :: op[N-2:0];
end;
```

Library pseudocode for shared/functions/float/fpabsmax/FPAbsMax

```
// FPAbsMax()
// ==========
// Compare absolute value of two operands and return the larger absolute
// value without rounding.

func FPAbsMax{N}(op1_in : bits(N), op2_in : bits(N), fpcr_in : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var done : boolean;
    var result : bits(N);
    var fpcr : FPCR_Type  = fpcr_in;
    fpcr.[AH,FIZ,FZ,FZ16] = '0000';

    let op1 = '0'::op1_in[N-2:0];
    let op2 = '0'::op2_in[N-2:0];
    let (type1,-,value1) = FPUnpack{N}(op1, fpcr);
    let (type2,-,value2) = FPUnpack{N}(op2, fpcr);

    (done,result) = FPProcessNaNs{N}(type1, type2, op1_in, op2_in, fpcr);

    if !done then
        // This condition covers all results other than NaNs,
        // including Zero & Infinity
        result = if value1 > value2 then op1 else op2;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpabsmin/FPAbsMin

```
// FPAbsMin()
// ==========
// Compare absolute value of two operands and return the smaller absolute
// value without rounding.

func FPAbsMin{N}(op1_in : bits(N), op2_in : bits(N), fpcr_in : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var done : boolean;
    var result : bits(N);
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.[AH,FIZ,FZ,FZ16] = '0000';

    let op1 = '0'::op1_in[N-2:0];
    let op2 = '0'::op2_in[N-2:0];
    let (type1,-,value1) = FPUnpack{N}(op1, fpcr);
    let (type2,-,value2) = FPUnpack{N}(op2, fpcr);

    (done,result) = FPProcessNaNs{N}(type1, type2, op1_in, op2_in, fpcr);

    if !done then
        // This condition covers all results other than NaNs,
        // including Zero & Infinity
        result = if value1 < value2 then op1 else op2;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpadd/FPAdd

```
// FPAdd()
// =======

func FPAdd{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let fpexc : boolean = TRUE;       // Generate floating-point exceptions
    return FPAdd{N}(op1, op2, fpcr, fpexc);
end;

// FPAdd()
// =======

func FPAdd{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type, fpexc : boolean) => bits(N)
begin

    assert N IN {16,32,64};
    let rounding : FPRounding = FPRoundingMode(fpcr);

    let (type1,sign1,value1) = FPUnpack{N}(op1, fpcr, fpexc);
    let (type2,sign2,value2) = FPUnpack{N}(op2, fpcr, fpexc);

    var (done,result) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr, fpexc);
    if !done then
        let inf1 : boolean  = (type1 == FPType_Infinity);
        let inf2 : boolean  = (type2 == FPType_Infinity);
        let zero1 : boolean = (type1 == FPType_Zero);
        let zero2 : boolean = (type2 == FPType_Zero);
        if inf1 && inf2 && sign1 == NOT(sign2) then
            result = FPDefaultNaN{N}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        elsif (inf1 && sign1 == '0') || (inf2 && sign2 == '0') then
            result = FPInfinity{N}('0');
        elsif (inf1 && sign1 == '1') || (inf2 && sign2 == '1') then
            result = FPInfinity{N}('1');
        elsif zero1 && zero2 && sign1 == sign2 then
            result = FPZero{N}(sign1);
        else
            let result_value : real = value1 + value2;
            if result_value == 0.0 then  // Sign of exact zero result depends on rounding mode
                let result_sign : bit = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{N}(result_sign);
            else
                result = FPRound{N}(result_value, fpcr, rounding, fpexc);
            end;
        end;

        if fpexc then FPProcessDenorms(type1, type2, N, fpcr); end;
    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpadd/FPAdd_ZA

```
// FPAdd_ZA()
// ==========
// Calculates op1+op2 for SME2 ZA-targeting instructions.

func FPAdd_ZA{N}(op1 : bits(N), op2 : bits(N), fpcr_in : FPCR_Type) => bits(N)
begin
    var fpcr : FPCR_Type = fpcr_in;
    let fpexc : boolean = FALSE; // Do not generate floating-point exceptions
    fpcr.DN = '1';               // Generate default NaN values
    return FPAdd{N}(op1, op2, fpcr, fpexc);
end;
```

Library pseudocode for shared/functions/float/fpbits/FPBits

```
// FPBits()
// ========
// Returns the minimum exponent, numbers of exponent and fraction bits.

func FPBits(N : integer, isbfloat16 : boolean) => FPBitsType
begin
    var F : FPFracBits;
    var minimum_exp : integer;
    if N == 16 then
        minimum_exp = -14;   F = 10;
    elsif N == 32 && isbfloat16 then
        minimum_exp = -126;  F = 7;
    elsif N == 32 then
        minimum_exp = -126;  F = 23;
    else  // N == 64
        minimum_exp = -1022; F = 52;
    end;

    return (F, minimum_exp);
end;
```

Library pseudocode for shared/functions/float/fpbits/FPBitsType

```
// FPBitsType
// ==========

type FPBitsType of (FPFracBits, integer);
```

Library pseudocode for shared/functions/float/fpbits/FPFracBits

```
// FPFracBits
// ==========

type FPFracBits of integer{2, 3, 7, 10, 23, 52};
```

Library pseudocode for shared/functions/float/fpcompare/FPCompare

```
// FPCompare()
// ===========

func FPCompare{N}(op1 : bits(N), op2 : bits(N), signal_nans : boolean, fpcr : FPCR_Type) => bits(4)
begin
    assert N IN {16,32,64};
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);

    var result : bits(4);
    if type1 IN {FPType_SNaN, FPType_QNaN} || type2 IN {FPType_SNaN, FPType_QNaN} then
        result = '0011';
        if type1 == FPType_SNaN || type2 == FPType_SNaN || signal_nans then
            FPProcessException(FPExc_InvalidOp, fpcr);
        end;
    else
        // All non-NaN cases can be evaluated on the values produced by FPUnpack()
        if value1 == value2 then
            result = '0110';
        elsif value1 < value2 then
            result = '1000';
        else  // value1 > value2
            result = '0010';
        end;

        FPProcessDenorms(type1, type2, N, fpcr);
    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpcompareeq/FPCompareEQ

```
// FPCompareEQ()
// =============

func FPCompareEQ{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => boolean
begin
    assert N IN {16,32,64};
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);

    var result : boolean;
    if type1 IN {FPType_SNaN, FPType_QNaN} || type2 IN {FPType_SNaN, FPType_QNaN} then
        result = FALSE;
        if type1 == FPType_SNaN || type2 == FPType_SNaN then
            FPProcessException(FPExc_InvalidOp, fpcr);
        end;
    else
        // All non-NaN cases can be evaluated on the values produced by FPUnpack()
        result = (value1 == value2);
        FPProcessDenorms(type1, type2, N, fpcr);

    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpcomparege/FPCompareGE

```
// FPCompareGE()
// =============

func FPCompareGE{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => boolean
begin
    assert N IN {16,32,64};
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);

    var result : boolean;
    if type1 IN {FPType_SNaN, FPType_QNaN} || type2 IN {FPType_SNaN, FPType_QNaN} then
        result = FALSE;
        FPProcessException(FPExc_InvalidOp, fpcr);
    else
        // All non-NaN cases can be evaluated on the values produced by FPUnpack()
        result = (value1 >= value2);
        FPProcessDenorms(type1, type2, N, fpcr);

    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpcomparegt/FPCompareGT

```
// FPCompareGT()
// =============

func FPCompareGT{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => boolean
begin
    assert N IN {16,32,64};
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);

    var result : boolean;
    if type1 IN {FPType_SNaN, FPType_QNaN} || type2 IN {FPType_SNaN, FPType_QNaN} then
        result = FALSE;
        FPProcessException(FPExc_InvalidOp, fpcr);
    else
        // All non-NaN cases can be evaluated on the values produced by FPUnpack()
        result = (value1 > value2);
        FPProcessDenorms(type1, type2, N, fpcr);

    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpconvert/FPConvert

```
// FPConvert()
// ===========

// Convert floating point 'op' with N-bit precision to M-bit precision,
// with rounding controlled by ROUNDING.
// This is used by the FP-to-FP conversion instructions and so for
// half-precision data ignores FZ16, but observes AHP.

func FPConvert{M, N}(op : bits(N), fpcr : FPCR_Type, rounding : FPRounding) => bits(M)
begin

    assert M IN {16,32,64};
    assert N IN {16,32,64};
    var result : bits(M);

    // Unpack floating-point operand optionally with flush-to-zero.
    let (fptype,sign,value) : (FPType, bit, real) = FPUnpackCV{N}(op, fpcr);

    let alt_hp : boolean = (M == 16) && (fpcr.AHP == '1');

    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        if alt_hp then
            result = FPZero{M}(sign);
        elsif fpcr.DN == '1' then
            result = FPDefaultNaN{M}(fpcr);
        else
            result = FPConvertNaN{M, N}(op);
        end;
        if fptype == FPType_SNaN || alt_hp then
            FPProcessException(FPExc_InvalidOp,fpcr);
        end;
    elsif fptype == FPType_Infinity then
        if alt_hp then
            result = sign::Ones{M-1};
            FPProcessException(FPExc_InvalidOp, fpcr);
        else
            result = FPInfinity{M}(sign);
        end;
    elsif fptype == FPType_Zero then
        result = FPZero{M}(sign);
    else
        result = FPRoundCV{M}(value, fpcr, rounding);
        FPProcessDenorm(fptype, N, fpcr);

    end;
    return result;
end;

// FPConvert()
// ===========

func FPConvert{M, N}(op : bits(N), fpcr : FPCR_Type) => bits(M)
begin
    return FPConvert{M, N}(op, fpcr, FPRoundingMode(fpcr));
end;
```

Library pseudocode for shared/functions/float/fpconvertnan/FPConvertNaN

```
// FPConvertNaN()
// ==============
// Converts a NaN of one floating-point type to another

func FPConvertNaN{M, N}(op : bits(N)) => bits(M)
begin
    assert N IN {16,32,64};
    assert M IN {16,32,64};
    var result : bits(M);
    var frac : bits(51);

    let sign : bit = op[N-1];

    // Unpack payload from input NaN
    case N of
        when 64 => frac = op[50:0];
        when 32 => frac = op[21:0]::Zeros{29};
        when 16 => frac = op[8:0]::Zeros{42};
    end;

    // Repack payload into output NaN, while
    // converting an SNaN to a QNaN.
    case M of
        when 64 => result = sign::Ones{M-52}::frac;
        when 32 => result = sign::Ones{M-23}::frac[50:29];
        when 16 => result = sign::Ones{M-10}::frac[50:42];
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpdecoderm/FPDecodeRM

```
// FPDecodeRM()
// ============

// Decode most common AArch32 floating-point rounding encoding.

func FPDecodeRM(rm : bits(2)) => FPRounding
begin
    var result : FPRounding;
    case rm of
        when '00' => result = FPRounding_TIEAWAY; // A
        when '01' => result = FPRounding_TIEEVEN; // N
        when '10' => result = FPRounding_POSINF;  // P
        when '11' => result = FPRounding_NEGINF;  // M
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpdecoderounding/FPDecodeRounding

```
// FPDecodeRounding()
// ==================

// Decode floating-point rounding mode and common AArch64 encoding.

func FPDecodeRounding(rmode : bits(2)) => FPRounding
begin
    case rmode of
        when '00' => return FPRounding_TIEEVEN; // N
        when '01' => return FPRounding_POSINF;  // P
        when '10' => return FPRounding_NEGINF;  // M
        when '11' => return FPRounding_ZERO;    // Z
    end;
end;
```

Library pseudocode for shared/functions/float/fpdefaultnan/FPDefaultNaN

```
// FPDefaultNaN()
// ==============

func FPDefaultNaN{N}(fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{} = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{} = N - (E + 1);
    let sign : bit = if IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() then fpcr.AH else '0';

    let exp : bits(E)  = Ones{};
    let frac : bits(F) = '1'::Zeros{F-1};
    return sign :: exp :: frac;
end;
```

Library pseudocode for shared/functions/float/fpdiv/FPDiv

```
// FPDiv()
// =======

func FPDiv{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);
    var (done,result) : (boolean, bits(N)) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr);

    if !done then
        let inf1 : boolean  = type1 == FPType_Infinity;
        let inf2 : boolean  = type2 == FPType_Infinity;
        let zero1 : boolean = type1 == FPType_Zero;
        let zero2 : boolean = type2 == FPType_Zero;

        if (inf1 && inf2) || (zero1 && zero2) then
            result = FPDefaultNaN{N}(fpcr);
            FPProcessException(FPExc_InvalidOp, fpcr);
        elsif inf1 || zero2 then
            result = FPInfinity{N}(sign1 XOR sign2);
            if !inf1 then FPProcessException(FPExc_DivideByZero, fpcr); end;
        elsif zero1 || inf2 then
            result = FPZero{N}(sign1 XOR sign2);
        else
            result = FPRound{N}(value1/value2, fpcr);
        end;

        if !zero2 then
            FPProcessDenorms(type1, type2, N, fpcr);
        end;
    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpdot/FPDot

```
// FPDot()
// =======
// Calculates single-precision result of 2-way 16-bit floating-point dot-product
// with a single rounding.
// The 'fpcr' argument supplies the FPCR control bits and 'isbfloat16'
// determines whether input operands are BFloat16 or half-precision type.
// and 'fpexc' controls the generation of floating-point exceptions.

func FPDot(op1_a : bits(16), op1_b : bits(16), op2_a : bits(16),
           op2_b : bits(16), fpcr : FPCR_Type, isbfloat16 : boolean) => bits(32)
begin
    let fpexc : boolean = TRUE;       // Generate floating-point exceptions
    return FPDot(op1_a, op1_b, op2_a, op2_b, fpcr, isbfloat16, fpexc);
end;

func FPDot(op1_a : bits(16), op1_b : bits(16), op2_a : bits(16),
           op2_b : bits(16), fpcr_in : FPCR_Type, isbfloat16 : boolean, fpexc : boolean) => bits(32)
begin
    var fpcr : FPCR_Type = fpcr_in;
    var result : bits(32);
    var done : boolean;
    fpcr.AHP = '0';           // Ignore alternative half-precision option
    let rounding : FPRounding = FPRoundingMode(fpcr);

    let (type1_a,sign1_a,value1_a) = FPUnpackBase{16}(op1_a, fpcr, fpexc, isbfloat16);
    let (type1_b,sign1_b,value1_b) = FPUnpackBase{16}(op1_b, fpcr, fpexc, isbfloat16);
    let (type2_a,sign2_a,value2_a) = FPUnpackBase{16}(op2_a, fpcr, fpexc, isbfloat16);
    let (type2_b,sign2_b,value2_b) = FPUnpackBase{16}(op2_b, fpcr, fpexc, isbfloat16);

    let inf1_a = (type1_a == FPType_Infinity); let zero1_a = (type1_a == FPType_Zero);
    let inf1_b = (type1_b == FPType_Infinity); let zero1_b = (type1_b == FPType_Zero);
    let inf2_a = (type2_a == FPType_Infinity); let zero2_a = (type2_a == FPType_Zero);
    let inf2_b = (type2_b == FPType_Infinity); let zero2_b = (type2_b == FPType_Zero);

    (done,result) = FPProcessNaNs4(type1_a, type1_b, type2_a, type2_b,
                                   op1_a, op1_b, op2_a, op2_b, fpcr, fpexc);

    if !done then
        // Determine sign and type products will have if it does not cause an Invalid
        // Operation.
        let signPa = sign1_a XOR sign2_a;
        let signPb = sign1_b XOR sign2_b;
        let infPa  = inf1_a || inf2_a;
        let infPb  = inf1_b || inf2_b;
        let zeroPa = zero1_a || zero2_a;
        let zeroPb = zero1_b || zero2_b;

        // Non SNaN-generated Invalid Operation cases are multiplies of zero
        // by infinity and additions of opposite-signed infinities.
        let invalidop = ((inf1_a && zero2_a) || (zero1_a && inf2_a) ||
                         (inf1_b && zero2_b) || (zero1_b && inf2_b) ||
                         (infPa && infPb && signPa != signPb));

        if invalidop then
            result = FPDefaultNaN{32}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;

       // Other cases involving infinities produce an infinity of the same sign.
        elsif (infPa && signPa == '0') || (infPb && signPb == '0') then
            result = FPInfinity{32}('0');
        elsif (infPa && signPa == '1') || (infPb && signPb == '1') then
            result = FPInfinity{32}('1');

        // Cases where the result is exactly zero and its sign is not determined by the
        // rounding mode are additions of same-signed zeros.
        elsif zeroPa && zeroPb && signPa == signPb then
            result = FPZero{32}(signPa);

        // Otherwise calculate fused sum of products and round it.
        else
            let result_value = (value1_a * value2_a) + (value1_b * value2_b);
            if result_value == 0.0 then  // Sign of exact zero result depends on rounding mode
                let result_sign = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{32}(result_sign);
            else
                result = FPRound{32}(result_value, fpcr, rounding, fpexc);
            end;
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpdot/FPDotAdd

```
// FPDotAdd()
// ==========
// Half-precision 2-way dot-product and add to single-precision.

func FPDotAdd(addend : bits(32), op1_a : bits(16), op1_b : bits(16),
              op2_a : bits(16), op2_b : bits(16), fpcr : FPCR_Type) => bits(32)
begin
    var prod : bits(32);
    let isbfloat16 : boolean = FALSE;
    let fpexc : boolean      = TRUE;     // Generate floating-point exceptions
    prod = FPDot(op1_a, op1_b, op2_a, op2_b, fpcr, isbfloat16, fpexc);
    let result : bits(32) = FPAdd{}(addend, prod, fpcr, fpexc);

    return result;
end;
```

Library pseudocode for shared/functions/float/fpdot/FPDotAdd_ZA

```
// FPDotAdd_ZA()
// =============
// Half-precision 2-way dot-product and add to single-precision
// for SME ZA-targeting instructions.

func FPDotAdd_ZA(addend : bits(32), op1_a : bits(16), op1_b : bits(16),
                 op2_a : bits(16), op2_b : bits(16), fpcr_in : FPCR_Type) => bits(32)
begin
    var fpcr : FPCR_Type = fpcr_in;
    var prod : bits(32);
    let isbfloat16 : boolean = FALSE;
    let fpexc : boolean      = FALSE; // Do not generate floating-point exceptions
    fpcr.DN = '1';                    // Generate default NaN values
    prod = FPDot(op1_a, op1_b, op2_a, op2_b, fpcr, isbfloat16, fpexc);
    let result : bits(32) = FPAdd{}(addend, prod, fpcr, fpexc);

    return result;
end;
```

Library pseudocode for shared/functions/float/fpexc/FPExc

```
// FPExc
// =====

type FPExc of enumeration {FPExc_InvalidOp, FPExc_DivideByZero, FPExc_Overflow,
                           FPExc_Underflow, FPExc_Inexact, FPExc_InputDenorm};
```

Library pseudocode for shared/functions/float/fpexceptrapsenabled/FPExcepTrapsEnabled

```
// FPExcepTrapsEnabled()
// =====================

func FPExcepTrapsEnabled() => boolean
begin
    return (ImpDefBool("IMPLEMENTED_trapping of Input Denormal floating-point exceptions") ||
            ImpDefBool("IMPLEMENTED_trapping of Inexact floating-point exceptions") ||
            ImpDefBool("IMPLEMENTED_trapping of Underflow floating-point exceptions") ||
            ImpDefBool("IMPLEMENTED_trapping of Overflow floating-point exceptions") ||
            ImpDefBool("IMPLEMENTED_trapping of Divide by Zero floating-point exceptions") ||
            ImpDefBool("IMPLEMENTED_trapping of Invalid Operation floating-point exceptions"));
end;
```

Library pseudocode for shared/functions/float/fpinfinity/FPInfinity

```
// FPInfinity()
// ============

func FPInfinity{N}(sign : bit) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{}  = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{}  = N - (E + 1);
    let exp : bits(E)  = Ones{};
    let frac : bits(F) = Zeros{};
    return sign :: exp :: frac;
end;
```

Library pseudocode for shared/functions/float/fpmatmul/FPMatMulAdd

```
// FPMatMulAdd()
// =============
//
// Floating point matrix multiply and add to same precision matrix
// result[2, 2] = addend[2, 2] + (op1[2, 2] * op2[2, 2])

func FPMatMulAdd{N}(addend : bits(N), op1 : bits(N), op2 : bits(N),
                    esize : ESize, fpcr : FPCR_Type) => bits(N)
begin
    assert N == esize * 2 * 2;
    var result : bits(N);
    var prod0, prod1, sum : bits(esize);

    for i = 0 to 1 do
        for j = 0 to 1 do
            sum   = addend[(2*i + j)*:esize];
            prod0 = FPMul{esize}(op1[(2*i + 0)*:esize],
                                 op2[(2*j + 0)*:esize], fpcr);
            prod1 = FPMul{esize}(op1[(2*i + 1)*:esize],
                                 op2[(2*j + 1)*:esize], fpcr);
            sum   = FPAdd{esize}(sum, FPAdd{esize}(prod0, prod1, fpcr), fpcr);
            result[(2*i + j)*:esize] = sum;
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpmatmulh/FPMatMulAddH

```
// FPMatMulAddH()
// ==============
// Half-precision matrix multiply and add to single-precision matrix
// result[2, 2] = addend[2, 2] + (op1[2, 4] * op2[4, 2])

func FPMatMulAddH{N}(addend : bits(N), op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N == 128;
    let M : integer{} = 32;
    var result : bits(N);

    let isbfloat16 : boolean = FALSE;
    for i = 0 to 1 do
        for j = 0 to 1 do
            var sum : bits(M) = addend[(2*i + j)*:M];
            var prod : array[[2]] of bits(M);
            for k = 0 to 1 do
                let elt1_a : bits(M DIV 2) = op1[(4*i + 2*k + 0)*:(M DIV 2)];
                let elt1_b : bits(M DIV 2) = op1[(4*i + 2*k + 1)*:(M DIV 2)];
                let elt2_a : bits(M DIV 2) = op2[(4*j + 2*k + 0)*:(M DIV 2)];
                let elt2_b : bits(M DIV 2) = op2[(4*j + 2*k + 1)*:(M DIV 2)];
                prod[[k]] = FPDot(elt1_a, elt1_b, elt2_a, elt2_b, fpcr, isbfloat16);
            end;
            sum = FPAdd{M}(sum, FPAdd{M}(prod[[0]], prod[[1]], fpcr), fpcr);
            result[(2*i + j)*:M] = sum;
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpmax/FPMax

```
// FPMax()
// =======

func FPMax{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let fpexc : boolean = TRUE;
    return FPMax{N}(op1, op2, fpcr, altfp, fpexc);
end;

// FPMax()
// =======

func FPMax{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type, altfp : boolean) => bits(N)
begin
    let fpexc : boolean = TRUE;
    return FPMax{N}(op1, op2, fpcr, altfp, fpexc);
end;

// FPMax()
// =======
// Compare two inputs and return the larger value after rounding. The
// 'fpcr' argument supplies the FPCR control bits and 'altfp' determines
// if the function should use alternative floating-point behavior.

func FPMax{N}(op1 : bits(N), op2 : bits(N), fpcr_in : FPCR_Type,
              altfp : boolean, fpexc : boolean) => bits(N)
begin
    assert N IN {16,32,64};
    var done : boolean;
    var result : bits(N);
    var fpcr : FPCR_Type = fpcr_in;
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr, fpexc);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr, fpexc);

    if altfp && type1 == FPType_Zero && type2 == FPType_Zero && sign1 != sign2 then
        // Alternate handling of zeros with differing sign
        return FPZero{N}(sign2);
    elsif altfp && (type1 IN {FPType_SNaN, FPType_QNaN} || type2 IN {FPType_SNaN, FPType_QNaN}) then
        // Alternate handling of NaN inputs
        if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        return (if type2 == FPType_Zero then FPZero{N}(sign2) else op2);
    end;

    (done,result) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr, fpexc);
    if !done then
        var fptype : FPType;
        var sign : bit;
        var value : real;
        if value1 > value2 then
            (fptype,sign,value) = (type1,sign1,value1);
        else
            (fptype,sign,value) = (type2,sign2,value2);
        end;
        if fptype == FPType_Infinity then
            result = FPInfinity{N}(sign);
        elsif fptype == FPType_Zero then
            sign = sign1 AND sign2;         // Use most positive sign
            result = FPZero{N}(sign);
        else
            // The use of FPRound() covers the case where there is a trapped underflow exception
            // for a denormalized number even though the result is exact.
            let rounding : FPRounding = FPRoundingMode(fpcr);
            if altfp then    // Denormal output is not flushed to zero
                fpcr.FZ = '0';
                fpcr.FZ16 = '0';
            end;

            result = FPRound{N}(value, fpcr, rounding, fpexc);
        end;
        if fpexc then FPProcessDenorms(type1, type2, N, fpcr); end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpmaxnormal/FPMaxNormal

```
// FPMaxNormal()
// =============

func FPMaxNormal{N}(sign : bit) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{}  = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{}  = N - (E + 1);
    let exp : bits(E)  = Ones{E-1}::'0';
    let frac : bits(F) = Ones{};
    return sign :: exp :: frac;
end;
```

Library pseudocode for shared/functions/float/fpmaxnum/FPMaxNum

```
// FPMaxNum()
// ==========

func FPMaxNum{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let fpexc : boolean = TRUE;
    return FPMaxNum{N}(op1, op2, fpcr, fpexc);
end;

// FPMaxNum()
// ==========

func FPMaxNum{N}(op1_in : bits(N), op2_in : bits(N), fpcr : FPCR_Type, fpexc : boolean) => bits(N)
begin
    assert N IN {16,32,64};
    var op1 : bits(N) = op1_in;
    var op2 : bits(N) = op2_in;
    let (type1,-,-) = FPUnpack{N}(op1, fpcr, fpexc);
    let (type2,-,-) = FPUnpack{N}(op2, fpcr, fpexc);

    let type1_nan : boolean = type1 IN {FPType_QNaN, FPType_SNaN};
    let type2_nan : boolean = type2 IN {FPType_QNaN, FPType_SNaN};
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';

    if !(altfp && type1_nan && type2_nan) then
        // Treat a single quiet-NaN as -Infinity.
        if type1 == FPType_QNaN && type2 != FPType_QNaN then
            op1 = FPInfinity{N}('1');
        elsif type1 != FPType_QNaN && type2 == FPType_QNaN then
            op2 = FPInfinity{N}('1');
        end;
    end;

    let altfmaxfmin : boolean = FALSE;    // Restrict use of FMAX/FMIN NaN propagation rules
    let result : bits(N) = FPMax{}(op1, op2, fpcr, altfmaxfmin, fpexc);

    return result;
end;
```

Library pseudocode for shared/functions/float/fpmerge/IsMerging

```
// IsMerging()
// ===========
// Returns TRUE if the output elements other than the lowest are taken from
// the destination register.

func IsMerging(fpcr : FPCR_Type) => boolean
begin
    let nep : bit = (if IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' &&
                        !IsFullA64Enabled() then '0' else fpcr.NEP);
    return IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && nep == '1';
end;
```

Library pseudocode for shared/functions/float/fpmin/FPMin

```
// FPMin()
// =======

func FPMin{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let fpexc : boolean = TRUE;
    return FPMin{N}(op1, op2, fpcr, altfp, fpexc);
end;

// FPMin()
// =======

func FPMin{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type, altfp : boolean) => bits(N)
begin
    let fpexc : boolean = TRUE;
    return FPMin{N}(op1, op2, fpcr, altfp, fpexc);
end;

// FPMin()
// =======
// Compare two inputs and return the smaller operand after rounding. The
// 'fpcr' argument supplies the FPCR control bits and 'altfp' determines
// if the function should use alternative floating-point behavior.

func FPMin{N}(op1 : bits(N), op2 : bits(N), fpcr_in : FPCR_Type,
              altfp : boolean, fpexc : boolean) => bits(N)
begin
    assert N IN {16,32,64};
    var done : boolean;
    var result : bits(N);
    var fpcr : FPCR_Type = fpcr_in;
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr, fpexc);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr, fpexc);

    if altfp && type1 == FPType_Zero && type2 == FPType_Zero && sign1 != sign2 then
        // Alternate handling of zeros with differing sign
        return FPZero{N}(sign2);
    elsif altfp && (type1 IN {FPType_SNaN, FPType_QNaN} || type2 IN {FPType_SNaN, FPType_QNaN}) then
        // Alternate handling of NaN inputs
        if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        return (if type2 == FPType_Zero then FPZero{N}(sign2) else op2);
    end;

    (done,result) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr, fpexc);
    if !done then
        var fptype : FPType;
        var sign : bit;
        var value : real;
        var rounding : FPRounding;
        if value1 < value2 then
            (fptype,sign,value) = (type1,sign1,value1);
        else
            (fptype,sign,value) = (type2,sign2,value2);
        end;
        if fptype == FPType_Infinity then
            result = FPInfinity{N}(sign);
        elsif fptype == FPType_Zero then
            sign = sign1 OR sign2;              // Use most negative sign
            result = FPZero{N}(sign);
        else
            // The use of FPRound() covers the case where there is a trapped underflow exception
            // for a denormalized number even though the result is exact.
            rounding = FPRoundingMode(fpcr);
            if altfp then    // Denormal output is not flushed to zero
                fpcr.FZ = '0';
                fpcr.FZ16 = '0';
            end;

            result = FPRound{N}(value, fpcr, rounding, fpexc);
        end;

        if fpexc then FPProcessDenorms(type1, type2, N, fpcr); end;
    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpminnum/FPMinNum

```
// FPMinNum()
// ==========

func FPMinNum{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let fpexc : boolean = TRUE;
    return FPMinNum{N}(op1, op2, fpcr, fpexc);
end;

// FPMinNum()
// ==========

func FPMinNum{N}(op1_in : bits(N), op2_in : bits(N), fpcr : FPCR_Type, fpexc : boolean) => bits(N)
begin
    assert N IN {16,32,64};
    var op1 : bits(N) = op1_in;
    var op2 : bits(N) = op2_in;
    let (type1,-,-) = FPUnpack{N}(op1, fpcr, fpexc);
    let (type2,-,-) = FPUnpack{N}(op2, fpcr, fpexc);

    let type1_nan : boolean = type1 IN {FPType_QNaN, FPType_SNaN};
    let type2_nan : boolean = type2 IN {FPType_QNaN, FPType_SNaN};
    let altfp : boolean     = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() &&
                                                                fpcr.AH == '1';

    if !(altfp && type1_nan && type2_nan) then
        // Treat a single quiet-NaN as +Infinity.
        if type1 == FPType_QNaN && type2 != FPType_QNaN then
            op1 = FPInfinity{N}('0');
        elsif type1 != FPType_QNaN && type2 == FPType_QNaN then
            op2 = FPInfinity{N}('0');
        end;
    end;

    let altfmaxfmin : boolean = FALSE;    // Restrict use of FMAX/FMIN NaN propagation rules
    let result : bits(N) = FPMin{}(op1, op2, fpcr, altfmaxfmin, fpexc);

    return result;
end;
```

Library pseudocode for shared/functions/float/fpmul/FPMul

```
// FPMul()
// =======

func FPMul{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);
    var (done,result) : (boolean, bits(N)) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr);
    if !done then
        let inf1 : boolean  = (type1 == FPType_Infinity);
        let inf2 : boolean  = (type2 == FPType_Infinity);
        let zero1 : boolean = (type1 == FPType_Zero);
        let zero2 : boolean = (type2 == FPType_Zero);

        if (inf1 && zero2) || (zero1 && inf2) then
            result = FPDefaultNaN{N}(fpcr);
            FPProcessException(FPExc_InvalidOp, fpcr);
        elsif inf1 || inf2 then
            result = FPInfinity{N}(sign1 XOR sign2);
        elsif zero1 || zero2 then
            result = FPZero{N}(sign1 XOR sign2);
        else
            result = FPRound{N}(value1*value2, fpcr);
        end;

        FPProcessDenorms(type1, type2, N, fpcr);

    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpmuladd/FPMulAdd

```
// FPMulAdd()
// ==========

func FPMulAdd{N}(addend : bits(N), op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let fpexc : boolean = TRUE;       // Generate floating-point exceptions
    return FPMulAdd{N}(addend, op1, op2, fpcr, fpexc);
end;

// FPMulAdd()
// ==========
//
// Calculates addend + op1*op2 with a single rounding. The 'fpcr' argument
// supplies the FPCR control bits, and 'fpexc' controls the generation of
// floating-point exceptions.

func FPMulAdd{N}(addend : bits(N), op1 : bits(N), op2 : bits(N),
                 fpcr : FPCR_Type, fpexc : boolean) => bits(N)
begin
    assert N IN {16,32,64};

    let (typeA,signA,valueA) : (FPType, bit, real) = FPUnpack{N}(addend, fpcr, fpexc);
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr, fpexc);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr, fpexc);
    let rounding : FPRounding = FPRoundingMode(fpcr);
    let inf1 : boolean = (type1 == FPType_Infinity); let zero1 : boolean = (type1 == FPType_Zero);
    let inf2 : boolean = (type2 == FPType_Infinity); let zero2 : boolean = (type2 == FPType_Zero);

    var (done,result) = FPProcessNaNs3{N}(typeA, type1, type2, addend, op1, op2, fpcr, fpexc);

    if !(IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1') then
        if typeA == FPType_QNaN && ((inf1 && zero2) || (zero1 && inf2)) then
            result = FPDefaultNaN{N}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        end;
    end;

    if !done then
        let infA : boolean = (typeA == FPType_Infinity);
        let zeroA : boolean = (typeA == FPType_Zero);

        // Determine sign and type product will have if it does not cause an
        // Invalid Operation.
        let signP : bit = sign1 XOR sign2;
        let infP : boolean  = inf1 || inf2;
        let zeroP : boolean = zero1 || zero2;

        // Non SNaN-generated Invalid Operation cases are multiplies of zero
        // by infinity and additions of opposite-signed infinities.
        let invalidop : boolean = ((inf1 && zero2) || (zero1 && inf2) ||
                                   (infA && infP && signA != signP));

        if invalidop then
            result = FPDefaultNaN{N}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        // Other cases involving infinities produce an infinity of the same sign.
        elsif (infA && signA == '0') || (infP && signP == '0') then
            result = FPInfinity{N}('0');
        elsif (infA && signA == '1') || (infP && signP == '1') then
            result = FPInfinity{N}('1');

        // Cases where the result is exactly zero and its sign is not determined by the
        // rounding mode are additions of same-signed zeros.
        elsif zeroA && zeroP && signA == signP then
            result = FPZero{N}(signA);

        // Otherwise calculate numerical result and round it.
        else
            let result_value : real = valueA + (value1 * value2);
            if result_value == 0.0 then  // Sign of exact zero result depends on rounding mode
                let result_sign : bit = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{N}(result_sign);
            else
                result = FPRound{N}(result_value, fpcr, rounding, fpexc);
            end;
        end;

        if !invalidop && fpexc then
            FPProcessDenorms3(typeA, type1, type2, N, fpcr);
        end;
    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpmuladd/FPMulAdd_ZA

```
// FPMulAdd_ZA()
// =============
// Calculates addend + op1*op2 with a single rounding for SME ZA-targeting
// instructions.

func FPMulAdd_ZA{N}(addend : bits(N), op1 : bits(N), op2 : bits(N), fpcr_in : FPCR_Type) => bits(N)
begin
    var fpcr : FPCR_Type = fpcr_in;
    let fpexc : boolean  = FALSE; // Do not generate floating-point exceptions
    fpcr.DN = '1';                // Generate default NaN values
    return FPMulAdd{N}(addend, op1, op2, fpcr, fpexc);
end;
```

Library pseudocode for shared/functions/float/fpmuladdh/FPMulAddH

```
// FPMulAddH()
// ===========
// Calculates addend + op1*op2.

func FPMulAddH(addend : bits(32), op1 : bits(16), op2 : bits(16), fpcr : FPCR_Type) => bits(32)
begin
    let fpexc : boolean = TRUE;       // Generate floating-point exceptions
    return FPMulAddH(addend, op1, op2, fpcr, fpexc);
end;

// FPMulAddH()
// ===========
// Calculates addend + op1*op2.

func FPMulAddH(addend : bits(32), op1 : bits(16), op2 : bits(16),
               fpcr : FPCR_Type, fpexc : boolean) => bits(32)
begin

    let rounding : FPRounding = FPRoundingMode(fpcr);
    let (typeA,signA,valueA)  = FPUnpack{32}(addend, fpcr, fpexc);
    let (type1,sign1,value1)  = FPUnpack{16}(op1, fpcr, fpexc);
    let (type2,sign2,value2)  = FPUnpack{16}(op2, fpcr, fpexc);
    let inf1 = (type1 == FPType_Infinity); let zero1 = (type1 == FPType_Zero);
    let inf2 = (type2 == FPType_Infinity); let zero2 = (type2 == FPType_Zero);

    var (done,result) = FPProcessNaNs3H(typeA, type1, type2, addend, op1, op2, fpcr, fpexc);

    if !(IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1') then
        if typeA == FPType_QNaN && ((inf1 && zero2) || (zero1 && inf2)) then
            result = FPDefaultNaN{32}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        end;
    end;

    if !done then
        let infA = (typeA == FPType_Infinity); let zeroA = (typeA == FPType_Zero);

        // Determine sign and type product will have if it does not cause an
        // Invalid Operation.
        let signP = sign1 XOR sign2;
        let infP  = inf1 || inf2;
        let zeroP = zero1 || zero2;

        // Non SNaN-generated Invalid Operation cases are multiplies of zero by infinity and
        // additions of opposite-signed infinities.
        let invalidop = (inf1 && zero2) || (zero1 && inf2) || (infA && infP && signA != signP);

        if invalidop then
            result = FPDefaultNaN{32}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;

        // Other cases involving infinities produce an infinity of the same sign.
        elsif (infA && signA == '0') || (infP && signP == '0') then
            result = FPInfinity{32}('0');
        elsif (infA && signA == '1') || (infP && signP == '1') then
            result = FPInfinity{32}('1');

        // Cases where the result is exactly zero and its sign is not determined by the
        // rounding mode are additions of same-signed zeros.
        elsif zeroA && zeroP && signA == signP then
            result = FPZero{32}(signA);

        // Otherwise calculate numerical result and round it.
        else
            let result_value = valueA + (value1 * value2);
            if result_value == 0.0 then // Sign of exact zero result depends on rounding mode
                let result_sign = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{32}(result_sign);
            else
                result = FPRound{32}(result_value, fpcr, rounding, fpexc);
            end;
        end;

        if !invalidop && fpexc then
            FPProcessDenorm(typeA, 32, fpcr);
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpmuladdh/FPMulAddH_ZA

```
// FPMulAddH_ZA()
// ==============
// Calculates addend + op1*op2 for SME2 ZA-targeting instructions.

func FPMulAddH_ZA(addend : bits(32), op1 : bits(16), op2 : bits(16),
                  fpcr_in : FPCR_Type) => bits(32)
begin
    var fpcr : FPCR_Type = fpcr_in;
    let fpexc : boolean = FALSE; // Do not generate floating-point exceptions
    fpcr.DN = '1';               // Generate default NaN values
    return FPMulAddH(addend, op1, op2, fpcr, fpexc);
end;
```

Library pseudocode for shared/functions/float/fpmuladdh/FPProcessNaNs3H

```
// FPProcessNaNs3H()
// =================

func FPProcessNaNs3H(type1 : FPType, type2 : FPType, type3 : FPType,
                     op1 : bits(32), op2 : bits(16), op3 : bits(16),
                     fpcr : FPCR_Type, fpexc : boolean) => (boolean, bits(32))
begin

    var result : bits(32);
    var type_nan : FPType;
    // When TRUE, use alternative NaN propagation rules.
    let altfp : boolean   = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let op1_nan : boolean = type1 IN {FPType_SNaN, FPType_QNaN};
    let op2_nan : boolean = type2 IN {FPType_SNaN, FPType_QNaN};
    let op3_nan : boolean = type3 IN {FPType_SNaN, FPType_QNaN};
    if altfp then
        if (type1 == FPType_SNaN || type2 == FPType_SNaN || type3 == FPType_SNaN) then
            type_nan = FPType_SNaN;
        else
            type_nan = FPType_QNaN;
        end;
    end;

    var done : boolean;
    if altfp && op1_nan && op2_nan && op3_nan then          // [n] register NaN selected
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type_nan, op2, fpcr, fpexc));
    elsif altfp && op2_nan && (op1_nan || op3_nan) then     // [n] register NaN selected
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type_nan, op2, fpcr, fpexc));
    elsif altfp && op3_nan && op1_nan then                  // [m] register NaN selected
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type_nan, op3, fpcr, fpexc));
    elsif type1 == FPType_SNaN then
        done = TRUE; result = FPProcessNaN{32}(type1, op1, fpcr, fpexc);
    elsif type2 == FPType_SNaN then
        done = TRUE; result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type2, op2, fpcr, fpexc));
    elsif type3 == FPType_SNaN then
        done = TRUE; result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type3, op3, fpcr, fpexc));
    elsif type1 == FPType_QNaN then
        done = TRUE; result = FPProcessNaN{32}(type1, op1, fpcr, fpexc);
    elsif type2 == FPType_QNaN then
        done = TRUE; result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type2, op2, fpcr, fpexc));
    elsif type3 == FPType_QNaN then
        done = TRUE; result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type3, op3, fpcr, fpexc));
    else
        done = FALSE; result = Zeros{32}; // 'Don't care' result
    end;
    return (done, result);
end;
```

Library pseudocode for shared/functions/float/fpmulx/FPMulX

```
// FPMulX()
// ========

func FPMulX{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var result : bits(N);
    var done : boolean;
    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr);

    (done,result) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr);
    if !done then
        let inf1 : boolean = (type1 == FPType_Infinity);
        let inf2 : boolean = (type2 == FPType_Infinity);
        let zero1 : boolean = (type1 == FPType_Zero);
        let zero2 : boolean = (type2 == FPType_Zero);

        if (inf1 && zero2) || (zero1 && inf2) then
            result = FPTwo{N}(sign1 XOR sign2);
        elsif inf1 || inf2 then
            result = FPInfinity{N}(sign1 XOR sign2);
        elsif zero1 || zero2 then
            result = FPZero{N}(sign1 XOR sign2);
        else
            result = FPRound{N}(value1*value2, fpcr);
        end;

        FPProcessDenorms(type1, type2, N, fpcr);

    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpneg/FPNeg

```
// FPNeg()
// =======

func FPNeg{N}(op : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    if !UsingAArch32() && IsFeatureImplemented(FEAT_AFP) then
        if fpcr.AH == '1' then
            let (fptype, -, -) = FPUnpack{N}(op, fpcr, FALSE);
            if fptype IN {FPType_SNaN, FPType_QNaN} then
                return op;        // When fpcr.AH=1, sign of NaN has no consequence
            end;
        end;
    end;
    return NOT(op[N-1]) :: op[N-2:0];
end;
```

Library pseudocode for shared/functions/float/fponepointfive/FPOnePointFive

```
// FPOnePointFive()
// ================

func FPOnePointFive{N}(sign : bit) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{}  = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{}  = N - (E + 1);
    let exp : bits(E)  = '0'::Ones{E-1};
    let frac : bits(F) = '1'::Zeros{F-1};
    let result : bits((F + E) + 1) = sign :: exp :: frac;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpprocessdenorms/FPProcessDenorm

```
// FPProcessDenorm()
// =================
// Handles denormal input in case of single-precision or double-precision
// when using alternative floating-point mode.

func FPProcessDenorm(fptype : FPType, N : integer, fpcr : FPCR_Type)
begin
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    if altfp && N != 16 && fptype == FPType_Denormal then
        FPProcessException(FPExc_InputDenorm, fpcr);
    end;
end;
```

Library pseudocode for shared/functions/float/fpprocessdenorms/FPProcessDenorms

```
// FPProcessDenorms()
// ==================
// Handles denormal input in case of single-precision or double-precision
// when using alternative floating-point mode.

func FPProcessDenorms(type1 : FPType, type2 : FPType, N : integer, fpcr : FPCR_Type)
begin
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    if altfp && N != 16 && (type1 == FPType_Denormal || type2 == FPType_Denormal) then
        FPProcessException(FPExc_InputDenorm, fpcr);
    end;
end;
```

Library pseudocode for shared/functions/float/fpprocessdenorms/FPProcessDenorms3

```
// FPProcessDenorms3()
// ===================
// Handles denormal input in case of single-precision or double-precision
// when using alternative floating-point mode.

func FPProcessDenorms3(type1 : FPType, type2 : FPType, type3 : FPType, N : integer,
                       fpcr : FPCR_Type)
begin
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    if altfp && N != 16 && (type1 == FPType_Denormal || type2 == FPType_Denormal ||
        type3 == FPType_Denormal) then
        FPProcessException(FPExc_InputDenorm, fpcr);
    end;
end;
```

Library pseudocode for shared/functions/float/fpprocessexception/FPProcessException

```
// FPProcessException()
// ====================
//
// The 'fpcr' argument supplies FPCR control bits. Status information is
// updated directly in the FPSR where appropriate.

func FPProcessException(except : FPExc, fpcr : FPCR_Type)
begin
    var cumul : integer;
    // Determine the cumulative exception bit number
    case except of
        when FPExc_InvalidOp =>     cumul = 0;
        when FPExc_DivideByZero =>  cumul = 1;
        when FPExc_Overflow =>      cumul = 2;
        when FPExc_Underflow =>     cumul = 3;
        when FPExc_Inexact =>       cumul = 4;
        when FPExc_InputDenorm =>   cumul = 7;
    end;
    // When FEAT_TRAPFPE is implemented, the control bits FPCR.{IDE, IXE, UFE, OFE, DZE, IOE}
    // can be independently set to 1.
    // Otherwise when FEAT_TRAPFPE is not implemented, bits FPCR.{IDE, IXE, UFE, OFE, DZE, IOE}
    // are RAZ/WI.
    let enable : integer = cumul + 8;
    if (fpcr[enable] == '1' && (!IsFeatureImplemented(FEAT_SME) || PSTATE.SM == '0' ||
          IsFullA64Enabled())) then
        // Trapping of the exception enabled.
        // It is IMPLEMENTATION DEFINED how exceptions and in what order that they may be
        // accumulated before calling FPTrappedException().
        var accumulated_exceptions : bits(8) = GetAccumulatedFPExceptions();
        accumulated_exceptions[cumul] = '1';
        if !ImpDefBool("Accumulate floating-point exceptions before trapping") then
            if UsingAArch32() then
                AArch32_FPTrappedException(accumulated_exceptions);
            else
                let is_ase : boolean = IsASEInstruction();
                AArch64_FPTrappedException(is_ase, accumulated_exceptions);
            end;
        else
            // The exceptions generated by this instruction are accumulated by the PE and
            // FPTrappedException is called later during its execution, before the next
            // instruction is executed. This field is cleared at the start of each FP instruction.
            SetAccumulatedFPExceptions(accumulated_exceptions);
        end;
    elsif UsingAArch32() then
        // Set the cumulative exception bit
        FPSCR()[cumul] = '1';
    else
        // Set the cumulative exception bit
        FPSR()[cumul] = '1';
    end;
    return;
end;
```

Library pseudocode for shared/functions/float/fpprocessnan/FPProcessNaN

```
// FPProcessNaN()
// ==============

func FPProcessNaN{N}(fptype : FPType, op : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let fpexc : boolean = TRUE;   // Generate floating-point exceptions
    return FPProcessNaN{N}(fptype, op, fpcr, fpexc);
end;

// FPProcessNaN()
// ==============
// Handle NaN input operands, returning the operand or default NaN value
// if fpcr.DN is selected. The 'fpcr' argument supplies the FPCR control bits.
// The 'fpexc' argument controls the generation of exceptions, regardless of
// whether 'fptype' is a signaling NaN or a quiet NaN.

func FPProcessNaN{N}(fptype : FPType, op : bits(N), fpcr : FPCR_Type, fpexc : boolean) => bits(N)
begin
    assert N IN {16,32,64};
    assert fptype IN {FPType_QNaN, FPType_SNaN};
    var topfrac : integer;

    case N of
        when 16 => topfrac =  9;
        when 32 => topfrac = 22;
        when 64 => topfrac = 51;
    end;

    var result : bits(N) = op;
    if fptype == FPType_SNaN then
        result[topfrac] = '1';
        if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
    end;
    if fpcr.DN == '1' then  // DefaultNaN requested
        result = FPDefaultNaN{N}(fpcr);
    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpprocessnans/FPProcessNaNs

```
// FPProcessNaNs()
// ===============

func FPProcessNaNs{N}(type1 : FPType, type2 : FPType, op1 : bits(N),
                      op2 : bits(N), fpcr : FPCR_Type) => (boolean, bits(N))
begin
    let fpexc : boolean = TRUE;     // Generate floating-point exceptions
    return FPProcessNaNs{N}(type1, type2, op1, op2, fpcr, fpexc);
end;

// FPProcessNaNs()
// ===============
//
// The boolean part of the return value says whether a NaN has been found and
// processed. The bits(N) part is only relevant if it has and supplies the
// result of the operation.
//
// The 'fpcr' argument supplies FPCR control bits and 'altfmaxfmin' controls
// alternative floating-point behavior for FMAX, FMIN and variants. 'fpexc'
// controls the generation of floating-point exceptions. Status information
// is updated directly in the FPSR where appropriate.

func FPProcessNaNs{N}(type1 : FPType, type2 : FPType, op1 : bits(N), op2 : bits(N),
                      fpcr : FPCR_Type, fpexc : boolean) => (boolean, bits(N))
begin
    assert N IN {16,32,64};
    var done : boolean;
    var result : bits(N);
    let altfp : boolean    = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let op1_nan : boolean  = type1 IN {FPType_SNaN, FPType_QNaN};
    let op2_nan : boolean  = type2 IN {FPType_SNaN, FPType_QNaN};
    let any_snan : boolean = type1 == FPType_SNaN || type2 == FPType_SNaN;
    let type_nan : FPType = if any_snan then FPType_SNaN else FPType_QNaN;

    if altfp && op1_nan && op2_nan then
        // [n] register NaN selected
        done = TRUE;  result = FPProcessNaN{N}(type_nan, op1, fpcr, fpexc);
    elsif type1 == FPType_SNaN then
        done = TRUE;  result = FPProcessNaN{N}(type1, op1, fpcr, fpexc);
    elsif type2 == FPType_SNaN then
        done = TRUE;  result = FPProcessNaN{N}(type2, op2, fpcr, fpexc);
    elsif type1 == FPType_QNaN then
        done = TRUE;  result = FPProcessNaN{N}(type1, op1, fpcr, fpexc);
    elsif type2 == FPType_QNaN then
        done = TRUE;  result = FPProcessNaN{N}(type2, op2, fpcr, fpexc);
    else
        done = FALSE;  result = Zeros{N};  // 'Don't care' result
    end;
    return (done, result);
end;
```

Library pseudocode for shared/functions/float/fpprocessnans3/FPProcessNaNs3

```
// FPProcessNaNs3()
// ================

func FPProcessNaNs3{N}(type1 : FPType, type2 : FPType, type3 : FPType,
                       op1 : bits(N), op2 : bits(N), op3 : bits(N),
                       fpcr : FPCR_Type) => (boolean, bits(N))
begin
    let fpexc : boolean = TRUE;   // Generate floating-point exceptions
    return FPProcessNaNs3{N}(type1, type2, type3, op1, op2, op3, fpcr, fpexc);
end;

// FPProcessNaNs3()
// ================
// The boolean part of the return value says whether a NaN has been found and
// processed. The bits(N) part is only relevant if it has and supplies the
// result of the operation.
//
// The 'fpcr' argument supplies FPCR control bits and 'fpexc' controls the
// generation of floating-point exceptions. Status information is updated
// directly in the FPSR where appropriate.

func FPProcessNaNs3{N}(type1 : FPType, type2 : FPType, type3 : FPType,
                       op1 : bits(N), op2 : bits(N), op3 : bits(N),
                       fpcr : FPCR_Type, fpexc : boolean) => (boolean, bits(N))
begin
    assert N IN {16,32,64};
    var result : bits(N);
    let op1_nan : boolean = type1 IN {FPType_SNaN, FPType_QNaN};
    let op2_nan : boolean = type2 IN {FPType_SNaN, FPType_QNaN};
    let op3_nan : boolean = type3 IN {FPType_SNaN, FPType_QNaN};

    let altfp : boolean   = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    var type_nan : FPType;
    if altfp then
        if type1 == FPType_SNaN || type2 == FPType_SNaN || type3 == FPType_SNaN then
            type_nan = FPType_SNaN;
        else
            type_nan = FPType_QNaN;
        end;
    end;

    var done : boolean;
    if altfp && op1_nan && op2_nan && op3_nan then
        // [n] register NaN selected
        done = TRUE;  result = FPProcessNaN{N}(type_nan, op2, fpcr, fpexc);
    elsif altfp && op2_nan && (op1_nan || op3_nan) then
        // [n] register NaN selected
        done = TRUE;  result = FPProcessNaN{N}(type_nan, op2, fpcr, fpexc);
    elsif altfp && op3_nan && op1_nan then
        // [m] register NaN selected
        done = TRUE;  result = FPProcessNaN{N}(type_nan, op3, fpcr, fpexc);
    elsif type1 == FPType_SNaN then
        done = TRUE;  result = FPProcessNaN{N}(type1, op1, fpcr, fpexc);
    elsif type2 == FPType_SNaN then
        done = TRUE;  result = FPProcessNaN{N}(type2, op2, fpcr, fpexc);
    elsif type3 == FPType_SNaN then
        done = TRUE;  result = FPProcessNaN{N}(type3, op3, fpcr, fpexc);
    elsif type1 == FPType_QNaN then
        done = TRUE;  result = FPProcessNaN{N}(type1, op1, fpcr, fpexc);
    elsif type2 == FPType_QNaN then
        done = TRUE;  result = FPProcessNaN{N}(type2, op2, fpcr, fpexc);
    elsif type3 == FPType_QNaN then
        done = TRUE;  result = FPProcessNaN{N}(type3, op3, fpcr, fpexc);
    else
        done = FALSE;  result = Zeros{N};  // 'Don't care' result
    end;
    return (done, result);
end;
```

Library pseudocode for shared/functions/float/fpprocessnans4/FPProcessNaNs4

```
// FPProcessNaNs4()
// ================
// The boolean part of the return value says whether a NaN has been found and
// processed. The bits(N) part is only relevant if it has and supplies the
// result of the operation.
//
// The 'fpcr' argument supplies FPCR control bits.
// Status information is updated directly in the FPSR where appropriate.
// The 'fpexc' controls the generation of floating-point exceptions.

func FPProcessNaNs4(type1 : FPType, type2 : FPType, type3 : FPType, type4 : FPType,
                    op1 : bits(16), op2 : bits(16), op3 : bits(16),
                    op4 : bits(16), fpcr : FPCR_Type, fpexc : boolean) => (boolean, bits(32))
begin

    var result : bits(32);
    var done : boolean;
    // The FPCR.AH control does not affect these checks
    if type1 == FPType_SNaN then
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type1, op1, fpcr, fpexc));
    elsif type2 == FPType_SNaN then
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type2, op2, fpcr, fpexc));
    elsif type3 == FPType_SNaN then
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type3, op3, fpcr, fpexc));
    elsif type4 == FPType_SNaN then
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type4, op4, fpcr, fpexc));
    elsif type1 == FPType_QNaN then
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type1, op1, fpcr, fpexc));
    elsif type2 == FPType_QNaN then
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type2, op2, fpcr, fpexc));
    elsif type3 == FPType_QNaN then
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type3, op3, fpcr, fpexc));
    elsif type4 == FPType_QNaN then
        done = TRUE;  result = FPConvertNaN{32, 16}(FPProcessNaN{16}(type4, op4, fpcr, fpexc));
    else
        done = FALSE;  result = Zeros{32};  // 'Don't care' result
    end;

    return (done, result);
end;
```

Library pseudocode for shared/functions/float/fprecipestimate/FPRecipEstimate

```
// FPRecipEstimate()
// =================

func FPRecipEstimate{N}(operand : bits(N), fpcr_in : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var fpcr : FPCR_Type = fpcr_in;
    var result : bits(N);
    var overflow_to_inf : boolean;
    // When using alternative floating-point behavior, do not generate
    // floating-point exceptions, flush denormal input and output to zero,
    // and use RNE rounding mode.
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let fpexc : boolean = !altfp;
    if altfp then fpcr.[FIZ,FZ] = '11'; end;
    if altfp then fpcr.RMode    = '00'; end;

    let (fptype,sign,value) : (FPType, bit, real) = FPUnpack{N}(operand, fpcr, fpexc);

    let rounding : FPRounding = FPRoundingMode(fpcr);
    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        result = FPProcessNaN{N}(fptype, operand, fpcr, fpexc);
    elsif fptype == FPType_Infinity then
        result = FPZero{N}(sign);
    elsif fptype == FPType_Zero then
        result = FPInfinity{N}(sign);
        if fpexc then FPProcessException(FPExc_DivideByZero, fpcr); end;
    elsif (
            (N == 16 && Abs(value) < 2.0^-16) ||
            (N == 32 && Abs(value) < 2.0^-128) ||
            (N == 64 && Abs(value) < 2.0^-1024)
          ) then
        case rounding of
            when FPRounding_TIEEVEN =>
                overflow_to_inf = TRUE;
            when FPRounding_POSINF =>
                overflow_to_inf = (sign == '0');
            when FPRounding_NEGINF =>
                overflow_to_inf = (sign == '1');
            when FPRounding_ZERO =>
                overflow_to_inf = FALSE;
        end;
        result = if overflow_to_inf then FPInfinity{N}(sign) else FPMaxNormal{N}(sign);
        if fpexc then
            FPProcessException(FPExc_Overflow, fpcr);
            FPProcessException(FPExc_Inexact, fpcr);
        end;
    elsif ((fpcr.FZ == '1' && N != 16) || (fpcr.FZ16 == '1' && N == 16))
          && (
               (N == 16 && Abs(value) >= 2.0^14) ||
               (N == 32 && Abs(value) >= 2.0^126) ||
               (N == 64 && Abs(value) >= 2.0^1022)
             ) then
        // Result flushed to zero of correct sign
        result = FPZero{N}(sign);

        // Flush-to-zero never generates a trapped exception.
        if UsingAArch32() then
            FPSCR().UFC = '1';
        else
            if fpexc then FPSR().UFC = '1'; end;
        end;
    else
        // Scale to a fixed point value in the range 0.5 <= x < 1.0 in steps of 1/512, and
        // calculate result exponent. Scaled value has copied sign bit,
        // exponent = 1022 = double-precision biased version of -1,
        // fraction = original fraction
        var fraction : bits(52);
        var exp : integer;
        case N of
            when 16 =>
                fraction = operand[9:0] :: Zeros{42};
                exp = UInt(operand[14:10]);
            when 32 =>
                fraction = operand[22:0] :: Zeros{29};
                exp = UInt(operand[30:23]);
            when 64 =>
                fraction = operand[51:0];
                exp = UInt(operand[62:52]);
        end;

        if exp == 0 then
            if fraction[51] == '0' then
                exp = -1;
                fraction = fraction[49:0]::'00';
            else
                fraction = fraction[50:0]::'0';
            end;
        end;
```

```
        var scaled : integer;
        let increasedprecision : boolean = N==32 && IsFeatureImplemented(FEAT_RPRES) && altfp;

        if !increasedprecision then
            scaled = UInt('1'::fraction[51:44]);
        else
            scaled = UInt('1'::fraction[51:41]);
        end;

        var result_exp : integer;
        case N of
            when 16 => result_exp = 29 - exp;   // In range 29-30 = -1 to 29+1 = 30
            when 32 => result_exp = 253 - exp;  // In range 253-254 = -1 to 253+1 = 254
            when 64 => result_exp = 2045 - exp; // In range 2045-2046 = -1 to 2045+1 = 2046
        end;

        // Scaled is in range 256 .. 511 or 2048 .. 4095 range representing a
        // fixed-point number in range [0.5 .. 1.0].
        let estimate : integer = RecipEstimate(scaled, increasedprecision);

        // Estimate is in the range 256 .. 511 or 4096 .. 8191 representing a
        // fixed-point result in the range [1.0 .. 2.0].
        // Convert to scaled floating point result with copied sign bit,
        // high-order bits from estimate, and exponent calculated above.
        if !increasedprecision then
            fraction = estimate[7:0] :: Zeros{44};
        else
            fraction = estimate[11:0] :: Zeros{40};
        end;

        if result_exp == 0 then
            fraction = '1' :: fraction[51:1];
        elsif result_exp == -1 then
            fraction = '01' :: fraction[51:2];
            result_exp = 0;
        end;

        case N of
            when 16 => result = sign :: result_exp[N-12:0] :: fraction[51:42];
            when 32 => result = sign :: result_exp[N-25:0] :: fraction[51:29];
            when 64 => result = sign :: result_exp[N-54:0] :: fraction[51:0];
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fprecipestimate/RecipEstimate

```
// RecipEstimate()
// ===============
// Compute estimate of reciprocal of 9-bit fixed-point number.
//
// a is in range 256 .. 511 or 2048 .. 4096 representing a number in
// the range 0.5 <= x < 1.0.
// increasedprecision determines if the mantissa is 8-bit or 12-bit.
// result is in the range 256 .. 511 or 4096 .. 8191 representing a
// number in the range 1.0 to 511/256 or 1.00 to 8191/4096.

func RecipEstimate(a_in : integer, increasedprecision : boolean) => integer
begin
    var a : integer = a_in;
    var r : integer;
    if !increasedprecision then
        assert 256 <= a && a < 512;
        a = a*2+1;                        // Round to nearest
        let b : integer = (2 ^ 19) DIVRM a;
        r = (b+1) DIVRM 2;                // Round to nearest
        assert 256 <= r && r < 512;
    else
        assert 2048 <= a && a < 4096;
        a = a*2+1;                        // Round to nearest
        let b : integer = (2 ^ 26) DIVRM a;
        r = (b+1) DIVRM 2;                // Round to nearest
        assert 4096 <= r && r < 8192;
    end;
    return r;
end;
```

Library pseudocode for shared/functions/float/fprecpx/FPRecpX

```
// FPRecpX()
// =========

func FPRecpX{N}(op : bits(N), fpcr_in : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var fpcr : FPCR_Type = fpcr_in;
    let isbfloat16 : boolean = FALSE;
    let (F, -) = FPBits(N, isbfloat16);
    let E = (N - F) - 1;
    var result : bits(N);
    var exp : bits(E);
    var max_exp : bits(E);
    let frac : bits(F) = Zeros{};

    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && fpcr.AH == '1';
    let fpexc : boolean = !altfp;            // Generate no floating-point exceptions
    if altfp then fpcr.[FIZ,FZ] = '11'; end; // Flush denormal input and output to zero
    let (fptype,sign,value) = FPUnpack{N}(op, fpcr, fpexc);
    exp = op[F+:E];
    max_exp = Ones{E} - 1;

    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        result = FPProcessNaN{N}(fptype, op, fpcr, fpexc);
    else
        if IsZero(exp) then                  // Zero and denormals
            result = ZeroExtend{N}(sign::max_exp::frac);
        else                                 // Infinities and normals
            result = ZeroExtend{N}(sign::NOT(exp)::frac);
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpround/FPRound

```
// FPRound()
// =========
// Generic conversion from precise, unbounded real data type to IEEE format.

func FPRound{N}(op : real, fpcr : FPCR_Type) => bits(N)
begin
    return FPRound{N}(op, fpcr, FPRoundingMode(fpcr));
end;

// FPRound()
// =========
// For directed FP conversion, includes an explicit 'rounding' argument.

func FPRound{N}(op : real, fpcr_in : FPCR_Type, rounding : FPRounding) => bits(N)
begin
    let fpexc : boolean = TRUE;    // Generate floating-point exceptions
    return FPRound{N}(op, fpcr_in, rounding, fpexc);
end;

// FPRound()
// =========
// For AltFP, includes an explicit FPEXC argument to disable exception
// generation and switches off Arm alternate half-precision mode.

func FPRound{N}(op : real, fpcr_in : FPCR_Type, rounding : FPRounding, fpexc : boolean) => bits(N)
begin
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.AHP = '0';
    let isbfloat16 : boolean = FALSE;
    return FPRoundBase{N}(op, fpcr, rounding, isbfloat16, fpexc);
end;
```

Library pseudocode for shared/functions/float/fpround/FPRoundBase

```
// FPRoundBase()
// =============
// For BFloat16, includes an explicit 'isbfloat16' argument.

func FPRoundBase{N}(op : real, fpcr : FPCR_Type, rounding : FPRounding,
                    isbfloat16 : boolean) => bits(N)
begin
    let fpexc : boolean = TRUE;    // Generate floating-point exceptions
    return FPRoundBase{N}(op, fpcr, rounding, isbfloat16, fpexc);
end;

// FPRoundBase()
// =============
// For FP8 multiply-accumulate, dot product, and outer product instructions, includes
// an explicit saturation overflow argument.

func FPRoundBase{N}(op : real, fpcr : FPCR_Type, rounding : FPRounding, isbfloat16 : boolean,
                    fpexc : boolean) => bits(N)
begin
    let satoflo : boolean = FALSE;
    return FPRoundBase{N}(op, fpcr, rounding, isbfloat16, fpexc, satoflo);
end;

// FPRoundBase()
// =============
// Convert a real number 'op' into an N-bit floating-point value using the
// supplied rounding mode 'rounding'.
//
// The 'fpcr' argument supplies FPCR control bits and 'fpexc' controls the
// generation of floating-point exceptions. Status information is updated
// directly in the FPSR where appropriate. The 'satoflo' argument
// controls whether overflow generates Infinity or MaxNorm for 8-bit floating-point
// data processing instructions.

func FPRoundBase{N}(op : real, fpcr : FPCR_Type, rounding : FPRounding, isbfloat16 : boolean,
                    fpexc : boolean, satoflo : boolean) => bits(N)
begin

    assert N IN {16,32,64};
    assert op != 0.0;
    assert rounding != FPRounding_TIEAWAY;
    var result : bits(N);

    // Obtain format parameters - minimum exponent, numbers of exponent and fraction bits.
    let (F, minimum_exp) = FPBits(N, isbfloat16);
    let zeros = if N == 32 && isbfloat16 then 16 else 0;
    let E = N - (F + 1 + zeros);
    // Split value into sign, unrounded mantissa and exponent.
    var sign : bit;
    var exponent : integer;
    var mantissa : real;
    if op < 0.0 then
        sign = '1';  mantissa = -op;
    else
        sign = '0';  mantissa = op;
    end;
    (mantissa, exponent) = NormalizeReal(mantissa);

    // When TRUE, detection of underflow occurs after rounding and the test for a
    // denormalized number for single and double precision values occurs after rounding.
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';

    // Deal with flush-to-zero before rounding if FPCR.AH != '1'.
    if (!altfp && ((fpcr.FZ == '1' && N != 16) || (fpcr.FZ16 == '1' && N == 16)) &&
          exponent < minimum_exp) then
        // Flush-to-zero never generates a trapped exception.
        if UsingAArch32() then
            FPSCR().UFC = '1';
        else
            if fpexc then FPSR().UFC = '1'; end;
        end;
        return FPZero{N}(sign);
    end;

    var biased_exp_unconstrained : integer = (exponent - minimum_exp) + 1;
    var int_mant_unconstrained : integer = RoundDown(mantissa * 2.0^F);
    let error_unconstrained : real = mantissa * 2.0^F - Real(int_mant_unconstrained);

    // Start creating the exponent value for the result. Start by biasing the actual exponent
    // so that the minimum exponent becomes 1, lower values 0 (indicating possible underflow).
    var biased_exp : integer = Max((exponent - minimum_exp) + 1, 0);
    if biased_exp == 0 then mantissa = mantissa / 2.0^(minimum_exp - exponent); end;

    // Get the unrounded mantissa as an integer, and the "units in last place" rounding error.
    // < 2.0^F if biased_exp == 0, >= 2.0^F if not
    var int_mant : integer = RoundDown(mantissa * 2.0^F);
    var error : real = mantissa * 2.0^F - Real(int_mant);

    // Underflow occurs if exponent is too small before rounding, and result is inexact or
```

```
// the Underflow exception is trapped. This applies before rounding if FPCR.AH != '1'.
let trapped_UF : boolean = fpcr.UFE == '1' && (!InStreamingMode() || IsFullA64Enabled());
if !altfp && biased_exp == 0 && (error != 0.0 || trapped_UF) then
    if fpexc then FPProcessException(FPExc_Underflow, fpcr); end;
end;

// Round result according to rounding mode.
var round_up_unconstrained : boolean;
var round_up : boolean;
var overflow_to_inf : boolean;
if altfp then

    case rounding of
        when FPRounding_TIEEVEN =>
            round_up_unconstrained = (error_unconstrained > 0.5 ||
               (error_unconstrained == 0.5 && int_mant_unconstrained[0] == '1'));
            round_up = (error > 0.5 || (error == 0.5 && int_mant[0] == '1'));
            overflow_to_inf = !satoflo;
        when FPRounding_POSINF =>
            round_up_unconstrained = (error_unconstrained != 0.0 && sign == '0');
            round_up = (error != 0.0 && sign == '0');
            overflow_to_inf = (sign == '0');
        when FPRounding_NEGINF =>
            round_up_unconstrained = (error_unconstrained != 0.0 && sign == '1');
            round_up = (error != 0.0 && sign == '1');
            overflow_to_inf = (sign == '1');
        when FPRounding_ZERO, FPRounding_ODD =>
            round_up_unconstrained = FALSE;
            round_up = FALSE;
            overflow_to_inf = FALSE;
    end;

    if round_up_unconstrained then
        int_mant_unconstrained = int_mant_unconstrained + 1;
        if int_mant_unconstrained == 2^(F+1) then    // Rounded up to next exponent
            biased_exp_unconstrained = biased_exp_unconstrained + 1;
            int_mant_unconstrained   = int_mant_unconstrained DIV 2;
        end;
    end;

    // Deal with flush-to-zero and underflow after rounding if FPCR.AH == '1'.
    if biased_exp_unconstrained < 1 && int_mant_unconstrained != 0 then
        // the result of unconstrained rounding is less than the minimum normalized number
        if (fpcr.FZ == '1' && N != 16) || (fpcr.FZ16 == '1' && N == 16) then   // Flush-to-zero
            if fpexc then
                FPSR().UFC = '1';
                FPProcessException(FPExc_Inexact, fpcr);
            end;
            return FPZero{N}(sign);
        elsif error != 0.0 || trapped_UF then
            if fpexc then FPProcessException(FPExc_Underflow, fpcr); end;
        end;
    end;
else    // altfp == FALSE
    case rounding of
        when FPRounding_TIEEVEN =>
            round_up = (error > 0.5 || (error == 0.5 && int_mant[0] == '1'));
            overflow_to_inf = !satoflo;
        when FPRounding_POSINF =>
            round_up = (error != 0.0 && sign == '0');
            overflow_to_inf = (sign == '0');
        when FPRounding_NEGINF =>
            round_up = (error != 0.0 && sign == '1');
            overflow_to_inf = (sign == '1');
        when FPRounding_ZERO, FPRounding_ODD =>
            round_up = FALSE;
            overflow_to_inf = FALSE;
    end;
end;

if round_up then
    int_mant = int_mant + 1;
    if int_mant == 2^F then      // Rounded up from denormalized to normalized
        biased_exp = 1;
    end;
    if int_mant == 2^(F+1) then  // Rounded up to next exponent
        biased_exp = biased_exp + 1;
        int_mant = int_mant DIV 2;
    end;
end;

// Handle rounding to odd
if error != 0.0 && rounding == FPRounding_ODD && int_mant[0] == '0' then
    int_mant = int_mant + 1;
end;

// Deal with overflow and generate result.
if N != 16 || fpcr.AHP == '0' then  // Single, double or IEEE half precision
    if biased_exp >= 2^E - 1 then
```

```
            result = if overflow_to_inf then FPInfinity{N}(sign) else FPMaxNormal{N}(sign);
            if fpexc then FPProcessException(FPExc_Overflow, fpcr); end;
            error = 1.0;  // Ensure that an Inexact exception occurs
        else
            result = sign :: biased_exp[E-1:0] :: int_mant[F-1:0] :: Zeros{N-(E+F+1)};
        end;
    else                                    // Alternative half precision
        if biased_exp >= 2^E then
            result = sign :: Ones{N-1};
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
            error = 0.0;  // Ensure that an Inexact exception does not occur
        else
            result = sign :: biased_exp[E-1:0] :: int_mant[F-1:0] :: Zeros{N-(E+F+1)};
        end;
    end;

    // Deal with Inexact exception.
    if error != 0.0 then
        if fpexc then FPProcessException(FPExc_Inexact, fpcr); end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fpround/FPRoundCV

```
// FPRoundCV()
// ===========
// Used for FP to FP conversion instructions.
// For half-precision data ignores FZ16 and observes AHP.

func FPRoundCV{N}(op : real, fpcr_in : FPCR_Type, rounding : FPRounding) => bits(N)
begin
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.FZ16 = '0';
    let fpexc : boolean = TRUE;    // Generate floating-point exceptions
    let isbfloat16 : boolean = FALSE;
    return FPRoundBase{N}(op, fpcr, rounding, isbfloat16, fpexc);
end;
```

Library pseudocode for shared/functions/float/fpround/FPRound_FP8

```
// FPRound_FP8()
// =============
// Used by FP8 multiply-accumulate, dot product, and outer product instructions
// which observe FPMR.OSM.

func FPRound_FP8{N}(op : real, fpcr_in : FPCR_Type, rounding : FPRounding,
                    satoflo : boolean) => bits(N)
begin
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.AHP = '0';
    let fpexc : boolean = FALSE;
    let isbfloat16 : boolean = FALSE;
    return FPRoundBase{N}(op, fpcr, rounding, isbfloat16, fpexc, satoflo);
end;
```

Library pseudocode for shared/functions/float/fprounding/FPRounding

```
// FPRounding
// ==========
// The conversion and rounding functions take an explicit
// rounding mode enumeration instead of booleans or FPCR values.

type FPRounding of enumeration {FPRounding_TIEEVEN, FPRounding_POSINF,
                                FPRounding_NEGINF,  FPRounding_ZERO,
                                FPRounding_TIEAWAY, FPRounding_ODD};
```

Library pseudocode for shared/functions/float/fproundingmode/FPRoundingMode

```
// FPRoundingMode()
// ================
// Return the current floating-point rounding mode.

func FPRoundingMode(fpcr : FPCR_Type) => FPRounding
begin
    return FPDecodeRounding(fpcr.RMode);
end;
```

Library pseudocode for shared/functions/float/fproundint/FPRoundInt

```
// FPRoundInt()
// ============

// Round op to nearest integral floating point value using rounding mode in FPCR/FPSCR.
// If EXACT is TRUE, set FPSR.IXC if result is not numerically equal to op.

func FPRoundInt{N}(op : bits(N), fpcr : FPCR_Type, rounding : FPRounding,
                   exact : boolean) => bits(N)
begin
    assert rounding != FPRounding_ODD;
    assert N IN {16,32,64};

    // When alternative floating-point support is TRUE, do not generate
    // Input Denormal floating-point exceptions.
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let fpexc : boolean = !altfp;

    // Unpack using FPCR to determine if subnormals are flushed-to-zero.
    let (fptype,sign,value) = FPUnpack{N}(op, fpcr, fpexc);

    var result : bits(N);
    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        result = FPProcessNaN{N}(fptype, op, fpcr);
    elsif fptype == FPType_Infinity then
        result = FPInfinity{N}(sign);
    elsif fptype == FPType_Zero then
        result = FPZero{N}(sign);
    else
        // Extract integer component.
        var int_result : integer = RoundDown(value);
        let error : real = value - Real(int_result);

        // Determine whether supplied rounding mode requires an increment.
        var round_up : boolean;
        case rounding of
            when FPRounding_TIEEVEN =>
                round_up = (error > 0.5 || (error == 0.5 && int_result[0] == '1'));
            when FPRounding_POSINF =>
                round_up = (error != 0.0);
            when FPRounding_NEGINF =>
                round_up = FALSE;
            when FPRounding_ZERO =>
                round_up = (error != 0.0 && int_result < 0);
            when FPRounding_TIEAWAY =>
                round_up = (error > 0.5 || (error == 0.5 && int_result >= 0));
        end;

        if round_up then int_result = int_result + 1; end;

        // Convert integer value into an equivalent real value.
        let real_result : real = Real(int_result);

        // Re-encode as a floating-point value, result is always exact.
        if real_result == 0.0 then
            result = FPZero{N}(sign);
        else
            result = FPRound{N}(real_result, fpcr, FPRounding_ZERO);
        end;

        // Generate inexact exceptions.
        if error != 0.0 && exact then
            FPProcessException(FPExc_Inexact, fpcr);
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fproundintn/FPRoundIntN

```
// FPRoundIntN()
// =============

func FPRoundIntN{N}(op : bits(N), fpcr : FPCR_Type,
                    rounding : FPRounding, intsize : integer) => bits(N)
begin
    assert rounding != FPRounding_ODD;
    assert N IN {32,64};
    assert intsize IN {32, 64};
    var exp : integer;
    var result : bits(N);
    var round_up : boolean;
    let E : integer{} = (if N == 32 then 8 else 11);
    let F : integer{} = N - (E + 1);

    // When alternative floating-point support is TRUE, do not generate
    // Input Denormal floating-point exceptions.
    let altfp = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let fpexc = !altfp;

    // Unpack using FPCR to determine if subnormals are flushed-to-zero.
    let (fptype,sign,value) = FPUnpack{N}(op, fpcr, fpexc);

    if fptype IN {FPType_SNaN, FPType_QNaN, FPType_Infinity} then
        if N == 32 then
            exp    = 126 + intsize;
            result = '1'::exp[(E-1):0]::Zeros{F};
        else
            exp    = 1022+intsize;
            result = '1'::exp[(E-1):0]::Zeros{F};
        end;
        FPProcessException(FPExc_InvalidOp, fpcr);
    elsif fptype == FPType_Zero then
        result = FPZero{N}(sign);
    else
        // Extract integer component.
        var int_result = RoundDown(value);
        var error      = value - Real(int_result);

        // Determine whether supplied rounding mode requires an increment.
        case rounding of
            when FPRounding_TIEEVEN =>
                round_up = error > 0.5 || (error == 0.5 && int_result[0] == '1');
            when FPRounding_POSINF =>
                round_up = error != 0.0;
            when FPRounding_NEGINF =>
                round_up = FALSE;
            when FPRounding_ZERO =>
                round_up = error != 0.0 && int_result < 0;
            when FPRounding_TIEAWAY =>
                round_up = error > 0.5 || (error == 0.5 && int_result >= 0);
        end;

        if round_up then int_result = int_result + 1; end;
        let overflow = int_result > 2^(intsize-1)-1 || int_result < -1*2^(intsize-1);

        if overflow then
            if N == 32 then
                exp    = 126 + intsize;
                result = '1'::exp[(E-1):0]::Zeros{F};
            else
                exp    = 1022 + intsize;
                result = '1'::exp[(E-1):0]::Zeros{F};
            end;
            FPProcessException(FPExc_InvalidOp, fpcr);
            // This case shouldn't set Inexact.
            error = 0.0;

        else
            // Convert integer value into an equivalent real value.
            let real_result : real = Real(int_result);

            // Re-encode as a floating-point value, result is always exact.
            if real_result == 0.0 then
                result = FPZero{N}(sign);
            else
                result = FPRound{N}(real_result, fpcr, FPRounding_ZERO);
            end;
        end;

        // Generate inexact exceptions.
        if error != 0.0 then
            FPProcessException(FPExc_Inexact, fpcr);
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fprsqrtestimate/FPRSqrtEstimate

```
// FPRSqrtEstimate()
// =================

func FPRSqrtEstimate{N}(operand : bits(N), fpcr_in : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var fpcr : FPCR_Type = fpcr_in;

    // When using alternative floating-point behavior, do not generate
    // floating-point exceptions and flush denormal input to zero.
    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let fpexc : boolean = !altfp;
    if altfp then fpcr.[FIZ,FZ] = '11'; end;

    let (fptype,sign,value) : (FPType, bit, real) = FPUnpack{N}(operand, fpcr, fpexc);

    var result : bits(N);
    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        result = FPProcessNaN{N}(fptype, operand, fpcr, fpexc);
    elsif fptype == FPType_Zero then
        result = FPInfinity{N}(sign);
        if fpexc then FPProcessException(FPExc_DivideByZero, fpcr); end;
    elsif sign == '1' then
        result = FPDefaultNaN{N}(fpcr);
        if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
    elsif fptype == FPType_Infinity then
        result = FPZero{N}('0');
    else
        // Scale to a fixed-point value in the range 0.25 <= x < 1.0 in steps of 512, with the
        // evenness or oddness of the exponent unchanged, and calculate result exponent.
        // Scaled value has copied sign bit, exponent = 1022 or 1021 = double-precision
        // biased version of -1 or -2, fraction = original fraction extended with zeros.

        var fraction : bits(52);
        var exp : integer;
        case N of
            when 16 =>
                fraction = operand[9:0] :: Zeros{42};
                exp      = UInt(operand[14:10]);
            when 32 =>
                fraction = operand[22:0] :: Zeros{29};
                exp      = UInt(operand[30:23]);
            when 64 =>
                fraction = operand[51:0];
                exp      = UInt(operand[62:52]);
        end;

        if exp == 0 then
            while fraction[51] == '0' looplimit 51 do
                fraction = fraction[50:0] :: '0';
                exp      = exp - 1;
            end;
            fraction = fraction[50:0] :: '0';
        end;

        var scaled : integer;
        let increasedprecision : boolean = N==32 && IsFeatureImplemented(FEAT_RPRES) && altfp;

        if !increasedprecision then
            if exp[0] == '0' then
                scaled = UInt('1'::fraction[51:44]);
            else
                scaled = UInt('01'::fraction[51:45]);
            end;
        else
            if exp[0] == '0' then
                scaled = UInt('1'::fraction[51:41]);
            else
                scaled = UInt('01'::fraction[51:42]);
            end;
        end;

        var result_exp : integer;
        case N of
            when 16 => result_exp = (  44 - exp) DIVRM 2;
            when 32 => result_exp = ( 380 - exp) DIVRM 2;
            when 64 => result_exp = (3068 - exp) DIVRM 2;
        end;

        let estimate : integer = RecipSqrtEstimate(scaled, increasedprecision);

        // Estimate is in the range 256 .. 511 or 4096 .. 8191 representing a
        // fixed-point result in the range [1.0 .. 2.0].
        // Convert to scaled floating point result with copied sign bit and high-order
        // fraction bits, and exponent calculated above.
        case N of
            when 16 => result = '0' :: result_exp[N-12:0] :: estimate[7:0]::Zeros{2};
            when 32 =>
                if !increasedprecision then
```

```
                    result = '0' :: result_exp[N-25:0] :: estimate[7:0]::Zeros{15};
                else
                    result = '0' :: result_exp[N-25:0] :: estimate[11:0]::Zeros{11};
                end;
            when 64 => result = '0' :: result_exp[N-54:0] :: estimate[7:0]::Zeros{44};
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fprsqrtestimate/RecipSqrtEstimate

```
// RecipSqrtEstimate()
// ===================
// Compute estimate of reciprocal square root of 9-bit fixed-point number.
//
// a_in is in range 128 .. 511 or 1024 .. 4095, with increased precision,
// representing a number in the range 0.25 <= x < 1.0.
// increasedprecision determines if the mantissa is 8-bit or 12-bit.
// result is in the range 256 .. 511 or 4096 .. 8191, with increased precision,
// representing a number in the range 1.0 to 511/256 or 8191/4096.

func RecipSqrtEstimate(a_in : integer, increasedprecision : boolean) => integer
begin
    var a : integer = a_in;
    var r : integer;
    if !increasedprecision then
        assert 128 <= a && a < 512;
        if a < 256 then                     // a in [128, 255], represents a value in [0.25, 0.5)
            a = a*2+1;                      // promote to 9-bit range in units of 1/512
        else                                // a in [256, 511], represents a value in [0.5, 1.0)
            a = (a >> 1) << 1;              // Discard bottom bit
            a = (a+1)*2;                    // round up to nearest 1/256 and convert
        end;
        var b : integer = 512;
        while a*(b+1)*(b+1) < 2^28 looplimit 510 do
            b = b+1;
        end;
        // b = largest b such that b < 2^14 / sqrt(a)
        r = (b+1) DIVRM 2;                  // Round to nearest
        assert 256 <= r && r < 512;
    else
        assert 1024 <= a && a < 4096;
        if a < 2048 then                    // a in [1024, 2047], represents a value in [0.25, 0.5)
            a = a*2 + 1;                    // promote to 13-bit range in units of 1/8192
        else                                // a in [2048, 4095], represents a value in [0.5, 1.0)
            a = (a >> 1) << 1;              // Discard bottom bit
            a = (a+1)*2;                    // round up to nearest 1/4096 and convert
        end;
        var b : integer = 8192;
        while a*(b+1)*(b+1) < 2^39 looplimit (2^39 - (2049 * 8193^2)) do
            b = b+1;
        end;
        r = (b+1) DIVRM 2;                  // Round to nearest
        assert 4096 <= r && r < 8192;
    end;
    return r;
end;
```

Library pseudocode for shared/functions/float/fpsqrt/FPSqrt

```
// FPSqrt()
// ========

func FPSqrt{N}(op : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    assert N IN {16,32,64};
    var (fptype,sign,value) : (FPType, bit, real) = FPUnpack{N}(op, fpcr);

    var result : bits(N);
    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        result = FPProcessNaN{N}(fptype, op, fpcr);
    elsif fptype == FPType_Zero then
        result = FPZero{N}(sign);
    elsif fptype == FPType_Infinity && sign == '0' then
        result = FPInfinity{N}(sign);
    elsif sign == '1' then
        result = FPDefaultNaN{N}(fpcr);
        FPProcessException(FPExc_InvalidOp, fpcr);
    else
        var prec : integer;
        if N == 16 then
            prec = 13; // 10 fraction bits + 3
        elsif N == 32 then
            prec = 26; // 23 fraction bits + 3
        else // N == 64
            prec = 55; // 52 fraction bits + 3
        end;
        value = SqrtRounded(value, prec);
        result = FPRound{N}(value, fpcr);
        FPProcessDenorm(fptype, N, fpcr);

    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpsub/FPSub

```
// FPSub()
// =======

func FPSub{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type) => bits(N)
begin
    let fpexc : boolean = TRUE;       // Generate floating-point exceptions
    return FPSub{N}(op1, op2, fpcr, fpexc);
end;

// FPSub()
// =======

func FPSub{N}(op1 : bits(N), op2 : bits(N), fpcr : FPCR_Type, fpexc : boolean) => bits(N)
begin

    assert N IN {16,32,64};
    let rounding : FPRounding = FPRoundingMode(fpcr);

    let (type1,sign1,value1) : (FPType, bit, real) = FPUnpack{N}(op1, fpcr, fpexc);
    let (type2,sign2,value2) : (FPType, bit, real) = FPUnpack{N}(op2, fpcr, fpexc);

    var (done,result) : (boolean, bits(N)) = FPProcessNaNs{N}(type1, type2, op1, op2, fpcr, fpexc);
    if !done then
        let inf1 : boolean = (type1 == FPType_Infinity);
        let inf2 : boolean = (type2 == FPType_Infinity);
        let zero1 : boolean = (type1 == FPType_Zero);
        let zero2 : boolean = (type2 == FPType_Zero);

        if inf1 && inf2 && sign1 == sign2 then
            result = FPDefaultNaN{N}(fpcr);
            if fpexc then FPProcessException(FPExc_InvalidOp, fpcr); end;
        elsif (inf1 && sign1 == '0') || (inf2 && sign2 == '1') then
            result = FPInfinity{N}('0');
        elsif (inf1 && sign1 == '1') || (inf2 && sign2 == '0') then
            result = FPInfinity{N}('1');
        elsif zero1 && zero2 && sign1 == NOT(sign2) then
            result = FPZero{N}(sign1);
        else
            let result_value : real = value1 - value2;
            if result_value == 0.0 then  // Sign of exact zero result depends on rounding mode
                let result_sign : bit = if rounding == FPRounding_NEGINF then '1' else '0';
                result = FPZero{N}(result_sign);
            else
                result = FPRound{N}(result_value, fpcr, rounding, fpexc);
            end;
        end;

        if fpexc then FPProcessDenorms(type1, type2, N, fpcr); end;
    end;
    return result;
end;
```

Library pseudocode for shared/functions/float/fpsub/FPSub_ZA

```
// FPSub_ZA()
// ==========
// Calculates op1-op2 for SME2 ZA-targeting instructions.

func FPSub_ZA{N}(op1 : bits(N), op2 : bits(N), fpcr_in : FPCR_Type) => bits(N)
begin
    var fpcr : FPCR_Type = fpcr_in;
    let fpexc : boolean = FALSE; // Do not generate floating-point exceptions
    fpcr.DN = '1';               // Generate default NaN values
    return FPSub{N}(op1, op2, fpcr, fpexc);
end;
```

Library pseudocode for shared/functions/float/fpthree/FPThree

```
// FPThree()
// =========

func FPThree{N}(sign : bit) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{} = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{}  = N - (E + 1);
    let exp : bits(E)  = '1'::Zeros{E-1};
    let frac : bits(F) = '1'::Zeros{F-1};
    let result : bits((F + E) + 1) = sign :: exp :: frac;

    return result;
end;
```

Library pseudocode for shared/functions/float/fptofixed/FPToFixed

```
// FPToFixed()
// ===========

// Convert N-bit precision floating point 'op' to M-bit fixed point with
// FBITS fractional bits, controlled by UNSIGNED and ROUNDING.

func FPToFixed{M, N}(op : bits(N), fbits : integer, unsigned : boolean, fpcr : FPCR_Type,
                     rounding : FPRounding) => bits(M)
begin
    assert N IN {16,32,64};
    assert M IN {8,16,32,64};
    assert fbits >= 0;
    assert rounding != FPRounding_ODD;

    // When alternative floating-point support is TRUE, do not generate
    // Input Denormal floating-point exceptions.
    let altfp = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32() && fpcr.AH == '1';
    let fpexc = !altfp;

    // Unpack using fpcr to determine if subnormals are flushed-to-zero.
    var (fptype,sign,value) : (FPType, bit, real) = FPUnpack{N}(op, fpcr, fpexc);

    // If NaN, set cumulative flag or take exception.
    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        FPProcessException(FPExc_InvalidOp, fpcr);
    end;

    // Scale by fractional bits and produce integer rounded towards minus-infinity.
    value = value * 2.0^fbits;
    var int_result : integer = RoundDown(value);
    let error : real         = value - Real(int_result);

    // Determine whether supplied rounding mode requires an increment.
    var round_up : boolean;
    case rounding of
        when FPRounding_TIEEVEN =>
            round_up = (error > 0.5 || (error == 0.5 && int_result[0] == '1'));
        when FPRounding_POSINF =>
            round_up = (error != 0.0);
        when FPRounding_NEGINF =>
            round_up = FALSE;
        when FPRounding_ZERO =>
            round_up = (error != 0.0 && int_result < 0);
        when FPRounding_TIEAWAY =>
            round_up = (error > 0.5 || (error == 0.5 && int_result >= 0));
    end;

    if round_up then int_result = int_result + 1; end;

    // Generate saturated result and exceptions.
    let (result, overflow) : (bits(M), boolean) = SatQ{M}(int_result, unsigned);
    if overflow then
        FPProcessException(FPExc_InvalidOp, fpcr);
    elsif error != 0.0 then
        FPProcessException(FPExc_Inexact, fpcr);
    end;

    return result;
end;
```

Library pseudocode for shared/functions/float/fptofixedjs/FPToFixedJS

```
// FPToFixedJS()
// =============

// Converts a double precision floating point input value
// to a signed integer, with rounding to zero.

func FPToFixedJS(op : bits(64), fpcr : FPCR_Type) => (bits(32), bit)
begin

    // If FALSE, never generate Input Denormal floating-point exceptions.
    let fpexc_idenorm : boolean = !(IsFeatureImplemented(FEAT_AFP) &&
                                    !UsingAArch32() && fpcr.AH == '1');

    // Unpack using fpcr to determine if subnormals are flushed-to-zero.
    let (fptype,sign,value) = FPUnpack{64}(op, fpcr, fpexc_idenorm);

    var z : bit = '1';
    // If NaN, set cumulative flag or take exception.
    if fptype == FPType_SNaN || fptype == FPType_QNaN then
        FPProcessException(FPExc_InvalidOp, fpcr);
        z = '0';
    end;

    var int_result = RoundDown(value);
    let error = value - Real(int_result);

    // Determine whether supplied rounding mode requires an increment.

    let round_it_up = (error != 0.0 && int_result < 0);
    if round_it_up then int_result = int_result + 1; end;

    var result : integer;
    if int_result < 0 then
        result = int_result - 2^32*RoundUp(Real(int_result)/Real(2^32));
    else
        result = int_result - 2^32*RoundDown(Real(int_result)/Real(2^32));
    end;

    // Generate exceptions.
    if int_result < -(2^31) || int_result > (2^31)-1 then
        FPProcessException(FPExc_InvalidOp, fpcr);
        z = '0';
    elsif error != 0.0 then
        FPProcessException(FPExc_Inexact, fpcr);
        z = '0';
    elsif sign == '1' && value == 0.0 then
        z = '0';
    elsif sign == '0' && value == 0.0 && !IsZero(op[51:0]) then
        z = '0';
    end;

    if fptype == FPType_Infinity then result = 0; end;

    return (result[31:0], z);
end;
```

Library pseudocode for shared/functions/float/fptwo/FPTwo

```
// FPTwo()
// =======

func FPTwo{N}(sign : bit) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{} = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{}  = N - (E + 1);
    let exp : bits(E)  = '1'::Zeros{E-1};
    let frac : bits(F) = Zeros{};
    let result : bits((F + E) + 1) = sign :: exp :: frac;
    return result;
end;
```

Library pseudocode for shared/functions/float/fptype/FPType

```
// FPType
// ======

type FPType of enumeration {FPType_Zero,
                            FPType_Denormal,
                            FPType_Nonzero,
                            FPType_Infinity,
                            FPType_QNaN,
                            FPType_SNaN};
```

Library pseudocode for shared/functions/float/fpunpack/FPUnpack

```
// FPUnpack()
// ==========

func FPUnpack{N}(fpval : bits(N), fpcr_in : FPCR_Type) => (FPType, bit, real)
begin
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.AHP = '0';
    let fpexc : boolean = TRUE;   // Generate floating-point exceptions
    let (fp_type, sign, value) : (FPType, bit, real) = FPUnpackBase{N}(fpval, fpcr, fpexc);
    return (fp_type, sign, value);
end;

// FPUnpack()
// ==========
//
// Used by data processing, int/fixed to FP and FP to int/fixed conversion instructions.
// For half-precision data it ignores AHP, and observes FZ16.

func FPUnpack{N}(fpval : bits(N), fpcr_in : FPCR_Type, fpexc : boolean) => (FPType, bit, real)
begin
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.AHP = '0';
    let (fp_type, sign, value) : (FPType, bit, real) = FPUnpackBase{N}(fpval, fpcr, fpexc);
    return (fp_type, sign, value);
end;
```

Library pseudocode for shared/functions/float/fpunpack/FPUnpackBase

```
// FPUnpackBase()
// ==============

func FPUnpackBase{N}(fpval : bits(N), fpcr : FPCR_Type, fpexc : boolean) => (FPType, bit, real)
begin
    let isbfloat16 : boolean = FALSE;
    let (fp_type, sign, value) = FPUnpackBase{N}(fpval, fpcr, fpexc, isbfloat16);
    return (fp_type, sign, value);
end;

// FPUnpackBase()
// ==============
//
// Unpack a floating-point number into its type, sign bit and the real number
// that it represents. The real number result has the correct sign for numbers
// and infinities, is very large in magnitude for infinities, and is 0.0 for
// NaNs. (These values are chosen to simplify the description of comparisons
// and conversions.)
//
// The 'fpcr_in' argument supplies FPCR control bits, 'fpexc' controls the
// generation of floating-point exceptions and 'isbfloat16' determines whether
// N=16 signifies BFloat16 or half-precision type. Status information is updated
// directly in the FPSR where appropriate.

func FPUnpackBase{N}(fpval : bits(N), fpcr_in : FPCR_Type, fpexc : boolean,
                     isbfloat16 : boolean) => (FPType, bit, real)
begin
    assert N IN {16,32,64};

    let fpcr : FPCR_Type = fpcr_in;

    let altfp : boolean = IsFeatureImplemented(FEAT_AFP) && !UsingAArch32();
    let fiz : boolean   = altfp && fpcr.FIZ == '1';
    let fz : boolean    = fpcr.FZ == '1' && !(altfp && fpcr.AH == '1');
    var value : real;
    var sign : bit;
    var fptype : FPType;

    if N == 16 && !isbfloat16 then
        sign   = fpval[15];
        let exp16 : bits(5)   = fpval[14:10];
        let frac16 : bits(10) = fpval[9:0];
        if IsZero(exp16) then
            if IsZero(frac16) || fpcr.FZ16 == '1' then
                fptype = FPType_Zero;  value = 0.0;
            else
                fptype = FPType_Denormal;  value = 2.0^-14 * (Real(UInt(frac16)) * 2.0^-10);
            end;
        elsif IsOnes(exp16) && fpcr.AHP == '0' then  // Infinity or NaN in IEEE format
            if IsZero(frac16) then
                fptype = FPType_Infinity;  value = 2.0^1000000;
            else
                fptype = if frac16[9] == '1' then FPType_QNaN else FPType_SNaN;
                value = 0.0;
            end;
        else
            fptype = FPType_Nonzero;
            value = 2.0^(UInt(exp16)-15) * (1.0 + Real(UInt(frac16)) * 2.0^-10);
        end;

    elsif N == 32 || isbfloat16 then
        var exp32 : bits(8);
        var frac32 : bits(23);
        if isbfloat16 then
            sign   = fpval[15];
            exp32  = fpval[14:7];
            frac32 = fpval[6:0] :: Zeros{16};
        else
            sign   = fpval[31];
            exp32  = fpval[30:23];
            frac32 = fpval[22:0];
        end;

        if IsZero(exp32) then
            if IsZero(frac32) then
                // Produce zero if value is zero.
                fptype = FPType_Zero;  value = 0.0;
            elsif fz || fiz then        // Flush-to-zero if FIZ==1 or AH,FZ==01
                fptype = FPType_Zero;  value = 0.0;
                // Check whether to generate Input Denormal floating-point exception.
                // fpcr.FIZ==1 does not generate Input Denormal exception.
                if fz then
                    // Denormalized input flushed to zero
                    if fpexc then FPProcessException(FPExc_InputDenorm, fpcr); end;
                end;
            else
                fptype = FPType_Denormal;  value = 2.0^-126 * (Real(UInt(frac32)) * 2.0^-23);
            end;
        elsif IsOnes(exp32) then
```

```
            if IsZero(frac32) then
                fptype = FPType_Infinity;  value = 2.0^1000000;
            else
                fptype = if frac32[22] == '1' then FPType_QNaN else FPType_SNaN;
                value = 0.0;
            end;
        else
            fptype = FPType_Nonzero;
            value = 2.0^(UInt(exp32)-127) * (1.0 + Real(UInt(frac32)) * 2.0^-23);
        end;

    else // N == 64
        sign   = fpval[63];
        let exp64 : bits(11)  = fpval[62:52];
        let frac64 : bits(52) = fpval[51:0];

        if IsZero(exp64) then
            if IsZero(frac64) then
                // Produce zero if value is zero.
                fptype = FPType_Zero;  value = 0.0;
            elsif fz || fiz then         // Flush-to-zero if FIZ==1 or AH,FZ==01
                fptype = FPType_Zero;  value = 0.0;
                // Check whether to generate Input Denormal floating-point exception.
                // fpcr.FIZ==1 does not generate Input Denormal exception.
                if fz then
                    // Denormalized input flushed to zero
                    if fpexc then FPProcessException(FPExc_InputDenorm, fpcr); end;
                end;
            else
                fptype = FPType_Denormal;  value = 2.0^-1022 * (Real(UInt(frac64)) * 2.0^-52);
            end;
        elsif IsOnes(exp64) then
            if IsZero(frac64) then
                fptype = FPType_Infinity;  value = 2.0^1000000;
            else
                fptype = if frac64[51] == '1' then FPType_QNaN else FPType_SNaN;
                value  = 0.0;
            end;
        else
            fptype = FPType_Nonzero;
            value  = 2.0^(UInt(exp64)-1023) * (1.0 + Real(UInt(frac64)) * 2.0^-52);
        end;
    end;

    if sign == '1' then value = -value; end;

    return (fptype, sign, value);
end;
```

Library pseudocode for shared/functions/float/fpunpack/FPUnpackCV

```
// FPUnpackCV()
// ============
//
// Used for FP to FP conversion instructions.
// For half-precision data ignores FZ16 and observes AHP.

func FPUnpackCV{N}(fpval : bits(N), fpcr_in : FPCR_Type) => (FPType, bit, real)
begin
    var fpcr : FPCR_Type = fpcr_in;
    fpcr.FZ16 = '0';
    let fpexc : boolean = TRUE;   // Generate floating-point exceptions
    let (fp_type, sign, value) : (FPType, bit, real) = FPUnpackBase{N}(fpval, fpcr, fpexc);
    return (fp_type, sign, value);
end;
```

Library pseudocode for shared/functions/float/fpzero/FPZero

```
// FPZero()
// ========

func FPZero{N}(sign : bit) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{}  = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{}  = N - (E + 1);
    let exp : bits(E)  = Zeros{};
    let frac : bits(F) = Zeros{};
    let result : bits((F + E) + 1) = sign :: exp :: frac;
    return result;
end;
```

Library pseudocode for shared/functions/float/vfpexpandimm/VFPExpandImm

```
// VFPExpandImm()
// ==============

func VFPExpandImm{N}(imm8 : bits(8)) => bits(N)
begin
    assert N IN {16,32,64};
    let E : integer{} = (if N == 16 then 5 else (if N == 32 then 8 else 11));
    let F : integer{} = (N - E) - 1;
    let sign : bit = imm8[7];
    let exp : bits(E)  = NOT(imm8[6])::Replicate{E - 3}(imm8[6])::imm8[5:4];
    let frac : bits(F) = imm8[3:0]::Zeros{F-4};
    let result : bits((F + E) + 1) = sign :: exp :: frac;

    return result;
end;
```

Library pseudocode for shared/functions/impdef/ImpDef

```
// ImpDef()
// ========
// IMPLEMENTATION DEFINED behavior identified by the string parameter.

impdef func ImpDef(x : string)
begin
    return;
end;
```

Library pseudocode for shared/functions/impdef/ImpDefBit

```
// ImpDefBit()
// ===========
// Return an IMPLEMENTATION DEFINED bit value identified by the string parameter.

readonly impdef func ImpDefBit(x : string) => bit
begin
    return ARBITRARY : bit;
end;
```

Library pseudocode for shared/functions/impdef/ImpDefBits

```
// ImpDefBits()
// ============
// Return an IMPLEMENTATION DEFINED value of bits identified by the string parameter.

readonly impdef func ImpDefBits{N}(x : string) => bits(N)
begin
    return ARBITRARY : bits(N);
end;
```

Library pseudocode for shared/functions/impdef/ImpDefBool

```
// ImpDefBool()
// ============
// Return an IMPLEMENTATION DEFINED boolean value identified by the string parameter.

readonly impdef func ImpDefBool(x : string) => boolean
begin
    return ARBITRARY : boolean;
end;
```

Library pseudocode for shared/functions/impdef/ImpDefInt

```
// ImpDefInt()
// ===========
// Return an IMPLEMENTATION DEFINED integer value identified by the string parameter.

readonly impdef func ImpDefInt(x : string) => integer
begin
    return ARBITRARY : integer;
end;

type UnimplementedException of exception{-};
```

Library pseudocode for shared/functions/impdef/Unimplemented

```
// Unimplemented()
// ===============
// Behavior that is not implemented by this pseudocode.

noreturn impdef func Unimplemented()
begin
    throw UnimplementedException{-};
end;
```

Library pseudocode for shared/functions/integer/AddWithCarry

```
// AddWithCarry()
// ==============
// Integer addition with carry input, returning result and NZCV flags

func AddWithCarry{N}(x : bits(N), y : bits(N), carry_in : bit) => (bits(N), bits(4))
begin
    let unsigned_sum : integer = UInt(x) + UInt(y) + UInt(carry_in);
    let signed_sum : integer = SInt(x) + SInt(y) + UInt(carry_in);
    let result : bits(N) = unsigned_sum[N-1:0]; // same value as signed_sum[N-1:0]
    let n : bit = result[N-1];
    let z : bit = if IsZero(result) then '1' else '0';
    let c : bit = if UInt(result) == unsigned_sum then '0' else '1';
    let v : bit = if SInt(result) == signed_sum then '0' else '1';
    return (result, n::z::c::v);
end;
```

Library pseudocode for shared/functions/interrupts/InterruptID

```
// InterruptID
// ===========

type InterruptID of enumeration {
    InterruptID_PMUIRQ,
    InterruptID_COMMIRQ,
    InterruptID_CTIIRQ,
    InterruptID_COMMRX,
    InterruptID_COMMTX,
    InterruptID_CNTP,
    InterruptID_CNTHP,
    InterruptID_CNTHPS,
    InterruptID_CNTPS,
    InterruptID_CNTV,
    InterruptID_CNTHV,
    InterruptID_CNTHVS,
    InterruptID_PMBIRQ,
    InterruptID_HACDBSIRQ,

    InterruptID_TRBIRQ,
};
```

Library pseudocode for shared/functions/interrupts/SetInterruptRequestLevel

```
// SetInterruptRequestLevel()
// ==========================
// Set a level-sensitive interrupt to the specified level.

impdef func SetInterruptRequestLevel(id : InterruptID, level : Signal)
begin
    return;
end;
```

Library pseudocode for shared/functions/memory/AArch64_BranchAddr

```
// AArch64_BranchAddr()
// ====================
// Return the virtual address with tag bits removed.
// This is typically used when the address will be stored to the program counter.

func AArch64_BranchAddr(vaddress : bits(64), el : bits(2)) => bits(64)
begin
    assert !UsingAArch32();
    let msbit : integer{} = AddrTop(vaddress, TRUE, el);
    if msbit == 63 then
        return vaddress;
    elsif (el IN {EL0, EL1} || ELIsInHost(el)) && vaddress[msbit] == '1' then
        return SignExtend{64}(vaddress[msbit:0]);
    else
        return ZeroExtend{64}(vaddress[msbit:0]);
    end;
end;
```

Library pseudocode for shared/functions/memory/AccessDescriptor

```
// AccessDescriptor
// ================
// Memory access or translation invocation details that steer architectural behavior


type AccessDescriptor of record {
    acctype : AccessType,
    el : bits(2),                   // Acting EL for the access
    ss : SecurityState,             // Acting Security State for the access
    acqsc : boolean,                // Acquire with Sequential Consistency
    acqpc : boolean,                // FEAT_LRCPC: Acquire with Processor Consistency
    relsc : boolean,                // Release with Sequential Consistency
    limitedordered : boolean,       // FEAT_LOR: Acquire/Release with limited ordering
    exclusive : boolean,            // Access has Exclusive semantics
    atomicop : boolean,             // FEAT_LSE: Atomic read-modify-write access
    modop : MemAtomicOp,            // FEAT_LSE: The modification operation in the 'atomicop' access
    nontemporal : boolean,          // Hints the access is non-temporal
    read : boolean,                 // Read from memory or only require read permissions
    write : boolean,                // Write to memory or only require write permissions
    cacheop : CacheOp,              // DC/IC: Cache operation
    opscope : CacheOpScope,         // DC/IC: Scope of cache operation
    cachetype : CacheType,          // DC/IC: Type of target cache
    pan : boolean,                  // FEAT_PAN: The access is subject to PSTATE.PAN
    nonfault : boolean,             // SVE: Non-faulting load
    firstfault : boolean,           // SVE: First-fault load
    first : boolean,                // SVE: First-fault load for the first active element
    contiguous : boolean,           // SVE: Contiguous load/store not gather load/scatter store
    predicated : boolean,           // SVE: Predicated load/store
    streamingsve : boolean,         // SME: Access made by PE while in streaming SVE mode
    ls64 : boolean,                 // FEAT_LS64: Accesses by accelerator support loads/stores
    withstatus : boolean,           // FEAT_LS64: Store with status result
    mops : boolean,                 // FEAT_MOPS: Memory operation (CPY/SET) accesses
    rcw : boolean,                  // FEAT_THE: Read-Check-Write access
    rcws : boolean,                 // FEAT_THE: Read-Check-Write Software access
    toplevel : boolean,             // FEAT_THE: Translation table walk access for TTB address
    varange : VARange,              // FEAT_THE: The corresponding TTBR supplying the TTB
    a32lsmd : boolean,              // A32 Load/Store Multiple Data access
    tagchecked : boolean,           // FEAT_MTE2: Access is tag checked
    tagaccess : boolean,            // FEAT_MTE: Access targets the tag bits
    stzgm : boolean,                // FEAT_MTE: Accesses that store Allocation tags to Device
                                    //           memory are CONSTRAINED UNPREDICTABLE
    Rt : integer,                   // Register named Rt in the instruction
    Rt2 : integer,                  // Register named Rt2 in the instruction
    Rs : integer,                   // Register named Rs in the instruction
    Rs2 : integer,                  // Register named Rs2 in the instruction
    ispair : boolean,               // Access represents a Load/Store pair access
    highestaddressfirst : boolean, // FEAT_LRCPC3: Highest address is accessed first
    lowestaddress : boolean,        // Is the current access the lowest address accessed by
                                    // this instruction
    mpam : MPAMinfo                 // FEAT_MPAM: MPAM information
};
```

Library pseudocode for shared/functions/memory/AccessType

```
// AccessType
// ==========

type AccessType of enumeration {
    AccessType_IFETCH,      // Instruction FETCH
    AccessType_GPR,         // Software load/store to a General Purpose Register
    AccessType_FP,          // Software load/store to an FP register
    AccessType_ASIMD,       // Software ASIMD extension load/store instructions
    AccessType_SVE,         // Software SVE load/store instructions
    AccessType_SME,         // Software SME load/store instructions
    AccessType_IC,          // Sysop IC
    AccessType_DC,          // Sysop DC (not DC {Z,G,GZ}VA)
    AccessType_DCZero,      // Sysop DC {Z,G,GZ}VA
    AccessType_AT,          // Sysop AT
    AccessType_NV2,         // NV2 memory redirected access
    AccessType_SPE,         // Statistical Profiling buffer access
    AccessType_GCS,         // Guarded Control Stack access
    AccessType_TRBE,        // Trace Buffer access
    AccessType_GPTW,        // Granule Protection Table Walk
    AccessType_HACDBS,      // Access to the HACDBS structure
    AccessType_HDBSS,       // Access to entries in HDBSS
    AccessType_TTW          // Translation Table Walk
};
```

Library pseudocode for shared/functions/memory/AddrTop

```
// AddrTop()
// =========
// Return the MSB number of a virtual address in the stage 1 translation regime for "el".
// If EL1 is using AArch64 then addresses from EL0 using AArch32 are zero-extended to 64 bits.

func AddrTop(address : bits(64), IsInstr : boolean, el : bits(2)) => AddressSize
begin
    assert HaveEL(el);
    let regime : bits(2) = S1TranslationRegime(el);
    if ELUsingAArch32(regime) then
        // AArch32 translation regime.
        return 31;
    else
        if EffectiveTBI(address, IsInstr, el) == '1' then
            return 55;
        else
            return 63;
        end;
    end;
end;
```

Library pseudocode for shared/functions/memory/AddressSize

```
// AddressSize
// ============

type AddressSize of integer{3..64};
```

Library pseudocode for shared/functions/memory/AlignmentEnforced

```
// AlignmentEnforced()
// ===================
// For the active translation regime, determine if alignment is required by all accesses

func AlignmentEnforced() => boolean
begin
    let regime : Regime = TranslationRegime(PSTATE.EL);
    var A : bit;
    case regime of
        when Regime_EL30 => A = SCTLR().A;
        when Regime_EL3 =>  A = SCTLR_EL3().A;
        when Regime_EL2 =>  A = if ELUsingAArch32(EL2) then HSCTLR().A else SCTLR_EL2().A;
        when Regime_EL20 => A = SCTLR_EL2().A;
        when Regime_EL10 => A = if ELUsingAArch32(EL1) then SCTLR().A  else SCTLR_EL1().A;
        otherwise => unreachable;
    end;
    return A == '1';
end;
```

Library pseudocode for shared/functions/memory/AllInAlignedQuantity

```
// AllInAlignedQuantity()
// ======================
// Returns TRUE if all accessed bytes are within one aligned quantity, FALSE otherwise.

readonly func AllInAlignedQuantity{N : integer{32, 64}}(address : bits(N),
                                                        size : integer,
                                                        alignment : integer) => boolean
begin
    return (AlignDownSize(address+(size-1), alignment as AddressSize) ==
            AlignDownSize(address, alignment as AddressSize));
end;
```

Library pseudocode for shared/functions/memory/Allocation

```
// Allocation hints
// ================

constant MemHint_No : bits(2)  = '00';    // No Read-Allocate, No Write-Allocate
constant MemHint_WA : bits(2)  = '01';    // No Read-Allocate, Write-Allocate
constant MemHint_RA : bits(2)  = '10';    // Read-Allocate, No Write-Allocate
constant MemHint_RWA : bits(2) = '11';    // Read-Allocate, Write-Allocate
```

Library pseudocode for shared/functions/memory/BigEndian

```
// BigEndian()
// ===========

func BigEndian(acctype : AccessType) => boolean
begin
    var bigend : boolean;
    if IsFeatureImplemented(FEAT_NV2) && acctype == AccessType_NV2 then
        return SCTLR_EL2().EE == '1';
    end;

    if UsingAArch32() then
        bigend = (PSTATE.E != '0');
    elsif PSTATE.EL == EL0 then
        bigend = (SCTLR_ELx().E0E != '0');
    else
        bigend = (SCTLR_ELx().EE != '0');
    end;
    return bigend;
end;
```

Library pseudocode for shared/functions/memory/BigEndianReverse

```
// BigEndianReverse()
// ==================

pure func BigEndianReverse{width}(value : bits(width)) => bits(width)
begin
    assert width IN {8, 16, 32, 64, 128, 256};

    if width == 8 then return value; end;
    return Reverse{width}(value, 8);
end;
```

Library pseudocode for shared/functions/memory/CacheOp

```
// CacheOp
// =======

type CacheOp of enumeration {
    CacheOp_Clean,
    CacheOp_Invalidate,
    CacheOp_CleanInvalidate
};
```

Library pseudocode for shared/functions/memory/CacheOpScope

```
// CacheOpScope
// ============

type CacheOpScope of enumeration {
    CacheOpScope_SetWay,
    CacheOpScope_PoU,
    CacheOpScope_PoC,
    CacheOpScope_PoE,
    CacheOpScope_PoP,
    CacheOpScope_PoDP,
    CacheOpScope_PoPA,
    CacheOpScope_PoPS,
    CacheOpScope_OuterCache,
    CacheOpScope_ALLU,
    CacheOpScope_ALLUIS_CnP,
    CacheOpScope_ALLUIS
};
```

Library pseudocode for shared/functions/memory/CachePASpace

```
// CachePASpace
// ============

type CachePASpace of enumeration {
    CPAS_NonSecure,
    CPAS_Any,                   // Applicable only for DC *SW / IC IALLU* in Root state:
                                // match entries from any PA Space
    CPAS_RealmNonSecure,        // Applicable only for DC *SW / IC IALLU* in Realm state:
                                // match entries from Realm or Non-Secure PAS
    CPAS_Realm,
    CPAS_Root,
    CPAS_SystemAgent,           // Applicable only for DC by PA:
                                // match entries from the System Agent PAS
    CPAS_NonSecureProtected,    // Applicable only for DC by PA:
                                // match entries from the Non-Secure Protected PAS
    CPAS_NA6,                   // Reserved
    CPAS_NA7,                   // Reserved
    CPAS_SecureNonSecure,       // Applicable only for DC *SW / IC IALLU* in Secure state:
                                // match entries from Secure or Non-Secure PAS
    CPAS_Secure
};
```

Library pseudocode for shared/functions/memory/CacheType

```
// CacheType
// =========

type CacheType of enumeration {
    CacheType_Data,
    CacheType_Tag,
    CacheType_Data_Tag,
    CacheType_TagWrite,
    CacheType_TagZero,
    CacheType_Instruction
};
```

Library pseudocode for shared/functions/memory/Cacheability

```
// Cacheability attributes
// =======================

constant MemAttr_NC : bits(2) = '00';     // Non-cacheable
constant MemAttr_WT : bits(2) = '10';     // Write-through
constant MemAttr_WB : bits(2) = '11';     // Write-back
```

Library pseudocode for shared/functions/memory/CreateAccDescA32LSMD

```
// CreateAccDescA32LSMD()
// ======================
// Access descriptor for A32 loads/store multiple general purpose registers

func CreateAccDescA32LSMD(memop : MemOp) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.pan             = TRUE;
    accdesc.a32lsmd         = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescASIMD

```
// CreateAccDescASIMD()
// ====================
// Access descriptor for ASIMD&FP loads/stores

func CreateAccDescASIMD(memop : MemOp, nontemporal : boolean, tagchecked : boolean,
                        privileged : boolean) => AccessDescriptor
begin
    let ispair : boolean = FALSE;
    return CreateAccDescASIMD(memop, nontemporal, tagchecked, privileged, ispair);
end;

// CreateAccDescASIMD()
// ====================

func CreateAccDescASIMD(memop : MemOp, nontemporal : boolean, tagchecked : boolean,
                        privileged : boolean, ispair : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_ASIMD);


    accdesc.nontemporal     = nontemporal;
    accdesc.el              = if !privileged then EL0 else PSTATE.EL;
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.ispair          = ispair;
    accdesc.pan             = TRUE;
    accdesc.streamingsve    = InStreamingMode();
    if (accdesc.streamingsve && ImpDefBool(
          "No tag checking of SIMD&FP loads and stores in Streaming SVE mode")) then
        accdesc.tagchecked  = FALSE;
    else
        accdesc.tagchecked  = tagchecked;
    end;
    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescASIMDAcqRel

```
// CreateAccDescASIMDAcqRel()
// ==========================
// Access descriptor for ASIMD&FP loads/stores with ordering semantics

func CreateAccDescASIMDAcqRel(memop : MemOp, tagchecked : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_ASIMD);

    accdesc.acqpc           = memop == MemOp_LOAD;
    accdesc.relsc           = memop == MemOp_STORE;
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.pan             = TRUE;
    accdesc.streamingsve    = InStreamingMode();
    if (accdesc.streamingsve && ImpDefBool(
          "No tag checking of SIMD&FP loads and stores in Streaming SVE mode")) then
        accdesc.tagchecked  = FALSE;
    else
        accdesc.tagchecked  = tagchecked;
    end;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescAT

```
// CreateAccDescAT()
// =================
// Access descriptor for address translation operations

func CreateAccDescAT(ss : SecurityState, el : bits(2), ataccess : ATAccess) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_AT);

    accdesc.el              = el;
    accdesc.ss              = ss;
    if ImpDefBool("MPAM uses the EL targeted by the AT instruction") then
        accdesc.mpam = GenMPAMAtEL(AccessType_AT, el);
    end;
    case ataccess of
        when ATAccess_Read =>
            accdesc.(read, write, pan) = (TRUE, FALSE, FALSE);
        when ATAccess_ReadPAN =>
            accdesc.(read, write, pan) = (TRUE, FALSE, TRUE);
        when ATAccess_Write =>
            accdesc.(read, write, pan) = (FALSE, TRUE, FALSE);
        when ATAccess_WritePAN =>
            accdesc.(read, write, pan) = (FALSE, TRUE, TRUE);
        when ATAccess_Any =>
            accdesc.(read, write, pan) = (FALSE, FALSE, FALSE);
    end;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescAcqRel

```
// CreateAccDescAcqRel()
// =====================
// Access descriptor for general purpose register loads/stores with ordering semantics

func CreateAccDescAcqRel(memop : MemOp, tagchecked : boolean, acqsc : boolean) => AccessDescriptor
begin
    let Rt : integer = -1;
    return CreateAccDescAcqRel(memop, tagchecked, acqsc, Rt);
end;

func CreateAccDescAcqRel(memop : MemOp, tagchecked : boolean,
                         acqsc : boolean, Rt : integer) => AccessDescriptor
begin
    let ispair : boolean = FALSE;
    let Rt2 : integer = -1;
    return CreateAccDescAcqRel(memop, tagchecked, ispair, acqsc, Rt, Rt2);
end;

func CreateAccDescAcqRel(memop : MemOp, tagchecked : boolean, ispair : boolean,
                         acqsc : boolean, Rt : integer, Rt2 : integer) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.acqsc           = acqsc;
    accdesc.relsc           = memop == MemOp_STORE;
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.pan             = TRUE;
    accdesc.tagchecked      = tagchecked;
    accdesc.ispair          = ispair;
    accdesc.Rt              = Rt;
    accdesc.Rt2             = Rt2;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescAtomicOp

```
// CreateAccDescAtomicOp()
// =======================
// Access descriptor for atomic read-modify-write memory accesses

func CreateAccDescAtomicOp(modop : MemAtomicOp, acquire : boolean, release : boolean,
                           tagchecked : boolean, privileged : boolean, Rt : integer,
                           Rs : integer) => AccessDescriptor
begin
    let Rt2 : integer = -1;
    let Rs2 : integer = -1;

    return CreateAccDescAtomicOp(modop, acquire, release, tagchecked, privileged, Rt, Rt2, Rs, Rs2);
end;

func CreateAccDescAtomicOp(modop : MemAtomicOp, acquire : boolean, release : boolean,
                           tagchecked : boolean, privileged : boolean, Rt : integer,
                           Rt2 : integer, Rs : integer, Rs2 : integer) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.acqsc           = acquire;
    accdesc.el              = if !privileged then EL0 else PSTATE.EL;
    accdesc.relsc           = release;
    accdesc.atomicop        = TRUE;
    accdesc.modop           = modop;
    accdesc.read            = TRUE;
    accdesc.write           = TRUE;
    accdesc.pan             = TRUE;
    accdesc.tagchecked      = tagchecked;
    accdesc.Rs              = Rs;
    accdesc.Rs2             = Rs2;
    accdesc.Rt              = Rt;
    accdesc.Rt2             = Rt2;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescDC

```
// CreateAccDescDC()
// =================
// Access descriptor for data cache operations

func CreateAccDescDC(cache : CacheRecord) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_DC);

    accdesc.cacheop         = cache.cacheop;
    accdesc.cachetype       = cache.cachetype;
    accdesc.opscope         = cache.opscope;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescDCZero

```
// CreateAccDescDCZero()
// =====================
// Access descriptor for data cache zero operations

func CreateAccDescDCZero(cachetype : CacheType) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_DCZero);

    accdesc.write           = TRUE;
    accdesc.pan             = TRUE;
    accdesc.tagchecked      = cachetype == CacheType_Data;
    accdesc.tagaccess       = cachetype IN {CacheType_Tag,
                                            CacheType_TagWrite,
                                            CacheType_TagZero,
                                            CacheType_Data_Tag};
    accdesc.cachetype       = cachetype;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescExLDST

```
// CreateAccDescExLDST()
// =====================
// Access descriptor for general purpose register loads/stores with exclusive semantics

func CreateAccDescExLDST(memop : MemOp, acqrel : boolean, tagchecked : boolean,
                         privileged : boolean) => AccessDescriptor
begin
    let Rt : integer = -1;
    return CreateAccDescExLDST(memop, acqrel, tagchecked, privileged, Rt);
end;

// CreateAccDescExLDST()
// =====================

func CreateAccDescExLDST(memop : MemOp, acqrel : boolean, tagchecked : boolean,
                         privileged : boolean, Rt : integer) => AccessDescriptor
begin
    let ispair : boolean = FALSE;
    let Rt2 : integer = -1;
    return CreateAccDescExLDST(memop, acqrel, tagchecked, privileged, ispair, Rt, Rt2);
end;

// CreateAccDescExLDST()
// =====================

func CreateAccDescExLDST(memop : MemOp, acqrel : boolean, tagchecked : boolean,
                         privileged : boolean, ispair : boolean,
                         Rt : integer, Rt2 : integer) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.acqsc           = acqrel && memop == MemOp_LOAD;
    accdesc.relsc           = acqrel && memop == MemOp_STORE;
    accdesc.exclusive       = TRUE;
    accdesc.el              = if !privileged then EL0 else PSTATE.EL;
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.ispair          = ispair;
    accdesc.pan             = TRUE;
    accdesc.tagchecked      = tagchecked;
    accdesc.Rt              = Rt;
    accdesc.Rt2             = Rt2;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescFPAtomicOp

```
// CreateAccDescFPAtomicOp()
// =========================
// Access descriptor for FP atomic read-modify-write memory accesses

func CreateAccDescFPAtomicOp(modop : MemAtomicOp, acquire : boolean, release : boolean,
                             tagchecked : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_FP);

    accdesc.acqsc           = acquire;
    accdesc.relsc           = release;
    accdesc.atomicop        = TRUE;
    accdesc.modop           = modop;
    accdesc.read            = TRUE;
    accdesc.write           = TRUE;
    accdesc.pan             = TRUE;
    accdesc.streamingsve    = InStreamingMode();
    if (accdesc.streamingsve && ImpDefBool(
          "No tag checking of SIMD&FP loads and stores in Streaming SVE mode")) then
        accdesc.tagchecked  = FALSE;
    else
        accdesc.tagchecked  = tagchecked;
    end;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescGCS

```
// CreateAccDescGCS()
// ==================
// Access descriptor for memory accesses to the Guarded Control Stack

func CreateAccDescGCS(memop : MemOp, privileged : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GCS);

    accdesc.el              = if !privileged then EL0 else PSTATE.EL;
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescGCSSS1

```
// CreateAccDescGCSSS1()
// =====================
// Access descriptor for memory accesses to the Guarded Control Stack that switch stacks

func CreateAccDescGCSSS1(privileged : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GCS);

    accdesc.el              = if !privileged then EL0 else PSTATE.EL;
    accdesc.atomicop        = TRUE;
    accdesc.modop           = MemAtomicOp_GCSSS1;
    accdesc.read            = TRUE;
    accdesc.write           = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescGPR

```
// CreateAccDescGPR()
// ==================
// Access descriptor for general purpose register loads/stores
// without exclusive or ordering semantics

func CreateAccDescGPR(memop : MemOp, nontemporal : boolean, privileged : boolean,
                      tagchecked : boolean) => AccessDescriptor
begin
    let Rt : integer = -1;
    return CreateAccDescGPR(memop, nontemporal, privileged, tagchecked, Rt);
end;

func CreateAccDescGPR(memop : MemOp, nontemporal : boolean, privileged : boolean,
                      tagchecked : boolean, Rt : integer) => AccessDescriptor
begin
    let ispair : boolean = FALSE;
    let Rt2 : integer = -1;
    return CreateAccDescGPR(memop, nontemporal, privileged, tagchecked, ispair, Rt, Rt2);
end;

func CreateAccDescGPR(memop : MemOp, nontemporal : boolean, privileged : boolean,
                      tagchecked : boolean, ispair : boolean,
                      Rt : integer, Rt2 : integer) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.el              = if !privileged then EL0 else PSTATE.EL;
    accdesc.nontemporal     = nontemporal;
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.pan             = TRUE;
    accdesc.tagchecked      = tagchecked;
    accdesc.Rt              = Rt;
    accdesc.Rt2             = Rt2;
    accdesc.ispair          = ispair;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescGPTW

```
// CreateAccDescGPTW()
// ===================
// Access descriptor for Granule Protection Table walks

func CreateAccDescGPTW(accdesc_in : AccessDescriptor) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPTW, accdesc_in.mpam);

    accdesc.el              = accdesc_in.el;
    accdesc.ss              = accdesc_in.ss;
    accdesc.read            = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescHACDBS

```
// CreateAccDescHACDBS()
// =====================
// Access descriptor for memory accesses to the HACDBS structure.

func CreateAccDescHACDBS() => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_HACDBS);

    accdesc.read            = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescHDBSS

```
// CreateAccDescHDBSS()
// ====================
// Access descriptor for appending entries to the HDBSS

func CreateAccDescHDBSS(accdesc_in : AccessDescriptor) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_HDBSS, accdesc_in.mpam);

    accdesc.el              = accdesc_in.el;
    accdesc.ss              = accdesc_in.ss;
    accdesc.write           = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescIC

```
// CreateAccDescIC()
// =================
// Access descriptor for instruction cache operations

func CreateAccDescIC(cache : CacheRecord) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_IC);

    accdesc.cacheop         = cache.cacheop;
    accdesc.cachetype       = cache.cachetype;
    accdesc.opscope         = cache.opscope;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescIFetch

```
// CreateAccDescIFetch()
// =====================
// Access descriptor for instruction fetches

func CreateAccDescIFetch() => AccessDescriptor
begin
    let accdesc : AccessDescriptor = NewAccDesc(AccessType_IFETCH);

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescLDAcqPC

```
// CreateAccDescLDAcqPC()
// ======================
// Access descriptor for general purpose register loads with local ordering semantics

func CreateAccDescLDAcqPC(tagchecked : boolean, acquire : boolean, Rt : integer) => AccessDescriptor
begin
    let ispair : boolean = FALSE;
    let Rt2 : integer = -1;
    return CreateAccDescLDAcqPC(tagchecked, ispair, acquire, Rt, Rt2);
end;

func CreateAccDescLDAcqPC(tagchecked : boolean, ispair : boolean, acquire : boolean,
                          Rt : integer, Rt2 : integer) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.acqpc           = acquire;
    accdesc.read            = TRUE;
    accdesc.pan             = TRUE;
    accdesc.tagchecked      = tagchecked;
    accdesc.ispair          = ispair;
    accdesc.Rt              = Rt;
    accdesc.Rt2             = Rt2;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescLDGSTG

```
// CreateAccDescLDGSTG()
// =====================
// Access descriptor for tag memory loads/stores

func CreateAccDescLDGSTG(memop : MemOp, stzgm : boolean, Rt : integer) => AccessDescriptor
begin
    let ispair : boolean = FALSE;
    let Rt2 : integer = -1;
    return CreateAccDescLDGSTG(memop, stzgm, ispair, Rt, Rt2);
end;

// CreateAccDescLDGSTG()
// =====================

func CreateAccDescLDGSTG(memop : MemOp, stzgm : boolean, ispair : boolean, Rt : integer,
                         Rt2 : integer) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.pan             = TRUE;
    accdesc.tagaccess       = TRUE;
    accdesc.stzgm           = stzgm;
    accdesc.ispair          = ispair;
    accdesc.Rt              = Rt;
    accdesc.Rt2             = Rt2;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescLOR

```
// CreateAccDescLOR()
// ==================
// Access descriptor for general purpose register loads/stores with limited ordering semantics

func CreateAccDescLOR(memop : MemOp, tagchecked : boolean,
                      acqsc : boolean, Rt : integer) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.acqsc           = acqsc;
    accdesc.relsc           = memop == MemOp_STORE;
    accdesc.limitedordered  = TRUE;
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.pan             = TRUE;
    accdesc.tagchecked      = tagchecked;
    accdesc.Rt              = Rt;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescLS64

```
// CreateAccDescLS64()
// ===================
// Access descriptor for accelerator-supporting memory accesses

func CreateAccDescLS64(memop : MemOp, withstatus : boolean, acqrel : boolean,
                       tagchecked : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.acqsc           = memop == MemOp_LOAD && acqrel;
    accdesc.relsc           = memop == MemOp_STORE && acqrel;
    accdesc.pan             = TRUE;
    accdesc.ls64            = TRUE;
    accdesc.withstatus      = withstatus;
    accdesc.tagchecked      = tagchecked;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescMOPS

```
// CreateAccDescMOPS()
// ===================
// Access descriptor for data memory copy and set instructions

func CreateAccDescMOPS(memop : MemOp, privileged : boolean,
                       nontemporal : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.el              = if !privileged then EL0 else PSTATE.EL;
    accdesc.nontemporal     = nontemporal;
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.pan             = TRUE;
    accdesc.mops            = TRUE;
    accdesc.tagchecked      = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescNV2

```
// CreateAccDescNV2()
// ==================
// Access descriptor nested virtualization memory indirection loads/stores

func CreateAccDescNV2(memop : MemOp) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_NV2);

    accdesc.el              = EL2;
    accdesc.ss              = SecurityStateAtEL(EL2);
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescRCW

```
// CreateAccDescRCW()
// ==================
// Access descriptor for atomic read-check-write memory accesses

func CreateAccDescRCW(modop : MemAtomicOp, soft : boolean, acquire : boolean,
                      release : boolean, tagchecked : boolean,
                      Rt : integer, Rs : integer) => AccessDescriptor
begin
    let Rt2 : integer = -1;
    let Rs2 : integer = -1;

    return CreateAccDescRCW(modop, soft, acquire, release, tagchecked, Rt, Rt2, Rs, Rs2);
end;

func CreateAccDescRCW(modop : MemAtomicOp, soft : boolean, acquire : boolean,
                      release : boolean, tagchecked : boolean, Rt : integer, Rt2 : integer,
                      Rs : integer, Rs2 : integer) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.acqsc           = acquire;
    accdesc.relsc           = release;
    accdesc.rcw             = TRUE;
    accdesc.rcws            = soft;
    accdesc.atomicop        = TRUE;
    accdesc.modop           = modop;
    accdesc.read            = TRUE;
    accdesc.write           = TRUE;
    accdesc.pan             = TRUE;
    accdesc.tagchecked      = tagchecked;
    accdesc.Rt              = Rt;
    accdesc.Rt2             = Rt2;
    accdesc.Rs              = Rs;
    accdesc.Rs2             = Rs2;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescS1TTW

```
// CreateAccDescS1TTW()
// ====================
// Access descriptor for stage 1 translation table walks

func CreateAccDescS1TTW(toplevel : boolean,
                        varange : VARange,
                        accdesc_in : AccessDescriptor) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_TTW, accdesc_in.mpam);

    accdesc.el              = accdesc_in.el;
    accdesc.ss              = accdesc_in.ss;
    accdesc.read            = TRUE;
    accdesc.toplevel        = toplevel;
    accdesc.varange         = varange;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescS2TTW

```
// CreateAccDescS2TTW()
// ====================
// Access descriptor for stage 2 translation table walks

func CreateAccDescS2TTW(accdesc_in : AccessDescriptor) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_TTW, accdesc_in.mpam);

    accdesc.el              = accdesc_in.el;
    accdesc.ss              = accdesc_in.ss;
    accdesc.read            = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescSME

```
// CreateAccDescSME()
// ==================
// Access descriptor for SME loads/stores

func CreateAccDescSME(memop : MemOp, nontemporal : boolean, contiguous : boolean,
                      tagchecked : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_SME);

    accdesc.nontemporal     = nontemporal;
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.pan             = TRUE;
    accdesc.contiguous      = contiguous;
    accdesc.streamingsve    = TRUE;
    if ImpDefBool("No tag checking of SME LDR & STR instructions") then
        accdesc.tagchecked  = FALSE;
    else
        accdesc.tagchecked  = tagchecked;
    end;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescSPE

```
// CreateAccDescSPE()
// ==================
// Access descriptor for memory accesses by Statistical Profiling unit

func CreateAccDescSPE(owning_ss : SecurityState, owning_el : bits(2)) => AccessDescriptor
begin
    let mpaminfo : MPAMinfo = GenMPAMAtEL(AccessType_SPE, owning_el);
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_SPE, mpaminfo);

    accdesc.el              = owning_el;
    accdesc.ss              = owning_ss;
    accdesc.write           = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescSTGMOPS

```
// CreateAccDescSTGMOPS()
// ======================
// Access descriptor for tag memory set instructions

func CreateAccDescSTGMOPS(privileged : boolean, nontemporal : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_GPR);

    accdesc.el              = if !privileged then EL0 else PSTATE.EL;
    accdesc.nontemporal     = nontemporal;
    accdesc.write           = TRUE;
    accdesc.pan             = TRUE;
    accdesc.mops            = TRUE;
    accdesc.tagaccess       = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescSVE

```
// CreateAccDescSVE()
// ==================
// Access descriptor for general SVE loads/stores

func CreateAccDescSVE(memop : MemOp, nontemporal : boolean, contiguous : boolean,
                      tagchecked : boolean) => AccessDescriptor
begin
    let predicated : boolean = FALSE;
    return CreateAccDescSVE(memop, nontemporal, contiguous, predicated, tagchecked);
end;

func CreateAccDescSVE(memop : MemOp, nontemporal : boolean, contiguous : boolean,
                      predicated : boolean, tagchecked : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_SVE);

    accdesc.nontemporal     = nontemporal;
    accdesc.read            = memop == MemOp_LOAD;
    accdesc.write           = memop == MemOp_STORE;
    accdesc.pan             = TRUE;
    accdesc.contiguous      = contiguous;
    accdesc.streamingsve    = InStreamingMode();
    if (accdesc.streamingsve && ImpDefBool(
          "No tag checking of SIMD&FP loads and stores in Streaming SVE mode")) then
        accdesc.tagchecked  = FALSE;
    else
        accdesc.tagchecked  = tagchecked;
    end;
    accdesc.predicated      = predicated;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescSVEFF

```
// CreateAccDescSVEFF()
// ====================
// Access descriptor for first-fault SVE loads

func CreateAccDescSVEFF(contiguous : boolean, tagchecked : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_SVE);

    accdesc.read            = TRUE;
    accdesc.pan             = TRUE;
    accdesc.firstfault      = TRUE;
    accdesc.first           = TRUE;
    accdesc.contiguous      = contiguous;
    accdesc.streamingsve    = InStreamingMode();
    if (accdesc.streamingsve && ImpDefBool(
          "No tag checking of SIMD&FP loads and stores in Streaming SVE mode")) then
        accdesc.tagchecked  = FALSE;
    else
        accdesc.tagchecked  = tagchecked;
    end;
    accdesc.predicated      = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescSVENF

```
// CreateAccDescSVENF()
// ====================
// Access descriptor for non-fault SVE loads

func CreateAccDescSVENF(contiguous : boolean, tagchecked : boolean) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_SVE);

    accdesc.read            = TRUE;
    accdesc.pan             = TRUE;
    accdesc.nonfault        = TRUE;
    accdesc.contiguous      = contiguous;
    accdesc.streamingsve    = InStreamingMode();
    if (accdesc.streamingsve && ImpDefBool(
          "No tag checking of SIMD&FP loads and stores in Streaming SVE mode")) then
        accdesc.tagchecked  = FALSE;
    else
        accdesc.tagchecked  = tagchecked;
    end;
    accdesc.predicated      = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescTRBE

```
// CreateAccDescTRBE()
// ===================
// Access descriptor for memory accesses by Trace Buffer Unit

func CreateAccDescTRBE(owning_ss : SecurityState, owning_el : bits(2)) => AccessDescriptor
begin
    var mpam : MPAMinfo;
    if SelfHostedTraceEnabled() then
        mpam = GenMPAMAtEL(AccessType_TRBE, owning_el);
    else
        mpam = GenMPAMTRBE();
    end;
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_TRBE, mpam);

    accdesc.el              = owning_el;
    accdesc.ss              = owning_ss;
    accdesc.write           = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/CreateAccDescTTEUpdate

```
// CreateAccDescTTEUpdate()
// ========================
// Access descriptor for translation table entry HW update

func CreateAccDescTTEUpdate(accdesc_in : AccessDescriptor) => AccessDescriptor
begin
    var accdesc : AccessDescriptor = NewAccDesc(AccessType_TTW, accdesc_in.mpam);

    accdesc.el              = accdesc_in.el;
    accdesc.ss              = accdesc_in.ss;
    accdesc.atomicop        = TRUE;
    accdesc.modop           = MemAtomicOp_CAS;
    accdesc.read            = TRUE;
    accdesc.write           = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/DataMemoryBarrier

```
// DataMemoryBarrier()
// ===================

impdef func DataMemoryBarrier(types : MBReqTypes)
begin
    return;
end;
```

Library pseudocode for shared/functions/memory/DataSynchronizationBarrier

```
// DataSynchronizationBarrier()
// ============================

impdef func DataSynchronizationBarrier(scope : MBMaintenanceScope,
                                       types : MBReqTypes,
                                       nXS : boolean)
begin
    return;
end;
```

Library pseudocode for shared/functions/memory/DeviceType

```
// DeviceType
// ==========
// Extended memory types for Device memory.

type DeviceType of enumeration {DeviceType_GRE, DeviceType_nGRE,
                                DeviceType_nGnRE, DeviceType_nGnRnE};
```

Library pseudocode for shared/functions/memory/EffectiveMTX

```
// EffectiveMTX()
// ==============
// Returns the effective MTX in the AArch64 stage 1 translation regime for "el".

readonly func EffectiveMTX(address : bits(64), is_instr : boolean, el : bits(2)) => bit
begin
    var mtx : bit;
    assert HaveEL(el);
    let regime : bits(2) = S1TranslationRegime(el);
    assert(!ELUsingAArch32(regime));

    if is_instr then
        mtx = '0';
    else
        case regime of
            when EL1 =>
                mtx = if address[55] == '1' then TCR_EL1().MTX1 else TCR_EL1().MTX0;
            when EL2 =>
                if IsFeatureImplemented(FEAT_VHE) && ELIsInHost(el) then
                    mtx = if address[55] == '1' then TCR_EL2().MTX1 else TCR_EL2().MTX0;
                else
                    mtx = TCR_EL2().MTX;
                end;
            when EL3 =>
                mtx = TCR_EL3().MTX;
        end;
    end;

    return mtx;
end;
```

Library pseudocode for shared/functions/memory/EffectiveTBI

```
// EffectiveTBI()
// ==============
// Returns the effective TBI in the AArch64 stage 1 translation regime for "el".

readonly func EffectiveTBI(address : bits(64), IsInstr : boolean, el : bits(2)) => bit
begin
    var tbi : bit;
    var tbid : bit;
    assert HaveEL(el);
    let regime : bits(2) = S1TranslationRegime(el);
    assert(!ELUsingAArch32(regime));

    case regime of
        when EL1 =>
            tbi = if address[55] == '1' then TCR_EL1().TBI1 else TCR_EL1().TBI0;
            if IsFeatureImplemented(FEAT_PAuth) then
                tbid = if address[55] == '1' then TCR_EL1().TBID1 else TCR_EL1().TBID0;
            end;
        when EL2 =>
            if IsFeatureImplemented(FEAT_VHE) && ELIsInHost(el) then
                tbi = if address[55] == '1' then TCR_EL2().TBI1 else TCR_EL2().TBI0;
                if IsFeatureImplemented(FEAT_PAuth) then
                    tbid = if address[55] == '1' then TCR_EL2().TBID1 else TCR_EL2().TBID0;
                end;
            else
                tbi = TCR_EL2().TBI;
                if IsFeatureImplemented(FEAT_PAuth) then tbid = TCR_EL2().TBID; end;
            end;
        when EL3 =>
            tbi = TCR_EL3().TBI;
            if IsFeatureImplemented(FEAT_PAuth) then tbid = TCR_EL3().TBID; end;
    end;

    return (if (tbi == '1' && (!IsFeatureImplemented(FEAT_PAuth) || tbid == '0' ||
            !IsInstr)) then '1' else '0');
end;
```

Library pseudocode for shared/functions/memory/EffectiveTCMA

```
// EffectiveTCMA()
// ===============
// Returns the effective TCMA of a virtual address in the stage 1 translation regime for "el".

readonly func EffectiveTCMA(address : bits(64), el : bits(2)) => bit
begin
    var tcma : bit;
    assert HaveEL(el);
    let regime : bits(2) = S1TranslationRegime(el);
    assert(!ELUsingAArch32(regime));

    case regime of
        when EL1 =>
            tcma = if address[55] == '1' then TCR_EL1().TCMA1 else TCR_EL1().TCMA0;
        when EL2 =>
            if IsFeatureImplemented(FEAT_VHE) && ELIsInHost(el) then
                tcma = if address[55] == '1' then TCR_EL2().TCMA1 else TCR_EL2().TCMA0;
            else
                tcma = TCR_EL2().TCMA;
            end;
        when EL3 =>
            tcma = TCR_EL3().TCMA;
    end;

    return tcma;
end;
```

Library pseudocode for shared/functions/memory/ErrorState

```
// ErrorState
// ==========
// The allowed error states that can be returned by memory and used by the PE.

type ErrorState of enumeration {
                        ErrorState_UC,            // Uncontainable
                        ErrorState_UEU,           // Unrecoverable state
                        ErrorState_UEO,           // Restartable state
                        ErrorState_UER,           // Recoverable state
                        ErrorState_CE};           // Corrected
```

Library pseudocode for shared/functions/memory/Fault

```
// Fault
// =====
// Fault types.

type Fault of enumeration {
                    Fault_None,
                    Fault_AccessFlag,
                    Fault_Alignment,
                    Fault_Background,
                    Fault_Domain,
                    Fault_Permission,
                    Fault_Translation,
                    Fault_AddressSize,
                    Fault_SyncExternal,
                    Fault_SyncExternalOnWalk,
                    Fault_SyncParity,
                    Fault_SyncParityOnWalk,
                    Fault_GPCFOnWalk,
                    Fault_GPCFOnOutput,
                    Fault_AsyncParity,
                    Fault_AsyncExternal,
                    Fault_TagCheck,
                    Fault_Debug,
                    Fault_TLBConflict,
                    Fault_BranchTarget,
                    Fault_UnsupportedAtomicHWUpdate,
                    Fault_Lockdown,
                    Fault_Exclusive,
                    Fault_ICacheMaint};
```

Library pseudocode for shared/functions/memory/FaultRecord

```
// FaultRecord
// ===========
// Fields that relate only to Faults.

type FaultRecord of record {
    statuscode : Fault,             // Fault Status
    accessdesc : AccessDescriptor,  // Details of the faulting access
    vaddress   : bits(64),          // Faulting virtual address
    ipaddress : FullAddress,        // Intermediate physical address
    paddress : FullAddress,         // Physical address
    gpcf : GPCFRecord,              // Granule Protection Check Fault record
    gpcfs2walk : boolean,           // GPC for a stage 2 translation table walk
    s2fs1walk : boolean,            // Is on a Stage 1 translation table walk
    write : boolean,                // TRUE for a write, FALSE for a read
    s1tagnotdata : boolean,         // TRUE for a fault due to tag not accessible at stage 1.
    tagaccess : boolean,            // TRUE for a fault due to NoTagAccess permission.
    level : integer,                // For translation, access flag and Permission faults
    extflag : bit,                  // IMPLEMENTATION DEFINED syndrome for External aborts
    secondstage : boolean,          // Is a Stage 2 abort
    assuredonly : boolean,          // Stage 2 Permission fault due to AssuredOnly attribute
    toplevel    : boolean,          // Stage 2 Permission fault due to TopLevel
    overlay : boolean,              // Fault due to overlay permissions
    dirtybit : boolean,             // Fault due to dirty state
    domain : bits(4),               // Domain number, AArch32 only
    merrorstate : ErrorState,       // Incoming error state from memory
    hdbssf : boolean,               // Fault caused by HDBSS
    watchptinfo : WatchpointInfo,   // Watchpoint related fields
    debugmoe : bits(4)              // Debug method of entry, from AArch32 only
};
```

Library pseudocode for shared/functions/memory/FullAddress

```
// FullAddress
// ===========
// Physical or Intermediate Physical Address type.
// Although AArch32 only has access to 40 bits of physical or intermediate physical address space,
// the full address type has NUM_PABITS bits to allow interprocessing with AArch64.
// The maximum physical or intermediate physical address size is IMPLEMENTATION DEFINED,
// but never exceeds NUM_PABITS bits.

type FullAddress of record {
    paspace : PASpace,
    address : bits(NUM_PABITS)
};
```

Library pseudocode for shared/functions/memory/GPCF

```
// GPCF
// ====
// Possible Granule Protection Check Fault reasons

type GPCF of enumeration {
    GPCF_None,        // No fault
    GPCF_AddressSize, // GPT address size fault
    GPCF_Walk,        // GPT walk fault
    GPCF_EABT,        // Synchronous External abort on GPT fetch
    GPCF_Fail         // Granule protection fault
};
```

Library pseudocode for shared/functions/memory/GPCFRecord

```
// GPCFRecord
// ==========
// Full details of a Granule Protection Check Fault

type GPCFRecord of record {
    gpf : GPCF,
    level : integer
};
```

Library pseudocode for shared/functions/memory/Hint_Prefetch

```
// Hint_Prefetch()
// ===============
// Signals the memory system that memory accesses of type HINT to or from the specified address are
// likely in the near future. The memory system may take some action to speed up the memory
// accesses when they do occur, such as pre-loading the specified address into one or more
// caches as indicated by the innermost cache level target (0=L1, 1=L2, etc) and non-temporal hint
// stream. Any or all prefetch hints may be treated as a NOP. A prefetch hint must not cause a
// synchronous abort due to Alignment or Translation faults and the like. Its only effect on
// software-visible state should be on caches and TLBs associated with address, which must be
// accessible by reads, writes or execution, as defined in the translation regime of the current
// Exception level. It is guaranteed not to access Device memory.
// A Prefetch_EXEC hint must not result in an access that could not be performed by a speculative
// instruction fetch, therefore if all associated MMUs are disabled, then it cannot access any
// memory location that cannot be accessed by instruction fetches.

impdef func Hint_Prefetch(address : bits(64),
                          hint : PrefetchHint,
                          target : integer,
                          stream : boolean)
begin
    return;
end;
```

Library pseudocode for shared/functions/memory/Hint_RangePrefetch

```
// Hint_RangePrefetch()
// ====================
// Signals the memory system that data memory accesses from a specified range
// of addresses are likely to occur in the near future. The memory system can
// respond by taking actions that are expected to speed up the memory accesses
// when they do occur, such as preloading the locations within the specified
// address ranges into one or more caches.

impdef func Hint_RangePrefetch(address : bits(64), length : integer, stride : integer,
                               count : integer, reuse : integer, operation : bits(6))
begin
    return;
end;
```

Library pseudocode for shared/functions/memory/IsContiguousSVEAccess

```
// IsContiguousSVEAccess()
// =======================
// Return TRUE if memory access is contiguous load/stores in an SVE mode.

func IsContiguousSVEAccess(accdesc : AccessDescriptor) => boolean
begin
    return accdesc.acctype == AccessType_SVE && accdesc.contiguous;
end;
```

Library pseudocode for shared/functions/memory/IsRelaxedWatchpointAccess

```
// IsRelaxedWatchpointAccess()
// ===========================
// Return TRUE if memory access is one of -
// - SIMD&FP load/store instruction when the PE is in Streaming SVE mode
// - SVE contiguous vector load/store instruction.
// - SME load/store instruction

func IsRelaxedWatchpointAccess(accdesc : AccessDescriptor) => boolean
begin
    return (IsContiguousSVEAccess(accdesc) ||
            IsSMEAccess(accdesc) ||
            (IsSIMDFPAccess(accdesc) && InStreamingMode()));
end;
```

Library pseudocode for shared/functions/memory/IsSIMDFPAccess

```
// IsSIMDFPAccess()
// ================
// Return TRUE if access is SIMD&FP.

func IsSIMDFPAccess(accdesc : AccessDescriptor) => boolean
begin
    return accdesc.acctype == AccessType_ASIMD;
end;
```

Library pseudocode for shared/functions/memory/IsSMEAccess

```
// IsSMEAccess()
// =============
// Return TRUE if access is of SME load/stores.

func IsSMEAccess(accdesc : AccessDescriptor) => boolean
begin
    return accdesc.acctype == AccessType_SME;
end;
```

Library pseudocode for shared/functions/memory/IsWatchpointableAccess

```
// IsWatchpointableAccess()
// ========================
// Return TRUE if access should be checked for watchpoints.

func IsWatchpointableAccess(accdesc : AccessDescriptor) => boolean
begin
    return (!(accdesc.acctype IN {AccessType_IFETCH,
                                  AccessType_TTW,
                                  AccessType_DC,
                                  AccessType_IC,
                                  AccessType_SPE,
                                  AccessType_TRBE,
                                  AccessType_AT}) ||
            (accdesc.acctype == AccessType_DC &&
             accdesc.cacheop == CacheOp_Invalidate &&
             (!UsingAArch32() || (ImpDefBool("DCIMVAC generates watchpoint")))));
end;
```

Library pseudocode for shared/functions/memory/MBMaintenanceScope

```
// MBMaintenanceScope
// ==================
// Memory barrier scope.

type MBMaintenanceScope of enumeration {MBMaintenanceScope_Nonshareable,
                                        MBMaintenanceScope_InnerShareable,
                                        MBMaintenanceScope_OuterShareable,
                                        MBMaintenanceScope_None};
```

Library pseudocode for shared/functions/memory/MBReqTypes

```
// MBReqTypes
// ==========
// Memory barrier read/write.

type MBReqTypes of enumeration {MBReqTypes_Reads, MBReqTypes_Writes, MBReqTypes_All};
```

Library pseudocode for shared/functions/memory/MemAtomicOp

```
// MemAtomicOp
// ===========
// Atomic data processing instruction types.

type MemAtomicOp of enumeration {
    MemAtomicOp_GCSSS1,
    MemAtomicOp_ADD,
    MemAtomicOp_BIC,
    MemAtomicOp_EOR,
    MemAtomicOp_ORR,
    MemAtomicOp_SMAX,
    MemAtomicOp_SMIN,
    MemAtomicOp_UMAX,
    MemAtomicOp_UMIN,
    MemAtomicOp_SWP,
    MemAtomicOp_CAS,
    MemAtomicOp_FPADD,
    MemAtomicOp_FPMAX,
    MemAtomicOp_FPMIN,
    MemAtomicOp_FPMAXNM,
    MemAtomicOp_FPMINNM,
    MemAtomicOp_BFADD,
    MemAtomicOp_BFMAX,
    MemAtomicOp_BFMIN,
    MemAtomicOp_BFMAXNM,
    MemAtomicOp_BFMINNM
};
```

Library pseudocode for shared/functions/memory/MemAttrHints

```
// MemAttrHints
// ============
// Attributes and hints for Normal memory.

type MemAttrHints of record {
    attrs : bits(2),  // See MemAttr_*, Cacheability attributes
    hints : bits(2),  // See MemHint_*, Allocation hints
    transient : boolean
};
```

Library pseudocode for shared/functions/memory/MemOp

```
// MemOp
// =====
// Memory access instruction types.

type MemOp of enumeration {MemOp_LOAD, MemOp_STORE, MemOp_PREFETCH};
```

Library pseudocode for shared/functions/memory/MemType

```
// MemType
// =======
// Basic memory types.

type MemType of enumeration {MemType_Normal, MemType_Device};
```

Library pseudocode for shared/functions/memory/Memory

```
// Memory Tag type
// ===============

type MemTagType of enumeration {
    MemTag_AllocationTagged,
    MemTag_CanonicallyTagged,
    MemTag_Untagged
};
```

Library pseudocode for shared/functions/memory/MemoryAttributes

```
// MemoryAttributes
// ================
// Memory attributes descriptor

type MemoryAttributes of record {
    memtype : MemType,
    device : DeviceType,         // For Device memory types
    inner : MemAttrHints,        // Inner hints and attributes
    outer : MemAttrHints,        // Outer hints and attributes
    shareability : Shareability, // Shareability attribute
    tags : MemTagType,           // MTE tag type for this memory.
    notagaccess : boolean,       // Allocation Tag access permission
    xs : bit                     // XS attribute
};
```

Library pseudocode for shared/functions/memory/NewAccDesc

```
// NewAccDesc()
// ============
// Create a new AccessDescriptor with initialized fields

func NewAccDesc(acctype : AccessType) => AccessDescriptor
begin
    let mpaminfo : MPAMinfo = GenMPAMCurEL(acctype);
    return NewAccDesc(acctype, mpaminfo);
end;

func NewAccDesc(acctype : AccessType, mpam : MPAMinfo) => AccessDescriptor
begin
    var accdesc : AccessDescriptor;

    accdesc.acctype             = acctype;
    accdesc.el                  = PSTATE.EL;
    accdesc.ss                  = SecurityStateAtEL(PSTATE.EL);
    accdesc.acqsc               = FALSE;
    accdesc.acqpc               = FALSE;
    accdesc.relsc               = FALSE;
    accdesc.limitedordered      = FALSE;
    accdesc.exclusive           = FALSE;
    accdesc.rcw                 = FALSE;
    accdesc.rcws                = FALSE;
    accdesc.atomicop            = FALSE;
    accdesc.nontemporal         = FALSE;
    accdesc.read                = FALSE;
    accdesc.write               = FALSE;
    accdesc.pan                 = FALSE;
    accdesc.nonfault            = FALSE;
    accdesc.firstfault          = FALSE;
    accdesc.first               = FALSE;
    accdesc.contiguous          = FALSE;
    accdesc.predicated          = FALSE;
    accdesc.streamingsve        = FALSE;
    accdesc.ls64                = FALSE;
    accdesc.withstatus          = FALSE;
    accdesc.mops                = FALSE;
    accdesc.a32lsmd             = FALSE;
    accdesc.tagchecked          = FALSE;
    accdesc.tagaccess           = FALSE;
    accdesc.stzgm               = FALSE;
    accdesc.mpam                = mpam;
    accdesc.Rs                  = -1;
    accdesc.Rs2                 = -1;
    accdesc.Rt                  = -1;
    accdesc.Rt2                 = -1;
    accdesc.ispair              = FALSE;
    accdesc.highestaddressfirst = FALSE;
    accdesc.lowestaddress       = TRUE;

    return accdesc;
end;
```

Library pseudocode for shared/functions/memory/PASpace

```
// PASpace
// =======
// Physical address spaces

type PASpace of enumeration {
    PAS_Root,
    PAS_SystemAgent,
    PAS_NonSecureProtected,
    PAS_NA6,                // Reserved
    PAS_NA7,                // Reserved
    PAS_Realm,
    PAS_Secure,
    PAS_NonSecure
};
```

Library pseudocode for shared/functions/memory/Permissions

```
// Permissions
// ===========
// Access Control bits in translation table descriptors

type Permissions of record {
    ap_table  : bits(2),  // Stage 1 hierarchical access permissions
    xn_table  : bit,      // Stage 1 hierarchical execute-never for single EL regimes
    pxn_table : bit,      // Stage 1 hierarchical privileged execute-never
    uxn_table : bit,      // Stage 1 hierarchical unprivileged execute-never
    ap  : bits(3),        // Stage 1 access permissions
    xn  : bit,            // Stage 1 execute-never for single EL regimes
    uxn : bit,            // Stage 1 unprivileged execute-never
    pxn : bit,            // Stage 1 privileged execute-never
    ppi : bits(4),        // Stage 1 privileged indirect permissions
    upi : bits(4),        // Stage 1 unprivileged indirect permissions
    ndirty : bit,         // Stage 1 dirty state for indirect permissions scheme
    s2pi    : bits(4),    // Stage 2 indirect permissions
    s2dirty : bit,        // Stage 2 dirty state
    po_index : bits(4),   // Stage 1 overlay permissions index
    s2po_index : bits(4), // Stage 2 overlay permissions index
    s2ap : bits(2),       // Stage 2 access permissions
    s2tag_na : bit,       // Stage 2 tag access
    s2xnx : bit,          // Stage 2 extended execute-never
    dbm : bit,            // Dirty bit management
    s2xn : bit            // Stage 2 execute-never
};
```

Library pseudocode for shared/functions/memory/PhysMemRead

```
// PhysMemRead()
// =============
// Returns the value read from memory, and a status.
// Returned value is UNKNOWN if an External abort occurred while reading the
// memory.
// Otherwise the PhysMemRetStatus statuscode is Fault_None.

impdef func PhysMemRead{size : integer{8, 16, 32, 64, 128, 256, 512}}(
                                                desc : AddressDescriptor,
                                                accdesc : AccessDescriptor
                                            ) => (PhysMemRetStatus, bits(size))
begin
    return (ARBITRARY : PhysMemRetStatus, ARBITRARY : bits(size));
end;
```

Library pseudocode for shared/functions/memory/PhysMemRetStatus

```
// PhysMemRetStatus
// ================
// Fields that relate only to return values of PhysMem functions.

type PhysMemRetStatus of record {
    statuscode : Fault,       // Fault Status
    extflag : bit,            // IMPLEMENTATION DEFINED syndrome for External aborts
    merrorstate : ErrorState, // Optional error state returned on a physical memory access
    store64bstatus : bits(64) // Status of 64B store
};
```

Library pseudocode for shared/functions/memory/PhysMemWrite

```
// PhysMemWrite()
// ==============
// Writes the value to memory, and returns the status of the write.
// If there is an External abort on the write, the PhysMemRetStatus indicates this.
// Otherwise the statuscode of PhysMemRetStatus is Fault_None.

impdef func PhysMemWrite{size : integer{8, 16, 32, 64, 128, 256, 512}}(desc : AddressDescriptor,
                                                                       accdesc : AccessDescriptor,
                                                  value : bits(size)) => PhysMemRetStatus
begin
    return ARBITRARY : PhysMemRetStatus;
end;
```

Library pseudocode for shared/functions/memory/PrefetchHint

```
// PrefetchHint
// ============
// Prefetch hint types.

type PrefetchHint of enumeration {
    Prefetch_READ,
    Prefetch_WRITE,
    Prefetch_IR,
    Prefetch_EXEC
};
```

Library pseudocode for shared/functions/memory/S1AccessControls

```
// S1AccessControls
// ================
// Effective access controls defined by stage 1 translation

type S1AccessControls of record {
    r : bit,                // Stage 1 base read permission
    w : bit,                // Stage 1 base write permission
    x : bit,                // Stage 1 base execute permission
    gcs : bit,              // Stage 1 GCS permission
    overlay : boolean,      // Stage 1 FEAT_S1POE overlay applies
    or : bit,               // Stage 1 FEAT_S1POE overlay read permission
    ow : bit,               // Stage 1 FEAT_S1POE overlay write permission
    ox : bit,               // Stage 1 FEAT_S1POE overlay execute permission
    wxn : bit               // Stage 1 write permission implies execute-never

};
```

Library pseudocode for shared/functions/memory/S2AccessControls

```
// S2AccessControls
// ================
// Effective access controls defined by stage 2 translation

type S2AccessControls of record {
    r : bit,                // Stage 2 read permission.
    w : bit,                // Stage 2 write permission.
    x : bit,                // Stage 2 execute permission.
    r_rcw : bit,            // Stage 2 Read perms for RCW instruction.
    w_rcw : bit,            // Stage 2 Write perms for RCW instruction.
    r_mmu : bit,            // Stage 2 Read perms for TTW data.
    w_mmu : bit,            // Stage 2 Write perms for TTW data.
    toplevel0 : bit,        // IPA as top level table for TTBR0_EL1.
    toplevel1 : bit,        // IPA as top level table for TTBR1_EL1.
    overlay : boolean,      // Overlay enable
    or : bit,               // Stage 2 overlay read permission.
    ow : bit,               // Stage 2 overlay write permission.
    ox : bit,               // Stage 2 overlay execute permission.
    or_rcw : bit,           // Stage 2 overlay Read perms for RCW instruction.
    ow_rcw : bit,           // Stage 2 overlay Write perms for RCW instruction.
    or_mmu : bit,           // Stage 2 overlay Read perms for TTW data.
    ow_mmu : bit,           // Stage 2 overlay Write perms for TTW data.
};
```

Library pseudocode for shared/functions/memory/Shareability

```
// Shareability
// ============

type Shareability of enumeration {
    Shareability_NSH,
    Shareability_ISH,
    Shareability_OSH
};
```

Library pseudocode for shared/functions/memory/SpeculativeStoreBypassBarrierToPA

```
// SpeculativeStoreBypassBarrierToPA()
// ===================================

impdef func SpeculativeStoreBypassBarrierToPA()
begin
    // Since there is no speculation in ASL model, this is a NOP.
    return;
end;
```

Library pseudocode for shared/functions/memory/SpeculativeStoreBypassBarrierToVA

```
// SpeculativeStoreBypassBarrierToVA()
// ===================================

impdef func SpeculativeStoreBypassBarrierToVA()
begin
    // Since there is no speculation in ASL model, this is a NOP.
    return;
end;
```

Library pseudocode for shared/functions/memory/Tag

```
// Tag Granule size
// ================

constant LOG2_TAG_GRANULE : integer{} = 4;
constant TAG_GRANULE : integer{} = 1 << LOG2_TAG_GRANULE;
```

Library pseudocode for shared/functions/memory/VARange

```
// VARange
// =======
// Virtual address ranges

type VARange of enumeration {
    VARange_LOWER,
    VARange_UPPER
};
```

Library pseudocode for shared/functions/mpam/AltPARTIDSpace

```
// AltPARTIDSpace()
// ================
// From the Security state, EL and ALTSP configuration, determine
// whether to primary space or the alt space is selected and which
// PARTID space is the alternative space. Return that alternative
// PARTID space if selected or the primary space if not.

func AltPARTIDSpace(el : bits(2), security : SecurityState,
                    primaryPIDSpace : PARTIDSpaceType) => PARTIDSpaceType
begin
    case security of
        when SS_NonSecure =>
            assert el != EL3;
            return primaryPIDSpace;
        when SS_Secure =>
            assert el != EL3;
            if primaryPIDSpace == PIDSpace_NonSecure then
                return primaryPIDSpace;
            end;
            return AltPIDSecure(el, primaryPIDSpace);
        when SS_Root =>
            assert el == EL3;
            if MPAM3_EL3().ALTSP_EL3 == '1' then
                if MPAM3_EL3().RT_ALTSP_NS == '1' then
                    return PIDSpace_NonSecure;
                else
                    return PIDSpace_Secure;
                end;
            else
                return primaryPIDSpace;
            end;
        when SS_Realm =>
            assert el != EL3;
            return AltPIDRealm(el, primaryPIDSpace);
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/functions/mpam/AltPIDRealm

```
// AltPIDRealm()
// =============
// Compute PARTID space as either the primary PARTID space or
// alternative PARTID space in the Realm Security state.
// Helper for AltPARTIDSpace.

func AltPIDRealm(el : bits(2), primaryPIDSpace : PARTIDSpaceType) => PARTIDSpaceType
begin
    var PIDSpace : PARTIDSpaceType = primaryPIDSpace;
    case el of
        when EL0 =>
            if ELIsInHost(EL0) then
                if !UsePrimarySpaceEL2() then
                    PIDSpace = PIDSpace_NonSecure;
                end;
            elsif !UsePrimarySpaceEL10() then
                PIDSpace = PIDSpace_NonSecure;
            end;
        when EL1 =>
            if !UsePrimarySpaceEL10() then
                PIDSpace = PIDSpace_NonSecure;
            end;
        when EL2 =>
            if !UsePrimarySpaceEL2() then
                PIDSpace = PIDSpace_NonSecure;
            end;
        otherwise =>
            unreachable;
    end;
    return PIDSpace;
end;
```

Library pseudocode for shared/functions/mpam/AltPIDSecure

```
// AltPIDSecure()
// ==============
// Compute PARTID space as either the primary PARTID space or
// alternative PARTID space in the Secure Security state.
// Helper for AltPARTIDSpace.

func AltPIDSecure(el : bits(2), primaryPIDSpace : PARTIDSpaceType) => PARTIDSpaceType
begin
    var PIDSpace : PARTIDSpaceType = primaryPIDSpace;
    case el of
        when EL0 =>
            if EL2Enabled() then
                if ELIsInHost(EL0) then
                    if !UsePrimarySpaceEL2() then
                        PIDSpace = PIDSpace_NonSecure;
                    end;
                elsif !UsePrimarySpaceEL10() then
                    PIDSpace = PIDSpace_NonSecure;
                end;
            elsif MPAM3_EL3().ALTSP_HEN == '0' && MPAM3_EL3().ALTSP_HFC == '1' then
                PIDSpace = PIDSpace_NonSecure;
            end;
        when EL1 =>
            if EL2Enabled() then
                if !UsePrimarySpaceEL10() then
                    PIDSpace = PIDSpace_NonSecure;
                end;
            elsif MPAM3_EL3().ALTSP_HEN == '0' && MPAM3_EL3().ALTSP_HFC == '1' then
                PIDSpace = PIDSpace_NonSecure;
            end;
        when EL2 =>
            if !UsePrimarySpaceEL2() then
                PIDSpace = PIDSpace_NonSecure;
            end;
        otherwise =>
            unreachable;
    end;
    return PIDSpace;
end;
```

Library pseudocode for shared/functions/mpam/DefaultMPAMInfo

```
// DefaultMPAMInfo()
// =================
// Returns default MPAM info.

func DefaultMPAMInfo() => MPAMinfo
begin
    assert IsFeatureImplemented(FEAT_MPAMv2);

    var defaultinfo : MPAMinfo;
    defaultinfo.partid = DEFAULT_PARTID;
    defaultinfo.pmg    = DEFAULT_PMG;
    return defaultinfo;
end;

// DefaultMPAMInfo()
// =================
// Returns default MPAM info.  The partidspace argument sets
// the PARTID space of the default MPAM information returned.

func DefaultMPAMInfo(partidspace : PARTIDSpaceType) => MPAMinfo
begin
    assert !IsFeatureImplemented(FEAT_MPAMv2);

    var defaultinfo : MPAMinfo;
    defaultinfo.mpam_sp = partidspace;
    defaultinfo.partid  = DEFAULT_PARTID;
    defaultinfo.pmg     = DEFAULT_PMG;
    return defaultinfo;
end;
```

Library pseudocode for shared/functions/mpam/GenMPAM

```
// GenMPAM()
// =========
// Returns MPAMinfo for Exception level el.
// If mpamdata.sm is TRUE returns MPAM information using MPAMSM_EL1.
// If mpamdata.ind is TRUE and alternative ID is enabled returns MPAM information
// using MPAMn_ELx.[altPARTID, altPMG].
// Otherwise returns MPAM information using MPAMn_ELx.

func GenMPAM(el_in : bits(2), mpamdata : MPAMdata) => MPAMinfo
begin
    var returninfo : MPAMinfo;
    var partidel : PARTIDType;
    var perr : boolean;
    var el : bits(2) = el_in;

    // Check if the guest OS application is locked by the EL2 hypervisor to
    // only use the EL1 virtual machine's PARTIDs.
    if (!IsFeatureImplemented(FEAT_MPAMv2) && el == EL0 && EL2Enabled() &&
          MPAMHCR_EL2().GSTAPP_PLK == '1' && HCR_EL2().TGE == '0') then
        el = EL1;
    end;

    (partidel, perr)      = GenPARTID(el, mpamdata);
    let groupel : PMGType = GenPMG(el, mpamdata, perr);

    returninfo.partid  = partidel;
    returninfo.pmg     = groupel;
    return returninfo;
end;
```

Library pseudocode for shared/functions/mpam/GenMPAMAtEL

```
// GenMPAMAtEL()
// =============
// Returns MPAMinfo for the specified EL.
// May be called if MPAM is not implemented (but in an version that supports
// MPAM), MPAM is disabled, or in AArch32_  In AArch32, convert the mode to
// EL if can and use that to drive MPAM information generation.  If mode
// cannot be converted, MPAM is not implemented, or MPAM is disabled return
// default MPAM information for the current security state.

func GenMPAMAtEL(acctype : AccessType, el : bits(2)) => MPAMinfo
begin
    let mpamEL : bits(2) = if acctype == AccessType_NV2 then EL2 else el;

    var pspace : PARTIDSpaceType;
    let security : SecurityState = SecurityStateAtEL(el);
    var mpamdata : MPAMdata      = GenNewMPAMData();
    if !IsFeatureImplemented(FEAT_MPAMv2) then
        pspace = PARTIDSpaceFromSS(security);
        if IsFeatureImplemented(FEAT_MPAMv0p1) && MPAMIDR_EL1().HAS_FORCE_NS == '1' then
            if MPAM3_EL3().FORCE_NS == '1' && security == SS_Secure then
                pspace = PIDSpace_NonSecure;
            end;
        end;

        if IsFeatureImplemented(FEAT_RME) && MPAMIDR_EL1().HAS_ALTSP == '1' then
            // Substitute alternative PARTID space if selected
            pspace = AltPARTIDSpace(mpamEL, security, pspace);
        end;
        if !MPAMIsEnabled() then
            return DefaultMPAMInfo(pspace);
        end;
    else
        if !MPAMIsEnabled() then
            return DefaultMPAMInfo();
        end;
    end;

    case acctype of
        when AccessType_IFETCH, AccessType_IC =>
            mpamdata.ind = TRUE;
        when AccessType_SME =>
            mpamdata.sm = (ImpDefBool("Shared SMCU") ||
                           ImpDefBool("MPAMSM_EL1 label precedence"));
        when AccessType_FP, AccessType_ASIMD, AccessType_SVE =>
            mpamdata.sm = (IsFeatureImplemented(FEAT_SME) && PSTATE.SM == '1' &&
                           (ImpDefBool("Shared SMCU") ||
                            ImpDefBool("MPAMSM_EL1 label precedence")));
        when AccessType_TRBE =>
            // GenMPAMTRBE() is called when the Trace Buffer Unit is using External mode.
            assert SelfHostedTraceEnabled();
        otherwise =>
            // Other access types are DATA accesses
            mpamdata.ind = FALSE;
    end;

    if ((IsFeatureImplemented(FEAT_MPAMv0p1) || IsFeatureImplemented(FEAT_MPAMv1p1)) &&
          MPAMIDR_EL1().HAS_SDEFLT == '1') then
        if MPAM3_EL3().SDEFLT == '1' && security == SS_Secure then
            return DefaultMPAMInfo(pspace);
        end;
    end;
    var mpaminfo : MPAMinfo = GenMPAM(mpamEL, mpamdata);
    if !IsFeatureImplemented(FEAT_MPAMv2) then
        mpaminfo.mpam_sp = pspace;
    end;

    return mpaminfo;
end;
```

Library pseudocode for shared/functions/mpam/GenMPAMCurEL

```
// GenMPAMCurEL()
// ==============
// Returns MPAMinfo for the current EL and security state.
// May be called if MPAM is not implemented (but in an version that supports
// MPAM), MPAM is disabled, or in AArch32_  In AArch32, convert the mode to
// EL if can and use that to drive MPAM information generation.  If mode
// cannot be converted, MPAM is not implemented, or MPAM is disabled return
// default MPAM information for the current security state.

func GenMPAMCurEL(acctype : AccessType) => MPAMinfo
begin
    return GenMPAMAtEL(acctype, PSTATE.EL);
end;
```

Library pseudocode for shared/functions/mpam/GenMPAMTRBE

```
// GenMPAMTRBE()
// =============
// Returns MPAMinfo for Trace Buffer Unit in external mode.

func GenMPAMTRBE() => MPAMinfo
begin
    assert !SelfHostedTraceEnabled();
    var pspace : PARTIDSpaceType;
    if IsFeatureImplemented(FEAT_MPAMv2) then
        if !IsFeatureImplemented(FEAT_TRBE_MPAM) || TRBMPAM_EL1().EN == '0' then
            return DefaultMPAMInfo();
        end;
    else
        case TRBMAR_EL1().PAS of
            when '00' => pspace = PIDSpace_Secure;
            when '01' => pspace = PIDSpace_NonSecure;
            when '10' => pspace = PIDSpace_Root;
            when '11' => pspace = PIDSpace_Realm;
        end;

        if !IsFeatureImplemented(FEAT_TRBE_MPAM) || TRBMPAM_EL1().EN == '0' then
            return DefaultMPAMInfo(pspace);
        end;
    end;

    assert IsFeatureImplemented(FEAT_TRBE_MPAM);

    var mpaminfo : MPAMinfo;
    if !IsFeatureImplemented(FEAT_MPAMv2) then mpaminfo.mpam_sp = pspace; end;
    mpaminfo.partid = (if IsFeatureImplemented(FEAT_MPAMv2) then
                           TRBMPAM_EL1().MPAMv2.PARTID
                       else
                           TRBMPAM_EL1().MPAMv0or1px.PARTID);
    var partid_out_of_range : boolean = FALSE;

    // When the PARTID is out-of-range, the behavior is a CONSTRAINED UNPREDICTABLE choice of:
    //     - Replace the out-of-range PARTID with any in-range PARTID
    //     - Replace the out-of-range PARTID with the default PARTID
    let partid_max : integer = GetMPAM_PARTID_MAX();
    if UInt(mpaminfo.partid) > partid_max then
        partid_out_of_range = TRUE;
        var c : Constraint;
        var partid : integer;
        (c, partid) = ConstrainUnpredictableInteger(0, partid_max,
                                                    Unpredictable_OUT_OF_RANGE_ID);
        assert c IN {Constraint_DEFAULT_ID, Constraint_ANY_ID};
        case c of
            when Constraint_DEFAULT_ID => mpaminfo.partid = DEFAULT_PARTID;
            when Constraint_ANY_ID     => mpaminfo.partid = partid[15:0];
        end;
    end;

    if partid_out_of_range && ConstrainUnpredictableBool(Unpredictable_USE_DEFAULT_PMG) then
        mpaminfo.pmg = DEFAULT_PMG;
    else
        mpaminfo.pmg = (if IsFeatureImplemented(FEAT_MPAMv2) then
                            TRBMPAM_EL1().MPAMv2.PMG
                        else
                            ZeroExtend{16}(TRBMPAM_EL1().MPAMv0or1px.PMG));
    end;

    let pmg_max : integer = GetMPAM_PMG_MAX();
    if UInt(mpaminfo.pmg) > pmg_max then
        var c : Constraint;
        var pmg : integer;
        (c, pmg) = ConstrainUnpredictableInteger(0, pmg_max,
                                                 Unpredictable_OUT_OF_RANGE_ID);
        assert c IN {Constraint_DEFAULT_ID, Constraint_ANY_ID};
        case c of
            when Constraint_DEFAULT_ID => mpaminfo.pmg = DEFAULT_PMG;
            when Constraint_ANY_ID     => mpaminfo.pmg = pmg[15:0];
        end;
    end;

    return mpaminfo;
end;
```

Library pseudocode for shared/functions/mpam/GenNewMPAMData

```
// GenNewMPAMData()
// ================

func GenNewMPAMData() => MPAMdata
begin
    var mpamdata : MPAMdata;
    mpamdata.ind = FALSE;
    mpamdata.sm = FALSE;
    return mpamdata;
end;
```

Library pseudocode for shared/functions/mpam/GenPARTID

```
// GenPARTID()
// ===========
// Returns physical PARTID and error boolean for Exception level el.
// If mpamdata.sm is TRUE then PARTID is from MPAMSM_EL1.
// If mpamdata.ind is TRUE and alternative ID is enabled then PARTID is from MPAMn_ELx.altPARTID.
// Otherwise, the PARTID is from MPAMn_ELx.

func GenPARTID(el : bits(2), mpamdata : MPAMdata) => (PARTIDType, boolean)
begin
    var partidel : PARTIDType = GetMPAM_PARTID(el, mpamdata);
    var partid_err : boolean  = FALSE;

    // When the PARTID is out-of-range, the behavior is a CONSTRAINED UNPREDICTABLE choice of:
    //     - Replace the out-of-range PARTID with any in-range PARTID
    //     - Replace the out-of-range PARTID with the default PARTID
    if UInt(partidel) > GetMPAM_PARTID_MAX() then
        var c : Constraint;
        var partid : integer;
        partid_err = TRUE;
        (c, partid) = ConstrainUnpredictableInteger(0, GetMPAM_PARTID_MAX(),
                                                    Unpredictable_OUT_OF_RANGE_ID);
        assert c IN {Constraint_DEFAULT_ID, Constraint_ANY_ID};
        case c of
            when Constraint_DEFAULT_ID => partidel = DEFAULT_PARTID;
            when Constraint_ANY_ID     => partidel = partid[15:0];
        end;
    end;

    if MPAMIsVirtual(el, mpamdata) then
        return MAP_vPARTID(partidel, partid_err);
    else
        return (partidel, partid_err);
    end;
end;
```

Library pseudocode for shared/functions/mpam/GenPMG

```
// GenPMG()
// ========
// Returns PMG for Exception level el.
// if mpamdata.sm is TRUE then PMG is from MPAMSM_EL1.
// If mpamdata.ind is TRUE and alternative id is enabled, then the PMG is from MPAMn_ELx.altPMG.
// Otherwise, PMG is from MPAMn_ELx.
// If PARTID generation (GenPARTID) encountered an error, GenPMG() should be
// called with partid_err as TRUE.

func GenPMG(el : bits(2), mpamdata : MPAMdata, partid_err : boolean) => PMGType
begin
    if (!IsFeatureImplemented(FEAT_MPAMv2) && partid_err &&
          ConstrainUnpredictableBool(Unpredictable_USE_DEFAULT_PMG)) then
        return DEFAULT_PMG;
    end;

    let groupel : PMGType = GetMPAM_PMG(el, mpamdata);
    let pmg_max : integer = GetMPAM_PMG_MAX();
    if UInt(groupel) <= pmg_max then
        return groupel;
    end;
    return DEFAULT_PMG;
end;
```

Library pseudocode for shared/functions/mpam/GetMPAM_PARTID

```
// GetMPAM_PARTID()
// ================
// Returns a PARTID
// If mpamdata.sm is TRUE then PARTID is from MPAMSM_EL1.
// If mpamdata.ind is TRUE and alternative ID is enabled then PARTID is from MPAMn_ELx.altPARTID.
// Otherwise, the PARTID is from MPAMn_ELx.

func GetMPAM_PARTID(MPAMn : bits(2), mpamdata : MPAMdata) => PARTIDType
begin
    if mpamdata.sm then
        return (if IsFeatureImplemented(FEAT_MPAMv2) then MPAMSM_EL1().PARTID
                                                     else MPAMSM_EL1().PARTID_D);
    end;

    if (IsFeatureImplemented(FEAT_MPAMv2_ALT_ID) && mpamdata.ind &&
          MPAMIDR_EL1().HAS_INSTR_ALT_ID == '1') then
        case MPAMn of
            when '11' =>
                if MPAMCTL_EL3().EN_ALT_IPARTID == '1' then
                    return MPAM3_EL3().altPARTID;
                else
                    return MPAM3_EL3().PARTID;
                end;
            when '10' =>
                if !EL2Enabled() then
                    return DEFAULT_PARTID;
                elsif MPAMCTL_EL2().EN_ALT_IPARTID == '1' then
                    return MPAM2_EL2().altPARTID;
                else
                    return MPAM2_EL2().PARTID;
                end;
            when '01' =>
                if MPAMCTL_EL1().EN_ALT_IPARTID == '1' then
                    return MPAM1_EL1().altPARTID;
                else
                    return MPAM1_EL1().PARTID;
                end;
            when '00' =>
                if ((ELIsInHost(EL0) && MPAMCTL_EL2().EN_ALT_IPARTID_EL0 == '1') ||
                      (!ELIsInHost(EL0) && MPAMCTL_EL1().EN_ALT_IPARTID_EL0 == '1')) then
                    return MPAM0_EL1().altPARTID;
                else
                    return MPAM0_EL1().PARTID;
                end;
        end;
    end;

    if IsFeatureImplemented(FEAT_MPAMv2) then
        case MPAMn of
            when '11' => return MPAM3_EL3().PARTID;
            when '10' => return (if EL2Enabled() then MPAM2_EL2().PARTID else DEFAULT_PARTID);
            when '01' => return MPAM1_EL1().PARTID;
            when '00' => return MPAM0_EL1().PARTID;
        end;
    end;

    if mpamdata.ind then
        case MPAMn of
            when '11' => return MPAM3_EL3().PARTID_I;
            when '10' => return (if EL2Enabled() then MPAM2_EL2().PARTID_I else DEFAULT_PARTID);
            when '01' => return MPAM1_EL1().PARTID_I;
            when '00' => return MPAM0_EL1().PARTID_I;
        end;
    else
        case MPAMn of
            when '11' => return MPAM3_EL3().PARTID_D;
            when '10' => return (if EL2Enabled() then MPAM2_EL2().PARTID_D else DEFAULT_PARTID);
            when '01' => return MPAM1_EL1().PARTID_D;
            when '00' => return MPAM0_EL1().PARTID_D;
        end;
    end;
end;
```

Library pseudocode for shared/functions/mpam/GetMPAM_PARTID_MAX

```
// GetMPAM_PARTID_MAX()
// ====================
// Returns the maximum supported PARTID value for the implemented MPAM version.

func GetMPAM_PARTID_MAX() => integer
begin
    if IsFeatureImplemented(FEAT_MPAMv2) then
        return UInt(MPAMIDR_EL1().MPAMv2.PARTID_MAX);
    end;
    return UInt(MPAMIDR_EL1().MPAMv0or1px.PARTID_MAX);
end;
```

Library pseudocode for shared/functions/mpam/GetMPAM_PMG

```
// GetMPAM_PMG()
// =============
// Returns a PMG.
// if mpamdata.sm is TRUE then PMG is from MPAMSM_EL1.
// If mpamdata.ind is TRUE and alternative id is enabled, then the PMG is from MPAMn_ELx.altPMG.
// Otherwise, PMG is from MPAMn_ELx.

func GetMPAM_PMG(MPAMn : bits(2), mpamdata : MPAMdata) => PMGType
begin
    if mpamdata.sm then
        return (if IsFeatureImplemented(FEAT_MPAMv2) then MPAMSM_EL1().PMG
                                                     else ZeroExtend{16}(MPAMSM_EL1().PMG_D));
    end;

    if (IsFeatureImplemented(FEAT_MPAMv2_ALT_ID) && mpamdata.ind &&
          MPAMIDR_EL1().HAS_INSTR_ALT_ID == '1') then
        case MPAMn of
            when '11' =>
                if MPAMCTL_EL3().EN_ALT_IPMG == '1' then
                    return MPAM3_EL3().altPMG;
                else
                    return MPAM3_EL3().PMG;
                end;
            when '10' =>
                if !EL2Enabled() then
                    return DEFAULT_PMG;
                elsif MPAMCTL_EL2().EN_ALT_IPMG == '1' then
                    return MPAM2_EL2().altPMG;
                else
                    return MPAM2_EL2().PMG;
                end;
            when '01' =>
                if MPAMCTL_EL1().EN_ALT_IPMG == '1' then
                    return MPAM1_EL1().altPMG;
                else
                    return MPAM1_EL1().PMG;
                end;
            when '00' =>
                if ((ELIsInHost(EL0) && MPAMCTL_EL2().EN_ALT_IPMG_EL0 == '1') ||
                      (!ELIsInHost(EL0) && MPAMCTL_EL1().EN_ALT_IPMG_EL0 == '1')) then
                    return MPAM0_EL1().altPMG;
                else
                    return MPAM0_EL1().PMG;
                end;
        end;
    end;

    if IsFeatureImplemented(FEAT_MPAMv2) then
        case MPAMn of
            when '11' => return MPAM3_EL3().PMG;
            when '10' => return (if EL2Enabled() then MPAM2_EL2().PMG else DEFAULT_PMG);
            when '01' => return MPAM1_EL1().PMG;
            when '00' => return MPAM0_EL1().PMG;
        end;
    end;

    if mpamdata.ind then
        case MPAMn of
            when '11'=>
                return ZeroExtend{16}(MPAM3_EL3().PMG_I);
            when '10' =>
                return (if EL2Enabled() then ZeroExtend{16}(MPAM2_EL2().PMG_I) else DEFAULT_PMG);
            when '01' =>
                return ZeroExtend{16}(MPAM1_EL1().PMG_I);
            when '00' =>
                return ZeroExtend{16}(MPAM0_EL1().PMG_I);
        end;
    else
        case MPAMn of
            when '11' =>
                return ZeroExtend{16}(MPAM3_EL3().PMG_D);
            when '10' =>
                return (if EL2Enabled() then ZeroExtend{16}(MPAM2_EL2().PMG_D) else DEFAULT_PMG);
            when '01' =>
                return ZeroExtend{16}(MPAM1_EL1().PMG_D);
            when '00' =>
                return ZeroExtend{16}(MPAM0_EL1().PMG_D);
        end;
    end;
end;
```

Library pseudocode for shared/functions/mpam/GetMPAM_PMG_MAX

```
// GetMPAM_PMG_MAX()
// =================
// Returns the maximum supported PMG value for the implemented MPAM version.

func GetMPAM_PMG_MAX() => integer
begin
    if IsFeatureImplemented(FEAT_MPAMv2) then
        return UInt(MPAMIDR_EL1().MPAMv2.PMG_MAX);
    end;
    return UInt(MPAMIDR_EL1().MPAMv0or1px.PMG_MAX);
end;
```

Library pseudocode for shared/functions/mpam/MAP_vPARTID

```
// MAP_vPARTID()
// =============
// Performs conversion of virtual PARTID into physical PARTID
// Contains all of the error checking and implementation
// choices for the conversion.

func MAP_vPARTID(vpartid : PARTIDType, partid_err : boolean) => (PARTIDType, boolean)
begin
    // should not ever be called if EL2 is not implemented
    // or is implemented but not enabled in the current
    // security state.
    var ret : PARTIDType;
    var err : boolean  = partid_err;
    var virt : integer = UInt(vpartid);

    // The largest vpartid supported
    let vpartid_max : integer = (UInt(MPAMIDR_EL1().VPMR_MAX) << 2) + 3;

    if virt > vpartid_max then
        let (c, vid) = ConstrainUnpredictableInteger(0, vpartid_max,
                                                     Unpredictable_OUT_OF_RANGE_VID);
        assert c IN {Constraint_DEFAULT_ID, Constraint_ANY_ID};
        case c of
            when Constraint_DEFAULT_ID => virt = UInt(DEFAULT_PARTID);
            when Constraint_ANY_ID     => virt = vid;
        end;
    end;

    // Check for valid mapping entry.
    if MPAMVPMV_EL2()[virt] == '1' then
        // vpartid has a valid mapping, access the map.
        ret = mapvpmw(virt);

    // Check for default virtual PARTID valid
    elsif MPAMVPMV_EL2()[0] == '1' then
        // Use default mapping for vpartid == 0.
        ret = MPAMVPM0_EL2()[0 +: 16];

    // Neither is valid, use default physical PARTID.
    else
        ret = DEFAULT_PARTID;
        err = TRUE;
    end;

    // Check that the physical PARTID is in-range.
    // This physical PARTID came from a virtual mapping entry.
    let partid_max : integer = GetMPAM_PARTID_MAX();
    if UInt(ret) > partid_max then
        // Out of range, so return default physical PARTID
        ret = DEFAULT_PARTID;
        err = TRUE;
    end;
    return (ret, err);
end;
```

Library pseudocode for shared/functions/mpam/MPAM

```
// MPAM Types
// ==========

type PARTIDType of bits(16);
type PMGType of bits(16);

type PARTIDSpaceType of enumeration {
    PIDSpace_Secure,
    PIDSpace_Root,
    PIDSpace_Realm,
    PIDSpace_NonSecure
};

type MPAMinfo of record {
     mpam_sp : PARTIDSpaceType,
     partid : PARTIDType,
     pmg : PMGType
};
constant DEFAULT_PARTID : PARTIDType = 0[15:0];
constant DEFAULT_PMG : PMGType = 0[15:0];

type MPAMdata of record {
    sm : boolean,         // TRUE for SME, SVE, SIMD&FP access, and SVE prefetch
                          // instructions, when the PE is in Streaming mode
    ind : boolean         // Instruction not data, TRUE for instruction accesses
};
```

Library pseudocode for shared/functions/mpam/MPAMIsEnabled

```
// MPAMIsEnabled()
// ===============
// Returns TRUE if MPAM is enabled.

func MPAMIsEnabled() => boolean
begin
    let el : bits(2) = HighestEL();
    if IsFeatureImplemented(FEAT_MPAMv2) then
        case el of
            when EL3 => return MPAMCTL_EL3().MPAMEN == '1';
            when EL2 => return MPAMCTL_EL2().MPAMEN == '1';
            when EL1 => return MPAMCTL_EL1().MPAMEN == '1';
        end;
    end;

    case el of
        when EL3 => return MPAM3_EL3().MPAMEN == '1';
        when EL2 => return MPAM2_EL2().MPAMEN == '1';
        when EL1 => return MPAM1_EL1().MPAMEN == '1';
    end;
end;
```

Library pseudocode for shared/functions/mpam/MPAMIsVirtual

```
// MPAMIsVirtual()
// ===============
// If FEAT_MPAMv2 is not implemented, returns TRUE if MPAM is configured to be virtual
// at the given el.

func MPAMIsVirtual(el : bits(2), mpamdata : MPAMdata) => boolean
begin
    if IsFeatureImplemented(FEAT_MPAMv2) then return FALSE; end;

    return (MPAMIDR_EL1().HAS_HCR == '1' && EL2Enabled() &&
            ((el == EL0 && MPAMHCR_EL2().EL0_VPMEN == '1' && !ELIsInHost(EL0)) ||
             (el == EL1 && MPAMHCR_EL2().EL1_VPMEN == '1')));
end;
```

Library pseudocode for shared/functions/mpam/PARTIDSpaceFromSS

```
// PARTIDSpaceFromSS()
// ===================
// Returns the primary PARTID space from the Security State.

func PARTIDSpaceFromSS(security : SecurityState) => PARTIDSpaceType
begin
    case security of
        when SS_NonSecure =>
            return PIDSpace_NonSecure;
        when SS_Root =>
            return PIDSpace_Root;
        when SS_Realm =>
            return PIDSpace_Realm;
        when SS_Secure =>
            return PIDSpace_Secure;
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/functions/mpam/UsePrimarySpaceEL10

```
// UsePrimarySpaceEL10()
// =====================
// Checks whether Primary space is configured in the
// MPAM3_EL3 and MPAM2_EL2 ALTSP control bits that affect
// MPAM ALTSP use at EL1 and EL0.

func UsePrimarySpaceEL10() => boolean
begin
    if MPAM3_EL3().ALTSP_HEN == '0' then
        return MPAM3_EL3().ALTSP_HFC == '0';
    end;
    return !MPAMIsEnabled() || !EL2Enabled() || MPAM2_EL2().ALTSP_HFC == '0';
end;
```

Library pseudocode for shared/functions/mpam/UsePrimarySpaceEL2

```
// UsePrimarySpaceEL2()
// ====================
// Checks whether Primary space is configured in the
// MPAM3_EL3 and MPAM2_EL2 ALTSP control bits that affect
// MPAM ALTSP use at EL2.

func UsePrimarySpaceEL2() => boolean
begin
    if MPAM3_EL3().ALTSP_HEN == '0' then
        return MPAM3_EL3().ALTSP_HFC == '0';
    end;
    return !MPAMIsEnabled() || MPAM2_EL2().ALTSP_EL2 == '0';
end;
```

Library pseudocode for shared/functions/mpam/mapvpmw

```
// mapvpmw()
// =========
// Map a virtual PARTID into a physical PARTID using
// the MPAMVPMn_EL2 registers.
// vpartid is now assumed in-range and valid (checked by caller)
// returns physical PARTID from mapping entry.

func mapvpmw(vpartid : integer) => PARTIDType
begin
    var vpmw : bits(64);
    let wd : integer = vpartid DIVRM 4;
    case wd of
        when 0    => vpmw = MPAMVPM0_EL2();
        when 1    => vpmw = MPAMVPM1_EL2();
        when 2    => vpmw = MPAMVPM2_EL2();
        when 3    => vpmw = MPAMVPM3_EL2();
        when 4    => vpmw = MPAMVPM4_EL2();
        when 5    => vpmw = MPAMVPM5_EL2();
        when 6    => vpmw = MPAMVPM6_EL2();
        when 7    => vpmw = MPAMVPM7_EL2();
        otherwise => vpmw = Zeros{64};
    end;
    // vpme_lsb selects LSB of field within register
    let vpme_lsb : integer = (vpartid MOD 4) * 16;
    return vpmw[vpme_lsb +: 16];
end;
```

Library pseudocode for shared/functions/predictionrestrict/ASID

```
// ASID
// ====
// Effective ASID.

func ASID() => bits(NUM_ASIDBITS)
begin
    if ELIsInHost(EL0) then
        var asid : bits(NUM_ASIDBITS);
        if TCR_EL2().A1 == '1' then
            asid = TTBR1_EL2().ASID;
        else
            asid = TTBR0_EL2().ASID;
        end;
        if IsFeatureImplemented(FEAT_ASID16) && TCR_EL2().AS == '1' then
            return asid;
        else
            return ZeroExtend{NUM_ASIDBITS}(asid[7:0]);
        end;
    end;
    if !ELUsingAArch32(EL1) then
        var asid : bits(NUM_ASIDBITS);
        if TCR_EL1().A1 == '1' then
            asid = TTBR1_EL1().ASID;
        else
            asid = TTBR0_EL1().ASID;
        end;
        if IsFeatureImplemented(FEAT_ASID16) && TCR_EL1().AS == '1' then
            return asid;
        else
            return ZeroExtend{NUM_ASIDBITS}(asid[7:0]);
        end;
    else
        if TTBCR().EAE == '0' then
            return ZeroExtend{NUM_ASIDBITS}(CONTEXTIDR().ASID);
        else
            if TTBCR().A1 == '1' then
                return ZeroExtend{NUM_ASIDBITS}(TTBR1().ASID);
            else
                return ZeroExtend{NUM_ASIDBITS}(TTBR0().ASID);
            end;
        end;
    end;
end;
```

Library pseudocode for shared/functions/predictionrestrict/ExecutionCntxt

```
// ExecutionCntxt
// ==============
// Context information for prediction restriction operation.

type ExecutionCntxt of record {
    is_vmid_valid     : boolean,        // is vmid valid for current context
    all_vmid          : boolean,        // should the operation be applied for all vmids
    vmid              : bits(NUM_VMIDBITS),
                                        // if all_vmid = FALSE, vmid to which operation is applied
    is_asid_valid     : boolean,        // is asid valid for current context
    all_asid          : boolean,        // should the operation be applied for all asids
    asid              : bits(NUM_ASIDBITS),
                                        // if all_asid = FALSE, ASID to which operation is applied
    software_context  : bits(64),       // SCXTNUM_ELx
    target_el         : bits(2),        // target EL at which operation is performed
    security          : SecurityState,
    restriction       : RestrictType    // type of restriction operation
};
```

Library pseudocode for shared/functions/predictionrestrict/RESTRICT_PREDICTIONS

```
// RESTRICT_PREDICTIONS()
// ======================
// Clear all speculated values.

impdef func RESTRICT_PREDICTIONS(c : ExecutionCntxt)
begin
    return;
end;
```

Library pseudocode for shared/functions/predictionrestrict/RestrictType

```
// RestrictType
// ============
// Type of restriction on speculation.

type RestrictType of enumeration {
    RestrictType_DataValue,
    RestrictType_ControlFlow,
    RestrictType_CachePrefetch,
    RestrictType_Other              // Any other trained speculation mechanisms than those above
};
```

Library pseudocode for shared/functions/predictionrestrict/TargetSecurityState

```
// TargetSecurityState()
// =====================
// Decode the target security state for the prediction context.

func TargetSecurityState(NS : bit, NSE : bit) => SecurityState
begin
    let curr_ss : SecurityState =  SecurityStateAtEL(PSTATE.EL);
    if curr_ss == SS_NonSecure then
        return SS_NonSecure;
    elsif curr_ss == SS_Secure then
        case NS of
            when '0' => return SS_Secure;
            when '1' => return SS_NonSecure;
        end;
    elsif IsFeatureImplemented(FEAT_RME) then
        if curr_ss == SS_Root then
            case NSE::NS of
                when '00' => return SS_Secure;
                when '01' => return SS_NonSecure;
                when '11' => return SS_Realm;
                when '10' => return SS_Root;
            end;
        elsif curr_ss == SS_Realm then
            return SS_Realm;
        end;
    end;
    unreachable;
end;
```

Library pseudocode for shared/functions/registers/BranchTo

```
// BranchTo()
// ==========
// Set program counter to a new address, with a branch type.
// Parameter branch_conditional indicates whether the executed branch has a conditional encoding.
// In AArch64 state the address might include a tag in the top eight bits.

func BranchTo{N}(target : bits(N), branch_type : BranchType, branch_conditional : boolean)
begin
    Hint_Branch(branch_type);
    if N == 32 then
        assert UsingAArch32();
        _PC = ZeroExtend{64}(target);
    else
        assert N == 64 && !UsingAArch32();
        let target_vaddress : bits(64) = AArch64_BranchAddr(target[63:0], PSTATE.EL);
        if (IsFeatureImplemented(FEAT_BRBE) &&
              branch_type IN {BranchType_DIR, BranchType_INDIR,
                              BranchType_DIRCALL, BranchType_INDCALL,
                              BranchType_RET}) then
            BRBEBranch(branch_type, branch_conditional, target_vaddress);
        end;
        let branch_taken : boolean = TRUE;

        if IsFeatureImplemented(FEAT_SPE) then
            SPEBranch{N}(target, branch_type, branch_conditional, branch_taken);
        end;

        _PC = target_vaddress;
    end;
    return;
end;
```

Library pseudocode for shared/functions/registers/BranchToAddr

```
// BranchToAddr()
// ==============
// Set program counter to a new address, with a branch type.
// In AArch64 state the address does not include a tag in the top eight bits.

func BranchToAddr{N}(target : bits(N), branch_type : BranchType)
begin
    Hint_Branch(branch_type);
    if N == 32 then
        assert UsingAArch32();
        _PC = ZeroExtend{64}(target);
    else
        assert N == 64 && !UsingAArch32();
        _PC = target[63:0];
    end;
    return;
end;
```

Library pseudocode for shared/functions/registers/BranchType

```
// BranchType
// ==========
// Information associated with a change in control flow.

type BranchType of enumeration {
    BranchType_DIRCALL,     // Direct Branch with link
    BranchType_INDCALL,     // Indirect Branch with link
    BranchType_ERET,        // Exception return (indirect)
    BranchType_DBGEXIT,     // Exit from Debug state
    BranchType_RET,         // Indirect branch with function return hint
    BranchType_DIR,         // Direct branch
    BranchType_INDIR,       // Indirect branch
    BranchType_EXCEPTION,   // Exception entry
    BranchType_RESET,       // Reset
    BranchType_UNKNOWN};    // Other
```

Library pseudocode for shared/functions/registers/Branchtypetaken

```
// Branchtypetaken
// ===============
var Branchtypetaken : BranchType;
```

Library pseudocode for shared/functions/registers/ESize

```
// ESize
// =====

type ESize of integer{8, 16, 32, 64, 128};
```

Library pseudocode for shared/functions/registers/EffectiveFPCR

```
// EffectiveFPCR()
// ===============
// Returns the effective FPCR value

func EffectiveFPCR() => FPCR_Type
begin
    if UsingAArch32() then
        var fpcr : FPCR_Type = ZeroExtend{64}(FPSCR());
        fpcr[7:0]   = '00000000';
        fpcr[31:27] = '00000';
        return fpcr;
    end;
    return FPCR();
end;
```

Library pseudocode for shared/functions/registers/Hint_Branch

```
// Hint_Branch()
// =============
// Report the hint passed to BranchTo() and BranchToAddr(), for consideration when processing
// the next instruction.

impdef func Hint_Branch(hint : BranchType)
begin
    Branchtypetaken = hint;
    return;
end;
```

Library pseudocode for shared/functions/registers/NextInstrAddr

```
// NextInstrAddr()
// ===============
// Return address of the sequentially next instruction.

impdef func NextInstrAddr{N}() => bits(N)
begin
    return Zeros{N};
end;
```

Library pseudocode for shared/functions/registers/ResetExternalDebugRegisters

```
// ResetExternalDebugRegisters()
// =============================
// Reset the External Debug registers in the Core power domain.

impdef func ResetExternalDebugRegisters(cold_reset : boolean)
begin
    return;
end;
```

Library pseudocode for shared/functions/registers/ThisInstrAddr

```
// ThisInstrAddr()
// ===============
// Return address of the current instruction.

func ThisInstrAddr{N}() => bits(N)
begin
    assert N == 64 || (N == 32 && UsingAArch32());
    return _PC[N-1:0];
end;
```

Library pseudocode for shared/functions/registers/UnimplementedIDRegister

```
// UnimplementedIDRegister()
// =========================
// Trap access to unimplemented encodings in the feature ID register space.

func UnimplementedIDRegister()
begin
    if IsFeatureImplemented(FEAT_IDST) then
        if EffectivelyAtEL0InHost() || EffectivelyAtEL0NotInHost() then
            AArch64_SystemAccessTraptoEL1orEL2(0x18);
        else
            AArch64_SystemAccessTrap(PSTATE.EL, 0x18);
        end;
    end;
    Undefined();
end;
```

Library pseudocode for shared/functions/registers/V

```
// V - accessor
// ============

accessor V{width : ESize}(n : integer) <=> value : bits(width)
begin
    // Read from SIMD&FP register with implicit slice of 8, 16
    // 32, 64 or 128 bits.
    getter
        assert n >= 0 && n <= 31;
        return _Z[[n]][width-1:0];
    end;

    // Write to SIMD&FP register with implicit extension from
    // 8, 16, 32, 64 or 128 bits.
    setter
        assert n >= 0 && n <= 31;
        if ConstrainUnpredictableBool(Unpredictable_SVEZEROUPPER) then
            _Z[[n]] = ZeroExtend{MAX_VL}(value);
        else
            let vlen : VecLen = if IsSVEEnabled(PSTATE.EL) then CurrentVL() else 128;
            _Z[[n]][vlen-1:0] = ZeroExtend{vlen}(value);
        end;
    end;
end;
```

Library pseudocode for shared/functions/registers/Vpart

```
// Vpart - accessor
// ================

accessor Vpart{width : ESize}(n : integer, part : integer) <=> value : bits(width)
begin
    // Reads a 128-bit SIMD&FP register in up to two parts:
    //  part 0 returns the bottom 8, 16, 32 or 64 bits of a value held in the register;
    //  part 1 returns the top half of the bottom 64 bits or the top half of the 128-bit
    //  value held in the register.
    getter
        assert n >= 0 && n <= 31;
        assert part IN {0, 1};
        if part == 0 then
            assert width < 128;
            return V{width}(n);
        else
            assert width IN {32,64};
            let vreg : bits(128) = V{}(n);
            return vreg[(width * 2)-1:width];
        end;
    end;

    setter
    // Writes a 128-bit SIMD&FP register in up to two parts:
    //  part 0 zero extends a 8, 16, 32, or 64-bit value to fill the whole register;
    //  part 1 inserts a 64-bit value into the top half of the register.
        assert n >= 0 && n <= 31;
        assert part IN {0, 1};
        if part == 0 then
            assert width < 128;
            V{width}(n) = value;
        else
            assert width == 64;
            let vreg : bits(64) = V{}(n);
            V{128}(n) = value[63:0] :: vreg;
        end;
    end;
end;
```

Library pseudocode for shared/functions/registers/_PC

```
// _PC - the program counter
// =========================

var _PC : bits(64);
```

Library pseudocode for shared/functions/registers/_R

```
// _R - the general-purpose register file array
// ============================================


var _R : array [[31]] of bits(64);
```

Library pseudocode for shared/functions/sysregisters/SPSR_ELx

```
// SPSR_ELx - accessor
// ===================

accessor SPSR_ELx() <=> value :  bits(64)
begin
    getter
        var result : bits(64);
        case PSTATE.EL of
            when EL1  => result = SPSR_EL1()[63:0];
            when EL2  => result = SPSR_EL2()[63:0];
            when EL3  => result = SPSR_EL3()[63:0];
            otherwise => unreachable;
        end;
        return result;
    end;

    setter
        case PSTATE.EL of
            when EL1   => SPSR_EL1()[63:0] = value[63:0];
            when EL2   => SPSR_EL2()[63:0] = value[63:0];
            when EL3   => SPSR_EL3()[63:0] = value[63:0];
            otherwise  => unreachable;
        end;
    end;
end;
```

Library pseudocode for shared/functions/sysregisters/SPSR_curr

```
// SPSR_curr - accessor
// ====================

accessor SPSR_curr() <=> value : bits(32)
begin
    getter
        var result : bits(32);
        case PSTATE.M of
            when M32_FIQ     => result = SPSR_fiq()[31:0];
            when M32_IRQ     => result = SPSR_irq()[31:0];
            when M32_Svc     => result = SPSR_svc()[31:0];
            when M32_Monitor => result = SPSR_mon()[31:0];
            when M32_Abort   => result = SPSR_abt()[31:0];
            when M32_Hyp     => result = SPSR_hyp()[31:0];
            when M32_Undef   => result = SPSR_und()[31:0];
            otherwise        => unreachable;
        end;
        return result;
    end;

    setter
        case PSTATE.M of
            when M32_FIQ     => SPSR_fiq()[31:0] = value[31:0];
            when M32_IRQ     => SPSR_irq()[31:0] = value[31:0];
            when M32_Svc     => SPSR_svc()[31:0] = value[31:0];
            when M32_Monitor => SPSR_mon()[31:0] = value[31:0];
            when M32_Abort   => SPSR_abt()[31:0] = value[31:0];
            when M32_Hyp     => SPSR_hyp()[31:0] = value[31:0];
            when M32_Undef   => SPSR_und()[31:0] = value[31:0];
            otherwise        => unreachable;
        end;
    end;
end;
```

Library pseudocode for shared/functions/system/AArch64_ChkFeat

```
// AArch64_ChkFeat()
// =================
// Indicates the status of some features

func AArch64_ChkFeat(feat_select : bits(64)) => bits(64)
begin
    var feat_en : bits(64) = Zeros{};
    feat_en[0] = if IsFeatureImplemented(FEAT_GCS) && GCSEnabled(PSTATE.EL) then '1' else '0';
    return feat_select AND NOT(feat_en);
end;
```

Library pseudocode for shared/functions/system/AddressAdd

```
// AddressAdd()
// ============
// Add an address with an offset and return the result.
// If FEAT_CPA2 is implemented, the pointer arithmetic is checked.

func AddressAdd(base : bits(64), offset : integer, accdesc : AccessDescriptor) => bits(64)
begin
    return AddressAdd(base, offset[63:0], accdesc);
end;

func AddressAdd(base : bits(64), offset : bits(64), accdesc : AccessDescriptor) => bits(64)
begin
    var result : bits(64) = base + offset;
    result = PointerAddCheckAtEL(accdesc.el, result, base);
    return result;
end;
```

Library pseudocode for shared/functions/system/AddressInNaturallyAlignedBlock

```
// AddressInNaturallyAlignedBlock()
// ================================
// Returns TRUE if the given addresses are in the same naturally aligned block, and FALSE otherwise.
// An address is in a naturally aligned block if it meets any of the below conditions:
// * is a power-of-two size.
// * Is no larger than the DC ZVA block size if ESR_ELx.FnP is being set to 0b0, or EDHSR is not
//   implemented or EDHSR.FnP is being set to 0b0 (as appropriate).
// * Is no larger than the smallest implemented translation granule if ESR_ELx.FnP, or EDHSR.FnP
//   (as appropriate) is being set to 0b1.
// * Contains a watchpointed address accessed by the memory access or set of contiguous memory
//   accesses that triggered the watchpoint.

impdef func AddressInNaturallyAlignedBlock(address1 : bits(64), address2 : bits(64)) => boolean
begin
    // Returns TRUE if address in within a naturally aligned block of memory as the watchpointed
    // location, else FALSE.
    return FALSE;
end;
```

Library pseudocode for shared/functions/system/AddressIncrement

```
// AddressIncrement()
// ==================
// Increment an address and return the result.
// If FEAT_CPA2 is implemented, the pointer arithmetic may be checked.

func AddressIncrement(base : bits(64), increment : integer, accdesc : AccessDescriptor) => bits(64)
begin
    return AddressIncrement(base, increment[63:0], accdesc);
end;

func AddressIncrement(base : bits(64), increment : bits(64), accdesc : AccessDescriptor) => bits(64)
begin
    var result : bits(64) = base + increment;
    // Checking the Pointer Arithmetic on an increment is equivalent to checking the
    // bytes in a sequential access crossing the 0xXXFF_FFFF_FFFF_FFFF boundary.
    if ConstrainUnpredictableBool(Unpredictable_CPACHECK) then
        result = PointerAddCheckAtEL(accdesc.el, result, base);
    end;
    return result;
end;
```

Library pseudocode for shared/functions/system/BranchTargetCheck

```
// BranchTargetCheck()
// ===================
// This function is executed checks if the current instruction is a valid target for a branch
// taken into, or inside, a guarded page. It is executed on every cycle once the current
// instruction has been decoded and the values of InGuardedPage and BTypeCompatible have been
// determined for the current instruction.

func BranchTargetCheck()
begin
    assert IsFeatureImplemented(FEAT_BTI) && !UsingAArch32();

    // The branch target check considers the following state variables:
    // * InGuardedPage, which is evaluated during instruction fetch.
    // * BTypeCompatible, which is evaluated during instruction decode.
    if Halted() then
        return;
    elsif IsZero(PSTATE.BTYPE) then
        return;
    elsif InGuardedPage && !BTypeCompatible then
        let pc : bits(64) = ThisInstrAddr{}();
        AArch64_BranchTargetException(pc[51:0]);
    end;
end;
```

Library pseudocode for shared/functions/system/ClearEventRegister

```
// ClearEventRegister()
// ====================
// Clear the Event Register of this PE.

func ClearEventRegister()
begin
    EventRegister = '0';
    return;
end;
```

Library pseudocode for shared/functions/system/ConditionHolds

```
// ConditionHolds()
// ================
// Return TRUE iff COND currently holds

func ConditionHolds(cond : bits(4)) => boolean
begin
    // Evaluate base condition.
    var result : boolean;
    case cond[3:1] of
        when '000' => result = (PSTATE.Z == '1');                          // EQ or NE
        when '001' => result = (PSTATE.C == '1');                          // CS or CC
        when '010' => result = (PSTATE.N == '1');                          // MI or PL
        when '011' => result = (PSTATE.V == '1');                          // VS or VC
        when '100' => result = (PSTATE.C == '1' && PSTATE.Z == '0');       // HI or LS
        when '101' => result = (PSTATE.N == PSTATE.V);                     // GE or LT
        when '110' => result = (PSTATE.N == PSTATE.V && PSTATE.Z == '0');  // GT or LE
        when '111' => result = TRUE;                                       // AL
    end;

    // Condition flag values in the set '111x' indicate always true
    // Otherwise, invert condition if necessary.
    if cond[0] == '1' && cond != '1111' then
        result = !result;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/system/ConsumptionOfSpeculativeDataBarrier

```
// ConsumptionOfSpeculativeDataBarrier()
// =====================================

impdef func ConsumptionOfSpeculativeDataBarrier()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/CurrentInstrSet

```
// CurrentInstrSet()
// =================

readonly func CurrentInstrSet() => InstrSet
begin
    var result : InstrSet;
    if UsingAArch32() then
        result = if PSTATE.T == '0' then InstrSet_A32 else InstrSet_T32;
        // PSTATE.J is RES0. Implementation of T32EE or Jazelle state not permitted.
    else
        result = InstrSet_A64;
    end;
    return result;
end;
```

Library pseudocode for shared/functions/system/CurrentSecurityState

```
// CurrentSecurityState()
// ======================
// Returns the effective security state at the exception level based off current settings.

readonly func CurrentSecurityState() => SecurityState
begin
    return SecurityStateAtEL(PSTATE.EL);
end;
```

Library pseudocode for shared/functions/system/DSBAlias

```
// DSBAlias
// ========
// Aliases of DSB.

type DSBAlias of enumeration {DSBAlias_SSBB, DSBAlias_PSSBB, DSBAlias_DSB};
```

Library pseudocode for shared/functions/system/EL0

```
// EL0-3
// =====
// PSTATE.EL Exception level bits.

constant EL3 : bits(2) = '11';
constant EL2 : bits(2) = '10';
constant EL1 : bits(2) = '01';
constant EL0 : bits(2) = '00';
```

Library pseudocode for shared/functions/system/EL2Enabled

```
// EL2Enabled()
// ============
// Returns TRUE if EL2 is present and executing
// - with the PE in Non-secure state when Non-secure EL2 is implemented, or
// - with the PE in Realm state when Realm EL2 is implemented, or
// - with the PE in Secure state when Secure EL2 is implemented and enabled, or
// - when EL3 is not implemented.

readonly func EL2Enabled() => boolean
begin
    var scr_ns : bit;
    if HaveEL(EL3) then
        if ELUsingAArch32(EL3) then
            scr_ns = SCR().NS;
        else
            scr_ns = SCR_EL3().NS;
        end;
    end;
    return HaveEL(EL2) && (!HaveEL(EL3) || scr_ns == '1' || IsSecureEL2Enabled());
end;
```

Library pseudocode for shared/functions/system/EL3SDDUndef

```
// EL3SDDUndef()
// =============
// Returns TRUE if in Debug state and EDSCR.SDD is set.

func EL3SDDUndef() => boolean
begin
    if Halted() && EDSCR().SDD == '1' then
        assert (PSTATE.EL != EL3  &&
                (IsFeatureImplemented(FEAT_RME) || CurrentSecurityState() != SS_Secure));
        return TRUE;
    else
        return FALSE;
    end;
end;
```

Library pseudocode for shared/functions/system/EL3SDDUndefPriority

```
// EL3SDDUndefPriority()
// =====================
// Returns TRUE if in Debug state, EDSCR.SDD is set, and an EL3 trap by an
// EL3 control register has priority over other traps.
// The IMPLEMENTATION DEFINED priority may be different for each case.

func EL3SDDUndefPriority() => boolean
begin
    return EL3SDDUndef() && ImpDefBool("EL3 trap priority when SDD == '1'");
end;
```

Library pseudocode for shared/functions/system/ELFromM32

```
// ELFromM32()
// ===========

readonly func ELFromM32(mode : bits(5)) => (boolean,bits(2))
begin
    // Convert an AArch32 mode encoding to an Exception level.
    // Returns (valid,EL):
    //   'valid' is TRUE if 'mode[4:0]' encodes a mode that is both valid for this implementation
    //           and the current value of SCR.NS/SCR_EL3.NS.
    //   'EL'    is the Exception level decoded from 'mode'.
    var el : bits(2);
    // Check for modes that are not valid for this implementation
    var valid : boolean = !BadMode(mode);
    let effective_nse_ns : bits(2) = EffectiveSCR_EL3_NSE() :: EffectiveSCR_EL3_NS();

    case mode of
        when M32_Monitor =>
            el = EL3;
        when M32_Hyp =>
            el = EL2;
        when M32_FIQ, M32_IRQ, M32_Svc, M32_Abort, M32_Undef, M32_System =>
            // If EL3 is implemented and using AArch32, then these modes are EL3 modes in Secure
            // state, and EL1 modes in Non-secure state. If EL3 is not implemented or is using
            // AArch64, then these modes are EL1 modes.
            el = (if HaveEL(EL3) && !HaveAArch64() && SCR().NS == '0' then EL3 else EL1);
        when M32_User =>
            el = EL0;
        otherwise =>
            valid = FALSE;           // Passed an illegal mode value
    end;

    if valid && el == EL2 && HaveEL(EL3) then
        if ELUsingAArch32(EL3) && SCR().NS == '0' then
            valid = FALSE; // EL2 only valid in Non-secure state in AArch32
        elsif !ELUsingAArch32(EL3) && SCR_EL3().NS == '0' then
            valid = FALSE; // EL2 only valid in Non-secure state in AArch32
        end;
    elsif valid && IsFeatureImplemented(FEAT_RME) && effective_nse_ns == '10' then
        valid = FALSE;               // Illegal Exception Return from EL3 if SCR_EL3.[NSE,NS]
                                     // selects a reserved encoding
    end;

    if !valid then el = ARBITRARY : bits(2); end;
    return (valid, el);
end;
```

Library pseudocode for shared/functions/system/ELFromSPSR

```
// ELFromSPSR()
// ============

// Convert an SPSR value encoding to an Exception level.
// Returns (valid,EL):
//   'valid' is TRUE if 'spsr[4:0]' encodes a valid mode for the current state.
//   'EL'    is the Exception level decoded from 'spsr'.

readonly func ELFromSPSR{N}(spsr : bits(N)) => (boolean,bits(2))
begin
    var el : bits(2);
    var valid : boolean;
    if spsr[4] == '0' then      // AArch64 state
        el = spsr[3:2];
        if (!Halted() && IsFeatureImplemented(FEAT_NV3) && spsr[3:2] == '10' && PSTATE.EL == EL1 &&
              EffectiveHCRX_EL2_NVTGE() == '1') then
            el = EL1;
        end;

        let effective_nse_ns : bits(2) = EffectiveSCR_EL3_NSE() :: EffectiveSCR_EL3_NS();
        if !HaveAArch64() then
            valid = FALSE;      // No AArch64 support
        elsif !HaveEL(el) then
            valid = FALSE;      // Exception level not implemented
        elsif spsr[1] == '1' then
            valid = FALSE;      // M[1] must be 0
        elsif el == EL0 && spsr[0] == '1' then
            valid = FALSE;      // for EL0, M[0] must be 0
        elsif IsFeatureImplemented(FEAT_RME) && el != EL3 && effective_nse_ns == '10' then
            valid = FALSE;      // Only EL3 valid in Root state
        elsif el == EL2 && HaveEL(EL3) && !IsSecureEL2Enabled() && EffectiveSCR_EL3_NS() == '0' then
            valid = FALSE;      // Unless Secure EL2 is enabled, EL2 valid only in Non-secure state
        else
            valid = TRUE;
        end;
    elsif HaveAArch32() then    // AArch32 state
        (valid, el) = ELFromM32(spsr[4:0]);
    else
        valid = FALSE;
    end;

    if !valid then el = ARBITRARY : bits(2); end;
    return (valid,el);
end;
```

Library pseudocode for shared/functions/system/ELIsInHost

```
// ELIsInHost()
// ============

readonly func ELIsInHost(el : bits(2)) => boolean
begin
    if !IsFeatureImplemented(FEAT_VHE) || ELUsingAArch32(EL2) then
        return FALSE;
    end;
    case el of
        when EL3 =>
            return FALSE;
        when EL2 =>
            return EL2Enabled() && EffectiveHCR_EL2_E2H() == '1';
        when EL1 =>
            return FALSE;
        when EL0 =>
            return EL2Enabled() && EffectiveHCR_EL2_E2H()::HCR_EL2().TGE == '11';
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/functions/system/ELStateUsingAArch32

```
// ELStateUsingAArch32()
// =====================

readonly func ELStateUsingAArch32(el : bits(2), secure : boolean) => boolean
begin
    // See ELStateUsingAArch32K() for description. Must only be called in circumstances where
    // result is valid (typically, that means 'el IN {EL1,EL2,EL3}').
    let (known, aarch32) : (boolean, boolean) = ELStateUsingAArch32K(el, secure);
    assert known;
    return aarch32;
end;
```

Library pseudocode for shared/functions/system/ELStateUsingAArch32K

```
// ELStateUsingAArch32K()
// ======================
// Returns (known, aarch32):
//   'known'   is FALSE for EL0 if the current Exception level is not EL0 and EL1 is
//             using AArch64, since it cannot determine the state of EL0; TRUE otherwise.
//   'aarch32' is TRUE if the specified Exception level is using AArch32; FALSE otherwise.

readonly func ELStateUsingAArch32K(el : bits(2), secure : boolean) => (boolean, boolean)
begin
    assert HaveEL(el);

    if !HaveAArch32EL(el) then
        return (TRUE, FALSE);   // Exception level is using AArch64
    elsif secure && el == EL2 then
        return (TRUE, FALSE);   // Secure EL2 is using AArch64
    elsif !HaveAArch64() then
        return (TRUE, TRUE);    // Highest Exception level, therefore all levels are using AArch32
    end;

    // Remainder of function deals with the interprocessing cases when highest
    // Exception level is using AArch64.

    if el == EL3 then
        return (TRUE, FALSE);
    end;

    if (HaveEL(EL3) && EffectiveSCR_EL3_RW() == '0' &&
          (!secure || !IsFeatureImplemented(FEAT_SEL2) || SCR_EL3().EEL2 == '0')) then
        // AArch32 below EL3.
        return (TRUE, TRUE);
    end;

    if el == EL2 then
        return (TRUE, FALSE);
    end;

    if (HaveEL(EL2) && !ELIsInHost(EL0) && HCR_EL2().RW == '0' &&
          (!secure || (IsFeatureImplemented(FEAT_SEL2) && SCR_EL3().EEL2 == '1'))) then
        // AArch32 below EL2.
        return (TRUE, TRUE);
    end;

    if el == EL1 then
        return (TRUE, FALSE);
    end;

    // The execution state of EL0 is only known from PSTATE.nRW when executing at EL0.
    if PSTATE.EL == EL0 then
        return (TRUE, PSTATE.nRW == '1');
    else
        return (FALSE, ARBITRARY : boolean);
    end;
end;
```

Library pseudocode for shared/functions/system/ELUsingAArch32

```
// ELUsingAArch32()
// ================

readonly func ELUsingAArch32(el : bits(2)) => boolean
                                    recurselimit 1
begin
    return ELStateUsingAArch32(el, IsSecureBelowEL3());
end;
```

Library pseudocode for shared/functions/system/ELUsingAArch32K

```
// ELUsingAArch32K()
// =================

readonly func ELUsingAArch32K(el : bits(2)) => (boolean,boolean)
begin
    return ELStateUsingAArch32K(el, IsSecureBelowEL3());
end;
```

Library pseudocode for shared/functions/system/EffectiveEA

```
// EffectiveEA()
// =============
// Returns the Effective value of SCR_EL3().EA

readonly func EffectiveEA() => bit
begin
    if !HaveEL(EL3) || Halted() then
        return '0';
    else
        return if HaveAArch64() then SCR_EL3().EA else SCR().EA;
    end;
end;
```

Library pseudocode for shared/functions/system/EffectiveHCR_EL2_E2H

```
// EffectiveHCR_EL2_E2H()
// ======================
// Returns the Effective value of HCR_EL2().E2H.

readonly func EffectiveHCR_EL2_E2H() => bit
begin
    if !IsFeatureImplemented(FEAT_VHE) then
        return '0';
    end;

    if !IsFeatureImplemented(FEAT_E2H0) then
        return '1';
    end;

    return HCR_EL2().E2H;
end;
```

Library pseudocode for shared/functions/system/EffectiveHCR_EL2_NVx

```
// EffectiveHCR_EL2_NVx()
// ======================
// Returns the Effective value of HCR_EL2().[NV2,NV1,NV].

readonly func EffectiveHCR_EL2_NVx() => bits(3)
begin
    if !EL2Enabled() || !IsFeatureImplemented(FEAT_NV) then
        return '000';
    end;

    var nv1 : bit = HCR_EL2().NV1;
    if (!IsFeatureImplemented(FEAT_E2H0) &&
          ImpDefBool("HCR_EL2.NV1 is implemented as RAZ")) then
        nv1 = '0';
    end;

    if HCR_EL2().NV == '0' then
        if nv1 == '1' then
            case ConstrainUnpredictable(Unpredictable_NVNV1) of
                when Constraint_NVNV1_00 => return '000';
                when Constraint_NVNV1_01 => return '010';
                when Constraint_NVNV1_11 => return '011';
            end;
        else
            return '000';
        end;
    end;

    if !IsFeatureImplemented(FEAT_NV2) then
        return '0' :: nv1 :: '1';
    end;

    var nv2 : bit = HCR_EL2().NV2;
    if (nv2 == '0' && ImpDefBool(
          "Programming HCR_EL2.<NV,NV2> to '10' behaves as '11'")) then
        nv2 = '1';
    end;

    return nv2 :: nv1 :: '1';
end;
```

Library pseudocode for shared/functions/system/EffectiveHCR_EL2_TERR

```
// EffectiveHCR_EL2_TERR()
// =======================
// Returns the Effective value of HCR_EL2().TERR.

readonly func EffectiveHCR_EL2_TERR() => bit
begin
    assert IsFeatureImplemented(FEAT_RAS);
    if !EL2Enabled() then return '0'; end;

    assert !ELUsingAArch32(EL2);

    if (NUM_ERROR_RECORDS == 0 &&
           ImpDefBool("HCR_EL2.TERR,SCR_EL3.TERR/TWERR=0 when ERRIDR_EL1.NUM==0")) then
        return '0';
    end;

    return HCR_EL2().TERR;
end;
```

Library pseudocode for shared/functions/system/EffectiveIESB

```
// EffectiveIESB()
// ===============
// Returns the Effective value of SCTLR_ELx().IESB.

func EffectiveIESB(target_el : bits(2)) => bit
begin
    if EffectiveNMEA(target_el) == '1' then return '1'; end;

    return SCTLR_EL(target_el).IESB;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_ADEn

```
// EffectiveSCR_EL3_ADEn()
// =======================
// Returns the Effective value of SCR_EL3().ADEn.

readonly func EffectiveSCR_EL3_ADEn() => bit
begin
    assert IsFeatureImplemented(FEAT_LS64_ACCDATA);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().ADEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_AIEn

```
// EffectiveSCR_EL3_AIEn()
// =======================
// Returns the Effective value of SCR_EL3().AIEn.

readonly func EffectiveSCR_EL3_AIEn() => bit
begin
    assert IsFeatureImplemented(FEAT_AIE);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().AIEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_AMVOFFEN

```
// EffectiveSCR_EL3_AMVOFFEN()
// ===========================
// Returns the Effective value of SCR_EL3().AMVOFFEN.
readonly func EffectiveSCR_EL3_AMVOFFEN() => bit
begin
    assert IsFeatureImplemented(FEAT_AMUv1p1);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().AMVOFFEN;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_API

```
// EffectiveSCR_EL3_API()
// ======================
// Returns the Effective value of SCR_EL3().API.

readonly func EffectiveSCR_EL3_API() => bit
begin
    assert IsFeatureImplemented(FEAT_PAuth);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().API;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_APK

```
// EffectiveSCR_EL3_APK()
// ======================
// Returns the Effective value of SCR_EL3().APK.

readonly func EffectiveSCR_EL3_APK() => bit
begin
    assert IsFeatureImplemented(FEAT_PAuth);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().APK;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_ATA

```
// EffectiveSCR_EL3_ATA()
// ======================
// Returns the Effective value of SCR_EL3().ATA.

readonly func EffectiveSCR_EL3_ATA() => bit
begin
    assert IsFeatureImplemented(FEAT_MTE2);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().ATA;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_D128En

```
// EffectiveSCR_EL3_D128En()
// =========================
// Returns the Effective value of SCR_EL3().D128En.

readonly func EffectiveSCR_EL3_D128En() => bit
begin
    assert IsFeatureImplemented(FEAT_D128);
    if !HaveEL(EL3) then return '1'; end;

    return SCR_EL3().D128En;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_DSE

```
// EffectiveSCR_EL3_DSE()
// ======================
// Returns the Effective value of SCR_EL3().DSE.

readonly func EffectiveSCR_EL3_DSE() => bit
begin
    assert IsFeatureImplemented(FEAT_E3DSE);
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    if EffectiveSCR_EL3_EnDSE() == '0' then return '0'; end;

    return SCR_EL3().DSE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EA

```
// EffectiveSCR_EL3_EA()
// =====================
// Returns the Effective value of SCR_EL3().EA.

readonly func EffectiveSCR_EL3_EA() => bit
begin
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().EA;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EASE

```
// EffectiveSCR_EL3_EASE()
// =======================
// Returns the Effective value of SCR_EL3().EASE.

readonly func EffectiveSCR_EL3_EASE() => bit
begin
    assert IsFeatureImplemented(FEAT_DoubleFault);
    assert HaveEL(EL3);
    assert !ELUsingAArch32(EL3);

    return SCR_EL3().EASE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_ECVEn

```
// EffectiveSCR_EL3_ECVEn()
// ========================
// Returns the Effective value of SCR_EL3().ECVEn.

readonly func EffectiveSCR_EL3_ECVEn() => bit
begin
    assert IsFeatureImplemented(FEAT_ECV_POFF);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().ECVEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EEL2

```
// EffectiveSCR_EL3_EEL2()
// =======================
// Returns the Effective value of SCR_EL3().EEL2.

readonly func EffectiveSCR_EL3_EEL2() => bit
begin
    assert IsFeatureImplemented(FEAT_SEL2);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().EEL2;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EnAS0

```
// EffectiveSCR_EL3_EnAS0()
// ========================
// Returns the Effective value of SCR_EL3().EnAS0.

readonly func EffectiveSCR_EL3_EnAS0() => bit
begin
    assert IsFeatureImplemented(FEAT_LS64_ACCDATA);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().EnAS0;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EnDSE

```
// EffectiveSCR_EL3_EnDSE()
// ========================
// Returns the Effective value of SCR_EL3().EnDSE.

readonly func EffectiveSCR_EL3_EnDSE() => bit
begin
    assert IsFeatureImplemented(FEAT_E3DSE);
    assert !ELUsingAArch32(EL3);

    if SCR_EL3().EA == '0' && EffectiveSCR_EL3_TMEA() == '0' then return '0'; end;

    return SCR_EL3().EnDSE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EnFPM

```
// EffectiveSCR_EL3_EnFPM()
// ========================
// Returns the Effective value of SCR_EL3().EnFPM.

readonly func EffectiveSCR_EL3_EnFPM() => bit
begin
    assert IsFeatureImplemented(FEAT_FPMR);
    if !HaveEL(EL3) then return '1'; end;

    return SCR_EL3().EnFPM;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EnIDCP128

```
// EffectiveSCR_EL3_EnIDCP128()
// ============================
// Returns the Effective value of SCR_EL3().EnIDCP128.

readonly func EffectiveSCR_EL3_EnIDCP128() => bit
begin
    assert IsFeatureImplemented(FEAT_SYSREG128);
    if !HaveEL(EL3) then return '1'; end;

    return SCR_EL3().EnIDCP128;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EnSCXT

```
// EffectiveSCR_EL3_EnSCXT()
// =========================
// Returns the Effective value of SCR_EL3().EnSCXT.

readonly func EffectiveSCR_EL3_EnSCXT() => bit
begin
    assert (IsFeatureImplemented(FEAT_CSV2_2) || IsFeatureImplemented(FEAT_CSV2_1p2));
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().EnSCXT;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_EnTP2

```
// EffectiveSCR_EL3_EnTP2()
// ========================
// Returns the Effective value of SCR_EL3().EnTP2.

readonly func EffectiveSCR_EL3_EnTP2() => bit
begin
    assert IsFeatureImplemented(FEAT_SME);
    if !HaveEL(EL3) then return '1'; end;

    return SCR_EL3().EnTP2;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_FGTEn

```
// EffectiveSCR_EL3_FGTEn()
// ========================
// Returns the Effective value of SCR_EL3().FGTEn.

readonly func EffectiveSCR_EL3_FGTEn() => bit
begin
    assert IsFeatureImplemented(FEAT_FGT);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().FGTEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_FGTEn2

```
// EffectiveSCR_EL3_FGTEn2()
// =========================
// Returns the Effective value of SCR_EL3().FGTEn2.

readonly func EffectiveSCR_EL3_FGTEn2() => bit
begin
    assert IsFeatureImplemented(FEAT_FGT2);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().FGTEn2;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_FIEN

```
// EffectiveSCR_EL3_FIEN()
// =======================
// Returns the Effective value of SCR_EL3().FIEN.

readonly func EffectiveSCR_EL3_FIEN() => bit
begin
    assert IsFeatureImplemented(FEAT_RASv1p1);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().FIEN;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_FIQ

```
// EffectiveSCR_EL3_FIQ()
// ======================
// Returns the Effective value of SCR_EL3().FIQ.

readonly func EffectiveSCR_EL3_FIQ() => bit
begin
    if !HaveEL(EL3) || ELUsingAArch32(EL3) then
        return '0';
    end;
    if IsFeatureImplemented(FEAT_GCIE) && IsFeatureImplemented(FEAT_EL3) then
        return '1';
    end;
    return SCR_EL3().FIQ;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_GCSEn

```
// EffectiveSCR_EL3_GCSEn()
// ========================
// Returns the Effective value of SCR_EL3().GCSEn.

readonly func EffectiveSCR_EL3_GCSEn() => bit
begin
    assert IsFeatureImplemented(FEAT_GCS);
    if !HaveEL(EL3) then return '1'; end;

    return SCR_EL3().GCSEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_GPF

```
// EffectiveSCR_EL3_GPF()
// ======================
// Returns the Effective value of SCR_EL3().GPF.

readonly func EffectiveSCR_EL3_GPF() => bit
begin
    assert IsFeatureImplemented(FEAT_RME);
    if !HaveEL(EL3) then return '0'; end;

    return SCR_EL3().GPF;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_HACDBSEn

```
// EffectiveSCR_EL3_HACDBSEn()
// ===========================
// Returns the Effective value of SCR_EL3().HACDBSEn.

readonly func EffectiveSCR_EL3_HACDBSEn() => bit
begin
    assert IsFeatureImplemented(FEAT_HACDBS);
    if !HaveEL(EL3) then return '1'; end;

    return SCR_EL3().HACDBSEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_HCE

```
// EffectiveSCR_EL3_HCE()
// ======================
// Returns the Effective value of SCR_EL3().HCE.

readonly func EffectiveSCR_EL3_HCE() => bit
begin
    if !HaveEL(EL2) then return '0'; end;
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().HCE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_HDBSSEn

```
// EffectiveSCR_EL3_HDBSSEn()
// ==========================
// Returns the Effective value of SCR_EL3().HDBSSEn.

readonly func EffectiveSCR_EL3_HDBSSEn() => bit
begin
    assert IsFeatureImplemented(FEAT_HDBSS);
    if !HaveEL(EL3) then return '1'; end;

    return SCR_EL3().HDBSSEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_HXEn

```
// EffectiveSCR_EL3_HXEn()
// =======================
// Returns the Effective value of SCR_EL3().HXEn.

readonly func EffectiveSCR_EL3_HXEn() => bit
begin
    assert IsFeatureImplemented(FEAT_HCX);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().HXEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_IRQ

```
// EffectiveSCR_EL3_IRQ()
// ======================
// Returns the Effective value of SCR_EL3().IRQ.

readonly func EffectiveSCR_EL3_IRQ() => bit
begin
    if !HaveEL(EL3) || ELUsingAArch32(EL3) then
        return '0';
    end;
    if IsFeatureImplemented(FEAT_GCIE) then
        return '0';
    end;
    return SCR_EL3().IRQ;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_MECEn

```
// EffectiveSCR_EL3_MECEn()
// ========================
// Returns the Effective value of SCR_EL3().MECEn.

readonly func EffectiveSCR_EL3_MECEn() => bit
begin
    assert IsFeatureImplemented(FEAT_MEC);
    if !HaveEL(EL3) then return '1'; end;

    return SCR_EL3().MECEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_NMEA

```
// EffectiveSCR_EL3_NMEA()
// =======================
// Returns the Effective value of SCR_EL3().NMEA.

readonly func EffectiveSCR_EL3_NMEA() => bit
begin
    assert IsFeatureImplemented(FEAT_DoubleFault);
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    if !IsFeatureImplemented(FEAT_DoubleFault2) && EffectiveSCR_EL3_EA() == '0' then
        return '0';
    end;

    return SCR_EL3().NMEA;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_NS

```
// EffectiveSCR_EL3_NS()
// =====================
// Returns the Effective value of SCR_EL3().NS.

readonly func EffectiveSCR_EL3_NS() => bit
begin
    if !HaveSecureState() then
        return '1';
    elsif !HaveEL(EL3) then
        return '0';
    elsif ELUsingAArch32(EL3) then
        return SCR().NS;
    else
        return SCR_EL3().NS;
    end;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_NSE

```
// EffectiveSCR_EL3_NSE()
// ======================
// Returns the Effective value of SCR_EL3().NSE.

readonly func EffectiveSCR_EL3_NSE() => bit
begin
    return if !IsFeatureImplemented(FEAT_RME) then '0' else SCR_EL3().NSE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_PFAREn

```
// EffectiveSCR_EL3_PFAREn()
// =========================
// Returns the Effective value of SCR_EL3().PFAREn.

readonly func EffectiveSCR_EL3_PFAREn() => bit
begin
    assert IsFeatureImplemented(FEAT_PFAR);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().PFAREn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_PIEn

```
// EffectiveSCR_EL3_PIEn()
// =======================
// Returns the Effective value of SCR_EL3().PIEn.

readonly func EffectiveSCR_EL3_PIEn() => bit
begin
    assert (IsFeatureImplemented(FEAT_S1PIE) || IsFeatureImplemented(FEAT_S2PIE) ||
            IsFeatureImplemented(FEAT_S1POE) || IsFeatureImplemented(FEAT_S2POE));
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().PIEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_RCWMASKEn

```
// EffectiveSCR_EL3_RCWMASKEn()
// ============================
// Returns the Effective value of SCR_EL3().RCWMASKEn.

readonly func EffectiveSCR_EL3_RCWMASKEn() => bit
begin
    assert IsFeatureImplemented(FEAT_THE);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().RCWMASKEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_RW

```
// EffectiveSCR_EL3_RW()
// =====================
// Returns the Effective value of SCR_EL3().RW.

readonly func EffectiveSCR_EL3_RW() => bit
begin
    if !HaveAArch64() then
        return '0';
    end;
    if !HaveAArch32EL(EL2) && !HaveAArch32EL(EL1) then
        return '1';
    end;
    if HaveAArch32EL(EL1) then
        if !HaveAArch32EL(EL2) && EffectiveSCR_EL3_NS() == '1' then
            return '1';
        end;
        if (IsFeatureImplemented(FEAT_SEL2) && SCR_EL3().EEL2 == '1' &&
              EffectiveSCR_EL3_NS() == '0') then
            return '1';
        end;
    end;
    return SCR_EL3().RW;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_SCTLR2En

```
// EffectiveSCR_EL3_SCTLR2En()
// ===========================
// Returns the Effective value of SCR_EL3().SCTLR2En.

readonly func EffectiveSCR_EL3_SCTLR2En() => bit
begin
    assert IsFeatureImplemented(FEAT_SCTLR2);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().SCTLR2En;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_SIF

```
// EffectiveSCR_EL3_SIF()
// ======================
// Returns the Effective value of SCR_EL3().SIF.

readonly func EffectiveSCR_EL3_SIF() => bit
begin
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    if IsFeatureImplemented(FEAT_RME) && !IsFeatureImplemented(FEAT_SEL2) then
        return '0';
    end;

    return SCR_EL3().SIF;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_SMD

```
// EffectiveSCR_EL3_SMD()
// ======================
// Returns the Effective value of SCR_EL3().SMD.

readonly func EffectiveSCR_EL3_SMD() => bit
begin
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().SMD;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_SRMASKEn

```
// EffectiveSCR_EL3_SRMASKEn()
// ===========================
// Returns the Effective value of SCR_EL3().SRMASKEn.

readonly func EffectiveSCR_EL3_SRMASKEn() => bit
begin
    assert IsFeatureImplemented(FEAT_SRMASK);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().SRMASKEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_ST

```
// EffectiveSCR_EL3_ST()
// =====================
// Returns the Effective value of SCR_EL3().ST.

readonly func EffectiveSCR_EL3_ST() => bit
begin
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().ST;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TCR2En

```
// EffectiveSCR_EL3_TCR2En()
// =========================
// Returns the Effective value of SCR_EL3().TCR2En.

readonly func EffectiveSCR_EL3_TCR2En() => bit
begin
    assert IsFeatureImplemented(FEAT_TCR2);
    if !HaveEL(EL3) then return '1'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().TCR2En;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TERR

```
// EffectiveSCR_EL3_TERR()
// =======================
// Returns the Effective value of SCR_EL3().TERR.

readonly func EffectiveSCR_EL3_TERR() => bit
begin
    assert IsFeatureImplemented(FEAT_RAS);
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    if (NUM_ERROR_RECORDS == 0 &&
           ImpDefBool("HCR_EL2.TERR,SCR_EL3.TERR/TWERR=0 when ERRIDR_EL1.NUM==0")) then
        return '0';
    end;

    return SCR_EL3().TERR;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TID3

```
// EffectiveSCR_EL3_TID3()
// =======================
// Returns the Effective value of SCR_EL3().TID3.

readonly func EffectiveSCR_EL3_TID3() => bit
begin
    assert IsFeatureImplemented(FEAT_IDTE3);
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().TID3;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TID5

```
// EffectiveSCR_EL3_TID5()
// =======================
// Returns the Effective value of SCR_EL3().TID5.

readonly func EffectiveSCR_EL3_TID5() => bit
begin
    assert IsFeatureImplemented(FEAT_IDTE3);
    assert IsFeatureImplemented(FEAT_MTE2);
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().TID5;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TLOR

```
// EffectiveSCR_EL3_TLOR()
// =======================
// Returns the Effective value of SCR_EL3().TLOR.

readonly func EffectiveSCR_EL3_TLOR() => bit
begin
    assert IsFeatureImplemented(FEAT_LOR);
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().TLOR;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TMEA

```
// EffectiveSCR_EL3_TMEA()
// =======================
// Returns the Effective value of SCR_EL3().TMEA

readonly func EffectiveSCR_EL3_TMEA() => bit
begin
    if (IsFeatureImplemented(FEAT_DoubleFault2) && HaveEL(EL3) &&
          !ELUsingAArch32(EL3)) then
        return SCR_EL3().TMEA;
    else
        return '0';
    end;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TRNDR

```
// EffectiveSCR_EL3_TRNDR()
// ========================
// Returns the Effective value of SCR_EL3().TRNDR.

readonly func EffectiveSCR_EL3_TRNDR() => bit
begin
    assert IsFeatureImplemented(FEAT_RNG_TRAP);
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().TRNDR;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TWE

```
// EffectiveSCR_EL3_TWE()
// ======================
// Returns the Effective value of SCR_EL3().TWE.

readonly func EffectiveSCR_EL3_TWE() => bit
begin
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().TWE;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TWEDEL

```
// EffectiveSCR_EL3_TWEDEL()
// =========================
// Returns the Effective value of SCR_EL3().TWEDEL.

readonly func EffectiveSCR_EL3_TWEDEL() => bits(4)
begin
    assert IsFeatureImplemented(FEAT_TWED);
    if !HaveEL(EL3) then return '0000'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().TWEDEL;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TWEDEn

```
// EffectiveSCR_EL3_TWEDEn()
// =========================
// Returns the Effective value of SCR_EL3().TWEDEn.

readonly func EffectiveSCR_EL3_TWEDEn() => bit
begin
    assert IsFeatureImplemented(FEAT_TWED);
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().TWEDEn;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TWERR

```
// EffectiveSCR_EL3_TWERR()
// ========================
// Returns the Effective value of SCR_EL3().TWERR.

readonly func EffectiveSCR_EL3_TWERR() => bit
begin
    assert IsFeatureImplemented(FEAT_RASv2);
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    if (NUM_ERROR_RECORDS == 0 &&
            ImpDefBool("HCR_EL2.TERR,SCR_EL3.TERR/TWERR=0 when ERRIDR_EL1.NUM==0")) then
        return '0';
    end;

    return SCR_EL3().TWERR;
end;
```

Library pseudocode for shared/functions/system/EffectiveSCR_EL3_TWI

```
// EffectiveSCR_EL3_TWI()
// ======================
// Returns the Effective value of SCR_EL3().TWI.

readonly func EffectiveSCR_EL3_TWI() => bit
begin
    if !HaveEL(EL3) then return '0'; end;

    assert !ELUsingAArch32(EL3);

    return SCR_EL3().TWI;
end;
```

Library pseudocode for shared/functions/system/EffectiveTGE

```
// EffectiveTGE()
// ==============
// Returns the Effective value of TGE.

readonly func EffectiveTGE() => bit
begin
    if EL2Enabled() then
        return if ELUsingAArch32(EL2) then HCR().TGE else HCR_EL2().TGE;
    else
        return '0';        // Effective value of TGE is zero
    end;
end;
```

Library pseudocode for shared/functions/system/EffectiveTGEforEL0Traps

```
// EffectiveTGEforEL0Traps()
// =========================
// Returns the Effective value of HCR_EL2().TGE for direct accesses to system or special-purpose
// registers, and execution of system instructions, that may trap when either
// EffectivelyAtEL0NotInHost() or EffectivelyAtEL0InHost() is TRUE.

func EffectiveTGEforEL0Traps() => bit
begin
    if !EL2Enabled() then
        return '0';
    end;

    return HCR_EL2().TGE;
end;
```

Library pseudocode for shared/functions/system/EffectivelyAtEL0InHost

```
// EffectivelyAtEL0InHost()
// ========================
// Accesses from EL0 when the Effective value of HCR_EL2().[E2H, TGE] is {1, 1} will be trapped
// or Undefined.

func EffectivelyAtEL0InHost() => boolean
begin
    return (PSTATE.EL == EL0 && ELIsInHost(EL0));
end;
```

Library pseudocode for shared/functions/system/EffectivelyAtEL0NotInHost

```
// EffectivelyAtEL0NotInHost()
// ===========================
// Accesses from EL0 when the Effective value of HCR_EL2().[E2H, TGE] is not {1, 1} will be
// trapped or Undefined.

func EffectivelyAtEL0NotInHost() => boolean
begin
    return (PSTATE.EL == EL0 && !ELIsInHost(EL0));
end;
```

Library pseudocode for shared/functions/system/EndOfInstruction

```
// EndOfInstruction()
// ==================
// Terminate processing of the current instruction.

noreturn impdef func EndOfInstruction(EndReason : EndOfInstructionReason)
begin
    throw EndOfInstructionException{reason=EndReason};
end;
```

Library pseudocode for shared/functions/system/EndOfInstructionException

```
// EndOfInstructionException
// =========================
// End of Instruction exception type.

type EndOfInstructionException of exception {
    reason : EndOfInstructionReason
};
```

Library pseudocode for shared/functions/system/EndOfInstructionReason

```
// EndOfInstructionReason
// ======================

type EndOfInstructionReason of enumeration {
    EndOfInstructionReason_Retired,
    EndOfInstructionReason_Exception
};
```

Library pseudocode for shared/functions/system/EnterLowPowerState

```
// EnterLowPowerState()
// ====================
// PE enters a low-power state.


impdef func EnterLowPowerState()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/EventRegister

```
// EventRegister
// =============
// Event Register for this PE.

var EventRegister : bits(1);
```

Library pseudocode for shared/functions/system/ExceptionalOccurrenceTargetState

```
// ExceptionalOccurrenceTargetState
// ================================
// Enumeration to represent the target state of an Exceptional Occurrence.
// The Exceptional Occurrence can be either Exception or Debug State entry.

type ExceptionalOccurrenceTargetState of enumeration {
    AArch32_NonDebugState,
    AArch64_NonDebugState,
    DebugState
};
```

Library pseudocode for shared/functions/system/ExecuteAsNOP

```
// ExecuteAsNOP()
// ==============

func ExecuteAsNOP()
begin
    EndOfInstruction(EndOfInstructionReason_Retired);
end;
```

Library pseudocode for shared/functions/system/FIQPending

```
// FIQPending()
// ============
// Returns a tuple indicating if there is any pending physical FIQ
// and if the pending FIQ has superpriority.

readonly impdef func FIQPending() => (boolean, boolean)
begin
    return (FALSE, FALSE);
end;
```

Library pseudocode for shared/functions/system/GetAccumulatedFPExceptions

```
// GetAccumulatedFPExceptions()
// ============================
// Returns FP exceptions accumulated by the PE.

impdef func GetAccumulatedFPExceptions() => bits(8)
begin
    return ARBITRARY : bits(8);
end;
```

Library pseudocode for shared/functions/system/GetLoadStoreType

```
// GetLoadStoreType()
// ==================
// Returns the Load/Store Type. Used when a Translation fault,
// Access flag fault, or Permission fault generates a Data Abort.

impdef func GetLoadStoreType() => bits(2)
begin
    return ARBITRARY : bits(2);
end;
```

Library pseudocode for shared/functions/system/GetPSRFromPSTATE

```
// GetPSRFromPSTATE()
// ==================
// Return a PSR value which represents the current PSTATE

func GetPSRFromPSTATE{N}(targetELState : ExceptionalOccurrenceTargetState) => bits(N)
begin
    if UsingAArch32() && targetELState == AArch32_NonDebugState then
        assert N == 32;
    else
        assert N == 64;
    end;

    var spsr : bits(N) = Zeros{};
    if IsFeatureImplemented(FEAT_UINJ) && targetELState == DebugState then
        spsr[36] = PSTATE.UINJ;
    end;
    spsr[31:28] = PSTATE.[N,Z,C,V];
    if IsFeatureImplemented(FEAT_PAN) then spsr[22] = PSTATE.PAN; end;
    spsr[20]     = PSTATE.IL;
    if PSTATE.nRW == '1' then                           // AArch32 state
        spsr[27]     = PSTATE.Q;
        spsr[26:25]  = PSTATE.IT[1:0];
        if IsFeatureImplemented(FEAT_SSBS) then spsr[23] = PSTATE.SSBS; end;
        if IsFeatureImplemented(FEAT_DIT) then
            if targetELState == AArch32_NonDebugState then
                spsr[21] = PSTATE.DIT;
            else                                        // AArch64_NonDebugState or DebugState
                spsr[24] = PSTATE.DIT;
            end;
        end;
        if targetELState IN {AArch64_NonDebugState, DebugState} then
            spsr[21] = PSTATE.SS;
        end;
        spsr[19:16]  = PSTATE.GE;
        spsr[15:10]  = PSTATE.IT[7:2];
        spsr[9]      = PSTATE.E;
        spsr[8:6]    = PSTATE.[A,I,F];                  // No PSTATE.D in AArch32 state
        spsr[5]      = PSTATE.T;
        assert PSTATE.M[4] == PSTATE.nRW;               // bit [4] is the discriminator
        spsr[4:0]    = PSTATE.M;
    else                                                // AArch64 state
        if IsFeatureImplemented(FEAT_PAuth_LR) then spsr[35] = PSTATE.PACM; end;
        if IsFeatureImplemented(FEAT_GCS) then spsr[34] = PSTATE.EXLOCK; end;
        if (IsFeatureImplemented(FEAT_EBEP) || IsFeatureImplemented(FEAT_SPE_EXC) ||
              IsFeatureImplemented(FEAT_TRBE_EXC)) then
            spsr[32] = PSTATE.PM;
        end;
        if IsFeatureImplemented(FEAT_MTE) then spsr[25] = PSTATE.TCO; end;
        if IsFeatureImplemented(FEAT_DIT) then spsr[24] = PSTATE.DIT; end;
        if IsFeatureImplemented(FEAT_UAO) then spsr[23] = PSTATE.UAO; end;
        spsr[21]    = PSTATE.SS;
        if IsFeatureImplemented(FEAT_NMI) then spsr[13] = PSTATE.ALLINT; end;
        if IsFeatureImplemented(FEAT_SSBS) then spsr[12] = PSTATE.SSBS; end;
        if IsFeatureImplemented(FEAT_BTI) then spsr[11:10] = PSTATE.BTYPE; end;
        spsr[9:6]    = PSTATE.[D,A,I,F];
        spsr[4]      = PSTATE.nRW;
        spsr[3:2]    = PSTATE.EL;
        spsr[0]      = PSTATE.SP;
    end;
    return spsr;
end;
```

Library pseudocode for shared/functions/system/HaveAArch32

```
// HaveAArch32()
// =============
// Return TRUE if AArch32 state is supported from at least EL0.

readonly func HaveAArch32() => boolean
begin
    return IsFeatureImplemented(FEAT_AA32EL0);
end;
```

Library pseudocode for shared/functions/system/HaveAArch32EL

```
// HaveAArch32EL()
// ===============
// Return TRUE if Exception level 'el' supports AArch32 in this implementation

readonly func HaveAArch32EL(el : bits(2)) => boolean
begin
    case el of
        when EL0 => return IsFeatureImplemented(FEAT_AA32EL0);
        when EL1 => return IsFeatureImplemented(FEAT_AA32EL1);
        when EL2 => return IsFeatureImplemented(FEAT_AA32EL2);
        when EL3 => return IsFeatureImplemented(FEAT_AA32EL3);
    end;
end;
```

Library pseudocode for shared/functions/system/HaveAArch64

```
// HaveAArch64()
// =============
// Return TRUE if the highest Exception level is using AArch64 state.

readonly func HaveAArch64() => boolean
begin
    return (IsFeatureImplemented(FEAT_AA64EL0)
            || IsFeatureImplemented(FEAT_AA64EL1)
            || IsFeatureImplemented(FEAT_AA64EL2)
            || IsFeatureImplemented(FEAT_AA64EL3)
            );
end;
```

Library pseudocode for shared/functions/system/HaveEL

```
// HaveEL()
// ========
// Return TRUE if Exception level 'el' is supported

readonly func HaveEL(el : bits(2)) => boolean
begin
    case el of
        when EL1,EL0 =>
            return TRUE;                          // EL1 and EL0 must exist
        when EL2 =>
            return IsFeatureImplemented(FEAT_AA64EL2) || IsFeatureImplemented(FEAT_AA32EL2);
        when EL3 =>
            return IsFeatureImplemented(FEAT_AA64EL3) || IsFeatureImplemented(FEAT_AA32EL3);
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/functions/system/HaveELUsingSecurityState

```
// HaveELUsingSecurityState()
// ==========================
// Returns TRUE if Exception level 'el' with Security state 'ss' is supported, FALSE otherwise.

readonly func HaveELUsingSecurityState(el : bits(2), ss : SecurityState) => boolean
begin
    if !HaveEL(el) then
        return FALSE;
    end;

    case el of
        when EL3 =>
            if IsFeatureImplemented(FEAT_RME) then
                return ss == SS_Root;
            end;

            return ss == SS_Secure;
        when EL2 =>
            case ss of
                when SS_Realm     => return IsFeatureImplemented(FEAT_RME);
                when SS_NonSecure => return !SecureOnlyImplementation();
                when SS_Secure    => return IsFeatureImplemented(FEAT_SEL2);
                otherwise         => return FALSE;
            end;
        when EL1 =>
            case ss of
                when SS_Realm     => return IsFeatureImplemented(FEAT_RME);
                when SS_NonSecure => return !SecureOnlyImplementation();
                when SS_Secure    => return IsFeatureImplemented(FEAT_Secure) && HaveAArch64();
                otherwise         => return FALSE;
            end;
        when EL0 =>
            case ss of
                when SS_Realm     => return IsFeatureImplemented(FEAT_RME);
                when SS_NonSecure => return !SecureOnlyImplementation();
                when SS_Secure    => return IsFeatureImplemented(FEAT_Secure);
                otherwise         => return FALSE;
            end;
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/functions/system/HaveSecureState

```
// HaveSecureState()
// =================
// Return TRUE if Secure State is supported.

readonly func HaveSecureState() => boolean
begin
    return IsFeatureImplemented(FEAT_Secure);
end;
```

Library pseudocode for shared/functions/system/HighestEL

```
// HighestEL()
// ===========
// Returns the highest implemented Exception level.

readonly func HighestEL() => bits(2)
begin
    if HaveEL(EL3) then
        return EL3;
    elsif HaveEL(EL2) then
        return EL2;
    else
        return EL1;
    end;
end;
```

Library pseudocode for shared/functions/system/Hint_CLRBHB

```
// Hint_CLRBHB()
// =============
// Provides a hint to clear the branch history for the current context.

impdef func Hint_CLRBHB()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/Hint_DGH

```
// Hint_DGH()
// ==========
// Provides a hint to close any gathering occurring within the implementation.

impdef func Hint_DGH()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/Hint_ScopeRelease

```
// Hint_ScopeRelease()
// ===================
// Signals to the memory system that if the next instruction in program order has Release semantics
// or is a Data Memory Barrier that defines a Barrier scope with a preceding Scoped Release
// Boundary, then these instructions do not create a Barrier-ordered-before relationship between
// any Effect in program order before the Scoped Release Boundary and the Effect with Release
// semantics or any Effect in program order after the DMB Effect.

impdef func Hint_ScopeRelease(stshstrm : boolean)
begin
    return;
end;
```

Library pseudocode for shared/functions/system/Hint_ScopedReleaseBoundary

```
// Hint_ScopedReleaseBoundary()
// ============================
// Signals to the memory system that a corresponding instruction with Release semantics or Data
// Memory Barrier will subsequently occur in program order, and if that corresponding instruction
// defines a Barrier scope with the Scoped Release Boundary, then these instructions do not create
// a Barrier-ordered-before relationship between any Effect in program order before the Scoped
// Release Boundary and the Effect with Release semantics or any Effect in program order after the
// DMB Effect.

impdef func Hint_ScopedReleaseBoundary()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/Hint_SharedUpdate

```
// Hint_SharedUpdate()
// ===================
// Provides a hint to the memory system that if the next instruction in program order generates an
// Explicit Memory Effect, then it is to a location that one or more other threads of execution
// are likely to subsequently update, and there is a performance benefit to ensuring subsequent
// updates by other threads of execution may complete with minimal latency.

impdef func Hint_SharedUpdate(priority : boolean)
begin
    return;
end;
```

Library pseudocode for shared/functions/system/Hint_StoreConcurrentPriority

```
// Hint_StoreConcurrentPriority()
// ==============================
// Provides a hint to the memory system that if the next instruction in program order generates an
// Explicit Memory Write Effect, then there is a performance benefit if the corresponding Explicit
// Memory Write Effect is sequenced before Explicit Memory Effects from other threads of execution
// in the coherence order to the same location.

impdef func Hint_StoreConcurrentPriority()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/Hint_StoreShared

```
// Hint_StoreShared()
// ==================
// Provides a hint that if the next instruction is an explicit write it is being waited on by
// observers and as such the data should propagate to them with minimum latency.
// A stream value of FALSE indicates KEEP whilst a value of TRUE indicates STRM.

impdef func Hint_StoreShared(stream : boolean)
begin
    return;
end;
```

Library pseudocode for shared/functions/system/Hint_WFE

```
// Hint_WFE()
// ==========
// Provides a hint indicating that the PE can enter a low-power state and
// remain there until a wakeup event occurs.

func Hint_WFE()
begin
    if IsEventRegisterSet() then
        ClearEventRegister();
        return;
    end;

    var trap : boolean;
    var target_el : bits(2);
    (trap, target_el) = AArch64_CheckForWFxTrap(WFxType_WFE);
    if trap then
        if IsFeatureImplemented(FEAT_TWED) then
            // Determine if trap delay is enabled and delay amount
            var delay_enabled : boolean;
            var delay : integer;
            (delay_enabled, delay) = WFETrapDelay(target_el);
            if WaitForEventUntilDelay(delay_enabled, delay) then
                // Event arrived before delay
                return;
            end;
        end;

        // Proceed with trapping
        if target_el == EL3 && EL3SDDUndef() then
            Undefined();
        else
            AArch64_WFxTrap(WFxType_WFE, target_el);
        end;
    else
        WaitForEvent();
    end;
end;
```

Library pseudocode for shared/functions/system/Hint_WFET

```
// Hint_WFET()
// ===========
// Provides a hint indicating that the PE can enter a low-power state
// and remain there until a wakeup event occurs or, for WFET,  a local
// timeout event is generated when the virtual timer value equals or
// exceeds the supplied threshold value.

func Hint_WFET(localtimeout : integer)
begin
    if IsEventRegisterSet() then
        ClearEventRegister();
        return;
    end;

    if IsFeatureImplemented(FEAT_WFxT) && LocalTimeoutEvent(localtimeout) then
        // No further operation if the local timeout has expired.
        EndOfInstruction(EndOfInstructionReason_Retired);
    end;

    var trap : boolean;
    var target_el : bits(2);
    (trap, target_el) = AArch64_CheckForWFxTrap(WFxType_WFET);
    if trap then
        if IsFeatureImplemented(FEAT_TWED) then
            // Determine if trap delay is enabled and delay amount
            var delay_enabled : boolean;
            var delay : integer;
            (delay_enabled, delay) = WFETrapDelay(target_el);
            if WaitForEventUntilDelay(delay_enabled, delay) then
                // Event arrived before the delay expired
                return;
            end;
        end;

        // Proceed with trapping
        if target_el == EL3 && EL3SDDUndef() then
            Undefined();
        else
            AArch64_WFxTrap(WFxType_WFET, target_el);
        end;
    else
        WaitForEvent(localtimeout);
    end;
end;
```

Library pseudocode for shared/functions/system/Hint_WFI

```
// Hint_WFI()
// ==========
// Provides a hint indicating that the PE can enter a low-power state and
// remain there until a wakeup event occurs.

func Hint_WFI()
begin
    if AArch64_InterruptPending() then
        // No further operation if an interrupt is pending.
        EndOfInstruction(EndOfInstructionReason_Retired);
    end;

    var trap : boolean;
    var target_el : bits(2);
    (trap, target_el) = AArch64_CheckForWFxTrap(WFxType_WFI);
    if trap then
        if target_el == EL3 && EL3SDDUndef() then
            Undefined();
        end;
        AArch64_WFxTrap(WFxType_WFI, target_el);
    else
        WaitForInterrupt();
    end;
end;
```

Library pseudocode for shared/functions/system/Hint_WFIT

```
// Hint_WFIT()
// ===========
// Provides a hint indicating that the PE can enter a low-power state and
// remain there until a wakeup event occurs or, for WFIT, a local timeout
// event is generated when the virtual timer value equals or exceeds the
// supplied threshold value.

func Hint_WFIT(localtimeout : integer)
begin
    if (AArch64_InterruptPending() || (IsFeatureImplemented(FEAT_WFxT) &&
          LocalTimeoutEvent(localtimeout))) then
        // No further operation if an interrupt is pending or the local timeout has expired.
        EndOfInstruction(EndOfInstructionReason_Retired);
    end;

    var trap : boolean;
    var target_el : bits(2);
    (trap, target_el) = AArch64_CheckForWFxTrap(WFxType_WFIT);
    if trap then
        if target_el == EL3 && EL3SDDUndef() then
            Undefined();
        end;
        AArch64_WFxTrap(WFxType_WFIT, target_el);
    else
        WaitForInterrupt(localtimeout);
    end;
end;
```

Library pseudocode for shared/functions/system/Hint_Yield

```
// Hint_Yield()
// ============
// Provides a hint that the task performed by a thread is of low
// importance so that it could yield to improve overall performance.

impdef func Hint_Yield()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/IRQPending

```
// IRQPending()
// ============
// Returns a tuple indicating if there is any pending physical IRQ
// and if the pending IRQ has superpriority.

readonly impdef func IRQPending() => (boolean, boolean)
begin
    return (FALSE, FALSE);
end;
```

Library pseudocode for shared/functions/system/IllegalExceptionReturn

```
// IllegalExceptionReturn()
// ========================

readonly func IllegalExceptionReturn{N}(spsr : bits(N)) => boolean
begin
    // Check for illegal return:
    //   * To an unimplemented Exception level.
    //   * To EL2 in Secure state, when SecureEL2 is not enabled.
    //   * To EL0 using AArch64 state, with SPSR.M[0]==1.
    //   * To AArch64 state with SPSR.M[1]==1.
    //   * To AArch32 state with an illegal value of SPSR.M.
    let (valid, target) : (boolean, bits(2)) = ELFromSPSR{N}(spsr);
    if !valid then return TRUE; end;

    // Check for return to higher Exception level.
    if UInt(target) > UInt(PSTATE.EL) then return TRUE; end;

    let spsr_mode_is_aarch32 : boolean = (spsr[4] == '1');

    // Check for illegal return:
    //   * To EL1, EL2 or EL3 with register width specified in the SPSR different from the
    //     Execution state used in the Exception level being returned to, as determined by
    //     the SCR_EL3.RW or HCR_EL2.RW bits, or as configured from reset.
    //   * To EL0 using AArch64 state when EL1 is using AArch32 state as determined by the
    //     SCR_EL3.RW or HCR_EL2.RW bits or as configured from reset.
    //   * To AArch64 state from AArch32 state (should be caught by above).
    let (known, target_el_is_aarch32) : (boolean, boolean) = ELUsingAArch32K(target);
    assert known || (target == EL0 && !ELUsingAArch32(EL1));
    if known && spsr_mode_is_aarch32 != target_el_is_aarch32 then return TRUE; end;

    // Check for illegal return from AArch32 to AArch64.
    if UsingAArch32() && !spsr_mode_is_aarch32 then return TRUE; end;
    // Check for illegal return to EL1 when HCR_EL2.TGE is set and when either of
    // * SecureEL2 is enabled.
    // * SecureEL2 is not enabled and EL1 is in Non-secure state.
    if EL2Enabled() && target == EL1 && HCR_EL2().TGE == '1' then
        if (!IsSecureBelowEL3() || IsSecureEL2Enabled()) then return TRUE; end;
    end;

    // Check for an illegal return to EL1 by a Guest Hypervisor at EL1 when HCRX_EL2.NVTGE is set.
    if (!Halted() && IsFeatureImplemented(FEAT_NV3) && PSTATE.EL == EL1 && spsr[3:2] == '01' &&
          EffectiveHCR_EL2_NVx() == '101' && IsHCRXEL2Enabled() &&  HCRX_EL2().NVTGE == '1' &&
          NVHCR_EL2().TGE == '1') then
        return TRUE;
    end;

    if (IsFeatureImplemented(FEAT_GCS) && PSTATE.EXLOCK == '0' &&
          PSTATE.EL == target && GetCurrentEXLOCKEN()) then
        return TRUE;
    end;

    return FALSE;
end;
```

Library pseudocode for shared/functions/system/InstrSet

```
// InstrSet
// ========

type InstrSet of enumeration {InstrSet_A64, InstrSet_A32, InstrSet_T32};
```

Library pseudocode for shared/functions/system/InstructionFetchBarrier

```
// InstructionFetchBarrier()
// =========================

impdef func InstructionFetchBarrier()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/InstructionSynchronizationBarrier

```
// InstructionSynchronizationBarrier()
// ===================================

impdef func InstructionSynchronizationBarrier()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/IsASEInstruction

```
// IsASEInstruction()
// ==================
// Returns TRUE if the current instruction is an ASIMD or SVE vector instruction.

impdef func IsASEInstruction() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/system/IsCurrentSecurityState

```
// IsCurrentSecurityState()
// ========================
// Returns TRUE if the current Security state matches
// the given Security state, and FALSE otherwise.

func IsCurrentSecurityState(ss : SecurityState) => boolean
begin
    return CurrentSecurityState() == ss;
end;
```

Library pseudocode for shared/functions/system/IsEventRegisterSet

```
// IsEventRegisterSet()
// ====================
// Return TRUE if the Event Register of this PE is set, and FALSE if it is clear.

func IsEventRegisterSet() => boolean
begin
    return EventRegister == '1';
end;
```

Library pseudocode for shared/functions/system/IsHighestEL

```
// IsHighestEL()
// =============
// Returns TRUE if given exception level is the highest exception level implemented

readonly func IsHighestEL(el : bits(2)) => boolean
begin
    return HighestEL() == el;
end;
```

Library pseudocode for shared/functions/system/IsInHost

```
// IsInHost()
// ==========

readonly func IsInHost() => boolean
begin
    return ELIsInHost(PSTATE.EL);
end;
```

Library pseudocode for shared/functions/system/IsSecureBelowEL3

```
// IsSecureBelowEL3()
// ==================
// Return TRUE if an Exception level below EL3 is in Secure state
// or would be following an exception return to that level.
//
// That is, if at AArch64 EL3 or in AArch32 Monitor mode, whether an
// exception return would pass to Secure or Non-secure state.

readonly func IsSecureBelowEL3() => boolean
begin
    if HaveEL(EL3) then
        return if !HaveAArch64() then SCR().NS == '0' else SCR_EL3().NS == '0';
    elsif HaveEL(EL2) && (!IsFeatureImplemented(FEAT_SEL2) || !HaveAArch64()) then
        // If Secure EL2 is not an architecture option then we must be Non-secure.
        return FALSE;
    else
        // TRUE if PE is Secure or FALSE if Non-secure.
        return SecureOnlyImplementation();
    end;
end;
```

Library pseudocode for shared/functions/system/IsSecureEL2Enabled

```
// IsSecureEL2Enabled()
// ====================
// Returns TRUE if Secure EL2 is enabled, FALSE otherwise.

readonly func IsSecureEL2Enabled() => boolean
begin
    if HaveEL(EL2) && IsFeatureImplemented(FEAT_SEL2) then
        if HaveEL(EL3) then
            if !ELUsingAArch32(EL3) && SCR_EL3().EEL2 == '1' then
                return TRUE;
            else
                return FALSE;
            end;
        else
            return SecureOnlyImplementation();
        end;
    else
        return FALSE;
    end;
end;
```

Library pseudocode for shared/functions/system/IsWFEsleep

```
// IsWFEsleep
// ==========
// Set if PE is in low power mode due to WFE(T) instruction

var IsWFEsleep : boolean;
```

Library pseudocode for shared/functions/system/IsWFIsleep

```
// IsWFIsleep
// ==========
// Set if PE is in low power mode due to WFI(T) instruction

var IsWFIsleep : boolean;
```

Library pseudocode for shared/functions/system/LocalTimeoutEvent

```
// LocalTimeoutEvent()
// ===================
// Returns TRUE if CNTVCT_EL0 equals or exceeds the localtimeout value.

func LocalTimeoutEvent(localtimeout : integer) => boolean
begin
    assert localtimeout >= 0;

    let cntvct : bits(64) = VirtualCounterTimer();
    if UInt(cntvct) >= localtimeout then
        return TRUE;
    end;

    IsLocalTimeoutEventPending = TRUE;
    LocalTimeoutVal = localtimeout[63:0];   // Store value to compare against
                                            // Virtual Counter Timer at subsequent clock ticks
    return FALSE;
end;
```

Library pseudocode for shared/functions/system/Mode

```
// Mode bits
// =========
// AArch32 PSTATE.M mode bits.

constant M32_User : bits(5)    = '10000';
constant M32_FIQ : bits(5)     = '10001';
constant M32_IRQ : bits(5)     = '10010';
constant M32_Svc : bits(5)     = '10011';
constant M32_Monitor : bits(5) = '10110';
constant M32_Abort : bits(5)   = '10111';
constant M32_Hyp : bits(5)     = '11010';
constant M32_Undef : bits(5)   = '11011';
constant M32_System : bits(5)  = '11111';
```

Library pseudocode for shared/functions/system/NonSecureOnlyImplementation

```
// NonSecureOnlyImplementation()
// =============================
// Returns TRUE if the security state is always Non-secure for this implementation.

readonly func NonSecureOnlyImplementation() => boolean
begin
    return !IsFeatureImplemented(FEAT_Secure) && !HaveEL(EL3);
end;
```

Library pseudocode for shared/functions/system/PLOfEL

```
// PLOfEL()
// ========

func PLOfEL(el : bits(2)) => PrivilegeLevel
begin
    case el of
        when EL3 =>  return if !HaveAArch64() then PL1 else PL3;
        when EL2 =>  return PL2;
        when EL1 =>  return PL1;
        when EL0 =>  return PL0;
    end;
end;
```

Library pseudocode for shared/functions/system/PSTATE

```
// PSTATE
// ======
// PE state bits.
// There is no significance to the field order.

var PSTATE : collection {
    N : bits (1),        // Negative condition flag
    Z : bits (1),        // Zero condition flag
    C : bits (1),        // Carry condition flag
    V : bits (1),        // Overflow condition flag
    D : bits (1),        // Debug mask bit                      [AArch64 only]
    A : bits (1),        // SError interrupt mask bit
    I : bits (1),        // IRQ mask bit
    F : bits (1),        // FIQ mask bit
    EXLOCK : bits (1),   // Lock exception return state
    PAN : bits (1),      // Privileged Access Never Bit         [v8.1]
    UAO : bits (1),      // User Access Override                 [v8.2]
    DIT : bits (1),      // Data Independent Timing             [v8.4]
    TCO : bits (1),      // Tag Check Override                  [v8.5, AArch64 only]
    PM : bits (1),       // PMU exception Mask
    BTYPE : bits (2),    // Branch Type                         [v8.5]
    PACM : bits (1),     // PAC instruction modifier
    ZA : bits (1),       // Accumulation array enabled          [SME]
    SM : bits (1),       // Streaming SVE mode enabled          [SME]
    ALLINT : bits (1),   // Interrupt mask bit
    UINJ : bits (1),     // Undefined Exception Injection
    SS : bits (1),       // Software step bit
    IL : bits (1),       // Illegal Execution state bit
    EL : bits (2),       // Exception level
    nRW : bits (1),      // Execution state: 0=AArch64, 1=AArch32
    SP : bits (1),       // Stack pointer select: 0=SP0, 1=SPx [AArch64 only]
    Q : bits (1),        // Cumulative saturation flag          [AArch32 only]
    GE : bits (4),       // Greater than or Equal flags         [AArch32 only]
    SSBS : bits (1),     // Speculative Store Bypass Safe
    IT : bits (8),       // If-then bits, RES0 in CPSR          [AArch32 only]
    J : bits (1),        // J bit, RES0                         [AArch32 only, RES0 in SPSR and CPSR]
    T : bits (1),        // T32 bit, RES0 in CPSR               [AArch32 only]
    E : bits (1),        // Endianness bit                      [AArch32 only]
    M : bits (5)         // Mode field                          [AArch32 only]
};
```

Library pseudocode for shared/functions/system/PhysicalCountInt

```
// PhysicalCountInt()
// ==================
// Returns the integral part of physical count value of the System counter.

func PhysicalCountInt() => bits(64)
begin
    return PhysicalCount[87:24];
end;
```

Library pseudocode for shared/functions/system/PrivilegeLevel

```
// PrivilegeLevel
// ==============
// Privilege Level abstraction.

type PrivilegeLevel of enumeration {PL3, PL2, PL1, PL0};
```

Library pseudocode for shared/functions/system/RestoredITBits

```
// RestoredITBits()
// ================
// Get the value of PSTATE.IT to be restored on this exception return.

func RestoredITBits{N}(spsr : bits(N)) => bits(8)
begin
    let it : bits(8) = spsr[15:10,26:25];

    // When PSTATE.IL is set, it is CONSTRAINED UNPREDICTABLE whether the IT bits are each set
    // to zero or copied from the SPSR.
    if PSTATE.IL == '1' then
        if ConstrainUnpredictableBool(Unpredictable_ILZEROIT) then return '00000000';
        else return it;
        end;
    end;

    // The IT bits are forced to zero when they are set to a reserved value.
    if !IsZero(it[7:4]) && IsZero(it[3:0]) then
        return '00000000';
    end;

    // The IT bits are forced to zero when returning to A32 state, or when returning to an EL
    // with the ITD bit set to 1, and the IT bits are describing a multi-instruction block.
    let itd : bit = if PSTATE.EL == EL2 then HSCTLR().ITD else SCTLR().ITD;
    if (spsr[5] == '0' && !IsZero(it)) || (itd == '1' && !IsZero(it[2:0])) then
        return '00000000';
    else
        return it;
    end;
end;
```

Library pseudocode for shared/functions/system/SecureOnlyImplementation

```
// SecureOnlyImplementation()
// ==========================
// Returns TRUE if the security state is always Secure for this implementation.

readonly func SecureOnlyImplementation() => boolean
begin
    return IsFeatureImplemented(FEAT_Secure) && !HaveEL(EL3);
end;
```

Library pseudocode for shared/functions/system/SecurityState

```
// SecurityState
// =============
// The Security state of an execution context

type SecurityState of enumeration {
    SS_NonSecure,
    SS_Root,
    SS_Realm,
    SS_Secure
};
```

Library pseudocode for shared/functions/system/SecurityStateAtEL

```
// SecurityStateAtEL()
// ===================
// Returns the effective security state at the exception level based off current settings.

readonly func SecurityStateAtEL(EL : bits(2)) => SecurityState
begin
    if IsFeatureImplemented(FEAT_RME) then
        if EL == EL3 then return SS_Root; end;
        let effective_nse_ns : bits(2) = EffectiveSCR_EL3_NSE() :: EffectiveSCR_EL3_NS();
        case effective_nse_ns of
            when '00' =>
                if IsFeatureImplemented(FEAT_SEL2) then
                    return SS_Secure;
                else
                    unreachable;
                end;
            when '01' =>
                return SS_NonSecure;
            when '11' =>
                return SS_Realm;
            otherwise =>
                unreachable;
        end;
    end;

    if !HaveEL(EL3) then
        if SecureOnlyImplementation() then
            return SS_Secure;
        else
            return SS_NonSecure;
        end;
    elsif EL == EL3 then
        return SS_Secure;
    else
        // For EL2 call only when EL2 is enabled in current security state
        assert(EL != EL2 || EL2Enabled());
        if !ELUsingAArch32(EL3) then
            return if EffectiveSCR_EL3_NS() == '1' then SS_NonSecure else SS_Secure;
        else
            return if SCR().NS == '1' then SS_NonSecure else SS_Secure;
        end;
    end;
end;
```

Library pseudocode for shared/functions/system/SendEvent

```
// SendEvent()
// ===========
// Signal an event to all PEs in a multiprocessor system to set their Event Registers.
// When a PE executes the SEV instruction, it causes this function to be executed.

impdef func SendEvent()
begin
    SendEventLocal();
end;
```

Library pseudocode for shared/functions/system/SendEventLocal

```
// SendEventLocal()
// ================
// Set the local Event Register of this PE.
// When a PE executes the SEVL instruction, it causes this function to be executed.

func SendEventLocal()
begin
    EventRegister = '1';
    return;
end;
```

Library pseudocode for shared/functions/system/SetAccumulatedFPExceptions

```
// SetAccumulatedFPExceptions()
// ============================
// Stores FP Exceptions accumulated by the PE.

impdef func SetAccumulatedFPExceptions(accumulated_exceptions : bits(8))
begin
    return;
end;
```

Library pseudocode for shared/functions/system/SetPSTATEFromPSR

```
// SetPSTATEFromPSR()
// ==================

func SetPSTATEFromPSR{N}(spsr : bits(N))
begin
    let illegal_psr_state : boolean = IllegalExceptionReturn{N}(spsr);
    SetPSTATEFromPSR{N}(spsr, illegal_psr_state);
end;

// SetPSTATEFromPSR()
// ==================
// Set PSTATE based on a PSR value

func SetPSTATEFromPSR{N}(spsr_in : bits(N), illegal_psr_state : boolean)
begin
    var spsr : bits(N) = spsr_in;
    let from_aarch64 : boolean = !UsingAArch32();
    PSTATE.SS = DebugExceptionReturnSS{N}(spsr);
    ShouldAdvanceSS = FALSE;
    if illegal_psr_state then
        PSTATE.IL = '1';
        if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = ARBITRARY : bit; end;
        if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = ARBITRARY : bits(2); end;
        if IsFeatureImplemented(FEAT_UAO) then PSTATE.UAO = ARBITRARY : bit; end;
        if IsFeatureImplemented(FEAT_DIT) then PSTATE.DIT = ARBITRARY : bit; end;
        if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = ARBITRARY : bit; end;
        if IsFeatureImplemented(FEAT_PAuth_LR) then PSTATE.PACM = ARBITRARY : bit; end;
        if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = '0'; end;
    else
        // State that is reinstated only on a legal exception return
        PSTATE.IL = spsr[20];
        if IsFeatureImplemented(FEAT_UINJ) then PSTATE.UINJ = spsr[36]; end;
        if spsr[4] == '1' then                    // AArch32 state
            AArch32_WriteMode(spsr[4:0]);         // Sets PSTATE.EL correctly
            if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = spsr[23]; end;
        else                                      // AArch64 state
            PSTATE.nRW = '0';
            (-, PSTATE.EL) = ELFromSPSR{N}(spsr);
            PSTATE.SP  = spsr[0];
            if IsFeatureImplemented(FEAT_BTI) then PSTATE.BTYPE = spsr[11:10]; end;
            if IsFeatureImplemented(FEAT_SSBS) then PSTATE.SSBS = spsr[12]; end;
            if IsFeatureImplemented(FEAT_UAO) then PSTATE.UAO = spsr[23]; end;
            if IsFeatureImplemented(FEAT_DIT) then PSTATE.DIT = spsr[24]; end;
            if IsFeatureImplemented(FEAT_MTE) then PSTATE.TCO = spsr[25]; end;
            if IsFeatureImplemented(FEAT_GCS) then PSTATE.EXLOCK = spsr[34]; end;
            if IsFeatureImplemented(FEAT_PAuth_LR) then
                PSTATE.PACM = if IsPACMEnabled() then spsr[35] else '0';
            end;
        end;
    end;

    // If PSTATE.IL is set, it is CONSTRAINED UNPREDICTABLE whether the T bit is set to zero or
    // copied from SPSR.
    if PSTATE.IL == '1' && PSTATE.nRW == '1' then
        if ConstrainUnpredictableBool(Unpredictable_ILZEROT) then spsr[5] = '0'; end;
    end;

    // State that is reinstated regardless of illegal exception return
    PSTATE.[N,Z,C,V] = spsr[31:28];
    if IsFeatureImplemented(FEAT_PAN) then PSTATE.PAN = spsr[22]; end;
    if PSTATE.nRW == '1' then                     // AArch32 state
        PSTATE.Q         = spsr[27];
        PSTATE.IT        = RestoredITBits{N}(spsr);
        ShouldAdvanceIT  = FALSE;
        if IsFeatureImplemented(FEAT_DIT) then
            PSTATE.DIT = (if (Restarting() || from_aarch64) then spsr[24] else spsr[21]);
        end;
        PSTATE.GE        = spsr[19:16];
        PSTATE.E         = spsr[9];
        PSTATE.[A,I,F]   = spsr[8:6];             // No PSTATE.D in AArch32 state
        PSTATE.T         = spsr[5];               // PSTATE.J is RES0
    else                                          // AArch64 state
        if (IsFeatureImplemented(FEAT_EBEP) || IsFeatureImplemented(FEAT_SPE_EXC) ||
              IsFeatureImplemented(FEAT_TRBE_EXC)) then
            PSTATE.PM    = spsr[32];
        end;
        if IsFeatureImplemented(FEAT_NMI) then PSTATE.ALLINT  = spsr[13]; end;
        PSTATE.[D,A,I,F] = spsr[9:6];             // No PSTATE.[Q,IT,GE,E,T] in AArch64 state
    end;
    return;
end;
```

Library pseudocode for shared/functions/system/ShouldAdvanceHS

```
// ShouldAdvanceHS
// ===============
// Cleared if we should not advance the EDESR.SS after the current instruction.

var ShouldAdvanceHS : boolean;
```

Library pseudocode for shared/functions/system/ShouldAdvanceIT

```
// ShouldAdvanceIT
// ===============
// Cleared if we should not advance the PSTATE.IT after the current instruction.

var ShouldAdvanceIT : boolean;
```

Library pseudocode for shared/functions/system/ShouldAdvanceSS

```
// ShouldAdvanceSS
// ===============
// Cleared if PSTATE.SS is written by the current instruction.

var ShouldAdvanceSS : boolean;
```

Library pseudocode for shared/functions/system/SpeculationBarrier

```
// SpeculationBarrier()
// ====================

impdef func SpeculationBarrier()
begin
    // Since there is no speculation in asl model, this is a NOP.
    return;
end;
```

Library pseudocode for shared/functions/system/SynchronizeContext

```
// SynchronizeContext()
// ====================
// Context Synchronization event, includes Instruction Fetch Barrier effect

impdef func SynchronizeContext()
begin
    InstructionFetchBarrier();
end;
```

Library pseudocode for shared/functions/system/SynchronizeErrors

```
// SynchronizeErrors()
// ===================
// Implements the error synchronization event.

impdef func SynchronizeErrors()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/TakeUnmaskedPhysicalSErrorInterrupts

```
// TakeUnmaskedPhysicalSErrorInterrupts()
// ======================================
// Take any pending unmasked physical SError interrupt.

impdef func TakeUnmaskedPhysicalSErrorInterrupts(iesb_req : boolean)
begin
    return;
end;
```

Library pseudocode for shared/functions/system/TakeUnmaskedSErrorInterrupts

```
// TakeUnmaskedSErrorInterrupts()
// ==============================
// Take any pending unmasked physical SError interrupt or unmasked virtual SError
// interrupt.

impdef func TakeUnmaskedSErrorInterrupts()
begin
    return;
end;
```

Library pseudocode for shared/functions/system/ThisInstr

```
// ThisInstr()
// ===========
// Return the bitstring encoding of the currently executing instruction.

impdef func ThisInstr() => bits(32)
begin
    Unimplemented();
end;
```

Library pseudocode for shared/functions/system/ThisInstrLength

```
// ThisInstrLength()
// =================
// Return the length of the current instruction.

impdef func ThisInstrLength() => integer
begin
    Unimplemented();
end;
```

Library pseudocode for shared/functions/system/UndefinedInjectionCheck

```
// UndefinedInjectionCheck()
// =========================
// Check PSTATE.UINJ to determine if execution of the current
// instruction should cause an Undefined exception.

func UndefinedInjectionCheck()
begin
    if IsFeatureImplemented(FEAT_UINJ) && PSTATE.UINJ == '1' then
        Undefined();
    end;
end;
```

Library pseudocode for shared/functions/system/UsingAArch32

```
// UsingAArch32()
// ==============
// Return TRUE if the current Exception level is using AArch32, FALSE if using AArch64_

readonly func UsingAArch32() => boolean
begin
    let aarch32 : boolean = (PSTATE.nRW == '1');
    if !HaveAArch32() then assert !aarch32; end;
    if !HaveAArch64() then assert aarch32; end;
    return aarch32;
end;
```

Library pseudocode for shared/functions/system/ValidSecurityStateAtEL

```
// ValidSecurityStateAtEL()
// ========================
// Returns TRUE if the current settings and architecture choices for this
// implementation permit a valid Security state at the indicated EL.

func ValidSecurityStateAtEL(el : bits(2)) => boolean
begin
    if !HaveEL(el) then
        return FALSE;
    end;

    if el == EL3 then
        return TRUE;
    end;

    if IsFeatureImplemented(FEAT_RME) then
        let effective_nse_ns : bits(2) = EffectiveSCR_EL3_NSE() :: EffectiveSCR_EL3_NS();
        if effective_nse_ns == '10' then
            return FALSE;
        end;
    end;

    if el == EL2 then
        return EL2Enabled();
    end;

    return TRUE;
end;
```

Library pseudocode for shared/functions/system/VirtualFIQPending

```
// VirtualFIQPending()
// ==================
// Returns TRUE if there is any pending virtual FIQ.

impdef func VirtualFIQPending() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/system/VirtualIRQPending

```
// VirtualIRQPending()
// ==================
// Returns TRUE if there is any pending virtual IRQ.

impdef func VirtualIRQPending() => boolean
begin
    return FALSE;
end;
```

Library pseudocode for shared/functions/system/WFxType

```
// WFxType
// =======
// WFx instruction types.

type WFxType of enumeration {WFxType_WFE, WFxType_WFI, WFxType_WFET, WFxType_WFIT};
```

Library pseudocode for shared/functions/system/WaitForEvent

```
// WaitForEvent()
// ==============
// PE optionally suspends execution until one of the following occurs:
// - A WFE wakeup event.
// - A reset.
// - The implementation chooses to resume execution.
// It is IMPLEMENTATION DEFINED whether restarting execution after the period of
// suspension causes the Event Register to be cleared.

func WaitForEvent()
begin
    if Halted() then return; end;
    if !IsEventRegisterSet() then
        EnterLowPowerState();
    end;
    return;
end;

// WaitForEvent()
// ==============
// PE optionally suspends execution until one of the following occurs:
// - A WFE wakeup event.
// - A reset.
// - The implementation chooses to resume execution.
// - A Wait for Event with Timeout (WFET) is executing, and a local timeout event occurs
// It is IMPLEMENTATION DEFINED whether restarting execution after the period of
// suspension causes the Event Register to be cleared.

func WaitForEvent(localtimeout : integer)
begin
    if Halted() then return; end;
    if !(IsEventRegisterSet() || LocalTimeoutEvent(localtimeout)) then
        EnterLowPowerState();
    end;
    return;
end;
```

Library pseudocode for shared/functions/system/WaitForInterrupt

```
// WaitForInterrupt()
// ==================
// PE optionally suspends execution until one of the following occurs:
// - A WFI wakeup event.
// - A reset.
// - The implementation chooses to resume execution.

func WaitForInterrupt()
begin
    if Halted() then return; end;
    EnterLowPowerState();
    return;
end;

// WaitForInterrupt()
// ==================
// PE optionally suspends execution until one of the following occurs:
// - A WFI wakeup event.
// - A reset.
// - The implementation chooses to resume execution.
// - A Wait for Interrupt with Timeout (WFIT) is executing, and a local timeout event occurs.

func WaitForInterrupt(localtimeout : integer)
begin
    if Halted() then return; end;
    if !LocalTimeoutEvent(localtimeout) then
        EnterLowPowerState();
    end;
    return;
end;
```

Library pseudocode for shared/functions/system/WatchpointRelatedSyndrome

```
// WatchpointRelatedSyndrome()
// ===========================
// Update common Watchpoint related fields.

func WatchpointRelatedSyndrome(fault : FaultRecord) => bits(24)
begin
    var syndrome : bits(24) = Zeros{};

    if fault.watchptinfo.maybe_false_match then
        syndrome[16] = '1';                                     // WPF
    elsif IsFeatureImplemented(FEAT_Debugv8p2) then
        syndrome[16] = ImpDefBit("WPF value on TRUE Watchpoint match");
    end;

    if IsRelaxedWatchpointAccess(fault.accessdesc) then
        if HaltOnBreakpointOrWatchpoint() then
            if ImpDefBool("EDWAR is not valid on watchpoint debug event") then
                syndrome[10] = '1';                             // FnV
            end;
        else
            if ImpDefBool("FAR is not valid on watchpoint exception") then
                syndrome[10] = '1';                             // FnV
            end;
        end;
    else
        if WatchpointFARNotPrecise(fault) then
            syndrome[15] = '1';                                 // FnP
        end;
    end;

    // Watchpoint number is valid if FEAT_Debugv8p9 is implemented or
    // if Feat_Debugv8p2 is implemented and below set of conditions are satisfied:
    // - Any of FnV = 1 or FnP = 1 or WPF = 1.
    // - If the address recorded in FAR is not within a naturally-aligned block of memory.
    // Otherwise, it is IMPLEMENTATION DEFINED if watchpoint number is valid.
    if IsFeatureImplemented(FEAT_Debugv8p9) then
        syndrome[17]    = '1';                                  // WPTV
        syndrome[23:18] = fault.watchptinfo.watchpt_num[5:0];   // WPT
    elsif IsFeatureImplemented(FEAT_Debugv8p2) then
        if syndrome[15] == '1' || syndrome[10] == '1' || syndrome[16] == '1' then
            syndrome[17] = '1';                                 // WPTV
        elsif !AddressInNaturallyAlignedBlock(fault.vaddress, fault.watchptinfo.vaddress) then
            syndrome[17] = '1';                                 // WPTV
        elsif ImpDefBool("WPTV field is valid") then
            syndrome[17] = '1';
        end;
        if syndrome[17] == '1' then
            syndrome[23:18] = fault.watchptinfo.watchpt_num[5:0];    // WPT
        else
            syndrome[23:18] = ARBITRARY : bits(6);
        end;
    end;

    return syndrome;
end;
```

Library pseudocode for shared/functions/tlbi

```
constant VMID_NONE : bits(NUM_VMIDBITS) = Zeros{};
constant ASID_NONE : bits(NUM_ASIDBITS) = Zeros{};
```

Library pseudocode for shared/functions/tlbi/Broadcast

```
// Broadcast
// =========

type Broadcast of enumeration {
    Broadcast_ISH,
    Broadcast_ForcedISH,
    Broadcast_ISH_CnP,          // Broadcast only to PEs which would share TLB entries if CnP is 1.
    Broadcast_OSHnISH,          // Broadcast only to PEs in the Outer shareability domain, but not
                                // in the Inner shareability domain.
    Broadcast_OSH,
    Broadcast_NSH
};
```

Library pseudocode for shared/functions/tlbi/BroadcastTLBI

```
// BroadcastTLBI()
// ===============
// IMPLEMENTATION DEFINED function to broadcast TLBI operation within the indicated broadcast
// domain.

impdef func BroadcastTLBI(broadcast : Broadcast, r : TLBIRecord)
begin
    BroadcastTLBI(broadcast, r, DEFAULT_TLBI_DOMAIN);
end;

impdef func BroadcastTLBI(broadcast : Broadcast, r : TLBIRecord, domains : bits(16))
begin
    return;
end;
```

Library pseudocode for shared/functions/tlbi/ExcludeXS

```
// ExcludeXS()
// ===========
// Returns TRUE if a TLBI maintenance instruction without the nXS qualifier executed at EL1
// behaves in the same way as the corresponding TLBI maintenance instruction with the nXS qualifier.

func ExcludeXS() => boolean
begin
    return (IsFeatureImplemented(FEAT_XS) && PSTATE.EL == EL1 &&
            IsHCRXEL2Enabled() && HCRX_EL2().FnXS == '1');
end;
```

Library pseudocode for shared/functions/tlbi/TLBI

```
// TLBI()
// ======
// Invalidates TLB entries for which TLBIMatch() returns TRUE.

impdef func TLBI(r : TLBIRecord)
begin
    return;
end;
```

Library pseudocode for shared/functions/tlbi/TLBILevel

```
// TLBILevel
// =========

type TLBILevel of enumeration {
    TLBILevel_Any,        // this applies to TLB entries at all levels
    TLBILevel_Last        // this applies to TLB entries at last level only
};
```

Library pseudocode for shared/functions/tlbi/TLBIMemAttr

```
// TLBIMemAttr
// ===========
// Defines the attributes of the memory operations that must be completed in
// order to deem the TLBI operation as completed.

type TLBIMemAttr of enumeration {
    TLBI_AllAttr,         // All TLB entries within the scope of the invalidation
    TLBI_ExcludeXS        // Only TLB entries with XS=0 within the scope of the invalidation
};
```

Library pseudocode for shared/functions/tlbi/TLBIOp

```
// TLBIOp
// ======

type TLBIOp of enumeration {
    TLBIOp_DALL,          // AArch32 Data TLBI operations - deprecated
    TLBIOp_DASID,
    TLBIOp_DVA,
    TLBIOp_IALL,          // AArch32 Instruction TLBI operations - deprecated
    TLBIOp_IASID,
    TLBIOp_IVA,
    TLBIOp_ALL,
    TLBIOp_ASID,
    TLBIOp_IPAS2,
    TLBIPOp_IPAS2,
    TLBIOp_VAA,
    TLBIOp_VA,
    TLBIPOp_VAA,
    TLBIPOp_VA,
    TLBIOp_VMALL,
    TLBIOp_VMALLS12,
    TLBIOp_RIPAS2,
    TLBIPOp_RIPAS2,
    TLBIOp_RVAA,
    TLBIOp_RVA,
    TLBIPOp_RVAA,
    TLBIPOp_RVA,
    TLBIOp_RPA,
    TLBIOp_PAALL,
    TLBIOp_VMALLWS2
};
```

Library pseudocode for shared/functions/tlbi/TLBIRecord

```
// TLBIRecord
// ==========
// Details related to a TLBI operation.

type TLBIRecord of record {
    op : TLBIOp,
    from_aarch64 : boolean, // originated as an AArch64 operation
    security : SecurityState,
    regime : Regime,
    use_vmid : boolean,
    vmid : bits(NUM_VMIDBITS),
    asid : bits(NUM_ASIDBITS),
    level : TLBILevel,
    attr : TLBIMemAttr,
    ipaspace : PASpace,     // For operations that take IPA as input address
    address : bits(64),     // input address, for range operations, start address
    end_address : bits(64), // for range operations, end; address
    d64 : boolean,          // For operations that evict VMSAv8-64 based TLB entries
    d128 : boolean,         // For operations that evict VMSAv9-128 based TLB entries
    ttl : bits(4),          // translation table walk level holding the leaf entry
                            // for the address being invalidated
                            // For Non-Range Invalidations:
                            //   When the ttl is
                            //     '00xx'    : this applies to all TLB entries
                            //     Otherwise : TLBIP instructions invalidates D128 TLB
                            //                 entries only
                            //                 TLBI instructions invalidates D64 TLB
                            //                 entries only
                            // For Range Invalidations:
                            //   When the ttl is
                            //     '00'      : this applies to all TLB entries
                            //     Otherwise : TLBIP instructions invalidates D128 TLB
                            //                 entries only
                            //                 TLBI instructions invalidates D64 TLB
                            //                 entries only
    tg : bits(2)            // for range operations, translation granule
};
```

Library pseudocode for shared/functions/tlbi/VMID

```
// VMID
// ====
// Effective VMID.

func VMID() => bits(NUM_VMIDBITS)
begin
    if EL2Enabled() then
        if !ELUsingAArch32(EL2) then
            if IsFeatureImplemented(FEAT_VMID16) && VTCR_EL2().VS == '1' then
                return VTTBR_EL2().VMID;
            else
                return ZeroExtend{NUM_VMIDBITS}(VTTBR_EL2().VMID[7:0]);
            end;
        else
            return ZeroExtend{NUM_VMIDBITS}(VTTBR().VMID);
        end;
    elsif HaveEL(EL2) && IsFeatureImplemented(FEAT_SEL2) then
        return Zeros{NUM_VMIDBITS};
    else
        return VMID_NONE;
    end;
end;
```

Library pseudocode for shared/functions/unbounded_configs/Maximum

```
// Maximum walk levels
// ===================
let MAX_WALK_LEVELS : integer{} = (if IsFeatureImplemented(FEAT_D128) then 6
                                   else (if IsFeatureImplemented(FEAT_LPA2) then 5
                                   else 4));
```

Library pseudocode for shared/functions/unbounded_configs/Unbounded

```
// Unbounded Variables
// ===================
// Sets the upper limit for a specific loop or function.

constant Unbounded : integer{} = 2^128;
config Unbounded_DiscardTransactionalWrites : integer{0..Unbounded} = Unbounded;
config Unbounded_DescriptorUpdate : integer{0..Unbounded} = Unbounded;
config Unbounded_semihost : integer{0..Unbounded} = Unbounded;
config Unbounded_ParseTrace : integer{0..Unbounded} = Unbounded;
config Unbounded_ParseExtensionPacket : integer{0..Unbounded} = Unbounded;
config Unbounded_ULEB128 : integer{0..Unbounded} = Unbounded;
config Unbounded_ULEB128n : integer{0..Unbounded} = Unbounded;
config Unbounded_FloorPow2 : integer{0..Unbounded} = Unbounded;
config Unbounded_NormalizeReal : integer{0..Unbounded} = Unbounded;
```

Library pseudocode for shared/functions/unpredictable/ConstrainUnpredictable

```
// ConstrainUnpredictable()
// ========================
// Return the appropriate Constraint result to control the caller's behavior.
// The return value is IMPLEMENTATION DEFINED within a permitted list for each
// UNPREDICTABLE case.
// (The permitted list is determined by an assert or case statement at the call site.)

readonly impdef func ConstrainUnpredictable(which : Unpredictable) => Constraint
begin
    case which of
        when Unpredictable_VMSR =>
            return Constraint_UNDEF;
        when Unpredictable_WBOVERLAPLD =>
            return Constraint_WBSUPPRESS; // return loaded value
        when Unpredictable_WBOVERLAPST =>
            return Constraint_NONE;      // store pre-writeback value
        when Unpredictable_LDPOVERLAP =>
            return Constraint_UNDEF;     // instruction is UNDEFINED
        when Unpredictable_BASEOVERLAP =>
            return Constraint_UNKNOWN;   // use UNKNOWN address
        when Unpredictable_DATAOVERLAP =>
            return Constraint_UNKNOWN;   // store UNKNOWN value
        when Unpredictable_DEVPAGE2 =>
            return Constraint_FAULT;     // take an alignment fault
        when Unpredictable_RESTCF =>
            return Constraint_UNKNOWN;   // Do not take a fault
        when Unpredictable_DEVICETAGSTORE =>
            return Constraint_NONE;      // Do not take a fault
        when Unpredictable_INSTRDEVICE =>
            return Constraint_NONE;      // Do not take a fault
        when Unpredictable_RESCPACR =>
            return Constraint_TRUE;      // Map to UNKNOWN value
        when Unpredictable_RESMAIR =>
            return Constraint_UNKNOWN;   // Map to UNKNOWN value
        when Unpredictable_S1CTAGGED =>
            return Constraint_FALSE;     // SCTLR_ELx.C == '0' marks address as untagged
        when Unpredictable_S2RESMEMATTR =>
            return Constraint_NC;        // Map to Noncacheable value
        when Unpredictable_RESTEXCB =>
            return Constraint_UNKNOWN;   // Map to UNKNOWN value
        when Unpredictable_RESDACR =>
            return Constraint_UNKNOWN;   // Map to UNKNOWN value
        when Unpredictable_RESPRRR =>
            return Constraint_UNKNOWN;   // Map to UNKNOWN value
        when Unpredictable_RESVTCRS =>
            return Constraint_UNKNOWN;   // Map to UNKNOWN value
        when Unpredictable_RESTnSZ =>
            return Constraint_FORCE;     // Map to the limit value
        when Unpredictable_ESRCONDPASS =>
            return Constraint_FALSE;     // Report as "AL"
        when Unpredictable_ILZEROIT =>
            return Constraint_FALSE;     // Do not zero PSTATE.IT
        when Unpredictable_ILZEROT =>
            return Constraint_FALSE;     // Do not zero PSTATE.T
        when Unpredictable_BPVECTORCATCHPRI =>
            return Constraint_TRUE;      // Debug Vector Catch: match on 2nd halfword
        when Unpredictable_VCMATCHHALF =>
            return Constraint_FALSE;     // No match
        when Unpredictable_VCMATCHDAPA =>
            return Constraint_FALSE;     // No match on Data Abort or Prefetch abort
        when Unpredictable_WPMASKANDBAS =>
            return Constraint_FALSE;     // Watchpoint disabled
        when Unpredictable_WPBASCONTIGUOUS =>
            return Constraint_FALSE;     // Watchpoint disabled
        when Unpredictable_RESWPMASK =>
            return Constraint_DISABLED; // Watchpoint disabled
        when Unpredictable_WPMASKEDBITS =>
            return Constraint_FALSE;     // Watchpoint disabled
        when Unpredictable_RESBPWPCTRL =>
            return Constraint_DISABLED; // Breakpoint/watchpoint disabled
        when Unpredictable_BPNOTIMPL =>
            return Constraint_DISABLED; // Breakpoint disabled
        when Unpredictable_RESBPTYPE =>
            return Constraint_DISABLED; // Breakpoint disabled
        when Unpredictable_BPNOTCTXCMP =>
            return Constraint_DISABLED; // Breakpoint disabled
        when Unpredictable_RESMDSELR =>
            return Constraint_UNKNOWN;   // Map to UNKNOWN value
        when Unpredictable_BPMATCHHALF =>
            return Constraint_FALSE;     // No match
        when Unpredictable_BPMISMATCHHALF =>
            return Constraint_FALSE;     // No match
        when Unpredictable_BPLINKINGDISABLED =>
            return Constraint_FALSE;     // Breakpoint does not match
        when Unpredictable_RESTARTALIGNPC =>
            return Constraint_FALSE;     // Do not force alignment
        when Unpredictable_RESTARTZEROUPPERPC =>
            return Constraint_TRUE;      // Force zero extension
        when Unpredictable_ZEROUPPER =>
```

```
        return Constraint_TRUE;     // zero top halves of X registers
    when Unpredictable_ERETZEROUPPERPC =>
        return Constraint_TRUE;     // zero top half of PC
    when Unpredictable_A32FORCEALIGNPC =>
        return Constraint_FALSE;    // Do not force alignment
    when Unpredictable_SMD =>
        return Constraint_UNDEF;    // disabled SMC is Unallocated
    when Unpredictable_NONFAULT =>
        return Constraint_FALSE;    // Speculation enabled
    when Unpredictable_SVEZEROUPPER =>
        return Constraint_TRUE;     // zero top bits of Z registers
    when Unpredictable_SVELDNFDATA =>
        return Constraint_TRUE;     // Load mem data in NF loads
    when Unpredictable_SVELDNFZERO =>
        return Constraint_TRUE;     // Write zeros in NF loads
    when Unpredictable_CHECKSPNONEACTIVE =>
        return Constraint_TRUE;     // Check SP alignment
    when Unpredictable_SMEZEROUPPER =>
        return Constraint_TRUE;     // zero top bits of ZA registers
    when Unpredictable_NVNV1 =>
        return Constraint_NVNV1_00; // Map unpredictable configuration of HCR_EL2[NV,NV1]
                                    // to NV = 0 and NV1 = 0
    when Unpredictable_BADDR_RESS =>
        return Constraint_RESS;     // Values behave as RESS
    when Unpredictable_Shareability =>
        return Constraint_OSH;      // Map reserved encoding of shareability to Outer Shareable
    when Unpredictable_AFUPDATE =>  // AF update for alignment or Permission fault
        return Constraint_TRUE;
    when Unpredictable_DBUPDATE =>  // DB State update for alignment fault
        return Constraint_TRUE;
    when Unpredictable_Unsupported_Atomic_HW_Update =>
        return Constraint_FALSE;    // Do not raise Unsupported Atomic HW Update fault
                                    // when encountering unsupported memory attributes
    when Unpredictable_IESBinDebug =>  // Use SCTLR_ELx[].IESB in Debug state
        return Constraint_TRUE;
    when Unpredictable_BADPMSFCR =>    // Bad settings for PMSFCR_EL1/PMSEVFR_EL1/PMSLATFR_EL1
        return Constraint_TRUE;
    when Unpredictable_PMBPTR_Fault_AddressSize =>
        return Constraint_FALSE;
    when Unpredictable_RES_PMU_VS =>   // Bad setting for SVE Streaming mode filter
        return Constraint_FALSE;
    when Unpredictable_COUNT_CHAIN =>  // Chained PMU counters idx, idx+1 are not in same range
        return Constraint_FALSE;
    when Unpredictable_EL1TIMESTAMP => // Bad settings for TRFCR_EL1
        return Constraint_EL1TIMESTAMP;
    when Unpredictable_EL2TIMESTAMP => // Bad settings for TRFCR_EL2
        return Constraint_EL2TIMESTAMP;
    when Unpredictable_ZEROBTYPE =>
        return Constraint_TRUE;           // Save BTYPE in SPSR_ELx/DPSR_EL0 as '00'
    when Unpredictable_CLEARERRITEZERO => // Clearing sticky errors when instruction in flight
        return Constraint_FALSE;
    when Unpredictable_ALUEXCEPTIONRETURN =>
        return Constraint_UNDEF;
    when Unpredictable_DBGxVR_RESS =>
        return Constraint_FALSE;
    when Unpredictable_PMSCR_PCT =>
        return Constraint_PMSCR_PCT_VIRT;
    when Unpredictable_WFxTDEBUG =>
        return Constraint_FALSE;    // WFxT in Debug state does not execute as a NOP
    // Accesses are not single-copy atomic above the byte level
    when Unpredictable_LS64UNSUPPORTED =>
        return Constraint_LIMITED_ATOMICITY;
    // Unaligned exclusives, atomics, acquire/release
    // to a region that is not to Normal inner write-back
    // outer write-back shareable generate an Alignment fault.
    when Unpredictable_LSE2_ALIGNMENT_FAULT =>
        return Constraint_FALSE;
    when Unpredictable_LSE128OVERLAP =>
        return Constraint_UNDEF;    // instruction is UNDEFINED
    when Unpredictable_IGNORETRAPINDEBUG =>
        return Constraint_FALSE;    // Trap to register access in debug state is enabled
    when Unpredictable_PMUEVENTCOUNTER =>
        return Constraint_UNDEF;    // Accesses to the register are UNDEFINED
    when Unpredictable_RES_HPMN =>
        return Constraint_UNKNOWN;  // The counter is reserved for EL2 use
    when Unpredictable_RES_EPMN =>
        return Constraint_UNKNOWN;  // The counter is reserved for external use
    when Unpredictable_BRBFILTRATE =>
        return Constraint_FALSE;    // BRB_FILTRATE event not generated on BRB injection
    when Unpredictable_PMUSNAPSHOTEVENT => // PMU_SNAPSHOT event not generated in Debug state
        return Constraint_FALSE;
    when Unpredictable_RESEPMSSAD =>
        // External access to PMUv3 Snapshot extension allowed in Secure state
        // when FEAT_RME is not implemented or Root state otherwise
        return Constraint_UNKNOWN;
    when Unpredictable_RESPMSSE =>
        return Constraint_DISABLED;   // PMU capture events disabled
    when Unpredictable_RESPMEE =>
        return Constraint_DISABLED;   // PMU Profiling exception disabled, PMUIRQ enabled
```

```
        when Unpredictable_RESPMSEE =>
            return Constraint_DISABLED;   // SPE Profiling exception disabled
        when Unpredictable_RESTRFEE =>
            return Constraint_DISABLED;   // TRBE Profiling exception disabled
        when Unpredictable_TRBPTR_Fault_AddressSize =>
            return Constraint_TRUE;       // TRBPTR_EL1[63:(OAMax)] NonZero causes Address Size fault
        when Unpredictable_RESTC =>
            return Constraint_DISABLED;   // Threshold features disabled
        when Unpredictable_MOPSOVERLAP => // Instruction is UNDEFINED
            return Constraint_UNDEF;
        when Unpredictable_MOPS_R31 =>    // Instruction is UNDEFINED
            return Constraint_UNDEF;
        when Unpredictable_CASRETURNOLDVALUE =>
            return Constraint_TRUE;
        when Unpredictable_WRITEFAILEDCAS =>
            return Constraint_FALSE;
        when Unpredictable_WRITEFAILEDRCWCAS =>
            return Constraint_FALSE;
        when Unpredictable_STRONLYTAGCHECKEDCAS =>
            return Constraint_TRUE;      // CAS with compare fail does a Tag Check
        when Unpredictable_STRONLYTAGCHECKEDRCWSCAS =>
            return Constraint_TRUE;      // RCW(S)CAS with RCW(S) fail/compare fail does a Tag Check
        when Unpredictable_RESERVEDNSxB =>
            return Constraint_MAPTOALLOCATED;
        when Unpredictable_RESERVEDNSxB_Trap =>
            return Constraint_TRUE;
        when Unpredictable_RES_ETBAD =>
            // ExternalTracebufferaccess disabled for res values
            return Constraint_DISABLED;
        when Unpredictable_RESBPMASK =>
            return Constraint_DISABLED; // Mask set to 0
        when Unpredictable_BPMASK =>
            return Constraint_FALSE;     // Breakpoint match will fail
        when Unpredictable_BPMASKEDBITS =>
            return Constraint_FALSE;     // Breakpoint match will fail
        when Unpredictable_BPLINKEDADDRMATCH =>
            return Constraint_FALSE;     // Breakpoint match will fail
        when Unpredictable_16BYTEROUNDEDUPACCESS =>
            // Watchpoint match of 16 byte rounded range in case of SVE access
            return Constraint_TRUE;
        when Unpredictable_16BYTEROUNDEDDOWNACCESS =>
            // Watchpoint match of 16 byte rounded range in case of SVE access
            return Constraint_TRUE;
        when Unpredictable_NODTRTAGCHK =>
            // The load or store operation does not perform the Tag check in Debug state
            return Constraint_TRUE;
        when Unpredictable_CPACHECK =>      // Do not perform Checked Pointer Arithmetic
            return Constraint_FALSE;
        when Unpredictable_Atomic_SYNC_ABORT =>
            return Constraint_FALSE;
        when Unpredictable_Atomic_SERROR =>
            return Constraint_FALSE;
        when Unpredictable_Atomic_NOP =>
            return Constraint_FALSE;
        when Unpredictable_Atomic_MMU_IMPDEF_FAULT =>
            return Constraint_FALSE;
        when Unpredictable_MPAM_TRUNCATE_VID =>
            return Constraint_TRUE;
        when Unpredictable_USE_DEFAULT_PMG =>
            return Constraint_TRUE;
        when Unpredictable_OUT_OF_RANGE_ID =>
            return Constraint_DEFAULT_ID;
        when Unpredictable_OUT_OF_RANGE_VID =>
            return Constraint_DEFAULT_ID;
        when Unpredictable_BankedRegister =>
            return Constraint_NOP;
        when Unpredictable_UnimplementedRegister =>
            return Constraint_NOP;
        when Unpredictable_RES_RA =>
            return Constraint_TRUE;
        when Unpredictable_BPNOTLINKING =>
            return Constraint_FALSE;
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/functions/unpredictable/ConstrainUnpredictableBits

```
// ConstrainUnpredictableBits()
// ============================

// This is a variant of ConstrainUnpredictable for when the result can be Constraint_UNKNOWN.
// If the result is Constraint_UNKNOWN then the function also returns UNKNOWN value, but that
// value is always an allocated value; that is, one for which the behavior is not itself
// CONSTRAINED.

readonly impdef func ConstrainUnpredictableBits{width}(which : Unpredictable
                                                       ) => (Constraint, bits(width))
begin
    let c : Constraint = ConstrainUnpredictable(which);

    case c of
        when Constraint_UNKNOWN =>
            return (c, Zeros{width});     // See notes; this is an example implementation only
        when Constraint_PMSCR_PCT_VIRT =>
            return (c, Zeros{width});
        when Constraint_NC where which == Unpredictable_S2RESMEMATTR =>
            assert width == 4;
            return (c, ZeroExtend{width}('0101'));
        when Constraint_WT where which == Unpredictable_S2RESMEMATTR =>
            assert width == 4;
            return (c, ZeroExtend{width}('1010'));
        when Constraint_WB where which == Unpredictable_S2RESMEMATTR =>
            assert width == 4;
            return (c, ZeroExtend{width}('1111'));
        when Constraint_EL1TIMESTAMP =>
            return (c, ZeroExtend{width}('01'));
        when Constraint_EL2TIMESTAMP =>
            return (c, ZeroExtend{width}('01'));
        when Constraint_MAPTOALLOCATED =>
            return (c, ZeroExtend{width}('010'));  // return non-secure state_bits
        otherwise =>
            return (c, ARBITRARY : bits(width));   // bits result not used
    end;
end;
```

Library pseudocode for shared/functions/unpredictable/ConstrainUnpredictableBool

```
// ConstrainUnpredictableBool()
// ============================
// This is a variant of the ConstrainUnpredictable function where the result is either
// Constraint_TRUE or Constraint_FALSE.

readonly impdef func ConstrainUnpredictableBool(which : Unpredictable) => boolean
begin
    let c : Constraint = ConstrainUnpredictable(which);
    assert c IN {Constraint_TRUE, Constraint_FALSE};
    return (c == Constraint_TRUE);
end;
```

Library pseudocode for shared/functions/unpredictable/ConstrainUnpredictableInteger

```
// ConstrainUnpredictableInteger()
// ===============================
// This is a variant of ConstrainUnpredictable for when the result can be Constraint_UNKNOWN.
// If the result is Constraint_UNKNOWN then the function also returns an UNKNOWN
// value in the range low to high, inclusive.

impdef func ConstrainUnpredictableInteger(low : integer,
                                          high : integer,
                                          which : Unpredictable) => (Constraint,integer)
begin
    let c : Constraint = ConstrainUnpredictable(which);
    if c == Constraint_UNKNOWN then
        return (c, low);                 // See notes; this is an example implementation only
    else
        return (c, ARBITRARY : integer); // integer result not used
    end;
end;
```

Library pseudocode for shared/functions/unpredictable/ConstrainUnpredictableProcedure

```
// ConstrainUnpredictableProcedure()
// =================================
// This is a variant of ConstrainUnpredictable that implements a Constrained
// Unpredictable behavior for a given Unpredictable situation.
// The behavior is within permitted behaviors for a given Unpredictable situation,
// these are documented in the textual part of the architecture specification.
//
// This function is expected to be refined in an IMPLEMENTATION DEFINED manner.
// The details of possible outcomes may not be present in the code and must be interpreted
// for each use with respect to the CONSTRAINED UNPREDICTABLE specifications
// for the specific area.

impdef func ConstrainUnpredictableProcedure(which : Unpredictable)
begin
    case which of
        when Unpredictable_PMUEVENTCOUNTER =>
            Undefined();
        when Unpredictable_MRC_APSR_TARGET =>
            PSTATE.[N,Z,C,V] = DBGDSCRint()[31:28];
        otherwise =>
            ImpDef("IMPLEMENTATION_DEFINED");
    end;
    return;
end;
```

Library pseudocode for shared/functions/unpredictable/Constraint

```
// Constraint
// ==========
// List of Constrained Unpredictable behaviors.

type Constraint of enumeration {
                          // General
                          Constraint_NONE,              // Instruction executes with
                                                        // no change or side-effect
                                                        // to its described behavior
                          Constraint_UNKNOWN,           // Destination register
                                                        // has UNKNOWN value
                          Constraint_UNDEF,             // Instruction is UNDEFINED
                          Constraint_UNDEFEL0,          // Instruction is UNDEFINED at EL0 only
                          Constraint_NOP,               // Instruction executes as NOP
                          Constraint_TRUE,
                          Constraint_FALSE,
                          Constraint_DISABLED,
                          Constraint_UNCOND,            // Instruction executes unconditionally
                          Constraint_COND,              // Instruction executes conditionally
                          Constraint_ADDITIONAL_DECODE, // Instruction executes
                                                        // with additional decode
                          // Load-store
                          Constraint_WBSUPPRESS,
                          Constraint_FAULT,
                          Constraint_LIMITED_ATOMICITY, // Accesses are not
                                                        // single-copy atomic
                                                        // above the byte level
                          Constraint_NVNV1_00,
                          Constraint_NVNV1_01,
                          Constraint_NVNV1_11,
                          Constraint_EL1TIMESTAMP,      // Constrain to Virtual Timestamp
                          Constraint_EL2TIMESTAMP,      // Constrain to Virtual Timestamp
                          Constraint_DEFAULT_ID,        // Use default PARTID/PM
                          Constraint_ANY_ID,            // Use any in-range PARTID/PMG
                          Constraint_OSH,               // Constrain to Outer Shareable
                          Constraint_ISH,               // Constrain to Inner Shareable
                          Constraint_NSH,               // Constrain to Nonshareable

                          Constraint_NC,                // Constrain to Noncacheable
                          Constraint_WT,                // Constrain to Writethrough
                          Constraint_WB,                // Constrain to Writeback
                          Constraint_RESS,              // Bits behave as RESS for all purposes
                                                        // other than reading back the register
                          Constraint_ALLRESS,           // Bits behave as RESS for all purposes

                          // IPA too large
                          Constraint_FORCE, Constraint_FORCENOSLCHECK,
                          // An unallocated System register value maps onto an allocated value
                          Constraint_MAPTOALLOCATED,
                          // PMSCR_PCT reserved values select Virtual timestamp
                          Constraint_PMSCR_PCT_VIRT,
                          Constraint_ANYREG,            // Any allocated register is chosen
};
```

Library pseudocode for shared/functions/unpredictable/Unpredictable

```
// Unpredictable
// =============
// List of Constrained Unpredictable situations.

type Unpredictable of enumeration {
                            // VMSR on MVFR
                            Unpredictable_VMSR,
                            // Writeback/transfer register overlap (load)
                            Unpredictable_WBOVERLAPLD,
                            // Writeback/transfer register overlap (store)
                            Unpredictable_WBOVERLAPST,
                            // Load Pair transfer register overlap
                            Unpredictable_LDPOVERLAP,
                            // Store-exclusive base/status register overlap
                            Unpredictable_BASEOVERLAP,
                            // Store-exclusive data/status register overlap
                            Unpredictable_DATAOVERLAP,
                            // Load-store alignment checks
                            Unpredictable_DEVPAGE2,
                            // Instruction fetch from Device memory
                            Unpredictable_INSTRDEVICE,
                            // Reserved CPACR value
                            Unpredictable_RESCPACR,
                            // Reserved MAIR value
                            Unpredictable_RESMAIR,
                            // Effect of SCTLR_ELx.C on Tagged attribute
                            Unpredictable_S1CTAGGED,
                            // Reserved Stage 2 MemAttr value
                            Unpredictable_S2RESMEMATTR,
                            // Reserved TEX::C::B value
                            Unpredictable_RESTEXCB,
                            // Reserved PRRR value
                            Unpredictable_RESPRRR,
                            // Reserved DACR field
                            Unpredictable_RESDACR,
                            // Reserved VTCR.S value
                            Unpredictable_RESVTCRS,
                            // Reserved or out-of-range TCR.TnSZ value
                            Unpredictable_RESTnSZ,
                            // Reserved SCTLR_ELx.TCF value
                            Unpredictable_RESTCF,
                            // Tag stored to Device memory
                            Unpredictable_DEVICETAGSTORE,
                            // Syndrome for a known-passing conditional A32 instruction
                            Unpredictable_ESRCONDPASS,
                            // Illegal State exception: zero PSTATE.IT
                            Unpredictable_ILZEROIT,
                            // Illegal State exception: zero PSTATE.T
                            Unpredictable_ILZEROT,
                            // Debug: prioritization of Vector Catch
                            Unpredictable_BPVECTORCATCHPRI,
                            // Debug Vector Catch: match on 2nd halfword
                            Unpredictable_VCMATCHHALF,
                            // Debug Vector Catch: match on Data Abort
                            // or Prefetch abort
                            Unpredictable_VCMATCHDAPA,
                            // Debug watchpoints: nonzero MASK and non-ones BAS
                            Unpredictable_WPMASKANDBAS,
                            // Debug watchpoints: non-contiguous BAS
                            Unpredictable_WPBASCONTIGUOUS,
                            // Debug watchpoints: reserved MASK
                            Unpredictable_RESWPMASK,
                            // Debug watchpoints: nonzero MASKed bits of address
                            Unpredictable_WPMASKEDBITS,
                            // Debug breakpoints and watchpoints: reserved control bits
                            Unpredictable_RESBPWPCTRL,
                            // Debug breakpoints: not implemented
                            Unpredictable_BPNOTIMPL,
                            // Debug breakpoints: reserved type
                            Unpredictable_RESBPTYPE,
                            // Debug breakpoints and watchpoints: reserved MDSELR_EL1.BANK
                            Unpredictable_RESMDSELR,
                            // Debug breakpoints: not-context-aware breakpoint
                            Unpredictable_BPNOTCTXCMP,
                            // Debug breakpoints: match on 2nd halfword of instruction
                            Unpredictable_BPMATCHHALF,
                            // Debug breakpoints: mismatch on 2nd halfword of instruction
                            Unpredictable_BPMISMATCHHALF,
                            // Debug breakpoints: a breakpoint is linked to that is not
                            // programmed with linking enabled
                            Unpredictable_BPLINKINGDISABLED,
                            // Debug breakpoints: reserved MASK
                            Unpredictable_RESBPMASK,
                            // Debug breakpoints: MASK is set for a Context matching
                            // breakpoint or when DBGBCR_EL1(n).BAS != '1111'
                            Unpredictable_BPMASK,
                            // Debug breakpoints: nonzero MASKed bits of address
                            Unpredictable_BPMASKEDBITS,
                            // Debug breakpoints: A linked breakpoint is
```

```
                        // linked to an address matching breakpoint
                        Unpredictable_BPLINKEDADDRMATCH,
                        // Debug: restart to a misaligned AArch32 PC value
                        Unpredictable_RESTARTALIGNPC,
                        // Debug: restart to a not-zero-extended AArch32 PC value
                        Unpredictable_RESTARTZEROUPPERPC,
                        // Zero top 32 bits of X registers in AArch32 state
                        Unpredictable_ZEROUPPER,
                        // Zero top 32 bits of PC on illegal return to
                        // AArch32 state
                        Unpredictable_ERETZEROUPPERPC,
                        // Force address to be aligned when interworking
                        // branch to A32 state
                        Unpredictable_A32FORCEALIGNPC,
                        // SMC disabled
                        Unpredictable_SMD,
                        // FF speculation
                        Unpredictable_NONFAULT,
                        // Zero top bits of Z registers in EL change
                        Unpredictable_SVEZEROUPPER,
                        // Load mem data in NF loads
                        Unpredictable_SVELDNFDATA,
                        // Write zeros in NF loads
                        Unpredictable_SVELDNFZERO,
                        // SP alignment fault when predicate is all zero
                        Unpredictable_CHECKSPNONEACTIVE,
                        // Zero top bits of ZA registers in EL change
                        Unpredictable_SMEZEROUPPER,
                        // Watchpoint match of last rounded up memory access in case of
                        // 16 byte rounding
                        Unpredictable_16BYTEROUNDEDUPACCESS,
                        // Watchpoint match of first rounded down memory access in case of
                        // 16 byte rounding
                        Unpredictable_16BYTEROUNDEDDOWNACCESS,
                        // HCR_EL2.[NV,NV1] == '01'
                        Unpredictable_NVNV1,
                        // Upper bits of a BADDR are not RESS
                        Unpredictable_BADDR_RESS,
                        // Reserved shareability encoding
                        Unpredictable_Shareability,
                        // Access Flag Update by HW
                        Unpredictable_AFUPDATE,
                        // Dirty Bit State Update by HW
                        Unpredictable_DBUPDATE,
                        // Unsupported memory attributes for HW update
                        Unpredictable_Unsupported_Atomic_HW_Update,
                        // Consider SCTLR_ELx().IESB in Debug state
                        Unpredictable_IESBinDebug,
                        // Bad settings for PMSFCR_EL1/PMSEVFR_EL1/PMSLATFR_EL1
                        Unpredictable_BADPMSFCR,
                        // Zero saved BType value in SPSR_ELx/DPSR_EL0
                        Unpredictable_ZEROBTYPE,
                        // Timestamp constrained to virtual or physical
                        Unpredictable_EL2TIMESTAMP,
                        Unpredictable_EL1TIMESTAMP,
                         // Reserved MDCR_EL3.[NSTBE,NSTB] or MDCR_EL3.[NSPBE,NSPB] value
                         Unpredictable_RESERVEDNSxB,
                         Unpredictable_RESERVEDNSxB_Trap,
                        // WFET or WFIT instruction in Debug state
                        Unpredictable_WFxTDEBUG,
                        // Address does not support LS64 instructions
                        Unpredictable_LS64UNSUPPORTED,
                        // Unaligned exclusives, atomics, acquire/release
                        // to a region that is not to Normal inner write-back
                        // outer write-back shareable generate an Alignment fault.
                        Unpredictable_LSE2_ALIGNMENT_FAULT,
                        // 128-bit Atomic or 128-bit RCW{S} transfer register overlap
                        Unpredictable_LSE128OVERLAP,
                        // Clearing DCC/ITR sticky flags when instruction is in flight
                        Unpredictable_CLEARERRITEZERO,
                        // ALUEXCEPTIONRETURN when in user/system mode in
                        // A32 instructions
                        Unpredictable_ALUEXCEPTIONRETURN,
                        // Trap to register in debug state are ignored
                        Unpredictable_IGNORETRAPINDEBUG,
                        // Compare DBGBVR.RESS for BP/WP
                        Unpredictable_DBGxVR_RESS,
                        // Inaccessible event counter
                        Unpredictable_PMUEVENTCOUNTER,
                        // Reserved PMSCR.PCT behavior
                        Unpredictable_PMSCR_PCT,
                        // MDCR_EL2.HPMN or HDCR.HPMN is larger than PMCR.N or
                        // FEAT_HPMN0 is not implemented and HPMN is 0.
                        Unpredictable_RES_HPMN,
                        // Chained PMU counters idx and idx+1 are not in same range
                        Unpredictable_COUNT_CHAIN,
                        // PMCCR.EPMN is larger than PMCR.N
                        Unpredictable_RES_EPMN,
                        // Generate BRB_FILTRATE event on BRB injection
```

```
                          Unpredictable_BRBFILTRATE,
                          // Generate PMU_SNAPSHOT event in Debug state
                          Unpredictable_PMUSNAPSHOTEVENT,
                          // Reserved MDCR_EL3.EPMSSAD value
                          Unpredictable_RESEPMSSAD,
                          // Reserved PMECR_EL1.SSE value
                          Unpredictable_RESPMSSE,
                          // Enable for PMU Profiling exception and PMUIRQ
                          Unpredictable_RESPMEE,
                          // Enables for SPE Profiling exceptions and PMSIRQ
                          Unpredictable_RESPMSEE,
                          // Enables for TRBE Profiling exceptions and PMSIRQ
                          Unpredictable_RESTRFEE,
                          // NonZero TRBPTR_EL1[63:(OAMax)]
                          Unpredictable_TRBPTR_Fault_AddressSize,
                          // Operands for CPY*/SET* instructions overlap
                          Unpredictable_MOPSOVERLAP,
                          // Operands for CPY*/SET* instructions use 0b11111
                          // as a register specifier
                          Unpredictable_MOPS_R31,
                          // PMBPTR_EL1[63:(OAMax)] NonZero causes Address Size fault
                          Unpredictable_PMBPTR_Fault_AddressSize,
                          // Chooses which value to return in a non failed Atomic Compare and Swap
                          Unpredictable_CASRETURNOLDVALUE,
                          // Enables write of the newvalue in a failed Atomic Compare and Swap
                          Unpredictable_WRITEFAILEDCAS,
                          // Enables write of the newvalue in a failed Atomic Compare and Swap
                          // or failed RCW{S}
                          Unpredictable_WRITEFAILEDRCWCAS,
                          // Store-only Tag checking on a failed Atomic Compare and Swap
                          Unpredictable_STRONLYTAGCHECKEDCAS,
                          // Store-only Tag checking on a failed RCW(S) checks
                          // or RCW(S) Atomic Compare and Swap
                          Unpredictable_STRONLYTAGCHECKEDRCWSCAS,
                          // Reserved MDCR_EL3.ETBAD value
                          Unpredictable_RES_ETBAD,
                          // Invalid Streaming Mode filter bits
                          Unpredictable_RES_PMU_VS,
                          // Apply Checked Pointer Arithmetic on a sequential access to bytes
                          // that cross the 0xXXFF_FFFF_FFFF_FFFF boundary.
                          Unpredictable_CPACHECK,
                          // Reserved PMEVTYPER_EL0(n).[TC,TE,TLC] values
                          Unpredictable_RESTC,
                          // When FEAT_MTE is implemented, if Memory access mode is enabled
                          // and PSTATE.TCO is 0,  Reads and writes to the external debug
                          // interface DTR registers are CONSTRAINED UNPREDICTABLE for tagcheck
                          Unpredictable_NODTRTAGCHK,
                          // Use the default PMG when the default PARTID is generated
                          // due to MPAM error
                          Unpredictable_USE_DEFAULT_PMG,
                          // MPAM PARTID/PMG exceeding PARTID_MAX/PMG_MAX
                          Unpredictable_OUT_OF_RANGE_ID,
                          // MPAM vPARTID exceeding (VPMR_MAX << 2) + 3
                          Unpredictable_OUT_OF_RANGE_VID,
                          // If the atomic instructions are not atomic in regard to other
                          // agents that access memory, then the instruction can have one or
                          // more of the following effects
                          Unpredictable_Atomic_SYNC_ABORT,
                          Unpredictable_Atomic_SERROR,
                          Unpredictable_Atomic_NOP,
                          Unpredictable_Atomic_MMU_IMPDEF_FAULT,
                          // Truncate virtual identifier to bottom 12 bits if it exceeds 2^12
                          Unpredictable_MPAM_TRUNCATE_VID,
                          // Accessing DBGDSCRint via MRC in debug state
                          Unpredictable_MRC_APSR_TARGET,
                          // Accessing Banked register not accessible from the PE mode
                          Unpredictable_BankedRegister,
                          // Accessing unimplemented Banked register
                          Unpredictable_UnimplementedRegister,
                          // Reserved ERRACR.{RLRA,SRA,NSRA} values
                          Unpredictable_RES_RA,
                          // Linked target does not support linking
                          Unpredictable_BPNOTLINKING
};
```

Library pseudocode for shared/functions/vector/AdvSIMDExpandImm

```
// AdvSIMDExpandImm()
// ==================

func AdvSIMDExpandImm(op : bit, cmode : bits(4), imm8 : bits(8)) => bits(64)
begin
    var imm64 : bits(64);
    case cmode[3:1] of
        when '000' =>
            imm64 = Replicate{64}(Zeros{24}::imm8);
        when '001' =>
            imm64 = Replicate{64}(Zeros{16}::imm8::Zeros{8});
        when '010' =>
            imm64 = Replicate{64}(Zeros{8}::imm8::Zeros{16});
        when '011' =>
            imm64 = Replicate{64}(imm8::Zeros{24});
        when '100' =>
            imm64 = Replicate{64}(Zeros{8}::imm8);
        when '101' =>
            imm64 = Replicate{64}(imm8::Zeros{8});
        when '110' =>
            if cmode[0] == '0' then
                imm64 = Replicate{64}(Zeros{16}::imm8::Ones{8});
            else
                imm64 = Replicate{64}(Zeros{8}::imm8::Ones{16});
            end;
        when '111' =>
            if cmode[0] == '0' && op == '0' then
                imm64 = Replicate{64}(imm8);
            end;
            if cmode[0] == '0' && op == '1' then
                let imm8a : bits(8) = Replicate{}(imm8[7]);
                let imm8b : bits(8) = Replicate{}(imm8[6]);
                let imm8c : bits(8) = Replicate{}(imm8[5]);
                let imm8d : bits(8) = Replicate{}(imm8[4]);
                let imm8e : bits(8) = Replicate{}(imm8[3]);
                let imm8f : bits(8) = Replicate{}(imm8[2]);
                let imm8g : bits(8) = Replicate{}(imm8[1]);
                let imm8h : bits(8) = Replicate{}(imm8[0]);
                imm64 = imm8a::imm8b::imm8c::imm8d::imm8e::imm8f::imm8g::imm8h;
            end;
            if cmode[0] == '1' && op == '0' then
                let imm32 : bits(32) = (imm8[7]::NOT(imm8[6])::Replicate{5}(imm8[6])::
                                        imm8[5:0]::Zeros{19});
                imm64 = Replicate{64}(imm32);
            end;
            if cmode[0] == '1' && op == '1' then
                if UsingAArch32() then ReservedEncoding(); end;
                imm64 = imm8[7]::NOT(imm8[6])::Replicate{8}(imm8[6])::imm8[5:0]::Zeros{48};
            end;
    end;

    return imm64;
end;
```

Library pseudocode for shared/functions/vector/MatMulAdd

```
// MatMulAdd()
// ===========
//
// Signed or unsigned 8-bit integer matrix multiply and add to 32-bit integer matrix
// result[2, 2] = addend[2, 2] + (op1[2, 8] * op2[8, 2])

func MatMulAdd(addend : bits(128), op1 : bits(128), op2 : bits(128), op1_unsigned : boolean,
               op2_unsigned : boolean) => bits(128)
begin

    var result : bits(128);
    var sum : bits(32);
    var prod : integer;

    for i = 0 to 1 do
        for j = 0 to 1 do
            sum = addend[(2*i + j)*:32];
            for k = 0 to 7 do
                let opelt1 : bits(8) = op1[(8*i + k)*:8];
                let opelt2 : bits(8) = op2[(8*j + k)*:8];
                let element1 : integer = if op1_unsigned then UInt(opelt1) else SInt(opelt1);
                let element2 : integer = if op2_unsigned then UInt(opelt2) else SInt(opelt2);
                prod = element1 * element2;
                sum  = sum + prod;
            end;
            result[(2*i + j)*:32] = sum;
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/functions/vector/PolynomialMult

```
// PolynomialMult()
// ================

func PolynomialMult{M, N}(op1 : bits(M), op2 : bits(N)) => bits(M+N)
begin
    var result : bits(N + M) = Zeros{M+N};
    let extended_op2 : bits(N + M) = ZeroExtend{M+N}(op2);
    for i=0 to M-1 do
        if op1[i] == '1' then
            result = result XOR LSL(extended_op2, i);
        end;
    end;
    return result;
end;
```

Library pseudocode for shared/functions/vector/SatQ

```
// SatQ()
// ======

func SatQ{N}(i : integer, unsigned : boolean) => (bits(N), boolean)
begin
    let (result, sat) : (bits(N), boolean) = (if unsigned then UnsignedSatQ{N}(i)
                                                          else SignedSatQ{N}(i));
    return (result, sat);
end;
```

Library pseudocode for shared/functions/vector/ShiftSat

```
// ShiftSat()
// ==========

func ShiftSat(shift : integer, esize : integer) => integer
begin
    if shift > esize+1 then return esize+1;
    elsif shift < -(esize+1) then return -(esize+1); end;
    return shift;
end;
```

Library pseudocode for shared/functions/vector/SignedSat

```
// SignedSat()
// ===========

func SignedSat{N}(i : integer) => bits(N)
begin
    let (result, -) = SignedSatQ{N}(i);
    return result;
end;
```

Library pseudocode for shared/functions/vector/SignedSatQ

```
// SignedSatQ()
// ============

func SignedSatQ{N}(i : integer) => (bits(N), boolean)
begin
    var result : integer;
    var saturated : boolean;
    if i > 2^(N-1) - 1 then
        result = 2^(N-1) - 1;  saturated = TRUE;
    elsif i < -(2^(N-1)) then
        result = -(2^(N-1));  saturated = TRUE;
    else
        result = i;  saturated = FALSE;
    end;
    return (result[N-1:0], saturated);
end;
```

Library pseudocode for shared/functions/vector/UnsignedRSqrtEstimate

```
// UnsignedRSqrtEstimate()
// =======================

func UnsignedRSqrtEstimate{N}(operand : bits(N)) => bits(N)
begin
    assert N == 32;
    var result : bits(N);
    if operand[N-1:N-2] == '00' then  // Operands <= 0x3FFFFFFF produce 0xFFFFFFFF
        result = Ones{N};
    else
        // input is in the range 0x40000000 .. 0xffffffff representing [0.25 .. 1.0)
        // estimate is in the range 256 .. 511 representing [1.0 .. 2.0)
        let increasedprecision : boolean = FALSE;
        let estimate : integer = RecipSqrtEstimate(UInt(operand[31:23]), increasedprecision);
        // result is in the range 0x80000000 .. 0xff800000 representing [1.0 .. 2.0)
        result = estimate[8:0] :: Zeros{N-9};
    end;

    return result;
end;
```

Library pseudocode for shared/functions/vector/UnsignedRecipEstimate

```
// UnsignedRecipEstimate()
// =======================

func UnsignedRecipEstimate{N}(operand : bits(N)) => bits(N)
begin
    assert N == 32;
    var result : bits(N);
    if operand[N-1] == '0' then  // Operands <= 0x7FFFFFFF produce 0xFFFFFFFF
        result = Ones{N};
    else
        // input is in the range 0x80000000 .. 0xffffffff representing [0.5 .. 1.0)

        // estimate is in the range 256 to 511 representing [1.0 .. 2.0)
        let increasedprecision : boolean = FALSE;
        let estimate : integer = RecipEstimate(UInt(operand[31:23]), increasedprecision);

        // result is in the range 0x80000000 .. 0xff800000 representing [1.0 .. 2.0)
        result = estimate[8:0] :: Zeros{N-9};
    end;

    return result;
end;
```

Library pseudocode for shared/functions/vector/UnsignedSat

```
// UnsignedSat()
// =============

func UnsignedSat{N}(i : integer) => bits(N)
begin
    let (result, -) = UnsignedSatQ{N}(i);
    return result;
end;
```

Library pseudocode for shared/functions/vector/UnsignedSatQ

```
// UnsignedSatQ()
// ==============

func UnsignedSatQ{N}(i : integer) => (bits(N), boolean)
begin
    var result : integer;
    var saturated : boolean;
    if i > 2^N - 1 then
        result = 2^N - 1;  saturated = TRUE;
    elsif i < 0 then
        result = 0;  saturated = TRUE;
    else
        result = i;  saturated = FALSE;
    end;
    return (result[N-1:0], saturated);
end;
```

Library pseudocode for shared/trace/Common/DebugMemWrite

```
// DebugMemWrite()
// ===============
// Write data to memory one byte at a time. Starting at the passed virtual address.
// Used by SPE and TRBE.

func DebugMemWrite(vaddress : bits(64), accdesc : AccessDescriptor,
                   aligned : boolean, data : bits(8)) => (PhysMemRetStatus, AddressDescriptor)
begin

    var memstatus : PhysMemRetStatus = ARBITRARY : PhysMemRetStatus;

    // Translate virtual address
    var addrdesc : AddressDescriptor;
    let size : integer = 1;
    addrdesc = AArch64_TranslateAddress(vaddress, accdesc, aligned, size);

    if IsFault(addrdesc) then
        return (memstatus, addrdesc);
    end;

    memstatus = PhysMemWrite{8}(addrdesc, accdesc, data);

    return (memstatus, addrdesc);
end;
```

Library pseudocode for shared/trace/Common/DebugWriteExternalAbort

```
// DebugWriteExternalAbort()
// =========================
// Populate the syndrome register for an External abort caused by a call of DebugMemWrite().

func DebugWriteExternalAbort(memstatus : PhysMemRetStatus, addrdesc : AddressDescriptor,
                             start_vaddr : bits(64))
begin

    let iswrite : boolean = TRUE;

    var handle_as_SError : boolean = FALSE;
    case addrdesc.fault.accessdesc.acctype of
        when AccessType_SPE =>
            handle_as_SError = ImpDefBool("Report SPE ExtAbort as SError");
        when AccessType_TRBE =>
            handle_as_SError = ImpDefBool("Report TRBE ExtAbort as SError");
        otherwise =>
            unreachable;
    end;

    let ttw_abort : boolean = addrdesc.fault.statuscode IN {Fault_SyncExternalOnWalk,
                                                            Fault_SyncParityOnWalk};
    let statuscode : Fault = (if ttw_abort then addrdesc.fault.statuscode
                              else memstatus.statuscode);

    if statuscode IN {Fault_AsyncExternal, Fault_AsyncParity} || handle_as_SError then
        // Report the abort as an SError
        var fault : FaultRecord = NoFault();
        let parity : boolean = statuscode IN {Fault_SyncParity, Fault_AsyncParity,
                                              Fault_SyncParityOnWalk};
        fault.statuscode = if parity then Fault_AsyncParity else Fault_AsyncExternal;
        if IsFeatureImplemented(FEAT_RAS) then
            fault.merrorstate = memstatus.merrorstate;
        end;
        let extflag : bit = if ttw_abort then addrdesc.fault.extflag else memstatus.extflag;
        fault.extflag = extflag;
        fault.accessdesc.acctype = addrdesc.fault.accessdesc.acctype;
        PendSErrorInterrupt(fault);
        return;
    end;

    // Generate a buffer management event, modifying the existing syndrome.
    var handle_async : boolean = FALSE;
    var syndrome : bits(64);
    case addrdesc.fault.accessdesc.acctype of
        when AccessType_SPE =>
            handle_async = ImpDefBool("Report SPE ExtAbort asynchronously");
            assert !IsFeatureImplemented(FEAT_SPE_EXC);
            syndrome = PMBSR_EL1();
        when AccessType_TRBE =>
            handle_async = ImpDefBool("Report TRBE ExtAbort asynchronously");
            assert !IsFeatureImplemented(FEAT_TRBE_EXC);
            syndrome = TRBSR_EL1();
        otherwise =>
            unreachable;
    end;

    var ec : bits(6);
    if (IsFeatureImplemented(FEAT_RME) && addrdesc.fault.gpcf.gpf != GPCF_None &&
          addrdesc.fault.gpcf.gpf != GPCF_Fail) then
        ec = '011110';
    else
        ec = if addrdesc.fault.secondstage then '100101' else '100100';
    end;

    let mss2 : bits(24) = Zeros{};
    var mss : bits(16) = Zeros{};
    if handle_async then                // FSC bits
        mss[5:0] = '010001';
    else
        mss[5:0] = EncodeLDFSC(statuscode, addrdesc.fault.level);
    end;

    // The following values are always updated in the syndrome register.
    if (addrdesc.fault.accessdesc.acctype == AccessType_SPE &&
          (handle_async || start_vaddr != addrdesc.vaddress)) then
        syndrome[19] = '1';             // DL bit (SPE only)
    end;

    syndrome[18] = '1';                 // EA bit

    // The following values are not modified if a previous buffer management event
    // has not been handled. Note that in this simple sequential model, this test
    // will never fail.
    if syndrome[17] == '0' then         // Check previous 'S' bit.
        syndrome[55:32] = mss2;         // MSS2 bits
        syndrome[31:26] = ec;           // EC bits
        if addrdesc.fault.accessdesc.acctype == AccessType_TRBE then
            syndrome[22] = '1';         // IRQ bit (TRBE only)
```

```
        end;
        syndrome[17] = '1';             // S bit
        syndrome[15:0] = mss;           // MSS bits
    end;

    case addrdesc.fault.accessdesc.acctype of
        when AccessType_SPE =>
            PMBSR_EL1() = syndrome;
        when AccessType_TRBE =>
            TRBSR_EL1() = syndrome;
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/trace/Common/DebugWriteFault

```
// DebugWriteFault()
// =================
// Populate the syndrome register for a fault caused by a call of DebugMemWrite().

func DebugWriteFault(vaddress : bits(64), fault : FaultRecord)
begin
    var ec : bits(6);
    if (IsFeatureImplemented(FEAT_RME) && fault.gpcf.gpf != GPCF_None &&
          fault.gpcf.gpf != GPCF_Fail) then
        ec = '011110';
    else
        ec = if fault.secondstage then '100101' else '100100';
    end;

    var mss2 : bits(24) = Zeros{};
    var mss : bits(16) = Zeros{};

    if fault.gpcf.gpf IN {GPCF_None, GPCF_Fail} then
        mss2[11] = if fault.hdbssf then '1' else '0';            // HDBSSF
        mss2[8]  = if fault.toplevel then '1' else '0';          // TopLevel bit
        mss2[7]  = if fault.assuredonly then '1' else '0';       // AssuredOnly bit
        mss2[6]  = if fault.overlay then '1' else '0';           // Overlay bit
        mss2[5]  = if fault.dirtybit then '1' else '0';          // DirtyBit

        mss[5:0] = EncodeLDFSC(fault.statuscode, fault.level);  // FSC bits
        if fault.s2fs1walk then
            if (IsFeatureImplemented(FEAT_SPE_EXC) &&
                  fault.accessdesc.acctype == AccessType_SPE) then
                mss[7] = '1';                                    // S1PTW bit
            end;
            if (IsFeatureImplemented(FEAT_TRBE_EXC) &&
                  fault.accessdesc.acctype == AccessType_TRBE) then
                mss[7] = '1';                                    // S1PTW bit
            end;
        end;
    end;

    // Generate a buffer management event, modifying the existing syndrome.
    var target_el : bits(2);
    var syndrome : bits(64);
    case fault.accessdesc.acctype of
        when AccessType_SPE =>
            target_el = ReportSPEEvent(ec, mss[5:0]);
            syndrome  = PMBSR_EL(target_el);
        when AccessType_TRBE =>
            target_el = ReportTRBEEvent(ec, mss[5:0]);
            syndrome  = TRBSR_EL(target_el);
        otherwise =>
            unreachable;
    end;

    // The following values are not modified if a previous buffer management event
    // has not been handled. Note that in this simple sequential model, this test
    // will never fail.
    if syndrome[17] == '0' then          // Check previous 'S' bit.
        syndrome[55:32] = mss2;          // MSS2 bits
        syndrome[31:26] = ec;            // EC bits
        if fault.accessdesc.acctype == AccessType_TRBE then
            syndrome[22] = '1';          // IRQ bit (TRBE only)
        end;
        syndrome[17] = '1';              // S bit
        syndrome[15:0] = mss;            // MSS bits
    end;

    // For SPE, PMBPTR_EL1 points to the address that generated the fault, and writing
    // to memory never started. Therefore, there isno data loss and DL is unchanged.

    case fault.accessdesc.acctype of
        when AccessType_SPE =>
            PMBSR_EL(target_el) = syndrome;

        when AccessType_TRBE =>
            TRBSR_EL(target_el) = syndrome;

        otherwise =>
            unreachable;
    end;

    return;
end;
```

Library pseudocode for shared/trace/Common/GetTimestamp

```
// GetTimestamp()
// ==============
// Returns the Timestamp depending on the type

func GetTimestamp(timeStampType : TimeStamp) => bits(64)
begin
    case timeStampType of
        when TimeStamp_Physical =>
            return PhysicalCountInt();
        when TimeStamp_Virtual =>
            return PhysicalCountInt() - CNTVOFF_EL2();
        when TimeStamp_OffsetPhysical =>
            let physoff : bits(64) = if PhysicalOffsetIsValid() then CNTPOFF_EL2() else Zeros{64};
            return PhysicalCountInt() - physoff;
        when TimeStamp_None =>
            return Zeros{64};
        when TimeStamp_CoreSight =>
            return ImpDefBits{64}("CoreSight timestamp");
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/trace/Common/PhysicalOffsetIsValid

```
// PhysicalOffsetIsValid()
// =======================
// Returns whether the Physical offset for the timestamp is valid

func PhysicalOffsetIsValid() => boolean
begin
    if !HaveAArch64() then
        return FALSE;
    elsif !HaveEL(EL2) || !IsFeatureImplemented(FEAT_ECV_POFF) then
        return FALSE;
    elsif HaveEL(EL3) && EffectiveSCR_EL3_NS() == '1' && EffectiveSCR_EL3_RW() == '0' then
        return FALSE;
    elsif HaveEL(EL3) && SCR_EL3().ECVEn == '0' then
        return FALSE;
    elsif CNTHCTL_EL2().ECV == '0' then
        return FALSE;
    else
        return TRUE;
    end;
end;
```

Library pseudocode for shared/trace/Common/TRBCRManStopWrite

```
// TRBCRManStopWrite()
// ===================
// Called on a write of 1 to TRBCR.ManStop.

func TRBCRManStopWrite()
begin
    TraceUnitFlush();
    OtherTRBEManagementEvent('000011');
    TryAssertTRBIRQ();
end;
```

Library pseudocode for shared/trace/TraceBranch/BranchNotTaken

```
// BranchNotTaken()
// ================
// Called when a branch is not taken.

func BranchNotTaken(branchtype : BranchType, branch_conditional : boolean)
begin
    let branchtaken : boolean = FALSE;
    if IsFeatureImplemented(FEAT_SPE) then
        SPEBranch{64}(ARBITRARY : bits(64), branchtype, branch_conditional, branchtaken);
    end;
    return;
end;
```

Library pseudocode for shared/trace/TraceBuffer/AllowExternalTraceBufferAccess

```
// AllowExternalTraceBufferAccess()
// ================================
// Returns TRUE if an external debug interface access to the Trace Buffer
// registers is allowed for the given Security state, FALSE otherwise.
// The access may also be subject to OS Lock, power-down, etc.

func AllowExternalTraceBufferAccess(addrdesc : AddressDescriptor) => boolean
begin
    assert IsFeatureImplemented(FEAT_TRBE_EXT);
    // FEAT_Debugv8p4 is always implemented when FEAT_TRBE_EXT is implemented.
    assert IsFeatureImplemented(FEAT_Debugv8p4);

    var etbad : bits(2) = if HaveEL(EL3) then MDCR_EL3().ETBAD else '11';

    // Check for reserved values
    if !IsFeatureImplemented(FEAT_RME) && etbad IN {'01','10'} then
        (-, etbad) = ConstrainUnpredictableBits{2}(Unpredictable_RES_ETBAD);
        // The value returned by ConstrainUnpredictableBits must be a
        // non-reserved value
        assert etbad IN {'00', '11'};
    end;

    case etbad of
        when '00' =>
            if IsFeatureImplemented(FEAT_RME) then
                return addrdesc.paddress.paspace == PAS_Root;
            else
                return addrdesc.paddress.paspace == PAS_Secure;
            end;
        when '01' =>
            assert IsFeatureImplemented(FEAT_RME);
            return addrdesc.paddress.paspace IN {PAS_Root, PAS_Realm};
        when '10' =>
            assert IsFeatureImplemented(FEAT_RME);
            return addrdesc.paddress.paspace IN {PAS_Root, PAS_Secure};
        when '11' =>
            return TRUE;
    end;
end;
```

Library pseudocode for shared/trace/TraceBuffer/CheckForTRBEException

```
// CheckForTRBEException()
// ======================
// Take a TRBE Profiling exception if pending, permitted, and unmasked.

func CheckForTRBEException()
begin
    if !IsFeatureImplemented(FEAT_TRBE_EXC) || !SelfHostedTraceEnabled() then
        return;
    end;

    if Halted() || Restarting() then
        return;
    end;

    var route_to_el3 : boolean = FALSE;
    var route_to_el2 : boolean = FALSE;
    var route_to_el1 : boolean = FALSE;

    if HaveEL(EL3) && MDCR_EL3().TRBEE == '1x' then
        let pending : boolean = TRBSR_EL3().IRQ == '1';
        let masked  : boolean = PSTATE.EL       == EL3;
        route_to_el3 = pending && !masked;
    end;

    var owning_ss : SecurityState;
    var owning_el : bits(2);
    (owning_ss, owning_el) = TraceBufferOwner();
    let in_owning_ss : boolean = IsCurrentSecurityState(owning_ss);

    if EffectiveTRFCR_EL2_EE() IN {'1x'} then
        let pending : boolean = TRBSR_EL2().IRQ == '1';
        let masked : boolean = (!in_owning_ss || PSTATE.EL == EL3 ||
                                (PSTATE.EL == EL2 && (TRFCR_EL2().EE != '11' ||
                                                      TRFCR_EL2().KE == '0' ||
                                                      PSTATE.PM == '1')));
        route_to_el2 = pending && !masked;
    end;

    if EffectiveTRFCR_EL1_EE() == '11' then
        let pending : boolean = TRBSR_EL1().IRQ == '1';
        let masked  : boolean = (!in_owning_ss || PSTATE.EL IN {EL3, EL2} ||
                                (PSTATE.EL == EL1 && (TRFCR_EL1().KE == '0' ||
                                                      PSTATE.PM == '1')));
        if EffectiveTGE() == '1' then
            route_to_el2 = route_to_el2 || (pending && !masked);
        else
            route_to_el1 = pending && !masked;
        end;
    end;

    let fsc : bits(5) = '00010';        // TRBE exception
    let synchronous : boolean = FALSE;

    // The relative priorities of the following checks is IMPLEMENTATION DEFINED
    if route_to_el3 then
        TakeProfilingException(EL3, fsc, synchronous);
    end;
    if route_to_el2 then
        TakeProfilingException(EL2, fsc, synchronous);
    end;
    if route_to_el1 then
        TakeProfilingException(EL1, fsc, synchronous);
    end;
end;
```

Library pseudocode for shared/trace/TraceBuffer/CheckMDCR_EL3_NSTBTrap

```
// CheckMDCR_EL3_NSTBTrap()
// ========================
// Check if the register access is trappable by MDCR_EL3.[NSTBE, NSTB]

func CheckMDCR_EL3_NSTBTrap() => boolean
begin
    var state_bits : bits(3);
    var reserved   : boolean;
    (state_bits, reserved) = EffectiveMDCR_EL3_NSTB();
    return ((reserved && ConstrainUnpredictableBool(Unpredictable_RESERVEDNSxB_Trap)) ||
             state_bits[0] == '0' || state_bits[1] != EffectiveSCR_EL3_NS() ||
             (IsFeatureImplemented(FEAT_RME) && state_bits[2] != EffectiveSCR_EL3_NSE()));
end;
```

Library pseudocode for shared/trace/TraceBuffer/CollectTrace

```
// CollectTrace()
// ==============
// Called for each byte generated by the trace unit.
// Returns TRUE if the Trace Buffer Unit accepts or discards the trace
// data, and FALSE if the Trace Buffer Unit rejects the trace data.

func CollectTrace(datum : bits(8)) => boolean recurselimit ElementStreamSize
begin
    if !TraceBufferEnabled() then  // Trace buffer disabled
        // 'datum' is discarded
        if HaveImpDefTraceOutput() then
            return ImpDefTraceOutput(datum);
        else
            return TRUE;           // Discard the trace byte
        end;
    end;

    // If the TRBE cannot accept the trace data, it must return FALSE
    if TRBEInternalBufferFull() then
        return FALSE;
    end;

    if TraceBufferRunning() then   // Accept the data
        let address : bits(64)          = TRBPTR_EL1();
        var ttw_abort : boolean         = FALSE;
        let ttw_abort_as_fault: boolean = ImpDefBool("Report TRBE ExtAbort on TTW as fault");
        var addrdesc : AddressDescriptor;
        var memstatus : PhysMemRetStatus;

        if !SelfHostedTraceEnabled() then
            // The Trace Buffer Unit is using External mode.
            if IsFeatureImplemented(FEAT_RME) && !ExternalRootInvasiveDebugEnabled() then
                if IsZero(GPCCR_EL3().[TBGPCD, GPC]) then
                    return FALSE;
                end;
            end;

            let pas : bits(2) = TRBMAR_EL1().PAS;
            let paspace : PASpace = DecodePASpace('0', pas[1], pas[0]);
            if !IsPASValid(pas) || !InvasiveDebugPermittedPAS(paspace) then
                OtherTRBEManagementEvent('000000');
                TryAssertTRBIRQ();
                return TRUE;
            end;

            if IsFeatureImplemented(FEAT_TRBE_MPAM) && TRBMPAM_EL1().EN == '1' then
                let mpam_sp : bits(2) = TRBMPAM_EL1().MPAM_SP;
                let mpam_pas : PASpace = DecodePASpace('0', mpam_sp[1], mpam_sp[0]);

                if !InvasiveDebugPermittedPAS(mpam_pas) then
                    OtherTRBEManagementEvent('000000');
                    TryAssertTRBIRQ();
                    return TRUE;
                end;
            end;

            let el : bits(2)               = ARBITRARY : bits(2);
            let ss : SecurityState         = ARBITRARY : SecurityState;
            let accdesc : AccessDescriptor = CreateAccDescTRBE(ss, el);

            var pa : FullAddress;
            pa.address = address[NUM_PABITS-1:0];
            pa.paspace = paspace;
            let memattrs : MemoryAttributes = S1DecodeMemAttrs(TRBMAR_EL1().Attr,
                                                               TRBMAR_EL1().SH, TRUE);
            addrdesc = CreateAddressDescriptor(pa, memattrs, accdesc);
            addrdesc.mecid = DEFAULT_MECID;

            let oamax : AddressSize = AArch64_OAMax();
            if !IsZero(address[63:oamax]) then
                if ConstrainUnpredictableBool(Unpredictable_TRBPTR_Fault_AddressSize) then
                    addrdesc.fault.statuscode = Fault_AddressSize;
                else
                    pa.address = ZeroExtend{NUM_PABITS}(pa.address[oamax-1:0]);
                end;
            end;

            if (!IsFault(addrdesc.fault.statuscode) && IsFeatureImplemented(FEAT_RME) &&
                  !ExternalRootInvasiveDebugEnabled()) then
                let gpcf : GPCFRecord = GranuleProtectionCheck(addrdesc, accdesc);
                if gpcf.gpf == GPCF_None then
                    memstatus = PhysMemWrite{8}(addrdesc, accdesc, datum);
                else
                    addrdesc.fault.gpcf       = gpcf;
                    addrdesc.fault.statuscode = Fault_GPCFOnOutput;
                end;
            end;
            if !IsFault(addrdesc.fault.statuscode) then
                memstatus = PhysMemWrite{8}(addrdesc, accdesc, datum);
```

```
            end;

        else
            // The Trace Buffer Unit is using Self-hosted mode.
            var owning_ss : SecurityState;
            var owning_el : bits(2);
            (owning_ss, owning_el) = TraceBufferOwner();
            let accdesc : AccessDescriptor = CreateAccDescTRBE(owning_ss, owning_el);

            let aligned : boolean = TRUE;
            (memstatus, addrdesc) = DebugMemWrite(address, accdesc, aligned, datum);

            ttw_abort = addrdesc.fault.statuscode IN {Fault_SyncExternalOnWalk,
                                                      Fault_SyncParityOnWalk};
        end;

        if IsFault(addrdesc.fault.statuscode) && (!ttw_abort || ttw_abort_as_fault) then
            DebugWriteFault(address, addrdesc.fault);
            TryAssertTRBIRQ();
            return TRUE;
        elsif IsFault(memstatus) || (ttw_abort && !ttw_abort_as_fault) then
            DebugWriteExternalAbort(memstatus, addrdesc, address);
            TryAssertTRBIRQ();
            return TRUE;
        end;

        // Check for Trigger Event
        let target_el : bits(2) = DefaultTRBEEvent();
        let triggered : boolean = TRBSR_EL(target_el).TRG == '1';
        if triggered && !IsZero(TRBTRG_EL1().TRG) then
            TRBTRG_EL1().TRG = (TRBTRG_EL1().TRG - 1)[31:0];
            if IsZero(TRBTRG_EL1().TRG) && TRBLIMITR_EL1().TM != '11' then
                TraceUnitFlush();
                TraceUnitFlushOnTriggerComplete();
            end;
        end;

        // Increment the pointer
        var next_address : bits(64) = TRBPTR_EL1() + 1;
        if next_address[63:12] == TRBLIMITR_EL1().LIMIT then
            next_address = TRBBASER_EL1().BASE::Zeros{12};
            TRBSR_EL(target_el).WRAP = '1';
            CTI_SignalEvent(CrossTriggerIn_TRBEWrap);
            if TRBLIMITR_EL1().FM == '00' then     // Fill mode
                let bsc : bits(6) = '000001'; // Buffer full event
                OtherTRBEManagementEvent(bsc);
            elsif TRBLIMITR_EL1().FM != '11' then  // Not Circular Buffer mode
                if TRBSR_EL(target_el).IRQ == '0' then
                    TRBSR_EL(target_el).IRQ = '1';   // Assert interrupt or exception
                    CTI_SignalEvent(CrossTriggerIn_TRBEMgmt);
                end;
            end;
        end;
        TRBPTR_EL1() = next_address[63:0];

        TryAssertTRBIRQ();
    end;
    return TRUE;
end;
```

Library pseudocode for shared/trace/TraceBuffer/DefaultTRBEEvent

```
// DefaultTRBEEvent()
// ==================
// Return the target ELx for an indirect write to TRBSR_ELx for an Other buffer management
// event or anything other than a buffer management event.

func DefaultTRBEEvent() => bits(2)
begin
    return ReportTRBEEvent(Zeros{6}, ARBITRARY : bits(6));
end;
```

Library pseudocode for shared/trace/TraceBuffer/DetectedTraceTrigger

```
// DetectedTraceTrigger()
// ======================
// Called when the trace unit detects a trace trigger

func DetectedTraceTrigger()
begin
    if TraceBufferRunning() then
        let target_el : bits(2) = DefaultTRBEEvent();
        if TRBSR_EL(target_el).TRG == '0' then
            TRBSR_EL(target_el).TRG = '1';
            if IsZero(TRBTRG_EL1().TRG) && TRBLIMITR_EL1().TM != '11' then
                TraceUnitFlush();
                TraceUnitFlushOnTriggerComplete();
            end;
        end;
    end;
end;
```

Library pseudocode for shared/trace/TraceBuffer/EffectiveMDCR_EL3_NSTB

```
// EffectiveMDCR_EL3_NSTB()
// ========================
// Return the Effective value of MDCR_EL3().[NSTBE, NSTB] field and whether it is a reserved value.

readonly func EffectiveMDCR_EL3_NSTB() => (bits(3), boolean)
begin
    var state_bits : bits(3);
    var reserved   : boolean = FALSE;

    if IsFeatureImplemented(FEAT_RME) then
        state_bits = MDCR_EL3().[NSTBE, NSTB];
        if state_bits == '10x' || (!IsFeatureImplemented(FEAT_Secure) && state_bits == '00x') then
            // Reserved value
            reserved = TRUE;
            (-, state_bits) = ConstrainUnpredictableBits{3}(Unpredictable_RESERVEDNSxB);
        end;
    else
        state_bits = '0' :: MDCR_EL3().NSTB;
    end;

    return (state_bits, reserved);
end;
```

Library pseudocode for shared/trace/TraceBuffer/EffectiveTRBLIMITR_EL1_nVM

```
// EffectiveTRBLIMITR_EL1_nVM()
// ============================

readonly func EffectiveTRBLIMITR_EL1_nVM() => bit
begin
    if !SelfHostedTraceEnabled() then
        // If SelfHostedTraceEnabled() is FALSE, then this function is only called when
        // FEAT_TRBE_EXT is implemented.
        assert IsFeatureImplemented(FEAT_TRBE_EXT);
        return '1';
    end;
    if IsFeatureImplemented(FEAT_TRBEv1p1) && HaveEL(EL2) then
        let (owning_ss, owning_el) = TraceBufferOwner();
        if ((owning_ss != SS_Secure || IsSecureEL2Enabled()) && owning_el == EL1 &&
              TRFCR_EL2().DnVM == '1') then
            return '0';
        end;
    end;
    return TRBLIMITR_EL1().nVM;
end;
```

Library pseudocode for shared/trace/TraceBuffer/EffectiveTRFCR_EL1_EE

```
// EffectiveTRFCR_EL1_EE()
// =======================
// Return the Effective value of TRFCR_EL1.EE for the purpose of controlling the
// TRBE Profiling exception.

func EffectiveTRFCR_EL1_EE() => bits(2)
begin
    if EffectiveTRFCR_EL2_EE() == '00' then
        return '00';
    end;

    var ee : bits(2) = TRFCR_EL1().EE;
    if ee IN {'01', '10'} then            // Reserved value
        if IsFeatureImplemented(FEAT_NV) then
            ee[0] = ee[1];
        else
            var c : Constraint;
            (c, ee) = ConstrainUnpredictableBits{2}(Unpredictable_RESTRFEE);
            assert c IN {Constraint_DISABLED, Constraint_UNKNOWN};
            if c == Constraint_DISABLED then
                ee = '00';
            end;
            // Otherwise the value returned by ConstrainUnpredictableBits must be
            // a non-reserved value
        end;
    end;

    return ee;
end;
```

Library pseudocode for shared/trace/TraceBuffer/EffectiveTRFCR_EL2_EE

```
// EffectiveTRFCR_EL2_EE()
// =======================
// Return the Effective value of TRFCR_EL2.EE.

func EffectiveTRFCR_EL2_EE() => bits(2)
begin
    if !IsFeatureImplemented(FEAT_TRBE_EXC) || !SelfHostedTraceEnabled() then
        return '00';
    end;

    if HaveEL(EL3) && MDCR_EL3().TRBEE == '00' then
        return '00';
    end;

    let check_el2 : boolean = HaveEL(EL2) && (EffectiveSCR_EL3_NS() == '1' ||
                                              IsSecureEL2Enabled());
    return if check_el2 then TRFCR_EL2().EE else '01';
end;
```

Library pseudocode for shared/trace/TraceBuffer/ElementStreamSize

```
// ElementStreamSize
// =================

config ElementStreamSize : integer{0..4096} = 4096;
```

Library pseudocode for shared/trace/TraceBuffer/GetTRBSR_EL1_FSC

```
// GetTRBSR_EL1_FSC()
// ==================
// Query the TRBSR_EL1.FSC field.

func GetTRBSR_EL1_FSC() => bits(6)
begin
    var FSC : bits(6);

    FSC = TRBSR_EL1()[5:0];
    return FSC;
end;
```

Library pseudocode for shared/trace/TraceBuffer/GetTRBSR_EL2_FSC

```
// GetTRBSR_EL2_FSC()
// ==================
// Query the TRBSR_EL2.FSC field.

func GetTRBSR_EL2_FSC() => bits(6)
begin
    var FSC : bits(6);

    FSC = TRBSR_EL2()[5:0];
    return FSC;
end;
```

Library pseudocode for shared/trace/TraceBuffer/GetTRBSR_EL3_FSC

```
// GetTRBSR_EL3_FSC()
// ==================
// Query the TRBSR_EL3.FSC field.

func GetTRBSR_EL3_FSC() => bits(6)
begin
    var FSC : bits(6);

    FSC = TRBSR_EL3()[5:0];
    return FSC;
end;
```

Library pseudocode for shared/trace/TraceBuffer/HaveImpDefTraceOutput

```
// HaveImpDefTraceOutput()
// =======================

func HaveImpDefTraceOutput() => boolean
begin
    return ImpDefBool("Has Enabled External Trace Port");
end;
```

Library pseudocode for shared/trace/TraceBuffer/ImpDefTraceOutput

```
// ImpDefTraceOutput()
// ===================

func ImpDefTraceOutput(datum : bits(8)) => boolean
begin
    // Send 'datum' to an IMPLEMENTATION DEFINED trace output port
    // return TRUE if the byte is sent
    return FALSE;
end;
```

Library pseudocode for shared/trace/TraceBuffer/OtherTRBEManagementEvent

```
// OtherTRBEManagementEvent()
// ==========================
// Report an Other buffer management event, with the status code 'bsc'

func OtherTRBEManagementEvent(bsc : bits(6))
begin
    ReportTRBEManagementEvent('000000', bsc);
end;
```

Library pseudocode for shared/trace/TraceBuffer/ReportTRBEEvent

```
// ReportTRBEEvent()
// =================
// Return the target ELx for an indirect write to TRBSR_ELx.
// When the indirect write is due to a buffer management event:
// 'ec_bits' is the Event Class for the management event.
// 'fsc_bits' is the Fault Status Code when this is a fault, ignored otherwise.
// Otherwise, 'ec_bits' should be Zeros().

func ReportTRBEEvent(ec_bits : bits(6), fsc_bits : bits(6)) => bits(2)
begin
    var target_el : bits(2);
    var route_to_el3 : boolean = FALSE;
    var route_to_el2 : boolean = FALSE;

    if IsFeatureImplemented(FEAT_TRBE_EXC) && SelfHostedTraceEnabled() then
        if ec_bits == '011111' then                     // implementation defined fault
            return ImpDefBits{2}("target_el for TRBE ImpDef Fault");
        end;

        let s1fault : boolean = (ec_bits == '100100');   // Stage 1 fault
        let s2fault : boolean = (ec_bits == '100101');   // Stage 2 fault

        var gpcfault, gpfault : boolean;
        if IsFeatureImplemented(FEAT_RME) then
            // Granule Protection Check fault, other than GPF. That is, a GPT address size fault,
            // GPT walk fault, or synchronous External abort on GPT fetch.
            gpcfault = (ec_bits == '011110');
            // Other Granule Protection Fault, reported as Stage 1 or Stage 2 fault.
            gpfault  =  ((s1fault || s2fault) && fsc_bits IN {'10001x', '1001xx', '101000'});
        else
            gpcfault = FALSE;
            gpfault  = FALSE;
        end;
        let sync_ext_abort : boolean = ((s1fault || s2fault) &&
                                        fsc_bits IN {'010000', '01001x', '0101xx', '011011'});

        var owning_ss : SecurityState;
        var owning_el : bits(2);
        (owning_ss, owning_el) = TraceBufferOwner();

        if HaveEL(EL3) && MDCR_EL3().TRBEE == '1x' then
            route_to_el3 = (MDCR_EL3().TRBEE == '11' ||
                            gpcfault || (gpfault && SCR_EL3().GPF == '1') ||
                            (sync_ext_abort && EffectiveEA() == '1'));
        end;

        if EffectiveTRFCR_EL2_EE() == '1x' then
            route_to_el2 = (TRFCR_EL2().EE == '11' || (s1fault && owning_el == EL2) || s2fault ||
                            gpcfault || (gpfault && HCR_EL2().GPF == '1') ||
                            (sync_ext_abort && EffectiveHCR_TEA() == '1'));
        end;
    end;

    if route_to_el3 then
        target_el = EL3;
    elsif route_to_el2 then
        target_el = EL2;
    else
        target_el = EL1;
    end;

    return target_el;
end;
```

Library pseudocode for shared/trace/TraceBuffer/ReportTRBEManagementEvent

```
// ReportTRBEManagementEvent()
// ===========================
// Report a buffer management event with the event class 'ec' and status code 'bsc'

func ReportTRBEManagementEvent(ec : bits(6), bsc : bits(6))
begin
    let target_el : bits(2) = DefaultTRBEEvent();
    if TRBSR_EL(target_el).S == '0' then
        TRBSR_EL(target_el).S    = '1';     // Stop collection
        if TRBSR_EL(target_el).IRQ == '0' then
            TRBSR_EL(target_el).IRQ  = '1';     // Assert interrupt or exception
            CTI_SignalEvent(CrossTriggerIn_TRBEMgmt);
        end;
        TRBSR_EL(target_el).EC   = ec;
        TRBSR_EL(target_el).MSS  = ZeroExtend{16}(bsc);
        TRBSR_EL(target_el).MSS2 = Zeros{24};
        CTI_SignalEvent(CrossTriggerIn_TRBEStop);
    end;
end;
```

Library pseudocode for shared/trace/TraceBuffer/TRBEInternalBufferFull

```
// TRBEInternalBufferFull()
// ========================

func TRBEInternalBufferFull() => boolean
begin
    // In the simple sequential model, the internal buffer never fills
    return FALSE;
end;
```

Library pseudocode for shared/trace/TraceBuffer/TRBEInterruptEnabled

```
// TRBEInterruptEnabled()
// ======================
// Return TRUE if the TRBE interrupt request (TRBIRQ) is enabled, FALSE otherwise.

func TRBEInterruptEnabled() => boolean
begin
    return EffectiveTRFCR_EL1_EE() == '00';
end;
```

Library pseudocode for shared/trace/TraceBuffer/TRBE_TRBIDR_P_Read

```
// TRBE_TRBIDR_P_Read()
// ====================
// Called when TRBIDR_EL1 is read, returns the value of TRBIDR_EL1.P

func TRBE_TRBIDR_P_Read() => bit
begin
    var owning_ss : SecurityState;
    var owning_el : bits(2);
    (owning_ss, owning_el) = TraceBufferOwner();

    // Reads as one if the Trace Buffer is owned by a higher Exception
    // Level or another Security state.
    if (UInt(owning_el) > UInt(PSTATE.EL) ||
          (PSTATE.EL != EL3 && owning_ss != CurrentSecurityState())) then
        return '1';
    else
        return '0';
    end;
end;
```

Library pseudocode for shared/trace/TraceBuffer/TRBSR_EL

```
// TRBSR_EL - accessor
// ===================

accessor TRBSR_EL(el : bits(2)) <=> value : TRBSRType
begin
    getter
        var r : bits(64);
        case el of
            when EL1 =>   r = TRBSR_EL1();
            when EL2 =>   r = TRBSR_EL2();
            when EL3 =>   r = TRBSR_EL3();
            otherwise =>  unreachable;
        end;
        return r;
    end;

    setter
        let r : bits(64) = value;
        case el of
            when EL1 =>   TRBSR_EL1() = r;
            when EL2 =>   TRBSR_EL2() = r;
            when EL3 =>   TRBSR_EL3() = r;
            otherwise =>  unreachable;
        end;
        return;
    end;
end;
```

Library pseudocode for shared/trace/TraceBuffer/TraceBufferEnabled

```
// TraceBufferEnabled()
// ====================

readonly func TraceBufferEnabled() => boolean
begin
    if !IsFeatureImplemented(FEAT_TRBE) then
        return FALSE;
    elsif SelfHostedTraceEnabled() then
        if TRBLIMITR_EL1().E == '0' then
            return FALSE;
        end;
        var el : bits(2);
        (-, el) = TraceBufferOwner();
        return !ELUsingAArch32(el);
    elsif IsFeatureImplemented(FEAT_TRBE_EXT) then
        return TRBLIMITR_EL1().XE == '1';
    else
        return FALSE;
    end;
end;
```

Library pseudocode for shared/trace/TraceBuffer/TraceBufferOwner

```
// TraceBufferOwner()
// ==================
// Return the owning Security state and Exception level. Must only be called
// when SelfHostedTraceEnabled() is TRUE.

readonly func TraceBufferOwner() => (SecurityState, bits(2))
begin
    assert IsFeatureImplemented(FEAT_TRBE);

    var owning_ss : SecurityState;
    var state_bits :  bits(3);
    if HaveEL(EL3) then
        (state_bits, -) = EffectiveMDCR_EL3_NSTB();
    else
        state_bits = if SecureOnlyImplementation() then '001' else '011';
    end;

    case state_bits of
        when '00x' => owning_ss = SS_Secure;
        when '01x' => owning_ss = SS_NonSecure;
        when '11x' => owning_ss = SS_Realm;
    end;
    var owning_el : bits(2);
    if HaveEL(EL2) && (owning_ss != SS_Secure || IsSecureEL2Enabled()) then
        owning_el = if MDCR_EL2().E2TB == '00' then EL2 else EL1;
    else
        owning_el = EL1;
    end;
    return (owning_ss, owning_el);
end;
```

Library pseudocode for shared/trace/TraceBuffer/TraceBufferRunning

```
// TraceBufferRunning()
// ====================

func TraceBufferRunning() => boolean
begin
    if !TraceBufferEnabled() then
        return FALSE;
    end;

    var stopped : boolean = TRBSR_EL1().S == '1';
    if IsFeatureImplemented(FEAT_TRBE_EXC) && SelfHostedTraceEnabled() then
        if HaveEL(EL3) && MDCR_EL3().TRBEE == '1x' then
            stopped = stopped || (TRBSR_EL3().S == '1');
        end;
        if EffectiveTRFCR_EL2_EE() == '1x' then
            stopped = stopped || (TRBSR_EL2().S == '1');
        end;
    end;
    return !stopped;
end;
```

Library pseudocode for shared/trace/TraceBuffer/TraceUnitFlushOnTriggerComplete

```
// TraceUnitFlushOnTriggerComplete()
// =================================
// Called when a trace unit flush completes following a call to
// TraceUnitFlush() due to a trace trigger.

func TraceUnitFlushOnTriggerComplete()
begin
    if TRBLIMITR_EL1().TM == '00' then     // Stop on trigger
        let bsc : bits(6) = '000010';      // Trigger event
        OtherTRBEManagementEvent(bsc);
    elsif TRBLIMITR_EL1().TM != '11' then  // Not Ignore trigger
        let target_el : bits(2) = DefaultTRBEEvent();
        TRBSR_EL(target_el).IRQ = '1';     // Assert interrupt or exception
    end;
end;
```

Library pseudocode for shared/trace/TraceBuffer/TryAssertTRBIRQ

```
// TryAssertTRBIRQ()
// =================
// Assert TRBIRQ pin when appropriate.

func TryAssertTRBIRQ()
begin
    if TRBEInterruptEnabled() && TRBSR_EL1().IRQ == '1' then
        SetInterruptRequestLevel(InterruptID_TRBIRQ, HIGH);
    else
        SetInterruptRequestLevel(InterruptID_TRBIRQ, LOW);
    end;
    return;
end;
```

Library pseudocode for shared/trace/TraceInstrumentationAllowed/TraceInstrumentationAllowed

```
// TraceInstrumentationAllowed()
// =============================
// Returns TRUE if Instrumentation Trace is allowed
// in the given Exception level and Security state.

func TraceInstrumentationAllowed(ss : SecurityState, el : bits(2)) => boolean
begin
    if !IsFeatureImplemented(FEAT_ITE) then return FALSE; end;
    if ELUsingAArch32(el) then return FALSE; end;

    if TraceAllowed(el) then
        var ite_bit : bit;
        case el of
            when EL3 => ite_bit = '0';
            when EL2 => ite_bit = TRCITECR_EL2().E2E;
            when EL1 => ite_bit = TRCITECR_EL1().E1E;
            when EL0 =>
                if EffectiveTGE() == '1' then
                    ite_bit = TRCITECR_EL2().E0HE;
                else
                    ite_bit = TRCITECR_EL1().E0E;
                end;
        end;

        if SelfHostedTraceEnabled() then
            return ite_bit == '1';
        else
            var el_bit : bit;
            var ss_bit : bit;
            case el of
                when EL0 => el_bit = TRCITEEDCR().E0;
                when EL1 => el_bit = TRCITEEDCR().E1;
                when EL2 => el_bit = TRCITEEDCR().E2;
                when EL3 => el_bit = TRCITEEDCR().E3;
            end;
            case ss of
                when SS_Realm =>     ss_bit = TRCITEEDCR().RL;
                when SS_Secure =>    ss_bit = TRCITEEDCR().S;
                when SS_NonSecure => ss_bit = TRCITEEDCR().NS;
                otherwise => ss_bit = '1';
            end;

            let ed_allowed : boolean = ss_bit == '1' && el_bit == '1';

            if TRCCONFIGR().ITO == '1' then
                return ed_allowed;
            else
                return ed_allowed && ite_bit == '1';
            end;
        end;
    else
        return FALSE;
    end;
end;
```

Library pseudocode for shared/trace/TraceProcessElements/TraceUnitFlush

```
// TraceUnitFlush()
// ================
// Called when a trace unit flush is requested, to output previous recorded trace.

impdef func TraceUnitFlush()
begin
    return;
end;
```

Library pseudocode for shared/trace/claim

```
var TRCClaimTags : bits(32);
```

Library pseudocode for shared/trace/claim/Write_TRCCLAIMCLR

```
// Write_TRCCLAIMCLR()
// ===================
// System register write to TRCCLAIMCLR.

func Write_TRCCLAIMCLR(value_in : bits(64))
begin
    assert NUM_TRC_CLAIM_TAGS <= 32;

    TRCClaimTags = TRCClaimTags AND NOT ZeroExtend{32}(value_in[0 +: NUM_TRC_CLAIM_TAGS]);
end;
```

Library pseudocode for shared/trace/claim/Write_TRCCLAIMSET

```
// Write_TRCCLAIMSET()
// ===================
// System register write to TRCCLAIMSET.

func Write_TRCCLAIMSET(value_in : bits(64))
begin
    assert NUM_TRC_CLAIM_TAGS <= 32;

    TRCClaimTags = TRCClaimTags OR ZeroExtend{32}(value_in[0 +: NUM_TRC_CLAIM_TAGS]);
end;
```

Library pseudocode for shared/trace/selfhosted/CheckTSBTrap

```
// CheckTSBTrap()
// ==============
// Checks whether TSB should trap to EL2.

func CheckTSBTrap()
begin
    if (IsFeatureImplemented(FEAT_FGT2) && IsFeatureImplemented(FEAT_TRBEv1p1) &&
          PSTATE.EL IN {EL0, EL1} && EL2Enabled() && !IsInHost() &&
          (!HaveEL(EL3) || SCR_EL3().FGTEn2 == '1') && HFGITR2_EL2().TSBCSYNC == '1') then
        let target_el : bits(2) = EL2;
        let iss : bits(25)      = 0x4[24:0];
        AArch64_OtherInstrTrap(target_el, iss);
    end;
end;

func TraceSynchronizationBarrier()
begin
    if IsFeatureImplemented(FEAT_TRBE) && !TraceAllowed(PSTATE.EL) then
        TraceUnitFlush();
    end;
    return;
end;
```

Library pseudocode for shared/trace/selfhosted/EffectiveE0HTRE

```
// EffectiveE0HTRE()
// =================
// Returns effective E0HTRE value

func EffectiveE0HTRE() => bit
begin
    return if ELUsingAArch32(EL2) then HTRFCR().E0HTRE else TRFCR_EL2().E0HTRE;
end;
```

Library pseudocode for shared/trace/selfhosted/EffectiveE0TRE

```
// EffectiveE0TRE()
// ================
// Returns effective E0TRE value

func EffectiveE0TRE() => bit
begin
    return if ELUsingAArch32(EL1) then TRFCR().E0TRE else TRFCR_EL1().E0TRE;
end;
```

Library pseudocode for shared/trace/selfhosted/EffectiveE1TRE

```
// EffectiveE1TRE()
// ================
// Returns effective E1TRE value

func EffectiveE1TRE() => bit
begin
    return if UsingAArch32() then TRFCR().E1TRE else TRFCR_EL1().E1TRE;
end;
```

Library pseudocode for shared/trace/selfhosted/EffectiveE2TRE

```
// EffectiveE2TRE()
// ================
// Returns effective E2TRE value

func EffectiveE2TRE() => bit
begin
    return if UsingAArch32() then HTRFCR().E2TRE else TRFCR_EL2().E2TRE;
end;
```

Library pseudocode for shared/trace/selfhosted/SelfHostedTraceEnabled

```
// SelfHostedTraceEnabled()
// ========================
// Returns TRUE if Self-hosted Trace is enabled.

readonly func SelfHostedTraceEnabled() => boolean
begin
    var secure_trace_enable : bit = '0';
    if !(HaveTraceExt() && IsFeatureImplemented(FEAT_TRF)) then return FALSE; end;
    if EDSCR().TFO == '0' then return TRUE; end;
    if IsFeatureImplemented(FEAT_RME) then
        secure_trace_enable = if IsFeatureImplemented(FEAT_SEL2) then MDCR_EL3().STE else '0';
        return ((secure_trace_enable == '1' && !ExternalSecureNoninvasiveDebugEnabled()) ||
                (MDCR_EL3().RLTE == '1' && !ExternalRealmNoninvasiveDebugEnabled()));
    end;
    if HaveEL(EL3) then
        secure_trace_enable = if ELUsingAArch32(EL3) then SDCR().STE else MDCR_EL3().STE;
    else
        secure_trace_enable = if SecureOnlyImplementation() then '1' else '0';
    end;

    if secure_trace_enable == '1' && !ExternalSecureNoninvasiveDebugEnabled() then
        return TRUE;
    end;

    return FALSE;
end;
```

Library pseudocode for shared/trace/selfhosted/TraceAllowed

```
// TraceAllowed()
// ==============
// Returns TRUE if Self-hosted Trace is allowed in the given Exception level.

func TraceAllowed(el : bits(2)) => boolean
begin
    if !HaveTraceExt() then
        return FALSE;
    end;
    // If in Debug state then tracing is not allowed
    if Halted() && !Restarting() then
        return FALSE;
    end;
    if SelfHostedTraceEnabled() then
        var trace_allowed : boolean;
        let ss : SecurityState = SecurityStateAtEL(el);
        // Detect scenarios where tracing in this Security state is never allowed.
        case ss of
            when SS_NonSecure =>
                trace_allowed = TRUE;
            when SS_Secure =>
                var trace_bit : bit;
                if HaveEL(EL3) then
                    trace_bit = if ELUsingAArch32(EL3) then SDCR().STE else MDCR_EL3().STE;
                else
                    trace_bit = '1';
                end;
                trace_allowed = trace_bit == '1';
            when SS_Realm =>
                trace_allowed = MDCR_EL3().RLTE == '1';
            when SS_Root =>
                trace_allowed = FALSE;
        end;

        // Tracing is prohibited if the trace buffer owning security state is not the
        // current Security state or the owning Exception level is a lower Exception level.
        if IsFeatureImplemented(FEAT_TRBE) && TraceBufferEnabled() then
            let (owning_ss, owning_el) = TraceBufferOwner();
            if (ss != owning_ss || UInt(owning_el) < UInt(el) ||
                (EffectiveTGE() == '1' && owning_el == EL1)) then
                trace_allowed = FALSE;
            end;
        end;

        var TRE_bit : bit;
        case el of
            when EL3 =>  TRE_bit = if !HaveAArch64() then TRFCR().E1TRE else '0';
            when EL2 =>  TRE_bit = EffectiveE2TRE();
            when EL1 =>  TRE_bit = EffectiveE1TRE();
            when EL0 =>
                if EffectiveTGE() == '1' then
                    TRE_bit = EffectiveE0HTRE();
                else
                    TRE_bit = EffectiveE0TRE();
                end;
        end;

        return trace_allowed && TRE_bit == '1';
    else
        return ExternalNoninvasiveDebugAllowed(el);
    end;
end;
```

Library pseudocode for shared/trace/selfhosted/TraceContextIDR2

```
// TraceContextIDR2()
// ==================

func TraceContextIDR2() => boolean
begin
    if !TraceAllowed(PSTATE.EL)|| !HaveEL(EL2) then return FALSE; end;
    return (!SelfHostedTraceEnabled() || TRFCR_EL2().CX == '1');
end;
```

Library pseudocode for shared/trace/selfhosted/TraceTimeStamp

```
// TraceTimeStamp()
// ================

func TraceTimeStamp() => TimeStamp
begin
    if SelfHostedTraceEnabled() then
        if HaveEL(EL2) then
            var TS_el2 : bits(2) = TRFCR_EL2().TS;
            if !IsFeatureImplemented(FEAT_ECV) && TS_el2 == '10' then
                // Reserved value
                (-, TS_el2) = ConstrainUnpredictableBits{2}(Unpredictable_EL2TIMESTAMP);
            end;

            case TS_el2 of
                when '00' =>
                    // Falls out to check TRFCR_EL1.TS
                    pass;
                when '01' =>
                    return TimeStamp_Virtual;
                when '10' =>
                    // Otherwise ConstrainUnpredictableBits removes this case
                    assert IsFeatureImplemented(FEAT_ECV);
                    return TimeStamp_OffsetPhysical;
                when '11' =>
                    return TimeStamp_Physical;
            end;
        end;

        var TS_el1 : bits(2) = TRFCR_EL1().TS;
        if TS_el1 == '00' || (!IsFeatureImplemented(FEAT_ECV) && TS_el1 == '10') then
            // Reserved value
            (-, TS_el1) = ConstrainUnpredictableBits{2}(Unpredictable_EL1TIMESTAMP);
        end;

        case TS_el1 of
            when '01' =>
                return TimeStamp_Virtual;
            when '10' =>
                assert IsFeatureImplemented(FEAT_ECV);
                return TimeStamp_OffsetPhysical;
            when '11' =>
                return TimeStamp_Physical;
            otherwise =>
                unreachable;         // ConstrainUnpredictableBits removes this case
        end;
    else
        return TimeStamp_CoreSight;
    end;
end;
```

Library pseudocode for shared/trace/system/HaveTraceExt

```
// HaveTraceExt()
// ==============
// Returns TRUE if Trace functionality as described by the Trace Architecture
// is implemented.

readonly func HaveTraceExt() => boolean
begin
    return IsFeatureImplemented(FEAT_ETE) || IsFeatureImplemented(FEAT_ETMv4);
end;
```

Library pseudocode for shared/trace/system/IsTraceCorePowered

```
// IsTraceCorePowered()
// ====================
// Returns TRUE if the trace unit Core power domain is powered up

impdef func IsTraceCorePowered() => boolean
begin
    return TRUE;
end;
```

Library pseudocode for shared/translation/at

```
type TranslationStage of enumeration {
    TranslationStage_1,
    TranslationStage_12
};

type ATAccess of enumeration {
    ATAccess_Read,
    ATAccess_Write,
    ATAccess_Any,
    ATAccess_ReadPAN,
    ATAccess_WritePAN
};
```

Library pseudocode for shared/translation/at/EncodePARAttrs

```
// EncodePARAttrs()
// ================
// Convert orthogonal attributes and hints to 64-bit PAR ATTR field.

func EncodePARAttrs(memattrs : MemoryAttributes) => bits(8)
begin
    var result : bits(8);

    if IsFeatureImplemented(FEAT_MTE) && memattrs.tags == MemTag_AllocationTagged then
        if IsFeatureImplemented(FEAT_MTE_PERM) && memattrs.notagaccess then
            result[7:0] = '11100000';
        else
            result[7:0] = '11110000';
        end;
        return result;
    end;

    if memattrs.memtype == MemType_Device then
        result[7:4] = '0000';
        case memattrs.device of
            when DeviceType_nGnRnE => result[3:0] = '0000';
            when DeviceType_nGnRE  => result[3:0] = '0100';
            when DeviceType_nGRE   => result[3:0] = '1000';
            when DeviceType_GRE    => result[3:0] = '1100';
            otherwise              => unreachable;
        end;
        result[0] = NOT memattrs.xs;
    else
        if memattrs.xs == '0' then
            if (memattrs.outer.attrs == MemAttr_WT && memattrs.inner.attrs == MemAttr_WT &&
                  !memattrs.outer.transient && memattrs.outer.hints == MemHint_RA) then
                return '10100000';
            elsif memattrs.outer.attrs == MemAttr_NC && memattrs.inner.attrs == MemAttr_NC then
                return '01000000';
            end;
        end;

        if memattrs.outer.attrs == MemAttr_WT then
            result[7:6] = if memattrs.outer.transient then '00' else '10';
            result[5:4] = memattrs.outer.hints;
        elsif memattrs.outer.attrs == MemAttr_WB then
            result[7:6] = if memattrs.outer.transient then '01' else '11';
            result[5:4] = memattrs.outer.hints;
        else // MemAttr_NC
            result[7:4] = '0100';
        end;

        if memattrs.inner.attrs == MemAttr_WT then
            result[3:2] = if memattrs.inner.transient then '00' else '10';
            result[1:0] = memattrs.inner.hints;
        elsif memattrs.inner.attrs == MemAttr_WB then
            result[3:2] = if memattrs.inner.transient then '01' else '11';
            result[1:0] = memattrs.inner.hints;
        else // MemAttr_NC
            result[3:0] = '0100';
        end;
    end;

    return result;
end;
```

Library pseudocode for shared/translation/at/PAREncodeShareability

```
// PAREncodeShareability()
// ======================
// Derive 64-bit PAR SH field.

func PAREncodeShareability(mematTrs : MemoryAttributes) => bits(2)
begin
    if (mematTrs.memtype == MemType_Device ||
            (mematTrs.inner.attrs == MemAttr_NC &&
             mematTrs.outer.attrs == MemAttr_NC)) then
        // Force Outer-Shareable on Device and Normal Non-Cacheable memory
        return '10';
    end;

    case mematTrs.shareability of
        when Shareability_NSH => return '00';
        when Shareability_ISH => return '11';
        when Shareability_OSH => return '10';
    end;
end;
```

Library pseudocode for shared/translation/at/ReportedPARAttrs

```
// ReportedPARAttrs()
// ==================
// The value returned in this field can be the resulting attribute, as determined by any permitted
// implementation choices and any applicable configuration bits, instead of the value that appears
// in the translation table descriptor.

impdef func ReportedPARAttrs(parattrs : bits(8)) => bits(8)
begin
    return parattrs;
end;
```

Library pseudocode for shared/translation/at/ReportedPARShareability

```
// ReportedPARShareability()
// =========================
// The value returned in SH field can be the resulting attribute, as determined by any
// permitted implementation choices and any applicable configuration bits, instead of
// the value that appears in the translation table descriptor.

impdef func ReportedPARShareability(sh : bits(2)) => bits(2)
begin
    return sh;
end;
```

Library pseudocode for shared/translation/attrs/DecodeDevice

```
// DecodeDevice()
// ==============
// Decode output Device type

readonly func DecodeDevice(device : bits(2)) => DeviceType
begin
    case device of
        when '00' => return DeviceType_nGnRnE;
        when '01' => return DeviceType_nGnRE;
        when '10' => return DeviceType_nGRE;
        when '11' => return DeviceType_GRE;
    end;
end;
```

Library pseudocode for shared/translation/attrs/DecodeLDFAttr

```
// DecodeLDFAttr()
// ===============
// Decode memory attributes using LDF (Long Descriptor Format) mapping

readonly func DecodeLDFAttr(attr : bits(4)) => MemAttrHints
begin
    var ldfattr : MemAttrHints;

    if    attr == 'x0xx' then ldfattr.attrs = MemAttr_WT; // Write-through
    elsif attr == '0100' then ldfattr.attrs = MemAttr_NC; // Non-cacheable
    elsif attr == 'x1xx' then ldfattr.attrs = MemAttr_WB; // Write-back
    else                      unreachable;
    end;

    // Allocation hints are applicable only to cacheable memory.
    if ldfattr.attrs != MemAttr_NC then
        case attr[1:0] of
            when '00' => ldfattr.hints = MemHint_No;  // No allocation hints
            when '01' => ldfattr.hints = MemHint_WA;  // Write-allocate
            when '10' => ldfattr.hints = MemHint_RA;  // Read-allocate
            when '11' => ldfattr.hints = MemHint_RWA; // Read/Write allocate
        end;
    end;

    // The Transient hint applies only to cacheable memory with some allocation hints.
    if ldfattr.attrs != MemAttr_NC && ldfattr.hints != MemHint_No then
        ldfattr.transient = attr[3] == '0';
    end;

    return ldfattr;
end;
```

Library pseudocode for shared/translation/attrs/DecodeSDFAttr

```
// DecodeSDFAttr()
// ===============
// Decode memory attributes using SDF (Short Descriptor Format) mapping

func DecodeSDFAttr(rgn : bits(2)) => MemAttrHints
begin
    var sdfattr : MemAttrHints;

    case rgn of
        when '00' =>                    // Non-cacheable (no allocate)
            sdfattr.attrs = MemAttr_NC;
        when '01' =>                    // Write-back, Read and Write allocate
            sdfattr.attrs = MemAttr_WB;
            sdfattr.hints = MemHint_RWA;
        when '10' =>                    // Write-through, Read allocate
            sdfattr.attrs = MemAttr_WT;
            sdfattr.hints = MemHint_RA;
        when '11' =>                    // Write-back, Read allocate
            sdfattr.attrs = MemAttr_WB;
            sdfattr.hints = MemHint_RA;
    end;

    sdfattr.transient = FALSE;

    return sdfattr;
end;
```

Library pseudocode for shared/translation/attrs/DecodeShareability

```
// DecodeShareability()
// ====================
// Decode shareability of target memory region

readonly func DecodeShareability(sh : bits(2)) => Shareability
begin
    case sh of
        when '10' => return Shareability_OSH;
        when '11' => return Shareability_ISH;
        when '00' => return Shareability_NSH;
        otherwise =>
            case ConstrainUnpredictable(Unpredictable_Shareability) of
                when Constraint_OSH => return Shareability_OSH;
                when Constraint_ISH => return Shareability_ISH;
                when Constraint_NSH => return Shareability_NSH;
            end;
    end;
end;
```

Library pseudocode for shared/translation/attrs/EffectiveShareability

```
// EffectiveShareability()
// =======================
// Force Outer Shareability on Device and Normal iNCoNC memory

func EffectiveShareability(mematTrs : MemoryAttributes) => Shareability
begin
    if (mematTrs.memtype == MemType_Device ||
          (mematTrs.inner.attrs == MemAttr_NC &&
           mematTrs.outer.attrs == MemAttr_NC)) then
        return Shareability_OSH;
    else
        return mematTrs.shareability;
    end;
end;
```

Library pseudocode for shared/translation/attrs/IsTaggableMemAttr

```
// IsTaggableMemAttr()
// ===================
// Determine whether the current memory attributes support MTE

pure func IsTaggableMemAttr(attrs : MemoryAttributes) => boolean
begin
    return ((attrs.memtype == MemType_Normal)  &&
            (attrs.inner.attrs == MemAttr_WB)  &&
            (attrs.inner.hints == MemHint_RWA) &&
            (!attrs.inner.transient)           &&
            (attrs.outer.attrs == MemAttr_WB)  &&
            (attrs.outer.hints == MemHint_RWA) &&
            (!attrs.outer.transient));
end;
```

Library pseudocode for shared/translation/attrs/IsWBShareable

```
// IsWBShareable()
// ===============
// Determines whether the given memory attributes are iWBoWB Shareable

func IsWBShareable(mematTrs : MemoryAttributes) => boolean
begin

    return (mematTrs.memtype == MemType_Normal &&
            mematTrs.inner.attrs == MemAttr_WB &&
            mematTrs.outer.attrs == MemAttr_WB &&
            mematTrs.shareability IN {Shareability_ISH, Shareability_OSH});
end;
```

Library pseudocode for shared/translation/attrs/NormalNCMemAttr

```
// NormalNCMemAttr()
// =================
// Normal Non-cacheable memory attributes

func NormalNCMemAttr() => MemoryAttributes
begin
    var non_cacheable : MemAttrHints;
    non_cacheable.attrs = MemAttr_NC;

    var nc_mematTrs : MemoryAttributes;
    nc_mematTrs.memtype      = MemType_Normal;
    nc_mematTrs.outer        = non_cacheable;
    nc_mematTrs.inner        = non_cacheable;
    nc_mematTrs.shareability = Shareability_OSH;
    nc_mematTrs.tags         = MemTag_Untagged;
    nc_mematTrs.notagaccess  = FALSE;

    return nc_mematTrs;
end;
```

Library pseudocode for shared/translation/attrs/S1ConstrainUnpredictableRESMAIR

```
// S1ConstrainUnpredictableRESMAIR()
// =================================
// Determine whether a reserved value occupies MAIR_ELx.AttrN

readonly func S1ConstrainUnpredictableRESMAIR(attr : bits(8), s1aarch64 : boolean) => boolean
begin
    case attr of
        when '0000xx01' => return !(s1aarch64 && IsFeatureImplemented(FEAT_XS));
        when '0000xxxx' => return attr[1:0] != '00';
        when '01000000' => return !(s1aarch64 && IsFeatureImplemented(FEAT_XS));
        when '10100000' => return !(s1aarch64 && IsFeatureImplemented(FEAT_XS));
        when '11110000' => return !(s1aarch64 && IsFeatureImplemented(FEAT_MTE2));
        when 'xxxx0000' => return TRUE;
        otherwise       => return FALSE;
    end;
end;
```

Library pseudocode for shared/translation/attrs/S1DecodeMemAttrs

```
// S1DecodeMemAttrs()
// ==================
// Decode MAIR-format memory attributes assigned in stage 1

readonly func S1DecodeMemAttrs(attr_in : bits(8), sh : bits(2),
                               s1aarch64 : boolean) => MemoryAttributes
begin
    var attr : bits(8) = attr_in;
    if S1ConstrainUnpredictableRESMAIR(attr, s1aarch64) then
        // Map reserved encodings to an allocated encoding
        (-, attr) = ConstrainUnpredictableBits{8}(Unpredictable_RESMAIR);
    end;

    var memattrs : MemoryAttributes;
    case attr of
        when '0000xxxx' => // Device memory
            memattrs.memtype = MemType_Device;
            memattrs.device  = DecodeDevice(attr[3:2]);
            memattrs.xs      = if s1aarch64 then NOT attr[0] else '1';
        when '01000000' =>
            assert s1aarch64 && IsFeatureImplemented(FEAT_XS);
            memattrs.memtype = MemType_Normal;
            memattrs.outer.attrs = MemAttr_NC;
            memattrs.inner.attrs = MemAttr_NC;
            memattrs.xs          = '0';
        when '10100000' =>
            assert s1aarch64 && IsFeatureImplemented(FEAT_XS);
            memattrs.memtype = MemType_Normal;
            memattrs.outer.attrs     = MemAttr_WT;
            memattrs.outer.hints     = MemHint_RA;
            memattrs.outer.transient = FALSE;
            memattrs.inner.attrs     = MemAttr_WT;
            memattrs.inner.hints     = MemHint_RA;
            memattrs.inner.transient = FALSE;
            memattrs.xs              = '0';
        when '11110000' => // Tagged memory
            assert s1aarch64 && IsFeatureImplemented(FEAT_MTE2);
            memattrs.memtype = MemType_Normal;
            memattrs.outer.attrs     = MemAttr_WB;
            memattrs.outer.hints     = MemHint_RWA;
            memattrs.outer.transient = FALSE;
            memattrs.inner.attrs     = MemAttr_WB;
            memattrs.inner.hints     = MemHint_RWA;
            memattrs.inner.transient = FALSE;
            memattrs.xs              = '0';
        otherwise =>
            memattrs.memtype = MemType_Normal;
            memattrs.outer   = DecodeLDFAttr(attr[7:4]);
            memattrs.inner   = DecodeLDFAttr(attr[3:0]);

            if (memattrs.inner.attrs == MemAttr_WB &&
                    memattrs.outer.attrs == MemAttr_WB) then
                memattrs.xs = '0';
            else
                memattrs.xs = '1';
            end;
    end;

    memattrs.shareability = DecodeShareability(sh);
    memattrs.tags = MemTag_Untagged;
    memattrs.notagaccess = FALSE;

    return memattrs;
end;

// S1DecodeMemAttrs()
// ==================
// Decode MAIR-format memory attributes assigned in stage 1

readonly func S1DecodeMemAttrs(attr_in : bits(8), sh : bits(2), s1aarch64 : boolean,
                               walkparams : S1TTWParams, acctype : AccessType) => MemoryAttributes
begin
    var memattrs : MemoryAttributes = S1DecodeMemAttrs(attr_in, sh, s1aarch64);
    if s1aarch64 && attr_in == '11110000' then
        memattrs.tags = MemTag_AllocationTagged;
    elsif IsFeatureImplemented(FEAT_MTE_CANONICAL_TAGS) && s1aarch64 && walkparams.mtx == '1' then
        memattrs.tags = MemTag_CanonicallyTagged;
    end;

    return memattrs;
end;
```

Library pseudocode for shared/translation/attrs/S2CombineS1AttrHints

```
// S2CombineS1AttrHints()
// ======================
// Determine resultant Normal memory cacheability and allocation hints from
// combining stage 1 Normal memory attributes and stage 2 cacheability attributes.

func S2CombineS1AttrHints(s1_attrhints : MemAttrHints, s2_attrhints : MemAttrHints) => MemAttrHints
begin
    var attrhints : MemAttrHints;

    if s1_attrhints.attrs == MemAttr_NC || s2_attrhints.attrs == MemAttr_NC then
        attrhints.attrs = MemAttr_NC;
    elsif s1_attrhints.attrs == MemAttr_WT || s2_attrhints.attrs == MemAttr_WT then
        attrhints.attrs = MemAttr_WT;
    else
        attrhints.attrs = MemAttr_WB;
    end;

    // Stage 2 does not assign any allocation hints
    // Instead, they are inherited from stage 1
    if attrhints.attrs != MemAttr_NC then
        attrhints.hints     = s1_attrhints.hints;
        attrhints.transient = s1_attrhints.transient;
    end;

    return attrhints;
end;
```

Library pseudocode for shared/translation/attrs/S2CombineS1Device

```
// S2CombineS1Device()
// ===================
// Determine resultant Device type from combining output memory attributes
// in stage 1 and Device attributes in stage 2

func S2CombineS1Device(s1_device : DeviceType, s2_device : DeviceType) => DeviceType
begin
    if s1_device == DeviceType_nGnRnE || s2_device == DeviceType_nGnRnE then
        return DeviceType_nGnRnE;
    elsif s1_device == DeviceType_nGnRE || s2_device == DeviceType_nGnRE then
        return DeviceType_nGnRE;
    elsif s1_device == DeviceType_nGRE || s2_device == DeviceType_nGRE then
        return DeviceType_nGRE;
    else
        return DeviceType_GRE;
    end;
end;
```

Library pseudocode for shared/translation/attrs/S2CombineS1MemAttrs

```
// S2CombineS1MemAttrs()
// =====================
// Combine stage 2 with stage 1 memory attributes

func S2CombineS1MemAttrs(s1_mematttrs : MemoryAttributes, s2_mematttrs : MemoryAttributes,
                         s2aarch64 : boolean) => MemoryAttributes
begin
    var mematttrs : MemoryAttributes;

    if s1_mematttrs.memtype == MemType_Device && s2_mematttrs.memtype == MemType_Device then
        mematttrs.memtype = MemType_Device;
        mematttrs.device  = S2CombineS1Device(s1_mematttrs.device, s2_mematttrs.device);
    elsif s1_mematttrs.memtype == MemType_Device then    // S2 Normal, S1 Device
        mematttrs = s1_mematttrs;
    elsif s2_mematttrs.memtype == MemType_Device then    // S2 Device, S1 Normal
        mematttrs = s2_mematttrs;
    else                                                 // S2 Normal, S1 Normal
        mematttrs.memtype = MemType_Normal;
        mematttrs.inner   = S2CombineS1AttrHints(s1_mematttrs.inner, s2_mematttrs.inner);
        mematttrs.outer   = S2CombineS1AttrHints(s1_mematttrs.outer, s2_mematttrs.outer);
    end;

    mematttrs.tags = S2MemTagType(mematttrs, s1_mematttrs.tags);

    if !IsFeatureImplemented(FEAT_MTE_PERM) then
        mematttrs.notagaccess = FALSE;
    else
        mematttrs.notagaccess = (s2_mematttrs.notagaccess &&
                                 s1_mematttrs.tags == MemTag_AllocationTagged);
    end;
    mematttrs.shareability = S2CombineS1Shareability(s1_mematttrs.shareability,
                                                     s2_mematttrs.shareability);

    if (mematttrs.memtype == MemType_Normal &&
            mematttrs.inner.attrs == MemAttr_WB &&
            mematttrs.outer.attrs == MemAttr_WB) then
        mematttrs.xs = '0';
    elsif s2aarch64 then
        mematttrs.xs = s2_mematttrs.xs AND s1_mematttrs.xs;
    else
        mematttrs.xs = s1_mematttrs.xs;
    end;

    mematttrs.shareability = EffectiveShareability(mematttrs);
    return mematttrs;
end;
```

Library pseudocode for shared/translation/attrs/S2CombineS1Shareability

```
// S2CombineS1Shareability()
// =========================
// Combine stage 2 shareability with stage 1

func S2CombineS1Shareability(s1_shareability : Shareability,
                             s2_shareability : Shareability) => Shareability
begin

    if (s1_shareability == Shareability_OSH ||
            s2_shareability == Shareability_OSH) then
        return Shareability_OSH;
    elsif (s1_shareability == Shareability_ISH ||
            s2_shareability == Shareability_ISH) then
        return Shareability_ISH;
    else
        return Shareability_NSH;
    end;
end;
```

Library pseudocode for shared/translation/attrs/S2DecodeCacheability

```
// S2DecodeCacheability()
// ======================
// Determine the stage 2 cacheability for Normal memory

func S2DecodeCacheability(attr : bits(2)) => MemAttrHints
begin
    var s2attr : MemAttrHints;

    case attr of
        when '01' => s2attr.attrs = MemAttr_NC;  // Non-cacheable
        when '10' => s2attr.attrs = MemAttr_WT;  // Write-through
        when '11' => s2attr.attrs = MemAttr_WB;  // Write-back
        otherwise => unreachable;
    end;

    // Stage 2 does not assign hints or the transient property
    // They are inherited from stage 1 if the result of the combination allows it
    s2attr.hints     = ARBITRARY : bits(2);
    s2attr.transient = ARBITRARY : boolean;

    return s2attr;
end;
```

Library pseudocode for shared/translation/attrs/S2DecodeMemAttrs

```
// S2DecodeMemAttrs()
// ==================
// Decode stage 2 memory attributes when FWB is 0

func S2DecodeMemAttrs(attr_in : bits(4), sh : bits(2), s2aarch64 : boolean) => MemoryAttributes
begin
    var memattrs : MemoryAttributes;

    var attr : bits(4);
    if S2ResMemAttr(s2aarch64, attr_in) then
        // Map reserved encodings to an allocated encoding
        (-, attr) = ConstrainUnpredictableBits{4}(Unpredictable_S2RESMEMATTR);
    else
        attr = attr_in;
    end;

    case attr of
        when '00xx' => // Device memory
            memattrs.memtype      = MemType_Device;
            memattrs.device       = DecodeDevice(attr[1:0]);
        when '0100' => // Normal, Inner+Outer WB cacheable NoTagAccess memory
            assert s2aarch64 && IsFeatureImplemented(FEAT_MTE_PERM);
            memattrs.memtype      = MemType_Normal;
            memattrs.outer        = S2DecodeCacheability('11'); // Write-back
            memattrs.inner        = S2DecodeCacheability('11'); // Write-back
        otherwise =>   // Normal memory
            memattrs.memtype      = MemType_Normal;
            memattrs.outer        = S2DecodeCacheability(attr[3:2]);
            memattrs.inner        = S2DecodeCacheability(attr[1:0]);
    end;

    memattrs.shareability = DecodeShareability(sh);

    if s2aarch64 && IsFeatureImplemented(FEAT_MTE_PERM) then
        memattrs.notagaccess = attr == '0100';
    else
        memattrs.notagaccess = FALSE;
    end;

    return memattrs;
end;
```

Library pseudocode for shared/translation/attrs/S2MemTagType

```
// S2MemTagType()
// ==============
// Determine whether the combined output memory attributes of stage 1 and
// stage 2 indicate tagged memory

readonly func S2MemTagType(s2_memattrs : MemoryAttributes, s1_tagtype : MemTagType) => MemTagType
begin

    if !IsFeatureImplemented(FEAT_MTE2) then
        return MemTag_Untagged;
    end;

    if s1_tagtype == MemTag_AllocationTagged && IsTaggableMemAttr(s2_memattrs) then
        return MemTag_AllocationTagged;
    end;

    // Return what stage 1 asked for if we can, otherwise Untagged.
    if s1_tagtype != MemTag_AllocationTagged then
        return s1_tagtype;
    end;

    return MemTag_Untagged;
end;
```

Library pseudocode for shared/translation/attrs/S2ResMemAttr

```
// S2ResMemAttr()
// ==============
// Determine whether a reserved value occupies stage 2 MemAttr field when FWB is 0

func S2ResMemAttr(s2aarch64 : boolean, attr : bits(4)) => boolean
begin
    case attr of
        when '0100' => return !(s2aarch64 && IsFeatureImplemented(FEAT_MTE_PERM));
        when '1x00' => return TRUE;
        otherwise   => return FALSE;
    end;
end;
```

Library pseudocode for shared/translation/attrs/WalkMemAttrs

```
// WalkMemAttrs()
// ==============
// Retrieve memory attributes of translation table walk

func WalkMemAttrs(sh : bits(2), irgn : bits(2), orgn : bits(2)) => MemoryAttributes
begin
    var walkmemattrs : MemoryAttributes;

    walkmemattrs.memtype      = MemType_Normal;
    walkmemattrs.shareability = DecodeShareability(sh);
    walkmemattrs.inner        = DecodeSDFAttr(irgn);
    walkmemattrs.outer        = DecodeSDFAttr(orgn);
    walkmemattrs.tags         = MemTag_Untagged;
    if (walkmemattrs.inner.attrs == MemAttr_WB &&
            walkmemattrs.outer.attrs == MemAttr_WB) then
        walkmemattrs.xs = '0';
    else
        walkmemattrs.xs = '1';
    end;
    walkmemattrs.notagaccess = FALSE;

    return walkmemattrs;
end;
```

Library pseudocode for shared/translation/faults/AlignmentFault

```
// AlignmentFault()
// ================
// Return a fault record indicating an Alignment fault not due to memory type has occurred
// for a specific access

func AlignmentFault(accdesc : AccessDescriptor, vaddress : bits(64)) => FaultRecord
begin
    var fault : FaultRecord = NoFault(accdesc, vaddress);
    fault.statuscode        = Fault_Alignment;

    return fault;
end;
```

Library pseudocode for shared/translation/faults/ExclusiveFault

```
// ExclusiveFault()
// ================
// Return a fault record indicating a fault for an unsupported Exclusive access

func ExclusiveFault(accdesc : AccessDescriptor, vaddress : bits(64)) => FaultRecord
begin
    var fault : FaultRecord = NoFault(accdesc, vaddress);
    fault.statuscode        = Fault_Exclusive;

    return fault;
end;
```

Library pseudocode for shared/translation/faults/NoFault

```
// NoFault()
// =========
// Return a clear fault record indicating no faults have occurred

func NoFault() => FaultRecord
begin
    var fault : FaultRecord;

    fault.vaddress = ARBITRARY : bits(64);
    fault.statuscode  = Fault_None;
    fault.accessdesc  = ARBITRARY : AccessDescriptor;
    fault.secondstage = FALSE;
    fault.s2fs1walk   = FALSE;
    fault.dirtybit    = FALSE;
    fault.overlay     = FALSE;
    fault.toplevel    = FALSE;
    fault.assuredonly = FALSE;
    fault.s1tagnotdata = FALSE;
    fault.tagaccess    = FALSE;
    fault.gpcfs2walk   = FALSE;
    fault.gpcf         = GPCNoFault();
    fault.hdbssf       = FALSE;

    return fault;
end;

// NoFault()
// =========
// Return a clear fault record indicating no faults have occurred for a specific access

func NoFault(accdesc : AccessDescriptor) => FaultRecord
begin
    var fault : FaultRecord;

    fault.statuscode  = Fault_None;
    fault.accessdesc  = accdesc;
    fault.secondstage = FALSE;
    fault.s2fs1walk   = FALSE;
    fault.dirtybit    = FALSE;
    fault.overlay     = FALSE;
    fault.toplevel    = FALSE;
    fault.assuredonly = FALSE;
    fault.s1tagnotdata = FALSE;
    fault.tagaccess   = FALSE;
    fault.write       = !accdesc.read && accdesc.write;
    fault.gpcfs2walk  = FALSE;
    fault.gpcf        = GPCNoFault();
    fault.hdbssf      = FALSE;

    return fault;
end;

// NoFault()
// =========

func NoFault(accdesc : AccessDescriptor, vaddress : bits(64)) => FaultRecord
begin
    var fault : FaultRecord = NoFault();
    fault.accessdesc  = accdesc;
    fault.write       = !accdesc.read && accdesc.write;
    fault.vaddress    = vaddress;

    return fault;
end;
```

Library pseudocode for shared/translation/gpc/AbovePPS

```
// AbovePPS()
// ==========
// Returns TRUE if an address exceeds the range configured in GPCCR_EL3.PPS.

func AbovePPS(address : bits(NUM_PABITS)) => boolean
begin
    let pps : integer{} = DecodePPS();
    if pps >= NUM_PABITS then
        return FALSE;
    end;

    return !IsZero(address[NUM_PABITS-1:pps]);
end;
```

Library pseudocode for shared/translation/gpc/DecodeGPTBlock

```
// DecodeGPTBlock()
// ================
// Decode a GPT Block descriptor.

func DecodeGPTBlock(pgs : PGSe, gpt_entry : bits(64)) => GPTEntry
begin
    assert gpt_entry[3:0] == GPT_Block;
    var result : GPTEntry;
    result.gpi   = gpt_entry[7:4];
    result.level = 0;
    result.istable = FALSE;

    // GPT information from a level 0 GPT Block descriptor is permitted
    // to be cached in a TLB as though the Block is a contiguous region
    // of granules each of the size configured in GPCCR_EL3.PGS.
    case pgs of
        when PGS_4KB =>  result.size = GPTRange_4KB;
        when PGS_16KB => result.size = GPTRange_16KB;
        when PGS_64KB => result.size = GPTRange_64KB;
        otherwise => unreachable;
    end;
    result.contig_size = GPTL0Size();

    return result;
end;
```

Library pseudocode for shared/translation/gpc/DecodeGPTContiguous

```
// DecodeGPTContiguous()
// =====================
// Decode a GPT Contiguous descriptor.

func DecodeGPTContiguous(pgs : PGSe, gpt_entry : bits(64)) => GPTEntry
begin
    assert gpt_entry[3:0] == GPT_Contig;
    var result : GPTEntry;
    result.gpi = gpt_entry[7:4];

    case pgs of
        when PGS_4KB =>  result.size = GPTRange_4KB;
        when PGS_16KB => result.size = GPTRange_16KB;
        when PGS_64KB => result.size = GPTRange_64KB;
        otherwise     => unreachable;
    end;

    case gpt_entry[9:8] of
        when '01' => result.contig_size = GPTRange_2MB;
        when '10' => result.contig_size = GPTRange_32MB;
        when '11' => result.contig_size = GPTRange_512MB;
        otherwise => unreachable;
    end;

    result.level = 1;
    result.istable = FALSE;

    return result;
end;
```

Library pseudocode for shared/translation/gpc/DecodeGPTGranules

```
// DecodeGPTGranules()
// ===================
// Decode a GPT Granules descriptor.

func DecodeGPTGranules(pgs : PGSe, index : integer, gpt_entry : bits(64)) => GPTEntry
begin
    var result : GPTEntry;
    result.gpi = gpt_entry[index*4 +:4];

    case pgs of
        when PGS_4KB  => result.size = GPTRange_4KB;
        when PGS_16KB => result.size = GPTRange_16KB;
        when PGS_64KB => result.size = GPTRange_64KB;
        otherwise     => unreachable;
    end;

    result.contig_size = result.size; // No contiguity
    result.level = 1;
    result.istable = FALSE;

    return result;
end;
```

Library pseudocode for shared/translation/gpc/DecodeGPTTable

```
// DecodeGPTTable()
// ================
// Decode a GPT Table descriptor.

func DecodeGPTTable(pgs : PGSe, gpt_entry : bits(64)) => GPTTable
begin
    assert gpt_entry[3:0] == GPT_Table;
    var result : GPTTable;

    // Descriptor bits [L0GPTSZ-PGS-2:12] are RES0
    var s : integer{30, 34, 36, 39};
    case GPTL0Size() of
        when GPTRange_1GB   => s = 30;
        when GPTRange_16GB  => s = 34;
        when GPTRange_64GB  => s = 36;
        when GPTRange_512GB => s = 39;
        otherwise           => unreachable;
    end;

    var p : integer{12, 14, 16};
    case pgs of
        when PGS_4KB  => p = 12;
        when PGS_16KB => p = 14;
        when PGS_64KB => p = 16;
        otherwise     => unreachable;
    end;

    let align_bits :  integer{} = (s - p) - 1;
    result.address              = AlignDownP2(gpt_entry[NUM_PABITS-1:0],align_bits);

    return result;
end;
```

Library pseudocode for shared/translation/gpc/DecodePGS

```
// DecodePGS()
// ===========

pure func DecodePGS(pgs : bits(2)) => PGSe
begin
    case pgs of
        when '00' => return PGS_4KB;
        when '10' => return PGS_16KB;
        when '01' => return PGS_64KB;
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for shared/translation/gpc/DecodePGSRange

```
// DecodePGSRange()
// ================

pure func DecodePGSRange(pgs : PGSe) => AddressSize
begin
    case pgs of
        when PGS_4KB =>  return GPTRange_4KB;
        when PGS_16KB => return GPTRange_16KB;
        when PGS_64KB => return GPTRange_64KB;
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for shared/translation/gpc/DecodePPS

```
// DecodePPS()
// ===========
// Size of region protected by the GPT, in bits.

readonly func DecodePPS() => AddressSize
begin
    case GPCCR_EL3().[PPS3, PPS] of
        when '0000' => return 32;
        when '0001' => return 36;
        when '0010' => return 40;
        when '0011' => return 42;
        when '0100' => return 44;
        when '0101' => return 48;
        when '0110' => return 52;
        when '0111' => assert IsFeatureImplemented(FEAT_RME_GPC3); return 56;
        when '1000' => assert IsFeatureImplemented(FEAT_RME_GPC3); return 46;
        when '1001' => assert IsFeatureImplemented(FEAT_RME_GPC3); return 47;
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for shared/translation/gpc/GPCBW_EL3BWSTRIDEValid

```
// GPCBW_EL3BWSTRIDEValid()
// ========================
// Returns whether the current GPCBW_EL3.BWSTRIDE value is valid

readonly func GPCBW_EL3BWSTRIDEValid() => boolean
begin
    assert IsFeatureImplemented(FEAT_RME_GPC3);
    return GPCBW_EL3().BWSTRIDE IN {
        '00000',
        '00010',
        '00100',
        '00110',
        '00111',
        '01000',
        '01001',
        '01010',
        '10000'
    };
end;
```

Library pseudocode for shared/translation/gpc/GPCFault

```
// GPCFault()
// ==========
// Constructs and returns a GPCF

func GPCFault(gpf : GPCF, level : integer) => GPCFRecord
begin
    var fault : GPCFRecord;
    fault.gpf   = gpf;
    fault.level = level;
    return fault;
end;
```

Library pseudocode for shared/translation/gpc/GPCNoFault

```
// GPCNoFault()
// ============
// Returns the default properties of a GPCF that does not represent a fault

func GPCNoFault() => GPCFRecord
begin
    var result : GPCFRecord;
    result.gpf = GPCF_None;
    return result;
end;
```

Library pseudocode for shared/translation/gpc/GPCRegistersConsistent

```
// GPCRegistersConsistent()
// ========================
// Returns whether the GPT registers are configured correctly.
// This returns false if any fields select a Reserved value.

readonly func GPCRegistersConsistent() => boolean
begin
    // Check for Reserved register values
    if IsFeatureImplemented(FEAT_RME_GPC3) then
        if  ! GPCCR_EL3().[PPS3, PPS] IN {'0xxx', '100x'} then
            return FALSE;
        end;

        if GPCCR_EL3().GPCBW == '1' then
            if ! GPCBW_EL3().BWSIZE IN {'00x', '1x0', '010'} then
                return FALSE;
            end;

            if !GPCBW_EL3BWSTRIDEValid() then
                return FALSE;
            end;

            let bwstride : integer = 1 << (40 + UInt(GPCBW_EL3().BWSTRIDE));
            let bwaddr : integer = UInt(GPCBW_EL3().BWADDR) << 30;
            if bwstride <= bwaddr then
                return FALSE;
            end;

            if !IsAlignedSize{26}(GPCBW_EL3().BWADDR, 1 << UInt(GPCBW_EL3().BWSIZE)) then
                return FALSE;
            end;
        end;
    else
        if GPCCR_EL3().PPS == '111' then
            return FALSE;
        end;
    end;

    if DecodePPS() > AArch64_PAMax() then
        return FALSE;
    end;
    if GPCCR_EL3().PGS == '11' then
        return FALSE;
    end;
    if GPCCR_EL3().SH == '01' then
        return FALSE;
    end;

    // Inner and Outer Non-cacheable requires Outer Shareable
    if GPCCR_EL3().[ORGN, IRGN] == '0000' && GPCCR_EL3().SH != '10' then
        return FALSE;
    end;

    return TRUE;
end;
```

Library pseudocode for shared/translation/gpc/GPICheck

```
// GPICheck()
// ==========
// Returns whether an access to a given physical address space is permitted
// given the configured GPI value.
// paspace: Physical address space of the access
// gpi: Value read from GPT for the access
// ss: Security state of the access

readonly func GPICheck(paspace : PASpace, gpi : bits(4), ss : SecurityState) => boolean
begin
    case gpi of
        when GPT_NoAccess =>
            return FALSE;
        when GPT_Secure =>
            assert IsFeatureImplemented(FEAT_SEL2);
            return paspace == PAS_Secure;
        when GPT_NonSecure =>
            return paspace == PAS_NonSecure;
        when GPT_Root =>
            return paspace == PAS_Root;
        when GPT_Realm =>
            return paspace == PAS_Realm;
        when GPT_NonSecureOnly =>
            assert IsFeatureImplemented(FEAT_RME_GPC2);
            return paspace == PAS_NonSecure && (ss IN {SS_Root, SS_NonSecure});
        when GPT_SystemAgent =>
            assert IsFeatureImplemented(FEAT_RME_GDI);
            return paspace == PAS_SystemAgent;
        when GPT_NonSecureProtected =>
            assert IsFeatureImplemented(FEAT_RME_GDI);
            return paspace == PAS_NonSecureProtected;
        when GPT_NA6 =>
            assert IsFeatureImplemented(FEAT_RME_GDI);
            return FALSE;
        when GPT_NA7 =>
            assert IsFeatureImplemented(FEAT_RME_GDI);
            return FALSE;
        when GPT_Any =>
            return TRUE;
        otherwise =>
            unreachable;
    end;
end;
```

Library pseudocode for shared/translation/gpc/GPIIndex

```
// GPIIndex()
// ==========

func GPIIndex(pa : bits(NUM_PABITS)) => integer
begin
    case DecodePGS(GPCCR_EL3().PGS) of
        when PGS_4KB =>  return UInt(pa[15:12]);
        when PGS_16KB => return UInt(pa[17:14]);
        when PGS_64KB => return UInt(pa[19:16]);
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for shared/translation/gpc/GPIValid

```
// GPIValid()
// ==========
// Returns whether a given value is a valid encoding for a GPI value

readonly func GPIValid(gpi : bits(4)) => boolean
begin
    case gpi of
        when GPT_NoAccess =>
            return TRUE;
        when GPT_NonSecureProtected =>
            return IsFeatureImplemented(FEAT_RME_GDI) && GPCCR_EL3().NSP == '1';
        when GPT_SystemAgent =>
            return IsFeatureImplemented(FEAT_RME_GDI) && GPCCR_EL3().SA  == '1';
        when GPT_NA6 =>
            return IsFeatureImplemented(FEAT_RME_GDI) && GPCCR_EL3().NA6 == '1';
        when GPT_NA7 =>
            return IsFeatureImplemented(FEAT_RME_GDI) && GPCCR_EL3().NA7 == '1';
        when GPT_Secure =>
            return IsFeatureImplemented(FEAT_SEL2);
        when GPT_NonSecure =>
            return TRUE;
        when GPT_Realm =>
            return TRUE;
        when GPT_Root =>
            return TRUE;
        when GPT_NonSecureOnly =>
            return IsFeatureImplemented(FEAT_RME_GPC2) && GPCCR_EL3().NSO == '1';
        when GPT_Any =>
            return TRUE;
        otherwise =>
            return FALSE;
    end;
end;
```

Library pseudocode for shared/translation/gpc/GPT

```
// GPT dimensions
// ==============

constant GPTRange_4KB   : AddressSize = 12;
constant GPTRange_16KB  : AddressSize = 14;
constant GPTRange_64KB  : AddressSize = 16;
constant GPTRange_2MB   : AddressSize = 21;
constant GPTRange_32MB  : AddressSize = 25;
constant GPTRange_512MB : AddressSize = 29;
constant GPTRange_1GB   : AddressSize = 30;
constant GPTRange_16GB  : AddressSize = 34;
constant GPTRange_64GB  : AddressSize = 36;
constant GPTRange_512GB : AddressSize = 39;
```

Library pseudocode for shared/translation/gpc/GPTBlockDescriptorValid

```
// GPTBlockDescriptorValid()
// =========================
// Returns TRUE if the given GPT Block descriptor is valid, and FALSE otherwise.

func GPTBlockDescriptorValid(level_0_entry : bits(64)) => boolean
begin
    assert level_0_entry[3:0] == GPT_Block;
    return IsZero(level_0_entry[63:8]) && GPIValid(level_0_entry[7:4]);
end;
```

Library pseudocode for shared/translation/gpc/GPTContigDescriptorValid

```
// GPTContigDescriptorValid()
// ==========================
// Returns TRUE if the given GPT Contiguous descriptor is valid, and FALSE otherwise.

readonly func GPTContigDescriptorValid(level_1_entry : bits(64), pgs : PGSe) => boolean
begin
    assert level_1_entry[3:0] == GPT_Contig;
    return (IsZero(level_1_entry[63:10]) &&
            !IsZero(level_1_entry[9:8])  &&
            GPIValid(level_1_entry[7:4]));
end;
```

Library pseudocode for shared/translation/gpc/GPTEntry

```
// GPTEntry
// ========

type GPTEntry of record {
    gpi         : bits(4),         // GPI value for this region
    size        : AddressSize,     // Region size
    contig_size : AddressSize,     // Contiguous region size
    istable     : boolean,         // Flag a table entry
    level       : integer,         // Level of GPT lookup
    pa          : bits(NUM_PABITS) // PA uniquely identifying the GPT entry
};
```

Library pseudocode for shared/translation/gpc/GPTGranulesDescriptorValid

```
// GPTGranulesDescriptorValid()
// ============================
// Returns TRUE if the given GPT Granules descriptor is valid, and FALSE otherwise.

readonly func GPTGranulesDescriptorValid(level_1_entry : bits(64)) => boolean
begin
    for i = 0 to 15 do
        if !GPIValid(level_1_entry[i*4 +:4]) then
            return FALSE;
        end;
    end;
    return TRUE;
end;
```

Library pseudocode for shared/translation/gpc/GPTL0Size

```
// GPTL0Size()
// ===========
// Returns number of bits covered by a level 0 GPT entry

readonly func GPTL0Size() => AddressSize
begin
    case GPCCR_EL3().L0GPTSZ of
        when '0000' => return GPTRange_1GB;
        when '0100' => return GPTRange_16GB;
        when '0110' => return GPTRange_64GB;
        when '1001' => return GPTRange_512GB;
        otherwise   => unreachable;
    end;
    return 30;
end;
```

Library pseudocode for shared/translation/gpc/GPTLevel0EntryValid

```
// GPTLevel0EntryValid()
// =====================
// Returns TRUE if the given level 0 gpt descriptor is valid, and FALSE otherwise.

func GPTLevel0EntryValid(gpt_entry : bits(64)) => boolean
begin
    case gpt_entry[3:0] of
        when GPT_Block => return GPTBlockDescriptorValid(gpt_entry);
        when GPT_Table => return GPTTableDescriptorValid(gpt_entry);
        otherwise =>      return FALSE;
    end;
end;
```

Library pseudocode for shared/translation/gpc/GPTLevel0Index

```
// GPTLevel0Index()
// ================
// Compute the level 0 index based on input PA.

func GPTLevel0Index(pa : bits(NUM_PABITS)) => integer
begin
    // Input address and index bounds
    let pps : integer{} = DecodePPS();
    let l0sz : integer{} = GPTL0Size();
    if pps <= l0sz then
        return 0;
    end;

    return UInt(pa[pps-1:l0sz]);
end;
```

Library pseudocode for shared/translation/gpc/GPTLevel1EntryValid

```
// GPTLevel1EntryValid()
// =====================
// Returns TRUE if the given level 1 gpt descriptor is valid, and FALSE otherwise.

func GPTLevel1EntryValid(gpt_entry : bits(64), pgs : PGSe) => boolean
begin
    case gpt_entry[3:0] of
        when GPT_Contig => return GPTContigDescriptorValid(gpt_entry, pgs);
        otherwise =>       return GPTGranulesDescriptorValid(gpt_entry);
    end;
end;
```

Library pseudocode for shared/translation/gpc/GPTLevel1Index

```
// GPTLevel1Index()
// ================
// Compute the level 1 index based on input PA.

func GPTLevel1Index(pa : bits(NUM_PABITS)) => integer
begin
    // Input address and index bounds
    let l0sz : integer{} = GPTL0Size();
    case DecodePGS(GPCCR_EL3().PGS) of
        when PGS_4KB =>  return UInt(pa[l0sz-1:16]);
        when PGS_16KB => return UInt(pa[l0sz-1:18]);
        when PGS_64KB => return UInt(pa[l0sz-1:20]);
        otherwise => unreachable;
    end;
end;
```

Library pseudocode for shared/translation/gpc/GPTTable

```
// GPTTable
// ========

type GPTTable of record {
    address : bits(NUM_PABITS)    // Base address of next table
};
```

Library pseudocode for shared/translation/gpc/GPTTableDescriptorValid

```
// GPTTableDescriptorValid()
// =========================
// Returns TRUE if the given GPT Table descriptor is valid, and FALSE otherwise.

readonly func GPTTableDescriptorValid(level_0_entry : bits(64)) => boolean
begin
    assert level_0_entry[3:0] == GPT_Table;
    let l0sz : integer{} = GPTL0Size();
    let pgs : PGSe       = DecodePGS(GPCCR_EL3().PGS);
    let p : integer{}    = DecodePGSRange(pgs);
    let top : integer{}  = if DecodePPS() == 56 then 56 else 52;
    return IsZero(level_0_entry[63:top,11:4]) && IsZero(level_0_entry[(l0sz-p)-2:12]);
end;
```

Library pseudocode for shared/translation/gpc/GPTWalk

```
// GPTWalk()
// =========
// Get the GPT entry for a given physical address, pa

func GPTWalk(pa : bits(NUM_PABITS), accdesc : AccessDescriptor) => (GPCFRecord, GPTEntry)
begin
    // GPT base address
    var base : bits(NUM_PABITS);
    let pgs : PGSe = DecodePGS(GPCCR_EL3().PGS);

    // The level 0 GPT base address is aligned to the greater of:
    // * the size of the level 0 GPT, determined by GPCCR_EL3().[PPS, L0GPTSZ].
    // * 4KB
    base = ZeroExtend{NUM_PABITS}(GPTBR_EL3().BADDR::Zeros{12});
    let pps : AddressSize = DecodePPS();
    let l0sz : integer{} = GPTL0Size();
    let alignment : integer{} = Max((pps - l0sz) + 3, 12) as AddressSize;
    base[alignment-1:0] = Zeros{alignment};

    let gptaccdesc : AccessDescriptor = CreateAccDescGPTW(accdesc);

    // Access attributes and address for GPT fetches
    var gptaddrdesc : AddressDescriptor;
    gptaddrdesc.memattrs = WalkMemAttrs(GPCCR_EL3().SH, GPCCR_EL3().IRGN, GPCCR_EL3().ORGN);
    gptaddrdesc.fault    = NoFault(gptaccdesc);

    gptaddrdesc.paddress.paspace = PAS_Root;
    gptaddrdesc.paddress.address = base + GPTLevel0Index(pa) * 8;

    // Fetch L0GPT entry
    var level_0_entry : bits(64);
    var memstatus : PhysMemRetStatus;
    (memstatus, level_0_entry) = PhysMemRead{64}(gptaddrdesc, gptaccdesc);
    if IsFault(memstatus) then
        return (GPCFault(GPCF_EABT, 0), ARBITRARY : GPTEntry);
    end;

    if !GPTLevel0EntryValid(level_0_entry) then
        return (GPCFault(GPCF_Walk, 0), ARBITRARY : GPTEntry);
    end;

    var result : GPTEntry;
    var table : GPTTable;
    case level_0_entry[3:0] of
        when GPT_Block =>
            // Decode the GPI value and return that
            result = DecodeGPTBlock(pgs, level_0_entry);
            result.pa = pa;
            return (GPCNoFault(), result);
        when GPT_Table =>
            // Decode the table entry and continue walking
            table = DecodeGPTTable(pgs, level_0_entry);
            // The address must be within the range covered by the GPT
            if AbovePPS(table.address) then
                return (GPCFault(GPCF_AddressSize, 0), ARBITRARY : GPTEntry);
            end;
        otherwise =>
            // An invalid encoding would be caught by GPTLevel0EntryValid()
            unreachable;
    end;

    // Must be a GPT Table entry
    assert level_0_entry[3:0] == GPT_Table;

    // Address of level 1 GPT entry
    let offset : integer = GPTLevel1Index(pa) * 8;

    var level_1_entry : bits(64);

    if IsFeatureImplemented(FEAT_RME_GDI) then
        // When FEAT_RME_GDI is implemented, the descriptor validation checks are performed
        // on a pair of descriptors within a naturally aligned 16-byte region of memory.
        gptaddrdesc.paddress.address = AlignDownSize{56}(table.address + offset, 16);
        var level_1_entry_lower : bits(64);
        (memstatus, level_1_entry_lower) = PhysMemRead{64}(gptaddrdesc, gptaccdesc);
        if IsFault(memstatus) then
            return (GPCFault(GPCF_EABT, 1), ARBITRARY : GPTEntry);
        end;

        gptaddrdesc.paddress.address = gptaddrdesc.paddress.address + 8;
        var level_1_entry_upper : bits(64);
        (memstatus, level_1_entry_upper) = PhysMemRead{64}(gptaddrdesc, gptaccdesc);
        if IsFault(memstatus) then
            return (GPCFault(GPCF_EABT, 1), ARBITRARY : GPTEntry);
        end;

        // An individual GPT descriptor is valid when both descriptors within the pair are valid.
        if (!GPTLevel1EntryValid(level_1_entry_upper, pgs) ||
              !GPTLevel1EntryValid(level_1_entry_lower, pgs)) then
```

```
            return (GPCFault(GPCF_Walk, 1), ARBITRARY : GPTEntry);
        end;

        if offset[3] == '1' then
            level_1_entry = level_1_entry_upper;
        else
            level_1_entry = level_1_entry_lower;
        end;
    else
        gptaddrdesc.paddress.address = table.address + offset;
        // Fetch L1GPT entry
        (memstatus, level_1_entry) = PhysMemRead{64}(gptaddrdesc, gptaccdesc);
        if IsFault(memstatus) then
            return (GPCFault(GPCF_EABT, 1), ARBITRARY : GPTEntry);
        end;

        if !GPTLevel1EntryValid(level_1_entry, pgs) then
            return (GPCFault(GPCF_Walk, 1), ARBITRARY : GPTEntry);
        end;
    end;

    case level_1_entry[3:0] of
        when GPT_Contig =>
            result = DecodeGPTContiguous(pgs, level_1_entry);
        otherwise =>
            let gpi_index : integer = GPIIndex(pa);
            result = DecodeGPTGranules(pgs, gpi_index, level_1_entry);
    end;

    result.pa = pa;
    return (GPCNoFault(), result);
end;
```

Library pseudocode for shared/translation/gpc/GranuleProtectionCheck

```
// GranuleProtectionCheck()
// ========================
// Returns whether a given access is permitted, according to the
// granule protection check.
// addrdesc and accdesc describe the access to be checked.

func GranuleProtectionCheck(addrdesc : AddressDescriptor, accdesc : AccessDescriptor) => GPCFRecord
begin

    assert IsFeatureImplemented(FEAT_RME);
    // The address to be checked
    let address = addrdesc.paddress;

    // Bypass mode - all accesses pass
    if GPCCR_EL3().GPC == '0' then
        return GPCNoFault();
    end;

    // Configuration consistency check
    if !GPCRegistersConsistent() then
        return GPCFault(GPCF_Walk, 0);
    end;

    if IsFeatureImplemented(FEAT_RME_GPC2) then
        var access_disabled : boolean;

        case address.paspace of
            when PAS_Secure =>    access_disabled = GPCCR_EL3().SPAD  == '1';
            when PAS_NonSecure => access_disabled = GPCCR_EL3().NSPAD == '1';
            when PAS_Realm =>     access_disabled = GPCCR_EL3().RLPAD == '1';
            when PAS_Root =>      access_disabled = FALSE;
            otherwise =>          unreachable;
        end;

        if access_disabled then
            return GPCFault(GPCF_Fail, 0);
        end;
    end;

    // Input address size check
    if AbovePPS(address.address) then
        if (address.paspace == PAS_NonSecure ||
              (IsFeatureImplemented(FEAT_RME_GPC2) && GPCCR_EL3().APPSAA == '1')) then
            return GPCNoFault();
        else
            return GPCFault(GPCF_Fail, 0);
        end;
    end;

    if (IsFeatureImplemented(FEAT_RME_GPC3) && GPCCR_EL3().GPCBW == '1' &&
          PAWithinGPCBypassWindow(address.address)) then
        return GPCNoFault();
    end;

    // GPT base address size check
    let gpt_base : bits(NUM_PABITS) = ZeroExtend{}(GPTBR_EL3().BADDR::Zeros{12});
    if AbovePPS(gpt_base) then
        return GPCFault(GPCF_AddressSize, 0);
    end;

    // GPT lookup
    var (gpcf, gpt_entry) = GPTWalk(address.address, accdesc);
    if gpcf.gpf != GPCF_None then
        return gpcf;
    end;

    // Check input physical address space against GPI
    let permitted : boolean = GPICheck(address.paspace, gpt_entry.gpi, accdesc.ss);

    if !permitted then
        gpcf = GPCFault(GPCF_Fail, gpt_entry.level);
        return gpcf;
    end;

    // Check passed

    return GPCNoFault();
end;
```

Library pseudocode for shared/translation/gpc/IsGranuleProtectionCheckedAccess

```
// IsGranuleProtectionCheckedAccess()
// ==================================
// Check if the access should be subject to Granule protection check returns
// true if it is, false otherwise

func IsGranuleProtectionCheckedAccess(accdesc : AccessDescriptor) => boolean
begin
    if accdesc.acctype == AccessType_DC then
        return ImpDefBool("GPC Fault on DC operations");
    end;

    return TRUE;
end;
```

Library pseudocode for shared/translation/gpc/PAWithinGPCBypassWindow

```
// PAWithinGPCBypassWindow()
// =========================
// Check if the supplied address is within a GPC Bypass window.

func PAWithinGPCBypassWindow(pa_in : bits(56)) => boolean
begin
    // Only check the top 26 bits as the minimum window size is 1GB
    let pa : bits(26) = pa_in[55:30];

    let gpcbwl : integer{} = UInt(GPCBW_EL3().BWSIZE);
    let gpcbwu : integer{} = 9 + UInt(GPCBW_EL3().BWSTRIDE);

    return pa[gpcbwu:gpcbwl] == GPCBW_EL3().BWADDR[gpcbwu:gpcbwl];
end;
```

Library pseudocode for shared/translation/gpc/PGS

```
// PGS
// ===
// Physical granule size

type PGSe of enumeration {
    PGS_4KB,
    PGS_16KB,
    PGS_64KB
};
```

Library pseudocode for shared/translation/gpc/Table

```
// Table format information
// ========================
// Granule Protection Table constants

constant GPT_NoAccess : bits(4)      = '0000';
constant GPT_Table : bits(4)         = '0011';
constant GPT_Block : bits(4)         = '0001';
constant GPT_Contig : bits(4)        = '0001';
constant GPT_SystemAgent : bits(4)   = '0100';
constant GPT_NonSecureProtected : bits(4) = '0101';
constant GPT_NA6 : bits(4)           = '0110';
constant GPT_NA7 : bits(4)           = '0111';
constant GPT_Secure : bits(4)        = '1000';
constant GPT_NonSecure : bits(4)     = '1001';
constant GPT_Root : bits(4)          = '1010';
constant GPT_Realm : bits(4)         = '1011';
constant GPT_NonSecureOnly : bits(4) = '1101';
constant GPT_Any : bits(4)           = '1111';
```

Library pseudocode for shared/translation/translation/S1TranslationRegime

```
// S1TranslationRegime()
// =====================
// Stage 1 translation regime for the given Exception level

readonly func S1TranslationRegime(el : bits(2)) => bits(2)
begin
    if el != EL0 then
        return el;

    elsif HaveEL(EL3) && ELUsingAArch32(EL3) && SCR().NS == '0' then
        return EL3;
    elsif IsFeatureImplemented(FEAT_VHE) && ELIsInHost(el) then
        return EL2;
    else
        return EL1;
    end;
end;

// S1TranslationRegime()
// =====================
// Returns the Exception level controlling the current Stage 1 translation regime. For the most
// part this is unused in code because the System register accessors (SCTLR_ELx, etc.) implicitly
// return the correct value.

readonly func S1TranslationRegime() => bits(2)
begin
    return S1TranslationRegime(PSTATE.EL);
end;
```

Library pseudocode for shared/translation/vmsa/AddressDescriptor

```
// AddressDescriptor
// =================
// Descriptor used to access the underlying memory array.

type AddressDescriptor of record {
    fault : FaultRecord,      // fault.statuscode indicates whether the address is valid
    memattrs : MemoryAttributes,
    paddress : FullAddress,
    s1assured : boolean,      // Stage 1 Assured Translation Property
    s2fs1mro : boolean,       // Stage 2 MRO permission for Stage 1
    mecid : bits(16),         // FEAT_MEC: Memory Encryption Context ID
    vaddress : bits(64)
};

constant FINAL_LEVEL : integer = 3;
```

Library pseudocode for shared/translation/vmsa/ContiguousSize

```
// ContiguousSize()
// ================
// Return the number of entries log 2 marking a contiguous output range

readonly func ContiguousSize(d128 : bit, tgx : TGx, walkstate : TTWState) => integer
begin
    if d128 == '1' then
        case tgx of
            when TGx_4KB =>
                assert walkstate.level IN {1, 2, 3};
                return if walkstate.level == 1 then 2 else 4;
            when TGx_16KB =>
                assert walkstate.level IN {1, 2, 3};
                if walkstate.level == 1 then
                    return 2;
                elsif walkstate.level == 2 then
                    return 4;
                else
                    return 6;
                end;
            when TGx_64KB =>
                assert walkstate.level IN {2, 3};
                return if walkstate.level == 2 then 6 else 4;
        end;
    end;

    case tgx of
        when TGx_4KB =>
            assert walkstate.level IN {1, 2, 3};
            return 4;
        when TGx_16KB =>
            assert walkstate.level IN {2, 3};
            return if walkstate.level == 2 then 5 else 7;
        when TGx_64KB =>
            assert walkstate.level IN {2, 3};
            return 5;
    end;
end;
```

Library pseudocode for shared/translation/vmsa/CreateAddressDescriptor

```
// CreateAddressDescriptor()
// =========================
// Set internal members for address descriptor type to valid values

func CreateAddressDescriptor(pa : FullAddress, memattrs : MemoryAttributes,
                             accdesc : AccessDescriptor) => AddressDescriptor
begin
    var addrdesc : AddressDescriptor;

    addrdesc.paddress = pa;
    addrdesc.memattrs = memattrs;
    addrdesc.fault    = NoFault(accdesc);

    return addrdesc;
end;

// CreateAddressDescriptor()
// =========================
// Set internal members for address descriptor type to valid values

func CreateAddressDescriptor(va : bits(64), pa : FullAddress,
                             memattrs : MemoryAttributes,
                             accdesc : AccessDescriptor) => AddressDescriptor
begin
    var addrdesc : AddressDescriptor;

    addrdesc.paddress  = pa;
    addrdesc.vaddress  = va;
    addrdesc.memattrs  = memattrs;
    addrdesc.fault     = NoFault(accdesc);
    addrdesc.s1assured = FALSE;

    return addrdesc;
end;
```

Library pseudocode for shared/translation/vmsa/CreateFaultyAddressDescriptor

```
// CreateFaultyAddressDescriptor()
// ===============================
// Set internal members for address descriptor type with values indicating error

func CreateFaultyAddressDescriptor(fault : FaultRecord) => AddressDescriptor
begin
    var addrdesc : AddressDescriptor;

    addrdesc.vaddress = fault.vaddress;
    addrdesc.fault    = fault;

    return addrdesc;
end;
```

Library pseudocode for shared/translation/vmsa/DecodePASpace

```
// DecodePASpace()
// ===============
// Decode the target PA Space

pure func DecodePASpace(nse2 : bit, nse : bit, ns : bit) => PASpace
begin
    case nse2::nse::ns of
        when '000' =>   return PAS_Secure;
        when '001' =>   return PAS_NonSecure;
        when '010' =>   return PAS_Root;
        when '011' =>   return PAS_Realm;
        when '100' =>   return PAS_SystemAgent;
        when '101' =>   return PAS_NonSecureProtected;
        when '110' =>   return PAS_NA6;
        when '111' =>   return PAS_NA7;
        otherwise =>    unreachable;
    end;
end;
```

Library pseudocode for shared/translation/vmsa/DescriptorType

```
// DescriptorType
// ==============
// Translation table descriptor formats

type DescriptorType of enumeration {
    DescriptorType_Table,
    DescriptorType_Leaf,
    DescriptorType_Invalid
};
```

Library pseudocode for shared/translation/vmsa/Domains

```
// Domains
// =======
// Short-descriptor format Domains

constant Domain_NoAccess : bits(2) = '00';
constant Domain_Client   : bits(2) = '01';
constant Domain_Manager  : bits(2) = '11';
```

Library pseudocode for shared/translation/vmsa/FetchDescriptor

```
// FetchDescriptor()
// =================
// Fetch a translation table descriptor

func FetchDescriptor{N : integer{32, 64, 128}}(ee : bit, walkaddress : AddressDescriptor,
                                               walkaccess : AccessDescriptor,
                                               fault_in : FaultRecord) => (FaultRecord, bits(N))
begin
    var descriptor : bits(N);
    var fault : FaultRecord = fault_in;

    if IsFeatureImplemented(FEAT_RME) then
        fault.gpcf = GranuleProtectionCheck(walkaddress, walkaccess);
        if fault.gpcf.gpf != GPCF_None then
            fault.statuscode = Fault_GPCFOnWalk;
            fault.paddress   = walkaddress.paddress;
            fault.gpcfs2walk = fault.secondstage;
            return (fault, ARBITRARY : bits(N));
        end;
    end;

    var memstatus : PhysMemRetStatus;
    (memstatus, descriptor) = PhysMemRead{N}(walkaddress, walkaccess);
    if IsFault(memstatus) then
        let iswrite : boolean = FALSE;
        fault = HandleExternalTTWAbort(memstatus, iswrite, walkaddress,
                                       walkaccess, N DIV 8, fault);
        if IsFault(fault.statuscode) then
            return (fault, ARBITRARY : bits(N));
        end;
    end;

    if ee == '1' then
        descriptor = BigEndianReverse{N}(descriptor);
    end;

    return (fault, descriptor);
end;
```

Library pseudocode for shared/translation/vmsa/HasUnprivileged

```
// HasUnprivileged()
// =================
// Returns whether a translation regime serves EL0 as well as a higher EL

readonly func HasUnprivileged(regime : Regime) => boolean
begin
    return (regime IN {
        Regime_EL20,
        Regime_EL30,
        Regime_EL10
    });
end;
```

Library pseudocode for shared/translation/vmsa/NUM_PABITS

```
// NUM_PABITS
// ==========

constant NUM_PABITS : integer{} = 56;
```

Library pseudocode for shared/translation/vmsa/Regime

```
// Regime
// ======
// Translation regimes

type Regime of enumeration {
    Regime_EL3,             // EL3
    Regime_EL30,            // EL3&0 (PL1&0 when EL3 is AArch32)
    Regime_EL2,             // EL2
    Regime_EL20,            // EL2&0
    Regime_EL10             // EL1&0
};
```

Library pseudocode for shared/translation/vmsa/RegimeUsingAArch32

```
// RegimeUsingAArch32()
// ====================
// Determine if the EL controlling the regime executes in AArch32 state

func RegimeUsingAArch32(regime : Regime) => boolean
begin
    case regime of
        when Regime_EL10 => return ELUsingAArch32(EL1);
        when Regime_EL30 => return TRUE;
        when Regime_EL20 => return FALSE;
        when Regime_EL2  => return ELUsingAArch32(EL2);
        when Regime_EL3  => return FALSE;
    end;
end;
```

Library pseudocode for shared/translation/vmsa/S1TTWParams

```
// S1TTWParams
// ===========
// Register fields corresponding to stage 1 translation
// For A32-VMSA, if noted, they correspond to A32-LPAE (Long descriptor format)

type S1TTWParams of record {
// A64-VMSA exclusive parameters
    ha : bit,          // TCR_ELx.HA
    hd : bit,          // TCR_ELx.HD
    tbi : bit,         // TCR_ELx.TBI{x}
    tbid : bit,        // TCR_ELx.TBID{x}
    nfd : bit,         // TCR_EL1.NFDx or TCR_EL2.NFDx when HCR_EL2.E2H == '1'
    e0pd : bit,        // TCR_EL1.E0PDx or TCR_EL2.E0PDx when HCR_EL2.E2H == '1'
    d128 : bit,        // TCR_ELx.D128
    aie : bit,         // (TCR2_ELx/TCR_EL3).AIE
    mair2 : MAIRType,  // MAIR2_ELx
    ds : bit,          // TCR_ELx.DS
    ps : bits(3),      // TCR_ELx.{I}PS
    txsz : bits(6),    // TCR_ELx.TxSZ
    epan : bit,        // SCTLR_EL1.EPAN or SCTLR_EL2.EPAN when HCR_EL2.E2H == '1'
    dct : bit,         // HCR_EL2.DCT
    nv1 : bit,         // HCR_EL2.NV1
    cmow : bit,        // SCTLR_EL1.CMOW or SCTLR_EL2.CMOW when HCR_EL2.E2H == '1'
    pnch : bit,        // TCR{2}_ELx.PnCH
    disch : bit,       // TCR{2}_ELx.DisCH
    haft : bit,        // TCR{2}_ELx.HAFT
    mtx : bit,         // TCR_ELx.MTX{y}
    skl : bits(2),     // TTBRn_ELx.SKL
    pie : bit,         // TCR2_ELx.PIE or TCR_EL3.PIE
    pir : S1PIRType,   // PIR_ELx
    pire0 : S1PIRType,// PIRE0_EL1 or PIRE0_EL2 when HCR_EL2.E2H == '1'
    emec : bit,        // SCTLR2_EL2.EMEC or SCTLR2_EL3.EMEC
    amec : bit,        // TCR2_EL2.AMEC0 or TCR2_EL2.AMEC1 when HCR_EL2.E2H == '1'
    fng : bit,         // TCR2_EL1.FNGx or TCR2_EL2.FNGx when HCR_EL2.E2H == '1'
    fngna : bit,       // TCR2_EL1.FNGx

// A32-VMSA exclusive parameters
    t0sz : bits(3),    // TTBCR.T0SZ
    t1sz : bits(3),    // TTBCR.T1SZ
    uwxn : bit,        // SCTLR.UWXN

// Parameters common to both A64-VMSA & A32-VMSA (A64/A32)
    tgx : TGx,         // TCR_ELx.TGx      / Always TGx_4KB
    irgn : bits(2),    // TCR_ELx.IRGNx    / TTBCR.IRGNx or HTCR.IRGN0
    orgn : bits(2),    // TCR_ELx.ORGNx    / TTBCR.ORGNx or HTCR.ORGN0
    sh : bits(2),      // TCR_ELx.SHx      / TTBCR.SHx or HTCR.SH0
    hpd : bit,         // TCR_ELx.HPD{x}   / TTBCR2.HPDx or HTCR.HPD
    ee : bit,          // SCTLR_ELx.EE     / SCTLR.EE or HSCTLR.EE
    wxn : bit,         // SCTLR_ELx.WXN    / SCTLR.WXN or HSCTLR.WXN
    ntlsmd : bit,      // SCTLR_ELx.nTLSMD / SCTLR.nTLSMD or HSCTLR.nTLSMD
    dc : bit,          // HCR_EL2.DC       / HCR.DC
    sif : bit,         // SCR_EL3.SIF      / SCR.SIF
    mair : MAIRType    // MAIR_ELx         / MAIR1:MAIR0 or HMAIR1:HMAIR0
};
```

Library pseudocode for shared/translation/vmsa/S2TTWParams

```
// S2TTWParams
// ===========
// Register fields corresponding to stage 2 translation.

type S2TTWParams of record {
// A64-VMSA exclusive parameters
    ha : bit,           // VTCR_EL2.HA
    hd : bit,           // VTCR_EL2.HD
    sl2 : bit,          // V{S}TCR_EL2.SL2
    ds : bit,           // VTCR_EL2.DS
    d128 : bit,         // VTCR_ELx.D128
    sw : bit,           // VSTCR_EL2.SW
    nsw : bit,          // VTCR_EL2.NSW
    sa : bit,           // VSTCR_EL2.SA
    nsa : bit,          // VTCR_EL2.NSA
    ps : bits(3),       // VTCR_EL2.PS
    txsz : bits(6),     // V{S}TCR_EL2.T0SZ
    fwb : bit,          // HCR_EL2.FWB
    cmow : bit,         // HCRX_EL2.CMOW
    skl : bits(2),      // VTTBR_EL2.SKL
    s2pie : bit,        // VTCR_EL2.S2PIE
    s2pir : S2PIRType,  // S2PIR_EL2
    tl0 : bit,          // VTCR_EL2.TL0
    tl1 : bit,          // VTCR_EL2.TL1
    assuredonly : bit,  // VTCR_EL2.AssuredOnly
    haft : bit,         // VTCR_EL2.HAFT
    emec : bit,         // SCTLR2_EL2.EMEC
    hdbss : bit,        // VTCR_EL2.HDBSS

// A32-VMSA exclusive parameters
    s : bit,            // VTCR.S
    t0sz : bits(4),     // VTCR.T0SZ

// Parameters common to both A64-VMSA & A32-VMSA if implemented (A64/A32)
    tgx : TGx,          // V{S}TCR_EL2.TG0  / Always TGx_4KB
    sl0 : bits(2),      // V{S}TCR_EL2.SL0  / VTCR.SL0
    irgn : bits(2),     // VTCR_EL2.IRGN0   / VTCR.IRGN0
    orgn : bits(2),     // VTCR_EL2.ORGN0   / VTCR.ORGN0
    sh : bits(2),       // VTCR_EL2.SH0     / VTCR.SH0
    ee : bit,           // SCTLR_EL2.EE     / HSCTLR.EE
    ptw : bit,          // HCR_EL2.PTW      / HCR.PTW
    vm : bit            // HCR_EL2.VM       / HCR.VM
};
```

Library pseudocode for shared/translation/vmsa/SDFType

```
// SDFType
// =======
// Short-descriptor format type

type SDFType of enumeration {
    SDFType_Table,
    SDFType_Invalid,
    SDFType_Supersection,
    SDFType_Section,
    SDFType_LargePage,
    SDFType_SmallPage
};
```

Library pseudocode for shared/translation/vmsa/SecurityStateForRegime

```
// SecurityStateForRegime()
// ========================
// Return the Security State of the given translation regime

func SecurityStateForRegime(regime : Regime) => SecurityState
begin
    case regime of
        when Regime_EL3 =>     return SecurityStateAtEL(EL3);
        when Regime_EL30 =>    return SS_Secure; // A32 EL3 is always Secure
        when Regime_EL2 =>     return SecurityStateAtEL(EL2);
        when Regime_EL20 =>    return SecurityStateAtEL(EL2);
        when Regime_EL10 =>    return SecurityStateAtEL(EL1);
    end;
end;
```

Library pseudocode for shared/translation/vmsa/Stage1OA

```
// Stage1OA()
// ==========
// Given the final walk state (a page or block descriptor), map the untranslated
// input address bits to the output address

func Stage1OA(ia : bits(64), walkparams : S1TTWParams, walkstate : TTWState) => FullAddress
begin
    // Output Address
    var oa : FullAddress;
    let tsize : integer = TranslationSize(walkparams.d128, walkparams.tgx, walkstate.level);

    var csize : integer;
    if walkstate.contiguous == '1' then
        csize = ContiguousSize(walkparams.d128, walkparams.tgx, walkstate);
    else
        csize = 0;
    end;

    let ia_msb : AddressSize = (tsize + csize) as AddressSize;
    oa.paspace = walkstate.baseaddress.paspace;
    oa.address = walkstate.baseaddress.address[NUM_PABITS-1:ia_msb]::ia[ia_msb-1:0];

    return oa;
end;
```

Library pseudocode for shared/translation/vmsa/Stage2OA

```
// Stage2OA()
// ==========
// Given the final walk state (a page or block descriptor), map the untranslated
// IPA bits to the output PA.

func Stage2OA(ipa : bits(NUM_PABITS), walkparams : S2TTWParams, walkstate : TTWState) => FullAddress
begin
    // Output Address
    var oa : FullAddress;
    let tsize : integer = TranslationSize(walkparams.d128, walkparams.tgx, walkstate.level);

    var csize : integer;
    if walkstate.contiguous == '1' then
        csize = ContiguousSize(walkparams.d128, walkparams.tgx, walkstate);
    else
        csize = 0;
    end;

    let ipa_msb : AddressSize = (tsize + csize) as AddressSize;
    oa.paspace = walkstate.baseaddress.paspace;
    oa.address = walkstate.baseaddress.address[NUM_PABITS-1:ipa_msb]::ipa[ipa_msb-1:0];

    return oa;
end;
```

Library pseudocode for shared/translation/vmsa/TGx

```
// TGx
// ===
// Translation granules sizes

type TGx of enumeration {
    TGx_4KB,
    TGx_16KB,
    TGx_64KB
};
```

Library pseudocode for shared/translation/vmsa/TGxGranuleBits

```
// TGxGranuleBits()
// ================
// Retrieve the address size, in bits, of a granule

pure func TGxGranuleBits(tgx : TGx) => AddressSize
begin
    case tgx of
        when TGx_4KB =>  return 12;
        when TGx_16KB => return 14;
        when TGx_64KB => return 16;
    end;
end;
```

Library pseudocode for shared/translation/vmsa/TLBContext

```
// TLBContext
// ==========
// Translation context compared on TLB lookups and invalidations, promoting a TLB hit on match

type TLBContext of record {
    ss : SecurityState,
    regime : Regime,
    vmid : bits(NUM_VMIDBITS),
    asid : bits(NUM_ASIDBITS),
    nG : bit,
    ipaspace : PASpace,    // Used in stage 2 lookups & invalidations only
    includes_s1 : boolean,
    includes_s2 : boolean,
    use_vmid : boolean,
    includes_gpt : boolean,
    ia : bits(64),         // Input Address
    tg : TGx,
    cnp : bit,
    level : integer,       // Assist TLBI level hints (FEAT_TTL)
    isd128 : boolean,
    xs : bit               // XS attribute (FEAT_XS)
};
```

Library pseudocode for shared/translation/vmsa/TLBRecord

```
// TLBRecord
// =========
// Translation output as a TLB payload

type TLBRecord of record {
    context : TLBContext,
    walkstate : TTWState,
    blocksize : AddressSize,  // Number of bits directly mapped from IA to OA
    contigsize : integer,     // Number of entries log 2 marking a contiguous output range
    s1descriptor : bits(128), // Stage 1 leaf descriptor in memory (valid if the TLB caches stage 1)
    s2descriptor : bits(128)  // Stage 2 leaf descriptor in memory (valid if the TLB caches stage 2)
};
```

Library pseudocode for shared/translation/vmsa/TTWState

```
// TTWState
// ========
// Translation table walk state

type TTWState of record {
    istable : boolean,
    level : integer,
    baseaddress : FullAddress,
    contiguous : bit,
    s1assured : boolean,  // Stage 1 Assured Translation Property
    s2assuredonly : bit,  // Stage 2 AssuredOnly attribute
    disch : bit,          // Stage 1 Disable Contiguous Hint
    nG : bit,
    guardedpage : bit,
    sdftype : SDFType,    // AArch32 Short-descriptor format walk only
    domain : bits(4),     // AArch32 Short-descriptor format walk only
    memattrs : MemoryAttributes,
    permissions : Permissions
};
```

Library pseudocode for shared/translation/vmsa/TranslationRegime

```
// TranslationRegime()
// ===================
// Select the translation regime given the target EL and PE state

readonly func TranslationRegime(el : bits(2)) => Regime
begin
    if el == EL3 then
        return if ELUsingAArch32(EL3) then Regime_EL30 else Regime_EL3;
    elsif el == EL2 then
        return if ELIsInHost(EL2) then Regime_EL20 else Regime_EL2;
    elsif el == EL1 then
        return Regime_EL10;
    elsif el == EL0 then
        if CurrentSecurityState() == SS_Secure && ELUsingAArch32(EL3) then
            return Regime_EL30;
        elsif ELIsInHost(EL0) then
            return Regime_EL20;
        else
            return Regime_EL10;
        end;
    else
        unreachable;
    end;
end;
```

Library pseudocode for shared/translation/vmsa/TranslationSize

```
// TranslationSize()
// =================
// Compute the number of bits directly mapped from the input address
// to the output address

func TranslationSize(d128 : bit, tgx : TGx, level : integer) => AddressSize
begin
    let granulebits : AddressSize = TGxGranuleBits(tgx);
    let descsizelog2 : integer{} = if d128 == '1' then 4 else 3;
    let blockbits : integer   = (FINAL_LEVEL - level) * (granulebits - descsizelog2);

    return (granulebits + blockbits) as AddressSize;
end;
```

Library pseudocode for shared/translation/vmsa/UseASID

```
// UseASID()
// =========
// Determine whether the translation context for the access requires ASID or is a global entry

func UseASID(accesscontext : TLBContext) => boolean
begin
    return HasUnprivileged(accesscontext.regime);
end;
```

Library pseudocode for shared/translation/vmsa/UseVMID

```
// UseVMID()
// =========
// Determine whether the translation context for the access requires VMID to match a TLB entry

func UseVMID(regime : Regime) => boolean
begin
    return regime == Regime_EL10 && EL2Enabled();
end;
```

2026-09_rel 2026-09-21 14:22:38

Copyright © 2010-2026 Arm Limited or its affiliates. All rights reserved. This document is Non-Confidential.